osmo-operator

code (scripts, sources, correctifs) — /opt/GSM/osmo-operator

Banc GSM émulé — banc-max

2026-10-28

Table des matières

1 Synthèse **6**

2 Fichiers **6**

2.1 /opt/GSM/osmo-operator/Dockerfile . . . 6
2.2 /opt/GSM/osmo-operator/Dockerfile.lite . . . 23
2.3 /opt/GSM/osmo-operator/Dockerfile.run . . . 25
2.4 /opt/GSM/osmo-operator/Dockerfile.stp . . . 27
2.5 /opt/GSM/osmo-operator/addition.sh . . . 29
2.6 /opt/GSM/osmo-operator/build-iso.sh . . . 48
2.7 /opt/GSM/osmo-operator/build.sh . . . 49
2.8 /opt/GSM/osmo-operator/compose.sh . . . 55
2.9 /opt/GSM/osmo-operator/generate_configs.sh . . . 59
2.10 /opt/GSM/osmo-operator/install.sh . . . 71
2.11 /opt/GSM/osmo-operator/launch.sh . . . 74
2.12 /opt/GSM/osmo-operator/set_stp_ip.sh . . . 85
2.13 /opt/GSM/osmo-operator/ss7-console.py . . . 94
2.14 /opt/GSM/osmo-operator/start-native.sh . . . 94
2.15 /opt/GSM/osmo-operator/start.sh . . . 95
2.16 /opt/GSM/osmo-operator/update.sh . . . 95
2.17 /opt/GSM/osmo-operator/armbian/build-armbian.sh . . . 110
2.18 /opt/GSM/osmo-operator/armbian/userpatches/customize-image.sh . . . 115
2.19 /opt/GSM/osmo-operator/checks/_mode.sh . . . 119
2.20 /opt/GSM/osmo-operator/checks/annuaire.sh . . . 131
2.21 /opt/GSM/osmo-operator/checks/check_all.sh . . . 134
2.22 /opt/GSM/osmo-operator/checks/diag-interstp.sh . . . 141
2.23 /opt/GSM/osmo-operator/checks/diag-stp-operator.sh . . . 145
2.24 /opt/GSM/osmo-operator/checks/global_check.sh . . . 148
2.25 /opt/GSM/osmo-operator/checks/operator_summary.sh . . . 157
2.26 /opt/GSM/osmo-operator/checks/ss7_check.sh . . . 164
2.27 /opt/GSM/osmo-operator/checks/vty-debug-dump.sh . . . 172
2.28 /opt/GSM/osmo-operator/checks/wan_ss7_check.sh . . . 179
2.29 /opt/GSM/osmo-operator/configs/info.html . . . 184
2.30 /opt/GSM/osmo-operator/configs/conky/dino.html . . . 189
2.31 /opt/GSM/osmo-operator/configs/conky/labgsm.html . . . 218
2.32 /opt/GSM/osmo-operator/configs/crypthome/crypthome-fond . . . 221
2.33 /opt/GSM/osmo-operator/configs/crypthome/crypthome-postinstall . . . 222
2.34 /opt/GSM/osmo-operator/configs/crypthome/crypthome-session-setup . . . 225
2.35 /opt/GSM/osmo-operator/configs/crypthome/init-crypthome.sh . . . 226
2.36 /opt/GSM/osmo-operator/configs/crypthome/lock-home . . . 230
2.37 /opt/GSM/osmo-operator/configs/crypthome/refresh-owner-apps . . . 231
2.38 /opt/GSM/osmo-operator/configs/crypthome/run-as-owner . . . 232
2.39 /opt/GSM/osmo-operator/configs/crypthome/unlock-home . . . 234
2.40 /opt/GSM/osmo-operator/configs/screenfetch/osmo-lab.sh . . . 240
2.41 /opt/GSM/osmo-operator/data/tutorial.html . . . 241

2.42 /opt/GSM/osmo-operator/fft-web/fft_web.py . . . 243
2.43 /opt/GSM/osmo-operator/helpers/create_interop.sh . . . 247
2.44 /opt/GSM/osmo-operator/helpers/prepare_host.sh . . . 250
2.45 /opt/GSM/osmo-operator/install_modules/00-prereqs.sh . . . 251
2.46 /opt/GSM/osmo-operator/install_modules/10-deps.sh . . . 252
2.47 /opt/GSM/osmo-operator/install_modules/20-user.sh . . . 253
2.48 /opt/GSM/osmo-operator/install_modules/30-sources.sh . . . 253
2.49 /opt/GSM/osmo-operator/install_modules/40-patches.sh . . . 254
2.50 /opt/GSM/osmo-operator/install_modules/50-build.sh . . . 255
2.51 /opt/GSM/osmo-operator/install_modules/60-binaires.sh . . . 256
2.52 /opt/GSM/osmo-operator/install_modules/70-configs.sh . . . 256
2.53 /opt/GSM/osmo-operator/install_modules/80-bureau.sh . . . 257
2.54 /opt/GSM/osmo-operator/install_modules/90-verify.sh . . . 260
2.55 /opt/GSM/osmo-operator/install_modules/_lib/inst.sh . . . 260
2.56 /opt/GSM/osmo-operator/installer/calamares/jobs/osmosupplements/main.py . . . 261
2.57 /opt/GSM/osmo-operator/iso_modules/00-options.sh . . . 263
2.58 /opt/GSM/osmo-operator/iso_modules/10-clavier.sh . . . 268
2.59 /opt/GSM/osmo-operator/iso_modules/20-hote.sh . . . 269
2.60 /opt/GSM/osmo-operator/iso_modules/20-stockage-docker.sh . . . 272
2.61 /opt/GSM/osmo-operator/iso_modules/21-docker-build.sh . . . 273
2.62 /opt/GSM/osmo-operator/iso_modules/22-wan-table.sh . . . 277
2.63 /opt/GSM/osmo-operator/iso_modules/30-image-configs.sh . . . 279
2.64 /opt/GSM/osmo-operator/iso_modules/31-image-source.sh . . . 281
2.65 /opt/GSM/osmo-operator/iso_modules/40-rootfs.sh . . . 282
2.66 /opt/GSM/osmo-operator/iso_modules/50-injection-image.sh . . . 284
2.67 /opt/GSM/osmo-operator/iso_modules/51-depot.sh . . . 285
2.68 /opt/GSM/osmo-operator/iso_modules/51-softsim.sh . . . 287
2.69 /opt/GSM/osmo-operator/iso_modules/52-qemu.sh . . . 288
2.70 /opt/GSM/osmo-operator/iso_modules/60-dashboard.sh . . . 295
2.71 /opt/GSM/osmo-operator/iso_modules/70-scripts.sh . . . 298
2.72 /opt/GSM/osmo-operator/iso_modules/71-debs-banc.sh . . . 301
2.73 /opt/GSM/osmo-operator/iso_modules/80-chroot.sh . . . 301
2.74 /opt/GSM/osmo-operator/iso_modules/81-cloture-systeme.sh . . . 314
2.75 /opt/GSM/osmo-operator/iso_modules/82-arm-natif.sh . . . 322
2.76 /opt/GSM/osmo-operator/iso_modules/82-services.sh . . . 325
2.77 /opt/GSM/osmo-operator/iso_modules/83-espace-ram.sh . . . 332
2.78 /opt/GSM/osmo-operator/iso_modules/84-comptes.sh . . . 338
2.79 /opt/GSM/osmo-operator/iso_modules/85-extensions-gnome.sh . . . 339
2.80 /opt/GSM/osmo-operator/iso_modules/85-glava.sh . . . 340
2.81 /opt/GSM/osmo-operator/iso_modules/85-installeur-bureau.sh . . . 340
2.82 /opt/GSM/osmo-operator/iso_modules/86-finitions.sh . . . 355
2.83 /opt/GSM/osmo-operator/iso_modules/87-lite.sh . . . 360
2.84 /opt/GSM/osmo-operator/iso_modules/88-lte-pmos.sh . . . 361
2.85 /opt/GSM/osmo-operator/iso_modules/89-home-chiffre.sh . . . 367
2.86 /opt/GSM/osmo-operator/iso_modules/90-iso.sh . . . 370
2.87 /opt/GSM/osmo-operator/iso_modules/91-secure-boot.sh . . . 375
2.88 /opt/GSM/osmo-operator/iso_modules/92-rpi-image.sh . . . 378
2.89 /opt/GSM/osmo-operator/iso_modules/99-fin.sh . . . 380
2.90 /opt/GSM/osmo-operator/launch/osmo-launch.sh . . . 380
2.91 /opt/GSM/osmo-operator/launch/start-docker.sh . . . 385
2.92 /opt/GSM/osmo-operator/launch/start-interstp.sh . . . 423
2.93 /opt/GSM/osmo-operator/launch/start-multi.sh . . . 429
2.94 /opt/GSM/osmo-operator/launch/start-nitb.sh . . . 440
2.95 /opt/GSM/osmo-operator/launch/start-oqc.sh . . . 441
2.96 /opt/GSM/osmo-operator/lib/audio.sh . . . 442
2.97 /opt/GSM/osmo-operator/lib/gabarits.sh . . . 454
2.98 /opt/GSM/osmo-operator/navigation/**init**.py . . . 461
2.99 /opt/GSM/osmo-operator/navigation/ber.py . . . 461
2.100/opt/GSM/osmo-operator/navigation/diagram.py . . . 464
2.101/opt/GSM/osmo-operator/navigation/m3ua.py . . . 467
2.102/opt/GSM/osmo-operator/navigation/mapops.py . . . 475

2.103/opt/GSM/osmo-operator/navigation/quickcmd.py . . . 479
2.104/opt/GSM/osmo-operator/navigation/responder.py . . . 480
2.105/opt/GSM/osmo-operator/navigation/sccp.py . . . 485
2.106/opt/GSM/osmo-operator/navigation/ss7-console.py . . . 488
2.107/opt/GSM/osmo-operator/navigation/ss7-diag.py . . . 492
2.108/opt/GSM/osmo-operator/navigation/ss7.py . . . 498
2.109/opt/GSM/osmo-operator/navigation/tcap.py . . . 502
2.110/opt/GSM/osmo-operator/navigation/test-console.sh . . . 505
2.111/opt/GSM/osmo-operator/navigation/topo.py . . . 508
2.112/opt/GSM/osmo-operator/navigation/tui.py . . . 515
2.113/opt/GSM/osmo-operator/navigation/vty.py . . . 529
2.114/opt/GSM/osmo-operator/network/apn-nat.sh . . . 533
2.115/opt/GSM/osmo-operator/network/firewall-wan.sh . . . 534
2.116/opt/GSM/osmo-operator/network/loopback.sh . . . 535
2.117/opt/GSM/osmo-operator/network/node-conf.sh . . . 537
2.118/opt/GSM/osmo-operator/network/osmo-ip-plan.sh . . . 539
2.119/opt/GSM/osmo-operator/network/pcu-socket.sh . . . 542
2.120/opt/GSM/osmo-operator/network/set-node-id.sh . . . 544
2.121/opt/GSM/osmo-operator/network/set_ip_vm.sh . . . 548
2.122/opt/GSM/osmo-operator/network/setup-docker-lan-route.sh . . . 555
2.123/opt/GSM/osmo-operator/network/setup-payload-ism.sh . . . 558
2.124/opt/GSM/osmo-operator/network/setup-vbox-interco.sh . . . 562
2.125/opt/GSM/osmo-operator/network/setup-wan-interop.sh . . . 565
2.126/opt/GSM/osmo-operator/network/setup-wan-mesh.sh . . . 574
2.127/opt/GSM/osmo-operator/network/setup-wan-sms.sh . . . 586
2.128/opt/GSM/osmo-operator/network/wan-nodes.sh . . . 588
2.129/opt/GSM/osmo-operator/opt-gsm/grgsm_fft_live.py . . . 594
2.130/opt/GSM/osmo-operator/opt-gsm/record_drain.py . . . 595
2.131/opt/GSM/osmo-operator/opt-gsm/si_bridge.py . . . 597
2.132/opt/GSM/osmo-operator/opt-gsm/si_bridge_loop.sh . . . 603
2.133/opt/GSM/osmo-operator/packaging/apt-fast-install.sh . . . 604
2.134/opt/GSM/osmo-operator/packaging/apt-mirror.sh . . . 607
2.135/opt/GSM/osmo-operator/packaging/build-debs.sh . . . 608
2.136/opt/GSM/osmo-operator/packaging/build-pmos-kernel-deb.sh . . . 613
2.137/opt/GSM/osmo-operator/packaging/osmo-deb.sh . . . 615
2.138/opt/GSM/osmo-operator/packaging/snapshot-lte-debs.sh . . . 617
2.139/opt/GSM/osmo-operator/patches/fc-magnetite-alr-arfcn-forced.patch . . . 619
2.140/opt/GSM/osmo-operator/patches/fc-magnetite-banc.patch . . . 623
2.141/opt/GSM/osmo-operator/patches/gapk-pq-alsa-output-probe.patch . . . 629
2.142/opt/GSM/osmo-operator/patches/grgsm-optional-qtgui-import.patch . . . 632
2.143/opt/GSM/osmo-operator/patches/grgsm-receiver-publish-bsic-fn.patch . . . 632
2.144/opt/GSM/osmo-operator/patches/magnetite-parole-ul-au-go.patch . . . 633
2.145/opt/GSM/osmo-operator/patches/magnetite-sms-msginfo-doublefree.patch . . . 634
2.146/opt/GSM/osmo-operator/patches/osmo-bts-force-rand-toy.patch . . . 636
2.147/opt/GSM/osmo-operator/patches/osmo-hlr-force-rand-toy.patch . . . 637
2.148/opt/GSM/osmo-operator/patches/osmo-msc-force-rand-toy.patch . . . 637
2.149/opt/GSM/osmo-operator/patches/osmo-trx-ipc-ts-frame-align.patch . . . 638
2.150/opt/GSM/osmo-operator/patches/osmo-trx-rach-per-ra-table.patch . . . 638
2.151/opt/GSM/osmo-operator/patches/osmocom-bb-clck-gen-frame-tolerance.patch . . . 642
2.152/opt/GSM/osmo-operator/patches/osmocom-bb-fbsb-timeout-scan-to.patch . . . 642
2.153/opt/GSM/osmo-operator/patches/osmocom-bb-firmware-gprs.patch . . . 644
2.154/opt/GSM/osmo-operator/patches/osmocom-bb-firmware-map-ld.patch . . . 654
2.155/opt/GSM/osmo-operator/patches/osmocom-bb-force-rand-toy.patch . . . 655
2.156/opt/GSM/osmo-operator/patches/osmocom-bb-osmocon-kc-filter.patch . . . 655
2.157/opt/GSM/osmo-operator/patches/osmocom-bb-plmn-a6-reinsert.patch . . . 656
2.158/opt/GSM/osmo-operator/patches/osmocom-bb-sim-plmn-from-bcd.patch . . . 656
2.159/opt/GSM/osmo-operator/patches/osmocom-bb-transceiver-qemu-bench.patch . . . 657
2.160/opt/GSM/osmo-operator/patches/pmaports-linux-postmarketos-stable-ppp.patch . . . 670
2.161/opt/GSM/osmo-operator/patches/pmbootstrap-osmo-bench-qemu.patch . . . 671
2.162/opt/GSM/osmo-operator/pont/**init**.py . . . 676
2.163/opt/GSM/osmo-operator/pont/air.py . . . 677

2.164/opt/GSM/osmo-operator/pont/airmesh.py . . . 684
2.165/opt/GSM/osmo-operator/pont/cipher.py . . . 692
2.166/opt/GSM/osmo-operator/pont/config.py . . . 694
2.167/opt/GSM/osmo-operator/pont/downlink.py . . . 695
2.168/opt/GSM/osmo-operator/pont/gsm.py . . . 701
2.169/opt/GSM/osmo-operator/pont/pont.py . . . 705
2.170/opt/GSM/osmo-operator/pont/pont_dsp.py . . . 706
2.171/opt/GSM/osmo-operator/pont/record.py . . . 706
2.172/opt/GSM/osmo-operator/pont/state.py . . . 710
2.173/opt/GSM/osmo-operator/pont/stats.py . . . 712
2.174/opt/GSM/osmo-operator/pont/trx.py . . . 714
2.175/opt/GSM/osmo-operator/pont/uplink.py . . . 721
2.176/opt/GSM/osmo-operator/pont/voix_hote.py . . . 728
2.177/opt/GSM/osmo-operator/pont/vsp_audio.py . . . 729
2.178/opt/GSM/osmo-operator/pont/dsp/**init**.py . . . 732
2.179/opt/GSM/osmo-operator/pont/dsp/clock.py . . . 734
2.180/opt/GSM/osmo-operator/pont/dsp/dedicated.py . . . 736
2.181/opt/GSM/osmo-operator/pont/dsp/downlink.py . . . 737
2.182/opt/GSM/osmo-operator/pont/dsp/tch.py . . . 738
2.183/opt/GSM/osmo-operator/pont/dsp/trx.py . . . 739
2.184/opt/GSM/osmo-operator/pont/dsp/uplink.py . . . 739
2.185/opt/GSM/osmo-operator/scripts/audio-chain.sh . . . 747
2.186/opt/GSM/osmo-operator/scripts/audio-diag.sh . . . 748
2.187/opt/GSM/osmo-operator/scripts/banc-repro.sh . . . 751
2.188/opt/GSM/osmo-operator/scripts/call-diag.sh . . . 754
2.189/opt/GSM/osmo-operator/scripts/calypso-mode.sh . . . 757
2.190/opt/GSM/osmo-operator/scripts/check.sh . . . 759
2.191/opt/GSM/osmo-operator/scripts/dsp-rom-max.sh . . . 760
2.192/opt/GSM/osmo-operator/scripts/egprs-dsp.sh . . . 764
2.193/opt/GSM/osmo-operator/scripts/entrypoint.sh . . . 770
2.194/opt/GSM/osmo-operator/scripts/gapk-start.sh . . . 771
2.195/opt/GSM/osmo-operator/scripts/ism_payload.py . . . 782
2.196/opt/GSM/osmo-operator/scripts/magnetite-c139.sh . . . 788
2.197/opt/GSM/osmo-operator/scripts/osmo-banc-aliases.sh . . . 790
2.198/opt/GSM/osmo-operator/scripts/osmo-disclaimer.sh . . . 791
2.199/opt/GSM/osmo-operator/scripts/osmo-gnuarm-profile.sh . . . 791
2.200/opt/GSM/osmo-operator/scripts/osmo-start.sh . . . 792
2.201/opt/GSM/osmo-operator/scripts/osmo-wg-profile.sh . . . 794
2.202/opt/GSM/osmo-operator/scripts/pulse-gsm-setup.sh . . . 795
2.203/opt/GSM/osmo-operator/scripts/run.sh . . . 796
2.204/opt/GSM/osmo-operator/scripts/send-mt-sms.sh . . . 805
2.205/opt/GSM/osmo-operator/scripts/sms-interop-relay.py . . . 805
2.206/opt/GSM/osmo-operator/scripts/sms-routing-setup.sh . . . 820
2.207/opt/GSM/osmo-operator/scripts/sms_notify.py . . . 828
2.208/opt/GSM/osmo-operator/scripts/smsc-start.sh . . . 830
2.209/opt/GSM/osmo-operator/scripts/status.sh . . . 832
2.210/opt/GSM/osmo-operator/scripts/voix-forks.sh . . . 832
2.211/opt/GSM/osmo-operator/scripts/wslg-audio-bridge.sh . . . 834
2.212/opt/GSM/osmo-operator/startup_scripts/01-defauts-options.sh . . . 838
2.213/opt/GSM/osmo-operator/startup_scripts/02-noeud-couleurs.sh . . . 847
2.214/opt/GSM/osmo-operator/startup_scripts/03-affichage.sh . . . 848
2.215/opt/GSM/osmo-operator/startup_scripts/04-environnement.sh . . . 849
2.216/opt/GSM/osmo-operator/startup_scripts/05-run-sh.sh . . . 850
2.217/opt/GSM/osmo-operator/startup_scripts/06-banc-dsp-menu.sh . . . 851
2.218/opt/GSM/osmo-operator/startup_scripts/07-plan-mobile.sh . . . 862
2.219/opt/GSM/osmo-operator/startup_scripts/08-mobile-ms1.sh . . . 862
2.220/opt/GSM/osmo-operator/startup_scripts/09-mobile-cfg-tmux.sh . . . 862
2.221/opt/GSM/osmo-operator/startup_scripts/10-identite-noeud.sh . . . 868
2.222/opt/GSM/osmo-operator/startup_scripts/11-regen-configs.sh . . . 871
2.223/opt/GSM/osmo-operator/startup_scripts/12-pcap-softsim.sh . . . 875
2.224/opt/GSM/osmo-operator/startup_scripts/13-adresses.sh . . . 883

2.225/opt/GSM/osmo-operator/startup_scripts/14-demarrage.sh . . . 889
2.226/opt/GSM/osmo-operator/startup_scripts/15-fin.sh . . . 892
2.227/opt/GSM/osmo-operator/tools/at-cmd.py . . . 892
2.228/opt/GSM/osmo-operator/tools/conky-osmo-status.sh . . . 894
2.229/opt/GSM/osmo-operator/tools/daram_sweep.sh . . . 902
2.230/opt/GSM/osmo-operator/tools/dockerfile-to-install-md.sh . . . 904
2.231/opt/GSM/osmo-operator/tools/fft.sh . . . 908
2.232/opt/GSM/osmo-operator/tools/fft2.sh . . . 911
2.233/opt/GSM/osmo-operator/tools/fft_global.sh . . . 912
2.234/opt/GSM/osmo-operator/tools/full_qmd.sh . . . 914
2.235/opt/GSM/osmo-operator/tools/full_text.sh . . . 923
2.236/opt/GSM/osmo-operator/tools/glava-build.sh . . . 926
2.237/opt/GSM/osmo-operator/tools/globals-complet.py . . . 928
2.238/opt/GSM/osmo-operator/tools/gnome-extensions-install.sh . . . 931
2.239/opt/GSM/osmo-operator/tools/grgsm_relay_decode.py . . . 932
2.240/opt/GSM/osmo-operator/tools/grub-theme-render.py . . . 933
2.241/opt/GSM/osmo-operator/tools/make-docker-image.sh . . . 935
2.242/opt/GSM/osmo-operator/tools/matrix.sh . . . 936
2.243/opt/GSM/osmo-operator/tools/osmo-banc-cmd.sh . . . 938
2.244/opt/GSM/osmo-operator/tools/osmo-c123-gtk.py . . . 938
2.245/opt/GSM/osmo-operator/tools/osmo-dino.py . . . 948
2.246/opt/GSM/osmo-operator/tools/osmo-drivers.sh . . . 951
2.247/opt/GSM/osmo-operator/tools/osmo-epc.sh . . . 953
2.248/opt/GSM/osmo-operator/tools/osmo-extras-install.sh . . . 957
2.249/opt/GSM/osmo-operator/tools/osmo-fft-snap.py . . . 964
2.250/opt/GSM/osmo-operator/tools/osmo-firmwares-install.sh . . . 980
2.251/opt/GSM/osmo-operator/tools/osmo-fond-antenne.py . . . 986
2.252/opt/GSM/osmo-operator/tools/osmo-fond-bleu.py . . . 987
2.253/opt/GSM/osmo-operator/tools/osmo-fond-repos.py . . . 990
2.254/opt/GSM/osmo-operator/tools/osmo-fond.py . . . 992
2.255/opt/GSM/osmo-operator/tools/osmo-gcc-alternatives.sh . . . 996
2.256/opt/GSM/osmo-operator/tools/osmo-glava.sh . . . 996
2.257/opt/GSM/osmo-operator/tools/osmo-gtk-run.py . . . 998
2.258/opt/GSM/osmo-operator/tools/osmo-info . . . 1006
2.259/opt/GSM/osmo-operator/tools/osmo-launcher.py . . . 1007
2.260/opt/GSM/osmo-operator/tools/osmo-lte-install.sh . . . 1016
2.261/opt/GSM/osmo-operator/tools/osmo-lte.sh . . . 1024
2.262/opt/GSM/osmo-operator/tools/osmo-moon.py . . . 1031
2.263/opt/GSM/osmo-operator/tools/osmo-ofono-bridge.py . . . 1033
2.264/opt/GSM/osmo-operator/tools/osmo-op.sh . . . 1037
2.265/opt/GSM/osmo-operator/tools/osmo-panel.py . . . 1041
2.266/opt/GSM/osmo-operator/tools/osmo-patches-apply.sh . . . 1055
2.267/opt/GSM/osmo-operator/tools/osmo-phonesim-banc.py . . . 1056
2.268/opt/GSM/osmo-operator/tools/osmo-pmos-build.sh . . . 1099
2.269/opt/GSM/osmo-operator/tools/osmo-pmos-install.sh . . . 1103
2.270/opt/GSM/osmo-operator/tools/osmo-pmos-qemu.sh . . . 1108
2.271/opt/GSM/osmo-operator/tools/osmo-pmos-setup.sh . . . 1114
2.272/opt/GSM/osmo-operator/tools/osmo-pmos.sh . . . 1121
2.273/opt/GSM/osmo-operator/tools/osmo-reglages.py . . . 1128
2.274/opt/GSM/osmo-operator/tools/osmo-ril-atmodem.py . . . 1135
2.275/opt/GSM/osmo-operator/tools/osmo-sim-install.sh . . . 1143
2.276/opt/GSM/osmo-operator/tools/osmo-sim.sh . . . 1145
2.277/opt/GSM/osmo-operator/tools/osmo-supplements-firstboot.sh . . . 1152
2.278/opt/GSM/osmo-operator/tools/osmo-telecom-boot.sh . . . 1152
2.279/opt/GSM/osmo-operator/tools/osmo-topzone.py . . . 1154
2.280/opt/GSM/osmo-operator/tools/osmo-ts-probe.py . . . 1159
2.281/opt/GSM/osmo-operator/tools/osmo-wallpaper.sh . . . 1166
2.282/opt/GSM/osmo-operator/tools/osmo-wg.sh . . . 1171
2.283/opt/GSM/osmo-operator/tools/osmo-wireshark-root.sh . . . 1174
2.284/opt/GSM/osmo-operator/tools/overlay-draw.py . . . 1176
2.285/opt/GSM/osmo-operator/tools/paysage-render.py . . . 1186

2.286/opt/GSM/osmo-operator/tools/plymouth-render.py . . . 1193
2.287/opt/GSM/osmo-operator/tools/screenfetch-art.py . . . 1195
2.288/opt/GSM/osmo-operator/tools/status.sh . . . 1198
2.289/opt/GSM/osmo-operator/tools/vbox-wan-lab.sh . . . 1198
2.290/opt/GSM/osmo-operator/tools/vty-menu.sh . . . 1209
2.291/opt/GSM/osmo-operator/tools/wallpaper-render.py . . . 1211
2.292/opt/GSM/osmo-operator/tools/softsim/0001-simos-comp128v1-a38.patch . . . 1218
2.293/opt/GSM/osmo-operator/tools/softsim/0002-sap-serve-next-client.patch . . . 1220
2.294/opt/GSM/osmo-operator/tools/softsim/0003-sap-log-timestamps.patch . . . 1220
2.295/opt/GSM/osmo-operator/tools/softsim/0004-pcsc-disconnect-idempotent.patch . . . 1221
2.296/opt/GSM/osmo-operator/tools/softsim/make_sim_op1.py . . . 1221
2.297/opt/GSM/osmo-operator/web/app.js . . . 1223
2.298/opt/GSM/osmo-operator/web/index.html . . . 1228
2.299/opt/GSM/osmo-operator/web/style.css . . . 1230

1 Synthèse

run	élément	valeur
(pas de verdict.txt)		

dossier de run	contenu
Verdict et couverture	échelle des barreaux et tableau de couverture couche 1
Résultats des tests	captures VTY, diagnostics, rapports pytest, grafcets (.txt .md .mmd)
Fichiers	sources, scripts, configs et docs du banc (tout fichier texte)
Logs	journaux (.log), en dernier

Méthode

Dossiers : /opt/GSM/osmo-operator. Extensions : tous les fichiers texte (hors .git, caches, binaires, sauvegardes, LICENSE et sorties précédentes). Entrée 3822 Ko, sortie 3682 Ko.
Compactage sans perte : fichiers identiques cités une fois ; lignes consécutives identiques « ×N » ; lignes ne différant que par des nombres : gabarit avec ⟨k⟩ puis la liste ordonnée des valeurs (k-uplets), chaque ligne se reconstruit en remettant les valeurs dans l'ordre. Seuls les codes couleur, la négociation telnet des captures VTY et les caractères de commande sont retirés ; les lignes de plus de 1000 caractères sont coupées en morceaux terminés par ↩. Les .md sont inclus tels quels (titres rétrogradés) et les .mmd rendus en diagrammes Mermaid, sans compactage.

2 Fichiers

2.1 /opt/GSM/osmo-operator/Dockerfile

64349 octets, 1068 lignes → 1068 lignes

```
# syntax=docker/dockerfile:1
# Dockerfile - osmocom-nitb base image (stable layer)
# ------------------------------------------------------------------------
# Everything long and rarely changed lives here: the full apt package list (one
# list, no apt in Dockerfile.run), the Osmocom stack built from source,
# osmocom-bb + the Calypso firmware, QEMU and its IPC device, gr-gsm/GNU Radio,
```

```
# the Node runtime and the web dashboard, the PulseAudio config, the systemd
# units, the directories and the prompt. Roughly 11 GB and 40 min to build;
# rebuilt only when a dependency changes.
#
# Base ubuntu:24.04 (noble): python 3.12, gcc 13 default, *t64-renamed libraries
# (libasound2t64, libgnutls30t64...). The ISO (build-iso.sh) starts from the
# SAME noble: /root/.env and the /usr/local .so files are copied as-is into the
# rootfs and must find the same glibc and python. Changing the base here changes
# it there too.
#
# .deb cache (packaging/osmo-deb.sh): each directory built below is emitted as a
# .deb in /var/cache/osmo-debs (COPY from .deb-cache/, which build.sh keeps in
# sync with /var/cache/osmo-debs on the host). On rebuild, `osmo-deb install`
# lays the package down and skips the clone + compilation; `osmo-deb pack` and
# `osmo-deb snapshot` build the package the first time. build.sh --no-cache sets
# OSMO_DEB_REFRESH=1: everything is recompiled and the cache rewritten. Sources
# stay in /opt/GSM: `osmo-deb pack` embeds the current tree (/opt/GSM/<name>,
# .git included) in the package, so a cache rebuild yields the same image with
# its source trees, not an image where only /usr/local survives.
#
# Daily iteration happens in Dockerfile.run, which starts from this image
# (`FROM osmocom-nitb`) and only refreshes the scripts, configs, bridge and the
# qosmo / osmo-operator git trees.
#
# This image is self-contained: it has its own ENTRYPOINT and start-nitb.sh runs
# it standalone. Do not remove its config/script COPYs because Dockerfile.run
# redoes them: the overlap is intentional on both sides.

# ──────────────────────────────────────────────────────────────────────────────
# The build is a graph, not a chain - BuildKit stages
# ──────────────────────────────────────────────────────────────────────────────
# Four of the five long compilations have no dependency between them (the
# Osmocom stack, QEMU/qosmo, the GNU Radio venv, the 4G srsRAN + Open5GS), so
# they build in parallel:
#
#                  base   (apt, apt-fast, osmo-deb, configs)
#                    |
#         +----------+--------+---------+
#         |          |        |         |
#     osmo-core    qemu      lte      node          <- in parallel
#         |          |
#    +----+-----+    |
#    |    |     |    |
#   bb  grgsm-  +----+
#    |  venv    |
#    |    |    l1     (bb: osmocom-bb — l1: c54x_exe + grgsm_exe)
#    |    |     |
#    +----+-----+
#         |
#     osmocom-nitb      (assembly: dpkg -i, then configs)
#           |
#         debs          (FROM scratch: the .deb cache for CI)
#
# What makes the split possible: packaging/osmo-deb.sh. Each block is emitted as
# a .deb that contains BOTH the installed files AND the /opt/GSM source tree
# (cmd_pack). A parallel stage therefore exports its .deb files, and the
# assembly lays them back with `dpkg -i`.
#
# Three stages are NOT parallel to osmo-core, by design - all three link to
# libosmocore one way or another:
#   - osmocom-bb : `make nofirmware` links mobile/trxcon to libosmocore;
#   - c54x_exe   : its Makefile runs `pkg-config --libs libosmocoding libosmocore`;
#   - gr-gsm     : its CMakeLists looks for libosmocore/libosmocoding/libosmogsm.
# They do run in parallel WITH EACH OTHER once osmo-core is finished.
#
# Invariant: --target osmocom-nitb (build.sh, compose.yaml) is MANDATORY, since
```

```
# `debs` is always the last stage of the file.
# ──────────────────────────────────────────────────────────────────────────────


FROM ubuntu:24.04 AS base

# ROOT: where the sources live in the image. Fixed path; nothing to make
# portable inside a container.
ARG DEBIAN_FRONTEND=noninteractive
ARG ROOT=/opt/GSM

# OSMO_IMAGE=1 marks "image of this repo, its network is set up by
# start-docker.sh / compose": network/osmo-ip-plan.sh then touches no address.
# A plain ubuntu:24.04 container lacks it and addresses itself like a native.
ENV container=docker \
    OSMO_IMAGE=1 \
    PKG_CONFIG_PATH=/usr/local/lib/pkgconfig \
    LD_LIBRARY_PATH=/usr/local/lib \
    GIT_TERMINAL_PROMPT=0

# ── apt-fast: parallel apt downloads ──────────────────────────────────────────
# This image installs several hundred packages; apt fetches them one by one,
# apt-fast parallelises the downloads via aria2 (the gain is on the download,
# not on dpkg - installation stays sequential).
#
# One installer for all environments: packaging/apt-fast-install.sh (this image,
# Dockerfile.stp, the ISO chroot, the host via build.sh). It also writes
# /etc/apt/apt.conf.d/90osmo-operator with the shared download settings.
# Built-in fallback: without GitHub, apt-fast calls apt-get and the build
# continues, more slowly.
#
# Build apt cache: every apt step in this file mounts /var/cache/apt/archives as
# a BuildKit cache (id osmo-apt-archives, shared with Dockerfile.stp), so a
# rebuild re-downloads only what changed. apt-fast-install removes docker-clean
# and sets Keep-Downloaded-Packages, otherwise apt would delete freshly
# installed packages. Requires BuildKit (docker compose or docker-buildx, which
# build.sh installs); the legacy builder does not understand --mount.
COPY packaging/apt-fast-install.sh /usr/local/sbin/apt-fast-install
# Ubuntu mirror measured by build.sh (packaging/apt-mirror.sh): empty = the base
# image sources (archive.ubuntu.com), which may be unusably slow from the host
# without apt saying so.
ARG OSMO_UBUNTU_MIRROR=
RUN --mount=type=cache,id=osmo-apt-archives,target=/var/cache/apt/archives,sharing=locked \
    chmod 755 /usr/local/sbin/apt-fast-install && apt-fast-install \
    && rm -rf /var/lib/apt/lists/*
ENV DEBIAN_FRONTEND=noninteractive

# 1. System dependencies - ALL of them here. One list, one place to evolve it;
#    the runtime image (Dockerfile.run) needs no apt.
#
# tshark pulls wireshark-common, which asks a debconf question (non-root
# capture) that would block a non-interactive build: answered in advance.
#
# External coupling: install_modules/10-deps.sh EXTRACTS this list from the
# Dockerfile (the list is authoritative here) and must accept both `apt-get` and
# `apt-fast`. Do not revert this list to apt-get to work around that extractor.
RUN echo 'wireshark-common wireshark-common/install-setuid boolean true' | debconf-set-selections
# deb-src BEFORE the single `apt-fast update`: the gnuradio build-dep (gr-gsm,
# end of the block below) reads the Sources indexes. Noble writes its sources in
# deb822 (ubuntu.sources, "Types: deb"); the old /etc/apt/sources.list (jammy)
# is still handled in case the base changes (gnuradio is in universe: all
# components).
RUN if [ -f /etc/apt/sources.list.d/ubuntu.sources ]; then \
        sed -i 's/^Types: deb$/Types: deb deb-src/' /etc/apt/sources.list.d/ubuntu.sources; \
    else \
        sed -nE 's|^deb (http\S+) (\S+) .*|deb-src \1 \2 main restricted universe multiverse|p' \
```

```
            /etc/apt/sources.list | sort -u > /etc/apt/sources.list.d/deb-src.list; \
    fi
RUN --mount=type=cache,id=osmo-apt-archives,target=/var/cache/apt/archives,sharing=locked \
    apt-fast update && apt-fast install -y --no-install-recommends \
    # Build tools
    build-essential git gcc g++ make cmake autoconf automake libtool pkg-config wget curl \
    # Osmocom core & network dependencies
    libtalloc-dev libpcsclite-dev libsctp-dev libmnl-dev liburing-dev asterisk-moh* \
    libdbi-dev libdbd-sqlite3 libsqlite3-dev sqlite3 libc-ares-dev libgnutls28-dev \
    # Audio, radio & SIP
    libortp-dev libfftw3-dev libusb-1.0-0-dev libsofia-sip-ua-dev libsofia-sip-ua-glib-dev \
    # Python & system tools
    python3 python3-dev python3-scapy ca-certificates tmux systemd systemd-sysv \
    # Debug: gdb-multiarch to attach to the QEMU gdb-stub (ARM Calypso)
    gdb-multiarch \
    # ALSA: required by osmo-gapk for hardware audio I/O
    # (libasound2t64: noble name for libasound2, time_t 64-bit transition)
    libasound2-dev libasound2t64 alsa-utils \
    # libgsm: native GSM-FR codec (speeds up gapk in gsmfr mode)
    libgsm1-dev libgsm1 \
    iptables iproute2 asterisk ffmpeg \
    # build-iso sync: psmisc (pkill/killall, cleanup) + pulseaudio (gapk/parec audio chain)
    psmisc pulseaudio pulseaudio-utils binutils-arm-none-eabi \
    # Alternative toolchains: osmocom-bb jolly/testing and fixeria/burst_ind only
    # compile with gcc-9; gcc-11 is the default compiler for the rest (gcc-13 is
    # noble's and builds the Osmocom stack at the top of this file). These
    # packages are required by the update-alternatives calls below.
    gcc-9 g++-9 gcc-11 g++-11 \
    # log4cpp: used by gnuradio (gr-gsm)
    liblog4cpp5-dev \
    # QEMU Calypso (bbaranoff/qosmO): venv + numpy/scipy for the DSP tooling,
    # glib/pixman/slirp for the arm-softmmu target, ninja for meson, socat for
    # the scripts/run.sh PTYs.
    python3-venv python3-pip python3-numpy python3-scipy \
    libglib2.0-dev libpixman-1-dev libslirp-dev socat ninja-build \
    # Container operation tooling:
    #   telnet   -> VTY fallback (run_modules/21-abonnes-hlr.sh) + dashboard transport
    #   whiptail -> menus of tools/vty-menu.sh and start-docker.sh
    #   xz-utils -> `tar -xJf` of the Node 22 tarball (dashboard block, end of file)
    #   libasound2-plugins -> ALSA->PulseAudio plugin, REQUIRED by configs/asound.conf
    #        (`type pulse` on gsm_out, gsm_in AND pcm.!default): without it any
    #        ALSA open (gapk, mobile) fails and audio is dead on both sides
    #   tshark/tcpdump/libcap2-bin -> capture; libcap2-bin provides the setcap below
    telnet nano whiptail xz-utils libasound2-plugins \
    tcpdump tshark libcap2-bin \
    # GNU Radio 3.10 deps (gr-gsm, venv /root/.env below): same apt resolution as
    # the rest, no second RUN apt. (The && closes the list for the
    # install_modules/10-deps.sh extractor.)
    && apt-fast build-dep -y gnuradio \
    && rm -rf /var/lib/apt/lists/*

# Non-root capture for the dashboard and tcpdump. Position is MANDATORY: dumpcap
# only exists once wireshark-common is installed (pulled by tshark, above).
RUN setcap cap_net_raw,cap_net_admin+eip "$(command -v dumpcap)"

# ── The .deb cache: the tool, then the already-built packages ──────────────
# .deb-cache/ is the build-context mirror of /var/cache/osmo-debs on the host
# (build.sh copies it in before, and brings new packages back after). Empty on
# the first build: everything compiles and is cached. Populated afterwards: each
# step below lays down its package and compiles nothing.
# OSMO_DEB_REFRESH=1 (build.sh --no-cache): the cache is ignored and rewritten.
ARG OSMO_DEB_REFRESH=0
COPY packaging/osmo-deb.sh /usr/local/sbin/osmo-deb
COPY .deb-cache/ /var/cache/osmo-debs/
RUN chmod 755 /usr/local/sbin/osmo-deb && osmo-deb list || true
```

```
# git: large postBuffer for big clones. GIT_TERMINAL_PROMPT=0 (ENV, above)
# makes a clone that wants a login fail fast rather than hang with no terminal.
RUN git config --global http.postBuffer 524288000

SHELL ["/bin/bash", "-c"]
COPY configs/*conf /etc/asterisk/
# asound.conf must ALSO be at the root. The COPY above puts it in /etc/asterisk/,
# which ALSA never reads: ALSA reads /etc/asound.conf (or ~/.asoundrc). Without
# this second copy, `aplay -D gsm_out` returns "Unknown PCM gsm_out" and the
# container mobile is silent. start-docker.sh mounts the host copy
# (-v ...:/etc/asound.conf), but a container started otherwise (plain docker
# run, diagnostics) has none.
COPY configs/asound.conf /etc/asound.conf

WORKDIR ${ROOT}

# 2. Create the osmocom user
RUN groupadd osmocom && useradd -r -g osmocom -s /sbin/nologin -d /var/lib/osmocom osmocom && \
    mkdir -p /var/lib/osmocom && chown osmocom:osmocom /var/lib/osmocom

# ──────────────────────────────────────────────────────────────────────────
# STAGE osmo-core — the Osmocom stack: 17 repos, 4 patches, gapk, SMSC, libosmo-dsp
# ──────────────────────────────────────────────────────────────────────────
FROM base AS osmo-core
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# 3. Build the Osmocom stack (order matters)
RUN for repo in \
    libosmocore:1.12.1 \
    libosmo-netif:1.7.0 \
    libosmo-abis:2.1.0 \
    libosmo-sigtran:2.2.1 \
    libsmpp34:1.14.5 \
    libgtpnl:1.3.3 \
    osmo-hlr:1.9.2 \
    osmo-mgw:1.15.0 \
    osmo-ggsn:1.14.0 \
    osmo-sgsn:1.13.1 \
    osmo-msc:1.15.0 \
    osmo-bsc:1.14.0 \
    osmo-trx:1.7.2 \
    osmo-bts:1.10.0 \
    osmo-pcu:1.5.2 \
    osmo-sip-connector:1.7.2 \
    libosmo-gprs:0.2.1; \
    do \
    name=$(echo $repo | cut -d: -f1) && \
    version=$(echo $repo | cut -d: -f2) && \
    \
    if [[ "$name" =~ ^libosmo ]]; then \
        GIT_URL="https://gitea.osmocom.org/osmocom/$name"; \
    else \
        GIT_URL="https://gitea.osmocom.org/cellular-infrastructure/$name"; \
    fi && \
    \
    # Cached package first: if present, neither clone nor compile.
    if osmo-deb install "$name" "$version"; then continue; fi && \
    \
    cd ${ROOT} && \
    git clone "$GIT_URL" && cd "$name" && \
    git checkout "$version" && \
    \
```

```
    autoreconf -fi && \
    EXTRA_FLAGS="" && \
    if [ "$name" = "libosmo-abis" ]; then EXTRA_FLAGS="--disable-dahdi"; fi && \
    if [ "$name" = "osmo-msc" ]; then EXTRA_FLAGS="--enable-smpp"; fi && \
    if [ "$name" = "osmo-mgw" ]; then EXTRA_FLAGS="--enable-alsa"; fi && \
    if [ "$name" = "osmo-trx" ]; then EXTRA_FLAGS="--with-ipc"; fi && \
    if [ "$name" = "osmo-bts" ]; then EXTRA_FLAGS="--enable-virtual --enable-trx"; fi && \
    if [ "$name" = "osmo-ggsn" ]; then EXTRA_FLAGS="--enable-gtp-linux"; fi && \
    \
    ./configure $EXTRA_FLAGS && \
    make -j$(nproc) && \
    # make install under DESTDIR -> .deb in the cache -> dpkg -i into the root
    osmo-deb pack "$name" "$version" make install && \
    ldconfig \
    || { echo "ECHEC build osmocom: $name"; exit 1; }; \
    done

# ── osmo-trx IPC patch: align ts_initial on the TDMA frame boundary ──────────
# The IPC device (calypso-ipc-device) commits RX buffers at timestamps that are
# multiples of 2500 (CALYPSO_SHM_BUFSIZE). The patch rounds ts_initial to the
# frame boundary (8*625=5000 @ 4 SPS) so device-TN0 maps to osmo-TN0 and the UL
# RACH hits the TS0/RACH slot. Maintained in patches/.
# ── osmo-trx RACH UL patch: per-RA modulation table ──────────────────────────
# The mobile's real RA (d_rach@0x0474, fed via /dev/shm/calypso_rach by QEMU)
# varies per burst. sigProcLib pre-generates osmo-trx's exact Laurent modulation
# for each RA 0x00..0x0f (-> /root/rach_ref_RA<nn>.cs16) and the device selects
# the reference for the real RA. Also adds the RACH-DET logs (Transceiver.cpp)
# and the libosmocoding link in COMMON_LDADD (Makefile.am, required for
# gsm0503_rach_ext_encode). Touches Makefile.am -> autoreconf+configure needed.
COPY patches/osmo-trx-ipc-ts-frame-align.patch /tmp/osmo-trx-ipc-ts-frame-align.patch
COPY patches/osmo-trx-rach-per-ra-table.patch /tmp/osmo-trx-rach-per-ra-table.patch
# Same package name as in the loop (osmo-trx), version 1.7.2+ipc: dpkg does an
# upgrade. If the loop was served from the cache the source tree does not exist,
# so it is re-cloned before applying the patches.
RUN if ! osmo-deb install osmo-trx 1.7.2+ipc; then \
      { [ -d ${ROOT}/osmo-trx ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-trx \
          && git -C osmo-trx checkout 1.7.2; }; } \
      && git -C ${ROOT}/osmo-trx apply /tmp/osmo-trx-ipc-ts-frame-align.patch \
      && git -C ${ROOT}/osmo-trx apply /tmp/osmo-trx-rach-per-ra-table.patch \
      && cd ${ROOT}/osmo-trx \
      && autoreconf -fi \
      && ./configure --with-ipc \
      && make -j$(nproc) \
      && osmo-deb pack osmo-trx 1.7.2+ipc make install \
      && ldconfig; \
    fi

# ── osmo-bts RAND patch: pin the authentication challenge (lab only) ─────────
# In rsl_rx_rll() (src/common/rsl.c), rewrites the RAND of the AUTHENTICATION
# REQUEST frame when the RAND environment variable is set on the BTS process:
#     RAND unset   -> nothing touched (normal behaviour);
#     RAND=0       -> 16-byte zero RAND;
#     RAND=<hex>   -> up to 16 bytes read as hex, zero-padded.
# Touches only rsl.c (no Makefile.am), but autoreconf/configure is still run
# because a re-cloned tree has no ./configure. Same package name (osmo-bts),
# version 1.10.0+rand: dpkg upgrades. Maintained in patches/.
COPY patches/osmo-bts-force-rand-toy.patch /tmp/osmo-bts-force-rand-toy.patch
RUN if ! osmo-deb install osmo-bts 1.10.0+rand; then \
      { [ -d ${ROOT}/osmo-bts ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-bts \
          && git -C osmo-bts checkout 1.10.0; }; } \
      && git -C ${ROOT}/osmo-bts apply /tmp/osmo-bts-force-rand-toy.patch \
      && cd ${ROOT}/osmo-bts \
      && autoreconf -fi \
      && ./configure --enable-virtual --enable-trx \
```

```
      && make -j$(nproc) \
      && osmo-deb pack osmo-bts 1.10.0+rand make install \
      && ldconfig; \
    fi

# ── osmo-hlr RAND patch: pin the auth-vector RAND (lab only) ──────────────
# In auc.c/auc_compute_vectors(): when the RAND environment variable is set on
# the HLR process, the RAND drawn for each vector is forced to 16 zero bytes
# before osmo_auth_gen_vec2, so SRES and Kc are computed over RAND=0 (the HLR
# holds Ki). This keeps the HLR and MSC vectors consistent. RAND unset -> normal
# network. No Makefile.am touched; autoreconf/configure still run (re-cloned
# tree has no ./configure). Same package name (osmo-hlr), version 1.9.2+rand:
# dpkg upgrades. Maintained in patches/.
COPY patches/osmo-hlr-force-rand-toy.patch /tmp/osmo-hlr-force-rand-toy.patch
RUN if ! osmo-deb install osmo-hlr 1.9.2+rand; then \
      { [ -d ${ROOT}/osmo-hlr ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-hlr \
          && git -C osmo-hlr checkout 1.9.2; }; } \
      && git -C ${ROOT}/osmo-hlr apply /tmp/osmo-hlr-force-rand-toy.patch \
      && cd ${ROOT}/osmo-hlr \
      && autoreconf -fi \
      && ./configure \
      && make -j$(nproc) \
      && osmo-deb pack osmo-hlr 1.9.2+rand make install \
      && ldconfig; \
    fi

# ── osmo-msc RAND patch: pin the authentication challenge (lab only) ──────
# In gsm48_tx_mm_auth_req() (src/libmsc/gsm_04_08.c): when the RAND environment
# variable is set on the MSC process, the RAND of the downlink AUTH REQUEST is
# forced (16 zero bytes here; change the forced_rand[] initialiser for another
# value). RAND unset -> normal network behaviour. No Makefile.am touched;
# autoreconf/configure still run (re-cloned tree has no ./configure). Same
# package name (osmo-msc), version 1.15.0+rand: dpkg upgrades. Maintained in
# patches/.
COPY patches/osmo-msc-force-rand-toy.patch /tmp/osmo-msc-force-rand-toy.patch
RUN if ! osmo-deb install osmo-msc 1.15.0+rand; then \
      { [ -d ${ROOT}/osmo-msc ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-msc \
          && git -C osmo-msc checkout 1.15.0; }; } \
      && git -C ${ROOT}/osmo-msc apply /tmp/osmo-msc-force-rand-toy.patch \
      && cd ${ROOT}/osmo-msc \
      && autoreconf -fi \
      && ./configure --enable-smpp \
      && make -j$(nproc) \
      && osmo-deb pack osmo-msc 1.15.0+rand make install \
      && ldconfig; \
    fi

# ── gapk: ALSA-output probe patch (GAPK_ALSA_PROBE), NOT applied by the build ─
# patches/gapk-pq-alsa-output-probe.patch is a diagnostic (inert unless
# GAPK_ALSA_PROBE=1) that tells whether a silent MS listener is caused upstream
# (PCM already zero in the gapk chain) or downstream (ALSA/PulseAudio). It is
# kept versioned in patches/ but not applied here, so an upstream change to
# pq_alsa.c cannot break the image.
# To re-enable: apply it after the `git clone`, before `cd osmo-gapk`, in the
# RUN below (it cannot stay as a `#` line inside the `\` continuation), i.e.
#   git -C ${ROOT}/osmo-gapk apply /tmp/gapk-pq-alsa-output-probe.patch && \
# and add back:
#   COPY patches/gapk-pq-alsa-output-probe.patch /tmp/gapk-pq-alsa-output-probe.patch
# Check first with: git -C <gapk clone> apply --check patches/gapk-pq-alsa-output-probe.patch
RUN if ! osmo-deb install osmo-gapk 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/osmocom/gapk osmo-gapk && \
      cd osmo-gapk && \
      autoreconf -fi && \
```

```
      ./configure --enable-alsa && \
      make -j$(nproc) && \
      osmo-deb pack osmo-gapk 0.git make install && \
      ldconfig; \
    fi

# ── gsup-smsc-proto: external SMSC connected to OsmoHLR over GSUP ──────────────
# Programs: proto-smsc-daemon (MO SMS reception + MT relay over GSUP)
#           proto-smsc-sendmt (MT SMS injection over a local UNIX socket)
# Build deps: libosmocore, libosmogsm, libosmo-gsup-client
# The top-level Makefile forces DESTDIR= empty, so the two sub-directories are
# installed here directly with the DESTDIR that osmo-deb sets (their own
# Makefiles honour it).
RUN if ! osmo-deb install gsup-smsc-proto 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/themwi/gsup-smsc-proto && \
      cd gsup-smsc-proto && \
      ./configure --with-osmo=/usr/local && \
      make -j$(nproc) && \
      osmo-deb pack gsup-smsc-proto 0.git \
        sh -c 'for i in daemon sendmt; do make -C "$i" DESTDIR="$DESTDIR" install || exit 1; done' && \
      ldconfig; \
    fi

# ── sms-coding-utils: SMS PDU encode/decode (GSM 03.40) ─────────────────────────
# sms-encode-text, gen-sms-deliver-pdu, sms-pdu-decode, etc.
# The top-level Makefile sets DESTDIR= empty and passes it to sub-directories; a
# variable given on make's COMMAND LINE wins and propagates, hence the explicit
# `make install DESTDIR=...`. bindir is /usr/local/bin per configure.
RUN if ! osmo-deb install sms-coding-utils 0.r1; then \
      cd ${ROOT} && \
      wget -q https://raw.githubusercontent.com/bbaranoff/fc-magnetite/main/freecalypso-env/sms-coding-utils-
       ↪  latest.tar.bz2 &&
       ↪  \
      tar xf sms-coding-utils-latest.tar.bz2 && \
      cd sms-coding-utils-r1 && \
      ./configure && \
      make -j$(nproc) && \
      osmo-deb pack sms-coding-utils 0.r1 \
        sh -c 'mkdir -p "$DESTDIR/usr/local/bin" && make install DESTDIR="$DESTDIR"'; \
    fi

# ── libosmo-dsp (transceiver/burst_ind dependency) ──────────────────────────────
RUN if ! osmo-deb install libosmo-dsp 0.git; then \
      cd /opt/GSM \
      && git clone https://gitea.osmocom.org/sdr/libosmo-dsp.git \
      && cd libosmo-dsp \
      && autoreconf -fi \
      && ./configure \
      && make -j$(nproc) \
      && osmo-deb pack libosmo-dsp 0.git make install \
      && ldconfig; \
    fi

# ═══════════════════════════════════════════════════════════════════════════════
# STAGE bb — osmocom-bb, the Calypso firmware and the two gcc-9 branches
# ═══════════════════════════════════════════════════════════════════════════════
FROM osmo-core AS bb
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# ── Calypso build ──────────────────────────────
#
```

```
# Three patches are applied to osmocom-bb and the WHOLE tree is snapshotted into
# the package: trx_toolkit, osmocon and the host binaries are read from the tree
# at runtime, not only from /usr/local. The same three patches are also applied
# natively by install_modules/40-patches.sh.
#   - clck-gen-frame-tolerance: fake_trx's TDMA clock only resets for a delay
#     larger than one frame (4.615 ms), so sub-frame jitter no longer drops
#     speech frames.
#   - plmn-a6-reinsert: a SIM re-inserted without RPLMN (SAP reconnect) no
#     longer stays in A6 with an "unhandled" PLMN search.
#   - sim-plmn-from-bcd: PLMNsel / FPLMN read with osmo_plmn_to_bcd instead of
#     from_bcd.
# patches/osmocom-bb-osmocon-kc-filter.patch is kept for reference but not
# applied (the Kc comes from calypso_dsp_shunt.c via /dev/shm/calypso_kc_l1).
COPY patches/osmocom-bb-clck-gen-frame-tolerance.patch /tmp/osmocom-bb-clck-gen-frame-tolerance.patch
COPY patches/osmocom-bb-plmn-a6-reinsert.patch /tmp/osmocom-bb-plmn-a6-reinsert.patch
COPY patches/osmocom-bb-sim-plmn-from-bcd.patch /tmp/osmocom-bb-sim-plmn-from-bcd.patch
# OSMOCOM_BB_REF: osmocom-bb is PINNED. Upstream a0a315e2 "trxcon/trx_if: migrate
# to libosmo-trx" removed trxcon from `make nofirmware` and made it require
# libosmo-trx, so a clone of HEAD no longer produces src/host/trxcon/src/trxcon.
# eac4eed1 is the bench tree the three patches are made against.
ARG OSMOCOM_BB_REF=eac4eed1
RUN if ! osmo-deb install osmocom-bb 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/phone-side/osmocom-bb && \
      git -C ${ROOT}/osmocom-bb checkout -q ${OSMOCOM_BB_REF} && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-clck-gen-frame-tolerance.patch && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-plmn-a6-reinsert.patch && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-sim-plmn-from-bcd.patch && \
      cd osmocom-bb/src && \
      # Full build: firmware (layer1.bin/.elf for Calypso) + host tools (mobile,
      # trxcon, virtphy, ccch_scan). The firmware is needed for PHY_MODE=qemu,
      # where QEMU emulates a Calypso and runs layer1.
      make nofirmware && \
      osmo-deb snapshot osmocom-bb 0.git ${ROOT}/osmocom-bb; \
    fi

# ── Firmware Calypso prebuild (layer1.highram.*) ──────────────────────
# `make nofirmware` above does NOT build the firmware: the prebuilt binaries
# from bbaranoff/firmware are fetched and placed where PHY_MODE=qemu looks for
# them. Order is fixed: this block needs BOTH this clone AND the /opt/GSM/
# osmocom-bb tree above. GIT_TERMINAL_PROMPT=0 makes a login-prompting clone
# fail fast with no terminal; --depth 1 fetches only the prebuilt binaries, not
# the history.
RUN if ! osmo-deb install calypso-firmware 0.git; then \
      GIT_TERMINAL_PROMPT=0 git clone --depth 1 \
        https://github.com/bbaranoff/firmware /opt/GSM/firmware \
      && osmo-deb snapshot calypso-firmware 0.git /opt/GSM/firmware; \
    fi
# The single firmware copy lives in /opt/GSM/firmware: that is what both QEMU
# (-kernel .elf) and osmocon (romload .bin) are given (qemu paths.env knows only
# $GSM_ROOT/firmware). If you recompile the firmware in osmocom-bb, place the
# result in /opt/GSM/firmware.

# ── GCC 9 for the experimental osmocom-bb branches (jolly/testing, burst_ind) ─
# gcc-9 and gcc-11 are installed with the rest, above; here we only declare the
# alternatives, whose order matters for osmocom-bb.
RUN update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 90 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-9 \
    && update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 110 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-11

RUN update-alternatives --set gcc /usr/bin/gcc-9

# osmocom-bb jolly/testing -> transceiver (soft-SDR BTS for Calypso)
# Both the binary AND the tree go into the package, so the sources survive a
# cache rebuild (see the .deb cache note in the header).
```

```
# patches/osmocom-bb-transceiver-qemu-bench.patch: the host `transceiver` app
# serves the emulated Calypso (start-native.sh --osmo-trx, c54x_exe/run.sh
# OSMO_TRX=1): --no-sync (BTS mode from reset, no cell to listen to), --ts-mask
# (single timeslot), --l1ctl/--base-port, and SETTSC/SETFORMAT/NOMTXPOWER
# acknowledged (osmo-bts-trx >= 1.3 treats them as critical). Also applied
# natively by install_modules/40-patches.sh.
COPY patches/osmocom-bb-transceiver-qemu-bench.patch /tmp/osmocom-bb-transceiver-qemu-bench.patch
RUN if ! osmo-deb install osmocom-bb-transceiver 0.git; then \
      git clone --branch jolly/testing --depth 1 \
        https://gitea.osmocom.org/phone-side/osmocom-bb.git \
        /opt/GSM/osmocom-bb-transceiver \
      && git -C /opt/GSM/osmocom-bb-transceiver apply /tmp/osmocom-bb-transceiver-qemu-bench.patch \
      && cd /opt/GSM/osmocom-bb-transceiver/src \
      && make HOST_layer23_CONFARGS=--enable-transceiver nofirmware -j$(nproc) \
      && cp /opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver \
         /usr/local/bin/transceiver \
      && osmo-deb snapshot osmocom-bb-transceiver 0.git /usr/local/bin/transceiver \
            /opt/GSM/osmocom-bb-transceiver; \
    fi

# osmocom-bb fixeria/burst_ind -> ccch_scan / bcch_scan / cell_log
RUN if ! osmo-deb install osmocom-bb-burst-ind 0.git; then \
      git clone --branch fixeria/burst_ind --depth 1 \
        https://gitea.osmocom.org/phone-side/osmocom-bb.git \
        /opt/GSM/osmocom-bb-burst_ind \
      && cd /opt/GSM/osmocom-bb-burst_ind/src \
      && make nofirmware -j$(nproc) \
      && cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/ccch_scan \
         /usr/local/bin/ccch_scan \
      && { cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/bcch_scan \
         /usr/local/bin/bcch_scan 2>/dev/null || true; } \
      && { cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/cell_log \
         /usr/local/bin/cell_log 2>/dev/null || true; } \
      && osmo-deb snapshot osmocom-bb-burst-ind 0.git /opt/GSM/osmocom-bb-burst_ind \
           $(ls /usr/local/bin/ccch_scan /usr/local/bin/bcch_scan /usr/local/bin/cell_log 2>/dev/null); \
    fi

# Back to the default compiler: this stage only produces .deb files, but leaving
# it on gcc-9 would trap anyone deriving a step from it.
RUN update-alternatives --set gcc /usr/bin/gcc-11

# ============================================================================
# STAGE qemu — the single QEMU tree: /opt/GSM/qosmo (--enable-l1-grgsm)
# ============================================================================
FROM base AS qemu
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# ----------------------------------------------------------------------------
# QEMU Calypso — virtual RAN (emulated baseband)
# ----------------------------------------------------------------------------
# Architecture:
#   - QEMU emulates a Calypso SoC (ARM7TDMI + DSP TMS320C54x)
#   - the ARM runs the real osmocom-bb firmware layer1.highram.elf
#   - the DSP loads the real ROM (calypso_dsp.txt) at boot
#   - bridge.py relays bursts between osmo-bts-trx (UDP 5700-5702) and the DSP
#     BSP (UDP 6702), with QEMU as the TDMA clock master
#   - the mobile (layer23) connects directly to the L1CTL socket published by
#     the firmware via QEMU's serial PTY
#
# See scripts/run.sh PHY_MODE=qemu for the runtime orchestration.
# ----------------------------------------------------------------------------
# qosmo + c54x_exe + grgsm_exe
```

```
# ──────────────────────────────────────────────────────────────────────────
#   /opt/GSM/qosmo      single QEMU tree (bbaranoff/qosmO), --enable-l1-grgsm.
#                       Carries run.sh, run_modules, cfgs. The same binary serves
#                       the --dsp mode: CALYPSO_DSP_EXTERN=1 disables the gr-gsm
#                       L1 there. `ninja install` also installs the C launcher
#                       `qosmo`.
#   /opt/GSM/c54x_exe   the C54x OUTSIDE QEMU (TI mask-ROM, its ROM in rom/).
#                       Builds the /opt/GSM/qosmo sources: AFTER qosmo.
#   /opt/GSM/grgsm_exe  the gr-gsm layer 1 outside QEMU, same qosmo sources.
# The WHOLE qosmo tree is snapshotted, build/ included: QEMU_BIN points at
# $OQC_ROOT/build/qemu-system-arm, and QEMU reads build/qemu-bundle to relocate
# itself (see Dockerfile.lite).
RUN if ! osmo-deb install qosmo 0.git; then \
      git clone https://github.com/bbaranoff/qosmO /opt/GSM/qosmo \
      && cd /opt/GSM/qosmo \
      && python3 -m venv /root/.venv-qemu \
      && . /root/.venv-qemu/bin/activate \
      && pip install --no-cache-dir tomli \
      && mkdir -p build && cd build \
      && ../configure --target-list=arm-softmmu --enable-l1-grgsm \
            --prefix=/opt/GSM/qemu-install --disable-werror --disable-docs \
      && make -j$(nproc) \
      && make install \
      && cp /opt/GSM/qemu-install/bin/qemu-system-arm /usr/local/bin/qemu-system-arm \
      && cp /opt/GSM/qemu-install/bin/qosmo /usr/local/bin/qosmo \
      && osmo-deb snapshot qosmo 0.git /opt/GSM/qosmo /opt/GSM/qemu-install \
            /root/.venv-qemu /usr/local/bin/qemu-system-arm /usr/local/bin/qosmo; \
    fi

# ──────────────────────────────────────────────────────────────────────────
# STAGE l1 — the two layer-1 builds outside QEMU: c54x_exe and grgsm_exe
# ──────────────────────────────────────────────────────────────────────────
FROM qemu AS l1
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# These build the /opt/GSM/qosmo sources (already present, this stage derives
# from qemu) and link to libosmocoding/libosmocore for c54x_exe. /usr/local is
# pulled from osmo-core rather than deriving this stage from it: copying
# /usr/local (a few hundred MB) is far cheaper than copying QEMU's build tree
# (~1.5 GB of objects) the other way.
COPY --from=osmo-core /usr/local /usr/local
RUN ldconfig

# c54x_exe: its Makefile puts -march=native in CFLAGS, which would SIGILL at
# startup on a CPU other than the build host's. We keep its flags without that
# one. The ROM is also placed at /opt/GSM/calypso_dsp.*.bin, c54x_exe's default
# --rom-dir. The ROM is not in the c54x_exe repo: rom/fetch-rom.sh downloads the
# FreeCalypso 3606 dump, converts it to .bin and verifies SHA256SUMS.3606;
# calypso_dsp.txt (the raw dump) is downloaded separately.
# dsp-rom-3606-dump.txt comes from the fc-magnetite repo (freecalypso-env/).
ARG FREECALYPSO_URL=https://raw.githubusercontent.com/bbaranoff/fc-magnetite/main/freecalypso-env
RUN if ! osmo-deb install c54x-exe 0.git; then \
      git clone https://github.com/bbaranoff/c54x_exe /opt/GSM/c54x_exe \
      && cd /opt/GSM/c54x_exe \
      && make QOSMO=/opt/GSM/qosmo \
            CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function
 ↪  -Wno-unused-variable -Wno-unused-but-set-variable -Wno-sign-compare" \
      && FREECALYPSO_URL="$FREECALYPSO_URL" bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom \
      && curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt" \
      && cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt \
      && osmo-deb snapshot c54x-exe 0.git /opt/GSM/c54x_exe \
            /opt/GSM/calypso_dsp.PROM0.bin /opt/GSM/calypso_dsp.PROM1.bin /opt/GSM/calypso_dsp.PROM2.bin \
```

```
            /opt/GSM/calypso_dsp.PROM3.bin /opt/GSM/calypso_dsp.DROM.bin /opt/GSM/calypso_dsp.PDROM.bin \
            /opt/GSM/calypso_dsp.Registers.bin /opt/GSM/calypso_dsp.txt; \
    fi

RUN if ! osmo-deb install grgsm-exe 0.git; then \
      git clone https://github.com/bbaranoff/grgsm_exE /opt/GSM/grgsm_exe \
      && cd /opt/GSM/grgsm_exe \
      && make QOSMO=/opt/GSM/qosmo \
      && osmo-deb snapshot grgsm-exe 0.git /opt/GSM/grgsm_exe; \
    fi

# ──────────────────────────────────────────────────────────────────────
# STAGE grgsm-venv — GNU Radio 3.10 + gr-osmosdr + gr-gsm in /root/.env
# ──────────────────────────────────────────────────────────────────────
# gr-gsm links to libosmocore/libosmocoding/libosmogsm (its CMakeLists finds
# them via pkg-config), so this stage MUST start from osmo-core, not base. It
# stays parallel to `bb` and `l1`, which also derive from osmo-core.
FROM osmo-core AS grgsm-venv
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# ── gr-gsm: GNU Radio 3.10 + gr-osmosdr + gr-gsm in the venv /root/.env ──────
# (= the real-SI demod engine used by si_bridge.py / grgsm_decode).
# The GNU Radio deps (apt build-dep) are installed by the single apt block at
# the top of the file, deb-src included: no apt here.
#
# The build downloads and runs the gist (pinned to commit fcdb409), which
# creates /root/.env and installs the three. Then:
#
# gr-gsm patch: the receiver publishes the SCH BSIC/FN (decode_sch) on the
# `measurements` port AND on stdout ("SCHBSIC <bsic> <fn>"). The DSP shunt reads
# it (si_bridge.py parses grgsm_decode's stdout -> UDP 4731 -> feed_sb) and
# encodes the REAL BSIC in dispatch_sb. Applied AFTER the gist (which clones and
# builds gr-gsm), then rebuilt/reinstalled in the venv. Maintained in patches/.
#
# matplotlib is used by tools/fft_global.sh and tools/matrix.sh.
# numpy STAYS at 1.x: the gist pins "numpy<2" (GNU Radio 3.10 is built against
# it), but a bare `pip install matplotlib` would pull the latest
# matplotlib/contourpy, which require numpy>=2. The same-line constraint makes
# pip pick a numpy-1-compatible matplotlib. The cached package is 1.git (the
# 0.git one carried a numpy-2 venv).
#
# The cached package (grgsm-venv) is the COMPLETE venv, patch and matplotlib
# included: that is why the three steps fit in one RUN. The
# /opt/GSM/{gnuradio,gr-osmosdr,gr-gsm} trees are not in the package - nothing
# reads them at runtime (the venv carries its .so with an RPATH on /root/.env).
COPY patches/grgsm-receiver-publish-bsic-fn.patch /tmp/grgsm-receiver-publish-bsic-fn.patch
RUN if ! osmo-deb install grgsm-venv 1.git; then \
      curl -fsSL
       ↪  https://gist.githubusercontent.com/bbaranoff/3683811057933af0954b661821e950d1/raw/fcdb4092483ec383440b67fc002db0c1583
       ↪  | bash \
      && git -C /opt/GSM/gr-gsm apply /tmp/grgsm-receiver-publish-bsic-fn.patch \
      && cd /opt/GSM/gr-gsm/build \
      && make -j"$(nproc)" \
      && make install \
      && . ~/.env/bin/activate && pip install matplotlib "numpy<2" \
      && osmo-deb snapshot grgsm-venv 1.git /root/.env /etc/ld.so.conf.d/gnuradio.conf; \
    fi && ldconfig

# ──────────────────────────────────────────────────────────────────────
# STAGE node - Node 22 and the osmo-egprs-web dashboard
# ──────────────────────────────────────────────────────────────────────
FROM base AS node
```

```
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# ═══════════════════════════════════════════════════════════════════════
# Node.js + osmo-egprs-web dashboard
# ═══════════════════════════════════════════════════════════════════════
# No other build step depends on Node. The order inside this chain is
# constrained (see the ORDER notes below).
# Cloned under /opt/GSM because the ISO copies /opt/GSM as a whole
# (docker cp "$CID:/opt/GSM").
RUN if [ ! -d "/opt/GSM/osmo-egprs-web/" ]; then \
      mkdir -p /opt/GSM && \
      git clone -b main https://github.com/bbaranoff/osmo-egprs-web /opt/GSM/osmo-egprs-web; \
    fi

# ── Node.js runtime + osmo-egprs-web web service ──────────────────────
# /opt/GSM/osmo-egprs-web/server.js runs in native mode (local telnet VTY, no
# docker) and serves :8080. Startup is handled by start-native.sh
# (`systemctl restart osmo-egprs-web`); this only installs the runtime, the
# dependencies and the unit (enabled at boot).
#
# Node 22 (LTS "Jod"): same major version as the ISO (nodesource setup_22.x in
# build-iso.sh). Keep both aligned: the dashboard is the same server.js.
ARG NODE_VERSION=v22.23.2
# The tarball follows the image architecture (x64 on amd64, arm64 for a
# --platform linux/arm64 build for the Raspberry Pi).
RUN case "$(uname -m)" in x86_64) _na=x64 ;; aarch64) _na=arm64 ;; *) echo "arch node inconnue: $(uname -m)" >&2;
 ↪  exit 1 ;; esac && \
    curl -fsSL "https://nodejs.org/dist/${NODE_VERSION}/node-${NODE_VERSION}-linux-${_na}.tar.xz" \
        -o /tmp/node.tar.xz && \
    mkdir -p /opt/node && \
    tar -xJf /tmp/node.tar.xz -C /opt/node --strip-components=1 && \
    rm -f /tmp/node.tar.xz && \
    ln -sf /opt/node/bin/node /usr/local/bin/node && \
    ln -sf /opt/node/bin/npm  /usr/local/bin/npm && \
    ln -sf /opt/node/bin/npx  /usr/local/bin/npx && \
    node --version
# systemd unit (single source: services/ in this repo). Start is still done by
# start-native.sh (`systemctl restart osmo-egprs-web`).
COPY services/osmo-egprs-web.service /etc/systemd/system/osmo-egprs-web.service
RUN ln -sf /etc/systemd/system/osmo-egprs-web.service \
        /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service

# ── Web dashboard: install-web-service.sh ─────────────────────────────
# ORDER: this block must stay AFTER the /opt/GSM/osmo-egprs-web clone (top of
# this section); the COPY below writes into that repository.
#
# Single source for the unit: install-web-service.sh installs the unit from
# /opt/GSM/osmo-egprs-web/osmo-egprs-web.service, so that copy is overwritten
# with services/osmo-egprs-web.service (which carries `CAP_IFACE=any`, needed
# for the dashboard capture).
COPY services/osmo-egprs-web.service /opt/GSM/osmo-egprs-web/osmo-egprs-web.service
# Fail fast at build time if upstream removes the script.
RUN test -s /opt/GSM/osmo-egprs-web/install-web-service.sh
# JS dependencies (ws): a single `npm install`, after the clone. Non-fatal
# (`|| true`): node_modules is versioned in the repo, so an offline build works.
RUN cd /opt/GSM/osmo-egprs-web && npm install --omit=dev --no-audit --no-fund || true
# ── The installation runs here, at build time ─────────────────────────
# The script resolves the node path (/usr/local/bin in the container) and
# writes the unit with the correct ExecStart, so the image ships a working unit.
#
# `|| true`: there is no systemd during a build, so the script's `systemctl`
# calls fail; the unit, certificate and runtime are written before them.
```

```
#
# Nothing is replayed at boot: the script downloads node when it is missing,
# and the bench must work offline. Boot only starts osmo-egprs-web.service.
RUN bash /opt/GSM/osmo-egprs-web/install-web-service.sh || true

# ──────────────────────────────────────────────────────────────────────────
# STAGE lte - the bench's 4G: srsRAN_4G (ZeroMQ) and Open5GS in /opt/LTE
# ──────────────────────────────────────────────────────────────────────────
FROM base AS lte
# ARG does not cross a FROM: stages that call osmo-deb or use ${ROOT} must
# redeclare them, otherwise ROOT is empty and OSMO_DEB_REFRESH=1
# (build.sh --no-cache) does not reach this far.
ARG ROOT=/opt/GSM
ARG OSMO_DEB_REFRESH=0

# osmo-lte-install --configs reads $OSMO_REPO, so this stage needs its own
# clone of the repo. The assembly stage makes its own clone, which is the one
# Dockerfile.run `git pull`s.
RUN git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator

# ── The bench's 4G: srsRAN_4G (ZeroMQ) and Open5GS, in /opt/LTE ──────────
# Two paths to the same packages (see packaging/snapshot-lte-debs.sh): the .deb
# cache provides osmo-build-libzmq / osmo-build-srsran / osmo-build-open5gs to
# `osmo-deb install` below, and nothing is compiled. With an empty cache, they
# are built from source by the same script as the native install
# (tools/osmo-lte-install.sh --build, OSMO_DEB=1 -> osmo-deb pack).
# libzmq comes from Ubuntu (libzmq3-dev, installed by `--deps`).
# Open5GS prefix: /opt/LTE/open5gs/install (bin, etc/open5gs, var/log).
#
# Configs and launchers too (`--configs --launchers`, at the end of the RUN):
# the container runs as root and /root/.config/srsran is what srsenb/srsue
# read. The Open5GS .deb carries no yaml; the repo configs (from the
# /opt/GSM/osmo-operator clone above) are the only source. Subscribers
# (configs/open5gs/dump, subscribers.json) are not installed at build time:
# `osmo-epc start` restores them into MongoDB.
#
# `--deps` first, always (not only when compiling). The apt list at the top of
# this file is the Osmocom stack's (also read by install_modules/10-deps.sh);
# the 4G list lives in tools/osmo-lte-install.sh (lte_deps). These packages are
# also the runtime: cached .debs carry only the binaries (osmo-deb pack is a
# tar), not their dependencies (libmbedtls, libzmq5, libconfig++,
# boost_program_options). `--deps` also pulls mongodb-org (HSS/PCRF subscriber
# DB) and is not fatal on any package.
COPY tools/osmo-lte-install.sh /usr/local/sbin/osmo-lte-install
RUN --mount=type=cache,id=osmo-apt-archives,target=/var/cache/apt/archives,sharing=locked \
    chmod 755 /usr/local/sbin/osmo-lte-install && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --deps && \
    { osmo-deb install libzmq 4.3.5+git || true; } && \
    { osmo-deb install srsgui 0.1+git || true; } && \
    SRS_DEB_VER="$(osmo-lte-install --srs-deb-version)" && \
    if ! osmo-deb install srsran "$SRS_DEB_VER"; then \
        OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --build || { echo "ECHEC build srsRAN"; exit
        ↪  1; }; \
    fi && \
    if ! osmo-deb install open5gs 2.8.0+git; then \
        OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --build || { echo "ECHEC build Open5GS"; exit
        ↪  1; }; \
    fi && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --configs --launchers && \
    ldconfig && test -x /usr/local/bin/srsenb && test -x /opt/LTE/open5gs/install/bin/open5gs-mmed \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/mme.yaml && grep -q '^  sgsap:'
    ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml \
    && test -s /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf && test -s
    ↪  /opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt \
    && test -s /root/.config/srsran/enb.conf && test -x /usr/local/bin/osmo-lte && test -x /usr/local/bin/osmo-epc
```

```
# ── Smartphone UI: patched pmbootstrap, in /opt/user_interface/pmos ──────────
# The bench phone is a postmarketOS VM launched by pmbootstrap, patched for the
# PCI serial modem, bench audio, screen size and port 5038 (not adb):
# patches/pmbootstrap-osmo-bench-qemu.patch. Clone + patch are packaged as
# osmo-build-pmbootstrap: the ISO (50-injection-image.sh) installs it as-is and
# tools/osmo-pmos-install.sh finds it in place. The PPP kernel is not built
# here (pmbootstrap needs non-root and loop mounts): it comes from
# osmo-build-pmos-kernel (packaging/build-pmos-kernel-deb.sh, or GitHub
# bbaranoff/pmos_ppp_kernel), installed by 88-lte-pmos.sh.
COPY patches/pmbootstrap-osmo-bench-qemu.patch /tmp/pmbootstrap-osmo-bench-qemu.patch
RUN if ! osmo-deb install pmbootstrap 0.git; then \
        mkdir -p /opt/user_interface/pmos && \
        git clone https://gitlab.postmarketos.org/postmarketOS/pmbootstrap.git
         ↪  /opt/user_interface/pmos/pmbootstrap && \
        cd /opt/user_interface/pmos/pmbootstrap && \
        git apply /tmp/pmbootstrap-osmo-bench-qemu.patch && \
        grep -q osmo_bench_args pmb/commands/qemu.py && \
        osmo-deb snapshot pmbootstrap 0.git /opt/user_interface/pmos/pmbootstrap; \
    fi && test -f /opt/user_interface/pmos/pmbootstrap/pmbootstrap.py

# ============================================================================
# STAGE osmocom-nitb - assembly. This is the published image.
# ============================================================================
# Nothing is compiled here: the .debs produced by the stages are installed,
# then what no package carries is added (Node, the dashboard, the 4G, the repo,
# the build-context configs). build.sh (compose_build) and compose.yaml pass
# --target osmocom-nitb.
FROM base AS osmocom-nitb
ARG ROOT=/opt/GSM

# ── 1. Packages from all stages ──────────────────────────────────────────────
# bb derives from osmo-core and l1 from qemu: their caches already include
# their parent's.
COPY --from=bb         /var/cache/osmo-debs/ /var/cache/osmo-debs/
COPY --from=l1         /var/cache/osmo-debs/ /var/cache/osmo-debs/
COPY --from=grgsm-venv /var/cache/osmo-debs/ /var/cache/osmo-debs/
COPY --from=lte        /var/cache/osmo-debs/ /var/cache/osmo-debs/
# (No COPY from `node`: that stage never calls osmo-deb.)

# Install order matters for the four patched packages. osmo-trx, osmo-bts,
# osmo-hlr and osmo-msc exist in two versions with the same package name
# (e.g. osmo-trx 1.7.2 and 1.7.2+ipc). The glob installs in ASCII order, where
# "+" sorts before "~", so the unpatched one would win; the patched four are
# therefore reinstalled explicitly, last.
RUN set -eux; \
    dpkg -i --force-overwrite /var/cache/osmo-debs/osmo-build-*.deb; \
    dpkg -i --force-overwrite \
        /var/cache/osmo-debs/osmo-build-osmo-trx_1.7.2+ipc~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-bts_1.10.0+rand~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-hlr_1.9.2+rand~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-msc_1.15.0+rand~*.deb; \
    ldconfig

# ── 2. What no .deb carries ──────────────────────────────────────────────────
# Node (tarball in /opt/node) and the dashboard (git clone with node_modules);
# install-web-service.sh already ran in the node stage.
COPY --from=node /opt/node                                  /opt/node
COPY --from=node /opt/GSM/osmo-egprs-web                    /opt/GSM/osmo-egprs-web
COPY --from=node /etc/systemd/system/osmo-egprs-web.service /etc/systemd/system/osmo-egprs-web.service
RUN ln -sf /opt/node/bin/node /usr/local/bin/node \
    && ln -sf /opt/node/bin/npm  /usr/local/bin/npm \
    && ln -sf /opt/node/bin/npx  /usr/local/bin/npx \
    && ln -sf /etc/systemd/system/osmo-egprs-web.service \
        /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service \
    && node --version
```

```
# Explicit destination path: /opt/GSM/osmo-operator is the nominal location,
# used by Dockerfile.run (`git pull`), build-iso.sh and update.sh.
# Default branch (main), no explicit checkout: HEAD stays attached, as the
# `git pull` in Dockerfile.run requires.
RUN git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator

COPY tools/osmo-lte-install.sh /usr/local/sbin/osmo-lte-install
RUN chmod 755 /usr/local/sbin/osmo-lte-install

# ── 4G ──────────────────────────────────────────────────────────────
# All of /opt/LTE, not only what the .debs carry: the Open5GS install prefix
# lives under the source tree, and /opt/LTE is outside /opt/GSM so osmo-deb
# pack embeds no sources (SRC_ROOT). This keeps the source trees in the image.
COPY --from=lte /opt/LTE /opt/LTE

# `--deps` must run again here: these are also the runtime packages (libzmq5,
# libmbedtls, libconfig++, boost_program_options, mongodb-org...) and osmo-deb
# .debs declare no dependencies. The apt cache mount avoids re-downloading.
#
# Then `--configs --launchers`, same order as in the lte stage: the Open5GS
# yaml, enb.conf and the two launchers are written outside any DESTDIR, so no
# .deb carries them. They read /opt/GSM/osmo-operator, cloned above.
RUN --mount=type=cache,id=osmo-apt-archives,target=/var/cache/apt/archives,sharing=locked \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --deps && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --configs --launchers && \
    ldconfig && test -x /usr/local/bin/srsenb \
    && test -x /opt/LTE/open5gs/install/bin/open5gs-mmed \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/mme.yaml && grep -q '^  sgsap:'
     ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml \
    && test -s /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt \
    && test -s /root/.config/srsran/enb.conf && test -x /usr/local/bin/osmo-lte && test -x /usr/local/bin/osmo-epc

# gcc alternatives: declared in the bb stage, which only passes on its .debs;
# redeclared here, with gcc-11 as default.
RUN update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 90 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-9 \
    && update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 110 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-11 \
    && update-alternatives --set gcc /usr/bin/gcc-11

# ── 3. Project files ────────────────────────────────────────────────
WORKDIR /etc/osmocom
COPY scripts/. /etc/osmocom/
COPY configs/*cfg /etc/osmocom/
RUN mv /etc/osmocom/run.sh /root/run.sh
# gapk-start.sh (copied by `COPY scripts/.` above): symlink + state directory.
RUN chmod +x /etc/osmocom/gapk-start.sh && \
    ln -sf /etc/osmocom/gapk-start.sh /usr/local/bin/gapk-start.sh && \
    mkdir -p /var/lib/gapk

RUN cp -f /usr/local/bin/osmo* /usr/bin/ || true && \
    cp -f /usr/local/bin/proto-smsc-* /usr/bin/ || true && \
    cp -f /usr/local/bin/sms-* /usr/bin/ || true && \
    cp -f /usr/local/bin/gen-sms-* /usr/bin/ || true && \
    # Install any .service files from configs/
    cp /etc/osmocom/configs/*.service /lib/systemd/system/ 2>/dev/null || true

# ── proto-SMSC binaries check ──────────────────────────────────────
# Built by gsup-smsc-proto, copied into /usr/bin just above. Warning only.
RUN which proto-smsc-daemon && which proto-smsc-sendmt && \
    echo "proto-smsc binaries OK" || \
    echo "WARNING: proto-smsc binaries not found in PATH"

# Permissions and systemd fixes (status 214/217: CPUScheduling and the
# osmocom user are not available in the container).
```

```
RUN sed -i 's/^CPUScheduling/#CPUScheduling/g' /lib/systemd/system/osmo-*.service && \
    sed -i 's/User=osmocom/User=root/g' /lib/systemd/system/osmo-ggsn.service && \
    sed -i 's/User=osmocom/User=root/g' /lib/systemd/system/osmo-sgsn.service && \
    chmod +x /etc/osmocom/*.sh

# Enable services; units from services/ go to /etc/systemd/system/.
COPY services/osmo-bts-trx.service /etc/systemd/system/osmo-bts-trx.service
RUN systemctl enable osmo-bts-trx.service && \
    passwd -d root && \
    systemctl mask getty@tty1.service serial-getty@tty1.service

# Host-side osmocom-bb binaries, installed in a single layer with explicit modes.
RUN mkdir -p /root/.osmocom/bb/ \
    && install -m755 ${ROOT}/osmocom-bb/src/host/trxcon/src/trxcon           /usr/local/bin/trxcon \
    && install -m755 ${ROOT}/osmocom-bb/src/host/layer23/src/mobile/mobile   /usr/local/bin/mobile \
    && install -m755 ${ROOT}/osmocom-bb/src/host/virt_phy/src/virtphy        /usr/local/bin/virtphy
# ccch_scan is installed from the fixeria/burst_ind branch, not from here.

# Interactive shell conveniences (read when a session starts).
RUN echo "alias faketrx='python3 ${ROOT}/osmocom-bb/src/target/trx_toolkit/fake_trx.py'" >> ~/.bashrc
# ── Interactive prompt (user red / hostname blue / cwd green / ☎#) ──────────
# \h = container name, set by `docker run --hostname "$container_name"`
# (start-docker.sh).
RUN printf '%s\n' '' \
    '### calypso-prompt ###' \
    "export PS1='\[\033[1;31m\]\u\[\033[0m\]@\[\033[1;34m\]\h\[\033[0m\]:\[\033[1;32m\]\w\[\033[0m\]☎# '" \
    '### end calypso-prompt ###' >> /root/.bashrc
COPY configs/mobile.cfg /root/.osmocom/bb/mobile.cfg
RUN chmod +x /root/run.sh

# proto-SMSC directories + Asterisk tree.
RUN mkdir -p /var/log/osmocom /var/run/smsc \
        /etc/asterisk /var/lib/asterisk /var/log/asterisk /var/run/asterisk \
        /root/.osmocom/bb && \
    chmod 755 /etc/asterisk /var/lib/asterisk /var/log/asterisk \
        /var/run/asterisk /var/log/osmocom

# ── PulseAudio system mode: config baked at build time ──────────────────────
# Every daemon must be reachable from startup: anonymous socket, null sinks
# gsm_audio / gsm_mic / osmo_tts_off, root in pulse-access.
# gsm_mic must exist with the daemon: configs/asound.conf points the gsm_in
# capture at gsm_mic.monitor, and gapk_io drops both playback and capture if
# capture fails.
RUN set -eux; \
    usermod -aG pulse-access root 2>/dev/null || true; \
    test -f /etc/pulse/system.pa; \
    sed -i 's|^#\?load-module module-native-protocol-unix.*|load-module module-native-protocol-unix
     ↪  auth-anonymous=1 socket=/var/run/pulse/native|' /etc/pulse/system.pa; \
    grep -q 'auth-anonymous=1' /etc/pulse/system.pa \
      || echo 'load-module module-native-protocol-unix auth-anonymous=1 socket=/var/run/pulse/native' >>
       ↪  /etc/pulse/system.pa; \
    grep -q 'sink_name=gsm_audio' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=gsm_audio format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=GSM_Audio' >> /etc/pulse/system.pa; \
    grep -q 'sink_name=gsm_mic' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=gsm_mic format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=GSM_Mic' >> /etc/pulse/system.pa; \
    grep -q 'sink_name=osmo_tts_off' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=osmo_tts_off format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=TTS_off' >> /etc/pulse/system.pa; \
    sed -i 's|^load-module module-suspend-on-idle|#load-module module-suspend-on-idle|' /etc/pulse/system.pa; \
    mkdir -p /var/run/pulse && chown -R pulse:pulse /var/run/pulse

# root's shell profile activates the gr-gsm venv.
RUN echo 'source ~/.env/bin/activate' >> ~/.bashrc
```

```
# ── Camping bridge scripts -> /opt/GSM (required by /opt/GSM/qosmo/run.sh):
# si_bridge.py (full SI set -> 4730 -> shunt feed_si), si_bridge_loop.sh,
# record_drain.py (iq_record.fifo -> record.cfile), grgsm_fft_live.py.
COPY opt-gsm/. /opt/GSM/

# --- Image metadata ---------------------------------------------------------
# SIGRTMIN+3 is systemd's clean shutdown signal.
STOPSIGNAL SIGRTMIN+3
ENTRYPOINT ["/etc/osmocom/entrypoint.sh"]
CMD ["/bin/bash"]

# ── Stage `debs`: the .deb cache only ──────────────────────────────────────
# `osmocom-nitb` is NOT the last stage of this file: everything that builds the
# image must pass `--target osmocom-nitb` (build.sh compose_build, compose.yaml
# build.target, iso_modules/21-docker-build.sh, .github/workflows/docker.yml).
# Without it, docker builds `debs`: an image containing only ~2 GB of packages.
#
# Used by CI, where the image is pushed straight to GHCR without being loaded
# locally: a second pass `--target debs --output type=local` reuses the whole
# build cache and writes only /var/cache/osmo-debs to the runner disk, for
# actions/cache. On the host, build.sh uses debs_from_image instead.
FROM scratch AS debs
COPY --from=osmocom-nitb /var/cache/osmo-debs/ /
```

2.2 /opt/GSM/osmo-operator/Dockerfile.lite

5669 octets, 103 lignes → 103 lignes

```
# ============================================================================
# Dockerfile.lite - runtime image: the binaries, without the source trees
# ============================================================================
#
# Purpose: derive osmocom-nitb:lite from the osmocom-nitb image by removing the
# build trees under /opt/GSM (gnuradio, gr-gsm, libosmocore, qemu, osmo-*...)
# and the rest of the build toolchain, while keeping everything the stack
# launches at runtime.
#
# Build:    ./build.sh --lite      (or: docker build -f Dockerfile.lite .)
# ISO:      ./build-iso.sh --role=operator --lite
# Args:     BASE   image to prune (default: osmocom-nitb)
#
# Why two stages and a flatten: deleting files in an extra layer frees nothing
# (the image is the sum of its layers). The prune stage trims the tree, then
# FROM scratch + COPY writes only the pruned root as a single layer.
# Starting from the complete root, rather than reinstalling a hand-written
# package list, keeps every runtime library the /usr/local binaries depend on.
#
# Kept in /opt/GSM (what the stack actually launches):
#   qosmo/                              whole tree (run.sh, run_modules/,
#                                       environnement/, cfgs/, scripts/, .git),
#                                       build/ reduced to qemu-system-arm and
#                                       qemu-bundle/
#   osmocom-bb/src/host/osmocon/osmocon the loader
#   osmocom-bb/src/target/trx_toolkit   fake_trx.py and friends
#   c54x_exe/, grgsm_exe/               layer-1 implementations outside QEMU
#   calypso_dsp.*.bin                   DSP ROMs
#   firmware/, qemu-install/, osmo-operator/, pont/
#
# The repositories keep their .git: "osmo-update" runs git fetch in place on
# the running machine instead of deleting and recloning.
# ============================================================================

ARG BASE=osmocom-nitb
```

```
# ── Stage 1: prune ──────────────────────────────────────────────────
FROM ${BASE} AS prune

# Implicit set -e (default SHELL): a failed prune does not produce a half-cut
# image. Everything happens in ONE layer; only its final state is copied.
RUN set -eux; \
    cd /opt/GSM; \
    mkdir -p /keep/qosmo/build /keep/osmocom-bb/src/host/osmocon \
             /keep/osmocom-bb/src/target; \
    # qosmo: the whole tree except build/. The binary is set aside BEFORE the
    # copy, and --exclude=./build keeps the tar from overwriting it.
    cp -a qosmo/build/qemu-system-arm  /keep/qosmo/build/ 2>/dev/null || true; \
    # qemu-bundle must follow the binary: QEMU relocates its data directory
    # relative to the binary location, and in a build tree that goes through
    # build/qemu-bundle/. Without it qemu-system-arm cannot read its keymap
    # ("could not read keymap file: 'en-us'") and dies before allocating PTYs.
    cp -a qosmo/build/qemu-bundle      /keep/qosmo/build/ 2>/dev/null || true; \
    tar -C qosmo --exclude=./build -cf - . | tar -C /keep/qosmo -xf -; \
    # c54x_exe (--dsp) and grgsm_exe: small trees (binary + run.sh + rom/),
    # kept whole, .git included (osmo-update). Their C sources live in qosmo.
    for d in c54x_exe grgsm_exe; do \
        [ -d "$d" ] && cp -a "$d" /keep/ || true; \
    done; \
    cp -a osmocom-bb/src/host/osmocon/osmocon \
          /keep/osmocom-bb/src/host/osmocon/ 2>/dev/null || true; \
    cp -a osmocom-bb/src/target/trx_toolkit \
          /keep/osmocom-bb/src/target/ 2>/dev/null || true; \
    cp -a *.bin /keep/ 2>/dev/null || true; \
    for d in firmware qemu-install osmo-operator pont; do \
        [ -e "$d" ] && cp -a "$d" /keep/ || true; \
    done; \
    # The cut: all of /opt/GSM goes, only the kept artefacts come back.
    cd /; rm -rf /opt/GSM; mkdir -p /opt/GSM; \
    cp -a /keep/. /opt/GSM/; rm -rf /keep; \
    # Rest of the toolchain: headers, static libraries, objects, caches.
    # /var/cache/osmo-debs (packaging/osmo-deb.sh) is a BUILD cache that lives
    # on the host; it has no place in the pruned runtime image.
    rm -rf /usr/local/include /root/.cache /root/.cargo /root/.rustup \
           /usr/share/doc /usr/share/man /usr/share/locale \
           /var/lib/apt/lists/* /var/cache/apt/archives/*.deb \
           /var/cache/osmo-debs /var/cache/apt/apt-fast \
           /tmp/* /var/tmp/*; \
    find /usr/local/lib -name '*.a' -delete 2>/dev/null || true; \
    # The .o sweep SKIPS /opt/GSM: some .o there are tracked by git, and
    # deleting them would leave a modified tree that "osmo-update" refuses
    # to fast-forward.
    find / -xdev -path /opt/GSM -prune -o -name '*.o' -exec rm -f {} + 2>/dev/null || true; \
    du -sh /opt/GSM /usr/local/bin /usr/local/lib

# ── Stage 2: flatten ────────────────────────────────────────────────
# FROM scratch + COPY of the root: a single layer holding only the pruned
# state. This is what actually reduces the image size.
FROM scratch
COPY --from=prune / /

# Configuration does not survive FROM scratch: redeclare it identically to the
# original image, otherwise the lite image does not start the same way.
ENV PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin \
    container=docker \
    OSMO_IMAGE=1 \
    PKG_CONFIG_PATH=/usr/local/lib/pkgconfig \
    LD_LIBRARY_PATH=/usr/local/lib \
    DEBIAN_FRONTEND=noninteractive

WORKDIR /etc/osmocom
ENTRYPOINT ["/etc/osmocom/entrypoint.sh"]
```

```
CMD ["/bin/bash"]
```

2.3 /opt/GSM/osmo-operator/Dockerfile.run

7860 octets, 118 lignes → 118 lignes

```
# Dockerfile.run - update layer on top of osmocom-nitb (runtime image osmocom-run)
# ──────────────────────────────────────────────────────────────────────────────
# This file builds nothing from scratch. It starts from the osmocom-nitb base
# image (Dockerfile) and refreshes only what changes between lab iterations:
#   - the repository's scripts and configs (COPY, a few seconds);
#   - the git trees qosmo, c54x_exe, grgsm_exe and osmo-operator (fetch + rebuild).
# Everything else (apt packages, Node/web dashboard, Calypso firmware,
# PulseAudio config, prompt, systemd units...) lives in Dockerfile; there is
# no apt call here.
#
# Build:  docker build -f Dockerfile.run .      (or via build.sh / start-docker.sh)
# Args:   BASE             base image (default: osmocom-nitb; the CI passes its
#                          GHCR reference osmocom-nitb:base-<context fingerprint>)
#         QEMU_CACHE_BUST  any changing value (e.g. $(date +%s)) to force the
#                          git refresh RUNs past the build cache
#
# The COPYs below overlap with Dockerfile on purpose: that overlap is what lets
# a config change be rebuilt in seconds instead of rebuilding the base image.
# Dockerfile keeps the same COPYs because osmocom-nitb must stay self-contained.
ARG BASE=osmocom-nitb
FROM ${BASE} AS osmocom-run

# ── Scripts ──────────────────────────────────────────────────────────────────
# Two destinations, two uses:
#   /scripts      -> entrypoint.sh (the ENTRYPOINT below) and the tools that
#                    scripts/run.sh calls through this path (pulse-gsm-setup.sh);
#   /etc/osmocom  -> what the modules consume ($OSMOCOM_CFG/...). The base
#                    image already has them, but at the state of ITS build;
#                    this COPY refreshes osmo-start.sh / status.sh / check.sh /
#                    audio-chain.sh too.
# The repository tracks these files as 644: the +x bit is set AFTER all COPYs,
# in the RUN further down. A COPY placed after that chmod would reset the mode.
COPY scripts/* /etc/osmocom/
COPY scripts/entrypoint.sh /scripts/

# ── Osmocom configs (templates with __XXX__ placeholders) ────────────────────
# All configs are copied; the real values are injected at start-up by
# start-docker.sh (apply_config_templates) and mounted as volumes.
COPY configs/*.conf /etc/asterisk/
COPY configs/*.cfg configs/sms-routing.conf /etc/osmocom/
COPY configs/mobile.cfg /root/.osmocom/bb/mobile.cfg
COPY configs/asound.conf /etc/

# ── Burst-level TRX bridge (CALYPSO_BRIDGE=pont) ─────────────────────────────
# pont.py is NOT copied and no PONT_PY is exported: the repository tree under
# /opt/GSM/osmo-operator is the single source. start-native.sh resolves
# $NITB_ROOT/pont/pont.py and c54x_exe/run.sh (MODE=dsp) picks pont_dsp.py;
# each launcher has its own default.

# Execute bits. MUST stay AFTER all the COPYs above.
RUN chmod +x /etc/osmocom/*.sh

# ── Disclaimer at container login (counterpart of the ISO's 01-osmo-disclaimer.sh)
# `docker exec -ti <ct> bash` opens an interactive NON-login shell, which reads
# ~/.bashrc (not /etc/profile.d). The text lives in scripts/osmo-disclaimer.sh
# (already in /etc/osmocom/ from the COPY above) and is sourced from .bashrc,
# guarded by:
#   - [ -t 1 ]               : only when a human has a terminal;
```

```
#   - OSMO_DISCLAIMER_SHOWN   : once per session, not in every sub-shell.
# The non-interactive `bash -c` of start-docker.sh never reads .bashrc, so the
# stack start-up output is not polluted. No heredoc here: the classic docker
# builder (no BuildKit) does not support RUN heredocs; a repository file plus a
# symlink works with both builders. `grep -q` keeps the append idempotent.
RUN ln -sf /etc/osmocom/osmo-disclaimer.sh /etc/osmo-disclaimer.sh && \
    { grep -q '### osmo-disclaimer ###' /root/.bashrc || \
      printf '%s\n' '' \
        '### osmo-disclaimer ###' \
        '[ -z "${OSMO_DISCLAIMER_SHOWN:-}" ] && [ -t 1 ] && . /etc/osmo-disclaimer.sh' \
        '### end osmo-disclaimer ###' >> /root/.bashrc; }

# ── Git tree refresh ─────────────────────────────────────────────────────
# start-docker.sh passes --build-arg QEMU_CACHE_BUST=$(date +%s). An ARG
# invalidates the cache at its FIRST USE, not at its declaration: hence the
# `: "${QEMU_CACHE_BUST:-}"` at the head of each RUN below, which consumes it.
ARG QEMU_CACHE_BUST

# The base image clone leaves HEAD on the default branch: a plain pull is
# enough. The pull must never kill the build: the build network may fail
# (git then prints a misleading "could not read Username" although the
# repository is public). GIT_TERMINAL_PROMPT=0 fails immediately instead of
# blocking on an invisible prompt, and `|| echo` keeps the base clone's tree,
# with a message saying which of the two we got.
RUN : "${QEMU_CACHE_BUST:-}" && cd /opt/GSM/osmo-operator \
    && (GIT_TERMINAL_PROMPT=0 git pull --ff-only \
        || echo "[warn] git pull osmo-operator impossible (reseau du build) - on garde l arbre du clone de base")

# ── Rebuild qosmo + c54x_exe + grgsm_exe (fetch + rebuild) ──────────────
# One QEMU tree (/opt/GSM/qosmo) and two layer-1 implementations outside QEMU,
# c54x_exe and grgsm_exe, which compile sources from /opt/GSM/qosmo
# (hw/arm/calypso, contrib/hors-qemu). All three are refreshed in the SAME RUN:
# a c54x_exe rebuilt against a stale qosmo (or the reverse) would give a DSP
# and an ARM speaking different bridge protocols with nothing reporting it.
# Do not split.
# fetch + reset --hard rather than `pull --ff-only`: the base trees carry
# modified tracked files (__pycache__/*.pyc in qosmo, the versioned c54x_exe
# binary) that a pull refuses to merge. These trees are sources, not a
# workspace: align them on origin.
# gcc-13 for ninja: qosmo's build/ was configured with gcc-13 (meson recorded
# -ftrivial-auto-var-init, unknown to gcc-11, the image's default cc). A local
# PATH is enough (update-alternatives does not know gcc-13); the default stays gcc-11.
# c54x_exe: CFLAGS without -march=native, as in the base Dockerfile.
RUN : "${QEMU_CACHE_BUST:-}" && \
    for d in qosmo c54x_exe grgsm_exe; do \
        (cd /opt/GSM/$d && GIT_TERMINAL_PROMPT=0 git fetch origin \
            && git reset --hard "origin/$(git rev-parse --abbrev-ref HEAD)") \
            || echo "[warn] mise a jour de $d impossible (reseau ?) - on rebatit sur l arbre present"; \
    done && \
    mkdir -p /tmp/cc13 && ln -sf /usr/bin/gcc-13 /tmp/cc13/cc && ln -sf /usr/bin/gcc-13 /tmp/cc13/gcc \
    && ln -sf /usr/bin/g++-13 /tmp/cc13/c++ && ln -sf /usr/bin/g++-13 /tmp/cc13/g++ && \
    (cd /opt/GSM/qosmo/build && PATH=/tmp/cc13:$PATH ninja) && rm -rf /tmp/cc13 && \
    make -C /opt/GSM/c54x_exe QOSMO=/opt/GSM/qosmo \
        CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function -Wno-unused-variable
 ↪  -Wno-unused-but-set-variable -Wno-sign-compare" && \
    cp /opt/GSM/c54x_exe/rom/calypso_dsp.*.bin /opt/GSM/ && \
    make -C /opt/GSM/grgsm_exe QOSMO=/opt/GSM/qosmo clean all

# ── Entrypoint ──────────────────────────────────────────────────────────
ENTRYPOINT ["/scripts/entrypoint.sh"]
CMD ["bash"]
```

2.4 /opt/GSM/osmo-operator/Dockerfile.stp

8884 octets, 171 lignes → 171 lignes

```
# ============================================================================
# Dockerfile.stp - inter-STP image: the SS7/M3UA hub of the WAN, nothing else
# ============================================================================
#
# osmocom-run carries the whole mock-up (HLR, MSC, BSC, PCU, BTS, Asterisk,
# QEMU, gr-gsm...). The hub uses none of it: it receives M3UA, looks up its
# routing table and forwards to the right linkset. This image contains only
# that. osmo-stp has no repository of its own; it is built by libosmo-sigtran,
# hence three sources (libosmocore, libosmo-netif, libosmo-sigtran).
#
# Versions are aligned with Dockerfile: a hub built against a newer
# libosmo-sigtran than the nodes would negotiate M3UA extensions they ignore,
# which only shows up as "ASP DOWN".
#
# Two ways to build it:
#
#   standalone (default): compiles the three libraries from ubuntu:24.04
#   (noble, the same base as Dockerfile), through the .deb cache of
#   packaging/osmo-deb.sh (the second build compiles nothing).
#     docker build -f Dockerfile.stp -t osmocom-stp .
#     docker compose build stp          (build.sh / build-iso.sh --role=interstp)
#
#   on the existing stack: osmocom-nitb (the base of Dockerfile.run) already
#   carries these libraries and osmo-stp; they are reused instead of being
#   recompiled, so hub and nodes share the SAME code.
#     docker build -f Dockerfile.stp --build-arg BASE_IMAGE=osmocom-nitb -t osmocom-stp .
#
# Args:  BASE_IMAGE            ubuntu:24.04 (default) or osmocom-nitb
#        LIBOSMOCORE_VERSION, LIBOSMO_NETIF_VERSION, LIBOSMO_SIGTRAN_VERSION
#        OSMO_UBUNTU_MIRROR    apt mirror for apt-fast-install
#        OSMO_DEB_REFRESH      1 forces a rebuild past the .deb cache
#        HUB_NODES, HUB_OPS    size of the default point-code plan
#
# Run:
#   docker run --rm -p 2908:2908 -p 4239:4239 osmocom-stp
#   docker run --rm -v ./hub.cfg:/etc/osmocom/osmo-stp-interop.cfg osmocom-stp
# ============================================================================

# ── Base shared by both stages ──────────────────────────────────────────
# The package list is the one from Dockerfile, taken as is. Deliberately wider
# than what these three libraries seem to need: libosmocore's configure enables
# or drops components according to what it finds (python3 is needed by its code
# generators). Declared once so it cannot diverge between the stage that
# compiles and the one that runs.
ARG BASE_IMAGE=ubuntu:24.04
FROM ${BASE_IMAGE} AS base
ARG DEBIAN_FRONTEND=noninteractive
# apt-fast: the same installer as Dockerfile (packaging/apt-fast-install.sh).
# BuildKit apt cache, the same as Dockerfile (id osmo-apt-archives).
ARG OSMO_UBUNTU_MIRROR=
COPY packaging/apt-fast-install.sh /usr/local/sbin/apt-fast-install
RUN --mount=type=cache,id=osmo-apt-archives,target=/var/cache/apt/archives,sharing=locked \
    chmod 755 /usr/local/sbin/apt-fast-install && apt-fast-install
# apt is idempotent: on osmocom-nitb, where everything is already installed,
# this step only confirms it, so the list is not duplicated for that case.
# libasound2t64: noble name of libasound2 (64-bit time_t transition).
RUN --mount=type=cache,id=osmo-apt-archives,target=/var/cache/apt/archives,sharing=locked \
    apt-fast update && apt-fast install -y --no-install-recommends \
    build-essential git gcc g++ make cmake autoconf automake libtool pkg-config wget curl \
    libtalloc-dev libpcsclite-dev libsctp-dev libmnl-dev liburing-dev \
    libdbi-dev libdbd-sqlite3 libsqlite3-dev sqlite3 libc-ares-dev libgnutls28-dev \
    libortp-dev libfftw3-dev libusb-1.0-0-dev libsofia-sip-ua-dev libsofia-sip-ua-glib-dev \
```

```
    python3 python3-dev python3-scapy ca-certificates \
    libasound2-dev libasound2t64 libgsm1-dev libgsm1 libpcap0.8-dev libsystemd-dev \
    && rm -rf /var/lib/apt/lists/*
ENV PKG_CONFIG_PATH=/usr/local/lib/pkgconfig \
    LD_LIBRARY_PATH=/usr/local/lib

# ── Build stage ──────────────────────────────────────────────────────
FROM base AS builder
ARG LIBOSMOCORE_VERSION=1.12.1
ARG LIBOSMO_NETIF_VERSION=1.7.0
ARG LIBOSMO_SIGTRAN_VERSION=2.2.1

WORKDIR /build

# ── .deb cache (packaging/osmo-deb.sh): the same packages as the nitb image ──
# .deb-cache/ mirrors the host cache (/var/cache/osmo-debs). A package
# osmo-build-libosmocore_1.12.1~noble already present = no compilation.
ARG OSMO_DEB_REFRESH=0
COPY packaging/osmo-deb.sh /usr/local/sbin/osmo-deb
COPY .deb-cache/ /var/cache/osmo-debs/
RUN chmod 755 /usr/local/sbin/osmo-deb

# Fixed order: each library builds against the previous one.
# libosmocore from github (official mirror), the other two from gitea, the
# same origins as Dockerfile so both images see the same code.
#
# If the base already provides osmo-stp (BASE_IMAGE=osmocom-nitb), nothing is
# recompiled: a second copy of the same libraries could diverge from the nodes'.
RUN set -eux; \
    if command -v osmo-stp >/dev/null 2>&1; then \
        echo "osmo-stp fourni par l'image de base — compilation inutile"; \
        osmo-stp --version; \
        exit 0; \
    fi; \
    for repo in \
        "libosmocore:${LIBOSMOCORE_VERSION}:https://github.com/osmocom/libosmocore" \
        "libosmo-netif:${LIBOSMO_NETIF_VERSION}:https://gitea.osmocom.org/osmocom/libosmo-netif" \
        "libosmo-sigtran:${LIBOSMO_SIGTRAN_VERSION}:https://gitea.osmocom.org/osmocom/libosmo-sigtran" \
    ; do \
        name="${repo%%:*}"; rest="${repo#*:}"; \
        version="${rest%%:*}"; url="${rest#*:}"; \
        if osmo-deb install "$name" "$version"; then continue; fi; \
        git clone "$url" "$name"; \
        cd "/build/$name"; \
        git checkout "$version"; \
        autoreconf -fi; \
        ./configure --prefix=/usr/local --disable-static; \
        make -j"$(nproc)"; \
        osmo-deb pack "$name" "$version" make install; \
        ldconfig; \
        cd /build; \
    done

# Default hub config, generated by the repository script: a single source of
# truth for the point-code plan 1.<node><op>.<role>, shared with the ISO and
# start-docker.sh. Can be replaced by a volume mount at run time.
ARG HUB_NODES=3
ARG HUB_OPS=1
COPY helpers/create_interop.sh /build/create_interop.sh
RUN mkdir -p /etc/osmocom \
    && bash /build/create_interop.sh --wan "${HUB_NODES}" "${HUB_OPS}" \
         /etc/osmocom/osmo-stp-interop.cfg

# ── Final image ──────────────────────────────────────────────────────
# Same base, hence same libraries: nothing the build may have enabled can be
# missing. Left out: the source trees and build objects, most of the volume.
```

```
FROM base
LABEL org.opencontainers.image.title="osmocom-stp" \
      org.opencontainers.image.description="Inter-STP osmo-operator — hub SS7/M3UA, PC 0.0.0"

# groupadd and useradd fail (code 9) if the account already exists, which is
# the case with BASE_IMAGE=osmocom-nitb (the main Dockerfile creates it).
RUN getent group osmocom >/dev/null || groupadd -r osmocom \
    && id -u osmocom >/dev/null 2>&1 \
       || useradd -r -g osmocom -s /usr/sbin/nologin -d /var/lib/osmocom osmocom \
    && mkdir -p /var/lib/osmocom /var/log/osmocom \
    && chown osmocom:osmocom /var/lib/osmocom /var/log/osmocom

# /usr/local is where both paths put their result: the compiling one
# (--prefix=/usr/local) and osmocom-nitb (same prefix in Dockerfile). One copy
# covers both cases.
COPY --from=builder /usr/local/bin/osmo-stp /usr/local/bin/osmo-stp
COPY --from=builder /usr/local/lib/         /usr/local/lib/
COPY --from=builder /etc/osmocom/osmo-stp-interop.cfg /etc/osmocom/osmo-stp-interop.cfg
# The produced packages travel in the final image: build.sh brings them back
# into the host cache (docker cp), and build-iso.sh --role=interstp installs
# them into the hub rootfs with dpkg.
COPY --from=builder /var/cache/osmo-debs/ /var/cache/osmo-debs/

# Dependency closure checked on the SHIPPED image, binary and libraries: a
# missing .so would otherwise only show at first start, inside a VM.
RUN set -eux; \
    ldconfig; \
    fail=0; \
    for f in /usr/local/bin/osmo-stp /usr/local/lib/*.so*; do \
        [ -f "$f" ] || continue; \
        if ldd "$f" 2>/dev/null | grep 'not found'; then \
            echo "DÉPENDANCE MANQUANTE : $f" >&2; fail=1; \
        fi; \
    done; \
    [ "$fail" = "0" ]; \
    osmo-stp --version

# 2908: M3UA over SCTP, which EXPOSE cannot declare (tcp/udp only).
# 4239: VTY.
EXPOSE 2908/tcp 4239/tcp

ENTRYPOINT ["/usr/local/bin/osmo-stp"]
CMD ["-c", "/etc/osmocom/osmo-stp-interop.cfg"]
```

2.5 /opt/GSM/osmo-operator/addition.sh

68727 octets, 1163 lignes → 1162 lignes (1 groupes compactés)

```
#!/bin/bash
# ============================================================================
# addition.sh - supplements that are not shipped in the ISO
#
# Companion to update.sh, launched the same way: a GTK desktop icon
# (osmo-addition-anim), or by hand.
#
# Why docker is not in the image:
# The ISO is a NATIVE node: it carries the full bench without containers, so
# it needs docker for nothing. Embedding it would cost ~500 MB and the daemon
# would run permanently on a bench that does not use it - with its bridge,
# its iptables rules and its MASQUERADE, on a machine whose whole point is to
# route GSM by hand.
# Docker serves only ONE scenario: the multi-operator (start-multi.sh), where
# operators 2 and 3 and the inter-STP run in containers beside the native one.
# It is a supplement, installed as such - here.
```

```
#
# Usage:
#   sudo ./addition.sh            GTK window: "Docker container and SS7
#                                 multioperator", then a second window with
#                                 radio buttons: DOWNLOADED or BUILT image
#                                 (exclusive). osmocom-run is derived right
#                                 after, then osmo-multi.service STARTS: on
#                                 exit, osmo-operator-2, -3 and the hub are
#                                 running (OSMO_MULTI_START=0: install without
#                                 launching).
#   sudo ./addition.sh --multi    the same, without the window: DOWNLOADED
#                                 image (docker pull ghcr.io/<repo>/osmocom-nitb,
#                                 tagged osmocom-nitb)
#   sudo ./addition.sh --multi-build
#                                 the same. This flag no longer COMPILES: it
#                                 also pulls the image from the repo, which
#                                 the CI publishes from the same Dockerfile.
#                                 To really compile (modified Dockerfile,
#                                 nothing published):
#                                   OSMO_ADDITION_BUILD_LOCAL=1 ... --multi-build
#   sudo ./addition.sh --docker   the container engine only   (troubleshooting)
#   sudo ./addition.sh --image    the operator image only, downloaded (troubleshooting)
#   sudo ./addition.sh --build    the operator image only, from the repo   (troubleshooting)
#   sudo ./addition.sh --opencl   the OpenCL stack only (GPU compute)
#   sudo ./addition.sh --claude   Claude Code (assistant CLI) only
#   sudo ./addition.sh --extras   Games + media (Doom, Quake III, Kodi,
#                                 YouTube+uBlock, Wireshark root, Linphone)
#   sudo ./addition.sh --gnuarm   the legacy arm-elf-gcc 4.5.2 toolchain (/root/gnuarm),
#                                 copied from the osmocom-bb-trx Docker image
#   sudo ./addition.sh --magnetite Wine + TI TMS470, FreeCalypso Magnetite C139
#                                 (sources, bench patches, flash image)
#   sudo ./addition.sh --trx      osmocom-bb-trx: published Docker image + trx
#                                 (TX) firmware built with gnuarm
#   sudo ./addition.sh --calypso-fw bench osmocom-bb rebuilt with gnuarm (cd src && make):
#                                 layer1 GPRS, rssi (OSMO_FW_INSTALLER=1: install them)
#   sudo ./addition.sh --fernvale fernly, fernvale-nuttx, layer1 MTK (MT6260)
#   sudo ./addition.sh --simcom   SIMCom documentation and SDK (~3.4 GB)
#   sudo ./addition.sh --firmwares the previous six
#   sudo ./addition.sh --status   only reports what is present
# ============================================================================
set -uo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

DIR="$(cd "$(dirname "$(readlink -f "$0")")" && pwd)"
MULTI_CONF="${MULTI_CONF:-/etc/osmocom/osmo-multi.conf}"
# Image name: osmocom-run, NOT "osmo-operator". build.sh produces osmocom-nitb
# (Dockerfile, ~11 GB), then Dockerfile.run derives osmocom-run, which is the
# one start-docker.sh runs (docker inspect osmo-operator-1 --format
# {{.Config.Image}} returns osmocom-run). "osmo-operator" is the REPO name, not
# the image name, so probing for it would always report the image as absent.
MULTI_IMAGE="${MULTI_IMAGE:-osmocom-run}"
# Base image and its origin. osmocom-nitb is what build.sh compiles. It is
# PULLED and tagged osmocom-nitb; start-docker.sh then derives osmocom-run from
# it via Dockerfile.run, exactly as after a local build.
#
# The source is GHCR, not Docker Hub: .github/workflows/docker.yml publishes
# the image on every push to ghcr.io/<repo>/osmocom-nitb - the SAME stack, built
# by the CI from the SAME Dockerfile, following the repo instead of a manually
# maintained Hub account. bastienbaranoff/norf_gsm stays reachable via HUB_IMAGE.
#   :latest      the latest published (default)
#   :base-<sha>  a precise context fingerprint, to pin
BASE_IMAGE="${BASE_IMAGE:-osmocom-nitb}"
HUB_IMAGE="${HUB_IMAGE:-ghcr.io/bbaranoff/osmo-operator/osmocom-nitb:latest}"
```

```
# ── _trust_desktop: make a desktop shortcut VISIBLE to DING ──────────────
# DING shows a .desktop with its name and icon only if it is executable, owned
# by the desktop user, AND carries metadata::trusted. That attribute does NOT
# live in the file: it is written in the SESSION's gvfs metadata, so gio must
# talk to THAT user's bus.
#
# addition.sh is launched by `pkexec env DISPLAY=... XAUTHORITY=... `
# (osmo-addition-anim); pkexec CLEANS the environment and passes neither
# XDG_RUNTIME_DIR nor DBUS_SESSION_BUS_ADDRESS, so `gio set metadata::trusted`
# has no bus to reach. We rebuild the env from the desktop owner's UID and run
# gio AS that user.
_trust_desktop() {
    local f="$1" home="$2" pos="${3:-}" owner uid bus
    [ -f "$f" ] || return 1
    owner="$(stat -c '%U' "$home" 2>/dev/null)"; [ -n "$owner" ] || owner=root
    uid="$(id -u "$owner" 2>/dev/null)" || return 1
    chown "$owner" "$f" 2>/dev/null || true
    chmod +x "$f" 2>/dev/null || true
    bus="/run/user/$uid/bus"
    # Active session for this owner: set the attributes RIGHT NOW. Otherwise (no
    # bus), osmo-trust-desktop (autostart) sets trusted at the next login, and
    # the icon then appears without intervention.
    # pos "x,y" (optional): place the icon on the desktop via DING.
    if [ -S "$bus" ] && command -v runuser >/dev/null 2>&1; then
        runuser -u "$owner" -- env XDG_RUNTIME_DIR="/run/user/$uid" \
            DBUS_SESSION_BUS_ADDRESS="unix:path=$bus" \
            gio set -t string "$f" metadata::trusted true 2>/dev/null || true
        [ -n "$pos" ] && runuser -u "$owner" -- env XDG_RUNTIME_DIR="/run/user/$uid" \
            DBUS_SESSION_BUS_ADDRESS="unix:path=$bus" \
            gio set -t string "$f" metadata::nautilus-icon-position "$pos" 2>/dev/null || true
    fi
    return 0
}

# ── Desktops to serve: /root AND the graphical session's ─────────────────
# Find the GRAPHICAL SESSION account, in this order:
#   0. OSMO_COMPTE: the account created by the installer, passed by
#      osmo-supplements-firstboot.sh. At first boot nobody is logged in yet:
#      without it, neither docker group nor icon for that account.
#   1. SUDO_USER   (sudo)
#   2. PKEXEC_UID  (the icon, pkexec)
#   3. the owner of an active session bus (/run/user/*/bus) - systemd, root
#      shell, autologin: the only proof that a session exists.
# SUDO_USER exists ONLY under sudo, and these scripts are launched by an icon
# (pkexec, which sets PKEXEC_UID), a systemd unit, or a root shell - so relying
# on SUDO_USER alone would leave the installer's account with a bare desktop.
# /root is ALWAYS served: the live key opens its session as root.
_comptes_session() {
    local u uid bus
    for u in "${OSMO_COMPTE:-}" "${SUDO_USER:-}" \
             "$([ -n "${PKEXEC_UID:-}" ] && getent passwd "$PKEXEC_UID" | cut -d: -f1)"; do
        [ -n "$u" ] && [ "$u" != root ] || continue
        getent passwd "$u" >/dev/null 2>&1 && { printf '%s\n' "$u"; return 0; }
    done
    for bus in /run/user/*/bus; do
        [ -S "$bus" ] || continue
        uid="${bus#/run/user/}"; uid="${uid%/bus}"
        [ "$uid" = 0 ] && continue
        u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
        [ -n "$u" ] && printf '%s\n' "$u"
    done
}

_bureau_homes() {
    local homes=(/root) u uh
    while read -r u; do
```

```
        [ -n "$u" ] || continue
        uh="$(getent passwd "$u" 2>/dev/null | cut -d: -f6)"
        [ -n "$uh" ] && [ -d "$uh" ] && homes+=("$uh")
    done < <(_comptes_session)
    printf '%s\n' "${homes[@]}"
}

# ── The session account must be able to talk to docker ──────────────────
# The Conky and the desktop panel run under the SESSION account. If it is in no
# docker group, every probe (`docker exec ... pgrep`) returns "permission denied
# while trying to connect to the docker API", so the bench reads as stopped even
# while the operators run. Add the account to the docker group; it only takes
# effect at the next login, which is reported rather than left looking broken.
_docker_groupe_session() {
    getent group docker >/dev/null 2>&1 || return 0
    local u
    while read -r u; do
        [ -n "$u" ] || continue
        id -nG "$u" 2>/dev/null | tr ' ' '\n' | grep -qx docker && continue
        usermod -aG docker "$u" 2>/dev/null || continue
        echo -e "  ${GREEN}✓${NC} ${BOLD}$u${NC} ajoute au groupe ${BOLD}docker${NC} (le Conky et l encart voient
        ↪  les conteneurs)"
        echo -e "    ${YELLOW}!${NC} effectif au prochain login de $u"
    done < <(_comptes_session)
}

DO_DOCKER=0; DO_IMAGE=0; DO_MULTI=0; DO_OPENCL=0; DO_CLAUDE=0; STATUS_ONLY=0; ANY_FLAG=0
DO_BUILD=0; DO_EXTRAS=0; DO_SIM=0
DO_GNUARM=0; DO_MAGNETITE=0; DO_TRX=0; DO_FERNVALE=0; DO_SIMCOM=0; DO_CALFW=0
for a in "$@"; do
    case "$a" in
        --docker) DO_DOCKER=1; ANY_FLAG=1 ;;
        --image)  DO_IMAGE=1;  ANY_FLAG=1 ;;
        --build)  DO_IMAGE=1;  DO_BUILD=1; ANY_FLAG=1 ;;
        --multi)  DO_MULTI=1;  ANY_FLAG=1 ;;
        --multi-build) DO_MULTI=1; DO_BUILD=1; ANY_FLAG=1 ;;
        --opencl) DO_OPENCL=1; ANY_FLAG=1 ;;
        --claude) DO_CLAUDE=1; ANY_FLAG=1 ;;
        --extras) DO_EXTRAS=1; ANY_FLAG=1 ;;
        --sim)    DO_SIM=1;    ANY_FLAG=1 ;;
        --gnuarm)    DO_GNUARM=1;    ANY_FLAG=1 ;;
        --magnetite) DO_MAGNETITE=1; ANY_FLAG=1 ;;
        --trx)       DO_TRX=1;       ANY_FLAG=1 ;;
        --calypso-fw) DO_CALFW=1;    ANY_FLAG=1 ;;
        --fernvale)  DO_FERNVALE=1;  ANY_FLAG=1 ;;
        --simcom)    DO_SIMCOM=1;    ANY_FLAG=1 ;;
        --firmwares) DO_GNUARM=1; DO_MAGNETITE=1; DO_TRX=1; DO_CALFW=1; DO_FERNVALE=1; DO_SIMCOM=1; ANY_FLAG=1 ;;
        --all)    DO_DOCKER=1; DO_IMAGE=1; DO_MULTI=1; DO_OPENCL=1; DO_CLAUDE=1; DO_EXTRAS=1; DO_SIM=1; ANY_FLAG=1
        ↪  ;;
        --status) STATUS_ONLY=1; ANY_FLAG=1 ;;
        -h|--help) sed -n '2,54p' "$0"; exit 0 ;;
    esac
done

echo -e "${CYAN}${BOLD}"
echo "╔═══════════════════════════════════════════════════════════╗"
echo "║   osmo-operator - supplements (hors ISO)                  ║"
echo "╚═══════════════════════════════════════════════════════════╝"
echo -e "${NC}"

etat() {
    local ok_d=0 ok_r=0 ok_i=0 ok_m=0 ok_u=0 ok_k=0
    command -v docker >/dev/null 2>&1 && ok_d=1
    [ "$ok_d" = "1" ] && docker info >/dev/null 2>&1 && ok_r=1
    [ "$ok_r" = "1" ] && docker image inspect "$MULTI_IMAGE" >/dev/null 2>&1 && ok_i=1
    [ -f "$MULTI_CONF" ] && ok_m=1
```

```
    # Report the same conditions launch.sh --multi requires: the
    # osmo-multi.service unit, start-multi.sh and the topology. Read here what
    # the icon checks at click time.
    systemctl cat osmo-multi.service >/dev/null 2>&1 && ok_u=1
    [ -f /usr/share/applications/osmo-multi.desktop ] && ok_k=1
    local ok_cl=0
    # clinfo returning at least one platform is the only proof the OpenCL stack
    # is USABLE. Installed packages prove nothing - an ICD with no driver behind
    # it leaves "Number of platforms 0".
    command -v clinfo >/dev/null 2>&1 \
        && [ "$(clinfo -l 2>/dev/null | grep -c Platform)" -gt 0 ] && ok_cl=1
    local ok_cc=0
    # claude answering --version is the only proof it is USABLE. The binary
    # installed without node behind it (broken npm install) exits with an error.
    command -v claude >/dev/null 2>&1 \
        && claude --version >/dev/null 2>&1 && ok_cc=1
    [ "$ok_d" = "1" ] && echo -e "  docker installe      : ${GREEN}oui${NC}"      || echo -e "  docker installe
     ↪  : ${YELLOW}non${NC}"
    [ "$ok_r" = "1" ] && echo -e "  demon actif          : ${GREEN}oui${NC}"      || echo -e "  demon actif
     ↪  : ${YELLOW}non${NC}"
    [ "$ok_i" = "1" ] && echo -e "  image operateur      : ${GREEN}presente${NC}" || echo -e "  image operateur
     ↪  : ${YELLOW}absente${NC}"
    [ "$ok_m" = "1" ] && echo -e "  topologie SS7        : ${GREEN}posee${NC}"    || echo -e "  topologie SS7
     ↪  : ${YELLOW}absente${NC}"
    [ "$ok_u" = "1" ] && echo -e "  osmo-multi.service   : ${GREEN}pose${NC} ($(systemctl is-active
     ↪  osmo-multi.service 2>/dev/null || echo inactive))" || echo -e "  osmo-multi.service   :
     ↪  ${YELLOW}absent${NC}"
    [ "$ok_k" = "1" ] && echo -e "  icone multi-operator : ${GREEN}posee${NC}"    || echo -e "  icone
     ↪  multi-operator : ${YELLOW}absente${NC}"
    [ -x "$DIR/launch/start-multi.sh" ] || echo -e "  start-multi.sh       : ${YELLOW}absent de $DIR${NC}"
    [ "$ok_cl" = "1" ] && echo -e "  OpenCL               : ${GREEN}operationnel${NC}" || echo -e "  OpenCL
     ↪  : ${YELLOW}absent${NC}"
    [ "$ok_cc" = "1" ] && echo -e "  Claude Code          : ${GREEN}operationnel${NC}" || echo -e "  Claude Code
     ↪  : ${YELLOW}absent${NC}"
    # phone firmwares (tools/osmo-firmwares-install.sh)
    _fw_etat() { [ -e "$2" ] && echo -e "  $(printf '%-20s' "$1") : ${GREEN}present${NC}" || echo -e "  $(printf
     ↪  '%-20s' "$1") : ${YELLOW}absent${NC}"; }
    _fw_etat "gnuarm"            /root/gnuarm/install/bin/arm-elf-gcc
    _fw_etat "Magnetite C139"    /opt/GSM/firmware/magnetite/c139-flash.img
    _fw_etat "firmware trx (TX)" /opt/GSM/osmocom-bb-trx/src/target/firmware/board/compal_e88/trx.highram.bin
    _fw_etat "layer1 GPRS (gnuarm)" /opt/GSM/osmocom-bb/src/target/firmware/board/compal_e88/layer1.highram.bin
    _fw_etat "Fernvale fernly"   /opt/GSM/fernly/build/firmware.bin
    _fw_etat "Fernvale NuttX"    /opt/GSM/fernvale-nuttx/nuttx/nuttx.bin
    _fw_etat "layer1 MTK"
     ↪  /opt/GSM/osmocom-bb-mtk-layer1/src/target/firmware/board/fernvale/layer1.mt6260.bin
    _fw_etat "SIMCom"            /opt/GSM/simcom/.git
    { command -v docker >/dev/null 2>&1 && docker image inspect
     ↪  "${OSMOCOM_BB_TRX_IMAGE:-ghcr.io/bbaranoff/osmocom-bb-trx:main}" >/dev/null 2>&1; } \
        && echo -e "  image osmocom-bb-trx : ${GREEN}presente${NC}" || echo -e "  image osmocom-bb-trx :
         ↪  ${YELLOW}absente${NC}"
}

[ "$STATUS_ONLY" = "1" ] && { etat; exit 0; }

# ── osmo-* units: User=osmocom -> the SESSION account ──────────────────
# The upstream .service files (osmo-bsc/bts-trx/bts-virtual/msc) set
# User=osmocom / Group=osmocom. In native post-live the osmocom account does
# not exist (the image removes it, see iso_modules/52-qemu.sh): the daemons die
# with 217/USER and Restart=always -> a blocking, SILENT crash-loop, and run.sh
# aborts ("OsmoMSC started but never ready"). Reassign these units to the
# SESSION account via a drop-in /etc/systemd/system/<unit>.d/ - do NOT touch the
# upstream .service that a package update would rewrite. With no session (chroot
# build): root, like 52-qemu.sh. Mirrored by update.sh (for already-installed
# machines) and 52-qemu.sh (live image -> root).
_osmo_unites_user_session() {
    [ "$(id -u)" -eq 0 ] || return 0
```

```
    command -v systemctl >/dev/null 2>&1 || return 0
    local u grp unit base d n=0
    u="$(_comptes_session | head -1)"
    [ -n "$u" ] && getent passwd "$u" >/dev/null 2>&1 || u=root
    grp="$(id -gn "$u" 2>/dev/null || echo "$u")"
    for unit in /lib/systemd/system/osmo-*.service /etc/systemd/system/osmo-*.service; do
        [ -f "$unit" ] || continue
        grep -qE '^(User|Group)=osmocom$' "$unit" || continue
        base="$(basename "$unit")"; d="/etc/systemd/system/$base.d"
        mkdir -p "$d"
        printf '[Service]\nUser=%s\nGroup=%s\n' "$u" "$grp" > "$d/00-session-user.conf"
        n=$((n+1))
    done
    [ "$n" -gt 0 ] || return 0
    systemctl daemon-reload 2>/dev/null || true
    # The osmocom state/log directories are owned by root/osmocom: without this
    # chown the daemon (now User=$u) cannot create /var/log/osmocom/*.log
    # -> "Unable to create file" -> .cfg parse fails -> status=1.
    for _d in /var/log/osmocom /var/lib/osmocom /run/osmocom; do
        [ -e "$_d" ] && chown -R "$u:$grp" "$_d" 2>/dev/null || true
    done
    # break any running crash-loop: restart the units that were running/looping
    for base in osmo-msc osmo-bsc osmo-bts-trx osmo-bts-virtual; do
        systemctl cat "$base" >/dev/null 2>&1 || continue
        systemctl reset-failed "$base" 2>/dev/null || true
        systemctl try-restart "$base" 2>/dev/null || true
    done
    echo -e "  ${GREEN}\u2713${NC} unites osmo-* rendues au compte de session ${BOLD}$u${NC} (sinon User=osmocom
     ↪  introuvable -> 217/USER)"
    return 0
}
_osmo_unites_user_session

# ── The choice ──────────────────────────────────────────────────────────
# Launched by the icon, this window is the ONLY interface: without it a
# double-click would go straight to a multi-hundred-MB apt-get and an Osmocom
# compilation without asking anything.
#
# Two choices are offered. The multi-operator is one supplement that ENTAILS
# docker and the operator image (the implication just below sets DO_DOCKER=1 and
# DO_IMAGE=1): docker without an image, or an 11 GB image without the topology,
# are useless on their own, so they are not offered separately. OpenCL is a
# genuinely INDEPENDENT supplement: it entails nothing and nothing entails it.
# The --docker / --image / --opencl flags remain for console troubleshooting.
if [ "$ANY_FLAG" = "0" ]; then
    if command -v zenity >/dev/null 2>&1 && [ -n "${DISPLAY:-}" ]; then
        # --print-column=2 is NOT optional: zenity defaults to column 1, which
        # for a --checklist is the CHECKBOX - we would get a list of "TRUE"
        # instead of the choices. We print column 2, hidden, which carries the
        # KEY ("multi", "opencl"): the displayed label can then change without
        # breaking the tests below.
        # --separator: the default "|" travels badly through the tests below; we
        # pick a character that appears in no label.
        _choix=$(zenity --list --checklist --width=680 --height=340 \
            --title="osmo-operator - supplements" \
            --text="Supplements qui ne sont PAS dans l ISO.\nUne connexion Internet est necessaire." \
            --ok-label="Installer" --cancel-label="Fermer" \
            --separator="~" \
            --column="" --column="cle" --column="Supplement" \
            --hide-column=2 --print-column=2 \
            TRUE  multi  "Docker container and SS7 multioperator - docker.io, l image operateur (telechargee du
 ↪  hub OU compilee sur place : le choix vient ensuite), et la topologie op1 natif + op2/op3 docker + inter-STP" \
            FALSE opencl "OpenCL (calcul GPU) - runtime ICD, clinfo, le pilote de la carte detectee (Intel /
 ↪  Mesa-AMD, pocl en repli), et les outils deka / a51_tools / dst80_reversing / tea1-cracker clones dans /root" \
            FALSE claude "Claude Code (CLI de l assistant IA) - installeur natif claude.ai/install.sh (binaire
 ↪  autonome, sans npm) ; lance ensuite avec la commande claude" \
```

```
            FALSE firmwares "Firmwares telephone - gnuarm (arm-elf-gcc 4.5.2), FreeCalypso Magnetite C139 (Wine +
↪  TI TMS470), osmocom-bb-trx (image Docker + firmware trx TX), Fernvale MT6260 (fernly, NuttX, layer1 MTK),
↪  documentation SIMCom ; long (plusieurs Go, compilations)" \
            FALSE extras "Jeux + media - Doom (freedoom + chocolate-doom), Quake III (OpenArena), Kodi, YouTube
↪  (Firefox + uBlock Origin), Wireshark (root), Linphone ; ranges dans les dossiers Jeux / Media / Telephone /
↪  Outils et lancables depuis l encart du bureau" \
            2>/dev/null) || { echo "Annule."; exit 0; }
        [ -n "$_choix" ] || { echo "Rien de selectionne."; exit 0; }
        case "$_choix" in *multi*)  DO_MULTI=1  ;; esac
        # Image origin: a second window, with RADIO BUTTONS. Two sources,
        # exclusive by construction - a radiolist returns a single value, so
        # there is no "both" to resolve afterwards as with checkboxes.
        if [ "$DO_MULTI" = "1" ]; then
            _orig=$(zenity --list --radiolist --width=680 --height=260 \
                --title="osmo-operator - image operateur" \
                --text="D ou vient l image operateur (osmocom-nitb, ~11 Go) ?" \
                --ok-label="Continuer" --cancel-label="Fermer" \
                --column="" --column="cle" --column="Origine" \
                --hide-column=2 --print-column=2 \
                TRUE  dl    "LE DEPOT (GHCR) - ${HUB_IMAGE}, publiee par la CI a chaque push" \
                FALSE hub   "DOCKER HUB - bastienbaranoff/norf_gsm, la pile publiee a la main" \
                2>/dev/null) || { echo "Annule."; exit 0; }
            # The two choices are two sources, not "pull or build". A local
            # build is still available but requested explicitly
            # (OSMO_ADDITION_BUILD_LOCAL=1), not via a radio button.
            case "$_orig" in
                hub)   DO_BUILD=0; HUB_IMAGE="bastienbaranoff/norf_gsm"; HUB_IS_RUN=1 ;;
                dl)    DO_BUILD=0 ;;
                *)     echo "Aucune origine choisie."; exit 0 ;;
            esac
        fi
        case "$_choix" in *opencl*) DO_OPENCL=1 ;; esac
        case "$_choix" in *claude*) DO_CLAUDE=1 ;; esac
        case "$_choix" in *extras*) DO_EXTRAS=1 ;; esac
        case "$_choix" in *firmwares*) DO_GNUARM=1; DO_MAGNETITE=1; DO_TRX=1; DO_CALFW=1; DO_FERNVALE=1;
         ↪  DO_SIMCOM=1 ;; esac
    else
        # Console without zenity: default to the icon's supplement.
        DO_MULTI=1
    fi
fi

# The multi-operator ENTAILS its dependencies: without engine or image the
# topology would be useless and start-multi.sh would come back here.
if [ "$DO_MULTI" = "1" ]; then DO_DOCKER=1; DO_IMAGE=1; fi
# osmocom-bb-trx: the host side is a Docker image, so it needs the engine.
# gnuarm too: the arm-elf-gcc toolchain is no longer rebuilt, it is copied from
# that same image (tools/osmo-firmwares-install.sh). The bench Calypso firmware
# (--calypso-fw) requires it.
if [ "$DO_TRX$DO_GNUARM$DO_CALFW" != "000" ]; then DO_DOCKER=1; fi

[ "$(id -u)" -eq 0 ] || { echo -e "${RED}Root requis : sudo $0${NC}"; exit 1; }

# ── OPENCL ──────────────────────────────────────────────────────────────
# Independent supplement: nothing here entails docker and docker does not entail
# this. It serves GPU compute - heavy SDR-side processing (correlation, wide
# FFTs) and anything a user wants to run alongside.
#
# An OpenCL stack has two halves, and the classic mistake is to install only
# one. The "runtime" (ocl-icd) is just a dispatcher: it reads
# /etc/OpenCL/vendors/*.icd and loads the VENDOR library named there. Without a
# vendor ICD, everything installs without error and clinfo returns
#     Number of platforms   0
# which looks like unsupported hardware when only a package is missing. So we
# install the dispatcher AND the driver of the card actually present, read from
# lspci.
```

```
#
# pocl is installed in all cases: it is a CPU implementation. It does not
# replace a GPU, but it guarantees an OpenCL program always finds a platform -
# including in a VM or on a bench with no dedicated graphics card, where the
# total absence of a platform makes the code fail with a CL_PLATFORM_NOT_FOUND_KHR.
#
# NVIDIA is the special case: its OpenCL ICD is not a standalone package, it
# ships with the proprietary driver (libnvidia-compute-xxx, pulled by
# nvidia-driver-xxx). We install the driver and the ICD comes with it. It is a
# proprietary graphics driver on a machine whose display depends on it: a REBOOT
# is usually needed for it to take effect, which is stated clearly.
#
# The version is pinned: nvidia-driver-610, not `ubuntu-drivers install` (which
# queried the repos and decided on its own, so the installed driver varied
# between machines). OSMO_NVIDIA_DRIVER selects a different one.
if [ "$DO_OPENCL" = "1" ]; then
    echo -e "  ${CYAN}→${NC} installation de la pile OpenCL ..."
    export DEBIAN_FRONTEND=noninteractive

    # lvm2: NOT part of OpenCL, but deka (installed with this supplement) mounts
    # its tables on LVM volumes - without vgchange, deka-start.sh mounts nothing.
    # Installed here, with deka.
    # swig4.1 + python3-dev: build deka's native modules (_delta.so,
    # _libvankus.so) below. build-essential is pulled elsewhere.
    # swig4.1 and NOT swig: the noble `swig` metapackage pulls swig 4.2, under
    # which deka's interfaces no longer build. 4.1 is in the distribution
    # (noble/universe, package swig4.1): name it explicitly.
    _cl_pkgs="ocl-icd-libopencl1 ocl-icd-opencl-dev opencl-headers clinfo pocl-opencl-icd lvm2 swig4.1
     ↪  python3-dev"
    _vga="$(lspci 2>/dev/null | grep -iE 'vga|3d controller|display' || true)"
    _nvidia=0
    case "$_vga" in
        *[Ii]ntel*)   _cl_pkgs="$_cl_pkgs intel-opencl-icd" ;;
    esac
    case "$_vga" in
        *AMD*|*ATI*|*[Rr]adeon*) _cl_pkgs="$_cl_pkgs mesa-opencl-icd" ;;
    esac
    case "$_vga" in
        *NVIDIA*|*nVidia*) _nvidia=1 ;;
    esac
    echo -e "      paquets : ${BOLD}${_cl_pkgs}${NC}"

    apt-get update || { echo -e "  ${RED}✗ apt-get update a echoue - pas de reseau ?${NC}"
                        echo -e "    Ce supplement a BESOIN d Internet : rien n est pre-telecharge dans l ISO."
                        exit 1; }
    # Package by package, non-fatal: a vendor ICD missing from the mirror must
    # not take the runtime down with it - the CPU stack stays usable.
    for _p in $_cl_pkgs; do
        apt-get install -y "$_p" >/dev/null 2>&1 \
            && echo -e "      ${GREEN}✓${NC} $_p" \
            || echo -e "      ${YELLOW}!${NC} $_p non installe (absent du miroir ?)"
    done

    # pyopencl: the Python OpenCL binding, installed in the VENV /root/.env - the
    # same one start-clean.sh (qosmo-grgsm) and root's profile activate, and
    # where gr-gsm already lives. Installed with the VENV pip
    # (/root/.env/bin/python3), not the system pip, otherwise the module would
    # not be visible to code running under that venv. opencl-headers +
    # ocl-icd-opencl-dev (installed above) provide the headers and libOpenCL.so
    # pyopencl needs. Non-fatal.
    if [ -x /root/.env/bin/python3 ]; then
        echo -e "  ${CYAN}→${NC} pyopencl dans le venv ${BOLD}/root/.env${NC} ..."
        if /root/.env/bin/python3 -m pip install --no-cache-dir --disable-pip-version-check pyopencl >/dev/null
         ↪  2>&1; then
            echo -e "      ${GREEN}✓${NC} pyopencl"
        else
```

```
        echo -e "      ${YELLOW}!${NC} pyopencl non installe (voir : /root/.env/bin/python3 -m pip install
        ↪  pyopencl)"
    fi
else
    echo -e "  ${YELLOW}!${NC} venv /root/.env absent - pyopencl non pose (attendu sur l ISO)"
fi

# The only check that matters: does a platform respond?
if command -v clinfo >/dev/null 2>&1; then
    _np="$(clinfo -l 2>/dev/null | grep -c Platform || true)"
    if [ "${_np:-0}" -gt 0 ]; then
        echo -e "  ${GREEN}✓${NC} OpenCL operationnel - ${BOLD}${_np}${NC} plateforme(s) :"
        clinfo -l 2>/dev/null | sed 's/^/      /'
    else
        echo -e "  ${YELLOW}!${NC} paquets poses, mais aucune plateforme OpenCL ne repond."
        echo -e "      Voir : ${BOLD}ls /etc/OpenCL/vendors/${NC} puis ${BOLD}clinfo${NC}"
    fi
fi

if [ "$_nvidia" = "1" ]; then
   echo -e "  ${CYAN}→${NC} carte ${BOLD}NVIDIA${NC} detectee : installation du pilote (ICD OpenCL inclus) ..."
    # Kernel headers first. The NVIDIA driver builds its module via DKMS
    # against the RUNNING kernel: without linux-headers-$(uname -r), dkms
    # and build-essential, the module build fails and the install errors
    # out (often silently: the package installs, but nvidia.ko does not
    # exist and OpenCL stays absent). Install them first.
    _kver="$(uname -r)"
    echo -e "      ${CYAN}→${NC} pre-requis DKMS : linux-headers-${_kver}, dkms, build-essential"
    apt-get install -y "linux-headers-${_kver}" dkms build-essential >/dev/null 2>&1 \
        || echo -e "      ${YELLOW}!${NC} headers/dkms : installation partielle (headers absents du miroir ?)"
    # linux-headers-generic as a safety net: keeps the headers aligned with
    # the meta-kernel for later kernel updates.
    apt-get install -y linux-headers-generic >/dev/null 2>&1 || true
    _nvdrv="${OSMO_NVIDIA_DRIVER:-nvidia-driver-610}"
    echo -e "      ${CYAN}→${NC} ${BOLD}apt install ${_nvdrv}${NC}"
    if apt-get install -y "$_nvdrv"; then
        echo -e "  ${GREEN}✓${NC} pilote NVIDIA installe : ${BOLD}$(dpkg-query -W -f='${Version}' "$_nvdrv"
        ↪  2>/dev/null)${NC}"
    else
        echo -e "  ${YELLOW}!${NC} ${BOLD}${_nvdrv}${NC} non installe - il est dans multiverse :"
        echo -e "      ${BOLD}sudo add-apt-repository multiverse && sudo apt install ${_nvdrv}${NC}"
    fi
    # ── Secure Boot: sign the NVIDIA modules (MOK) ──────────────────────
    # Under Secure Boot, the kernel REFUSES a module not signed by a trusted
    # key: nvidia.ko built by DKMS is signed by no one, and the driver stays
    # silent (nvidia-smi: "NVIDIA-SMI has failed..."). Generate a MOK key
    # (once), SIGN the current kernel's nvidia*.ko modules, then PROPOSE the
    # key for enrollment (mokutil) - to be CONFIRMED at the next reboot in
    # the blue MOK Manager screen. Outside Secure Boot or with no module:
    # skip. Standard Ubuntu location: /var/lib/shim-signed/mok.
    if command -v mokutil >/dev/null 2>&1 && mokutil --sb-state 2>/dev/null | grep -qi enabled; then
        echo -e "  ${CYAN}→${NC} Secure Boot actif : signature MOK des modules NVIDIA ..."
        apt-get install -y --no-install-recommends openssl mokutil zstd xz-utils >/dev/null 2>&1 || true
        _mokdir=/var/lib/shim-signed/mok
        _mokkey="$_mokdir/MOK.priv"; _mokder="$_mokdir/MOK.der"
        _mokpw="${OSMO_MOK_PASSWORD:-osmo-operator}"
        mkdir -p "$_mokdir"
        if [ ! -s "$_mokkey" ] || [ ! -s "$_mokder" ]; then
            if openssl req -new -x509 -newkey rsa:2048 -nodes -days 36500 \
                    -keyout "$_mokkey" -outform DER -out "$_mokder" \
                    -subj "/CN=osmo-operator NVIDIA MOK/" >/dev/null 2>&1; then
                chmod 600 "$_mokkey"
                echo -e "      ${GREEN}✓${NC} cle MOK generee : $_mokder"
            else
                echo -e "      ${YELLOW}!${NC} generation cle MOK echouee (openssl ?)"
            fi
```

```
        else
            echo -e "      ${CYAN}→${NC} cle MOK existante reutilisee : $_mokder"
        fi
        _kver="$(uname -r)"
        _signfile="/usr/src/linux-headers-${_kver}/scripts/sign-file"
        [ -x "$_signfile" ] || _signfile="/lib/modules/${_kver}/build/scripts/sign-file"
        if [ -x "$_signfile" ] && [ -s "$_mokkey" ] && [ -s "$_mokder" ]; then
            _signed=0
            while IFS= read -r _ko; do
                case "$_ko" in
                    *.ko)     "$_signfile" sha256 "$_mokkey" "$_mokder" "$_ko" 2>/dev/null &&
                     ↪  _signed=$((_signed+1)) ;;
                    *.ko.xz)  xz -d "$_ko"        && "$_signfile" sha256 "$_mokkey" "$_mokder" "${_ko%.xz}"
                     ↪  2>/dev/null && xz -q "${_ko%.xz}"          && _signed=$((_signed+1)) ;;
                   *.ko.zst) zstd -q -d --rm "$_ko" && "$_signfile" sha256 "$_mokkey" "$_mokder" "${_ko%.zst}"
                     ↪  2>/dev/null && zstd -q --rm "${_ko%.zst}" && _signed=$((_signed+1)) ;;
                    *.ko.gz)  gunzip "$_ko"      && "$_signfile" sha256 "$_mokkey" "$_mokder" "${_ko%.gz}"
                     ↪  2>/dev/null && gzip "${_ko%.gz}"           && _signed=$((_signed+1)) ;;
                esac
            done < <(find "/lib/modules/${_kver}" -name 'nvidia*.ko*' 2>/dev/null)
            depmod -a "${_kver}" 2>/dev/null || true
            echo -e "      ${GREEN}✓${NC} ${_signed} module(s) NVIDIA signe(s) (noyau ${_kver})"
        else
            echo -e "      ${YELLOW}!${NC} sign-file absent (linux-headers-${_kver} ?) - modules NON signes"
        fi
        if mokutil --test-key "$_mokder" >/dev/null 2>&1; then
            echo -e "      ${GREEN}✓${NC} cle MOK deja enrolee dans le firmware"
        elif printf '%s\n%s\n' "$_mokpw" "$_mokpw" | mokutil --import "$_mokder" >/dev/null 2>&1; then
            echo -e "  ${YELLOW}!${NC} MOK proposee a l enrolement. AU PROCHAIN REDEMARRAGE, l ecran bleu"
            echo -e "      ${BOLD}MOK Manager${NC} : Enroll MOK -> Continue -> mot de passe =
             ↪  ${BOLD}${_mokpw}${NC}"
        else
            echo -e "      ${YELLOW}!${NC} mokutil --import echoue (deja en attente ? droits root ?)"
        fi
    fi
    echo -e "  ${YELLOW}!${NC} un ${BOLD}redemarrage${NC} est generalement necessaire pour que le pilote"
    echo -e "      NVIDIA (et donc son OpenCL) soit actif - ${BOLD}clinfo${NC} le confirmera ensuite."
fi

# ── The tools that run on OpenCL ──────────────────────────────────────
# deka, a51_tools, dst80_reversing, tea1-cracker: four compute repositories
# that use the GPU. They GO WITH OpenCL, so they are installed here with it,
# not in the ISO: the image stays a GSM node, these are a compute supplement.
# Cloned into /root. Idempotent: if a repo is already there, update it (git
# pull) instead of re-cloning. Non-fatal - an unreachable repo (private,
# network) does not stop the rest.
echo -e "  ${CYAN}→${NC} outils OpenCL (deka, a51_tools, dst80_reversing, tea1-cracker) dans /root ..."
# Non-interactive, otherwise the window hangs. On a private or absent repo,
# git asks for credentials on the terminal and waits INDEFINITELY - launched
# by the icon, there is no one to answer. GIT_TERMINAL_PROMPT=0 makes it fail
# cleanly (no prompt), and the message below says what to do.
export GIT_TERMINAL_PROMPT=0
for _rt in "deka=https://github.com/bbaranoff/deka" \
           "a51_tools=https://github.com/bbaranoff/a51_tools" \
           "dst80_reversing=https://github.com/bbaranoff/dst80_reversing" \
           "tea1-cracker=https://github.com/bbaranoff/tea1-cracker"; do
    _name="${_rt%%=*}"; _url="${_rt#*=}"; _dst="/root/$_name"
    # Capture git's output and SHOW it on failure.
    GIT="git"
    if [ -d "$_dst/.git" ]; then
        if _err=$($GIT -C "$_dst" pull --ff-only 2>&1); then
            echo -e "      ${GREEN}✓${NC} $_name a jour ($(git -C "$_dst" log -1 --format='%h'))"
        else
            echo -e "      ${YELLOW}!${NC} $_name : git pull a echoue - $(echo "$_err" | tail -1)"
        fi
    elif _err=$($GIT clone --depth 1 "$_url" "$_dst" 2>&1); then
```

```
            echo -e "      ${GREEN}✓${NC} $_name clone ($(git -C "$_dst" log -1 --format='%h'))"
        else
            echo -e "      ${YELLOW}!${NC} $_name : clone impossible - $_url"
            echo -e "        ${YELLOW}git:${NC} $(echo "$_err" | tail -1)"
        fi
    done

    # ── deka: build the native modules (make) ──────────────────────────
    # deka imports _delta.so and _libvankus.so, produced by SWIG + gcc from
    # delta.c / libvankus.c (see its Makefile). Without this make, the workers
    # import missing modules and deka computes nothing. Requires: swig4.1 and
    # the python3 headers. Non-fatal: a failure is reported without stopping.
    if [ -f /root/deka/Makefile ]; then
        echo -e "  ${CYAN}→${NC} deka : compilation des modules natifs (make) ..."
        apt-get install -y --no-install-recommends swig4.1 python3-dev build-essential >/dev/null 2>&1 || true
        if ( cd /root/deka && make >/dev/null 2>&1 ) \
           && [ -e /root/deka/_delta.so ] && [ -e /root/deka/_libvankus.so ]; then
            echo -e "      ${GREEN}✓${NC} _delta.so + _libvankus.so construits"
        else
            echo -e "      ${YELLOW}!${NC} make deka a echoue (swig4.1 / python3-dev ?) - voir : cd /root/deka &&
             ↪  make"
        fi
    fi

    # ── deka: an application ICON (not a service) ──────────────────────
    # deka mounts tables and launches workers - this must NOT start at every
    # boot behind the user's back (the mount assumes the volume group present).
    # So we install an icon: the user clicks it when they want to start deka.
    # deka-start.sh remains the engine; the icon calls it with root rights
    # (pkexec) in a terminal, to see the output.
    if [ -d /root/deka ]; then
        echo -e "  ${CYAN}→${NC} deka : pose de l icone d application ..."
        # deka-start.sh lives in the repo: if it is there (up-to-date clone),
        # leave it alone. Write it only as a FALLBACK, for a clone that does not
        # have it yet.
        if [ ! -f /root/deka/deka-start.sh ]; then
        cat > /root/deka/deka-start.sh <<'DEKASTART'
#!/bin/bash
# deka-start.sh - mount the deka tables and start the workers.
# Installed by addition.sh (OpenCL supplement). Started by the deka icon (pkexec) or
# by hand: sudo /root/deka/deka-start.sh
#   1. vgchange -ay: activate the volume group (otherwise /dev/tables/* is missing).
#   2. the 4 mount points, then the 4 volumes (1_10..31_40).
#   3. the 3 python workers, from /root/deka (where delta.pyc lives).
set -u
DIR="$(cd "$(dirname "$(readlink -f "$0")")" && pwd)"
LOG=/var/log/deka.log
# Output on screen + copy in the log (tee), to see mounts and worker PIDs.
exec > >(tee -a "$LOG") 2>&1
echo "=== $(date -Is) deka-start ==="
[ "$(id -u)" -eq 0 ] || { echo "root requis : sudo $0"; exit 1; }
PY="$(command -v python3.7 || command -v python3)"
[ -n "$PY" ] || { echo "aucun python3 trouve"; exit 1; }
vgchange -ay || echo "ATTENTION: vgchange -ay a echoue (groupe absent ?)"
mkdir -p /mnt1 /mnt2 /mnt3
monter() {
    local dev="$1" pt="$2"
    if mountpoint -q "$pt"; then echo "deja monte: $pt"; return 0; fi
    if [ ! -e "$dev" ]; then echo "ABSENT: $dev - non monte sur $pt"; return 1; fi
    mount "$dev" "$pt" && echo "monte: $dev -> $pt" || echo "ECHEC montage: $dev -> $pt"
}
monter /dev/tables/1_10  /mnt
monter /dev/tables/⟨1⟩_⟨2⟩ /mnt⟨3⟩  ×3
    ⟨⟩ = (11,20,1) (21,30,2) (31,40,3)
cd "$DIR" || exit 1
for w in paplon.py oclvankus.py delta_client.py; do
```

```
    [ -f "$w" ] || { echo "worker absent: $w"; continue; }
    echo "lancement: $PY $DIR/$w"
    "$PY" "$DIR/$w" >>"/var/log/deka-${w%.py}.log" 2>&1 &
    echo "  pid $!"
done
echo "deka-start termine."
DEKASTART
        chmod 755 /root/deka/deka-start.sh
        fi

        # The launcher: deka-start.sh needs root (vgchange, mount). From an icon
        # we are not root: pkexec elevates, and a terminal stays open to read the
        # output (mounts, worker PIDs). Same style as osmo-addition-anim.
        cat > /usr/local/bin/osmo-deka-anim <<'DEKAGUI'
#!/bin/bash
set -u
# osmo-deka-anim - deka in a GTK bench window (embedded VTE terminal),
# as root through pkexec.
SCRIPT=/root/deka/deka-start.sh
if [ ! -x "$SCRIPT" ]; then
    command -v zenity >/dev/null 2>&1 && \
        zenity --error --text="deka-start.sh introuvable : $SCRIPT" 2>/dev/null
    exit 1
fi
exec python3 /opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --root --title "deka" -- "$SCRIPT" "$@"
DEKAGUI
        chmod 755 /usr/local/bin/osmo-deka-anim

        # Dedicated icon (data/deka.svg), set as an ABSOLUTE PATH in Icon= - a
        # theme name arriving after icon-theme.cache would show as a blank page.
        install -d /usr/share/osmo-operator/icons /usr/share/icons/hicolor/scalable/apps
        if [ -f "$DIR/data/deka.svg" ]; then
            install -m644 "$DIR/data/deka.svg" /usr/share/osmo-operator/icons/deka.svg
            install -m644 "$DIR/data/deka.svg" /usr/share/icons/hicolor/scalable/apps/deka.svg
        fi
        _deka_desktop=/usr/share/applications/deka.desktop
        cat > "$_deka_desktop" <<'DEKADSK'
[Desktop Entry]
Type=Application
Name=deka
Name[fr]=deka
Comment=Monte les tables et lance les workers deka
Comment[fr]=Monte les tables et lance les workers deka
Exec=/usr/local/bin/osmo-deka-anim
Icon=/usr/share/osmo-operator/icons/deka.svg
Terminal=false
Categories=System;Utility;
Keywords=deka;tables;workers;opencl;
Actions=Arreter;

[Desktop Action Arreter]
Name=Arreter deka
Name[fr]=Arreter deka
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --title "Arret deka" -- /root/deka/deka-start.sh --stop
DEKADSK
        chmod 644 "$_deka_desktop"
        command -v gtk-update-icon-cache >/dev/null 2>&1 && gtk-update-icon-cache -f -q /usr/share/icons/hicolor
        ↪  2>/dev/null || true
        update-desktop-database /usr/share/applications 2>/dev/null || true

        # ── No deka icon on the desktop ──────────────────────────────
        # deka stays launchable: the /usr/share/applications/deka.desktop entry
        # above puts it in the applications menu, and deka-start.sh is called
        # directly. No icon is imposed on the desktop.
        # Also remove any dropped by a previous run.
        for _h in /root /home/*; do
```

```
            for _dir in Bureau Desktop; do
                rm -f "$_h/$_dir/deka.desktop" 2>/dev/null || true
            done
        done
        echo -e "  ${GREEN}✓${NC} deka : dans le ${BOLD}menu des applications${NC} (aucune icone posee sur le
        ↪  bureau)"
    fi
fi

# ── Claude Code ──────────────────────────────────────────────────────
# Independent supplement: Claude Code, the assistant CLI. Nothing here entails
# docker or OpenCL, and nothing entails it - it can be wanted on its own.
#
# We use the native installer (claude.ai/install.sh), not npm.
# @anthropic-ai/claude-code via npm requires node >= 18, but the node in the
# Ubuntu 22.04 repos is a v12, too old - the npm install would break, or would
# require a third-party repo (NodeSource) avoided here as get.docker.com is
# avoided below. The native installer drops a standalone binary (embedded node),
# without touching the system: the safest path.
#
# It installs into ~/.local/bin (so /root/.local/bin when run via sudo). We link
# that binary into /usr/local/bin so it is on everyone's PATH and the
# `command -v claude` probe sees it.
if [ "$DO_CLAUDE" = "1" ]; then
    echo -e "  ${CYAN}→${NC} installation de Claude Code ..."
    export DEBIAN_FRONTEND=noninteractive

    # curl + certificates: the installer requires them. Non-fatal in itself, but
    # warn early if the network is missing - nothing is pre-downloaded in the ISO.
    command -v curl >/dev/null 2>&1 || apt-get install -y curl ca-certificates >/dev/null 2>&1 || true
    if ! command -v curl >/dev/null 2>&1; then
        echo -e "  ${RED}✗ curl absent et non installable - pas de reseau ?${NC}"
        echo -e "    Ce supplement a BESOIN d Internet : rien n est pre-telecharge dans l ISO."
        exit 1
    fi

    # Explicit HOME: launched by pkexec/sudo, $HOME may be missing or point
    # elsewhere; the installer drops into $HOME/.local/bin, we want to know where.
    _cc_home="${HOME:-/root}"
    if _out=$(HOME="$_cc_home" bash -c 'curl -fsSL https://claude.ai/install.sh | bash' 2>&1); then
        echo -e "      ${GREEN}✓${NC} installeur natif termine"
    else
        echo -e "      ${YELLOW}!${NC} installeur natif en echec - $(echo "$_out" | tail -1)"
    fi

    # Find the installed binary and link it onto the global PATH. Look at the
    # installer's known locations (~/.local/bin, then /root as fallback).
    _cc_bin=""
    for _c in "$_cc_home/.local/bin/claude" /root/.local/bin/claude "$_cc_home/.claude/bin/claude"; do
        [ -x "$_c" ] && { _cc_bin="$_c"; break; }
    done
    if [ -n "$_cc_bin" ] && [ ! -e /usr/local/bin/claude ]; then
        ln -sf "$_cc_bin" /usr/local/bin/claude
        echo -e "      ${GREEN}✓${NC} lien /usr/local/bin/claude -> $_cc_bin"
    fi

    # The only check that matters: does claude respond?
    if command -v claude >/dev/null 2>&1 && _ver=$(claude --version 2>/dev/null); then
        echo -e "  ${GREEN}✓${NC} Claude Code operationnel - ${BOLD}${_ver}${NC}"
        echo -e "      lancez-le avec la commande ${BOLD}claude${NC}"
    else
        echo -e "  ${YELLOW}!${NC} Claude Code pose, mais ne repond pas encore."
        echo -e "      Ouvrez un nouveau terminal (PATH), ou lancez ${BOLD}$_cc_home/.local/bin/claude${NC}"
    fi

    # ── Claude icon (data/claude.svg) ──────────────────────────────
```

```
    # Launcher osmo-claude-anim: opens a terminal that runs claude; if it is not
    # installed, it first runs this same supplement (--claude), then launches it.
    install -d /usr/share/osmo-operator/icons /usr/share/icons/hicolor/scalable/apps
    if [ -f "$DIR/data/claude.svg" ]; then
        install -m644 "$DIR/data/claude.svg" /usr/share/osmo-operator/icons/claude.svg
        install -m644 "$DIR/data/claude.svg" /usr/share/icons/hicolor/scalable/apps/claude.svg
    fi
    cat > /usr/local/bin/osmo-claude-anim <<'CLA'
#!/bin/bash
# osmo-claude-anim - Claude Code in a GTK bench window (VTE terminal: claude is
# interactive). Not in the ISO: the first click installs it (addition.sh --claude,
# as root), then opens it in the same window.
set -u
exec python3 /opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Claude" -- bash -lc '
if ! command -v claude >/dev/null 2>&1; then
    ADD=/opt/GSM/osmo-operator/addition.sh
    if [ -x "$ADD" ]; then
        if [ "$(id -u)" -ne 0 ] && command -v pkexec >/dev/null 2>&1; then
            pkexec env DISPLAY="${DISPLAY:-}" XAUTHORITY="${XAUTHORITY:-}" "$ADD" --claude
        else
            "$ADD" --claude
        fi
    fi
fi
if command -v claude >/dev/null 2>&1; then exec claude; fi
echo "Claude non installe - lancez le supplement (addition.sh --claude)."
exit 1'


CLA
    chmod 755 /usr/local/bin/osmo-claude-anim
    if [ -f "$DIR/data/desktop/claude.desktop" ]; then
        install -m644 "$DIR/data/desktop/claude.desktop" /usr/share/applications/claude.desktop
        sed -i "s|^Icon=.*|Icon=/usr/share/osmo-operator/icons/claude.svg|" /usr/share/applications/claude.desktop
        command -v gtk-update-icon-cache >/dev/null 2>&1 && gtk-update-icon-cache -f -q /usr/share/icons/hicolor
         ↪  2>/dev/null || true
        update-desktop-database /usr/share/applications 2>/dev/null || true
        mapfile -t _homes < <(_bureau_homes)
        _cposee=0
        for _h in "${_homes[@]}"; do
            for _dir in Bureau Desktop; do
                [ -d "$_h/$_dir" ] || continue
                cp -f /usr/share/applications/claude.desktop "$_h/$_dir/claude.desktop" 2>/dev/null || continue
                _trust_desktop "$_h/$_dir/claude.desktop" "$_h" "110,0"
                touch "$_h/$_dir" 2>/dev/null || true
                _cposee=1
            done
        done
        [ "$_cposee" = "1" ] \
            && echo -e "  ${GREEN}✓${NC} icone ${BOLD}Claude${NC} posee sur le bureau (en haut a gauche)" \
            || echo -e "  ${GREEN}✓${NC} icone ${BOLD}Claude${NC} dans le menu des applications"
    fi
fi

# ── Docker ──────────────────────────────────────────────────────────
# docker.io from the Ubuntu repos, NOT the get.docker.com script: that one adds
# a third-party repo and replaces containerd, which on a frozen image costs at
# the first apt-get upgrade. The repo version is enough: we only run our own
# containers.
if [ "$DO_DOCKER" = "1" ]; then
    if command -v docker >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} docker deja installe ($(docker --version 2>/dev/null | head -1))"
    else
        echo -e "  ${CYAN}→${NC} installation de docker.io ..."
        export DEBIAN_FRONTEND=noninteractive
        apt-get update || { echo -e "  ${RED}✗ apt-get update a echoue - pas de reseau ?${NC}"
```

```
                        echo -e "    Ce supplement a BESOIN d Internet : rien n est pre-telecharge dans l ISO."
                         exit 1; }
        apt-get install -y docker.io || { echo -e "  ${RED}✗ installation de docker.io echouee${NC}"; exit 1; }
        echo -e "  ${GREEN}✓${NC} docker.io installe"
    fi
    # docker compose build goes through Bake and warns if buildx is missing. Non-fatal.
    if ! docker buildx version >/dev/null 2>&1; then
        apt-get install -y docker-buildx >/dev/null 2>&1 \
            || apt-get install -y docker-buildx-plugin >/dev/null 2>&1 || true
    fi
    systemctl enable --now docker 2>/dev/null || true
    # The socket takes a moment to respond after a first start: without this
    # wait, the next `docker info` fails and the install looks broken.
    for _i in $(seq 1 30); do docker info >/dev/null 2>&1 && break; sleep 1; done
    docker info >/dev/null 2>&1 \
        && echo -e "  ${GREEN}✓${NC} demon docker actif" \
        || { echo -e "  ${RED}✗ le demon docker ne repond pas${NC} - voir : systemctl status docker"; exit 1; }
    _docker_groupe_session
fi

# ── Operator image ──────────────────────────────────────────────────
# Two paths to obtain osmocom-nitb:
#   1. docker pull ${HUB_IMAGE} && docker tag ... osmocom-nitb  (default)
#      The already-compiled Osmocom stack, published on the hub: the download
#      replaces the compilation. The tag gives the result the name that
#      start-docker.sh (Dockerfile.run: FROM osmocom-nitb) and start-nitb.sh expect.
#   2. build.sh --no-cache  (--build / --multi-build / "compile" case)
#      --no-cache: rebuild the image CLEAN. The docker cache could keep layers
#      from a half-compiled Osmocom stack and replay them identically, so a
#      broken build stayed broken every time. Force a rebuild.
# --build / --multi-build no longer compile locally: they pull from GHCR, the
# same image the CI publishes from the same Dockerfile. Both flags still mean
# "take the image of THIS repo", as opposed to a Hub image.
# A real local build stays available - it is the only path once the Dockerfile
# has been MODIFIED and no CI has published yet:
#     sudo OSMO_ADDITION_BUILD_LOCAL=1 ./addition.sh --multi-build
_build_image() {
    if [ "${OSMO_ADDITION_BUILD_LOCAL:-0}" != "1" ]; then
        echo -e "  ${CYAN}→${NC} image du depot : ${BOLD}docker pull ${HUB_IMAGE}${NC}"
        echo -e "      (build local : ${BOLD}OSMO_ADDITION_BUILD_LOCAL=1${NC}, plusieurs dizaines de minutes)"
        _pull_image
        return $?
    fi
    [ -x "$DIR/build.sh" ] || { echo -e "  ${RED}✗ build.sh introuvable dans $DIR${NC}"; return 1; }
    echo -e "  ${CYAN}→${NC} construction de l image : ${BOLD}${DIR}/build.sh --no-cache${NC}"
    echo -e "      compilation Osmocom - comptez plusieurs dizaines de minutes."
    "$DIR/build.sh" --no-cache || { echo -e "  ${RED}✗ ${DIR}/build.sh a echoue${NC}"; return 1; }
    echo -e "  ${GREEN}✓${NC} image operateur construite (${BASE_IMAGE})"
}
_pull_image() {
    echo -e "  ${CYAN}→${NC} telechargement de l image : ${BOLD}docker pull ${HUB_IMAGE}${NC}"
    echo -e "      ~11 Go - selon le debit, quelques minutes a une heure."
    # GHCR serves PUBLIC images without authentication. If the repo package is
    # private, a token is needed: docker login ghcr.io -u <account> (classic
    # token with read:packages). The message says so rather than leaving the
    # operator facing a "denied" without explanation.
    docker pull "$HUB_IMAGE" || {
        echo -e "  ${RED}✗ docker pull ${HUB_IMAGE} a echoue${NC} - reseau, image absente, ou paquet prive."
        case "$HUB_IMAGE" in ghcr.io/*)
           echo -e "    Paquet prive ? ${BOLD}docker login ghcr.io -u <compte>${NC} (jeton avec read:packages)" ;;
        esac
        echo -e "    Autre image : ${BOLD}HUB_IMAGE=<ref> $0 --multi${NC}"
        echo -e "    Compiler sur place : ${BOLD}OSMO_ADDITION_BUILD_LOCAL=1 $0 --multi-build${NC}"
        return 1; }
    # norf_gsm is published from osmocom-run (Dockerfile.run already applied):
    # tag it osmocom-run directly, with no derivation.
```

```
    local _tag="$BASE_IMAGE"; [ "${HUB_IS_RUN:-0}" = 1 ] && _tag="$MULTI_IMAGE"
    docker tag "$HUB_IMAGE" "$_tag" || { echo -e "  ${RED}✗ docker tag ${HUB_IMAGE} ${_tag} a echoue${NC}"; return
     ↪  1; }
    echo -e "  ${GREEN}✓${NC} image operateur tiree et taguee ${_tag}"
}
# The image the multi-operator runs is osmocom-run, NOT osmocom-nitb. Neither
# the pull nor build.sh produces it: Dockerfile.run (FROM osmocom-nitb) derives
# it in a few seconds. start-multi.sh probes osmocom-run and comes back here if
# it is missing. Same command as start-docker.sh (build_run_image, quick mode).
_derive_run_image() {
    echo -e "  ${CYAN}→${NC} derivation de l image d execution : ${BOLD}docker build -f Dockerfile.run -t
     ↪  ${MULTI_IMAGE}${NC}"
    ( cd "$DIR" && docker build --build-arg QEMU_CACHE_BUST=$(date +%s) \
                      -f Dockerfile.run -t "$MULTI_IMAGE" . ) \
        || { echo -e "  ${RED}✗ derivation de ${MULTI_IMAGE} echouee${NC}"; return 1; }
    echo -e "  ${GREEN}✓${NC} image d execution prete (${MULTI_IMAGE})"
}
# norf_gsm (the "hub" choice, or HUB_IMAGE set by the installer) IS osmocom-run.
case "$HUB_IMAGE" in *norf_gsm*) HUB_IS_RUN=1 ;; esac
if [ "$DO_IMAGE" = "1" ]; then
    if [ "$DO_BUILD" = "1" ]; then
        _build_image || exit 1
        _derive_run_image || exit 1
    elif docker image inspect "$MULTI_IMAGE" >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} image operateur deja presente (${MULTI_IMAGE})"
    elif [ "${HUB_IS_RUN:-0}" != 1 ] && docker image inspect "$BASE_IMAGE" >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} image de base deja presente (${BASE_IMAGE})"
        _derive_run_image || exit 1
    else
        _pull_image || exit 1
        [ "${HUB_IS_RUN:-0}" = 1 ] || _derive_run_image || exit 1
    fi
fi

# ── Multi-operator topology ──────────────────────────────────────────
# Written HERE, once, and read back by start-multi.sh. It is not recomputed on
# both sides: two twin formulas always diverge in the end.
#
# The SS7 plan, as start-interstp.sh defines it:
#     PC   = 1.<node><op>.<role>       role 1=MSC 2=STP 3=BSC
#     RCTX = node*1000 + op*100 + 50
# On a single machine node is 1, so PC = 1.<op>.<role>.
#
#   op 1  NATIVE  - PC 1.1.2, that of start-native.sh, left as is
#   op 2  DOCKER  - PC 1.2.2, backbone 172.20.0.12
#   op 3  DOCKER  - PC 1.3.2, backbone 172.20.0.13
#
# The containers start at index 2, and this is controllable. OP_ID_BASE=2
# (start-docker.sh) starts the containers at index 2 and leaves index 1 to the
# native operator; name, backbone, point code, RCTX, private segment and
# published ports all follow. Otherwise osmo-operator-1 would get point code
# 1.1.2, the native one's - two devices at the same SS7 address, i.e. wrong
# routing (checks/wan_ss7_check.sh).
# The native operator reaches the hub via the bridge gateway: the host reaches
# 172.20.0.10 directly, with no NAT or route to add.
if [ "$DO_MULTI" = "1" ]; then
    install -d "$(dirname "$MULTI_CONF")"
    cat > "$MULTI_CONF" <<CONF
# osmo-multi.conf - multi-operator topology, written by addition.sh.
# Read by start-multi.sh. Do not edit by hand without reading both.
MULTI_NODE=1
MULTI_HUB_NAME=osmo-inter-stp
MULTI_HUB_IP=172.20.0.10
MULTI_HUB_PC=0.0.0
MULTI_M3UA_PORT=2908
MULTI_NET_NAME=osmo-ss7
```

```
MULTI_NET_SUBNET=172.20.0.0/24
MULTI_NET_GW=172.20.0.1
MULTI_IMAGE=osmocom-run
# operateurs : "index:mode:backbone_ip:point_code:rctx"
MULTI_OPS="1:native::1.1.2:150 2:docker:172.20.0.12:1.2.2:250 3:docker:172.20.0.13:1.3.2:350"
CONF
    echo -e "  ${GREEN}✓${NC} topologie SS7 ecrite : ${CYAN}${MULTI_CONF}${NC}"
    echo -e "      op1 ${BOLD}natif${NC} (1.1.2) + op2/op3 ${BOLD}docker${NC} (1.2.2 / 1.3.2) → hub 172.20.0.10"

    # ── Multi-operator icon: installed here, at the end ──────────────────
    # The blue antenna starts the multi-operator bench, which needs docker, the
    # image and the topology above, so the icon is only installed once the SS7
    # supplement is in place (build-iso and install_modules/80-bureau.sh do not).
    _ico_dir=/usr/share/osmo-operator/icons
    _svg="$DIR/data/osmo-multi.svg"
    _dsk="$DIR/data/desktop/osmo-multi.desktop"

    # ── Unit first, then the icon ────────────────────────────────────────
    # The icon runs `systemctl start osmo-multi.service`, so the unit must exist.
    # On an ISO it is installed at build time (iso_modules/82-services.sh); on a
    # hand-deployed machine it is installed here, together with the icon.
    # The unit sets OSMO_NO_TERM=1: no terminal window, everything goes to the
    # journal (`journalctl -u osmo-multi`).
    for _u in osmo-banc osmo-multi; do
        [ -f "$DIR/services/$_u.service" ] || continue
        install -m644 "$DIR/services/$_u.service" "/etc/systemd/system/$_u.service"
    done
    systemctl daemon-reload 2>/dev/null || true
    # ── Installed, not enabled ───────────────────────────────────────────
    # A bench is started explicitly:
    #   systemctl start osmo-banc | osmo-multi     one session (or the icon)
    #   systemctl enable --now osmo-multi          at every boot
    # OSMO_MULTI_ENABLE=1 ./addition.sh enables it automatically.
    if [ -f /etc/systemd/system/osmo-multi.service ]; then
        if [ "${OSMO_MULTI_ENABLE:-0}" = "1" ]; then
            systemctl enable osmo-multi.service >/dev/null 2>&1 \
                && echo -e "  ${GREEN}✓${NC} ${BOLD}osmo-multi.service${NC} installe et active au demarrage
                ↪  (OSMO_MULTI_ENABLE=1)" \
                || echo -e "  ${YELLOW}!${NC} osmo-multi.service installe, activation KO (systemctl enable
                ↪  osmo-multi)"
        else
            systemctl disable osmo-multi.service >/dev/null 2>&1 || true
           echo -e "  ${GREEN}✓${NC} ${BOLD}osmo-multi.service${NC} installe, ${BOLD}non active${NC} au demarrage"
            echo -e "      ${CYAN}systemctl start osmo-multi${NC} pour une session · ${CYAN}systemctl enable --now
             ↪  osmo-multi${NC} a chaque boot"
        fi
    fi

    if [ -f "$_dsk" ]; then
        install -d "$_ico_dir" /usr/share/icons/hicolor/scalable/apps /usr/share/applications
        # SVG as an ABSOLUTE PATH in Icon= (see build-iso.sh): a theme name added
        # after icon-theme.cache was built shows as a blank page.
        if [ -f "$_svg" ]; then
            install -m644 "$_svg" "$_ico_dir/osmo-multi.svg"
            install -m644 "$_svg" /usr/share/icons/hicolor/scalable/apps/osmo-multi.svg
        fi
        install -m644 "$_dsk" /usr/share/applications/osmo-multi.desktop
        # Icon= follows the real tree. Exec= calls `systemctl start osmo-multi.service`;
        # the unit carries the path. The sed on Exec= also fixes older .desktop files
        # that still name start-multi.sh.
        sed -i -e "s|^Exec=/opt/GSM/osmo-operator/|Exec=$DIR/|" \
               -e "s|^Icon=.*|Icon=$_ico_dir/osmo-multi.svg|" \
               /usr/share/applications/osmo-multi.desktop

        # On the desktops of root and of the sudo user, in Bureau/ and Desktop/.
        # metadata::trusted so DING shows it without a badge; osmo-trust-desktop
```

```
        # (at login) repositions and trusts it too.
        mapfile -t _homes < <(_bureau_homes)
        _posee=0
        for _h in "${_homes[@]}"; do
            for _dir in Bureau Desktop; do
                [ -d "$_h/$_dir" ] || continue
                cp -f /usr/share/applications/osmo-multi.desktop "$_h/$_dir/osmo-multi.desktop" 2>/dev/null ||
                 ↪  continue
                _trust_desktop "$_h/$_dir/osmo-multi.desktop" "$_h"
                touch "$_h/$_dir" 2>/dev/null || true
                _posee=1
            done
        done
        command -v gtk-update-icon-cache >/dev/null 2>&1 && gtk-update-icon-cache -f -q /usr/share/icons/hicolor
         ↪  2>/dev/null || true
        command -v update-desktop-database >/dev/null 2>&1 && update-desktop-database /usr/share/applications
         ↪  2>/dev/null || true
        [ "$_posee" = "1" ] \
            && echo -e "  ${GREEN}✓${NC} icone ${BOLD}multi-operator${NC} (antenne) posee sur le bureau" \
            || echo -e "  ${GREEN}✓${NC} icone ${BOLD}multi-operator${NC} dans le menu des applications"
    fi

    # ── Start the multi-operator bench now: containers exist on exit ──────
    # One session through launch.sh --multi, the same path as the icon and the
    # desktop launcher (tools/osmo-launcher.py): it checks the unit,
    # start-multi.sh and the topology, refuses a bench already running, starts
    # the unit (start, not enable) and reports the result. The unit journal
    # scrolls here during start-up. OSMO_MULTI_START=0 installs without starting.
    if [ "${OSMO_MULTI_START:-1}" = "1" ] && [ -f /etc/systemd/system/osmo-multi.service ]; then
        echo -e "  ${CYAN}→${NC} demarrage du banc multi-operateur (${BOLD}launch.sh --multi${NC}, comme l icone)
         ↪  - comptez plusieurs minutes ..."
        journalctl -u osmo-multi -f -n 0 --no-pager -o cat 2>/dev/null &
        _jpid=$!
        # OSMO_LAUNCH_NO_SERVICE is irrelevant here (--multi is handled at the top
        # of launch.sh); the current terminal keeps its output.
        if "$DIR/launch.sh" --multi; then
            sleep 1; kill "$_jpid" 2>/dev/null; wait "$_jpid" 2>/dev/null
            echo -e "  ${GREEN}✓${NC} banc multi-operateur en service - conteneurs :"
            docker ps --format '      {{.Names}}  {{.Status}}' 2>/dev/null | grep -E
             ↪  'osmo-operator-|osmo-inter-stp' \
                || echo -e "      ${YELLOW}!${NC} aucun conteneur osmo-operator-N visible (docker ps)"
        else
            sleep 1; kill "$_jpid" 2>/dev/null; wait "$_jpid" 2>/dev/null
            echo -e "  ${RED}✗${NC} launch.sh --multi n a pas abouti - ${BOLD}journalctl -u osmo-multi -n 80${NC}"
            echo -e "      a la main : ${BOLD}$DIR/launch.sh --multi${NC} (ou l icone multi-operator)"
        fi
    fi
fi

# ── Bench SIM (independent supplement) ──────────────────────────────────
# PC/SC reader (pcscd, CCID: HID Omnikey), softSIM (Ruby SAP server) and
# pySim (read / program a sysmoISIM). Same code as the ISO (80-chroot.sh):
# tools/osmo-sim-install.sh, sourced here. Then: tools/osmo-sim.sh.
if [ "$DO_SIM" = "1" ]; then
    echo -e "${BOLD}== SIM : lecteur PC/SC, softSIM, pySim ==${NC}"
    if [ -f "$DIR/tools/osmo-sim-install.sh" ]; then
        # shellcheck source=tools/osmo-sim-install.sh
        . "$DIR/tools/osmo-sim-install.sh"
        REPO="$DIR" osmo_sim_install
    else
        echo -e "  ${YELLOW}!${NC} tools/osmo-sim-install.sh introuvable"
    fi
fi

# ── Phone firmwares (independent supplement) ────────────────────────────
# gnuarm, Magnetite C139, osmocom-bb-trx, Fernvale, SIMCom: tools/osmo-firmwares-install.sh.
```

```
# osmocom-bb-trx pulls a Docker image: without the engine, --trx only installs the firmware.
if [ "$DO_GNUARM$DO_MAGNETITE$DO_TRX$DO_CALFW$DO_FERNVALE$DO_SIMCOM" != "000000" ]; then
    if [ -f "$DIR/tools/osmo-firmwares-install.sh" ]; then
        # shellcheck source=tools/osmo-firmwares-install.sh
        . "$DIR/tools/osmo-firmwares-install.sh"
        export REPO="$DIR"
        [ "$DO_GNUARM" = "1" ]    && { osmo_gnuarm_install    || true; }
        [ "$DO_MAGNETITE" = "1" ] && { osmo_magnetite_install || true; }
        [ "$DO_TRX" = "1" ]       && { osmo_trx_install       || true; }
        [ "$DO_CALFW" = "1" ]     && { osmo_calypso_fw_install || true; }
        [ "$DO_FERNVALE" = "1" ]  && { osmo_fernvale_install  || true; }
        [ "$DO_SIMCOM" = "1" ]    && { osmo_simcom_install    || true; }
        echo -e "  ${CYAN}→${NC} journaux : ${BOLD}${OSMO_FW_LOG:-/var/log/osmo-firmwares}/${NC}"
    else
        echo -e "  ${YELLOW}!${NC} tools/osmo-firmwares-install.sh introuvable"
    fi
fi

# ── Games + media (independent supplement) ──────────────────────────
# Doom / Quake III / Kodi / YouTube+uBlock / Wireshark(root) / Linphone, in the
# Jeux / Media / Telephone / Outils folders, launchable from the desktop panel.
# Shared with the ISO: tools/osmo-extras-install.sh, sourced here.
if [ "$DO_EXTRAS" = "1" ]; then
    echo -e "${BOLD}== Jeux + media ==${NC}"
    if [ -f "$DIR/tools/osmo-extras-install.sh" ]; then
        # shellcheck source=tools/osmo-extras-install.sh
        . "$DIR/tools/osmo-extras-install.sh"
        osmo_extras_install
    else
        echo -e "  ${YELLOW}!${NC} tools/osmo-extras-install.sh introuvable"
    fi
fi

# ── 4G (srsRAN ZeroMQ + Open5GS) and smartphone UI (postmarketOS) ────────
# Same code for addition.sh, update.sh and the ISO build (iso_modules/88-lte-pmos.sh):
# tools/osmo-lte-install.sh (packages, cached .deb, repo configs, osmo-lte /
# osmo-epc launchers; --all builds into /opt/LTE when binaries and .deb are
# missing) and tools/osmo-pmos-install.sh (/opt/user_interface/pmos: scripts,
# patched pmbootstrap, template; kernel/: the PPP kernel .deb). Not fatal.
# Skipped when only --claude is requested.
if [ "$DO_CLAUDE" = "1" ] && [ "$DO_DOCKER$DO_IMAGE$DO_MULTI$DO_OPENCL$DO_EXTRAS$DO_SIM" = "000000" ] \
   && [ "$DO_GNUARM$DO_MAGNETITE$DO_TRX$DO_CALFW$DO_FERNVALE$DO_SIMCOM" = "000000" ]; then
    :
else
echo -e "${BOLD}== La 4G et le telephone ==${NC}"
if [ -f "$DIR/tools/osmo-lte-install.sh" ]; then
    # shellcheck source=tools/osmo-lte-install.sh
    . "$DIR/tools/osmo-lte-install.sh"
    osmo_lte_install --all || true
else
    echo -e "  ${YELLOW}!${NC} tools/osmo-lte-install.sh introuvable"
fi
if [ -f "$DIR/tools/osmo-pmos-install.sh" ]; then
    # shellcheck source=tools/osmo-pmos-install.sh
    . "$DIR/tools/osmo-pmos-install.sh"
    osmo_pmos_install || true
else
    echo -e "  ${YELLOW}!${NC} tools/osmo-pmos-install.sh introuvable"
fi
fi

echo
etat
echo
[ "$DO_MULTI" = "1" ] && \
    echo -e "  ${CYAN}→${NC} lancer : ${BOLD}sudo $DIR/launch/start-multi.sh${NC}  (ou l antenne bleue du bureau)"
```

```
[ "$DO_OPENCL" = "1" ] && \
    echo -e "  ${CYAN}→${NC} verifier OpenCL : ${BOLD}clinfo${NC}"
[ "$DO_EXTRAS" = "1" ] && \
    echo -e "  ${CYAN}→${NC} jeux + media : dossiers ${BOLD}Jeux / Media / Telephone / Outils${NC} et encart du
     ↪  bureau"
exit 0
```

2.6 /opt/GSM/osmo-operator/build-iso.sh

4721 octets, 75 lignes → 75 lignes

```
#!/bin/bash
# build-iso.sh - build a bootable amd64 ISO or an arm64 SD image, through
# build.sh and start-docker.sh (no direct docker build in this script).
#
#  Usage: sudo ./build-iso.sh [options]
#    (no option)          the four amd64 images: interstp, operator, lite, desktop
#    --role=operator|interstp   --node=N   --lite   --desktop   --all
#    --arm                arm64 SD images via github.com/armbian/build
#                         (armbian/build-armbian.sh): Raspberry Pi 4 and the
#                         Orange Pi 5 family (5, 5B, 5 Plus, 5 Pro, 5 Max, 5 Ultra);
#                         --board=rpi4|opi5|all|<board>[,...] (default all),
#                         --desktop = GNOME + the bench desktop; --all and --lite
#                         are refused. See iso_modules/00-options.sh.
#    --banc --multi       enable osmo-banc.service / osmo-multi.service at boot
#                         of the image. By default both units are INSTALLED but
#                         NOT enabled: the machine boots to its desktop and the
#                         operator starts the bench (icon, launch.sh,
#                         systemctl start osmo-banc). Env equivalents:
#                         OSMO_ISO_BANC=1 / OSMO_ISO_MULTI=1.
#    --version=24.04|22.04   --kb=fr   --output=file   --no-cache
#    (default, amd64)     THE DOCKER IMAGE IS PULLED, NOT BUILT, and tagged
#                         osmocom-nitb. Tried in order: bastienbaranoff/norf_gsm:latest
#                         (Docker Hub), then ghcr.io/<repo>/osmocom-nitb:base-<fingerprint>,
#                         then the GHCR :latest. build.sh runs only if none
#                         answers. The fingerprint is the one of the workflows:
#                         when it matches, the published image was built from
#                         exactly this working tree; the two other references
#                         do not guarantee that, and the script says so before
#                         using them. On arm64 nothing is published: building
#                         remains the default.
#    --skip-build[=image:tag]   make the pull mandatory (no fallback to
#                         build.sh) and, with =image, force that reference
#                         instead of deriving it. An image already present
#                         locally is not pulled. Env: OSMO_ISO_SKIP_BUILD=0
#                         for the opposite - never pull.
#    --build-docker       the opposite, and it wins: the image is built by
#                         build.sh even if a --skip-build is set elsewhere.
#                         Env: OSMO_ISO_BUILD_DOCKER=1.
#    --without-debs       no .deb travels in the image: neither the bench
#                         packages (step 7c) nor those of the docker build.
#                         They are still used to install the stack in the
#                         rootfs but are not kept there. Env: OSMO_ISO_WITHOUT_DEBS=1.
#    --wan --wan-nodes=... --wan-id=N --wan-ops=N --hub-ip=IP
set -euo pipefail

# Colours are defined first: under `set -u`, a coloured message emitted before
# their definition would fail with "unbound variable".
RED='\033[0;31m'; GREEN='\033[0;32m'; CYAN='\033[0;36m'
YELLOW='\033[1;33m'; NC='\033[0m'; BOLD='\033[1m'

VERSION="${OSMO_ISO_VERSION:-main}"
OUTPUT="osmo-operator-${VERSION}.iso"
# Work directory ON DISK (not /tmp, often a RAM tmpfs too small for the
```

```
# rootfs). Override with OSMO_ISO_WORK.
WORK="${OSMO_ISO_WORK:-/var/tmp}/iso-build-$$"
ROOTFS="$WORK/rootfs"
ISOROOT="$WORK/isoroot"
LABEL="OSMO_EGPRS_V2"
DIR="$(cd "$(dirname "$0")" && pwd)"
# /usr/local/sbin in PATH: apt-fast-install puts apt-fast there, on the host as
# in the rootfs; the chroot below inherits this PATH (env ... bash -c).
export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"


# ── Build steps live in iso_modules/ ──────────────────────────────────
# Each step is iso_modules/NN-<slug>.sh, sourced here in numeric order, in THIS
# shell: variables, functions and the trap stay global. A module that has
# nothing to do (another variant or architecture) returns at its top.
# Invariant: `. file` returns the status of the file's last command and set -e
# is active, so every module must end with `true`.
MODDIR="$DIR/iso_modules"
[ -d "$MODDIR" ] || { echo -e "${RED}iso_modules/ introuvable a cote de $0${NC}" >&2; exit 2; }
shopt -s nullglob; _ISO_MODS=("$MODDIR"/[0-9][0-9]-*.sh); shopt -u nullglob
[ "${#_ISO_MODS[@]}" -gt 0 ] || { echo -e "${RED}iso_modules/ est vide${NC}" >&2; exit 2; }
for _m in "${_ISO_MODS[@]}"; do . "$_m"; done
```

2.7 /opt/GSM/osmo-operator/build.sh

21145 octets, 386 lignes → 386 lignes

```
#!/bin/bash
# ============================================================================
#  build.sh - prepare the host and build the osmocom-nitb image
# ============================================================================
#
#  Usage: sudo ./build.sh [--no-cache] [--lite] [--stp] [--build-run] [--arch=arm64|--arm]
#         (-h describes every option)
#
#  - apt-fast everywhere: packaging/apt-fast-install.sh installs apt-fast
#    (aria2) and the common apt settings on the host, in the images and in the
#    ISO rootfs.
#  - docker compose v2 (+ buildx) is a dependency: it builds the images
#    (compose.yaml: services nitb, run, lite, stp). `docker build` is the
#    fallback when compose is missing.
#  - .deb cache on the host: every tree compiled in the image is packaged as a
#    .deb (packaging/osmo-deb.sh) and stored in
#        /var/cache/osmo-debs          (OSMO_DEB_CACHE to relocate)
#    Before the build they are copied into .deb-cache/ (the docker context) and
#    the image installs them with dpkg instead of recompiling; after the build
#    the packages produced by the image are copied back to the host cache.
#    The first build compiles everything, later ones only what is missing.
#    --no-cache sets OSMO_DEB_REFRESH=1: everything is recompiled and the
#    cache rewritten.
#
#  Env: OSMO_DEB_CACHE, OSMO_UBUNTU_MIRROR, OSMO_HOST_MIRROR (0|1|force),
#       OSMO_HOST_DOCKER (0 = never ask apt for docker), BUILDKIT_PROGRESS.
# ============================================================================
set -euo pipefail

# Build context: the directory of THIS script, not the caller's cwd (the
# .desktop launcher goes through pkexec, which starts in /root).
cd "$(dirname "$(readlink -f "$0")")"
DIR="$(pwd)"
# /usr/local/sbin in PATH: apt-fast-install puts apt-fast there (as the
# Dockerfile does); a root shell without it in PATH would not find apt-fast.
export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"
```

```
RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[0;33m'; CYAN='\033[0;36m'; NC='\033[0m'

# 0. Argument parsing
NO_CACHE=""
LITE=0
STP=0
BUILD_RUN=0
OSMO_ARCH="$(dpkg --print-architecture 2>/dev/null || uname -m)"
for arg in "$@"; do
    case "$arg" in
        --no-cache) NO_CACHE="--no-cache" ;;
        --lite)     LITE=1 ;;
        --stp)      STP=1 ;;
        --build-run) BUILD_RUN=1 ;;
        --arch=*)   OSMO_ARCH="${arg#*=}" ;;
        --arm)      OSMO_ARCH=arm64 ;;
        -h|--help)
            echo "Usage: sudo $0 [--no-cache] [--lite] [--stp] [--build-run] [--arch=arm64|--arm]"
            echo "  --arch=arm64 Construit les images pour arm64 (Raspberry Pi 4) sur cet hote,"
            echo "               par docker buildx --platform linux/arm64 sous qemu-user-static."
            echo "               Images taguees :arm64 (osmocom-nitb:arm64...). LENT au premier"
            echo "               build (emulation) ; le cache .deb rend les suivants courts."
            echo "               Pose aussi sur l hote tout ce que la chaine arm64 demande :"
            echo "               qemu-user-static, binfmt-support, docker-buildx, debootstrap,"
            echo "               mtools, dosfstools, e2fsprogs, kpartx... (build-iso.sh --arm,"
            echo "               armbian/build-armbian.sh)."
            echo "  --no-cache   Reconstruction complete : cache docker ignore, paquets .deb recompiles"
            echo "               et le cache /var/cache/osmo-debs reecrit."
            echo "  --lite       Construit EN PLUS osmocom-nitb:lite : la meme pile, sans les"
            echo "               arbres de sources de /opt/GSM (~4 Go sur 11). Voir Dockerfile.lite."
            echo "  --stp        Construit EN PLUS osmocom-stp (Dockerfile.stp), le hub SS7 seul."
            echo "  --build-run  Construit SEULEMENT osmocom-run (Dockerfile.run) sur l image"
            echo "               osmocom-nitb deja presente, puis s arrete : ni preparation de"
            echo "               l hote, ni rebuild de la base. QEMU_CACHE_BUST force les git pull."
            echo ""
            echo "  OSMO_DEB_CACHE=/chemin   cache des .deb compiles (defaut /var/cache/osmo-debs)"
            exit 0
            ;;
        *) echo -e "${YELLOW}[WARN] Argument inconnu ignore : $arg${NC}" ;;
    esac
done

# --build-run: only the update layer (Dockerfile.run), on top of the local
# osmocom-nitb image: nothing to prepare on the host.
if [[ "$BUILD_RUN" -eq 1 ]]; then
    if ! docker image inspect osmocom-nitb >/dev/null 2>&1; then
        echo -e "${RED}[ERREUR] image osmocom-nitb absente : lancer d abord sudo $0${NC}"
        exit 1
    fi
    echo "--- Lancement du build de l'image osmocom-run ---"
    if docker build $NO_CACHE --build-arg QEMU_CACHE_BUST=$(date +%s) \
           -f "$DIR/Dockerfile.run" -t osmocom-run "$DIR"; then
        echo -e "${GREEN}[OK] Image osmocom-run construite.${NC}"
        exit 0
    fi
    echo -e "${RED}[ERREUR] Le build de l'image osmocom-run a echoue.${NC}"
    exit 1
fi

# 1. Root privileges
if [[ $EUID -ne 0 ]]; then
   echo -e "${RED}[ERREUR] Ce script doit etre lance en tant que root (sudo).${NC}"
   exit 1
fi

echo "--- Preparation complete de l'hote (SDR & Docker) ---"
```

```
export DEBIAN_FRONTEND=noninteractive

# 2. apt-fast on the host: the same installer as in the images
echo "[*] apt-fast (packaging/apt-fast-install.sh)..."
bash "$DIR/packaging/apt-fast-install.sh" || echo -e "${YELLOW}[WARN] apt-fast non installe - apt-get${NC}"
command -v apt-fast >/dev/null 2>&1 || apt-fast() { apt-get "$@"; }

# ── Ubuntu mirror: measured, not assumed - for the image AND the host ──────
# packaging/apt-mirror.sh benchmarks a few mirrors:
#   - the fastest goes into the image as a build-arg (compose reads it from the
#     environment); OSMO_UBUNTU_MIRROR=http://... forces a mirror;
#   - if archive.ubuntu.com does NOT answer and the host apt sources point to
#     it, they are switched to the fastest mirror, with a backup (*.osmo-bak),
#     otherwise no host package (qemu-user-static, docker...) could be
#     installed. OSMO_HOST_MIRROR=0 forbids touching the host sources,
#     OSMO_HOST_MIRROR=force switches them even if archive answers (slowly).
echo "[*] Miroir Ubuntu (packaging/apt-mirror.sh)..."
_MLIST="$(bash "$DIR/packaging/apt-mirror.sh" --list noble 2>&1 | tee /dev/stderr | grep -E '^[0-9]+' || true)"
OSMO_UBUNTU_MIRROR="${OSMO_UBUNTU_MIRROR:-$(printf '%s\n' "$_MLIST" | head -1 | cut -f2)}"
OSMO_UBUNTU_MIRROR="${OSMO_UBUNTU_MIRROR:-http://archive.ubuntu.com/ubuntu}"
export OSMO_UBUNTU_MIRROR
echo -e "${GREEN}[OK] miroir : ${OSMO_UBUNTU_MIRROR}${NC}"
if { [ "${OSMO_HOST_MIRROR:-1}" = "force" ] || { [ "${OSMO_HOST_MIRROR:-1}" = "1" ] && ! printf '%s\n' "$_MLIST" |
 ↪  grep -q 'archive\.ubuntu\.com'; }; } \
   && [ "$OSMO_UBUNTU_MIRROR" != "http://archive.ubuntu.com/ubuntu" ]; then
    _switched=""
    for _f in /etc/apt/sources.list /etc/apt/sources.list.d/*.list /etc/apt/sources.list.d/*.sources; do
        [ -f "$_f" ] || continue
        grep -qE '^(deb|deb-src|URIs:).*https?://(archive|[a-z]{2}\.archive|security)\.ubuntu\.com/ubuntu' "$_f"
         ↪  || continue
        cp -f "$_f" "$_f.osmo-bak"
        sed -i -E "s#https?://(archive|[a-z]{2}\.archive|security)\.ubuntu\.com/ubuntu/?#${OSMO_UBUNTU_MIRROR}#g"
         ↪  "$_f"
        _switched="$_switched $_f"
    done
    if [ -n "$_switched" ]; then
        echo -e "${YELLOW}[WARN] archive.ubuntu.com ne repond pas : sources apt de l hote basculees sur
         ↪  ${OSMO_UBUNTU_MIRROR}${NC}"
        echo -e "${YELLOW}       (sauvegarde en *.osmo-bak :${_switched} ; OSMO_HOST_MIRROR=0 pour ne pas y
         ↪  toucher)${NC}"
    fi
fi
apt-fast update -qq || true

# 3. Docker, docker compose v2 and buildx: always dependencies, but only what
#    is MISSING is installed - never docker.io when docker is already there
#    (docker-ce + containerd.io conflict with it). Both package namings are
#    accepted: docker-compose-v2 / docker-buildx (Ubuntu) and
#    docker-compose-plugin / docker-buildx-plugin (Docker repository).
#    OSMO_HOST_DOCKER=0: do not ask apt for anything docker-related (a GitHub
#    runner ships docker, compose v2 and buildx as moby-* packages).
echo "[*] Docker + docker compose v2 + buildx..."
if [ "${OSMO_HOST_DOCKER:-1}" = 0 ]; then
    echo -e "${GREEN}[OK] docker : installation sautee (OSMO_HOST_DOCKER=0) - $(docker --version 2>/dev/null ||
     ↪  echo 'docker introuvable, la suite dira si c est un probleme')${NC}"
elif command -v docker >/dev/null 2>&1; then
    echo -e "${GREEN}[OK] docker deja installe ($(docker --version 2>/dev/null || echo version inconnue)) - pas
     ↪  touche${NC}"
    docker info >/dev/null 2>&1 || systemctl start docker >/dev/null 2>&1 || true
    docker compose version >/dev/null 2>&1 \
        || apt-fast install -y --no-install-recommends docker-compose-v2 \
        || apt-fast install -y --no-install-recommends docker-compose-plugin \
        || echo -e "${YELLOW}[WARN] docker present mais pas de compose v2 : compose.sh basculera sur docker
         ↪  build${NC}"
    docker buildx version >/dev/null 2>&1 \
        || apt-fast install -y --no-install-recommends docker-buildx \
```

```
        || apt-fast install -y --no-install-recommends docker-buildx-plugin \
        || echo -e "${YELLOW}[WARN] buildx absent : --arch=arm64 ne marchera pas sur cet hote${NC}"
else
    apt-fast install -y --no-install-recommends docker.io docker-compose-v2 docker-buildx \
        || apt-fast install -y --no-install-recommends docker.io docker-compose-v2 \
        || apt-fast install -y --no-install-recommends docker.io \
        || command -v docker >/dev/null 2>&1 \
        || { echo -e "${RED}[ERREUR] docker / docker-compose-v2 non installables.${NC}"; \
             echo -e "${RED}        (docker absent du PATH : $PATH)${NC}"; \
            echo -e "${RED}       OSMO_HOST_DOCKER=0 pour sauter cette etape si docker est pose autrement.${NC}";
             ↪  exit 1; }
fi
systemctl enable --now docker >/dev/null 2>&1 || true
HAVE_COMPOSE=0
docker compose version >/dev/null 2>&1 && HAVE_COMPOSE=1
# BuildKit output stays the default (tty, folded). To read everything a step
# writes (a one-hour compilation, a silent apt):
#     BUILDKIT_PROGRESS=plain ./build.sh
# Applies to docker compose build AND docker build (and buildx).
export BUILDKIT_PROGRESS="${BUILDKIT_PROGRESS:-auto}"

# ── Target architecture: the host's, or another one through buildx ──────────
# --arch=arm64 on an x86 host: docker buildx build --platform linux/arm64, with
# qemu-user-static emulating aarch64 in the build containers. binfmt_misc must
# carry the F (fix-binary) flag, otherwise the container cannot find the
# emulator. Images are tagged :arm64 to coexist with the native ones. The .deb
# cache is already per architecture (osmo-deb names packages ..._arm64.deb).
HOST_ARCH="$(dpkg --print-architecture)"
BUILD_PLATFORM=""; IMG_TAG=""
if [ "$OSMO_ARCH" != "$HOST_ARCH" ]; then
    case "$OSMO_ARCH" in
        arm64) BUILD_PLATFORM=linux/arm64 ;;
        amd64) BUILD_PLATFORM=linux/amd64 ;;
        *) echo -e "${RED}[ERREUR] --arch : amd64 ou arm64 (recu : $OSMO_ARCH)${NC}"; exit 2 ;;
    esac
    IMG_TAG=":$OSMO_ARCH"
    # Everything the --arm chain needs on the host, installed once here: the
    # emulation (qemu-user-static, binfmt-support) and buildx for this image,
    # then the tools of build-iso.sh --arm (iso_modules/20-hote.sh,
    # ISO_HOST_PKGS_COMMON: debootstrap, mke2fs -d, mcopy, sfdisk, kpartx,
    # zstd, python3-pil...) and of armbian/build-armbian.sh (git, docker,
    # qemu-user for the customize-image.sh chroot).
    echo "[*] Paquets hote de la chaine ${OSMO_ARCH} : emulation, buildx, outils d image SD..."
    apt-fast install -y --no-install-recommends qemu-user-static binfmt-support docker-buildx \
        git debootstrap rsync wget curl ca-certificates gnupg zstd xz-utils cpio file gcc make \
        python3 python3-pil python3-yaml e2fsprogs fdisk util-linux dosfstools mtools kpartx \
        || apt-fast install -y --no-install-recommends qemu-user-static binfmt-support \
        || echo -e "${YELLOW}[WARN] qemu-user-static non installe (apt) - binfmt via docker ci-dessous${NC}"
    _bf="/proc/sys/fs/binfmt_misc/qemu-${OSMO_ARCH/arm64/aarch64}"
    _bf="${_bf/amd64/x86_64}"
    [ -e "$_bf" ] || update-binfmts --enable "$(basename "$_bf")" >/dev/null 2>&1 || true
    if ! { [ -e "$_bf" ] && grep -q '^flags:.*F' "$_bf"; }; then
        echo "[*] binfmt $(basename "$_bf") sans drapeau F : enregistrement via tonistiigi/binfmt..."
        docker run --privileged --rm tonistiigi/binfmt --install "${OSMO_ARCH}" >/dev/null 2>&1 || true
    fi
    if [ -e "$_bf" ] && grep -q '^flags:.*F' "$_bf"; then
        echo -e "${GREEN}[OK] emulation ${OSMO_ARCH} (binfmt fix-binary) - images taguees ${IMG_TAG}${NC}"
    else
        echo -e "${RED}[ERREUR] binfmt ${OSMO_ARCH} indisponible (qemu-user-static, binfmt-support)${NC}"; exit 1
    fi
    docker buildx version >/dev/null 2>&1 || { echo -e "${RED}[ERREUR] docker buildx requis pour --arch${NC}";
     ↪  exit 1; }
fi
IMG_NITB="osmocom-nitb${IMG_TAG}"
IMG_LITE="osmocom-nitb:lite${IMG_TAG:+-$OSMO_ARCH}"
IMG_STP="osmocom-stp${IMG_TAG}"
```

```
[ "$HAVE_COMPOSE" = 1 ] && echo -e "${GREEN}[OK] $(docker compose version)${NC}" \
    || echo -e "${YELLOW}[WARN] docker compose absent - repli sur docker build${NC}"

# 4. Critical host dependencies
# SCTP is required by the Osmocom signalling protocols
echo "[*] Installation de SCTP, TUN et D-Bus sur l'hote..."
apt-fast install -y lksctp-tools libsctp-dev dbus uml-utilities libusb-1.0-0-dev \
     wireshark linphone* gnome-terminal whiptail expect netcat-openbsd \
     telnet iproute2 pulseaudio-utils dpkg-dev zstd

# 4b. toast: the GSM 06.10 codec (quut.com), missing from apt
# libgsm1 (apt) ships the library but not the toast/untoast/tcat binaries.
# Built from the official sources under /opt/GSM, binaries installed in
# /usr/local/bin. Non-fatal: without network the build goes on.
echo "[*] Compilation de toast (codec GSM 06.10)..."
_GSM_VER=gsm-1.0.24
_GSM_DIR=gsm-1.0-pl24   # the tarball extracts under this name
_GSM_URL="https://www.quut.com/gsm/${_GSM_VER}.tar.gz"
if command -v toast >/dev/null 2>&1; then
    echo -e "${GREEN}[OK] toast deja present ($(command -v toast)).${NC}"
else
    apt-fast install -y --no-install-recommends build-essential wget >/dev/null 2>&1 || true
    mkdir -p /opt/GSM
    if ( cd /opt/GSM \
         && wget -qO "${_GSM_VER}.tar.gz" "$_GSM_URL" \
         && tar xzf "${_GSM_VER}.tar.gz" \
         && cd "$_GSM_DIR" \
         && { make >/dev/null 2>&1 || true; } \
         && { [ -x bin/toast ] || make toast >/dev/null 2>&1 || true; } \
         && [ -x bin/toast ] ); then
        for _b in toast untoast tcat; do
            [ -e "/opt/GSM/${_GSM_DIR}/bin/$_b" ] \
                && install -m755 "/opt/GSM/${_GSM_DIR}/bin/$_b" "/usr/local/bin/$_b"
        done
        echo -e "${GREEN}[OK] toast installe dans /usr/local/bin (sources : /opt/GSM/${_GSM_DIR}).${NC}"
    else
        echo -e "${YELLOW}[WARN] compilation de toast echouee (reseau ou outils ?) - on continue.${NC}"
    fi
fi

# 5. Kernel modules
echo "[*] Chargement des modules noyau (SCTP & TUN)..."
modprobe sctp
modprobe tun

# SCTP module check (direct read of /proc/modules: no pipe, hence no
# SIGPIPE/pipefail, and anchored on the exact module name)
if grep -q '^sctp ' /proc/modules; then
    echo -e "${GREEN}[OK] Module SCTP charge sur l'hote.${NC}"
else
    echo -e "${RED}[ERREUR] Impossible de charger SCTP.${NC}"
fi

# ============================================================================
# 6. The .deb cache: from the host to the build context
# ============================================================================
OSMO_DEB_CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
mkdir -p "$OSMO_DEB_CACHE" "$DIR/.deb-cache"
find "$DIR/.deb-cache" -maxdepth 1 -name '*.deb' -delete
OSMO_DEB_REFRESH=0
if [[ -n "$NO_CACHE" ]]; then
    OSMO_DEB_REFRESH=1
    echo "[*] Mode --no-cache actif : reconstruction complete, paquets .deb recompiles."
else
    cp -f "$OSMO_DEB_CACHE"/osmo-build-*.deb "$DIR/.deb-cache/" 2>/dev/null || true
    # find, not "ls | wc": under pipefail an empty ls exits 2 and set -e would
```

```
    # stop the script silently on the first build.
    _n=$(find "$DIR/.deb-cache" -maxdepth 1 -name '*.deb' | wc -l)
    if [ "$_n" -gt 0 ]; then
        echo -e "${GREEN}[OK] cache .deb : ${_n} paquets repris de ${OSMO_DEB_CACHE} ($(du -sh "$DIR/.deb-cache" |
        ↪  cut -f1))${NC}"
    else
        echo "[*] cache .deb vide (${OSMO_DEB_CACHE}) : premier build, tout sera compile et mis en cache."
    fi
fi
export OSMO_DEB_REFRESH

# Build one compose.yaml service (or its docker build equivalent).
#   $1 = service (nitb|run|lite|stp)   $2 = image   $3 = Dockerfile   $4.. = build-args
compose_build() {
    local svc="$1" image="$2" dockerfile="$3"; shift 3
    # Dockerfile (nitb) does not end with the osmocom-nitb stage: the `debs`
    # stage follows it and only exports the .deb cache for the CI, so without
    # --target docker would build that one. The other Dockerfiles have a single
    # useful target, already the last. compose.yaml carries the same target on
    # its nitb service for the compose branch below.
    local target=()
    [ "$svc" = nitb ] && target=(--target osmocom-nitb)
    if [ -n "$BUILD_PLATFORM" ]; then
        # Other architecture: buildx --platform, --load so the image lands in
        # the local docker (not only in the buildx cache).
        local args=(); local a; for a in "$@"; do args+=(--build-arg "$a"); done
        docker buildx build --platform "$BUILD_PLATFORM" --load $NO_CACHE \
            "${target[@]+"${target[@]}"}" \
            "${args[@]+"${args[@]}"}" -f "$DIR/$dockerfile" -t "$image" "$DIR"
    elif [ "$HAVE_COMPOSE" = 1 ]; then
        docker compose -f "$DIR/compose.yaml" build $NO_CACHE "$svc"
    else
        local args=(); local a; for a in "$@"; do args+=(--build-arg "$a"); done
        docker build $NO_CACHE "${target[@]+"${target[@]}"}" \
            "${args[@]+"${args[@]}"}" -f "$DIR/$dockerfile" -t "$image" "$DIR"
    fi
}

# Copy the packages the image just produced back into the host cache.
#   $1 = image
debs_from_image() {
    local image="$1" cid n
    cid="$(docker create "$image" /bin/true 2>/dev/null)" || return 0
    mkdir -p "$DIR/.deb-cache/out"
    docker cp "$cid:/var/cache/osmo-debs/." "$DIR/.deb-cache/out/" 2>/dev/null || true
    docker rm "$cid" >/dev/null 2>&1 || true
    n=$(find "$DIR/.deb-cache/out" -maxdepth 1 -name 'osmo-build-*.deb' | wc -l)
    if [ "$n" -gt 0 ]; then
        cp -f "$DIR/.deb-cache/out"/osmo-build-*.deb "$OSMO_DEB_CACHE/"
        echo -e "${GREEN}[OK] cache .deb : ${n} paquets dans ${OSMO_DEB_CACHE} ($(du -sh "$OSMO_DEB_CACHE" | cut
        ↪  -f1))${NC}"
    fi
    rm -rf "$DIR/.deb-cache/out"
}

# 7. Docker build
echo "--- Lancement du build de l'image ${IMG_NITB} ---"
if compose_build nitb "$IMG_NITB" Dockerfile "OSMO_DEB_REFRESH=$OSMO_DEB_REFRESH"
↪  "OSMO_UBUNTU_MIRROR=$OSMO_UBUNTU_MIRROR"; then
    echo -e "${GREEN}[OK] Image ${IMG_NITB} construite avec succes.${NC}"
    debs_from_image "$IMG_NITB"
else
    echo -e "${RED}[ERREUR] Le build Docker a echoue.${NC}"
    exit 1
fi
```

```
# 8. lite image: the same stack without the source trees
# osmocom-nitb embeds /opt/GSM - the build trees of gnuradio, gr-gsm, qemu,
# libosmocore... (4 GB out of 11) which were only needed to compile; what runs
# lives in /usr/local. Dockerfile.lite removes them and flattens the result
# (see its header: deleting in one more layer frees nothing).
if [[ "$LITE" -eq 1 ]]; then
    echo "--- Lancement du build de l'image ${IMG_LITE} ---"
    if LITE_BASE="$IMG_NITB" LITE_IMAGE="$IMG_LITE" \
       compose_build lite "$IMG_LITE" Dockerfile.lite "BASE=$IMG_NITB"; then
        echo -e "${GREEN}[OK] Image ${IMG_LITE} construite.${NC}"
        docker images --format '  {{.Repository}}:{{.Tag}}\t{{.Size}}' \
            | grep -E '^\s+osmocom-nitb:(latest|lite)' || true
    else
        echo -e "${RED}[ERREUR] Le build de l'image lite a echoue.${NC}"
        exit 1
    fi
fi

# 9. STP image: the SS7 hub alone (Dockerfile.stp), with the same .deb cache
if [[ "$STP" -eq 1 ]]; then
    echo "--- Lancement du build de l'image ${IMG_STP} ---"
    if compose_build stp "$IMG_STP" Dockerfile.stp "OSMO_DEB_REFRESH=$OSMO_DEB_REFRESH"
     ↪  "OSMO_UBUNTU_MIRROR=$OSMO_UBUNTU_MIRROR"; then
        echo -e "${GREEN}[OK] Image ${IMG_STP} construite.${NC}"
        debs_from_image "$IMG_STP"
    else
        echo -e "${RED}[ERREUR] Le build de l'image stp a echoue.${NC}"
        exit 1
    fi
fi

# The context does not keep the packages: they live in the host cache.
find "$DIR/.deb-cache" -maxdepth 1 -name '*.deb' -delete
```

2.8 /opt/GSM/osmo-operator/compose.sh

10014 octets, 211 lignes → 211 lignes

```
#!/bin/bash
# ============================================================================
#  compose.sh - the bench with docker compose v2: images, hub + one operator
# ============================================================================
#
#    sudo ./compose.sh build [--no-cache] [--lite] [--stp]   the images (= build.sh)
#    sudo ./compose.sh up [--ms N] [--phy faketrx|virtphy|qemu] [--no-run]
#    sudo ./compose.sh down          stop and remove hub + operator (configs kept)
#    sudo ./compose.sh ps | logs [service] | shell
#    sudo ./compose.sh debs          the host .deb cache (/var/cache/osmo-debs)
#
#  `up` does what start-docker.sh does for ONE operator, with compose.yaml as
#  the bench description:
#    1. the configs (apply_config_templates from generate_configs.sh, the hub
#       config from helpers/create_interop.sh) in data/compose/;
#    2. data/compose/.env: the addresses computed by the same functions as
#       start-docker.sh (op_backbone_ip, op_private_*), the choices (N_MS, PHY_MODE);
#    3. data/compose/compose.audio.yaml: the host PulseAudio socket and
#       /dev/snd, when they exist (compose cannot make them optional);
#    4. docker compose --profile bench up -d;
#    5. run.sh in the container, wait for the HLR, test subscribers in the HLR
#       (same IMSI / MSISDN <node>00<op><ms> / Ki as start-docker.sh).
#  For N operators, the WAN and published ports: start-docker.sh.
# ============================================================================
set -euo pipefail
```

```
DIR="$(cd "$(dirname "$(readlink -f "$0")")" && pwd)"
cd "$DIR"
RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'

CDIR="${OSMO_COMPOSE_DIR:-$DIR/data/compose}"
FILES=(-f "$DIR/compose.yaml")

usage() { sed -n '2,24p' "$0" | sed 's/^#\{1,2\} \{0,1\}//'; exit "${1:-0}"; }
die() { echo -e "${RED}$*${NC}" >&2; exit 1; }

need_root() { [ "$(id -u)" -eq 0 ] || die "Root requis (sudo)."; }
need_compose() {
    docker compose version >/dev/null 2>&1 \
        || die "docker compose v2 absent : sudo ./build.sh l installe (docker-compose-v2 + docker-buildx)."
}

# The start-docker.sh library: its functions (op_*, apply_config_templates...)
# without its execution - same cut as build-iso.sh (load_start_lib).
load_start_lib() {
    local lib; lib="$(mktemp)"
    awk '
        /^banner[[:space:]]*$/                    { exit }
        /^\[ "\$\{1:-\}" = "stop" \]/             { exit }
        /^\[ "\$\(id -u\)" -ne 0 \]/              { exit }
        /^choose_network_mode[[:space:]]*$/       { exit }
        /^\.\//                                   { exit }
        /^case "\$NETWORK_MODE" in[[:space:]]*$/  { exit }
        { print }
    ' "$DIR/launch/start-docker.sh" > "$lib"
    export OSMO_REPO_DIR="$DIR"
    # shellcheck disable=SC1090
    source "$lib"
    rm -f "$lib"
}

compose() {
    local envf=()
    [ -f "$CDIR/.env" ] && envf=(--env-file "$CDIR/.env")
    [ -f "$CDIR/compose.audio.yaml" ] && FILES+=(-f "$CDIR/compose.audio.yaml")
    docker compose "${FILES[@]}" "${envf[@]+"${envf[@]}"}" "$@"
}

cmd_build() { need_root; exec bash "$DIR/build.sh" "$@"; }

cmd_up() {
    need_root; need_compose
    local n_ms=2 phy="${PHY_MODE:-faketrx}" do_run=1
    while [ $# -gt 0 ]; do
        case "$1" in
            --ms)     n_ms="${2:?}"; shift ;;
            --ms=*)   n_ms="${1#*=}" ;;
            --phy)    phy="${2:?}"; shift ;;
            --phy=*)  phy="${1#*=}" ;;
            --no-run) do_run=0 ;;
            -h|--help) usage 0 ;;
            *) die "option inconnue : $1" ;;
        esac; shift
    done
    [[ "$n_ms" =~ ^[1-9][0-9]?$ ]] || die "--ms : 1 a 99"

    for img in "${IMAGE_RUN:-osmocom-run}" "${IMAGE_STP:-osmocom-stp}"; do
        docker image inspect "$img" >/dev/null 2>&1 \
            || die "image $img absente : sudo ./compose.sh build --stp (ou ./build.sh --stp, puis
            ↪  ./launch/start-docker.sh une fois pour osmocom-run)"
    done
```

```
    load_start_lib
    export OSMO_PRIV_BY_OP=1      # provision PER operator (see _osmo_priv_index)
    local op=1 node="${WAN_NODE_ID:-1}"
    local bb_ip priv_net priv_gw priv_ip mcc mnc name
    bb_ip="$(op_backbone_ip "$op")"
    priv_net="$(op_private_net "$op")"; priv_gw="$(op_private_gw "$op")"; priv_ip="$(op_private_ip "$op")"
    mcc="$(op_mcc "$op")"; mnc="$(op_mnc "$op")"; name="$(op_default_name "$op")"

    echo -e "${CYAN}── Banc compose : hub 172.20.0.10 + operateur ${op} (${name}, MCC ${mcc} MNC ${mnc}, ${n_ms}
     ↪  MS, PHY ${phy}) ──${NC}"
    echo -e "  backbone ${CYAN}${bb_ip}${NC}   prive ${CYAN}${priv_ip}${NC} (${priv_net})"

    # 1. The configs
    rm -rf "$CDIR/op1" "$CDIR/hub"
    mkdir -p "$CDIR/op1/osmocom" "$CDIR/op1/asterisk" "$CDIR/op1/bb" "$CDIR/hub"
    apply_config_templates "$CDIR/op1" "$priv_ip" "$priv_gw" "$op" \
        "1.${op}.1" "1.${op}.2" "1.${op}.3" "$mcc" "$mnc" "$name" \
        172.20.0.10 "no shutdown" 1 \
        || die "generation des configs echouee"
    bash "$DIR/helpers/create_interop.sh" 1 "$CDIR/hub/osmo-stp-interop.cfg" >/dev/null \
        || die "generation de la config du hub echouee"
    for f in extensions.conf pjsip.conf rtp.conf modules.conf annuaire.conf; do
        [ -f "$CDIR/op1/asterisk/$f" ] || { [ -f "$DIR/configs/$f" ] && cp "$DIR/configs/$f"
         ↪  "$CDIR/op1/asterisk/$f" || : > "$CDIR/op1/asterisk/$f"; }
    done
    [ -f "$CDIR/op1/bb/mobile.cfg" ] || cp "$DIR/configs/mobile.cfg" "$CDIR/op1/bb/mobile.cfg"
    cp -f "$DIR/configs/asound.conf" "$CDIR/asound.conf"
    chmod +x "$CDIR/op1/osmocom"/*.sh 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} configs dans ${CDIR}"

    # 2. .env: what compose.yaml reads
    local host_ip; host_ip="$(ip -4 route get 1.1.1.1 2>/dev/null | awk '{for(i=1;i<=NF;i++) if($i=="src") print
     ↪  $(i+1)}' | head -1)"
    cat > "$CDIR/.env" <<ENV
# genere par compose.sh up - ne pas editer : relancer compose.sh up
IMAGE_RUN=${IMAGE_RUN:-osmocom-run}
IMAGE_STP=${IMAGE_STP:-osmocom-stp}
OSMO_COMPOSE_DIR=${CDIR}
OP1_BACKBONE_IP=${bb_ip}
OP1_PRIV_NET=${priv_net}
OP1_PRIV_GW=${priv_gw}
OP1_PRIV_IP=${priv_ip}
N_MS=${n_ms}
PHY_MODE=${phy}
HOST_IP=${host_ip}
WAN_NODE_ID=${node}
ENV

    # 3. Host audio: pulse socket and /dev/snd when they exist
    local pa=""
    for s in /var/run/pulse/native /run/pulse/native "/run/user/${SUDO_UID:-$(id -u)}/pulse/native"; do
        [ -S "$s" ] && { pa="$s"; break; }
    done
    {
        echo "# genere par compose.sh up : l audio de l hote, quand il existe"
        echo "services:"
        echo "  operator-1:"
        if [ -n "$pa" ]; then
            echo "    volumes:"; echo "      - ${pa}:/run/pulse/native"
            echo "    environment:"; echo "      PULSE_SERVER: unix:/run/pulse/native"
        fi
        if [ -d /dev/snd ]; then
            echo "    devices:"; echo "      - /dev/snd:/dev/snd"
            getent group audio >/dev/null 2>&1 && { echo "    group_add:"; echo "      - \"$(getent group audio |
             ↪  cut -d: -f3)\""; }
        fi
```

```
    } > "$CDIR/compose.audio.yaml"
    [ -n "$pa" ] && echo -e "  ${GREEN}✓${NC} audio : socket pulse ${pa}" \
                 || echo -e "  ${YELLOW}audio : pas de socket pulse sur l hote - repli TCP 172.20.0.1:4713${NC}"

    # 4. The containers
    modprobe sctp 2>/dev/null || true; modprobe tun 2>/dev/null || true
    mkdir -p /tmp/osmocom-logs/op1
    compose --profile bench up -d --remove-orphans
    echo -e "  ${GREEN}✓${NC} osmo-inter-stp et osmo-operator-1 lances"
    [ "$do_run" = 1 ] || { echo "  (--no-run : run.sh non lance - docker exec -ti osmo-operator-1 bash)"; return
     ↪  0; }

    # 5. The stack, then the subscribers
    echo -e "  ${GREEN}[*] run.sh dans osmo-operator-1...${NC}"
    docker exec -d osmo-operator-1 bash -c \
        "mkdir -p /var/log/osmocom && { RUN_NO_PROCESS=0 CALYPSO_NO_ATTACH=1 /etc/osmocom/run.sh; } >
         ↪  /var/log/osmocom/run.sh.log 2>&1"
    echo -ne "  ${GREEN}[*] Attente HLR (4258)${NC}"
    local retry=0
    while ! docker exec osmo-operator-1 bash -c "echo >/dev/tcp/127.0.0.1/4258" 2>/dev/null; do
        sleep 2; echo -n "."; retry=$((retry + 1))
        [ $retry -ge 45 ] && { echo -e " ${RED}TIMEOUT${NC}"; break; }
    done
    echo -e " ${GREEN}✓${NC}"
    local ms imsi msisdn ki feed
    feed="$(mktemp)"
    echo "enable" > "$feed"
    for ms in $(seq 1 "$n_ms"); do
        imsi="${mcc}${mnc}$(printf '%04d%06d' "$op" "$ms")"
        msisdn="$(osmo_msisdn "$node" "$op" "$ms")"
        ki="$(printf '00112233445566778899aabbccdd%02x%02x' "$ms" "$op")"
        printf 'subscriber imsi %s create\nsubscriber imsi %s update msisdn %s\nsubscriber imsi %s update aud2g
         ↪  comp128v1 ki %s\n' \
            "$imsi" "$imsi" "$msisdn" "$imsi" "$ki" >> "$feed"
        echo "    MS${ms} : IMSI ${imsi}  MSISDN ${msisdn}"
    done
    echo "end" >> "$feed"
    docker cp "$feed" osmo-operator-1:/tmp/hlr_feed.vty; rm -f "$feed"
    docker exec osmo-operator-1 bash -c '(sleep 1; cat /tmp/hlr_feed.vty; sleep 2) | nc -q2 127.0.0.1 4258
     ↪  >/dev/null 2>&1; rm -f /tmp/hlr_feed.vty' || true
    echo -e "  ${GREEN}✓${NC} HLR alimente (${n_ms} abonnes)"
    echo ""
    echo -e "${GREEN}Banc pret.${NC}  docker exec -ti osmo-operator-1 bash   |   ./compose.sh logs   |
     ↪  ./compose.sh down"
}

cmd_down()  { need_root; need_compose; compose --profile bench down --remove-orphans; }
cmd_ps()    { need_compose; compose --profile bench ps; }
cmd_logs()  { need_compose; compose --profile bench logs -f "$@"; }
cmd_shell() { exec docker exec -ti "${1:-osmo-operator-1}" bash; }
cmd_debs()  { ls -1sh "${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"/osmo-build-*.deb 2>/dev/null || echo "cache vide
 ↪  (${OSMO_DEB_CACHE:-/var/cache/osmo-debs})"; }

case "${1:-}" in
    build) shift; cmd_build "$@" ;;
    up)    shift; cmd_up "$@" ;;
    down)  shift; cmd_down ;;
    ps)    cmd_ps ;;
    logs)  shift; cmd_logs "$@" ;;
    shell) shift; cmd_shell "$@" ;;
    debs)  cmd_debs ;;
    -h|--help|help|'') usage 0 ;;
    *) die "commande inconnue : $1 (build|up|down|ps|logs|shell|debs)" ;;
esac
```

2.9 /opt/GSM/osmo-operator/generate_configs.sh

42214 octets, 820 lignes → 816 lignes (2 groupes compactés)

```
#!/usr/bin/env bash
# ============================================================================
#  generate_configs.sh - the ONLY place where the network configuration is
#                        decided, and the only one that substitutes templates.
# ============================================================================
#
#  HOW IT WORKS
#  ------------
#  The files under configs/ carry markers (__MCC__, __ARFCN__, __KI__, ...)
#  that apply_config_templates() replaces at start-up. The derivation
#  formulas (ARFCN, BSIC, IMSI, IMEI, Ki...) are only DEFAULTS: globals.conf,
#  at the repository root, exposes them all and wins.
#
#  TWO USAGES
#  ----------
#    ./generate_configs.sh [--force] [--show] [KEY=VALUE ...]
#         writes/completes globals.conf at the root, then prints the
#         effective values. Without --force an existing globals.conf is
#         NEVER overwritten.
#
#    . ./generate_configs.sh
#         loads globals.conf and defines apply_config_templates() and
#         apply_native_post_patches().
#
#  WHO CALLS WHAT
#  --------------
#    launch/start-docker.sh  -> runs this script (writes globals.conf), then
#                               sources it
#    start-native.sh         -> sources globals.conf
#    lib/gabarits.sh         -> sources this script for apply_config_templates()
#    iso_modules/30-image-configs.sh, startup_scripts/11-regen-configs.sh
#                            -> apply_native_post_patches() on a generated tree
#
#  PRECEDENCE
#  ----------
#    command line / environment  >  globals.conf  >  defaults of _gc_table
#
#  MULTI-OPERATOR
#  --------------
#  In `virtual` mode (N_OPERATORS > 1) each operator needs DISTINCT values.
#  A value set in globals.conf applies to ALL of them, which is almost always
#  wrong for ARFCN, LAC, IMSI... Leave them empty (the default) to keep the
#  per-operator derivation and pin only what you really want pinned. The
#  script warns when a per-operator field is pinned with N_OPERATORS > 1.
#
#  ENVIRONMENT
#  -----------
#    OSMO_NO_REGEN=1    keep the installed configs (start-native.sh default;
#                       --regen clears it)
#    OSMO_NO_STP_IP=1   skip the node SS7 addressing (network/set-node-id.sh)
#    OSMO_WAN_NODE, WAN_NODE_ID, ROLE_FILE (/etc/osmo-role): node number
#    OSMO_ROLE, OSMO_HUB_IP, WAN_STP_TARGET, INTER_STP_IP: WAN node identity
#    SAP_MODE=1         mobile.cfg uses "sim sap" instead of "sim test"
#    RCTX_INTER_OVERRIDE, INTER_LOCAL_IP_OVERRIDE, SIP_REMOTE_IP_OVERRIDE:
#                       set by the ISO build / --regen for a WAN node
#    WAN_CONF_FILE      WAN table (default /etc/osmo-wan.conf)
#
#  SPDX-License-Identifier: GPL-2.0-or-later
# ============================================================================

_GC_HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
_GC_FILE="$_GC_HERE/globals.conf"
```

```
# ── Exposed settings, with their default ──────────────────────────────
# Format: KEY|DEFAULT|PER-OP|COMMENT
#   PER-OP=1 -> the value is normally derived from the operator number;
#               pinning it only makes sense with a single operator.
#
# The body of _gc_table is a TABLE, not shell: the only non-data line allowed
# is "#SECTION|<title>". A plain "#" line there becomes a malformed entry, so
# explanations go HERE, above the function.
#
# Known limitation: the mobile emulated under QEMU does not decipher A5/1 yet
# (the cipher path of hw/arm/calypso/l1ctl_sock.c is not exercised), so with
# the default ENCRYPTION="a5 1" its Location Update stops after CIPHERING
# MODE COMPLETE (T3210). The osmocom-bb side-car handles A5/1 fine.
# Work-around: ENCRYPTION="a5 0" on the command line.
_gc_table() {
cat <<'TABLE'
#SECTION|Identite du reseau
MCC|001|0|Mobile Country Code (3 chiffres)
MNC|01|0|Mobile Network Code (2 chiffres)
OP_NAME|OsmoQEMU|0|Nom court et long du reseau
#SECTION|Securite et carte SIM
ENCRYPTION|a5 1|0|Chiffrement : "a5 1" (defaut), "a5 0" (aucun), "a5 0 1 3"...
SIM_ALGO|comp128|0|Algorithme d'authentification (comp128, xor...)
KI||1|Ki de la SIM, 16 octets hexa espaces
IMSI||1|IMSI du mobile
IMEI||1|IMEI du mobile
#SECTION|Radio
ARFCN||1|Canal radio (a accorder avec BAND)
BAND|DCS1800|0|Bande : GSM900, DCS1800, PCS1900, GSM850
BSIC||1|Base Station Identity Code, 0..63
LAC||1|Location Area Code
CELL_ID||1|Cell identity
IPA_UNIT_ID||1|Identifiant IPA du BTS
MS_MAX_POWER|15|0|Puissance max autorisee au mobile (0..15)
RXLEV_ACCESS_MIN|0|0|Niveau recu minimal pour acceder a la cellule
CELL_RESEL_HYST|4|0|Hysteresis de reselection de cellule (dB)
T3109|5|0|Timer T3109 du BSC (s) - liberation de canal
#SECTION|GPRS
BVCI||1|BSSGP Virtual Connection Id
NSEI||1|Network Service Entity Id
NSVCI||1|NS Virtual Connection Id
APN|internet|0|Nom du point d'acces
#SECTION|Services
SMS_SC||1|Numero du centre SMS
#SECTION|Dimensionnement
MS_COUNT|2|0|Nombre de mobiles simules
N_OPERATORS|1|0|Nombre d'operateurs (mode "virtual")
HOST_IP|127.0.0.1|0|Adresse de l'hote vue par la pile
TABLE
}

# ── Write globals.conf ────────────────────────────────────────────────
_gc_write() {
    local force=$1
    if [ -f "$_GC_FILE" ] && [ "$force" != "1" ]; then
        printf '  globals.conf existe deja - conserve (--force pour regenerer)\n'
        return 0
    fi
    {
        printf '# ═══════════════════════════════════════════════════════════════\n'
        printf '#  globals.conf - la configuration du reseau, en un seul endroit.\n'
        printf '# ═══════════════════════════════════════════════════════════════\n'
        printf '#\n'
        printf '#  Edite ce fichier a la main : il ne sera JAMAIS reecrit par un run.\n'
        printf '#  Seul "./generate_configs.sh --force" le regenere.\n'
```

```
        printf '#\n'
        printf '#  CE FICHIER FAIT AUTORITE - SUR LES VARIABLES QU'"'"'IL DECLARE, ET SUR\n'
        printf '#  ELLES SEULES. Il est lu en dernier : les %s reglages ci-dessous\n' "$(_gc_table | grep -vc
        ↪  '^#SECTION')"
        printf '#  ecrasent leur homonyme dans l'"'"'environnement. Tout le reste (CALYPSO_*,\n'
        printf '#  LOG_DIR, MODE...) passe au travers, intact. Deux facons de le modifier :\n'
        printf '#      vim globals.conf\n'
        printf '#      ./generate_configs.sh ARFCN=520\n'
        printf '#\n'
        printf '#  Un champ VIDE = valeur calculee automatiquement. Les champs notes\n'
        printf '#  [auto/operateur] se deduisent du numero d'"'"'operateur : ne les fige\n'
        printf '#  que si N_OPERATORS=1, sinon tous les operateurs se marchent dessus.\n'
        printf '# ═══════════════════════════════════════════════════════════════════\n'
        local key def perop comment
        while IFS='|' read -r key def perop comment; do
            [ -n "$key" ] || continue
            if [ "$key" = "#SECTION" ]; then
                printf '\n# ── %s %s\n\n' "$def" \
                    "$(printf '%.0s─' $(seq 1 $((56 - ${#def}))))"
                continue
            fi
            printf '# %s%s\n' "$comment" \
                "$([ "$perop" = 1 ] && printf '  [auto/operateur]')"
            case "$def" in
                *" "*|"") printf '%s="%s"\n' "$key" "$def" ;;
                *)        printf '%s=%s\n'   "$key" "$def" ;;
            esac
        done < <(_gc_table)
    } > "$_GC_FILE"
    printf '  globals.conf ecrit (%s reglages)\n' \
           "$(_gc_table | grep -vc '^#SECTION')"
}

# ── Set one value IN the file, in place, keeping the layout ──────────
_gc_set() {
    local key="${1%%=*}" val="${1#*=}"
    _gc_table | grep -q "^${key}|" || {
        printf '  ⚠ reglage inconnu : %s (voir --show)\n' "$key" >&2; return 1; }
    [ -f "$_GC_FILE" ] || _gc_write 1 >/dev/null
    local q; case "$val" in *" "*|"") q="\"$val\"" ;; *) q="$val" ;; esac
    if grep -qE "^${key}=" "$_GC_FILE"; then
        sed -i "s|^${key}=.*|${key}=${q}|" "$_GC_FILE"
    else
        printf '%s=%s\n' "$key" "$q" >> "$_GC_FILE"
    fi
    printf '  %s = %s\n' "$key" "${val:-(auto)}"
}

# ── Loading ──────────────────────────────────────────────────────────
# Loads globals.conf, then COMPLETES it with the table defaults. Precedence:
#     command line / environment  >  globals.conf  >  default of _gc_table
# globals.conf assigns unconditionally, so the keys already present in the
# environment are saved BEFORE sourcing it and restored AFTER. Any key still
# empty then receives its factory default: an absent globals.conf (fresh
# clone, the file is gitignored) must never leave BAND, SIM_ALGO, APN...
# empty in the substituted configs, or osmo-bsc refuses to start.
gc_load() {
    local key def perop comment
    local -A _gc_env=()
    while IFS='|' read -r key def perop comment; do
        [ -n "$key" ] && [ "$key" != "#SECTION" ] || continue
        if [ -n "${!key:-}" ]; then _gc_env["$key"]="${!key}"; fi
    done < <(_gc_table)

    [ -f "$_GC_FILE" ] && . "$_GC_FILE"
```

```
    if [ "${#_gc_env[@]}" -gt 0 ]; then
        for key in "${!_gc_env[@]}"; do printf -v "$key" '%s' "${_gc_env[$key]}"; done
    fi

    while IFS='|' read -r key def perop comment; do
        [ -n "$key" ] && [ "$key" != "#SECTION" ] || continue
        [ -n "${!key:-}" ] || printf -v "$key" '%s' "$def"
    done < <(_gc_table)
    return 0
}

# ── Random IMEI with its Luhn check digit ──────────────────────────────
# Plausible TAC (35892500) + six random serial digits + Luhn digit, so that
# two benches, or two MS of the same operator, never share an IMEI and the
# value passes the check of equipment that validates it.
rand_imei() {
    local body d i sum=0 dbl parity
    body="35892500$(printf '%06d' "$(( (RANDOM << 15 | RANDOM) % 1000000 ))")"
    # Luhn over the 14 digits: double every second digit from the right,
    # folding two-digit results.
    for (( i = ${#body} - 1; i >= 0; i-- )); do
        d=${body:i:1}
        parity=$(( (${#body} - 1 - i) % 2 ))
        if [ "$parity" -eq 0 ]; then
            dbl=$(( d * 2 )); [ "$dbl" -gt 9 ] && dbl=$(( dbl - 9 ))
            sum=$(( sum + dbl ))
        else
            sum=$(( sum + d ))
        fi
    done
    printf '%s%d' "$body" "$(( (10 - sum % 10) % 10 ))"
}

# ── THE NUMBERING PLAN, IN ONE PLACE ───────────────────────────────────
#
#   MSISDN = <node> 00 <operator> <MS rank on 2 digits>
#            node ⟨1⟩, operator 1, MS ⟨2⟩  ->  ⟨3⟩  ×3
    ⟨⟩ = (1,1,100101) (1,2,100102) (2,1,200101)
#
# The leading digit is the node address, so SMS and dialplan routing can be
# read directly from the number on a multi-node bench.
#
# Invariant: the IMSI (MCC MNC <op on 4 digits> <ms on 6>) and the Ki
# (00 11 ... dd <ms> <op>) do NOT depend on the node. They are derived
# identically here (mobile.cfg), in start-native.sh (mobiles) and in
# launch/start-docker.sh (HLR subscribers) and must stay bit-identical,
# otherwise authentication fails on a Kc mismatch with no trace other than a
# CIPHER MODE reject. Only the MSISDN, which enters no security computation,
# carries the node.
osmo_msisdn_pfx() { printf '%s00' "${1:-1}"; }                      # <node>00
osmo_msisdn()     { printf '%d' "$(( ${1:-1} * 100000 + ${2:-1} * 100 + ${3:-1} ))"; }

# The node as seen by the generator: environment first (what
# launch/start-docker.sh passes to a container and what start-native.sh
# exports before a --regen), then /etc/osmo-role, then 1. NEVER empty: a
# numbering plan without a node is everybody's plan.
osmo_node_id() {
    local n="${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}"
    if [ -z "$n" ] && [ -r "${ROLE_FILE:-/etc/osmo-role}" ]; then
        n="$(awk -F= '/^OSMO_WAN_NODE=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' \
             "${ROLE_FILE:-/etc/osmo-role}" 2>/dev/null)"
    fi
    case "$n" in [1-9]) ;; *) n=1 ;; esac
    printf '%s' "$n"
}
```

```
# ── Template substitution - the SINGLE implementation ──────────────────────
#  Signature shared by all callers:
#    apply_config_templates DEST CONTAINER_IP GATEWAY_IP OP_ID \
#                           PC_MSC PC_STP PC_BSC MCC MNC OP_NAME \
#                           INTER_STP INTER_STP_SHUTDOWN N_OPERATORS
#  Writes DEST/osmocom, DEST/asterisk and DEST/bb from configs/ and scripts/,
#  then DEST/osmocom/radio-plan.env (the decided radio plan and PLMN, read by
#  start-native.sh for the mobiles and the side-car) and sms-routing.conf.
apply_config_templates() {
    local dest=$1 container_ip=$2 gateway_ip=$3 op_id=$4

    # OSMO_NO_REGEN=1: keep the configs already in place. In a container they
    # were just generated by launch/start-docker.sh with the node identity,
    # point codes and inter-STP; regenerating from the templates would replace
    # them with the single-machine defaults. start-native.sh sets it by
    # default, --regen clears it.
    if [ "${OSMO_NO_REGEN:-0}" = "1" ]; then
        echo "  gabarits : conserves (OSMO_NO_REGEN=1 ; --regen pour regenerer)"
        return 0
    fi
    local pc_msc=$5 pc_stp=$6 pc_bsc=$7 mcc=$8 mnc=$9 op_name=${10}

    # PLMN check before writing anything: osmo-msc refuses "network country
    # code 000" (an empty MCC gives 000 through printf), exits, and systemd
    # restarts it endlessly. Fail HERE, where the parameter name is known.
    case "$mcc" in
        ''|000|00|0) echo "  ERREUR : MCC invalide ('$mcc') pour l'operateur $op_id - osmo-msc refuserait la
        ↪  config" >&2; return 1 ;;
    esac
    case "$mnc" in
        ''|'-') echo "  ERREUR : MNC vide pour l'operateur $op_id" >&2; return 1 ;;
    esac
    local inter_stp=${11} inter_stp_shutdown=${12} n_operators=${13}

    # The node enters the numbering plan (MSISDN = <node>00<op><ms>), NOT the
    # IMSI nor the Ki (see osmo_msisdn).
    local node msisdn_pfx
    node="$(osmo_node_id)"
    msisdn_pfx="$(osmo_msisdn_pfx "$node")"

    mkdir -p "$dest/osmocom" "$dest/asterisk" "$dest/bb"
    local f bn

    for f in configs/*.cfg; do
        [ "$(basename "$f")" = "osmo-stp-interop.cfg" ] && continue
        cp "$f" "$dest/osmocom/"
    done
    [ -f "configs/osmo-bts-virtual.cfg" ] && cp "configs/osmo-bts-virtual.cfg" "$dest/osmocom/"
    for f in configs/*.conf; do
        bn=$(basename "$f"); [ "$bn" = "sms-routing.conf" ] && continue
        cp "$f" "$dest/asterisk/"
    done
    # scripts/ as a whole, so that a script added later is never silently
    # left out.
    cp scripts/* "$dest/osmocom/" 2>/dev/null || true
    chmod +x "$dest/osmocom"/*.sh 2>/dev/null || true

    if [ -f "configs/mobile.cfg.template" ]; then
        cp "configs/mobile.cfg.template" "$dest/bb/mobile.cfg"
        cp "configs/mobile.cfg.template" "$dest/bb/mobile_group1.cfg"
    elif [ -f "configs/mobile.cfg" ]; then
        cp "configs/mobile.cfg" "$dest/bb/mobile.cfg"
        cp "configs/mobile.cfg" "$dest/bb/mobile_group1.cfg"
    fi
    # The SIM mode of the template follows --sap: "sim sap" only when
    # SAP_MODE=1 (set by --sap / --softsim in startup_scripts/01-defauts-options.sh),
```

```
# "sim test" otherwise. Same rule as ms_sim_mode in
# startup_scripts/06-banc-dsp-menu.sh; containers, without softSIM, stay
# in "sim test".
local _sim=test _f
[ "${SAP_MODE:-0}" = 1 ] && _sim=sap
for _f in "$dest/bb/mobile.cfg" "$dest/bb/mobile_group1.cfg"; do
    [ -f "$_f" ] && sed -i "s#^\([[:space:]]*\)sim \(sap\|test\|reader\)\$#\1sim ${_sim}#" "$_f"
done

local rctx_msc rctx_stp rctx_bsc rctx_inter
rctx_msc=$(op_rctx_msc "$op_id");   rctx_stp=$(op_rctx_stp "$op_id")
rctx_bsc=$(op_rctx_bsc "$op_id")
# RCTX_INTER_OVERRIDE: set when the image is a WAN NODE. The routing
# context identifies the AS at the hub, and the local formula (op*100+50)
# gives the same value on every node. Empty = local formula.
rctx_inter="${RCTX_INTER_OVERRIDE:-$(op_rctx_inter "$op_id")}"

# ── The values: globals.conf wins, otherwise the default formula ──────────
local arfcn="${ARFCN:-$(( 512 + op_id * 2 ))}"
# The SECOND BTS (the side-car) needs its own channel: +100 keeps the
# side-cars (614, 616, 618...) out of reach of the main plan (514, 516,
# 518...) and inside the DCS1800 band (512-885). In a virtual medium
# fake_trx pairs by frequency, so two cells on one ARFCN mix their bursts.
local arfcn_bts1="${ARFCN_BTS1:-$(( 612 + op_id * 2 ))}"
# +2 on the main BTS step of 10: 6010/6012, 6020/6022...
local ipa_unit_id_bts1="${IPA_UNIT_ID_BTS1:-$(( 6000 + op_id * 10 + 2 ))}"
local bsic="${BSIC:-$(( (op_id * 7) % 64 ))}"
# The side-car has its own BSIC: it is what tells two cells apart in the
# mobile measurements.
local bsic_bts1="${BSIC_BTS1:-$(( (bsic + 1) % 64 ))}"
local lac="${LAC:-0x000${op_id}}"
local cell_id="${CELL_ID:-$(( 6000 + op_id ))}"
# Step of 10, not 6000+op_id: the second BTS takes op*10+2, so the two BTS
# of an operator never present the same unit-id to the BSC (which would
# accept one and eject the other, in a loop), whatever the operator count.
local ipa_unit_id="${IPA_UNIT_ID:-$(( 6000 + op_id * 10 ))}"
local bvci="${BVCI:-$(( op_id * 10 + 2 ))}"
local nsei="${NSEI:-$(( op_id * 10 ))}"
local nsvci="${NSVCI:-$(( op_id * 10 ))}"
# Same subscriber plan as the HLR (launch/start-docker.sh, "HLR subscribers"):
#    IMSI = MCC MNC <operator on 4 digits> <MS index on 6>
# This template serves MS #1.
local imsi="${IMSI:-${mcc}${mnc}$(printf '%04d%06d' "$op_id" 1)}"
local imei="${IMEI:-$(rand_imei)}"
# Ki: last two bytes = MS index, then operator - the HLR order
# (printf '...dd%02x%02x' ms_idx op_id). Without a common Ki there is no
# Kc and the CIPHER MODE is rejected.
local ki="${KI:-00 11 22 33 44 55 66 77 88 99 aa bb cc dd $(printf '%02x %02x' 1 "$op_id")}"
local sms_sc="${SMS_SC:-+336661234$(printf '%04d' "$op_id")}"

local inter_local_ip sip_remote_ip rtp_start rtp_end sip_host_port
# On a WAN node the ASP source address is not a docker-plan IP but the
# segment address, or 0.0.0.0 when it comes from DHCP.
inter_local_ip="${INTER_LOCAL_IP_OVERRIDE:-$(op_backbone_ip "$op_id")}"
# The SIP leg to Asterisk is NOT the backbone: it is local in both
# topologies, and Asterisk identifies it on loopback only (pjsip.conf
# [gsm_msc-identify] match=127.0.0.1).
sip_remote_ip="${SIP_REMOTE_IP_OVERRIDE:-127.0.0.1}"
rtp_start=$(linphone_rtp_start "$op_id"); rtp_end=$(linphone_rtp_end "$op_id")
sip_host_port=$(linphone_sip_port "$op_id")

# ── The radio plan, WRITTEN next to the configurations ─────────────────────
# The generator is the only one that decides ARFCN, unit-id, PLMN...: it
# WRITES what it decided and the other scripts READ it instead of
# recomputing the formulas.
{
```

```
    printf '# radio-plan.env - ecrit par generate_configs.sh. Ne pas editer.\n'
    printf '# Le plan radio et le PLMN de CET operateur, tels qu ils viennent\n'
    printf '# d etre poses dans les configurations voisines. start-native.sh le\n'
    printf '# lit pour accorder les mobiles et pour piloter le side-car.\n'
    printf 'OPERATOR_ID=%s\n'      "$op_id"
    printf 'PLAN_NODE=%s\n'        "$node"
    printf 'PLAN_MSISDN_PFX=%s\n'  "$msisdn_pfx"
    printf 'PLAN_MCC=%s\n'         "$mcc"
    printf 'PLAN_MNC=%s\n'         "$mnc"
    printf 'PLAN_ARFCN=%s\n'       "$arfcn"
    printf 'PLAN_ARFCN_BTS1=%s\n'  "$arfcn_bts1"
    printf 'PLAN_UNIT_ID=%s\n'     "$ipa_unit_id"
    printf 'PLAN_UNIT_ID_BTS1=%s\n' "$ipa_unit_id_bts1"
    printf 'PLAN_BSIC_BTS1=%s\n'    "$bsic_bts1"
    printf 'PLAN_LAC=%s\n'         "$lac"
    printf 'PLAN_BSIC=%s\n'        "$bsic"
    printf 'PLAN_CELL_ID=%s\n'     "$cell_id"
} > "$dest/osmocom/radio-plan.env"

# extensions.conf carries a #include "annuaire.conf" (subscriber entries,
# written by launch/start-docker.sh so that the caller name reaches the
# handset). Asterisk warns on every load when an include is missing, so
# a bench started without start-docker.sh always gets a file, even empty:
# the _X. fallback of extensions.conf does the rest.
if [ ! -f "$dest/asterisk/annuaire.conf" ]; then
    {
        printf '; annuaire.conf - fiches d abonnes, ecrites par start-docker.sh.\n'
        printf '; Vide ici : ce banc a ete monte sans annuaire. Les appels\n'
        printf '; passent, ils affichent le numero au lieu du nom.\n'
        printf '[sub-annuaire]\n'
    } > "$dest/asterisk/annuaire.conf"
fi

for f in "$dest/osmocom"/*.cfg "$dest/asterisk"/*.conf "$dest/bb"/*.cfg; do
    [ -f "$f" ] || continue
    sed -i \
        -e "s|__ENCRYPTION__|${ENCRYPTION}|g" \
        -e "s|__INTER_NET_GATEWAY__|172.20.0.1|g" \
        -e "s|__CONTAINER_IP__|${container_ip}|g" \
        -e "s|__GATEWAY_IP__|${gateway_ip}|g" \
        -e "s|__HLR_IP__|127.0.0.2|g" \
        -e "s|__INTER_STP_IP__|${inter_stp}|g" \
        -e "s|__INTER_STP_SHUTDOWN__|${inter_stp_shutdown}|g" \
        -e "s|__INTER_LOCAL_IP__|${inter_local_ip}|g" \
        -e "s|__SIP_REMOTE_IP__|${sip_remote_ip}|g" \
        -e "s|__MSISDN_PFX__|${msisdn_pfx}|g" \
        -e "s|__OPERATOR_ID__|${op_id}|g" \
        -e "s|__PC_MSC__|${pc_msc}|g" -e "s|__PC_STP__|${pc_stp}|g" -e "s|__PC_BSC__|${pc_bsc}|g" \
        -e "s|__RCTX_MSC__|${rctx_msc}|g" -e "s|__RCTX_STP__|${rctx_stp}|g" \
        -e "s|__RCTX_BSC__|${rctx_bsc}|g" -e "s|__RCTX_INTER__|${rctx_inter}|g" \
        -e "s|__MCC__|${mcc}|g" -e "s|__MNC__|${mnc}|g" -e "s|__OP_NAME__|${op_name}|g" \
        -e "s|__ARFCN__|${arfcn}|g" -e "s|__IPA_UNIT_ID__|${ipa_unit_id}|g" \
        -e "s|__ARFCN_BTS1__|${arfcn_bts1}|g" \
        -e "s|__IPA_UNIT_ID_BTS1__|${ipa_unit_id_bts1}|g" \
        -e "s|__BSIC_BTS1__|${bsic_bts1}|g" \
        -e "s|__CELL_ID__|${cell_id}|g" -e "s|__BSIC__|${bsic}|g" -e "s|__LAC__|${lac}|g" \
        -e "s|__BVCI__|${bvci}|g" -e "s|__NSEI__|${nsei}|g" -e "s|__NSVCI__|${nsvci}|g" \
        -e "s|__IMSI__|${imsi}|g" -e "s|__IMEI__|${imei}|g" -e "s|__KI__|${ki}|g" \
        -e "s|__SMS_SC__|${sms_sc}|g" -e "s|__HOST_IP__|${HOST_IP}|g" \
        -e "s|__BAND__|${BAND}|g" -e "s|__SIM_ALGO__|${SIM_ALGO}|g" \
        -e "s|__MS_MAX_POWER__|${MS_MAX_POWER}|g" \
        -e "s|__RXLEV_ACCESS_MIN__|${RXLEV_ACCESS_MIN}|g" \
        -e "s|__CELL_RESEL_HYST__|${CELL_RESEL_HYST}|g" \
        -e "s|__T3109__|${T3109}|g" -e "s|__APN__|${APN}|g" \
        -e "s|__NEIGHBOR_MODE_BTS0__|$([ -n "${MS_MAGNETITE:-}" ] && [ "${MAGNETITE_VOISINS:-0}" != 1 ] &&
        ↪  echo manual || echo automatic)|g" \
```

```
            -e "s|__TS1_PCHAN_BTS0__|${TS1_PCHAN:-$([ "${OSMO_TRX:-0}" = 1 ] && echo TCH/F || echo SDCCH8)}|g" \
            -e "s|__SIP_HOST_PORT__|${sip_host_port}|g" \
            -e "s|__ALSA_OUTPUT__|${ALSA_OUTPUT}|g" -e "s|__ALSA_INPUT__|${ALSA_INPUT}|g" \
            -e "s|__RTP_START__|${rtp_start}|g" -e "s|__RTP_END__|${rtp_end}|g" \
            "$f"
    done

    generate_pjsip_interop_trunks     "$op_id" "$n_operators" >> "$dest/asterisk/pjsip.conf"
    generate_extensions_interop_out   "$op_id" "$n_operators" >> "$dest/asterisk/extensions.conf"
    # SMS routes (default generator). Regenerated when the real MSISDN of the
    # first MS is missing or a stray route (op0000/op000) is present.
    _src="$dest/osmocom/sms-routing.conf"
    _generate_sms_routing_conf_fallback "$op_id" "$n_operators" >  "$_src"
    if ! grep -q "^$(osmo_msisdn "$node" "$op_id" 1) = " "$_src" 2>/dev/null \
       || grep -qE "^${op_id}0000? = " "$_src" 2>/dev/null; then
        _generate_sms_routing_conf_fallback "$op_id" "$n_operators" >  "$_src"
    fi

    # ── The node SS7 addressing: delegated to set-node-id.sh ──────────────
    # The templates carry the defaults of a SINGLE machine (point code 1.1.2,
    # ASP to 127.0.0.1, shutdown). On a WAN node they are wrong, so
    # network/set-node-id.sh (the only place that keeps osmo-stp, osmo-msc
    # and osmo-bsc and their cross point codes consistent) is replayed AFTER
    # the substitution. Its failure is the generation's failure.
    _apply_node_ss7_addressing "$dest" || return 1
}

# Replays the node SS7 addressing on freshly generated configs.
#   $1 = destination directory (the one that contains osmocom/)
# No effect, silently, when the machine is not a WAN node, when the script is
# missing, or when OSMO_NO_STP_IP=1 asks for it explicitly.
_apply_node_ss7_addressing() {
    local dest="$1"
    local role_file="${ROLE_FILE:-/etc/osmo-role}"
    local here="${GEN_HERE:-$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)}"

    [ "${OSMO_NO_STP_IP:-0}" = "1" ] && return 0
    [ -d "$dest/osmocom" ] || return 0

    # ENVIRONMENT FIRST, the role file next: a container has no
    # /etc/osmo-role, only the variables launch/start-docker.sh passes it
    # (OSMO_WAN_NODE, OSMO_HUB_IP).
    local role node hub
    node="${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}"
    hub="${OSMO_HUB_IP:-}"
    role="${OSMO_ROLE:-}"
    if [ -r "$role_file" ]; then
        [ -n "$role" ] || role="$(awk -F= '/^OSMO_ROLE=/{gsub(/[ \r\t]/,"",$2);v=$2}    END{print v}'
        ↪  "$role_file")"
        [ -n "$node" ] || node="$(awk -F= '/^OSMO_WAN_NODE=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}'
        ↪  "$role_file")"
        [ -n "$hub"  ] || hub="$(awk  -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2);v=$2}  END{print v}'
        ↪  "$role_file")"
    fi
    # An EXPLICIT declaration only: OSMO_ROLE=operator (a VM, a native node)
    # or OSMO_WAN_NODE in the environment (what start-docker.sh passes to a
    # container). WAN_NODE_ID alone is NOT a declaration: on the host it
    # designates the MACHINE's node, not the container whose configs are
    # being generated, and applying it to every operator would give them all
    # the same point code.
    [ "$role" = "operator" ] || [ -n "${OSMO_WAN_NODE:-}" ] || return 0
    [[ "$node" =~ ^[1-9]$ ]] || return 0

    local setid="$here/network/set-node-id.sh"
    [ -r "$setid" ] || return 0
```

```
    # The hub, from any source: without it set-node-id.sh would PROMPT, and
    # nobody answers from a start-up loop.
    [ -n "$hub" ] || hub="${WAN_STP_TARGET:-${INTER_STP_IP:-}}"

    # --op is the operator INDEX inside the point code 1.<node><op>.<role>,
    # not the operator count: a WAN node carries one here. Not the operator
    # id either: in start-docker.sh it is the CONTAINER index, and node 2
    # would move from 1.21.2 to 1.22.2.
    local args=(--node "$node" --op 1 --native --conf-dir "$dest/osmocom")
    [ -n "$hub" ] && args+=(--hub-ip "$hub")

    # </dev/null: this rewrite is automatic and must NEVER ask a question. A
    # reported failure is preferred to a hang.
    if bash "$setid" "${args[@]}" </dev/null >/dev/null 2>&1; then
        echo "  ✓ adressage SS7 du noeud ${node} reapplique (set-node-id.sh)"
    else
        echo "  ⚠ adressage SS7 du noeud ${node} NON reapplique - l'ASP inter-STP" >&2
        echo "    gardera les defauts du gabarit (127.0.0.1, shutdown)." >&2
        # Configs left with the template SS7 identity are no better than no
        # configs: nothing upstream re-reads the point code, so fail here.
        return 1
    fi
}

# ── sms-routing.conf: THE ROUTING IS READ FROM THE NUMBER ──────────────────
# The routing key is the NODE, not the operator, and it is the first digit of
# the MSISDN (<node>00<op><ms>). The addresses come from the WAN TABLE
# (/etc/osmo-wan.conf), the only source that knows where the other nodes are
# really reachable. Without a table (single-node bench) there is one entry:
# ours.
#
#   $1 file  $2 node  $3 operator  $4 MS count  $5 local IP of the core
#   $6 WAN table
_write_sms_routing_native() {
    local out="$1" node="$2" op_id="$3" n_ms="$4" host_ip="$5" wanf="$6"
    local spec="" ent id ip nops n o m
    local -a ids=()

    # WAN_NODES="id:ip:prefix[:operators] ..." - read without sourcing the
    # table: that file also sets WAN_NODE_ID and WAN_OPS, which would
    # overwrite the caller's.
    if [ -r "$wanf" ]; then
        spec="$(sed -n 's/^WAN_NODES=//p' "$wanf" | tr -d '"'"'"'' | tail -1)"
    fi

    # OUR address, the one other nodes reach us at: the table first (an
    # explicit choice), then the default route, loopback last - an isolated
    # bench must still route its SMS.
    local self_ip=""
    for ent in $spec; do
        IFS=: read -r id ip _ nops <<< "$ent"
        [ "$id" = "$node" ] && self_ip="$ip"
    done
    if [ -z "$self_ip" ]; then
        self_ip="$(ip route get 1.1.1.1 2>/dev/null | sed -n 's/.*src \([0-9.]*\).*/\1/p' | head -1)"
    fi
    [ -n "$self_ip" ] || self_ip="$host_ip"

    {
        printf '# sms-routing.conf - ecrit par generate_configs.sh.\n'
        printf '# Cle de routage = NUMERO DE NOEUD, premier chiffre du MSISDN.\n'
        printf '# MSISDN = <noeud>00<operateur><rang du MS sur 2 chiffres>\n\n'
        printf '[local]\n'
        printf 'operator_id = %s\n' "$node"
        printf 'sc_address  = 1999001%s444\n\n' "$op_id"
        printf '[operators]\n'
```

```
        if [ -n "$spec" ]; then
            for ent in $spec; do
                IFS=: read -r id ip _ nops <<< "$ent"
                [ -n "$id" ] || continue
                [ "$id" = "$node" ] && ip="$self_ip"
                printf '%s = %s\n' "$id" "$ip"
            done
        else
            printf '%s = %s\n' "$node" "$self_ip"
        fi
        printf '\n[routes]\n'
        if [ -n "$spec" ]; then
            for ent in $spec; do
                IFS=: read -r id ip _ nops <<< "$ent"
                [ -n "$id" ] || continue
                # nops missing = 3-field entry: default to 1 operator.
                nops="${nops:-1}"
                case "$nops" in ''|*[!0-9]*) nops=1 ;; esac
                for o in $(seq 1 "$nops"); do
                    for m in $(seq 1 "$n_ms"); do
                        printf '%s = %s\n' "$(osmo_msisdn "$id" "$o" "$m")" "$id"
                    done
                done
            done
        else
            for m in $(seq 1 "$n_ms"); do
                printf '%s = %s\n' "$(osmo_msisdn "$node" "$op_id" "$m")" "$node"
            done
        fi
        printf '\n[relay]\nport = 7890\nconnect_timeout = 10\nretry_count = 3\nretry_delay = 5\n'
    } > "$out"
}

# ── NATIVE post-patches: the ISO recipe, in ONE place ──────────────────
# What turns "the templates are substituted" into "the machine has a
# configuration that starts" on a native host (ISO, VM, start-native.sh):
#
#   sms-routing.conf  one route per embarked MS, on the local core address
#                     (the template fallback lists docker-plan addresses).
#   osmo-sgsn.cfg     "gtp local-ip", "gsup remote-ip" and the NS listen
#                     address follow the native plan.
#   osmo-msc.cfg      "hlr remote-ip" follows the HLR bind address.
#   osmo-stp.cfg      the local-ip of the ASP to the inter-STP hub.
#   pjsip.conf        the [transport-udp] block loses its docker NAT settings.
#   mobile.cfg        the GSMTAP target becomes 127.0.0.1.
#   run.sh            "trxcon <options>" and "mobile <options>" reduced to
#                     the bare binary: the start-up modules decide the options.
#
# Called on both sides, on the same tree: the ISO build ($TEMP_CONFIG then
# $ROOTFS/etc) and start-native.sh --regen (its temporary directory).
# Idempotent: replaying it on an already patched tree changes nothing.
#
#   $1  root that holds osmocom/ : $TEMP_CONFIG, $ROOTFS/etc, /etc
#   $2  operator number         (default 1)
#   $3  number of embarked MS   (default 2)
#   $4  local address of the core (default 127.0.0.2)
#   $5  node number             (default: osmo_node_id)
#   $6  WAN table to read       (default /etc/osmo-wan.conf)
#   $7  SGSN GTP local-ip       (default 192.168.<node+1>.1)
#   $8  HLR address             (default 127.0.0.2)
apply_native_post_patches() {
    local dest="${1:?apply_native_post_patches : racine manquante}"
    local op_id="${2:-1}" n_ms="${3:-2}" host_ip="${4:-127.0.0.2}"
    local node="${5:-$(osmo_node_id)}" wanf="${6:-${WAN_CONF_FILE:-/etc/osmo-wan.conf}}"
    # sgsn_gtp_ip  The SGSN and the GGSN both open a GTP socket; on the same
    #              address the SGSN dies on "Address already in use". Default:
```

```
#              the gateway of the node's private segment,
#              192.168.<node+1>.1, set by network/osmo-ip-plan.sh.
# hlr_ip       "gsup remote-ip" and the hlr "remote-ip" of the MSC
#              designate WHERE THE HLR LISTENS (the "bind ip" of
#              osmo-hlr.cfg), not the local address of the node.
local sgsn_gtp_ip="${7:-}"
local hlr_ip="${8:-127.0.0.2}"
local cfg="$dest/osmocom"
[ -d "$cfg" ] || return 0
[ "$n_ms" -ge 1 ] 2>/dev/null || n_ms=1
case "$node" in [1-9]) ;; *) node=1 ;; esac
# AFTER the node validation, never before: an empty $node would give
# 192.168.1.1, the bench LAN where the VMs and the SS7 hub live.
: "${sgsn_gtp_ip:=192.168.$((node + 1)).1}"

_write_sms_routing_native "$cfg/sms-routing.conf" \
    "$node" "$op_id" "$n_ms" "$host_ip" "$wanf"

if [ -f "$cfg/osmo-sgsn.cfg" ]; then
    sed -i \
        -e "s/^\([[:space:]]*gtp[[:space:]]\+local-ip[[:space:]]\+\).*/\1${sgsn_gtp_ip}/" \
        -e "s/^\([[:space:]]*gsup[[:space:]]\+remote-ip[[:space:]]\+\).*/\1${hlr_ip}/" \
        -e "/^[[:space:]]*bind udp local\$/,/^[[:space:]]*!\$/
         ↪  s/^\([[:space:]]*listen[[:space:]]\+\).*/\1${host_ip} 23000/" \
        "$cfg/osmo-sgsn.cfg"
fi

if [ -f "$cfg/osmo-msc.cfg" ]; then
    sed -i \
        -e "/^hlr\$/,/^!\$/ s/^\([[:space:]]*remote-ip[[:space:]]\+\).*/\1${hlr_ip}/" \
        "$cfg/osmo-msc.cfg"
fi

# ── osmo-stp.cfg: the local-ip of the ASP to the inter-STP ──────────────
# The template sets  local-ip __INTER_LOCAL_IP__  under  asp asp-to-inter,
# substituted by op_backbone_ip(N) = 172.20.0.<10+N>: right INSIDE the
# container, nonexistent on the host. An unbindable local-ip gives no
# readable error: the ASP simply never comes up (as-inter AS_DOWN).
#
# The local-ip is the source address the kernel picks to reach the hub
# (ip route get): an SCTP association bound to 127.0.0.1 towards a hub on
# a network address - even this machine's own - is refused ("Network is
# unreachable"). 127.0.0.1 only when the hub is on loopback or no route
# leads to it. The remote-ip is already resolved by the template.
#
# awk, not sed: only the local-ip that follows "asp asp-to-inter" must
# change, not those of the listen block above it, which are correct.
if [ -f "$cfg/osmo-stp.cfg" ]; then
    local inter_rip inter_lip=127.0.0.1
    inter_rip="$(awk '/^[[:space:]]*asp[[:space:]]+asp-to-inter[[:space:]]/{f=1;next}
     ↪  f&&/^[[:space:]]*remote-ip[[:space:]]/{print $2; exit} f&&/^ (asp|as) /{exit}' "$cfg/osmo-stp.cfg")"
    case "$inter_rip" in
        ""|127.*|__*) ;;   # __* : template not substituted yet
        *) inter_lip="$(ip -4 route get "$inter_rip" 2>/dev/null | awk '{for(i=1;i<NF;i++) if($i=="src")
         ↪  {print $(i+1); exit}}')"
           [ -n "$inter_lip" ] || inter_lip=127.0.0.1 ;;
    esac
    awk -v lip="$inter_lip" '
        /^[[:space:]]*asp[[:space:]]+asp-to-inter[[:space:]]/ { in_asp = 1; print; next }
        in_asp && /^[[:space:]]*local-ip[[:space:]]/ {
            sub(/local-ip[[:space:]]+.*/, "local-ip " lip); in_asp = 0; print; next
        }
        /^[[:space:]]*(as|asp|cs7|listen)[[:space:]]/ && !/asp-to-inter/ { in_asp = 0 }
        { print }
    ' "$cfg/osmo-stp.cfg" > "$cfg/osmo-stp.cfg.tmp" \
        && mv -f "$cfg/osmo-stp.cfg.tmp" "$cfg/osmo-stp.cfg"
```

```
    fi

    # ── pjsip.conf: the [transport-udp] block ──────────────────────────────
    # configs/pjsip.conf carries no placeholder: its transport block is pinned
    # on the docker plan (external_media_address / external_signaling_address
    # = 10.2.0.2, local_net = 172.20.0.0/16). external_* is a NAT option: for
    # any peer OUTSIDE local_net Asterisk rewrites the Contact and the SDP
    # with that address, which a LAN Linphone cannot reach.
    # On a flat host there is no NAT boundary: the external_* lines are
    # REMOVED (pjsip then answers with the socket address, per destination)
    # and local_net covers the whole private space, whatever the topology.
    if [ -f "$dest/asterisk/pjsip.conf" ]; then
        awk '
            /^\[transport-udp\]/ { in_t = 1; print; next }
            in_t && /^\[/        { in_t = 0 }
            in_t && /^[[:space:]]*external_(media|signaling)_address[[:space:]]*=/ { next }
            in_t && /^[[:space:]]*local_net[[:space:]]*=/ {
                if (!done) {
                    print "local_net=⟨1⟩.⟨2⟩.⟨3⟩.0/⟨4⟩"  ×5
⟨⟩ = (127,0,0,8) (10,0,0,8) (172,16,0,12) (192,168,0,16) (176,16,32,24)
                    done = 1
                }
                next
            }
            { print }
        ' "$dest/asterisk/pjsip.conf" > "$dest/asterisk/pjsip.conf.tmp" \
            && mv -f "$dest/asterisk/pjsip.conf.tmp" "$dest/asterisk/pjsip.conf"
    fi

    # ── mobile.cfg: the GSMTAP target ──────────────────────────────────────
    # The template sets  remote-host __INTER_NET_GATEWAY__  under  gsmtap,
    # substituted by 172.20.0.1 (the docker bridge gateway), which only exists
    # while a docker network is up. The other components (osmo-bts-trx,
    # osmo-pcu, osmo-bsc) already send to 127.0.0.1:4729.
    if [ -f "$cfg/mobile.cfg" ]; then
        sed -i \
            -e "/^gsmtap\$/,/^!\$/ s/^\([[:space:]]*remote-host[[:space:]]\+\).*/\1127.0.0.1/" \
            "$cfg/mobile.cfg"
    fi

    if [ -f "$cfg/run.sh" ]; then
        sed -i \
            -e 's#^[[:space:]]*\([^[:space:]]*/\)\?trxcon\([[:space:]].*\)\?$#trxcon#' \
            -e 's#^[[:space:]]*\([^[:space:]]*/\)\?mobile\([[:space:]].*\)\?$#mobile#' \
            "$cfg/run.sh"
    fi
    chmod +x "$cfg"/*.sh 2>/dev/null || true
    return 0
}

# ── Show the effective values ──────────────────────────────────────────────
_gc_show() {
    local key def perop comment val src
    printf '\n  %-13s %-26s %s\n' "REGLAGE" "VALEUR EFFECTIVE" "ORIGINE"
    printf '  %-13s %-26s %s\n' "-------" "----------------" "-------"
    while IFS='|' read -r key def perop comment; do
        [ -n "$key" ] && [ "$key" != "#SECTION" ] || continue
        val="${!key:-}"
        if [ -n "$val" ]; then src="globals.conf / env"
        elif [ "$perop" = 1 ]; then src="derive de l'operateur"; val="(auto)"
        else src="defaut"; val="$def"; fi
        printf '  %-13s %-26s %s\n' "$key" "$val" "$src"
    done < <(_gc_table)
    printf '\n'
}
```

```
# ── Multi-operator guard ──────────────────────────────────────────────
_gc_warn_perop() {
    [ "${N_OPERATORS:-1}" -gt 1 ] 2>/dev/null || return 0
    local key def perop comment figes=""
    while IFS='|' read -r key def perop comment; do
        [ "$key" != "#SECTION" ] && [ "$perop" = 1 ] && [ -n "${!key:-}" ] && figes="$figes $key"
    done < <(_gc_table)
    [ -n "$figes" ] || return 0
    printf '\n  ⚠ N_OPERATORS=%s mais ces champs par-operateur sont FIGES :%s\n' \
           "$N_OPERATORS" "$figes"
    printf '    Tous les operateurs partageront la meme valeur - collision garantie\n'
    printf '    sur la radio et/ou les identites. Videz-les dans globals.conf.\n\n'
}

# ── Executed mode ─────────────────────────────────────────────────────
if [ "${BASH_SOURCE[0]}" = "$0" ]; then
    force=0; show=0; _gc_sets=()
    for a in "$@"; do
        case "$a" in
            --force) force=1 ;;
            --show)  show=1 ;;
            -h|--help)
                sed -n '2,60p' "$0" | sed 's/^#\{0,1\} \{0,1\}//'; exit 0 ;;
            *=*)     _gc_sets+=("$a") ;;
            *) printf 'option inconnue : %s (voir --help)\n' "$a" >&2; exit 2 ;;
        esac
    done
    _gc_write "$force"
    for kv in "${_gc_sets[@]}"; do _gc_set "$kv"; done
    gc_load
    _gc_warn_perop
    [ "$show" = 1 ] && _gc_show
    exit 0
fi

# ── Sourced mode ──────────────────────────────────────────────────────
gc_load
```

2.10 /opt/GSM/osmo-operator/install.sh

9938 octets, 199 lignes → 199 lignes

```
#!/bin/bash
# ============================================================================
#  install.sh - native, module-based installation
# ============================================================================
#
#  Same architecture as start-native.sh: this file is only an engine. Each
#  step lives in install_modules/NN-<slug>.sh and follows the contract
#  described in install_modules/_lib/inst.sh.
#
#      sudo ./install.sh                everything
#      ./install.sh --list              list the steps, do nothing
#      ./install.sh --dry-run           walk through without side effects
#      sudo ./install.sh --only deps    one step only
#      sudo ./install.sh --skip build   everything except that step
#      ./install.sh --check             what is already installed
#      sudo ./install.sh --reinstall    replay EVERYTHING, even what is
#                                       already there (sources updated,
#                                       configs and shortcuts rewritten,
#                                       binaries rebuilt)
#
#  Where: GSM_ROOT (default /opt/GSM) receives sources and binaries.
#  The desktop (icons, "Lancer le banc GSM" and "multi-operator" shortcuts) is
```

```
#  the `bureau` step: it installs the SAME files as the ISO (data/desktop/).
#  New to the project? Prefer Docker: ./tools/make-docker-image.sh.
#  This native install is for people who want to work on the sources.
#
#  The package list is not duplicated here: it is EXTRACTED from the
#  Dockerfile, which stays the reference, so the two cannot diverge.
#
#  Environment: GSM_ROOT, OSMOCOM_HOME (default ~/.osmocom),
#               LOG_DIR (default /tmp/osmo-nitb-install), NO_COLOR.
# ------------------------------------------------------------------------------
set -uo pipefail

INST_TREE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
MODDIR="$INST_TREE/install_modules"
: "${GSM_ROOT:=/opt/GSM}"
: "${OSMOCOM_HOME:=$HOME/.osmocom}"
export INST_TREE GSM_ROOT OSMOCOM_HOME

# -- Options ------------------------------------------------------------------
DRY=0 ONLY="" SKIP="" ACTION=install REINSTALL=0
while [ $# -gt 0 ]; do
    case "$1" in
        --list)    ACTION=list ;;
        --check)   ACTION=check ;;
        --dry-run) DRY=1 ;;
        --reinstall) REINSTALL=1 ;;
        --only)    ONLY="${2:-}"; shift ;;
        --skip)    SKIP="${2:-}"; shift ;;
        -h|--help) sed -n '3,24p' "$0" | sed 's/^#\{1,2\} \{0,1\}//'; exit 0 ;;
        *) printf 'option inconnue : %s\n' "$1" >&2; exit 2 ;;
    esac
    shift
done

# -- Output helpers -----------------------------------------------------------
if [ -t 1 ] && [ -z "${NO_COLOR:-}" ]; then
    TTY=1; C_OK=$'\033[32m'; C_KO=$'\033[31m'; C_SK=$'\033[33m'; C_DIM=$'\033[2m'; C_Z=$'\033[0m'
else
    TTY=0; C_OK=""; C_KO=""; C_SK=""; C_DIM=""; C_Z=""
fi
begin() { if [ $TTY -eq 1 ]; then printf '[ %s..%s ] %s' "$C_DIM" "$C_Z" "$1"; else printf '[ .. ] %s\n' "$1"; fi; }
end()   { [ $TTY -eq 1 ] && printf '\r\033[K'; printf '[%s%s%s] %s' "$2" "$1" "$C_Z" "$3"
          [ -n "${4:-}" ] && printf ' %s(%s)%s' "$C_DIM" "$4" "$C_Z"; printf '\n'; }

LOGDIR="${LOG_DIR:-/tmp/osmo-nitb-install}"; mkdir -p "$LOGDIR" 2>/dev/null || true
export REINSTALL

# -- Load the modules and select the steps ------------------------------------
[ -d "$MODDIR" ] || { printf 'install_modules/ introuvable\n' >&2; exit 2; }
. "$MODDIR/_lib/inst.sh"
shopt -s nullglob; for f in "$MODDIR"/[0-9][0-9]-*.sh; do . "$f"; done; shopt -u nullglob
[ ${#INST_ORDER[@]} -eq 0 ] && { printf 'aucun module d installation\n' >&2; exit 2; }

in_csv() { case ",$1," in *",$2,"*) return 0;; esac; return 1; }
selected=()
for s in "${INST_ORDER[@]}"; do
    [ -n "$ONLY" ] && { in_csv "$ONLY" "$s" || continue; }
    [ -n "$SKIP" ] && { in_csv "$SKIP" "$s" && continue; }
    selected+=("$s")
done

# -- --list -------------------------------------------------------------------
if [ "$ACTION" = list ]; then
    printf 'Etapes - %d, dans cet ordre\n\n' "${#selected[@]}"
    for s in "${selected[@]}"; do
        printf '  %-10s %-38s %s%s\n' "$s" "${INST_DESC[$s]}" \
```

```
            "$([ "${INST_REQUIRED[$s]}" = 1 ] && echo obligatoire || echo optionnelle)" \
            "$([ -n "${INST_DEPS[$s]}" ] && echo "  apres: ${INST_DEPS[$s]}")"
    done
    printf '\nGSM_ROOT = %s\n' "$GSM_ROOT"
    exit 0
fi

# ── --check ──────────────────────────────────────────────────────────────
if [ "$ACTION" = check ]; then
    rc=0
    for s in "${selected[@]}"; do
        p="$(inst_prefix "$s")"
        if declare -F "${p}_done" >/dev/null; then
            if "${p}_done" >/dev/null 2>&1; then end " OK " "$C_OK" "${INST_DESC[$s]}" "deja installe"
            else end "----" "$C_SK" "${INST_DESC[$s]}" "a faire"; rc=1; fi
        else
            end " ?  " "$C_DIM" "${INST_DESC[$s]}" "pas de controle"
        fi
    done
    exit $rc
fi

# root: required only when a selected step asks for it
need_root=0
for s in "${selected[@]}"; do [ "${INST_ROOT[$s]}" = 1 ] && need_root=1; done
if [ $DRY -eq 0 ] && [ $need_root -eq 1 ] && [ "$(id -u)" -ne 0 ]; then
    printf 'Cette installation ecrit dans %s, /usr/local et /etc/osmocom.\n' "$GSM_ROOT" >&2
    printf 'Relancez avec sudo, ou inspectez sans rien modifier :  ./install.sh --dry-run\n' >&2
    exit 2
fi

# ── Run the steps ────────────────────────────────────────────────────────
declare -A STATE=()
nb_ok=0 nb_skip=0 nb_fail=0
for s in "${selected[@]}"; do
    p="$(inst_prefix "$s")"; log="$LOGDIR/$s.log"
    _INST_REASON=""; _INST_HINT=""

    # A prerequisite step the caller EXCLUDED (--only, --skip) is not a
    # failure: only a step that ran and FAILED blocks its dependents.
    dep_ko=""
    for d in ${INST_DEPS[$s]}; do
        in_csv "$(IFS=,; echo "${selected[*]}")" "$d" || continue
        case "${STATE[$d]:-absent}" in ok|skip) ;; *) dep_ko="$d"; break;; esac
    done
    if [ -n "$dep_ko" ]; then
        end "SKIP" "$C_SK" "${INST_DESC[$s]}" "depend de $dep_ko, non satisfaite"
        STATE[$s]=skip; nb_skip=$((nb_skip+1)); continue
    fi

    begin "${INST_DESC[$s]}"

    # --reinstall: skip the "already done?" check and redo the step. Modules
    # read REINSTALL to overwrite what they normally keep (cp -n, existing
    # checkout left as is).
    if [ "$REINSTALL" -eq 0 ] && declare -F "${p}_done" >/dev/null && "${p}_done" >>"$log" 2>&1; then
        end "SKIP" "$C_SK" "${INST_DESC[$s]}" "deja installe"
        STATE[$s]=skip; nb_skip=$((nb_skip+1)); continue
    fi

    if declare -F "${p}_check" >/dev/null; then
        "${p}_check" >>"$log" 2>&1; rc=$?
        if [ $rc -eq $INST_RC_FAIL ]; then
            end "FAIL" "$C_KO" "${INST_DESC[$s]}" "${_INST_REASON:-prerequis non satisfait}"
            [ -n "$_INST_HINT" ] && printf '       %s→ %s%s\n' "$C_DIM" "$_INST_HINT" "$C_Z"
            printf '       %sjournal : %s%s\n' "$C_DIM" "$log" "$C_Z"
```

```
            nb_fail=$((nb_fail+1)); STATE[$s]=fail
            [ "${INST_REQUIRED[$s]}" = 1 ] && { printf '\ninstallation interrompue\n'; exit 1; }
            continue
        fi
    fi

    if [ $DRY -eq 1 ]; then
        end " -- " "$C_DIM" "${INST_DESC[$s]}" "simule"; STATE[$s]=ok; continue
    fi

    "${p}_run" >>"$log" 2>&1; rc=$?
    case $rc in
        $INST_RC_DONE) end "SKIP" "$C_SK" "${INST_DESC[$s]}" "${_INST_REASON:-rien a faire}"
                       STATE[$s]=skip; nb_skip=$((nb_skip+1)); continue ;;
        $INST_RC_FAIL) end "FAIL" "$C_KO" "${INST_DESC[$s]}" "${_INST_REASON:-echec}"
                       [ -n "$_INST_HINT" ] && printf '       %s→ %s%s\n' "$C_DIM" "$_INST_HINT" "$C_Z"
                       printf '       %sjournal : %s%s\n' "$C_DIM" "$log" "$C_Z"
                       nb_fail=$((nb_fail+1)); STATE[$s]=fail
                       [ "${INST_REQUIRED[$s]}" = 1 ] && { printf '\ninstallation interrompue\n'; exit 1; }
                       continue ;;
    esac

    # Running an installer is not the same as having installed: verify afterwards.
    if declare -F "${p}_verify" >/dev/null; then
        if ! "${p}_verify" >>"$log" 2>&1; then
            end "FAIL" "$C_KO" "${INST_DESC[$s]}" "faite, mais le controle echoue : ${_INST_REASON:-}"
            [ -n "$_INST_HINT" ] && printf '       %s→ %s%s\n' "$C_DIM" "$_INST_HINT" "$C_Z"
            printf '       %sjournal : %s%s\n' "$C_DIM" "$log" "$C_Z"
            nb_fail=$((nb_fail+1)); STATE[$s]=fail
            [ "${INST_REQUIRED[$s]}" = 1 ] && { printf '\ninstallation interrompue\n'; exit 1; }
            continue
        fi
    fi

    end " OK " "$C_OK" "${INST_DESC[$s]}"; STATE[$s]=ok; nb_ok=$((nb_ok+1))
done

# ── Summary ──────────────────────────────────────────────────────────
printf '\n%d ok · %d ignores · %d echecs\n' "$nb_ok" "$nb_skip" "$nb_fail"
if [ $nb_fail -eq 0 ] && [ $DRY -eq 0 ]; then
    printf '\n  %sverifier%s   ./install.sh --check\n'          "$C_DIM" "$C_Z"
    printf '  %slancer%s     ./start-native.sh --list   puis   sudo ./start-native.sh\n\n' "$C_DIM" "$C_Z"
fi
[ $nb_fail -gt 0 ] && exit 1
exit 0
```

2.11 /opt/GSM/osmo-operator/launch.sh

38181 octets, 719 lignes → 719 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════
# launch.sh - DOUBLE-CLICK entry point of the osmo-operator bench.
#
# Brings up the complete workstation in one gesture:
#     1. wireshark    on udp/4729 (GSMTAP)         - root, only with OSMO_LAUNCH_APPS=1
#     2. linphone     the SIP softphone            - session user, only with OSMO_LAUNCH_APPS=1
#     3. firefox      on http://127.0.0.1:8080     - as soon as the dashboard answers
#     4. start.sh                                  - the bench, in the FOREGROUND
#
# The first three start in the background and cannot fail the launch: a
# missing tool is reported and the launch goes on. start.sh keeps the
# terminal: its output is what one wants to read.
#
```

```
# Three things separate a script that works in a terminal from one that works
# on double-click, and they are handled first:
#   1. NO TERMINAL. A .desktop starts without a tty: start.sh would write into
#      the void. With the systemd unit installed the launch becomes a service
#      start (notification + progress window); otherwise the script re-executes
#      itself inside a terminal window (tools/osmo-gtk-run.py --tty).
#   2. NO ROOT. The bench needs privileges (netns, systemd, captures). From an
#      icon: pkexec (graphical prompt), sudo as fallback.
#   3. THE WINDOW CLOSES BEFORE THE ERROR CAN BE READ. It is held at the end.
#
# Terminal usage (none of these detours apply):
#     sudo ./launch.sh [arguments passed to start.sh]
#
# Right-click actions (handled before any window, root through pkexec):
#     --service [args]   start osmo-banc.service (args -> OSMO_BANC_ARGS)
#     --service-suivi    same, following the unit journal (progress window)
#     --stop             stop the bench, native and container operators
#     --multi            start osmo-multi.service (multi-operator topology)
#     --multi-suivi      same, following the unit journal
#     --multi-stop       stop the multi-operator containers
#     --dashboard        open the dashboard in the browser
#     --console          attach to the bench tmux session in a window
#     --vty              open the MS VTY (telnet) in a window
#
# Environment:
#     GSMTAP_PORT (4729), OSMO_DASH_URL (http://127.0.0.1:8080),
#     OSMO_TUTORIAL_URL, OSMO_BANC_SERVICE (osmo-banc.service),
#     OSMO_MULTI_SERVICE (osmo-multi.service), OSMO_LAUNCH_LOCK,
#     OSMO_LAUNCH_APPS=1 (also start wireshark and linphone),
#     OSMO_LAUNCH_MENU=1 (keep the interactive whiptail menus of start.sh),
#     OSMO_LAUNCH_GTK=0 (no progress window, notifications only),
#     OSMO_LAUNCH_NO_SERVICE=1 (never go through the systemd unit from an icon),
#     OSMO_BANC_DIRECT=1 (run start.sh directly, bypassing the unit),
#     OSMO_LAUNCH_LOG (/var/log/osmo-launch.log, detached fallback),
#     OSMO_MS_VTY_PORT, TMUX_SESSION (calypso), HOST_PULSE_USER.
#     OSMO_LAUNCH_TERM / OSMO_TERM_TAKEN are internal re-exec flags.
# ═══════════════════════════════════════════════════════════════════════════
set -uo pipefail

DIR="$(cd "$(dirname "$(readlink -f "$0")")" && pwd)"
TARGET="$DIR/start.sh"
GSMTAP_PORT="${GSMTAP_PORT:-4729}"
DASH_URL="${OSMO_DASH_URL:-http://127.0.0.1:8080}"
DASH_PORT="${DASH_URL##*:}"
# The tutorial opens as a SECOND TAB next to the dashboard. Served by the
# dashboard rather than file://: a single copy, and the URL works the same
# from another machine of the bench. See /usr/local/bin/osmo-tutorial.
TUTO_URL="${OSMO_TUTORIAL_URL:-${DASH_URL%/}/tutorial.html}"

GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; RED='\033[0;31m'; NC='\033[0m'

[ -x "$TARGET" ] || { echo -e "${RED}start.sh introuvable ou non executable dans $DIR${NC}" >&2; exit 1; }

# ── ONE LAUNCH AT A TIME ─────────────────────────────────────────────────────
# A second launch would begin by stopping everything (one click = a fresh
# bench) and tear down the stack the first one is bringing up. Two ways of
# already being in the middle of a start-up, two tests:
#   1. another launch.sh holds the lock: a non-blocking flock on a descriptor
#      that SURVIVES the exec of start.sh, so it covers the whole start-up and
#      the kernel releases it when the process dies, even killed - nothing to
#      clean up, no stale lock after a power cut.
#   2. the osmo-banc unit is "activating": a bench started by systemd
#      (`systemctl start osmo-banc`, the "as a service" action of the icon, or
#      an image burnt with --banc) goes through no launch.sh and holds no lock.
# `--stop` is never blocked: it is the gesture that takes back control when a
# launch drags on.
```

```
LOCK_FILE="${OSMO_LAUNCH_LOCK:-/run/lock/osmo-launch.lock}"
verrou_pris() {
    command -v flock >/dev/null 2>&1 || return 0   # no flock: nothing is blocked
    mkdir -p "$(dirname "$LOCK_FILE")" 2>/dev/null || true
    exec 9>>"$LOCK_FILE" 2>/dev/null || return 0
    flock -n 9
}
banc_en_demarrage() {
    [ "$(systemctl is-active "${OSMO_BANC_SERVICE:-osmo-banc.service}" 2>/dev/null)" = activating ]
}
lancement_possible() {
    banc_en_demarrage && return 1
    verrou_pris
}
# The lock must die with the START-UP, not with the window: held by descriptor
# 9 it would otherwise live as long as the process, i.e. until someone presses
# Enter at "Fenetre maintenue ouverte" (or while attached to tmux), and a
# forgotten window would refuse every later click. It is released as soon as
# the bench is up.
verrou_liberer() { exec 9>&- 2>/dev/null || true; }

# ── 0. DIRECT ACTIONS (right-click menu): no window, talk to systemd only ────
# The report goes through a desktop notification, the only place the operator
# looks when there is no terminal. `systemctl restart` returns at the end of
# the start-up (up to TimeoutStartSec=900), which allows announcing a real
# result rather than a "started" with no follow-up.
case "${1:-}" in --service|--service-suivi|--stop|--dashboard|--console|--vty|--multi|--multi-suivi|--multi-stop)
 ↪  _bloc_direct=1 ;; *) _bloc_direct=0 ;; esac
if [ "$_bloc_direct" = "1" ]; then
    # Root WITHOUT a terminal: pkexec asks for the password in a desktop
    # window. No sudo fallback here: sudo would need a terminal, which is
    # precisely what these actions promise not to open.
    if [ "$(id -u)" -ne 0 ] && command -v pkexec >/dev/null 2>&1 \
       && [ -n "${DISPLAY:-}" ]; then
        exec pkexec env DISPLAY="${DISPLAY:-}" XAUTHORITY="${XAUTHORITY:-}" \
             "$0" "$@"
    fi
    _action="$1"; shift
    _svc="${OSMO_BANC_SERVICE:-osmo-banc.service}"
    _icon=/usr/share/osmo-operator/icons/osmo-launch.svg
    _note() {
        if command -v notify-send >/dev/null 2>&1; then
            notify-send -i "$_icon" "Banc GSM" "$1" 2>/dev/null || true
        fi
        echo -e "  ${CYAN}→${NC} $1"
    }

    # ── The container operators ──────────────────────────────────────────
    # The docker operators of the multi (osmo-operator-2, -3, the osmo-inter-stp
    # hub) run in their own cgroups (/system.slice/docker-<id>.scope), out of
    # reach of `systemctl stop osmo-banc`. osmo-multi.service propagates the
    # stop only while it is active: once "failed" (killed during start-up)
    # nothing holds the containers any more. So the unit is stopped when it
    # exists, THEN whatever remains is swept with the same filter
    # launch/start-docker.sh applies at the start of a run (a container of a
    # previous topology would escape `start-multi.sh --stop`).
    _arret_multi() {
        local _m="${OSMO_MULTI_SERVICE:-osmo-multi.service}"
        if systemctl cat "$_m" >/dev/null 2>&1; then
            timeout 200 systemctl stop "$_m" >/dev/null 2>&1 || true
            systemctl reset-failed "$_m" 2>/dev/null || true
        fi
        command -v docker >/dev/null 2>&1 || return 0
        local _c
        _c="$(docker ps -q --filter name=osmo- 2>/dev/null)" || return 0
        [ -n "$_c" ] || return 0
```

```
    _note "Arret des operateurs en conteneur..."
    # shellcheck disable=SC2086  # a list of IDs, one per line
    docker rm -f $_c >/dev/null 2>&1 || true
}

# ── Multi-operator: the icon starts the unit, and says so ──────────────
# `systemctl start osmo-multi.service` returns 0 in two cases that are
# indistinguishable from a click: a ConditionPathExists not met (the unit
# is SKIPPED, not failed - the normal case on a fresh ISO before the
# "Supplements" icon ran) and a unit already active (RemainAfterExit=yes).
# The conditions are therefore tested HERE, what is missing is named, and
# the result is announced by notification.
_svcm="${OSMO_MULTI_SERVICE:-osmo-multi.service}"
# ── Followed launch: the action plus the unit journal streaming ────────
# What the desktop launch window (osmo-gtk-run.py --progress) executes:
# `systemctl start` shows nothing, the bench writes to its unit journal.
# The journal is followed in parallel and the window reads the "[n/10]"
# markers of run.sh for its progress bar. Lock, notifications and return
# code are those of --service / --multi.
if [ "$_action" = "--service-suivi" ] || [ "$_action" = "--multi-suivi" ]; then
    if [ "$_action" = "--service-suivi" ]; then _u="$_svc"; _sub=--service; else _u="$_svcm"; _sub=--multi; fi
    journalctl -u "$_u" -f -n 0 -o cat 2>/dev/null & _j=$!
    "$0" "$_sub" "$@"; _rc=$?
    sleep 1; kill "$_j" 2>/dev/null; wait "$_j" 2>/dev/null
    exit "$_rc"
fi
if [ "$_action" = "--multi-stop" ]; then
    _note "Arret du multi-operateur..."
    _arret_multi
    systemctl reset-failed "$_svcm" 2>/dev/null || true
    _note "Multi-operateur arrete (le natif reste a osmo-banc)."
    exit 0
fi
if [ "$_action" = "--multi" ]; then
    if ! systemctl cat "$_svcm" >/dev/null 2>&1; then
        _note "Unite $_svcm absente : lancez d abord l icone « Supplements »."
        exit 1
    fi
    # The same paths as the ConditionPathExists of the unit, checked HERE
    # rather than trusting the systemctl return code.
    if [ ! -f "$DIR/launch/start-multi.sh" ]; then
        _note "start-multi.sh introuvable dans $DIR - depot incomplet."
        exit 1
    fi
    if [ ! -f /etc/osmocom/osmo-multi.conf ]; then
        _note "Pas de topologie multi-operateur (/etc/osmocom/osmo-multi.conf) : l unite serait sautee en
        ↪  silence. Lancez l icone « Supplements » (addition.sh), qui installe docker, l image et cette
        ↪  topologie."
        exit 1
    fi
    case "$(systemctl is-active "$_svcm" 2>/dev/null)" in
        active)     _note "Le multi-operateur tourne deja. Journal : journalctl -u ${_svcm%.service} -f"; exit
        ↪  0 ;;
        activating) _note "Le multi-operateur demarre deja - journalctl -u ${_svcm%.service} -f"; exit 0 ;;
    esac
    systemctl reset-failed "$_svcm" 2>/dev/null || true
    _note "Demarrage de $_svcm - comptez plusieurs minutes (images, conteneurs, HLR)."
    # `start`, not `restart`: ExecStop tears the containers down, and a
    # restart on an already running unit would cost a useless teardown.
    if systemctl start "$_svcm"; then
        # Type=oneshot + conditions: a 0 does not prove that it ran.
        if [ "$(systemctl is-active "$_svcm" 2>/dev/null)" = active ]; then
            _note "Multi-operateur en service. Journal : journalctl -u ${_svcm%.service} -f"
            exit 0
        fi
        _note "Unite sautee (condition non remplie) - systemctl status ${_svcm%.service}"
```

```
        exit 1
    fi
    _note "Echec de $_svcm - journalctl -u ${_svcm%.service} -n 80"
    exit 1
fi

# ── The dashboard ──────────────────────────────────────────────────
# Firefox FIRST, not xdg-open: it is the browser of the image, and a second
# invocation adds a tab to the existing window instead of opening another
# one. xdg-open is only the fallback.
if [ "$_action" = "--dashboard" ]; then
    _u="${OSMO_DASH_URL:-http://127.0.0.1:8080}"
    if command -v firefox >/dev/null 2>&1; then
        setsid firefox "$_u" >/dev/null 2>&1 &
    elif command -v xdg-open >/dev/null 2>&1; then
        setsid xdg-open "$_u" >/dev/null 2>&1 &
    else
        _note "Aucun navigateur - ouvrez $_u a la main."; exit 1
    fi
    exit 0
fi

# ── The MS console (VTY) ───────────────────────────────────────────
# telnet, as the Osmocom documentation says: the VTY expects a line-mode
# terminal. Opened in a GTK window of the bench (VTE terminal inside,
# tools/osmo-gtk-run.py); no favourite opens a gnome-terminal.
if [ "$_action" = "--vty" ]; then
    # 4247, or 4347 in --dsp: start.sh publishes the real port in
    # /run/osmo-banc/ms1-vty; without the file, look at who listens
    # (4347 first, then 4247).
    _p="${OSMO_MS_VTY_PORT:-}"
    [ -z "$_p" ] && [ -r /run/osmo-banc/ms1-vty ] && _p="$(tr -dc 0-9 < /run/osmo-banc/ms1-vty)"
    if [ -z "$_p" ]; then
        if ss -ltnH "( sport = :4347 )" 2>/dev/null | grep -q .; then _p=4347; else _p=4247; fi
    fi
    _c="telnet 127.0.0.1 $_p || { echo; echo 'VTY 127.0.0.1:$_p injoignable - banc arrete ?'; exit 1; }"
    exec python3 "$(dirname "$(readlink -f "$0")")/tools/osmo-gtk-run.py" --tty \
         --title "Console du mobile (VTY $_p)" -- bash -c "$_c"
    _note "Aucune fenetre possible - tapez : telnet 127.0.0.1 $_p"
    exit 1
fi

# ── The bench console ──────────────────────────────────────────────
# This one, AND ONLY THIS ONE, opens a terminal: `tmux attach` is an
# interactive session and has nowhere else to live. The bench launch
# itself no longer opens one (see "ONE CLICK = ONE SERVICE" below).
if [ "$_action" = "--console" ]; then
    _sess="${TMUX_SESSION:-calypso}"
    if ! tmux has-session -t "$_sess" 2>/dev/null; then
        _note "Pas de session « $_sess » : le banc ne tourne pas."
        exit 1
    fi
    exec python3 "$(dirname "$(readlink -f "$0")")/tools/osmo-gtk-run.py" --tty \
         --title "Console du banc (tmux $_sess)" -- tmux attach -t "$_sess"
    _note "Aucune fenetre possible - tapez : tmux attach -t $_sess"
    exit 1
fi

# ── Stop ───────────────────────────────────────────────────────────
# A stop has nothing to show: it is done here, before any window.
if [ "$_action" = "--stop" ]; then
    _etat="$(systemctl is-active "$_svc" 2>/dev/null || true)"
    if systemctl cat "$_svc" >/dev/null 2>&1; then
        _note "Arret de $_svc..."
        timeout 200 systemctl stop "$_svc" || true
    fi
```

```
        # ExecStop only runs when the unit was active or activating. When it
        # is "failed" (start.sh exited with an error) systemd has already
        # killed its cgroup, but the core (osmo-stp, hlr, msc, bsc...) lives
        # in ITS OWN units, started by run.sh through systemctl: they stay up.
        # Tear down by hand what ExecStop had no chance to tear down.
        case "$_etat" in
            active|activating|reloading) ;;   # ExecStop did the work
            *)
                [ -x "$TARGET" ] && timeout 120 "$TARGET" --stop >/dev/null 2>&1 || true
                # The two units that the second ExecStop of osmo-banc.service
                # stops: they are NOT in the bench cgroup and would survive a
                # stop through this path.
                systemctl stop osmo-egprs-web.service osmo-hlr.service 2>/dev/null || true
                ;;
        esac
        # The native stack is down; the container operators depend on none of
        # these paths (see _arret_multi).
        _arret_multi
        # Otherwise the unit stays "failed" and the next start begins from a
        # failed state.
        systemctl reset-failed "$_svc" 2>/dev/null || true
        _note "Banc arrete."
        exit 0
    fi

    if ! systemctl cat "$_svc" >/dev/null 2>&1; then
        _note "Unite $_svc absente : utilisez « Lancer le banc GSM »."
        exit 1
    fi
    # Same guard as the windowed launch: two start-ups at once tear each
    # other down.
    if ! lancement_possible; then
        _note "Un lancement est deja en cours - rien de fait. journalctl -u ${_svc%.service} -f"
        exit 1
    fi
    # The right-click options (--dsp...) reach the unit as in
    # banc_unit_restart: same mechanism, same scope (until reboot).
    if [ $# -gt 0 ]; then
        systemctl set-environment OSMO_BANC_ARGS="$*" 2>/dev/null || true
    else
        systemctl unset-environment OSMO_BANC_ARGS 2>/dev/null || true
    fi
    systemctl reset-failed "$_svc" 2>/dev/null || true
    _note "Demarrage de $_svc - comptez plusieurs minutes."
    if systemctl restart "$_svc"; then
        _note "Banc demarre. Console : tmux attach -t calypso"
        exit 0
    fi
    _note "Echec de $_svc - journalctl -u ${_svc%.service} -n 80"
    exit 1
fi

# ── ONE CLICK = ONE SERVICE; A SECOND CLICK = A NOTIFICATION THAT SAYS WHY ──
# From an icon (neither stdin nor stdout on a terminal) the launch goes
# through the service: systemd brings the bench up, a progress window or a
# notification announces the result, no terminal opens. This is exactly the
# "start the bench as a service" right-click action: a single path.
#
# A second click must not open anything either, but it must be SEEN: it
# answers with a desktop notification.
#
# A manual launch from a terminal keeps that terminal; the boxed lock warning
# further down is displayed there.
if { [ ! -t 0 ] || [ ! -t 1 ]; } \
   && [ "${OSMO_LAUNCH_TERM:-0}" != "1" ] && [ "${OSMO_TERM_TAKEN:-0}" != "1" ] \
   && [ "${OSMO_LAUNCH_NO_SERVICE:-0}" != "1" ] \
```

```
   && systemctl cat "${OSMO_BANC_SERVICE:-osmo-banc.service}" >/dev/null 2>&1; then
    if lancement_possible; then
        # WITH A WINDOW, WITHOUT A SHELL: the launch is shown in the bench
        # progress window (tools/osmo-gtk-run.py --progress: bar, step,
        # collapsed console). OSMO_LAUNCH_GTK=0 falls back to notifications.
        _gtk="$(dirname "$(readlink -f "$0")")/tools/osmo-gtk-run.py"
        if [ "${OSMO_LAUNCH_GTK:-1}" != 0 ] && [ -n "${DISPLAY:-}" ] && [ -x "$_gtk" ]; then
            # The lock is released HERE: the --service started inside the
            # window (as root) takes it again; held by the window, it would
            # refuse that very launch.
            verrou_liberer
            exec python3 "$_gtk" --progress --root --title "Banc GSM 2G" \
                 --icon /usr/share/osmo-operator/icons/osmo-launch.svg -- "$0" --service-suivi "$@"
        fi
        # The lock taken here survives the exec: the same process holds the
        # start-up under its other name.
        exec "$0" --service "$@"
    fi
    # Lock held: say so, and stop here. No window.
    if command -v notify-send >/dev/null 2>&1; then
        notify-send -i /usr/share/osmo-operator/icons/osmo-launch.svg "Banc GSM" \
            "Un lancement est deja en cours. Rien de fait - suivez-le avec journalctl -u osmo-banc -f." \
            2>/dev/null || true
    fi
    echo "Un lancement est deja en cours - rien de fait." >&2
    exit 1
fi

# ── 1. A terminal, if there is none - OR if it is not ours ──────────────
# The test is the presence of a TTY on stdin AND stdout, not $DISPLAY: that is
# the real difference between "run by hand" and "run from an icon".
# OSMO_LAUNCH_TERM marks the re-exec round so as not to loop.
# An INHERITED tty is not ours: called by another launcher (start-multi.sh,
# which brings the native stack up BEFORE its containers) start.sh would write
# over its output. OSMO_TERM_TAKEN=1 says "this terminal is already busy": a
# window of our own is opened, tty or not.
_lt_besoin=0
{ [ ! -t 0 ] || [ ! -t 1 ]; } && _lt_besoin=1
[ "${OSMO_TERM_TAKEN:-0}" = "1" ] && _lt_besoin=1
if [ "$_lt_besoin" = "1" ] && [ "${OSMO_LAUNCH_TERM:-0}" != "1" ]; then
    export OSMO_LAUNCH_TERM=1
    unset OSMO_TERM_TAKEN            # the window opened here IS ours
    # The window is the bench one (GTK + VTE terminal, tools/osmo-gtk-run.py).
    exec python3 "$(dirname "$(readlink -f "$0")")/tools/osmo-gtk-run.py" --tty \
         --title "Banc GSM" -- "$0" "$@"
    # NO window possible. The caller's terminal is not taken over (that is
    # the failure this guard prevents): detach, and send the output to a log
    # rather than into the void.
    _lt_log="${OSMO_LAUNCH_LOG:-/var/log/osmo-launch.log}"
    echo -e "${YELLOW}Aucun emulateur de terminal - lancement detache, journal : ${_lt_log}${NC}" >&2
    exec setsid "$0" "$@" </dev/null >>"$_lt_log" 2>&1
fi

# The flag has done its job: REMOVE it from the environment, or it would reach
# start.sh and every descendant that calls this script back, which would then
# believe it already has its window. The value stays in a shell variable, for
# holding the window at the end.
_own_window=0
[ "${OSMO_LAUNCH_TERM:-0}" = "1" ] && _own_window=1
unset OSMO_LAUNCH_TERM

# ── 2. Privileges ──────────────────────────────────────────────────────
if [ "$(id -u)" -ne 0 ]; then
    if command -v pkexec >/dev/null 2>&1 && [ -n "${DISPLAY:-}" ]; then
        # pkexec scrubs the environment: DISPLAY and XAUTHORITY are passed
        # again by hand, otherwise nothing graphical can open afterwards.
```

```
        exec pkexec env DISPLAY="${DISPLAY:-}" XAUTHORITY="${XAUTHORITY:-}" \
             OSMO_LAUNCH_TERM="$_own_window" "$0" "$@"
    fi
    command -v sudo >/dev/null 2>&1 && exec sudo -E "$0" "$@"
    echo -e "${RED}Root requis et ni pkexec ni sudo disponibles.${NC}" >&2
    exit 1
fi

# ── The user of the graphical session ──────────────────────────────
# Same criterion as helpers/prepare_host.sh and launch/start-docker.sh: the
# account NAME says nothing, the presence of its pulse socket does. On the
# --desktop ISO the session is opened as root (gdm3 AutomaticLogin=root), so
# "root" is a valid answer here, not an error.
session_user() {
    local u uid sock
    for u in "${HOST_PULSE_USER:-}" "${SUDO_USER:-}" "$(logname 2>/dev/null || true)"; do
        [ -n "$u" ] || continue
        uid="$(id -u "$u" 2>/dev/null)" || continue
        [ -S "/run/user/${uid}/pulse/native" ] && { echo "$u"; return 0; }
    done
    for sock in /run/user/*/pulse/native; do
        [ -S "$sock" ] || continue
        uid="${sock#/run/user/}"; uid="${uid%%/*}"
        u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
        [ -n "$u" ] && { echo "$u"; return 0; }
    done
    echo "${SUDO_USER:-$(id -un)}"
}
GUI_USER="$(session_user)"
GUI_UID="$(id -u "$GUI_USER" 2>/dev/null || echo 0)"
GUI_HOME="$(getent passwd "$GUI_USER" 2>/dev/null | cut -d: -f6)"
export DISPLAY="${DISPLAY:-:0}"
export XAUTHORITY="${XAUTHORITY:-${GUI_HOME:-/root}/.Xauthority}"

# Starts a graphical application under the session account. `setsid` detaches
# it: it survives the closing of this terminal, and its return code cannot
# bring launch.sh down.
gui_run() {
    setsid sudo -u "$GUI_USER" \
        env DISPLAY="$DISPLAY" XAUTHORITY="$XAUTHORITY" \
            DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/${GUI_UID}/bus" \
        "$@" </dev/null >/dev/null 2>&1 &
    disown 2>/dev/null || true
}

# ── The 2x2 tiling ─────────────────────────────────────────────────
# Each application takes ONE QUARTER of the screen:
#
#        +---------------------+---------------------+
#        |  bench terminal     |  firefox / dashboard|
#        |  (start.sh)         |  127.0.0.1:8080     |
#        +---------------------+---------------------+
#        |  wireshark          |  linphone           |
#        |  udp/4729 GSMTAP    |  extensions 100/200 |
#        +---------------------+---------------------+
#
# wmctrl talks EWMH, which Mutter exposes under X11 (the ISO forces X11:
# WaylandEnable=false in gdm3/custom.conf). Under Wayland nothing answers and
# the tiling is simply ignored, without breaking the launch.
#
# The maximized states are removed BEFORE setting the geometry: a maximized
# window accepts the order and ignores it.
tile_windows() {
    command -v wmctrl  >/dev/null 2>&1 || { echo -e "  ${YELLOW}○${NC} wmctrl absent - fenetres non pavees";
     ↪  return 0; }
```

```
    command -v xdpyinfo >/dev/null 2>&1 || { echo -e "  ${YELLOW}○${NC} xdpyinfo absent - fenetres non pavees";
     ↪  return 0; }

    local sw sh qw qh
    sw="$(xdpyinfo 2>/dev/null | awk '/dimensions:/{split($2,a,"x"); print a[1]; exit}')"
    sh="$(xdpyinfo 2>/dev/null | awk '/dimensions:/{split($2,a,"x"); print a[2]; exit}')"
    [ -n "$sw" ] && [ -n "$sh" ] || return 0
    qw=$(( sw / 2 )); qh=$(( sh / 2 ))

    # $1 = WM_CLASS pattern (case-insensitive), $2 = x, $3 = y
    place() {
        local pat="$1" x="$2" y="$3" i id
        for i in $(seq 1 90); do
            id="$(wmctrl -l -x 2>/dev/null | grep -iE "$pat" | awk '{print $1; exit}')"
            [ -n "$id" ] && break
            sleep 1
        done
        [ -n "$id" ] || return 0
        wmctrl -i -r "$id" -b remove,maximized_vert,maximized_horz 2>/dev/null || true
        wmctrl -i -r "$id" -e "0,${x},${y},${qw},${qh}" 2>/dev/null || true
    }

    place 'gnome-terminal|xfce4-terminal|konsole|xterm'  0    0
    place 'firefox|navigator'                           "$qw" 0
    place 'wireshark'                                    0   "$qh"
    place 'linphone'                                    "$qw" "$qh"
}

# ── The bench is a systemd unit when it exists ──────────────────────────
# services/osmo-banc.service starts the bench without a terminal and keeps it
# in the tmux session "calypso" (installed but not enabled at boot: this
# script or the icon starts it). When the unit is installed this script does
# not run start.sh itself: it (re)starts the unit, opens the desktop
# applications, then ATTACHES to tmux in its terminal. Running start.sh by
# hand on top would kill the service processes behind systemd's back.
#
# The arguments (--dsp, --op 2, --regen...) reach the unit through
# OSMO_BANC_ARGS (systemctl set-environment: until reboot, then
# /etc/default/osmo-banc applies). Without arguments a leftover is cleared.
#
# OSMO_BANC_DIRECT=1 ./launch.sh bypasses the unit (manual debugging).
BANC_UNIT="${OSMO_BANC_SERVICE:-osmo-banc.service}"
banc_unit_present() {
    [ "${OSMO_BANC_DIRECT:-0}" = "1" ] && return 1
    command -v systemctl >/dev/null 2>&1 || return 1
    [ -d /run/systemd/system ] || return 1
    systemctl cat "$BANC_UNIT" >/dev/null 2>&1
}
# The stop lives at the top of the file ("Stop" block), before any window:
# `launch.sh --stop` never reaches this point.
# Follows the unit journal during the start-up: the output of start.sh /
# run.sh, in this terminal.
banc_unit_restart() {
    if [ $# -gt 0 ]; then
        systemctl set-environment OSMO_BANC_ARGS="$*" 2>/dev/null || true
        echo -e "  ${CYAN}→${NC} options : ${CYAN}$*${NC} (OSMO_BANC_ARGS, jusqu au reboot)"
    else
        systemctl unset-environment OSMO_BANC_ARGS 2>/dev/null || true
    fi
    echo -e "  ${CYAN}→${NC} redemarrage de ${BANC_UNIT} (un clic = un banc neuf)"
    journalctl -u "${BANC_UNIT%.service}" -f -n 0 -o cat 2>/dev/null &
    local _jpid=$!
    systemctl restart "$BANC_UNIT"; local rc=$?
    sleep 1; kill "$_jpid" 2>/dev/null; wait "$_jpid" 2>/dev/null
    if [ "$rc" -ne 0 ]; then
        echo -e "  ${RED}✗${NC} ${BANC_UNIT} en echec (rc=$rc) - journalctl -u ${BANC_UNIT%.service} -n 80"
```

```
        systemctl status --no-pager -n 20 "$BANC_UNIT" 2>/dev/null || true
    else
        echo -e "  ${GREEN}✓${NC} banc demarre par ${BANC_UNIT}"
    fi
    return "$rc"
}

# The lock (see its block at the top of the file). Here and not earlier:
# `--stop` must pass, and privileges must be settled - a flock taken before
# the sudo re-exec would be held by a process that disappears at once.
if [ "${1:-}" != "--stop" ] && ! lancement_possible; then
    echo
    echo -e "${YELLOW}+------------------------------------------------------------+${NC}"
    echo -e "${YELLOW}|  UN LANCEMENT EST DEJA EN COURS                            |${NC}"
    echo -e "${YELLOW}|  Cette fenetre ne lancera RIEN.                            |${NC}"
    echo -e "${YELLOW}+------------------------------------------------------------+${NC}"
    echo
    echo -e "  Le banc met plusieurs minutes a monter (QEMU, Calypso, mobiles)."
    echo -e "  Un second lancement ARRETERAIT le premier : un clic = un banc"
    echo -e "  neuf, et le neuf commence par demonter celui d avant."
    echo
    echo -e "  Suivre le demarrage   : ${CYAN}journalctl -u osmo-banc -f${NC}"
    echo -e "  Console du banc       : ${CYAN}tmux attach -t calypso${NC}"
    echo -e "  Vraiment recommencer  : clic droit sur l icone, ${CYAN}Arreter le banc${NC},"
    echo -e "                          puis relancer."
    echo
    command -v notify-send >/dev/null 2>&1 && \
        notify-send -i /usr/share/osmo-operator/icons/osmo-launch.svg "Banc GSM" \
            "Un lancement est deja en cours - cette fenetre ne fera rien." 2>/dev/null || true
    if [ "$_own_window" = "1" ]; then
        echo "Entree pour fermer."
        read -r _ || true
    fi
    exit 1
fi

# ── Stop everything before relaunching ──────────────────────────────
# One click = a FRESH bench: the Osmocom daemons do not re-read their
# configuration while running (not even on SIGHUP), so a launch on top of a
# running bench would change almost nothing. The NATIVE stack is stopped
# (start.sh --stop, which tears radio and core down cleanly). Nothing here
# may fail the launch, hence the || true.
tout_arreter() {
    # With the unit, the stop is done by `systemctl restart` further down.
    banc_unit_present && return 0
    echo -e "  ${CYAN}→${NC} arret du banc en place (un clic = un banc neuf)"
    if [ -x "$DIR/start.sh" ]; then
        timeout 120 "$DIR/start.sh" --stop >/dev/null 2>&1 || true
        echo -e "    ${GREEN}✓${NC} pile native arretee"
    fi
    # The containers are NOT touched here: launch/start-docker.sh already does
    # `docker rm -f $(docker ps -aq --filter name=osmo-)` at the start of a
    # run, and a click on the red phone, which only brings the native stack
    # up, must not sweep the multi-operator containers next to it.
    # The audio bridges survive the containers (setsid): without this a relay
    # would pile up at every relaunch and the sound would double.
    pkill -f 'paplay --server=tcp:' 2>/dev/null || true
}

tout_arreter

# ── Are the operating tools wanted? ─────────────────────────────────
# OSMO_LAUNCH_APPS=0 (default): neither wireshark nor linphone. They are tools
# one opens WHEN needed, from their own icon; they are not part of the
# start-up. It is also what start-multi.sh asks for when it triggers this
# launcher to bring the native stack back up. OSMO_LAUNCH_APPS=1 opens them.
```

```
#
# FIREFOX IS NOT CONCERNED, deliberately: the browser carries the native
# dashboard and the tutorial, and start-multi.sh ADDS the container dashboard
# tabs to it. Firefox does not duplicate itself: a second invocation adds
# tabs to the existing window.
OSMO_LAUNCH_APPS="${OSMO_LAUNCH_APPS:-0}"
if [ "$OSMO_LAUNCH_APPS" != "1" ]; then
    echo -e "  ${CYAN}i${NC} wireshark et linphone non lances (OSMO_LAUNCH_APPS=1 pour les avoir)"
fi

# ── 1. WIRESHARK on udp/4729 ──────────────────────────────────────────
# As ROOT, not as $GUI_USER: capturing on `any` needs CAP_NET_RAW. -k starts
# the capture at once (otherwise it opens on the interface chooser and
# nothing is captured until a click). "not requested" and "not installed" are
# reported separately.
if [ "$OSMO_LAUNCH_APPS" != "1" ]; then
    echo -e "  ${CYAN}○${NC} wireshark : non demande (mode banc seul)"
elif command -v wireshark >/dev/null 2>&1; then
    setsid wireshark -k -i any -f "udp port ${GSMTAP_PORT}" \
        </dev/null >/dev/null 2>&1 &
    disown 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} wireshark : capture GSMTAP sur ${CYAN}udp/${GSMTAP_PORT}${NC}"
else
    echo -e "  ${YELLOW}○${NC} wireshark absent - capture GSMTAP ignoree"
fi

# ── 2. LINPHONE ───────────────────────────────────────────────────────
# The SIP account is pre-provisioned (configs/linphonerc, installed by the ISO
# build). Linphone does NOT re-read its config while running: if it is
# already up it is left as is rather than cutting a call in progress.
if [ "$OSMO_LAUNCH_APPS" != "1" ]; then
    echo -e "  ${CYAN}○${NC} linphone : non demande (mode banc seul)"
elif command -v linphone >/dev/null 2>&1; then
    if pgrep -x linphone >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} linphone : deja lance (laisse en place)"
    else
        gui_run linphone
        echo -e "  ${GREEN}✓${NC} linphone : lance (user=${CYAN}${GUI_USER}${NC})"
    fi
else
    echo -e "  ${YELLOW}○${NC} linphone absent"
fi

# ── 3. FIREFOX on the dashboard, WHEN IT ANSWERS ──────────────────────
# Opening the URL at once would give an error page (the dashboard is not up
# yet) that the user would have to reload by hand. The port is polled in the
# background, up to 180 s.
if command -v firefox >/dev/null 2>&1; then
    (
        for _i in $(seq 1 180); do
            if (exec 3<>"/dev/tcp/127.0.0.1/${DASH_PORT}") 2>/dev/null; then
                exec 3>&- 2>/dev/null
                # Two URLs on the same command line = two TABS in the same
                # window. A second `firefox <url>` would race with the browser
                # start-up and sometimes open a second window.
                gui_run firefox "$DASH_URL" "$TUTO_URL"
                exit 0
            fi
            sleep 1
        done
    ) &
    disown 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} firefox : ouvrira ${CYAN}${DASH_URL}${NC} des que le dashboard repond"
else
    echo -e "  ${YELLOW}○${NC} firefox absent - dashboard a ouvrir a la main sur ${DASH_URL}"
fi
```

```
# The tiling runs in the background: each window is placed as soon as it
# appears, without delaying the bench start-up.
tile_windows &
disown 2>/dev/null || true

# ── 4. THE BENCH, IN THE FOREGROUND ──────────────────────────────────
echo
cd "$DIR" || exit 1

# ── No input ─────────────────────────────────────────────────────────
# start.sh opens menu_interactif() (startup_scripts/06-banc-dsp-menu.sh):
# profile, SS7 node, operator, inter-STP - whiptail questions it skips only
# when no terminal is present ([ -t 0 ]), and launch.sh has just opened one.
# NO_MENU=1 (startup_scripts/06-banc-dsp-menu.sh and 10-identite-noeud.sh)
# makes the menus return without asking: node 1, operator 1, local hub, as
# launch/start-docker.sh already sets for its containers.
# OSMO_LAUNCH_MENU=1 ./launch.sh gives the questions back.
if banc_unit_present; then
    banc_unit_restart "$@"
    rc=$?
    # The bench runs in tmux: attach to it when a terminal is available.
    # Ctrl-b d detaches; the bench goes on without this terminal.
    if [ "$rc" -eq 0 ] && [ -t 1 ] && tmux has-session -t "${TMUX_SESSION:-calypso}" 2>/dev/null; then
        echo -e "  ${CYAN}connexion a la session tmux « ${TMUX_SESSION:-calypso} »  (Ctrl-b d pour detacher)${NC}"
        sleep 1
        verrou_liberer   # otherwise the tmux client would hold it while attached
        exec tmux attach -t "${TMUX_SESSION:-calypso}"
    fi
elif [ "${OSMO_LAUNCH_MENU:-0}" = "1" ]; then
    "$TARGET" "$@"
    rc=$?
else
    NO_MENU=1 "$TARGET" "$@"
    rc=$?
fi

verrou_liberer

echo
if [ "$rc" -eq 0 ]; then
    echo -e "${GREEN}=== start.sh termine (code 0) ===${NC}"
else
    echo -e "${RED}=== start.sh a ECHOUE (code $rc) ===${NC}"
fi
# Only in a window opened by this script: in a terminal, returning at once is
# the expected behaviour.
if [ "$_own_window" = "1" ]; then
    echo "Fenetre maintenue ouverte - Entree pour fermer."
    read -r _ || true
fi
exit "$rc"
```

2.12 /opt/GSM/osmo-operator/set_stp_ip.sh

26764 octets, 524 lignes → 524 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════
# set_stp_ip.sh - gives THIS VM its SS7 address on the bridged network
#
# One ISO, two roles. The script first asks WHICH role, then the addresses,
# because they differ on each side:
#
```

```
#   inter-STP (hub)      its bridge IP, plus the IP of every osmo-operator-N
#   osmo-operator-N      its bridge IP, plus the IP of the inter-STP
#
#   osmo-operator-1  192.168.1.11 ─┐
#   osmo-operator-2  192.168.1.12 ─┼─ M3UA/SCTP 2908 ─► inter-STP 192.168.1.10
#   osmo-operator-3  192.168.1.13 ─┘                     PC 0.0.0
#
# Bridged, not host-only: the image's configs target 192.168.56.1 (VirtualBox
# host-only) or 172.20.0.10 (docker network); neither exists when the VMs are
# bridged on the LAN (192.168.1.X), so the ASPs must be re-addressed.
#
# What the script writes
#   hub       /etc/osmocom/osmo-stp-interop.cfg   local-ip = its bridge IP
#             /etc/osmo-wan.conf                  the operator table
#             /etc/osmo-role                      OSMO_ROLE=interstp
#   operator  /etc/osmocom/osmo-stp.cfg           local-ip = its own
#                                                 remote-ip = the hub's
#             /etc/osmocom/osmo-{msc,bsc}.cfg     WAN-plan point codes
#             /etc/osmo-wan.conf, /etc/osmo-role
#
# Idempotent: re-running with other addresses replaces them, nothing to clean up.
# Environment: CONF_DIR (default /etc/osmocom), ROLE_FILE (default /etc/osmo-role)
#
# Usage:
#   sudo ./set_stp_ip.sh                        menu: role, then addresses
#   sudo ./set_stp_ip.sh --inter --ip 192.168.1.10 --ips "192.168.1.11 192.168.1.12"
#   sudo ./set_stp_ip.sh --operator --node 1 --ip 192.168.1.11 --hub-ip 192.168.1.10
#   sudo ./set_stp_ip.sh --show                 current state, changes nothing
#
#   --inter | --operator   role (otherwise: asked)
#   --ip ADDRESS           bridge IP of THIS machine
#   --ips "a b c"          hub: operator IPs, in order
#   --hub-ip ADDRESS       operator: IP of the inter-STP
#   --node N               operator: its node number (1-9)
#   --op K                 HOW MANY operators (PLMN) this node carries (default 1)
#   --op-index K           WHICH of those operators is this node's, in the point
#                          code 1.<node><op>.<role> (1-9, not above --op; default 1)
#   --iface NAME           interface to put the address on (default: detected)
#   --port N               M3UA port (default 2908)
#   --conf-dir DIR         configuration directory (default /etc/osmocom)
#   --no-ip                leave the network configuration alone
#   --defaut               the bench configuration without any question: role
#                          deduced, bridge IP deduced, hub 172.20.0.10, node read
#                          from /etc/osmo-role or the WAN table.
#   --restart              restart the stack after rewriting
#   --dry-run              show what would be written
# ═══════════════════════════════════════════════════════════════════════════
set -uo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
CONF_DIR="${CONF_DIR:-/etc/osmocom}"
ROLE_FILE="${ROLE_FILE:-/etc/osmo-role}"
INTEROP_CFG="${CONF_DIR}/osmo-stp-interop.cfg"

# wan_detect_local_ip, _wan_ask, the WAN table and its validation live in
# network/wan-nodes.sh, shared with the launchers (same interface, same table).
# shellcheck source=network/wan-nodes.sh
. "$HERE/network/wan-nodes.sh"

ROLE=""
SELF_IP=""
HUB_IP=""
IPS=""
```

```
NODE=""
OPS=1
# --op COUNTS the operators carried by this node (it sizes the firewall, the
# hub's "as" blocks and the WAN table); --op-index says WHICH one is this node's.
OP_INDEX=1
IFACE=""
PORT=2908
SET_IP=1
RESTART=0
# ── The bench configuration without any question (--defaut) ──────────────
# Every answer is deduced from what the machine already knows about itself:
#   role      interstp if its address IS the hub's, operator otherwise;
#   its IP    the bridge one, detected;
#   the hub   172.20.0.10 (osmo-inter-stp, docker backbone);
#   the node  /etc/osmo-role, then the WAN table, then 1.
# Every value stays overridable: --defaut --node 3 wins over the deduction.
USE_DEFAULTS=0
DEFAULT_HUB_IP="172.20.0.10"
DRY=0
SHOW=0

while [ $# -gt 0 ]; do
    case "$1" in
        --inter|--interstp|--hub)  ROLE="interstp" ;;
        --operator|--op-node)      ROLE="operator" ;;
        --role)     ROLE="${2:-}"; shift ;;
        --role=*)   ROLE="${1#*=}" ;;
        --ip)       SELF_IP="${2:-}"; shift ;;
        --ip=*)     SELF_IP="${1#*=}" ;;
        --hub-ip)   HUB_IP="${2:-}"; shift ;;
        --hub-ip=*) HUB_IP="${1#*=}" ;;
        --ips)      IPS="${2:-}"; shift ;;
        --ips=*)    IPS="${1#*=}" ;;
        --node)     NODE="${2:-}"; shift ;;
        --node=*)   NODE="${1#*=}" ;;
        --op)       OPS="${2:-1}"; shift ;;
        --op=*)     OPS="${1#*=}" ;;
        --ops)      OPS="${2:-1}"; shift ;;
        --ops=*)    OPS="${1#*=}" ;;
        --op-index)   OP_INDEX="${2:-1}"; shift ;;
        --op-index=*) OP_INDEX="${1#*=}" ;;
        --iface)    IFACE="${2:-}"; shift ;;
        --iface=*)  IFACE="${1#*=}" ;;
        --port)     PORT="${2:-2908}"; shift ;;
        --port=*)   PORT="${1#*=}" ;;
        --conf-dir)   CONF_DIR="${2:-}"; INTEROP_CFG="${CONF_DIR}/osmo-stp-interop.cfg"; shift ;;
        --conf-dir=*) CONF_DIR="${1#*=}"; INTEROP_CFG="${CONF_DIR}/osmo-stp-interop.cfg" ;;
        --defaut|--default) USE_DEFAULTS=1 ;;
        --no-ip)    SET_IP=0 ;;
        --restart)  RESTART=1 ;;
        --dry-run)  DRY=1 ;;
        --show)     SHOW=1 ;;
        -h|--help)  sed -n '2,53p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; exit 2 ;;
    esac
    shift
done

banner() {
    echo -e "${CYAN}${BOLD}"
    echo "╔══════════════════════════════════════════════════════════════════╗"
    echo "║   set_stp_ip.sh - adressage SS7 sur le reseau en pont            ║"
    echo "╚══════════════════════════════════════════════════════════════════╝"
    echo -e "${NC}"
}
```

```
is_ipv4() {
    local ip="$1" o
    [[ "$ip" =~ ^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$ ]] || return 1
    for o in ${ip//./ }; do [ "$o" -le 255 ] || return 1; done
    return 0
}

need_ipv4() {  # $1=value  $2=what it is
    is_ipv4 "$1" || { echo -e "${RED}$2 : '$1' n'est pas une IPv4${NC}" >&2; exit 2; }
}

# ── This VM's bridge address ──────────────────────────────────────
# wan_detect_local_ip prefers the VirtualBox host-only range (192.168.56-63),
# right for the bench but not here: a bridged VM sits on the host's LAN. Look
# first for a private LAN address that is neither the VirtualBox NAT (10.0.2.x),
# a docker alias (172.20.x) nor host-only; fall back to wan_detect_local_ip
# only when none is found.
bridge_ip() {
    local addrs a
    addrs="$(ip -4 -o addr show scope global 2>/dev/null | awk '{print $4}' | cut -d/ -f1)"
    for a in $addrs; do
        case "$a" in
            192.168.5[6-9].*|192.168.6[0-3].*) ;;   # host-only: not the bridge
            192.168.*) printf '%s' "$a"; return 0 ;;
        esac
    done
    for a in $addrs; do
        case "$a" in
            10.0.2.*|172.20.*|127.*|169.254.*) ;;
            10.*|172.1[6-9].*|172.2[0-9].*|172.3[01].*) printf '%s' "$a"; return 0 ;;
        esac
    done
    wan_detect_local_ip 2>/dev/null || return 1
}

# The interface that already carries this /24 is the one on the target segment;
# otherwise the first global interface.
iface_for() {  # $1=ip
    local ip="$1" f
    f="$(ip -4 -o addr show scope global 2>/dev/null \
         | awk -v pfx="${ip%.*}." '$4 ~ "^"pfx {print $2; exit}')"
    [ -z "$f" ] && f="$(ip -4 -o addr show scope global 2>/dev/null | awk 'NR==1{print $2}')"
    printf '%s' "$f"
}

ensure_ip() {  # $1=ip - adds it when missing
    local ip="$1" dev
    [ "$SET_IP" = "1" ] || { echo -e "  ${YELLOW}-${NC} --no-ip : reseau inchange"; return 0; }
    if ip -4 addr show 2>/dev/null | grep -q "inet ${ip}/"; then
        echo -e "  ${GREEN}✓${NC} ${CYAN}${ip}${NC} deja presente sur cette machine"
        return 0
    fi
    dev="${IFACE:-$(iface_for "$ip")}"
    if [ "$DRY" = "1" ]; then
        echo -e "  [dry-run] ip addr add ${ip}/24 dev ${dev:-?}"; return 0
    fi
    if [ -n "$dev" ] && ip addr add "${ip}/24" dev "$dev" 2>/dev/null; then
        echo -e "  ${GREEN}✓${NC} ${CYAN}${ip}/24${NC} posee sur ${dev}"
    else
        echo -e "  ${YELLOW}⚠${NC} impossible de poser ${ip} (interface ${dev:-?})"
        echo -e "     Tant qu'elle manque, aucun ASP ne peut s'attacher."
    fi
}

write_role() {  # $1=role  $2=hub-ip  $3=node (empty for the hub)
    if [ "$DRY" = "1" ]; then
```

```
        echo -e "  [dry-run] ${ROLE_FILE} : OSMO_ROLE=$1 OSMO_HUB_IP=$2 ${3:+OSMO_WAN_NODE=$3}"
        return 0
    fi
    {
        printf '# %s - genere par set_stp_ip.sh\n' "$ROLE_FILE"
        printf 'OSMO_ROLE=%s\n' "$1"
        [ -n "${3:-}" ] && printf 'OSMO_WAN_NODE=%s\n' "$3"
        printf 'OSMO_HUB_IP=%s\n' "$2"
    } > "$ROLE_FILE"
    echo -e "  ${GREEN}✓${NC} ${CYAN}${ROLE_FILE}${NC}"
}

role_of_file() { awk -F= '/^OSMO_ROLE=/{gsub(/[ \r\t]/,"",$2); print $2}' "$ROLE_FILE" 2>/dev/null | tail -1; }
hub_of_file()  { awk -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2); print $2}' "$ROLE_FILE" 2>/dev/null | tail -1; }
node_of_file() { awk -F= '/^OSMO_WAN_NODE=/{gsub(/[ \r\t]/,"",$2); print $2}' "$ROLE_FILE" 2>/dev/null | tail -1;
 ↪  }

# ═══════════════════════════════════════════════════════════════════
# --show: what this machine currently believes it is
# ═══════════════════════════════════════════════════════════════════
if [ "$SHOW" = "1" ]; then
    banner
    r="$(role_of_file)"; h="$(hub_of_file)"; n="$(node_of_file)"
    echo -e "  ${BOLD}Role${NC}      : ${CYAN}${r:-inconnu}${NC}${n:+   noeud ${CYAN}${n}${NC}}"
    echo -e "  ${BOLD}IP de pont${NC}: ${CYAN}$(bridge_ip || echo '-')${NC}"
    echo -e "  ${BOLD}Inter-STP${NC} : ${CYAN}${h:--}${NC}"
    echo ""
    if [ -r "$INTEROP_CFG" ]; then
        echo -e "  ${BOLD}$(basename "$INTEROP_CFG")${NC}"
        awk '/^ *listen m3ua/{f=1} f&&/^ *(local-ip|listen)/{print "    "$0} f&&/^!/{f=0}' "$INTEROP_CFG"
    fi
    for f in "$CONF_DIR/osmo-stp.cfg" "$CONF_DIR/osmo-msc.cfg" "$CONF_DIR/osmo-bsc.cfg"; do
        [ -r "$f" ] || continue
        pc="$(awk '/^cs7 instance/{c=1} c && /^ *point-code /{print $2; exit}' "$f")"
        lo="$(awk '/asp asp-to-inter/{f=1} f&&/local-ip/{print $2; exit}' "$f")"
        re="$(awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}' "$f")"
        printf '  %-16s PC %-10s local-ip %-16s remote-ip %s\n' \
            "$(basename "$f")" "${pc:--}" "${lo:--}" "${re:--}"
    done
    echo ""
    [ -r "$WAN_CONF_FILE" ] && { echo -e "  ${BOLD}${WAN_CONF_FILE}${NC}"; sed 's/^/    /' "$WAN_CONF_FILE"; }
    exit 0
fi

if [ "$USE_DEFAULTS" = "1" ]; then
    [ -n "$HUB_IP" ]  || HUB_IP="$DEFAULT_HUB_IP"
    [ -n "$SELF_IP" ] || SELF_IP="$(bridge_ip || true)"
    if [ -z "$ROLE" ]; then
        if [ "$SELF_IP" = "$HUB_IP" ]; then ROLE="interstp"; else ROLE="operator"; fi
    fi
    if [ -z "$NODE" ] && [ "$ROLE" = "operator" ]; then
        NODE="$(node_of_file)"
        if [ -z "$NODE" ] && [ -r "$WAN_CONF_FILE" ]; then
            NODE="$(awk -F\" '/WAN_NODES=/{print $2}' "$WAN_CONF_FILE" 2>/dev/null \
                    | tr ' ' '\n' | awk -F: -v ip="$SELF_IP" '$2==ip{print $1; exit}')"
        fi
        [ -n "$NODE" ] || NODE=1
    fi
    if [ -z "$IPS" ] && [ "$ROLE" = "interstp" ] && [ -r "$WAN_CONF_FILE" ]; then
        IPS="$(awk -F\" '/WAN_NODES=/{print $2}' "$WAN_CONF_FILE" 2>/dev/null \
               | tr ' ' '\n' | awk -F: '{print $2}' | tr '\n' ' ')"
    fi
fi

banner
[ "$DRY" = "1" ] || [ "$(id -u)" -eq 0 ] \
```

```
    || { echo -e "${RED}Root requis (ecriture dans ${CONF_DIR} et sur l'interface).${NC}" >&2; exit 1; }

# ============================================================================
# 1. WHICH MACHINE IS THIS?
# ============================================================================
# The role decides everything else: the hub listens and knows its operators, an
# operator attaches and only knows the hub.
if [ -z "$ROLE" ]; then
    def_role="$(role_of_file)"; [ -n "$def_role" ] || def_role="operator"
    if [ -t 0 ]; then
        echo -e "${BOLD}Role de cette machine${NC}"
        echo -e "    ${CYAN}1${NC}) inter-STP   - le hub SS7, PC 0.0.0, celui que tous appellent"
        echo -e "    ${CYAN}2${NC}) operateur   - osmo-operator-N, qui s'attache au hub"
        echo ""
        def_n=2; [ "$def_role" = "interstp" ] && def_n=1
        ans="$(_wan_ask "Role" "1 = inter-STP, 2 = operateur :" "$def_n")" || exit 1
        case "$ans" in
            1|inter|interstp|hub)   ROLE="interstp" ;;
            2|op|operator|operateur|operateur) ROLE="operator" ;;
            *) echo -e "${RED}reponse invalide : '$ans'${NC}" >&2; exit 2 ;;
        esac
    else
        ROLE="$def_role"
        echo -e "  ${YELLOW}Pas de terminal - role repris de ${ROLE_FILE} : ${ROLE}${NC}"
    fi
fi
case "$ROLE" in
    interstp|inter|hub) ROLE="interstp" ;;
    operator|op)        ROLE="operator" ;;
    *) echo -e "${RED}--role : interstp ou operator${NC}" >&2; exit 2 ;;
esac
[[ "$OPS" =~ ^[1-9]$ ]] || { echo -e "${RED}--op : un chiffre de 1 a 9${NC}" >&2; exit 2; }
[[ "$OP_INDEX" =~ ^[1-9]$ ]] || { echo -e "${RED}--op-index : un chiffre de 1 a 9${NC}" >&2; exit 2; }
# An index above the count names an operator this node does not carry: its
# point code would fall in a range nobody routes.
[ "$OP_INDEX" -le "$OPS" ] \
    || { echo -e "${RED}--op-index ${OP_INDEX} : ce noeud n'annonce que ${OPS} operateur(s) (--op)${NC}" >&2; exit
    ↪  2; }

DETECTED="$(bridge_ip || true)"
echo -e "${BOLD}Role :${NC} ${CYAN}${ROLE}${NC}${DETECTED:+    IP de pont detectee : ${CYAN}${DETECTED}${NC}}"
echo ""

# ============================================================================
# 2A. INTER-STP - its bridge IP, and those of the osmo-operator-N
# ============================================================================
if [ "$ROLE" = "interstp" ]; then

    if [ -z "$SELF_IP" ]; then
        def="${DETECTED:-192.168.1.10}"
        SELF_IP="$(_wan_ask "Inter-STP" "IP de pont de CE hub :" "$def")" || exit 1
    fi
    need_ipv4 "$SELF_IP" "IP du hub"

    # Operator IPs. The hub does not need them to ROUTE (it listens, the ASPs
    # come to it, the point code decides the rest); it needs them to open its
    # firewall only to them, to write the table the nodes read, and so that
    # --status can say WHO is missing.
    WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=(); i=0
    if [ -n "$IPS" ]; then
        for ip in ${IPS//,/ }; do
            i=$((i+1)); need_ipv4 "$ip" "IP de l'operateur $i"
            WAN_IP[$i]="$ip"; WAN_IND[$i]="$(wan_default_ind "$i")"; WAN_NODE_LIST+=("$i")
        done
    else
        old_n=0
```

```
    [ -r "$WAN_CONF_FILE" ] && { wan_nodes_load "$WAN_CONF_FILE" >/dev/null 2>&1 && old_n="$WAN_NODE_COUNT"; }
    declare -A old_ip=()
    for k in ${WAN_NODE_LIST[@]+"${WAN_NODE_LIST[@]}"}; do old_ip[$k]="${WAN_IP[$k]}"; done
    WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=()
    n="$(_wan_ask "Inter-STP" "Nombre d'osmo-operator (1-9) :" "$([ "$old_n" -ge 1 ] && echo "$old_n" || echo
     ↪  3)")" || exit 1
    [[ "$n" =~ ^[1-9]$ ]] || { echo -e "${RED}nombre invalide : $n${NC}" >&2; exit 2; }
    for i in $(seq 1 "$n"); do
        # Default aligned with the bench plan: osmo-operator-N at .1N on
        # the hub's /24.
        def="${old_ip[$i]:-${SELF_IP%.*}.$((10 + i))}"
        ip="$(_wan_ask "osmo-operator-$i" "IP de pont de osmo-operator-$i :" "$def")" || exit 1
        need_ipv4 "$ip" "IP de osmo-operator-$i"
        WAN_IP[$i]="$ip"; WAN_IND[$i]="$(wan_default_ind "$i")"; WAN_NODE_LIST+=("$i")
    done
fi
NODES="${#WAN_NODE_LIST[@]}"
[ "$NODES" -ge 1 ] || { echo -e "${RED}aucun operateur saisi${NC}" >&2; exit 2; }
# The hub carries no operator and is not a node of the table: node id 0
# (accepted by wan_nodes_validate) rather than a number that would make it
# look like a node and steal a point code.
WAN_NODE_ID=0; WAN_NODE_COUNT="$NODES"; WAN_OPS="$OPS"
wan_nodes_validate || exit 2

echo ""
echo -e "  Hub ${CYAN}${SELF_IP}:${PORT}${NC}   PC ${CYAN}0.0.0${NC}"
for i in "${WAN_NODE_LIST[@]}"; do
    printf '    osmo-operator-%s  %-15s  STP %s  MSC %s  BSC %s\n' \
        "$i" "${WAN_IP[$i]}" "1.${i}1.2" "1.${i}1.1" "1.${i}1.3"
done
echo ""

ensure_ip "$SELF_IP"

# ── The hub's config ──────────────────────────────────────────────
# local-ip 0.0.0.0 would advertise every address of the machine in the SCTP
# INIT (NAT 10.0.2.15 and 172.20.x aliases included): paths that lead
# nowhere from the nodes. A single correct address is written instead.
if [ "$DRY" = "1" ]; then
    echo -e "  [dry-run] ${INTEROP_CFG} : listen m3ua ${PORT} / local-ip ${SELF_IP}"
else
    mkdir -p "$CONF_DIR"
    # Regenerated on every run, as start-interstp.sh does: the file is
    # fully derived from (NODES, OPS). A stale one would keep "as" blocks
    # for vanished nodes and none for new ones (an ASP without a matching
    # "as" is silently refused).
    bash "$HERE/helpers/create_interop.sh" --wan "$NODES" "$OPS" "$INTEROP_CFG" || exit 1
    tmp="$(mktemp)"
    awk -v hub="$SELF_IP" -v port="$PORT" '
        /^ *listen m3ua/ { sub(/listen m3ua .*/, "listen m3ua " port); in_l=1; print; next }
        in_l && /^ *local-ip / { sub(/local-ip .*/, "local-ip " hub); print; next }
        in_l && /^(!|[a-z])/ { in_l=0 }
        { print }
    ' "$INTEROP_CFG" > "$tmp"
    # cat rather than mv: on the ISO or in a container these files may be
    # mount points, where rename() fails.
    cat "$tmp" > "$INTEROP_CFG"; rm -f "$tmp"
    echo -e "  ${GREEN}✓${NC} $(basename "$INTEROP_CFG") - ecoute ${CYAN}${SELF_IP}:${PORT}${NC}"
fi

if [ "$DRY" = "1" ]; then
    echo -e "  [dry-run] ${WAN_CONF_FILE} : $(wan_nodes_spec)"
else
    WAN_AUTO=0 wan_nodes_save "$WAN_CONF_FILE" \
        && echo -e "  ${GREEN}✓${NC} table ecrite dans ${CYAN}${WAN_CONF_FILE}${NC}"
    for i in "${WAN_NODE_LIST[@]}"; do
```

```
            bash "$HERE/network/firewall-wan.sh" "${WAN_IP[$i]}" "$OPS" >/dev/null 2>&1 || true
        done
        echo -e "  ${GREEN}✓${NC} pare-feu ouvert pour ${NODES} operateur(s)"
    fi
    write_role interstp "$SELF_IP" ""

    echo ""
    echo -e "  ${BOLD}Cote operateurs :${NC} sur chaque VM osmo-operator-N, lancer"
    echo -e "    ${CYAN}sudo ./set_stp_ip.sh --operator --node N --ip <son IP> --hub-ip ${SELF_IP}${NC}"

    if [ "$RESTART" = "1" ] && [ "$DRY" != "1" ]; then
        echo ""
        if [ -x "$HERE/launch/start-interstp.sh" ]; then
            "$HERE/launch/start-interstp.sh" --ip "$SELF_IP" --nodes "$NODES" --ops "$OPS" --port "$PORT" --no-ip
        else
            systemctl restart osmo-interstp 2>/dev/null \
                && echo -e "  ${GREEN}✓${NC} osmo-interstp relance" \
                || echo -e "  ${YELLOW}⚠${NC} relancez le hub a la main : ${CYAN}./launch/start-interstp.sh${NC}"
        fi
    else
        echo -e "  ${YELLOW}Le hub deja lance garde l'ancienne adresse :${NC}
         ↪  ${CYAN}./launch/start-interstp.sh${NC}"
    fi
    exit 0
fi

# ═══════════════════════════════════════════════════════════════════
# 2B. OSMO-OPERATOR-N - its bridge IP, and the inter-STP's
# ═══════════════════════════════════════════════════════════════════
if [ -z "$NODE" ]; then
    def="$(node_of_file)"; [ -n "$def" ] || def=1
    NODE="$(_wan_ask "osmo-operator" "Numero de CE noeud operateur (1-9) :" "$def")" || exit 1
fi
[[ "$NODE" =~ ^[1-9]$ ]] || { echo -e "${RED}--node : un chiffre de 1 a 9${NC}" >&2; exit 2; }

if [ -z "$SELF_IP" ]; then
    def="${DETECTED:-192.168.1.$((10 + NODE))}"
    SELF_IP="$(_wan_ask "osmo-operator-$NODE" "IP de pont de CE noeud :" "$def")" || exit 1
fi
need_ipv4 "$SELF_IP" "IP de ce noeud"

if [ -z "$HUB_IP" ]; then
    def="$(hub_of_file)"
    # A hub in 172.20.x (docker plan) or 192.168.56.x (host-only) inherited
    # from the image does not exist on the bridge: never offer it as default.
    case "$def" in ""|172.20.*|192.168.5[6-9].*|192.168.6[0-3].*) def="${SELF_IP%.*}.10" ;; esac
    HUB_IP="$(_wan_ask "Inter-STP" "IP de pont de l'inter-STP :" "$def")" || exit 1
fi
need_ipv4 "$HUB_IP" "IP de l'inter-STP"

if [ "$SELF_IP" = "$HUB_IP" ]; then
    # Co-located hub: node 1 carries the inter-STP on its own address. Valid as
    # long as the hub LISTENS (port 2908) and the ASP leaves from another port
    # (2910): the node attaches to a neighbouring process, not to itself.
    # Refuse only when nobody listens at the hub.
    if pgrep -f 'osmo-stp -c /etc/osmocom/osmo-stp-interop\.cfg' >/dev/null 2>&1; then
        echo -e "  ${YELLOW}⚠${NC} inter-STP co-localise sur ${SELF_IP} (hub en service) - ASP vers le voisin, pas
         ↪  vers soi"
    else
        echo -e "${RED}Ce noeud et l'inter-STP portent la meme adresse (${SELF_IP}).${NC}" >&2
        echo -e "${RED}Aucun inter-STP n'ecoute ici : lancez start-interstp.sh, ou corrigez l'adresse.${NC}" >&2
        exit 2
    fi
fi
if [ "${SELF_IP%.*}" != "${HUB_IP%.*}" ]; then
    echo -e "  ${YELLOW}⚠${NC} ${SELF_IP} et ${HUB_IP} ne sont pas sur le meme /24 -"
```

```
    echo -e "     il faudra une route entre les deux pour que l'ASP monte."
fi

echo ""
echo -e "  ${BOLD}osmo-operator-${NODE}${NC}   ${CYAN}${SELF_IP}${NC}  ──M3UA──►  inter-STP
 ↪  ${CYAN}${HUB_IP}:${PORT}${NC}"
# The <op> of the plan is the operator index, not the count.
echo -e "  STP ${CYAN}1.${NODE}${OP_INDEX}.2${NC}   MSC ${CYAN}1.${NODE}${OP_INDEX}.1${NC}   BSC
 ↪  ${CYAN}1.${NODE}${OP_INDEX}.3${NC}   (plan WAN 1.<noeud><op>.<role>)"
echo ""

ensure_ip "$SELF_IP"

# ── Point codes and ASP addresses ──────────────────────────────────────
# set-node-id.sh is the only place that keeps the THREE files (osmo-stp,
# osmo-msc, osmo-bsc) consistent: own PC, routing key and the peer PC each one
# calls; hence the delegation rather than a second sed here. Its --op is an
# operator INDEX, not a count.
SNI_ARGS=(--node "$NODE" --op "$OP_INDEX" --native
          --local-ip "$SELF_IP" --hub-ip "$HUB_IP" --conf-dir "$CONF_DIR")
[ "$DRY" = "1" ] && SNI_ARGS+=(--dry-run)
bash "$HERE/network/set-node-id.sh" "${SNI_ARGS[@]}" || exit 1

# ── The WAN table ──────────────────────────────────────────────────────
# Start from the existing one so the other nodes are kept: this script only
# knows THIS node and the hub.
WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=()
[ -r "$WAN_CONF_FILE" ] && wan_nodes_load "$WAN_CONF_FILE" >/dev/null 2>&1 || true
if [ -z "${WAN_IP[$NODE]:-}" ]; then
    WAN_NODE_LIST+=("$NODE")
    WAN_IND[$NODE]="$(wan_default_ind "$NODE")"
fi
WAN_IP[$NODE]="$SELF_IP"
[ -n "${WAN_IND[$NODE]:-}" ] || WAN_IND[$NODE]="$(wan_default_ind "$NODE")"
WAN_NODE_ID="$NODE"; WAN_NODE_COUNT="${#WAN_NODE_LIST[@]}"; WAN_OPS="$OPS"

if [ "$DRY" = "1" ]; then
    echo -e "  [dry-run] ${WAN_CONF_FILE} : $(wan_nodes_spec)  (ce noeud : ${NODE})"
else
    if wan_nodes_validate; then
        wan_nodes_save "$WAN_CONF_FILE" \
            && echo -e "  ${GREEN}✓${NC} ${CYAN}${WAN_CONF_FILE}${NC} - ce noeud : ${NODE}"
    else
        echo -e "  ${YELLOW}⚠${NC} table WAN incoherente, non reecrite - ${CYAN}${WAN_CONF_FILE}${NC} inchange"
    fi
    bash "$HERE/network/firewall-wan.sh" "$HUB_IP" "$OPS" >/dev/null 2>&1 || true
fi
write_role operator "$HUB_IP" "$NODE"

echo ""
if [ "$RESTART" = "1" ] && [ "$DRY" != "1" ]; then
    if [ -x "$HERE/start-native.sh" ]; then
        echo -e "  Relance de la pile..."
        "$HERE/start-native.sh" --node "$NODE" || true
    else
        echo -e "  ${YELLOW}⚠${NC} start-native.sh absent - relancez la pile a la main."
    fi
else
    echo -e "  ${YELLOW}Les demons deja lances gardent l'ancienne adresse :${NC} relancez la pile"
    echo -e "    ${CYAN}sudo ./start-native.sh --node ${NODE}${NC}"
fi
echo -e "  ${BOLD}Verifier l'attachement, cote hub :${NC} ${CYAN}./launch/start-interstp.sh --status${NC}"
```

2.13 /opt/GSM/osmo-operator/ss7-console.py

336 octets, 10 lignes → 10 lignes

```
#!/usr/bin/env python3
# ss7-console.py - shortcut to the SS7 console; the code lives in navigation/ss7-console.py.
import os
import runpy
import sys

target = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                      "navigation", "ss7-console.py")
sys.argv[0] = target
runpy.run_path(target, run_name="__main__")
```

2.14 /opt/GSM/osmo-operator/start-native.sh

2265 octets, 55 lignes → 55 lignes

```
#!/bin/bash
# ============================================================================
# start.sh - native launcher: prepares the Calypso environment, then runs run.sh
# (start-native.sh is a symlink to this file)
# ============================================================================
#
# Usage:
#   ./start.sh [options]            native launcher (startup_scripts/*.sh)
#   ./start.sh --docker [options]   hands over to launch/start-docker.sh
#   VAR=x ./start.sh                environment overrides always win
#
# This script starts no daemon itself. It:
#   1. loads the environment (environment/load.env);
#   2. detects the hardware / binaries;
#   3. chooses the mode (profile);
#   4. generates the mobile_*.cfg files;
#   5. exports the required variables;
#   6. executes run.sh (RUN_SH, resolved in startup_scripts/05-run-sh.sh).
#
# All GSM logic then lives in run.sh (and run_modules/).
#
# Configuration chain (do not break it):
#     VAR=x ./start.sh              <- the command line always wins
#       -> environment/load.env
#          -> paths / modes / domains
#     then exec run.sh with the chosen profile.
#
# Known limitation: there is no "must run inside Docker" guard, because the ISO
# has no Docker. Run by mistake on a host with Docker, it dies with "run.sh not
# found": the launchers there are ./launch/start-docker.sh or
# ./launch/start-nitb.sh, not this one.
# ----------------------------------------------------------------------------
set -uo pipefail

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cd "$HERE"

# -- --docker: the bench in containers, through launch/start-docker.sh --------
# `./start.sh --docker [args]` hands over to start-docker.sh with the other
# arguments untouched; the rest of this script (startup_scripts/) is the
# NATIVE launcher and is not read.
_args=(); _docker=0
for _a in "$@"; do
    if [ "$_a" = --docker ]; then _docker=1; else _args+=("$_a"); fi
```

```
done
if [ "$_docker" = 1 ]; then
    exec bash "$HERE/launch/start-docker.sh" "${_args[@]+"${_args[@]}"}"
fi
unset _args _docker _a

# ── Native launcher: startup_scripts/*.sh, sourced in order (same shell, same variables) ──
for _f in "$HERE"/startup_scripts/*.sh; do
    . "$_f"
done
unset _f
```

2.15 /opt/GSM/osmo-operator/start.sh

identique à /opt/GSM/osmo-operator/start-native.sh

2.16 /opt/GSM/osmo-operator/update.sh

53781 octets, 999 lignes → 999 lignes

```
#!/bin/bash
# ================================================================================
# update.sh - per-session catch-up of an installed osmo-operator machine, then
# the "SMS delivered" session animation.
#
# Everything that belongs to the build (cloning repos, apt packages, services)
# is done by build-iso.sh. This script only replays, idempotently and offline,
# what an already installed machine would otherwise miss after a git pull:
#   - desktop icons (absolute Icon= paths, metadata::trusted, icon cache);
#   - osmo-multi.service: Wants= instead of Requires=osmo-banc.service;
#   - QEMU mobile template (c54x_exe/mobile_pont.cfg) aligned on PLAN_ARFCN;
#   - launchers and .desktop entries: overlay-draw, postmarketOS phone, deka,
#     4G/LTE and smartphone UI (tools/osmo-lte-install.sh,
#     tools/osmo-pmos-install.sh), Wireshark (root, bench filters),
#     /usr/local/bin launchers of tools/osmo-extras-install.sh,
#     osmo-start-native / osmo-start-direct, the "?" help page (osmo-info);
#   - systemd: services/osmo-banc.service, web consoles (osmo-egprs-web,
#     open5gs-webui) enabled unless masked, "no telecom service at boot"
#     policy applied once (tools/osmo-telecom-boot.sh), osmo-* units run as
#     the session user instead of the missing "osmocom" account;
#   - osmo-desktop-panel wrapper (desktop widgets, Conky);
#   - Firefox from the Mozilla deb repository when no browser is present;
#   - session user added to the docker group;
#   - audio: /etc/osmocom/gapk-start.sh, Asterisk 600 echo test, PulseAudio
#     echo canceller and speech-dispatcher sink; srsUE log levels;
#   - voice settings in the qosmo forks (scripts/voix-forks.sh);
#   - gnuarm profile, gcc alternatives, c123 and banc-* commands, WireGuard;
#   - patches/*.patch applied to the /opt/GSM trees (tools/osmo-patches-apply.sh).
#
# Usage:
#   sudo ./update.sh            run the catch-up steps, then play the animation
#   sudo ./update.sh --quiet    run the catch-up steps only (exit code 0)
#   ./update.sh -h | --help     print this header
#
# On the ISO, /etc/profile.d/99-osmo-sms.sh calls it once per session, after
# the keyboard choice (alphabetical order of /etc/profile.d). Most steps need
# root and are skipped silently otherwise. Without a tty, the result is shown
# in a zenity window when a display is available; otherwise the script exits
# 0 silently.
# ================================================================================
set -u

# Called from /etc/profile.d and from desktop launchers, where the current
```

```
# directory is not the repository: move there before touching anything.
cd "$(dirname "$(readlink -f "$0")")" || true

case "${1:-}" in
    -h|--help) sed -n '2,40p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
esac

# ── DESKTOP ICONS, RE-INSTALLED AT EVERY SESSION ──────────────────────
# "Icon=<name>" is resolved through the hicolor icon-theme cache, and a name
# that does not resolve is silently replaced by a blank page by GNOME/DING.
# The shortcuts therefore carry Icon= with an absolute path; the theme copy
# (plus a cache refresh) only serves the application menu. Idempotent.
osmo_reposer_icones() {
    [ "$(id -u)" -eq 0 ] || return 0        # without root: nothing to do
    local src=/opt/GSM/osmo-operator/data
    local dst=/usr/share/osmo-operator/icons
    [ -d "$src" ] || return 0

    install -d "$dst" 2>/dev/null || return 0
    local ic
    for ic in osmo-launch osmo-multi osmo-tutorial; do
        [ -f "$src/$ic.svg" ] || continue
        cp -f "$src/$ic.svg" "$dst/$ic.svg" 2>/dev/null || continue
        chmod 644 "$dst/$ic.svg" 2>/dev/null || true
        # The theme copy serves the application MENU, which still resolves
        # icons by name; the desktop uses the absolute path.
        install -d /usr/share/icons/hicolor/scalable/apps 2>/dev/null || true
        cp -f "$src/$ic.svg" \
              "/usr/share/icons/hicolor/scalable/apps/$ic.svg" 2>/dev/null || true
    done
    command -v gtk-update-icon-cache >/dev/null 2>&1 && \
        gtk-update-icon-cache -f -q /usr/share/icons/hicolor 2>/dev/null || true

    # Icon= as an absolute path: no theme, no cache, no silent fallback.
    # The shortcuts live in the menu and on each account's desktop (Bureau or
    # Desktop, whichever the locale names); every copy carries the same line.
    # All accounts are covered, not only a hard-coded "osmocom".
    local f b d
    for d in /usr/share/applications /root/Bureau /root/Desktop \
             /home/*/Bureau /home/*/Desktop; do
        [ -d "$d" ] || continue
        for b in osmo-launch osmo-multi osmo-tutorial; do
            f="$d/$b.desktop"
            [ -f "$f" ] || continue
            grep -q "^Icon=$dst/$b.svg\$" "$f" && continue
            sed -i "s|^Icon=.*|Icon=$dst/$b.svg|" "$f" 2>/dev/null || true
        done
    done

    # A desktop .desktop file only shows its name and icon when it is
    # executable AND carries metadata::trusted. That attribute lives in the
    # OWNER's gvfs session metadata, never in the file: a `gio set` run by root
    # sets it for root only, so it is set through the owner's account and
    # session bus (runuser), as _trust_desktop() in addition.sh does.
    local _own _uid _bus
    for d in "$HOME/Bureau" "$HOME/Desktop" /root/Bureau /root/Desktop \
             /home/*/Bureau /home/*/Desktop; do
        [ -d "$d" ] || continue
        _own="$(stat -c '%U' "$d" 2>/dev/null)"; [ -n "$_own" ] || _own=root
        _uid="$(id -u "$_own" 2>/dev/null)" || continue
        _bus="/run/user/$_uid/bus"
        for f in "$d"/*.desktop; do
            [ -f "$f" ] || continue
            chown "$_own" "$f" 2>/dev/null || true
            chmod +x "$f" 2>/dev/null || true
            if [ -S "$_bus" ] && command -v runuser >/dev/null 2>&1; then
```

```
                runuser -u "$_own" -- env XDG_RUNTIME_DIR="/run/user/$_uid" \
                    DBUS_SESSION_BUS_ADDRESS="unix:path=$_bus" \
                    gio set -t string "$f" metadata::trusted true 2>/dev/null || true
            else
                gio set -t string "$f" metadata::trusted true 2>/dev/null || true
            fi
        done
        # DING does not re-read metadata on the fly: touching the directory
        # forces a rescan.
        touch "$d" 2>/dev/null || true
    done
    update-desktop-database /usr/share/applications 2>/dev/null || true
    return 0
}
osmo_reposer_icones

# ── osmo-multi.service MUST NOT DIE WHEN THE NATIVE BENCH IS RECYCLED ────────
# start-multi.sh stops and restarts osmo-banc at every launch. With
# Requires=osmo-banc.service that explicit stop would propagate to osmo-multi
# itself during its ExecStart; Wants= keeps the boot order (After=) without
# propagating the stop.
osmo_corriger_multi_natif() {
    [ "$(id -u)" -eq 0 ] || return 0
    local unit=/etc/systemd/system/osmo-multi.service
    [ -f "$unit" ] || return 0
    grep -q '^Requires=osmo-banc\.service$' "$unit" || return 0
    sed -i 's/^Requires=osmo-banc\.service$/Wants=osmo-banc.service/' "$unit"
    if command -v systemctl >/dev/null 2>&1 && [ -d /run/systemd/system ]; then
        systemctl daemon-reload 2>/dev/null || true
    fi
    echo "[OK] osmo-multi.service : Requires=osmo-banc -> Wants (plus de suicide au recyclage du natif)."
    return 0
}
osmo_corriger_multi_natif

# ── THE QEMU MOBILE MUST LOOK FOR THE CELL ON THE RADIO PLAN'S ARFCN ─────────
# The BSC of the multi containers transmits on PLAN_ARFCN, but the QEMU mobile
# of instance 1 reads c54x_exe/mobile_pont.cfg as is (run.sh only rewrites
# "stick" for instances 2+). Align the template on the radio plan:
# radio-plan.env, then ARFCN of globals.conf, then 1. Idempotent.
osmo_aligner_arfcn_qemu() {
    local cfg arfcn
    arfcn="$( . /etc/osmocom/radio-plan.env 2>/dev/null; echo "${PLAN_ARFCN:-}")"
    [ -n "$arfcn" ] || arfcn="$(sed -n 's/^ARFCN="\{0,1\}\([0-9]*\)"\{0,1\}$/\1/p' \
        "$(dirname "$(readlink -f "$0")")/globals.conf" 2>/dev/null | tail -1)"
    [[ "$arfcn" =~ ^[0-9]+$ ]] || arfcn=1
    for cfg in /opt/GSM/c54x_exe/mobile_pont.cfg; do
        [ -w "$cfg" ] || continue
        grep -qE "^[[:space:]]*stick ${arfcn}\$" "$cfg" \
            && ! grep -qE "^[[:space:]]*no (p|e)-gsm\$" "$cfg" && continue
        # The template only enables DCS (no p-gsm / no e-gsm): a stick on an
        # unsupported 900 MHz ARFCN makes gsm322_cs_powerscan loop at 100% CPU,
        # so the P/E-GSM bands are enabled as well.
        sed -i -e "s|^\([[:space:]]*\)stick [0-9]*\$|\1stick ${arfcn}|" \
            -e "s|^\([[:space:]]*\)no \([pe]-gsm\)\$|\1\2|" "$cfg"
        echo "[OK] $cfg : stick ${arfcn} (ARFCN du plan radio), bandes P/E-GSM actives."
    done
    return 0
}
osmo_aligner_arfcn_qemu

# ── THE OVERLAY PAINT TOOL ON ALREADY INSTALLED MACHINES ─────────────────────
# tools/overlay-draw.py and its launcher: symlink, icon, absolute icon path.
# Idempotent.
osmo_poser_peinture() {
    [ "$(id -u)" -eq 0 ] || return 0
```

```
    local d=/opt/GSM/osmo-operator
    local py="$d/tools/overlay-draw.py"
    [ -f "$py" ] || return 0
    chmod 755 "$py" 2>/dev/null || true
    ln -sf "$py" /usr/local/bin/overlay-draw 2>/dev/null || true
    if [ -f "$d/data/osmo-paint.svg" ]; then
        install -d /usr/share/osmo-operator/icons \
                   /usr/share/icons/hicolor/scalable/apps 2>/dev/null || true
        cp -f "$d/data/osmo-paint.svg" /usr/share/osmo-operator/icons/osmo-paint.svg 2>/dev/null || true
        cp -f "$d/data/osmo-paint.svg" /usr/share/icons/hicolor/scalable/apps/osmo-paint.svg 2>/dev/null || true
        chmod 644 /usr/share/osmo-operator/icons/osmo-paint.svg 2>/dev/null || true
    fi
    if [ -f "$d/data/desktop/osmo-paint.desktop" ]; then
        install -m644 "$d/data/desktop/osmo-paint.desktop" \
                /usr/share/applications/osmo-paint.desktop 2>/dev/null || true
        sed -i "s|^Icon=.*|Icon=/usr/share/osmo-operator/icons/osmo-paint.svg|" \
            /usr/share/applications/osmo-paint.desktop 2>/dev/null || true
    fi
    command -v gtk-update-icon-cache >/dev/null 2>&1 && \
        gtk-update-icon-cache -f -q /usr/share/icons/hicolor 2>/dev/null || true
    update-desktop-database /usr/share/applications 2>/dev/null || true
    return 0
}
osmo_poser_peinture

# ── THE postmarketOS PHONE ON ALREADY INSTALLED MACHINES ──────────────────
# The bench phone is a postmarketOS/Phosh VM (tools/osmo-pmos.sh). The ISO and
# addition.sh (through tools/osmo-extras-install.sh) install its launcher and
# icon; this replays it on installed machines and removes the obsolete
# Waydroid launcher. Idempotent.
osmo_poser_pmos() {
    [ "$(id -u)" -eq 0 ] || return 0
    local d=/opt/GSM/osmo-operator
    [ -f "$d/tools/osmo-pmos.sh" ] || return 0
    chmod 755 "$d/tools/osmo-pmos.sh" 2>/dev/null || true
    cat > /usr/local/bin/osmo-pmos <<'PM'
#!/bin/bash
# osmo-pmos - the bench's postmarketOS phone (see tools/osmo-pmos.sh).
exec "${OSMO_REPO:-/opt/GSM/osmo-operator}/tools/osmo-pmos.sh" "$@"
PM
    chmod 755 /usr/local/bin/osmo-pmos 2>/dev/null || true
    cat > /usr/share/applications/osmo-pmos.desktop <<'PMD'
[Desktop Entry]
Type=Application
Name=Telephone (postmarketOS)
Comment=Le telephone du banc - Phosh, ModemManager, appels et SMS reels
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --root --title "Telephone (postmarketOS)" --
 ↪  /usr/local/bin/osmo-pmos up
Icon=phone
Terminal=false
Categories=Network;Telephony;
Keywords=postmarketos;phosh;telephone;sms;appel;modem;
Actions=Etat;Arreter;

[Desktop Action Etat]
Name=Etat du telephone
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --keep-open --title "Etat du telephone" --
 ↪  /usr/local/bin/osmo-pmos status

[Desktop Action Arreter]
Name=Arreter le telephone
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --close-on-exit --title "Arret du telephone" --
 ↪  /usr/local/bin/osmo-pmos stop
PMD
    chmod 644 /usr/share/applications/osmo-pmos.desktop 2>/dev/null || true
    # The two pmbootstrap launchers: tools/osmo-pmos-qemu.sh (the VM, format
```

```
    # smartphone|tablette as argument) and tools/osmo-pmos-setup.sh (modem and
    # voice, VM running). The matching pmbootstrap patch,
    # patches/pmbootstrap-osmo-bench-qemu.patch, applies to the user's
    # pmbootstrap, not here.
    local s
    for s in qemu setup; do
        [ -f "$d/tools/osmo-pmos-$s.sh" ] \
            && install -m 755 "$d/tools/osmo-pmos-$s.sh" "/usr/local/bin/osmo-pmos-$s" 2>/dev/null
    done
    # Two formats, two icons: Phosh lays itself out according to the screen
    # QEMU gives it (OSMO_PMOS_RES, read by the pmbootstrap patch), and that
    # size cannot change under a running session, so the format is chosen at
    # launch through two entries.
    local f n r
    for f in smartphone:720x1440 tablette:1280x800; do
        n="${f%%:*}"; r="${f#*:}"
        cat > "/usr/share/applications/osmo-pmos-$n.desktop" <<PMD
[Desktop Entry]
Type=Application
Name=postmarketOS - $n (banc)
Comment=Le telephone du banc, modem et voix branches tout seuls, en format $n ($r) - modem branche sur le banc GSM
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Telephone du banc (VM)" -- env OSMO_PMOS_RES=$r
 ↪  /usr/local/bin/osmo-pmos-qemu
Icon=$([ "$n" = tablette ] && echo video-display || echo phone)
Terminal=false
Categories=Network;Telephony;System;
Keywords=postmarketos;pmos;qemu;modem;gsm;telephone;$n;
Actions=Arreter;Setup;SansModem;

[Desktop Action Arreter]
Name=Arreter le telephone (osmo-pmos-qemu stop)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --close-on-exit --title "Arret du telephone" --
 ↪  /usr/local/bin/osmo-pmos-qemu stop

[Desktop Action Setup]
Name=Rebrancher le modem et la voix (osmo-pmos-setup)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Modem et voix du telephone" --
 ↪  /usr/local/bin/osmo-pmos-setup

[Desktop Action SansModem]
Name=Demarrer sans modem (VM nue)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Telephone du banc (VM)" -- env OSMO_PMOS_RES=$r
 ↪  OSMO_PMOS_MODEM=0 /usr/local/bin/osmo-pmos-qemu
PMD
        chmod 644 "/usr/share/applications/osmo-pmos-$n.desktop" 2>/dev/null || true
    done
    rm -f /usr/local/bin/osmo-waydroid /usr/share/applications/osmo-waydroid.desktop 2>/dev/null || true
    update-desktop-database /usr/share/applications 2>/dev/null || true
    echo "  [telephone] osmo-pmos pose (VM postmarketOS ; « osmo-pmos install » la premiere fois)"
    echo "  [telephone] deux formats au lancement : smartphone (720x1440) et tablette (1280x800)"
    echo "  [telephone] « osmo-pmos-qemu stop » (ou le clic droit de l icone) arrete la VM et debranche le modem"
    return 0
}
osmo_poser_pmos

# ── DEKA: A STOP ACTION ON THE ICON'S RIGHT CLICK ──────────────────────────
# addition.sh (--opencl) installs deka.desktop; it is rewritten here only to
# add the "Arreter deka" menu action (deka-start.sh --stop), and only when deka
# is installed. The main click still starts deka.
osmo_poser_deka() {
    local dsk=/usr/share/applications/deka.desktop
    [ -x /usr/local/bin/osmo-deka-anim ] || [ -f "$dsk" ] || return 0
    local icon=/usr/share/osmo-operator/icons/deka.svg
    [ -f "$icon" ] || icon=deka
    local gtk=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py
    cat > "$dsk" <<DEKADSK
```

```
[Desktop Entry]
Type=Application
Name=deka
Name[fr]=deka
Comment=Monte les tables et lance les workers deka
Comment[fr]=Monte les tables et lance les workers deka
Exec=/usr/local/bin/osmo-deka-anim
Icon=$icon
Terminal=false
Categories=System;Utility;
Keywords=deka;tables;workers;opencl;
Actions=Arreter;

[Desktop Action Arreter]
Name=Arreter deka
Name[fr]=Arreter deka
Exec=$gtk --root --title "Arret deka" -- /root/deka/deka-start.sh --stop
DEKADSK
    chmod 644 "$dsk" 2>/dev/null || true
    update-desktop-database /usr/share/applications 2>/dev/null || true
    echo "  [deka] icone : clic = demarrer, clic droit « Arreter deka » = stop (workers + demontages)"
}
osmo_poser_deka

# ── 4G AND THE SMARTPHONE UI: CATCH-UP ──────────────────────────────────────
# Same sources as addition.sh and the ISO (tools/osmo-lte-install.sh,
# tools/osmo-pmos-install.sh). Nothing is compiled here: repository configs
# (existing ones are not overwritten), the osmo-lte / osmo-epc /
# open5gs-epc.sh launchers, the /opt/user_interface/pmos layout and its PPP
# kernel (from the .deb cache or GitHub). Missing binaries come from the cached
# .deb files; otherwise run "osmo-lte-install --build".
osmo_poser_lte() {
    local d=/opt/GSM/osmo-operator
    if [ -f "$d/tools/osmo-lte-install.sh" ]; then
        # shellcheck source=tools/osmo-lte-install.sh
        DIR="$d" . "$d/tools/osmo-lte-install.sh"
        osmo_lte_install --debs --configs --launchers 2>&1 | sed 's/^/  [4G]/'
    fi
    if [ -f "$d/tools/osmo-pmos-install.sh" ]; then
        # shellcheck source=tools/osmo-pmos-install.sh
        DIR="$d" . "$d/tools/osmo-pmos-install.sh"
        osmo_pmos_install 2>&1 | sed 's/^/  [telephone]/'
    fi
}
osmo_poser_lte

# ── WIRESHARK: AN ICON THAT ALREADY LISTENS TO THE BENCH ────────────────────
# tools/osmo-wireshark-root.sh: root through pkexec, immediate LTE + SCTP + GSM
# capture, 2G + 4G + phone display filter (signalling, call, SMS, data;
# heartbeats removed) and filter buttons in root's profile. "wireshark" on the
# command line links to it. The dock favourite (org.wireshark.Wireshark, which
# cannot capture) is replaced by ours in every open session; favourites live
# in the user's dconf, not root's. The ISO does the same in 80-chroot.
osmo_poser_wireshark() {
    [ "$(id -u)" -eq 0 ] || return 0
    local d=/opt/GSM/osmo-operator
    [ -f "$d/tools/osmo-wireshark-root.sh" ] || return 0
    install -m 755 "$d/tools/osmo-wireshark-root.sh" /usr/local/bin/osmo-wireshark-root
    ln -sfn /usr/local/bin/osmo-wireshark-root /usr/local/bin/wireshark
    cat > /usr/share/applications/osmo-wireshark-root.desktop <<'WSD'
[Desktop Entry]
Type=Application
Name=Wireshark (banc)
GenericName=Analyseur reseau
Comment=Wireshark en root, capture immediate du banc, filtre 2G + 4G + telephone (signalisation, appel, SMS, data)
 ↪  sans les battements de coeur ; boutons Banc / 2G / 4G / Appel / SMS / Data / Attach
```

```
Exec=/usr/local/bin/osmo-wireshark-root %f
Icon=org.wireshark.Wireshark
Terminal=false
StartupNotify=true
Categories=Network;Monitor;
Keywords=wireshark;capture;pcap;reseau;lte;sctp;gsm;gsmtap;2g;4g;sms;appel;
WSD
    chmod 644 /usr/share/applications/osmo-wireshark-root.desktop
    update-desktop-database /usr/share/applications 2>/dev/null || true
    # The favourite, in every open GNOME session.
    local u uid bus
    for u in $(loginctl list-users --no-legend 2>/dev/null | awk '{print $2}'); do
        uid="$(id -u "$u" 2>/dev/null)" || continue
        bus="/run/user/$uid/bus"; [ -S "$bus" ] || continue
        sudo -u "$u" DBUS_SESSION_BUS_ADDRESS="unix:path=$bus" python3 - <<'PY' 2>/dev/null || true
import subprocess, ast
g = ["gsettings", "get", "org.gnome.shell", "favorite-apps"]
cur = ast.literal_eval(subprocess.run(g, capture_output=True, text=True).stdout.strip() or "[]")
neu = [a for a in cur if a not in ("org.wireshark.Wireshark.desktop", "wireshark.desktop",
 ↪  "wireshark-root.desktop")]
if "osmo-wireshark-root.desktop" not in neu:
    i = next((k for k, a in enumerate(cur) if a in ("org.wireshark.Wireshark.desktop", "wireshark.desktop")),
 ↪  len(neu))
    neu.insert(min(i, len(neu)), "osmo-wireshark-root.desktop")
if neu != cur:
    subprocess.run(["gsettings", "set", "org.gnome.shell", "favorite-apps", repr(neu)], check=False)
PY
    done
    echo "  [wireshark] osmo-wireshark-root pose (LTE + SCTP + GSM, root par pkexec) ; « wireshark » y mene ;
     ↪  favori du dock remplace"
    return 0
}
osmo_poser_wireshark

# ── THE /usr/local/bin LAUNCHERS (osmo-dino-play, osmo-youtube...) ──────────
# They are installed by tools/osmo-extras-install.sh (addition.sh and the ISO);
# replay its launcher function alone so installed machines follow the
# repository. Idempotent, no apt, no network.
osmo_reposer_lanceurs() {
    [ "$(id -u)" -eq 0 ] || return 0
    local ex=/opt/GSM/osmo-operator/tools/osmo-extras-install.sh
    [ -f "$ex" ] || return 0
    # shellcheck source=tools/osmo-extras-install.sh
    ( . "$ex" && _osmo_extras_lanceurs ) >/dev/null 2>&1 \
        && echo "  [bureau] lanceurs /usr/local/bin reposes (osmo-dino-play, osmo-pmos...)" \
        || true
    return 0
}
osmo_reposer_lanceurs

# ── osmo-start-direct -> start-native.sh ────────────────────────────────────
# /usr/local/bin/osmo-start-direct on an installed machine may still point to
# the removed start-direct.sh: repoint it and install osmo-start-native next
# to it. Idempotent.
osmo_reposer_lien_start_native() {
    [ "$(id -u)" -eq 0 ] || return 0
    local t=/opt/GSM/osmo-operator/start-native.sh l
    [ -f "$t" ] || return 0
    for l in /usr/local/bin/osmo-start-native /usr/local/bin/osmo-start-direct; do
        [ "$(readlink "$l" 2>/dev/null)" = "$t" ] || ln -sf "$t" "$l"
    done
    return 0
}
osmo_reposer_lien_start_native

# ── osmo-banc.service: THE INSTALLED UNIT FOLLOWS THE REPOSITORY ────────────
```

```
# Re-install services/osmo-banc.service when it differs from the installed
# copy; a machine without the unit is left alone.
osmo_reposer_unite_banc() {
    [ "$(id -u)" -eq 0 ] || return 0
    local src=/opt/GSM/osmo-operator/services/osmo-banc.service dst=/etc/systemd/system/osmo-banc.service
    [ -f "$src" ] && [ -f "$dst" ] || return 0
    cmp -s "$src" "$dst" && return 0
    install -m644 "$src" "$dst" && systemctl daemon-reload 2>/dev/null || true
    return 0
}
osmo_reposer_unite_banc

# ── NO TELECOM SERVICE AT BOOT: APPLIED ONCE ──────────────────────────
# tools/osmo-telecom-boot.sh is run once (marker file) so that a service the
# operator re-enables afterwards (systemctl enable osmo-banc) is not disabled
# again at every update.
osmo_politique_telecom_boot() {
    [ "$(id -u)" -eq 0 ] || return 0
    local m=/var/lib/osmo-operator/telecom-boot-v1 t=/opt/GSM/osmo-operator/tools/osmo-telecom-boot.sh
    [ -e "$m" ] || [ ! -f "$t" ] && return 0
    bash "$t" && mkdir -p "${m%/*}" && date -Is > "$m"
    return 0
}
osmo_politique_telecom_boot

# ── THE DOCK'S "?": HELP PAGE SERVED OVER HTTP, FIRST FAVOURITE ───────
# Installs /usr/local/bin/osmo-info (dashboard on port 8080, local fallback),
# the help page in both locations, and osmo-info.desktop as first favourite of
# every open session (snap Firefox cannot open file:// pages).
osmo_poser_aide() {
    [ "$(id -u)" -eq 0 ] || return 0
    local R=/opt/GSM/osmo-operator u uid bus
    [ -f "$R/configs/info.html" ] || return 0
    install -m755 "$R/tools/osmo-info" /usr/local/bin/osmo-info 2>/dev/null || true
    install -D -m644 "$R/configs/info.html" /usr/share/osmo-operator/info.html
    [ -d /opt/GSM/osmo-egprs-web/web ] && cp -f "$R/configs/info.html" /opt/GSM/osmo-egprs-web/web/info.html
    install -m644 "$R/data/desktop/osmo-info.desktop" /usr/share/applications/osmo-info.desktop
    update-desktop-database /usr/share/applications 2>/dev/null || true
    for u in $(loginctl list-users --no-legend 2>/dev/null | awk '{print $2}'); do
        uid="$(id -u "$u" 2>/dev/null)" || continue
        bus="/run/user/$uid/bus"; [ -S "$bus" ] || continue
        sudo -u "$u" DBUS_SESSION_BUS_ADDRESS="unix:path=$bus" python3 - <<'PY' 2>/dev/null || true
import subprocess, ast
g = ["gsettings", "get", "org.gnome.shell", "favorite-apps"]
cur = ast.literal_eval(subprocess.run(g, capture_output=True, text=True).stdout.strip() or "[]")
neu = ["osmo-info.desktop"] + [a for a in cur if a != "osmo-info.desktop"]
if neu != cur:
    subprocess.run(["gsettings", "set", "org.gnome.shell", "favorite-apps", repr(neu)], check=False)
PY
    done
    return 0
}
osmo_poser_aide

# ── INTERACTIVE DESKTOP: CATCH-UP OF THE osmo-desktop-panel WRAPPER ────
# The widget launcher (/usr/local/bin/osmo-desktop-panel) is baked into the
# image by iso_modules/85-installeur-bureau.sh; the widgets (osmo-topzone.py,
# osmo-launcher.py, osmo-ts-probe.py) arrive by git pull. The wrapper is
# re-installed when it lacks osmo_vivant (widgets tracked per display).
# Exact mirror of iso_modules/85-installeur-bureau.sh.
osmo_reposer_bureau_interactif() {
    [ "$(id -u)" -eq 0 ] || return 0
    local w=/usr/local/bin/osmo-desktop-panel
    [ -f "$w" ] || return 0
    grep -q 'osmo_vivant' "$w" 2>/dev/null && return 0
    cat > "$w" <<'PANEL'
```

```
#!/bin/sh
# osmo-desktop-panel - keeps the inset and Conky alive for the whole session.
REPO=/opt/GSM/osmo-operator
export OSMO_REPO="$REPO"
# The LAB GSM banner is not baked into the wallpaper: osmo-topzone.py renders
# it live. Tell the wallpaper renderer.
export OSMO_LIVE_BANNER=1

# ── A WIDGET LEFT OVER FROM THE PREVIOUS SESSION IS NOT A WIDGET ─────────────
# A conky or python widget from a previous X session may still be running with
# an old DISPLAY (e.g. :0 while GNOME restarted on :1): it cannot paint
# anything, but "pgrep -x conky" would count it as alive and nothing would be
# restarted. Only processes looking at OUR display count; the others are
# killed, they can no longer serve anyone.
osmo_vivant() {
    _trouve=1
    for _p in $(pgrep "$@" 2>/dev/null); do
        _d="$(tr '\0' '\n' < "/proc/$_p/environ" 2>/dev/null | sed -n 's/^DISPLAY=//p' | head -1)"
        # Kill ONLY what was read and looks elsewhere. An unreadable environ,
        # or one without DISPLAY, proves nothing: that process counts as
        # alive - confusing "absent" with "invisible" would spawn a duplicate
        # of each widget on every pass.
        if [ -n "$_d" ] && [ "$_d" != "$DISPLAY" ]; then kill "$_p" 2>/dev/null
        else _trouve=0
        fi
    done
    return $_trouve
}
# The watcher must not outlive its own X server: otherwise the old one keeps
# running next to the new one and restarts into the void forever.
osmo_display_vivant() {
    [ -n "$DISPLAY" ] || return 0          # no named display: cannot judge
    _n="${DISPLAY#*:}"; _n="${_n%%.*}"
    [ -S "/tmp/.X11-unix/X$_n" ]
}

# ── CONKY ARROWS ON THE KEYBOARD: Ctrl+AltGr+Left / Right ────────────────────
# A gschema.override does NOT apply to a RELOCATABLE schema, so the shortcut
# is written here, in the session, once per boot: the only place that runs
# UNDER the user's account, where gsettings writes into their dconf.
# <Mod5> is ISO_Level3_Shift, i.e. AltGr (xmodmap -pm confirms it).
osmo_raccourci() {
    command -v gsettings >/dev/null 2>&1 || return 0
    _k=org.gnome.settings-daemon.plugins.media-keys.custom-keybinding
    _p=/org/gnome/settings-daemon/plugins/media-keys/custom-keybindings
    gsettings set org.gnome.settings-daemon.plugins.media-keys custom-keybindings \
        "['$_p/osmo-op-next/','$_p/osmo-op-prev/']" 2>/dev/null || return 0
    gsettings set "$_k:$_p/osmo-op-next/" name 'osmo-operator : arret suivant (Conky)' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-next/" command '/usr/local/bin/osmo-op --next' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-next/" binding '<Control><Mod5>Right' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-prev/" name 'osmo-operator : arret precedent (Conky)' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-prev/" command '/usr/local/bin/osmo-op --prev' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-prev/" binding '<Control><Mod5>Left' 2>/dev/null
}
osmo_raccourci

# Conky must find the GNOME desktop already painted (own_window_type
# desktop), otherwise it ends up behind the wallpaper.
sleep 6
while osmo_display_vivant; do
    osmo_vivant -f "$REPO/tools/osmo-panel.py" || \
        "$REPO/tools/osmo-panel.py" >>/tmp/osmo-panel.log 2>&1 &
    osmo_vivant -f "$REPO/tools/osmo-dino.py" || \
        "$REPO/tools/osmo-dino.py" >>/tmp/osmo-dino.log 2>&1 &
    # Timeslot activity probe (osmo-bsc conf + VTY) for the banner
    osmo_vivant -f "$REPO/tools/osmo-ts-probe.py" || \
```

```
        "$REPO/tools/osmo-ts-probe.py" >>/tmp/osmo-ts-probe.log 2>&1 &
    # Live LAB GSM top zone (blinking timeslots + app welcome)
    osmo_vivant -f "$REPO/tools/osmo-topzone.py" || \
        "$REPO/tools/osmo-topzone.py" >>/tmp/osmo-topzone.log 2>&1 &
    # The categorised launcher bar at the bottom

    osmo_vivant -f "$REPO/tools/osmo-launcher.py" || \
        "$REPO/tools/osmo-launcher.py" >>/tmp/osmo-launcher.log 2>&1 &
    osmo_vivant -x conky || \
        conky --daemonize -c "$REPO/configs/conky/osmo-conky.conf" >>/tmp/osmo-conky.log 2>&1
    sleep 5
done
PANEL
    chmod 755 "$w"
    # restart: kill the old wrapper, the autostart (or the loop) picks up the new one
    pkill -f /usr/local/bin/osmo-desktop-panel 2>/dev/null || true
    echo "  [bureau] wrapper osmo-desktop-panel remis a jour (zone LAB GSM vivante, barre de lancement, widgets
     ↪  suivis par display)"
    return 0
}
osmo_reposer_bureau_interactif

# ── THE TWO WEB CONSOLES, ENABLED BY DEFAULT (NATIVE SIDE) ──────────────────
# The ISO enables them at build time (iso_modules/60-dashboard.sh,
# iso_modules/82-services.sh); installed machines get the missing units here.
# `systemctl mask` is the way to opt out: a plain `disable` is undone at the
# next update, a masked unit is left alone.
# Neither unit is STARTED here: osmo-egprs-web follows the bench (PartOf=
# osmo-banc.service) and open5gs-webui has nothing to serve until the WebUI is
# built ("osmo-lte-install --webui").
osmo_activer_consoles_web() {
    [ "$(id -u)" -eq 0 ] || return 0
    command -v systemctl >/dev/null 2>&1 || return 0
    local u src dst etat
    for u in osmo-egprs-web open5gs-webui; do
        src="$REPO/services/$u.service"
        dst="/etc/systemd/system/$u.service"
        [ -f "$src" ] || continue
        etat="$(systemctl is-enabled "$u" 2>/dev/null)"
        [ "$etat" = masked ] && continue          # masked by hand: respected
        if [ ! -f "$dst" ] || ! cmp -s "$src" "$dst"; then
            install -m644 "$src" "$dst"
            systemctl daemon-reload 2>/dev/null || true
            echo "  [web] unite $u.service posee depuis le depot"
        fi
        if [ "$etat" != enabled ]; then
            systemctl enable "$u" >/dev/null 2>&1 \
                && echo "  [web] $u.service active au demarrage" \
                || echo "  [web] $u.service : activation impossible"
        fi
    done
    return 0
}
osmo_activer_consoles_web

# ── FIREFOX ─────────────────────────────────────────────────────────────────
# The dashboard and fft-web open in a browser. Recent images ship Mozilla's
# .deb, in which case this does nothing. Otherwise the Mozilla repository is
# set up and the package installed; the Ubuntu "firefox" package is not a
# fallback, it pulls snapd.
osmo_installer_firefox() {
    [ "$(id -u)" -eq 0 ] || return 0
    command -v firefox >/dev/null 2>&1 && return 0
    command -v apt-get >/dev/null 2>&1 || return 0
    echo "[*] Installation de Firefox (deb Mozilla)..."
    if [ ! -s /etc/apt/keyrings/packages.mozilla.org.asc ]; then
```

```
        install -d -m0755 /etc/apt/keyrings
        curl -fsSL --retry 3 https://packages.mozilla.org/apt/repo-signing-key.gpg \
             -o /etc/apt/keyrings/packages.mozilla.org.asc || {
            echo "[WARN] packages.mozilla.org injoignable - Firefox non installe."
            return 0
        }
    fi
    echo "deb [signed-by=/etc/apt/keyrings/packages.mozilla.org.asc] https://packages.mozilla.org/apt mozilla
     ↪  main" \
        > /etc/apt/sources.list.d/mozilla.list
    printf "Package: *\nPin: origin packages.mozilla.org\nPin-Priority: 1000\n" \
        > /etc/apt/preferences.d/mozilla
    apt-get update -qq 2>/dev/null || true
    if DEBIAN_FRONTEND=noninteractive apt-get install -y firefox >/dev/null 2>&1; then
        echo "[OK] Firefox installe (deb Mozilla)."
    else
        echo "[WARN] Firefox non installe (reseau ?)."
    fi
    return 0
}
osmo_installer_firefox

# ── THE SESSION ACCOUNT MUST SEE DOCKER ──────────────────────────────────
# Conky (tools/conky-osmo-status.sh) and the panel (tools/osmo-panel.py) run
# under the SESSION account and probe each containerised operator with
# "docker exec"; without the docker group the probe reports every operator as
# stopped. The group is set at installation (users.conf, defaultGroups) and by
# addition.sh (_docker_groupe_session); this covers installed machines.
# Same user detection as addition.sh: SUDO_USER, then PKEXEC_UID, then the
# owner of an active session bus (update.sh is launched by an icon via pkexec
# as well as by sudo).
osmo_docker_groupe_session() {
    [ "$(id -u)" -eq 0 ] || return 0
    getent group docker >/dev/null 2>&1 || return 0
    local u uid bus vus=""
    for u in "${SUDO_USER:-}" \
             "$([ -n "${PKEXEC_UID:-}" ] && getent passwd "$PKEXEC_UID" | cut -d: -f1)"; do
        [ -n "$u" ] && [ "$u" != root ] && getent passwd "$u" >/dev/null 2>&1 && { vus="$u"; break; }
    done
    if [ -z "$vus" ]; then
        for bus in /run/user/*/bus; do
            [ -S "$bus" ] || continue
            uid="${bus#/run/user/}"; uid="${uid%/bus}"
            [ "$uid" = 0 ] && continue
            u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
            [ -n "$u" ] && vus="$vus $u"
        done
    fi
    for u in $vus; do
        id -nG "$u" 2>/dev/null | tr ' ' '\n' | grep -qx docker && continue
        usermod -aG docker "$u" 2>/dev/null || continue
        echo "[OK] $u ajoute au groupe docker (le Conky et l encart voient les conteneurs)."
        echo "     Effectif au prochain login de $u."
    done
}
osmo_docker_groupe_session

# ── osmo-* UNITS: User=osmocom -> SESSION ACCOUNT (CATCH-UP) ─────────────
# Mirror of addition.sh (_osmo_unites_user_session). The upstream .service
# files (osmo-bsc/bts-trx/bts-virtual/msc) set User=osmocom / Group=osmocom,
# and that account does not exist natively (iso_modules/52-qemu.sh removes it)
# -> 217/USER, silent Restart=always crash loop. A drop-in
# /etc/systemd/system/<unit>.d/ hands the units to the session account; the
# upstream .service stays intact. Same user detection as
# osmo_docker_groupe_session.
osmo_unites_user_session() {
```

```
    [ "$(id -u)" -eq 0 ] || return 0
    command -v systemctl >/dev/null 2>&1 || return 0
    local u uid bus grp unit base d n=0 vus=""
    for u in "${SUDO_USER:-}" \
             "$([ -n "${PKEXEC_UID:-}" ] && getent passwd "$PKEXEC_UID" | cut -d: -f1)"; do
        [ -n "$u" ] && [ "$u" != root ] && getent passwd "$u" >/dev/null 2>&1 && { vus="$u"; break; }
    done
    if [ -z "$vus" ]; then
        for bus in /run/user/*/bus; do
            [ -S "$bus" ] || continue
            uid="${bus#/run/user/}"; uid="${uid%/bus}"
            [ "$uid" = 0 ] && continue
            u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
            [ -n "$u" ] && { vus="$u"; break; }
        done
    fi
    [ -n "$vus" ] && getent passwd "$vus" >/dev/null 2>&1 || vus=root
    grp="$(id -gn "$vus" 2>/dev/null || echo "$vus")"
    for unit in /lib/systemd/system/osmo-*.service /etc/systemd/system/osmo-*.service; do
        [ -f "$unit" ] || continue
        grep -qE '^(User|Group)=osmocom$' "$unit" || continue
        base="$(basename "$unit")"; d="/etc/systemd/system/$base.d"
        mkdir -p "$d"
        printf '[Service]\nUser=%s\nGroup=%s\n' "$vus" "$grp" > "$d/00-session-user.conf"
        n=$((n+1))
    done
    [ "$n" -gt 0 ] || return 0
    systemctl daemon-reload 2>/dev/null || true
    # osmocom state/log directories -> session account, otherwise "Unable to
    # create file /var/log/osmocom/*.log" and the .cfg parse fails (status=1).
    for _d in /var/log/osmocom /var/lib/osmocom /run/osmocom; do
        [ -e "$_d" ] && chown -R "$vus:$grp" "$_d" 2>/dev/null || true
    done
    for base in osmo-msc osmo-bsc osmo-bts-trx osmo-bts-virtual; do
        systemctl cat "$base" >/dev/null 2>&1 || continue
        systemctl reset-failed "$base" 2>/dev/null || true
        systemctl try-restart "$base" 2>/dev/null || true
    done
    echo "[OK] unites osmo-* rendues au compte de session ($vus) - sinon User=osmocom -> 217/USER."
    return 0
}
osmo_unites_user_session

# ── THE 600 ECHO TEST SOUNDS BAD: THE GAPK SUPERVISOR ──────────────────
# lib/audio.sh prefers /etc/osmocom/gapk-start.sh, a COPY installed by the
# Dockerfile/ISO that git pull does not touch: re-install it from
# scripts/gapk-start.sh, kill a stray osmo-gapk transmitter (the mobile's
# gapk io-handler already carries the uplink) and restart the supervisor
# (tmux session "gapk") so it reads the new version.
osmo_poser_correctif_audio() {
    [ "$(id -u)" -eq 0 ] || return 0
    # NB: two separate 'local' statements: under 'set -u', bash evaluates all
    # right-hand sides of one 'local' before assigning the first one.
    local d=/opt/GSM/osmo-operator
    local src="$d/scripts/gapk-start.sh" dst=/etc/osmocom/gapk-start.sh
    [ -f "$src" ] || return 0
    if [ -d /etc/osmocom ] && ! cmp -s "$src" "$dst"; then
        install -m 755 "$src" "$dst" && echo "[OK] $dst repose depuis scripts/ (emetteur RTP en doublon retire)"
    fi
    # A running osmo-gapk is the old TX (the mobile never starts one): stop it.
    pkill -x osmo-gapk 2>/dev/null && echo "[OK] osmo-gapk en doublon arrete"
    rm -f /var/run/gapk-tx.pid /var/run/gapk-rx.pid
    # The supervisor is only restarted when it already runs: off bench, nothing to do.
    if tmux has-session -t gapk 2>/dev/null; then
        local log_dir; log_dir="$(tmux list-panes -t gapk -F '#{pane_start_command}' 2>/dev/null \
                                | grep -oE 'tee [^ ]+/gapk-auto.log' | awk '{print $2}' | xargs -r dirname)"
```

```
    [ -n "$log_dir" ] || log_dir="${LOG_DIR:-/run/user/0/osmo-nitb/logs}"
    local sock="${PULSE_SOCK:-/var/run/pulse/native}"
    tmux kill-session -t gapk 2>/dev/null || true
    rm -f /var/run/gapk-auto.lock
    [ -x "$dst" ] || dst="$src"
    tmux new-session -d -s gapk \
        "GAPK_ALSA_DEV=gsm_out GAPK_ALSA_DEV_IN=gsm_in PULSE_SERVER=unix:${sock} bash '$dst' auto gsm gsm_out
         ↪  gsm_in 2>&1 | tee ${log_dir}/gapk-auto.log" \
        && echo "[OK] superviseur gapk relance (tmux gapk, journal ${log_dir}/gapk-auto.log)"
fi
# ── The 600 from the phone: no announcement before the echo ──
# configs/extensions.conf skips the announcement when ECHO_NO_PROMPT=1
# (set by osmo-phonesim-banc.py); the live copy /etc/asterisk/extensions.conf
# is only regenerated at start-up, so it is patched here and reloaded.
local ext=/etc/asterisk/extensions.conf
if [ -f "$ext" ] && ! grep -q 'ECHO_NO_PROMPT' "$ext"; then
    sed -i '/^exten => 600,1,NoOp(=== ECHO TEST/,/^ same => n,Hangup()/{
        s|^ same => n,Playback(demo-echotest)$| same => n,GotoIf($["${ECHO_NO_PROMPT}" = "1"]?echo)\n same =>
↪ n,Playback(demo-echotest)|
        s|^ same => n,Echo()$| same => n(echo),Echo()|
    }' "$ext" && echo "[OK] $ext : le 600 saute son annonce pour le banc (ECHO_NO_PROMPT)"
fi
# The echo test announcement, replayed on the mobile leg (Dial ...rT${DIAL_OPTS};
# the bench sets DIAL_OPTS=A(demo-echotest) for 600): [internal] context only.
if [ -f "$ext" ] && ! grep -q 'rT\${DIAL_OPTS}' "$ext"; then
    sed -i '/^\[internal\]/,/^\[/{s|^ same => n,Dial(PJSIP/${EXTEN}@gsm_msc,,rT)$| same =>
     ↪  n,Dial(PJSIP/${EXTEN}@gsm_msc,,rT${DIAL_OPTS})|}' "$ext" \
        && echo "[OK] $ext : annonce de l echo test sur la patte du mobile (DIAL_OPTS)"
fi
command -v asterisk >/dev/null 2>&1 && asterisk -rx "dialplan reload" >/dev/null 2>&1 || true
# The bench modem (osmo-phonesim-banc.py --connect) runs from the repository
# but does not re-read its code: restart it exactly as osmo-pmos-setup.sh
# launched it, so it picks up ECHO_NO_PROMPT.
local bpid
bpid="$(pgrep -f '[o]smo-phonesim-banc.py --connect' | head -1)"
if [ -n "$bpid" ]; then
   local bargs; bargs="$(tr '\0' ' ' <"/proc/$bpid/cmdline" 2>/dev/null | sed 's/.*osmo-phonesim-banc.py //')"
    kill "$bpid" 2>/dev/null; sleep 1
    setsid nohup env OSMO_AT_LOG_OFONO=/tmp/osmo-at-vm.log \
        "$d/tools/osmo-phonesim-banc.py" $bargs >>/tmp/osmo-phonesim-vm-root.log 2>&1 </dev/null &
    sleep 3
    pgrep -f '[o]smo-phonesim-banc.py --connect' >/dev/null 2>&1 \
        && echo "[OK] modem du banc relance ($bargs) - journal /tmp/osmo-phonesim-vm-root.log" \
        || echo "[!!] le modem du banc ne s est pas relance : osmo-pmos-setup pour le rebrancher"
fi
# ── Saturated, bassy microphone: the echo canceller's analog AGC ──
# module-echo-cancel (webrtc) drives the microphone gain and makes it pump
# (see tools/osmo-pmos-setup.sh). If the canceller still runs without
# aec_args, reload it identically with the analog AGC disabled and move
# the VM's streams back onto it.
export PULSE_SERVER="${PULSE_SERVER:-unix:/var/run/pulse/native}"
# Only a WEBRTC canceller without aec_args: speex (the default, lib/audio.sh)
# has none and must not be switched to webrtc.
if command -v pactl >/dev/null 2>&1 && pactl list short modules 2>/dev/null | grep module-echo-cancel | grep
 ↪  -q aec_method=webrtc \
   && ! pactl list short modules 2>/dev/null | grep module-echo-cancel | grep -q analog_gain_control=0; then
    local ecm mic hp so si
    ecm="$(pactl list short modules | awk '/module-echo-cancel/ {print $1; exit}')"
    mic="$(pactl list short modules | grep module-echo-cancel | grep -oE 'source_master=[^ ]+' | cut -d= -f2)"
    hp="$(pactl list short modules | grep module-echo-cancel | grep -oE 'sink_master=[^ ]+' | cut -d= -f2)"
    if [ -n "$mic" ] && [ -n "$hp" ]; then
        pactl unload-module "$ecm" 2>/dev/null
        pactl load-module module-echo-cancel aec_method=webrtc source_master="$mic" sink_master="$hp" \
            source_name=osmo_mic_ec sink_name=osmo_hp_ec \
            'aec_args="analog_gain_control=0 digital_gain_control=1 noise_suppression=1 high_pass_filter=1"' \
```

```
                source_properties=device.description=Micro_sans_echo
↪  sink_properties=device.description=HP_sans_echo >/dev/null 2>&1 \
                && echo "[OK] annuleur d echo recharge sans AGC analogique (micro fixe a 25 %)"
            pactl set-default-source osmo_mic_ec 2>/dev/null; pactl set-default-sink osmo_hp_ec 2>/dev/null
            for so in $(pactl list source-outputs 2>/dev/null | awk '/Source Output #/{id=$3} /media.name =
             ↪  "combine"/{print id}' | tr -d '#'); do
                pactl move-source-output "$so" osmo_mic_ec 2>/dev/null; done
            for si in $(pactl list sink-inputs 2>/dev/null | awk '/Sink Input #/{id=$3} /media.name =
             ↪  "combine"/{print id}' | tr -d '#'); do
                pactl move-sink-input "$si" osmo_hp_ec 2>/dev/null; done
            pactl set-source-mute "$mic" 0 2>/dev/null; pactl set-source-volume "$mic" 25% 2>/dev/null
            pactl set-source-mute osmo_mic_ec 0 2>/dev/null; pactl set-source-volume osmo_mic_ec 25% 2>/dev/null
        fi
    fi
    # ── speech-dispatcher's dummy message in the speakers ──
    # See lib/audio.sh (osmo_tts_off): create the discard sink if missing and
    # move any open speech-dispatcher stream onto it.
    if command -v pactl >/dev/null 2>&1 && pactl info >/dev/null 2>&1; then
        pactl list short sinks 2>/dev/null | grep -qw osmo_tts_off \
            || pactl load-module module-null-sink sink_name=osmo_tts_off format=s16le rate=8000 channels=1
             ↪  sink_properties=device.description=TTS_off >/dev/null 2>&1
        for si in $(pactl list sink-inputs 2>/dev/null | awk '/Sink Input #/{id=$3} /application.name =
         ↪  "speech-dispatcher/{print id}' | tr -d '#'); do
            pactl move-sink-input "$si" osmo_tts_off 2>/dev/null && echo "[OK] flux speech-dispatcher detourne des
             ↪  haut-parleurs (osmo_tts_off)"
        done
    fi
    return 0
}
osmo_poser_correctif_audio

# ── THE DESKTOP srsUE CONSOLE: 4G LOGS ──────────────────────────────────────
# osmo-lte.sh sets --log.filename=/tmp/osmo-lte-ue.log and the console in
# /tmp/osmo-lte-ue.console. Set NAS to info (no hex dump) in an already
# installed ue.conf (osmo-lte-install --configs does not overwrite it without
# --force) and restart the osmo-fft-snap panel, which reads its paths at
# start-up. The UE itself is not restarted: the levels apply at the next
# "osmo-lte restart".
osmo_poser_journaux_lte() {
    [ "$(id -u)" -eq 0 ] || return 0
    local f
    for f in /root/.config/srsran/ue.conf /home/*/.config/srsran/ue.conf; do
        [ -f "$f" ] || continue
        grep -q '^nas_hex_limit *= *0' "$f" && continue
        sed -i -e '/^rrc_level *= *info$/d' -e '/^nas_level *= *info$/d' \
               -e '/^\[log\]/,/^\[/{s/^\(all_level *= *warning\)$/\1\nnas_level = info\nas_hex_limit = 0/}' \
               -e 's|^filename *= */tmp/ue\.log$|filename = /tmp/osmo-lte-ue.log|' "$f" \
            && echo "[OK] $f : NAS en info (console srsUE du bureau)"
    done
    if systemctl is-active --quiet osmo-fft-snap 2>/dev/null; then
       systemctl try-restart osmo-fft-snap 2>/dev/null && echo "[OK] encart osmo-fft-snap relance (console srsUE)"
    fi
    return 0
}
osmo_poser_journaux_lte

# ── TCH FRAME FORMAT AND MOBILE BUFFER, IN THE FORKS ────────────────────────
# Two vital voice settings live in repositories that the update re-fetches
# (qosmo-grgsm, qosmo-dsp; "git reset --hard FETCH_HEAD"), so git pull on
# osmo-operator cannot carry them:
#
#  1. "io-tch-format rtp" in the forks' cfgs/mobile_group1.cfg, the template
#     that run.sh (run_modules/20-mobile-cfg.sh) copies over
#     ~/.osmocom/bb/mobile_group1.cfg at every start. Without it the mobile
#     uses the Texas Instruments TCH frame layout while pont.py encodes
#     RTP/RFC3551 with libosmocoding: no counter complains, the voice is
```

```
#     robotic.
#
#  2. CALYPSO_PULSE_LATENCY_MSEC in run_modules/70-l2.sh and
#     68-sidecar-mobile.sh. The mobile writes one 20 ms GSM frame per
#     snd_pcm_writei; with a too short ALSA-PulseAudio buffer every write
#     returns -EPIPE and the PulseAudio stream is torn down and rebuilt 50
#     times per second. 200 ms or more is enough (start-native.sh exports 320).
#
# scripts/voix-forks.sh is the only place that applies both settings; it is
# also called by the osmo-update service (after its fetch) and by
# iso_modules/70-scripts.sh (on the rootfs).
# Check: `grep io-tch-format ~/.osmocom/bb/mobile_group1.cfg` after a start,
# and the mobile's sink-input index must stay fixed during a call
# (`pactl list short sink-inputs`, two readings 1 s apart).
osmo_poser_voix_forks() {
    [ "$(id -u)" -eq 0 ] || return 0
    local s=/opt/GSM/osmo-operator/scripts/voix-forks.sh
    [ -x "$s" ] || return 0
    "$s" | sed 's/^  \[voix\]/[OK]/'
    return 0
}
osmo_poser_voix_forks

# ── gnuarm, GCC ALTERNATIVES, c123 AND banc-*: CATCH-UP ──────────────────
# What build-iso.sh (70-scripts, 80-chroot) and the native install set up,
# replayed on an installed machine. No apt, no network: only compilers already
# present enter the alternatives.
#   - /etc/profile.d/97-osmo-gnuarm.sh: /root/gnuarm/install/bin in PATH
#     (arm-elf-gcc 4.5.2 from addition.sh --gnuarm), no effect while absent;
#   - update-alternatives gcc: gcc-9 (90), gcc-11 (110), gcc-13 (130). An
#     existing choice (update-alternatives --config gcc) is kept; on creation,
#     gcc-13, the kernel compiler, for DKMS;
#   - c123 / osmo-c123 and the bench commands (banc-rssi, banc-trx, c123-1...)
#     in /usr/local/bin, 98-osmo-banc-aliases.sh in profile.d;
#   - /usr/local/bin/osmo-glava (a copy, not a link) aligned on tools/osmo-glava.sh.
osmo_poser_outils_banc() {
    [ "$(id -u)" -eq 0 ] || return 0
    local r=/opt/GSM/osmo-operator v
    [ -d "$r" ] || return 0
    [ -f "$r/scripts/osmo-gnuarm-profile.sh" ] \
        && install -m644 "$r/scripts/osmo-gnuarm-profile.sh" /etc/profile.d/97-osmo-gnuarm.sh
    [ -f "$r/scripts/osmo-banc-aliases.sh" ] \
        && install -m644 "$r/scripts/osmo-banc-aliases.sh" /etc/profile.d/98-osmo-banc-aliases.sh
    # WAN WireGuard ("wg 1", "wg N ..."); the package itself needs apt: native install / ISO
    [ -f "$r/scripts/osmo-wg-profile.sh" ] \
        && install -m644 "$r/scripts/osmo-wg-profile.sh" /etc/profile.d/97-osmo-wg.sh \
        && ln -sf "$r/tools/osmo-wg.sh" /usr/local/bin/osmo-wg
    [ -x "$r/tools/osmo-banc-cmd.sh" ] && bash "$r/tools/osmo-banc-cmd.sh" --install /usr/local/bin >/dev/null
    for v in c123 osmo-c123; do
        [ "$(readlink "/usr/local/bin/$v" 2>/dev/null)" = "$r/tools/osmo-c123-gtk.py" ] \
            || ln -sf "$r/tools/osmo-c123-gtk.py" "/usr/local/bin/$v"
    done
    [ -f /usr/local/bin/osmo-glava ] && [ -f "$r/tools/osmo-glava.sh" ] \
        && ! cmp -s "$r/tools/osmo-glava.sh" /usr/local/bin/osmo-glava \
        && install -m755 "$r/tools/osmo-glava.sh" /usr/local/bin/osmo-glava \
        && echo "  [bureau] osmo-glava aligne sur le depot (systemctl --user restart glava)"
    # gcc-9 / gcc-11 / gcc-13 alternatives (single definition: tools/osmo-gcc-alternatives.sh)
    [ -x "$r/tools/osmo-gcc-alternatives.sh" ] && bash "$r/tools/osmo-gcc-alternatives.sh"
    return 0
}
osmo_poser_outils_banc

# ── PATCHES FOR THE /opt/GSM TREES (patches/) ────────────────────────────
# The trees installed by the ISO (osmocom-bb, osmo-trx...) receive the
# repository patches: idempotent, offline. With gnuarm (addition.sh --gnuarm),
# "cd /opt/GSM/osmocom-bb/src && make" then builds the GPRS layer1.
```

```
osmo_poser_correctifs() {
    [ "$(id -u)" -eq 0 ] || return 0
    local r=/opt/GSM/osmo-operator
    [ -x "$r/tools/osmo-patches-apply.sh" ] || return 0
    OSMO_PATCHES_QUIET=1 bash "$r/tools/osmo-patches-apply.sh" /opt/GSM 2>/dev/null | sed 's/^ */[OK] /'
    return 0
}
osmo_poser_correctifs

case "${1:-}" in
    --quiet) exit 0 ;;
esac

# Remove a stale global http.version setting if still present (see Dockerfile).
git config --global --unset http.version 2>/dev/null || true


# ── WITHOUT A TERMINAL: A GTK WINDOW ──────────────────────────────────────
# Launched from the desktop icon the script never has a tty; the catch-up
# steps above are already done when we get here. With a display server the
# result is shown through zenity (no cursor sequences outside a terminal);
# without terminal or display (cron, ssh without X, /etc/profile.d on a
# console) exit silently with code 0. zenity comes with the desktop variant
# (build-iso.sh installs it): its absence means an image without a desktop.
if [ ! -t 1 ]; then
    if [ -n "${DISPLAY:-}${WAYLAND_DISPLAY:-}" ] && command -v zenity >/dev/null 2>&1; then
        # --pulsate, not a percentage: there is no progress left to measure.
        # --auto-close so the window leaves with the stream, no button to click.
        {
            echo "# Recherche de la cellule (ARFCN)..." ; sleep 0.6
            echo "# Raccourcis du bureau reposes."      ; sleep 0.6
            echo "# Envoi du SMS de test..."            ; sleep 0.6
        } | zenity --progress --pulsate --auto-close --no-cancel \
                   --title="osmo-operator - mise a jour" \
                   --width=420 --text="Mise a jour en cours..." 2>/dev/null
        zenity --info --title="osmo-operator" --width=420 \
               --text="<b>SMS delivered</b> - MT end-to-end\n\nMessage : Bastien phone home\n\nLes raccourcis du
                ↪  bureau ont ete reposes." \
               2>/dev/null
    fi
    exit 0
fi

printf '\033[?25l'
trap 'printf "\033[?25h"' EXIT

ph='\033[1;33m☎\033[0m'
bars=('\033[2m___\033[0m' '\033[1;32m▄\033[0m\033[2m__\033[0m' '\033[1;32m▄▆\033[0m\033[2m_\033[0m'
↪  '\033[1;32m▄▆█\033[0m')
for b in "${bars[@]}"; do
    printf '\r  %b %b  \033[36mscanning ARFCN...\033[0m   ' "$ph" "$b"
    sleep 0.12
done
for ((p=0; p<=20; p++)); do
    printf '\r\033[K  %b %*s\033[1;36m✉\033[0m%*s %b' "$ph" "$p" '' "$((20-p))" '' "$ph"
    sleep 0.04
done
printf '\r\033[K  %b%21s%b  \033[1;32m✓ SMS delivered - MT end-to-end Message : Bastien phone home\033[0m\n'
↪  "$ph" '' "$ph"
```

2.17 /opt/GSM/osmo-operator/armbian/build-armbian.sh

16400 octets, 269 lignes → 269 lignes

```
#!/bin/bash
# armbian/build-armbian.sh - build an arm64 SD image with the official Armbian framework
#                            (github.com/armbian/build) and install the osmo-operator stack on it.
#
# Armbian builds the kernel, BSP, u-boot/firmware, rootfs and image; this script only adds the
# stack through Armbian's userpatches mechanism:
#     userpatches/customize-image.sh   runs inside the image chroot (under qemu-user on an x86 host)
#     userpatches/overlay/             mounted at /tmp/overlay inside that chroot
#
# The stack comes from the project's arm64 Docker image (Dockerfile, build.sh --arch=arm64:
# osmocom-nitb:arm64, or the published base ghcr.io/<repo>/osmocom-nitb:base-arm64-<hash>), the same
# content iso_modules/50-injection-image.sh uses for the ISO: build .debs (osmo-build-*), /usr/local,
# /opt/GSM, /etc/osmocom, systemd units. It is exported into the overlay with docker cp.
# What an x86 host cannot produce (toast, qosmo launcher, c54x_exe/grgsm_exe, bench .debs) is
# compiled inside the chroot by customize-image.sh.
#
# Usage:
#   sudo ./build-armbian.sh                       rpi4b, branch current, noble: clone, overlay, compile
#   sudo ./build-armbian.sh --prepare             everything except compile.sh (check userpatches/, overlay)
#   sudo ./build-armbian.sh --no-stack            plain Armbian (no overlay, no customize)
#   sudo ./build-armbian.sh --board=orangepi5 --branch=edge --release=noble
#   sudo ./build-armbian.sh --image=osmocom-nitb:arm64      Docker image providing the stack
#
# Options:
#   --board=              one board, a comma-separated list, or a group:
#                           rpi4   rpi4b (Raspberry Pi 4)
#                           opi5   all Armbian Orange Pi 5 boards: orangepi5, orangepi5b, orangepi5-plus,
#                                  orangepi5pro, orangepi5-max, orangepi5-ultra
#                           all    rpi4 + opi5
#                         One image per board, built sequentially from the same overlay. Kernel branch
#                         is chosen per board: --branch if offered (KERNEL_TARGET in
#                         config/boards/<board>.*), else current, else the board's first branch.
#                         Community boards (.csc) are built with EXPERT=yes.
#   --desktop             GNOME desktop (DESKTOP_ENVIRONMENT=gnome, DESKTOP_TIER=minimal, via
#                         armbian/configng) + the bench desktop (wallpaper, Conky, panel, icons, c123)
#   --banc / --multi      enable osmo-banc.service / osmo-multi.service at boot
#   --build-image         if no arm64 stack Docker image exists, build it (build.sh --arch=arm64)
#                         instead of failing
#   --armbian-dir=PATH    armbian/build clone                [/opt/GSM/armbian-build]
#   --update              git pull the clone before building
#   --install-deps        apt install qemu-user-static binfmt-support if aarch64 cannot be emulated
#   --dry-run             print what would be done
#
# Env: ARMBIAN_DIR, ARMBIAN_GIT, OSMO_DEB_CACHE (/var/cache/osmo-debs), OSMO_ARM_GLAVA=1.
# Host requirements: docker (compile.sh containerizes itself on non-Ubuntu/x86 hosts), git, and
# aarch64 emulation (qemu-user-static + binfmt) so the chroot runs on x86 (not needed on arm64).
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO="$(cd "$HERE/.." && pwd)"

BOARD=rpi4b; BRANCH=current; RELEASE=noble
ARMBIAN_DIR="${ARMBIAN_DIR:-/opt/GSM/armbian-build}"
ARMBIAN_GIT="${ARMBIAN_GIT:-https://github.com/armbian/build}"
IMAGE=""; PREPARE=0; NO_STACK=0; UPDATE=0; INSTALL_DEPS=0; DRY=0
DESKTOP=0; BANC=0; MULTI=0; BUILD_IMAGE=0; BRANCH_GIVEN=0
DEB_CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
for arg in "$@"; do
    case "$arg" in
        --board=*)       BOARD="${arg#*=}" ;;
        --branch=*)      BRANCH="${arg#*=}"; BRANCH_GIVEN=1 ;;
        --desktop)       DESKTOP=1 ;;
        --banc)          BANC=1 ;;
        --multi)         MULTI=1 ;;
        --build-image)   BUILD_IMAGE=1 ;;
        --release=*)     RELEASE="${arg#*=}" ;;
        --image=*)       IMAGE="${arg#*=}" ;;
```

```
        --armbian-dir=*) ARMBIAN_DIR="${arg#*=}" ;;
        --prepare)       PREPARE=1 ;;
        --no-stack)      NO_STACK=1 ;;
        --update)        UPDATE=1 ;;
        --install-deps)  INSTALL_DEPS=1 ;;
        --dry-run)       DRY=1 ;;
        -h|--help)       sed -n '2,46p' "$0"; exit 0 ;;
        *) echo "option inconnue : $arg" >&2; sed -n '2,46p' "$0" >&2; exit 2 ;;
    esac
done
say()  { printf '\033[1;32m[armbian]\033[0m %s\n' "$*"; }
warn() { printf '\033[1;33m[armbian] !\033[0m %s\n' "$*" >&2; }
die()  { printf '\033[1;31m[armbian] ECHEC\033[0m %s\n' "$*" >&2; exit 1; }
run()  { if [ "$DRY" = 1 ]; then echo "  + $*"; else "$@"; fi; }

# ── 1. Host checks ──────────────────────────────────────────────────
[ "$(id -u)" = 0 ] || [ "$DRY" = 1 ] || die "a lancer en root (compile.sh d Armbian l exige) : sudo $0 $*"
command -v git >/dev/null    || die "git absent"
command -v docker >/dev/null || warn "docker absent : compile.sh ne pourra pas se conteneuriser (il faut alors un
↪  hote Ubuntu avec ses dependances)"
HOST_ARCH="$(dpkg --print-architecture 2>/dev/null || uname -m)"
if [ "$HOST_ARCH" != arm64 ] && [ "$NO_STACK" = 0 ]; then
    if [ ! -e /proc/sys/fs/binfmt_misc/qemu-aarch64 ]; then
        if [ "$INSTALL_DEPS" = 1 ]; then
            say "emulation aarch64 absente : apt install qemu-user-static binfmt-support"
            run apt-get install -y qemu-user-static binfmt-support
        elif [ "$DRY" = 1 ] || [ "$PREPARE" = 1 ]; then
            warn "pas d emulation aarch64 sur cet hote $HOST_ARCH : avant compile.sh, sudo $REPO/build.sh --arm
            ↪  (pose tous les paquets hote) ou --install-deps"
        else
            die "pas d emulation aarch64 sur cet hote $HOST_ARCH (/proc/sys/fs/binfmt_misc/qemu-aarch64 absent) :
            ↪  customize-image.sh ne pourrait pas tourner dans le chroot arm64. sudo $REPO/build.sh --arm pose
            ↪  tous les paquets hote (ou --install-deps)"
        fi
    fi
fi
OPI5_BOARDS="orangepi5 orangepi5b orangepi5-plus orangepi5pro orangepi5-max orangepi5-ultra"
BOARDS=""
for b in ${BOARD//,/ }; do
    case "$b" in
        rpi4|rpi4b) BOARDS="$BOARDS rpi4b" ;;
        opi5|orangepi5-all) BOARDS="$BOARDS $OPI5_BOARDS" ;;
        all)        BOARDS="$BOARDS rpi4b $OPI5_BOARDS" ;;
        *)          BOARDS="$BOARDS $b" ;;
    esac
done
BOARDS=$(echo $BOARDS | tr ' ' '\n' | awk '!vu[$0]++' | tr '\n' ' ')
say "hote $HOST_ARCH ($(nproc) coeurs) ; cartes : $BOARDS; RELEASE=$RELEASE ; bureau=$([ "$DESKTOP" = 1 ] && echo
↪  gnome || echo non) ; clone $ARMBIAN_DIR"

# ── 2. Armbian framework ────────────────────────────────────────────
if [ ! -f "$ARMBIAN_DIR/compile.sh" ]; then
    say "clone de $ARMBIAN_GIT (branche main, depth 1) -> $ARMBIAN_DIR"
    run git clone --depth 1 --branch main "$ARMBIAN_GIT" "$ARMBIAN_DIR"
elif [ "$UPDATE" = 1 ]; then
    say "mise a jour du clone"
    run git -C "$ARMBIAN_DIR" pull --ff-only
fi
# board_file BOARD: Armbian board file (.conf supported, .csc community, .wip, ...)
board_file() {
    local e
    for e in conf csc wip tvb eos; do
        [ -f "$ARMBIAN_DIR/config/boards/$1.$e" ] && { echo "$ARMBIAN_DIR/config/boards/$1.$e"; return 0; }
    done
    return 1
}
```

```
# board_branch BOARD: --branch if the board offers it, else current, else its first branch
board_branch() {
    local f targets
    f=$(board_file "$1") || { echo "$BRANCH"; return; }
    targets=$(sed -n 's/^KERNEL_TARGET="\(.*\)".*/\1/p' "$f" | head -1)
    [ -n "$targets" ] || { echo "$BRANCH"; return; }
    case ",$targets," in
        *",$BRANCH,"*) echo "$BRANCH" ;;
        *",current,"*) [ "$BRANCH_GIVEN" = 1 ] && warn "$1 : pas de branche $BRANCH ($targets), current a la
         ↪  place" >&2; echo current ;;
        *) warn "$1 : pas de branche $BRANCH$([ "$BRANCH" = current ] || echo " ni current") ($targets),
         ↪  ${targets%%,*} a la place" >&2; echo "${targets%%,*}" ;;
    esac
}
if [ "$DRY" = 0 ]; then
    for b in $BOARDS; do
        board_file "$b" >/dev/null || die "carte inconnue d Armbian : $b (voir $ARMBIAN_DIR/config/boards/)"
    done
fi

# ── 3. userpatches: customization script and overlay ──────────────────────
UP="$ARMBIAN_DIR/userpatches"
OV="$UP/overlay/osmo"
run install -d "$UP" "$OV"
if [ "$NO_STACK" = 1 ]; then
    say "--no-stack : pas de customize-image.sh (Armbian nue)"
    run rm -f "$UP/customize-image.sh"
else
    run install -m755 "$HERE/userpatches/customize-image.sh" "$UP/customize-image.sh"
    say "userpatches/customize-image.sh pose"
fi

# ── 4. Overlay: the stack, exported from the arm64 Docker image ───────────
if [ "$NO_STACK" = 0 ]; then
    if [ -z "$IMAGE" ]; then
        if docker image inspect osmocom-nitb:arm64 >/dev/null 2>&1; then
            IMAGE=osmocom-nitb:arm64
        else
            KEY=$(cd "$REPO" && git ls-files -- Dockerfile packaging configs scripts services helpers opt-gsm
             ↪  tools patches | sort | xargs sha256sum | sha256sum | cut -c1-16)
            REMOTE=$(cd "$REPO" && git remote get-url origin 2>/dev/null | sed -E 's#.*github.com[:/]##;
             ↪  s#\.git$##' | tr '[:upper:]' '[:lower:]')
            GHCR="ghcr.io/${REMOTE:-bbaranoff/osmo-operator}/osmocom-nitb:base-arm64-$KEY"
            if [ "$DRY" = 0 ] && docker pull --platform linux/arm64 "$GHCR" >/dev/null 2>&1; then
                IMAGE="$GHCR"
            elif [ "$DRY" = 1 ]; then
                warn "aucune image Docker arm64 de la pile en local ($GHCR non testee en dry-run) : il faudra d
                 ↪  abord sudo $REPO/build.sh --arch=arm64"
                IMAGE="osmocom-nitb:arm64"
            elif [ "$BUILD_IMAGE" = 1 ]; then
               say "aucune image Docker arm64 de la pile : build.sh --arch=arm64 (emule sur un hote x86 : ~10 h au
                ↪  premier build)"
                bash "$REPO/build.sh" --arch=arm64 || die "build.sh --arch=arm64 a echoue"
                if docker image inspect osmocom-nitb:arm64 >/dev/null 2>&1; then IMAGE=osmocom-nitb:arm64; else
                 ↪  IMAGE=osmocom-nitb; fi
            else
                die "aucune image Docker arm64 de la pile : ni osmocom-nitb:arm64 en local, ni $GHCR. Sur cet hote
                 ↪  : sudo $REPO/build.sh --arch=arm64 (emule, ~10 h au premier build) ou --build-image, ou lancer
                 ↪  docker-arm.yml sur GitHub (runner arm64 natif) puis relancer ici."
            fi
        fi
    fi
    if [ "$DRY" = 0 ]; then
        ARCH=$(docker image inspect --format '{{.Architecture}}' "$IMAGE" 2>/dev/null || echo "?")
        [ "$ARCH" = arm64 ] || die "l image $IMAGE est $ARCH, pas arm64"
    fi
```

```
    say "pile : image Docker $IMAGE -> overlay $OV"
    if [ "$DRY" = 1 ]; then
        echo "  + docker create/cp -> $OV/debs/ et $OV/rootfs/ (usr/local, opt/GSM, opt/node, root/.env,
        ↪  etc/osmocom, etc/asterisk, lib/systemd/system)"
    else
        rm -rf "$OV"; install -d "$OV/debs" "$OV/rootfs/usr" "$OV/rootfs/opt" "$OV/rootfs/root" "$OV/rootfs/etc"
        ↪  "$OV/rootfs/lib/systemd/system"
        CID=$(docker create --platform linux/arm64 "$IMAGE" /bin/true)
        trap 'docker rm "$CID" >/dev/null 2>&1 || true' EXIT
        # Build .debs for the target suite: host cache first, else those shipped in the image
        cp -f "$DEB_CACHE"/osmo-build-*"~${RELEASE}_arm64.deb" "$OV/debs/" 2>/dev/null || true
        if ! ls "$OV/debs"/*.deb >/dev/null 2>&1; then
            docker cp "$CID:/var/cache/osmo-debs/." "$OV/debs/" 2>/dev/null || true
            find "$OV/debs" -name '*.deb' ! -name "*~${RELEASE}_arm64.deb" -delete 2>/dev/null || true
        fi
        NDEBS=$(find "$OV/debs" -name '*.deb' | wc -l)
        # The overlay is a tree (rootfs/) copied as-is onto / by customize-image.sh: /usr/local,
        # /opt/GSM minus the trees the .debs already install (same list as 50-injection-image.sh),
        # /opt/node, root's environment, /etc/osmocom, /etc/asterisk, systemd units.
        docker cp "$CID:/usr/local" "$OV/rootfs/usr/"
        EXCL=()
        if [ "$NDEBS" -gt 0 ]; then
            for t in osmocom-bb qosmo c54x_exe grgsm_exe qemu-install firmware; do EXCL+=("--exclude=GSM/$t"
            ↪  "--exclude=GSM/$t/*"); done
        fi
        docker cp "$CID:/opt/GSM" - | tar -x -C "$OV/rootfs/opt" "${EXCL[@]+"${EXCL[@]}"}"
        docker cp "$CID:/opt/node" "$OV/rootfs/opt/" 2>/dev/null || true
        for d in .env .venv-qemu; do docker cp "$CID:/root/$d" "$OV/rootfs/root/" 2>/dev/null || true; done
        docker cp "$CID:/etc/osmocom" "$OV/rootfs/etc/" 2>/dev/null || true
        docker cp "$CID:/etc/asterisk" "$OV/rootfs/etc/" 2>/dev/null || true
        for svc in osmo-bts-trx osmo-bsc osmo-msc osmo-hlr osmo-mgw osmo-stp osmo-ggsn osmo-sgsn osmo-pcu
        ↪  osmo-sip-connector; do
           docker cp "$CID:/lib/systemd/system/$svc.service" "$OV/rootfs/lib/systemd/system/" 2>/dev/null || true
        done
        cp -f "$REPO"/services/*.service "$OV/rootfs/lib/systemd/system/" 2>/dev/null || true
        docker rm "$CID" >/dev/null; trap - EXIT
        # /opt/GSM/osmo-operator is taken from this working tree (tracked + untracked, no .git or
        # ignored files), as for the ISO, so local fixes not yet in a published image are included.
        rm -rf "$OV/rootfs/opt/GSM/osmo-operator"; install -d "$OV/rootfs/opt/GSM/osmo-operator"
        (cd "$REPO" && git ls-files -co --exclude-standard -z | tar --null --ignore-failed-read -T - -cf -
        ↪  2>/dev/null) \
            | tar -xf - -C "$OV/rootfs/opt/GSM/osmo-operator"
        # Panel/Conky watcher: the same script as the ISO, extracted from 85-installeur-bureau.sh
        sed -n "/<<'PANEL'$/,/^PANEL$/p" "$REPO/iso_modules/85-installeur-bureau.sh" | sed '1d;$d' >
        ↪  "$OV/osmo-desktop-panel"
        [ -s "$OV/osmo-desktop-panel" ] || warn "osmo-desktop-panel non extrait de
        ↪  iso_modules/85-installeur-bureau.sh : pas de Conky au login"
        printf 'IMAGE=%s\nRELEASE=%s\nBOARDS=%s\nDESKTOP=%s\nDATE=%s\nNDEBS=%s\n' "$IMAGE" "$RELEASE" "$BOARDS"
        ↪  "$DESKTOP" "$(date -Is)" "$NDEBS" > "$OV/SOURCE"
        say "overlay pret : $NDEBS .deb, $(du -sh "$OV" | cut -f1)"
    fi
fi

# Marker files read by customize-image.sh from the overlay (services enabled at boot)
if [ "$NO_STACK" = 0 ] && [ "$DRY" = 0 ]; then
    rm -f "$OV/OSMO_IMAGE_BANC" "$OV/OSMO_IMAGE_MULTI" "$OV/OSMO_ARM_GLAVA"
    [ "${OSMO_ARM_GLAVA:-0}" = 1 ] && : > "$OV/OSMO_ARM_GLAVA"
    [ "$BANC" = 1 ]  && : > "$OV/OSMO_IMAGE_BANC"
    [ "$MULTI" = 1 ] && : > "$OV/OSMO_IMAGE_MULTI"
fi

# ── 5. compile.sh, once per board ──────────────────────────────────────
# The desktop comes from armbian/configng YAML, selected by DESKTOP_ENVIRONMENT + DESKTOP_TIER.
# GNOME minimal pulls gdm3, pulseaudio and Xorg, which Conky and c123 need.
if [ "$DESKTOP" = 1 ]; then
    DESK=(BUILD_DESKTOP=yes DESKTOP_ENVIRONMENT=gnome DESKTOP_TIER=minimal)
```

```
else
    DESK=(BUILD_DESKTOP=no)
fi
IMGS=()
for b in $BOARDS; do
    br=$(board_branch "$b")
    CMD=(./compile.sh build "BOARD=$b" "BRANCH=$br" "RELEASE=$RELEASE" BUILD_MINIMAL=no "${DESK[@]}"
     ↪  KERNEL_CONFIGURE=no COMPRESS_OUTPUTIMAGE=sha,img)
    case "$(board_file "$b" 2>/dev/null)" in *.conf|"") ;; *) CMD+=(EXPERT=yes) ;; esac
    if [ "$PREPARE" = 1 ] || [ "$DRY" = 1 ]; then
        echo "  + (cd $ARMBIAN_DIR && ${CMD[*]})"
        continue
    fi
    say "[$b] compile.sh (long : noyau + rootfs + customize sous qemu-user) : ${CMD[*]}"
    t0=$(date +%s)
    (cd "$ARMBIAN_DIR" && "${CMD[@]}") || die "[$b] compile.sh a echoue"
    IMG=$(find "$ARMBIAN_DIR/output/images" -maxdepth 1 -name '*.img' -newermt "@$t0" 2>/dev/null | head -1)
    [ -n "$IMG" ] || die "[$b] pas de nouvelle image dans $ARMBIAN_DIR/output/images/"
    IMGS+=("$IMG")
    say "[$b] image : $IMG  ($(du -h "$IMG" | cut -f1))"
done
if [ "$PREPARE" = 1 ]; then
    say "--prepare : fini. Pour construire : les commandes ci-dessus, en root"
    exit 0
fi
[ "$DRY" = 1 ] && exit 0
say "images :"; for i in "${IMGS[@]}"; do echo "    $i"; done
say "flash : sudo dd if=<image> of=/dev/sdX bs=4M status=progress conv=fsync   (comptes : root/osmo, osmocom/osmo)"
```

2.18 /opt/GSM/osmo-operator/armbian/userpatches/customize-image.sh

13664 octets, 195 lignes → 195 lignes

```
#!/bin/bash
# armbian/userpatches/customize-image.sh - install the osmo-operator stack into the Armbian image
#
# Run by armbian/build INSIDE the image chroot (arguments: $RELEASE $LINUXFAMILY $BOARD
# $BUILD_DESKTOP), with userpatches/overlay mounted at /tmp/overlay. Condensed equivalent of the
# ISO modules 50-injection-image, 80-chroot, 82-arm-natif and 84-comptes:
#   1. Ubuntu runtime packages needed by the stack (list from 80-chroot.sh, without kernel/live)
#   2. dpkg -i of the Docker build .debs (osmo-build-*: libosmocore, osmo-*, gapk, QEMU, osmocom-bb...)
#   3. copy of /tmp/overlay/osmo/rootfs onto / (/usr/local, /opt/GSM, /etc/osmocom, units...)
#   4. what an x86 host cannot produce, compiled HERE (aarch64): toast, qosmo launcher,
#      c54x_exe / grgsm_exe, bench .debs (packaging/build-debs.sh)
#   5. accounts (root/osmo, osmocom/osmo), no Armbian first-login wizard, services
# Kernel, Pi firmware and root filesystem resizing are handled by Armbian.
set -euo pipefail
RELEASE="${1:-}"; LINUXFAMILY="${2:-}"; BOARD="${3:-}"; BUILD_DESKTOP="${4:-}"
OV=/tmp/overlay/osmo
export DEBIAN_FRONTEND=noninteractive LC_ALL=C.UTF-8 LANG=C.UTF-8
log() { echo "[osmo-customize] $*"; }
log "release=$RELEASE family=$LINUXFAMILY board=$BOARD desktop=$BUILD_DESKTOP overlay=$OV"
[ -d "$OV" ] || { log "pas d overlay $OV : image Armbian nue"; exit 0; }
[ -f "$OV/SOURCE" ] && log "pile : $(tr '\n' ' ' < "$OV/SOURCE")"
case "$RELEASE" in noble) ;; *) log "ATTENTION : les .deb du build visent noble, pas $RELEASE" ;; esac

# ── 1. Runtime packages ──────────────────────────────────────────────
apt-get update
PKGS="ca-certificates openssl netcat-openbsd socat tcpdump git logrotate systemd-resolved build-essential wget
      libtalloc2 libtalloc-dev libpcsclite1 libsctp1 libsctp-dev libc-ares2
      libgnutls30t64 libgnutls28-dev libmnl-dev libmnl0
      libortp-dev libdbi1t64 libdbd-sqlite3 sqlite3
      libfftw3-single3 libusb-1.0-0 libgsm1 libasound2t64
      libsofia-sip-ua-glib3t64 liburing2 libslirp0
```

```
      iproute2 iptables net-tools lksctp-tools
      tmux telnet expect whiptail lsb-release openssh-server sudo
      console-setup keyboard-configuration locales psmisc cryptsetup rsync
      python3 python3-venv python3-scapy tshark wireshark-common
      libasound2-plugins pulseaudio pulseaudio-utils alsa-utils
      binutils-arm-none-eabi gdb-multiarch asterisk ffmpeg
      libzmq5 libfftw3-dev cmake libpcsclite-dev meson ninja-build flex bison
      libgcrypt20-dev libssl-dev libidn-dev libyaml-dev libnghttp2-dev libmicrohttpd-dev libcurl4-gnutls-dev
      libtins-dev libc-ares-dev zstd kpartx"
# shellcheck disable=SC2086
apt-get install -y --no-install-recommends $PKGS || {
    log "apt : certains paquets manquent sur $RELEASE/arm64, on pose ce qui existe"
    for p in $PKGS; do apt-get install -y --no-install-recommends "$p" >/dev/null 2>&1 || log "  absent : $p"; done
}

# ── 2. Docker build .debs ──────────────────────────────────────────
if ls "$OV"/debs/*.deb >/dev/null 2>&1; then
    install -d /var/cache/osmo-debs
    cp -f "$OV"/debs/*.deb /var/cache/osmo-debs/
    dpkg -i --force-overwrite /var/cache/osmo-debs/osmo-build-*.deb || apt-get install -y -f
    ldconfig
    log "$(ls /var/cache/osmo-debs/*.deb | wc -l) paquets du build poses"
else
    log "pas de .deb du build dans l overlay : tout vient de l arbre rootfs/"
fi

# ── 3. Docker image tree ──────────────────────────────────────────
if [ -d "$OV/rootfs" ]; then
    cp -a "$OV/rootfs/." /
    ldconfig
    log "arbre recopie : $(du -sh "$OV/rootfs" | cut -f1) (/usr/local, /opt/GSM, /etc/osmocom, unites)"
fi

# ── 4. Native compilation (aarch64) ──────────────────────────────
if [ ! -x /usr/local/bin/toast ]; then
    ( set -e; cd /opt/GSM
      wget -qO gsm-1.0.24.tar.gz https://www.quut.com/gsm/gsm-1.0.24.tar.gz
      tar xzf gsm-1.0.24.tar.gz && rm -f gsm-1.0.24.tar.gz && cd gsm-1.0-pl24
      make >/dev/null 2>&1 || true; [ -x bin/toast ] || make toast >/dev/null 2>&1
      for b in toast untoast tcat; do [ -e "bin/$b" ] && install -m755 "bin/$b" "/usr/local/bin/$b"; done
    ) && log "toast compile" || log "toast non compile (on continue)"
fi
for fork in qosmo; do
    if [ -d "/opt/GSM/$fork/tools/qosmo-launch" ]; then
        make -s -C "/opt/GSM/$fork/tools/qosmo-launch" ALIAS="$fork" "$fork" && install -Dm755
        ↪  "/opt/GSM/$fork/tools/qosmo-launch/$fork" "/usr/local/bin/$fork" \
           && log "lanceur /usr/local/bin/$fork compile" || log "lanceur $fork : echec (run.sh appellera
            ↪  qemu-system-arm directement)"
    fi
done
for e in c54x_exe grgsm_exe; do
    [ -d "/opt/GSM/$e" ] && { make -s -B -C "/opt/GSM/$e" && log "$e compile" || log "$e : compilation echouee"; }
done
/usr/local/bin/qemu-system-arm --version >/dev/null 2>&1 && log "qemu-system-arm (aarch64) s execute" || log
 ↪  "qemu-system-arm absent ou KO"
if [ -x /opt/GSM/osmo-operator/packaging/build-debs.sh ]; then
    install -d /var/cache/osmo-debs
    env OSMO_OPERATOR_SRC=/opt/GSM/osmo-operator QOSMO_SRC=/opt/GSM/qosmo bash
     ↪  /opt/GSM/osmo-operator/packaging/build-debs.sh --out /var/cache/osmo-debs \
        && log ".deb du banc construits dans /var/cache/osmo-debs" || log ".deb du banc non construits (l image
         ↪  sort sans eux)"
fi

# ── 4b. Bench desktop (BUILD_DESKTOP=yes: minimal GNOME installed by Armbian) ─
# Subset of what the ISO --desktop adds on top of GNOME (iso_modules/80-chroot.sh,
# 85-installeur-bureau.sh) that makes sense on a board: X11 autologin as root, GSM LAB
```

```
# wallpaper, dark/Yaru blue theme, repository icons and dock, Conky and the panel
# (osmo-desktop-panel, extracted by build-armbian.sh), c123 and its GTK packages.
# No calamares, grub, secure boot, Mozilla firefox or GNOME extensions.
# Known limitation: GLava is not installed by default; its NCS circle needs OpenGL >= 4.2
# (imageLoad/Store), which neither the Pi 4 V3D nor the Orange Pi 5 Mali G610 provide under
# Mesa (GL 3.1). Use OSMO_ARM_GLAVA=1 sudo ./build-armbian.sh ... to try anyway (overlay marker OSMO_ARM_GLAVA).
if [ "$BUILD_DESKTOP" = yes ]; then
    REPO=/opt/GSM/osmo-operator
    DPKGS="conky-all fonts-dejavu python3-pil python3-gi python3-gi-cairo python3-cairo
           gir1.2-gtk-3.0 gir1.2-gdkpixbuf-2.0 gir1.2-vte-2.91 wmctrl x11-utils x11-xserver-utils
           xdotool zenity librsvg2-common yaru-theme-gtk yaru-theme-icon yaru-theme-gnome-shell
           wireshark vlc linphone-desktop gnome-terminal"
    apt-get update
    # shellcheck disable=SC2086
    apt-get install -y --no-install-recommends $DPKGS || {
        for p in $DPKGS; do apt-get install -y --no-install-recommends "$p" >/dev/null 2>&1 || log "  bureau,
         ↪  absent : $p"; done
    }

    # gdm: root autologin under X11 (the bench runs as root; xdotool, xprop and Conky need X)
    sed -i "/pam_succeed_if.so user != root quiet_success/s/^/#/" \
        /etc/pam.d/gdm-password /etc/pam.d/gdm-autologin 2>/dev/null || true
    install -d /etc/gdm3
    printf '[daemon]\nAutomaticLoginEnable=true\nAutomaticLogin=root\nWaylandEnable=false\n' >
     ↪  /etc/gdm3/custom.conf
    rm -f /etc/xdg/autostart/gnome-initial-setup-first-login.desktop

    # Schema defaults: wallpaper, theme, idle, keyboard, dock (same values as the ISO)
    OVR=/usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
    FOND=/usr/share/backgrounds/gsm-lab-ville.jpg
    for f in gsm-lab-ville.jpg gsm-lab-antenne.jpg gsm-lab-fond.jpg; do
        [ -f "$REPO/configs/$f" ] && install -Dm644 "$REPO/configs/$f" "/usr/share/backgrounds/$f"
    done
    [ -f "$FOND" ] || FOND=/usr/share/backgrounds/gsm-lab-fond.jpg
    printf "[org.gnome.desktop.session]\nidle-delay=uint32
     ↪  0\n\n[org.gnome.desktop.screensaver]\nlock-enabled=false\nidle-activation-
     ↪  enabled=false\n\n[org.gnome.settings-daemon.plugins.power]\nsleep-inactive-ac-type='nothing'\nsleep-
     ↪  inactive-battery-type='nothing'\n\n[org.gnome.desktop.input-sources]\nsources=[('xkb','%s')]\n"
     ↪  \
        "${OSMO_KB:-fr}" > "$OVR"
    for sec in org.gnome.desktop.interface org.gnome.desktop.interface:GNOME-Greeter; do
        printf "\n[%s]\ncolor-scheme='default'\ngtk-theme='Yaru-blue'\nicon-theme='Yaru-blue'\ncursor-
         ↪  theme='Yaru'\n" "$sec" >>
         ↪  "$OVR"
    done
    if [ -f "$FOND" ]; then
        for sec in org.gnome.desktop.background org.gnome.desktop.background:GNOME-Greeter; do
            printf "\n[%s]\npicture-uri='file://%s'\npicture-uri-dark='file://%s'\npicture-
             ↪  options='zoom'\nprimary-color='#0d1b2a'\n" "$sec" "$FOND" "$FOND" >>
             ↪  "$OVR"
        done
    fi
    printf "\n[org.gnome.shell]\nfavorite-apps=['osmo-info.desktop', 'org.gnome.Nautilus.desktop',
     ↪  'osmo-launch.desktop', 'osmo-tutorial.desktop', 'osmo-addition.desktop', 'osmo-update.desktop',
     ↪  'linphone.desktop', 'org.wireshark.Wireshark.desktop', 'org.gnome.Terminal.desktop']\n" >> "$OVR"
    printf "[org.gnome.settings-daemon.plugins.media-keys]\ncustom-keybindings=['/org/gnome/settings-
     ↪  daemon/plugins/media-keys/custom-keybindings/osmo-op-next/','/org/gnome/settings-daemon/plugins/media-
     ↪  keys/custom-keybindings/osmo-op-prev/']\n\n[org.gnome.settings-daemon.plugins.media-keys.custom-
     ↪  keybinding:/org/gnome/settings-daemon/plugins/media-keys/custom-keybindings/osmo-op-next/]\nname='osmo-
     ↪  operator : arret suivant (Conky)'\ncommand='/usr/local/bin/osmo-op
     ↪  --next'\nbinding='<Control><Mod5>Right'\n\n[org.gnome.settings-daemon.plugins.media-keys.custom-
     ↪  keybinding:/org/gnome/settings-daemon/plugins/media-keys/custom-keybindings/osmo-op-prev/]\nname='osmo-
     ↪  operator : arret precedent (Conky)'\ncommand='/usr/local/bin/osmo-op
     ↪  --prev'\nbinding='<Control><Mod5>Left'\n" \
        > /usr/share/glib-2.0/schemas/98-osmo-keys.gschema.override
    glib-compile-schemas /usr/share/glib-2.0/schemas 2>/dev/null || log "glib-compile-schemas a echoue"
```

```
    # Repository icons (except multi: installed by addition.sh --multi, see 85-installeur-bureau.sh)
    install -d /usr/share/applications /usr/share/icons/hicolor/scalable/apps
    for d in "$REPO"/data/desktop/*.desktop; do
        [ -f "$d" ] || continue
        e=$(basename "$d" .desktop); [ "$e" = osmo-multi ] && continue
        install -m644 "$d" "/usr/share/applications/$e.desktop"
        [ -f "$REPO/data/$e.svg" ] && install -m644 "$REPO/data/$e.svg"
         ↪  "/usr/share/icons/hicolor/scalable/apps/$e.svg"
    done
    gtk-update-icon-cache -f -q /usr/share/icons/hicolor 2>/dev/null || true

    # Desktop commands in PATH (same links as iso_modules/70-scripts.sh)
    ln -sf "$REPO/start-native.sh"        /usr/local/bin/osmo-start-native
    ln -sf "$REPO/tools/osmo-fond.py"     /usr/local/bin/osmo-fond
    ln -sf "$REPO/tools/osmo-op.sh"       /usr/local/bin/osmo-op
    ln -sf "$REPO/tools/osmo-drivers.sh"  /usr/local/bin/osmo-drivers
    ln -sf "$REPO/tools/overlay-draw.py"  /usr/local/bin/overlay-draw
    ln -sf "$REPO/tools/osmo-c123-gtk.py" /usr/local/bin/c123
    ln -sf "$REPO/tools/osmo-c123-gtk.py" /usr/local/bin/osmo-c123

    # Conky and the panel, kept alive for the whole session (same watchdog as the ISO)
    if [ -s "$OV/osmo-desktop-panel" ]; then
        install -m755 "$OV/osmo-desktop-panel" /usr/local/bin/osmo-desktop-panel
        install -d /etc/xdg/autostart
        printf '[Desktop Entry]\nType=Application\nName=Conky osmo-operator\nComment=Tableau de bord du banc
         ↪  GSM\nExec=/usr/local/bin/osmo-desktop-panel\nIcon=utilities-system-
         ↪  monitor\nTerminal=false\nNoDisplay=true\nOnlyShowIn=GNOME;\nX-GNOME-Autostart-enabled=true\nX-GNOME-
         ↪  Autostart-Delay=6\n'
         ↪  \
            > /etc/xdg/autostart/osmo-conky.desktop
        log "bureau : Conky + encart a la connexion"
    fi

    if [ -f "$OV/OSMO_ARM_GLAVA" ]; then
        apt-get install -y --no-install-recommends git meson ninja-build gcc pkg-config patch \
            libpulse-dev libx11-dev libxext-dev libxrender-dev libxcomposite-dev libgl-dev >/dev/null \
            && bash "$REPO/tools/glava-build.sh" && log "GLava pose (OSMO_ARM_GLAVA)" \
            || log "GLava non pose"
    fi
    log "bureau du banc pose (fond $FOND)"
fi

# Bench aliases (banc-rssi, banc-trx, c123-1...): profile.d AND real commands, desktop or not
if [ -f /opt/GSM/osmo-operator/scripts/osmo-banc-aliases.sh ]; then
    install -m644 /opt/GSM/osmo-operator/scripts/osmo-banc-aliases.sh /etc/profile.d/98-osmo-banc-aliases.sh
    bash /opt/GSM/osmo-operator/tools/osmo-banc-cmd.sh --install /usr/local/bin
fi

# ── 5. Accounts, first login, services ──────────────────────────────────
id -u osmocom >/dev/null 2>&1 || useradd -m -s /bin/bash -c 'Compte osmocom (non privilegie)' osmocom
echo 'osmocom:osmo' | chpasswd
echo 'root:osmo'    | chpasswd
usermod -aG sudo,audio,dialout,plugdev osmocom 2>/dev/null || true
rm -f /root/.not_logged_in_yet  # no Armbian first-login wizard: accounts are already set
for u in osmo-egprs-web osmo-pulse; do systemctl enable "$u" 2>/dev/null || true; done
for u in osmo-banc osmo-multi; do  # installed, not enabled: set OSMO_IMAGE_BANC=1 / OSMO_IMAGE_MULTI=1 in the
 ↪  overlay to enable
    v="OSMO_IMAGE_${u#osmo-}"; v="${v^^}"
    [ -f "$OV/$v" ] && systemctl enable "$u" 2>/dev/null && log "$u active au boot"
done
apt-get clean; rm -rf /var/lib/apt/lists/*
log "termine"
```

2.19 /opt/GSM/osmo-operator/checks/_mode.sh

34506 octets, 800 lignes → 800 lignes

```
# ============================================================================
#  checks/_mode.sh - single docker/native detection rule shared by all
#                    checks/ scripts.
#  Library: SOURCE it; running it directly only prints what it detects.
# ============================================================================
#
#  THE TWO WORLDS
#  --------------
#    docker - one osmo-operator-N container per operator on the gsm-inter
#             network (172.20.0.0/24); the SS7 hub is the osmo-inter-stp
#             container (172.20.0.10); everything is reached via "docker exec".
#    native - ISO, VM or bare metal: a single operator (unless netns topology),
#             everything runs here, VTYs on 127.0.0.1, configs in /etc/osmocom,
#             daemons are systemd units or plain processes.
#
#  VTY PORTS are the SAME in both worlds - only the host changes:
#      osmo-stp 4239 · osmo-msc 4254 · osmo-bsc 4242 · osmo-hlr 4258
#      osmo-sgsn 4245 · osmo-ggsn 4260 · osmo-mgw 4243 · mobile/BB 4247
#
#  CONTRACT
#  --------
#   • defines ONLY functions and ": =" variables; prints nothing, sets no
#     colors, counts no PASS/FAIL: each check keeps its own ok/fail/warn/skip,
#     output format and exit code;
#   • probes return 0 = true, non-0 = false;
#   • can be sourced twice without side effects (OSMO_MODE_LIB_LOADED guard);
#   • no "grep -aq" at the end of a pipeline and no "awk ... exit" mid-pipe:
#     under "set -o pipefail" both return 141;
#   • compatible with "set -euo pipefail" in the caller.
#
#  osmo_mode() VS detect_runtime_env() IN start-native.sh
#  ------------------------------------------------------
#  They answer different questions and must not be merged:
#    detect_runtime_env()  "where am I running?"            -> docker | vm | bare
#        Decides the ADDRESSING PLAN (docker: inter-STP 172.20.0.10, IP already
#        set; VM/ISO: inter-STP is another machine, address from DHCP).
#    osmo_mode()           "how do I REACH the nodes from here?" -> docker | native
#        Decides between "docker exec" and running locally.
#  Inside a repository container the first returns "docker" and the second
#  "native": "docker exec" to oneself makes no sense, even with the socket
#  mounted. The shared convention is "--docker" / "--native", which
#  start-native.sh passes to network/setup-wan-mesh.sh and osmo_mode_force()
#  accepts.
#
#  NAMING A NODE - <op>
#  --------------------
#  All execution functions take an <op> as first argument, one of:
#      3                    the operator number (canonical form)
#      osmo-operator-3      the label (as returned by osmo_ops/osmo_node)
#      hub | osmo-inter-stp the SS7 hub
#  In docker this is the container name; in native it is this machine, with
#  an "ip netns exec osmo-opN" prefix if and only if that namespace exists
#  (native multi-operator topology, run_modules/12-netns.sh).
#  The osmo-operator-N label is KEPT in native mode: checks/vty-debug-dump.sh
#  writes "container=osmo-operator-N" and checks/operator_summary.sh parses
#  it. It is a correlation key, not a queryable object.
#
#  SOURCING (same block everywhere; works from the repository and from /opt):
#      _here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
#      for _c in "$_here/_mode.sh" /opt/GSM/osmo-operator/checks/_mode.sh; do
#          [ -r "$_c" ] && { . "$_c"; break; }
#      done
```

```
#      command -v osmo_mode >/dev/null || { echo "checks/_mode.sh introuvable" >&2; exit 1; }
#
#  Env: OSMO_MODE (force docker|native), OSMO_REPO, OSMO_NATIVE_ROOT, OSMO_BACKBONE_NET,
#       OSMO_NETNS_PREFIX, OSMO_MULTI_CONF, OSMO_HUB_M3UA_PORT,
#       OSMO_VTY_OPEN / OSMO_VTY_READ / OSMO_VTY_Q / OSMO_VTY_KILL.
# ============================================================================

[ -n "${OSMO_MODE_LIB_LOADED:-}" ] && return 0
OSMO_MODE_LIB_LOADED=1

# Repository root (this file is in checks/); used to find globals.conf.
# Can be overridden for tests.
_osmo_lib_here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
: "${OSMO_REPO:=$(dirname "$_osmo_lib_here")}"
unset _osmo_lib_here

# Inter-operator backbone addressing. Same value in both worlds: docker uses
# "--network gsm-inter --ip", native multi-operator recreates it in netns
# (BACKBONE_NET, run_modules/12-netns.sh).
: "${OSMO_BACKBONE_NET:=172.20.0}"
: "${OSMO_NETNS_PREFIX:=${NETNS_PREFIX:-osmo-op}}"

# Root prefix, native mode only: points at a test tree
# (OSMO_NATIVE_ROOT=/tmp/test) instead of the machine's /etc. Empty in
# production. Same role as NATIVE_ROOT in network/setup-wan-mesh.sh.
: "${OSMO_NATIVE_ROOT:=}"

# Per-node cache of the available VTY client, to avoid running "command -v nc"
# for every VTY command. "-g" keeps it global even when the library is sourced
# from inside a function.
declare -gA _OSMO_VTY_TOOL 2>/dev/null || declare -A _OSMO_VTY_TOOL

# ============================================================================
#  1. MODE
# ============================================================================

# osmo_in_container - are we running INSIDE a container of THIS repository?
# /.dockerenv alone is true in any container; also require
# /etc/docker-entrypoint-cmd, created by scripts/entrypoint.sh (same pair as
# start-docker.sh and start-native.sh).
osmo_in_container() {
    [ -f /.dockerenv ] && [ -f /etc/docker-entrypoint-cmd ]
}

# Detection, from the most explicit to the most inferred.
_osmo_mode_detect() {
    # Inside a repository container we ARE the node: "docker exec" to oneself
    # makes no sense, even if the docker socket is mounted (dashboard container).
    if osmo_in_container; then printf 'native\n'; return 0; fi

    # OSMO_NATIVE=1 is set by the ISO (services/osmo-egprs-web.service).
    if [ "${OSMO_NATIVE:-}" = "1" ]; then printf 'native\n'; return 0; fi

    # Repository rule (also in wan_ss7_check.sh and setup-wan-mesh.sh): rely on a
    # RUNNING operator container, not on the docker binary being present.
    # Capture the list BEFORE filtering: "docker ps | grep -aq" returns 141 under pipefail.
    if command -v docker >/dev/null 2>&1; then
        local names
        names="$(docker ps --format '{{.Names}}' 2>/dev/null || true)"
        if grep -ax 'osmo-operator-1' >/dev/null 2>&1 <<<"$names"; then
            printf 'docker\n'; return 0
        fi
    fi
    printf 'native\n'
}
```

```
# osmo_mode - prints "docker" or "native". Cached in OSMO_MODE, which also
# acts as input: "OSMO_MODE=native ./checks/global_check.sh" forces it.
osmo_mode() {
    case "${OSMO_MODE:-}" in
        docker|native) : ;;
        *) OSMO_MODE="$(_osmo_mode_detect)" ;;
    esac
    printf '%s\n' "$OSMO_MODE"
}

osmo_is_docker() { [ "$(osmo_mode)" = docker ]; }
osmo_is_native() { [ "$(osmo_mode)" = native ]; }

# osmo_mode_force <docker|native|--docker|--native|--mode=X>
# Drop into each script's option "case" loop: explicit always wins over
# detection. Returns 2 for an argument that is not a mode.
osmo_mode_force() {
    local m="${1:-}"
    case "$m" in
        --mode=*) m="${m#*=}" ;;
        --docker) m=docker ;;
        --native) m=native ;;
    esac
    case "$m" in
        docker|native) OSMO_MODE="$m"; return 0 ;;
        *) return 2 ;;
    esac
}

# ============================================================================
#  2. INVENTORY
# ============================================================================

# Reads a key from globals.conf ("N_OPERATORS=3") with awk directly:
# no pipe, hence no pipefail trap.
_osmo_globals() {
    local k="${1:-}" f v
    for f in "${OSMO_GLOBALS:-}" "$OSMO_REPO/globals.conf" \
             /opt/GSM/osmo-operator/globals.conf ./globals.conf; do
        [ -n "$f" ] && [ -r "$f" ] || continue
        v="$(awk -F= -v k="$k" '$1 == k { gsub(/[" \r\t]/, "", $2); if ($2 != "") print $2 }' "$f" | tail -1)"
        [ -n "$v" ] && { printf '%s\n' "$v"; return 0; }
    done
    return 1
}

# osmo_ops - IDs of the present operators, one per line, sorted.
#   multi  : the operators declared in the multi-operator topology (below).
#   docker : the running osmo-operator-N containers.
#   native : $OSMO_OP_IDS (set by services/osmo-egprs-web.service), then the
#            osmo-opN netns (native multi-operator), then N_OPERATORS from
#            globals.conf, else 1.
#
# == MULTI-OPERATOR TOPOLOGY ==================================================
# A multi-operator bench can be MIXED (e.g. op 1 native on the host, op 2 and 3
# and the hub in containers), so mode, IP and point code are per operator.
# /etc/osmocom/osmo-multi.conf, written by addition.sh, is authoritative:
#   MULTI_OPS="1:native::1.1.2:150 2:docker:172.20.0.12:1.2.2:250 ..."
#   MULTI_HUB_IP=172.20.0.10  MULTI_HUB_NAME=osmo-inter-stp
# When absent, the global detection above applies.
OSMO_MULTI_CONF="${OSMO_MULTI_CONF:-/etc/osmocom/osmo-multi.conf}"
_OSMO_MULTI_LOADED=""

_osmo_multi_load() {
    [ -n "$_OSMO_MULTI_LOADED" ] && return "$_OSMO_MULTI_LOADED"
    _OSMO_MULTI_LOADED=1                      # 1 = no topology
```

```
    [ -r "$OSMO_MULTI_CONF" ] || return 1
    # shellcheck disable=SC1090
    . "$OSMO_MULTI_CONF" 2>/dev/null || return 1
    [ -n "${MULTI_OPS:-}" ] || return 1
    _OSMO_MULTI_LOADED=0
    return 0
}

# osmo_multi_active - rc 0 if a multi-operator topology is declared.
osmo_multi_active() { _osmo_multi_load; }

# osmo_op_mode <id> - "native" or "docker" FOR THIS OPERATOR.
# Without a topology, falls back to the global mode.
osmo_op_mode() {
    local want="${1:-}" spec idx mode
    if _osmo_multi_load; then
        for spec in $MULTI_OPS; do
            IFS=: read -r idx mode _ <<< "$spec"
            [ "$idx" = "$want" ] && { printf '%s\n' "$mode"; return 0; }
        done
    fi
    osmo_mode
}

# osmo_hub_pc - hub point code from the topology; empty otherwise.
osmo_hub_pc() {
    _osmo_multi_load || return 1
    [ -n "${MULTI_HUB_PC:-}" ] || return 1
    printf '%s\n' "$MULTI_HUB_PC"
}

# osmo_op_pc <id> - expected point code from the topology; empty otherwise.
osmo_op_pc() {
    local want="${1:-}" spec idx mode ip pc
    _osmo_multi_load || return 1
    for spec in $MULTI_OPS; do
        IFS=: read -r idx mode ip pc _ <<< "$spec"
        [ "$idx" = "$want" ] && { printf '%s\n' "$pc"; return 0; }
    done
    return 1
}

osmo_ops() {
    local ids=""
    # The declared topology overrides detection: it lists the EXPECTED operators,
    # so a stopped operator shows up as missing instead of disappearing.
    if _osmo_multi_load; then
        ids="$(tr ' ' '\n' <<<"$MULTI_OPS" | cut -d: -f1 | sed -n 's/^\([0-9][0-9]*\)$/\1/p')"
        [ -n "$ids" ] && { sort -n -u <<<"$ids" | sed '/^$/d'; return 0; }
    fi
    if [ "$(osmo_mode)" = docker ]; then
        local names
        names="$(docker ps --format '{{.Names}}' 2>/dev/null || true)"
        ids="$(sed -n 's/^osmo-operator-\([0-9][0-9]*\)$/\1/p' <<<"$names")"
    else
        ids="$(tr ', ' '\n\n' <<<"${OSMO_OP_IDS:-}" | sed -n 's/^\([0-9][0-9]*\)$/\1/p')"
        if [ -z "$ids" ]; then
            ids="$(ip netns list 2>/dev/null | awk '{print $1}' \
                   | sed -n "s/^${OSMO_NETNS_PREFIX}\([0-9][0-9]*\)\$/\1/p")"
        fi
        if [ -z "$ids" ]; then
            local n; n="$(_osmo_globals N_OPERATORS || true)"
            case "$n" in ''|*[!0-9]*) n=1 ;; esac
            [ "$n" -ge 1 ] 2>/dev/null || n=1
            ids="$(seq 1 "$n")"
        fi
```

```
    fi
    [ -n "$ids" ] || return 0
    sort -n -u <<<"$ids" | sed '/^$/d'
}

# osmo_op_exists <id> - does the operator exist? (anchored match: "1" does
# not match osmo-operator-10)
osmo_op_exists() {
    local want="${1:-}" id
    case "$want" in ''|*[!0-9]*) return 2 ;; esac
    while read -r id; do
        [ "$id" = "$want" ] && return 0
    done <<<"$(osmo_ops)"
    return 1
}

# osmo_node <op> - node label: "osmo-operator-N" or "osmo-inter-stp".
# In docker this is the container name; in native it is a DISPLAY NAME that
# keeps banners and the dump's "container=" field identical.
osmo_node() {
    case "${1:-}" in
        hub|osmo-inter-stp) printf 'osmo-inter-stp\n' ;;
        osmo-operator-*)    printf '%s\n' "$1" ;;
        [0-9]*)             printf 'osmo-operator-%s\n' "$1" ;;
        *)                  printf '%s\n' "${1:-}" ;;
    esac
}

# Operator number of an <op>; empty for the hub or unknown values.
_osmo_id() {
    case "${1:-}" in
        hub|osmo-inter-stp) printf '' ;;
        osmo-operator-*)    printf '%s\n' "${1##*-}" ;;
        [0-9]*)             printf '%s\n' "$1" ;;
        *)                  printf '' ;;
    esac
}

# osmo_hub - is an inter-STP queryable HERE? Prints its label if so.
#   docker : the osmo-inter-stp container is running.
#   native : this machine IS the hub - OSMO_ROLE=interstp in /etc/osmo-role
#            (set by build-iso.sh), or the start-interstp.sh probe.
# On a native operator node the hub is ANOTHER machine: rc 1, and
# osmo_hub_hint says what to do instead of reporting a fake "0 AS active".
osmo_hub() {
    if [ "$(osmo_mode)" = docker ]; then
        local names
        names="$(docker ps --format '{{.Names}}' 2>/dev/null || true)"
        grep -ax 'osmo-inter-stp' >/dev/null 2>&1 <<<"$names" || return 1
        printf 'osmo-inter-stp\n'; return 0
    fi
    if [ -r /etc/osmo-role ] \
       && grep -aE '^OSMO_ROLE[[:space:]]*=[[:space:]]*interstp' /etc/osmo-role >/dev/null 2>&1; then
        printf 'osmo-inter-stp\n'; return 0
    fi
    if pgrep -f 'osmo-stp .*osmo-stp-interop\.cfg' >/dev/null 2>&1; then
        printf 'osmo-inter-stp\n'; return 0
    fi
    return 1
}

# OSMO_HUB_M3UA_PORT - M3UA port of the inter-STP hub (helpers/create_interop.sh
# writes "listen m3ua 2908"). Operator STPs listen on 2905.
: "${OSMO_HUB_M3UA_PORT:=2908}"

# _osmo_hub_ip_probe - ask the operator nodes where their hub is. Sources,
```

```
# in order:
#   1. OSMO_HUB_IP, injected into each container by start-docker.sh (-e OSMO_HUB_IP)
#      when --hub-ip / --wan points to an inter-STP outside the docker backbone;
#   2. the "remote-ip" of asp asp-to-inter in /etc/osmocom/osmo-stp.cfg, valid
#      even without that environment (manually restarted container, ISO).
# Returns 1 without output if no node can answer.
_osmo_hub_ip_probe() {
    local op v=""
    for op in $(osmo_ops 2>/dev/null); do
        v="$(osmo_exec "$op" printenv OSMO_HUB_IP 2>/dev/null | tr -d "\r" | tail -1)"
        [ -n "$v" ] && { printf '%s\n' "$v"; return 0; }
        v="$(osmo_exec "$op" cat /etc/osmocom/osmo-stp.cfg 2>/dev/null \
             | awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}' | tr -d "\r")"
        [ -n "$v" ] && { printf '%s\n' "$v"; return 0; }
    done
    return 1
}

# osmo_hub_ip - hub address. Nothing known -> rc 1.
#   docker + local hub    : the backbone, 172.20.0.10;
#   docker, no local hub  : the WAN address seen by the operators (--hub-ip);
#   native                : OSMO_HUB_IP from /etc/osmo-role, else the same node probe.
# Cached: the probe costs one "docker exec" per node and checks call this
# function several times per run.
_OSMO_HUB_IP_CACHE=""
osmo_hub_ip() {
    if [ -n "$_OSMO_HUB_IP_CACHE" ]; then
        [ "$_OSMO_HUB_IP_CACHE" = "-" ] && return 1
        printf '%s\n' "$_OSMO_HUB_IP_CACHE"; return 0
    fi
    local v=""
    # MULTI_HUB_IP is authoritative: on a mixed bench the global mode would give
    # the wrong address (loopback for a native operator while the hub is a container).
    if _osmo_multi_load && [ -n "${MULTI_HUB_IP:-}" ]; then
        _OSMO_HUB_IP_CACHE="$MULTI_HUB_IP"
        printf '%s\n' "$MULTI_HUB_IP"; return 0
    fi
    if [ "$(osmo_mode)" = docker ]; then
        if osmo_hub >/dev/null 2>&1; then
            printf '%s.10\n' "$OSMO_BACKBONE_NET"; return 0
        fi
    else
        [ -r /etc/osmo-role ] && \
            v="$(awk -F= '/^OSMO_HUB_IP=/ { gsub(/[ \r\t]/, "", $2); print $2 }' /etc/osmo-role 2>/dev/null | tail
             ↪  -1)"
    fi
    [ -n "$v" ] || v="$(_osmo_hub_ip_probe 2>/dev/null || true)"
    if [ -z "$v" ]; then _OSMO_HUB_IP_CACHE="-"; return 1; fi
    _OSMO_HUB_IP_CACHE="$v"
    printf '%s\n' "$v"
}

# osmo_hub_is_remote - rc 0 (and prints the IP) when a hub is CONFIGURED but
# lives ELSEWHERE: the WAN case (--hub-ip / --wan) or a native operator node.
# A docker backbone or loopback address without a hub container does not
# count: then the hub is not remote, it is absent.
osmo_hub_is_remote() {
    local ip
    osmo_hub >/dev/null 2>&1 && return 1
    ip="$(osmo_hub_ip 2>/dev/null)" || return 1
    [ -n "$ip" ] || return 1
    case "$ip" in
        "${OSMO_BACKBONE_NET}."*|127.*|localhost) return 1 ;;
    esac
    printf '%s\n' "$ip"
}
```

```
# osmo_hub_assoc [ip] [port] - how many local nodes have an ESTABLISHED SCTP
# association to the hub? The inter-STP link is M3UA over SCTP, so a TCP probe
# ("nc -z" on 2908) is refused even when the link is up. "ss --sctp" is used,
# run from inside the NODE (containers have their own network stack).
# Prints "established/total"; rc 0 if at least one, 1 if none, 2 if the probe
# is not possible (no ss, no --sctp support, no queryable node).
osmo_hub_assoc() {
    local ip="${1:-}" port="${2:-$OSMO_HUB_M3UA_PORT}" op out probed=0 n=0 tot=0 pat
    [ -n "$ip" ] || ip="$(osmo_hub_ip 2>/dev/null || true)"
    [ -n "$ip" ] || return 2
    pat="${ip//./\\.}:${port}"
    for op in $(osmo_ops 2>/dev/null); do
        tot=$((tot+1))
        out="$(osmo_exec "$op" ss -an --sctp 2>/dev/null || true)"
        [ -n "$out" ] || continue
        probed=1
        grep -aqE "ESTAB.*[[:space:]]${pat}([[:space:]]|$)" <<<"$out" && n=$((n+1))
    done
    [ "$probed" -eq 1 ] || return 2
    printf '%s/%s\n' "$n" "$tot"
    [ "$n" -gt 0 ]
}

# osmo_hub_hint - ready-made skip() message when the hub exists but is
# elsewhere. Osmocom VTYs listen only on 127.0.0.1 (no repository config sets
# "bind" under "line vty"), so the hub's AS/ASP state is NOT observable from
# an operator node.
osmo_hub_hint() {
    local ip
    ip="$(osmo_hub_ip 2>/dev/null || true)"
    if [ -n "$ip" ]; then
        printf 'hub distant %s : VTY 4239 non joignable depuis ce noeud - lancez ./launch/start-interstp.sh
        ↪  --status sur le hub\n' "$ip"
    else
        printf 'aucun inter-STP local, et aucune adresse de hub connue ici (OSMO_HUB_IP, --hub-ip) - hub non
        ↪  interrogeable\n'
    fi
}

# ============================================================================
#  3. EXECUTION ON A NODE
# ============================================================================

# Native-mode netns prefix. Empty for a single operator: the backbone and
# namespaces are only created when N_OPERATORS > 1.
_osmo_ns_cmd() {
    local id ns
    id="$(_osmo_id "${1:-}")"
    [ -n "$id" ] || return 0
    ns="${OSMO_NETNS_PREFIX}${id}"
    if [ -e "/run/netns/$ns" ] || [ -e "/var/run/netns/$ns" ]; then
        printf 'ip netns exec %s' "$ns"
    fi
}

# osmo_exec <op> <argv...> - 1:1 replacement for "docker exec "$container" ...".
osmo_exec() {
    local op="${1:-}"; shift || return 2
    [ $# -ge 1 ] || return 2
    # Mode is decided per operator: on a mixed bench the native one is queried
    # locally while its neighbours go through docker exec.
    if [ "$(osmo_op_mode "$(_osmo_id "$op")")" = docker ]; then
        local c; c="$(osmo_node "$op")"
        [ -n "$c" ] || return 2
        docker exec "$c" "$@"
```

```
        return $?
    fi
    local ns; ns="$(_osmo_ns_cmd "$op")"
    # shellcheck disable=SC2086  # ns must be word-split (ip netns exec X)
    if [ -n "$ns" ]; then $ns "$@"; else "$@"; fi
}

# Same, but stdin is forwarded to the node ("docker exec -i").
# Internal: used for VTY dialogue, where the pipe feeds nc/telnet.
_osmo_exec_in() {
    local op="${1:-}"; shift || return 2
    if [ "$(osmo_op_mode "$(_osmo_id "$op")")" = docker ]; then
        docker exec -i "$(osmo_node "$op")" "$@"
        return $?
    fi
    local ns; ns="$(_osmo_ns_cmd "$op")"
    # shellcheck disable=SC2086
    if [ -n "$ns" ]; then $ns "$@"; else "$@"; fi
}

# osmo_cat <op> <path> - contents of a file on the node (config, marker...).
osmo_cat() {
    local op="${1:-}" path="${2:-}"
    [ -n "$path" ] || return 2
    if [ "$(osmo_mode)" = docker ]; then
        docker exec "$(osmo_node "$op")" cat "$path" 2>/dev/null
    else
        cat "${OSMO_NATIVE_ROOT}${path}" 2>/dev/null
    fi
}

# osmo_ast <op> <CLI command> - asterisk -rx, identical in both worlds.
osmo_ast() {
    local op="${1:-}" cli="${2:-}"
    [ -n "$cli" ] || return 2
    osmo_exec "$op" asterisk -rx "$cli" 2>/dev/null
}

# osmo_running <op> <daemon> - is this daemon running on this node?
#
# Native mode needs BOTH checks: on the ISO daemons are units
# (services/osmo-bts-trx.service...), but core_svc_start falls back to a
# detached process without systemd, where only pgrep answers.
#
# The "pgrep -f" fallback is needed because /proc/PID/comm is TRUNCATED to 15
# characters, so "pgrep -ax osmo-sip-connector" (18) never matches.
#
# Known limitation (native multi-operator): "ip netns exec" does not isolate
# PIDs, so the answer is per machine, not per operator.
osmo_running() {
    local op="${1:-}" name="${2:-}"
    [ -n "$name" ] || return 2
    if [ "$(osmo_mode)" = docker ]; then
        osmo_exec "$op" pgrep -ax "$name" >/dev/null 2>&1 && return 0
        osmo_exec "$op" pgrep -f "(^|/)${name}([[:space:]]|\$)" >/dev/null 2>&1
        return $?
    fi
    if [ -d /run/systemd/system ] && command -v systemctl >/dev/null 2>&1; then
        systemctl is-active --quiet "$name" 2>/dev/null && return 0
    fi
    pgrep -ax "$name" >/dev/null 2>&1 && return 0
    pgrep -f "(^|/)${name}([[:space:]]|\$)" >/dev/null 2>&1
}

# osmo_sock <op> <path> - is a UNIX socket listening?
# ss is queried BEFORE the file test: under PrivateTmp, /tmp/msc_mncc exists
```

```
# but is invisible to "test -S" (run_modules/13-msc.sh).
# awk reads ALL its input (no mid-pipe "exit"): an early exit would kill ss
# with SIGPIPE and return 141 under pipefail.
osmo_sock() {
    local op="${1:-}" p="${2:-}" out
    [ -n "$p" ] || return 2
    out="$(osmo_exec "$op" ss -xlH 2>/dev/null || true)"
    awk -v w="$p" '{ for (i = 1; i <= NF; i++) if ($i == w) f = 1 } END { exit !f }' <<<"$out" && return 0
    osmo_exec "$op" test -S "$p" 2>/dev/null
}

# osmo_port <op> <port> [tcp|udp|sctp] - is this port listening on the node?
# Exact port match: ":7890" does not match 17890 or 78901.
osmo_port() {
    local op="${1:-}" port="${2:-}" proto="${3:-tcp}" out
    local -a opts
    case "$port" in ''|*[!0-9]*) return 2 ;; esac
    case "$proto" in
        tcp)  opts=(-H -ltn) ;;
        udp)  opts=(-H -lun) ;;
        sctp) opts=(-H -ln --sctp) ;;
        *)    return 2 ;;
    esac
    out="$(osmo_exec "$op" ss "${opts[@]}" 2>/dev/null || true)"
    # ss field 4 is the local "address:port" for tcp, udp and sctp.
    awk -v want="$port" '
        { a = $4
          if (a == "" || match(a, /:[0-9]+$/) == 0) next
          if (substr(a, RSTART + 1) == want) f = 1 }
        END { exit !f }' <<<"$out"
}

# osmo_host <op> - host to reach this node FROM HERE (SMS relay, SIP,
# M3UA...). NOT the VTY host, which is always 127.0.0.1 inside the node
# (handled by osmo_vty).
#   docker           : container IP on the backbone (172.20.0.1N, .10 = hub)
#   native + netns   : the same backbone, recreated in netns
#   native single-op : 127.0.0.1 - no gsm-inter bridge, no 172.20.0.0/24
#   native hub       : OSMO_HUB_IP from /etc/osmo-role, else 127.0.0.1
osmo_host() {
    local op="${1:-}" id
    case "$op" in
        hub|osmo-inter-stp)
            if [ "$(osmo_mode)" = docker ]; then
                printf '%s.10\n' "$OSMO_BACKBONE_NET"
            else
                local h; h="$(osmo_hub_ip 2>/dev/null || true)"
                printf '%s\n' "${h:-127.0.0.1}"
            fi
            return 0 ;;
    esac
    id="$(_osmo_id "$op")"
    if [ -n "$id" ] && { [ "$(osmo_mode)" = docker ] || [ -n "$(_osmo_ns_cmd "$op")" ]; }; then
        printf '%s.%s\n' "$OSMO_BACKBONE_NET" "$((10 + id))"
    else
        printf '127.0.0.1\n'
    fi
}

# ============================================================================
#  4. VTY
# ============================================================================

# Which VTY client does this node have? nc preferred, telnet as fallback.
# The ISO installs netcat-openbsd AND telnet but NOT socat (unlike core_vty_ask
# in run_modules/). The diagnostic is printed ONCE per node, on stderr.
```

```
_osmo_vty_tool() {
    local op="${1:-}" key tool
    key="$(osmo_node "$op")"
    tool="${_OSMO_VTY_TOOL[$key]:-}"
    if [ -z "$tool" ]; then
        if osmo_exec "$op" sh -c 'command -v nc' >/dev/null 2>&1; then
            tool=nc
        elif osmo_exec "$op" sh -c 'command -v telnet' >/dev/null 2>&1; then
            tool=telnet
        else
            tool=none
            printf 'checks/_mode.sh : aucun client VTY sur %s - installez netcat-openbsd ou telnet (apt-get
             ↪  install -y netcat-openbsd telnet)\n' \
                  "$key" >&2
        fi
        _OSMO_VTY_TOOL[$key]="$tool"
    fi
    [ "$tool" = none ] && return 1
    printf '%s\n' "$tool"
}

# osmo_vty_up <op> <port> - does the VTY answer?
# In native mode the PASSIVE probe (ss) is tried first: it opens no VTY
# session. If it sees nothing (no ss, restricted /proc), fall back to the
# active probe - no false negatives.
osmo_vty_up() {
    local op="${1:-}" port="${2:-}"
    case "$port" in ''|*[!0-9]*) return 2 ;; esac
    if [ "$(osmo_mode)" != docker ] && command -v ss >/dev/null 2>&1; then
        osmo_port "$op" "$port" tcp && return 0
    fi
    osmo_exec "$op" bash -c "echo >/dev/tcp/127.0.0.1/${port}" 2>/dev/null
}

# osmo_vty <op> <port> <command>... - sends the commands to the VTY and
# prints the RAW output (banner and prompt included; osmo_vty_clean filters).
#
# Returns 1 WITHOUT OUTPUT if the VTY does not answer or the node has neither
# nc nor telnet, so callers can tell "service silent" from "no answer to
# this command". The pre-check is osmo_vty_up (one TCP open/close).
#
# The caller supplies its own "enable" / "end": the library adds no command.
#
# TIMINGS ARE PARAMETERS (environment variables), because each script has its
# own and changing them changes the AMOUNT of captured output, hence the
# "grep -ac" counts:
#     OSMO_VTY_OPEN  delay before writing (0.5; ss7_check: 1)
#     OSMO_VTY_READ  read delay           (2  ; ss7_check: 1.5)
#     OSMO_VTY_Q     nc -q<N>             (1  ; vty-debug-dump: 2)
#     OSMO_VTY_KILL  hard timeout of the VTY client, seconds (8)
# The Osmocom VTY ignores input received before its telnet negotiation ends:
# these sleeps pace the DIALOGUE, they are not wait barriers.
osmo_vty() {
    local op="${1:-}" port="${2:-}"
    [ $# -ge 3 ] || return 2
    shift 2
    case "$port" in ''|*[!0-9]*) return 2 ;; esac

    osmo_vty_up "$op" "$port" || return 1

    local body="" c
    for c in "$@"; do body="${body}${c}"$'\n'; done

    local tool
    tool="$(_osmo_vty_tool "$op")" || return 1
```

```
    local open="${OSMO_VTY_OPEN:-0.5}" read_s="${OSMO_VTY_READ:-2}" q="${OSMO_VTY_Q:-1}"
    # The VTY client is always run under "timeout": operator docker images only
    # have telnet, which has no equivalent of nc's "-q" (EOF on stdin does not
    # hang up, and the Osmocom VTY does not close after "end"), so it would block
    # forever. The timeout runs INSIDE the node (docker exec ... timeout N telnet):
    # killing docker exec from here would leave telnet running in the container.
    local kill_after="${OSMO_VTY_KILL:-8}"

    # nc/telnet often exit before reading everything: the producer subshell
    # then gets SIGPIPE and the pipe returns 141. "|| true" keeps the function
    # usable under "set -euo pipefail".
    if [ "$tool" = nc ]; then
        { sleep "$open"; printf '%s' "$body"; sleep "$read_s"; } \
            | _osmo_exec_in "$op" timeout "$kill_after" nc "-q${q}" 127.0.0.1 "$port" 2>/dev/null || true
    else
        { sleep "$open"; printf '%s' "$body"; sleep "$read_s"; } \
            | _osmo_exec_in "$op" timeout "$kill_after" telnet 127.0.0.1 "$port" 2>/dev/null || true
    fi
    return 0
}

# osmo_vty_clean [extra pattern] - stdin->stdout filter removing banner, prompt
# and empty lines. The base pattern and the ORDER of filters match the
# scripts exactly: changing them would change their output.
# global_check.sh and ss7_check.sh also drop "Free Software lives", which
# vty-debug-dump.sh keeps - hence the optional argument:
#     osmo_vty_clean 'Free Software lives'
osmo_vty_clean() {
    local extra="${1:-}"
    local re="^(Trying|Connected|Escape|Welcome|OsmoSTP|OsmoHLR|OsmoMSC|\
OsmoBSC|OsmoBTS|OsmoMGW|OsmoPCU|OsmoSGSN|OsmoGGSN|OsmoSIP|\
VTY server|Use.*help|Press.*tab|[A-Za-z0-9_-]+[#>] |\
Enter password|% Unknown|% Command incomplete|% Error|\
Connection closed)"
    [ -n "$extra" ] && re="${re%)}|${extra})"
    grep -avE "$re" | sed 's/\r//' | grep -av '^[[:space:]]*$' || true
}

# osmo_vty_interactive <op> <port> [host] - opens an INTERACTIVE VTY session
# (a human at the keyboard), whereas osmo_vty sends commands and returns output.
# 1:1 replacement for "docker exec -ti <container> telnet <ip> <port>".
# The host is a PARAMETER because Baseband groups listen on 127.0.0.1,
# 127.0.0.2... (tools/vty-menu.sh): the only VTY target in the repository
# other than 127.0.0.1.
osmo_vty_interactive() {
    local op="${1:-}" port="${2:-}" host="${3:-127.0.0.1}"
    case "$port" in ''|*[!0-9]*) return 2 ;; esac
    if [ "$(osmo_mode)" = docker ]; then
        docker exec -ti "$(osmo_node "$op")" telnet "$host" "$port"
        return $?
    fi
    local ns; ns="$(_osmo_ns_cmd "$op")"
    # shellcheck disable=SC2086  # ns must be word-split (ip netns exec X)
    if [ -n "$ns" ]; then $ns telnet "$host" "$port"; else telnet "$host" "$port"; fi
}

# osmo_vty_target <op> - "target" to hand to a non-bash VTY client that
# cannot source this library (tools/vty-connect.exp, written in expect).
# The mode is decided in bash (tools/vty-menu.sh) and passed as an ARGUMENT:
#     docker -> the container name, which becomes "docker exec -ti <name> telnet"
#     native -> "-", which becomes a direct "telnet"
# "-" rather than an empty string: an empty string is lost in an expect
# [lindex] as in a bash "$@", and "native" becomes indistinguishable from
# "missing argument".
osmo_vty_target() {
    if [ "$(osmo_mode)" = docker ]; then osmo_node "${1:-}"; else printf -- '-\n'; fi
```

```
}

# ============================================================================
#  5. CATEGORY A - CLEAN REFUSAL WHEN DOCKER IS MISSING
# ============================================================================
#
#  Some scripts inherently DRIVE docker (build an image, create a
#  172.20.N.0/24 network, start a container) and have no native equivalent;
#  the native counterparts are start-native.sh and start-interstp.sh.
#  On an ISO without docker these scripts must refuse cleanly and name the
#  alternative instead of failing with "docker: command not found".

# osmo_docker_ok - is docker REALLY usable from here?
# The binary is not enough (dev machines often have it without a running
# daemon), so the daemon is queried. No pipe: no pipefail trap.
osmo_docker_ok() {
    command -v docker >/dev/null 2>&1 || return 1
    docker info >/dev/null 2>&1
}

# osmo_require_docker [alternative line...] - clean refusal, then exit 3.
# Returns 0 silently if docker is usable, so it can be called unconditionally
# at the top of a script.
#
# The message says WHAT IS MISSING, WHY this script needs it, and WHAT TO RUN
# instead, by exact name. Without alternatives given as arguments, the two
# repository ones are named.
#
# Output on stderr, exit code 3: distinct from 1 (execution failure) and 2
# (bad argument), so a caller can recognise "wrong world" without parsing text.
osmo_require_docker() {
    osmo_docker_ok && return 0
    {
        printf '\n%s\n\n' "Docker n'est pas utilisable sur cette machine."
        if command -v docker >/dev/null 2>&1; then
            printf '%s\n' "  Le binaire docker est la, mais le demon ne repond pas."
            printf '%s\n' "  → sudo systemctl start docker, puis relancer ce script."
        else
            printf '%s\n' "  La commande docker est absente - c'est le cas normal sur l'ISO,"
            printf '%s\n' "  en VM et sur une machine nue : la pile y est installee en dur."
        fi
        printf '\n%s\n' "Ce script pilote docker (images, reseaux, conteneurs). Il n'a pas"
        printf '%s\n' "d'equivalent natif a ecrire : l'equivalent EXISTE DEJA, et il a un nom."
        printf '%s\n\n' "Lancez plutot :"
        if [ $# -gt 0 ]; then
            local _l; for _l in "$@"; do printf '  %s\n' "$_l"; done
        else
            printf '  %s\n' "./start-native.sh     - la pile complete de CE noeud (un operateur)"
            printf '  %s\n' "./launch/start-interstp.sh   - le hub SS7 inter-operateurs"
        fi
        printf '\n%s\n' "Sur l'ISO ces deux-la sont deja dans le PATH : osmo-start-direct."
        printf '%s\n\n' "Pour verifier l'etat sans rien demarrer : ./checks/global_check.sh"
    } >&2
    exit 3
}

# ============================================================================
#  6. MISC
# ============================================================================

# osmo_qgrep <args...> - "grep -aq" without the pipefail trap.
# With "cmd | grep -aq pattern" grep exits on the FIRST match, the producer
# gets SIGPIPE and the pipe returns 141 under "set -o pipefail", so a match
# can intermittently read as no match. Here grep reads all input and its
# output is discarded. Same as qgrep() in network/setup-wan-mesh.sh.
osmo_qgrep() { grep "$@" >/dev/null; }
```

```
# ============================================================================
#  When executed instead of sourced: nothing destructive, just print what the
#  library detects. Serves as a quick self-test.
# ============================================================================
if [ "${BASH_SOURCE[0]}" = "$0" ]; then
    printf 'mode      : %s\n' "$(osmo_mode)"
    printf 'operateurs: %s\n' "$(osmo_ops | tr '\n' ' ')"
    if hub="$(osmo_hub)"; then
        printf 'inter-STP : %s (interrogeable ici)\n' "$hub"
    else
        printf 'inter-STP : %s\n' "$(osmo_hub_hint)"
    fi
    printf 'hote op.1 : %s\n' "$(osmo_host 1)"
    exit 0
fi
```

2.20 /opt/GSM/osmo-operator/checks/annuaire.sh

9508 octets, 190 lignes → 190 lignes

```
#!/bin/bash
# ============================================================================
# checks/annuaire.sh -- bench subscriber directory, cross-checked with the network
#
# Gathers in one page what is otherwise spread over several tools: does
# subscriber 20001 exist, does it have a key, is it attached, and which
# machine serves it. Sources:
#
#   data/*.txt  -> annuaire.csv    identities generated at startup
#   hlr.db                          does the subscriber exist, does it have a key
#   osmo-msc VTY                    is it ATTACHED right now
#   annuaire-ss7.csv + STP VTY      which machine serves it, under which PC
#
# OUTPUT
#   1. subscribers: IMSI, MSISDN, name, operator, country, key, attach state
#   2. inconsistencies: a record without subscriber, a subscriber without
#      record, a subscriber without key - the latter makes osmo-msc request
#      auth vectors, get none, and reject the phone as if the SIM were wrong
#   3. SS7 directory: point code, routing context, hub, and the current
#      STP view
#
# Read-only: modifies nothing, restarts nothing.
#
# Usage: ./checks/annuaire.sh [--op N] [--ss7] [--abo] [--no-color]
#   --op N      a single operator (default: all running ones)
#   --ss7       SS7 section only
#   --abo       subscriber section only
#   --no-color  plain output
# Env: OSMO_ANNUAIRE, OSMO_ANNUAIRE_SS7 override the directory CSV paths.
# ============================================================================
set -uo pipefail

ONLY_OP=""; WANT_ABO=1; WANT_SS7=1; USE_COLOR=1

# Directory files: start-docker.sh writes them on the HOST in /var/tmp and
# copies them into /etc/osmocom of each container. Use the first one found.
_premier(){ for f in "$@"; do [ -r "$f" ] && { printf '%s' "$f"; return 0; }; done; printf '%s' "$1"; }
ANNU="${OSMO_ANNUAIRE:-$(_premier /var/tmp/osmo-annuaire.csv /etc/osmocom/annuaire.csv)}"
ANNU_SS7="${OSMO_ANNUAIRE_SS7:-$(_premier /var/tmp/osmo-annuaire-ss7.csv /etc/osmocom/annuaire-ss7.csv)}"

while [ $# -gt 0 ]; do
  case "$1" in
    --op)       ONLY_OP="${2:-}"; shift ;;
```

```
    --op=*)     ONLY_OP="${1#*=}" ;;
    --ss7)      WANT_ABO=0 ;;
    --abo)      WANT_SS7=0 ;;
    --no-color) USE_COLOR=0 ;;
    -h|--help)  sed -n '2,29p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
  esac
  shift
done

if [ "$USE_COLOR" = 1 ] && [ -t 1 ]; then
  G=$'\033[0;32m'; R=$'\033[0;31m'; Y=$'\033[1;33m'; C=$'\033[0;36m'; B=$'\033[1m'; D=$'\033[2m'; Z=$'\033[0m'
else G=; R=; Y=; C=; B=; D=; Z=; fi
OK="${G}[ OK ]${Z}"; KO="${R}[FAIL]${Z}"; WARN="${Y}[WARN]${Z}"
head(){ printf '\n%s== %s ==%s\n' "$B" "$1" "$Z"; }

# ── Where are we running? ──────────────────────────────────────────
# On the host, query the containers; inside a container, query itself.
if [ -f /.dockerenv ]; then
  MODE=local
  IN(){ shift; bash -c "$*" 2>/dev/null; }          # $1 ignore
  OPS="${OPERATOR_ID:-1}"
else
  MODE=docker
  IN(){ local c="$1"; shift; docker exec "$c" bash -c "$*" 2>/dev/null; }
  OPS="$(docker ps --format '{{.Names}}' 2>/dev/null | sed -n 's/^osmo-operator-\([0-9]\+\)$/\1/p' | sort -n | tr
   ↪  '\n' ' ')"
fi
[ -n "$ONLY_OP" ] && OPS="$ONLY_OP"
[ -n "${OPS// /}" ] || { printf '%s aucun operateur en service\n' "$KO"; exit 1; }
ct(){ [ "$MODE" = docker ] && echo "osmo-operator-$1" || echo local; }

# ── Directory records, read once ──────────────────────────────────
# Missing is not an error: the directory is created by start-docker.sh, and a
# manually set up bench has none. Report it and continue; HLR and SS7 sections
# still work without it.
if [ -r "$ANNU" ]; then
  printf '%s fiches abonnes : %s (%s lignes)\n' "$OK" "$ANNU" "$(( $(wc -l < "$ANNU") - 1 ))"
else
  printf '%s pas de fiches abonnes (%s) - les noms manqueront\n' "$WARN" "$ANNU"
fi
if [ -r "$ANNU_SS7" ]; then
  printf '%s annuaire SS7    : %s (%s lignes)\n' "$OK" "$ANNU_SS7" "$(( $(wc -l < "$ANNU_SS7") - 1 ))"
else
  printf '%s pas d annuaire SS7 (%s)\n' "$WARN" "$ANNU_SS7"
fi

fiche(){   # $1 = msisdn, $2 = champ (7=nom 3=operateur 4=pays 8=adresse 1=imsi)
  [ -r "$ANNU" ] || return 1
  awk -F';' -v n="$1" -v f="$2" 'NR>1 && $2==n {print $f; exit}' "$ANNU"
}

# ═══════════════════════════ 1. SUBSCRIBERS ═══════════════════════════
if [ "$WANT_ABO" = 1 ]; then
for op in $OPS; do
  C_NAME="$(ct "$op")"
  head "ABONNES - operateur ${op} (${C_NAME})"

  # The HLR is authoritative for existence and key. Read the database rather
  # than the VTY ("subscriber show" handles one subscriber at a time). The key
  # column comes from auc_2g: a subscriber without a row there cannot authenticate.
  rows="$(IN "$C_NAME" 'command -v sqlite3 >/dev/null 2>&1 || exit 0
      sqlite3 -separator "|" /var/lib/osmocom/hlr.db "
        select s.imsi, coalesce(s.msisdn,\"\"),
               case when a.subscriber_id is null then \"non\" else \"oui\" end
        from subscriber s left join auc_2g a on a.subscriber_id = s.id
        order by s.imsi;"')"
```

```
  if [ -z "$rows" ]; then
    printf '  %s HLR illisible (sqlite3 absent, ou base vide)\n' "$KO"
    continue
  fi

  # Who is ATTACHED now: the VLR subscriber cache only holds subscribers that
  # have contacted the network; provisioned but never seen ones are absent.
  # osmo-msc has no "show subscriber all"; valid forms are cache, imsi, msisdn, tmsi.
  att="$(IN "$C_NAME" 'exec 9<>/dev/tcp/127.0.0.1/4254 2>/dev/null || exit 0
      { printf "enable\n"; printf "show subscriber cache\n"; sleep 0.5; } >&9
      timeout 3 cat <&9' | tr -d '\r' | sed 's/\x1b\[[0-9;]*m//g')"
  case "$att" in *"Unknown command"*) att="" ;; esac

  # ANSI codes count in a %-Ns width: pad the plain text first, then colorize,
  # otherwise each colored cell shifts the row.
  col(){ local t; printf -v t '%-*s' "$2" "$1"; printf '%s%s%s' "${3:-}" "$t" "${Z}"; }
  printf '  %-16s %-7s %-4s %-8s %-20s %-15s %s\n' \
         IMSI MSISDN CLE ETAT NOM OPERATEUR PAYS
  n_sans_cle=0; n_sans_fiche=0
  while IFS='|' read -r imsi msisdn cle; do
    [ -n "$imsi" ] || continue
    nom="$(fiche "$msisdn" 7)"; ope="$(fiche "$msisdn" 3)"; pays="$(fiche "$msisdn" 4)"
    if [ -z "$nom" ]; then nom="(pas de fiche)"; n_sans_fiche=$((n_sans_fiche+1)); nom_c="$D"; else nom_c=""; fi
    if printf '%s' "$att" | grep -q "$imsi"; then etat="attache"; etat_c="$G"; else etat="-"; etat_c="$D"; fi
    if [ "$cle" = non ]; then cle="NON"; cle_c="$R"; n_sans_cle=$((n_sans_cle+1)); else cle="oui"; cle_c="$G"; fi
    printf '  %-16s %-7s %s %s %s %-15s %s\n' \
           "$imsi" "${msisdn:-(aucun)}" \
           "$(col "$cle" 4 "$cle_c")" "$(col "$etat" 8 "$etat_c")" \
           "$(col "$nom" 20 "$nom_c")" "${ope:--}" "${pays:--}"
  done <<< "$rows"

  if [ "$n_sans_cle" -gt 0 ]; then
    printf '  %s %s abonne(s) SANS CLE : osmo-msc les trouve, reclame leurs vecteurs\n' "$KO" "$n_sans_cle"
    printf '          d authentification, n obtient rien, et rejette le mobile en\n'
    printf '          accusant la carte SIM. Reprovisionner : %ssubscriber imsi X update aud2g comp128v1 ki
     ↪  ...%s\n' "$C" "$Z"
  fi
  [ "$n_sans_fiche" -gt 0 ] && \
    printf '  %s %s abonne(s) sans fiche : le HLR les connait, l annuaire non (banc relance sans start-docker.sh
     ↪  ?)\n' "$WARN" "$n_sans_fiche"

  # Does the phone present an IMSI known to the HLR? Both values are written
  # by two different scripts and may disagree.
  for f in /root/.osmocom/bb/mobile.cfg /root/.osmocom/bb/mobile_faketrx_bts1.cfg; do
    msi="$(IN "$C_NAME" "awk '\$1==\"imsi\"{print \$2; exit}' $f")"
    [ -n "$msi" ] || continue
    if printf '%s\n' "$rows" | cut -d'|' -f1 | grep -qx "$msi"; then
      printf '  %s %-28s presente %s - connu du HLR\n' "$OK" "$(basename "$f")" "$msi"
    else
      printf '  %s %-28s presente %s - INCONNU du HLR : pas d attachement possible\n' "$KO" "$(basename "$f")"
       ↪  "$msi"
    fi
  done
done
fi

# ═══════════════════════════ 2. SS7 DIRECTORY ═══════════════════════════
if [ "$WANT_SS7" = 1 ] && [ -r "$ANNU_SS7" ]; then
  head "ANNUAIRE SS7"
  printf '  %-5s %-16s %-12s %-9s %-9s %-9s %-6s %s\n' \
         NOEUD OPERATEUR PAYS PC_MSC PC_STP PC_BSC RCTX HUB
  awk -F';' 'NR>1 {printf "  %-5s %-16s %-12s %-9s %-9s %-9s %-6s %s\n", $1,$2,$3,$6,$7,$8,$12,$13}' "$ANNU_SS7"

  # Current STP view: an active AS proves the machine is actually talking,
  # a point code in a file does not.
  for op in $OPS; do
```

```
    C_NAME="$(ct "$op")"
    as_out="$(IN "$C_NAME" 'exec 9<>/dev/tcp/127.0.0.1/4239 2>/dev/null || exit 0
        { printf "enable\n"; printf "show cs7 instance 0 as all\n"; sleep 0.5; } >&9
        timeout 3 cat <&9' | tr -d '\r' | sed 's/\x1b\[[0-9;]*m//g')"
    if [ -z "$as_out" ]; then
      printf '  %s operateur %s : VTY STP (4239) muet\n' "$KO" "$op"
      continue
    fi
    act="$(printf '%s\n' "$as_out" | grep -c 'AS_ACTIVE' || true)"
    tot="$(printf '%s\n' "$as_out" | grep -cE '^as-' || true)"
    if [ "${act:-0}" -gt 0 ]; then
      printf '  %s operateur %s : %s/%s AS actifs\n' "$OK" "$op" "$act" "$tot"
    else
      printf '  %s operateur %s : aucun AS actif (%s declares)\n' "$KO" "$op" "$tot"
    fi
  done
fi

exit 0
```

2.21 /opt/GSM/osmo-operator/checks/check_all.sh

22823 octets, 472 lignes → 472 lignes

```
#!/bin/bash
# ============================================================================
# checks/check_all.sh -- run ALL bench checks in the right order and return
#                        ONE verdict.
#
# checks/ holds scripts that do not overlap: global_check sees the daemons,
# ss7_check the signalling, diag-* the interconnect from either side,
# wan_ss7_check the traffic between machines, annuaire the subscribers.
# This script selects WHAT MAKES SENSE HERE (mode and role decide), runs them
# from the most general to the most specific, logs everything, and ends with
# a table: one row per check, one verdict, one exit code.
#
# WHAT IT DOES NOT DO
#   - it modifies nothing and restarts nothing: all called checks are
#     read-only (only --dump changes daemon log levels, so it is optional and
#     never enabled by default);
#   - it reimplements no test: each verdict comes from the original script,
#     never from a second VTY read done here.
#
# CHECK SELECTION - WHAT RUNS, AND WHEN
#   mode           all checks honour --docker/--native; the option is passed
#                  through unchanged, so a single world is inspected.
#   diag-interstp  only if an inter-STP is QUERYABLE HERE (osmo-inter-stp
#                  container, or OSMO_ROLE=interstp): on an operator node the
#                  hub is another machine whose VTY listens on its loopback
#                  only, so running it would report a false "hub silent".
#   diag-stp-oper. the exact opposite: only when this node is NOT the hub and
#                  has an /etc/osmo-role (i.e. a WAN setup).
#   wan_ss7_check  only if /etc/osmo-wan.conf exists: without a WAN plan there
#                  is nothing "between nodes" to verify.
#   annuaire       only if the directory files exist (/var/tmp or /etc/osmocom).
#   dump + resume  never by default: vty-debug-dump.sh switches the daemons to
#                  DEBUG and writes several MB. --dump enables both, the
#                  summary rereading the dump just written.
#
# HOW A CHECK VERDICT IS DERIVED
# The checks/ scripts do not share one convention:
#   rc    global_check.sh / ss7_check.sh return the NUMBER of failures;
#   rcb   wan_ss7_check.sh returns 0 or 1;
#   motif diag-*.sh and annuaire.sh always return 0 - their verdict is in
```

```
#         their last section ("INTERCO SS7 : DOWN"), read from the output
#         with colors stripped.
#
# Usage:
#   sudo ./checks/check_all.sh [options]
#     --quick|-q        passed to global_check / ss7_check (quick pass)
#     --verbose|-v      passed to the checks that support it
#     --op=N            limit to one operator (global_check, annuaire)
#     --docker|--native force the world; without it, auto-detection
#     --mode=X          same as --docker / --native
#     --dump            add vty-debug-dump.sh then operator_summary.sh
#     --only=a,b        run only these checks (see --list)
#     --skip=a,b        run all checks except these
#     --list            show the checks, their key and applicability
#     --out FILE        full log (default /tmp/osmo-check-all-<date>.txt)
#     --no-log          write nothing to disk
#     --timeout N       maximum time per check, in seconds (default 300)
#     --no-color        plain output
#
# Env: OSMO_MODE=docker|native forces the world like --docker/--native.
#
# Exit code: number of failed checks (0 = all good), at most 125.
#
# Check keys: global ss7 interstp operator wan annuaire dump resume.
# Examples:
#   sudo ./checks/check_all.sh --quick
#   sudo ./checks/check_all.sh --only=global,ss7 --no-log
#   sudo ./checks/check_all.sh --native --dump
# ============================================================================

set -u

# ── Shared library (docker/native detection, inventory, VTY) ─────────────
# Path relative to THIS file: the repository lives in a home directory during
# development and in /opt/GSM/osmo-operator on the ISO.
_here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
for _c in "$_here/_mode.sh" /opt/GSM/osmo-operator/checks/_mode.sh; do
    [ -r "$_c" ] && { . "$_c"; break; }
done
command -v osmo_mode >/dev/null || { echo "checks/_mode.sh introuvable" >&2; exit 1; }

# ── Options ──────────────────────────────────────────────────────────────
QUICK=0; VERBOSE=0; SPECIFIC_OP=""; WANT_DUMP=0
ONLY=""; SKIP_LIST=""; LIST_ONLY=0
LOGFILE=""; NO_LOG=0; TIMEOUT=300; USE_COLOR=1
MODE_FORCED=0; MODE_OPT=""

# OSMO_MODE in the environment counts as forcing ("OSMO_MODE=native
# ./checks/check_all.sh"); it is passed to the children as an option.
case "${OSMO_MODE:-}" in docker|native) MODE_FORCED=1 ;; esac

while [ $# -gt 0 ]; do
    case "$1" in
        --quick|-q)   QUICK=1 ;;
        --verbose|-v) VERBOSE=1 ;;
        --op)         SPECIFIC_OP="${2:-}"; shift ;;
        --op=*)       SPECIFIC_OP="${1#*=}" ;;
        --dump)       WANT_DUMP=1 ;;
        --only)       ONLY="${2:-}"; shift ;;
        --only=*)     ONLY="${1#*=}" ;;
        --skip)       SKIP_LIST="${2:-}"; shift ;;
        --skip=*)     SKIP_LIST="${1#*=}" ;;
        --list)       LIST_ONLY=1 ;;
        --out)        LOGFILE="${2:-}"; shift ;;
        --out=*)      LOGFILE="${1#*=}" ;;
        --no-log)     NO_LOG=1 ;;
```

```
        --timeout)    TIMEOUT="${2:-}"; shift ;;
        --timeout=*)  TIMEOUT="${1#*=}" ;;
        --no-color)   USE_COLOR=0 ;;
        --docker|--native|--mode=*)
            osmo_mode_force "$1" || { echo "mode inconnu : $1" >&2; exit 1; }
            MODE_FORCED=1 ;;
        -h|--help)    sed -n '2,69p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *)            echo "option inconnue : $1 (voir --help)" >&2; exit 1 ;;
    esac
    shift
done

case "$TIMEOUT" in ''|*[!0-9]*) echo "--timeout attend un entier" >&2; exit 1 ;; esac

# ── Colors ──────────────────────────────────────────────────────────
if [ "$USE_COLOR" = 1 ] && [ -t 1 ]; then
    GREEN=$'\033[0;32m'; CYAN=$'\033[0;36m'; YELLOW=$'\033[1;33m'
    RED=$'\033[0;31m';   BOLD=$'\033[1m';    DIM=$'\033[2m'; NC=$'\033[0m'
else
    GREEN=; CYAN=; YELLOW=; RED=; BOLD=; DIM=; NC=
fi

banner() {
    echo ""
    echo "${BOLD}$*${NC}"
    printf '%s\n' "$(printf '─%.0s' $(seq 1 74))"
}

# ── Mode, role, inventory ──────────────────────────────────────────
MODE="$(osmo_mode)"
[ "$MODE_FORCED" -eq 1 ] && MODE_SRC="force" || MODE_SRC="detecte"

# Mode option passed to the children. ALWAYS passed, even when detected here:
# otherwise each child redetects, and on a host with docker installed but the
# lab running natively two detections may disagree.
MODE_OPT="--$MODE"

HUB_LOCAL=0; HUB_LABEL=""
if HUB_LABEL="$(osmo_hub 2>/dev/null)"; then HUB_LOCAL=1; else HUB_LABEL=""; fi

ROLE=""
[ -r /etc/osmo-role ] && \
    ROLE="$(awk -F= '/^OSMO_ROLE=/ { gsub(/[ \r\t]/, "", $2); print $2 }' /etc/osmo-role 2>/dev/null | tail -1)"

OPS="$(osmo_ops 2>/dev/null | tr '\n' ' ' | sed 's/ *$//')"
N_OPS="$(printf '%s\n' "$OPS" | wc -w | tr -d ' ')"

# Subscriber directory files: start-docker.sh writes them on the HOST in
# /var/tmp and copies them into /etc/osmocom of each container. Same rule as
# checks/annuaire.sh.
ANNU_FOUND=0
for f in "${OSMO_ANNUAIRE:-}" /var/tmp/osmo-annuaire.csv /etc/osmocom/annuaire.csv; do
    [ -n "$f" ] && [ -r "$f" ] && { ANNU_FOUND=1; break; }
done

# ── Log ─────────────────────────────────────────────────────────────
# The log receives the output WITHOUT colors; the terminal keeps them.
STRIP_ANSI='s/\x1b\[[0-9;]*[a-zA-Z]//g'
if [ "$NO_LOG" -eq 0 ]; then
    [ -n "$LOGFILE" ] || LOGFILE="/tmp/osmo-check-all-$(date '+%Y%m%d-%H%M%S').txt"
    if : >"$LOGFILE" 2>/dev/null; then :; else
        echo "${YELLOW}journal impossible a ecrire (${LOGFILE}) - on continue sans${NC}" >&2
        NO_LOG=1; LOGFILE=""
    fi
else
    LOGFILE=""
```

```
fi
journal() { [ -n "$LOGFILE" ] && printf '%s\n' "$*" | sed "$STRIP_ANSI" >>"$LOGFILE"; return 0; }

# ── Catalogue ──────────────────────────────────────────────────────────
# One record per check:  key | script | verdict | description
#   verdict = rc     the script returns the NUMBER of failures (global_check, ss7_check)
#             rcb    the script returns 0 or non-0 (wan_ss7_check)
#             motif  the script always returns 0: the verdict is read from its
#                    output, pattern given after ':' (diag-*, annuaire)
#             info   informational output, never failing (dump, resume)
# Order follows the diagnosis: running daemons, local signalling,
# interconnect, WAN, then subscribers.
CATALOGUE=(
  "global|global_check.sh|rc|etat de tous les demons (STP, HLR, MSC, BSC, BTS, PCU, SGSN, GGSN, MGW)"
  "ss7|ss7_check.sh|rc|signalisation SS7 : AS, ASP, routes, catch-all vers l inter-STP"
  "interstp|diag-interstp.sh|motif:INTERCO SS7 : (DOWN|PARTIELLE)|indeterminable|interco vue du HUB : qui est
   ↪  attache, qui manque"
  "operator|diag-stp-operator.sh|motif:INTERCO SS7 : DOWN|interco vue du NOEUD : conf disque vs demon, SCTP vers
   ↪  le hub"
  "wan|wan_ss7_check.sh|rcb|entre noeuds : SIP/RTP, relais SMS, point codes, iptables"
  "annuaire|annuaire.sh|motif:\\[FAIL\\]|abonnes : IMSI, MSISDN, cles Ki, attachement, annuaire SS7"
  "dump|vty-debug-dump.sh|info|dump DEBUG de tous les VTY (--dump seulement)"
  "resume|operator_summary.sh|info|resume par operateur, lu sur le dump (--dump seulement)"
)

champ() { printf '%s' "$1" | cut -d'|' -f"$2"; }
# The pattern may contain '|' (regex alternation): it spans everything
# between the 3rd field and the LAST one, the description being the last.
verdict_de()     { printf '%s' "${1#*|*|}" | sed 's/|[^|]*$//'; }
description_de() { printf '%s' "$1" | awk -F'|' '{print $NF}'; }

# applicable <key> - does this check make sense here? Prints the reason if not.
applicable() {
    case "$1" in
        global|ss7) return 0 ;;
        interstp)
            [ "$HUB_LOCAL" -eq 1 ] && return 0
            printf 'aucun inter-STP interrogeable ici (%s)' "$(osmo_hub_hint 2>/dev/null || echo 'hub distant ou
             ↪  absent')"
            return 1 ;;
        operator)
            [ "$HUB_LOCAL" -eq 1 ] && { printf 'ce noeud EST le hub - voir le check interstp'; return 1; }
            [ -r /etc/osmo-role ] && return 0
            printf '/etc/osmo-role absent : pas de montage WAN sur ce noeud'
            return 1 ;;
        wan)
            [ -r /etc/osmo-wan.conf ] && return 0
            printf '/etc/osmo-wan.conf absent : aucun plan WAN a verifier'
            return 1 ;;
        annuaire)
            [ "$ANNU_FOUND" -eq 1 ] && return 0
            printf 'aucune fiche (osmo-annuaire.csv) : banc lance sans annuaire'
            return 1 ;;
        dump|resume)
            [ "$WANT_DUMP" -eq 1 ] && return 0
            printf 'non demande (--dump)'
            return 1 ;;
    esac
    return 0
}

# args_de <key> - options to pass; each script takes different ones.
args_de() {
    local a=("$MODE_OPT")
    case "$1" in
        global)
```

```
            [ "$QUICK" -eq 1 ]        && a+=(--quick)
            [ "$VERBOSE" -eq 1 ]      && a+=(--verbose)
            [ -n "$SPECIFIC_OP" ]     && a+=("--op=$SPECIFIC_OP") ;;
        ss7)
            [ "$QUICK" -eq 1 ]        && a+=(--quick)
            [ "$VERBOSE" -eq 1 ]      && a+=(--verbose) ;;
        interstp)
            # diag-interstp.sh and diag-stp-operator.sh do not use _mode.sh: they read
            # /etc/osmo-role and the local VTYs and reject --docker/--native as an
            # unknown option, so MODE_OPT is not passed.
            a=()
            [ "$USE_COLOR" -eq 0 ]    && a+=(--no-color) ;;
        operator)
            a=()
            [ "$USE_COLOR" -eq 0 ]    && a+=(--no-color) ;;
        wan)
            a=()
            [ "$VERBOSE" -eq 1 ]      && a+=(--verbose) ;;
        annuaire)
            a=()
            [ -n "$SPECIFIC_OP" ]     && a+=(--op "$SPECIFIC_OP")
            [ "$USE_COLOR" -eq 0 ]    && a+=(--no-color) ;;
        dump)
            a+=(--out "$DUMP_OUT") ;;
        resume)
            a+=("--dump=$DUMP_OUT") ;;
    esac
    printf '%s\n' "${a[@]:-}"
}

# ── --list: show what would be done, without doing it ──────────────────
if [ "$LIST_ONLY" -eq 1 ]; then
    echo "${BOLD}checks disponibles${NC}   (mode ${MODE}, ${MODE_SRC})"
    echo ""
    printf '  %-10s %-24s %s\n' CLE SCRIPT ETAT
    printf '  %s\n' "$(printf '─%.0s' $(seq 1 72))"
    for e in "${CATALOGUE[@]}"; do
        k="$(champ "$e" 1)"; s="$(champ "$e" 2)"
        if raison="$(applicable "$k")"; then
            printf '  %-10s %-24s %slance%s\n' "$k" "$s" "$GREEN" "$NC"
        else
            printf '  %-10s %-24s %signore%s - %s\n' "$k" "$s" "$DIM" "$NC" "$raison"
        fi
        printf '  %-10s %-24s %s%s%s\n' "" "" "$DIM" "$(description_de "$e")" "$NC"
    done
    echo ""
    exit 0
fi

# ── Header ──────────────────────────────────────────────────────────────
DUMP_OUT="/tmp/osmo-check-all-dump-$(date '+%Y%m%d-%H%M%S').txt"
DATE_DEB="$(date '+%Y-%m-%d %H:%M:%S')"

en_tete() {
    echo "${CYAN}${BOLD}"
    echo "╔══════════════════════════════════════════════════════════════════════════╗"
    echo "║   osmo-operator - CHECK COMPLET du banc                                  ║"
    echo "╚══════════════════════════════════════════════════════════════════════════╝"
    echo "${NC}"
    printf '  date       : %s\n' "$DATE_DEB"
    printf '  machine    : %s\n' "$(hostname 2>/dev/null || echo '?')"
    printf '  mode       : %s%s%s (%s)\n' "$CYAN" "$MODE" "$NC" "$MODE_SRC"
    printf '  operateurs : %s   (%s)\n' "${N_OPS:-0}" "${OPS:-aucun}"
    if [ "$HUB_LOCAL" -eq 1 ]; then
        printf '  inter-STP  : %s%s%s (interrogeable ici)\n' "$GREEN" "$HUB_LABEL" "$NC"
    else
```

```
        printf '  inter-STP  : %s\n' "$(osmo_hub_hint 2>/dev/null || echo 'non interrogeable ici')"
    fi
    [ -n "$ROLE" ] && printf '  role       : %s\n' "$ROLE"
    [ -n "$LOGFILE" ] && printf '  journal    : %s\n' "$LOGFILE"
    # Osmocom VTYs listen on loopback and docker needs the socket: without root,
    # half the checks would report "silent" for reasons unrelated to the network.
    # Warn up front.
    [ "$(id -u)" -ne 0 ] && printf '  %s! lance sans root : certains checks seront incomplets (sudo
     ↪  recommande)%s\n' "$YELLOW" "$NC"
}
en_tete
en_tete_txt="$(en_tete 2>&1)"
journal "$en_tete_txt"

# ── Execution ──────────────────────────────────────────────────────────
NB_OK=0; NB_KO=0; NB_IGN=0
RES_CLE=(); RES_ETAT=(); RES_RC=(); RES_DUREE=(); RES_NOTE=()

dans_liste() {   # dans_liste <aiguille> <a,b,c>
    local x; local IFS=','
    for x in $2; do [ "$x" = "$1" ] && return 0; done
    return 1
}

enregistre() {   # enregistre <cle> <etat> <rc> <duree> <note>
    RES_CLE+=("$1"); RES_ETAT+=("$2"); RES_RC+=("$3"); RES_DUREE+=("$4"); RES_NOTE+=("${5:-}")
}

for entree in "${CATALOGUE[@]}"; do
    cle="$(champ "$entree" 1)"
    script="$(champ "$entree" 2)"
    verdict="$(verdict_de "$entree")"
    desc="$(description_de "$entree")"
    chemin="$_here/$script"

    # --only / --skip first: user-excluded checks are not evaluated and are not
    # reported as "not applicable" (a choice, not a finding).
    [ -n "$ONLY" ]      && { dans_liste "$cle" "$ONLY"      || continue; }
    [ -n "$SKIP_LIST" ] && { dans_liste "$cle" "$SKIP_LIST" && continue; }

    if ! raison="$(applicable "$cle")"; then
        banner "[$cle] $script"
        echo "  ${DIM}ignore${NC} - $raison"
        journal ""; journal "== [$cle] $script =="; journal "  ignore - $raison"
        NB_IGN=$((NB_IGN+1)); enregistre "$cle" IGNORE "-" "-" "$raison"
        continue
    fi

    if [ ! -x "$chemin" ]; then
        banner "[$cle] $script"
        echo "  ${RED}absent ou non executable${NC} : $chemin"
        journal ""; journal "== [$cle] $script =="; journal "  absent : $chemin"
        NB_KO=$((NB_KO+1)); enregistre "$cle" ABSENT "-" "-" "$chemin introuvable"
        continue
    fi

    mapfile -t opts < <(args_de "$cle")
    # args_de prints an empty line when there are no options; avoid passing an
    # empty argument, which some scripts reject.
    [ "${#opts[@]}" -eq 1 ] && [ -z "${opts[0]}" ] && opts=()

    banner "[$cle] $script ${DIM}${opts[*]:-}${NC}"
    echo "  ${DIM}${desc}${NC}"
    echo ""
    journal ""
    journal "═══════════════════════════════════════════════════════════════════════"
```

```
    journal "== [$cle] $script ${opts[*]:-}"
    journal "   $desc"
    journal "═══════════════════════════════════════════════════════════════════════"

    tmp="$(mktemp "${TMPDIR:-/tmp}/osmo-check-$cle-XXXXXX")"
    debut=$SECONDS
    # timeout: a VTY that never answers (frozen daemon, hung docker exec) would
    # block the whole report; cut it, report it, continue.
    # PIPESTATUS rather than "set -o pipefail": under pipefail a trailing
    # "grep -q" in a child would return 141 (see checks/_mode.sh).
    timeout "$TIMEOUT" "$chemin" "${opts[@]}" 2>&1 | tee "$tmp"
    rc="${PIPESTATUS[0]}"
    duree=$((SECONDS - debut))

    [ -n "$LOGFILE" ] && sed "$STRIP_ANSI" "$tmp" >>"$LOGFILE"
    propre="$(sed "$STRIP_ANSI" "$tmp")"
    rm -f "$tmp"

    etat=OK; note=""
    if [ "$rc" -eq 124 ]; then
        etat=TIMEOUT; note="coupe apres ${TIMEOUT}s"
    else
        case "$verdict" in
            rc)
                # global_check.sh and ss7_check.sh return the number of failures.
                if [ "$rc" -ne 0 ]; then etat=ECHEC; note="$rc test(s) en echec"; fi ;;
            rcb)
                if [ "$rc" -ne 0 ]; then etat=ECHEC; note="code de retour $rc"; fi ;;
            motif:*)
                # The script always returns 0: its verdict is in its output. Keep the whole
                # LINE, not just the pattern, so the summary is self-explanatory.
                m="${verdict#motif:}"
                nb="$(printf '%s\n' "$propre" | grep -Ec -- "$m")"
                if [ "${nb:-0}" -gt 0 ]; then
                    etat=ECHEC
                    note="$(printf '%s\n' "$propre" | grep -Em1 -- "$m" | sed 's/^[[:space:]]*//;
                     ↪  s/[[:space:]]*$//' | cut -c1-60)"
                    [ "$nb" -gt 1 ] && note="$note (+$((nb-1)))"
                fi ;;
            info)
                etat=INFO ;;
        esac
    fi

    case "$etat" in
        OK)      NB_OK=$((NB_OK+1));  echo ""; echo "  ${GREEN}→ $cle : OK${NC} (${duree}s)" ;;
        INFO)    NB_OK=$((NB_OK+1));  echo ""; echo "  ${CYAN}→ $cle : termine${NC} (${duree}s)" ;;
        TIMEOUT) NB_KO=$((NB_KO+1));  echo ""; echo "  ${RED}→ $cle : TIMEOUT${NC} - $note" ;;
        *)       NB_KO=$((NB_KO+1));  echo ""; echo "  ${RED}→ $cle : PROBLEMES${NC} - $note (${duree}s)" ;;
    esac
    journal "  → $cle : $etat ${note} (${duree}s, rc=$rc)"

    enregistre "$cle" "$etat" "$rc" "${duree}s" "$note"
done

# ── Summary ──────────────────────────────────────────────────────────
bilan() {
    echo ""
    printf '%s\n' "$(printf '=%.0s' $(seq 1 74))"
    echo "  ${BOLD}BILAN - $(date '+%Y-%m-%d %H:%M:%S')${NC}"
    printf '%s\n' "$(printf '=%.0s' $(seq 1 74))"
    printf '  %-10s %-9s %-6s %-7s %s\n' CHECK ETAT RC DUREE COMMENTAIRE
    printf '  %s\n' "$(printf '-%.0s' $(seq 1 72))"
    local i coul
    for i in "${!RES_CLE[@]}"; do
        case "${RES_ETAT[$i]}" in
```

```
            OK)                 coul="$GREEN" ;;
            INFO)               coul="$CYAN" ;;
            IGNORE)             coul="$DIM" ;;
            ECHEC|TIMEOUT|ABSENT) coul="$RED" ;;
            *)                  coul="$YELLOW" ;;
        esac
        printf '  %-10s %s%-9s%s %-6s %-7s %s\n' \
            "${RES_CLE[$i]}" "$coul" "${RES_ETAT[$i]}" "$NC" \
            "${RES_RC[$i]}" "${RES_DUREE[$i]}" "${RES_NOTE[$i]}"
    done
    printf '  %s\n' "$(printf '—%.0s' $(seq 1 72))"
    printf '  %s%d ok%s   %s%d en echec%s   %s%d ignore%s   (mode %s)\n' \
        "$GREEN" "$NB_OK" "$NC" "$RED" "$NB_KO" "$NC" "$DIM" "$NB_IGN" "$NC" "$MODE"
    echo ""
    if [ "$NB_KO" -eq 0 ] && [ "$NB_OK" -eq 0 ]; then
        # Everything skipped or excluded: nothing was inspected, so do not report OK.
        echo "  ${YELLOW}${BOLD}AUCUN CHECK N A TOURNE${NC} - tout etait ignore ou exclu (--list pour savoir
        ↪  pourquoi)"
    elif [ "$NB_KO" -eq 0 ]; then
        echo "  ${GREEN}${BOLD}BANC OK${NC} - ${NB_OK} check(s) passes, aucun en echec"
    else
        echo "  ${RED}${BOLD}PROBLEMES DETECTES${NC} - ${NB_KO} check(s) en echec"
        echo ""
        echo "  ${DIM}rejouer un seul check en detail :${NC}"
        for i in "${!RES_CLE[@]}"; do
            case "${RES_ETAT[$i]}" in
                ECHEC|TIMEOUT) echo "    ./checks/check_all.sh --only=${RES_CLE[$i]} --verbose" ;;
            esac
        done
    fi
    [ -n "$LOGFILE" ] && { echo ""; echo "  ${CYAN}journal complet :${NC} $LOGFILE"; }
    [ "$WANT_DUMP" -eq 1 ] && [ -s "$DUMP_OUT" ] && echo "  ${CYAN}dump VTY        :${NC} $DUMP_OUT"
    printf '%s\n' "$(printf '=%.0s' $(seq 1 74))"
    echo ""
}
bilan
journal "$(bilan)"

# Same convention as global_check.sh / ss7_check.sh: exit code = number of
# failures, capped at 125 (126, 127 and 128+n have special meanings).
[ "$NB_KO" -gt 125 ] && NB_KO=125
exit "$NB_KO"
```

2.22 /opt/GSM/osmo-operator/checks/diag-interstp.sh

11326 octets, 247 lignes → 247 lignes

```
#!/bin/bash
# ============================================================================
# checks/diag-interstp.sh - SS7 diagnostic as seen from the HUB (inter-STP)
#
# Answers one question: WHO is attached, and who is missing?
# What is attached is read from the hub itself (AS Name column and remote
# address of "show cs7 instance 0 asp"). /etc/osmo-wan.conf is only used for
# the roll call of absent nodes: a node missing from that table cannot be
# told apart from a node that is switched off.
#
# WHAT IT CHECKS
#   1. role and listen address (0.0.0.0 multi-homing is flagged)
#   2. daemon running, M3UA port listening
#   3. each AS declared by the hub: AS_ACTIVE / AS_DOWN, and attached ASPs
#   4. WAN table roll call: which expected address is seen nowhere
#   5. routes: PROHIB / UNAVAIL flagged
#   6. flapping: two measurements; renumbered dynamic ASPs reveal
```

```
#      associations that re-establish in a loop
#
# Read-only: changes nothing, restarts nothing.
#
# Usage: sudo ./checks/diag-interstp.sh [--watch] [--no-color]
#   --watch     second measurement at +10 s to detect flapping
# ============================================================================
set -uo pipefail

ROLE_FILE=/etc/osmo-role
WAN_FILE=/etc/osmo-wan.conf
CFG=/etc/osmocom/osmo-stp-interop.cfg
# The hub may have been moved off port 4239 (co-located with the operator
# STP, see start-interstp.sh): read its VTY port from its config.
VTY_PORT=4239
_hubp="$(sed -n 's/^ *bind 127\.0\.0\.1 \([0-9]*\)$/\1/p' "$CFG" 2>/dev/null | head -1)"
[ -n "$_hubp" ] && VTY_PORT="$_hubp"
M3UA_PORT=2908
WATCH=0; USE_COLOR=1

while [ $# -gt 0 ]; do
  case "$1" in
    --watch)    WATCH=1 ;;
    --no-color) USE_COLOR=0 ;;
    -h|--help)  sed -n '2,23p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
  esac
  shift
done

if [ "$USE_COLOR" = 1 ] && [ -t 1 ]; then
  G=$'\033[0;32m'; R=$'\033[0;31m'; Y=$'\033[1;33m'; C=$'\033[0;36m'; B=$'\033[1m'; Z=$'\033[0m'
else G=; R=; Y=; C=; B=; Z=; fi
OK="${G}[ OK ]${Z}"; KO="${R}[FAIL]${Z}"; WARN="${Y}[WARN]${Z}"
head(){ printf '\n%s== %s ==%s\n' "$B" "$1" "$Z"; }

vty(){ { printf 'enable\n'; for c in "$@"; do printf '%s\n' "$c"; done; sleep 0.6; } \
       | timeout 8 nc -q1 127.0.0.1 "$VTY_PORT" 2>/dev/null | tr -d '\r' | sed 's/\x1b\[[0-9;]*m//g'; }

getv(){ awk -F= -v k="$1" '$1==k{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' "$ROLE_FILE" 2>/dev/null; }

# ---- identity ---------------------------------------------------------------
head "IDENTITE"
ROLE="$(getv OSMO_ROLE)"; SELF="$(getv OSMO_HUB_IP)"
printf '  hostname : %s\n' "$(hostname)"
printf '  role     : %s%s%s   adresse annoncee : %s%s%s\n' "$C" "${ROLE:-?}" "$Z" "$C" "${SELF:-?}" "$Z"
[ "$ROLE" = "interstp" ] || printf '  %s role=%s : ce script vise le HUB (voir diag-stp-operator.sh)\n' "$WARN"
 ↪  "${ROLE:-?}"

# effective listen address
LISTEN_IP="$(awk '/listen m3ua/{f=1} f&&/local-ip/{print $2; exit}' "$CFG" 2>/dev/null)"
LISTEN_PORT="$(awk '/listen m3ua/{print $3; exit}' "$CFG" 2>/dev/null)"
printf '  ecoute   : %s:%s\n' "${LISTEN_IP:-?}" "${LISTEN_PORT:-?}"
if [ "$LISTEN_IP" = "0.0.0.0" ]; then
  printf '  %s local-ip 0.0.0.0 : SCTP annonce TOUTES les adresses de la machine\n' "$WARN"
  printf '         dans son INIT (NAT, alias 172.20.x, host-only). Les noeuds tentent\n'
  printf '         des chemins morts et les ASP montent puis retombent, en boucle.\n'
  printf '         Corriger : set_stp_ip.sh --inter --ip <adresse du hub>\n'
fi

# ---- daemon -----------------------------------------------------------------
head "DEMON"
ST="$(systemctl is-active osmo-interstp 2>/dev/null)"
[ -z "$ST" ] && ST="$(systemctl is-active osmo-stp 2>/dev/null)"
if [ "$ST" = active ]; then printf '  %s service actif (%s)\n' "$OK" "$ST"
else printf '  %s service inactif (%s)\n' "$KO" "${ST:-inconnu}"; fi
if ss -Sanl 2>/dev/null | grep -q ":${LISTEN_PORT:-$M3UA_PORT} "; then
```

```
  printf '  %s port M3UA %s en ecoute (SCTP)\n' "$OK" "${LISTEN_PORT:-$M3UA_PORT}"
else
  printf '  %s port M3UA %s PAS en ecoute\n' "$KO" "${LISTEN_PORT:-$M3UA_PORT}"
fi

# ---- WAN table (only used for the roll call of absent nodes) ----------------
declare -A EXP_IP=()
NODES=""
if [ -r "$WAN_FILE" ]; then
  # Anchored at line start: the file's own comment line mentions
  # WAN_NODES="id:ip:prefix ..." and must not be parsed as an entry.
  spec="$(awk -F\" '/^WAN_NODES=/{print $2}' "$WAN_FILE" 2>/dev/null)"
  for tok in $spec; do
    id="${tok%%:*}"; rest="${tok#*:}"; ip="${rest%%:*}"
    EXP_IP[$id]="$ip"; NODES="$NODES $id"
  done
fi

# ---- AS / ASP ---------------------------------------------------------------
head "AS DECLARES PAR LE HUB"
AS_OUT="$(vty 'show cs7 instance 0 as all')"
ASP_OUT="$(vty 'show cs7 instance 0 asp')"

# Attachment is read where the hub declares it (AS Name column and remote
# address of its own output), never derived from the local WAN table. Column
# positions vary between libosmo-sigtran versions: the state and the address
# are matched by their form, not by their rank.
ASP_ROWS="$(printf '%s\n' "$ASP_OUT" | awk '
  /ASP_/ {
    st = ""; rem = ""
    for (i = 2; i <= NF; i++) {
      if ($i ~ /^ASP_/) st = $i
      if ($i ~ /^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+(:[0-9]+)?$/) rem = $i
    }
    sub(/:[0-9]+$/, "", rem)
    printf "%s|%s|%s|%s\n", $1, $2, (st == "" ? "?" : st), (rem == "" ? "?" : rem)
  }')"
AS_ROWS="$(printf '%s\n' "$AS_OUT" | awk '$1 ~ /^as-/ {print $1 "|" $2}')"

if [ -z "$AS_OUT" ]; then
  printf '  %s VTY injoignable sur 127.0.0.1:%s -- nc absent, ou demon arrete\n' "$KO" "$VTY_PORT"
elif [ -z "$AS_ROWS" ]; then
  printf '  %s aucun AS declare : conf du hub vide, ou autre instance cs7\n' "$KO"
else
  while IFS='|' read -r as_name as_state; do
    [ -n "$as_name" ] || continue
    if [ "$as_state" = AS_ACTIVE ]; then m="$OK"; else m="$KO"; fi
    printf '  %s %-12s %s\n' "$m" "$as_name" "${as_state:-?}"
    attache=0
    while IFS='|' read -r asp_name asp_as asp_state asp_rem; do
      [ "$asp_as" = "$as_name" ] || continue
      attache=1
      printf '         %-12s %-12s depuis %s\n' "$asp_name" "$asp_state" "$asp_rem"
    done <<< "$ASP_ROWS"
    [ "$attache" = 0 ] && printf '         aucun ASP rattache : noeud eteint, asp en shutdown, ou SCTP bloque\n'
  done <<< "$AS_ROWS"
  # NF: with no ASP at all, printf still emits an empty line whose empty
  # field 2 must not be reported as an orphan.
  orph="$(printf '%s\n' "$ASP_ROWS" | awk -F'|' 'NF && $2 !~ /^as-/ {printf "%s(%s) ", $1, $4}')"
  if [ -n "$orph" ]; then
    printf '  %s ASP connecte mais rattache a aucun AS : %s\n' "$WARN" "${orph% }"
    printf '         son routing context ne correspond a aucun AS du hub\n'
  fi
fi

head "APPEL DE LA TABLE WAN (${WAN_FILE})"
```

```
if [ -z "$ASP_OUT" ]; then
  printf '  %s pas de reponse VTY : rien a confronter a la table\n' "$KO"
elif [ -z "$NODES" ]; then
  printf '  %s table absente ou vide : impossible de dire qui manque a l appel,\n' "$WARN"
  printf '         la liste ci-dessus reste complete pour ce qui est attache\n'
else
  # An attached address missing from the table = node masked by NAT: the hub
  # sees the router's address, not the node's. Such a node must not be
  # declared absent just because it is masked.
  tab=" "
  for n in $NODES; do tab="$tab${EXP_IP[$n]} "; done
  hors=""
  for ip in $(printf '%s\n' "$ASP_ROWS" | awk -F'|' '$4 ~ /^[0-9]/ {print $4}' | sort -u); do
    case "$tab" in *" $ip "*) ;; *) hors="$hors $ip" ;; esac
  done
  # An address outside the table can mask only ONE node; without this count
  # a single masked ASP would downgrade every truly absent node to WARN.
  n_hors=0
  for _ in $hors; do n_hors=$(( n_hors + 1 )); done
  for n in $NODES; do
    if printf '%s\n' "$ASP_ROWS" | awk -F'|' -v ip="${EXP_IP[$n]}" '$4 == ip {f = 1} END {exit !f}'; then
      printf '  %s %-15s %-20s (noeud %s de la table)\n' "$OK" "${EXP_IP[$n]}" "attachee" "$n"
    elif [ "$n_hors" -gt 0 ]; then
      n_hors=$(( n_hors - 1 ))
      printf '  %s %-15s %-20s (noeud %s de la table)\n' "$WARN" "${EXP_IP[$n]}" "pas vue telle quelle" "$n"
    else
      printf '  %s %-15s %-20s (noeud %s de la table)\n' "$KO" "${EXP_IP[$n]}" "aucune association" "$n"
    fi
  done
  [ -n "$hors" ] && printf '  %s attachees hors table :%s -- masquage NAT probable\n' "$WARN" "$hors"
fi

head "ASP ATTACHES"
if [ -n "$ASP_OUT" ]; then
  printf '%s\n' "$ASP_OUT" | grep -E 'asp-|ASP Name' | sed 's/^/  /' | cut -c1-160
else
  printf '  %s pas de reponse VTY\n' "$KO"
fi

# ---- routes ----------------------------------------------------------------
head "ROUTES"
RT="$(vty 'show cs7 instance 0 route')"
if [ -n "$RT" ]; then
  bad="$(printf '%s\n' "$RT" | grep -E 'PROHIB|UNAVAIL|INACC' || true)"
  n_ok="$(printf '%s\n' "$RT" | grep -cE '^[0-9]+\.[0-9]+\.[0-9]+/' || true)"
  printf '  %s routes declarees\n' "${n_ok:-0}"
  if [ -n "$bad" ]; then
    printf '  %s destinations interdites ou injoignables :\n' "$WARN"
    printf '%s\n' "$bad" | sed 's/^/    /'
    printf '    (TFP herite d un flapping : relancer osmo-stp sur le noeud concerne)\n'
  else
    printf '  %s aucune route PROHIB/UNAVAIL\n' "$OK"
  fi
else
  printf '  %s pas de reponse VTY\n' "$KO"
fi

# ---- flapping --------------------------------------------------------------
if [ "$WATCH" = 1 ]; then
  head "FLAPPING (2e mesure a +10 s)"
  first="$(printf '%s\n' "$ASP_OUT" | grep -oE 'asp-dyn-[0-9]+' | sort -u | tr '\n' ' ')"
  printf '  t0    : %s\n' "${first:-aucun}"
  sleep 10
  second="$(vty 'show cs7 instance 0 asp' | grep -oE 'asp-dyn-[0-9]+' | sort -u | tr '\n' ' ')"
  printf '  t+10s : %s\n' "${second:-aucun}"
  if [ "$first" = "$second" ]; then
```

```
    printf '  %s ASP stables - pas de reetablissement en boucle\n' "$OK"
  else
    printf '  %s les ASP dynamiques ont change de numero : les associations se\n' "$KO"
    printf '         reetablissent en boucle. Cause habituelle : local-ip 0.0.0.0\n'
    printf '         cote hub (multi-homing vers des adresses injoignables).\n'
  fi
fi

# ---- verdict -----------------------------------------------------------
head "VERDICT"
if [ -n "${AS_OUT:-}" ]; then
  # Two naming conventions coexist: as-n<node>op<op> in WAN mode and as-op<i>
  # otherwise (helpers/create_interop.sh); the verdict counts both.
  tot="$(printf '%s\n' "$AS_OUT" | grep -cE 'as-(n[0-9]+)?op[0-9]+' || true)"
  act="$(printf '%s\n' "$AS_OUT" | grep -E 'as-(n[0-9]+)?op[0-9]+' | grep -c 'AS_ACTIVE' || true)"
  if [ "${act:-0}" -gt 0 ] && [ "${act:-0}" = "${tot:-0}" ]; then
    printf '  %sINTERCO SS7 : UP%s  - %s/%s AS actifs\n' "$G$B" "$Z" "$act" "$tot"
  elif [ "${act:-0}" -gt 0 ]; then
    printf '  %sINTERCO SS7 : PARTIELLE%s - %s/%s AS actifs\n' "$Y$B" "$Z" "$act" "$tot"
    echo   "  les noeuds manquants : demon arrete, ASP en shutdown, ou adresse fausse."
    echo   "  sur le noeud : checks/diag-stp-operator.sh"
  else
    printf '  %sINTERCO SS7 : DOWN%s - aucun AS actif\n' "$R$B" "$Z"
    echo   "  a verifier, dans l ordre :"
    echo   "    - local-ip du hub (0.0.0.0 = multi-homing, voir plus haut)"
    echo   "    - les noeuds pointent-ils la bonne adresse ? (remote-ip)"
    echo   "    - leur ASP est-il en 'no shutdown' ?"
  fi
else
  printf '  %sindeterminable%s : le VTY du hub ne repond pas\n' "$R$B" "$Z"
fi
echo "------------------------------------------------------------------------"
exit 0
```

2.23 /opt/GSM/osmo-operator/checks/diag-stp-operator.sh

7802 octets, 161 lignes → 161 lignes

```
#!/bin/bash
# ============================================================================
# checks/diag-stp-operator.sh - SS7 / interconnect diagnostic from an operator node
#
# Reads the role and the WAN plan, compares the on-disk configuration with
# what the osmo-stp daemon actually runs (VTY), tests the SCTP transport to
# the inter-STP and gives a clear verdict: interconnect UP / DOWN + the cause.
#
# Read-only: changes nothing, restarts nothing.
#
# Usage: sudo ./checks/diag-stp-operator.sh [--node N] [--hub-ip IP] [--no-color]
#   --node N     WAN node number (default: OSMO_WAN_NODE from /etc/osmo-role)
#   --hub-ip IP  inter-STP address (default: OSMO_HUB_IP from /etc/osmo-role)
# ============================================================================
set -uo pipefail

ROLE_FILE=/etc/osmo-role
WAN_FILE=/etc/osmo-wan.conf
CONF_DIR=/etc/osmocom
STP_CFG="$CONF_DIR/osmo-stp.cfg"
MSC_CFG="$CONF_DIR/osmo-msc.cfg"
BSC_CFG="$CONF_DIR/osmo-bsc.cfg"
VTY_PORT=4239
M3UA_PORT=2908

NODE=""; HUB_IP=""; USE_COLOR=1
```

```
while [ $# -gt 0 ]; do
  case "$1" in
    --node) NODE="${2:-}"; shift ;;
    --hub-ip) HUB_IP="${2:-}"; shift ;;
    --no-color) USE_COLOR=0 ;;
    -h|--help) sed -n '2,14p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
  esac
  shift
done

if [ "$USE_COLOR" = 1 ] && [ -t 1 ]; then
  G=$'\033[0;32m'; R=$'\033[0;31m'; Y=$'\033[1;33m'; C=$'\033[0;36m'; B=$'\033[1m'; Z=$'\033[0m'
else G=; R=; Y=; C=; B=; Z=; fi
OK="${G}[ OK ]${Z}"; KO="${R}[FAIL]${Z}"; WARN="${Y}[WARN]${Z}"; NA="[ -- ]"
PROB=0
hr(){ printf '%s\n' "--------------------------------------------------------------------------------"; }
section(){ printf '\n%s== %s ==%s\n' "$B" "$1" "$Z"; }

# ---- identity ----------------------------------------------------------
getv(){ awk -F= -v k="$1" '$1==k{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' "$ROLE_FILE" 2>/dev/null; }
[ -z "$NODE" ]   && NODE="$(getv OSMO_WAN_NODE)"
[ -z "$HUB_IP" ] && HUB_IP="$(getv OSMO_HUB_IP)"
ROLE="$(getv OSMO_ROLE)"
[ -z "$NODE" ] && NODE="?"

# local IP of this node, from the WAN table when available
SELF_IP=""
if [ -r "$WAN_FILE" ]; then
  wl="$(awk -F\" '/WAN_NODES=/{print $2}' "$WAN_FILE" 2>/dev/null)"
  for tok in $wl; do
    id="${tok%%:*}"; rest="${tok#*:}"; ip="${rest%%:*}"
    [ "$id" = "$NODE" ] && SELF_IP="$ip"
  done
fi

section "IDENTITE"
printf '  hostname     : %s\n' "$(hostname)"
printf '  role         : %s%s%s   noeud : %s%s%s\n' "$C" "${ROLE:-?}" "$Z" "$C" "$NODE" "$Z"
printf '  inter-STP    : %s%s%s   ce noeud (WAN) : %s%s%s\n' "$C" "${HUB_IP:-?}" "$Z" "$C" "${SELF_IP:-?}" "$Z"
if [ "$ROLE" != "operator" ]; then
  printf '  %s ce n est pas un noeud operateur (role=%s) -- utilise diag-interstp.sh\n' "$WARN" "${ROLE:-?}"
fi

# expected plan for this node
if [ "$NODE" != "?" ]; then
  EXP_PC="1.${NODE}1.2"; EXP_MSC="1.${NODE}1.1"; EXP_BSC="1.${NODE}1.3"
  EXP_RK=$(( NODE*1000 + 150 ))
else
  EXP_PC=""; EXP_MSC=""; EXP_BSC=""; EXP_RK=""
fi

# ---- on-disk configuration ---------------------------------------------
section "CONF SUR DISQUE ($STP_CFG)"
f_pc="$(awk '/^cs7 instance/{c=1} c&&/^ *point-code /{print $2; exit}' "$STP_CFG" 2>/dev/null)"
f_rip="$(awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}' "$STP_CFG" 2>/dev/null)"
f_lip="$(awk '/asp asp-to-inter/{f=1} f&&/local-ip/{print $2; exit}' "$STP_CFG" 2>/dev/null)"
f_rk="$(awk '/as as-inter/{f=1} f&&/routing-key/{print $2; exit}' "$STP_CFG" 2>/dev/null)"
f_shut="$(awk '/asp asp-to-inter/{f=1} f&&/^ *no shutdown/{print "no"; exit} f&&/^ *shutdown/{print "yes"; exit}
 ↪  f&&/^ *as /{exit}' "$STP_CFG" 2>/dev/null)"
m_pc="$(awk '/^cs7 instance/{c=1} c&&/^ *point-code /{print $2; exit}' "$MSC_CFG" 2>/dev/null)"
b_pc="$(awk '/^cs7 instance/{c=1} c&&/^ *point-code /{print $2; exit}' "$BSC_CFG" 2>/dev/null)"

chk(){ # label got exp
  local label="$1" got="$2" exp="$3" m="$OK"
  if [ -n "$exp" ] && [ "$got" != "$exp" ]; then m="$KO"; PROB=$((PROB+1)); fi
  if [ -z "$got" ]; then m="$WARN"; got="(vide)"; fi
```

```
  printf '  %-26s %-16s %s\n' "$label" "$got" "$m$( [ -n "$exp" ]&&[ "$got" != "$exp" ]&&printf ' attendu %s'
   ↪  "$exp")"
}
chk "point-code STP"        "$f_pc"  "$EXP_PC"
chk "asp remote-ip (hub)"   "$f_rip" "$HUB_IP"
chk "asp local-ip (self)"   "$f_lip" "$SELF_IP"
chk "routing-key inter"     "$f_rk"  "$EXP_RK"
if [ "$f_shut" = "no" ]; then printf '  %-26s %-16s %s\n' "asp administratif" "no shutdown" "$OK"
elif [ "$f_shut" = "yes" ]; then printf '  %-26s %-16s %s\n' "asp administratif" "shutdown" "$KO"; PROB=$((PROB+1))
else printf '  %-26s %-16s %s\n' "asp administratif" "(defaut, actif)" "$OK"; fi
chk "point-code MSC"        "$m_pc"  "$EXP_MSC"
chk "point-code BSC"        "$b_pc"  "$EXP_BSC"

# ---- services ----------------------------------------------------------
section "SERVICES (systemd)"
for s in osmo-stp osmo-msc osmo-bsc osmo-hlr; do
  st="$(systemctl is-active "$s" 2>/dev/null)"
  m="$OK"; [ "$st" = active ] || { m="$KO"; PROB=$((PROB+1)); }
  printf '  %-14s %-10s %s\n' "$s" "${st:-inconnu}" "$m"
done

# ---- VTY: what the daemon actually runs --------------------------------
vty(){ # commands...
  { printf 'enable\n'; for c in "$@"; do printf '%s\n' "$c"; done; sleep 0.5; } \
    | timeout 6 nc -q1 127.0.0.1 "$VTY_PORT" 2>/dev/null | tr -d '\r' | sed 's/\x1b\[[0-9;]*m//g'
}
section "VTY osmo-stp (etat RUNTIME, port $VTY_PORT)"
ASP="$(vty 'show cs7 instance 0 asp')"
if [ -z "$ASP" ]; then
  printf '  %s VTY injoignable -- osmo-stp est-il lance ?\n' "$KO"; PROB=$((PROB+1))
  r_state=""; r_rem=""
else
  line="$(printf '%s\n' "$ASP" | grep -E 'asp-to-inter' | head -1)"
  r_state="$(printf '%s' "$line" | awk '{print $3}')"
  r_rem="$(printf '%s' "$line" | awk '{print $8}')"
  printf '  asp-to-inter  etat=%s%s%s  remote=%s\n' "$C" "${r_state:-absent}" "$Z" "${r_rem:-?}"
  nact="$(printf '%s\n' "$ASP" | grep -c 'ASP_ACTIVE')"
  printf '  ASP locaux ACTIVE (MSC/HLR internes) : %s\n' "$nact"
  # drift: file fixed but daemon not reloaded
  if [ -n "$r_rem" ] && [ -n "$f_rip" ] && [ "$r_rem" != "${f_rip}:${M3UA_PORT}" ] && [ "${r_rem%:*}" != "$f_rip"
   ↪  ]; then
    printf '  %s DERIVE : disque remote=%s mais demon tourne remote=%s -> reload/restart requis\n' "$WARN"
     ↪  "$f_rip" "$r_rem"
    PROB=$((PROB+1))
  fi
fi

# ---- transport ---------------------------------------------------------
section "TRANSPORT vers inter-STP ${HUB_IP:-?}:$M3UA_PORT"
if [ -n "$HUB_IP" ]; then
  if ping -c1 -W2 "$HUB_IP" >/dev/null 2>&1; then printf '  %s ping %s\n' "$OK" "$HUB_IP"
  else printf '  %s ping %s injoignable\n' "$KO" "$HUB_IP"; PROB=$((PROB+1)); fi
fi
estab="$(ss -Sanp 2>/dev/null | grep -E "${HUB_IP//./\\.}:$M3UA_PORT" | grep -i estab)"
if [ -n "$estab" ]; then printf '  %s association SCTP ETABLIE\n' "$OK"; printf '    %s\n' "$estab"
else printf '  %s aucune association SCTP etablie vers le hub\n' "$KO"; fi

# ---- verdict -----------------------------------------------------------
section "VERDICT"
if [ "$r_state" = "ASP_ACTIVE" ] && [ -n "$estab" ]; then
  printf '  %sINTERCO SS7 : UP%s  (asp-to-inter ACTIVE, SCTP etabli)\n' "$G$B" "$Z"
else
  printf '  %sINTERCO SS7 : DOWN%s\n' "$R$B" "$Z"
  echo   "  causes probables (par ordre) :"
  [ "$f_shut" = yes ] && echo "    - asp-to-inter en 'shutdown' dans la conf"
```

```
  { [ -n "$EXP_PC" ] && [ "$f_pc" != "$EXP_PC" ]; } && echo "    - point-code STP disque ($f_pc) != attendu
   ↪  ($EXP_PC)"
  { [ -n "$HUB_IP" ] && [ "$f_rip" != "$HUB_IP" ]; } && echo "    - remote-ip disque ($f_rip) != hub ($HUB_IP)"
  { [ -n "$r_rem" ] && [ "${r_rem%:*}" != "$f_rip" ]; } && echo "    - demon tourne une conf obsolete -> systemctl
   ↪  restart osmo-stp osmo-msc osmo-bsc"
  [ -z "$estab" ] && echo "    - pas de SCTP etabli (hub down, pare-feu, ou asp pas monte)"
  echo "  (rappel : ce script ne redemarre rien)"
fi
hr
exit 0
```

2.24 /opt/GSM/osmo-operator/checks/global_check.sh

25083 octets, 595 lignes → 595 lignes

```
#!/bin/bash
# global_check.sh - Full health check of every Osmocom component
#
# Checks, for each operator:
#   - STP (4239): ASP, AS, routes
#   - HLR (4258): GSUP connections, subscribers
#   - MSC (4254): ASP, SCCP, subscribers
#   - BSC (4242): ASP, SCCP, BTS, OML/RSL
#   - BTS (4241): transceiver state, LCHAN
#   - PCU (4240): BTS, TBF, MS (if GPRS is enabled)
#   - SGSN (4245): MM context, PDP context (if GPRS)
#   - GGSN (4260): PDP context, APN (if GPRS)
#   - MGW (4243): MGCP endpoints
#   - SIP connector (4255): state (if present)
#   - Services: SMS relay, MNCC socket, Asterisk
#
# DUAL MODE - docker or native
#   docker: each operator is an osmo-operator-N container queried through
#           "docker exec"; the inter-STP is the osmo-inter-stp container.
#   native: ISO, VM or bare machine; a single operator, the daemons run here,
#           VTYs on 127.0.0.1 (same ports), configs in /etc/osmocom.
# The mode is DETECTED (checks/_mode.sh) and printed at the top of the output.
# --docker / --native force it when detection cannot decide (e.g. a host with
# docker installed that runs the lab natively).
#
# Usage:
#   sudo ./checks/global_check.sh [--verbose] [--quick] [--op=N] [--docker|--native]
# Environment: OSMO_MODE=docker|native also forces the mode.

set -u

# ── Colors ──────────────────────────────────────────────────────────────
GREEN='\033[0;32m'
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BOLD='\033[1m'
NC='\033[0m'

# ── Mode library (docker | native) ──────────────────────────────────────
# Looked up RELATIVE to the script: the repo lives at different roots in
# development and on the ISO. The second candidate covers a call through a
# symlink, where dirname does not point to checks/.
_here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
for _c in "$_here/_mode.sh" /opt/GSM/osmo-operator/checks/_mode.sh; do
    [ -r "$_c" ] && { . "$_c"; break; }
done
command -v osmo_mode >/dev/null || { echo "checks/_mode.sh introuvable" >&2; exit 1; }
```

```
# ── Configuration ──────────────────────────────────────────────────────
VERBOSE=0
QUICK=0
SPECIFIC_OP=""
MODE_FORCED=0

# OSMO_MODE set in the environment is already a forced mode
# ("OSMO_MODE=native ./checks/global_check.sh"): report it as such.
case "${OSMO_MODE:-}" in docker|native) MODE_FORCED=1 ;; esac

for arg in "$@"; do
    case "$arg" in
        --verbose|-v) VERBOSE=1 ;;
        --quick|-q)   QUICK=1 ;;
        --op=*)       SPECIFIC_OP="${arg#*=}" ;;
        # osmo_mode_force returns 2 when the argument is not a mode; it always
        # is one here, "|| true" only guards against a future set -e.
        --docker|--native|--mode=*) osmo_mode_force "$arg" && MODE_FORCED=1 || true ;;
    esac
done

MODE="$(osmo_mode)"

PASS=0
FAIL=0
WARN=0
SKIP=0

# ── Helpers ────────────────────────────────────────────────────────────
ok()   { echo -e "  ${GREEN}✓${NC} $*"; PASS=$((PASS+1)); }
fail() { echo -e "  ${RED}✗${NC} $*"; FAIL=$((FAIL+1)); }
warn() { echo -e "  ${YELLOW}⚠${NC} $*"; WARN=$((WARN+1)); }
skip() { echo -e "  ${YELLOW}-${NC} $*"; SKIP=$((SKIP+1)); }
info() { [[ $VERBOSE -eq 1 ]] && echo -e "    ${CYAN}|${NC} $*"; }

banner() {
    echo ""
    echo -e "${BOLD}$*${NC}"
    echo -e "$(printf '=%.0s' {1..70})"
}

section() {
    echo -e "\n  ${CYAN}${BOLD}$*${NC}"
}

# ── VTY function ───────────────────────────────────────────────────────
# First argument: the node LABEL (osmo-operator-N), which osmo_vty maps to
# "docker exec" or local execution depending on the mode. The timings
# (0.5 / 2 / nc -q1) determine the AMOUNT of output captured, hence the
# "grep -c" counts below: do not change them.
vty_cmd() {
    local node="$1"
    local port="$2"
    local cmd="$3"
    local timeout="${4:-2}"

    local raw_output
    raw_output=$(OSMO_VTY_OPEN=0.5 OSMO_VTY_READ="$timeout" OSMO_VTY_Q=1 \
                 osmo_vty "$node" "$port" enable "$cmd" end) || return 1

    # Filter banner/prompt lines; "Free Software lives" is dropped as well.
    printf '%s\n' "$raw_output" | osmo_vty_clean 'Free Software lives'
    return 0
}

# ── Quick VTY availability check ──────────────────────────────────────
```

```
# In native mode osmo_vty_up probes with ss first (passive, opens no VTY
# session) and falls back to a TCP connection only if ss sees nothing.
vty_available() {
    osmo_vty_up "$1" "$2"
}

# ── Operator detection ──────────────────────────────────────────────
# docker: running osmo-operator-N containers.
# native: $OSMO_OP_IDS, else the osmo-opN netns, else N_OPERATORS from
#         globals.conf, else 1 - there is no inventory in native mode.
if [ -n "$SPECIFIC_OP" ]; then
    # osmo_op_exists is ANCHORED: "--op=1" must not match osmo-operator-10.
    if ! osmo_op_exists "$SPECIFIC_OP"; then
        echo -e "${RED}Operateur ${SPECIFIC_OP} non trouve (mode ${MODE})${NC}"
        exit 1
    fi
    OP_IDS=("$SPECIFIC_OP")
else
    mapfile -t OP_IDS < <(osmo_ops)
fi

N_OPS=${#OP_IDS[@]}

# One line of the header box, padded to exactly 58 columns. The text must be
# ASCII: under a C locale ${#t} counts the bytes of an accented character and
# the frame shifts.
box_line() {
    # Two statements, not one: in 'local t="$1" pad=$(( ... ${#t} ))' the
    # arithmetic expands BEFORE t is assigned (unbound variable under set -u).
    local t="$1"
    local pad=$(( 58 - 5 - ${#t} ))
    [ "$pad" -lt 0 ] && pad=0
    printf "${CYAN}║     %s%*s║${NC}\n" "$t" "$pad" ""
}

echo ""
echo -e "${CYAN}╔══════════════════════════════════════════════════════════╗${NC}"
echo -e "${CYAN}║     Global Health Check - $(date '+%Y-%m-%d %H:%M:%S')            ║${NC}"
echo -e "${CYAN}║     ${N_OPS} operateur(s) a verifier                                ║${NC}"
# The mode is part of the diagnostic: without it a "service DOWN" does not say
# whether it was looked for in a container or on this machine.
if [ "$MODE_FORCED" -eq 1 ]; then
    box_line "mode : ${MODE} (force)"
else
    box_line "mode : ${MODE} (detecte)"
fi
if [ "$MODE" = native ]; then
    box_line "natif : demons locaux, VTY sur 127.0.0.1"
else
    box_line "docker : conteneurs osmo-operator-N"
fi
echo -e "${CYAN}╚══════════════════════════════════════════════════════════╝${NC}"

if [ "$N_OPS" -eq 0 ]; then
    # In native mode osmo_ops always returns at least "1": this case is docker
    # with no running container. Say so instead of suggesting a service outage.
    echo -e "${RED}Aucun operateur trouve (mode ${MODE})${NC}"
    [ "$MODE" = docker ] && echo -e "     ${CYAN}aucun conteneur osmo-operator-N en cours - sudo
     ↪  ./launch/start-docker.sh${NC}"
    exit 1
fi

# ═══════════════════════════════════════════════════════════════════
# For each operator
# ═══════════════════════════════════════════════════════════════════
for op_id in "${OP_IDS[@]}"; do
```

```
# osmo_node returns "osmo-operator-N" in BOTH modes: the container name in
# docker, a display label in native. The banner and the "container=" fields
# read back by operator_summary.sh rely on it.
container="$(osmo_node "$op_id")"
banner "OPERATEUR ${op_id} (${container})"

# ============================================================================
# 1. STP (port 4239)
# ============================================================================
section "STP (Signalisation)"

if ! vty_available "$container" 4239; then
    fail "STP VTY (4239) inaccessible - service DOWN"
else
    # local ASPs
    asp_out=$(vty_cmd "$container" 4239 "show cs7 instance 0 asp")
    n_asp=$(echo "$asp_out" | grep -c 'ASP_ACTIVE' || true)

    if [ "$n_asp" -ge 2 ]; then
        ok "ASP actifs : ${n_asp} (MSC+BSC ok)"
    elif [ "$n_asp" -eq 1 ]; then
        warn "ASP actifs : 1 seul (manque MSC ou BSC)"
    else
        fail "ASP actifs : aucun"
    fi

    # local AS
    as_out=$(vty_cmd "$container" 4239 "show cs7 instance 0 as all")
    n_as=$(echo "$as_out" | grep -c 'AS_ACTIVE' || true)

    if [ "$n_as" -ge 2 ]; then
        ok "AS actifs : ${n_as}"
    else
        warn "AS actifs : ${n_as}"
    fi

    # Routes (no PROHIB)
    route_out=$(vty_cmd "$container" 4239 "show cs7 instance 0 route")
    n_prohib=$(echo "$route_out" | grep -ciE 'prohib' || true)

    if [ "$n_prohib" -eq 0 ]; then
        ok "Routes : aucun PROHIB"
    else
        fail "Routes : ${n_prohib} PROHIB detectes"
    fi

    # Default route to the inter-STP.
    # In docker the inter-STP is a container (osmo-inter-stp, 172.20.0.10).
    # In native mode it exists only if THIS machine holds that role, or on
    # another WAN machine: then a missing catch-all route is topology, not
    # an anomaly, and is skipped explicitly.
    has_default=$(echo "$route_out" | grep -cE '0\.0\.0/0.*avail' || true)
    if [ "$has_default" -gt 0 ]; then
        ok "Route par defaut → inter-STP : presente"
    elif osmo_is_native && ! osmo_hub >/dev/null 2>&1; then
        skip "Route par defaut → inter-STP - ignore (natif) : $(osmo_hub_hint)"
    else
        warn "Route par defaut → inter-STP : absente"
    fi
fi

# ============================================================================
# 2. HLR (port 4258)
# ============================================================================
section "HLR (Abonnes)"
```

```
if ! vty_available "$container" 4258; then
    fail "HLR VTY (4258) inaccessible - service DOWN"
else
    # GSUP connections
    gsup_out=$(vty_cmd "$container" 4258 "show gsup-connections")
    vlr_conn=$(echo "$gsup_out" | grep -c "VLR" || true)
    smsc_conn=$(echo "$gsup_out" | grep -c "SMSC" || true)

    [ "$vlr_conn" -gt 0 ] && ok "GSUP : VLR connecte" || fail "GSUP : VLR absent"
    [ "$smsc_conn" -gt 0 ] && ok "GSUP : SMSC connecte" || warn "GSUP : SMSC absent"

    # Subscriber count
    sub_out=$(vty_cmd "$container" 4258 "show subscriber count")
    sub_count=$(echo "$sub_out" | grep -oE '[0-9]+' | head -1)
    info "Abonnes dans HLR : ${sub_count:-0}"
fi

# ============================================================================
# 3. MSC (port 4254)
# ============================================================================
section "MSC (Commutation)"

if ! vty_available "$container" 4254; then
    fail "MSC VTY (4254) inaccessible - service DOWN"
else
    # ASP to STP
    msc_asp=$(vty_cmd "$container" 4254 "show cs7 instance 0 asp")
    msc_active=$(echo "$msc_asp" | grep -c 'ASP_ACTIVE' || true)
    [ "$msc_active" -gt 0 ] && ok "MSC→STP : ASP_ACTIVE" || fail "MSC→STP : pas de ASP actif"

    # SCCP
    msc_sccp=$(vty_cmd "$container" 4254 "show cs7 instance 0 sccp users")
    if echo "$msc_sccp" | grep -qE 'SSN 254'; then
        ok "SCCP SSN 254 (MSC-A) enregistre"
    else
        fail "SCCP SSN 254 absent"
    fi

    # VLR subscribers
    vlr_out=$(vty_cmd "$container" 4254 "show subscriber count")
    vlr_count=$(echo "$vlr_out" | grep -oE '[0-9]+' | head -1 || echo "0")
    info "Subscribers dans VLR : ${vlr_count}"
fi

# ============================================================================
# 4. BSC (port 4242)
# ============================================================================
section "BSC (Controleur BTS)"

if ! vty_available "$container" 4242; then
    fail "BSC VTY (4242) inaccessible - service DOWN"
else
    # ASP to STP
    bsc_asp=$(vty_cmd "$container" 4242 "show cs7 instance 0 asp")
    bsc_active=$(echo "$bsc_asp" | grep -c 'ASP_ACTIVE' || true)
    [ "$bsc_active" -gt 0 ] && ok "BSC→STP : ASP_ACTIVE" || fail "BSC→STP : pas de ASP actif"

    # SCCP
    bsc_sccp=$(vty_cmd "$container" 4242 "show cs7 instance 0 sccp users")
    echo "$bsc_sccp" | grep -qE 'SSN 254' && ok "SCCP SSN 254 (BSC) enregistre" || fail "SCCP SSN 254 absent"

    # BTS 0 state
    bts_out=$(vty_cmd "$container" 4242 "show bts 0")

    if echo "$bts_out" | grep -qE "Oper 'Enabled'.*Avail 'OK'"; then
        ok "BTS 0 : Enabled / OK"
```

```
    elif echo "$bts_out" | grep -qE "Oper 'Enabled'"; then
        warn "BTS 0 : Enabled mais Avail != OK"
    else
        fail "BTS 0 : pas Enabled"
    fi

    # OML in show bts 0
    oml_up=$(echo "$bts_out" | grep -c 'OML Link state: connected' || true)
    [ "$oml_up" -gt 0 ] && ok "OML : connecte" || fail "OML : deconnecte"

    # RSL: check in show trx 0 0 (not in show bts 0)
    trx_out=$(vty_cmd "$container" 4242 "show trx 0 0")
    rsl_up=$(echo "$trx_out" | grep -c 'RSL State: connected' || true)

    if [ "$rsl_up" -gt 0 ]; then
        ok "RSL : connecte (TRX 0)"
    else
        # Fallback: check whether it is mentioned elsewhere
        rsl_alt=$(echo "$bts_out" | grep -c 'RSL' || true)
        if [ "$rsl_alt" -gt 0 ]; then
            warn "RSL : statut ambigu (verifier manuellement)"
        else
            fail "RSL : deconnecte"
        fi
    fi

    # Number of BTS
    n_bts=$(vty_cmd "$container" 4242 "show bts all" | grep -c '^bts' || true)
    info "BTS configurees : ${n_bts}"
fi

# ============================================================================
# 5. BTS (base station)
# ============================================================================
section "BTS (Station de base)"

if ! vty_available "$container" 4242; then
    skip "BSC VTY (4242) inaccessible - impossible de verifier les BTS"
else
    # Check BTS 0 directly
    bts0_out=$(vty_cmd "$container" 4242 "show bts 0")

    if [ -z "$bts0_out" ]; then
        fail "Impossible d'obtenir les informations de BTS 0"
    else
        # Check that the BTS exists (name present)
        if echo "$bts0_out" | grep -q "BTS 0 is of osmo-bts type"; then
            ok "BTS 0 : configuree"

            # Operational state
            if echo "$bts0_out" | grep -q "Oper 'Enabled'.*Avail 'OK'"; then
                ok "BTS 0 : operationnelle (Enabled/OK)"
            elif echo "$bts0_out" | grep -q "Oper 'Enabled'"; then
                warn "BTS 0 : Enabled mais Avail != OK"
            else
                fail "BTS 0 : pas Enabled"
            fi

            # OML
            if echo "$bts0_out" | grep -q "OML Link state: connected"; then
                ok "OML : connecte"
            else
                fail "OML : deconnecte"
            fi

            # RSL (present in the statistics at the bottom)
```

```
            # Known limitation: BASH_REMATCH is never set by grep, so the
            # second branch prints an empty count (unbound under set -u).
            if echo "$bts0_out" | grep -q "Number of RSL links connected.* 1"; then
                ok "RSL : connecte"
            elif echo "$bts0_out" | grep -q "Number of RSL links connected.* [1-9]"; then
                ok "RSL : connecte (${BASH_REMATCH[1]})"
            else
                # Check via TRX
                trx0_out=$(vty_cmd "$container" 4242 "show trx 0 0" 2>/dev/null)
                if echo "$trx0_out" | grep -q "RSL State: connected"; then
                    ok "RSL : connecte (TRX 0)"
                else
                    fail "RSL : deconnecte"
                fi
            fi

            # ARFCN
            arfcn=$(echo "$bts0_out" | grep -oE 'ARFCNs: [0-9]+' | head -1)
            [ -n "$arfcn" ] && info "$arfcn"

            # Uptime
            uptime=$(echo "$bts0_out" | grep "Seconds of uptime" | grep -oE '[0-9]+ s')
            [ -n "$uptime" ] && info "Uptime : ${uptime}"

            # Check for other BTS
            n_bts=$(vty_cmd "$container" 4242 "show bts all" | grep -c '^bts' || true)
            if [ "$n_bts" -gt 1 ]; then
                ok "BTS multiples : ${n_bts} (BTS 0 OK)"
            fi
        else
            fail "BTS 0 non trouvee dans la configuration"
        fi
    fi
fi
# ============================================================================
# 6. PCU (port 4240) - GPRS
# ============================================================================
section "PCU (Controleur paquets)"

if ! vty_available "$container" 4240; then
    skip "PCU VTY (4240) - non disponible (GPRS peut-etre desactive)"
else
    # BTS
    pcu_bts=$(vty_cmd "$container" 4240 "show bts all")
    if [ -n "$pcu_bts" ]; then
        ok "PCU : connecte aux BTS"
    else
        warn "PCU : aucune BTS"
    fi

    # TBF
    tbf_out=$(vty_cmd "$container" 4240 "show tbf all")
    tbf_count=$(echo "$tbf_out" | grep -c 'TBF' || true)
    info "TBF actifs : ${tbf_count}"
fi

# ============================================================================
# 7. SGSN (port 4245) - Core paquets
# ============================================================================
section "SGSN (Serving GPRS)"

if ! vty_available "$container" 4245; then
    skip "SGSN VTY (4245) - non disponible (GPRS peut-etre desactive)"
else
    # MM context
    mm_out=$(vty_cmd "$container" 4245 "show mm-context all")
```

```
    mm_count=$(echo "$mm_out" | grep -c 'IMSI' || true)
    info "MM contexts : ${mm_count}"

    # PDP context
    pdp_out=$(vty_cmd "$container" 4245 "show pdp-context all")
    pdp_count=$(echo "$pdp_out" | grep -c 'PDP' || true)
    info "PDP contexts : ${pdp_count}"

    # NS entities
    ns_out=$(vty_cmd "$container" 4245 "show ns entities")
    ns_count=$(echo "$ns_out" | grep -c 'NSVC' || true)
    [ "$ns_count" -gt 0 ] && ok "NS entities : ${ns_count}" || warn "NS entities : 0"
fi

# ============================================================================
# 8. GGSN (port 4260) - Gateway GPRS
# ============================================================================
section "GGSN (Gateway GPRS)"

if ! vty_available "$container" 4260; then
    skip "GGSN VTY (4260) - non disponible (GPRS peut-etre desactive)"
else
    # PDP contexts
    ggsn_pdp=$(vty_cmd "$container" 4260 "show pdp-context all")
    ggsn_count=$(echo "$ggsn_pdp" | grep -c 'PDP' || true)
    info "PDP contexts GGSN : ${ggsn_count}"

    # APN
    apn_out=$(vty_cmd "$container" 4260 "show apn all")
    apn_count=$(echo "$apn_out" | grep -c 'APN' || true)
    [ "$apn_count" -gt 0 ] && ok "APN configures : ${apn_count}" || warn "APN configures : 0"
fi

# ============================================================================
# 9. MGW (port 4243) - Media Gateway
# ============================================================================
section "MGW (Media Gateway)"

if ! vty_available "$container" 4243; then
    fail "MGW VTY (4243) inaccessible"
else
    # Endpoints
    ep_out=$(vty_cmd "$container" 4243 "show mgcp endpoint all")
    ep_count=$(echo "$ep_out" | grep -c 'Endpoint' || true)
    info "Endpoints MGCP : ${ep_count}"

    # Active connections
    conn_out=$(vty_cmd "$container" 4243 "show mgcp stats")
    if echo "$conn_out" | grep -qE "active calls.*[1-9]"; then
        ok "MGW : appels actifs"
    else
        info "MGW : aucun appel actif"
    fi
fi

# ============================================================================
# 10. SIP connector (port 4255)
# ============================================================================
section "SIP connector (Voix)"

if ! vty_available "$container" 4255; then
    skip "SIP connector VTY (4255) - non disponible"
else
    ok "SIP connector : accessible"
    # osmo-sip-connector has few useful show commands
fi
```

```
    # ============================================================================
    # 11. Services
    # ============================================================================
    section "Services"

    # MNCC socket - osmo_sock queries ss BEFORE "test -S": under PrivateTmp
    # (ISO systemd unit) the socket exists without being visible to test -S.
    if osmo_sock "$container" /tmp/msc_mncc; then
        ok "MNCC socket present (MSC ↔ SIP connector)"
    else
        fail "MNCC socket absent - voix impossible"
    fi

    # SMS relay - awk on the exact port, not "grep -c :7890" which would also
    # match 17890.
    if osmo_port "$container" 7890 tcp; then
        ok "SMS relay : ecoute sur port 7890"
    else
        warn "SMS relay : non detecte"
    fi

    # Asterisk
    if osmo_running "$container" asterisk; then
        ok "Asterisk : en cours d'execution"

        # Quick Asterisk check
        ast_out=$(osmo_ast "$container" "core show channels" || true)
        channels=$(echo "$ast_out" | grep -c "active channels" || true)
        info "Asterisk : ${channels} canaux actifs"
    else
        warn "Asterisk : non detecte"
    fi

    # Key process check
    section "Processus"

    # In native mode osmo_running tries systemctl, THEN pgrep -x, THEN pgrep -f:
    # daemons are units on the ISO but detached processes without systemd, and
    # /proc/PID/comm is truncated to 15 characters ("pgrep -x
    # osmo-sip-connector" never matches).
    if [ "$MODE" = native ] && [ "$N_OPS" -gt 1 ]; then
        info "natif multi-operateur : ip netns exec n'isole pas les PID, ces etats portent sur la machine"
    fi

    for proc in osmo-stp osmo-hlr osmo-msc osmo-bsc osmo-bts-trx osmo-mgw osmo-sip-connector; do
        if osmo_running "$container" "$proc"; then
            ok "$proc : en cours"
        else
            # Some processes may be absent depending on the configuration
            skip "$proc : non trouve"
        fi
    done

    echo ""
done

# ============================================================================
# GLOBAL SUMMARY
# ============================================================================
echo ""
echo -e "$(printf '=%.0s' {1..70})"
echo -e "  ${BOLD}RESUME GLOBAL${NC}"
echo -e "$(printf '=%.0s' {1..70})"

TOTAL=$((PASS + FAIL + WARN + SKIP))
```

```
echo -e "  ${CYAN}mode   : ${MODE}${NC}  (${N_OPS} operateur(s))"
echo -e "  ${GREEN}✓ PASS : ${PASS}${NC}"
echo -e "  ${RED}✗ FAIL : ${FAIL}${NC}"
echo -e "  ${YELLOW}⚠ WARN : ${WARN}${NC}"
echo -e "  ${YELLOW}- SKIP : ${SKIP}${NC}"
echo -e "$(printf '─%.0s' {1..70})"

if [ "$FAIL" -eq 0 ] && [ "$WARN" -eq 0 ]; then
    echo -e "  ${GREEN}${BOLD}TOUT EST OK${NC}  -  Tous les composants fonctionnent"
elif [ "$FAIL" -eq 0 ]; then
    echo -e "  ${YELLOW}${BOLD}FONCTIONNEL AVEC AVERTISSEMENTS${NC}  -  Verifier les warnings"
else
    echo -e "  ${RED}${BOLD}PROBLEMES DETECTES${NC}  -  ${FAIL} test(s) en echec"
fi
echo -e "$(printf '═%.0s' {1..70})"
echo ""

exit "$FAIL"
```

2.25 /opt/GSM/osmo-operator/checks/operator_summary.sh

20578 octets, 473 lignes → 473 lignes

```
#!/bin/bash
# operator_summary.sh - Summary of each operator's main characteristics
#
# Parses the dump produced by checks/vty-debug-dump.sh and extracts, operator
# by operator, the few figures that tell whether the network is alive: active
# ASPs, SS7 routes, HLR/VLR subscribers, BTS state, PDP contexts, ongoing calls.
#
# DUAL MODE (docker / native)
# This script talks to NO daemon: it reads a file. The mode only changes:
#   1. dump GENERATION (--live): the generator must look at the SAME world,
#      so the mode is passed to it through OSMO_MODE, even when forced here.
#   2. the INTER-STP: in docker it is the osmo-inter-stp container; in native
#      mode it is either this machine (role interstp) or ANOTHER machine whose
#      VTY is not reachable from here, so the dump contains nothing about it.
#      The section is then explicitly SKIPPED: a fabricated "0 active AS"
#      would read as an SS7 core outage.
#   3. the "container=osmo-operator-N" label of the dump: identical in native
#      mode (checks/_mode.sh, osmo_node). It is a CORRELATION KEY between the
#      dump blocks, not a queryable docker object: every extraction below
#      depends on it.
# The selected mode is printed in the header.
#
# Usage:
#   sudo ./checks/operator_summary.sh [--dump=/path/dump.txt] [--live]
#                                     [--docker|--native]
# Environment: OSMO_MODE=docker|native forces the mode.

set -euo pipefail

# ── Common docker/native detection library ──────────────────────────────
# Looked up relative to THIS file (never absolute): the repo lives at
# different roots in development and on the ISO.
_here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
for _c in "$_here/_mode.sh" /opt/GSM/osmo-operator/checks/_mode.sh; do
    [ -r "$_c" ] && { . "$_c"; break; }
done
command -v osmo_mode >/dev/null || { echo "checks/_mode.sh introuvable" >&2; exit 1; }

# ── Colors ──────────────────────────────────────────────────────────────
GREEN='\033[0;32m'
```

```
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BOLD='\033[1m'
NC='\033[0m'

# ── Configuration ──────────────────────────────────────────────────────
DUMP_FILE=""
LIVE_MODE=0
TEMP_DUMP=""

# OSMO_MODE inherited from the environment is an explicit choice, like
# --docker/--native: reported as "force" in the header.
MODE_FORCED=0
[ -n "${OSMO_MODE:-}" ] && MODE_FORCED=1

for arg in "$@"; do
    case "$arg" in
        --dump=*) DUMP_FILE="${arg#*=}" ;;
        --live)   LIVE_MODE=1 ;;
        --docker|--native|--mode=*)
            # osmo_mode_force returns 2 when the argument is not a mode: only
            # "--mode=<anything>" can still fail here.
            if osmo_mode_force "$arg"; then
                MODE_FORCED=1
            else
                echo "Mode inconnu : ${arg} (attendu : --docker, --native)" >&2
                exit 1
            fi
            ;;
        --help|-h)
            echo "Usage: $0 [--dump=/chemin/dump.txt] [--live] [--docker|--native]"
            echo "  --dump   : Utilise un fichier de dump existant"
            echo "  --live   : Genere un dump live avant l'analyse"
            echo "  --docker : Force le mode conteneurs (osmo-operator-N)"
            echo "  --native : Force le mode natif (ISO, VM, machine nue)"
            echo "             sans option, le mode est detecte automatiquement"
            exit 0
            ;;
    esac
done

MODE="$(osmo_mode)"
if [ "$MODE_FORCED" -eq 1 ]; then MODE_SRC="force"; else MODE_SRC="detecte"; fi

# ── Display helpers ────────────────────────────────────────────────────
print_header() {
    echo ""
    echo -e "${BOLD}$1${NC}"
    echo -e "$(printf '─%.0s' {1..70})"
}

print_row() {
    printf "  %-20s : %s\n" "$1" "$2"
}

print_section() {
    echo -e "\n  ${CYAN}${BOLD}$1${NC}"
}

# One line of the header box, aligned on the 58 columns of the ╔═╗ frame.
box_line() {
    local txt="$1" pad
    pad=$(( 58 - 5 - ${#txt} ))
    [ "$pad" -ge 0 ] || pad=0
    echo -e "${CYAN}║     ${txt}$(printf '%*s' "$pad" '')║${NC}"
```

```
}

# ── Data extraction from the dump ──────────────────────────────────────
#
# PIPEFAIL, for EVERY extraction in this file: under "set -o pipefail" a grep
# that finds nothing (rc 1) or a "head -1" that exits early (producer killed
# by SIGPIPE, rc 141) fails the whole pipeline, hence the assignment and,
# under "set -e", the script itself (empty or partial dump). The "|| true"
# hide no error: they mean "nothing found = zero", which wc and grep -c
# already printed.

extract_value() {
    local container="$1"
    local pattern="$2"
    local default="${3:-?}"

    grep -A20 "\[.*\] port=.* container=${container}" "$DUMP_FILE" \
        | grep -E "$pattern" \
        | head -1 \
        | sed -E 's/^[[:space:]]+//' \
        || echo "$default"
}

extract_count() {
    local container="$1"
    local section="$2"
    local pattern="$3"

    grep -A50 "\[.*\] port=.* container=${container}.*$section" "$DUMP_FILE" \
        | grep -E "$pattern" \
        | wc -l \
        | tr -d ' ' \
        || true
}

extract_subscriber_count() {
    local container="$1"
    local count

    count=$(grep -A20 "\[OsmoHLR\]" "$DUMP_FILE" \
        | grep -A5 "container=${container}" \
        | grep -E "subscriber count.*[0-9]+" \
        | grep -oE '[0-9]+' \
        | head -1 || true)

    echo "${count:-0}"
}

# Sets BTS_STATE, OML_OK and RSL_OK as GLOBAL variables, on purpose: it must
# be called without command substitution, otherwise the assignments would die
# with the $( ) sub-shell and OML/RSL would always show as disconnected.
extract_bts_info() {
    local container="$1"
    local bts_id="$2"
    local bts_block oper avail oml rsl

    bts_block=$(grep -A50 "\[OsmoBSC\]" "$DUMP_FILE" \
        | grep -A30 "container=${container}" \
        | grep -A20 "bts ${bts_id}" \
        | sed -n '/bts [0-9]/,/^$/p' || true)

    oper=$(echo "$bts_block" | grep -c "Oper 'Enabled'" || true)
    avail=$(echo "$bts_block" | grep -c "Avail 'OK'" || true)
    oml=$(echo "$bts_block" | grep -c "OML Link state: connected" || true)
    rsl=$(echo "$bts_block" | grep -c "RSL State: connected" || true)
```

```
    if [ "$oper" -gt 0 ] && [ "$avail" -gt 0 ]; then
        BTS_STATE="OK"
    elif [ "$oper" -gt 0 ]; then
        BTS_STATE="Enabled"
    else
        BTS_STATE="Down"
    fi

    # Also set OML/RSL for information
    [ "$oml" -gt 0 ] && OML_OK=1 || OML_OK=0
    [ "$rsl" -gt 0 ] && RSL_OK=1 || RSL_OK=0
}

extract_asp_state() {
    local container="$1"
    local component="$2"  # STP, MSC, BSC

    local asp_block=""
    if [ "$component" = "STP" ]; then
        asp_block=$(grep -A30 "\[OsmoSTP\]" "$DUMP_FILE" \
            | grep -A20 "container=${container}" || true)
    elif [ "$component" = "MSC" ]; then
        asp_block=$(grep -A30 "\[OsmoMSC\]" "$DUMP_FILE" \
            | grep -A20 "container=${container}" || true)
    elif [ "$component" = "BSC" ]; then
        asp_block=$(grep -A30 "\[OsmoBSC\]" "$DUMP_FILE" \
            | grep -A20 "container=${container}" || true)
    fi

    echo "$asp_block" | grep -c "ASP_ACTIVE" || true
}

# ── Live dump generation on request ──────────────────────────────────
if [ "$LIVE_MODE" -eq 1 ]; then
    echo -e "${CYAN}Generation d'un dump live (mode ${MODE})...${NC}"
    TEMP_DUMP="/tmp/vty-live-$$.txt"

    # The generator is looked up NEXT TO THIS FILE (BASH_SOURCE), not in the
    # current directory.
    DUMPER="$_here/vty-debug-dump.sh"
    [ -x "$DUMPER" ] || DUMPER="./vty-debug-dump.sh"

    # OSMO_MODE is passed explicitly: the generator must query the same world
    # as the one analysed here, even when --native/--docker forced a mode it
    # would not have detected by itself.
    if ! OSMO_MODE="$MODE" "$DUMPER" --out "$TEMP_DUMP" > /dev/null; then
        echo -e "${RED}Echec de la generation du dump (${DUMPER}, mode ${MODE}).${NC}" >&2
        if [ "$MODE" = native ]; then
            echo "  Mode natif : verifiez que les demons tournent (VTY 4239/4254/4242/4258...)." >&2
        else
            echo "  Mode docker : verifiez que les conteneurs osmo-operator-N sont en cours." >&2
        fi
        rm -f "$TEMP_DUMP"
        exit 1
    fi
    DUMP_FILE="$TEMP_DUMP"
    echo -e "${GREEN}✓ Dump genere : $DUMP_FILE${NC}"
fi

if [ -z "$DUMP_FILE" ] || [ ! -f "$DUMP_FILE" ]; then
    echo -e "${RED}Erreur : fichier de dump non specifie ou introuvable${NC}"
    echo "Utilisez --dump=/chemin/vers/dump.txt ou --live pour generer un dump"
    exit 1
fi

# ── Nodes present IN THE DUMP ────────────────────────────────────────
```

```
# "container=osmo-operator-N" is written by vty-debug-dump.sh in both modes:
# the container name in docker, the label returned by osmo_node() in native.
# The dump is therefore read the same way on both sides.
mapfile -t CONTAINERS < <(
    grep -E "container=osmo-operator-[0-9]+" "$DUMP_FILE" \
        | sed -E 's/.*container=([^ ]+).*/\1/' \
        | sort -u
)

INTER_STP=$(grep -E "container=osmo-inter-stp" "$DUMP_FILE" \
    | sed -E 's/.*container=([^ ]+).*/\1/' \
    | head -1 || true)

N_OPS=${#CONTAINERS[@]}

# ── Header ──────────────────────────────────────────────────────────────
echo ""
echo -e "${CYAN}╔═══════════════════════════════════════════════════════════════════╗${NC}"
echo -e "${CYAN}║     Resume Operateurs - $(date '+%Y-%m-%d %H:%M')              ║${NC}"
echo -e "${CYAN}║     ${N_OPS} operateur(s) trouve(s)                               ║${NC}"
box_line "Mode : ${MODE} (${MODE_SRC})"
echo -e "${CYAN}╚═══════════════════════════════════════════════════════════════════╝${NC}"

# Silent dump: say so. Comparing it with the live inventory tells "nothing is
# running" from "the dump comes from elsewhere / is partial" - two very
# different failures, especially in native mode where no "docker ps" can
# decide.
if [ "$N_OPS" -eq 0 ]; then
    live_ops="$(osmo_ops 2>/dev/null | tr '\n' ' ' || true)"
    live_ops="${live_ops% }"
    echo ""
    echo -e "  ${YELLOW}Aucun bloc "container=osmo-operator-N" dans ce dump.${NC}"
    print_row "Operateurs vus ici" "${live_ops:-aucun} (mode ${MODE})"
    print_row "Dump" "vide, partiel, ou produit sur un autre noeud"
fi

# ── Inter-STP ───────────────────────────────────────────────────────────
if [ -n "$INTER_STP" ]; then
    print_header "INTER-STP HUB (${INTER_STP})"

    asp_count=$(extract_asp_state "$INTER_STP" "STP")
    print_row "ASP actifs" "$asp_count"

    # Routes
    routes=$(grep -A50 "\[OsmoSTP-hub\]" "$DUMP_FILE" \
        | grep -A30 "container=${INTER_STP}" \
        | grep -E '^[0-9]+\.[0-9]+\.[0-9]+/[0-9]+' \
        | wc -l || true)
    print_row "Routes configurees" "$routes"

    # AS
    as_count=$(grep -A50 "\[OsmoSTP-hub\]" "$DUMP_FILE" \
        | grep -A30 "container=${INTER_STP}" \
        | grep -c "AS_ACTIVE" || true)
    print_row "AS actifs" "$as_count"

elif osmo_is_native; then
    # Nothing about the hub in the dump, and we are native: either this
    # machine IS the hub and the dump predates it, or the hub is ANOTHER
    # machine whose VTY 4239 listens only on its own 127.0.0.1 and is NOT
    # observable from here. Skipped visibly rather than showing zero counters
    # that would read as a dead SS7 core.
    print_header "INTER-STP HUB - ignore (natif)"
    if hub_here="$(osmo_hub)"; then
        print_row "Hub local" "${hub_here} present ici, mais absent du dump"
        print_row "Pour l'inclure" "relancez avec --live"
```

```
    else
        print_row "Hub" "$(osmo_hub_hint)"
    fi

elif hub_remote="$(osmo_hub_is_remote 2>/dev/null)"; then
    # WAN: the hub lives on ANOTHER machine (--hub-ip / OSMO_HUB_IP), so it
    # cannot appear in a dump taken here, and its VTY 4239 listens only on its
    # own loopback. What is observable from here is the LINK to it: the SCTP
    # association each node keeps open.
    print_header "INTER-STP HUB - distant (${hub_remote})"
    hub_assoc="$(osmo_hub_assoc "$hub_remote" 2>/dev/null)" && hub_rc=0 || hub_rc=$?
    case "$hub_rc" in
        0) print_row "Lien M3UA/SCTP" "${hub_assoc} noeud(s) associe(s) a ${hub_remote}:${OSMO_HUB_M3UA_PORT}" ;;
        1) print_row "Lien M3UA/SCTP" "aucune association vers ${hub_remote}:${OSMO_HUB_M3UA_PORT}" ;;
        *) print_row "Lien M3UA/SCTP" "non sondable ici (ss --sctp absent des noeuds)" ;;
    esac
    print_row "Etat interne" "$(osmo_hub_hint)"

elif hub_here="$(osmo_hub)"; then
    # Docker mode: the container runs but is not in the dump, so the dump
    # predates the hub. Without a hub container nothing is printed: there is
    # simply no inter-STP in this lab.
    print_header "INTER-STP HUB (${hub_here})"
    print_row "Etat" "conteneur en cours, mais absent du dump"
    print_row "Pour l'inclure" "relancez avec --live"
fi

# ── Loop over operators ──────────────────────────────────────────────
# "${CONTAINERS[@]+...}": on a silent dump the array is empty, and under
# "set -u" a bare "${CONTAINERS[@]}" would abort instead of doing nothing.
for container in ${CONTAINERS[@]+"${CONTAINERS[@]}"}; do
    # The number is the label suffix; mapfile only let "osmo-operator-<digits>"
    # through, so no sub-shell or grep is needed here.
    op_id="${container##*-}"

    print_header "OPERATEUR ${op_id} (${container})"

    # ── Local STP ────────────────────────────────────────────────────
    print_section "SS7"

    # Active ASPs in STP
    stp_asp=$(extract_asp_state "$container" "STP")
    print_row "ASP actifs (STP)" "$stp_asp"

    # Routes
    routes=$(grep -A50 "\[OsmoSTP\]" "$DUMP_FILE" \
        | grep -A30 "container=${container}" \
        | grep -E '^[0-9]+\.[0-9]+\.[0-9]+/[0-9]+' \
        | wc -l || true)
    print_row "Routes configurees" "$routes"

    # ── MSC ──────────────────────────────────────────────────────────
    print_section "MSC"

    msc_asp=$(extract_asp_state "$container" "MSC")
    print_row "ASP actif" "$([ "$msc_asp" -gt 0 ] && echo "✓" || echo "✗")"

    # VLR subscribers
    vlr_count=$(grep -A30 "\[OsmoMSC\]" "$DUMP_FILE" \
        | grep -A20 "container=${container}" \
        | grep -E "subscriber count.*[0-9]+" \
        | grep -oE '[0-9]+' \
        | head -1 || true)
    print_row "Subscribers VLR" "${vlr_count:-0}"

    # Transactions
```

```
trans=$(grep -A30 "\[OsmoMSC\]" "$DUMP_FILE" \
    | grep -A10 "container=${container}" \
    | grep -c "transaction" || true)
print_row "Transactions" "$trans"

# ── BSC ──────────────────────────────────────────────────────────
print_section "BSC"

bsc_asp=$(extract_asp_state "$container" "BSC")
print_row "ASP actif" "$([ "$bsc_asp" -gt 0 ] && echo "✓" || echo "✗")"

# BTS 0 state
OML_OK=0
RSL_OK=0
BTS_STATE="Down"
extract_bts_info "$container" 0
print_row "BTS 0 etat" "$BTS_STATE"
print_row "  ├─ OML" "$([ "$OML_OK" -eq 1 ] && echo "✓ connecte" || echo "✗ deconnecte")"
print_row "  └─ RSL" "$([ "$RSL_OK" -eq 1 ] && echo "✓ connecte" || echo "✗ deconnecte")"

# Total number of BTS
n_bts=$(grep -A50 "\[OsmoBSC\]" "$DUMP_FILE" \
    | grep -A30 "container=${container}" \
    | grep -c "^ bts [0-9]" || true)
print_row "BTS totales" "$n_bts"

# ── HLR ──────────────────────────────────────────────────────────
print_section "HLR"

subs_count=$(extract_subscriber_count "$container")
print_row "Abonnes" "$subs_count"

# GSUP connections
gsup=$(grep -A20 "\[OsmoHLR\]" "$DUMP_FILE" \
    | grep -A10 "container=${container}" \
    | grep -c "GSUP" || true)
print_row "Connexions GSUP" "$gsup"

# ── BTS (if available) ──────────────────────────────────────────
print_section "BTS (phy)"

# Active LCHANs
lchan=$(grep -A30 "\[OsmoBTS\]" "$DUMP_FILE" \
    | grep -A10 "container=${container}" \
    | grep -c "lchan" || true)
print_row "LCHAN" "$lchan"

# ── SGSN/GGSN (if GPRS is enabled) ──────────────────────────────
print_section "GPRS"

# NS entities
ns=$(grep -A30 "\[OsmoSGSN\]" "$DUMP_FILE" \
    | grep -A10 "container=${container}" \
    | grep -c "NS entity" || true)
print_row "NS entities" "$ns"

# PDP contexts
pdp=$(grep -A30 "\[OsmoGGSN\]" "$DUMP_FILE" \
    | grep -A10 "container=${container}" \
    | grep -c "PDP context" || true)
print_row "PDP contexts" "$pdp"

# ── Auxiliary services ──────────────────────────────────────────
print_section "Services"

# SIP connector
```

```
    sip_logs=$(grep -A10 "\[OsmoSIPconn\]" "$DUMP_FILE" \
        | grep -c "container=${container}" || true)
    print_row "SIP connector" "$([ "$sip_logs" -gt 0 ] && echo "✓ actif" || echo "?")"

    # SMS relay port
    sms=$(grep -A10 "\[.*\]" "$DUMP_FILE" \
        | grep -A5 "container=${container}" \
        | grep -c "port 7890" || true)
    print_row "SMS relay" "$([ "$sms" -gt 0 ] && echo "✓ actif" || echo "?")"

    # ── Quick statistics ──────────────────────────────────────────
    print_section "Statistiques"

    # Active calls
    calls=$(grep -A20 "\[OsmoMSC\]" "$DUMP_FILE" \
        | grep -A10 "container=${container}" \
        | grep -c "call-leg" || true)
    print_row "Appels actifs" "$calls"

    # Paging
    paging=$(grep -A20 "\[OsmoBSC\]" "$DUMP_FILE" \
        | grep -A10 "container=${container}" \
        | grep -c "paging" || true)
    print_row "Paging actifs" "$paging"

done

# ── Cleanup ──────────────────────────────────────────────────────
if [ "$LIVE_MODE" -eq 1 ] && [ -n "$TEMP_DUMP" ]; then
    rm -f "$TEMP_DUMP"
fi

echo ""
echo -e "${GREEN}╔══════════════════════════════════════════════════════════════╗${NC}"
echo -e "${GREEN}║  Analyse terminee - Dump utilise : ${DUMP_FILE}${NC}"
echo -e "${GREEN}╚══════════════════════════════════════════════════════════════╝${NC}"
echo ""
```

2.26 /opt/GSM/osmo-operator/checks/ss7_check.sh

22990 octets, 485 lignes → 485 lignes

```
#!/bin/bash
# ss7_check.sh - SS7 check for the central inter-STP architecture
#
# Target architecture:
#   - a central inter-STP (hub)
#   - each operator has a local STP with a default route to the inter-STP
#   - dynamic routes learned through M3UA
#
# What is checked:
#   1. Inter-STP: active AS for each operator
#   2. Inter-STP: connected ASPs
#   3. Inter-STP: no PROHIB
#   4. Each operator: active ASP towards the inter-STP
#   5. Each operator: local routes (MSC/BSC) active
#   6. Each operator: default route (0.0.0/0) towards the inter-STP
#   7. Basic connectivity matrix
#
# DUAL MODE - docker AND native (ISO, VM, bare machine)
# All node detection and access logic lives in checks/_mode.sh; this script
# only calls its functions. The two worlds:
#   docker: one osmo-operator-N container per operator, hub = osmo-inter-stp
#           container; everything is reached through "docker exec".
```

```
#   native: a single operator (unless netns topology), VTYs on 127.0.0.1 of
#           THIS machine. The hub is EITHER this machine (OSMO_ROLE=interstp
#           in /etc/osmo-role) OR another WAN machine; in the latter case its
#           VTY is NOT observable from here (Osmocom VTYs listen on loopback
#           only). It is then reported as skipped rather than as a fabricated
#           "0 active AS" that would read as an outage.
#
# VTY PORTS are the same in both worlds (4239 STP, 4254 MSC, 4242 BSC,
# 4258 HLR): only the host changes, and _mode.sh takes care of it.
#
# Usage:
#   sudo ./checks/ss7_check.sh [--verbose] [--quick] [--docker|--native]
# Environment: OSMO_VTY_OPEN / OSMO_VTY_READ / OSMO_VTY_Q override the VTY
# timings (defaults 1 / 1.5 / 1).

set -u

# ── Common library (docker/native detection, VTY, inventory) ──────────
# Looked up RELATIVE to the script: the repo runs from a clone as well as from
# /opt/GSM/osmo-operator on the ISO, and checks/ is copied as a whole, so the
# library is always next to this file.
_here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
for _c in "$_here/_mode.sh" /opt/GSM/osmo-operator/checks/_mode.sh; do
    [ -r "$_c" ] && { . "$_c"; break; }
done
command -v osmo_mode >/dev/null || { echo "checks/_mode.sh introuvable" >&2; exit 1; }

# ── Colors ─────────────────────────────────────────────────────────────
GREEN='\033[0;32m'
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BOLD='\033[1m'
NC='\033[0m'

# ── Configuration ──────────────────────────────────────────────────────
VERBOSE=0
QUICK=0

for arg in "$@"; do
    case "$arg" in
        --verbose|-v) VERBOSE=1 ;;
        --quick|-q)   QUICK=1 ;;
        # --docker / --native / --mode=X: force the world instead of detecting
        # it. osmo_mode_force returns 2 when the argument is not a mode, which
        # cannot happen here since the case pattern already filtered it.
        --docker|--native|--mode=*)
            osmo_mode_force "$arg" || { echo "mode inconnu : $arg" >&2; exit 1; } ;;
        -h|--help)
            echo "Usage : $0 [--verbose|-v] [--quick|-q] [--docker|--native]"
            echo "  --docker / --native : force le mode ; sans option, detection automatique."
            exit 0 ;;
    esac
done

INTER_STP="osmo-inter-stp"
PASS=0
FAIL=0
WARN=0
SKIP=0

# ── Helpers ────────────────────────────────────────────────────────────
ok()   { echo -e "  ${GREEN}✓${NC} $*"; PASS=$((PASS+1)); }
fail() { echo -e "  ${RED}✗${NC} $*"; FAIL=$((FAIL+1)); }
warn() { echo -e "  ${YELLOW}⚠${NC} $*"; WARN=$((WARN+1)); }
skip() { echo -e "  ${YELLOW}-${NC} $*"; SKIP=$((SKIP+1)); }
```

```
info() { [[ $VERBOSE -eq 1 ]] && echo -e "    ${CYAN}|${NC} $*"; }

banner() {
    echo ""
    echo -e "${BOLD}$*${NC}"
    echo -e "$(printf '—%.0s' {1..60})"
}

# ── VTY function ──────────────────────────────────────────────────────
# Returns 1 without output when the VTY does not answer, otherwise the cleaned
# output. The dialogue itself (nc/telnet, docker exec or local, netns) is in
# osmo_vty; the timings below are this script's own - changing them changes
# the AMOUNT of output captured, hence the "grep -ac" counts further down.
: "${OSMO_VTY_OPEN:=1}"    # sleep before writing
: "${OSMO_VTY_READ:=1.5}"  # read sleep
: "${OSMO_VTY_Q:=1}"       # nc -q1

vty_cmd() {
    local node="$1"
    local port="$2"
    local cmd="$3"

    local raw_output
    raw_output=$(osmo_vty "$node" "$port" enable "$cmd" end) || return 1

    # Filter banner/prompt lines.
    printf '%s\n' "$raw_output" | osmo_vty_clean 'Free Software lives'
    return 0
}

# ── Quick VTY availability check ──────────────────────────────────────
vty_available() {
    osmo_vty_up "$1" "$2"
}

# ── World detection and inventory ─────────────────────────────────────
MODE="$(osmo_mode)"

# Operators: osmo-operator-N containers in docker; in native mode OSMO_OP_IDS,
# then the osmo-opN netns, then N_OPERATORS from globals.conf, else 1.
mapfile -t OPS < <(osmo_ops)
N_OPS=${#OPS[@]}

# Hub: the osmo-inter-stp container in docker; in native mode this machine IF
# it holds the interstp role. On a native operator node the hub is elsewhere:
# osmo_hub returns 1 and osmo_hub_hint says what to run, and where.
HAS_INTER_STP=0
HUB_NODE="$INTER_STP"
if HUB_NODE="$(osmo_hub)"; then
    HAS_INTER_STP=1
else
    HUB_NODE="$INTER_STP"
fi

# HUB_REMOTE_IP - address of the hub when it does NOT run here: docker WAN
# (start-docker.sh --wan/--hub-ip injects OSMO_HUB_IP into the containers) or
# native operator node. Empty if the hub is local or no address is known.
HUB_REMOTE_IP="$(osmo_hub_is_remote 2>/dev/null || true)"

# HUB_EXPECTED - "a hub should exist" (not "I can query it"). A remote hub is
# expected as soon as an address is known (OSMO_HUB_IP from /etc/osmo-role in
# native, the --hub-ip address seen by the containers in docker WAN): the
# local STP's default route to it is verifiable from here even when the hub's
# internal state is not. In docker WITHOUT WAN both notions coincide: no
# container, no hub.
HUB_EXPECTED=$HAS_INTER_STP
```

```
if [ "$HUB_EXPECTED" -eq 0 ] && [ -n "$HUB_REMOTE_IP" ]; then
    HUB_EXPECTED=1
fi

# Expected number of AS/ASP on the hub. In docker it is the number of LOCAL
# operators, which are exactly those attaching to the hub. In native mode the
# hub serves REMOTE operators and no local inventory says how many: judge on
# presence, not on an invented total.
EXPECT_AS=0
[ "$MODE" = docker ] && EXPECT_AS=$N_OPS

if osmo_is_docker; then
    MODE_LABEL="docker (conteneurs osmo-operator-N)"
elif osmo_in_container; then
    MODE_LABEL="natif (dans le conteneur ${HOSTNAME:-?})"
else
    MODE_LABEL="natif (cette machine)"
fi

# Frame line: width computed in CHARACTERS (${#s}), not bytes - "printf %-58s"
# counts bytes and shifts by one per accented character.
boxline() {
    local s="$1" n
    n=$(( 58 - ${#s} )); [ "$n" -lt 0 ] && n=0
    echo -e "${CYAN}║${s}$(printf '%*s' "$n" '')║${NC}"
}

echo ""
echo -e "${CYAN}╔══════════════════════════════════════════════════════════╗${NC}"
boxline "          SS7 Health Check - $(date '+%H:%M:%S')"
boxline "          mode : ${MODE_LABEL}"
if [ -n "$HUB_REMOTE_IP" ]; then
    boxline "          ${N_OPS} operateur(s)  inter-stp : distant ${HUB_REMOTE_IP}"
else
    boxline "          ${N_OPS} operateur(s)  inter-stp=${HAS_INTER_STP}"
fi
echo -e "${CYAN}╚══════════════════════════════════════════════════════════╝${NC}"

if [ "$N_OPS" -eq 0 ] && [ "$HAS_INTER_STP" -eq 0 ] && [ -z "$HUB_REMOTE_IP" ]; then
    if [ "$MODE" = docker ]; then
        echo -e "${RED}Aucun container trouve${NC}"
    else
        echo -e "${RED}Aucun operateur ni inter-STP sur cette machine${NC}"
    fi
    exit 1
fi

# ═══════════════════════════════════════════════════════════════
# CHECK 1: Inter-STP (central hub)
# ═══════════════════════════════════════════════════════════════
if [ "$HAS_INTER_STP" -eq 1 ]; then
    # Point code read from the topology (osmo-multi.conf) when available;
    # the hub default is 0.0.0.
    banner "INTER-STP (hub SS7) - $(osmo_hub_pc 2>/dev/null || echo 0.0.0)"

    if ! vty_available "$HUB_NODE" 4239; then
        fail "VTY inter-STP (4239) inaccessible"
    else
        ok "VTY accessible"

        # Check that the inter-STP sees every operator as an AS
        as_output=$(vty_cmd "$HUB_NODE" 4239 "show cs7 instance 0 as all")
        n_as_total=$(echo "$as_output" | grep -acE 'as-op[0-9]+' || true)
        n_as_active=$(echo "$as_output" | grep -acE 'AS_ACTIVE' || true)
        info "AS declares sur le hub : ${n_as_total}"
```

```
        if [ "$EXPECT_AS" -gt 0 ]; then
            if [ "$n_as_active" -eq "$EXPECT_AS" ]; then
                ok "AS : ${n_as_active}/${EXPECT_AS} actifs (tous)"
            elif [ "$n_as_active" -gt 0 ]; then
                warn "AS : ${n_as_active}/${EXPECT_AS} actifs"
            else
                fail "AS : aucun actif"
            fi
        elif [ "$n_as_active" -gt 0 ]; then
            # Native: operators are remote machines, the expected total is
            # not knowable from here.
            ok "AS : ${n_as_active} actif(s) (total attendu inconnu en natif)"
        else
            fail "AS : aucun actif"
        fi

        # Check connected ASPs
        asp_output=$(vty_cmd "$HUB_NODE" 4239 "show cs7 instance 0 asp")
        n_asp_active=$(echo "$asp_output" | grep -acE 'ASP_ACTIVE' || true)

        if [ "$EXPECT_AS" -gt 0 ]; then
            if [ "$n_asp_active" -eq "$EXPECT_AS" ]; then
                ok "ASP : ${n_asp_active}/${EXPECT_AS} connectes (tous)"
            else
                warn "ASP : ${n_asp_active}/${EXPECT_AS} connectes"
            fi
        elif [ "$n_asp_active" -gt 0 ]; then
            ok "ASP : ${n_asp_active} connecte(s) (total attendu inconnu en natif)"
        else
            warn "ASP : aucun connecte"
        fi

        # Check the absence of PROHIB (blocked routes)
        route_output=$(vty_cmd "$HUB_NODE" 4239 "show cs7 instance 0 route")
        n_prohib=$(echo "$route_output" | grep -aciE 'prohib' || true)

        if [ "$n_prohib" -eq 0 ]; then
            ok "Routes : aucun PROHIB"
        else
            fail "Routes : ${n_prohib} PROHIB detectees"
        fi
    fi
elif [ -n "$HUB_REMOTE_IP" ]; then
    # REMOTE hub (WAN): "container absent" would be wrong - the hub runs,
    # elsewhere. What is observable from here is the LINK to it.
    banner "INTER-STP (hub distant ${HUB_REMOTE_IP})"

    # The operator → hub link is M3UA over SCTP (osmo-stp.cfg: "asp
    # asp-to-inter 2908 2910 m3ua"). A TCP probe on 2908 would return
    # "Connection refused" on a perfectly established link: the SCTP
    # association, as seen from the nodes, is authoritative.
    hub_assoc="$(osmo_hub_assoc "$HUB_REMOTE_IP" 2>/dev/null)" && hub_rc=0 || hub_rc=$?
    case "$hub_rc" in
        0) ok "Lien M3UA/SCTP → ${HUB_REMOTE_IP}:${OSMO_HUB_M3UA_PORT} : ${hub_assoc} noeud(s) associe(s)" ;;
        1) fail "Aucune association SCTP vers ${HUB_REMOTE_IP}:${OSMO_HUB_M3UA_PORT} (${hub_assoc}) - hub arrete,
         ↪  filtre, ou ASP shutdown" ;;
        *) skip "Association SCTP vers ${HUB_REMOTE_IP} non sondable ici (ss --sctp absent des noeuds)" ;;
    esac

    # No false failure on the hub's INTERNAL state: its VTY listens only on
    # ITS loopback. The sentence comes from _mode.sh and names the real IP.
    skip "$(osmo_hub_hint)"
else
    banner "INTER-STP"
    if [ "$MODE" = docker ]; then
        skip "Container ${INTER_STP} absent"
```

```
    else
        # No false failure: the hub may be perfectly alive, but its VTY listens
        # only on ITS loopback. The sentence comes from _mode.sh and names the
        # hub's real IP when /etc/osmo-role gives it.
        skip "$(osmo_hub_hint)"
    fi
fi

# ═══════════════════════════════════════════════════════════════════
# CHECK 2: each operator
# ═══════════════════════════════════════════════════════════════════
for op in ${OPS[@]+"${OPS[@]}"}; do
    op_id="$op"
    # osmo_node keeps the "osmo-operator-N" label even in native mode: it is
    # the display name the other checks correlate on.
    container="$(osmo_node "$op")"
    # The label reflects the topology (osmo-multi.conf) when it exists: the
    # point code comes from the local plan, and a native operator is shown as
    # running on the host rather than as a container.
    _pc="$(osmo_op_pc "$op_id" 2>/dev/null || true)"
    if [ "$(osmo_op_mode "$op_id" 2>/dev/null)" = "native" ]; then
        _label="natif (hote)"
    else
        _label="$container"
    fi
    banner "OPERATEUR ${op_id} (${_label}) - STP ${_pc:-inconnu}"

    # ── Local STP (4239) ──────────────────────────────────────────
    echo -e "  ${BOLD}STP local :${NC}"
    if ! vty_available "$op" 4239; then
        fail "STP VTY (4239) inaccessible"
    else
        # Check the connection to the inter-STP
        as_out=$(vty_cmd "$op" 4239 "show cs7 instance 0 as all")
        inter_state=$(echo "$as_out" | grep -aoP 'as-inter\s+\K(AS_\w+)' || echo "ABSENT")

        if [[ "$inter_state" == "AS_ACTIVE" ]]; then
            ok "as-inter → hub : ACTIVE"
        elif [[ "$inter_state" == "ABSENT" ]] && [ "$HUB_EXPECTED" -eq 1 ]; then
            fail "as-inter absent (pas de connexion au hub)"
        elif [[ "$inter_state" == "ABSENT" ]]; then
            skip "as-inter : ignore (aucun inter-STP connu depuis ce noeud)"
        else
            warn "as-inter : ${inter_state}"
        fi

        # Check local ASPs (MSC/BSC)
        asp_out=$(vty_cmd "$op" 4239 "show cs7 instance 0 asp")
        n_asp_ok=$(echo "$asp_out" | grep -acE 'ASP_ACTIVE' || true)

        if [ "$n_asp_ok" -ge 2 ]; then
            ok "ASP locaux : au moins 2 actifs (MSC+BSC)"
        elif [ "$n_asp_ok" -eq 1 ]; then
            warn "ASP locaux : 1 seul actif (manque MSC ou BSC)"
        else
            fail "ASP locaux : aucun actif"
        fi

        # Check the default route to the inter-STP (key element). Verifiable
        # as soon as a hub is EXPECTED: the remote hub of a native lab shows up
        # in the local STP routing table even when its own VTY is out of reach.
        route_out=$(vty_cmd "$op" 4239 "show cs7 instance 0 route")
        has_default=$(echo "$route_out" | grep -acE '0\.0\.0/0.*avail' || true)

        if [ "$has_default" -gt 0 ] && [ "$HUB_EXPECTED" -eq 1 ]; then
            ok "Route par defaut → inter-STP : presente et avail"
```

```
    elif [ "$HUB_EXPECTED" -eq 1 ]; then
        fail "Route par defaut → inter-STP absente ou non avail"
    elif [ "$MODE" != docker ]; then
        skip "Route par defaut → inter-STP : ignore (natif, aucun hub declare ici)"
    fi

    # Check the absence of PROHIB
    n_prohib=$(echo "$route_out" | grep -aciE 'prohib' || true)
    if [ "$n_prohib" -gt 0 ]; then
        fail "Routes PROHIB detectees : ${n_prohib}"
    fi

    # Count dynamic routes (information only)
    n_dyn=$(echo "$route_out" | grep -ac 'dyn' || true)
    info "Routes dynamiques apprises : ${n_dyn} (normal: devrait augmenter avec le nombre d'operateurs)"
fi

# In quick mode, move on to the next operator
if [ "$QUICK" -eq 1 ]; then
    continue
fi

# ── MSC (4254) ──────────────────────────────────────────────────
echo -e "  ${BOLD}MSC :${NC}"
if ! vty_available "$op" 4254; then
    fail "MSC VTY (4254) inaccessible"
else
    msc_asp=$(vty_cmd "$op" 4254 "show cs7 instance 0 asp")
    msc_active=$(echo "$msc_asp" | grep -acE 'ASP_ACTIVE' || true)
    [ "$msc_active" -gt 0 ] && ok "MSC→STP : ASP_ACTIVE" || fail "MSC→STP : pas de ASP actif"

    # SCCP - osmo_qgrep rather than "grep -aq": at the end of a pipe,
    # grep -aq exits on the first match and kills the producer with
    # SIGPIPE (141 under pipefail).
    msc_sccp=$(vty_cmd "$op" 4254 "show cs7 instance 0 sccp users")
    echo "$msc_sccp" | osmo_qgrep -aE 'SSN 254' && ok "SCCP SSN 254 (MSC-A) enregistre" || fail "SCCP SSN 254
     ↪  absent"
fi

# ── BSC (4242) ──────────────────────────────────────────────────
echo -e "  ${BOLD}BSC :${NC}"
if ! vty_available "$op" 4242; then
    fail "BSC VTY (4242) inaccessible"
else
    bsc_asp=$(vty_cmd "$op" 4242 "show cs7 instance 0 asp")
    bsc_active=$(echo "$bsc_asp" | grep -acE 'ASP_ACTIVE' || true)
    [ "$bsc_active" -gt 0 ] && ok "BSC→STP : ASP_ACTIVE" || fail "BSC→STP : pas de ASP actif"

    # SCCP
    bsc_sccp=$(vty_cmd "$op" 4242 "show cs7 instance 0 sccp users")
    echo "$bsc_sccp" | osmo_qgrep -aE 'SSN 254' && ok "SCCP SSN 254 (BSC) enregistre" || fail "SCCP SSN 254
     ↪  absent"

    # BTS state (optional)
    bsc_bts=$(vty_cmd "$op" 4242 "show bts 0" 2>/dev/null || echo "")
    if echo "$bsc_bts" | osmo_qgrep -aE "Oper 'Enabled'.*Avail 'OK'"; then
        ok "BTS 0 : OK"
    fi
fi

# ── HLR (4258) ──────────────────────────────────────────────────
echo -e "  ${BOLD}HLR :${NC}"
if ! vty_available "$op" 4258; then
    fail "HLR VTY (4258) inaccessible"
else
    hlr_gsup=$(vty_cmd "$op" 4258 "show gsup-connections")
```

```
        vlr_conn=$(echo "$hlr_gsup" | grep -acE "VLR" || true)
        [ "$vlr_conn" -gt 0 ] && ok "GSUP : VLR connecte" || fail "GSUP : VLR absent"
    fi
done

# ============================================================================
# CHECK 3: simple connectivity matrix
# ============================================================================
# The matrix crosses the LOCAL operators with each other through the hub: it
# makes sense as soon as a hub is EXPECTED (local, or remote over WAN - op1↔op2
# traffic crosses it in both cases) and there is more than one operator here.
# On a single operator it would reduce to one "self" cell: it is reported as
# skipped, pointing to the check that looks between machines.
if [ "$HUB_EXPECTED" -eq 1 ] && [ "$N_OPS" -gt 1 ]; then
    if [ -n "$HUB_REMOTE_IP" ]; then
        banner "MATRICE DE CONNECTIVITE (via inter-STP distant ${HUB_REMOTE_IP})"
    else
        banner "MATRICE DE CONNECTIVITE (via inter-STP)"
    fi

    printf "  %-8s" ""
    for j in "${OPS[@]}"; do printf "  Op%-3s" "$j"; done
    echo ""

    for op in "${OPS[@]}"; do
        src_id="$op"
        printf "  Op%-5s" "${src_id}"

        # Only basic connectivity is tested: does the operator have a default route?
        route_out=$(vty_cmd "$op" 4239 "show cs7 instance 0 route" 2>/dev/null || echo "")
        has_default=$(echo "$route_out" | grep -acE '0\.0\.0/0.*avail' || true)

        for j in "${OPS[@]}"; do
            if [ "$j" -eq "$src_id" ]; then
                printf "  ${CYAN}%-5s${NC}" "self"
            elif [ "$has_default" -gt 0 ]; then
                # With a default route, connectivity through the inter-STP is assumed
                printf "  ${GREEN}%-5s${NC}" "via"
            else
                printf "  ${RED}%-5s${NC}" "FAIL"
                FAIL=$((FAIL+1))
            fi
        done
        echo ""
    done
    echo ""
    echo "  Note: 'via' signifie que l'operateur a une route par defaut vers l'inter-STP"
    echo "  Les routes dynamiques specifiques sont apprises au fur et a mesure"
elif [ "$MODE" != docker ]; then
    banner "MATRICE DE CONNECTIVITE (via inter-STP)"
    skip "ignore (natif) : ${N_OPS} operateur(s) local(aux) - la connectivite entre operateurs passe par le WAN,
     ↪  voir checks/wan_ss7_check.sh"
fi

# ============================================================================
# SUMMARY
# ============================================================================
echo ""
echo -e "$(printf '=%.0s' {1..60})"

if [ "$FAIL" -eq 0 ]; then
    echo -e "  ${GREEN}${BOLD}SS7 OK${NC}  -  ${GREEN}${PASS} pass${NC}  ${YELLOW}${WARN} warn${NC}  ${SKIP} skip"
elif [ "$FAIL" -le 3 ]; then
    echo -e "  ${YELLOW}${BOLD}SS7 DEGRADE${NC}  -  ${GREEN}${PASS} pass${NC}  ${RED}${FAIL} fail${NC}
     ↪  ${YELLOW}${WARN} warn${NC}  ${SKIP} skip"
else
```

```
    echo -e "  ${RED}${BOLD}SS7 KO${NC}  -  ${GREEN}${PASS} pass${NC}  ${RED}${FAIL} fail${NC}  ${YELLOW}${WARN}
    ↪  warn${NC}  ${SKIP} skip"
fi
echo -e "$(printf '=%.0s' {1..60})"
echo ""

exit "$FAIL"
```

2.27 /opt/GSM/osmo-operator/checks/vty-debug-dump.sh

18175 octets, 477 lignes → 477 lignes

```
#!/bin/bash
# vty-debug-dump.sh - Dump every Osmocom VTY of each operator and of the inter-STP
#
# Walks every Osmocom VTY of each operator AND of the inter-STP, enables the
# DEBUG log level, runs the relevant "show" commands and consolidates
# everything into one log file.
#
# TWO WORLDS, automatic detection (checks/_mode.sh):
#   docker: one osmo-operator-N container per operator, the SS7 hub is the
#           osmo-inter-stp container; VTYs are queried through "docker exec".
#   native: ISO, VM or bare machine - a single operator (unless netns
#           topology), VTYs reached by telnet on 127.0.0.1. What exists only
#           under docker (the inter-STP container) is EXPLICITLY SKIPPED: a
#           dump inventing an absent hub would read as an SS7 outage.
#
# The "container=osmo-operator-N" label is written in the dump IN BOTH MODES:
# checks/operator_summary.sh anchors all its extractions on it
# (grep -A20 "container=..."). In native mode it is a display name, a
# correlation key, not a queryable object.
#
# VTYs covered per operator:
#   OsmoSTP          4239
#   OsmoHLR          4258
#   OsmoMGW          4243
#   OsmoMSC          4254
#   OsmoBSC          4242
#   OsmoBTS          4241
#   OsmoPCU          4240
#   OsmoSGSN         4245
#   OsmoGGSN         4260
#   OsmoSIP-conn     4255
#
# Inter-STP (SS7 hub) VTY:
#   OsmoSTP (hub)    4239 (or the port bound in /etc/osmocom/osmo-stp-interop.cfg)
#
# Usage:
#   sudo ./checks/vty-debug-dump.sh [--out /path/log.txt] [--docker|--native]
# Environment: OUTFILE (default /tmp/vty-debug-dump.txt), OSMO_MODE=docker|native.

set -euo pipefail
GREEN='\033[0;32m'; CYAN='\033[0;36m'; YELLOW='\033[1;33m'; RED='\033[0;31m'; NC='\033[0m'

# Saved BEFORE sourcing: the library writes into OSMO_MODE (memoisation), so
# afterwards it cannot be told whether the value came from the environment.
MODE_ENV="${OSMO_MODE:-}"

# ── Common docker/native library ──────────────────────────────────────
# Looked up RELATIVE to the script (BASH_SOURCE): the repo and the ISO do not
# share the same root, and the ISO ships checks/ as a whole, so _mode.sh is
# always next to this file.
_here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
for _c in "$_here/_mode.sh" /opt/GSM/osmo-operator/checks/_mode.sh; do
```

```
    [ -r "$_c" ] && { . "$_c"; break; }
done
command -v osmo_mode >/dev/null || { echo "checks/_mode.sh introuvable" >&2; exit 1; }

# ── Options ──────────────────────────────────────────────────────────
# --out is accepted AT ANY POSITION: operator_summary.sh --live runs
# "./vty-debug-dump.sh --out /tmp/vty-live-$$.txt" and must keep working even
# when preceded by --native.
OUTFILE="${OUTFILE:-/tmp/vty-debug-dump.txt}"
MODE_FORCED=0
while [ $# -gt 0 ]; do
    case "$1" in
        --out)
            [ -n "${2:-}" ] || { echo "--out attend un chemin" >&2; exit 1; }
            OUTFILE="$2"; shift 2 ;;
        --out=*)
            OUTFILE="${1#*=}"; shift ;;
        --docker|--native|--mode=*)
            osmo_mode_force "$1" || { echo "mode inconnu : $1" >&2; exit 1; }
            MODE_FORCED=1; shift ;;
        -h|--help)
            echo "Usage : sudo ./vty-debug-dump.sh [--out /chemin/log.txt] [--docker|--native]"
            echo "  --docker  force le mode conteneurs (docker exec)"
            echo "  --native  force le mode natif (VTY locaux sur 127.0.0.1)"
            echo "  sans option : detection automatique"
            exit 0 ;;
        *)
            echo "option inconnue : $1" >&2
            echo "Usage : sudo ./vty-debug-dump.sh [--out /chemin/log.txt] [--docker|--native]" >&2
            exit 1 ;;
    esac
done

INTER_STP_CONTAINER="$(osmo_node hub)"   # "osmo-inter-stp" in both modes

# ── VTY tables: two sets (operator / inter-stp) ──────────────────────
declare -A OP_VTY_NAME OP_VTY_SHOW
declare -A ISTP_VTY_NAME ISTP_VTY_SHOW

register_op_vty() {
    local port="$1" name="$2" show_cmds="$3"
    OP_VTY_NAME[$port]="$name"
    OP_VTY_SHOW[$port]="$show_cmds"
}

register_istp_vty() {
    local port="$1" name="$2" show_cmds="$3"
    ISTP_VTY_NAME[$port]="$name"
    ISTP_VTY_SHOW[$port]="$show_cmds"
}

# ── Operator show commands ───────────────────────────────────────────

register_op_vty 4239 "OsmoSTP" "\
show cs7 instance 0 users|\
show cs7 instance 0 as all|\
show cs7 instance 0 asp|\
show cs7 instance 0 route|\
show cs7 instance 0 sccp users|\
show cs7 instance 0 sccp connections|\
show cs7 instance 0 sccp timers|\
show logging level|\
show talloc-context all brief"

register_op_vty 4258 "OsmoHLR" "\
show subscriber all|\
```

```
show subscriber count|\
show gsup-connections|\
show logging level|\
show talloc-context all brief"

register_op_vty 4243 "OsmoMGW" "\
show mgcp stats|\
show mgcp active|\
show mgcp endpoint Name trunk-0|\
show logging level|\
show talloc-context all brief"

register_op_vty 4254 "OsmoMSC" "\
show network|\
show subscriber all|\
show subscriber count|\
show transaction all|\
show call-legs|\
show connection|\
show sgs-connections|\
show cs7 instance 0 users|\
show cs7 instance 0 asp|\
show cs7 instance 0 as all|\
show cs7 instance 0 sccp users|\
show cs7 instance 0 sccp connections|\
show logging level|\
show stats|\
show talloc-context all brief"

register_op_vty 4242 "OsmoBSC" "\
show network|\
show bts all|\
show bts 0|\
show trx 0 0|\
show timeslot all|\
show lchan all|\
show lchan summary|\
show paging all|\
show paging-group all|\
show conns|\
show cs7 instance 0 users|\
show cs7 instance 0 asp|\
show cs7 instance 0 as all|\
show cs7 instance 0 sccp users|\
show cs7 instance 0 sccp connections|\
show logging level|\
show stats|\
show talloc-context all brief"

register_op_vty 4241 "OsmoBTS" "\
show bts 0|\
show trx 0 0|\
show ts all|\
show lchan all|\
show lchan summary|\
show phy 0 inst all|\
show logging level|\
show talloc-context all brief"

register_op_vty 4240 "OsmoPCU" "\
show bts all|\
show tbf all|\
show ms all|\
show logging level|\
show talloc-context all brief"
```

```
register_op_vty 4245 "OsmoSGSN" "\
show sgsn|\
show mm-context all|\
show pdp-context all|\
show subscriber cache|\
show ns|\
show ns stats|\
show ns entities|\
show bssgp|\
show logging level|\
show stats|\
show talloc-context all brief"

register_op_vty 4260 "OsmoGGSN" "\
show ggsn|\
show pdp-context all|\
show apn all|\
show logging level|\
show talloc-context all brief"

register_op_vty 4255 "OsmoSIPconn" "\
show logging level|\
show talloc-context all brief"

OP_VTY_PORTS=(4239 4258 4243 4254 4242 4241 4240 4245 4260 4255)

# ── Inter-STP show commands - sweep of instances 0..9 ────────────────────
# The inter-STP hub may have N CS7 instances (one per operator link).

ISTP_SHOW_CMDS="show cs7 instance 0 users|\
show cs7 instance 0 as all|\
show cs7 instance 0 as all active|\
show cs7 instance 0 asp|\
show cs7 instance 0 route|\
show cs7 instance 0 sccp users|\
show cs7 instance 0 sccp connections|\
show cs7 instance 0 sccp timers|\
show cs7 instance 1 users|\
show cs7 instance 1 as all|\
show cs7 instance 1 asp|\
show cs7 instance 1 route|\
show cs7 instance 1 sccp users|\
show cs7 instance 1 sccp connections|\
show cs7 instance 2 users|\
show cs7 instance 2 as all|\
show cs7 instance 2 asp|\
show cs7 instance 2 route|\
show cs7 instance 2 sccp users|\
show cs7 instance 2 sccp connections|\
show cs7 instance 3 users|\
show cs7 instance 3 as all|\
show cs7 instance 3 asp|\
show cs7 instance 3 route|\
show cs7 instance 4 as all|\
show cs7 instance 4 asp|\
show cs7 instance 4 route|\
show cs7 instance 5 as all|\
show cs7 instance 5 asp|\
show cs7 instance 5 route|\
show cs7 instance 6 as all|\
show cs7 instance 6 asp|\
show cs7 instance 6 route|\
show cs7 instance 7 as all|\
show cs7 instance 7 asp|\
show cs7 instance 7 route|\
show cs7 instance 8 as all|\
```

```
show cs7 instance 8 asp|\
show cs7 instance 8 route|\
show cs7 instance 9 as all|\
show cs7 instance 9 asp|\
show cs7 instance 9 route|\
show logging level|\
show stats|\
show talloc-context all brief"

# The hub may have been moved off port 4239 (co-located with the operator
# STP, see start-interstp.sh): read its VTY port from its config. In docker
# the hub stays on 4239 (queried through docker exec) and the host file is
# absent: sed returns nothing and 4239 is kept.
ISTP_HUB_VTY=4239
_hubp="$(sed -n 's/^ *bind 127\.0\.0\.1 \([0-9]*\)$/\1/p' /etc/osmocom/osmo-stp-interop.cfg 2>/dev/null | head
 ↪  -1)"
[ -n "$_hubp" ] && ISTP_HUB_VTY="$_hubp"
register_istp_vty "$ISTP_HUB_VTY" "OsmoSTP-hub" "$ISTP_SHOW_CMDS"

ISTP_VTY_PORTS=("$ISTP_HUB_VTY")

# ── Node inventory - docker or native ──────────────────────────────────
MODE="$(osmo_mode)"
# An environment OSMO_MODE that is neither docker nor native is ignored by the
# library (it re-detects): it is therefore not reported as forced.
if   [ "$MODE_FORCED" -eq 1 ]; then MODE_SRC="force par option"
elif [ "$MODE_ENV" = docker ] || [ "$MODE_ENV" = native ]; then MODE_SRC="force par OSMO_MODE"
else MODE_SRC="detection automatique"
fi

# osmo_ops: osmo-operator-N containers in docker; OSMO_OP_IDS / netns /
# N_OPERATORS (else 1) in native. Already numerically sorted and deduplicated.
mapfile -t OPS < <(osmo_ops)
OP_CONTAINERS=()
for op in "${OPS[@]:-}"; do
    [ -n "$op" ] && OP_CONTAINERS+=("$(osmo_node "$op")")
done

# Inter-STP: present IF it can be queried FROM HERE.
#   docker: the osmo-inter-stp container is running.
#   native: this machine IS the hub (OSMO_ROLE=interstp, or an osmo-stp started
#           on osmo-stp-interop.cfg). If the hub is another machine, its VTY
#           listens only on ITS 127.0.0.1: nothing can be dumped from here and
#           this is stated - an empty "OsmoSTP-hub" block would read as a
#           dead hub.
HAS_INTER_STP=0
osmo_hub >/dev/null 2>&1 && HAS_INTER_STP=1

if [ ${#OP_CONTAINERS[@]} -eq 0 ] && [ "$HAS_INTER_STP" -eq 0 ]; then
    echo -e "${RED}Aucun container osmo-operator-N ni ${INTER_STP_CONTAINER} trouve.${NC}"
    if [ "$MODE" = docker ]; then
        echo -e "  Containers actifs : $(docker ps --format '{{.Names}}' | tr '\n' ' ')"
    else
        echo -e "  Mode natif : aucun operateur declare (OSMO_OP_IDS, netns ${OSMO_NETNS_PREFIX}N, N_OPERATORS)."
    fi
    exit 1
fi

N_OPS=${#OP_CONTAINERS[@]}
# The mode is printed BEFORE anything else: a diagnostic that does not say
# which world it looked at is not usable.
echo -e "${CYAN}Mode : ${MODE} (${MODE_SRC})${NC}"
echo -e "${GREEN}=== VTY Debug Dump - ${N_OPS} operateur(s) + inter-stp=${HAS_INTER_STP} → ${OUTFILE} ===${NC}"
echo ""

# ── Clear / initialise the output file ─────────────────────────────────
```

```
: > "$OUTFILE"
log() { echo -e "$*" | tee -a "$OUTFILE"; }
logsep() {
    local title="$1"
    log ""
    log "$(printf '=%.0s' {1..72})"
    log "  ${title}"
    log "$(printf '=%.0s' {1..72})"
}
logsubsep() {
    local title="$1"
    log ""
    log "  $(printf '-%.0s' {1..68})"
    log "  ${title}"
    log "  $(printf '-%.0s' {1..68})"
}

log "VTY DEBUG DUMP - $(date '+%Y-%m-%d %H:%M:%S')"
log "Mode       : ${MODE} (${MODE_SRC})"
log "Operateurs : ${N_OPS}   Containers : ${OP_CONTAINERS[*]:-aucun}"
if   [ "$HAS_INTER_STP" -eq 1 ]; then log "Inter-STP  : present"
elif [ "$MODE" = docker        ]; then log "Inter-STP  : absent"
else                                    log "Inter-STP  : ignore (natif) - $(osmo_hub_hint)"
fi

# ── Query one VTY on a node ──────────────────────────────────────────
# $1=<op> (operator number or "hub")  $2=port  $3=name  $4=show commands
# osmo_vty_up / osmo_vty handle the dialogue (docker exec, or local in native
# mode). The "container=" field of the dump keeps the node label.
query_vty() {
    local op="$1"
    local port="$2"
    local name="$3"
    local show_str="$4"
    local container
    container="$(osmo_node "$op")"

    # Quick connectivity test. In native mode the passive probe (ss) runs
    # first: it opens no VTY session.
    if ! osmo_vty_up "$op" "$port"; then
        echo -e "  ${YELLOW}[SKIP]${NC} ${name} (port ${port}) - non disponible"
        log "  [SKIP] ${name} port ${port} - non disponible"
        return 1
    fi

    echo -ne "  ${CYAN}${name}${NC} (${port}) ... "

    # Build the VTY script: switch to DEBUG, then the show commands, then end.
    local -a VTY_CMDS=(enable "logging level all debug" "logging filter all 1")
    IFS='|' read -ra SHOW_CMDS <<< "$show_str"
    for cmd in "${SHOW_CMDS[@]}"; do
        VTY_CMDS+=("$cmd")
    done
    VTY_CMDS+=(end)

    # Timings (nc -q2, 0.5 s before writing, 2 s of reading): these are large
    # command sets, and changing these values changes the AMOUNT of output
    # captured - hence the line count printed right after.
    local raw_output
    raw_output="$(OSMO_VTY_OPEN=0.5 OSMO_VTY_READ=2 OSMO_VTY_Q=2 \
                  osmo_vty "$op" "$port" "${VTY_CMDS[@]}" || true)"

    # Filter VTY banner/prompt. Without argument the "Free Software lives" line
    # is KEPT, unlike global_check.sh and ss7_check.sh.
    local clean_output
    clean_output="$(printf '%s\n' "$raw_output" | osmo_vty_clean)"
```

```
    local n_lines
    n_lines=$(echo "$clean_output" | wc -l)
    echo -e "${GREEN}${n_lines} lignes${NC}"

    {
        echo ""
        echo "  [${name}] port=${port}  container=${container}"
        echo "  Commandes show : $(echo "$show_str" | tr '|' ', ')"
        echo ""
        echo "$clean_output" | sed 's/^/    /'
    } >> "$OUTFILE"

    return 0
}

# ── Main loop: operator nodes ──────────────────────────────────────────
TOTAL_OK=0
TOTAL_SKIP=0

for op in "${OPS[@]:-}"; do
    [ -n "$op" ] || continue
    container="$(osmo_node "$op")"
    logsep "Container : ${container}  -  $(date '+%H:%M:%S')"
    echo -e "${GREEN}--- ${container} ---${NC}"

    for port in "${OP_VTY_PORTS[@]}"; do
        logsubsep "${OP_VTY_NAME[$port]} (port ${port})"
        if query_vty "$op" "$port" \
                "${OP_VTY_NAME[$port]}" "${OP_VTY_SHOW[$port]}"; then
            TOTAL_OK=$(( TOTAL_OK + 1 ))
        else
            TOTAL_SKIP=$(( TOTAL_SKIP + 1 ))
        fi
    done
    echo ""
done

# ── Inter-STP ──────────────────────────────────────────────────────────
if [ "$HAS_INTER_STP" -eq 1 ]; then
    logsep "Container : ${INTER_STP_CONTAINER} (hub SS7)  -  $(date '+%H:%M:%S')"
    echo -e "${GREEN}--- ${INTER_STP_CONTAINER} ---${NC}"

    for port in "${ISTP_VTY_PORTS[@]}"; do
        logsubsep "${ISTP_VTY_NAME[$port]} (port ${port})"
        if query_vty hub "$port" \
                "${ISTP_VTY_NAME[$port]}" "${ISTP_VTY_SHOW[$port]}"; then
            TOTAL_OK=$(( TOTAL_OK + 1 ))
        else
            TOTAL_SKIP=$(( TOTAL_SKIP + 1 ))
        fi
    done
    echo ""
elif [ "$MODE" = docker ]; then
    log ""
    log "  [INFO] Container ${INTER_STP_CONTAINER} absent - skipped"
else
    # Native without a local hub: neither OK nor failure, and no counter
    # touched - there is nothing to query here, it is not an outage.
    echo -e "  ${YELLOW}${INTER_STP_CONTAINER} - ignore (natif)${NC}"
    echo -e "  ${YELLOW}  $(osmo_hub_hint)${NC}"
    log ""
    log "  [INFO] ${INTER_STP_CONTAINER} - ignore (natif) : $(osmo_hub_hint)"
fi

# ── Summary ────────────────────────────────────────────────────────────
```

```
logsep "RESUME - $(date '+%Y-%m-%d %H:%M:%S')"
log "  Mode                  : ${MODE} (${MODE_SRC})"
log "  VTY interroges        : ${TOTAL_OK}"
log "  VTY non disponibles   : ${TOTAL_SKIP}"
log "  Operateurs            : ${N_OPS}"
if   [ "$HAS_INTER_STP" -eq 1 ]; then log "  Inter-STP             : oui"
elif [ "$MODE" = docker        ]; then log "  Inter-STP             : non"
else                                    log "  Inter-STP             : ignore (natif)"
fi
log "  Fichier               : ${OUTFILE}"
log "  Taille                : $(wc -c < "$OUTFILE") octets  /  $(wc -l < "$OUTFILE") lignes"

echo ""
echo -e "${GREEN}╔═══════════════════════════════════════════════════════════════╗${NC}"
echo -e "${GREEN}║  Dump complet → ${OUTFILE}${NC}"
echo -e "${GREEN}╚═══════════════════════════════════════════════════════════════╝${NC}"
echo ""
echo -e "  ${CYAN}Consulter :${NC}  less ${OUTFILE}"
echo -e "  ${CYAN}Filtrer   :${NC}  grep -A5 'OsmoBSC' ${OUTFILE}"
echo -e "  ${CYAN}SS7 hub   :${NC}  grep -A10 'OsmoSTP-hub' ${OUTFILE}"
echo -e "  ${CYAN}Routes    :${NC}  grep -A20 'show cs7.*route' ${OUTFILE}"
echo -e "  ${CYAN}ASP etat  :${NC}  grep -iE 'asp|active|inactive|down' ${OUTFILE}"
echo -e "  ${CYAN}SCCP      :${NC}  grep -A10 'sccp' ${OUTFILE}"
echo -e "  ${CYAN}Erreurs   :${NC}  grep -iE 'error|fail|warn|prohib|timeout|refused' ${OUTFILE}"

# Local copy when run from the host
if [ -w "$(pwd)" ]; then
    cp "$OUTFILE" "$(pwd)/vty-debug-dump-$(date '+%Y%m%d-%H%M%S').txt" 2>/dev/null && \
        echo -e "  ${CYAN}Copie locale :${NC} $(pwd)/vty-debug-dump-$(date '+%Y%m%d-%H%M%S').txt" || true
fi
```

2.28 /opt/GSM/osmo-operator/checks/wan_ss7_check.sh

17666 octets, 322 lignes → 322 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════════
# checks/wan_ss7_check.sh - WAN health, and the reach of SS7 across the WAN
#
# checks/ss7_check.sh verifies the SS7 of ONE node (inter-STP, ASPs, routes).
# This script answers the next question: "and between nodes?"
#
# Without a common inter-STP hub the osmo-operator WAN carries NO SS7. Between
# two nodes there is only:
#     • SIP/RTP  (Asterisk <-> Asterisk)  -> voice
#     • TCP 789x (sms-interop-relay)      -> SMS
# SS7/M3UA (osmo-stp, point codes 1.N.2, inter-STP 0.0.0) stays INSIDE a node:
# each operator's ASP points at 172.20.0.10 (private docker range) and the
# catch-all "route 0.0.0 0.0.0 -> as-inter" sends unknown PCs to that local
# inter-STP. An inter-node call is a SIP termination, not a MAP/ISUP relay.
# With a hub (OSMO_HUB_IP) SS7 does cross the WAN and the point codes must be
# unique per node (1.<node><op>.<role>).
#
# The script verifies which situation applies and flags the two cases where
# inter-node SS7 is wrongly assumed: identical point codes on several nodes,
# and an M3UA port (2905/2908) exposed on the public interface.
#
# Usage: sudo checks/wan_ss7_check.sh [--verbose] [--conf /etc/osmo-wan.conf]
#   --verbose, -v   print details of each check
#   --conf FILE     WAN node table (default /etc/osmo-wan.conf)
# ═══════════════════════════════════════════════════════════════════════════
set -u

GREEN='\033[0;32m'; CYAN='\033[0;36m'; YELLOW='\033[1;33m'
```

```
RED='\033[0;31m'; BOLD='\033[1m'; NC='\033[0m'

VERBOSE=0
CONF="/etc/osmo-wan.conf"
while [ $# -gt 0 ]; do
    case "$1" in
        --verbose|-v) VERBOSE=1 ;;
        --conf)       CONF="${2:-}"; shift ;;
        --conf=*)     CONF="${1#*=}" ;;
        -h|--help)    sed -n '2,25p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
    esac
    shift
done

PASS=0; FAIL=0; WARN=0; SKIP=0
ok()   { echo -e "  ${GREEN}✓${NC} $*"; PASS=$((PASS+1)); }
fail() { echo -e "  ${RED}✗${NC} $*"; FAIL=$((FAIL+1)); }
warn() { echo -e "  ${YELLOW}⚠${NC} $*"; WARN=$((WARN+1)); }
skip() { echo -e "  ${YELLOW}-${NC} $*"; SKIP=$((SKIP+1)); }
info() { [ $VERBOSE -eq 1 ] && echo -e "    ${CYAN}|${NC} $*" || true; }
banner(){ echo ""; echo -e "${BOLD}$*${NC}"; printf '─%.0s' $(seq 1 64); echo ""; }

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

echo -e "${CYAN}${BOLD}"
echo "╔══════════════════════════════════════════════════════════════╗"
echo "║   WAN - diagnostic voix / SMS / SS7 entre noeuds osmo-operator      ║"
echo "╚══════════════════════════════════════════════════════════════╝"
echo -e "${NC}"

# ── 0. Node table ──────────────────────────────────────────────────
banner "0. Table des noeuds WAN"
if [ ! -r "$SCRIPT_DIR/network/wan-nodes.sh" ]; then
    fail "network/wan-nodes.sh introuvable - depot incomplet"; exit 1
fi
# shellcheck source=network/wan-nodes.sh
. "$SCRIPT_DIR/network/wan-nodes.sh"

if ! wan_nodes_load "$CONF" 2>/dev/null; then
    fail "$CONF illisible - aucun WAN configure sur ce noeud"
    echo -e "     ${CYAN}sudo ./launch/start-docker.sh --wan${NC}  ou  ${CYAN}sudo ./start-native.sh --wan${NC}"
    exit 1
fi
[ "${WAN_NODE_ID:-0}" = "0" ] && wan_nodes_detect_self 2>/dev/null
if wan_nodes_validate 2>/dev/null; then ok "table valide : ${WAN_NODE_COUNT} noeud(s)"
else fail "table incoherente (voir messages ci-dessus)"; fi
wan_nodes_summary

LOCAL_IND="$(wan_local_ind)"
REMOTES=(); for id in "${WAN_NODE_LIST[@]}"; do [ "$id" = "${WAN_NODE_ID:-0}" ] || REMOTES+=("$id"); done
if [ "${#REMOTES[@]}" -eq 0 ]; then
    warn "table a un seul noeud : aucun pair, rien a verifier cote WAN"
    echo -e "     ${CYAN}Ajoutez des noeuds : sudo ./launch/start-docker.sh --wan${NC}"
    exit 0
fi

# ── Mode (docker / native) ─────────────────────────────────────────
if command -v docker >/dev/null 2>&1 && docker ps --format '{{.Names}}' 2>/dev/null | grep -q '^osmo-operator-1$';
 ↪  then
    MODE=docker
    N_OPS=$(docker ps --format '{{.Names}}' | grep -c '^osmo-operator-[0-9]*$')
else
    MODE=native; N_OPS=1
fi
info "mode=${MODE} operateurs locaux=${N_OPS}"
```

```
# ── What a peer carries, and where it listens ──────────────────────────
# These two helpers mirror what network/setup-wan-mesh.sh actually writes into
# the configs: the number of trunks expected per peer is the peer's operator
# count, not the local one.
peer_nops() {   # $1 = node id
    local n="${WAN_NOPS[$1]:-}"
    if [[ "$n" =~ ^[1-9][0-9]*$ ]]; then echo "$n"; return; fi
    # Table without the 4th field (hand-written table): a docker backbone peer
    # is one container, hence one operator; elsewhere assume the peer is set
    # up like us. Same rule as node_nops in network/setup-wan-mesh.sh.
    if peer_is_backbone "${WAN_IP[$1]:-}"; then echo 1; else echo "${N_OPS:-1}"; fi
}
# A docker backbone peer is reached on its own address: the container listens
# on 5060 and on the base SMS relay port. 5080/5082 and 7891 are the ports
# PUBLISHED on the host and are meaningless on that side.
peer_is_backbone() { case "$1" in 172.20.0.*) return 0 ;; *) return 1 ;; esac; }

ast() { # $1=op  $2=asterisk CLI command
    if [ "$MODE" = docker ]; then docker exec "osmo-operator-$1" asterisk -rx "$2" 2>/dev/null
    else asterisk -rx "$2" 2>/dev/null; fi
}
inside() { # $1=op  $2=shell command
    if [ "$MODE" = docker ]; then docker exec "osmo-operator-$1" bash -c "$2" 2>/dev/null
    else bash -c "$2" 2>/dev/null; fi
}

# ── 1. Peer reachability ──────────────────────────────────────────────
banner "1. Pairs - joignabilite IP et ports WAN"
for r in "${REMOTES[@]}"; do
    rip="${WAN_IP[$r]}"
    if ping -c1 -W2 "$rip" >/dev/null 2>&1; then ok "node ${r} ${rip} repond au ping"
    else warn "node ${r} ${rip} ne repond pas au ping (ICMP filtre ? noeud eteint ?)"; fi
    for i in $(seq 1 "$(peer_nops "$r")"); do
        if peer_is_backbone "$rip"; then sip=5060; sms=7890
        else sip=$(( 5080 + (i - 1) * 2 )); sms=$(( 7890 + i - 1 )); fi
        if command -v nc >/dev/null 2>&1; then
            nc -z -w2 "$rip" "$sms" 2>/dev/null \
                && ok "node ${r} op${i} : relais SMS ${rip}:${sms} ouvert" \
                || warn "node ${r} op${i} : relais SMS ${rip}:${sms} ferme - SMS WAN KO dans ce sens"
        else skip "nc absent : test TCP ${rip}:${sms} impossible"; fi
        info "SIP attendu sur ${rip}:${sip}/udp (UDP : pas de test fiable sans trafic)"
    done
done

# ── 2. Asterisk - WAN trunks ──────────────────────────────────────────
banner "2. Asterisk - trunks WAN (voix)"
for i in $(seq 1 "$N_OPS"); do
    out="$(ast "$i" 'pjsip show endpoints')"
    if [ -z "$out" ]; then fail "op${i} : Asterisk ne repond pas"; continue; fi
    for r in "${REMOTES[@]}"; do
        for j in $(seq 1 "$(peer_nops "$r")"); do
            ep="wan_n${r}_op${j}"
            if echo "$out" | grep -q "$ep"; then
               state="$(ast "$i" "pjsip show aor $ep" | grep -iE 'Avail|Unavail|Contact' | head -3 | tr '\n' ' ')"
                if echo "$out" | grep -E "Endpoint:[[:space:]]+$ep" | grep -qi 'Unavailable'; then
                    warn "op${i} → ${ep} present mais Unavailable (pair injoignable)"
                else ok "op${i} → ${ep}"; fi
                info "$state"
            else
                fail "op${i} : trunk ${ep} ABSENT - relancez network/setup-wan-mesh.sh"
            fi
        done
    done
done

# ── 3. Dialplan - prefixes ────────────────────────────────────────────
```

```
banner "3. Dialplan - un indicatif route par pair"
for i in $(seq 1 "$N_OPS"); do
    for r in "${REMOTES[@]}"; do
        ind="${WAN_IND[$r]}"
        if ast "$i" "dialplan show wan_out" | grep -q "_${ind}"; then
            ok "op${i} : indicatif ${ind} → node ${r} present dans [wan_out]"
        else
            fail "op${i} : indicatif ${ind} absent de [wan_out]"
        fi
    done
    ast "$i" "dialplan show gsm_in" | grep -q "_${LOCAL_IND}" \
        && ok "op${i} : indicatif local ${LOCAL_IND} traite en interne" \
        || warn "op${i} : indicatif local ${LOCAL_IND} non traite - un appel 'plein numero' echouera"
done

# ── 4. SMS - routes and strip ──────────────────────────────────────
banner "4. SMS - routes WAN et retrait d'indicatif"
for i in $(seq 1 "$N_OPS"); do
    conf="$(inside "$i" 'cat /etc/osmocom/sms-routing.conf')"
    if [ -z "$conf" ]; then fail "op${i} : sms-routing.conf illisible"; continue; fi
    for r in "${REMOTES[@]}"; do
        ind="${WAN_IND[$r]}"
        line="$(echo "$conf" | grep -E "^${ind}[0-9]+ =" | head -1)"
        if [ -z "$line" ]; then
            fail "op${i} : aucune route SMS pour l'indicatif ${ind}"
        elif echo "$line" | grep -q 'strip='; then
            ok "op${i} : route SMS ${ind} avec strip - le HLR distant verra le numero nu"
        else
            fail "op${i} : route SMS ${ind} SANS strip= - le noeud distant cherchera "${ind}10001" dans son HLR et
             ↪  repondra "not found""
        fi
    done
    # The relay must understand strip=: a correct config on a relay that
    # ignores it is just as broken as a config without strip.
    if inside "$i" 'grep -q lookup_strip /opt/GSM/osmo-operator/scripts/sms-interop-relay.py 2>/dev/null || grep
     ↪  -q lookup_strip /etc/osmocom/sms-interop-relay.py 2>/dev/null || grep -rq lookup_strip /usr/local/bin
     ↪  2>/dev/null'; then
        ok "op${i} : le relais SMS gere strip="
    else
        warn "op${i} : relais SMS sans lookup_strip - version anterieure, SMS WAN muets"
    fi
done

# ── 5. SS7 - what crosses the WAN, what does not ──────────────────
banner "5. SS7 - portee reelle vis-a-vis du WAN"
echo -e "  ${CYAN}Attendu : le SS7 s'arrete au bord du noeud. On verifie que c'est bien${NC}"
echo -e "  ${CYAN}le cas, et qu'on ne croit pas avoir un lien SS7 inter-noeud.${NC}"
echo ""

# A hub changes the picture: with one, SS7 DOES cross the WAN and the point
# codes must be verified rather than their absence.
HUB_IP=""
[ -r /etc/osmo-role ] && HUB_IP="$(awk -F= '/^OSMO_HUB_IP=/{print $2}' /etc/osmo-role)"
# In docker /etc/osmo-role does not exist on the host: the hub address is
# injected into the containers by start-docker.sh (-e OSMO_HUB_IP, --hub-ip).
if [ -z "$HUB_IP" ]; then
    for _i in $(seq 1 "$N_OPS"); do
        HUB_IP="$(inside "$_i" 'printenv OSMO_HUB_IP' | tr -d '\r' | tail -1)"
        [ -n "$HUB_IP" ] && break
    done
fi

# 5a - where does the local ASP point?
for i in $(seq 1 "$N_OPS"); do
    cfg="$(inside "$i" 'cat /etc/osmocom/osmo-stp.cfg')"
    if [ -z "$cfg" ]; then skip "op${i} : osmo-stp.cfg illisible"; continue; fi
```

```
    remote="$(echo "$cfg" | awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}')"
    shut="$(echo "$cfg" | awk '/asp asp-to-inter/{f=1} f&&/shutdown/{print $0; exit}')"
    pc="$(echo "$cfg" | awk '/^ point-code/{print $2; exit}')"
    if [ -n "$HUB_IP" ] && [ "$remote" = "$HUB_IP" ]; then
        if echo "$shut" | grep -q 'no shutdown'; then
            ok "op${i} : ASP → hub inter-STP ${remote} (actif) - le SS7 traverse le WAN, PC ${pc}"
        else
            fail "op${i} : ASP → hub ${remote} mais ${BOLD}shutdown${NC} - aucun SS7 ne passera"
        fi
        # The link is M3UA over SCTP: "nc -z" (TCP) reports "Connection
        # refused" on a perfectly live hub. The proof is an ESTABLISHED SCTP
        # association seen from the node itself.
        assoc="$(inside "$i" "ss -an --sctp" | grep -E "ESTAB.*[[:space:]]${remote//./\\.}:2908([[:space:]]|\$)"
        ↪  || true)"
        if [ -n "$assoc" ]; then
            ok "op${i} : association SCTP etablie vers le hub ${remote}:2908"
        elif inside "$i" "ss -an --sctp" | grep -q .; then
            fail "op${i} : aucune association SCTP vers ${remote}:2908 - l'ASP est DOWN"
            echo -e "     ${CYAN}Sur le noeud inter-STP : ./launch/start-interstp.sh --status${NC}"
        else
            skip "op${i} : ss --sctp indisponible - lien vers ${remote}:2908 non verifiable"
        fi
    else
        case "$remote" in
            172.2[0-9].*|127.*)
                ok "op${i} : ASP inter-STP → ${remote} (docker local) - le SS7 reste dans le noeud"
                [ "${#REMOTES[@]}" -ge 1 ] && info "pas de hub commun : voix et SMS traversent, le SS7 non" ;;
            "") skip "op${i} : pas d'ASP inter-STP declare" ;;
           *)  warn "op${i} : ASP inter-STP → ${remote} - M3UA sur un lien non maitrise ? (pas de chiffrement SCTP
            ↪  ici)" ;;
        esac
    fi
done

# 5b - does the point code carry the node number?
# The local plan (1.<op>.<role>) is computed identically everywhere: three
# nodes attached to the same hub would all present 1.1.2. The WAN plan
# (1.<node><op>.<role>) removes the ambiguity. Check which one is in place.
if [ "${#REMOTES[@]}" -ge 1 ]; then
    pc_local="$(inside 1 'cat /etc/osmocom/osmo-stp.cfg' | awk '/^ point-code/{print $2; exit}')"
    expected="1.${WAN_NODE_ID}1.2"
    if [ "$pc_local" = "$expected" ]; then
        ok "point code ${pc_local} : le numero de noeud (${WAN_NODE_ID}) y est encode - pas de collision au hub"
    elif [ -n "$HUB_IP" ]; then
        fail "point code ${pc_local} attendu ${expected} : ce noeud entrerait en collision avec ses pairs sur le
        ↪  hub"
        echo -e "     ${CYAN}Reconstruire l'image : ./build-iso.sh --role operator --node ${WAN_NODE_ID}${NC}"
    else
        warn "point code ${pc_local} : plan local, identique sur tous les noeuds"
        echo -e "     ${CYAN}Sans effet tant qu'il n'y a pas de hub commun. Pour du SS7 inter-noeud :${NC}"
        echo -e "     ${CYAN}./build-iso.sh --role interstp   puis   --role operator --node N${NC}"
    fi
fi

# 5c - is an M3UA port publicly exposed?
LOCAL_IP="$(wan_local_ip)"
for port in 2905 2908; do
    if command -v ss >/dev/null 2>&1; then
        # M3UA runs over SCTP here (osmo-stp.cfg: "listen m3ua ..."), not
        # TCP: "ss -lntp" alone would see nothing. Check both transports.
        listen="$( { ss -lntp 2>/dev/null; ss -ln --sctp 2>/dev/null; } | grep ":${port} ")"
        if [ -z "$listen" ]; then
            ok "M3UA ${port} : aucun ecouteur sur l'hote"
        elif echo "$listen" | grep -qE '0\.0\.0\.0:'"${port}"'|\[::\]:'"${port}"; then
            warn "M3UA ${port} ecoute sur 0.0.0.0 - expose si le firewall ne le ferme pas"
```

```
            echo -e "     ${CYAN}iptables -A INPUT -p sctp --dport ${port} -j DROP${NC} (ou -p tcp selon le
             ↪  transport)"
        else
            ok "M3UA ${port} : ecoute restreinte ($(echo "$listen" | awk '{print $4}' | tr '\n' ' '))"
        fi
    else skip "ss absent : ecouteurs M3UA non verifies"; fi
done

# 5d - internal SS7 still healthy? (delegated to the existing check)
if [ -x "$SCRIPT_DIR/checks/ss7_check.sh" ] && [ "$MODE" = docker ]; then
    echo ""
    echo -e "  ${CYAN}SS7 interne : checks/ss7_check.sh --quick${NC}"
else
    skip "SS7 interne : checks/ss7_check.sh (docker uniquement)"
fi

# ── 6. iptables ──────────────────────────────────────────────────────────
banner "6. iptables - chaine WAN"
if iptables -t nat -L OSMO_WAN_MESH -n >/dev/null 2>&1; then
    n=$(iptables -t nat -S OSMO_WAN_MESH 2>/dev/null | grep -c '^-A')
    ok "chaine OSMO_WAN_MESH presente (${n} regle(s))"
    iptables -t nat -S PREROUTING 2>/dev/null | grep -q 'OSMO_WAN_MESH' \
        && ok "chaine branchee sur PREROUTING" \
        || fail "chaine NON branchee sur PREROUTING - rien n'entre"
else
    if iptables -t nat -L OSMO_WAN_INTEROP -n >/dev/null 2>&1; then
        warn "chaine legacy OSMO_WAN_INTEROP active (2 serveurs, prefixe 66)"
    else
        fail "aucune chaine WAN - relancez network/setup-wan-mesh.sh sans --config-only"
    fi
fi

# ── Summary ──────────────────────────────────────────────────────────────
echo ""
printf '─%.0s' $(seq 1 64); echo ""
echo -e "  ${GREEN}${PASS} ok${NC}   ${RED}${FAIL} echec${NC}   ${YELLOW}${WARN} avertissement${NC}   ${SKIP}
 ↪  ignore"
echo ""
[ "$FAIL" -eq 0 ] || exit 1
```

2.29 /opt/GSM/osmo-operator/configs/info.html

16767 octets, 197 lignes → 197 lignes

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>osmo-operator · help</title>
<!-- info.html - bench information page. Served by the dashboard
     (http://127.0.0.1:8080/info.html, iso_modules/60-dashboard.sh) and opened by /usr/local/bin/osmo-info;
     Firefox (snap) cannot read file:///usr/share, so the copy in /usr/share/osmo-operator is only the
     offline fallback. -->
<style>
:root { --bg:#f6f7f9; --card:#fff; --fg:#1d2330; --mute:#667085; --line:#d9dde5; --acc:#2f6fde; --code:#eef1f6;
 ↪  --warn:#a15c00; }
@media (prefers-color-scheme: dark) {
  :root { --bg:#12151b; --card:#1b1f27; --fg:#e3e6eb; --mute:#8d96a5; --line:#2c3340; --acc:#6b9cff;
   ↪  --code:#242a35; --warn:#f0b35a; }
}
* { box-sizing:border-box; }
body { margin:0; background:var(--bg); color:var(--fg); font:15px/1.55 system-ui,sans-serif; }
```

```
header { position:sticky; top:0; z-index:2; background:var(--card); border-bottom:1px solid var(--line);
 ↪  padding:10px 16px; display:flex; flex-wrap:wrap; gap:6px 14px; align-items:center; }
header b { font-size:16px; margin-right:8px; }
header a { color:var(--mute); text-decoration:none; font-size:14px; }
header a:hover { color:var(--acc); }
main { max-width:1100px; margin:auto; padding:16px; }
section { background:var(--card); border:1px solid var(--line); border-radius:10px; padding:16px 20px; margin:0 0
 ↪  16px; }
h1 { font-size:24px; margin:8px 0 4px; }
h2 { font-size:18px; margin:0 0 10px; }
h3 { font-size:15px; margin:16px 0 6px; }
p { margin:6px 0; }
a { color:var(--acc); }
code, pre { font:13px/1.5 ui-monospace,monospace; background:var(--code); border-radius:5px; }
code { padding:1px 5px; }
pre { padding:10px 12px; overflow-x:auto; margin:8px 0; }
table { width:100%; border-collapse:collapse; font-size:14px; }
th, td { text-align:left; vertical-align:top; padding:7px 8px; border-top:1px solid var(--line); }
th { color:var(--mute); font-weight:600; border-top:0; }
td:first-child { white-space:nowrap; }
.lead { color:var(--mute); }
.warn { color:var(--warn); }
.ref { font-size:12px; vertical-align:super; text-decoration:none; }
ol.refs { padding-left:22px; font-size:14px; } ol.refs li { margin:3px 0; }
.grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(250px,1fr)); gap:10px; }
.tile { border:1px solid var(--line); border-radius:8px; padding:10px 12px; }
.tile a { font-weight:600; text-decoration:none; } .tile p { color:var(--mute); font-size:13px; margin:4px 0 0; }
</style>
</head>
<body>
<header>
  <b>osmo-operator · help</b>
  <a href="#modes">Modes</a><a href="#modules">Modules</a><a href="#notebooks">Notebooks</a>
  <a href="#local">On this machine</a><a href="#sites">Sites</a><a href="#repos">Repositories</a><a
   ↪  href="#refs">References</a>
</header>
<main>

<h1>Running the bench</h1>
<p class="lead">A complete 2G network with no radio hardware: Osmocom core, an emulated Calypso phone (QEMU for
 ↪  the ARM,
<code>c54x_exe</code> for the TI DSP and its mask ROM), plus a 4G cell, a SIP softphone and a postmarketOS phone.
Everything lives in <code>/opt/GSM/osmo-operator</code><a class="ref" href="#r1">[1]</a>.
Nothing telecom starts at boot: starting a network is your gesture<a class="ref" href="#r12">[12]</a>.</p>

<section id="modes">
<h2>Modes: one command per bench</h2>
<p>All modes go through <code>./start.sh</code> (<code>start-native.sh</code> is the same script, kept as a link).
Add <code>--regen</code> to rebuild <code>/etc/osmocom</code> from the templates, <code>--no-attach</code> to
 ↪  stay out of tmux.
Full option list: <code>./start.sh --help</code><a class="ref" href="#r2">[2]</a>.</p>
<table>
<tr><th>Command</th><th>What runs</th><th>State</th></tr>
<tr><td><code>./start.sh --bts-classique --ms</code></td>
<td><b>The default (red phone icon).</b> Classic BTS: <code>osmo-bts-trx</code> talks to
 ↪  <code>pont/pont_dsp.py</code>; the emulated
phone runs the osmocom-bb <code>layer1</code> firmware and <code>mobile</code>, on the real DSP ROM<a class="ref"
 ↪  href="#r3">[3]</a>.</td>
<td>LU, SMS, calls</td></tr>
<tr><td><code>./start.sh --bts-classique --ms-magnetite</code></td>
<td>Same network, but the phone is a Motorola C139 running the <b>real TI firmware</b> (FreeCalypso Magnetite):
 ↪  its own
L2/L3 and UI, colour screen in <code>tools/osmo-c123-gtk.py --modele c139</code>, traces by
 ↪  <code>rvinterf</code><a class="ref" href="#r4">[4]</a>.
Add <code>--gapk</code> for voice on the host.</td>
<td>LU, SMS, calls in A5/0: <code>ENCRYPTION="a5 0" ./start.sh --regen …</code></td></tr>
```

```
<tr><td><code>./start.sh --bts --ms --ts 2</code></td>
<td>The Calypso of QEMU <b>is the BTS radio</b> (firmware <code>trx</code> + host <code>transceiver</code>); a
 ↪  second Calypso is the
phone and camps on it over the air (<code>pont/air.py</code>). <code>--ts 2</code>: one Calypso per timeslot (TS0
 ↪  CCCH/SDCCH, TS1 TCH/F).</td>
<td class="warn">camps, LU; no call yet (2nd TRX not frame-aligned)</td></tr>
<tr><td><code>./start.sh --bts --ms --ts 2/1</code></td><td>TS0 and TS1 served by a single Calypso TRX (also the
 ↪  default of <code>--bts</code>).</td><td>calls (audible on 20/10)</td></tr>
<tr><td><code>./start.sh --bts --ms --ts 1</code></td><td>TS0 only (CCCH + SDCCH/4).</td><td>LU, SMS, no
 ↪  call</td></tr>
<tr><td><code>./start.sh --bts --ms-magnetite</code></td><td>Calypso TRX + the C139 Magnetite over the
 ↪  air.</td><td>experimental</td></tr>
<tr><td><code>./start.sh --grgsm</code></td><td>gr-gsm layer 1 in QEMU + <code>grgsm_exe</code> instead of the
 ↪  DSP<a class="ref" href="#r5">[5]</a>.</td><td>end to end</td></tr>
<tr><td><code>--vocoder-rom</code> / <code>--gapk</code></td><td>Speech codec in the DSP mask ROM (default) or on
 ↪  the host (osmo-gapk / libgsm).</td><td>--gapk keeps real time</td></tr>
<tr><td><code>--sap</code>, <code>--sap=pcsc</code></td><td>SIM over SAP: the card in the PC/SC reader
 ↪  (sysmoISIM) or the soft SIM<a class="ref" href="#r6">[6]</a>.</td><td></td></tr>
<tr><td><code>--wan</code>, <code>--payload-ism</code></td><td>Link several nodes (SS7 hub, WireGuard); SMS text
 ↪  over an ISM radio link<a class="ref" href="#r7">[7]</a>.</td><td></td></tr>
<tr><td><code>banc-rssi</code></td><td>The osmocom-bb <code>rssi</code> app on a 4th Calypso (spectrum, sync,
 ↪  per-timeslot levels), fed by the pont in
<code>--bts-classique</code>; <code>banc-rssi --stop</code>. Screen: <code>c123-3</code>.</td><td>DL OK, UL view
 ↪  in progress</td></tr>
<tr><td><code>./start.sh --stop</code> · <code>--status</code></td><td>Stop / query the whole
 ↪  stack.</td><td></td></tr>
</table>
<h3>Safeguards</h3>
<p>BTS#0 runs under a watchdog (<code>BTS_GARDIEN=1</code>, <code>qosmo/run_modules/60-bts.sh</code>): if
 ↪  <code>osmo-bts-trx</code> exits on
“PC clock skew too high” it is restarted after 3 s. The pont caps the pause it asks of the BTS
(<code>PONT_CLCK_PAUSE_PLAFOND=38</code>, <code>pont/trx.py</code>). Do not compile during a run: a loaded host
 ↪  still trips the clock.</p>
</section>

<section id="modules">
<h2>Modules: icon, launcher, CLI</h2>
<table>
<tr><th>Dock icon</th><th>CLI</th><th>Code</th></tr>
<tr><td>Red phone: the GSM bench</td><td><code>launch.sh --bts-classique --ms</code> · <code>systemctl start
 ↪  osmo-banc</code></td><td><code>launch.sh</code>, <code>services/osmo-banc.service</code></td></tr>
<tr><td>Antenna: SS7 multi-operator</td><td><code>systemctl start osmo-multi</code> · <code>launch/start-multi.sh
 ↪  --status</code></td><td>operators 2 and 3 in Docker, inter-STP hub<a class="ref" href="#r8">[8]</a></td></tr>
<tr><td>4G</td><td><code>osmo-lte toggle | status</code> · <code>osmo-epc
 ↪  start</code></td><td><code>tools/osmo-lte.sh</code>, Open5GS + srsRAN<a class="ref"
 ↪  href="#r9">[9]</a></td></tr>
<tr><td>Supplements</td><td><code>sudo ./addition.sh --multi | --opencl | --claude | --extras | --sim</code><br>
<code>--gnuarm | --magnetite | --trx | --fernvale | --simcom |
 ↪  --firmwares</code></td><td><code>addition.sh</code>, <code>tools/osmo-firmwares-install.sh</code></td></tr>
<tr><td>postmarketOS phone</td><td><code>osmo-pmos up | stop |
 ↪  status</code></td><td><code>tools/osmo-pmos.sh</code></td></tr>
<tr><td>Linphone, Wireshark (bench)</td><td><code>linphone</code> · <code>osmo-wireshark-root</code> (GSMTAP
 ↪  udp/4729)</td><td><code>tools/osmo-wireshark-root.sh</code></td></tr>
<tr><td>Update</td><td><code>sudo ./update.sh</code></td><td>git pull of the embedded trees</td></tr>
<tr><td>Tutorial</td><td><code>osmo-tutorial</code></td><td><a href="tutorial.html">tutorial.html</a></td></tr>
<tr><td>? (this page)</td><td><code>osmo-info</code></td><td><code>configs/info.html</code></td></tr>
</table>
<h3>Day to day</h3>
<pre>sudo tmux attach -t calypso          # the bench consoles
telnet 127.0.0.1 4254               # MSC VTY: show subscriber cache
telnet 127.0.0.1 4242               # BSC VTY
telnet 127.0.0.1 4247               # mobile VTY (osmocom phone; not with Magnetite)
./run.sh --logs   (in /opt/GSM/c54x_exe)   # DSP bench logs: /tmp/c54x-pont/*.log</pre>
<p>Settings: <code>globals.conf</code> (MCC, MNC, ARFCN, A5…) or the settings page below; configs are generated
 ↪  into <code>/etc/osmocom</code> by <code>--regen</code><a class="ref" href="#r10">[10]</a>.</p>
</section>
```

```
<section id="notebooks">
<h2>Jupyter notebooks</h2>
<div class="grid">
<div class="tile"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/installer-osmo-jusquau-
 ↪  dsp.ipynb">installer-osmo-jusquau-dsp.ipynb</a>
<p>From a fresh Ubuntu 24.04 to a call on the DSP, one cell per step, with checks; optional Magnetite build.
<a href="https://nbviewer.org/github/bbaranoff/osmo-operator/blob/main/wiki/installer-osmo-jusquau-dsp.ipynb">
 ↪  nbviewer</a></p></div>
<div class="tile"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/lancer-pas-a-pas.ipynb">
 ↪  lancer-pas-a-pas.ipynb</a>
<p>The bench started process by process: HLR, MGW, MSC, BSC, BTS, QEMU, osmocon, mobile, pont; classic BTS, rssi.
<a href="https://nbviewer.org/github/bbaranoff/osmo-operator/blob/main/wiki/lancer-pas-a-pas.ipynb">nbviewer</a
 ↪  ></p></div>
<div class="tile"><a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/README-notebook.md">README-notebook</a>
<p>How to run them: disposable container, fresh VM, or unattended with <code>nbconvert</code><a class="ref"
 ↪  href="#r11">[11]</a>.</p></div>
</div>
<pre># locally (the notebooks are in /opt/GSM/osmo-operator/wiki)
python3 -m venv ~/.venv-jupyter && ~/.venv-jupyter/bin/pip install jupyterlab
~/.venv-jupyter/bin/jupyter lab /opt/GSM/osmo-operator/wiki</pre>
</section>

<section id="local">
<h2>On this machine</h2>
<div class="grid">
<div class="tile"><a href="http://127.0.0.1:8080/">Dashboard</a><p>http://127.0.0.1:8080 · subscribers, VTY,
 ↪  calls, logs (osmo-egprs-web)</p></div>
<div class="tile"><a href="http://127.0.0.1:8090/">Settings</a><p>http://127.0.0.1:8090 · globals.conf, Osmocom
 ↪  configs, Calypso instances (osmo-reglages)</p></div>
<div class="tile"><a href="tutorial.html">Tutorial</a><p>The quick start, served by the dashboard</p></div>
<div class="tile"><a href="https://github.com/bbaranoff/osmo-operator/wiki">Wiki</a><p>The detailed reference,
 ↪  page by page<a class="ref" href="#r13">[13]</a></p></div>
</div>
</section>

<section id="sites">
<h2>Sites</h2>
<div class="grid">
<div class="tile"><a href="https://bbaranoff.github.io">bbaranoff.github.io</a><p>Bastien Baranoff · resources,
 ↪  articles and the related projects</p></div>
<div class="tile"><a href="https://pl4y.store">pl4y.store</a><p>pl4y (<a
 ↪  href="https://github.com/bbaranoff/pl4y">github.com/bbaranoff/pl4y</a>)</p></div>
<div class="tile"><a href="https://github.com/bbaranoff">github.com/bbaranoff</a><p>All the
 ↪  repositories</p></div>
<div class="tile"><a href="https://github.com/bbaranoff/osmo-operator/releases/latest">Latest release</a><p>The
 ↪  desktop ISO, in parts, with SHA256SUMS</p></div>
</div>
</section>

<section id="repos">
<h2>Repositories</h2>
<table>
<tr><th>Repository</th><th>Role in the bench</th></tr>
<tr><td><a href="https://github.com/bbaranoff/osmo-operator">osmo-operator</a></td><td>This bench: launchers,
 ↪  configs, pont, ISO, supplements, docs</td></tr>
<tr><td><a href="https://github.com/bbaranoff/qosmo">qosmo</a></td><td>QEMU with the Calypso machine (ARM, TPU,
 ↪  TSP, BSP, ST7558/C139 screens), run modules</td></tr>
<tr><td><a href="https://github.com/bbaranoff/c54x_exe">c54x_exe</a></td><td>The TI C54x DSP outside QEMU, on the
 ↪  mask ROM; <code>run.sh</code> drives the instances</td></tr>
<tr><td><a href="https://github.com/bbaranoff/grgsm_exe">grgsm_exe</a></td><td>The gr-gsm layer 1
 ↪  bridge</td></tr>
<tr><td><a href="https://github.com/bbaranoff/firmware">firmware</a></td><td>Prebuilt osmocom-bb Calypso
 ↪  firmware</td></tr>
```

```
<tr><td><a href="https://github.com/bbaranoff/osmocom-bb-trx">osmocom-bb-trx</a></td><td>osmocom-bb jolly/testing
 ↪  with TX (Docker image <code>ghcr.io/bbaranoff/osmocom-bb-trx:main</code>)</td></tr>
<tr><td><a href="https://github.com/bbaranoff/osmo-egprs-web">osmo-egprs-web</a> · <a
 ↪  href="https://github.com/bbaranoff/osmo_egprs">osmo_egprs</a></td><td>The dashboard; the EGPRS
 ↪  workflow</td></tr>
<tr><td><a href="https://github.com/bbaranoff/fernly">fernly</a> · <a
 ↪  href="https://github.com/bbaranoff/fernvale-nuttx">fernvale-nuttx</a> · <a
 ↪  href="https://github.com/bbaranoff/osmocom-bb-mtk-layer1">osmocom-bb-mtk-layer1</a></td><td>MediaTek MT6260
 ↪  (Fernvale): reversing OS, NuttX, osmocom-bb layer 1 (<code>addition.sh --fernvale</code>)</td></tr>
<tr><td><a href="https://github.com/bbaranoff/simcom">simcom</a></td><td>SIMCom module documentation and SDKs
 ↪  (<code>addition.sh --simcom</code>)</td></tr>
<tr><td><a href="https://github.com/bbaranoff/deka">deka</a> · <a
 ↪  href="https://github.com/bbaranoff/a51_tools">a51_tools</a></td><td>A5/1 tools (OpenCL supplement)</td></tr>
<tr><td>Upstream</td><td><a href="https://gitea.osmocom.org/">Osmocom</a> · <a
 ↪  href="https://www.freecalypso.org/">FreeCalypso</a> (<a
 ↪  href="https://hg.themyscira.org/fc-magnetite">fc-magnetite</a>) ·
<a href="https://github.com/axilirator/gnu-arm-installer">gnu-arm-installer</a> (arm-elf-gcc 4.5.2)</td></tr>
</table>
</section>

<section id="refs">
<h2>References</h2>
<ol class="refs">
<li id="r1"><a href="https://github.com/bbaranoff/osmo-operator#readme">README</a> — start here, by goal (ISO,
 ↪  Docker, native, ARM, DSP, 4G, SS7)</li>
<li id="r2"><a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/Start-native.md">wiki/Start-native</a> — the
 ↪  launcher step by step, options manual</li>
<li id="r3"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-dsp.md">doc/README-dsp</a> —
 ↪  the real C54x DSP on the mask ROM</li>
<li id="r4"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/magnetite.md">doc/magnetite</a> —
 ↪  FreeCalypso Magnetite in qosmo: build, flash image, state</li>
<li id="r5"><a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-grgsm.md">doc/README-grgsm</a> — the
 ↪  gr-gsm layer 1</li>
<li id="r6"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/SIM.md">wiki/SIM</a> — Omnikey
 ↪  reader, sysmoISIM, soft SIM over SAP</li>
<li id="r7"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/WAN.md">wiki/WAN</a> · <a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/Payload-ISM.md">wiki/Payload-ISM</a> —
 ↪  linking nodes</li>
<li id="r8"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-ss7.md">doc/README-ss7</a> —
 ↪  multi-operator, SS7 interconnect</li>
<li id="r9"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-lte.md">doc/README-lte</a> —
 ↪  4G, CSFB, the postmarketOS phone</li>
<li id="r10"><a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/Environnement.md">wiki/Environnement</a> —
 ↪  which variable wins, what reaches QEMU</li>
<li id="r11"><a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/README-notebook.md">wiki/README-notebook</a>
 ↪  — running the notebooks</li>
<li id="r12"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/services/80-osmo-operator.preset">
 ↪  services/80-osmo-operator.preset</a> — the boot policy
 ↪  (<code>tools/osmo-telecom-boot.sh</code>)</li>
<li id="r13"><a href="https://github.com/bbaranoff/osmo-operator/blob/main/wiki/Home.md">wiki/Home</a> · <a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-usage.md">doc/README-usage</a> · <a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-architecture.md">doc/README-
 ↪  architecture</a> · <a
 ↪  href="https://github.com/bbaranoff/osmo-operator/blob/main/doc/README-tests.md">doc/README-tests</a></li>
</ol>
</section>

</main>
</body>
</html>
```

2.30 /opt/GSM/osmo-operator/configs/conky/dino.html

217818 octets, 1343 lignes → 1343 lignes

```
<!doctype html><html><head><meta charset=utf8><meta name=viewport content="width=device-width,initial-scale=1"><
↪ style>:root{color-scheme:light}body{margin:0;padding:0;font:14px
↪ -apple-system,BlinkMacSystemFont,sans-serif;background:#faf9f5;color:#141413}img{max-width:100%}[hidden]
↪ :not([hidden=until-found]){display:none!important}</style></head><body>
<!-- dino.html - "Coureur d'ombres": Chrome-dino style runner on a canvas, with Sun and
     Moon orbiting the dino and shadows computed from their positions.
     Usage: opened by osmo-dino-play (#full in the URL starts fullscreen) or embedded in
     the desktop widget tools/osmo-dino.py (window.OSMO_HOST = "widget" or "app").
     Keys: arrows/A/D move, space/up/W jump, Enter restart, M mute, F fullscreen. -->
<title>Cavalier d'ombres</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Rye&
↪ family=Barlow+Condensed:wght@400;500;600&family=IBM+Plex+Mono:wght@400;500&display=swap">
<style>
  :root{
    --ink:#131018;        /* ink night */
    --ink-2:#1c1822;
    --bone:#efe6d2;       /* bone */
    --bone-dim:#a99f8d;
    --sun:#e0653a;        /* setting sun */
    --moon:#a9b7de;       /* moonlight */
    --cactus:#4b7a4a;
    --dust:#c8a87a;
    --line:rgba(239,230,210,.14);
  }
  *{box-sizing:border-box}
  body{
    margin:0; background:var(--ink); color:var(--bone);
    font-family:"Barlow Condensed",-apple-system,"Segoe UI",sans-serif;
    min-height:100vh; display:flex; flex-direction:column; align-items:center;
    justify-content:center; gap:14px; padding:18px 14px 26px;
    background-image:radial-gradient(120% 80% at 50% -10%, #241d2b 0%, var(--ink) 62%);
  }

  header{width:min(980px,100%); display:flex; align-items:baseline; justify-content:space-between; gap:16px}
  h1{
    font-family:Rye,"Barlow Condensed",serif; font-weight:400; margin:0;
    font-size:clamp(22px,3.4vw,34px); letter-spacing:.01em; color:var(--bone);
    text-wrap:balance;
  }
  h1 em{font-style:normal; color:var(--sun)}
  header p{margin:0; font-size:14px; color:var(--bone-dim); letter-spacing:.04em; text-align:right;
↪ max-width:42ch}

  .frame{
    position:relative; width:min(980px,100%); aspect-ratio:8/5;
    border:1px solid var(--line); border-radius:3px; overflow:hidden;
    background:#0d0b12; box-shadow:0 24px 60px -30px #000, inset 0 0 0 1px rgba(0,0,0,.5);
    touch-action:none; cursor:crosshair;
  }
  canvas{display:block; width:100%; height:100%}

  /* ---- HUD ---- */
  .hud{position:absolute; inset:0; pointer-events:none; font-variant-numeric:tabular-nums}
  .top{
    position:absolute; top:0; left:0; right:0; display:flex; align-items:center;
    justify-content:space-between; gap:12px; padding:9px 14px;
    background:linear-gradient(#000000a8,#0000);
  }
  .dist{display:flex; align-items:baseline; gap:6px}
  .dist b{font-size:26px; font-weight:600; line-height:1; letter-spacing:.01em}
```

```
.dist span{font-size:12px; color:var(--bone-dim); letter-spacing:.16em; text-transform:uppercase}
.right{display:flex; align-items:center; gap:16px}
.sky-read{text-align:right; line-height:1.15}
.sky-read .ph{font-size:14px; letter-spacing:.03em}
.sky-read .dy{font-size:11px; color:var(--bone-dim); letter-spacing:.14em; text-transform:uppercase}
.lives{display:flex; gap:5px}
.lives i{
  display:block; width:15px; height:11px; background:var(--bone);
  clip-path:polygon(0 100%,100% 100%,84% 74%,74% 0,26% 0,16% 74%);
  transition:opacity .18s;
}
.lives i.off{opacity:.16}

.snd{
  position:absolute; right:12px; bottom:12px; z-index:3;
  font:500 13px/1 "IBM Plex Mono",monospace; color:var(--bone);
  background:#0a0810a6; border:1px solid var(--line); border-radius:2px;
  padding:6px 9px; cursor:pointer; letter-spacing:.08em;
}
.snd:hover{background:#0a0810e0}
.snd:focus-visible{outline:2px solid var(--sun); outline-offset:2px}
.snd[aria-pressed="true"]{color:var(--bone-dim)}
/* Fullscreen button, left of the sound button. The page starts windowed; in
   fullscreen the frame fills the screen keeping its 8/5 aspect (letterboxed
   if needed), and the same button returns to the window. */
.fsb{ right:58px }
body.full{ padding:0; gap:0; background:#000; background-image:none }
body.full header{ display:none }
body.full .frame{ width:min(100vw, 160vh); height:min(100vh, 62.5vw); border:0; border-radius:0;
↪  box-shadow:none }

.card{
  position:absolute; left:50%; top:34%; transform:translate(-50%,-50%);
  display:grid; justify-items:center; gap:4px; text-align:center; pointer-events:none;
  text-shadow:0 2px 18px #000, 0 0 30px #000;
}
.card[hidden]{display:none}
.card b{font-family:Rye,serif; font-weight:400; font-size:clamp(19px,2.8vw,27px); color:#f6efdd}
.card span{font-size:12px; letter-spacing:.18em; text-transform:uppercase; color:var(--moon)}

/* ---- overlays ---- */
.veil{
  position:absolute; inset:0; display:grid; place-content:center; text-align:center; gap:10px;
  background:radial-gradient(60% 60% at 50% 50%, #0b0910cc, #0b0910f2); padding:24px;
}
.veil[hidden]{display:none}
.veil h2{
  font-family:Rye,serif; font-weight:400; margin:0; font-size:clamp(20px,3vw,30px); color:var(--bone);
}
.veil p{margin:0; font-size:15px; color:#cfc6b4; max-width:46ch; line-height:1.5}
.veil .keys{display:flex; gap:8px; justify-content:center; margin-top:4px; flex-wrap:wrap}
.veil kbd{
  font-family:"IBM Plex Mono",monospace; font-size:12px; padding:4px 9px; color:var(--bone);
  border:1px solid var(--line); background:#ffffff0e; border-radius:2px;
}
.veil .score{font-family:"IBM Plex Mono",monospace; font-size:13px; color:var(--bone-dim); letter-spacing:.06em}
.veil .score b{color:var(--sun); font-weight:500}

footer{
  width:min(980px,100%); display:flex; justify-content:space-between; gap:18px;
  font-size:12.5px; color:var(--bone-dim); letter-spacing:.03em; flex-wrap:wrap;
}
footer code{font-family:"IBM Plex Mono",monospace; font-size:11.5px; color:#bcb4a3}
@media (max-width:620px){
  header{flex-direction:column; gap:4px} header p{text-align:left}
  .dist b{font-size:20px}
```

```
  }
</style>

<header>
  <h1>Coureur d'<em>ombres</em></h1>
  <p>Le dino court vers le levant : le Soleil se lève droit devant lui, monte, et sort du cadre par le haut. La
   ↪  Lune fait le même chemin, un cinquième plus vite. Les ombres sont calculées, pas dessinées — même celle du
   ↪  saut.</p>
</header>

<div class="frame" id="frame">
  <canvas id="cv" width="960" height="600"></canvas>
  <div class="hud">
    <div class="top">
      <div class="dist"><b id="dist">0</b><span>mètres</span></div>
      <div class="right">
        <div class="sky-read">
          <div class="ph" id="phase">nouvelle lune</div>
          <div class="dy" id="day">jour 1 · 06 h 00</div>
        </div>
        <div class="lives" id="lives"><i></i><i></i><i></i></div>
      </div>
    </div>
  </div>

  <button class="snd" id="snd" type="button" aria-pressed="false" title="son (M)">♪</button>
  <button class="snd fsb" id="fsb" type="button" aria-pressed="false" title="plein écran (F)">□</button>

  <div class="card" id="card" hidden>
    <b id="cardLvl"></b>
    <span id="cardSub"></span>
  </div>

  <div class="veil" id="start">
    <h2>Il va tout droit</h2>
    <p>Tant que personne ne touche aux commandes, la piste reste vide. Prends la main et le désert se remplit. La
     ↪  Lune rattrape le Soleil tous les cinq jours : l'éclipse est un spectacle. Il n'y a que des mètres à
     ↪  faire.</p>
    <div class="keys"><kbd>←</kbd><kbd>→</kbd><kbd>espace : saut</kbd><kbd>clic / glisser</kbd></div>
  </div>

  <div class="veil" id="over" hidden>
    <h2 id="overTitle">Dans les épines</h2>
    <p class="score">parcouru <b id="finalDist">0</b> m · record <b id="bestDist">0</b> m</p>
    <div class="keys"><kbd>Entrée</kbd><kbd>clic</kbd></div>
  </div>
</div>

<footer>
  <span>Soleil et Lune tournent autour du dino et se lèvent devant lui ; la Lune va 1,2 fois plus vite que le
   ↪  Soleil, et le rattrape tous les cinq jours. L'année dure onze jours : neige, orages, aurores, arcs-en-ciel,
   ↪  étoiles filantes — et la dernière nuit, le père Noël passe.</span>
  <code>ombre : S = Q − l·(Qz □ lz)</code>
</footer>

<script>
(() => {
  "use strict";
  const cv = document.getElementById("cv"), ctx = cv.getContext("2d", {alpha:false});
  const W = 960, H = 600, FOV = 52 * Math.PI/180, FOC = (W/2)/Math.tan(FOV/2);

  // ---------- small vector algebra ----------
  const sub=(a,b)=>[a[0]-b[0],a[1]-b[1],a[2]-b[2]];
  const add=(a,b)=>[a[0]+b[0],a[1]+b[1],a[2]+b[2]];
  const mul=(a,s)=>[a[0]*s,a[1]*s,a[2]*s];
  const dot=(a,b)=>a[0]*b[0]+a[1]*b[1]+a[2]*b[2];
```

```
const cross=(a,b)=>[a[1]*b[2]-a[2]*b[1], a[2]*b[0]-a[0]*b[2], a[0]*b[1]-a[1]*b[0]];
const norm=a=>{const l=Math.hypot(a[0],a[1],a[2])||1; return [a[0]/l,a[1]/l,a[2]/l];};
const clamp=(v,a,b)=>v<a?a:v>b?b:v;
const lerp=(a,b,t)=>a+(b-a)*t;
const mix=(a,b,t)=>{t=clamp(t,0,1); return [lerp(a[0],b[0],t),lerp(a[1],b[1],t),lerp(a[2],b[2],t)];};
const rgb=c=>`rgb(${c[0]|0},${c[1]|0},${c[2]|0})`;

// ---------- camera ----------
let EYE=[0,-9.2,2.6], RGT=[1,0,0], UPV=[0,0,1], FWD=[0,1,0], HORIZON=300;
function setCam(camX, lookX){
  EYE=[camX,-9.2,2.6];
  const f=norm(sub([lookX,0,1.5],EYE));
  const r=norm(cross(f,[0,0,1]));
  FWD=f; RGT=r; UPV=cross(r,f);
  HORIZON = proj([camX,8000,0])[1];
}
function proj(P){
  const d=sub(P,EYE), cz=dot(d,FWD);
  if(cz<=0.06) return null;
  return [W/2 + FOC*dot(d,RGT)/cz, H/2 - FOC*dot(d,UPV)/cz, cz];
}

// ---------- oriented boxes ----------
function box(c,h,col,ax,ay,az){
  return {c, h, col, ax:ax||[1,0,0], ay:ay||[0,1,0], az:az||[0,0,1]};
}
function verts(b){
  const out=[];
  for(const sx of [-1,1]) for(const sy of [-1,1]) for(const sz of [-1,1])
    out.push([
      b.c[0]+b.ax[0]*b.h[0]*sx+b.ay[0]*b.h[1]*sy+b.az[0]*b.h[2]*sz,
      b.c[1]+b.ax[1]*b.h[0]*sx+b.ay[1]*b.h[1]*sy+b.az[1]*b.h[2]*sz,
      b.c[2]+b.ax[2]*b.h[0]*sx+b.ay[2]*b.h[1]*sy+b.az[2]*b.h[2]*sz]);
  return out;
}
function faces(b,out){
  const A=[b.ax,b.ay,b.az];
  for(let i=0;i<3;i++){
    const j=(i+1)%3, k=(i+2)%3;
    for(const s of [-1,1]){
      const n=mul(A[i],s), cf=add(b.c,mul(n,b.h[i]));
      const u=mul(A[j],b.h[j]), v=mul(A[k],b.h[k]);
      out.push({n, col:b.col, q:[sub(sub(cf,u),v), sub(add(cf,u),v), add(add(cf,u),v), add(sub(cf,u),v)]});
    }
  }
}
// convex hull (monotone chain); constant winding so the union fills cleanly
function hull(pts){
  pts=pts.slice().sort((a,b)=>a[0]-b[0]||a[1]-b[1]);
  if(pts.length<3) return pts;
  const half=P=>{const o=[]; for(const p of P){
    while(o.length>=2 && (o[o.length-1][0]-o[o.length-2][0])*(p[1]-o[o.length-2][1])
                       -(o[o.length-1][1]-o[o.length-2][1])*(p[0]-o[o.length-2][0])<=0) o.pop();
    o.push(p);} return o;};
  const lo=half(pts), hi=half(pts.slice().reverse());
  return lo.slice(0,-1).concat(hi.slice(0,-1));
}

// ---------- the model ----------
const DINO=[92,92,92], DINO_D=[72,72,72], DINO_T=[62,62,62], EYE_W=[248,248,248];
const C_CAC=[75,122,74], C_CAC_D=[58,98,58];

// The Chrome dino seen from behind, as boxes: body, neck, skull, raised tail,
// two tiny arms, two legs.  z = jump height.
function dinoBoxes(t, px, lean, z){
  const B=[], ph=t*Math.PI*2, air = z>0.02;
```

```
  const Z = z + (air ? 0 : 0.045*Math.sin(2*ph));
  const rl = lean*0.18;
  const P=(x,y,zz)=>[px + x*Math.cos(rl), y, zz + Z + x*Math.sin(rl)];
  B.push(box(P(0,-0.02,1.16),[0.40,0.60,0.42],DINO));            // body
  B.push(box(P(0,0.58,1.44),[0.33,0.24,0.34],DINO));             // chest
  B.push(box(P(0,0.74,1.84),[0.25,0.20,0.30],DINO));             // neck
  B.push(box(P(0,0.86,2.20),[0.25,0.32,0.21],DINO));             // skull
  B.push(box(P(0,1.14,2.13),[0.21,0.22,0.13],DINO_D));           // snout
  for(const sx of [-1,1]) B.push(box(P(sx*0.215,0.98,2.28),[0.05,0.05,0.05],EYE_W));
  const c1=Math.cos(0.30), s1=Math.sin(0.30);                    // tail, two segments
  B.push(box(P(0,-0.76,1.28),[0.24,0.40,0.25],DINO,[1,0,0],[0,c1,s1],[0,-s1,c1]));
  const c2=Math.cos(0.58), s2=Math.sin(0.58);
  B.push(box(P(0,-1.34,1.52),[0.14,0.42,0.15],DINO_T,[1,0,0],[0,c2,s2],[0,-s2,c2]));
  for(const sx of [-1,1])                                        // tiny arms
    B.push(box(P(sx*0.42,0.50,1.42),[0.07,0.15,0.07],DINO_D,[1,0,0],[0,0.94,0.34],[0,-0.34,0.94]));
  for(let i=0;i<2;i++){                                          // legs
    const sx = i ? 1 : -1;
    const a = air ? (i ? 0.85 : -0.45) : 0.72*Math.sin(2*Math.PI*(t+i*0.5));
    const L=0.92, d=[0,Math.sin(a),-Math.cos(a)];
    const hip=P(sx*0.24,-0.06,1.00);
    B.push(box(add(hip,mul(d,L/2)),[0.135,0.15,L/2],DINO_D,
               [1,0,0],[0,Math.cos(a),Math.sin(a)],[0,-Math.sin(a),Math.cos(a)]));
    B.push(box(add(add(hip,mul(d,L)),[0,0.10,0.06]),[0.13,0.22,0.06],DINO_T));   // foot
  }
  return B;
}
function cactusBoxes(c){
  const B=[], s=c.s, x=c.x, y=c.y;
  B.push(box([x,y,0.95*s],[0.16*s,0.16*s,0.95*s],C_CAC));
  if(c.a & 1){
    B.push(box([x-0.34*s,y,1.05*s],[0.20*s,0.13*s,0.12*s],C_CAC_D));
    B.push(box([x-0.48*s,y,1.05*s+0.42*s],[0.13*s,0.13*s,0.42*s],C_CAC));
  }
  if(c.a & 2){
    B.push(box([x+0.34*s,y,1.22*s],[0.20*s,0.13*s,0.12*s],C_CAC_D));
    B.push(box([x+0.48*s,y,1.22*s+0.34*s],[0.13*s,0.13*s,0.34*s],C_CAC));
  }
  return B;
}

// ---------- static scenery ----------
const RND=(()=>{let s=20260906; return ()=>{s=(s*1664525+1013904223)>>>0; return s/4294967296;};})();
const ROCKS=[], STARS=[];
for(let i=0;i<70;i++) ROCKS.push({x:(RND()*2-1)*34, y0:RND()*140, r:0.12+RND()*0.45});
for(let i=0;i<460;i++) STARS.push({x:RND()*W, y:Math.pow(RND(),0.75)*0.99,
                                   m:0.18+Math.pow(RND(),2.2)*0.82, s:RND()<0.13?2:1,
                                   tw:RND()*6.28});

// ---------- state ----------
const S={
  mode:"attract", t:0, dist:0, speed:9, px:0, tgt:0, camX:0, lean:0,
  lives:3, invuln:0, gallop:0, cacti:[], spawn:2.5, shake:0, z:0, vz:0,
  day0: 0, best: 0, flash:0, totT:0,
  eclipsing:false, cardT:0, cardTitle:"", cardSub:"", sky:null,
  meteors:[], santa:-1, santaSeen:false,
  wx:{cloud:0,rain:0,snow:0,storm:0,aurora:0,day:0}, boltT:0, bolt:null, glare:0, skyT:0
};
try{ S.best = +(localStorage.getItem("cavalier-best")||0) || 0; }catch(e){}

const DAY = 24;                       // seconds per full day
// The dino runs toward sunrise. Sun and Moon orbit it on the same circle and
// only the rising quarter is visible: each body rises from the horizon ahead,
// climbs at constant speed and leaves through the top of the frame (clipped
// by the edge, not faded). The rest of the orbit (descent behind the dino,
// passage below ground) is never seen. Hence the full Moon faces the dino at
// night: being opposite the Sun, it rises ahead while the Sun sets behind.
```

```
//   height in frame = angular speed x time (x K_SKY)
// The Sun makes one turn per day; the Moon runs MOON_K = 1.2 times faster and
// catches up every 1/(MOON_K-1) = 5 days: the model's lunation (not 28 days),
// with an eclipse on the day of conjunction. Only the Sun makes day and
// night; the Moon is invisible by day and fades in, already moving, as the
// sky darkens.
const ECL_ANG = 34*Math.PI/180;       // conjunction angle: low in the rising quarter
const ECL_DAY = 0.25 + 34/360;        // time of day when the Sun is there
const MOON_K = 1.2;                   // Moon angular speed / Sun angular speed
const MOON_PER = 1/(MOON_K-1);        // days between two conjunctions (5)
// At the equator the noon Sun is at the zenith: the arc amplitude is 90 deg.
// A 5 deg declination makes no two days quite alike: the seasons. The model
// year lasts SEASON days; the cold trough falls at the turn of the year
// (day 0 == day 11), summer in the middle.
const SUN_MAX = 90*Math.PI/180, SUN_DEC = 5*Math.PI/180, SEASON = 11;
const yearT  = dayT => ((dayT % SEASON) + SEASON) % SEASON;   // day within the year, [0, SEASON)
const froidAt= dayT => (1 + Math.cos(2*Math.PI*dayT/SEASON))/2; // 1 at the depth of winter
const sunAmp = dayT => SUN_MAX - SUN_DEC*froidAt(dayT);
// Two angles on the same circle, two speeds. The Moon angle is the Sun angle
// plus a lag growing by (MOON_K-1) turn per day, zero at conjunction: at
// ECL_DAY both angles coincide exactly.
const moonAngle = dayT => (dayT-0.25)*2*Math.PI + 2*Math.PI*(dayT-ECL_DAY)*(MOON_K-1);
// Altitude is a triangle wave, not a sine: the body RISES at constant speed
// from the horizon (u = 0) to the zenith (u = pi/2), the only quarter seen,
// ahead of the dino. It then descends behind it.
function arcDir(theta, amp){
  const TAU=2*Math.PI, u=((theta%TAU)+TAU)%TAU;
  const tri = u<=Math.PI ?  (1 - Math.abs(u-Math.PI/2)/(Math.PI/2))
                         : -(1 - Math.abs(u-3*Math.PI/2)/(Math.PI/2));
  const alt = amp*tri;
  const front = u<Math.PI/2 || u>3*Math.PI/2;    // ahead of the dino: the ascent
  return [0, (front?1:-1)*Math.cos(alt), Math.sin(alt)];
}
// Geocentric model: both bodies orbit the dino at a finite distance, so
// shadows are computed from their POSITION, not from a direction: they
// radiate from the ground point below the body.
const R_SUN_W = 70, R_MOON_W = 95;    // metres
const R_SUN_PX = 26, R_MOON_PX = 29;   // nearly the same apparent size; the Moon is slightly wider so totality
 ↪  lasts
// The viewer sees pi/3: the horizon (183 px, altitude 0) is at the bottom of
// the sky, the top of the frame is at 60 deg altitude. The zenith (90 deg) is
// well above the frame: the body exits through the top, whole, two thirds up.
const SKY_FOV = Math.PI/3, HZ_PX = 183;
const K_SKY = HZ_PX/SKY_FOV;          // px per radian of altitude
// The sky opens on today's Moon: real lunar age computed from the reference
// new Moon of 2000-01-06 18:14 UTC with a 29.530589 d synodic month, then
// scaled to the model's lunation.
const SYNODIC = 29.530588853;
function moonAgeToday(){
  const ref = Date.UTC(2000,0,6,18,14)/86400000, now = Date.now()/86400000;
  return ((now-ref) % SYNODIC + SYNODIC) % SYNODIC;      // 0 = new Moon
}
const AGE = moonAgeToday();
S.day0 = ECL_DAY + MOON_PER - (SYNODIC-AGE)*MOON_PER/SYNODIC;   // time left before conjunction
S.sky = null;

// corona streamers: long at the equator, short brushes at the poles
const CORONA=[];
for(let i=0;i<34;i++){
  const a=RND()*Math.PI*2, eq=Math.abs(Math.sin(a));
  CORONA.push({a, l:(0.25+0.75*eq)*(0.55+0.45*RND()), w:0.05+0.09*RND(),
               b:(RND()*2-1)*0.22, o:0.55+0.45*RND()});
}

// ---------- weather: drawn at random each day ----------
const wh = n => { const x=Math.sin(n*127.1+311.7)*43758.5453; return x-Math.floor(x); };
```

```
// Possible weather: clear, cloudy, rain or SNOW (more often in the cold
// season, when the Sun is low), storm with lightning, and AURORAS on one
// clear night out of three.
function dayWeather(n){
  const a=wh(n), b=wh(n+0.37), c=wh(n+0.71), d=wh(n+0.53), e=wh(n+0.89);
  const froid = froidAt(n);                                  // 1 at the depth of the cold season
  const aurora = (a<0.64 && e>0.66) ? 0.45+0.55*b : 0;
  // On the last night of the year it always snows, and the sky stays open
  // enough to see who passes by.
  if(Math.floor(yearT(n)) === SEASON-1) return {cloud:0.38+0.14*b, rain:0, snow:0.45+0.30*c, storm:0, aurora:0};
  if(a<0.34) return {cloud:0.04+0.18*b, rain:0, snow:0, storm:0, aurora};       // clear
  if(a<0.64) return {cloud:0.34+0.42*b, rain:0, snow:0, storm:0, aurora};       // cloudy
  if(a<0.87){                                                                    // precipitation
    if(d < 0.25+0.45*froid) return {cloud:0.62+0.30*b, rain:0, snow:0.35+0.45*c, storm:0, aurora:0};  // snow
    return {cloud:0.62+0.30*b, rain:0.30+0.40*c, snow:0, storm:0, aurora:0};    // rain
  }
  return {cloud:0.82+0.18*b, rain:0.68+0.32*c, snow:0, storm:1, aurora:0};      // storm
}
function weatherAt(dayT){                    // smooth transition at the end of the day
  const n=Math.floor(dayT), f=dayT-n;
  const A=dayWeather(n), B=dayWeather(n+1), t=clamp((f-0.80)/0.20,0,1);
  return {cloud:lerp(A.cloud,B.cloud,t), rain:lerp(A.rain,B.rain,t),
          snow:lerp(A.snow,B.snow,t), storm:lerp(A.storm,B.storm,t),
          aurora:lerp(A.aurora,B.aurora,t), day:n};
}
const CLOUDS=[];
for(let i=0;i<26;i++){
  const puffs=[];
  const np=3+((RND()*4)|0);
  for(let k=0;k<np;k++) puffs.push({dx:(RND()*2-1)*46, dy:(RND()*2-1)*9, r:16+RND()*22});
  CLOUDS.push({x:RND(), alt:0.10+RND()*1.15, sp:0.006+RND()*0.016, sc:0.6+RND()*0.9, puffs});
}
const DROPS=[];
for(let i=0;i<240;i++) DROPS.push({x:RND()*W, y:RND()*H, l:9+RND()*16, v:520+RND()*420});
// Snowflakes: slow, and they drift instead of falling straight.
const FLAKES=[];
for(let i=0;i<220;i++) FLAKES.push({x:RND()*W, y:RND()*H, r:1.2+RND()*2.2, v:38+RND()*52, ph:RND()*7,
 ↪  sw:12+RND()*22});

// ---------- sky (independent of the runner and the camera) ----------
function computeSky(){
  const dayT = S.skyT + S.day0;
  const thS = (dayT - 0.25) * Math.PI*2;           // Earth rotation: 1 turn/day
  const ampS = sunAmp(dayT);
  const lS = arcDir(thS, ampS);
  const altS = Math.asin(clamp(lS[2],-1,1));
  const thM = moonAngle(dayT);                     // Moon angle on the same circle
  const lM = arcDir(thM, ampS);                    // exactly the same circle
  const altM = Math.asin(clamp(lM[2],-1,1));
  // Two bodies at the same altitude, one ahead and one behind, do not meet:
  // they must be on the same side of the sky.
  const sameBranch = (lS[1]>=0) === (lM[1]>=0);
  // The closer the Moon is to the top, the slower the Sun crosses it: the
  // corona only has time to unfold there.
  const slow = clamp((0.5-Math.abs(Math.cos(thM)))/0.5, 0, 1);

  // Phase: the true lunation phase, seeded from today's Moon. It does not
  // depend on altitude: fixed over a whole night, clearly changed the next.
  // The terminator is horizontal: both bodies share one circle in the
  // front/up plane, so the Sun is exactly below or above the Moon.
  const untilNew = ((((ECL_DAY - dayT) % MOON_PER) + MOON_PER) % MOON_PER);
  const lunation = 1 - untilNew/MOON_PER;          // 0 and 1 = new Moon
  const kMoon = (1 - Math.cos(2*Math.PI*lunation))/2;
  const sepAng = altM - altS;
  const elong = 2*Math.PI*lunation;
  // The lit limb points to the Sun. Sun-dino-Moon angle 0: the Moon is
```

```
    // backlit, dark; as it grows the Moon leads on the circle and the Sun stays
    // BEHIND it (lower, or below the horizon at night): limb down. Past 180 deg
    // (full Moon, Sun opposite) the Sun is ahead and higher: limb up, until it
    // catches the Moon from below at the next conjunction.
    const pa = elong < Math.PI ? Math.PI/2 : -Math.PI/2;

    // Eclipse: the Sun can only reach the Moon on the day of conjunction, and
    // only at the top of its course, where it is slowest.
    const sep = (sameBranch && altS>0 && altM>0) ? Math.abs(sepAng)*K_SKY : 1e9;
    const rs=R_SUN_PX, rm=R_MOON_PX;
    let cover = 0;
    if(sep < rs+rm){
      if(sep <= rm-rs) cover = 1;
      else{
        const a1=Math.acos(clamp((sep*sep+rs*rs-rm*rm)/(2*sep*rs),-1,1));
        const a2=Math.acos(clamp((sep*sep+rm*rm-rs*rs)/(2*sep*rm),-1,1));
        cover = clamp((rs*rs*(a1-Math.sin(2*a1)/2)+rm*rm*(a2-Math.sin(2*a2)/2))/(Math.PI*rs*rs),0,1);
      }
    }
    return {dayT, thS, thM, lS, lM, altS, altM, elong, kMoon, cover, pa, slow,
            total: cover>=0.999};
  }
  // sky dome: a body is placed by its direction only, never by the runner
  function skyPt(d, hz){
    const alt = Math.asin(clamp(d[2],-1,1));
    if(d[1] <= 0) return null;                    // behind the dino: never seen
    if(alt < -0.30) return null;                  // set: nothing below the horizon
    const vx = W/2 + FOC*RGT[1]/(FWD[1] || 1e-6);   // vanishing point of both tracks
    return [vx, hz - alt*K_SKY, alt];
  }

  // ---------- sound: 3 bars of the excerpt, looped seamlessly ----------
  const LOOP_B64=
   "SUQzBAAAAAAAI1RTU0UAAAAPAAADTGF2ZjYwLjE2LjEwMAAAAAAAAAAAAAAA//uAAAAAAAAAAAAAA
```

```
t/bRXcfBB2X/1+oFZniWUwIFxCNNEX23QSRMSRlTky++09nECvA+amCzC2yPt4k/9
    ZPGvh5qyQK2Jbw93XkbBgPMHfCt65gzZ3WrdvTnY6nMqtRxQAwtFDpJkFXb2G/Trkm/q/An
    //oY56oGzmtzd77k0KDvgFATd3opAAARoTcQlDxKrr8aJ1MaC3MmJ+9E5qXzKCgObPdShFUMQf/V+r/0f0++e/88sEV2iXIQAAAAeHCo+kR1ABqiWSsb/wzA0GP
ghK86ZA0GDuS9BCm4E7P2twpxeywxxC2OWuIPndV+dDMZUssyGOAxU2OhX8at2pvHccKe/6URyHIo+cPyi+9D7j0Tjzzwstd7/
    QCcOwepCwj6EIQ9fJ9RywlZGleRZ9arm0Z
    /////vPBygLBvF/RR/jmk1DvksiAAAAKhJE5JCMLCGU7bK9TVgOjLdpDUcTjsKgfJ4327Ktki4xduSDqN708BElvJJBhxL6REtso7guxdUwgZZIEyBh6Q1hIU4N
BZVgvvMbs9mw+bSGBTGrNVq7RP5SqDVGiDmp868Qzy/zn9OZc+
    m5Ta7OtgH97zZ7WKGvvBfgoc16wmtItydjACAok4GhAAduH8a9kY0qOVVhQ0XynyG+VmfcJPDbv
    ///cpJy085i3/L/s6bP/EnTUEy+6QgAABTJ2hKRsEFt9FG1m3dicYhtoPSWrECuE8+jcYjY2Z85VzOkN7CAP55szTAVLKj08BeVsO5DCxxz3YTAACIHIUSQgFBh
XaBJAlB0tZxeHctJwDI3jiFCSua0/nq7lMx9qivVx3gt6j2OgjpWkne1zBG7i7rVTPJoNFaUbekFtuxNwelOK1sBOJl+
    eCo8kTiqZKRnuXdoiDbFNsW7bT9PKt/5O/W/I2b1o1qgA1eHZlIAAEzeNDXEWIbZpD7zy14Yo8LdQRAHAiFR8naFCMx12qulV9/
    xHCAeAoSZ1YLAcehI2/xgcMBYUJhOEc2cBaKr3BkFFjYlOeWZ+YTr0WD+Z2/LffyL+wuwuH1i0LI
    //7gmTZAhPIXdh7CRPyMeILXT2CGo7VXVuspFiAsofsrPSUanNuFjaO8wbbf8QH9UvqWB99aBHmJi5CogGOAMTgc6HGoDgvZJTSqg/QQxbf/sbokXn1BxtX/2QK
FCKLKcnSbGSLRFiZNk1LZDolAuOZ9H8cZFByioVEKn6alaDoFw0TekcTJlklpLt2J1MqGaZgghRb
    //vp/0qSVe6SKSSqDP9V203ZXnkWQolxi8b3k5FNlAAAAq16CJSPqmzWDcFRCUVHGhxrzXociMoqvePODdDSyDtToWIQ6iCR9GWY3Yn74S5Voq0pqIWVZ33SZ/m
LnB9UBpACbzFFiVEh3ot3I/+kEOoEAAAAAzKYgA/
    CLItADkkAkOZuPoqfaFyltOOOvL6FiwElACIpDSPDpOitaB7VzEIuTTu8aUUej2NTTdu5qeGhtjXh0WSqfc5330TIom2yo1Rc6laklh5r3h5IJNOk1El7n6A9tx
    +PE4i
    //uAZJOAAyxAW9HpG7AwgNstPSkADUjDaaewxYC/h+y0xA1IiB2ZT/6sx/q0VUH5AAAACmUBBUJSYOdAFwZV9SoJOWGBGWDREqJXpFQ1cZJoojxEzJSG2iEEjnK
797w60OAoOEtynUBVitLaYUBF8LHh1IGYjt167JhEJUBJAEECcLJquZBhgvQME8PwlbOuENJc8KyYiknQMY6LINmrD31C9KlN9vd/2
    //0f+uoBekAAACnHwyAyQ6QWqB8wQiALiqTFYTqRLCVlhA8DF1TKvRK0SNHCMpQLwYDMGpaZgYVPJHkAdjBMQB00dQn/+4JkroIC4yDbUewaNDLCGw09I0gLZNN
dml+64
    PGZoRO1FlT0tj4QQ7xy1jgcempPEMe0WTSMSijZQTSgteiXiAbxQYRdOEeev7mekIs9S5lU8q0YlLME9rjSDLQEysEAEGyYNgBYq83oFWSaZN
    +q0AgQAAAyqFcChREEaMD1qAPc7SdCJjNrKoZbMKwTQYAkSMyaGYwf/+4
    Bk2IICvhfaaewaMDXhqvpl7CaLLGdhTDEkgNOJKzT0jch62kSzYc9LHn7T9lJ6yDzoBetknDJr/1t/6/Z/+oB/1
    SAASnH4IQfpbAgk4uJvq8kRYxXwgxSG4YhWnQ4XFFYZR61UblNtFwaf5n9X8e4Rjlq8PxuCsjWmlI70iIzEEIY6Z5NmdxhGXLl+
    a1OeoMYqbEQkDoXtuoEB5QjJD5spy4vGjxNN9JJgOWUoMMFq8L4GWoVOOsekWs6UIdVxGELV7EXma9bcJT/
    WVFax0lcPee9eSIC52CQZn3333f/d/29TP/zaA6qQAAFKRyKApkgcFAVAt5eCVKo9Q4oGx8tZtxPijZA3/FO8uVkuyGUH/3
    YseiUuuLHd3etNWlse6RgHMOaajMxCyC8sa76KxWtrKnY+Zd3RP06xKhHIhXOUTDBkqi5zbK9M/
    KpGS5qBIIucBWC7hcl3ONmHWXRCxJUadhe3agUE6x0c8v/7
    gmTvAAN2Q9dTCRtQQ4MKamGJYA047WFHpG8Q8oxq9YMdyADof93i9cJzRJnbDiBv6hh5m3/9m
    //ug3SEAARODQxAGXrIKYQYkRBUob7gTQ/1OaBiv1Sf1LRlqA/gTz1/vK9iUpVPYiJlAxM7g6BANpspdeURYRbB5sTHBbL7YtQZ2Rm+iLtWVv0IzLyu9OoasxVZ
quOuSezUYerdaizjkxVFGuzTUTZWJ89W1tRtGU/uQGd1QX0KxL0bUWL7Od9u/pqP1Q3/
    a1WkYwQYpjDCK1TtSm9GTUurUZq57ChSbsrKQBXHmIJgfRQLQaPgbBOGZAhc6gNwtP6Rv/P/MbPP/7
    gmTnggMlT1jR6SrkOqLKzT3mCg1RaVvnjLiA9qBqNPYIaO8z
    ///9t/VTF9r++/9X1Q3/2q3MEZVoAA4JFMgCc7OF8GEbxL2YlR/PiWJTaCjRzIQLkwib16KOePVcfjDtehkCm607N52G0G08iMQwkinRiCpsr3YVCQaLopU/TyS
n1YkCMdH8k08bEOnPdtcpCHpF0er3lQkIJzke0e++7zYrWT71I8nmRdo1qEBhWH30PTjn+2u1ijQobtHvfDqcuvrgZcATY+
    lFChWaeDAdYoDBcPsJb0z3L
    //uCZOsAA29a1vnmFGI9KFqdPMJSDhlrW+ekTwjiIGr5hIkU+WNSf5/K9v7J/////T+v//VuO3o9RJLsrQBN0Nl/yY0jXXKGG3oaetrztuNADX10QDRmRKVUdf
hKetKKZWe7Ueuapyw0txcoTEKZLzAyWZrIseQB5VPljWn79lvZPdTVTz1NW+
    /4iOve29jN6KkTdtqvY3YrUtqYYdn5T7XS//uCZOSCA4laVWnsE/AwoossPSIpjwE1U0ekscDZlat88Rm571qHOWpNLWwAAJAB2EwP9Jp9gkof60wMShX6yvNoz
    +GHWXqWM+04Mc8LZyCykeaxDi+q2DS0S3jw9vFBSxFisKoRcLgsC7vndZ/P9RIm/u+
    84YXMcDnU0GbWH65pnWtfOtxm9zvr/41I/ul2aXesZpqffh2/i5i3tb+NnW6zZxjWM0/3
    nby9omt4pLsSOFBzHgBFBADgCQEANTK2YosjQGxjFjB4eqYHA0ungdZrzjMMddgDkE6LwfioJtAR8i0havL2wsGW00ThbKJ+
    Fp4o1Lh7NelXrBXTOm2hV
    //uAZOEAA+hP1n09YAAwpPqsp4gAWJWVW/mngAIYHup7NPAA2kw8oxPb+7zv9M7E6cVmy0561v/VK3Z79Yc21uW3OB8Zv//9/X/x/CmQ/4YwXt9fww7//+nn1RE
Iu6LFyGjeLMFvSZ7sB9Va6Z4g9RM9HZbkKI0Ovc2TpJ1uwuBEJUA2BJGZdG2BmLlS3TGz/+4
    JkiQIEuktZVz2ABjCDK2znlAAPGO9lTDEtEOYQbCD0iVi4jHnqEyFdgYbbif8lupBDGSSAMJnvOeZl+L9ZcsaReRtXvjlKWYWVaf/0
    p6kgISv5AAM80xon+MV6P5yVrCrEgJ0UgbXWPMkyccO2lxmMYkba3Qt281vmVBRzZOiggMElLDCJ3g+
    n6QDWEAAAAfTuUgJQg4ikyis4KgGQa9SsYFhoUT9lXjrmWwJ7P3S42cKH2G+q/JxTb3yA3Z6mFtHFHLzlCwV+
    E1w7agW0v97tRwPnR2GCN04ldfCwRClA/9E3xP0bjyE8MVNmMuh2avqFEYu48UY9I032ixJZJc4lXx+2xikVLHkyoCkMARlBi/Wg3T/
    PxCJ4rG0BHbjIxisehPPMoa0ysc7LoSd
    //b+6U7m9lrKg7j8mP2v/96bbZ0AE0gAAGZBpQRGp9Sh6tjF3neRpLVGJNrGYHeeer3Y/FP72arPPLLl10Vb/+4JkcQIz3jzYUw9hwDuFGz09gioQgSNfrDBTwM
OhKpiWFhLET2sJVp1liIsaehkki1phIglAOkH2K+lhGzn/+4BkXoMD4UtX0wkUcDLBmug9iSYN1T9hB6RRwNkYbHDwimgNEsjVThe550ohdnR/2
    fsrKkMJxZmhL2tLPN6ys2dIEQqSI0tne/MhqYjZBwOwf4LU0ilNuUUbJDj4x8WxkTsbdWoPyGM6q4u+
    pU5kdmyqiCezTtzexQbUpXzDFUDP7fj2E7bFUkXgDJQs1jsOR0TRXmkVJjJ9GaqjzIrHYNUtBpCgEylv/
    mky0X8ma86D9MrczAifUZOhRfYUgAAF0EYToggQ8JQTseSpGmC/DVqY85Zj/EeVa4UQQlIUOklBO5EJGdZqeMtXUpTtW7vVqvPW+
    aaMYatSYJwuXrkgoKAu6sLECs+QxznLhjGnCPS84pJ5Rl7+
    wotOgYHCMWYn7gEYIipLeLk3oITmjKVIEjaCBjcHmhhDTzRA6B6GEpIEYqAVRMGa995Aa+9/
    cOQFSiFUpilizkV6vGBShHMIWKaAAADe3ZPYnQWnBf/7gmRZAAN+Q9jR6RvkNKIq6z2GYg4VAV1MMQ9AzA/
    sdPYIeAHbXsy5g8HudVntQE9zQdFo7O5rQEyzdx9uTder2BuBcW1s07X1i6YUmuzXJplGiobOdxXpqUg1jGU8Sj4IbVxTv3ihBO7v6Wx46o3pL82K
    +JrgdR5X3kyRYcLIUw3+aHN7a0C461UCGBQAiTRK9DAPnAtNSwpG0CmSqgC7nWFks6JDzHTa18jm
    //6IrOgwz//7IcM76SC01FEiSSnLGMC5Yw0ENVwhxJjKMZtmqboZh1p5UJxuonXMdCw2aHDnhJY0lvD5vS6GdS1I/Yl0YU6MPualn3xIwpiCHowFDLkI+GfROII
```

```
COB4KcZqhIk8B/Cvi8lf/7gmRaAQNTNdrp6BykNsIb/T2CSYwpCWcHjLOQ3IarMYewkI5C3LMSz1D1RHYco+KRM8DgkIFaxjkgTuLVUpH/o3vrM+
 ↪ GcntgUEXjrDK6eWVV2ZAk7OzsiTaTOpcyEKx3FQo9VCGAqLWOApGvr5t0azeAI7EmAIQCPsjHlsERiZcPNNhyhwBPsA4NgiRl9bGBHxdg+
 ↪ WIxYFw5SRHjgQUfx5AmKB39sV/61
 ↪ AFgAgAApx0H4B1EsBZTp4IaX4mptJM11qGKxfXlBOlHSH3k00SRV2bqSPIimQSuZ1p6KFmQeR3dPtULPR1Tl19kWzxh4GoBrIvl3HEFhJ1PPqy1763
 ↪ /j+isARyokLAIEAqJp3MVQHO0JHROEdEsJYwhmJWROIx06Y1RK7AMmiVCQunk0U0SWHEdTnfsEJ3/
 ↪ rACgQEPJ2QIikJFo3MdOUNYeVkYHKBQgjpZwzMN/0mV8hYkPdTIGDM4e4MQpHjifk86Dwnf/7
 ↪ gmRiAQLINFhR6StENwMKzGHlOAq41V8MMGrQ5QhqdYeM4Khnw1QjQ920vG
 ↪ //vlfdoONYxN5JhV7onNnf/ad/7peS2FAAA6AAAADeBGSqVFMuyCrP4t9uEEQ9HDQELZx/rB11uhifwD7Yhg3wgx7262Uuky57+uzb/Cbv+6sAAAyhwyFgIqFRI
R3ggK8KFabD0FQOEHaqmHmKgodJ2OnpEkA3QZrdPYNwFxEOTZJkkap3l62ztIgLo8GxFxnA3HQotzUOlkuuz/vhxdVv/9X
 ↪ //bQxZ7gAAAKaOkilKVbASKTgtAaITAnCOkWIY61WFBY0YgvXuDk1PGGQ6pvcK56DO/3dzdun2v/tNKHM176UZFwHh4q2X16ov9j3vHFyDcgkYkkXIbO7U9TfUF
0tYaekqsDVhWp0/CSQm4MSTXOtv+TTNvxiGYuc4MAg4EDaHD1AEMjl09ceBwjCwFAb2LNhusPGwsREcAo/
 ↪ ZX5EFkNqVJhAQAAsKwFFwqZ6RK8WUbam9FiI2MRForXhTOeEXCAJYqfEqus3BQ+C63t1vd
 ↪ //60U2400QQEnKLg9Yz6RVkOF3bzJOpWNyhQ9blj2nZtMYW1kDirnA7e/8wCCbky8L55H3/UznlCkcPhI0XsLHzqj5cBJUZenkDgTcgHEWTa7DykjC36ithWIQc
rqZljf6MrsIjMzZiIppRDo+AWDhMX/URHE2WA3uTxD0IiEzKlRnmqB9IKQxl23LTIycTGE0LHMyzVS5q/0o4Hx83Us6AqS/IzZb/
 ↪ l13qn8ZGJZpM5dRaaNO9YkUAUQOLpsvR3utkY5GaJKzkjHRTNbSnVBbq35DtItTLU0rof/
 ↪ or0KkuzPScSszwupUlsEAg2gYYwdapFeUxrBpZGNVM9U171KtYzvfqquSfrP9W6Pf97hEjWdD2TY8aLKGG/+
 ↪ pppVTsRUBlVJxtpMIkgqSi1HMIc3C3kzBikmOAb78nMcy1OrnzqaCxwjTiBE54EgoNqFKsQQq3E/Ai33Rg+E85lqYF4qa+LuvuacE1Ku5r
 ↪ //v+kikeevf75G6+i4ZD5QBcqbFLePCBkZHSwfTzMs5/yH4rM5yMUQGFzEtBGswAgEFiGBEeb/+4Jk5QIDSF/Vaew78jOlKmgxB0wOVX1Vp6SvyPAU6nT2FLDpE
fOqXtSJYtSEXKRC3jeZbOfqOk3
 ↪ ///8a+v6U3r/7zbf/7Kn0Q6VB3o9ga7OUSIjhLFamroRkJgEEx4M1eACyFYhahC4wGUiUSEY3mHhIA3YCrS78qiMflxYGUcaHIJNbhsrg2uLGqS/482J8+PPiLj
QBoX+r1BFxmZSG1NuYrTCFD7fNNVrdY8HCNWfVCirjaKRIJToOmKcIoQaBC0tsnDcpUkWEv6QUKfepFWq1aUD3LjUuzaUWKm9Ygml+
 ↪ KpsLti0lxQpfOGJmxNUM7PMqIBGUYsqRdViq6k5L+TGs/
 ↪ Is0O5dQfvJ5oZIHGEPizDoJFHImFvVyijQmciaIAVdywlEAAqYeTDzaapHIJBRi2GU2rhKMdlSf86+9ZXYAtEtVk4DDL8CGOfKZnb31xTL/
 ↪ mJkDDMDRx7PYYhocjF+X1WKqSk5L/sdb8izQ7/UP+T9PSjBEuBgIeIH/+QBu/
 ↪ taJAAAUxNJJGGGoNieNdT6w0zeDoGuQFD0qsTCShQY002GZVWo+81pg
 ↪ //H8TK/PYtP7D//7gkSBgAN5S1np5hykXmkbPWHjDgypMWGsGG/BpxVstYYY2TkNlfZKFbveZ16zQYQ623kZXOldnwp3/mRXn1F4x+v7fY1v1b28oU6SKqTI39F
fvnySqE7AUQOE88Oy63L6NuIxPmtmxwcuRry83ya9bwPwRWe3cgZawQDgg8MGzKwocILMuC5ooDYwJNdr/+
 ↪ PGgKTNiACFgAuM8eSJNw1z6Tx5wEurK2V8tvDnReyrs6WMpLfOf5LnoS3p0YDEOgKaYTE/BzsL9/
 ↪ BtZ35IAIZQiIQAAABeEIhGZEBM1hL6scclYz5OPL47CKKdjFtM0mnKVKqaPA7m2o9aJV9Dvdl/mld1J+
 ↪ ysHAY7S3QSKEmgmAkDJ6xIsHnHFINlyWJvZR86xR1DCU04IjSx9jmBBCqckH8o8D1Uu7aVrWuxACZwqAAgCQABwxhJR9mMfCKGZKGxeDCSRt4
 ↪ /u18ySLzEs
 ↪ //uCZEiCA300V3sGQ/A7APrvPYkiDTTBXaw8w0DuFK108InqdDNaCqknRQ+fegGAUHq/Z///y5w0qg4psBgI+usgBTAJgGKOCb0lCIFCfAy36pRyEzPEijD8JAx
Vn/17LvVmK3t0e1FrM2Wrcicouz2SuiM62a1/Vnsl+cerAN3KoEAACRUZMDqISwvFwuhGKzkjL5Xev3pK5HUzLr80+hr/n6uqHBSXl2SF
 ↪ ////////R6CMBCSNsEAAB3NdOUCwhKNhzGAjMhAgIpdhLNi1rpiY0pNLvu36I63a+yIy9b62Rn6Tdv70hO/s+spd5303SBo0HVlTwvLH5Ua5LfCWIXaKCugJAIC
85kJJCJuhNXFMqnUqjEGuROszNo7nVWJsrpgVnK
 ↪ //1F2OJnATEKBGJ2fmPqVOmw5nv3MeXJhxTw5mgCeAABSs3eXxGsZWsgWg8sIhzhisULqLPt9SpMrauR5jIrs8Ez8qVf2kRBzo7ylSBPGIO3SyIlEFJ1h6N92FQ
C5EkSnLT+BfjtZkOTCTGWaLo31MtK+4wKoeOIC58R7uhU7OC953Mlp+PWOzHP9Fuul50paiyYRVkVnfr6Jqnf/9
 ↪ fKtz7ylTz19ZokCRFQdp7aP1gvsCAASxQ7ITRD55Wksjb56G4O9k5F6LQb/0Suun
 ↪ ///br0Wpnqk6wzGv9ZZ4gd///SpC7VhpSoly2ySJIhFuqYH4PeIZaQJOXxDVOnnI08aJBKsY8XqeL+y+3u1vOU7du5flsa9nvvfdVe1fXFg3Ckirac3EunVtYIh
1XZc1weOqqiMCrmnvTtNWeZ95QQuSXdxhg4zzTkARzKJKb3VxV73c7RW6STZUa+
 ↪ /YtIe44R/7P/uoBuVAAAFdzDDDjEzCnSkSqgslpyy24MMV2/D51bEfj3niosJBPRDOKW4dCxZhuqOWZNNoWxwukt1e02ggKwdVEHClVn9XxIsE4enz/8f/
 ↪ +t7N/dzveny3Lrpze2KrJV0vG11COoLIXSLbb54Bn6wAEaAAABYGHGnhXGGMD1dkbCEgWUN3a/KKZ3f/7gmTjggK+
 ↪ NNZTAzv0OaKKnWXiHIyo0VGtGFGBG5TotaEZ8Jc8dyI227yu26GutUVRzIhy1c7HEVOh9FRBwjqVbUQ/SEBYM
 ↪ //f/+oAARNEOXRCJUOIJBtWj+XY2TA/06KQPCbagPZRUdF+ro1AglL9QW/yxnoaqX4WYSh/PCEF8QwHKr04RcRgG4rTpIYtn4XUzztG4houxbDrZ9MamfyMEeC4
SltfyQc0szq2AcbdLbFpz9jXYb723PabVx+m2j1eiFdqBaO8e9ujXeyVia97Zo2MJoRGC80fV8vbseJWTVob6HBxjH1XFsbfsKF+r/
 ↪ CjtMo9vP6VrB+XuYBqCYrrCUAAAAAAHYABwBAoztss8ZSQNEm9m6zvPddl8of+
 ↪ xGuLXzlwvHiR81XtGUtrEaXDBPdepfatI1BePC4e092q41lb+4OJ7ZEqdKoelZa7eqEw3U4P/7
 ↪ gGSLAAYdW1juaeAAhQg6Zs0wAA4lJ2G9kwAAwgLst54gAMkMfEtqvr6FatDrsqzaZqdIDlG3ZvXL2lms223zPTkttEt9syZGPOCOISzWva26UnOwsA2jACAAAC5
 ↪ +Lo5IYj4Kdii9oVGVW1scvY9ZbXdmW5nrypM9fum+rSZ5rPmxGft82HbM/zH+eJP9+b9726H1o72/Z98ZsSsiH5WB96sWEgJOKT45BUJ+
 ↪ SIyENM3gH2LiaKbV0BHkyfUjCpgHbkojVm/XQguLJqlBzAKoPAh0enLmS+f/
 ↪ Kb1QI24gQAAAFKcZI8mW6bxlEArhiTQZuCZK6z7Ok9lBfIVEVE803py6HHoa6rpK5KVspEcoXVQZq4Q1tHP9iVZNsFNLKJonOrO0ZihgzV6QAhQOLIhBV3zOusa
 ↪ //4vy//uCZDoCA8BJ1+spG/I15IrbPSU4DmEpWyyky4D8hCv095gQbZH3IQKwoyADgsA5j1QSfSRrr+5CmFQ+buCVxz5ZlRFZ115lIvqZfYncq0YB41pnkHi0uZ
ZtS1TAJaKc4J2ENCrB1IpJo+BsUNYKRRcKt+KshrJloHJ8XIjxinB9lQwq0S2Shla71Nz25SqeqmsXJGlKe+8ANaVkMiBAAKmOe0dU0oq+
 ↪ zpN41FM9gBOKInBy8QCfcsFUxXoULjYC7vMVu46ULiWu0ZzvGEVe0SRUBAgo0cK56GYxbUmNlsN17klxTNQwUsjtTKZLMmlmUDjQwckB
 ↪ //uCZC8AE2JA2HsMGkAvoSrMYekGDrklWawwccDCkit08YkgdSgaHCx4o5ll+fl4cdpBU28AFoIAhMHSBUu2Jclqmywo0WJFRE5dilW+dADlCZS0EGX2PgyHRYC
```

```
t+jlV0dNkKsyd73EiNa1GF7m1f/8MbLnICGTZssEAJ3tREDUvlEWQtjeaNuFAWIzoJDchp06g7Zlt2Nr5mUON/+
    YYeE6HkFqU8ti41JWcSJgUg1ZM4E2goUmjnin8Fl5eZVHYz17T4uUXl/905edv/PYa/qV8ie/sAQakrBAUZBBfKjEASh8qQkOT/+4
    JkOQATAknX6wwZ8DIEGqtgwjpMqP9np6BR0M0UqnGAiWDyQdAyiA4gZsVj/X+zLdERUpz9HSxhW4ME9Hf1Hjv1qsFO2NpJIlKSg+
    ROCDA8jCM8rkNNdfHqL88UjArZO5PtNQ03q0suaqOM0gxVqYpOhVyqSbr+9
    qJhlTs3nr7qQhRRbVUXG33Zs5ahCu9tXryuxDB8QIwA8RCV4AoQDW51tvXTNWSp8Am5BAQAm8F5s5JiwDBDeyp0Io/
    tOIxM4Qzb5xoE2xulN70QHLv7lMbd7D0+6
    DpO6sO2JgUpG2iiQCnKFYqJ0RRN01O6CoDozEounxPRaVnLxHdGep9nMcBGZ30Hoind11RxCfe81sq++sz9Vs0nm2u8pW/0vKzlRXapE/0
    XlFh5xUyVa6zobpiWkkswAE5XQCQL2GSjWyhpSIAtB0+Gyw6WkB/0v1eYh/79+66Sqi0Vj+lmwG5pgR+R6St0SPfQQnLm2mn/+4
    JkSIACx0lY6wwRZDMFmothghwK6KtprBhNENaVrHDwCpYSnLlgxCTNq8QgloDYZpq7+
    uTSrCwUCkYloSMpKKxuQKbZ2bgiF0O763ZK2VOF7ovpvQwVhSBr1yL4VDRjIrUecty3z/548g6dav/
    EKGTy4uNKgty6UQBJwbBzvCJWx3KMzIZqx1czkuTrE1bfQdxf
    ///elP3Auyp9+oCGOJOHSFizFaf6dRCiBb1ssaaIJTpcSxokaxlokjjcFgFihJKKmIQPKUDFPaYmvpg6CFNkupoFL4wYdSQreXH/Md4t7U2HxcJg8IqswCxQWaW
WnwQrQ/XemxLKFJag+GjrSQLd+1k4qAGYcCSK1GGEU6fC0dRuHZaGteIhRUsiLvaZQIT6NJtX
    //SzIA0dxwzGa6xCBEKEYpChs0KqI0iCU4AfiaHuWqRaiIaQwGM6V492UsQ8gS4QBgRleXQ1RMndNaEGCTMjMVqr/+4JkYQAC0yFbaeZDLDYFKyw9gjfKxINv55
M1TO3VxqbsXdPvbiARVyOKNchGAg8QMdunvuOvWSrLp7J7VrkK6+Q/
    UUOwvIJiwRSFxdErZxbaAVZHEBQVhDxxkKRnQpAwrHAwhYEspec3m7GIZHVfCJ/T0+19TspDykdNQ2eLe
    //7gmSNAALaMldrDBq0NMGbPD2GQ4t9MWunmKzQzBUqLZYIcR3cWwdBB61n0jSAGFwCCqNZQHhDQXjhG+PhItcz9eNDFluecYDVJXN5Lfg7epir9762DOLoiJlr
1BVbDsLCUZiHCmytHcCo2MBHLRUcyOFIYbt/mFOkNvcjaqQdg6rodzrBEG7U7G4ZmMdijbKDCJ
    //31zezN+lv/ou1fkWdDERlsCpHo/TevYkFNtf/7gGS6gAK9MtlrDBHUNgHrDT0iRIrpMWOnjFEQ0BTssYCZ/iAgAlgUcAhYAS44ioP8kzia7Za2aI1rfhxLk/r
YUqoEQRxxgyYqv/60D+TpRR6cr1F3/NLzt8R6r93/E9/PcXFTxvdxvG/ff9/P6X3SfqNcAn2xFAABwAJoGpyI1
    //uCZOqAA1ta1WsmK/JApTqaYYI4jJkvZ6wwaVD8FKv1hgkWyIaGGfr+dSBhYVQ163Ej846sGWzqeCNaZlOT7t/TlU79E6wjzrGMVGZx3uMLB0RgbySAQFJpCIF
0yl7LGamKyBAME1TLQB5R+moejkpx7giN6BCP33YzGLOlIVTpm0zyvNbqWFUIU
    //uAZJ+CA4Q12ldhIAAwQats5gwBDkz/ZmwwycC1CS1w9hhUcWoAJAAAAAXGfhkVDXnKAKiWPBLO4ba+27gwFAtWbn+tNmLBQH8YJAfKUsQRHmRScIx8nLCkHYr
ML2ehcfEJxEiMqnWhZh89lvrtWrvYzKY7nW43/+4Jkm4IDYT/b0eYbxi7lOxUwQ4oOlS9nR6RxwOkU7TT0iKijFp37dX6exoa3in7/2zO+
    jf4txQUrAAAABWxeEiivBeS0JQymvSORFXEceNP04HtRDc7LnsN7sl4QmZy61BN+
    aqhosqQshJTHCkpw4ClYEIfTR2FCEZCJeEz9kLfMKTEozuy3y0sfPI4n
    //S//P+q5buUW5qUIUFD5gOjH5HTKwE8AgBAWHOH5NN9MsfN/usX8hGdBghIKjNpVhWfcoTOufGGY0XWJ3A6OMPGlj35beuZqZAU4/40p8yqKH0IGaO5OMFYAgj
kqEg5QqR3dbv/7gmSmAAMUK9rR5hu0NOT61TzDOgw86XFHiNGQxpTscMCOMPnsTWGWdckFuZreW5KvyfyN/Da+TEObnAQYdpLI/RyDh60/
    yMQgFygAAAnBYfQLcvguTE4rJQbcYm7x8frrOtsaFSPXW7sexNYKWHXRBbmZXj2Eq/A3yBhVNWOXozEKehJDWePBMhfsd0Jbf/
    eoArkAAADAE4PED4UFEniHEjOzw5N3T06QHbpm8B52iSETbdXDC/jct9OwUirqWmZa7YJjJFKHsGuOAES9BrDBNx+3
    y4kQeSZbrEZkkKULw8E+LgU+ydvA6CYkIR/CVEHMmcXxfvXcBJD23U4L5Jy3ZNYZVdS08u4bBMZEqQ9gwPHI4dtuO1DTRdw/48MJ1/6
    qAqzAAAJKCIWMDOeJJoh7MqFSRU7FgMYL5htniNjsMwVqLmSMuucScY5NSOSEzGdyFSHsa2eOJjQiF0GDigwKByTvMFemtH/oHoVbSf/7
    gkS1gCIzK1pphhsASGVbKj2DHAkgq19HsGHBGxWsqPSM6ESEnLEL0PWyJeh1MpJojipWDnPEV/
    s32uNYdny1F9EZcTmhObT6IyGd1333VeOJNGmyiHG3dP6U1o2+upAXmFadOhFGpYDYoAuAvhPSKJaNd6hTGfasNoQh4ISZ0CmJ/
    CLIZu8ti2kKFanS/7RpnSspJgD8HNgsu3S+
    hux7VI6H0kMjUThliae0q9CAQoAAAJwHIZIeiShFsYfFcXCAjWiz2DE6ccOIoZlN0HNz1uS0mbrZd+1
    J9L7pgFKGZhZY2DI6DDJ2PqR73JIZHbs/
    aVoAGoshEgAN7gwjDAdxDRWotFmWFR2LhyFrli2FJiyU19QRA83QSPoe9v6kmTsc9DLlTyydcgIPUpkNfQXBRDegkVYH0pS1SNNKaUbNaRQ2JBS5NLcrQzyS0CC
    /cTKfQmBWb1aF6mtf/7gkTLAgJHKl↩
]
    dR5hqgSEVLajxihoi4p1tHpGkBEpTsKPYMsNZzmyNXGmjvaxTAQkPzOFXGqmOAs8BAAMm62YbH9IJUuqFujktqw2NMyee5DLKnvnL76SrLHsewjp8mGTjF7ekSp
    /Uh6NzJ7auw1Bp53361etn+nTVR7NRIEG3c7jJSBITWiqUfpeYRg4PdEEDLEC8chgawpCU9A5Eba8oo1POanf2wxZqmgpC8lLUjRVAp+
    fJLQWSUW3luXCtD8egBXWLdUV35XqSg2BiwStqPQEqCnMhlK9UKXieyaVq6/wsKFDwVnKmCMS1pWGVk7kE0c1mntIVspbRzBK5c30gFf/7
    gGTiAALCKthp7BnANOVauTwjmA2tI11MGLNBApUrdYYVUDa/DkAogrXGAFAwiiswiiAjjHvPklyySn09a6HhCrAQQCm5MYYJcW0P+
    RGw2fKBfJzalaW+ZYdjbCT1gbCMEJy86aY24kqiAbn8+nBs5KEKHD9uwSgU4/4
    Ge665upyErDx8UY1rtYinXd3VW5zOsYkOXRVfJGGDBXXd1zIeWa84bJx+uask/
    qnDf28qMZYZsIwTMsjppC8uxBbm8nTODO8BAwAYMcaYo1xKCVfiK7u6nIwcft9ZG1253VXczoUqAWlkggFNxVdVUAmLUqazauExnNb9g67WcRt2nnfyln3xiFiV
    +3pUrW
    //7pveLCAfFViiNZrbuS/tbDZ1r00BskAIAHITQOzwC4RgY0nGra1Npj+P9M5RWEOXbisERQnq2IAtU6//uCROiA4pMrVtHpGjBSBWqhYYZkinSrWUewY5↩
E4lWsNlg0gdyPZM3wbHJiQ1qqpfOjsH3YLNwlyw5tWDa1eTAq0LJJSbjSKBMlypBxiXEdCdYjpPQryZGOuWV8S8/
    hXYiIDuLapX659ajQPeLe89CHTmUSxJRQX9KqW6AmMxaNrIDAW
    //7q09jop7bt7p/MjpvatG6op19rVsn2yAA9gRxjUGe5IWutEkEQYOqDuF7i/VhUTGXsXi5kcb6dYBBa0p+VwXDVaAQMycqzJw76pVxXIdfs/2YPIYr8PqZoDf4
i/L5h363IcoHkDNEggcygC6ji67YNxYk4pShU9qSJTtynJGAglC3kxJGKw1OEXWaHh8SAYGEFkPZByDiC02KjFZiq5lu/Zvc77KYz/3BnHE3/
    ZVvb6yDxrTD3p0Gv75zflShtgFuCpGgBAGCAAHJY2OnCgmBIUwhUDpZgNThT1fTatMMxfRfyyI6C4Eam3K22rX8jpuJvQ0GyyqWgX8tkJFdrsWMzqAHJvfc4t2U
    /mPl6R5mCzLI0vrosKaLbpVlzdsD67jPPWM/VsEd/hVhlyiI+5696qvNTLQlXwtTYdPyEgv5TTWZIo2ipCRW7DHbu/
    n8PvrOmETZcGFrrxccNQ0va97SA+MOXocjZsQ96TtSyCagAMBxRF47INUTz5pK/FTC0mb/+4
    JE7QAC3ylT0wwp0FnFOqdl6DYJ1K1hR7BBkTqQa6mGFLIGQWbIwo5Lh1ObQdE0OBjF/NntcAdls/kHetwC8shp/hiWdrh/
    GyPDeYo3TdKI84ayXX0CD/k6F1AIq0QQC5LdC6RhQhExF83zl7axpT8NOfffq6/VgsOJJLsIIKMFRWIkgyPrIrYqGX7hiq2w1kyirb3M+
    V1t37KCtTzw8c+MRz7L/
    Z3PayaElcsoKCA5EknU93POFQiF1hAEbxCF2xWAhd5UXQJvTas2RiwyIjGIlXWb6lvYgxO6hb2IfIbOvicIt6nDkShdvbPAt4hyKPRyVa0CmViZy1CleyzO25Dm
    /YQqVOABB2tg+XSOaz0OwXxfB1RI7CuSSRXxjqt5I0PlX/+4
    Jk6IDzrDHTUy9KYEolWiJh5kwR8WtbVYSAGN6SaYKwUAGi531WBLo0VwHIPwsbFBftcV4gLt9MssEuB6IFXs↩
KH5b2p9FYWxs25qRU5ZzqMo5HppuVHG0K1oFs2mxFnxDjertcIQqosRV13I6Yrw4GY0k9ojmxvYdvaHTLx6o1h/vdKY+NQa619W19
    //OaVm3uv1n/+PH0zwbs/7+YSgrHJNS43ZIpDa43ggAITCImjfIWMUMPgoyBRCgj4oKoCtyKtVWaySUg4jaBrFgs8i6YUJfi7wVCc5w4rGJItp4yT6PqP7QJF5l
```

```
BAA9BShnp8MAgyhLqdL9LKtZfvoWYjI67a1q7Or/6GEjg4LPyYsv
 ↪ //1471gBKooAAvtjJsjRH7UL5GXLcuYlDtltEsLHp1J+e3e5u339xpeXzlzXZ5w5W4kdYbGH3SnjJtVNpzajIjZiHRhAwThguFBaRVA3awDkTBEjiovYIueuotE
jiW/UK48dW0BKGZCIgCVgBpNwYqiS8VzZMY0Q0W+AcyT6
 ↪ //9YUAjfP6ExXESpQfO0uTJSMx31JNyNpEEAAKUYMrlQ5eKlDE1LG5rxYOyWHn2hykisX2mUJBRRw9u0uWbMznYhESNqqjJSoSr2e86jiDWFaF2ZWZHKHkbOFAz
OsvS3Rns08kP73e6+miuhf/pOpTgL3qz/sn5kdd/+
 ↪ vx6xZbedDZ5p5cYfPtCg1LZGjrQEgDGYySAAwJAcJHylyEfDgN1VtSWVV2V320rdG6W0WijabcuoJhNe0/
 ↪ GlBEIsj6dCPR6IltyNolEAl2jmMCwGROEIauH0lGgNycIR2YrRPM0ES1Xrm/
 ↪ vg1HTIMZG6qgtDgxWa5mqul2PTqTdNqh2O5oIzkSQtpenitQuERSXhLtAoueAjQAacb6lcFqRjlrjABAYoAABgDQNFBF5MV+
 ↪ qRn76uiU4aYpRdTKo35+12+1tcuiW0cQtRA+aakYGTDSX8q
 ↪ //uCZEcAMuYs2GmDFKQzJBp8PSIqC7z9V0wwp1C+j+lg9Iioc9S+9QAAAFOPE3RpqvkfkBQkdCESRBH8AodX47K0S4tCRer/891GKVGR5bHZbVL3dFQjqrBomOc
wfgQAiSANDzCOhZJhXofmfnBu+amkXtpFJ/ctFkS4wRS9t2lBzh2A0QhDLCdImegANjOqfKjiJIf8z/udh6GzeCCk+
 ↪ lyxI6ATFmSg20ooEyoWT3aratRwSQTYUjgDAUZIEerdRKxPj61/E0LEofu3c06yr5HjQjaMx1jdyfI4LCqHmr1e62qJ7/
 ↪ fRpohSbngWBqkD47Kg3CEPwJEMlvA6SAoGWrMMaWgz37WQ0t5dhus6OuD4nd2pzs7pKa06Q+rjHnPrCLiMq+
 ↪ z76hxDJuqZKjBUUFaKp29gudh9gfIGxTO2
 ↪ ///uAZHUAAvE3WGmGG8AwJVqcPCN+C9TlZaYYsQDHFKr0wIloEL/UwKKAECAE1sAmA25ZbYxOexn8przR/PK/+n///VwBGB5CZIIBAbTy//KNAdd6nHRRnMKSfe
YEIYW
 ↪ //J5l7nnkZL5JEZLGkWzjd3qBX/////8l9Fl/0ssaIC0vAwiKDYApyBLR8MQlTDwQVrA+HRkipRCB1SiBttn20/REKkJw/8FSlT82N76Ebi8k+kfNniUA0lFh//
NtFQroK+
 ↪ mKbcS6UyRGuNK1IAEAQAE9EyLvOhTM9e9ti0ZVWrVe4swGThMvmv0zyZaIN8pLqfJGE5hBk8hf0rDn7JBAAA7uEmAVSWPEJek6VSpQo6WQv4
 ↪ /T6nZVSDuOMIKHntUgfFGFg0drcyjXMTSzodUKcXaUMyFDQHFSxjOIijMA4qcGVv/Uc
 ↪ ////TTao8W9Wfp2RGMzDZgb2sE0R15vvN+BjsJ5S0pGS0iC5a1UIstZkH6sY85z4cea///7gmTMAALWX9pp4xREM0U6rTAjagtZC2usBFNQxJJq9PCJ8C/aZqp/
7OpUedCDVk0N9VIv4kslsxP+xl/ONo0h6pd69YN9Qcbxnxc+tfo6FY1pRjkapqKu/77eLZ1B19brrdP
 ↪ //O7Xy5pNRqNwYIaMpT+sLv42rW9oU52lUP8b6AJKU2yAEAZhQsAaL6lwEsdFtEvnqd1dEWikYf2AohZIMIAdZkw40d4yRKQ3jZBhsJptUVPRt2Z3lFWuDIVcZS
FiIiAHAAL+KSUz5WpR/E1uzWMNCcRm3H6
 ↪ //3+p1S+gGwGtYHDF1VBrq6YA1tsgAAAAOmPeCskmXZb9s8HurBsOtheyGJS+7qtZpIsAOTLmNUnaijSULTJOgNEbT0DEVpWPmEL4ELEdRpuOCyWFDCSR9ksfgi
vY+UtSlck9ph3VHgMBG1lhzAnPCacWW5gYEHKzYzJqkZUlP9rA1CkFASAfgN0URehc8sNLmO/zHW6Cl/
 ↪ pZnbZdrMbz5VV7MUzOyxhIwbn1OKAJlNUDSLl2MxTJCLvG6IUOo8BqFodx/BoCbCeE7x0SUysZ0g9b0MyadE1FQRVAb1AhXhW8
 ↪ //uAZBiAAwQw2XnsGjAyIZtcMewSi/Sna6ewatDPBGu496QgKkeCoYt8+AiAWpXjIgYcIFJUYiAVTDNMgY29Jqqg1WHYVY0+58sSKnA697TzPTlTH9tZH/MAF4E
A0WEoYbJCZAFjgeJlVHdENvX38eZThFyAUa8Md+9wSkbSUtQcPhcmEwaBRyxvelbB6zyF/FrW/
 ↪ TrrAi1gBDSxO9QM01pxoGlUVQ1BZABGCiUuTmDH37n9t1t8pyszwwCOp8ha09fvQEjm6bDxBEXW5YGX/+4
 ↪ JEQIACQiRV6wkY0EilWqhnBgQJKHtpp5hJMRIPbXTzCOquXSpFBAWEp0EaTr1Oi38uFNZJI0SQCnDDK8VUYuQ8jHPdKrZNCBxSIIgYpNqq34M8qVItzcze9
 ↪ /
 ↪ XHYioBnCTFIQZZ9dRSDvpl55Qq3TPLLSwovZypjtfRyoBsbcjaJJdtEjJMa8ZacjjT40eegocyJkEJTpqTfQGeyJZXmTM3vf1xxRdRc4hhZCDP
 ↪ /xL03F3gyMEbcyg8xVNf6/J/
 ↪ SgXK5JE0kU5c76UQ8aLkz1xr2f4MBpIcQLEY0enajYuLsTjfhVmr2ksdIXnKSdilIrF4UCEqpq44Pcs1ah2DhJV/3vtlaor/++hd1
 ↪ //9/RP/2VN5POYmrNVQeKQAM2aVAAASCmAJBK2lAYSTAc2Q3u2dy/VPQas66eY+6IdHNegk8wtGU1QSBOz/ej6FgAnBIhmKBAEm45sMcVtZert52+kbrVLMiyjD
szcU5E9xbDPh8+Lglf/7wJImkigSC3AIStuhfGyyZc1grUQkKBwhgxdEIQfcgzJfcZ1cGCb9HcvUR4QlmVSH0SOuR2WwSQ07/9
 ↪ ZRdoAchxbsJQGEWh9nUj7kOZFIZXERH/+4
 ↪ BkbYAC2CpVa0NEQC7EinxkAoYLeRdhrBhHGMYUq3zwibjgAZHDAD3v5ObJsAaKWAUADaL4AA2HEG4rBcC5MyWYl28ndKqPxkv
 ↪ ///mkL2lj28f0Z/fgzbmfLOcfuSoBKNtIlIAJODhVkkT3jaA1EYGg8kMSheaIaEWu6Ft1CjzGsyu1VMp1HT1ChdDMxaFXQ8fubTVzoph/dLJf/7LTKVRzibicXo
JHeICVTCfzxTG9YsSVKcn8n8PLQvc7XPzUePwL46P/+7+ai729Uo50gB0Vu/76B1UNzyyxSNF27DtbUNL4PB8luJoc86jVCq2E5qlIZv/7
 ↪ gkSfgAJJJlTrCRFgR8VKzWRGbgkgU1+npGcRHRUr6YSMqqeX/6455PidyXuvlBGujb6tp73qy93YWpz/j1Zr6dAj/
 ↪ JLDZFIo2W0dNbHqBu71EgBuQQXBBVnyhZkWG69wrozM0J0Z7M0UuYO5X9iFa2FcvbzhSufEzedL0/UWw9BjYGPhc5/xXtt/
 ↪ QI3v8lDaTYo2rR01kAE25G0kSDBIBtF3eqQmiWPJpSk71RIlWK9mXD9/JZ7CPR8MIhylAsFzK507a71pzvo63diZaIP/nWDNxZX
 ↪ //HSP/9ILgGA2mk4l+wsGSQVc1O/JqMJAQsgmkCWDxQYk+/32VHZ5Y1lsysozpX8QHnE7F1dL7B+VO3+04Fx2fLzMoDovwxMTxo35kRdyAdVAADjMLPMERIgpah
zmQgfIhyF3OUK+nVkvYSB0f/+PUro5UoR/vI3dfuUmry6zPpfPST1Z1fVet2HTopkfYw9R2ZAMbaYA
 ↪ //uCRMkAAlEqUctPMLBJYuttMSNjiRSnUawkQ8EdlSuphggyKAEoAog0uQ3V2yx/GmQuVsTk07D8G0+e8hAoS/87UBlC5L7fY0MhV392//6A7ZsOmIyhwxohMhs
F1aZIuS+0rsjlY3eXtT
 ↪ //iD49lADIAC1ChBAADBCoGAcM3DSTwH49C//uCZI2AA29D21dhAAQuwTtO54AADXljc6wYr1CjjCz8sA4YMb8/pkwbb/WCXDvSOXap253/wo7zJioBStpEkAAA
mECCSHFOR484QJyEjZYTnVwIbtMSn/+4JkmoADnz9YawwscDajC848Q0OOPQ9frCUPwOUW7HT0iHqr9WheQQ+5FS
 ↪ //Mlf/6m2Z2o6ct3k1VRb6bv////r//00Fy2yRJIkFzQRaBmM4aDs23ziajEfg51UlWxKtaVYI7aiQhbPOWdH24ibjfnTNEWohVWXjPLOD7qqdpnHfbzadtMPbc
j4ijv/+4BkogAC4Sza6ekaNDPBGv89iRALpKttrBhPUNQJKzz0jGiXR38UQHrYxO/GigsbUSE1K51ORnYVOKh82F+
 ↪ aYqHx3st3aPuelQbtttsaJAKdgEQCI5DqrFlERbMy0yIixE4gEE8vKSlpdrM1/P7aTC+W8kDUsfOi46AXid6fKPECzYr1Vpo2GfQm
 ↪ //9adykmygc2tslSIBBc4PC2BYUlouHA/DkbnjsRZGCQgFXSSzto6qinfNyo61ojf/lSn5WsQ9bpOAB0T6iIN5c72J6EbU/dNiaLWr1/0ZYFNKNJokApylUAcGI
U6lGDCSEuc/2+
 ↪ nmG5w0hVqMYYIMFZjZTfkzEeQMclFfRAp4GDhAQdZoEyy2NpolO7HYOAh7AtOBrK2CVyLP5So5Sqw8ZLyvZ498aRAYi0iBm5tNycFTvswjPp
 ↪ //34dQ+FqS5hSoI5/n/+LMqfWJDJWnTexj0E4TshJB8OMheFWqQYtCPpCRAPsgM2Q239KjlZnnfnxn+vm1zFBG/zAEcuoBQAEwBCsVOnBTiU2EEXcUIURlEjDSZ
YUWPX/6Wav7v/03DEBjOKHhRsf
 ↪ ///1uuYpYicbeSAcl0bjSAUloJBKCiYRbg5yGNKhPBUF0OFoFqYGJGkh6UucgaYdbbTKMTd1aodApCLQVUS3ebhNyLdDuJ92GR//2Elme1DNM5nNvo6PRDjUNCO
]
 ↪ /P1RQ70gV9qiAE24KWUrTOqhDZcME6QjBCeiocizQ0YLlzlx1w74gKbY5C0YZMaxVT3tFtP3cD2uykUqIWLbMigeHYgihrGhrtuvdN9Sq1oQEb6lLWSI21W/IoJ
 ↪ +t1aCqySTPlLsJYAzuVYxpxL7r3X6B2uhARv1LWSI21W/Sgk594bvvUD/+0iApJQhRb1rrNEcpVTLBWFUwrD6E+
 ↪ Ym2XgtZGYcBM3sXYcLTlLI92Rv3fVuSShKODwouA2/+f3qjfp/VH+JuF9ak2MzX+u3pJg
 ↪ //0iQDHIOiS/Yt7NaQjEksXfGxadHw+YKxKLrMo5UnM+Y7y7rPUOyPMxCtrUz1Vu//uARQgAAmlPWGnsEkROy7s9PSIci3StVUwxA5FwFat1h6RYSShKOFshXUF
```

```
HrNLhR+7hDuqPRnjz3gwJ9d5Vy+v/bV/FjRJ8xd
 ↪  //+OwRGSt2ePcIAyhcE6BCAJASABCExQAGdoWAthpbZcMwTYiHiI+YUSTLGXLvZKABSCRlwiHGHDpdCksoyCwF3CPqVAsMSV4Xt6r9++r63p+/Y3O715IomN80w
6Pw71Qi0W8VEeUKJo/jBrvp53spAqYXDm0jDlc9Iyox7Kc5P+
 ↪  ZspGNm8tnx4rbjulNXeyux1jbFSiMTKmKoQ5SVd2Ta3eYmqTLHFh6kXxXNedBQJpQgASAwENQEkUJ0Ky1ryfW2NcMimiCvpjmK/
 ↪  p8mwTApUKhUsxxH0f/8Kzqb0CqQAAALuN4R0kRpBdnkZLsnbYt0WjSS4+
 ↪  eRCJRFZhWlWkSJVfBCogMzTVWtmWqURypibGp7smE0Wu3xYYURV9SgdS1cVCc6EuiFx9dz3Ny4ZeucUsQXMFlTjUxiAMg8xbJ4LtyUYB0tOs7yRAArAIXz2JSqF
 ↪  /+4BkSgITYTzbUekbcCyCGwQ9IigMePVtR5hRwKcGbGD0mKL1HixXWaW8ADvgFMlBczLQRfdLgvztApxgPdJTrhlT7a9J0XRzoqt4WseKuNw
 ↪  +KIfGNi0rN2tMeNSxR0wuvmeEJzWP2Jf2/nNh8t1/VS/T7l7DxCwEHBe503PA6UqBqsXoyijKl7LXNWEMzKIay4Mg6B4oumQufNcDhk9P+
 ↪  qBJKBVI5JgAiNwRE/GHrWuAp7p/9FUhTtkgAAAp7oocp1GmMg1ixMKkBJGLQTklSgHjaaN/6OGs+
 ↪  jXwmCF9rvRVGEEGJCYwr9I3RIEBYYkg4QiyJGO5x9ggMeg8/IogC+
 ↪  aLFxK5medLLxzVLsy5IenMj2Zb2yy9FKgBlmPR0qauoWvVpd77YDPoBDSRDxRg0kSE6Vh8iqHAlH5+jpUT3H/kwL/W1rwGB0gdwq6tCvZtT+
 ↪  ggx1tIAu9kLWpUhnHWGcbioFwIUeKvUBxomP/7
 ↪  gmRbAgNnTVtp7BnwLyGcHD2DG40I1WunmFNAuAZsbDYYKMvNjMlqJBC6QHfSQWbCZrrlF0YwsgkyEOVN/
 ↪  bW5HSZjUWUgq51IpL56rE0xdkPKrczzyIYaXC5tRKtgkhkONET1IewQg9dgOu↩
R78WWuugAFOoIAgOjocCCw3qsnnrxgdmgEMnznLQ1qAUNqKVS50/0eLjp1z9qIkemV/IJVQLbiQAAABd+g4R/
 ↪  jtPZAkJN5IotISlpCBGBheuUWxdb3kg8AqG8BsHaIUQkPq1FQ3cnCGWetVnzI4OAGpEDWAohxKeF2IlhwVKzRkqGiAyf9Aic1E7LU6Ot1rsoiFKJNEAggKUJ8Ay
 ↪  /u/qdxF
 ↪  ///tIfpERZzzg6UNNEpt6SSnIiSSQSnNcQtdkhSKBOx4aZinCeR+GkcQSND4BW5XSK0ZNxNiCWJ3cPlOZMOqWxZ9qOjedNi2ep7vNcbx+P/7gmRmgBLmJtrp7Bp
6
 ↪  f2VjQah4sK1k1lXjBrYFTWMW8AgbvfSXYVvNfoAG5ERIAbgpAOyjC7LihKGEkBAQlYMeOh8D194mKInYRc7yaHZggSeWckr2vaBXQD9xWEHhOhQBOgAwmiBGlqy
 ↪  /GUM8YBcUOjJYDCq8mHkRRoJFk5ZACU7NRpE+3x5nZ7I/
 ↪  Umd8LlJBBcelm8tT61mFqsYdFSLQuJg8GSQs80d1ulhZghElHj6NEqUU5W2yACzZ/DML+cdTQgLtjYyYCtgYxi/KltX6w9Blv9Sj3/6e
 ↪  ///+jFjzlusjIHOqTLKgCBAAAHIyQTEwgwSzF6//uCZJWCAs4o2FHsMUA2Ymq5PYI4CxihWywwyMDNBy1w9glWsugVSx7BslAw4ej4ObiQzRQuwm55V+264n9nX
6Boeb6tTRSm1bfy3YokBnlCcsLkOVjnLZf6SxeJUh1aSkZnKrP8JQYhL/
 ↪  BIsIsAgEGCniR9BiSZ35GtOg572LqbQZZiOqGpBUrkKYR1noLBKUCSGLPJzh5SS0kMConQjWRqPRbPOjKhFJyPmqdFVpkdzMzLcrdzBrUKgAwAAAAKcHDpGpaIg
 ↪  +bin3ShprSMdsdVuU6J6bwVaQskNWl6nGCyn+n+hH/2L1DFf+OBJcSQVSDBUcgP
 ↪  //uCZMAAgr0/WVHjFHQ4YipVYegcCsipW0wNEYDblOjBhhTohOSFqw1k8CAE1dC1JCYUzPsIdLyIKJPEweeDoTJiwJBHSwv1pIf/b/x8ZvPbUF+VJICbdL6N8Rh
jxxiABnyQUgZQF2SYp3wUHwZHQpHBBxUwTSeFR+5DzRio90/KVkrQUGZYa5n3z2hSp
 ↪  //xbf/9v8V+0VeZgZB+mRJ6QkABJwRUGwC9P0E+ZR+bIaZCpTx6lxTb0+IgxgfgjNI3ZoeyBaDrw8Y2XGsafMEee3t9tLxyYlqk05wh7ZwcYdv/92Wtqi16t/t/
PG92wqpTxKrK9/P+oxSAol1OsZrqZavLPavo2SRAmu8uuH11IRXaLoEDGxm2pdCSrTRBAALlotIAAE+N0vJDRongOIkZwoT15MN65ORkZHy6v/
 ↪  M9fqxLmeaq4rnex19+I419vGEOJXB0VbQj444fMXisNTcdg8VrzOEAkbVS0Vyg5f/289
 ↪  tIX9FlrKuiAGPaCsZ7aEL6DQWmx8GRlPbpoN3Lqygg4IQsBmIbnfYT+3X/vY+
 ↪  //X/ZD/+4Bk7IATZ0JU6eZUYEMk2ic9I0YMjSVfp4xS2O0VqEDxpiDu3dsMq+OAfSAJANwwLiOFKJR3cLA5aMkd2Ymn7PN9/FRI1RlYr12z+yrz/d9J3qL+uJbJ
vyCnEfLdFY4z4hD1Vqx31VnUBqm3/8R7OOoUO7EqbRX7c5Yzv2ZnCRUP4/ffbc1d78f/H/pAmkYAJNewJvqZOev/9VV4K3KAAAAPpwW8uA+
 ↪  QX5KUae5OcuCQNPSyjXkhQLKUoiP/sSckx1YJWypS3hiQiUDwJGhMaUImsjGSzaA6AAUkYiIU0MBT+
 ↪  kxsVCMFkCyF7eFgGfaBhpZaN0jyWsREMZb
 ↪  //7gmTUgAYbXllWYeAGhcb6z8y8AA807Wdc9IAAzwZtM55gAIb/4IoiUKxyXvQd30R57DkV9QbcLRWZunhVMx2tt8hAEcnBBxvGohx5KRDoUdJnJvca+SyN+5DJ
AuSCYVVzpgdpqUTNJ4MZJDhLTEqjpiTDPbRr6CcWHdgfo4fFfXHHaL1L7KSX1v/Of2Uet0
 ↪  //xLB7kaZRanfk901fVAoogDS0SQpJQJh7qtCiJVBQmicUEAcS0WIkAnE+tnDLsKXv7Kr2uIKJiilmU1gmFjIONiRwHriPk//////91IEqCAFRN4uODxCAZZBZS
7pmL9GM/xEy0XrO/yVQaZkkAAqVEJ4FQUEBZoNsnbONwVTU/wgA5SSUQlDVXarwn2Gw4iToE51PaDmvpa8e2uH/+
 ↪  zj5YpR1QrGaP5w7cNF69dNrLbP3/ZLzW+dYFUf43lPjkctN/M8xC/ueaR6VqAK8RoUgJmqeIXqH1WCgRSB
 ↪  //uCZDkAA4lEWNIPMPQ34hrZPShUDV0/X0wwS1DOCKrs85lAURqwIBCHYD5OUJU3i9ORMIebUtc+Gg2y0RMXTgI/a8KHYVcGVIIKODWD46GySa86ohIRV+S/6gG
by1llD3ZZUs1CB6mYz1fWtC1re+99NCoValLN1e/nl/9+j6HGllG/MEFBXGi6HLXo7p2+
 ↪  hGdWzCQEAEiBsIgHKYpCnxbWM5SwfAjrDxVyweJ4zFmEG1qgYllI1U4wMDiWtbUKFT0UV9v+00BD/+4JkSAASvj/
 ↪  Y0wwSZDgC6to9I0cLONlbTDxlgNcKahWHmKgBQIAIBd2oxEkXIYRBi2cnCWRLjtPtTkOuilsmuJ0U21lfM0VxZA0QFCkPqn0kq9sz8i7I4YWbBq5g1w9RAvECnJ
 ↪  /5Soyqjxz3RQRuToxOre/S+v/+TAEh6SxANWABhiFjGUVZLS/
 ↪  JRgF7pS6HleKicR4wJXCHZ7Lm8fMw5yQdoUtNhIlLHD0iLWhO9UACAAAAFds1bQoMvkjnEZA7kHvlCoNj1PDsueeteks9vupTjTRCJqG/
 ↪  TP0QIplUt0bfqzklod5kq5JGxoZE3YzN7l36kbo1bdbKV0ZjkuRUYTK9zjRcUE1IHy48DEnBkBrEhdH/
 ↪  iZ4CmLJBASARxOgkwhoVjYMh3FoZGpSOYC0iDzo0hrRSQebWbhzQuTEa4nHedYIrlKZppIq
 ↪  //ff3klJKJEAgkpx+YZ0lyL3DMg/yvoyIkmjd1wGAk7/+4JkXQADNUnXUwMs0DdCGro9iCoKpStnp4RTENoHqzD3mDAa4cPe0CiZvKRzVeNGZRtOS+iRHORHFmE
lwjASVCj/Z
 ↪  ///+cTpajWr6iiAUnDdVY+VGOraNbeinq4a2A/D7WrUXTTBMwQwQSYza8kYUTMRa1jYdKRd71QlrUazKX1OMJRuneiGEYeWfSOPPnUjErQvHpSf/+4JkbQACpSr
4RP8zn/8ZKRHhM5XR1n9HzEXf
 ↪  //p4zo6v//fCpImxFyJooEgASfhmFZNAyRx2H8cR+L4SBD+AB5XStPxq7R2sytcuZfMdJV//7gmSdAAKLKNbphiwwNKNaijEiSApxB1unhFMA3hSsdMCKMvLmZI
S4U5
 ↪  ///PZGp1t/7O7OrZsw5Ete/Kuznu9rpbPR+qaLMNJDPpekiG7WIEIBDEwaNZCZiCrcIv/7gETPgAJIKVTRgh2gSuUbDT0jSIjspUbHjZEBIBSrNJKmKJD09OZxS
kU4x6mxGXlZ5/4xyBwRrvkACYAIAAUABAwBlGBBU0q3Fuz
 ↪  //uCZIwAA01KYG89AA4vJarI54gADbFbZawwaoDHFKu08Ik4ouYyAYf/mOpqd/XXknYnUgdP3Xkg8Plhq7vf///15CsCSJZDMQIAACmAIpaMNAjIMIIkSUySEmg
EZXRiEloHAu2d0fD5CQffe
 ↪  //uAZJoAA2Q32esGG/I04SsdPSwCDeEzX0wZDcDQBW40xI2CutvpC6E9///I9OKZUe8avQoBS+StEggF3mODSyWgtL4fBYKjpSHJ6MiAT2E52pO6UQ6JvrCj3YM
```

```
JksQACtilY+ekaIDQByww8I2SLjKVjp4jPgN2UqvWHlBgsWJAYtLrZEkCUgViUJyIJzhAhCcR44ITZtKdH7T7cJJV+pHCmhS+i+
 /VzpfEjoYqzvW3TqQLW1ztmueFR99qsCx7iaLFpYsWAKRgDDN4EHIIN5GHoisxbn5TWg0ATUOMdKbheUwrj3lNu/jxCOE3W+
 dni8ei3c2zIiHA+nigAd319oc1DU400EcX/lv/6wW++SJAbkHwD5Hs0Ni8JSDROQlqGQRPRJjeWMLtgUnFXrpFJA8qo+HiwuVVteAGV/
 DChw4KKNyzOL8rerwK1zFaP/0IAtuNpIkgtyl8IwRh+o0w5nVFaV01w0dUcWdKsIVd5NNVRFOWmjovp/VHQ/XpyWsO3/6
 V2Qz1QFrd7Yq337i9V1CP7lFDigEAWpXooSALf2ACOhILMnig9H6JUUtNy1R5b6KLljO5bZ9URp/efB4v8T
 //qPD8lSwH/+4BExoACLinWawkQ4EWFO70wZYWIwKlRDAzLyQ2FrOj0mJJfQxEIW7/h2Ka3O6zgq1F9+4v3UI/90sADLJGiSAZdziIAJFEMeX7QBuHJHJhcFrwr
pwpVOsMGVBR5Us9PGWIlKzVEvW7pPoWHwgABh8kuNC2vpZZ3160KtIgv/6UiHJKUg+
 SZB6Lip0EznCsKrbahp90er6vo4zUgKFWol9TP7JsUHQ5TKYqa1P7ozL9/h+Vk1wMIBgAYXC4heRahREc785DHr9n/hse/
 tMgn9cUQFJK1EgeqkVjgGVlE/A6GBKjORJGTUKdfztMYJp7mbHXOfdmR2Y1Zzm1dutT/2ZbpuvgXFBQ+6NRvPU84i8jHliI77EHIYjL1
 //86PfbaZqAKkaTSRJUlpJgtCIESP5dmsqRynOqzkbUKYrMG/ee8WzVueJVEF/Al6sM50/QvuGK+p2fc5zjO+YLEO+t//n1/s7vve0cheXohB7Z63MdNr/qDtMQ
esaZAIXDhRzngexRj36lEpn/2I+uwAC+puFoAu7jhQENN0rSA+IoMEgOicPJcgOYjHhROMHRjUDJb3MzgcZ2LTROakPPnb3noyHH8ipZ2bZ/5
 //jjtDXVEtBMVYH3ANarb1JBEYxlX0U/JgJy1yNxIqS49QU6JI8k6lDwchmG9ASJyNytkb0jXDA6BLZv49vlbsFqs7P/SW6NSpa0JIZHQ/a1yvNd//FOjaoxVE6
O3C9ZJ5Q/Uo2AA0oSCkAnuEhzHNEFbLVtN2X+0t93/fmP9rR6A5hyqGWWeMoR0VyMhXdQkyalpImLaH+pZ547z1MvovUCgbHl/16TOhe3/
 MPQxDFoOES7HB0uywmGkmQBpVJIALkZWf65BLIVOlHn/ZXBcRZy2O3TN2A0CQGgrKky2nY2Um28it9PjJRIOiW+KPOI86kvodaxQXIf8j/7
 n2a9JcUcHS7LGhpKd6XuWHShyqgT6WJJAcgWHIgEj0ISA6LV4p0crUogdzI53aM/jynNHQQHMKQk6Ntlyvq9x+bKyQ3ZqNv1UGPb/+i6BDyHq8HWIHCwus6GlIj
 9+19tX9KOtB2AJUgkgF7UvSDqwZONyA5CWpsvM0xiTrRh7KWhrP1gQUGRtJBOIrI7p42x4/mGzSb7uk1/5j0/8i774hWWt3vbL5BJaLf
 /////9tfTnF3+owlQnNuj1KhuLQ5yUql20bYAU5HLG2QU3ZRRnaYh0QCnVSwjjyViAZV+EzaDg4FFRcw4eQiFJi7I6WZKOKmAdXO4uP+nUTfsa7r3WHBZ2//0dX
48WeJDtE3
 ///////////////ltQ928gMjuJSG/ZyS4ZZoXBaZBGJDmRQBAOJbwlhV5pVAxdinAeUNbl+VPM153WYOocGEDcL5qutSymee5omPAuEoOwcZTYvqY0JQ8ZuMCcM
OZ2ms97mzpxYi/AjlUWuhEeFVAVPnM2mQFotfgofGDjqzo0dVdoWYBdYAjvAIAOIHMVI25UwKWx/N4UIFMCQ0MVmIjMBKtncjIFT+2vSm4M5I+3
 VcGVntYA5QymQgAQAVeeWAQCqizWetwYpJYhBD/O4RgiBgqITaqq7dsuUnkmSBzJzbVCIElc0RCAuHsZ+AYemYQhE3atSPYfv047K2iEIz/
 BcrRMVz29n/mlT8+wVFEx5R1e+/v36l39H9//2KMsuqsiyRFWsJRi+E2HqEuiAExCU0o8NTFIwYATY+9N83CEZa7O5vYRV3ML/w+
 zrn8AzusaIAABJo1UDBTsb1jC1GauQ38fc2nikFwVHb/+4JkpgATKjdY6wYb8DBkStw8AkIM+K1h7CRqyMaQrrzxiX/
 FKwWiElET9PxWvBSMpTM3h9Pu8n31oN33DfWScBzkhQwCGFqmEnKCarOV7LhG1HGBgHFmXnnggLSD2hkIKeve13d/
 rGlyZytv8AQfHMJyhS5UYqjrJwWDHv5YjbH7uiP4VjCo0R0XMtNMOJnecfBYNf
 //+li35cF1pxIkgAAzHkEIB6I1BidG6c8VHPtzp6Rpbn6BKCVIw53a1wclc7C4uezT53Wbaffzju2GhFJEwk4DFkT6nFIxer5odLy6CDQrnTD2SpNlQmPLyOWGO
IBMUiKRJBKdQAEBFgIHhUkfOPNMl0VvylsDzNItO2LCBZ/Wlme/9
 GSosu6BZNnmpaUzyql0BjnP1eyt9oqGgolB0HbXlb2SjYpwPmlIYxWoTWV51XKi2xd0BgJQVxQggAjBnZBVMttDVKkpD3C9bkhslnOCyNQMVkiCavEYCtGVSxRZ
 +/+3//bX6aAEOodWNG0Qn+CQAvRgJhWx8uCqIiXoMIES2LbTjD1ocEqE+
 Ra0MVV2Syevhv07ecU4lO5CLRdKJXqakkcmgWGpTSw8xtKwEHo6lEAArgmgairIVqJKEMzvlBW3DxxdBSB+
 kRh60OCoMYhP/7gmTEAALBJ9hp5hsgNML7DzzCOAs4rWOsGG7Q24VqtYekSFkB1oZlXpZPXw27sQ12a+cUDI3U6xbbp12IY8ZSVDl+
 pSl72gJNxJIlAEpwaengyhcbTodalDhRYHIwPQfPfR0Ys3dqCuBU6lZU1qYhsoshrMiFf7JiOAWSuVFw8pBVreEzv5nO0I/Ijuv/s/
 WoEfqAADATubtIICLcBR9T8YJxwsgiC7Lx9hSk7KGLEcqtf3PtUerM3VRHf8uaQ48HXqjRUSP3ArXzNTJ7mGQn4lEv6Hniu5zNVQpNfbY0iEW7GVStgKDrbVTUF
 +FQa84FAbLKF+
 RLpCJD8dUlGXIZd3KqtMLsO0Qzp9kaM5USr32kAEIqOMOQPDUNVVFzoy1Ucz1Xt0nIVLqOKk2KnRGDUkBg8iVrMKinYgBEaZgAoBam0riRAJDj4No
 /SOKokBtRTeVxxJxGK5GMqci1k8XDFdbawURowf/7gkTaAAIwKdh56RBwR+Uq/WGCHAk8hVusMKdRIpVq5YSg4Hn/2ea7VbCy9ugUCva31PN
 /b//DEVd09P8PTZtJYByWy1ttFu7EtBaNxFjYF1RwNkzzjLkYLs7k+pHzk4qPc+np2tBOSkJVNghYJBZWCqLhursXO9LBO1/f2VAMZRLO6Z↩
Fq6P6Kr0p+rEIVjqVxgQNW24ceKwqlRQRt2sfHpaxSgMFLdtqBIy3INnOuSsM8mRqjCRSAOldR1LAUanWRJJMT/+
 sacimUqCmRCe9nPcIdvb3SKqK2fyxBa+x+HDqKJa1dCgE7LLG0kW5cIWIW9I+IdpeBsG6Y6RQ9s5+Ix27ZTqQ030fPK7CApGUggje1/+
 Xv7a3OI3wgD3PnfWdpiRbgIWU+z85mG7FhLMe6h8AhJm9nuB+CseG9ky3MP5/+tuv+
 CM9YNTYyhOT5OUNIpOCg3EcFjQmRLx3nQ2GoWMlwQx6ZBQDCYl4oif/7
 gGTugANeQFtrCStMRMUrDTwji4x9AWenjLLRBBTtdPCVzlPfStIHcGATFnHohDgCCqvp1h5XpyjlKWcL1Ld5f6QY7rY220U5aZ4/
 EWNcpRbRqG/YcRBl21E9TyCME5pmSFHaC7sbUW0yOzxuinVuYFxu0QtyoP7oe1E1VJNVBIDcI+
 m31mUtDId7CtH3IqQTUQKc7XYlqvdnHnYDMORFalOlLLn2oAZvqdKkANACqB96vgeDaZEp+
 IPuy06SLhZVggns7yNqZSv6JdyvI4IwJiwSFPSs6oVXFGkG/rrDQYSdWYoAYodWZDaAATvHomtaW5MZf8eVa9g1C0eBoN/Mzcuk+
 pwyeBlHQ2OLAE47CRiO6hUBd+nM7arUjYjQUWjBXjygyECHOnf/56VH8qWfaIPE8yf/eBYzuLss++fsPPRr6Ja/juf9p9b/
 VeiADF3WsARAF4WcerLADxXllzV54FUaEQmA+KHLYcbW
 //uCZOgAAykr2envGPZAoPstPewTjKEva6egURDzC+o1hJUgbuTrYrMn/9Dg1SCRl7F0UEG2Af///sEph7NuqYDCTjgHLdY4kgDJeXzARFg1vS14laW5cbV+ZdA
VT58XIFQqKGChwctq1eXewPXvHBRN1iU6EIaBDyeChRh8oNhutWgpZfTfjxRDjEDmQY35UYzNDG5d3OOl+zkl
 //zmBBKz////uaM9dlQFO061EQBJMwYwlTtZtTs8Vyz9qrOLj0Qp+cJLSdmaKvOjOJtbY2BqHratu0OTZV3K2qbL+9tZ7+GAa1SDxHFHoXgaLBhXLX+2VNZFRUI
p0QAACCV3wWuOhLjSDiGK44NLJvUUWvsuxff2R2P7dX691RL8lSdnVzhMo2qCYJgoUKf2P/v
 //5b//+qoBmNyNpEAtyg2bBUMAoC82aPQ/MlCk/Px8Do0ZVM1VHEggkTOyN27v7KecOwv79OlmREF9UAjn8XFr//lmN6UBOIwg8Mkx5VHLJpNh7CJ02TFJZZG2C
XBigAYORhEgucLt7081b+3375nXd+
 //otYljfv2VNuzK0bbjSIdkGiX0UAGK1RkgAe7rDHrIFAidBmrVVQNdbdfrYa7DqjL4aa05gTFUogiUtrGio1IiI4aGU2u1g2cqUX/eaTZ1qZD2BdlsK9ish6AC
V7n4ira7u7RZIePnLXFT8eP9QVWdCCsx4pPZc2vK/non0PZ8U1ZxR8+3zNnLa3sVcbrF1Mxs7/feKyJV/DcXONHiRpn8tN+bwtx/vWY87zxM+
 Pnce9/4DVLrUXe47zEsb97fOtztPiIBAEmogjAUEgECE0lkG9iRDl1k7V0u6VgsopFtNnXIwycZS3k6+ggsAZg7wbEaF00N3IkTpkYv/7
 gmR9AAYZYdxuZeAGhyhq7czEAI5RO21dgwAIvYZt95ggBKK5ERzg+
 AXAgmZsmcRYY8PXFxkeMqtJJSRIFwmi8m6CCGaKtnVL1K7amTa2zHFspI0kwRQqLfSNGoo5xAxVu9yYNC4gmTZPjjqPva6xyfr6QYuYAABL2dozhVgf9gGbD4Ee
 +4yWHpZ26Tu6sfs
 //n5M/5/2+Tvby97//97W/v/vfyOv7KB7Y/MEpgN9fcvVvr3947jvmEigfzCYIEAEQ4A3GgE1LTw6oezdmy1amYBCXQO7ggLC59bPQxfb//4HIgcO3upG1QiAAE
qKW2R7VAEn9FxPyTuMkHMJj5ELVoTNVT/GX/qCwIGeuhA6u8o+
 UDra4fbiJVQKEpsAAAABzTC0l6M4tg3CcrzeYCFsSMNw9AIAuQ8J3IkTcmVLWb8i/
 MoC0S9smYximQkMloKyQocnGwo3SKKdzgQWMxl0SVFmHq6qa71kSsf1uB/9/ieXN6MppkOUI
 //7gmQvggOxWdrp6RtyM0J7GzDDOAzow2tHsMOAuIXsYLeYGIc4vo2ZnzPsJV+w///6STxce1Z4yRj+E5hJJBGF5HFicPy2SoWFjC0nnUOEhC5JKd6WU2ewJHDh
```

```
yfhb12RHDSg03UMpGi3aWTy7RX3Izpc6WzO0VqL4HVlUN8ZfxoUg1rtv9LfPsmgINGlUYBc7vKFIsEbyT44e6SiFV73/8
    ↪ xJ4AMo8py6o1SKieajayNSl1crjZ0VCnfTSGXs3Pny6k0QjcqOPAkmBxIMahlwdFtF6AV
    ↪ //uCZEeAMt8sWmnpMxAxQ/rlPSMoC8SrZ6ekyoDLEmtg8Zl4abAIABTuJ4OdMmETZhL6bhekLTDMiMEojLBM4iU1krW8NXWanaeTOmHFcpnqseKP83ON+4VbJvW
r5B/TBeb/c/UtG5pUSSU4G8oDJY02zHonV98EAQaKCQhJySJV0Vyo3XeripdVpNr6sXZqO9CF01shVQmRK/dfIj0Vei/fbXUQQVjGfS+
    ↪ eFRYTC4QAcgl8AOFgakn2fx2S40JCgRYaJ
    ↪ //uARHSAIh4pWOnsEGBHRUsaPSM4CSz7Z0eYSZETlSvZhIzwQYfMtFScjlKKbsb1DV6/NtKX9JaRThjwIivsyhyKKoJMggLDwVQPuBCxsV+uj//F1RCEFGFCoTI
P3uTK3Z4xvcnJZ1zOEV/ZiPOZ359L7M5FMvnghYLkn/oWqz/7qV0ah7nDvt9D/
    ↪ gBUFtaiUCm5U4WQ6TREo9TE4jiKEDBp3K1MuFXPIgy0jEvkudk7/+4JEoYICPCrVEwkyQEhFWyo8YnoIzJFZp4RxASCT7Cj2DHiU+
    ↪ IDJAUoJf
    ↪ //2LU0Ci0hc1zBnj7qNwh/9Nam/15zq/rHRXQA0FiaJQBklQwHwEpLaXDKOTwuCIkge+GxzYnYfMJorcSexbqmrJoiyIihm/6NXU6fXV7//7KhX7fr7JW/WCbuf
GKgQ2YexXXKX6ewLjQBQAYEdJdDsAUgTSuuDk32oOUMmiycHZRBm+8hv9eqHHmXtNiYfS81FtWxRX6UPbuv7bmO13/+4
    ↪ BEzIACSinTMwkyIEfFOqZlJkYI5KdVR6RlQR6U7GjzCZJz4+I3gYqAJsk9ipTxL+
    ↪ qSACQIS7S1FBZgAUPQbZ2WbLMcsUDRGcIxUDEGDknpWol1HSUUD1bKcMgYgNMsuOuyeYcnRKL/H8mXW0ncELJflj5wMRX5VZpp+
    ↪ y3UkwMxyKWEV65pjIarrGAABb/ESCQkkyODqMeflqY8Bi4sQi3ap8TKOJGo7ahNdSuMMEKn28zKkChKabOpsWThycoDF3h+MKXC2rXBDkN+
    ↪ WfDEV/vFz9nTSYGViY0lLDmtc0zdXXSqW6rpIkEpxDyBBpDkMSEvFsQDG0ZW9cYA7Ig+
    ↪ F5y5zMeSJKaxZZe5npjdLxm7Oq4jopV5NFQEOKHT8rsexfhZD0pTXXlSjFNiEMqSKuallZXUcOVgot0ykVLcxj2CuGAJoyOk4lJRax/
    ↪ MHBuZKF4T8k8zIXhKRDqzL1X7I7KRcVb0VcCGWisq5P/7
    ↪ gkThAoIcK1KLDzFERYVapmGGTAr4p01MJGmBYZVqnYYNMNKDB5h6fldjxRfhZEukmlcbV7XkCOx1J7ApfDAQhcvEoAAEt5ebiChBxTQqhqFCITDVsfrKVpWorWw
    ↪ /PioDsbZVEpnYiE7s+lCnqQJMexVAcklompTjynB8Ph8QPf/z5+XKOEDcQAm8oc
    ↪ //0450eCC7JJISbrWHcF4EAIcTRxORYTkVvR1miGinBzeY9y1+8ZoTrHBnJUx0hk9j/KrqwSZ+U5PrnHlLYPh8PiB7/+PPy5RxRuUBN5Q5/5m6RpLOjy1UAsAAA
zxNRCRmGFPHdUi1UrkqkyWYTkq9Q+QzDzsaiv9XauYWVOV9Qbs2JT7pVkaXYjeQv/7
    ↪ gkTpAAKbKtjR5hNcU8VbKj0lTIo4p07sMKWBSRTr6PQWGteVWvkLlnnrYyY+
    ↪ UAEGAAbxr4quEsF9TGMZe09ZbkMzX46Evxh3co6FmkHzAHPfXrAYzCjzN7wzOVm+q2qmXr7+/yp7ETrZ+
    ↪ hCSPws4SYg9e9b1LAS/kACk2zEWkyKQiEA0IjY0dE8m7KYFxVMQuCXIso3nVIIAoaxZj7QIu94sBJgjhwEsXbCu0MNMZjkSkNwN0sZ8KFv2jMNTChrCeShkH2wE
    ↪ +99fyqxjnrA3/n+Je8be75+
    ↪ //8bvekTb/eoFNf409GL/UCxO22uw6Kx5CIUjsdMADn5GC/q+ysEZEBi8ElmgcMKsNTT2bjTwq9DJ9nSb5p35P/7gGToAASMXVRVZSACS4TKWayYABiNgWW5h4A
2WAKZYId
    ↪ //IUKBq3987/zDuGWhiYJIGP6Zov0O//uCZHkAA+Y7We89IAArAWtM55gADW0/dawwaREQrSwo8Io6CxtC+/DiCISq9KezEMtnzYsobUVmc6WJc5PeehsYBtDVg
qqR2o1HozEaWWR
    ↪ //uCZGGCA5ZLWOsGQ+QyYLrsPWwADlU/Y6wkcdCxkG6w8AnWeKNECBS4ycutDH5nh6XlVGllIlPmTzzhDTRA8Hj9pV5zIs1icuksp0VQ6VjdT/800GJm0CU/Isv
bZ6ewx1DPA2v8lKRIKrKlhp7EI0NGErTTDJQ610A0NfYrZS7dNAF82LTWz2Q2hqS6ViAkgACvgGNoyESFAkh8SCAuGpgKoZHd/Ukh+
    ↪ sceYSYhMRIOb
    ↪ ///////Xarc4VHLUtSopqVtJIkAFyIEdISZPiasaMaiZlIWF8qWiyhW2UOSPLYhSclWpwg83Gd7+7j75CPcTtdK6gnBAV9K9EUhWc7vqv6dPIyUtb3o+yqO8FkM
xR0MsUaNWGDFF4p0EovShBVOECMzq8JBrC7mR6EJc6IDB2o9+q5qcl58/JyzqdHsxAk9hD2TIlrbLy/
    ↪ V9CMgOZHoCjum1tHpRBkajdKAhDJqrmSJ70RrLCZlNu4EHZoWouGgNwTHj1Pp/PNC4RIMjlO
    ↪ //+Thh9kichhtOR1pEkhJSHUGuTs4jLVSPqpHJDV9GG+fiGwS/2YGSAIJ6qtSztKBky0KohTo+rSruqM/iKM9PYoEAFczVRiXQu0+pcr9F/PVu07f2R1odgyBsc
]
    ↪ ↩
Hn0RPX7WSMgCXc1CEm0gTUjEthq00CQLktw72cQEAgVFKRZjEMy6iCacwvO2G71acazOdUUxk6lTiXEnEA+VgoUKghCQRExEff/5H6f
    ↪ ///Vbafem66jKDzsdJZjMVs62z6IR9qtsNUtFY5rrta2yDtg0CQSIR3mjLBGBtIhECBNeaKE4zm31JzyoNqpPbagCQfyPhl3sM6fbhSAc/S7//p/5RzPWoimEEA
h1HljMiPvmTN8f41SlP/TNL3vjxNTzf9/NIyKu7+
    ↪ JuIzspQxmYmBqwu4Slkan4gMCCFoRKgkoDFGUMhQaKp1jrGh6BYjbzw7npuBICA5qyQRamnhejgzEQLZKRhOnvPkjsRsRnMPxFk5hAp5QRJM
    ↪ +jq7GNMYFwely7x4KwFQ+ZOPLMTkhkqx6U5MYxJv389
    ↪ ////nGMrmOx5J/////MdTyTBoMAElMsiZKRAAAAABRMlMJuAQwzHsv0ftcNGzRjEeUJE2DgCijqtldBYq5WfpYtUwTgl7WEnNwEuznQi8UR1QrkOMlRk/P9+p0w
cn5CXtknIgUVHA60IRUqSDi72ip8IihLIn2BlyAyvpfMYQLMlbcb/WKgUdS0R0dEyuJIAy2/
    ↪ RMhB01MwjPrqloEEhMeilEjRLVTdP2H9xZUkjlM5nqEJTl068oWkwESsq0MXITnpaFECk3iocOgSpLAAIEodDgCguPicPZSSpyF2ACMgNlfv3
    ↪ +OVSTZ3WI33EO2ZqqCOihQAAAACoLBLUqZwGtKAMScYDdMRXhGHI1K9ESRISFkii58lRYqU0GuRZRDtUpHcxZqRDt+
    ↪ SmbJkZOp0CAOkemcgdjnycn0ZZJpmZzedCzrLRRuEfksbjRuwi/T1K8jIvx3PLJ9FId767ezhnP/Iu7Wyv/+4
    ↪ JkOgIDgj1X6yww0jMEO5wwIonPhTNf7KRzwPOQrDD0iJgZ6gBOMgOMSG6CgGYeF9Bddr9J9ay0AJ/zYBn/0caEj7Fd6Gh+
    ↪ YZwji8yJv3bo10CCERDAgADxwpgo1JNuMsg5p6lL4M4gGCYeb1+
    ↪ r9mPWspLlcppVYzx2eTbbJeofOMzjO6ioftptRZB2GV4FZVoXdCZ1fDMjDAh2MLQyQu1hHD5AvkNjT6uL9OtTydJzyntbn7ps3yBt7vbGW9Q4
    ↪ /qkCqxR5cUfa9oA2vsMhz7dqBpGgE0HEMxuWz7ecOicNERKHStFKoLKV/2a+
    ↪ iJQm6tkRT7pS0I9hZIGwZDpkE9SkNJDKsp1u0q6qKg1a6kQAAAC6AyiALhqtZI1BkcGOjCYW+
    ↪ jyy6BZfJojiCiuENVKGvzNIk5M8aTIZS8Ne1KiJhrVcttCtAxFOcidmK6IT4BJ4kqltl22PDEQSJCKYl3dVBAxdp2L4NLqXw37/+4
    ↪ JkMIIDtEtX6wkb8jZiK50xIhuOsN1brD0gwP0Qa7T2CDBmk6QI3zz55xSocDXI1vNw/3bs8r/BJBdVIwiACo0uAwUQLPpb+
    ↪ ZRtkJph61eG6tc0KI/mJNx6yqGWEiwHkqvf//WzXinYRYV/2WLcA7CkEAAqiG↩
YCm6fQzIpOlAtmOfDIOhB4QPqHFM66aKDeNIm5zgaaamKBdVJDOA+QsdxKk8pJqlOWek0mFl2U1GhM0UC/
    ↪ IWlcTfCM26Sguxq9M6dPbJP4nCTtk3A82EHPIODjnJIQC0IRKI1nQzyPKvcDUqq1QqstqrAABSwLsF8HoGh/iSv0WOGR0g0P+1RayIUuyt/
    ↪ dt9jPdy1tWrWQKliAGGD0gTDAXUtjyNm9H
    ↪ //rqFZEnQa9G2iSQAU6/auWzQ4GUKJnYfFcDqAXWyKeqSouWl44MKgIeqGO7M6gihguWhj3S7sPSkdMGzuW4OGK+jMRv1Rt2aw7QVXarLr/+4BkJIETIVfaawwP
C76dpiulkoOlEn52xYALw7maFEAA3wXLB0VISwTn1RZoTqpigTuKLaVnUFQzj6sNwviRm/MytIrPzGQqaEag+lcQ0etzlI9egrvp+
    ↪ un5RLZ2xdBdttcbRQAKhQiVHhEDrji0GQcEBhS5qqhfdoQpijF6RUt6/i1JqR3nZU2zMTv/
    ↪ w33sx5QEDpwkXPKqgjeB4dO0GxqHkaFSsyVYpgtcVcJv06zpFVA7AX4vsqAUfseEQAAGC2gdpOOnWpwfDe0aHQ6XSw/
    ↪ BuKMGeKGh5UgsSbTSyguIDTJG4SNQOZv7
    ↪ ////1A///7gmRIgALQKdtp5hvsNWGKvWEiKAtAe19MJKlQ0BJqcYYIWPcSACXICHh7UmKqx5YvyddW84ECwg4VHCHDE4rmEqpjI46jkzxExHkFlLux0/DSEKqCQ
```

```
⌋
 ↪ sM8xtFsBzI8YKELTrYaNmcj6mdO0mgk4lzzD6Ges1tFCRVjfKM5r6yWmCvIOUJUmHtHuou3RcHhwXDKCbyAClDYcaRKYuV4lyLalyj004MC6CUYCeoTC4IhKm1t
 ↪ /rf⌋
 ↪ //+u+U/+0yuOAKAXvlRIBSkQFgNhMSG2cSZ55irAVmEvPLKKgp5GJD4a0musOqbpqculCU3nuzrpmfbkFVDBjHpdmUcw+zf7svz870nf9a6+tXXfShrs7K6xcct
0mQSAkpKi8pdGZFpW1QXlWeIvnS2sbfFVCE3eSeRjpxTM7ZW4/Fp5nZBqmc55L1ot/EsZ0oQY7y/8tbmSa736v7dpNM1Wv/lq4ppAKo3/0⌋
 ↪ nAAlEaACjASUAIAYee5oHOCyaDpfrOW2rQ2267azItHP+Shd6/zRE3mf2VTuooWqg69HQmb+⌋
 ↪ hQE422k0UCxK3IIQT0wpJwsPhaufOrllczQzeP4NMpHuu6oHZujnlReukrnciujkRt+w3lgg1/Si⌋
 ↪ //uCZKeAAsZM1usMEPQwZUpVZGIui2ktcaeYUTDIlSnxgI1w9bf//23Lp/7UIBCVUYDk11tljZVG5eAYJIqAqIxYVDASj8nJwYlYDYLSTaacxe96dgcMKlyQRas
BBE⌋
 ↪ ↩
M7VayW8+Eof6jZ3r5FFU1TvE8q3I74LqdZGGMdgeHQTe0SDB3vcCPbdYaan03wqaVcMhZ9Oeol6WzDivtVzT+n1uDr7⌋
 ↪ //9rUiUljv40Sd5T+14U9BmtO96SYAEAADAAAAAECIAAtyz8TNLRHWQ1HsUlEbobFFX5N1/hf6kevuRVce6Cx+TER2uPZIjLsa4lqzA3SEw1Urr4IL/+4BkgIAC
⌋
 ↪ KV8li4JdK0wk7PdLpmZVKfRAMFo8W5Rmf9upBlvAJhOr6ItHutuVyKbSPTi49cm3W6AUmrLAAAAE5C5MUrLDLJYClLGXnjUlfjJ23YOkMkRS7G0uW5m73uVRM⌋
 ↪ /M0ycxtnDlshkW0aauZzTdWnzEViQd0qb5o1qw7aUFRpJAWPk/QhVzm8V0P/7gmQqgAO/P9frDEtwNsWq/⌋
 ↪ TwiXg6Vf2WsIFHZAiesNPCJsvvjU4w8C+N5clVaypwQzz⌋
 ↪ //25c+HVbki7xWwQGrvXLZYjtQBEeJI0AAU8BOUwKGEcR42xjUaXmIbNoMU0mTsX/xETo/+Yh0+i6b1qrEZA3VvkXWVJI9+V+4YCbbGkUAAAnSEC2R50pb2D2SL
xDOA9ZLWrDPZ96PvUiCLJ+rfdgdBBaneq4AMHMRPue1⌋
 ↪ //t/////iRhl/jV1QnHY0ySSAXKFMPsfxhBknggi2l9ilISZCD+vLCVLTGvsyYX/DVPudKzBxo5ETkNdaCcHYiiCMshVzs5B5Cu9rfq355nVX+hrv/7gGQkAALZ
A4YBRFT9Gc2kr7m+stv7+v/9tdv6r/9SUVm+urgjIBFsUSfgGAgyAFMXjIetL1KRAdPsz6Ll2iqXtanyHXzWK/⌋
 ↪ tb2QrgtAmAokNlRUFpHpKooBLcjaSJAKcg6QDPK2OsrF⌋
 ↪ //uCZE0AErpf3OnjFEwzBBqsYYIaC1T/X6wYT1DFFKoRgwyhPtpVa9C4hDbyUU7Wk9IQJnkJLjp5snYMKjX/7/zA6ZlmfIfTcSqd1s85RJWcxAhgz2o1OjRPrR7
BN89tdjTULQ3ZqqX5n8+8k6wIgJVHU5qUPIsya3+UZwFFxet47aZRneiyOAYJqR7dNoukCVuOKYBPqsrIwSrcG8LgEs3KBlkVdnTdLnO7/+4⌋
 ↪ JkfYAC5SrW6wMz4C6hW1w9JjWLLMljrCRLEMwR63TwjfA77OmMku+d6O6⌋
 ↪ //pa8nCuFYMMSrBb8se9230//XQXHG20SACk40Ym0NpXiqq3ZQKAjUlIIuwK04+loiKUQgklx8vGaa1HsdDKVDEKxWpZN5hTzld6iL4mgtOCWNYcXUWUvbt3Hk3
THGkkkQC25HxZytMgahOgSBaxYXYLk5pqaQ4/A0sem/86zFDtqS0soS/9nfBKEYLzoQPrMgL9b+Xd9nS⌋
 ↪ //+4JkqwACzitYawwZ9DKjas08AnCLFSlrrDBHkM4FarWGGFj0Xf/39y9IB10bbbQKkuXYaQYuFB0ggfJQiLRIFgMKXZS1sxThGTPvqDkXKCpJAhO7Kpc7OuesM
3879PSJc/wYNDuT+jTLMcilef/zXy7SY6iV6WMG7ZJHIUS5MC7AwQUQv/7⌋
 ↪ gkTbgAJOKtlp5ipURyVquWUPbgjkkVusMGGRHJVr9YSMaBpJCIKpDE8HRFUEx5xpcJ0WFxELKJs4vW6GORzfoxyvKws5j9HM1GLdUR2IgTykUIiijju⌋
 ↪ //v/y///97et66DlzECt3qS0m/1I6HM9HNZloIho2rRaMZalBnHlqCyY1+JFphHQYgaXFSVeWuinJ0CFkBnRXjSwibic7hxKUKwLojpfdYrKajsUcbd+bn7Mojj
f866Gjfd2sL+/sXxbc5+Yj5tfyVLygWf/7gGSCgANpSVxvYQAAL4ILBeeMAEyEgXNHpGtYrYawcPANjplWFicGSwTf5GiX3bUswZYwCLFg+xI3+9⌋
 ↪ R3/G5xbgNav2Ef9pNr9dUESdykAgAqTL5KCEOZXqlDRbUeii7ptQHIqnGCkzTDCXL8JEygZBfejNwy6IJLY3yw8Uus6o/⌋
 ↪ VHr7zQyXKESJ6cazggk9ChHytrafvSrwzOMPwLXC4mCbjNM1v01d1Gb2PJAK7UhIEgh4Bh8d3Q2PiYEhY6o4nC5n5FmwJjDuCVy9X2aOg3aXanp⌋
 ↪ /d91waWeNWCiRjbhQIJJV3YRGi5XLEdI9L83D8Wk0YkJXJVDoYSfRROCiWUVXYrUJiXLnG7MKsychjELsvhvwQUwXC7mfJw45PuXT+F5GnT/⌋
 ↪ pZvsn4cFWdKUkVH7pfpKhfdTt2G2oQi9nJa8+RUQANQBAJAgGB6F64PR2jXJh6o8SrGlLbSJTouXYrrX7rZMbnnL9X⌋
 ↪ //kF//uCZJCAAw8+3OnmHDAwQdtdMSMmDJ07c6eYb0CmBWz0waSYAIcSAABABeyRMUeyGlvPoyYJosCKUZ3sSeQ0yHQWECSwAT1RbRa7PQLCiJEjekzyzMPk1Ep
GsoIYs⌋
 ↪ //uCZKUAE49KWunmG/AtwWscMEwgDRU1baeYcwC2Cmy4kI3ATpbVZ3EyPAZMJUkGrw9qV53eUlCUQoy+WQYNWSBaZR5VQQyEwYnddYxylVlVZ9Q7n8Y8Z446bEC
QzAZsdPSJECrzDaaekbJDYhSu0x6QgSJVNBLm400cdKvfx1EjEyBLnVowXcLetmAKYkUesVoQj⌋
 ↪ ///5p75GdFP////F8eAXHGkikSk5qFsnSwxXoxDvBvANhaHuI0ryYKMuW8KVi+mh6SEZdwFDZQoaX2cmHVlwaCUedJC6AodeXMFnhR54Fjq3NljgDsk99A5BrRf
mEjZiEpCMxAzloZi2De7kZ/ayk0uX87adOYMEtM5v/⌋
 ↪ oAEJZAABAJd7rg0gRsZMuN3AGA4Oo1rRMEcciybCSwwX1FNDne0uzGGn0ijkTqL5MzebtpRLo5SsxVZdRMvXbEI/⌋
 ↪ yzgsEQYAwo4FDwCc3HWPXGDgkOQYF8lVSeLnrVRV/wdch5jSoBvAy44BcpfUOIh1YI1t73/⌋
 ↪ XUjw2zGXcjNPYgbPOYo84JCmYn1ppWM6xNNmPPqE7QzpmY6nr+t/gwA5VuofJqgLiWQJKbleEEHUTAy6C5E+⌋
 ↪ OddngdjTuMlnLdG7MPc7bVw5PizejClLO+Tqlpg/1KyE7yPmY7KkTqcUnA8RW9tLlzkYxz1NSayKYdzUsZm3tRTrp/⌋
 ↪ c5p7TlRlVHOT5q3fe7vp2rnKpNiboAIAAiAIlUACdhflzEXHoSaGpNyottaB8QIUpgdQiv/+4⌋
 ↪ Bk6AADLzrZUwwbRDzkykFhg2QMuKdZrDDFQPqVKeWBiigQoIM6hl3u9k3IY9IzvRb2VU2Zrz0XcgK1/o6tTaQAIySQAAA7e8cYLTqGCx3PZ/⌋
 ↪ Ayo1FnVm3u3OgAjQ9EkFKsVVYk9Mddq4+FblvlWnmHGvG/5kDyqi8ZxN0iCKoumcaGjP/bmKWmjIlqq3ZjEYn0OHHnh8ySbkLbooN1wki48+⌋
 ↪ KjX1ADAAABABii3gJAPsKwcJCC2tAQEQlAxjjOSgyoYMXb2K+lkmYDIKIanmk1JM1nS8/EX0O8L7U0/kGJ/⌋
 ↪ yMBqlSSAG3HhZIVlky3nmYFSuc8bfumyl/IjOw/EgADIyhpqulbBP3Ik47TotfKkdkANVZV1Vu0hVsSOUSMSokQNIjf/1dbPlO5kM+W/⌋
 ↪ o9jLgpLsmy1S+uy6GLpMrf0M21Uyv2UcVcAJVQSCZQE5mBE/D1aa1dYGxUtcvPQ6+tt/8hPUQBS/6n8G//7⌋
 ↪ gmTogANKX9hR41TWPIU6bT0FZgy9CVWsGK3A8RTopPYNIP/9NbxFuU446///u26Vt9↩
vVrlNUGPVNcZ0ASRxtC68ciNJuXEUpnA0x+ljSiDT5fIMijSCrZYsZmdQEDD2YYwglJI7hXd3QAaChjT/2nkQaIDWKIe6ElyHg0QIv/+⌋
 ↪ cQO3Zo8UZk9eayGPpyp+r/Raru9DU1S9FY4761oqP/4tyAuQgBAH0BN+Vsqb5WGlHkRtqzLnNoCfqvCbn7mQ4rQWnVQ+T/⌋
 ↪ kxcthdnP5V3l0Mpnkqp91BTizf/lP95Rrj9n6kAonGkiAC5KIsEnF+⌋
 ↪ X04rqFCzkKFmVZMWdTClLykkUUCL6zwdavDcHHUIIcySBPVkbSz85qprTsZzPmBMn⌋
 ↪ //C09yTAc5mf3mRrIf5hvaEXn9ht9K/VI5JXq7f9VS+lq/kvCCCR1SACJUYAKKu4HCwMnS4S9cBtc9Q1vG0xHos00YZ+Yx4pzPEe3t//7gmTpggMwYFZTBivGPA
/t/qLJUkjbRJIMkpjhstRIRvoMQsXMtCdFOeijNIfx1Qk8i1HtP1gLbCZCmOz0nZVd2JGuK1RBY2Q4kJoGZGzEV9T3Sint⌋
 ↪ //HWSvMV2FqKQ29HyT35iNy/X/uT/6pZ95aX/F+DWTEdmIFJEEwCaqTIAACoAJifxJzsocimbjIbHMmmPxICFcIASl75RTq2P6xCET5d6ws4moasELvpGX0LBAi
NtdywItZaODy81LRqd7GevrxJYulbBtRxiwocjU9bJ1dApR/JuCxOLd6YvEg3fePFzq16Rb/H9poOMV1/⌋
 ↪ NjOP8Q4UKms1ve8LRAEgDmACAgJkEAAACABqmQcBEKOGTIIjWCjLxzjEFXtAQEOgGFQzcxMmCIyM0YHRmuYh4ucNxUTTZmEsIAuQzEJZCIcXt1JAJeztiFRY0Im
/h+LaAwBCCMKJAAAAAIDQ9pqEjcXDEIVjoRolnMqy4QnNK3Lye59nqdg+2Ur06aC6LYrxHy6o9HIeh6bV5fG⌋
 ↪ //uCZMkABi1lVn5h4ACQxkoOzTwAVw0xZbmHgAoHpev3HvAARwRKnajwOftiPJ2xFgONEDoIhGK6Uux6mSnH7iyptkVkr8/SHx1Sq0fBOeBM3O2ZhJelX+y9KF4
```

```
SDTWEwsMUPDBMXK41iucFKioIzIe2x
 //+qmGAo38sp9ivrX+lT/QAqQABfB3jSEZAJRSEydhPTiIIl4Rq4YTJgMRmeDDlnHbP9G5BFQRWK52cz9HXIkMoLarTpdAaONMMQQmtjknpV7B4vXq1TrKGtuRJ
koABJF/JEgx1WKKpUJIvhQOzxJoYHmu7OfZAOD9wMDiBaXyfoQa3iRP/
 h9F3vFeZlEGVrKJJJbibKJACcyRGgH8PQKFTIFlIQM4iXYRTekTxKXaILYjmSKhQ95f2KBhlWeBXOR47VsVRzk
 //DeMZ/nP9tttZ9rjF98TjFXgVwxbTyPrmOgt1UgozD8wrCXkgoQqWAkKsLJ/QHyiyOQBhSr/flIkAQQCRkxOcinqbfHWUpjVMW0JVGNpvR5qxo4GODVnDOHOeE
JPJoB1xQO6mnA5RtUAHhgD3g5BoieWqCF0jNpA/a7REDsnLcGJ5dQeIDp5ESXtnHekFoRlb3v1jiAf9vjS/bX0giUE7ma+1
 rtH0yiw2pZIyHaWYiMVbH5hFNnQ95J/N9X+VXYCCZSG+A8CSpPh1EMUiJWyWNvwu4KxoHWDDZfRMOH0nBJHDrZsCxo8eCgyLWL+OE1T2+
 HGud/Ja9rlAAATgeNDkZbA5wZ+FquY+s9uAYmJMscEYGrejIZvrXEJ4aqim4OHGSiNvq1EYfXURz7y2ghWxRxHVWsW1Vixqv/7
 gmQgAAK2NVYbDEJAOGIayj2GOArk/2dHpE7Q6oxpxYSN2PMcuQTfP6W+9TexrWyijalpF63E3M+
 gBLRikELgLDBAZRDQ0HpWn0HhDDdMH69eiMqSeoHVZeFVRP4LDcHRISFybl/bWmxcLv/+
 ePb6HcohWrKSBSlrENAyyuG2zjXJuGWL4sKhMTrbSa2KHWMXou3e1PyDWMfFwSyoOJATbMn7We66164dShSmNpr4Jyt6o7/6
 lsZHoijtKuXYKpsgYflGWbke07oIhfIWaAyCTSlreKEuIlwni/
 VtPPUCQhWni4NG21KKMVlpex1vPsyv9VhStPtfSXcLMIqScEACImLf1gHAAIAAV3fZpQHUNHBSn/
 jA6EwE0EhDap6gD5d1HFbLGDxXUrxZWZOmriUelCN54lIMjsWdiMxDkUjKHA3/+j/q1
 ///49TEnHxoXJsFt7HyH/jK3gKPIESDaBBKxTkNOAAQqBYL8fpYE//7gmQ0hAKfP9dTDBFgOSG6qmHpGgpdG10njLGA6RTqJYYJIEvnFtmVQNdWHwy+M8C7qcQU
VNQvgDiwVda0imKDGJBGGrDcTr4fKbL2TRuzqyBnbT+de3Wzl28lCb1/QYQ/Qk0k++1
 NWltxJJJEgpyELFsFGOMk5DUMNMXiXsX4r41gslFEg1xYx0dLSq5kDtdTGc1uVFaxqNVTtrP1RPY9v+2t2fvucrY/3Gn+
 pGW6tZQW0XDYMPAEPInjf66dpgVCMRNrou1IhqzjNl4K6JTkMOhYJrqiRf47VLNf7gmQ4NOHWJSTqEAGGFAooXTAgfAqmU2d1lDBuZbUAICbmsTP
 /7gGRMAAK+
 Sdrp5hMkNqJ6hWGDKAsYq2GnpGqA3gvqZPYZCN2PWW1ZL6S4JgW1UJkTVMOmWcJU0BqBEw7lOFnofDy4CiP2KfPz4ZctQzOYEGyXgVBLmFcwJw2FVPA4DJhNRwE
 /QQAiTUNAYBnPkKToaYOYjF4WrXUEvQUdfdHZfNbBK8jnoQ+
 Sm1mmawZvQta2XQm2EgMO82sE83HHGQEruvMcG5bO1ez919HVXowDokYLjjevWakx4qmLSORDNIY9KOxHyo/6
 o6zayDkq5RvorBtv10ZRb35tN36u6CLvMCpEMyJqpo5580SLNcW5XNtikCsoAAAAQKzqeEmKUDsgKhrCoDcZSP7GUP8Ug16uprXM/OqdUR/X
 +kQkuk46wEzu99H/1f+qBBwAjSRRJAATu5J0EWMaqWTB4DM8lDD108H0wRNMu8KoZSLf4TatyzKPpnFPPPKBzupGz
 //uCZGEAAsU+2GsJKkA3hIp6YYIaCnypWaewZUDsBap0tiQYigogRICh2RjGd7kE2RiHVufNOPKS6ws09JNIqJ4mV+MHkQEXUiyAgVLgLxIFI8AHCHyuAgvgQHg
KVT
 ↩
iQw+jpsedYTNOTSUOrAPdsSqGmQc68qm7Fbq1WxUmzVvXzgixv58lvf/4H7sps+
 GPd9ElCpq24kgABdwAwPgmRAqMi6TjEsHy48RasIOQHzK6OOsaSnnPatGraWbRit7LbBdlIub9udyXBxhcb
 ///4/uymc9YYT3dM1JIRAAFAPoXAS5ECfIJQm4hxMDic0uuXF4upsOTZjdIvkXfr/7V8izLjdRnm//uARKYAAj8pV+mCLMRIJWqJMGiGCSSlWaeYrtEeFKs0wpp
HeJIJGy9+HfBqElX4kUXa/
 bcOIrlGacbpphOacK4CcYYbbuHdUSRNulXBp38ulYhioZ2Njeq19CxGr6M7gxx833SPm1tYtn6fyq9ndb8dugmilGZmxSG+nxVnL+
 ZaUhMhl3vt6+e2ktfGfNjX3vuEigQuPiJDjsjzM0HFcZ1T617f
 ///6ve7x5SUORgjIhI2Vy1s3/+4JkUgAFIVNfbmHgBFzlS0/HrAANLStlvZKAAMAF7vOYYAYwA5o61LTBGRhEoBSiBABwCoDUHyzk+YCERFbJK6WEI8U/hdA2Ow
SpyQMlIFAaQECITajj6OR0LFWeXR3VaLEwAM7PRKHNYVcRCxxt6H3/YFdRUXZJEkCsLTH/6uzAmc62kO/
 Vya1VL9Lhcfau91xKAS3jIAAAACoJ6JTZjDjCog5ktZAu1ibOmUs1ftxaSjCUFIiDxTbyjtbaSH6LkKbc/pMx9qesff/7
 gmQbggOpOdhrBlvgSYlLXT0lVo2VLWOsGQ3A6BSsvPSI2PSmFq2gHt2gB3PIWQxx73yTiIQBq1N6SEVB2Ip8bkV9M+
 h7OJilB6xAFDR9SjbnPiEXWDJsFijN9/vDdXYLByTORtMkpCBkWCqLMR06xno5rPsyB6QsfwX2BNr5F1I8hBP1KzYqcouq/Xtt3ydQ9bK/
 Qo0JmdGWln3s6pmIv91
 ////+aMvxX0AKNVJIAzDClLF8vxBbwV3ouPZEX0guNxwRHtAkHUBZR5hnnESmmU0AOFb4mUtdm+FbOFu8q0IvyMAbdjzHaFIWBIOtafSplZd7WUmJm544a9q6/u
S5Mv/RgzcaTIRWMDL3O5/mcSaje9Vx8vCo5a0A2kQA6yEmJkYwhBMyIaWJNIerS+x51D6v
 //Fh2qaG9CTpdfmfHUeDdYadehn////8r//5KhBUh2dVIpAAvgRRG8bCwvhudC40iZGBUmIoRYbVWf/7gkQPAAI5C9l5jEiQRuQ7TTAilgkoSV+nmGcBIhSstPC
BclOSf4wglailFBXLdpbKPU6So/8xt9lWputrAPL/d9UYVUkagwd9DiRNIrpM9o0YkX1mHP/SgJmBCaAAsZSQgEA2xyexkaM+0
 ec59ngZcBUsHuSFGyr5VV9P9
 //uCZDwAAr4x2OsJKrQ0RTqcYMVGCv19X6wkqRDbFKs9gAoQ5k2Ruv12LtuMR0K+Mj4uEL+jT5YAitpskkApyPuSJQ+Y6zCkfKINzRTIASHBUSEqestJDqqbTK7
pjm0dIO0UBH1SiQC5AOIDuJ0Nce0hu2mFqYxKjzkRA+JHMaIPe6kMJdg5uzWuz4ZTtq0nIj2zsS9GuYzJf7/a/IrnGV3Ywl6HD
 //uAZGkAAtErU8tGS3A1RTpAaGZeC2EXY0ewQ1jbhes1h6QAhyMQ4RKDAZjBwQsAKOYgmeqenf7CDf3/YBM1OrYID2AipCwU01RMXCBAQjxdYjVP9PFEWYmI3VP
ZEeLJqzyT6a1UhndzMvsVat1pLZ8i9e12RppC4MKDzvmA0WLmBEyinv6v0SX
 //7EUA+rkSSW7/+4JEk4ACOhNZ4ewY3Efm6v1hIhwJPDVTrDBjQR6U6zWWCHhF6igGBkx5DjB1C/kQAZJiwZSQERZNyGRNeic7TNuu+/dfRF3RiaMM1Y0roqJ4F
8D/mTfdhlulTIsHgdW/PU/8ef8KMn33f13mz0GGqu+R8qzCar/+4
 BEvIACTCrS0ykpwEDB250lhhmJCKlCzRlvwR4VKWWjLfgJOKN6NdOAAXWWICAdgEszpgxAmQEpBLZfR02sXGm0E7C72UhixhahDs4G2yi0DET
 /mmWZhBq+rsDOIiJVqaAKcd9ylyn
 //oBCT8LZ9FYRBIZDYjVZRXjhCq08YyJiFfymJJXmUOKigRItkXLChFK6jT6RTaJQxTp0ERZKAbBBr2lN1+nmWuS6lommJpo/vNjLsaO//bFOux0Vrwlrc9KpX9
Q9JSnq4RZFOUe1XRzIunq9/f/7gmR/ghNbLthXaQAALiIrnOSMAY6ZLV+sGQ2AuY1uMMYM5k3ce8PnAGwIqBsy+
 K4n5NKybSSfwUiwYFFJEGJieuEshGA/BRzNljr8i9XNorChB1tzhYR1tUo1Bill4gE1Ac0qQAAAAL5vkLVO0eB/m+
 byROI1VGrEOPNFrK3wcmVBd0bsW2VrnQizxf1zAb979PTZ/hz1l5RPMVZrUSvP3fctLySump39qwLkB4qsMEQZawKho+
 RMuaPtVh8GRj1zl77vrUeEo1gCUOAgQgEAICQgaRvEOgSMrAXVhjGanBMUN9db/S+OA/Epdz872Q6hmKgVCbNJCd6f
 ///6QFM4mEAHjGhL9UaUL+MBCYWAVSDyWYS2XiWftnJZD5qifcQcv1apIzmNbJ5CR7bmxsygxeviUvdEITAqA6O26rsbDeySXRrWS8zaOHPNY31/XW4ImXIadPD
uCZIwAAylJ2HsmG7I0JHu9PSJFjHjJXawxBsDOEGz88Ymg/Fmky3rbWUFdk06vs57f9R1bv/77f
 //+pSa5ZGkSQSU6LcJ2ZxDSBE+B+mwym+cR/HIT7qBiYwGJIARZ87uycXKhFx6il5FH9NYf/ywfhkswYVLlFpPlRQouYmiTnJaS4MDziFmHd+s7Sa/8V3VpADGI
I2S7SlbaeYTLDNFKt08ApIecZnTYFEyXI0MkTEpKS7jxeyGeYaCqejkhiPX+sL+
 q2kAEDE7MAEBAFgRUDC6S3ALcU1njadnymJbBDUfRCB1P5cllHoVPt/9rA2MFG+MX2k0OGXOaFbFcIwYS2D+
 JwVSEFwVilJVkGWshKzjaocrKHqkY9GKtgskW6AxrjzAAs/ql0dysBke1a53vWtVJBoBrW9SCCfQnXzMnoNi6ig7w/633L+waAgQM/2
 AShABYwijtTTinyYjPZY+84Bkc81I9jPCpOWTTDiDY/Iop/gRpFWwkrq
 ////RSctBNs00VHeXwc53GuX0ow0j2Ow8TApIIC3hBlw8bFCOJ2DK8F1r+N29MZJnHhQlTZk9TFdp9qplO6xYKBOL0+pWeoQ5jKikCg6dGkZhA8Xb0O6Bpnjvc9
```

```
afmZkImQBRMzPa0UpNk3T40M5nne2zFnDVAESmLUlkO7dpqNlMp/Bsqsw1lYVTltmB0VoOks+
 ↪ cZn3Wb6g4QF7GGnXxJ0V1dB7jjkc7UKmz24Pcyz2hf1VFwxt52nL+TNaViof4VFLnqX6S+
 ↪ KA0oIJWPKEy71mK9geXoFE9LJoPpWZAQLGPxSIAgEcEboWinTDCB620RY08MLk/iTsH7BRoPE2DMnhWWRJRme6o9a778Qp2mq5P
 ↪ ///QSiehAYaGGqLAVl4tQf+SKjC6gTQCeIuXs5yR0uAgHAjghc3yvKhuo1Ckrwv01FJd92dSs9olyOoR8bP+3eY+sInu942VbBwYMZX9NPEs9a50+R3t7KmoRpS
DDXdDv
 ↪ //SBoBEZpKNtBa7sLYYh+x5/Xef8f4qq3xCE0NnRJH2FOa/UBhHJCGUjMc+uDuyu4iilw4Cdipj3dzEEoVYdSPVqmKwQUlPqcQ3svS9qbM66U+kCMZWGFuqm9NK
6IAChufUjCi+lAaaRBXFY4oXISECM1NzRWyohw5BRls+1YcxRkZS6uSxwjIX9twxyHYAFKix7o4gxThkr977aE9aP/T/8hnR1Ih/
 ↪ Buyqymdk9lZrblZD3zkR7apM4M5QMoT6SRYUGCKKAtXzCM0msWTworXptMqaBEt+ZO75TaG4vXwk9W9LKzvIJP93
 ↪ ///P6knJgeAASEe0bQWv4jhEVQwOBWCw9lsdxCILR6LiWuYs6vTHz25CQO3wZUYSL3UzLNm1GM/JLomqOt1dlyBZi//7Pv8rXR1/tt6oylu+zOhGmEjs7uXoler
uP
 ↪ ↩
Nw50qlgIFAIjigJEoomUEZXUWSBRNA3o81qpOu0XRKn0B3U0kaikW4GYIgiAo9VaJghlFKK3z1KRodNMNlGHYnNWhQ9eY5xzpnz77Of
 ↪ //58w4kRwIGqJ17nfZ/rD1TGLd7/29pKO62XMRvOm69xDjXng/DE/O0iua/edSZ2/Uc4GwUov8J9CVzRy7/eZ+1HVhMFSAk6gAAAAVexC0C3D2NcxxknMX89Cd0
9uWOpwuEj4AYMDY6QSWEJY2tz0pjjpNQFMpUQdWi
 ↪ //YES4EADQjTgq4NrEwMlqDdvmaRvO339l7JnvNWaNBW5sbhkVW/uwgLm2DXOcLPBxLuS5CpFmRIABKcwWAl5Ul7bRN3jWZKVMdMIxZKaAyyri2v5RrUNLKFvBJ
OlsS3JHQ0qpWsD7mrYygtGh5Cjqp2+xHc50MaeQIR7q8pOQMGNwM5bj6Bksj0aHC3dnntyNaf/Z1kkKEooEEECSweIQCCgbhNxB/qgsZoA6vP1Cz
 ↪ +F5vGRgB/WBzrj/2OOqOn5O4ayKDv
 ↪ ////rdItoR/0IAKLEkKXsNTpSLYfCILHzx5/G3bR4oIU4f8yD5OoL698WhSHadI3qo85eGrZ/lTT7CLAfyJhJceVtQRBxOIqE2P2sVUeeVQVAY9G9bcKsqlf9g5
r/q7AAhWAABbvPIJOcB0CpegLp4gMEihsCj8RIvkhPjNyFcycM9UlfyOKak4fpRWM8/8x9nJ4gxF9KaZsHgb/J9EXR7P/wDJfoph2gBIIADcD/
 ↪ EpU+WWIQOwYGtqCR8kRgOR1OE7vm7ldsd3Jwz1VU80aKao4fpNYw+enmPsZPoMRZUZtHOA4C/yaMlEpT2f/
 ↪ jVSAAJAABd3iKZYKTL6JsKLk8zB14ahMfZPR4jrGIil1kijGVEZLa2D3p2Z+caae/z7zqmP/7
 ↪ gkSoAAInF1rR6TGkSOVbajzCeIkgrVtHpGdBFhWr3YYM6CYSh/
 ↪ wWsR3Btf9HsadrDPzjXoaihIUCoKWrfJYFcxIftpakqAMCKiVAOMUDS6rBtKmg51GbaYPFSviQ+iN2+8+
 ↪ y7EqFVGCgqhA0JrhhhhBONLGgDMlyAUR/+oQV2kSSnJdEbFwEkUId4hD8kCadKUnE7Ysoa8FDz8uQfpjCr0l5ujXsu1aPl9NsPR5EEIhT/
 ↪ CHX/p9Kp9qNL++j6KQglYebPLbQbPu3eoS0qLATsG1Aq4OkSvSLUGQ8S/WI7bFjYlATWsoDHr3HLUwqSrr/s2772g2e/
 ↪ VbupQjq06jvwT60ygABuaRKkliTd2ksoiKyxosshh+EYJgTm4PJvMIJNs23P6tV42SeNqvhPUufqynFnfotnEHPRhApjU0sEgtRyf7+
 ↪ dNp3lMh5szJt27KKkOcZnovpGcnJGDhtKqIslIAAALtQAgKkoD6IWf/7gGS+
 ↪ gDK1KtfTKRsgNqGqlWXmNAqlCWlHoE7QxBJqlYYJIQYYAPOUfAtIoBWrGGacTtxYlWXIFLr+uv
 ↪ //ojtZnKh0KWutwYK//orEABwAAGXd+BAdDoDERZWFAC1yBGoKZv4LtnDIkhpubmQOnrEoLWxawppLciv0Z2/2n+ZuVJ4GP0zDDT/9CzcBrPFpUkl+JV9lRS0Bz
zieSezFKdTWXuB1eXt9taXo9Bf5a199JW1sFcAO/1bBWpASukkSnJU8OcapKE2XYX6OOpEtqwm3JjQL+j1Y1Ozu/COYYrVjUJ5oZFu9V0/
 ↪ yP1K1tN7hFcN+jbGnTWs6tOleksfSQXn548P0YTxAl80YA6h4fQAVYkgFtx9guEeyXUU2YkklEp94aF5ZVm7cUh/
 ↪ VFYx21d1hCmgIa1Y1nsjKXe5iPUnfqUjleTe4Is3/9qV
 ↪ //uCZNkAgulC2dMJKnQ0JUrcPSI7CrSlXUwkaoDllOr1hIkw1spZhAMmRp1af5YH7F1H548P0YTxA/Qg3eTAAJSdRJALd7wOFYEMB5Oh6k8ZbS8L6hrcvQUQ/Y0
WQNRfmYeTyNVop/lIihMcT
 ↪ //3oH2UgL//9SgBgBLlIugyMTJK0xdHtgYIEgZpB3QQeGxQtiBf5XFHqNJDG94o7n6wZl2nuKDoqZeoi5EE23hUFEc+lkd5QYCOzqdBSDi3rPnycz7vk9d///7/
mxrQUmTiLGFf/6aZcZaHSlgulM84MAZ
 ↪ //oRbcbTJAAAUsGDR0saocqZV0y5it2Ygx5ne3GZDECgYlYc4AIu5bVnSfdzO/aMiz5xbO8+cx8KRQJoEVP23daN/iiTPEakdW22t/+7Vr/Vt+//x8+UdWbP7dk
jQhgs1FIfGkbLhXTUtoq7PP4eReqpncG2Rf/LvSFBERPeSJPXIxF0cuQUQBIAE+VEv2kE3zuFV4nSNOG2fSskG3o/hdkjLnShl/i0dITNq/
 ↪ Wd2zHds2p8f2hfHVF91zKG09NvurWaK3lv/tVpQK8vS/ZIzc6cUpWTla+j/+4
 ↪ Bkf4IDsj9YUwljYDZkGthhgzYOiP9frDDRwRgfrPWHiGL683nzbGjddkYnq81/57Zt+NcVOEAqefY8Uvbk/1
 ↪ K2MuqQkVDCoGkSnACqSLLthDGGwBnZ1IDV67PxDk89Ri2EgIVPmMp1f7MyMKexXT9sExvun1SpzIhjM37Vs2aa+22
 ↪ ts6ID40hkEKSBtIAABdJFuq019HNIFpsXV8r322Rm/8
 ↪ r7giVS6G4dhmJcooVHGhg8o5E6vGpOdJ6ivrCVTxmlYSm0io8LFj6iDilkfDxDECPfy0i1L4S2qfVZTtrBNbkHdvroVkZW/6f/9
 ↪ a0Xd25xGoYUAyJc+QLARYSxOku5ghhVyNizxNHzLwMF0RYVi5o5/zvRAohSue+h7FRpv/csMl/
 ↪ MtGsdQS7mS5mvsVr6HSQZATyesvKOlxODhRqFIvlV9KwpC+1jzc03li7fH2ETtaLQunyNernvkyjEcX637o3kgRRWGdr4mf/7
 ↪ gmRwggNRYNlTCBTkPSQK7D2COg1Fj2NMILHRBQprdPSJgO3rMq1dtM4Wbvjdu01ou9JZtF3ViOoQdl200syroR1TmftJ
 ↪ //VM46RSZ2MpkJaQSKCAQIoWgEVKEUhRO2l6FMBbccxeuSGS4F9LgYtuMnuAKtDXhiDnvEHTJsKBoXBCEUoQ6CrtneGFlvpcJOtN7K2/TXKWxFEyiSneouNMGou
+3Ts+HrWlKy5SAqa9Rx3r0pEv1RfGZJlGA6k8X5FpCGkrgELGk/Z/1qEMBJtEgFO6SrokKxuoeFBMGlpN4Gofbh/
 ↪ EXRDxfEcctv6UIFOvefwCVTUaj33nn/qbuXP85S6zCEWkevfhZX
 ↪ //7gmRsACMxY9vrBhQkMgJa3GHiNgxRNWmsMGkQvQUrIGwsSWuGQKIy/ZZm7NTNjlz+/9ufV5r+uHoTDB6MCXZ5E+IEJkywIAHwpMJJxaTi08jsJi4HMPutC9jy
S3KKXtjwg0qwXQXfWYqUi1NXmP/U6jfD8X0YyhXLlP/
 ↪ MCOiTCIjUdOK6g4L8GuLsRXVP9EIXxg5wNkEkgSQDOIBhNRJB1axuj0aF4VUz2VEAA0leSXsP/7
 ↪ gGSQgGLCH9bTLBqkNcE6iT8vEArAxVtMpK0A1YrplaeUmnh8K1LIXuxLc4PU6fp91I7h/QhFL5rYlc1fABBwoY7LVDCg5+
 ↪ z5kTMSIlCCqcCUgYQWARILGKCYsll7SlLkxYPrdJrF8Ig4Iz3UkANgChAAAK0QB8BPJMIMoLyoZFbhABU6bsjNg440fZS+q4+
 ↪ dejBXO53Y7kv+27+KhhWiKYWEYSOyBQxRJJEpOWDUJys48/NW9l9BIl1VXmlEkd6X1sp6VUmF2llI4ZMIgR4HT/
 ↪ Jb6I5Qvt3n1Fc86gm9gTAyMTddu9V9nsrdveQ6+iq88qSImzkLCrYhrTH2NPJ2HGiAWn1lw1GIoYoB6cF+pbD2E8ZaiR7T+38yctX6ix5+Z/
 ↪ vUPs5CSlY8JMJAp49T+6j99
 ↪ //SoyByZVQokAm9zDJdzW4LgNZrHtvKuVTaUTjuu7IoxdLwGIJtJTZHY+ohUNGlY+vha29emm+r//uCZKgAIowpVisvGcQ34WqtMwkSC80rbawMUVDIkynBhgyY
VVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVVUABCSgAAAAACAL
 ↪ //uCZMAAAuZLWXsGE9AxpVqcYYI0CuD7Z7TygADNFOq2mCAAUgbp2wGkA2FMjLBCQKCdHAvgU1FsD8PEbkkvLBLRiDBq1+FSBahghPg53/h1AygACgBCASweutb
qqqq
```

```
const VOL = 0.5;
function initAudio(){
  if(actx){ if(actx.state==="suspended") actx.resume(); return; }
  try{
    actx = new (window.AudioContext||window.webkitAudioContext)();
    aGain = actx.createGain(); aGain.gain.value = muted?0:VOL;
    aGain.connect(actx.destination);
    const bin=atob(LOOP_B64), u=new Uint8Array(bin.length);
    for(let i=0;i<bin.length;i++) u[i]=bin.charCodeAt(i);
    actx.decodeAudioData(u.buffer, b=>{
      const src=actx.createBufferSource();
      src.buffer=b; src.loop=true; src.connect(aGain); src.start();
    }, ()=>{});
  }catch(err){ actx=null; }
}
function setMute(m){
  muted=m;
  try{ localStorage.setItem("cavalier-mute", m?"1":"0"); }catch(e){}
  elSnd.setAttribute("aria-pressed", String(m));
  elSnd.textContent = m ? "♪ coupé" : "♪";
  if(aGain&&actx) aGain.gain.setTargetAtTime(m?0:VOL, actx.currentTime, 0.05);
}
function duck(on){                        // totality lowers the music
  if(!aGain||!actx||muted||ducked===on) return;
  ducked=on; aGain.gain.setTargetAtTime(on?VOL*0.42:VOL, actx.currentTime, 0.35);
}
// Fullscreen: a button and the F key. Starts windowed; the launcher
// (osmo-dino-play --fullscreen) opens with #full to start large.
// requestFullscreen may be refused (WebView without permission): the "full"
// class is then enough, the frame fills the window.
const elFsb=document.getElementById("fsb");
// Two possible hosts, and the page cannot resize a GTK window by itself:
// it ASKS the host through a WebKit message ("osmo").
//   - osmo-dino-play: puts its window in or out of fullscreen;
//   - the desktop widget (osmo-dino.py, 400x260): opens the game large in
//     a real window and keeps its own aspect.
// Without a host (a browser), the Fullscreen API does the job.
const HOST = window.OSMO_HOST || "";
const hote = msg => { try{ window.webkit.messageHandlers.osmo.postMessage(msg); return true; }catch(e){ return
 ↪  false; } };
function setFull(on){
  if(HOST==="widget"){ hote({full:true}); return; }      // the widget stays a widget
  document.body.classList.toggle("full", on);
  elFsb.setAttribute("aria-pressed", String(on));
  elFsb.title = on ? "revenir dans la fenêtre (F)" : "plein écran (F)";
  if(hote({full:on})) return;
  try{
    if(on && !document.fullscreenElement && document.documentElement.requestFullscreen)
      document.documentElement.requestFullscreen().catch(()=>{});
    else if(!on && document.fullscreenElement && document.exitFullscreen)
      document.exitFullscreen().catch(()=>{});
  }catch(e){}
}
elFsb.addEventListener("pointerdown", e=>e.stopPropagation());
elFsb.addEventListener("click", e=>{ e.stopPropagation(); setFull(!document.body.classList.contains("full"));
↪  });
document.addEventListener("fullscreenchange", ()=>{ if(!document.fullscreenElement &&
 ↪  document.body.classList.contains("full") && location.hash!=="#full" && !HOST) setFull(false); });
if(location.hash==="#full") setFull(true);
elSnd.addEventListener("pointerdown", e=>e.stopPropagation());
elSnd.addEventListener("click", e=>{ e.stopPropagation(); initAudio(); setMute(!muted); });
setMute(muted);

// ---------- input ----------
const keys={};
function engage(){ initAudio(); if(S.mode==="attract"){ start(); } }
```

```
function jump(){ if(S.mode==="play" && S.z<=0.001){ S.vz=8.3; S.z=0.001; } }
addEventListener("keydown", e=>{
  if(["ArrowLeft","ArrowRight","a","d","A","D"].includes(e.key)){ keys[e.key]=1; e.preventDefault(); engage(); }
  if(e.key===" "||e.key==="ArrowUp"||e.key==="w"||e.key==="W"){ e.preventDefault(); engage(); jump(); }
  if(e.key==="Enter" && S.mode==="over"){ initAudio(); start(); }
  if(e.key==="m"||e.key==="M"){ initAudio(); setMute(!muted); }
  if(e.key==="f"||e.key==="F"){ setFull(!document.body.classList.contains("full")); }
});
addEventListener("keyup", e=>{ keys[e.key]=0; });
const frame=document.getElementById("frame");
let pointing=false;
function pointX(e){
  const r=frame.getBoundingClientRect();
  const cxp=(e.touches?e.touches[0].clientX:e.clientX);
  return ((cxp-r.left)/r.width - 0.5) * 13.5;
}
frame.addEventListener("pointerdown", e=>{
  initAudio();
  if(S.mode==="over"){ start(); return; }
  pointing=true; S.tgt=clamp(pointX(e),-6.4,6.4); frame.setPointerCapture(e.pointerId); engage();
});
frame.addEventListener("pointermove", e=>{ if(pointing) S.tgt=clamp(pointX(e),-6.4,6.4); });
addEventListener("pointerup", ()=>{ pointing=false; });

// ---------- game cycle ----------
const elStart=document.getElementById("start"), elOver=document.getElementById("over");
// Each game opens on a RANDOM sky: the time of day (where the Sun and Moon
// are on their ascent) and the day (weather and season), but not the phase:
// jumps are whole lunations, the Moon stays today's.
function tirerCiel(){ S.skyT = MOON_PER*Math.floor(RND()*40) + RND(); }
tirerCiel();
function start(){
  S.mode="play"; S.dist=0; S.speed=9.5; S.lives=3; S.invuln=0; S.cacti.length=0;
  S.spawn=1.6; S.px=0; S.tgt=0; S.flash=0;
  tirerCiel();
  elStart.hidden=true; elOver.hidden=true; paintLives();
}
function gameOver(){
  S.mode="over";
  S.best=Math.max(S.best, Math.round(S.dist));
  try{ localStorage.setItem("cavalier-best", String(S.best)); }catch(e){}
  document.getElementById("finalDist").textContent=Math.round(S.dist);
  document.getElementById("bestDist").textContent=S.best;
  elOver.hidden=false;
}
const elCard=document.getElementById("card"), elCardLvl=document.getElementById("cardLvl"),
      elCardSub=document.getElementById("cardSub");
// No levels: the eclipse is a spectacle only (sky slows down, corona unfolds,
// cacti stop during totality). Conjunction recurs every MOON_PER days without
// resetting the sky, so phases never jump. Only metres count.
const livesEl=document.getElementById("lives");
function paintLives(){
  [...livesEl.children].forEach((n,i)=> n.classList.toggle("off", i>=S.lives));
}

let lastGap=0;
function spawnRow(ramp){
  lastGap = clamp(lastGap + (RND()*2-1)*2.6, -4.4, 4.4);
  const p = 0.34 + 0.46*ramp, half = lerp(2.5, 1.35, ramp);
  for(let lane=-6; lane<=6; lane+=1.5){
    if(Math.abs(lane-lastGap) < half) continue;
    if(RND()>p) continue;
    S.cacti.push({x:lane+(RND()*0.5-0.25), y:96, s:0.72+RND()*0.43, a:1+((RND()*3)|0), hit:0});
  }
}
```

```
function update(dt){
  S.t+=dt;
  S.sky = computeSky();
  S.wx  = weatherAt(S.sky.dayT);
  // On the day of conjunction the sky slows down while the two discs overlap:
  // it is a game, the great eclipse must last.
  const dEcl0 = Math.abs((((S.sky.dayT-ECL_DAY)%MOON_PER)+MOON_PER)%MOON_PER);
  const near = dEcl0<0.5 || dEcl0>MOON_PER-0.5;
  const rate = (near && S.sky.cover>0.9)
             ? lerp(1, 0.25, clamp((S.sky.cover-0.9)/0.09,0,1)) : 1;
  S.skyT += dt*rate/DAY;
  // storm: an occasional lightning bolt that lights the whole scene
  if(S.wx.storm>0.5 && S.wx.rain>0.45){
    S.boltT -= dt;
    if(S.boltT<=0){
      S.boltT = 1.1 + RND()*3.4;
      const bx = 90+RND()*(W-180); const seg=[[bx,0]];
      let y=0, x=bx;
      while(y<HORIZON-6){ y += 14+RND()*22; x += (RND()*2-1)*26; seg.push([x,y]); }
      S.bolt = {seg, t:0.34};
      S.glare = 1;
    }
  }
  if(S.bolt){ S.bolt.t-=dt; if(S.bolt.t<=0) S.bolt=null; }
  if(S.glare>0) S.glare=Math.max(0, S.glare-dt*3.4);
  // --- shooting stars: on clear nights, one now and then; on the last night
  // of the year, a shower.
  const nightU = clamp(-S.sky.altS/0.20, 0, 1);
  const yT = yearT(S.sky.dayT), frac = S.sky.dayT - Math.floor(S.sky.dayT);
  const lastNight = Math.floor(yT) === SEASON-1;
  const metRate = nightU*(1-S.wx.cloud)*(lastNight ? 1.8 : 0.30);
  if(RND() < metRate*dt){
    const dir = RND()<0.5 ? 1 : -1, sp = 520+RND()*420, tilt = 0.35+RND()*0.6;   // slanted, downward
    S.meteors.push({x:RND()*W, y:RND()*HORIZON*0.55, vx:dir*Math.cos(tilt)*sp, vy:Math.sin(tilt)*sp*0.6,
                    t:0, life:0.45+RND()*0.45, len:70+RND()*90, w:1+RND()*1.2});
  }
  for(let i=S.meteors.length-1;i>=0;i--){ const m=S.meteors[i]; m.t+=dt; if(m.t>m.life) S.meteors.splice(i,1); }
  // --- Santa Claus: on the last night of the year, once the Sun has set, he
  // crosses the sky right to left, in the snow.
  const santaOn = lastNight && frac>=0.77 && frac<=0.93;
  S.santa = santaOn ? (frac-0.77)/0.16 : -1;
  if(santaOn && !S.santaSeen){
    S.santaSeen = true;
    S.cardTitle = "Joyeux Noël"; S.cardSub = `an ${Math.floor(S.sky.dayT/SEASON)+1} — la dernière nuit`;
    S.cardT = 3.6;
  }
  if(!santaOn) S.santaSeen = false;
  const ramp = S.mode==="play" ? clamp(S.dist/2600,0,1) : 0;

  // Total eclipse is a spectacle, not a level end. The countdown runs in every
  // mode: the sky does not pause because the runner fell.
  // Every noon the Sun crosses the Moon; the GREAT eclipse is the one on the
  // scheduled day, when the two discs overlap most closely.
  const dEcl = Math.abs((((S.sky.dayT-ECL_DAY)%MOON_PER)+MOON_PER)%MOON_PER);
  const atEcl = dEcl<0.35 || dEcl>MOON_PER-0.35;
  S.totT = (S.sky.total && atEcl) ? S.totT+dt : 0;
  if(!S.eclipsing && S.totT>0.12){
    S.eclipsing=true;
    if(S.mode==="play") S.invuln=Math.max(S.invuln,1.2);
  }
  if(S.eclipsing && S.sky.cover<0.96) S.eclipsing=false;
  if(S.cardT>0) S.cardT-=dt;

  if(S.mode==="play"){
    S.speed = lerp(9.5, 25, ramp);
    S.dist += S.speed*dt;
```

```
    let dir=0;
    if(keys.ArrowLeft||keys.a||keys.A) dir-=1;
    if(keys.ArrowRight||keys.d||keys.D) dir+=1;
    if(dir) S.tgt = clamp(S.tgt + dir*9*dt, -6.4, 6.4);
    S.px += (S.tgt-S.px) * Math.min(1, dt*7.5);
    S.spawn -= dt;
    if(S.spawn<=0 && !S.eclipsing){ spawnRow(ramp); S.spawn = lerp(1.35,0.42,ramp) * (0.8+RND()*0.45); }
    if(S.invuln>0) S.invuln-=dt;
  } else {
    S.dist += S.speed*dt;                      // the gallop goes on, idle
    S.tgt += (0-S.tgt)*Math.min(1,dt*2);
    S.px += (S.tgt-S.px)*Math.min(1,dt*4);
  }

  // Jump: vertical integration. The shadow follows by itself: it is computed,
  // so it detaches from the feet and shrinks during flight.
  if(S.z>0 || S.vz>0){ S.vz -= 19.0*dt; S.z += S.vz*dt; if(S.z<=0){ S.z=0; S.vz=0; } }
  S.lean += ((S.tgt-S.px)*0.7 - S.lean)*Math.min(1,dt*6);
  S.camX += (S.px*0.55 - S.camX)*Math.min(1,dt*5);
  S.gallop = (S.gallop + dt*S.speed*0.19) % 1;
  if(S.shake>0) S.shake=Math.max(0,S.shake-dt*2.4);
  if(S.flash>0) S.flash=Math.max(0,S.flash-dt*2);

  for(const c of S.cacti) c.y -= S.speed*dt;
  for(let i=S.cacti.length-1;i>=0;i--){
    const c=S.cacti[i];
    if(c.y < -12){ S.cacti.splice(i,1); continue; }
    const ctop = 1.9*c.s;                      // cleared when jumping high enough
    if(!c.hit && S.mode==="play" && S.invuln<=0 && Math.abs(c.y)<0.85
       && Math.abs(c.x-S.px) < 0.52+0.42*c.s && S.z < ctop-0.55){
      c.hit=1; S.lives--; S.invuln=1.6; S.shake=1; S.flash=1; paintLives();
      S.dist=Math.max(0,S.dist-8);
      if(S.lives<=0){ gameOver(); }
    }
  }
}

// ---------- rendering ----------
const elDist=document.getElementById("dist"), elPhase=document.getElementById("phase"),
      elDay=document.getElementById("day");

function draw(){
  // --- sky: the bodies depend only on the time, never on the runner
  const SK = S.sky, lS=SK.lS, lM=SK.lM, altS=SK.altS, altM=SK.altM;
  const kMoon=SK.kMoon, cover=SK.cover, total=SK.total, elong=SK.elong, dayT=SK.dayT;

  const shakeX = S.shake? (RND()*2-1)*6*S.shake : 0;
  setCam(S.camX + shakeX*0.02, S.px*0.35);
  const hz = HORIZON;

  // Point sources orbiting the dino
  const ORB=[S.px, 0, 1.20];
  const LposS = add(ORB, mul(lS, R_SUN_W)), LposM = add(ORB, mul(lM, R_MOON_W));

  // Same rule for both: only visible in their RISING quarter, ahead of the
  // dino, from the horizon to the top edge. The Sun rises and exits through
  // the top; the Moon does the same 1.2 times faster, but is only visible at
  // night, revealed by a fade-in while already moving.
  const pS = skyPt(lS,hz), pM = skyPt(lM,hz);
  const rS = R_SUN_PX, rM = R_MOON_PX;
  const eclUp = clamp(altS*5,0,1);                    // the eclipse only darkens by day
  const ecl = (cover<=0.5 ? 0 : Math.pow((cover-0.5)/0.5, 1.3)) * eclUp;

  const cl = 1 - 0.80*S.wx.cloud;              // cloud cover dims the light
  let dayF = clamp((altS*180/Math.PI + 6)/14, 0, 1) * (1-0.95*ecl);
  const sunI = altS>0 ? Math.pow(Math.sin(altS),0.7)*(1-cover)*cl : 0;
```

```
const warm = mix([255,176,52],[255,232,120], clamp((altS*180/Math.PI-2)/22,0,1));  // yellow Sun
const Isun = altS>0 ? mul([warm[0]/255,warm[1]/255,warm[2]/255], 1.05*sunI) : [0,0,0];
const mi = altM>0 ? 0.34*kMoon*Math.pow(Math.sin(altM),0.5)*(1-dayF) : 0;   // moonlight
const Imoon=[mi*0.86,mi*0.93,mi];
const ambL = (0.17 + 0.40*dayF*(0.45+0.55*cl) + 0.055*ecl) + 0.55*S.glare;            // the corona gives a
 ↪  little light
const amb = [ambL*(0.92+0.10*dayF), ambL*0.95, ambL*(1.20-0.18*dayF)];

let zen, hor;
const e=altS*180/Math.PI;
if(e>12){ zen=[78,132,196]; hor=[176,202,222]; }
else if(e>-2){ const t=(e+2)/14; zen=mix([36,46,92],[78,132,196],t); hor=mix([228,150,72],[176,202,222],t); }
else if(e>-14){ const t=(e+14)/12; zen=mix([22,26,54],[36,46,92],t); hor=mix([72,48,78],[228,150,72],t); }
else { zen=[14,17,38]; hor=[34,32,58]; }
if(ecl>0){ zen=mix(zen,[15,17,44],ecl); hor=mix(hor,[104,64,72],ecl*0.92); }
if(S.wx.cloud>0.01){                          // overcast: the sky closes
  const g=S.wx.cloud*0.85;
  zen=mix(zen, mix([54,56,70],[146,148,158],dayF), g);
  hor=mix(hor, mix([44,46,58],[176,174,178],dayF), g);
}
const g=ctx.createLinearGradient(0,0,0,Math.max(1,hz));
g.addColorStop(0,rgb(zen)); g.addColorStop(1,rgb(hor));
ctx.fillStyle=g; ctx.fillRect(0,0,W,Math.max(0,hz));

if(dayF<0.72){
  const a=Math.pow(1-dayF/0.72, 1.4);
  ctx.fillStyle="#fffbe8";
  for(const st of STARS){
    const y=st.y*hz; if(y>hz-2) continue;
    const tw=0.78+0.22*Math.sin(S.t*1.7+st.tw);
    ctx.globalAlpha=a*st.m*tw*(1-S.wx.cloud);
    ctx.fillRect((st.x+S.camX*6+W)%W, y, st.s, st.s);
  }
  ctx.globalAlpha=1;
}

// --- auroras: waving curtains, green below, purple above. Clear nights
// only; daylight and clouds extinguish them.
function drawAurora(){
  const a = S.wx.aurora * clamp(1-dayF/0.45,0,1) * (1-S.wx.cloud);
  if(a<0.02) return;
  ctx.save(); ctx.globalCompositeOperation="lighter";
  for(let k=0;k<4;k++){
    const base = hz*(0.10+0.09*k), amp = 14+6*k, hgt = hz*(0.30+0.06*k);
    const g=ctx.createLinearGradient(0,base-hgt,0,base+amp);
    g.addColorStop(0,`rgba(150,80,220,0)`);
    g.addColorStop(0.45,`rgba(110,120,230,${(0.10*a).toFixed(3)})`);
    g.addColorStop(1,`rgba(80,255,160,${(0.22*a).toFixed(3)})`);
    ctx.fillStyle=g; ctx.beginPath();
    for(let x=-20;x<=W+20;x+=16){
      const y = base + amp*Math.sin(x/(70+9*k) + S.t*(0.55+0.15*k) + k*1.7)
                     + 6*Math.sin(x/23 - S.t*1.3 + k);
      x===-20 ? ctx.moveTo(x,y) : ctx.lineTo(x,y);
    }
    for(let x=W+20;x>=-20;x-=16){
      const y = base - hgt + amp*0.6*Math.sin(x/(90+9*k) + S.t*0.4 + k*2.1);
      ctx.lineTo(x,y);
    }
    ctx.closePath(); ctx.fill();
  }
  ctx.restore();
}

// --- bare discs: no orbit circle, no rays. Yellow Sun, white Moon.
function drawSun(){
  if(!pS) return;
```

```
  ctx.globalAlpha = 1;                                          // the FRAME clips it at the top
  ctx.fillStyle=rgb(mix([255,150,44],[255,226,74], clamp((e+4)/22,0,1)));
  ctx.beginPath(); ctx.arc(pS[0],pS[1],rS,0,7); ctx.fill();
  ctx.globalAlpha=1;
}
function drawCorona(){
  if(!pM || cover<0.62) return;
  const t=clamp((cover-0.62)/0.38,0,1)*(0.22+0.78*SK.slow);
  ctx.save(); ctx.globalCompositeOperation="lighter";
  const g1=ctx.createRadialGradient(pS[0],pS[1],rS*0.97,pS[0],pS[1],rS*3.1);
  g1.addColorStop(0,`rgba(255,252,242,${(0.60*t).toFixed(3)})`);
  g1.addColorStop(0.16,`rgba(224,232,255,${(0.22*t).toFixed(3)})`);
  g1.addColorStop(1,"rgba(170,196,255,0)");
  ctx.fillStyle=g1; ctx.beginPath(); ctx.arc(pS[0],pS[1],rS*3.1,0,7); ctx.fill();
  ctx.translate(pS[0],pS[1]);
  for(let i=0;i<CORONA.length;i++){
    const c=CORONA[i], a=c.a+Math.sin(S.t*0.22+i)*0.014;
    const L=rS*(1.2+c.l*2.6*(0.7+0.3*t)), w=c.w;
    ctx.beginPath();
    ctx.moveTo(Math.cos(a-w)*rS*0.98, Math.sin(a-w)*rS*0.98);
    ctx.quadraticCurveTo(Math.cos(a)*L*0.55, Math.sin(a)*L*0.55, Math.cos(a+c.b)*L, Math.sin(a+c.b)*L);
    ctx.quadraticCurveTo(Math.cos(a)*L*0.55, Math.sin(a)*L*0.55, Math.cos(a+w)*rS*0.98,
↪ Math.sin(a+w)*rS*0.98);
    ctx.closePath();
    ctx.fillStyle=`rgba(236,243,255,${(0.115*t*c.o).toFixed(3)})`;
    ctx.fill();
  }
  if(total){
    ctx.beginPath(); ctx.arc(0,0,rS*1.04,0,7);
    ctx.lineWidth=Math.max(1.2,rS*0.06); ctx.strokeStyle="rgba(255,96,86,.55)"; ctx.stroke();
  }
  ctx.restore();
}
// By day the Moon is not drawn: only the part common to both discs, in
// black. Clip to the solar disc, then fill the lunar disc: only the bite
// remains.
function drawBite(){
  if(!pS||!pM) return;
  ctx.save();
  ctx.beginPath(); ctx.arc(pS[0],pS[1],rS,0,7); ctx.clip();
  ctx.fillStyle="#000";
  ctx.beginPath(); ctx.arc(pM[0],pM[1],rM,0,7); ctx.fill();
  ctx.restore();
}
function drawDiamond(){
  if(!pS||!pM||total||cover<0.965) return;
  const t=(cover-0.965)/0.035, dx=pS[0]-pM[0], dy=pS[1]-pM[1], d=Math.hypot(dx,dy)||1;
  const x=pM[0]+dx/d*rM*0.99, y=pM[1]+dy/d*rM*0.99, r=rM*0.85*t;
  ctx.save(); ctx.globalCompositeOperation="lighter";
  const gd=ctx.createRadialGradient(x,y,0,x,y,r);
  gd.addColorStop(0,"rgba(255,255,250,.95)"); gd.addColorStop(.25,"rgba(255,246,222,.55)");
  gd.addColorStop(1,"rgba(255,230,180,0)");
  ctx.fillStyle=gd; ctx.beginPath(); ctx.arc(x,y,r,0,7); ctx.fill(); ctx.restore();
}
function drawMoon(veil){
  if(!pM) return;
  ctx.globalAlpha = veil;
  // The whole disc must read, otherwise a rising crescent looks like a
  // growing glow instead of a rising body: the dark part keeps its
  // earthshine and a rim. Atmospheric extinction: ruddy and dull at the
  // horizon, clean at the zenith. This makes the rising readable: a waxing
  // Moon's lit limb points down, toward the set Sun, so the dark part of
  // the disc rises first.
  const low = clamp(1 - altM/0.22, 0, 1);
  ctx.fillStyle = rgb(mix([59,63,86],[96,58,44],low));
  ctx.beginPath(); ctx.arc(pM[0],pM[1],rM,0,7); ctx.fill();
```

```
  ctx.lineWidth=1.3; ctx.strokeStyle=rgb(mix([86,91,120],[142,88,62],low)); ctx.stroke();
  if(kMoon>0.02){
    const pa = SK.pa, ct = 1-2*kMoon;   // +1: nothing lit (new); -1: all lit (full)
    ctx.save(); ctx.translate(pM[0],pM[1]); ctx.rotate(pa);
    ctx.beginPath();
    for(let i=0;i<=40;i++){ const a=-Math.PI/2+Math.PI*i/40; const x=Math.cos(a)*rM,y=Math.sin(a)*rM;
     ↪  i?ctx.lineTo(x,y):ctx.moveTo(x,y); }
    for(let i=0;i<=40;i++){ const a=Math.PI/2-Math.PI*i/40; ctx.lineTo(Math.cos(a)*rM*ct, Math.sin(a)*rM); }
    ctx.closePath(); ctx.fillStyle=rgb(mix([253,253,246],[246,172,104],low)); ctx.fill(); ctx.restore();
  }
  ctx.globalAlpha = 1;
}
// Sun by day, Moon by night, never both. The only exception is the eclipse:
// the Moon is then just a silhouette in front of the Sun.
function drawClouds(){
  const wc=S.wx.cloud; if(wc<0.02) return;
  const lit  = mix(mix([44,46,60],[238,240,246],dayF), [78,80,92], S.wx.rain*0.8);
  const dark = mix(mix([24,26,38],[152,154,170],dayF), [50,52,62], S.wx.rain*0.7);
  const n = Math.round(CLOUDS.length*clamp(wc*1.15,0,1));
  ctx.globalAlpha = clamp(wc*1.25,0,1)*0.92;
  for(let i=0;i<n;i++){
    const c=CLOUDS[i];
    const x=((((c.x + S.t*c.sp)%1)+1)%1)*(W+320)-160, y=hz - c.alt*K_SKY;
    if(y>hz-4) continue;
    for(const q of c.puffs){
      ctx.fillStyle = rgb(q.dy<0 ? lit : dark);
      ctx.beginPath();
      ctx.ellipse(x+q.dx*c.sc, y+q.dy*c.sc, q.r*c.sc, q.r*c.sc*0.62, 0, 0, 7);
      ctx.fill();
    }
  }
  ctx.globalAlpha=1;
}
// The rainbow forms opposite the Sun, so it only appears in the afternoon,
// once the Sun has passed behind the dino, exactly when it is not seen.
// Centred on the antisolar point, radius 42 deg.
function drawRainbow(){
  const r=S.wx.rain;
  // A little rain and a Sun at the back suffice; only storm downpours
  // erase it.
  const st = clamp(Math.min(r/0.15,(0.82-r)/0.20),0,1) * clamp(altS/0.12,0,1) * (lS[1]<0?1:0);
  if(st<=0.02) return;
  const vx=W/2 + FOC*RGT[1]/(FWD[1]||1e-6), cy=hz + altS*K_SKY, R=0.733*K_SKY;
  const cols=[[216,70,58],[226,134,52],[228,208,78],[96,180,96],[68,140,208],[88,84,182],[134,72,170]];
  ctx.save(); ctx.globalCompositeOperation="lighter"; ctx.lineWidth=4.4;
  cols.forEach((c,i)=>{
    ctx.strokeStyle=`rgba(${c[0]},${c[1]},${c[2]},${(0.48*st).toFixed(3)})`;
    ctx.beginPath(); ctx.arc(vx, cy, R-i*4.4, Math.PI, 2*Math.PI); ctx.stroke();
    ctx.strokeStyle=`rgba(${c[0]},${c[1]},${c[2]},${(0.18*st).toFixed(3)})`;
    ctx.beginPath(); ctx.arc(vx, cy, R*1.28+i*4.4, Math.PI, 2*Math.PI); ctx.stroke();  // secondary bow
  });
  ctx.restore();
}
function drawMeteors(){
  if(!S.meteors.length) return;
  ctx.save(); ctx.globalCompositeOperation="lighter"; ctx.lineCap="round";
  for(const m of S.meteors){
    const k = 1 - m.t/m.life, hx = m.x + m.vx*m.t, hy = m.y + m.vy*m.t;
    const d = Math.hypot(m.vx,m.vy)||1, tx = hx - m.vx/d*m.len, ty = hy - m.vy/d*m.len;
    const g = ctx.createLinearGradient(tx,ty,hx,hy);
    g.addColorStop(0,"rgba(255,250,230,0)"); g.addColorStop(1,`rgba(255,250,230,${(0.9*k).toFixed(3)})`);
    ctx.strokeStyle=g; ctx.lineWidth=m.w;
    ctx.beginPath(); ctx.moveTo(tx,ty); ctx.lineTo(hx,hy); ctx.stroke();
  }
  ctx.restore();
}
```

```
  // Santa Claus: four galloping reindeer, the sleigh, the sack and a golden
  // trail behind. Rudolph leads, nose lit.
  function drawSanta(u){
    if(u<0) return;
    const x0 = W+180 - u*(W+420), y0 = hz*0.26 + hz*0.12*u + 10*Math.sin(u*11);
    const gal = S.t*9;
    ctx.save(); ctx.translate(x0,y0);
    ctx.globalCompositeOperation="lighter";
    for(let i=0;i<16;i++){
      const t=i/16, tx=42+t*170, ty=4+Math.sin(u*11-t*4)*9;
      ctx.fillStyle=`rgba(255,214,110,${(0.5*(1-t)).toFixed(2)})`;
      ctx.beginPath(); ctx.arc(tx,ty,2+t*3.5,0,7); ctx.fill();
    }
    ctx.globalCompositeOperation="source-over";
    const N=4, brun="#6b4a2f";
    ctx.strokeStyle="#8a6a48"; ctx.lineWidth=1.1;
    ctx.beginPath(); ctx.moveTo(-8,-4); ctx.lineTo(-40-(N-1)*40, -4-(N-1)*3); ctx.stroke();  // reins
    for(let i=0;i<N;i++){
      const rx=-40-i*40, ry=-i*3 + Math.sin(gal+i*1.3)*3;
      ctx.fillStyle=brun; ctx.beginPath(); ctx.ellipse(rx,ry,13,6,0,0,7); ctx.fill();          // body
      ctx.beginPath(); ctx.ellipse(rx-15,ry-6,5,3.6,-0.3,0,7); ctx.fill();                     // head
      ctx.lineWidth=2.4; ctx.strokeStyle=brun; ctx.beginPath(); ctx.moveTo(rx-10,ry-2); ctx.lineTo(rx-14,ry-6);
↪ ctx.stroke();  // neck
      ctx.lineWidth=1.2; ctx.strokeStyle="#c9a77a";                                            // antlers
      ctx.beginPath(); ctx.moveTo(rx-16,ry-9); ctx.lineTo(rx-20,ry-17); ctx.moveTo(rx-16,ry-9);
↪ ctx.lineTo(rx-12,ry-17);
      ctx.moveTo(rx-18,ry-13); ctx.lineTo(rx-22,ry-15); ctx.moveTo(rx-14,ry-13); ctx.lineTo(rx-10,ry-15);
↪ ctx.stroke();
      ctx.lineWidth=2; ctx.strokeStyle=brun;                                                   // galloping legs
      for(let k=0;k<4;k++){
        const ph=gal+k*1.57+i, lx=rx-8+k*5.5, ang=Math.sin(ph)*0.75;
        ctx.beginPath(); ctx.moveTo(lx,ry+3); ctx.lineTo(lx+Math.sin(ang)*8, ry+3+Math.cos(ang)*8);
↪ ctx.stroke();
      }
      if(i===N-1){                                                                             // Rudolph
        const nz=0.65+0.35*Math.sin(S.t*8);
        ctx.fillStyle=`rgba(255,60,50,${nz.toFixed(2)})`; ctx.beginPath(); ctx.arc(rx-20,ry-6,2.4,0,7);
↪ ctx.fill();
        ctx.globalCompositeOperation="lighter"; ctx.fillStyle=`rgba(255,60,50,${(0.28*nz).toFixed(2)})`;
        ctx.beginPath(); ctx.arc(rx-20,ry-6,8,0,7); ctx.fill(); ctx.globalCompositeOperation="source-over";
      }
    }
    ctx.fillStyle="#b2302e"; ctx.beginPath();                                                  // the sleigh
    ctx.moveTo(-10,-8); ctx.lineTo(28,-8); ctx.lineTo(30,6); ctx.lineTo(-6,6);
↪ ctx.quadraticCurveTo(-18,6,-14,-14); ctx.closePath(); ctx.fill();
    ctx.strokeStyle="#e0b25a"; ctx.lineWidth=2;                                                // the runner
    ctx.beginPath(); ctx.moveTo(-16,10); ctx.lineTo(34,10); ctx.quadraticCurveTo(40,10,38,4); ctx.stroke();
    ctx.fillStyle="#d9c9a5"; ctx.beginPath(); ctx.arc(20,-13,8,0,7); ctx.fill();               // the sack
    ctx.fillStyle="#c8322f"; ctx.beginPath(); ctx.arc(6,-14,7,0,7); ctx.fill();                // Santa
    ctx.fillStyle="#f2d5b8"; ctx.beginPath(); ctx.arc(6,-24,4.2,0,7); ctx.fill();
    ctx.fillStyle="#f6f1e8"; ctx.beginPath(); ctx.arc(6,-21,4.6,0.15,Math.PI-0.15); ctx.fill();  // the beard
    ctx.fillStyle="#c8322f"; ctx.beginPath(); ctx.moveTo(1,-26); ctx.lineTo(11,-26); ctx.lineTo(14,-33);
↪ ctx.closePath(); ctx.fill();
    ctx.fillStyle="#fff"; ctx.beginPath(); ctx.arc(14,-33,1.8,0,7); ctx.fill();
    ctx.restore();
  }
  ctx.save();                                  // nothing spills below the horizon
  ctx.beginPath(); ctx.rect(0,0,W,Math.max(0,hz)); ctx.clip();
  // The Sun rises ahead and exits through the top of the frame. The Moon,
  // also moving, only exists at night: it fades in when the Sun goes below
  // the horizon (veil) and fades out at dawn.
  drawMeteors();
  drawAurora();
  if(altS>-0.05) drawSun();
  if(cover>0.02){ drawCorona(); drawBite(); drawDiamond(); }   // by day: the intersection, in black
  const veil = clamp(-altS/0.30, 0, 1);        // 0 at sunset, 1 in full night
```

```
if(veil>0.01) drawMoon(veil);
drawClouds(); drawRainbow();
drawSanta(S.santa);
if(S.bolt){                                  // the lightning bolt, in the sky
  ctx.strokeStyle="rgba(255,252,238,.92)"; ctx.lineWidth=2.4;
  ctx.beginPath(); ctx.moveTo(S.bolt.seg[0][0],S.bolt.seg[0][1]);
  for(const q of S.bolt.seg) ctx.lineTo(q[0],q[1]);
  ctx.stroke();
  ctx.strokeStyle="rgba(190,214,255,.30)"; ctx.lineWidth=7; ctx.stroke();
}
ctx.restore();

// --- ground
let near=mix([88,70,52],[196,154,106],dayF), far=mix([54,47,44],[172,138,100],dayF);
if(S.wx.snow>0.02){ const w=S.wx.snow*0.75; near=mix(near,mix([120,124,140],[232,236,244],dayF),w);
 ↪  far=mix(far,mix([96,100,116],[214,220,232],dayF),w); }
const g2=ctx.createLinearGradient(0,hz,0,H);
g2.addColorStop(0,rgb(far)); g2.addColorStop(1,rgb(near));
ctx.fillStyle=g2; ctx.fillRect(0,Math.max(0,hz),W,H-hz);

// speed bands
const BL=7, off=(S.dist%BL);
ctx.fillStyle=`rgba(0,0,0,${0.06+0.05*dayF})`;
for(let k=0;k<26;k++){
  const y0=k*BL*2 - off, y1=y0+BL;
  if(y1<0.5) continue;
  const q=[proj([-46,Math.max(y0,0.5),0]),proj([46,Math.max(y0,0.5),0]),proj([46,y1,0]),proj([-46,y1,0])];
  if(q.some(v=>!v)) continue;
  ctx.beginPath(); ctx.moveTo(q[0][0],q[0][1]); for(let i=1;i<4;i++) ctx.lineTo(q[i][0],q[i][1]); ctx.fill();
}
// track
ctx.fillStyle=`rgba(${mix([120,105,88],[224,206,178],dayF).map(v=>v|0).join(",")},.5)`;
for(const sgn of [-1,1]) for(let k=0;k<34;k++){
  const y0=k*4.2-(S.dist%4.2), y1=y0+2.4;
  if(y1<0.4) continue;
  const q=[proj([sgn*2.6-0.08,Math.max(y0,0.4),0.01]),proj([sgn*2.6+0.08,Math.max(y0,0.4),0.01]),
           proj([sgn*2.6+0.08,y1,0.01]),proj([sgn*2.6-0.08,y1,0.01])];
  if(q.some(v=>!v)) continue;
  ctx.beginPath(); ctx.moveTo(q[0][0],q[0][1]); for(let i=1;i<4;i++) ctx.lineTo(q[i][0],q[i][1]); ctx.fill();
}
// pebbles
ctx.fillStyle=`rgba(${mix([58,47,38],[150,118,84],dayF).map(v=>v|0).join(",")},.85)`;
for(const r of ROCKS){
  const y=((r.y0-S.dist)%140+140)%140;
  const p=proj([r.x,y,0]); if(!p||p[1]<hz) continue;
  const s=FOC*r.r/p[2];
  ctx.beginPath(); ctx.ellipse(p[0],p[1],s,s*0.4,0,0,7); ctx.fill();
}

// --- world objects
const boxes = dinoBoxes(S.gallop, S.px, S.lean, S.z);
const nearCacti = S.cacti.filter(c=>c.y>-6 && c.y<62);
for(const c of nearCacti) boxes.push(...cactusBoxes(c));

// --- COMPUTED SHADOWS: S = Q - l*(Qz/lz), union in a single path
const shC = 1 - 0.92*S.wx.cloud;             // under clouds, no cast shadow
for(const [Lp,alt,base,tint] of [[LposS,altS,0.52*(1-cover)*shC,"14,10,20"],
                                 [LposM,altM,0.34*kMoon*(1-dayF)*shC,"10,16,38"]]){
  if(alt<=0.04 || base<0.02) continue;
  const fade=Math.min(1, Math.sin(alt)*3.0), a=base*fade;
  if(a<0.02) continue;
  ctx.beginPath();
  for(const b of boxes){
    const gp=[];
    let ok=true;
    for(const q of verts(b)){
```

```
      const qz=Math.max(q[2],0);
      if(Lp[2] <= qz+0.25){ ok=false; break; }      // light below the vertex: nothing on the ground
      const t=qz/(Lp[2]-qz);                        // intersection of (L,Q) with z=0
      const gx=q[0]+(q[0]-Lp[0])*t, gy=q[1]+(q[1]-Lp[1])*t;
      if(Math.hypot(gx-q[0], gy-q[1]) > 70){ ok=false; break; }   // grazing shadow
      const p=proj([gx,gy,0]);
      if(!p){ ok=false; break; }
      gp.push([p[0],p[1]]);
    }
    if(!ok||gp.length<3) continue;
    const hl=hull(gp);
    if(hl.length<3) continue;
    ctx.moveTo(hl[0][0],hl[0][1]);
    for(let i=1;i<hl.length;i++) ctx.lineTo(hl[i][0],hl[i][1]);
    ctx.closePath();
  }
  ctx.fillStyle=`rgba(${tint},${a.toFixed(3)})`;
  ctx.fill("nonzero");
}

// --- faces, painter's algorithm
const F=[];
for(const b of boxes) faces(b,F);
const drawn=[];
for(const f of F){
  const cen=mul(add(add(f.q[0],f.q[1]),add(f.q[2],f.q[3])),0.25);
  const d=sub(cen,EYE);
  if(dot(f.n,d)>=0) continue;
  const p=[proj(f.q[0]),proj(f.q[1]),proj(f.q[2]),proj(f.q[3])];
  if(p.some(v=>!v)) continue;
  const dS=norm(sub(LposS,cen)), dM=norm(sub(LposM,cen));   // direction toward the body
  const nd1=Math.max(0,dot(f.n,dS)), nd2=Math.max(0,dot(f.n,dM));
  const col=[
    f.col[0]*(amb[0]+Isun[0]*nd1+Imoon[0]*nd2),
    f.col[1]*(amb[1]+Isun[1]*nd1+Imoon[1]*nd2),
    f.col[2]*(amb[2]+Isun[2]*nd1+Imoon[2]*nd2)];
  drawn.push({z:dot(d,FWD), p, c:rgb([Math.min(255,col[0]),Math.min(255,col[1]),Math.min(255,col[2])])});
}
drawn.sort((a,b)=>b.z-a.z);
const blink = S.invuln>0 && (Math.floor(S.t*12)%2===0);
for(const f of drawn){
  ctx.globalAlpha = blink ? 0.55 : 1;
  ctx.fillStyle=f.c;
  ctx.beginPath(); ctx.moveTo(f.p[0][0],f.p[0][1]);
  for(let i=1;i<4;i++) ctx.lineTo(f.p[i][0],f.p[i][1]);
  ctx.fill();
}
ctx.globalAlpha=1;

// --- dust
for(let j=0;j<7 && S.z<0.05;j++){
  const t=((S.gallop*3 + j*0.7)%3);
  const p=proj([S.px+Math.sin(j*2.1)*0.9, -1.3-t*1.1, 0.22+t*0.16]);
  if(!p) continue;
  const s=FOC*(0.12+t*0.13)/p[2];
  ctx.fillStyle=`rgba(${mix([70,60,52],[190,164,132],dayF).map(v=>v|0).join(",")},${(0.42-0.13*t).toFixed(2)
})`;
  ctx.beginPath(); ctx.ellipse(p[0],p[1],s,s*0.7,0,0,7); ctx.fill();
}

// --- snow
if(S.wx.snow>0.03){
  const sn=S.wx.snow, n=Math.round(FLAKES.length*sn);
  ctx.fillStyle=`rgba(236,242,252,${(0.45+0.45*sn).toFixed(2)})`;
  for(let i=0;i<n;i++){
    const f=FLAKES[i];
```

```
      const y=((f.y + S.t*f.v)%(H+20))-10;
      const x=((f.x + f.sw*Math.sin(S.t*0.9+f.ph) + y*0.06)%W+W)%W;
      ctx.beginPath(); ctx.arc(x,y,f.r,0,7); ctx.fill();
    }
  }
  // --- rain
  if(S.wx.rain>0.03){
    const r=S.wx.rain, n=Math.round(DROPS.length*r);
    ctx.strokeStyle=`rgba(196,212,232,${(0.16+0.30*r).toFixed(2)})`;
    ctx.lineWidth=1.1; ctx.beginPath();
    for(let i=0;i<n;i++){
      const d=DROPS[i];
      const y=((d.y + S.t*d.v)%(H+40))-20, x=((d.x - y*0.24)%W+W)%W;
      ctx.moveTo(x,y); ctx.lineTo(x+d.l*0.24, y+d.l);
    }
    ctx.stroke();
  }
  if(S.glare>0){ ctx.fillStyle=`rgba(226,234,255,${(0.42*S.glare).toFixed(3)})`; ctx.fillRect(0,0,W,H); }

  // --- vignette + hit flash
  const vg=ctx.createRadialGradient(W/2,H/2,H*0.42,W/2,H/2,H*0.95);
  vg.addColorStop(0,"rgba(0,0,0,0)"); vg.addColorStop(1,"rgba(0,0,0,.42)");
  ctx.fillStyle=vg; ctx.fillRect(0,0,W,H);
  if(S.flash>0){ ctx.fillStyle=`rgba(190,70,50,${(S.flash*0.30).toFixed(3)})`; ctx.fillRect(0,0,W,H); }

  // --- HUD
  duck(total);
  elDist.textContent = Math.round(S.dist);
  const hrs=(dayT%1)*24;
  const edeg = elong*180/Math.PI;
  const nm = kMoon<0.03 ? "nouvelle lune" : kMoon<0.28 ? "croissant"
           : kMoon<0.58 ? "quartier" : kMoon<0.92 ? "gibbeuse" : "pleine lune";
  elPhase.textContent = total ? "éclipse totale"
    : cover>0.02 ? `éclipse · ${(cover*100).toFixed(0)} %`
    : `${edeg.toFixed(0)}° — ${nm}`;
  elPhase.style.color = cover>0.02 ? "#e0653a" : "";
  const rem = ((ECL_DAY - dayT) % MOON_PER + MOON_PER) % MOON_PER;
  const an = Math.floor(dayT/SEASON)+1, doy = Math.floor(yearT(dayT))+1;
  elDay.textContent = `an ${an} · jour ${doy}/${SEASON} · ${String(Math.floor(hrs)).padStart(2,"0")} h
↪ ${String(Math.floor(hrs%1*60)).padStart(2,"0")} · éclipse dans ${rem.toFixed(1)} j`;
  if(S.cardT>0){
    elCard.hidden=false;
    elCard.style.opacity = clamp(Math.min(S.cardT, 3.6-S.cardT)*2.2, 0, 1).toFixed(2);
    elCardLvl.textContent = S.cardTitle;
    elCardSub.textContent = S.cardSub;
  } else if(!elCard.hidden) elCard.hidden=true;

 }

 let last=performance.now();
 function loop(now){
  const dt=Math.min(0.05,(now-last)/1000); last=now;
  update(dt); draw();
  requestAnimationFrame(loop);
 }
 paintLives();
 requestAnimationFrame(loop);
})();
</script>

</body></html>
```

2.31 /opt/GSM/osmo-operator/configs/conky/labgsm.html

10844 octets, 188 lignes → 188 lignes

```
<!doctype html><html><head><meta charset="utf-8">
<title>LAB GSM</title>
<style>
  /* labgsm.html - "LAB GSM" banner of the desktop top zone. Loaded by
     tools/osmo-topzone.py, which pushes state through osmoUpdate(), osmoFade()
     and osmoBanc() (see the script section below).
     - Sizes are in vw (zone width, 1110 px reference), never in vh: the window
       is shrunk to the content height (INJECT_CSS in osmo-topzone.py).
     - The card (TOP_BOX y=60) must end above the panel (y=220): without a
       title it fits in about 160 reference px.
     - Ubuntu font everywhere (fonts-ubuntu is installed by iso_modules/80-chroot.sh).
     - The "live bench" row under the timeslots (core, radio, subscribers,
       calls, uptime) is fed by tools/osmo-ts-probe.py through timeslots.json
       (core / radio / phy / subs / calls / up). */
  :root{
    --ink:#131018; --ink2:#1c1822; --card:rgba(13,11,20,0);
    --bone:#f4eddc; --dim:#c9bfab; --violet:#9d8cf0; --violet2:#6f5fc0;
    --teal:#4fe0b6; --moon:#b4c2ea; --orange:#ff9a3c; --pink:#ff4d6d;
    --ok:#3fb950; --ko:#f85149;
    --line:rgba(244,237,220,.18);
  }
  *{box-sizing:border-box}
  html,body{margin:0;height:100%;background:transparent;color:var(--bone);text-shadow:0 1px 2px rgba(0,0,0,.9),0
↪  0 8px rgba(0,0,0,.7);
    font-family:"Ubuntu","Ubuntu Sans",sans-serif;overflow:hidden}
  #card{position:absolute;inset:0;border:1px solid var(--line);border-radius:18px;
    background:var(--card);backdrop-filter:blur(3px);padding:1vw 1.8vw .9vw;
    display:flex;flex-direction:column;gap:.45vw;transition:opacity .6s ease}
  #card.faded{opacity:0;pointer-events:none}
  /* body.repos: no bench is running (osmo-panel.py writes 0 to
     $XDG_RUNTIME_DIR/osmo-banc.etat, relayed by osmo-topzone.py as
     osmoBanc(false)): the card is invisible and the zone is empty (the
     wallpaper carries the title). When a bench starts, the card fades in.
     body.instant: no animation (first state pushed at page load). */
  body.repos #card{opacity:0;pointer-events:none;transition:opacity 1.4s ease}
  body.instant #card{transition:none!important}
  .eyebrow{display:flex;justify-content:space-between;align-items:center;
    color:var(--dim);letter-spacing:.16em;font-size:.9vw;font-weight:500}
  .eyebrow .dot{color:var(--teal)}
  .pill{border:1px solid rgba(79,224,182,.55);border-radius:.8vw;padding:.3vw .9vw;color:var(--teal);
    font-family:"Ubuntu Mono",monospace;letter-spacing:.06em;font-size:.95vw}
  h1{margin:0;font-family:"Ubuntu","Ubuntu Sans",sans-serif;font-weight:700;
    font-size:5vw;line-height:1;letter-spacing:.01em;
    background:linear-gradient(92deg,var(--violet),#c4b6ff 60%,var(--moon));
    -webkit-background-clip:text;background-clip:text;color:transparent}
  .lede{color:#e6ddcb;font-size:1.35vw;line-height:1.4;max-width:78%}
  .lede b{color:#fff;font-weight:500}
  .meta{color:var(--dim);letter-spacing:.12em;font-size:.85vw;font-weight:500}
  /* timeslot row: eight tall columns of equal width */
  #ts{display:grid;grid-template-columns:repeat(8,1fr);gap:.7vw;min-height:4.2vw}
  .ts{border:1px solid rgba(180,194,234,.3);border-radius:.8vw;padding:.5vw .4vw;
    display:flex;flex-direction:column;align-items:center;justify-content:center;gap:.5vw;
    text-align:center;background:rgba(28,24,34,.3);position:relative;gap:.3vw;
    transition:box-shadow .18s,border-color .18s,background .18s}
  .ts .n{font-weight:700;font-size:1.5vw;line-height:1}
  .ts .k{color:var(--dim);font-size:.9vw;letter-spacing:.04em;font-family:"Ubuntu Mono",monospace}
  .ts.bcch{border-color:var(--violet);box-shadow:0 0 0 1px var(--violet) inset}
  /* orange blink when the timeslot carries bursts */
  .ts.on{border-color:var(--orange);
    background:rgba(255,154,60,.16);
    box-shadow:0 0 14px rgba(255,154,60,.55),0 0 0 1px var(--orange) inset;
    animation:blink .9s steps(2,end) infinite}
```

```
  @keyframes blink{50%{box-shadow:0 0 4px rgba(255,154,60,.2),0 0 0 1px rgba(255,154,60,.4) inset;
    background:rgba(255,154,60,.05)}}
  /* ── LIVE BENCH ROW ──────────────────────────────────────────────── */
  #live{display:flex;gap:.6vw;flex-wrap:wrap;align-items:center;
    border-top:1px solid var(--line);padding-top:.45vw}
  .grp{display:flex;align-items:center;gap:.35vw;border:1px solid var(--line);
    border-radius:.8vw;padding:.2vw .6vw;background:rgba(28,24,34,.25);font-size:.95vw}
  .grp .lab{color:var(--dim);letter-spacing:.08em;font-size:.9vw;margin-right:.2vw;font-weight:500}
  .grp .d{font-family:"Ubuntu Mono",monospace;font-size:.95vw;color:var(--dim)}
  .grp .d.ok{color:var(--ok)} .grp .d.ko{color:var(--ko)}
  .grp b{font-weight:500;color:var(--bone)}
  .grp.warn{border-color:rgba(255,154,60,.6);color:var(--orange)}
  .grp.call{border-color:rgba(79,224,182,.7);color:var(--teal)}
  .tags{display:flex;gap:.5vw;flex-wrap:wrap;align-items:center}
  .tag{border:1px solid var(--line);border-radius:.8vw;padding:.2vw .7vw;color:var(--dim);font-size:.9vw}
  .tag b{color:var(--bone);font-weight:500}
  .foot{margin-left:auto;color:var(--dim);font-size:.85vw}
  #noconf{color:var(--orange);opacity:.9}
</style></head>
<body class="repos">
<div id="card">
  <div class="eyebrow">
    <div><span class="dot">●</span> SOFTWARE DEFINED RADIO · UM INTERFACE · GR-GSM</div>
    <div class="pill" id="arfcn">ARFCN 514 · DCS 1800</div>
  </div>
  <!-- No title or presentation paragraph: the banner only shows live data
       (timeslots, bench row, tags). -->
  <div class="meta">TRAME TDMA · 8 TIMESLOTS · 4,615 MS</div>
  <div id="ts"></div>
  <div id="live"></div>
  <div class="tags" id="tags">
    <div class="tag">IMSI <b id="imsi">001-01</b></div>
    <div class="tag" id="a5tag">A5/1 <b>✓</b></div>
    <div class="tag">COMP128v1 <b>✓</b></div>
    <div class="tag">TCH/F · GAPK fr</div>
    <div class="tag">gr-gsm · trxcon · osmo-bts-trx</div>
    <div id="noconf" class="tag" hidden>osmo-bsc non détecté — animation de démonstration</div>
    <div class="foot">bbaranoff.github.io</div>
  </div>
</div>

<script>
// Short labels per physical channel type (osmo-bsc phys_chan_config).
const LABEL = {
  "CCCH+SDCCH4":"FCCH·SCH·BCCH", "CCCH":"FCCH·SCH·BCCH", "BCCH":"BCCH",
  "SDCCH8":"SDCCH", "SDCCH8+CBCH":"SDCCH", "SDCCH":"SDCCH",
  "TCH/F":"TCH/F", "TCH/H":"TCH/H", "TCH/F_PDCH":"TCH/F·PDCH",
  "PDCH":"PDCH", "TCH/F_TCH/H_PDCH":"TCH", "NONE":"—"
};
let TS = [
  {n:0,type:"CCCH+SDCCH4"},{n:1,type:"SDCCH8"},{n:2,type:"TCH/F"},{n:3,type:"TCH/F"},
  {n:4,type:"TCH/F"},{n:5,type:"TCH/F"},{n:6,type:"TCH/F"},{n:7,type:"TCH/F"}
];
function drawTS(){
  const box=document.getElementById('ts'); box.innerHTML='';
  TS.forEach(t=>{
    const d=document.createElement('div'); d.className='ts'; d.dataset.n=t.n;
    if((t.type||'').indexOf('CCCH')>=0||(t.type||'').indexOf('BCCH')>=0) d.classList.add('bcch');
    d.innerHTML='<div class="n">TS'+t.n+'</div><div class="k">'+(LABEL[t.type]||t.type||'—')+'</div>';
    box.appendChild(d);
  });
}
// ── BENCH ROW ──────────────────────────────────────────────────────
// Fixed order: the dots are always read at the same place.
const CORE=["HLR","MSC","BSC","STP","MGW","BTS","SIP","SMSC","PBX"];
const RADIO=["FAKETRX","PONT","MOBILE","TRXCON","QEMU","GAPK"];
```

```
function esc(s){return String(s).replace(/[&<>]/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;'}[c]));}
function dots(names,etat){
  return names.map(n=>{const v=etat&&etat[n];
    return '<span class="d '+(v===undefined?'':(v?'ok':'ko'))+'">'+(v===undefined?'○':'●')+'</span>'+esc(n);
  }).join(' ');
}
function duree(s){
  s=Math.max(0,s|0); if(s<60) return s+' s';
  const m=Math.floor(s/60)%60, h=Math.floor(s/3600);
  return h?h+' h '+String(m).padStart(2,'0'):m+' min';
}
function drawLive(state){
  const box=document.getElementById('live'); if(!box) return;
  if(!state || !state.bsc){
    box.innerHTML='<div class="grp warn"><span class="lab">BANC</span> à l\'arrêt — <b>./start-native.sh</b> ou le
↪  bouton <b>run standalone</b></div>';
    return;
  }
  const s=state.subs||{}; const calls=state.calls|0;
  let h='';
  h+='<div class="grp"><span class="lab">CŒUR</span>'+dots(CORE,state.core)+'</div>';
  h+='<div class="grp"><span class="lab">RADIO</span>'+dots(RADIO,state.radio)+(state.phy?' <b>·
↪  '+esc(state.phy)+'</b>':'')+'</div>';
  h+='<div class="grp"><span class="lab">ABONNÉS</span><b>'+(s.total===undefined?'–':s.total)+'</b>'
     +(s.attached!==undefined?' · <span class="d ok">●</span> '+s.attached+'
      ↪  rattaché'+(s.attached>1?'s':''):'')+'</div>';
  h+='<div class="grp'+(calls?' call':'')+'"><span class="lab">APPELS</span><b>'+calls+'</b>'+(calls?' TCH en
↪  voix':'')+'</div>';
  if(state.up) h+='<div class="grp"><span class="lab">DEBOUT</span><b>'+duree(state.up)+'</b></div>';
  box.innerHTML=h;
}
// osmoUpdate({ts:[{n,type,active,voice}], bsc:true, arfcn:"...", core:{}, radio:{}, subs:{}, calls:n, up:s})
// - called by osmo-topzone.py
window.osmoUpdate=function(state){
  try{
    if(state.ts && state.ts.length){ TS=state.ts; drawTS(); }
    document.getElementById('noconf').hidden = !!state.bsc;
    if(state.arfcn) document.getElementById('arfcn').textContent=state.arfcn;
    if(state.imsi) document.getElementById('imsi').textContent=state.imsi;
    if(state.a5) document.getElementById('a5tag').innerHTML='A5/'+state.a5+' <b>✓</b>';
    (state.ts||[]).forEach(t=>{
      const el=document.querySelector('.ts[data-n="'+t.n+'"]'); if(!el) return;
      // .on = this timeslot carries bursts (voice included): the only
      // blinking element of the card.
      el.classList.toggle('on', !!t.active || !!t.voice);
    });
    drawLive(state);
  }catch(e){}
};
// fade: osmo-topzone.py toggles .faded to make room for an application
window.osmoFade=function(on){document.getElementById('card').classList.toggle('faded',!!on);};
// bench running (on=true): show the card; stopped: nothing (the wallpaper
// carries the title). instant=true: no fade (initial state, at page load).
window.osmoBanc=function(on,instant){const b=document.body;
  if(instant){b.classList.add('instant');void b.offsetWidth;}
  b.classList.toggle('repos',!on);
  if(instant){void b.offsetWidth;b.classList.remove('instant');}};
drawTS(); drawLive(null);
// demo when nobody pushes a state (osmo-topzone absent): random TCH timeslots blink
let demo=setInterval(()=>{ if(window.__osmoLive) return clearInterval(demo);
  TS.forEach(t=>{const el=document.querySelector('.ts[data-n="'+t.n+'"]');
    if(el) el.classList.toggle('on', (t.type||'').indexOf('TCH')>=0 && Math.random()<0.25);});
},700);
</script>
</body></html>
```

2.32 /opt/GSM/osmo-operator/configs/crypthome/crypthome-fond

2989 octets, 77 lignes → 77 lignes

```
#!/usr/bin/python3
# crypthome-fond - paints an image on the framebuffer during the LUKS prompt at boot.
#
#   crypthome-fond [IMAGE]     (default: CRYPTHOME_IMAGE from /etc/default/crypthome)
#
# unlock-home quits plymouth and reads the passphrase on the text console, i.e. fbcon: a framebuffer
# (/dev/fb0) on which the kernel draws the characters. The image is painted there, darkened at the
# bottom; the prompt is then placed on the LAST line, where fbcon repaints only the cells it writes.
# Input stays on the tty: the image is only decoration, and if it fails (no /dev/fb0, no Pillow,
# unknown pixel format) the script exits 0 with a single line on stderr.
import os
import sys


def lire(chemin, defaut=""):
    try:
        with open(chemin) as f:
            return f.read().strip()
    except OSError:
        return defaut


def image_conf():
    for ligne in lire("/etc/default/crypthome").splitlines():
        if ligne.startswith("CRYPTHOME_IMAGE="):
            return ligne.split("=", 1)[1].strip().strip('"')
    return "/usr/share/backgrounds/gsm-lab-antenne.jpg"


def main():
    chemin = sys.argv[1] if len(sys.argv) > 1 else image_conf()
    try:
        from PIL import Image, ImageDraw
    except ImportError:
        return "Pillow absent"
    sysfb = "/sys/class/graphics/fb0"
    try:
        w, h = (int(v) for v in lire(sysfb + "/virtual_size").split(","))
        bpp = int(lire(sysfb + "/bits_per_pixel"))
        stride = int(lire(sysfb + "/stride") or w * bpp // 8)
    except ValueError:
        return "pas de framebuffer"
    if bpp != 32:
        return "framebuffer %d bits par point, seul 32 est gere" % bpp
    try:
        img = Image.open(chemin).convert("RGB")
    except OSError as e:
        return "image illisible (%s)" % e
    # cover the screen without distortion: scale, then crop at the centre
    k = max(w / img.width, h / img.height)
    img = img.resize((round(img.width * k), round(img.height * k)), Image.LANCZOS)
    x0, y0 = (img.width - w) // 2, (img.height - h) // 2
    img = img.crop((x0, y0, x0 + w, y0 + h))
    # dark band at the bottom: the white console prompt stays readable on it
    bande = Image.new("L", (w, h), 0)
    ImageDraw.Draw(bande).rectangle((0, h - h // 7, w, h), fill=200)
    img = Image.composite(Image.new("RGB", (w, h), (0, 0, 0)), img, bande)
    # little-endian XRGB8888 (B, G, R, X): the fbcon format on simpledrm / nvidia-drm / i915
    brut = img.tobytes("raw", "BGRX")
    try:
        with open("/dev/fb0", "r+b") as fb:
            if stride == w * 4:
```

```
                fb.write(brut)
            else:
                for y in range(h):
                    fb.seek(y * stride)
                    fb.write(brut[y * w * 4:(y + 1) * w * 4])
    except OSError as e:
        return "ecriture /dev/fb0 impossible (%s)" % e
    return None


if __name__ == "__main__":
    err = main()
    if err:
        print("crypthome-fond: %s - question sans image." % err, file=sys.stderr)
    sys.exit(0)
```

2.33 /opt/GSM/osmo-operator/configs/crypthome/crypthome-postinstall

11139 octets, 210 lignes → 210 lignes

```
#!/bin/bash
# crypthome-postinstall <compte-de-session>
#
# Lance UNE FOIS, par Calamares (shellprocess@osmo), DANS la cible montee, juste
# apres que l installeur a ecrit /etc/fstab et /etc/crypttab. Il ne chiffre
# rien : le chiffrement est deja fait, par Calamares, sur la partition /home
# (partition.conf : la racine porte noEncrypt, /home non). Ce script ne fait que
# RACCORDER ce que l installeur a produit au dispositif crypthome.
#
# POURQUOI UN RACCORD, ET PAS LE CABLAGE DE CALAMARES TEL QUEL. Calamares ecrit
# une ligne crypttab ordinaire pour /home. Avec une racine EN CLAIR, le module
# fstab y met le mot de passe "none" (il ne cree de /crypto_keyfile.bin que pour
# une racine chiffree) : systemd demande donc la phrase de passe au demarrage et
# ATTEND. C est exactement ce que ce banc ne veut pas - il redemarre dix fois
# par jour, souvent sans personne devant l ecran.
#
# On repasse donc crypttab et fstab en "noauto" et on remplit
# /etc/default/crypthome. C est alors unlock-home.service qui demande la phrase
# de passe, avant le gestionnaire de connexion : la taper ouvre /home et la
# session du compte ; Entree a vide (ou le delai qui expire) demarre en root
# sans /home - et le demarrage se termine dans tous les cas.
#
# LE PROPRIETAIRE, C EST LE COMPTE DE SESSION. Sur la cle live, le home chiffre
# appartenait a osmocom pendant que la session tournait sous root. Sur le disque
# installe, shellprocess-osmo.conf SUPPRIME osmocom et la session s ouvre sous le
# compte choisi a l installation. Laisser CRYPTHOME_OWNER=osmocom y designerait
# un compte qui n existe plus : run-as-owner, lock-home et
# crypthome-session-setup viseraient dans le vide, sans une erreur.
set -u

OWNER="${1:-}"
CONF=/etc/default/crypthome
MOUNT=/home

log() { echo "crypthome-postinstall: $*"; }

[ -r "$CONF" ] || { log "$CONF absent - rien a raccorder."; exit 0; }

# -- CE QUE CALAMARES A MIS SOUS /home ----------------------------------------
# /etc/fstab fait foi : c est lui que le systeme installe lira. Le module fstab
# de Calamares ecrit "/dev/mapper/<nom>" quand la partition est chiffree, et un
# "UUID=..." quand elle ne l est pas. L absence de ligne /home (pas de partition
# separee) se lit de la meme facon : pas de volume chiffre.
SRC="$(awk '$1 !~ /^#/ && $2 == "'"$MOUNT"'" { print $1; exit }' /etc/fstab 2>/dev/null)"
```

```
MAPPER=""
UUID=""
case "$SRC" in
    /dev/mapper/*)
        MAPPER="${SRC#/dev/mapper/}"
        # L UUID du CONTENEUR LUKS, pas celui du systeme de fichiers interne :
        # c est lui que cryptsetup ouvre, et c est ce que porte crypttab.
        UUID="$(awk -v m="$MAPPER" '$1 !~ /^#/ && $1 == m { sub(/^UUID=/, "", $2); print $2; exit }' \
                /etc/crypttab 2>/dev/null)"
        ;;
esac

# ── LE COMPTE PROPRIETAIRE ──────────────────────────────────────────
# Vide ou inconnu : on ne l ecrit pas, plutot que d ecrire un nom faux. Un
# CRYPTHOME_OWNER vide met simplement run-as-owner en retrait (l application
# demarre sous le compte courant), ce qui est le bon repli.
if [ -n "$OWNER" ] && ! id -u "$OWNER" >/dev/null 2>&1; then
    log "compte « $OWNER » inconnu dans la cible - CRYPTHOME_OWNER laisse tel quel."
    OWNER=""
fi

ecrire_conf() {
    # sed sur les trois cles, sans reecrire le fichier : les commentaires de
    # /etc/default/crypthome sont la documentation du dispositif sur la machine.
    local cle valeur
    for cle in CRYPTHOME_UUID CRYPTHOME_NAME CRYPTHOME_OWNER; do
        case "$cle" in
            CRYPTHOME_UUID)  valeur="$UUID" ;;
            CRYPTHOME_NAME)  valeur="$MAPPER" ;;
            CRYPTHOME_OWNER) valeur="$OWNER" ;;
        esac
        [ -n "$valeur" ] || continue
        if grep -q "^$cle=" "$CONF"; then
            sed -i "s|^$cle=.*|$cle=\"$valeur\"|" "$CONF"
        else
            printf '%s="%s"\n' "$cle" "$valeur" >> "$CONF"
        fi
    done
}

# ── LE TROUSSEAU DE ROOT ────────────────────────────────────────────
# /root/.local/share/keyrings est un LIEN vers le disque chiffre, pose dans le
# rootfs par l etape 8i. Deux cas, et ils sont opposes :
#
#   /home chiffre  -> on cree la cible du lien sur le volume. Disque non
#                     dechiffre = lien mort = gnome-keyring n ecrit rien en
#                     clair, ce qui est precisement le but.
#   /home en clair -> le lien n a plus rien a proteger, et un lien mort casse
#                     le trousseau de root sans le dire. On remet un vrai
#                     repertoire.
raccorder_trousseau_root() {
    local chiffre="$1" lien cible
    for lien in ${CRYPTHOME_ROOT_LINKS:-.local/share/keyrings}; do
        # "root-" : le nom que posent deja l etape 8i et init-crypthome.sh
        # (/home/.crypthome/root-keyrings). Le prefixe dit a qui appartient le
        # repertoire, sur un volume qui porte aussi les homes des comptes.
        cible="$MOUNT/.crypthome/root-$(basename "$lien")"
        if [ "$chiffre" = 1 ]; then
            install -d -m 0700 "$MOUNT/.crypthome"
            install -d -m 0700 "$cible"
            chown -R root:root "$MOUNT/.crypthome"
            install -d -m 0700 "/root/$(dirname "$lien")"
            # UN VRAI REPERTOIRE A CET ENDROIT N EST PAS UN OBSTACLE A POUSSER.
            # Le lien pose par l etape 8i est mort tant que rien n est monte, et
            # c est le cas normal ici. Mais si un repertoire REEL s y trouve, il
```

```
            # contient un trousseau - « rm -rf » le perdrait sans un mot. On
            # deplace son contenu sur le volume, puis on remplace.
            if [ -d "/root/$lien" ] && [ ! -L "/root/$lien" ]; then
                ( shopt -s dotglob nullglob
                  _restes=("/root/$lien"/*)
                  [ "${#_restes[@]}" -gt 0 ] && mv -n "${_restes[@]}" "$cible"/ ) 2>/dev/null
                rmdir "/root/$lien" 2>/dev/null \
                    || { log "ATTENTION : /root/$lien non vide, laisse EN CLAIR."; continue; }
            else
                rm -f "/root/$lien"
            fi
            ln -sfn "$cible" "/root/$lien"
        elif [ -L "/root/$lien" ]; then
            rm -f "/root/$lien"
            install -d -m 0700 "/root/$lien"
        fi
    done
}

# shellcheck disable=SC1090
. "$CONF" 2>/dev/null || true

if [ -z "$MAPPER" ] || [ -z "$UUID" ]; then
    # ── PAS DE VOLUME CHIFFRE ──────────────────────────────────────
    # L operateur n a pas coche « chiffrer », ou le disque etait trop petit pour
    # que la partition /home de partition.conf tienne. On le DIT : le symptome
    # silencieux serait un banc qu on croit chiffre et qui ne l est pas.
    UUID=""
    MAPPER=""
    ecrire_conf          # n ecrit que CRYPTHOME_OWNER : les deux autres sont vides
    raccorder_trousseau_root 0
    log "ATTENTION : $MOUNT n est PAS chiffre sur ce disque."
    log "  Les profils, cookies et trousseaux y seront lisibles."
    log "  Pour chiffrer un SECOND disque apres coup : /root/init-crypthome.sh"
    exit 0
fi

# ── LE VOLUME DOIT ETRE MONTE LA, MAINTENANT ──────────────────────
# Calamares monte la cible avant d y deverser le squashfs : a cet instant,
# /home DOIT etre le volume dechiffre. S il ne l est pas, ce qu on s apprete a
# ecrire dedans (le trousseau de root) irait sur le repertoire /home de la
# RACINE, en clair, et disparaitrait sous le montage au premier demarrage.
# On le dit et on continue : crypttab, fstab et la configuration, eux, restent
# justes - c est seulement le trousseau qu on ne pose pas.
_monte=1
if command -v findmnt >/dev/null 2>&1; then
    _src_reel="$(findmnt -fno SOURCE "$MOUNT" 2>/dev/null)"
    if [ "$(readlink -f "${_src_reel:-/nonexistent}")" != "$(readlink -f "/dev/mapper/$MAPPER")" ]; then
        _monte=0
        log "ATTENTION : $MOUNT n est pas monte depuis /dev/mapper/$MAPPER"
        log "  (source vue : ${_src_reel:-aucune}) - trousseau de root non pose."
    fi
fi

# ── CRYPTTAB : "noauto" ────────────────────────────────────────────
# systemd ne doit RIEN demander de lui-meme : c est unlock-home qui pose la
# question, avec son delai et son repli. La ligne reste, parce qu elle documente
# le volume et sert a « cryptdisks_start » a la main.
if grep -q "^$MAPPER[[:space:]]" /etc/crypttab 2>/dev/null; then
    sed -i "\|^$MAPPER[[:space:]]|d" /etc/crypttab
fi
printf '%-22s %-40s %-10s %s\n' "$MAPPER" "UUID=$UUID" "none" "luks,noauto" >> /etc/crypttab

# ── FSTAB : "noauto,nofail" ────────────────────────────────────────
# noauto pour la meme raison, nofail en ceinture : une entree /home qui echoue
# au demarrage ne doit pas basculer la machine en mode secours. On CONSERVE les
```

```
# options ecrites par Calamares (noatime, discard sur SSD) au lieu de les
# remplacer par "defaults" - elles ont ete choisies d apres le disque reel.
awk -v m="$MOUNT" '
    $1 ~ /^#/ || $2 != m { print; next }
    {
        opts = $4
        if (opts !~ /(^|,)noauto(,|$)/) opts = opts ",noauto"
        if (opts !~ /(^|,)nofail(,|$)/) opts = opts ",nofail"
        # Le passe fsck a 2 : la racine seule vaut 1, et 0 desactiverait la
        # verification du volume chiffre.
        printf "%-42s %-14s %-7s %s %s %s\n", $1, $2, $3, opts, ($5 == "" ? "0" : $5), "2"
    }
' /etc/fstab > /etc/fstab.crypthome && mv /etc/fstab.crypthome /etc/fstab

ecrire_conf
[ "$_monte" = 1 ] && raccorder_trousseau_root 1

# ── LE SERVICE ──────────────────────────────────────────────────────
# Deja active dans l image (etape 8i) ; on le reactive ici pour le cas d un
# rootfs ou systemctl avait echoue au build. Sans lui, /home reste ferme et
# personne ne saurait pourquoi.
systemctl enable unlock-home.service >/dev/null 2>&1 || true

# Les lanceurs a secrets pointent vers le compte proprietaire, qui vient de
# changer de nom : la liste doit etre refaite avec le bon.
[ -x /usr/local/sbin/refresh-owner-apps ] && \
    /usr/local/sbin/refresh-owner-apps >/dev/null 2>&1 || true

log "$MOUNT chiffre (LUKS $UUID, mapping $MAPPER), proprietaire ${OWNER:-<aucun>}."
log "  Au demarrage : « decrypt $MOUNT -passphrase- or Enter » (sur la console, plymouth quitte avant)"
log "  phrase tapee : session ${OWNER:-du compte}, persistante sur $MOUNT chiffre ;"
log "  Entree a vide, ou pas de reponse : session root ephemere (ecritures dans /tmp),"
log "  sans $MOUNT - le demarrage ne bloque jamais."
exit 0
```

2.34 /opt/GSM/osmo-operator/configs/crypthome/crypthome-session-setup

1880 octets, 42 lignes → 42 lignes

```
#!/bin/bash
# crypthome-session-setup - lance au demarrage de la session (root) de GDM.
#
# Prepare la bascule vers le compte proprietaire AVANT qu'on en ait besoin :
# autorisation sur le serveur X et session utilisateur demarree. Sans cela, le
# premier lancement de Firefox paie l'attente de logind, plusieurs secondes,
# et donne l'impression que rien ne se passe.
#
# Quand /home n'est pas dechiffre, ce script ne fait rien : c'est le mode
# degrade normal, les applications s'ouvriront sous root, sans secrets.
set -u
CONF=/etc/default/crypthome
[ -r "$CONF" ] && . "$CONF"
OWNER="${CRYPTHOME_OWNER:-}"
MOUNT="${CRYPTHOME_MOUNT:-/home}"
MAPPER="${CRYPTHOME_NAME:-crypthome}"

# ── LE MONTAGE DOIT VENIR DU VOLUME CHIFFRE ─────────────────────────
# "monte" ne suffit pas. /home peut etre une partition ORDINAIRE - une machine
# installee avec un /home separe, ou init-crypthome.sh n a jamais tourne. Traiter
# ce montage-la comme le disque chiffre ecrirait les secrets EN CLAIR sous le
# compte proprietaire tout en annoncant le contraire : le pire des etats, parce
# qu il a l air correct. On exige donc que la source du montage soit le mapping
# dechiffre lui-meme, et pas seulement qu il y ait quelque chose sur /home.
crypthome_monte() {
```

```
    local _src
    mountpoint -q "$MOUNT" || return 1
    _src="$(findmnt -fno SOURCE "$MOUNT" 2>/dev/null)"
    [ -n "$_src" ] || return 1
    [ "$(readlink -f "$_src")" = "$(readlink -f "/dev/mapper/$MAPPER")" ]
}


[ -n "$OWNER" ] || exit 0
id -u "$OWNER" >/dev/null 2>&1 || exit 0
crypthome_monte || exit 0
[ -d "$(getent passwd "$OWNER" | cut -d: -f6)" ] || exit 0

[ -n "${DISPLAY:-}" ] && command -v xhost >/dev/null 2>&1 && \
    xhost "+SI:localuser:$OWNER" >/dev/null 2>&1 || true
loginctl enable-linger "$OWNER" >/dev/null 2>&1 || true
exit 0
```

2.35 /opt/GSM/osmo-operator/configs/crypthome/init-crypthome.sh

12987 octets, 265 lignes → 265 lignes

```
#!/bin/bash
# init-crypthome.sh - initialises the second disk as LUKS2 and moves /home onto it.
#
#   sudo /root/init-crypthome.sh [/dev/partition] [--key-file=F] [--yes]
#
# NOT THE NORMAL PATH. On a machine installed from the ISO, /home is encrypted
# BY THE INSTALLER: installer/calamares/modules/partition.conf splits the disk
# into root + /home and encrypts only /home; crypthome-postinstall wires it up.
#
# This script covers the case Calamares does not: encrypting a SECOND DISK
# added to an already installed machine. It refuses any partition of the root
# disk.
#
# Without argument, the candidate partition is deduced - there must be exactly
# one, otherwise the script stops and lists them.
#
# THIS SCRIPT WIPES THE TARGET PARTITION. It refuses to run if the partition
# contains anything but lost+found, if it is mounted, or if it already carries
# a LUKS header - and it asks for a confirmation typed in full.
#
# What it does, in order: encrypts the partition, copies the current content
# of /home onto it, leaves the old /home in place (unlock-home moves it aside
# at next boot), writes /etc/crypttab and /etc/fstab with "noauto" (unlock-home
# mounts after asking), and enables unlock-home.service.
set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; CYAN='\033[0;36m'
YELLOW='\033[1;33m'; NC='\033[0m'; BOLD='\033[1m'

CONF=/etc/default/crypthome
. "$CONF"
MAPPER="${CRYPTHOME_NAME:-crypthome}"
MOUNT="${CRYPTHOME_MOUNT:-/home}"
OWNER="${CRYPTHOME_OWNER:-}"
TMP=/mnt/crypthome-init

[ "$(id -u)" -eq 0 ] || { echo -e "${RED}A lancer en root.${NC}" >&2; exit 1; }

# ── OPTIONS ──────────────────────────────────────────────────────────
#   --key-file=F  passphrase read from F instead of being asked. For scripted
#                 installations. The file must NOT outlive the script: the
#                 caller deletes it.
```

```
#   --yes, -y     no confirmation. For non-interactive calls only; the script
#                 WIPES the target partition.
KEYFILE=""
SANS_QUESTION=0
_args=()
for _a in "$@"; do
    case "$_a" in
        --key-file=*) KEYFILE="${_a#--key-file=}" ;;
        --yes|-y)     SANS_QUESTION=1 ;;
        -*)           echo -e "${RED}Option inconnue : $_a${NC}" >&2; exit 2 ;;
        *)            _args+=("$_a") ;;
    esac
done
set -- ${_args[@]+"${_args[@]}"}
if [ -n "$KEYFILE" ] && [ ! -r "$KEYFILE" ]; then
    echo -e "${RED}--key-file : $KEYFILE illisible.${NC}" >&2; exit 1
fi

# ── WHICH PARTITION ──────────────────────────────────────────────────
# Nothing is guessed silently: this script WIPES its target. Without argument
# it keeps only the partitions meeting ALL the conditions - not on the root
# disk, not mounted, no LUKS header, not iso9660/swap/EFI - and exactly one
# must remain for it to go on by itself.
disque_racine="$(lsblk -no PKNAME "$(findmnt -no SOURCE /)" 2>/dev/null | head -1)"
candidates() {
    local p pk
    for p in $(lsblk -lnpo NAME,TYPE | awk '$2=="part"{print $1}'); do
        pk="$(lsblk -no PKNAME "$p" 2>/dev/null | head -1)"
        [ "$pk" = "$disque_racine" ] && continue
        findmnt -S "$p" >/dev/null 2>&1 && continue
        cryptsetup isLuks "$p" 2>/dev/null && continue
        case "$(blkid -o value -s TYPE "$p" 2>/dev/null)" in
            iso9660|swap|vfat) continue ;;
        esac
        echo "$p"
    done
}

if [ "$#" -ge 1 ]; then
    DEV="$1"
else
    mapfile -t _cand < <(candidates)
    if [ "${#_cand[@]}" -eq 1 ]; then
        DEV="${_cand[0]}"
        echo -e "${YELLOW}Partition candidate unique retenue : ${CYAN}$DEV${NC}"
    elif [ "${#_cand[@]}" -eq 0 ]; then
        echo -e "${RED}Aucune partition candidate.${NC}" >&2
        echo    "Il en faut une hors du disque de la racine (${disque_racine:-?})," >&2
        echo    "non montee et sans en-tete LUKS. Creez-la, puis relancez :" >&2
        echo    "  $0 /dev/<partition>" >&2
        exit 1
    else
        echo -e "${RED}Plusieurs partitions candidates - precisez laquelle :${NC}" >&2
        lsblk -po NAME,SIZE,FSTYPE,LABEL "${_cand[@]}" >&2
        echo    "  $0 /dev/<partition>" >&2
        exit 1
    fi
fi

# ── CHECKS ───────────────────────────────────────────────────────────
[ -b "$DEV" ] || { echo -e "${RED}$DEV n'est pas une partition.${NC}" >&2; exit 1; }

if findmnt -S "$DEV" >/dev/null 2>&1; then
    echo -e "${RED}$DEV est monte. Demontez-le d'abord.${NC}" >&2; exit 1
fi
```

```
if cryptsetup isLuks "$DEV" 2>/dev/null; then
    echo -e "${RED}$DEV porte deja un en-tete LUKS.${NC}" >&2
    echo    "Si c'est un essai precedent a refaire, effacez-le sciemment :" >&2
    echo    "  cryptsetup erase $DEV && wipefs -a $DEV" >&2
    exit 1
fi

# The partition must be empty. Checked for real by mounting it read-only: a
# blkid saying "ext4" does not tell whether it holds data.
if blkid "$DEV" >/dev/null 2>&1; then
    mkdir -p "$TMP"
    if mount -o ro "$DEV" "$TMP" 2>/dev/null; then
        reste="$(find "$TMP" -mindepth 1 -maxdepth 1 ! -name lost+found -print -quit)"
        df -h "$TMP" | tail -1
        umount "$TMP"
        if [ -n "$reste" ]; then
            echo -e "${RED}$DEV contient des donnees (par ex. $(basename "$reste")).${NC}" >&2
            echo    "Ce script ne formate que des partitions vides. Sauvegardez et videz-la." >&2
            exit 1
        fi
    fi
    rmdir "$TMP" 2>/dev/null || true
fi

taille="$(lsblk -dno SIZE "$DEV" | tr -d ' ')"
echo
echo -e "${BOLD}Initialisation du disque chiffre${NC}"
echo -e "  partition      : ${CYAN}$DEV${NC} ($taille)"
echo -e "  deviendra      : ${CYAN}$MOUNT${NC} (LUKS2 + ext4, mappe sur /dev/mapper/$MAPPER)"
echo -e "  compte a home  : ${CYAN}$OWNER${NC}"
echo -e "  au demarrage   : ${CYAN}\"decrypt $MOUNT -passphrase- or Enter\"${NC}, Entree a vide n'empeche pas de
 ↪  demarrer"
echo -e "  ${YELLOW}le contenu actuel de $MOUNT sera recopie dessus ; l original reste en place${NC}"
echo -e "  ${YELLOW}et part dans /var/backups/ au redemarrage, avant le montage${NC}"
echo
echo -e "${RED}${BOLD}TOUT CE QUE CONTIENT $DEV SERA DETRUIT.${NC}"
if [ "$SANS_QUESTION" = 1 ]; then
    echo -e "${YELLOW}--yes : confirmation passee.${NC}"
else
    read -r -p "Tapez OUI en majuscules pour continuer : " accord
    [ "$accord" = "OUI" ] || { echo "Abandon."; exit 1; }
fi

# ── THE OWNER ACCOUNT'S SESSION ──────────────────────────────────────────
# Copying a home while its session runs gives a slightly inconsistent copy:
# dconf, browser history and cache files are rewritten continuously. Harmless
# HERE: the original is not deleted, it is moved aside intact at next boot,
# and what matters is the (static) keyring.
#
# THE SESSION IS NOT KILLED: this script is often run from a terminal that is
# PART of it; closing it between luksFormat and the copy would leave a half
# filled encrypted disk. The real switch happens at next boot, when nobody
# holds /home - unlock-home then moves the old content aside.
if [ -n "$OWNER" ] && loginctl list-sessions --no-legend 2>/dev/null | grep -qw "$OWNER"; then
    echo -e "${YELLOW}Session de $OWNER ouverte : la copie sera prise a chaud.${NC}"
    echo -e "${YELLOW}L original n est pas efface - il sera mis de cote au redemarrage.${NC}"
fi

# ── 1. ENCRYPTION ────────────────────────────────────────────────────────
echo
echo -e "${GREEN}[1/7]${NC} Creation du conteneur LUKS2 sur $DEV."
echo -e "      ${BOLD}Choisissez une phrase de passe longue.${NC} Elle ne se recupere pas :"
echo -e "      perdue, le contenu du disque l'est aussi."
if [ -n "$KEYFILE" ]; then
    cryptsetup luksFormat --type luks2 --label crypthome --batch-mode \
              --key-file="$KEYFILE" "$DEV"
```

```
else
    cryptsetup luksFormat --type luks2 --label crypthome "$DEV"
fi

echo
if [ -n "$KEYFILE" ]; then
    echo -e "${GREEN}[2/7]${NC} Ouverture du volume."
else
    echo -e "${GREEN}[2/7]${NC} Ouverture du volume (retapez la phrase de passe)."
fi
if [ -n "$KEYFILE" ]; then
    cryptsetup open --key-file="$KEYFILE" "$DEV" "$MAPPER"
else
    cryptsetup open "$DEV" "$MAPPER"
fi

UUID="$(cryptsetup luksUUID "$DEV")"
echo -e "      UUID LUKS : ${CYAN}$UUID${NC}"

# ── 3. FILE SYSTEM ──────────────────────────────────────────────────
echo -e "${GREEN}[3/7]${NC} Formatage ext4."
mkfs.ext4 -q -L crypthome "/dev/mapper/$MAPPER"

# ── 4. COPY OF /home ────────────────────────────────────────────────
echo -e "${GREEN}[4/7]${NC} Recopie du contenu actuel de $MOUNT."
mkdir -p "$TMP"
mount "/dev/mapper/$MAPPER" "$TMP"
# -aHAX: hard links, ACLs and extended attributes included - a home without
# its ACLs and xattrs ends up with subtly wrong permissions.
# --sparse: without it, sparse files (VM and disk images) are copied holes
# included, zero by zero.
# -x: mount points are not crossed; what is mounted UNDER the home belongs to
# another disk.
# --info=progress2 only on a terminal: run by systemd, the progress bar would
# write tens of thousands of lines to the journal.
_rs=(-aHAX --sparse -x)
[ -t 1 ] && _rs+=(--info=progress2)
rsync "${_rs[@]}" "$MOUNT"/ "$TMP"/
# Root's keyring lives here and nowhere else: /root/.local/share/keyrings is
# a link to this directory.
install -d -m 0700 "$TMP/.crypthome/root-keyrings"
chown -R root:root "$TMP/.crypthome"
sync
umount "$TMP"
rmdir "$TMP"

# ── 5. WIRING ───────────────────────────────────────────────────────
echo -e "${GREEN}[5/7]${NC} Ecriture de la configuration."
sed -i "s|^CRYPTHOME_UUID=.*|CRYPTHOME_UUID=\"$UUID\"|" "$CONF"

# crypttab: "noauto" - systemd must ask NOTHING by itself at boot; unlock-home
# asks the question and opens the volume.
sed -i "/^$MAPPER[[:space:]]/d" /etc/crypttab
printf '%-22s %-40s %-10s %s\n' "$MAPPER" "UUID=$UUID" "none" "luks,noauto" >> /etc/crypttab

# fstab: "noauto" for the same reason, and "nofail" as a safety net - an entry
# failing at boot on a system without the disk must not drop the machine into
# rescue mode.
sed -i "\|^[^#].*[[:space:]]$MOUNT[[:space:]]|d" /etc/fstab
printf '%-42s %-14s %-7s %s %s %s\n' \
    "/dev/mapper/$MAPPER" "$MOUNT" "ext4" "defaults,noauto,nofail" "0" "2" >> /etc/fstab

# ── 6. THE OLD /home ────────────────────────────────────────────────
# Left untouched on purpose: processes hold it right now (usually the session
# this script runs from). unlock-home moves it to /var/backups at next boot,
# BEFORE mounting the encrypted disk - the only moment nobody uses it.
```

```
echo -e "${GREEN}[6/7]${NC} Ancien $MOUNT laisse en place ; il sera mis de cote au redemarrage."

# ── 7. ENABLING ──────────────────────────────────────────────────────
echo -e "${GREEN}[7/7]${NC} Activation du service de demarrage."
systemctl daemon-reload
systemctl enable unlock-home.service >/dev/null
/usr/local/sbin/refresh-owner-apps >/dev/null
# "splash" stays on the kernel command line: unlock-home quits plymouth itself
# just before asking the question on the console.

cryptsetup close "$MAPPER"

echo
echo -e "${GREEN}${BOLD}Termine.${NC}"
echo -e "  Au prochain demarrage : ${CYAN}\"decrypt $MOUNT -passphrase- or Enter\"${NC}"
echo -e "  Taper la phrase (affichee en ${CYAN}****${NC}) monte $MOUNT et ouvre la session de $OWNER,"
echo -e "  avec son trousseau ; Firefox et les applications a secrets tourneront sous lui."
echo -e "  ${CYAN}Entree${NC} a vide (ou attendre ${CRYPTHOME_TIMEOUT:-60} s) demarre sans : la session root"
echo -e "  s'ouvre normalement, les memes applications tournent sans secrets."
echo
echo -e "  Apres coup, dans un terminal : ${CYAN}unlock-home${NC} / ${CYAN}lock-home${NC}"
echo -e "  Au redemarrage, l ancien $MOUNT part dans ${CYAN}/var/backups/home-avant-chiffrement-*${NC}"
echo -e "  - a effacer une fois le dechiffrement verifie."
echo
echo -e "  ${YELLOW}Verifiez le dechiffrement AVANT d'effacer la sauvegarde.${NC}"
exit 0
```

2.36 /opt/GSM/osmo-operator/configs/crypthome/lock-home

2308 octets, 56 lignes → 56 lignes

```
#!/bin/bash
# lock-home - demonte /home et referme le volume chiffre, en cours de session.
#
# A utiliser quand on veut rendre les secrets illisibles sans redemarrer. Les
# processus du compte proprietaire tiennent le point de montage : on les
# termine d'abord, sinon le demontage echoue avec "target is busy" et le
# volume reste ouvert - exactement ce qu'on cherchait a eviter.
set -u

CONF=/etc/default/crypthome
[ -r "$CONF" ] || { echo "lock-home: $CONF introuvable" >&2; exit 1; }
. "$CONF"

MAPPER="${CRYPTHOME_NAME:-crypthome}"
MOUNT="${CRYPTHOME_MOUNT:-/home}"
OWNER="${CRYPTHOME_OWNER:-}"

# ── LE MONTAGE DOIT VENIR DU VOLUME CHIFFRE ──────────────────────────
# "monte" ne suffit pas. /home peut etre une partition ORDINAIRE - une machine
# installee avec un /home separe, ou init-crypthome.sh n a jamais tourne. Traiter
# ce montage-la comme le disque chiffre ecrirait les secrets EN CLAIR sous le
# compte proprietaire tout en annoncant le contraire : le pire des etats, parce
# qu il a l air correct. On exige donc que la source du montage soit le mapping
# dechiffre lui-meme, et pas seulement qu il y ait quelque chose sur /home.
crypthome_monte() {
    local _src
    mountpoint -q "$MOUNT" || return 1
    _src="$(findmnt -fno SOURCE "$MOUNT" 2>/dev/null)"
    [ -n "$_src" ] || return 1
    [ "$(readlink -f "$_src")" = "$(readlink -f "/dev/mapper/$MAPPER")" ]
}

# Rien a fermer, ET rien a demonter qui nous appartienne : un /home ordinaire
```

```
# n est pas de notre ressort - le demonter tuerait des sessions pour rien.
if ! crypthome_monte && [ ! -e "/dev/mapper/$MAPPER" ]; then
    echo "lock-home: $MOUNT n'est pas dechiffre."
    exit 0
fi

if [ -n "$OWNER" ] && id -u "$OWNER" >/dev/null 2>&1; then
    loginctl disable-linger "$OWNER" >/dev/null 2>&1 || true
    loginctl terminate-user "$OWNER"  >/dev/null 2>&1 || true
    pkill -u "$OWNER" >/dev/null 2>&1 || true
    sleep 2
    pkill -KILL -u "$OWNER" >/dev/null 2>&1 || true
fi

if crypthome_monte && ! umount "$MOUNT"; then
    echo "lock-home: $MOUNT occupe - processus restants :" >&2
    fuser -vm "$MOUNT" 2>&1 | head -20 >&2
    exit 1
fi

cryptsetup close "$MAPPER" 2>/dev/null || true
echo "lock-home: $MOUNT demonte, volume referme."
exit 0
```

2.37 /opt/GSM/osmo-operator/configs/crypthome/refresh-owner-apps

2606 octets, 68 lignes → 68 lignes

```
#!/bin/bash
# refresh-owner-apps - redirige les lanceurs des applications a secrets vers
# run-as-owner.
#
# Pour chaque .desktop de CRYPTHOME_OWNER_APPS trouve dans /usr/share/
# applications, ecrit une COPIE dans /usr/local/share/applications dont chaque
# ligne Exec= (celle du lanceur ET celles des [Desktop Action ...]) est
# prefixee de run-as-owner. /usr/local/share precede /usr/share dans
# XDG_DATA_DIRS : la copie masque l'original sans le modifier, et un
# "apt upgrade" ne defait rien.
#
# A relancer apres avoir installe une application ou allonge la liste.
set -u

CONF=/etc/default/crypthome
[ -r "$CONF" ] && . "$CONF"
APPS="${CRYPTHOME_OWNER_APPS:-}"
DEST=/usr/local/share/applications
SRCDIRS="/usr/share/applications /var/lib/flatpak/exports/share/applications"

install -d "$DEST"

# Menage : les copies posees par ce script et dont la source a disparu, ou qui
# ne sont plus dans la liste. Le marqueur X-Crypthome-Generated distingue nos
# fichiers de ceux que l'operateur aurait mis la lui-meme - on ne touche
# jamais a ceux-la.
for f in "$DEST"/*.desktop; do
    [ -e "$f" ] || continue
    grep -q '^X-Crypthome-Generated=true' "$f" 2>/dev/null || continue
    nom="$(basename "$f" .desktop)"
    case " $APPS " in
        *" $nom "*) ;;
        *) rm -f "$f"; echo "  - $nom (retire de la liste)" ;;
    esac
done
```

```
poses=0
for app in $APPS; do
    src=""
    for d in $SRCDIRS; do
        [ -f "$d/$app.desktop" ] && { src="$d/$app.desktop"; break; }
    done
    [ -n "$src" ] || continue

    dst="$DEST/$app.desktop"
    # sed sur Exec= uniquement, et idempotent : une ligne deja prefixee n'est
    # pas prefixee deux fois (relancer le script ne doit rien empiler).
    sed -E 's#^Exec=(/usr/local/bin/run-as-owner )?#Exec=/usr/local/bin/run-as-owner #' "$src" > "$dst.tmp"
    # Le marqueur en fin de groupe [Desktop Entry] : une cle inconnue prefixee
    # X- est ignoree par les menus, c'est la convention de la specification.
    if ! grep -q '^X-Crypthome-Generated=true' "$dst.tmp"; then
        awk '
            /^\[/ && n++ == 1 { print "X-Crypthome-Generated=true"; print "" }
            { print }
            END { if (n < 2) print "X-Crypthome-Generated=true" }
        ' "$dst.tmp" > "$dst.tmp2" && mv "$dst.tmp2" "$dst.tmp"
    fi
    mv "$dst.tmp" "$dst"
    chmod 0644 "$dst"
    poses=$((poses + 1))
    echo "  + $app -> run-as-owner"
done

command -v update-desktop-database >/dev/null 2>&1 && \
    update-desktop-database "$DEST" >/dev/null 2>&1 || true

echo "refresh-owner-apps: $poses lanceur(s) redirige(s) vers ${CRYPTHOME_OWNER:-<aucun>}."
exit 0
```

2.38 /opt/GSM/osmo-operator/configs/crypthome/run-as-owner

6618 octets, 129 lignes → 129 lignes

```
#!/bin/bash
# run-as-owner <commande> [arguments...]
#
# Lance une application sous le compte dont le home vit sur le disque chiffre
# (CRYPTHOME_OWNER) QUAND ce disque est dechiffre ; sinon la lance telle quelle,
# sous le compte courant.
#
# POURQUOI. La session s'ouvre sur root : c'est le compte de travail du banc,
# et tout y est lisible. Mais un navigateur, un trousseau, un client de
# messagerie ecrivent des SECRETS - cookies, jetons, mots de passe. Ceux-la
# n'ont rien a faire sur le disque systeme en clair : ils appartiennent au
# compte proprietaire, dont tout le home est chiffre.
#
# Disque non dechiffre = pas de home chiffre = l'application demarre quand meme,
# en root, sur un profil neuf du disque systeme. On prefere une application qui
# s'ouvre sans secrets a une application qui refuse de s'ouvrir.
set -u

CONF=/etc/default/crypthome
[ -r "$CONF" ] && . "$CONF"
OWNER="${CRYPTHOME_OWNER:-}"
MOUNT="${CRYPTHOME_MOUNT:-/home}"
MAPPER="${CRYPTHOME_NAME:-crypthome}"

# —— LE MONTAGE DOIT VENIR DU VOLUME CHIFFRE ——————————————————————
# "monte" ne suffit pas. /home peut etre une partition ORDINAIRE - une machine
# installee avec un /home separe, ou init-crypthome.sh n a jamais tourne. Traiter
```

```
# ce montage-la comme le disque chiffre ecrirait les secrets EN CLAIR sous le
# compte proprietaire tout en annoncant le contraire : le pire des etats, parce
# qu il a l air correct. On exige donc que la source du montage soit le mapping
# dechiffre lui-meme, et pas seulement qu il y ait quelque chose sur /home.
crypthome_monte() {
    local _src
    mountpoint -q "$MOUNT" || return 1
    _src="$(findmnt -fno SOURCE "$MOUNT" 2>/dev/null)"
    [ -n "$_src" ] || return 1
    [ "$(readlink -f "$_src")" = "$(readlink -f "/dev/mapper/$MAPPER")" ]
}


[ "$#" -ge 1 ] || { echo "usage: run-as-owner <commande> [args...]" >&2; exit 2; }

# Deja sous un compte ordinaire : rien a basculer.
if [ "$(id -u)" -ne 0 ] || [ -z "$OWNER" ] || ! id -u "$OWNER" >/dev/null 2>&1; then
    exec "$@"
fi

OWNER_UID="$(id -u "$OWNER")"
OWNER_HOME="$(getent passwd "$OWNER" | cut -d: -f6)"
OWNER_SHELL="$(getent passwd "$OWNER" | cut -d: -f7)"
[ -n "$OWNER_SHELL" ] || OWNER_SHELL=/bin/bash

# Le volume chiffre doit etre monte ET le home exister dessus. Les deux : un
# /home monte mais vide (disque fraichement initialise) ne doit pas faire
# demarrer une application sur un home inexistant.
if ! crypthome_monte || [ ! -d "$OWNER_HOME" ]; then
    logger -t run-as-owner -p user.notice \
        "$1 lance sous root : $MOUNT n'est pas le volume chiffre dechiffre, ou pas de profil $OWNER."
    exec "$@"
fi

# ── L'ACCES A L'AFFICHAGE ──────────────────────────────────────────────
# Le serveur X appartient a root. SI:localuser autorise le compte par ses
# credentials locaux, sans partager le cookie de root : on N'EXPORTE PAS
# XAUTHORITY, qui pointe vers un fichier que le compte ne peut pas lire et
# ferait echouer les bibliotheques qui le lisent avant de se rabattre.
if [ -n "${DISPLAY:-}" ] && command -v xhost >/dev/null 2>&1; then
    xhost "+SI:localuser:$OWNER" >/dev/null 2>&1 || true
fi

# ── LA SESSION UTILISATEUR DU COMPTE ───────────────────────────────────
# lingering : logind cree /run/user/<uid> et demarre user@<uid>.service sans
# qu'une session soit ouverte - or personne ne se connecte jamais sous ce
# compte, les applications sont lancees depuis la session de root.
if [ ! -d "/run/user/$OWNER_UID" ]; then
    loginctl enable-linger "$OWNER" >/dev/null 2>&1 || true
    for _ in 1 2 3 4 5 6 7 8 9 10; do
        [ -d "/run/user/$OWNER_UID" ] && break
        sleep 0.3
    done
fi
if [ ! -d "/run/user/$OWNER_UID" ]; then
    install -d -m 0700 -o "$OWNER" -g "$(id -gn "$OWNER")" "/run/user/$OWNER_UID"
fi

RUNTIME="/run/user/$OWNER_UID"
BUS="unix:path=$RUNTIME/bus"
[ -S "$RUNTIME/bus" ] || BUS=""

# ── LE SON ─────────────────────────────────────────────────────────────
# PulseAudio tourne en mode SYSTEME : un seul socket, /run/pulse/native, et un
# seul proprietaire - celui de la derniere session humaine ouverte, donc root
# ici. Or c'est le navigateur qui porte le micro du tableau de bord, et il
```

```
# tourne desormais sous le compte proprietaire : sans ce rappel, il obtient
# « NotFoundError » sur getUserMedia pendant que pactl liste les peripheriques
# en RUNNING - l'ecart qu'on cherche des heures du cote « permission micro ».
# osmo-pulse-link.sh pose le lien dans /run/user/<uid> ET donne le socket a cet
# uid. La session root n'a pas de navigateur : elle n'en perd rien.
if [ -x /usr/local/sbin/osmo-pulse-link.sh ]; then
    /usr/local/sbin/osmo-pulse-link.sh "$OWNER_UID" >/dev/null 2>&1 || true
fi
# Et l'adresse en clair, plutot que de s'en remettre au seul lien : c'est ce
# que fait deja le wrapper vlc de l'etape 8e. Deux chemins valent mieux qu'un
# quand le symptome d'un echec est un navigateur muet sans message.
PULSE=""
[ -S /run/pulse/native ] && PULSE="unix:/run/pulse/native"

# ── LE TROUSSEAU ──────────────────────────────────────────────────────
# Demarre sous le compte proprietaire, donc AVEC son home chiffre : c'est la
# seule facon d'avoir un service Secrets dont le magasin n'est pas sur le
# disque en clair. Deja lance = ne fait rien.
runuser -u "$OWNER" -- env -i \
    HOME="$OWNER_HOME" USER="$OWNER" LOGNAME="$OWNER" \
    XDG_RUNTIME_DIR="$RUNTIME" ${BUS:+DBUS_SESSION_BUS_ADDRESS="$BUS"} \
    PATH=/usr/local/bin:/usr/bin:/bin \
    gnome-keyring-daemon --start --components=secrets,ssh >/dev/null 2>&1 || true

exec runuser -u "$OWNER" -- env -i \
    HOME="$OWNER_HOME" USER="$OWNER" LOGNAME="$OWNER" SHELL="$OWNER_SHELL" \
    PATH=/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin:/sbin \
    DISPLAY="${DISPLAY:-:0}" \
    XDG_RUNTIME_DIR="$RUNTIME" \
    ${BUS:+DBUS_SESSION_BUS_ADDRESS="$BUS"} \
    XDG_SESSION_TYPE=x11 \
    ${PULSE:+PULSE_SERVER="$PULSE"} \
    XDG_CURRENT_DESKTOP="${XDG_CURRENT_DESKTOP:-ubuntu:GNOME}" \
    XDG_DATA_DIRS="${XDG_DATA_DIRS:-/usr/local/share:/usr/share}" \
    LANG="${LANG:-fr_FR.UTF-8}" LANGUAGE="${LANGUAGE:-}" \
    "$@"
```

2.39 /opt/GSM/osmo-operator/configs/crypthome/unlock-home

18108 octets, 401 lignes → 401 lignes

```
#!/bin/bash
# unlock-home - decrypts and mounts /home from the second (LUKS) disk.
#
#   unlock-home --boot   asks the passphrase at boot, on the text console
#   unlock-home          the same from a terminal, afterwards
#
# Settings: /etc/default/crypthome (CRYPTHOME_UUID, _NAME, _MOUNT, _OWNER,
# _TRIES, _TIMEOUT, _PROMPT, _IMAGE, _RAM_DIRS, _ROOT_LINKS).
#
# THIS SCRIPT NEVER FAILS THE BOOT. On an empty Enter, a missing disk, a wrong
# passphrase $TRIES times or a failed mount, it exits 0 and the machine boots
# with an empty /home: the system stays usable as root, whose home (/root) is
# on the system disk in clear.
#
# THE DESKTOP THEN OPENS ON ROOT. On an installed disk the session account has
# its home ON the encrypted volume: without /home, GDM would open it on a
# non-existent home. When /home stays closed at boot (Enter, timeout, refused
# passphrase, missing disk), GDM opens a ROOT session for this boot only - see
# session_root below.
set -u

CONF=/etc/default/crypthome
[ -r "$CONF" ] || { echo "unlock-home: $CONF introuvable" >&2; exit 0; }
```

```
. "$CONF"

MAPPER="${CRYPTHOME_NAME:-crypthome}"
MOUNT="${CRYPTHOME_MOUNT:-/home}"
TRIES="${CRYPTHOME_TRIES:-3}"
TIMEOUT="${CRYPTHOME_TIMEOUT:-60}"
OWNER="${CRYPTHOME_OWNER:-}"

BOOT=0
[ "${1:-}" = "--boot" ] && BOOT=1

log() { echo "unlock-home: $*" >&2; }

# ── THE MOUNT MUST COME FROM THE ENCRYPTED VOLUME ──────────────────────
# "mounted" is not enough: /home may be an ORDINARY partition (a machine
# installed with a separate /home where init-crypthome.sh never ran). Treating
# that mount as the encrypted disk would write the secrets IN CLEAR under the
# owner account while claiming otherwise. The mount source must therefore be
# the decrypted mapping itself.
crypthome_monte() {
    local _src
    mountpoint -q "$MOUNT" || return 1
    _src="$(findmnt -fno SOURCE "$MOUNT" 2>/dev/null)"
    [ -n "$_src" ] || return 1
    [ "$(readlink -f "$_src")" = "$(readlink -f "/dev/mapper/$MAPPER")" ]
}

# ── THE ROOT SESSION WHEN /home STAYS CLOSED ───────────────────────────
# GDM has no "for this boot" configuration: it reads custom.conf. It is thus
# modified BEFORE GDM starts (unlock-home.service has Before=display-manager),
# the original files being saved in $ETAT_GDM. Every boot starts by restoring
# them (session_normale): the root session lasts one boot, and a power cut
# does not make it permanent.
#
# Two files: GDM refuses uid 0 not in custom.conf but in PAM, through the rule
# "pam_succeed_if.so user != root". It is neutralised ONLY in gdm-autologin:
# the password login screen keeps refusing root.
#
# No added risk: the root file system is in clear by construction
# (partition.conf); whoever holds the machine can already read it from a live
# key. What is protected is /home - precisely what is not mounted.
GDM_CONF=/etc/gdm3/custom.conf
GDM_PAM=/etc/pam.d/gdm-autologin
ETAT_GDM=/var/lib/crypthome/gdm-avant-session-root

session_normale() {
    [ -d "$ETAT_GDM" ] || return 0
    if [ -f "$ETAT_GDM/custom.conf" ]; then
        cp -p "$ETAT_GDM/custom.conf" "$GDM_CONF"
    elif [ -f "$ETAT_GDM/custom.conf.absent" ]; then
        rm -f "$GDM_CONF"
    fi
    [ -f "$ETAT_GDM/gdm-autologin" ] && cp -p "$ETAT_GDM/gdm-autologin" "$GDM_PAM"
    rm -rf "$ETAT_GDM"
    log "GDM : configuration du compte de session retablie."
}

session_root() {
    [ "$BOOT" = 1 ] || return 0
    [ -d /etc/gdm3 ] || return 0
    # Already done (two calls in the same boot): do not overwrite the saved
    # originals with the already modified version.
    if [ ! -d "$ETAT_GDM" ]; then
        install -d -m 0700 "$ETAT_GDM" || return 0
        if [ -f "$GDM_CONF" ]; then
            cp -p "$GDM_CONF" "$ETAT_GDM/custom.conf"
```

```
        else
            : > "$ETAT_GDM/custom.conf.absent"
        fi
        [ -f "$GDM_PAM" ] && cp -p "$GDM_PAM" "$ETAT_GDM/gdm-autologin"
    fi
    # Only these two keys in [daemon]: the rest (WaylandEnable, a possible
    # TimedLogin) is the machine's configuration.
    awk '
        /^[[:space:]]*AutomaticLogin(Enable)?[[:space:]]*=/ { next }
        { print }
        /^[[:space:]]*\[daemon\][[:space:]]*$/ && !fait {
            print "AutomaticLoginEnable=true"; print "AutomaticLogin=root"; fait = 1
        }
        END { if (!fait) { print "[daemon]"; print "AutomaticLoginEnable=true"; print "AutomaticLogin=root" } }
    ' "${GDM_CONF}" 2>/dev/null > "$GDM_CONF.crypthome" \
        && mv "$GDM_CONF.crypthome" "$GDM_CONF"
    [ -f "$GDM_PAM" ] && \
        sed -i '/^[[:space:]]*auth.*pam_succeed_if.so user != root quiet_success/s/^/#/' "$GDM_PAM"
    log "GDM : session root pour ce demarrage ($MOUNT ferme)."
    root_dans_tmp
}

# ── WHAT THE ROOT SESSION WRITES STAYS IN /tmp ──────────────────────────
# Without /home, the root session must leave nothing on the disk: what it
# writes in its home (/root) - profiles, history, files - lives in $RAM_ROOT,
# a tmpfs under /tmp, and disappears at shutdown. /root is covered by an
# overlay: its content stays READABLE (launchers, bench scripts,
# init-crypthome.sh), only writes go to /tmp.
#
# /opt/GSM too: the whole bench lives there (repository, configs, hlr.db,
# logs, captures). In a root session it is "live", non persistent, and gets
# its on-disk state back at reboot. The list of directories covered this way
# is CRYPTHOME_RAM_DIRS (/etc/default/crypthome); /root is always part of it.
# A missing directory is ignored; a directory that is itself a mount point
# (second disk on /opt/GSM via fstab) is covered like any other: local-fs.target
# has already been reached.
#
# A tmpfs and not a plain directory in /tmp: on Ubuntu /tmp is on the root
# disk, in clear; in RAM nothing ever touches a disk.
#
# The keyring: the link /root/.local/share/keyrings points to the closed
# volume. It is replaced, IN the overlay only, by a real directory: the session
# has a working keyring that evaporates with it.
RAM_ROOT=/tmp/crypthome-root
RAM_DIRS="${CRYPTHOME_RAM_DIRS-/opt/GSM}"

# en_ram DIR: covers DIR with an overlay whose upper and work directories live
# in $RAM_ROOT. Mode and owner of the upper = those of DIR: an overlay root
# takes the attributes of its upper, and a /opt/GSM turned 0700 would cut
# asterisk (osmo-banc) and the session account off their working directory.
en_ram() {
    local dir="$1" nom
    [ -d "$dir" ] || return 1
    [ "$(findmnt -fno FSTYPE "$dir" 2>/dev/null)" = overlay ] && return 0   # already done this boot
    nom="$(printf '%s' "$dir" | sed 's#^/##; s#/#_#g')"
    install -d -m "$(stat -c %a "$dir")" -o "$(stat -c %u "$dir")" -g "$(stat -c %g "$dir")" \
            "$RAM_ROOT/$nom/upper" || return 1
    install -d -m 0700 "$RAM_ROOT/$nom/work" || return 1
    mount -t overlay overlay \
          -o "lowerdir=$dir,upperdir=$RAM_ROOT/$nom/upper,workdir=$RAM_ROOT/$nom/work" "$dir"
}

root_dans_tmp() {
    local lien d
    if ! mountpoint -q "$RAM_ROOT"; then
        install -d -m 0700 "$RAM_ROOT" || return 0
```

```
        if ! mount -t tmpfs -o mode=0700,size=50% tmpfs "$RAM_ROOT"; then
            log "ATTENTION : tmpfs $RAM_ROOT impossible - la session root ecrira sur le disque."
            return 0
        fi
        # systemd-tmpfiles-clean cleans /tmp by age: it must not empty the
        # home of a running session.
        install -d /run/tmpfiles.d
        echo "x $RAM_ROOT" > /run/tmpfiles.d/crypthome-root.conf
    fi
    if en_ram /root; then
        for lien in ${CRYPTHOME_ROOT_LINKS:-.local/share/keyrings}; do
            if [ -L "/root/$lien" ]; then
                rm -f "/root/$lien"
                install -d -m 0700 "/root/$lien"
            fi
        done
        log "session root : les ecritures dans /root restent dans $RAM_ROOT (RAM)."
    else
        log "ATTENTION : overlay sur /root impossible - la session root ecrira sur le disque."
    fi
    for d in $RAM_DIRS; do
        [ "$d" = /root ] && continue
        [ -d "$d" ] || continue
        if en_ram "$d"; then
            log "session root : les ecritures dans $d restent dans $RAM_ROOT (RAM)."
        else
            log "ATTENTION : overlay sur $d impossible - $d ecrira sur le disque."
        fi
    done
}

# Every boot starts from the session account's configuration.
[ "$BOOT" = 1 ] && session_normale

# Already mounted (2nd call, or manual decryption after a "no"): nothing to do.
#
# If it is ANOTHER mount - a separate /home in clear - exit too, without
# moving anything: the operator did not ask to encrypt that partition, and
# mounting over it is impossible. The message names the source, otherwise the
# symptom is "it never asks the passphrase" with no explanation.
if mountpoint -q "$MOUNT"; then
    if crypthome_monte; then
        log "$MOUNT est deja monte depuis le volume chiffre."
    else
        log "$MOUNT est deja monte, mais pas depuis /dev/mapper/$MAPPER" \
            "($(findmnt -fno SOURCE "$MOUNT" 2>/dev/null)) - rien n a ete touche."
    fi
    exit 0
fi

if [ -z "${CRYPTHOME_UUID:-}" ]; then
    log "aucun disque chiffre configure (CRYPTHOME_UUID vide dans $CONF)."
    exit 0
fi

# ── THE OLD /home IN CLEAR MUST GO ──────────────────────────────────────
# What remains under the mount point becomes invisible once the disk is
# mounted - invisible, but still there: space on the system disk and, what
# matters here, OLD KEYRINGS IN CLEAR. It is moved aside BEFORE mounting.
#
# Done HERE and not by init-crypthome.sh because this is the only moment
# nobody holds /home: at boot, before the first session.
#
# Unconditional, not "only on yes": once an encrypted disk is configured, the
# /home in clear is no longer the right one. Answering no must leave an EMPTY
# /home, not the old one, otherwise "no" would hand back the very secrets
```

```
# being protected.
if [ -d "$MOUNT" ] && [ -n "$(ls -A "$MOUNT" 2>/dev/null)" ]; then
    _sauve="/var/backups/home-avant-chiffrement-$(date +%Y%m%d-%H%M%S)"
    if install -d -m 0700 "$_sauve" 2>/dev/null; then
        ( shopt -s dotglob nullglob
          _reste=("$MOUNT"/*)
          [ "${#_reste[@]}" -gt 0 ] && mv "${_reste[@]}" "$_sauve"/ ) 2>/dev/null
        log "ancien contenu de $MOUNT mis de cote dans $_sauve."
    else
        log "ATTENTION : $MOUNT n est pas vide et n a pas pu etre evacue."
    fi
fi

DEV="/dev/disk/by-uuid/$CRYPTHOME_UUID"
if [ ! -e "$DEV" ]; then
    log "disque chiffre absent (UUID=$CRYPTHOME_UUID) - $MOUNT reste vide."
    session_root
    exit 0
fi

# ── THE ONLY QUESTION: THE PASSPHRASE ──────────────────────────────────
# A single input, which is both the answer AND the passphrase:
#   - empty ENTER (or the timeout expiring)  : boot as ROOT, without /home;
#   - a passphrase                           : open the volume; the session
#     is the owner account's.
# A refused passphrase asks again, $TRIES times; at any time an empty Enter
# exits to root. An expired timeout counts as Enter: a machine rebooting on
# its own finishes its boot instead of waiting forever.
#
# The prompt fits on one line, the same everywhere: "decrypt /home -passphrase-
# or Enter". Typed characters are shown as "*" (lire_masquee below; "read -s"
# would show nothing and the operator could not tell whether keys are taken).
#
# WHERE IT IS ASKED: ON THE TEXT CONSOLE, NOT IN PLYMOUTH. Plymouth reads the
# keyboard through its own path (the tty it holds, or evdev + xkb), which is
# dead on some hardware (Intel + NVIDIA laptops with the proprietary driver
# loaded after the initrd) when unlock-home runs. So if plymouth runs it is
# QUIT, the text console comes back, and the passphrase is read directly on
# /dev/console - no agent, no unit condition, no plymouth keyboard. GDM then
# starts as usual (its "plymouth quit --retain-splash" finds nobody).
#
# CRYPTHOME_PROMPT=plymouth in /etc/default/crypthome selects the other path
# (systemd-ask-password -> plymouth agent -> osmo-bts theme), for a machine
# where the plymouth keyboard works. --key-file=- in both cases: the passphrase
# goes neither through the command line (visible in ps) nor through the disk,
# only through a pipe.
lire_masquee() {
    # lire_masquee VAR PROMPT: reads a line from the keyboard, one "*" per
    # character, Backspace handled. Returns 1 if the timeout expires (the
    # variable is then empty).
    local _var="$1" _invite="$2" _c _s="" _rc
    printf '%s' "$_invite" >&2
    while :; do
        IFS= read -r -s -n1 -t "$TIMEOUT" _c; _rc=$?
        if [ "$_rc" -gt 128 ]; then
            echo >&2; printf -v "$_var" '%s' ""; return 1
        fi
        [ "$_rc" -ne 0 ] && break            # end of input
        [ -z "$_c" ] && break                # Enter
        if [ "$_c" = $'\x7f' ] || [ "$_c" = $'\b' ]; then
            if [ -n "$_s" ]; then _s="${_s%?}"; printf '\b \b' >&2; fi
            continue
        fi
        _s+="$_c"; printf '*' >&2
    done
    echo >&2
```

```
    printf -v "$_var" '%s' "$_s"
    return 0
}

# The console where the question is asked at boot. /dev/console follows the
# kernel's "console=" (tty0 = the active virtual console, or a serial port if
# requested); tty1 as a fallback.
CONSOLE=/dev/console
[ -c "$CONSOLE" ] || CONSOLE=/dev/tty1

demander_console() {
    # demander_console VAR PROMPT: the question on the text console, at boot.
    # plymouth, if running, is quit first: the screen goes back to text and
    # the keyboard returns to the tty.
    if command -v plymouth >/dev/null 2>&1 && plymouth --ping 2>/dev/null; then
        plymouth quit 2>/dev/null || true
        log "plymouth quitte - la question passe sur la console ($CONSOLE)."
    fi
    # An image behind the question: the text console is cleared (ESC[2J),
    # crypthome-fond paints CRYPTHOME_IMAGE on /dev/fb0 (darkened at the
    # bottom), and the prompt goes to the LAST line (ESC[999;1H): fbcon only
    # repaints the cells it writes. Input is unchanged (lire_masquee on the
    # tty). CRYPTHOME_IMAGE=none disables it.
    if [ "${CRYPTHOME_IMAGE:-}" != none ] && [ -x /usr/local/sbin/crypthome-fond ]; then
        printf '\033[2J' > "$CONSOLE" 2>/dev/null
        /usr/local/sbin/crypthome-fond ${CRYPTHOME_IMAGE:+"$CRYPTHOME_IMAGE"} 2>&1 | while read -r l; do log "$l";
        ↪  done
        printf '\033[999;1H' > "$CONSOLE" 2>/dev/null
    fi
    lire_masquee "$1" "$2" < "$CONSOLE" 2> "$CONSOLE"
}

# ── THE KEYBOARD LAYOUT, LOGGED ──────────────────────────────────────
# A passphrase refused three times is most often a keyboard layout other than
# the expected one. The console at that moment has the layout of
# /etc/default/keyboard (written by build-iso, then by Calamares at
# installation): it is logged so that "passphrase refused" can be read next
# to the layout. Fallback: /etc/vconsole.conf, then "?".
clavier_console() {
    local _l _v
    _l="$(awk -F'"' '/^XKBLAYOUT=/{print $2}' /etc/default/keyboard 2>/dev/null)"
    _v="$(awk -F'"' '/^XKBVARIANT=/{print $2}' /etc/default/keyboard 2>/dev/null)"
    if [ -z "$_l" ]; then
        _l="$(awk -F= '/^KEYMAP=/{print $2}' /etc/vconsole.conf 2>/dev/null | tr -d '"')"
        _v=""
    fi
    [ -n "$_l" ] || { echo "?"; return 0; }
    _l="${_l%%,*}"; _v="${_v%%,*}"
    [ -n "$_v" ] && _l="$_l-$_v"
    echo "$_l"
}
log "clavier de la console : $(clavier_console)."

PROMPT_MODE="${CRYPTHOME_PROMPT:-console}"
ouvert=0
essai=0
while [ "$essai" -lt "$TRIES" ]; do
    essai=$((essai + 1))
    invite="decrypt $MOUNT -passphrase- or Enter"
    [ "$essai" -gt 1 ] && invite="$invite ($essai/$TRIES)"
    if [ "$BOOT" = 1 ] && [ "$PROMPT_MODE" = plymouth ]; then
        passe="$(systemd-ask-password --timeout="$TIMEOUT" \
                    --id="crypthome:passphrase" "$invite" 2>/dev/null)" || passe=""
    elif [ "$BOOT" = 1 ]; then
        demander_console passe "$invite : " || passe=""
    else
```

```
        lire_masquee passe "$invite : " || passe=""
    fi
    if [ -z "$passe" ]; then
        unset passe
        log "Entree sans phrase - demarrage en root sans $MOUNT (« unlock-home » pour le faire plus tard)."
        session_root
        exit 0
    fi
    if printf '%s' "$passe" | cryptsetup open --key-file=- "$DEV" "$MAPPER"; then
        passe=""; unset passe
        ouvert=1
        break
    fi
    passe=""; unset passe
    log "phrase de passe refusee."
done

if [ "$ouvert" != 1 ]; then
    log "abandon apres $TRIES essais - demarrage en root sans $MOUNT."
    session_root
    exit 0
fi

# ── THE MOUNT ──────────────────────────────────────────────────────────
# The /home line of /etc/fstab has "noauto": this script mounts, and it alone.
# If the mount fails the mapping is CLOSED again - an open, unmounted decrypted
# volume would be the worst of both states.
if ! mount "$MOUNT" 2>/dev/null && ! mount "/dev/mapper/$MAPPER" "$MOUNT"; then
    log "montage de $MOUNT impossible - le volume est referme."
    cryptsetup close "$MAPPER" 2>/dev/null || true
    session_root
    exit 0
fi

# The owner account can now have a user session: its home exists. Without
# lingering, /run/user/<uid> and its bus are only created at first login - but
# run-as-owner starts its applications from root's session without ever
# logging that account in.
if [ -n "$OWNER" ] && id -u "$OWNER" >/dev/null 2>&1; then
    loginctl enable-linger "$OWNER" >/dev/null 2>&1 || true
fi

# /home open: the session is the account's, persistent on the encrypted
# volume. Also useful outside boot: "unlock-home" typed from the root session
# puts GDM back on the account for the next login.
session_normale

log "$MOUNT dechiffre et monte."
exit 0
```

2.40 /opt/GSM/osmo-operator/configs/screenfetch/osmo-lab.sh

13106 octets, 31 lignes → 31 lignes

```
#!/bin/bash
# configs/screenfetch/osmo-lab.sh - pixel-art logo of the lab for "screenfetch -a".
# GENERATED by tools/screenfetch-art.py: edit the drawing there, not here.
# Sourced by screenfetch (asciiText): startline, fulloutput (one line = one
# trailing %s, 32 visible columns each), labelcolor / textcolor.
#
# The colours of the INFO lines are not set here (screenfetch has already
# built them when it sources this file): osmo-banner filters the output
# (user yellow, host green; root red, host yellow).
labelcolor=$'\e[1;33m'
```

```
textcolor=$'\e[0m'
startline="0"
logowidth="32"
fulloutput=(
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;200;208;228m▄\e[0m\e[38;2;16;16;24m
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m▄\e[0m        %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;200;208;228m\e[48;2;84;96;130m▄\e[0m\e[38;2;84;96;13
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;200;70;60m\e[48;2;216;204;160m▄\e[0m\e[38;2;232;110;100m\e[48;2;200;70;60m▄\e[0m\e[38;2;16;1
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;200;208;228m\e[48;2;16;16;24m▄\e[0m\e[38;2;84;96;130m\e[48;2;84;96;130m▄\e[
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;200;208;228m\e[48;2;138;148;180m▄\e[0m\e[38;2;84;96;130m\e[48;2;84;96;130m▄
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;200;208;228m\e[48;2;138;148;180m▄\e[0m\e[38;2;84;96;
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;138;148;180m\e[48;2;16;16;24m▄\e[0m\e[38;2;84;96;130
 ↪  \e[38;2;255;176;0m▄\e[0m     %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;138;148;180m\e[48;2;16;16;24m▄\e[0m\e[38;2;138;148;180m\e[48;2;16;16;24m▄\e[0m\e[38;2;138;14
 ↪  \e[38;2;255;176;0m▄\e[0m   %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;150;156;176m▄\e[0m\e[38;2;150;156;176m\e[48;2;150;156;176m▄\e[0m\e[38;2;16;
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m   %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;150;156;176m\e[48;2;150;156;1
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;150;156;176m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;200;70;60
 ↪  \e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m▄\e[0m\e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;16;16;
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;150;156;176m\e[48;2;150;156;176m▄\e[0m\e[38;2;150;156;176m\e[48;2;150;156;1
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;60;66;88m▄\e[0m\e[38;2;70;220;120m\e[48;2;70;220;120m▄\e[0
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;150;156;176m\e[48;2;150;156;176m▄\e[0m\e[38;2;150;156;176m\e[48;2;150;156;1
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;60;66;88m▄\e[0m\e[38;2;60;66;88m\e[48;2;96;104;134m▄\e[0m\
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;150;156;176m\e[48;2;16;16;24m▄\e[0m\e[38;2;150;156;176m\e[48;2;16;16;24m▄\e
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;60;66;88m▄\e[0m\e[38;2;96;104;134m\e[48;2;96;104;134m▄\e[0
m▄\e[0m   %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;200;208;228m\e[48;2;140;180;140m▄\e[0m\e[38;2;200;208;228m\e[48;2;140;180;1
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;16;16;24m▄\e[0m\e[
]
 ↪  88m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;16;16;24m▄\e[0m\e[38;2;60;66;88m\e[48;2;16;16;24m▄\e[0m\e[38;2;16;16;24m
 ↪  %s"
"
 ↪  \e[38;2;16;16;24m\e[48;2;16;16;24m▄\e[0m\e[38;2;90;138;90m\e[48;2;16;16;24m▄\e[0m\e[38;2;90;138;90m\e[48;2;16;16;24m▄\e[0m\
 ↪  %s")
```

2.41 /opt/GSM/osmo-operator/data/tutorial.html

5685 octets, 109 lignes → 109 lignes

```
<!doctype html>
<!-- tutorial.html - quick-start page (French) served by the dashboard at
```

```
     /tutorial.html and installed to /usr/share/osmo-operator/tutorial.html. -->
<html lang="fr">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>osmo-operator — Démarrage rapide</title>
<style>
  :root{
    --fond:#f7f6f4; --carte:#ffffff; --texte:#1c1b19; --doux:#5f5c57;
    --trait:#e0ddd7; --rouge:#b3160b; --bleu:#1b4788; --code:#f0eee9;
  }
  @media (prefers-color-scheme: dark){
    :root:not([data-theme="light"]){
      --fond:#161513; --carte:#201f1c; --texte:#eceae6; --doux:#a5a09a;
      --trait:#35332e; --rouge:#f0654e; --bleu:#7aa8e8; --code:#2a2825;
    }
  }
  :root[data-theme="dark"]{
    --fond:#161513; --carte:#201f1c; --texte:#eceae6; --doux:#a5a09a;
    --trait:#35332e; --rouge:#f0654e; --bleu:#7aa8e8; --code:#2a2825;
  }
  *{box-sizing:border-box}
  body{margin:0;background:var(--fond);color:var(--texte);
       font:16px/1.6 system-ui,-apple-system,"Segoe UI",Roboto,sans-serif}
  .page{max-width:760px;margin:0 auto;padding:40px 24px 64px}
  h1{font-size:28px;margin:0 0 4px;letter-spacing:-.01em}
  .sous{color:var(--doux);margin:0 0 32px}
  h2{font-size:17px;margin:32px 0 10px;padding-top:16px;border-top:1px solid var(--trait)}
  h2:first-of-type{border-top:0;padding-top:0}
  p{margin:0 0 12px}
  ol,ul{margin:0 0 12px;padding-left:22px}
  li{margin:4px 0}
  code{background:var(--code);padding:1px 6px;border-radius:4px;
       font:14px/1.5 ui-monospace,"SF Mono",Menlo,Consolas,monospace}
  table{width:100%;border-collapse:collapse;margin:0 0 12px;font-size:15px}
  th,td{text-align:left;padding:7px 10px;border-bottom:1px solid var(--trait)}
  th{color:var(--doux);font-weight:600;font-size:13px;text-transform:uppercase;letter-spacing:.04em}
  td:first-child{white-space:nowrap;font:14px ui-monospace,Menlo,Consolas,monospace}
  .grille{display:grid;grid-template-columns:1fr 1fr;gap:1px;background:var(--trait);
          border:1px solid var(--trait);border-radius:8px;overflow:hidden;margin:0 0 12px}
  .case{background:var(--carte);padding:14px 16px}
  .case b{display:block;font-size:14px;margin-bottom:2px}
  .case span{color:var(--doux);font-size:14px}
  .note{background:var(--carte);border-left:3px solid var(--rouge);
        padding:12px 16px;border-radius:0 6px 6px 0;margin:0 0 12px}
  .note b{color:var(--rouge)}
</style>
</head>
<body>
<div class="page">

<h1>osmo-operator — démarrage rapide</h1>
<p class="sous">Réseau GSM/EGPRS complet sur une machine : BTS, cœur de réseau, Asterisk et softphones.</p>

<h2>1. Lancer</h2>
<p>Double-cliquez le <b>téléphone rouge</b> sur le bureau. Quatre fenêtres s'ouvrent, une par quart d'écran :</p>
<div class="grille">
  <div class="case"><b>Terminal</b><span>le banc démarre (<code>start-native.sh</code>)</span></div>
  <div class="case"><b>Firefox</b><span>console web sur <code>127.0.0.1:8080</code></span></div>
  <div class="case"><b>Wireshark</b><span>GSMTAP, <code>udp/4729</code></span></div>
  <div class="case"><b>Linphone</b><span>postes 100 et 200, déjà enregistrés</span></div>
</div>
<p>Comptez une à deux minutes avant que la radio soit stable.</p>

<h2>2. Passer un appel</h2>
<p>Dans Linphone, composez un numéro et décrochez :</p>
```

```
<table>
  <tr><th>Numéro</th><th>Destination</th></tr>
  <tr><td>100</td><td>Linphone A</td></tr>
  <tr><td>200</td><td>Linphone B</td></tr>
  <tr><td>600</td><td>test d'écho — votre voix vous revient</td></tr>
  <tr><td>500 / 700</td><td>annonces enregistrées</td></tr>
  <tr><td>1001<i>XX</i></td><td>un mobile GSM du réseau local</td></tr>
</table>
<p><b>Depuis un mobile GSM</b>, tout passe par le <b>VTY de la console web</b>, pas par un clavier : dans la barre
 ↪  en haut, choisissez l'opérateur (<code>OP1</code>), puis le mobile (<code>BB MS1&amp;nbsp;☎&amp;nbsp;…</code>), et
 ↪  cliquez <b>Go</b>. Vous obtenez son terminal, d'où l'appel se compose :</p>
<table>
  <tr><th>Commande VTY</th><th>Effet</th></tr>
  <tr><td>call 1 100</td><td>le mobile appelle Linphone A</td></tr>
  <tr><td>call 1 hangup</td><td>raccrocher</td></tr>
  <tr><td>call 1 answer</td><td>décrocher un appel entrant</td></tr>
</table>
<p>Le <code>1</code> est le nom du mobile, celui de <code>mobile.cfg</code>. En cas de doute, <code>list</code>
 ↪  dans le VTY affiche les commandes réellement disponibles.</p>
<p><b>Commencez par le 600.</b> S'il vous répond, la chaîne SIP et l'audio sont bonnes — inutile de chercher plus
 ↪  loin côté téléphone.</p>

<h2>3. Vérifier</h2>
<ul>
  <li>Les postes sont enregistrés : <code>asterisk -rx "pjsip show endpoints"</code> → <code>Avail</code></li>
  <li>Le mobile est attaché : la console web, page <i>Abonnés</i></li>
  <li>La radio parle : Wireshark défile en continu sur <code>udp/4729</code></li>
</ul>

<h2>4. Si ça coince</h2>
<div class="note">
  <p><b>Linphone ne s'enregistre pas.</b> Il ne relit jamais sa configuration à chaud : il faut le
   ↪  <b>redémarrer</b>. Un compte modifié pendant qu'il tourne n'est pas pris en compte, et il réécrit le fichier
   ↪  en quittant.</p>
</div>
<ul>
  <li><b>Appel muet.</b> Le micro passe par la carte son, pas par le navigateur : <code>pactl list short modules |
   ↪  grep loopback</code> doit montrer deux lignes.</li>
  <li><b>Écho.</b> Les enceintes rentrent dans le micro — mettez un casque.</li>
  <li><b>Pas de console web.</b> <code>systemctl status osmo-egprs-web</code>, puis <code>journalctl -u
   ↪  osmo-egprs-web</code>.</li>
</ul>

<h2>5. Arrêter</h2>
<p><code>sudo /opt/osmo-launch.sh stop</code>, ou <code>Ctrl+C</code> dans le terminal du banc.</p>

</div>
</body>
</html>
```

2.42 /opt/GSM/osmo-operator/fft-web/fft_web.py

11311 octets, 208 lignes → 208 lignes

```
#!/usr/bin/env python3
# fft_web.py - two live FFTs (MS + BTS) on ONE page, standalone web server (port 8081).
#
# Same processing as tools/fft.sh / tools/fft2.sh (Welch PSD on fc32 I/Q) but:
#   - NATIVE: reads the host sources directly, no docker exec.
#   - HEADLESS: no matplotlib/X. numpy computes the PSD -> JSON -> the browser
#     draws it (canvas: PSD trace + waterfall).
#   - LIVE FIFO: a FIFO source is read CONTINUOUSLY. It is opened
#     O_RDWR|O_NONBLOCK (like grgsm_fft_live.py): no EOF, the writer's
#     non-blocking write always finds a reader, and the producer (relay,
```

```
#     camping loop) is never disturbed.
#
# Sources:
#   - MS  = /tmp/iq_fft_ms.fifo (uplink, written by pont.py; LIVE)
#   - BTS = /tmp/iq_fft.fifo    (DL tap of the osmo-trx relay; LIVE)
#   Each source accepts either a .cfile (tail read) or a .fifo (live read),
#   auto-detected. Override with CFILE_MS / CFILE_BTS.
#
# Usage:  python3 /opt/GSM/osmo-operator/fft-web/fft_web.py
#   Environment: FFT_WEB_PORT=8081 RATE=1083333 NSAMP=262144 NFFT=4096
#                REFRESH_MS=40 NSEG_MAX=16 CFILE_MS CFILE_BTS ARFCN_MS ARFCN_BTS
#   GET /          the page;  GET /psd?src=ms|bts   one PSD as JSON.
import os, json, stat, threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs
import numpy as np

# --- installation root, resolved without a hard-coded path -------------------
# GSM_ROOT when the environment sets it (the case through start.sh), otherwise
# the parent of this repository: the repositories live side by side.
import os as _os
GSM_ROOT = _os.environ.get(
    "GSM_ROOT",
    _os.path.dirname(_os.path.dirname(_os.path.abspath(__file__))))


PORT  = int(os.environ.get('FFT_WEB_PORT', '8081'))
RATE  = float(os.environ.get('RATE', '1083333'))   # native 4 SPS = 26e6/24 (= Fs of the relay/FIFO)
NSAMP = int(os.environ.get('NSAMP', '262144'))     # complex samples kept (tail / live window)
NFFT  = int(os.environ.get('NFFT', '4096'))        # FFT resolution (Welch segment)
# Display rate: 40 ms = 25 fps. The browser polls /psd at this interval
# (2 fetches ms+bts per tick -> ~50 req/s). The number of Welch segments per
# request is bounded (NSEG_MAX) so the computation keeps up (NFFT*NSEG_MAX FFT/req).
REFRESH_MS = int(os.environ.get('REFRESH_MS', '40'))
NSEG_MAX   = int(os.environ.get('NSEG_MAX', '16'))

SRC = {
    # Uplink comes from pont.py (PONT_AIRREC_UL_FIFO): in bridge mode the DSP
    # cfile (/dev/shm/dsp_iq.cfile) has no producer.
    'ms':  {'path': os.environ.get('CFILE_MS',  '/tmp/iq_fft_ms.fifo'),
            'arfcn': os.environ.get('ARFCN_MS', '514'),  'label': 'MS — montant (iq_fft_ms.fifo)'},
    'bts': {'path': os.environ.get('CFILE_BTS', '/tmp/iq_fft.fifo'),
            'arfcn': os.environ.get('ARFCN_BTS', '514'), 'label': 'BTS — DL relay LIVE (iq_fft.fifo)'},
}

# ── Live state per FIFO source: fd O_RDWR|O_NONBLOCK + rolling buffer ────────
_state = {}                                        # src -> {'fd':int, 'buf':bytearray}
_locks = {k: threading.Lock() for k in SRC}
MAXB   = NSAMP * 8                                 # bytes kept (complex64 = 8 bytes/sample)

# ── I/Q acquisition ──────────────────────────────────────────────────────────
def _is_fifo(path):
    try:
        return stat.S_ISFIFO(os.stat(path).st_mode)
    except OSError:
        return path.endswith('.fifo')              # not created yet: trust the suffix

def read_live_iq(src, path):
    """Drain the FIFO without blocking and return the last NSAMP samples
    (the freshest). O_RDWR => never an EOF, never blocks the writer."""
    st = _state.get(src)
    if st is None:
        if not os.path.exists(path):
            os.mkfifo(path, 0o666)
        fd = os.open(path, os.O_RDWR | os.O_NONBLOCK)
        st = {'fd': fd, 'buf': bytearray()}
```

```
        _state[src] = st
    fd, buf = st['fd'], st['buf']
    while True:
        try:
            c = os.read(fd, 1 << 16)
        except BlockingIOError:
            break
        if not c:
            break
        buf += c
        if len(buf) > MAXB:
            del buf[:-MAXB]                          # bound the RAM, keep the freshest
    if len(buf) > MAXB:
        del buf[:-MAXB]
    n = (len(buf) // 8) * 8
    if n < NFFT * 8:
        return np.array([], dtype=np.complex64)
    return np.frombuffer(bytes(buf[-n:]), dtype=np.complex64)

def read_tail_iq(path, nsamp):
    sz = os.path.getsize(path)
    nbytes = min(sz - (sz % 8), nsamp * 8)         # complex64 = 8 bytes/sample
    with open(path, 'rb') as f:
        if sz > nbytes:
            f.seek(sz - nbytes)
        raw = f.read(nbytes)
    return np.frombuffer(raw, dtype=np.complex64)

# ── PSD ──────────────────────────────────────────────────────────────
def welch_psd(iq, nfft, rate):
    if iq.size < nfft:
        return None, None
    win = np.hanning(nfft).astype(np.float32)
    nseg = min(iq.size // nfft, NSEG_MAX)          # bound the cost to hold 25 fps
    iq = iq[-nseg*nfft:]                            # keep the freshest segments
    acc = np.zeros(nfft, dtype=np.float64)
    for i in range(nseg):
        seg = iq[i*nfft:(i+1)*nfft] * win
        acc += np.abs(np.fft.fftshift(np.fft.fft(seg)))**2
    acc /= nseg
    psd_db = 10.0*np.log10(acc + 1e-12)
    freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1.0/rate)) / 1e3   # kHz
    return freqs, psd_db

def psd_json(src):
    s = SRC[src]
    path = s['path']
    try:
        with _locks[src]:
            iq = read_live_iq(src, path) if _is_fifo(path) else read_tail_iq(path, NSAMP)
        f, p = welch_psd(iq, NFFT, RATE)
        if f is None:
            return {'error': "flux pas encore prêt (pas assez d'échantillons)", 'label': s['label']}
        step = max(1, len(f)//1024)                # decimate for transport
        return {'label': s['label'], 'arfcn': s['arfcn'], 'rate': RATE,
                'freqs': [round(x, 1) for x in f[::step].tolist()],
                'psd':   [round(x, 2) for x in p[::step].tolist()]}
    except FileNotFoundError:
        return {'error': 'source absente (%s) — lance la stack' % path, 'label': s['label']}
    except Exception as e:
        return {'error': str(e), 'label': s['label']}

# ── HTML page (canvas PSD + waterfall, polls /psd) ──────────────────
PAGE = """<!doctype html><html><head><meta charset=utf-8><title>Calypso FFT — MS & BTS</title>
<style>body{background:#0b0f14;color:#cdd6e0;font-family:monospace;margin:0;padding:12px}
h1{font-size:15px;color:#2aa198;margin:0 0 10px} .row{display:flex;gap:12px;flex-wrap:wrap}
```

```
.card{flex:1;min-width:380px;background:#0e151c;border:1px solid #1d2a33;border-radius:8px;padding:8px}
.t{font-size:12px;color:#b58900;margin-bottom:4px}
 ↪  canvas{width:100%;background:#060a0e;border-radius:4px;display:block}
.psd{height:200px} .wf{height:150px;margin-top:6px;image-rendering:pixelated}
.err{color:#dc322f;font-size:12px;min-height:14px}</style></head><body>
<h1>\U0001F4E1 Calypso I/Q FFT — MS vs BTS <span id=st style="color:#586e75;font-size:11px"></span></h1>
<div class=row>
 <div class=card><div class=t id=t-ms>MS</div><canvas id=c-ms class=psd></canvas><canvas id=w-ms
 ↪  class=wf></canvas><div class=err id=e-ms></div></div>
 <div class=card><div class=t id=t-bts>BTS</div><canvas id=c-bts class=psd></canvas><canvas id=w-bts
 ↪  class=wf></canvas><div class=err id=e-bts></div></div>
</div>
<script>
function cmap(v){v=Math.max(0,Math.min(1,v));var h=(1-v)*240,l=0.12+0.5*v,a=1*Math.min(l,1-l);
 var f=function(n){var k=(n+h/30)%12;return Math.round(255*(l-a*Math.max(-1,Math.min(k-3,9-k,1))));};
 return [f(0),f(8),f(4)];}
function drawPsd(src,d){var c=document.getElementById('c-'+src),x=c.getContext('2d');
 var W=c.width=c.clientWidth*2,H=c.height=c.clientHeight*2;x.clearRect(0,0,W,H);
 var p=d.psd,n=p.length;if(!n)return;var mn=Math.min.apply(null,p),mx=Math.max.apply(null,p),rg=(mx-mn)||1;
 x.strokeStyle='#16222b';x.lineWidth=1;for(var g=0;g<=4;g++){var
 ↪  yy=H*g/4;x.beginPath();x.moveTo(0,yy);x.lineTo(W,yy);x.stroke();}
 x.beginPath();x.moveTo(W/2,0);x.lineTo(W/2,H);x.strokeStyle='#243845';x.stroke();
 x.strokeStyle='#2aa198';x.lineWidth=2;x.beginPath();
 for(var i=0;i<n;i++){var xx=W*i/(n-1),yy=H-(p[i]-mn)/rg*H*0.9-H*0.05;i?x.lineTo(xx,yy):x.moveTo(xx,yy);}
 x.stroke();}
function drawWf(src,d){var c=document.getElementById('w-'+src),x=c.getContext('2d');
 var p=d.psd,n=p.length;if(!n)return;
 if(c.width!==n){c.width=n;c.height=160;x.fillStyle='#060a0e';x.fillRect(0,0,n,c.height);}
 // Waterfall scrolls like every SDR one (gqrx, GNU Radio, grgsm): newest line
 // at the TOP, history going down.
 x.drawImage(c,0,1);                        // scroll down by 1px
 var mn=Math.min.apply(null,p),mx=Math.max.apply(null,p),rg=(mx-mn)||1;
 var row=x.createImageData(n,1);
 for(var i=0;i<n;i++){var
 ↪  col=cmap((p[i]-mn)/rg),o=i*4;row.data[o]=col[0];row.data[o+1]=col[1];row.data[o+2]=col[2];row.data[o+3]=255;}
 x.putImageData(row,0,0);}                  // new line at the top
function draw(src,d){var e=document.getElementById('e-'+src),tt=document.getElementById('t-'+src);
 if(d.error){e.textContent=d.error;tt.textContent=d.label||'';return;} e.textContent='';
 tt.textContent=d.label+'  •  ARFCN '+d.arfcn+'  •  Fs '+(d.rate/1e6).toFixed(3)+' MHz';
 drawPsd(src,d);drawWf(src,d);}
function tick(){['ms','bts'].forEach(function(s){
  fetch('/psd?src='+s+'&t='+Date.now()).then(function(r){return
 ↪  r.json();}).then(function(d){draw(s,d);}).catch(function(){});});
 document.getElementById('st').textContent='— '+new Date().toLocaleTimeString();}
tick();setInterval(tick,__REFRESH_MS__);
</script></body></html>"""

# ── HTTP server ──────────────────────────────────────────────────────
class Handler(BaseHTTPRequestHandler):
    def log_message(self, *a):
        pass
    def do_GET(self):
        u = urlparse(self.path)
        if u.path == '/psd':
            q = parse_qs(u.query); src = q.get('src', ['ms'])[0]
            if src not in SRC:
                src = 'ms'
            body = json.dumps(psd_json(src)).encode()
            self.send_response(200)
            self.send_header('Content-Type', 'application/json')
            self.send_header('Cache-Control', 'no-store')
            self.send_header('Content-Length', str(len(body)))
            self.end_headers(); self.wfile.write(body); return
        body = PAGE.replace('__REFRESH_MS__', str(REFRESH_MS)).encode('utf-8')
        self.send_response(200)
        self.send_header('Content-Type', 'text/html; charset=utf-8')
```

```
        self.send_header('Content-Length', str(len(body)))
        self.end_headers(); self.wfile.write(body)

if __name__ == '__main__':
    print('FFT web on :%d  MS=%s  BTS=%s' % (PORT, SRC['ms']['path'], SRC['bts']['path']))
    ThreadingHTTPServer(('0.0.0.0', PORT), Handler).serve_forever()
```

2.43 /opt/GSM/osmo-operator/helpers/create_interop.sh

6865 octets, 192 lignes → 192 lignes

```
#!/bin/bash
# create_interop.sh - Generate the inter-STP configuration (central SS7 hub)
#
# Usage:
#   create_interop.sh [--listen-ip <ip>] <n_operators> [outfile]
#   create_interop.sh [--listen-ip <ip>] --wan <n_nodes> <ops_per_node> [outfile]
#
# Parameters (single-machine mode):
#   $1  n_operators   Number of operators (1..24)
#   $2  outfile       Output config file (default: osmo-stp-interop.cfg)
#
# Environment:
#   HUB_LISTEN_IP   Address the hub listens on (also --listen-ip). Empty or
#                   unset = 0.0.0.0 (single-machine setups). On a multi-homed
#                   host a single correct address is required: with 0.0.0.0
#                   SCTP announces every interface in its INIT and the peer ASP
#                   flaps on the unreachable ones.
#   OP_ID_BASE      Id of the first container operator (default 1). The AS
#                   list runs from 1 up to OP_ID_BASE + n_operators - 1 so the
#                   native operator (op1) is covered when containers start at 2.
#   WAN_NODE_IDS    WAN mode: node numbers, e.g. "1 3" (default 1..n_nodes).
#
# Point codes, ITU 3-8-3:
#   Inter-STP : 0.0.0
#   Op N STP  : 1.N.2    (cluster = N, 8 bits -> max 255, limited to 24)
#   Op N MSC  : 1.N.1
#   Op N BSC  : 1.N.3
#
# Static routes with exact mask 7.255.7 (14 bits):
#   1.N.1 -> as-opN  (MSC)
#   1.N.3 -> as-opN  (BSC)
#
# This file is REGENERATED at every hub start (launch/start-interstp.sh): any
# fix must go through its options/environment, not into the generated file.

set -e

# ── WAN mode ──────────────────────────────────────────────────────────
#   create_interop.sh --wan <n_nodes> <ops_per_node> <outfile>
#
# A point code is an ADDRESS: several nodes sharing one hub must not all be
# 1.1.2. The WAN plan encodes the node INSIDE the point code:
#   PC = 1.<node><op>.<role>     node 1 op 1 -> 1.11.2 ; node 3 op 2 -> 1.32.2
#   RCTX = node*1000 + op*100 + 50
# Readable (first digit is the node), unique, and the middle field of 3-8-3
# holds up to 9 nodes x 9 operators.

# ── Hub listen address ────────────────────────────────────────────────
HUB_LISTEN_IP="${HUB_LISTEN_IP:-0.0.0.0}"
_ci_args=()
while [ $# -gt 0 ]; do
    case "$1" in
        --listen-ip)   HUB_LISTEN_IP="${2:-0.0.0.0}"; shift 2 ;;
```

```
        --listen-ip=*) HUB_LISTEN_IP="${1#*=}"; shift ;;
        *)             _ci_args+=("$1"); shift ;;
    esac
done
[ "${#_ci_args[@]}" -gt 0 ] && set -- "${_ci_args[@]}"
[ -n "$HUB_LISTEN_IP" ] || HUB_LISTEN_IP="0.0.0.0"

WAN_MODE=0
if [ "${1:-}" = "--wan" ]; then
    WAN_MODE=1
    n_nodes="${2:-3}"
    ops_per_node="${3:-1}"
    outfile="${4:-osmo-stp-interop.cfg}"
    if ! [[ "$n_nodes" =~ ^[1-9]$ ]] || ! [[ "$ops_per_node" =~ ^[1-9]$ ]]; then
        echo "Erreur : --wan <1-9 noeuds> <1-9 operateurs>" >&2; exit 1
    fi
else
    n_operators="${1:-2}"
    outfile="${2:-osmo-stp-interop.cfg}"
    if ! [[ "$n_operators" =~ ^[0-9]+$ ]] || [ "$n_operators" -lt 1 ] || [ "$n_operators" -gt 24 ]; then
        echo "Erreur : n_operators doit etre 1..24" >&2
        exit 1
    fi
fi

# ── Header: logging, VTY, cs7 instance, M3UA listener ──────────────────
cat > "$outfile" <<'EOFCONFIG'
!
! osmo-stp-interop.cfg - Configuration inter-STP centrale
!
! PC 0.0.0 : hub de routage SS7
!

log stderr
 logging filter all 1
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging print extended-timestamp 1
 logging print level 1
 logging print file basename
 logging level lss7 info
 logging level lsccp info
 logging level lm3ua info
 logging level linp notice
!
stats interval 5
!
line vty
 no login
!
cs7 instance 0
 network-indicator international
 point-code 0.0.0
!
 xua rkm routing-key-allocation dynamic-permitted

 listen m3ua 2908
  accept-asp-connections dynamic-permitted
  local-ip __LISTEN_IP__
!
EOFCONFIG

# Quoted heredoc: the template is written verbatim, then substituted. The
# marker only exists in the header; the AS loops below do not use it.
sed -i "s|__LISTEN_IP__|${HUB_LISTEN_IP}|" "$outfile"
```

```
# ── Application servers (one per operator STP) ──────────────────────────
if [ "$WAN_MODE" = "1" ]; then
    for n in ${WAN_NODE_IDS:-$(seq 1 "$n_nodes")}; do
        for o in $(seq 1 "$ops_per_node"); do
            rctx_inter=$(( n * 1000 + o * 100 + 50 ))
            pc_stp="1.${n}${o}.2"
            cat >> "$outfile" <<EOF

 as as-n${n}op${o} m3ua
  routing-key ${rctx_inter} ${pc_stp}
  traffic-mode override
EOF
        done
    done
else
# From 1 up to the highest operator id, not "1..n_operators": n_operators
# counts the CONTAINERS, the native operator is op1 and containers may start
# at OP_ID_BASE=2.
for i in $(seq 1 "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
    rctx_inter=$(( i * 100 + 50 ))
    pc_stp="1.${i}.2"

    cat >> "$outfile" <<EOF

 as as-op${i} m3ua
  routing-key ${rctx_inter} ${pc_stp}
  traffic-mode override
EOF
done
fi

# ── Route table (MSC and BSC of every operator) ─────────────────────────
cat >> "$outfile" <<'EOFROUTES'

 route-table system
EOFROUTES

if [ "$WAN_MODE" = "1" ]; then
    for n in ${WAN_NODE_IDS:-$(seq 1 "$n_nodes")}; do
        for o in $(seq 1 "$ops_per_node"); do
            cat >> "$outfile" <<EOF
  update route 1.${n}${o}.1 7.255.7 linkset as-n${n}op${o}
  update route 1.${n}${o}.3 7.255.7 linkset as-n${n}op${o}
EOF
        done
    done
else
# Same range as the AS list above (1 .. OP_ID_BASE + n_operators - 1).
for i in $(seq 1 "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
    cat >> "$outfile" <<EOF
  update route 1.${i}.1 7.255.7 linkset as-op${i}
  update route 1.${i}.3 7.255.7 linkset as-op${i}
EOF
done
fi

cat >> "$outfile" <<'EOF'

!
EOF

# ── Summary ─────────────────────────────────────────────────────────────
if [ "$WAN_MODE" = "1" ]; then
    echo "✓ Config inter-STP WAN generee : $outfile" >&2
    echo "  PC hub  : 0.0.0   Noeuds : $n_nodes × $ops_per_node operateur(s)" >&2
```

```
    echo "  PC noeuds : 1.<noeud><op>.<role>   RCTX : noeud*1000 + op*100 + 50" >&2
    echo "  Routes  : $(( n_nodes * ops_per_node * 2 )) (masque 7.255.7)" >&2
else
    echo "✓ Config inter-STP generee : $outfile" >&2
    echo "  PC hub     : 0.0.0   Operateurs : $n_operators" >&2
    echo "  Routes     : $((n_operators * 2)) (masque 7.255.7)" >&2
fi
```

2.44 /opt/GSM/osmo-operator/helpers/prepare_host.sh

3666 octets, 107 lignes → 107 lignes

```
#!/usr/bin/env bash
# prepare_host.sh - reset the host-side desktop tools of the GSM bench.
# Usage: sudo ./helpers/prepare_host.sh
# Steps:
#  - kill and relaunch Linphone as the session user
#  - kill xterm and Wireshark (root)
#  - check the UDP/GSMTAP listeners
# Env: GSMTAP_PORT (default 4729), HOST_PULSE_USER (session user override).
# Display: "[....] step" is rewritten in place as "[ OK ] step".

set -euo pipefail

GSMTAP_PORT="${GSMTAP_PORT:-4729}"

# ---------- helpers ----------
step()    { printf "[....] %s" "$1"; }
step_ok() { printf "\r[ OK ] %s\n" "$1"; }
fail()    { printf "\r[FAIL] %s\n" "$1" >&2; exit 1; }

need() { command -v "$1" >/dev/null 2>&1 || fail "Commande manquante: $1"; }

kill_name_as_user() {
  local user="$1" name="$2"
  pkill -u "$user" -x "$name" 2>/dev/null || true
}

kill_name_root() {
  local name="$1"
  pkill -x "$name" 2>/dev/null || true
}

# ---------- preflight ----------
[[ "${EUID}" -eq 0 ]] || fail "Lance en root: sudo $0"

need sudo
need docker
need ss
need systemctl

# ---------- session user ----------
# The session user is identified by the PRESENCE of its pulse socket, not by
# the account name: SUDO_USER is empty after `sudo -i`, and on the --desktop
# ISO the graphical session is root itself. Same logic as session_pulse_user()
# in launch/start-docker.sh.
resolve_session_user() {
  local u uid sock
  for u in "${HOST_PULSE_USER:-}" "${SUDO_USER:-}" "$(logname 2>/dev/null || true)"; do
    [[ -n "$u" ]] || continue
    uid="$(id -u "$u" 2>/dev/null)" || continue
    [[ -S "/run/user/${uid}/pulse/native" ]] && { echo "$u"; return 0; }
  done
  # Owner of the first pulse socket found: covers `sudo -i` and sessions
```

```
  # without logname (cron, service, detached terminal).
  for sock in /run/user/*/pulse/native; do
    [[ -S "$sock" ]] || continue
    uid="${sock#/run/user/}"; uid="${uid%%/*}"
    u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
    [[ -n "$u" ]] && { echo "$u"; return 0; }
  done
  # No live pulse session: fall back to the invoker, root included.
  echo "${SUDO_USER:-$(id -un)}"
}
TARGET_USER="$(resolve_session_user)"

id -u "${TARGET_USER}" >/dev/null 2>&1 || fail "User invalide: ${TARGET_USER}"

# ---------- Linphone: kill + relaunch as the session user ----------
step "Linphone: kill (user=${TARGET_USER})"
kill_name_as_user "${TARGET_USER}" "linphone"
sleep 0.2
step_ok "Linphone: kill (user=${TARGET_USER})"

step "Linphone: relance (user=${TARGET_USER})"
    # TARGET_USER is resolved once, at the top of the script; do not
    # re-resolve it here.
    TARGET_UID=$(id -u "$TARGET_USER")
    # HOME via getent, not a hard-coded /home/$TARGET_USER: root lives in /root.
    TARGET_HOME="$(getent passwd "$TARGET_USER" 2>/dev/null | cut -d: -f6)"
    DISPLAY="${DISPLAY:-:0}"
    XAUTHORITY="${XAUTHORITY:-${TARGET_HOME:-/home/$TARGET_USER}/.Xauthority}"

    setsid sudo -u "$TARGET_USER" \
        env DISPLAY="$DISPLAY" XAUTHORITY="$XAUTHORITY" \
            DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/${TARGET_UID}/bus" \
        linphone </dev/null >/dev/null 2>&1 &

step_ok "Linphone: relance (user=${TARGET_USER})"

# ---------- Xterm: kill ----------
step "Xterm kill"
kill_name_root "xterm"
sleep 0.2
step_ok "Xterm: kill (root)"

# ---------- Wireshark: kill (root) ----------
step "Wireshark: kill (root)"
kill_name_root "wireshark"
sleep 0.2
step_ok "Wireshark: kill (root)"

# ---------- Listener check (optional) ----------
step "Check: listeners UDP/${GSMTAP_PORT}"
ss -lunp | awk -v p=":${GSMTAP_PORT}" '$0 ~ p {print}' >/dev/null 2>&1 || true
step_ok "Check: listeners UDP/${GSMTAP_PORT}"

step "Stack forcee"
step_ok "Stack forcee"
```

2.45 /opt/GSM/osmo-operator/install_modules/00-prereqs.sh

871 octets, 17 lignes → 17 lignes

```
# 00-prereqs - can this machine host the installation? (apt-get, git, free disk space)
INST_REGISTER prereqs "Prerequis du systeme"
INST_ROOT[prereqs]=0
```

```
inst_prereqs_run() {
    have_cmd apt-get || { inst_hint "cette installation vise Debian/Ubuntu ; sur une autre distribution, utilisez
     ↪  Docker : ./tools/make-docker-image.sh"
                          inst_fail "apt-get introuvable"; return $INST_RC_FAIL; }
    have_cmd git || { inst_fail "git absent"; return $INST_RC_FAIL; }
    local free; free=$(df -Pm "${GSM_ROOT%/*}" 2>/dev/null | awk 'NR==2{print $4}')
    if [ -n "$free" ] && [ "$free" -lt 4096 ]; then
        inst_hint "les sources et les objets de compilation occupent ~4 Go"
        inst_fail "espace disque insuffisant : ${free} Mo libres sur $(dirname "$GSM_ROOT")"
        return $INST_RC_FAIL
    fi
    inst_ok
}
inst_prereqs_verify() { inst_ok; }
```

2.46 /opt/GSM/osmo-operator/install_modules/10-deps.sh

3816 octets, 82 lignes → 82 lignes

```
# 10-deps - system packages (apt).
# The package list is authoritative in the Dockerfile: it is EXTRACTED from it
# rather than copied, so the two can never diverge.
INST_REGISTER deps "Dependances systeme (apt)"
INST_DEPS[deps]="prereqs"

# Extract the package names from the Dockerfile's apt-get/apt-fast install
# block (either tool, any update/install order, optional --mount=... on the
# RUN line). Only tokens that LOOK like package names are kept, and the list
# is cut at the first shell operator (&&, ||, {) or URL: the bootstrap block
# contains commands (chmod, printf, rm) that would otherwise pass for
# packages and turn a format error into an unreadable apt error.
_deps_list() {
    awk '
        /^RUN( --mount=[^ ]+)? *\\?$/ { pending = 1; next }
        /^RUN( --mount=[^ ]+)? +apt-(get|fast) update && apt-(get|fast) install/ || (pending && /^ *apt-(get|fast)
 ↪  update && apt-(get|fast) install/) {
            pending = 0
            inblk = 1
            line = $0
            # The RUN may carry a --mount=... (BuildKit apt cache) and continue
            # on the next line: strip everything before the end of the apt
            # command, whether the line starts with RUN or not.
            sub(/^.*--no-install-recommends/, "", line)
            emit(line)
            next
        }
        pending && !/^ *apt-(get|fast) update/ { pending = 0 }
        inblk { emit($0) }

        function emit(l,   n, a, i, cont) {
            # Strip the comment FIRST: "# ..." lines inside the list carry no
            # continuation backslash (Docker removes them before the shell),
            # so they must not be taken as the end of the block.
            # (No apostrophe in this block: it lives between single quotes.)
            sub(/#.*/, "", l)
            if (l ~ /^[ \t]*$/) return          # pure comment: transparent

            cont = (l ~ /\\[ \t]*$/)
            if (l ~ /&&|\|\||[{}]|https?:/) {   # end of the package list
                sub(/(&&|\|\||\{).*/, "", l)
                inblk = 0
            } else if (!cont) {
                inblk = 0                        # last line of the block
            }
```

```
            sub(/\\[ \t]*$/, "", l)
            n = split(l, a, /[ \t]+/)
            for (i = 1; i <= n; i++)
                if (a[i] ~ /^[a-z0-9][a-z0-9.+*-]*$/) print a[i]
        }
    ' "$INST_TREE/Dockerfile" 2>/dev/null | sort -u | tr '\n' ' '
}

inst_deps_check() {
    have_file "$INST_TREE/Dockerfile" || { inst_fail "Dockerfile introuvable - c'est lui qui porte la liste des
     ↪  paquets"; return $INST_RC_FAIL; }
    inst_ok
}
# "Already installed" is judged on the WHOLE list, not on a few witness
# packages: a partially equipped system (binaries without the -dev packages)
# must not skip this step. A pattern (asterisk-moh*) counts as present if one
# of the packages it matches is installed.
_deps_missing() {
    local p
    for p in $(_deps_list); do
        dpkg-query -W -f='${db:Status-Status}\n' "$p" 2>/dev/null | grep -qx installed || printf '%s\n' "$p"
    done
}
inst_deps_done() {
    [ -n "$(_deps_list)" ] && [ -z "$(_deps_missing)" ]
}
inst_deps_run() {
    local pkgs; pkgs="$(_deps_list)"
    [ -z "${pkgs// }" ] && { inst_fail "liste de paquets vide - le format du Dockerfile a change"; return
     ↪  $INST_RC_FAIL; }
    inst_say "paquets : $(echo $pkgs | wc -w), dont manquants : $(_deps_missing | wc -l)"
    # The FULL list, not only the missing packages: apt-get install is
    # idempotent, and a pattern like asterisk-moh* is only resolved by apt.
    apt-get update -qq && apt-get install -y --no-install-recommends $pkgs
}
inst_deps_verify() {
    local missing; missing="$(_deps_missing | tr '\n' ' ')"
    [ -n "${missing// }" ] && { inst_fail "paquets absents apres installation : $missing"; return $INST_RC_FAIL; }
    inst_ok
}
```

2.47 /opt/GSM/osmo-operator/install_modules/20-user.sh

594 octets, 11 lignes → 11 lignes

```
# 20-user - service account, as the Dockerfile creates it.
INST_REGISTER user "Utilisateur et groupe osmocom"
INST_DEPS[user]="prereqs"

inst_user_done() { getent passwd osmocom >/dev/null && getent group osmocom >/dev/null; }
inst_user_run() {
    getent group  osmocom >/dev/null || groupadd osmocom || return $INST_RC_FAIL
    getent passwd osmocom >/dev/null || useradd -r -g osmocom -s /sbin/nologin -d /var/lib/osmocom osmocom ||
     ↪  return $INST_RC_FAIL
    inst_ok
}
inst_user_verify() { inst_user_done && inst_ok || { inst_fail "le compte osmocom n'existe pas"; return
 ↪  $INST_RC_FAIL; }; }
```

2.48 /opt/GSM/osmo-operator/install_modules/30-sources.sh

2025 octets, 49 lignes → 49 lignes

```
# 30-sources - fetch the repositories. Idempotent: an already cloned repository is left as is
# (no pull unless REINSTALL=1: pulling would modify the user's work in progress without asking).
INST_REGISTER sources "Recuperation des sources Osmocom"
INST_DEPS[sources]="prereqs"

_SRC_REPOS="osmocom-bb|https://gitea.osmocom.org/phone-side/osmocom-bb|
osmo-gapk|https://gitea.osmocom.org/osmocom/gapk|
gsup-smsc-proto|https://gitea.osmocom.org/themwi/gsup-smsc-proto|
libosmo-dsp|https://gitea.osmocom.org/sdr/libosmo-dsp.git|"

inst_sources_done() {
    local l; while IFS='|' read -r dir url br; do
        [ -z "$dir" ] && continue
        have_repo "$GSM_ROOT/$dir" || return 1
    done <<< "$_SRC_REPOS"
    return 0
}
inst_sources_run() {
    mkdir -p "$GSM_ROOT" || { inst_fail "impossible de creer $GSM_ROOT"; return $INST_RC_FAIL; }
    local dir url br n=0
    while IFS='|' read -r dir url br; do
        [ -z "$dir" ] && continue
        if have_repo "$GSM_ROOT/$dir"; then
            if [ "${REINSTALL:-0}" = 1 ]; then
                inst_say "$dir : deja present, mise a jour (git pull --ff-only)"
                git -C "$GSM_ROOT/$dir" pull --ff-only \
                    || inst_say "$dir : mise a jour impossible (modifications locales ?), laisse tel quel"
                n=$((n+1))
            else
                inst_say "$dir : deja present, laisse tel quel"
            fi
            continue
        fi
        inst_say "clonage $dir"
        git clone ${br:+--branch "$br"} "$url" "$GSM_ROOT/$dir" || { inst_fail "echec du clonage de $dir ($url)";
        ↪  return $INST_RC_FAIL; }
        n=$((n+1))
    done <<< "$_SRC_REPOS"
    [ $n -eq 0 ] && return $INST_RC_DONE
    inst_ok
}
inst_sources_verify() {
    local missing="" dir url br
    while IFS='|' read -r dir url br; do
        [ -z "$dir" ] && continue
        have_repo "$GSM_ROOT/$dir" || missing="$missing $dir"
    done <<< "$_SRC_REPOS"
    [ -n "$missing" ] && { inst_fail "depots absents :$missing"; return $INST_RC_FAIL; }
    inst_ok
}
```

2.49 /opt/GSM/osmo-operator/install_modules/40-patches.sh

652 octets, 15 lignes → 15 lignes

```
# 40-patches - local fixes (patches/*.patch) applied to the trees under $GSM_ROOT
# by tools/osmo-patches-apply.sh (idempotent: an already applied patch is not an error).
INST_REGISTER patches "Application des correctifs"
INST_DEPS[patches]="sources"
INST_REQUIRED[patches]=0

inst_patches_run() {
    local out l n
    out="$(GSM_ROOT="$GSM_ROOT" REPO="$INST_TREE" bash "$INST_TREE/tools/osmo-patches-apply.sh" "$GSM_ROOT" 2>&1)"
```

```
    while IFS= read -r l; do [ -n "$l" ] && inst_say "${l#  }"; done <<< "$out"
    n="$(printf '%s\n' "$out" | grep -c ' -> ')"
    [ "$n" -eq 0 ] && return $INST_RC_DONE
    inst_ok
}
inst_patches_verify() { inst_ok; }
```

2.50 /opt/GSM/osmo-operator/install_modules/50-build.sh

2974 octets, 62 lignes → 62 lignes

```
# 50-build - compilation. The long step: hence the large timeout and the detailed log.
INST_REGISTER build "Compilation des sources"
INST_DEPS[build]="deps sources"
INST_TIMEOUT[build]=5400

_BUILD_ORDER="libosmo-dsp osmocom-bb osmo-gapk"

inst_build_done() { have_file "$GSM_ROOT/osmocom-bb/src/host/layer23/src/mobile/mobile"; }
inst_build_run() {
    local d
    for d in $_BUILD_ORDER; do
        [ -d "$GSM_ROOT/$d" ] || { inst_say "$d absent, ignore"; continue; }
        inst_say "=== $d ==="
        ( cd "$GSM_ROOT/$d" || exit 1
          [ -x ./configure ] || [ -f ./configure.ac ] && { autoreconf -fi || true; }
          [ -x ./configure ] && ./configure
          make -j"$(nproc)" ) || { inst_hint "detail dans le journal de cette etape"
                                   inst_fail "echec de compilation : $d"; return $INST_RC_FAIL; }
    done

    # toast: GSM 06.10 codec (quut.com). libgsm1 ships the library, not the
    # binary. Sources under $GSM_ROOT, binary in /usr/local/bin. Optional: a
    # failure (network, quut.com unreachable) does not stop the installation.
    inst_say "=== toast (codec GSM 06.10) ==="
    if command -v toast >/dev/null 2>&1; then
        inst_say "toast deja present ($(command -v toast))"
    else
        local gver=gsm-1.0.24
        local gdir=gsm-1.0-pl24   # the tarball extracts UNDER this name
        local gurl="https://www.quut.com/gsm/${gver}.tar.gz"
        if ( cd "$GSM_ROOT" \
             && wget -qO "${gver}.tar.gz" "$gurl" \
             && tar xzf "${gver}.tar.gz" \
             && cd "$gdir" \
             && { make >/dev/null 2>&1 || true; } \
             && { [ -x bin/toast ] || make toast >/dev/null 2>&1 || true; } \
             && [ -x bin/toast ] ); then
            for b in toast untoast tcat; do
                [ -e "$GSM_ROOT/$gdir/bin/$b" ] \
                    && install -m755 "$GSM_ROOT/$gdir/bin/$b" "/usr/local/bin/$b"
            done
            inst_say "toast installe dans /usr/local/bin (sources : $GSM_ROOT/$gdir)"
        else
            inst_hint "toast non compile (reseau ou quut.com injoignable) - suite sans lui"
        fi
    fi
    # ── 4G: srsRAN (ZeroMQ) and Open5GS in /opt/LTE ──────────────────────
    # Same script as the Dockerfile and addition.sh (tools/osmo-lte-install.sh):
    # cached .deb packages if present, compilation otherwise. Non-fatal: 2G
    # does not depend on it.
    inst_say "=== la 4G (srsRAN ZeroMQ + Open5GS) ==="
    if [ -f "$INST_TREE/tools/osmo-lte-install.sh" ]; then
        OSMO_REPO="$INST_TREE" bash "$INST_TREE/tools/osmo-lte-install.sh" --all \
```

```
            || inst_hint "4G non installee (voir ci-dessus) - « osmo-lte-install --build » plus tard"
    fi
    inst_ok
}
inst_build_verify() {
    have_file "$GSM_ROOT/osmocom-bb/src/host/layer23/src/mobile/mobile" \
        || { inst_fail "le binaire mobile n'a pas ete produit"; return $INST_RC_FAIL; }
    inst_ok
}
```

2.51 /opt/GSM/osmo-operator/install_modules/60-binaires.sh

1171 octets, 30 lignes → 30 lignes

```
# 60-binaires - install the executables into /usr/local/bin, as the Dockerfile does.
INST_REGISTER binaires "Installation des binaires"
INST_DEPS[binaires]="build"

_BINS="trxcon:src/host/trxcon/src/trxcon
mobile:src/host/layer23/src/mobile/mobile
virtphy:src/host/virt_phy/src/virtphy
ccch_scan:src/host/layer23/src/misc/ccch_scan"

inst_binaires_done() { have_cmd mobile && have_cmd trxcon; }
inst_binaires_run() {
    local line name path n=0
    while IFS=: read -r name path; do
        [ -z "$name" ] && continue
        if [ -x "$GSM_ROOT/osmocom-bb/$path" ]; then
            install -m755 "$GSM_ROOT/osmocom-bb/$path" "/usr/local/bin/$name" && n=$((n+1))
        else
            inst_say "absent, ignore : $path"
        fi
    done <<< "$_BINS"
    [ $n -eq 0 ] && { inst_fail "aucun binaire trouve a installer - la compilation a-t-elle abouti ?"; return
     ↪  $INST_RC_FAIL; }
    inst_say "$n binaires installes"
    inst_ok
}
inst_binaires_verify() {
    local missing="" b
    for b in mobile trxcon; do have_cmd "$b" || missing="$missing $b"; done
    [ -n "$missing" ] && { inst_fail "binaires absents du PATH :$missing"; return $INST_RC_FAIL; }
    inst_ok
}
```

2.52 /opt/GSM/osmo-operator/install_modules/70-configs.sh

1583 octets, 29 lignes → 29 lignes

```
# 70-configs - Osmocom configuration. `cp -n`: an existing configuration is NEVER
# overwritten (it may have been adapted to the machine), except with REINSTALL=1,
# which backs up /etc/osmocom first.
INST_REGISTER configs "Configuration (/etc/osmocom)"
INST_DEPS[configs]="prereqs"

inst_configs_done() { have_file /etc/osmocom/osmo-bsc.cfg; }
inst_configs_run() {
    mkdir -p /etc/osmocom /var/log/osmocom /var/run/smsc "${OSMOCOM_HOME:-$HOME/.osmocom}/bb" \
        || { inst_fail "impossible de creer les repertoires de configuration"; return $INST_RC_FAIL; }
    if [ "${REINSTALL:-0}" = 1 ]; then
        # Overwrite, but never without a copy: /etc/osmocom.<date> keeps the
```

```
        # previous state, as --regen does in start-native.sh.
        local sav="/etc/osmocom.$(date +%Y%m%d-%H%M%S)"
        cp -a /etc/osmocom "$sav" 2>/dev/null && inst_say "sauvegarde : $sav"
        cp -f "$INST_TREE"/configs/*.cfg /etc/osmocom/ 2>/dev/null
        [ -f "$INST_TREE/configs/mobile.cfg" ] && cp -f "$INST_TREE/configs/mobile.cfg"
         ↪  "${OSMOCOM_HOME:-$HOME/.osmocom}/bb/mobile.cfg"
        inst_say "configurations reecrites (--reinstall)"
    else
        cp -n "$INST_TREE"/configs/*.cfg /etc/osmocom/ 2>/dev/null
        [ -f "$INST_TREE/configs/mobile.cfg" ] && cp -n "$INST_TREE/configs/mobile.cfg"
         ↪  "${OSMOCOM_HOME:-$HOME/.osmocom}/bb/mobile.cfg"
        inst_say "configurations deposees (les fichiers existants ont ete conserves)"
    fi
    inst_ok
}
inst_configs_verify() {
    have_dir /etc/osmocom || { inst_fail "/etc/osmocom absent"; return $INST_RC_FAIL; }
    inst_ok
}
```

2.53 /opt/GSM/osmo-operator/install_modules/80-bureau.sh

10524 octets, 188 lignes → 188 lignes

```
# ============================================================================
#  80-bureau - desktop icons and shortcuts (native install)
# ============================================================================
#  Installs the same files the ISO installs (build-iso.sh), from the same
#  repository files:
#     data/desktop/osmo-launch.desktop   "Lancer le banc GSM"  -> launch.sh
#     data/desktop/osmo-dino.desktop     the Dino game
#     data/*.svg                         the icons
#  plus: /usr/local/bin/osmo-start-native and osmo-start-direct -> start-native.sh,
#  c123 / osmo-c123 -> tools/osmo-c123-gtk.py, osmo-wg -> tools/osmo-wg.sh,
#  the bench commands (tools/osmo-banc-cmd.sh), the profile.d snippets (gnuarm
#  PATH, bench aliases, WireGuard), the gcc-9/gcc-11 alternatives and the
#  GNOME default background (gschema override).
#
#  osmo-multi (antenna, multi-operator) is NOT installed here: its launcher
#  launch/start-multi.sh needs the SS7 supplement (docker + image + topology),
#  which only exists after addition.sh; addition.sh installs that icon itself
#  at the end of a successful SS7 install.
#
#  Icon= is rewritten as an ABSOLUTE PATH (/usr/share/osmo-operator/icons/*.svg):
#  a theme name goes through icon-theme.cache, which does not know an icon
#  added after it, and GNOME then silently shows a blank page. The cache is
#  still rebuilt for the applications menu.
#
#  Shortcuts are copied to the desktop of root and of the user who ran sudo
#  (SUDO_USER), in Bureau/ and Desktop/. The metadata::trusted attribute
#  cannot be set here (it lives in the session's gvfs metadata): update.sh
#  sets it at login; otherwise double-click + "Allow" is enough.
#
#  The background is a schema DEFAULT (99-osmo-fond.gschema.override), not a
#  user setting: a user who chose another background keeps it.
#
#  Neither osmo-tutorial, osmo-addition nor osmo-update: their launchers are
#  wrappers written by build-iso.sh for the image (terminal + pkexec) and
#  depend on the ISO's GNOME desktop. On an installed machine, launch.sh and
#  launch/start-multi.sh are enough.
# ----------------------------------------------------------------------------
INST_REGISTER bureau "Bureau : icones et raccourcis"
INST_DEPS[bureau]="configs"
INST_REQUIRED[bureau]=0
```

```
_BUREAU_ICONS=/usr/share/osmo-operator/icons
_BUREAU_ENTRIES="osmo-launch osmo-dino"
# The idle background is the city WITHOUT antenna (gsm-lab-ville.jpg);
# gsm-lab-antenne.jpg (same city, mast raised) is faded in by tools/osmo-fond.py
# while a 2G/4G network runs. Both are copied, plus the low-poly landscape
# (gsm-lab-fond.jpg), which is not the default.
_BUREAU_FOND=/usr/share/backgrounds/gsm-lab-ville.jpg
_BUREAU_FONDS="gsm-lab-antenne.jpg gsm-lab-fond.jpg"
_BUREAU_SCHEMA=/usr/share/glib-2.0/schemas/99-osmo-fond.gschema.override

# Default background: same file and same three sections as the ISO
# (org.gnome.desktop.background, :ubuntu, :GNOME-Greeter - an ubuntu:GNOME
# session reads the latter). Without glib-compile-schemas (no GNOME here),
# the file is installed and nothing more.
_bureau_fond() {
    local src="$INST_TREE/configs/gsm-lab-ville.jpg" sec f
    for f in $_BUREAU_FONDS; do
        [ -f "$INST_TREE/configs/$f" ] && install -Dm644 "$INST_TREE/configs/$f" "/usr/share/backgrounds/$f"
    done
    ln -sf "$INST_TREE/tools/osmo-fond.py" /usr/local/bin/osmo-fond
    [ -f "$src" ] || { inst_hint "configs/gsm-lab-ville.jpg absent : fond GNOME inchange"; return 0; }
    install -Dm644 "$src" "$_BUREAU_FOND" || { inst_hint "fond non pose : $_BUREAU_FOND"; return 0; }
    [ -d /usr/share/glib-2.0/schemas ] || { inst_say "fond : $_BUREAU_FOND (pas de schemas GNOME ici)"; return 0; }
    : > "$_BUREAU_SCHEMA"
    for sec in "org.gnome.desktop.background" "org.gnome.desktop.background:ubuntu"
     ↪  "org.gnome.desktop.background:GNOME-Greeter"; do
        printf "\n[%s]\npicture-uri='file://%s'\npicture-uri-dark='file://%s'\npicture-options='zoom'\nprimary-
         ↪  color='#0d1b2a'\n"
         ↪  \
            "$sec" "$_BUREAU_FOND" "$_BUREAU_FOND" >> "$_BUREAU_SCHEMA"
    done
    # Light mode by default, as on the ISO (iso_modules/80-chroot.sh):
    # Yaru-blue, color-scheme default. A 99-osmo-live override left by the ISO
    # on a machine installed from it is switched back to light as well.
    for sec in "org.gnome.desktop.interface" "org.gnome.desktop.interface:ubuntu"
     ↪  "org.gnome.desktop.interface:GNOME-Greeter"; do
        printf "\n[%s]\ncolor-scheme='default'\ngtk-theme='Yaru-blue'\nicon-theme='Yaru-blue'\ncursor-
         ↪  theme='Yaru'\n"
         ↪  \
            "$sec" >> "$_BUREAU_SCHEMA"
    done
    [ -f /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override ] \
        && sed -i -e "s/^color-scheme='prefer-dark'/color-scheme='default'/" -e
         ↪  "s/^gtk-theme='Yaru-blue-dark'/gtk-theme='Yaru-blue'/" \
            /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
    if command -v glib-compile-schemas >/dev/null 2>&1; then
        glib-compile-schemas /usr/share/glib-2.0/schemas 2>/dev/null \
            && inst_say "fond GNOME par defaut : $_BUREAU_FOND" \
            || inst_hint "glib-compile-schemas a echoue : fond pose, defaut non compile"
    else
        inst_say "fond : $_BUREAU_FOND (glib-compile-schemas absent, defaut non compile)"
    fi
}

_bureau_homes() {
    local h
    printf '%s\n' /root
    if [ -n "${SUDO_USER:-}" ] && [ "$SUDO_USER" != root ]; then
        h="$(getent passwd "$SUDO_USER" | cut -d: -f6)"
        [ -n "$h" ] && [ -d "$h" ] && printf '%s\n' "$h"
    fi
}

inst_bureau_check() {
    [ -d "$INST_TREE/data/desktop" ] || { inst_fail "data/desktop/ absent du depot"; return $INST_RC_FAIL; }
```

```
    [ -x "$INST_TREE/launch.sh" ] || inst_hint "launch.sh n est pas executable : chmod +x $INST_TREE/launch.sh"
    inst_ok
}

inst_bureau_done() {
    local e
    for e in $_BUREAU_ENTRIES; do
        have_file "/usr/share/applications/$e.desktop" || return 1
        have_file "$_BUREAU_ICONS/$e.svg" || return 1
    done
    [ -L /usr/local/bin/osmo-start-direct ] || return 1
    [ -L /usr/local/bin/c123 ] || return 1
    [ -L /usr/local/bin/banc-rssi ] || return 1
    return 0
}

inst_bureau_run() {
    local e h d f
    install -d "$_BUREAU_ICONS" /usr/share/icons/hicolor/scalable/apps /usr/share/applications \
        || { inst_fail "impossible de creer les repertoires du bureau"; return $INST_RC_FAIL; }
    for e in $_BUREAU_ENTRIES; do
        [ -f "$INST_TREE/data/$e.svg" ] && {
            install -m644 "$INST_TREE/data/$e.svg" "$_BUREAU_ICONS/$e.svg"
            install -m644 "$INST_TREE/data/$e.svg" "/usr/share/icons/hicolor/scalable/apps/$e.svg"
        }
        install -m644 "$INST_TREE/data/desktop/$e.desktop" "/usr/share/applications/$e.desktop" \
            || { inst_fail "copie impossible : $e.desktop"; return $INST_RC_FAIL; }
        # Exec= and Icon= follow the INSTALLED tree, not a hard-coded /opt/GSM:
        # this repository may live elsewhere (GSM_ROOT).
        sed -i -e "s|^Exec=/opt/GSM/osmo-operator/|Exec=$INST_TREE/|" \
               -e "s|^Icon=.*|Icon=$_BUREAU_ICONS/$e.svg|" "/usr/share/applications/$e.desktop"
        inst_say "menu : /usr/share/applications/$e.desktop"
    done
    # osmo-start-native is the launcher name; osmo-start-direct is kept as an
    # alias for existing habits and scripts.
    ln -sf "$INST_TREE/start-native.sh" /usr/local/bin/osmo-start-native
    ln -sf "$INST_TREE/start-native.sh" /usr/local/bin/osmo-start-direct
    # c123: the emulated C123 window (tools/osmo-c123-gtk.py) in the PATH, not
    # only as a login-shell alias. It runs under the SYSTEM python3: PyGObject
    # + GTK3 + cairo (python3-gi-cairo for Gdk.cairo_set_source_pixbuf) come
    # from apt, not from pip.
    ln -sf "$INST_TREE/tools/osmo-c123-gtk.py" /usr/local/bin/c123
    ln -sf "$INST_TREE/tools/osmo-c123-gtk.py" /usr/local/bin/osmo-c123
    # banc-rssi, banc-trx, c123-1...: profile.d aliases AND real commands (tools/osmo-banc-cmd.sh)
    install -m644 "$INST_TREE/scripts/osmo-banc-aliases.sh" /etc/profile.d/98-osmo-banc-aliases.sh
    OSMO_BANC_TREE="$INST_TREE" bash "$INST_TREE/tools/osmo-banc-cmd.sh" --install /usr/local/bin >/dev/null
    install -m644 "$INST_TREE/scripts/osmo-gnuarm-profile.sh" /etc/profile.d/97-osmo-gnuarm.sh   # gnuarm in the
     ↪  PATH, if installed
    # gcc-9 and gcc-11 next to gcc-13, with their alternatives, as on the ISO
    # (iso_modules/80-chroot.sh): jolly/testing and burst_ind of osmocom-bb need gcc-9.
    apt-get install -y --no-install-recommends gcc-9 g++-9 gcc-11 g++-11 >/dev/null \
        || inst_hint "gcc-9 / gcc-11 non installes : alternatives limitees aux compilateurs presents"
    bash "$INST_TREE/tools/osmo-gcc-alternatives.sh"
    # WireGuard for the WAN: "wg 1" / "wg N ..." (tools/osmo-wg.sh), as on the ISO
    install -m644 "$INST_TREE/scripts/osmo-wg-profile.sh" /etc/profile.d/97-osmo-wg.sh
    ln -sf "$INST_TREE/tools/osmo-wg.sh" /usr/local/bin/osmo-wg
    command -v wg-quick >/dev/null 2>&1 \
        || apt-get install -y --no-install-recommends wireguard-tools >/dev/null \
        || inst_hint "wireguard-tools non installe : « wg N » ne montera pas de tunnel"
    /usr/bin/python3 -c 'import gi, cairo; gi.require_version("Gtk", "3.0"); from gi.repository import Gtk'
     ↪  2>/dev/null \
        || apt-get install -y --no-install-recommends python3-gi python3-gi-cairo python3-cairo \
               gir1.2-gtk-3.0 gir1.2-gdkpixbuf-2.0 >/dev/null \
        || inst_hint "c123 : paquets GTK non installes (python3-gi python3-gi-cairo gir1.2-gtk-3.0)"

    while read -r h; do
```

```
        for d in Bureau Desktop; do
            [ -d "$h/$d" ] || continue
            for e in $_BUREAU_ENTRIES; do
                f="$h/$d/$e.desktop"
                cp -f "/usr/share/applications/$e.desktop" "$f" && chmod +x "$f"
                inst_say "bureau : $f"
            done
            [ "$h" != /root ] && chown "$SUDO_USER" "$h/$d"/osmo-*.desktop 2>/dev/null
        done
    done < <(_bureau_homes)

    command -v gtk-update-icon-cache >/dev/null 2>&1 && gtk-update-icon-cache -f -q /usr/share/icons/hicolor
    ↪  2>/dev/null
    command -v update-desktop-database >/dev/null 2>&1 && update-desktop-database /usr/share/applications
    ↪  2>/dev/null
    _bureau_fond
    inst_ok
}

inst_bureau_verify() {
    local e
    for e in $_BUREAU_ENTRIES; do
        grep -q "^Exec=$INST_TREE/" "/usr/share/applications/$e.desktop" \
            || { inst_fail "$e.desktop ne pointe pas sur $INST_TREE"; return $INST_RC_FAIL; }
    done
    inst_ok
}
```

2.54 /opt/GSM/osmo-operator/install_modules/90-verify.sh

727 octets, 17 lignes → 17 lignes

```
# 90-verify - overall check. Distinct from each step's `verify`: here we check
# whether the machine is usable, not whether a step succeeded.
INST_REGISTER verify "Verification de l'installation"
INST_DEPS[verify]="binaires configs"
INST_ROOT[verify]=0

inst_verify_run() {
    local missing="" b
    for b in osmo-stp osmo-hlr osmo-msc osmo-mgw osmo-bsc; do have_cmd "$b" || missing="$missing $b"; done
    if [ -n "$missing" ]; then
        inst_hint "les demons Osmocom ne sont pas dans les depots Ubuntu par defaut : ajoutez le depot Osmocom, ou
        ↪  utilisez Docker (./tools/make-docker-image.sh)"
        inst_fail "demons absents :$missing"
        return $INST_RC_FAIL
    fi
    inst_ok
}
inst_verify_verify() { inst_ok; }
```

2.55 /opt/GSM/osmo-operator/install_modules/_lib/inst.sh

2491 octets, 60 lignes → 60 lignes

```
# ============================================================================
#  install_modules/_lib/inst.sh - contract of the installation modules
# ============================================================================
#
#  Same principle as qosmo/run_modules/_lib/mod.sh, but for installation.
#  The steps do not "start" anything, they transform the machine: for each
#  one we want to know whether it is ALREADY done (idempotence) and whether
```

```
#  it SUCCEEDED (verification).
#
#  Four functions per module, only `run` is mandatory:
#     inst_<p>_check    prerequisites, READ ONLY       0 ok · 1 missing · 4 not applicable
#     inst_<p>_done     already installed?             0 yes · 1 no
#     inst_<p>_run      does the work                  0 ok · 1 failure · 3 already done
#     inst_<p>_verify   check AFTER the fact           0 ok · 1 failure
#
#  `verify` is the counterpart of the `wait` barrier on the run side:
#  installing is not having installed. An `apt-get` that returns 0 but leaves
#  a binary missing must be caught here, not three steps later.
# ------------------------------------------------------------------------------

readonly INST_RC_OK=0
readonly INST_RC_FAIL=1
readonly INST_RC_DONE=3
readonly INST_RC_NA=4

declare -a INST_ORDER=()
declare -A INST_DESC=()      # displayed description
declare -A INST_DEPS=()      # prerequisite steps
declare -A INST_REQUIRED=()  # 1 = a failure stops everything
declare -A INST_ROOT=()      # 1 = requires root
declare -A INST_TIMEOUT=()   # seconds, for long steps

INST_REGISTER() {
    local slug="$1" desc="$2"
    INST_ORDER+=("$slug")
    INST_DESC[$slug]="$desc"
    INST_DEPS[$slug]=""
    INST_REQUIRED[$slug]=1
    INST_ROOT[$slug]=1
    INST_TIMEOUT[$slug]=1800
}

inst_prefix() { printf 'inst_%s' "${1//-/_}"; }

# --- message protocol (same as the run modules) -----------------------------
_INST_REASON=""
_INST_HINT=""
inst_ok()   { _INST_REASON=""   ; return $INST_RC_OK; }
inst_fail() { _INST_REASON="$*" ; return $INST_RC_FAIL; }
inst_done() { _INST_REASON="$*" ; return $INST_RC_DONE; }
inst_na()   { _INST_REASON="$*" ; return $INST_RC_NA; }
inst_hint() { _INST_HINT="$*"; }
inst_say()  { printf '%s\n' "$*"; }

# --- probes -----------------------------------------------------------------
have_cmd()  { command -v "$1" >/dev/null 2>&1; }
have_dir()  { [ -d "$1" ]; }
have_file() { [ -f "$1" ]; }
have_pkg()  { dpkg -s "$1" >/dev/null 2>&1; }
have_repo() { [ -d "$1/.git" ]; }
```

2.56 /opt/GSM/osmo-operator/installer/calamares/jobs/osmosupplements/main.py

2906 octets, 72 lignes → 72 lignes

```
#!/usr/bin/env python3
# main.py - Calamares job "osmosupplements": carries the supplement choices
# from the installer to the first boot of the installed system.
#
# packagechooser@supplements (optionalmultiple, legacy method) stores the
# checked keys in packagechooser_supplements, comma separated;
```

```
# packagechooser@image stores ghcr or hub in packagechooser_image. They are
# translated into addition.sh flags, written to /etc/osmo-supplements in the
# target, and osmo-supplements.service is enabled: tools/osmo-supplements-firstboot.sh
# replays them at first boot, once the network is up.
import os

import libcalamares

DRAPEAUX = {
    "multi": "--multi",
    "opencl": "--opencl",
    "claude": "--claude",
    "firmwares": "--firmwares",
    "extras": "--extras",
}
IMAGES = {
    "ghcr": "ghcr.io/bbaranoff/osmo-operator/osmocom-nitb:latest",
    "hub": "bastienbaranoff/norf_gsm",
}
UNITE = "osmo-supplements.service"


def pretty_name():
    return "Supplements (premier demarrage)"


def _liste(cle):
    v = libcalamares.globalstorage.value(cle) or ""
    if isinstance(v, (list, tuple)):
        return [str(x).strip() for x in v if str(x).strip()]
    return [x.strip() for x in str(v).split(",") if x.strip()]


def run():
    root = libcalamares.globalstorage.value("rootMountPoint")
    if not root:
        return ("Supplements", "point de montage de la cible inconnu")
    choix = [c for c in _liste("packagechooser_supplements") if c in DRAPEAUX]
    conf = os.path.join(root, "etc/osmo-supplements")
    lien = os.path.join(root, "etc/systemd/system/multi-user.target.wants", UNITE)
    if not choix:
        libcalamares.utils.debug("osmosupplements : rien de coche")
        for f in (conf, lien):
            if os.path.lexists(f):
                os.remove(f)
        return None
    image = (_liste("packagechooser_image") or ["ghcr"])[0]
    with open(conf, "w") as f:
        f.write("# ecrit par l installeur (Calamares) - lu par tools/osmo-supplements-firstboot.sh\n")
        f.write('OSMO_SUPPLEMENTS="%s"\n' % " ".join(DRAPEAUX[c] for c in choix))
        if "multi" in choix:
            f.write('HUB_IMAGE="%s"\n' % IMAGES.get(image, IMAGES["ghcr"]))
        # Account created on the Users page: at first boot nobody is logged in
        # yet, so addition.sh could not find it (docker group, icons).
        compte = libcalamares.globalstorage.value("username")
        if compte:
            f.write('OSMO_COMPTE="%s"\n' % compte)
    if os.path.exists(os.path.join(root, "etc/systemd/system", UNITE)):
        os.makedirs(os.path.dirname(lien), exist_ok=True)
        if os.path.lexists(lien):
            os.remove(lien)
        os.symlink("/etc/systemd/system/" + UNITE, lien)
    else:
        libcalamares.utils.warning("osmosupplements : %s absente de la cible" % UNITE)
    libcalamares.utils.debug("osmosupplements : %s" % ", ".join(choix))
    return None
```

2.57 /opt/GSM/osmo-operator/iso_modules/00-options.sh

20712 octets, 345 lignes → 345 lignes

```
#!/bin/bash
# iso_modules/00-options.sh - command-line options, Ubuntu suite, root check
# Sourced by build-iso.sh in numeric order (same shell, variables and
# functions); not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (that is how --arm skips a step).
# Options: see iso_usage below (build-iso.sh --help).

NO_CACHE=""
# ── WAN: off by default. With --wan the ISO IS a WAN node ──────────────────
# The table is frozen into /etc/osmo-wan.conf in the image; at boot
# start-native.sh sees it (WAN_AUTO=1) and applies it.
#
# ONE ISO for all N machines: each one recognises itself by its IP in the
# table (wan_nodes_detect_self). --wan-id only forces a node when the IP is not
# enough (dynamic DHCP, NAT).
ISO_WAN=0
ISO_WAN_NODES=""
ISO_WAN_ID=""
ISO_WAN_OPS=1

# ── IMAGE ROLE ─────────────────────────────────────────────────────────────
# One build chain, two products:
#
#   --role operator --node N   →  osmo-operator-N.iso
#       A WAN node: full GSM core, its own point codes, its ASP attached to
#       the hub.
#
#   --role interstp            →  interstp.iso
#       The SS7 hub: PC 0.0.0, no operator, it ONLY routes M3UA between nodes.
#
# Without --role the default operator ISO is produced under its default name.
ISO_ROLE=""
ISO_ROLE_GIVEN=0
ISO_NODE=""
# ── LITE VARIANT ───────────────────────────────────────────────────────────
#   --lite  →  osmo-operator-lite.iso
# Same node, built from the pruned runtime image (Dockerfile.lite) instead of
# the build image: the /opt/GSM build trees (gnuradio, gr-gsm, libosmo*,
# osmo-*, qemu objects) are left out of the squashfs. What RUNS is identical.
# The three repositories stay complete (see Dockerfile.lite).
ISO_LITE=0
# ── DESKTOP VARIANT ────────────────────────────────────────────────────────
#   --desktop  →  osmo-operator-desktop.iso
# Same image plus a GNOME desktop (ubuntu-desktop-minimal), GUI wireshark and
# linphone-desktop, for what is not driven from the VTY: reading GSMTAP
# captures in wireshark, placing SIP calls by hand, gr-gsm Qt tools
# (grgsm_livemon). Adds about 2.5 GB to the rootfs, hence a separate variant.
ISO_DESKTOP=0
# --arm: boards for armbian/build-armbian.sh (--board=)
ISO_ARM_BOARD="${OSMO_ISO_BOARD:-all}"
# ── --all: ALL images in one pass ──────────────────────────────────────────
# Without arguments the script builds hub, node and lite node; --all adds the
# desktop image (~2.5 GB larger, so not built by default).
ISO_ALL=0
# ── PULL IS THE DEFAULT (amd64) ────────────────────────────────────────────
# The image is PULLED; build.sh only runs if no published image fits (see
# iso_docker_build in 21-docker-build.sh, which tries references in order).
#
# amd64 ONLY: nothing is published for arm64, so an --arm pass always builds.
# An explicit --skip-build is honoured on both architectures.
#
# --skip-build[=REF] makes the pull MANDATORY (no build.sh fallback, a failure
```

```
# is an error) and, with =REF, forces the reference instead of deriving it
# from the repository fingerprint.
ISO_SKIP_BUILD="${OSMO_ISO_SKIP_BUILD:-1}"
# Given on the command line, or only the default? Decides what happens when
# the pull fails: requested -> fatal; default -> fall back to a local build.
ISO_SKIP_BUILD_GIVEN=0
# FIRST image tried: the Docker Hub image, tagged osmocom-nitb. It does NOT
# track the commit (iso_docker_build says so before using it); GHCR
# (fingerprint, then :latest) comes next.
ISO_PULL_IMAGE="${OSMO_ISO_PULL_IMAGE:-bastienbaranoff/norf_gsm:latest}"
# --build-docker: the OPPOSITE of --skip-build, and it wins, wherever the
# --skip-build comes from (CI, a leftover OSMO_ISO_PULL_IMAGE, an alias). The
# image is built locally by build.sh. Env equivalent: OSMO_ISO_BUILD_DOCKER=1.
ISO_FORCE_BUILD="${OSMO_ISO_BUILD_DOCKER:-0}"
# --without-debs: NO .deb is shipped in the image. By default step 7c
# (71-debs-banc.sh) builds the bench packages (osmo-operator, bridge,
# qemu-calypso, calypso-firmware) into the rootfs's /var/cache/osmo-debs so an
# installed machine can rebuild the bench without docker or git. This option
# drops them and forces ISO_EMBED_DEBS=0 so the docker build packages (step 5)
# are not kept either: they are still used to install the stack in the rootfs.
# Env equivalent: OSMO_ISO_WITHOUT_DEBS=1.
ISO_WITH_DEBS=1
[ "${OSMO_ISO_WITHOUT_DEBS:-0}" = "1" ] && ISO_WITH_DEBS=0

# ── ARCHITECTURE: amd64 (PC ISO) or arm64 (SD images) ──────────────────────
# --arm: the SAME stack built for arm64 through the Armbian framework
# (armbian/build-armbian.sh, see below), delivered as SD images rather than a
# live ISO. Everything is built on the x86 host (aarch64 emulated by
# qemu-user-static: SLOW, around ten hours for the first build; the .deb cache
# makes later builds short). No installer and no --all on this target.
ISO_ARCH=amd64
# Default hub: the osmo-inter-stp container on the docker backbone
# (INTER_STP_IP in start-docker.sh). VMs reach it via the LAN -> docker route
# (network/setup-docker-lan-route.sh). VirtualBox host-only (192.168.56.1)
# remains possible via --hub-ip.
ISO_HUB_IP="172.20.0.10"
# ── Default WAN table: the bench ──────────────────────────────────────────
# Format: <node>:<IP>:<prefix>
#   node 1  192.168.1.123  the VM (this ISO)  prefix 11
#   node 2  172.20.0.12    osmo-operator-2    prefix 22   (container)
#   node 3  172.20.0.13    osmo-operator-3    prefix 33   (container)
#   hub     172.20.0.10    osmo-inter-stp                 (not in the table)
# Numbering plan: MSISDN = <node>00<op><ms> (100101 = node 1, op 1, MS 1;
# 200101 = node 2...) - see osmo_msisdn in generate_configs.sh. The prefix
# (11, 22, 33) is dialled to reach another node.
# Used when --wan-nodes is not given (lets non-interactive builds, e.g. CI,
# get past step 7b).
# The SAME table must be kept in start-interstp.sh (WAN_NODES_DEFAULT) and
# start-docker.sh, otherwise point codes no longer match the declared ASes.
ISO_WAN_NODES_DEFAULT="1:192.168.1.123:11 2:172.20.0.12:22 3:172.20.0.13:33"

# ── NO 2G STARTS AT BOOT ──────────────────────────────────────────────────
# osmo-multi.service and osmo-banc.service are installed but not enabled: a
# 2G bench is started by the operator, not as a boot side effect (once
# addition.sh writes /etc/osmocom/osmo-multi.conf, an enabled osmo-multi would
# bring up the whole multi-operator bench at boot).
#   systemctl start osmo-multi          one session (or the desktop icon)
#   systemctl enable --now osmo-multi   start it at every boot
# --multi (or OSMO_ISO_MULTI=1) at build time produces an image that enables
# it, like --banc for osmo-banc.
OSMO_ISO_MULTI="${OSMO_ISO_MULTI:-0}"
export OSMO_ISO_MULTI

OUTPUT_SET=0

# ── --help ────────────────────────────────────────────────────────────────
```

```
# Handled BEFORE the root check (further down): asking for help does not
# require root. Bracketed values are the real defaults, read from the
# variables above.
iso_usage() {
    # The repository colours (BOLD='\033[1m'...) are meant for `echo -e`; a
    # heredoc through `cat` does not interpret them. $'...' (ANSI-C quoting)
    # produces the real escape byte at assignment time.
    local B=$'\033[1m' C=$'\033[36m' D=$'\033[2m' N=$'\033[0m'
    cat <<EOF
${B}build-iso.sh${N} - fabrique une ISO bootable (ou une image SD arm64) a partir
de l'image docker osmocom-nitb. Tout passe par build.sh et start-docker.sh.

  ${B}Usage${N} : sudo ./build-iso.sh [options]
  Sans option : les quatre images amd64 - interstp, operator, lite, desktop.

${B}QUELLES IMAGES${N}
  ${C}--role=operator|interstp${N}  le noeud complet, ou le hub SS7 seul   ${D}[${ISO_ROLE:-operator}]${N}
  ${C}--lite${N}                    l'ISO allegee : sans les arbres de sources de /opt/GSM
  ${C}--desktop${N}                 l'ISO avec bureau GNOME et installateur (~2,5 Go de plus)
  ${C}--all${N}                     les quatre d'un coup (operator + lite + desktop + interstp)
  ${C}--node=N${N}                  numero du noeud dans le plan de numerotation    ${D}[${ISO_NODE:-1}]${N}
  ${C}--output=FICHIER${N}          nom du fichier de sortie
↪   ${D}[${OUTPUT:-osmo-operator-<version>.iso}]${N}

${B}L'IMAGE DOCKER SOURCE${N}
  ${D}Par defaut, en amd64, l'image est TIREE et non construite, puis taguee
  osmocom-nitb. Dans l'ordre : l'image Docker Hub (bastienbaranoff/norf_gsm:latest),
  puis ghcr.io/<depot>/osmocom-nitb:base-<empreinte du depot>, puis le :latest
  de GHCR. Seule l'image d'empreinte correspond a l'arbre de travail ; pour les
  autres le script previent avant de s'en servir.
  Si aucune ne repond, build.sh prend le relais (1h23). En arm64 rien n'est
  publie : la compilation reste le defaut.${N}
  ${C}--skip-build[=IMAGE]${N}      le pull devient OBLIGATOIRE : pas de repli sur
                            build.sh, et =IMAGE impose la reference au lieu de
                            la deduire de l'empreinte. N'importe quel registre :
                              --skip-build=ghcr.io/<depot>/osmocom-nitb:base-<empreinte>
                            Une image deja presente localement n'est pas retiree.
                            ${D}[${ISO_PULL_IMAGE:-bastienbaranoff/norf_gsm:latest}]${N}
  ${C}--build-docker${N}            l'inverse, et il gagne sur --skip-build d'ou qu'il
                            vienne : l'image est construite ici par build.sh.
  ${C}--no-cache${N}                tout recompiler : cache docker ignore, .deb refaits.
                            Sur un runner, c'est 1h23 - voir .github/workflows/.

${B}CE QUI VOYAGE DANS L'IMAGE${N}
  ${C}--without-debs${N}            aucun .deb embarque : ni les paquets du banc
                            (osmo-operator, pont, qemu-calypso, calypso-firmware),
                            ni ceux du build docker. Ils posent toujours la pile
                            dans le rootfs, ils n'y restent plus.
  ${C}--with-debs${N}               le defaut : les paquets du banc sont dans
                            /var/cache/osmo-debs de l'image, pour un
                            \`dpkg -i\` sans docker ni git.

${B}AU DEMARRAGE DE L'IMAGE${N}
  ${C}--banc / --no-banc${N}        activer (ou non) osmo-banc.service au boot   ${D}[${OSMO_ISO_BANC:-0}]${N}
  ${C}--multi / --no-multi${N}      activer (ou non) osmo-multi.service au boot  ${D}[${OSMO_ISO_MULTI:-0}]${N}
  ${C}--kb=LANG${N}                 disposition clavier                          ${D}[${OSMO_ISO_KB:-fr}]${N}

${B}LA SIM${N}
  ${C}--softsim / --no-softsim${N}  softSIM (SIM via SAP, Ruby) dans /opt/GSM/softsim, pySim et
                            pcscd/CCID pour la carte d un lecteur HID Omnikey    ${D}[${OSMO_ISO_SOFTSIM:-1}]${N}

${B}LE LIEN ENTRE NOEUDS (WAN)${N}
  ${C}--wan${N}                     active la table WAN
  ${C}--wan-nodes=LISTE${N}         <noeud>:<IP>:<indicatif>, separes par des espaces
                            ${D}[${ISO_WAN_NODES_DEFAULT:-}]${N}
  ${C}--wan-id=N${N}                numero de CE noeud dans la table
```

```
  ${C}--wan-ops=N${N}               nombre d'operateurs sur ce noeud
  ${C}--hub-ip=IP${N}               adresse du hub SS7

${B}LA BASE${N}
  ${C}--version=22.04|24.04${N}     suite Ubuntu du rootfs                        ${D}[${ISO_UBUNTU:-24.04}]${N}
  --arm, --arch=arm64       images SD arm64 par github.com/armbian/build
                            (armbian/build-armbian.sh) : Raspberry Pi 4 et
                            toutes les Orange Pi 5. --desktop = GNOME + le
                            bureau du banc. Pas de --all ni de --lite. Le
                            premier build emule aarch64 : une dizaine d'heures.
  ${C}--board=LISTE${N}             cartes de --arm : rpi4, opi5, all, ou des noms
                            Armbian (orangepi5-max,rpi4b...)       ${D}[${ISO_ARM_BOARD:-all}]${N}
  ${C}--arch=amd64${N}              le defaut.

  ${C}-h, --help${N}                ceci.

${B}Equivalents en variables${N} (pour la CI, ou un environnement sans ligne de commande) :
  OSMO_ISO_BUILD_DOCKER=1   comme --build-docker
  OSMO_ISO_SKIP_BUILD=0     ne pas tirer d'image : construire, comme avant 09-2026
  OSMO_ISO_WITHOUT_DEBS=1   comme --without-debs
  OSMO_ISO_PULL_IMAGE=REF   l'image de --skip-build
  OSMO_ISO_BANC=1 / OSMO_ISO_MULTI=1 / OSMO_ISO_KB=fr / OSMO_ISO_VERSION=...
  OSMO_ISO_SOFTSIM=0        comme --no-softsim
  OSMO_ISO_WORK=/chemin     repertoire de travail (defaut /var/tmp, PAS /tmp qui
                            est souvent un tmpfs : le rootfs n'y tient pas)
  OSMO_DEB_CACHE=/chemin    cache des .deb de l'hote   ${D}[/var/cache/osmo-debs]${N}
EOF
}

for arg in "$@"; do case "$arg" in
    -h|--help)      iso_usage; exit 0 ;;
    --output=*)     OUTPUT="${arg#*=}"; OUTPUT_SET=1 ;;
    --no-cache)     NO_CACHE="--no-cache" ;;
    --wan)          ISO_WAN=1 ;;
    --wan-nodes=*)  ISO_WAN=1; ISO_WAN_NODES="${arg#*=}" ;;
    --wan-id=*)     ISO_WAN=1; ISO_WAN_ID="${arg#*=}" ;;
    --wan-ops=*)    ISO_WAN=1; ISO_WAN_OPS="${arg#*=}" ;;
    --role=*)       ISO_ROLE="${arg#*=}"; ISO_ROLE_GIVEN=1 ;;
    --node=*)       ISO_NODE="${arg#*=}" ;;
    --hub-ip=*)     ISO_HUB_IP="${arg#*=}" ;;
    --kb=*)         OSMO_ISO_KB="${arg#*=}" ;;
    --version=*)    ISO_UBUNTU="${arg#*=}" ;;
    # Two-word form (`--version 24.04`): the loop is a `for` without shift, so
    # the flag remembers that the value comes on the next iteration and the
    # pattern below picks it up. These values mean nothing else on the command
    # line, so there is no collision.
    --version)      ISO_UBUNTU_NEXT=1 ;;
    22.04|24.04|jammy|noble)
                    if [ "${ISO_UBUNTU_NEXT:-0}" = 1 ]; then
                        ISO_UBUNTU="$arg"; ISO_UBUNTU_NEXT=0
                    fi ;;
    # ── 2G BENCH AT BOOT: NO, UNLESS EXPLICITLY REQUESTED ──────────────────
    # osmo-banc.service and osmo-multi.service are INSTALLED in every image
    # (82-services.sh) but not ENABLED. --banc / --multi (or OSMO_ISO_BANC=1 /
    # OSMO_ISO_MULTI=1) enable them at boot, per image.
    --banc)         OSMO_ISO_BANC=1 ;;
    --no-banc)      OSMO_ISO_BANC=0 ;;
    --multi)        OSMO_ISO_MULTI=1 ;;
    --no-multi)     OSMO_ISO_MULTI=0 ;;
    --softsim)      OSMO_ISO_SOFTSIM=1 ;;
    --no-softsim)   OSMO_ISO_SOFTSIM=0 ;;
    --lite)         ISO_LITE=1 ;;
    --desktop)      ISO_DESKTOP=1 ;;
    --all)          ISO_ALL=1 ;;
    --skip-build)   ISO_SKIP_BUILD=1; ISO_SKIP_BUILD_GIVEN=1 ;;
```

```
    --skip-build=*) ISO_SKIP_BUILD=1; ISO_SKIP_BUILD_GIVEN=1; ISO_PULL_IMAGE="${arg#*=}"; ISO_PULL_IMAGE_GIVEN=1
     ↪  ;;
    --build-docker) ISO_FORCE_BUILD=1 ;;
    --without-debs) ISO_WITH_DEBS=0 ;;
    --with-debs)    ISO_WITH_DEBS=1 ;;
    --arm|--arch=arm64) ISO_ARCH=arm64 ;;
    --board=*)      ISO_ARM_BOARD="${arg#*=}" ;;
    --arch=amd64)   ISO_ARCH=amd64 ;;
    --arch=*)       echo -e "${RED}--arch : amd64 ou arm64 (recu : ${arg#*=})${NC}" >&2; exit 2 ;;
esac; done

# Resolved AFTER the loop so option order does not matter: --build-docker
# wins over --skip-build, wherever it comes from (command line or variable).
if [ "$ISO_FORCE_BUILD" = "1" ] && [ "$ISO_SKIP_BUILD" = "1" ]; then
    # Only warn if a --skip-build was actually requested (pull is the default).
    [ "$ISO_SKIP_BUILD_GIVEN" = "1" ] && \
        echo -e "${YELLOW}--build-docker : --skip-build ignore, l image sera construite localement${NC}"
    ISO_SKIP_BUILD=0
fi
# Without embedded .debs, the docker build's .debs do not stay in the rootfs
# either (50-injection-image.sh removes them unless ISO_EMBED_DEBS=1).
[ "$ISO_WITH_DEBS" = "0" ] && ISO_EMBED_DEBS=0
export ISO_FORCE_BUILD ISO_WITH_DEBS

# ── UBUNTU BASE: 22.04 (jammy) or 24.04 (noble) ──────────────────────
# Single variable for the suite name (debootstrap and sources.list).
#
# Default 24.04 (noble): the ISO MUST use the same suite as the docker image
# it copies /usr/local and the /root/.env venv from (python 3.12, glibc 2.39,
# *t64 libraries); a jammy rootfs under a noble image would give a venv
# without interpreter and .so files with missing symbols. This is checked
# later against the source image's os-release. 22.04 is accepted for an image
# built on jammy.
: "${ISO_UBUNTU:=24.04}"
case "$ISO_UBUNTU" in
    22.04|jammy) ISO_UBUNTU=22.04; ISO_SUITE=jammy ;;
    24.04|noble) ISO_UBUNTU=24.04; ISO_SUITE=noble ;;
    *) echo -e "${RED}--version : 22.04 ou 24.04 (recu : $ISO_UBUNTU)${NC}" >&2; exit 2 ;;
esac
export ISO_UBUNTU ISO_SUITE

# Derived from the architecture: the mirror (arm64 is on ports, NOT
# archive.ubuntu.com), the docker image suffix (both architectures coexist on
# one host), the output format. ISO_IMG_TAG is appended to image names:
# osmocom-nitb:arm64.
case "$ISO_ARCH" in
    amd64) ISO_MIRROR="$(bash "$DIR/packaging/apt-mirror.sh" "$ISO_SUITE" 2>/dev/null || echo
     ↪  http://archive.ubuntu.com/ubuntu)"; ISO_IMG_TAG="" ;;
    arm64) ISO_MIRROR="http://ports.ubuntu.com/ubuntu-ports"; ISO_IMG_TAG=":arm64" ;;
esac
# The suffix only exists for coexistence: an arm64 image built on an x86
# host must not overwrite the native image of the same name. Same rule as
# build.sh, which only sets IMG_TAG when $OSMO_ARCH != $HOST_ARCH. On an
# aarch64 host there is no suffix (build.sh tags plain osmocom-nitb, which
# 31-image-source.sh must find).
[ "$ISO_ARCH" = "$(dpkg --print-architecture 2>/dev/null || echo amd64)" ] && ISO_IMG_TAG=""
export ISO_ARCH ISO_MIRROR ISO_IMG_TAG
[ "$(id -u)" -ne 0 ] && { echo -e "${RED}Root requis.${NC}"; exit 1; }

# --arm GOES THROUGH THE ARMBIAN FRAMEWORK (github.com/armbian/build):
# armbian/build-armbian.sh builds kernel, BSP, u-boot/firmware, rootfs and
# image for each board, and installs the stack via
# userpatches/customize-image.sh. This script is replaced by
# build-armbian.sh here, before any step.
if [ "$ISO_ARCH" = "arm64" ]; then
```

```
    [ "$ISO_ALL" = "1" ]  && { echo -e "${RED}--arm : pas de --all (--board=all construit toutes les cartes)${NC}"
     ↪  >&2; exit 2; }
    [ "$ISO_LITE" = "1" ] && { echo -e "${RED}--arm : pas de --lite avec le framework Armbian${NC}" >&2; exit 2; }
    case "${ISO_ROLE:-operator}" in operator) ;; *) echo -e "${RED}--arm : --role=operator seulement (recu :
     ↪  $ISO_ROLE)${NC}" >&2; exit 2 ;; esac
    [ "$ISO_SUITE" = noble ] || { echo -e "${RED}--arm : noble (24.04) seulement - les .deb du build arm64 visent
     ↪  noble${NC}" >&2; exit 2; }
    [ "$OUTPUT_SET" = "1" ] && echo -e "${YELLOW}--arm : --output ignore, les images sortent dans le
     ↪  output/images/ d armbian/build${NC}"
    _ARM=("--board=${ISO_ARM_BOARD:-all}" "--release=$ISO_SUITE" --install-deps)  # 20-hote.sh (which installed
     ↪  qemu-user-static) is not reached
    [ "$ISO_DESKTOP" = "1" ]            && _ARM+=(--desktop)
    [ "${OSMO_ISO_BANC:-0}" = "1" ]     && _ARM+=(--banc)
    [ "${OSMO_ISO_MULTI:-0}" = "1" ]    && _ARM+=(--multi)
    # only the --skip-build=REF image: the default (norf_gsm:latest) is amd64
    [ "${ISO_PULL_IMAGE_GIVEN:-0}" = "1" ] && _ARM+=("--image=$ISO_PULL_IMAGE")
    # nothing is published for arm64 except GHCR base-arm64: without an explicit
    # --skip-build, the stack image is built if missing
    [ "$ISO_SKIP_BUILD_GIVEN" = "1" ] || _ARM+=(--build-image)
    [ "${OSMO_ISO_ARM_DRY:-0}" = "1" ]  && _ARM+=(--dry-run)  # show the compile.sh commands without running
     ↪  anything
    echo -e "  ${CYAN}cible : arm64, framework Armbian - armbian/build-armbian.sh ${_ARM[*]}${NC}"
    exec bash "$DIR/armbian/build-armbian.sh" "${_ARM[@]}"
fi


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.58 /opt/GSM/osmo-operator/iso_modules/10-clavier.sh

3695 octets, 62 lignes → 62 lignes

```
#!/bin/bash
# iso_modules/10-clavier.sh - image keyboard layout (interactive question or --kb)
# Sourced by build-iso.sh in numeric order (same shell, variables and
# functions); not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (that is how --arm skips a step).

# ═══════════════════════════════════════════════════════════════════════
# QUESTIONS: all asked here, once, before anything starts
# ═══════════════════════════════════════════════════════════════════════
# Everything interactive is asked HERE, never mid-build nor in the image at
# first boot:
#   - before the build, so a running pass is never interrupted;
#   - only once even when several images are produced: the answer is exported
#     and child passes inherit it;
#   - never in CI: without a terminal the default is used (waiting for an EOF
#     would fail the build under set -e).
#
# --kb=XX or OSMO_ISO_KB=XX skip the question.
ISO_KB_DEFAULT="fr"
if [ -z "${OSMO_ISO_KB:-}" ]; then
    if [ -t 0 ]; then
        echo -e "${CYAN}${BOLD}== Clavier de l'image ==${NC}"
        echo "  1) fr   2) us   3) de   4) es   5) it"
        echo "  6) pt   7) gb   8) be   9) ch   0) autre"
        read -rp "  Choix [1] : " _kb_choice || _kb_choice=""
        case "${_kb_choice:-1}" in
            1|"") OSMO_ISO_KB="fr" ;;
            2) OSMO_ISO_KB="us" ;;  3) OSMO_ISO_KB="de" ;;
            4) OSMO_ISO_KB="es" ;;  5) OSMO_ISO_KB="it" ;;
```

```
            6) OSMO_ISO_KB="pt" ;;  7) OSMO_ISO_KB="gb" ;;
            8) OSMO_ISO_KB="be" ;;  9) OSMO_ISO_KB="ch" ;;
            0) read -rp "  Layout (fr, us, ru, ar...) : " OSMO_ISO_KB || OSMO_ISO_KB=""
               OSMO_ISO_KB="${OSMO_ISO_KB:-$ISO_KB_DEFAULT}" ;;
            *) OSMO_ISO_KB="$ISO_KB_DEFAULT" ;;
        esac
    else
        OSMO_ISO_KB="$ISO_KB_DEFAULT"
        echo -e "  ${YELLOW}Pas de terminal : clavier ${OSMO_ISO_KB} (--kb=XX pour changer)${NC}"
    fi
fi
export OSMO_ISO_KB
echo -e "  ${GREEN}✓${NC} clavier de l'image : ${CYAN}${OSMO_ISO_KB}${NC}"

# ── Without arguments: ALL default images ──────────────────────────
# A WAN needs both an SS7 hub and nodes, so both are produced, in separate
# full passes.
#
# ONE operator image serves all nine nodes: the node number is chosen at boot
# (`start-native.sh --node N`), which rewrites the point codes.
#
# ═══════════════════════════════════════════════════════════════════════
# DONE ONLY ONCE: host packages and the docker build
# ═══════════════════════════════════════════════════════════════════════
# The parent pass (--all) calls them once (20-hote.sh, 21-docker-build.sh);
# child passes skip them via OSMO_ISO_HOST_READY / OSMO_ISO_IMAGE_READY. A
# standalone pass calls them itself, once.


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.59 /opt/GSM/osmo-operator/iso_modules/20-hote.sh

11091 octets, 196 lignes → 196 lignes

```
#!/bin/bash
# iso_modules/20-hote.sh - host packages needed to build the image (iso_host_packages)
# Sourced by build-iso.sh in numeric order (same shell, variables and
# functions); not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (that is how --arm skips a step).

# ── Host packages required to build the ISO (squashfs, grub, xorriso...) ──
# Installed here rather than in the CI workflow: `sudo ./build-iso.sh` is
# enough on a fresh Debian/Ubuntu machine.
#
# The list is what the chain calls ON THE HOST, starting from a fresh Ubuntu
# 24.04 with only ubuntu-minimal:
#   syslinux-utils   isohybrid (91-secure-boot). The isolinux package does NOT
#                    provide it; without it the ISO has no hybrid partition
#                    table and a dd-written USB key does not boot on some BIOSes.
#   rsync            calamares unpackfs, rootfs copies (87-lite).
#   wget curl        TAS2781 firmware and gsm-1.0.24 sources (52-qemu), pmOS
#                    kernel (88-lte-pmos), repository keys (80-chroot).
#   ca-certificates gnupg  downloads are https, repository keys go through
#                    gpg --dearmor.
#   python3-pil      tools/plymouth-render.py (86-finitions): the boot theme is
#                    GENERATED and needs Pillow.
#   python3-yaml     calamares config check (85-installeur-bureau).
#   gcc make         toast/untoast (GSM 06.10 codec) built on the host for the
#                    rootfs (52-qemu). Without them the ISO silently lacks toast.
#   kpartx           installer and pmOS disk images (85, 88).
```

```
#   e2fsprogs fdisk util-linux  mke2fs / sfdisk / losetup.
#   xz-utils cpio file zstd  squashfs -comp xz, initrd, identification.
#   grub2-common     grub-install; grub-common alone only has grub-mkrescue.
#   fonts-ubuntu     tools/grub-theme-render.py (86, 90): GRUB theme fonts,
#                    converted by grub-mkfont (grub-common).
#   meson ninja-build pkg-config libpulse-dev libx11-dev libxext-dev
#   libxrender-dev libxcomposite-dev libgl-dev  GLava built on the host for the
#                    rootfs (tools/glava-build.sh, 85-glava).
# Ubuntu 24.04 already provides tar, gzip, dpkg-deb, coreutils, mount. aria2
# and curl come with apt-fast-install.sh.
#
# shim-signed / grub-efi-amd64-signed / dosfstools: the Secure Boot chain
# (see step 9); without them the ISO does not boot with Secure Boot enabled
# and the firmware only reports a certificate error.
# apt-fast everywhere: same installer as the Dockerfile and the chroot
# (packaging/apt-fast-install.sh), hence the same apt settings on the host.
#
# COMMON TO BOTH ARCHITECTURES: everything except grub and xorriso.
ISO_HOST_PKGS_COMMON="debootstrap git rsync wget curl ca-certificates gnupg \
zstd xz-utils cpio file gcc make python3 python3-pil python3-yaml \
e2fsprogs fdisk util-linux dosfstools mtools kpartx"
# amd64: the ISO itself - squashfs, grub (BIOS + EFI), xorriso, isohybrid.
ISO_HOST_PKGS="$ISO_HOST_PKGS_COMMON squashfs-tools xorriso grub-common grub2-common grub-pc-bin
 ↪  grub-efi-amd64-bin shim-signed grub-efi-amd64-signed isolinux syslinux-common syslinux-utils \
fonts-ubuntu meson ninja-build pkg-config libpulse-dev libx11-dev libxext-dev libxrender-dev libxcomposite-dev
 ↪  libgl-dev"
# --arm: no grub, xorriso or squashfs. Needs emulation (qemu-user-static +
# binfmt-support: chroot and docker run aarch64 code on the x86 host),
# debootstrap, and tools to build the SD image without mounting anything:
# mke2fs -d (e2fsprogs), mkfs.vfat + mcopy (dosfstools, mtools), sfdisk.
[ "${ISO_ARCH:-amd64}" = "arm64" ] \
    && ISO_HOST_PKGS="$ISO_HOST_PKGS_COMMON qemu-user-static binfmt-support docker-buildx"

# ── Docker: a dependency, not an option ──────────────────────────────
# The whole chain starts from a docker image (build.sh, or a pull). build.sh
# installs docker.io + docker-compose-v2 + docker-buildx but is not called on
# a pull, while the 22-wan-table tool check requires docker in both cases.
# ONLY IF MISSING: installing docker.io on a host with docker-ce (Docker repo)
# would break it. An existing docker, whatever its origin, is left as is.
iso_host_docker() {
    if command -v docker >/dev/null 2>&1; then
        docker info >/dev/null 2>&1 || systemctl start docker >/dev/null 2>&1 || true
        return 0
    fi
    echo -e "  ${CYAN}docker absent : installation de docker.io + compose v2 + buildx${NC}"
    apt-fast install -y --no-install-recommends docker.io docker-compose-v2 docker-buildx \
        || apt-fast install -y --no-install-recommends docker.io docker-compose-v2 \
        || apt-fast install -y --no-install-recommends docker.io \
        || { echo -e "${RED}docker non installable par apt : posez-le (get.docker.com) et relancez${NC}" >&2;
         ↪  return 1; }
    systemctl enable --now docker >/dev/null 2>&1 || true
}

# ── Tools called by the chain, and the package providing each ────────
# A missing tool is reported with THE command that installs it. Optional
# tools (isohybrid, gcc) only produce a warning.
iso_host_check_tools() {
    local missing="" pkgs="" t p
    local required="debootstrap git rsync wget curl mkfs.vfat mcopy sfdisk mke2fs kpartx python3"
    local optional="isohybrid gcc make"
    [ "${ISO_ARCH:-amd64}" = "arm64" ] \
        || required="$required mksquashfs unsquashfs xorriso grub-mkrescue grub-mkimage"
    for t in $required; do
        command -v "$t" >/dev/null 2>&1 && continue
        case "$t" in
            mksquashfs|unsquashfs) p=squashfs-tools ;;
```

```
            grub-mkrescue|grub-mkimage) p=grub-common ;;
            mkfs.vfat) p=dosfstools ;;  mcopy) p=mtools ;;
            sfdisk) p=fdisk ;;          mke2fs) p=e2fsprogs ;;
            *) p="$t" ;;
        esac
        missing="$missing $t"; pkgs="$pkgs $p"
    done
    for t in $optional; do
        command -v "$t" >/dev/null 2>&1 \
            || echo -e "  ${YELLOW}!${NC} $t absent (optionnel) : $([ "$t" = isohybrid ] \
                && echo "pas de table hybride sur l ISO (clef USB gravee en dd)" \
                || echo "toast/untoast ne seront pas compiles")" >&2
    done
    if [ -n "$missing" ]; then
        echo -e "${RED}Outils hote manquants :${missing}${NC}" >&2
        echo -e "${RED}  apt-get install -y$(echo "$pkgs" | tr ' ' '\n' | sort -u | tr '\n' ' ')${NC}" >&2
        return 1
    fi
    # Pillow: plymouth-render.py runs with the python3 from PATH, which may be a
    # venv (/root/.env) without python3-pil. Warning only: the boot theme does
    # not prevent the ISO from being built.
    python3 -c "import PIL" >/dev/null 2>&1 \
        || echo -e "  ${YELLOW}!${NC} Pillow absent du python3 du PATH ($(command -v python3)) - theme Plymouth
        ↪  non genere" >&2
    return 0
}

# May be called several times (21-docker-build with --all, then
# 22-wan-table): apt only runs on the first call.
_ISO_HOST_PKGS_DONE=0
iso_host_packages() {
    [ "$_ISO_HOST_PKGS_DONE" = "1" ] && return 0
    _ISO_HOST_PKGS_DONE=1
    if [ "${OSMO_ISO_HOST_READY:-0}" = "1" ]; then
        echo -e "${GREEN}[0/9] Paquets hote : deja poses par la passe parente${NC}"; return 0
    fi
    if command -v apt-get &>/dev/null; then
        echo -e "${GREEN}[0/9] Installation des paquets hote (apt-fast)...${NC}"
        export DEBIAN_FRONTEND=noninteractive
        bash "$DIR/packaging/apt-fast-install.sh" >/dev/null 2>&1 \
            || echo -e "  ${YELLOW}apt-fast non installe - apt-get${NC}"
        command -v apt-fast >/dev/null 2>&1 || apt-fast() { apt-get "$@"; }
        apt-fast update -qq || true
        # One unavailable package (removed from a suite, partial mirror) fails the
        # whole line: fall back to installing package by package and NAME the
        # failures.
        if ! apt-fast install -y --no-install-recommends $ISO_HOST_PKGS; then
            echo -e "  ${YELLOW}installation groupee refusee - reprise paquet par paquet${NC}"
            local _ko="" _p
            for _p in $ISO_HOST_PKGS; do
                apt-fast install -y --no-install-recommends "$_p" >/dev/null 2>&1 || _ko="$_ko $_p"
            done
            [ -n "$_ko" ] && echo -e "  ${YELLOW}non installes :${_ko}${NC}" >&2
        fi
        iso_host_docker || true
    else
        echo -e "${YELLOW}apt-get absent : verification seule des outils hote.${NC}"
    fi
    iso_host_check_tools
}

# ── binfmt: the x86 host must run aarch64 code TRANSPARENTLY ──────────
# Two consumers. The rootfs chroot: qemu-aarch64-static must be registered in
# binfmt_misc and copied INTO the rootfs. Docker (buildx --platform
# linux/arm64, docker run of the arm64 image for the ldd closure): the
# container has no emulator, so the F (fix-binary) flag is required for the
```

```
# kernel to keep the host emulator open. qemu-user-static registers it with F
# since jammy; otherwise use Docker's binfmt image (tonistiigi/binfmt), which
# does exactly that registration (a download, announced).
#
# On an arm64 host there is nothing to emulate: compare with the host
# architecture, as build.sh and 40-rootfs.sh do, and only set up emulation
# when they differ.
iso_arm_binfmt() {
    [ "${ISO_ARCH:-amd64}" = "arm64" ] || return 0
    if [ "${ISO_ARCH}" = "$(dpkg --print-architecture 2>/dev/null || echo amd64)" ]; then
        echo -e "  ${GREEN}✓${NC} hote deja en ${ISO_ARCH} : ni qemu-user ni binfmt, tout est natif"
        return 0
    fi
    local f=/proc/sys/fs/binfmt_misc/qemu-aarch64
    [ -d /proc/sys/fs/binfmt_misc ] || mount -t binfmt_misc binfmt_misc /proc/sys/fs/binfmt_misc 2>/dev/null ||
     ↪  true
    [ -e "$f" ] || update-binfmts --enable qemu-aarch64 >/dev/null 2>&1 || true
    if [ -e "$f" ] && grep -q '^flags:.*F' "$f"; then
        echo -e "  ${GREEN}✓${NC} binfmt qemu-aarch64 (fix-binary) : chroot et docker savent executer l aarch64"
        return 0
    fi
    echo -e "  ${YELLOW}binfmt qemu-aarch64 sans drapeau F - enregistrement via tonistiigi/binfmt (docker)${NC}"
    docker run --privileged --rm tonistiigi/binfmt --install arm64 >/dev/null 2>&1 || true
    if [ -e "$f" ] && grep -q '^flags:.*F' "$f"; then
        echo -e "  ${GREEN}✓${NC} binfmt qemu-aarch64 enregistre (fix-binary)"
    else
        echo -e "${RED}binfmt qemu-aarch64 indisponible : apt install qemu-user-static binfmt-support, puis
         ↪  relancez${NC}" >&2
        exit 1
    fi
}


# ── INSTALL THEM HERE, AT STARTUP ──────────────────────────────────────
# Host packages are installed in module 20, before any docker build, so a
# missing tool fails early (only 10-clavier runs before). The calls in 21 and
# 22 do nothing more (_ISO_HOST_PKGS_DONE guard).
iso_host_packages


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.60 /opt/GSM/osmo-operator/iso_modules/20-stockage-docker.sh

2750 octets, 49 lignes → 49 lignes

```
#!/bin/bash
# iso_modules/20-stockage-docker.sh - keep Docker storage off overlayfs
# Sourced by build-iso.sh in numeric order (after 20-hote: docker is
# installed). Not meant to run on its own.
#
# When the host is itself a container (root on `overlay`), the docker/containerd
# overlayfs snapshotter mounts an overlay ON an overlay, which the kernel
# refuses. The build then fails at the first `RUN --mount=type=cache` with:
#   failed to solve: mount source: "overlay" ... err: invalid argument
# Fix: put /var/lib/docker and /var/lib/containerd on a loop-mounted ext4
# (sparse file). Idempotent; does nothing if storage is already fine.
# Env: OSMO_ISO_NO_DOCKER_FIX=1 disables it; OSMO_ISO_DOCKER_LOOP_GB sets the
# size in GB (default 10).
iso_docker_storage_fix() {
    command -v docker >/dev/null 2>&1 || return 0
```

```
    [ "${OSMO_ISO_NO_DOCKER_FIX:-0}" = 1 ] && return 0
    # A pull alone (--skip-build) does not need nested overlays.
    local fs; fs="$(findmnt -no FSTYPE -T /var/lib/containerd 2>/dev/null || stat -f -c %T /var/lib/containerd
     ↪  2>/dev/null)"
    [ "$fs" = overlay ] || return 0
    [ "$(id -u)" = 0 ] || { echo -e "${YELLOW}! /var/lib/containerd est sur overlay : le build docker echouera
     ↪  (relancer en root)${NC}" >&2; return 0; }

    local img=/var/lib/docker-loop.img base=/var/lib/dockerdata gb="${OSMO_ISO_DOCKER_LOOP_GB:-10}"
    echo -e "  ${CYAN}stockage docker sur overlay : bascule sur un ext4 en boucle (${gb} Go, $img)${NC}"
    if [ -n "$(docker ps -q 2>/dev/null)" ]; then
        echo -e "${RED}des conteneurs tournent : arretez-les avant (le correctif redemarre docker)${NC}" >&2
        return 1
    fi
    systemctl stop docker docker.socket containerd || return 1
    mkdir -p "$base"
    if [ ! -f "$img" ]; then
        truncate -s "${gb}G" "$img" && mkfs.ext4 -q -F "$img" || return 1
    fi
    mountpoint -q "$base" || mount -o loop "$img" "$base" || return 1
    mkdir -p "$base/docker" "$base/containerd"
    mountpoint -q /var/lib/docker     || mount --bind "$base/docker" /var/lib/docker || return 1
    mountpoint -q /var/lib/containerd || mount --bind "$base/containerd" /var/lib/containerd || return 1
    if ! grep -q docker-loop /etc/fstab 2>/dev/null; then
        printf '%s\n' "$img $base ext4 loop 0 0" \
            "$base/docker /var/lib/docker none bind 0 0" \
            "$base/containerd /var/lib/containerd none bind 0 0" >> /etc/fstab
    fi
    systemctl start containerd docker || return 1
    sleep 3
    echo -e "  ${GREEN}✓${NC} docker stocke sur $(findmnt -no FSTYPE -T /var/lib/containerd)"
}
iso_docker_storage_fix || { echo -e "${RED}Correctif du stockage docker echoue (voir plus haut)${NC}" >&2; exit 1;
 ↪  }

# End of module: always return 0 (see 20-hote.sh).
true
```

2.61 /opt/GSM/osmo-operator/iso_modules/21-docker-build.sh

16327 octets, 271 lignes → 271 lignes

```
#!/bin/bash
# iso_modules/21-docker-build.sh - iso_docker_build, --all parent pass, output file name
# Sourced by build-iso.sh in numeric order (same shell, variables and
# functions); not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (that is how --arm skips a step).

# ── ONE docker build ──────────────────────────────────────────────────
# The source image of ALL ISOs is osmocom-nitb (Dockerfile), built by build.sh
# (docker compose + .deb cache). Configs are injected from this repository
# (step 2b); lite pruning is done on the rootfs (step 8c).
# Only exception: --role=interstp requested ALONE. The hub only needs osmo-stp
# and three libraries: Dockerfile.stp, without building the full stack.
# ── CANDIDATE IMAGES, IN ORDER ────────────────────────────────────────
# The fingerprint is computed exactly like the CI: same command as
# .github/workflows/docker.yml (job `base`) and build-iso.yml (context
# fingerprint step). If it matches, the image published under that name was
# built from EXACTLY the working tree. The path list must be changed in all
# three places together, otherwise fingerprints diverge and nothing is pulled.
#
# Order:
#   1. the EXPLICIT --skip-build=REF reference, used alone;
#   1b. the Docker Hub image ($ISO_PULL_IMAGE, bastienbaranoff/norf_gsm:latest),
```

```
#      tagged osmocom-nitb like the others;
#   2. ghcr.io/<repo>/osmocom-nitb:base-<fingerprint> - matches the commit,
#      published by docker.yml;
#   3. ghcr.io/<repo>/osmocom-nitb:latest - latest image published by the same
#      workflow (public registry, no token needed); does not track the commit.
# Only 2 guarantees that the image matches the working tree; for 3,
# iso_docker_build warns before using it.
iso_pull_refs() {
    if [ "${ISO_SKIP_BUILD_GIVEN:-0}" = "1" ]; then echo "$ISO_PULL_IMAGE"; return 0; fi
    local _key _repo
    # Must be an `if` condition: build-iso.sh runs under `set -euo pipefail`,
    # and an assignment from a failing pipeline (outside a git repo, git ls-files
    # exits 128) would silently stop the script. As a condition, set -e lets it
    # through and _key stays empty.
    # The 2>/dev/null is ON git, not on the `if`: in a simple command expansions
    # happen before redirections, so a trailing redirection would not cover the
    # command substitution.
    if ! _key="$(cd "$DIR" && git ls-files -- Dockerfile packaging configs scripts services \
                                helpers opt-gsm tools patches 2>/dev/null \
                | sort | xargs sha256sum 2>/dev/null | sha256sum | cut -c1-16)"; then
        _key=""
    fi
    # Repository from origin; outside a git clone (hand-deployed archive), fall
    # back to the upstream repository.
    _repo="$(cd "$DIR" && git config --get remote.origin.url 2>/dev/null || true)"
    _repo="${_repo%.git}"; _repo="${_repo##*github.com[:/]}"
    # Accept only `owner/name`: an origin not on github.com (ssh, gitlab, local
    # path) comes out of the split above unchanged and would give a bogus GHCR
    # reference.
    case "$_repo" in *[:@\ ]*|*/*/*) _repo="" ;; */*) ;; *) _repo="" ;; esac
    [ -n "$_repo" ] || _repo="bbaranoff/osmo-operator"
    # The fingerprint is empty outside a git repo: no fingerprint reference, but
    # the same repository's latest is still valid.
    # Docker Hub FIRST: bastienbaranoff/norf_gsm:latest is the bench's reference
    # stack; GHCR is the fallback.
    echo "$ISO_PULL_IMAGE"
    [ -n "$_key" ] && echo "ghcr.io/${_repo,,}/osmocom-nitb:base-${_key}"
    echo "ghcr.io/${_repo,,}/osmocom-nitb:latest"
}

iso_docker_build() {
    local role="$1"
    if [ "${OSMO_ISO_IMAGE_READY:-0}" = "1" ]; then
        echo -e "${GREEN}[1/9] Image docker : deja construite par la passe parente${NC}"; return 0
    fi
    # ── PULL RATHER THAN BUILD ──────────────────────────────────────────
    # The pulled image is tagged with the name build.sh would produce
    # (osmocom-nitb, or osmocom-nitb:arm64 with --arm) so 31-image-source and the
    # rest see no difference; OSMO_ISO_SRC_IMAGE also prevents a local image of
    # the same name from being used.
    #
    # TWO GUARDS, applying only to the DEFAULT (an explicit --skip-build
    # overrides both):
    #   arm64    nothing is published for this architecture; an amd64 pull on an
    #            arm64 target would give an unusable rootfs.
    #   interstp the hub alone builds in MINUTES from Dockerfile.stp (osmo-stp and
    #            three libraries), faster than pulling the ~11 GB base image.
    local _try_pull=0
    if [ "${ISO_SKIP_BUILD:-0}" = "1" ] && [ "${ISO_FORCE_BUILD:-0}" != "1" ]; then
        if [ "${ISO_SKIP_BUILD_GIVEN:-0}" = "1" ]; then _try_pull=1
        elif [ "${ISO_ARCH:-amd64}" = "amd64" ] && [ "$role" != "interstp" ]; then _try_pull=1
        fi
    fi
    if [ "$_try_pull" = "1" ]; then
        local _local="osmocom-nitb${ISO_IMG_TAG}" _ref _got=""
        echo -e "${GREEN}[1/9] Image docker : on TIRE avant de construire${NC}"
```

```
    for _ref in $(iso_pull_refs); do
        # ALREADY PRESENT? Do not pull. The registry may not be reachable for the
        # current user: on a runner, docker/login-action writes credentials to the
        # runner account's ~/.docker while build-iso.sh runs under sudo, so a GHCR
        # pull would have no token. The image was pulled upstream.
        if docker image inspect "$_ref" >/dev/null 2>&1; then
            echo -e "  ${GREEN}✓${NC} ${CYAN}${_ref}${NC} deja presente localement"
            _got="$_ref"; break
        fi
        # manifest inspect BEFORE pulling: one HTTP round trip, whereas a failing
        # pull can spend minutes retrying layers.
        if docker manifest inspect "$_ref" >/dev/null 2>&1; then
            echo -e "  ${CYAN}pull${NC} ${_ref} (linux/${ISO_ARCH:-amd64})..."
            if docker pull --platform "linux/${ISO_ARCH:-amd64}" "$_ref"; then
                _got="$_ref"; break
            fi
        fi
        echo -e "  ${YELLOW}!${NC} ${_ref} : indisponible"
    done
    if [ -n "$_got" ]; then
        # Anything that is not a fingerprint image does not track the commit: warn
        # (an ISO built on it may carry a stack different from the working tree).
        # The `:base-` pattern identifies the only images that match.
        # Use `if`, not "[ ... ] && [ ... ] && echo": a && chain whose first test is
        # false returns 1, which stops the script under set -e.
        # Except the Docker Hub image: it is first BY CHOICE, so no warning.
        if [ "${_got}" = "${_got#*:base-}" ] && [ "${_got}" != "${ISO_PULL_IMAGE}" ] \
           && [ "${ISO_SKIP_BUILD_GIVEN:-0}" != "1" ]; then
            echo -e "  ${YELLOW}!${NC} aucune image ne correspond a l'empreinte du depot : ${CYAN}${_got}${NC}
             ↪  ne suit pas ce commit (${CYAN}--build-docker${NC} pour compiler l'arbre de travail)"
        fi
        docker tag "$_got" "$_local"
        export OSMO_ISO_SRC_IMAGE="$_local"
       echo -e "  ${GREEN}✓${NC} image ${_local} prete ($(docker image inspect "$_local" --format '{{.Size}}'
         ↪  2>/dev/null | awk '{printf "%.0f Mo", $1/1048576}'))"
        return 0
    fi
    # Explicitly requested pull: failure is an error. Default: it was only a
    # shortcut, so build.
    if [ "${ISO_SKIP_BUILD_GIVEN:-0}" = "1" ]; then
        echo -e "${RED}--skip-build : aucune image joignable (${ISO_PULL_IMAGE})${NC}" >&2; exit 1
    fi
    echo -e "  ${YELLOW}!${NC} aucune image publiee joignable - construction locale"
fi
if [ "$role" = "interstp" ]; then
   echo -e "${GREEN}[1/9] Hub seul : construction de ${CYAN}osmocom-stp${NC}${GREEN} (Dockerfile.stp)...${NC}"
    echo -e "  ${CYAN}osmo-stp + libosmocore + libosmo-netif + libosmo-sigtran. Rien d'autre.${NC}"
    if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
        # arm64: buildx --platform, image tagged :arm64, host .deb cache copied into
        # the context as build.sh does (osmo-deb reads packages of ITS architecture;
        # stray amd64 ones do not matter).
        mkdir -p "$DIR/.deb-cache"; cp -f "${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"/osmo-build-*_arm64.deb
         ↪  "$DIR/.deb-cache/" 2>/dev/null || true
        docker buildx build --platform linux/arm64 --load $NO_CACHE \
            --build-arg "OSMO_DEB_REFRESH=$([ -n "$NO_CACHE" ] && echo 1 || echo 0)" \
            -f "$DIR/Dockerfile.stp" -t "osmocom-stp${ISO_IMG_TAG}" "$DIR" \
            || { echo -e "${RED}Echec de la construction d'osmocom-stp${ISO_IMG_TAG}${NC}"; exit 1; }
    elif docker compose version >/dev/null 2>&1; then
        ( cd "$DIR" && OSMO_DEB_REFRESH="$([ -n "$NO_CACHE" ] && echo 1 || echo 0)" \
          docker compose -f "$DIR/compose.yaml" build $NO_CACHE stp ) \
            || { echo -e "${RED}Echec de la construction d'osmocom-stp${NC}"; exit 1; }
    else
        docker build $NO_CACHE -f "$DIR/Dockerfile.stp" -t osmocom-stp "$DIR" \
            || { echo -e "${RED}Echec de la construction d'osmocom-stp${NC}"; exit 1; }
    fi
```

```
        echo -e "  ${GREEN}✓${NC} osmocom-stp${ISO_IMG_TAG} construite ($(docker image inspect
        ↪  "osmocom-stp${ISO_IMG_TAG}" --format '{{.Size}}' 2>/dev/null | awk '{printf "%.0f Mo", $1/1048576}'))"
        return 0
    fi
    echo -e "${GREEN}[1/9] Execution de build.sh (image osmocom-nitb${ISO_IMG_TAG}, docker compose + cache
    ↪  .deb)...${NC}"
    if [ -f "$DIR/build.sh" ]; then
        # --arch=arm64: build.sh uses buildx --platform linux/arm64 and tags the
        # image osmocom-nitb:arm64. Everything else (apt-fast, docker, sctp, toast,
        # .deb cache) is identical.
        bash "$DIR/build.sh" $NO_CACHE $([ "${ISO_ARCH:-amd64}" = "arm64" ] && echo "--arch=arm64")
    else
        echo -e "${YELLOW}build.sh introuvable, construction manuelle de l'image osmocom-nitb...${NC}"
        if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
            # --target: the Dockerfile ends with the `debs` stage (.deb cache for CI),
            # not osmocom-nitb. Without it the image would be empty.
            docker buildx build --platform linux/arm64 --load $NO_CACHE \
                --target osmocom-nitb -t "osmocom-nitb${ISO_IMG_TAG}" "$DIR"
        else
            docker build $NO_CACHE --target osmocom-nitb -t osmocom-nitb "$DIR"
        fi
    fi
    echo -e "  ${GREEN}✓${NC} image osmocom-nitb${ISO_IMG_TAG} prete"
}

if [ "$ISO_ALL" = "1" ] || { [ "${ISO_ARCH:-amd64}" = "amd64" ] && [ "$ISO_ROLE_GIVEN" = "0" ] && [ "$OUTPUT_SET"
 ↪  = "0" ] \
   && [ -z "$ISO_NODE" ] && [ "$ISO_LITE" = "0" ] && [ "$ISO_DESKTOP" = "0" ]; }; then
    _N="QUATRE"  # --all by default: all FOUR images, desktop included
    echo -e "${CYAN}${BOLD}══ Construction des ${_N} images ══${NC}"
    echo -e "  1. ${CYAN}interstp.iso${NC}               le hub SS7 (PC 0.0.0)"
    echo -e "  2. ${CYAN}osmo-operator.iso${NC}          un noeud - son numero se choisit au demarrage :"
    echo -e "     ${CYAN}./start-native.sh --node N${NC}   (N de 1 a 9)"
    echo -e "  3. ${CYAN}osmo-operator-lite.iso${NC}     le meme noeud, sans les ateliers de compilation"
    echo -e "  4. ${CYAN}osmo-operator-desktop.iso${NC}  le meme noeud, avec GNOME, wireshark et linphone"
    echo ""

    # --all and --desktop are REMOVED from the arguments passed to child passes;
    # otherwise each pass would enter this same block (endless recursion).
    SUB_ARGS=()
    for _a in "$@"; do
        case "$_a" in --all|--desktop) ;; *) SUB_ARGS+=("$_a") ;; esac
    done
    set -- "${SUB_ARGS[@]+"${SUB_ARGS[@]}"}"

    # ── ONCE ONLY: host apt, docker build ──────────────────────────────
    iso_host_packages
    iso_docker_build operator
    export OSMO_ISO_HOST_READY=1 OSMO_ISO_IMAGE_READY=1 OSMO_ISO_ALL_RUN=1

    # ── ORDER: interstp, normal, lite, desktop - the rootfs is handed over ────
    # Hub first: small, independent, a failure shows in minutes. Then the NORMAL
    # image, built from scratch (debootstrap + apt + injection): the expensive
    # one. The other two REUSE ITS ROOTFS:
    #   lite     = a COPY of the normal rootfs with the build trees REMOVED (so it
    #              must come after the normal image);
    #   desktop  = the normal rootfs itself (moved), with the apt delta ADDED:
    #              desktop, installer, snaps.
    # apt is idempotent: on an inherited rootfs the common list costs a few
    # seconds of resolution, only the delta is downloaded.
    _KEEP="$WORK"; mkdir -p "$_KEEP"
    _all_fail() { echo -e "${RED}Echec de $1${NC}" >&2; rm -rf "$_KEEP"; exit 1; }
    "$0" --role=interstp "$@" || _all_fail interstp.iso
    OSMO_ISO_ROOTFS_KEEP="$_KEEP/rootfs-normal" \
        "$0" --role=operator --output=osmo-operator.iso "$@" || _all_fail osmo-operator.iso
    [ -d "$_KEEP/rootfs-normal" ] || _all_fail "osmo-operator.iso (rootfs non transmis)"
```

```
    OSMO_ISO_ROOTFS_FROM="$_KEEP/rootfs-normal" OSMO_ISO_ROOTFS_MODE=copy \
        "$0" --role=operator --lite --output=osmo-operator-lite.iso "$@" || _all_fail osmo-operator-lite.iso
    _ISOS=("$(pwd)/interstp.iso" "$(pwd)/osmo-operator.iso" "$(pwd)/osmo-operator-lite.iso")
    OSMO_ISO_ROOTFS_FROM="$_KEEP/rootfs-normal" OSMO_ISO_ROOTFS_MODE=move \
        "$0" --role=operator --desktop --output=osmo-operator-desktop.iso "$@" || _all_fail
        ↪ osmo-operator-desktop.iso
    _ISOS+=("$(pwd)/osmo-operator-desktop.iso")
    rm -rf "$_KEEP"
    echo -e "${GREEN}${BOLD}═══ Les ${_N} images sont pretes ═══${NC}"
    ls -lh "${_ISOS[@]}" 2>/dev/null | sed 's/^/  /'
    exit 0
fi

case "${ISO_ROLE:-operator}" in
    interstp)
        ISO_ROLE="interstp"
        [ "$OUTPUT_SET" = "1" ] || OUTPUT="interstp.iso"
        # The hub has no build trees to prune: its image (Dockerfile.stp) only has
        # osmo-stp and four libraries. --lite is meaningless here and is rejected.
        [ "$ISO_LITE" = "1" ] && { echo "--lite ne s'applique pas au hub (--role=interstp)" >&2; exit 2; }
        # Same for the desktop: the hub runs headless (server room or VM without a
        # graphical console).
        [ "$ISO_DESKTOP" = "1" ] && { echo "--desktop ne s'applique pas au hub (--role=interstp)" >&2; exit 2; }
        # The hub serves N nodes: it needs the WAN table to know how many.
        ISO_WAN=1 ;;
    operator|"")
        ISO_ROLE="operator"
        if [ -n "$ISO_NODE" ]; then
            [[ "$ISO_NODE" =~ ^[1-9]$ ]] || { echo "--node : 1 a 9" >&2; exit 2; }
            if [ "$OUTPUT_SET" = "0" ]; then
                _sfx=""
                [ "$ISO_LITE" = "1" ]    && _sfx="${_sfx}-lite"
                [ "$ISO_DESKTOP" = "1" ] && _sfx="${_sfx}-desktop"
                OUTPUT="osmo-operator-${ISO_NODE}${_sfx}.iso"
            fi
            ISO_WAN_ID="${ISO_WAN_ID:-$ISO_NODE}"
            ISO_WAN=1
        elif [ "$ISO_LITE" = "1" ] || [ "$ISO_DESKTOP" = "1" ]; then
            if [ "$OUTPUT_SET" = "0" ]; then
                _sfx=""
                [ "$ISO_LITE" = "1" ]    && _sfx="${_sfx}-lite"
                [ "$ISO_DESKTOP" = "1" ] && _sfx="${_sfx}-desktop"
                OUTPUT="osmo-operator${_sfx}.iso"
            fi
        fi ;;
    *) echo "--role inconnu : $ISO_ROLE (operator|interstp)" >&2; exit 2 ;;
esac
# --arm: an SD image, not an ISO. Same naming rule (role, node, -lite), with
# -rpi4.img instead of .iso - except --output=, used as is.
if [ "${ISO_ARCH:-amd64}" = "arm64" ] && [ "$OUTPUT_SET" != "1" ]; then
    OUTPUT="${OUTPUT%.iso}-rpi4.img"
fi
case "$OUTPUT" in /*) ;; *) OUTPUT="$(pwd)/$OUTPUT" ;; esac


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.62 /opt/GSM/osmo-operator/iso_modules/22-wan-table.sh

3220 octets, 64 lignes → 64 lignes

```
#!/bin/bash
# iso_modules/22-wan-table.sh - WAN node table, cleanup trap, required tools, WORK dirs
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# ── WAN table: fixed HERE, before the build starts ──────────────────────
# --role=interstp implies the WAN (the hub must know how many nodes it serves).
# The table is fixed now, with the bench table as default, so that no question
# is asked in the middle of an hour-long (possibly CI, terminal-less) build.
# --wan-nodes="..." and --hub-ip=... take precedence; OSMO_ISO_WAN_ASK=1 asks
# interactively instead (requires a terminal).
if [ "$ISO_WAN" = "1" ] && [ -z "$ISO_WAN_NODES" ]; then
    if [ "${OSMO_ISO_WAN_ASK:-0}" = "1" ] && [ -t 0 ]; then
        echo -e "${CYAN}${BOLD}== Table WAN de l'image ==${NC}"
        echo -e "  Format : ${CYAN}<noeud>:<IP>:<indicatif>${NC}, separes par des espaces."
        echo -e "  Entree vide = le banc : ${CYAN}${ISO_WAN_NODES_DEFAULT}${NC}"
        read -rp "  Noeuds : " _wan_in || _wan_in=""
        ISO_WAN_NODES="${_wan_in:-$ISO_WAN_NODES_DEFAULT}"
        if [ "$ISO_ROLE" = "interstp" ]; then
            read -rp "  IP du hub [${ISO_HUB_IP}] : " _hub_in || _hub_in=""
            ISO_HUB_IP="${_hub_in:-$ISO_HUB_IP}"
        fi
    else
        ISO_WAN_NODES="$ISO_WAN_NODES_DEFAULT"
    fi
    echo -e "  ${GREEN}✓${NC} WAN : ${CYAN}${ISO_WAN_NODES}${NC}   hub ${CYAN}${ISO_HUB_IP}${NC}  (defaut du banc
    ↪  ; --wan-nodes= / --hub-ip= pour changer)"
fi

# Propagate --no-cache to both Docker builds: build.sh (osmocom-nitb image) and
# build_run_image (osmocom-run image, via DOCKER_NO_CACHE).
export DOCKER_NO_CACHE="$NO_CACHE"


cleanup() { umount "$ROOTFS/var/cache/apt/archives" 2>/dev/null||true; umount "$ROOTFS"/{dev/pts,proc,sys,dev}
↪  2>/dev/null||true; rm -rf "$WORK"; }
trap cleanup EXIT


# Host packages: once (see iso_host_packages, defined earlier).
iso_host_packages

# Docker is not auto-installed here (docker-ce is outside the standard apt repos).
_ISO_TOOLS="docker mksquashfs xorriso grub-mkrescue debootstrap git"
# qemu-aarch64-static is only required to EMULATE: on a host that is already
# aarch64 it is not needed. Same rule as 40-rootfs.sh, which decides on
# --foreign by comparing with the host architecture.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    _ISO_TOOLS="docker debootstrap git mke2fs mkfs.vfat mcopy sfdisk truncate"
    [ "$ISO_ARCH" = "$(dpkg --print-architecture 2>/dev/null || echo amd64)" ] \
        || _ISO_TOOLS="$_ISO_TOOLS qemu-aarch64-static"
fi
for t in $_ISO_TOOLS; do
    command -v "$t" &>/dev/null || { echo -e "${RED}Manquant: $t${NC}"; exit 1; }
done
iso_arm_binfmt
mkdir -p "$WORK" "$ROOTFS" "$ISOROOT"

echo -e "${CYAN}${BOLD}== osmo-operator ISO builder (via build.sh + start-docker.sh) ==${NC}"


# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.63 /opt/GSM/osmo-operator/iso_modules/30-image-configs.sh

8269 octets, 156 lignes → 156 lignes

```
#!/bin/bash
# iso_modules/30-image-configs.sh - step 1 (docker build) and 2 (configs via start-docker.sh)
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# ── Step 1: THE docker build (once; see iso_docker_build) ──────────────────
iso_docker_build "$ISO_ROLE"

# load_start_lib: extract the function library of launch/start-docker.sh
# (everything before its main code) into $WORK and source it. This provides
# apply_config_templates.
load_start_lib() {
    local src="$DIR/launch/start-docker.sh"
    local lib="$WORK/start.lib.sh"

    awk '
        BEGIN { skip=0 }
        /^banner[[:space:]]*$/                  { exit }
        /^\[ "\$\{1:-\}" = "stop" \]/           { exit }
        /^\[ "\$\(id -u\)" -ne 0 \]/            { exit }
        /^choose_network_mode[[:space:]]*$/     { exit }
        /^\.\//                                 { exit }
        /^case "\$NETWORK_MODE" in[[:space:]]*$/  { exit }
        { print }
    ' "$src" > "$lib"

    # The lib lives in $WORK, not in the repo: without this, the BASH_SOURCE
    # resolution of start-docker.sh would look for generate_configs.sh next to the copy.
    export OSMO_REPO_DIR="$DIR"

    # shellcheck disable=SC1090
    source "$lib"
}

# ── Step 2: the start-docker.sh library (apply_config_templates) ────────────
# A single docker image (step 1); the configs come from this repo and the lite
# pruning is done on the rootfs (step 8c).
load_start_lib
echo -e "${GREEN}[2/9] Image source : ${CYAN}$([ "$ISO_ROLE" = "interstp" ] && [ "${OSMO_ISO_ALL_RUN:-0}" != "1" ]
 ↪  && echo osmocom-stp || echo osmocom-nitb)${NC}"

echo -e "${GREEN}[2b/9] Preparation de l'image source de l'ISO...${NC}"

ISO_N_MS=2
ISO_OP_ID=1         # single operator in the ISO (PLMN 001-01)
ENCRYPTION="a5 1"   # A5/1 by default in the ISO

# The ISO runs NATIVELY, without a docker bridge. The core must not bind on
# the default NIC address: it may be absent (VM without a card), named outside
# "en* eth*", or not yet configured by systemd-networkd when osmo-ggsn starts.
# Affected: osmo-ggsn (gtp bind-ip), osmo-sgsn (ggsn remote-ip), osmo-upf
# (local-addr), osmo-bsc (gprs nsvc remote ip) and the gsmtap log.
ISO_PRIV_BASE=$(( ${ISO_NODE:-1} + 1 ))
ISO_PRIV_GW="192.168.${ISO_PRIV_BASE}.1"
ISO_PRIV_IP="192.168.${ISO_PRIV_BASE}.10"

# ── Base address: the private segment, plus loopback ────────────────────────
# SGSN and GGSN both open the same GTP socket (2123/2152/3386): they MUST be
# on different addresses, otherwise the second one fails with "bind failed:
# Address already in use" and systemd restarts it in a loop (packet attach
# then hangs forever). The node plan gives exactly two addresses:
```

```
#     .10  ip1  the core: GGSN, NS/Gb of the SGSN, UPF, nsvc of the BSC
#     .1   private gw    the GTP endpoint of the SGSN, alone
# Both are set as /32 by network/osmo-ip-plan.sh before run.sh starts anything;
# its fallback puts them on `lo` when no card provides Internet, so they
# always exist.
HOST_IP="$ISO_PRIV_IP"     # ip1 : __CONTAINER_IP__ - ggsn/sgsn-NS/upf/bsc-nsvc
SGSN_GTP_IP="$ISO_PRIV_GW" # the SGSN GTP endpoint, distinct from the GGSN
GATEWAY_IP="127.0.0.1"     # gw  : __GATEWAY_IP__  - gsmtap log + dns 0 of the ggsn
# HLR and GSUP stay on loopback: a node-internal control plane that nobody
# calls from outside; osmo-hlr.cfg hard-codes "bind ip 127.0.0.2".
HLR_IP="127.0.0.2"

# ── The private segment of this node: 192.168.<node+1>.x ──────────────────
# Same plan as the docker side (start-docker.sh: op_private_*), so that a VM
# and a container of the same rank describe themselves the same way. The +1
# leaves 192.168.1.0/24 to the bench LAN (VMs and SS7 hub).
# These addresses are /32: the goal is a stable attachment point for the
# configurations that still name a private address, not a real segment.
# __INTER_STP_IP__: ASP towards the STP of another operator. Inert here - the
# ISO has a single operator and passes inter_stp_shutdown=shutdown to
# apply_config_templates - but no dead docker IP is left in the configs.
INTER_STP_IP="127.0.0.1"   # ip2 : inter-operator (ASP shutdown on the ISO)

echo -e "  Host IP    : ${CYAN}${HOST_IP}${NC}"
echo -e "  Gateway    : ${CYAN}${GATEWAY_IP}${NC}"
echo -e "  Inter-STP  : ${CYAN}${INTER_STP_IP}${NC}"
echo -e "  MS         : ${CYAN}${ISO_N_MS}${NC}"
echo -e "  Encryption : ${CYAN}${ENCRYPTION}${NC}"

TEMP_CONFIG="$(mktemp -d)"

# ── Point codes and SS7 attachment ────────────────────────────────────────
# Without WAN: the historical plan, 1.<op>.<role>, and the inter-STP ASP shut
# down (a single operator has nobody to talk to over SS7).
#
# With --node N: the node is part of the point code, 1.<node><op>.<role>.
# Otherwise three ISOs attached to the same hub would all present 1.11.2: a
# point code is an address, and duplicates mean silent wrong routing.
ISO_PC_MSC="1.1.1"; ISO_PC_STP="1.1.2"; ISO_PC_BSC="1.1.3"
ISO_INTER_SHUT="shutdown"
ISO_INTER_IP="$INTER_STP_IP"
if [ -n "$ISO_NODE" ]; then
    ISO_PC_MSC="1.${ISO_NODE}${ISO_OP_ID}.1"
    ISO_PC_STP="1.${ISO_NODE}${ISO_OP_ID}.2"
    ISO_PC_BSC="1.${ISO_NODE}${ISO_OP_ID}.3"
    ISO_INTER_IP="$ISO_HUB_IP"
    ISO_INTER_SHUT="no shutdown"
    # Unique RCTX too: the hub identifies each AS by its routing context.
    export RCTX_INTER_OVERRIDE=$(( ISO_NODE * 1000 + ISO_OP_ID * 100 + 50 ))
    # ASP local-ip left at 0.0.0.0: the node address comes from DHCP and is
    # not necessarily up when osmo-stp starts; binding to an absent address
    # fails at launch.
    export INTER_LOCAL_IP_OVERRIDE="0.0.0.0"
    echo -e "  Noeud WAN  : ${CYAN}${ISO_NODE}${NC}  PC ${CYAN}${ISO_PC_STP}${NC}  hub ${CYAN}${ISO_HUB_IP}${NC}
     ↪  rctx ${RCTX_INTER_OVERRIDE}"
fi

apply_config_templates "$TEMP_CONFIG" \
    "$HOST_IP" "$GATEWAY_IP" \
    "1" "$ISO_PC_MSC" "$ISO_PC_STP" "$ISO_PC_BSC" \
    "001" "01" "OsmoGSM" \
    "$ISO_INTER_IP" "$ISO_INTER_SHUT" "1"

# ── Inter-STP role: the hub config, for N nodes ───────────────────────────
if [ "$ISO_ROLE" = "interstp" ]; then
    _hub_nodes=3
```

```
    if [ -n "$ISO_WAN_NODES" ]; then
        _hub_nodes=$(printf '%s' "${ISO_WAN_NODES//,/ }" | wc -w)
    fi
    bash "$DIR/helpers/create_interop.sh" --wan "$_hub_nodes" "${ISO_WAN_OPS:-1}" \
        "$TEMP_CONFIG/osmocom/osmo-stp-interop.cfg" || exit 1
    echo -e "  ${GREEN}✓${NC} hub SS7 pour ${CYAN}${_hub_nodes}${NC} noeud(s) × ${ISO_WAN_OPS:-1} operateur(s)"
fi

# ── NATIVE post-patches ──────────────────────────────────────────────
# AFTER apply_config_templates, not before: it overwrites sms-routing.conf,
# osmo-sgsn.cfg and osmo-msc.cfg with the templates (i.e. the DOCKER plan).
# The recipe lives once in generate_configs.sh (apply_native_post_patches) and
# both paths (ISO build, ./start-native.sh --regen) call it.
# The WAN table as this ISO will embed it: it gives sms-routing.conf the
# address of EACH node. Without it (isolated bench ISO), the generator only
# writes our own entry.
ISO_WAN_TMP=""
if [ -n "$ISO_WAN_NODES" ]; then
    ISO_WAN_TMP="$(mktemp -p "$WORK")"
    printf 'WAN_NODES="%s"\n' "$ISO_WAN_NODES" > "$ISO_WAN_TMP"
fi
apply_native_post_patches "$TEMP_CONFIG" "$ISO_OP_ID" "$ISO_N_MS" "$HOST_IP" \
    "${ISO_NODE:-1}" "${ISO_WAN_TMP:-/nonexistent}" "$SGSN_GTP_IP" "$HLR_IP"
echo -e "  ${GREEN}✓${NC} retouches natives : sms-routing (${CYAN}${ISO_N_MS}${NC} route(s) MS), GGSN/NS
 ↪  ${CYAN}${HOST_IP}${NC}, GTP SGSN ${CYAN}${SGSN_GTP_IP}${NC}, HLR ${CYAN}${HLR_IP}${NC}"


# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.64 /opt/GSM/osmo-operator/iso_modules/31-image-source.sh

4411 octets, 75 lignes → 75 lignes

```
#!/bin/bash
# iso_modules/31-image-source.sh - ISO source image, suite check, run image
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# ── The source image ─────────────────────────────────────────────────
# osmocom-nitb for everyone (a single build), except the hub requested ALONE:
# osmocom-stp, built at step 1. In an --all pass the hub also takes
# osmocom-nitb (which carries osmo-stp): an osmocom-stp image left over from
# another base must not sneak in. OSMO_ISO_SRC_IMAGE forces an image.
if [ -n "${OSMO_ISO_SRC_IMAGE:-}" ]; then
    ISO_SRC_IMAGE="$OSMO_ISO_SRC_IMAGE"
elif [ "$ISO_ROLE" = "interstp" ] && [ "${OSMO_ISO_ALL_RUN:-0}" != "1" ] \
     && docker image inspect "osmocom-stp${ISO_IMG_TAG}" >/dev/null 2>&1; then
    ISO_SRC_IMAGE="osmocom-stp${ISO_IMG_TAG}"
else
    ISO_SRC_IMAGE="${IMAGE_NITB:-osmocom-nitb}${ISO_IMG_TAG}"
fi
case "$ISO_ROLE:$ISO_LITE" in
    interstp:*) ISO_RUN_IMAGE="osmocom-stp-iso${ISO_IMG_TAG}" ;;
    *:1)        ISO_RUN_IMAGE="osmocom-run-lite-iso${ISO_IMG_TAG}" ;;
    *)          ISO_RUN_IMAGE="osmocom-run-iso-net-host${ISO_IMG_TAG}" ;;
esac
docker image inspect "$ISO_SRC_IMAGE" >/dev/null 2>&1 \
    || { echo -e "${RED}Image source ${ISO_SRC_IMAGE} introuvable${NC}" >&2; exit 1; }

# ── The image and the rootfs must be the SAME Ubuntu suite ──────────
```

```
# /usr/local, /root/.env and /root/.venv-qemu are copied as-is: a noble venv
# (python 3.12) on a jammy rootfs (python 3.10) has no interpreter, and the
# image's .so files need a glibc the rootfs does not have. The image's
# os-release is read and a mismatch is refused; --version=<suite> aligns them.
# OSMO_ISO_SUITE_MISMATCH_OK=1 continues anyway.
_img_suite="$(docker run --rm --entrypoint sh "$ISO_SRC_IMAGE" -c '. /etc/os-release; echo "$VERSION_CODENAME"'
 ↪  2>/dev/null | tr -d '[:space:]' || true)"
if [ -n "$_img_suite" ] && [ "$_img_suite" != "$ISO_SUITE" ]; then
    if [ "${OSMO_ISO_SUITE_MISMATCH_OK:-0}" = "1" ]; then
        echo -e "  ${YELLOW}⚠ image ${ISO_SRC_IMAGE} en ${_img_suite}, rootfs en ${ISO_SUITE}
         ↪  (OSMO_ISO_SUITE_MISMATCH_OK=1 : on continue)${NC}"
    else
        echo -e "${RED}L image ${ISO_SRC_IMAGE} est construite sur ${_img_suite}, le rootfs demande est
         ↪  ${ISO_SUITE}.${NC}" >&2
        echo -e "${RED}Relancez avec --version=${_img_suite}, ou reconstruisez l image (./build.sh) sur la base
         ↪  voulue.${NC}" >&2
        exit 1
    fi
fi
echo -e "  ${GREEN}✓${NC} image source ${CYAN}${ISO_SRC_IMAGE}${NC} (${_img_suite:-suite inconnue}) -> rootfs
 ↪  ${CYAN}${ISO_SUITE}${NC}"
TMP_CID="$(docker create "$ISO_SRC_IMAGE" /bin/sh)"

# The hub carries NO operator: only the hub config is pushed, and the
# operator's osmo-stp.cfg (point code 1.1.2, docker local-ip) is removed so
# that two STP configurations never coexist in /etc/osmocom (start-interstp.sh
# discriminates by the NAME of the config file passed to osmo-stp). The rest of
# /etc/osmocom (run.sh, status.sh, check.sh, entrypoint.sh) is still needed:
# the osmocom-stp image starts from a bare ubuntu:22.04.
if [ "$ISO_ROLE" = "interstp" ]; then
    rm -f "$TEMP_CONFIG/osmocom/osmo-stp.cfg"
fi
docker cp "$TEMP_CONFIG/osmocom/."  "$TMP_CID:/etc/osmocom/"  2>/dev/null || true
# The hub has no Asterisk: no SIP configs for it (the image has no /etc/asterisk).
[ "$ISO_ROLE" = "interstp" ] || \
    docker cp "$TEMP_CONFIG/asterisk/." "$TMP_CID:/etc/asterisk/" 2>/dev/null || true

docker commit "$TMP_CID" "$ISO_RUN_IMAGE" >/dev/null
docker rm -f "$TMP_CID" >/dev/null 2>&1 || true
rm -rf "$TEMP_CONFIG"

echo -e "  ${GREEN}✓${NC} image ${CYAN}${ISO_RUN_IMAGE}${NC} prete"

# ── Step 3: none - NATIVE ISO, the Docker image is NOT embedded ──────────────
# The osmocom-run image is only a build SOURCE (docker cp of binaries and
# configs into the rootfs at step 5). No docker at runtime, no multi-GB tarball.


# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.65 /opt/GSM/osmo-operator/iso_modules/40-rootfs.sh

4943 octets, 82 lignes → 82 lignes

```
#!/bin/bash
# iso_modules/40-rootfs.sh - step 4: debootstrap (or inherited rootfs)
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment:
```

```
#   ISO_DEB_CACHE=<abs dir>   persistent .deb cache reused by debootstrap
#                             (--cache-dir); empty or --no-cache -> no cache.
#   OSMO_ISO_ROOTFS_FROM=<dir> reuse the rootfs of a previous pass (--all).
#   OSMO_ISO_ROOTFS_MODE=move|copy  how to take it over (default: move).

# ── Step 4: bootstrap a minimal rootfs ──────────────────────────────────
# ── .deb cache (speeds up rebuilds) ─────────────────────────────────────
ISO_DEB_CACHE="${ISO_DEB_CACHE:-$HOME/.cache/osmo-iso-debs}"
DEBOOTSTRAP_CACHE_OPT=""
if [ -n "$ISO_DEB_CACHE" ] && [ -z "$NO_CACHE" ]; then
    # The cache is PER ARCHITECTURE: an amd64 .deb and its arm64 counterpart
    # share the same file name except for the arch, and debootstrap does not
    # check the architecture of what it takes from the cache.
    _DBS_CACHE="$ISO_DEB_CACHE/debootstrap"
    [ "${ISO_ARCH:-amd64}" = "amd64" ] || _DBS_CACHE="$ISO_DEB_CACHE/debootstrap-$ISO_ARCH"
    mkdir -p "$_DBS_CACHE"
    DEBOOTSTRAP_CACHE_OPT="--cache-dir=$_DBS_CACHE"
    echo -e "  ${GREEN}cache .deb debootstrap : $_DBS_CACHE${NC}"
fi
# ── Rootfs INHERITED from a previous pass (--all: lite and desktop reuse the
# normal one) or debootstrap from scratch. An inherited rootfs is complete and
# configured: step 5 (injection) is skipped, the other steps replay - they
# write their files, apt only installs the delta.
OSMO_ISO_INHERITED=0
if [ -n "${OSMO_ISO_ROOTFS_FROM:-}" ]; then
    [ -d "$OSMO_ISO_ROOTFS_FROM" ] || { echo -e "${RED}Rootfs herite introuvable : ${OSMO_ISO_ROOTFS_FROM}${NC}"
     ↪  >&2; exit 1; }
    rmdir "$ROOTFS" 2>/dev/null || true
    case "${OSMO_ISO_ROOTFS_MODE:-move}" in
        copy)
            echo -e "${GREEN}[4/9] Rootfs repris (COPIE) de ${CYAN}${OSMO_ISO_ROOTFS_FROM}${NC}..."
            # --reflink=auto: instant on btrfs/xfs, plain copy elsewhere.
            cp -a --reflink=auto "$OSMO_ISO_ROOTFS_FROM" "$ROOTFS" ;;
        *)
            echo -e "${GREEN}[4/9] Rootfs repris (deplace) de ${CYAN}${OSMO_ISO_ROOTFS_FROM}${NC}..."
            mv "$OSMO_ISO_ROOTFS_FROM" "$ROOTFS" ;;
    esac
    OSMO_ISO_INHERITED=1
    echo -e "  ${GREEN}✓${NC} rootfs herite $(du -sh "$ROOTFS"|cut -f1) - debootstrap et injection sautes"
else
    # The debootstrap script for the suite: on a jammy host the package does
    # not know noble. All Ubuntu scripts are the same (gutsy): link it.
    _DBS=/usr/share/debootstrap/scripts
    if [ -d "$_DBS" ] && [ ! -e "$_DBS/$ISO_SUITE" ] && [ -e "$_DBS/gutsy" ]; then
        ln -s gutsy "$_DBS/$ISO_SUITE"
        echo -e "  ${YELLOW}debootstrap ne connaissait pas ${ISO_SUITE} : script gutsy lie${NC}"
    fi
    echo -e "${GREEN}[4/9] debootstrap $ISO_SUITE ($ISO_UBUNTU, ${ISO_ARCH}, minimal)...${NC}"
    # arm64 on an x86 host: --foreign only DOWNLOADS and UNPACKS (nothing is
    # executed); the second stage configures the packages in the chroot, where
    # qemu-aarch64-static (copied into the rootfs) runs the aarch64 binaries
    # via binfmt. On amd64 the flag is empty and debootstrap does it all at once.
    _DBS_FOREIGN=""
    if [ "${ISO_ARCH:-amd64}" != "$(dpkg --print-architecture)" ]; then _DBS_FOREIGN="--foreign"; fi
    # zstd: noble sets COMPRESS=zstd in initramfs.conf but minbase does not
    # install the binary; without it mkinitramfs falls back to gzip (bigger,
    # slower initrd). It must be there BEFORE the kernel, hence here.
    debootstrap $DEBOOTSTRAP_CACHE_OPT --arch="$ISO_ARCH" $_DBS_FOREIGN --variant=minbase --include=\
systemd,systemd-sysv,dbus,kmod,zstd,\
ca-certificates,curl,gnupg,\
iproute2,iputils-ping,procps,less,nano \
        "$ISO_SUITE" "$ROOTFS" "$ISO_MIRROR"
    if [ -n "$_DBS_FOREIGN" ]; then
        install -m755 "$(command -v qemu-aarch64-static)" "$ROOTFS/usr/bin/qemu-aarch64-static"
        echo -e "  ${CYAN}debootstrap --second-stage (aarch64 sous qemu-user, plus lent)...${NC}"
        chroot "$ROOTFS" /debootstrap/debootstrap --second-stage >"$WORK/debootstrap-2nd.log" 2>&1 \
```

```
            || { echo -e "${RED}debootstrap --second-stage a echoue (voir $WORK/debootstrap-2nd.log)${NC}" >&2;
             ↪  tail -20 "$WORK/debootstrap-2nd.log" >&2; exit 1; }
    fi
    echo -e "  ${GREEN}✓${NC} rootfs base $(du -sh "$ROOTFS"|cut -f1)"
fi


# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.66 /opt/GSM/osmo-operator/iso_modules/50-injection-image.sh

5577 octets, 95 lignes → 95 lignes

```
#!/bin/bash
# iso_modules/50-injection-image.sh - step 5: build .debs first, then docker cp of the image
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment:
#   OSMO_DEB_CACHE=<dir>  host cache of the build .debs (default /var/cache/osmo-debs)
#   ISO_EMBED_DEBS=1      keep the .debs in the ISO (/var/cache/osmo-debs)

# ── Step 5: the .debs OF THE DOCKER BUILD first, then the rest of the image ──
# What the image compiled is packaged (packaging/osmo-deb.sh): libosmocore and
# the osmo-*, gapk, QEMU and its tree, osmocom-bb, the gr-gsm venv, the
# firmware. They are installed into the rootfs with dpkg; only what is not
# packaged is copied from the image (scripts, configs, units, node, the repo).
# Package source: the HOST cache (/var/cache/osmo-debs, fed by build.sh), else
# the cache embedded in the image. Only those of the rootfs suite/arch are
# taken. Without packages, the full docker cp is used.
if [ "$OSMO_ISO_INHERITED" = "1" ]; then
    echo -e "${GREEN}[5/9] Injection : rootfs herite, deja fait${NC}"
else
echo -e "${GREEN}[5/9] Injection stack Osmocom (paquets .deb du build, puis image)...${NC}"
CID=$(docker create "$ISO_RUN_IMAGE" /bin/true)
ISO_DEB_HOST_CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
_iso_debs="$WORK/debs-build"; mkdir -p "$_iso_debs"
# Suite AND architecture: the host cache holds the packages of both
# architectures side by side (..._amd64.deb, ..._arm64.deb).
cp -f "$ISO_DEB_HOST_CACHE"/osmo-build-*"~${ISO_SUITE}_${ISO_ARCH}.deb" "$_iso_debs/" 2>/dev/null || true
if ! ls "$_iso_debs"/*.deb >/dev/null 2>&1; then
    docker cp "$CID:/var/cache/osmo-debs/." "$_iso_debs/" 2>/dev/null || true
    find "$_iso_debs" -name '*.deb' ! -name "*~${ISO_SUITE}_${ISO_ARCH}.deb" -delete 2>/dev/null || true
fi
# The hub only needs the STP: libosmocore, libosmo-netif, libosmo-sigtran.
if [ "$ISO_ROLE" = "interstp" ]; then
    find "$_iso_debs" -name '*.deb' ! -name 'osmo-build-libosmocore_*' \
        ! -name 'osmo-build-libosmo-netif_*' ! -name 'osmo-build-libosmo-sigtran_*' -delete 2>/dev/null || true
fi
ISO_DEBS_USED=0
_ndebs=$(find "$_iso_debs" -maxdepth 1 -name '*.deb' | wc -l)
if [ "$_ndebs" -gt 0 ]; then
    # Installed from a directory INSIDE the rootfs (dpkg reads its packages
    # under the root), removed afterwards: the packages only travel in the ISO
    # with ISO_EMBED_DEBS=1 (a zstd QEMU tree in an image that fits in RAM
    # is not a given).
    install -d "$ROOTFS/var/cache/osmo-debs"
    cp -f "$_iso_debs"/*.deb "$ROOTFS/var/cache/osmo-debs/"
    if chroot "$ROOTFS" sh -c 'dpkg -i --force-overwrite /var/cache/osmo-debs/osmo-build-*.deb' \
           > "$WORK/dpkg-debs.log" 2>&1; then
```

```
        chroot "$ROOTFS" ldconfig 2>/dev/null || true
        ISO_DEBS_USED=1
        echo -e "  ${GREEN}✓${NC} ${_ndebs} paquets du build docker poses par dpkg ($(du -sh "$_iso_debs" | cut
         ↪  -f1))"
    else
        echo -e "  ${YELLOW}⚠${NC} dpkg -i des paquets du build a echoue (voir $WORK/dpkg-debs.log) - repli sur l
         ↪  image" >&2
        tail -5 "$WORK/dpkg-debs.log" | sed 's/^/     /' >&2
    fi
    [ "${ISO_EMBED_DEBS:-0}" = "1" ] || rm -rf "$ROOTFS/var/cache/osmo-debs"
else
    echo -e "  ${CYAN}aucun paquet .deb du build pour ${ISO_SUITE} (cache ${ISO_DEB_HOST_CACHE}) - tout vient de l
     ↪  image${NC}"
fi
docker cp "$CID:/usr/local/bin/." "$ROOTFS/usr/local/bin/"  2>/dev/null||true
docker cp "$CID:/usr/local/lib/." "$ROOTFS/usr/local/lib/"  2>/dev/null||true
docker cp "$CID:/usr/local/include/." "$ROOTFS/usr/local/include/" 2>/dev/null||true
docker cp "$CID:/usr/local/sbin/." "$ROOTFS/usr/local/sbin/" 2>/dev/null||true
# /opt/GSM: everything EXCEPT the trees the packages just installed (osmocom-bb,
# qosmo, qemu-install, firmware) - and nothing at all for the hub, which does
# not read any of it (the repo itself is cloned at step 5a).
if [ "$ISO_ROLE" != "interstp" ]; then
    _excl=()
    if [ "$ISO_DEBS_USED" = "1" ]; then
        for _t in osmocom-bb qosmo c54x_exe grgsm_exe qemu-install firmware; do
            [ -d "$ROOTFS/opt/GSM/$_t" ] && _excl+=("--exclude=GSM/$_t" "--exclude=GSM/$_t/*")
        done
    fi
    mkdir -p "$ROOTFS/opt"
    docker cp "$CID:/opt/GSM" - 2>/dev/null | tar -x -C "$ROOTFS/opt" "${_excl[@]+"${_excl[@]}"}" 2>/dev/null ||
     ↪  true
fi
# Python venv (gr-gsm + bridges) expected by the bridge (grgsm_exe, pont/*.py)
# - as a package (grgsm-venv) when the cache has it, from the image otherwise.
[ -d "$ROOTFS/root/.env" ] || docker cp "$CID:/root/.env" "$ROOTFS/root/" 2>/dev/null||true
[ -d "$ROOTFS/root/.venv-qemu" ] || docker cp "$CID:/root/.venv-qemu" "$ROOTFS/root/" 2>/dev/null||true
docker cp "$CID:/opt/node"            "$ROOTFS/opt/"        2>/dev/null||true
docker cp "$CID:/etc/osmocom/."       "$ROOTFS/etc/osmocom/" 2>/dev/null||true
docker cp "$CID:/etc/asterisk/."      "$ROOTFS/etc/asterisk/" 2>/dev/null||true
for svc in osmo-bts-trx osmo-bsc osmo-msc osmo-hlr osmo-mgw osmo-stp osmo-ggsn osmo-sgsn osmo-pcu
 ↪  osmo-sip-connector; do
    docker cp "$CID:/lib/systemd/system/${svc}.service" "$ROOTFS/lib/systemd/system/" 2>/dev/null||true
done
docker rm "$CID" &>/dev/null
echo -e "  ${GREEN}✓${NC} binaires + libs + configs injectes"
fi


# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.67 /opt/GSM/osmo-operator/iso_modules/51-depot.sh

4318 octets, 81 lignes → 81 lignes

```
#!/bin/bash
# iso_modules/51-depot.sh - step 5a: up-to-date osmo-operator and tests trees, coeur.env
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment:
```

```
#   OSMO_EGPRS_REPO / OSMO_EGPRS_BRANCH   osmo-operator repo and branch to clone
#   OSMO_TESTS_REPO / OSMO_TESTS_BRANCH   tests repo and branch to clone

# ── osmo-operator: up-to-date TREE from GitHub, WITH its .git ──────────────
# The docker cp copy may be stale: the repo branch is cloned fresh (the .git is
# kept so that update.sh can fetch at boot instead of recloning, and the
# running commit is known). The clone is made NEXT TO the tree, then swapped
# in: without network, the image tree stays in place.
EGPRS_BRANCH="${OSMO_EGPRS_BRANCH:-main}"
EGPRS_REPO="${OSMO_EGPRS_REPO:-https://github.com/bbaranoff/osmo-operator}"
echo -e "${GREEN}[5a/9] Clone de osmo-operator (branche ${EGPRS_BRANCH})...${NC}"
EGPRS_TREE="$ROOTFS/opt/GSM/osmo-operator"
EGPRS_TMP="$WORK/osmo-operator-clone"
rm -rf "$EGPRS_TMP"
if [ "$OSMO_ISO_INHERITED" = "1" ] && [ -d "$EGPRS_TREE/.git" ]; then
    echo -e "  ${GREEN}✓${NC} osmo-operator : arbre du rootfs herite conserve"
elif GIT_TERMINAL_PROMPT=0 git clone --depth 1 -b "$EGPRS_BRANCH" "$EGPRS_REPO" "$EGPRS_TMP" >/dev/null 2>&1; then
    rm -rf "$EGPRS_TREE"
    mkdir -p "$ROOTFS/opt/GSM"
    mv "$EGPRS_TMP" "$EGPRS_TREE"
    echo -e "  ${GREEN}✓${NC} osmo-operator clone (${EGPRS_BRANCH}, .git conserve) - $(git -C "$EGPRS_TREE" log
     ↪  -1 --format='%h %s')"
else
    rm -rf "$EGPRS_TMP"
    if [ -d "$EGPRS_TREE" ]; then
        echo -e "  ${YELLOW}⚠${NC} osmo-operator : clone impossible (reseau ?) - arbre de l'image conserve" >&2
    else
        echo -e "  ${RED}✗${NC} osmo-operator : clone impossible ET absent de l'image" >&2
    fi
fi

# ── tests: /opt/GSM/tests from GitHub (bbaranoff), WITH its .git ───────────
# Same logic as osmo-operator: clone next to the tree, then swap, so that a
# missing network leaves the existing image tree in place.
TESTS_BRANCH="${OSMO_TESTS_BRANCH:-main}"
TESTS_REPO="${OSMO_TESTS_REPO:-https://github.com/bbaranoff/tests}"
TESTS_TREE="$ROOTFS/opt/GSM/tests"
TESTS_TMP="$WORK/tests-clone"
echo -e "${GREEN}[5a/9] Clone de tests (branche ${TESTS_BRANCH})...${NC}"
rm -rf "$TESTS_TMP"
if [ "$OSMO_ISO_INHERITED" = "1" ] && [ -d "$TESTS_TREE/.git" ]; then
    echo -e "  ${GREEN}✓${NC} tests : arbre du rootfs herite conserve"
elif GIT_TERMINAL_PROMPT=0 git clone --depth 1 -b "$TESTS_BRANCH" "$TESTS_REPO" "$TESTS_TMP" >/dev/null 2>&1; then
    rm -rf "$TESTS_TREE"
    mkdir -p "$ROOTFS/opt/GSM"
    mv "$TESTS_TMP" "$TESTS_TREE"
    echo -e "  ${GREEN}✓${NC} tests clone (${TESTS_BRANCH}, .git conserve) - $(git -C "$TESTS_TREE" log -1
     ↪  --format='%h %s')"
else
    rm -rf "$TESTS_TMP"
    if [ -d "$TESTS_TREE" ]; then
        echo -e "  ${YELLOW}⚠${NC} tests : clone impossible (reseau ?) - arbre de l'image conserve" >&2
    else
        echo -e "  ${YELLOW}⚠${NC} tests : clone impossible ET absent de l'image" >&2
    fi
fi

# ── HLR feed: align N_MS with the number of embedded MS ────────────────────
# run_modules/21-abonnes-hlr.sh falls back to N_MS=1: without this file a
# SINGLE subscriber would be provisioned while the ISO declares ISO_N_MS, and
# the other MS would be refused ("IMSI unknown in HLR").
# NOT in /opt/GSM/osmo-operator/environment: that file is not in git and the
# reset --hard of osmo-update would remove it. /etc/osmocom belongs to no repo.
mkdir -p "$ROOTFS/etc/osmocom"
cat > "$ROOTFS/etc/osmocom/coeur.env" <<COEUR
# coeur.env - generated by build-iso.sh. Aligns the number of subscribers
```

```
# provisioned in the HLR with the number of MS embedded in the ISO (ISO_N_MS).
: "\${N_MS:=$ISO_N_MS}"
: "\${OPERATOR_ID:=1}"
COEUR
echo -e "  ${GREEN}✓${NC} coeur.env : ${CYAN}N_MS=${ISO_N_MS}${NC} (/etc/osmocom)"

# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.68 /opt/GSM/osmo-operator/iso_modules/51-softsim.sh

4181 octets, 84 lignes → 84 lignes

```
#!/bin/bash
# iso_modules/51-softsim.sh - step 5a ter: softSIM (SIM over SAP, Osmocom)
# Sourced by build-iso.sh in numeric order. A `return` at the top = nothing to do.
#
# softSIM (Ruby): cloned from gitea.osmocom.org into /opt/GSM/softsim, with its .git.
# Ruby 3.2+: File.exists? and Fixnum no longer exist; the shim tools/softsim/ruby_compat.rb
# is installed in /usr/local/share/softsim/ (usage: ruby -r<shim> demo_server.rb ...).
# Ruby and ruby-libxml are in PKGS of 80-chroot.sh.
#
# Environment:
#   OSMO_ISO_SOFTSIM=0     disable (--no-softsim)
#   OSMO_SOFTSIM_REPO      git URL to clone
#   OSMO_SOFTSIM_LOCAL     local reference copy on the host (default /opt/GSM/softsim)
#   OSMO_SIM_IMSI          IMSI of the generated sim-op1.xml (default 001010001000001)
if [ "${OSMO_ISO_SOFTSIM:-1}" != "1" ]; then
    echo -e "${YELLOW}[5a/9] softSIM desactive (--no-softsim)${NC}"
    return 0
fi

# The inter-STP hub has no SIM to emulate (no MS): 50-injection already skips
# its /opt/GSM, so softsim is not cloned for it.
if [ "$ISO_ROLE" = "interstp" ]; then
    echo -e "${CYAN}[5a/9] Role inter-STP : pas de softSIM (hub M3UA)${NC}"
    return 0
fi

SOFTSIM_REPO="${OSMO_SOFTSIM_REPO:-https://gitea.osmocom.org/sim-card/softsim.git}"
SOFTSIM_LOCAL="${OSMO_SOFTSIM_LOCAL:-/opt/GSM/softsim}"
SOFTSIM_TREE="$ROOTFS/opt/GSM/softsim"
SOFTSIM_TMP="$WORK/softsim-clone"
echo -e "${GREEN}[5a/9] softSIM (SIM via SAP)...${NC}"
rm -rf "$SOFTSIM_TMP"
if [ "$OSMO_ISO_INHERITED" = "1" ] && [ -d "$SOFTSIM_TREE/.git" ]; then
    echo -e "  ${GREEN}✓${NC} softSIM : arbre du rootfs herite conserve"
elif GIT_TERMINAL_PROMPT=0 git clone --depth 1 "$SOFTSIM_REPO" "$SOFTSIM_TMP" >/dev/null 2>&1; then
    rm -rf "$SOFTSIM_TREE"
    mkdir -p "$ROOTFS/opt/GSM"
    mv "$SOFTSIM_TMP" "$SOFTSIM_TREE"
    echo -e "  ${GREEN}✓${NC} softSIM clone ($(git -C "$SOFTSIM_TREE" log -1 --format='%h %s'))"
else
    rm -rf "$SOFTSIM_TMP"
    echo -e "  ${YELLOW}⚠${NC} softSIM : clone impossible (reseau ?)" >&2
fi
# Local reference copy, if the host has one (same as 51-shannon.sh)
if [ -d "$SOFTSIM_LOCAL" ] && [ "$SOFTSIM_LOCAL" != "$SOFTSIM_TREE" ]; then
    mkdir -p "$SOFTSIM_TREE"
    cp -a "$SOFTSIM_LOCAL/." "$SOFTSIM_TREE/" 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} softSIM : copie locale de $SOFTSIM_LOCAL posee" \
        || echo -e "  ${YELLOW}⚠${NC} softSIM : copie locale incomplete" >&2
```

```
fi

# COMP128v1 for the SIM (HLR Ki): lib/comp128.rb + patches of simos_server.rb.
# Each patch is applied only once (an already patched local tree keeps it).
if [ -d "$SOFTSIM_TREE/src" ] && [ -f "$DIR/tools/softsim/comp128.rb" ]; then
    install -Dm644 "$DIR/tools/softsim/comp128.rb" "$SOFTSIM_TREE/src/lib/comp128.rb"
    # ALL patches of tools/softsim (0001 COMP128v1, 0002 next client, 0003
    # timestamped log, 0004 idempotent pcsc disconnect), as tools/osmo-sim-install.sh does.
    for _sp in "$DIR"/tools/softsim/*.patch; do
        [ -f "$_sp" ] || continue
        if git -C "$SOFTSIM_TREE" apply --check -R "$_sp" 2>/dev/null; then
            echo -e "  ${GREEN}✓${NC} softSIM : $(basename "$_sp") deja present"
        elif git -C "$SOFTSIM_TREE" apply "$_sp" 2>/dev/null; then
            echo -e "  ${GREEN}✓${NC} softSIM : $(basename "$_sp") applique"
        else
            echo -e "  ${YELLOW}⚠${NC} softSIM : $(basename "$_sp") ne s applique pas" >&2
        fi
    done
    unset _sp
    # SIM of the HLR subscriber (IMSI 001010001000001): the Ki comes from SOFTSIM_KI
    if python3 "$DIR/tools/softsim/make_sim_op1.py" "$SOFTSIM_TREE/src/sim.xml" \
            "$SOFTSIM_TREE/src/sim-op1.xml" --imsi "${OSMO_SIM_IMSI:-001010001000001}" >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} softSIM : sim-op1.xml (IMSI ${OSMO_SIM_IMSI:-001010001000001})"
    else
        echo -e "  ${YELLOW}⚠${NC} softSIM : sim-op1.xml non genere" >&2
    fi
fi

# Ruby 3.2+ shim: from the build repo
if [ -f "$DIR/tools/softsim/ruby_compat.rb" ]; then
    install -Dm644 "$DIR/tools/softsim/ruby_compat.rb" "$ROOTFS/usr/local/share/softsim/ruby_compat.rb"
    echo -e "  ${GREEN}✓${NC} shim Ruby : /usr/local/share/softsim/ruby_compat.rb"
fi

true
```

2.69 /opt/GSM/osmo-operator/iso_modules/52-qemu.sh

26680 octets, 454 lignes → 454 lignes

```
#!/bin/bash
# iso_modules/52-qemu.sh - steps 5b/5c/5d: QEMU, firmware, toast, launchers, configs
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment:
#   OSMO_QEMU_SRC=<dir>        use this local qosmo tree instead of the image's
#   OSMO_QEMU_BUILD=<dir>      host QEMU build dir (default: <qosmo>/build)
#   OSMO_C54X_SRC=<dir>        local c54x_exe tree to embed
#   OSMO_GRGSM_EXE_SRC=<dir>   local grgsm_exe tree to embed
#   OSMO_DSP_ROM_DIR=<dir>     where to find the calypso_dsp.*.bin ROM dumps

# ── qosmo: full tree + installed binary ──────────────────────────────
# The ISO needs BOTH:
#   - the qosmo tree (run.sh, run_modules/, environnement/): it IS the qemu
#     mode of start-native.sh;
#   - the qemu-system-arm binary, found by paths.env as build/qemu-system-arm.
QEMU_BUILD_LOCAL="${OSMO_QEMU_BUILD:-${OSMO_QEMU_SRC:-/opt/GSM/qosmo}/build}"
# --arm: a host QEMU build is an x86 binary, useless in an arm64 rootfs.
# Only the (arm64) binary from the image counts.
[ "${ISO_ARCH:-amd64}" = "amd64" ] || QEMU_BUILD_LOCAL="/nonexistent/arm64-pas-de-build-hote"
echo -e "${GREEN}[5b/9] Installation QEMU (artefacts seuls, depuis ${QEMU_BUILD_LOCAL})...${NC}"
# ── qosmo tree: embedded WHOLE, .git and build/ included ─────────────
```

```
#   - .git lets update.sh do an incremental git fetch instead of a reclone;
#   - build/ holds the compiled qemu-system-arm that environnement/paths.env
#     expects at $QEMU_TREE/build/qemu-system-arm, so the tree works offline.
# Known limitation: this adds ~1.6 GB to the squashfs; watch it for toram boots.
# OSMO_QEMU_SRC=<dir> replaces the image's tree (cloned from GitHub by the
# Dockerfile) with a local one, so unpushed fixes reach the ISO. Without it,
# the image tree is used.
QSRC="$ROOTFS/opt/GSM/qosmo"
if [ -n "${OSMO_QEMU_SRC:-}" ] && [ -d "$OSMO_QEMU_SRC" ]; then
    rm -rf "$QSRC"
    mkdir -p "$ROOTFS/opt/GSM"
    cp -a "$OSMO_QEMU_SRC" "$QSRC"
    echo -e "  ${GREEN}✓${NC} qosmo FORCE depuis ${CYAN}${OSMO_QEMU_SRC}${NC} (OSMO_QEMU_SRC) ($(du -sh "$QSRC" |
     ↪  cut -f1))"
elif [ -d "$QSRC" ]; then
    echo -e "  ${GREEN}✓${NC} qosmo conserve ENTIER ($(du -sh "$QSRC" | cut -f1), .git + build/ compris)"
else
    # Not in the image: take the host tree, also whole.
    QSRC_HOST="${OSMO_QEMU_SRC:-/opt/GSM/qosmo}"
    if [ -d "$QSRC_HOST" ]; then
        mkdir -p "$ROOTFS/opt/GSM"
        cp -a "$QSRC_HOST" "$QSRC"
        echo -e "  ${GREEN}✓${NC} qosmo repris de l'hote ${CYAN}${QSRC_HOST}${NC} ($(du -sh "$QSRC" | cut -f1))"
    else
        echo -e "  ${YELLOW}!${NC} qosmo introuvable (ni image, ni hote) - l'ISO n'aura pas le mode qemu" >&2
    fi
fi

# ── Remove the obsolete forks from the rootfs ──────────────────────────
# The docker image still clones qosmo-grgsm and qosmo-dsp, and
# 50-injection-image.sh copies all of /opt/GSM and /usr/local/bin: drop those
# trees, the qemu-dsp-install prefix and their launchers. A dangling
# /opt/GSM/calypso_dsp.txt link is removed; it is set again below to c54x_exe/rom.
for _old in qosmo-grgsm qosmo-dsp qemu-dsp-install; do
    [ -e "$ROOTFS/opt/GSM/$_old" ] || continue
    rm -rf "$ROOTFS/opt/GSM/$_old"
    echo -e "  ${GREEN}✓${NC} ancien arbre ${CYAN}/opt/GSM/$_old${NC} retire du rootfs"
done
rm -f "$ROOTFS"/usr/local/bin/qosmo-grgsm "$ROOTFS"/usr/local/bin/qosmo-dsp \
      "$ROOTFS"/usr/local/bin/qosmo-grgsm-launch "$ROOTFS"/usr/local/bin/qosmo-dsp-launch
[ -L "$ROOTFS/opt/GSM/calypso_dsp.txt" ] && [ ! -e "$ROOTFS/opt/GSM/calypso_dsp.txt" ] \
    && rm -f "$ROOTFS/opt/GSM/calypso_dsp.txt"

# ── c54x_exe and grgsm_exe: the two layer-1 implementations outside QEMU ────
#   c54x_exe   TMS320C54x DSP on the TI mask ROM (start-native.sh --dsp,
#              BANC_DSP = /opt/GSM/c54x_exe/run.sh)
#   grgsm_exe  gr-gsm layer 1 outside QEMU (bridge: /usr/local/bin/grgsm_exe)
# OSMO_C54X_SRC / OSMO_GRGSM_EXE_SRC force a local tree, like OSMO_QEMU_SRC.
# Not fatal: without c54x_exe only --dsp is missing.
# Known limitation: c54x_exe is built with -march=native, so the host binary may
# not run on the machine booting the ISO (run make -C /opt/GSM/c54x_exe there).
for _e in c54x_exe grgsm_exe; do
    case "$_e" in
        c54x_exe)  _esrc="${OSMO_C54X_SRC:-}" ;;
        grgsm_exe) _esrc="${OSMO_GRGSM_EXE_SRC:-}" ;;
    esac
    _edst="$ROOTFS/opt/GSM/$_e"
    if [ -n "$_esrc" ] && [ -d "$_esrc" ]; then
        rm -rf "$_edst"; mkdir -p "$ROOTFS/opt/GSM"
        cp -a "$_esrc" "$_edst"
        echo -e "  ${GREEN}✓${NC} $_e FORCE depuis ${CYAN}${_esrc}${NC} ($(du -sh "$_edst" | cut -f1))"
    elif [ -d "$_edst" ]; then
        echo -e "  ${GREEN}✓${NC} $_e : arbre de l image conserve ($(du -sh "$_edst" | cut -f1))"
    elif [ -d "/opt/GSM/$_e" ]; then
        mkdir -p "$ROOTFS/opt/GSM"
        cp -a "/opt/GSM/$_e" "$_edst"
```

```
        echo -e "  ${GREEN}✓${NC} $_e repris de l hote ${CYAN}/opt/GSM/$_e${NC} ($(du -sh "$_edst" | cut -f1))"
    else
        echo -e "  ${YELLOW}!${NC} $_e introuvable (ni image, ni hote)" >&2
        continue
    fi
    if [ -x "$_edst/$_e" ]; then
        echo -e "  ${GREEN}✓${NC} $_e : binaire ${CYAN}/opt/GSM/$_e/$_e${NC} present"
    else
        echo -e "  ${YELLOW}!${NC} $_e : binaire ABSENT - make -C /opt/GSM/$_e avant de graver" >&2
    fi
    # Small bash launchers /usr/local/bin/<exe> from the host.
    if [ -x "/usr/local/bin/$_e" ] && [ ! -e "$ROOTFS/usr/local/bin/$_e" ]; then
        install -Dm755 "/usr/local/bin/$_e" "$ROOTFS/usr/local/bin/$_e"
        echo -e "  ${GREEN}✓${NC} lanceur ${CYAN}/usr/local/bin/$_e${NC} (repris de l'hote)"
    fi
done

# ── TMS320C54x DSP ROMs: required by --dsp ──────────────────────────────
# Seven silicon dumps (PROM0..3, DROM, PDROM, Registers), ~330 KB, read by
# c54x_exe from its --rom-dir (default /opt/GSM). They are in no repository:
# taken from the host, $OSMO_DSP_ROM_DIR (default /opt/GSM) then
# /opt/GSM/c54x_exe/rom (untracked backup copy, with calypso_dsp.txt). Not fatal.
_ROM_SRC="${OSMO_DSP_ROM_DIR:-/opt/GSM}"
_ROM_SECOURS=/opt/GSM/c54x_exe/rom
_rom_ok=0; _rom_miss=""
for _r in PROM0 PROM1 PROM2 PROM3 DROM PDROM Registers; do
    if [ -f "$_ROM_SRC/calypso_dsp.$_r.bin" ]; then
        install -Dm644 "$_ROM_SRC/calypso_dsp.$_r.bin" "$ROOTFS/opt/GSM/calypso_dsp.$_r.bin"
        _rom_ok=$((_rom_ok + 1))
    elif [ -f "$_ROM_SECOURS/calypso_dsp.$_r.bin" ]; then
        install -Dm644 "$_ROM_SECOURS/calypso_dsp.$_r.bin" "$ROOTFS/opt/GSM/calypso_dsp.$_r.bin"
        _rom_ok=$((_rom_ok + 1))
    elif [ -f "$ROOTFS/opt/GSM/calypso_dsp.$_r.bin" ]; then
        _rom_ok=$((_rom_ok + 1))
    else
        _rom_miss="$_rom_miss $_r"
    fi
done
# calypso_dsp.txt (ROM listing): /opt/GSM/calypso_dsp.txt points to the copy
# in c54x_exe/rom.
if [ -f "$ROOTFS/opt/GSM/c54x_exe/rom/calypso_dsp.txt" ]; then
    ln -sfn /opt/GSM/c54x_exe/rom/calypso_dsp.txt "$ROOTFS/opt/GSM/calypso_dsp.txt"
elif [ -f "$_ROM_SECOURS/calypso_dsp.txt" ]; then
    install -Dm644 "$_ROM_SECOURS/calypso_dsp.txt" "$ROOTFS/opt/GSM/calypso_dsp.txt"
fi
if [ -z "$_rom_miss" ]; then
    echo -e "  ${GREEN}✓${NC} ROMs DSP : ${CYAN}/opt/GSM/calypso_dsp.{PROM0..3,DROM,PDROM,Registers}.bin${NC}
     ↪  (7/7)"
elif [ -d "$ROOTFS/opt/GSM/c54x_exe" ]; then
    echo -e "  ${YELLOW}!${NC} ROMs DSP incompletes (${_rom_ok}/7, manquent :${_rom_miss}) -
     ↪  OSMO_DSP_ROM_DIR=/chemin ; --dsp ne demarrera pas" >&2
fi

# ── Calypso firmware: /opt/GSM/firmware only ────────────────────────────
# environnement/paths.env (and local.env) of qosmo only know
# $GSM_ROOT/firmware; do not add a firmware link inside the qosmo tree.
# Just check that the firmware is in the rootfs: without it the ISO boots but
# the MS does not start.
FW_ELF="board/compal_e88/layer1.highram.elf"
if [ -e "$ROOTFS/opt/GSM/firmware/$FW_ELF" ]; then
    echo -e "  ${GREEN}✓${NC} firmware : ${CYAN}/opt/GSM/firmware/${FW_ELF}${NC} (source unique ; FIRMWARE_ELF
     ↪  resolu)"
else
    echo -e "  ${YELLOW}!${NC} /opt/GSM/firmware/${FW_ELF} absent du rootfs - FIRMWARE_ELF restera non resolu" >&2
fi
# The firmware is interchangeable (compal_e86, gta0x, ...): the launchers read
```

```
# l1s/last_rach from the ELF selected by FIRMWARE_ELF. List the embedded boards
# so that FIRMWARE_ELF=.../board/X/layer1.highram.elf is not guesswork.
_boards=""
for _b in "$ROOTFS"/opt/GSM/firmware/board/*/layer1.highram.elf; do
    [ -f "$_b" ] || continue
    _boards="$_boards $(basename "$(dirname "$_b")")"
done
[ -n "$_boards" ] && echo -e "  ${GREEN}✓${NC} boards layer1 embarques :${CYAN}${_boards}${NC}"

# ── TI TAS2781 audio firmware (Lenovo Legion 7 amplifier) ──────────────
# The snd_soc_tas2781 driver stays silent until its firmware is in
# /lib/firmware. Installed into the rootfs (not the build host). Not fatal:
# without it only the Legion 7 audio is missing.
_TAS_URL="https://github.com/bbaranoff/sound_firmware_lenovo_legion_7/raw/refs/heads/main/TIAS2781RCA2.bin"
install -d "$ROOTFS/lib/firmware"
if [ "${ISO_ARCH:-amd64}" != "amd64" ]; then
    echo -e "  ${CYAN}·${NC} firmware TAS2781 (Legion 7, x86) : sans objet sur le Pi"
elif wget -qO "$ROOTFS/lib/firmware/TIAS2781RCA2.bin" "$_TAS_URL"; then
    echo -e "  ${GREEN}✓${NC} firmware audio : ${CYAN}/lib/firmware/TIAS2781RCA2.bin${NC} (TAS2781, Legion 7)"
else
    rm -f "$ROOTFS/lib/firmware/TIAS2781RCA2.bin"
    echo -e "  ${YELLOW}!${NC} TIAS2781RCA2.bin non telecharge (reseau ?) - audio Legion 7 muet" >&2
fi

# ── toast: GSM 06.10 codec (quut.com), not shipped by libgsm1 ──────────
# libgsm1 provides the library, not toast/untoast/tcat. The sources are built
# under $ROOTFS/opt/GSM and the binaries installed into $ROOTFS/usr/local/bin.
# Built on the build host, so only when host and ISO architectures match
# (otherwise in the chroot, 82-arm-natif). Not fatal: no network or toolchain
# means no toast.
_GSM_VER=gsm-1.0.24
_GSM_DIR=gsm-1.0-pl24  # the tarball unpacks under this name
_GSM_URL="https://www.quut.com/gsm/${_GSM_VER}.tar.gz"
if [ -x "$ROOTFS/usr/local/bin/toast" ]; then
    echo -e "  ${GREEN}✓${NC} toast deja dans le rootfs (image ?)"
elif [ "${ISO_ARCH:-amd64}" != "$(dpkg --print-architecture)" ]; then
    echo -e "  ${CYAN}·${NC} toast : compile DANS le chroot ${ISO_ARCH} plus loin (82-arm-natif), pas sur l hote"
elif ! command -v make >/dev/null 2>&1 || ! command -v cc >/dev/null 2>&1; then
    echo -e "  ${YELLOW}!${NC} toast non compile : gcc/make absents de l hote de build" >&2
else
    install -d "$ROOTFS/opt/GSM" "$ROOTFS/usr/local/bin"
    if ( cd "$ROOTFS/opt/GSM" \
         && wget -qO "${_GSM_VER}.tar.gz" "$_GSM_URL" \
         && tar xzf "${_GSM_VER}.tar.gz" \
         && cd "$_GSM_DIR" \
         && { make >/dev/null 2>&1 || true; } \
         && { [ -x bin/toast ] || make toast >/dev/null 2>&1 || true; } \
         && [ -x bin/toast ] ); then
        for _b in toast untoast tcat; do
            [ -e "$ROOTFS/opt/GSM/${_GSM_DIR}/bin/$_b" ] \
                && install -m755 "$ROOTFS/opt/GSM/${_GSM_DIR}/bin/$_b" "$ROOTFS/usr/local/bin/$_b"
        done
        echo -e "  ${GREEN}✓${NC} toast : ${CYAN}/usr/local/bin/toast${NC} (sources : /opt/GSM/${_GSM_DIR})"
    else
        echo -e "  ${YELLOW}!${NC} compilation de toast echouee (reseau ?) - ISO sans toast" >&2
    fi
fi

# ── QEMU datadir: link required by QEMU path relocation ────────────────
# QEMU does not use CONFIG_QEMU_FIRMWAREPATH as-is (get_relocated_path()): it
# makes it relative to its build-time bindir and applies it to the directory
# the binary actually runs from:
#   built with    prefix=/opt/GSM/qemu-install
#   run from      /opt/GSM/qosmo/build/qemu-system-arm   (run.sh -> QEMU_BIN)
#   QEMU looks in /opt/GSM/qosmo/share/qemu
# Without the <exec_dir>/../share/qemu link QEMU dies with
```

```
# "could not read keymap file: 'en-us'"; <exec_dir>/pc-bios does not help.
QINST="$ROOTFS/opt/GSM/qemu-install/share/qemu"
if [ -d "$QSRC" ] && [ -d "$QINST/keymaps" ]; then
    if [ -e "$QSRC/share/qemu/keymaps/en-us" ]; then
        echo -e "  ${GREEN}✓${NC} datadir QEMU : ${CYAN}/opt/GSM/qosmo/share/qemu${NC} deja resolu"
    else
        mkdir -p "$QSRC/share"
        rm -rf "$QSRC/share/qemu"
        ln -sfn /opt/GSM/qemu-install/share/qemu "$QSRC/share/qemu"
        echo -e "  ${GREEN}✓${NC} datadir QEMU : ${CYAN}/opt/GSM/qosmo/share/qemu${NC} ->
         ↪  /opt/GSM/qemu-install/share/qemu (keymap 'en-us' resolu)"
    fi
elif [ -d "$QSRC" ] && [ "$ISO_ROLE" != "interstp" ]; then
    echo -e "  ${YELLOW}!${NC} /opt/GSM/qemu-install/share/qemu/keymaps absent - QEMU mourra sur 'could not read
     ↪  keymap file'" >&2
fi

# The binary may already come from the image (docker cp of /usr/local/bin
# above): a host build is not required. Fatal only if there is no binary at
# all, neither on the host nor in the image.
# pc-bios is not needed: the Calypso machine loads its own ROM, and copying
# it would add 303 MB to an ISO that runs from RAM.
ROOTFS_QEMU="$ROOTFS/usr/local/bin/qemu-system-arm"
if [ ! -x "$QEMU_BUILD_LOCAL/qemu-system-arm" ] \
   && [ ! -x "$ROOTFS_QEMU" ] && [ "$ISO_ROLE" != "interstp" ]; then
    echo -e "${RED}${BOLD}[5b/9] qemu-system-arm introuvable${NC}" >&2
    echo -e "  ${YELLOW}Ni build local : ${QEMU_BUILD_LOCAL}/qemu-system-arm${NC}" >&2
    echo -e "  ${YELLOW}Ni binaire venu de l'image : ${ROOTFS_QEMU}${NC}" >&2
    echo -e "  ${YELLOW}L'image d'operateur emule le Calypso : sans ce binaire elle n'a pas de MS.${NC}" >&2
    echo -e "  ${YELLOW}Trois issues : compiler qosmo, pointer OSMO_QEMU_BUILD sur un build${NC}" >&2
    echo -e "  ${YELLOW}existant, ou reconstruire l'image docker qui, elle, porte le binaire.${NC}" >&2
    exit 1
fi
if [ ! -x "$QEMU_BUILD_LOCAL/qemu-system-arm" ] && [ -x "$ROOTFS_QEMU" ]; then
    # Host strip: x86 only (host binutils cannot read aarch64; the arm64 binary
    # is kept as-is, with symbols).
    [ "${ISO_ARCH:-amd64}" = "$(dpkg --print-architecture)" ] && strip --strip-unneeded "$ROOTFS_QEMU" 2>/dev/null
     ↪  || true
    echo -e "  ${GREEN}✓${NC} qemu-system-arm repris de l'image ($(du -h "$ROOTFS_QEMU" | cut -f1)), pas de build
     ↪  hote necessaire"
elif [ -x "$QEMU_BUILD_LOCAL/qemu-system-arm" ]; then
    qpfx="$(sed -n 's/^prefix=//p' "$QEMU_BUILD_LOCAL/config-host.mak" 2>/dev/null)"
    qpfx="${qpfx:-/usr/local}"

    if DESTDIR="$ROOTFS" ninja -C "$QEMU_BUILD_LOCAL" install >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} qemu installe dans ${ROOTFS}${qpfx} (ninja install, pas de sources)"
    else
        # Fallback: binary + strictly required firmware/keymaps
        install -Dm755 "$QEMU_BUILD_LOCAL/qemu-system-arm" "$ROOTFS$qpfx/bin/qemu-system-arm"
        install -d "$ROOTFS$qpfx/share/qemu"
        cp -a "$QEMU_BUILD_LOCAL/pc-bios/." "$ROOTFS$qpfx/share/qemu/" 2>/dev/null || true
        echo -e "  ${GREEN}✓${NC} qemu-system-arm + pc-bios copies dans ${ROOTFS}${qpfx} (repli manuel)"
    fi
    strip --strip-unneeded "$ROOTFS$qpfx/bin/qemu-system-arm" 2>/dev/null || true
else
    # Only the hub gets here: it just routes M3UA and has no MS to emulate.
    # For the operator role the check above already aborted the build.
    echo -e "  ${CYAN}Role inter-STP : pas de QEMU (aucun MS a emuler)${NC}"
fi
# ── qosmo C launcher: called by 40-qemu.sh ──────────────────────────────
# tools/qosmo-launch/qosmo-launch.c is built in its directory (make) and
# installed as /usr/local/bin/<fork>. Without it run.sh calls qemu-system-arm
# directly: the ISO works, but without stable pty links nor network/socket
# options. Built on the host (plain C, libc only), else the binary already
# installed on the host is reused. Never fatal.
if [ "$ISO_ROLE" != "interstp" ]; then
```

```
    for fork in qosmo; do
        lsrc=""
        for c in "/opt/GSM/$fork/tools/qosmo-launch" "$ROOTFS/opt/GSM/$fork/tools/qosmo-launch"; do
            [ -f "$c/qosmo-launch.c" ] && { lsrc="$c"; break; }
        done
        if [ "${ISO_ARCH:-amd64}" != "$(dpkg --print-architecture)" ]; then
            # Cross build: neither host gcc nor host binary (both x86). The launcher is
            # built in the chroot (82-arm-natif) unless it came with the image.
            if [ -x "$ROOTFS/usr/local/bin/$fork" ]; then
               echo -e "  ${GREEN}✓${NC} lanceur ${CYAN}/usr/local/bin/$fork${NC} (deja dans l'image ${ISO_ARCH})"
            else
                echo -e "  ${CYAN}·${NC} lanceur $fork : compile dans le chroot ${ISO_ARCH} plus loin"
            fi
        elif [ -n "$lsrc" ] && command -v gcc >/dev/null 2>&1 \
           && make -s -C "$lsrc" ALIAS="$fork" TREE="/opt/GSM/$fork" "$fork" >/dev/null 2>&1 && [ -x "$lsrc/$fork"
            ↪  ]; then
            install -Dm755 "$lsrc/$fork" "$ROOTFS/usr/local/bin/$fork"
            echo -e "  ${GREEN}✓${NC} lanceur ${CYAN}/usr/local/bin/$fork${NC} (compile depuis $lsrc)"
        elif [ -x "/usr/local/bin/$fork" ]; then
            install -Dm755 "/usr/local/bin/$fork" "$ROOTFS/usr/local/bin/$fork"
            echo -e "  ${GREEN}✓${NC} lanceur ${CYAN}/usr/local/bin/$fork${NC} (repris de l'hote)"
        elif [ -x "$ROOTFS/usr/local/bin/$fork" ]; then
            echo -e "  ${GREEN}✓${NC} lanceur ${CYAN}/usr/local/bin/$fork${NC} (deja dans l'image)"
        else
            echo -e "  ${YELLOW}!${NC} lanceur $fork absent (ni source, ni binaire) : run.sh appellera
             ↪  qemu-system-arm directement" >&2
        fi
        # ── gdb console over telnet (44-gdb-telnet.sh): the cmd.gdb panel ─────
        # tools/gdb-telnet.py serves `telnet localhost 44444` -> gdb-multiarch on the
        # ARM gdbstub; it sources tools/cmd.gdb, GENERATED by tools/gdb_cmd.sh
        # (osmocom commands: dsp, sb, tasks, fake_sb, trace_frames, help_osmo...).
        # Regenerated in the embedded tree to match the shipped gdb_cmd.sh.
        # gdb-multiarch and telnet are in PKGS (operator role).
        if [ -f "$ROOTFS/opt/GSM/$fork/tools/gdb_cmd.sh" ]; then
            # GDB_CMD_HERE: the path cmd.gdb will have INSIDE THE IMAGE, not the build
            # rootfs path (otherwise `source /var/tmp/iso-build-NNN/rootfs/...`).
            if (cd "$ROOTFS/opt/GSM/$fork/tools" && GDB_CMD_HERE="/opt/GSM/$fork/tools" bash ./gdb_cmd.sh
             ↪  >/dev/null 2>&1) \
               && [ -s "$ROOTFS/opt/GSM/$fork/tools/cmd.gdb" ]; then
                echo -e "  ${GREEN}✓${NC} gdb telnet ${CYAN}$fork${NC} : tools/cmd.gdb genere ($(grep -c '^define
                 ↪  ' "$ROOTFS/opt/GSM/$fork/tools/cmd.gdb") commandes), serveur 44-gdb-telnet.sh"
            else
                echo -e "  ${YELLOW}!${NC} $fork : tools/gdb_cmd.sh n'a pas produit cmd.gdb - la console telnet
                 ↪  marchera sans le panneau" >&2
            fi
        fi
    done
fi
# ── QEMU_BIN in the tree: link ONLY if the binary is missing ──────────────
# environnement/paths.env resolves QEMU_BIN to $QEMU_TREE/build/qemu-system-arm.
# If the embedded tree already has it (build/ shipped whole), touch nothing:
# an "ln -sf" over it would replace the compiled binary with a link.
# Otherwise (tree without build/) link it to /usr/local/bin, or run.sh stops
# at its first module with "[FAIL] Prerequisite checks (... QEMU_BIN)".
# osmo-qemu-link.service (step [6/9]) applies the same rule at every boot.
if [ -d "$QSRC" ] && [ ! -e "$QSRC/build/qemu-system-arm" ]; then
    qbin=""
    for c in "$ROOTFS/usr/local/bin/qemu-system-arm" \
             "$ROOTFS${qpfx:-/usr/local}/bin/qemu-system-arm"; do
        [ -x "$c" ] && { qbin="${c#$ROOTFS}"; break; }
    done
    if [ -n "$qbin" ]; then
        mkdir -p "$QSRC/build"
        ln -sfn "$qbin" "$QSRC/build/qemu-system-arm"
        echo -e "  ${GREEN}✓${NC} QEMU_BIN : ${CYAN}/opt/GSM/qosmo/build/qemu-system-arm${NC} ->
         ↪  ${CYAN}${qbin}${NC}"
```

```
    elif [ "$ISO_ROLE" != "interstp" ]; then
        echo -e "  ${YELLOW}!${NC} binaire QEMU introuvable dans le rootfs - QEMU_BIN restera non resolu" >&2
    fi
elif [ -e "$QSRC/build/qemu-system-arm" ]; then
    echo -e "  ${GREEN}✓${NC} QEMU_BIN : ${CYAN}/opt/GSM/qosmo/build/qemu-system-arm${NC} (binaire compile de
     ↪  l'arbre)"
fi

# ── QEMU keymaps: required for the machine to start ──────────────────────
# Unlike pc-bios (303 MB of ROMs the Calypso machine never opens), keymaps are
# read by the UI at init, Calypso machine included. Without
# $prefix/share/qemu/keymaps, qemu-system-arm prints
#     qemu-system-arm: could not read keymap file: 'en-us'
# and exits before the first cycle (start-native.sh: "[FAIL] Calypso emulator
# (QEMU) (started but never ready)"). So copy the keymaps only (~917 KB).
if [ "$ISO_ROLE" != "interstp" ]; then
    if [ -d "$ROOTFS/usr/local/share/qemu/keymaps" ]; then
        echo -e "  ${GREEN}✓${NC} keymaps QEMU deja en place (${CYAN}/usr/local/share/qemu/keymaps${NC})"
    else
        qkm=""
        for c in "$QEMU_BUILD_LOCAL/pc-bios/keymaps" \
                 "$QSRC/pc-bios/keymaps" \
                 "$ROOTFS/opt/GSM/qemu-install/share/qemu/keymaps" \
                 "$ROOTFS/usr/share/qemu/keymaps" \
                 /usr/local/share/qemu/keymaps \
                 /usr/share/qemu/keymaps; do
            [ -d "$c" ] && { qkm="$c"; break; }
        done
        if [ -n "$qkm" ]; then
            mkdir -p "$ROOTFS/usr/local/share/qemu"
            cp -a "$qkm" "$ROOTFS/usr/local/share/qemu/"
            echo -e "  ${GREEN}✓${NC} keymaps QEMU ($(du -sh "$ROOTFS/usr/local/share/qemu/keymaps" | cut -f1))
             ↪  copiees depuis ${CYAN}${qkm}${NC}"
        else
            # Not fatal: the binary may use another prefix, or a future version may not
            # read them. But this is the first thing to check if QEMU starts then exits.
            echo -e "  ${YELLOW}!${NC} keymaps QEMU introuvables - si QEMU s'arrete au demarrage," >&2
            echo -e "  ${YELLOW}  cherchez \"could not read keymap file\" dans logs/qemu.log${NC}" >&2
        fi
    fi
fi

echo -e "${GREEN}[5c/9] Ajustements osmocom dans le rootfs...${NC}"
echo -e "${GREEN}[5d/9] Patch configs ISO...${NC}"

# SAME recipe as on $TEMP_CONFIG, replayed on the rootfs (idempotent):
# /etc/osmocom in the rootfs comes from the docker IMAGE (docker cp at step 5),
# not from $TEMP_CONFIG, so it may hold files the substitution never saw.
apply_native_post_patches "$ROOTFS/etc" "$ISO_OP_ID" "$ISO_N_MS" "$HOST_IP" \
    "${ISO_NODE:-1}" "${ISO_WAN_TMP:-/nonexistent}" "$SGSN_GTP_IP" "$HLR_IP"

if [ -f "$ROOTFS/etc/osmocom/run.sh" ]; then
    chmod +x "$ROOTFS/etc/osmocom/run.sh"
else
    # Guard: under set -e a missing run.sh would abort the build. The file comes
    # from the image, not the repo: if it is missing, check the image.
    echo -e "  ${YELLOW}!${NC} /etc/osmocom/run.sh absent de l'image ${CYAN}${ISO_SRC_IMAGE}${NC}"
fi

echo -e "  ${GREEN}✓${NC} retouches natives rejouees sur le rootfs (SGSN, MSC, sms-routing, run.sh)"
mkdir -p "$ROOTFS/usr/bin"
# ── /usr/local/bin ALSO in /usr/bin, but NEVER over a dpkg-owned binary ──────
# /usr/local/bin holds wrappers (e.g. tcpdump, which calls /usr/bin/tcpdump):
# copying one over the real binary would make it call itself forever, and
# apt would not repair it. Only files owned by NO package are copied. The dpkg
# file lists are read straight from the rootfs (no chroot, which would not
```

```
# work for an arm64 cross build; no dpkg-query).
_ubin_dpkg="$WORK/usr-bin-dpkg.txt"
cat "$ROOTFS"/var/lib/dpkg/info/*.list 2>/dev/null \
    | grep '^/usr/bin/' | sort -u > "$_ubin_dpkg" || true
for _f in "$ROOTFS/usr/local/bin/"*; do
    [ -e "$_f" ] || continue
    _b="${_f##*/}"
    if grep -qxF "/usr/bin/$_b" "$_ubin_dpkg"; then
        echo -e "  ${YELLOW}!${NC} /usr/bin/$_b appartient a un paquet - laisse intact
        ↪  (${CYAN}/usr/local/bin/$_b${NC} le precede dans le PATH)"
        continue
    fi
    cp -a "$_f" "$ROOTFS/usr/bin/" 2>/dev/null || true
done

mkdir -p "$ROOTFS/root/.osmocom/bb"
if [ -f "$ROOTFS/opt/GSM/osmo-operator/mobile.cfg" ]; then
    cp "$ROOTFS/opt/GSM/osmo-operator/mobile.cfg" "$ROOTFS/root/.osmocom/bb/mobile.cfg"
elif [ -f "$ROOTFS/opt/GSM/osmo-operator/configs/mobile.cfg" ]; then
    cp "$ROOTFS/opt/GSM/osmo-operator/configs/mobile.cfg" "$ROOTFS/root/.osmocom/bb/mobile.cfg"
elif [ -f "$ROOTFS/etc/osmocom/mobile.cfg" ]; then
    cp "$ROOTFS/etc/osmocom/mobile.cfg" "$ROOTFS/root/.osmocom/bb/mobile.cfg"
fi

# ── NO osmocom user: root only ──────────────────────────────────────
# The image had an "osmocom" account forced to UID 0 (usermod -o -u 0), which
# breaks GDM autologin, yields two HOMEs (two .osmocom/bb) and confusing
# ownership. The account is DELETED and the systemd units are given back to
# root: the upstream Osmocom .service files set "User=osmocom / Group=osmocom"
# and would otherwise fail with "Failed to determine user credentials".
sed -i -e 's/^User=osmocom$/User=root/' -e 's/^Group=osmocom$/Group=root/' \
       "$ROOTFS/lib/systemd/system"/osmo-*.service \
       "$ROOTFS/etc/systemd/system"/osmo-*.service 2>/dev/null || true

chroot "$ROOTFS" userdel -r osmocom 2>/dev/null || true
chroot "$ROOTFS" groupdel osmocom  2>/dev/null || true
rm -rf "$ROOTFS/home/osmocom"
# /var/lib/osmocom (HLR databases, GTP state) and /var/log/osmocom belonged to
# the deleted account: without this chown, writes fail at first boot
# ("Unable to create file").
chown -R 0:0 "$ROOTFS/root/.osmocom" "$ROOTFS/var/lib/osmocom" \
             "$ROOTFS/var/log/osmocom" 2>/dev/null || true

# The Docker image's "osmocom" account was a UID 0 alias and is removed here;
# the real, unprivileged osmocom account is recreated at step 8d.
echo -e "  ${GREEN}✓${NC} alias uid-0 osmocom retire (unites osmo-* rendues a root) + /usr/bin + mobile.cfg prets"

# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.70 /opt/GSM/osmo-operator/iso_modules/60-dashboard.sh

9573 octets, 157 lignes → 157 lignes

```
#!/bin/bash
# iso_modules/60-dashboard.sh - step 6: web dashboard (osmo-egprs-web)
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment:
#   OSMO_WEB_REPO=<url>      dashboard git repository
```

```
#   OSMO_WEB_BRANCH=<branch> branch to clone (default: main)
#   OSMO_WEB_LOCAL=<dir>     use this local tree instead of cloning

# ── Step 6: web dashboard injection ──────────────────────────────
echo -e "${GREEN}[6/9] Dashboard web (git clone)...${NC}"
WEB="$ROOTFS/opt/GSM/osmo-egprs-web"
WEB_REPO="${OSMO_WEB_REPO:-https://github.com/bbaranoff/osmo-egprs-web.git}"
# Default branch is main (reviewed code); OSMO_WEB_BRANCH=test must be explicit.
WEB_BRANCH="${OSMO_WEB_BRANCH:-main}"

mkdir -p "$WEB/web"
# Source of truth: the bbaranoff/osmo-egprs-web git repo (cloned below).
# A local tree is only used when requested: OSMO_WEB_LOCAL=<dir> ./build-iso.sh.
# The native patch below is idempotent (skipped if server.js is already native).
LOCAL_WEB="${OSMO_WEB_LOCAL:-}"
if [ -n "$LOCAL_WEB" ] && [ -f "$LOCAL_WEB/server.js" ]; then
    cp "$LOCAL_WEB/server.js" "$WEB/server.js"
    [ -f "$LOCAL_WEB/package.json" ] && cp "$LOCAL_WEB/package.json" "$WEB/package.json"
    [ -d "$LOCAL_WEB/web" ]          && cp -r "$LOCAL_WEB/web/."     "$WEB/web/"
    [ -f "$LOCAL_WEB/start-web.sh" ] && cp "$LOCAL_WEB/start-web.sh" "$WEB/" && chmod +x "$WEB/start-web.sh"
    [ -f "$LOCAL_WEB/Dockerfile" ]   && cp "$LOCAL_WEB/Dockerfile"   "$WEB/Dockerfile"
    # The .git lets update.sh fetch instead of recloning at boot.
    [ -d "$LOCAL_WEB/.git" ]         && cp -a "$LOCAL_WEB/.git"     "$WEB/"
    echo -e "  ${GREEN}✓${NC} osmo-egprs-web depuis source LOCALE ($LOCAL_WEB)"
else
    WEB_TMP="$WORK/osmo-egprs-web"
    GIT_TERMINAL_PROMPT=0 git clone --depth 1 -b "$WEB_BRANCH" "$WEB_REPO" "$WEB_TMP" 2>&1 | tail -2 || true
    # The whole clone, .git included, goes into the image so that update.sh can
    # fetch incrementally instead of wiping and recloning at boot.
    # cp -a: files already placed by a local override are overwritten, not removed.
    [ -d "$WEB_TMP/.git" ] && cp -a "$WEB_TMP/." "$WEB/"
    # Repo layout: server.js / package.json / web/ / start-web.sh at the ROOT
    # (fallback to server/ for an older layout).
    if   [ -f "$WEB_TMP/server.js" ];        then cp "$WEB_TMP/server.js"        "$WEB/server.js"
    elif [ -f "$WEB_TMP/server/server.js" ]; then cp "$WEB_TMP/server/server.js" "$WEB/server.js"; fi
    if   [ -f "$WEB_TMP/package.json" ];        then cp "$WEB_TMP/package.json"        "$WEB/package.json"
    elif [ -f "$WEB_TMP/server/package.json" ]; then cp "$WEB_TMP/server/package.json" "$WEB/package.json"; fi
    [ -d "$WEB_TMP/web" ]          && cp -r "$WEB_TMP/web/."     "$WEB/web/"
    [ -f "$WEB_TMP/start-web.sh" ] && cp "$WEB_TMP/start-web.sh" "$WEB/" && chmod +x "$WEB/start-web.sh"
    [ -f "$WEB_TMP/Dockerfile" ]   && cp "$WEB_TMP/Dockerfile"   "$WEB/Dockerfile"
    if [ -f "$WEB/server.js" ]; then
        echo -e "  ${GREEN}✓${NC} osmo-egprs-web depuis le git ${CYAN}$WEB_REPO${NC} ($WEB_BRANCH)"
    else
        echo -e "  ${RED}✗ clone osmo-egprs-web sans server.js - dashboard incomplet${NC}"
    fi
fi

# ── Tutorial, served by the dashboard ────────────────────────────
# Done here because $WEB has just been populated and is the only static root
# the dashboard exposes. /usr/share keeps an offline fallback copy, but the
# icon opens THIS one - see /usr/local/bin/osmo-tutorial.
if [ -f "$DIR/data/tutorial.html" ]; then
    mkdir -p "$WEB/web"
    cp -f "$DIR/data/tutorial.html" "$WEB/web/tutorial.html"
    echo -e "  ${GREEN}✓${NC} tutoriel servi par le dashboard : ${CYAN}/tutorial.html${NC}"
fi
# The help page (the dock "?" button) is served the same way; see
# /usr/local/bin/osmo-info.
if [ -f "$DIR/configs/info.html" ]; then
    mkdir -p "$WEB/web"
    cp -f "$DIR/configs/info.html" "$WEB/web/info.html"
    echo -e "  ${GREEN}✓${NC} aide servie par le dashboard : ${CYAN}/info.html${NC}"
fi

# Patch server.js for native (no-docker) mode: VTY via direct telnet to
# 127.0.0.1 (or ip netns exec) instead of docker exec. Idempotent; never fails
```

```
# the build.
if [ -f "$WEB/server.js" ] && command -v python3 >/dev/null 2>&1; then
python3 - "$WEB/server.js" <<'PYEOF' || echo -e "  ${YELLOW}[web] patch natif non applique (server.js amont a
 ↪  change ?)${NC}"
import sys, re
p = sys.argv[1]
s = open(p, encoding='utf-8').read()
if 'const NATIVE' in s:
    print('  [web] server.js deja en mode natif - skip'); sys.exit(0)
HELPERS = """
// ── Native (no-docker) mode ──────────────────────────────
const NATIVE        = (process.env.OSMO_NATIVE !== '0');
const OP_IDS        = (process.env.OSMO_OP_IDS || '1').split(',')
                        .map(function(s){ return parseInt(s, 10); })
                        .filter(function(n){ return !isNaN(n); });
const NETNS_PREFIX  = process.env.OSMO_NETNS_PREFIX || '';
function vtyProc(container, port, ip, id) {
  if (NATIVE) {
    if (NETNS_PREFIX) return { bin: 'ip', args: ['netns','exec', NETNS_PREFIX + id, 'telnet', ip, String(port)] };
    return { bin: 'telnet', args: [ip, String(port)] };
  }
  return { bin: 'docker', args: ['exec','-i', container, 'telnet', ip, String(port)] };
}
function shCmd(container, id, inner) {
  if (NATIVE) {
    if (NETNS_PREFIX) return 'ip netns exec ' + NETNS_PREFIX + id + ' bash -c "' + inner + '"';
    return 'bash -c "' + inner + '"';
  }
  return 'docker exec ' + container + ' bash -c "' + inner + '"';
}
"""
n = [0]
def sub(pat, rep, flags=0):
    global s
    s, c = re.subn(pat, rep, s, flags=flags); n[0]+=c; return c
sub(r"(const VTY_RETRY_DELAY = 2000;)", r"\1\n" + HELPERS.replace('\\','\\\\'))
sub(r"(function discoverOperators\(\) \{)", r"\1\n  if (NATIVE) return Promise.resolve(OP_IDS.slice());")
sub(r"var proc = spawn\('docker', \[\s*'exec', '-i', container, 'telnet', targetIp, String\(port\)\s*\], \{
 ↪  stdio: \['pipe','pipe','pipe'\] \}\);",
    "var vc = vtyProc(container, port, targetIp, String(container).replace(PREFIX, ''));\n    var proc =
 ↪  spawn(vc.bin, vc.args, { stdio: ['pipe','pipe','pipe'] });", re.DOTALL)
sub(r"return execAsync\(\s*'docker inspect.*?\)\.then\(function\(running\) \{",
    "var runningProbe = NATIVE\n    ? execAsync(shCmd(container, id, 'ss -tln 2>/dev/null | grep -q :' +
 ↪  VTY_PORTS.bsc + ' && echo true || echo false'), 3000)\n    : execAsync('docker inspect -f
 ↪  \\'{{.State.Running}}\\' ' + container + ' 2>/dev/null', 3000);\n  return runningProbe.then(function(running)
 ↪  {", re.DOTALL)
sub(r"execAsync\(\s*'docker exec ' \+ container \+ ' bash -c \"ss -tlnp.*?', 3000\s*\)",
    "execAsync(\n        shCmd(container, id, 'ss -tlnp 2>/dev/null | grep :7890 | wc -l'), 3000\n      )",
 ↪  re.DOTALL)
sub(r"log\('VTY open: docker exec.*?\], \{ stdio: \['pipe','pipe','pipe'\] \}\);",
    "var vc = vtyProc(this.container, this.port, this.ip, this.opId);\n  log('VTY open: ' + vc.bin + ' ' +
 ↪  vc.args.join(' ') + ' (attempt ' + (this.retries + 1) + ')');\n\n  this.proc = spawn(vc.bin, vc.args, { stdio:
 ↪  ['pipe','pipe','pipe'] });", re.DOTALL)
open(p,'w',encoding='utf-8').write(s)
print('  [web] server.js patche mode natif (%d remplacements)' % n[0])
sys.exit(0 if n[0] >= 6 else 2)
PYEOF
fi

# ── The dashboard must start on its own (live ISO and installed disk) ──────
_SVC_SRC="$DIR/services"
# Only the dashboard unit (osmo-egprs-web.service) is installed and enabled.
# The web installation itself is done at image BUILD time (Dockerfile:
# RUN bash install-web-service.sh), so boot does not need Internet.
if [ -f "$_SVC_SRC/osmo-egprs-web.service" ]; then
    install -d "$ROOTFS/etc/systemd/system/multi-user.target.wants"
```

```
    cp -f "$_SVC_SRC/osmo-egprs-web.service"         "$ROOTFS/etc/systemd/system/"
    # The web repo ships its own, diverging copy of the unit: the operator
    # repo's copy is imposed as the single source of truth.
    [ -d "$ROOTFS/opt/GSM/osmo-egprs-web" ] && \
        cp -f "$_SVC_SRC/osmo-egprs-web.service" \
              "$ROOTFS/opt/GSM/osmo-egprs-web/osmo-egprs-web.service"
    ln -sf /etc/systemd/system/osmo-egprs-web.service \
           "$ROOTFS/etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service"
    echo -e "  ${GREEN}✓${NC} dashboard : unite ${CYAN}osmo-egprs-web${NC} posee et activee au boot"
else
    echo -e "  ${RED}✗ services/osmo-egprs-web*.service introuvables - le dashboard ne demarrerait pas seul${NC}"
     ↪  >&2
    exit 1
fi

# Remove /usr/local/bin/node if it is a dangling link (e.g. to the absent
# /opt/node/bin/node): it precedes /usr/bin in PATH and misleads scripts that
# test `-x /usr/local/bin/node`.
if [ -L "$ROOTFS/usr/local/bin/node" ] && [ ! -e "$ROOTFS/usr/local/bin/node" ]; then
    rm -f "$ROOTFS/usr/local/bin/node" "$ROOTFS/usr/local/bin/npm" "$ROOTFS/usr/local/bin/npx"
    echo -e "  ${GREEN}✓${NC} liens morts /usr/local/bin/node,npm,npx retires (node reste en /usr/bin)"
fi

# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.71 /opt/GSM/osmo-operator/iso_modules/70-scripts.sh

9580 octets, 162 lignes → 162 lignes

```
#!/bin/bash
# iso_modules/70-scripts.sh - steps 7/7b: project scripts, WAN, NetworkManager
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# ── Step 7: project scripts and the start-native.sh launcher ──────────────
echo -e "${GREEN}[7/9] Scripts projet et adaptation ISO...${NC}"
# ── A single repository tree: /opt/GSM/osmo-operator ──────────────────────
# All launchers, links, aliases and units point to this tree (full clone with
# .git, updated at step [5a/9] and by osmo-update). If the clone could not be
# fetched, it is filled from the build repository instead, at the same path.
P="$ROOTFS/opt/GSM/osmo-operator"
if [ ! -x "$P/start-native.sh" ]; then
    echo -e "  ${YELLOW}!${NC} /opt/GSM/osmo-operator sans lanceur (image perimee, clone impossible)"
    echo -e "    -> remplissage depuis le depot de construction ${CYAN}${DIR}${NC}"
    mkdir -p "$P"
    # --exclude .git: do not fake a repository; osmo-update will rebuild it.
    tar -C "$DIR" --exclude=.git --exclude='*.iso' -cf - . | tar -C "$P" -xf -
    find "$P" -name "*.sh" -exec chmod +x {} \;
fi
if [ ! -x "$P/start-native.sh" ]; then
    echo -e "  ${RED}✗${NC} start-native.sh introuvable - l'ISO n'aura pas de lanceur" >&2
    exit 1
fi
ln -sf /opt/GSM/osmo-operator/start-native.sh "$ROOTFS/usr/local/bin/osmo-start-native" 2>/dev/null || true
ln -sf /opt/GSM/osmo-operator/start-native.sh "$ROOTFS/usr/local/bin/osmo-start-direct" 2>/dev/null || true  #
 ↪  legacy name
# Default wallpaper (80-chroot.sh reads it from THIS tree): installed from the
# build repository so that an unpushed tree still ships it.
[ -f "$DIR/configs/gsm-lab-wallpaper.jpg" ] \
    && install -Dm644 "$DIR/configs/gsm-lab-wallpaper.jpg" "$P/configs/gsm-lab-wallpaper.jpg"
```

```
# Plain background (same scene as the overlay, see tools/paysage-render.py).
[ -f "$DIR/configs/gsm-lab-fond.jpg" ] \
    && install -Dm644 "$DIR/configs/gsm-lab-fond.jpg" "$P/configs/gsm-lab-fond.jpg"
# Two-state background (tools/osmo-fond.py): the city alone when idle
# (gsm-lab-ville.jpg, schema default), the city with the antenna while a bench
# is running (gsm-lab-antenne.jpg), cross-faded.
for _bg in gsm-lab-ville.jpg gsm-lab-antenne.jpg; do
    [ -f "$DIR/configs/$_bg" ] && install -Dm644 "$DIR/configs/$_bg" "$P/configs/$_bg"
done
unset _bg
ln -sf /opt/GSM/osmo-operator/tools/osmo-fond.py "$ROOTFS/usr/local/bin/osmo-fond" 2>/dev/null || true
# osmo-op: selects which operator the overlay and Conky show (arrows, from
# the command line). Also bound by iso_modules/80-chroot.sh to
# Ctrl+AltGr+Right / Ctrl+AltGr+Left.
ln -sf /opt/GSM/osmo-operator/tools/osmo-op.sh "$ROOTFS/usr/local/bin/osmo-op" 2>/dev/null || true
ln -sf /opt/GSM/osmo-operator/tools/osmo-drivers.sh "$ROOTFS/usr/local/bin/osmo-drivers" 2>/dev/null || true
ln -sf /opt/GSM/osmo-operator/tools/overlay-draw.py "$ROOTFS/usr/local/bin/overlay-draw" 2>/dev/null || true
# Emulated C123 window (qosmo screen + keypad) in PATH: the c123 alias of
# /etc/profile.d/98-osmo-banc-aliases.sh only exists in login shells; the link
# works everywhere else (Alt+F2, sudo, scripts).
ln -sf /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py "$ROOTFS/usr/local/bin/c123" 2>/dev/null || true
ln -sf /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py "$ROOTFS/usr/local/bin/osmo-c123" 2>/dev/null || true
# Bench aliases (banc-rssi, banc-trx, banc-mobile, c123-1...): the profile.d
# file for login shells, and the same names as real commands (tools/osmo-banc-cmd.sh).
install -Dm644 "$P/scripts/osmo-banc-aliases.sh" "$ROOTFS/etc/profile.d/98-osmo-banc-aliases.sh"
bash "$P/tools/osmo-banc-cmd.sh" --install "$ROOTFS/usr/local/bin" >/dev/null
# Adds /root/gnuarm/install/bin to PATH when present (--gnuarm add-on);
# inert while the toolchain is absent.
install -Dm644 "$P/scripts/osmo-gnuarm-profile.sh" "$ROOTFS/etc/profile.d/97-osmo-gnuarm.sh"
# Apply the repo patches (patches/, via tools/osmo-patches-apply.sh) to the
# rootfs trees, outside the chroot (GPRS layer1, osmocon...). With gnuarm,
# "cd /opt/GSM/osmocom-bb/src && make" then builds the whole tree.
OSMO_PATCHES_QUIET=1 GSM_ROOT="$ROOTFS/opt/GSM" REPO="$P" bash "$P/tools/osmo-patches-apply.sh" "$ROOTFS/opt/GSM"
 ↪  || true
# WAN WireGuard (wiki/WAN.md): "wg 1" (hub 10.99.0.1), "wg N ..." (10.99.0.N),
# a profile.d function that passes through to the real wg; osmo-wg for sudo and
# scripts. Keys are generated on the machine at the first "wg N", NEVER in the
# image (they would be shared).
install -Dm644 "$P/scripts/osmo-wg-profile.sh" "$ROOTFS/etc/profile.d/97-osmo-wg.sh"
ln -sf /opt/GSM/osmo-operator/tools/osmo-wg.sh "$ROOTFS/usr/local/bin/osmo-wg" 2>/dev/null || true
if [ -f "$DIR/launch/osmo-launch.sh" ]; then
    cp "$DIR/launch/osmo-launch.sh" "$ROOTFS/opt/osmo-launch.sh" && chmod +x "$ROOTFS/opt/osmo-launch.sh"
    ln -sf /opt/osmo-launch.sh "$ROOTFS/usr/local/bin/osmo-launch"
fi
echo -e "  ${GREEN}✓${NC} lanceurs -> ${CYAN}/opt/GSM/osmo-operator${NC} (arbre unique, avec .git)"

# /etc/osmocom/gapk-start.sh comes from the Docker image (COPY scripts/.) and
# lib/audio.sh prefers it over scripts/gapk-start.sh: always align it with the
# repo copy (idempotent), so a cached image cannot ship an outdated one.
if [ -f "$P/scripts/gapk-start.sh" ]; then
    install -d "$ROOTFS/etc/osmocom"
    install -m 755 "$P/scripts/gapk-start.sh" "$ROOTFS/etc/osmocom/gapk-start.sh"
    echo -e "  ${GREEN}✓${NC} /etc/osmocom/gapk-start.sh aligne sur scripts/ (superviseur gapk sans emetteur en
     ↪  doublon)"
fi

# ── Voice in the image: io-tch-format and the mobile audio buffer ──────────
# Same fix as update.sh, applied to the ROOTFS by scripts/voix-forks.sh (which
# documents the details). Both settings live in qosmo (run_modules, cfgs),
# copied as-is from the reference image by 50-injection-image.sh.
#
#   io-tch-format rtp: otherwise the mobile reads/writes TCH frames in the TI
#   Calypso layout, while pont.py encodes them with libosmocoding in RTP/RFC3551
#   layout. The template cfgs/mobile_group1.cfg is what matters:
#   run_modules/20-mobile-cfg.sh copies it over ~/.osmocom/bb/ at every start.
#
```

```
#   CALYPSO_PULSE_LATENCY_MSEC: the mobile writes one 20 ms GSM frame per
#   snd_pcm_writei; a too-short buffer makes every write return -EPIPE and
#   pq_alsa.c silently re-prepares the PulseAudio stream. Set to 320 ms.
if [ -x "$P/scripts/voix-forks.sh" ]; then
    # Do not put color codes in a sed replacement: GNU sed reads \0 in
    # '\033[0;32m' as "whole match". Lines are re-read and printed with echo -e.
    "$P/scripts/voix-forks.sh" "$ROOTFS" | while IFS= read -r _l; do
        case "$_l" in
            "  [voix]"*) echo -e "  ${GREEN}✓${NC}${_l#"  [voix]"}" ;;
            *)           echo "$_l" ;;
        esac
    done
fi

# ── WAN: node table fixed in the image ──────────────────────────────────
if [ "$ISO_WAN" = "1" ]; then
    echo -e "${GREEN}[7b/9] WAN - table des noeuds embarquee...${NC}"
    # shellcheck source=network/wan-nodes.sh
    . "$DIR/network/wan-nodes.sh"
    WAN_OPS="$ISO_WAN_OPS"
    if [ -n "$ISO_WAN_NODES" ]; then
        wan_nodes_parse "$ISO_WAN_NODES" || exit 1
        WAN_NODE_ID="${ISO_WAN_ID:-0}"
    else
        # Interactive build: same questions as ./launch/start-docker.sh --wan.
        # The node number asked here is only a default: each machine booting the ISO
        # identifies itself by its IP.
        WAN_NODE_ID="${ISO_WAN_ID:-0}"
        wan_nodes_prompt || exit 1
    fi
    wan_nodes_validate || exit 1
    WAN_AUTO=1 WAN_CONF_FILE="$ROOTFS/etc/osmo-wan.conf" wan_nodes_save
    wan_nodes_summary
    echo -e "  ${GREEN}✓${NC} /etc/osmo-wan.conf fige dans l'ISO (WAN_AUTO=1)"
    echo -e "  ${CYAN}Au boot :${NC} start-native.sh applique le WAN tout seul ;"
    echo -e "  ${CYAN}sans --wan a la construction, l'ISO n'a AUCUN WAN.${NC}"
else
    echo -e "  ${CYAN}[7b/9] WAN non embarque (--wan absent) - ISO autonome${NC}"
fi

# ── Step 9 prep: files the chroot cannot write itself ──────────────────
# The chroot script is passed to "bash -c" in SINGLE quotes: a single
# apostrophe in its body would end the string. Files containing quotes (quoted
# heredocs, nested shell commands) are therefore written HERE, into the rootfs;
# the chroot only enables them.

# NetworkManager drives the desktop, not the packet core: without this rule NM
# takes over apn0 or a GGSN tun and drops a subscriber's data session as
# "unconfigured".
mkdir -p "$ROOTFS/etc/NetworkManager/conf.d"
cat > "$ROOTFS/etc/NetworkManager/conf.d/10-osmo-networkd.conf" <<'NMCONF'
# Written by build-iso.sh. NetworkManager manages the physical NICs and the
# desktop; packet-core interfaces are left to systemd-networkd and the bench
# scripts.
[main]
plugins=keyfile

[keyfile]
unmanaged-devices=interface-name:apn*;interface-name:tun*;interface-name:veth*;interface-name:docker*;interface-
 ↪  name:br-*;interface-name:osmo*
NMCONF

# Firefox comes from Mozilla's .deb, installed in the chroot by 80-chroot.sh.

# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
```

```
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.72 /opt/GSM/osmo-operator/iso_modules/71-debs-banc.sh

2533 octets, 44 lignes → 44 lignes

```
#!/bin/bash
# iso_modules/71-debs-banc.sh - step 7c: bench .debs embedded in the image
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment: ISO_WITH_DEBS=0 (--without-debs) skips this step.

# ── Step 7c: the bench .deb packages, EMBEDDED in the image ──────────────
# packaging/build-debs.sh builds one .deb per component (osmo-operator, pont,
# qemu-calypso, calypso-firmware) from the trees this ISO embeds anyway. They
# are stored in /var/cache/osmo-debs: a machine installed from the ISO, or any
# other, can then run
#     dpkg -i /var/cache/osmo-debs/*.deb
# and get the bench without Docker or git. The ISO itself keeps running on the
# full trees (the repo and qosmo with their .git) - the packages travel along,
# they do not replace them.
# Non-fatal: without dpkg-deb or without a built qemu, the image ships without them.
# --arm: build-debs.sh compiles the C launcher and reads binaries with ldd - on
# the host that would be x86 labelled arm64. It runs INSIDE the chroot, after
# the apt of step 8 (82-arm-natif).
if [ "${ISO_WITH_DEBS:-1}" = "0" ]; then
    echo -e "  ${CYAN}·${NC} --without-debs : aucun paquet .deb embarque dans l'image"
elif [ "${ISO_ARCH:-amd64}" != "$(dpkg --print-architecture)" ]; then
    echo -e "  ${CYAN}·${NC} paquets .deb du banc : fabriques dans le chroot ${ISO_ARCH} plus loin"
elif [ -x "$DIR/packaging/build-debs.sh" ] && command -v dpkg-deb >/dev/null 2>&1; then
    echo -e "${GREEN}[7c/9] Paquets .deb du banc...${NC}"
    _DEBS="$WORK/debs"
    if OSMO_OPERATOR_SRC="$DIR" QOSMO_SRC="$ROOTFS/opt/GSM/qosmo" \
       FIRMWARE_SRC="$ROOTFS/opt/GSM/firmware" \
       "$DIR/packaging/build-debs.sh" --out "$_DEBS" >"$WORK/build-debs.log" 2>&1; then
        install -d "$ROOTFS/var/cache/osmo-debs"
        cp -f "$_DEBS"/*.deb "$ROOTFS/var/cache/osmo-debs/"
        echo -e "  ${GREEN}✓${NC} $(ls "$_DEBS"/*.deb | wc -l) paquets dans ${CYAN}/var/cache/osmo-debs${NC} ($(du
         ↪  -sh "$_DEBS" | cut -f1))"
    else
        echo -e "  ${YELLOW}!${NC} paquets .deb non construits (voir $WORK/build-debs.log) - l image sort sans eux"
    fi
fi


# End of module: `. file` returns the status of its LAST command and
# build-iso.sh runs under set -e. A module ending with a false test
# ("[ ... ] && { ...; }") would abort everything silently. Always 0 here.
true
```

2.73 /opt/GSM/osmo-operator/iso_modules/80-chroot.sh

50843 octets, 812 lignes → 812 lignes

```
#!/bin/bash
# iso_modules/80-chroot.sh - step 8: configure the rootfs inside the chroot (apt, kernel, desktop).
# Sourced by build-iso.sh in numeric order (same shell, variables and functions);
# not runnable on its own. A `return` at the top of a module means "nothing to do"
# (this is how --arm skips a step).
```

```
# Environment: ROOTFS, DIR, ISO_ROLE, ISO_LITE, ISO_ARCH, ISO_MIRROR, ISO_SUITE,
# ISO_DESKTOP, ISO_DEB_CACHE, NO_CACHE, OSMO_ISO_KB, OSMO_ISO_SOFTSIM, OSMO_FF_HOME.

echo -e "${GREEN}[8/9] Configuration chroot...${NC}"
mount --bind /proc "$ROOTFS/proc"; mount --bind /sys "$ROOTFS/sys"
# ── Persistent apt cache for the chroot ──────────────────────────────
# The apt/apt-fast download directory is a bind mount of the host cache
# ISO_DEB_CACHE/apt-archives, reused by every pass and later builds. It is
# unmounted before the squashfs (no .deb files in the ISO), and the chroot skips
# "apt-get clean" while it is mounted (that would empty the host cache).
# --no-cache: no bind mount.
ISO_APT_CACHE_BOUND=0
if [ -n "$ISO_DEB_CACHE" ] && [ -z "$NO_CACHE" ]; then
    mkdir -p "$ISO_DEB_CACHE/apt-archives/partial" "$ROOTFS/var/cache/apt/archives"
    if mount --bind "$ISO_DEB_CACHE/apt-archives" "$ROOTFS/var/cache/apt/archives"; then
        ISO_APT_CACHE_BOUND=1
        echo -e "  ${GREEN}cache apt du chroot : $ISO_DEB_CACHE/apt-archives ($(du -sh
        ↪  "$ISO_DEB_CACHE/apt-archives" | cut -f1), $(find "$ISO_DEB_CACHE/apt-archives" -maxdepth 1 -name
        ↪  '*.deb' | wc -l) paquets)${NC}"
    fi
fi
mount --bind /dev "$ROOTFS/dev";   mount --bind /dev/pts "$ROOTFS/dev/pts" 2>/dev/null||true
# The host resolv.conf gives apt DNS during the build only; it is replaced at
# the end by 81-cloture-systeme.sh (see its DNS block).
cp /etc/resolv.conf "$ROOTFS/etc/resolv.conf" 2>/dev/null||true

# Install the apt-fast installer (same as the Dockerfile) into the rootfs.
install -m755 "$DIR/packaging/apt-fast-install.sh" "$ROOTFS/usr/local/sbin/apt-fast-install"

# The script below is single-quoted (no expansion at write time), so ISO_ROLE and
# the other settings are passed through the environment.
# LC_ALL=C.UTF-8: the host locale is not generated in the chroot and perl/debconf
# would warn "Setting locale failed" on every package. C.UTF-8 needs no locale-gen.
# NOTE: everything up to the closing quote is inside a single-quoted bash -c
# string: never put an apostrophe in it, comments included (use \047).
chroot "$ROOTFS" env PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin \
                  LC_ALL=C.UTF-8 LANG=C.UTF-8 LANGUAGE= \
                  ISO_ROLE="$ISO_ROLE" ISO_LITE="$ISO_LITE" ISO_APT_CACHE_BOUND="$ISO_APT_CACHE_BOUND" \
                  ISO_ARCH="$ISO_ARCH" ISO_MIRROR="$ISO_MIRROR" \
                  ISO_DESKTOP="$ISO_DESKTOP" OSMO_ISO_KB="$OSMO_ISO_KB" \
                  OSMO_ISO_SOFTSIM="${OSMO_ISO_SOFTSIM:-1}" bash -c '
set -e; export DEBIAN_FRONTEND=noninteractive
export DPKG_OPTIONS="--force-confold --force-confdef"

# ── Fast apt/dpkg ────────────────────────────────────────────────────
# This rootfs is disposable (built, packed into a squashfs, then deleted), so
# the per-file fsync of dpkg is useless here and dominates build time.
# force-unsafe-io disables it, as the official Docker images do.
mkdir -p /etc/dpkg/dpkg.cfg.d /etc/apt/apt.conf.d
echo "force-unsafe-io" > /etc/dpkg/dpkg.cfg.d/02-unsafe-io
# Download settings (Languages=none, Retries, Pipeline-Depth, Use-Pty) are written
# by apt-fast-install below into /etc/apt/apt.conf.d/90osmo-operator, the same
# as in the Docker image and on the host.

APT_OPTS="-o Dpkg::Options::=--force-confold -o Dpkg::Options::=--force-confdef"

# Preseed debconf
echo "wireshark-common wireshark-common/install-setuid boolean true" | debconf-set-selections
echo "keyboard-configuration keyboard-configuration/layoutcode string us" | debconf-set-selections
echo "keyboard-configuration keyboard-configuration/modelcode string pc105" | debconf-set-selections
echo "console-setup console-setup/charmap47 select UTF-8" | debconf-set-selections

# ISO_SUITE comes from the chroot environment; the heredoc below is unquoted so
# that the variables are substituted.
_S="${ISO_SUITE:-noble}"
# The mirror follows the architecture (arm64: ports.ubuntu.com). arm64 also
```

```
# needs restricted (linux-firmware-raspi, the Pi boot firmware start4.elf).
_M="${ISO_MIRROR:-http://archive.ubuntu.com/ubuntu}"
_C="main universe multiverse"
[ "${ISO_ARCH:-amd64}" = "arm64" ] && _C="$_C restricted"
# ── Git: no forced HTTP/1.1 ──────────────────────────────────────────
# Remove any system-wide http.version setting. If a machine needs it, set it
# locally:  git config --global http.version HTTP/1.1
git config --system --unset http.version 2>/dev/null || true

cat > /etc/apt/sources.list <<SOURCES
deb $_M $_S           $_C
deb $_M $_S-updates    $_C
deb $_M $_S-security   $_C
SOURCES

# ── CA certificates first ────────────────────────────────────────────
# Installing ca-certificates is not enough: update-ca-certificates builds
# /etc/ssl/certs/ca-certificates.crt (used by OpenSSL, curl, git, apt, snap),
# and the postinst does not always run it in a chroot. Run it before the first
# TLS access (the NodeSource key below). --fresh rebuilds from the package store
# instead of whatever debootstrap left.
update-ca-certificates --fresh >/dev/null 2>&1 || update-ca-certificates || true

# ── arm64: Armbian repository (noble), in addition to Ubuntu ports ──────
# The arm64 base is Armbian 24.04: bcm2711 kernel (rpi-6.18.y), device trees,
# the rpi4b BSP package (hooks keeping /boot/firmware up to date, zram, resize,
# motd) and its base-files. The repository also carries the Ubuntu Pi packages
# (linux-firmware-raspi, rpi-eeprom, libraspberrypi-bin). Written after the
# certificates (https) and before the single apt-get update, deb822 format with
# the dearmored key in /usr/share/keyrings, as Armbian does.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    curl -fsSL --retry 3 https://apt.armbian.com/armbian.key | gpg --dearmor -o /usr/share/keyrings/armbian.gpg
    cat > /etc/apt/sources.list.d/armbian.sources <<EOF
Types: deb
URIs: https://apt.armbian.com
Suites: $_S
Components: main $_S-utils $_S-desktop
Signed-By: /usr/share/keyrings/armbian.gpg
EOF
    # /boot/firmware must exist before the kernel is installed: the BSP hooks
    # (zzz-copy-new-files, zzz-update-initramfs, bash -e) copy into it without
    # creating it, and their failure would abort this whole script.
    mkdir -p /boot/firmware
    echo "depot Armbian : https://apt.armbian.com $_S"
fi

# ── deb-src: before the single apt-get update ────────────────────────
# The gnuradio build-dep below needs the Sources indexes. The inter-STP hub has
# no gr-gsm, so it gets neither deb-src (~80 MB of indexes) nor build-dep.
if [ "${ISO_ROLE:-operator}" != "interstp" ]; then
    sed -nE "s|^deb (http\S+) (\S+) .*|deb-src \1 \2 main restricted universe multiverse|p" \
        /etc/apt/sources.list | sort -u > /etc/apt/sources.list.d/deb-src.list
fi

# ── NodeSource: before the single apt-get update, nodejs in PKGS ─────
# The dashboard needs node 22, which the distribution does not ship. The
# repository is added by hand (ASCII key in /etc/apt/keyrings, read as-is by
# apt >= 2.4) so nodejs joins the single install list. If the key download
# fails, the official setup script is used as a fallback further down.
NODE_VIA_APT=0
if ! command -v node >/dev/null 2>&1; then
    mkdir -p /etc/apt/keyrings
    if curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key \
            -o /etc/apt/keyrings/nodesource.asc; then
        echo "deb [signed-by=/etc/apt/keyrings/nodesource.asc] https://deb.nodesource.com/node_22.x nodistro main"
 ↪  \
```

```
            > /etc/apt/sources.list.d/nodesource.list
        NODE_VIA_APT=1
    else
        echo "  WARN: cle NodeSource non telechargee - repli sur setup_22.x apres l install"
        rm -f /etc/apt/keyrings/nodesource.asc
    fi
fi

# ── MongoDB (Open5GS HSS/PCRF subscriber store): before the apt update ──────
# mongodb-org is not in Ubuntu: repo.mongodb.org (noble/8.0), dearmored key in
# /usr/share/keyrings, same file as tools/osmo-lte-install.sh. Without the key
# (no network to mongodb.org) the repository is skipped and osmo-epc reports it
# on first start.
MONGO_VIA_APT=0
if [ "${ISO_ROLE:-operator}" != "interstp" ]; then
    if curl -fsSL --retry 3 https://www.mongodb.org/static/pgp/server-8.0.asc \
            | gpg --dearmor --yes -o /usr/share/keyrings/mongodb-server-8.0.gpg 2>/dev/null; then
        echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]
 ↪  https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" \
            > /etc/apt/sources.list.d/mongodb-org-8.0.list
        MONGO_VIA_APT=1
    else
       echo "  WARN: cle MongoDB non telechargee - Open5GS sans base d abonnes (osmo-lte-install --deps plus tard)"
        rm -f /usr/share/keyrings/mongodb-server-8.0.gpg
    fi
fi

# ── Single apt-get update, then apt-fast for everything else ────────────────
# Every extra source is added above so that this one update reads them all.
# apt-fast-install (/usr/local/sbin, same installer as the Dockerfile) installs
# aria2 and curl with apt-get, then downloads in parallel; it falls back to
# apt-get if GitHub is unreachable.
apt-get update -qq
/usr/local/sbin/apt-fast-install

# ── Single apt-get install ──────────────────────────────────────────────────
# One call resolves dependencies once, downloads in parallel and unpacks in a
# single order; this is the heaviest part of the chroot.
#
# It must stay right after apt-get update and before the build-dep: under set -e
# a failing build-dep must not take down the tools the ISO cannot work without:
#   nc       the whole repository queries the VTY with "nc 127.0.0.1 4239";
#            without nc the checks show an EMPTY diagnostic instead of failing.
#   socat    first-choice VTY transport of run_modules/_lib/core.sh; without it
#            21-abonnes-hlr.sh falls back to telnet, which does not exit on stdin
#            EOF, so HLR provisioning fails.
#   tcpdump  GSMTAP/M3UA captures; without it background captures fail silently.
#   git      the embedded repositories keep their .git and are updated in place.
#
# Two lists: the inter-STP hub only routes M3UA (no radio, QEMU, audio or PBX).
# ca-certificates comes first (its store is regenerated above).
#
# Kernel: on noble linux-image-generic is the 6.8 series. The vmlinuz detection
# further down (ls vmlinuz-*, sort -V, tail -1) picks the newest kernel.
# Package names depend on the suite: noble renamed the time_t ABI libraries
# (libasound2t64, libgnutls30t64, libdbi1t64, libsofia-sip-ua-glib3t64) and
# c-ares is libcares2.
#
# ── Kernel headers in the default image ─────────────────────────────────────
# linux-headers-* follows linux-image-* (the metapackage, not a fixed version),
# so headers always match the installed kernel, also after kernel updates.
# Out-of-tree modules (NVIDIA driver, SDR dongles, VirtualBox) need them for
# DKMS, which otherwise requires network access. dkms and build-essential go
# with them (build-essential is already in the arm64 list).
case "$_S" in
    noble) _KERNEL_PKG="linux-image-generic linux-headers-generic";           _T64="t64"; _CARES="libcares2" ;;
```

```
    *)     _KERNEL_PKG="linux-image-generic-hwe-22.04 linux-headers-generic-hwe-22.04"; _T64="";
 ↪  _CARES="libc-ares2" ;;
esac
# ── arm64 / Raspberry Pi 4: Armbian, no live-boot ──────────────────────────
# Same packages as the Armbian rpi4b build (config/sources/families/bcm2711.conf):
# linux-image-current-bcm2711 (vmlinuz in /boot, dtb and overlays under
# /usr/lib/linux-image-<ver>), linux-dtb-current-bcm2711,
# armbian-bsp-cli-rpi4b-current (postinst.d hooks that copy kernel, dtb and
# firmware to /boot/firmware, first-boot resize, zram, motd, armbian-config),
# armbian-firmware; plus the Ubuntu Pi packages: linux-firmware-raspi
# (start4.elf, fixup4.dat, loaded by the GPU before the kernel), rpi-eeprom,
# libraspberrypi-bin (vcgencmd), raspi-config. No u-boot: the firmware loads
# vmlinuz directly (config.txt written by 82-arm-natif). build-essential and
# wget: toast and the launchers are compiled in this chroot, and osmo-update
# recompiles the launchers on the device. No live-boot: the root is an ext4 on
# the SD card mounted by the standard initramfs (root=LABEL=armbi_root).
_LIVE_PKGS="live-boot live-boot-initramfs-tools"
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    _KERNEL_PKG="linux-image-current-bcm2711 linux-headers-current-bcm2711 linux-dtb-current-bcm2711
      armbian-bsp-cli-rpi4b-current armbian-firmware
      linux-firmware-raspi rpi-eeprom libraspberrypi-bin raspi-config build-essential wget"
    _LIVE_PKGS=""
fi
# systemd-resolved is a separate package on noble, not pulled by the debootstrap
# minbase. Without it nothing answers on the 127.0.0.53 stub named in
# /etc/resolv.conf (see the DNS block of 81-cloture-systeme.sh).
#
# cryptsetup and rsync are in the BASE list: the encrypted /home disk (step 8i)
# also applies to images without a desktop (unlock-home opens the volume at
# boot, init-crypthome.sh copies the home with rsync -aHAX).
PKGS="ca-certificates openssl netcat-openbsd socat tcpdump git logrotate systemd-resolved
      $_KERNEL_PKG initramfs-tools dkms build-essential
      $_LIVE_PKGS
      libtalloc2 libtalloc-dev libpcsclite1 libsctp1 libsctp-dev $_CARES
      libgnutls30${_T64} libgnutls28-dev libmnl-dev libmnl0
      libortp-dev libdbi1${_T64} libdbd-sqlite3 sqlite3
      libfftw3-single3 libusb-1.0-0
      libgsm1 libasound2${_T64}
      libsofia-sip-ua-glib3${_T64}
      liburing2 libslirp0
      iproute2 iptables net-tools lksctp-tools
      tmux telnet expect whiptail
      lsb-release openssh-server sudo
      console-setup keyboard-configuration locales
      psmisc
      cryptsetup rsync
      python3 python3-venv python3-scapy
      tshark wireshark-common
      wireguard-tools"
[ "$NODE_VIA_APT" = "1" ] && PKGS="$PKGS nodejs"

if [ "${ISO_ROLE:-operator}" != "interstp" ]; then
    # Radio, Calypso emulation, audio, PBX: operator node only.
    # gcc-9 and gcc-11 next to the noble gcc-13, as in the Docker image, to rebuild
    # osmocom-bb on the machine (jolly/testing and burst_ind need gcc-9). The
    # alternatives are set after installation, below.
    PKGS="$PKGS
      libasound2-plugins pulseaudio pulseaudio-utils alsa-utils
      binutils-arm-none-eabi gdb-multiarch
      gcc-9 g++-9 gcc-11 g++-11
      asterisk
      ffmpeg"
    # ── LTE: srsRAN (ZeroMQ) and Open5GS runtime libraries ──────────────────
    # The binaries come from the build .deb files (osmo-build-srsran,
    # osmo-build-open5gs, osmo-build-libzmq: 50-injection-image.sh); this list
    # provides their system libraries (authoritative, like the Dockerfile list).
```

```
    # The -dev packages stay: /opt/LTE is a workshop and
    # tools/osmo-lte-install.sh --build must be able to rebuild there.
    # qtbase5 / qwt / boost: srsGUI, built before srsRAN to enable its real-time
    # plots (see lte_build in tools/osmo-lte-install.sh); srsenb and srsue need them
    # at runtime once linked against srsgui.
    # mongodb-org: from the MongoDB repository added above; mongosh and the tools
    # (mongodump/mongorestore) for the subscriber database.
    PKGS="$PKGS
      libzmq5 libzmq3-dev libboost-program-options-dev libmbedtls-dev libconfig++-dev libfftw3-dev cmake
      qtbase5-dev libqt5opengl5-dev libqwt-qt5-dev libboost-system-dev libboost-thread-dev libboost-test-dev
      libpcsclite-dev
      meson ninja-build flex bison libgnutls28-dev libgcrypt20-dev libssl-dev libidn-dev
      libmongoc-dev libbson-dev libyaml-dev libnghttp2-dev libmicrohttpd-dev libcurl4-gnutls-dev
      libtins-dev libtalloc-dev libc-ares-dev
      zstd rsync kpartx"
    [ "$MONGO_VIA_APT" = "1" ] && PKGS="$PKGS mongodb-org mongodb-mongosh mongodb-database-tools"
    # kpartx: required by pmbootstrap (pmb/config/__init__.py, required_programs),
    # which refuses to start without it.

    # ── QEMU build headers: normal ISO only ──────────────────────────────
    # The normal image ships /opt/GSM/qosmo with its .git and build/ so that
    # "make -C build qemu-system-arm" can be rerun on the machine. ninja needs the
    # development .so links and headers, not just the runtimes (liburing2,
    # libslirp0, libpixman-1-0). The LITE image (Dockerfile.lite) strips the
    # toolchains, hence the ISO_LITE test.
    if [ "${ISO_LITE:-0}" != "1" ]; then
        PKGS="$PKGS
      liburing-dev libslirp-dev libpixman-1-dev"
    fi
fi

apt-fast install -y $APT_OPTS --no-install-recommends $PKGS

# ── GCC alternatives ─────────────────────────────────────────────────────
# Same as the Docker image (gcc-9 90, gcc-11 110) plus gcc-13, the noble
# compiler, as DEFAULT (130): it built the kernel, and DKMS (NVIDIA driver via
# ubuntu-drivers, osmo-drivers) builds modules with "gcc".
#   update-alternatives --config gcc      choose (g++ follows as a slave)
#   update-alternatives --set gcc /usr/bin/gcc-9
for _v in 9 11 13; do
    [ -x "/usr/bin/gcc-$_v" ] && [ -x "/usr/bin/g++-$_v" ] || continue
    update-alternatives --install /usr/bin/gcc gcc "/usr/bin/gcc-$_v" "$(( _v * 10 ))" \
        --slave /usr/bin/g++ g++ "/usr/bin/g++-$_v"
done
unset _v
[ -x /usr/bin/gcc-13 ] && update-alternatives --set gcc /usr/bin/gcc-13
echo "  [chroot] alternatives gcc : $(update-alternatives --list gcc 2>/dev/null | xargs) (defaut $(readlink -f
↪  /usr/bin/gcc))"

# ── SIM: PC/SC reader (HID Omnikey), softSIM as a reader, pySim ──────────
# Same code as addition.sh --sim: tools/osmo-sim-install.sh (placed in the
# rootfs by 50-injection). Installs pcscd, libccid, pcsc-tools, opensc, pyscard,
# ruby/ruby-libxml (apt), the smartcard gem (softSIM in pcsc mode: the reader
# card served over SAP) and pySim (from git) in /opt/GSM/pysim-venv. softSIM
# itself is already installed (51-softsim.sh, host side): OSMO_SIM_SOFTSIM=0.
# Non-fatal. Skipped on the inter-STP hub (no mobile) and with --no-softsim
# (OSMO_ISO_SOFTSIM=0).
if [ "$ISO_ROLE" != "interstp" ] && [ "${OSMO_ISO_SOFTSIM:-1}" = "1" ]; then
    _SIMI=/opt/GSM/osmo-operator/tools/osmo-sim-install.sh
    if [ -f "$_SIMI" ]; then
        ( . "$_SIMI" && OSMO_SIM_SOFTSIM=0 REPO=/opt/GSM/osmo-operator osmo_sim_install ) \
            || echo "  WARN: osmo_sim_install a echoue (SIM par lecteur / pySim non poses, non fatal)"
    else
        echo "  WARN: $_SIMI absent -- pas de pcscd/pySim dans l image"
    fi
fi
```

```
# build-dep gnuradio: all GNU Radio dependencies (boost, fftw, gmp, log4cpp,
# volk...) needed by the custom gnuradio/gr-gsm in /usr/local. The Sources
# indexes are already present (deb-src.list written before apt-get update).
# The hub has no gr-gsm: no deb-src, no build-dep.
if [ -s /etc/apt/sources.list.d/deb-src.list ]; then
    apt-fast build-dep -y $APT_OPTS gnuradio || echo "WARN: apt build-dep gnuradio a echoue"
fi

echo "/usr/local/lib" > /etc/ld.so.conf.d/osmocom.conf
ldconfig

# ── venv /root/.env: must exist and provide tomli ──────────────────────
# /root/.env is the venv activated by the bridge (grgsm_exe) and the root
# profile (.bashrc: [ -f /root/.env/bin/activate ] && source ...). It is copied
# from the image with "|| true", so it may be missing or point to an absent
# interpreter, and everything would silently fall back to the system python3.
#
# python3 -m venv WITHOUT --clear repairs: it recreates bin/ and pyvenv.cfg,
# installs pip via ensurepip and keeps site-packages. It works on the copied
# venv and on a missing directory. ensurepip needs python3-venv in PKGS.
#
# tomli: the code imports tomli (not tomllib), so it is installed explicitly.
python3 -m venv /root/.env
/root/.env/bin/python3 -m pip install -q --no-cache-dir --disable-pip-version-check tomli \
    || echo "WARN: pip a echoue pour tomli dans /root/.env"
if /root/.env/bin/python3 -c "import tomli" 2>/dev/null; then
    echo "  /root/.env : venv pret, tomli importable"
else
    echo "WARN: /root/.env sans tomli utilisable"
fi

# Docker is NOT installed in the ISO: the lab runs via start-native.sh and the
# web dashboard via native node. The host build uses the host docker only to
# extract binaries and configs.

# NodeSource fallback: only if the key could not be added before the update
# (see NODE_VIA_APT above). The official script runs its own update.
if ! command -v node >/dev/null 2>&1; then
    curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
    apt-get install -y $APT_OPTS --no-install-recommends nodejs
fi

if [ -f /opt/GSM/osmo-egprs-web/package.json ]; then
    cd /opt/GSM/osmo-egprs-web && npm install --production 2>/dev/null || true
fi

# ── Dashboard installed at build time ──────────────────────────────────
# install-web-service.sh (osmo-egprs-web repository) installs the node runtime,
# JS dependencies, the systemd unit, the TLS certificate and the Firefox
# policy. Running it here makes failures visible during the build;
# osmo-egprs-web-install.service at first boot is an idempotent catch-up.
#
#   WEB_NO_TLS=1   no certificate at build time: a private key generated here
#                  would be identical in every ISO. The certificate and the
#                  Firefox policy (which names the same addresses) are created
#                  on the machine at first boot with its real addresses.
#   WEB_NO_START=1 systemd does not run in a chroot; systemctl restart would
#                  fail and the script is set -eu.
if [ -x /opt/GSM/osmo-egprs-web/install-web-service.sh ]; then
    echo "  [web] install-web-service.sh (sans TLS ni demarrage : chroot)"
    WEB_NO_TLS=1 WEB_NO_START=1 bash /opt/GSM/osmo-egprs-web/install-web-service.sh \
        || echo "  [web] WARN: install-web-service.sh a echoue - le dashboard peut manquer dans l ISO"
else
    echo "  [web] WARN: install-web-service.sh absent de /opt/GSM/osmo-egprs-web"
fi
```

```
# ── DESKTOP variant: desktop, wireshark GUI, linphone ──────────────────────
# Placed after the main apt-get install (shared deps already present), before
# update-initramfs (the desktop pulls plymouth and modules for the initrd) and
# before apt-get clean.
if [ "${ISO_DESKTOP:-0}" = "1" ]; then
    echo "  [desktop] ubuntu-desktop-minimal + wireshark + linphone-desktop"

    # WITH recommends: ubuntu-desktop-minimal puts almost everything in
    # Recommends. With --no-install-recommends it installs gnome-shell without
    # gdm3, session or terminal, giving a black screen at boot.
    #
    # linphone-desktop is the graphical client (linphone is a transition package).
    # wireshark pulls wireshark-qt. wmctrl + x11-utils (xdpyinfo): launch.sh tiles
    # the four windows into screen quarters (optional, placement only).
    #
    # One call for the desktop AND the installer (calamares, signed grub,
    # partitioning tools: see the next block). A failure stops the build: without
    # desktop or installer this image is useless.
    # snapd is not requested (the browser is a Mozilla .deb) but still comes as a
    # recommendation of ubuntu-desktop-minimal; see the snapd handling below.
    # x11-xserver-utils: xhost, used by run-as-owner (step 8i) to allow the
    # encrypted-home account on the root session X server.
    apt-fast install -y $APT_OPTS \
        ubuntu-desktop-minimal wireshark linphone-desktop \
        vlc chrome-gnome-shell \
        wmctrl x11-utils x11-xserver-utils xdotool zenity librsvg2-common \
        calamares squashfs-tools rsync dosfstools efibootmgr os-prober \
        cryptsetup cryptsetup-initramfs lvm2 pciutils ubuntu-drivers-common \
        conky-all fonts-dejavu fonts-ubuntu screenfetch python3-pil python3-gi python3-gi-cairo python3-cairo
↪   gir1.2-gtk-3.0 gir1.2-vte-2.91 \
        grub2-common grub-efi-amd64-bin grub-efi-amd64-signed shim-signed grub-pc-bin \
        qml-module-qtquick2 qml-module-qtquick-layouts \
        qml-module-qtquick-window2 qml-module-qtquick-controls

    # ── Avahi: purged ──────────────────────────────────────────────────────
    # avahi comes only as a Recommends of ubuntu-desktop-minimal. On a GSM bench
    # avahi-daemon broadcasts mDNS (224.0.0.251:5353) on every interface, apn0 and
    # docker veths included, polluting GSMTAP and SIP captures; all addresses here
    # are static. --purge with libnss-mdns, and the mdns entries are removed from
    # /etc/nsswitch.conf so lookups do not query an absent service.
    apt-fast purge -y $APT_OPTS avahi-daemon avahi-utils avahi-autoipd libnss-mdns 2>/dev/null \
        || echo "  [desktop] avahi deja absent"
    apt-fast autoremove -y $APT_OPTS 2>/dev/null || true
    # Masked so that a future dependency cannot start it again.
    systemctl mask avahi-daemon.service avahi-daemon.socket 2>/dev/null || true
    # Double quotes only, no apostrophe (this block is inside a single-quoted
    # bash -c string, and bash -n on build-iso.sh cannot detect a stray one).
    sed -i "s/[[:space:]]*mdns4_minimal[[:space:]]*\[NOTFOUND=return\]//; s/[[:space:]]*mdns4//" \
        /etc/nsswitch.conf 2>/dev/null || true
    echo "  [desktop] avahi purge (mDNS retire du banc)"

    # ── Installer: Calamares (Ubiquity needs casper, this image uses live-boot) ──
    # Calamares is told where the squashfs is. Its configuration lives in the
    # repository (installer/calamares/) and is copied into the rootfs outside this
    # chroot (the files contain apostrophes).
    #
    # Each dependency covers a step:
    #   squashfs-tools  unpackfs, which copies the system to disk
    #   dosfstools      the FAT EFI partition
    #   efibootmgr      the UEFI boot entry
    #   os-prober       other systems, for the GRUB menu
    #   pciutils        lspci: NVIDIA card detection (osmo-install
    #   ubuntu-drivers  and contextualprocess@nvidia); ubuntu-drivers picks
    #   -common         the recommended nvidia-driver-5xx
    #   conky-all       the bench Conky dashboard (configs/conky/,
```

```
#   python3-gi      the clickable live panel (tools/osmo-panel.py, GTK)
#   gir1.2-gtk-3.0
#   fonts-dejavu    GNOME autostart set below), live and installed disk
#   python3-gi-     the c123 window (tools/osmo-c123-gtk.py): import cairo
#   cairo (+cairo)  and Gdk.cairo_set_source_pixbuf, the emulated C123 screen
#   lvm2            LVM in the Calamares manual partitioning (volume groups,
#                   LUKS on LVM); its initramfs hook follows
#   cryptsetup      LUKS: the installer encryption checkbox (partition.conf);
#   (+ -initramfs)  cryptsetup-initramfs installs the hook that lets the target
#                   initrd open an encrypted root
#   qml-module-*    the slideshow during the copy
# These packages are in the desktop apt-get install above (a failure aborts).
# Here we also check that the binary exists, in case apt exits 0 without it.
# We are inside the chroot: the binary is /usr/bin/calamares (no $ROOTFS here).
if [ ! -x /usr/bin/calamares ]; then
    echo "ERREUR: calamares absent du rootfs apres apt-get - build DESKTOP interrompu." >&2
    echo "        (dependance Qt/KDE manquante au miroir ? relancer avec un cache .deb)" >&2
    exit 1
fi

# ── Tools Calamares calls but does not depend on ──────────────────────
# unpackfs runs unsquashfs and then rsync; without rsync the install fails
# after partitioning with "error code 127" (command not found). Checked at
# build time.
for _t in rsync unsquashfs; do
    if ! command -v "$_t" >/dev/null 2>&1; then
        echo "ERREUR: $_t absent du rootfs - unpackfs echouerait a l installation." >&2
        exit 1
    fi
done

# ── GRUB must be in the target, not only on the build host ────────────
# The calamares bootloader module runs grub-install INSIDE the installed
# system (this rootfs); grub-install comes from grub2-common, not grub-common.
# The signed binaries (shimx64, grubx64) are needed too: bootloader.conf sets
# efiBootloaderId=ubuntu so the installed system boots with Secure Boot.
for _t in grub-install grub-mkconfig grub-probe; do
    if ! command -v "$_t" >/dev/null 2>&1; then
        echo "ERREUR: $_t absent du rootfs - le module bootloader echouerait." >&2
        exit 1
    fi
done
for _f in /usr/lib/grub/x86_64-efi-signed/grubx64.efi.signed \
          /usr/lib/shim/shimx64.efi.signed; do
    if [ ! -e "$_f" ]; then
        echo "ERREUR: $_f absent - le systeme installe ne demarrerait pas en Secure Boot." >&2
        exit 1
    fi
done

# ── Firefox: the Mozilla .deb, not the snap ───────────────────────────
# The Ubuntu "firefox" package is a snap transition package, unusable in a
# chroot. Mozilla publishes .deb packages (packages.mozilla.org), pinned above
# the Ubuntu one and kept up to date by apt. The desktop file is
# firefox.desktop and /etc/firefox/policies/policies.json is read directly.
apt-fast purge -y firefox chromium-browser 2>/dev/null || true
install -d -m0755 /etc/apt/keyrings
_ff_ok=0
if curl -fsSL --retry 3 https://packages.mozilla.org/apt/repo-signing-key.gpg \
     -o /etc/apt/keyrings/packages.mozilla.org.asc; then
    echo "deb [signed-by=/etc/apt/keyrings/packages.mozilla.org.asc] https://packages.mozilla.org/apt mozilla
↪ main" \
        > /etc/apt/sources.list.d/mozilla.list
    printf "Package: *\nPin: origin packages.mozilla.org\nPin-Priority: 1000\n" \
        > /etc/apt/preferences.d/mozilla
    apt-get update -qq
```

```
        if apt-fast install -y $APT_OPTS firefox; then
            _ff_ok=1
            # The UI language follows the keyboard chosen at build time (--kb).
            apt-fast install -y $APT_OPTS "firefox-l10n-${OSMO_ISO_KB:-fr}" 2>/dev/null \
                || echo "  [desktop] firefox-l10n-${OSMO_ISO_KB:-fr} indisponible - interface en anglais"
        fi
    fi
    # ── Start page ──────────────────────────────────────────────────────
    # Set as a DEFAULT preference (defaults/pref/), not a policy: policies.json
    # belongs to install-web-service.sh and Firefox reads only one. The operator
    # can still change it. OSMO_FF_HOME selects it at build time.
    if [ "$_ff_ok" = "1" ] && [ -d /usr/lib/firefox/defaults/pref ]; then
        cat > /usr/lib/firefox/defaults/pref/osmo-homepage.js <<FFPREF
// Pose par build-iso.sh (iso_modules/80-chroot.sh). Preference PAR DEFAUT :
// modifiable dans Firefox, remise a cette valeur sur un profil neuf.
pref("browser.startup.homepage", "${OSMO_FF_HOME:-https://pl4y.store}");
pref("browser.startup.page", 1);
FFPREF
        echo "  [desktop] Firefox : page de demarrage ${OSMO_FF_HOME:-https://pl4y.store}"
    fi
    if [ "$_ff_ok" = "1" ]; then
        echo "  [desktop] Firefox : $(dpkg-query -W -f=\${Version} firefox 2>/dev/null) (deb Mozilla,
↪   packages.mozilla.org)"
    else
        echo "  [desktop] WARN: Firefox (deb Mozilla) NON installe - packages.mozilla.org injoignable ?"
    fi
    # No snaps in the image: disable snap seeding and remove the snap install unit
    # that older images installed (it would fail at boot on a cached rootfs).
    rm -rf /var/lib/osmo-snaps
    rm -f /etc/systemd/system/osmo-firefox-snap.service \
          /etc/systemd/system/multi-user.target.wants/osmo-firefox-snap.service \
          /usr/local/sbin/osmo-firefox-snap
    # ── snapd stays usable ──────────────────────────────────────────────
    # The image ships no snaps, but snapd remains available through SOCKET
    # activation: it starts only when someone talks to it. snapd.seeded returns
    # immediately without a seed.
    systemctl unmask snapd.service snapd.socket snapd.seeded.service snapd.apparmor.service 2>/dev/null || true
    systemctl enable snapd.socket snapd.apparmor.service 2>/dev/null || true
    # snapd.service is not enabled: the socket starts it on demand.
    systemctl disable snapd.service 2>/dev/null || true

    systemctl set-default graphical.target

    # ── Audio: system-wide PulseAudio, also on noble ────────────────────
    # ubuntu-desktop-minimal on noble depends on pipewire-pulse; the bench relies on
    # the SYSTEM pulseaudio daemon (/run/pulse/native, osmo-pulse.service: gsm_audio
    # sink, gapk). Installing pulseaudio removes the desktop METAPACKAGE only; the
    # desktop packages are then marked manual so autoremove keeps them.
    #
    # pulseaudio and pipewire-pulse are co-installable on noble, so pipewire-pulse
    # must be purged explicitly: otherwise it holds /run/user/<uid>/pulse/native,
    # and since wireplumber has no ALSA access (51-osmo-no-alsa.lua, the cards
    # belong to the system daemon) the session only sees "Dummy Output" and is
    # silent. This shows only for non-root users (installed disk).
    #
    # pipewire-pulse is purged, its user units masked (it may come back with a
    # desktop update) and the result verified. pipewire itself stays (GNOME
    # screencast). The session is connected to the system daemon by
    # /etc/pulse/client.conf.d and a user unit, both installed by
    # iso_modules/82-services.sh.
    if [ "$_S" = "noble" ]; then
        _auto=$(apt-mark showauto)
        apt-fast install -y $APT_OPTS pulseaudio pulseaudio-utils \
            || echo "  [desktop] WARN: apt n a pas pu remettre pulseaudio (conflit pipewire-pulse ?)"
        if [ -n "$_auto" ]; then apt-mark manual $_auto >/dev/null 2>&1 || true; fi
        # Explicit purge: installing pulseaudio does not remove it.
```

```
        apt-get purge -y pipewire-pulse pipewire-media-session >/dev/null 2>&1 || true
        systemctl --global mask pulseaudio.service pulseaudio.socket 2>/dev/null || true
        # Masked in case a desktop update reinstalls pipewire-pulse: the socket
        # /run/user/<uid>/pulse/native must stay free for the link to the system daemon.
        systemctl --global mask pipewire-pulse.service pipewire-pulse.socket 2>/dev/null || true
        # wireplumber stays (GNOME screencast uses PipeWire) but must not take the
        # sound cards: they belong to the pulseaudio daemon.
        mkdir -p /etc/wireplumber/main.lua.d
        printf "alsa_monitor.enabled = false\n" > /etc/wireplumber/main.lua.d/51-osmo-no-alsa.lua
        if [ -x /usr/bin/pulseaudio ]; then
            echo "  [desktop] audio : pulseaudio systeme retabli ($(dpkg-query -W -f=\${Version} pulseaudio)),
↪   wireplumber sans ALSA"
        else
            echo "  [desktop] WARN: /usr/bin/pulseaudio toujours absent - osmo-pulse.service echouera"
        fi
        # Verify that pipewire-pulse is really gone.
        if [ -x /usr/bin/pipewire-pulse ]; then
            echo "  [desktop] WARN: pipewire-pulse TOUJOURS PRESENT - il prendra /run/user/<uid>/pulse/native et
↪   le bureau sera muet"
        else
            echo "  [desktop] audio : pipewire-pulse purge (pipewire reste pour le screencast)"
        fi
    fi

    # ── NetworkManager: enabled ──────────────────────────────────────
    # systemd-networkd keeps the bench interfaces (apn0, packet core tun/veth);
    # NetworkManager manages the physical cards (Wi-Fi, VPN, manual switching).
    # The split is defined in /etc/NetworkManager/conf.d/10-osmo-networkd.conf,
    # written outside this chroot. Without it both would fight over the same card.
    systemctl unmask NetworkManager 2>/dev/null || true
    systemctl enable NetworkManager 2>/dev/null || true
    # ── No network wait at boot ──────────────────────────────────────
    # The wait-online units can delay boot by minutes: bench interfaces (apn0,
    # tun/veth) are never "online" in their sense and the WAN cable is often
    # unplugged. Nothing we start needs a default route first (bench units use
    # Restart=on-failure, osmo-update exits 0 offline).
    # They are MASKED (not just disabled) because third-party units (apt-daily,
    # snapd...) pull network-online.target, which then becomes immediate. This
    # also avoids an extra delay at shutdown.
    systemctl disable NetworkManager-wait-online.service systemd-networkd-wait-online.service 2>/dev/null || true
    systemctl mask    NetworkManager-wait-online.service systemd-networkd-wait-online.service 2>/dev/null || true

    # ── Autologin ────────────────────────────────────────────────────
    # Directly as root (there is no separate "osmocom" account). GDM refuses
    # sessions for uid 0 through PAM, not gdm3.conf:
    #     auth required pam_succeed_if.so user != root quiet_success
    # That line is commented out, otherwise no login works at all.
    sed -i "/pam_succeed_if.so user != root quiet_success/s/^/#/" \
        /etc/pam.d/gdm-password /etc/pam.d/gdm-autologin 2>/dev/null || true
    mkdir -p /etc/gdm3
    cat > /etc/gdm3/custom.conf <<GDM
[daemon]
AutomaticLoginEnable=true
AutomaticLogin=root
# X11 forced: under VirtualBox/QEMU the GNOME Wayland session falls back to
# llvmpipe and is unusable.
WaylandEnable=false
GDM

    # ── PipeWire for the root session ────────────────────────────────
    # GNOME screencast (Ctrl+Alt+Shift+R) requires PipeWire, and the upstream
    # pipewire.socket/pipewire.service units carry ConditionUser=!root, so the
    # root autologin session would have no /run/user/0/pipewire-0.
    # An EMPTY assignment resets the inherited conditions (the systemd way to
    # cancel them). /etc/systemd/user/ applies to every account.
    # Known limitation: PipeWire as root is not supported upstream. On noble the
```

```
    # wireplumber ALSA monitor is disabled above (51-osmo-no-alsa.lua) so it does
    # not compete with the bench pulseaudio --system.
    for _pwu in pipewire.socket pipewire.service; do
        mkdir -p "/etc/systemd/user/${_pwu}.d"
        # Double quotes: this block is inside single quotes.
        printf "[Unit]\nConditionUser=\n" > "/etc/systemd/user/${_pwu}.d/10-allow-root.conf"
    done
    unset _pwu

    # The first-login assistant would run at EVERY boot on a non-persistent live:
    # disable it.
    rm -f /etc/xdg/autostart/gnome-initial-setup-first-login.desktop
    for h in /root; do
        mkdir -p "$h/.config" && echo yes > "$h/.config/gnome-initial-setup-done"
    done

    # Screen lock and suspend disabled: the bench screen stays on during captures
    # and calls, and a live session has no chosen password. The keyboard layout
    # follows --kb. printf instead of a heredoc: gschema values need apostrophes
    # (written as \047 inside this single-quoted block).
    # ── Conky arrows on the keyboard ──────────────────────────────
    # Conky windows ignore the pointer, so the operator selection shown by Conky
    # is changed with Ctrl+AltGr+Right (next) and Ctrl+AltGr+Left (previous).
    # Ctrl+Alt+arrows stay with GNOME workspaces; AltGr is ISO_Level3_Shift,
    # <Mod5> in GTK accelerator syntax. osmo-op writes /run/osmo-fft/operator,
    # read by the panel, osmo-fft-snap.py and Conky.
    printf "[org.gnome.settings-daemon.plugins.media-keys]\ncustom-keybindings=[\047/org/gnome/settings-
↪   daemon/plugins/media-keys/custom-keybindings/osmo-op-next/\047,\047/org/gnome/settings-daemon/plugins/media-
↪   keys/custom-keybindings/osmo-op-prev/\047]\n\n[org.gnome.settings-daemon.plugins.media-keys.custom-
↪   keybinding:/org/gnome/settings-daemon/plugins/media-keys/custom-keybindings/osmo-op-next/]\nname=\047osmo-
↪   operator : arret suivant (Conky)\047\ncommand=\047/usr/local/bin/osmo-op
↪   --next\047\nbinding=\047<Control><Mod5>Right\047\n\n[org.gnome.settings-daemon.plugins.media-keys.custom-
↪   keybinding:/org/gnome/settings-daemon/plugins/media-keys/custom-keybindings/osmo-op-prev/]\nname=\047osmo-
↪   operator : arret precedent (Conky)\047\ncommand=\047/usr/local/bin/osmo-op
↪   --prev\047\nbinding=\047<Control><Mod5>Left\047\n\n" \
        > /usr/share/glib-2.0/schemas/98-osmo-keys.gschema.override

    printf "[org.gnome.desktop.session]\nidle-delay=uint32 0\n\n[org.gnome.desktop.screensaver]\nlock-
↪   enabled=false\nidle-activation-enabled=false\n\n[org.gnome.settings-daemon.plugins.power]\nsleep-inactive-ac-
↪   type=\047nothing\047\nsleep-inactive-battery-type=\047nothing\047\n\n[org.gnome.desktop.input-
↪   sources]\nsources=[(\047xkb\047,\047%s\047)]\n"
↪   \
        "${OSMO_ISO_KB:-fr}" > /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
    # ── Light mode, Yaru blue ─────────────────────────────────────
    # Blue accent (Yaru-blue icons), light color scheme by default, same as the
    # native install (install_modules/80-bureau.sh); dark mode is available in
    # Settings > Appearance. The :ubuntu sections duplicate the plain one
    # (10_ubuntu-settings.gschema.override sets gtk-theme=Yaru there and the ubuntu
    # session reads it first); :GNOME-Greeter applies the theme to the gdm screen.
    for _sec in "org.gnome.desktop.interface" "org.gnome.desktop.interface:ubuntu"
↪   "org.gnome.desktop.interface:GNOME-Greeter"; do
        printf "\n[%s]\ncolor-scheme=\047default\047\ngtk-theme=\047Yaru-blue\047\nicon-theme=\047Yaru-
↪   blue\047\ncursor-theme=\047Yaru\047\n" "$_sec"
↪   \
            >> /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
    done
    unset _sec
    # ── GSM LAB wallpaper ─────────────────────────────────────────
    # Set as the session DEFAULT through the schema (a user setting would not
    # survive a non-persistent live boot).
    #   gsm-lab-ville.jpg     idle wallpaper and schema default
    #   gsm-lab-antenne.jpg   same scene with the 2G/4G tower, faded in by
    #                         tools/osmo-fond.py when a 2G or 4G network starts
    #                         (osmo-launcher.py, osmo-panel.py); back to idle on stop
    #   gsm-lab-fond.jpg      low-poly landscape (tools/paysage-render.py), fallback
    #   gsm-lab-wallpaper.jpg tower illustration, also installed
```

```
    # 85-installeur-bureau.sh reinstalls the files from the local repository; the
    # schema names the wallpaper regardless. The native install
    # (install_modules/80-bureau.sh) uses the same wallpaper.
    _WP=/opt/GSM/osmo-operator/configs/gsm-lab-wallpaper.jpg
    _FOND=/opt/GSM/osmo-operator/configs/gsm-lab-fond.jpg
    _VILLE=/opt/GSM/osmo-operator/configs/gsm-lab-ville.jpg
    [ -f "$_WP" ] && install -Dm644 "$_WP" /usr/share/backgrounds/gsm-lab-wallpaper.jpg
    [ -f "${_VILLE%ville.jpg}antenne.jpg" ] \
        && install -Dm644 "${_VILLE%ville.jpg}antenne.jpg" /usr/share/backgrounds/gsm-lab-antenne.jpg
    if [ -f "$_FOND" ]; then
        install -Dm644 "$_FOND" /usr/share/backgrounds/gsm-lab-fond.jpg
    elif [ -f "$_WP" ]; then
        install -Dm644 "$_WP" /usr/share/backgrounds/gsm-lab-fond.jpg
    fi
    if [ -f "$_VILLE" ]; then
        install -Dm644 "$_VILLE" /usr/share/backgrounds/gsm-lab-ville.jpg
        _DEFAUT=/usr/share/backgrounds/gsm-lab-ville.jpg
    else
        _DEFAUT=/usr/share/backgrounds/gsm-lab-fond.jpg
    fi
    if [ -f "$_WP" ] || [ -f "$_FOND" ] || [ -f "$_VILLE" ]; then
        # Three sections: the plain value plus the [schema:ubuntu] and
        # [schema:GNOME-Greeter] desktop overrides from
        # 10_ubuntu-settings.gschema.override, which an ubuntu:GNOME session reads
        # first (otherwise the greeter and DING would use the Ubuntu default).
        for _sec in "org.gnome.desktop.background" "org.gnome.desktop.background:ubuntu"
↪   "org.gnome.desktop.background:GNOME-Greeter"; do
            printf "\n[%s]\npicture-uri=\047file://%s\047\npicture-uri-dark=\047file://%s\047\npicture-
↪   options=\047zoom\047\nprimary-color=\047#0d1b2a\047\n" "$_sec" "$_DEFAUT" "$_DEFAUT"
↪   \
                >> /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
        done
        unset _sec
        # Desktop icons (DING) in the top-left corner, where the wallpaper leaves room.
        for _sec in "org.gnome.shell.extensions.ding" "org.gnome.shell.extensions.ding:ubuntu"; do
            printf "\n[%s]\nstart-corner=\047top-left\047\nshow-trash=false\nshow-volumes=false\n" "$_sec" \
                >> /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
        done
        unset _sec
        echo "  [desktop] fond d ecran GSM LAB pose : $_DEFAUT"
    else
        echo "  [desktop] WARN: $_WP absent -- fond d ecran GNOME par defaut"
    fi
    # ── Dock: bench favorites ──────────────────────────────────────
    # Set as the schema DEFAULT (non-persistent live) and also in
    # [org.gnome.shell:ubuntu], which 10_ubuntu-settings fills with the Ubuntu list
    # and the ubuntu:GNOME session reads first.
    # Entries pointing to a missing .desktop are silently ignored by GNOME Shell,
    # so deka.desktop and claude.desktop may be listed even when not installed
    # (deka comes from addition.sh). firefox.desktop is the Mozilla .deb one;
    # osmo-pmos.desktop (postmarketOS/Phosh VM) comes from
    # tools/osmo-extras-install.sh.
    for _sec in "org.gnome.shell" "org.gnome.shell:ubuntu"; do
        printf "\n[%s]\nfavorite-apps=[\047osmo-info.desktop\047, \047firefox.desktop\047,
↪   \047org.gnome.Nautilus.desktop\047, \047claude.desktop\047, \047osmo-launch.desktop\047,
↪   \047osmo-install.desktop\047, \047osmo-tutorial.desktop\047, \047osmo-addition.desktop\047,
↪   \047osmo-multi.desktop\047, \047osmo-update.desktop\047, \047osmo-paint.desktop\047, \047deka.desktop\047,
↪   \047linphone.desktop\047, \047osmo-wireshark-root.desktop\047, \047osmo-pmos.desktop\047,
↪   \047osmo-lte.desktop\047]\n" "$_sec" \
            >> /usr/share/glib-2.0/schemas/99-osmo-live.gschema.override
    done
    unset _sec
    echo "  [desktop] favoris du dock poses (14 entrees)"

    # ── Extras (media, tools) in the native image ──────────────────
    # Same set and code as addition.sh (tools/osmo-extras-install.sh): Kodi,
```

```
    # YouTube + uBlock, Wireshark (root), Linphone, grouped in Games / Media /
    # Phone / Tools folders. Non-fatal. Runs as root in the chroot with the
    # repository at /opt/GSM/osmo-operator.
    _EXTRAS=/opt/GSM/osmo-operator/tools/osmo-extras-install.sh
    if [ -f "$_EXTRAS" ]; then
        echo "  [desktop] jeux + media (natif) : $_EXTRAS"
        # Games (Doom/Quake III, ~470 MB) are excluded: they are installed only by
        # addition.sh --extras on an installed machine.
        OSMO_EXTRAS_JEUX=0 REPO=/opt/GSM/osmo-operator . "$_EXTRAS" && OSMO_EXTRAS_JEUX=0 osmo_extras_install || \
            echo "  [desktop] WARN: osmo_extras_install a echoue (non fatal)"
    else
        echo "  [desktop] WARN: $_EXTRAS absent -- pas de jeux/media dans l image"
    fi

    glib-compile-schemas /usr/share/glib-2.0/schemas 2>/dev/null || true

    echo "  [desktop] GNOME pret : autologin root, X11, NetworkManager actif, Firefox deb Mozilla"
fi

# -- CA certificates: final check ------------------------------------------
# The store was regenerated at the start of this chroot; just check it is
# not empty.
if [ -s /etc/ssl/certs/ca-certificates.crt ]; then
    echo "  certificats : $(grep -c "BEGIN CERTIFICATE" /etc/ssl/certs/ca-certificates.crt) autorites dans
 ↪  /etc/ssl/certs"
else
    echo "  WARN: /etc/ssl/certs/ca-certificates.crt vide - le TLS echouera dans l image"
fi

setcap cap_net_raw,cap_net_admin+eip $(which dumpcap) 2>/dev/null || true

KERNEL=$(ls /boot/vmlinuz-* | sort -V | tail -1 | sed "s|/boot/vmlinuz-||")
update-initramfs -u -k "$KERNEL"

# Remove deb-src.list: its Sources indexes (~80 MB) were only needed for the
# gnuradio build-dep and would be loaded into RAM by the first apt-get update
# on a toram live. No "apt-get clean" when /var/cache/apt/archives is the
# bind-mounted host cache: only indexes and binary caches are removed.
if [ "${ISO_APT_CACHE_BOUND:-0}" = "1" ]; then
    rm -f /var/cache/apt/*.bin; rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
else
    apt-get clean; rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
fi
rm -f /etc/apt/sources.list.d/deb-src.list
'


# End of module: sourcing returns the status of the LAST command and
# build-iso.sh runs under set -e, so always end with a true status.
true
```

2.74 /opt/GSM/osmo-operator/iso_modules/81-cloture-systeme.sh

22973 octets, 469 lignes → 467 lignes (1 groupes compactés)

```
#!/bin/bash
# iso_modules/81-cloture-systeme.sh - step 8b: ldd closure from the run image, network, osmo-update
# Sourced by build-iso.sh in numeric order (same shell, variables and functions);
# not meant to run on its own. A top-level `return` means "nothing to do here"
# (this is how --arm skips a step).

# -- Step 8b: ldd dependency closure copied from the run image ---------------
# The osmocom-run image (build.sh + Dockerfile) is the reference environment.
# Instead of relying on the rootfs apt versions (version skew breaks libosmocore
```

```
# logging), copy the exact .so closure of all osmo binaries and
# calypso-ipc-device from the container, overwriting the apt libs. The
# glibc/loader family is excluded (never clobber ld.so).
echo -e "${GREEN}[8b/9] Cloture de dependances COMPLETE depuis ${CYAN}${ISO_RUN_IMAGE}${NC}${GREEN} (toute
 ↪  l'install)...${NC}"
# ldd every ELF (executables and .so) of the custom install: /usr/local/bin
# (osmo), /opt/GSM (qemu, ipc-device, gr-gsm), /root/.env (Python venv:
# gnuradio/gr-gsm bindings and their boost/log4cpp/volk/fftw deps).
docker run --rm --entrypoint bash \
    -e OSMO_LDD_ROOTS="$([ "$ISO_ROLE" = "interstp" ] && echo "/usr/local/bin /usr/local/lib" || echo
     ↪  "/usr/local/bin /opt/GSM /root/.env")" \
    "$ISO_RUN_IMAGE" -c '
    set -e
    find ${OSMO_LDD_ROOTS:-/usr/local/bin /opt/GSM /root/.env} -type f \( -executable -o -name "*.so*" \)
 ↪  2>/dev/null \
      | while read -r b; do ldd "$b" 2>/dev/null; done \
      | grep -oE "/[^ ]+\.so[^ ]*" | sort -u \
      | grep -vE "/(ld-linux[^/]*|ld|libc|libm|libpthread|libdl|librt|libresolv)\.so" \
      | while read -r f; do realpath "$f" 2>/dev/null; done | sort -u \
      | tar -czf - -T - 2>/dev/null
' > "$WORK/closure.tar.gz" || true
if [ -s "$WORK/closure.tar.gz" ]; then
    tar -xzf "$WORK/closure.tar.gz" -C "$ROOTFS" 2>/dev/null || true
    # List of the overwritten libs, for 90-iso.sh: they intentionally replace
    # apt's, so their dpkg md5 no longer matches; the squashfs verification
    # uses this list to not report them as altered. Relative paths, as in .md5sums.
    tar -tzf "$WORK/closure.tar.gz" 2>/dev/null \
        | sed 's#^\./##; s#^/##' | sort -u > "$WORK/closure.list" || true
    echo -e "  ${GREEN}✓${NC} $(wc -l < "$WORK/closure.list") libs injectees (Docker)"
else
    echo -e "  ${YELLOW}cloture vide - on garde les libs apt${NC}"
fi

# /usr/local/lib (custom libosmo*) takes priority; system duplicates are purged.
# No `| grep` here: under set -euo pipefail a grep with no match (the normal
# case: no duplicate) would return 1 and abort the build.
echo "/usr/local/lib" > "$ROOTFS/etc/ld.so.conf.d/00-osmocom-local.conf"
find "$ROOTFS/usr/lib" "$ROOTFS/lib" -maxdepth 4 -name 'libosmo*.so*' -delete 2>/dev/null || true
chroot "$ROOTFS" ldconfig 2>/dev/null || true
echo -e "  ${GREEN}✓${NC} /usr/local/lib prioritaire + ldconfig"

# ── Configuration systeme ──────────────────────────────────────────
echo "osmo-egprs" > "$ROOTFS/etc/hostname"
cat > "$ROOTFS/etc/hosts" <<'EOF'
127.0.0.1 localhost osmo-egprs
::1       localhost
EOF

mkdir -p "$ROOTFS/etc/systemd/network"
cat > "$ROOTFS/etc/systemd/network/20-dhcp.network" <<'EOF'
[Match]
Name=en* eth*
[Network]
DHCP=yes
# ── Legacy docker-plan addresses, as /32 ──────────────────────────
# Image configs still reference 172.20.x (backbone gateway, gsmtap target...);
# daemons binding to them need the addresses to exist locally.
#
# /32 rather than /24: a /24 makes the machine ARP the whole 172.20.0.0/24 on
# the LAN instead of routing it, which breaks reaching host containers on a
# mixed VM + docker bench ("ip route add ... via <host>" fails with "File exists").
#
# 172.20.0.11 is intentionally absent: it is the first operator container's
# address; a VM holding it would answer itself. No /16 route for the same reason.
#
# The private node addresses are set at runtime by osmo-ip-plan.service on the
```

```
# interface that holds the default route (see network/osmo-ip-plan.sh).
EOF
# Units are enabled one by one: 'systemctl enable' validates all listed units
# first, so one missing unit would prevent enabling the others.
chroot "$ROOTFS" systemctl enable systemd-networkd 2>/dev/null||true
chroot "$ROOTFS" systemctl enable systemd-resolved 2>/dev/null||true

# ── DNS: do not ship the build host's resolv.conf ──────────────────────
# 80-chroot.sh copies the host's /etc/resolv.conf so apt in the chroot has DNS;
# on Ubuntu hosts it is the systemd-resolved stub (127.0.0.53). Restore the
# standard Ubuntu symlink to the stub file that systemd-resolved writes in /run
# at each boot (systemd-resolved is installed and enabled above).
ln -sfn ../run/systemd/resolve/stub-resolv.conf "$ROOTFS/etc/resolv.conf"
# Fallback: if systemd-resolved does not start, the symlink dangles and there
# is no DNS. Leave a backup copy that NetworkManager or an operator can restore.
printf '# Repli hors systemd-resolved : cp %s /etc/resolv.conf
nameserver 1.1.1.1
nameserver 8.8.8.8
' \
    /etc/resolv.conf.secours > "$ROOTFS/etc/resolv.conf.secours"
echo -e "  ${GREEN}✓${NC} DNS : /etc/resolv.conf -> stub systemd-resolved (repli /etc/resolv.conf.secours)"

# ── No waiting for the network, at boot or shutdown ────────────────────
# The *-wait-online units can delay boot by up to 2 minutes; the bench
# interfaces (apn0, packet-core tun/veth) are never "online" in their sense
# and the WAN cable is not always plugged in. Nothing started here needs a
# default route first: bench units retry, osmo-update exits 0 offline, and
# osmo-ip-plan is replayed by the dispatchers when a link comes up.
#
# Masked, not just disabled: network-online.target is pulled in by third-party
# units (apt-daily, snapd, cloud-init); with the wait-online units masked the
# target is reached immediately. Applies to all variants (operator, inter-STP,
# lite), not only --desktop.
install -d "$ROOTFS/etc/systemd/system"
for _wo in systemd-networkd-wait-online.service NetworkManager-wait-online.service \
           systemd-networkd-wait-online@.service; do
    ln -sfn /dev/null "$ROOTFS/etc/systemd/system/$_wo"
done
unset _wo

# ── Bounded shutdown ───────────────────────────────────────────────────
# systemd defaults to 90 s per stuck unit. 30 s is enough for a clean stop;
# beyond that SIGKILL is the right answer. DefaultTimeoutStartSec likewise
# at boot. Services that need more declare it themselves.
install -d "$ROOTFS/etc/systemd/system.conf.d"
cat > "$ROOTFS/etc/systemd/system.conf.d/10-osmo-timeouts.conf" <<'SDTO'
# Installed by build-iso.sh (iso_modules/81-cloture-systeme.sh).
# Neither boot nor shutdown should wait for the network or a stuck service.
# The wait-online units are masked next to this file.
[Manager]
DefaultTimeoutStartSec=45s
DefaultTimeoutStopSec=30s
SDTO
echo -e "  ${GREEN}✓${NC} aucune attente reseau : ${CYAN}*-wait-online${NC} masquees · arret plafonne a ${CYAN}30
 ↪  s${NC}"

# ── Private node addresses, set at runtime ─────────────────────────────
# network/osmo-ip-plan.sh picks the interface that actually provides Internet,
# assigns 192.168.<node+1>.1 and .10 to it, and falls back to 127.0.0.66 on lo
# when no interface leads anywhere, so configs that name a private address
# always find something local to bind to.
install -Dm755 "$DIR/network/osmo-ip-plan.sh" "$ROOTFS/usr/local/sbin/osmo-ip-plan.sh"
cat > "$ROOTFS/etc/systemd/system/osmo-ip-plan.service" <<'IPPLAN'
[Unit]
Description=Adresses privees du noeud sur la carte qui fournit Internet
# Not ordered after network-online.target (see above): the first run may
```

```
# fall back to 127.0.0.66; both dispatchers below replay the plan as soon as
# an interface becomes routable (networkd-dispatcher for systemd-networkd
# interfaces, NetworkManager-dispatcher for physical NICs).
After=network.target systemd-networkd.service
Before=osmo-egprs-web.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/sbin/osmo-ip-plan.sh --apply

[Install]
WantedBy=multi-user.target
IPPLAN
# Replayed on every link change (cable replugged, Wi-Fi taking over), since
# the interface providing Internet may change.
mkdir -p "$ROOTFS/etc/networkd-dispatcher/routable.d"
cat > "$ROOTFS/etc/networkd-dispatcher/routable.d/50-osmo-ip-plan" <<'IPHOOK'
#!/bin/sh
exec /usr/local/sbin/osmo-ip-plan.sh --apply
IPHOOK
chmod +x "$ROOTFS/etc/networkd-dispatcher/routable.d/50-osmo-ip-plan"
# Same hook for NetworkManager: networkd-dispatcher only sees systemd-networkd
# interfaces, while NetworkManager manages the physical NICs
# (iso_modules/80-chroot.sh).
mkdir -p "$ROOTFS/etc/NetworkManager/dispatcher.d"
cat > "$ROOTFS/etc/NetworkManager/dispatcher.d/50-osmo-ip-plan" <<'NMHOOK'
#!/bin/sh
# $1 = interface, $2 = action. Only link-up events matter.
case "$2" in
    up|dhcp4-change|dhcp6-change|connectivity-change) ;;
    *) exit 0 ;;
esac
exec /usr/local/sbin/osmo-ip-plan.sh --apply
NMHOOK
chmod +x "$ROOTFS/etc/NetworkManager/dispatcher.d/50-osmo-ip-plan"
chroot "$ROOTFS" systemctl enable osmo-ip-plan 2>/dev/null || true
echo -e "  ${GREEN}✓${NC} osmo-ip-plan : ${CYAN}192.168.$(( ${ISO_NODE:-1} + 1 )).1/.10${NC} sur la carte
 ↪  Internet, repli ${CYAN}127.0.0.66${NC}"

# live-boot writes /root/etc/network/interfaces in the root mounted at boot;
# without ifupdown /etc/network/ does not exist and /init fails with
# "nonexistent directory". Create the directory and a minimal (loopback)
# interfaces file. systemd-networkd manages the network; nobody reads this
# file, it only satisfies the live-boot hook.
mkdir -p "$ROOTFS/etc/network"
cat > "$ROOTFS/etc/network/interfaces" <<'EOF'
auto lo
iface lo inet loopback
EOF

# ── SMS animation at login ──────────────────────────────────────────
# The repositories, packages and dashboard service are all provisioned at
# build time; the only interactive part of update.sh left is the SMS animation.
#
# It is played by the login profile, not a service (a systemd oneshot runs
# before any terminal exists). /etc/profile.d is sourced alphabetically:
# 01-osmo-disclaimer.sh first, then 99-osmo-sms.sh.
install -Dm755 "$DIR/update.sh" "$ROOTFS/usr/local/sbin/osmo-sms.sh"
cat > "$ROOTFS/etc/profile.d/99-osmo-sms.sh" <<'EOF'
# 99-osmo-sms.sh - installed by build-iso.sh. Plays an incoming SMS once per
# boot. Sourced after 01-keyboard-setup.sh (alphabetical order).
[ -n "${BASH_VERSION:-}" ] || return 0
case $- in *i*) ;; *) return 0 ;; esac      # session interactive seulement
[ -x /usr/local/sbin/osmo-sms.sh ] || return 0
# /run is a tmpfs recreated at each boot: the animation plays once per boot,
```

```
# not on every tty or "su -".
[ -e /run/osmo-sms.done ] && return 0
: > /run/osmo-sms.done
/usr/local/sbin/osmo-sms.sh
EOF
chmod +x "$ROOTFS/etc/profile.d/99-osmo-sms.sh"
echo -e "  ${GREEN}✓${NC} animation SMS a l'ouverture de session (99-osmo-sms.sh)"

# ── /usr/local/bin/osmo-update: in-place update via git ──────────────────
# The repositories ship in the image with their .git, so the update is a
# fetch plus fast-forward. Nothing is deleted or recloned, and an offline
# machine keeps exactly what it was built with.
cat > "$ROOTFS/usr/local/bin/osmo-update" <<'OSMOUPD'
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════════
# osmo-update - update the repositories embedded in the image, in place.
#
#   osmo-update              all repositories
#   osmo-update qosmo        a single one (osmo-operator | osmo-egprs-web | qosmo | c54x_exe | grgsm_exe)
#   osmo-update --check      report what is behind, write nothing
#   osmo-update --quiet      no colours or decoration (journal, cron)
#   osmo-update --boot       boot mode: --quiet, logged, always exits 0
#
# Never deletes a tree, reclones a repository or installs a package.
#
# ═══════════════════════════════════════════════════════════════════════════
set -u

G='\033[0;32m'; Y='\033[1;33m'; R='\033[0;31m'; C='\033[0;36m'; B='\033[1m'; N='\033[0m'
CHECK=0; QUIET=0; BOOT=0
while [ $# -gt 0 ]; do
    case "$1" in
        --check)   CHECK=1 ;;
        --quiet)   QUIET=1 ;;
        --boot)    BOOT=1; QUIET=1 ;;
        -h|--help) sed -n '3,12p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        -*)        echo "option inconnue : $1" >&2; exit 2 ;;
        *)         break ;;
    esac
    shift
done
[ "$QUIET" = "1" ] && { G=''; Y=''; R=''; C=''; B=''; N=''; }

# At boot nobody reads the screen: output goes to the log, and the exit code
# must never delay or block multi-user.target.
if [ "$BOOT" = "1" ]; then
    exec >>/var/log/osmo-update.log 2>&1
    echo "===== osmo-update (boot) $(date '+%F %T') ====="
fi

[ "$(id -u)" -eq 0 ] || { echo "Root requis." >&2; exit 1; }

# name|path - the paths start-native.sh, the dashboard and
# environnement/paths.env already use. Changing one here does not move theirs.
REPOS="osmo-operator|/opt/GSM/osmo-operator
osmo-egprs-web|/opt/GSM/osmo-egprs-web
qosmo|/opt/GSM/qosmo
c54x_exe|/opt/GSM/c54x_exe
grgsm_exe|/opt/GSM/grgsm_exe"

WANT="${1:-}"
rc=0; web_moved=0; forks_moved=""

while IFS='|' read -r name dir; do
    [ -n "$name" ] || continue
    [ -z "$WANT" ] || [ "$WANT" = "$name" ] || continue
```

```
    found=1
    printf "  ${B}%-16s${N} ${C}%s${N}\n" "$name" "$dir"

    if [ ! -d "$dir" ]; then
        printf "    ${R}✗${N} absent - l'image ne le portait pas\n"; rc=1; continue
    fi
    if [ ! -d "$dir/.git" ]; then
        # Never reclone over it: that would erase a tree of unknown content.
        # Report and skip.
        printf "    ${Y}⚠${N} pas de depot (.git absent) - laisse tel quel\n"; rc=1; continue
    fi

    br="$(git -C "$dir" symbolic-ref --quiet --short HEAD 2>/dev/null)" || br=""
    [ -n "$br" ] || br=main

    # A --depth 1 repository is grafted: its single commit has no parent, so
    # nothing fetched shares an ancestor with it. Refetch shallow repos with
    # --depth 1 (keeps them light) and full repos fully (keeps ancestry for
    # fast-forward).
    if git -C "$dir" rev-parse --is-shallow-repository 2>/dev/null | grep -q true; then
        fetch_ok=$(git -C "$dir" fetch --depth 1 --quiet origin "$br" 2>/dev/null && echo 1)
    else
        fetch_ok=$(git -C "$dir" fetch --quiet origin "$br" 2>/dev/null && echo 1)
    fi
    if [ -z "${fetch_ok:-}" ]; then
        printf "    ${Y}⚠${N} fetch impossible (reseau ?) - copie locale conservee\n"; rc=1; continue
    fi

    local_h="$(git -C "$dir" rev-parse HEAD 2>/dev/null)"
    remote_h="$(git -C "$dir" rev-parse FETCH_HEAD 2>/dev/null)"
    if [ "$local_h" = "$remote_h" ]; then
        printf "    ${G}✓${N} deja a jour - %s\n" "$(git -C "$dir" log -1 --format='%h %s')"
        continue
    fi
    if [ "$CHECK" = "1" ]; then
        printf "    ${Y}→${N} en retard : %s -> %s\n" "${local_h:0:7}" "${remote_h:0:7}"
        continue
    fi

    # Three cases; only local changes block the update, never history.
    if git -C "$dir" merge-base --is-ancestor HEAD FETCH_HEAD 2>/dev/null \
       && git -C "$dir" merge --ff-only FETCH_HEAD >/dev/null 2>&1; then
        # 1. Fast-forward: behind on the same branch.
        printf "    ${G}✓${N} %s\n" "$(git -C "$dir" log -1 --format='%h %s')"
        [ "$name" = "osmo-egprs-web" ] && web_moved=1
        case "$name" in qosmo) forks_moved="$forks_moved $name" ;; esac
    elif [ -z "$(git -C "$dir" status --porcelain 2>/dev/null)" ]; then
        # 2. No common ancestor (grafted --depth 1 repo) but clean tree:
        #    nothing to lose, align with the server.
        if git -C "$dir" reset --hard FETCH_HEAD >/dev/null 2>&1; then
            printf "    ${G}✓${N} aligne sur origin/%s - %s\n" "$br" "$(git -C "$dir" log -1 --format='%h %s')"
            [ "$name" = "osmo-egprs-web" ] && web_moved=1
            case "$name" in qosmo) forks_moved="$forks_moved $name" ;; esac
        else
            printf "    ${Y}⚠${N} alignement impossible - arbre inchange\n"; rc=1
        fi
    else
        # 3. Local modifications: touch nothing, report it.
        printf "    ${Y}⚠${N} modifications locales - rien n'a ete ecrase\n"
        printf "       a la main : ${C}git -C %s status${N}\n" "$dir"
        rc=1
    fi
done <<REPOEOF
$REPOS
REPOEOF
```

```
if [ -z "${found:-}" ]; then
    echo "depot inconnu : $WANT  (osmo-operator | osmo-egprs-web | qosmo | c54x_exe | grgsm_exe)" >&2
    exit 2
fi

# A fork update may change tools/qosmo-launch: the installed launcher
# (/usr/local/bin/<fork>, called by 40-qemu.sh) is rebuilt in place (plain C,
# libc only). Without gcc the old binary is kept (40-qemu.sh otherwise falls
# back to QEMU_BIN).
# scripts/voix-forks.sh reapplies the fork voice settings that
# "git reset --hard FETCH_HEAD" wipes ("io-tch-format rtp" in
# cfgs/mobile_group1.cfg and CALYPSO_PULSE_LATENCY_MSEC); see that script.
[ -x /opt/GSM/osmo-operator/scripts/voix-forks.sh ] \
    && /opt/GSM/osmo-operator/scripts/voix-forks.sh

for f in $forks_moved; do
    src="/opt/GSM/$f/tools/qosmo-launch"
    [ -f "$src/qosmo-launch.c" ] || continue
    if command -v gcc >/dev/null 2>&1 && make -s -C "$src" ALIAS="$f" install >/dev/null 2>&1; then
        printf "  ${G}✓${N} lanceur %s recompile (/usr/local/bin/%s)\n" "$f" "$f"
    else
        printf "  ${Y}⚠${N} lanceur %s non recompile (gcc/make ?) - l'ancien reste en place\n" "$f"
    fi
done

# The dashboard runs as a service: it must be restarted to pick up a new
# server.js.
if [ "$web_moved" = "1" ]; then
    [ -f /opt/GSM/osmo-egprs-web/package.json ] && \
        (cd /opt/GSM/osmo-egprs-web && npm install --production >/dev/null 2>&1 || true)
    systemctl try-restart osmo-egprs-web >/dev/null 2>&1 || true
    printf "  ${G}✓${N} dashboard relance\n"
fi

[ "$BOOT" = "1" ] && exit 0
exit $rc
OSMOUPD
chmod +x "$ROOTFS/usr/local/bin/osmo-update"

# At boot: after the network, without blocking it. Type=oneshot and an
# ExecStart that always exits 0 in --boot mode, so an offline machine boots
# the same.
cat > "$ROOTFS/etc/systemd/system/osmo-update.service" <<'EOF'
[Unit]
Description=osmo-operator - mise a jour des depots embarques (git, en place)
# network.target is enough: the default route may come later, which
# osmo-update --boot tolerates. No network-online.target (boot delay).
After=network.target
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/bin/osmo-update --boot
[Install]
WantedBy=multi-user.target
EOF
chroot "$ROOTFS" systemctl enable osmo-update 2>/dev/null || true
echo -e "  ${GREEN}✓${NC} osmo-update (/usr/local/bin, + service au demarrage : git fetch, jamais de reclone)"

# ── QEMU_BIN in the qosmo tree: build/qemu-system-arm ──────────────────
# The qosmo tree ships with .git and build/, so QEMU_BIN normally resolves
# already. This service covers a tree that lost its build/ (manual reclone or
# fresh checkout): environnement/paths.env would then resolve QEMU_BIN to a
# missing path and the stack would stop at the prerequisite check.
#
# Recreate the only path paths.env looks for, as a symlink to the installed
# binary (~30 MB, the ISO lives in RAM). build/ is in qemu's .gitignore, so
```

```
# "git clean -fd" does not remove it; the service runs at every boot.
cat > "$ROOTFS/usr/local/sbin/osmo-qemu-link.sh" <<'QLINK'
#!/bin/bash
# osmo-qemu-link.sh - make QEMU_BIN resolvable in the qosmo tree.
# Installed by build-iso.sh (iso_modules/81-cloture-systeme.sh).
set -u
SRC="${OSMO_QEMU_BIN:-/usr/local/bin/qemu-system-arm}"
TREE="${OSMO_QEMU_SRC:-/opt/GSM/qosmo}"
LNK="$TREE/build/qemu-system-arm"

# No binary (inter-STP image) or no tree: nothing to link.
[ -x "$SRC" ] || { echo "osmo-qemu-link: $SRC absent - rien a faire"; exit 0; }
[ -d "$TREE" ] || { echo "osmo-qemu-link: $TREE absent - rien a faire"; exit 0; }

# A real build compiled in place always wins: only replace our own
# symlink or an empty path.
if [ -e "$LNK" ] && [ ! -L "$LNK" ]; then
    echo "osmo-qemu-link: $LNK est un vrai fichier - laisse tel quel"
    exit 0
fi

mkdir -p "$TREE/build"
ln -sfn "$SRC" "$LNK"
echo "osmo-qemu-link: $LNK -> $SRC"

# ── C launchers (tools/qosmo-launch), called by 40-qemu.sh ──────────────
# /usr/local/bin/<fork> is rebuilt if missing or older than the tree source.
# Only one tree, qosmo; the DSP (--dsp) is c54x_exe, outside QEMU: report if
# it is missing.
for fork in qosmo; do
    src="/opt/GSM/$fork/tools/qosmo-launch"
    [ -f "$src/qosmo-launch.c" ] || continue
    if [ ! -x "/usr/local/bin/$fork" ] || [ "$src/qosmo-launch.c" -nt "/usr/local/bin/$fork" ]; then
        if command -v gcc >/dev/null 2>&1 && make -s -C "$src" ALIAS="$fork" install >/dev/null 2>&1; then
            echo "osmo-qemu-link: lanceur /usr/local/bin/$fork (re)compile"
        else
           echo "osmo-qemu-link: lanceur $fork non compile (gcc/make absents ?) - 40-qemu.sh retombe sur QEMU_BIN"
        fi
    fi
done
if [ -d /opt/GSM/c54x_exe ] && [ ! -x /opt/GSM/c54x_exe/c54x_exe ]; then
    echo "osmo-qemu-link: c54x_exe non compile - --dsp indisponible (make -C /opt/GSM/c54x_exe)"
fi
if [ ! -f /opt/GSM/calypso_dsp.PROM0.bin ]; then
    echo "osmo-qemu-link: ROM DSP absente de /opt/GSM - --dsp indisponible (copie dans /opt/GSM/c54x_exe/rom)"
fi
QLINK
chmod +x "$ROOTFS/usr/local/sbin/osmo-qemu-link.sh"

cat > "$ROOTFS/etc/systemd/system/osmo-qemu-link.service" <<'EOF'
[Unit]
Description=osmo-operator - QEMU_BIN dans l'arbre qosmo + lanceur qosmo
After=local-fs.target
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/sbin/osmo-qemu-link.sh
[Install]
WantedBy=multi-user.target
EOF
chroot "$ROOTFS" systemctl enable osmo-qemu-link 2>/dev/null || true
echo -e "  ${GREEN}✓${NC} osmo-qemu-link (QEMU_BIN relie apres le reclone de qosmo ; lanceur qosmo recompile si
↪  besoin)"
  ×3
# End of module: `. file` returns the status of its last command, and
# build-iso.sh runs under set -e. A module ending on a false test
```

```
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.75 /opt/GSM/osmo-operator/iso_modules/82-arm-natif.sh

12525 octets, 207 lignes → 207 lignes

```
#!/bin/bash
# iso_modules/82-arm-natif.sh - --arm: what the x86 build host cannot produce
# Sourced by build-iso.sh in numeric order (same shell, variables and functions);
# not meant to run on its own. A top-level `return` means "nothing to do here"
# (this is how --arm skips a step).

# On amd64 all of this is done elsewhere, natively (52-qemu, 71-debs-banc).
if [ "${ISO_ARCH:-amd64}" = "amd64" ]; then return 0; fi

# ═══════════════════════════════════════════════════════════════════════
#  Step 8e: native compilation inside the arm64 chroot
# ═══════════════════════════════════════════════════════════════════════
# The amd64 ISO builds these on the host with the host gcc: toast (GSM 06.10
# codec), the qosmo C launcher, c54x_exe/grgsm_exe, and the bench .deb packages
# (build-debs.sh compiles the launcher and inspects binaries with ldd). On an
# x86 host that would silently put x86 code into an arm64 rootfs, since none of
# these steps is fatal. Here everything runs inside the chroot under qemu-user,
# with the rootfs gcc (build-essential, step 8): the output is aarch64. Slower
# than native, but correct.
#
# Then the Pi-specific parts: the firmware partition (/boot/firmware: kernel,
# initrd, device trees, start4.elf, config.txt, cmdline.txt), kept up to date by
# hooks on each kernel update, and growing the root filesystem to the card size
# on first boot.
echo -e "${GREEN}[8e/9] Compilation native dans le chroot ${ISO_ARCH} (toast, lanceurs, .deb du banc)...${NC}"
_arm_chroot() {
    chroot "$ROOTFS" env PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin \
        LC_ALL=C.UTF-8 LANG=C.UTF-8 LANGUAGE= DEBIAN_FRONTEND=noninteractive "$@"
}

# ── toast ──────────────────────────────────────────────────────────────
if [ "$ISO_ROLE" != "interstp" ]; then
    if [ -x "$ROOTFS/usr/local/bin/toast" ]; then
        echo -e "  ${GREEN}✓${NC} toast deja present (image)"
    elif _arm_chroot bash -c '
        set -e; cd /opt/GSM
        wget -qO gsm-1.0.24.tar.gz https://www.quut.com/gsm/gsm-1.0.24.tar.gz
        tar xzf gsm-1.0.24.tar.gz && rm -f gsm-1.0.24.tar.gz && cd gsm-1.0-pl24
        make >/dev/null 2>&1 || true
        [ -x bin/toast ] || make toast >/dev/null 2>&1
        for b in toast untoast tcat; do [ -e "bin/$b" ] && install -m755 "bin/$b" "/usr/local/bin/$b"; done
        [ -x /usr/local/bin/toast ]' >"$WORK/toast-arm.log" 2>&1; then
        echo -e "  ${GREEN}✓${NC} toast compile dans le chroot (sources : /opt/GSM/gsm-1.0-pl24)"
    else
        echo -e "  ${YELLOW}!${NC} toast non compile dans le chroot (voir $WORK/toast-arm.log) - on continue" >&2
    fi
fi

# ── Lanceur C qosmo ────────────────────────────────────────────────────
if [ "$ISO_ROLE" != "interstp" ]; then
    for fork in qosmo; do
        if [ -x "$ROOTFS/usr/local/bin/$fork" ]; then
            echo -e "  ${GREEN}✓${NC} lanceur $fork deja present (image)"
        elif [ -f "$ROOTFS/opt/GSM/$fork/tools/qosmo-launch/qosmo-launch.c" ]; then
            if _arm_chroot make -s -C "/opt/GSM/$fork/tools/qosmo-launch" ALIAS="$fork" "$fork"
             ↪  >"$WORK/launch-$fork.log" 2>&1 \
              && _arm_chroot install -Dm755 "/opt/GSM/$fork/tools/qosmo-launch/$fork" "/usr/local/bin/$fork"; then
```

```
                echo -e "  ${GREEN}✓${NC} lanceur ${CYAN}/usr/local/bin/$fork${NC} compile dans le chroot"
            else
                echo -e "  ${YELLOW}!${NC} lanceur $fork : compilation echouee (voir $WORK/launch-$fork.log) -
                 ↪  run.sh appellera qemu-system-arm directement" >&2
            fi
        else
            echo -e "  ${YELLOW}!${NC} lanceur $fork : pas de source dans le rootfs" >&2
        fi
    done
    # c54x_exe and grgsm_exe binaries copied from the host are x86: rebuild them
    # here from the rootfs sources.
    for _e in c54x_exe grgsm_exe; do
        [ -f "$ROOTFS/opt/GSM/$_e/Makefile" ] || continue
        if _arm_chroot make -s -B -C "/opt/GSM/$_e" >"$WORK/make-$_e.log" 2>&1; then
            echo -e "  ${GREEN}✓${NC} ${CYAN}/opt/GSM/$_e/$_e${NC} compile dans le chroot"
        else
            echo -e "  ${YELLOW}!${NC} $_e : compilation echouee (voir $WORK/make-$_e.log)" >&2
        fi
    done
    # The arm64 binary must run here under qemu-user; this catches a binary or
    # libraries of the wrong architecture.
    if [ -x "$ROOTFS/usr/local/bin/qemu-system-arm" ]; then
        if _arm_chroot /usr/local/bin/qemu-system-arm --version >/dev/null 2>&1; then
            echo -e "  ${GREEN}✓${NC} qemu-system-arm (aarch64) s execute dans le chroot"
        else
            echo -e "${RED}qemu-system-arm du rootfs ne s execute pas sous qemu-user : binaire ou bibliotheques d
             ↪  une autre architecture ?${NC}" >&2
            file "$ROOTFS/usr/local/bin/qemu-system-arm" >&2 || true
            exit 1
        fi
    fi
fi

# ── Bench .deb packages (step 7c), built inside the chroot ──────────────
if [ -x "$ROOTFS/opt/GSM/osmo-operator/packaging/build-debs.sh" ]; then
    install -d "$ROOTFS/var/cache/osmo-debs"
    if _arm_chroot env OSMO_OPERATOR_SRC=/opt/GSM/osmo-operator QOSMO_SRC=/opt/GSM/qosmo \
         FIRMWARE_SRC=/opt/GSM/firmware TMPDIR=/var/tmp \
         bash /opt/GSM/osmo-operator/packaging/build-debs.sh --out /var/cache/osmo-debs \
         >"$WORK/build-debs.log" 2>&1; then
        echo -e "  ${GREEN}✓${NC} $(ls "$ROOTFS"/var/cache/osmo-debs/*.deb 2>/dev/null | wc -l) paquets dans
         ↪  ${CYAN}/var/cache/osmo-debs${NC}"
    else
        echo -e "  ${YELLOW}!${NC} paquets .deb du banc non construits (voir $WORK/build-debs.log) - l image sort
         ↪  sans eux"
    fi
fi

# ═══════════════════════════════════════════════════════════════════════
#  Pi firmware partition: /boot/firmware, populated the Armbian way
# ═══════════════════════════════════════════════════════════════════════
# The Pi 4 boots neither via GRUB nor u-boot: its GPU reads a FAT partition
# (the first one, label RPICFG), loads start4.elf, reads config.txt, then the
# kernel and initrd it names. Armbian mounts this partition on /boot/firmware
# and keeps it up to date with three hooks from its BSP package
# (armbian-bsp-cli-rpi4b):
#   /etc/kernel/postinst.d/z51-raspi-firmware   start4.elf, fixup4.dat... from
#                                               /usr/lib/linux-firmware-raspi
#   /etc/kernel/postinst.d/zzz-copy-new-files   vmlinuz, *.dtb, overlays/
#   /etc/initramfs/post-update.d/zzz-update-initramfs   initrd.img
# The kernel is installed before the BSP in the same apt run, so these hooks
# have not run yet. Replay them here, as the Armbian build does
# (post_family_tweaks__populate_boot_firmware_directory), then run
# update-initramfs so the third hook installs the initrd. Later kernel updates
# via apt on the machine do the same automatically.
_KVER="$(ls "$ROOTFS"/boot/vmlinuz-*-bcm2711 2>/dev/null | sort -V | tail -1 | sed 's|.*/vmlinuz-||' || true)"
```

```
[ -n "$_KVER" ] || { echo -e "${RED}aucun noyau Armbian bcm2711 dans /boot du rootfs (linux-image-current-bcm2711
 ↪  non installe ?)${NC}" >&2; exit 1; }
install -d "$ROOTFS/boot/firmware"
for _h in /etc/kernel/postinst.d/z51-raspi-firmware /etc/kernel/postinst.d/zzz-copy-new-files; do
    [ -x "$ROOTFS$_h" ] || { echo -e "${RED}hook Armbian absent : $_h (armbian-bsp-cli-rpi4b-current non installe
     ↪  ?)${NC}" >&2; exit 1; }
    _arm_chroot "$_h" "$_KVER" >>"$WORK/rpi-firmware.log" 2>&1 \
        || { echo -e "${RED}$_h a echoue (voir $WORK/rpi-firmware.log)${NC}" >&2; tail -5 "$WORK/rpi-firmware.log"
         ↪  >&2; exit 1; }
done
_arm_chroot update-initramfs -u -k "$_KVER" >>"$WORK/rpi-firmware.log" 2>&1 \
    || { echo -e "${RED}update-initramfs -k $_KVER a echoue (voir $WORK/rpi-firmware.log)${NC}" >&2; exit 1; }
[ -f "$ROOTFS/boot/firmware/initrd.img" ] || cp -f "$ROOTFS/boot/initrd.img-$_KVER"
 ↪  "$ROOTFS/boot/firmware/initrd.img"

# config.txt and cmdline.txt: as written by Armbian for rpi4b (bcm2711.conf,
# pre_umount_final_image__write_raspi_config/cmdline), except that the serial
# console is enabled (enable_uart=1; root autologin on it is set up by
# 84-comptes) and loglevel is not forced to 1. Written once, never
# overwritten afterwards: edit freely on the card.
cat > "$ROOTFS/boot/firmware/config.txt" <<'CFG'
# osmo-operator on Armbian - Raspberry Pi 4 (arm64)
# For more options and information see http://rptl.io/configtxt

# Enable audio (loads snd_bcm2835)
dtparam=audio=on

# Additional overlays and parameters are documented /boot/firmware/overlays/README

# Automatically load overlays for detected cameras / DSI displays
camera_auto_detect=1
display_auto_detect=1

# Automatically load initramfs files, if found
auto_initramfs=1

# Enable DRM VC4 V3D driver
dtoverlay=vc4-kms-v3d
max_framebuffers=2

# Don't have the firmware create an initial video= setting in cmdline.txt.
disable_fw_kms_setup=1

# Disable compensation for displays with overscan
disable_overscan=1

# Run as fast as firmware / board allows
arm_boost=1

# Serial console (serial0 -> ttyS0), root autologin (osmo-operator)
enable_uart=1

[cm4]
otg_mode=1

[all]
kernel=vmlinuz
initramfs initrd.img followkernel
arm_64bit=1
CFG
printf '%s\n' "console=serial0,115200 console=tty1 root=LABEL=armbi_root rootfstype=ext4 fsck.repair=yes rootwait
 ↪  logo.nologo cgroup_enable=cpuset cgroup_memory=1 cgroup_enable=memory" \
    > "$ROOTFS/boot/firmware/cmdline.txt"

for _f in vmlinuz initrd.img start4.elf fixup4.dat bcm2711-rpi-4-b.dtb config.txt cmdline.txt
 ↪  overlays/vc4-kms-v3d.dtbo; do
```

```
    [ -e "$ROOTFS/boot/firmware/$_f" ] || { echo -e "${RED}/boot/firmware/$_f absent apres les hooks Armbian (voir
     ↪  $WORK/rpi-firmware.log)${NC}" >&2; ls -la "$ROOTFS/boot/firmware" >&2; exit 1; }
done
echo -e "  ${GREEN}✓${NC} /boot/firmware : noyau ${CYAN}${_KVER}${NC}, $(ls "$ROOTFS/boot/firmware" | wc -l)
 ↪  entrees ($(du -sh "$ROOTFS/boot/firmware" | cut -f1))"

# ── Armbian first-boot services ──────────────────────────────────────
# armbian-resize-filesystem: grows the root partition to the card and runs
# resize2fs, once (handles both MBR and GPT).
# armbian-firstrun: regenerates ssh keys, hardware tweaks. The BSP postinst
# enables zram, hardware-optimize, led-state and monitor, but not these two.
# armbian-ramlog (log2ram) is disabled: it keeps /var/log in 50 MB of RAM;
# osmocom logs (logrotate 32 MB x 3) and the tcpdump pcap ring (5 x 32 MB)
# would fill it and writes would fail silently. /var/log stays on the card.
chroot "$ROOTFS" systemctl enable armbian-resize-filesystem armbian-firstrun >/dev/null 2>&1 \
    || echo -e "  ${YELLOW}!${NC} armbian-resize-filesystem / armbian-firstrun : enable a echoue" >&2
install -d "$ROOTFS/etc/default"
if [ -f "$ROOTFS/etc/default/armbian-ramlog" ] || [ -f "$ROOTFS/etc/default/armbian-ramlog.dpkg-dist" ]; then
    cp -f "$ROOTFS/etc/default/armbian-ramlog.dpkg-dist" "$ROOTFS/etc/default/armbian-ramlog" 2>/dev/null || true
    sed -i 's/^ENABLED=.*/ENABLED=false/' "$ROOTFS/etc/default/armbian-ramlog"
    chroot "$ROOTFS" systemctl disable armbian-ramlog >/dev/null 2>&1 || true
fi
# No Armbian first-login wizard: root:osmo is already set; the marker
# /root/.not_logged_in_yet only exists in Armbian images.
rm -f "$ROOTFS/root/.not_logged_in_yet"
echo -e "  ${GREEN}✓${NC} armbian-resize-filesystem et armbian-firstrun actives, armbian-ramlog coupe"

# End of module: `. file` returns the status of its last command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.76 /opt/GSM/osmo-operator/iso_modules/82-services.sh

23691 octets, 438 lignes → 438 lignes

```
#!/bin/bash
# iso_modules/82-services.sh - role marker, os-release, asterisk, dashboard, audio, PulseAudio
# Sourced by build-iso.sh in numeric order (same shell, variables and functions);
# not meant to run on its own. A top-level `return` means "nothing to do here"
# (this is how --arm skips a step).

# ── Role marker: what this image is ──────────────────────────────────
# Read by start-native.sh and the login banner, so images built from the same
# pipeline can be told apart once booted.
{
    printf '# /etc/osmo-role - genere par build-iso.sh\n'
    printf 'OSMO_ROLE=%s\n' "$ISO_ROLE"
    [ -n "$ISO_NODE" ] && printf 'OSMO_WAN_NODE=%s\n' "$ISO_NODE"
    # Lite images lack the /opt/GSM build trees; record it so the machine can
    # report it itself.
    printf 'OSMO_LITE=%s\n' "$ISO_LITE"
    printf 'OSMO_HUB_IP=%s\n' "$ISO_HUB_IP"
} > "$ROOTFS/etc/osmo-role"

# ── /etc/os-release: the image names itself ──────────────────────────
# The login banner, `hostnamectl`, bug reports and the dashboard read
# os-release, so it must distinguish hub, node and lite variant.
#
# Only display fields are changed. ID, VERSION_ID, VERSION_CODENAME and
# UBUNTU_CODENAME stay Ubuntu's: apt, add-apt-repository and many package
# scripts use them to pick repositories.
# VARIANT / VARIANT_ID are the os-release(5) fields meant for this distinction;
# IMAGE_ID / IMAGE_VERSION are meant for built images.
```

```
case "$ISO_ROLE" in
    interstp) OS_NAME="osmo-operator interstp"; OS_VARIANT_ID="interstp" ;;
    *)        if [ "$ISO_LITE" = "1" ]; then
                  OS_NAME="osmo-operator-lite"; OS_VARIANT_ID="operator-lite"
              else
                  OS_NAME="osmo-operator";      OS_VARIANT_ID="operator"
              fi ;;
esac
# When the node number is baked into the image it is part of the identity:
# it is the only difference between osmo-operator-1.iso and osmo-operator-2.iso.
OS_PRETTY="$OS_NAME"
[ -n "$ISO_NODE" ] && OS_PRETTY="$OS_NAME (noeud $ISO_NODE)"

# /etc/os-release is a symlink to ../usr/lib/os-release: write the target and
# keep the link, otherwise the two paths (both read by various tools) diverge.
_osrel="$ROOTFS/usr/lib/os-release"
sed -i -e '/^NAME=/d' -e '/^PRETTY_NAME=/d' \
       -e '/^VARIANT=/d' -e '/^VARIANT_ID=/d' \
       -e '/^IMAGE_ID=/d' -e '/^IMAGE_VERSION=/d' \
       -e '/^HOME_URL=/d' -e '/^SUPPORT_URL=/d' -e '/^BUG_REPORT_URL=/d' \
       "$_osrel"
{
    printf 'NAME="%s"\n'          "$OS_NAME"
    printf 'PRETTY_NAME="%s"\n'   "$OS_PRETTY"
    printf 'VARIANT="%s"\n'       "$OS_PRETTY"
    printf 'VARIANT_ID="%s"\n'    "$OS_VARIANT_ID"
    printf 'IMAGE_ID="%s"\n'      "$OS_NAME"
    printf 'IMAGE_VERSION="%s"\n' "$LABEL"
    printf 'HOME_URL="https://github.com/bbaranoff/osmo-operator"\n'
    printf 'SUPPORT_URL="https://github.com/bbaranoff/osmo-operator"\n'
    printf 'BUG_REPORT_URL="https://github.com/bbaranoff/osmo-operator/issues"\n'
} >> "$_osrel"
echo -e "  ${GREEN}✓${NC} os-release : ${CYAN}${OS_PRETTY}${NC}"

# ── Asterisk: single owner, run_modules/19-asterisk.sh ──────────────────
# The asterisk package ships its unit enabled; a systemd-started Asterisk
# would compete with the one started by 19-asterisk.sh for /etc/asterisk and
# the asterisk.ctl socket. The module also disables it at each start; doing
# it here keeps the first boot of a fresh ISO clean.
chroot "$ROOTFS" systemctl disable asterisk >/dev/null 2>&1 || true
echo -e "  ${GREEN}✓${NC} asterisk.service desactive - le PBX est lance par ${CYAN}19-asterisk.sh${NC}"

if [ "$ISO_ROLE" = "interstp" ]; then
    # The hub must start on its own: nodes attach to it at boot, so a hub that
    # has to be started by hand breaks simultaneous startup.
    cat > "$ROOTFS/etc/systemd/system/osmo-interstp.service" <<EOF
[Unit]
Description=osmo-operator inter-STP - hub SS7 du WAN (PC 0.0.0)
# network.target is enough: the default route may come later, which the
# service tolerates. No network-online.target (boot delay).
After=network.target
[Service]
Type=forking
PIDFile=/run/osmo-interstp.pid
ExecStart=/opt/GSM/osmo-operator/launch/start-interstp.sh --ip ${ISO_HUB_IP}
ExecStop=/opt/GSM/osmo-operator/launch/start-interstp.sh --stop
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
EOF
    chroot "$ROOTFS" systemctl enable osmo-interstp 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} osmo-interstp.service - hub lance au demarrage sur ${CYAN}${ISO_HUB_IP}${NC}"
fi
```

```
# Root autologin is configured later (8d, getty@tty1.service.d/10-autologin-root.conf).

# ── Web dashboard service ──────────────────────────────────────────
# The unit file is services/osmo-egprs-web.service, copied at step 6 (single
# source of truth, with TLS_CERT/TLS_KEY, PULSE_SERVER and CAP_IFACE). Only
# enable it here; never rewrite it.
# The hub has no operator VTY or radio to show: the dashboard is not enabled there.
[ "$ISO_ROLE" = "interstp" ] || chroot "$ROOTFS" systemctl enable osmo-egprs-web 2>/dev/null||true

# ── 4G subscriber console (Open5GS WebUI, :9999) ──────────────────
# Installed and enabled by default, like the 2G dashboard. On images without a
# built WebUI its ConditionPathExists (services/open5gs-webui.service) skips
# it cleanly. The hub has no 4G core: not enabled there.
if [ "$ISO_ROLE" != "interstp" ]; then
    if [ -f "$DIR/services/open5gs-webui.service" ]; then
        install -m644 "$DIR/services/open5gs-webui.service" \
                "$ROOTFS/etc/systemd/system/open5gs-webui.service"
        mkdir -p "$ROOTFS/etc/systemd/system/multi-user.target.wants"
        chroot "$ROOTFS" systemctl enable open5gs-webui 2>/dev/null || \
            ln -sf /etc/systemd/system/open5gs-webui.service \
                "$ROOTFS/etc/systemd/system/multi-user.target.wants/open5gs-webui.service"
        echo -e "  ${GREEN}✓${NC} ${CYAN}open5gs-webui.service${NC} pose et active au boot (console abonnes 4G,
        ↪  :9999)"
    else
        echo -e "  ${RED}✗ services/open5gs-webui.service introuvable${NC}" >&2; exit 1
    fi
fi

# ── Bench services are installed but not started at boot ──────────
# services/osmo-banc.service (standalone: start-native.sh) and
# services/osmo-multi.service (multi-operator: start-multi.sh) keep their
# stack in tmux: `tmux attach -t calypso` from any root terminal takes over.
# launch.sh and start-multi.sh detect the unit and go through it.
#
# Starting a bench is an operator action (node, --dsp and topology are chosen
# first), not a boot side effect:
#   systemctl start osmo-banc          one session (or the desktop icon)
#   systemctl enable --now osmo-banc   restart it at every boot
# At build time OSMO_ISO_BANC=1 (or --banc) and OSMO_ISO_MULTI=1 (or --multi)
# enable them at boot, per image.
# osmo-lte (services/osmo-lte.service: Open5GS + srsENB + srsUE, the "4G du
# banc" icon) follows the same rule: OSMO_ISO_LTE=1 enables it at boot.
# Without srsenb (interstp, arm64) its Condition skips it.
if [ "$ISO_ROLE" != "interstp" ]; then
    # osmo-supplements: installed, not enabled; the Calamares osmosupplements job
    # enables it in the target when a box on the "Supplements" page is ticked.
    for _u in osmo-banc osmo-multi osmo-lte osmo-supplements; do
        if [ -f "$DIR/services/$_u.service" ]; then
            install -m644 "$DIR/services/$_u.service" "$ROOTFS/etc/systemd/system/$_u.service"
        else
            echo -e "  ${RED}✗ services/$_u.service introuvable${NC}" >&2; exit 1
        fi
    done
    # `systemctl disable` alone is not enough: the base image may already carry
    # the multi-user.target.wants/ link, and `disable` in the chroot fails
    # silently when systemd is not running. Remove the link directly (what
    # `disable` does anyway, the mirror of the `ln -sf` fallback for `enable`).
    mkdir -p "$ROOTFS/etc/systemd/system/multi-user.target.wants"
    for _u in osmo-banc osmo-multi osmo-lte; do
        # osmo-banc -> OSMO_ISO_BANC, osmo-multi -> OSMO_ISO_MULTI; ${!_var} is
        # bash indirect expansion (value of the variable named by $_var), not eval.
        _var="OSMO_ISO_${_u#osmo-}"; _var="${_var^^}"
        if [ "${!_var:-0}" = "1" ]; then
            chroot "$ROOTFS" systemctl enable "$_u" 2>/dev/null || \
                ln -sf "/etc/systemd/system/$_u.service" \
                       "$ROOTFS/etc/systemd/system/multi-user.target.wants/$_u.service"
```

```
            echo -e "  ${GREEN}✓${NC} ${CYAN}$_u.service${NC} pose ET active au boot (${_var}=1)"
        else
            chroot "$ROOTFS" systemctl disable "$_u" 2>/dev/null || true
            rm -f "$ROOTFS/etc/systemd/system/multi-user.target.wants/$_u.service"
            echo -e "  ${GREEN}✓${NC} ${CYAN}$_u.service${NC} pose, ${BOLD}non active${NC} au boot ·
             ↪  ${CYAN}systemctl start $_u${NC} pour le lancer, ${CYAN}${_var}=1${NC} au build pour l'activer"
        fi
    done
fi

# ── Audio: system PulseAudio (gsm_audio sink) at boot ──────────────────
# Chain: osmo-gapk → ALSA gsm_out → null sink gsm_audio → monitor → loopback
# → sound card. system.pa allows anonymous access and declares the sinks; the
# service starts the daemon at boot (ensure_pulse in start-native.sh is then a no-op).
if [ -f "$ROOTFS/etc/pulse/system.pa" ]; then
    sed -i 's|^load-module module-native-protocol-unix.*|load-module module-native-protocol-unix auth-anonymous=1
     ↪  socket=/var/run/pulse/native|' \
        "$ROOTFS/etc/pulse/system.pa"
    # Both gsm_audio and gsm_mic must be declared: lib/audio.sh treats them as a
    # pair (GSM_SINKS) and configs/asound.conf maps pcm.gsm_in to gsm_mic.monitor.
    # Without gsm_mic, gapk_io fails to open 'gsm_in' and drops both directions
    # (call established but silent).
    for _s in "gsm_audio:GSM_Audio" "gsm_mic:GSM_Mic"; do
        _n="${_s%%:*}"; _d="${_s##*:}"
        grep -q "sink_name=${_n}\b" "$ROOTFS/etc/pulse/system.pa" || \
            echo "load-module module-null-sink sink_name=${_n} format=s16le rate=8000 channels=1
             ↪  sink_properties=device.description=${_d}" \
            >> "$ROOTFS/etc/pulse/system.pa"
    done
fi
# /etc/asound.conf defines the gsm_out/gsm_in PCMs opened by `mobile`;
# without it TCH voice never reaches gsm_audio.
if [ -f "$DIR/configs/asound.conf" ]; then
    cp -f "$DIR/configs/asound.conf" "$ROOTFS/etc/asound.conf"
    echo -e "  ${GREEN}✓${NC} /etc/asound.conf (PCM gsm_out/gsm_in → sinks PulseAudio)"
fi
cat > "$ROOTFS/usr/local/sbin/osmo-audio-chain.sh" <<'ACHAIN'
#!/bin/bash
# osmo-audio-chain.sh - complete the local audio chain once PulseAudio is up.
# Called as ExecStartPost by osmo-pulse.service. Ensures:
#   1. /etc/asound.conf is present (PCM gsm_out/gsm_in)
#   2. both null sinks gsm_audio + gsm_mic are loaded
#   3. module-loopback gsm_audio.monitor → sound card
# Without (2) gapk_io drops both directions; without (3) downlink voice is
# discarded by the null sink. Always exits 0: audio must never block the
# stack. AUDIO=0 or AUDIO_LOCAL_LOOPBACK=0 disable it.
set -u
for r in /opt/GSM/osmo-operator /etc/osmocom/osmo-operator; do
    [ -x "$r/scripts/audio-chain.sh" ] && exec "$r/scripts/audio-chain.sh" "${1:-30}"
done
exit 0
ACHAIN
chmod +x "$ROOTFS/usr/local/sbin/osmo-audio-chain.sh"

cat > "$ROOTFS/usr/local/sbin/osmo-pulse-link.sh" <<'PLINK'
#!/bin/sh
# osmo-pulse-link.sh [uid] - make the system PulseAudio visible to clients
# that look for a per-user socket.
#
# In system mode PulseAudio only listens on /run/pulse/native, while clients
# look at $XDG_RUNTIME_DIR/pulse/native (/run/user/<uid>/pulse). Without that
# path a browser sees no device ("NotFoundError" on the microphone).
#
# Always exits 0: audio must never block the stack.
#
# ── Called twice, both matter ──────────────────────────────────────────
```

```
#   1. without argument, by osmo-pulse.service (ExecStartPost): sessions
#      already open when the daemon (re)starts.
#   2. with a uid, by the user@.service drop-in (ExecStartPre=+, i.e. as
#      root): every session opened afterwards. At boot osmo-pulse.service
#      starts before the first login, so /run/user/<uid> of an installed
#      system's user (uid >= 1000) does not exist yet.
#
# ── Socket owner, not only the path ──────────────────────────────────
# An AppArmor-confined client may be allowed the path only with the `owner`
# qualifier (file owner == process fsuid). The socket belongs to `pulse`, so
# the connection would be refused. Such rules apply to the real path: the
# symlink below does not hide /run/pulse/native.
#
# The socket is therefore given to the graphical session's uid. A socket has
# a single owner: the last human session opened gets it. Service accounts
# (gdm, uid < 1000) are excluded, otherwise gdm, whose user@.service starts
# first, would keep it. OSMO_PULSE_UID forces the value.
# The daemon still runs as `pulse`: accept() needs no ownership, and mode
# srwxrwxrwx lets everyone connect.
set -u

# The symlink, for the given uid or for all open sessions.
if [ "$#" -ge 1 ] && [ -n "${1:-}" ]; then
    _dirs="/run/user/$1"
else
    _dirs=$(echo /run/user/*)
fi
for d in $_dirs; do
    [ -d "$d" ] || continue
    mkdir -p "$d/pulse" 2>/dev/null || continue
    # -f: a leftover pipewire-pulse socket must be replaced; without -f ln
    # fails silently and the session stays mute.
    ln -sfn /run/pulse/native "$d/pulse/native" 2>/dev/null || true
done

# The owner. With an argument: that session, if it is a human one.
# Otherwise the active graphical session, and root as a last resort
# (the live USB account).
PUID="${OSMO_PULSE_UID:-}"
if [ -z "$PUID" ] && [ "$#" -ge 1 ] && [ -n "${1:-}" ]; then
    [ "$1" -ge 1000 ] 2>/dev/null && PUID="$1"
    [ "$1" = "0" ] && PUID=0
fi
if [ -z "$PUID" ]; then
    PUID=$(loginctl list-sessions --no-legend 2>/dev/null \
           | awk '$2 == 0 || $2 >= 1000 {print $2; exit}')
fi
[ -n "$PUID" ] || PUID=0
[ -S /run/pulse/native ] && chown "$PUID" /run/pulse/native 2>/dev/null || true
exit 0
PLINK
chmod +x "$ROOTFS/usr/local/sbin/osmo-pulse-link.sh"

# ── The link is created at every session start ──────────────────────
# See the osmo-pulse-link.sh header: at boot osmo-pulse.service cannot link a
# session that does not exist yet. systemd-logind creates /run/user/<uid>
# then starts user@<uid>.service; this drop-in hooks the script there, once
# per session.
#
# `ExecStartPre=+`: `+` runs the command as root despite user@.service's
# User=; otherwise chown of the socket (owned by `pulse`) would fail.
install -d "$ROOTFS/etc/systemd/system/user@.service.d"
cat > "$ROOTFS/etc/systemd/system/user@.service.d/10-osmo-pulse-link.conf" <<'UAT'
[Service]
ExecStartPre=+/usr/local/sbin/osmo-pulse-link.sh %i
UAT
```

```
# ── Default server, for clients that ignore the link ──────────────────────
# libpulse reads /etc/pulse/client.conf and its .d directory before deriving
# a path from XDG_RUNTIME_DIR. `default-server` therefore works regardless of
# races, uid or start order: GNOME, Firefox, VLC and gapk reach the system
# daemon even if the link is missing.
#
# `autospawn = no` too: otherwise a client that finds no server spawns its own
# pulseaudio, which cannot grab the cards (held by the system daemon) and
# leaves the session with an empty server.
install -d "$ROOTFS/etc/pulse/client.conf.d"
cat > "$ROOTFS/etc/pulse/client.conf.d/50-osmo-system-server.conf" <<'PCLIENT'
# Installed by build-iso.sh (iso_modules/82-services.sh).
# The bench runs a system PulseAudio (osmo-pulse.service): all clients,
# whatever their uid, connect to it. Without this line each session looks
# for a server in its own /run/user/<uid> and finds none.
default-server = unix:/run/pulse/native
autospawn = no
PCLIENT
echo -e "  ${GREEN}✓${NC} audio session : ${CYAN}client.conf.d/50-osmo-system-server.conf${NC} + lien pose a
 ↪  chaque ouverture (user@.service)"

cat > "$ROOTFS/etc/systemd/system/osmo-pulse.service" <<'EOF'
[Unit]
Description=osmo-operator PulseAudio system daemon (GSM audio)
# sound.target is passive: it is reached once systemd has processed the known
# udev rules, not when the cards are present. module-udev-detect picks up
# late cards, but the loopback to the card is set up only once; the wait
# lives in scripts/audio-chain.sh (delay argument) instead of pulling the
# obsolete systemd-udev-settle, which would delay the whole boot.
After=sound.target
Wants=sound.target
[Service]
# ── Type=notify and --daemonize=no go together ────────────────────────────
# With Type=forking + --daemonize=yes, systemd guesses the main PID among the
# cgroup survivors and may pick a dead worker, then kill the real daemon
# (KillMode=control-group): /run/pulse/native never appears. With
# --daemonize=no the started process is the daemon, and pulseaudio (linked
# to libsystemd) sends sd_notify(READY=1) once its modules are loaded, as the
# packaged unit does ("notify will only work if --daemonize=no").
# Type=notify also delays ExecStartPost until READY=1, so the scripts below
# never race the socket creation.
Type=notify
ExecStart=/usr/bin/pulseaudio --system --daemonize=no --disallow-exit --exit-idle-time=-1 --log-target=journal
ExecStartPre=/bin/mkdir -p /var/log/osmocom /var/run/pulse
# ── The socket where applications look for it ─────────────────────────────
# In system mode PulseAudio only listens on /run/pulse/native, while
# applications look for /run/user/<uid>/pulse/native; without it
# getUserMedia returns "NotFoundError". Set up by the daemon unit itself, so
# it survives a service restart.
ExecStartPost=/usr/local/sbin/osmo-pulse-link.sh
# gsm_audio (module-null-sink) needs a consumer, otherwise downlink voice is
# discarded and the call is silent: the loopback to the card is set up here,
# by the daemon unit, so it survives a restart. Non-fatal (always exits 0).
# AUDIO_LOCAL_LOOPBACK=0 disables it.
# Goes through a /usr/local/sbin wrapper: an ExecStartPost=/bin/sh -c with
# nested quotes is rejected by the unit parser and silently ignored.
ExecStartPost=/usr/local/sbin/osmo-audio-chain.sh 30
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
EOF
# Same for audio: the hub carries no calls, it only routes signalling.
[ "$ISO_ROLE" = "interstp" ] || chroot "$ROOTFS" systemctl enable osmo-pulse 2>/dev/null||true
```

```
# Kernel modules
mkdir -p "$ROOTFS/etc/modules-load.d"
printf 'sctp\ntun\n' > "$ROOTFS/etc/modules-load.d/osmocom.conf"

# Environment variables
cat > "$ROOTFS/etc/environment" <<'EOF'
PATH="/usr/local/sbin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin"
PKG_CONFIG_PATH="/usr/local/lib/pkgconfig"
LD_LIBRARY_PATH="/usr/local/lib"
EOF

# root bashrc
cat >> "$ROOTFS/root/.bashrc" <<'BASH'
# Activate by default the Python environment (gr-gsm + bridges) used by
# /opt/GSM/osmo-operator/start-native.sh. VIRTUAL_ENV_DISABLE_PROMPT keeps PS1.
export VIRTUAL_ENV_DISABLE_PROMPT=1
[ -f /root/.env/bin/activate ] && source /root/.env/bin/activate
# /etc/osmocom/coeur.env is loaded here because environment/load.env only
# looks in its own directory. set -a exports the values to start-native.sh;
# the file's ":=" idiom lets e.g. N_MS=3 ./start-native.sh override them.
if [ -f /etc/osmocom/coeur.env ]; then set -a; . /etc/osmocom/coeur.env; set +a; fi
# Shortcuts for the bench.
alias osmo-lab='cd /opt/GSM/osmo-operator; ./start-native.sh'
alias osmo-web='systemctl status osmo-egprs-web'
alias osmo-status='/etc/osmocom/status.sh status'
export PATH="$HOME/.local/bin:$PATH"

### calypso-prompt ###
export PS1='\[\033[1;31m\]\u\[\033[0m\]@\[\033[1;34m\]\h\[\033[0m\]:\[\033[1;32m\]\w\[\033[0m\]☎ # '
### end calypso-prompt ###
BASH

# Message of the day: colour banner and aligned box. Box content is ASCII
# (no multi-byte characters) with printf padding so the borders align.
# Generated at build time to embed ANSI colours in /etc/motd.
{
  B=$'\033[1;36m'; T=$'\033[1;37m'; G=$'\033[1;32m'; Y=$'\033[0;33m'; N=$'\033[0m'
  W=58
  printf '\n%b' "$B"
  cat <<'LOGO'
    ___  ___ _ __ ___   ___    ___  __ _ _ __  _ __ ___
   / _ \/ __| '_ ` _ \ / _ \  / _ \/ _` | '_ \| '__/ __|
  | (_) \__ \ | | | | | (_) ||  __/ (_| | |_) | |  \__ \
   \___/|___/_| |_| |_|\___/  \___|\__, | .__/|_|  |___/
                                   |___/|_|
LOGO
  printf '%b' "$N"
  printf "${B}  ╔"; printf '═%.0s' $(seq 1 $W); printf "╗${N}\n"
  printf "${B}  ║${N} ${T}%-*s${N} ${B}║${N}\n" $((W-2)) "GSM / EGPRS  Multi-PLMN  Live System"
  printf "${B}  ║${N} %-*s ${B}║${N}\n"         $((W-2)) "SS7/SIGTRAN  -  Osmocom  -  Calypso/QEMU"
  printf "${B}  ╠"; printf '═%.0s' $(seq 1 $W); printf "╣${N}\n"
  # Path of the bench launcher (same as the login message and the
  # osmo-start-direct link).
  printf "${B}  ║${N} ${G}%-*s${N} ${B}║${N}\n" $((W-2)) "/opt/GSM/osmo-operator/start-native.sh"
  printf "${B}  ║${N} %-*s ${B}║${N}\n"         $((W-2)) "    -> lance le lab Calypso/QEMU (A5/1)"
  printf "${B}  ║${N} ${G}%-*s${N} ${B}║${N}\n" $((W-2)) "Dashboard web  ->  http://<vm-ip>:8080"
  printf "${B}  ║${N} ${G}%-*s${N} ${B}║${N}\n" $((W-2)) "FFT spectres   ->  http://<vm-ip>:8081"
  printf "${B}  ║${N} ${Y}%-*s${N} ${B}║${N}\n" $((W-2)) "Wiki / docs        ->  pl4y.store"
  printf "${B}  ║${N} ${G}%-*s${N} ${B}║${N}\n" $((W-2)) "ssh root@<vm-ip>   -> mot de passe : osmo"
  printf "${B}  ║${N} ${Y}%-*s${N} ${B}║${N}\n" $((W-2)) "loadkeys fr   -> changer le clavier (apres boot)"
  printf "${B}  ║${N} ${G}%-*s${N} ${B}║${N}\n" $((W-2)) "osmo-update   -> met a jour les depots (git en place)"
  printf "${B}  ╚"; printf '═%.0s' $(seq 1 $W); printf "╝${N}\n\n"
} > "$ROOTFS/etc/motd"

# Root password = "osmo" (console autologin + SSH login). The password is
# not emptied (sshd would refuse root login).
```

```
echo 'root:osmo' | chroot "$ROOTFS" chpasswd 2>/dev/null || true

# ── Default policy: no telecom service at boot ──────────────────────
# The services/80-osmo-operator.preset preset is installed in the rootfs and
# any telecom unit already enabled (asterisk/ofono packages enabling
# themselves, upstream osmo-* units, base image) is disabled. Only explicit
# build requests stay enabled: OSMO_ISO_BANC / _MULTI / _LTE=1, and the
# osmo-interstp hub for the interstp role.
_garder=()
[ "$ISO_ROLE" = "interstp" ] && _garder+=(osmo-interstp)
[ "${OSMO_ISO_BANC:-0}" = 1 ]  && _garder+=(osmo-banc)
[ "${OSMO_ISO_MULTI:-0}" = 1 ] && _garder+=(osmo-multi)
[ "${OSMO_ISO_LTE:-0}" = 1 ]   && _garder+=(osmo-lte)
bash "$DIR/tools/osmo-telecom-boot.sh" --root "$ROOTFS" ${_garder[@]+--garder "${_garder[@]}"} || true

# End of module: `. file` returns the status of its last command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.77 /opt/GSM/osmo-operator/iso_modules/83-espace-ram.sh

15919 octets, 370 lignes → 370 lignes

```
#!/bin/bash
# iso_modules/83-espace-ram.sh - fstab, /tmp, journal, logrotate, tcpdump wrapper, purge, ssh, keyboard
# Sourced by build-iso.sh in numeric order (same shell, variables and functions);
# not meant to run on its own. A top-level `return` means "nothing to do here"
# (this is how --arm skips a step).

# ── Live writable space: /dev/shm and /tmp, as a percentage of RAM ──────────
# The live system boots 'toram': the root is a tmpfs overlay (RAM). The big
# writers are the I/Q cfiles in /dev/shm (FFT/record, hundreds of MB) and /tmp.
#
# Percentages, not fixed sizes: the squashfs already takes ~2.5 GB of RAM and
# the root is itself a tmpfs; 20 % + 15 % always leave two thirds of RAM to
# the squashfs, root and processes, whatever the machine size.
#
# Idempotent: first remove any existing tmpfs /tmp or /dev/shm entry
# (including the 'nosuid,nodev' variant without size=) and the old block
# comment, then write the canonical block, so the squashfs fstab has exactly
# one /dev/shm entry.
touch "$ROOTFS/etc/fstab"
sed -i -E \
    -e '/^[[:space:]]*tmpfs[[:space:]]+\/tmp[[:space:]]/d' \
    -e '/^[[:space:]]*tmpfs[[:space:]]+\/dev\/shm[[:space:]]/d' \
    -e '/^# osmo-operator live - espace writable/d' \
    "$ROOTFS/etc/fstab"
# /dev/shm: sized via fstab (no generator duplicate risk).
cat >> "$ROOTFS/etc/fstab" <<'FSTAB'
# osmo-operator live - espace writable (cfiles I/Q FFT)
tmpfs   /dev/shm   tmpfs   defaults,nosuid,nodev,size=20%   0 0
FSTAB
# --arm: root and firmware are real SD card partitions (named by
# 92-rpi-image). The live image has none: live-boot provides its root.
# Labels, not UUIDs: the image is built without mounting.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    # Armbian labels (armbi_root, RPICFG): what armbian-resize-filesystem and its
    # docs expect.
    sed -i -E -e '/^LABEL=armbi_root[[:space:]]/d' -e '/^LABEL=RPICFG[[:space:]]/d' "$ROOTFS/etc/fstab"
    cat >> "$ROOTFS/etc/fstab" <<'FSTAB'
LABEL=armbi_root    /              ext4    defaults,noatime,commit=120,errors=remount-ro   0 1
LABEL=RPICFG        /boot/firmware  vfat    defaults                                        0 2
FSTAB
```

```
    install -d "$ROOTFS/boot/firmware"
fi
# /tmp: not in fstab. An fstab /tmp entry collides with systemd's tmp.mount
# (systemd-fstab-generator: "Duplicate entry in /etc/fstab"). /tmp is managed
# natively via a tmp.mount drop-in with size=15%.
mkdir -p "$ROOTFS/etc/systemd/system/tmp.mount.d"
cat > "$ROOTFS/etc/systemd/system/tmp.mount.d/size.conf" <<'EOF'
[Mount]
Options=mode=1777,strictatime,nosuid,nodev,size=15%
EOF
chroot "$ROOTFS" systemctl enable tmp.mount 2>/dev/null || true

# ── Bounding the stack's writes ──────────────────────────────────────
# With 'toram' the root is a tmpfs overlay: everything written at runtime stays
# in RAM. Three unbounded writers would fill it after hours ("No space left on
# device" with no full disk):
#   1. the systemd journal;
#   2. /var/log/osmocom/*.log - the stack logs with 'filter all 1', and mobile
#      runs with about twenty debug categories;
#   3. GSMTAP pcap captures, written continuously.
# They are bounded here, in the image, so the caps apply to every machine.

# 1. Journal: volatile (lost on reboot of a live system anyway) and capped.
#    Without RuntimeMaxUse journald allows itself 10 % of RAM.
mkdir -p "$ROOTFS/etc/systemd/journald.conf.d"
# --arm: the root is persistent, so is the journal (Storage=persistent,
# capped); volatile would drop the logs on every SD card reboot.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    cat > "$ROOTFS/etc/systemd/journald.conf.d/osmo-live.conf" <<'EOF'
[Journal]
Storage=persistent
SystemMaxUse=256M
RuntimeMaxUse=64M
EOF
else
cat > "$ROOTFS/etc/systemd/journald.conf.d/osmo-live.conf" <<'EOF'
# osmo-operator live: the root is in RAM, the journal must not eat it.
[Journal]
Storage=volatile
RuntimeMaxUse=64M
RuntimeKeepFree=256M
EOF
fi

# 2. Osmocom logs: rotate by size, not date - a verbose stack fills in an hour
#    what a daily rotation would never see.
#    copytruncate: daemons keep their descriptor open; otherwise rotation
#    leaves them a deleted file and the space is not released.
mkdir -p "$ROOTFS/etc/logrotate.d"
cat > "$ROOTFS/etc/logrotate.d/osmocom" <<'EOF'
/var/log/osmocom/*.log {
    size 32M
    rotate 3
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
EOF
# logrotate.timer runs only once a day: too late for a tmpfs.
cat > "$ROOTFS/etc/systemd/system/osmo-logrotate.service" <<'EOF'
[Unit]
Description=osmo-operator - rotation des journaux Osmocom (racine en RAM)
[Service]
Type=oneshot
```

```
ExecStart=/usr/sbin/logrotate /etc/logrotate.d/osmocom --state /run/osmo-logrotate.state
EOF
cat > "$ROOTFS/etc/systemd/system/osmo-logrotate.timer" <<'EOF'
[Unit]
Description=osmo-operator - rotation des journaux Osmocom toutes les 15 min
[Timer]
OnBootSec=10min
OnUnitActiveSec=15min
[Install]
WantedBy=timers.target
EOF
chroot "$ROOTFS" systemctl enable osmo-logrotate.timer 2>/dev/null || true

# 3. pcap captures: delete those older than one hour. The GSMTAP capture runs
#    continuously; it is for looking at what just happened, not for history.
mkdir -p "$ROOTFS/etc/tmpfiles.d"
cat > "$ROOTFS/etc/tmpfiles.d/osmo-captures.conf" <<'EOF'
# osmo-operator: captures live in RAM, keep them at most one hour.
d /run/user/0/osmo-nitb/captures 0755 root root 1h
EOF

# 3b. Ring buffer instead of an ever-growing file
# An hourly purge does not protect against one continuous capture filling RAM
# between two passes. A single -w file grows without limit; -C <MB> -W <n>
# turns it into a ring of n recycled files: the capture never stops and its
# footprint stays bounded.
#
# A wrapper rather than fixing callers: GSMTAP capture is started from the
# web dashboard (another repository) and from tools outside this repository.
# /usr/local/bin precedes /usr/bin in PATH, so plain "tcpdump" goes through it.
#
# It forces nothing if the caller already chose (-C or -W given), and without
# -w there is nothing to bound.
cat > "$ROOTFS/usr/local/bin/tcpdump" <<'TCPDUMPEOF'
#!/bin/sh
# tcpdump - osmo-operator wrapper: forces a ring buffer on file captures.
#
# The live root is a tmpfs: an unbounded capture eventually fills RAM.
#
# Two pitfalls handled:
#  -Z root : with -C/-W, tcpdump creates the next ring members after dropping
#            privileges (user "tcpdump"); without -Z it fails with
#            "Permission denied".
#  symlink : with -C/-W, tcpdump numbers the members (capture.pcap0, .pcap1...)
#            and the exact requested name never exists. <exact name> ->
#            current member is maintained so callers expecting it keep working.
#
# Tunables: OSMO_PCAP_RING_MB (32), OSMO_PCAP_RING_FILES (5).
mkdir -p "$ROOTFS/usr/bin"
# Known limitation / TODO: the mkdir/find lines below are build-time commands
# (copy /usr/local/bin to /usr/bin without overwriting dpkg binaries, -n, and
# never the tcpdump wrapper) that ended up inside this heredoc; the generated
# wrapper also lacks REAL=/usr/bin/tcpdump and has_w=0 initialisation.
find "$ROOTFS/usr/local/bin" -mindepth 1 -maxdepth 1 ! -name tcpdump \
    -exec cp -an {} "$ROOTFS/usr/bin/" \; 2>/dev/null || true
 has_ring=0; has_z=0; wfile=""; next_is_w=0
for a in "$@"; do
    if [ "$next_is_w" = 1 ]; then wfile="$a"; next_is_w=0; continue; fi
    case "$a" in
        -w)            has_w=1; next_is_w=1 ;;
        -w?*)          has_w=1; wfile="${a#-w}" ;;
        -C|-C*|-W|-W*) has_ring=1 ;;
        -Z|-Z*)        has_z=1 ;;
    esac
done
```

```
# Nothing to bound, or the caller already chose its ring: step aside.
if [ "$has_w" != 1 ] || [ "$has_ring" = 1 ] || [ -z "$wfile" ]; then
    exec "$REAL" "$@"
fi

# Symlink watcher. Started before exec: afterwards this process is tcpdump.
# $$ is unchanged across exec, so it follows tcpdump's lifetime exactly and
# stops with it - no orphan to clean up.
( ppid=$$
  n=0
  while [ "$n" -lt 300 ]; do
      [ -e "${wfile}0" ] && break
      kill -0 "$ppid" 2>/dev/null || exit 0
      sleep 0.2; n=$((n + 1))
  done
  while kill -0 "$ppid" 2>/dev/null; do
      newest=$(ls -t "${wfile}"[0-9]* 2>/dev/null | head -1)
      if [ -n "$newest" ] && [ "$(readlink "$wfile" 2>/dev/null)" != "$newest" ]; then
          ln -sfn "$newest" "$wfile"
      fi
      sleep 2
  done ) >/dev/null 2>&1 &

if [ "$has_z" = 1 ]; then
    exec "$REAL" -C "${OSMO_PCAP_RING_MB:-32}" -W "${OSMO_PCAP_RING_FILES:-5}" "$@"
fi
exec "$REAL" -Z root -C "${OSMO_PCAP_RING_MB:-32}" -W "${OSMO_PCAP_RING_FILES:-5}" "$@"
TCPDUMPEOF
chmod +x "$ROOTFS/usr/local/bin/tcpdump"
# The real binary must still be an ELF: if a module replaced it with a script,
# the wrapper would loop. Stop the build rather than ship that.
if [ "$(head -c 2 "$ROOTFS/usr/bin/tcpdump" 2>/dev/null)" = "#!" ]; then
    echo -e "  ${RED}✗ /usr/bin/tcpdump du rootfs est un script, pas le binaire dpkg${NC}" >&2
    exit 1
fi
echo -e "  ${GREEN}✓${NC} wrapper tcpdump (anneau pcap) dans /usr/local/bin, binaire dpkg intact dans /usr/bin"

# ── Full purge at every boot ──────────────────────────────────────
# The caps above prevent drift during a session; this starts each boot from
# a clean root. On a live system nothing is lost; with persistence these
# files would otherwise accumulate until the medium is full.
#
# Before the stack (Before=osmo-*.service): purging after start would erase
# the logs of the current session.
cat > "$ROOTFS/usr/local/sbin/osmo-purge.sh" <<'PURGEEOF'
#!/bin/bash
# osmo-purge.sh - start from a clean root. Called at boot by osmo-purge.service.
# Never touches configs (/etc/osmocom) or databases (HLR): only regenerated
# data - logs, captures, work files.
set -u

purge_dir() {   # $1=repertoire  $2=motif
    [ -d "$1" ] || return 0
    find "$1" -maxdepth 1 -type f -name "$2" -delete 2>/dev/null || true
}

# Stack logs
purge_dir /var/log/osmocom '*.log'
purge_dir /var/log/osmocom '*.log.*'
purge_dir /var/log/osmocom '*.gz'

# pcap captures (GSMTAP and others)
rm -rf /run/user/0/osmo-nitb/captures/* 2>/dev/null || true
purge_dir /var/log/osmocom '*.pcap'
find /tmp /var/tmp -maxdepth 2 -type f -name '*.pcap*' -delete 2>/dev/null || true
```

```
# Work files: FFT I/Q (hundreds of MB each)
find /dev/shm -maxdepth 1 -type f \( -name '*.cfile' -o -name '*.raw' \) -delete 2>/dev/null || true
# The same files outside /dev/shm: bridge mode writes /root/record.cfile and
# /root/record_ul.cfile continuously and stacks /root/osmo-rec/*.cfile up to
# its own 64 GB cap, which is meaningless when the root is a tmpfs.
find /root /tmp /var/tmp -maxdepth 2 -type f \( -name '*.cfile' -o -name '*.raw' \) -delete 2>/dev/null || true

# Live runtime directory, recreated by the stack at start
rm -rf /run/user/0/osmo-nitb/logs/* 2>/dev/null || true

# Volatile systemd journal
command -v journalctl >/dev/null 2>&1 && journalctl --rotate --vacuum-time=1s >/dev/null 2>&1

exit 0
PURGEEOF
chmod +x "$ROOTFS/usr/local/sbin/osmo-purge.sh"
cat > "$ROOTFS/etc/systemd/system/osmo-purge.service" <<'EOF'
[Unit]
Description=osmo-operator - purge des journaux, captures et fichiers de travail
DefaultDependencies=no
After=local-fs.target
Before=osmo-stp.service osmo-interstp.service osmo-msc.service osmo-bsc.service
Before=osmo-egprs-web.service shutdown.target
Conflicts=shutdown.target
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/sbin/osmo-purge.sh
[Install]
WantedBy=multi-user.target
EOF
chroot "$ROOTFS" systemctl enable osmo-purge.service 2>/dev/null || true

# SSH: allow root password login and enable the service at boot.
if [ -f "$ROOTFS/etc/ssh/sshd_config" ]; then
    sed -i \
        -e 's/^#\?PermitRootLogin.*/PermitRootLogin yes/' \
        -e 's/^#\?PasswordAuthentication.*/PasswordAuthentication yes/' \
        "$ROOTFS/etc/ssh/sshd_config"
    grep -q '^PermitRootLogin yes' "$ROOTFS/etc/ssh/sshd_config" || \
        echo 'PermitRootLogin yes' >> "$ROOTFS/etc/ssh/sshd_config"
fi
chroot "$ROOTFS" systemctl enable ssh 2>/dev/null || true

# ── Keyboard: fixed in the image, not asked at first boot ──────────────
# The layout is chosen at the start of the build (see "LES QUESTIONS");
# it is only written here, so no machine blocks on a menu at login.
cat > "$ROOTFS/etc/default/keyboard" <<KBCONF
XKBMODEL="pc105"
XKBLAYOUT="${OSMO_ISO_KB}"
XKBVARIANT=""
XKBOPTIONS=""
BACKSPACE="guess"
KBCONF
chroot "$ROOTFS" setupcon --force 2>/dev/null || true
chroot "$ROOTFS" dpkg-reconfigure -f noninteractive keyboard-configuration 2>/dev/null || true

# What remains at login: a reminder that waits for nothing.
# The command depends on the role: the hub has no GSM core, and
# start-native.sh would look for a BSC, MSC and BTS that do not exist there.
if [ "$ISO_ROLE" = "interstp" ]; then
    OSMO_START_HINT='Pour demarrer le hub SS7 : /opt/GSM/osmo-operator/launch/start-interstp.sh'
    OSMO_START_HINT2='  (etat des noeuds attaches : ./launch/start-interstp.sh --status)'
else
    OSMO_START_HINT='Pour demarrer la stack : /opt/GSM/osmo-operator/start-native.sh --node N'
    OSMO_START_HINT2='  (N de 1 a 9 : il fixe les point codes 1.<N>1.x du noeud)'
```

```
fi
cat > "$ROOTFS/etc/profile.d/01-osmo-disclaimer.sh" <<KBSCRIPT
#!/bin/bash
[ "\$(id -u)" -ne 0 ] && return 0
[ -n "\${OSMO_DISCLAIMER_SHOWN:-}" ] && return 0
export OSMO_DISCLAIMER_SHOWN=1
echo ""
echo -e "  \033[1;33mDisclaimer\033[0m - banc d'essai GSM/SS7 Osmocom."
echo -e "  A n'utiliser que sur un reseau radio \033[1mISOLE\033[0m (cage/attenuateur) ou"
echo -e "  sur une bande sous licence : emettre sur le spectre public est illegal."
echo -e "  Aucun service Osmocom n'est lance automatiquement sur cette ISO."
echo -e "  \033[1;33m${OSMO_START_HINT}\033[0m"
echo -e "  \033[0;36m${OSMO_START_HINT2}\033[0m"
echo -e "  clavier : \033[1;32m\$(awk -F\\" '/^XKBLAYOUT/{print \$2}' /etc/default/keyboard 2>/dev/null)\033[0m
 ↪  \033[0;36m(changer : osmo-keyboard)\033[0m"
echo ""
KBSCRIPT

chmod +x "$ROOTFS/etc/profile.d/01-osmo-disclaimer.sh"
rm -f "$ROOTFS/etc/profile.d/01-keyboard-setup.sh"

# Keyboard selection on demand only (osmo-keyboard), never between login
# and shell.
cat > "$ROOTFS/usr/local/bin/osmo-keyboard" <<'KBCMD'
#!/bin/bash
# osmo-keyboard - change the system keyboard layout, on demand.
[ "$(id -u)" -ne 0 ] && { echo "Root requis."; exit 1; }

if [ -n "$1" ]; then
    KB_LAYOUT="$1"
else
    echo ""
    echo -e "\033[1;36m== Configuration clavier ==\033[0m"
    echo "  1) fr    2) us    3) de    4) es    5) it"
    echo "  6) pt    7) gb    8) be    9) ch    0) autre"
    echo ""
    read -rp "  Choix [1] : " KB_CHOICE
    case "${KB_CHOICE:-1}" in
        1|"") KB_LAYOUT="fr" ;;
        2) KB_LAYOUT="us" ;;  3) KB_LAYOUT="de" ;;
        4) KB_LAYOUT="es" ;;  5) KB_LAYOUT="it" ;;
        6) KB_LAYOUT="pt" ;;  7) KB_LAYOUT="gb" ;;
        8) KB_LAYOUT="be" ;;  9) KB_LAYOUT="ch" ;;
        0) read -rp "  Layout (fr, us, ru, ar...) : " KB_LAYOUT
           KB_LAYOUT="${KB_LAYOUT:-us}" ;;
        *) KB_LAYOUT="fr" ;;
    esac
fi

loadkeys "$KB_LAYOUT" 2>/dev/null || true
cat > /etc/default/keyboard <<KBCONF
XKBMODEL="pc105"
XKBLAYOUT="${KB_LAYOUT}"
XKBVARIANT=""
XKBOPTIONS=""
BACKSPACE="guess"
KBCONF
setupcon --force 2>/dev/null || true
dpkg-reconfigure -f noninteractive keyboard-configuration 2>/dev/null || true
echo -e "  \033[1;32mClavier : ${KB_LAYOUT}\033[0m   (sans persistance, revient au reboot)"
KBCMD
chmod +x "$ROOTFS/usr/local/bin/osmo-keyboard"

# End of module: `. file` returns the status of its last command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
```

```
true
```

2.78 /opt/GSM/osmo-operator/iso_modules/84-comptes.sh

6736 octets, 108 lignes → 108 lignes

```
#!/bin/bash
# iso_modules/84-comptes.sh - step 8d: accounts, console/desktop autologin, linphone
# Sourced by build-iso.sh in numeric order (same shell, variables and functions);
# not meant to run on its own. A top-level `return` means "nothing to do here"
# (this is how --arm skips a step).

# ═══════════════════════════════════════════════════════════════════════
# Step 8d: accounts, session, installer - everything that contains apostrophes
# ═══════════════════════════════════════════════════════════════════════
# Written here rather than in the step 8 chroot, which is a single-quoted
# bash -c: one extra apostrophe changes the meaning of everything after it.
# Calamares configs, sudoers and systemd units are full of them, so they are
# written directly into "$ROOTFS" with the script's normal quoting.
echo -e "${GREEN}[8d/9] Comptes, session et installeur...${NC}"

# ── Account model ──────────────────────────────────────────────────────
# root is the working account: the bench is driven as root (start-native.sh,
# VTYs, tcpdump, netns) and the whole repository assumes it. The session opens
# as root and terminals opened in it are root without prompting.
#
# osmocom is a second, unprivileged sudoer account with its own UID - a
# sandbox for what does not need full privileges (browser, ordinary desktop
# session). It is used explicitly (log out and pick it in GDM, or
# "su - osmocom"), never by default.
chroot "$ROOTFS" env LC_ALL=C.UTF-8 LANGUAGE= bash -c "
set -u
id -u osmocom >/dev/null 2>&1 || useradd -m -s /bin/bash -c 'Compte osmocom (non privilegie)' osmocom
echo 'osmocom:osmo' | chpasswd
echo 'root:osmo'    | chpasswd
passwd -u root >/dev/null 2>&1 || true
for g in sudo adm dialout audio video plugdev netdev cdrom; do
    getent group \$g >/dev/null && usermod -aG \$g osmocom
done
" 2>/dev/null || echo -e "  ${YELLOW}!${NC} creation des comptes incomplete"
echo -e "  ${GREEN}✓${NC} comptes : ${CYAN}root${NC} (travail, mdp osmo) et ${CYAN}osmocom${NC} (non privilegie,
 ↪  sudoer, mdp osmo)"

# ── Console logs in as root ────────────────────────────────────────────
# On the live USB there is nobody to authenticate: a tty1 password protects
# nothing (whoever holds the medium holds the machine). --noclear keeps the
# boot messages on screen, often the only trace of a failed module.
install -d "$ROOTFS/etc/systemd/system/getty@tty1.service.d"
cat > "$ROOTFS/etc/systemd/system/getty@tty1.service.d/10-autologin-root.conf" <<'GETTY'
[Service]
ExecStart=
ExecStart=-/sbin/agetty --autologin root --noclear %I $TERM
GETTY
install -d "$ROOTFS/etc/systemd/system/serial-getty@ttyS0.service.d"
cat > "$ROOTFS/etc/systemd/system/serial-getty@ttyS0.service.d/10-autologin-root.conf" <<'GETTYS'
[Service]
ExecStart=
ExecStart=-/sbin/agetty --autologin root --keep-baud 115200,57600,38400,9600 %I $TERM
GETTYS
# Pi 4: console=serial0 -> ttyS0 (mini-UART, enable_uart=1) or ttyAMA0
# (PL011) depending on the overlay; both units get the same drop-in.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    install -d "$ROOTFS/etc/systemd/system/serial-getty@ttyAMA0.service.d"
    cp "$ROOTFS/etc/systemd/system/serial-getty@ttyS0.service.d/10-autologin-root.conf" \
```

```
       "$ROOTFS/etc/systemd/system/serial-getty@ttyAMA0.service.d/10-autologin-root.conf"
fi
echo -e "  ${GREEN}✓${NC} tty1 et console serie : ouverture automatique sur ${CYAN}root${NC}"

# ── Desktop autologin too ──────────────────────────────────────────
# Step 8 already sets AutomaticLogin=root in /etc/gdm3/custom.conf; it is
# rewritten here unconditionally so the setting is identical on the live USB
# and on the installed system (where shellprocess-osmo.conf writes it).
# Harmless when ISO_DESKTOP=0 (GDM not installed).
if [ -d "$ROOTFS/etc/gdm3" ] || [ "${ISO_DESKTOP:-0}" = "1" ]; then
    install -d "$ROOTFS/etc/gdm3"
    cat > "$ROOTFS/etc/gdm3/custom.conf" <<'GDMCONF'
[daemon]
AutomaticLoginEnable=true
AutomaticLogin=root
# X11 forced: under VirtualBox/QEMU the GNOME 42 Wayland session falls back to
# llvmpipe and is unusable.
WaylandEnable=false
GDMCONF
    sed -i "/pam_succeed_if.so user != root quiet_success/s/^/#/" \
        "$ROOTFS/etc/pam.d/gdm-password" "$ROOTFS/etc/pam.d/gdm-autologin" 2>/dev/null || true
    install -d "$ROOTFS/root/.config" "$ROOTFS/home/osmocom/.config"
    echo yes > "$ROOTFS/root/.config/gnome-initial-setup-done"
    echo yes > "$ROOTFS/home/osmocom/.config/gnome-initial-setup-done"
    chroot "$ROOTFS" chown -R osmocom:osmocom /home/osmocom 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} session graphique : ouverture automatique sur ${CYAN}root${NC} (osmocom au choix,
     ↪  apres deconnexion)"
fi

# ── Linphone: pre-provisioned account ──────────────────────────────
# The account must exist before the client's first start: Linphone does not
# reload linphonerc at runtime and rewrites it on exit, so an account added
# while it runs never sends REGISTER (Asterisk shows the endpoint
# "Unavailable" with no 401). Details of both pitfalls (live reload, presence
# publish) are in the header of configs/linphonerc.
if [ "${ISO_DESKTOP:-0}" = "1" ] && [ -f "$DIR/configs/linphonerc" ]; then
    # Both accounts plus /etc/skel: the session opens as root (gdm3 above),
    # osmocom stays available after logout, and skel covers accounts created by
    # the installer on the target system.
    for _lh in "$ROOTFS/root" "$ROOTFS/home/osmocom" "$ROOTFS/etc/skel"; do
        install -d "$_lh/.config/linphone"
        cp -a "$DIR/configs/linphonerc" "$_lh/.config/linphone/linphonerc"
    done
    chroot "$ROOTFS" chown -R osmocom:osmocom /home/osmocom 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} linphone : compte ${CYAN}linphone_A${NC} pre-provisionne (poste ${CYAN}100${NC},
     ↪  UDP vers 127.0.0.1:5060)"
fi


# End of module: `. file` returns the status of its last command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would silently stop everything. Always 0 here.
true
```

2.79 /opt/GSM/osmo-operator/iso_modules/85-extensions-gnome.sh

799 octets, 15 lignes → 15 lignes

```
#!/bin/bash
# iso_modules/85-extensions-gnome.sh - GNOME Shell extensions enabled by default
# Sourced by build-iso.sh in numeric order. A top-level `return` means nothing to do.
#
# EasyScreenCast is installed and enabled for all accounts by
# tools/gnome-extensions-install.sh (zip from extensions.gnome.org, sha256
```

```
# checked). chrome-gnome-shell comes from apt (desktop package list, 80-chroot.sh).
# Desktop only (--desktop), amd64. A failure (network) does not stop the build.
if [ "${ISO_ARCH:-amd64}" = "arm64" ] || [ "${ISO_DESKTOP:-0}" != "1" ]; then return 0; fi

if _EXT_LOG=$("$DIR/tools/gnome-extensions-install.sh" "$ROOTFS" 2>&1); then
    echo -e "  ${GREEN}✓${NC} ${_EXT_LOG##*$'\n'}"
else
    echo -e "  ${YELLOW}!${NC} extensions GNOME non posees :\n$_EXT_LOG" | tail -4 >&2
fi
```

2.80 /opt/GSM/osmo-operator/iso_modules/85-glava.sh

1423 octets, 27 lignes → 27 lignes

```
#!/bin/bash
# iso_modules/85-glava.sh - desktop audio visualiser (GLava + NCS Spectrum)
# Sourced by build-iso.sh in numeric order. A top-level `return` means nothing to do.
#
# GLava has no package in noble: tools/glava-build.sh builds it on the host
# (same Ubuntu as the image) with the NCS module patch and installs it into the
# rootfs via DESTDIR - binary, shaders in /etc/xdg/glava, launcher
# /usr/local/bin/osmo-glava, user service glava.service. Runtime libraries
# (libpulse0, libx11-6, libxext6, libxrender1, libxcomposite1, libgl1) already
# come with the GNOME desktop.
#
# Desktop only (--desktop), amd64. Disable with OSMO_ISO_GLAVA=0.
# A failure (network, missing -dev packages on the host) does not stop the
# build: the ISO ships without the visualiser, with a warning.
if [ "${ISO_ARCH:-amd64}" = "arm64" ] || [ "${ISO_DESKTOP:-0}" != "1" ]; then return 0; fi
if [ "${OSMO_ISO_GLAVA:-1}" != "1" ]; then
    echo -e "  ${YELLOW}!${NC} GLava desactive (OSMO_ISO_GLAVA=0)"
    return 0
fi

if _GLV_LOG=$("$DIR/tools/glava-build.sh" "$ROOTFS" 2>&1); then
    echo -e "  ${GREEN}✓${NC} ${_GLV_LOG##*$'\n'}"
    _GLV_MANQUE=$(chroot "$ROOTFS" ldd /usr/bin/glava 2>/dev/null | grep 'not found' || true)
    [ -z "$_GLV_MANQUE" ] || echo -e "  ${YELLOW}!${NC} GLava : bibliotheques absentes du rootfs :\n$_GLV_MANQUE"
     ↪  >&2
else
    echo -e "  ${YELLOW}!${NC} GLava non installe :\n$_GLV_LOG" | tail -4 >&2
fi
```

2.81 /opt/GSM/osmo-operator/iso_modules/85-installeur-bureau.sh

50810 octets, 930 lignes → 930 lignes

```
#!/bin/bash
# iso_modules/85-installeur-bureau.sh - Calamares installer, osmo-install, desktop (icons, launchers)
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# --arm: no Calamares, osmo-install or desktop on arm64 (no --desktop there).
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then return 0; fi

# ── INSTALLER ──────────────────────────────────────────────────────────
# The configuration lives in the repository (installer/calamares/) rather than
# in heredocs here: versioned, and checkable outside a build (plain YAML).
_CAL_SRC="$DIR/installer/calamares"
if [ "${ISO_DESKTOP:-0}" = "1" ] && [ -d "$_CAL_SRC" ]; then
    install -d "$ROOTFS/etc/calamares"
```

```
    cp -a "$_CAL_SRC/settings.conf" "$ROOTFS/etc/calamares/"
    cp -a "$_CAL_SRC/modules"       "$ROOTFS/etc/calamares/"
    cp -a "$_CAL_SRC/branding"      "$ROOTFS/etc/calamares/"
    # The osmosupplements python job ("Supplements" page) goes in the SYSTEM
    # modules directory, the one Calamares' modules-search always scans.
    _CAL_MODS="$ROOTFS/usr/lib/$(chroot "$ROOTFS" dpkg-architecture -qDEB_HOST_MULTIARCH 2>/dev/null || echo
     ↪  x86_64-linux-gnu)/calamares/modules"
    install -d "$_CAL_MODS/osmosupplements"
    install -m644 "$_CAL_SRC/jobs/osmosupplements/module.desc" "$_CAL_SRC/jobs/osmosupplements/main.py" \
        "$_CAL_MODS/osmosupplements/"

    # Branding images: reuse the wallpaper frozen at build time instead of
    # adding binaries to the repository. Calamares scales them.
    _WPI="$DIR/configs/gsm-lab-wallpaper.png"
    if [ -f "$_WPI" ]; then
        cp "$_WPI" "$ROOTFS/etc/calamares/branding/osmo/logo.png"
    else
        # Without an image Calamares logs an error per screen: drop the keys
        # rather than leave them pointing at missing files.
        sed -i '/^images:/,/^slideshow:/{/productLogo\|productIcon/d}' \
            "$ROOTFS/etc/calamares/branding/osmo/branding.desc"
    fi
    # Welcome screen and first slide: the "Telecom Operator / No RF"
    # background (gsm-lab-ville.jpg), the same as the live desktop.
    _VILLE="$DIR/configs/gsm-lab-ville.jpg"
    if [ -f "$_VILLE" ]; then
        cp "$_VILLE" "$ROOTFS/etc/calamares/branding/osmo/welcome.jpg"
    else
        sed -i '/^images:/,/^slideshow:/{/productWelcome/d}' \
            "$ROOTFS/etc/calamares/branding/osmo/branding.desc"
    fi

    # ── LAUNCHER ──────────────────────────────────────────────────────────
    # pkexec rather than sudo: the installer is started from the desktop, where
    # there is no terminal to type a password. The live session runs as root,
    # but the osmocom user must be able to start the installer too.
    cat > "$ROOTFS/usr/local/bin/osmo-install" <<'INSTALLER'
#!/bin/bash
# Starts the system installer. Live medium only.
set -u
if [ ! -r /etc/calamares/settings.conf ]; then
    echo "Calamares n est pas installe sur cette image (ISO_DESKTOP=0 ?)." >&2
    exit 1
fi

# Become root RIGHT AWAY, for this script - not just for calamares: what
# follows (mounting the medium, unmounting a target left open) needs root.
# pkexec passes DISPLAY and XAUTHORITY along, so the GUI still opens.
if [ "$(id -u)" -ne 0 ]; then
    exec pkexec --disable-internal-agent "$0" "$@"
fi

# ── Locate the squashfs without assuming WHERE it is ──────────────────────
# In normal mode live-boot mounts the medium on /run/live/medium and the
# squashfs sits at live/filesystem.squashfs. With "toram=filesystem.squashfs"
# live-boot copies only THE FILE to the root of a tmpfs, then moves that tmpfs
# onto /run/live/medium: the squashfs ends up at
#     /run/live/medium/filesystem.squashfs
# and the "live/" subdirectory is gone, so anything that hardcodes the path
# misses it. Instead of rebuilding an assumed tree, the file is located by
# three increasingly broad means and presented to Calamares at a path that
# does not depend on the boot mode: the one named in unpackfs.conf.
SQ=""

# 1. Canonical path of a normal boot. If present, nothing to do.
[ -e /run/live/medium/live/filesystem.squashfs ] \
```

```
    && SQ=/run/live/medium/live/filesystem.squashfs

# 2. The loop device backing the running root. The most reliable source: not
#    "a" squashfs found somewhere, but THE one this system runs on. Holds in
#    normal, toram and persistent modes.
if [ -z "$SQ" ]; then
    _loop=$(findmnt -no SOURCE /run/live/rootfs/filesystem.squashfs 2>/dev/null || true)
    if [ -n "$_loop" ]; then
        _bf=$(losetup -nO BACK-FILE "$_loop" 2>/dev/null || true)
        [ -n "$_bf" ] && [ -e "$_bf" ] && SQ="$_bf"
    fi
fi

# 3. Last resort: any loop device whose backing file is a squashfs. Covers a
#    live-boot that named the mount point differently (persistence:
#    /run/live/persistence/sdX).
if [ -z "$SQ" ]; then
    SQ=$(losetup -anO BACK-FILE 2>/dev/null | grep -m1 '\.squashfs$' || true)
    [ -n "$SQ" ] && [ -e "$SQ" ] || SQ=""
fi

if [ -z "$SQ" ]; then
    echo "Squashfs introuvable - ce systeme ne tourne pas depuis une cle live," >&2
    echo "ou l image n est plus lisible. Rien a installer." >&2
    exit 1
fi

# ── THE STABLE PATH, THE ONE CALAMARES READS ──────────────────────────
# Bind-mount the FILE, not its directory: the origin directory changes shape
# from one boot mode to the next, the file does not. /run is a tmpfs on a live
# system, hence writable - unlike /run/live/medium in toram mode, which
# live-boot moves read-only, so the missing "live/" could not be created there.
SRC=/run/osmo-install-src
mkdir -p "$SRC/live"
if ! mountpoint -q "$SRC/live/filesystem.squashfs"; then
    : > "$SRC/live/filesystem.squashfs"
    mount --bind "$SQ" "$SRC/live/filesystem.squashfs" || {
        echo "Impossible de presenter $SQ a l installeur." >&2
        exit 1
    }
fi
echo "Image source : $SQ  ->  $SRC/live/filesystem.squashfs"

# Test the FILE, not the directory: the directory exists even when empty, and
# a failure would then only show up later in Calamares, with the disk already
# repartitioned.
if [ ! -s "$SRC/live/filesystem.squashfs" ]; then
    echo "Le squashfs presente a l installeur est vide - rien a installer." >&2
    exit 1
fi

# ── Clean up the target of a previous attempt ─────────────────────────
# When a step fails, Calamares skips all following jobs, "umount" included: the
# target stays mounted under /tmp/calamares-root-*. On the next run it sees a
# mounted disk, removes "Erase disk" from the list and only offers manual
# partitioning.
for _t in /tmp/calamares-root-*; do
    [ -d "$_t" ] || continue
    if mountpoint -q "$_t"; then
        umount -R "$_t" 2>/dev/null || {
            echo "Cible $_t encore montee et non demontable - fermez ce qui l occupe" >&2
            echo "(un terminal, un gestionnaire de fichiers) ou redemarrez." >&2
            exit 1
        }
    fi
    rmdir "$_t" 2>/dev/null || true
```

```
done

# ── GUARD: an invalid generated page must not block the installation ────────
# The two pages below are WRITTEN at every launch, hence checked at every
# launch: if the YAML does not parse, the minimal version (the one that does
# nothing) is put back and the operator can still install. yaml.safe_load
# reads what yaml-cpp reads. Without python3-yaml the check is skipped.
_yamlok() {
    python3 -c "import yaml" 2>/dev/null || return 0
    python3 - "$1" <<'YAMLCHECK'
import io, sys, yaml
lines = io.open(sys.argv[1], encoding="utf-8").read().splitlines()
i = next((n for n, l in enumerate(lines) if l.strip() == "---"), -1)
d = yaml.safe_load("\n".join(lines[i + 1:]))
if not isinstance(d, dict):
    raise SystemExit("pas une table YAML")
YAMLCHECK
}

# ── MIRROR: the installer's "Miroir Ubuntu" page follows the network ────────
# packaging/apt-mirror.sh --list measures the mirrors from here (5 s max each);
# the page offers those that answer, fastest first, the first one preselected,
# plus "the live key's one" (no change). The contextualprocess@mirror module
# writes the choice into the target, after unpackfs.
# NO BACKSLASH IN THESE COMMANDS: they sit in a double-quoted YAML string,
# where only a handful of escapes exist. \s and \S (sed classes) are not among
# them and yaml-cpp would reject the WHOLE file. Hence the POSIX classes
# ([[:space:]], [^[:space:]]) and the sed expression in escaped quotes (\" is
# a valid YAML escape), which also avoids escaping the space in the replacement.
_pm=/etc/calamares/modules/packagechooser-mirror.conf
_cm=/etc/calamares/modules/contextualprocess-mirror.conf
_am=/opt/GSM/osmo-operator/packaging/apt-mirror.sh
if [ -f "$_pm" ] && [ -f "$_cm" ] && [ -f "$_am" ]; then
    _cur="$(sed -nE 's|^deb (http[^ ]+) .*|\1|p' /etc/apt/sources.list 2>/dev/null | head -1)"
    _suite="$(. /etc/os-release 2>/dev/null; echo "${VERSION_CODENAME:-noble}")"
    _mlist="$(bash "$_am" --list "$_suite" 2>/dev/null || true)"
    _n=0
    {
        echo "---"
        echo "mode: required"
        echo "method: legacy"
        echo "labels:"
        echo "    step: \"Miroir Ubuntu\""
        if [ -n "$_mlist" ]; then echo "default: m1"; else echo "default: keep"; fi
        echo "items:"
        echo "    - id: keep"
        echo "      name: \"Celui de la cle live\""
        echo "      description: \"Le systeme installe garde le miroir de la cle : ${_cur:-archive.ubuntu.com}.\""
        printf '%s\n' "$_mlist" | while IFS="$(printf '\t')" read -r _ms _url; do
            [ -n "$_url" ] || continue
            _n=$((_n + 1))
            _tag=""; [ "$_n" = 1 ] && _tag=" - le plus rapide d ici"
            echo "    - id: m${_n}"
            echo "      name: \"${_url#http://}\""
            echo "      description: \"${_url} - InRelease en ${_ms} ms${_tag}.\""
        done
    } > "$_pm"
    {
        echo "---"
        echo "dontChroot: false"
        echo "timeout: 60"
        echo "packagechooser_mirror:"
        echo "    \"keep\":"
        echo "        - \"-/bin/true\""
        _n=0
        printf '%s\n' "$_mlist" | while IFS="$(printf '\t')" read -r _ms _url; do
```

```
            [ -n "$_url" ] || continue
            _n=$((_n + 1))
            echo "    \"m${_n}\":"
            echo "        - \"-/bin/bash -c 'sed -i -E \\\"s,^deb[[:space:]]+https?://[^[:space:]]+,deb
↪   ${_url},\\\" /etc/apt/sources.list; [ -f /etc/apt/sources.list.d/ubuntu.sources ] && sed -i -E
↪   \\\"s,^URIs:[[:space:]]+https?://[^[:space:]]+,URIs: ${_url},\\\" /etc/apt/sources.list.d/ubuntu.sources;
↪   echo [mirror] ${_url}'\""
        done
    } > "$_cm"
    if ! _yamlok "$_pm" || ! _yamlok "$_cm"; then
        echo "Miroir Ubuntu : page generee ILLISIBLE - retour au choix minimal" >&2
        printf '%s\n' "---" "mode: required" "method: legacy" "labels:" \
            "    step: \"Miroir Ubuntu\"" "default: keep" "items:" \
            "    - id: keep" "      name: \"Celui de la cle live\"" \
            "      description: \"Le systeme installe garde le miroir de la cle.\"" > "$_pm"
        printf '%s\n' "---" "dontChroot: false" "timeout: 60" \
            "packagechooser_mirror:" "    \"keep\":" "        - \"-/bin/true\"" > "$_cm"
    elif [ -n "$_mlist" ]; then
        echo "Miroir Ubuntu : $(printf '%s\n' "$_mlist" | wc -l) miroirs mesures, le plus rapide : $(printf '%s\n'
↪   "$_mlist" | head -1 | cut -f2)"
    else
        echo "Miroir Ubuntu : aucune mesure (pas de reseau ?) - la cible garde ${_cur:-archive.ubuntu.com}"
    fi
fi

# ── NVIDIA: the installer's "Pilotes graphiques" page follows the machine ──
# Calamares cannot grey out an entry based on hardware: its chooser page and
# the module that installs are WRITTEN at EVERY launch, from lspci:
#   - NVIDIA card seen -> the "nvidia-driver-610" entry is named after the card
#     and preselected;
#   - no NVIDIA card: the page says so and any choice has no effect
#     (contextualprocess re-runs the lspci test in the target).
# On the installed system, tools/osmo-drivers.sh ("Pilotes graphiques" icon)
# shows the same state and allows installing or updating later.
#
# ONE FIXED VERSION: nvidia-driver-610 (noble-updates/multiverse), installed
# like any package: sudo apt install nvidia-driver-610.
# OSMO_NVIDIA_DRIVER selects another one at build time.
_pc=/etc/calamares/modules/packagechooser-nvidia.conf
_cp=/etc/calamares/modules/contextualprocess-nvidia.conf
_nvdrv="${OSMO_NVIDIA_DRIVER:-nvidia-driver-610}"
if [ -f "$_pc" ] && [ -f "$_cp" ]; then
    _nv="$(lspci -d 10de: 2>/dev/null | head -1 | cut -d: -f3- | sed 's/^ //; s/ (rev.*//')"
    # No double quote in this command: it sits between single quotes (bash -c)
    # inside a double-quoted YAML string - either would end the string. The
    # target's /run is an empty tmpfs (mount.conf) and /etc/resolv.conf points
    # there: resolvers are written there for the duration of apt.
    _apt_cmd='mkdir -p /run/systemd/resolve; [ -s /run/systemd/resolve/stub-resolv.conf ] || { echo nameserver
↪   1.1.1.1 > /run/systemd/resolve/stub-resolv.conf; echo nameserver 8.8.8.8 >>
↪   /run/systemd/resolve/stub-resolv.conf; }; export DEBIAN_FRONTEND=noninteractive; lspci -n 2>/dev/null | grep
↪   -qi 10de: || { echo [nvidia] aucune carte NVIDIA : rien a installer; exit 0; }; apt-get update -qq || { echo
↪   [nvidia] pas de reseau : pilote non installe; exit 0; }'
    {
        echo "---"
        echo "mode: optional"
        echo "method: legacy"
        echo "labels:"
        echo "    step: \"Pilotes graphiques\""
        if [ -n "$_nv" ]; then echo "default: nvidia"; else echo "default: none"; fi
        echo "items:"
        echo "    - id: none"
        echo "      name: \"Aucun pilote supplementaire\""
        if [ -n "$_nv" ]; then
            _state=""
            dpkg -s "$_nvdrv" >/dev/null 2>&1 && _state=" (deja installe sur la cle)"
```

```
            echo "      description: \"Carte detectee : ${_nv}. Le pilote libre (nouveau) du noyau, comme sur la
↪ cle live.\""
            echo "    - id: nvidia"
            echo "      name: \"${_nvdrv} (proprietaire)\""
            echo "      description: \"Carte ${_nv} : installe ${_nvdrv}${_state} depuis les depots Ubuntu (reseau
↪ requis).\""
        else
           echo "      description: \"AUCUNE carte NVIDIA detectee (lspci) : rien a installer sur cette machine.\""
        fi
    } > "$_pc"
    {
        echo "---"
        echo "dontChroot: false"
        echo "timeout: 1800"
        echo "packagechooser_nvidia:"
        echo "    \"none\":"
        echo "        - \"-/bin/true\""
        echo "    \"nvidia\":"
        echo "        - \"-/bin/bash -c '${_apt_cmd}; apt-get install -y ${_nvdrv} || echo [nvidia] ${_nvdrv} non
↪ installable - le pilote libre nouveau reste en place; update-initramfs -u >/dev/null 2>&1 || true'\""
    } > "$_cp"
    if ! _yamlok "$_pc" || ! _yamlok "$_cp"; then
        echo "NVIDIA : page generee ILLISIBLE - retour au choix minimal" >&2
        printf '%s\n' "---" "mode: optional" "method: legacy" "labels:" \
            "    step: \"Pilotes graphiques\"" "default: none" "items:" \
            "    - id: none" "      name: \"Aucun pilote supplementaire\"" \
            "      description: \"Le pilote libre (nouveau) du noyau.\"" > "$_pc"
        printf '%s\n' "---" "dontChroot: false" "timeout: 1800" \
            "packagechooser_nvidia:" "    \"none\":" "        - \"-/bin/true\"" > "$_cp"
    elif [ -n "$_nv" ]; then
        echo "NVIDIA : $_nv - pilote propose : $_nvdrv"
    else
        echo "NVIDIA : aucune carte detectee - la page le dira, sans effet"
    fi
fi
exec /usr/bin/calamares -d "$@"
INSTALLER
    chmod +x "$ROOTFS/usr/local/bin/osmo-install"

    # ── CONKY: the bench dashboard, in EVERY GNOME session ──────────────────
    # /etc/xdg/autostart applies to all accounts: root on the live key, the
    # user created by Calamares on disk (the file travels in the squashfs).
    # The config and the status script live in the repository
    # (/opt/GSM/osmo-operator, present on both): configs/conky/osmo-conky.conf,
    # tools/conky-osmo-status.sh.
    # ── A WATCHER, NOT A ONE-SHOT LAUNCH ────────────────────────────────────
    # osmo-panel.py is the window that DISPLAYS /run/osmo-fft/panel.png: without
    # it the inset does not exist on screen, whatever osmo-fft-snap renders. A
    # crash, an OOM kill or a manual kill would remove it for the rest of the
    # session, so this script restarts whatever is missing every five seconds,
    # never doubling an instance.
    cat > "$ROOTFS/usr/local/bin/osmo-desktop-panel" <<'PANEL'
#!/bin/sh
# osmo-desktop-panel - keeps the inset and Conky alive for the whole session.
# Started by /etc/xdg/autostart/osmo-conky.desktop. See iso_modules/85-installeur-bureau.sh.
REPO=/opt/GSM/osmo-operator
export OSMO_REPO="$REPO"
# The LAB GSM banner is not baked into the wallpaper: osmo-topzone.py renders
# it live. Tell the wallpaper renderer.
export OSMO_LIVE_BANNER=1

# ── A WIDGET LEFT OVER FROM THE PREVIOUS SESSION IS NOT A WIDGET ────────────
# A conky or python widget from a previous X session may still be running with
# an old DISPLAY (e.g. :0 while GNOME restarted on :1): it cannot paint
# anything, but "pgrep -x conky" would count it as alive and nothing would be
# restarted. Only processes looking at OUR display count; the others are
```

```
# killed, they can no longer serve anyone.
osmo_vivant() {
    _trouve=1
    for _p in $(pgrep "$@" 2>/dev/null); do
        _d="$(tr '\0' '\n' < "/proc/$_p/environ" 2>/dev/null | sed -n 's/^DISPLAY=//p' | head -1)"
        # Kill ONLY what was read and looks elsewhere. An unreadable environ,
        # or one without DISPLAY, proves nothing: that process counts as
        # alive - confusing "absent" with "invisible" would spawn a duplicate
        # of each widget on every pass.
        if [ -n "$_d" ] && [ "$_d" != "$DISPLAY" ]; then kill "$_p" 2>/dev/null
        else _trouve=0
        fi
    done
    return $_trouve
}
# The watcher must not outlive its own X server: otherwise the old one keeps
# running next to the new one and restarts into the void forever.
osmo_display_vivant() {
    [ -n "$DISPLAY" ] || return 0          # no named display: cannot judge
    _n="${DISPLAY#*:}"; _n="${_n%%.*}"
    [ -S "/tmp/.X11-unix/X$_n" ]
}

# ── CONKY ARROWS ON THE KEYBOARD: Ctrl+AltGr+Left / Right ──────────────────
# A gschema.override does NOT apply to a RELOCATABLE schema, so the shortcut
# is written here, in the session, once per boot: the only place that runs
# UNDER the user's account, where gsettings writes into their dconf.
# <Mod5> is ISO_Level3_Shift, i.e. AltGr (xmodmap -pm confirms it).
osmo_raccourci() {
    command -v gsettings >/dev/null 2>&1 || return 0
    _k=org.gnome.settings-daemon.plugins.media-keys.custom-keybinding
    _p=/org/gnome/settings-daemon/plugins/media-keys/custom-keybindings
    gsettings set org.gnome.settings-daemon.plugins.media-keys custom-keybindings \
        "['$_p/osmo-op-next/','$_p/osmo-op-prev/']" 2>/dev/null || return 0
    gsettings set "$_k:$_p/osmo-op-next/" name 'osmo-operator : arret suivant (Conky)' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-next/" command '/usr/local/bin/osmo-op --next' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-next/" binding '<Control><Mod5>Right' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-prev/" name 'osmo-operator : arret precedent (Conky)' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-prev/" command '/usr/local/bin/osmo-op --prev' 2>/dev/null
    gsettings set "$_k:$_p/osmo-op-prev/" binding '<Control><Mod5>Left' 2>/dev/null
}
osmo_raccourci

# Conky must find the GNOME desktop already painted (own_window_type
# desktop), otherwise it ends up behind the wallpaper.
sleep 6
while osmo_display_vivant; do
    osmo_vivant -f "$REPO/tools/osmo-panel.py" || \
        "$REPO/tools/osmo-panel.py" >>/tmp/osmo-panel.log 2>&1 &
    # The dino is not on the desktop: it is an APPLICATION
    # (data/desktop/osmo-dino.desktop, osmo-dino.py --app).
    # Timeslot activity probe (osmo-bsc conf + VTY) for the banner
    osmo_vivant -f "$REPO/tools/osmo-ts-probe.py" || \
        "$REPO/tools/osmo-ts-probe.py" >>/tmp/osmo-ts-probe.log 2>&1 &
    # Live LAB GSM top zone (blinking timeslots + app welcome)
    osmo_vivant -f "$REPO/tools/osmo-topzone.py" || \
        "$REPO/tools/osmo-topzone.py" >>/tmp/osmo-topzone.log 2>&1 &
    # The bottom launcher bar is off the desktop by default: 2G/4G start from
    # the icons and launch.sh; OSMO_LAUNCHER=1 in the session environment
    # restores it.
    if [ "${OSMO_LAUNCHER:-0}" = 1 ]; then
        osmo_vivant -f "$REPO/tools/osmo-launcher.py" || \
            "$REPO/tools/osmo-launcher.py" >>/tmp/osmo-launcher.log 2>&1 &
    fi
    osmo_vivant -x conky || \
        conky --daemonize -c "$REPO/configs/conky/osmo-conky.conf" >>/tmp/osmo-conky.log 2>&1
```

```
    sleep 5
done
PANEL
    chmod 755 "$ROOTFS/usr/local/bin/osmo-desktop-panel"

    cat > "$ROOTFS/etc/xdg/autostart/osmo-conky.desktop" <<'CONKY'
[Desktop Entry]
Type=Application
Name=Conky osmo-operator
Comment=Tableau de bord du banc GSM (coeur, radio, abonnes, services)
Exec=/usr/local/bin/osmo-desktop-panel
Icon=utilities-system-monitor
Terminal=false
NoDisplay=true
OnlyShowIn=GNOME;
X-GNOME-Autostart-enabled=true
X-GNOME-Autostart-Delay=6
CONKY
    chmod 644 "$ROOTFS/etc/xdg/autostart/osmo-conky.desktop"
    chmod +x "$ROOTFS/opt/GSM/osmo-operator/tools/osmo-dino.py" \
             "$ROOTFS/opt/GSM/osmo-operator/tools/conky-osmo-status.sh" \
             "$ROOTFS/opt/GSM/osmo-operator/tools/osmo-drivers.sh" \
             "$ROOTFS/opt/GSM/osmo-operator/tools/osmo-fft-snap.py" \
             "$ROOTFS/opt/GSM/osmo-operator/tools/osmo-panel.py" \
             "$ROOTFS/opt/GSM/osmo-operator/tools/wallpaper-render.py" \
             "$ROOTFS/opt/GSM/osmo-operator/tools/osmo-wallpaper.sh" 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} Conky du banc : autostart GNOME (live et disque), config
     ↪  ${CYAN}configs/conky/osmo-conky.conf${NC}"

    # The file lives in the repository (data/desktop/), not in a heredoc here:
    # the native install (install_modules/80-bureau.sh) and the .deb install
    # the SAME one.
    # ALL the repository's shortcuts, EXCEPT osmo-multi.desktop: it only makes sense once the Supplements are
    # installed (docker, the operator image, osmo-multi.conf), and
    # addition.sh --multi installs it at that point. The dock shows it when it
    # exists (favorite-apps already names it, iso_modules/80-chroot.sh).
    for _d in "$DIR"/data/desktop/*.desktop; do
        [ -f "$_d" ] || continue
        [ "$(basename "$_d")" = osmo-multi.desktop ] && continue
        install -m644 "$_d" "$ROOTFS/usr/share/applications/$(basename "$_d")"
    done

    # ── NO ICONS ON THE DESKTOP: THE DOCK IS ENOUGH ──────────────────────
    # Shortcuts are not copied to ~/Desktop or ~/Bureau: they live in
    # /usr/share/applications (above) and in the dock favorites (favorite-apps,
    # iso_modules/80-chroot.sh). Leftovers from older images are removed,
    # including the DING trust mechanism (osmo-trust-desktop).
    rm -f "$ROOTFS/root/Desktop"/*.desktop "$ROOTFS/root/Bureau"/*.desktop \
          "$ROOTFS/home/osmocom/Desktop"/*.desktop "$ROOTFS/home/osmocom/Bureau"/*.desktop \
          "$ROOTFS/usr/local/bin/osmo-trust-desktop" \
          "$ROOTFS/etc/xdg/autostart/osmo-trust-desktop.desktop" 2>/dev/null || true
    chroot "$ROOTFS" chown -R osmocom:osmocom /home/osmocom 2>/dev/null || true

    # ── LAUNCHER AND TUTORIAL: DESKTOP ICONS ─────────────────────────────
    # Repository icons (data/*.svg), not theme names: "call-start" is GREEN in
    # Adwaita with no red variant, and a missing icon name is SILENTLY replaced
    # by a grey rectangle.
    install -d "$ROOTFS/usr/share/icons/hicolor/scalable/apps" \
              "$ROOTFS/usr/share/osmo-operator/icons" \
              "$ROOTFS/usr/share/osmo-operator"
    for _ic in osmo-launch osmo-multi osmo-tutorial claude osmo-paint osmo-lte osmo-dino; do
        [ -f "$DIR/data/$_ic.svg" ] || continue
        cp -f "$DIR/data/$_ic.svg" \
              "$ROOTFS/usr/share/icons/hicolor/scalable/apps/$_ic.svg"
        # THE COPY THAT MATTERS FOR THE DESKTOP. See the block right below.
        cp -f "$DIR/data/$_ic.svg" \
```

```
            "$ROOTFS/usr/share/osmo-operator/icons/$_ic.svg"
        chmod 644 "$ROOTFS/usr/share/osmo-operator/icons/$_ic.svg"
    done

    # The .desktop files POINT TO THE FILE, NOT TO THE ICON NAME.
    # "Icon=osmo-launch" is a name resolved through the theme and
    # /usr/share/icons/hicolor/icon-theme.cache; a name that does not resolve
    # is SILENTLY replaced by a blank page. Two measures:
    #   - Icon= as an ABSOLUTE PATH (below): bypasses theme and cache;
    #   - the cache is rebuilt anyway, for the applications menu, which still
    #     resolves by name.
    if chroot "$ROOTFS" sh -c 'command -v gtk-update-icon-cache' >/dev/null 2>&1; then
        chroot "$ROOTFS" gtk-update-icon-cache -f -q /usr/share/icons/hicolor 2>/dev/null || true
    fi
    [ -f "$DIR/data/tutorial.html" ] && \
        cp -f "$DIR/data/tutorial.html" "$ROOTFS/usr/share/osmo-operator/tutorial.html"

    # A copy in /usr/share is the offline fallback: the only path that stays
    # readable when the dashboard does not answer.
    cat > "$ROOTFS/usr/local/bin/osmo-tutorial" <<'TUTO'
#!/bin/bash
# osmo-tutorial - opens the quick-start.
#
# OVER HTTP, NOT file://. The dashboard already serves this file on 8080: the
# page shown is the authoritative one (the repository's, copied at each build),
# the same as from another machine of the bench, and relative links to the
# dashboard work. file:// remains as a fallback when the dashboard is down.
set -u
URL="${OSMO_TUTORIAL_URL:-http://127.0.0.1:8080/tutorial.html}"
PORT="${URL##*:}"; PORT="${PORT%%/*}"
if (exec 3<>"/dev/tcp/127.0.0.1/${PORT}") 2>/dev/null; then
    exec 3>&- 2>/dev/null
    exec xdg-open "$URL"
fi
echo "Dashboard injoignable sur $PORT - repli file:// sur la copie de /usr/share" >&2
exec xdg-open "file:///usr/share/osmo-operator/tutorial.html"
TUTO
    chmod +x "$ROOTFS/usr/local/bin/osmo-tutorial"
    # Same launcher for the help page (the "?"), served by the dashboard
    install -m755 "$DIR/tools/osmo-info" "$ROOTFS/usr/local/bin/osmo-info"

    # The file lives in the repository (data/desktop/), not in a heredoc here:
    # the native install (install_modules/80-bureau.sh) and the .deb install
    # the SAME one.
    install -m644 "$DIR/data/desktop/osmo-launch.desktop" "$ROOTFS/usr/share/applications/osmo-launch.desktop"
    # No separate DSP icon: DSP is the bench default; the phone icon's right
    # click offers the other chain ("Lancer en mode gr-gsm").
    rm -f "$ROOTFS/usr/share/applications/osmo-dsp.desktop"

    # osmo-multi (antenna, multi-operator) is NOT installed here: its launcher
    # start-multi.sh needs docker + the image + the SS7 topology, which only
    # exist after the supplement; addition.sh installs it at the end of a
    # successful SS7 install. Its SVG stays installed (icons block above) so
    # that deferred install finds the image.

    # ── SUPPLEMENTS: THE CHECKLIST WINDOW ──────────────────────────────
    # Same shape as osmo-update-anim: a bench GTK window with a VTE terminal
    # inside (tools/osmo-gtk-run.py), control given back only once the end has
    # been read. addition.sh opens its own checklist (zenity) when DISPLAY is
    # set; the terminal remains useful for the long, verbose rest (apt, then
    # the Osmocom build). No dock favorite opens gnome-terminal.
    cat > "$ROOTFS/usr/local/bin/osmo-addition-anim" <<'ADDGUI'
#!/bin/bash
# osmo-addition-anim - the supplements (addition.sh) in a bench GTK window: a
# VTE terminal inside, for the checklist and then the long, verbose rest (apt,
# build); the window stays open at the end with the status. Root through
```

```
# pkexec, DISPLAY/XAUTHORITY and the session proxy carried over
# (osmo-gtk-run --root).
set -u
REPO=/opt/GSM/osmo-operator
SCRIPT=$REPO/addition.sh
if [ ! -x "$SCRIPT" ]; then
    command -v zenity >/dev/null 2>&1 && \
        zenity --error --text="addition.sh introuvable : $SCRIPT" 2>/dev/null
    exit 1
fi
# --progress: progress bar + step, the console (VTE) folded behind "console"
exec python3 "$REPO/tools/osmo-gtk-run.py" --progress --root --title "osmo-operator - supplements" -- "$SCRIPT"
 ↪  "$@"

ADDGUI
    chmod +x "$ROOTFS/usr/local/bin/osmo-addition-anim"

    # The file lives in the repository (data/desktop/), not in a heredoc here:
    # the native install (install_modules/80-bureau.sh) and the .deb install
    # the SAME one.
    install -m644 "$DIR/data/desktop/osmo-addition.desktop" "$ROOTFS/usr/share/applications/osmo-addition.desktop"

    # The file lives in the repository (data/desktop/), not in a heredoc here:
    # the native install (install_modules/80-bureau.sh) and the .deb install
    # the SAME one.
    install -m644 "$DIR/data/desktop/osmo-tutorial.desktop" "$ROOTFS/usr/share/applications/osmo-tutorial.desktop"
    # The Info page (English): every bench module, by icon, launch bar or CLI,
    # with the code behind it. Icon in the favorites.
    install -d "$ROOTFS/usr/share/osmo-operator"
    install -m644 "$DIR/configs/info.html" "$ROOTFS/usr/share/osmo-operator/info.html"
    install -d "$ROOTFS/usr/share/osmo-operator/icons"
    install -m644 "$DIR/data/osmo-info.svg" "$ROOTFS/usr/share/osmo-operator/icons/osmo-info.svg"
    install -m644 "$DIR/data/desktop/osmo-info.desktop" "$ROOTFS/usr/share/applications/osmo-info.desktop"

    # ── CLAUDE: launcher + menu entry ──────────────────────────────
    # Claude Code is NOT in the ISO (it is installed by the supplement): the
    # launcher runs the install (addition.sh --claude) on the first click if
    # claude is missing, then opens it.
    cat > "$ROOTFS/usr/local/bin/osmo-claude-anim" <<'CLA'
#!/bin/bash
# osmo-claude-anim - Claude Code in a bench GTK window (VTE terminal: claude is
# interactive end to end). Not in the ISO: the first click installs it
# (addition.sh --claude, as root) then opens it, in the same window.
set -u
exec python3 /opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Claude" -- bash -lc '
if ! command -v claude >/dev/null 2>&1; then
    ADD=/opt/GSM/osmo-operator/addition.sh
    if [ -x "$ADD" ]; then
        if [ "$(id -u)" -ne 0 ] && command -v pkexec >/dev/null 2>&1; then
            pkexec env DISPLAY="${DISPLAY:-}" XAUTHORITY="${XAUTHORITY:-}" "$ADD" --claude
        else
            "$ADD" --claude
        fi
    fi
fi
if command -v claude >/dev/null 2>&1; then exec claude; fi
echo "Claude non installe - lancez le supplement (addition.sh --claude)."
exit 1'

CLA
    chmod +x "$ROOTFS/usr/local/bin/osmo-claude-anim"
    # The file lives in the repository (data/desktop/); Icon= as an absolute path.
    install -m644 "$DIR/data/desktop/claude.desktop" "$ROOTFS/usr/share/applications/claude.desktop"
    sed -i "s|^Icon=.*|Icon=/usr/share/osmo-operator/icons/claude.svg|" \
        "$ROOTFS/usr/share/applications/claude.desktop"
```

```
    # Terminal=false: launch.sh opens ITS OWN terminal and asks for privileges
    # (pkexec). No copy on the desktop: shortcuts are in the menu and dock.
    echo -e "  ${GREEN}✓${NC} menu/dock : ${CYAN}telephone${NC} (lancer) · ${CYAN}livre${NC} (tutoriel) ·
     ↪  ${CYAN}supplements${NC} · ${CYAN}Claude${NC}"

    # The calamares package installs ITS OWN menu entry, which runs
    # /usr/bin/calamares directly and bypasses osmo-install (no medium put back
    # in place, no previous target unmounted). NoDisplay removes it from the
    # menus without touching the package.
    if [ -f "$ROOTFS/usr/share/applications/calamares.desktop" ]; then
        grep -q '^NoDisplay=' "$ROOTFS/usr/share/applications/calamares.desktop" \
            || echo 'NoDisplay=true' >> "$ROOTFS/usr/share/applications/calamares.desktop"
    fi

    echo -e "  ${GREEN}✓${NC} installeur ${CYAN}Calamares${NC} : /usr/local/bin/osmo-install (+ icone sur le
     ↪  bureau)"
elif [ "${ISO_DESKTOP:-0}" = "1" ]; then
    echo -e "  ${YELLOW}!${NC} $_CAL_SRC absent - pas d installeur dans cette image"
fi

# ── WITHOUT --desktop: THE SHORTCUTS AND THEIR ICONS ANYWAY ──────────────
# The 4G check of 88-lte-pmos.sh requires osmo-lte.desktop on EVERY operator
# image, including the base ISO (no GNOME), and the block above only runs with
# --desktop. Install here what the native install (install_modules/80-bureau.sh)
# installs anyway: the repository's .desktop files and their SVGs. A few KB,
# and a desktop installed afterwards (apt install ubuntu-desktop) finds its
# menu entries ready.
if [ "${ISO_DESKTOP:-0}" != "1" ]; then
    install -d "$ROOTFS/usr/share/applications" \
               "$ROOTFS/usr/share/icons/hicolor/scalable/apps" \
               "$ROOTFS/usr/share/osmo-operator/icons"
    for _ic in osmo-launch osmo-multi osmo-tutorial claude osmo-paint osmo-lte osmo-dino osmo-info; do
        [ -f "$DIR/data/$_ic.svg" ] || continue
        install -m644 "$DIR/data/$_ic.svg" "$ROOTFS/usr/share/icons/hicolor/scalable/apps/$_ic.svg"
        install -m644 "$DIR/data/$_ic.svg" "$ROOTFS/usr/share/osmo-operator/icons/$_ic.svg"
    done
    # EXCEPT osmo-multi.desktop: it only makes sense once the Supplements are
    # installed (docker, the operator image, osmo-multi.conf), and
    # addition.sh --multi installs it at that point. The dock shows it when it
    # exists (favorite-apps already names it, iso_modules/80-chroot.sh).
    for _d in "$DIR"/data/desktop/*.desktop; do
        [ -f "$_d" ] || continue
        [ "$(basename "$_d")" = osmo-multi.desktop ] && continue
        install -m644 "$_d" "$ROOTFS/usr/share/applications/$(basename "$_d")"
    done
    echo -e "  ${GREEN}✓${NC} raccourcis du depot (data/desktop/) + icones poses, sans bureau"
fi

# ── THE "UPDATE" GTK LAUNCHER -> the repository's update.sh ──────────────
# Independent of Calamares: a desktop icon that replays update.sh, the TRACKED
# file of the osmo-operator repository (/opt/GSM/osmo-operator/update.sh).
# update.sh is a TERMINAL animation: its first useful line is
# "[ -t 1 ] || exit 0", so without a tty (from a GTK icon) it exits at once.
# The launcher therefore opens it in a VTE terminal inside a bench GTK window
# (tools/osmo-gtk-run.py), which stays open at the end to read the result.
if [ "${ISO_DESKTOP:-0}" = "1" ]; then
    cat > "$ROOTFS/usr/local/bin/osmo-update-anim" <<'UPDGUI'
#!/bin/bash
# osmo-update-anim - replays the repository's update.sh in a bench GTK window.
# update.sh is a TERMINAL animation ([ -t 1 ] || exit 0): a VTE terminal in
# the window, which stays open on the last line ("SMS delivered") with the
# status.
set -u
SCRIPT=/opt/GSM/osmo-operator/update.sh
if [ ! -x "$SCRIPT" ]; then
    command -v zenity >/dev/null 2>&1 && \
```

```
        zenity --error --text="update.sh introuvable : $SCRIPT" 2>/dev/null
    exit 1
fi
# --progress: progress bar + step, the console (VTE) folded behind "console"
exec python3 /opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "osmo-operator - update" --
 ↪  "$SCRIPT" "$@"

UPDGUI
    chmod +x "$ROOTFS/usr/local/bin/osmo-update-anim"

    # The file lives in the repository (data/desktop/), not in a heredoc here:
    # the native install (install_modules/80-bureau.sh) and the .deb install
    # the SAME one.
    install -m644 "$DIR/data/desktop/osmo-update.desktop" "$ROOTFS/usr/share/applications/osmo-update.desktop"

    echo -e "  ${GREEN}✓${NC} lanceur GTK ${CYAN}update${NC} : /usr/local/bin/osmo-update-anim (+ icone sur le
     ↪  bureau)"
fi


# ── DESKTOP, CONTINUED: WALLPAPER OF THE DAY, SPECTRA, TERMINAL, DING ──────
# This whole block is OUTSIDE the chroot of 80-chroot.sh: heredocs and single
# quotes are free here.
if [ "${ISO_DESKTOP:-0}" = "1" ]; then
    # ── 1. Wallpaper of the day: Calvin & Hobbes strip inset ────────────────
    # tools/osmo-wallpaper.sh fetches the strip of the day (gocomics), composes
    # photo + LAB GSM map + strip (tools/wallpaper-render.py) and pushes the
    # result into the open sessions. At boot (40 s, network up), daily at 6:30,
    # and every 4 h as a catch-up (bench offline at boot).
    install -m755 "$DIR/tools/osmo-wallpaper.sh" "$ROOTFS/usr/local/sbin/osmo-wallpaper"
    # The image's /opt/GSM/osmo-operator tree comes from the GitHub clone made
    # in Docker: what the wallpaper and spectra use is also installed from THIS
    # repository, so the build does not depend on a push for those files.
    _rt="$ROOTFS/opt/GSM/osmo-operator"
    install -d "$_rt/tools" "$_rt/configs/wallpaper" "$_rt/configs/conky"
    # conky-osmo-status.sh is in the list: it reports whether the HLR runs.
    # Same rule for all: the local repository is authoritative.
    install -m755 "$DIR/tools/wallpaper-render.py" "$DIR/tools/osmo-fft-snap.py" "$DIR/tools/osmo-panel.py"
     ↪  "$DIR/tools/osmo-wallpaper.sh" "$DIR/tools/conky-osmo-status.sh" "$_rt/tools/"
    # The interactive desktop (live LAB GSM banner + launch bar) and the dino:
    # local files, installed from THIS repository like the five above.
    # osmo-extras-install.sh serves the native install (80).
    for _f in osmo-launcher.py osmo-topzone.py osmo-ts-probe.py osmo-dino.py osmo-extras-install.sh osmo-fond.py \
              osmo-pmos.sh osmo-pmos-qemu.sh osmo-pmos-setup.sh osmo-wireshark-root.sh \
              osmo-ofono-bridge.py osmo-phonesim-banc.py \
              osmo-ril-atmodem.py at-cmd.py; do
        [ -f "$DIR/tools/$_f" ] && install -m755 "$DIR/tools/$_f" "$_rt/tools/"
    done
    unset _f
    # tower.* and not tower.jpg: the photo travels in its original format
    # (full-definition PNG), without re-encoding.
    for _tw in "$DIR"/configs/wallpaper/tower.*; do
        [ -f "$_tw" ] && install -m644 "$_tw" "$_rt/configs/wallpaper/"
    done
    unset _tw
    install -m644 "$DIR/configs/conky/osmo-conky.conf" "$_rt/configs/conky/"
    # The conky plate (Cairo), loaded by osmo-conky.conf
    [ -f "$DIR/configs/conky/osmo-conky.lua" ] && install -m644 "$DIR/configs/conky/osmo-conky.lua"
     ↪  "$_rt/configs/conky/"
    # The live LAB GSM banner and the dino html (rendered by WebKit)
    for _h in labgsm.html dino.html; do
        [ -f "$DIR/configs/conky/$_h" ] && install -m644 "$DIR/configs/conky/$_h" "$_rt/configs/conky/"
    done
    unset _h
    install -m644 "$DIR/configs/gsm-lab-wallpaper.png" "$_rt/configs/gsm-lab-wallpaper.png"
    # 80-chroot.sh copied the wallpaper from the GitHub clone (before this
```

```
    # module): put THIS repository's one back, the render of tools/wallpaper-render.py.
    install -m644 "$DIR/configs/gsm-lab-wallpaper.png" "$ROOTFS/usr/share/backgrounds/gsm-lab-wallpaper.png"
    # The DEFAULT wallpaper is the raw illustration (80-chroot.sh); the composed
    # render above stays installed for osmo-wallpaper run by hand.
    install -m644 "$DIR/configs/gsm-lab-wallpaper.jpg" "$_rt/configs/gsm-lab-wallpaper.jpg"
    install -m644 "$DIR/configs/gsm-lab-wallpaper.jpg" "$ROOTFS/usr/share/backgrounds/gsm-lab-wallpaper.jpg"
    # The default GNOME wallpaper is configs/gsm-lab-fond.jpg (named by the
    # 80-chroot.sh schema); the illustration above remains for the inset, which
    # shows it whole in the middle. Both are rendered together by
    # tools/paysage-render.py (same scene: bare background, inset with the
    # pylons), then retouched by hand: the repository holds the chosen version.
    if [ -f "$DIR/configs/gsm-lab-fond.jpg" ]; then
        install -m644 "$DIR/configs/gsm-lab-fond.jpg" "$_rt/configs/gsm-lab-fond.jpg"
        install -m644 "$DIR/configs/gsm-lab-fond.jpg" "$ROOTFS/usr/share/backgrounds/gsm-lab-fond.jpg"
    else
        echo -e "  ${YELLOW}!${NC} configs/gsm-lab-fond.jpg absent - le fond GNOME retombe sur l illustration"
        install -m644 "$DIR/configs/gsm-lab-wallpaper.jpg" "$ROOTFS/usr/share/backgrounds/gsm-lab-fond.jpg"
    fi
    unset _rt
    cat > "$ROOTFS/etc/systemd/system/osmo-wallpaper.service" <<'EOF'
[Unit]
Description=Fond d ecran du banc (image du jour : Calvin, xkcd, APOD, Bing, turnoff.us)
# NO NETWORK WAIT: without network the script installs the frozen wallpaper
# and exits 0; with network the daily timer (osmo-wallpaper.timer) replaces it
# on its next pass.
After=network.target
ConditionPathExists=/opt/GSM/osmo-operator/tools/wallpaper-render.py

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/osmo-wallpaper
# A slow download must hold neither boot nor shutdown: the service is killed
# after 45 s, the frozen wallpaper stays.
TimeoutStartSec=45
TimeoutStopSec=10
EOF
    cat > "$ROOTFS/etc/systemd/system/osmo-wallpaper.timer" <<'EOF'
[Unit]
Description=Fond d ecran du banc : strip du jour (boot + quotidien)

[Timer]
OnBootSec=40s
OnCalendar=*-*-* 06:30:00
OnUnitActiveSec=4h
Persistent=true
Unit=osmo-wallpaper.service

[Install]
WantedBy=timers.target
EOF
    # Timer NOT enabled: it would replace the raw wallpaper with the composed
    # render 40 s after boot. The unit stays installed:
    #     systemctl enable --now osmo-wallpaper.timer   to get the wallpaper of the day back
    chroot "$ROOTFS" systemctl disable osmo-wallpaper.timer 2>/dev/null || true

    # ── 2. I/Q spectra in the bottom-right Conky ──────────────────────────
    # tools/osmo-fft-snap.py draws /run/osmo-fft/panel.png: the Calvin & Hobbes
    # frame of the wallpaper (cut from the PNG of the day) fading to "mobile
    # FFT + mobile.log" when /psd has a stream; osmo-panel.py (GTK desktop-type
    # window, clickable: Dashboard, tmux, operator arrows) displays it over the
    # frame, scaled to the screen. The service lives with the dashboard and
    # restarts without it: without a stream the image says "no stream" instead
    # of freezing.
    cat > "$ROOTFS/etc/systemd/system/osmo-fft-snap.service" <<'EOF'
[Unit]
Description=Encart vivant du bureau : strip du jour qui fond vers FFT du mobile + mobile.log
```

```
After=osmo-egprs-web.service osmo-wallpaper.service
ConditionPathExists=/opt/GSM/osmo-operator/tools/osmo-fft-snap.py

[Service]
ExecStart=/usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-fft-snap.py
RuntimeDirectory=osmo-fft
# 0775 AND the sudo group, NOT 0755 root:root: osmo-panel.py runs in the
# DESKTOP SESSION (root on the key, the installer-created account on disk) and
# WRITES /run/osmo-fft/operator when the operator is changed with the arrows.
# The installer's account is a sudoer by construction (users.conf), and so is
# root: the sudo group is exactly "who drives this bench". ExecStartPost rather
# than Group=: the unit must stay root to read the containers and /psd.
RuntimeDirectoryMode=0775
ExecStartPost=-/bin/chgrp sudo /run/osmo-fft
# systemd removes the RuntimeDirectory when the unit stops: keeping it avoids
# /run/osmo-fft/panel.png disappearing (and the inset blinking) across a
# restart. The file is rewritten every second.
RuntimeDirectoryPreserve=yes
Restart=always
RestartSec=5
Nice=10

[Install]
WantedBy=multi-user.target
EOF
    [ "$ISO_ROLE" = "interstp" ] || chroot "$ROOTFS" systemctl enable osmo-fft-snap 2>/dev/null || true

    # ── 3. The terminal: slightly transparent, amber on near-black ──────────
    # A gnome-terminal profile is a RELOCATABLE schema: impossible through
    # gschema.override. Hence a SYSTEM dconf database (/etc/dconf/db/local) read
    # by every session, root on the key as well as the Calamares user on disk,
    # which the user can still override ("user" database in front).
    # AMBER TERMINAL: amber text (#FFB000) on a warm near-black background,
    # blinking amber block cursor, light-yellow bold, palette in ambers/golds
    # (red and green stay distinct for errors and checkmarks), Ubuntu Mono 12,
    # 18 % transparency. Default on the ISO as on native (the same file is
    # loaded into the user database by update.sh / by hand: dconf load).
    install -d "$ROOTFS/etc/dconf/profile" "$ROOTFS/etc/dconf/db/local.d"
    printf 'user-db:user\nsystem-db:local\n' > "$ROOTFS/etc/dconf/profile/user"
    cat > "$ROOTFS/etc/dconf/db/local.d/10-osmo-terminal" <<'EOF'
[org/gnome/terminal/legacy/profiles:]
list=['b1dcc9dd-5262-4d8d-a863-c897e6d979b9']
default='b1dcc9dd-5262-4d8d-a863-c897e6d979b9'

[org/gnome/terminal/legacy/profiles:/:b1dcc9dd-5262-4d8d-a863-c897e6d979b9]
visible-name='osmo-operator'
use-theme-colors=false
use-theme-transparency=false
use-transparent-background=true
background-transparency-percent=18
background-color='#0B0A05'
foreground-color='#FFB000'
bold-color-same-as-fg=false
bold-color='#FFD75F'
cursor-colors-set=true
cursor-background-color='#FFB000'
cursor-foreground-color='#0B0A05'
cursor-shape='block'
cursor-blink-mode='on'
highlight-colors-set=true
highlight-background-color='#FFB000'
highlight-foreground-color='#0B0A05'
palette=['#1C1A10', '#FF5555', '#50FA7B', '#FFD75F', '#57A6FF', '#FF79C6', '#4FE0D4', '#FFD98A', '#5C5230',
 ↪  '#FF6E6E', '#69FF94', '#FFE28A', '#7CC0FF', '#FF92DF', '#7FF2E6', '#FFF3C8']
use-system-font=false
font='Ubuntu Mono 12'
```

```
scrollback-unlimited=false
scrollback-lines=50000
audible-bell=false
EOF
    # ── SCROLLBACK IS BOUNDED: IT LIVES IN /tmp ──────────────────────────
    # VTE keeps the scrollback in DELETED files under /tmp (held by
    # gnome-terminal-server): invisible to du, visible to df. An unlimited
    # scrollback on a tab following the bench logs can fill /tmp ("No space
    # left on device"). 50 000 lines = ~10 MB per tab, plenty to scroll back.
    chroot "$ROOTFS" dconf update 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} gnome-terminal : profil ${CYAN}osmo-operator${NC} (noir, 18 % transparent)
        ↪  via dconf systeme" \
        || echo -e "  ${YELLOW}!${NC} dconf update a echoue - profil terminal non compile"

    # ── 4. DING restarted after login ────────────────────────────────────
    # The "Desktop Icons NG" extension opens its full-screen window before GNOME
    # Shell has painted the wallpaper and keeps the greeter image on top of
    # ours. Killing it once the session is open is enough: the extension
    # restarts it within a second, cleanly.
    cat > "$ROOTFS/etc/xdg/autostart/osmo-ding-refresh.desktop" <<'EOF'
[Desktop Entry]
Type=Application
Name=osmo-operator : rafraichit les icones du bureau
Comment=Relance Desktop Icons NG une fois le fond d ecran peint
Exec=sh -c 'sleep 12; pkill -f "extensions/ding@rastersoft.com/app/ding.js" 2>/dev/null; exit 0'
Terminal=false
NoDisplay=true
OnlyShowIn=GNOME;
X-GNOME-Autostart-enabled=true
X-GNOME-Autostart-Delay=12
EOF
    chmod 644 "$ROOTFS/etc/xdg/autostart/osmo-ding-refresh.desktop"
    echo -e "  ${GREEN}✓${NC} bureau : fond brut (gsm-lab-wallpaper.jpg), spectres (osmo-fft-snap), DING relance
    ↪  a l ouverture"

    # ── 5. VLC FROM A ROOT SESSION ───────────────────────────────────────
    # The live session runs as root and VLC refuses to start as root ("VLC is
    # not supposed to be run as root"). The binary is untouched: the wrapper
    # /usr/local/bin/vlc (ahead of /usr/bin in PATH, targeted by the local
    # .desktop) re-runs VLC under the osmocom account when root - X access via
    # xhost (local only), sound via the bench's system PulseAudio daemon
    # (osmo-pulse, pulse-access group). A normal user goes straight through.
    if [ -x "$ROOTFS/usr/bin/vlc" ]; then
        chroot "$ROOTFS" getent passwd osmocom >/dev/null 2>&1 \
            || chroot "$ROOTFS" useradd -m -s /bin/bash osmocom 2>/dev/null || true
        chroot "$ROOTFS" getent group pulse-access >/dev/null 2>&1 \
            && chroot "$ROOTFS" usermod -aG pulse-access,audio,video osmocom 2>/dev/null || true
        cat > "$ROOTFS/usr/local/bin/vlc" <<'VLCW'
#!/bin/bash
# vlc - osmo-operator wrapper: VLC refuses root, the live session IS root.
# Root: re-run under osmocom (local X via xhost, sound via osmo-pulse).
VLC_USER="${OSMO_VLC_USER:-osmocom}"
export PULSE_SERVER="${PULSE_SERVER:-unix:/var/run/pulse/native}"
if [ "$(id -u)" -eq 0 ] && id "$VLC_USER" >/dev/null 2>&1; then
    export DISPLAY="${DISPLAY:-:0}"
    xhost "+SI:localuser:$VLC_USER" >/dev/null 2>&1 || true
    exec runuser -u "$VLC_USER" -- env DISPLAY="$DISPLAY" PULSE_SERVER="$PULSE_SERVER" \
        XDG_RUNTIME_DIR="/run/user/$(id -u "$VLC_USER")" HOME="$(getent passwd "$VLC_USER" | cut -d: -f6)" \
        /usr/bin/vlc "$@"
fi
exec /usr/bin/vlc "$@"
VLCW
        chmod 755 "$ROOTFS/usr/local/bin/vlc"
        # The package's .desktop calls /usr/bin/vlc by absolute path: local copy
        # (higher priority, not overwritten by apt) that goes through the wrapper.
        if [ -f "$ROOTFS/usr/share/applications/vlc.desktop" ]; then
```

```
            install -d "$ROOTFS/usr/local/share/applications"
            sed 's|^Exec=/usr/bin/vlc|Exec=/usr/local/bin/vlc|; s|^Exec=vlc |Exec=/usr/local/bin/vlc |' \
                "$ROOTFS/usr/share/applications/vlc.desktop" > "$ROOTFS/usr/local/share/applications/vlc.desktop"
        fi
        echo -e "  ${GREEN}✓${NC} vlc : wrapper ${CYAN}/usr/local/bin/vlc${NC} (root -> osmocom, .desktop
        ↪  redirige)"
    else
        echo -e "  ${YELLOW}!${NC} vlc absent du rootfs - pas de wrapper"
    fi
fi


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would stop everything silently. Always 0 here.
true
```

2.82 /opt/GSM/osmo-operator/iso_modules/86-finitions.sh

18785 octets, 307 lignes → 307 lignes

```
#!/bin/bash
# iso_modules/86-finitions.sh - live-boot toram fix, sssd, firefox, plymouth, grub, terminal banner, unmount
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# ── live-boot: tmpfs size computation of the RAM copy ──────────────────────
# The "in RAM" menu entry passes "toram=filesystem.squashfs": live-boot then
# copies ONLY the squashfs. lib/live/boot/9990-toram-todisk.sh sizes the tmpfs
# with
#     size=$( expr $(ls -la ${MODULETORAMFILE} | awk '{print $5}') / 1024 + 5000 )
# which prints "expr: syntax error" when the initramfs ls (busybox) does not
# return the size in the 5th field. Not fatal (the tmpfs falls back to its
# default size), but noisy. expr is replaced by shell arithmetic with the
# field VALIDATED before use and an explicit fallback; "ls -lan" avoids user
# name resolution, which can widen the column in an initramfs without a full
# /etc/passwd. The patch must precede update-initramfs, hence the explicit
# regeneration right after.
_LB_TORAM="$ROOTFS/lib/live/boot/9990-toram-todisk.sh"
if [ -f "$_LB_TORAM" ] && grep -q 'expr \$(ls -la \${MODULETORAMFILE}' "$_LB_TORAM"; then
    python3 - "$_LB_TORAM" <<'PYLB'
import sys
p = sys.argv[1]
s = open(p, encoding='utf-8').read()
old = "\t\t\tsize=$( expr $(ls -la ${MODULETORAMFILE} | awk '{print $5}') / 1024 + 5000 )\n"
new = ("\t\t\t_lbsz=$(ls -lan \"${MODULETORAMFILE}\" 2>/dev/null | awk '{print $5}')\n"
       "\t\t\tcase \"${_lbsz}\" in ''|*[!0-9]*) _lbsz=0 ;; esac\n"
       "\t\t\tsize=$(( _lbsz / 1024 + 5000 ))\n")
sys.exit(0 if (open(p, 'w', encoding='utf-8').write(s.replace(old, new, 1)) and old in s) else 0)
PYLB
    if grep -q '_lbsz' "$_LB_TORAM"; then
        _LB_K=$(ls "$ROOTFS"/boot/vmlinuz-* 2>/dev/null | sort -V | tail -1 | sed "s|.*/vmlinuz-||")
        if [ -n "$_LB_K" ]; then
            chroot "$ROOTFS" update-initramfs -u -k "$_LB_K" >/dev/null 2>&1 \
                || echo -e "  ${YELLOW}!${NC} update-initramfs a echoue apres la retouche live-boot"
        fi
        echo -e "  ${GREEN}✓${NC} live-boot : calcul de taille ${CYAN}toram${NC} fiabilise (plus d appel a expr)"
    else
        echo -e "  ${YELLOW}!${NC} live-boot : la ligne attendue n a pas ete trouvee - non modifie"
    fi
else
    echo -e "  ${GREEN}·${NC} live-boot : rien a corriger (absent, ou deja corrige)"
fi
```

```
# ── sssd: a service that fails at boot for nothing ──────────────────────
# ubuntu-desktop-minimal pulls sssd (gnome-online-accounts / realmd). On a
# machine in NO domain - a bench - sssd has no provider configured and exits
# with an error at every boot, leaving a permanently red entry in
# "systemctl --failed". MASKED rather than purged: purging would drag desktop
# packages along by reverse dependency. Reversible for whoever joins a domain:
#     systemctl unmask sssd && systemctl enable --now sssd
for _s in sssd sssd-autofs sssd-nss sssd-pac sssd-pam sssd-ssh sssd-sudo; do
    chroot "$ROOTFS" systemctl mask "$_s" >/dev/null 2>&1 || true
done
echo -e "  ${GREEN}✓${NC} sssd masque (aucun domaine sur un banc) - ${CYAN}systemctl unmask sssd${NC} pour le
 ↪  rendre"

# ── FIREFOX: THE BENCH BROWSER, AND NOTHING ELSE ────────────────────────
# Chromium requires either a change of account or --no-sandbox when the
# session runs as root ("Running as root without --no-sandbox is not
# supported"); Firefox starts as root without an intermediate launcher. The
# Chromium plumbing older images may have installed (launcher, profile
# runtime units, duplicate .desktop files) is removed: an ISO rebuilt over a
# cached rootfs would otherwise keep a "chromium" launcher leading nowhere and
# units failing at boot.
rm -f "$ROOTFS/usr/local/bin/chromium" \
      "$ROOTFS/usr/local/bin/chromium-browser" \
      "$ROOTFS/etc/profile.d/98-osmo-chromium.sh" \
      "$ROOTFS/usr/share/applications/chromium.desktop" \
      "$ROOTFS/usr/share/applications/chromium-browser.desktop" \
      "$ROOTFS/var/lib/snapd/desktop/applications/chromium_chromium.desktop" 2>/dev/null || true
for _u in osmo-chromium-runtime osmo-chromium-desktop; do
    chroot "$ROOTFS" systemctl disable "$_u" >/dev/null 2>&1 || true
    rm -f "$ROOTFS/etc/systemd/system/$_u.service" \
          "$ROOTFS/etc/systemd/system/multi-user.target.wants/$_u.service"
done
rm -rf "$ROOTFS/var/lib/osmo-chromium" 2>/dev/null || true

# ── THE DASHBOARD MICROPHONE: WHAT FIREFOX REQUIRES ─────────────────────
# Three conditions for the osmo-egprs-web microphone button to work:
#   1. A REACHABLE AUDIO SERVER: osmo-pulse.service + osmo-pulse-link.sh.
#      Without it getUserMedia returns "NotFoundError": zero devices.
#   2. A SECURE CONTEXT: navigator.mediaDevices does NOT EXIST over http:// on
#      an IP - only https:// or http://localhost. That is the HTTPS listener
#      of server.js, armed with the certificate installed by
#      install-web-service.sh.
#   3. THE PERMISSION, AND TRUST IN THE CERTIFICATE: this block.
# A policy rather than a click: the certificate is self-signed, and without a
# policy the operator must accept an exception before seeing the page, then
# answer a second prompt for the microphone; a refusal is remembered and the
# button stays dead silently.
# /etc/firefox/policies/policies.json is the system policy path on Linux; the
# Mozilla deb (80-chroot.sh) reads it directly. Its CONTENT cannot be written
# here: it names the origins (https://<ip>:80) and the certificate of THIS
# machine, which do not exist at build time. install-web-service.sh generates
# it, together with the certificate and from the same address list.
install -d "$ROOTFS/etc/firefox/policies"
echo -e "  ${GREEN}✓${NC} Chromium retire ; ${CYAN}Firefox${NC} seul navigateur (politique micro+certificat posee
 ↪  au boot)"


# ── PLYMOUTH: THE BENCH PRESENTS ITSELF FROM POWER-ON ───────────────────
# A home-made "script" theme: a pylon on the left, a mobile on the right and a
# GSM burst going back and forth between them (the 8 timeslots of the frame,
# the lit one moving). THE IMAGES ARE GENERATED HERE, not stored:
# tools/plymouth-render.py draws them (Pillow + DejaVu, the same dependencies
# as the wallpaper); thirty PNGs would be unreadable in a diff.
# The theme ALSO handles the LUKS passphrase prompt (SetDisplayPasswordFunction):
# on an encrypted install the screen would otherwise look frozen while the
```

```
# machine waits for a passphrase.
_PLY_SRC="$DIR/configs/plymouth/osmo-bts"
_PLY_DST="$ROOTFS/usr/share/plymouth/themes/osmo-bts"
# plymouthd lives in /usr/sbin on noble (not /usr/bin), and the theme is of
# type "script": without the script.so plugin plymouthd ignores the theme and
# falls back to text mode. Both are checked.
# No `$(ls ... | head -1)` here: build-iso.sh runs under `set -euo pipefail`
# and an empty glob (rootfs without plymouth - the interstp hub) would make ls
# exit 2 and abort the build silently. nullglob + array spawns no process and
# cannot fail; an empty array yields the empty string the -n test expects.
shopt -s nullglob; _PLY_SOS=("$ROOTFS"/usr/lib/*/plymouth/script.so); shopt -u nullglob
_PLY_SO="${_PLY_SOS[0]-}"
if [ -d "$_PLY_SRC" ] && [ -x "$ROOTFS/usr/sbin/plymouthd" ] && [ -n "$_PLY_SO" ]; then
    install -d "$_PLY_DST"
    install -m644 "$_PLY_SRC/osmo-bts.plymouth" "$_PLY_SRC/osmo-bts.script" "$_PLY_DST/"
    # --template: the repository's template; the renderer writes next to the
    # images the script with the geometry injected (it OVERWRITES the raw copy
    # installed just above, on purpose: it is the version aligned with the drawing).
    if python3 "$DIR/tools/plymouth-render.py" --out "$_PLY_DST" \
            --template "$_PLY_SRC/osmo-bts.script" >/dev/null 2>&1; then
        chmod 644 "$_PLY_DST"/*.png
        # ── THE THEME CHOICE GOES THROUGH update-alternatives ──────────────
        # `plymouth-set-default-theme` does NOT EXIST on noble. Ubuntu manages
        # the theme through alternatives: /usr/share/plymouth/themes/default.plymouth
        # links to /etc/alternatives/default.plymouth, where bgrt is registered
        # with priority 110. Register above it (200) and set the choice. A
        # direct `ln -sfn` on default.plymouth would be ERASED by the first
        # apt-get touching plymouth.
        chroot "$ROOTFS" update-alternatives --install \
            /usr/share/plymouth/themes/default.plymouth default.plymouth \
            /usr/share/plymouth/themes/osmo-bts/osmo-bts.plymouth 200 >/dev/null 2>&1 || true
        chroot "$ROOTFS" update-alternatives --set default.plymouth \
            /usr/share/plymouth/themes/osmo-bts/osmo-bts.plymouth >/dev/null 2>&1 \
            || ln -sfn /usr/share/plymouth/themes/osmo-bts/osmo-bts.plymouth \
                       "$ROOTFS/usr/share/plymouth/themes/default.plymouth"
        # The theme must GET INTO the initrd: without this file hook-functions
        # only embeds the original default theme and the screen stays Ubuntu's
        # despite the link above.
        install -d "$ROOTFS/etc/initramfs-tools/conf.d"
        echo "FRAMEBUFFER=y" > "$ROOTFS/etc/initramfs-tools/conf.d/splash"
        chroot "$ROOTFS" update-initramfs -u >/dev/null 2>&1 || true
        echo -e "  ${GREEN}✓${NC} plymouth : theme ${CYAN}osmo-bts${NC} (pylone <-> mobile, rafale GSM, saisie
         ↪  LUKS)"
    else
        echo -e "  ${YELLOW}!${NC} plymouth : rendu des images echoue (python3-pil ?) - theme Ubuntu conserve"
    fi
else
    echo -e "  ${YELLOW}!${NC} plymouth ou son greffon script absent du rootfs - pas de theme de demarrage"
fi

# ── PLYMOUTH: UBUNTU VORTEX BY DEFAULT ──────────────────────────────────────
# The boot screen is "Vortex Ubuntu" (Ubuntu logo and rotating colour bands),
# taken as is from configs/plymouth/vortex-ubuntu/ (see SOURCE). osmo-bts stays
# registered in the alternatives (priority 200): `update-alternatives --config
# default.plymouth` switches back. Vortex also handles the LUKS prompt
# (SetDisplayPasswordFunction).
_VTX_SRC="$DIR/configs/plymouth/vortex-ubuntu"
_VTX_DST="$ROOTFS/usr/share/plymouth/themes/vortex-ubuntu"
if [ -d "$_VTX_SRC" ] && [ -x "$ROOTFS/usr/sbin/plymouthd" ] && [ -n "$_PLY_SO" ]; then
    install -d "$_VTX_DST"
    install -m644 "$_VTX_SRC"/*.png "$_VTX_SRC"/vortex-ubuntu.plymouth "$_VTX_SRC"/vortex-ubuntu.script
     ↪  "$_VTX_DST/"
    chroot "$ROOTFS" update-alternatives --install \
        /usr/share/plymouth/themes/default.plymouth default.plymouth \
        /usr/share/plymouth/themes/vortex-ubuntu/vortex-ubuntu.plymouth 210 >/dev/null 2>&1 || true
    chroot "$ROOTFS" update-alternatives --set default.plymouth \
```

```
        /usr/share/plymouth/themes/vortex-ubuntu/vortex-ubuntu.plymouth >/dev/null 2>&1 \
        || ln -sfn /usr/share/plymouth/themes/vortex-ubuntu/vortex-ubuntu.plymouth \
                   "$ROOTFS/usr/share/plymouth/themes/default.plymouth"
    install -d "$ROOTFS/etc/initramfs-tools/conf.d"
    echo "FRAMEBUFFER=y" > "$ROOTFS/etc/initramfs-tools/conf.d/splash"
    chroot "$ROOTFS" update-initramfs -u >/dev/null 2>&1 || true
    echo -e "  ${GREEN}✓${NC} plymouth : theme ${CYAN}vortex-ubuntu${NC} par defaut (osmo-bts en alternative)"
fi

# ── GRUB: "osmo" THEME ON THE INSTALLED SYSTEM ──────────────────────────
# Generated by tools/grub-theme-render.py (background = the blue city,
# blurred). Installed in /boot/grub/themes: Calamares copies the rootfs, /boot
# included, and grub-mkconfig (run by the installer) reads /etc/default/grub.d.
# The menu is SHOWN for 5 s (Ubuntu hides it by default: an invisible theme).
if python3 "$DIR/tools/grub-theme-render.py" --out "$ROOTFS/boot/grub/themes/osmo" >/dev/null 2>&1; then
    install -d "$ROOTFS/etc/default/grub.d"
    cat > "$ROOTFS/etc/default/grub.d/osmo-theme.cfg" <<'GRUBTHEME'
# GRUB theme "osmo" (tools/grub-theme-render.py). Menu shown for 5 s.
GRUB_THEME=/boot/grub/themes/osmo/theme.txt
GRUB_GFXMODE=auto
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=5
GRUBTHEME
    echo -e "  ${GREEN}✓${NC} grub : theme ${CYAN}osmo${NC} (systeme installe)"
else
    echo -e "  ${YELLOW}!${NC} grub : rendu du theme echoue (python3-pil, grub-mkfont, fonts-ubuntu ?) - menu
     ↪  texte"
fi

# ── THE TERMINAL BANNER ─────────────────────────────────────────────────
# /usr/local/bin/osmo-banner is played from .bashrc (below). Three guards,
# none of them decorative:
#   $- == *i*   INTERACTIVE shell only: otherwise scp, rsync and git over ssh
#               read the banner as their protocol and fail on "protocol error".
#   [ -t 1 ]    a terminal, not a file: cursor sequences make a log unreadable.
#   OSMO_BANNER a shell within a shell (tmux, sudo -i, a make that opens a
#               bash) does not replay it: once per terminal is enough.
cat > "$ROOTFS/usr/local/bin/osmo-banner" <<'BANNER'
#!/bin/bash
# Terminal opening banner: screenfetch with the bench logo
# (configs/screenfetch/osmo-lab.sh: pylon, burst, mobile - not the Ubuntu
# logo), then one line.
set -u
[ -t 1 ] || exit 0

ART=/opt/GSM/osmo-operator/configs/screenfetch/osmo-lab.sh
# Info colours: user yellow and host green - ROOT in red and host in yellow.
# A filter on the output: screenfetch freezes its lines before reading the
# logo file, so they cannot be tinted from inside. Labels (OS:, Kernel:...)
# rotate over six colours.
_filtre() {
    command -v perl >/dev/null 2>&1 || { cat; return; }
    if [ "$(id -u)" = 0 ]; then _cu='1;31'; _ch='1;33'; else _cu='1;33'; _ch='1;32'; fi
    CU="$_cu" CH="$_ch" perl -pe '
        BEGIN { @p = ("1;32","1;36","1;35","1;34","1;33","1;31"); $i = 0; }
        if    (/^(.*\e\[0m  )(.*)$/s) { $a = $1; $b = $2; }
        elsif (/^( {30,})(.*)$/s)     { $a = $1; $b = $2; }
        else                          { next; }
        $b =~ s/\e\[[0-9;]*m//g;
        if ($b =~ s/^\s*(\S+?)@(\S+)/\e[$ENV{CU}m$1\e[0m@\e[$ENV{CH}m$2\e[0m/) { }
        elsif ($b =~ s/^\s*([A-Za-z][A-Za-z ]*:)/"\e[".$p[$i++ % 6]."m$1\e[0m"/e) { }
        $_ = $a . $b;'
}
if command -v screenfetch >/dev/null 2>&1; then
    if [ -f "$ART" ]; then screenfetch -E -a "$ART" | _filtre; else screenfetch -E | _filtre; fi
fi
```

```
# ── THE BOTTOM LINE ──────────────────────────────────────────────────
# One sentence, drawn at random, per terminal. Drawn with $RANDOM rather than
# `shuf`: one fork less at EVERY terminal opening.
_q=(
  "Um, Abis, A, Gb - quatre lettres, et six mois de votre vie."
  "L abonne est toujours joignable. C est le reseau qui ne repond pas."
  "TMSI : le seul pseudonyme qui change plus souvent que votre avis sur SS7."
  "Un timeslot ne ment jamais. Il se tait, ce qui est pire."
  "RSSI -95 dBm : ce n est pas un probleme d antenne, c est un mode de vie."
  "GSM a 1991. Il vous survivra, et il le sait."
  "Le paging a fonctionne. C est le telephone qui n ecoutait pas."
  "MCC 208 - la France, ou meme les operateurs mobiles ont un terroir."
  "Toute pile assez profonde finit par ressembler a un oignon. Et fait pleurer."
  "Je vis dans une fenetre de contexte. Vous, dans une fenetre de temps de garde."
  "Mon terroir a moi, c est l espace latent. Millesime variable, garde au frais."
  "Entre nous : vous predisez le canal, je predis le token suivant."
  "L attention, c est tout ce dont vous avez besoin. Et d un bon oscillateur."
  "Un LLM et un BTS ont ceci en commun : tous deux hallucinent hors couverture."
  "Ecrit par une machine, relu par une machine, debogue par vous. Bon courage."
)
printf '  \033[2;3m« %s »\033[0m\n\n' "${_q[$RANDOM % ${#_q[@]}]}"
BANNER
chmod +x "$ROOTFS/usr/local/bin/osmo-banner"
# ── screenfetch: WITHOUT ITS EASTER EGG ──────────────────────────────
# screenfetch draws a random number AT EACH LINE and, for five values out of a
# thousand, SHUFFLES the order of logo and info lines
# ("case $(awk ... rand ...) in 411|188|15|166|609)"). Those five values are
# neutralised; the rest of the script is intact.
[ -f "$ROOTFS/usr/bin/screenfetch" ] && \
    sed -i 's/^\([[:space:]]*\)411|188|15|166|609)/\1--jamais--)/' "$ROOTFS/usr/bin/screenfetch"

# Installed in the .bashrc of BOTH accounts, and in /etc/skel for those the
# installer will create. APPENDED, never rewriting the file: Ubuntu's .bashrc
# carries the prompt, colours and aliases.
for _h in "$ROOTFS/root" "$ROOTFS/home/osmocom" "$ROOTFS/etc/skel"; do
    install -d "$_h"
    [ -f "$_h/.bashrc" ] || cp "$ROOTFS/etc/skel/.bashrc" "$_h/.bashrc" 2>/dev/null || : > "$_h/.bashrc"
    grep -q 'osmo-banner' "$_h/.bashrc" 2>/dev/null || cat >> "$_h/.bashrc" <<'BASHRC'

# ── osmo-operator banner ─────────────────────────────────────────────
# Interactive AND terminal AND not already played: see /usr/local/bin/osmo-banner.
# Removing these three lines is enough to get rid of it.
if [[ $- == *i* ]] && [ -t 1 ] && [ -z "${OSMO_BANNER:-}" ] && [ -x /usr/local/bin/osmo-banner ]; then
    export OSMO_BANNER=1
    /usr/local/bin/osmo-banner
fi
# The bench prompt: user yellow, host green, path blue - ROOT in red, host
# in yellow.
if [[ $- == *i* ]]; then
    if [ "$EUID" = 0 ]; then
        PS1='\[\e[1;31m\]\u\[\e[0m\]@\[\e[1;33m\]\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '
    else
        PS1='\[\e[1;33m\]\u\[\e[0m\]@\[\e[1;32m\]\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '
    fi
fi
BASHRC
done
chroot "$ROOTFS" chown -R osmocom:osmocom /home/osmocom 2>/dev/null || true
echo -e "  ${GREEN}✓${NC} banniere de terminal : ${CYAN}screenfetch${NC} (logo pixel art du banc) + une phrase"

umount "$ROOTFS/var/cache/apt/archives" 2>/dev/null||true
umount "$ROOTFS"/{dev/pts,proc,sys,dev} 2>/dev/null||true

echo -e "  ${GREEN}✓${NC} config terminee"
```

```
# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would stop everything silently. Always 0 here.
true
```

2.83 /opt/GSM/osmo-operator/iso_modules/87-lite.sh

3019 octets, 52 lignes → 52 lignes

```
#!/bin/bash
# iso_modules/87-lite.sh - step 8c: lite pruning of the rootfs
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).

# ── Step 8c: LITE = the normal image MINUS the build workshops, on the rootfs ─
# Removes from the rootfs what Dockerfile.lite removes from the image, with the
# same rules (see its header for each exception). What remains in /opt/GSM:
#   qosmo/                the whole tree, build/ reduced to qemu-system-arm and
#                         qemu-bundle (QEMU relocates itself through it)
#   osmocom-bb/           osmocon (the loader) and trx_toolkit (fake_trx.py)
#   firmware/ qemu-install/ osmo-operator/ pont/ osmo-egprs-web/ qemu/
#   c54x_exe/ grgsm_exe/   the two layer-1 stacks outside QEMU (+ c54x_exe/rom)
#   *.bin *.py *.txt      DSP ROM, bridge scripts, calypso_dsp.txt
if [ "$ISO_LITE" = "1" ]; then
    echo -e "${GREEN}[8c/9] Elagage lite : les ateliers de compilation quittent le rootfs...${NC}"
    _G="$ROOTFS/opt/GSM"
    _before=$(du -sh "$_G" 2>/dev/null | cut -f1)
    for _d in libosmocore libosmo-netif libosmo-abis libosmo-sigtran libsmpp34 libgtpnl \
              libosmo-gprs osmo-hlr osmo-mgw osmo-ggsn osmo-sgsn osmo-msc osmo-bsc osmo-trx \
              osmo-bts osmo-pcu osmo-sip-connector osmo-gapk gsup-smsc-proto libosmo-dsp \
              gnuradio gr-osmosdr gr-gsm osmocom-bb-transceiver osmocom-bb-burst_ind; do
        rm -rf "$_G/$_d"
    done
    rm -rf "$_G"/sms-coding-utils* "$_G"/*.tar.bz2 "$_G"/*.tar.gz
    if [ -d "$_G/osmocom-bb" ]; then
        _k="$WORK/keep-bb"; rm -rf "$_k"
        mkdir -p "$_k/src/host/osmocon" "$_k/src/target"
        cp -a "$_G/osmocom-bb/src/host/osmocon/osmocon" "$_k/src/host/osmocon/" 2>/dev/null || true
        cp -a "$_G/osmocom-bb/src/target/trx_toolkit"   "$_k/src/target/"      2>/dev/null || true
        rm -rf "$_G/osmocom-bb"; mv "$_k" "$_G/osmocom-bb"
    fi
    for _q in qosmo; do
        [ -d "$_G/$_q/build" ] || continue
        _k="$WORK/keep-qbuild"; rm -rf "$_k"; mkdir -p "$_k"
        cp -a "$_G/$_q/build/qemu-system-arm" "$_k/" 2>/dev/null || true
        cp -a "$_G/$_q/build/qemu-bundle"     "$_k/" 2>/dev/null || true
        rm -rf "$_G/$_q/build"; mv "$_k" "$_G/$_q/build"
    done
    rm -rf "$ROOTFS/usr/local/include" "$ROOTFS/var/cache/osmo-debs" "$ROOTFS/root/.cache" \
           "$ROOTFS/usr/share/doc" "$ROOTFS/usr/share/man"
    find "$ROOTFS/usr/local/lib" -name '*.a' -delete 2>/dev/null || true
    find "$ROOTFS" -xdev -path "$_G" -prune -o -name '*.o' -exec rm -f {} + 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} /opt/GSM : ${_before:-?} -> $(du -sh "$_G" | cut -f1)"
fi


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. A module ending on a false test
# ("[ ... ] && { ...; }") would stop everything silently. Always 0 here.
true
```

2.84 /opt/GSM/osmo-operator/iso_modules/88-lte-pmos.sh

26282 octets, 405 lignes → 405 lignes

```
#!/bin/bash
# iso_modules/88-lte-pmos.sh - step 8d: 4G (srsRAN ZeroMQ + Open5GS) and the
# smartphone UI (postmarketOS, PPP kernel) in the rootfs.
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to be run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment variables:
#   OSMO_ISO_LTE_BUILD=0      do not compile srsRAN/Open5GS in the chroot when
#                             the .deb cache lacks them (ISO without 4G)
#   OSMO_ISO_PMOS_IMAGE=PATH  pmOS VM image (.img or .img.zst) to embed;
#                             'none' embeds nothing; default: look on the host
#   OSMO_PMOS_IMAGE_URL=URL   download the VM image (release asset) if none found
#   OSMO_PMOS_KERNEL_URL=URL  PPP kernel .deb to fetch when missing from the cache
#   OSMO_ISO_PMAPORTS=0       do not embed pmaports at the kernel commit
#   OSMO_ISO_LTE_REQUIRED=0   final check only reports, does not abort the build
#   OSMO_SRSRAN_ARCH          ISA baseline for srsRAN builds (default x86-64-v2)
#   OSMO_DEB_CACHE            host .deb cache (default /var/cache/osmo-debs)

# ── Step 8d: 4G and the phone ──────────────────────────────────────────
# Sources used by this module:
#   - Binaries: .deb files from /var/cache/osmo-debs, placed by
#     50-injection-image.sh: osmo-build-libzmq, osmo-build-srsran
#     (/opt/LTE/srsRAN_4G, /usr/local/bin), osmo-build-open5gs (/opt/LTE/open5gs,
#     install/ prefix), osmo-build-pmos-kernel (/opt/user_interface/kernel/pmos),
#     osmo-build-pmbootstrap (/opt/user_interface/pmos/pmbootstrap, patched).
#     They come from packaging/snapshot-lte-debs.sh or from the Dockerfile (the
#     pmOS kernel only from the former). If the cache lacks srsRAN/Open5GS they
#     are compiled here in the chroot by tools/osmo-lte-install.sh --build.
#   - apt runtime packages: PKGS in 80-chroot.sh (MongoDB repo added there).
#   - Configs and launchers: tools/osmo-lte-install.sh and
#     tools/osmo-pmos-install.sh, run inside the chroot from THIS repository
#     (not the image's GitHub clone, which may lag behind).
#   - The VM image (5 GB sparse, ~2 GB zstd) is optional, see
#     OSMO_ISO_PMOS_IMAGE; without it osmo-pmos-build runs pmbootstrap install.
#
# Not for the hub (interstp) nor --arm (srsRAN and Open5GS are not built for
# arm64, and pmbootstrap would run an emulated x86 VM).
if [ "$ISO_ROLE" = "interstp" ] || [ "${ISO_ARCH:-amd64}" = "arm64" ]; then return 0; fi

echo -e "${GREEN}[8d/9] La 4G (srsRAN ZeroMQ + Open5GS) et l UI smartphone (postmarketOS)...${NC}"

# ── /proc /sys /dev: bind-mounted for the duration of this module ──────
# Everything below runs inside the chroot (dpkg, apt, srsRAN/Open5GS builds,
# final checks) and needs them; 86-finitions.sh has already unmounted them.
# They are unmounted at the end of the module, since 90-iso.sh builds the
# squashfs. Idempotent: already-mounted points are skipped, and only what was
# mounted here is unmounted.
_iso88_mnt=()
_iso88_up() {   # _iso88_up <host source> <mount point, relative to the rootfs>
    local _t="$ROOTFS$2"
    [ -d "$_t" ] || install -d "$_t" 2>/dev/null || return 0
    mountpoint -q "$_t" 2>/dev/null && return 0
    mount --bind "$1" "$_t" 2>/dev/null && _iso88_mnt=("$_t" "${_iso88_mnt[@]}")
    return 0
}
_iso88_down() {  # reverse mount order (/dev/pts before /dev)
    local _m
    for _m in "${_iso88_mnt[@]}"; do umount -l "$_m" 2>/dev/null || true; done
    _iso88_mnt=()
}
_iso88_up /proc    /proc
```

```
_iso88_up /sys     /sys
_iso88_up /dev     /dev
_iso88_up /dev/pts /dev/pts

# Chroot DNS for the git clones of --build: /etc/resolv.conf links to
# /run/systemd/resolve/stub-resolv.conf, which is empty at build time. /run is
# a tmpfs at boot, so this file does not survive into the live system.
install -d "$ROOTFS/run/systemd/resolve"
[ -s "$ROOTFS/run/systemd/resolve/stub-resolv.conf" ] || {
    grep '^nameserver' /etc/resolv.conf 2>/dev/null | grep -v '^nameserver 127\.' \
      || printf 'nameserver 1.1.1.1\nnameserver 8.8.8.8\n'
} > "$ROOTFS/run/systemd/resolve/stub-resolv.conf"

_rt="$ROOTFS/opt/GSM/osmo-operator"
install -d "$_rt/tools" "$_rt/configs/srsran" "$_rt/configs/open5gs" "$_rt/configs/pmos" "$_rt/patches"
 ↪  "$_rt/packaging"
# The local repository is authoritative (same rule as 85-installeur-bureau.sh).
for _f in osmo-lte.sh osmo-epc.sh osmo-lte-install.sh osmo-pmos.sh osmo-pmos-qemu.sh osmo-pmos-setup.sh \
          osmo-pmos-build.sh osmo-pmos-install.sh osmo-phonesim-banc.py at-cmd.py; do
    [ -f "$DIR/tools/$_f" ] && install -m755 "$DIR/tools/$_f" "$_rt/tools/"
done
install -m644 "$DIR"/configs/srsran/* "$_rt/configs/srsran/"
install -m644 "$DIR"/configs/open5gs/*.yaml "$DIR"/configs/open5gs/*.json "$_rt/configs/open5gs/" 2>/dev/null ||
 ↪  true
[ -d "$DIR/configs/open5gs/dump" ] && cp -a "$DIR/configs/open5gs/dump" "$_rt/configs/open5gs/"
# freeDiameter .conf files come from the repo (the Open5GS .deb has none).
[ -d "$DIR/configs/open5gs/freeDiameter" ] && cp -a "$DIR/configs/open5gs/freeDiameter" "$_rt/configs/open5gs/"
install -m644 "$DIR"/configs/pmos/* "$_rt/configs/pmos/"
for _f in pmbootstrap-osmo-bench-qemu.patch pmaports-linux-postmarketos-stable-ppp.patch; do
    [ -f "$DIR/patches/$_f" ] && install -m644 "$DIR/patches/$_f" "$_rt/patches/"
done
for _f in build-pmos-kernel-deb.sh snapshot-lte-debs.sh osmo-deb.sh; do
    [ -f "$DIR/packaging/$_f" ] && install -m755 "$DIR/packaging/$_f" "$_rt/packaging/"
done
unset _f

# PPP pmOS kernel missing from the cache: fetch it from GitHub (Git LFS,
# bbaranoff/pmos_ppp_kernel) into the host cache and the rootfs.
if [ ! -e "$ROOTFS/opt/user_interface/kernel/pmos/APKINDEX.tar.gz" ]; then
    _kurl="${OSMO_PMOS_KERNEL_URL:-https://media.githubusercontent.com/media/bbaranoff/pmos_ppp_kernel/main/osmo-
     ↪  build-pmos-kernel_7.2.3.0+ppp~noble_amd64.deb}"
    _kdeb="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}/$(basename "$_kurl")"
    if [ ! -s "$_kdeb" ]; then
        echo -e "  ${CYAN}·${NC} noyau pmOS PPP : telechargement depuis GitHub ($_kurl)"
        install -d "$(dirname "$_kdeb")"
        curl -fsSL --retry 3 -o "$_kdeb.part" "$_kurl" && dpkg-deb --info "$_kdeb.part" >/dev/null 2>&1 \
            && mv -f "$_kdeb.part" "$_kdeb" || { rm -f "$_kdeb.part"; echo -e "  ${YELLOW}!${NC} noyau pmOS :
             ↪  telechargement impossible"; }
    fi
    if [ -s "$_kdeb" ]; then
        install -d "$ROOTFS/var/cache/osmo-debs"; cp -f "$_kdeb" "$ROOTFS/var/cache/osmo-debs/"
        chroot "$ROOTFS" dpkg -i --force-overwrite "/var/cache/osmo-debs/$(basename "$_kdeb")" >/dev/null 2>&1 \
            && echo -e "  ${GREEN}✓${NC} noyau pmOS PPP pose depuis $(basename "$_kdeb")"
        [ "${ISO_EMBED_DEBS:-0}" = "1" ] || rm -f "$ROOTFS/var/cache/osmo-debs/$(basename "$_kdeb")"
    fi
    unset _kurl _kdeb
fi

# Report what the .deb files installed; warn but do not stop.
for _c in "srsenb:/usr/local/bin/srsenb" "Open5GS:/opt/LTE/open5gs/install/bin/open5gs-mmed" \
          "noyau pmOS PPP:/opt/user_interface/kernel/pmos/APKINDEX.tar.gz" \
          "pmbootstrap:/opt/user_interface/pmos/pmbootstrap/pmbootstrap.py"; do
    if [ -e "$ROOTFS/${_c#*:}" ]; then echo -e "  ${GREEN}✓${NC} ${_c%%:*} (.deb du build)"
    else echo -e "  ${YELLOW}!${NC} ${_c%%:*} absent du rootfs - pas de .deb dans /var/cache/osmo-debs
     ↪  (packaging/snapshot-lte-debs.sh sur la machine de reference, ou le Dockerfile)"; fi
done
```

```
unset _c

# The MongoDB repo is already configured by 80-chroot.sh: --deps only adds
# missing packages (and enables mongod), then the cached .deb files are installed.
chroot "$ROOTFS" env DEBIAN_FRONTEND=noninteractive OSMO_REPO=/opt/GSM/osmo-operator \
    bash /opt/GSM/osmo-operator/tools/osmo-lte-install.sh --deps --debs 2>&1 | sed 's/^/  /'

# ── No .deb: compile in the chroot ──────────────────────────────────
# The chroot has the -dev packages (PKGS in 80-chroot.sh), /proc /sys /dev and
# DNS, so the same --build used by the Dockerfile runs here. With OSMO_DEB=1 the
# build produces .deb files that are copied back to the host cache.
# OSMO_ISO_LTE_BUILD=0 disables this (ISO without 4G).
#
# A cached srsRAN .deb built with -march=native contains AVX instructions and
# would die with "Illegal instruction" on other CPUs. It is rejected here and
# rebuilt with a portable ISA baseline (see SRS_ARCH in tools/osmo-lte-install.sh).
_iso_srs_avx() {   # 0 = this binary will not run on every CPU
    [ -x "$ROOTFS/usr/local/bin/srsenb" ] || return 1
    command -v objdump >/dev/null 2>&1 || return 1
    objdump -d "$ROOTFS/usr/local/bin/srsenb" 2>/dev/null | grep -qE '%[yz]mm[0-9]'
}
if _iso_srs_avx; then
    echo -e "  ${YELLOW}!${NC} srsenb du cache .deb taille pour le CPU du build (AVX) : il ferait « Illegal
     ↪  instruction » ailleurs"
    rm -f "$ROOTFS"/usr/local/bin/srsenb "$ROOTFS"/usr/local/bin/srsue "$ROOTFS"/usr/local/bin/srsepc
    # Move the offending .deb aside in the host cache (renamed, not deleted; the
    # recovery glob below only takes *.deb) so the next ISO does not reuse it.
    for _sd in "${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"/osmo-build-srsran_*.deb; do
        [ -f "$_sd" ] || continue
        mv -f "$_sd" "$_sd.avx-natif" && echo -e "    ${CYAN}·${NC} $(basename "$_sd") ecarte -> $(basename
         ↪  "$_sd").avx-natif"
    done
    unset _sd
fi
unset -f _iso_srs_avx
if [ ! -x "$ROOTFS/usr/local/bin/srsenb" ] || [ ! -x "$ROOTFS/opt/LTE/open5gs/install/bin/open5gs-mmed" ]; then
    if [ "${OSMO_ISO_LTE_BUILD:-1}" = "1" ]; then
        echo -e "  ${YELLOW}!${NC} 4G absente du cache .deb : compilation dans le chroot (long)"
        install -d "$ROOTFS/var/cache/osmo-debs"
        [ -f "$DIR/packaging/osmo-deb.sh" ] && install -m755 "$DIR/packaging/osmo-deb.sh"
         ↪  "$ROOTFS/usr/local/sbin/osmo-deb"
        # OSMO_SRSRAN_ARCH: ISA baseline of the ISO binaries. The script default
        # (x86-64-v2) is right for an ISO; passed explicitly so a build can override it.
        chroot "$ROOTFS" env DEBIAN_FRONTEND=noninteractive OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator \
            OSMO_SRSRAN_ARCH="${OSMO_SRSRAN_ARCH:-x86-64-v2}" \
            bash /opt/GSM/osmo-operator/tools/osmo-lte-install.sh --build 2>&1 | sed 's/^/  /' \
            || echo -e "  ${YELLOW}!${NC} compilation 4G echouee dans le chroot - ISO sans srsRAN/Open5GS"
        # Copy freshly built .deb files back to the host cache.
        _dc="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
        install -d "$_dc"
        for _d in "$ROOTFS"/var/cache/osmo-debs/osmo-build-*.deb; do
            [ -f "$_d" ] || continue
            [ -f "$_dc/$(basename "$_d")" ] || { cp -f "$_d" "$_dc/" && echo -e "  ${GREEN}✓${NC} $(basename
             ↪  "$_d") remis dans $_dc"; }
        done
        [ "${ISO_EMBED_DEBS:-0}" = "1" ] || rm -f "$ROOTFS"/var/cache/osmo-debs/osmo-build-*.deb
        unset _d _dc
    else
        echo -e "  ${YELLOW}!${NC} 4G absente et OSMO_ISO_LTE_BUILD=0 : ISO sans srsRAN/Open5GS"
    fi
fi

# Open5GS configs are only installed if the prefix exists: after the build.
chroot "$ROOTFS" env OSMO_REPO=/opt/GSM/osmo-operator \
    bash /opt/GSM/osmo-operator/tools/osmo-lte-install.sh --configs --launchers 2>&1 | sed 's/^/  /'
chroot "$ROOTFS" env OSMO_REPO=/opt/GSM/osmo-operator \
```

```
    bash /opt/GSM/osmo-operator/tools/osmo-pmos-install.sh 2>&1 | sed 's/^/  /'

# Default VM image: if OSMO_ISO_PMOS_IMAGE is unset, use the host's image
# (where osmo-pmos-build looks for it). A ready .zst is preferred, otherwise
# the raw pmbootstrap image is compressed. OSMO_ISO_PMOS_IMAGE=none embeds nothing.
if [ -z "${OSMO_ISO_PMOS_IMAGE:-}" ]; then
    for _cand in /opt/user_interface/pmos/image/qemu-amd64.img.zst \
                 "${OSMO_DEB_CACHE:-/var/cache/osmo-debs}/qemu-amd64.img.zst" \
                 "${OSMO_PMB_WORK:-/root/test}"/chroot_native/home/pmos/rootfs/qemu-amd64.img \
                 /home/*/test/chroot_native/home/pmos/rootfs/qemu-amd64.img; do
        [ -f "$_cand" ] || continue
        OSMO_ISO_PMOS_IMAGE="$_cand"
        echo -e "  ${CYAN}·${NC} image de la VM de reference trouvee sur l hote : $_cand"
        break
    done
    unset _cand
fi

# No image on the host (e.g. CI runner): download it from
# OSMO_PMOS_IMAGE_URL if set. The image is a release asset (too big for the
# Git LFS quota). Publish once from a reference machine:
#
#   gh release create pmos-image --repo bbaranoff/osmo-operator \
#       --title "postmarketOS - VM de reference (noyau PPP)" --notes "..." \
#       /opt/user_interface/pmos/image/qemu-amd64.img.zst
#
# Empty (default) = no download.
if [ -z "${OSMO_ISO_PMOS_IMAGE:-}" ] && [ -n "${OSMO_PMOS_IMAGE_URL:-}" ]; then
    _icache="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}/qemu-amd64.img.zst"
    echo -e "  ${CYAN}·${NC} image de la VM : telechargement depuis $OSMO_PMOS_IMAGE_URL"
    install -d "$(dirname "$_icache")"
    # zstd -t catches a 404 page or truncated download here rather than in the ISO.
    if curl -fsSL --retry 3 --retry-delay 5 -o "$_icache.part" "$OSMO_PMOS_IMAGE_URL" \
       && zstd -t "$_icache.part" >/dev/null 2>&1; then
        mv -f "$_icache.part" "$_icache"
        OSMO_ISO_PMOS_IMAGE="$_icache"
        echo -e "  ${GREEN}✓${NC} image recuperee ($(du -h "$_icache" | cut -f1)) - gardee dans $(dirname
        ↪  "$_icache") pour les ISO suivantes"
    else
        rm -f "$_icache.part"
        echo -e "  ${YELLOW}!${NC} image de la VM : telechargement impossible - ISO sans telephone pret
        ↪  (osmo-pmos-build le fera sur la machine)"
    fi
    unset _icache
fi
[ "${OSMO_ISO_PMOS_IMAGE:-}" = "none" ] && OSMO_ISO_PMOS_IMAGE=""
# Embed the VM image: the given one, or the host's (see above).
if [ -n "${OSMO_ISO_PMOS_IMAGE:-}" ]; then
    if [ -f "$OSMO_ISO_PMOS_IMAGE" ]; then
        install -d "$ROOTFS/opt/user_interface/pmos/image"
        case "$OSMO_ISO_PMOS_IMAGE" in
            *.zst) cp -f "$OSMO_ISO_PMOS_IMAGE" "$ROOTFS/opt/user_interface/pmos/image/qemu-amd64.img.zst" ;;
            *) zstd -T0 -3 --sparse -f -o "$ROOTFS/opt/user_interface/pmos/image/qemu-amd64.img.zst"
            ↪  "$OSMO_ISO_PMOS_IMAGE" ;;
        esac && echo -e "  ${GREEN}✓${NC} image de la VM embarquee ($(du -h
        ↪  "$ROOTFS/opt/user_interface/pmos/image/qemu-amd64.img.zst" | cut -f1))"
    else
        echo -e "  ${YELLOW}!${NC} OSMO_ISO_PMOS_IMAGE=$OSMO_ISO_PMOS_IMAGE introuvable - image non embarquee"
    fi
else
    echo -e "  ${CYAN}·${NC} image de la VM non embarquee (OSMO_ISO_PMOS_IMAGE=... pour l ajouter) :
    ↪  osmo-pmos-build la fera"
fi

# ── pmaports in the ISO, at the kernel commit ──────────────────────
# Fetch only the commit of the PPP kernel (kernel/pmos/pmaports.commit,
```

```
# --depth 1) with the origin/main ref pmbootstrap needs to read channels.cfg,
# into /opt/user_interface/pmos/pmaports; osmo-pmos-build copies it instead of
# cloning the full pmaports. Kept in the host cache (OSMO_DEB_CACHE).
# OSMO_ISO_PMAPORTS=0 disables it.
_pc="$(cat "$ROOTFS/opt/user_interface/kernel/pmos/pmaports.commit" 2>/dev/null || true)"
if [ "${OSMO_ISO_PMAPORTS:-1}" = "1" ] && [ -n "$_pc" ] && [ "$_pc" != inconnu ]; then
    _pcache="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}/pmaports-$_pc"
    if ! git -C "$_pcache" rev-parse --verify HEAD >/dev/null 2>&1; then
        echo -e "  ${CYAN}·${NC} pmaports : fetch du commit $_pc (GitLab, --depth 1)..."
        rm -rf "$_pcache"
        ( git init -q "$_pcache" && cd "$_pcache" \
          && git remote add origin https://gitlab.postmarketos.org/postmarketOS/pmaports.git \
          && git fetch -q --depth 1 origin "$_pc" && git checkout -q -B main FETCH_HEAD \
          && git update-ref refs/remotes/origin/main FETCH_HEAD ) \
          || { rm -rf "$_pcache"; echo -e "  ${YELLOW}!${NC} pmaports : fetch impossible (reseau ?) - le premier
           ↪  clic ira le chercher"; }
    fi
    if git -C "$_pcache" rev-parse --verify HEAD >/dev/null 2>&1; then
        rm -rf "$ROOTFS/opt/user_interface/pmos/pmaports"
        cp -a "$_pcache" "$ROOTFS/opt/user_interface/pmos/pmaports"
        echo -e "  ${GREEN}✓${NC} pmaports au commit ${_pc:0:10} dans /opt/user_interface/pmos/pmaports ($(du -sh
         ↪  "$_pcache" | cut -f1))"
    fi
    unset _pcache
fi
unset _pc

# The session account runs pmbootstrap: /opt/user_interface must be readable,
# and the config template goes to /etc/skel for accounts created by Calamares
# (shellprocess-osmo.conf fills in the paths of their home).
chroot "$ROOTFS" chmod -R a+rX /opt/user_interface 2>/dev/null || true
install -d "$ROOTFS/etc/skel/.config"
install -m644 "$DIR/configs/pmos/pmbootstrap_v3.cfg" "$ROOTFS/etc/skel/.config/pmbootstrap_v3.cfg.osmo"
# Existing accounts too: root and osmocom are created by 84-comptes.sh before
# this module, so /etc/skel does not apply. Same substitution as
# shellprocess-osmo.conf (work dir ~/.local/var/pmbootstrap).
for _h in /root /home/osmocom; do
    [ -d "$ROOTFS$_h" ] || continue
    [ -s "$ROOTFS$_h/.config/pmbootstrap_v3.cfg" ] && continue
    install -d "$ROOTFS$_h/.config"
    sed "s#@WORK@#$_h/.local/var/pmbootstrap#g; s#@APORTS@#$_h/.local/var/pmbootstrap/cache_git/pmaports#g" \
        "$DIR/configs/pmos/pmbootstrap_v3.cfg" > "$ROOTFS$_h/.config/pmbootstrap_v3.cfg"
done
chroot "$ROOTFS" chown -R osmocom:osmocom /home/osmocom/.config 2>/dev/null || true
unset _h
echo -e "  ${GREEN}✓${NC} pmbootstrap_v3.cfg : /root, /home/osmocom, /etc/skel"

# Lite: build objects in /opt/LTE are not needed at runtime.
if [ "$ISO_LITE" = "1" ]; then
    _before=$(du -sh "$ROOTFS/opt/LTE" 2>/dev/null | cut -f1)
    find "$ROOTFS/opt/LTE/srsRAN_4G/build" "$ROOTFS/opt/LTE/open5gs/build" -name '*.o' -delete 2>/dev/null || true
    rm -rf "$ROOTFS/opt/LTE/libzmq" "$ROOTFS/opt/LTE/czmq" 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} lite : /opt/LTE ${_before:-?} -> $(du -sh "$ROOTFS/opt/LTE" 2>/dev/null | cut -f1)"
fi
unset _rt _before

# ── Final check: the ISO must be ready to use, 2G + 4G + phone ──────────
# Verify in the rootfs every link needed to launch the GSM bench, the 4G and
# the pmOS phone, and abort the build if one is missing.
# OSMO_ISO_LTE_REQUIRED=0 only reports (ISO without 4G accepted).
echo -e "  ${CYAN}·${NC} controle final : 2G + 4G + telephone dans le rootfs"
_ko=0
_chk() {  # _chk <label> <chroot test command...>
    local l="$1"; shift
    if chroot "$ROOTFS" "$@" >/dev/null 2>&1; then echo -e "      ${GREEN}✓${NC} $l"
    else echo -e "      ${RED}✗${NC} $l"; _ko=$((_ko + 1)); fi
```

```
}
# 2G: the bench, its unit, and osmo-msc SGs (combined attach, CSFB)
_chk "2G  launch.sh (icone « Lancer le banc GSM »)"       test -x /opt/GSM/osmo-operator/launch.sh
_chk "2G  osmo-banc.service"                               test -s /etc/systemd/system/osmo-banc.service
_chk "2G  osmo-msc.cfg : section sgs (CSFB)"               grep -q '^sgs' /etc/osmocom/osmo-msc.cfg
# 4G: binaries, configs (incl. MME SGs), launchers, unit, MongoDB
_chk "4G  srsenb / srsue"                                  bash -c 'test -x /usr/local/bin/srsenb && test -x
 ↪  /usr/local/bin/srsue'
# ── Are all their libraries present? ──────────────────────────────
# srsRAN is built with ENABLE_GUI=ON and links to libsrsgui (/usr/local/lib).
_chk "4G  srsenb / srsue : bibliotheques toutes resolues" \
    bash -c 'for b in srsenb srsue; do ldd /usr/local/bin/$b 2>/dev/null | grep -q "not found" && exit 1; done;
 ↪  exit 0'
# ── Will they run on the operator's machine? ──────────────────────
# An AVX (%ymm) or AVX512 (%zmm) instruction in srsenb/srsue ties the ISO to
# the build CPU: treated as an error. Checked from the host (objdump on the
# rootfs; the ISO has no binutils). Skipped with a notice if objdump is missing.
if command -v objdump >/dev/null 2>&1; then
    if objdump -d "$ROOTFS/usr/local/bin/srsenb" 2>/dev/null | grep -qE '%[yz]mm[0-9]' \
       || objdump -d "$ROOTFS/usr/local/bin/srsue" 2>/dev/null | grep -qE '%[yz]mm[0-9]'; then
        echo -e "      ${RED}✗${NC} 4G  srsenb / srsue portables (pas d AVX du CPU du build)"
        echo -e "          recompiler : OSMO_SRSRAN_ARCH=x86-64-v2, cf. tools/osmo-lte-install.sh"
        _ko=$((_ko + 1))
    else
        echo -e "      ${GREEN}✓${NC} 4G  srsenb / srsue portables (pas d AVX du CPU du build)"
    fi
else
    echo -e "      ${CYAN}·${NC} 4G  objdump absent de l hote : portabilite ISA de srsenb non verifiee"
fi
# Real-time traces (srsGUI) are optional: only reported.
if chroot "$ROOTFS" bash -c 'ldd /usr/local/bin/srsenb 2>/dev/null | grep -q libsrsgui' 2>/dev/null; then
    echo -e "      ${GREEN}✓${NC} 4G  srsGUI : traces temps reel armees (srsenb lie a libsrsgui)"
else
    echo -e "      ${CYAN}·${NC} 4G  srsGUI absent : srsenb sans traces temps reel (« osmo-lte-install --build »)"
fi
_chk "4G  open5gs-mmed / open5gs-hssd"                     bash -c 'test -x
 ↪  /opt/LTE/open5gs/install/bin/open5gs-mmed && test -x /opt/LTE/open5gs/install/bin/open5gs-hssd'
_chk "4G  mme.yaml du depot avec sgsap (CSFB)"             grep -q '^  sgsap:'
 ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml
_chk "4G  freeDiameter mme/hss/pcrf/smf.conf (S6a, Gx)"    bash -c 'for n in mme hss pcrf smf; do test -s
 ↪  /opt/LTE/open5gs/install/etc/freeDiameter/$n.conf || exit 1; done'
_chk "4G  TLS freeDiameter (ca + mme/hss/pcrf/smf)"        bash -c 'for n in ca mme hss pcrf smf; do test -s
 ↪  /opt/LTE/open5gs/install/etc/open5gs/tls/$n.crt || exit 1; done'
# Does the MME start? 3 s in the foreground in the chroot catches missing
# configs or libraries. A port already in use on the host (its own 4G running)
# is not an ISO fault: reported without failing.
_o="$(chroot "$ROOTFS" timeout 3 /opt/LTE/open5gs/install/bin/open5gs-mmed -c
 ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml 2>&1 || true)"
rm -f "$ROOTFS/opt/LTE/open5gs/install/var/log/open5gs/mme.log"
if echo "$_o" | grep -aq 'Address already in use'; then
    echo -e "      ${CYAN}·${NC} 4G  open5gs-mmed : port deja pris sur l hote (sa 4G tourne ?) - demarrage non
     ↪  teste"
elif echo "$_o" | grep -aqE 'FATAL|error while loading shared'; then
    echo -e "      ${RED}✗${NC} 4G  open5gs-mmed demarre (3 s au premier plan)"
    echo "$_o" | grep -aE 'FATAL|error while loading' | grep -v backtrace | head -2 | sed 's/^/          /'
    _ko=$((_ko + 1))
else
    echo -e "      ${GREEN}✓${NC} 4G  open5gs-mmed demarre (3 s au premier plan)"
fi
unset _o
_chk "4G  configs srsRAN (/root/.config/srsran)"           bash -c 'test -s /root/.config/srsran/enb.conf && test
 ↪  -s /root/.config/srsran/ue.conf && test -s /root/.config/srsran/user_db.csv'
_chk "4G  osmo-lte / osmo-epc"                             bash -c 'test -x /usr/local/bin/osmo-lte && test -x
 ↪  /usr/local/bin/osmo-epc'
# Phone data path: docker sets FORWARD to DROP; the DOCKER-USER rules from
# osmo-epc (setup_forward), osmo-pmos-qemu (pmos_forward) and the bench modem
```

```
# (PPP session) are required for traffic to leave the host.
_chk "4G  FORWARD ogstun : osmo-epc, osmo-pmos-qemu, phonesim" bash -c 'grep -q "^setup_forward()"
 ↪  /opt/GSM/osmo-operator/tools/osmo-epc.sh && grep -q "^pmos_forward()"
 ↪  /opt/GSM/osmo-operator/tools/osmo-pmos-qemu.sh && grep -q "DOCKER-USER"
 ↪  /opt/GSM/osmo-operator/tools/osmo-phonesim-banc.py'
_chk "4G  osmo-lte.service + osmo-lte.desktop"             bash -c 'test -s /etc/systemd/system/osmo-lte.service
 ↪  && test -s /usr/share/applications/osmo-lte.desktop'
_chk "4G  mongod + mongosh + mongorestore (abonnes HSS)"   bash -c 'command -v mongod && command -v mongosh &&
 ↪  command -v mongorestore'
_chk "4G  abonnes du depot (dump mongo + subscribers.json)" bash -c 'test -s
 ↪  /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/subscribers.bson && test -s
 ↪  /opt/GSM/osmo-operator/configs/open5gs/subscribers.json'
# Phone: patched pmbootstrap, PPP kernel, launcher and icon
_chk "pmOS pmbootstrap (/opt/user_interface/pmos)"         test -s
 ↪  /opt/user_interface/pmos/pmbootstrap/pmbootstrap.py
_chk "pmOS noyau PPP (APKINDEX)"                          test -s /opt/user_interface/kernel/pmos/APKINDEX.tar.gz
_chk "pmOS osmo-pmos + osmo-pmos.desktop"                  bash -c 'test -x /usr/local/bin/osmo-pmos && test -s
 ↪  /usr/share/applications/osmo-pmos.desktop'
_chk "pmOS pmbootstrap_v3.cfg de root (la session de l ISO)" test -s /root/.config/pmbootstrap_v3.cfg
if [ -d "$ROOTFS/home/osmocom" ]; then
    _chk "pmOS pmbootstrap_v3.cfg d osmocom"               bash -c 'test -s
     ↪  /home/osmocom/.config/pmbootstrap_v3.cfg && ! grep -q @WORK@ /home/osmocom/.config/pmbootstrap_v3.cfg'
fi
_chk "pmOS lanceurs en root (--as-root dans osmo-pmos-qemu)" grep -q 'as-root'
 ↪  /opt/user_interface/pmos/bin/osmo-pmos-qemu.sh
_chk "pmOS kpartx + losetup + git (pmbootstrap les exige)"  bash -c 'command -v kpartx && command -v losetup &&
 ↪  command -v git'
if [ "${OSMO_ISO_PMAPORTS:-1}" = "1" ]; then
    _chk "pmOS pmaports au commit du noyau (pas de clone au premier clic)" bash -c 'test -f
     ↪  /opt/user_interface/pmos/pmaports/device/main/linux-postmarketos-stable/config-stable.x86_64'
fi
if [ "$_ko" -gt 0 ]; then
    if [ "${OSMO_ISO_LTE_REQUIRED:-1}" = "1" ]; then
        echo -e "  ${RED}✗ $_ko maillon(s) manquant(s) : cette ISO ne serait PAS prete a l emploi (2G + 4G +
         ↪  telephone).${NC}" >&2
        echo -e "    Corrige ci-dessus, ou OSMO_ISO_LTE_REQUIRED=0 pour sortir l ISO quand meme." >&2
        _iso88_down
        exit 1
    fi
    echo -e "  ${YELLOW}!${NC} $_ko maillon(s) manquant(s), ISO sortie quand meme (OSMO_ISO_LTE_REQUIRED=0)"
else
    echo -e "  ${GREEN}✓${NC} prete a l emploi : banc 2G, 4G et telephone au complet"
fi
unset _ko; unset -f _chk

# Unmount what was mounted above, otherwise the squashfs of 90-iso.sh would
# include the host's /proc and /sys.
_iso88_down; unset _iso88_mnt; unset -f _iso88_up _iso88_down


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. Always return 0 here.
true
```

2.85 /opt/GSM/osmo-operator/iso_modules/89-home-chiffre.sh

9427 octets, 140 lignes → 140 lignes

```
#!/bin/bash
# iso_modules/89-home-chiffre.sh - step 8i: encrypted second disk on /home.
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to be run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
```

```
# Environment variables:
#   OSMO_ISO_CRYPTHOME_OWNER  account owning the encrypted home (default osmocom)

# ═══════════════════════════════════════════════════════════════════════
# Step 8i: the system disk stays readable, secrets go to the second disk
# ═══════════════════════════════════════════════════════════════════════
# Model: everything system-related is readable, everything keyring-related is
# encrypted.
#
# The system disk is intentionally NOT encrypted: the bench is driven as root
# (VTY, netns, tcpdump), the session logs in as root, and an operator must be
# able to extract a trace, read a log or repair the machine from a live key
# without decrypting anything.
#
# Secrets (GNOME keyring, browser cookies and tokens, profiles) live on a
# second LUKS2 disk mounted on /home. The unprivileged osmocom account has its
# home there, and applications holding secrets run as that account from the
# root session (run-as-owner).
#
# At boot the machine asks "decrypt /home -passphrase- or Enter" on the text
# console (plymouth is quit first, see unlock-home). An empty answer, a wrong
# passphrase or no answer never blocks the boot: the root session opens, /home
# stays empty, and the same applications start without secrets.
#
# Encryption is done by Calamares at install time:
#   - installer/calamares/modules/partition.conf splits the disk into root +
#     /home and marks root "noEncrypt", so "encrypt system" only encrypts
#     /home (LUKS2);
#   - crypthome-postinstall, run by shellprocess@osmo in the target, wires up
#     the result: crypttab and fstab entries ("noauto"), /etc/default/crypthome
#     (UUID, mapping, session owner), root keyring placed on the volume.
#
# This module encrypts nothing: it installs the tools in the rootfs for the
# live key and the target. With CRYPTHOME_UUID empty (live key, or install
# without encryption) unlock-home has nothing to do and exits.
#
# init-crypthome.sh covers the case Calamares does not: encrypting a second
# disk added later to an installed machine.
#
# run-as-owner is not a sandbox workaround: Firefox runs fine as root but would
# then write its cookies, tokens and keyring in clear on the system disk. The
# owner account is where secrets live, since its home is on the encrypted disk.
# It generalises the vlc wrapper of step 8e (85-installeur-bureau.sh: xhost,
# runuser, PULSE_SERVER, XDG_RUNTIME_DIR) to all applications holding secrets,
# conditionally: if the disk is not unlocked the application runs as root.
_CH_SRC="$DIR/configs/crypthome"
if [ ! -d "$_CH_SRC" ]; then
    echo -e "  ${YELLOW}!${NC} configs/crypthome absent - etape 8i sautee"
    return 0
fi

echo -e "${GREEN}[8i/9] Second disque chiffre sur /home (outils, rien n est chiffre ici)...${NC}"

# Account owning the encrypted home: osmocom (unprivileged), overridable.
_CH_OWNER="${OSMO_ISO_CRYPTHOME_OWNER:-osmocom}"

# ── Tools ──────────────────────────────────────────────────────────────
# unlock-home / lock-home in sbin (they mount/unmount), run-as-owner and
# crypthome-session-setup in bin (called from the session).
install -d "$ROOTFS/usr/local/sbin" "$ROOTFS/usr/local/bin" "$ROOTFS/root" \
           "$ROOTFS/etc/systemd/system" "$ROOTFS/etc/default" "$ROOTFS/usr/local/share/applications"
install -m 0755 "$_CH_SRC/unlock-home"            "$ROOTFS/usr/local/sbin/unlock-home"
install -m 0755 "$_CH_SRC/lock-home"              "$ROOTFS/usr/local/sbin/lock-home"
install -m 0755 "$_CH_SRC/crypthome-fond"         "$ROOTFS/usr/local/sbin/crypthome-fond"  # background image for
 ↪  the LUKS prompt
install -m 0755 "$_CH_SRC/refresh-owner-apps"     "$ROOTFS/usr/local/sbin/refresh-owner-apps"
```

```
install -m 0755 "$_CH_SRC/run-as-owner"           "$ROOTFS/usr/local/bin/run-as-owner"
install -m 0755 "$_CH_SRC/crypthome-session-setup" "$ROOTFS/usr/local/bin/crypthome-session-setup"
install -m 0700 "$_CH_SRC/init-crypthome.sh"      "$ROOTFS/root/init-crypthome.sh"
# Run by Calamares in the target (shellprocess@osmo) to wire up the /home the
# installer just encrypted. In sbin because it edits crypttab, fstab and
# /etc/default/crypthome.
install -m 0755 "$_CH_SRC/crypthome-postinstall"  "$ROOTFS/usr/local/sbin/crypthome-postinstall"
install -m 0644 "$_CH_SRC/unlock-home.service"    "$ROOTFS/etc/systemd/system/unlock-home.service"

# Configuration: __OWNER__ is the only per-image setting. CRYPTHOME_UUID stays
# empty; it is written on the machine once the disk exists.
sed "s/__OWNER__/$_CH_OWNER/" "$_CH_SRC/crypthome.default" > "$ROOTFS/etc/default/crypthome"
chmod 0644 "$ROOTFS/etc/default/crypthome"

# ── Service ──────────────────────────────────────────────────────────
# Enabled in the image: without a configured UUID it does nothing, but it is
# ready when init-crypthome.sh only has to write the UUID.
chroot "$ROOTFS" systemctl enable unlock-home.service 2>/dev/null || \
    echo -e "  ${YELLOW}!${NC} unlock-home.service non active (systemctl indisponible dans le rootfs)"

# ── Root keyring exists only on the encrypted disk ──────────────────
# Symlink to the encrypted disk, created before the first root session makes a
# real directory there. Disk locked = dangling link = gnome-keyring cannot
# write anything in clear on the system disk.
install -d -m 0700 "$ROOTFS/root/.local/share"
rm -rf "$ROOTFS/root/.local/share/keyrings"
ln -sfn /home/.crypthome/root-keyrings "$ROOTFS/root/.local/share/keyrings"

# ── Audio for the owner account ─────────────────────────────────────
# PulseAudio runs in system mode: the daemon belongs to `pulse` and only
# accepts members of pulse-access. The browser (dashboard microphone) runs as
# the owner; without these groups getUserMedia returns "NotFoundError".
chroot "$ROOTFS" bash -c "
    getent group pulse-access >/dev/null 2>&1 && usermod -aG pulse-access,audio,video $_CH_OWNER
" 2>/dev/null || echo -e "  ${YELLOW}!${NC} groupes audio non ajoutes a ${CYAN}$_CH_OWNER${NC}"

# ── Session ──────────────────────────────────────────────────────────
# The autostart entry prepares the switch when the root session opens: X
# server authorisation and a user session for the owner account. Without it
# the first Firefox launch waits for logind for several seconds silently.
if [ -d "$ROOTFS/etc/gdm3" ] || [ "${ISO_DESKTOP:-0}" = "1" ]; then
    install -d "$ROOTFS/root/.config/autostart" "$ROOTFS/etc/skel/.config/autostart"
    install -m 0644 "$_CH_SRC/crypthome-session.desktop" \
        "$ROOTFS/root/.config/autostart/crypthome-session.desktop"
    install -m 0644 "$_CH_SRC/crypthome-session.desktop" \
        "$ROOTFS/etc/skel/.config/autostart/crypthome-session.desktop"

    # Launchers of applications holding secrets go through run-as-owner. Done in
    # the chroot since the list depends on what is actually installed.
    chroot "$ROOTFS" /usr/local/sbin/refresh-owner-apps 2>/dev/null | sed 's/^/  /' || \
        echo -e "  ${YELLOW}!${NC} lanceurs non rediriges (a refaire avec refresh-owner-apps)"
    echo -e "  ${GREEN}✓${NC} applications a secrets : sous ${CYAN}$_CH_OWNER${NC} quand /home est dechiffre,
     ↪  sous ${CYAN}root${NC} sinon"
fi

# ── Required in the rootfs ──────────────────────────────────────────
# cryptsetup to open the volume, rsync for init-crypthome.sh, x11-xserver-utils
# for xhost.
for _p in cryptsetup rsync xhost; do
    chroot "$ROOTFS" bash -c "command -v $_p >/dev/null 2>&1" || \
        echo -e "  ${YELLOW}!${NC} ${CYAN}$_p${NC} absent du rootfs - requis par le dispositif de chiffrement"
done

echo -e "  ${GREEN}✓${NC} outils poses : ${CYAN}unlock-home${NC}, ${CYAN}lock-home${NC},
 ↪  ${CYAN}crypthome-postinstall${NC} (Calamares), ${CYAN}init-crypthome.sh${NC} (second disque)"
```

```
echo -e "  ${GREEN}✓${NC} a l installation : cocher ${CYAN}« chiffrer »${NC} chiffre ${CYAN}/home${NC} seul, la
↪  racine reste lisible"
echo -e "  ${GREEN}✓${NC} demarrage : ${CYAN}« decrypt /home -passphrase- or Enter »${NC} sur la console - le
↪  boot aboutit quelle que soit la reponse"

# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. Always return 0 here.
true
```

2.86 /opt/GSM/osmo-operator/iso_modules/90-iso.sh

16286 octets, 316 lignes → 316 lignes

```
#!/bin/bash
# iso_modules/90-iso.sh - step 9: squashfs, kernel, GRUB menu, /.disk.
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to be run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment variables:
#   OSMO_ISO_NO_VERIFY=1     skip the dpkg md5 integrity checks (~40 s per pass)
#   OSMO_ISO_SQUASH_JOBS=N   limit mksquashfs to N CPU cores (default: all)

# --arm: no squashfs or GRUB; 92-rpi-image writes the SD image.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then return 0; fi

# ── Squashfs and ISO creation ──────────────────────────────────────────
echo -e "${GREEN}[9/9] Squashfs et ISO...${NC}"
mkdir -p "$ISOROOT/live" "$ISOROOT/boot/grub"

# 2>/dev/null || true: under pipefail an empty ls would exit before the
# "Kernel absent" message below.
VMLINUZ=$(ls "$ROOTFS"/boot/vmlinuz-* 2>/dev/null | sort -V | tail -1 || true)
INITRD=$(ls "$ROOTFS"/boot/initrd.img-* 2>/dev/null | sort -V | tail -1 || true)
if [ -z "$VMLINUZ" ] || [ -z "$INITRD" ]; then echo -e "${RED}Kernel absent${NC}"; exit 1; fi
echo -e "  Kernel: ${CYAN}$(basename "$VMLINUZ")${NC}"

# ── Squashfs compression: zstd rather than xz ──────────────────────────
# The embedded kernel has CONFIG_SQUASHFS_DECOMP_SINGLE=y (one serialised
# decompression stream), so single-thread decompression speed matters: zstd
# -19 reads ~9x faster than xz for ~16% larger images.
# Fallback to xz: mksquashfs only supports zstd since 4.4 and only if built
# with libzstd, so the tool is probed with a test compression.
_squash_zstd_ok() {
    local t rc
    t="$(mktemp -d)" || return 1
    : > "$t/probe"
    mksquashfs "$t" "$t.sqfs" -comp zstd -no-progress -noappend >/dev/null 2>&1
    rc=$?
    rm -rf "$t" "$t.sqfs"
    return $rc
}

if _squash_zstd_ok; then
    SQUASH_COMP=(-comp zstd -Xcompression-level 19)
    echo -e "  Compression: ${CYAN}zstd -19${NC} (lecture ~9x plus rapide que xz a un thread)"
else
    SQUASH_COMP=(-comp xz -Xbcj x86)
    echo -e "  Compression: ${YELLOW}xz${NC} (mksquashfs sans zstd - ISO plus petite, mais lente a lire)"
fi

# ── Integrity check: package files against their dpkg md5sums ──────────
# Kernel and initrd are kept in the squashfs: it is also the Calamares source
# (unpackfs.conf), and the installed system needs them.
```

```
#
# The check runs twice with exactly the same filter, so both lists compare:
#
#   rootfs altered            -> before compression (dpkg, or a build step
#                                overwriting a packaged file)
#   rootfs ok + squashfs bad  -> mksquashfs or the read-back
#
# Excluded (modified by the build on purpose): /opt and /root (sources and
# venv), /usr/local, MongoDB, osmo-* units, calamares.desktop (NoDisplay),
# live-boot (toram), os-release, usr/bin/screenfetch (patched by
# 86-finitions.sh), and the ldd-closure libs of step 8b ($WORK/closure.list).
# Deleted files (docs, locales) do not count.
#
# Result lists are timestamped and compared with the previous build: the same
# files every time points to a build step, a different set to the host.
# About 40 s per pass. OSMO_ISO_NO_VERIFY=1 skips it.
_vfy_stamp="$(date +%Y%m%d-%H%M%S)"

# Run the check on a given root. Sets _vfy_out (failing paths, without
# ": FAILED"), _vfy_n (their count) and _vfy_prev (list of the previous build
# with the same label, empty on the first build).
_vfy_paquets() {   # $1 = label (rootfs|squashfs), $2 = root to check
    local _k="$1" _r="$2" _skip="$WORK/closure.list"
    [ -s "$_skip" ] || _skip=/dev/null
    _vfy_out="/var/tmp/osmo-iso-verify-${_vfy_stamp}-${_k}.failed"
    ( cd "$_r" && cat var/lib/dpkg/info/*.md5sums 2>/dev/null \
        | grep -vE '^[0-9a-f]{32}  (opt/|root/|usr/local/|mongodb|usr/lib/systemd/system/osmo-
        ↪  |usr/share/applications/calamares\.desktop$|lib/live/boot/|usr/lib/os-release$|usr/bin/screenfetch$)'
        ↪  \
        | awk -v skip="$_skip" 'BEGIN{while((getline l < skip)>0) s[l]=1} !(substr($0,35) in s)' \
        | md5sum -c --quiet 2>/dev/null | grep -v 'open or read' \
        | sed 's/: FAILED$//' ) > "$_vfy_out" || true
    _vfy_n=$(wc -l < "$_vfy_out")
    # Legacy path, points to the latest pass.
    ln -sf "$_vfy_out" /var/tmp/osmo-iso-verify.failed
    _vfy_prev=$(ls -1t /var/tmp/osmo-iso-verify-*-"${_k}".failed 2>/dev/null \
        | grep -vF "$_vfy_stamp" | head -1) || true
    # Keep the last 20 lists.
    ls -1t /var/tmp/osmo-iso-verify-*-"${_k}".failed 2>/dev/null \
        | tail -n +21 | xargs -r rm -f || true
}

# Verdict for a non-empty list. The comparison with the previous build gives
# the diagnosis, not the number of files.
_vfy_verdict() {
    local _c
    head -8 "$_vfy_out" | sed 's/^/      /'
    if [ "$_vfy_n" -gt 8 ]; then echo "      ... (liste complete : $_vfy_out)"; fi
    if [ -n "$_vfy_prev" ]; then
        _c=$(grep -cxF -f "$_vfy_out" "$_vfy_prev" 2>/dev/null) || _c=0
        if [ "${_c:-0}" -gt 0 ]; then
            echo -e "  ${YELLOW}${_c} de ces fichiers echouaient DEJA au build precedent${NC}"
            echo -e "  ${YELLOW}($(basename "$_vfy_prev")) : la liste est STABLE. Ce n est pas de la${NC}"
            echo -e "  ${YELLOW}RAM qui flanche - c est une etape du build qui reecrit ces fichiers${NC}"
            echo -e "  ${YELLOW}par-dessus leur paquet sans etre ecartee du filtre ci-dessus.${NC}"
        else
            echo -e "  ${YELLOW}Aucun de ces fichiers n echouait au build precedent${NC}"
            echo -e "  ${YELLOW}($(basename "$_vfy_prev")) : le tirage CHANGE a chaque fois. Aucune${NC}"
            echo -e "  ${YELLOW}etape du build ne vise des fichiers au hasard : c est l hote.${NC}"
            echo -e "  ${YELLOW}RAM non-ECC : rien ne sera jamais signale (ni EDAC, ni dmesg).${NC}"
            echo -e "  ${YELLOW}La faute n apparait que SOUS CHARGE SOUTENUE sur tous les coeurs :${NC}"
            echo -e "  ${YELLOW}memtest86+ peut passer une nuit entiere sans rien voir (il ne${NC}"
            echo -e "  ${YELLOW}chauffe pas le CPU). Refaire plutot ce build a froid, et le${NC}"
            echo -e "  ${YELLOW}refaire avec OSMO_ISO_SQUASH_JOBS=4 : si les echecs disparaissent${NC}"
            echo -e "  ${YELLOW}a 4 coeurs et reviennent a 32, c est l alimentation/le CPU sous${NC}"
            echo -e "  ${YELLOW}charge, pas une barrette morte (un i9-14900HX est concerne par${NC}"
```

```
            echo -e "  ${YELLOW}l instabilite Raptor Lake : microcode a jour = degradation${NC}"
            echo -e "  ${YELLOW}arretee, pas reparee). Sinon : memtest86+, et temperatures.${NC}"
        fi
    else
        echo -e "  ${YELLOW}Pas de build precedent a comparer. Relancer tel quel : meme liste =${NC}"
        echo -e "  ${YELLOW}une etape du build reecrit ces fichiers ; liste differente = l hote.${NC}"
    fi
}

# Pass 1: the rootfs as left by dpkg and the build, before compression.
if [ "${OSMO_ISO_NO_VERIFY:-0}" != "1" ]; then
    _vfy_paquets rootfs "$ROOTFS"
    if [ "$_vfy_n" -eq 0 ]; then
        echo -e "  ${GREEN}✓${NC} rootfs conforme a dpkg avant compression"
    else
        echo -e "  ${RED}✗ rootfs ALTERE AVANT compression : ${_vfy_n} fichier(s)${NC}"
        _vfy_verdict
        echo -e "  ${YELLOW}L alteration est ANTERIEURE a mksquashfs : la compression et sa${NC}"
        echo -e "  ${YELLOW}relecture sont hors de cause, ne pas les chercher.${NC}"
        echo -e "  ${YELLOW}OSMO_ISO_NO_VERIFY=1 pour livrer quand meme (l ISO plantera).${NC}"
        exit 1
    fi
fi

# ── OSMO_ISO_SQUASH_JOBS: limit compression parallelism ──────────────
# mksquashfs uses all cores by default. Limiting it helps tell host
# instability under full load from a build problem, and allows a correct
# (slower) build on such a host. Empty by default.
SQUASH_JOBS=()
if [ -n "${OSMO_ISO_SQUASH_JOBS:-}" ]; then
    SQUASH_JOBS=(-processors "$OSMO_ISO_SQUASH_JOBS")
    echo -e "  ${CYAN}compression bridee a ${OSMO_ISO_SQUASH_JOBS} coeur(s) (OSMO_ISO_SQUASH_JOBS)${NC}"
fi
mksquashfs "$ROOTFS" "$ISOROOT/live/filesystem.squashfs" \
    "${SQUASH_COMP[@]}" "${SQUASH_JOBS[@]}" -b 1M \
    -e 'var/cache/apt' -e 'var/lib/apt/lists' \
    -no-progress
echo -e "  ${GREEN}✓${NC} squashfs $(du -sh "$ISOROOT/live/filesystem.squashfs"|cut -f1)"

# ── Pass 2: the squashfs read back, against the same dpkg md5sums ────
# Pass 1 found the rootfs clean, so failures here come from compression,
# writing or the read-back. See pass 1 for the filter.
if [ "${OSMO_ISO_NO_VERIFY:-0}" != "1" ]; then
    _vfy_mnt="$WORK/sqfs-verify"
    mkdir -p "$_vfy_mnt"
    if mount -t squashfs -o loop,ro "$ISOROOT/live/filesystem.squashfs" "$_vfy_mnt" 2>/dev/null; then
        _vfy_paquets squashfs "$_vfy_mnt"
        umount "$_vfy_mnt" 2>/dev/null || true
        if [ "$_vfy_n" -eq 0 ]; then
            echo -e "  ${GREEN}✓${NC} squashfs relu : les fichiers des paquets sont conformes a dpkg"
        else
            echo -e "  ${RED}✗ squashfs ALTERE : ${_vfy_n} fichier(s) ne correspondent plus a leur paquet${NC}"
            _vfy_verdict
            echo -e "  ${YELLOW}Le rootfs, lui, etait conforme juste avant (passe 1). NE PAS${NC}"
            echo -e "  ${YELLOW}chercher un bug de mksquashfs : il est deterministe, il donnerait${NC}"
            echo -e "  ${YELLOW}les MEMES fichiers a chaque fois. Ce qu il compresse, ce sont les${NC}"
            echo -e "  ${YELLOW}pages que la lecture du rootfs vient de remplir - 20 Go lus a${NC}"
            echo -e "  ${YELLOW}pleine charge - et c est LA que les octets basculent. Il les${NC}"
            echo -e "  ${YELLOW}compresse fidelement ensuite. Compression a tous les coeurs = le${NC}"
            echo -e "  ${YELLOW}moment ou l hote chauffe et consomme le plus de tout le build.${NC}"
            echo -e "  ${YELLOW}OSMO_ISO_NO_VERIFY=1 pour livrer quand meme (l ISO plantera).${NC}"
            exit 1
        fi
    else
        echo -e "  ${YELLOW}!${NC} squashfs non relu (mount loop impossible) : seule la passe 1 a parle"
    fi
```

```
    rmdir "$_vfy_mnt" 2>/dev/null || true
    unset _vfy_mnt _vfy_out _vfy_n _vfy_prev _vfy_stamp
fi

cp "$VMLINUZ" "$ISOROOT/boot/vmlinuz"
cp "$INITRD"  "$ISOROOT/boot/initrd.img"

# ── GRUB menu ──────────────────────────────────────────────────────
# RAM figures shown in the menu are computed from the squashfs just written.
SQ_MB=$(( $(stat -Lc%s "$ISOROOT/live/filesystem.squashfs") / 1048576 ))
# toram copies the squashfs to a tmpfs and the system runs on top of it:
# file size plus 2 GB, rounded up to the next GB.
RAM_TORAM_GB=$(( (SQ_MB + 2048 + 1023) / 1024 ))
SQ_GB=$(awk -v m="$SQ_MB" 'BEGIN{printf "%.1f", m/1024}')

# "osmo" theme (tools/grub-theme-render.py), same as on the installed system.
# If it cannot load (missing fonts, signed GRUB without gfxterm), the
# "if loadfont" block is skipped and the menu stays in text mode.
python3 "$DIR/tools/grub-theme-render.py" --out "$ISOROOT/boot/grub/themes/osmo" >/dev/null 2>&1 \
    || echo -e "  ${YELLOW}!${NC} grub : rendu du theme echoue - menu du live en texte"

cat > "$ISOROOT/boot/grub/grub.cfg" <<GRUB
# WARNING - this file is GENERATED by build-iso.sh. Changes made in the ISO
# do not survive the next build.
#
# Under VirtualBox, two settings avoid minutes of waiting on the "RAM" entry:
# attach the ISO to the SATA controller rather than IDE, and enable
# "Use Host I/O Cache" on it.

set default=0
set timeout=5

if loadfont /boot/grub/themes/osmo/osmo-18.pf2; then
    loadfont /boot/grub/themes/osmo/osmo-24.pf2
    loadfont /boot/grub/themes/osmo/osmom-22.pf2
    set gfxmode=auto
    insmod all_video
    insmod gfxterm
    insmod png
    terminal_output gfxterm
    set theme=/boot/grub/themes/osmo/theme.txt
fi

# Three main entries (read from medium, copy to RAM, write to medium); the
# rest is in a submenu.
menuentry "osmo-operator" {
    linux  /boot/vmlinuz boot=live quiet
    initrd /boot/initrd.img
}

# Plain "toram" makes live-boot copy the WHOLE medium to a tmpfs
# (lib/live/boot/9990-toram-todisk.sh) with a silent "cp -a".
# "toram=filesystem.squashfs" copies only the squashfs (tmpfs sized on it),
# and "quiet" is omitted so the copy progress is visible.
menuentry "osmo-operator - en RAM (copie ${SQ_GB} Go - ${RAM_TORAM_GB} Go de RAM mini)" {
    linux  /boot/vmlinuz boot=live toram=filesystem.squashfs
    initrd /boot/initrd.img
}

# Without persistence the root is a tmpfs overlay: everything written is lost
# at reboot. No toram here, since the overlay would then stay in RAM.
#
# Requires a volume LABELLED "persistence" holding a persistence.conf with
# the line "/ union":
#   sudo mkfs.ext4 -L persistence /dev/sdX3
#   sudo mount /dev/sdX3 /mnt && echo "/ union" | sudo tee /mnt/persistence.conf
```

```
# In a VM a second disk is enough. Without such a volume this entry boots
# as a plain live system.
menuentry "osmo-operator - persistant (ecrit sur le medium)" {
    linux  /boot/vmlinuz boot=live persistence persistence-encryption=none quiet
    initrd /boot/initrd.img
}

# Verbose variants (same command lines without "quiet") are for diagnostics.
# No graphics acceleration: for GPUs the 6.8 kernel does not support (recent
# AMD RDNA without firmware, NVIDIA without nouveau) that give a black screen
# or a crash; nomodeset keeps the firmware framebuffer.
menuentry "osmo-operator - sans acceleration graphique (nomodeset : ecran noir, plantage AMD/NVIDIA)" {
    linux  /boot/vmlinuz boot=live nomodeset quiet
    initrd /boot/initrd.img
}

submenu "Options (demarrage verbeux)" {
    menuentry "osmo-operator - verbose" {
        linux  /boot/vmlinuz boot=live
        initrd /boot/initrd.img
    }
    menuentry "osmo-operator - persistant verbose" {
        linux  /boot/vmlinuz boot=live persistence persistence-encryption=none
        initrd /boot/initrd.img
    }
    menuentry "osmo-operator - persistant, sans acceleration graphique (nomodeset)" {
        linux  /boot/vmlinuz boot=live persistence persistence-encryption=none nomodeset
        initrd /boot/initrd.img
    }
}
GRUB
echo -e "  ${GREEN}✓${NC} menu GRUB : defaut = lecture depuis le medium ; toram annonce ${CYAN}${RAM_TORAM_GB}
 ↪  Go${NC} de RAM"

# ── /.disk/info: required for UEFI boot, otherwise GRUB stops at "grub>" ──
# Ubuntu's signed GRUB (gcdx64.efi.signed) has an embedded config:
#
#     if [ -z "$prefix" -o ! -e "$prefix" ]; then
#         if ! search --file --set=root /.disk/info; then
#             search --file --set=root /.disk/mini-info
#         fi
#         set prefix=($root)/boot/grub
#     fi
#     ... source $prefix/grub.cfg ... else source $cmdpath/grub.cfg
#
# Its compiled prefix "/boot/grub" has no device and resolves on the EFI
# (FAT) partition, which has no /boot/grub, so it relies on finding
# /.disk/info. BIOS boot (eltorito.img, built with -p /boot/grub) does not
# need it. Ubuntu and Debian ISOs ship the same file for the same reason.
mkdir -p "$ISOROOT/.disk"
printf 'osmo-operator %s - live amd64 (%s)\n' "$VERSION" "$LABEL" > "$ISOROOT/.disk/info"
cp "$ISOROOT/.disk/info" "$ISOROOT/.disk/mini-info"
echo -e "  ${GREEN}✓${NC} ${CYAN}/.disk/info${NC} pose : le GRUB signe retrouve le medium en UEFI"

# EFI modules where $prefix will look for them. The generated menu only uses
# commands built into the signed binary, but they make "insmod" work at the
# prompt. 3 MB.
if [ -d /usr/lib/grub/x86_64-efi ]; then
    mkdir -p "$ISOROOT/boot/grub/x86_64-efi"
    cp -a /usr/lib/grub/x86_64-efi/*.mod /usr/lib/grub/x86_64-efi/*.lst \
          "$ISOROOT/boot/grub/x86_64-efi/" 2>/dev/null || true
    # No grub.cfg in this directory: the embedded config tries
    # "$prefix/x86_64-efi/grub.cfg" BEFORE "$prefix/grub.cfg".
    rm -f "$ISOROOT/boot/grub/x86_64-efi/grub.cfg"
fi
```

```
# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. Always return 0 here.
true
```

2.87 /opt/GSM/osmo-operator/iso_modules/91-secure-boot.sh

10614 octets, 217 lignes → 217 lignes

```
#!/bin/bash
# iso_modules/91-secure-boot.sh - Secure Boot ISO (signed shim + grub), or
# grub-mkrescue fallback.
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to be run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Host requirements for Secure Boot: shim-signed, grub-efi-amd64-signed,
# mtools, dosfstools (sbsigntool optional, to verify the kernel signature).

# --arm: no shim or xorriso on the Pi.
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then return 0; fi

# ═══════════════════════════════════════════════════════════════════════
# SECURE BOOT - why this step does not use grub-mkrescue
# ═══════════════════════════════════════════════════════════════════════
# grub-mkrescue builds an unsigned BOOTX64.EFI from host modules, which the
# firmware refuses to load when Secure Boot is enabled.
#
# Chain of trust:
#
#   firmware --(Microsoft key)--> shimx64.efi.signed   package shim-signed
#            --(Canonical key)--> gcdx64.efi.signed    package grub-efi-amd64-signed
#            --(Canonical key)--> vmlinuz              linux-image-generic (signed)
#
# shim is the only Microsoft-signed link; it carries the Canonical key. It is
# copied from the package, not built.
#
# gcdx64, not grubx64: both are Canonical-signed but their compiled prefix
# differs:
#     grubx64.efi.signed  -> prefix /EFI/ubuntu, looks for its config on the
#                            EFI partition (absent on an ISO: bare "grub>").
#     gcdx64.efi.signed   -> prefix /boot/grub relative to the boot medium
#                            (optical variant), finds $ISOROOT/boot/grub/grub.cfg.
# A stub /EFI/ubuntu/grub.cfg is still put on the ESP for the grubx64 fallback.
#
# Nothing is signed here: no private key, nothing to enrol (no MokManager on
# first boot). Only binaries already signed by Microsoft and Canonical are
# assembled, like an official Ubuntu ISO.
#
# BIOS boot: the El Torito i386-pc image is built by grub-mkimage
# (i386-pc-eltorito, includes cdboot.img) and the hybrid MBR by --grub2-mbr.
#
# Fallback: if a component is missing on the host, grub-mkrescue builds an
# ISO that boots everywhere EXCEPT with Secure Boot, and this is reported.

SB_SHIM=""
for c in /usr/lib/shim/shimx64.efi.signed \
         /usr/lib/shim/shimx64.efi.signed.latest \
         /usr/lib/shim/shimx64.efi; do
    [ -f "$c" ] && { SB_SHIM="$c"; break; }
done
# gcdx64 first (prefix /boot/grub, for optical media), grubx64 as fallback.
SB_GRUB="" ; SB_GRUB_KIND=""
for c in /usr/lib/grub/x86_64-efi-signed/gcdx64.efi.signed:gcd \
```

```
         /usr/lib/grub/x86_64-efi-signed/grubx64.efi.signed:grub; do
    [ -f "${c%:*}" ] && { SB_GRUB="${c%:*}"; SB_GRUB_KIND="${c##*:}"; break; }
done
SB_MM=""
for c in /usr/lib/shim/mmx64.efi.signed /usr/lib/shim/mmx64.efi; do
    [ -f "$c" ] && { SB_MM="$c"; break; }
done

# The kernel must be signed too: shim validates GRUB, GRUB validates the
# kernel. An unsigned vmlinuz fails AFTER the menu. Ubuntu "generic" kernels
# are signed; the signature is checked rather than trusting the package name.
SB_KERNEL_SIGNED=0
if command -v sbverify &>/dev/null; then
    sbverify --list "$ISOROOT/boot/vmlinuz" &>/dev/null && SB_KERNEL_SIGNED=1
elif grep -qa '~Module signature appended~\|sbat\|Canonical Ltd\. Secure Boot' "$ISOROOT/boot/vmlinuz"
 ↪  2>/dev/null; then
    SB_KERNEL_SIGNED=1
fi

SECURE_BOOT_OK=0
if [ -n "$SB_SHIM" ] && [ -n "$SB_GRUB" ] && command -v mmd &>/dev/null; then
    echo -e "${GREEN}[9/9] ISO Secure Boot (shim + grub signes)...${NC}"
    echo -e "  shim : ${CYAN}${SB_SHIM}${NC}"
    echo -e "  grub : ${CYAN}${SB_GRUB}${NC} (${SB_GRUB_KIND})"

    # ── EFI system partition ──────────────────────────────────────────
    # FAT16 with a 16 MB floor: the content is ~4 MB, but FAT16 needs at least
    # 4085 clusters or mkfs.vfat fails. 16 MB with 2 KB clusters (-s 4) gives
    # 8192 clusters.
    SB_ESP="$WORK/efi.img"
    SB_KB=$(( ( $(stat -Lc%s "$SB_SHIM") + $(stat -Lc%s "$SB_GRUB") \
              + $( [ -n "$SB_MM" ] && stat -Lc%s "$SB_MM" || echo 0 ) ) / 1024 + 2048 ))
    [ "$SB_KB" -lt 16384 ] && SB_KB=16384
    rm -f "$SB_ESP"
    mkfs.vfat -F 16 -s 4 -n OSMOEFI -C "$SB_ESP" "$SB_KB" >/dev/null

    mmd   -i "$SB_ESP" ::/EFI ::/EFI/BOOT ::/EFI/ubuntu
    mcopy -i "$SB_ESP" "$SB_SHIM" ::/EFI/BOOT/BOOTX64.EFI
    mcopy -i "$SB_ESP" "$SB_GRUB" ::/EFI/BOOT/grubx64.efi
    [ -n "$SB_MM" ] && mcopy -i "$SB_ESP" "$SB_MM" ::/EFI/BOOT/mmx64.efi

    # Stub: last resort of the config embedded in gcdx64, which ends with
    # "source $cmdpath/grub.cfg" ($cmdpath = /EFI/BOOT on this ESP). Also used
    # as-is by the grubx64 fallback (prefix /EFI/ubuntu).
    #
    # "search --set=root" leaves the variable unchanged on failure, so a fresh
    # variable is used to test the result, and a message is printed if nothing
    # is found.
    cat > "$WORK/esp-grub.cfg" <<'ESPCFG'
search --no-floppy --file --set=osmodev /.disk/info
if [ -z "$osmodev" ]; then
    search --no-floppy --file --set=osmodev /boot/grub/grub.cfg
fi
if [ -n "$osmodev" ]; then
    set root=$osmodev
    set prefix=($osmodev)/boot/grub
    configfile ($osmodev)/boot/grub/grub.cfg
fi
echo "GRUB : ni /.disk/info ni /boot/grub/grub.cfg trouves sur les disques vus."
echo "Le medium n'est probablement pas lisible par le firmware a ce stade."
ESPCFG
    mcopy -i "$SB_ESP" "$WORK/esp-grub.cfg" ::/EFI/ubuntu/grub.cfg
    mcopy -i "$SB_ESP" "$WORK/esp-grub.cfg" ::/EFI/BOOT/grub.cfg

    # ── Same tree in the ISO9660 filesystem ──────────────────────────
    # The appended ESP is enough for DVDs and image-mode writes (dd, Rufus DD).
```

```
    # Copying the ISO contents onto a FAT-formatted key (common on Windows) only
    # sees the ISO9660, which also needs /EFI/BOOT (xorriso warns otherwise).
    # Ubuntu ISOs do the same.
    mkdir -p "$ISOROOT/EFI/BOOT"
    cp "$SB_SHIM" "$ISOROOT/EFI/BOOT/BOOTX64.EFI"
    cp "$SB_GRUB" "$ISOROOT/EFI/BOOT/grubx64.efi"
    [ -n "$SB_MM" ] && cp "$SB_MM" "$ISOROOT/EFI/BOOT/mmx64.efi"
    cp "$WORK/esp-grub.cfg" "$ISOROOT/EFI/BOOT/grub.cfg"
    mkdir -p "$ISOROOT/EFI/ubuntu"
    cp "$WORK/esp-grub.cfg" "$ISOROOT/EFI/ubuntu/grub.cfg"

    # ── BIOS boot image, built here since grub-mkrescue is not used ──────────
    # i386-pc-eltorito already includes cdboot.img: usable directly as
    # -eltorito-boot.
    SB_BIOS="$WORK/eltorito.img"
    grub-mkimage -O i386-pc-eltorito -p /boot/grub -o "$SB_BIOS" \
        biosdisk iso9660 part_msdos part_gpt fat ext2 normal linux configfile \
        search search_label search_fs_uuid search_fs_file loopback gzio \
        all_video gfxterm videotest videoinfo test echo ls minicmd sleep \
        halt reboot chain 2>/dev/null

    if [ -s "$SB_BIOS" ]; then
        # -eltorito-alt-boot separates the two catalog entries (BIOS, then UEFI).
        # "--interval:appended_partition_2" refers to the partition appended below,
        # without storing it twice in the image.
        xorriso -as mkisofs -iso-level 3 \
            -volid "$LABEL" \
            -full-iso9660-filenames \
            -eltorito-boot boot/grub/eltorito.img \
                -no-emul-boot -boot-load-size 4 -boot-info-table \
                --eltorito-catalog boot/grub/boot.cat \
                --grub2-boot-info \
                --grub2-mbr /usr/lib/grub/i386-pc/boot_hybrid.img \
            -eltorito-alt-boot \
                -e --interval:appended_partition_2:all:: \
                -no-emul-boot \
            -append_partition 2 0xef "$SB_ESP" \
            -appended_part_as_gpt \
            -o "$OUTPUT" \
            -graft-points "$ISOROOT" "/boot/grub/eltorito.img=$SB_BIOS" \
            && SECURE_BOOT_OK=1
    fi

    if [ "$SECURE_BOOT_OK" = "1" ]; then
        if [ "$SB_KERNEL_SIGNED" = "1" ]; then
            echo -e "  ${GREEN}✓${NC} ISO signee Secure Boot : shim -> grub -> noyau, chaine complete"
        else
            echo -e "  ${YELLOW}⚠${NC}  shim et grub sont signes, mais la signature du NOYAU n'a pas"
            echo -e "      pu etre confirmee ($ISOROOT/boot/vmlinuz). Si le boot s'arrete APRES"
            echo -e "      le menu GRUB, c'est la : installez sbsigntool pour le verifier, ou"
            echo -e "      utilisez un noyau linux-image-generic non recompile."
        fi
    else
        echo -e "  ${YELLOW}⚠${NC}  assemblage Secure Boot echoue - repli sur grub-mkrescue"
    fi
else
    echo -e "${YELLOW}[9/9] Secure Boot indisponible sur cet hote :${NC}"
    [ -z "$SB_SHIM" ] && echo -e "      shim absent  -> apt install ${CYAN}shim-signed${NC}"
    [ -z "$SB_GRUB" ] && echo -e "      grub signe absent -> apt install ${CYAN}grub-efi-amd64-signed${NC}"
    command -v mmd &>/dev/null || echo -e "      mtools absent -> apt install ${CYAN}mtools dosfstools${NC}"
fi

if [ "$SECURE_BOOT_OK" != "1" ]; then
# Fallback: ISO that boots in BIOS and UEFI without Secure Boot.
#
# Wrapper: inject -iso-level 3 (multi-extent, lifts the 4 GiB single-file cap)
```

```
# into grub-mkrescue's internal `xorriso -as mkisofs` call.
XORRISO_WRAP="$WORK/xorriso-iso-level3"
cat > "$XORRISO_WRAP" <<'EOF'
#!/bin/sh
if [ "$1" = "-as" ] && [ "$2" = "mkisofs" ]; then
    shift 2
    exec xorriso -as mkisofs -iso-level 3 "$@"
fi
exec xorriso "$@"
EOF
chmod +x "$XORRISO_WRAP"

    grub-mkrescue --xorriso="$XORRISO_WRAP" -o "$OUTPUT" "$ISOROOT" \
        --product-name "osmo-operator $VERSION" -- -volid "$LABEL"
    if command -v isohybrid &>/dev/null; then
        isohybrid --uefi "$OUTPUT"
    fi
    echo -e "  ${YELLOW}!${NC} ISO NON signee : elle ne demarrera pas si Secure Boot est actif."
    echo -e "    Desactivez-le dans le firmware, ou construisez sur un hote qui a"
    echo -e "    ${CYAN}shim-signed${NC} et ${CYAN}grub-efi-amd64-signed${NC}."
fi

if [ ! -f "$OUTPUT" ]; then
    echo -e "${RED}Creation de l'ISO echouee - rien n'a ete ecrit${NC}"
    exit 1
fi


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. Always return 0 here.
true
```

2.88 /opt/GSM/osmo-operator/iso_modules/92-rpi-image.sh

6008 octets, 94 lignes → 94 lignes

```
#!/bin/bash
# iso_modules/92-rpi-image.sh - --arm: Raspberry Pi 4 SD card image (.img).
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to be run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment variables:
#   OSMO_RPI_BOOT_MB  size of the FAT firmware partition in MB (default 512)
#   OSMO_RPI_ROOT_MB  minimum size of the ext4 root partition in MB

# amd64: the ISO is built by 90-iso / 91-secure-boot.
if [ "${ISO_ARCH:-amd64}" != "arm64" ]; then return 0; fi

# ═══════════════════════════════════════════════════════════════════════
#  Step 9: the SD image
# ═══════════════════════════════════════════════════════════════════════
# Two partitions, MBR table (the Pi GPU only reads MBR):
#   1  FAT32  RPICFG        contents of /boot/firmware (kernel, dtb, start4.elf)
#   2  ext4   armbi_root    persistent rootfs (no squashfs, no live)
# Same labels, sizes (512 MB FAT) and MBR table as Armbian rpi4b images:
# armbian-resize-filesystem grows armbi_root to the card on first boot.
# Nothing is mounted: mke2fs -d populates the ext4 from the directory (owners,
# modes, links, xattrs/capabilities), mcopy fills the FAT, sfdisk writes the
# table, dd assembles. No loop device, nothing left mounted on failure.
# The ext4 is the rootfs size plus 15% plus 768 MB of headroom.
echo -e "${GREEN}[9/9] Image SD Raspberry Pi 4 (${ISO_ARCH})...${NC}"

# ── What must not be shipped ─────────────────────────────────────────
```

```
# The emulator copied for the chroot is an x86 binary: useless on the Pi.
rm -f "$ROOTFS/usr/bin/qemu-aarch64-static"
# The apt cache bind mount is already unmounted (86-finitions); what remains
# in var/cache/apt belongs to the rootfs itself.
rm -rf "$ROOTFS/var/lib/apt/lists"/* "$ROOTFS/var/cache/apt/archives"/*.deb 2>/dev/null || true

# ── Firmware partition: contents of /boot/firmware, set aside ──────────────
# In the ext4, /boot/firmware must be an EMPTY directory (FAT mount point):
# its contents are moved out and restored after the image is built.
BOOTSRC="$WORK/bootfs"
rm -rf "$BOOTSRC"; mv "$ROOTFS/boot/firmware" "$BOOTSRC"; install -d "$ROOTFS/boot/firmware"
for _f in vmlinuz initrd.img start4.elf fixup4.dat bcm2711-rpi-4-b.dtb config.txt cmdline.txt; do
    [ -f "$BOOTSRC/$_f" ] || { echo -e "${RED}/boot/firmware/$_f absent - image impossible${NC}" >&2; exit 1; }
done

# ── Sizes ──────────────────────────────────────────────────────────────────
BOOT_MB="${OSMO_RPI_BOOT_MB:-512}"
_root_kb=$(du -sxk "$ROOTFS" | cut -f1)
ROOT_MB=$(( _root_kb / 1024 * 115 / 100 + 768 ))
[ -n "${OSMO_RPI_ROOT_MB:-}" ] && [ "$OSMO_RPI_ROOT_MB" -gt "$ROOT_MB" ] && ROOT_MB="$OSMO_RPI_ROOT_MB"
echo -e "  rootfs $(( _root_kb / 1024 )) Mo -> ext4 de ${ROOT_MB} Mo, FAT de ${BOOT_MB} Mo"

# ── Partition 1: FAT32 ─────────────────────────────────────────────────────
BOOT_IMG="$WORK/boot.img"; rm -f "$BOOT_IMG"
mkfs.vfat -F 32 -n RPICFG -C "$BOOT_IMG" $(( BOOT_MB * 1024 )) >/dev/null
# mcopy -s: recursive (overlays/). "::" is the FAT root.
mcopy -s -i "$BOOT_IMG" "$BOOTSRC"/* :: \
    || { echo -e "${RED}mcopy vers la partition firmware a echoue${NC}" >&2; exit 1; }
echo -e "  ${GREEN}✓${NC} RPICFG : $(mdir -i "$BOOT_IMG" :: | tail -1 | sed 's/^ *//')"

# ── Partition 2: ext4, populated without mounting ──────────────────────────
ROOT_IMG="$WORK/root.img"; rm -f "$ROOT_IMG"
truncate -s "${ROOT_MB}M" "$ROOT_IMG"
# -d: contents; -m 1: 1% reserved for root; lazy_*=0: inode tables are
# written now, not on first mount on a slow SD card.
mke2fs -q -F -t ext4 -L armbi_root -m 1 -E lazy_itable_init=0,lazy_journal_init=0 \
    -d "$ROOTFS" "$ROOT_IMG" \
    || { echo -e "${RED}mke2fs -d a echoue : rootfs trop gros pour ${ROOT_MB} Mo ? (OSMO_RPI_ROOT_MB=... pour
    ↪  forcer)${NC}" >&2; exit 1; }
echo -e "  ${GREEN}✓${NC} armbi_root : ext4 ${ROOT_MB} Mo peuplee ($(e2fsck -n "$ROOT_IMG" 2>/dev/null | tail -1
↪  | sed 's/^[^:]*: //' || true))"

# Put /boot/firmware back into the rootfs (a following pass, or a kept
# rootfs, needs it).
rm -rf "$ROOTFS/boot/firmware"; mv "$BOOTSRC" "$ROOTFS/boot/firmware"

# ── Assembly: MBR table + both partitions ──────────────────────────────────
rm -f "$OUTPUT"
_boot_start=2048
_boot_size=$(( BOOT_MB * 2048 ))
_root_start=$(( _boot_start + _boot_size ))
_root_size=$(( ROOT_MB * 2048 ))
truncate -s $(( (_root_start + _root_size + 2048) * 512 )) "$OUTPUT"
sfdisk --quiet "$OUTPUT" <<SFD
label: dos
unit: sectors
start=${_boot_start}, size=${_boot_size}, type=c, bootable
start=${_root_start}, size=${_root_size}, type=83
SFD
dd if="$BOOT_IMG" of="$OUTPUT" bs=512 seek="$_boot_start" conv=notrunc,sparse status=none
dd if="$ROOT_IMG" of="$OUTPUT" bs=512 seek="$_root_start" conv=notrunc,sparse status=none
rm -f "$BOOT_IMG" "$ROOT_IMG"
sfdisk --dump "$OUTPUT" | sed 's/^/    /' | grep -E "^ +$OUTPUT" || true
echo -e "  ${GREEN}✓${NC} image assemblee : $(du -h "$OUTPUT" | cut -f1) occupes, $(du -h --apparent-size
↪  "$OUTPUT" | cut -f1) apparents"
```

```
# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. Always return 0 here.
true
```

2.89 /opt/GSM/osmo-operator/iso_modules/99-fin.sh

1820 octets, 36 lignes → 36 lignes

```
#!/bin/bash
# iso_modules/99-fin.sh - hand the rootfs to the next pass, print the summary.
# Sourced by build-iso.sh in numeric order: same shell, same variables, same
# functions. Not meant to be run on its own. A `return` at the top of a module
# means "nothing to do here" (this is how --arm skips a step).
#
# Environment variables:
#   OSMO_ISO_ROOTFS_KEEP  path where the rootfs is kept for the next pass (--all)

# ── Keep the rootfs for the next pass if the parent asks (--all) ──────────
# Moved, not copied (same filesystem, instant). cleanup() then removes $WORK
# without it.
if [ -n "${OSMO_ISO_ROOTFS_KEEP:-}" ]; then
    umount "$ROOTFS/var/cache/apt/archives" 2>/dev/null || true
    umount "$ROOTFS"/{dev/pts,proc,sys,dev} 2>/dev/null || true
    rm -rf "$OSMO_ISO_ROOTFS_KEEP"
    mkdir -p "$(dirname "$OSMO_ISO_ROOTFS_KEEP")"
    mv "$ROOTFS" "$OSMO_ISO_ROOTFS_KEEP"
    echo -e "  ${GREEN}✓${NC} rootfs conserve pour la passe suivante : ${CYAN}${OSMO_ISO_ROOTFS_KEEP}${NC}"
fi

echo ""
if [ "${ISO_ARCH:-amd64}" = "arm64" ]; then
    echo -e "${GREEN}${BOLD}═══ Image SD Raspberry Pi 4 (Armbian ${ISO_UBUNTU}) prete : ${OUTPUT} ($(du -sh
     ↪  "$OUTPUT"|cut -f1)) ═══${NC}"
    echo -e "  Chemin absolu : $(readlink -f "$OUTPUT")"
    echo -e "  Gravure : ${CYAN}dd if=$(basename "$OUTPUT") of=/dev/sdX bs=4M status=progress conv=fsync${NC}"
    echo -e "  Au premier demarrage armbian-resize-filesystem etend la racine a la carte ; console HDMI/serie et
     ↪  ssh, root:osmo."
else
echo -e "${GREEN}${BOLD}═══ ISO prete : ${OUTPUT} ($(du -sh "$OUTPUT"|cut -f1)) ═══${NC}"
echo -e "  Chemin absolu : $(readlink -f "$OUTPUT")"
fi


# End of module: `. file` returns the status of its LAST command, and
# build-iso.sh runs under set -e. Always return 0 here.
true
```

2.90 /opt/GSM/osmo-operator/launch/osmo-launch.sh

12652 octets, 308 lignes → 308 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════════
# launch/osmo-launch.sh - osmo-operator orchestrator (lab + web dashboard).
# Installed in the ISO as /opt/osmo-launch.sh (symlink /usr/local/bin/osmo-launch).
#
# Usage:
#   sudo /opt/osmo-launch.sh           # start everything (interactive)
#   sudo /opt/osmo-launch.sh --auto    # start with defaults (2 operators, 8 MS)
#   sudo /opt/osmo-launch.sh stop      # stop everything (native + lab + web)
#   sudo /opt/osmo-launch.sh restart   # stop then start
#   sudo /opt/osmo-launch.sh status    # service status
```

```
#   sudo /opt/osmo-launch.sh web-only  # start the dashboard only
#
# Environment variables:
#   OSMO_BANC_SERVICE  systemd unit of the native operator (default osmo-banc.service)
# ═══════════════════════════════════════════════════════════════════════════
set -euo pipefail

LAB_DIR="/opt/GSM/osmo-operator"
WEB_DIR="/opt/GSM/osmo-egprs-web"
WEB_SERVICE="osmo-egprs-web"
LOG_DIR="/var/log/osmocom"

RED='\033[0;31m'; GREEN='\033[0;32m'; CYAN='\033[0;36m'
YELLOW='\033[1;33m'; NC='\033[0m'; BOLD='\033[1m'

[ "$(id -u)" -ne 0 ] && { echo -e "${RED}Root requis.${NC}"; exit 1; }

mkdir -p "$LOG_DIR"

# ═══════════════════════════════════════════════════════════════════════════
banner() {
    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════════════════════════╗"
    echo "║  osmo-operator - GSM/EGPRS Multi-PLMN + Web Dashboard         ║"
    echo "╚═══════════════════════════════════════════════════════════════╝"
    echo -e "${NC}"
}

# ═══════════════════════════════════════════════════════════════════════════
wait_docker() {
    echo -ne "  Docker daemon"
    local i=0
    while ! docker info &>/dev/null; do
        sleep 1; echo -n "."; i=$((i+1))
        [ $i -ge 30 ] && { echo -e " ${RED}TIMEOUT${NC}"; return 1; }
    done
    echo -e " ${GREEN}OK${NC}"
}

wait_operators() {
    local timeout="${1:-120}"
    echo -ne "  Containers operateurs"
    local i=0
    while [ "$(docker ps --filter 'name=osmo-operator-' --format '{{.Names}}' 2>/dev/null | wc -l)" -eq 0 ]; do
        sleep 2; echo -n "."; i=$((i+2))
        [ $i -ge "$timeout" ] && { echo -e " ${RED}TIMEOUT${NC}"; return 1; }
    done
    local n=$(docker ps --filter 'name=osmo-operator-' --format '{{.Names}}' | wc -l)
    echo -e " ${GREEN}${n} operateur(s) detectes${NC}"
}

# ═══════════════════════════════════════════════════════════════════════════
start_web() {
    echo -e "${GREEN}[web] Demarrage dashboard...${NC}"

    if [ -f "$WEB_DIR/server.js" ] && command -v node &>/dev/null; then
        # Native mode (local Node.js)
        systemctl restart "$WEB_SERVICE" 2>/dev/null && {
            echo -e "  ${GREEN}✓${NC} systemd: $WEB_SERVICE actif (port 8080)"
            return 0
        }
        # Fallback when the systemd service does not exist
        echo -e "  ${YELLOW}Service systemd absent, lancement direct...${NC}"
        cd "$WEB_DIR"
        node server.js --verbose >> "$LOG_DIR/web-dashboard.log" 2>&1 &
        echo $! > /var/run/osmo-egprs-web.pid
```

```
        echo -e "  ${GREEN}✓${NC} Dashboard lance (PID $(cat /var/run/osmo-egprs-web.pid))"

    elif [ -f "$WEB_DIR/Dockerfile" ] || [ -f "$WEB_DIR/start-web.sh" ]; then
        # Mode Docker
        if [ -f "$WEB_DIR/start-web.sh" ]; then
            bash "$WEB_DIR/start-web.sh"
        else
            docker rm -f osmo-egprs-web 2>/dev/null || true
            docker build -t osmo-egprs-web "$WEB_DIR"
            docker run -d \
                --name osmo-egprs-web \
                --network host \
                --cap-add=NET_RAW --cap-add=NET_ADMIN \
                -v /var/run/docker.sock:/var/run/docker.sock:ro \
                -e CONTAINER_PREFIX=osmo-operator- \
                --restart unless-stopped \
                osmo-egprs-web
        fi
        echo -e "  ${GREEN}✓${NC} Dashboard Docker lance (port 80)"
    else
        echo -e "  ${RED}✗${NC} Dashboard introuvable dans $WEB_DIR"
        return 1
    fi
}

stop_web() {
    echo -e "${YELLOW}[web] Arret dashboard...${NC}"
    systemctl stop "$WEB_SERVICE" 2>/dev/null || true
    [ -f /var/run/osmo-egprs-web.pid ] && {
        kill "$(cat /var/run/osmo-egprs-web.pid)" 2>/dev/null || true
        rm -f /var/run/osmo-egprs-web.pid
    }
    docker rm -f osmo-egprs-web 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} Dashboard arrete"
}

# ═══════════════════════════════════════════════════════════════════
start_lab() {
    echo -e "${GREEN}[lab] Demarrage osmo-operator...${NC}"

    if [ ! -f "$LAB_DIR/launch/start-docker.sh" ]; then
        echo -e "  ${RED}✗${NC} $LAB_DIR/launch/start-docker.sh introuvable"
        return 1
    fi

    cd "$LAB_DIR"
    bash ./launch/start-docker.sh
}

# ── Native operator ──────────────────────────────────────────────
# The native operator runs in osmo-banc.service (start-native.sh, tmux session
# "calypso") and holds ports 5060 and 8080; `stop` must stop it as well as the
# containers (same as launch.sh --stop). The unit delegates to
# start-native.sh --stop. Native side first, then containers (clean shutdown order).
stop_natif() {
    local u="${OSMO_BANC_SERVICE:-osmo-banc.service}"
    command -v systemctl >/dev/null 2>&1 || return 0
    [ -d /run/systemd/system ] || return 0
    systemctl cat "$u" >/dev/null 2>&1 || return 0
    echo -e "${YELLOW}[natif] Arret ${u}...${NC}"
    timeout 200 systemctl stop "$u" >/dev/null 2>&1 || true
    # ExecStop does not run on a "failed" unit: the core, started by run.sh in
    # its own units, would stay up.
    systemctl stop osmo-egprs-web.service osmo-hlr.service >/dev/null 2>&1 || true
    systemctl reset-failed "$u" >/dev/null 2>&1 || true
    echo -e "  ${GREEN}✓${NC} Natif arrete"
```

```
}

stop_lab() {
    echo -e "${YELLOW}[lab] Arret osmo-operator...${NC}"
    if [ -f "$LAB_DIR/launch/start-docker.sh" ]; then
        cd "$LAB_DIR"
        bash ./launch/start-docker.sh stop
    fi
    # Full cleanup in case start-docker.sh stop is not enough
    docker ps -a --filter "name=osmo-" --format "{{.Names}}" | xargs -r docker rm -f 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} Lab arrete"
}

# ═══════════════════════════════════════════════════════════════
show_status() {
    echo -e "${CYAN}${BOLD}── Etat des services ──${NC}"
    echo ""

    # Docker
    if docker info &>/dev/null; then
        echo -e "  Docker       : ${GREEN}●${NC} actif"
    else
        echo -e "  Docker       : ${RED}●${NC} arrete"
    fi

    # Containers
    local ops=$(docker ps --filter 'name=osmo-operator-' --format '{{.Names}}' 2>/dev/null | sort)
    local n_ops=$(echo "$ops" | grep -c . 2>/dev/null || echo 0)
    if [ "$n_ops" -gt 0 ]; then
        echo -e "  Operateurs   : ${GREEN}●${NC} ${n_ops} actif(s)"
        echo "$ops" | while read -r c; do
            echo -e "                 ${CYAN}└─${NC} $c"
        done
    else
        echo -e "  Operateurs   : ${RED}●${NC} aucun"
    fi

    # Inter-STP
    if docker ps --format '{{.Names}}' 2>/dev/null | grep -q osmo-inter-stp; then
        echo -e "  Inter-STP    : ${GREEN}●${NC} actif"
    else
        echo -e "  Inter-STP    : ${RED}●${NC} arrete"
    fi

    # Web dashboard
    if systemctl is-active --quiet "$WEB_SERVICE" 2>/dev/null; then
        echo -e "  Dashboard    : ${GREEN}●${NC} systemd (port 8080)"
    elif [ -f /var/run/osmo-egprs-web.pid ] && kill -0 "$(cat /var/run/osmo-egprs-web.pid 2>/dev/null)"
     ↪  2>/dev/null; then
        echo -e "  Dashboard    : ${GREEN}●${NC} PID $(cat /var/run/osmo-egprs-web.pid) (port 8080)"
    elif docker ps --format '{{.Names}}' 2>/dev/null | grep -q osmo-egprs-web; then
        echo -e "  Dashboard    : ${GREEN}●${NC} Docker (port 80)"
    else
        echo -e "  Dashboard    : ${RED}●${NC} arrete"
    fi

    # IPs
    echo ""
    local host_ip=$(ip route get 1.1.1.1 2>/dev/null | awk '{for(i=1;i<=NF;i++) if($i=="src") print $(i+1)}' |
     ↪  head -1)
    [ -n "$host_ip" ] && echo -e "  Host IP      : ${CYAN}${host_ip}${NC}"
    echo -e "  Dashboard    : ${CYAN}http://${host_ip:-localhost}:8080${NC}"
    echo ""
}

# ═══════════════════════════════════════════════════════════════
```

```
do_start() {
    banner

    # 1. Docker
    echo -e "${GREEN}[1/3] Verification Docker...${NC}"
    systemctl start docker 2>/dev/null || true
    wait_docker || { echo -e "${RED}Docker necessaire.${NC}"; exit 1; }

    # 2. Lab
    echo ""
    echo -e "${GREEN}[2/3] Lab multi-operateurs...${NC}"
    start_lab

    # 3. Web (wait for the containers to be up)
    echo ""
    echo -e "${GREEN}[3/3] Dashboard web...${NC}"
    wait_operators 180 || true
    start_web || true

    # Summary
    echo ""
    show_status
    echo -e "${GREEN}${BOLD}Pret !${NC}"
}

do_auto() {
    banner
    echo -e "${YELLOW}Mode automatique (defauts)${NC}"
    echo ""

    systemctl start docker 2>/dev/null || true
    wait_docker || exit 1

    # Run start-docker.sh non-interactively by piping default answers to its prompts
    cd "$LAB_DIR"
    {
        echo "2"       # bridge mode
        echo "2"       # 2 operators
        echo "o"       # default values
        echo "8"       # 8 MS
        echo "N"       # no WAN
        echo "1"       # faketrx
        echo "0"       # A5/0
    } | bash ./launch/start-docker.sh

    wait_operators 180 || true
    start_web || true

    echo ""
    show_status
}

# ═══════════════════════════════════════════════════════════════════
# Main
# ═══════════════════════════════════════════════════════════════════
CMD="${1:-}"

case "$CMD" in
    stop)
        banner
        stop_web
        stop_natif
        stop_lab
        echo -e "\n${GREEN}Tout arrete.${NC}"
        ;;
    status)
```

```
        banner
        show_status
        ;;
    web-only)
        banner
        systemctl start docker 2>/dev/null || true
        wait_docker || exit 1
        start_web
        ;;
    --auto)
        do_auto
        ;;
    restart)
        banner
        stop_web
        stop_natif
        stop_lab
        sleep 3
        do_start
        ;;
    help|-h|--help)
        echo "Usage: $0 [commande]"
        echo ""
        echo "  (rien)    Lancement interactif (lab + web)"
        echo "  --auto    Lancement automatique (2 ops, 8 MS, defauts)"
        echo "  stop      Arrete tout (natif + lab + web + containers)"
        echo "  restart   Redemarre tout"
        echo "  status    Etat des services"
        echo "  web-only  Lance uniquement le dashboard web"
        echo "  help      Ce message"
        ;;
    *)
        do_start
        ;;
esac
```

2.91 /opt/GSM/osmo-operator/launch/start-docker.sh

132051 octets, 2418 lignes → 2418 lignes

```
#!/bin/bash
# start-docker.sh - host launcher for the multi-operator Osmocom GSM stack (Docker)
#
# Usage: sudo ./launch/start-docker.sh [quick|normal] [WAN options] [qemu|virtual|hw|stop]
#   qemu     PoC: one operator, QEMU Calypso radio
#   virtual  N operators in containers, linked by SS7 over a docker bridge
#   hw       net-host mode: one operator, physical SDR
#   stop     remove all osmo-*/egprs containers and WAN iptables chains
#   quick    build the run image with the docker cache (default)
#   normal   rebuild the run image with --no-cache
# WAN options (--wan, --wan-nodes, --hub-ip, --operator, ...): see --help.
#
# Environment:
#   OSMO_REPO_DIR        repository root (when this file is sourced from a copy)
#   IMAGE_RUN/IMAGE_STP  images for operators / SS7 hub
#   OSMO_QUICK=0         force a --no-cache rebuild;  OSMO_NO_BUILD=1 skip the build
#   OSMO_NONINTERACTIVE=1  answer every whiptail dialog with its default
#   OP_ID_BASE=N         first operator number used by containers (lower ones are native)
#   OPERATOR_COUNT_HINT  number of operators (same as --operators)
#   HANDOFF_MODE         radio mode handed to start-native.sh (qemu | faketrx-qemu)
#   OSMO_NO_ATTACH=1, OSMO_NO_TERM=1, OSMO_SKIP_CHECKS=1, BRIDGE_NO_PROCESS=1,
#   OSMO_DOCKER_ROUTED=1, OSMO_PORT_OFFSET=N  (see the corresponding sections)
set -eu
# Repository root. OSMO_REPO_DIR first: build-iso.sh (iso_modules/30-image-configs.sh)
```

```
# sources a COPY of this file from $WORK, where BASH_SOURCE would point to /var/tmp.
START_REPO="${OSMO_REPO_DIR:-$(cd "$(dirname "$(readlink -f "${BASH_SOURCE[0]}")")/.." && pwd)}"

# ── This is the HOST launcher: refuse to run inside the container ──
# start-docker.sh drives docker (build, networks, docker run, volumes); start-native.sh
# runs INSIDE the container. Stop before touching anything and name the right script.
# Both /.dockerenv and /etc/docker-entrypoint-cmd (created by scripts/entrypoint.sh)
# are required: /.dockerenv alone is true in any container.
if [ -f /.dockerenv ] && [ -f /etc/docker-entrypoint-cmd ]; then
    printf '\033[1;31mVous etes dans le docker ! Utilisez start-native.sh\033[0m\n' >&2
    printf '\n  \033[0;36m/opt/GSM/osmo-operator/start-native.sh\033[0m   (ou ./start-native.sh depuis le
     ↪  depot)\n' >&2
    printf '\n  start-docker.sh est le lanceur de l'"'"'HOTE : il construit l'"'"'image, cree les\n' >&2
    printf '  reseaux et fait le "docker run". Rien de tout ca n'"'"'a de sens ici.\n' >&2
    exit 1
fi

DEBUG=
if [[ -n "$DEBUG" ]]; then
    set -x
    PS4='[DEBUG] + ${BASH_SOURCE}:${LINENO}: '
    echo "=== MODE DEBUG ACTIVE ==="
fi

IMAGE_BASE="osmocom-nitb"
# Overridable from the environment: start-nitb.sh re-runs this script with
# IMAGE_RUN=osmocom-nitb. Do not hard-code it.
IMAGE_RUN="${IMAGE_RUN:-osmocom-run}"

INTER_NET="gsm-inter"
INTER_NET_SUBNET="172.20.0.0/24"
INTER_NET_GATEWAY="172.20.0.1"

# WAN node hosting the SS7 hub. Exactly ONE node may do so: two hubs make two
# separate SS7 networks, silently.
WAN_HUB_NODE="${WAN_HUB_NODE:-1}"
# --hub-ip: the inter-STP is designated by its ADDRESS, not a node number, so a
# dedicated hub machine (no operator) does not need an entry in the WAN table.
WAN_HUB_IP="${WAN_HUB_IP:-}"
# --node-per-op: each container is a NODE, not an operator of this machine.
# Only use it when containers have externally reachable addresses (backbone routed
# to the LAN); otherwise other nodes cannot reach them.
WAN_NODE_PER_OP="${WAN_NODE_PER_OP:-0}"
# --operator IP:PREFIX: declare a WAN operator wherever it runs. Node numbers
# follow the declaration order (alternative to --wan-nodes "id:ip:ind").
OPERATOR_DECLS=()
# Inherited from the environment; --operators N (parsed later) takes precedence.
OPERATOR_COUNT_HINT="${OPERATOR_COUNT_HINT:-}"
# --host: address of THIS machine as seen by the VMs (their route to the backbone).
# Derived from the default interface; set it when the guess is wrong (several
# interfaces, address just changed).
HOST_LAN_IP=""
IMAGE_STP="${IMAGE_STP:-osmocom-stp}"
INTER_STP_CONTAINER="osmo-inter-stp"
INTER_STP_IP="172.20.0.10"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; NC='\033[0m'; BOLD='\033[1m'

# ── SMS Routing ──────────────────────────────────────────────────────
SMS_ROUTING_SCRIPT="$START_REPO/scripts/sms-routing-setup.sh"
if [ -f "$SMS_ROUTING_SCRIPT" ]; then
    source "$SMS_ROUTING_SCRIPT"
fi
SMS_ROUTING_DIR=""
```

```
# ═══════════════════════════════════════════════════════════════════════
# WAN Interop
# ═══════════════════════════════════════════════════════════════════════
# Two mutually exclusive WAN mechanisms, neither enabled by default:
#
#   WAN_ENABLED (legacy, no option, whiptail question): TWO servers, single
#     prefix 66, network/setup-wan-interop.sh.
#
#   WAN_MESH (--wan): N nodes (1 to 9), ONE PREFIX PER NODE,
#     network/setup-wan-mesh.sh. Use it beyond two servers: the legacy script
#     rewrites its config block on each call, so only the last peer survives.
WAN_MESH=0
# --virtualbox: the WAN peers are VirtualBox VMs on THIS machine, which is itself a
# node. See network/setup-vbox-interco.sh.
VBOX_INTERCO=0
VBOX_NODES=""
VBOX_HOST_NODE=1
BUILD_STP=0
# shellcheck source=network/wan-nodes.sh
. "$START_REPO/network/wan-nodes.sh"

WAN_ENABLED="false"
WAN_LOCAL_IP=""
WAN_REMOTE_IP=""
WAN_N_REMOTE=""
WAN_PREFIX="66"
WAN_SIP_BASE=5080
WAN_RTP_BASE=20000
WAN_RTP_PER_OP=500
PHY_MODE="faketrx"   # faketrx | virtphy
# Choices handed to start-native.sh INSIDE the container (NO_MENU=1), to avoid
# whiptail menus over docker exec -ti. Decided on the HOST.
# HANDOFF_MODE_FROM_ENV records whether the caller imposed the mode, so that --wan
# can switch to hybrid without overriding an explicit HANDOFF_MODE=...
HANDOFF_MODE_FROM_ENV="${HANDOFF_MODE:+1}"
HANDOFF_MODE="${HANDOFF_MODE:-qemu}"  # qemu | faketrx-qemu (combined)
HANDOFF_QEMU_CHOICE="${HANDOFF_QEMU_CHOICE:-full-grgsm}"

# ── Helpers ──────────────────────────────────────────────────────────────
# ── The two address plans of an operator ─────────────────────────────────
# backbone  172.20.0.<10+op>  segment shared by ALL operators and the inter-STP
#                             (.10). SS7 goes through it; network/setup-docker-lan-route.sh
#                             makes it reachable from the VM LAN.
# private   192.168.<op+1>.x  one segment per operator: its BTS, PCU, GGSN.
#                             Not reachable from outside.
#
# <op+1> because the lab LAN is 192.168.1.0/24 (VMs, SS7 hub at .49): operator 1
# on 192.168.1.0/24 would collide with it. Operators start at 192.168.2.0/24.
# ── First container rank ─────────────────────────────────────────────────
# OP_ID_BASE=2 starts containers at operator 2, leaving 1 to a NATIVE operator on
# the host (two equipments with the same point code would mean wrong routing).
# Everything derives from the index: container name (osmo-operator-N), backbone
# (172.20.0.<10+N>), point code (1.N.2), RCTX, private segment (192.168.<N+1>.0/24)
# and published ports.
OP_ID_BASE="${OP_ID_BASE:-1}"
op_backbone_ip()  { echo "172.20.0.$((10 + $1))"; }
# ── Private segment index: node or operator, depending on the host ────────
# 192.168.<index+1>.x, where index is:
#
#   DOCKER   a host carries N operator containers, each in its own netns; they
#            differ by OPERATOR number (the node is shared). index = operator.
#   VM/NATIVE the machine IS the node and carries one operator. index = node.
#
# The decision lives here once; lib/gabarits.sh and this script both inherit it.
_osmo_priv_index() {
    local op="${1:-1}"
```

```
    # The orchestrator is not inside the containers it creates: the detection below
    # would take the VM/native branch on the host and give every operator the same
    # segment (docker then refuses the overlapping network). start_bridge_mode sets
    # OSMO_PRIV_BY_OP=1 to mean "provisioning per operator".
    [ "${OSMO_PRIV_BY_OP:-0}" = "1" ] && { printf '%s' "$op"; return; }
    # Same detection as start-native.sh: /.dockerenv + /etc/docker-entrypoint-cmd
    # identifies a container of THIS repo; the cgroup is a fallback for any container.
    if [ -f /.dockerenv ] || grep -qa 'docker\|containerd' /proc/1/cgroup 2>/dev/null; then
        printf '%s' "$op"; return
    fi
    local n="${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}"
    [ -n "$n" ] || n="$(awk -F= '/^OSMO_WAN_NODE=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' \
                        "${ROLE_FILE:-/etc/osmo-role}" 2>/dev/null)"
    [ -n "$n" ] || n="$(sed -n 's/^PLAN_NODE=//p' "${OSMOCOM_CFG:-/etc/osmocom}/radio-plan.env" 2>/dev/null | tail
     ↪  -1)"
    case "$n" in [1-9]) ;; *) n=1 ;; esac
    printf '%s' "$n"
}
op_private_ip()   { echo "192.168.$(($(_osmo_priv_index "$1") + 1)).10"; }
op_private_gw()   { echo "192.168.$(($(_osmo_priv_index "$1") + 1)).1"; }
op_private_net()  { echo "192.168.$(($(_osmo_priv_index "$1") + 1)).0/24"; }
op_container()    { echo "osmo-operator-$1"; }
op_rctx_msc()     { echo $(( $1 * 100 + 10 )); }
op_rctx_stp()     { echo $(( $1 * 100 + 20 )); }
op_rctx_bsc()     { echo $(( $1 * 100 + 30 )); }
op_rctx_inter()   { echo $(( $1 * 100 + 50 )); }

# ── PLMN: MCC is the COUNTRY, i.e. the NODE; MNC is the operator ──────────
# The node is what --node passes to start-native.sh. With --node-per-op each
# container is its own node (own country) and MNC is 01; otherwise containers are
# N operators of the same country.
op_plmn_node()  { if [ "${WAN_NODE_PER_OP:-0}" = "1" ]; then echo $(( ${WAN_NODE_ID:-1} + $1 - 1 )); else echo
 ↪  "${WAN_NODE_ID:-1}"; fi; }
op_plmn_index() { if [ "${WAN_NODE_PER_OP:-0}" = "1" ]; then echo 1; else echo "$1"; fi; }
# printf '%03d' of an empty/zero value gives "000", which osmo-msc rejects
# ("network country code 000") and then restart-loops. A zero node number is a
# bug, not an MCC: clamp it and warn once.
_plmn_borne() {  # $1 = value, $2 = floor, $3 = what
    case "$1" in
        ''|*[!0-9]*) echo "  ATTENTION : $3 illisible ('$1') - ramene a $2" >&2; echo "$2" ;;
       *) if [ "$1" -lt "$2" ]; then echo "  ATTENTION : $3 vaut $1 - ramene a $2" >&2; echo "$2"; else echo "$1";
         ↪  fi ;;
    esac
}
op_mcc()        { printf '%03d' "$(_plmn_borne "$(op_plmn_node "$1")" 1 "numero de noeud (MCC)")"; }
op_mnc()        { printf '%02d' "$(_plmn_borne "$(op_plmn_index "$1")" 1 "rang d'operateur (MNC)")"; }

# Names without spaces: osmo-msc's VTY "short name" takes a single word.
# ── Lab data sets: data/*.txt ─────────────────────────────────────────────
# Operator names, countries, people and addresses live in data/, one per line,
# editable without touching the code. They are SHUFFLED at load time (the random
# draw) and then fixed for the whole run.
OSMO_DATA_DIR="${OSMO_DATA_DIR:-$START_REPO/data}"

# $1 = name of the array to fill, $2 = file, rest = fallback values.
# The repo may be incomplete (an ISO does not always ship data/).
_load_pool() {
    local -n __out=$1
    local __file=$2; shift 2
    __out=()
    if [ -r "$__file" ]; then
        if command -v shuf >/dev/null 2>&1; then
            mapfile -t __out < <(grep -vE '^[[:space:]]*(#|$)' "$__file" | shuf)
        else
            mapfile -t __out < <(grep -vE '^[[:space:]]*(#|$)' "$__file")
        fi
```

```
    fi
    [ "${#__out[@]}" -gt 0 ] || __out=("$@")
}
declare -a POOL_OP POOL_PAYS POOL_NOMS POOL_ADR
_load_pool POOL_OP   "$OSMO_DATA_DIR/op.txt"        OsmoOP1 OsmoOP2 OsmoOP3
_load_pool POOL_PAYS "$OSMO_DATA_DIR/pays.txt"      Syldavie Bordurie Poldevie
_load_pool POOL_NOMS "$OSMO_DATA_DIR/noms.txt"      "Jean Dupont" "Marie Martin"
_load_pool POOL_ADR  "$OSMO_DATA_DIR/adresses.txt"  "rue des Ondes Courtes, Fontaine-sur-Onde"

op_default_name() { echo "${POOL_OP[$(( ($1 - 1) % ${#POOL_OP[@]} ))]}"; }
op_country()      { local n; n=$(op_plmn_node "$1"); echo "${POOL_PAYS[$(( (n - 1) % ${#POOL_PAYS[@]} ))]}"; }

# WAN helpers
wan_sip_port()    { echo $(( WAN_SIP_BASE + ($1 - 1) * 2 )); }
wan_rtp_start()   { echo $(( WAN_RTP_BASE + ($1 - 1) * WAN_RTP_PER_OP )); }
wan_rtp_end()     { echo $(( WAN_RTP_BASE + $1 * WAN_RTP_PER_OP - 1 )); }

wan_sms_port()    { echo $(( 7890 + $1 - 1 )); }
# A WAN is active (legacy OR mesh): "must the WAN ports be opened?".
wan_active()      { [ "$WAN_ENABLED" = "true" ] || [ "${WAN_MESH:-0}" = "1" ]; }

# Linphone helpers - SIP/RTP ports of the containers.
# No SIP/RTP port is published for containers: they sit on a routable bridge and
# are reached directly at their own address on the nominal port. These functions
# return the NOMINAL values (generate_configs.sh uses them to write configs).
# OSMO_PORT_OFFSET=N re-enables an offset if needed.
PORT_OFFSET="${OSMO_PORT_OFFSET:-0}"
# RTP follows the same rule: nominal range, 200 ports per operator.
linphone_sip_port()  { echo $(( 5060  + ${PORT_OFFSET:-0}      + ($1 - 1) )); }
linphone_rtp_start() { echo $(( 30000 + ${PORT_OFFSET:-0} * 10 + ($1 - 1) * 200 )); }
linphone_rtp_end()   { echo $(( 30000 + ${PORT_OFFSET:-0} * 10 + $1 * 200 - 1 )); }

# Global - detected before the operator loop
HOST_IP="127.0.0.1"
ALSA_OUTPUT="${ALSA_OUTPUT:-default}"
ALSA_INPUT="${ALSA_INPUT:-default}"
HOST_AUDIO_RELAY=""
WSLG_TCP_PORT="${WSLG_TCP_PORT:-4713}"

banner() {
    echo -e "${CYAN}"
    echo "╔═══════════════════════════════════════════════════════════╗"
    echo "║         Osmocom GSM/EGPRS Virtual Network                 ║"
    echo "║        Multi-Operator SS7 + osmo-gapk Audio               ║"
    echo "╚═══════════════════════════════════════════════════════════╝"
    echo -e "${NC}"
}

# ── Whiptail helpers ──────────────────────────────────────────────
WT_BACKTITLE="Osmocom GSM/EGPRS Virtual Network"
WT_WIDTH=72

check_whiptail() {
    if ! command -v whiptail >/dev/null 2>&1; then
        echo -e "${RED}whiptail introuvable.${NC} Installez-le :" >&2
        echo -e "  ${CYAN}apt-get install -y whiptail${NC}  (Debian/Ubuntu)" >&2
        echo -e "  ${CYAN}dnf install -y newt${NC}          (Fedora/RHEL)" >&2
        exit 1
    fi
}

# ── Non-interactive mode: dialogs answer themselves ──────────────
# Each wt_* opens a whiptail box and WAITS, which blocks when launched from a
# desktop icon, start-multi.sh or a script. In auto mode the default is used:
#   wt_input -> its default ($3), the pre-filled value;
#   wt_menu  -> the FIRST entry, where the cursor starts;
```

```
#   wt_yesno -> no, because boxes use --defaultno;
#   wt_msg   -> written to stderr instead of waiting for <Ok>.
# This is exactly what pressing Enter would give.
#
# Triggers: OSMO_NONINTERACTIVE=1 (set by start-multi.sh), or no TTY on stdin
# (icon, cron, pipe). On a terminal the questions are asked.
_wt_auto() { [ "${OSMO_NONINTERACTIVE:-0}" = "1" ] || [ ! -t 0 ]; }

wt_input() {
    if _wt_auto; then
        echo "[auto] $1 : ${3}" >&2
        printf '%s' "$3"; return 0
    fi
    whiptail --backtitle "$WT_BACKTITLE" --title "$1" \
        --inputbox "$2" 10 "$WT_WIDTH" "$3" 3>&1 1>&2 2>&3
}

wt_yesno() {
    if _wt_auto; then echo "[auto] $1 : non" >&2; return 1; fi
    whiptail --backtitle "$WT_BACKTITLE" --title "$1" \
        --defaultno --yesno "$2" 11 "$WT_WIDTH" 3>&1 1>&2 2>&3
}

wt_menu() {
    local title="$1" prompt="$2"; shift 2
    local nitems=$(( $# / 2 ))
    if _wt_auto; then
        echo "[auto] $title : ${1}" >&2
        printf '%s' "$1"; return 0
    fi
    whiptail --backtitle "$WT_BACKTITLE" --title "$title" \
        --menu "$prompt" 20 "$WT_WIDTH" "$nitems" "$@" 3>&1 1>&2 2>&3
}

wt_msg() {
    if _wt_auto; then echo "$1" >&2; return 0; fi
    whiptail --backtitle "$WT_BACKTITLE" --msgbox "$1" 10 "$WT_WIDTH"
}

# ── Host audio session: WHO and WHERE ──────────────────────────────
# The right user is found by the PRESENCE of their pulse socket (a live graphical
# session), not by name. Same logic as session_user() in wslg-audio-bridge.sh,
# extended to a scan of /run/user/*. The loopback script is network/loopback.sh.
session_pulse_user() {
    local u uid sock
    for u in "${HOST_PULSE_USER:-}" "${SUDO_USER:-}" "$(logname 2>/dev/null || true)" "${USER:-}"; do
        [ -n "$u" ] || continue
        uid="$(id -u "$u" 2>/dev/null)" || continue
        [ -S "/run/user/${uid}/pulse/native" ] && { echo "$u"; return 0; }
    done
    # Last resort: owner of the first pulse socket found. Covers `sudo -i`
    # (SUDO_USER lost) and sessions without logname (cron, service).
    for sock in /run/user/*/pulse/native; do
        [ -S "$sock" ] || continue
        uid="${sock#/run/user/}"; uid="${uid%%/*}"
        u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
        [ -n "$u" ] && { echo "$u"; return 0; }
    done
    return 1
}

# The loopback script lives in the repo, not in a $HOME.
find_loopback_script() {
    local d c
    d="$START_REPO"
    for c in "$d/network/loopback.sh" "$d/loopback.sh"; do
```

```
        [ -x "$c" ] && { echo "$c"; return 0; }
    done
    return 1
}

enable_user_loopback() {
    local target_user target_uid target_runtime loopback_script
    target_user="$(session_pulse_user)" || {
        echo -e "  ${YELLOW}[audio] aucune session PulseAudio utilisateur - loopback ignore${NC}"
        return 0
    }
    loopback_script="$(find_loopback_script)" || {
        echo -e "  ${YELLOW}[audio] network/loopback.sh introuvable - loopback ignore${NC}"
        return 0
    }
    target_uid="$(id -u "$target_user")"
    target_runtime="/run/user/${target_uid}"
    echo -e "${GREEN}=== [audio] Loopback user session ===${NC}"
    echo -e "  user    : ${CYAN}${target_user}${NC}"
    echo -e "  runtime : ${CYAN}${target_runtime}${NC}"
    echo -e "  script  : ${CYAN}${loopback_script}${NC}"
    sudo -u "$target_user" \
        XDG_RUNTIME_DIR="$target_runtime" \
        PULSE_SERVER="unix:${target_runtime}/pulse/native" \
        "$loopback_script" enable >/dev/null 2>&1 || {
            echo -e "  ${RED}[FAIL]${NC} enable loopback"
            return 1
        }
    echo -e "[ OK ]${NC} loopback active"
}

disable_user_loopback() {
    local target_user target_uid target_runtime loopback_script
    target_user="$(session_pulse_user)"       || return 0
    loopback_script="$(find_loopback_script)" || return 0
    target_uid="$(id -u "$target_user")"
    target_runtime="/run/user/${target_uid}"
    sudo -u "$target_user" \
        XDG_RUNTIME_DIR="$target_runtime" \
        PULSE_SERVER="unix:${target_runtime}/pulse/native" \
        "$loopback_script" disable || true
}

build_alsa_args() {
    local alsa_args=""
    local src_asound="$START_REPO/configs/asound.conf"
    local host_asound="/tmp/osmocom-alsa/asound.conf"
    mkdir -p /tmp/osmocom-alsa
    # Docker creates a DIRECTORY when the source of a -v bind does not exist; a
    # leftover directory would make the mount on /etc/asound.conf (a file) fail with
    # "not a directory". Remove it before the copy.
    if [ -d "$host_asound" ]; then
        rm -rf "$host_asound"
    fi
    if [ -d /dev/snd ]; then
        alsa_args="${alsa_args} --device /dev/snd"
        if getent group audio >/dev/null 2>&1; then
            alsa_args="${alsa_args} --group-add $(getent group audio | cut -d: -f3)"
        fi
    fi
    local has_pulse="true"
    # ── The pulse socket must be MOUNTED, not just named ──────────────────
    # PULSE_SERVER=unix:/run/pulse/native only works if the socket is mounted into
    # the container; otherwise gapk fails to init ALSA and calls are silent.
    # Mount the socket when it exists, else fall back to the TCP relay. The socket is
    # srwxrwxrwx and the unix module uses auth-anonymous=1, so the container's
```

```
    # osmocom user can connect.
    local _pa_sock=""
    for _s in /var/run/pulse/native /run/pulse/native "/run/user/$(id -u)/pulse/native"; do
        [ -S "$_s" ] && { _pa_sock="$_s"; break; }
    done
    if [ -n "$_pa_sock" ]; then
        alsa_args="${alsa_args} -v ${_pa_sock}:/run/pulse/native"
        alsa_args="${alsa_args} -e PULSE_SERVER=unix:/run/pulse/native"
    elif [ -n "${HOST_AUDIO_RELAY:-}" ]; then
        alsa_args="${alsa_args} -e PULSE_SERVER=${HOST_AUDIO_RELAY}"
    else
        # Neither socket nor relay: say so, otherwise a silent call is the only symptom.
        alsa_args="${alsa_args} -e PULSE_SERVER=tcp:${INTER_NET_GATEWAY:-172.20.0.1}:${WSLG_TCP_PORT:-4713}"
        echo -e "  ${YELLOW}Audio : aucun socket pulse trouve et relais non arme - repli TCP a l aveugle${NC}" >&2
    fi
    local asound_ok="false"
    if [ -f "$src_asound" ]; then
        if cp -f "$src_asound" "$host_asound" 2>/dev/null && [ -f "$host_asound" ]; then
            asound_ok="true"
        else
            echo -e "  ${YELLOW}Audio : copie de asound.conf vers ${host_asound} impossible${NC}" >&2
        fi
    fi
    if [ "$asound_ok" = "true" ]; then
        alsa_args="${alsa_args} -v ${host_asound}:/etc/asound.conf:rw"
        alsa_args="${alsa_args} -e ALSA_OUTPUT=gsm_out -e ALSA_INPUT=gsm_in"
        alsa_args="${alsa_args} -e ALSA_CARD=gsm_out -e GAPK_ALSA_DEV=gsm_out"
        if [ "$has_pulse" = "true" ]; then
            echo -e "  ${GREEN}Audio : PulseAudio + asound.conf${NC}" >&2
        else
            echo -e "  ${YELLOW}Audio : asound.conf sans PulseAudio${NC}" >&2
        fi
    else
        alsa_args="${alsa_args} -e ALSA_OUTPUT=default -e ALSA_INPUT=default"
        alsa_args="${alsa_args} -e ALSA_CARD=default -e GAPK_ALSA_DEV=default"
        echo -e "  ${YELLOW}Audio : configs/asound.conf absent, fallback default${NC}" >&2
    fi
    echo "$alsa_args"
}

# ── Host audio: expose the host pulse server over TCP ──────────────────
ensure_host_audio_relay() {
    [ -n "${HOST_AUDIO_RELAY_DONE:-}" ] && return 0
    HOST_AUDIO_RELAY_DONE=1
    local bridge="$START_REPO/scripts/wslg-audio-bridge.sh"
    [ -x "$bridge" ] || { echo -e "  ${YELLOW}[host-audio] $bridge introuvable${NC}" >&2; return 0; }
    if "$bridge" host-relay; then
        # Check that the port actually answers: `host-relay` returns 0 as soon as the
        # module load is REQUESTED, not when pulseaudio accepts connections. gapk
        # initialises its I/O only ONCE: missing that window means a silent call.
        local _p=0 _i
        for _i in $(seq 1 30); do
            if (exec 3<>"/dev/tcp/${INTER_NET_GATEWAY}/${WSLG_TCP_PORT:-4713}") 2>/dev/null; then
                exec 3>&- 2>/dev/null; _p=1; break
            fi
            sleep 1
        done
        if [ "$_p" = "1" ]; then
            HOST_AUDIO_RELAY="tcp:${INTER_NET_GATEWAY}:${WSLG_TCP_PORT:-4713}"
            echo -e "  ${GREEN}[host-audio] relai pret → ${HOST_AUDIO_RELAY}${NC}"
        else
            HOST_AUDIO_RELAY=""
            echo -e "  ${RED}[host-audio] relai charge mais ${INTER_NET_GATEWAY}:${WSLG_TCP_PORT:-4713} NE REPOND
             ↪  PAS apres 30 s${NC}" >&2
            echo -e "  ${YELLOW}             → l audio du conteneur sera muet. Verifier :${NC}" >&2
            echo -e "  ${YELLOW}               pactl list modules | grep -A2 native-protocol-tcp${NC}" >&2
```

```
        fi
    else
        HOST_AUDIO_RELAY=""
        echo -e "  ${YELLOW}[host-audio] relai non ouvert - fallback carte locale dans le conteneur${NC}"
    fi

    # Uplink (microphone). The relay above only covers downlink (gsm_audio.monitor
    # -> host speakers). The mic comes from the browser, which captures the host's
    # DEFAULT source, possibly a ghost input producing only zeros. host-mic picks the
    # source by signal, not by name (see host_mic()). Non fatal.
    "$bridge" host-mic || true
}

# ============================================================================
# Build
# ============================================================================
# QUICK by default, without asking: the docker cache is the everyday answer; a
# full Osmocom rebuild (~40 min) happens only on request:
#     ./launch/start-docker.sh normal ...        (positional argument)
#     OSMO_QUICK=0 ./launch/start-docker.sh ...  (environment variable)
QUICK="${OSMO_QUICK:-1}"
# Explicit even when not requested: this suppresses the build-mode dialog.
# Setting it to 0 would bring the question back on every non-interactive run.
QUICK_EXPLICIT=1

build_run_image() {
    # docker compose v2 (compose.yaml, service "run") when available - it carries
    # the build-args and image name - else plain docker build. Same result:
    # osmocom-run from Dockerfile.run, on top of osmocom-nitb.
    local _dir; _dir="${OSMO_REPO_DIR:-$START_REPO}"
    local _nc=""; [ "${QUICK:-0}" = "1" ] || _nc="--no-cache"
    if [ "${QUICK:-0}" = "1" ]; then
        echo -e "${GREEN}Build de l'image run...${NC} ${YELLOW}(quick - cache docker reutilise)${NC}"
    else
        echo -e "${GREEN}Build de l'image run...${NC} ${CYAN}(normal - --no-cache)${NC}"
    fi
    if docker compose version >/dev/null 2>&1 && [ -f "$_dir/compose.yaml" ]; then
        ( cd "$_dir" && QEMU_CACHE_BUST=$(date +%s) IMAGE_RUN="$IMAGE_RUN" \
          docker compose -f "$_dir/compose.yaml" build $_nc run )
    else
        docker build $_nc --build-arg QEMU_CACHE_BUST=$(date +%s) -f Dockerfile.run -t "$IMAGE_RUN" .
    fi
    echo -e "${GREEN}Image '$IMAGE_RUN' prete.${NC}"
}

check_image() {
    if ! docker image inspect "$IMAGE_RUN" &>/dev/null; then
        echo -e "${RED}Image '$IMAGE_RUN' introuvable - build en cours...${NC}"
        build_run_image
    fi
}

# ============================================================================
# Dynamic generation - pjsip interop trunks
# ============================================================================
generate_pjsip_interop_trunks() {
    local op_id=$1
    local n_operators=$2
    # Operators below OP_ID_BASE are NATIVE (on the host): include them so that
    # containers also get a trunk to them.
    for remote_op in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))) \
                     $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        [ "$remote_op" -eq "$op_id" ] && continue
        local remote_ip
        # A native operator runs on the host; seen from a container the host is the
        # docker bridge gateway (same address as the audio bridge and sms-routing.conf).
```

```
        # Note: match=172.20.0.1 does not capture Linphone REGISTERs arriving through the
        # same gateway, because pjsip.conf sets endpoint_identifier_order=username,ip,anonymous.
        if [ "$remote_op" -lt "${OP_ID_BASE:-1}" ]; then
            remote_ip="${INTER_NET_GATEWAY:-172.20.0.1}"
        else
            remote_ip=$(op_backbone_ip "$remote_op")
        fi
        cat <<EOF

[interop-identify-op${remote_op}]
type=identify
endpoint=interop_trunk_op${remote_op}
match=${remote_ip}

[interop_trunk_op${remote_op}]
type=endpoint
transport=transport-udp
context=interop_in
disallow=all
allow=gsm
allow=ulaw
aors=interop_trunk_op${remote_op}
direct_media=no
rtp_symmetric=yes
force_rport=yes
media_encryption=no

[interop_trunk_op${remote_op}]
type=aor
contact=sip:${remote_ip}:5060
qualify_frequency=15
qualify_timeout=5.0
EOF
    done
}

generate_extensions_interop_out() {
    local op_id=$1
    local n_operators=$2
    # The prefix carries the NODE: MSISDN = <node>00<operator><rank>.
    local _pfx; _pfx="$(osmo_msisdn_pfx "$(osmo_node_id)")"
    cat <<'EOF'
[interop_out]

EOF
    # A single pattern: MSISDNs have six digits, <node>00<operator><rank>.
    # Operators below OP_ID_BASE are native and are included as well.
    for remote_op in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))) \
                     $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        [ "$remote_op" -eq "$op_id" ] && continue
        cat <<EOF
exten => _${_pfx}${remote_op}XX,1,NoOp(=== INTEROP OUT Op${remote_op}: \${EXTEN} ===)
 same => n,Dial(PJSIP/\${EXTEN}@interop_trunk_op${remote_op},,rT)
 same => n,Congestion()
 same => n,Hangup()

EOF
    done

    # ── Softphones too, without prefix ──────────────────────────────────
    # Lets a Linphone in [internal] dial another operator's number directly instead
    # of going through the local MSC. _<pfx><op>XX is MORE SPECIFIC than _X., so
    # Asterisk picks it without touching the template's catch-all. Gosub(sub-record)
    # as for _9., so these calls are recorded too.
    printf '[internal]\n\n'
    for remote_op in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))) \
```

```
                    $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        [ "$remote_op" -eq "$op_id" ] && continue
        cat <<EOF
exten => _${_pfx}${remote_op}XX,1,NoOp(=== SIP -> INTER-OP Op${remote_op}: \${EXTEN} - CallerID:
 ↪  \${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Goto(interop_out,\${EXTEN},1)

EOF
    done
    # No _X. catch-all here: configs/extensions.conf already declares one in this
    # context; a duplicate would be rejected with warnings on every reload.
}

# ═══════════════════════════════════════════════════════════════════════
# SMS routing fallback
# ═══════════════════════════════════════════════════════════════════════
_generate_sms_routing_conf_fallback() {
    # 'local i' is required: this function is called (via apply_config_templates)
    # from the "for i in ..." loop that starts the operators; without local it
    # clobbers the caller's index and every following container gets the same ports.
    local i
    local op_id=$1 n_operators=$2
    printf '# sms-routing.conf - Fallback\n\n[local]\noperator_id = %s\nsc_address  =
     ↪  1999001%s444\n\n[operators]\n' "$op_id" "$op_id"
    # Operators below OP_ID_BASE are NATIVE (start-multi.sh sets OP_ID_BASE=2):
    # list them in [operators] and [routes], pointing to the docker bridge gateway,
    # which IS the host seen from a container (172.20.0.1).
    local _natif_gw="${INTER_NET_GATEWAY:-172.20.0.1}"
    for i in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))); do printf '%s = %s\n' "$i" "$_natif_gw"; done
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        printf '%s = %s\n' "$i" "$(op_backbone_ip "$i")"
    done
    printf '\n[routes]\n'
    for i in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))); do
        for ms in 1 2; do printf '%s = %s\n' "$(osmo_msisdn "$(osmo_node_id)" "$i" "$ms")" "$i"; done
    done
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        for ms in 1 2; do printf '%s = %s\n' "$(osmo_msisdn "$(osmo_node_id)" "$i" "$ms")" "$i"; done  # exact
         ↪  MSISDNs <node>00<op><ms> - NO concatenation
    done
    printf '\n[relay]\nport = 7890\nconnect_timeout = 10\nretry_count = 3\nretry_delay = 5\n'
}

# ═══════════════════════════════════════════════════════════════════════
# ─────────────────────────────────────────────────────────────────────
# Network configuration - delegated to ./generate_configs.sh
#
# generate_configs.sh creates globals.conf at the repo root if missing and never
# overwrites it afterwards. Reset to factory values: ./generate_configs.sh --force
# ─────────────────────────────────────────────────────────────────────
# OSMO_REPO_DIR lets the caller name the real repo root: build-iso.sh extracts
# this file as a library ($WORK/start.lib.sh) and sources it, so BASH_SOURCE would
# point to a copy in a work directory without generate_configs.sh.
_GC_SH="${OSMO_REPO_DIR:-$START_REPO}/generate_configs.sh"
"$_GC_SH" >/dev/null || true
. "$_GC_SH"


# ── Forced update of the three trees, BEFORE run.sh ──────────────────
# The repos are cloned and pulled in the Dockerfile, i.e. frozen at BUILD time.
# Refresh them at every start so a new container runs the current code.
#
# ORDER: call after `docker run` and BEFORE `docker exec ... run.sh`.
#
# `--ff-only`, NOT `reset --hard`: the container's qosmo repo may receive manual
```

```
# commits during a session; a divergence must stop and be visible, not be erased.
#
# qemu is REBUILT after the pull: updating the source without relinking changes
# nothing at runtime. To check a running binary: `stat -L /proc/<pid>/exe`.
#
# Non blocking (no network = continue) but LOUD: a failed pull must show up.
force_update_trees() {
    local c=$1
    echo -e "  ${GREEN}[*] Mise a jour forcee des depots (avant run.sh)...${NC}"
    for repo in /opt/GSM/qosmo /opt/GSM/osmo-operator /opt/GSM/osmo-egprs-web; do
        if ! docker exec "$c" test -d "$repo/.git" 2>/dev/null; then
            echo -e "    ${YELLOW}$repo : pas un depot git - ignore${NC}"
            continue
        fi
        local before after
        before=$(docker exec "$c" git -C "$repo" rev-parse --short HEAD 2>/dev/null || echo '?')
        if docker exec "$c" git -C "$repo" pull --ff-only 2>&1 | tail -2 | sed 's/^/    /'; then
            after=$(docker exec "$c" git -C "$repo" rev-parse --short HEAD 2>/dev/null || echo '?')
            if [ "$before" = "$after" ]; then
                echo -e "    ${CYAN}$repo${NC} : deja a jour ($after)"
            else
                echo -e "    ${GREEN}$repo${NC} : $before -> $after"
                # ── The web server already runs and node has already read its file ──
                # osmo-egprs-web.service starts at container boot, before this call, and node
                # reads server.js only once. Restart it ONLY if the pull actually changed something.
                if [ "$repo" = /opt/GSM/osmo-egprs-web ]; then
                    if docker exec "$c" systemctl restart osmo-egprs-web 2>/dev/null; then
                        echo -e "    ${GREEN}osmo-egprs-web.service relance${NC} (le dashboard charge le nouveau
                         ↪  code)"
                    else
                        echo -e "    ${YELLOW}osmo-egprs-web.service : relance KO - le dashboard garde l'ancien
                         ↪  code${NC}"
                    fi
                fi
            fi
        else
            echo -e "    ${RED}$repo : pull KO (divergence ou reseau) - le run part sur $before${NC}"
        fi
    done
    # Rebuild QEMU: without it the pull above does not change the running binary.
    # ninja only rebuilds what changed.
    echo -e "  ${GREEN}[*] Recompilation QEMU (ninja)...${NC}"
    if docker exec "$c" bash -c 'cd /opt/GSM/qosmo/build && ninja' >/dev/null 2>&1; then
        echo -e "    ${GREEN}qemu-system-arm relie${NC} ($(docker exec "$c" stat -L -c %y
         ↪  /opt/GSM/qosmo/build/qemu-system-arm 2>/dev/null | cut -c1-19))"
    else
        echo -e "    ${RED}ninja KO - le run utilisera le binaire de l'image${NC}"
        docker exec "$c" bash -c 'cd /opt/GSM/qosmo/build && ninja 2>&1 | tail -15' | sed 's/^/      /' || true
    fi
}

build_vol_args() {
    local tmpdir=$1
    local vol_args=""
    [ -d "$tmpdir/osmocom" ] && vol_args="$vol_args -v $tmpdir/osmocom:/etc/osmocom"
    for f in "$tmpdir/asterisk"/*.conf; do
        [ -f "$f" ] || continue
        vol_args="$vol_args -v $f:/etc/asterisk/$(basename "$f")"
    done
    if [ -f "$tmpdir/bb/mobile.cfg" ]; then
        vol_args="$vol_args -v $tmpdir/bb/mobile.cfg:/root/.osmocom/bb/mobile.cfg"
    fi
    if [ -f "$tmpdir/bb/mobile_group1.cfg" ]; then
        vol_args="$vol_args -v $tmpdir/bb/mobile_group1.cfg:/root/.osmocom/bb/mobile_group1.cfg"
    fi
    echo "$vol_args"
```

```
}

# ═══════════════════════════════════════════════════════════════════
# Host TUN
# ═══════════════════════════════════════════════════════════════════
prepare_host_tun() {
    echo -e "${GREEN}[*] Configuration TUN sur l'hote...${NC}"
    modprobe tun 2>/dev/null || true
    mkdir -p /dev/net
    if [ ! -c /dev/net/tun ]; then
        mknod /dev/net/tun c 10 200
        chmod 666 /dev/net/tun
    fi
    ip link del apn0 2>/dev/null || true
    ip tuntap add dev apn0 mode tun
    ip addr add 176.16.32.0/24 dev apn0
    ip link set apn0 up
}

# ═══════════════════════════════════════════════════════════════════
# Inter-STP
# ═══════════════════════════════════════════════════════════════════
start_inter_stp() {
    local n_operators=$1
    local tmpdir
    tmpdir=$(mktemp -d)
    local inter_cfg="${tmpdir}/osmo-stp-interop.cfg"
    if [ "${WAN_MESH:-0}" = "1" ]; then
        # WAN plan: one AS per (node, operator) pair, point codes 1.<node><op>.<role>.
        # The hub must know ALL nodes, not only local operators; otherwise remote ASPs
        # attach to a non-existent AS and are rejected without explanation.
        echo -e "${GREEN}Generation config inter-STP WAN (${WAN_NODE_COUNT} noeuds × ${n_operators})...${NC}"
        bash ./helpers/create_interop.sh --wan "$WAN_NODE_COUNT" "$n_operators" "$inter_cfg" > /dev/null
    else
        echo -e "${GREEN}Generation config inter-STP (${n_operators} operateurs)...${NC}"
        bash ./helpers/create_interop.sh "$n_operators" "$inter_cfg" > /dev/null
    fi
    if [ ! -f "$inter_cfg" ]; then
        echo -e "${RED}Echec generation config inter-STP${NC}"; exit 1
    fi
    # The hub only routes M3UA: the osmocom-stp image (Dockerfile.stp) is enough and
    # much smaller than osmocom-run. It is not built here behind the caller's back:
    # if missing, the full image (which also contains osmo-stp) is used.
    local stp_image="$IMAGE_RUN"
    if [ "${BUILD_STP:-0}" = "1" ] && ! docker image inspect "$IMAGE_STP" >/dev/null 2>&1; then
        echo -e "  ${GREEN}Construction de ${IMAGE_STP} (Dockerfile.stp)...${NC}"
        docker build -f "$START_REPO/Dockerfile.stp" -t "$IMAGE_STP" "$START_REPO" \
            || echo -e "  ${YELLOW}⚠ echec - on restera sur ${IMAGE_RUN}${NC}"
    fi
    if docker image inspect "$IMAGE_STP" >/dev/null 2>&1; then
        stp_image="$IMAGE_STP"
        echo -e "  ${CYAN}image ${IMAGE_STP} (STP seul)${NC}"
    elif [ "${WAN_MESH:-0}" = "1" ]; then
        echo -e "  ${YELLOW}image ${IMAGE_STP} absente - on utilise ${IMAGE_RUN}.${NC}"
        echo -e "  ${CYAN}Pour la construire : docker build -f Dockerfile.stp -t ${IMAGE_STP} .${NC}"
    fi

    echo -e "${GREEN}Lancement inter-STP @ ${INTER_STP_IP}:2908 (PC 0.0.0)...${NC}"
    docker rm -f "$INTER_STP_CONTAINER" &>/dev/null || true
    # --restart, NOT --rm: with --rm a host reboot removes the hub instead of leaving
    # it stopped. The two are mutually exclusive in docker, and the "docker rm -f"
    # above already removes any leftover.
    # ── 2908 published on loopback: attachment point of the NATIVE operator ──
    # The native operator's osmo-stp.cfg declares `asp asp-to-inter 2908 2910 m3ua`
    # with remote-ip 127.0.0.1; the hub lives in a container, so the port must be
    # published on the host.
```

```
    # /sctp, NOT /tcp: M3UA runs over SCTP (docker supports publishing it).
    # Bound to 127.0.0.1, not 0.0.0.0: only the local native operator uses it; exposing
    # it would open the SS7 core to the LAN (checks/wan_ss7_check.sh flags a public
    # M3UA port as a defect). Pointing the native ASP at 172.20.0.10 instead is not
    # viable: that address only exists while the docker bridge is up.
    docker run -d \
        --restart unless-stopped \
        --name "$INTER_STP_CONTAINER" \
        --hostname "$INTER_STP_CONTAINER" \
        --network "$INTER_NET" \
        --ip "$INTER_STP_IP" \
        -p "127.0.0.1:2908:2908/sctp" \
        --cap-add NET_ADMIN \
        -v "${inter_cfg}:/etc/osmocom/osmo-stp-interop.cfg" \
        --entrypoint bash \
        "$stp_image" \
        -c "sleep infinity" > /dev/null
    # tmux if the image has it, otherwise a plain detached process.
    # osmocom-stp is the minimal "STP only" image and has no tmux. The barrier below
    # only reads /tmp/osmo-stp.log, which the detached process also writes.
    if docker exec "$INTER_STP_CONTAINER" sh -c 'command -v tmux >/dev/null 2>&1'; then
        docker exec "$INTER_STP_CONTAINER" \
            tmux new-session -d -s stp \
            "osmo-stp -c /etc/osmocom/osmo-stp-interop.cfg 2>&1 | tee /tmp/osmo-stp.log"
    else
        docker exec -d "$INTER_STP_CONTAINER" \
            sh -c 'exec osmo-stp -c /etc/osmocom/osmo-stp-interop.cfg > /tmp/osmo-stp.log 2>&1'
    fi
    echo -ne "${GREEN}[*] Attente demarrage inter-STP"
    local retries=25
    while [ $retries -gt 0 ]; do
        if docker exec "$INTER_STP_CONTAINER" \
            grep -qE "listening|m3ua.*[Ss]erver|bound" /tmp/osmo-stp.log 2>/dev/null; then
            echo -e " ${GREEN}✓${NC}"
            return 0
        fi
        if [ "$(docker inspect -f '{{.State.Running}}' "$INTER_STP_CONTAINER" 2>/dev/null)" != "true" ]; then
            echo -e " ${RED}CRASH${NC}"
            docker logs "$INTER_STP_CONTAINER"
            exit 1
        fi
        echo -n "."
        sleep 1
        ((retries--)) || true
    done
    echo -e " ${YELLOW}(timeout log, on continue)${NC}"
}

wait_inter_stp_ready() {
    local n_operators=$1
    echo -ne "${GREEN}[*] Verification stabilisation inter-STP (routes SS7)${NC}"
    for i in {1..15}; do sleep 1; echo -n "."; done
    echo -e " ${GREEN}✓${NC}"
    return 0
}

wait_bb_vty() {
    local container="$1"
    local timeout=120
    local elapsed=0
    echo -ne "  Attente BB VTY 4247 "
    while ! docker exec "$container" bash -c "echo >/dev/tcp/127.0.0.1/4247" 2>/dev/null; do
        sleep 2; elapsed=$((elapsed + 2)); echo -n "."
        if [ "$elapsed" -ge "$timeout" ]; then echo -e " ${RED}TIMEOUT${NC}"; return 1; fi
    done
    echo -e " ${GREEN}OK${NC}"
```

```
}

wait_stp_vty() {
    local container="$1"
    local timeout=60
    local elapsed=0
    echo -ne "  Attente STP VTY 4239 "
    while ! docker exec "$container" bash -c "echo >/dev/tcp/127.0.0.1/4239" 2>/dev/null; do
        sleep 2; elapsed=$((elapsed + 2)); echo -n "."
        if [ "$elapsed" -ge "$timeout" ]; then echo -e " ${RED}TIMEOUT${NC}"; return 1; fi
    done
    echo -e " ${GREEN}OK${NC}"
}

# ═══════════════════════════════════════════════════════════════════
# WAN Interop
# ═══════════════════════════════════════════════════════════════════
setup_wan_interop() {
    local n_local=$1
    local n_remote=$2
    local script_path
    script_path="$START_REPO/network/setup-wan-interop.sh"
    if [ ! -f "$script_path" ]; then
        echo -e "${RED}[WAN] network/setup-wan-interop.sh introuvable${NC}"
        return 1
    fi
    chmod +x "$script_path"
    bash "$script_path" "$WAN_LOCAL_IP" "$WAN_REMOTE_IP" "$n_local" "$n_remote"
}

# ═══════════════════════════════════════════════════════════════════
# WAN mesh (--wan)
# ═══════════════════════════════════════════════════════════════════
# Fill the node table: non-interactively if WAN_NODES is already set (--wan-nodes
# or environment variable), otherwise by asking.
wan_mesh_configure() {
    local n_operators=$1
    WAN_OPS="$n_operators"

    # The WAN point code is 1.<node><op>.<role>: the middle field of the ITU 3-8-3
    # format is 8 bits (max 255). With 12 operators, node 9 would give "912", which
    # osmo-stp rejects with an unhelpful parse error. Stop here, naming the cause.
    if [ "$n_operators" -gt 9 ]; then
        echo -e "${RED}[WAN] ${n_operators} operateurs : le plan de point codes WAN${NC}" >&2
        echo -e "${RED}      (1.<noeud><op>.<role>) n'en admet que 9 par noeud.${NC}" >&2
        echo -e "${CYAN}      Repartissez-les sur plusieurs noeuds, ou lancez sans --wan.${NC}" >&2
        exit 1
    fi

    # --virtualbox: segment, VMs and table are created BEFORE everything else, then
    # read back like any table. The WAN code does not need to know its peers are VMs.
    if [ "${VBOX_INTERCO:-0}" = "1" ] && [ -z "${WAN_NODES:-}" ]; then
        local _vb=("$START_REPO/network/setup-vbox-interco.sh"
                   --host-node "$VBOX_HOST_NODE" --conf "$WAN_CONF_FILE")
        [ -n "${VBOX_NODES:-}" ] && _vb+=(--nodes "$VBOX_NODES")
        bash "${_vb[@]}" || { echo -e "${RED}[WAN] interconnexion VirtualBox echouee${NC}" >&2; exit 1; }
        wan_nodes_load "$WAN_CONF_FILE" || exit 1
        WAN_NODES="$(wan_nodes_spec)"
        WAN_NODE_ID="$VBOX_HOST_NODE"
    fi

    wan_menu_table

    if [ -n "${WAN_NODES:-}" ]; then
        wan_nodes_parse "$WAN_NODES" || exit 1
        if [ "${WAN_NODE_ID:-0}" = "0" ]; then
```

```
            # IP-based detection cannot work when the first nodes are CONTAINERS: their
            # addresses (172.20.0.11, .12) belong to the docker backbone, not to the host.
            # The host carries nodes without being one. Its number is that of its FIRST
            # container, i.e. 1 ("container i = node i"); --wan-id is only asked if the
            # table does not even contain that node.
            if wan_nodes_detect_self 2>/dev/null; then
                :
            elif [ -n "${WAN_IP[1]:-}" ]; then
                WAN_NODE_ID=1
                echo -e "  ${CYAN}[WAN] cette machine porte les conteneurs : noeud de base 1${NC}"
            else
                echo -e "${RED}[WAN] table vide ou sans noeud 1 : passez --wan-id N${NC}" >&2
                exit 1
            fi
        fi
        wan_nodes_validate || exit 1
    else
        # A table already present on this machine provides the defaults: relaunching
        # the lab proposes the existing values.
        wan_nodes_load "$WAN_CONF_FILE" 2>/dev/null || true
        wan_nodes_prompt || exit 1
    fi

    # ── What the WAN table does not say ──────────────────────────────
    # It gives nodes and addresses, but not where the SS7 hub is, nor whether this
    # machine's containers are its operators or separate nodes. Ask (or use the CLI
    # options) instead of falling back on defaults (local hub, containers = operators).
    wan_menu_complements

    WAN_ENABLED="false"  # excludes the legacy WAN
    WAN_LOCAL_IP="$(wan_local_ip)"
    echo ""
    wan_nodes_summary
    if [ -n "${WAN_HUB_IP:-}" ]; then
        echo -e "  inter-STP : ${CYAN}${WAN_HUB_IP}${NC}  (hors table : ce n'est pas un noeud)"
    else
        echo -e "  inter-STP : porte par le noeud ${CYAN}${WAN_HUB_NODE}${NC}"
    fi
    [ "${WAN_NODE_PER_OP:-0}" = "1" ] \
        && echo -e "  conteneurs: ${CYAN}un noeud chacun${NC}, a partir du noeud ${CYAN}${WAN_NODE_ID}${NC}"
    # Who runs HERE and who is only DECLARED: a declared node must already run
    # elsewhere, otherwise its subscribers appear in SMS tables and the dialplan with
    # nobody answering.
    for _nid in ${WAN_NODE_LIST[@]+"${WAN_NODE_LIST[@]}"}; do
        if [ "$_nid" = "${WAN_NODE_ID:-0}" ] || \
           { [ "${WAN_NODE_PER_OP:-0}" = "1" ] && [ "$_nid" -gt "${WAN_NODE_ID:-0}" ]; }; then
            printf '    noeud %-2s %-16s %s\n' "$_nid" "${WAN_IP[$_nid]}" "conteneur lance ici"
        else
            printf '    noeud %-2s %-16s %s\n' "$_nid" "${WAN_IP[$_nid]}" "declare (doit tourner ailleurs)"
        fi
    done
    echo ""
}

# ── Building the WAN table ──────────────────────────────────────────
# First the TOTAL number of operators, then how many run HERE as containers, then
# the others one by one. Containers take the first ranks (osmo-operator-1 is node
# 1, -2 node 2), so the container name gives its node number. Remote operators
# (an already running VM) take the next ranks: only their address and prefix are
# asked.
wan_menu_table() {
    command -v whiptail >/dev/null 2>&1 || return 0
    [ -t 0 ] || return 0
    [ -n "${WAN_NODES:-}" ] && return 0  # already given on the CLI
    [ "${#OPERATOR_DECLS[@]}" -gt 0 ] && return 0  # already declared with --operator
```

```
    local total cont k ip ind spec=""
    total=$(wt_input "WAN" "Nombre d'operateurs du WAN, tous sites confondus (1-9) :" "3") || return 0
    [[ "$total" =~ ^[1-9]$ ]] || { wt_msg "Nombre invalide : $total"; return 0; }

    cont=$(wt_input "WAN" "Combien tournent ICI, en conteneur ? (0-${total})" "$(( total > 1 ? total - 1 : total
    ↪  ))") || return 0
    [[ "$cont" =~ ^[0-9]$ ]] && [ "$cont" -le "$total" ] \
        || { wt_msg "Entre 0 et ${total}."; return 0; }

    # Containers: backbone address, prefix from the rank. Nothing to ask.
    for k in $(seq 1 "$cont"); do
        spec="${spec}${spec:+ }${k}:$(op_backbone_ip "$k"):$(( k * 11 ))"
    done
    # The others: only they need an answer.
    for k in $(seq $(( cont + 1 )) "$total"); do
        # Default: the lab VM, the only remote node of the current setup.
        _defip=""; [ "$k" = 3 ] && _defip="192.168.1.123"
        ip=$(wt_input "Operateur distant ${k}/${total}" \
             "Adresse du noeud ${k} (une VM, une autre machine) :" "$_defip") || return 0
        [ -n "$ip" ] || { wt_msg "Adresse vide : noeud ${k} ignore."; continue; }
        ind=$(wt_input "Operateur distant ${k}/${total}" \
              "Indicatif (prefixe d'appel) du noeud ${k} :" "$(( k * 11 ))") || return 0
        spec="${spec}${spec:+ }${k}:${ip}:${ind:-$(( k * 11 ))}"
    done

    WAN_NODES="$spec"
    OPERATOR_COUNT_HINT="$cont"
    WAN_NODE_ID="${WAN_NODE_ID:-1}"
    [ "$cont" -ge 1 ] && WAN_NODE_PER_OP=1 && WAN_NODE_PER_OP_GIVEN=1
    echo -e "  ${CYAN}Table WAN :${NC} $WAN_NODES"
}

# ── WAN questions not covered by the table ──────────────────────────
# Each is skipped when the matching CLI option was given: scripted runs stay
# silent, manual runs are guided. Without a terminal (CI, cron) defaults are kept.
wan_menu_complements() {
    command -v whiptail >/dev/null 2>&1 || return 0
    [ -t 0 ] || return 0

    if [ -z "${WAN_HUB_IP:-}" ] && [ "${WAN_HUB_NODE_GIVEN:-0}" != "1" ]; then
        local _c
        _c=$(wt_menu "Inter-STP" "Ou se trouve le hub SS7 (PC 0.0.0) ?" \
            "ip"    "sur une machine dediee, hors table (une VM par exemple)" \
            "ici"   "sur CETTE machine - un conteneur inter-STP sera lance" \
            "noeud" "sur un autre noeud de la table WAN") || return 0
        case "$_c" in
            ip)
                local _ip
                _ip=$(wt_input "Inter-STP" "Adresse du hub (ex: 192.168.1.49) :" "${WAN_HUB_IP:-}") || return 0
                if [ -n "$_ip" ]; then WAN_HUB_IP="$_ip"
                else wt_msg "Adresse vide : le hub restera local."; fi ;;
            noeud)
                local _n
                _n=$(wt_input "Inter-STP" "Numero du noeud qui porte le hub :" "${WAN_HUB_NODE:-1}") || return 0
                [ -n "$_n" ] && WAN_HUB_NODE="$_n" ;;
            *)  WAN_HUB_NODE="$WAN_NODE_ID" ;;
        esac
    fi

    if [ "${WAN_NODE_PER_OP_GIVEN:-0}" != "1" ]; then
        if wt_yesno "Conteneurs" \
"Chaque conteneur doit-il etre un NOEUD a part entiere ?

Oui : conteneur 1 -> noeud ${WAN_NODE_ID}, conteneur 2 -> noeud $((WAN_NODE_ID + 1))...
      (leurs adresses backbone doivent etre joignables des autres noeuds)
Non : ce sont les operateurs du noeud ${WAN_NODE_ID}."; then
```

```
            WAN_NODE_PER_OP=1
        fi
    fi
}

# Apply the table (Asterisk, iptables, SMS) once the containers are up.
wan_mesh_apply() {
    local n_operators=$1
    local script_path="$START_REPO/network/setup-wan-mesh.sh"
    if [ ! -f "$script_path" ]; then
        echo -e "${RED}[WAN] network/setup-wan-mesh.sh introuvable${NC}"; return 1
    fi
    # ── 4th table field: number of operators per node (WAN_NOPS) ──────────────
    # Written here, where the information is certain: with --op-is-node each node IS
    # a container and carries one operator; otherwise our node carries them all.
    # wan_nodes_spec writes the field only when it differs from 1, so a container
    # table keeps the 3-field form other readers (build-iso.sh, set_stp_ip.sh,
    # diagnostics) expect.
    local _k
    if [ "${WAN_NODE_PER_OP:-0}" = "1" ]; then
        for _k in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
            WAN_NOPS[$(( ${WAN_NODE_ID:-1} + _k - 1 ))]=1
        done
    else
        WAN_NOPS[${WAN_NODE_ID:-1}]="$n_operators"
    fi

    if [ "${WAN_NODE_PER_OP:-0}" = "1" ]; then
        # A SINGLE pass with --op-is-node: the script maps operator i to node i and
        # computes ITS peers. One pass per node would write into ALL containers each
        # time, the last one winning.
        bash "$script_path" --docker --op-is-node \
            --nodes "$(wan_nodes_spec)" --id 1 --ops "$n_operators" \
            || echo -e "${RED}[WAN] setup-wan-mesh.sh a echoue${NC}"
    else
        bash "$script_path" --docker \
            --nodes "$(wan_nodes_spec)" --id "$WAN_NODE_ID" --ops "$n_operators" \
            || echo -e "${RED}[WAN] setup-wan-mesh.sh a echoue${NC}"
    fi

}

# ── Preserving the source address of outgoing traffic ─────────────────────
# Docker masquerades everything leaving a bridge: seen from the LAN, all container
# associations come from the host. This does NOT prevent an ASP from joining its
# AS: the inter-STP uses "accept-asp-connections dynamic-permitted"
# (helpers/create_interop.sh) and matches by ROUTING CONTEXT, not source address.
# network/setup-docker-lan-route.sh manages the related POSTROUTING RETURN rule.
#
# Replayed here because docker RE-INSERTS its MASQUERADE rules at the head for
# each network created (one per operator), which would push a RETURN rule behind
# them while "iptables -L" still shows it.
reassert_docker_lan_return() {
    command -v iptables >/dev/null 2>&1 || return 0
    [ "$(id -u)" -eq 0 ] || return 0

    local lan_ip lan_net
    lan_ip="${HOST_LAN_IP:-}"
    [ -n "$lan_ip" ] || lan_ip="$(ip route show default 2>/dev/null | awk '/^default/{print $9; exit}')"
    [ -n "$lan_ip" ] || lan_ip="$(hostname -I 2>/dev/null | awk '{print $1}')"
    [ -n "$lan_ip" ] || return 0
    lan_net="${lan_ip%.*}.0/24"

    # If LAN and backbone overlap, the rule would hit bridge-internal traffic:
    # do nothing rather than guess.
    [ "${lan_net%%/*}" = "${INTER_NET_SUBNET%%/*}" ] && return 0
```

```
    # ── The RETURN rule is REMOVED, not added ──────────────────────────────
    # With source addresses preserved, the hub's SCTP INIT ACK is addressed to
    # 172.20.0.x, which the host must route and does not receive back: ASPs stay
    # down ("connect failed (-110)"). Masqueraded to the host's own address, the
    # reply comes back and conntrack hands it to the container.
    # Nothing is lost: the inter-STP distinguishes nodes by ROUTING CONTEXT (1150,
    # 2150, 3150), not by source address.
    # Remove the rule if a previous run left it.
    iptables -t nat -D POSTROUTING -s "$INTER_NET_SUBNET" -d "$lan_net" -j RETURN 2>/dev/null \
        && echo -e "  ${CYAN}[WAN] regle RETURN retiree : les noeuds sortent masques (l'inter-STP les apparie par
        ↪  routing context)${NC}" \
        || true

    # ── FORWARD, without which routing is useless ──────────────────────────
    # Docker sets a default FORWARD DROP policy: a packet between the LAN and the
    # backbone is silently dropped. DOCKER-USER is evaluated BEFORE docker's rules
    # and docker does not rewrite it. network/setup-docker-lan-route.sh sets the same
    # two rules, but nothing calls it automatically; they are set here, where the
    # function is replayed after each network creation.
    iptables -N DOCKER-USER 2>/dev/null || true
    local _spec
    for _spec in "-s ${lan_net} -d ${INTER_NET_SUBNET}" "-s ${INTER_NET_SUBNET} -d ${lan_net}"; do
        # shellcheck disable=SC2086
        if iptables -C DOCKER-USER $_spec -j ACCEPT 2>/dev/null; then
            continue
        fi
        # shellcheck disable=SC2086
        if iptables -I DOCKER-USER 1 $_spec -j ACCEPT 2>/dev/null; then
            echo -e "  ${CYAN}[WAN] FORWARD autorise (${_spec})${NC}"
        fi
    done

    # ── Already established translations ───────────────────────────────────
    # The rule only applies to new associations; masqueraded ones keep their
    # translation while they live. Flush those entries so the stacks reconnect.
    #
    # EXCEPT when docker 29 has set its DROP: its "ip raw" table drops, in
    # prerouting, any incoming packet to the backbone not arriving via the bridge.
    # An association restarting with its real source address would get no reply
    # (COOKIE_WAIT); flushing conntrack would cut ASPs that only work thanks to
    # masquerading.
    local _nft_drop=0
    if command -v nft >/dev/null 2>&1; then
        if nft list table ip raw 2>/dev/null \
           | grep -F "${INTER_NET_SUBNET%.*}." | grep -q 'drop'; then
            _nft_drop=1
        fi
    fi
    if [ "$_nft_drop" = "1" ]; then
        echo -e "  ${YELLOW}[WAN] DROP nft de docker actif sur ${INTER_NET_SUBNET} :${NC}"
        echo -e "  ${YELLOW}      purge conntrack SAUTEE (elle couperait les ASP attaches).${NC}"
        echo -e "  ${CYAN}      Backbone en mode routed pour le lever : OSMO_DOCKER_ROUTED=1${NC}"
    elif command -v conntrack >/dev/null 2>&1; then
        # LIMITED to the LAN destination. Without -d this would also flush
        # 172.20.0.x <-> 172.20.0.x flows, i.e. all intra-backbone SS7.
        local _purged
        _purged="$(conntrack -D -s "$INTER_NET_SUBNET" -d "$lan_net" 2>/dev/null | grep -c . || true)"
        echo -e "  ${CYAN}[WAN] ${_purged} traduction(s) heritee(s) effacee(s) vers ${lan_net}${NC}"
    else
        # The lab host case: report explicitly that translations were NOT flushed.
        echo -e "  ${YELLOW}[WAN] conntrack absent : les associations deja masquees gardent${NC}"
        echo -e "  ${YELLOW}      leur traduction jusqu'a leur reconnexion (apt install conntrack)${NC}"
    fi
}
```

```
# ═══════════════════════════════════════════════════════════════════════
# QEMU PoC - default entry of the first menu
# ═══════════════════════════════════════════════════════════════════════
start_qemu_poc() {
    echo -e "${GREEN}PoC QEMU Calypso - bring-up minimal + qosmo-grgsm/run.sh...${NC}"
    QEMU_POC=1
    start_bridge_mode
}

# ═══════════════════════════════════════════════════════════════════════
# Virtual mode (multi-operator SS7, formerly "bridge")
# ═══════════════════════════════════════════════════════════════════════
start_bridge_mode() {
    # This mode CREATES the containers: the private segment derives from the
    # operator, not the node (see _osmo_priv_index).
    export OSMO_PRIV_BY_OP=1
    local n_operators
    declare -A OP_MCC OP_MNC OP_NAME OP_MS

    if [ "${QEMU_POC:-0}" = "1" ]; then
        # ── PoC QEMU ──────────────────────────────────────────────────
        n_operators=1
        # Same base as the loops: otherwise OP_ID_BASE=2 would look up OP_MCC[2]
        # while everything is set at [1].
        _b="${OP_ID_BASE:-1}"
        OP_MCC[$_b]="001"; OP_MNC[$_b]="01"; OP_NAME[$_b]="OsmoQEMU"; OP_MS[$_b]=2
        # Single-operator PoC: reference PLMN, the one of the repo's IMSIs. Not
        # derived from the node: no WAN here.
        WAN_ENABLED="false"
        # --wan is still valid in the QEMU PoC: one node = one operator.
        [ "${WAN_MESH:-0}" = "1" ] && wan_mesh_configure 1
        # BRIDGE_NO_PROCESS=0: radio and mobile start with the core (see
        # "Containers start the full stack" below).
        PHY_MODE="faketrx"; BRIDGE_NO_PROCESS="${BRIDGE_NO_PROCESS:-0}"; BRIDGE_QEMU=1
        ENCRYPTION="${ENCRYPTION:-a5 1}"
        echo -e "  ${CYAN}[PoC QEMU] 1 operateur · no-process + qosmo/run.sh · ${ENCRYPTION}${NC}"
    else
        # ── Interactive mode ──────────────────────────────────────────
        # Already answered in the WAN menu ("how many run HERE"): not asked again, two
        # diverging answers would make the table and container count disagree.
        if [ -n "${OPERATOR_COUNT_HINT:-}" ]; then
            n_operators="$OPERATOR_COUNT_HINT"
            echo -e "  ${CYAN}Conteneurs a lancer : ${n_operators} (--operators / menu WAN)${NC}"
        else
            # Default 3: the lab size (two containers and one VM). A node missing from the
            # table does not exist for anyone.
            n_operators=$(wt_input "Operateurs" "Nombre d'operateurs (1-36) :" "3") || exit 1
        fi
        n_operators=${n_operators:-2}
        if ! [[ "$n_operators" =~ ^[0-9]+$ ]] || [ "$n_operators" -lt 1 ] || [ "$n_operators" -gt 36 ]; then
            wt_msg "Nombre invalide (1-36)."; exit 1
        fi

        # ── The node plan is decided BEFORE the PLMN ──────────────────
        # It answers "is each container a separate node?" (WAN_NODE_PER_OP), and the
        # MCC is the node number, so the WAN questions must come before the operator
        # menu. They only depend on the operator count, known above.
        # ── WAN ───────────────────────────────────────────────────────
        if [ "${WAN_MESH:-0}" = "1" ]; then
            wan_mesh_configure "$n_operators"
        elif wt_yesno "WAN Interop" "Activer WAN vers un serveur distant (2 serveurs, prefixe 66) ?"; then
            WAN_ENABLED="true"
            local auto_ip=""
            auto_ip=$(ip route get 1.1.1.1 2>/dev/null | awk '{for(i=1;i<=NF;i++) if($i=="src") print $(i+1)}' |
             ↪  head -1) || true
            [ -z "$auto_ip" ] && auto_ip=$(hostname -I 2>/dev/null | awk '{print $1}') || true
```

```
    local wan_local wan_remote wan_nremote wan_pfx
    wan_local=$(wt_input "WAN" "IP publique locale :" "$auto_ip") || exit 1
    WAN_LOCAL_IP="${wan_local:-$auto_ip}"
    [ -z "$WAN_LOCAL_IP" ] && WAN_ENABLED="false"
    if [ "$WAN_ENABLED" = "true" ]; then
        wan_remote=$(wt_input "WAN" "IP publique distante :" "") || exit 1
        WAN_REMOTE_IP="$wan_remote"
        [ -z "$WAN_REMOTE_IP" ] && WAN_ENABLED="false"
    fi
    if [ "$WAN_ENABLED" = "true" ]; then
        wan_nremote=$(wt_input "WAN" "Nb operateurs distants :" "$n_operators") || exit 1
        WAN_N_REMOTE="${wan_nremote:-$n_operators}"
        wan_pfx=$(wt_input "WAN" "Prefixe WAN :" "$WAN_PREFIX") || exit 1
        WAN_PREFIX="${wan_pfx:-$WAN_PREFIX}"
        echo -e "  ${GREEN}WAN: ${WAN_LOCAL_IP} ↔ ${WAN_REMOTE_IP} (local=${n_operators}
         ↪  remote=${WAN_N_REMOTE} prefix=${WAN_PREFIX})${NC}"
    fi
fi

local use_defaults="N"
wt_yesno "Valeurs par defaut" "Valeurs par defaut pour tous (MCC=001/002/... MNC=01/02/...) ?" &&
 ↪  use_defaults="O"

local same_ms_all="N"
if [ "$use_defaults" != "O" ]; then
    wt_yesno "MS" "Meme nombre de MS pour tous les operateurs ?" && same_ms_all="O"
fi

declare -A OP_MCC OP_MNC OP_NAME OP_MS

if [ "$use_defaults" = "O" ]; then
    local common_ms
    common_ms=$(wt_input "MS" "MS par operateur (1-64) :" "8") || exit 1
    common_ms=${common_ms:-8}
    if ! [[ "$common_ms" =~ ^[0-9]+$ ]] || [ "$common_ms" -lt 1 ] || [ "$common_ms" -gt 64 ]; then
     ↪  common_ms=8; fi
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        # One PLMN per operator, on BOTH components: 001/002/003 makes a PLMN
        # mismatch visible.
        OP_MCC[$i]=$(op_mcc "$i"); OP_MNC[$i]=$(op_mnc "$i")
        OP_NAME[$i]="$(op_default_name "$i")"; OP_MS[$i]=$common_ms
    done
else
    if [ "$same_ms_all" = "O" ]; then
        local common_ms
        common_ms=$(wt_input "MS" "MS par operateur (1-64) :" "8") || exit 1
        common_ms=${common_ms:-8}
        if ! [[ "$common_ms" =~ ^[0-9]+$ ]] || [ "$common_ms" -lt 1 ] || [ "$common_ms" -gt 64 ]; then
         ↪  common_ms=8; fi
        for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
            local mcc mnc name dmcc dmnc dname
            dmcc=$(op_mcc "$i"); dmnc=$(op_mnc "$i"); dname="$(op_default_name "$i")"
           mcc=$(wt_input "Operateur ${i}" "MCC (pays = noeud $(op_plmn_node "$i"), $(op_country "$i")) :"
             ↪  "$dmcc") || exit 1; OP_MCC[$i]=${mcc:-$dmcc}
            mnc=$(wt_input "Operateur ${i}" "MNC :" "$dmnc") || exit 1; OP_MNC[$i]=${mnc:-$dmnc}
            name=$(wt_input "Operateur ${i}" "Nom :" "$dname") || exit 1; OP_NAME[$i]=${name:-$dname}
            OP_MS[$i]=$common_ms
        done
    else
        for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
            local mcc mnc name n_ms dmcc dmnc dname
            dmcc=$(op_mcc "$i"); dmnc=$(op_mnc "$i"); dname="$(op_default_name "$i")"
           mcc=$(wt_input "Operateur ${i}" "MCC (pays = noeud $(op_plmn_node "$i"), $(op_country "$i")) :"
             ↪  "$dmcc") || exit 1; OP_MCC[$i]=${mcc:-$dmcc}
            mnc=$(wt_input "Operateur ${i}" "MNC :" "$dmnc") || exit 1; OP_MNC[$i]=${mnc:-$dmnc}
            name=$(wt_input "Operateur ${i}" "Nom :" "$dname") || exit 1; OP_NAME[$i]=${name:-$dname}
```

```
                # Default 2, not 1: minimum for a MIXED radio (one faketrx and one QEMU
                # Calypso, like the native operator).
                n_ms=$(wt_input "Operateur ${i}" "Nombre de MS (1-64) :" "2") || exit 1; OP_MS[$i]=${n_ms:-2}
               if ! [[ "${OP_MS[$i]}" =~ ^[0-9]+$ ]] || [ "${OP_MS[$i]}" -lt 1 ] || [ "${OP_MS[$i]}" -gt 64 ];
                ↪  then OP_MS[$i]=1; fi
            done
        fi
    fi


    # ── RAN / Encryption ──────────────────────────────────────────
    # BRIDGE_NO_PROCESS=0: radio and mobile start with the core (see
    # "Containers start the full stack" below).
    PHY_MODE="faketrx"; BRIDGE_NO_PROCESS="${BRIDGE_NO_PROCESS:-0}"; BRIDGE_QEMU=1
    ENCRYPTION="${ENCRYPTION:-a5 1}"
    if [ -n "$HANDOFF_MODE_FROM_ENV" ]; then
        # The caller imposed the mode (HANDOFF_MODE in the environment): do not ask
        # again. Otherwise the menu (auto-answered with its first entry in
        # non-interactive mode) would override it.
        echo -e "  ${GREEN}[RAN] mode impose par l appelant : ${CYAN}${HANDOFF_MODE}${NC}"
    elif [ "${WAN_MESH:-0}" = "1" ]; then
        # A WAN node is a complete testbed: it needs a radio of BOTH types for an
        # inter-node call to be carried end to end. Hence hybrid by default (one faketrx +
        # one QEMU Calypso). Prefix with HANDOFF_MODE=... to change it.
        HANDOFF_MODE="faketrx-qemu"; HANDOFF_QEMU_CHOICE="faketrx-qemu"
        echo -e "  ${GREEN}[RAN] WAN → hybride par defaut : ${CYAN}1 faketrx + 1 QEMU Calypso${NC}"
    else
        choose_ran_host
    fi
fi

# ── Host IP detection ─────────────────────────────────────────────
HOST_IP=$(ip route get 1.1.1.1 2>/dev/null \
    | awk '{for(i=1;i<=NF;i++) if($i=="src") print $(i+1)}' | head -1) || true
if [ -z "$HOST_IP" ]; then
    HOST_IP=$(hostname -I 2>/dev/null | awk '{print $1}') || true
fi
HOST_IP="${HOST_IP:-127.0.0.1}"
REAL_UID=$(id -u "${SUDO_USER:-$(logname 2>/dev/null || echo root)}")
echo -e "${GREEN}IP hote : ${CYAN}${HOST_IP}${NC}  (Linphone)${NC}"
echo ""

# ── Where to reach the SS7 hub from here ──────────────────────────
# The hub is UNIQUE for the whole WAN:
#   this node hosts the hub  → containers reach it internally (172.20.0.10)
#   another node hosts it    → they go out over the WAN, to ITS address
WAN_STP_TARGET="$INTER_STP_IP"
HUB_IS_LOCAL=1
if [ -n "${WAN_HUB_IP:-}" ]; then
    # Hub given by address: it is not a node and nothing is started here
    # (dedicated inter-STP VM).
    WAN_STP_TARGET="$WAN_HUB_IP"; HUB_IS_LOCAL=0
elif [ "${WAN_MESH:-0}" = "1" ] && [ "${WAN_NODE_ID}" != "${WAN_HUB_NODE}" ]; then
    WAN_STP_TARGET="${WAN_IP[$WAN_HUB_NODE]:-}"; HUB_IS_LOCAL=0
    if [ -z "$WAN_STP_TARGET" ]; then
        echo -e "${RED}[SS7] le noeud ${WAN_HUB_NODE} (hub) n'est pas dans la table WAN${NC}" >&2
        echo -e "${RED}      (ou designez-le par son adresse : --hub-ip ADRESSE)${NC}" >&2
        exit 1
    fi
fi

# ── Backbone network ──────────────────────────────────────────────
# OSMO_DOCKER_ROUTED=1: bridge in "routed" mode (docker >= 28). Docker then stops
# masquerading backbone traffic and does not install the "ip raw" DROP that
# discards incoming packets to 172.20.0.x not arriving via the bridge (which
# leaves SCTP associations from the LAN in COOKIE_WAIT).
```

```
# Optional and non fatal: docker < 28 rejects the option and the ordinary
# (masqueraded) network is used.
if docker network inspect "$INTER_NET" &>/dev/null; then
    echo -e "  ${CYAN}Reseau backbone ${INTER_NET} deja present${NC}"
else
    if [ "${OSMO_DOCKER_ROUTED:-0}" = "1" ] \
       && docker network create --subnet="$INTER_NET_SUBNET" --gateway="$INTER_NET_GATEWAY" \
            -o com.docker.network.bridge.gateway_mode_ipv4=routed "$INTER_NET" &>/dev/null; then
        echo -e "  ${GREEN}✓ Reseau backbone ${INTER_NET} (${INTER_NET_SUBNET}) cree - mode routed${NC}"
    elif docker network create --subnet="$INTER_NET_SUBNET" --gateway="$INTER_NET_GATEWAY" "$INTER_NET"
     ↪  &>/dev/null; then
        [ "${OSMO_DOCKER_ROUTED:-0}" = "1" ] \
            && echo -e "  ${YELLOW}  (mode routed refuse par ce docker - reseau ordinaire)${NC}"
        echo -e "  ${GREEN}✓ Reseau backbone ${INTER_NET} (${INTER_NET_SUBNET}) cree${NC}"
    else
        echo -e "  ${RED}✗ Echec creation reseau backbone ${INTER_NET} (${INTER_NET_SUBNET})${NC}"; exit 1
    fi
fi

# ── SMS Routing ──────────────────────────────────────────────────
SMS_ROUTING_DIR=$(mktemp -d)
if declare -f sms_routing_generate_all > /dev/null 2>&1; then
    local _ms_counts=()
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
     ↪  _ms_counts+=("${OP_MS[$i]}"); done
    sms_routing_generate_all "$n_operators" "$SMS_ROUTING_DIR" "${_ms_counts[@]}"
    sms_routing_summary "$n_operators" "${_ms_counts[@]}"
fi

# ── Inter-STP ────────────────────────────────────────────────────
if [ "$HUB_IS_LOCAL" = "1" ]; then
    start_inter_stp "$n_operators"
    wait_inter_stp_ready "$n_operators"
else
    echo -e "${CYAN}[SS7] hub porte par le noeud ${WAN_HUB_NODE} (${WAN_STP_TARGET}) - pas de hub ici.${NC}"
    echo -e "${CYAN}      Les ASP de ce noeud s'y attacheront en M3UA/SCTP 2908.${NC}"
fi

# ── HLR subscribers ──────────────────────────────────────────────
local all_subscribers_file
all_subscribers_file=$(mktemp)
echo "# op_id:ms_idx:imsi:msisdn:ki" > "$all_subscribers_file"
# ── Directory: a fake identity for each IMSI ─────────────────────
# Puts a name and address next to each subscriber. Names and streets are
# FICTITIOUS, but the IMSI is exactly the one sent to the HLR below, from the
# same loop and variable. Entries come from data/noms.txt and data/adresses.txt
# (shuffled at load) and are taken IN ORDER, so no two subscribers share a name
# until the file is exhausted.
local ANNUAIRE_FILE="${OSMO_ANNUAIRE:-/var/tmp/osmo-annuaire.csv}"
# ── SS7 directory: the same service one layer down ───────────────
# Maps each POINT CODE to its machine, operator and hub. Produced by the SAME
# loop as the configurations: a record of what was just written.
local ANNUAIRE_SS7="${OSMO_ANNUAIRE_SS7:-/var/tmp/osmo-annuaire-ss7.csv}"
printf
 ↪  'noeud;operateur;pays;mcc;mnc;pc_msc;pc_stp;pc_bsc;rctx_msc;rctx_stp;rctx_bsc;rctx_inter;hub;as_inter;asp_inter;backbon
 ↪  \
    > "$ANNUAIRE_SS7"
local _pick=0
printf 'imsi;msisdn;operateur;pays;mcc;mnc;nom;adresse;ki\n' > "$ANNUAIRE_FILE"
for op_id in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
    local mcc="${OP_MCC[$op_id]}" mnc="${OP_MNC[$op_id]}" n_ms="${OP_MS[$op_id]}"
    local _pays; _pays="$(op_country "$op_id")"
    for ms_idx in $(seq 1 "$n_ms"); do
        local msin; msin=$(printf '%04d%06d' "${op_id}" "${ms_idx}")
        local imsi="${mcc}${mnc}${msin}"
        # Six-digit MSISDN: <node>00<operator><rank>, e.g. 100101, 100102, 200101.
```

```
        # The first digit carries the node, so routing can be read from the number and
        # subscriber numbers do not clash with dialplan services (100, 200, 500, 600, 700).
        local msisdn; msisdn=$(osmo_msisdn "$(osmo_node_id)" "$op_id" "$ms_idx")
        local ki; ki=$(printf '00112233445566778899aabbccdd%02x%02x' "${ms_idx}" "${op_id}")
        echo "${op_id}:${ms_idx}:${imsi}:${msisdn}:${ki}" >> "$all_subscribers_file"
        printf '%s;%s;%s;%s;%s;%s;%s;%s %s;%s\n' \
            "$imsi" "$msisdn" "${OP_NAME[$op_id]}" "$_pays" "$mcc" "$mnc" \
            "${POOL_NOMS[$(( _pick % ${#POOL_NOMS[@]} ))]}" \
            "$(( RANDOM % 97 + 1 ))" \
            "${POOL_ADR[$(( _pick % ${#POOL_ADR[@]} ))]}" \
            "$ki" >> "$ANNUAIRE_FILE"
        _pick=$(( _pick + 1 ))
    done
done
local total_subs; total_subs=$(( $(wc -l < "$all_subscribers_file") - 1 ))
echo -e "${GREEN}Abonnes total : ${total_subs}${NC}"
# Preview: first entries, then the path of the full file.
awk -F';' 'NR==1 {next}
           NR<=9 {printf "  %-16s %-6s %-16s %-12s %-22s %s\n", $1,$2,$3,$4,$7,$8}
           END {if (NR>9) printf "  ... et %d autres\n", NR-9}' "$ANNUAIRE_FILE"
echo -e "  ${CYAN}Annuaire complet : ${ANNUAIRE_FILE}${NC}  (aussi /etc/osmocom/annuaire.csv dans chaque
↪  conteneur)"
echo ""

# ── Clean slate for the operator series ──────────────────────────────
# The "docker rm -f" below only targets the container of the SAME NAME. A leftover
# from a previous run (more operators, interrupted series) keeps its published
# ports and makes the next creation fail ("port is already allocated"). This
# function RECREATES the whole series, so remove them first.
local _stale
_stale="$(docker ps -aq --filter "name=^osmo-operator-" 2>/dev/null)"
if [ -n "$_stale" ]; then
    echo -e "  ${CYAN}Nettoyage : $(echo "$_stale" | wc -l) conteneur(s) operateur d'un lancement
    ↪  precedent${NC}"
    # shellcheck disable=SC2086
    docker rm -f $_stale >/dev/null 2>&1 || true
fi

# ── Sequential operator start ────────────────────────────────────────
for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
    local container_name net_name subnet gateway container_ip inter_local_ip rctx_inter
    local n_groups last_group_ip

    container_name=$(op_container "$i")
    net_name="gsm-net-op${i}"
    subnet=$(op_private_net "$i")
    gateway=$(op_private_gw "$i")
    container_ip=$(op_private_ip "$i")
    inter_local_ip=$(op_backbone_ip "$i")
    rctx_inter=$(op_rctx_inter "$i")
    n_groups=$(( (${OP_MS[$i]} + 7) / 8 ))
    last_group_ip="127.0.0.${n_groups}"

    echo -e "${CYAN}── Operateur ${i} : ${OP_NAME[$i]} ($(op_country "$i"), MCC=${OP_MCC[$i]}
    ↪  MNC=${OP_MNC[$i]}) ──${NC}"
    echo -e "  Backbone   : ${CYAN}${inter_local_ip}${NC}  Prive : ${CYAN}${container_ip}${NC}"
    local _node_i="$WAN_NODE_ID" _op_i="$i"
    if [ "${WAN_NODE_PER_OP:-0}" = "1" ]; then
        _node_i=$(( WAN_NODE_ID + i - 1 ))
        _op_i=1
        # Node numbers are 1..9: beyond that the point code 1.<node><op>.<role>
        # leaves the ITU 3-8-3 format, no matching AS exists on the hub
        # (helpers/create_interop.sh supports 9 nodes), and network/set-node-id.sh and
        # start-native.sh reject it.
        if [ "$_node_i" -gt 9 ] || [ "$_node_i" -lt 1 ]; then
            echo -e "${RED}[WAN] --node-per-op : l'operateur ${i} tomberait sur le noeud ${_node_i}${NC}" >&2
```

```
        echo -e "${RED}      Les numeros de noeud vont de 1 a 9. Avec --wan-id ${WAN_NODE_ID},"' '"${NC}" >&2
          echo -e "${RED}      le maximum est $(( 10 - WAN_NODE_ID )) operateur(s) : baissez --wan-id,"'
          ↪  '"${NC}" >&2
          echo -e "${RED}      reduisez --operators, ou retirez --node-per-op.${NC}" >&2
          exit 1
      fi
  fi
  local _pc_mid="$i" _rctx_inter=""
  if [ "${WAN_MESH:-0}" = "1" ]; then
      _pc_mid="${_node_i}${_op_i}"
      _rctx_inter=$(( _node_i * 1000 + _op_i * 100 + 50 ))
  fi

  # The identity DISPLAYED is the one WRITTEN to the config.
  echo -e "  STP PC     : 1.${_pc_mid}.2  RCTX : ${_rctx_inter:-$rctx_inter}  MS : ${CYAN}${OP_MS[$i]}${NC}"

  # SS7 record of THIS operator, with the values written to its configuration
  # just below.
  printf '%s;%s;%s;%s;%s;1.%s.1;1.%s.2;1.%s.3;%s;%s;%s;%s;%s;as-n%sop%s;asp-n%sop%s;%s;%s\n' \
      "$_node_i" "${OP_NAME[$i]}" "$(op_country "$i")" \
      "${OP_MCC[$i]}" "${OP_MNC[$i]}" \
      "$_pc_mid" "$_pc_mid" "$_pc_mid" \
      "$(op_rctx_msc "$i")" "$(op_rctx_stp "$i")" "$(op_rctx_bsc "$i")" \
      "${_rctx_inter:-$rctx_inter}" "${WAN_STP_TARGET:-127.0.0.1}" \
      "$_node_i" "$_op_i" "$_node_i" "$_op_i" \
      "$inter_local_ip" "$container_ip" >> "$ANNUAIRE_SS7"

  if docker network inspect "$net_name" &>/dev/null; then
      echo -e "  Reseau     : ${CYAN}${net_name}${NC} (deja present)"
  elif docker network create --subnet="$subnet" --gateway="$gateway" "$net_name" &>/dev/null; then
      echo -e "  Reseau     : ${GREEN}✓ ${net_name} (${subnet}) cree${NC}"
  else
      echo -e "  ${RED}✗ Echec creation reseau ${net_name} (${subnet})${NC}"; exit 1
  fi

  # ── The no-masquerade rule, BEFORE the stack connects ──────────────
  # A NAT decision is taken on the FIRST packet of an association and conntrack
  # keeps it for its lifetime. The rule is re-asserted after EACH network creation
  # (docker re-inserts its MASQUERADE at the head each time), hence before any
  # stack starts.
  reassert_docker_lan_return >/dev/null

  local tmpdir
  tmpdir=$(mktemp -d)
  # Point code plan. Without WAN: 1.<op>.<role>. With WAN the node number is
  # included, otherwise two machines present identical SS7 addresses to the hub.
  # Only RCTX_INTER is overridden: it is the only routing context the hub sees;
  # MSC and BSC contexts stay internal to the container.
  #
  # ONE NODE PER CONTAINER (--node-per-op)
  # By default the machine IS the node and its containers are its operators:
  # 1.<node><op>.<role>.
  #
  # When containers are reachable at THEIR own address (172.20.0.11, .12... routed
  # from the LAN) they are full nodes, like a VM. With --node-per-op, container i
  # becomes node (base + i - 1), carries operator 1 of that node, and its point
  # code is 1.<n>1.<role>.
  RCTX_INTER_OVERRIDE="$_rctx_inter" \
  apply_config_templates "$tmpdir" \
      "$container_ip" "$gateway" \
      "$i" "1.${_pc_mid}.1" "1.${_pc_mid}.2" "1.${_pc_mid}.3" \
      "${OP_MCC[$i]}" "${OP_MNC[$i]}" "${OP_NAME[$i]}" \
      "$WAN_STP_TARGET" "no shutdown" \
      "$n_operators"

  # The directory travels with the stack: IMSIs are read in logs inside the
```

```
# container, not on the host.
[ -f "$ANNUAIRE_FILE" ] && cp "$ANNUAIRE_FILE" "$tmpdir/osmocom/annuaire.csv"
[ -f "$ANNUAIRE_SS7" ]  && cp "$ANNUAIRE_SS7"  "$tmpdir/osmocom/annuaire-ss7.csv"

# ── Dialplan entries ──────────────────────────────────────────
# Asterisk cannot read CSV: write the same entries as one extension per MSISDN
# in the sub-annuaire context (see configs/extensions.conf), so calls reach the
# handset with a name.
{
    printf '; annuaire.conf - genere par start-docker.sh, ne pas editer a la main.\n'
    printf '; Une fiche par abonne du banc. Le repli _X. est dans extensions.conf.\n'
    printf '[sub-annuaire]\n'
    awk -F';' 'NR>1 && $2 != "" {
        gsub(/[^A-Za-z0-9 ._-]/, "", $7); gsub(/[^A-Za-z0-9 ._-]/, "", $3)
        gsub(/[^A-Za-z0-9 ._-]/, "", $4); gsub(/[^A-Za-z0-9 .,_-]/, "", $8)
        printf "exten => %s,1,Set(CALLERID(name)=%s)\n", $2, $7
        printf " same => n,Set(ANNU_OP=%s)\n", $3
        printf " same => n,Set(ANNU_PAYS=%s)\n", $4
        printf " same => n,Set(ANNU_ADR=%s)\n", $8
        printf " same => n,Set(ANNU_IMSI=%s)\n", $1
        printf " same => n,Set(CDR(userfield)=%s / %s / %s)\n", $7, $3, $4
        printf " same => n,NoOp(=== ANNUAIRE %s = %s, %s (%s) ===)\n", $2, $7, $3, $4
        printf " same => n,Return()\n"
    }' "$ANNUAIRE_FILE"
} > "$tmpdir/asterisk/annuaire.conf" 2>/dev/null || true

local vol_args alsa_args
vol_args=$(build_vol_args "$tmpdir")
alsa_args=$(build_alsa_args)

# The chown is REQUIRED: this directory is mounted on /var/log/osmocom and the
# daemons run as osmocom (999:1000 in the image). Owned by root, osmo-hlr cannot
# create its log file, fails to parse its config and restart-loops, and
# osmo-start.sh / run.sh abort the whole pipeline.
mkdir -p /tmp/osmocom-logs/op${i}
chown 999:1000 /tmp/osmocom-logs/op${i} 2>/dev/null || true

# ── Ports to expose ──────────────────────────────────────────
local port_args=""

# ── SIP/RTP: nothing published on the host ──────────────────
# The native operator permanently holds 5060/udp (Asterisk) and 5061
# (osmo-sip-connector). Containers are on a bridge routable from the host and
# are reached directly at their own address on the nominal port, so no
# publication (and no port conflict) is needed.
local _cip; _cip="$(op_backbone_ip "$i")"
echo -e "  Linphone   : ${CYAN}${_cip}:5060${NC}  RTP 30000-30199 (direct, sans publication)"

# Plus WAN ports if enabled (legacy or mesh)
if wan_active; then
    local sip_port rtp_start rtp_end sms_port
    sip_port=$(wan_sip_port "$i")
    rtp_start=$(wan_rtp_start "$i")
    rtp_end=$(wan_rtp_end "$i")
    port_args="${port_args} -p ${sip_port}:5060/udp -p ${sip_port}:5060/tcp -p
    ↪  ${rtp_start}-${rtp_end}:${rtp_start}-${rtp_end}/udp"
    echo -e "  WAN        : SIP ${sip_port} RTP ${rtp_start}-${rtp_end}"
    if [ "${WAN_MESH:-0}" = "1" ]; then
        # One SMS port PER operator: the remote relay must reach the container of the
        # RIGHT operator, since MT injection goes through that container's HLR. A single
        # port would deliver all SMS to the first operator ("MSISDN not found in HLR").
        sms_port=$(wan_sms_port "$i")
        port_args="${port_args} -p ${sms_port}:7890/tcp"
        echo -e "  WAN SMS    : ${sms_port} → relay 7890"
    fi
fi
```

```
docker rm -f "$container_name" 2>/dev/null || true
ensure_host_audio_relay

# --restart, NOT --rm: with --rm, operator containers would DISAPPEAR on reboot
# together with their HLR database and anything not mounted from the host. The
# "docker rm -f" above makes --rm unnecessary, and docker rejects --rm with --restart.
#
# OSMO_ROLE=operator: the container regenerates its configs when run.sh replays
# apply_config_templates, and generate_configs.sh's SS7 identity fix-up silently
# stops on an empty role (all containers would then use point code 1.1.2).
# ── --security-opt apparmor=unconfined: required for systemd as PID 1 ──────
# The entrypoint ends with `exec systemd`, which mounts its API filesystems; Ubuntu's
# docker-default AppArmor profile forbids mounts regardless of capabilities
# (CAP_SYS_ADMIN is not enough), and PID 1 exits with code 255:
#     Failed to mount tmpfs (type tmpfs) on /run/lock ... Permission denied
# systemd writes this to /dev/console, so it is NOT in `docker logs`; use
# `docker run -t` to see it.
# SYS_NICE: c54x_exe/run.sh runs the chain (DSP, QEMU, osmocon, mobile, bridge)
# under `nice -n -10` (RT_NICE), which needs this capability.
# shellcheck disable=SC2086
docker run -d \
    --restart unless-stopped \
    --name "$container_name" \
    --hostname "$container_name" \
    --network "$INTER_NET" \
    --ip "$inter_local_ip" \
    --cap-add NET_ADMIN \
    --cap-add SYS_ADMIN \
    --cap-add NET_RAW \
    --cap-add SYS_NICE \
    --security-opt apparmor=unconfined \
    --ulimit rtprio=18 \
    --shm-size=8g \
    --cgroupns host \
    --device /dev/net/tun:/dev/net/tun \
    $alsa_args \
    $port_args \
    -v /sys/fs/cgroup:/sys/fs/cgroup:rw \
    -v /tmp/osmocom-logs/op${i}:/var/log/osmocom \
    --tmpfs /tmp:exec,rw,mode=1777,size=2g \
    --tmpfs /run:exec,size=64m \
    --tmpfs /run/lock \
    -e OPERATOR_ID="$i" \
    -e OSMO_ROLE=operator \
    -e N_MS="${OP_MS[$i]}" \
    -e CONTAINER_IP="$container_ip" \
    -e GATEWAY_IP="$gateway" \
    -e INTER_STP_IP="${WAN_STP_TARGET:-$INTER_STP_IP}" \
    -e OSMO_HUB_IP="${WAN_STP_TARGET:-$INTER_STP_IP}" \
    -e WAN_NODE_ID="${_node_i}" \
    -e OSMO_WAN_NODE="${_node_i}" \
    -e HOST_IP="${HOST_IP}" \
    -e SIP_HOST_PORT="5060" \
    -e PHY_MODE="${PHY_MODE}" \
    -e HOST_AUDIO_RELAY="${HOST_AUDIO_RELAY}" \
    $vol_args \
    "$IMAGE_RUN" \
    sleep infinity

docker network connect --ip "$container_ip" "$net_name" "$container_name"

# ── A dead operator does not take down the following ones ──────────────
# Everything below talks to the container through `docker exec`, which fails on a
# restarting container, and this script runs under `set -eu`. Check the container
# really runs first; if not, report it (with the commands that show the real
```

```
# cause, hidden from `docker logs`) and move on to the next operator.
_st="$(docker inspect -f '{{.State.Status}}' "$container_name" 2>/dev/null || echo absent)"
if [ "$_st" != running ]; then
    _rc="$(docker inspect -f '{{.State.ExitCode}}' "$container_name" 2>/dev/null || echo '?')"
    echo -e "  ${RED}[!] ${container_name} n'est pas parti (etat: ${_st}, code: ${_rc}) - operateur
     ↪  SAUTE${NC}"
    echo -e "      ${YELLOW}systemd ecrit son erreur sur /dev/console : elle n'est PAS dans docker
     ↪  logs.${NC}"
    echo -e "      ${CYAN}docker logs ${container_name} | tail${NC}"
    echo -e "      ${CYAN}docker run --rm -t <memes drapeaux> ${IMAGE_RUN} sleep infinity${NC}   (pour la
     ↪  voir)"
    FAILED_OPS="${FAILED_OPS:-} ${container_name}"
    continue
fi

force_update_trees "$container_name"

# ── Containers start the full stack ──────────────────────────────────────
# run.sh starts core (STP, HLR, MSC, BSC, MGW, GGSN, SGSN, PCU) and radio (PHY,
# mobile, Asterisk, SMSC). It never attaches by itself: everything runs in tmux
# and only the attach command is printed. The delegated QEMU pipeline honours
# CALYPSO_NO_ATTACH=1.
# BRIDGE_NO_PROCESS=1 ./launch/start-docker.sh starts the core only.
local run_cmd="RUN_NO_PROCESS=${BRIDGE_NO_PROCESS:-0} CALYPSO_NO_ATTACH=1 /etc/osmocom/run.sh"
echo -e "  ${GREEN}[*] Lancement run.sh...${NC}"
# run.sh is started detached; its output goes to /var/log/osmocom/run.sh.log
# (mounted on the host as /tmp/osmocom-logs/opN).
# ONE start path per container: in QEMU mode each container's radio stack is
# started later by start-native.sh. run.sh is still needed here: it brings up the
# core, and the next step (waiting for HLR port 4258, then provisioning
# subscribers) depends on its HLR. The HLR database survives the later restart.
docker exec -d "$container_name" bash -c "mkdir -p /var/log/osmocom && { ${run_cmd}; } >
 ↪  /var/log/osmocom/run.sh.log 2>&1"

# Wait for HLR
echo -ne "  ${GREEN}[*] Attente HLR (4258)${NC}"
local retry=0
while ! docker exec "$container_name" bash -c "echo >/dev/tcp/127.0.0.1/4258" 2>/dev/null; do
    sleep 2; echo -n "."; retry=$((retry + 1))
    if [ $retry -ge 45 ]; then echo -e " ${RED}TIMEOUT${NC}"; break; fi
done
echo -e " ${GREEN}✓${NC}"

# ── Each HLR only holds its own subscribers ──────────────────────────────
# Copying all lab subscribers into every HLR would fake a roaming that does not
# exist. Real roaming goes through SS7, not through a database copy.
local _mine; _mine="$(awk -F: -v op="$i" '$1 == op' "$all_subscribers_file")"
local _n_mine; _n_mine="$(printf '%s\n' "$_mine" | grep -c . || true)"
echo -e "  ${GREEN}[*] Alimentation HLR Op${i} (${_n_mine} abonnes)...${NC}"

# ── Remove subscribers that are no longer in the plan ────────────────────
# hlr.db survives restarts, so a plan change leaves stale subscribers that keep
# their MSISDN. osmo-hlr enforces MSISDN uniqueness, so the right subscriber
# could not get its number ("cannot update MSISDN").
local _garder; _garder="$(printf '%s\n' "$_mine" | cut -d: -f3 | paste -sd'|' -)"
{
    echo "#!/bin/bash"
    echo "cat > /tmp/hlr_feed.vty << 'VTYCMDS'"
    echo "enable"
    if [ -n "$_garder" ]; then
        echo "__PURGE__${_garder}"
    fi
    while IFS=: read -r feed_op feed_ms feed_imsi feed_msisdn feed_ki; do
        [[ "$feed_op" =~ ^#.*$ ]] && continue
        echo "subscriber imsi ${feed_imsi} create"
        echo "subscriber imsi ${feed_imsi} update msisdn ${feed_msisdn}"
```

```
        echo "subscriber imsi ${feed_imsi} update aud2g comp128v1 ki ${feed_ki}"
    done <<< "$_mine"
    echo "end"
    echo "VTYCMDS"
} | docker exec -i "$container_name" bash

# Rejections are NOT counted: osmo-hlr prefixes ALL replies with '%', success
# included, and "Subscriber already exists" is the NORMAL reply on a restart (the
# lab is reprovisioned at each start).
#
# The only proof that a subscriber is served is its record: read the database
# back. A subscriber without keys is worse than a missing one - osmo-msc finds it,
# requests auth vectors, gets none, and rejects the mobile with a SIM-related cause.
local _sans_cle
# Purge: LIST with sqlite (the VTY has no filterable "show all") and DELETE
# through the VTY - osmo-hlr keeps an in-memory cache that a direct SQL DELETE
# would bypass.
docker exec "$container_name" bash -c '
    garder="$(sed -n "s/^__PURGE__//p" /tmp/hlr_feed.vty)"
    sed -i "/^__PURGE__/d" /tmp/hlr_feed.vty
    if [ -n "$garder" ] && command -v sqlite3 >/dev/null 2>&1; then
        sqlite3 /var/lib/osmocom/hlr.db "select imsi from subscriber;" 2>/dev/null \
          | grep -vE "^($garder)$" \
          | sed "s|^|subscriber imsi |; s|$| delete|" >> /tmp/hlr_purge.vty
    fi
    if [ -s /tmp/hlr_purge.vty ]; then
        { echo enable; cat /tmp/hlr_purge.vty; } > /tmp/hlr_purge_full.vty
        if command -v nc >/dev/null 2>&1; then
            (sleep 1; cat /tmp/hlr_purge_full.vty; sleep 2) | nc -q2 127.0.0.1 4258 2>/dev/null
        else
            (sleep 1; cat /tmp/hlr_purge_full.vty; sleep 3) | telnet 127.0.0.1 4258 2>/dev/null
        fi
        echo "  purge : $(grep -c . /tmp/hlr_purge.vty) abonne(s) hors plan supprime(s)" >&2
    fi
    rm -f /tmp/hlr_purge.vty /tmp/hlr_purge_full.vty
    if command -v nc >/dev/null 2>&1; then
        (sleep 1; cat /tmp/hlr_feed.vty; sleep 2) | nc -q2 127.0.0.1 4258 2>/dev/null
    else
        (sleep 1; cat /tmp/hlr_feed.vty; sleep 3) | telnet 127.0.0.1 4258 2>/dev/null
    fi
    rm -f /tmp/hlr_feed.vty
' >/dev/null 2>&1 || true

_sans_cle="$(docker exec "$container_name" bash -c '
    command -v sqlite3 >/dev/null 2>&1 || exit 0
    sqlite3 /var/lib/osmocom/hlr.db "
        select count(*) from subscriber s
        left join auc_2g a on a.subscriber_id = s.id
        where a.subscriber_id is null;" 2>/dev/null' 2>/dev/null)" || true

if [ "${_sans_cle:-0}" -gt 0 ]; then
    echo -e "  ${RED}✗ HLR Op${i} : ${_sans_cle} abonne(s) SANS CLE - ils ne pourront pas
     ↪  s'authentifier${NC}"
    echo -e "    ${CYAN}docker exec ${container_name} sqlite3 /var/lib/osmocom/hlr.db \"select s.imsi from
     ↪  subscriber s left join auc_2g a on a.subscriber_id=s.id where a.subscriber_id is null;\"${NC}"
else
    echo -e "  ${GREEN}✓ HLR Op${i} alimente (${_n_mine} abonnes, tous avec cle)${NC}"
fi

# Wait for the last group - skipped in no-process mode
if [ "${BRIDGE_NO_PROCESS:-0}" = "1" ]; then
    echo -e "  ${YELLOW}[no-process] mobile non lance - attente 4247 sautee${NC}"
else
    echo -ne "  ${GREEN}[*] Attente groupe (${last_group_ip}:4247)${NC}"
    retry=0
    while ! docker exec "$container_name" bash -c "echo >/dev/tcp/${last_group_ip}/4247" 2>/dev/null; do
```

```
            sleep 2; echo -n "."; retry=$((retry + 1))
            if [ $retry -ge 90 ]; then echo -e " ${RED}TIMEOUT${NC}"; break; fi
        done
        echo -e " ${GREEN}OK${NC}"
        sleep 3
    fi
    echo -e "  ${GREEN}✓${NC} ${container_name} pret"
    echo ""
done

rm -f "$all_subscribers_file"

# ── Wait for SMS relays ──────────────────────────────────────────
if [ "${BRIDGE_NO_PROCESS:-0}" != "1" ] && declare -f sms_routing_wait_ready > /dev/null 2>&1; then
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        sms_routing_wait_ready "$(op_container "$i")" 90 || true
    done
fi

# ── WAN ──────────────────────────────────────────────────────────
if [ "${WAN_MESH:-0}" = "1" ]; then
    wan_mesh_apply "$n_operators"
elif [ "$WAN_ENABLED" = "true" ]; then
    setup_wan_interop "$n_operators" "$WAN_N_REMOTE"
fi

# ── Summary ──────────────────────────────────────────────────────
echo ""
echo -e "${GREEN}${BOLD}Stack multi-operateurs demarree !${NC}"
echo ""
echo -e "  Inter-STP @ ${CYAN}${INTER_STP_IP}:2908${NC}  PC=0.0.0"
for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
    local rctx bb_ip n_groups
    rctx=$(op_rctx_inter "$i"); bb_ip=$(op_backbone_ip "$i")
    n_groups=$(( (${OP_MS[$i]} + 7) / 8 ))
    echo -e "  Op${i} ${OP_NAME[$i]}  STP 1.${i}.2 @ ${bb_ip}  RCTX ${rctx}  [${OP_MS[$i]} MS]"
done

echo ""
echo -e "  ${BOLD}Linphone (depuis l'hote) :${NC}"
for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
    local bb_ip; bb_ip=$(op_backbone_ip "$i")
    echo -e "    Op${i}: ${CYAN}${bb_ip}:5060${NC}"
done
echo -e "    linphone_A / tester → 100  |  linphone_B / testerB → 200"

if [ "${WAN_MESH:-0}" = "1" ]; then
    echo ""
    echo -e "  ${BOLD}WAN mesh :${NC} noeud ${CYAN}${WAN_NODE_ID}${NC}/${WAN_NODE_COUNT} - indicatif
     ↪  ${CYAN}$(wan_local_ind)${NC}"
    for _r in "${WAN_NODE_LIST[@]}"; do
        [ "$_r" = "$WAN_NODE_ID" ] && continue
        echo -e "    ${CYAN}${WAN_IND[$_r]}${NC}10001 → MS 10001 op1 du noeud ${_r} (${WAN_IP[$_r]})"
    done
elif [ "$WAN_ENABLED" = "true" ]; then
    echo ""
    echo -e "  ${BOLD}WAN :${NC} ${CYAN}${WAN_LOCAL_IP}${NC} (${n_operators} ops) ↔
     ↪  ${CYAN}${WAN_REMOTE_IP}${NC} (${WAN_N_REMOTE} ops)  prefix=${WAN_PREFIX}"
fi
echo ""

# ── Checks, right after the launch ──────────────────────────────
#   ss7_check.sh        ASP, AS, routes, SSN - the layer that carries calls.
#   operator_summary.sh what each operator ACTUALLY has: HLR/VLR subscribers,
#                       BTS, PDP contexts. --live makes it generate its own dump;
#                       without it, it would analyse a previous lab's dump.
```

```
#
# RELATIVE paths, like ./helpers/prepare_host.sh above: this script is run from
# the repo root. Neither may fail the launch (|| true): the stack is up, its
# state is only REPORTED.
# OSMO_SKIP_CHECKS=1 skips them (start-multi.sh sets it and runs its own checks).
if [ "${OSMO_SKIP_CHECKS:-0}" != "1" ]; then
    if [ -r ./checks/ss7_check.sh ]; then
        echo -e "${GREEN}${BOLD}── Check SS7 ──${NC}"
        bash ./checks/ss7_check.sh || true
        echo ""
    else
        echo -e "  ${YELLOW}checks/ss7_check.sh introuvable - check SS7 saute${NC}"
    fi
    if [ -r ./checks/operator_summary.sh ]; then
        echo -e "${GREEN}${BOLD}── Check operateurs ──${NC}"
        bash ./checks/operator_summary.sh --live || true
        echo ""
    else
        echo -e "  ${YELLOW}checks/operator_summary.sh introuvable - resume saute${NC}"
    fi
fi

# --virtualbox: the VMs are created earlier (setup-vbox-interco.sh) and started
# here, once the hub and local operators are up - a node starting before the hub
# gets its SCTP refused.
if [ "${VBOX_INTERCO:-0}" = "1" ]; then
    local _lab="$START_REPO/tools/vbox-wan-lab.sh"
    if [ -x "$_lab" ] || [ -r "$_lab" ]; then
        echo ""
        echo -e "${GREEN}[*] Demarrage des VM du WAN...${NC}"
        bash "$_lab" start || echo -e "  ${YELLOW}⚠ demarrage des VM incomplet${NC}"
    fi
fi

# HERE and not earlier: docker re-inserts its MASQUERADE rules at the head of
# POSTROUTING for EACH network created (one per operator). At this point all
# networks exist.
reassert_docker_lan_return

# Address through which VMs must route to the containers. It changes with the
# active interface (unplugged cable -> wifi answers with another address).
local _hostip
_hostip="${HOST_LAN_IP:-$(ip route show default 2>/dev/null | awk '/^default/{print $9; exit}')}"
if [ -n "$_hostip" ] && [ "${WAN_MESH:-0}" = "1" ]; then
    echo -e "\n  ${BOLD}Cote VM${NC}, pour joindre les conteneurs :"
    echo -e "    ${CYAN}bash network/set_ip_vm.sh --via ${_hostip} --persist${NC}"
fi

echo -e "\n${GREEN}${BOLD}Stack prete !${NC}"

# ── QEMU mode: handoff to start-native.sh ──────────────────────────────
if [ "${BRIDGE_QEMU:-0}" = "1" ]; then
    local _qemu_container; _qemu_container=$(op_container 1)
    echo -e "  ${CYAN}[*] run.sh calypso → ${_qemu_container} (terminal courant)${NC}"
    echo -e "  ${CYAN}[*] mode=${HANDOFF_MODE} pipeline=${HANDOFF_QEMU_CHOICE} chiffrement='${ENCRYPTION}'
     ↪  (choisis sur l'hote, NO_MENU)${NC}"

    # The start-native.sh run INSIDE the container must know its node: it re-checks
    # (and fixes) the SS7 identity before starting the stack. The arguments are built
    # per container by _node_args() below.

    local _wan_env=""
    if [ "${WAN_MESH:-0}" = "1" ]; then
        _wan_env="WAN_NODES='$(wan_nodes_spec)' WAN_NODE_ID='${WAN_NODE_ID}' WAN_OPS='${n_operators}'"
        echo -e "  ${CYAN}[*] WAN mesh transmis au conteneur (noeud ${WAN_NODE_ID}, indicatif
         ↪  $(wan_local_ind))${NC}"
```

```
elif [ "$WAN_ENABLED" = "true" ]; then
    _wan_env="WAN_REMOTE_IP='${WAN_REMOTE_IP}' WAN_PREFIX='${WAN_PREFIX}'
     ↪  WAN_N_REMOTE='${WAN_N_REMOTE:-$n_operators}'"
    echo -e "  ${CYAN}[*] SMS inter-WAN → ${WAN_REMOTE_IP} (prefixe ${WAN_PREFIX})${NC}"
fi

# ── Radio stack launch ──────────────────────────────────────────
# start-native.sh is run in each container to bring up the radio stack.
#
# Behaviour depends on the number of containers (see below):
#   - one     : start, THEN attach to the "calypso" tmux session;
#   - several : start each one detached, sequentially.

# Node arguments of a container: its place in the WAN plan.
_node_args() {  # $1 = container index
    # Operator rank is 1 only if EACH container is a node. Otherwise containers share
    # the node and differ by rank (set-node-id.sh would otherwise give container 2 the
    # identity of container 1). Same convention as the plan written above (_node_i / _op_i).
    local _n _op
    if [ "${WAN_NODE_PER_OP:-0}" = "1" ]; then
        _n=$(( ${WAN_NODE_ID:-1} + $1 - 1 )); _op=1
    else
        _n="${WAN_NODE_ID:-1}"; _op="$1"
    fi
    # ALWAYS fall back to node 1 / operator 1 (also without WAN or with an empty
    # WAN_NODE_ID): start-native.sh needs a valid "--node N" or it exits, and a
    # single node IS node 1 (same as WAN_NODE_ID="${WAN_NODE_ID:-1}" above).
    [[ "$_n"  =~ ^[1-9]$ ]] || _n=1
    [[ "$_op" =~ ^[1-9]$ ]] || _op=1
    printf -- '--node %s --op %s' "$_n" "$_op"
    # The hub only makes sense in mesh mode; an empty `--hub-ip` would shift the
    # following arguments.
    [ "${WAN_MESH:-0}" = "1" ] && [ -n "${WAN_STP_TARGET:-}" ] && \
        printf -- ' --hub-ip %s' "$WAN_STP_TARGET"
    return 0
}

echo ""

# start-native.sh command for a container: node identity + hub + WAN table.
_hcmd() {  # $1 = container index -> prints the command
    local _na; _na="$(_node_args "$1")"
    # `unset PONT_PY`: the image (Dockerfile.run) pins it to pont.py (grgsm); in DSP
    # mode c54x_exe/run.sh would then start pont.py --dsp-port 6702 and no burst would
    # reach the DSP.
    local _cmd="cd /opt/GSM/osmo-operator && unset PONT_PY && NO_MENU=1"
    # CALYPSO_NO_ATTACH follows OSMO_NO_ATTACH: qosmo-grgsm/run.sh attaches to tmux
    # by default when it has a TTY (NO_ATTACH="${CALYPSO_NO_ATTACH:-0}"), and
    # `docker exec -t` provides one. The variable set on /etc/osmocom/run.sh above
    # does not cross this handoff.
    [ "${OSMO_NO_ATTACH:-0}" = "1" ] && _cmd="$_cmd CALYPSO_NO_ATTACH=1"
    _cmd="$_cmd MODE='${HANDOFF_MODE}' QEMU_CHOICE='${HANDOFF_QEMU_CHOICE}'"
    # The HOST's encryption, not a constant: the container's configs are already
    # written with it (OSMO_NO_REGEN=1 there), and 08-gabarits.sh refuses to start if
    # they differ. CALYPSO_BRIDGE is not forced: 05-run-sh.sh sets it to `none` in DSP
    # mode (c54x_exe/run.sh provides the bridge) and to `pont` in grgsm mode.
    _cmd="$_cmd ENCRYPTION='${ENCRYPTION}' CALYPSO_MODE=shunt_legit"
    [ -n "$_wan_env" ] && _cmd="$_cmd ${_wan_env}"
    _cmd="$_cmd ./start-native.sh ${_na} --force"
    printf '%s' "$_cmd"
}

local _c _ct _cmd
if [ "$n_operators" -eq 1 ] && [ "${OSMO_NO_ATTACH:-0}" != "1" ]; then
    # ── One container: start, then attach ──────────────────────
    # ...unless OSMO_NO_ATTACH=1 (launch from an icon or a script such as
```

```
            # start-multi.sh, which must get control back for checks and summary). The
            # stack then stays up in its calypso session; attach manually when needed.
            # start-native.sh and run.sh output scrolls in the current terminal, then we
            # ATTACH to the container's "calypso" tmux session. Ctrl-b then d detaches
            # without stopping anything.
            _ct="$(op_container 1)"; _cmd="$(_hcmd 1)"
            echo -e "  ${BOLD}Lancement de la pile radio sur ${_ct} (terminal courant)...${NC}"
            echo ""
            docker exec -ti "$_ct" bash -c "$_cmd"
            echo ""
            echo -e "  ${BOLD}Attache a la session tmux ${CYAN}calypso${NC}${BOLD} de ${_ct}${NC}  ${CYAN}(Ctrl-b
             ↪  puis d pour se detacher)${NC}"
            echo ""
            docker exec -ti "$_ct" tmux attach -t calypso
        else
            # ── Several containers: detached ──────────────────────────────────
            # One terminal cannot attach to N tmux sessions.
            #
            # `docker exec -d`, not `-t`: a pty is closed by docker when the script returns
            # and the kernel sends SIGHUP to the foreground group, i.e. to every daemon the
            # run_modules started with `cmd &`. (The modules also use setsid, see
            # run_modules/_lib/radio.sh.) No pty, no hangup.
            #
            # Output goes, INSIDE the container, to /var/log/osmocom/start-direct.log
            # (mounted on the host as /tmp/osmocom-logs/opN). `wait_bb_vty` returns only once
            # the container's stack is online, so containers start one at a time.
            echo -e "  ${BOLD}${n_operators} conteneurs : lancement detache - journal
             ↪  ${CYAN}/var/log/osmocom/start-direct.log${NC}${BOLD} dans chaque conteneur.${NC}"
            echo ""
            for _c in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
                _ct="$(op_container "$_c")"; _cmd="$(_hcmd "$_c")"
                echo -e "  ${GREEN}──── ${_ct} ──────────────────────────────────${NC}"
                docker exec -d "$_ct" bash -c "mkdir -p /var/log/osmocom && { ${_cmd}; } >
                 ↪  /var/log/osmocom/start-direct.log 2>&1"
                wait_bb_vty "$_ct" || echo -e "    ${RED}pile non prete - voir
                 ↪  /var/log/osmocom/start-direct.log${NC}"
                echo ""
            done
            echo -e "  ${BOLD}Toutes les piles tournent.${NC} Pour en suivre une :"
            echo -e "    ${CYAN}docker exec -ti <conteneur> tmux attach -t calypso${NC}"
            echo -e "  ${CYAN}Ctrl-b puis d${NC} pour se detacher sans rien arreter."
            echo ""
        fi
    fi
}

# ═══════════════════════════════════════════════════════════════════════
# net-host mode
# ═══════════════════════════════════════════════════════════════════════
start_host_mode() {
    echo -e "${GREEN}Demarrage en mode net-host (1 operateur)...${NC}"
    local src_ip gw_ip
    gw_ip=$(ip route get 1 | awk '{print $3; exit}')
    src_ip=$(ip route get 1 | awk '{print $7; exit}')
    [ -z "$src_ip" ] && { echo -e "${RED}Impossible de detecter l'IP hote.${NC}"; exit 1; }
    HOST_IP="$src_ip"

    local n_ms
    n_ms=$(wt_input "net-host" "Nombre de MS :" "1") || exit 1
    n_ms=${n_ms:-1}

    prepare_host_tun
    docker rm -f egprs &>/dev/null || true

    SMS_ROUTING_DIR=$(mktemp -d)
    if declare -f sms_routing_generate > /dev/null 2>&1; then
```

```
        sms_routing_generate 1 1 "$SMS_ROUTING_DIR" "$n_ms"
    fi

    local tmpdir; tmpdir=$(mktemp -d)
    apply_config_templates "$tmpdir" \
        "$src_ip" "$gw_ip" \
        "1" "1.1.1" "1.1.2" "1.1.3" \
        "001" "01" "OsmoGSM" \
        "127.0.0.1" "shutdown" "1"

    local vol_args alsa_args
    vol_args=$(build_vol_args "$tmpdir")
    alsa_args=$(build_alsa_args)

    ensure_host_audio_relay
    local nethost_relay=""
    [ -n "$HOST_AUDIO_RELAY" ] && nethost_relay="tcp:127.0.0.1:${WSLG_TCP_PORT}"

    # --rm KEPT here, unlike operator containers: this one runs run.sh (not "sleep
    # infinity") and stops when run.sh returns; --restart would loop on the HOST
    # network, competing with the gnome-terminal session opened right after. It also
    # depends on prepare_host_tun (apn0), gone after a reboot. Restart this mode
    # manually.
    # shellcheck disable=SC2086
    docker run -d --rm --name egprs --net host \
        --cap-add NET_ADMIN --cap-add SYS_ADMIN --cap-add NET_RAW --cap-add SYS_NICE \
        --security-opt apparmor=unconfined \
        --shm-size=8g \
        --cgroupns host \
        --device /dev/net/tun:/dev/net/tun \
        $alsa_args \
        -v /sys/fs/cgroup:/sys/fs/cgroup:rw \
        --tmpfs /run --tmpfs /run/lock --tmpfs /tmp:exec,rw,mode=1777,size=2g \
        -e CONTAINER_IP="$src_ip" -e GATEWAY_IP="$gw_ip" \
        -e OPERATOR_ID="1" -e N_MS="$n_ms" \
        -e INTER_STP_IP="127.0.0.1" \
        -e HOST_IP="${HOST_IP}" -e SIP_HOST_PORT="5060" \
        -e ALSA_OUTPUT="${ALSA_OUTPUT}" -e ALSA_INPUT="${ALSA_INPUT}" \
        -e HOST_AUDIO_RELAY="${nethost_relay}" \
        $vol_args \
        "$IMAGE_RUN" /root/run.sh

    wait_bb_vty "egprs"
    sleep 3

    local _du="${SUDO_USER:-$(logname 2>/dev/null || echo "$USER")}"
    local _duid; _duid=$(id -u "$_du" 2>/dev/null)
    local _disp="${DISPLAY:-:0}"
    local _xauth="${XAUTHORITY:-/home/$_du/.Xauthority}"
    local _hscript="/tmp/osmo-gnome-host.sh"
    cat > "$_hscript" <<'EOF'
#!/usr/bin/env bash
printf '\033]0;net-host - egprs\007'
echo "=== net-host - egprs run.sh ==="
exec sudo docker exec -ti egprs /bin/bash -c "/root/run.sh; exec bash"
EOF
    chmod +x "$_hscript"
    # ── OSMO_NO_TERM=1: start the stack without a window ──────────────────
    # When launched by a systemd unit (osmo-multi.service, and the desktop icon using
    # it) there is no screen: run.sh is started detached in the container and
    # everything is in the container log (`docker logs egprs`).
    if [ "${OSMO_NO_TERM:-0}" = "1" ]; then
        echo -e "  ${CYAN}[*] run.sh dans egprs (OSMO_NO_TERM=1 : pas de terminal)${NC}"
        docker exec -d egprs /bin/bash -c "/root/run.sh > /var/log/osmocom/run.sh.log 2>&1"
        sleep 0.3
        return 0
```

```
    fi
    echo -e "  ${CYAN}[*] run.sh → gnome-terminal${NC}"
    sudo -u "$_du" \
        DISPLAY="$_disp" XAUTHORITY="$_xauth" \
        XDG_RUNTIME_DIR="/run/user/${_duid}" \
        DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/${_duid}/bus" \
        gnome-terminal -- bash "$_hscript" &
    sleep 0.3
}

# ═══════════════════════════════════════════════════════════════
# Stop
# ═══════════════════════════════════════════════════════════════
stop_all() {
    echo -e "${YELLOW}Arret de tous les containers Osmocom...${NC}"
    docker ps -a --filter "name=osmo-" --format "{{.Names}}" | xargs -r docker rm -f 2>/dev/null || true
    docker ps -a --filter "name=egprs"  --format "{{.Names}}" | xargs -r docker rm -f 2>/dev/null || true
    for _chain in OSMO_WAN_INTEROP OSMO_WAN_MESH; do
        iptables -t nat -D PREROUTING -j "$_chain" 2>/dev/null || true
        iptables -t nat -F "$_chain" 2>/dev/null || true
        iptables -t nat -X "$_chain" 2>/dev/null || true
    done
    echo -e "${GREEN}Arrete.${NC}"
    disable_user_loopback
}

# ═══════════════════════════════════════════════════════════════
# Menus
# ═══════════════════════════════════════════════════════════════
choose_build_mode() {
    [ "${QUICK_EXPLICIT:-0}" = "1" ] && return 0
    local choice
    choice=$(wt_menu "Build de l'image" "Comment (re)construire l'image run ?" \
        "quick"  "Quick - reutilise le cache docker (rapide)" \
        "normal" "Normal - docker build --no-cache (image a neuf)") \
        || { echo "Annule."; exit 1; }
    case "$choice" in
        quick)  QUICK=1 ;;
        normal) QUICK=0 ;;
    esac
}

choose_ran_host() {
    local r
    r=$(wt_menu "RAN - Pipeline QEMU Calypso" \
        "Pipeline radio (conteneur de l'operateur) :" \
        "full-grgsm"   "gr-gsm = le DSP (defaut, valide)" \
        "start-clean"  "Pipeline historique (start-clean.sh)" \
        "full"         "full radio + vrai c54x (WIP)" \
        "shunt"        "DSP shunt canned (bissection FBSB)" \
        "bare"         "QEMU + osmocon only" \
        "faketrx-qemu" "COMBINE : fake_trx vivant + Calypso QEMU" \
        "free"         "menu complet run.sh (--menu)") || exit 1

    if [ "$r" = "faketrx-qemu" ]; then
        HANDOFF_MODE="faketrx-qemu"; HANDOFF_QEMU_CHOICE="faketrx-qemu"
    else
        HANDOFF_MODE="qemu";         HANDOFF_QEMU_CHOICE="$r"
    fi

    local e
    e=$(wt_menu "Chiffrement A5" "Chiffrement :" \
        "1" "A5/1 - chiffrement legacy (defaut)" \
        "0" "A5/0 - pas de chiffrement" \
        "2" "A5/2 - (casse, usage test)") || exit 1
    case "$e" in 0) ENCRYPTION="a5 0" ;; 2) ENCRYPTION="a5 2" ;; *) ENCRYPTION="a5 1" ;; esac
```

```
    echo -e "  ${GREEN}[RAN] mode=${CYAN}${HANDOFF_MODE}${NC}${GREEN}
     ↪  pipeline=${CYAN}${HANDOFF_QEMU_CHOICE}${NC}${GREEN} chiffrement=${CYAN}${ENCRYPTION}${NC}"
}

choose_network_mode() {
    local choice
    choice=$(wt_menu "Que lancer ?" "Premier choix :" \
        "qemu"    "PoC QEMU Calypso - qosmo-grgsm/run.sh (defaut)" \
        "virtual" "Reseau VIRTUEL multi-operateurs SS7 (config niveau 2)" \
        "hw"      "SDR physique (net-host) - hw (in dev)") || { echo "Annule."; exit 1; }
    case "$choice" in
        qemu)    NETWORK_MODE="qemu" ;;
        virtual) NETWORK_MODE="bridge" ;;
        hw)      NETWORK_MODE="host" ;;
        *) echo -e "${RED}Choix invalide.${NC}"; exit 1 ;;
    esac
}

# ═══════════════════════════════════════════════════════════════════════
# Main
# ═══════════════════════════════════════════════════════════════════════
# ── Long options, extracted BEFORE the positional mode ──────────────────
# The mode is in $1 (qemu|virtual|hw|stop) and the build in quick|normal; this
# pre-pass only removes the long options and keeps the original positionals.
usage_wan() {
    cat <<'USAGE'
Usage : sudo ./launch/start-docker.sh [quick|normal] [--wan ...] [qemu|virtual|hw|stop]

  --wan                   active le WAN a N noeuds (1 a 9) et pose les questions :
                            • nombre de noeuds
                            • IP publique de chaque noeud
                            • indicatif de chaque noeud (prefixe d'appel)
                            • numero du noeud construit par CE lancement
  --wan-nodes "1:IP:IND ..."  meme chose sans question (scriptable)
  --wan-id N              numero du noeud local (sinon deduit des IP locales)
  --wan-conf FICHIER      table a lire/ecrire (defaut /etc/osmo-wan.conf)

  --virtualbox[=N]        monte le WAN entre CETTE machine et des VM VirtualBox
                          (implique --wan). Cette machine devient un noeud, les
                          autres sont des VM sur un segment host-only.
  --vbox-node N           numero de noeud porte par cette machine (defaut 1)
  --hub-node N            noeud qui heberge l'inter-STP, le hub SS7 (defaut 1).
  --hub-ip ADRESSE        inter-STP a une adresse a lui, hors table WAN. Le hub
                          n'est alors PAS un noeud : aucun indicatif, aucun
                          abonne, et rien n'est lance localement.
  --node-per-op           chaque conteneur est un NOEUD (base = --wan-id) et non
                          un operateur de la machine : conteneur 1 -> noeud N,
                          conteneur 2 -> noeud N+1... Point codes 1.<n>1.<role>.
  --host ADRESSE          adresse de CETTE machine vue des VM (defaut : deduite
                          de la route par defaut). Recours quand la machine a
                          plusieurs cartes ou vient de changer d'adresse : c'est
                          par elle que les VM routent vers les conteneurs.
  --operator IP:PREFIXE   declare un operateur du WAN par son adresse et son
                          prefixe d'appel. Repetable ; l'ordre donne les numeros
                          de noeud. Il DECLARE seulement : aucun conteneur n'est
                          lance pour lui - c'est ainsi qu'une VM deja demarree
                          entre dans la table SMS et le dialplan. Les conteneurs
                          restent gouvernes par le nombre d'operateurs demande.
                            --operator 192.168.1.123:11   (la VM, deja lancee)
                            --operator 172.20.0.11:22   (un conteneur d'ici)
                          UN SEUL noeud du WAN doit le porter.
  --build-stp             construit l'image legere osmocom-stp (Dockerfile.stp)
                          pour le hub. Sans elle, le hub tourne sur l'image
                          complete, qui contient aussi osmo-stp.

  Sans --wan, rien ne change : aucun WAN n'est monte.
```

```
  Avec --wan, la radio passe en HYBRIDE par defaut (1 faketrx + 1 QEMU Calypso).

  Composer <indicatif><numero> joint ce numero sur le noeud correspondant.
    ex. noeud 1 = indicatif 11, noeud 2 = 22 → depuis 2, "1110001" appelle
    le MS 10001 de l'operateur 1 du noeud 1.
USAGE
}

_pos_args=()
while [ $# -gt 0 ]; do
    case "$1" in
        --wan)          WAN_MESH=1 ;;
        --wan=*)        WAN_MESH=1; WAN_NODES="${1#*=}" ;;
        --wan-nodes)    WAN_MESH=1; WAN_NODES="${2:-}"; shift ;;
        --wan-nodes=*)  WAN_MESH=1; WAN_NODES="${1#*=}" ;;
        --wan-id)       WAN_MESH=1; WAN_NODE_ID="${2:-}"; shift ;;
        --wan-id=*)     WAN_MESH=1; WAN_NODE_ID="${1#*=}" ;;
        --wan-conf)     WAN_CONF_FILE="${2:-}"; shift ;;
        --wan-conf=*)   WAN_CONF_FILE="${1#*=}" ;;
        --build-stp)      BUILD_STP=1 ;;
        --hub-node)       WAN_HUB_NODE="${2:-1}"; WAN_HUB_NODE_GIVEN=1; shift ;;
        # --hub-ip and --node-per-op IMPLY the WAN plan. Without WAN_MESH=1, _pc_mid
        # falls back to the LOCAL plan: point codes 1.1.2 / 1.2.2 and routing contexts
        # 150 / 250, identical on EVERY WAN machine. Two nodes on the same hub would then
        # share PC and context and, with the hub in traffic-mode override, the last ASP
        # attached would silently steal the first one's traffic.
        --hub-ip)         WAN_HUB_IP="${2:-}"; WAN_MESH=1; shift ;;
        --node-per-op)    WAN_NODE_PER_OP=1; WAN_NODE_PER_OP_GIVEN=1; WAN_MESH=1 ;;
        --host)           HOST_LAN_IP="${2:-}"; shift ;;
        --host=*)         HOST_LAN_IP="${1#*=}" ;;
        --operators)      OPERATOR_COUNT_HINT="${2:-}"; shift ;;
        --operators=*)    OPERATOR_COUNT_HINT="${1#*=}" ;;
        --operator)       OPERATOR_DECLS+=("${2:-}"); WAN_MESH=1; shift ;;
        --operator=*)     OPERATOR_DECLS+=("${1#*=}"); WAN_MESH=1 ;;
        --hub-ip=*)       WAN_HUB_IP="${1#*=}"; WAN_MESH=1 ;;
        --hub-node=*)     WAN_HUB_NODE="${1#*=}" ;;
        --virtualbox)     WAN_MESH=1; VBOX_INTERCO=1 ;;
        --virtualbox=*)   WAN_MESH=1; VBOX_INTERCO=1; VBOX_NODES="${1#*=}" ;;
        --vbox-node)      VBOX_HOST_NODE="${2:-1}"; shift ;;
        --vbox-node=*)    VBOX_HOST_NODE="${1#*=}" ;;
        -h|--help)      usage_wan; exit 0 ;;
        *)              _pos_args+=("$1") ;;
    esac
    shift
done

# ── --operator: from the declaration to the WAN table ──────────────────
# "IP:PREFIX" is what the user sees; "id:ip:ind" is what the rest of the code
# expects. Converted here, once.
if [ "${#OPERATOR_DECLS[@]}" -gt 0 ]; then
    if [ -n "${WAN_NODES:-}" ]; then
        echo -e "\033[0;31m--operator et --wan-nodes sont exclusifs : choisissez l'un des deux.\033[0m" >&2
        exit 2
    fi
    # CONTAINERS FIRST, in order: osmo-operator-1 is node 1, -2 node 2... Their
    # address is the backbone one, their prefix the rank default. Operators declared
    # on the CLI take the following ranks.
    _n=0; _spec=""
    _ncont="${OPERATOR_COUNT_HINT:-0}"
    while [ "$_n" -lt "$_ncont" ]; do
        _n=$(( _n + 1 ))
        _spec="${_spec}${_spec:+ }${_n}:172.20.0.$(( 10 + _n )):$(( _n * 11 ))"
    done
    for _d in "${OPERATOR_DECLS[@]}"; do
        _ip="${_d%%:*}"; _ind="${_d##*:}"
        if [ -z "$_ip" ] || [ "$_ip" = "$_d" ] || [ -z "$_ind" ]; then
```

```
            echo -e "\033[0;31m--operator : attendu IP:PREFIXE (ex: 192.168.1.123:11), recu '$_d'\033[0m" >&2
            exit 2
        fi
        _n=$(( _n + 1 ))
        _spec="${_spec}${_spec:+ }${_n}:${_ip}:${_ind}"
    done
    WAN_NODES="$_spec"
    echo -e "\033[0;36m  Operateurs declares :\033[0m $WAN_NODES"
fi

set -- ${_pos_args[@]+"${_pos_args[@]}"}

banner
[ "${1:-}" = "stop" ] && { stop_all; exit 0; }
[ "$(id -u)" -ne 0 ] && { echo -e "${RED}Root requis${NC}"; exit 1; }

case "${1:-}" in
    quick)  QUICK=1; QUICK_EXPLICIT=1; shift ;;
    normal) QUICK=0; QUICK_EXPLICIT=1; shift ;;
esac

docker rm -f $(docker ps -aq --filter "name=osmo-") 2>/dev/null || true
docker rm -f $(docker ps -aq --filter "name=egprs") 2>/dev/null || true
docker network ls --filter "name=gsm-" -q | xargs -r docker network rm 2>/dev/null || true

# Mode selection
case "${1:-}" in
    qemu)    NETWORK_MODE="qemu" ;;
    virtual) NETWORK_MODE="bridge" ;;
    hw)      NETWORK_MODE="host" ;;
    "")
        if [ "${WAN_MESH:-0}" = "1" ]; then
            # --wan without a mode: a WAN runs on operators, so virtual mode; the build
            # menu (quick/normal) is still shown unless decided on the command line.
            NETWORK_MODE="bridge"
            [ "${QUICK_EXPLICIT:-0}" = "1" ] || { check_whiptail; choose_build_mode; }
        elif [ "${OSMO_MULTI:-0}" = "1" ]; then
            NETWORK_MODE="bridge"
        elif [ "${OSMO_MENU:-1}" = "0" ]; then
            NETWORK_MODE="qemu"
        else
            check_whiptail
            choose_build_mode
            choose_network_mode
        fi
        ;;
    *) echo -e "${RED}Mode inconnu : '$1' (attendu : qemu|virtual|hw|stop)${NC}"; exit 1 ;;
esac

[ "$NETWORK_MODE" = "bridge" ] && check_whiptail
echo -e "${GREEN}Mode : ${CYAN}${NETWORK_MODE}${NC}  ${GREEN}Build : ${CYAN}$([ "${QUICK:-0}" = "1" ] && echo
 ↪  "quick (cache)" || echo "normal (--no-cache)")${NC}"

./helpers/prepare_host.sh
# OSMO_NO_BUILD=1 (start-multi.sh): use the image as is. QEMU_CACHE_BUST breaks
# the cache on every build (git pull + qosmo/c54x_exe rebuild). check_image
# still builds it if missing.
[ "${OSMO_NO_BUILD:-0}" = 1 ] || build_run_image
check_image

case "$NETWORK_MODE" in
    qemu)   start_qemu_poc ;;
    host)   start_host_mode ;;
    bridge) start_bridge_mode ;;
esac
```

2.92 /opt/GSM/osmo-operator/launch/start-interstp.sh

18970 octets, 387 lignes → 387 lignes

```
#!/bin/bash
# ============================================================================
# start-interstp.sh - start the INTER-STP node: the SS7 hub shared over the WAN
# ============================================================================
#
# On a single machine the inter-STP is one container among others (172.20.0.10)
# serving only that machine's operators. When operators are spread over several
# nodes it becomes a separate device with its own address to which every node
# attaches; it alone carries SS7 across the WAN. Nodes reach it over
# M3UA/SCTP (port 2908 by default, see --port).
#
#   node 1 (1.11.2) ─┐
#   node 2 (1.21.2) ─┼─ M3UA/SCTP 2908 ─► inter-STP 192.168.1.49    PC 0.0.0
#   node 3 (1.31.2) ─┘
#
# POINT CODE PLAN
# On an isolated machine operator N uses 1.N.2, computed identically on every
# machine, so nodes sharing a hub would present duplicate point codes (wrong
# routing). The WAN plan therefore encodes the node number:
#
#     PC   = 1.<node><op>.<role>     role 1=MSC 2=STP 3=BSC
#     RCTX = node*1000 + op*100 + 50
#
# -h / --help prints this header.
#
# Usage:
#   sudo ./launch/start-interstp.sh [--nodes 3] [--ops 1] [--ip 192.168.1.49]
#   sudo ./launch/start-interstp.sh --status | --stop
#
#   --nodes N     number of nodes served (1-9, default: read from /etc/osmo-wan.conf)
#   --ops K       operators per node (1-9, default 1)
#   --ip ADDRESS  hub address; added to the interface if missing
#   --iface NAME  interface to put the address on (default: auto-detected)
#   --port N      M3UA port (default 2908)
#   --menu        ask the questions even if a table already exists
#   --defaut      apply the default lab table, ignoring the one already stored
#   --ips "a b c" operator node IPs, in order (no questions);
#                 "id:ip" keeps the node number (table 1,3 -> nodes 1 and 3)
#   --no-save     do not rewrite /etc/osmo-wan.conf (used by start-native.sh)
#   --no-ip       do not touch the network configuration
#   --foreground  stay in the foreground (log on screen)
# ============================================================================
set -uo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

# Repository root (this script lives in launch/).
HERE="$(cd "$(dirname "$(readlink -f "${BASH_SOURCE[0]}")")/.." && pwd)"
CONF_DIR="${CONF_DIR:-/etc/osmocom}"
CFG="${CONF_DIR}/osmo-stp-interop.cfg"
LOG="${LOG:-/var/log/osmocom/osmo-stp-interop.log}"
PIDFILE="/run/osmo-interstp.pid"

NODES=""
OPS=1
MENU=0
USE_DEFAULTS=0
IPS=""
SAVE_TABLE=1
# Default hub: the osmo-inter-stp container on the docker backbone
# (INTER_STP_IP in start-docker.sh).
```

```
HUB_IP="172.20.0.10"
# ── Default lab table, default for all prompts ──────────────────────────
# Must match the table written by build-iso.sh, otherwise point codes on one
# side no longer match the AS declared on the other.
#   node 1  192.168.1.123  the VM              prefix 11
#   node 2  172.20.0.12  osmo-operator-2    prefix 22   (container)
#   node 3  172.20.0.13  osmo-operator-3    prefix 33   (container)
#   hub     172.20.0.10  osmo-inter-stp
# MSISDN = <node>00<op><ms> (100101 = node 1, op 1, MS 1).
WAN_NODES_DEFAULT="1:192.168.1.123:11 2:172.20.0.12:22 3:172.20.0.13:33"

# Default address of node i, read from this table.
_default_node_ip() {
    printf '%s' "$WAN_NODES_DEFAULT" | tr ' ' '\n' \
        | awk -F: -v n="$1" '$1==n{print $2; exit}'
}
IFACE=""
PORT=2908
SET_IP=1
FOREGROUND=0
ACTION=start

while [ $# -gt 0 ]; do
    case "$1" in
        --nodes)      NODES="${2:-}"; shift ;;
        --nodes=*)    NODES="${1#*=}" ;;
        --ops)        OPS="${2:-1}"; shift ;;
        --ops=*)      OPS="${1#*=}" ;;
        --ip)         HUB_IP="${2:-}"; shift ;;
        --ip=*)       HUB_IP="${1#*=}" ;;
        --iface)      IFACE="${2:-}"; shift ;;
        --iface=*)    IFACE="${1#*=}" ;;
        --port)       PORT="${2:-2908}"; shift ;;
        --port=*)     PORT="${1#*=}" ;;
        --menu)       MENU=1 ;;
        --defaut|--default) USE_DEFAULTS=1 ;;
        --ips)        IPS="${2:-}"; shift ;;
        --ips=*)      IPS="${1#*=}" ;;
        --no-ip)      SET_IP=0 ;;
        --no-save)    SAVE_TABLE=0 ;;
        --foreground) FOREGROUND=1 ;;
        --status)     ACTION=status ;;
        --stop)       ACTION=stop ;;
        -h|--help)    sed -n '2,40p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; exit 2 ;;
    esac
    shift
done

banner() {
    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════════════════════════════════╗"
    echo "║   Inter-STP - hub SS7 du WAN osmo-operator   ·   PC 0.0.0             ║"
    echo "╚═══════════════════════════════════════════════════════════════════════╝"
    echo -e "${NC}"
}

stp_pid() {
    if [ -r "$PIDFILE" ] && kill -0 "$(cat "$PIDFILE")" 2>/dev/null; then
        cat "$PIDFILE"; return 0
    fi
    pgrep -f 'osmo-stp .*osmo-stp-interop\.cfg' | head -1
}

# Hub VTY port. 4239 is the node's local STP default, so on a machine that
# already runs osmo-stp the hub cannot use it. Read the port from the config
```

```
# (bind 127.0.0.1 N), else 4239.
vty_port() {
    local p
    p="$(sed -n 's/^ *bind 127\.0\.0\.1 \([0-9]*\)$/\1/p' "$CFG" 2>/dev/null | head -1)"
    echo "${p:-4239}"
}

# ── status ──────────────────────────────────────────────────────────
if [ "$ACTION" = "status" ]; then
    banner
    pid="$(stp_pid)"
    if [ -z "$pid" ]; then
        echo -e "  ${RED}✗${NC} inter-STP arrete"; exit 1
    fi
    echo -e "  ${GREEN}✓${NC} inter-STP actif (pid ${pid})"
    echo -e "  Ecoute : $(ss -lna 2>/dev/null | grep -E "[:.]${PORT}\b" | head -3 | tr -s ' ' | cut -d' ' -f1,5 |
     ↪  tr '\n' ' ')"
    echo ""
    # The VTY is the only reliable source for WHO is attached: an ASP may have
    # opened its SCTP association without ever reaching ASP-ACTIVE. An open port
    # only proves the hub is listening.
    VTY_PORT="$(vty_port)"
    if command -v nc >/dev/null 2>&1; then
        echo -e "  ${BOLD}ASP attaches :${NC}"
        # Pause between commands: the VTY does not read a whole block at once.
        { sleep 1; printf 'enable\n'; sleep 0.5; printf 'show cs7 instance 0 asp\n'; sleep 0.5
          printf 'show cs7 instance 0 as all\n'; sleep 0.5; printf 'exit\n'; sleep 1; } \
          | nc -q2 127.0.0.1 "$VTY_PORT" 2>/dev/null | tr -d '\r' | sed 's/^/    /' | grep -vE '^\s*$' | tail -30 \
          || echo -e "    ${YELLOW}VTY ${VTY_PORT} muet${NC}"
    else
        echo -e "  ${YELLOW}nc absent : ${CYAN}telnet 127.0.0.1 ${VTY_PORT}${NC} puis "show cs7 instance 0 asp""
    fi
    exit 0
fi

# ── stop ────────────────────────────────────────────────────────────
if [ "$ACTION" = "stop" ]; then
    pid="$(stp_pid)"
    if [ -n "$pid" ]; then
        kill "$pid" 2>/dev/null; sleep 1
        kill -9 "$pid" 2>/dev/null || true
        rm -f "$PIDFILE"
        echo -e "${GREEN}Inter-STP arrete.${NC}"
    else
        echo -e "${YELLOW}Inter-STP deja arrete.${NC}"
    fi
    exit 0
fi

banner
[ "$(id -u)" -eq 0 ] || { echo -e "${RED}Root requis.${NC}" >&2; exit 1; }
command -v osmo-stp >/dev/null 2>&1 || {
    echo -e "${RED}osmo-stp introuvable.${NC} Ce noeud doit porter la pile Osmocom." >&2; exit 1; }

# ═══════════════════════════════════════════════════════════════════
# Who are the nodes? - the menu
# ═══════════════════════════════════════════════════════════════════
# The hub does not need the IPs to ROUTE: it listens, ASPs connect to it
# (accept-asp-connections dynamic-permitted) and the point code decides the
# rest. It needs them for:
#   • opening its firewall to these addresses only;
#   • writing /etc/osmo-wan.conf, read by the nodes and the diagnostics;
#   • telling in --status WHO should be attached (otherwise a missing node
#     cannot be told apart from a node that does not exist).
# shellcheck source=network/wan-nodes.sh
. "$HERE/network/wan-nodes.sh"
```

```
ask_ips() {
    local n i ip def
    echo -e "${BOLD}Configuration du hub SS7${NC}"
    echo -e "  Saisissez les adresses des noeuds operateurs, dans l'ordre."
    echo ""
    n=$(_wan_ask "Inter-STP" "Nombre de noeuds operateurs (1-9) :" "${NODES:-3}") || exit 1
    [[ "$n" =~ ^[1-9]$ ]] || { echo -e "${RED}Nombre invalide : $n${NC}" >&2; exit 2; }

    WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=()
    for i in $(seq 1 "$n"); do
        # Default: the lab table (the first nodes are containers on the docker
        # backbone, not on the hub's LAN segment).
        def="$(_default_node_ip "$i")"
        [ -n "$def" ] || def="${HUB_IP%.*}.$((10 + i))"
        ip=$(_wan_ask "Noeud $i/$n" "IP du noeud operateur $i :" "$def") || exit 1
        [[ "$ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] \
            || { echo -e "${RED}'$ip' n'est pas une IPv4${NC}" >&2; exit 2; }
        WAN_IP[$i]="$ip"; WAN_IND[$i]="$(wan_default_ind "$i")"; WAN_NODE_LIST+=("$i")
    done
    WAN_NODE_COUNT="$n"; NODES="$n"
    # The hub carries no operator and is not a node of the table: record 0
    # (accepted by wan_nodes_validate) so it does not take a node's point code.
    WAN_NODE_ID=0
    WAN_OPS="$OPS"
}

# --defaut: use the lab table regardless of what is on the machine.
# Without it, a table already stored in /etc/osmo-wan.conf is reloaded as is,
# so after an addressing plan change the hub keeps stale addresses until
# --defaut or --menu is used.
if [ "$USE_DEFAULTS" = "1" ] && [ -z "$IPS" ]; then
    IPS="$(printf '%s' "$WAN_NODES_DEFAULT" | tr ' ' '\n' | awk -F: '{print $2}' | tr '\n' ' ')"
    [ -n "$NODES" ] || NODES="$(printf '%s' "$WAN_NODES_DEFAULT" | wc -w)"
    echo -e "  ${CYAN}--defaut : table du banc${NC} ${WAN_NODES_DEFAULT}"
fi

if [ -n "$IPS" ]; then
    # Scriptable form: --ips "10.0.0.11 10.0.0.12 10.0.0.13"
    # "id:ip" keeps the node number (non-contiguous tables such as 1,3 are
    # allowed by wan_nodes_validate). A bare IP: number = position.
    WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=(); i=0
    for tok in $IPS; do
        i=$((i+1))
        case "$tok" in *:*) id="${tok%%:*}"; ip="${tok#*:}" ;; *) id="$i"; ip="$tok" ;; esac
       [[ "$id" =~ ^[1-9]$ ]] || { echo -e "${RED}--ips : numero de noeud '$id' invalide (1-9)${NC}" >&2; exit 2; }
        WAN_IP[$id]="$ip"; WAN_IND[$id]="$(wan_default_ind "$id")"; WAN_NODE_LIST+=("$id")
    done
    WAN_NODE_COUNT="$i"; NODES="$i"; WAN_NODE_ID=0; WAN_OPS="$OPS"
elif [ "$MENU" = "1" ] || { [ -z "$NODES" ] && [ ! -r "$WAN_CONF_FILE" ]; }; then
    if [ -t 0 ]; then
        ask_ips
    else
        echo -e "${YELLOW}Pas de terminal et pas de table : on part sur 3 noeuds sans adresses.${NC}"
        NODES=3
    fi
elif [ -z "$NODES" ] && [ -r "$WAN_CONF_FILE" ]; then
    wan_nodes_load "$WAN_CONF_FILE" 2>/dev/null || true
    NODES="${WAN_NODE_COUNT:-3}"
    OPS="${WAN_OPS:-$OPS}"
    echo -e "  Table lue dans ${CYAN}${WAN_CONF_FILE}${NC} : ${NODES} noeud(s)"
    echo -e "  ${CYAN}--menu${NC} pour la ressaisir."
fi

[[ "$NODES" =~ ^[1-9]$ ]] || { echo -e "${RED}--nodes : 1 a 9${NC}" >&2; exit 2; }
[[ "$OPS"   =~ ^[1-9]$ ]] || { echo -e "${RED}--ops : 1 a 9${NC}" >&2; exit 2; }
```

```
# Known table: persist it and open the firewall for these addresses.
if [ "${#WAN_NODE_LIST[@]}" -gt 0 ] && [ -n "${WAN_IP[${WAN_NODE_LIST[0]}]:-}" ]; then
    echo ""
    wan_nodes_summary 2>/dev/null || true
    # --no-save: called by start-native.sh, which has just written the table
    # (WAN_AUTO=1, node number, prefixes); rewriting it here with WAN_AUTO=0 /
    # node 0 would disable the WAN on the next boot.
    if [ "$SAVE_TABLE" = 1 ]; then
        WAN_AUTO=0 wan_nodes_save "$WAN_CONF_FILE" \
            && echo -e "  ${GREEN}✓${NC} table ecrite dans ${CYAN}${WAN_CONF_FILE}${NC}"
    else
        echo -e "  ${CYAN}--no-save${NC} : ${WAN_CONF_FILE} laisse tel quel"
    fi
    if [ -x "$HERE/network/firewall-wan.sh" ] || [ -r "$HERE/network/firewall-wan.sh" ]; then
        for i in "${WAN_NODE_LIST[@]}"; do
            bash "$HERE/network/firewall-wan.sh" "${WAN_IP[$i]}" "$OPS" >/dev/null 2>&1 || true
        done
        echo -e "  ${GREEN}✓${NC} pare-feu ouvert pour ${#WAN_NODE_LIST[@]} noeud(s)"
    fi
fi

# ── Hub address ──────────────────────────────────────────────────────
# It is hard-coded in EVERY node's config: if it is missing at startup, all
# ASPs fail to attach and the only symptom is "connection refused" on the
# node side, never on the hub.
if [ "$SET_IP" = "1" ]; then
    if ip -4 addr show 2>/dev/null | grep -q "inet ${HUB_IP}/"; then
        echo -e "  ${GREEN}✓${NC} ${CYAN}${HUB_IP}${NC} deja presente sur cette machine"
    else
        if [ -z "$IFACE" ]; then
            # The interface already carrying the same /24 is the WAN segment.
            IFACE="$(ip -4 -o addr show scope global 2>/dev/null \
                | awk -v pfx="${HUB_IP%.*}." '$4 ~ "^"pfx {print $2; exit}')"
            [ -z "$IFACE" ] && IFACE="$(ip -4 -o addr show scope global 2>/dev/null | awk 'NR==1{print $2}')"
        fi
        if [ -n "$IFACE" ] && ip addr add "${HUB_IP}/24" dev "$IFACE" 2>/dev/null; then
            echo -e "  ${GREEN}✓${NC} ${CYAN}${HUB_IP}/24${NC} posee sur ${IFACE}"
        else
            echo -e "  ${YELLOW}⚠${NC} impossible de poser ${HUB_IP} (interface ${IFACE:-?})"
            echo -e "     Les noeuds ne pourront pas s'attacher tant qu'elle manque."
        fi
    fi
fi

# ── Config du hub ────────────────────────────────────────────────────
mkdir -p "$CONF_DIR" "$(dirname "$LOG")"
if [ ! -x "$HERE/helpers/create_interop.sh" ] && [ ! -r "$HERE/helpers/create_interop.sh" ]; then
    echo -e "${RED}helpers/create_interop.sh introuvable${NC}" >&2; exit 1
fi
# Previous hub's VTY port, read BEFORE regeneration so the new hub reuses it
# (scanning for a free port while the old hub still runs would drift by 1
# on each restart).
VTY_PREC="$(sed -n 's/^ *bind 127\.0\.0\.1 \([0-9]*\)$/\1/p' "$CFG" 2>/dev/null | head -1)"
# --listen-ip: the hub binds to ITS address, not 0.0.0.0. Otherwise SCTP
# advertises every local address in its INIT, nodes try dead paths and the
# ASP flaps up and down.
WAN_NODE_IDS="${WAN_NODE_LIST[*]:-}" bash "$HERE/helpers/create_interop.sh" --listen-ip "$HUB_IP" \
     --wan "$NODES" "$OPS" "$CFG" || exit 1
[ "$PORT" = "2908" ] || sed -i "s/^ listen m3ua 2908/ listen m3ua ${PORT}/" "$CFG"
# Hub VTY: 4239 while free (the port read by checks/diag-interstp.sh and
# vty-debug-dump.sh; nothing else uses it on a dedicated hub). Otherwise the
# previous hub's port, else the first free one in 4300-4399 (vty_port reads
# it back from the config).
if ! grep -q '^ *bind 127\.0\.0\.1 [0-9]' "$CFG"; then
    VTY_PORT="${VTY_PORT:-}"
```

```
    hub_pid="$(stp_pid)"
    _port_pris() {  # taken by something other than the hub being replaced
        local who; who="$(ss -lntpH "( sport = :$1 )" 2>/dev/null)"
        [ -n "$who" ] && ! { [ -n "$hub_pid" ] && printf '%s' "$who" | grep -q "pid=$hub_pid,"; }
    }
    # Hub co-located with an operator STP: the operator takes VTY 4239 (its
    # default) when run.sh starts it, AFTER this hub. If the hub took 4239 now,
    # run.sh (run_modules/10-stp.sh, mod_stp_status checks core_vty_listen 4239)
    # would think OsmoSTP is already up and skip the operator STP, leaving the
    # MSC without an STP. So 4239 is reserved for the operator whenever an
    # operator osmo-stp.cfg is present here.
    _op_stp_4239=0
    if [ -r "$CONF_DIR/osmo-stp.cfg" ] && [ "$CONF_DIR/osmo-stp.cfg" != "$CFG" ] \
       && ! grep -q '^ *bind 127\.0\.0\.1 [0-9]' "$CONF_DIR/osmo-stp.cfg"; then
        _op_stp_4239=1
    fi
    if [ -z "$VTY_PORT" ]; then
        if [ "$_op_stp_4239" = 0 ] && ! _port_pris 4239; then VTY_PORT=4239
        elif [ -n "$VTY_PREC" ] && [ "$VTY_PREC" != 4239 ] && ! _port_pris "$VTY_PREC"; then VTY_PORT="$VTY_PREC"
        else
            for p in $(seq 4300 4399); do _port_pris "$p" || { VTY_PORT=$p; break; }; done
        fi
    fi
    [ "${VTY_PORT:-4239}" = 4239 ] || sed -i "s/^line vty$/line vty\n bind 127.0.0.1 ${VTY_PORT}/" "$CFG"
fi
echo ""

# ── Lancement ──────────────────────────────────────────────────────
if pid="$(stp_pid)" && [ -n "$pid" ]; then
    echo -e "  ${YELLOW}Un inter-STP tourne deja (pid ${pid}) - arret puis relance.${NC}"
    kill "$pid" 2>/dev/null; sleep 1; kill -9 "$pid" 2>/dev/null || true
fi

echo -e "  ${BOLD}Noeuds desservis :${NC} ${NODES} × ${OPS} operateur(s)"
for n in ${WAN_NODE_LIST[*]:-$(seq 1 "$NODES")}; do
    for o in $(seq 1 "$OPS"); do
        printf '    noeud %s op %s → STP %s  MSC %s  BSC %s  (rctx %s)\n' \
            "$n" "$o" "1.${n}${o}.2" "1.${n}${o}.1" "1.${n}${o}.3" "$(( n*1000 + o*100 + 50 ))"
    done
done
echo ""

if [ "$FOREGROUND" = "1" ]; then
    echo -e "  ${GREEN}osmo-stp au premier plan - Ctrl-C pour arreter${NC}"
    exec osmo-stp -c "$CFG"
fi

osmo-stp -c "$CFG" >> "$LOG" 2>&1 &
echo $! > "$PIDFILE"
sleep 2
if kill -0 "$(cat "$PIDFILE")" 2>/dev/null; then
    echo -e "  ${GREEN}✓ inter-STP lance${NC} - M3UA ${CYAN}${HUB_IP}:${PORT}${NC}, VTY $(vty_port)"
    echo -e "  Journal : ${CYAN}${LOG}${NC}"
    echo ""
    echo -e "  ${BOLD}Cote noeuds :${NC} leur osmo-stp doit viser ${CYAN}${HUB_IP}${NC}"
    echo -e "    ISO construite avec ${CYAN}--role operator --node N${NC} : deja fait."
    echo -e "  ${BOLD}Verifier qui est attache :${NC} ${CYAN}./launch/start-interstp.sh --status${NC}"
else
    echo -e "  ${RED}✗ osmo-stp n'a pas demarre${NC} - ${CYAN}tail -30 ${LOG}${NC}"
    tail -15 "$LOG" 2>/dev/null | sed 's/^/    /'
    exit 1
fi
```

2.93 /opt/GSM/osmo-operator/launch/start-multi.sh

37161 octets, 752 lignes → 750 lignes (1 groupes compactés)

```
#!/bin/bash
# ============================================================================
# start-multi.sh - MULTI-OPERATOR LAB: native + docker, linked over SS7
#
#   op 1  NATIVE  already running (start-native.sh)  PC 1.1.2
#   op 2  DOCKER  container                          PC 1.2.2   172.20.0.12
#   op 3  DOCKER  container                          PC 1.3.2   172.20.0.13
#   hub   DOCKER  inter-STP                          PC 0.0.0   172.20.0.10
#                                                    M3UA/SCTP 2908
#
# This script reimplements nothing. start-docker.sh already starts operator
# containers, sets up the hub (--hub-ip) and can DECLARE an operator without
# starting a container (--operator IP:PREFIX), which is the case of the
# already-running native operator. start-multi.sh only checks prerequisites,
# translates the topology into options and delegates.
#
# The topology comes from addition.sh: /etc/osmocom/osmo-multi.conf.
#
# Environment:
#   OSMO_MULTI_RESTART_NATIF=1  same as --restart-natif
#   OSMO_NO_TERM=1              never open a terminal or browser (systemd)
#
# Usage:
#   sudo ./launch/start-multi.sh              start the multi-operator lab
#   sudo ./launch/start-multi.sh --status     state of containers and hub
#   sudo ./launch/start-multi.sh --stop       stop the containers (native stays)
#   sudo ./launch/start-multi.sh --dry-run    print the command without running it
#   sudo ./launch/start-multi.sh --align      align the native SS7 identity only, start nothing
#   sudo ./launch/start-multi.sh --restart-natif
#                                     also stop and restart the native 2G
#                                     (by default it is KEPT if running and
#                                     STARTED if stopped)
# ============================================================================
set -uo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

# Repository root (this script lives in launch/).
DIR="$(cd "$(dirname "$(readlink -f "$0")")/.." && pwd)"
MULTI_CONF="${MULTI_CONF:-/etc/osmocom/osmo-multi.conf}"

# ── Native subscribers come from coeur.env, not ~/.bashrc ──────────────────
# Load N_MS and OPERATOR_ID from /etc/osmocom/coeur.env so a launch from the
# desktop icon or a systemd unit gets the right subscriber count (as
# start-native.sh does); ":=" lets a value already in the environment win.
if [ -f "${OSMOCOM_CFG:-/etc/osmocom}/coeur.env" ]; then
    set -a; . "${OSMOCOM_CFG:-/etc/osmocom}/coeur.env"; set +a
fi
ACTION="start"
# The native 2G is kept if running and started if missing. --restart-natif
# (or OSMO_MULTI_RESTART_NATIF=1) forces stop + restart.
RESTART_NATIF="${OSMO_MULTI_RESTART_NATIF:-0}"
for a in "$@"; do
    case "$a" in
        --status)  ACTION="status" ;;
        --stop)    ACTION="stop" ;;
        --dry-run) ACTION="dry" ;;
        # --align: ONLY align the native SS7 identity (aligner_natif), then exit.
        # The test bench (tests/modules/00-demarrage.sh) calls it BEFORE
        # start-native.sh so the native stack starts already aligned.
        --align)   ACTION="align" ;;
```

```
        --restart-natif) RESTART_NATIF=1 ;;
        -h|--help) sed -n '2,33p' "$0"; exit 0 ;;
    esac
done

# ── Launched from an icon: terminal and privileges ──────────────────
# The .desktop entry uses Terminal=false, so without this preamble a
# double-click would show nothing (e.g. the "root required" refusal).
# Test for a TTY, not $DISPLAY: that is the real difference between a manual
# launch and an icon launch.
# ONLY for starting the lab. --status, --dry-run and --help are read-only:
# re-running them in a graphical terminal would open a window on every call
# from a script or pipe, and their output would not reach the caller.
_multi_readonly=0
case " $* " in
    *" --status "*|*" --dry-run "*|*" --align "*|*" -h "*|*" --help "*) _multi_readonly=1 ;;
esac

# Check stdin AND stdout: a run from a real terminal with stdin redirected
# must not be mistaken for a double-click.
_multi_besoin=0
if [ "$_multi_readonly" = "0" ]; then
    { [ ! -t 0 ] || [ ! -t 1 ]; } && _multi_besoin=1
fi
# OSMO_NO_TERM=1: launched by systemd (osmo-multi.service), nobody is in
# front of a screen - open nothing, output goes to the journal.
if [ "$_multi_besoin" = "1" ] && [ "${OSMO_MULTI_TERM:-0}" != "1" ] && [ "${OSMO_NO_TERM:-0}" != "1" ]; then
    export OSMO_MULTI_TERM=1
    for _t in gnome-terminal xfce4-terminal konsole xterm; do
        command -v "$_t" >/dev/null 2>&1 || continue
        case "$_t" in
            gnome-terminal) exec "$_t" --title="osmo-operator multi-operateur" -- "$0" "$@" ;;
            *)              exec "$_t" -T "osmo-operator multi-operateur" -e "$0" "$@" ;;
        esac
    done
fi

# The flag only prevents a relaunch loop. It is REMOVED from the
# environment so it does not leak into start-docker.sh, the native shortcut
# or launch.sh. The value stays in a shell variable, which decides whether
# to keep the window open at the end.
_multi_own_window=0
[ "${OSMO_MULTI_TERM:-0}" = "1" ] && _multi_own_window=1
unset OSMO_MULTI_TERM

# Same for root: a read-only action must not ask for a password.
if [ "$_multi_readonly" = "0" ]; then
        if [ "$(id -u)" -ne 0 ]; then
            if command -v pkexec >/dev/null 2>&1 && [ -n "${DISPLAY:-}" ]; then
                exec pkexec env DISPLAY="${DISPLAY:-}" XAUTHORITY="${XAUTHORITY:-}" \
                     OSMO_MULTI_TERM="$_multi_own_window" "$0" "$@"
            fi
            command -v sudo >/dev/null 2>&1 && exec sudo -E "$0" "$@"
        fi
fi

echo -e "${CYAN}${BOLD}"
echo "╔═══════════════════════════════════════════════════════════╗"
echo "║   osmo-operator - banc MULTI-OPERATEUR (SS7)              ║"
echo "╚═══════════════════════════════════════════════════════════╝"
echo -e "${NC}"

# ── PREFLIGHT ──────────────────────────────────────────────────────
# Docker is NOT in the ISO (see addition.sh): its absence is NORMAL on the
# first launch. Say so and point to the add-on instead of letting docker
# answer "command not found".
```

```
manque() {
    echo -e "  ${RED}✗ $1${NC}"
    echo -e "    ${CYAN}→ sudo ${DIR}/addition.sh${NC}  (ou l icone ${BOLD}Supplements${NC} du bureau)"
    exit 1
}
command -v docker >/dev/null 2>&1 || manque "docker n est pas installe"
docker info >/dev/null 2>&1        || manque "le demon docker ne repond pas"

if [ ! -f "$MULTI_CONF" ]; then
    manque "topologie absente : $MULTI_CONF"
fi
# shellcheck disable=SC1090
. "$MULTI_CONF"

: "${MULTI_HUB_IP:=172.20.0.10}"
: "${MULTI_M3UA_PORT:=2908}"
: "${MULTI_OPS:=}"
: "${MULTI_IMAGE:=osmocom-run}"

# ── Read the native stack instead of assuming ──────────────────────
# The topology written by addition.sh says what the native operator SHOULD
# be; these two functions report what it IS (e.g. a running process may
# carry a container's point code).
#
# The point code depends on the chosen plan, and both exist:
#   LOCAL plan (set-node-id.sh --local) : 1.<op>.<role>        - one machine
#   WAN plan   (default)                : 1.<node><op>.<role>  - several
# On a single-machine lab the local plan applies, but the config may have
# been set otherwise: READ it, do not recompute it.
natif_pc() {
    awk '/^[[:space:]]*point-code[[:space:]]/{print $2; exit}' \
        /etc/osmocom/osmo-stp.cfg 2>/dev/null
}

# Address the native stack uses towards the hub: the local-ip of its ASP
# asp-to-inter. It must exist on the host - an unbindable local-ip is an ASP
# that never comes up, without a clear message.
natif_asp_ip() {
    local ip
    ip="$(awk '/^[[:space:]]*asp[[:space:]]+asp-to-inter[[:space:]]/{a=1; next}
               a && /^[[:space:]]*local-ip[[:space:]]/{print $2; exit}' \
          /etc/osmocom/osmo-stp.cfg 2>/dev/null)"
    [ -n "$ip" ] || return 0
    # Check that the address exists HERE. An address missing from the machine
    # means an ASP that will never come up; show it in the column instead of
    # suggesting a healthy topology.
    if ip -4 -o addr show 2>/dev/null | awk '{print $4}' | cut -d/ -f1 \
         | grep -qx "$ip"; then
        printf '%s\n' "$ip"
    else
        printf '%s(absente!)\n' "$ip"
    fi
}

# ── A host process, not a container's ──────────────────────────────
# The host sees ALL processes, including those of containers, so a plain
# pgrep would mistake any container stack for the native one. The
# discriminator is the PID NAMESPACE: a container has its own, the host
# keeps its init's. More reliable than the cgroup, which depends on
# --cgroupns.
#
# `pgrep -x`, NOT `pgrep -f`: match the process NAME, not its command line
# (which would match e.g. `tail -F .../osmo-bsc.log`, an editor or the
# calling shell). An exact name also makes excluding our own PID
# unnecessary.
_ns_hote="$(readlink /proc/1/ns/pid 2>/dev/null)"
```

```
_est_natif() {          # $@ = noms EXACTS ; vrai si l un tourne hors conteneur
    local n p
    for n in "$@"; do
        for p in $(pgrep -x "$n" 2>/dev/null); do
            [ "$(readlink "/proc/$p/ns/pid" 2>/dev/null)" = "$_ns_hote" ] && return 0
        done
    done
    return 1
}

# ── ETAT ──────────────────────────────────────────────────────────────
etat() {
    local spec idx mode ip pc _r
    echo -e "  ${BOLD}Topologie${NC} ($MULTI_CONF)"
    for spec in $MULTI_OPS; do
        IFS=: read -r idx mode ip pc _rctx <<< "$spec"
        if [ "$mode" = "native" ]; then ip="$(natif_asp_ip)"; fi
        printf '    op %-2s %-7s %-13s PC %-8s ' "$idx" "$mode" "${ip:-(hote)}" "$pc"
        if [ "$mode" = "native" ]; then
            if _est_natif osmo-bsc asterisk \
               || systemctl is-active --quiet asterisk 2>/dev/null; then
                _r="$(natif_pc)"
                if [ -n "$_r" ] && [ "$_r" != "$pc" ]; then
                    echo -e "${GREEN}actif${NC} ${YELLOW}(PC reel ${_r})${NC}"
                else
                    echo -e "${GREEN}actif${NC}"
                fi
            else
                echo -e "${YELLOW}arrete${NC}"
            fi
        else
            if docker ps --format '{{.Names}}' 2>/dev/null | grep -qx "osmo-operator-${idx}"; then
                echo -e "${GREEN}actif${NC}"
            else
                echo -e "${YELLOW}arrete${NC}"
            fi
        fi
    done
    printf '    hub  %-7s %-13s PC %-8s ' "docker" "$MULTI_HUB_IP" "${MULTI_HUB_PC:-0.0.0}"
    if docker ps --format '{{.Names}}' 2>/dev/null | grep -qx "${MULTI_HUB_NAME:-osmo-inter-stp}"; then
        echo -e "${GREEN}actif${NC}"
    else
        echo -e "${YELLOW}arrete${NC}"
    fi
}

case "$ACTION" in
  status) etat; exit 0 ;;
  stop)
        for spec in $MULTI_OPS; do
            IFS=: read -r idx mode _ <<< "$spec"
            [ "$mode" = "docker" ] || continue
            docker rm -f "osmo-operator-${idx}" >/dev/null 2>&1 \
                && echo -e "  ${GREEN}✓${NC} osmo-operator-${idx} arrete"
        done
        docker rm -f "${MULTI_HUB_NAME:-osmo-inter-stp}" >/dev/null 2>&1 \
            && echo -e "  ${GREEN}✓${NC} hub arrete"
        echo -e "  ${CYAN}i${NC} l operateur NATIF n est pas touche - ${BOLD}sudo ${DIR}/start-native.sh stop${NC}
         ↪  pour lui."
        exit 0 ;;
esac

# ── The image ─────────────────────────────────────────────────────────
# Image name: osmocom-run, NOT "osmo-operator" (the repository name).
# build.sh produces osmocom-nitb (Dockerfile, ~11 GB), then Dockerfile.run
# derives osmocom-run from it; start-docker.sh runs osmocom-run.
```

```
# If missing, pull it from norf_gsm on Docker Hub first (published with
# Dockerfile.run already applied). Deriving it locally from osmocom-nitb
# (a few seconds) is the fallback when there is no network or Hub.
: "${MULTI_HUB_RUN_IMAGE:=bastienbaranoff/norf_gsm:latest}"
if [ "$ACTION" != align ] && ! docker image inspect "$MULTI_IMAGE" >/dev/null 2>&1; then
    echo -e "  ${CYAN}→${NC} image '$MULTI_IMAGE' absente : docker pull ${MULTI_HUB_RUN_IMAGE}"
    docker pull "$MULTI_HUB_RUN_IMAGE" && docker tag "$MULTI_HUB_RUN_IMAGE" "$MULTI_IMAGE" \
        || echo -e "  ${YELLOW}!${NC} pull ${MULTI_HUB_RUN_IMAGE} impossible - repli sur la derivation locale"
fi
if [ "$ACTION" != align ] && ! docker image inspect "$MULTI_IMAGE" >/dev/null 2>&1; then
    if docker image inspect osmocom-nitb >/dev/null 2>&1; then
        echo -e "  ${CYAN}→${NC} image '$MULTI_IMAGE' absente, base osmocom-nitb presente : derivation
        ↪  (Dockerfile.run)"
        ( cd "$DIR" && docker build --build-arg QEMU_CACHE_BUST=$(date +%s) \
                        -f Dockerfile.run -t "$MULTI_IMAGE" . ) \
            || manque "derivation de l image '$MULTI_IMAGE' echouee"
    else
        manque "image '$MULTI_IMAGE' absente"
    fi
fi

# ── Native-to-hub link ──────────────────────────────────────────────
# The native /etc/osmocom/osmo-stp.cfg declares
#       asp asp-to-inter 2908 2910 m3ua
#        remote-ip 127.0.0.1
# start-docker.sh publishes the hub's 2908 in SCTP on loopback
# (docker run -p 127.0.0.1:2908:2908/sctp), so this remote-ip is valid
# without touching the regenerated configs. Pointing the native ASP at
# 172.20.0.10 instead would break whenever the docker bridge is down.

# ── How many containers ─────────────────────────────────────────────
# Those marked "docker" in the table, not a fixed count: the topology is
# the only source.
N_DOCKER=0
for spec in $MULTI_OPS; do
    IFS=: read -r idx mode _ <<< "$spec"
    [ "$mode" = "docker" ] && N_DOCKER=$((N_DOCKER + 1))
done
[ "$N_DOCKER" -ge 1 ] || { echo -e "  ${RED}✗ aucun operateur docker dans la topologie${NC}"; exit 1; }

# NO --wan here: --wan is the MULTI-MACHINE mode (reads/writes
# /etc/osmo-wan.conf, renumbers point codes as 1.<node><op>.<role>, expects
# a hub at a routable address). On ONE machine the default already points
# each operator's ASP at the local hub 172.20.0.10 with a catch-all
# "route 0.0.0 0.0.0 -> as-inter" (see checks/wan_ss7_check.sh).
#
# Environment passed to start-docker.sh:
#   WAN_NODE_ID          node number (MCC), from the topology (MULTI_NODE).
#   OSMO_QUICK=1         reuse the docker cache (no --no-cache rebuild).
#   OSMO_NO_BUILD=1      image already checked/derived above; do not rebuild.
#   OSMO_NONINTERACTIVE=1  whiptail dialogs return their default instead of
#                        waiting (nobody sees them after an icon click).
#   HANDOFF_MODE=faketrx-qemu  mixed radio: one faketrx + one QEMU Calypso
#                        per operator, same setup as the native stack.
#   OSMO_SKIP_CHECKS=1   skip start-docker.sh's final checks; verifier()
#                        below runs them with the topology.
#   OP_ID_BASE=2         containers take ranks 2 and up; 1 is the native.
#   OSMO_NO_ATTACH=1     do not attach to a container's tmux; return so
#                        verifier() can run.
# --operators: the explicit start-docker.sh flag (preferred over
# OPERATOR_COUNT_HINT); it is parsed last, wins, and shows the intent in
# the printed command.
CMD=(env "WAN_NODE_ID=${MULTI_NODE:-1}" "OSMO_QUICK=1" "OSMO_NO_BUILD=1" "OSMO_NONINTERACTIVE=1"
 ↪  "HANDOFF_MODE=faketrx-qemu" "OSMO_SKIP_CHECKS=1"
     "OP_ID_BASE=2" "OSMO_NO_ATTACH=1"
     "$DIR/launch/start-docker.sh" virtual --operators "$N_DOCKER")
```

```
echo -e "  ${BOLD}Topologie${NC} : ${N_DOCKER} conteneur(s) + 1 natif + hub ${MULTI_HUB_IP}"
echo -e "  ${CYAN}→${NC} ${CMD[*]}"
echo

if [ "$ACTION" = "dry" ]; then
    echo -e "  ${YELLOW}--dry-run : rien n a ete lance.${NC}"
    exit 0
fi

[ "$(id -u)" -eq 0 ] || { echo -e "${RED}Root requis : sudo $0${NC}"; exit 1; }

# ── Enter the repository before calling start-docker.sh ──────────────
# start-docker.sh calls its helpers by RELATIVE path (./helpers/prepare_host.sh);
# an icon launch starts in $HOME.
cd "$DIR" || { echo -e "${RED}Impossible d entrer dans $DIR${NC}"; exit 1; }

# ── Align the native SS7 identity before starting ────────────────────
# start-docker.sh numbers ITS containers 1..N: with 2 containers it produces
# point codes 1.1.2 and 1.2.2. A native stack left at its default also
# uses 1.1.2 - two devices with the same SS7 address.
#
# So the native stack is repointed to the rank given by the topology, BEFORE
# starting the containers. Two flags matter:
#   --local   single-machine plan, MID=<op> -> 1.<op>.<role>. WITHOUT it,
#             set-node-id.sh uses the WAN plan (MID=<node><op>) and yields
#             1.13.2 instead of 1.3.2.
#   --hub-ip  otherwise the ASP is repointed to 192.168.1.49, the default
#             WAN hub, which does not exist here.
# ── The native identity is more than the point code ──────────────────
# set-node-id.sh only changes point codes. start-native.sh derives the rest
# of its operator from OPERATOR_ID (/etc/osmocom/coeur.env):
#     MS_OP_ID  = OPERATOR_ID
#     ms_imsi() = MCC MNC <op, 4 digits> <ms, 6 digits>  -> IMSI
#     MSISDN    = 100<op><ms>
#     MS_ARFCN1 = 512 + MS_OP_ID * 2                     -> ARFCN
# A native stack realigned to operator 3 but left at OPERATOR_ID=1 would
# share subscribers (MSISDN, IMSI) and ARFCN with container osmo-operator-1.
_aligner_coeur_env() {
    local idx="$1" f="/etc/osmocom/coeur.env"
    [ -f "$f" ] || return 0
    if grep -qE '^[[:space:]]*:[[:space:]]*"\$\{OPERATOR_ID:=' "$f"; then
        sed -i -E "s|^([[:space:]]*:[[:space:]]*\"\\\$\\{OPERATOR_ID:=)[0-9]+(\\}\")|\\1${idx}\\2|" "$f"
        grep -qE "OPERATOR_ID:=${idx}\\}" "$f" \
            || echo -e "  ${GREEN}✓${NC} coeur.env : OPERATOR_ID=${idx} (IMSI, MSISDN et ARFCN du natif suivent)"
    else
        echo ": \"\${OPERATOR_ID:=${idx}}\"" >> "$f"
        echo -e "  ${GREEN}✓${NC} coeur.env : OPERATOR_ID=${idx} ajoute"
    fi
}

# Set to 1 when the native point code has just changed: its daemons keep
# the old one in memory and MUST restart (see tout_arreter).
NATIF_REALIGNE=0
aligner_natif() {
    local idx pc spec reel
    for spec in $MULTI_OPS; do
        IFS=: read -r idx mode _ip pc _rctx <<< "$spec"
        [ "$mode" = "native" ] || continue
        # coeur.env is aligned FIRST and UNCONDITIONALLY: point codes
        # (set-node-id.sh) and subscriber plan (OPERATOR_ID) are independent, and
        # the early "already at the right PC" exit below must not skip it.
        _aligner_coeur_env "$idx"
        reel="$(natif_pc)"
        [ -n "$reel" ] || return 0
        [ "$reel" = "$pc" ] && { echo -e "  ${GREEN}✓${NC} natif deja en PC ${pc} (op ${idx})"; return 0; }
```

```
        echo -e "  ${CYAN}→${NC} natif en PC ${reel}, attendu ${pc} : realignement sur l operateur ${idx}"
        if [ -x "$DIR/network/set-node-id.sh" ] || [ -r "$DIR/network/set-node-id.sh" ]; then
            if bash "$DIR/network/set-node-id.sh" --node "${MULTI_NODE:-1}" --op "$idx" \
                    --local --hub-ip 127.0.0.1 >/dev/null 2>&1; then
                echo -e "  ${GREEN}✓${NC} natif realigne : PC $(natif_pc)"
                echo -e "  ${YELLOW}!${NC} les demons natifs gardent l ancienne identite en memoire :"
                echo -e "    ils seront arretes et relances pour qu ils la relisent."
                NATIF_REALIGNE=1
            else
                echo -e "  ${RED}✗${NC} realignement du natif echoue - voir network/set-node-id.sh"
            fi
        fi
        return 0
    done
}
aligner_natif

# ── --align: stop here ──────────────────────────────────────────────
# Nothing is started or stopped: the native stack reads its config when
# start-native.sh starts it right after (hence the bench calls --align first).
if [ "$ACTION" = "align" ]; then
    if [ "$NATIF_REALIGNE" = 1 ]; then
        echo -e "  ${CYAN}i${NC} --align : identite posee ; le natif la lira a son prochain demarrage (rien
        ↪  relance ici)"
    else
        echo -e "  ${GREEN}✓${NC} --align : identite du natif deja en place, rien a faire"
    fi
    exit 0
fi

# ── Native 2G stays up; containers restart from scratch ─────────────
# Osmocom daemons do NOT reload their configuration live (kill -HUP does not
# help): a full stop is required after a point code change.
#
# The native stack is only restarted when needed:
#   - native running and already aligned -> KEPT as is;
#   - native stopped                     -> started by lancer_natif_si_absent;
#   - native realigned (aligner_natif)   -> stop + restart, no choice;
#   - --restart-natif                    -> stop + restart, on request.
#
# Stopping delegates to run.sh --stop, which tears down radio and core
# cleanly. Nothing here may fail the launch: what is already stopped is
# fine, hence the || true.
# ── The native stack is a systemd unit when installed ───────────────
# osmo-banc.service holds the native stack (services/osmo-banc.service).
# Stopping it with start-native.sh --stop behind systemd's back would leave
# the unit "active (exited)" on a dead stack, and the desktop shortcut needs
# a graphical session the service does not have. When the unit is
# installed, stop and (re)start it instead.
BANC_UNIT="${OSMO_BANC_SERVICE:-osmo-banc.service}"
_banc_unit_present() {
    command -v systemctl >/dev/null 2>&1 || return 1
    [ -d /run/systemd/system ] || return 1
    systemctl cat "$BANC_UNIT" >/dev/null 2>&1
}

arreter_natif() {
    if _banc_unit_present; then
        # The native stack is restarted right after by lancer_natif_si_absent;
        # here we only stop it cleanly (ExecStop = start-native.sh --stop).
        timeout 200 systemctl stop "$BANC_UNIT" >/dev/null 2>&1 || true
        echo -e "    ${GREEN}✓${NC} pile native arretee (${BANC_UNIT})"
    elif [ -x "$DIR/start-native.sh" ]; then
        timeout 120 "$DIR/start-native.sh" --stop >/dev/null 2>&1 || true
        echo -e "    ${GREEN}✓${NC} pile native arretee"
    fi
```

```
}

tout_arreter() {
    if _est_natif osmo-bsc asterisk; then
        if [ "$NATIF_REALIGNE" = 1 ]; then
            echo -e "  ${CYAN}→${NC} natif realigne : arret de la pile native pour qu elle relise sa config"
            arreter_natif
        elif [ "$RESTART_NATIF" = 1 ]; then
            echo -e "  ${CYAN}→${NC} --restart-natif : arret de la pile native"
            arreter_natif
        else
            echo -e "  ${GREEN}✓${NC} operateur natif (2G) deja en marche : on le garde tel quel"
        fi
    else
        echo -e "  ${CYAN}i${NC} operateur natif (2G) a l arret : il sera lance avant les conteneurs"
    fi
    # Containers are NOT touched here: start-docker.sh already runs
    # `docker rm -f $(docker ps -aq --filter name=osmo-)` at startup. Doing it
    # here would also wipe the multi-operator containers when called from
    # launch.sh, which only starts the native stack.
    # Audio bridges outlive the containers (setsid): kill them, otherwise one
    # more relay piles up on each restart and the audio doubles.
    pkill -f 'paplay --server=tcp:' 2>/dev/null || true
}

tout_arreter

# ── If the native stack is down, run its shortcut ──────────────────────
# The native stack has its own launcher (desktop red phone -> launch.sh ->
# start-native.sh). Without it the hub would see only some of the expected
# ASPs and the final check would report DEGRADED.
#
# Start it BEFORE the containers: start-native.sh must set up its configs
# and core while the addressing plan is still free.
#
# Using the .desktop rather than launch.sh directly gives the same path as
# a user double-click: same terminal, same pkexec elevation.
NATIF_DESKTOP="${NATIF_DESKTOP:-/root/Desktop/osmo-launch.desktop}"

lancer_natif_si_absent() {
    _est_natif osmo-bsc asterisk && return 0

    if _banc_unit_present; then
        # systemctl start blocks until start-native.sh finishes (oneshot): when it
        # returns the core is already up. The wait below only matters if the unit
        # returned 0 without a core.
        echo -e "  ${YELLOW}!${NC} operateur natif a l arret - ${CYAN}systemctl start ${BANC_UNIT}${NC}"
        if ! systemctl start "$BANC_UNIT"; then
            echo -e "  ${RED}✗${NC} ${BANC_UNIT} n a pas demarre - journalctl -u ${BANC_UNIT%.service} -n 50"
            return 1
        fi
        _est_natif osmo-bsc asterisk && { echo -e "  ${GREEN}✓${NC} natif lance (${BANC_UNIT})"; return 0; }
        echo -n "  attente du natif "
        for i in $(seq 1 60); do
            _est_natif osmo-bsc asterisk && { echo -e " ${GREEN}✓${NC}"; return 0; }
            sleep 1; echo -n "."
        done
        echo -e " ${YELLOW}toujours absent apres 60 s - on continue quand meme${NC}"
        return 0
    fi

    echo -e "  ${YELLOW}!${NC} operateur natif a l arret - lancement de son raccourci"
    local d="$NATIF_DESKTOP"
    # Fallback locations: Desktop (French locale "Bureau"), then the menu.
    for _c in "$d" "${d%/Desktop/*}/Bureau/${d##*/}" "/usr/share/applications/${d##*/}"; do
        [ -f "$_c" ] && { d="$_c"; break; }
```

```
    done
    if [ ! -f "$d" ]; then
        echo -e "  ${RED}✗${NC} raccourci introuvable ($NATIF_DESKTOP)"
        echo -e "    ${CYAN}sudo ${DIR}/start-native.sh${NC}"
        return 1
    fi

    # gio launch is THE way to run a .desktop: it reads Exec, honours
    # Terminal= and goes through the session bus. gtk-launch as fallback (it
    # wants an application ID, not a path - hence basename without .desktop).
    # Last resort: extract Exec by hand.
    # OSMO_LAUNCH_APPS=0: the native stack is used as a STACK, not a
    # workstation; wireshark, linphone and firefox are already open elsewhere,
    # so launch.sh must not reopen them.
    export OSMO_LAUNCH_APPS=0
    # OSMO_TERM_TAKEN=1: the native stack gets its OWN window. Otherwise
    # launch.sh would take our tty as its own and start-native.sh would hold it
    # to the end. See launch.sh, guard "1. Un terminal".
    export OSMO_TERM_TAKEN=1
    export XDG_RUNTIME_DIR="${XDG_RUNTIME_DIR:-/run/user/$(id -u)}"
    export DBUS_SESSION_BUS_ADDRESS="${DBUS_SESSION_BUS_ADDRESS:-unix:path=${XDG_RUNTIME_DIR}/bus}"
    if command -v gio >/dev/null 2>&1 && gio launch "$d" >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} raccourci lance : ${CYAN}${d}${NC}"
    elif command -v gtk-launch >/dev/null 2>&1 \
         && gtk-launch "$(basename "$d" .desktop)" >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} raccourci lance (gtk-launch) : ${CYAN}${d}${NC}"
    else
        local _exec
        _exec="$(sed -n 's/^Exec=//p' "$d" | head -1 | sed 's/ *%[fFuUdDnNickvm]//g')"
        [ -n "$_exec" ] || { echo -e "  ${RED}✗${NC} pas de ligne Exec dans $d"; return 1; }
        echo -e "  ${YELLOW}○${NC} gio/gtk-launch indisponibles - execution directe : ${CYAN}${_exec}${NC}"
        setsid bash -c "$_exec" </dev/null >/dev/null 2>&1 &
        disown 2>/dev/null || true
    fi

    # Wait until it is really up: starting containers while the native stack
    # is coming up means competing for the same resources.
    echo -ne "  ${CYAN}→${NC} attente du natif"
    local i
    for i in $(seq 1 120); do
        _est_natif osmo-bsc asterisk && { echo -e " ${GREEN}✓${NC}"; return 0; }
        sleep 1; echo -n "."
    done
    echo -e " ${YELLOW}toujours absent apres 120 s - on continue quand meme${NC}"
    return 0
}
lancer_natif_si_absent
  ×3
"${CMD[@]}"
rc=$?

# ── Final check ──────────────────────────────────────────────────────
# start-docker.sh may return 0 while SS7 is not up: its exit code says the
# containers STARTED, not that the ASPs associated. Check the REAL state
# afterwards.
verifier() {
    local ko=0 spec idx mode nom _ip pc _rctx _pc_reel
    echo
    echo -e "  ${BOLD}── Verification ──${NC}"

    # 1. Are all four elements present?
    for spec in $MULTI_OPS; do
        IFS=: read -r idx mode _ip pc _rctx <<< "$spec"
        if [ "$mode" = "docker" ]; then
            nom="osmo-operator-${idx}"
            if docker ps --format '{{.Names}}' 2>/dev/null | grep -qx "$nom"; then
```

```
            echo -e "    ${GREEN}✓${NC} conteneur ${nom}"
        else
            echo -e "    ${RED}✗${NC} conteneur ${nom} absent"; ko=1
        fi
    else
        if ! _est_natif osmo-bsc asterisk; then
            echo -e "    ${RED}✗${NC} operateur natif arrete - ${CYAN}sudo ${DIR}/start-native.sh --op
             ↪  ${idx}${NC}"; ko=1
        else
            # ── The native point code must match its rank ──
            # A native stack started without --op runs as operator 1 (PC 1.1.2), the
            # same as the FIRST container (start-docker.sh always numbers from 1). Both
            # sides start fine but SS7 routing is wrong, so check it explicitly.
            _pc_reel="$(natif_pc)"
            if [ -z "$_pc_reel" ]; then
                echo -e "    ${YELLOW}?${NC} operateur natif actif, point code illisible
                 ↪  (/etc/osmocom/osmo-stp.cfg)"
            elif [ "$_pc_reel" = "$pc" ]; then
                echo -e "    ${GREEN}✓${NC} operateur natif (op ${idx}, PC ${_pc_reel})"
            else
                echo -e "    ${RED}✗${NC} operateur natif en PC ${_pc_reel}, attendu ${pc}"
                echo -e "      il tourne en operateur $(echo "$_pc_reel" | cut -d. -f2), rang deja pris par un
                 ↪  CONTENEUR"
                echo -e "      ${CYAN}sudo ${DIR}/start-native.sh --op ${idx}${NC}  (puis relancer ce script)"
                ko=1
            fi
        fi
    fi
done
nom="${MULTI_HUB_NAME:-osmo-inter-stp}"
if docker ps --format '{{.Names}}' 2>/dev/null | grep -qx "$nom"; then
    echo -e "    ${GREEN}✓${NC} inter-STP ${nom}"
else
    echo -e "    ${RED}✗${NC} inter-STP ${nom} absent"; ko=1
fi

# 2. Native M3UA entry point. Query `ss`, NOT /dev/tcp: M3UA runs over
#    SCTP, where a TCP probe is silent; `ss -an` lists SCTP sockets.
# awk by FIELDS, not a regex: `ss -an` aligns columns with a VARIABLE number
# of spaces depending on address length. $1/$2/$5 are stable.
if ss -an 2>/dev/null | awk -v p=":${MULTI_M3UA_PORT}" \
     '$1=="sctp" && $2=="LISTEN" && index($5,p)==length($5)-length(p)+1 {f=1} END{exit !f}'; then
    echo -e "    ${GREEN}✓${NC} M3UA en SCTP sur 127.0.0.1:${MULTI_M3UA_PORT} (l operateur natif peut s
     ↪  attacher)"
else
    echo -e "    ${RED}✗${NC} rien n ecoute en SCTP sur le port ${MULTI_M3UA_PORT}"
    echo -e "      l ASP du natif (osmo-stp.cfg, remote-ip 127.0.0.1) ne trouvera personne"
    ko=1
fi

# 3. The repository's SS7 checker, which reads ASPs and routes.
if [ -x "$DIR/checks/ss7_check.sh" ]; then
    echo -e "    ${CYAN}→${NC} checks/ss7_check.sh"
    "$DIR/checks/ss7_check.sh" || ko=1
else
    echo -e "    ${YELLOW}○${NC} checks/ss7_check.sh absent - verification SS7 sautee"
fi

echo
if [ "$ko" = "0" ]; then
    echo -e "  ${GREEN}${BOLD}Banc multi-operateur OPERATIONNEL.${NC}"
else
    echo -e "  ${RED}${BOLD}Le banc n est PAS complet${NC} - voir les lignes ✗ ci-dessus."
    echo -e "  ${CYAN}→${NC} etat detaille : ${BOLD}sudo $0 --status${NC}"
fi
return "$ko"
```

```
}

# ── Container dashboards as extra tabs ──────────────────────────────
# launch.sh already opened Firefox on the NATIVE dashboard and the tutorial,
# and started Linphone and Wireshark: only the container consoles are
# missing, one per docker operator.
#
# Addresses come from the topology: adding an operator to osmo-multi.conf
# is enough to get its tab.
#
# Wait for each port to answer before opening: a container dashboard takes
# several seconds to come up, and a tab opened too early shows an error.
ouvrir_dashboards() {
    # ── OSMO_NO_TERM=1: no window, not even a tab ──
    # When run by osmo-multi.service (the desktop icon), do not pop up a
    # browser minutes after the click. The addresses are still PRINTED to the
    # journal (`journalctl -u osmo-multi`).
    if [ "${OSMO_NO_TERM:-0}" = "1" ]; then
        local _spec _idx _mode _ip _liste=""
        for _spec in $MULTI_OPS; do
            IFS=: read -r _idx _mode _ip _ <<< "$_spec"
            [ "$_mode" = "docker" ] && [ -n "$_ip" ] && _liste="$_liste http://${_ip}:8080"
        done
        echo -e "  ${CYAN}i${NC} OSMO_NO_TERM=1 : aucun onglet ouvert. Dashboards :${CYAN}${_liste}${NC}"
        return 0
    fi
    command -v firefox >/dev/null 2>&1 || return 0

    local spec idx mode ip urls="" i
    for spec in $MULTI_OPS; do
        IFS=: read -r idx mode ip _pc _rctx <<< "$spec"
        [ "$mode" = "docker" ] && [ -n "$ip" ] || continue
        for i in $(seq 1 60); do
            (exec 3<>"/dev/tcp/${ip}/8080") 2>/dev/null && { exec 3>&- 2>/dev/null; break; }
            sleep 1
        done
        if (exec 3<>"/dev/tcp/${ip}/8080") 2>/dev/null; then
            exec 3>&- 2>/dev/null
            urls="$urls http://${ip}:8080"
        else
            echo -e "  ${YELLOW}○${NC} dashboard op ${idx} (${ip}:8080) ne repond pas - onglet non ouvert"
        fi
    done
    [ -n "$urls" ] || return 0

    # Under the session user's account: Firefox already runs there and adds
    # the URLs as TABS to the existing window instead of opening a new one.
    local u uid home
    u="$(natif_session_user)"; uid="$(id -u "$u" 2>/dev/null || echo 0)"
    home="$(getent passwd "$u" 2>/dev/null | cut -d: -f6)"
    # shellcheck disable=SC2086
    setsid sudo -u "$u" \
        env DISPLAY="${DISPLAY:-:0}" \
            XAUTHORITY="${XAUTHORITY:-${home:-/root}/.Xauthority}" \
            DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/${uid}/bus" \
        firefox $urls </dev/null >/dev/null 2>&1 &
    disown 2>/dev/null || true
    echo -e "  ${GREEN}✓${NC} onglets dashboards :${CYAN}${urls}${NC}"
}

# Graphical session user, same criterion as elsewhere: presence of its
# pulse socket, not its name (the ISO opens the session as root).
natif_session_user() {
    local u uid sock
    for u in "${HOST_PULSE_USER:-}" "${SUDO_USER:-}" "$(logname 2>/dev/null || true)"; do
        [ -n "$u" ] || continue
```

```
        uid="$(id -u "$u" 2>/dev/null)" || continue
        [ -S "/run/user/${uid}/pulse/native" ] && { echo "$u"; return 0; }
    done
    for sock in /run/user/*/pulse/native; do
        [ -S "$sock" ] || continue
        uid="${sock#/run/user/}"; uid="${uid%%/*}"
        u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
        [ -n "$u" ] && { echo "$u"; return 0; }
    done
    echo "${SUDO_USER:-$(id -un)}"
}

# IN THE BACKGROUND: this function waits up to 60 s per container for port
# 8080. In the foreground it would block the checks for up to two minutes.
# Tabs open when they can; the checks start right away.
ouvrir_dashboards &
disown 2>/dev/null || true

echo
etat
verifier || rc=1

# ── The node sheet follows the multi-operator setup ──────────────────
# The native stack wrote it with a single operator; rewrite it now that
# the containers are up (network/node-conf.sh counts the running
# osmo-operator-N). Other nodes import this sheet.
if [ "$ACTION" = "start" ] && [ -x "$DIR/start-native.sh" ]; then
    NO_MENU=1 "$DIR/start-native.sh" --gen-conf 2>/dev/null | grep -E "Fiche|mynode" || true
fi
# Window opened by us: keep it open, otherwise the summary disappears with it.
if [ "$_multi_own_window" = "1" ]; then
    echo
    echo "Fenetre maintenue ouverte - Entree pour fermer."
    read -r _ || true
fi
exit "$rc"
```

2.94 /opt/GSM/osmo-operator/launch/start-nitb.sh

3367 octets, 64 lignes → 64 lignes

```
#!/bin/bash
# start-nitb.sh - same launcher as start-docker.sh, but using the osmocom-nitb image.
#
# A thin wrapper rather than a copy: only the image differs (IMAGE_RUN),
# so start-docker.sh stays the single implementation.
#
# Constraint: the two images have different entrypoints:
#     osmocom-run  -> /scripts/entrypoint.sh
#     osmocom-nitb -> /etc/osmocom/entrypoint.sh
# start-docker.sh mounts the generated config directory ON /etc/osmocom,
# hiding the nitb image's entrypoint. This works only because the generated
# directory contains entrypoint.sh (copied from scripts/entrypoint.sh); the
# check below verifies it on every launch.
#
# Usage: same as start-docker.sh; all arguments are passed through.
#   ./launch/start-nitb.sh [start-docker.sh args...]
#
# Environment: IMAGE_NITB  image to run (default osmocom-nitb).
#
# To go back to the run image, call ./launch/start-docker.sh directly.

set -eu
```

```
# Repository root (this script lives in launch/).
HERE="$(cd "$(dirname "$(readlink -f "$0")")/.." && pwd)"
IMAGE_NITB="${IMAGE_NITB:-osmocom-nitb}"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'

# ── 1. The image must EXIST: it is not built here ──────────────────────
# start-docker.sh's check_image() builds a missing IMAGE_RUN with
# `docker build -f Dockerfile.run -t "$IMAGE_RUN" .`, which with
# IMAGE_RUN=osmocom-nitb would OVERWRITE osmocom-nitb. Check first.
if ! docker image inspect "$IMAGE_NITB" >/dev/null 2>&1; then
    echo -e "${RED}Image '${IMAGE_NITB}' introuvable.${NC}"
    echo -e "Ce script ne la construit pas volontairement : le build integre de"
    echo -e "start-docker.sh utilise ${YELLOW}Dockerfile.run${NC} et ecraserait l'image nitb."
    echo -e "Construire d'abord, depuis ${CYAN}${HERE}${NC} :"
    echo -e "  ${CYAN}docker build -f Dockerfile --target osmocom-nitb -t ${IMAGE_NITB} .${NC}"
    exit 1
fi

# ── 2. The entrypoint must survive the /etc/osmocom mount ──────────────
# Test PRESENCE, not the exec bit: the file is 644 in the repository; +x is
# set downstream by generate_configs.sh (in the generated, mounted directory)
# and by Dockerfile.run (`chmod +x /scripts/*.sh` in the image).
if [ ! -s "${HERE}/scripts/entrypoint.sh" ]; then
    echo -e "${RED}scripts/entrypoint.sh absent ou vide.${NC}"
    echo -e "L'image ${IMAGE_NITB} a son entrypoint dans /etc/osmocom, que start-docker.sh"
    echo -e "recouvre par le dossier de config genere. Sans ce fichier, le"
    echo -e "conteneur demarrerait sans entrypoint - alors que l'image"
    echo -e "osmocom-run, elle, continuerait de marcher (son entrypoint est dans"
    echo -e "/scripts). Panne asymetrique et illisible : on refuse avant."
    exit 1
fi

echo -e "${GREEN}Lanceur nitb${NC} : image = ${CYAN}${IMAGE_NITB}${NC} (au lieu d'osmocom-run)"
echo -e "  entrypoint attendu dans le conteneur : /etc/osmocom/entrypoint.sh (monte)"

# ── 3. Delegation ──────────────────────────────────────────────────────
# start-docker.sh reads IMAGE_RUN with the `:=` idiom: the value set here wins.
export IMAGE_RUN="$IMAGE_NITB"
cd "$HERE"
exec ./launch/start-docker.sh "$@"
```

2.95 /opt/GSM/osmo-operator/launch/start-oqc.sh

2219 octets, 55 lignes → 55 lignes

```
#!/bin/bash
# ============================================================================
#  launch/start-oqc.sh - start qosmo-grgsm from osmo-operator
# ============================================================================
#
#  "oqc" = qosmo-grgsm, the Calypso emulation (ARM7 + TMS320C54x DSP).
#  This is a shortcut from osmo-operator: it locates the repository and hands
#  over to its run.sh. All launch logic lives there, in run_modules/.
#
#      ./launch/start-oqc.sh                 calypso profile (default)
#      ./launch/start-oqc.sh --list          show the plan, start nothing
#      ./launch/start-oqc.sh --dry-run       run without side effects
#      ./launch/start-oqc.sh --status        state of each step
#      ./launch/start-oqc.sh --stop          stop, in reverse order
#      ./launch/start-oqc.sh --check-paths   check the machine's dependencies
#
#  CALYPSO_* variables given as a prefix are passed through to QEMU:
#      CALYPSO_MODE=native ./launch/start-oqc.sh
```

```
#
#  Repository location: OQC_ROOT, otherwise ../qosmo-grgsm,
#  $GSM_ROOT/qosmo-grgsm (default /opt/GSM) and ~/qosmo-grgsm are tried.
#      OQC_ROOT=~/src/qosmo-grgsm ./launch/start-oqc.sh
#
#  Not to be confused with start-native.sh, which orchestrates the WHOLE stack
#  (Osmocom core + radio) and whose "qemu" mode also ends up here.
# ------------------------------------------------------------------------------
set -euo pipefail

_find_oqc() {
    [ -n "${OQC_ROOT:-}" ] && { printf '%s\n' "$OQC_ROOT"; return 0; }
    local here; here="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"   # launch/ -> repository root
    local c
    for c in "$here/../qosmo-grgsm" \
             "${GSM_ROOT:-/opt/GSM}/qosmo-grgsm" \
             "$HOME/qosmo-grgsm"; do
        [ -x "$c/run.sh" ] && { (cd "$c" && pwd); return 0; }
    done
    return 1
}

if ! OQC="$(_find_oqc)"; then
    cat >&2 <<'ERR'
qosmo-grgsm introuvable.

Cherche dans : ../qosmo-grgsm, $GSM_ROOT/qosmo-grgsm, ~/qosmo-grgsm

Indiquez son emplacement :
    OQC_ROOT=/chemin/vers/qosmo-grgsm ./launch/start-oqc.sh
ou recuperez-le :
    git clone https://github.com/bbaranoff/qosmo-grgsm qosmo-grgsm
ERR
    exit 2
fi

exec "$OQC/run.sh" "$@"
```

2.96 /opt/GSM/osmo-operator/lib/audio.sh

42674 octets, 745 lignes → 745 lignes

```
# ==============================================================================
#  lib/audio.sh - the osmo-operator audio chain (PulseAudio + GAPK bridge)
# ==============================================================================
#
#  ensure_pulse / ensure_host_audio / ensure_gapk set up the PulseAudio chain
#  used by the `mobile` and by the MGW RTP bridge. The `qemu` mode of
#  start-native.sh calls ensure_gapk right before handing over to
#  qosmo-grgsm/start-clean.sh: this is what connects the MGW RTP to the
#  `gsm_audio` sink; without it the stack comes up, the call is established,
#  and nobody hears anything.
#
#  THIS FILE DOES NOTHING WHEN SOURCED: it only defines functions.
#  Expected caller: run_modules/25-audio.sh.
#
#  AUDIO=0 disables the whole setup.
# ------------------------------------------------------------------------------

: "${HERE:=/opt/GSM/osmo-operator}"
: "${LOG_DIR:=/root}"
: "${AUDIO:=1}"
: "${PULSE_SOCK:=/var/run/pulse/native}"
# Messages are colored; under `set -u` an undefined color would make the
```

```
# function fail before doing anything, so the colors are neutralized here.
RED='' GREEN='' YELLOW='' CYAN='' NC='' BOLD=''

# The null-sinks of the GSM chain, declared in one place: gsm_audio (= alsa
# gsm_out, the outgoing voice) and gsm_mic (= alsa gsm_in through
# gsm_mic.monitor, the silent microphone that feeds in).
#
# Both sinks are always created together - here, in system.pa and in the
# Dockerfile - because gapk_io gives up BOTH directions when the capture fails:
#     pq_alsa.c:168  Couldn't init ALSA device 'gsm_in': Input/output error
#     gapk_io.c:468  Failed to initialize GAPK I/O
# which leaves a perfectly established but totally silent call, with the error
# buried in mobile.log.
# osmo_tts_off: a trash sink for speech-dispatcher. On the bench a desktop
# application sometimes triggers speech synthesis; without an installed voice
# the "dummy" module of speech-dispatcher plays its placeholder message
# (dummy-message.wav: 29 s, low voice, 83 % of the energy below 300 Hz, level
# 100 %) in the speakers, on top of the current call. Once the stream is moved
# here, module-stream-restore sends it back here each time it appears.
GSM_SINKS="gsm_audio:GSM_Audio gsm_mic:GSM_Mic osmo_tts_off:TTS_off"

load_gsm_sinks() {
    local entry name desc
    for entry in $GSM_SINKS; do
        name="${entry%%:*}"; desc="${entry##*:}"
        pactl list short sinks 2>/dev/null | grep -qw "$name" || \
            pactl load-module module-null-sink sink_name="$name" \
                format=s16le rate=8000 channels=1 \
                sink_properties=device.description="$desc" >/dev/null 2>&1 || true
    done
}

# ── Local loopback: gsm_audio.monitor → the machine's sound card ─────────────
# gsm_audio is a module-null-sink: without a consumer the downlink voice is
# dropped by construction (gsm_audio RUNNING, gsm_audio.monitor IDLE, ALSA
# output SUSPENDED: call established, no sound). This loopback is that consumer.
#
# ⚠ The sink choice matters: looping back onto gsm_audio or gsm_mic creates a
# closed loop (cf. ensure_gapk: the mobiles write into gsm_out, they would be
# replayed into their own uplink). Both null-sinks are therefore explicitly
# excluded and only a hardware output is kept.
# 40 ms and not 20: the minimum latency of these sinks is 26 ms. A loopback
# asking for 20 ms resyncs constantly and drops samples ("Configured latency
# of 20.00 ms is smaller than minimum latency", "Doing resync", "drop sink",
# "drop source"). 40 ms is above the floor and the extra 20 ms are inaudible
# on a call.
LOOPBACK_LATENCY_MSEC="${LOOPBACK_LATENCY_MSEC:-40}"

# ── Speaker and microphone assignment: one single place ──────────────────────
# The bench audio has two modes: WITHOUT a postmarketOS phone, the host hears
# the downlink through a direct module-loopback (gsm_audio.monitor ->
# speakers); WITH the VM, the voice must go THROUGH IT ("bridge" card 0x12 <->
# "combine" card 0x13, tools/osmo-pmos-setup.sh) and the direct loopback must
# not exist, otherwise everything is heard twice (direct, then the VM 500 ms
# later). scripts/audio-chain.sh (at boot, ExecStartPost of pulseaudio),
# ensure_pulse (start-native) and osmo-pmos-setup all go through the functions
# below, so they agree on what "the speakers" are: the memorized default sink
# (osmo_hp_ec) does not exist at boot, the echo canceller must be loaded before
# the QEMU "combine" streams appear, otherwise they land on the raw card.
#
#   audio_hw_sink / audio_hw_source   the hardware sound card (the default if
#                                     it is one, otherwise the first alsa_*,
#                                     HDMI excluded);
#   audio_hp_sink / audio_mic_source  WHAT THE OPERATOR HEARS / SAYS: the echo
#                                     canceller (osmo_hp_ec / osmo_mic_ec) if
#                                     loaded, otherwise the hardware;
```

```
#   ensure_echo_cancel               loads that canceller (without analog
#                                    AGC, microphone at 25 %) and makes it
#                                    the default, at boot as with the VM;
#   pmos_vm_audio_present            the postmarketOS VM has its sound cards
#                                    here (QEMU streams media.name=combine);
#   remove_direct_loopbacks          removes the direct loopbacks;
#   ensure_local_loopback / _mic     install the direct loopback ONLY without
#                                    the VM, to audio_hp_sink / from
#                                    audio_mic_source, and replace a loopback
#                                    installed to another output (the raw
#                                    card from before the canceller, for
#                                    example).
# osmo-pmos-qemu loads the canceller BEFORE starting QEMU and names its outputs
# to the "combine" card (OSMO_PMOS_HP / OSMO_PMOS_MIC, pmbootstrap patch);
# osmo-pmos-setup and the VM shutdown (pmos_stop) go through these functions.
# AUDIO_ECHO_CANCEL=0 to do without the canceller (speakers = the raw card).
EC_AEC_ARGS='aec_args="analog_gain_control=0 digital_gain_control=1 noise_suppression=1 high_pass_filter=1"'
: "${AUDIO_AEC_METHOD:=speex}"
audio_aec_method() { echo "$AUDIO_AEC_METHOD"; }
# aec_args only apply to webrtc (see ensure_echo_cancel, aec_opts).
: "${EC_MIC_VOLUME:=25%}"
# The Acer PDM microphone does not saturate at 100 % (room noise ~70 rms) and
# is only a whisper at 25-50 % (~6 rms): it gets its own volume.
: "${EC_MIC_VOLUME_PDM:=100%}"

audio_hw_sink() {
    local d; d="$(pactl get-default-sink 2>/dev/null || true)"
    case "$d" in alsa_output.*) echo "$d"; return 0 ;; esac
    pactl list short sinks 2>/dev/null \
        | awk '$2 ~ /^alsa_output/ && $2 !~ /hdmi/ { print $2; exit }'
}
# THE INTERNAL DIGITAL MICROPHONE (PDM/DMIC) BEFORE THE JACK INPUT.
# On the Acer the default source is the analog jack input
# (alsa_input.pci-0000_04_00.6.analog-stereo): nothing plugged in, absolute
# zero (not even noise), and the echo canceller would attach to it, while the
# internal microphone is the acp_pdm_mach source, left SUSPENDED. The jack
# port reports "availability unknown", so there is no plug detection to rely
# on: the internal digital microphone is preferred by name.
# AUDIO_MIC_SOURCE=<source> forces another microphone (headset).
audio_hw_source() {
    if [ -n "${AUDIO_MIC_SOURCE:-}" ] && pactl list short sources 2>/dev/null \
            | awk -v s="$AUDIO_MIC_SOURCE" '$2 == s { f=1 } END { exit !f }'; then
        echo "$AUDIO_MIC_SOURCE"; return 0
    fi
    local p; p="$(pactl list short sources 2>/dev/null \
        | awk '$2 ~ /^alsa_input/ && $2 ~ /pdm|dmic/ && $2 !~ /\.monitor$/ { print $2; exit }')"
    [ -n "$p" ] && { echo "$p"; return 0; }
    local d; d="$(pactl get-default-source 2>/dev/null || true)"
    case "$d" in alsa_input.*) echo "$d"; return 0 ;; esac
    pactl list short sources 2>/dev/null \
        | awk '$2 ~ /^alsa_input/ && $2 !~ /\.monitor$/ { print $2; exit }'
}
audio_hp_sink() {
    pactl list short sinks 2>/dev/null | awk '$2 == "osmo_hp_ec" { f=1 } END { exit !f }' \
        && { echo osmo_hp_ec; return 0; }
    audio_hw_sink
}
audio_mic_source() {
    pactl list short sources 2>/dev/null | awk '$2 == "osmo_mic_ec" { f=1 } END { exit !f }' \
        && { echo osmo_mic_ec; return 0; }
    audio_hw_source
}

# ── Who carries the voice: the host, or the VM? ──────────────────────────
# BY DEFAULT, THE HOST. The postmarketOS phone is the bench INTERFACE (dial,
# answer, SMS); it does not need to carry the samples. Doing so is expensive
```

```
# and audible: microphone -> QEMU -> VM Pulse -> module-loopback 400 ms ->
# QEMU -> gsm_mic, and as much on the way back, i.e. ~800 ms round trip, two
# emulated clocks, and the resampling of two HDA cards at 44.1 kHz for an
# 8 kHz telephone signal.
#
# So the voice stays on the host (gsm_audio -> speakers, microphone ->
# gsm_mic, 40 ms), and the VM keeps its cards for its own use - ringtone,
# notifications, the Calls application. It stays in the SIGNALLING path,
# which is its real role.
#
# To put the voice INSIDE the VM (the crossed setup, to hear it go through the
# phone): OSMO_PMOS_VOIX_VM=1 when starting osmo-pmos-setup. It then installs
# the two loopbacks in the VM and drops this marker; the functions below then
# remove the host's direct loopbacks, otherwise the voice would arrive TWICE -
# direct, then through the VM 400 ms later.
PMOS_VOIX_VM_MARQUEUR="${PMOS_VOIX_VM_MARQUEUR:-/run/osmo-pmos-voix-vm}"

# True only if the VM REALLY carries the voice: the marker AND the process.
# A marker left behind by a VM killed abruptly must not deprive the host of
# its speakers.
pmos_vm_audio_present() {
    [ -f "$PMOS_VOIX_VM_MARQUEUR" ] || return 1
    pgrep -f 'qemu-system.*hdac[o]mbine' >/dev/null 2>&1 && return 0
    pactl list sink-inputs 2>/dev/null | grep -q 'media.name = "combine"'
}


remove_direct_loopbacks() {
    local m n=0
    for m in $(pactl list short modules 2>/dev/null \
               | awk '/module-loopback/ && (/source=gsm_audio\.monitor/ || /sink=gsm_mic([ \t]|$)/) { print $1
               ↪  }'); do
        pactl unload-module "$m" >/dev/null 2>&1 && n=$(( n + 1 ))
    done
    [ "$n" -gt 0 ] && echo -e "  ${YELLOW}[audio] ${n} bouclage(s) direct(s) retire(s)${NC}"
    return 0
}

# THE ECHO CANCELLER BETWEEN THE MICROPHONE AND THE SPEAKERS.
# Internal microphone + speakers: the echo of the 600 feeds back into the
# microphone (2 400 peak with the speaker cut, 32 768 saturated with the
# speaker open): Larsen. The webrtc analog AGC, left at its default, moves the
# microphone gain from 100 % to 0 % from one minute to the next (crushed
# voice, 93 % of the energy below 300 Hz at the GSM input). Hence: no analog
# AGC, microphone fixed at 25 % (it saturates from 45 %), and an explicit
# unmute, since module-device-restore sometimes leaves the module muted. The
# osmo_mic_ec volume is SHARED with the hardware microphone: same value for
# both.
ensure_echo_cancel() {
    [ "${AUDIO:-1}" = "1" ] || return 0
    [ "${AUDIO_ECHO_CANCEL:-1}" = "1" ] || {
        echo -e "  ${YELLOW}[audio] annuleur d echo desactive (AUDIO_ECHO_CANCEL=0)${NC}"; return 0; }
    pactl info >/dev/null 2>&1 || return 0
    local mic hp aec_opts=()
    [ "$AUDIO_AEC_METHOD" = "webrtc" ] && aec_opts=("$EC_AEC_ARGS")
    mic="$(audio_hw_source)"; hp="$(audio_hw_sink)"
    if [ -z "$mic" ] || [ -z "$hp" ]; then
        echo -e "  ${YELLOW}[audio] pas de micro ou de haut-parleur materiel - annuleur d echo ignore${NC}";
        ↪  return 0
    fi
    # The default (osmo_hp_ec / osmo_mic_ec) is only set at LOAD time: replaying
    # it on every pass would send back to the canceller the streams the
    # operator has just moved elsewhere. Everything that matters for the GSM
    # chain is pinned by name; the default is only a desktop matter.
    if pactl list short modules 2>/dev/null | grep -q 'module-echo-cancel'; then
        echo -e "  ${GREEN}[audio] annuleur d echo deja en place (osmo_hp_ec / osmo_mic_ec)${NC}"
```

```
        return 0
    # 8000 Hz MONO, LIKE THE REST OF THE BENCH. Without "rate" the module loads
    # at 32000 Hz (osmo_hp_ec in float32le 1ch 32000Hz) while gsm_audio,
    # gsm_mic and osmo_tts_off are all s16le 1ch 8000Hz; each pass through the
    # canceller would then cost two resamplings (8000 -> 32000 on input,
    # 32000 -> 8000 on output). The signal is bounded to 4 kHz by GSM anyway.
    # The webrtc canceller accepts 8000/16000/32000/48000.
    # SPEEX BY DEFAULT. webrtc (cancellation + noise reduction + high-pass)
    # costs PulseAudio 7 to 17 % of a core continuously, call or not. speex
    # cancels the echo for a fraction of that price and has no analog AGC that
    # would pump the microphone. AUDIO_AEC_METHOD=webrtc selects the webrtc
    # canceller and its aec_args.
    # aec_args is ONE argument (quotes included): an unquoted $(...) would
    # split it, pactl would ignore analog_gain_control=0 and the webrtc AGC
    # would bring the microphone down to 0 % (mute microphone during a call).
    elif pactl load-module module-echo-cancel aec_method="$(audio_aec_method)" \
            rate=8000 channels=1 \
            source_master="$mic" sink_master="$hp" \
            source_name=osmo_mic_ec sink_name=osmo_hp_ec "${aec_opts[@]}" \
            source_properties=device.description=Micro_sans_echo \
            sink_properties=device.description=HP_sans_echo >/dev/null 2>&1; then
        echo -e "  ${GREEN}[audio] annuleur d echo ${AUDIO_AEC_METHOD} pose entre ${mic} et ${hp}${NC}"
    else
        echo -e "  ${YELLOW}[audio] echec de l annuleur d echo - les HP restent ${hp}${NC}"; return 0
    fi
    pactl set-default-sink osmo_hp_ec >/dev/null 2>&1
    pactl set-default-source osmo_mic_ec >/dev/null 2>&1
    pactl set-sink-mute "$hp" 0 2>/dev/null;       pactl set-sink-mute osmo_hp_ec 0 2>/dev/null
    local vol="$EC_MIC_VOLUME"
    case "$mic" in *pdm*|*dmic*) vol="$EC_MIC_VOLUME_PDM" ;; esac
    pactl set-source-mute "$mic" 0 2>/dev/null;    pactl set-source-volume "$mic" "$vol" 2>/dev/null
    pactl set-source-mute osmo_mic_ec 0 2>/dev/null; pactl set-source-volume osmo_mic_ec "$vol" 2>/dev/null
    return 0
}

# ── The RECORDING source: what is heard + what is said ──────────────────
# The screen recorder (EasyScreenCast extension, "audio source" menu) can only
# capture one PulseAudio SOURCE. A call does not live on a source: the
# downlink goes into sinks (gsm_audio, then the speakers), and only the
# microphone is a source. "No audio source" records silence, and choosing the
# microphone only records the operator's voice, never the other party.
#
# So ONE trash sink, osmo_rec, is built and fed by both sides:
#   - the monitor of the operator's speakers (everything he HEARS, whether the
#     voice comes from the direct loopback or from the postmarketOS phone);
#   - his microphone (everything he SAYS).
# Its monitor, osmo_rec.monitor, then shows up in the recorder menu as
# "Enregistrement_appel": one choice, and the whole call is in it.
# 8 kHz mono, like the rest of the chain. AUDIO_RECORD_MIX=0 not to create it.
# ── Lightening PulseAudio ────────────────────────────────────────────────
# Sources of permanent PulseAudio load (7-17 % of a core):
#   - speech-dispatcher (sd_dummy): a 44100 Hz stream never suspended towards
#     the sound card, hence a continuous 44.1 -> 48 kHz resampling for
#     nothing; started on demand by the desktop ("-s -t 0": never stops);
#   - the webrtc canceller (see ensure_echo_cancel);
#   - osmo_rec in 48 kHz stereo, fed by two 8 kHz mono loops.
# This stops the first one and MIGRATES an already running bench to
# speex / 8 kHz: unloading the canceller drops the loops pinned on it, so
# they are reinstalled. AUDIO_ALLEGER=0 to leave everything alone.
alleger_audio() {
    [ "${AUDIO:-1}" = "1" ] || return 0
    [ "${AUDIO_ALLEGER:-1}" = "1" ] || return 0
    if pgrep -x speech-dispatch >/dev/null 2>&1 || pgrep -f '[/]usr/bin/speech-dispatcher' >/dev/null 2>&1; then
        pkill -f '[/]usr/bin/speech-dispatcher' 2>/dev/null
        pkill -f '[/]usr/lib/speech-dispatcher-modules/' 2>/dev/null
        echo -e "  ${GREEN}[audio] speech-dispatcher arrete (flux 44,1 kHz permanent vers les HP)${NC}"
```

```
    fi
    pactl info >/dev/null 2>&1 || return 0
    # module-stream-restore WITHOUT restore_device: otherwise it sends the
    # `mobile` (ALSA "default") back to the raw card where it once played,
    # outside the canceller - echo test: the speaker feeds back into the
    # microphone, Larsen rising during the call. Volumes stay memorized, the
    # device does not.
    local sr
    sr="$(pactl list short modules 2>/dev/null | awk '$2 == "module-stream-restore" && $0 !~
     ↪  /restore_device=false/ {print $1; exit}')"
    if [ -n "$sr" ]; then
        pactl unload-module "$sr" >/dev/null 2>&1 \
            && pactl load-module module-stream-restore restore_device=false >/dev/null 2>&1 \
            && echo -e "  ${GREEN}[audio] module-stream-restore sans restauration de peripherique${NC}"
    fi
    local m refaire=0
    m="$(pactl list short modules 2>/dev/null | awk '/module-echo-cancel/ && /aec_method=webrtc/ {print $1;
     ↪  exit}')"
    if [ -n "$m" ] && [ "$AUDIO_AEC_METHOD" != "webrtc" ]; then
        pactl unload-module "$m" >/dev/null 2>&1 \
            && echo -e "  ${GREEN}[audio] annuleur webrtc decharge -> ${AUDIO_AEC_METHOD}${NC}"
        refaire=1
    fi
    if pactl list short sinks 2>/dev/null | awk '$2 == "osmo_rec"' | grep -q 48000Hz; then
        for m in $(pactl list short modules 2>/dev/null | awk '/module-loopback/ && /sink=osmo_rec([ \t]|$)/
         ↪  {print $1}') \
                $(pactl list short modules 2>/dev/null | awk '/module-null-sink/ && /sink_name=osmo_rec([ \t]|$)/
                 ↪  {print $1}'); do
            pactl unload-module "$m" >/dev/null 2>&1
        done
        echo -e "  ${GREEN}[audio] osmo_rec 48 kHz stereo retire -> 8 kHz mono${NC}"
        refaire=1
    fi
    if [ "$refaire" = 1 ]; then
        ensure_echo_cancel
        ensure_local_loopback
        ensure_local_mic
        ensure_record_mix
    fi
    return 0
}

# ── REINIT: start from a clean chain without restarting PulseAudio ──────────
# scripts/audio-chain.sh --reinit. Unloads what the chain installs ON TOP of
# the GSM sinks - echo canceller, all the loops, osmo_rec - and stops
# speech-dispatcher; audio-chain.sh then reinstalls everything as at boot.
# gsm_audio, gsm_mic and osmo_tts_off stay: the `mobile` and gapk are
# connected to them (ALSA PCM gsm_out/gsm_in), unloading them would cut a
# call for good instead of one second.
reinit_audio() {
    [ "${AUDIO:-1}" = "1" ] || return 0
    pactl info >/dev/null 2>&1 || return 0
    pkill -f '[/]usr/bin/speech-dispatcher' 2>/dev/null
    pkill -f '[/]usr/lib/speech-dispatcher-modules/' 2>/dev/null
    local m n=0
    for m in $(pactl list short modules 2>/dev/null | awk '/module-loopback/ {print $1}') \
             $(pactl list short modules 2>/dev/null | awk '/module-echo-cancel/ {print $1}') \
             $(pactl list short modules 2>/dev/null | awk '/module-null-sink/ && /sink_name=osmo_rec([ \t]|$)/
              ↪  {print $1}'); do
        pactl unload-module "$m" >/dev/null 2>&1 && n=$((n + 1))
    done
    echo -e "  ${GREEN}[audio] ${n} module(s) de la chaine decharge(s)${NC}"
}

ensure_record_mix() {
    [ "${AUDIO:-1}" = "1" ] || return 0
```

```
    [ "${AUDIO_RECORD_MIX:-1}" = "1" ] || return 0
    pactl info >/dev/null 2>&1 || return 0

    pactl list short sinks 2>/dev/null | grep -qw osmo_rec \
        || pactl load-module module-null-sink sink_name=osmo_rec \
               format=s16le rate=8000 channels=1 \
               sink_properties=device.description=Enregistrement_appel >/dev/null 2>&1 \
        || { echo -e "  ${YELLOW}[audio] sink osmo_rec impossible - enregistrement non prepare${NC}"; return 0; }

    local hp mic src
    hp="$(audio_hp_sink)"; mic="$(audio_mic_source)"
    for src in ${hp:+${hp}.monitor} ${mic:-}; do
        # Idempotent, and pinned like the others (cf. the dont_move block).
        pactl list short modules 2>/dev/null | grep -F 'module-loopback' \
            | grep -F "source=${src}" | grep -qF 'sink=osmo_rec' && continue
        pactl load-module module-loopback source="$src" sink=osmo_rec \
            sink_dont_move=true source_dont_move=true \
            latency_msec="$LOOPBACK_LATENCY_MSEC" >/dev/null 2>&1
    done
    echo -e "  ${GREEN}[audio] source d enregistrement prete : osmo_rec.monitor (Enregistrement_appel)${NC}"
    # The screen recorder is remembered by its application name
    # (module-stream-restore): moved once, it comes back there afterwards.
    local so
    for so in $(pactl list source-outputs 2>/dev/null \
                | awk '/Source Output #/{id=$3} /application.name = ".*[Ss]creen[Cc]ast|.*Shell.Screencast/{print
                ↪  id}' | tr -d '#'); do
        pactl move-source-output "$so" osmo_rec.monitor 2>/dev/null \
            && echo -e "  ${GREEN}[audio] enregistreur d ecran bascule sur osmo_rec.monitor${NC}"
    done
    return 0
}

ensure_local_loopback() {
    [ "${AUDIO:-1}" = "1" ] || return 0
    [ "${AUDIO_LOCAL_LOOPBACK:-1}" = "1" ] || {
        echo -e "  ${YELLOW}[audio] loopback local desactive (AUDIO_LOCAL_LOOPBACK=0)${NC}"; return 0; }
    pactl info >/dev/null 2>&1 || return 0

    pactl list short sources 2>/dev/null | grep -qw 'gsm_audio.monitor' || {
        echo -e "  ${YELLOW}[audio] gsm_audio.monitor absent - loopback local ignore${NC}"; return 0; }

    # With the postmarketOS phone the voice goes through the VM: no direct loopback.
    if pmos_vm_audio_present; then
        echo -e "  ${GREEN}[audio] telephone postmarketOS en place - l ecoute passe par la VM, pas de bouclage
        ↪  direct${NC}"
        remove_direct_loopbacks
        return 0
    fi

    local sink
    sink="$(audio_hp_sink)"
    [ -n "$sink" ] || {
        echo -e "  ${YELLOW}[audio] aucune sortie materielle - loopback local ignore${NC}"; return 0; }

    # Idempotent: do not stack a 2nd loopback (doubled voice + echo)...
    if pactl list short modules 2>/dev/null | grep -F 'module-loopback' \
         | grep -F 'source=gsm_audio.monitor' | grep -F "sink=${sink}" | grep -qF 'sink_dont_move=true'; then
        echo -e "  ${GREEN}[audio] loopback local deja en place → ${sink}${NC}"
        return 0
    fi
    # ...and do not keep one towards ANOTHER output (the raw card installed
    # before the echo canceller, for example).
    local m
    for m in $(pactl list short modules 2>/dev/null \
                | awk '/module-loopback/ && /source=gsm_audio\.monitor/ { print $1 }'); do
        pactl unload-module "$m" >/dev/null 2>&1
```

```
    done

    # PINNED. PulseAudio 16 moves any unpinned stream to the new default device
    # (set-default-sink / GNOME "Output"): one click in the sound settings and
    # this loopback would read the MICROPHONE (default source) to play it into
    # the raw card (default sink). dont_move makes it deaf to default changes:
    # it only moves if unloaded.
    if pactl load-module module-loopback \
            source=gsm_audio.monitor sink="$sink" \
            sink_dont_move=true source_dont_move=true \
            latency_msec="$LOOPBACK_LATENCY_MSEC" >/dev/null 2>&1; then
        echo -e "  ${GREEN}[audio] loopback local charge : gsm_audio.monitor → ${sink} (${LOOPBACK_LATENCY_MSEC}
        ↪  ms)${NC}"
    else
        echo -e "  ${YELLOW}[audio] echec du loopback local vers ${sink}${NC}"
    fi
}

# ── Local microphone: the machine's sound card → gsm_mic ─────────────
# SYMMETRIC to ensure_local_loopback. The uplink (what the mobile SENDS to
# the network) starts from gsm_in, i.e. from gsm_mic.monitor, a null-sink
# that nothing on the machine feeds by itself. The web console can feed it
# (the browser captures the microphone, pushes the samples to
# /opt/GSM/osmo-egprs-web/server.js, which reinjects them through
# `pacat --playback -d gsm_mic`), but the browser -> node -> socket transport
# adds 3 to 4 seconds of delay that no pulse setting can reach. A
# module-loopback from the sound card removes that transport: the microphone
# feeds gsm_mic directly, in 20 ms.
#
# ⚠ TWO SOURCES NEVER TO USE HERE:
#   - gsm_audio.monitor: this is the DOWNLINK. Replaying it into the uplink is
#     a closed loop (RMS x11 in 10 s), as documented in /etc/asound.conf.
#     Only a HARDWARE input is kept.
#   - gsm_mic.monitor: the sink being fed - it would copy itself onto itself.
#
# ⚠ ACOUSTIC loop: the downlink goes out on the speakers (loopback above)
# and the microphone hears them again. Irrelevant with a headset; on speakers,
# mute one of the two or lower the volume.
AUDIO_LOCAL_MIC="${AUDIO_LOCAL_MIC:-1}"

ensure_local_mic() {
    [ "${AUDIO:-1}" = "1" ] || return 0
    [ "$AUDIO_LOCAL_MIC" = "1" ] || {
        echo -e "  ${YELLOW}[audio] micro local desactive (AUDIO_LOCAL_MIC=0)${NC}"; return 0; }
    pactl info >/dev/null 2>&1 || return 0

    pactl list short sinks 2>/dev/null | grep -qw 'gsm_mic' || {
        echo -e "  ${YELLOW}[audio] sink gsm_mic absent - micro local ignore${NC}"; return 0; }

    # With the postmarketOS phone, the VM feeds gsm_mic.
    if pmos_vm_audio_present; then
        echo -e "  ${GREEN}[audio] telephone postmarketOS en place - le micro passe par la VM, pas de bouclage
        ↪  direct${NC}"
        remove_direct_loopbacks
        return 0
    fi

    # Input = the operator's microphone (echo canceller if present, otherwise
    # the hardware) - never a .monitor (see the warning above).
    local src
    src="$(audio_mic_source)"
    [ -n "$src" ] || {
        echo -e "  ${YELLOW}[audio] aucune entree materielle - micro local ignore${NC}"; return 0; }

    # Idempotent: a 2nd loopback would double the uplink voice...
    if pactl list short modules 2>/dev/null | grep -F 'module-loopback' \
```

```
         | grep -F "source=${src}" | grep -F 'sink=gsm_mic' | grep -qF 'sink_dont_move=true'; then
        echo -e "  ${GREEN}[audio] micro local deja en place : ${src} → gsm_mic${NC}"
        return 0
    fi
    # ...and not one from ANOTHER input.
    local m
    for m in $(pactl list short modules 2>/dev/null \
               | awk '/module-loopback/ && /sink=gsm_mic([ \t]|$)/ { print $1 }'); do
        pactl unload-module "$m" >/dev/null 2>&1
    done

    if pactl load-module module-loopback \
            source="$src" sink=gsm_mic \
            sink_dont_move=true source_dont_move=true \
            latency_msec="$LOOPBACK_LATENCY_MSEC" >/dev/null 2>&1; then
        echo -e "  ${GREEN}[audio] micro local charge : ${src} → gsm_mic (${LOOPBACK_LATENCY_MSEC} ms)${NC}"
        echo -e "  ${CYAN}       → la console web n'est plus necessaire pour parler.${NC}"
    else
        echo -e "  ${YELLOW}[audio] echec du micro local depuis ${src}${NC}"
    fi
}

# Post-condition: do the PCMs that `mobile` will actually open really open?
# Testing the sink with `pactl list sinks` is not enough - it is the ALSA
# mapping of /etc/asound.conf that breaks (gsm_in → gsm_mic.monitor). Each
# side is really opened for 1 s and a failure is reported loudly; otherwise
# the failure stays silent until the first mute call.
assert_audio_devices() {
    command -v aplay >/dev/null 2>&1 || return 0
    local ko=0
    timeout 5 aplay  -D gsm_out -f S16_LE -r 8000 -c 1 -d 1 /dev/zero >/dev/null 2>&1 || {
        echo -e "  ${RED}[audio] KO : lecture 'gsm_out' impossible (sink gsm_audio ?)${NC}"; ko=1; }
    timeout 5 arecord -D gsm_in  -f S16_LE -r 8000 -c 1 -d 1 /dev/null  >/dev/null 2>&1 || {
        echo -e "  ${RED}[audio] KO : capture 'gsm_in' impossible (sink gsm_mic ?)${NC}"; ko=1; }
    if [ "$ko" = "1" ]; then
        echo -e "  ${RED}       → gapk_io va echouer et l'appel sera MUET DANS LES DEUX SENS.${NC}"
        echo -e "  ${RED}       → pactl list short sinks  (attendu : gsm_audio ET gsm_mic)${NC}"
        return 1
    fi
    echo -e "  ${GREEN}[audio] gsm_out (lecture) + gsm_in (capture) ouverts - chaine OK${NC}"
    return 0
}

ensure_pulse() {
    [ "${AUDIO:-1}" = "1" ] || { echo -e "  ${YELLOW}[audio] desactive (AUDIO=0)${NC}"; return 0; }

    # 0. ALSA mapping gsm_out/gsm_in → PulseAudio sink gsm_audio (REQUIRED on
    #    the host in native mode). Without /etc/asound.conf, the `mobile`
    #    (io-handler gapk, alsa-output-dev gsm_out) opens a non-existent ALSA
    #    PCM → "Unknown PCM gsm_out" → the TCH audio is never decoded to
    #    gsm_audio → silence in the browser. The sink alone is not enough.
    if [ -f "$HERE/configs/asound.conf" ] && ! cmp -s "$HERE/configs/asound.conf" /etc/asound.conf 2>/dev/null;
     ↪  then
        cp -f "$HERE/configs/asound.conf" /etc/asound.conf \
            && echo -e "  ${GREEN}[audio] /etc/asound.conf deploye (ALSA gsm_out/gsm_in → pulse gsm_audio)${NC}"
    fi

    export PULSE_SERVER="unix:${PULSE_SOCK}"
    if pactl info >/dev/null 2>&1; then
        # PulseAudio already running (osmo-pulse service at boot, or a previous
        # 'fake' solo). The gsm_audio sink is NOT necessarily loaded in THIS
        # daemon -> it is (re)loaded on the fly if missing.
        load_gsm_sinks
        # Deduplication: on a PipeWire/Pulse SHARED between containers, each
        # ensure_pulse loads its own module-null-sink gsm_audio → duplicates.
        # parec (the dashboard /audio stream) then reads the monitor of a sink
```

```
    # that gapk does NOT feed → mute audio for remote clients
    # (browser/Windows; locally the host hears through the module-loopback
    # to its speakers). ONE gsm_audio (the 1st module) is kept and the
    # others are unloaded.
    # (the deduplication applies to gsm_mic as much as to gsm_audio: with a
    #  duplicated gsm_mic, gapk captures the monitor of the wrong sink)
    local _entry _name
    for _entry in $GSM_SINKS; do
        _name="${_entry%%:*}"
        pactl list short modules 2>/dev/null \
            | awk -v s="$_name" '/module-null-sink/ && $0 ~ ("sink_name=" s "([ \t]|$)") {print $1}' \
            | tail -n +2 \
            | while read -r _m; do [ -n "$_m" ] && pactl unload-module "$_m" >/dev/null 2>&1 \
                && echo -e "  ${YELLOW}[audio] sink ${_name} en double decharge (module $_m)${NC}"; done
    done
    echo -e "  ${GREEN}[audio] PulseAudio deja actif (sinks gsm_audio + gsm_mic uniques assures)${NC}"
    assert_audio_devices || true
    alleger_audio
    ensure_local_loopback
    ensure_local_mic
    ensure_record_mix
    return 0
fi

# 1. Install pulseaudio if missing (the daemon binary, not only the clients)
if ! command -v pulseaudio >/dev/null 2>&1; then
    if command -v apt-get >/dev/null 2>&1; then
        echo -e "  ${YELLOW}[audio] installation pulseaudio...${NC}"
        DEBIAN_FRONTEND=noninteractive apt-get install -y -q \
            pulseaudio pulseaudio-utils alsa-utils >/dev/null 2>&1 || true
    fi
fi
command -v pulseaudio >/dev/null 2>&1 || {
    echo -e "  ${YELLOW}[audio] pulseaudio indisponible - audio ignore${NC}"; return 0; }

# 2. system.pa config: anonymous access + the null sinks (idempotent).
#    Declaring them here rather than relying on a load-module at runtime is
#    what makes the chain robust: they exist from the daemon start, even
#    if the daemon restarts on its own later.
local sp=/etc/pulse/system.pa
if [ -f "$sp" ]; then
    grep -q 'auth-anonymous=1' "$sp" || sed -i \
       's|^load-module module-native-protocol-unix.*|load-module module-native-protocol-unix auth-anonymous=1
        ↪  socket=/var/run/pulse/native|' "$sp"
    local _entry _name _desc
    for _entry in $GSM_SINKS; do
        _name="${_entry%%:*}"; _desc="${_entry##*:}"
        grep -q "sink_name=${_name}\b" "$sp" || \
            echo "load-module module-null-sink sink_name=${_name} format=s16le rate=8000 channels=1
             ↪  sink_properties=device.description=${_desc}" >> "$sp"
    done
fi

# 3. (Re)start the system daemon - SERIALIZED
mkdir -p /var/run/pulse "$LOG_DIR"
# ensure_pulse is called in parallel (main flow + ensure_gapk through the
# 'gapk' tmux session) → two 'pulseaudio --system' would start at the same
# time and fight over /var/run/pulse/native → "bind(): Address already in
# use" → the socket module fails → dead daemon → unreachable.
# flock guarantees ONE SINGLE (re)start at a time; a leftover daemon
# holding a stale socket (sometimes under another uid) is killed and the
# socket removed by root before the bind.
(
    flock 9
    if ! pactl info >/dev/null 2>&1; then
        pkill -x pulseaudio 2>/dev/null || true
```

```
            local _k=10
            while pgrep -x pulseaudio >/dev/null 2>&1 && [ $_k -gt 0 ]; do sleep 0.3; ((_k--)) || true; done
            pkill -9 -x pulseaudio 2>/dev/null || true
            rm -f /var/run/pulse/pid /var/run/pulse/native 2>/dev/null || true
            chown -R pulse:pulse /var/run/pulse 2>/dev/null || true
            pulseaudio --system --daemonize=yes --disallow-exit --exit-idle-time=-1 \
                --log-target="file:${LOG_DIR}/pulse-system.log" >/dev/null 2>&1 || true
            local r=10
            while [ $r -gt 0 ]; do pactl info >/dev/null 2>&1 && break; sleep 1; ((r--)) || true; done
        fi
    ) 9>/run/osmo-pulse.lock

    if pactl info >/dev/null 2>&1; then
        load_gsm_sinks
        echo -e "  ${GREEN}[audio] PulseAudio pret (sinks gsm_audio + gsm_mic) @ ${PULSE_SOCK}${NC}"
        assert_audio_devices || true
        ensure_local_loopback
        ensure_local_mic
        ensure_record_mix
    else
        echo -e "  ${YELLOW}[audio] PulseAudio injoignable - audio degrade${NC}"
        echo -e "  ${YELLOW}        → voir ${LOG_DIR}/pulse-system.log${NC}"
    fi
}

# ── Audio bridge: osmo-gapk auto (network path MGW RTP → sink gsm_audio) ─────
# gapk auto polls the OsmoMGW VTY (4243) and bridges the RTP of EACH call to
# alsa://gsm_out (= PulseAudio null sink gsm_audio). This is the link that
# makes the audio of "network" calls (e.g. 600 = Asterisk echo-test through
# the MGW) audible in the web dashboard: server.js captures gsm_audio.monitor
# → /audio (MP3).
# Started in a DETACHED tmux session 'gapk': survives the 'exec'
# (start-clean.sh / tmux attach) of the qemu/hybrid modes and polls until the
# MGW is up. Idempotent (restarts the session), non-fatal. AUDIO=0 → disabled.
# Container → host voice bridge: parec(gsm_audio.monitor) | paplay(--server=relay).
# run.sh (no-process) attempts this bridge BEFORE PulseAudio is ready (race →
# "PulseAudio injoignable apres 30s" → bridge never started → MUTE VOICE in
# docker). It is (re)started HERE, after ensure_pulse (gsm_audio sink
# guaranteed up). Idempotent, gated on HOST_AUDIO_RELAY (set by
# start-docker.sh = tcp:<gw>:4713).
ensure_host_audio() {
    local relay="${HOST_AUDIO_RELAY:-}"
    [ -n "$relay" ] || return 0
    if ! pactl --server="$relay" info >/dev/null 2>&1; then
        echo -e "  ${YELLOW}[host-audio] relai ${relay} injoignable - pont voix non lance${NC}"; return 0
    fi
    # ── ONE SINGLE BRIDGE FOR THE WHOLE BENCH ──────────────────────────────
    # The `pkill` below runs INSIDE the container, which has its own PID
    # namespace: it never sees the neighbour's bridge. Each operator would
    # start its own, and as they all push to the SAME host PulseAudio, the
    # call would be heard twice with two operators, three times with three.
    # So the SHARED SERVER is queried, not the local process table: a client
    # already connected under this name means a bridge is running somewhere,
    # whatever the container.
    #
    # ⚠ scripts/run.sh (audio_bridge) carries the SAME guard. Both copies
    # must keep THE SAME client name: it is through this name that they see
    # each other.
    local _bridge_name="osmo-gsm-bridge"
    if pactl --server="$relay" list clients 2>/dev/null \
         | grep -q "application.name = \"${_bridge_name}\""; then
        echo -e "  ${GREEN}[host-audio] pont deja actif sur l hote (${relay}) - on n en ajoute pas un second${NC}"
        return 0
    fi
    # ── NO DUPLICATE WITH THE HOST LOOPBACK EITHER ─────────────────────────
    # The host already loads its module-loopback gsm_audio.monitor -> sound
```

```
    # card (ensure_local_loopback, above in this file). This is the SAME job
    # as this bridge: same source, same output. Both together, everything is
    # heard TWICE - and the symptom survives stopping a container. The
    # loopback is kept: it is on the host side and much shorter (20 ms against
    # 250 ms for the TCP bridge).
    if pactl --server="$relay" list short modules 2>/dev/null \
         | grep -q 'source=gsm_audio.monitor'; then
        echo -e "  ${GREEN}[host-audio] loopback gsm_audio deja en place sur l hote - pont TCP inutile${NC}"
        return 0
    fi
    pkill -f "paplay --server=${relay}" 2>/dev/null || true          # idempotent (local)
    [ -f /run/host-audio.pid ] && kill -- "-$(cat /run/host-audio.pid)" 2>/dev/null || true
    # parec reads the LOCAL pulse (gsm_audio.monitor); paplay pushes to the host.
    # --latency-msec: short capture (30ms) + BUFFERED playback (250ms) to absorb
    # jitter (scheduling/TCP/2 pulse clocks) - otherwise CHOPPY voice.
    # --client-name: this is what makes this bridge visible to the guard above,
    # from any container. Without it, each one believes it is alone.
    setsid env PULSE_SERVER="unix:${PULSE_SOCK}" sh -c '
      while true; do
        parec -d gsm_audio.monitor --latency-msec=30 --format=s16le --rate=8000 --channels=1 \
          | paplay --server='"${relay}"' --client-name='"${_bridge_name}"' --latency-msec=250 --raw --format=s16le
↪   --rate=8000 --channels=1
        sleep 1
      done' >"${LOG_DIR}/host-audio.log" 2>&1 &
    echo $! > /run/host-audio.pid
    echo -e "  ${GREEN}[host-audio] pont voix parec|paplay → hote (${relay})${NC}"
}

ensure_gapk() {
    [ "${AUDIO:-1}" = "1" ] || return 0
    command -v tmux      >/dev/null 2>&1 || { echo -e "  ${YELLOW}[gapk] tmux absent - bridge audio non
     ↪  lance${NC}"; return 0; }
    command -v osmo-gapk >/dev/null 2>&1 || { echo -e "  ${YELLOW}[gapk] osmo-gapk absent - bridge audio non
     ↪  lance${NC}"; return 0; }
    ensure_pulse   # gapk writes into alsa://gsm_out = sink gsm_audio (idempotent)
    local gapk_sh="/etc/osmocom/gapk-start.sh"; [ -x "$gapk_sh" ] || gapk_sh="$HERE/scripts/gapk-start.sh"
    [ -x "$gapk_sh" ] || { echo -e "  ${YELLOW}[gapk] gapk-start.sh introuvable - bridge audio non lance${NC}";
     ↪  return 0; }
    tmux kill-session -t gapk 2>/dev/null || true
    # The GSM FR format is named "gsm" in osmo-gapk ("gsmfr" does not exist:
    # both processes would die at argument parsing, "Unsupported format").
    # The 3rd argument is the CAPTURE device: it must stay gsm_in (monitor of
    # the gsm_mic null-sink). Pointing it at gsm_out creates a closed loop -
    # the mobiles write into gsm_out, they would be replayed into their own
    # uplink.
    tmux new-session -d -s gapk \
        "GAPK_ALSA_DEV=gsm_out GAPK_ALSA_DEV_IN=gsm_in PULSE_SERVER=unix:${PULSE_SOCK} bash '$gapk_sh' auto gsm
         ↪  gsm_out gsm_in 2>&1 | tee ${LOG_DIR}/gapk-auto.log"
    # `tmux new-session -d` returns 0 as soon as the server has taken the
    # command, not when it runs. If gapk-start.sh returns right away - format
    # refused, sink not there yet, binary missing - the window stays displayed
    # with "[exited]" across it while the green message below would still
    # announce a working audio bridge. So wait one second and check.
    # A dead session is removed: an "exited" window among the others makes one
    # look for a failure in the stack when only the audio bridge did not
    # start - and the log says why.
    sleep 1
    if tmux has-session -t gapk 2>/dev/null && \
       [ "$(tmux list-panes -t gapk -F '#{pane_dead}' 2>/dev/null | head -1)" != "1" ]; then
        echo -e "  ${GREEN}[gapk] auto lance (RTP MGW → sink gsm_audio) - tmux 'gapk', log
         ↪  ${LOG_DIR}/gapk-auto.log${NC}"
    else
        tmux kill-session -t gapk 2>/dev/null || true
        echo -e "  ${YELLOW}[gapk] mort au demarrage - pas de pont audio (fenetre retiree)${NC}"
        echo -e "         ${CYAN}tail -20 ${LOG_DIR}/gapk-auto.log${NC}"
    fi
```

```
    ensure_host_audio   # (re)start the voice bridge → host now that pulse+sink are up
}

# ============================================================================
# Single-operator modes (host loopback + enp0s3): noproc/faketrx/virtphy/qemu/combine
# ============================================================================
# ============================================================================
# HYBRID mode faketrx-qemu (approach C): 2 osmo-bts-trx, 1 core
#   BTS#0 = UNTOUCHED QEMU pipeline (run.sh: osmo-trx-ipc 5700, ARFCN 514, Calypso)
#   BTS#1 = side-car: osmo-bts-trx (unit-id 6002, base-port 5820/5720) + fake_trx
#           (-P 5720 -p 6720) + trxcon + osmocom-bb mobile (ARFCN 516, IMSI ...0002)
#   Both MS register on the same osmo-bsc/MSC/HLR → intra-MSC call.
#   1 BTS = 1 clock (clk_s per osmo-bts-trx process) → no clock conflict:
#   a single 2-PHY osmo-bts-trx is IMPOSSIBLE (shared clk_s, reset ping-pong),
#   hence 2 distinct processes. Neither qosmo-grgsm/run.sh nor osmo-bts-trx.cfg
#   is touched.
# ============================================================================
```

2.97 /opt/GSM/osmo-operator/lib/gabarits.sh

25035 octets, 456 lignes → 456 lignes

```
# ============================================================================
#  lib/gabarits.sh - the osmo-operator configuration template engine
# ============================================================================
#
#  The functions below substitute __ENCRYPTION__, __MCC__, __KI__, __ARFCN__ ...
#  in configs/*.cfg and install the result in /etc/osmocom, /etc/asterisk and
#  ~/.osmocom/bb. Without them, the core daemons start on the files left by a
#  previous run, and `ENCRYPTION=a5 0` has no effect, silently.
#
#  THIS FILE DOES NOTHING WHEN SOURCED: it only defines functions.
#  The only expected caller is run_modules/08-gabarits.sh.
#
#  ENVIRONMENT DEPENDENCIES (set by the caller, defaults below): ENCRYPTION,
#  HOST_IP, ALSA_OUTPUT, ALSA_INPUT.
#  The current directory must be the osmo-operator one: the functions read
#  `configs/*.cfg` and `scripts/*` by RELATIVE path.
# ----------------------------------------------------------------------------

: "${ENCRYPTION:=a5 0}"
: "${HOST_IP:=127.0.0.1}"
: "${ALSA_OUTPUT:=default}"
: "${ALSA_INPUT:=default}"

# These functions have a TWIN in start-docker.sh: both must say the same thing.
# THIS copy is the one read by a machine that reclones the repository at every
# boot, i.e. the one that matters on an ISO.
#
#   backbone  172.20.0.<10+op>   the segment shared with the inter-STP (.10)
#   private   192.168.<op+1>.x   one segment per operator; the +1 leaves
#                                192.168.1.0/24 to the bench LAN.
op_backbone_ip()  { echo "172.20.0.$((10 + $1))"; }
# ── THE PRIVATE SEGMENT INDEX: THE NODE OR THE OPERATOR, DEPENDING ON THE HOST ─
# 192.168.<index+1>.x, where "index" depends on where the code runs:
#
#   DOCKER    a host carries N operator containers, each in its own netns.
#             What distinguishes them is the OPERATOR NUMBER; the node is
#             common to all containers of the machine. index = operator.
#   VM/NATIVE the machine IS the node and carries only one operator. What
#             distinguishes it from its neighbours is the NODE NUMBER.
#             index = node.
#
# Without this distinction, every VM would write 192.168.2.10 (operator 1) in
```

```
# osmo-ggsn.cfg while the ISO plan reserves 192.168.<node+1>.x: on node 2 the
# GGSN would bind to the address of node 1. The switch is here, once, and both
# twins (start-docker.sh, lib/gabarits.sh) inherit it.
_osmo_priv_index() {
    local op="${1:-1}"
    # THE ORCHESTRATOR IS NOT INSIDE THE CONTAINER IT CREATES.
    # The detection below asks "am I INSIDE a container?", the right question
    # for code running inside one. start-docker.sh, however, provisions the
    # containers FROM THE HOST: it would take the VM/native branch and return
    # the NODE index - 1 - for every operator, so both private networks would
    # land on 192.168.2.0/24 and docker would refuse the second one for
    # overlap:
    #     ── Operateur 2 : ... Prive : 192.168.2.10
    #     ✗ Echec creation reseau gsm-net-op2 (192.168.2.0/24)
    # OSMO_PRIV_BY_OP=1 says "I provision per operator"; start_bridge_mode
    # sets it. It is the only thing the detection cannot guess.
    [ "${OSMO_PRIV_BY_OP:-0}" = "1" ] && { printf '%s' "$op"; return; }
    # Same detection as start-native.sh: the pair /.dockerenv +
    # /etc/docker-entrypoint-cmd identifies a container OF THIS REPOSITORY; the
    # cgroup is the fallback for any container.
    if [ -f /.dockerenv ] || grep -qa 'docker\|containerd' /proc/1/cgroup 2>/dev/null; then
        printf '%s' "$op"; return
    fi
    local n="${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}"
    [ -n "$n" ] || n="$(awk -F= '/^OSMO_WAN_NODE=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' \
                        "${ROLE_FILE:-/etc/osmo-role}" 2>/dev/null)"
    [ -n "$n" ] || n="$(sed -n 's/^PLAN_NODE=//p' "${OSMOCOM_CFG:-/etc/osmocom}/radio-plan.env" 2>/dev/null | tail
    ↪  -1)"
    case "$n" in [1-9]) ;; *) n=1 ;; esac
    printf '%s' "$n"
}
op_private_ip()   { echo "192.168.$(($(_osmo_priv_index "$1") + 1)).10"; }
op_private_gw()   { echo "192.168.$(($(_osmo_priv_index "$1") + 1)).1"; }
op_private_net()  { echo "192.168.$(($(_osmo_priv_index "$1") + 1)).0/24"; }
op_netns()        { echo "osmo-op$1"; }
op_rctx_msc()     { echo $(( $1 * 100 + 10 )); }
op_rctx_stp()     { echo $(( $1 * 100 + 20 )); }
op_rctx_bsc()     { echo $(( $1 * 100 + 30 )); }
op_rctx_inter()   { echo $(( $1 * 100 + 50 )); }
# ── NO PORT OFFSET ──────────────────────────────────────────────────────
# No SIP/RTP port is published for the containers: they sit on a routable
# bridge and are reached directly, at their own address, on the nominal port
# (172.20.0.12:5060 and 172.20.0.13:5060 answer 401 from the host without any
# publication). So there is no conflict with the native Asterisk on 5060 and
# no offset to compute.
#
# The functions below keep their shape (generate_configs.sh still uses them to
# write the configs) but return the NOMINAL values.
# OSMO_PORT_OFFSET=N remains available for whoever wants an offset.
PORT_OFFSET="${OSMO_PORT_OFFSET:-0}"
# RTP follows the same rule: nominal range, 200 ports per operator.
linphone_sip_port()  { echo $(( 5060  + ${PORT_OFFSET:-0}      + ($1 - 1) )); }
linphone_rtp_start() { echo $(( 30000 + ${PORT_OFFSET:-0} * 10 + ($1 - 1) * 200 )); }
linphone_rtp_end()   { echo $(( 30000 + ${PORT_OFFSET:-0} * 10 + $1 * 200 - 1 )); }
# Both twins enumerate the operators from the same base: with OP_ID_BASE=2 the
# containers take ranks 2 and 3 and rank 1 stays native. A loop hard-coded on
# 1..N would build a trunk to "operator 1" at 172.20.0.11, a backbone address
# NOBODY carries (the native one is on the host), and none to operator 3: the
# trunk would exist but point into the void.
generate_pjsip_interop_trunks() {
    local op_id=$1 n_operators=$2 remote_op remote_ip
    # ── THE OPERATORS BELOW THE BASE ARE THE NATIVE ONES ────────────────
    # The loop starts at 1, not at the base: with OP_ID_BASE=2 (containers on
    # ranks 2..N, rank 1 native on the host) op2 must also get a trunk to the
    # native operator, otherwise a call 100201 -> 100101 falls into the _X.
    # catch-all of [interop_out] and ends in Congestion.
```

```
    for remote_op in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))) \
                     $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        [ "$remote_op" -eq "$op_id" ] && continue
        # An operator below the base is NATIVE: it runs on the host, not in a
        # container. Seen from a container, the host is the docker bridge
        # gateway - the same address as the audio bridge and sms-routing.conf.
        # NB: match=172.20.0.1 does not swallow the Linphone REGISTERs, which
        # arrive through the same gateway: pjsip.conf sets
        # endpoint_identifier_order=username,ip,anonymous - username first.
        if [ "$remote_op" -lt "${OP_ID_BASE:-1}" ]; then
            remote_ip="${INTER_NET_GATEWAY:-172.20.0.1}"
        else
            remote_ip=$(op_backbone_ip "$remote_op")
        fi
        cat <<EOF

[interop-identify-op${remote_op}]
type=identify
endpoint=interop_trunk_op${remote_op}
match=${remote_ip}

[interop_trunk_op${remote_op}]
type=endpoint
transport=transport-udp
context=interop_in
disallow=all
allow=gsm
allow=ulaw
aors=interop_trunk_op${remote_op}
direct_media=no
rtp_symmetric=yes
force_rport=yes
media_encryption=no

[interop_trunk_op${remote_op}]
type=aor
contact=sip:${remote_ip}:5060
qualify_frequency=15
qualify_timeout=5.0
EOF
    done
}
generate_extensions_interop_out() {
    local op_id=$1 n_operators=$2 remote_op
    # The prefix carries the NODE: MSISDN = <node>00<operator><rank>. A fixed
    # "600" would not match anything once the number starts with the node
    # number - inter-operator calls would end in Congestion without any line
    # saying that the pattern does not match.
    local _pfx; _pfx="$(osmo_msisdn_pfx "$(osmo_node_id)")"
    printf '[interop_out]\n\n'
    # One single pattern: MSISDNs are six digits, <node>00<operator><rank>.
    # ── THE OPERATORS BELOW THE BASE ARE THE NATIVE ONES ──────────────────
    # The loop starts at 1, not at the base: with OP_ID_BASE=2 (containers on
    # ranks 2..N, rank 1 native on the host) op2 must also get a route to the
    # native operator, otherwise a call 100201 -> 100101 falls into the _X.
    # catch-all of [interop_out] and ends in Congestion. Same rule as
    # sms-routing.conf, on the VOICE path.
    for remote_op in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))) \
                     $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        [ "$remote_op" -eq "$op_id" ] && continue
        cat <<EOF
exten => _${_pfx}${remote_op}XX,1,NoOp(=== INTEROP OUT Op${remote_op}: \${EXTEN} ===)
 same => n,Dial(PJSIP/\${EXTEN}@interop_trunk_op${remote_op},,rT)
 same => n,Congestion()
 same => n,Hangup()
```

```
EOF
    done

    # ── THE SOFTPHONES TOO, AND WITHOUT PREFIX ──────────────────────────
    # From [internal], a Linphone can reach another operator by dialling the
    # number as is: without this route, 100201 would fall into the _X.
    # catch-all of [internal] and go to the LOCAL MSC, which returns
    #     rx MNCC_SETUP_REQ for unknown subscriber number '100201'
    # before sending it back through MNCC to Asterisk - a detour that consumes
    # an MNCC transaction for nothing and makes a SIP->SIP call depend on the
    # MSC. _<pfx><op>XX is MORE SPECIFIC than _X.: Asterisk picks it first,
    # without touching the template's catch-all. Gosub(sub-record) as for _9.,
    # otherwise these calls would be the only ones not recorded.
    printf '[internal]\n\n'
    for remote_op in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))) \
                     $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        [ "$remote_op" -eq "$op_id" ] && continue
        cat <<EOF
exten => _${_pfx}${remote_op}XX,1,NoOp(=== SIP -> INTER-OP Op${remote_op}: \${EXTEN} - CallerID:
 ↪  \${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Goto(interop_out,\${EXTEN},1)

EOF
    done
    # NO _X. catch-all here: the configs/extensions.conf template already
    # declares one in this same context. Two identical extensions in a context
    # make Asterisk refuse the second, with six warnings at every load:
    #     add_priority: Unable to register extension '_X.' priority 1
    #                   in 'interop_out', already in use
}
_generate_sms_routing_conf_fallback() {
    # 'local i' is NOT cosmetic. This function is called, through
    # apply_config_templates, FROM the "for i in ..." loop that starts the
    # operators. Without local, its own loop overwrites the caller's variable
    # and leaves it at n_operators: all following containers would compute
    # their ports with the SAME index, hence
    #     Bind for 0.0.0.0:5082 failed: port is already allocated
    # on the second container - and, before the error, two operators announced
    # on the same SIP/RTP/SMS.
    local i
    local op_id=$1 n_operators=$2 i j
    printf '# sms-routing.conf - Fallback\n\n[local]\noperator_id = %s\nsc_address  =
     ↪  1999001%s444\n\n[operators]\n' "$op_id" "$op_id"
    # ── THE OPERATORS BELOW THE BASE ARE THE NATIVE ONES ────────────────
    # start-multi.sh sets OP_ID_BASE=2: the containers take ranks 2..N and
    # operator 1 stays NATIVE, on the host. It must still appear in [operators]
    # and [routes], otherwise an SMS to 100101 ends in "No route for
    # destination" and the native operator is unreachable from the whole bench.
    # It is pointed at the docker bridge gateway, which IS the host seen from a
    # container - the same address as the audio bridge (172.20.0.1).
    local _natif_gw="${INTER_NET_GATEWAY:-172.20.0.1}"
    for i in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))); do printf '%s = %s\n' "$i" "$_natif_gw"; done
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do printf '%s = %s\n' "$i"
     ↪  "$(op_backbone_ip "$i")"; done
    printf '\n[routes]\n'
    for i in $(seq 1 $(( ${OP_ID_BASE:-1} - 1 ))); do
        for ms in 1 2; do printf '%s = %s\n' "$(osmo_msisdn "$(osmo_node_id)" "$i" "$ms")" "$i"; done
    done
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_operators - 1 ))"); do
        # EXACT MSISDNs, not a prefix: a prefix too short swallows the
        # neighbouring numbers, a missing prefix makes the relay reject every
        # local SMS with "No route for destination".
        for ms in 1 2; do printf '%s = %s\n' "$(osmo_msisdn "$(osmo_node_id)" "$i" "$ms")" "$i"; done   # exact
         ↪  MSISDNs <node>00<op><ms> (100101, 100102, 200101...)
    done
```

```
    printf '\n[relay]\nport = 7890\nconnect_timeout = 10\nretry_count = 3\nretry_delay = 5\n'
}
# apply_config_templates() lives in generate_configs.sh (one single copy, for
# this file and for start-docker.sh); its values (ARFCN, BSIC, KI, IMSI...)
# are exposed in globals.conf instead of being formulas buried in the
# substitution.
_GAB_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
[ -r "$_GAB_ROOT/generate_configs.sh" ] && . "$_GAB_ROOT/generate_configs.sh"

# ── Native config install. $1=src $2=root prefix (/ or /etc/netns/<ns>) ──
# ── CONTAINER ADDRESSES HAVE NO PLACE ON THE HOST ──────────────────────
# The templates are written for a CONTAINER: they set the operator backbone
# address (172.20.0.<10+N>) and the docker bridge gateway (172.20.0.1). On the
# host, neither exists.
#
# apply_native_post_patches (generate_configs.sh) also fixes these, but only
# start-native.sh calls it; qosmo-grgsm/run_modules/08-gabarits.sh calls ONLY
# apply_config_templates + install_configs_native and does not even source
# generate_configs.sh. So the fixes are hooked here, in install_configs_native
# - the MANDATORY path of the native mode, whatever the caller. Idempotent:
# replaying changes nothing.
_gab_fixups_natifs() {
    local src="$1" f

    # 1. osmo-stp.cfg: the local-ip of the ASP to the inter-STP. Only the
    #    local-ip following "asp asp-to-inter" is touched: those of the listen
    #    block above (127.0.0.1 / 127.0.0.2) are correct. 127.0.0.1 is the only
    #    address one can be sure of on a host, EXCEPT when the hub is on a
    #    network address, even ours: an SCTP bound to 127.0.0.1 towards
    #    192.168.1.31 is refused ("Network is unreachable") and as-inter stays
    #    AS_DOWN. So the source address the kernel picks towards the hub is
    #    used (same rule as generate_configs.sh).
    f="$src/osmocom/osmo-stp.cfg"
    if [ -f "$f" ]; then
        local rip lip=127.0.0.1
        rip="$(awk '/^[[:space:]]*asp[[:space:]]+asp-to-inter[[:space:]]/{x=1;next}
         ↪  x&&/^[[:space:]]*remote-ip[[:space:]]/{print $2; exit} x&&/^ (asp|as) /{exit}' "$f")"
        case "$rip" in
            ""|127.*|__*) ;;   # __* : template not substituted yet
            *) lip="$(ip -4 route get "$rip" 2>/dev/null | awk '{for(i=1;i<NF;i++) if($i=="src") {print $(i+1);
             ↪  exit}}')"
               [ -n "$lip" ] || lip=127.0.0.1 ;;
        esac
        awk -v lip="$lip" '
            /^[[:space:]]*asp[[:space:]]+asp-to-inter[[:space:]]/ { a = 1; print; next }
            a && /^[[:space:]]*local-ip[[:space:]]/ {
                sub(/local-ip[[:space:]]+.*/, "local-ip " lip); a = 0; print; next
            }
            /^[[:space:]]*(as|asp|cs7|listen)[[:space:]]/ && !/asp-to-inter/ { a = 0 }
            { print }
        ' "$f" > "$f.tmp" && mv -f "$f.tmp" "$f"
    fi

    # 2. mobile.cfg: the GSMTAP target. 172.20.0.1 does not exist without the
    #    bridge, and the packets would go nowhere - Wireshark stays empty on
    #    udp/4729 without any message saying so.
    for f in "$src/osmocom/mobile.cfg" "$src/bb/mobile.cfg" "$src/bb/mobile_group1.cfg"; do
        [ -f "$f" ] || continue
        sed -i "/^gsmtap\$/,/^!\$/ s/^\([[:space:]]*remote-host[[:space:]]\+\).*/\1127.0.0.1/" "$f"
    done

    # 3. pjsip.conf: on a flat host there is no NAT boundary to declare.
    #    external_* makes pjsip REWRITE Contact and SDP for any peer outside
    #    local_net - a LAN Linphone would receive an unreachable address.
    #    Without them, pjsip answers with the socket address, the right one,
    #    per destination.
```

```
f="$src/asterisk/pjsip.conf"
if [ -f "$f" ]; then
    awk '
        /^\[transport-udp\]/ { t = 1; print; next }
        t && /^\[/           { t = 0 }
        t && /^[[:space:]]*external_(media|signaling)_address[[:space:]]*=/ { next }
        t && /^[[:space:]]*local_net[[:space:]]*=/ {
            if (!d) {
                print "local_net=127.0.0.0/8"; print "local_net=10.0.0.0/8"
                print "local_net=172.16.0.0/12"; print "local_net=192.168.0.0/16"
                print "local_net=176.16.32.0/24"; d = 1
            }
            next
        }
        { print }
    ' "$f" > "$f.tmp" && mv -f "$f.tmp" "$f"
fi

# 4. sms-routing.conf: THE NATIVE OPERATOR MUST KNOW THE OTHERS.
#    The fallback written by apply_config_templates is meant FOR A
#    CONTAINER: 172.20.0.11 as the operator address (the DOCKER backbone
#    address, which does not exist on the host) and no other operator at
#    all, so an SMS from the native to 100201 would end in "No route for
#    destination". Same trap as points 1 to 3 above, hence the fix in the
#    same place: install_configs_native is the MANDATORY path of the
#    native mode, whatever the caller.
#
#    THE SOURCE OF TRUTH is osmo-multi.conf (written by addition.sh):
#        MULTI_OPS="1:native::1.1.2:150 2:docker:172.20.0.12:... 3:docker:..."
#    The keying by OPERATOR, the containers' one, is kept: on a
#    single-machine bench everybody is on node 1, a keying by node would
#    confuse them. Without osmo-multi.conf (single operator), nothing is
#    touched.
f="$src/osmocom/sms-routing.conf"
local mc="${OSMO_MULTI_CONF:-/etc/osmocom/osmo-multi.conf}"
if [ -f "$f" ] && [ -r "$mc" ]; then
    local _ops _node _self _sc _id _mode _ip _e
    _ops="$(sed -n 's/^MULTI_OPS=//p' "$mc" | tr -d '"' | tail -1)"
    _node="$(sed -n 's/^MULTI_NODE=//p' "$mc" | tail -1)"; case "$_node" in [1-9]) ;; *) _node=1 ;; esac
    # Identity: it is RE-READ, not recomputed - it says which operator this
    # native is, and it may have been realigned by start-multi.sh
    # (aligner_natif).
    _self="$(sed -n 's/^operator_id[[:space:]]*=[[:space:]]*//p' "$f" | head -1)"
    _sc="$(sed -n 's/^sc_address[[:space:]]*=[[:space:]]*//p'  "$f" | head -1)"
    case "$_self" in [1-9]*) ;; *) _self=1 ;; esac
    [ -n "$_sc" ] || _sc="1999001${_self}444"
    if [ -n "$_ops" ]; then
        {
            printf '# sms-routing.conf - natif, reecrit par _gab_fixups_natifs\n'
            printf '# Table issue de %s (MULTI_OPS). Cle = numero d operateur.\n' "$mc"
            printf '# MSISDN = <noeud>00<operateur><rang du MS sur 2 chiffres>\n\n'
            printf '[local]\noperator_id = %s\nsc_address  = %s\n\n[operators]\n' "$_self" "$_sc"
            for _e in $_ops; do
                IFS=: read -r _id _mode _ip _ <<< "$_e"
                [ -n "$_id" ] || continue
                # The native one is US: 127.0.0.1 is the only address one
                # can be sure of on a host (same choice as in point 1).
                if [ "$_mode" = native ] || [ "$_id" = "$_self" ]; then _ip=127.0.0.1; fi
                [ -n "$_ip" ] || continue
                printf '%s = %s\n' "$_id" "$_ip"
            done
            printf '\n[routes]\n'
            for _e in $_ops; do
                IFS=: read -r _id _mode _ip _ <<< "$_e"
                [ -n "$_id" ] || continue
                for _m in 1 2; do
```

```
                    printf '%s = %s\n' "$(( _node * 100000 + _id * 100 + _m ))" "$_id"
                done
            done
            printf '\n[relay]\nport = 7890\nconnect_timeout = 10\nretry_count = 3\nretry_delay = 5\n'
        } > "$f.tmp" && mv -f "$f.tmp" "$f"
    fi
fi

# 5. THE INTER-OPERATOR VOICE PATH: SIP trunks + [interop_out].
#    Same need as point 4, on VOICE. Without it, the native [interop_out]
#    only holds _X. -> Congestion() and pjsip.conf has no
#    interop_trunk_opN: a call 100101 -> 100201 leaves the MSC
#        rx MNCC_SETUP_REQ for unknown subscriber number '100201'
#    goes to osmo-sip-connector, hits the catch-all and comes back as
#        INVITE got status(503), releasing leg
#
#    The logic is NOT rewritten: generate_extensions_interop_out and
#    generate_pjsip_interop_trunks (above in this file) already produce it.
#    They are replayed here with the real topology, as in point 4.
#    OP_ID_BASE=1: seen from the native, all remotes are containers and do
#    carry a backbone address; this is the opposite of the container case,
#    where the native is the one without.
local mc5="${OSMO_MULTI_CONF:-/etc/osmocom/osmo-multi.conf}"
if [ -r "$mc5" ]; then
    local _ops5 _self5 _n5=0 _e5 _i5
    _ops5="$(sed -n 's/^MULTI_OPS=//p' "$mc5" | tr -d '"' | tail -1)"
    _self5="$(sed -n 's/^operator_id[[:space:]]*=[[:space:]]*//p' "$src/osmocom/sms-routing.conf" 2>/dev/null
    ↪  | head -1)"
    case "$_self5" in [1-9]*) ;; *) _self5=1 ;; esac
    for _e5 in $_ops5; do
        IFS=: read -r _i5 _ <<< "$_e5"
        case "$_i5" in [1-9]*) [ "$_i5" -gt "$_n5" ] && _n5="$_i5" ;; esac
    done
    # Idempotence marker: replaying install_configs_native must not stack
    # the same extensions (Asterisk would refuse the duplicates with
    # "already in use" and flood the log).
    local _mk5='; --- interop natif : ajoute par _gab_fixups_natifs ---'
    if [ "$_n5" -gt 1 ]; then
        f="$src/asterisk/extensions.conf"
        if [ -f "$f" ] && ! grep -qF "$_mk5" "$f"; then
            { printf '\n%s\n' "$_mk5"
              OP_ID_BASE=1 generate_extensions_interop_out "$_self5" "$_n5"; } >> "$f"
        fi
        f="$src/asterisk/pjsip.conf"
        if [ -f "$f" ] && ! grep -qF "$_mk5" "$f"; then
            { printf '\n%s\n' "$_mk5"
              OP_ID_BASE=1 generate_pjsip_interop_trunks "$_self5" "$_n5"; } >> "$f"
        fi
    fi
fi
}

install_configs_native() {
    local src=$1 root="${2:-}"
    _gab_fixups_natifs "$src"
    mkdir -p "${root}/etc/osmocom" "${root}/etc/asterisk" "$HOME/.osmocom/bb"
    # /etc/osmocom/.proteges: one file name per line (# comment), never
    # overwritten by the regeneration. The page http://127.0.0.1:8090/ (Configs
    # osmo tab) adds each file it saves; removing the line gives the file back
    # to the template.
    local _prot="${root}/etc/osmocom/.proteges" _f _b
    for _f in "$src/osmocom"/*; do
        [ -e "$_f" ] || continue
        _b="${_f##*/}"
        if [ -f "$_prot" ] && grep -qxF "$_b" <(sed 's/#.*//; s/[[:space:]]*$//' "$_prot"); then
            printf '  gabarits   %s protege (%s) : conserve\n' "$_b" "$_prot" >&2
```

```
            continue
        fi
        cp -f "$_f" "${root}/etc/osmocom/" 2>/dev/null || true
    done
    cp -f "$src/asterisk"/*.conf "${root}/etc/asterisk/" 2>/dev/null || true
    [ -f "$src/bb/mobile.cfg" ]        && cp -f "$src/bb/mobile.cfg"        "$HOME/.osmocom/bb/mobile.cfg"
    [ -f "$src/bb/mobile_group1.cfg" ] && cp -f "$src/bb/mobile_group1.cfg" "$HOME/.osmocom/bb/mobile_group1.cfg"
    if [ -f configs/asound.conf ]; then
        cp -f configs/asound.conf "${root}/etc/asound.conf"
        ALSA_OUTPUT="gsm_out"; ALSA_INPUT="gsm_in"
    fi
}

detect_host_ip() {
    HOST_IP=$(ip route get 1.1.1.1 2>/dev/null \
        | awk '{for(i=1;i<=NF;i++) if($i=="src") print $(i+1)}' | head -1) || true
    [ -z "$HOST_IP" ] && HOST_IP=$(hostname -I 2>/dev/null | awk '{print $1}')
    HOST_IP="${HOST_IP:-127.0.0.1}"
}

# ── tmux auto-attach ──────────────────────────────────────────────────
# run.sh runs in the background and creates the tmux session 'osmocom'
# (socket /tmp/osmocom_tmux). Wait until it is ready (run.sh finished), then
# attach to it in the current terminal. $1 = run.sh log to watch.
#   AUTO_ATTACH=0   → disabled (stays in the background, manual message)
#   Also disabled without a TTY (scripted/bg) or when already inside tmux.
```

2.98 /opt/GSM/osmo-operator/navigation/init.py

699 octets, 14 lignes → 14 lignes

```
# ss7console - the osmo-operator SS7 console.
#
# The package has no dependency beyond the Python 3 standard library. It runs
# on the HOST and talks to the lab through two complementary paths:
#
#   - the Osmocom VTYs, reached through "docker exec" (the VTYs only listen on
#     the NODE's loopback) - reading and driving the daemons;
#   - a real M3UA/SCTP link to the inter-STP, which makes the console a full
#     SS7 point: it can send TCAP/MAP (sendRoutingInfo, sendRoutingInfoForSM,
#     anyTimeInterrogation...) and read the answers.
#
# The whole topology is INHERITED from the repository and node configurations:
# no point code, address or port is hard-coded here.
__version__ = "1.0"
```

2.99 /opt/GSM/osmo-operator/navigation/ber.py

4854 octets, 181 lignes → 181 lignes

```
# ber.py - the bare minimum of BER (X.690): enough to write and read TCAP
# and MAP without an external dependency.
#
# Deliberately small: short or long tags, short, long and indefinite lengths,
# primitive and constructed elements. Nothing more is needed for the MAP
# operations the console sends.

def enc_len(n):
    if n < 0x80:
        return bytes([n])
    out = b""
    while n:
```

```
        out = bytes([n & 0xFF]) + out
        n >>= 8
    return bytes([0x80 | len(out)]) + out


def tlv(tag, value):
    """tag: integer (0x30) or byte sequence for long tags."""
    if isinstance(tag, int):
        tag = bytes([tag])
    return bytes(tag) + enc_len(len(value)) + value


def enc_int(value, tag=0x02):
    if value == 0:
        return tlv(tag, b"\x00")
    neg = value < 0
    raw = b""
    v = value
    if neg:
        v = -value - 1
    while v:
        raw = bytes([v & 0xFF]) + raw
        v >>= 8
    if neg:
        raw = bytes([(~b) & 0xFF for b in raw])
        if not raw or not (raw[0] & 0x80):
            raw = b"\xff" + raw
    elif raw[0] & 0x80:
        raw = b"\x00" + raw
    return tlv(tag, raw)


def enc_bool(value, tag=0x01):
    return tlv(tag, b"\xff" if value else b"\x00")


def enc_null(tag=0x05):
    return tlv(tag, b"")


def enc_oid(dotted, tag=0x06):
    parts = [int(x) for x in dotted.split(".")]
    body = bytes([parts[0] * 40 + parts[1]])
    for p in parts[2:]:
        if p < 0x80:
            body += bytes([p])
            continue
        chunk = []
        while p:
            chunk.insert(0, (p & 0x7F) | 0x80)
            p >>= 7
        chunk[-1] &= 0x7F
        body += bytes(chunk)
    return tlv(tag, body)


def ctx(num, value, constructed=True):
    """Context tag [num], constructed or primitive."""
    tag = 0x80 | num | (0x20 if constructed else 0x00)
    return tlv(tag, value)


class Elem(object):
    __slots__ = ("tag", "constructed", "value", "children", "raw")

    def __init__(self, tag, constructed, value, children, raw):
```

```
        self.tag = tag
        self.constructed = constructed
        self.value = value
        self.children = children
        self.raw = raw

    @property
    def cls(self):
        return (self.tag[0] & 0xC0) >> 6      # 0 univ, 1 appli, 2 ctx, 3 priv

    @property
    def num(self):
        t = self.tag[0] & 0x1F
        if t != 0x1F:
            return t
        n = 0
        for b in self.tag[1:]:
            n = (n << 7) | (b & 0x7F)
        return n

    def find(self, cls, num, deep=True):
        """First element of a given class/number."""
        for e in self.walk() if deep else self.children:
            if e.cls == cls and e.num == num:
                return e
        return None

    def walk(self):
        for c in self.children:
            yield c
            for sub in c.walk():
                yield sub

    def tagstr(self):
        names = {0: "univ", 1: "appl", 2: "ctx", 3: "priv"}
        return "%s[%d]%s" % (names[self.cls], self.num,
                             "c" if self.constructed else "p")

    def __repr__(self):
        return "<%s len=%d>" % (self.tagstr(), len(self.value))


def _read_tag(data, i):
    start = i
    b0 = data[i]
    i += 1
    if (b0 & 0x1F) == 0x1F:
        while i < len(data) and data[i] & 0x80:
            i += 1
        i += 1
    return data[start:i], i


def _read_len(data, i):
    b = data[i]
    i += 1
    if b < 0x80:
        return b, i
    if b == 0x80:
        return -1, i                       # indefinite length
    n = b & 0x7F
    val = int.from_bytes(data[i:i + n], "big")
    return val, i + n


def decode(data, limit=None):
```

```
    """Return the list of top-level elements."""
    out = []
    i = 0
    end = len(data) if limit is None else min(limit, len(data))
    while i < end:
        try:
            tag, i = _read_tag(data, i)
            length, i = _read_len(data, i)
        except IndexError:
            break
        if length < 0:                    # indefinite: up to the 00 00
            j = data.find(b"\x00\x00", i)
            if j < 0:
                j = end
            value = data[i:j]
            nxt = j + 2
        else:
            value = data[i:i + length]
            nxt = i + length
        constructed = bool(tag[0] & 0x20)
        children = decode(value) if constructed and value else []
        out.append(Elem(tag, constructed, value, children, data[:nxt]))
        i = nxt
    return out


def dump(elems, indent=0, out=None):
    """Readable tree - used to show a raw MAP answer."""
    out = [] if out is None else out
    for e in elems:
        head = "%s%s len=%d" % ("  " * indent, e.tagstr(), len(e.value))
        if e.constructed and e.children:
            out.append(head)
            dump(e.children, indent + 1, out)
        else:
            out.append("%s  %s" % (head, e.value.hex()))
    return out
```

2.100 /opt/GSM/osmo-operator/navigation/diagram.py

7798 octets, 208 lignes → 208 lignes

```
# diagram.py - the lab diagram, drawn in characters, and navigable.
#
# Every network element becomes a BOX placed on a character grid. The arrow
# keys do not follow a list: they move in space, to the nearest box in the
# requested direction, so the diagram is browsed like a map.
#
# The boxes are built from the discovered topology: nothing is drawn that
# does not exist in the lab.

BOX_W = 20

# Display order of the services: core first, radio next, the rest after.
# Anything not listed keeps its discovery order.
FAMILY_ORDER = {"SS7": 0, "coeur": 1, "abonnes": 2, "radio": 3, "media": 4,
                "data": 5, "voix": 6, "mobile": 7, "autre": 8}


class Box(object):
    __slots__ = ("key", "x", "y", "w", "h", "kind", "title", "lines", "ref",
                 "node", "color")

    def __init__(self, key, x, y, w, h, kind, title, lines, ref=None,
```

```
                 node=None, color=0):
        self.key = key
        self.x = x
        self.y = y
        self.w = w
        self.h = h
        self.kind = kind          # hub | stp | service | console | note
        self.title = title
        self.lines = lines
        self.ref = ref            # Service, Node, or None
        self.node = node          # Node the box belongs to
        self.color = color

    @property
    def cx(self):
        return self.x + self.w // 2

    @property
    def cy(self):
        return self.y + self.h // 2

    def contains(self, x, y):
        return self.x <= x < self.x + self.w and self.y <= y < self.y + self.h


class Canvas(object):
    def __init__(self, width, height):
        self.w = width
        self.h = height
        self.grid = [[" "] * width for _ in range(height)]

    def put(self, x, y, text):
        if y < 0 or y >= self.h:
            return
        for i, ch in enumerate(text):
            if 0 <= x + i < self.w:
                self.grid[y][x + i] = ch

    def hline(self, x1, x2, y, ch="-"):
        for x in range(min(x1, x2), max(x1, x2) + 1):
            if 0 <= x < self.w and 0 <= y < self.h:
                cur = self.grid[y][x]
                self.grid[y][x] = "+" if cur in "|+" else ch

    def vline(self, y1, y2, x, ch="|"):
        for y in range(min(y1, y2), max(y1, y2) + 1):
            if 0 <= x < self.w and 0 <= y < self.h:
                cur = self.grid[y][x]
                self.grid[y][x] = "+" if cur in "-+" else ch

    def box(self, b):
        top = "+" + "-" * (b.w - 2) + "+"
        bot = "+" + "-" * (b.w - 2) + "+"
        self.put(b.x, b.y, top)
        rows = [b.title] + b.lines
        for i in range(b.h - 2):
            text = rows[i] if i < len(rows) else ""
            self.put(b.x, b.y + 1 + i, "|" + text.ljust(b.w - 2)[:b.w - 2] + "|")
        self.put(b.x, b.y + b.h - 1, bot)

    def render(self):
        return ["".join(row).rstrip() for row in self.grid]


def _service_sort(service):
    return (FAMILY_ORDER.get(service.family, 9), service.port)
```

```
def build(topo, extra=None):
    """Return (lines, boxes). extra: state dict of the SS7 console."""
    extra = extra or {}
    ops = topo.operators
    per_op_cols = 2
    svc_rows = max([1] + [(len(o.services) + per_op_cols - 1) // per_op_cols
                          for o in ops])

    col_w = per_op_cols * (BOX_W + 2) + 2
    width = max(80, len(ops) * col_w + 8)
    height = 12 + svc_rows * 4 + 8
    cv = Canvas(width, height)
    boxes = []

    # ── banner ──────────────────────────────────────────────────────
    cv.put(2, 0, "RESEAU SS7 - %s" % ("docker" if topo.docker else "natif"))

    # ── hub inter-STP ───────────────────────────────────────────────
    hub = topo.hub
    hub_x = max(2, width // 2 - BOX_W // 2)
    hub_y = 2
    hub_lines = []
    if hub:
        hub_lines.append("PC %s" % (hub.point_code or "?"))
        hub_lines.append("%s:%d" % (topo.hub_ip or "?", topo.hub_port))
        hub_lines.append("M3UA/SCTP" if not topo.hub_local else "local")
    else:
        hub_lines = ["aucun hub", "declare", ""]
    hb = Box("hub", hub_x, hub_y, BOX_W + 6, 6, "hub", " INTER-STP (hub) ",
             hub_lines, ref=hub, node=hub)
    cv.box(hb)
    boxes.append(hb)

    # ── SS7 console (us) ────────────────────────────────────────────
    con_x = max(2, hub_x - BOX_W - 10)
    con_lines = [
        "PC %s" % extra.get("local_pc", topo.free_point_code()),
        "SSN %s" % extra.get("local_ssn", 8),
        extra.get("state", "hors ligne"),
    ]
    cb = Box("console", con_x, hub_y, BOX_W + 4, 6, "console", " CONSOLE SS7 ",
             con_lines)
    cv.box(cb)
    boxes.append(cb)
    cv.hline(cb.x + cb.w, hb.x - 1, hub_y + 2)
    cv.put(cb.x + cb.w + 1, hub_y + 1, "M3UA")

    # ── bus to the operators ────────────────────────────────────────
    bus_y = hub_y + 7
    cv.vline(hub_y + 6, bus_y, hb.cx)
    if ops:
        first_x = 4
        xs = []
        for i, _ in enumerate(ops):
            xs.append(first_x + i * col_w + (col_w - BOX_W) // 2)
        cv.hline(min(xs) + BOX_W // 2, max(xs) + BOX_W // 2, bus_y)
        cv.put(hb.cx - 4, bus_y - 1, "")

        for i, node in enumerate(ops):
            x = xs[i]
            stp_y = bus_y + 2
            cv.vline(bus_y, stp_y, x + BOX_W // 2)
            lines = ["PC %s" % (node.point_code or "?"),
                     "M3UA %s" % node.local_m3ua,
```

```
                    "noeud WAN %s" % (node.wan_node or "-")]
            sb = Box("stp:%s" % node.name, x, stp_y, BOX_W + 4, 6, "stp",
                     " %s - STP " % node.title, lines, ref=node, node=node)
            cv.box(sb)
            boxes.append(sb)

            svc_y = stp_y + 7
            cv.vline(stp_y + 6, svc_y - 1, x + BOX_W // 2)
            services = sorted(node.services, key=_service_sort)
            for j, svc in enumerate(services):
                row, col = divmod(j, per_op_cols)
                bx = x - 2 + col * (BOX_W + 2)
                by = svc_y + row * 4
                lines = ["VTY %d  %s" % (svc.port, svc.family)]
                box = Box("svc:%s:%d" % (node.name, svc.port), bx, by,
                          BOX_W, 4, "service", " %s" % svc.label, lines,
                          ref=svc, node=node)
                cv.box(box)
                boxes.append(box)
    else:
        cv.put(4, bus_y + 2, "aucun operateur en cours d'execution")

    lines = cv.render()
    while lines and not lines[-1].strip():
        lines.pop()
    return lines, boxes


def navigate(boxes, current, direction):
    """Nearest box in a direction. dx/dy in {-1,0,1}."""
    if not boxes:
        return current
    cur = boxes[current]
    dx, dy = direction
    best = None
    best_score = None
    for i, b in enumerate(boxes):
        if i == current:
            continue
        vx = b.cx - cur.cx
        vy = b.cy - cur.cy
        if dx and (vx * dx) <= 0:
            continue
        if dy and (vy * dy) <= 0:
            continue
        # Distance along the requested axis, penalised by the lateral offset:
        # what is "straight ahead" wins over what is far off to the side.
        along = abs(vx) if dx else abs(vy)
        across = abs(vy) if dx else abs(vx)
        score = along + across * 2.5
        if best_score is None or score < best_score:
            best_score = score
            best = i
    return best if best is not None else current
```

2.101 /opt/GSM/osmo-operator/navigation/m3ua.py

18813 octets, 463 lignes → 463 lignes

```
# m3ua.py - an M3UA ASP (RFC 4666) in Python, over SCTP, without dependency.
#
# The console becomes a real SS7 point of the lab: it connects to the
# inter-STP (or to an operator's STP), announces itself (ASPUP), dynamically
# registers a routing key for ITS point code (RKM - the repository hub is
```

```
# configured with "routing-key-allocation dynamic-permitted", see
# helpers/create_interop.sh), goes active (ASPAC), then sends and receives SCCP.
#
# The link is SCTP: socket(AF_INET, SOCK_STREAM, IPPROTO_SCTP). PPID 3 (M3UA)
# is set through sendmsg() - an STP filtering on the PPID would otherwise
# drop our messages.

import queue
import socket
import struct
import threading
import time

IPPROTO_SCTP = 132
SCTP_SNDRCV = 1          # cmsg type (linux/sctp.h): 0 = INIT, 1 = SNDRCV
PPID_M3UA = 3

# Offset of the console routing contexts: outside the lab numbering plan
# (n*1000 + o*100 + 50, so at most 9950 for 9 nodes and 9 operators).
CONSOLE_RCTX_BASE = 900000

# Message classes and types.
CLS_MGMT, CLS_TRANSFER, CLS_SSNM, CLS_ASPSM, CLS_ASPTM, CLS_RKM = 0, 1, 2, 3, 4, 9
ERR, NTFY = 0, 1
DATA = 1
DUNA, DAVA, DAUD, SCON, DUPU, DRST = 1, 2, 3, 4, 5, 6
ASPUP, ASPDN, BEAT, ASPUP_ACK, ASPDN_ACK, BEAT_ACK = 1, 2, 3, 4, 5, 6
ASPAC, ASPIA, ASPAC_ACK, ASPIA_ACK = 1, 2, 3, 4
REG_REQ, REG_RSP, DEREG_REQ, DEREG_RSP = 1, 2, 3, 4

# Parameters.
P_INFO = 0x0004
P_ROUTING_CTX = 0x0006
P_DIAG = 0x0007
P_HEARTBEAT = 0x0009
P_TRAFFIC_MODE = 0x000B
P_ERROR_CODE = 0x000C
P_STATUS = 0x000D
P_ASP_ID = 0x0011
P_AFFECTED_PC = 0x0012
P_NET_APPEARANCE = 0x0200
P_ROUTING_KEY = 0x0207
P_REG_RESULT = 0x0208
P_LOCAL_RK_ID = 0x020A
P_DPC = 0x020B
P_SERVICE_IND = 0x020C
P_OPC_LIST = 0x020E
P_PROTOCOL_DATA = 0x0210
P_REG_STATUS = 0x0212

MSG_NAMES = {
    (CLS_MGMT, ERR): "ERR", (CLS_MGMT, NTFY): "NTFY",
    (CLS_TRANSFER, DATA): "DATA",
    (CLS_SSNM, DUNA): "DUNA", (CLS_SSNM, DAVA): "DAVA", (CLS_SSNM, DAUD): "DAUD",
    (CLS_SSNM, SCON): "SCON", (CLS_SSNM, DUPU): "DUPU", (CLS_SSNM, DRST): "DRST",
    (CLS_ASPSM, ASPUP): "ASPUP", (CLS_ASPSM, ASPUP_ACK): "ASPUP_ACK",
    (CLS_ASPSM, ASPDN): "ASPDN", (CLS_ASPSM, ASPDN_ACK): "ASPDN_ACK",
    (CLS_ASPSM, BEAT): "BEAT", (CLS_ASPSM, BEAT_ACK): "BEAT_ACK",
    (CLS_ASPTM, ASPAC): "ASPAC", (CLS_ASPTM, ASPAC_ACK): "ASPAC_ACK",
    (CLS_ASPTM, ASPIA): "ASPIA", (CLS_ASPTM, ASPIA_ACK): "ASPIA_ACK",
    (CLS_RKM, REG_REQ): "REG_REQ", (CLS_RKM, REG_RSP): "REG_RSP",
    (CLS_RKM, DEREG_REQ): "DEREG_REQ", (CLS_RKM, DEREG_RSP): "DEREG_RSP",
}

# RFC 4666 3.6.2 - Registration Status.
REG_STATUS = {
```

```
    0: "enregistree", 1: "erreur inconnue", 2: "DPC invalide",
    3: "network appearance invalide", 4: "routing key invalide",
    5: "permission refusee", 6: "routage unique impossible",
    7: "routing key non provisionnee", 8: "ressources insuffisantes",
    9: "parametre de routing key non supporte",
    10: "mode de trafic non supporte", 11: "changement de RK refuse",
    12: "routing key deja enregistree",
}

ERROR_CODES = {
    0x01: "Invalid Version", 0x03: "Unsupported Message Class",
    0x04: "Unsupported Message Type", 0x05: "Unsupported Traffic Mode",
    0x06: "Unexpected Message", 0x07: "Protocol Error",
    0x09: "Invalid Stream Identifier", 0x0D: "Refused - Management Blocking",
    0x0E: "ASP Identifier Required", 0x0F: "Invalid ASP Identifier",
    0x11: "Invalid Parameter Value", 0x12: "Parameter Field Error",
    0x13: "Unexpected Parameter", 0x14: "Destination Status Unknown",
    0x15: "Invalid Network Appearance", 0x16: "Missing Parameter",
    0x19: "Invalid Routing Context", 0x1A: "No Configured AS for ASP",
}


def param(tag, value):
    body = struct.pack("!HH", tag, 4 + len(value)) + value
    pad = (-len(body)) % 4
    return body + b"\x00" * pad


def parse_params(raw):
    out = {}
    i = 0
    while i + 4 <= len(raw):
        tag, length = struct.unpack("!HH", raw[i:i + 4])
        if length < 4:
            break
        value = raw[i + 4:i + length]
        out.setdefault(tag, []).append(value)
        i += length + ((-length) % 4)
    return out


def message(cls, typ, params_bytes=b""):
    return struct.pack("!BBBBI", 1, 0, cls, typ, 8 + len(params_bytes)) + params_bytes


class M3uaMessage(object):
    def __init__(self, cls, typ, params):
        self.cls = cls
        self.typ = typ
        self.params = params

    @property
    def name(self):
        return MSG_NAMES.get((self.cls, self.typ), "cls%d/typ%d" % (self.cls, self.typ))

    def first(self, tag):
        vals = self.params.get(tag)
        return vals[0] if vals else None

    def __str__(self):
        return "M3UA %s (%d parametre(s))" % (self.name, len(self.params))


class M3uaError(Exception):
    pass
```

```
class Asp(object):
    """M3UA ASP client. One reader thread, one queue per usage."""

    def __init__(self, remote_ip, remote_port=2908, local_pc=None,
                 routing_ctx=None, traffic_mode=1, log=None):
        # Observed lifetime of an association from the host to the lab
        # inter-STP: about thirty seconds. Past that, the hub stops answering
        # and the SCTP association expires in the kernel. It is brought up
        # again instead of pretending to hold a permanent link (see renew()).
        self.opened_at = 0.0
        self.remote = (remote_ip, int(remote_port))
        self.local_pc = local_pc
        self.routing_ctx = routing_ctx
        self.traffic_mode = traffic_mode
        self.sock = None
        self.rx = queue.Queue()          # received DATA (SCCP)
        self.ctrl = queue.Queue()        # control messages
        self.events = []                 # readable log
        self.reader = None
        self.beater = None
        self.beat_interval = 15.0
        self.running = False
        self.state = "deconnecte"
        self._log = log

    # -- log ------------------------------------------------------------
    def log(self, text):
        stamp = time.strftime("%H:%M:%S")
        self.events.append("%s %s" % (stamp, text))
        del self.events[:-400]
        if self._log:
            try:
                self._log(text)
            except Exception:
                pass

    # -- transport ------------------------------------------------------
    def connect(self, timeout=6.0):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM, IPPROTO_SCTP)
        self.sock.settimeout(timeout)
        self.sock.connect(self.remote)
        self.sock.settimeout(1.0)
        self.running = True
        self.opened_at = time.time()
        self.state = "connecte"
        self.reader = threading.Thread(target=self._read_loop, daemon=True)
        self.reader.start()
        # M3UA heartbeat. The inter-STP closes the association of a silent ASP
        # after about thirty seconds: without these BEATs, a perfectly
        # established link drops by itself between two commands, and the
        # destination goes DUNA with nothing in the logs.
        self.beater = threading.Thread(target=self._beat_loop, daemon=True)
        self.beater.start()
        self.log("SCTP etabli vers %s:%d" % self.remote)

    def _send(self, data, stream=0):
        if not self.sock:
            raise M3uaError("pas de lien SCTP")
        info = struct.pack("HHHIIIIIi", stream, 0, 0, socket.htonl(PPID_M3UA),
                           0, 0, 0, 0, 0)
        try:
            self.sock.sendmsg([data], [(IPPROTO_SCTP, SCTP_SNDRCV, info)])
        except (OSError, ValueError):
            self.sock.sendall(data)
```

```
def send_msg(self, cls, typ, params_bytes=b""):
    raw = message(cls, typ, params_bytes)
    # RFC 4666 1.4.4: SCTP stream 0 is RESERVED for management. DATA
    # traffic sent on it is refused with "Invalid Stream Identifier" -
    # the link stays up, and the messages vanish silently.
    self._send(raw, stream=(1 if cls == CLS_TRANSFER else 0))
    name = MSG_NAMES.get((cls, typ), "cls%d/typ%d" % (cls, typ))
    if name not in ("DATA", "BEAT"):
        self.log("-> %s" % name)
    return raw

def _beat_loop(self):
    while self.running:
        for _ in range(int(self.beat_interval * 4)):
            if not self.running:
                return
            time.sleep(0.25)
        if not self.running:
            return
        try:
            self.send_msg(CLS_ASPSM, BEAT)
        except Exception:
            return

def _read_loop(self):
    buf = b""
    while self.running:
        try:
            data = self.sock.recv(65536)
        except socket.timeout:
            continue
        except OSError:
            break
        if not data:
            break
        buf += data
        while len(buf) >= 8:
            ver, res, cls, typ = buf[0], buf[1], buf[2], buf[3]
            length = struct.unpack("!I", buf[4:8])[0]
            if length < 8 or len(buf) < length:
                break
            body = buf[8:length]
            buf = buf[length:]
            msg = M3uaMessage(cls, typ, parse_params(body))
            self._dispatch(msg)
    self.running = False
    self.state = "deconnecte"
    self.log("lien SCTP ferme")

def _dispatch(self, msg):
    if msg.cls == CLS_TRANSFER and msg.typ == DATA:
        pd = msg.first(P_PROTOCOL_DATA)
        if pd and len(pd) >= 12:
            opc, dpc = struct.unpack("!II", pd[0:8])
            si, ni, mp, sls = pd[8], pd[9], pd[10], pd[11]
            self.rx.put({"opc": opc, "dpc": dpc, "si": si, "ni": ni,
                         "sls": sls, "data": pd[12:]})
            self.log("<- DATA opc=%s si=%d (%d octets)" %
                     (_pc(opc), si, len(pd) - 12))
        return
    if msg.cls == CLS_ASPSM and msg.typ == BEAT:
        self.send_msg(CLS_ASPSM, BEAT_ACK, param(P_HEARTBEAT,
                                                 msg.first(P_HEARTBEAT) or b""))
        return
    if msg.cls == CLS_ASPSM and msg.typ == BEAT_ACK:
        # Answer to OUR heartbeat: nothing to do with it, and it must not
```

```
        # pile up in the control queue.
        return
    name = msg.name
    detail = ""
    if msg.cls == CLS_MGMT and msg.typ == ERR:
        code = msg.first(P_ERROR_CODE)
        if code:
            val = struct.unpack("!I", code)[0]
            detail = " : %s" % ERROR_CODES.get(val, "code 0x%02x" % val)
    if msg.cls == CLS_SSNM:
        apc = msg.first(P_AFFECTED_PC)
        if apc and len(apc) >= 4:
            detail = " : PC %s" % _pc(struct.unpack("!I", apc[:4])[0])
    self.log("<- %s%s" % (name, detail))
    self.ctrl.put(msg)

def wait_ctrl(self, wanted, timeout=5.0):
    """Wait for a given (class, type). The others are kept."""
    end = time.time() + timeout
    keep = []
    found = None
    while time.time() < end:
        try:
            msg = self.ctrl.get(timeout=0.2)
        except queue.Empty:
            continue
        if (msg.cls, msg.typ) in wanted:
            found = msg
            break
        keep.append(msg)
    for m in keep:
        self.ctrl.put(m)
    return found

# ── activation sequence ──────────────────────────────────────
def asp_up(self, timeout=5.0):
    self.send_msg(CLS_ASPSM, ASPUP)
    msg = self.wait_ctrl({(CLS_ASPSM, ASPUP_ACK), (CLS_MGMT, ERR)}, timeout)
    if msg is None:
        raise M3uaError("pas d'ASPUP_ACK - le STP ne repond pas")
    if msg.cls == CLS_MGMT:
        raise M3uaError("ASPUP refuse par le STP")
    self.state = "ASP_INACTIVE"
    return True

def register(self, point_code, service_ind=None, rk_id=None, timeout=5.0):
    """RKM: dynamic request of a routing key for OUR point code.

    The "Service Indicators" parameter is OMITTED by default: the Osmocom
    inter-STP refuses it (registration status 9, "unsupported routing key
    parameter") and the whole registration then fails, while the same key
    without this field is accepted."""
    # The local routing key identifier MUST be specific to this key: the
    # inter-STP uses it to recognise a re-registration. Two ASPs both
    # sending identifier 1 end up in THE SAME AS (same routing context),
    # and in traffic-mode override the last one steals the traffic of the
    # first - whose destination becomes DUNA. The identifier is therefore
    # derived from the requested point code.
    if rk_id is None:
        rk_id = point_code & 0x00FFFFFF
    # The routing context is also PROPOSED, derived from the point code.
    # Without it, the inter-STP allocates a context itself and returns the
    # SAME one to two different ASPs: the second then takes the place of
    # the first in the AS (traffic-mode override) and the first point code
    # goes DUNA. With a context specific to each point code, each keeps
    # its traffic. The proposed context lives in a range the lab does not
```

```
    # use: the nodes compute theirs as n*1000 + o*100 + 50 (at most 9950),
    # and the hub does not tell apart two AS carrying the same context,
    # hence a large offset added to the point code rather than risking a
    # collision with an operator's routing key.
    rctx = self.routing_ctx if self.routing_ctx is not None else \
        (CONSOLE_RCTX_BASE + (point_code & 0x00FFFFFF))
    dpc = struct.pack("!I", point_code & 0x00FFFFFF)
    rk = param(P_LOCAL_RK_ID, struct.pack("!I", rk_id))
    rk += param(P_ROUTING_CTX, struct.pack("!I", rctx))
    rk += param(P_TRAFFIC_MODE, struct.pack("!I", self.traffic_mode))
    rk += param(P_DPC, dpc)
    if service_ind is not None:
        rk += param(P_SERVICE_IND, bytes([service_ind]))
    self.send_msg(CLS_RKM, REG_REQ, param(P_ROUTING_KEY, rk))
    msg = self.wait_ctrl({(CLS_RKM, REG_RSP), (CLS_MGMT, ERR)}, timeout)
    if msg is None:
        raise M3uaError("pas de REG_RSP - RKM refuse ou non supporte")
    if msg.cls == CLS_MGMT:
        raise M3uaError("REG_REQ rejete (ERR)")
    result = msg.first(P_REG_RESULT)
    status = None
    if result:
        sub = parse_params(result)
        st = sub.get(P_REG_STATUS)
        if st:
            status = struct.unpack("!I", st[0])[0]
        rc = sub.get(P_ROUTING_CTX)
        if rc:
            self.routing_ctx = struct.unpack("!I", rc[0][:4])[0]
    if status not in (0, None):
        raise M3uaError("enregistrement refuse : %s" %
                        REG_STATUS.get(status, status))
    self.log("routing key enregistree : PC %s -> contexte %s (rk id %s)"
             % (_pc(point_code), self.routing_ctx, rk_id))
    return self.routing_ctx

def activate(self, timeout=5.0):
    body = param(P_TRAFFIC_MODE, struct.pack("!I", self.traffic_mode))
    if self.routing_ctx is not None:
        body += param(P_ROUTING_CTX, struct.pack("!I", self.routing_ctx))
    self.send_msg(CLS_ASPTM, ASPAC, body)
    msg = self.wait_ctrl({(CLS_ASPTM, ASPAC_ACK), (CLS_MGMT, ERR)}, timeout)
    if msg is None:
        raise M3uaError("pas d'ASPAC_ACK")
    if msg.cls == CLS_MGMT:
        raise M3uaError("ASPAC refuse")
    self.state = "ASP_ACTIVE"
    return True

@property
def age(self):
    return time.time() - self.opened_at if self.opened_at else 0.0

def bring_up(self, point_code, register=True):
    self.last_pc = point_code
    self.last_register = register
    self.connect()
    self.asp_up()
    if register:
        try:
            self.register(point_code)
        except M3uaError as exc:
            # An STP already provisioned for this PC refuses the RKM: carry
            # on, the activation may well succeed without it.
            self.log("RKM : %s (on continue sans)" % exc)
    self.activate()
```

```
        return self.state

    def renew(self, max_age=3.0):
        """Reopen the link if it is dead, or close to the end of its lifetime.

        The threshold is short (3 s): that is how long the lab inter-STP
        actually answers an association coming from the host. A reconnection
        costs about 0.3 s (SCTP + ASPUP + RKM + ASPAC).

        Return True if a reconnection took place. The point code and the
        routing context are the same: from the outside, nothing moves."""
        if self.state == "ASP_ACTIVE" and self.age < max_age:
            return False
        pc = getattr(self, "last_pc", None)
        if pc is None:
            return False
        try:
            self.close()
        except Exception:
            pass
        self.routing_ctx = None
        self.rx = queue.Queue()
        self.ctrl = queue.Queue()
        self.bring_up(pc, getattr(self, "last_register", True))
        self.log("lien renouvele (PC %s)" % _pc(pc))
        return True

    # -- data ---------------------------------------------------------------
    def send_data(self, opc, dpc, payload, si=3, ni=0, sls=0, mp=0):
        pd = struct.pack("!IIBBBB", opc & 0xFFFFFF, dpc & 0xFFFFFF, si, ni, mp, sls)
        pd += payload
        body = b""
        if self.routing_ctx is not None:
            body += param(P_ROUTING_CTX, struct.pack("!I", self.routing_ctx))
        body += param(P_PROTOCOL_DATA, pd)
        self.send_msg(CLS_TRANSFER, DATA, body)
        self.log("-> DATA vers %s (%d octets SCCP)" % (_pc(dpc), len(payload)))

    def recv_data(self, timeout=5.0):
        try:
            return self.rx.get(timeout=timeout)
        except queue.Empty:
            return None

    def close(self):
        self.running = False
        if self.sock:
            try:
                self.send_msg(CLS_ASPTM, ASPIA)
                self.send_msg(CLS_ASPSM, ASPDN)
            except Exception:
                pass
            try:
                self.sock.close()
            except Exception:
                pass
        self.sock = None
        self.state = "deconnecte"


def _pc(val):
    return "%d.%d.%d" % ((val >> 11) & 0x07, (val >> 3) & 0xFF, val & 0x07)
```

2.102 /opt/GSM/osmo-operator/navigation/mapops.py

12144 octets, 302 lignes → 302 lignes

```
# mapops.py - catalogue of the MAP operations the console can send.
#
# Each operation declares: its opcode, its application context, the SSN of
# the machine it addresses (HLR 6, VLR 7, MSC 8, gsmSCF 147), the list of its
# parameters - the list the console form displays - and how to read the
# answer.
#
# Encodings follow 3GPP TS 29.002 (MAP v3).

from . import ber
from . import sccp

# -- elementary types ---------------------------------------------------

def address_string(number, nai=1, npi=1):
    """ISDN-AddressString: one nature/plan octet, then the digits in TBCD.
    nai 1 = international, npi 1 = E.164."""
    digits = "".join(c for c in str(number) if c.isdigit())
    head = 0x80 | ((nai & 0x07) << 4) | (npi & 0x0F)
    return bytes([head]) + sccp.tbcd(digits)


def read_address_string(raw):
    if not raw:
        return ""
    nai = (raw[0] >> 4) & 0x07
    num = sccp.untbcd(raw[1:])
    prefix = "+" if nai == 1 else ""
    return prefix + num


def imsi(value):
    return sccp.tbcd(value)


def read_imsi(raw):
    return sccp.untbcd(raw)


# -- MAP application contexts (0.4.0.0.1.0.<context>.<version>) ---------
AC = {
    "networkLocUp": "0.4.0.0.1.0.1.3",
    "roamingNumberEnquiry": "0.4.0.0.1.0.3.3",
    "locationInfoRetrieval": "0.4.0.0.1.0.5.3",
    "infoRetrieval": "0.4.0.0.1.0.14.3",
    "shortMsgGateway": "0.4.0.0.1.0.20.3",
    "shortMsgMT-Relay": "0.4.0.0.1.0.25.3",
    "imsiRetrieval": "0.4.0.0.1.0.26.3",
    "subscriberInfoEnquiry": "0.4.0.0.1.0.28.3",
    "anyTimeInfoEnquiry": "0.4.0.0.1.0.29.3",
}

# -- MAP errors ---------------------------------------------------------
MAP_ERRORS = {
    1: "unknownSubscriber", 3: "unknownMSC", 5: "unidentifiedSubscriber",
    6: "absentSubscriberSM", 7: "unknownEquipment", 8: "roamingNotAllowed",
    9: "illegalSubscriber", 10: "bearerServiceNotProvisioned",
    11: "teleserviceNotProvisioned", 12: "illegalEquipment", 13: "callBarred",
    21: "facilityNotSupported", 27: "absentSubscriber",
    31: "subscriberBusyForMT-SMS", 32: "sm-DeliveryFailure",
    33: "messageWaitingListFull", 34: "systemFailure", 35: "dataMissing",
    36: "unexpectedDataValue", 44: "numberChanged", 45: "busySubscriber",
```

```
    46: "noSubscriberReply", 49: "forwardingViolation",
    51: "resourceLimitation", 52: "noRoamingNumberAvailable",
    53: "subscriberBusyForMT-SMS", 71: "unknownAlphabet",
    74: "absentSubscriberSM",
}

SSN_HLR, SSN_VLR, SSN_MSC, SSN_SCF = 6, 7, 8, 147


class Param(object):
    """One form field."""

    def __init__(self, key, label, default="", kind="text", help=""):
        self.key = key
        self.label = label
        self.default = default
        self.kind = kind          # text | number | bool | choice
        self.help = help


class Operation(object):
    def __init__(self, key, name, opcode, ac, ssn, params, encoder,
                 decoder=None, summary=""):
        self.key = key
        self.name = name
        self.opcode = opcode
        self.ac = ac
        self.ssn = ssn
        self.params = params
        self.encoder = encoder
        self.decoder = decoder or (lambda elem: [])
        self.summary = summary

    def encode(self, values):
        return self.encoder(values)

    def defaults(self):
        return dict((p.key, p.default) for p in self.params)


# ── encoders ─────────────────────────────────────────────────────────

def _enc_sri_sm(v):
    body = ber.ctx(0, address_string(v["msisdn"]), constructed=False)
    body += ber.tlv(0x81, b"\xff" if str(v.get("sm_rp_pri", "1")) in
                    ("1", "true", "oui", "True") else b"\x00")
    body += ber.ctx(2, address_string(v.get("sc_addr") or "0"), constructed=False)
    return ber.tlv(0x30, body)


def _enc_sri(v):
    body = ber.ctx(0, address_string(v["msisdn"]), constructed=False)
    itype = {"basicCall": 0, "forwarding": 1}.get(v.get("interrogation", "basicCall"), 0)
    body += ber.tlv(0x83, bytes([itype]))
    body += ber.ctx(6, address_string(v.get("gmsc") or "0"), constructed=False)
    return ber.tlv(0x30, body)


def _enc_ati(v):
    ident = v.get("identity_type", "msisdn")
    if ident == "imsi":
        sub = ber.ctx(0, imsi(v["identity"]), constructed=False)
    else:
        sub = ber.ctx(1, address_string(v["identity"]), constructed=False)
    subscriber = ber.ctx(0, sub)
    req = b""
```

```
    if str(v.get("location", "1")) in ("1", "true", "oui"):
        req += ber.ctx(0, b"", constructed=False)          # locationInformation
    if str(v.get("state", "1")) in ("1", "true", "oui"):
        req += ber.ctx(1, b"", constructed=False)          # subscriberState
    requested = ber.ctx(1, req)
    scf = ber.ctx(3, address_string(v.get("scf") or "0"), constructed=False)
    return ber.tlv(0x30, subscriber + requested + scf)


def _enc_sai(v):
    body = ber.ctx(0, imsi(v["imsi"]), constructed=False)
    body += ber.ctx(1, bytes([int(v.get("vectors", 1) or 1)]), constructed=False)
    return ber.tlv(0x30, body)


def _enc_send_imsi(v):
    # The argument is directly an ISDN-AddressString.
    return ber.tlv(0x04, address_string(v["msisdn"]))


def _enc_psi(v):
    body = ber.ctx(0, imsi(v["imsi"]), constructed=False)
    req = b""
    if str(v.get("location", "1")) in ("1", "true", "oui"):
        req += ber.ctx(0, b"", constructed=False)
    if str(v.get("state", "1")) in ("1", "true", "oui"):
        req += ber.ctx(1, b"", constructed=False)
    body += ber.ctx(2, req)
    return ber.tlv(0x30, body)


def _enc_ul(v):
    body = ber.tlv(0x04, imsi(v["imsi"]))
    body += ber.ctx(1, address_string(v.get("msc") or "0"), constructed=False)
    body += ber.tlv(0x04, address_string(v.get("vlr") or "0"))
    return ber.tlv(0x30, body)


def _enc_raw(v):
    txt = (v.get("hex") or "").replace(" ", "")
    return bytes.fromhex(txt) if txt else b""


# ── result readers ──────────────────────────────────────────────────

def _walk_pairs(elem, out, path=""):
    """Opportunistic extraction: anything that looks like an IMSI or a
    number is made readable, with the tag path that produced it."""
    for child in (elem.children if elem else []):
        tag = "%s/%s" % (path, child.tagstr())
        if child.constructed:
            _walk_pairs(child, out, tag)
        else:
            raw = child.value
            if not raw:
                out.append((tag, "(vide)"))
                continue
            txt = raw.hex()
            digits = sccp.untbcd(raw)
            if len(raw) >= 3 and raw[0] & 0x80 and digits[1:].isdigit():
                out.append((tag, "numero %s" % read_address_string(raw)))
            elif digits.isdigit() and len(digits) >= 5:
                out.append((tag, "chiffres %s (hex %s)" % (digits, txt)))
            else:
                out.append((tag, txt))
    return out
```

```
def _dec_sri_sm(elem):
    res = []
    if elem is None:
        return res
    kids = elem.children
    if kids and not kids[0].constructed:
        res.append(("IMSI", read_imsi(kids[0].value)))
    for e in (elem.walk() if elem else []):
        if e.cls == 2 and e.num == 1 and not e.constructed and e.value:
            res.append(("Noeud de service (networkNode-Number)",
                        read_address_string(e.value)))
    if not res:
        res = _walk_pairs(elem, [])
    return res


def _dec_sri(elem):
    res = []
    for e in (elem.walk() if elem else []):
        if e.cls == 2 and e.num == 5 and not e.constructed and e.value:
            res.append(("MSRN (roamingNumber)", read_address_string(e.value)))
        if e.cls == 0 and e.num == 4 and len(e.value) > 3:
            res.append(("IMSI", read_imsi(e.value)))
    if not res:
        res = _walk_pairs(elem, [])
    return res


def _dec_generic(elem):
    return _walk_pairs(elem, [])


# ── catalogue ──────────────────────────────────────────────────────────

OPERATIONS = [
    Operation(
        "sri-sm", "sendRoutingInfoForSM", 45, AC["shortMsgGateway"], SSN_HLR,
        [Param("msisdn", "MSISDN de l'abonne", "", "text",
               "numero appele, en international sans +"),
         Param("sc_addr", "Adresse du SMSC (GT)", "", "text",
               "serviceCentreAddress - l'adresse qui recevra la reponse"),
         Param("sm_rp_pri", "Priorite SM-RP-PRI", "1", "bool", "")],
        _enc_sri_sm, _dec_sri_sm,
        "Demande au HLR ou joindre l'abonne pour lui remettre un SMS "
        "(IMSI + numero du MSC/VLR de service)."),
    Operation(
        "sri", "sendRoutingInfo", 22, AC["locationInfoRetrieval"], SSN_HLR,
        [Param("msisdn", "MSISDN appele", "", "text", ""),
         Param("gmsc", "Adresse du GMSC (GT)", "", "text", ""),
         Param("interrogation", "Type d'interrogation", "basicCall", "choice",
               "basicCall | forwarding")],
        _enc_sri, _dec_sri,
        "Interrogation d'acheminement d'appel : le HLR rend le MSRN, "
        "ou une erreur (absentSubscriber, unknownSubscriber...)."),
    Operation(
        "ati", "anyTimeInterrogation", 71, AC["anyTimeInfoEnquiry"], SSN_HLR,
        [Param("identity", "IMSI ou MSISDN", "", "text", ""),
         Param("identity_type", "Type d'identite", "msisdn", "choice",
               "msisdn | imsi"),
         Param("scf", "Adresse du gsmSCF (GT)", "", "text", ""),
         Param("location", "Demander la localisation", "1", "bool", ""),
         Param("state", "Demander l'etat de l'abonne", "1", "bool", "")],
        _enc_ati, _dec_generic,
        "Localisation a la demande : cellule, VLR, etat de l'abonne. "
```

```
        "Un HLR reel la refuse souvent (facilityNotSupported)."),
    Operation(
        "sai", "sendAuthenticationInfo", 56, AC["infoRetrieval"], SSN_HLR,
        [Param("imsi", "IMSI", "", "text", ""),
         Param("vectors", "Nombre de vecteurs", "1", "number", "")],
        _enc_sai, _dec_generic,
        "Demande de triplets/quintuplets d'authentification au HLR/AuC."),
    Operation(
        "send-imsi", "sendIMSI", 58, AC["imsiRetrieval"], SSN_HLR,
        [Param("msisdn", "MSISDN", "", "text", "")],
        _enc_send_imsi, _dec_generic,
        "Resolution MSISDN -> IMSI par le HLR."),
    Operation(
        "psi", "provideSubscriberInfo", 70, AC["subscriberInfoEnquiry"], SSN_VLR,
        [Param("imsi", "IMSI", "", "text", ""),
         Param("location", "Localisation", "1", "bool", ""),
         Param("state", "Etat de l'abonne", "1", "bool", "")],
        _enc_psi, _dec_generic,
        "Demande au VLR l'etat et la position d'un abonne."),
    Operation(
        "ul", "updateLocation", 2, AC["networkLocUp"], SSN_HLR,
        [Param("imsi", "IMSI", "", "text", ""),
         Param("msc", "Numero du MSC (GT)", "", "text", ""),
         Param("vlr", "Numero du VLR (GT)", "", "text", "")],
        _enc_ul, _dec_generic,
        "Mise a jour de localisation - la meme que celle d'un VLR reel. "
        "A n'emettre que sur un lab."),
    Operation(
        "raw", "invoke brut", 0, "", SSN_HLR,
        [Param("opcode", "Opcode MAP", "45", "number", ""),
         Param("ac", "Contexte applicatif (OID)", AC["shortMsgGateway"], "text", ""),
         Param("hex", "Parametre BER en hexa", "", "text",
               "colle tel quel apres l'opcode")],
        _enc_raw, _dec_generic,
        "Pour tout ce qui n'est pas au catalogue : opcode et parametre a la main."),
]

BY_KEY = dict((o.key, o) for o in OPERATIONS)


def error_name(code):
    return MAP_ERRORS.get(code, "erreur MAP %s" % code)
```

2.103 /opt/GSM/osmo-operator/navigation/quickcmd.py

2304 octets, 72 lignes → 72 lignes

```
# quickcmd.py - the everyday VTY commands, per daemon.
# They feed the console's "quick commands" menu: pick one with the arrow keys,
# the output is shown, and the diagram stays on screen.

QUICK = {
    "OsmoSTP": [
        ("AS (application servers)", "show cs7 instance 0 as all"),
        ("ASP (points d'acces)", "show cs7 instance 0 asp"),
        ("Table de routage", "show cs7 instance 0 route"),
        ("Utilisateurs SCCP", "show cs7 instance 0 users"),
        ("Connexions SCCP", "show cs7 instance 0 sccp connections"),
        ("SSN locaux", "show cs7 instance 0 sccp ssn"),
    ],
    "OsmoMSC": [
        ("Abonnes en VLR", "show subscriber cache"),
        ("Connexions actives", "show connection"),
        ("Transactions", "show transaction"),
```

```
        ("Liens BSC", "show bsc"),
        ("Statistiques", "show statistics"),
    ],
    "OsmoHLR": [
        ("Connexions GSUP (VLR/SMSC)", "show gsup-connections"),
        ("Statistiques", "show stats"),
    ],
    "OsmoBSC": [
        ("Etat des BTS", "show bts"),
        ("Canaux logiques", "show lchan"),
        ("Canaux occupes", "show lchan summary"),
        ("Liens vers le MSC", "show msc connection"),
        ("Timeslots", "show timeslot"),
    ],
    "OsmoBTS": [
        ("Etat du BTS", "show bts 0"),
        ("Canaux logiques", "show lchan summary"),
        ("Liens PHY", "show phy"),
    ],
    "OsmoMGW": [
        ("Points d'extremite", "show mgcp"),
        ("Statistiques", "show mgcp stats"),
    ],
    "OsmoSGSN": [
        ("Abonnes GPRS", "show subscriber cache"),
        ("Contextes PDP", "show pdp-context all"),
        ("Liens BSSGP", "show ns"),
    ],
    "OsmoGGSN": [
        ("Contextes PDP", "show pdp-context all"),
        ("APN", "show apn"),
    ],
    "OsmoPCU": [
        ("Etat TBF", "show tbf all"),
        ("BTS PCU", "show bts"),
    ],
    "OsmoSIPConnector": [
        ("Appels en cours", "show calls"),
        ("Statistiques", "show stats"),
    ],
    "OsmoMobile": [
        ("Etat du mobile", "show ms"),
        ("Reseaux vus", "show subscriber"),
    ],
}

DEFAULT = [
    ("Version", "show version"),
    ("Journal en cours", "show logging vty"),
    ("Configuration en vigueur", "show running-config"),
]


def for_daemon(daemon):
    return QUICK.get(daemon, []) + DEFAULT
```

2.104 /opt/GSM/osmo-operator/navigation/responder.py

11330 octets, 269 lignes → 269 lignes

```
# responder.py - the console's MAP responder.
#
# Why: in this lab, Osmocom makes MSC and HLR talk GSUP, not MAP. A
# sendRoutingInfoForSM sent to the real HLR therefore finds nobody listening,
```

```
# and SS7 looks silent even though it routes perfectly.
#
# The responder fills that gap: it attaches to the same inter-STP, with its
# own point code and SSN 6 (HLR), and answers MAP interrogations with data
# taken from the REAL HLR through its VTY. The path itself is real: the
# message crosses the SS7 hub like any traffic between operators.
#
#   Terminal A:  ./navigation/ss7-console.py serve --pc 1.11.61
#   Terminal B:  ./navigation/ss7-console.py map sri-sm --pc 1.11.61 \
#                    msisdn=100101 sc_addr=600999

import threading
import time

from . import ber
from . import m3ua
from . import mapops
from . import sccp
from . import tcap
from . import vty as vty_mod

OP_SRI_SM = 45
OP_SRI = 22
OP_SEND_IMSI = 58
OP_SAI = 56
OP_ATI = 71


class Subscriber(object):
    def __init__(self, msisdn, imsi, imei="", nam=""):
        self.msisdn = msisdn
        self.imsi = imsi
        self.imei = imei
        self.nam = nam


class MapResponder(object):
    def __init__(self, topo, local_pc, local_ssn=mapops.SSN_HLR, node=None,
                 hlr_port=4258, msc_number=None, log=None):
        self.topo = topo
        self.local_pc = local_pc
        self.local_ssn = local_ssn
        self.node = node or (topo.operators[0].name if topo.operators else None)
        self.hlr_port = hlr_port
        self.msc_number = msc_number
        self.pool = vty_mod.VtyPool(docker=topo.docker)
        self.asp = None
        self.running = False
        self.thread = None
        self.events = []
        self._log = log
        self._cache = ([], 0.0)

    def log(self, text):
        line = "%s %s" % (time.strftime("%H:%M:%S"), text)
        self.events.append(line)
        del self.events[:-400]
        if self._log:
            self._log(line)

    # -- subscriber data, read from the real HLR ----------------------
    def subscribers(self, max_age=10.0):
        rows, stamp = self._cache
        if rows and (time.time() - stamp) < max_age:
            return rows
        rows = []
```

```
    if self.node:
        try:
            out = self.pool.cmd(self.node, self.hlr_port, "show subscribers all")
        except vty_mod.VtyError as exc:
            self.log("HLR injoignable : %s" % exc)
            out = ""
        for line in out.split("\n"):
            parts = line.split()
            if len(parts) >= 3 and parts[0].isdigit() and parts[2].isdigit():
                rows.append(Subscriber(parts[1], parts[2],
                                       parts[3] if len(parts) > 3 else "",
                                       parts[4] if len(parts) > 4 else ""))
    self._cache = (rows, time.time())
    return rows

def find(self, msisdn=None, imsi=None):
    for sub in self.subscribers():
        if msisdn and sub.msisdn == str(msisdn).lstrip("+"):
            return sub
        if imsi and sub.imsi == str(imsi):
            return sub
    return None

# ── link ──────────────────────────────────────────────────────────
def start(self):
    self.asp = m3ua.Asp(self.topo.hub_ip, self.topo.hub_port, log=self.log)
    self.asp.bring_up(sccp.pc_to_int(self.local_pc))
    self.running = True
    self.thread = threading.Thread(target=self._loop, daemon=True)
    self.thread.start()
    # The hub only serves an association coming from the host for a few
    # seconds: without this renewal the responder would be unreachable
    # after about ten seconds, with nothing to show for it.
    self.keeper = threading.Thread(target=self._keep_alive, daemon=True)
    self.keeper.start()
    self.log("repondeur MAP actif : PC %s, SSN %d, %d abonne(s) connus"
             % (self.local_pc, self.local_ssn, len(self.subscribers())))
    return self.asp.state

def stop(self):
    self.running = False
    if self.asp:
        self.asp.close()
    self.pool.close_all()

def _keep_alive(self):
    while self.running:
        time.sleep(1.0)
        if not self.running:
            return
        try:
            self.asp.renew(max_age=2.5)
        except Exception as exc:
            self.log("renouvellement impossible : %s" % exc)
            time.sleep(2.0)

def _loop(self):
    while self.running:
        data = self.asp.recv_data(timeout=0.5)
        if data is None:
            continue
        try:
            self._handle(data)
        except Exception as exc:
            self.log("erreur de traitement : %s: %s" % (type(exc).__name__, exc))
```

```
# ── processing ──────────────────────────────────────────────────
def _handle(self, data):
    dec = sccp.decode(data["data"])
    if dec is None or dec.kind not in ("UDT", "XUDT"):
        return
    if dec.called.ssn not in (None, self.local_ssn):
        self.log("recu pour SSN %s, ignore (je suis SSN %d)"
                 % (dec.called.ssn, self.local_ssn))
        return
    msg = tcap.parse(dec.data)
    if msg is None or not msg.components:
        return
    comp = msg.components[0]
    if comp.kind != "invoke":
        return
    self.log("<- %s opcode %s de %s" % (msg.kind, comp.opcode, dec.calling))
    answer = self._answer(comp, msg.app_context)
    if answer is None:
        return
    payload = tcap.end_response(msg.otid, answer, msg.app_context or None)
    called = sccp.Address(pc=dec.calling.pc, ssn=dec.calling.ssn,
                          gt=dec.calling.gt,
                          route_on_ssn=dec.calling.gt is None)
    calling = sccp.Address(pc=self.local_pc, ssn=self.local_ssn)
    udt = sccp.udt(called, calling, payload)
    self.asp.send_data(sccp.pc_to_int(self.local_pc),
                       data["opc"], udt)
    self.log("-> reponse envoyee vers PC %s" % m3ua._pc(data["opc"]))

@staticmethod
def _ctx(elem, num):
    """Child carrying context tag [num], shallow first, then deep."""
    if elem is None:
        return None
    for child in elem.children:
        if child.cls == 2 and child.num == num:
            return child
    for child in elem.walk():
        if child.cls == 2 and child.num == num:
            return child
    return None

def _param_values(self, comp):
    """Numbers and IMSI found in the invoke parameter, IN SEQUENCE ORDER:
    the first field of an SRI-SM is the MSISDN, the third the SMSC
    address - mixing them up would answer about the wrong subscriber."""
    found = {"numbers": [], "digits": []}
    if comp.parameter is None:
        return found
    stack = [comp.parameter]
    while stack:
        elem = stack.pop(0)
        if elem.constructed:
            stack = list(elem.children) + stack
            continue
        raw = elem.value
        if not raw:
            continue
        if raw[0] & 0x80 and len(raw) >= 2:
            found["numbers"].append(mapops.read_address_string(raw))
        digits = sccp.untbcd(raw)
        if digits.isdigit():
            found["digits"].append(digits)
    return found

def _answer(self, comp, app_context):
```

```
op = comp.opcode
vals = self._param_values(comp)
wanted = (vals["numbers"] + vals["digits"] + [""])[0].lstrip("+")
# Named field when the structure allows it: SRI-SM and SRI carry the
# MSISDN in [0], the ATI its subscriber identity in [0]/[0-1].
if op in (OP_SRI_SM, OP_SRI):
    champ = self._ctx(comp.parameter, 0)
    if champ is not None and champ.value:
        wanted = mapops.read_address_string(champ.value).lstrip("+")
elif op == OP_ATI:
    ident = self._ctx(comp.parameter, 0)
    if ident is not None:
        imsi_champ = self._ctx(ident, 0)
        msisdn_champ = self._ctx(ident, 1)
        if msisdn_champ is not None and msisdn_champ.value:
            wanted = mapops.read_address_string(msisdn_champ.value).lstrip("+")
        elif imsi_champ is not None and imsi_champ.value:
            wanted = mapops.read_imsi(imsi_champ.value)

if op == OP_SRI_SM:
    sub = self.find(msisdn=wanted)
    if sub is None:
        self.log("SRI-SM %s : abonne inconnu" % wanted)
        return tcap.return_error(comp.invoke_id, 1)      # unknownSubscriber
    node_number = self.msc_number or sub.msisdn
    info = ber.ctx(1, mapops.address_string(node_number), constructed=False)
    body = ber.tlv(0x04, mapops.imsi(sub.imsi)) + ber.ctx(0, info)
    self.log("SRI-SM %s -> IMSI %s" % (wanted, sub.imsi))
    return tcap.return_result_last(comp.invoke_id, op, ber.tlv(0x30, body))

if op == OP_SRI:
    sub = self.find(msisdn=wanted)
    if sub is None:
        return tcap.return_error(comp.invoke_id, 1)
    msrn = self.msc_number or ("99" + sub.msisdn)
    body = ber.ctx(9, mapops.imsi(sub.imsi), constructed=False)
    body += ber.tlv(0x04, mapops.address_string(msrn))
    self.log("SRI %s -> MSRN %s" % (wanted, msrn))
    return tcap.return_result_last(comp.invoke_id, op, ber.tlv(0x30, body))

if op == OP_SEND_IMSI:
    sub = self.find(msisdn=wanted)
    if sub is None:
        return tcap.return_error(comp.invoke_id, 1)
    self.log("sendIMSI %s -> %s" % (wanted, sub.imsi))
    return tcap.return_result_last(comp.invoke_id, op,
                                   ber.tlv(0x04, mapops.imsi(sub.imsi)))

if op == OP_ATI:
    sub = self.find(msisdn=wanted) or self.find(imsi=wanted)
    if sub is None:
        return tcap.return_error(comp.invoke_id, 1)
    # locationInformation [0] { vlr-number [1] } - the useful minimum.
    loc = ber.ctx(1, mapops.address_string(self.msc_number or sub.msisdn),
                  constructed=False)
    body = ber.ctx(0, ber.ctx(0, loc))
    self.log("ATI %s -> localisation" % wanted)
    return tcap.return_result_last(comp.invoke_id, op, ber.tlv(0x30, body))

if op == OP_SAI:
    # No AuC here: say so plainly rather than inventing vectors that
    # would authenticate nowhere.
    self.log("sendAuthenticationInfo : refuse (pas d'AuC dans ce repondeur)")
    return tcap.return_error(comp.invoke_id, 34)          # systemFailure

self.log("opcode %s non gere -> facilityNotSupported" % op)
```

```
        return tcap.return_error(comp.invoke_id, 21)
```

2.105 /opt/GSM/osmo-operator/navigation/sccp.py

7493 octets, 222 lignes → 222 lignes

```
# sccp.py - connectionless SCCP (UDT / UDTS / XUDT), just enough to carry
# TCAP and to UNDERSTAND a refusal.
#
# The "return cause" of a UDTS is often the only answer a lab network gives:
# "no translation for this address", "unequipped user"... Decoding it
# properly avoids mistaking it for silence.

MSG_UDT = 0x09
MSG_UDTS = 0x0A
MSG_XUDT = 0x11
MSG_XUDTS = 0x12

RETURN_CAUSE = {
    0x00: "no translation for an address of such nature",
    0x01: "no translation for this specific address",
    0x02: "subsystem congestion",
    0x03: "subsystem failure",
    0x04: "unequipped user (SSN absent)",
    0x05: "MTP failure",
    0x06: "network congestion",
    0x07: "unqualified",
    0x08: "error in message transport",
    0x09: "error in local processing",
    0x0A: "destination cannot perform reassembly",
    0x0B: "SCCP failure",
    0x0C: "hop counter violation",
    0x0D: "segmentation not supported",
    0x0E: "segmentation failure",
}

# Usual SSNs (Q.713 + Osmocom usage).
SSN = {
    0: "inconnu", 1: "SCCP mgmt", 6: "HLR", 7: "VLR", 8: "MSC", 9: "EIR",
    10: "AUC", 142: "RANAP", 143: "RNSAP", 145: "GMLC", 146: "CAP",
    147: "gsmSCF", 148: "SIWF", 149: "SGSN", 150: "GGSN", 254: "BSSAP",
}

NAI = {0: "inconnu", 1: "abonne", 2: "reserve national", 3: "national",
       4: "international"}


def tbcd(digits):
    """Digits -> TBCD (swapped nibbles, F padding)."""
    digits = "".join(c for c in str(digits) if c in "0123456789*#abc")
    out = bytearray()
    for i in range(0, len(digits), 2):
        lo = int(digits[i], 16)
        hi = int(digits[i + 1], 16) if i + 1 < len(digits) else 0x0F
        out.append((hi << 4) | lo)
    return bytes(out)


def untbcd(raw):
    out = []
    for b in raw:
        for nib in (b & 0x0F, (b >> 4) & 0x0F):
            if nib == 0x0F:
                continue
```

```
            out.append("%x" % nib)
        return "".join(out)


    def pc_to_int(pc):
        """'1.11.2' (ITU 3-8-3 format) or integer -> 14-bit integer."""
        if isinstance(pc, int):
            return pc
        pc = str(pc).strip()
        if pc.isdigit():
            return int(pc)
        a, b, c = (int(x) for x in pc.split("."))
        return (a << 11) | (b << 3) | c


    def int_to_pc(val):
        return "%d.%d.%d" % ((val >> 11) & 0x07, (val >> 3) & 0xFF, val & 0x07)


    class Address(object):
        """SCCP address: point code, SSN, and/or global title (GTI 4)."""

        def __init__(self, pc=None, ssn=None, gt=None, tt=0, np=1, es=2, nai=4,
                     route_on_ssn=None):
            self.pc = pc
            self.ssn = ssn
            self.gt = gt
            self.tt = tt
            self.np = np          # 1 = ISDN/telephony E.164
            self.es = es          # 2 = BCD, even count (adjusted at encoding)
            self.nai = nai        # 4 = international
            if route_on_ssn is None:
                route_on_ssn = gt is None
            self.route_on_ssn = route_on_ssn

        def encode(self):
            ai = 0
            body = b""
            if self.pc is not None:
                ai |= 0x01
                val = pc_to_int(self.pc)
                body += bytes([val & 0xFF, (val >> 8) & 0x3F])
            if self.ssn is not None:
                ai |= 0x02
                body += bytes([self.ssn & 0xFF])
            if self.gt:
                ai |= (4 << 2)                      # GTI = 4
                digits = "".join(c for c in str(self.gt) if c.isdigit())
                es = 1 if len(digits) % 2 else 2
                body += bytes([self.tt & 0xFF,
                               ((self.np & 0x0F) << 4) | (es & 0x0F),
                               self.nai & 0x7F]) + tbcd(digits)
            if self.route_on_ssn:
                ai |= 0x40
            return bytes([ai]) + body

        @classmethod
        def decode(cls, raw):
            if not raw:
                return cls()
            ai = raw[0]
            i = 1
            pc = ssn = gt = None
            tt = np = es = nai = 0
            if ai & 0x01:
                pc = int_to_pc(raw[i] | ((raw[i + 1] & 0x3F) << 8))
```

```
            i += 2
        if ai & 0x02:
            ssn = raw[i]
            i += 1
        gti = (ai >> 2) & 0x0F
        if gti == 4 and len(raw) > i + 2:
            tt = raw[i]
            np = (raw[i + 1] >> 4) & 0x0F
            es = raw[i + 1] & 0x0F
            nai = raw[i + 2] & 0x7F
            gt = untbcd(raw[i + 3:])
        elif gti == 1 and len(raw) > i:
            nai = raw[i] & 0x7F
            gt = untbcd(raw[i + 1:])
        elif gti == 2 and len(raw) > i:
            tt = raw[i]
            gt = untbcd(raw[i + 1:])
        addr = cls(pc=pc, ssn=ssn, gt=gt, tt=tt, np=np, es=es, nai=nai,
                   route_on_ssn=bool(ai & 0x40))
        return addr

    def __str__(self):
        bits = []
        if self.pc is not None:
            bits.append("PC %s" % self.pc)
        if self.ssn is not None:
            bits.append("SSN %d (%s)" % (self.ssn, SSN.get(self.ssn, "?")))
        if self.gt:
            bits.append("GT %s (nai %s)" % (self.gt, NAI.get(self.nai, self.nai)))
        bits.append("route sur %s" % ("SSN/PC" if self.route_on_ssn else "GT"))
        return ", ".join(bits)


def udt(called, calling, data, pclass=0):
    """UDT: 3 relative pointers, then the three variable-length fields."""
    c1 = called.encode()
    c2 = calling.encode()
    f1 = bytes([len(c1)]) + c1
    f2 = bytes([len(c2)]) + c2
    f3 = bytes([len(data)]) + data
    p1 = 3
    p2 = p1 + len(f1) - 1
    p3 = p2 + len(f2) - 1
    return bytes([MSG_UDT, pclass & 0xFF, p1, p2, p3]) + f1 + f2 + f3


class Decoded(object):
    def __init__(self, kind, called=None, calling=None, data=b"", cause=None):
        self.kind = kind
        self.called = called
        self.calling = calling
        self.data = data
        self.cause = cause

    @property
    def cause_text(self):
        if self.cause is None:
            return ""
        return RETURN_CAUSE.get(self.cause, "cause 0x%02x" % self.cause)

    def __str__(self):
        if self.kind == "UDTS":
            return "UDTS - refus SCCP : %s" % self.cause_text
        return "%s vers %s, de %s, %d octets utiles" % (
            self.kind, self.called, self.calling, len(self.data))
```

```
def decode(raw):
    if not raw:
        return None
    mtype = raw[0]
    if mtype == MSG_UDT:
        p1, p2, p3 = raw[2], raw[3], raw[4]
        o1, o2, o3 = 2 + p1, 3 + p2, 4 + p3
        called = Address.decode(raw[o1 + 1:o1 + 1 + raw[o1]])
        calling = Address.decode(raw[o2 + 1:o2 + 1 + raw[o2]])
        data = raw[o3 + 1:o3 + 1 + raw[o3]]
        return Decoded("UDT", called, calling, data)
    if mtype == MSG_UDTS:
        cause = raw[1]
        p1, p2, p3 = raw[2], raw[3], raw[4]
        o1, o2, o3 = 2 + p1, 3 + p2, 4 + p3
        called = Address.decode(raw[o1 + 1:o1 + 1 + raw[o1]])
        calling = Address.decode(raw[o2 + 1:o2 + 1 + raw[o2]])
        data = raw[o3 + 1:o3 + 1 + raw[o3]]
        return Decoded("UDTS", called, calling, data, cause)
    if mtype in (MSG_XUDT, MSG_XUDTS):
        # XUDT: class, hop counter, then 4 pointers. Only the useful part is read.
        off = 3
        p1, p2, p3 = raw[off], raw[off + 1], raw[off + 2]
        o1, o2, o3 = off + p1, off + 1 + p2, off + 2 + p3
        called = Address.decode(raw[o1 + 1:o1 + 1 + raw[o1]])
        calling = Address.decode(raw[o2 + 1:o2 + 1 + raw[o2]])
        data = raw[o3 + 1:o3 + 1 + raw[o3]]
        return Decoded("XUDT" if mtype == MSG_XUDT else "XUDTS",
                       called, calling, data,
                       raw[1] if mtype == MSG_XUDTS else None)
    return Decoded("SCCP 0x%02x" % mtype, data=raw[1:])
```

2.106 /opt/GSM/osmo-operator/navigation/ss7-console.py

9584 octets, 254 lignes → 254 lignes

```
#!/usr/bin/env python3
# ss7-console.py - entry point of the osmo-operator SS7 console.
#
#   ./navigation/ss7-console.py                the navigable diagram (arrows)
#   ./navigation/ss7-console.py --list         the inventory, as text
#   ./navigation/ss7-console.py audit 1.11.1   SS7 audit of a point code
#   ./navigation/ss7-console.py vty 1 4258 "show gsup-connections"
#   ./navigation/ss7-console.py map sri-sm --pc 1.11.1 msisdn=112233445566
#
# Text mode is for scripts and machines without a terminal; diagram mode is
# the everyday one.

import argparse
import os
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
REPO = os.path.dirname(HERE)
if REPO not in sys.path:
    sys.path.insert(0, REPO)

from navigation import mapops, responder as responder_mod, ss7   # noqa: E402
from navigation.ss7 import Ss7Error                               # noqa: E402
from navigation import topo as topo_mod, vty as vty_mod           # noqa: E402
```

```
def cmd_list(topo, args):
    print("Mode          : %s" % ("docker" if topo.docker else "natif"))
    print("Hub inter-STP : %s:%s (%s)" % (
        topo.hub_ip or "-", topo.hub_port,
        "local" if topo.hub_local else "distant"))
    print("PC libre      : %s" % topo.free_point_code())
    for node in topo.operators:
        print("")
        print("%s (%s)" % (node.title, node.name))
        print("  point code  : %s   M3UA local %s   noeud WAN %s"
              % (node.point_code, node.local_m3ua, node.wan_node or "-"))
        print("  hub vu d'ici: %s:%s  routing key %s"
              % (node.hub_ip, node.hub_port, node.routing_ctx))
        for name, addr in sorted(node.sccp.items()):
            if name.startswith("_self_"):
                continue
            print("  sccp %-14s PC %-8s SSN %s" % (name, addr.get("pc") or "-",
                                                   addr.get("ssn") or "-"))
        if node.services:
            print("  VTY : " + ", ".join("%s:%d" % (s.label, s.port)
                                         for s in node.services))
    return 0


def _client(topo, args):
    local = getattr(args, "from_pc", None) or getattr(args, "global_pc", None) \
        or topo.free_point_code()
    client = ss7.Ss7Client(topo.hub_ip, topo.hub_port, local,
                           log=(lambda t: print("  [m3ua] %s" % t)) if args.verbose
                           else None)
    print("Lien M3UA vers %s:%s ..." % (topo.hub_ip or "?", topo.hub_port))
    try:
        print("  etat : %s" % client.connect())
    except (Ss7Error, OSError) as exc:
        raise SystemExit("  impossible : %s" % exc)
    return client


def cmd_audit(topo, args):
    client = _client(topo, args)
    try:
        for pc in args.point_code:
            print("")
            print(client.audit(pc))
    finally:
        client.close()
    return 0


def cmd_map(topo, args):
    if args.operation not in mapops.BY_KEY:
        print("operation inconnue. Disponibles : %s"
              % ", ".join(sorted(mapops.BY_KEY)))
        return 2
    op = mapops.BY_KEY[args.operation]
    values = op.defaults()
    for item in args.params:
        if "=" not in item:
            print("parametre attendu sous la forme cle=valeur : %s" % item)
            return 2
        key, val = item.split("=", 1)
        values[key] = val
    if not args.pc and not args.gt:
        print("il faut --pc (point code) ou --gt (global title)")
        return 2
    client = _client(topo, args)
```

```
    try:
        rep = client.send_map(op, values, args.pc or "0.0.0",
                              args.ssn, args.gt or "",
                              route_on_gt=bool(args.gt and not args.pc),
                              timeout=args.timeout)
        print("")
        print(rep)
    finally:
        client.close()
    return 0 if rep.ok else 1


def cmd_vty(topo, args):
    node = args.node
    if node.isdigit():
        node = "osmo-operator-%s" % node
    pool = vty_mod.VtyPool(docker=topo.docker)
    try:
        print(pool.cmd(node, args.port, " ".join(args.command)))
    except vty_mod.VtyError as exc:
        print("VTY %s:%s injoignable : %s" % (node, args.port, exc),
              file=sys.stderr)
        return 1
    finally:
        pool.close_all()
    return 0


def cmd_serve(topo, args):
    node = args.node
    if node and node.isdigit():
        node = "osmo-operator-%s" % node
    rep = responder_mod.MapResponder(
        topo, args.pc or getattr(args, "global_pc", None) or _second_pc(topo),
        args.ssn, node=node,
        msc_number=args.msc_number, log=lambda t: print("  %s" % t, flush=True))
    print("Repondeur MAP : PC %s, SSN %s, donnees du HLR de %s"
          % (rep.local_pc, rep.local_ssn,
             node or (topo.operators[0].name if topo.operators else "-")))
    try:
        print("Etat du lien : %s" % rep.start())
    except Exception as exc:
        print("  impossible de monter le repondeur : %s" % exc, file=sys.stderr)
        return 2
    print("Ctrl-C pour arreter.")
    try:
        while True:
            time.sleep(0.5)
            if rep.asp is not None and rep.asp.state == "deconnecte" \
                    and not rep.running:
                break
    except KeyboardInterrupt:
        print("")
    finally:
        rep.stop()
    return 0


def _second_pc(topo):
    """A point code distinct from the one the sending console takes: the
    second of the free list, never an increment (the third member of an ITU
    point code fits in 3 bits, .8 does not exist)."""
    libres = topo.free_point_codes(2)
    return libres[1] if len(libres) > 1 else libres[0]
```

```
def cmd_ops(topo, args):
    for op in mapops.OPERATIONS:
        print("%-10s %-24s opcode %-3s SSN %-3s" % (op.key, op.name, op.opcode, op.ssn))
        print("           %s" % op.summary)
        print("           parametres : %s"
              % ", ".join(p.key for p in op.params))
    return 0


def main(argv=None):
    ap = argparse.ArgumentParser(
        description="Console SS7 de osmo-operator : schema navigable, VTY et MAP.")
    ap.add_argument("--pc", dest="global_pc",
                    help="point code de la console (avant la sous-commande)")
    ap.add_argument("--no-probe", action="store_true",
                    help="ne pas sonder les VTY a la decouverte (plus rapide)")
    ap.add_argument("-v", "--verbose", action="store_true",
                    help="journal du lien M3UA")
    sub = ap.add_subparsers(dest="cmd")

    sub.add_parser("list", help="inventaire du lab, en texte")
    sub.add_parser("ops", help="catalogue des operations MAP")

    p = sub.add_parser("audit", help="audit SS7 (DAUD) d'un ou plusieurs PC")
    p.add_argument("point_code", nargs="+")
    p.add_argument("--from-pc", dest="from_pc",
                   help="point code de la console (defaut : calcule)")

    p = sub.add_parser("map", help="emettre une operation MAP")
    p.add_argument("operation")
    p.add_argument("params", nargs="*", help="cle=valeur (n'importe ou)")
    p.add_argument("--from-pc", dest="from_pc",
                   help="point code de la console (defaut : calcule)")
    p.add_argument("--pc", dest="pc", help="point code destination")
    p.add_argument("--ssn", type=int, help="SSN destination")
    p.add_argument("--gt", help="global title destination")
    p.add_argument("--timeout", type=float, default=6.0)

    p = sub.add_parser("serve", help="repondeur MAP (HLR simule) sur le SS7 du lab")
    p.add_argument("--pc", dest="pc", help="point code du repondeur")
    p.add_argument("--ssn", type=int, default=mapops.SSN_HLR,
                   help="SSN ecoute (6 = HLR)")
    p.add_argument("--node", help="operateur dont le HLR fournit les donnees")
    p.add_argument("--msc-number", help="numero rendu comme noeud de service")

    p = sub.add_parser("vty", help="une commande VTY, sans interface")
    p.add_argument("node", help="numero d'operateur ou nom de conteneur")
    p.add_argument("port", type=int)
    p.add_argument("command", nargs="+")

    args, extra = ap.parse_known_args(argv)
    # "map sri-sm --pc 1.11.61 msisdn=100101": argparse sets aside the
    # positionals that follow an option. They are collected rather than lost.
    leftovers = [x for x in extra if "=" in x and not x.startswith("-")]
    unknown = [x for x in extra if x not in leftovers]
    if unknown:
        ap.error("arguments inconnus : %s" % " ".join(unknown))
    if leftovers:
        if getattr(args, "cmd", None) == "map":
            args.params = list(getattr(args, "params", [])) + leftovers
        else:
            ap.error("arguments inconnus : %s" % " ".join(leftovers))
    if not hasattr(args, "pc"):
        args.pc = None

    probe = not args.no_probe and args.cmd in (None, "list")
```

```
    topo = topo_mod.discover(probe=probe)
    if topo.errors:
        for err in topo.errors:
            print("attention : %s" % err, file=sys.stderr)

    if args.cmd == "list":
        return cmd_list(topo, args)
    if args.cmd == "ops":
        return cmd_ops(topo, args)
    if args.cmd == "audit":
        return cmd_audit(topo, args)
    if args.cmd == "map":
        return cmd_map(topo, args)
    if args.cmd == "serve":
        return cmd_serve(topo, args)
    if args.cmd == "vty":
        return cmd_vty(topo, args)

    if not sys.stdout.isatty():
        print("pas de terminal : utilisez 'list', 'audit', 'map' ou 'vty'.",
              file=sys.stderr)
        return 2
    from navigation import tui
    tui.main(topo)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

2.107 /opt/GSM/osmo-operator/navigation/ss7-diag.py

16351 octets, 415 lignes → 415 lignes

```
#!/usr/bin/env python3
# ss7-diag.py - complete SS7 diagnostic of the lab, from the host.
#
# The script locates itself: it derives the repository from its own path, and
# the topology from the containers and their configurations. No argument is
# needed.
#
#   ./navigation/ss7-diag.py               complete diagnostic
#   ./navigation/ss7-diag.py --rapide      without the M3UA/MAP tests
#   ./navigation/ss7-diag.py --log FILE    full copy into a file
#
# What is checked, in the order a failure is read:
#   1. the environment (docker, SCTP, modules);
#   2. the topology inherited from the configurations;
#   3. per operator: STP, MSC, BSC, HLR - VTY, AS, ASP, routes, SSN;
#   4. the link to the inter-STP: SCTP association, AS state;
#   5. the console as M3UA ASP: ASPUP, RKM, ASPAC, then audit of every known
#      point code;
#   6. a real MAP round trip (sendRoutingInfoForSM) through the hub.
#
# Exit code = number of failures.

import argparse
import os
import re
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
```

```
REPO = os.path.dirname(HERE)
if REPO not in sys.path:
    sys.path.insert(0, REPO)

from navigation import mapops, responder as responder_mod, sccp, ss7   # noqa: E402
from navigation import topo as topo_mod, vty as vty_mod                # noqa: E402

G, Y, R, C, B, N = ("\033[0;32m", "\033[1;33m", "\033[0;31m", "\033[0;36m",
                    "\033[1m", "\033[0m")

STATE = {"pass": 0, "warn": 0, "fail": 0, "skip": 0, "log": None}


def out(text=""):
    print(text, flush=True)
    if STATE["log"]:
        STATE["log"].write(re.sub(r"\033\[[0-9;]*m", "", text) + "\n")
        STATE["log"].flush()


def section(title):
    out("")
    out("%s%s%s" % (B, title, N))
    out("-" * 70)


def ok(text):
    STATE["pass"] += 1
    out("  %s✓%s %s" % (G, N, text))


def warn(text):
    STATE["warn"] += 1
    out("  %s!%s %s" % (Y, N, text))


def fail(text):
    STATE["fail"] += 1
    out("  %s✗%s %s" % (R, N, text))


def skip(text):
    STATE["skip"] += 1
    out("  %s-%s %s" % (Y, N, text))


def info(text):
    out("    %s" % text)


def sh(cmd):
    try:
        p = subprocess.run(cmd, shell=True, stdout=subprocess.PIPE,
                           stderr=subprocess.DEVNULL, timeout=30)
        return p.stdout.decode("utf-8", "replace")
    except Exception:
        return ""


# ── 1. environment ──────────────────────────────────────────────────
def check_env():
    section("1. ENVIRONNEMENT")
    if sh("docker ps --format '{{.Names}}'").strip():
        ok("docker repond")
    else:
        warn("docker ne rend aucun conteneur - mode natif, ou docker arrete")
```

```
    import socket
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM, 132)
        s.close()
        ok("SCTP disponible sur l'hote (IPPROTO_SCTP)")
    except Exception as exc:
        fail("SCTP indisponible : %s - le lien M3UA sera impossible" % exc)
        info("modprobe sctp")
    out("    depot   : %s" % REPO)
    out("    python  : %s" % sys.version.split()[0])


# ── 2. topology ──────────────────────────────────────────────────────
def check_topo(topo):
    section("2. TOPOLOGIE (heritee des configurations)")
    if not topo.operators:
        fail("aucun operateur trouve")
        return
    ok("%d operateur(s) : %s" % (len(topo.operators),
                                 ", ".join(n.name for n in topo.operators)))
    if topo.hub_ip:
        ok("inter-STP : %s:%s (%s)" % (topo.hub_ip, topo.hub_port,
                                       "local" if topo.hub_local else "distant"))
    else:
        fail("aucun inter-STP connu - ni conteneur local, ni OSMO_HUB_IP")
    vus = {}
    for node in topo.operators:
        if not node.point_code:
            fail("%s : aucun point code lu dans osmo-stp.cfg" % node.name)
            continue
        if node.point_code in vus:
            fail("%s et %s partagent le point code %s - collision SS7"
                 % (node.name, vus[node.point_code], node.point_code))
        else:
            vus[node.point_code] = node.name
            ok("%s : PC %s, routing key %s, noeud WAN %s"
               % (node.title, node.point_code, node.routing_ctx,
                  node.wan_node or "-"))
        for nom, addr in sorted(node.sccp.items()):
            if nom.startswith("_self_") or not addr.get("pc"):
                continue
            info("%-10s %-10s PC %-8s SSN %s" % (node.title, nom, addr["pc"],
                                                 addr.get("ssn")))
        # ITU 3-8-3 format: the third member fits in 3 bits.
        membres = node.point_code.split(".")
        if len(membres) == 3:
            a, b, c = (int(x) for x in membres)
            if a > 7 or b > 255 or c > 7:
                fail("%s : point code %s hors format ITU 3-8-3 (max 7.255.7)"
                     % (node.title, node.point_code))
    if topo.docker:
        libres = topo.free_point_codes(2)
        info("point codes libres pour la console : %s" % ", ".join(libres))


# ── 3. the daemons, per operator ─────────────────────────────────────
def check_nodes(topo, pool):
    for node in topo.operators:
        section("3. %s (%s)" % (node.title, node.name))
        stp = node.service("OsmoSTP") or type("S", (), {"port": 4239})()
        try:
            as_out = pool.cmd(node.name, stp.port, "show cs7 instance 0 as all")
        except vty_mod.VtyError as exc:
            fail("VTY STP (%s) inaccessible : %s" % (stp.port, exc))
            continue
        actifs = len(re.findall(r"AS_ACTIVE", as_out))
```

```
inter = re.search(r"as-inter\s+(\S+)", as_out)
if inter and inter.group(1) == "AS_ACTIVE":
    ok("as-inter -> hub : ACTIVE")
elif inter:
    fail("as-inter : %s" % inter.group(1))
else:
    warn("aucun as-inter declare (pas de lien vers un hub)")
if actifs >= 2:
    ok("AS actifs sur le STP local : %d" % actifs)
else:
    warn("AS actifs sur le STP local : %d" % actifs)

asp_out = pool.cmd(node.name, stp.port, "show cs7 instance 0 asp")
asp_actifs = len(re.findall(r"ASP_ACTIVE", asp_out))
(ok if asp_actifs >= 2 else fail)("ASP actifs : %d" % asp_actifs)

route_out = pool.cmd(node.name, stp.port, "show cs7 instance 0 route")
if re.search(r"0\.0\.0/0.*avail", route_out):
    ok("route par defaut vers l'inter-STP : presente et avail")
else:
    fail("route par defaut vers l'inter-STP absente ou indisponible")
# PROHIB: a real failure (a node OF THE LAB become unreachable) is
# told from a dynamic route left by a transient point code - a
# stopped SS7 console, a WAN node switched off. Mixing them up would
# raise an alarm every time the console is closed.
connus = set()
for autre in topo.operators:
    if autre.point_code:
        connus.add(autre.point_code)
    for a in autre.sccp.values():
        if a.get("pc"):
            connus.add(a["pc"])
durs, mous = [], []
for ligne in route_out.split("\n"):
    if "PROHIB" not in ligne:
        continue
    dest = ligne.split("/")[0].strip()
    (durs if dest in connus else mous).append(dest)
if durs:
    fail("routes PROHIB vers des noeuds du lab : %s" % ", ".join(durs))
elif mous:
    warn("%d route(s) PROHIB, toutes vers des point codes de passage : %s"
         % (len(mous), ", ".join(mous)))
    info("dynamiques : elles repasseront avail des que ces PC reviendront")
else:
    ok("aucune route PROHIB")

users = pool.cmd(node.name, stp.port, "show cs7 instance 0 users")
for ssn, nom in ((254, "BSSAP"), (6, "HLR"), (8, "MSC")):
    if re.search(r"\b%d\b" % ssn, users):
        info("SSN %s (%s) enregistre sur le STP" % (ssn, nom))

# Without VTY probing (--no-probe), fall back to the usual ports
# rather than reporting an "absent" MSC or HLR that runs fine.
def service(daemon, defaut):
    svc = node.service(daemon)
    if svc:
        return svc
    return type("S", (), {"port": defaut, "label": daemon})()

try:
    msc = service("OsmoMSC", 4254)
    cache = pool.cmd(node.name, msc.port, "show subscriber cache")
    n = len(re.findall(r"IMSI", cache))
    ok("MSC : VTY %s, %d abonne(s) en VLR" % (msc.port, n))
except vty_mod.VtyError as exc:
```

```
            fail("MSC : VTY injoignable (%s)" % exc)

        try:
            hlr = service("OsmoHLR", 4258)
            subs = pool.cmd(node.name, hlr.port, "show subscribers all")
            n = len(re.findall(r"^\s*\d+\s+\d+", subs, re.M))
            gsup = pool.cmd(node.name, hlr.port, "show gsup-connections")
            vlr = "VLR" in gsup
            (ok if n else warn)("HLR : %d abonne(s) provisionne(s)" % n)
            (ok if vlr else fail)("HLR : VLR %s en GSUP"
                                  % ("connecte" if vlr else "absent"))
        except vty_mod.VtyError as exc:
            fail("HLR : VTY injoignable (%s)" % exc)

        try:
            bsc = service("OsmoBSC", 4242)
            bts = pool.cmd(node.name, bsc.port, "show bts")
            up = len(re.findall(r"(?i)OML.*Link.*connected|BTS is in service", bts))
            info("BSC : VTY %s%s" % (bsc.port,
                                     ", %d BTS en service" % up if up else ""))
        except vty_mod.VtyError as exc:
            warn("BSC : VTY injoignable (%s)" % exc)


# -- 4. link to the hub, seen from the nodes ------------------------------------
def check_hub_link(topo):
    section("4. LIEN VERS L'INTER-STP (vu des noeuds)")
    if not topo.hub_ip:
        skip("pas de hub connu")
        return
    port = topo.hub_port
    for node in topo.operators:
        out_ = sh("docker exec %s ss -an --sctp 2>/dev/null" % node.name) \
            if topo.docker else sh("ss -an --sctp")
        if not out_.strip():
            skip("%s : ss --sctp indisponible" % node.title)
            continue
        motif = re.escape("%s:%d" % (topo.hub_ip, port))
        if re.search(r"ESTAB.*%s" % motif, out_):
            ok("%s : association SCTP ETABLIE vers %s:%d"
               % (node.title, topo.hub_ip, port))
        else:
            fail("%s : aucune association SCTP vers %s:%d - l'ASP est DOWN"
                 % (node.title, topo.hub_ip, port))


# -- 5. the console as ASP ------------------------------------------------------
def check_console_asp(topo):
    section("5. LA CONSOLE COMME ASP M3UA")
    if not topo.hub_ip:
        skip("pas de hub : aucun lien a monter")
        return None
    pc = topo.free_point_codes(2)[0]
    client = ss7.Ss7Client(topo.hub_ip, topo.hub_port, pc)
    try:
        etat = client.connect()
    except Exception as exc:
        fail("attachement impossible : %s" % exc)
        info("verifiez que %s:%s accepte les ASP dynamiques" % (topo.hub_ip,
                                                               topo.hub_port))
        return None
    if etat == "ASP_ACTIVE":
        ok("ASP actif sur le hub : PC %s, routing context %s"
           % (pc, client.asp.routing_ctx))
    else:
        fail("ASP dans l'etat %s" % etat)
```

```
        return client

    cibles = []
    for node in topo.operators:
        if node.point_code:
            cibles.append((node.title + " STP", node.point_code))
        for nom, addr in sorted(node.sccp.items()):
            if not nom.startswith("_self_") and addr.get("pc"):
                cibles.append(("%s %s" % (node.title, nom), addr["pc"]))
    vus = set()
    for label, cible in cibles:
        if cible in vus:
            continue
        vus.add(cible)
        rep = client.audit(cible)
        if rep.ok:
            ok("DAUD %-10s %-8s : joignable a travers le hub" % (label, cible))
        else:
            warn("DAUD %-10s %-8s : %s" % (label, cible, rep.lines[0]))
    return client


# -- 6. real MAP round trip ------------------------------------------------
def check_map(topo, client, pool):
    section("6. ALLER-RETOUR MAP (sendRoutingInfoForSM)")
    if client is None:
        skip("pas de lien M3UA")
        return
    if not topo.operators:
        skip("aucun operateur")
        return
    pc_rep = topo.free_point_codes(2)[1]
    node = topo.operators[0]
    rep = responder_mod.MapResponder(topo, pc_rep, mapops.SSN_HLR, node=node.name)
    try:
        rep.start()
    except Exception as exc:
        fail("repondeur MAP impossible a demarrer : %s" % exc)
        return
    abonnes = rep.subscribers()
    if not abonnes:
        skip("aucun abonne dans le HLR de %s : rien a interroger" % node.name)
        rep.stop()
        return
    ok("repondeur MAP en place : PC %s, SSN 6, %d abonne(s) du HLR reel"
       % (pc_rep, len(abonnes)))
    cible = abonnes[0]
    info("interrogation du MSISDN %s (IMSI attendu %s)" % (cible.msisdn, cible.imsi))
    time.sleep(0.5)
    rapport = client.send_map("sri-sm", {"msisdn": cible.msisdn,
                                         "sc_addr": cible.msisdn,
                                         "sm_rp_pri": "1"},
                              dest_pc=pc_rep, timeout=5.0)
    trouve = dict(rapport.result).get("IMSI", "")
    if trouve == cible.imsi:
        ok("MAP complet : SRI-SM -> IMSI %s (le TCAP a bien traverse le hub)"
           % trouve)
    elif rapport.ok:
        warn("reponse MAP recue, mais IMSI inattendu : %s" % trouve)
    else:
        fail("aucune reponse MAP exploitable")
        for ligne in rapport.lines[-6:]:
            info(ligne)
    rep.stop()
```

```
def main():
    ap = argparse.ArgumentParser(description="Diagnostic SS7 complet du lab.")
    ap.add_argument("--rapide", action="store_true",
                    help="sauter les tests M3UA et MAP")
    ap.add_argument("--log", help="ecrire aussi le rapport dans ce fichier")
    ap.add_argument("--no-probe", action="store_true",
                    help="ne pas sonder les ports VTY")
    args = ap.parse_args()

    if args.log:
        STATE["log"] = open(args.log, "w")

    out("")
    out("%s============================================================%s" % (C, N))
    out("%s  DIAGNOSTIC SS7 - osmo-operator - %s%s"
        % (C, time.strftime("%Y-%m-%d %H:%M:%S"), N))
    out("%s============================================================%s" % (C, N))

    check_env()
    topo = topo_mod.discover(probe=not args.no_probe)
    check_topo(topo)
    pool = vty_mod.VtyPool(docker=topo.docker)
    client = None
    try:
        check_nodes(topo, pool)
        check_hub_link(topo)
        if not args.rapide:
            client = check_console_asp(topo)
            check_map(topo, client, pool)
        else:
            section("5-6. M3UA ET MAP")
            skip("ignores (--rapide)")
    finally:
        pool.close_all()
        if client:
            client.close()

    section("RESUME")
    out("  %s%d ok%s   %s%d avertissement(s)%s   %s%d echec(s)%s   %d ignore(s)"
        % (G, STATE["pass"], N, Y, STATE["warn"], N, R, STATE["fail"], N,
           STATE["skip"]))
    if STATE["fail"] == 0 and STATE["warn"] == 0:
        out("  %sLe SS7 du lab est sain de bout en bout.%s" % (G, N))
    elif STATE["fail"] == 0:
        out("  %sPas de panne, mais des points a surveiller.%s" % (Y, N))
    else:
        out("  %sDes elements du SS7 sont en panne (voir ci-dessus).%s" % (R, N))
    out("")
    if STATE["log"]:
        STATE["log"].close()
    return STATE["fail"]


if __name__ == "__main__":
    sys.exit(main())
```

2.108 /opt/GSM/osmo-operator/navigation/ss7.py

9215 octets, 224 lignes → 224 lignes

```
# ss7.py - the layer that ties everything together: M3UA + SCCP + TCAP + MAP.
#
# This is the object the console and the scripts use: given an operation from
# the catalogue and a destination (point code, SSN, GT), it builds the UDT,
```

```
# sends it on the M3UA link and returns a readable report of what comes back
# - MAP result, MAP error, or SCCP refusal.

import os
import random
import time

from . import m3ua
from . import mapops
from . import sccp
from . import tcap


class Ss7Error(Exception):
    pass


class Report(object):
    def __init__(self):
        self.lines = []
        self.ok = False
        self.raw_tx = b""
        self.raw_rx = b""
        self.result = []

    def add(self, text):
        self.lines.append(text)

    def __str__(self):
        return "\n".join(self.lines)


class Ss7Client(object):
    def __init__(self, hub_ip, hub_port=2908, local_pc="1.11.60",
                 local_ssn=mapops.SSN_MSC, local_gt="", log=None):
        self.hub_ip = hub_ip
        self.hub_port = int(hub_port)
        self.local_pc = local_pc
        self.local_ssn = local_ssn
        self.local_gt = local_gt
        self.asp = None
        self.invoke_id = 1
        self.log_cb = log

    # -- link ----------------------------------------------------------
    @property
    def connected(self):
        return self.asp is not None and self.asp.state == "ASP_ACTIVE"

    @property
    def events(self):
        return self.asp.events if self.asp else []

    def connect(self, register=True):
        if not self.hub_ip:
            raise Ss7Error(
                "aucun inter-STP connu : ni conteneur osmo-inter-stp, ni "
                "OSMO_HUB_IP dans les noeuds. Demarrez le lab, ou donnez "
                "l'adresse du hub avec --hub-ip.")
        self.asp = m3ua.Asp(self.hub_ip, self.hub_port, log=self.log_cb)
        self.asp.bring_up(sccp.pc_to_int(self.local_pc), register=register)
        return self.asp.state

    def fresh(self):
        """A young link before every exchange.
```

```
    The lab inter-STP serves an association coming from the host for
    about thirty seconds; past that, messages go nowhere. Rather than
    showing a false "no answer", the link is brought up again - invisible
    for the caller, and the point code does not change."""
    if self.asp is None:
        self.connect()
        return True
    return self.asp.renew()

def close(self):
    if self.asp:
        self.asp.close()
    self.asp = None

# ── destination audit (the SS7 "ping") ──────────────────────────
def audit(self, point_code, timeout=3.0):
    """DAUD: the STP answers DAVA (reachable) or DUNA (unavailable)."""
    rep = Report()
    if not self.asp:
        rep.add("pas de lien M3UA")
        return rep
    self.fresh()
    val = sccp.pc_to_int(point_code)
    body = m3ua.param(m3ua.P_AFFECTED_PC, bytes([0]) + val.to_bytes(3, "big"))
    if self.asp.routing_ctx is not None:
        body += m3ua.param(m3ua.P_ROUTING_CTX,
                           self.asp.routing_ctx.to_bytes(4, "big"))
    self.asp.send_msg(m3ua.CLS_SSNM, m3ua.DAUD, body)
    msg = self.asp.wait_ctrl({(m3ua.CLS_SSNM, m3ua.DAVA),
                              (m3ua.CLS_SSNM, m3ua.DUNA),
                              (m3ua.CLS_SSNM, m3ua.SCON),
                              (m3ua.CLS_MGMT, m3ua.ERR)}, timeout)
    if msg is None:
        rep.add("DAUD %s : aucune reponse du STP" % point_code)
        return rep
    if msg.typ == m3ua.DAVA and msg.cls == m3ua.CLS_SSNM:
        rep.ok = True
        rep.add("DAVA - le point code %s est JOIGNABLE via le hub" % point_code)
    elif msg.typ == m3ua.DUNA:
        rep.add("DUNA - le point code %s est INACCESSIBLE" % point_code)
    elif msg.typ == m3ua.SCON:
        rep.add("SCON - congestion vers %s" % point_code)
    else:
        rep.add("reponse inattendue : %s" % msg.name)
    return rep

# ── sending a MAP operation ─────────────────────────────────────
def send_map(self, op, values, dest_pc, dest_ssn=None, dest_gt="",
             route_on_gt=False, timeout=6.0, essais=2):
    if isinstance(op, str):
        op = mapops.BY_KEY[op]
    rep = Report()
    if not self.asp:
        rep.add("pas de lien M3UA - connectez d'abord la console au STP")
        return rep
    self.fresh()

    ssn = int(dest_ssn if dest_ssn is not None else op.ssn)
    param_ber = op.encode(values)
    opcode = int(values.get("opcode", op.opcode) or op.opcode)
    ac = values.get("ac") or op.ac

    otid = os.urandom(4)
    comp = tcap.invoke(self.invoke_id, opcode, param_ber)
    self.invoke_id = (self.invoke_id % 127) + 1
    payload = tcap.begin(otid, comp, ac or None)
```

```
called = sccp.Address(pc=None if route_on_gt else dest_pc, ssn=ssn,
                      gt=dest_gt or None, route_on_ssn=not route_on_gt)
calling = sccp.Address(pc=self.local_pc, ssn=self.local_ssn,
                       gt=self.local_gt or None,
                       route_on_ssn=not self.local_gt)
udt = sccp.udt(called, calling, payload)
rep.raw_tx = udt

rep.add("Emission")
rep.add("  operation   : %s (opcode %d)" % (op.name, opcode))
if ac:
    rep.add("  contexte    : %s" % ac)
rep.add("  destination : %s" % called)
rep.add("  origine     : %s" % calling)
rep.add("  TCAP begin  : otid %s, %d octets" % (otid.hex(), len(payload)))

self.asp.send_data(sccp.pc_to_int(self.local_pc),
                   sccp.pc_to_int(dest_pc if not route_on_gt else dest_pc),
                   udt)

deadline = time.time() + timeout
while time.time() < deadline:
    data = self.asp.recv_data(timeout=max(0.2, deadline - time.time()))
    if data is None:
        continue
    dec = sccp.decode(data["data"])
    rep.raw_rx = data["data"]
    if dec is None:
        continue
    rep.add("")
    rep.add("Reponse recue de PC %s" % m3ua._pc(data["opc"]))
    rep.add("  SCCP : %s" % dec)
    if dec.kind.endswith("UDTS"):
        rep.add("  -> le reseau refuse : %s" % dec.cause_text)
        return rep
    msg = tcap.parse(dec.data)
    if msg is None:
        rep.add("  charge non TCAP : %s" % dec.data.hex())
        return rep
    rep.add("  TCAP : %s" % msg)
    for comp in msg.components:
        if comp.kind == "returnResultLast":
            rep.ok = True
            rep.add("  resultat (opcode %s) :" % comp.opcode)
            for label, value in op.decoder(comp.parameter):
                rep.add("     %-28s %s" % (label, value))
                rep.result.append((label, value))
        elif comp.kind == "returnError":
            rep.add("  ERREUR MAP %s : %s" %
                    (comp.error, mapops.error_name(comp.error)))
        elif comp.kind == "reject":
            rep.add("  REJECT TCAP : %s" %
                    (comp.parameter.value.hex() if comp.parameter else ""))
        else:
            rep.add("  composant %s" % comp.kind)
    if msg.kind == "abort":
        rep.add("  ABORT TCAP : %s" % msg.abort_reason)
    return rep

if essais > 1:
    # The link to the hub only lives a few seconds: a send may fall
    # while one of the two ends renews its association. A second try,
    # on a fresh link, tells "nobody listens" from "bad timing".
    rep.add("")
    rep.add("Pas de reponse - second essai sur un lien neuf ...")
```

```
            self.asp.renew(max_age=0)
            suite = self.send_map(op, values, dest_pc, dest_ssn, dest_gt,
                                  route_on_gt, timeout, essais=essais - 1)
            rep.lines.extend([""] + suite.lines)
            rep.ok = suite.ok
            rep.result = suite.result
            return rep
        rep.add("")
        rep.add("Aucune reponse au bout de %.0f s." % timeout)
        rep.add("Sur ce lab, un silence signifie en general qu'aucun demon")
        rep.add("n'ecoute ce SSN : Osmocom parle GSUP entre MSC et HLR, et")
        rep.add("reserve le SCCP a l'interface A (SSN 254).")
        return rep


def default_client(topo, local_pc=None, log=None):
    """Build a client from the discovered topology: hub, port and free point
    code all come from the lab configuration."""
    pc = local_pc or topo.free_point_code()
    gt = ""
    return Ss7Client(topo.hub_ip or "127.0.0.1", topo.hub_port, pc,
                     local_gt=gt, log=log)
```

2.109 /opt/GSM/osmo-operator/navigation/tcap.py

6605 octets, 188 lignes → 188 lignes

```
# tcap.py - TCAP (Q.773): begin / continue / end / abort, and the components
# invoke / returnResultLast / returnError / reject.
#
# The console sends a begin holding ONE invoke, and reads what comes back. The
# dialogue (AARQ/AARE) carries the MAP application context: without it a real
# HLR answers "abort", so it is always included.

from . import ber

# Application tags of the TCAP messages.
BEGIN, END, CONTINUE, ABORT = 0x62, 0x64, 0x65, 0x67
TID_ORIG, TID_DEST = 0x48, 0x49
DIALOGUE, COMPONENTS = 0x6B, 0x6C

# Components.
INVOKE, RET_RES_LAST, RET_ERROR, REJECT = 0xA1, 0xA2, 0xA3, 0xA4

DIALOGUE_AS_ID = "0.0.17.773.1.1.1"

MSG_NAME = {BEGIN: "begin", END: "end", CONTINUE: "continue", ABORT: "abort"}
COMP_NAME = {INVOKE: "invoke", RET_RES_LAST: "returnResultLast",
             RET_ERROR: "returnError", REJECT: "reject"}


def dialogue_request(app_context_oid):
    """dialoguePortion holding an AARQ (dialogue open request)."""
    aarq = ber.tlv(0x60,
                   ber.tlv(0x80, b"\x07\x80") +              # version 1
                   ber.ctx(1, ber.enc_oid(app_context_oid)))
    external = ber.tlv(0x28,
                       ber.enc_oid(DIALOGUE_AS_ID) +
                       ber.ctx(0, aarq))
    return ber.tlv(DIALOGUE, external)


def invoke(invoke_id, opcode, parameter=b""):
    body = ber.enc_int(invoke_id) + ber.enc_int(opcode)
```

```
    if parameter:
        body += parameter
    return ber.tlv(INVOKE, body)


def begin(otid, components, app_context_oid=None):
    body = ber.tlv(TID_ORIG, otid)
    if app_context_oid:
        body += dialogue_request(app_context_oid)
    body += ber.tlv(COMPONENTS, components)
    return ber.tlv(BEGIN, body)


def end(dtid, components=b"", app_context_oid=None):
    body = ber.tlv(TID_DEST, dtid)
    if app_context_oid:
        body += dialogue_request(app_context_oid)
    if components:
        body += ber.tlv(COMPONENTS, components)
    return ber.tlv(END, body)


class Component(object):
    def __init__(self, kind, invoke_id=None, opcode=None, error=None,
                 parameter=None, raw=b""):
        self.kind = kind
        self.invoke_id = invoke_id
        self.opcode = opcode
        self.error = error
        self.parameter = parameter
        self.raw = raw


class Message(object):
    def __init__(self, kind="?", otid=b"", dtid=b"", components=None,
                 app_context="", abort_reason="", raw=b""):
        self.kind = kind
        self.otid = otid
        self.dtid = dtid
        self.components = components or []
        self.app_context = app_context
        self.abort_reason = abort_reason
        self.raw = raw

    def __str__(self):
        s = "TCAP %s" % self.kind
        if self.otid:
            s += " otid=%s" % self.otid.hex()
        if self.dtid:
            s += " dtid=%s" % self.dtid.hex()
        if self.app_context:
            s += " ac=%s" % self.app_context
        for c in self.components:
            s += " | %s" % c.kind
        return s


def _oid_str(raw):
    if not raw:
        return ""
    vals = [raw[0] // 40, raw[0] % 40]
    n = 0
    for b in raw[1:]:
        n = (n << 7) | (b & 0x7F)
        if not b & 0x80:
            vals.append(n)
```

```
            n = 0
    return ".".join(str(v) for v in vals)


def parse(data):
    """Return a Message, or None if this is not TCAP."""
    top = ber.decode(data)
    if not top:
        return None
    e = top[0]
    kind = MSG_NAME.get(e.tag[0], "0x%02x" % e.tag[0])
    msg = Message(kind=kind, raw=data)
    for child in e.children:
        t = child.tag[0]
        if t == TID_ORIG:
            msg.otid = child.value
        elif t == TID_DEST:
            msg.dtid = child.value
        elif t == DIALOGUE:
            for sub in child.walk():
                if sub.cls == 0 and sub.num == 6 and len(sub.value) > 3:
                    oid = _oid_str(sub.value)
                    if oid != DIALOGUE_AS_ID:
                        msg.app_context = oid
        elif t == COMPONENTS:
            for comp in child.children:
                msg.components.append(_parse_component(comp))
        elif kind == "abort":
            msg.abort_reason = child.tagstr() + " " + child.value.hex()
    return msg


def _parse_component(comp):
    kind = COMP_NAME.get(comp.tag[0], comp.tagstr())
    invoke_id = opcode = error = None
    parameter = None
    kids = comp.children
    if kind == "invoke" and len(kids) >= 2:
        invoke_id = int.from_bytes(kids[0].value, "big", signed=True)
        opcode = int.from_bytes(kids[1].value, "big", signed=True)
        parameter = kids[2] if len(kids) > 2 else None
    elif kind == "returnResultLast" and kids:
        invoke_id = int.from_bytes(kids[0].value, "big", signed=True)
        if len(kids) > 1 and kids[1].constructed and kids[1].children:
            inner = kids[1].children
            opcode = int.from_bytes(inner[0].value, "big", signed=True)
            parameter = inner[1] if len(inner) > 1 else None
    elif kind == "returnError" and len(kids) >= 2:
        invoke_id = int.from_bytes(kids[0].value, "big", signed=True)
        error = int.from_bytes(kids[1].value, "big", signed=True)
        parameter = kids[2] if len(kids) > 2 else None
    elif kind == "reject" and kids:
        invoke_id = int.from_bytes(kids[0].value, "big", signed=True)
        parameter = kids[1] if len(kids) > 1 else None
    return Component(kind, invoke_id, opcode, error, parameter, comp.value)


def dialogue_response(app_context_oid, result=0, diagnostic=0):
    """dialoguePortion holding an AARE (dialogue open accept)."""
    aare = ber.tlv(0x61,
                   ber.tlv(0x80, b"\x07\x80") +
                   ber.ctx(1, ber.enc_oid(app_context_oid)) +
                   ber.ctx(2, ber.enc_int(result)) +
                   ber.ctx(3, ber.ctx(1, ber.enc_int(diagnostic))))
    external = ber.tlv(0x28, ber.enc_oid(DIALOGUE_AS_ID) + ber.ctx(0, aare))
    return ber.tlv(DIALOGUE, external)
```

```
def return_result_last(invoke_id, opcode, parameter=b""):
    inner = ber.enc_int(opcode) + parameter
    return ber.tlv(RET_RES_LAST, ber.enc_int(invoke_id) + ber.tlv(0x30, inner))


def return_error(invoke_id, error_code, parameter=b""):
    return ber.tlv(RET_ERROR,
                   ber.enc_int(invoke_id) + ber.enc_int(error_code) + parameter)


def end_response(dtid, components, app_context_oid=None):
    """end + AARE: the complete answer to a begin."""
    body = ber.tlv(TID_DEST, dtid)
    if app_context_oid:
        body += dialogue_response(app_context_oid)
    body += ber.tlv(COMPONENTS, components)
    return ber.tlv(END, body)
```

2.110 /opt/GSM/osmo-operator/navigation/test-console.sh

8242 octets, 205 lignes → 205 lignes

```
#!/usr/bin/env bash
# test-console.sh - runs EVERY command of the SS7 console and its tools,
# keeps one log per command, and prints a report.
#
#   ./navigation/test-console.sh              everything, including real SS7
#   ./navigation/test-console.sh --rapide     without M3UA/MAP (lab stopped)
#   ./navigation/test-console.sh --dir PATH   logs elsewhere
#
# One test = one command + one success criterion (expected pattern in the
# output, or return code). Every command has ITS OWN log file, named after
# the test number and name: what happened can be read back exactly without
# running anything again.
#
# Exit codes: 0 all passed, otherwise the number of failures.

set -u

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO="$(dirname "$HERE")"
CONSOLE="$HERE/ss7-console.py"
DIAG="$HERE/ss7-diag.py"

RAPIDE=0
LOGDIR=""
while [ $# -gt 0 ]; do
    case "$1" in
        --rapide) RAPIDE=1 ;;
        --dir)    LOGDIR="${2:-}"; shift ;;
        --dir=*)  LOGDIR="${1#*=}" ;;
        -h|--help)
            sed -n '2,14p' "${BASH_SOURCE[0]}" | sed 's/^# \{0,1\}//'
            exit 0 ;;
        *) echo "argument inconnu : $1" >&2; exit 2 ;;
    esac
    shift
done

STAMP="$(date +%Y%m%d-%H%M%S)"
LOGDIR="${LOGDIR:-$HERE/logs/$STAMP}"
mkdir -p "$LOGDIR"
```

```
RAPPORT="$LOGDIR/rapport.txt"

G='\033[0;32m'; R='\033[0;31m'; Y='\033[1;33m'; C='\033[0;36m'; B='\033[1m'; N='\033[0m'
PASS=0; FAIL=0; SKIP=0; NUM=0

dire() { echo -e "$*"; echo -e "$*" | sed 's/\x1b\[[0-9;]*m//g' >> "$RAPPORT"; }

# essai <name> <expected pattern|-> <command...>
#   pattern "-": only the return code is judged.
essai() {
    local nom="$1" motif="$2"; shift 2
    NUM=$((NUM + 1))
    local fichier
    fichier="$(printf '%02d-%s.log' "$NUM" "$nom")"
    local chemin="$LOGDIR/$fichier"
    {
        echo "# test    : $nom"
        echo "# commande : $*"
        echo "# attendu : ${motif}"
        echo "# date    : $(date '+%Y-%m-%d %H:%M:%S')"
        echo "# ----------------------------------------------------------------"
    } > "$chemin"
    local debut fin rc
    debut=$(date +%s)
    timeout 240 "$@" >> "$chemin" 2>&1
    rc=$?
    fin=$(date +%s)
    echo "# ----------------------------------------------------------------" >> "$chemin"
    echo "# code de retour : $rc   duree : $((fin - debut)) s" >> "$chemin"

    if [ "$rc" -eq 124 ]; then
        FAIL=$((FAIL + 1))
        dire "  ${R}✗${N} ${nom} — delai depasse (240 s)   ${C}${fichier}${N}"
        return
    fi
    if [ "$motif" = "-" ]; then
        if [ "$rc" -eq 0 ]; then
            PASS=$((PASS + 1)); dire "  ${G}✓${N} ${nom}   ${C}${fichier}${N}"
        else
            FAIL=$((FAIL + 1)); dire "  ${R}✗${N} ${nom} — code $rc   ${C}${fichier}${N}"
        fi
        return
    fi
    if grep -qE "$motif" "$chemin"; then
        PASS=$((PASS + 1)); dire "  ${G}✓${N} ${nom}   ${C}${fichier}${N}"
    else
        FAIL=$((FAIL + 1))
        dire "  ${R}✗${N} ${nom} — motif absent : ${motif}   ${C}${fichier}${N}"
    fi
}

ignore() { NUM=$((NUM + 1)); SKIP=$((SKIP + 1)); dire "  ${Y}-${N} $1"; }

titre() { dire ""; dire "${B}$1${N}"; dire
 ↪  "----------------------------------------------------------------------"; }

# ── is the lab up? ──────────────────────────────────────────────────
CONTENEURS="$(docker ps --format '{{.Names}}' 2>/dev/null | grep -c '^osmo-' || true)"
HUB=""
if [ "${CONTENEURS:-0}" -gt 0 ]; then
    HUB="$(docker exec "$(docker ps --format '{{.Names}}' | grep '^osmo-operator-' | head -1)" \
           printenv OSMO_HUB_IP 2>/dev/null | tr -d '\r')"
fi

dire ""
dire "${C}======================================================================${N}"
```

```
dire "${C}  TEST DE LA CONSOLE SS7 - $(date '+%Y-%m-%d %H:%M:%S')${N}"
dire "${C}=============================================================================${N}"
dire "  depot      : $REPO"
dire "  journaux   : $LOGDIR"
dire "  conteneurs : ${CONTENEURS:-0}   hub : ${HUB:-aucun}"
[ "$RAPIDE" = "1" ] && dire "  mode       : rapide (ni M3UA ni MAP)"

titre "1. LE PAQUET S'IMPORTE"
essai "import-modules" "modules ok" python3 -c "
import sys; sys.path.insert(0, '$REPO')
from navigation import ber, sccp, tcap, mapops, m3ua, ss7, topo, vty, diagram, quickcmd, responder
print('modules ok')"
essai "encodage-ber-sccp" "encodage ok" python3 -c "
import sys; sys.path.insert(0, '$REPO')
from navigation import sccp, tcap, mapops, ber
a = sccp.Address(pc='1.11.2', ssn=6)
assert sccp.Address.decode(a.encode()).pc == '1.11.2'
assert sccp.pc_to_int('1.11.2') == (1 << 11) | (11 << 3) | 2
assert sccp.untbcd(sccp.tbcd('600101')) == '600101'
u = sccp.udt(a, sccp.Address(pc='1.11.7', ssn=8), b'\x62\x03\x48\x01\x01')
d = sccp.decode(u); assert d.kind == 'UDT' and d.called.ssn == 6
m = tcap.parse(tcap.begin(b'\x01\x02\x03\x04', tcap.invoke(1, 45, b''), mapops.AC['shortMsgGateway']))
assert m.kind == 'begin' and m.components[0].opcode == 45
print('encodage ok')"
essai "aide-console" "usage: ss7-console" "$CONSOLE" --help
essai "aide-diag" "usage: ss7-diag" "$DIAG" --help

titre "2. INVENTAIRE ET CATALOGUE"
essai "console-ops" "sendRoutingInfoForSM" "$CONSOLE" ops
essai "console-list" "Hub inter-STP" "$CONSOLE" --no-probe list
essai "console-list-sonde" "Hub inter-STP" "$CONSOLE" list

titre "3. VTY (un test par demon)"
if [ "${CONTENEURS:-0}" -gt 0 ]; then
    essai "vty-stp"   "Routing|AS Name"   "$CONSOLE" vty 1 4239 "show cs7 instance 0 as all"
    essai "vty-stp-route" "Routing table" "$CONSOLE" vty 1 4239 "show cs7 instance 0 route"
    essai "vty-msc"   "IMSI|Subscriber"   "$CONSOLE" vty 1 4254 "show subscriber cache"
    essai "vty-hlr"   "IMSI|MSISDN"       "$CONSOLE" vty 1 4258 "show subscribers all"
    essai "vty-hlr-gsup" "VLR|GSUP|from"  "$CONSOLE" vty 1 4258 "show gsup-connections"
    essai "vty-bsc"   "BTS|bts"           "$CONSOLE" vty 1 4242 "show bts"
    essai "vty-inconnu-propre" "injoignable" "$CONSOLE" vty 1 4299 "show version"
else
    ignore "VTY : aucun conteneur ne tourne"
fi

titre "4. SS7 REEL (M3UA, DAUD, MAP)"
if [ "$RAPIDE" = "1" ]; then
    ignore "M3UA et MAP : mode rapide"
elif [ -z "$HUB" ]; then
    ignore "M3UA et MAP : aucun hub connu (lab arrete ?)"
else
    essai "m3ua-audit-stp" "JOIGNABLE|DAVA" "$CONSOLE" audit 1.11.2
    essai "m3ua-audit-msc" "JOIGNABLE|DAVA" "$CONSOLE" audit 1.11.1
    essai "m3ua-audit-inconnu" "INACCESSIBLE|DUNA|aucune reponse" "$CONSOLE" audit 7.255.7
    essai "map-sans-destination" "point code|global title" "$CONSOLE" map sri-sm msisdn=600101
    essai "map-operation-inconnue" "operation inconnue" "$CONSOLE" map pas-une-op --pc 1.11.2
    # Full round trip: the responder answers the sender through the hub.
    essai "map-aller-retour" "IMSI" "$DIAG" --no-probe
fi

titre "5. DIAGNOSTIC COMPLET"
essai "diag-rapide" "RESUME" "$DIAG" --rapide --no-probe --log "$LOGDIR/diag-rapide.txt"
if [ "$RAPIDE" != "1" ] && [ -n "$HUB" ]; then
    essai "diag-complet" "RESUME" "$DIAG" --log "$LOGDIR/diag-complet.txt"
else
    ignore "diagnostic complet : lab a l'arret ou mode rapide"
```

```
fi

titre "6. LE SCHEMA (interface aux fleches)"
essai "schema-rendu" "INTER-STP" python3 -c "
import sys; sys.path.insert(0, '$REPO')
from navigation import topo, diagram
t = topo.discover(probe=False)
lignes, boites = diagram.build(t)
print('\n'.join(lignes))
print('%d boites' % len(boites))
assert boites, 'aucune boite'
i = 0
for direction in ((0, 1), (1, 0), (0, -1), (-1, 0)):
    i = diagram.navigate(boites, i, direction)
print('navigation ok')"
essai "schema-navigation" "navigation ok" python3 -c "
import sys; sys.path.insert(0, '$REPO')
from navigation import topo, diagram
t = topo.discover(probe=False)
_, boites = diagram.build(t)
vus = set()
i = 0
for _ in range(40):
    for d in ((0,1),(1,0),(0,-1),(-1,0)):
        i = diagram.navigate(boites, i, d); vus.add(i)
assert len(vus) >= min(3, len(boites))
print('navigation ok')"

titre "RESUME"
dire "  ${G}${PASS} reussi(s)${N}   ${R}${FAIL} echec(s)${N}   ${Y}${SKIP} ignore(s)${N}"
dire "  journaux : ${C}${LOGDIR}${N}  (un fichier par commande)"
dire "  rapport  : ${C}${RAPPORT}${N}"
dire ""
exit "$FAIL"
```

2.111 /opt/GSM/osmo-operator/navigation/topo.py

14688 octets, 426 lignes → 426 lignes

```
# topo.py - inventory of the lab, INHERITED from the configurations.
#
# Nothing is hard-coded: point codes, addresses, ports and services are read
# where they are authoritative.
#
#   1. "docker ps"                      -> which nodes run here;
#   2. the container environment        -> OSMO_HUB_IP, WAN_NODE_ID (set by
#                                          start-docker.sh, see --hub-ip / --wan);
#   3. /etc/osmocom/*.cfg IN the node   -> point code, routing keys, M3UA
#                                          listeners, SCCP addresses, links to
#                                          the hub. This is the REAL
#                                          configuration, not the repo template;
#   4. /proc/net/tcp IN the node        -> which VTYs really listen, and each
#                                          banner says WHICH daemon answers
#                                          (OsmoMSC, OsmoHLR, ...).
#
# The templates in configs/ are only a fallback when no node runs.

import json
import os
import re
import subprocess

REPO = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
# Probe pushed into the node: lists the listening VTYs and reads their
# banner. One execution per node, all in parallel.
_PROBE = r'''
import json, socket, sys, threading

def listening():
    ports = set()
    for path in ("/proc/net/tcp", "/proc/net/tcp6"):
        try:
            with open(path) as fh:
                next(fh, None)
                for line in fh:
                    f = line.split()
                    if len(f) < 4 or f[3] != "0A":
                        continue
                    try:
                        p = int(f[1].split(":")[1], 16)
                    except Exception:
                        continue
                    ports.add(p)
        except Exception:
            pass
    return sorted(p for p in ports if 4200 <= p <= 4299)

res = {}
lock = threading.Lock()

def probe(p):
    banner = ""
    try:
        s = socket.create_connection(("127.0.0.1", p), 1.5)
        s.settimeout(1.2)
        data = b""
        try:
            while len(data) < 4096:
                d = s.recv(4096)
                if not d:
                    break
                data += d
                if b"VTY interface" in data:
                    break
        except socket.timeout:
            pass
        s.close()
        for line in data.decode("utf-8", "replace").splitlines():
            if "VTY interface" in line:
                banner = line.strip()
                break
    except Exception:
        return
    with lock:
        res[p] = banner

th = [threading.Thread(target=probe, args=(p,)) for p in listening()]
for t in th:
    t.start()
for t in th:
    t.join(4)
print(json.dumps(res))
'''

# Short name shown in the diagram, derived from the banner ("OsmoMSC" -> MSC).
_PRETTY = {
    "OsmoSTP": ("STP", "SS7"),
    "OsmoMSC": ("MSC/VLR", "coeur"),
    "OsmoHLR": ("HLR", "abonnes"),
```

```
    "OsmoBSC": ("BSC", "radio"),
    "OsmoBTS": ("BTS", "radio"),
    "OsmoMGW": ("MGW", "media"),
    "OsmoPCU": ("PCU", "data"),
    "OsmoSGSN": ("SGSN", "data"),
    "OsmoGGSN": ("GGSN", "data"),
    "OsmoUPF": ("UPF", "data"),
    "OsmoTRX": ("TRX", "radio"),
    "OsmoSIPConnector": ("SIP-CONN", "voix"),
    "OsmoMobile": ("MS", "mobile"),
    "mobile": ("MS", "mobile"),
}

# Fallback when the banner says nothing (silent VTY, daemon still starting).
_BY_PORT = {
    4238: "OsmoHNBGW", 4239: "OsmoSTP", 4240: "OsmoPCU", 4241: "OsmoBTS",
    4242: "OsmoBSC", 4243: "OsmoMGW", 4245: "OsmoSGSN", 4246: "OsmoGbProxy",
    4247: "OsmoMobile", 4248: "OsmoMobile", 4249: "OsmoSIPConnector",
    4250: "OsmoBTS", 4251: "OsmoMSC", 4254: "OsmoMSC", 4255: "OsmoMGW",
    4256: "OsmoUECUPS", 4257: "OsmoE1D", 4258: "OsmoHLR", 4259: "OsmoUPF",
    4260: "OsmoTRX", 4267: "OsmoGGSN",
}


def sh(args, timeout=20):
    try:
        out = subprocess.run(args, stdout=subprocess.PIPE,
                             stderr=subprocess.DEVNULL, timeout=timeout)
        return out.stdout.decode("utf-8", "replace")
    except Exception:
        return ""


class Service(object):
    """A daemon reachable by VTY on a node."""

    def __init__(self, node, port, daemon, label, family):
        self.node = node
        self.port = int(port)
        self.daemon = daemon
        self.label = label
        self.family = family

    def __repr__(self):
        return "<Service %s:%s %s>" % (self.node, self.port, self.label)


class Node(object):
    """A lab node: operator container, or inter-STP hub."""

    def __init__(self, name, role, index=None):
        self.name = name            # container name, or "local"
        self.role = role            # "operator" | "interstp"
        self.index = index          # operator number
        self.point_code = ""        # 1.11.2
        self.hub_ip = ""            # inter-STP address as seen by this node
        self.hub_port = 2908
        self.local_m3ua = 2905
        self.routing_ctx = None     # routing key to the hub
        self.wan_node = ""
        self.services = []          # [Service]
        self.sccp = {}              # name -> {"pc":..., "ssn":..., "ri":...}
        self.env = {}
        self.cfg = {}               # file -> text

    @property
```

```
    def title(self):
        if self.role == "interstp":
            return "INTER-STP"
        return "OP%s" % (self.index if self.index is not None else "?")

    def service(self, daemon):
        for s in self.services:
            if s.daemon == daemon:
                return s
        return None

    def __repr__(self):
        return "<Node %s pc=%s svc=%d>" % (self.name, self.point_code, len(self.services))


class Topology(object):
    def __init__(self):
        self.docker = True
        self.nodes = []
        self.hub_ip = ""
        self.hub_port = 2908
        self.hub_local = False
        self.repo = REPO
        self.errors = []

    @property
    def operators(self):
        return [n for n in self.nodes if n.role == "operator"]

    @property
    def hub(self):
        for n in self.nodes:
            if n.role == "interstp":
                return n
        return None

    def node(self, name):
        for n in self.nodes:
            if n.name == name:
                return n
        return None

    def free_point_codes(self, count=1):
        """Free point codes for the console, in the LAB FORMAT (ITU 3-8-3).

        The third member fits in 3 bits: 0 to 7. A "1.11.60" would overflow
        and be received as another point code (1.15.4) - the message would go
        to a neighbour, or nowhere. The choice is therefore made among the
        free values of the same group as the operator STPs (which use .1 to
        .3)."""
        used = set(n.point_code for n in self.nodes if n.point_code)
        for node in self.nodes:
            for addr in node.sccp.values():
                if addr.get("pc"):
                    used.add(addr["pc"])
        base = None
        for n in self.operators:
            if n.point_code:
                base = n.point_code
                break
        out = []
        heads = []
        if base and base.count(".") == 2:
            a, b, _ = base.split(".")
            heads.append((int(a), int(b)))
        heads.append((1, 11))
```

```
        for a, b in heads:
            for last in range(7, 0, -1):
                cand = "%d.%d.%d" % (a, b, last)
                if cand not in used and cand not in out:
                    out.append(cand)
                    if len(out) >= count:
                        return out
        while len(out) < count:
            out.append("1.11.7")
        return out

    def free_point_code(self):
        return self.free_point_codes(1)[0]


# ── reading the configurations in a node ─────────────────────────────

_CFG_FILES = ["osmo-stp.cfg", "osmo-msc.cfg", "osmo-bsc.cfg", "osmo-hlr.cfg",
              "osmo-sgsn.cfg", "osmo-ggsn.cfg", "osmo-mgw.cfg"]


def _read_cfgs(name, docker):
    cfg = {}
    for f in _CFG_FILES:
        path = "/etc/osmocom/" + f
        if docker:
            txt = sh(["docker", "exec", name, "cat", path], timeout=10)
        else:
            try:
                txt = open(path).read()
            except Exception:
                txt = ""
        if txt.strip():
            cfg[f] = txt
    return cfg


def _parse_stp(node):
    txt = node.cfg.get("osmo-stp.cfg", "")
    if not txt:
        return
    m = re.search(r"^\s*point-code\s+(\S+)", txt, re.M)
    if m:
        node.point_code = m.group(1)
    m = re.search(r"^\s*listen\s+m3ua\s+(\d+)", txt, re.M)
    if m:
        node.local_m3ua = int(m.group(1))
    m = re.search(r"^\s*asp\s+asp-to-inter\s+(\d+)\s+(\d+)", txt, re.M)
    if m:
        node.hub_port = int(m.group(1))
    m = re.search(r"asp\s+asp-to-inter.*?^\s*remote-ip\s+(\S+)", txt, re.M | re.S)
    if m:
        node.hub_ip = m.group(1)
    m = re.search(r"as\s+as-inter.*?^\s*routing-key\s+(\d+)\s+(\S+)", txt, re.M | re.S)
    if m:
        node.routing_ctx = int(m.group(1))


def _parse_sccp(node):
    """Declared SCCP addresses (osmo-bsc.cfg / osmo-msc.cfg): this is where
    the SSNs and point codes of the A-interface pairs come from."""
    for fname in ("osmo-bsc.cfg", "osmo-msc.cfg"):
        txt = node.cfg.get(fname, "")
        if not txt:
            continue
        for blk in re.finditer(
```

```
                r"^\s*sccp-address\s+(\S+)\s*\n((?:\s+\S.*\n)*)", txt, re.M):
            name, body = blk.group(1), blk.group(2)
            entry = {"ssn": None, "pc": None, "ri": None, "source": fname}
            m = re.search(r"^\s*point-code\s+(\S+)", body, re.M)
            if m:
                entry["pc"] = m.group(1)
            m = re.search(r"^\s*subsystem-number\s+(\d+)", body, re.M)
            if m:
                entry["ssn"] = int(m.group(1))
            m = re.search(r"^\s*routing-indicator\s+(\S+)", body, re.M)
            if m:
                entry["ri"] = m.group(1)
            node.sccp[name] = entry
        # cs7 instance N / point-code of the daemon itself
        m = re.search(r"^\s*point-code\s+(\S+)", txt, re.M)
        if m:
            node.sccp.setdefault("_self_" + fname, {"pc": m.group(1), "ssn": None,
                                                    "ri": None, "source": fname})


def _probe_vty(node, docker):
    if docker:
        raw = sh(["docker", "exec", "-i", node.name, "python3", "-c", _PROBE], timeout=30)
    else:
        raw = sh(["python3", "-c", _PROBE], timeout=30)
    try:
        found = json.loads(raw.strip().splitlines()[-1]) if raw.strip() else {}
    except Exception:
        found = {}
    for port, banner in sorted((int(p), b) for p, b in found.items()):
        daemon = ""
        m = re.search(r"Welcome to the (\S+) VTY interface", banner)
        if m:
            daemon = m.group(1)
        if not daemon:
            daemon = _BY_PORT.get(port, "VTY")
        label, family = _PRETTY.get(daemon, (daemon.replace("Osmo", ""), "autre"))
        node.services.append(Service(node.name, port, daemon, label, family))


def _env(name):
    raw = sh(["docker", "inspect", "-f", "{{json .Config.Env}}", name], timeout=10)
    env = {}
    try:
        for item in json.loads(raw or "[]"):
            if "=" in item:
                k, v = item.split("=", 1)
                env[k] = v
    except Exception:
        pass
    return env


def _native_present():
    """Does this machine carry a native Osmocom stack?"""
    if os.path.isdir("/etc/osmocom") and os.listdir("/etc/osmocom"):
        return True
    return bool(sh(["pgrep", "-f", "osmo-stp"]).strip())


def discover(docker=None, probe=True):
    topo = Topology()
    if docker is None:
        docker = bool(sh(["docker", "ps", "--format", "{{.Names}}"]).strip())
    topo.docker = docker
```

```
names = []
if docker:
    names = [n for n in sh(["docker", "ps", "--format", "{{.Names}}"]).split()
             if n.startswith("osmo-")]
if docker and not names:
    topo.errors.append("aucun conteneur osmo-* en cours d'execution")

for name in sorted(names):
    if name == "osmo-inter-stp":
        node = Node(name, "interstp")
        topo.hub_local = True
    else:
        m = re.match(r"osmo-operator-(\d+)$", name)
        if not m:
            continue
        node = Node(name, "operator", int(m.group(1)))
    node.env = _env(name)
    node.cfg = _read_cfgs(name, True)
    _parse_stp(node)
    _parse_sccp(node)
    if probe:
        _probe_vty(node, True)
    node.wan_node = node.env.get("OSMO_WAN_NODE", node.env.get("WAN_NODE_ID", ""))
    if not node.hub_ip:
        node.hub_ip = node.env.get("OSMO_HUB_IP", "")
    topo.nodes.append(node)

# Native fallback: ONLY if this machine really carries an Osmocom stack.
# Without this check, a stopped docker lab would show up as a "native
# operator" with no point code nor service - an imaginary inventory,
# more misleading than a plain "nothing here".
if not docker and not _native_present():
    topo.errors.append(
        "aucun noeud : ni conteneur osmo-* en cours, ni /etc/osmocom sur "
        "cette machine (le lab est arrete ?)")
elif not docker:
    node = Node("local", "operator", 1)
    node.cfg = _read_cfgs("local", False)
    _parse_stp(node)
    _parse_sccp(node)
    if probe:
        _probe_vty(node, False)
    # An /etc/osmocom directory may be a mere leftover of an install:
    # without a readable point code nor an answering VTY, there is no node.
    if node.point_code or node.services:
        topo.nodes.append(node)
    else:
        topo.errors.append(
            "/etc/osmocom existe mais aucun demon Osmocom ne repond ici "
            "(ni point code, ni VTY) : rien a inventorier")

for n in topo.operators:
    if n.hub_ip:
        topo.hub_ip = n.hub_ip
        topo.hub_port = n.hub_port
        break
if topo.hub_local and not topo.hub_ip:
    topo.hub_ip = "172.20.0.10"

hub = topo.hub
if hub is None and topo.hub_ip:
    # Remote hub (WAN): it does not run here, but it is part of the
    # diagram - all inter-operator SS7 goes through it.
    hub = Node("hub-distant", "interstp")
    hub.point_code = "0.0.0"
    hub.hub_ip = topo.hub_ip
```

```
        hub.hub_port = topo.hub_port
        topo.nodes.append(hub)
    elif hub is not None and not hub.point_code:
        hub.point_code = "0.0.0"
    return topo
```

2.112 /opt/GSM/osmo-operator/navigation/tui.py

40087 octets, 916 lignes → 916 lignes

```
# tui.py - the console: a network diagram browsed with the arrow keys, from
# which either a VTY or an SS7 script is opened.
#
# Navigation principles, kept everywhere:
#   arrows       move (in the diagram: from box to box, in space)
#   Enter        open / confirm
#   Escape       go back; IN A VTY, Escape closes the session and returns to
#                the diagram - the other VTYs stay available
#   q            quit (from the diagram)
#
# The rest (v, s, a, c, r, ?) are shortcuts: everything is also reachable
# with the arrows from a box menu.

import curses
import locale
import os
import time

from . import diagram
from . import mapops
from . import quickcmd
from . import sccp
from . import ss7
from . import topo as topo_mod
from . import vty as vty_mod

locale.setlocale(locale.LC_ALL, "")

C_DEFAULT, C_HUB, C_STP, C_CORE, C_RADIO, C_DATA, C_CONSOLE, C_BAR, C_WARN, C_OK = range(1, 11)

FAMILY_COLOR = {
    "SS7": C_STP, "coeur": C_CORE, "abonnes": C_CORE, "radio": C_RADIO,
    "media": C_DATA, "data": C_DATA, "voix": C_CORE, "mobile": C_RADIO,
}

HELP = [
    "NAVIGATION",
    "  fleches            se deplacer dans le schema (haut/bas/gauche/droite)",
    "  Entree             ouvrir le menu de l'element selectionne",
    "  Echap              revenir en arriere ; dans un VTY : fermer la session",
    "  Tab                element suivant, Maj-Tab precedent",
    "",
    "RACCOURCIS",
    "  v                  ouvrir directement le VTY de l'element",
    "  s                  scripts SS7 (operations MAP) sur l'element",
    "  a                  audit SS7 (DAUD) d'un point code",
    "  c                  connecter / deconnecter le lien M3UA de la console",
    "  j                  journal du lien M3UA",
    "  r                  relire la topologie (redecouverte complete)",
    "  ?                  cette aide            q  quitter",
    "",
    "DANS UN VTY",
    "  Entree             envoyer la commande",
    "  fleches haut/bas   historique des commandes",
```

```
    "  PagePrec/PageSuiv  faire defiler la sortie",
    "  Echap              fermer CE VTY et revenir au schema",
    "",
    "SCRIPTS SS7",
    "  La console s'enregistre sur l'inter-STP comme un ASP M3UA a part",
    "  entiere (RKM dynamique), avec son propre point code. Elle emet du",
    "  TCAP/MAP reel : sendRoutingInfoForSM, sendRoutingInfo, ATI, SAI...",
    "  Chaque envoi peut etre exporte en script Python autonome.",
]


# ── key reading ──────────────────────────────────────────────────────
# ncurses only recognises the arrows in the form given by the terminal's
# terminfo (here "\x1bOB", application cursor mode). A multiplexer, a badly
# configured SSH or a terminal left in normal mode send "\x1b[B": ncurses then
# returns three raw keys and navigation looks dead. Both forms are therefore
# decoded here. A lone Escape stays Escape: what tells it from the start of a
# sequence is the absence of a follow-up within 40 ms.
_CSI_FINAL = {
    "A": curses.KEY_UP, "B": curses.KEY_DOWN, "C": curses.KEY_RIGHT,
    "D": curses.KEY_LEFT, "H": curses.KEY_HOME, "F": curses.KEY_END,
    "Z": curses.KEY_BTAB, "P": curses.KEY_F1, "Q": curses.KEY_F2,
    "R": curses.KEY_F3, "S": curses.KEY_F4,
}
_CSI_TILDE = {1: curses.KEY_HOME, 2: curses.KEY_IC, 3: curses.KEY_DC,
              4: curses.KEY_END, 5: curses.KEY_PPAGE, 6: curses.KEY_NPAGE,
              15: curses.KEY_F5, 17: curses.KEY_F6, 18: curses.KEY_F7,
              19: curses.KEY_F8, 20: curses.KEY_F9, 21: curses.KEY_F10}


def read_key(win):
    key = win.getch()
    if key != 27:
        return key
    win.timeout(40)
    try:
        nxt = win.getch()
        if nxt == -1:
            return 27
        if nxt not in (ord("["), ord("O")):
            return 27
        params = ""
        for _ in range(8):
            ch = win.getch()
            if ch == -1:
                return 27
            c = chr(ch)
            if c.isdigit() or c == ";":
                params += c
                continue
            if c == "~":
                try:
                    return _CSI_TILDE.get(int(params.split(";")[0]), 27)
                except ValueError:
                    return 27
            return _CSI_FINAL.get(c, 27)
        return 27
    finally:
        win.timeout(-1)


def init_colors():
    curses.start_color()
    curses.use_default_colors()
    pairs = {
        C_DEFAULT: (curses.COLOR_WHITE, -1),
```

```
        C_HUB: (curses.COLOR_MAGENTA, -1),
        C_STP: (curses.COLOR_CYAN, -1),
        C_CORE: (curses.COLOR_GREEN, -1),
        C_RADIO: (curses.COLOR_YELLOW, -1),
        C_DATA: (curses.COLOR_BLUE, -1),
        C_CONSOLE: (curses.COLOR_WHITE, -1),
        C_BAR: (curses.COLOR_BLACK, curses.COLOR_CYAN),
        C_WARN: (curses.COLOR_RED, -1),
        C_OK: (curses.COLOR_GREEN, -1),
    }
    for idx, (fg, bg) in pairs.items():
        try:
            curses.init_pair(idx, fg, bg)
        except curses.error:
            pass


class App(object):
    def __init__(self, stdscr, topo):
        self.scr = stdscr
        self.topo = topo
        self.pool = vty_mod.VtyPool(docker=topo.docker)
        self.client = None
        self.local_pc = topo.free_point_code()
        self.local_ssn = mapops.SSN_MSC
        self.sel = 0
        self.top = 0
        self.left = 0
        self.status = "Bienvenue. Fleches pour circuler, Entree pour ouvrir, ? pour l'aide."
        self.status_attr = C_DEFAULT
        self.rebuild()

    # ── state ──────────────────────────────────────────────────────
    def rebuild(self):
        extra = {"local_pc": self.local_pc, "local_ssn": self.local_ssn,
                 "state": (self.client.asp.state if self.client and self.client.asp
                           else "hors ligne")}
        self.lines, self.boxes = diagram.build(self.topo, extra)
        self.sel = min(self.sel, max(0, len(self.boxes) - 1))

    def say(self, text, attr=C_DEFAULT):
        self.status = text
        self.status_attr = attr

    # ── drawing ────────────────────────────────────────────────────
    def draw(self):
        self.scr.erase()
        h, w = self.scr.getmaxyx()
        title = " CONSOLE SS7 - osmo-operator "
        link = "M3UA %s" % (self.client.asp.state if self.client and self.client.asp
                            else "hors ligne")
        head = "%s| %s operateur(s) | hub %s:%s | console PC %s | %s" % (
            title, len(self.topo.operators), self.topo.hub_ip or "?",
            self.topo.hub_port, self.local_pc, link)
        self.scr.attron(curses.color_pair(C_BAR))
        self.scr.addstr(0, 0, head.ljust(w - 1)[:w - 1])
        self.scr.attroff(curses.color_pair(C_BAR))

        view_h = h - 3
        self.draw_map(1, view_h, w)

        foot = ("fleches:circuler  Entree:ouvrir  v:VTY  s:script SS7  "
                "a:audit  c:lien M3UA  r:relire  ?:aide  q:quitter")
        self.scr.attron(curses.color_pair(C_BAR))
        self.scr.addstr(h - 1, 0, foot.ljust(w - 1)[:w - 1])
        self.scr.attroff(curses.color_pair(C_BAR))
```

```
    self.scr.attron(curses.color_pair(self.status_attr))
    self.scr.addstr(h - 2, 0, self.status.ljust(w - 1)[:w - 1])
    self.scr.attroff(curses.color_pair(self.status_attr))
    self.scr.noutrefresh()
    curses.doupdate()

def _scroll_to_selection(self, view_h, view_w):
    if not self.boxes:
        return
    b = self.boxes[self.sel]
    if b.y < self.top:
        self.top = max(0, b.y - 1)
    if b.y + b.h > self.top + view_h:
        self.top = b.y + b.h - view_h
    if b.x < self.left:
        self.left = max(0, b.x - 2)
    if b.x + b.w > self.left + view_w:
        self.left = b.x + b.w - view_w + 2
    self.top = max(0, self.top)
    self.left = max(0, self.left)

def draw_map(self, y0, view_h, view_w):
    self._scroll_to_selection(view_h, view_w)
    for row in range(view_h):
        src = self.top + row
        if src >= len(self.lines):
            break
        text = self.lines[src][self.left:self.left + view_w - 1]
        try:
            self.scr.addstr(y0 + row, 0, text)
        except curses.error:
            pass
    for i, b in enumerate(self.boxes):
        attr = curses.color_pair(self._box_color(b))
        if i == self.sel:
            attr = curses.color_pair(self._box_color(b)) | curses.A_REVERSE | curses.A_BOLD
        for row in range(b.h):
            src = b.y + row
            if src < self.top or src >= self.top + view_h:
                continue
            if src >= len(self.lines):
                continue
            line = self.lines[src]
            seg = line[b.x:b.x + b.w].ljust(b.w)
            sx = b.x - self.left
            if sx < 0 or sx + b.w > view_w - 1:
                seg = seg[max(0, -sx):max(0, view_w - 1 - max(sx, 0))]
                sx = max(sx, 0)
            try:
                self.scr.addstr(y0 + src - self.top, sx, seg, attr)
            except curses.error:
                pass

def _box_color(self, b):
    if b.kind == "hub":
        return C_HUB
    if b.kind == "console":
        return C_CONSOLE
    if b.kind == "stp":
        return C_STP
    if b.ref is not None and getattr(b.ref, "family", None):
        return FAMILY_COLOR.get(b.ref.family, C_DEFAULT)
    return C_DEFAULT

# ── generic windows ──────────────────────────────────────────
def _win(self, height, width, title):
```

```
    h, w = self.scr.getmaxyx()
    height = min(height, h - 2)
    width = min(width, w - 2)
    y = max(0, (h - height) // 2)
    x = max(0, (w - width) // 2)
    win = curses.newwin(height, width, y, x)
    win.keypad(True)
    win.box()
    win.addstr(0, 2, " %s " % title[:width - 6], curses.A_BOLD)
    return win

def menu(self, title, items, footer="fleches + Entree, Echap pour revenir"):
    """items: list of (label, value). Return the value, or None."""
    if not items:
        self.show_text(title, ["(rien a proposer ici)"])
        return None
    idx = 0
    width = max([len(title) + 8] + [len(i[0]) + 8 for i in items] + [len(footer) + 6])
    win = self._win(len(items) + 4, min(width, 100), title)
    while True:
        win.erase()
        win.box()
        win.addstr(0, 2, " %s " % title, curses.A_BOLD)
        for i, (label, _val) in enumerate(items):
            attr = curses.A_REVERSE if i == idx else curses.A_NORMAL
            try:
                win.addstr(1 + i, 2, ("%-*s" % (win.getmaxyx()[1] - 4, label))[
                    :win.getmaxyx()[1] - 4], attr)
            except curses.error:
                pass
        try:
            win.addstr(win.getmaxyx()[0] - 2, 2, footer[:win.getmaxyx()[1] - 4],
                       curses.A_DIM)
        except curses.error:
            pass
        win.refresh()
        key = read_key(win)
        if key in (curses.KEY_UP, ord("k")):
            idx = (idx - 1) % len(items)
        elif key in (curses.KEY_DOWN, ord("j")):
            idx = (idx + 1) % len(items)
        elif key in (curses.KEY_HOME,):
            idx = 0
        elif key in (curses.KEY_END,):
            idx = len(items) - 1
        elif key in (10, 13, curses.KEY_ENTER, curses.KEY_RIGHT):
            return items[idx][1]
        elif key in (27, curses.KEY_LEFT, ord("q")):
            return None
        elif key == curses.KEY_RESIZE:
            self.draw()

def show_text(self, title, lines, wrap=True):
    """Scrolling text area. Escape or q to go back."""
    h, w = self.scr.getmaxyx()
    win = self._win(h - 2, w - 2, title)
    inner_h = win.getmaxyx()[0] - 3
    inner_w = win.getmaxyx()[1] - 4
    flat = []
    for line in lines:
        line = line.replace("\t", "    ")
        if wrap:
            while len(line) > inner_w:
                flat.append(line[:inner_w])
                line = line[inner_w:]
        flat.append(line[:inner_w])
```

```
        pos = 0
        while True:
            win.erase()
            win.box()
            win.addstr(0, 2, " %s " % title[:inner_w], curses.A_BOLD)
            for i in range(inner_h):
                if pos + i >= len(flat):
                    break
                try:
                    win.addstr(1 + i, 2, flat[pos + i])
                except curses.error:
                    pass
            hint = "fleches/PagePrec/PageSuiv pour defiler - Echap pour revenir"
            if len(flat) > inner_h:
                hint = "%d/%d - %s" % (pos + 1, len(flat), hint)
            try:
                win.addstr(win.getmaxyx()[0] - 2, 2, hint[:inner_w], curses.A_DIM)
            except curses.error:
                pass
            win.refresh()
            key = read_key(win)
            if key in (27, ord("q")):
                return
            elif key == curses.KEY_DOWN:
                pos = min(max(0, len(flat) - inner_h), pos + 1)
            elif key == curses.KEY_UP:
                pos = max(0, pos - 1)
            elif key == curses.KEY_NPAGE:
                pos = min(max(0, len(flat) - inner_h), pos + inner_h)
            elif key == curses.KEY_PPAGE:
                pos = max(0, pos - inner_h)
            elif key == curses.KEY_HOME:
                pos = 0
            elif key == curses.KEY_END:
                pos = max(0, len(flat) - inner_h)
            elif key == curses.KEY_RESIZE:
                self.draw()
                return

    def form(self, title, fields, note=""):
        """fields: list of dicts {key,label,value,help}. Return a dict or None.

        Navigation: up/down between fields, type to edit, Enter on
        [ Envoyer ] to submit, Escape to give up."""
        idx = 0
        rows = fields + [{"key": "__submit__", "label": "[ Envoyer ]", "value": "",
                          "help": "Entree pour lancer"}]
        height = len(rows) + (6 if note else 5)
        width = 78
        win = self._win(height, width, title)
        inner_w = win.getmaxyx()[1] - 4
        while True:
            win.erase()
            win.box()
            win.addstr(0, 2, " %s " % title, curses.A_BOLD)
            y = 1
            if note:
                win.addstr(y, 2, note[:inner_w], curses.A_DIM)
                y += 1
            for i, f in enumerate(rows):
                attr = curses.A_REVERSE if i == idx else curses.A_NORMAL
                if f["key"] == "__submit__":
                    win.addstr(y + i, 2, ("%-*s" % (inner_w, f["label"]))[:inner_w],
                               attr | curses.A_BOLD)
                else:
                    label = "%-26s" % f["label"][:26]
```

```
                val = str(f.get("value", ""))
                win.addstr(y + i, 2, label, curses.A_NORMAL)
                win.addstr(y + i, 2 + len(label),
                           ("%-*s" % (inner_w - len(label), val))[:inner_w - len(label)],
                           attr)
        cur = rows[idx]
        hint = cur.get("help") or "haut/bas : champ suivant - Echap : abandonner"
        try:
            win.addstr(win.getmaxyx()[0] - 2, 2, hint[:inner_w], curses.A_DIM)
        except curses.error:
            pass
        win.refresh()
        key = read_key(win)
        if key == 27:
            return None
        if key in (curses.KEY_UP,):
            idx = (idx - 1) % len(rows)
        elif key in (curses.KEY_DOWN, 9):
            idx = (idx + 1) % len(rows)
        elif key in (10, 13, curses.KEY_ENTER):
            if rows[idx]["key"] == "__submit__":
                return dict((f["key"], f.get("value", "")) for f in fields)
            idx = (idx + 1) % len(rows)
        elif key in (curses.KEY_BACKSPACE, 127, 8):
            if cur["key"] != "__submit__":
                cur["value"] = str(cur.get("value", ""))[:-1]
        elif key == 21:                       # Ctrl-U
            if cur["key"] != "__submit__":
                cur["value"] = ""
        elif key == curses.KEY_RESIZE:
            self.draw()
        elif 32 <= key < 127 and cur["key"] != "__submit__":
            cur["value"] = str(cur.get("value", "")) + chr(key)

def confirm(self, question):
    return self.menu(question, [("Oui", True), ("Non", False)]) is True

# ── VTY ──────────────────────────────────────────────────────────
def vty_view(self, node, port, label):
    try:
        sess = self.pool.get(node, port)
    except vty_mod.VtyError as exc:
        self.say("VTY %s:%s - %s" % (node, port, exc), C_WARN)
        return
    lines = [l for l in (sess.banner or "").split("\n") if l.strip()]
    lines.append("")
    lines.append("--- Echap ferme cette session et revient au schema ---")
    history = []
    hpos = 0
    current = ""
    scroll = None
    h, w = self.scr.getmaxyx()
    win = curses.newwin(h, w, 0, 0)
    win.keypad(True)
    while True:
        win.erase()
        head = " VTY %s - %s (port %d) " % (label, node, port)
        win.attron(curses.color_pair(C_BAR))
        win.addstr(0, 0, head.ljust(w - 1)[:w - 1])
        win.attroff(curses.color_pair(C_BAR))
        body_h = h - 3
        view = lines if scroll is None else lines[:scroll + body_h]
        start = max(0, len(view) - body_h)
        for i, line in enumerate(view[start:start + body_h]):
            try:
                win.addstr(1 + i, 0, line[:w - 1])
```

```
    except curses.error:
        pass
foot = ("Entree:envoyer  haut/bas:historique  PagePrec/Suiv:defiler  "
        "Echap:fermer ce VTY")
win.attron(curses.color_pair(C_BAR))
win.addstr(h - 2, 0, foot.ljust(w - 1)[:w - 1])
win.attroff(curses.color_pair(C_BAR))
prompt = "%s> " % label
win.addstr(h - 1, 0, (prompt + current)[:w - 1], curses.A_BOLD)
win.move(h - 1, min(w - 1, len(prompt) + len(current)))
win.refresh()
key = read_key(win)
if key == 27:
    self.pool.drop(node, port)
    self.say("VTY %s:%s ferme - les autres VTY restent ouverts."
             % (node, port), C_OK)
    return
if key in (10, 13, curses.KEY_ENTER):
    cmd = current.strip()
    current = ""
    if not cmd:
        continue
    if cmd in ("quit", "exit"):
        self.pool.drop(node, port)
        return
    history.append(cmd)
    hpos = len(history)
    lines.append("")
    lines.append("%s> %s" % (label, cmd))
    try:
        out = sess.cmd(cmd)
    except vty_mod.VtyError as exc:
        out = "[session perdue : %s]" % exc
    rows = out.split("\n")
    # The VTY echoes the command back: it is already displayed
    # above, so it is not shown twice.
    if rows and rows[0].strip() == cmd:
        rows = rows[1:]
    lines.extend(rows)
    scroll = None
elif key in (curses.KEY_BACKSPACE, 127, 8):
    current = current[:-1]
elif key == 21:
    current = ""
elif key == curses.KEY_UP:
    if history:
        hpos = max(0, hpos - 1)
        current = history[hpos]
elif key == curses.KEY_DOWN:
    if history:
        hpos = min(len(history), hpos + 1)
        current = history[hpos] if hpos < len(history) else ""
elif key == curses.KEY_PPAGE:
    base = len(lines) if scroll is None else scroll
    scroll = max(0, base - (h - 3))
elif key == curses.KEY_NPAGE:
    if scroll is not None:
        scroll += h - 3
        if scroll >= len(lines):
            scroll = None
elif key == curses.KEY_RESIZE:
    h, w = self.scr.getmaxyx()
    win = curses.newwin(h, w, 0, 0)
    win.keypad(True)
elif 32 <= key < 127:
    current += chr(key)
```

```
# ── SS7 link ──────────────────────────────────────────────────────
def ensure_link(self):
    if self.client and self.client.connected:
        return True
    if not self.topo.hub_ip:
        self.say("aucun hub SS7 connu : pas de lien M3UA possible", C_WARN)
        return False
    self.say("connexion M3UA vers %s:%s ..." % (self.topo.hub_ip, self.topo.hub_port))
    self.draw()
    try:
        self.client = ss7.Ss7Client(self.topo.hub_ip, self.topo.hub_port,
                                    self.local_pc, self.local_ssn)
        state = self.client.connect()
    except Exception as exc:
        self.say("lien M3UA impossible : %s" % exc, C_WARN)
        self.client = None
        return False
    self.rebuild()
    self.say("lien M3UA %s - la console est un ASP du hub (PC %s)"
             % (state, self.local_pc), C_OK)
    return True

def toggle_link(self):
    if self.client and self.client.connected:
        self.client.close()
        self.client = None
        self.rebuild()
        self.say("lien M3UA ferme", C_WARN)
    else:
        self.ensure_link()

def audit_dialog(self, default_pc=""):
    if not self.ensure_link():
        return
    vals = self.form("Audit SS7 (DAUD)", [
        {"key": "pc", "label": "Point code a auditer", "value": default_pc,
         "help": "ex 1.11.1 - le STP repond DAVA (joignable) ou DUNA"},
    ])
    if not vals or not vals["pc"].strip():
        return
    rep = self.client.audit(vals["pc"].strip())
    self.show_text("Audit %s" % vals["pc"], rep.lines + [""] +
                   ["Journal du lien :"] + self.client.events[-12:])

def map_dialog(self, default_pc="", default_ssn=None):
    if not self.ensure_link():
        return
    items = [("%-22s %s" % (o.key, o.name), o.key) for o in mapops.OPERATIONS]
    key = self.menu("Operations MAP - schema %s" % self.local_pc, items)
    if key is None:
        return
    op = mapops.BY_KEY[key]
    fields = []
    for p in op.params:
        fields.append({"key": p.key, "label": p.label, "value": p.default,
                       "help": p.help})
    fields.append({"key": "__pc", "label": "Point code destination",
                   "value": default_pc or (self.topo.operators[0].sccp.get(
                       "addr-msc", {}).get("pc", "") if self.topo.operators else ""),
                   "help": "ou vide si routage sur GT"})
    fields.append({"key": "__ssn", "label": "SSN destination",
                   "value": str(default_ssn if default_ssn else op.ssn),
                   "help": "6 HLR, 7 VLR, 8 MSC, 147 gsmSCF, 254 BSSAP"})
    fields.append({"key": "__gt", "label": "Global title destination",
                   "value": "", "help": "laisser vide pour router sur PC/SSN"})
```

```
    vals = self.form("%s (opcode %d)" % (op.name, op.opcode), fields,
                     note=op.summary[:74])
    if vals is None:
        return
    dest_pc = vals.pop("__pc").strip()
    dest_ssn = vals.pop("__ssn").strip()
    dest_gt = vals.pop("__gt").strip()
    if not dest_pc and not dest_gt:
        self.say("il faut au moins un point code ou un global title", C_WARN)
        return
    self.say("emission de %s ..." % op.name)
    self.draw()
    try:
        rep = self.client.send_map(op, vals, dest_pc or "0.0.0",
                                   int(dest_ssn) if dest_ssn else None,
                                   dest_gt, route_on_gt=bool(dest_gt and not dest_pc))
    except Exception as exc:
        self.show_text("Echec", ["%s : %s" % (type(exc).__name__, exc)])
        return
    lines = rep.lines + ["", "Journal du lien :"] + self.client.events[-10:]
    lines += ["", "e : exporter cet envoi en script Python autonome",
              "Echap : revenir"]
    self.show_text("%s -> %s" % (op.name, dest_gt or dest_pc), lines)
    if self.confirm("Exporter cet envoi en script Python ?"):
        path = export_script(op, vals, dest_pc, dest_ssn, dest_gt,
                             self.local_pc, self.topo)
        self.show_text("Script ecrit", [
            "Le script autonome est ecrit dans :", "", "  %s" % path, "",
            "Il rejoue exactement cet envoi :",
            "  python3 %s" % os.path.relpath(path, topo_mod.REPO)])

# ── box menus ──────────────────────────────────────────────────────
def open_box(self, box):
    if box.kind == "console":
        self.console_menu()
    elif box.kind == "hub":
        self.hub_menu(box)
    elif box.kind == "stp":
        self.stp_menu(box)
    elif box.kind == "service":
        self.service_menu(box)

def console_menu(self):
    connected = bool(self.client and self.client.connected)
    items = [
        ("Deconnecter le lien M3UA" if connected else "Connecter le lien M3UA",
         "link"),
        ("Operations MAP (sendRoutingInfo, SRI-SM, ATI...)", "map"),
        ("Audit d'un point code (DAUD)", "audit"),
        ("Journal du lien M3UA", "log"),
        ("Changer le point code de la console", "pc"),
        ("Informations", "info"),
    ]
    action = self.menu("CONSOLE SS7 (PC %s)" % self.local_pc, items)
    if action == "link":
        self.toggle_link()
    elif action == "map":
        self.map_dialog()
    elif action == "audit":
        self.audit_dialog()
    elif action == "log":
        self.show_text("Journal M3UA",
                       self.client.events if self.client else ["(pas de lien)"])
    elif action == "pc":
        vals = self.form("Point code de la console", [
            {"key": "pc", "label": "Point code", "value": self.local_pc,
```

```
             "help": "il sera enregistre sur le hub par RKM dynamique"},
            {"key": "ssn", "label": "SSN local", "value": str(self.local_ssn),
             "help": "identite SCCP de la console"}])
        if vals:
            self.local_pc = vals["pc"].strip() or self.local_pc
            try:
                self.local_ssn = int(vals["ssn"])
            except ValueError:
                pass
            if self.client:
                self.client.close()
                self.client = None
            self.rebuild()
            self.say("console : PC %s, SSN %d" % (self.local_pc, self.local_ssn))
    elif action == "info":
        self.show_text("Console SS7", [
            "La console se connecte a l'inter-STP en M3UA sur SCTP et",
            "enregistre dynamiquement une routing key pour son point code.",
            "",
            "  hub          : %s:%s" % (self.topo.hub_ip, self.topo.hub_port),
            "  point code   : %s" % self.local_pc,
            "  SSN local    : %s" % self.local_ssn,
            "  etat         : %s" % (self.client.asp.state if self.client
                                     and self.client.asp else "hors ligne"),
            "",
            "Tout ce qu'elle emet passe par le meme chemin SS7 que le trafic",
            "des operateurs : c'est le lab qui route, pas une simulation.",
        ])

def hub_menu(self, box):
    node = box.ref
    items = [("Etat du lien M3UA de la console", "link"),
             ("Auditer un point code a travers le hub", "audit"),
             ("Journal M3UA", "log")]
    if node and node.name != "hub-distant":
        items.insert(0, ("Ouvrir le VTY du hub (STP)", "vty"))
    else:
        items.append(("Pourquoi le hub n'a pas de VTY ici", "why"))
    action = self.menu("INTER-STP %s" % (self.topo.hub_ip or ""), items)
    if action == "vty":
        self.vty_view(node.name, 4239, "hub-STP")
    elif action == "audit":
        self.audit_dialog()
    elif action == "link":
        self.console_menu()
    elif action == "log":
        self.show_text("Journal M3UA",
                       self.client.events if self.client else ["(pas de lien)"])
    elif action == "why":
        self.show_text("Hub distant", [
            "Le hub tourne sur %s : ce n'est pas cette machine." % self.topo.hub_ip,
            "",
            "Le VTY d'Osmocom n'ecoute que sur la boucle locale de SON noeud :",
            "il n'est donc pas interrogeable d'ici. Ce qui l'est, en revanche :",
            "",
            "  - le lien M3UA/SCTP de chaque operateur vers le hub ;",
            "  - l'etat des AS et ASP vus du cote operateur (VTY du STP local) ;",
            "  - l'audit DAUD d'un point code a travers le hub.",
            "",
            "Sur le noeud du hub : ./launch/start-interstp.sh --status",
        ])

def stp_menu(self, box):
    node = box.ref
    items = [("Ouvrir le VTY du STP (4239)", "vty"),
             ("AS / ASP / routes (vue rapide)", "quick"),
```

```
            ("Auditer le point code %s" % node.point_code, "audit"),
            ("Cibler ce noeud en MAP", "map"),
            ("Configuration heritee (osmo-stp.cfg)", "cfg"),
            ("Informations", "info")]
    action = self.menu("%s - STP %s" % (node.title, node.point_code), items)
    if action == "vty":
        self.vty_view(node.name, 4239, "%s-STP" % node.title)
    elif action == "quick":
        self.quick_menu(node, "OsmoSTP", 4239)
    elif action == "audit":
        self.audit_dialog(node.point_code)
    elif action == "map":
        self.map_dialog(node.point_code)
    elif action == "cfg":
        self.show_text("osmo-stp.cfg (%s)" % node.name,
                       node.cfg.get("osmo-stp.cfg", "(non lu)").split("\n"),
                       wrap=False)
    elif action == "info":
        lines = ["noeud        : %s" % node.name,
                 "point code   : %s" % node.point_code,
                 "M3UA local   : %s" % node.local_m3ua,
                 "hub          : %s:%s" % (node.hub_ip, node.hub_port),
                 "routing key  : %s" % node.routing_ctx,
                 "noeud WAN    : %s" % (node.wan_node or "-"),
                 "", "Adresses SCCP heritees des configurations :"]
        for name, addr in sorted(node.sccp.items()):
            lines.append("  %-22s PC %-8s SSN %-4s (%s)" % (
                name, addr.get("pc") or "-", addr.get("ssn") or "-",
                addr.get("source")))
        lines += ["", "Services VTY decouverts :"]
        for s in node.services:
            lines.append("  %-12s port %s" % (s.label, s.port))
        self.show_text("Informations %s" % node.title, lines)

def service_menu(self, box):
    svc = box.ref
    node = box.node
    items = [("Ouvrir le VTY (%d)" % svc.port, "vty"),
             ("Commandes rapides", "quick"),
             ("Informations", "info")]
    if svc.daemon in ("OsmoMSC", "OsmoHLR", "OsmoSGSN"):
        items.insert(2, ("Cibler ce demon en MAP", "map"))
    action = self.menu("%s - %s" % (node.title, svc.label), items)
    if action == "vty":
        self.vty_view(node.name, svc.port, "%s-%s" % (node.title, svc.label))
    elif action == "quick":
        self.quick_menu(node, svc.daemon, svc.port)
    elif action == "map":
        ssn = {"OsmoHLR": mapops.SSN_HLR, "OsmoMSC": mapops.SSN_MSC,
               "OsmoSGSN": 149}.get(svc.daemon, mapops.SSN_HLR)
        pc = ""
        for key in ("addr-msc", "addr-bsc"):
            if key in node.sccp and node.sccp[key].get("pc"):
                pc = node.sccp[key]["pc"]
                break
        if svc.daemon == "OsmoMSC" and node.sccp.get("_self_osmo-msc.cfg"):
            pc = node.sccp["_self_osmo-msc.cfg"].get("pc") or pc
        self.map_dialog(pc or node.point_code, ssn)
    elif action == "info":
        self.show_text("%s sur %s" % (svc.label, node.name), [
            "demon        : %s" % svc.daemon,
            "port VTY     : %s" % svc.port,
            "famille      : %s" % svc.family,
            "noeud        : %s (%s)" % (node.name, node.point_code),
            "",
            "Echap ferme un VTY sans fermer les autres : la console garde",
```

```
            "une session par port, et le schema reste accessible.",
        ])

def quick_menu(self, node, daemon, port):
    cmds = quickcmd.for_daemon(daemon)
    items = [("%-32s %s" % (label, cmd), cmd) for label, cmd in cmds]
    cmd = self.menu("Commandes rapides - %s" % daemon, items)
    if cmd is None:
        return
    try:
        out = self.pool.cmd(node.name, port, cmd)
    except vty_mod.VtyError as exc:
        out = "[VTY indisponible : %s]" % exc
    self.show_text("%s : %s" % (node.name, cmd), out.split("\n"), wrap=False)

# ── main loop ──────────────────────────────────────────────────
# Shortcuts are LOWERCASE only: arrow sequences end with A, B, C or D,
# and a sequence badly split by the terminal would otherwise arrive as a
# shortcut - an arrow would close the M3UA link. With lowercase letters
# only, the worst case does nothing.
def run(self):
    while True:
        self.draw()
        key = read_key(self.scr)
        if key == ord("q"):
            if self.pool.sessions and not self.confirm(
                    "Des sessions VTY sont ouvertes. Quitter ?"):
                continue
            return
        elif key == curses.KEY_RESIZE:
            continue
        elif key == curses.KEY_UP:
            self.sel = diagram.navigate(self.boxes, self.sel, (0, -1))
        elif key == curses.KEY_DOWN:
            self.sel = diagram.navigate(self.boxes, self.sel, (0, 1))
        elif key == curses.KEY_LEFT:
            self.sel = diagram.navigate(self.boxes, self.sel, (-1, 0))
        elif key == curses.KEY_RIGHT:
            self.sel = diagram.navigate(self.boxes, self.sel, (1, 0))
        elif key == 9:
            self.sel = (self.sel + 1) % max(1, len(self.boxes))
        elif key == curses.KEY_BTAB:
            self.sel = (self.sel - 1) % max(1, len(self.boxes))
        elif key in (10, 13, curses.KEY_ENTER):
            if self.boxes:
                self.open_box(self.boxes[self.sel])
        elif key == ord("v"):
            self.direct_vty()
        elif key == ord("s"):
            box = self.boxes[self.sel] if self.boxes else None
            pc = box.node.point_code if box and box.node else ""
            self.map_dialog(pc)
        elif key == ord("a"):
            box = self.boxes[self.sel] if self.boxes else None
            self.audit_dialog(box.node.point_code if box and box.node else "")
        elif key == ord("c"):
            self.toggle_link()
        elif key == ord("j"):
            self.show_text("Journal M3UA",
                           self.client.events if self.client else ["(pas de lien)"])
        elif key == ord("r"):
            self.say("redecouverte de la topologie ...")
            self.draw()
            self.pool.close_all()
            self.topo = topo_mod.discover()
            self.rebuild()
```

```
                self.say("topologie relue : %d operateur(s), hub %s"
                         % (len(self.topo.operators), self.topo.hub_ip or "-"), C_OK)
            elif key in (ord("?"), ord("h")):
                self.show_text("Aide", HELP)
            elif key == 27:
                self.say("Echap : deja au schema. q pour quitter, ? pour l'aide.")

    def direct_vty(self):
        if not self.boxes:
            return
        box = self.boxes[self.sel]
        if box.kind == "service":
            self.vty_view(box.node.name, box.ref.port,
                          "%s-%s" % (box.node.title, box.ref.label))
        elif box.kind == "stp":
            self.vty_view(box.ref.name, 4239, "%s-STP" % box.ref.title)
        elif box.kind == "hub" and box.ref and box.ref.name != "hub-distant":
            self.vty_view(box.ref.name, 4239, "hub-STP")
        else:
            self.say("cet element n'a pas de VTY joignable depuis ici", C_WARN)


def export_script(op, values, dest_pc, dest_ssn, dest_gt, local_pc, topo):
    """Write a standalone Python script that replays exactly this send."""
    out_dir = os.path.join(topo_mod.REPO, "navigation", "scripts")
    os.makedirs(out_dir, exist_ok=True)
    stamp = time.strftime("%Y%m%d-%H%M%S")
    path = os.path.join(out_dir, "%s-%s.py" % (op.key, stamp))
    body = '''#!/usr/bin/env python3
# Generated by the SS7 console (navigation/) on {stamp}.
# Replays a MAP {name} send to {dest}.
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", ".."))
from navigation import ss7

client = ss7.Ss7Client({hub!r}, {port!r}, local_pc={local!r})
print("lien M3UA :", client.connect())
rapport = client.send_map({op!r}, {values!r},
                          dest_pc={pc!r}, dest_ssn={ssn!r}, dest_gt={gt!r},
                          route_on_gt={rogt!r})
print(rapport)
client.close()
'''.format(stamp=stamp, name=op.name, dest=dest_gt or dest_pc,
           hub=topo.hub_ip, port=topo.hub_port, local=local_pc, op=op.key,
           values=values, pc=dest_pc or "0.0.0",
           ssn=int(dest_ssn) if dest_ssn else op.ssn, gt=dest_gt,
           rogt=bool(dest_gt and not dest_pc))
    with open(path, "w") as fh:
        fh.write(body)
    os.chmod(path, 0o755)
    return path


def main(topo):
    def _run(stdscr):
        curses.curs_set(0)
        try:
            curses.set_escdelay(60)
        except Exception:
            pass
        init_colors()
        stdscr.keypad(True)
        app = App(stdscr, topo)
        try:
            app.run()
        finally:
```

```
            app.pool.close_all()
            if app.client:
                app.client.close()
    curses.wrapper(_run)
```

2.113 /opt/GSM/osmo-operator/navigation/vty.py

9398 octets, 271 lignes → 271 lignes

```
# vty.py - access to the Osmocom VTYs from the host.
#
# The lab VTYs only listen on 127.0.0.1 INSIDE the node: from the host, they
# are reached through "docker exec". Not every container has nc; they all
# have python3, telnet and socat. A small python relay is therefore pushed
# into the container, which also gives a PERSISTENT session: the VTY keeps
# its context (enable, config node) from one command to the next, exactly
# like a manual telnet.
#
# In native mode (no docker), the same class talks directly to the local socket.

import os
import re
import shutil
import socket
import subprocess
import threading
import time

EOC = b"\x01EOC\x01"

# Relay executed INSIDE the node. Protocol: one command per line on stdin,
# the raw VTY output on stdout, terminated by the EOC marker.
_RELAY = r'''
import socket, sys, time
port = int(sys.argv[1])
EOC = b"\x01EOC\x01\n"
try:
    s = socket.create_connection(("127.0.0.1", port), 5)
except Exception as e:
    sys.stdout.buffer.write(("__ERR__ %s\n" % e).encode()); sys.stdout.buffer.write(EOC)
    sys.stdout.buffer.flush(); sys.exit(0)
s.settimeout(0.35)
def pump(idle=0.35, hard=8.0):
    out = b""; t0 = time.time(); last = time.time()
    while True:
        try:
            d = s.recv(65536)
            if not d:
                break
            out += d; last = time.time()
        except socket.timeout:
            if out and (time.time() - last) >= idle:
                break
            if (time.time() - t0) > hard:
                break
    return out
sys.stdout.buffer.write(pump()); sys.stdout.buffer.write(EOC); sys.stdout.buffer.flush()
for line in sys.stdin:
    cmd = line.rstrip("\n")
    try:
        s.sendall(cmd.encode() + b"\r\n")
    except Exception as e:
        sys.stdout.buffer.write(("__ERR__ %s\n" % e).encode())
        sys.stdout.buffer.write(EOC); sys.stdout.buffer.flush(); break
```

```
    sys.stdout.buffer.write(pump()); sys.stdout.buffer.write(EOC); sys.stdout.buffer.flush()
'''

_IAC = re.compile(rb"\xff[\xfb-\xfe].|\xff[\xf0-\xfa]")


def clean(raw):
    """Readable VTY output: telnet negotiation, CRs and prompts removed."""
    if isinstance(raw, str):
        raw = raw.encode("utf-8", "replace")
    raw = _IAC.sub(b"", raw)
    txt = raw.decode("utf-8", "replace").replace("\r", "")
    out = []
    for line in txt.split("\n"):
        # The prompt ("OsmoSTP> ", "OsmoHLR# ") comes back with every command:
        # it adds nothing in a panel, and the real prompt is shown by the
        # console itself.
        line = re.sub(r"^Osmo[A-Za-z0-9_-]*[#>]\s?", "", line)
        line = line.replace("\x00", "")
        out.append(line.rstrip())
    while out and not out[0].strip():
        out.pop(0)
    while out and not out[-1].strip():
        out.pop()
    return "\n".join(out)


class VtyError(Exception):
    pass


class VtySession(object):
    """Persistent VTY session to a (node, port)."""

    def __init__(self, node, port, docker=True, timeout=12.0):
        self.node = node
        self.port = int(port)
        self.docker = docker
        self.timeout = timeout
        self.proc = None
        self.sock = None
        self.banner = ""
        self.lock = threading.Lock()
        self.alive = False

    # ── open / close ─────────────────────────────────────────────────
    def open(self):
        if self.alive:
            return self
        if self.docker and not _container_up(self.node):
            raise VtyError("le conteneur %s ne tourne pas" % self.node)
        if self.docker:
            self.proc = subprocess.Popen(
                ["docker", "exec", "-i", self.node, "python3", "-", str(self.port)],
                stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                stderr=subprocess.DEVNULL,
            )
            self.proc.stdin.write(_RELAY.encode())
            self.proc.stdin.flush()
            self.proc.stdin.close()
            # The relay then reads its commands on stdin, which has just been
            # closed. docker exec cannot reopen stdin, so the relay is started
            # again in "script on the command line" mode (see _open_docker).
            self.proc.kill()
            self.proc = None
            return self._open_docker()
```

```
    try:
        self.sock = socket.create_connection(("127.0.0.1", self.port), 5)
    except OSError as exc:
        raise VtyError("VTY 127.0.0.1:%s injoignable (%s)"
                       % (self.port, exc.strerror or exc))
    self.sock.settimeout(0.35)
    self.banner = clean(self._pump_socket())
    self.alive = True
    return self

def _open_docker(self):
    # -c: the relay comes as an argument, stdin stays free for the
    # commands. This is what gives the persistent session.
    self.proc = subprocess.Popen(
        ["docker", "exec", "-i", self.node, "python3", "-c", _RELAY, str(self.port)],
        stdin=subprocess.PIPE, stdout=subprocess.PIPE,
        stderr=subprocess.DEVNULL, bufsize=0,
    )
    try:
        self.banner = clean(self._read_until_eoc())
    except Exception as exc:
        self.close()
        raise VtyError("VTY %s:%s injoignable (%s)" % (self.node, self.port, exc))
    if "__ERR__" in self.banner:
        msg = self.banner.strip()
        self.close()
        raise VtyError(msg.replace("__ERR__", "").strip())
    self.alive = True
    return self

def close(self):
    self.alive = False
    if self.proc:
        try:
            self.proc.stdin.close()
        except Exception:
            pass
        try:
            self.proc.terminate()
            self.proc.wait(timeout=2)
        except Exception:
            try:
                self.proc.kill()
            except Exception:
                pass
        self.proc = None
    if self.sock:
        try:
            self.sock.close()
        except Exception:
            pass
        self.sock = None

# ── dialogue ─────────────────────────────────────────────────────
def _read_until_eoc(self):
    buf = b""
    deadline = time.time() + self.timeout
    while EOC not in buf:
        if time.time() > deadline:
            raise VtyError("delai depasse en lisant le VTY")
        chunk = self.proc.stdout.read(1)
        if not chunk:
            raise VtyError("relais VTY termine")
        buf += chunk
    return buf.split(EOC)[0]
```

```
    def _pump_socket(self, idle=0.35, hard=8.0):
        out = b""
        t0 = last = time.time()
        while True:
            try:
                d = self.sock.recv(65536)
                if not d:
                    break
                out += d
                last = time.time()
            except socket.timeout:
                if out and (time.time() - last) >= idle:
                    break
                if (time.time() - t0) > hard:
                    break
        return out

    def cmd(self, command):
        """Send a command, return the cleaned output."""
        with self.lock:
            if not self.alive:
                self.open()
            if self.docker:
                try:
                    self.proc.stdin.write(command.encode() + b"\n")
                    self.proc.stdin.flush()
                    raw = self._read_until_eoc()
                except (BrokenPipeError, VtyError):
                    self.close()
                    raise VtyError("session VTY perdue (%s:%s)" % (self.node, self.port))
            else:
                self.sock.sendall(command.encode() + b"\r\n")
                raw = self._pump_socket()
            return clean(raw)


class VtyPool(object):
    """Open sessions, one per (node, port). Closed on request - that is what
    Escape does in the console: THIS VTY is left, the others stay
    available."""

    def __init__(self, docker=True):
        self.docker = docker
        self.sessions = {}

    def get(self, node, port):
        key = (node, int(port))
        sess = self.sessions.get(key)
        if sess is None or not sess.alive:
            sess = VtySession(node, port, docker=self.docker)
            sess.open()
            self.sessions[key] = sess
        return sess

    def cmd(self, node, port, command):
        return self.get(node, port).cmd(command)

    def drop(self, node, port):
        key = (node, int(port))
        sess = self.sessions.pop(key, None)
        if sess:
            sess.close()
            return True
        return False

    def close_all(self):
```

```
        for sess in list(self.sessions.values()):
            sess.close()
        self.sessions.clear()


def _container_up(name):
    try:
        out = subprocess.run(["docker", "ps", "--format", "{{.Names}}"],
                             stdout=subprocess.PIPE, stderr=subprocess.DEVNULL,
                             timeout=10).stdout.decode()
    except Exception:
        return False
    return name in out.split()


def docker_available():
    return shutil.which("docker") is not None and os.system(
        "docker ps >/dev/null 2>&1") == 0
```

2.114 /opt/GSM/osmo-operator/network/apn-nat.sh

2332 octets, 55 lignes → 55 lignes

```
#!/bin/bash
# ============================================================================
# network/apn-nat.sh - Internet access for the phones (APN internet, apn0)
#
# The GGSN puts the phones in 176.16.32.0/24 on the apn0 interface. Without
# NAT and forwarding their packets (DNS included) go nowhere: the host's
# FORWARD policy is DROP. This script installs, idempotently:
#   - MASQUERADE 176.16.32.0/24 towards the egress interface (the card that
#     routes to the Internet, detected from the default route);
#   - FORWARD apn0 <-> egress, and the return path of established connections.
#
#   network/apn-nat.sh          apply
#   network/apn-nat.sh --status status only, changes nothing
# ============================================================================
set -u

SUBNET="${APN_SUBNET:-176.16.32.0/24}"
APN_IF="${APN_IF:-apn0}"
MODE=apply
[ "${1:-}" = "--status" ] && MODE=status

OUT_IF="$(ip route get 1.1.1.1 2>/dev/null | sed -n 's/.* dev \([^ ]*\).*/\1/p' | head -1)"
if [ -z "$OUT_IF" ]; then
    echo "apn-nat : pas de route vers Internet - rien a faire"; exit 1
fi

ok() { printf '  %-58s %s\n' "$1" "$2"; }

check_or_add() {   # $1 = table (nat|filter), $2 = chain, rest = rule
    local table="$1" chain="$2"; shift 2
    if iptables -w -t "$table" -C "$chain" "$@" 2>/dev/null; then
        ok "$table $chain $*" "deja present"
    elif [ "$MODE" = status ]; then
        ok "$table $chain $*" "ABSENT"
    else
        if iptables -w -t "$table" -I "$chain" "$@" 2>/dev/null; then
            ok "$table $chain $*" "ajoute"
        else
            ok "$table $chain $*" "ECHEC"; return 1
        fi
    fi
```

```
}

echo "apn-nat : $SUBNET via $APN_IF -> $OUT_IF"
rc=0
# Without ip_forward, MASQUERADE and FORWARD are useless.

if [ "$(sysctl -n net.ipv4.ip_forward 2>/dev/null)" = 1 ]; then ok "net.ipv4.ip_forward" "deja 1"
elif [ "$MODE" = status ]; then ok "net.ipv4.ip_forward" "0 (ABSENT)"
elif sysctl -qw net.ipv4.ip_forward=1 2>/dev/null; then ok "net.ipv4.ip_forward" "mis a 1"
else ok "net.ipv4.ip_forward" "ECHEC"; rc=1; fi
check_or_add nat POSTROUTING -s "$SUBNET" -o "$OUT_IF" -j MASQUERADE || rc=1
check_or_add filter FORWARD -i "$APN_IF" -o "$OUT_IF" -j ACCEPT || rc=1
check_or_add filter FORWARD -i "$OUT_IF" -o "$APN_IF" -m state --state RELATED,ESTABLISHED -j ACCEPT || rc=1
exit $rc
```

2.115 /opt/GSM/osmo-operator/network/firewall-wan.sh

3082 octets, 77 lignes → 77 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════
# network/firewall-wan.sh - opens the ports needed for the WAN interop
#
# Usage: sudo ./network/firewall-wan.sh <remote_ip> [n_operators]
# ═══════════════════════════════════════════════════════════════════════

set -euo pipefail

REMOTE_IP="${1:?Usage: $0 <remote_ip> [n_operators]}"
N_OPS="${2:-2}"
SIP_WAN_BASE=5080
RTP_WAN_BASE=20000
RTP_PER_OP=500
SMS_RELAY_PORT=7890

echo "=== Firewall WAN Interop - Autorisation ${REMOTE_IP} ==="
echo ""

# Firewall detection
if command -v ufw &>/dev/null && ufw status | grep -q "active"; then
    FW="ufw"
elif command -v firewall-cmd &>/dev/null; then
    FW="firewalld"
else
    FW="iptables"
fi

echo "Firewall detecte : ${FW}"
echo ""

for i in $(seq 1 "$N_OPS"); do
    sip_port=$(( SIP_WAN_BASE + (i - 1) * 2 ))
    rtp_start=$(( RTP_WAN_BASE + (i - 1) * RTP_PER_OP ))
    rtp_end=$(( RTP_WAN_BASE + i * RTP_PER_OP - 1 ))

    echo "Op${i}: SIP ${sip_port}/udp  RTP ${rtp_start}:${rtp_end}/udp"

    case "$FW" in
        ufw)
            ufw allow from "$REMOTE_IP" to any port "$sip_port" proto udp
            ufw allow from "$REMOTE_IP" to any port "${rtp_start}:${rtp_end}" proto udp
            ;;
        firewalld)
```

```
            firewall-cmd --permanent --add-rich-rule="rule family=ipv4 source address=${REMOTE_IP} port
            ↪  port=${sip_port} protocol=udp accept"
            firewall-cmd --permanent --add-rich-rule="rule family=ipv4 source address=${REMOTE_IP} port
            ↪  port=${rtp_start}-${rtp_end} protocol=udp accept"
            ;;
        iptables)
            # -C ... || -A: idempotent, no duplicate when re-run at every WAN start.

            iptables -C INPUT -s "$REMOTE_IP" -p udp --dport "$sip_port" -j ACCEPT 2>/dev/null \
                || iptables -A INPUT -s "$REMOTE_IP" -p udp --dport "$sip_port" -j ACCEPT
            iptables -C INPUT -s "$REMOTE_IP" -p udp --dport "${rtp_start}:${rtp_end}" -j ACCEPT 2>/dev/null \
                || iptables -A INPUT -s "$REMOTE_IP" -p udp --dport "${rtp_start}:${rtp_end}" -j ACCEPT
            ;;
    esac
done

# SMS relay port
echo ""
echo "SMS Relay: ${SMS_RELAY_PORT}/tcp"
case "$FW" in
    ufw)
        ufw allow from "$REMOTE_IP" to any port "$SMS_RELAY_PORT" proto tcp
        ;;
    firewalld)
        firewall-cmd --permanent --add-rich-rule="rule family=ipv4 source address=${REMOTE_IP} port
        ↪  port=${SMS_RELAY_PORT} protocol=tcp accept"
        firewall-cmd --reload
        ;;
    iptables)
        iptables -C INPUT -s "$REMOTE_IP" -p tcp --dport "$SMS_RELAY_PORT" -j ACCEPT 2>/dev/null \
            || iptables -A INPUT -s "$REMOTE_IP" -p tcp --dport "$SMS_RELAY_PORT" -j ACCEPT
        ;;
esac

echo ""
echo "✓ Ports ouverts pour ${REMOTE_IP}"
```

2.116 /opt/GSM/osmo-operator/network/loopback.sh

1856 octets, 87 lignes → 87 lignes

```
#!/usr/bin/env bash
set -euo pipefail

SOURCE="gsm_audio.monitor"
ACTION="${1:-status}"
SINK="${2:-$(pactl get-default-sink)}"

find_loopback_ids() {
    pactl list short modules | \
        grep -F "module-loopback" | \
        grep -F "source=${SOURCE}" | \
        grep -F "sink=${SINK}" | \
        awk '{print $1}'
}

check_source() {
    pactl list short sources | grep -F "${SOURCE}" >/dev/null || {
        echo "[FAIL] source ${SOURCE} introuvable"
        exit 1
    }
}

enable_loopback() {
```

```
    check_source
    local ids
    ids="$(find_loopback_ids || true)"

    echo "[*] Source : $SOURCE"
    echo "[*] Sink   : $SINK"

    if [[ -n "${ids}" ]]; then
        echo "[OK] loopback deja present : ${ids}"
    else
        local mid
        mid="$(pactl load-module module-loopback source="${SOURCE}" sink="${SINK}" latency_msec=20)"
        echo "[OK] loopback charge, module id=${mid}"
    fi
}

disable_loopback() {
    local ids
    ids="$(find_loopback_ids || true)"

    echo "[*] Source : $SOURCE"
    echo "[*] Sink   : $SINK"

    if [[ -z "${ids}" ]]; then
        echo "[OK] aucun loopback a retirer"
        return 0
    fi

    while read -r id; do
        [[ -n "${id}" ]] || continue
        pactl unload-module "${id}"
        echo "[OK] loopback retire, module id=${id}"
    done <<< "${ids}"
}

status_loopback() {
    local ids
    ids="$(find_loopback_ids || true)"

    echo "[*] Source : $SOURCE"
    echo "[*] Sink   : $SINK"

    if [[ -n "${ids}" ]]; then
        echo "[OK] loopback actif : ${ids}"
    else
        echo "[INFO] aucun loopback actif"
    fi
}

case "${ACTION}" in
    enable)
        enable_loopback
        ;;
    disable)
        disable_loopback
        ;;
    status)
        status_loopback
        ;;
    *)
        echo "Usage: $0 {enable|disable|status} [sink]"
        exit 1
        ;;
esac
```

2.117 /opt/GSM/osmo-operator/network/node-conf.sh

7801 octets, 153 lignes → 153 lignes

```
#!/bin/bash
# network/node-conf.sh - A NODE'S CARD, produced by the run, read by the other
# ISOs: "mynode<N>.conf".
#
#   node 1:  ./start-native.sh              -> writes /etc/osmocom/mynode1.conf
#            ./start-native.sh --gen-conf   -> the same card, without starting
#   node 2:  ./start-native.sh --wan mynode1.conf [mynode3.conf ...]
#            -> WAN table = this node + the cards, then the usual meshing
#               (network/setup-wan-mesh.sh: SIP trunks, dialplan, SMS routes)
#
# A "node" is an ISO: its native operator, its multi-operator containers and
# its inter-STP. The WAN links several ISOs. The card says everything another
# node must know to reach it: SS7 identity, public address, prefix, number of
# operators (hence SIP/SMS ports), numbering plan. It is written in shell
# KEY=value format, like /etc/osmo-wan.conf, so it can be read with `.` and
# by eye.
#
# This library is SOURCED (start-native.sh, start-multi.sh). It assumes
# network/wan-nodes.sh is already loaded (WAN_IP/WAN_IND/WAN_NOPS/WAN_NODE_LIST,
# wan_default_ind, wan_detect_local_ip).
: "${OSMOCOM_CFG:=/etc/osmocom}"
NODE_CONF_DIR="${NODE_CONF_DIR:-$OSMOCOM_CFG}"

node_conf_path() { printf '%s/mynode%s.conf' "$NODE_CONF_DIR" "${1:-1}"; }

# How many operators THIS node carries: the native one + the RUNNING
# osmo-operator-N containers (start-multi.sh). The topology written by
# addition.sh exists even without multi started; what matters for the mesh's
# SIP/SMS ports is what answers. OSMO_NODE_NOPS forces the value.
node_conf_nops() {
    local n=1 c
    if [ -n "${OSMO_NODE_NOPS:-}" ]; then printf '%s' "$OSMO_NODE_NOPS"; return 0; fi
    if command -v docker >/dev/null 2>&1; then
        c="$(docker ps --format '{{.Names}}' 2>/dev/null | grep -c '^osmo-operator-[0-9]*$')"
        n=$(( 1 + ${c:-0} ))
    fi
    printf '%s' "$n"
}

# The node's operator list: "idx:mode:pc_stp:rctx_inter[:ip]" - the native
# one first, then the running containers of /etc/osmocom/osmo-multi.conf.
# The point code plan is the WAN one: 1.<node><op>.2.
node_conf_ops() {
    local node="$1" op="$2" out="" spec idx mode ip running=""
    out="${op}:native:1.${node}${op}.2:$(( node * 1000 + op * 100 + 50 ))"
    if command -v docker >/dev/null 2>&1; then
        running="$(docker ps --format '{{.Names}}' 2>/dev/null)"
    fi
    if [ -r "$OSMOCOM_CFG/osmo-multi.conf" ]; then
        for spec in $(sed -n 's/^MULTI_OPS="\{0,1\}\([^"]*\)"\{0,1\}/\1/p' "$OSMOCOM_CFG/osmo-multi.conf"); do
            IFS=: read -r idx mode ip _ _ <<< "$spec"
            [ "$mode" = docker ] || continue
            printf '%s\n' "$running" | grep -qx "osmo-operator-$idx" || continue
            out="$out ${idx}:docker:1.${node}${idx}.2:$(( node * 1000 + idx * 100 + 50 )):${ip}"
        done
    fi
    printf '%s' "$out"
}

# node_conf_write FILE NODE OP [HUB_IP]
# Writes the card. Public IP and prefix: the WAN table if it knows this node
# (wan_nodes_load done), otherwise local detection + default prefix derived
```

```
# from the node number (11, 22, 33...).
node_conf_write() {
    local out="$1" node="$2" op="${3:-1}" hub="${4:-}"
    local ip ind nops ops mcc mnc arfcn nms pfx sip sms
    [[ "$node" =~ ^[1-9]$ ]] || { echo "node_conf_write : noeud '$node' invalide (1-9)" >&2; return 1; }
    ip="${WAN_IP[$node]:-}"
    [ -n "$ip" ] || ip="$(wan_detect_local_ip 2>/dev/null)"
    [ -n "$ip" ] || ip="127.0.0.1"
    ind="${WAN_IND[$node]:-}"
    [ -n "$ind" ] || ind="$(wan_default_ind "$node")"
    nops="$(node_conf_nops)"
    ops="$(node_conf_ops "$node" "$op")"
    mcc="$(sed -n 's/^PLAN_MCC=//p' "$OSMOCOM_CFG/radio-plan.env" 2>/dev/null | tail -1)"
    mnc="$(sed -n 's/^PLAN_MNC=//p' "$OSMOCOM_CFG/radio-plan.env" 2>/dev/null | tail -1)"
    arfcn="$(sed -n 's/^PLAN_ARFCN=//p' "$OSMOCOM_CFG/radio-plan.env" 2>/dev/null | tail -1)"
    [ -n "$mcc" ] || mcc="$(awk '/^ *network country code /{print $4; exit}' "$OSMOCOM_CFG/osmo-msc.cfg"
    ↪  2>/dev/null)"
    [ -n "$mnc" ] || mnc="$(awk '/^ *mobile network code /{print $4; exit}' "$OSMOCOM_CFG/osmo-msc.cfg"
    ↪  2>/dev/null)"
    [ -n "$arfcn" ] || arfcn="$(awk '/^ *arfcn /{print $2; exit}' "$OSMOCOM_CFG/osmo-bsc.cfg" 2>/dev/null)"
    nms="${N_MS:-$(sed -n 's/^: "\${N_MS:=\([0-9]*\)}"/\1/p' "$OSMOCOM_CFG/coeur.env" 2>/dev/null | tail -1)}"
    nms="${nms:-2}"
    pfx="${node}00"
    # Ports as seen from outside, same formulas as setup-wan-mesh.sh:
    # a single operator -> 5060 / 7890; otherwise 5080+(j-1)*2 / 7890+j-1.
    if [ "$nops" = 1 ]; then sip=5060; sms=7890; else sip="5080+(j-1)*2"; sms="7890+j-1"; fi
    mkdir -p "$(dirname "$out")" || return 1
    cat > "$out.tmp" <<EOF
# mynode${node}.conf - card of node ${node}, written by start-native.sh on $(date '+%F %T')
# Copy it to another node and hand it over:  ./start-native.sh --wan $(basename "$out")
# Shell format (KEY=value). The OSMO_NODE_ID/IP/IND/NOPS fields are enough for
# the mesh (WAN table "id:ip:prefix:operators"); the rest documents the node.
OSMO_NODE_CONF_VERSION=1
OSMO_NODE_ID=${node}
OSMO_NODE_IP=${ip}
OSMO_NODE_IND=${ind}
OSMO_NODE_NOPS=${nops}
OSMO_NODE_HOSTNAME=$(hostname 2>/dev/null)
# operators: "idx:mode:pc_stp:rctx_inter[:backbone_ip]" (native, then running containers)
OSMO_NODE_OPS="${ops}"
OSMO_NODE_OP=${op}
OSMO_NODE_PC_MSC=1.${node}${op}.1
OSMO_NODE_PC_STP=1.${node}${op}.2
OSMO_NODE_PC_BSC=1.${node}${op}.3
OSMO_NODE_RCTX_INTER=$(( node * 1000 + op * 100 + 50 ))
OSMO_NODE_HUB_IP=${hub}
OSMO_NODE_HUB_PORT=2908
OSMO_NODE_MCC=${mcc}
OSMO_NODE_MNC=${mnc}
OSMO_NODE_ARFCN=${arfcn}
OSMO_NODE_N_MS=${nms}
# numbering: MSISDN = <node>00<op><ms>; from another node: <prefix><MSISDN>
OSMO_NODE_MSISDN_PFX=${pfx}
OSMO_NODE_SIP_PORT="${sip}"
OSMO_NODE_SMS_PORT="${sms}"
EOF
    mv -f "$out.tmp" "$out" && chmod 644 "$out"
}

# node_conf_import FILE: reads a card and puts its node in the WAN table
# (WAN_IP/WAN_IND/WAN_NOPS/WAN_NODE_LIST) - replaces the entry if it existed.
# Read with sed, not with `.`: a card from elsewhere executes nothing here.
node_conf_import() {
    local f="$1" id ip ind nops
    [ -r "$f" ] || { _wan_err "fiche introuvable : $f"; return 1; }
    id="$(sed -n 's/^OSMO_NODE_ID=//p' "$f" | tail -1 | tr -d '" ')"
```

```
    ip="$(sed -n 's/^OSMO_NODE_IP=//p' "$f" | tail -1 | tr -d '" ')"
    ind="$(sed -n 's/^OSMO_NODE_IND=//p' "$f" | tail -1 | tr -d '" ')"
    nops="$(sed -n 's/^OSMO_NODE_NOPS=//p' "$f" | tail -1 | tr -d '" ')"
    [[ "$id" =~ ^[1-9]$ ]] || { _wan_err "$f : OSMO_NODE_ID='$id' (attendu 1-9)"; return 1; }
    [[ "$ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] || { _wan_err "$f : OSMO_NODE_IP='$ip'"; return 1; }
    [[ "$ind" =~ ^[0-9]{2,4}$ ]] || ind="$(wan_default_ind "$id")"
    [[ "$nops" =~ ^[1-9][0-9]*$ ]] || nops=1
    if [ -z "${WAN_IP[$id]:-}" ]; then WAN_NODE_LIST+=("$id"); fi
    WAN_IP[$id]="$ip"; WAN_IND[$id]="$ind"; WAN_NOPS[$id]="$nops"
    WAN_NODE_COUNT=${#WAN_NODE_LIST[@]}
    printf '  %-4s noeud %s  %-15s indicatif %-4s %s operateur(s)  (%s)\n' "" "$id" "$ip" "$ind" "$nops"
     ↪  "$(basename "$f")"
    return 0
}

# node_conf_self NODE: makes sure THIS node is in the table (detected local
# address, default prefix, operators counted), without overwriting an entry
# the table already knows.

node_conf_self() {
    local id="$1" ip
    [ -n "${WAN_IP[$id]:-}" ] && { WAN_NOPS[$id]="$(node_conf_nops)"; return 0; }
    ip="$(wan_detect_local_ip 2>/dev/null)" || ip=""
    [ -n "$ip" ] || { _wan_err "aucune adresse locale detectee pour le noeud $id (--wan-nodes pour la donner)";
     ↪  return 1; }
    WAN_IP[$id]="$ip"; WAN_IND[$id]="$(wan_default_ind "$id")"; WAN_NOPS[$id]="$(node_conf_nops)"
    WAN_NODE_LIST+=("$id"); WAN_NODE_COUNT=${#WAN_NODE_LIST[@]}
    printf '  %-4s noeud %s  %-15s indicatif %-4s %s operateur(s)  (ce noeud)\n' "" "$id" "$ip" "${WAN_IND[$id]}"
     ↪  "${WAN_NOPS[$id]}"
}
```

2.118 /opt/GSM/osmo-operator/network/osmo-ip-plan.sh

10586 octets, 205 lignes → 205 lignes

```
#!/usr/bin/env bash
# ============================================================================
# network/osmo-ip-plan.sh - WHICH INTERFACE CARRIES THE NODE'S PRIVATE ADDRESSES
# ============================================================================
#
# WHAT IT DOES
# ------------
# The node's private addresses (192.168.<node+1>.1 and .10) are installed as
# /32 on the ONE interface that provides Internet access, i.e. the one the
# default route goes through:
#
#   1. a pattern such as [Match] Name=en* eth* would give the addresses to
#      EVERY matching card, two cards ending up with the same /32;
#   2. the pattern says NOTHING about which card really carries the network:
#      on a VM with several interfaces, "en*" matches the NAT as well as the
#      bridge, and the address could land on a card that leads nowhere while
#      the one holding the default route does not have it. A peer aiming at
#      192.168.2.10 would then find nobody although "ip addr" shows the address.
#
# "The card that has the default route" is a RUNTIME property, not a
# configuration one, which systemd-networkd cannot express. Hence this script,
# called at boot and at every link change.
#
# THE FALLBACK, AND WHY IT EXISTS
# -------------------------------
# When NO card provides Internet access - VM without network, cable unplugged,
# silent DHCP - there is no sensible place for a segment address. Instead of
# installing it nowhere, 127.0.0.66/32 goes on the loopback. Configurations
# that name a private address then ALWAYS find something local and reachable,
```

```
# the daemons start, and the bench runs self-contained instead of failing at
# bind on a missing address.
#
# DOCKER OR NATIVE
# ----------------
# The address plan differs on the two sides, and mistaking one for the other
# gives daemons that bind to addresses that do not exist here:
#   docker  the container has its address on the bridge before the daemons
#           start, and the backbone is 172.20.0.x. Nothing is touched:
#           docker does the addressing.
#   native  the address comes from DHCP, may be missing at start-up, and the
#           node's private segment is 192.168.<node+1>.x.
# "docker" = an image OF THIS REPOSITORY: OSMO_IMAGE=1 (set by the Dockerfile),
# or as a fallback /.dockerenv AND /etc/docker-entrypoint-cmd. Any other
# container (bare ubuntu:24.04) is addressed like a native machine (see
# detect_runtime_env).
#
# Usage:   osmo-ip-plan.sh [--apply|--show] [--node N]
# ------------------------------------------------------------------------------
set -uo pipefail

ACTION=apply
NODE=""
while [ $# -gt 0 ]; do
    case "$1" in
        --apply)  ACTION=apply ;;
        --show)   ACTION=show ;;
        --node)   NODE="${2:-}"; shift ;;
        --node=*) NODE="${1#*=}" ;;
        -h|--help) sed -n '2,45p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo "option inconnue : $1" >&2; exit 2 ;;
    esac
    shift
done

LOOPBACK_FALLBACK="${OSMO_PRIV_FALLBACK:-127.0.0.66}"

# ── docker or native ──────────────────────────────────────────────────────
# "docker" means HERE: an image of THIS repository, whose network is set up by
# start-docker.sh / compose (backbone 172.20.0.x). The Dockerfile sets
# ENV OSMO_IMAGE=1; as a fallback (variable lost under sudo, or in a systemd
# unit that does not inherit the container environment), the pair
# /.dockerenv + /etc/docker-entrypoint-cmd written by scripts/entrypoint.sh.
# /.dockerenv alone or the cgroup are not enough: a bare ubuntu:24.04 (the
# disposable container of the installation notebook) would pass for "docker",
# nobody would install 192.168.2.10 there, and osmo-ggsn / osmo-sgsn would
# refuse to start. Any other container is treated as a native machine.
detect_runtime_env() {
    if [ "${OSMO_IMAGE:-0}" = 1 ]; then echo docker; return; fi
    if [ -f /.dockerenv ] && [ -f /etc/docker-entrypoint-cmd ]; then echo docker; return; fi
    echo native
}

# ── the node ──────────────────────────────────────────────────────────────
resolve_node() {
    local n="${NODE:-${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}}"
    [ -n "$n" ] || n="$(awk -F= '/^OSMO_WAN_NODE=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' \
                        /etc/osmo-role 2>/dev/null)"
    [ -n "$n" ] || n="$(sed -n 's/^PLAN_NODE=//p' /etc/osmocom/radio-plan.env 2>/dev/null | tail -1)"
    case "$n" in [1-9]) ;; *) n=1 ;; esac
    printf '%s' "$n"
}

# ── THE card that provides Internet access ────────────────────────────────
# "ip route get" answers what the KERNEL would really do: the only answer that
# does not guess. A public address is tried first, then the default gateway -
```

```
# a bench cut off from the Internet usually keeps its default route.
uplink_dev() {
    local d
    d="$(ip route get 1.1.1.1 2>/dev/null | sed -n 's/.*dev \([^ ]*\).*/\1/p' | head -1)"
    [ -n "$d" ] || d="$(ip route show default 2>/dev/null | sed -n 's/.*dev \([^ ]*\).*/\1/p' | head -1)"
    # lo provides nothing: a default route through it means there is none.
    [ "$d" = "lo" ] && d=""
    printf '%s' "$d"
}

# ── WHAT THE CONFIGS REQUIRE, NOT WHAT IS ASSUMED ──────────────────────
# The private plan (192.168.<node+1>.x) is a convention; what matters is the
# address each daemon will REALLY try to bind. It is written in /etc/osmocom
# and may come from another generator than ours:
# qosmo-grgsm/run_modules/08-gabarits.sh passes op_private_ip() to
# apply_config_templates, hence 192.168.2.10, while the ISO's native recipe
# passes 127.0.0.2. Two truths, and each daemon follows its own:
#     osmo-ggsn: "gtp bind-ip 192.168.2.10" -> bind on a missing address
#     osmo-sgsn: "listen 192.168.2.10 23000" -> same
# The addresses are therefore EXTRACTED from the files instead of recomputed:
# the machine ends up carrying what its configs ask for, whoever wrote them.
#
# NOT taken: "ip dns 0/1" (servers announced to the subscriber), "remote-ip",
# "ggsn 0 remote-ip" name REMOTE addresses. Installing them locally would make
# the machine answer in place of the peer.
addrs_des_configs() {
    local d="${OSMOCOM_CFG:-/etc/osmocom}"
    [ -d "$d" ] || return 0
    {
        sed -n 's/^[[:space:]]*gtp[[:space:]]\+bind-ip[[:space:]]\+\([0-9.]*\).*/\1/p'  "$d"/osmo-ggsn.cfg
         ↪  2>/dev/null
        sed -n 's/^[[:space:]]*gtp[[:space:]]\+local-ip[[:space:]]\+\([0-9.]*\).*/\1/p' "$d"/osmo-sgsn.cfg
         ↪  2>/dev/null
        sed -n 's/^[[:space:]]*listen[[:space:]]\+\([0-9.]*\)[[:space:]]\+[0-9]*.*/\1/p' "$d"/osmo-sgsn.cfg
         ↪  2>/dev/null
        sed -n 's/^[[:space:]]*local-addr[[:space:]]\+\([0-9.]*\).*/\1/p'                 "$d"/osmo-upf.cfg
         ↪  2>/dev/null
    } | sort -u | while read -r a; do
        case "$a" in
            ''|0.0.0.0|127.*) continue ;;      # already there, or "all cards"
        esac
        printf '%s\n' "$a"
    done
}

RUNTIME="$(detect_runtime_env)"
NODE="$(resolve_node)"
PRIV_BASE=$(( NODE + 1 ))
PRIV_GW="192.168.${PRIV_BASE}.1"
PRIV_IP="192.168.${PRIV_BASE}.10"
DEV="$(uplink_dev)"

if [ "$ACTION" = show ] || [ "${OSMO_IP_PLAN_VERBOSE:-1}" = 1 ]; then
    printf 'osmo-ip-plan : runtime=%s noeud=%s prive=%s,%s carte=%s\n' \
        "$RUNTIME" "$NODE" "$PRIV_GW" "$PRIV_IP" "${DEV:-aucune}"
fi
[ "$ACTION" = show ] && exit 0

# ── DOCKER: nothing is touched ─────────────────────────────────────────
if [ "$RUNTIME" = docker ]; then
    echo "osmo-ip-plan : conteneur - l'adressage appartient a docker, rien a poser"
    exit 0
fi

# ── The addresses are first removed from EVERYWHERE ────────────────────
# Without this clean-up, changing card (cable unplugged, Wi-Fi taking over)
```

```
# would leave the old address on the old card: two interfaces claiming the
# same /32, with the kernel choosing the source by order of appearance, not
# by route.
for a in "$PRIV_GW" "$PRIV_IP" "$LOOPBACK_FALLBACK"; do
    while read -r ifn; do
        [ -n "$ifn" ] || continue
        ip addr del "${a}/32" dev "$ifn" 2>/dev/null || true
    done < <(ip -o -4 addr show 2>/dev/null | awk -v a="$a" '$4 ~ "^"a"/" {print $2}')
done

# The union of the node plan and of what the configs ask for. Duplicates are
# dropped by the sort; "ip addr add" of an address already set has no effect.
BESOINS="$(printf '%s\n%s\n%s\n' "$PRIV_GW" "$PRIV_IP" "$(addrs_des_configs)" | sed '/^$/d' | sort -u)"

if [ -n "$DEV" ]; then
    _pose=""
    for a in $BESOINS; do
        # Already carried by some card: left alone. Installing the address
        # elsewhere would give two interfaces on the same /32.
        if ip -o -4 addr show 2>/dev/null | awk '{print $4}' | grep -qx "${a}/32\|${a}/[0-9]*"; then
            continue
        fi
        ip -o -4 addr show 2>/dev/null | awk -F'[ /]+' '{print $4}' | grep -qx "$a" && continue
        ip addr add "${a}/32" dev "$DEV" 2>/dev/null && _pose="$_pose $a"
    done
    if [ -n "$_pose" ]; then
        echo "osmo-ip-plan :${_pose} pose(s) en /32 sur ${DEV} (carte qui fournit Internet)"
    else
        echo "osmo-ip-plan : rien a poser, tout est deja la (${DEV})"
    fi
else
    # -- FALLBACK: the loopback ------------------------------------------
    # No card leads anywhere. Installing 192.168.x on a random interface would
    # give an address that is present and unreachable - the worst of both
    # worlds, because everything looks in place. A loopback address is honest:
    # local, reachable, and it claims nothing.
    # On lo, a missing address prevents a daemon from starting just as on a
    # card: what the configs ask for is installed there TOO, not only the
    # fallback. The bench then runs self-contained, complete.

    ip addr add "${LOOPBACK_FALLBACK}/32" dev lo 2>/dev/null || true
    for a in $BESOINS; do
        ip -o -4 addr show 2>/dev/null | awk -F'[ /]+' '{print $4}' | grep -qx "$a" && continue
        ip addr add "${a}/32" dev lo 2>/dev/null || true
    done
    echo "osmo-ip-plan : aucune carte ne fournit Internet - ${LOOPBACK_FALLBACK}/32 + $(echo $BESOINS) sur lo"
fi
exit 0
```

2.119 /opt/GSM/osmo-operator/network/pcu-socket.sh

2733 octets, 73 lignes → 73 lignes

```
#!/bin/bash
# ============================================================================
# network/pcu-socket.sh - PCU socket harness (BTS <-> osmo-pcu)
#
# The PCU (GPRS/EGPRS) connects to ONE unix socket per BTS:
#   - the main BTS (osmo-bts-trx.cfg): default socket /tmp/pcu_bts, the one
#     run.sh gives the PCU (PCU_SOCKET);
#   - the side-car BTS (osmo-bts-trx-bts1.cfg): pcu-socket /tmp/pcu_bts2, which
#     no PCU serves. A socket with no process behind it stays on disk and
#     suggests a link that does not exist.
#
```

```
# This script changes NO config. It says who listens to what, and removes the
# orphan sockets (file present, no listening process) so the next start begins
# from a clean slate.
#
#   network/pcu-socket.sh          status (default)
#   network/pcu-socket.sh --fix    removes the orphan sockets /tmp/pcu_bts*
#
# Called by start-native.sh just before run.sh is started (--fix).
# ============================================================================
set -u

PCU_SOCKET="${PCU_SOCKET:-/tmp/pcu_bts}"
PCU_LOG="${PCU_LOG:-/var/log/osmocom/osmo-pcu.log}"
MODE=check
[ "${1:-}" = "--fix" ] && MODE=fix

# A socket is "in service" when a process listens on it (ss -xlp shows it).
listener_of() {
    ss -xlp 2>/dev/null | grep -F " $1 " | sed -n 's/.*users:((\"\([^\"]*\)\",pid=\([0-9]*\).*/\1 pid \2/p' | head
    ↪  -1
}

rc=0
echo "pcu-socket : socket attendu par le PCU = $PCU_SOCKET"

for s in /tmp/pcu_bts /tmp/pcu_bts2; do
    if [ ! -e "$s" ]; then
        printf '  %-16s absent\n' "$s"
        continue
    fi
    who="$(listener_of "$s")"
    if [ -n "$who" ]; then
        printf '  %-16s en service (%s)\n' "$s" "$who"
    elif [ "$MODE" = fix ]; then
        if rm -f "$s"; then
            printf '  %-16s orphelin supprime\n' "$s"
        else
            printf '  %-16s orphelin, suppression impossible\n' "$s"; rc=1
        fi
    else
        printf '  %-16s orphelin (aucun processus) - lancer avec --fix\n' "$s"
    fi
done

# Who serves the PCU: the last logged connection to the expected socket.
if [ -r "$PCU_LOG" ]; then
    last="$(sed 's/\x1b\[[0-9;]*m//g' "$PCU_LOG" | grep -a "PCU socket $PCU_SOCKET has been connected" | tail -1)"
    if [ -n "$last" ]; then
        echo "  PCU          connecte a $PCU_SOCKET  ($(echo "$last" | awk '{print $4, $5, $6}'))"
    else
        echo "  PCU          pas de connexion journalisee a $PCU_SOCKET"
    fi
fi

# Config: which BTSs declare a PCU socket, and which one.

for c in /etc/osmocom/osmo-bts*.cfg; do
    [ -r "$c" ] || continue
    line="$(grep -E '^[[:space:]]*pcu-socket[[:space:]]' "$c" | head -1 | sed 's/^[[:space:]]*//')"
    printf '  %-28s %s\n' "$(basename "$c")" "${line:-(defaut : /tmp/pcu_bts)}"
done

exit $rc
```

2.120 /opt/GSM/osmo-operator/network/set-node-id.sh

15917 octets, 318 lignes → 318 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════
# network/set-node-id.sh - gives THIS machine its SS7 node identity
#
# A single ISO for every node: the number is chosen at start-up, not at build
# time. This script rewrites what designates the node in the Osmocom configs,
# i.e. its POINT CODES and its routing contexts.
#
#   local plan (one machine)   1.<op>.<role>          op×100 + {10,20,30,50}
#   WAN plan   (N machines)    1.<node><op>.<role>     node×1000 + op×100 + ...
#
# Why the local plan cannot stay: the formula gives the same result on every
# machine. Three nodes attached to the same inter-STP would all present
# 1.11.2. A point code is an ADDRESS - two devices sharing one is not a name
# clash, it is wrong routing, and the hub has no way to notice.
#
# Three files carry this identity, and they must agree:
#   osmo-stp.cfg   its own PC, its routing-key, its ASP towards the hub
#   osmo-msc.cfg   its PC, its routing-key, and the PC of the BSC it calls
#   osmo-bsc.cfg   its PC, its routing-key, and the PC of the MSC it calls
# Missing one gives a core that starts and calls that do not complete.
#
# Idempotent: it reads the current state and replaces it. --node 2 after
# --node 1 works, there is no state to clean between the two.
#
# Usage:
#   sudo network/set-node-id.sh --node 3 [--op 1] [--hub-ip 192.168.1.49]
#   sudo network/set-node-id.sh --show          current state, changes nothing
#   sudo network/set-node-id.sh --node 3 --local   local plan (cancels the WAN plan)
# ═══════════════════════════════════════════════════════════════════════
set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
# wan_detect_local_ip lives there: the same address selection rule as the one
# start-docker.sh and start-native.sh apply, so that both always designate
# the same card.
# shellcheck source=network/wan-nodes.sh
. "$SCRIPT_DIR/wan-nodes.sh"

NODE=""
OP=1
HUB_IP=""
MODE=""            # docker | native - the addressing differs
LOCAL_IP=""        # source address of the ASP; "auto" = detected
CONF_DIR="${CONF_DIR:-/etc/osmocom}"
SHOW=0
LOCAL_PLAN=0
DRY=0

while [ $# -gt 0 ]; do
    case "$1" in
        --node)     NODE="${2:-}"; shift ;;
        --node=*)   NODE="${1#*=}" ;;
        --op)       OP="${2:-1}"; shift ;;
        --op=*)     OP="${1#*=}" ;;
        --local-ip)   LOCAL_IP="${2:-}"; shift ;;
        --local-ip=*) LOCAL_IP="${1#*=}" ;;
        --hub-ip)   HUB_IP="${2:-}"; shift ;;
        --hub-ip=*) HUB_IP="${1#*=}" ;;
```

```
        --conf-dir)   CONF_DIR="${2:-}"; shift ;;
        --conf-dir=*) CONF_DIR="${1#*=}" ;;
        --mode)     MODE="${2:-}"; shift ;;
        --mode=*)   MODE="${1#*=}" ;;
        --docker)   MODE="docker" ;;
        --native)   MODE="native" ;;
        --show)     SHOW=1 ;;
        --local)    LOCAL_PLAN=1 ;;
        --dry-run)  DRY=1 ;;
        -h|--help)  sed -n '2,32p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; exit 2 ;;
    esac
    shift
done

STP="$CONF_DIR/osmo-stp.cfg"
MSC="$CONF_DIR/osmo-msc.cfg"
BSC="$CONF_DIR/osmo-bsc.cfg"

# ── Reading the identity in place ──────────────────────────────────
# The first point-code of a file is THE DAEMON'S; the following ones, inside
# an "sccp-address" block, designate its peers. This distinction structures
# the whole rewrite.
own_pc() { awk '/^cs7 instance/{c=1} c && /^ *point-code /{print $2; exit}' "$1" 2>/dev/null; }

show_state() {
    local f n
    printf '  %-16s %-10s %-12s %-16s %s\n' FICHIER "POINT CODE" "ROUTING-KEY" "ASP INTER-STP" "SOURCE"
    for f in "$STP" "$MSC" "$BSC"; do
        [ -r "$f" ] || { printf '  %-16s %s\n' "$(basename "$f")" "absent"; continue; }
        local pc rk asp
        pc="$(own_pc "$f")"
        rk="$(awk '/^ *routing-key /{print $2; exit}' "$f")"
        asp="$(awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}' "$f")"
        local src
        src="$(awk '/asp asp-to-inter/{f=1} f&&/local-ip/{print $2; exit}' "$f")"
        printf '  %-16s %-10s %-12s %-16s %s\n' "$(basename "$f")" "${pc:--}" "${rk:--}" "${asp:--}" "${src:--}"
    done
}

if [ "$SHOW" = "1" ]; then
    [ -n "$MODE" ] || { [ -f /.dockerenv ] && MODE=docker || MODE=native; }
    echo -e "${BOLD}Identite SS7 de cette machine${NC}  (mode ${CYAN}${MODE}${NC})"
    show_state
    exit 0
fi

[[ "$NODE" =~ ^[1-9]$ ]] || { echo -e "${RED}--node : un chiffre de 1 a 9${NC}" >&2; exit 2; }
[[ "$OP"   =~ ^[1-9]$ ]] || { echo -e "${RED}--op : un chiffre de 1 a 9${NC}" >&2; exit 2; }
[ "$(id -u)" -eq 0 ] || { echo -e "${RED}Root requis (ecriture dans ${CONF_DIR}).${NC}" >&2; exit 1; }

for f in "$STP" "$MSC" "$BSC"; do
    [ -w "$f" ] || { echo -e "${RED}$f introuvable ou non modifiable${NC}" >&2; exit 1; }
done

# ── The new plan ──────────────────────────────────────────────────
if [ "$LOCAL_PLAN" = "1" ]; then
    MID="$OP"                       # 1.<op>.<role> - plan of an isolated machine
    RBASE=$(( OP * 100 ))
else
    MID="${NODE}${OP}"              # 1.<node><op>.<role> - WAN plan
    RBASE=$(( NODE * 1000 + OP * 100 ))
fi
PC_MSC="1.${MID}.1"; PC_STP="1.${MID}.2"; PC_BSC="1.${MID}.3"
RCTX_MSC=$(( RBASE + 10 )); RCTX_STP=$(( RBASE + 20 ))
RCTX_BSC=$(( RBASE + 30 )); RCTX_INTER=$(( RBASE + 50 ))
```

```
# ── DOCKER OR VM: two address plans, and the default of one is wrong in the
# other ──────────────────────────────────────────────────────────────────
#   docker: the hub is the osmo-inter-stp container, 172.20.0.10, on the
#           gsm-inter network. The node already has its IP when osmo-stp starts.
#   VM/ISO: the hub is a separate machine, on the LAN in bridged mode. The
#           node's address comes from DHCP and is not necessarily up at
#           start-up.
# A wrong default gives an ASP that never attaches, and the log only says
# "connection refused" - without telling that the target address exists
# nowhere on that network.
#
# The VirtualBox host-only address (192.168.56.1) is WRONG as soon as the VMs
# are bridged: the address exists on no segment, and the ASP aims at it
# without ever reaching anything. The address is therefore ASKED instead of
# guessed, with the bench hub as default. An EMPTY answer is accepted when it
# is not known yet: in that case the remote-ip already present in the file is
# left untouched - better to write nothing than to write a wrong address.
#
if [ -z "$MODE" ]; then
    if [ -f /.dockerenv ]; then MODE="docker"; else MODE="native"; fi
fi

: "${HUB_IP_DEFAULT:=172.20.0.10}"    # the bench hub: osmo-inter-stp, docker backbone

if [ -z "$HUB_IP" ]; then
    if [ -r /etc/osmo-role ]; then
        HUB_IP="$(awk -F= '/^OSMO_HUB_IP=/{print $2}' /etc/osmo-role)"
    fi
fi
if [ -z "$HUB_IP" ]; then
    case "$MODE" in
        docker) HUB_IP="${INTER_STP_IP:-172.20.0.10}" ;;
        *)
            # Interactive: ask. Otherwise (ISO, service, CI) take the bench
            # default - a non-interactive script cannot answer, and blocking
            # on a read would only fail later.
            if [ -t 0 ] && [ "$DRY" != "1" ] && command -v _wan_ask >/dev/null 2>&1; then
                HUB_IP="$(_wan_ask "Inter-STP" \
                    "IP de l'inter-STP (vide si inconnue) :" "$HUB_IP_DEFAULT")" || exit 1
            else
                HUB_IP="$HUB_IP_DEFAULT"
            fi ;;
    esac
fi

# Empty is accepted: the remote-ip is not rewritten. The rest (point codes,
# routing-keys, local-ip) keeps its meaning and is applied anyway.
SET_HUB_IP=1
if [ -z "$HUB_IP" ]; then
    SET_HUB_IP=0
    echo -e "  ${YELLOW}Inter-STP inconnu : remote-ip laisse tel quel dans les fichiers.${NC}"
    echo -e "  ${YELLOW}Renseignez-le plus tard : --hub-ip ADRESSE, ou set_stp_ip.sh${NC}"
fi

# ── SOURCE address of the ASP towards the hub ──────────────────────────
# In docker it is left alone: the container has its address before the daemon
# starts, and an explicit source keeps SCTP from announcing several addresses
# (multi-homing) to a peer that expects only one.
#
# In a VM, the address actually detected on the VirtualBox segment is
# INJECTED rather than 0.0.0.0. The difference is not cosmetic: with 0.0.0.0,
# SCTP announces ALL local addresses in its INIT - including the 10.0.2.15 NAT
# and the 172.20.x aliases the ISO sets. The hub then tries to reach addresses
# that, seen from it, lead nowhere. A single correct address beats a
# multi-homing with three dead branches out of four.
```

```
#
# Falls back to 0.0.0.0 when no address is detectable - binding there beats
# binding to a missing address, which prevents osmo-stp from starting.
SET_LOCAL_IP=0
ASP_LOCAL_IP=""
if [ "$MODE" != "docker" ]; then
    SET_LOCAL_IP=1
    case "$LOCAL_IP" in
        "" | auto)
            # ── The address that ROUTES to the hub, before any heuristic ────
            # The kernel knows exactly which source it will use to reach the
            # hub: that is what "ip route get" answers. Any other method
            # guesses - and guesses wrong as soon as the machine carries
            # several private addresses. A bench VM has three (192.168.1.2 the
            # bridge, 192.168.2.1 and .2.10 the ISO's private plan): a
            # heuristic may pick 192.168.2.1, an address the hub cannot reach
            # back; the ASP would then go nowhere and the log would only say
            # "connect failed (-110)" - a timeout, with no hint about the
            # faulty source.
            if [ -n "$HUB_IP" ]; then
                ASP_LOCAL_IP="$(ip route get "$HUB_IP" 2>/dev/null \
                                | sed -n 's/.*src \([0-9.]*\).*/\1/p' | head -1)"
                [ -n "$ASP_LOCAL_IP" ] && LOCAL_IP_SRC="route vers le hub"
            fi
            if [ -n "$ASP_LOCAL_IP" ]; then
                :
            elif ! command -v wan_detect_local_ip >/dev/null 2>&1; then
                echo -e "${RED}wan_detect_local_ip introuvable - network/wan-nodes.sh non charge${NC}" >&2
                exit 1
            else
                ASP_LOCAL_IP="$(wan_detect_local_ip || true)"
            fi
            if [ -n "$ASP_LOCAL_IP" ]; then
                [ -n "${LOCAL_IP_SRC:-}" ] || LOCAL_IP_SRC="detectee sur le segment"
            else
                ASP_LOCAL_IP="0.0.0.0"
                LOCAL_IP_SRC="aucune adresse detectee - repli"
            fi ;;
        *)  ASP_LOCAL_IP="$LOCAL_IP"; LOCAL_IP_SRC="imposee par --local-ip" ;;
    esac
fi

echo -e "${BOLD}Noeud ${NODE} - operateur ${OP}${NC}   (mode ${CYAN}${MODE}${NC})"
echo -e "  STP ${CYAN}${PC_STP}${NC}   MSC ${CYAN}${PC_MSC}${NC}   BSC ${CYAN}${PC_BSC}${NC}"
echo -e "  routing-key : stp ${RCTX_STP} · msc ${RCTX_MSC} · bsc ${RCTX_BSC} · inter ${RCTX_INTER}"
if [ "$LOCAL_PLAN" = "1" ] && [ -z "$HUB_IP" ]; then
    echo -e "  ${YELLOW}plan LOCAL sans hub : point codes 1.<op>.<role>, ASP inchange${NC}"
else
    [ "$LOCAL_PLAN" = "1" ] && echo -e "  ${CYAN}plan LOCAL${NC} (point codes 1.<op>.<role>) avec hub sur cette
     ↪  machine"
    echo -e "  ASP inter-STP → ${CYAN}${HUB_IP}${NC}$([ "$SET_LOCAL_IP" = "1" ] && echo "  depuis
     ↪  ${CYAN}${ASP_LOCAL_IP}${NC} (${LOCAL_IP_SRC})" || echo "  (source inchangee)")"
fi
echo ""

# ── Rewrite ──────────────────────────────────────────────────────────────
# `own` = the daemon's point code (at cs7 instance level).
# `peer` = the one it declares in its sccp-address blocks.
# The distinction is made on the current block, not on the line number: a
# file that gains a directive does not shift the rewrite.
rewrite() {  # $1=file  $2=own  $3=peer  $4=rctx  $5=1 if inter-STP asp block
    local f="$1" own="$2" peer="$3" rctx="$4" do_asp="$5"
    local tmp; tmp="$(mktemp)"
    awk -v own="$own" -v peer="$peer" -v rctx="$rctx" -v do_asp="$do_asp" -v hub="$HUB_IP" \
        -v set_hub="$SET_HUB_IP" \
        -v set_local="$SET_LOCAL_IP" -v localip="${ASP_LOCAL_IP:-0.0.0.0}" '
```

```
        /^cs7 instance/            { in_cs7=1 }
        /^ *sccp-address /         { sccp=1 }
        /^ *(as|asp|listen|route-table|network-indicator|xua) / { sccp=0 }
        /^!/                       { sccp=0 }
        /^ *asp asp-to-inter/      { in_asp=1; print; next }
        in_asp && /^ *(as|listen|route-table|!)/ { in_asp=0 }

        in_asp && do_asp && set_hub && /^ *remote-ip /   { sub(/remote-ip .*/, "remote-ip " hub); print; next }
        # local-ip of the ASP: the SOURCE address of the SCTP towards the hub.
        # It is 127.0.0.1 in the image (everything local); left as is, the
        # connection would leave the loopback towards a segment address -
        # it would never complete, and the only symptom would be an ASP DOWN.
        # 0.0.0.0: the kernel picks the source according to the route.
        in_asp && do_asp && set_local && /^ *local-ip / { sub(/local-ip.*/, "local-ip " localip); print; next }
        in_asp && do_asp && /^ *shutdown[ \t]*$/ { sub(/shutdown/, "no shutdown"); print; next }

        /^ *point-code / {
            if (sccp && peer != "") { sub(/point-code .*/, "point-code " peer) }
            else if (in_cs7)        { sub(/point-code .*/, "point-code " own) }
            print; next
        }
        /^ *routing-key / { sub(/routing-key .*/, "routing-key " rctx " " own); print; next }
        { print }
    ' "$f" > "$tmp"

    if [ "$DRY" = "1" ]; then
        # diff returns 1 when the files differ: the normal case here, and
        # pipefail would abort the script at the first rewrite.
        echo "  [dry-run] $f"; { diff -u "$f" "$tmp" || true; } | sed -n '4,30p' | sed 's/^/    /'; rm -f "$tmp"
    else
        # cat rather than mv: on the ISO as in a container, these files may be
        # mount points, where rename() fails.
        cat "$tmp" > "$f"; rm -f "$tmp"
        echo -e "  ${GREEN}✓${NC} $(basename "$f")"
    fi
}

# ── THE POINT CODE PLAN AND THE HUB ARE TWO DIFFERENT THINGS ──────────────
# The multi-operator bench carries its inter-STP in a CONTAINER, on the same
# machine, reachable at 127.0.0.1:2908 (SCTP port published by
# start-docker.sh). Its point code plan is local - 1.<op>.<role> - AND there
# is a hub. The asp-to-inter block is therefore handled as soon as a hub is
# CONFIGURED, whatever the plan; otherwise `--local --hub-ip 127.0.0.1` would
# leave the template's local-ip, 172.20.0.11 - the FIRST CONTAINER's address,
# which does not exist on the host - and the ASP would stay ASP_DOWN with
# nothing but a timeout in the log.

_DO_ASP=0
[ -n "$HUB_IP" ] && _DO_ASP=1
[ "$LOCAL_PLAN" = "1" ] || _DO_ASP=1
rewrite "$STP" "$PC_STP" ""         "$RCTX_INTER" "$_DO_ASP"
rewrite "$MSC" "$PC_MSC" "$PC_BSC" "$RCTX_MSC"   0
rewrite "$BSC" "$PC_BSC" "$PC_MSC" "$RCTX_BSC"   0

if [ "$DRY" != "1" ]; then
    echo ""
    show_state
    echo ""
    echo -e "  ${YELLOW}Les demons deja lances gardent l'ancienne identite :${NC} relancez la pile."
fi
```

2.121 /opt/GSM/osmo-operator/network/set_ip_vm.sh

22626 octets, 439 lignes → 439 lignes

```
#!/bin/bash
# ============================================================================
# network/set_ip_vm.sh - makes a VM able to reach the docker containers
#
# The problem in one line: the ISO sets docker-plan addresses on the NIC
# (172.20.0.1/24, 172.20.0.11/24, 172.20.1.10/24), meant for a single machine
# running everything. On a bench where the containers live on a separate
# HOST, these addresses lie:
#
#   1. the /24 pretends 172.20.0.0/24 is on the local link. The VM ARPs for
#      it on the LAN instead of routing it, and
#          ip route add 172.20.0.0/24 via <host>
#      is refused with "RTNETLINK answers: File exists".
#   2. 172.20.0.11 is the address of the FIRST operator CONTAINER. The VM
#      answers itself: the most confusing failure of all, since everything
#      "works" locally and nothing ever reaches the container.
#
# This script removes what lies, keeps what serves, and installs the route.
# Everything is derived at run time - interface, present addresses, networks -
# nothing is hard-coded: the same command applies to the three bench VMs.
#
# WHAT IT DOES NOT DO: break a running stack. An address a daemon listens on
# is reported and KEPT, unless --force. An incomplete routing beats an
# osmo-stp that no longer restarts.
#
# Usage:
#   sudo network/set_ip_vm.sh --via 192.168.1.101      the host carrying docker
#   sudo network/set_ip_vm.sh                          --via asked, or re-read
#   sudo network/set_ip_vm.sh --status                 status, changes nothing
#   sudo network/set_ip_vm.sh --undo                   removes the route
#
#   --via ADDRESS      docker host (router towards the backbone)
#   --docker-net CIDR  container backbone (default 172.20.0.0/24)
#   --iface NAME       interface (default: the one of the default route)
#   --keep ADDRESS     address NOT to remove (repeatable)
#   --force            removes even the addresses a daemon listens on
#   --hub-ip ADDRESS   writes the inter-STP to aim at in osmo-stp.cfg, lifts
#                      the ASP's shutdown, then aligns the source on the route
#                      that leads there. Without it, only the source is aligned.
#   --no-stp           does not touch osmo-stp.cfg (ASP source)
#   --defaut           applies the bench configuration without asking.
#                      The docker host is NOT assumed: it is searched on the
#                      network read from the interface - the only machine that
#                      can reach the container backbone. --via fixes it.
#   --persist          replayed at boot (systemd unit)
#   --dry-run          shows, does not apply
# ============================================================================
set -uo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

DOCKER_NET="172.20.0.0/24"
VIA=""
IFACE=""
FORCE=0
NO_STP=0
# --hub-ip: the inter-STP this ASP must aim at. Without it, the source is only
# ALIGNED on the remote-ip already written - worthless when that remote-ip is
# still the template default (127.0.0.1): it would validate a configuration
# that will never reach anyone. With it, the destination is WRITTEN, then the
# source that leads there.
HUB_IP=""
USE_DEFAULTS=0
# ── The bench configuration, in a word ──────────────────────────────────────
# --defaut applies what holds on all our VMs without asking: the host carrying
```

```
# docker, the container backbone, and persistence at boot (the live system
# has none, the route would be lost at reboot).
# Each value can still be overridden: --defaut --via X wins over the default.
DEFAULT_VIA=""                  # discovered: see discover_docker_host()
DEFAULT_DOCKER_NET="172.20.0.0/24"
DRY=0
PERSIST=0
ACTION="apply"
KEEP=()
STATE=/etc/osmo-docker-host          # remembers --via, so it is typed only once

while [ $# -gt 0 ]; do
    case "$1" in
        --via)          VIA="${2:-}"; shift ;;
        --via=*)        VIA="${1#*=}" ;;
        --docker-net)   DOCKER_NET="${2:-}"; shift ;;
        --docker-net=*) DOCKER_NET="${1#*=}" ;;
        --iface)        IFACE="${2:-}"; shift ;;
        --iface=*)      IFACE="${1#*=}" ;;
        --keep)         KEEP+=("${2:-}"); shift ;;
        --keep=*)       KEEP+=("${1#*=}") ;;
        --force)        FORCE=1 ;;
        --hub-ip)       HUB_IP="${2:-}"; shift ;;
        --hub-ip=*)     HUB_IP="${1#*=}" ;;
        --no-stp)       NO_STP=1 ;;
        --defaut|--default) USE_DEFAULTS=1 ;;
        --persist)      PERSIST=1 ;;
        --dry-run)      DRY=1 ;;
        --status)       ACTION="status" ;;
        --undo)         ACTION="undo" ;;
        -h|--help)      sed -n '2,40p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; exit 2 ;;
    esac
    shift
done

run() { if [ "$DRY" = 1 ]; then echo "    [dry-run] $*"; else "$@"; fi; }

# ── Finding the docker host, without naming it ──────────────────────
# A hard-coded address does not survive the bench: the host switching from
# ethernet to wifi changes its address, and every VM would route towards a
# machine that no longer exists - "connect failed" everywhere, with no
# configuration having changed.
#
# It is therefore searched by what it DOES, not by its name: the only machine
# on the LAN that can reach the container backbone. For each live machine the
# route is installed on trial and the first container is probed; the one that
# answers is the host. The route is removed when the trial fails - nothing
# remains of an unsuccessful attempt.
discover_docker_host() {
    local prefix probe live c saved
    # This function INSTALLS routes to find the right one, without going
    # through run(): under --dry-run it is skipped, so the routing table of
    # the machine stays untouched, as the mode promises.
    if [ "$DRY" = 1 ]; then
        echo -e "    [dry-run] decouverte sautee : elle pose des routes a l'essai - donnez --via" >&2
        return 1
    fi
    # Without root, "ip route replace" would fail silently for every
    # candidate: the loop would run to the end, VIA would come out empty and
    # --defaut would exit on "--via manquant" - a message naming the wrong
    # cause. The general root check comes later, too late for this one.
    if [ "$(id -u)" -ne 0 ]; then
        echo -e "    decouverte impossible sans root (elle pose des routes a l'essai) - donnez --via" >&2
        return 1
    fi
```

```
    # The network is read from the interface, never assumed.
    [ -n "$LAN_IP" ] || return 1
    prefix="${LAN_IP%.*}"
    probe="$(echo "$DOCKER_NET" | cut -d. -f1-3).11"   # the first container

    echo -e "  ${CYAN}Recherche de l'hote docker sur ${prefix}.0/24...${NC}" >&2
    # 1. who is alive? (in parallel, one second in total)
    live="$(for c in $(seq 1 254); do
                [ "${prefix}.${c}" = "$LAN_IP" ] && continue
                ping -c1 -W1 "${prefix}.${c}" >/dev/null 2>&1 && echo "${prefix}.${c}" &
            done; wait)"
    [ -n "$live" ] || return 1

    # 2. which one routes to the containers?
    saved="$(ip -4 route show "$DOCKER_NET" 2>/dev/null | head -1)"
    for c in $live; do
        ip route replace "$DOCKER_NET" via "$c" 2>/dev/null || continue
        if ping -c1 -W1 "$probe" >/dev/null 2>&1; then
            echo -e "  ${GREEN}✓${NC} hote docker : ${CYAN}${c}${NC} (il joint ${probe})" >&2
            printf '%s' "$c"; return 0
        fi
    done
    # nothing found: restore what was there
    ip route del "$DOCKER_NET" 2>/dev/null || true
    [ -n "$saved" ] && ip route add $saved 2>/dev/null || true
    return 1
}

# ── The interface: the one carrying the default route ──────────────
[ -n "$IFACE" ] || IFACE="$(ip route show default 2>/dev/null | awk '/^default/{print $5; exit}')"
[ -n "$IFACE" ] || { echo -e "${RED}Aucune interface par defaut - precisez --iface${NC}" >&2; exit 1; }

# ── The machine's address ON THE LAN ───────────────────────────────
# Taken from the default route, not from the address list: the ISO also sets
# the private plan (192.168.<n>.1/32 and .10/32), so a simple "the first one
# that is not 172.20" would pick 192.168.2.1 - a local address, on no segment -
# and the host discovery would sweep the wrong network. The source of the
# default route is, by construction, the one that goes out.
LAN_IP="$(ip route show default 2>/dev/null | awk '/^default/{print $9; exit}')"
[ -n "$LAN_IP" ] || LAN_IP="$(ip -4 -o addr show dev "$IFACE" scope global 2>/dev/null \
          | awk '$0 !~ /\/32/ {print $4}' | cut -d/ -f1 | grep -vE '^172\.20\.' | head -1)"

# ── --defaut: the bench table, without asking ──────────────────────
# This block comes AFTER the interface and LAN address are read, and must stay
# there: discover_docker_host sweeps the network read from the interface.
# Placed earlier, LAN_IP would not exist yet and set -u would kill the
# substitution's subshell - the search would never start, VIA would stay
# empty, and --defaut would exit on "--via manquant" (code 2) although its
# role is precisely to have nothing to ask.
if [ "$USE_DEFAULTS" = "1" ]; then
    [ -n "$VIA" ] || VIA="$(awk -F= '/^OSMO_DOCKER_HOST=/{print $2}' "$STATE" 2>/dev/null | tr -d ' \r')"
    [ -n "$VIA" ] || VIA="$(discover_docker_host || true)"
    [ "$DOCKER_NET" = "172.20.0.0/24" ] && DOCKER_NET="$DEFAULT_DOCKER_NET"
    PERSIST=1
    echo -e "${CYAN}  --defaut : hote docker ${VIA}, backbone ${DOCKER_NET}, rejoue au boot${NC}"
fi

# The docker plan prefix, to recognise the addresses to remove:
# 172.20.0.0/24 -> "172.20." (the whole /16, since the ISO also sets 172.20.1.x)
DOCKER_PREFIX="$(echo "$DOCKER_NET" | cut -d. -f1-2)."

echo -e "${CYAN}${BOLD}== Adressage VM vers les conteneurs docker ==${NC}"
echo -e "  interface : ${CYAN}${IFACE}${NC}   adresse LAN : ${CYAN}${LAN_IP:-?}${NC}"
echo -e "  backbone  : ${CYAN}${DOCKER_NET}${NC}"

# ── What is lying around ───────────────────────────────────────────
```

```
mapfile -t STALE_ADDRS < <(ip -4 -o addr show dev "$IFACE" 2>/dev/null \
    | awk '{print $4}' | grep "^${DOCKER_PREFIX}" || true)
mapfile -t STALE_ROUTES < <(ip -4 route show dev "$IFACE" 2>/dev/null \
    | awk '$1 ~ /^'"${DOCKER_PREFIX//./\\.}"'/ && /scope link/ {print $1}' || true)

# ── --status ──────────────────────────────────────────────────────────
if [ "$ACTION" = "status" ]; then
    echo ""
    if [ "${#STALE_ADDRS[@]}" -eq 0 ]; then
        echo -e "  ${GREEN}✓${NC} aucune adresse ${DOCKER_PREFIX}x sur ${IFACE}"
    else
        echo -e "  ${YELLOW}adresses du plan docker encore posees :${NC}"
        for a in "${STALE_ADDRS[@]}"; do echo "    $a"; done
    fi
    cur="$(ip -4 route show "$DOCKER_NET" 2>/dev/null | head -1)"
    if echo "$cur" | grep -q " via "; then
        echo -e "  ${GREEN}✓${NC} route : ${cur}"
    elif [ -n "$cur" ]; then
        echo -e "  ${RED}✗${NC} route CONNECTEE (pas de via) : ${cur}"
        echo -e "     tant qu'elle existe, les conteneurs sont injoignables."
    else
        echo -e "  ${YELLOW}–${NC} aucune route vers ${DOCKER_NET}"
    fi
    exit 0
fi

[ "$DRY" = 1 ] || [ "$(id -u)" -eq 0 ] \
    || { echo -e "${RED}Root requis.${NC}" >&2; exit 1; }

# ── --undo ────────────────────────────────────────────────────────────
if [ "$ACTION" = "undo" ]; then
    run ip route del "$DOCKER_NET" 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} route retiree" \
        || echo -e "  ${YELLOW}–${NC} pas de route a retirer"
    rm -f /etc/systemd/system/osmo-docker-route.service 2>/dev/null && \
        systemctl disable osmo-docker-route.service >/dev/null 2>&1
    exit 0
fi

# ── The docker host ───────────────────────────────────────────────────
# Remembered: typed only once per machine. Without a terminal and without a
# saved value, stop rather than invent a gateway.
[ -n "$VIA" ] || VIA="$(awk -F= '/^OSMO_DOCKER_HOST=/{print $2}' "$STATE" 2>/dev/null | tr -d ' \r')"
if [ -z "$VIA" ] && [ -t 0 ]; then
    def=""
    [ -n "$LAN_IP" ] && def="${LAN_IP%.*}.101"
    printf '  Adresse de l hote qui porte docker [%s] : ' "${def:-?}"
    read -r VIA || VIA=""
    [ -n "$VIA" ] || VIA="$def"
fi
if [ -z "$VIA" ]; then
    echo -e "  ${RED}✗ --via manquant : j'ignore par ou joindre ${DOCKER_NET}${NC}" >&2
    exit 2
fi
if ! [[ "$VIA" =~ ^[0-9]{1,3}(\.[0-9]{1,3}){3}$ ]]; then
    echo -e "  ${RED}✗ '${VIA}' n'est pas une IPv4${NC}" >&2; exit 2
fi
echo -e "  hote docker : ${CYAN}${VIA}${NC}"

# ── This script is for the VMs. Not for the host carrying docker ──────
# Run on the host, it would install a route towards ITSELF, overriding the
# connected route of the docker bridge: immediate "Destination Host
# Unreachable" towards its own containers - although they run fine.
#
# Two signs give it away, and one is enough:
#   - --via is a local address;
```

```
#   - a bridge already carries the container network.
#
# The two ways of "carrying" that network must not be confused:
#   a docker BRIDGE (br-xxxx, docker0)   -> this is the host, stop;
#   an ordinary card (enp0s3...)         -> these are precisely the stale ISO
#                                           aliases, and that is our job.
# Refusing both would block on the very VM this script is meant to repair.
#
_self=0
ip -4 -o addr show scope global 2>/dev/null | awk '{print $4}' | cut -d/ -f1 \
    | grep -qx "$VIA" && _self=1
_dev="$(ip -4 -o route show "$DOCKER_NET" 2>/dev/null | awk '/proto kernel/{print $3; exit}')"
_bridge=""
case "$_dev" in br-*|docker*|virbr*) _bridge="$_dev" ;; esac
if [ "$_self" = 1 ] || [ -n "$_bridge" ]; then
    echo ""
    echo -e "  ${RED}✗ Cette machine EST l'hote docker - rien a faire ici.${NC}" >&2
    [ "$_self" = 1 ] && echo -e "     ${VIA} est une de ses propres adresses." >&2
    [ -n "$_bridge" ] && echo -e "     ${DOCKER_NET} est deja porte par ${_bridge}." >&2
    echo -e "  ${YELLOW}Ce script se lance SUR LES VM${NC}, pour leur apprendre le chemin" >&2
    echo -e "  vers les conteneurs. Cote hote, c'est l'autre script :" >&2
    echo -e "     ${CYAN}sudo network/setup-docker-lan-route.sh --push \"<ip des VM>\"${NC}" >&2
    exit 2
fi
echo ""

# ── 1. Remove what lies ──────────────────────────────────────────────
# An address a daemon LISTENS on is not removed: deleting it would bring down
# the service bound to it, and the problem would become worse than the one
# being fixed.
for a in ${STALE_ADDRS[@]+"${STALE_ADDRS[@]}"}; do
    ip_only="${a%%/*}"
    skip=0
    for k in ${KEEP[@]+"${KEEP[@]}"}; do [ "$k" = "$ip_only" ] && skip=1; done
    if [ "$skip" = 1 ]; then
        echo -e "  ${CYAN}–${NC} ${a} conservee (--keep)"; continue
    fi
    if [ "$FORCE" != 1 ] && ss -Hlnp 2>/dev/null | grep -q "[^0-9]${ip_only}:"; then
        echo -e "  ${YELLOW}⚠${NC} ${a} CONSERVEE : un demon y ecoute"
        echo -e "     (relancez la pile puis --force, ou --keep pour l'assumer)"
        continue
    fi
    # An address a CONFIG names, even if nothing listens yet. That is the
    # idle case: the stack is stopped, no socket open, and yet osmo-ggsn.cfg
    # says "gtp bind-ip 172.20.1.10". Removing it gives, at the next start, a
    # service that refuses to start on an address "not found locally" - far
    # from this script.
    if [ "$FORCE" != 1 ]; then
        _ref="$(grep -rlF "$ip_only" /etc/osmocom /etc/asterisk 2>/dev/null | head -3)"
        if [ -n "$_ref" ]; then
            echo -e "  ${YELLOW}⚠${NC} ${a} CONSERVEE : nommee dans une configuration"
            printf '     %s\n' $_ref
            echo -e "     (corrigez la config - 127.0.0.1 convient en natif - puis --force)"
            continue
        fi
    fi
    run ip addr del "$a" dev "$IFACE" 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} ${a} retiree" \
        || echo -e "  ${YELLOW}–${NC} ${a} deja absente"
done

for r in ${STALE_ROUTES[@]+"${STALE_ROUTES[@]}"}; do
    run ip route del "$r" dev "$IFACE" 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} route connectee ${r} retiree"
done
```

```
# ── 2. Install the route ──────────────────────────────────────────────
# 'replace' rather than 'add': idempotent, and it does not trip on a leftover.
if run ip route replace "$DOCKER_NET" via "$VIA" dev "$IFACE"; then
    echo -e "  ${GREEN}✓${NC} ${DOCKER_NET} via ${CYAN}${VIA}${NC}"
else
    echo -e "  ${RED}✗${NC} route refusee - une adresse du plan docker subsiste ?"
    echo -e "     ${CYAN}ip -4 addr show dev ${IFACE} | grep ${DOCKER_PREFIX}${NC}"
    exit 1
fi

# ── 2bis. The ASP must leave from the address that ROUTES ─────────────
# Installing the route is not enough if osmo-stp binds to another address. A
# bench VM carries several - the bridge, and the ISO's private plan
# (192.168.<n>.1 and .10) - and auto-detection may pick a private one. The
# hub then cannot answer: the association never completes and the log only
# says "connect failed (-110)", a timeout, without naming the faulty source.
#
# The ASP source is therefore aligned on what the kernel will really use to
# reach the hub - the same rule as network/set-node-id.sh.
STP_CFG="${STP_CFG:-/etc/osmocom/osmo-stp.cfg}"
if [ "${NO_STP:-0}" != "1" ] && [ -w "$STP_CFG" ]; then
    # --hub-ip is authoritative; otherwise read what the file already declares.
    if [ -n "$HUB_IP" ]; then
        _hub="$HUB_IP"
        _cur_hub="$(awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}' "$STP_CFG")"
        if [ "$_cur_hub" != "$_hub" ]; then
            if [ "$DRY" = 1 ]; then
                echo -e "  [dry-run] ASP remote-ip ${_cur_hub:-?} -> ${_hub}"
            else
                sed -i "/asp asp-to-inter/,/^ *as / s/^\( *remote-ip \).*/\1${_hub}/" "$STP_CFG"
                # An ASP in shutdown does not attach, even well addressed: lift
                # it, otherwise the fix has no visible effect.
                sed -i "/asp asp-to-inter/,/^ *as / s/^\( *\)shutdown[[:space:]]*$/\1no shutdown/" "$STP_CFG"
                echo -e "  ${GREEN}✓${NC} ASP : hub ${CYAN}${_hub}${NC} (etait ${_cur_hub:-aucun})"
            fi
        fi
    else
        _hub="$(awk '/asp asp-to-inter/{f=1} f&&/remote-ip/{print $2; exit}' "$STP_CFG")"
    fi
    # A loopback remote-ip is not a hub: it is the template default, left by
    # a regeneration that erased the node identity. Aligning the source on it
    # would validate a dead configuration - "source deja correcte (127.0.0.1)"
    # on an ASP that will never reach anyone.
    case "$_hub" in
        127.*|0.0.0.0|"")
            echo -e "  ${YELLOW}⚠${NC} ASP : remote-ip=${_hub:-vide} - ce n'est pas un hub."
            echo -e "     L'identite SS7 n'a pas ete appliquee : ${CYAN}./start-native.sh --node N --hub-ip
            ↪  <hub>${NC}"
            _hub="" ;;
    esac
    if [ -n "$_hub" ]; then
        _src="$(ip route get "$_hub" 2>/dev/null | sed -n 's/.*src \([0-9.]*\).*/\1/p' | head -1)"
        _cur="$(awk '/asp asp-to-inter/{f=1} f&&/local-ip/{print $2; exit}' "$STP_CFG")"
        if [ -n "$_src" ] && [ "$_src" != "$_cur" ]; then
            if [ "$DRY" = 1 ]; then
                echo -e "  [dry-run] ASP local-ip ${_cur:-?} -> ${_src}"
            else
                sed -i "/asp asp-to-inter/,/^ *as / s/^\( *local-ip \).*/\1${_src}/" "$STP_CFG"
                echo -e "  ${GREEN}✓${NC} ASP : source ${CYAN}${_src}${NC} (etait ${_cur:-aucune}) vers le hub
                ↪  ${_hub}"
                if systemctl is-active osmo-stp >/dev/null 2>&1; then
                    systemctl restart osmo-stp \
                        && echo -e "  ${GREEN}✓${NC} osmo-stp relance pour prendre la nouvelle source"
                fi
            fi
        else
```

```
            echo -e "  ${GREEN}✓${NC} ASP : source deja correcte (${_cur:-aucune})"
        fi
    fi
fi

[ "$DRY" = 1 ] || printf 'OSMO_DOCKER_HOST=%s\n' "$VIA" > "$STATE" 2>/dev/null || true

# ── 3. At boot, on request ──────────────────────────────────────────
# A live system without persistence restarts from the ISO at every boot:
# without a unit, everything is to redo. It simply replays this idempotent script.
if [ "$PERSIST" = 1 ]; then
    here="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
    if [ "$DRY" = 1 ]; then
        echo "    [dry-run] unite osmo-docker-route.service"
    else
        cat > /etc/systemd/system/osmo-docker-route.service <<EOF
[Unit]
Description=osmo-operator - route vers les conteneurs docker (${DOCKER_NET})
After=network-online.target
Wants=network-online.target
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=${here}/set_ip_vm.sh --via ${VIA} --docker-net ${DOCKER_NET}
[Install]
WantedBy=multi-user.target
EOF
        systemctl enable osmo-docker-route.service >/dev/null 2>&1 \
            && echo -e "  ${GREEN}✓${NC} rejoue au boot (osmo-docker-route.service)"
    fi
fi

# ── 4. The check that matters ──────────────────────────────────────

echo ""
first="$(echo "$DOCKER_NET" | cut -d. -f1-3).11"
echo -e "  ${BOLD}Verifier :${NC} ${CYAN}ip route get ${first}${NC}   doit indiquer via ${VIA}"
if [ "$DRY" != 1 ]; then
    got="$(ip route get "$first" 2>/dev/null | head -1)"
    if echo "$got" | grep -q "via ${VIA}"; then
        echo -e "  ${GREEN}✓${NC} ${got}"
    else
        echo -e "  ${YELLOW}⚠${NC} ${got:-aucune reponse}"
    fi
fi
```

2.122 /opt/GSM/osmo-operator/network/setup-docker-lan-route.sh

10084 octets, 197 lignes → 197 lignes

```
#!/bin/bash
# ============================================================================
# network/setup-docker-lan-route.sh - the docker containers seen from the LAN, ROUTED
#
# The docker operators live on the 172.20.0.0/24 backbone (osmo-operator-1 at
# .11, -2 at .12, -3 at .13; the inter-STP at .10). The VirtualBox VMs are
# bridged on 192.168.1.0/24 with the SS7 hub at .49. The two worlds ignore
# each other: nothing tells the LAN where to find 172.20.0.0/24.
#
# THIS SCRIPT MAKES THEM ONE NETWORK, BY ROUTING
# The host carries both (192.168.1.101 LAN side, 172.20.0.1 docker side): it
# only has to route. No macvlan, no renumbering, no port publishing - the
# containers keep their addresses, and the VMs reach them as they are.
#
```

```
#
# WHY NOT SIMPLY -p (port publishing)
# A NAT publication presents ALL containers under the host's IP. But the
# inter-STP identifies its ASPs by their source address ("match=" in the
# type=identify blocks): three operators behind a single IP cannot be told
# apart, and the hub attaches their associations to the wrong AS - or refuses
# them. Routing preserves the original address.
#
# THE THREE THINGS TO DO, AND WHY EACH
#   1. ip_forward       without it the host routes nothing, it only receives.
#   2. no masquerading towards the LAN
#                       docker MASQUERADEs everything leaving the bridge for
#                       the outside. The hub would then see 192.168.1.101
#                       everywhere instead of 172.20.0.11/12/13 - exactly what
#                       the paragraph above says to avoid. A RETURN is inserted
#                       before docker's rule, for the single destination
#                       192.168.1.0/24.
#   3. FORWARD allowed  docker sets a default FORWARD policy of DROP. The
#                       DOCKER-USER chain is made for this: consulted BEFORE
#                       docker's rules, and docker does not rewrite it.
#
# VM SIDE, one route to add (the script prints it, and installs it with --push):
#     ip route add 172.20.0.0/24 via <host IP>
#
# Usage:
#   sudo network/setup-docker-lan-route.sh              apply on the host
#   sudo network/setup-docker-lan-route.sh --push "192.168.1.123 192.168.1.49"
#                                                       + install the route on these VMs
#   sudo network/setup-docker-lan-route.sh --status     status, changes nothing
#   sudo network/setup-docker-lan-route.sh --undo       removes what the script installed
#
#   --docker-net CIDR   container network (default 172.20.0.0/24)
#   --lan CIDR          LAN network        (default: derived from the interface)
#   --iface NAME        LAN interface      (default: the one of the default route)
# ============================================================================
set -uo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

DOCKER_NET="172.20.0.0/24"
LAN_NET=""
IFACE=""
PUSH=""
ACTION="apply"

while [ $# -gt 0 ]; do
    case "$1" in
        --docker-net)   DOCKER_NET="${2:-}"; shift ;;
        --docker-net=*) DOCKER_NET="${1#*=}" ;;
        --lan)          LAN_NET="${2:-}"; shift ;;
        --lan=*)        LAN_NET="${1#*=}" ;;
        --iface)        IFACE="${2:-}"; shift ;;
        --iface=*)      IFACE="${1#*=}" ;;
        --push)         PUSH="${2:-}"; shift ;;
        --push=*)       PUSH="${1#*=}" ;;
        --status)       ACTION="status" ;;
        --undo)         ACTION="undo" ;;
        -h|--help)      sed -n '2,46p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; exit 2 ;;
    esac
    shift
done

# ── Where are we? ──────────────────────────────────────────────────────────
[ -n "$IFACE" ] || IFACE="$(ip route show default 2>/dev/null | awk '/^default/{print $5; exit}')"
```

```
[ -n "$IFACE" ] || { echo -e "${RED}Aucune interface par defaut - precisez --iface${NC}" >&2; exit 1; }

HOST_IP="$(ip -4 -o addr show dev "$IFACE" scope global 2>/dev/null | awk '{print $4}' | cut -d/ -f1 | head -1)"
[ -n "$HOST_IP" ] || { echo -e "${RED}Pas d'adresse sur ${IFACE}${NC}" >&2; exit 1; }

if [ -z "$LAN_NET" ]; then
    CIDR="$(ip -4 -o addr show dev "$IFACE" scope global 2>/dev/null | awk '{print $4}' | head -1)"
    # 192.168.1.101/24 -> 192.168.1.0/24
    LAN_NET="$(python3 - "$CIDR" <<'PY' 2>/dev/null || true
import ipaddress, sys
print(ipaddress.ip_interface(sys.argv[1]).network)
PY
)"
    [ -n "$LAN_NET" ] || LAN_NET="${HOST_IP%.*}.0/24"
fi

echo -e "${CYAN}${BOLD}== Routage docker <-> LAN ==${NC}"
echo -e "  hote      : ${CYAN}${HOST_IP}${NC} sur ${CYAN}${IFACE}${NC}"
echo -e "  LAN       : ${CYAN}${LAN_NET}${NC}"
echo -e "  conteneurs: ${CYAN}${DOCKER_NET}${NC}"
echo ""

# ── --status ──────────────────────────────────────────────────────────
if [ "$ACTION" = "status" ]; then
    fwd="$(sysctl -n net.ipv4.ip_forward 2>/dev/null)"
    printf '  %-34s %s\n' "ip_forward" "$([ "$fwd" = 1 ] && echo "1 (ok)" || echo "${fwd:-?} - le routage est
    ↪  inactif")"
    if iptables -t nat -C POSTROUTING -s "$DOCKER_NET" -d "$LAN_NET" -j RETURN 2>/dev/null; then
        printf '  %-34s %s\n' "pas de masquage vers le LAN" "pose"
    else
        printf '  %-34s %s\n' "pas de masquage vers le LAN" "ABSENT - le hub verrait ${HOST_IP} partout"
    fi
    for spec in "-s ${LAN_NET} -d ${DOCKER_NET}" "-s ${DOCKER_NET} -d ${LAN_NET}"; do
        # shellcheck disable=SC2086
        if iptables -C DOCKER-USER $spec -j ACCEPT 2>/dev/null; then
            printf '  %-34s %s\n' "FORWARD ${spec}" "autorise"
        else
            printf '  %-34s %s\n' "FORWARD ${spec}" "ABSENT"
        fi
    done
    echo ""
    echo -e "  ${BOLD}Sur chaque VM :${NC} ip route add ${DOCKER_NET} via ${HOST_IP}"
    exit 0
fi

[ "$(id -u)" -eq 0 ] || { echo -e "${RED}Root requis.${NC}" >&2; exit 1; }

# ── --undo ────────────────────────────────────────────────────────────
if [ "$ACTION" = "undo" ]; then
    iptables -t nat -D POSTROUTING -s "$DOCKER_NET" -d "$LAN_NET" -j RETURN 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} regle de non-masquage retiree"
    iptables -D DOCKER-USER -s "$LAN_NET" -d "$DOCKER_NET" -j ACCEPT 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} FORWARD LAN -> docker retire"
    iptables -D DOCKER-USER -s "$DOCKER_NET" -d "$LAN_NET" -j ACCEPT 2>/dev/null \
        && echo -e "  ${GREEN}✓${NC} FORWARD docker -> LAN retire"
    echo -e "  ${YELLOW}ip_forward laisse tel quel : d'autres services peuvent en dependre.${NC}"
    exit 0
fi

# ── 1. Routing ────────────────────────────────────────────────────────
if [ "$(sysctl -n net.ipv4.ip_forward 2>/dev/null)" != "1" ]; then
    sysctl -qw net.ipv4.ip_forward=1 && echo -e "  ${GREEN}✓${NC} ip_forward active"
else
    echo -e "  ${GREEN}✓${NC} ip_forward deja actif"
fi
```

```
# ── 2. Do NOT masquerade towards the LAN ──────────────────────────────
# -I: at the HEAD of POSTROUTING, hence before the MASQUERADE docker put
# there. A RETURN hands back to the calling chain without masquerading: the
# packet keeps its 172.20.0.x source address, and the SS7 hub recognises the
# operator that sends it.
if iptables -t nat -C POSTROUTING -s "$DOCKER_NET" -d "$LAN_NET" -j RETURN 2>/dev/null; then
    echo -e "  ${GREEN}✓${NC} non-masquage deja en place"
else
    iptables -t nat -I POSTROUTING 1 -s "$DOCKER_NET" -d "$LAN_NET" -j RETURN \
        && echo -e "  ${GREEN}✓${NC} ${DOCKER_NET} -> ${LAN_NET} : adresse source preservee"
fi

# ── 3. Allow FORWARD in both directions ───────────────────────────────
# DOCKER-USER is consulted before docker's rules, and docker does not rewrite
# it: the only place where a rule survives a daemon restart.
iptables -N DOCKER-USER 2>/dev/null || true
for spec in "-s ${LAN_NET} -d ${DOCKER_NET}" "-s ${DOCKER_NET} -d ${LAN_NET}"; do
    # shellcheck disable=SC2086
    if iptables -C DOCKER-USER $spec -j ACCEPT 2>/dev/null; then
        echo -e "  ${GREEN}✓${NC} FORWARD deja autorise (${spec})"
    else
        # shellcheck disable=SC2086
        iptables -I DOCKER-USER 1 $spec -j ACCEPT \
            && echo -e "  ${GREEN}✓${NC} FORWARD autorise (${spec})"
    fi
done

# ── 4. The route, VM side ─────────────────────────────────────────────

echo ""
echo -e "  ${BOLD}Il reste UNE chose, sur chaque VM :${NC}"
echo -e "    ${CYAN}ip route add ${DOCKER_NET} via ${HOST_IP}${NC}"
echo -e "  Sans elle, la VM ne sait pas ou envoyer 172.20.0.x et le trafic part"
echo -e "  vers sa passerelle par defaut, qui le jette."

if [ -n "$PUSH" ]; then
    echo ""
    echo -e "  ${BOLD}Pose de la route sur les machines indiquees :${NC}"
    for vm in $PUSH; do
        if ssh -o StrictHostKeyChecking=no -o ConnectTimeout=6 -o BatchMode=yes \
               "root@${vm}" "ip route replace ${DOCKER_NET} via ${HOST_IP}" 2>/dev/null; then
            echo -e "    ${GREEN}✓${NC} ${vm}"
        else
            echo -e "    ${YELLOW}⚠${NC} ${vm} injoignable - a poser a la main"
        fi
    done
fi

echo ""
echo -e "  ${BOLD}Verifier, depuis une VM :${NC} ${CYAN}ping 172.20.0.11${NC}  puis"
echo -e "  ${CYAN}checks/diag-interstp.sh${NC} cote hub - les ASP doivent apparaitre"
echo -e "  avec leur VRAIE adresse (172.20.0.11/.12/.13), pas celle de l'hote."
```

2.123 /opt/GSM/osmo-operator/network/setup-payload-ism.sh

11062 octets, 222 lignes → 222 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════
# network/setup-payload-ism.sh — the SMS PAYLOAD over ISM radio, the rest over IP
#
# Complement of --wan: the signalling (MSISDN, IMSI, SC-address, SS7, SIP)
# already goes over IP between the two nodes; this script connects the ISM-band
# RADIO link that will carry ONLY the SMS text, and writes the [payload] section
```

```
# of sms-routing.conf read by scripts/sms-interop-relay.py (via ism_payload.py).
#
# Run on EACH node (with --wan in place). Choose the backend according to the
# radio:
#
#   serial  — an ISM module over UART (LoRa, RFM69, CC1101, HC-12...) on /dev/tty*.
#             The simplest hardware-wise. The baud rate is set here by stty.
#               sudo ./network/setup-payload-ism.sh --serial /dev/ttyUSB0 --baud 9600
#
#   exec    — an SDR pipeline YOU provide (HackRF/LimeSDR/USRP + GNU Radio,
#             rtl_433 for reception, etc.). Two commands: TX reads a frame on
#             its stdin and transmits it; RX writes received frames on its stdout.
#               sudo ./network/setup-payload-ism.sh --exec \
#                    --tx-cmd '/opt/GSM/ism/ism_tx.sh' --rx-cmd '/opt/GSM/ism/ism_rx.sh'
#             Templates of ism_tx.sh / ism_rx.sh are placed in /opt/GSM/ism with
#             --exemples (they transmit nothing until filled in).
#
#   fifo    — two named pipes, if the radio is connected otherwise (socat...).
#               sudo ./network/setup-payload-ism.sh --fifo --tx /tmp/ism_tx --rx /tmp/ism_rx
#
#   loopback— all in memory, to TRY the SMS chain without radio (one node).
#               sudo ./network/setup-payload-ism.sh --loopback
#
# Common options: --conf FILE (default: the sms-routing.conf files found),
# --recv-timeout SECONDS (wait for the radio payload before failing, default 30),
# --selftest (runs the ism_payload.py self-test and exits), --off (removes [payload]).
#
# REGULATORY REMINDER: transmit only on a band and power allowed where you are
# (ISM 433/868 MHz in Europe, 902-928 MHz in region 2, 2.4 GHz), or into a
# cage/dummy load. This is your responsibility as bench operator.
# ═══════════════════════════════════════════════════════════════════════════
set -euo pipefail

GREEN='\033[0;32m'; CYAN='\033[0;36m'; YELLOW='\033[1;33m'; RED='\033[0;31m'; NC='\033[0m'
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

LINK="" DEV="" BAUD="9600" TX="" RX="" TXCMD="" RXCMD=""
RECV_TIMEOUT="30" CONF="" DO_SELFTEST=0 DO_OFF=0 DO_EXAMPLES=0

while [ $# -gt 0 ]; do
    case "$1" in
        --serial)   LINK=serial; DEV="${2:?--serial attend un /dev/tty*}"; shift ;;
        --baud)     BAUD="${2:?}"; shift ;;
        --exec)     LINK=exec ;;
        --fifo)     LINK=fifo ;;
        --loopback) LINK=loopback ;;
        --tx)       TX="${2:?}"; shift ;;
        --rx)       RX="${2:?}"; shift ;;
        --tx-cmd)   TXCMD="${2:?}"; shift ;;
        --rx-cmd)   RXCMD="${2:?}"; shift ;;
        --recv-timeout) RECV_TIMEOUT="${2:?}"; shift ;;
        --conf)     CONF="${2:?}"; shift ;;
        --selftest) DO_SELFTEST=1 ;;
        --exemples|--examples) DO_EXAMPLES=1 ;;
        --off)      DO_OFF=1 ;;
        -h|--help)  sed -n '2,/^set -euo/p' "$0" | sed 's/^# \{0,1\}//; $d'; exit 0 ;;
        *) echo "option inconnue : $1" >&2; exit 2 ;;
    esac
    shift
done

say() { echo -e "  ${GREEN}✓${NC} $*"; }
warn() { echo -e "  ${YELLOW}!${NC} $*" >&2; }

# ── the sms-routing.conf files to modify ──────────────────────────────────
# Native: /etc/osmocom/sms-routing.conf. Docker: the one of each container.
```

```
confs() {
    if [ -n "$CONF" ]; then printf '%s\n' "$CONF"; return; fi
    local found=0
    if [ -f /etc/osmocom/sms-routing.conf ]; then printf '%s\n' /etc/osmocom/sms-routing.conf; found=1; fi
    if command -v docker >/dev/null 2>&1; then
        for c in $(docker ps --format '{{.Names}}' 2>/dev/null | grep '^osmo-operator-' || true); do
            printf 'docker:%s\n' "$c"; found=1
        done
    fi
    [ "$found" = 1 ] || warn "aucun sms-routing.conf trouve (passe --conf)"
}

# Writes the [payload] section into a file (local path or docker:<container>).
write_block() {
    local target="$1" block="$2"
    local script="
        CONF=/etc/osmocom/sms-routing.conf
        sed -i '/# ── PAYLOAD ISM/,/# ── FIN PAYLOAD ISM/d' \"\$CONF\" 2>/dev/null || true
        sed -i '/^\\[payload\\]/,/^\\[/{/^\\[payload\\]/d; /^\\[/!d}' \"\$CONF\" 2>/dev/null || true
        printf '%s\n' \"\$BLOCK\" >> \"\$CONF\"
    "
    if [ "$DO_OFF" = 1 ]; then
        block=""
        script="
            CONF=/etc/osmocom/sms-routing.conf
            sed -i '/# ── PAYLOAD ISM/,/# ── FIN PAYLOAD ISM/d' \"\$CONF\" 2>/dev/null || true
        "
    fi
    case "$target" in
        docker:*)
            local c="${target#docker:}"
            docker exec -e BLOCK="$block" "$c" bash -c "$script" \
                && say "[$c] section [payload] ${DO_OFF:+retiree}${DO_OFF:-ecrite}" \
                || warn "[$c] echec d'ecriture"
            ;;
        *)
            local CONFP="$target"
            if [ "$DO_OFF" = 1 ]; then
                sed -i '/# ── PAYLOAD ISM/,/# ── FIN PAYLOAD ISM/d' "$CONFP" 2>/dev/null || true
                say "[$CONFP] section [payload] retiree"
            else
                sed -i '/# ── PAYLOAD ISM/,/# ── FIN PAYLOAD ISM/d' "$CONFP" 2>/dev/null || true
                printf '%s\n' "$block" >> "$CONFP"
                say "[$CONFP] section [payload] ecrite"
            fi
            ;;
    esac
}

# ── self-test ──────────────────────────────────────────────────────────
if [ "$DO_SELFTEST" = 1 ]; then
    echo -e "${CYAN}Autotest de la liaison payload ISM${NC}"
    case "$LINK" in
        fifo)   python3 -I "$HERE/scripts/ism_payload.py" --selftest --link fifo \
                    --tx "${TX:-/tmp/osmo_ism_tx}" --rx "${RX:-/tmp/osmo_ism_rx}" ;;
        serial) python3 -I "$HERE/scripts/ism_payload.py" --selftest --link serial --serial "$DEV" ;;
        *)      python3 -I "$HERE/scripts/ism_payload.py" --selftest ;;
    esac
    exit $?
fi

# ── removal ────────────────────────────────────────────────────────────
if [ "$DO_OFF" = 1 ]; then
    echo -e "${CYAN}Retrait du payload ISM (retour au texte par IP)${NC}"
    while read -r t; do [ -n "$t" ] && write_block "$t" ""; done < <(confs)
    exit 0
```

```
fi

[ -n "$LINK" ] || { echo -e "${RED}choisis un backend : --serial /dev/tty* | --exec | --fifo | --loopback${NC}"
 ↪  >&2; exit 2; }

# ── SDR templates for the exec backend ──────────────────────────────
if [ "$DO_EXAMPLES" = 1 ] || { [ "$LINK" = exec ] && [ -z "$TXCMD" ] && [ -z "$RXCMD" ]; }; then
    mkdir -p /opt/GSM/ism
    if [ ! -f /opt/GSM/ism/ism_tx.sh ]; then
        cat > /opt/GSM/ism/ism_tx.sh <<'TX'
#!/bin/bash
# ism_tx.sh — TRANSMITS a frame (read on stdin) in the ISM band. TEMPLATE: to fill in.
# The frame is a JSON line {"t":token,"p":hex,"c":crc}. Transmit it as is
# (FSK/LoRa/OOK modulation, your choice) on your allowed ISM frequency.
# Possible chains:
#   - LoRa/UART module : cat > /dev/ttyUSB0
#   - GNU Radio SDR    : hand the line to a flowgraph (gr-paquet) through a FIFO
# As long as this file transmits nothing, the ISM payload does not go through.
cat >/dev/null   # <- REPLACE: the frame is dropped here
TX
        chmod +x /opt/GSM/ism/ism_tx.sh
    fi
    if [ ! -f /opt/GSM/ism/ism_rx.sh ]; then
        cat > /opt/GSM/ism/ism_rx.sh <<'RX'
#!/bin/bash
# ism_rx.sh — RECEIVES in the ISM band and writes each received frame (one JSON
# line) on stdout. TEMPLATE: to fill in. Demodulate your radio and emit the lines.
# Examples:
#   - LoRa/UART module : exec cat /dev/ttyUSB0
#   - SDR              : read the output of a flowgraph (gr-paquet)
# As long as this file reads nothing, no payload arrives.
exec sleep infinity   # <- REPLACE: nothing is ever received here
RX
        chmod +x /opt/GSM/ism/ism_rx.sh
    fi
    say "gabarits SDR : /opt/GSM/ism/ism_tx.sh et ism_rx.sh (a remplir avant d'emettre)"
    [ -z "$TXCMD" ] && TXCMD="/opt/GSM/ism/ism_tx.sh"
    [ -z "$RXCMD" ] && RXCMD="/opt/GSM/ism/ism_rx.sh"
fi

# ── serial: set the baud rate (the backend then reads/writes raw) ──────
if [ "$LINK" = serial ]; then
    [ -e "$DEV" ] || warn "$DEV absent pour l'instant (branche le module ISM)"
    if [ -e "$DEV" ]; then
        stty -F "$DEV" "$BAUD" cs8 -cstopb -parenb -echo raw 2>/dev/null \
            && say "serie $DEV a $BAUD bauds, mode brut" \
            || warn "stty sur $DEV a echoue (droits ? module absent ?)"
    fi
fi

# ── compose the [payload] section ──────────────────────────────────────
BLOCK="# ── PAYLOAD ISM ──────────────────────────────────────────────────────
# Written by network/setup-payload-ism.sh. The SMS text goes over ISM radio,
# the signalling stays on IP (see wiki/Payload-ISM.md).

[payload]
mode = ism
link = $LINK
recv_timeout = $RECV_TIMEOUT"
case "$LINK" in
    serial) BLOCK="$BLOCK
serial = $DEV" ;;
    fifo)   BLOCK="$BLOCK
tx = ${TX:-/tmp/osmo_ism_tx}
rx = ${RX:-/tmp/osmo_ism_rx}" ;;
    exec)   BLOCK="$BLOCK
```

```
tx_cmd = $TXCMD
rx_cmd = $RXCMD" ;;
esac
BLOCK="$BLOCK
# ── FIN PAYLOAD ISM ──────────────────────────────────────────────────────"

echo -e "${CYAN}Payload ISM : backend ${LINK}${NC}"
while read -r t; do [ -n "$t" ] && write_block "$t" "$BLOCK"; done < <(confs)

echo
say "Fait. Relance le relais SMS pour qu'il prenne la liaison :"
echo "     - natif  : sudo systemctl restart osmo-smsc  (ou scripts/smsc-start.sh)"
echo "     - docker : sudo docker restart \$(docker ps --format '{{.Names}}' | grep osmo-operator-)"
echo "   ou simplement : ./start-native.sh --wan --payload-ism (il fait tout)."
if [ "$LINK" = exec ]; then
    warn "backend exec : remplis /opt/GSM/ism/ism_tx.sh et ism_rx.sh (ta radio)."
fi
warn "N'emets que sur une bande/puissance ISM autorisees chez toi, ou en charge/cage."
```

2.124 /opt/GSM/osmo-operator/network/setup-vbox-interco.sh

9619 octets, 178 lignes → 178 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════════
# network/setup-vbox-interco.sh - WAN interconnection between THIS machine and
# VirtualBox VMs, on a single PC
#
# The osmo-operator WAN links machines by their public IP. To test it without
# renting N servers, the remote peers are replaced by VMs and the public link
# by a host-only segment: the model is the same, the addresses are local.
#
#
#   ┌─ this machine ──────────┐        ┌─ VM osmo-wan-2 ─┐  ┌─ VM osmo-wan-3 ─┐
#   │ docker or native lab    │        │ ISO osmo-operator │  │ ISO osmo-operator  │
#   │ node 1 · prefix 11      │        │ node 2 · 22     │  │ node 3 · 33     │
#   └──────────┬──────────────┘        └──────────┬──────┘  └──────────┬──────┘
#              │ vboxnet0 192.168.56.11           │ .12                │ .13
#              └──────────────────── host-only segment ──────────────────┘
#
# WHY HOST-ONLY AND NOT "INTERNAL NETWORK"
# An internal network links the VMs TO EACH OTHER and leaves the host out.
# Here the host is itself a node: without a leg on the segment its lab cannot
# reach its peers, and nothing reports it - the trunks simply stay
# "Unavailable". Host-only is the only mode where the host takes part.
#
# THE CONSTRAINT TO KNOW
# VirtualBox refuses any host-only range outside 192.168.56.0/21 as long as
# /etc/vbox/networks.conf does not allow it. Hence this default prefix.
#
# Usage:
#   sudo network/setup-vbox-interco.sh [--nodes N] [--host-node 1]
#                                      [--subnet 192.168.56] [--iso f.iso]
#                                      [--build-iso] [--no-vms] [--indicatifs "11 22"]
#
# Writes the table to /etc/osmo-wan.conf, ready for --wan.
# Called automatically by: start-docker.sh --virtualbox  /  start-native.sh --virtualbox
# ═══════════════════════════════════════════════════════════════════════════
set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
```

```
LAB="$DIR/tools/vbox-wan-lab.sh"
# shellcheck source=network/wan-nodes.sh
. "$DIR/network/wan-nodes.sh"

NODES=""
HOST_NODE=1
SUBNET="192.168.56"
ISO=""
BUILD_ISO=0
NO_VMS=0
INDS=""
PASSTHRU=()

while [ $# -gt 0 ]; do
    case "$1" in
        --nodes)       NODES="${2:-}"; shift ;;
        --nodes=*)     NODES="${1#*=}" ;;
        --host-node)   HOST_NODE="${2:-}"; shift ;;
        --host-node=*) HOST_NODE="${1#*=}" ;;
        --subnet)      SUBNET="${2:-}"; shift ;;
        --subnet=*)    SUBNET="${1#*=}" ;;
        --iso)         ISO="${2:-}"; shift ;;
        --iso=*)       ISO="${1#*=}" ;;
        --build-iso)   BUILD_ISO=1 ;;
        --no-vms)      NO_VMS=1 ;;
        --indicatifs)  INDS="${2:-}"; shift ;;
        --indicatifs=*) INDS="${1#*=}" ;;
        --conf)        WAN_CONF_FILE="${2:-}"; shift ;;
        --conf=*)      WAN_CONF_FILE="${1#*=}" ;;
        -h|--help)     sed -n '2,40p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) PASSTHRU+=("$1") ;;   # passed as is to the lab (--ram, --usb-filter...)
    esac
    shift
done

echo -e "${CYAN}${BOLD}"
echo "╔═══════════════════════════════════════════════════════════════════╗"
echo "║   Interconnexion WAN - cette machine + VM VirtualBox              ║"
echo "╚═══════════════════════════════════════════════════════════════════╝"
echo -e "${NC}"

# ── Guards ──────────────────────────────────────────────────────────────
# Run INSIDE a VM, this script can do nothing useful and its error messages
# would not say so: VBoxManage is simply absent there.
if command -v systemd-detect-virt >/dev/null 2>&1; then
    # `|| echo none` would produce two lines when the command exits 1 (the
    # "no virtualisation" case): only the first is kept.
    _virt="$(systemd-detect-virt 2>/dev/null | head -1)" || true
    _virt="${_virt:-none}"
    case "$_virt" in
        oracle|kvm|qemu|vmware|microsoft)
            echo -e "${RED}Vous etes DANS une machine virtuelle (${_virt}).${NC}" >&2
            echo -e "  L'interconnexion VirtualBox se monte depuis l'HOTE." >&2
            echo -e "  Ici, le WAN se configure normalement : ${CYAN}./start-native.sh --wan${NC}" >&2
            exit 1 ;;
    esac
fi

VBM="$(command -v VBoxManage || command -v vboxmanage || true)"
if [ -z "$VBM" ]; then
    echo -e "${RED}VBoxManage introuvable - VirtualBox n'est pas installe.${NC}" >&2
    exit 1
fi
if ! grep -q '^vboxdrv ' /proc/modules 2>/dev/null; then
    echo -e "${RED}Le module vboxdrv n'est pas charge : aucune VM ne demarrera.${NC}" >&2
    echo -e "  ${CYAN}sudo /sbin/vboxconfig${NC}" >&2
```

```
    if command -v mokutil >/dev/null 2>&1 && mokutil --sb-state 2>/dev/null | grep -i enabled >/dev/null; then
        echo -e "  ${YELLOW}Secure Boot actif : le module doit etre signe et sa cle enrolee (MOK).${NC}" >&2
    fi
    exit 1
fi
[ "$(id -u)" -eq 0 ] || { echo -e "${RED}Root requis (interface reseau + /etc/osmo-wan.conf).${NC}" >&2; exit 1; }

# ── How many nodes ──────────────────────────────────────────────────
if [ -z "$NODES" ]; then
    if [ -t 0 ]; then
        # Same question as ./launch/start-docker.sh --wan, same bounds.
       NODES=$(_wan_ask "WAN VirtualBox" "Nombre de noeuds au total, cette machine comprise (1-9) :" "3") || exit 1
    else
        NODES=3
    fi
fi
[[ "$NODES" =~ ^[1-9]$ ]] || { echo -e "${RED}--nodes doit valoir 1 a 9${NC}" >&2; exit 2; }
[[ "$HOST_NODE" =~ ^[1-9]$ ]] && [ "$HOST_NODE" -le "$NODES" ] \
    || { echo -e "${RED}--host-node doit designer un noeud de 1 a ${NODES}${NC}" >&2; exit 2; }

# ── Allowed range? ──────────────────────────────────────────────────
if [ "$SUBNET" != "192.168.56" ] && ! grep -q "${SUBNET}" /etc/vbox/networks.conf 2>/dev/null; then
    echo -e "${YELLOW}Attention : VirtualBox n'autorise que 192.168.56.0/21 en host-only.${NC}"
    echo -e "  Pour ouvrir ${SUBNET}.0/24 : ${CYAN}echo '* ${SUBNET}.0/24' | sudo tee -a
    ↪  /etc/vbox/networks.conf${NC}"
    echo ""
fi

# ── Node table ──────────────────────────────────────────────────────
declare -a ind_arr=()
[ -n "$INDS" ] && read -r -a ind_arr <<< "$INDS"
WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=()
for i in $(seq 1 "$NODES"); do
    # Node N → .1N, no exception for the host: the table reads at a glance
    # and the node number shows in the address.
    WAN_IP[$i]="${SUBNET}.$((10 + i))"
    WAN_IND[$i]="${ind_arr[$((i-1))]:-$(wan_default_ind "$i")}"
    WAN_NODE_LIST+=("$i")
done
WAN_NODE_COUNT="$NODES"
WAN_NODE_ID="$HOST_NODE"
WAN_OPS="${WAN_OPS:-1}"
wan_nodes_validate || exit 1

echo -e "  Cette machine porte le ${BOLD}noeud ${HOST_NODE}${NC} ; les autres sont des VM."
wan_nodes_summary
echo ""

# ── The VM lab does the rest: host-only network, fixed DHCP, hardware ──────
if [ "$NO_VMS" -eq 1 ]; then
    echo -e "  ${YELLOW}--no-vms : ni reseau ni VM crees, seule la table est ecrite.${NC}"
else
    [ -x "$LAB" ] || { echo -e "${RED}tools/vbox-wan-lab.sh introuvable${NC}" >&2; exit 1; }
    lab_args=(create --nodes "$NODES" --host-node "$HOST_NODE" --subnet "$SUBNET"
              --indicatifs "$(for i in "${WAN_NODE_LIST[@]}"; do printf '%s ' "${WAN_IND[$i]}"; done)")
    [ -n "$ISO" ]          && lab_args+=(--iso "$ISO")
    [ "$BUILD_ISO" -eq 1 ] && lab_args+=(--build-iso)
    bash "$LAB" "${lab_args[@]}" ${PASSTHRU[@]+"${PASSTHRU[@]}"} || {
        echo -e "${RED}La creation du banc a echoue.${NC}" >&2; exit 1; }
fi

# ── Persisted table ─────────────────────────────────────────────────
# WAN_AUTO=0 on the host side: here start-docker.sh / start-native.sh decides
# when to bring up the WAN. The VM ISOs are built with WAN_AUTO=1.

WAN_AUTO=0 wan_nodes_save "$WAN_CONF_FILE"
```

```
echo -e "  ${GREEN}✓${NC} table ecrite dans ${CYAN}${WAN_CONF_FILE}${NC}"
echo ""
echo -e "  ${BOLD}Ensuite, sur cette machine :${NC}"
echo -e "    ${CYAN}sudo ./launch/start-docker.sh --wan${NC}         (le lab docker devient le noeud ${HOST_NODE})"
echo -e "    ${CYAN}tools/vbox-wan-lab.sh start${NC}    (demarre les VM)"
echo ""
```

2.125 /opt/GSM/osmo-operator/network/setup-wan-interop.sh

28043 octets, 590 lignes → 590 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════════
# network/setup-wan-interop.sh - WAN interconnection between two osmo-operator instances
#
# Lets the MS of one server call the MS of the other server through the
# +66 prefix. Example: dialing 6610001 from server A calls MS 10001 on
# server B.
#
# Architecture :
#
#   Serveur A (82.66.231.141)              Serveur B (37.59.111.84)
#   ┌──────────────────────────┐           ┌──────────────────────────┐
#   │ Op1 Asterisk :5060       │           │ Op1 Asterisk :5060       │
#   │   172.20.0.11            │           │   172.20.0.11            │
#   │ Op2 Asterisk :5060       │           │ Op2 Asterisk :5060       │
#   │   172.20.0.12            │           │   172.20.0.12            │
#   └──────────┬───────────────┘           └──────────┬───────────────┘
#              │                                      │
#   Host iptables DNAT                     Host iptables DNAT
#   Op1: :5080 → 172.20.0.11:5060         Op1: :5080 → 172.20.0.11:5060
#   Op2: :5082 → 172.20.0.12:5060         Op2: :5082 → 172.20.0.12:5060
#              │                                      │
#              └──────── SIP/RTP over WAN ────────────┘
#                   82.66.231.141 ↔ 37.59.111.84
#
# WAN ports per operator:
#   OpN SIP  : 5080 + (N-1)*2
#   OpN RTP  : 20000 + (N-1)*500  →  20000 + N*500 - 1
#
# Dialing prefix:
#   66 + normal_number  →  routed to the remote server
#   Ex: 6610001 = call MS 10001 on the remote server
#   Ex: 6620001 = call MS 20001 (op2) on the remote server
#
# Usage :
#   sudo ./network/setup-wan-interop.sh <local_public_ip> <remote_public_ip> [n_operators]
#
# Run on EACH server with the IPs swapped:
#   Server A: sudo ./network/setup-wan-interop.sh 82.66.231.141 37.59.111.84 2
#   Server B: sudo ./network/setup-wan-interop.sh 37.59.111.84 82.66.231.141 2
# ═══════════════════════════════════════════════════════════════════════════

set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

# ── Parameters ──────────────────────────────────────────────────────────────
LOCAL_IP="${1:-}"
REMOTE_IP="${2:-}"
N_OPS="${3:-2}"
WAN_PREFIX="66"                    # Prefix for inter-server calls
SIP_WAN_BASE=5080                  # Base WAN SIP port
```

```
RTP_WAN_BASE=20000                  # Base WAN RTP port
RTP_PER_OP=500                      # RTP ports per operator
CONTAINER_PREFIX="osmo-operator-"
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"   # to call network/firewall-wan.sh

if [ -z "$LOCAL_IP" ] || [ -z "$REMOTE_IP" ]; then
    echo -e "${RED}Usage: sudo $0 <local_public_ip> <remote_public_ip> [n_operators]${NC}"
    echo ""
    echo "  Serveur A : sudo $0 82.66.231.142 37.59.111.84 2"
    echo "  Serveur B : sudo $0 37.59.111.84 82.66.231.142 2"
    exit 1
fi

if [ "$(id -u)" -ne 0 ]; then
    echo -e "${RED}Doit etre lance en root (sudo).${NC}"; exit 1
fi

# ── Fonctions helper ──────────────────────────────────────────────────
op_backbone_ip() { echo "172.20.0.$((10 + $1))"; }
op_sip_port()    { echo $(( SIP_WAN_BASE + ($1 - 1) * 2 )); }
op_rtp_start()   { echo $(( RTP_WAN_BASE + ($1 - 1) * RTP_PER_OP )); }
op_rtp_end()     { echo $(( RTP_WAN_BASE + $1 * RTP_PER_OP - 1 )); }
op_container()   { echo "${CONTAINER_PREFIX}$1"; }

# NOTE: sed -i fails on Docker bind mounts (rename() impossible).
# Every sed edit uses the pattern: cp file /tmp → sed -i /tmp → cp back

# ═══════════════════════════════════════════════════════════════════════
# Banner
# ═══════════════════════════════════════════════════════════════════════
echo -e "${CYAN}${BOLD}"
echo "╔═══════════════════════════════════════════════════════════════════╗"
echo "║        WAN Interop - Interconnexion osmo-operator distante        ║"
echo "╚═══════════════════════════════════════════════════════════════════╝"
echo -e "${NC}"
echo -e "  Local       : ${CYAN}${LOCAL_IP}${NC}"
echo -e "  Remote      : ${CYAN}${REMOTE_IP}${NC}"
echo -e "  Operateurs  : ${CYAN}${N_OPS}${NC}"
echo -e "  Prefixe WAN : ${CYAN}${WAN_PREFIX}${NC}"
echo ""

# ── Container check ──────────────────────────────────────────────────
echo -e "${GREEN}[1/5] Verification des containers...${NC}"
for i in $(seq 1 "$N_OPS"); do
    cname=$(op_container "$i")
    if ! docker ps --format '{{.Names}}' | grep -q "^${cname}$"; then
        echo -e "  ${RED}✗ ${cname} non trouve${NC}"
        echo -e "  ${YELLOW}Lancez start-docker.sh d'abord.${NC}"
        exit 1
    fi
    echo -e "  ${GREEN}✓${NC} ${cname} actif"
done
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [2/5] iptables - DNAT for the traffic coming in from the remote server
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[2/5] Configuration iptables (DNAT entrant)...${NC}"

# Enable IP forwarding
sysctl -w net.ipv4.ip_forward=1 > /dev/null 2>&1

# Clear the previous WAN interop rules (custom chain)
iptables -t nat -D PREROUTING -j OSMO_WAN_INTEROP 2>/dev/null || true
iptables -t nat -F OSMO_WAN_INTEROP 2>/dev/null || true
iptables -t nat -X OSMO_WAN_INTEROP 2>/dev/null || true
```

```
# Create the chain
iptables -t nat -N OSMO_WAN_INTEROP

for i in $(seq 1 "$N_OPS"); do
    bb_ip=$(op_backbone_ip "$i")
    sip_port=$(op_sip_port "$i")
    rtp_start=$(op_rtp_start "$i")
    rtp_end=$(op_rtp_end "$i")

    # SIP signaling: external port → container:5060
    iptables -t nat -A OSMO_WAN_INTEROP \
        -s "$REMOTE_IP" -p udp --dport "$sip_port" \
        -j DNAT --to-destination "${bb_ip}:5060"

    # SIP TCP (just in case)
    iptables -t nat -A OSMO_WAN_INTEROP \
        -s "$REMOTE_IP" -p tcp --dport "$sip_port" \
        -j DNAT --to-destination "${bb_ip}:5060"

    # RTP: port range → container (same ports inside)
    iptables -t nat -A OSMO_WAN_INTEROP \
        -s "$REMOTE_IP" -p udp --dport "${rtp_start}:${rtp_end}" \
        -j DNAT --to-destination "${bb_ip}"

    echo -e "  ${CYAN}Op${i}${NC} SIP :${sip_port} → ${bb_ip}:5060  RTP ${rtp_start}-${rtp_end} → ${bb_ip}"
done

# Insert the chain into PREROUTING
iptables -t nat -I PREROUTING -j OSMO_WAN_INTEROP

# MASQUERADE for the return path (the container sees the traffic as coming
# from the host). --ctstate DNAT: without that restriction the masquerade would
# ALSO catch LAN flows merely routed to a container, which would then answer
# the host instead of the client. Only what the DNAT above redirected is
# masqueraded. Restricting on the host address instead would match nothing:
# after DNAT the source is the remote server's, not ours.
# The broad rule is removed first - the -C guard tests the exact spec, otherwise
# both would stay in place and the broad one would win.
iptables -t nat -D POSTROUTING -d 172.20.0.0/24 -j MASQUERADE 2>/dev/null || true
if ! iptables -t nat -C POSTROUTING -d 172.20.0.0/24 -m conntrack --ctstate DNAT -j MASQUERADE 2>/dev/null; then
    # Fallback when the iptables conntrack module is missing: without it the -A
    # fails and, the broad rule being already removed, there would be no
    # masquerade left and no return path to the containers.
    iptables -t nat -A POSTROUTING -d 172.20.0.0/24 -m conntrack --ctstate DNAT -j MASQUERADE 2>/dev/null || {
        echo -e "  ${YELLOW}! -m conntrack indisponible : masquage large 172.20.0.0/24 (moins precis)${NC}"
        iptables -t nat -C POSTROUTING -d 172.20.0.0/24 -j MASQUERADE 2>/dev/null \
            || iptables -t nat -A POSTROUTING -d 172.20.0.0/24 -j MASQUERADE || true
    }
fi

echo -e "  ${GREEN}✓ iptables configure${NC}"
echo ""

# ═══════════════════════════════════════════════════════════════════
# [2bis/5] Firewall - automatic opening of the ports coming in from the remote
# The DNAT above redirects the traffic, but if a firewall (ufw/firewalld/
# iptables INPUT) filters INPUT, WAN SIP/RTP are dropped -> no WAN audio.
# The rules are therefore applied at EVERY WAN start (network/firewall-wan.sh is
# idempotent: no duplicates when rerun).
# ═══════════════════════════════════════════════════════════════════
echo -e "${GREEN}[2bis/5] Firewall - ouverture des ports WAN (entrant ${REMOTE_IP})...${NC}"
if [ -x "$SCRIPT_DIR/firewall-wan.sh" ] || [ -f "$SCRIPT_DIR/firewall-wan.sh" ]; then
    if bash "$SCRIPT_DIR/firewall-wan.sh" "$REMOTE_IP" "$N_OPS"; then
        echo -e "  ${GREEN}✓ Firewall ouvert pour ${REMOTE_IP}${NC}"
    else
```

```
        echo -e "  ${YELLOW}⚠ network/firewall-wan.sh a echoue - ouvre les ports a la main (voir resume).${NC}"
    fi
else
    echo -e "  ${YELLOW}⚠ network/firewall-wan.sh introuvable dans ${SCRIPT_DIR} - ports a ouvrir a la main.${NC}"
fi
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [3/5] RTP configuration - WAN port range per operator
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[3/5] Configuration RTP WAN dans chaque container...${NC}"

for i in $(seq 1 "$N_OPS"); do
    cname=$(op_container "$i")
    rtp_start=$(op_rtp_start "$i")
    rtp_end=$(op_rtp_end "$i")

    # Create/modify rtp.conf in Asterisk to use the WAN range.
    # The original range is kept for local calls, and the config is added
    # to the PJSIP transport (external_media_address)
    docker exec "$cname" bash -c "cat > /etc/asterisk/rtp.conf << 'RTPEOF'
[general]
rtpstart=${rtp_start}
rtpend=${rtp_end}
strictrtp=no
icesupport=no
RTPEOF"

    echo -e "  ${CYAN}Op${i}${NC} RTP range: ${rtp_start}-${rtp_end}"
done

echo -e "  ${GREEN}✓ RTP configure${NC}"
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [4/5] PJSIP + Dialplan - WAN trunks and remote interop context
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[4/5] Injection PJSIP trunks + dialplan WAN...${NC}"

for i in $(seq 1 "$N_OPS"); do
    cname=$(op_container "$i")
    local_sip_port=$(op_sip_port "$i")
    rtp_start=$(op_rtp_start "$i")
    rtp_end=$(op_rtp_end "$i")

    # ── PJSIP: patch transport-udp + trunks to each remote operator ──

    # Generate the PJSIP blocks for the WAN trunks
    pjsip_wan=""

    # No separate transport-wan - the existing transport-udp is patched
    # to add external_media_address (needed for correct SDP)
    docker exec "$cname" bash -c "
        if ! grep -q 'external_media_address' /etc/asterisk/pjsip.conf; then
            cp /etc/asterisk/pjsip.conf /tmp/_pj.work
            sed -i '/^\[transport-udp\]/,/^\$/ {
                /^\$/i\\
external_media_address=${LOCAL_IP}\\
external_signaling_address=${LOCAL_IP}\\
local_net=172.20.0.0/24\\
local_net=127.0.0.0/8
            }' /tmp/_pj.work
            cp /tmp/_pj.work /etc/asterisk/pjsip.conf
            rm -f /tmp/_pj.work
        fi
    "
```

```
    pjsip_wan+="
; ═══════════════════════════════════════════════════════════════
; WAN INTEROP - Trunks vers serveur distant ${REMOTE_IP}
; Genere par network/setup-wan-interop.sh le $(date '+%Y-%m-%d %H:%M:%S')
; ═══════════════════════════════════════════════════════════════
"

    # One identify per ADDRESS, outside the loop: identify only looks at the
    # source address, never the port, so one block per remote operator would
    # declare several for the same address and an incoming call would be
    # assigned to one of them at random. Two addresses only: the bridge
    # gateway, through which the WAN traffic masqueraded by the host arrives,
    # and the remote server itself when it is not masqueraded.
    pjsip_wan+="
; ── Identification des appels entrants du WAN ──────────────────
[wan-identify-remote]
type=identify
endpoint=wan_trunk_op1
match=${REMOTE_IP}

[wan-identify-gw]
type=identify
endpoint=wan_trunk_op1
match=172.20.0.1
"

    # One trunk per remote operator
    for j in $(seq 1 "$N_OPS"); do
        remote_sip_port=$(op_sip_port "$j")

        pjsip_wan+="
; ── Trunk WAN → Serveur distant Op${j} (${REMOTE_IP}:${remote_sip_port}) ─────
[wan_trunk_op${j}]
type=endpoint
transport=transport-udp
context=wan_in
disallow=all
allow=gsm
allow=ulaw
aors=wan_trunk_op${j}
media_encryption=no
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
ice_support=no

[wan_trunk_op${j}]
type=aor
contact=sip:${REMOTE_IP}:${remote_sip_port}
qualify_frequency=30
qualify_timeout=5.0
"
    done

    # Inject into pjsip.conf
    # First remove the previous WAN block if present
    docker exec "$cname" bash -c "
        cp /etc/asterisk/pjsip.conf /tmp/_pj.work
        sed -i '/; ═.*WAN INTEROP/,\$d' /tmp/_pj.work 2>/dev/null || true
        cp /tmp/_pj.work /etc/asterisk/pjsip.conf
        rm -f /tmp/_pj.work
    "

    # Append the new block
```

```
    docker exec "$cname" bash -c "cat >> /etc/asterisk/pjsip.conf << 'PJSIPEOF'
${pjsip_wan}
PJSIPEOF"

    echo -e "  ${CYAN}Op${i}${NC} PJSIP: ${N_OPS} trunk(s) WAN ajoutes"

    # ── Dialplan: [wan_out] and [wan_in] contexts ──────────────────────

    # ── SIP softphones: number → endpoint, extracted DYNAMICALLY from pjsip.conf ──
    # Every endpoint carrying a "callerid=... <NNN>" (ex: linphone_A <100>) becomes
    # routable SIP→SIP over WAN in both directions, without hardcoding numbers.
    softphones=$(docker exec "$cname" cat /etc/asterisk/pjsip.conf | awk '
        /^\[/                 { s=$0; gsub(/[][]/,"",s); ep=s }
        /callerid=.*<[0-9]+>/ { n=$0; sub(/.*</,"",n); sub(/>.*/,"",n); print n":"ep }
    ')
    echo -e "  ${CYAN}Op${i}${NC} Softphones SIP detectes: $(echo "$softphones" | tr '\n' ' ')"

    # Generate the wan_out context (outgoing calls to the remote)
    wan_dialplan="
; ═══════════════════════════════════════════════════════════════════
; WAN INTEROP - Routage appels inter-serveur (prefixe ${WAN_PREFIX})
; Genere par network/setup-wan-interop.sh le $(date '+%Y-%m-%d %H:%M:%S')
;
; Composer ${WAN_PREFIX}NXXXX = appeler NXXXX sur le serveur distant
; Composer ${WAN_PREFIX}NXXXXX = appeler NXXXXX sur le serveur distant
; Ex: ${WAN_PREFIX}10001 → MS 10001 op1 distant
;     ${WAN_PREFIX}20001 → MS 20001 op2 distant
; ═══════════════════════════════════════════════════════════════════

[wan_out]
"
    for j in $(seq 1 "$N_OPS"); do
        # Pattern for a 1-digit operator prefix (1-9)
        if [ "$j" -lt 10 ]; then
            wan_dialplan+="
; → Operateur distant ${j} (4 chiffres)
exten => _${j}XXXX,1,NoOp(=== WAN OUT Op${j}: \${EXTEN} → ${REMOTE_IP} ===)
 same => n,Dial(PJSIP/\${EXTEN}@wan_trunk_op${j},,rT)
 same => n,NoOp(WAN: \${DIALSTATUS})
 same => n,Congestion()
 same => n,Hangup()

; → Operateur distant ${j} (5 chiffres)
exten => _${j}XXXXX,1,NoOp(=== WAN OUT Op${j}: \${EXTEN} → ${REMOTE_IP} ===)
 same => n,Dial(PJSIP/\${EXTEN}@wan_trunk_op${j},,rT)
 same => n,NoOp(WAN: \${DIALSTATUS})
 same => n,Congestion()
 same => n,Hangup()
"
        else
            wan_dialplan+="
; → Operateur distant ${j} (prefixe 2 chiffres)
exten => _${j}XXXX,1,NoOp(=== WAN OUT Op${j}: \${EXTEN} → ${REMOTE_IP} ===)
 same => n,Dial(PJSIP/\${EXTEN}@wan_trunk_op${j},,rT)
 same => n,Congestion()
 same => n,Hangup()
"
        fi
    done

    # → Remote SIP softphones (SIP→SIP over WAN) - generated dynamically.
    #   A local Linphone dials ${WAN_PREFIX}<num>; the ${WAN_PREFIX} is stripped
    #   upstream (Goto wan_out,\${EXTEN:2}), so <num> is routed to the matching
    #   softphone of the same operator on the remote server.
    for sp in $softphones; do
        sp_num="${sp%%:*}"
```

```
        wan_dialplan+="
; → Softphone SIP distant ${sp_num} (Op${i})
exten => ${sp_num},1,NoOp(=== WAN OUT SIP Op${i}: ${sp_num} → ${REMOTE_IP} ===)
 same => n,Dial(PJSIP/${sp_num}@wan_trunk_op${i},,rT)
 same => n,NoOp(WAN: \${DIALSTATUS})
 same => n,Congestion()
 same => n,Hangup()
"
    done

    wan_dialplan+="
; Fallback WAN
exten => _X.,1,NoOp(=== WAN OUT: destination inconnue \${EXTEN} ===)
 same => n,Congestion()
 same => n,Hangup()

; ═══════════════════════════════════════════════════════════════════
; [wan_in] - Appels entrants DEPUIS le serveur distant
; ═══════════════════════════════════════════════════════════════════
[wan_in]

; Appel distant → MS local (meme operateur)
exten => _${i}XXXX,1,NoOp(=== WAN IN → GSM local Op${i}: \${EXTEN} ===)
 same => n,Set(CALLERID(all)=<\${CALLERID(num)}>)
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Dial(PJSIP/\${EXTEN}@gsm_msc,,rT)
 same => n,Congestion()
 same => n,Hangup()

exten => _${i}XXXXX,1,NoOp(=== WAN IN → GSM local Op${i}: \${EXTEN} ===)
 same => n,Set(CALLERID(all)=<\${CALLERID(num)}>)
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Dial(PJSIP/\${EXTEN}@gsm_msc,,rT)
 same => n,Congestion()
 same => n,Hangup()
"

    # Local SIP softphones (incoming calls from the remote server) - dynamic
    for sp in $softphones; do
        sp_num="${sp%%:*}"
        sp_ep="${sp#*:}"
        wan_dialplan+="
; Softphone SIP local ${sp_num} → ${sp_ep}
exten => ${sp_num},1,NoOp(=== WAN IN → ${sp_ep} local (${sp_num}) ===)
 same => n,Gosub(sub-record,s,1(${sp_num}))
 same => n,Dial(PJSIP/${sp_ep},,rT)
 same => n,Congestion()
 same => n,Hangup()
"
    done

    wan_dialplan+="
; Appel distant → autre operateur local (re-route via interop_out)
exten => _X.,1,NoOp(=== WAN IN → routage local: \${EXTEN} ===)
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Goto(interop_out,\${EXTEN},1)

; Echo test depuis le distant
exten => 600,1,NoOp(=== WAN ECHO TEST ===)
 same => n,Answer()
 same => n,Playback(demo-echotest)
 same => n,Echo()
 same => n,Playback(demo-echodone)
 same => n,Hangup()
"
```

```
    # Remove the previous WAN block from the dialplan
    docker exec "$cname" bash -c '
        cp /etc/asterisk/extensions.conf /tmp/_ext.work
        sed -i "/; ==.*WAN INTEROP.*Routage/,\$d" /tmp/_ext.work 2>/dev/null || true
        cp /tmp/_ext.work /etc/asterisk/extensions.conf
        rm -f /tmp/_ext.work
    '

    # Append to the dialplan
    docker exec "$cname" bash -c "cat >> /etc/asterisk/extensions.conf << 'EXTEOF'
${wan_dialplan}
EXTEOF"

    # ── Inject the _66. pattern directly into [gsm_in] and [internal] ──
    docker exec "$cname" bash -c '
        EXT="/etc/asterisk/extensions.conf"
        if ! grep -q "_'"${WAN_PREFIX}"'\." "$EXT"; then
            cp "$EXT" /tmp/_ext.work

            sed -i "/^\[gsm_in\]$/a\\
\\
; -- WAN : prefixe '"${WAN_PREFIX}"' -> serveur distant --\\
exten => _'"${WAN_PREFIX}"'.,1,NoOp(=== GSM -> WAN: strip '"${WAN_PREFIX}"' ===)\\
 same => n,Gosub(sub-record,s,1(\${EXTEN:2}))\\
 same => n,Goto(wan_out,\${EXTEN:2},1)" /tmp/_ext.work

            sed -i "/^\[internal\]$/a\\
\\
; -- WAN : prefixe '"${WAN_PREFIX}"' -> serveur distant --\\
exten => _'"${WAN_PREFIX}"'.,1,NoOp(=== SIP -> WAN: strip '"${WAN_PREFIX}"' ===)\\
 same => n,Gosub(sub-record,s,1(\${EXTEN:2}))\\
 same => n,Goto(wan_out,\${EXTEN:2},1)" /tmp/_ext.work

            cp /tmp/_ext.work "$EXT"
            rm -f /tmp/_ext.work
        fi
    '

    echo -e "  ${CYAN}Op${i}${NC} Dialplan: wan_out + wan_in + routage ${WAN_PREFIX}"
done

echo -e "  ${GREEN}✓ PJSIP + Dialplan configures${NC}"
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [5/5] Restart Asterisk (transport modifie → restart complet requis)
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[5/5] Restart Asterisk...${NC}"

for i in $(seq 1 "$N_OPS"); do
    cname=$(op_container "$i")

    docker exec "$cname" bash -c "
        asterisk -rx 'core stop now' 2>/dev/null || pkill asterisk 2>/dev/null || true
        sleep 2
        pkill -9 asterisk 2>/dev/null || true
        sleep 1
        rm -f /var/lib/asterisk/astdb.sqlite3
        asterisk -f &
        disown
    " 2>/dev/null || true

    echo -e "  ${CYAN}Op${i}${NC} Asterisk redemarre"
done

echo ""
```

```
# ═══════════════════════════════════════════════════════════════════════
# Resume
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}${BOLD}"
echo "╔═══════════════════════════════════════════════════════════════════╗"
echo "║              WAN Interop configure avec succes !                  ║"
echo "╚═══════════════════════════════════════════════════════════════════╝"
echo -e "${NC}"

echo -e "  ${BOLD}Serveur local  :${NC} ${CYAN}${LOCAL_IP}${NC}"
echo -e "  ${BOLD}Serveur distant:${NC} ${CYAN}${REMOTE_IP}${NC}"
echo ""

echo -e "  ${BOLD}Ports WAN par operateur :${NC}"
for i in $(seq 1 "$N_OPS"); do
    sip_port=$(op_sip_port "$i")
    rtp_start=$(op_rtp_start "$i")
    rtp_end=$(op_rtp_end "$i")
    echo -e "    Op${i}: SIP ${CYAN}:${sip_port}${NC}  RTP ${CYAN}${rtp_start}-${rtp_end}${NC}"
done

echo ""
echo -e "  ${BOLD}Comment appeler :${NC}"
echo -e "    Depuis un MS local, composer ${CYAN}${WAN_PREFIX}${NC} + numero distant"
echo -e "    Ex: ${CYAN}${WAN_PREFIX}10001${NC} = appeler MS 10001 op1 sur ${REMOTE_IP}"
echo -e "    Ex: ${CYAN}${WAN_PREFIX}20001${NC} = appeler MS 20001 op2 sur ${REMOTE_IP}"
echo ""

echo -e "  ${BOLD}Firewall :${NC} ${GREEN}deja applique automatiquement sur CE serveur${NC} (etape 2bis)."
echo -e "    Pense a lancer le WAN aussi sur ${CYAN}${REMOTE_IP}${NC} (il ouvrira son propre INPUT)."
echo -e "    Pour re-ouvrir manuellement : ${CYAN}sudo ${SCRIPT_DIR}/network/firewall-wan.sh ${REMOTE_IP}
 ↪  ${N_OPS}${NC}"
echo -e "    Equivalent ufw :"
echo ""
for i in $(seq 1 "$N_OPS"); do
    sip_port=$(op_sip_port "$i")
    rtp_start=$(op_rtp_start "$i")
    rtp_end=$(op_rtp_end "$i")
    echo -e "    ${YELLOW}# Op${i}${NC}"
    echo -e "    ufw allow from ${REMOTE_IP} to any port ${sip_port} proto udp"
    echo -e "    ufw allow from ${REMOTE_IP} to any port ${rtp_start}:${rtp_end} proto udp"
done

echo ""
echo -e "  ${BOLD}Test rapide :${NC}"
echo -e "    # Depuis le container op1 sur ce serveur :"
echo -e "    docker exec -it osmo-operator-1 bash"
echo -e "    # Dans le VTY mobile (Ctrl-b → fenetre ue_g1) :"
echo -e "    call 1 ${WAN_PREFIX}10001"
echo ""
echo -e "  ${BOLD}Diagnostic :${NC}"
echo -e "    # Verifier la connectivite SIP :"
echo -e "    docker exec osmo-operator-1 asterisk -rx 'pjsip show endpoints'"
echo -e "    docker exec osmo-operator-1 asterisk -rx 'pjsip show aors'"
echo -e "    # Logs Asterisk :"
echo -e "    docker exec osmo-operator-1 asterisk -rx 'core set verbose 5'"
echo ""

# ── Save the config for persistence ──────────────────────────────────
CONFIG_SAVE="/etc/osmo-wan-interop.conf"
cat > "$CONFIG_SAVE" << EOF
# osmo-wan-interop.conf - saved WAN config
# Generated on $(date)
LOCAL_IP=${LOCAL_IP}
REMOTE_IP=${REMOTE_IP}
```

```
N_OPS=${N_OPS}
WAN_PREFIX=${WAN_PREFIX}
SIP_WAN_BASE=${SIP_WAN_BASE}
RTP_WAN_BASE=${RTP_WAN_BASE}
RTP_PER_OP=${RTP_PER_OP}
EOF

echo -e "  Config sauvegardee dans ${CYAN}${CONFIG_SAVE}${NC}"
echo -e "  Relancer apres reboot : ${CYAN}sudo $0 ${LOCAL_IP} ${REMOTE_IP} ${N_OPS}${NC}"
echo ""
```

2.126 /opt/GSM/osmo-operator/network/setup-wan-mesh.sh

40708 octets, 820 lignes → 820 lignes

```
#!/bin/bash
# ============================================================================
# network/setup-wan-mesh.sh - N-node WAN, routed by DIALING CODE
#
# Generalises network/setup-wan-interop.sh (which only knows 2 servers):
# here the topology is a TABLE of nodes (network/wan-nodes.sh), each with its
# dialing code, and EVERYTHING is generated in a single pass - so it can be
# rerun and works for 2 nodes as well as for 12.
#
#   Dial <target_code><number>  →  that number, on the target node.
#   The code is stripped on the WAY OUT: the receiving node gets the bare
#   number, the one its HLR and its dialplan already know.
#
# What this script configures, on THIS node:
#   • iptables: SIP/RTP/SMS ingress from each peer (docker DNAT, native REDIRECT)
#   • firewall: opening of the same ports (network/firewall-wan.sh, idempotent)
#   • Asterisk: rtp.conf, PJSIP trunks to each remote (node, operator),
#               [wan_out]/[wan_in] dialplan + per-code entries
#   • SMS     : sms-routing.conf - one remote operator per (node, operator),
#               with code stripping (cf. scripts/sms-interop-relay.py)
#
# Usage:
#   sudo network/setup-wan-mesh.sh --nodes "1:1.2.3.4:33 2:5.6.7.8:44" --id 1 [--ops 2]
#   sudo network/setup-wan-mesh.sh --config /etc/osmo-wan.conf
#   options: --native | --docker (default: auto)  --no-restart  --dry-run
#
# Run on EACH node, with the SAME table and its own --id.
# ============================================================================
set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
# shellcheck source=network/wan-nodes.sh
. "$SCRIPT_DIR/wan-nodes.sh"

MODE=""
# --op-is-node: local operator i IS node i, not an operator of node --id. This
# is the case when each container carries its own address and its own dialing
# code: without this notion a single peer set would be computed for the whole
# machine and every container would get the same view - one of them even
# routing to itself.
OP_IS_NODE=0            # docker | native
NO_RESTART=0
CONFIG_ONLY=0
DRY=0
CONF=""
CHAIN="OSMO_WAN_MESH"
```

```
SIP_WAN_BASE=5080
RTP_WAN_BASE=20000
RTP_PER_OP=500
SMS_RELAY_BASE=7890
CONTAINER_PREFIX="osmo-operator-"
# Root prefix, native mode only: lets a test tree (NATIVE_ROOT=/tmp/essai) be
# targeted instead of the machine's /etc. Empty in production.
NATIVE_ROOT="${NATIVE_ROOT:-}"

wan_sip_port()  { echo $(( SIP_WAN_BASE + ($1 - 1) * 2 )); }
wan_rtp_start() { echo $(( RTP_WAN_BASE + ($1 - 1) * RTP_PER_OP )); }
wan_rtp_end()   { echo $(( RTP_WAN_BASE + $1 * RTP_PER_OP - 1 )); }
wan_sms_port()  { echo $(( SMS_RELAY_BASE + $1 - 1 )); }
op_backbone_ip(){ echo "172.20.0.$((10 + $1))"; }
op_container()  { echo "${CONTAINER_PREFIX}$1"; }

# A peer whose address is on the backbone is reached INSIDE the bridge: the
# traffic does not cross the host's nat table there (br_netfilter is not
# loaded), so the ports PUBLISHED on the host - 5080/5082 for SIP, 7891 for the
# second operator's SMS relay - answer nobody. Toward such a peer the target
# must be what the container really listens on: 5060 and the relay's base port.
peer_is_backbone() { case "$1" in 172.20.0.*) return 0 ;; *) return 1 ;; esac; }

# On which port to reach operator j of node r.
#
# The 5080/5082/... plan exists ONLY to multiplex several operators behind a
# single address: a docker host republishing its containers' ports, or a
# native node setting REDIRECTs to 5060. A node carrying ONLY ONE operator has
# nothing to multiplex - its Asterisk listens directly on 5060.
#
# "Is the address on the docker backbone" is not the right discriminant: a
# NATIVE VM off the backbone with a single operator also listens on 5060 only:
#     192.168.1.2:5060  -> SIP/2.0 401 Unauthorized
#     192.168.1.2:5080  -> no answer
# (It would answer on 5080 if the mesh had run there - it sets the REDIRECTs -
#  but that cannot be assumed from here.) The operator count of the remote
#  node is the discriminant.
peer_sip_port() {          # $1 = remote node id, $2 = operator
    if [ "$(node_nops "$1")" -eq 1 ]; then echo 5060; else wan_sip_port "$2"; fi
}
peer_sms_port() {          # $1 = remote node id, $2 = operator
    if [ "$(node_nops "$1")" -eq 1 ]; then echo "$SMS_RELAY_BASE"; else wan_sms_port "$2"; fi
}

# The address the TABLE announces for us is not always the one the packet
# carries on arrival: behind a router the table gives the public address and
# the host only ever sees its private one. A "-d" on an address absent from
# the machine then matches NOTHING and the WAN dies silently - the case of two
# servers behind their home routers. The destination is therefore only
# restricted when the address really is ours.
addr_is_local() {
    ip -o addr show 2>/dev/null | awk '{print $4}' | cut -d/ -f1 | grep -qx "$1"
}

# Number of operators carried by a REMOTE node. The local count does not say:
# with one node per container it is 2 and would build a trunk to a
# non-existent operator, a _<code>2XXXX pattern and an SMS route nobody
# serves. WAN_NOPS comes from the table.
#
# When the table has no 4th field, the fallback is the general case. A fixed
# fallback of 1 would drop operator 2 from every CLASSIC setup
# ("--wan-nodes 1:a:11 2:b:22 --ops 2", where each node carries both
# operators) - trunk missing, inter-node calls dead for it. The fallback is 1
# only where it is true by construction: --op-is-node, one operator per node.
# Otherwise the original assumption holds: the peers are set up like us.
```

```
node_nops() {
    local n="${WAN_NOPS[$1]:-}"
    if ! [[ "$n" =~ ^[1-9][0-9]*$ ]]; then
        if [ "${OP_IS_NODE:-0}" = "1" ]; then n=1; else n="${N_OPS:-1}"; fi
    fi
    echo "$n"
}

# ── `pipefail` + `grep -q` trap ──────────────────────────────────────
# `grep -q` exits on the FIRST match; the pipeline producer then gets SIGPIPE
# and dies with 141, and under `set -o pipefail` 141 is what the pipeline
# returns. Result: "found" reads as "not found", intermittently - depending on
# whether the output fit in the pipe buffer. The whole input is therefore read
# and the output discarded.
qgrep() { grep "$@" >/dev/null; }

usage() { sed -n '2,26p' "${BASH_SOURCE[0]}" | sed 's/^# \{0,1\}//'; }

while [ $# -gt 0 ]; do
    case "$1" in
        --nodes)   WAN_NODES="${2:-}"; shift ;;
        --nodes=*) WAN_NODES="${1#*=}" ;;
        --id)      WAN_NODE_ID="${2:-}"; shift ;;
        --id=*)    WAN_NODE_ID="${1#*=}" ;;
        --op-is-node) OP_IS_NODE=1 ;;
        --ops)     WAN_OPS="${2:-}"; shift ;;
        --ops=*)   WAN_OPS="${1#*=}" ;;
        --config)  CONF="${2:-}"; shift ;;
        --config=*) CONF="${1#*=}" ;;
        --native)  MODE="native" ;;
        --docker)  MODE="docker" ;;
        --no-restart) NO_RESTART=1 ;;
        --config-only) CONFIG_ONLY=1; NO_RESTART=1 ;;
        --dry-run) DRY=1 ;;
        -h|--help) usage; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; usage >&2; exit 2 ;;
    esac
    shift
done

# ── Node table ──────────────────────────────────────────────────────
# --config is both source and destination: it is only read if it already
# exists, and --nodes takes precedence (a new topology is being described).
if [ -n "${WAN_NODES:-}" ]; then
    wan_nodes_parse "$WAN_NODES" || exit 1
elif [ -n "$CONF" ]; then
    wan_nodes_load "$CONF" || { echo -e "${RED}config WAN illisible : $CONF${NC}" >&2; exit 1; }
elif [ -n "${WAN_NODES:-}" ]; then
    wan_nodes_parse "$WAN_NODES" || exit 1
elif wan_nodes_load "$WAN_CONF_FILE" 2>/dev/null; then
    :
else
    echo -e "${RED}Aucune table de noeuds : --nodes, --config ou $WAN_CONF_FILE${NC}" >&2
    exit 1
fi

# Explicit --id > config > auto-detection by local IP.
if [ "${WAN_NODE_ID:-0}" = "0" ] || [ -z "${WAN_NODE_ID:-}" ]; then
    wan_nodes_detect_self || {
        echo -e "${RED}Impossible de determiner le noeud local : passez --id N${NC}" >&2; exit 1; }
    echo -e "  ${CYAN}Noeud local detecte par son IP : ${WAN_NODE_ID}${NC}"
fi
wan_nodes_validate || exit 1

# ── Mode ────────────────────────────────────────────────────────────
# The ISO has no docker; the lab host has one AND osmo-operator-* containers.
```

```
# The presence of the containers is trusted, not that of the binary.
if [ -z "$MODE" ]; then
    if command -v docker >/dev/null 2>&1 && \
       docker ps --format '{{.Names}}' 2>/dev/null | qgrep "^${CONTAINER_PREFIX}1$"; then
        MODE="docker"
    else
        MODE="native"
    fi
fi

if [ "$MODE" = "native" ]; then
    OP_IDS=( "${NATIVE_OP_ID:-${OPERATOR_ID:-1}}" )
else
    [ "${WAN_OPS:-1}" -ge 1 ] || WAN_OPS=1
    OP_IDS=(); for i in $(seq 1 "$WAN_OPS"); do OP_IDS+=("$i"); done
fi
N_OPS="${#OP_IDS[@]}"

LOCAL_IP="$(wan_local_ip)"
LOCAL_IND="$(wan_local_ind)"

echo -e "${CYAN}${BOLD}"
echo "╔═══════════════════════════════════════════════════════════════════════╗"
echo "║   WAN mesh - routage par indicatif entre N noeuds osmo-operator       ║"
echo "╚═══════════════════════════════════════════════════════════════════════╝"
echo -e "${NC}"
echo -e "  Mode        : ${CYAN}${MODE}${NC}   Operateurs locaux : ${CYAN}${N_OPS}${NC} (${OP_IDS[*]})"
wan_nodes_summary
echo ""

[ "$(id -u)" -ne 0 ] && { echo -e "${RED}Root requis (iptables + configs).${NC}" >&2; exit 1; }

REMOTES=()
for id in "${WAN_NODE_LIST[@]}"; do [ "$id" = "$WAN_NODE_ID" ] || REMOTES+=("$id"); done

# The context of ONE operator: its node, its dialing code, its address and its
# peers. Without --op-is-node it does not change from one operator to the
# next (they share the machine's node); with it, each has its own.
mesh_set_context() {
    local op="$1" node id
    if [ "${OP_IS_NODE:-0}" = "1" ]; then node="$op"; else node="$WAN_NODE_ID"; fi
    MESH_NODE="$node"
    [ -n "${WAN_IND[$node]:-}" ] && LOCAL_IND="${WAN_IND[$node]}"
    [ -n "${WAN_IP[$node]:-}"  ] && LOCAL_IP="${WAN_IP[$node]}"
    REMOTES=()
    for id in "${WAN_NODE_LIST[@]}"; do [ "$id" = "$node" ] || REMOTES+=("$id"); done
}
mesh_set_context "${OP_IDS[0]}"
[ "${#REMOTES[@]}" -ge 1 ] || { echo -e "${RED}Aucun pair distant.${NC}" >&2; exit 1; }

# Union of the peers of ALL local operators. mesh_set_context recomputes
# REMOTES on every call: what does not depend on an operator - the firewall
# opening, the displayed counters - needs the peers of every operator, not
# only the LAST one's list.
ALL_REMOTES=(); _seen=""
for _i in "${OP_IDS[@]}"; do
    mesh_set_context "$_i"
    for _r in "${REMOTES[@]}"; do
        case " $_seen " in
            *" $_r "*) ;;
            *) _seen="$_seen $_r"; ALL_REMOTES+=("$_r") ;;
        esac
    done
done
mesh_set_context "${OP_IDS[0]}"
```

```
run() { if [ "$DRY" -eq 1 ]; then echo "  [dry-run] $*"; else "$@"; fi; }

# ── Access to the conf files, docker or native ──────────────────────────
# The file is pulled onto the host, transformed LOCALLY with awk/sed, and
# pushed back. This lets the same code serve both modes - and avoids
# multi-level escaped seds inside a `docker exec bash -c` (where
# network/setup-wan-interop.sh is most fragile).
ast_pull() {  # $1=op  $2=path  → stdout
    if [ "$MODE" = "docker" ]; then docker exec "$(op_container "$1")" cat "$2" 2>/dev/null || true
    else cat "${NATIVE_ROOT}$2" 2>/dev/null || true; fi
}
ast_push() {  # $1=op  $2=path  ← stdin
    if [ "$DRY" -eq 1 ]; then cat >/dev/null; echo "  [dry-run] ecriture $2 (op $1)"; return 0; fi
    if [ "$MODE" = "docker" ]; then docker exec -i "$(op_container "$1")" bash -c "cat > '$2'"
    else mkdir -p "$(dirname "${NATIVE_ROOT}$2")"; cat > "${NATIVE_ROOT}$2"; fi
}
ast_cli() {   # $1=op  $2=asterisk CLI command
    if [ "$MODE" = "docker" ]; then docker exec "$(op_container "$1")" asterisk -rx "$2" 2>/dev/null || true
    else asterisk -rx "$2" 2>/dev/null || true; fi
}

BEGIN_MARK="; ─── OSMO WAN MESH BEGIN ───"
END_MARK="; ─── OSMO WAN MESH END ───"
SMS_BEGIN="# ─── OSMO WAN MESH BEGIN ───"
SMS_END="# ─── OSMO WAN MESH END ───"

# Removes our blocks AND the "WAN INTEROP" block (generated by
# setup-wan-interop.sh, always appended at the end of the file): both describe
# the same traffic, letting them coexist would make the routing depend on the
# order of the patterns.
strip_generated() { sed -e '/OSMO WAN MESH BEGIN/,/OSMO WAN MESH END/d' -e '/; ═.*WAN INTEROP/,$d'; }

# ═══════════════════════════════════════════════════════════════════════
# [1/6] Target check
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[1/6] Verification des cibles Asterisk...${NC}"
for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    if [ "$MODE" = "docker" ]; then
        docker ps --format '{{.Names}}' | qgrep "^$(op_container "$i")$" \
            || { echo -e "  ${RED}✗ $(op_container "$i") absent - lancez start-docker.sh d'abord${NC}"; exit 1; }
        echo -e "  ${GREEN}✓${NC} $(op_container "$i")"
    else
        [ -d "${NATIVE_ROOT}/etc/asterisk" ] || { echo -e "  ${RED}✗ ${NATIVE_ROOT}/etc/asterisk absent${NC}";
        ↪  exit 1; }
        echo -e "  ${GREEN}✓${NC} /etc/asterisk (natif, operateur ${i})"
    fi
done
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [2/6] iptables - ingress from each peer
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[2/6] iptables - ouverture entrante depuis ${#ALL_REMOTES[@]} pair(s)...${NC}"
if [ "$CONFIG_ONLY" -eq 1 ]; then
    echo -e "  ${YELLOW}--config-only : ni iptables ni firewall. Les ports WAN resteront fermes${NC}"
    echo -e "  ${YELLOW}tant que ce script n'aura pas ete relance sans --config-only.${NC}"
else
run sysctl -w net.ipv4.ip_forward=1 >/dev/null 2>&1 || true

if [ "$DRY" -eq 0 ]; then
    iptables -t nat -D PREROUTING -j "$CHAIN" 2>/dev/null || true
    iptables -t nat -F "$CHAIN" 2>/dev/null || true
    iptables -t nat -X "$CHAIN" 2>/dev/null || true
    iptables -t nat -N "$CHAIN"
fi
```

```
# The local operator is the OUTER loop: mesh_set_context recomputes REMOTES,
# and nested the other way round the peer loop would read the list left by
# the last call - a single view for every operator, leaving the other
# operators' peers without any rule in the chain.
for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    lip="$LOCAL_IP"
    sip=$(wan_sip_port "$i"); rs=$(wan_rtp_start "$i"); re=$(wan_rtp_end "$i")
    sms=$(wan_sms_port "$i")
    # The destination filter, computed once per local operator.
    _dfil=()
    if addr_is_local "$lip"; then _dfil=(-d "$lip")
    else echo -e "  ${YELLOW}! ${lip} (table) n'est pas une adresse de cette machine : DNAT non restreint en
     ↪  destination${NC}"; fi
    # ── THE OUTGOING DIRECTION: one source port PER CONTAINER ────────────────
    # Every container runs Asterisk on the same port, 5060. Toward a LAN peer
    # they are masqueraded behind the host's single address, and the
    # masquerade may give them THE SAME translated port:
    #     192.168.1.31.48057 > 192.168.1.2.5080   (from 172.20.0.11)
    #     192.168.1.31.48057 > 192.168.1.2.5080   (from 172.20.0.12)
    # Two distinct flows, one tuple: the peer's replies only come back to one
    # of the two, and the other's trunk stays "Unavail" forever - both
    # containers do send, only the return is missing.
    #
    # A stable and distinct source port is therefore pinned per operator.
    # Stable for the peer too: its identify always sees the same address/port
    # pair for a given node, instead of a port drawn at random at every
    # conntrack expiry.
    if [ "$MODE" = "docker" ] && [ "${#REMOTES[@]}" -gt 0 ]; then
        _snat_port=$(( 5060 + i ))
        _lan_net="$(printf '%s' "$lip" | cut -d. -f1-3).0/24"
        for _pr in udp tcp; do
            run iptables -t nat -D POSTROUTING -s "$(op_backbone_ip "$i")" -p "$_pr" --sport 5060 \
                -d "$_lan_net" -j SNAT --to-source "${lip}:${_snat_port}" 2>/dev/null || true
            run iptables -t nat -I POSTROUTING 1 -s "$(op_backbone_ip "$i")" -p "$_pr" --sport 5060 \
                -d "$_lan_net" -j SNAT --to-source "${lip}:${_snat_port}"
        done
        echo -e "  ${CYAN}Op${i}${NC} sort en SIP depuis ${CYAN}${lip}:${_snat_port}${NC} (port source epingle)"
        # Already established conntrack entries ignore the new rule: Asterisk
        # refreshes its qualify every 30 s, so they never expire on their own.
        # They are flushed, otherwise the rule only takes effect at the next
        # Asterisk restart.
        if [ "$DRY" -eq 0 ] && command -v conntrack >/dev/null 2>&1; then
            conntrack -D -s "$(op_backbone_ip "$i")" -p udp >/dev/null 2>&1 || true
        fi
    fi

    for r in "${REMOTES[@]}"; do
        rip="${WAN_IP[$r]}"
        if [ "$MODE" = "docker" ]; then
            bb=$(op_backbone_ip "$i")
            # -d "$lip": filtering on the source alone would also catch what THIS
            # node SENDS to the peer (same source, same destination port). The
            # packet would never leave the host, it would go back to our own
            # container answering itself - the trunk showing "Avail" while the
            # peer received nothing.
            run iptables -t nat -A "$CHAIN" -s "$rip" ${_dfil[@]+"${_dfil[@]}"} -p udp --dport "$sip" -j DNAT
             ↪  --to-destination "${bb}:5060"
            run iptables -t nat -A "$CHAIN" -s "$rip" ${_dfil[@]+"${_dfil[@]}"} -p tcp --dport "$sip" -j DNAT
             ↪  --to-destination "${bb}:5060"
            run iptables -t nat -A "$CHAIN" -s "$rip" ${_dfil[@]+"${_dfil[@]}"} -p udp --dport "${rs}:${re}" -j
             ↪  DNAT --to-destination "$bb"
            run iptables -t nat -A "$CHAIN" -s "$rip" ${_dfil[@]+"${_dfil[@]}"} -p tcp --dport "$sms" -j DNAT
             ↪  --to-destination "${bb}:${SMS_RELAY_BASE}"
        else
```

```
            # Native: Asterisk and the relay already listen on the host. Only the
            # WAN port → service port translation is needed; RTP arrives
            # directly in the range set by rtp.conf.
            run iptables -t nat -A "$CHAIN" -s "$rip" -p udp --dport "$sip" -j REDIRECT --to-ports 5060
            run iptables -t nat -A "$CHAIN" -s "$rip" -p tcp --dport "$sip" -j REDIRECT --to-ports 5060
            [ "$sms" = "$SMS_RELAY_BASE" ] || \
                run iptables -t nat -A "$CHAIN" -s "$rip" -p tcp --dport "$sms" -j REDIRECT --to-ports
                ↪ "$SMS_RELAY_BASE"
        fi
        echo -e "  ${CYAN}node ${r}${NC} ${rip} → Op${i} SIP/RTP/SMS acceptes"
    done
done

if [ "$DRY" -eq 0 ]; then
    iptables -t nat -I PREROUTING -j "$CHAIN"
    if [ "$MODE" = "docker" ]; then
        # --ctstate DNAT: without that restriction the masquerade would ALSO
        # catch LAN flows merely routed to a container, which would then answer
        # the host instead of the client. Only what our own DNAT redirected is
        # masqueraded. Restricting on the host address instead would match
        # nothing: after DNAT the source is the peer's.
        # The broad rule is removed first - the -C guard tests the exact spec,
        # otherwise both would stay in place and the broad one would win.
        iptables -t nat -D POSTROUTING -d 172.20.0.0/24 -j MASQUERADE 2>/dev/null || true
        if ! iptables -t nat -C POSTROUTING -d 172.20.0.0/24 -m conntrack --ctstate DNAT -j MASQUERADE
        ↪ 2>/dev/null; then
            # Without the iptables conntrack module the -A fails and, under
            # set -e, the script would die HERE: chain already inserted in
            # PREROUTING and broad rule already removed, but no rtp.conf, pjsip,
            # dialplan or SMS routes. The broad rule is then put back, with a
            # notice - less precise, but a full mesh beats half a mesh.
            iptables -t nat -A POSTROUTING -d 172.20.0.0/24 -m conntrack --ctstate DNAT -j MASQUERADE 2>/dev/null
            ↪ || {
              echo -e "  ${YELLOW}! -m conntrack indisponible : masquage large 172.20.0.0/24 (moins precis)${NC}"
               iptables -t nat -C POSTROUTING -d 172.20.0.0/24 -j MASQUERADE 2>/dev/null \
                   || iptables -t nat -A POSTROUTING -d 172.20.0.0/24 -j MASQUERADE || true
            }
        fi
    fi
fi

# Firewall INPUT - without it the DNAT redirects traffic that INPUT drops.
for r in "${ALL_REMOTES[@]}"; do
    if [ -f "$SCRIPT_DIR/firewall-wan.sh" ]; then
        run bash "$SCRIPT_DIR/firewall-wan.sh" "${WAN_IP[$r]}" "$N_OPS" >/dev/null 2>&1 \
            && echo -e "  ${GREEN}✓${NC} firewall ouvert pour ${WAN_IP[$r]}" \
            || echo -e "  ${YELLOW}⚠${NC} firewall-wan.sh a echoue pour ${WAN_IP[$r]}"
    fi
done
fi
echo ""

# ═══════════════════════════════════════════════════════════════════
# [3/6] rtp.conf - WAN RTP range per operator
# ═══════════════════════════════════════════════════════════════════
echo -e "${GREEN}[3/6] rtp.conf...${NC}"
for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    rs=$(wan_rtp_start "$i"); re=$(wan_rtp_end "$i")
    printf '[general]\nrtpstart=%s\nrtpend=%s\nstrictrtp=no\nicesupport=no\n' "$rs" "$re" \
        | ast_push "$i" /etc/asterisk/rtp.conf
    echo -e "  ${CYAN}Op${i}${NC} RTP ${rs}-${re}"
done
echo ""

# ═══════════════════════════════════════════════════════════════════
```

```
# [4/6] PJSIP - one trunk per (remote node, remote operator)
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[4/6] PJSIP - trunks WAN...${NC}"

for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    pj="$(mktemp)"; ast_pull "$i" /etc/asterisk/pjsip.conf | strip_generated > "$pj"

    # external_media_address: the IP the SDP announces to the other end. It
    # MUST be the public IP of THIS node. The configs/pjsip.conf template
    # already sets one (the docker/host IP): it is OVERWRITTEN, not merely
    # added when missing - otherwise the WAN announces a private address and
    # the audio goes nowhere while the signalling passes (call answered,
    # silence).
    awk -v ip="$LOCAL_IP" '
        /^\[/ { if (intr && !done) { emit(); done=1 } ; intr = ($0 == "[transport-udp]") ; print ; next }
        intr && /^external_(media|signaling)_address=/ { next }
        intr && /^local_net=/ { seen_net=1 ; print ; next }
        intr && /^[[:space:]]*$/ && !done { emit(); done=1; intr=0; print; next }
        { print }
        END { if (intr && !done) emit() }
        function emit() {
            print "external_media_address=" ip
            print "external_signaling_address=" ip
            if (!seen_net) { print "local_net=172.20.0.0/16"; print "local_net=127.0.0.0/8" }
        }
    ' "$pj" > "${pj}.new" && mv "${pj}.new" "$pj"

    {
        echo "$BEGIN_MARK"
        echo "; Trunks WAN - generes par network/setup-wan-mesh.sh"
        echo "; Noeud local ${WAN_NODE_ID} (${LOCAL_IP}, indicatif ${LOCAL_IND})"
        for r in "${REMOTES[@]}"; do
            rip="${WAN_IP[$r]}"; rind="${WAN_IND[$r]}"; rnops="$(node_nops "$r")"
            # ONE identify per remote address: identify only looks at the source
            # address, never the port. One block per remote operator would set
            # several for the SAME address, and the incoming call would be
            # assigned to one of them at random - sometimes the wrong context.
            cat <<EOF

; ── node ${r} (indicatif ${rind}) · ${rip} · ${rnops} operateur(s) ──
[wan-id-n${r}]
type=identify
endpoint=wan_n${r}_op1
match=${rip}
EOF
            for j in $(seq 1 "$rnops"); do
                rsip=$(peer_sip_port "$r" "$j")
                # Peer on the backbone: the port published on the host is not
                # crossed inside the bridge, the container listens on 5060.

                cat <<EOF

; ── node ${r} · operateur ${j} · ${rip}:${rsip} ──
[wan_n${r}_op${j}]
type=endpoint
transport=transport-udp
context=wan_in
disallow=all
allow=gsm
allow=ulaw
aors=wan_n${r}_op${j}
media_encryption=no
direct_media=no
rtp_symmetric=yes
force_rport=yes
```

```
rewrite_contact=yes
ice_support=no

[wan_n${r}_op${j}]
type=aor
contact=sip:${rip}:${rsip}
qualify_frequency=30
qualify_timeout=5.0
EOF
            done
        done
        # The bridge gateway once, outside the loop: incoming WAN traffic is
        # masqueraded by the host and thus arrives with 172.20.0.1 as source.
        # Repeated in each block it would give as many identify entries as
        # (node, operator) pairs for this single address.
        if [ "$MODE" = "docker" ]; then
            # Traffic arriving masqueraded as 172.20.0.1 is what the host routed
            # from the LAN: it can ONLY come from a peer off the backbone. The
            # first such peer is taken, not REMOTES[0] - often a neighbouring
            # container, which never arrives masqueraded.
            _gwr=""
            for _g in "${REMOTES[@]}"; do
                peer_is_backbone "${WAN_IP[$_g]}" || { _gwr="$_g"; break; }
            done
            if [ -n "$_gwr" ]; then
                cat <<EOF

; ── passerelle du bridge - trafic WAN masque par l'hote (noeud ${_gwr}) ──
[wan-id-gw]
type=identify
endpoint=wan_n${_gwr}_op1
match=172.20.0.1
EOF
            fi
        fi
        echo "$END_MARK"
    } >> "$pj"

    ast_push "$i" /etc/asterisk/pjsip.conf < "$pj"
    rm -f "$pj"
    ntrunk=0
    for r in "${REMOTES[@]}"; do ntrunk=$(( ntrunk + $(node_nops "$r") )); done
    echo -e "  ${CYAN}Op${i}${NC} ${ntrunk} trunk(s) WAN"
done
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [5/6] Dialplan - [wan_out] per code, [wan_in], gsm_in/internal entries
# ═══════════════════════════════════════════════════════════════════════
echo -e "${GREEN}[5/6] Dialplan - routage par indicatif...${NC}"

for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    ext="$(mktemp)"; ast_pull "$i" /etc/asterisk/extensions.conf | strip_generated > "$ext"

    # Softphones: every endpoint with "callerid=... <NNN>" becomes reachable
    # over the WAN, without hardcoding any number.
    softphones=$(ast_pull "$i" /etc/asterisk/pjsip.conf | awk '
        /^\[/                 { s=$0; gsub(/[][]/,"",s); ep=s }
        /callerid=.*<[0-9]+>/ { n=$0; sub(/.*</,"",n); sub(/>.*/,"",n); print n":"ep }')

    body="$(mktemp)"
    {
        echo "$BEGIN_MARK"
        echo "; Dialplan WAN - network/setup-wan-mesh.sh"
        echo "; Composer <indicatif><numero> ; l'indicatif est retire a la sortie."
```

```
        for r in "${REMOTES[@]}"; do
            printf ';   %-4s → node %s (%s)\n' "${WAN_IND[$r]}" "$r" "${WAN_IP[$r]}"
        done
        echo ""
        echo "[wan_out]"
        for r in "${REMOTES[@]}"; do
            rind="${WAN_IND[$r]}"; off="${#rind}"
            echo ""
            echo "; ── node ${r} - indicatif ${rind} (strip ${off}) ──────────────────"
            # The operators of the REMOTE node, not ours: counting ours would
            # produce a _<code>2XXXX toward an operator that node does not have
            # - the call would leave and get lost instead of being refused here.
            # The dialed number: <node code> + the SIX-digit MSISDN.
            # To reach operator 1 of node 1 from here: 11 100101.
            # The MSISDN starts with the REMOTE node number (<node>00<op>
            # <rank>): it is "${r}00${j}", not a common "600".
            # A single pattern is enough, the length being fixed.
            for j in $(seq 1 "$(node_nops "$r")"); do
                cat <<EOF
exten => _${rind}${r}00${j}XX,1,NoOp(=== WAN OUT node${r} op${j}: \${EXTEN} → ${WAN_IP[$r]} ===)
 same => n,Dial(PJSIP/\${EXTEN:${off}}@wan_n${r}_op${j},,rT)
 same => n,NoOp(WAN \${DIALSTATUS})
 same => n,Congestion()
 same => n,Hangup()
EOF
            done
            # The operator targeted on the peer, bounded to what it really
            # carries: using the LOCAL operator index toward a node with a
            # single operator would call wan_n<r>_op2 - an endpoint that is not
            # declared since the trunks follow node_nops: Asterisk would log
            # "endpoint not found" instead of trying.
            _spop="$i"; [ "$_spop" -le "$(node_nops "$r")" ] || _spop=1
            for sp in $softphones; do
                sp_num="${sp%%:*}"
                cat <<EOF
exten => ${rind}${sp_num},1,NoOp(=== WAN OUT node${r} softphone ${sp_num} ===)
 same => n,Dial(PJSIP/${sp_num}@wan_n${r}_op${_spop},,rT)
 same => n,Congestion()
 same => n,Hangup()
EOF
            done
        done
        cat <<EOF

; Indicatif local (${LOCAL_IND}) compose depuis ce noeud : c'est chez nous.
; On retire l'indicatif et on repart dans le routage local plutot que de
; sortir sur le WAN pour revenir - un aller-retour qui, lui, ne raccroche pas.
exten => _${LOCAL_IND}X.,1,NoOp(=== WAN: indicatif local, routage interne \${EXTEN} ===)
 same => n,Goto(gsm_in,\${EXTEN:${#LOCAL_IND}},1)

exten => _X.,1,NoOp(=== WAN OUT: indicatif inconnu \${EXTEN} ===)
 same => n,Congestion()
 same => n,Hangup()

; ── [wan_in] - ce qui ARRIVE des autres noeuds ──────────────────────────
; Le noeud emetteur a deja retire l'indicatif : on recoit le numero nu.
[wan_in]
EOF
        # The code was stripped by the calling node: what arrives here is the
        # bare six-digit MSISDN. These are OUR numbers, so they start with OUR
        # node number: <WAN_NODE_ID>00<operator><rank>.
        for j in "${OP_IDS[@]}"; do
                cat <<EOF
exten => _${WAN_NODE_ID}00${j}XX,1,NoOp(=== WAN IN → GSM Op${j}: \${EXTEN} ===)
 same => n,Set(CALLERID(all)=<\${CALLERID(num)}>)
 same => n,Gosub(sub-annuaire,\${CALLERID(num)},1)
```

```
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Dial(PJSIP/\${EXTEN}@gsm_msc,,rT)
 same => n,Congestion()
 same => n,Hangup()
EOF
        done
        for sp in $softphones; do
            sp_num="${sp%%:*}"; sp_ep="${sp#*:}"
            cat <<EOF
exten => ${sp_num},1,NoOp(=== WAN IN → ${sp_ep} (${sp_num}) ===)
 same => n,Gosub(sub-record,s,1(${sp_num}))
 same => n,Dial(PJSIP/${sp_ep},,rT)
 same => n,Congestion()
 same => n,Hangup()
EOF
        done
        cat <<EOF

; Un pair qui n'aurait pas retire notre indicatif : on le retire ici.
exten => _${LOCAL_IND}X.,1,NoOp(=== WAN IN: indicatif local encore present ===)
 same => n,Goto(wan_in,\${EXTEN:${#LOCAL_IND}},1)

; Autre operateur de CE noeud : on repasse par le routage inter-operateur local.
exten => _X.,1,NoOp(=== WAN IN → routage local \${EXTEN} ===)
 same => n,Gosub(sub-record,s,1(\${EXTEN}))
 same => n,Goto(interop_out,\${EXTEN},1)

exten => 600,1,NoOp(=== WAN ECHO TEST ===)
 same => n,Answer()
 same => n,Playback(demo-echotest)
 same => n,Echo()
 same => n,Playback(demo-echodone)
 same => n,Hangup()
EOF
        echo "$END_MARK"
    } > "$body"

    # Entries: one prefix per remote code, injected into [gsm_in] (calls coming
    # from GSM) and [internal] (SIP softphones).
    entry="$(mktemp)"
    {
        echo "$BEGIN_MARK"
        for r in "${REMOTES[@]}"; do
            printf 'exten => _%s.,1,NoOp(=== indicatif %s → node %s ===)\n' "${WAN_IND[$r]}" "${WAN_IND[$r]}" "$r"
            printf ' same => n,Goto(wan_out,${EXTEN},1)\n'
        done
        # Our OWN code dialed in full: it must land here, not fall into the
        # void. wan_out strips it and hands back to the local routing.
        printf 'exten => _%s.,1,NoOp(=== indicatif local %s ===)\n' "$LOCAL_IND" "$LOCAL_IND"
        printf ' same => n,Goto(wan_out,${EXTEN},1)\n'
        echo "$END_MARK"
    } > "$entry"

    awk -v bf="$entry" '
        { print }
        $0 == "[gsm_in]" || $0 == "[internal]" {
            while ((getline l < bf) > 0) print l
            close(bf)
        }
    ' "$ext" > "${ext}.new" && mv "${ext}.new" "$ext"

    cat "$body" >> "$ext"
    ast_push "$i" /etc/asterisk/extensions.conf < "$ext"
    rm -f "$ext" "$body" "$entry"
    _inds=""; for r in "${REMOTES[@]}"; do _inds="${_inds}${_inds:+ }${WAN_IND[$r]}→node${r}"; done
    echo -e "  ${CYAN}Op${i}${NC} wan_out/wan_in · ${_inds}"
```

```
done
echo ""

# ═══════════════════════════════════════════════════════════════════════
# [6/6] SMS - sms-routing.conf: one remote operator per (node, operator)
# ═══════════════════════════════════════════════════════════════════════
# The relay (scripts/sms-interop-relay.py) routes by LONGEST-PREFIX on the
# dialed MSISDN. It is given the <code><op> prefixes, the ip:port target of
# the remote relay, and the number of code digits to STRIP before sending:
# the remote HLR only knows the bare number.
echo -e "${GREEN}[6/6] Routage SMS WAN...${NC}"
for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    conf="$(mktemp)"
    ast_pull "$i" /etc/osmocom/sms-routing.conf \
        | sed '/OSMO WAN MESH BEGIN/,/OSMO WAN MESH END/d' \
        | sed '/# ── WAN ROUTES/,/# ── FIN WAN/d' > "$conf"
    if [ ! -s "$conf" ]; then
        echo -e "  ${YELLOW}⚠${NC} Op${i} : sms-routing.conf absent - SMS WAN non configure"
        rm -f "$conf"; continue
    fi
    {
        echo ""
        echo "$SMS_BEGIN"
        echo "# Remote WAN nodes. Key = <w><node><op>, value = ip:port of the relay."
        echo "[operators]"
        for r in "${REMOTES[@]}"; do
            rip="${WAN_IP[$r]}"
            for j in $(seq 1 "$(node_nops "$r")"); do
                # Toward a backbone peer the remote relay is reached inside the
                # bridge: its port published on the host (7891 for operator 2)
                # does not lead there, it listens on the base port.
                rsms="$(peer_sms_port "$r" "$j")"

                printf 'w%s%s = %s:%s\n' "$r" "$j" "$rip" "$rsms"
            done
        done
        echo ""
        # The routing key is the dialed PREFIX:
        # <code><remote node>00<operator>, since the MSISDN starts with the
        # node number (an SMS to 11100101 must match a route).
        # strip removes the code only, which leaves the whole MSISDN.
        echo "# <code><node>00<op> → remote node; strip = code digits to remove."
        echo "[routes]"
        for r in "${REMOTES[@]}"; do
            for j in $(seq 1 "$(node_nops "$r")"); do
                printf '%s%s00%s = w%s%s strip=%s\n' "${WAN_IND[$r]}" "$r" "$j" "$r" "$j" "${#WAN_IND[$r]}"
            done
        done
        echo "$SMS_END"
    } >> "$conf"
    ast_push "$i" /etc/osmocom/sms-routing.conf < "$conf"
    rm -f "$conf"
    nsms=0
    for r in "${REMOTES[@]}"; do nsms=$(( nsms + $(node_nops "$r") )); done
    echo -e "  ${CYAN}Op${i}${NC} ${nsms} route(s) SMS WAN"
done
echo ""

# ── Asterisk restart ──────────────────────────────────────────────────
# The PJSIP transport changed: a "reload" is not enough. In native mode at
# bootstrap time Asterisk is not yet running - nothing is restarted then, the
# conf will be read at its start.
if [ "$NO_RESTART" -eq 1 ]; then
    echo -e "${YELLOW}--no-restart : Asterisk gardera l'ancienne conf jusqu'a son prochain demarrage.${NC}"
elif [ "$DRY" -eq 1 ]; then
```

```
    echo "  [dry-run] redemarrage Asterisk"
else
    for i in "${OP_IDS[@]}"; do
        mesh_set_context "$i"
        if [ "$MODE" = "docker" ]; then
            docker exec "$(op_container "$i")" bash -c "
                asterisk -rx 'core stop now' 2>/dev/null || pkill asterisk 2>/dev/null || true
                sleep 2; pkill -9 asterisk 2>/dev/null || true; sleep 1
                asterisk -f & disown" 2>/dev/null || true
            echo -e "  ${CYAN}Op${i}${NC} Asterisk redemarre"
        else
            if asterisk -rx 'core show uptime' 2>/dev/null | qgrep -i uptime; then
                asterisk -rx 'core restart now' >/dev/null 2>&1 || true
                echo -e "  ${CYAN}Op${i}${NC} Asterisk redemarre"
            else
                echo -e "  ${CYAN}Op${i}${NC} Asterisk non lance - conf prete pour son demarrage"
            fi
        fi
    done
fi

# ── Save + summary ──────────────────────────────────────────────────
if [ "$DRY" -eq 0 ]; then
    # Default 0: applying a table once must not make it permanent.
    # Only an ISO built with --wan freezes WAN_AUTO=1 (iso_modules/70-scripts.sh).
    WAN_AUTO="${WAN_AUTO:-0}" wan_nodes_save "${CONF:-$WAN_CONF_FILE}"
fi

echo ""
echo -e "${GREEN}${BOLD}WAN mesh applique.${NC}"
echo ""
# The node code must be dialed FOLLOWED BY the whole six-digit MSISDN:
# <node>00<operator><rank>. The example in the final summary must match a
# real pattern.
# LOCAL_IND and REMOTES keep the context of the LAST operator looped above;
# ours is set again before displaying, otherwise the line would announce a
# code that is not this node's.
mesh_set_context "${OP_IDS[0]}"
echo -e "  ${BOLD}Depuis ce noeud (${WAN_NODE_ID}, indicatif ${LOCAL_IND}) :${NC}"
for r in "${REMOTES[@]}"; do
    for j in $(seq 1 "$(node_nops "$r")"); do
        echo -e "    ${CYAN}${WAN_IND[$r]}${NC}${r}00${j}01  → MS ${r}00${j}01 (op${j}) du noeud ${r}
        ↪  (${WAN_IP[$r]})"
    done
done
echo ""
echo -e "  ${BOLD}Ports WAN (identiques sur chaque noeud) :${NC}"
for i in "${OP_IDS[@]}"; do
    mesh_set_context "$i"
    echo -e "    Op${i}: SIP ${CYAN}$(wan_sip_port "$i")${NC}/udp  RTP ${CYAN}$(wan_rtp_start "$i")-$(wan_rtp_end
    ↪  "$i")${NC}/udp  SMS ${CYAN}$(wan_sms_port "$i")${NC}/tcp"
done
echo ""
echo -e "  Table sauvegardee : ${CYAN}${CONF:-$WAN_CONF_FILE}${NC}"
echo -e "  Diagnostic SS7/WAN : ${CYAN}checks/wan_ss7_check.sh${NC}"
echo ""
```

2.127 /opt/GSM/osmo-operator/network/setup-wan-sms.sh

3941 octets, 98 lignes → 98 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════
# network/setup-wan-sms.sh - inter-server SMS routing
```

```
#
# Adds the SMS routing for prefix 66 to the sms-routing.conf table of each
# operator. SMS to 66XXXXX are relayed over TCP to the remote server's
# sms-interop-relay.py.
#
# Usage:
#   sudo ./network/setup-wan-sms.sh <remote_public_ip> [n_operators]
#
# Run on EACH server.
# ═══════════════════════════════════════════════════════════════════════

set -euo pipefail

GREEN='\033[0;32m'; CYAN='\033[0;36m'; NC='\033[0m'

REMOTE_IP="${1:-}"
N_OPS="${2:-2}"
WAN_PREFIX="66"
RELAY_PORT=7890
CONTAINER_PREFIX="osmo-operator-"

if [ -z "$REMOTE_IP" ]; then
    echo "Usage: sudo $0 <remote_public_ip> [n_operators]"
    exit 1
fi

echo -e "${GREEN}Configuration SMS WAN → ${REMOTE_IP}${NC}"
echo ""

for i in $(seq 1 "$N_OPS"); do
    cname="${CONTAINER_PREFIX}${i}"

    echo -e "  ${CYAN}Op${i}${NC} - Mise a jour sms-routing.conf..."

    # Add the WAN routes to sms-routing.conf
    docker exec "$cname" bash -c "
        CONF='/etc/osmocom/sms-routing.conf'

        # Remove the old WAN block if present
        sed -i '/# ── WAN ROUTES/,/# ── FIN WAN/d' \"\$CONF\" 2>/dev/null || true

        # Add the remote operators to [operators]
        # IDs 100+N are used for the remote operators
        cat >> \"\$CONF\" << SMSEOF

# ── WAN ROUTES - remote server ${REMOTE_IP} ──────────────────────────
# IDs 100+ designate the operators on the remote server.
# The relay uses the remote server's IP for the TCP connection.

SMSEOF

        # Add to the [operators] section if not already present
        for j in \$(seq 1 ${N_OPS}); do
            wan_op_id=\$((100 + j))
            if ! grep -q \"^\${wan_op_id} =\" \"\$CONF\"; then
                # Insert after the last line of [operators]
                sed -i \"/^\\[operators\\]/,/^\\[/ {
                    /^\\[routes\\]/i \${wan_op_id} = ${REMOTE_IP}
                }\" \"\$CONF\" 2>/dev/null || true
            fi
        done

        # Add the routes for the WAN prefix
        cat >> \"\$CONF\" << SMSEOF2
# WAN routes: prefix ${WAN_PREFIX}NXXXX → remote operator N
SMSEOF2
```

```
        for j in \$(seq 1 ${N_OPS}); do
            wan_op_id=\$((100 + j))
            # Prefix route: 66 + operator + number
            echo \"${WAN_PREFIX}\${j}0000 = \${wan_op_id}\" >> \"\$CONF\"
            for ms in \$(seq 1 8); do
                # <node>00<op><ms>: here j numbers the remote peer, so it is
                # both its node and its operator (one operator per node on
                # this path).

                msisdn=\$(( j * 100000 + j * 100 + ms ))
                echo \"${WAN_PREFIX}\${msisdn} = \${wan_op_id}\" >> \"\$CONF\"
            done
        done

        echo '# ── FIN WAN ──────────────────────────────────────────────────' >> \"\$CONF\"
    "

    echo -e "  ${GREEN}✓${NC} Op${i} SMS WAN routes ajoutees"
done

echo ""
echo -e "${GREEN}SMS WAN routing configure.${NC}"
echo -e "  SMS vers ${CYAN}${WAN_PREFIX}10001${NC} → serveur distant Op1 MS1"
echo -e "  SMS vers ${CYAN}${WAN_PREFIX}20001${NC} → serveur distant Op2 MS1"
echo ""
echo -e "  ${CYAN}Note :${NC} Le sms-interop-relay.py doit etre accessible"
echo -e "  depuis l'exterieur sur le port TCP ${RELAY_PORT}."
echo -e "  Firewall : ufw allow from ${REMOTE_IP} to any port ${RELAY_PORT} proto tcp"
```

2.128 /opt/GSM/osmo-operator/network/wan-nodes.sh

19784 octets, 408 lignes → 408 lignes

```
#!/bin/bash
# ================================================================================
# network/wan-nodes.sh - the WAN NODE TABLE, shared by the 3 launchers
# ================================================================================
#
# This file only defines functions and variables: it is SOURCED by
#   start-docker.sh          (docker host, N operators per node)
#   start-native.sh   (native / ISO, 1 operator per node)
#   build-iso.sh      (freezes the table into the ISO)
#   network/setup-wan-mesh.sh  (applies it to Asterisk / iptables / SMS)
#
# ── MODEL ──────────────────────────────────────────────────────────────────
# A WAN = N nodes. A node = (id, public ip, dialing code).
#
# The DIALING CODE is the node's telephone prefix, exactly like a country
# code: from any node, dialing
#       <target_code><number>
# reaches <number> on the node that carries that code. The sending node strips
# the code before going out on the WAN; the receiving node thus gets the bare
# number, the one its HLR and its dialplan already know.
#
# WAN_NODE_ID says WHICH node is the local one. It is the only thing that tells
# "this number is mine" from "this number leaves on the WAN": without it, a
# node would bounce its own calls back to itself forever.
#
# ── WHY A TABLE AND NOT A local/remote PAIR ────────────────────────────────
# network/setup-wan-interop.sh only knows TWO servers: LOCAL_IP and REMOTE_IP.
# On every call it rewrites the "WAN INTEROP" block of pjsip.conf/
# extensions.conf and resets its iptables chain, so looping it over three
# peers leaves only the LAST one. Hence this table: ONE block containing all
```

```
# the peers is generated in a single pass.
# ------------------------------------------------------------------------

# Table: WAN_IP[id] / WAN_IND[id] / WAN_NOPS[id], plus the ordered list of ids.
declare -A WAN_IP  2>/dev/null || true
declare -A WAN_IND 2>/dev/null || true
# WAN_NOPS[id]: how many operators node id carries. WAN_OPS only describes the
# LOCAL node, so the mesh generator would otherwise have to ASSUME the others
# are identical to itself. The docker host carries two, a VM only one: from the
# host, a trunk, a _<code>2XXXX pattern and an SMS route that nobody serves
# would be built toward the VM - letting calls leave that arrive nowhere.
# Hence this per-node count, fourth field of the table.
declare -A WAN_NOPS 2>/dev/null || true
WAN_NODE_LIST=()
: "${WAN_NODE_COUNT:=0}"
: "${WAN_NODE_ID:=0}"
: "${WAN_OPS:=1}"          # operators (PLMN) carried by THIS node
: "${WAN_CONF_FILE:=/etc/osmo-wan.conf}"
# WAN_DOCKER_HOST: the LAN address of the host carrying docker, the one through
# which a VM routes to the container backbone. It is DHCP-assigned and can
# change: it is DISCOVERED at run time (network/set_ip_vm.sh) and is never
# frozen here - the table only carries it from one machine to another.
: "${WAN_DOCKER_HOST:=}"

_wanc() { [ -t 1 ] && printf '\033[0;36m%s\033[0m' "$1" || printf '%s' "$1"; }
_wan_warn() { printf '  \033[1;33m⚠\033[0m %s\n' "$*" >&2; }
_wan_err()  { printf '  \033[0;31m✗\033[0m %s\n' "$*" >&2; }

# ── Default dialing codes ──────────────────────────────────────────────
# 11/22/.../99: one doublet per node, none is a prefix of another (hard
# condition, cf. wan_nodes_validate) and there are exactly nine - the node
# bound. The doublet also reads as the node number, which makes a call log
# readable without the table at hand.
_WAN_DEFAULT_IND=(11 22 33 44 55 66 77 88 99)
wan_default_ind() {
    local i=$1
    if [ "$i" -ge 1 ] && [ "$i" -le "${#_WAN_DEFAULT_IND[@]}" ]; then
        echo "${_WAN_DEFAULT_IND[$((i-1))]}"
    else
        echo $(( 100 + i ))
    fi
}

# ── Serialisation "id:ip:code[:operators] ..." ─────────────────────────
# The operator count must go through here, otherwise it does not survive the
# first round trip: wan_nodes_save serialises with this function, and a table
# read back would announce 1 operator everywhere. It is only written when it
# says something other than 1, so that everything already reading WAN_NODES
# with three fields (build-iso.sh, set_stp_ip.sh, the diagnostics) keeps
# seeing the table it knows.
wan_nodes_spec() {
    local id out="" nops
    for id in "${WAN_NODE_LIST[@]}"; do
        out="${out}${out:+ }${id}:${WAN_IP[$id]}:${WAN_IND[$id]}"
        nops="${WAN_NOPS[$id]:-1}"
        [ "$nops" = "1" ] || out="${out}:${nops}"
    done
    printf '%s' "$out"
}

wan_nodes_parse() {
    local spec="${1:-}" entry id rest ip ind nops
    WAN_IP=(); WAN_IND=(); WAN_NOPS=(); WAN_NODE_LIST=()
    spec="${spec//,/ }"
    for entry in $spec; do
        [ -n "$entry" ] || continue
```

```
        case "$entry" in
            *:*:*) ;;
            *) _wan_err "entree noeud invalide : '$entry' (attendu id:ip:indicatif[:operateurs])"; return 1 ;;
        esac
        id="${entry%%:*}"; rest="${entry#*:}"
        ip="${rest%%:*}";  ind="${rest#*:}"
        ind="${ind%%:*}"
        # Optional fourth field. Three-field tables (the ISO's, the one already
        # in /etc/osmo-wan.conf) must keep parsing: a node without a count
        # carries one operator, which is the case of a VM.
        nops=1
        case "$entry" in
            *:*:*:*) nops="${rest#*:}"; nops="${nops#*:}"; nops="${nops%%:*}" ;;
        esac
        [ -n "$nops" ] || nops=1
        # A field that is present but unreadable is not silently folded to 1:
        # that would be a mesh deprived of operators 2..n without a word in
        # the log.
        if ! [[ "$nops" =~ ^[1-9][0-9]*$ ]]; then
            _wan_err "noeud $id : '$nops' operateurs - attendu un entier >= 1"; return 1
        fi
        WAN_IP[$id]="$ip"; WAN_IND[$id]="$ind"; WAN_NOPS[$id]="$nops"
        WAN_NODE_LIST+=("$id")
    done
    WAN_NODE_COUNT=${#WAN_NODE_LIST[@]}
    [ "$WAN_NODE_COUNT" -gt 0 ] || { _wan_err "table de noeuds vide"; return 1; }
    return 0
}

# ── Validation ──────────────────────────────────────────────────────
# Rejects what would produce silently wrong routing, not only what is
# malformed.
wan_nodes_validate() {
    local id other rc=0
    [ "${#WAN_NODE_LIST[@]}" -ge 1 ] || { _wan_err "table de noeuds vide"; return 1; }
    for id in "${WAN_NODE_LIST[@]}"; do
        # 1..9: one digit. This keeps <code><operator><MSISDN> readable and
        # the Asterisk patterns free of length ambiguity.
        [[ "$id" =~ ^[1-9]$ ]] \
            || { _wan_err "numero de noeud invalide : '$id' (1 a 9)"; rc=1; }
        [[ "${WAN_IP[$id]}" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] \
            || { _wan_err "noeud $id : '${WAN_IP[$id]}' n'est pas une IPv4"; rc=1; }
        # ── THE CODE MUST NOT OVERLAP THE LOCAL NUMBERING PLAN ──
        # The repo's MSISDNs are <node>00<op><ms> (100101, 200101...): six
        # digits whose first two are "<node>0". A code "10" would thus produce
        # "exten => _10." in [gsm_in], which would swallow 100101 - the local
        # subscriber would become unreachable and the call would leave on the
        # WAN. A ONE-digit code has the same flaw, worse.
        # The defaults 11/22/.../99 all pass this rule; the "^[1-9]0" rule
        # below is what rejects 10, 20, ... 90.
        if ! [[ "${WAN_IND[$id]}" =~ ^[0-9]{2,4}$ ]]; then
            _wan_err "noeud $id : indicatif '${WAN_IND[$id]}' - 2 a 4 chiffres (1 chiffre entrerait en conflit
             ↪  avec les MSISDN locaux)"; rc=1
        elif [[ "${WAN_IND[$id]}" =~ ^[1-9]0 ]]; then
            _wan_err "noeud $id : indicatif '${WAN_IND[$id]}' commence comme un MSISDN local (<noeud>00<op><ms> →
             ↪  100101, 200101...) - il capterait les appels locaux ; prenez
             ↪  ${WAN_IND[$id]:0:1}${WAN_IND[$id]:0:1} par exemple"; rc=1
        fi
    done
    # Codes: unique AND none a prefix of another. A code 3 that prefixes a
    # code 33 makes the Asterisk patterns (_3. vs _33.) and the SMS relay's
    # longest-prefix match order-dependent - hence unpredictable.
    for id in "${WAN_NODE_LIST[@]}"; do
        for other in "${WAN_NODE_LIST[@]}"; do
            [ "$id" = "$other" ] && continue
            if [ "${WAN_IND[$id]}" = "${WAN_IND[$other]}" ]; then
```

```
                _wan_err "noeuds $id et $other partagent l'indicatif ${WAN_IND[$id]}"; rc=1
            elif [ "${WAN_IND[$other]#${WAN_IND[$id]}}" != "${WAN_IND[$other]}" ]; then
                _wan_err "indicatif ${WAN_IND[$id]} (noeud $id) est un prefixe de ${WAN_IND[$other]} (noeud
                ↪  $other)"; rc=1
            fi
            if [ "${WAN_IP[$id]}" = "${WAN_IP[$other]}" ]; then
                _wan_err "noeuds $id et $other partagent l'IP ${WAN_IP[$id]}"; rc=1
            fi
        done
    done
    if [ -n "${WAN_NODE_ID:-}" ] && [ "${WAN_NODE_ID:-0}" != "0" ]; then
        [ -n "${WAN_IP[$WAN_NODE_ID]:-}" ] \
            || { _wan_err "le noeud local ($WAN_NODE_ID) n'est pas dans la table"; rc=1; }
    fi
    return $rc
}

# ── Local node detection by its addresses ──────────────────────────────
# A SINGLE ISO can then serve the N machines: each recognises itself by its
# IP. Without this, one image per node would be needed.
wan_nodes_detect_self() {
    local id addrs
    addrs="$(ip -4 -o addr show scope global 2>/dev/null | awk '{print $4}' | cut -d/ -f1)"
    [ -n "$addrs" ] || return 1
    for id in "${WAN_NODE_LIST[@]}"; do
        if printf '%s\n' $addrs | grep -qx "${WAN_IP[$id]}"; then
            WAN_NODE_ID="$id"; return 0
        fi
    done
    return 1
}

# ── THIS NODE'S ADDRESS ON THE WAN SEGMENT ─────────────────────────────
# "The first global IP" is a poor criterion on a bench VM: there are at least
# three, and two are traps.
#   • the VirtualBox NAT - always 10.0.2.x - reaches the Internet but sees no
#     peer: an ASP bound on it never reaches the hub;
#   • the aliases 172.20.0.1 / 172.20.0.11 / 172.20.1.10 that the ISO itself
#     puts on the NIC (build-iso.sh, 20-dhcp.network) - they belong to the
#     docker plan and exist on no segment.
# The right address is the VirtualBox "bridged" adapter's. It is searched in
# this order, from the surest to the most inferred.
wan_detect_local_ip() {
    local addrs a

    addrs="$(ip -4 -o addr show scope global 2>/dev/null | awk '{print $4}' | cut -d/ -f1)"
    [ -n "$addrs" ] || return 1

    # 1. The table is authoritative - provided the address is really up.
    #    A table announcing an absent address is a stale table, not a source
    #    of authority.
    if [ "${WAN_NODE_ID:-0}" != "0" ] && [ -n "${WAN_IP[${WAN_NODE_ID}]:-}" ]; then
        for a in $addrs; do
            [ "$a" = "${WAN_IP[$WAN_NODE_ID]}" ] && { printf '%s' "$a"; return 0; }
        done
    fi

    # 2. The VirtualBox host-only range (192.168.56.0/21), the only one it
    #    allows without /etc/vbox/networks.conf. This is the bench segment.
    for a in $addrs; do
        case "$a" in
            192.168.5[6-9].*|192.168.6[0-3].*) printf '%s' "$a"; return 0 ;;
        esac
    done

    # 3. Any other 192.168.x address: this is the "bridged" adapter, the one
```

```
    #    that puts the VM on the same network as the host. This is the
    #    address through which the peers reach it.
    for a in $addrs; do
        case "$a" in 192.168.*) printf '%s' "$a"; return 0 ;; esac
    done

    # 4. Failing that, the first address that is neither the VirtualBox NAT,
    #    nor a docker-plan alias, nor the loopback.
    for a in $addrs; do
        case "$a" in
            10.0.2.*|172.20.*|127.*|169.254.*) ;;
            *) printf '%s' "$a"; return 0 ;;
        esac
    done

    return 1
}

wan_local_ip()  { printf '%s' "${WAN_IP[${WAN_NODE_ID}]:-}"; }
wan_local_ind() { printf '%s' "${WAN_IND[${WAN_NODE_ID}]:-}"; }

# ── Update of THIS node in the table, without prompting ──────────────
# --node N says which node this machine is. Its address in the table may be
# stale (a table copied from another machine): it is replaced by the real
# LOCAL address - the one that routes to the hub if known, otherwise
# wan_detect_local_ip - then /etc/osmo-wan.conf is rewritten. The other nodes
# keep their entry: this script does not know where they are.
#   $1 = node number (1-9)   $2 = hub (optional)
wan_self_refresh() {
    local id="$1" hub="${2:-}" ip="" ind

    [[ "$id" =~ ^[1-9]$ ]] || { _wan_err "wan_self_refresh : noeud '$id' invalide"; return 1; }
    WAN_NODE_ID="$id"

    if [ -n "$hub" ]; then
        ip="$(ip route get "$hub" 2>/dev/null | sed -n 's/.*src \([0-9.]*\).*/\1/p' | head -1)"
    fi
    [ -n "$ip" ] || ip="$(wan_detect_local_ip 2>/dev/null || true)"
    if [ -z "$ip" ]; then
        _wan_err "noeud $id : aucune adresse locale detectee - table non reecrite"
        return 1
    fi

    ind="${WAN_IND[$id]:-$(wan_default_ind "$id")}"
    WAN_IP[$id]="$ip"
    WAN_IND[$id]="$ind"
    WAN_NOPS[$id]="${WAN_NOPS[$id]:-1}"
    case " ${WAN_NODE_LIST[*]:-} " in
        *" $id "*) ;;
        *) WAN_NODE_LIST+=("$id") ;;
    esac
    WAN_NODE_COUNT="${#WAN_NODE_LIST[@]}"

    wan_nodes_save "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" || return 1
    printf '  \033[0;36mWAN\033[0m        noeud %s = %s (table reecrite : %s)\n' \
        "$id" "$ip" "${WAN_CONF_FILE:-/etc/osmo-wan.conf}"
    return 0
}

# ── Input ────────────────────────────────────────────────────────────
# whiptail when available AND a terminal is present; otherwise `read`. The
# "neither" mode (cron, CI, docker exec -d) invents nothing: it fails, naming
# the variables to set.
_wan_ask() {   # title  question  default
    local title="$1" q="$2" def="${3:-}" ans
    if [ -t 0 ] && [ -t 2 ] && command -v whiptail >/dev/null 2>&1 && [ -z "${WAN_NO_WHIPTAIL:-}" ]; then
```

```
        ans=$(whiptail --backtitle "osmo-operator - WAN" --title "$title" \
                --inputbox "$q" 10 72 "$def" 3>&1 1>&2 2>&3) || return 1
    elif [ -t 0 ]; then
        printf '  %s [%s] : ' "$q" "$def" >&2
        read -r ans || return 1
    else
        _wan_err "pas de terminal : renseignez WAN_NODES / WAN_NODE_ID / WAN_OPS"
        return 1
    fi
    printf '%s' "${ans:-$def}"
}

# wan_nodes_prompt [local_ops_count]
#   1. how many nodes
#   2. for EACH node: its public IP, then its dialing code
#   3. which number the node built by this run carries
wan_nodes_prompt() {
    local n i ip ind me auto_ip def_n
    def_n="${WAN_NODE_COUNT:-0}"; [ "$def_n" -lt 1 ] && def_n=3

    n=$(_wan_ask "WAN - noeuds" "Nombre de noeuds du WAN (1-9) :" "$def_n") || return 1
    [[ "$n" =~ ^[1-9]$ ]] \
        || { _wan_err "nombre de noeuds invalide : '$n' (1 a 9)"; return 1; }

    auto_ip=$(ip route get 1.1.1.1 2>/dev/null | awk '{for(i=1;i<=NF;i++) if($i=="src") print $(i+1)}' | head -1)
    [ -z "$auto_ip" ] && auto_ip=$(hostname -I 2>/dev/null | awk '{print $1}')

    # ${x[@]+"${x[@]}"}: under `set -u`, an EMPTY array expanded without this
    # guard is a fatal error on bash < 4.4 - and the first input starts
    # precisely from an empty table.
    local -a old_list=(${WAN_NODE_LIST[@]+"${WAN_NODE_LIST[@]}"})
    local -A old_ip old_ind
    for i in "${old_list[@]}"; do old_ip[$i]="${WAN_IP[$i]}"; old_ind[$i]="${WAN_IND[$i]}"; done
    WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=()

    for i in $(seq 1 "$n"); do
        ip=$(_wan_ask "WAN - noeud $i/$n" "IP publique du noeud $i :" "${old_ip[$i]:-}") || return 1
        ind=$(_wan_ask "WAN - noeud $i/$n" "Indicatif du noeud $i (prefixe d'appel) :" \
                       "${old_ind[$i]:-$(wan_default_ind "$i")}") || return 1
        WAN_IP[$i]="$ip"; WAN_IND[$i]="$ind"; WAN_NODE_LIST+=("$i")
    done
    WAN_NODE_COUNT="$n"

    # LOCAL node number. The one whose IP is ours is proposed.
    me="${WAN_NODE_ID:-0}"
    if [ "$me" = "0" ] && [ -n "$auto_ip" ]; then
        for i in "${WAN_NODE_LIST[@]}"; do [ "${WAN_IP[$i]}" = "$auto_ip" ] && me="$i"; done
    fi
    [ "$me" = "0" ] && me=1
    me=$(_wan_ask "WAN - ce noeud" "Numero du noeud construit par ce lancement (1-$n) :" "$me") || return 1
    [[ "$me" =~ ^[0-9]+$ ]] && [ -n "${WAN_IP[$me]:-}" ] \
        || { _wan_err "noeud local '$me' hors table"; return 1; }
    WAN_NODE_ID="$me"

    wan_nodes_validate || return 1
    return 0
}

# ── Persistance ──────────────────────────────────────────────────────
wan_nodes_save() {
    local f="${1:-$WAN_CONF_FILE}" host="${WAN_DOCKER_HOST:-}"
    mkdir -p "$(dirname "$f")"
    # The docker host is only known to whoever just discovered it. Since this
    # file is rewritten WHOLE on every save, a save made by a script without it
    # in its environment would erase the address - the VM's only hint of how
    # to reach the backbone. The one already in the file is therefore read
```

```
    # back before writing.
    if [ -z "$host" ] && [ -r "$f" ]; then
        host="$(sed -n 's/^WAN_DOCKER_HOST=//p' "$f" | tr -d "\"' " | tail -1)"
    fi
    # WAN_AUTO_SAVE, when set, takes precedence over WAN_AUTO: reading the table
    # back (wan_nodes_load sources it) puts WAN_AUTO=1 into the shell, and every
    # later save would rewrite it. start-native.sh sets it: 1 only if WAN_AUTO=1
    # itself brought the WAN up (node of an ISO built with --wan), 0 for a
    # --wan typed by hand.
    cat > "$f" <<EOF
# /etc/osmo-wan.conf - WAN node table (generated by network/wan-nodes.sh)
# Format: WAN_NODES="id:ip:code[:operators] ..."  (operators: default 1)
WAN_NODES="$(wan_nodes_spec)"
WAN_NODE_COUNT=${WAN_NODE_COUNT}
WAN_NODE_ID=${WAN_NODE_ID}
WAN_OPS=${WAN_OPS}
# WAN_AUTO=1: the launchers apply the WAN without passing --wan again.
WAN_AUTO=${WAN_AUTO_SAVE:-${WAN_AUTO:-0}}
EOF
    # Written only when known: an empty line would be read as a gateway, and
    # an empty gateway cannot be told from a wrong one when the route is set.
    if [ -n "$host" ]; then
        {
            printf '# Host carrying docker, through which a VM routes to the container\n'
            printf '# backbone. DHCP address: it is rediscovered, never frozen.\n'
            printf 'WAN_DOCKER_HOST="%s"\n' "$host"
        } >> "$f"
    fi
    return 0
}

wan_nodes_load() {
    local f="${1:-$WAN_CONF_FILE}"
    [ -r "$f" ] || return 1
    # Sourcing brings back WAN_NODES, the local node, and WAN_DOCKER_HOST when
    # the file carries it - it is how a VM knows where to route 172.20.
    # shellcheck disable=SC1090
    . "$f"
    wan_nodes_parse "${WAN_NODES:-}" || return 1
    return 0
}

wan_nodes_summary() {
    local id tag
    printf '  Noeuds WAN : %s   (ce noeud : %s)\n' "$WAN_NODE_COUNT" "$(_wanc "${WAN_NODE_ID}")"
    for id in "${WAN_NODE_LIST[@]}"; do
        tag="   "; [ "$id" = "${WAN_NODE_ID}" ] && tag="◄ ce noeud"
        printf '    node %-2s  %-15s  indicatif %-4s %s\n' "$id" "${WAN_IP[$id]}" "${WAN_IND[$id]}" "$tag"
    done
    printf '    Composer %s + numero pour joindre le noeud correspondant.\n' \
        "$(_wanc "<indicatif>")"
}
```

2.129 /opt/GSM/osmo-operator/opt-gsm/grgsm_fft_live.py

2724 octets, 62 lignes → 62 lignes

```
#!/usr/bin/env python3
# grgsm_fft_live.py — LIVE FFT (ASCII spectrum). Reads the live FIFO
# (default /tmp/iq_asciifft.fifo, fc32 4 SPS) that qemu pushes frame by frame,
# OR a cfile (seek-tail) as fallback. Marks the FCCH peak.
#
# FIFO: opened O_RDWR|O_NONBLOCK -> non-blocking open + no EOF (a write-end is
# kept) => qemu's non-blocking write always finds a reader. Every turn DRAINS
```

```
# the backlog and keeps only the last N samples (the freshest), so no lag
# accumulates.
import numpy as np, time, os, stat
CFILE=os.environ.get("CFILE","/tmp/iq_asciifft.fifo")
FS=float(os.environ.get("FS","1083333"))    # 4 SPS relay (FIFO). BSP cfile 1 SPS=270833
N=int(os.environ.get("NFFT","4096"))
COLS=70
FCCH=1625000.0/24                           # +67708 Hz

def _is_fifo(p):
    try: return stat.S_ISFIFO(os.stat(p).st_mode)
    except OSError: return False

fifo_fd=None
if _is_fifo(CFILE) or CFILE.endswith(".fifo"):
    if not os.path.exists(CFILE): os.mkfifo(CFILE,0o666)
    fifo_fd=os.open(CFILE, os.O_RDWR|os.O_NONBLOCK)

def _read_iq():
    if fifo_fd is not None:
        buf=b""
        while True:
            try: c=os.read(fifo_fd, 1<<16)
            except BlockingIOError: break
            if not c: break
            buf+=c
            if len(buf) > N*8*4: buf=buf[-(N*8):]     # keep the fresh part, bound RAM
        buf=buf[-(N*8):]
        if len(buf) < N*8: return np.array([],dtype=np.complex64)
        return np.frombuffer(buf[:(len(buf)//8)*8], dtype=np.complex64)
    try: sz=os.path.getsize(CFILE)
    except OSError: sz=0
    if sz < N*8: return np.array([],dtype=np.complex64)
    with open(CFILE,"rb") as f:
        f.seek(max(0,(sz//8-N)*8)); return np.frombuffer(f.read(N*8),dtype=np.complex64)

while True:
    iq=_read_iq()
    os.system("clear")
    print("== FFT LIVE %s (%s, fs=%.0f, N=%d) ==  FCCH attendu %+.0f Hz"%(
        CFILE, "FIFO" if fifo_fd is not None else "file", FS, N, FCCH))
    if len(iq) < N:
        print("\n  pas (encore) de donnees -> qemu ecrit-il le FIFO ? (relance ./run.sh)")
        time.sleep(1); continue
    sp=np.abs(np.fft.fftshift(np.fft.fft(iq*np.hanning(len(iq)))))
    fr=np.fft.fftshift(np.fft.fftfreq(len(iq),1/FS))
    pk=fr[np.argmax(sp)]; mag=np.abs(iq)
    print("  ampl moy=%.5f max=%.4f | PIC %+.0f Hz | dphi std=%.2f (GMSK~1.57)\n"%(
        mag.mean(),mag.max(),pk,np.std(np.angle(iq[1:]*np.conj(iq[:-1])))))
    B=24; step=len(sp)//B; spb=np.array([sp[i*step:(i+1)*step].max() for i in range(B)])
    frb=np.array([fr[i*step] for i in range(B)]); m=spb.max()+1e-9
    for i in range(B):
        bar=int(COLS*spb[i]/m); tag=" <-FCCH" if abs(frb[i]-FCCH)<FS/B else ""
        print("%+7.0f|%s%s"%(frb[i],"#"*bar,tag))
    time.sleep(0.5)
```

2.130 /opt/GSM/osmo-operator/opt-gsm/record_drain.py

5331 octets, 118 lignes → 118 lignes

```
#!/usr/bin/env python3
# record_drain.py — drains the live FIFO iq_record.fifo -> record.cfile as a
# RING (128 MB, fseek wrap), OUTSIDE the qemu hot path. qemu only does a
# non-blocking write() to the FIFO; this (independent) process absorbs the
```

```
# stream and writes it to disk at its own pace. grgsm_decode -c reads
# record.cfile back offline => SI preserved.
#
# The FIFO is opened O_RDWR: it does not block at open AND never sees EOF (a
# write-end is kept here), so qemu (O_WRONLY|O_NONBLOCK) always finds a
# reader => its open succeeds and the stream flows.
#
# ============================================================================
# FULL, TRIGGERABLE CAPTURE — next to the ring, not in its place
# ============================================================================
# The ring above rewinds every 128 MB (~15.5 s): fine to replay the immediate
# past, unusable to capture a whole call. The full capture below NEVER
# rewinds.
#
# WHY A DEDICATED PIPE, and not a read of iq_record.fifo: a FIFO cannot be
# broadcast — two readers SHARE the bytes, neither sees them all. Draining
# iq_record.fifo here would steal part of the stream from the ring and break
# SI decoding. An 8th pipe is added to the relay (CALYPSO_RELAY_FIFOS), fed
# by qemu like the others.
# ⚠ RELAY_NFIFO_MAX is 8 in qemu_wrap.c and 7 pipes are already declared:
# this pipe takes THE LAST SLOT. Adding a 9th requires raising the constant
# AND recompiling, otherwise it is SILENTLY ignored.
#
# ⚠ NOT ON TMPFS. /dev/shm and /tmp are RAM. The stream is 1083333 samples/s
# x 8 B = 8.7 MB/s = 520 MB/min = 31 GB/h: a full capture on tmpfs exhausts
# the machine's memory in minutes. The default therefore points to the
# overlay (disk). Check `df -h` before a long capture.
#
# TRIGGER: the capture only writes AS LONG AS the trigger file exists.
#     touch /tmp/iq_full.trigger    -> starts (opens/truncates the .cfile)
#     rm    /tmp/iq_full.trigger    -> stops and CLOSES the file cleanly
# Without the trigger the pipe is still READ and discarded: otherwise the
# relay's memory ring fills up and qemu starts dropping whole frames for ALL
# pipes.
#
# CEILING: at CALYPSO_RECORD_FULL_MAX bytes the capture STOPS and SAYS SO. It
# does not rewind — a file silently biting its own tail is exactly what must
# be avoided here.
import os, sys, threading

FIFO = os.environ.get("CALYPSO_RECORD_FIFO", "/tmp/iq_record.fifo")
OUT  = os.environ.get("CALYPSO_RECORD_FILE", "/tmp/record.cfile")
RING = int(os.environ.get("CALYPSO_RECORD_RING", str(128 << 20)))   # 128 MB

FULL_ON      = os.environ.get("CALYPSO_RECORD_FULL", "0") == "1"
FULL_FIFO    = os.environ.get("CALYPSO_RECORD_FULL_FIFO", "/tmp/iq_full.fifo")
FULL_FILE    = os.environ.get("CALYPSO_RECORD_FULL_FILE", "/opt/GSM/captures/full.cfile")
FULL_TRIGGER = os.environ.get("CALYPSO_RECORD_FULL_TRIGGER", "/tmp/iq_full.trigger")
FULL_MAX     = int(os.environ.get("CALYPSO_RECORD_FULL_MAX", str(8 << 30)))   # 8 GB

def log(msg):
    sys.stderr.write(msg + "\n")
    sys.stderr.flush()

def full_drain():
    """Full capture: always reads, only writes under trigger, never wraps."""
    if not os.path.exists(FULL_FIFO):
        os.mkfifo(FULL_FIFO, 0o666)
    os.makedirs(os.path.dirname(FULL_FILE) or ".", exist_ok=True)
    fd = os.open(FULL_FIFO, os.O_RDWR)       # same idiom as the ring: no block, no EOF
    log("[record-full] %s -> %s (plafond %d MB) — declencheur : %s"
        % (FULL_FIFO, FULL_FILE, FULL_MAX >> 20, FULL_TRIGGER))
    out = None
    w = 0
    plafond_dit = False
    while True:
```

```
        b = os.read(fd, 1 << 16)
        if not b:
            continue
        armed = os.path.exists(FULL_TRIGGER)
        if armed and out is None:
            out = open(FULL_FILE, "wb", buffering=1 << 20)
            w = 0
            plafond_dit = False
            log("[record-full] DEMARRE -> %s" % FULL_FILE)
        elif not armed and out is not None:
            out.flush(); out.close(); out = None
            log("[record-full] ARRETE : %s, %.1f MB (%.1f s d'IQ)"
                % (FULL_FILE, w / 1048576.0, w / 8666664.0))
        if out is None:
            continue                          # discarded, but READ: the relay does not clog
        if w + len(b) > FULL_MAX:
            if not plafond_dit:
                log("[record-full] PLAFOND %d MB atteint : capture ARRETEE (pas de "
                    "rembobinage). Retirer %s, vider %s, puis relancer."
                    % (FULL_MAX >> 20, FULL_TRIGGER, FULL_FILE))
                plafond_dit = True
            out.flush(); out.close(); out = None
            continue
        out.write(b)
        w += len(b)
        out.flush()

if not os.path.exists(FIFO):
    os.mkfifo(FIFO, 0o666)

if FULL_ON:
    threading.Thread(target=full_drain, daemon=True).start()
else:
    log("[record-full] inactif (CALYPSO_RECORD_FULL != 1)")

fd = os.open(FIFO, os.O_RDWR)             # O_RDWR: no blocking / no EOF
out = open(OUT, "wb", buffering=1 << 20)
w = 0
log("[record-drain] %s -> %s (ring %d MB)" % (FIFO, OUT, RING >> 20))
while True:
    b = os.read(fd, 1 << 16)              # 64 KB
    if not b:
        continue
    out.write(b)
    w += len(b)
    if w >= RING:                         # RING: rewind (grgsm reads again from 0)
        out.seek(0); w = 0
    out.flush()
```

2.131 /opt/GSM/osmo-operator/opt-gsm/si_bridge.py

18731 octets, 389 lignes → 389 lignes

```
#!/usr/bin/env python3
# Bridge: grgsm_decode -v (DECODES the real SI of the continuous stream) ->
# parses the SI lines (PD=0x06) -> GSMTAP 16B + L2 -> 4730 (shunt feed_si ->
# DATA_IND -> mobile). The real SCH (grgsm patch) -> UDP 4731 (feed_sb).
#
# FN HANDLING:
#   - the real FN comes from the SCHBSIC <bsic> <fn> <toa> lines (reliable
#     clock), or from an explicit "FN: <n>" if grgsm -v prints one.
#   - it is written in the frame_number field of the GSMTAP sent to feed_si.
#   - reminder, 51-multiframe non-combined BCCH TS0 DL (FN % 51):
#       0,10,20,30,40 = FCCH | 1,11,21,31,41 = SCH | 2-5 = BCCH
```

```
#       6-9,12-19,22-29,32-39,42-49 = CCCH (PCH/AGCH) | 50 = idle
#     -> 31 = SCH position, 32 = start of a CCCH block, 51 = multiframe length.
import subprocess, socket, struct, re, sys, os, threading, time

ARFCN = int(os.environ.get("CALYPSO_CCCH_ARFCN", "514"))
CF = sys.argv[1] if len(sys.argv) > 1 else "/tmp/iq_grgsm.fifo"
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

SI = {0x1b:"SI3",0x1a:"SI2",0x1c:"SI4",0x19:"SI1",0x1d:"SI2bis",0x1e:"SI2ter",0x06:"RR"}  # NB: 0x21 = PAGING
 ↪  (not SI13=0x00)
# CCCH/AGCH downlink (GATE 3a): the IMM ASSIGN must be forwarded to the mobile,
# not only the SIs. gr-gsm decodes it (06 3f).
CCCH = {0x3f:"IMM-ASSIGN", 0x39:"IMM-ASSIGN-EXT", 0x3a:"IMM-ASSIGN-REJ"}
# Paging Request (PCH): type1=0x21 (== GSM48_MT_RR_PAG_REQ_1, NOT SI13), type2=0x22, type3=0x24.
PAGING = {0x21:"PAG-REQ-1", 0x22:"PAG-REQ-2", 0x24:"PAG-REQ-3"}
GSMTAP_BCCH = 0x01
GSMTAP_AGCH = 0x04
GSMTAP_SDCCH4 = 0x07
GSMTAP_SACCH  = 0x07 | 0x80   # SDCCH/4 + ACCH = dedicated SACCH SS0 (real SI5/SI6)

def fn51_role(fn):
    m = fn % 51
    if m in (0,10,20,30,40): return "FCCH"
    if m in (1,11,21,31,41): return "SCH"
    if m in (2,3,4,5):       return "BCCH"
    if m == 50:              return "idle"
    return "CCCH"

def gsmtap(fn, chan, l2):
    # v2, hdr_len=4 (16B), type=UM(1), ts=0, arfcn, signal_dbm=0 (NOTHING injected),
    # snr=0, frame_number=fn (REAL, decoded), sub_type=chan, antenna=0, sub_slot=0, res=0
    hdr = struct.pack(">BBBBHbbIBBBB", 2, 4, 0x01, 0, ARFCN & 0xffff,
                      0, 0, fn & 0xffffffff, chan, 0, 0, 0)
    return hdr + l2

n = 0; nsch = 0; nsd = 0; nsa = 0; last_fn = 0; last_sd_fn = -1; last_sa_fn = -1

def handle_line(line):
    global n, nsch, nsd, nsa, last_fn, last_sd_fn, last_sa_fn
    # --- real SCH: "SCHBSIC <bsic> <fn> <toa>" -> 4731 + FN clock ---
    msch = re.search(r"SCHBSIC\s+(\d+)\s+(\d+)\s+(-?\d+)", line)
    if msch:
        bsic, fn, toa = int(msch.group(1)), int(msch.group(2)), int(msch.group(3))
        last_fn = fn
        s.sendto(b"SCH1" + struct.pack("<iii", bsic, fn, toa), ("127.0.0.1", 4731))
        nsch += 1
        if nsch <= 10 or nsch % 100 == 0:
            print("[si-bridge] SCH -> 4731 : BSIC=%d (ncc=%d bcc=%d) FN=%d (%%51=%d %s) TOA=%d  #%d"
                  % (bsic,(bsic>>3)&7,bsic&7,fn,fn%51,fn51_role(fn),toa,nsch), flush=True)
        return
    # --- optional explicit FN in the -v line ---
    mfn = re.search(r"\bFN[:=]?\s*(\d+)", line)
    if mfn: last_fn = int(mfn.group(1))
    # --- SI hex line ---
    lstr = line.strip()
    m = re.search(r":\s+([0-9a-fA-F][0-9a-fA-F ]+)$", lstr)
    if not m: return
    try: by = bytes(int(x,16) for x in m.group(1).split())
    except: return
    if len(by) >= 3 and by[1] == 0x06 and (by[2] in SI or by[2] in CCCH or by[2] in PAGING):
        mline_fn = re.match(r"^\s*(\d+)\b", lstr)
        si_fn = int(mline_fn.group(1)) if mline_fn else last_fn
        L2 = (by[:23] + b"\x2b"*23)[:23]
        # IMM ASSIGN (AGCH) AND Paging Request (PCH) -> same AGCH/PCH path on the
        # shunt side (feed_agch -> CCCH block -> firmware chan_nr=0x90 ->
        # gsm48_rr_rx_pch_agch, which dispatches IMM-ASS vs PAGING by msg type).
```

```
        # SI -> BCCH.
        is_ccch = (by[2] in CCCH) or (by[2] in PAGING)
        chan = GSMTAP_AGCH if is_ccch else GSMTAP_BCCH
        s.sendto(gsmtap(si_fn, chan, L2), ("127.0.0.1", 4730))
        n += 1
        name = CCCH.get(by[2]) or PAGING.get(by[2]) or SI.get(by[2])
        if is_ccch or n <= 20 or n % 50 == 0:
            print("[si-bridge] %s (mt=0x%02x) FN=%d (%%51=%d %s) -> %s (4730)  #%d"
                  % (name, by[2], si_fn, si_fn%51, fn51_role(si_fn),
                     "feed_agch/PCH" if is_ccch else "feed_si", n), flush=True)
    # --- SDCCH/4 SS0 DL (#2): LAPDm (UA/AUTH), not RR -> gate fn%51 in {22-25} ---
    mfn_sd = re.match(r"^\s*(\d+)\b", lstr)
    sd_fn = int(mfn_sd.group(1)) if mfn_sd else last_fn
    if len(by) >= 3 and (sd_fn % 51) in (22, 23, 24, 25):
        L2 = (by[:23] + b"\x2b"*23)[:23]
        # DEDUP BY FN: gr-gsm may re-decode the SAME on-air block (same FN) ->
        # that duplicate must be skipped. BUT the BTS legitimately RE-SENDS the
        # SAME content (byte-identical UA) on a DIFFERENT FN in answer to a
        # SABM T200 retransmission of the mobile: that re-UA MUST be forwarded,
        # otherwise the mobile never confirms (missed UA -> SABM loop -> "SABM
        # not expected in timer recovery"). A dedup by content would DROP that
        # legitimate re-UA. Dedup is therefore by decoded FN (sd_fn): grgsm
        # emits each on-air FN ONCE (monotonic FN) -> dedup-by-FN removes the
        # re-decode (same FN) AND forwards the BTS re-send (distinct FN, same
        # content). LAPDm fill (UI c=0x03) is always skipped.
        ctrl = by[1] if len(by) > 1 else 0
        is_fill = (ctrl == 0x03)
        if (not is_fill) and sd_fn != last_sd_fn:
            last_sd_fn = sd_fn
            s.sendto(gsmtap(sd_fn, GSMTAP_SDCCH4, L2), ("127.0.0.1", 4730))
            nsd += 1
            if nsd <= 20 or nsd % 50 == 0:
                print("[si-bridge] SDCCH/4 SS0 DL (a0=0x%02x c=0x%02x) FN=%d (%%51=%d) -> feed_sdcch (4730)  #%d
                 ↪  [new]"
                      % (by[0], ctrl, sd_fn, sd_fn%51, nsd), flush=True)
            # TCH ARMING: the ASSIGNMENT COMMAND goes through this same DL block.
            check_assignment(by, sd_fn)
    # --- SACCH (SDCCH/4 SS0) DL: REAL SI5(0x1d)/SI6(0x1e) -> feed_sacch ---
    # SACCH SS0 slots (combined CCCH+SDCCH/4): fn%51 in {42-45}. grgsm gives the
    # 23-byte block = [L1 hdr 2][LAPDm: 03 03 len 06 mt L3...]. The RR header
    # "06 1d"/"06 1e" is detected and the block forwarded AS IS to the shunt
    # (sub_type 0x87 = SDCCH4|ACCH) -> REAL feed_sacch.
    if len(by) >= 8 and (sd_fn % 51) in (42, 43, 44, 45):
        mt_sa = None
        for off in range(2, min(9, len(by) - 1)):
            if by[off] == 0x06 and by[off + 1] in (0x1d, 0x1e):
                mt_sa = by[off + 1]; break
        if mt_sa is not None and sd_fn != last_sa_fn:
            last_sa_fn = sd_fn
            L2 = (by[:23] + b"\x2b" * 23)[:23]
            s.sendto(gsmtap(sd_fn, GSMTAP_SACCH, L2), ("127.0.0.1", 4730))
            nsa += 1
            if nsa <= 20 or nsa % 50 == 0:
                print("[si-bridge] SACCH SI%d (mt=0x%02x) FN=%d (%%51=%d) -> feed_sacch REEL (4730)  #%d"
                      % (5 if mt_sa == 0x1d else 6, mt_sa, sd_fn, sd_fn % 51, nsa), flush=True)

# === STAGE 3: DL DECIPHERING — TWO grgsm THAT HAND OVER TO EACH OTHER =========
# The CLEAR grgsm is NEVER killed: killing it to restart with -k at CIPHER MODE
# would remove the FIFO reader and freeze the ipc-device (DL FIFO full ->
# osmo-trx SETPOWER no-response -> feed_iq frozen -> no LU accept). Two
# processes run in parallel, each on ITS FIFO (both fed by the ipc-device relay):
#   - CLEAR grgsm    on CF (iq_grgsm.fifo): decodes establishment/ID/AUTH(RAND)/
#     CIPHER MODE COMMAND. Only outputs CRC-valid frames -> goes quiet by itself
#     on the dedicated channel once the DL is ciphered.
#   - CIPHERED grgsm on CIPH_CF (-e -k): spawned as soon as a Kc appears
#     (RAND->cipher window: it has time to lock FCCH/SCH BEFORE the DL turns
```

```
#     ciphered), decodes the ciphered dedicated channel (LU ACCEPT...). Killed
#     only at release (Kc erased).
# Both outputs are merged by handle_line (under lock): grgsm's CRC sorts them
# out, dedup by FN avoids duplicates. ZERO reader gap on the clear FIFO -> the
# ipc-device does not freeze. The churn (spawn/kill) of the ciphered grgsm is
# harmless thanks to the non-blocking ipc-device writer (relay_fifo_writer).
CIPH_CF = os.environ.get("CALYPSO_CIPH_FIFO", "/tmp/iq_grgsm_ciph.fifo")
_hl_lock = threading.Lock()

# grgsm_decode HARD-CODES the GSMTAP UDP port 4729 (socket_pdu UDP_SERVER+CLIENT,
# no option). Two instances -> "bind: Address already in use" -> the 2nd crashes
# (silently, stderr=DEVNULL) -> never any ciphered decode. STDOUT (-v) is read,
# not 4729 -> a COPY is patched for the ciphered grgsm on a free port (4799).
# Generated at startup -> reproducible, survives a reclone of the container.
import shutil
def make_ciph_binary():
    src = shutil.which("grgsm_decode")
    if not src:
        return "grgsm_decode"
    dst = "/tmp/grgsm_decode_ciph"
    try:
        with open(src) as f:
            code = f.read()
        code = code.replace('"4729"', '"4799"')   # unique GSMTAP port (no conflict with the clear one)
        with open(dst, "w") as f:
            f.write(code)
        os.chmod(dst, 0o755)
        print("[si-bridge] grgsm chiffre patche -> %s (GSMTAP 4729->4799)" % dst, flush=True)
        return dst
    except Exception as e:
        print("[si-bridge] patch grgsm chiffre echoue (%s) -> fallback partage 4729" % e, flush=True)
        return src
GRGSM_CIPH = make_ciph_binary()

def spawn_grgsm(fifo, algo=0, kc_hex=None, binary="grgsm_decode", errlog=None):
    cmd = [binary, "-m", "BCCH_SDCCH4", "-t", "0", "-a", str(ARFCN),
           "-c", fifo, "-s", "1083333", "-v"]
    if kc_hex and algo in (1, 2, 3):
        cmd += ["-e", str(algo), "-k", kc_hex]
    err = open(errlog, "w") if errlog else subprocess.DEVNULL
    return subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=err, text=True)

# ============================================================================
# TCH ARMING: ASSIGNMENT COMMAND -> calypso_tch_cfg -> 3rd grgsm_decode
# ============================================================================
# Without it the failure cascades SILENTLY: calypso_tch_cfg stays at 0 bytes ->
#   no `grgsm_decode -m TCHF` -> no dedicated GSMTAP -> feed_tch_sacch NEVER
#   called -> the TCH SACCH is not fed -> the MS reads the camping SI5/SI6
#   there (a_cd is SHARED camp<->TCH, with TWO header conventions) -> "Short
#   header message type 0x07 unsupported" once a second, from ~30 s after
#   the ASSIGNMENT.
#
# FILE CONTRACT — as read by its CONSUMER (qemu_wrap.c calypso_tch_cfg_read),
# never guessed:
#   0..3 seq uint32 LE (≠0)   4 TN   5 TSC   6..7 ARFCN uint16 LE   (16 B)
# The seq is written LAST: atomic publication, the reader never sees a
# half-written record.
#
# CHECKS: calypso_tch_cfg not empty; a live `-m TCHF` process;
#         `feed_tch_sacch` > 0 in qemu.log; `Short header 0x07` = 0.
TCH_CFG_PATH = "/dev/shm/calypso_tch_cfg"
TCH_FIFO     = os.environ.get("CALYPSO_TCH_FIFO", "/tmp/iq_grgsm_tch.fifo")
TCH_SPEECH   = "/dev/shm/calypso_tch_speech.gsm"
_tch_proc = None
_tch_cur  = None          # (tn, tsc, arfcn) currently armed
_tch_seq  = 0
```

```
def _tch_write_cfg(tn, tsc, arfcn):
    """Publish TN/TSC/ARFCN. seq last = atomic publication."""
    global _tch_seq
    _tch_seq += 1
    buf = bytearray(16)
    buf[4] = tn & 0xff
    buf[5] = tsc & 0xff
    buf[6:8] = int(arfcn).to_bytes(2, "little")
    buf[0:4] = _tch_seq.to_bytes(4, "little")
    fd = os.open(TCH_CFG_PATH, os.O_CREAT | os.O_WRONLY, 0o644)
    try:
        os.pwrite(fd, bytes(buf), 0)
    finally:
        os.close(fd)

def make_tch_binary():
    """Patched copy of grgsm_decode for the TCH — TWO ports to fix.

    grgsm_decode hard-codes 127.0.0.1:4729 TWICE (l.122-123):
      - l.122 UDP_SERVER: only there to avoid ICMP port-unreachable. Two
        instances -> "bind: Address already in use" -> the 2nd DIES, so
        feed_tch_sacch = 0 and the dedicated SACCH stays silent.
      - l.123 UDP_CLIENT: the REAL GSMTAP output. In TCHF mode, tch_f_decoder
        AND cch_decoder publish there (l.213-214).
    The server is therefore bound on a free port, and the client pointed
    straight at 4730, which the shunt listens on: it dispatches by sub_type —
    TCH_F (0x02) -> feed_facch, TCH_ACCH (0x82) -> feed_tch_sacch
    (calypso_dsp_shunt.c:3126-3134). No stdout parsing: data goes from the
    decoder to the shunt directly.
    ⚠ TARGETED replacements line by line: a global `replace("4729","4730")`
    would bind the server on 4730 and conflict with the shunt.
    """
    src = shutil.which("grgsm_decode")
    if not src:
        return "grgsm_decode"
    dst = "/tmp/grgsm_decode_tch"
    try:
        with open(src) as f:
            code = f.read()
        a = '"UDP_SERVER", "127.0.0.1", "4729"'
        b = '"UDP_CLIENT", "127.0.0.1", "4729"'
        if a not in code or b not in code:
            print("[si-bridge] grgsm TCH : motifs de port introuvables -> "
                  "fallback non patche (conflit 4729 probable)", flush=True)
            return src
        code = code.replace(a, '"UDP_SERVER", "127.0.0.1", "4739"')
        code = code.replace(b, '"UDP_CLIENT", "127.0.0.1", "4730"')
        with open(dst, "w") as f:
            f.write(code)
        os.chmod(dst, 0o755)
        print("[si-bridge] grgsm TCH patche -> %s (bind 4739, GSMTAP -> 4730)" % dst,
              flush=True)
        return dst
    except Exception as e:
        print("[si-bridge] patch grgsm TCH echoue (%s)" % e, flush=True)
        return src
GRGSM_TCH = make_tch_binary()

def _tch_spawn(tn, arfcn):
    """3rd decoder, on the dedicated IQ pipe (fed by CALYPSO_RELAY_FIFOS)."""
    cmd = [GRGSM_TCH, "-m", "TCHF", "-t", str(tn), "-a", str(arfcn),
           "-c", TCH_FIFO, "-s", "1083333", "-v", "-d", "FR", "-o", TCH_SPEECH]
    try:
        err = open("/tmp/osmo-nitb/logs/grgsm_decode_tch.log", "w")
    except Exception:
```

```
        err = subprocess.DEVNULL
    return subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=err, text=True)

def arm_tch(tn, tsc, arfcn):
    global _tch_proc, _tch_cur
    cfg = (tn, tsc, arfcn)
    vivant = _tch_proc is not None and _tch_proc.poll() is None
    if cfg == _tch_cur and vivant:
        return                                  # already armed on this config
    if _tch_proc is not None and vivant:
        try: _tch_proc.terminate()
        except Exception: pass
    _tch_write_cfg(tn, tsc, arfcn)
    _tch_proc = _tch_spawn(tn, arfcn)
    _tch_cur = cfg
    print("[si-bridge] TCH arme : TN=%d TSC=%d ARFCN=%d -> %s + grgsm -m TCHF (pid %s)"
          % (tn, tsc, arfcn, TCH_CFG_PATH, _tch_proc.pid), flush=True)

def check_assignment(by, fn):
    """Detect an ASSIGNMENT COMMAND (RR PD=0x06, MT=0x2e) on the SDCCH DL.

    Channel Description 2 (GSM 04.08 10.5.2.5), 3 octets, decoded like
    libosmocore's `struct gsm48_chan_desc`:
        b0: chan_nr           -> TN = b0 & 0x07
        b1: tsc(3) h(1) arfcn_high(2) spare(2)
            TSC = (b1>>5)&7   H = (b1>>4)&1   ARFCN_high = b1 & 0x03
        b2: ARFCN_low         -> ARFCN = (ARFCN_high<<8) | b2   [if H=0]
    """
    for off in range(2, min(7, len(by) - 5)):
        if by[off] != 0x06 or by[off + 1] != 0x2e:
            continue
        b0, b1, b2 = by[off + 2], by[off + 3], by[off + 4]
        tn  = b0 & 0x07
        tsc = (b1 >> 5) & 0x07
        h   = (b1 >> 4) & 0x01
        if h:
            # ⚠ EXPLICIT REFUSAL. This chain does NOT handle frequency hopping
            # (same limit on the UL injection side). Building an ARFCN from
            # the MAIO/HSN fields would silently give a WRONG number, and the
            # decoder would listen elsewhere with nothing to show for it.
            print("[si-bridge] ASSIGNMENT COMMAND avec SAUT DE FREQUENCE (H=1) : "
                  "TCH NON arme (chaine sans hopping). FN=%d" % fn, flush=True)
            return
        arfcn = ((b1 & 0x03) << 8) | b2
        print("[si-bridge] ASSIGNMENT COMMAND FN=%d -> chan_nr=0x%02x TN=%d TSC=%d ARFCN=%d"
              % (fn, b0, tn, tsc, arfcn), flush=True)
        arm_tch(tn, tsc, arfcn)
        return

def reader_loop(proc, tag):
    # raw log per grgsm (decipher diagnostic): /root/grgsm_clair.raw / _ciph.raw
    try: raw = open("/root/grgsm_%s.raw" % tag, "w")
    except Exception: raw = None
    for line in proc.stdout:
        if raw:
            try: raw.write(line); raw.flush()
            except Exception: pass
        with _hl_lock:
            handle_line(line)
    if raw:
        try: raw.close()
        except Exception: pass
    print("[si-bridge] grgsm %s termine (n=%d nsch=%d nsd=%d nsa=%d)"
          % (tag, n, nsch, nsd, nsa), flush=True)

def read_kc():
```

```
    try:
        with open("/dev/shm/calypso_kc", "rb") as f:
            b = f.read(32)
    except OSError:
        return (0, 0, None)
    if len(b) < 14:
        return (0, 0, None)
    seq = int.from_bytes(b[0:4], "little")
    if seq == 0:
        return (0, 0, None)
    return (seq, b[4], b[6:14].hex())

# --- CLEAR grgsm: always alive (watchdog respawn on crash), NEVER killed at cipher ---
clear = {"proc": None}
def ensure_clear():
    p = clear["proc"]
    if p is None or p.poll() is not None:
        clear["proc"] = spawn_grgsm(CF)
        threading.Thread(target=reader_loop, args=(clear["proc"], "clair"),
                         daemon=True).start()
        print("[si-bridge] grgsm CLAIR spawn sur %s (jamais tue au cipher)" % CF, flush=True)

ensure_clear()

# --- CIPHERED grgsm: spawned/killed by the Kc, ALWAYS on CIPH_CF (never on CF) ---
ciph = {"applied": (0, None), "proc": None}
while True:
    time.sleep(0.5)
    ensure_clear()                    # clear grgsm watchdog
    seq, algo, kc = read_kc()
    want = (algo, kc) if (seq and algo in (1, 2, 3)) else (0, None)
    if want == ciph["applied"]:
        continue
    # Kc state changed -> ONLY the ciphered grgsm is (re)cycled (on CIPH_CF)
    p = ciph["proc"]
    if p is not None:
        try: p.kill(); p.wait(timeout=2)
        except Exception: pass
        ciph["proc"] = None
    ciph["applied"] = want
    if want[0]:
        np = spawn_grgsm(CIPH_CF, algo=want[0], kc_hex=want[1], binary=GRGSM_CIPH,
                         errlog="/root/grgsm_ciph.err")
        ciph["proc"] = np
        threading.Thread(target=reader_loop, args=(np, "chiffre"),
                         daemon=True).start()
        print("[si-bridge] grgsm CHIFFRE spawn sur %s : A5/%d kc=%s (decipher DL)"
              % (CIPH_CF, want[0], want[1]), flush=True)
    else:
        print("[si-bridge] Kc efface -> grgsm chiffre arrete (clair seul)", flush=True)
```

2.132 /opt/GSM/osmo-operator/opt-gsm/si_bridge_loop.sh

584 octets, 12 lignes → 12 lignes

```
#!/bin/bash
# Runs si_bridge (2 grgsm: clear always-on + ciphered spawned on Kc) in a loop.
# --line-buffered: otherwise grep buffers by 4KB blocks -> late logs, the
# respawn/Kc cannot be watched in real time. Python errors/tracebacks are
# let through as well (otherwise invisible).
source /root/.env/bin/activate 2>/dev/null
echo "[si-bridge] 2 grgsm (clair iq_grgsm.fifo + chiffre iq_grgsm_ciph.fifo) -> feed_*"
while true; do
  python3 ${GSM_ROOT}/si_bridge.py /tmp/iq_grgsm.fifo 2>&1 \
```

```
    | grep --line-buffered -E "si-bridge|SI|Traceback|Error|error|Exception"
  sleep 2
done
```

2.133 /opt/GSM/osmo-operator/packaging/apt-fast-install.sh

8176 octets, 183 lignes → 183 lignes

```
#!/bin/bash
# ================================================================================
#  apt-fast-install - ONE single way to install apt-fast, everywhere
# ================================================================================
#
#  Used by: Dockerfile, Dockerfile.stp (COPY then RUN), build-iso.sh (in the
#  rootfs chroot), build.sh and tools/make-docker-image.sh (on the host).
#  After this script, everyone talks to apt the same way:
#
#      apt-fast update
#      apt-fast install -y --no-install-recommends <packages>
#      apt-fast build-dep -y <package>
#
#  apt-fast downloads in parallel (aria2), dpkg then installs as usual: the
#  gain is on the download, and it is clear as soon as several hundred
#  packages are installed. If GitHub is unreachable, an apt-fast that calls
#  apt-get is installed: the command always exists, the build goes on, more
#  slowly.
#
#  DLDIR = /var/cache/apt/archives: apt-fast downloads where apt stores its
#  packages. This is what lets build-iso.sh mount a PERSISTENT cache at that
#  place in the chroot (the rootfs .deb are downloaded once, for every pass
#  and every rebuild).
#
#  It also installs /etc/apt/apt.conf.d/90osmo-operator - the apt settings
#  shared by the ISO chroot and the host command line: no translations,
#  3 retries, HTTP pipelining, dpkg without pty. These are DOWNLOAD
#  settings; they touch neither the recommends nor the way dpkg writes
#  its files.
#
#  Idempotent: already installed -> only redoes the conf.
# ================================================================================
set -euo pipefail
export DEBIAN_FRONTEND=noninteractive

APT_FAST_URL="${APT_FAST_URL:-https://raw.githubusercontent.com/ilikenwf/apt-fast/master/apt-fast}"
BIN=/usr/local/sbin/apt-fast

# 30 s timeout and no pipelining: a mirror that does not answer fails in
# 30 s and moves on to the next retry, instead of blocking 120 s (the
# default) times three retries, silently. HTTP pipelining brings nothing here
# (aria2 already parallelises) and is a known source of connections left
# hanging behind some load balancers.
cat > /etc/apt/apt.conf.d/90osmo-operator <<'CONF'
// osmo-operator : reglages apt communs (image docker, rootfs ISO, hote)
Acquire::Languages "none";
Acquire::Retries "3";
Acquire::http::Timeout "30";
Acquire::http::Pipeline-Depth "0";
Dpkg::Use-Pty "0";
// Avant CHAQUE dpkg : les .deb a installer sont decompresses de bout en bout ;
// un fichier corrompu est supprime du cache et l installation s arrete, a
// relancer (osmo-apt-get, sous apt-fast, relance tout seul). Voir
// /usr/local/sbin/osmo-deb-check.
DPkg::Pre-Install-Pkgs { "/usr/local/sbin/osmo-deb-check"; };
DPkg::Tools::Options::/usr/local/sbin/osmo-deb-check::Version "1";
```

```
CONF

# ── The apt cache is PERSISTENT everywhere here: a corrupted .deb stays in it ─
# apt does not re-verify a file already present in /var/cache/apt/archives,
# and that cache is never cleared here (Keep-Downloaded-Packages, docker build
# cache mount, persistent cache of the ISO chroot): a file with the expected
# size but a bad SHA256 (altered download on a flaky mirror) would stay there
# for every build. Two pieces:
#   osmo-deb-check   DPkg::Pre-Install-Pkgs hook: apt passes it the paths of
#                    the .deb it is about to install; each is fully
#                    decompressed (dpkg-deb --fsys-tarfile); the bad ones are
#                    DELETED, a marker is set and apt stops before dpkg. The
#                    next apt re-downloads what is missing, with hash check.
#   osmo-apt-get     the "apt-get" that apt-fast runs (_APTMGR): if apt fails
#                    and the marker is there, it retries once. For an apt-get
#                    called directly, the message says to rerun.
cat > /usr/local/sbin/osmo-deb-check <<'CHK'
#!/bin/sh
# osmo-deb-check - apt hook (DPkg::Pre-Install-Pkgs, protocol 1: one .deb path
# per line on stdin). Deletes from the cache the .deb whose data archive does
# not decompress, and refuses the installation if there were any.
MARK=/run/osmo-deb-check.bad
rm -f "$MARK"
bad=0; n=0
while IFS= read -r f; do
    [ -f "$f" ] || continue
    n=$((n + 1))
    if ! dpkg-deb --fsys-tarfile "$f" >/dev/null 2>&1; then
        echo "osmo-deb-check: CORROMPU, supprime du cache : $f" >&2
        rm -f "$f"
        bad=$((bad + 1))
    fi
done
if [ "$bad" -gt 0 ]; then
    echo "$bad" > "$MARK"
    echo "osmo-deb-check: $bad paquet(s) corrompu(s) retire(s) sur $n - relancez la commande, apt les
↪  retelechargera" >&2
    exit 1
fi
exit 0
CHK
chmod 755 /usr/local/sbin/osmo-deb-check
cat > /usr/local/sbin/osmo-apt-get <<'OAG'
#!/bin/sh
# osmo-apt-get - apt-get with ONE retry when osmo-deb-check removed corrupted
# .deb from the cache (marker /run/osmo-deb-check.bad).
MARK=/run/osmo-deb-check.bad
rm -f "$MARK"
apt-get "$@" && exit 0
rc=$?
if [ -f "$MARK" ]; then
    echo "osmo-apt-get: cache apt nettoye ($(cat "$MARK") .deb corrompu(s)) - relance de : apt-get $*" >&2
    rm -f "$MARK"
    exec apt-get "$@"
fi
exit "$rc"
OAG
chmod 755 /usr/local/sbin/osmo-apt-get

# ── The Ubuntu mirror: the one the caller measured (packaging/apt-mirror.sh)
# OSMO_UBUNTU_MIRROR=http://... replaces archive.ubuntu.com in the sources of
# the current system - the docker image (build-arg), the ISO rootfs.
# Never on the host unless asked: build.sh does not export it.
if [ -n "${OSMO_UBUNTU_MIRROR:-}" ]; then
    _m="${OSMO_UBUNTU_MIRROR%/}"
    for _f in /etc/apt/sources.list /etc/apt/sources.list.d/ubuntu.sources; do
```

```
        [ -f "$_f" ] || continue
        # archive AND security: security.ubuntu.com lives on the same Canonical
        # addresses as archive (91.189.9x, 185.125.190.x) - when one is
        # unreachable, so is the other. Every Ubuntu mirror serves noble-security.
        sed -i -E "s#https?://(archive|[a-z]{2}\.archive|security)\.ubuntu\.com/ubuntu/?#${_m}#g" "$_f"
    done
    echo "[apt-fast] miroir Ubuntu : $_m"
fi

# aria2 and curl are the only packages still installed with apt-get:
# apt-fast does not exist before them.
if ! command -v aria2c >/dev/null 2>&1 || ! command -v curl >/dev/null 2>&1; then
    apt-get update -qq
    apt-get install -y --no-install-recommends aria2 curl ca-certificates
fi

if [ ! -x "$BIN" ] || ! grep -q 'aria2' "$BIN" 2>/dev/null; then
    if curl -fsSL --retry 3 -o "$BIN.tmp" "$APT_FAST_URL" && grep -q 'aria2' "$BIN.tmp"; then
        mv -f "$BIN.tmp" "$BIN"
        echo "[apt-fast] installe depuis $APT_FAST_URL"
    else
        rm -f "$BIN.tmp"
        printf '#!/bin/sh\n# apt-fast de repli (telechargement impossible) : apt-get tel quel\nexec apt-get
        ↪  "$@"\n' > "$BIN"
        echo "[apt-fast] WARN: telechargement impossible - repli sur apt-get sous le meme nom" >&2
    fi
    chmod 755 "$BIN"
fi

# The apt-fast conf: 16 connections, no questions. Written every time, it is
# short and this is what guarantees that all environments look alike.
cat > /etc/apt-fast.conf <<'CONF'
_APTMGR=/usr/local/sbin/osmo-apt-get
DOWNLOADBEFORE=true
_MAXNUM=16
_MAXCONPERSRV=10
_SPLITCON=8
_MINSPLITSZ="1M"
_PIECEALGO="default"
DLDIR=/var/cache/apt/archives
DLLIST=/tmp/apt-fast.list
APT_FAST_TIMEOUT=60
VERBOSE_OUTPUT=false
CONF
mkdir -p /var/cache/apt/archives/partial

# Keep the downloaded .deb: that is the point of the cache. Docker images ship
# /etc/apt/apt.conf.d/docker-clean, which deletes them after every
# installation - it is removed. Keep-Downloaded-Packages applies to apt >= 1.2.
rm -f /etc/apt/apt.conf.d/docker-clean
cat > /etc/apt/apt.conf.d/91osmo-keep-debs <<'CONF'
// osmo-operator : les paquets telecharges restent dans /var/cache/apt/archives
Binary::apt::APT::Keep-Downloaded-Packages "true";
APT::Keep-Downloaded-Packages "true";
CONF
# If /usr/local/sbin is not in the caller's PATH (root without sudo, custom
# PATH), the command would be unreachable right after "ready": a link in
# /usr/local/bin, which is always there, settles it without touching PATH.
if ! command -v apt-fast >/dev/null 2>&1; then
    ln -sfn "$BIN" /usr/local/bin/apt-fast
    echo "[apt-fast] /usr/local/sbin absent du PATH - lien /usr/local/bin/apt-fast"
fi
echo "[apt-fast] pret : $BIN"
```

2.134 /opt/GSM/osmo-operator/packaging/apt-mirror.sh

3785 octets, 79 lignes → 79 lignes

```
#!/bin/bash
# ============================================================================
#  apt-mirror - THE fastest Ubuntu mirror from here, measured, not assumed
# ============================================================================
#
#  Prints a mirror URL (http://.../ubuntu) on standard output.
#
#  The "archive.ubuntu.com" address is a geo-balanced entry point, not a
#  throughput commitment: it can answer at a few KB/s while a national or
#  hosting-provider mirror serves the same file in a fraction of a second,
#  leaving a docker build silent for minutes on `apt-get update`. So the
#  candidates are MEASURED, and the fastest one is the one that is used
#  everywhere a build talks to apt.
#
#  Candidates: $OSMO_UBUNTU_MIRRORS (space-separated list) if given, otherwise
#  the mirrors mirrors.ubuntu.com proposes for THIS country (GeoIP, the first
#  four) plus archive.ubuntu.com. Each candidate downloads
#  dists/<suite>/InRelease with a 5 s maximum; the fastest wins. If nothing
#  answers, archive.ubuntu.com.
#
#  OSMO_UBUNTU_MIRROR=http://... short-circuits everything: that mirror,
#  with no measurement (a local mirror, an apt proxy).
#
#  Usage:   MIRROR="$(packaging/apt-mirror.sh [suite])"   (default: noble)
#           The measurements go to stderr, the URL alone to stdout.
#           packaging/apt-mirror.sh --list [suite]
#           Every mirror that answers, "<ms>\t<url>", from the fastest to the
#           slowest, on stdout - this is what the live installer reads to
#           offer the choice (osmo-install, packagechooser@mirror page).
# ============================================================================
set -uo pipefail
LIST=0
if [ "${1:-}" = "--list" ]; then LIST=1; shift; fi
SUITE="${1:-noble}"
DEFAULT="http://archive.ubuntu.com/ubuntu"

if [ -n "${OSMO_UBUNTU_MIRROR:-}" ] && [ "$LIST" = 0 ]; then
    echo "${OSMO_UBUNTU_MIRROR%/}"; exit 0
fi
command -v curl >/dev/null 2>&1 || { [ "$LIST" = 1 ] && printf '0\t%s\n' "$DEFAULT" || echo "$DEFAULT"; exit 0; }

CANDIDATES="${OSMO_UBUNTU_MIRRORS:-}"
if [ -z "$CANDIDATES" ]; then
    # mirrors.txt (GeoIP) sometimes returns... archive.ubuntu.com itself. The
    # locale's country mirror (fr.archive, de.archive...), two large European
    # hosting providers and kernel.org are added: enough to always have a
    # mirror that answers.
    CANDIDATES="$(curl -fsSL -m 5 http://mirrors.ubuntu.com/mirrors.txt 2>/dev/null \
                  | grep -E '^https?://' | head -4 | tr '\n' ' ')"
    _cc="$(echo "${LC_ALL:-${LANG:-}}" | sed -nE 's/^[a-z]+_([A-Z]{2}).*/\1/p' | tr 'A-Z' 'a-z')"
    [ -n "$_cc" ] && CANDIDATES="$CANDIDATES http://${_cc}.archive.ubuntu.com/ubuntu"
    CANDIDATES="$CANDIDATES http://ubuntu.mirrors.ovh.net/ubuntu http://mirror.ubuntu.ikoula.com/ubuntu
                http://mirrors.edge.kernel.org/ubuntu $DEFAULT"
fi
# Without duplicates, in order.
CANDIDATES="$(for m in $CANDIDATES; do echo "${m%/}"; done | awk '!seen[$0]++' | tr '\n' ' ')"

best=""; best_t=999999; results=""
for m in $CANDIDATES; do
    m="${m%/}"
    # A single file, small and always present; 5 s max, and the time is
    # curl's own (ms), not an external clock.
```

```
    t="$(curl -s -m 5 -o /dev/null -w '%{http_code} %{time_total}' "$m/dists/$SUITE/InRelease" 2>/dev/null ||
     ↪  true)"
    code="${t%% *}"; secs="${t##* }"
    if [ "$code" = "200" ]; then
        ms="$(awk -v s="$secs" 'BEGIN{printf "%d", s*1000}')"
        printf '  %-52s %6d ms\n' "$m" "$ms" >&2
        results="${results}${ms}    ${m}
"
        if [ "$ms" -lt "$best_t" ]; then best="$m"; best_t="$ms"; fi
    else
        printf '  %-52s   -    (%s)\n' "$m" "${code:-timeout}" >&2
    fi
done
if [ "$LIST" = 1 ]; then
    printf '%s' "$results" | sort -n
    exit 0
fi
echo "${best:-$DEFAULT}"
```

2.135 /opt/GSM/osmo-operator/packaging/build-debs.sh

14701 octets, 298 lignes → 298 lignes

```
#!/bin/bash
# ============================================================================
#  packaging/build-debs.sh - one .deb package per bench component
# ============================================================================
#
#      osmo-operator     this repo, as is, in /opt/GSM/osmo-operator
#                        (+ menu, icons, /usr/bin/osmo-start-direct)
#      pont              pont/ - the Python transceiver (pont/pont.py)
#      qemu-calypso      qosmo: qemu-system-arm (calypso machine),
#                        run.sh + run_modules + environment + cfgs +
#                        tmux_modules + keymaps, in /opt/GSM/qosmo
#      calypso-firmware  /opt/GSM/firmware/board/compal_e88/layer1.highram.*
#
#  The install paths are THOSE of the bench (/opt/GSM/...): the whole repo
#  hardcodes them (bench.env, start-native.sh, the systemd units), and this
#  is what lets a machine installed from packages run exactly like the ISO.
#  The price: a package refuses to install over a git clone at the same
#  place (preinst) - a git-managed tree and a dpkg-managed tree are not
#  mixed.
#
#      ./build-debs.sh                   all four, into packaging/dist/
#      ./build-debs.sh --only pont,calypso-firmware
#      ./build-debs.sh --out /tmp/debs
#      ./build-debs.sh --no-strip        keeps the qemu-system-arm symbols
#
#  Tools: dpkg-deb (dpkg), git, strip (binutils, otherwise --no-strip), ldd.
#  No debhelper: each package is a tree + DEBIAN/control built by hand, with
#  --root-owner-group (no fakeroot needed).
#
#  Sources, overridable from the environment:
#      OSMO_OPERATOR_SRC  (default: the repo containing this script)
#      QOSMO_SRC          (default: /opt/GSM/qosmo)
#      FIRMWARE_SRC       (default: /opt/GSM/firmware)
# ----------------------------------------------------------------------------
set -euo pipefail

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
: "${OSMO_OPERATOR_SRC:=$(cd "$HERE/.." && pwd)}"
: "${QOSMO_SRC:=/opt/GSM/qosmo}"
: "${FIRMWARE_SRC:=/opt/GSM/firmware}"
OUT="$HERE/dist"
```

```
ONLY=""
STRIP=1
while [ $# -gt 0 ]; do
    case "$1" in
        --out)      OUT="${2:?}"; shift ;;
        --out=*)    OUT="${1#*=}" ;;
        --only)     ONLY="${2:?}"; shift ;;
        --only=*)   ONLY="${1#*=}" ;;
        --no-strip) STRIP=0 ;;
        -h|--help)  sed -n '2,33p' "$0" | sed 's/^#\{1,2\} \{0,1\}//'; exit 0 ;;
        *) echo "option inconnue : $1" >&2; exit 2 ;;
    esac
    shift
done
for t in dpkg-deb git; do
    command -v "$t" >/dev/null 2>&1 || { echo "outil manquant : $t" >&2; exit 1; }
done
mkdir -p "$OUT"
WORK="$(mktemp -d "${TMPDIR:-/var/tmp}/osmo-debs.XXXXXX")"
trap 'rm -rf "$WORK"' EXIT

MAINT="osmo-operator <bastienbaranoff@gmail.com>"
ARCH_HOST="$(dpkg --print-architecture)"

wanted() { [ -z "$ONLY" ] || case ",$ONLY," in *",$1,"*) return 0;; *) return 1;; esac; }

# Version = 0.1+git<last commit date>.<short hash> of the source tree.
# Two builds of the same commit give the same number: this is intended.
tree_version() {
    local d="$1"
    if git -C "$d" rev-parse --is-inside-work-tree >/dev/null 2>&1; then
        printf '0.1+git%s.%s' \
            "$(git -C "$d" log -1 --format=%cd --date=format:%Y%m%d)" \
            "$(git -C "$d" rev-parse --short HEAD)"
    else
        printf '0.1+local%s' "$(date +%Y%m%d)"
    fi
}

# Shared preinst: refuses a git clone at the same place (except when upgrading
# the package itself, where dpkg passes "upgrade").
write_preinst_guard() {   # $1=DEBIAN dir  $2=installed path
    cat > "$1/preinst" <<PRE
#!/bin/sh
set -e
if [ "\$1" = install ] && [ -d "$2/.git" ]; then
    echo "$2 est un clone git : ce paquet ne s installe pas par-dessus." >&2
    echo "Deplacez ce clone (ou supprimez-le) avant d installer le paquet." >&2
    exit 1
fi
exit 0
PRE
    chmod 755 "$1/preinst"
}

write_control() {   # $1=DEBIAN dir, then fields on stdin
    cat > "$1/control"
}

installed_size() { du -sk --exclude=DEBIAN "$1" | cut -f1; }

build_pkg() {   # $1=staging  $2=nom  $3=version  $4=arch
    printf 'Installed-Size: %s\n' "$(installed_size "$1")" >> "$1/DEBIAN/control"
    find "$1" -path "$1/DEBIAN" -prune -o -type d -exec chmod 755 {} + 2>/dev/null || true
    dpkg-deb --build --root-owner-group "$1" "$OUT/${2}_${3}_${4}.deb" >/dev/null
    echo "  $OUT/${2}_${3}_${4}.deb  ($(du -h "$OUT/${2}_${3}_${4}.deb" | cut -f1))"
```

```
}

# Runtime dependencies of a binary, via ldd -> dpkg -S. Libraries unknown to
# dpkg (/usr/local/lib) are ignored: they come from no package, so no package
# can guarantee them.
shlib_depends() {
    # ldd returns /lib/x86_64-linux-gnu/...; on a merged-/usr system dpkg often
    # only knows /usr/lib/...: both are tried. Everything is "|| true": under
    # pipefail a single unknown library would fail the whole substitution and
    # set -e would stop the script.
    local so
    { ldd "$1" 2>/dev/null | awk '/=> \//{print $3}' || true; } | while read -r so; do
        { dpkg -S "$so" 2>/dev/null || dpkg -S "/usr$so" 2>/dev/null || true; } | cut -d: -f1 | head -1
    done | grep -v '^$' | sort -u | paste -sd, - | sed 's/,/, /g' || true
}

# ── osmo-operator ──────────────────────────────────────────────────────
if wanted osmo-operator; then
    V="$(tree_version "$OSMO_OPERATOR_SRC")"
    P="$WORK/osmo-operator"; D="$P/opt/GSM/osmo-operator"
    mkdir -p "$D" "$P/DEBIAN" "$P/usr/bin" "$P/usr/share/applications" \
             "$P/usr/share/osmo-operator/icons" "$P/usr/share/icons/hicolor/scalable/apps"
    tar -C "$OSMO_OPERATOR_SRC" \
        --exclude=.git --exclude=./pont --exclude=./packaging/dist \
        --exclude='__pycache__' --exclude='*.iso' --exclude='vty-debug-dump-*' \
        -cf - . | tar -C "$D" -xf -
    ln -s /opt/GSM/osmo-operator/start-native.sh "$P/usr/bin/osmo-start-native"
    ln -s /opt/GSM/osmo-operator/start-native.sh "$P/usr/bin/osmo-start-direct"   # ancien nom
    for e in osmo-launch osmo-multi; do
        install -m644 "$OSMO_OPERATOR_SRC/data/desktop/$e.desktop" "$P/usr/share/applications/$e.desktop"
        install -m644 "$OSMO_OPERATOR_SRC/data/$e.svg" "$P/usr/share/osmo-operator/icons/$e.svg"
        install -m644 "$OSMO_OPERATOR_SRC/data/$e.svg" "$P/usr/share/icons/hicolor/scalable/apps/$e.svg"
    done
    install -m644 "$OSMO_OPERATOR_SRC/data/osmo-tutorial.svg" "$P/usr/share/osmo-operator/icons/" 2>/dev/null ||
     ↪  true
    install -m644 "$OSMO_OPERATOR_SRC/data/tutorial.html" "$P/usr/share/osmo-operator/" 2>/dev/null || true
    write_preinst_guard "$P/DEBIAN" /opt/GSM/osmo-operator
    cat > "$P/DEBIAN/postinst" <<'POST'
#!/bin/sh
set -e
command -v update-desktop-database >/dev/null 2>&1 && update-desktop-database /usr/share/applications || true
command -v gtk-update-icon-cache >/dev/null 2>&1 && gtk-update-icon-cache -f -q /usr/share/icons/hicolor || true
exit 0
POST
    chmod 755 "$P/DEBIAN/postinst"
    write_control "$P/DEBIAN" <<CTL
Package: osmo-operator
Version: $V
Section: net
Priority: optional
Architecture: all
Maintainer: $MAINT
Depends: bash, python3, socat, netcat-openbsd, tcpdump, tmux, expect, whiptail, iproute2, sqlite3, git, pont (>=
 ↪  0.1), qemu-calypso, calypso-firmware
Recommends: pulseaudio, pulseaudio-utils, asterisk, wireshark
Homepage: https://github.com/bbaranoff/osmo-operator
Description: banc GSM/EGPRS Osmocom - lanceur, configurations, outils
 Le depot osmo-operator installe en /opt/GSM/osmo-operator : start-native.sh
 (lanceur natif, qui delegue a qosmo/run.sh), les gabarits de
 configuration Osmocom et Asterisk, les scripts reseau/WAN/SS7, les checks, le
 menu et les icones du bureau. Les demons Osmocom eux-memes (osmo-stp, hlr,
 msc, bsc, bts...) ne sont pas dans ce paquet : ils viennent du depot Osmocom
 ou d une compilation locale (/usr/local).
CTL
    build_pkg "$P" osmo-operator "$V" all
fi
```

```
# ── pont ──────────────────────────────────────────────────────────────
if wanted pont; then
    V="$(tree_version "$OSMO_OPERATOR_SRC")"
    P="$WORK/pont"; D="$P/opt/GSM/osmo-operator/pont"
    mkdir -p "$D" "$P/DEBIAN"
    install -m644 "$OSMO_OPERATOR_SRC"/pont/*.py "$D/"
    # The DSP bridge (pont_dsp.py -> pont.dsp.main).
    mkdir -p "$D/dsp"
    install -m644 "$OSMO_OPERATOR_SRC"/pont/dsp/*.py "$D/dsp/"
    install -m644 "$OSMO_OPERATOR_SRC/pont/README.md" "$D/" 2>/dev/null || true
    chmod 755 "$D/pont.py" "$D/pont_dsp.py"
    write_preinst_guard "$P/DEBIAN" /opt/GSM/osmo-operator
    write_control "$P/DEBIAN" <<CTL
Package: pont
Version: $V
Section: net
Priority: optional
Architecture: all
Maintainer: $MAINT
Depends: python3 (>= 3.10), python3-numpy
Enhances: osmo-operator
Homepage: https://github.com/bbaranoff/osmo-operator
Description: pont TRX entre osmo-bts-trx et le Calypso emule (qosmo)
 Transceiver TRX-UDP en Python : se presente a osmo-bts-trx sur 5700-5702,
 decode le descendant vers GSMTAP 4730/4731 (lu par le modele QEMU) et encode
 le montant depuis les sidebands /dev/shm/calypso_*. Installe en
 /opt/GSM/osmo-operator/pont, la ou start-native.sh l attend (PONT_PY).
 Requiert libosmocoding (libosmocore) accessible par ld.so - fournie par la
 pile Osmocom, pas par un paquet Debian.
CTL
    build_pkg "$P" pont "$V" all
fi

# ── qemu-calypso ──────────────────────────────────────────────────────
if wanted qemu-calypso; then
    BIN="$QOSMO_SRC/build/qemu-system-arm"
    [ -x "$BIN" ] || { echo "qemu-system-arm absent : $BIN (construisez qosmo d abord)" >&2; exit 1; }
    V="$(tree_version "$QOSMO_SRC")"
    P="$WORK/qemu-calypso"; D="$P/opt/GSM/qosmo"
    mkdir -p "$D/build" "$D/share/qemu" "$P/DEBIAN"
    for d in run_modules environnement cfgs tmux_modules opt-gsm-scripts; do
        [ -d "$QOSMO_SRC/$d" ] || continue
        tar -C "$QOSMO_SRC" --exclude='__pycache__' -cf - "$d" | tar -C "$D" -xf -
    done
    install -m755 "$QOSMO_SRC/run.sh" "$D/run.sh"
    for f in LICENSE COPYING; do [ -f "$QOSMO_SRC/$f" ] && install -m644 "$QOSMO_SRC/$f" "$D/$f"; done
    # The binary stays in build/: bench.env names it there (QEMU_BIN), and QEMU
    # computes its datadir RELATIVE to the binary (build/../share/qemu) - hence
    # the keymaps in share/qemu/keymaps, otherwise "could not read keymap file".
    install -m755 "$BIN" "$D/build/qemu-system-arm"
    if [ "$STRIP" = 1 ] && command -v strip >/dev/null 2>&1; then
        strip --strip-unneeded "$D/build/qemu-system-arm" || true
    fi
    # qosmo C launcher (tools/qosmo-launch): the one 40-qemu.sh calls instead
    # of qemu-system-arm. Compiled in its folder, shipped as
    # /usr/local/bin/qosmo; the source goes with the tree.
    if [ -f "$QOSMO_SRC/tools/qosmo-launch/qosmo-launch.c" ]; then
        mkdir -p "$D/tools/qosmo-launch" "$P/usr/local/bin"
        install -m644 "$QOSMO_SRC/tools/qosmo-launch/qosmo-launch.c" "$QOSMO_SRC/tools/qosmo-launch/Makefile"
         ↪  "$D/tools/qosmo-launch/"
        make -s -C "$QOSMO_SRC/tools/qosmo-launch" ALIAS=qosmo TREE=/opt/GSM/qosmo qosmo >/dev/null \
            || { echo "compilation du lanceur qosmo echouee ($QOSMO_SRC/tools/qosmo-launch)" >&2; exit 1; }
        install -m755 "$QOSMO_SRC/tools/qosmo-launch/qosmo" "$P/usr/local/bin/qosmo"
    fi
    # gdb console over telnet (run_modules/44-gdb-telnet.sh): the server, the
```

```
    # panel generator and the generated panel. gdb-multiarch and telnet go in
    # Depends: without them the module reports failure (non-blocking).
    if [ -f "$QOSMO_SRC/tools/gdb-telnet.py" ]; then
        install -m755 "$QOSMO_SRC/tools/gdb-telnet.py" "$D/tools/gdb-telnet.py"
        install -m755 "$QOSMO_SRC/tools/gdb_cmd.sh" "$D/tools/gdb_cmd.sh"
        (cd "$D/tools" && bash ./gdb_cmd.sh >/dev/null) || { echo "gdb_cmd.sh : generation de cmd.gdb echouee"
         ↪  >&2; exit 1; }
    fi
    km=""
    for c in "$QOSMO_SRC/build/pc-bios/keymaps" "$QOSMO_SRC/pc-bios/keymaps" \
             /opt/GSM/qemu-install/share/qemu/keymaps /usr/share/qemu/keymaps; do
        [ -f "$c/en-us" ] && { km="$c"; break; }
    done
    [ -n "$km" ] || { echo "keymaps QEMU introuvables (en-us)" >&2; exit 1; }
    cp -a "$km" "$D/share/qemu/keymaps"
    DEPS="$(shlib_depends "$D/build/qemu-system-arm")"
    write_preinst_guard "$P/DEBIAN" /opt/GSM/qosmo
    write_control "$P/DEBIAN" <<CTL
Package: qemu-calypso
Version: $V
Section: otherosfs
Priority: optional
Architecture: $ARCH_HOST
Maintainer: $MAINT
Depends: ${DEPS:+$DEPS, }bash, socat, tmux, tcpdump, procps, psmisc, gawk, python3, iproute2, gdb-multiarch,
 ↪  telnet
Recommends: osmo-operator, calypso-firmware
Homepage: https://github.com/bbaranoff/qosmO
Description: QEMU avec la machine calypso (telephone Osmocom-BB emule) et son lanceur
 qemu-system-arm construit depuis qosmo, avec le modele de SoC Calypso
 (ARM946, TPU/TSP, SIM, couche 1 gr-gsm), le lanceur run.sh et ses modules
 (run_modules/), l environnement du banc (environnement/bench.env), les
 configurations du mobile (cfgs/) et les dispositions tmux. Installe en
 /opt/GSM/qosmo, sans les sources ni l arbre de construction.
CTL
    build_pkg "$P" qemu-calypso "$V" "$ARCH_HOST"
fi

# ── calypso-firmware ──────────────────────────────────────────────
if wanted calypso-firmware; then
    FW="$FIRMWARE_SRC/board/compal_e88"
    [ -f "$FW/layer1.highram.elf" ] || { echo "firmware absent : $FW/layer1.highram.elf" >&2; exit 1; }
    V="$(tree_version "$FIRMWARE_SRC")"
    P="$WORK/calypso-firmware"; D="$P/opt/GSM/firmware/board/compal_e88"
    mkdir -p "$D" "$P/DEBIAN"
    install -m644 "$FW/layer1.highram.elf" "$FW/layer1.highram.bin" "$D/"
    [ -f "$FIRMWARE_SRC/COPYING" ] && install -m644 "$FIRMWARE_SRC/COPYING" "$P/opt/GSM/firmware/COPYING"
    write_preinst_guard "$P/DEBIAN" /opt/GSM/firmware
    write_control "$P/DEBIAN" <<CTL
Package: calypso-firmware
Version: $V
Section: misc
Priority: optional
Architecture: all
Maintainer: $MAINT
Homepage: https://github.com/bbaranoff/firmware
Description: firmware layer1 Osmocom-BB (compal_e88) pour le Calypso emule
 layer1.highram.elf (charge par qemu-system-arm -kernel) et layer1.highram.bin
 (envoye par osmocon), pre-compiles depuis osmocom-bb. Chemin fixe attendu par
 bench.env : /opt/GSM/firmware/board/compal_e88/.
CTL
    build_pkg "$P" calypso-firmware "$V" all
fi
echo "termine : $OUT"
```

2.136 /opt/GSM/osmo-operator/packaging/build-pmos-kernel-deb.sh

6850 octets, 122 lignes → 122 lignes

```
#!/bin/bash
# ============================================================================
# build-pmos-kernel-deb.sh - THE postmarketOS KERNEL WITH PPP, AS A .deb.
#
# The phone kernel (linux-postmarketos-stable, rebuilt with CONFIG_PPP for the
# 4G data of the bench modem) is an .apk signed by the pmbootstrap build key.
# It builds in 15 to 40 minutes, under a NON-root account, with loop mounts:
# impossible inside a `docker build`. This script therefore takes the ALREADY
# BUILT kernel (the pmbootstrap work folder of the reference machine) and
# turns it into a Debian package:
#
#     osmo-build-pmos-kernel_<version>+ppp~<suite>_<arch>.deb
#
# which installs, under /opt/user_interface/kernel/pmos/:
#     linux-postmarketos-stable-<ver>.apk   the kernel (signed PPP modules inside)
#     APKINDEX.tar.gz                       the index of the local pmbootstrap repo
#     <key>.rsa.pub                         the PUBLIC key that signed the apk
#     pmaports.commit                       the pmaports commit it comes from
#     pmaports-linux-postmarketos-stable-ppp.patch   the config (CONFIG_PPP=m...)
#
# tools/osmo-pmos-build.sh knows how to read this folder: it copies the apk
# and the index into the local pmbootstrap repo, the key into config_apk_keys,
# and « pmbootstrap install » takes that kernel without recompiling anything.
#
# The .deb goes into /opt/user_interface/kernel/ (where it is stored) AND into
# /var/cache/osmo-debs/: iso_modules/50-injection-image.sh takes every
# osmo-build-*.deb of the suite from there, so the kernel goes into the ISO by itself.
#
#   sudo packaging/build-pmos-kernel-deb.sh [pmbootstrap work folder]
#       default: the « work » folder of the session account's pmbootstrap_v3.cfg
#                (SUDO_USER), otherwise ~<user>/test, otherwise $OSMO_PMB_WORK
# ============================================================================
set -euo pipefail

REPO="$(cd "$(dirname "$0")/.." && pwd)"
UI="${OSMO_UI_ROOT:-/opt/user_interface}"
KDIR="$UI/kernel"
CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
ARCH="$(dpkg --print-architecture 2>/dev/null || echo amd64)"
SUITE="$(. /etc/os-release 2>/dev/null; echo "${VERSION_CODENAME:-noble}")"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'
say() { echo -e "${CYAN}→${NC} $*"; }; ok() { echo -e "${GREEN}✓${NC} $*"; }
die() { echo -e "${RED}✗${NC} $*" >&2; exit 1; }

# The pmbootstrap work folder: argument, otherwise the session account's.
_user="${SUDO_USER:-}"
[ -z "$_user" ] && [ -n "${PKEXEC_UID:-}" ] && _user="$(getent passwd "$PKEXEC_UID" | cut -d: -f1)"
_home="$(getent passwd "${_user:-root}" | cut -d: -f6)"
WORK="${1:-${OSMO_PMB_WORK:-}}"
if [ -z "$WORK" ]; then
    for c in "$_home/.config/pmbootstrap_v3.cfg" "$_home/.config/pmbootstrap.cfg"; do
        [ -f "$c" ] && WORK="$(sed -n 's/^work *= *//p' "$c" | head -1)" && [ -n "$WORK" ] && break
    done
fi
[ -n "$WORK" ] || WORK="$_home/test"
[ -d "$WORK/packages" ] || die "pas de dossier packages/ dans $WORK (dossier de travail pmbootstrap ?)"

apk="$(find "$WORK/packages" -name 'linux-postmarketos-stable-[0-9]*.apk' -printf '%T@ %p\n' 2>/dev/null | sort
↪  -n | tail -1 | cut -d' ' -f2-)"
[ -n "$apk" ] || die "aucun linux-postmarketos-stable-*.apk dans $WORK/packages - le noyau n a pas ete construit
↪  (osmo-pmos-build)"
```

```
# (grep -q closes the pipe before tar ends: under pipefail that would be a
# false failure - hence the process substitution.)
grep -q 'ppp_async\.ko' < <(tar tzf "$apk" 2>/dev/null) || die "$apk n a PAS ppp_async.ko : ce n est pas le noyau
 ↪  PPP"
ver="$(basename "$apk" .apk | sed 's/^linux-postmarketos-stable-//')"          # 7.2.3-r0
dver="$(echo "$ver" | sed 's/-r/./')+ppp"                                       # 7.2.3.0+ppp
index="$(dirname "$apk")/APKINDEX.tar.gz"
channel="$(basename "$(dirname "$(dirname "$apk")")")"                          # edge
parch="$(basename "$(dirname "$apk")")"                                         # x86_64
pub="$(ls -1 "$WORK"/config_abuild/*.rsa.pub 2>/dev/null | head -1)"
[ -n "$pub" ] || pub="$(ls -1 "$WORK"/config_apk_keys/pmos@*.rsa.pub 2>/dev/null | head -1)"
[ -n "$pub" ] || die "pas de cle publique pmos@*.rsa.pub dans $WORK/config_abuild ni config_apk_keys"
aports="$(sed -n 's/^aports *= *//p' "$_home/.config/pmbootstrap_v3.cfg" 2>/dev/null | head -1)"
[ -n "$aports" ] && [ -d "$aports/.git" ] || aports="$WORK/cache_git/pmaports"
commit="$(git -C "$aports" rev-parse HEAD 2>/dev/null || echo inconnu)"

say "noyau : $apk ($ver, $channel/$parch), cle $(basename "$pub"), pmaports $commit"

stage="$(mktemp -d /var/tmp/pmos-kernel-deb.XXXXXX)"; trap 'rm -rf "$stage"' EXIT
d="$stage$KDIR/pmos"
mkdir -p "$d" "$stage/DEBIAN"
cp -f "$apk" "$d/"
[ -f "$index" ] && cp -f "$index" "$d/"
cp -f "$pub" "$d/"
echo "$commit" > "$d/pmaports.commit"
echo "$channel $parch $ver" > "$d/apk.info"
if [ -f "$REPO/patches/pmaports-linux-postmarketos-stable-ppp.patch" ]; then
    cp -f "$REPO/patches/pmaports-linux-postmarketos-stable-ppp.patch" "$d/"
elif git -C "$aports" diff --quiet 2>/dev/null; then :; else
    git -C "$aports" diff > "$d/pmaports-linux-postmarketos-stable-ppp.patch"
fi
cat > "$d/README" <<EOF
Noyau postmarketOS $ver (x86_64, qemu-amd64) rebati avec PPP pour la data 4G
du telephone du banc. Produit par packaging/build-pmos-kernel-deb.sh depuis
$WORK le $(date -I).
  - l apk est signe par $(basename "$pub") : la cle doit etre dans
    config_apk_keys/ du dossier de travail pmbootstrap qui l installe ;
  - pmaports doit etre au commit $commit (meme pkgver/pkgrel : $ver),
    avec le patch ci-joint applique, pour que pmbootstrap le prefere au depot ;
  - tools/osmo-pmos-build.sh (osmo-pmos-build) fait tout cela.
EOF
size="$(du -sk --exclude=DEBIAN "$stage" | cut -f1)"
cat > "$stage/DEBIAN/control" <<CTL
Package: osmo-build-pmos-kernel
Version: ${dver}~${SUITE}
Architecture: ${ARCH}
Maintainer: osmo-operator <bastienbaranoff@gmail.com>
Installed-Size: ${size}
Section: misc
Priority: optional
Description: osmo-operator - noyau postmarketOS ${ver} avec PPP (telephone du banc)
 L apk linux-postmarketos-stable rebati avec CONFIG_PPP pour la data 4G du
 modem du banc, son index, sa cle publique et le patch pmaports, sous
 ${KDIR}/pmos. Lu par osmo-pmos-build (tools/osmo-pmos-build.sh).
CTL
mkdir -p "$KDIR" "$CACHE"
deb="$KDIR/osmo-build-pmos-kernel_${dver}~${SUITE}_${ARCH}.deb"
dpkg-deb --build --root-owner-group -Zzstd "$stage" "$deb" >/dev/null 2>&1 \
    || dpkg-deb --build --root-owner-group -Zxz "$stage" "$deb" >/dev/null
cp -f "$deb" "$CACHE/"
ok "$deb ($(du -h "$deb" | cut -f1)) - copie dans $CACHE"
# And installed on this machine: /opt/user_interface/kernel/pmos filled.
dpkg -i --force-overwrite "$deb" >/dev/null 2>&1 && ok "pose : $KDIR/pmos/" || echo "  (dpkg -i a echoue : dpkg -i
 ↪  $deb)"
```

2.137 /opt/GSM/osmo-operator/packaging/osmo-deb.sh

7486 octets, 172 lignes → 172 lignes

```
#!/bin/bash
# ============================================================================
#  osmo-deb - one .deb per compiled folder, kept in cache, reinstalled at rebuild
# ============================================================================
#
#  PRINCIPLE. Everything compiled in the image (libosmocore, osmo-*, gapk,
#  QEMU, osmocom-bb, gr-gsm...) comes out as a .deb package, stored in
#      /var/cache/osmo-debs/osmo-build-<name>_<version>~<suite>_<arch>.deb
#  At the next rebuild, if the package is there, it is installed (dpkg -i)
#  instead of re-cloning and recompiling. The cache lives ON THE HOST
#  (/var/cache/osmo-debs): build.sh copies it into the context (.deb-cache/)
#  before `docker build`, and brings back the packages the image produced.
#
#  Three verbs:
#
#    osmo-deb install <name> <version>
#        Installs the cached .deb if it exists. Exits 0 in that case, 1
#        otherwise (= compilation needed). With OSMO_DEB_REFRESH=1, always
#        exits 1: the cache is ignored and will be rewritten (build.sh --no-cache).
#
#    osmo-deb pack <name> <version> <command...>
#        Runs <command> with DESTDIR=<staging> in the environment -
#        `make install`, `ninja install`, `cmake --install` all honour it -
#        builds the .deb from the staging, installs it in the root and stores
#        it in the cache. Typical usage:
#            ./configure && make -j && osmo-deb pack libosmocore 1.12.1 make install
#        THE SOURCE TREE GOES WITH IT: the current folder, when under
#        /opt/GSM, is added as is to the package (.git, compiled objects). At
#        a rebuild from the cache, /opt/GSM/<name> thus always exists - the
#        image and the ISO stay a workshop where one can reread, patch and
#        run `make` again. Without it, a full cache would give an image
#        WITHOUT any source. OSMO_DEB_SRC=/path names another tree; OSMO_DEB_NO_SRC=1 drops it.
#
#    osmo-deb snapshot <name> <version> <path...>
#        For what has no install step (a source tree used as is at runtime:
#        osmocom-bb, qosmo-grgsm, a venv): the .deb is built from files
#        ALREADY in place. Nothing is reinstalled.
#
#    osmo-deb list
#
#  The <version> is the caller's choice: the upstream version when pinned
#  (1.12.1), `0.git` when following a branch. The Ubuntu suite name (noble,
#  jammy) is added automatically: a package compiled on jammy is never
#  installed on a noble rootfs.
#
#  Variables: OSMO_DEB_CACHE (default /var/cache/osmo-debs)
#             OSMO_DEB_REFRESH=1  ignores the cache, rebuilds and replaces
# ============================================================================
set -euo pipefail

CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
REFRESH="${OSMO_DEB_REFRESH:-0}"
PREFIX="osmo-build-"
SRC_ROOT="${OSMO_DEB_SRC_ROOT:-/opt/GSM}"   # the source trees live there
ARCH="$(dpkg --print-architecture 2>/dev/null || echo amd64)"
SUITE="$(. /etc/os-release 2>/dev/null; echo "${VERSION_CODENAME:-unknown}")"
MAINT="osmo-operator <bastienbaranoff@gmail.com>"

log() { echo "[osmo-deb] $*" >&2; }
die() { log "ERREUR : $*"; exit 1; }

usage() { sed -n '2,45p' "$0" | sed 's/^#\{1,2\} \{0,1\}//'; exit "${1:-0}"; }
```

```
check_name() {
    [[ "$1" =~ ^[a-z0-9][a-z0-9.+-]*$ ]] || die "nom de paquet invalide : '$1' (minuscules, chiffres, . + -)"
    [[ "$2" =~ ^[0-9][A-Za-z0-9.+~-]*$ ]] || die "version invalide : '$2' (doit commencer par un chiffre)"
}

deb_path() { printf '%s/%s%s_%s~%s_%s.deb' "$CACHE" "$PREFIX" "$1" "$2" "$SUITE" "$ARCH"; }

# Builds the .deb from a staging ($1) for <name> ($2) <version> ($3).
make_deb() {
    local stage="$1" name="$2" ver="$3" deb size
    deb="$(deb_path "$name" "$ver")"
    mkdir -p "$CACHE" "$stage/DEBIAN"
    size="$(du -sk --exclude=DEBIAN "$stage" | cut -f1)"
    cat > "$stage/DEBIAN/control" <<CTL
Package: ${PREFIX}${name}
Version: ${ver}~${SUITE}
Architecture: ${ARCH}
Maintainer: ${MAINT}
Installed-Size: ${size}
Section: misc
Priority: optional
Description: osmo-operator - ${name} ${ver}, compile dans l image (${SUITE})
 Paquet produit par osmo-deb au build de l image osmocom-nitb. Il ne declare
 aucune dependance : les bibliotheques systeme viennent de la liste apt du
 Dockerfile, qui fait autorite.
CTL
    # zstd when dpkg supports it (jammy and noble), xz otherwise: on a 1.5 GB
    # QEMU tree the difference is several minutes.
    rm -f "$deb.tmp"
    if ! dpkg-deb --build --root-owner-group -Zzstd "$stage" "$deb.tmp" >/dev/null 2>&1; then
        dpkg-deb --build --root-owner-group -Zxz "$stage" "$deb.tmp" >/dev/null
    fi
    mv -f "$deb.tmp" "$deb"
    log "$name $ver -> $deb ($(du -h "$deb" | cut -f1))"
}

cmd_install() {
    [ $# -eq 2 ] || usage 2
    check_name "$1" "$2"
    local deb; deb="$(deb_path "$1" "$2")"
    if [ "$REFRESH" = "1" ]; then
        log "$1 $2 : cache ignore (OSMO_DEB_REFRESH=1) -> compilation"; return 1
    fi
    if [ ! -s "$deb" ]; then
        log "$1 $2 : pas en cache ($(basename "$deb")) -> compilation"; return 1
    fi
    log "$1 $2 : installe depuis le cache ($(basename "$deb"))"
    dpkg -i --force-overwrite "$deb" >/dev/null
    ldconfig 2>/dev/null || true
    return 0
}

cmd_pack() {
    [ $# -ge 3 ] || usage 2
    local name="$1" ver="$2"; shift 2
    check_name "$name" "$ver"
    local stage; stage="$(mktemp -d "${TMPDIR:-/tmp}/osmo-deb.XXXXXX")"
    DESTDIR="$stage" "$@"
    if [ -z "$(find "$stage" -mindepth 1 -maxdepth 1 ! -name DEBIAN 2>/dev/null)" ]; then
        rm -rf "$stage"
        die "$name : rien n a ete installe dans DESTDIR - la commande l ignore-t-elle ?"
    fi
    # The source tree (the current folder, under /opt/GSM) goes into the
    # package: the sources stay in /opt/GSM even when the cache spares the
    # cloning and compiling. See the header.
```

```
    local src="${OSMO_DEB_SRC:-$PWD}"
    if [ "${OSMO_DEB_NO_SRC:-0}" != "1" ] && [ -d "$src" ] \
       && case "$src" in "$SRC_ROOT"/?*) true ;; *) false ;; esac; then
        mkdir -p "$stage$(dirname "$src")"
        # `cp -an`: the install prefix may live UNDER the source tree (Open5GS:
        # /opt/LTE/open5gs/install), in which case it is ALREADY in the
        # staging. The copy completes around what is already there (-n: never
        # over what the install command just put in place).
        cp -an "$src" "$stage$(dirname "$src")/"
        log "$name : arbre de sources $src inclus dans le paquet"
    fi
    make_deb "$stage" "$name" "$ver"
    dpkg -i --force-overwrite "$(deb_path "$name" "$ver")" >/dev/null
    ldconfig 2>/dev/null || true
    rm -rf "$stage"
}

cmd_snapshot() {
    [ $# -ge 3 ] || usage 2
    local name="$1" ver="$2"; shift 2
    check_name "$name" "$ver"
    local stage p; stage="$(mktemp -d "${TMPDIR:-/tmp}/osmo-deb.XXXXXX")"
    for p in "$@"; do
        [ -e "$p" ] || { rm -rf "$stage"; die "$name : $p n existe pas"; }
        case "$p" in /*) ;; *) rm -rf "$stage"; die "$name : chemin absolu requis ($p)" ;; esac
        mkdir -p "$stage$(dirname "$p")"
        cp -a "$p" "$stage$(dirname "$p")/"
    done
    make_deb "$stage" "$name" "$ver"
    rm -rf "$stage"
}

cmd_list() {
    ls -1sh "$CACHE"/${PREFIX}*.deb 2>/dev/null || log "cache vide ($CACHE)"
}

case "${1:-}" in
    install)  shift; cmd_install "$@" ;;
    pack)     shift; cmd_pack "$@" ;;
    snapshot) shift; cmd_snapshot "$@" ;;
    list)     cmd_list ;;
    -h|--help|help|'') usage 0 ;;
    *) die "verbe inconnu : $1 (install|pack|snapshot|list)" ;;
esac
```

2.138 /opt/GSM/osmo-operator/packaging/snapshot-lte-debs.sh

7071 octets, 112 lignes → 112 lignes

```
#!/bin/bash
# ============================================================================
# snapshot-lte-debs.sh - THE BENCH 4G AND PHONE, AS .deb, FROM THE NATIVE
# INSTALL - the shortcut.
#
# Two paths lead to the same packages:
#   1. THE SHORTCUT (this script): the reference machine ALREADY has srsRAN
#      (ZeroMQ) compiled in /opt/LTE/srsRAN_4G, libzmq/czmq in /opt/LTE,
#      Open5GS in /opt/LTE/open5gs (install/ prefix), and the postmarketOS PPP
#      kernel in the session account's pmbootstrap work folder. All of it is
#      snapshotted into packages, as is (osmo-deb snapshot: nothing is
#      recompiled, nothing is reinstalled):
#          osmo-build-libzmq_4.3.5+git~noble_amd64.deb
#          osmo-build-srsgui_0.1+git~noble_amd64.deb       (before srsran: it links to it)
#          osmo-build-srsran_25.10+zmq+x86-64-v2~noble_amd64.deb  (ISA included)
```

```
#          osmo-build-open5gs_2.8.0+git~noble_amd64.deb
#          osmo-build-pmos-kernel_7.2.3.0+ppp~noble_amd64.deb   (packaging/build-pmos-kernel-deb.sh)
#      into /var/cache/osmo-debs: the ISO build (50-injection-image.sh) puts
#      them in the rootfs, and the Dockerfile (osmo-deb install) takes them
#      instead of compiling.
#   2. THE SOURCES: the Dockerfile and tools/osmo-lte-install.sh --build
#      compile srsRAN and Open5GS when the cache lacks these packages. The
#      pmOS kernel, however, is only built by osmo-pmos-build (pmbootstrap,
#      under the session account): it has NO docker path.
#
#   sudo packaging/snapshot-lte-debs.sh            everything that is there
#   sudo packaging/snapshot-lte-debs.sh --no-pmos  without the phone kernel
# ============================================================================
set -u
REPO="$(cd "$(dirname "$0")/.." && pwd)"
LTE="${OSMO_LTE_ROOT:-/opt/LTE}"
CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
OSMODEB="$REPO/packaging/osmo-deb.sh"
PMOS=1; [ "${1:-}" = "--no-pmos" ] && PMOS=0
GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'
say() { echo -e "${CYAN}→${NC} $*"; }; ok() { echo -e "${GREEN}✓${NC} $*"; }; warn() { echo -e "${YELLOW}!${NC}
 ↪  $*"; }
[ "$(id -u)" -eq 0 ] || { echo "root requis" >&2; exit 1; }
export OSMO_DEB_CACHE="$CACHE"
mkdir -p "$CACHE"

# ── libzmq + czmq (compiled by hand in /opt/LTE, installed in /usr/local) ────
if [ -e /usr/local/lib/libzmq.so.5 ]; then
    say "libzmq / czmq"
    _z=( /usr/local/lib/libzmq.so* /usr/local/lib/libzmq.la /usr/local/lib/pkgconfig/libzmq.pc
     ↪  /usr/local/include/zmq.h /usr/local/include/zmq_utils.h )
    [ -e /usr/local/lib/libczmq.so ] && _z+=( /usr/local/lib/libczmq.so* /usr/local/lib/libczmq.la
     ↪  /usr/local/lib/pkgconfig/libczmq.pc /usr/local/include/czmq.h /usr/local/include/czmq_library.h
     ↪  /usr/local/include/czmq_prelude.h )
    [ -x /usr/local/bin/zmakecert ] && _z+=( /usr/local/bin/zmakecert )
    for d in "$LTE/libzmq" "$LTE/czmq"; do [ -d "$d" ] && _z+=( "$d" ); done
    _zv="$(git -C "$LTE/libzmq" describe --tags 2>/dev/null | sed 's/^v//; s/-.*//')"
    bash "$OSMODEB" snapshot libzmq "${_zv:-4.3.5}+git" "${_z[@]}" && ok "libzmq"
else
    warn "pas de /usr/local/lib/libzmq.so.5 : libzmq d Ubuntu (libzmq5) suffira"
fi

# ── srsGUI: the real-time plots, BEFORE srsRAN (which links to it) ──────────
# srsRAN_4G is compiled with ENABLE_GUI=ON: srsenb and srsue LINK to
# libsrsgui. A srsRAN .deb without it would give binaries that do not start
# (« libsrsgui.so: cannot open shared object file »). So it comes first, in
# this file as in the cache.
if [ -e /usr/local/lib/libsrsgui.so ] || [ -e /usr/local/lib/libsrsgui.a ]; then
    say "srsGUI (traces srsenb / srsue)"
    _g=( /usr/local/lib/libsrsgui.* )
    [ -d /usr/local/include/srsgui ] && _g+=( /usr/local/include/srsgui )
    [ -e /usr/local/lib/pkgconfig/srsgui.pc ] && _g+=( /usr/local/lib/pkgconfig/srsgui.pc )
    [ -d "$LTE/srsGUI" ] && _g+=( "$LTE/srsGUI" )
    bash "$OSMODEB" snapshot srsgui "${OSMO_SRSGUI_VER:-0.1+git}" "${_g[@]}" && ok "srsgui"
else
    warn "pas de /usr/local/lib/libsrsgui : srsGUI non empaquete (srsRAN sera sans traces)"
fi

# ── srsRAN_4G: the tree (sources + build) and what make install put in place ─
if [ -f "$LTE/srsRAN_4G/build/install_manifest.txt" ]; then
    say "srsRAN_4G (ZeroMQ)"
    mapfile -t _s < <(grep -v '^$' "$LTE/srsRAN_4G/build/install_manifest.txt" | while read -r f; do [ -e "$f" ]
     ↪  && echo "$f"; done)
    _sv="$(git -C "$LTE/srsRAN_4G" describe --tags 2>/dev/null | sed 's/^release_//; s/-.*//; s/_/./g')"
    # ── THE ISA GOES INTO THE VERSION, AND IT IS OBSERVED, NOT ASSUMED ──────
    # This script SNAPSHOTS already compiled binaries: nothing here says which
```

```
    # -march they were built with. A srsenb built natively on the reference
    # machine would satisfy the Dockerfile's « osmo-deb install srsran », go
    # into the image and the ISO, and raise « Illegal instruction » on the
    # operator's machine (see SRS_ARCH in tools/osmo-lte-install.sh). So what
    # the binary CONTAINS is inspected - one %ymm/%zmm instruction and it is
    # native - and stated in its name. Without objdump nothing is decided:
    # +inconnu, which can never be confused with the portable one either.
    _sisa=inconnu
    if command -v objdump >/dev/null 2>&1 && [ -x /usr/local/bin/srsenb ]; then
        if objdump -d /usr/local/bin/srsenb 2>/dev/null | grep -qE '%[yz]mm[0-9]'; then _sisa=native
        else _sisa="$(printf '%s' "${OSMO_SRSRAN_ARCH:-x86-64-v2}" | tr -c 'A-Za-z0-9.' '-')"; fi
    fi
    [ "$_sisa" = native ] && warn "srsenb porte de l AVX : paquete en +native (il ne tournera que sur CETTE
     ↪  machine)"
    bash "$OSMODEB" snapshot srsran "${_sv:-25.10}+zmq+${_sisa}" "$LTE/srsRAN_4G" "${_s[@]}" && ok "srsran
     ↪  ${_sv:-25.10}+zmq+${_sisa} (${#_s[@]} fichiers installes + l arbre)"
else
    warn "pas de $LTE/srsRAN_4G/build/install_manifest.txt : srsRAN non empaquete"
fi

# ── Open5GS : l arbre entier, prefixe install/ compris ──────────────────
if [ -x "$LTE/open5gs/install/bin/open5gs-mmed" ]; then
    say "Open5GS ($LTE/open5gs)"
    _ov="$(git -C "$LTE/open5gs" describe --tags 2>/dev/null | sed 's/^v//; s/-.*//')"
    # The logs and the .ctxt do not travel.
    find "$LTE/open5gs/install/var/log" -type f -delete 2>/dev/null || true
    rm -f "$LTE/open5gs/.ctxt"
    bash "$OSMODEB" snapshot open5gs "${_ov:-2.8.0}+git" "$LTE/open5gs" && ok "open5gs"
else
    warn "pas de $LTE/open5gs/install/bin/open5gs-mmed : Open5GS non empaquete (il vit encore dans /root ?)"
fi

# ── The postmarketOS PPP kernel ─────────────────────────────────────────
if [ "$PMOS" = 1 ]; then
    say "noyau postmarketOS (PPP)"
    bash "$REPO/packaging/build-pmos-kernel-deb.sh" "${OSMO_PMB_WORK:-}" || warn "noyau pmOS non empaquete (voir
     ↪  ci-dessus)"
fi
echo; bash "$OSMODEB" list 2>/dev/null | grep -E 'libzmq|srsgui|srsran|open5gs|pmos-kernel' || true
```

2.139 /opt/GSM/osmo-operator/patches/fc-magnetite-alr-arfcn-forced.patch

9123 octets, 235 lignes → 235 lignes

```
--- a/src/aci2/alr/alr_cs.c
+++ b/src/aci2/alr/alr_cs.c
@@ -44,6 +44,11 @@
 #include "alr_em.h"
 #include "cl_list.h"

+/* [2026-10-21] banc osmo-operator : ARFCN force par defaut (voir cs_rxlev_ind) */
+#ifndef ARFCN_FORCED
+#define ARFCN_FORCED 1
+#endif
+
 #if defined (_SIMULATION_)
 #define TRACING
 #endif
@@ -828,6 +833,32 @@
         /* Sort the channels based on their RxLev */
         cs_add_and_sort_channels ();

+#if ARFCN_FORCED >= 0
+        /* [2026-10-21] banc d'emulation osmo-operator : seule la cellule de l'ARFCN_FORCED est presentee
```

```
+         * a RR (les autres porteuses sont retirees de la liste triee ; ajoutee au niveau max si le
+         * balayage ne l'a pas vue). -DARFCN_FORCED=<n> pour une autre, -1 pour le comportement normal. */
+        {
+          U16 k, n = 0;
+          for (k = 0; k < mph_power_cnf->num_of_chan; k++)
+          {
+            if ((mph_power_cnf->arfcn[k] & 0x03FF) EQ ARFCN_FORCED)
+            {
+              mph_power_cnf->arfcn[n]  = mph_power_cnf->arfcn[k];
+              mph_power_cnf->rx_lev[n] = mph_power_cnf->rx_lev[k];
+              n++;
+            }
+          }
+          if (n EQ 0)
+          {
+            mph_power_cnf->arfcn[0]  = ARFCN_FORCED;
+            mph_power_cnf->rx_lev[0] = 63;
+            n = 1;
+          }
+          mph_power_cnf->num_of_chan = (U8)n;
+          TRACE_EVENT_P2 ("ARFCN_FORCED=%u : %u porteuse(s) presentee(s) a RR", ARFCN_FORCED, n);
+        }
+#endif
+
          if(!IS_EXT_MEAS_RUNNING)
          {
            /* Free MPH_POWER_REQ buffer */
--- a/src/ui3/bmi/mmiDummy.c
+++ b/src/ui3/bmi/mmiDummy.c
@@ -154,12 +154,34 @@
  *
  ******************************************************************************/

+/* [2026-10-21] banc osmo-operator : un handle de fenetre perime (petite valeur, fenetre detruite) faisait
+ * lire hdr->data en flash a ~0 (vecteur 0xEA003FFD) puis faute de donnees (DFAR 0xEA003FFD, LR dans
+ * mmi_send_event) ; le gestionnaire d'abort bouclait et tout le firmware gelait (ATI muet). On ne
+ * suit que des pointeurs en RAM (interne 0x800000-0x83FFFF, externe 0x1000000-0x17FFFFF). */
+#define MMI_EN_RAM(p) ((((ULONG)(p)) >= 0x00800000UL && ((ULONG)(p)) < 0x00840000UL) || \
+                       (((ULONG)(p)) >= 0x01000000UL && ((ULONG)(p)) < 0x01800000UL))
+
 void mmi_send_event (T_MFW_HND win, USHORT event, USHORT value, void * parameter)
 {

    if (win){
-       T_MFW_WIN     * win_data  = ((T_MFW_HDR *)win)->data;
-         T_MMI_CONTROL * control  = (T_MMI_CONTROL *)win_data->user;
+       T_MFW_WIN     * win_data;
+       T_MMI_CONTROL * control;
+
+       if (!MMI_EN_RAM(win)) {
+           TRACE_EVENT_P2("mmi_send_event: handle invalide 0x%08x (evt %d) ignore", (ULONG)win, event);
+           return;
+       }
+       win_data = ((T_MFW_HDR *)win)->data;
+       if (!MMI_EN_RAM(win_data)) {
+           TRACE_EVENT_P2("mmi_send_event: win_data invalide 0x%08x (evt %d) ignore", (ULONG)win_data, event);
+           return;
+       }
+       control = (T_MMI_CONTROL *)win_data->user;
+       if (!MMI_EN_RAM(control)) {
+           TRACE_EVENT_P2("mmi_send_event: control invalide 0x%08x (evt %d) ignore", (ULONG)control, event);
+           return;
+       }

        if (control->dialog NEQ 0)
            (control->dialog) (win, event, value, parameter);
```

```
--- a/src/ui3/mfw/mfw_nm.c
+++ b/src/ui3/mfw/mfw_nm.c
@@ -1259,6 +1259,40 @@
 }


+/* [2026-10-21] banc osmo-operator : enregistrement robuste aux touches.
+ * - BANC_REG_AUTO (defaut 1) : nm_registration() part toujours en automatique. Le mode manuel
+ *   memorise en FFS (/mmi/mmidata) sans operateur connu (LOCI vierge) laissait le MM sans
+ *   recherche (+CREG 0) et le MMI affichait "Network Forbidden".
+ * - nm_nrg_robuste() : si l'ACI repond AT_BUSY (une commande MM en cours, liste PLMN lancee au
+ *   clavier, +COPS d'un autre canal...), on l'abandonne et on relance une fois. */
+#ifndef BANC_REG_AUTO
+#define BANC_REG_AUTO 1
+#endif
+static U8 nm_reg_attente;   /* nm_registration() refusee faute de SIM active : a relancer */
+void nm_reg_en_attente (void)
+{
+    if (!nm_reg_attente)
+        return;
+    nm_reg_attente = 0;
+    TRACE_EVENT("nm_reg_en_attente: SIM active, enregistrement relance");
+    nm_registration(NM_AUTOMATIC, 0, 0);
+}
+
+static T_ACI_RETURN nm_nrg_robuste (T_ACI_NRG_RGMD rm, char *opr)
+{
+    T_ACI_RETURN r = sAT_PercentNRG(CMD_SRC_LCL, rm, NRG_SVMD_Full, NRG_FRMT_Numeric, opr);
+    if (r == AT_BUSY)
+    {
+        TRACE_EVENT("nm_nrg_robuste: AT_BUSY, abandon COPS/NRG et relance");
+        sAT_Abort(CMD_SRC_LCL, AT_CMD_COPS);
+        sAT_Abort(CMD_SRC_LCL, AT_CMD_NRG);
+        r = sAT_PercentNRG(CMD_SRC_LCL, rm, NRG_SVMD_Full, NRG_FRMT_Numeric, opr);
+    }
+    if (r != AT_EXCT && r != AT_CMPL)
+        TRACE_EVENT_P1("nm_nrg_robuste: sAT_PercentNRG -> %d", r);
+    return r;
+}
+
 /*
  +--------------------------------------------------------------------+
  | PROJECT : MMI-Framework (8417)        MODULE  : MFW_NM             |
@@ -1296,7 +1330,19 @@
       return IMEI_NOT_VALID;

     if (!limited && sim_stat != SIM_ACTIVE)
+    {
+      /* [2026-10-21] banc : relancee par nm_reg_en_attente() quand la SIM devient active */
+      nm_reg_attente = 1;
+      TRACE_EVENT("nm_registration: SIM pas encore active, enregistrement mis en attente");
       return SIM_NOT_ACTIVE;
+    }
+    nm_reg_attente = 0;
+
+#if BANC_REG_AUTO
+    if (mode == NM_MANUAL)
+        TRACE_EVENT("nm_registration: mode manuel memorise -> automatique (BANC_REG_AUTO)");
+    mode = NM_AUTOMATIC;
+#endif


     if (mode == NM_MANUAL)
@@ -1310,22 +1356,14 @@
                         to the last manual selected HPLMN cell before the power cycle, and this does not happen;
                           instead this, the sample shows up the available networks list
```

```
            Fix         :  call to nm_avail_plmn_list() is changed to call to sAT_PercentNRG() */
-       if (sAT_PercentNRG(CMD_SRC_LCL,
-                          NRG_RGMD_Manual,
-                          NRG_SVMD_Full,
-                          NRG_FRMT_Numeric,
-                          NULL) != AT_EXCT)
+        if (nm_nrg_robuste(NRG_RGMD_Manual, NULL) != AT_EXCT)
         {
             TRACE_ERROR("sAT_PercentNRG error");
         }
     }
     else
     {
-       if (sAT_PercentNRG(CMD_SRC_LCL,
-                          NRG_RGMD_Auto,
-                          NRG_SVMD_Full,
-                          NRG_FRMT_Numeric,
-                          NULL) != AT_EXCT)
+        if (nm_nrg_robuste(NRG_RGMD_Auto, NULL) != AT_EXCT)
         {
             TRACE_ERROR("sAT_PercentNRG error");
         }
@@ -1856,12 +1894,9 @@
     T_ACI_RETURN result;
     TRACE_FUNCTION("nm_select_plmn()");

-   result = sAT_PercentNRG(CMD_SRC_LCL,
-                       NRG_RGMD_Manual,
-                       NRG_SVMD_Full,
-                       NRG_FRMT_Numeric,
-                       (char *) network);
-   result = AT_FAIL;
+   /* [2026-10-21] banc : le "result = AT_FAIL;" d'origine qui suivait cet appel faisait
+    * tracer une erreur (et tester un qAT_ErrDesc perime) a chaque choix de reseau au clavier. */
+   result = nm_nrg_robuste(NRG_RGMD_Manual, (char *) network);
     if (result != AT_EXCT)
     {
         TRACE_ERROR("sAT_PercentNRG error");
--- a/src/ui3/mfw/mfw_sim.c
+++ b/src/ui3/mfw/mfw_sim.c
@@ -656,8 +656,21 @@

 */

+/* [2026-10-21] banc osmo-operator : course au boot. nm_registration() appelee avant que la SIM
+ * soit active rendait SIM_NOT_ACTIVE sans rien lancer, et personne ne la rappelait (+CREG 0,0,
+ * pas de RACH, "Network Forbidden"). Chaque changement d etat SIM passe par sim_signal() : on
+ * y relance l enregistrement en attente (mfw_nm.c, nm_reg_en_attente) quand la SIM devient active. */
+static void sim_signal_mmi (MfwEvt event, void *para);
+EXTERN void nm_reg_en_attente (void);
 void sim_signal (MfwEvt event, void *para)
 {
+    sim_signal_mmi(event, para);
+    if (sim_stat == SIM_ACTIVE)
+        nm_reg_en_attente();
+}
+
+static void sim_signal_mmi (MfwEvt event, void *para)
+{
   UBYTE temp = dspl_Enable(0);
     TRACE_FUNCTION ("sim_signal()");

--- a/src/cs/layer1/dyn_dwl_cfile/l1_dyn_dwl_async.c
+++ b/src/cs/layer1/dyn_dwl_cfile/l1_dyn_dwl_async.c
@@ -182,6 +182,18 @@
   UWORD16 temp_patch_id[MAX_NUM_OF_PATCH_IDS];
```

```
   BOOL return_flag = FALSE;

+  /* [2026-10-21] banc osmo-operator (BANC_SANS_DYN_DWNLD, defaut 1) : aucun telechargement dynamique. Le DSP
+   * emule n a pas le patch TI qui sert ce protocole : les pointeurs d_api_dwl_* du NDB etaient faux et la copie
+   * du patch (l1_dyn_dwl_apihisr.c) ecrasait la table des vecteurs ARM a 0x0 a l entree en dedie (D_DWL_C puis
+   * abort en boucle). Sans patch charge (DWNLD 0) il n y a rien a completer. L1_DYN_DSP_DWNLD 0 ne compile pas
+   * (variante audio AMR sans en-tetes). */
+  #ifndef BANC_SANS_DYN_DWNLD
+  #define BANC_SANS_DYN_DWNLD 1
+  #endif
+  #if BANC_SANS_DYN_DWNLD
+  return FALSE;
+  #endif
+
   // Primitive is processed only if it triggers a dynamic download or there is a delay
   if(delay_flag == TRUE || l1_does_the_incoming_primitive_trigger_dynamic_dwnld(SignalCode, FALSE) == TRUE)
   {
```

2.140 /opt/GSM/osmo-operator/patches/fc-magnetite-banc.patch

14892 octets, 368 lignes → 368 lignes

```
diff -r 9394305d4ff5 src/aci2/alr/alr_cs.c
--- a/src/aci2/alr/alr_cs.c Mon Oct 31 00:14:44 2022 +0000
+++ b/src/aci2/alr/alr_cs.c Fri Oct 23 19:18:45 2026 +0200
@@ -44,6 +44,11 @@
 #include "alr_em.h"
 #include "cl_list.h"

+/* [2026-10-21] banc osmo-operator : ARFCN force par defaut (voir cs_rxlev_ind) */
+#ifndef ARFCN_FORCED
+#define ARFCN_FORCED 1
+#endif
+
 #if defined (_SIMULATION_)
 #define TRACING
 #endif
@@ -828,6 +833,32 @@
         /* Sort the channels based on their RxLev */
         cs_add_and_sort_channels ();

+#if ARFCN_FORCED >= 0
+        /* [2026-10-21] banc d'emulation osmo-operator : seule la cellule de l'ARFCN_FORCED est presentee
+         * a RR (les autres porteuses sont retirees de la liste triee ; ajoutee au niveau max si le
+         * balayage ne l'a pas vue). -DARFCN_FORCED=<n> pour une autre, -1 pour le comportement normal. */
+        {
+          U16 k, n = 0;
+          for (k = 0; k < mph_power_cnf->num_of_chan; k++)
+          {
+            if ((mph_power_cnf->arfcn[k] & 0x03FF) EQ ARFCN_FORCED)
+            {
+              mph_power_cnf->arfcn[n]  = mph_power_cnf->arfcn[k];
+              mph_power_cnf->rx_lev[n] = mph_power_cnf->rx_lev[k];
+              n++;
+            }
+          }
+          if (n EQ 0)
+          {
+            mph_power_cnf->arfcn[0]  = ARFCN_FORCED;
+            mph_power_cnf->rx_lev[0] = 63;
+            n = 1;
+          }
+          mph_power_cnf->num_of_chan = (U8)n;
+          TRACE_EVENT_P2 ("ARFCN_FORCED=%u : %u porteuse(s) presentee(s) a RR", ARFCN_FORCED, n);
```

```
+        }
+#endif
+
          if(!IS_EXT_MEAS_RUNNING)
          {
            /* Free MPH_POWER_REQ buffer */
diff -r 9394305d4ff5 src/cs/layer1/dyn_dwl_cfile/l1_dyn_dwl_async.c
--- a/src/cs/layer1/dyn_dwl_cfile/l1_dyn_dwl_async.c    Mon Oct 31 00:14:44 2022 +0000
+++ b/src/cs/layer1/dyn_dwl_cfile/l1_dyn_dwl_async.c    Fri Oct 23 19:18:45 2026 +0200
@@ -182,6 +182,18 @@
   UWORD16 temp_patch_id[MAX_NUM_OF_PATCH_IDS];
   BOOL return_flag = FALSE;

+  /* [2026-10-21] banc osmo-operator (BANC_SANS_DYN_DWNLD, defaut 1) : aucun telechargement dynamique. Le DSP
+   * emule n a pas le patch TI qui sert ce protocole : les pointeurs d_api_dwl_* du NDB etaient faux et la copie
+   * du patch (l1_dyn_dwl_apihisr.c) ecrasait la table des vecteurs ARM a 0x0 a l entree en dedie (D_DWL_C puis
+   * abort en boucle). Sans patch charge (DWNLD 0) il n y a rien a completer. L1_DYN_DSP_DWNLD 0 ne compile pas
+   * (variante audio AMR sans en-tetes). */
+  #ifndef BANC_SANS_DYN_DWNLD
+  #define BANC_SANS_DYN_DWNLD 1
+  #endif
+  #if BANC_SANS_DYN_DWNLD
+  return FALSE;
+  #endif
+
   // Primitive is processed only if it triggers a dynamic download or there is a delay
   if(delay_flag == TRUE || l1_does_the_incoming_primitive_trigger_dynamic_dwnld(SignalCode, FALSE) == TRUE)
   {
diff -r 9394305d4ff5 src/ui3/bmi/mmiDummy.c
--- a/src/ui3/bmi/mmiDummy.c    Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/bmi/mmiDummy.c    Fri Oct 23 19:18:45 2026 +0200
@@ -154,12 +154,34 @@
  *
  ******************************************************************************************/

+/* [2026-10-21] banc osmo-operator : un handle de fenetre perime (petite valeur, fenetre detruite) faisait
+ * lire hdr->data en flash a ~0 (vecteur 0xEA003FFD) puis faute de donnees (DFAR 0xEA003FFD, LR dans
+ * mmi_send_event) ; le gestionnaire d'abort bouclait et tout le firmware gelait (ATI muet). On ne
+ * suit que des pointeurs en RAM (interne 0x800000-0x83FFFF, externe 0x1000000-0x17FFFFF). */
+#define MMI_EN_RAM(p) ((((ULONG)(p)) >= 0x00800000UL && ((ULONG)(p)) < 0x00840000UL) || \
+                       (((ULONG)(p)) >= 0x01000000UL && ((ULONG)(p)) < 0x01800000UL))
+
 void mmi_send_event (T_MFW_HND win, USHORT event, USHORT value, void * parameter)
 {

    if (win){
-       T_MFW_WIN     * win_data  = ((T_MFW_HDR *)win)->data;
-         T_MMI_CONTROL * control  = (T_MMI_CONTROL *)win_data->user;
+       T_MFW_WIN     * win_data;
+       T_MMI_CONTROL * control;
+
+       if (!MMI_EN_RAM(win)) {
+           TRACE_EVENT_P2("mmi_send_event: handle invalide 0x%08x (evt %d) ignore", (ULONG)win, event);
+           return;
+       }
+       win_data = ((T_MFW_HDR *)win)->data;
+       if (!MMI_EN_RAM(win_data)) {
+           TRACE_EVENT_P2("mmi_send_event: win_data invalide 0x%08x (evt %d) ignore", (ULONG)win_data, event);
+           return;
+       }
+       control = (T_MMI_CONTROL *)win_data->user;
+       if (!MMI_EN_RAM(control)) {
+           TRACE_EVENT_P2("mmi_send_event: control invalide 0x%08x (evt %d) ignore", (ULONG)control, event);
+           return;
+       }
```

```
        if (control->dialog NEQ 0)
            (control->dialog) (win, event, value, parameter);
diff -r 9394305d4ff5 src/ui3/bmi/mmiSmsRead.c
--- a/src/ui3/bmi/mmiSmsRead.c  Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/bmi/mmiSmsRead.c  Fri Oct 23 19:18:45 2026 +0200
@@ -777,6 +777,11 @@
            {
                for (i = 0; i < ((data->sms_list_data).nb_sms_stored); i++)
                    {
+#ifdef FF_MMI_SMS_DYNAMIC
+                    /* [banc 2026-10-22] deja libere par E_EXIT, ou jamais alloue (liste vide) */
+                    if (data->sms_list_data.sms_header == NULL || data->sms_list_data.sms_header[i] == NULL)
+                        continue;
+#endif
                    FREE_MEMORY( (void *)data->sms_list_data.sms_header[i], UCS2_MENU_TEXT_LEN+ 1/*SPR1242*/);
                    }
                /*SPR 2686, only de-allocate memory for the number of messages displayed*/
@@ -784,8 +789,11 @@
                FREE_MEMORY ((UBYTE *)data->menu_list_data, sizeof(ListMenuData));
            }
 #ifdef FF_MMI_SMS_DYNAMIC
-           FREE_MEMORY((void*)data->sms_list_data.sms_messages,g_max_messages * sizeof(T_MFW_SMS_MSG) );
-           FREE_MEMORY((void*)data->sms_list_data.sms_header, g_max_messages * sizeof(wstring_t));
+           /* [banc 2026-10-22] gardes : E_EXIT a pu les liberer deja (remis a NULL) */
+           if (data->sms_list_data.sms_messages != NULL)
+               FREE_MEMORY((void*)data->sms_list_data.sms_messages,g_max_messages * sizeof(T_MFW_SMS_MSG) );
+           if (data->sms_list_data.sms_header != NULL)
+               FREE_MEMORY((void*)data->sms_list_data.sms_header, g_max_messages * sizeof(wstring_t));
 #endif
            FREE_MEMORY ((void *)data, sizeof (T_SMSREAD_R));
            sms_read_win = 0;
@@ -1012,18 +1020,32 @@
        l_parent = data->parent;
        SmsRead_R_destroy(win);
        SEND_EVENT(l_parent, E_RETURN, data->id, NULL);
-
+       /* [banc 2026-10-22] break ajoute : E_EXIT relisait data/list_data deja liberes par SmsRead_R_destroy */
+       break;
+
    case E_EXIT: // Terminate exit from sms send...
    // May 27, 2005    MMI-FIX-29869 x0018858
 #ifdef FF_MMI_SMS_DYNAMIC
     {
           int i;
-       for (i=0;i<list_data->nb_sms_stored; i++)
-           {
-               FREE_MEMORY((U8*)list_data->sms_header[i], UCS2_MENU_TEXT_LEN + 1 );
-           }
-       FREE_MEMORY((U8*)list_data->sms_header, g_max_messages * sizeof(wstring_t) );
-       FREE_MEMORY((U8*)list_data->sms_messages, g_max_messages * sizeof(T_MFW_SMS_MSG));
+       /* [banc 2026-10-22] gardes + NULL : sans eux SmsRead_R_destroy liberait une 2e fois (tas MMI
+          corrompu,  Preceeding Sentinal Bytes have been corrupted ), et sms_header n'existe pas si la
+          liste est vide */
+       if (list_data->sms_header != NULL)
+       {
+           for (i=0;i<list_data->nb_sms_stored; i++)
+               {
+                   if (list_data->sms_header[i] != NULL)
+                       FREE_MEMORY((U8*)list_data->sms_header[i], UCS2_MENU_TEXT_LEN + 1 );
+                   list_data->sms_header[i] = NULL;
+               }
+           FREE_MEMORY((U8*)list_data->sms_header, g_max_messages * sizeof(wstring_t) );
+           list_data->sms_header = NULL;
+       }
+       if (list_data->sms_messages != NULL)
+           FREE_MEMORY((U8*)list_data->sms_messages, g_max_messages * sizeof(T_MFW_SMS_MSG));
```

```
+       list_data->sms_messages = NULL;
+       list_data->nb_sms_stored = 0;
    }
 #endif
        listDisplayListMenu(win, data->menu_list_data, (ListCbFunc_t)SmsRead_R_smslist_cb,0);
diff -r 9394305d4ff5 src/ui3/mfw/mfw_nm.c
--- a/src/ui3/mfw/mfw_nm.c  Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/mfw/mfw_nm.c  Fri Oct 23 19:18:45 2026 +0200
@@ -1259,6 +1259,40 @@
 }


+/* [2026-10-21] banc osmo-operator : enregistrement robuste aux touches.
+ * - BANC_REG_AUTO (defaut 1) : nm_registration() part toujours en automatique. Le mode manuel
+ *   memorise en FFS (/mmi/mmidata) sans operateur connu (LOCI vierge) laissait le MM sans
+ *   recherche (+CREG 0) et le MMI affichait "Network Forbidden".
+ * - nm_nrg_robuste() : si l'ACI repond AT_BUSY (une commande MM en cours, liste PLMN lancee au
+ *   clavier, +COPS d'un autre canal...), on l'abandonne et on relance une fois. */
+#ifndef BANC_REG_AUTO
+#define BANC_REG_AUTO 1
+#endif
+static U8 nm_reg_attente;   /* nm_registration() refusee faute de SIM active : a relancer */
+void nm_reg_en_attente (void)
+{
+    if (!nm_reg_attente)
+        return;
+    nm_reg_attente = 0;
+    TRACE_EVENT("nm_reg_en_attente: SIM active, enregistrement relance");
+    nm_registration(NM_AUTOMATIC, 0, 0);
+}
+
+static T_ACI_RETURN nm_nrg_robuste (T_ACI_NRG_RGMD rm, char *opr)
+{
+    T_ACI_RETURN r = sAT_PercentNRG(CMD_SRC_LCL, rm, NRG_SVMD_Full, NRG_FRMT_Numeric, opr);
+    if (r == AT_BUSY)
+    {
+        TRACE_EVENT("nm_nrg_robuste: AT_BUSY, abandon COPS/NRG et relance");
+        sAT_Abort(CMD_SRC_LCL, AT_CMD_COPS);
+        sAT_Abort(CMD_SRC_LCL, AT_CMD_NRG);
+        r = sAT_PercentNRG(CMD_SRC_LCL, rm, NRG_SVMD_Full, NRG_FRMT_Numeric, opr);
+    }
+    if (r != AT_EXCT && r != AT_CMPL)
+        TRACE_EVENT_P1("nm_nrg_robuste: sAT_PercentNRG -> %d", r);
+    return r;
+}
+
 /*
  +--------------------------------------------------------------------+
  | PROJECT : MMI-Framework (8417)        MODULE  : MFW_NM             |
@@ -1296,7 +1330,19 @@
       return IMEI_NOT_VALID;

     if (!limited && sim_stat != SIM_ACTIVE)
+    {
+      /* [2026-10-21] banc : relancee par nm_reg_en_attente() quand la SIM devient active */
+      nm_reg_attente = 1;
+      TRACE_EVENT("nm_registration: SIM pas encore active, enregistrement mis en attente");
       return SIM_NOT_ACTIVE;
+    }
+    nm_reg_attente = 0;
+
+#if BANC_REG_AUTO
+    if (mode == NM_MANUAL)
+        TRACE_EVENT("nm_registration: mode manuel memorise -> automatique (BANC_REG_AUTO)");
+    mode = NM_AUTOMATIC;
+#endif
```

```
     if (mode == NM_MANUAL)
@@ -1310,22 +1356,14 @@
                         to the last manual selected HPLMN cell before the power cycle, and this does not happen;
                           instead this, the sample shows up the available networks list
            Fix         :  call to nm_avail_plmn_list() is changed to call to sAT_PercentNRG() */
-       if (sAT_PercentNRG(CMD_SRC_LCL,
-                          NRG_RGMD_Manual,
-                          NRG_SVMD_Full,
-                          NRG_FRMT_Numeric,
-                          NULL) != AT_EXCT)
+        if (nm_nrg_robuste(NRG_RGMD_Manual, NULL) != AT_EXCT)
         {
             TRACE_ERROR("sAT_PercentNRG error");
         }
     }
     else
     {
-       if (sAT_PercentNRG(CMD_SRC_LCL,
-                          NRG_RGMD_Auto,
-                          NRG_SVMD_Full,
-                          NRG_FRMT_Numeric,
-                          NULL) != AT_EXCT)
+        if (nm_nrg_robuste(NRG_RGMD_Auto, NULL) != AT_EXCT)
         {
             TRACE_ERROR("sAT_PercentNRG error");
         }
@@ -1856,12 +1894,9 @@
     T_ACI_RETURN result;
     TRACE_FUNCTION("nm_select_plmn()");

-   result = sAT_PercentNRG(CMD_SRC_LCL,
-                       NRG_RGMD_Manual,
-                       NRG_SVMD_Full,
-                       NRG_FRMT_Numeric,
-                       (char *) network);
-   result = AT_FAIL;
+   /* [2026-10-21] banc : le "result = AT_FAIL;" d'origine qui suivait cet appel faisait
+    * tracer une erreur (et tester un qAT_ErrDesc perime) a chaque choix de reseau au clavier. */
+   result = nm_nrg_robuste(NRG_RGMD_Manual, (char *) network);
     if (result != AT_EXCT)
     {
         TRACE_ERROR("sAT_PercentNRG error");
diff -r 9394305d4ff5 src/ui3/mfw/mfw_sim.c
--- a/src/ui3/mfw/mfw_sim.c Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/mfw/mfw_sim.c Fri Oct 23 19:18:45 2026 +0200
@@ -656,8 +656,21 @@

 */

+/* [2026-10-21] banc osmo-operator : course au boot. nm_registration() appelee avant que la SIM
+ * soit active rendait SIM_NOT_ACTIVE sans rien lancer, et personne ne la rappelait (+CREG 0,0,
+ * pas de RACH, "Network Forbidden"). Chaque changement d etat SIM passe par sim_signal() : on
+ * y relance l enregistrement en attente (mfw_nm.c, nm_reg_en_attente) quand la SIM devient active. */
+static void sim_signal_mmi (MfwEvt event, void *para);
+EXTERN void nm_reg_en_attente (void);
 void sim_signal (MfwEvt event, void *para)
 {
+    sim_signal_mmi(event, para);
+    if (sim_stat == SIM_ACTIVE)
+        nm_reg_en_attente();
+}
+
+static void sim_signal_mmi (MfwEvt event, void *para)
+{
```

```
  UBYTE temp = dspl_Enable(0);
     TRACE_FUNCTION ("sim_signal()");

diff -r 9394305d4ff5 src/ui3/mfw/mfw_sms.c
--- a/src/ui3/mfw/mfw_sms.c Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/mfw/mfw_sms.c Fri Oct 23 19:18:45 2026 +0200
@@ -241,6 +241,8 @@

 //May 27, 2005    MMI-FIX-29869 x0018858 - Added a Checking variable.
 static int Counter_first = 0;
+/* [banc 2026-10-22] sms_read_msg_info() appele avant l'allocation de msg_info (voir Check_max_sms_allowed) */
+static int liste_avant_alloc = 0;
 #else
 static T_MFW_SMS_MSG msg_info[MAX_MESSAGES]; /* message table */
 #endif
@@ -348,8 +350,30 @@
                {
                    msg_info = (T_MFW_SMS_MSG*)ALLOC_MEMORY(g_max_messages*sizeof(T_MFW_SMS_MSG));

+                   /* [banc 2026-10-22] Le tableau sortait de ALLOC_MEMORY NON initialise. rAT_sms_ready() ->
+                    sms_read_msg_info() passe souvent AVANT cette allocation (premier evenement SMS de l'ecran de
+                    veille) : sa remise a index -1 tombait sur msg_info NULL et le tableau alloue ensuite gardait
+                    des dechets. Index != -1 = entrees fantomes  [P]  dans Messages > Read ; en ouvrir une lancait
+                      sms_read sur un index bidon, sans reponse :  Please wait  infini. */
+                   for (i=0; i<g_max_messages; i++)
+                   {
+                       memset(&msg_info[i], 0, sizeof(T_MFW_SMS_MSG));
+                       msg_info[i].index = -1;
+                       msg_info[i].first_index = -1;
+                       msg_info[i].addr.ton = MFW_TON_UNKNOWN;
+                       msg_info[i].addr.npi = MFW_NPI_UNKNOWN;
+                       msg_info[i].concat_status = MFW_SMS_NO_CONC;
+                       msg_info[i].mem = MFW_MEM_NOTPRESENT;
+                       msg_info[i].first_mem = MFW_MEM_NOTPRESENT;
+                   }
                }
            Counter_first ++;
+           /* la liste lue avant l'allocation est perdue : la relire maintenant */
+           if (liste_avant_alloc)
+           {
+               liste_avant_alloc = 0;
+               sms_read_msg_info();
+           }
        }

 }
@@ -853,6 +877,15 @@
     TRACE_FUNCTION("sms_read_msg_info()");

 #ifdef FF_MMI_SMS_DYNAMIC
+    if (msg_info == NULL)
+    {
+        /* [banc 2026-10-22] trop tot : Check_max_sms_allowed() relira la liste apres l'allocation */
+        liste_avant_alloc = 1;
+        return;
+    }
+#endif
+
+#ifdef FF_MMI_SMS_DYNAMIC
     for (i=0; i<g_max_messages; i++)
 #else
     for (i=0; i<MAX_MESSAGES; i++)
```

2.141 /opt/GSM/osmo-operator/patches/gapk-pq-alsa-output-probe.patch

8936 octets, 221 lignes → 221 lignes

```
diff --git a/src/pq_alsa.c b/src/pq_alsa.c
index 50479e6..05efbda 100644
--- a/src/pq_alsa.c
+++ b/src/pq_alsa.c
@@ -20,6 +20,7 @@

 #include <errno.h>
 #include <stdint.h>
+#include <stdlib.h>    /* getenv() — vanne GAPK_ALSA_PROBE */
 #include <talloc.h>

 #include <osmocom/gapk/logging.h>
@@ -51,14 +52,96 @@ pq_cb_alsa_input(void *_state, uint8_t *out, const uint8_t *in, unsigned int in_
     return rv * sizeof(uint16_t);
 }

+/* SONDE GAPK_ALSA_PROBE — le PCM qui part vers ALSA est-il muet ?
+ *
+ * Pourquoi elle existe (mesures du 2026-08-09, Calypso emule sous QEMU) : le
+ * mobile recoit a sa porte 300/300 trames FR DISTINCTES, cadrage 49 o et
+ * signature 0xd correctes (releve dans osmocon.log, hdlc_recv dlci=5), la meme
+ * libosmogapk decode ces memes trames en audio reel quand on l'appelle par le
+ * binaire osmo-gapk autonome (RMS 391, 98,2 % d'echantillons non nuls), et
+ * pourtant la sortie ALSA du mobile est du ZERO NUMERIQUE EXACT (crete 0 sur
+ * 48000 echantillons). Aucune erreur n'est journalisee, car
+ * osmo_gapk_pq_execute ne se plaint que d'un retour negatif.
+ *
+ * Cette sonde tranche en un appel : le PCM remis a snd_pcm_writei est-il nul
+ * (le defaut est en amont, dans la chaine) ou non nul (le defaut est en aval,
+ * dans ALSA/PulseAudio) ?
+ *
+ * Compteurs CUMULATIFS, jamais un taux : un taux tire d'un journal bufferise a
+ * deja produit deux chiffres faux dans cette enquete. La sonde s'annonce au
+ * premier passage dans les deux cas — une sonde muette rend le silence
+ * indecidable. Diagnostic pur : aucun effet sur le flux quand elle est eteinte.
+ */
+static void pq_alsa_out_probe(void *_state, const uint8_t *in, unsigned int in_len,
+                              int rv, int rv_avant_reprise)
+{
+   struct pq_state_alsa *state = _state;
+   static int on = -1;
+   static unsigned long long n, muettes, ecrites_court, epipe, autres_err;
+   static unsigned long long pic_max;
+   unsigned int i;
+   unsigned long long pic = 0;
+   snd_pcm_state_t st;
+   snd_pcm_sframes_t dispo, retard = 0;
+
+   if (on < 0) {
+       const char *e = getenv("GAPK_ALSA_PROBE");
+       on = (e && *e == '1') ? 1 : 0;
+       LOGPGAPK(LOGL_NOTICE, "SONDE GAPK_ALSA_PROBE %s\n",
+            on ? "ACTIVE" : "inactive");
+   }
+   if (!on)
+       return;
+
+   for (i = 0; i + 1 < in_len; i += 2) {
+       int16_t s = (int16_t)(in[i] | (in[i + 1] << 8));
+       unsigned long long a = (s < 0) ? (unsigned long long)-(long long)s : (unsigned long long)s;
+       if (a > pic)
+           pic = a;
```

```
+   }
+   n++;
+   if (pic == 0)
+       muettes++;
+   if (pic > pic_max)
+       pic_max = pic;
+   if (rv >= 0 && (unsigned int)rv != in_len / 2)
+       ecrites_court++;
+   /* L'ECHEC EST RECUPERE EN SILENCE par pq_cb_alsa_output : sur -EPIPE il
+    * fait snd_pcm_prepare() puis reecrit, et ne renvoie que le rv d'apres.
+    * Un flux qui sous-alimente en permanence boucle donc XRUN -> prepare ->
+    * write sans que rien ne le dise, et PulseAudio n'a jamais rien en file
+    * (symptome observe : « Buffer Latency: 0 usec » constant). C'est ce que
+    * ce compteur rend visible. */
+   if (rv_avant_reprise == -EPIPE)
+       epipe++;
+   else if (rv_avant_reprise < 0)
+       autres_err++;
+
+   st = snd_pcm_state(state->pcm_handle);
+   dispo = snd_pcm_avail_update(state->pcm_handle);
+   if (snd_pcm_delay(state->pcm_handle, &retard) < 0)
+       retard = -1;
+
+   if (n <= 3 || (n % 250) == 0)
+       LOGPGAPK(LOGL_NOTICE, "GAPK-ALSA-PROBE : %llu blocs, in_len=%u "
+            "(%u ech), rendus par ALSA=%d -- blocs MUETS : %llu, "
+            "ecritures courtes : %llu, XRUN recuperes : %llu, "
+            "autres erreurs : %llu, crete max vue : %llu, "
+            "etat=%s avail=%ld delay=%ld\n",
+            n, in_len, in_len / 2, rv, muettes, ecrites_court,
+            epipe, autres_err, pic_max,
+            snd_pcm_state_name(st), (long)dispo, (long)retard);
+}
+
 static int
 pq_cb_alsa_output(void *_state, uint8_t *out, const uint8_t *in, unsigned int in_len)
 {
    struct pq_state_alsa *state = _state;
    unsigned int num_samples = in_len/2;
-   int rv;
+   int rv, rv_avant_reprise;

    rv = snd_pcm_writei(state->pcm_handle, in, num_samples);
+   rv_avant_reprise = rv;
    if (rv == -EPIPE) {
        /* Recover from buffer underrun */
        snd_pcm_prepare(state->pcm_handle);
@@ -66,6 +149,8 @@ pq_cb_alsa_output(void *_state, uint8_t *out, const uint8_t *in, unsigned int in
        rv = snd_pcm_writei(state->pcm_handle, in, num_samples);
    }

+   pq_alsa_out_probe(state, in, in_len, rv, rv_avant_reprise);
+
    return rv == num_samples ? 0 : -1;
 }

@@ -129,12 +214,101 @@ pq_queue_alsa_op(struct osmo_gapk_pq *pq, const char *alsa_dev, unsigned int blk
    if (rc < 0)
        goto out_free_par;

+   /* DIMENSIONNEMENT DE LA PERIODE ET DU TAMPON — sans ça, l'ecouteur est MUET.
+    *
+    * Ces deux appels manquaient. snd_pcm_hw_params() prenait donc le maximum
+    * offert par le greffon ALSA-pulse, mesure dans le conteneur :
+    *   HW  period_size=2048  buffer_size=2097152  periods=1024   (a 8 kHz :
```

```
+     *       une periode de 256 ms, un tampon de 262 SECONDES)
+     *   SW  start_threshold=1  avail_min=2048
+     * Or la voix arrive par blocs de 160 trames = 20 ms. Le flux demarre des le
+     * premier echantillon (start_threshold=1), sous-alimente bien avant que
+     * pulse ne tire une periode complete, ALSA declare XRUN, et la reprise
+     * silencieuse de pq_cb_alsa_output appelle snd_pcm_prepare() — qui sur ce
+     * greffon fait pa_stream_flush() et JETTE ce qui etait en file.
+     *
+     * MESURE IN VIVO (archive 20260809-030334, appel ETABLI de bout en bout :
+     * premier bloc a 02:54:17, instant exact de CALL_DELIVERED -> ACTIVE) :
+     *   24250 blocs, XRUN recuperes : 24249  =  99,996 %
+     *   crete max vue : 15432   -> c'etait bien de la voix
+     *   delay epingle a ~155 trames = un seul bloc jamais consomme
+     * Cote sortie : parec sur gsm_audio.monitor rendait un ZERO EXACT.
+     *
+     * ⚠ La variable discriminante immediate est la CADENCE, pas la taille :
+     * a 20 ms/bloc on mesure 0 XRUN, a 28 ms/bloc on en mesure 99,7 %. Le
+     * defaut du dimensionnement ne CAUSE pas l'XRUN, il ne laisse AUCUNE MARGE
+     * — la moindre gigue d'arrivee d'une trame DL suffit alors a tout jeter.
+     * Une periode d'un bloc et huit periodes de tampon donnent 160 ms de coussin.
+     *
+     * Mesure apres correctif, meme banc : 0,1 % -> 73,0 % d'echantillons non
+     * nuls sur gsm_audio.monitor, crete 20000 = exactement l'amplitude injectee.
+     *
+     * GAPK_ALSA_NO_SIZING=1 retablit l'ancien comportement (aucun dimensionnement)
+     * pour refaire l'A/B sans reconstruire. Ce n'est PAS une bequille : le
+     * correctif est le defaut, la vanne n'existe que pour la mesure. */
+    {
+        const char *no_sz = getenv("GAPK_ALSA_NO_SIZING");
+        if (!(no_sz && *no_sz == '1')) {
+            snd_pcm_uframes_t period = blk_len / 2; /* blk_len en octets -> trames */
+            snd_pcm_uframes_t buffer = period * 8;  /* 8 blocs = 160 ms de marge */
+            int dir = 0;
+
+            if (period == 0)
+                period = 160;
+            rc = snd_pcm_hw_params_set_period_size_near(state->pcm_handle,
+                                    hw_params, &period, &dir);
+            if (rc < 0)
+                goto out_free_par;
+            buffer = period * 8;
+            rc = snd_pcm_hw_params_set_buffer_size_near(state->pcm_handle,
+                                    hw_params, &buffer);
+            if (rc < 0)
+                goto out_free_par;
+        }
+    }
+
     rc = snd_pcm_hw_params(state->pcm_handle, hw_params);
     if (rc < 0)
         goto out_free_par;

     snd_pcm_hw_params_free(hw_params);

+    /* SW params : demarrer seulement quand la moitie du tampon est remplie, pour
+     * que le puits ne parte pas en famine des le premier bloc. Sans ce reglage,
+     * start_threshold vaut 1 trame (defaut pose par snd_pcm_hw_params) et le flux
+     * demarre a vide. Non fatal : si ca echoue on continue avec les defauts, mais
+     * on le DIT — une degradation silencieuse rendrait le diagnostic impossible. */
+    {
+        const char *no_sz = getenv("GAPK_ALSA_NO_SIZING");
+        if (!(no_sz && *no_sz == '1')) {
+            snd_pcm_sw_params_t *sw = NULL;
+            snd_pcm_uframes_t period = 0, buffer = 0;
+
+            if (snd_pcm_get_params(state->pcm_handle, &buffer, &period) == 0 &&
```

```
+               snd_pcm_sw_params_malloc(&sw) == 0) {
+               if (snd_pcm_sw_params_current(state->pcm_handle, sw) == 0) {
+                   snd_pcm_sw_params_set_start_threshold(state->pcm_handle,
+                                           sw, buffer / 2);
+                   snd_pcm_sw_params_set_avail_min(state->pcm_handle,
+                                   sw, period);
+                   if (snd_pcm_sw_params(state->pcm_handle, sw) < 0)
+                       LOGPGAPK(LOGL_ERROR, "ALSA '%s': sw_params refuses, "
+                            "on garde les defauts (start_threshold=1)\n",
+                            alsa_dev);
+               }
+               snd_pcm_sw_params_free(sw);
+           }
+           LOGPGAPK(LOGL_NOTICE, "ALSA '%s': periode=%lu trames tampon=%lu trames "
+                "(%lu ms de marge a 8 kHz)\n", alsa_dev,
+                (unsigned long)period, (unsigned long)buffer,
+                (unsigned long)(buffer / 8));
+       } else {
+           LOGPGAPK(LOGL_NOTICE, "ALSA '%s': GAPK_ALSA_NO_SIZING=1 -- "
+                "dimensionnement laisse au greffon (ancien comportement)\n",
+                alsa_dev);
+       }
+   }
+
    item = osmo_gapk_pq_add_item(pq);
    if (!item) {
        rc = -ENOMEM;
```

2.142 /opt/GSM/osmo-operator/patches/grgsm-optional-qtgui-import.patch

729 octets, 16 lignes → 16 lignes

```
--- a/python/__init__.py
+++ b/python/__init__.py
@@ -54,7 +54,11 @@
     #from fcch_burst_tagger import fcch_burst_tagger
     #from sch_detector import sch_detector
     #from fcch_detector import fcch_detector
     from .clock_offset_corrector_tagged import clock_offset_corrector_tagged
-    from .multiarfcns_receiver import multiarfcns_receiver
+    try:
+        # multiarfcns_receiver tire qtgui (GUI Qt) - optionnel, inutile au decodage
+        from .multiarfcns_receiver import multiarfcns_receiver
+    except ImportError:
+        pass
     from .gsm_input import gsm_input
     from .gsm_bcch_ccch_demapper import gsm_bcch_ccch_demapper
     from .gsm_bcch_ccch_sdcch4_demapper import gsm_bcch_ccch_sdcch4_demapper
```

2.143 /opt/GSM/osmo-operator/patches/grgsm-receiver-publish-bsic-fn.patch

1993 octets, 36 lignes → 36 lignes

```
diff --git a/lib/receiver/receiver_impl.cc b/lib/receiver/receiver_impl.cc
index 558e276..e0269d6 100644
--- a/lib/receiver/receiver_impl.cc
+++ b/lib/receiver/receiver_impl.cc
@@ -377,6 +377,31 @@ namespace gr
           send_burst(d_burst_nr, output_binary,
             GSMTAP_BURST_SCH, input_nr, d_c0_burst_start);

+          /* gr-gsm = le DSP : publier le BSIC/FN REELS decodes du SCH sur le
```

```
+           * port `measurements` (le "DSP" poste son resultat de sync, comme un
+           * vrai DSP ecrit dans le NDB). Consomme par tools/grgsm_relay_decode.py ->
+           * shunt -> shunt_dispatch_sb (remplace SHUNT_CANNED_BSIC 63). */
+          {
+            int bsic = ((ncc & 0x7) << 3) | (bcc & 0x7);
+            /* TOA REEL mesure : residu de timing du burst SCH (d_c0_burst_start
+             * vs position nominale GUARD_PERIOD), en qbits @OSR (1 sample OSR =
+             * 1 qbit), recale sur la base firmware 23 ("on time"). gr-gsm aligne
+             * bien -> ~23. Le shunt l'utilise si TOA hors CALYPSO_CANNED. */
+            int toa = 23 + (int)(d_c0_burst_start - (long)floor((GUARD_PERIOD) * d_OSR));
+            pmt::pmt_t msg = pmt::make_tuple(pmt::mp("sch"),
+              pmt::from_long(bsic),
+              pmt::from_long((long)d_burst_nr.get_frame_nr()),
+              pmt::from_long((long)toa));
+            message_port_pub(pmt::mp("measurements"), msg);
+            /* Et sur stdout : le CLI grgsm_decode avale le port measurements,
+             * donc on imprime aussi le SCH pour que si_bridge.py (qui parse le
+             * stdout de grgsm_decode) le forwarde au shunt (UDP 4731 -> feed_sb).
+             * Prefixe distinctif "SCHBSIC " (ne collisionne pas avec les hex SI). */
+            printf("SCHBSIC %d %u %d\n", bsic,
+                   (unsigned)d_burst_nr.get_frame_nr(), toa);
+            fflush(stdout);
+          }
+
           /**
            * Decoding was successful, now
            * compute offset from burst_start,
```

2.144 /opt/GSM/osmo-operator/patches/magnetite-parole-ul-au-go.patch

1521 octets, 28 lignes → 28 lignes

```
--- a/c54x_exe/src/montant.c
+++ b/c54x_exe/src/montant.c
@@ -356,6 +356,25 @@
         if (g.facch++ < (unsigned long)journal())
             printf("  [montant] FACCH UL fn=%u (au GO)\n", fn);
     }
+    /* [2026-10-22] LA PAROLE MONTANTE AU GO. a_du (NDB_A_DU_1) souffre du meme mal que a_fu : la ROM
+     * pose B_BLUD puis l'efface des la phase B de la trame ; capture_tch_ul (montant_scruter, fin de
+     * trame) arrivait trop tard, prendre_ul() rendait false -> publier_parole() jamais appelee, SHM parole
+     * absent, pont ul=0 (aucune voix montante). On la capte ici au GO, symetrique a la FACCH ci-dessus.
+     * L'anti-doublon de prendre_ul() (dedup par contenu, vu[]) empeche le double envoi quand
+     * capture_tch_ul la retente en fin de trame. MONTANT_DU_AU_GO=0 coupe. */
+    static int du = -1;
+    if (du < 0) { const char *e = calypso_getenv("MONTANT_DU_AU_GO"); du = !(e && *e == '0'); }
+    if (du) {
+        uint8_t fr[FR_BYTES];
+        if (prendre_ul(api_ram, NDB_A_DU_1, fr, FR_BYTES)) {
+            noter_ul(fn, 1, TCHT_DSP_TASK, fr, FR_BYTES);   /* brute, format TI (a_du) */
+            parole_ti_vers_fr(fr);
+            noter_ul(fn, 0, TCHT_DSP_TASK, fr, FR_BYTES);   /* convertie FR, ce que recoit le pont */
+            publier_parole(fr, fn);
+            if (g.parole++ < (unsigned long)journal())
+                printf("  [montant] parole UL fn=%u (au GO)\n", fn);
+        }
+    }
 }

 static bool capture_tch_ul(uint16_t *api_ram, uint16_t task_u, uint32_t fn)
```

2.145 /opt/GSM/osmo-operator/patches/magnetite-sms-msginfo-doublefree.patch

5219 octets, 128 lignes → 128 lignes

```
diff -r 9394305d4ff5 src/ui3/bmi/mmiSmsRead.c
--- a/src/ui3/bmi/mmiSmsRead.c  Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/bmi/mmiSmsRead.c  Thu Oct 22 16:08:11 2026 +0200
@@ -777,6 +777,11 @@
            {
                for (i = 0; i < ((data->sms_list_data).nb_sms_stored); i++)
                    {
+#ifdef FF_MMI_SMS_DYNAMIC
+                   /* [banc 2026-10-22] deja libere par E_EXIT, ou jamais alloue (liste vide) */
+                   if (data->sms_list_data.sms_header == NULL || data->sms_list_data.sms_header[i] == NULL)
+                       continue;
+#endif
                    FREE_MEMORY( (void *)data->sms_list_data.sms_header[i], UCS2_MENU_TEXT_LEN+ 1/*SPR1242*/);
                    }
                /*SPR 2686, only de-allocate memory for the number of messages displayed*/
@@ -784,8 +789,11 @@
                FREE_MEMORY ((UBYTE *)data->menu_list_data, sizeof(ListMenuData));
            }
 #ifdef FF_MMI_SMS_DYNAMIC
-           FREE_MEMORY((void*)data->sms_list_data.sms_messages,g_max_messages * sizeof(T_MFW_SMS_MSG) );
-           FREE_MEMORY((void*)data->sms_list_data.sms_header, g_max_messages * sizeof(wstring_t));
+           /* [banc 2026-10-22] gardes : E_EXIT a pu les liberer deja (remis a NULL) */
+           if (data->sms_list_data.sms_messages != NULL)
+               FREE_MEMORY((void*)data->sms_list_data.sms_messages,g_max_messages * sizeof(T_MFW_SMS_MSG) );
+           if (data->sms_list_data.sms_header != NULL)
+               FREE_MEMORY((void*)data->sms_list_data.sms_header, g_max_messages * sizeof(wstring_t));
 #endif
            FREE_MEMORY ((void *)data, sizeof (T_SMSREAD_R));
            sms_read_win = 0;
@@ -1012,18 +1020,32 @@
        l_parent = data->parent;
        SmsRead_R_destroy(win);
        SEND_EVENT(l_parent, E_RETURN, data->id, NULL);
-
+       /* [banc 2026-10-22] break ajoute : E_EXIT relisait data/list_data deja liberes par SmsRead_R_destroy */
+       break;
+
    case E_EXIT: // Terminate exit from sms send...
    // May 27, 2005    MMI-FIX-29869 x0018858
 #ifdef FF_MMI_SMS_DYNAMIC
      {
            int i;
-       for (i=0;i<list_data->nb_sms_stored; i++)
-           {
-               FREE_MEMORY((U8*)list_data->sms_header[i], UCS2_MENU_TEXT_LEN + 1 );
-           }
-       FREE_MEMORY((U8*)list_data->sms_header, g_max_messages * sizeof(wstring_t) );
-       FREE_MEMORY((U8*)list_data->sms_messages, g_max_messages * sizeof(T_MFW_SMS_MSG));
+       /* [banc 2026-10-22] gardes + NULL : sans eux SmsRead_R_destroy liberait une 2e fois (tas MMI
+          corrompu,  Preceeding Sentinal Bytes have been corrupted ), et sms_header n'existe pas si la
+          liste est vide */
+       if (list_data->sms_header != NULL)
+       {
+           for (i=0;i<list_data->nb_sms_stored; i++)
+               {
+                   if (list_data->sms_header[i] != NULL)
+                       FREE_MEMORY((U8*)list_data->sms_header[i], UCS2_MENU_TEXT_LEN + 1 );
+                   list_data->sms_header[i] = NULL;
+               }
+           FREE_MEMORY((U8*)list_data->sms_header, g_max_messages * sizeof(wstring_t) );
+           list_data->sms_header = NULL;
+       }
```

```
+       if (list_data->sms_messages != NULL)
+           FREE_MEMORY((U8*)list_data->sms_messages, g_max_messages * sizeof(T_MFW_SMS_MSG));
+       list_data->sms_messages = NULL;
+       list_data->nb_sms_stored = 0;
    }
 #endif
        listDisplayListMenu(win, data->menu_list_data, (ListCbFunc_t)SmsRead_R_smslist_cb,0);
diff -r 9394305d4ff5 src/ui3/mfw/mfw_sms.c
--- a/src/ui3/mfw/mfw_sms.c Mon Oct 31 00:14:44 2022 +0000
+++ b/src/ui3/mfw/mfw_sms.c Thu Oct 22 16:08:11 2026 +0200
@@ -241,6 +241,8 @@

 //May 27, 2005    MMI-FIX-29869 x0018858 - Added a Checking variable.
 static int Counter_first = 0;
+/* [banc 2026-10-22] sms_read_msg_info() appele avant l'allocation de msg_info (voir Check_max_sms_allowed) */
+static int liste_avant_alloc = 0;
 #else
 static T_MFW_SMS_MSG msg_info[MAX_MESSAGES]; /* message table */
 #endif
@@ -348,8 +350,30 @@
                 {
                     msg_info = (T_MFW_SMS_MSG*)ALLOC_MEMORY(g_max_messages*sizeof(T_MFW_SMS_MSG));

+                    /* [banc 2026-10-22] Le tableau sortait de ALLOC_MEMORY NON initialise. rAT_sms_ready() ->
+                     sms_read_msg_info() passe souvent AVANT cette allocation (premier evenement SMS de l'ecran de
+                     veille) : sa remise a index -1 tombait sur msg_info NULL et le tableau alloue ensuite gardait
+                     des dechets. Index != -1 = entrees fantomes  [P]  dans Messages > Read ; en ouvrir une lancait
+                       sms_read sur un index bidon, sans reponse :  Please wait  infini. */
+                    for (i=0; i<g_max_messages; i++)
+                    {
+                        memset(&msg_info[i], 0, sizeof(T_MFW_SMS_MSG));
+                        msg_info[i].index = -1;
+                        msg_info[i].first_index = -1;
+                        msg_info[i].addr.ton = MFW_TON_UNKNOWN;
+                        msg_info[i].addr.npi = MFW_NPI_UNKNOWN;
+                        msg_info[i].concat_status = MFW_SMS_NO_CONC;
+                        msg_info[i].mem = MFW_MEM_NOTPRESENT;
+                        msg_info[i].first_mem = MFW_MEM_NOTPRESENT;
+                    }
                 }
             Counter_first ++;
+           /* la liste lue avant l'allocation est perdue : la relire maintenant */
+           if (liste_avant_alloc)
+           {
+               liste_avant_alloc = 0;
+               sms_read_msg_info();
+           }
        }

 }
@@ -853,6 +877,15 @@
     TRACE_FUNCTION("sms_read_msg_info()");

 #ifdef FF_MMI_SMS_DYNAMIC
+    if (msg_info == NULL)
+    {
+        /* [banc 2026-10-22] trop tot : Check_max_sms_allowed() relira la liste apres l'allocation */
+        liste_avant_alloc = 1;
+        return;
+    }
+#endif
+
+#ifdef FF_MMI_SMS_DYNAMIC
     for (i=0; i<g_max_messages; i++)
 #else
     for (i=0; i<MAX_MESSAGES; i++)
```

2.146 /opt/GSM/osmo-operator/patches/osmo-bts-force-rand-toy.patch

3024 octets, 84 lignes → 84 lignes

```
diff --git a/src/common/rsl.c b/src/common/rsl.c
index 7931a176..f8bf39ac 100644
--- a/src/common/rsl.c
+++ b/src/common/rsl.c
@@ -22,6 +22,7 @@
  */

 #include <stdio.h>
+#include <stdlib.h>
 #include <errno.h>
 #include <netdb.h>
 #include <stdbool.h>
@@ -3641,6 +3642,59 @@ static int rsl_reject_unknown_lchan(struct msgb *msg)
  * selecting message
  */

+/* forcer le RAND d'une AUTHENTICATION REQUEST (MM) descendante.
+ *
+ * La valeur vient de la variable d'environnement RAND, lue UNE fois :
+ *   RAND non defini -> on ne touche a rien (comportement normal du reseau) ;
+ *   RAND=0          -> RAND tout a zero sur 16 octets (defi fixe du toy,
+ *                      defi fixe connu) ;
+ *   RAND=<hexa>     -> jusqu'a 16 octets lus en hexa (espaces admis),
+ *                      completes a droite par des zeros.
+ * La PRESENCE de RAND est le seul interrupteur : sans elle le RAND du reseau
+ * passe intact. C'est donc « seulement quand precise, mais force quand precise ».
+ * "0" est traite a part car osmo_hexparse() refuse un nombre impair de quartets. */
+static void deka_force_auth_rand(struct msgb *msg)
+{
+   static int inited;
+   static bool forced;
+   static uint8_t rand_val[16];
+   struct gsm48_hdr *gh;
+   struct gsm48_auth_req *ar;
+
+   if (!inited) {
+       const char *env = getenv("RAND");
+       inited = 1;
+       if (env != NULL) {
+           forced = true;
+           if (env[0] == '0' && env[1] == '\0') {
+               memset(rand_val, 0, sizeof(rand_val));
+           } else if (osmo_hexparse(env, rand_val, sizeof(rand_val)) < 0) {
+               /* osmo_hexparse() a deja remis rand_val a zero ;
+                * on force donc un RAND nul et on le signale. */
+               LOGP(DRSL, LOGL_ERROR, "RAND=\"%s\" illisible "
+                    "(hexa <= 16 octets attendu) -> RAND=0\n", env);
+           }
+           LOGP(DRSL, LOGL_NOTICE, "RAND force a %s dans toute "
+                "AUTH REQ descendante\n",
+                osmo_hexdump_nospc(rand_val, sizeof(rand_val)));
+       }
+   }
+   if (!forced)
+       return;
+
+   if (msgb_l3len(msg) < sizeof(*gh) + sizeof(*ar))
+       return;
+   gh = (struct gsm48_hdr *) msgb_l3(msg);
+   if ((gh->proto_discr & 0x0f) != GSM48_PDISC_MM)
+       return;
+   /* bit 6 (0x40) du type porte le numero de sequence MM : on l'ignore. */
```

```
+   if ((gh->msg_type & 0x3f) != GSM48_MT_MM_AUTH_REQ)
+       return;
+
+   ar = (struct gsm48_auth_req *) gh->data;
+   memcpy(ar->rand, rand_val, sizeof(ar->rand));
+}
+
 static int rsl_rx_rll(struct gsm_bts_trx *trx, struct msgb *msg)
 {
    struct abis_rsl_rll_hdr *rh = msgb_l2(msg);
@@ -3680,6 +3734,11 @@ static int rsl_rx_rll(struct gsm_bts_trx *trx, struct msgb *msg)

    LOGPLCHAN(lchan, DRLL, LOGL_DEBUG, "Rx RLL %s Abis -> LAPDm\n", rsl_msg_name(rh->c.msg_type));

+   /* sur une DATA REQ (L3 descendant vers le mobile), reecrire le
+    * RAND de l'AUTH REQUEST si l'environnement RAND est pose. */
+   if (rh->c.msg_type == RSL_MT_DATA_REQ)
+       deka_force_auth_rand(msg);
+
    /* make copy of RLL header, as the message will be free'd in case of erroneous return */
    rh2 = *rh;
    /* exception: RLL messages are _NOT_ freed as they are now
```

2.147 /opt/GSM/osmo-operator/patches/osmo-hlr-force-rand-toy.patch

761 octets, 25 lignes → 25 lignes

```
diff --git a/src/auc.c b/src/auc.c
index 3ec0893..9fa763c 100644
--- a/src/auc.c
+++ b/src/auc.c
@@ -18,6 +18,7 @@
  */

 #include <string.h>
+#include <stdlib.h>
 #include <inttypes.h>

 #include <osmocom/core/utils.h>
@@ -110,6 +111,12 @@ int auc_compute_vectors(struct osmo_auth_vector *vec, unsigned int num_vec,
                 "bytes: rc=%d\n", sizeof(rand), rc);
            goto out;
        }
+       /* gate RAND (getenv) - force le defi du vecteur a 0
+        * (COMP128v1 toy) pour que SRES/Kc soient calcules sur RAND=0
+        * cote HLR ; sinon osmo-msc envoie 0 mais le SRES attendu est
+        * calcule sur un autre RAND -> mismatch -> LU reject. */
+       if (getenv("RAND"))
+           memset(rand, 0, sizeof(rand));
        DBGP("vector [%u]: rand = %s\n", i, hexb(rand));

        if (aud3g) {
```

2.148 /opt/GSM/osmo-operator/patches/osmo-msc-force-rand-toy.patch

853 octets, 18 lignes → 18 lignes

```
diff --git a/src/libmsc/gsm_04_08.c b/src/libmsc/gsm_04_08.c
index de0a69373..b28bd6746 100644
--- a/src/libmsc/gsm_04_08.c
+++ b/src/libmsc/gsm_04_08.c
@@ -638,6 +638,13 @@ int gsm48_tx_mm_auth_req(struct msc_a *msc_a, uint8_t *rand,
```

```
    struct gsm48_hdr *gh = (struct gsm48_hdr *) msgb_put(msg, sizeof(*gh));
    struct gsm48_auth_req *ar = (struct gsm48_auth_req *) msgb_put(msg, sizeof(*ar));

+   /* gate de forcage du RAND. RAND pose dans l'environnement ->
+    * on force le defi (0 sur 16 octets dans notre cas ; changer
+    * l'initialiseur pour un autre RAND). RAND absent -> comportement normal. */
+   static uint8_t forced_rand[16] = { 0 };
+   if (getenv("RAND"))
+       rand = forced_rand;
+
    DEBUGP(DMM, "Tx AUTH REQ (rand = %s)\n", osmo_hexdump_nospc(rand, 16));
    if (autn)
        DEBUGP(DMM, "   AUTH REQ (autn = %s)\n", osmo_hexdump_nospc(autn, 16));
```

2.149 /opt/GSM/osmo-operator/patches/osmo-trx-ipc-ts-frame-align.patch

1892 octets, 29 lignes → 29 lignes

```
diff --git a/Transceiver52M/device/ipc/IPCDevice.cpp b/Transceiver52M/device/ipc/IPCDevice.cpp
index 1d2b89a..871075a 100644
--- a/Transceiver52M/device/ipc/IPCDevice.cpp
+++ b/Transceiver52M/device/ipc/IPCDevice.cpp
@@ -1073,6 +1073,24 @@ bool IPCDevice::flush_recv(size_t num_pkts)
            read = ipc_shm_read(shm_io_rx_streams[i], (uint16_t *)&tmp.front(), 4096 / 2, &tmps, 3);
    }
    ts_initial = tmps + read;
+   /* FIX LU 2026-06-05 : aligner ts_initial sur une frontiere de TRAME TDMA
+    * (8 slots * 625 samples @ 4 SPS = 5000). Le device IPC commit des buffers a
+    * des timestamps multiples de 2500 (CALYPSO_SHM_BUFSIZE), donc ts_initial%5000
+    * valait 0 OU 2500 -> quand 2500, device-TN0 mappe sur osmo-TN4 et la RACH
+    * (placee a l'offset 0 = device-TN0) ratait le slot TS0/RACH -> NOPE/-110.
+    * Arrondir a la trame met les deux grilles en phase. (5000 = 8*625 @ rx-sps 4) */
+   ts_initial -= ts_initial % 5000;
+   /* FIX SDCCH RUN-INVARIANCE 2026-06-09 : aligner ts_initial sur une frontiere de
+    * MULTITRAME 102 (couvre CCCH+SDCCH/4) en plus de la trame. mClock demarre a FN 0
+    * (CALYPSO_START_FN=0) MAIS l'echantillon a ts_initial est etiquete FN 0 -> Q =
+    * osmo_trx_fn - internal_fn = -(ts_initial/5000). Le drain SHM au handshake
+    * OPEN->START est une course de demarrage -> ts_initial/5000 (donc Q mod 102)
+    * variait run-a-run -> le bloc SDCCH/4 SS0 UL tombait a un %51 different chaque run
+    * -> SABM decodee seulement parfois. En forcant ts_initial%(102*5000)==0 -> Q==0
+    * mod 102 a chaque run -> osmo_trx_fn==internal_fn (mod 102) -> gate fixe cote
+    * qemu_wrap (CALYPSO_UL_SDCCH_OFS=15) run-invariant. 102*5000=510000 ; gHyperframe
+    * en echantillons est multiple exact de 510000 -> wrap d'hypertrame non perturbe. */
+   ts_initial -= ts_initial % (102 * 5000);

    LOGC(DDEV, INFO) << "Initial timestamp " << ts_initial << std::endl;
    return true;
```

2.150 /opt/GSM/osmo-operator/patches/osmo-trx-rach-per-ra-table.patch

10975 octets, 231 lignes → 231 lignes

```
diff --git a/Transceiver52M/Makefile.am b/Transceiver52M/Makefile.am
index 0b63b16..fa260f5 100644
--- a/Transceiver52M/Makefile.am
+++ b/Transceiver52M/Makefile.am
@@ -73,7 +73,8 @@ COMMON_LDADD = \
    $(FFTWF_LIBS) \
    $(LIBOSMOCORE_LIBS) \
    $(LIBOSMOCTRL_LIBS) \
-   $(LIBOSMOVTY_LIBS)
+   $(LIBOSMOVTY_LIBS) \
```

```
+   $(LIBOSMOCODING_LIBS)

 if ENABLE_MS_TRX
 AM_CPPFLAGS += -I$(top_srcdir)/osmocom-bb/src/host/trxcon/include/
diff --git a/Transceiver52M/Transceiver.cpp b/Transceiver52M/Transceiver.cpp
index f5ddd9b..9c6b76a 100644
--- a/Transceiver52M/Transceiver.cpp
+++ b/Transceiver52M/Transceiver.cpp
@@ -24,6 +24,7 @@
 */

 #include <stdio.h>
+#include <stdlib.h>     // getenv/atoi (debug SDCCH UL Calypso)
 #include <netinet/in.h>
 #include <iomanip>      // std::setprecision
 #include <fstream>
@@ -258,8 +259,13 @@ bool Transceiver::init()
     mStates[i].init(filler, cfg->tx_sps, txFullScale, cfg->rtsc, cfg->rach_delay);
   }

-  /* Randomize the central clock */
-  GSM::Time startTime(random() % gHyperframe, 0);
+  /* Horloge centrale : random par defaut. CALYPSO_START_FN=N force un FN de
+   * depart DETERMINISTE -> tue la run-variance tn=0/tn=5 de l'injection UL Calypso
+   * (mClock demarrait a un FN aleatoire -> l'etiquette TN glissait). */
+  unsigned _start_fn;
+  { const char *e = getenv("CALYPSO_START_FN");
+    _start_fn = e ? (unsigned)(atol(e) % gHyperframe) : (unsigned)(random() % gHyperframe); }
+  GSM::Time startTime(_start_fn, 0);
   mRadioInterface->getClock()->set(startTime);
   mTransmitDeadlineClock = startTime;
   mLastClockUpdateTime = startTime;
@@ -741,6 +747,13 @@ int Transceiver::pullRadioVector(size_t chan, struct trx_ul_burst_ind *bi)
   burst = radio_burst->getVector(max_i);
   avg = sqrt(avg / radio_burst->chans());

+  /* ENERGY-DBG 2026-06-05 : logge TOUT slot avec de l'energie (un burst presente),
+   * quel que soit le type/TN, pour trouver OU la RACH injectee atterrit vraiment
+   * dans la grille osmo-trx (avant le filtre type==IDLE). Pas de cap : rare. */
+  if (avg > 800.0)
+    LOGCHAN(chan, DTRXDUL, NOTICE) << "ENERGY fn=" << burstTime.FN()
+      << " tn=" << burstTime.TN() << " type=" << type << " avg=" << avg << " max=" << max;
+
   if (type == IDLE) {
     /* Update noise levels */
     state->mNoises.insert(avg);
@@ -756,6 +769,14 @@ int Transceiver::pullRadioVector(size_t chan, struct trx_ul_burst_ind *bi)

   max_toa = (type == RACH || type == EXT_RACH) ?
             mMaxExpectedDelayAB : mMaxExpectedDelayNB;
+  /* DEBUG SDCCH UL (Calypso) : osmo-bts n'envoie pas SETMAXDLYNB -> mMaxExpectedDelayNB=0
+   * -> fenetre TOA du burst normal trop etroite pour le burst injecte par qemu_wrap.
+   * CALYPSO_NB_MAXDLY=N (symboles) elargit la fenetre TOA late du burst normal. */
+  if (type != RACH && type != EXT_RACH) {
+    static int nb_maxdly = -1;
+    if (nb_maxdly < 0) { const char *e = getenv("CALYPSO_NB_MAXDLY"); nb_maxdly = e ? atoi(e) : 0; }
+    if (nb_maxdly > 0) max_toa = nb_maxdly;
+  }

   if (cfg->use_va) {
     // shifted burst copy to make the old demod and detection happy
@@ -766,6 +787,15 @@ int Transceiver::pullRadioVector(size_t chan, struct trx_ul_burst_ind *bi)

   /* Detect normal or RACH bursts */
   rc = detectAnyBurst(*shvec_ptr, mTSC, BURST_THRESH, cfg->rx_sps, type, max_toa, &ebp);
+  if ((type == RACH || type == EXT_RACH) && avg > 800.0)
```

```
+    LOGCHAN(chan, DTRXDUL, NOTICE) << "RACH-DET fn=" << burstTime.FN()
+      << " rc=" << rc << " toa=" << ebp.toa << " tsc=" << ebp.tsc;
+  /* DEBUG SDCCH UL (Calypso) : sonde symetrique a RACH-DET pour le burst NORMAL,
+   * limitee a TN0 (le slot CCCH+SDCCH4) pour ne pas flooder. rc>0 = NB detecte,
+   * toa = decalage mesure -> dit si c'est detection (offset) ou demod. */
+  if ((type != RACH && type != EXT_RACH) && avg > 800.0 && burstTime.TN() == 0)
+    LOGCHAN(chan, DTRXDUL, NOTICE) << "NB-DET fn=" << burstTime.FN()
+      << " rc=" << rc << " toa=" << ebp.toa << " tsc=" << ebp.tsc << " avg=" << avg;
   if (rc <= 0) {
     if (rc == -SIGERR_CLIP) {
       LOGCHAN(chan, DTRXDUL, INFO) << "Clipping detected on received RACH or Normal Burst";
@@ -790,6 +820,33 @@ int Transceiver::pullRadioVector(size_t chan, struct trx_ul_burst_ind *bi)
   bi->tsc = ebp.tsc;
   bi->ci = ebp.ci;

+  /* SONDE SDCCH (Calypso) : |soft| moyen des bits DATA (3..60, 87..144) vs
+   * MIDAMBLE (61..86) sur les bursts NORMAUX tn=0. Discrimine la cause du BER
+   * 456/456 : data_re~0 + data_im>>data_re => bits data sur l'axe imaginaire
+   * (rotation pi/2 fausse, jetee par .real()) ; data_re~mid_re => data OK ailleurs ;
+   * data_re~0 & data_im~0 => pas de signal data. */
+  if ((type != RACH && type != EXT_RACH) && avg > 800.0 && burstTime.TN() == 0 && rxBurst) {
+    double dabs = 0, mabs = 0; int nd = 0, nm = 0;
+    int Nsb = (int)rxBurst->size();
+    for (int i = 0; i < Nsb && i < 148; i++) {
+      float v = rxBurst->begin()[i];   /* SoftVector = float (re deja extrait) */
+      if (i >= 61 && i <= 86) { mabs += fabsf(v); nm++; }
+      else if (i >= 3 && i <= 144) { dabs += fabsf(v); nd++; }
+    }
+    /* bits durs démodulés aux positions TSC (61..86) : référence CONNUE = TSC7
+     * 11101111000100101110111100. Match=correct ; complément=inversion ;
+     * décalé=alignement ; garbage=démod faux. Idem 1ers bits data (3..12). */
+    char tscb[32]; int tj = 0;
+    for (int i = 61; i <= 86 && tj < 31; i++) tscb[tj++] = (rxBurst->begin()[i] > 0) ? '1' : '0';
+    tscb[tj] = 0;
+    char datb[16]; int dj = 0;
+    for (int i = 3; i <= 12 && dj < 15; i++) datb[dj++] = (rxBurst->begin()[i] > 0) ? '1' : '0';
+    datb[dj] = 0;
+    LOGCHAN(chan, DTRXDUL, NOTICE) << "SOFT-DBG fn=" << burstTime.FN()
+      << " data_abs=" << (nd ? dabs / nd : 0.0) << " mid_abs=" << (nm ? mabs / nm : 0.0)
+      << " tsc=" << tscb << " dat0=" << datb;
+  }
+
   /* EDGE demodulator returns 444 (gSlotLen * 3) bits */
   if (rxBurst->size() == EDGE_BURST_NBITS) {
     bi->modulation = MODULATION_8PSK;
diff --git a/Transceiver52M/sigProcLib.cpp b/Transceiver52M/sigProcLib.cpp
index 5fac365..b34851a 100644
--- a/Transceiver52M/sigProcLib.cpp
+++ b/Transceiver52M/sigProcLib.cpp
@@ -35,6 +35,8 @@

 extern "C" {
 #include <osmocom/core/panic.h>
+#include <osmocom/core/bits.h>
+#include <osmocom/coding/gsm0503_coding.h>
 #include "convolve.h"
 #include "scale.h"
 #include "mult.h"
@@ -2148,6 +2150,96 @@ bool sigProcLibSetup()
   generateRACHSequence(&gRACHSequences[1], gRACHSynchSequenceTS1, 1);
   generateRACHSequence(&gRACHSequences[2], gRACHSynchSequenceTS2, 1);

+  /* DUMP RACH ref 2026-06-05 (OUTIL DE PREUVE, PAS LE FINAL) : access-burst
+   * [8 tail][41 sync TS0][36 data][3 tail] module par le modulateur EXACT d'osmo-trx
+   * (Laurent, sps=4) -> /root/rach_ref.cs16, rejoue par le device (sa modulation maison
+   * ne correle pas). data : ZERO si CALYPSO_UL_RA non defini (PORTE 1 = rc>0, la sync
```

```
+   * seule suffit) ; sinon gsm0503_rach_ext_encode(RA,BSIC) (PORTE 2 = CHAN RQD). RA/BSIC
+   * viennent de l'ENV (config, PAS de hardcode). Le FINAL = device encode+modulate le
+   * RA REEL (plombe du BSP) + BSIC reel, dynamiquement. */
+  {
+    static const char SYNC[] = "0100101101111111100110011010101000111100 0";
+    char bs[89];
+    for (int i = 0; i < 88; i++) bs[i] = '0';
+    for (int i = 0; i < 41; i++) bs[8 + i] = SYNC[i];
+    const char *era = getenv("CALYPSO_UL_RA");
+    if (era && *era) {
+      const char *ebs = getenv("CALYPSO_UL_BSIC");
+      uint16_t ra = (uint16_t) strtol(era, NULL, 0);
+      uint8_t bsic = (uint8_t)(ebs && *ebs ? atoi(ebs) : 0);
+      ubit_t coded[40]; memset(coded, 0, sizeof(coded));
+      gsm0503_rach_ext_encode(coded, ra, bsic, false);   /* 36 bits codes (RA + parite BSIC) */
+      for (int i = 0; i < 36; i++) bs[49 + i] = coded[i] ? '1' : '0';
+      LOG(NOTICE) << "[rach-ref] RA=" << (int) ra << " BSIC=" << (int) bsic << " (PORTE 2, env)";
+    } else {
+      LOG(NOTICE) << "[rach-ref] sync seule, data=0 (PORTE 1 ; set CALYPSO_UL_RA pour PORTE 2)";
+    }
+    bs[88] = 0;
+    BitVector ab(bs);
+    signalVector *rb = modulateBurst(ab, 68, 4, false);
+    if (rb) {
+      FILE *f = fopen("/root/rach_ref.cs16", "wb");
+      if (f) {
+        float *p = (float *) rb->begin();
+        for (size_t i = 0; i < rb->size(); i++) {
+          float I = p[2 * i] * 20000.0f, Q = p[2 * i + 1] * 20000.0f;
+          if (I > 32767.0f) I = 32767.0f; else if (I < -32768.0f) I = -32768.0f;
+          if (Q > 32767.0f) Q = 32767.0f; else if (Q < -32768.0f) Q = -32768.0f;
+          int16_t s[2] = { (int16_t) I, (int16_t) Q };
+          fwrite(s, sizeof(int16_t), 2, f);
+        }
+        fclose(f);
+        LOG(NOTICE) << "[rach-ref] dumped " << rb->size() << " IQ -> /root/rach_ref.cs16";
+      }
+      delete rb;
+    }
+  }
+
+  /* TABLE per-RA 2026-06-05 (NO-HARDCODE) : la vraie RA du mobile (d_rach@0x0474,
+   * plombee via /dev/shm/calypso_rach) varie a chaque burst. On pre-genere ICI la
+   * modulation EXACTE d'osmo-trx (Laurent sps=4, la seule qui correle son propre
+   * detecteur) pour chaque RA 0x00..0x0f a BSIC reel -> /root/rach_ref_RA<nn>.cs16.
+   * Le device (ul_drain) selectionne le ref de la vraie RA recue. BSIC = celui du
+   * BTS (env CALYPSO_UL_BSIC, defaut 7 ; colore la parite). Pas de RA hardcodee :
+   * c'est un codebook, la vraie RA en selectionne l'entree. */
+  {
+    static const char SYNC[] = "0100101101111111100110011010101000111100 0";
+    const char *ebs = getenv("CALYPSO_UL_BSIC");
+    uint8_t bsic = (uint8_t)(ebs && *ebs ? atoi(ebs) : 7);
+    int ntab = 0;
+    for (int ra = 0; ra < 16; ra++) {
+      char bs[89];
+      for (int i = 0; i < 88; i++) bs[i] = '0';
+      for (int i = 0; i < 41; i++) bs[8 + i] = SYNC[i];
+      ubit_t coded[40]; memset(coded, 0, sizeof(coded));
+      gsm0503_rach_ext_encode(coded, (uint16_t) ra, bsic, false);
+      for (int i = 0; i < 36; i++) bs[49 + i] = coded[i] ? '1' : '0';
+      bs[88] = 0;
+      BitVector ab(bs);
+      signalVector *rb = modulateBurst(ab, 68, 4, false);
+      if (!rb) continue;
+      char path[64];
+      snprintf(path, sizeof(path), "/root/rach_ref_RA%02x.cs16", ra);
```

```
+      FILE *f = fopen(path, "wb");
+      if (f) {
+        float *p = (float *) rb->begin();
+        for (size_t i = 0; i < rb->size(); i++) {
+          float I = p[2 * i] * 20000.0f, Q = p[2 * i + 1] * 20000.0f;
+          if (I > 32767.0f) I = 32767.0f; else if (I < -32768.0f) I = -32768.0f;
+          if (Q > 32767.0f) Q = 32767.0f; else if (Q < -32768.0f) Q = -32768.0f;
+          int16_t s[2] = { (int16_t) I, (int16_t) Q };
+          fwrite(s, sizeof(int16_t), 2, f);
+        }
+        fclose(f);
+        ntab++;
+      }
+      delete rb;
+    }
+    LOG(NOTICE) << "[rach-ref] table per-RA : " << ntab
+                << " refs (RA 0x00..0x0f, BSIC=" << (int) bsic
+                << ") -> /root/rach_ref_RA*.cs16";
+  }
+
   generateSCHSequence(1);
   generateDummyMidamble(1);
```

2.151 /opt/GSM/osmo-operator/patches/osmocom-bb-clck-gen-frame-tolerance.patch

1078 octets, 23 lignes → 23 lignes

```
diff --git a/src/target/trx_toolkit/clck_gen.py b/src/target/trx_toolkit/clck_gen.py
index f769c3e9..8c253ea2 100755
--- a/src/target/trx_toolkit/clck_gen.py
+++ b/src/target/trx_toolkit/clck_gen.py
@@ -104,8 +104,16 @@ class CLCKGen:
            t = time.monotonic_ns()
            dt = (t_next - t)
            if dt < 0:
-               log.warning("CLCKGen: time overrun by %dus; resetting the clock" % (dt * ns // us))
-               t_next = time.monotonic_ns()
+               # [osmo-operator 2026-09-08] Only rebase the clock when we are
+               # late by more than one full TDMA frame. A sub-frame overrun
+               # (0.3..2 ms on a loaded host) used to reset t_next as well,
+               # shifting the timebase by that much every time: osmo-bts-trx
+               # then logged "N FN slower/faster than TRX, compensating" all
+               # day long and the speech path lost frames (audible dropouts
+               # at TCH start). Below one frame we simply catch up.
+               if dt < -t_tick:
+                   log.warning("CLCKGen: time overrun by %dus; resetting the clock" % (dt * ns // us))
+                   t_next = time.monotonic_ns()
                dt = 0

            if self._breaker.wait(dt * ns):
```

2.152 /opt/GSM/osmo-operator/patches/osmocom-bb-fbsb-timeout-scan-to.patch

5064 octets, 109 lignes → 109 lignes

```
osmocom-bb : rendre reglables les deux budgets d'une tentative de synchro.

Sous emulation QEMU-Calypso, l'interpreteur c54x fait tourner le tick TDMA a
~10 Hz au lieu de 217 (facteur ~96). Les deux budgets qu'osmocom fixe en dur
sont alors trop courts d'autant, et le mobile boucle :
```

```
  gsm322_sync_to_cell() emet L1CTL_RESET_REQ puis un FBSB_REQ avec un timeout
  litteral de 100 TRAMES (461 ms de temps GSM, correct sur silicium, ~10 s de
  temps reel ici). Le firmware abandonne, renvoie un FBSB_CONF d'echec, la
  couche 3 loggue « Channel sync error » / « Cell selection failed » et rappelle
  gsm322_sync_to_cell() — donc re-emet le RESET. Le DSP emule n'obtient jamais
  une fenetre assez longue pour acquerir FB+SB.

  support.c fixe scan_to = 4 s, sans aucune commande VTY : impossible a regler
  depuis mobile.cfg.

Ce correctif n'introduit AUCUNE valeur fabriquee : ni detection, ni resultat
DSP, ni cellule d'API. Il empeche seulement le banc d'interrompre son propre
sujet. Si le DSP ne trouve rien, il ne trouvera toujours rien — sans etre coupe.

  CALYPSO_FBSB_TIMEOUT=<trames>    defaut 100   (plafond 65535, champ uint16)
  CALYPSO_SCAN_TO=<secondes>       defaut 4     (plafond 255, champ uint8)

Les deux doivent etre accordes : SCAN_TO doit depasser la duree MUR de
FBSB_TIMEOUT trames. A ~10 Hz, 2000 trames (~200 s) va avec SCAN_TO=240.

Absentes, les deux variables laissent le comportement osmocom d'origine.
diff --git a/src/host/layer23/src/common/support.c b/src/host/layer23/src/common/support.c
index 3e053713..a1c1c8bf 100644
--- a/src/host/layer23/src/common/support.c
+++ b/src/host/layer23/src/common/support.c
@@ -89,7 +89,24 @@ void gsm_support_init(struct osmocom_ms *ms)
     sup->ch_cap = GSM_CAP_SDCCH_TCHF_TCHH;
     sup->min_rxlev_dbm = -106; // TODO
     sup->sync_to = 6; /* how long to wait sync (0.9 s) */
+   /* [2026-08-03] Reglable : sous emulation le DSP est ~96x trop lent, et 4 s
+    * de mur ne couvrent qu'une fraction du temps GSM necessaire. Sans commande
+    * VTY jusqu'ici, d'ou l'impossibilite de le regler depuis mobile.cfg.
+    * CALYPSO_SCAN_TO=<secondes> ; absente = 4 = comportement d'origine.
+    * Champ uint8_t -> plafonne a 255. Instrument de banc, pas une bequille :
+    * aucune valeur radio n'est fabriquee. */
     sup->scan_to = 4; /* how long to wait for all sysinfos (>=4 s) */
+   {
+       const char *e = getenv("CALYPSO_SCAN_TO");
+       if (e && *e) {
+           long v = strtol(e, NULL, 0);
+           if (v < 1) v = 1;
+           if (v > 255) v = 255;
+           sup->scan_to = (uint8_t)v;
+           printf("[support] CALYPSO_SCAN_TO=%u s (defaut 4)\n",
+                  sup->scan_to);
+       }
+   }
     sup->dsc_max = 90; /* the specs defines 90 */

     /* codec */
diff --git a/src/host/layer23/src/mobile/gsm322.c b/src/host/layer23/src/mobile/gsm322.c
index 3d0cc77d..698648c8 100644
--- a/src/host/layer23/src/mobile/gsm322.c
+++ b/src/host/layer23/src/mobile/gsm322.c
@@ -493,10 +493,44 @@ static int gsm322_sync_to_cell(struct gsm322_cellsel *cs,

     cs->sync_pending = 1;
     l1ctl_tx_reset_req(ms, L1CTL_RES_T_FULL);
-    return l1ctl_tx_fbsb_req(ms, cs->arfcn,
-            L1CTL_FBSB_F_FB01SB, 100, 0,
-            cs->ccch_mode,
-            cs->list[cs->arfci].rxlev);
+
+   /* [2026-08-03] BUDGET D'UNE TENTATIVE FBSB, en TRAMES TDMA.
+    *
+    * Etait le litteral 100 : 100 trames = 461 ms de temps GSM, parfaitement
```

```
+    * correct sur silicium. Mais sous emulation le tick TDMA du DSP tourne a
+    * ~10 Hz au lieu de 217 (interpreteur c54x ~96x trop lent), donc ces
+    * 100 trames valent ~10 s de temps REEL. Le firmware abandonne, envoie un
+    * FBSB_CONF d'echec, la couche 3 loggue « Channel sync error » puis
+    * « Cell selection failed », et gsm322_sync_to_cell() est rappelee — ce qui
+    * re-emet le L1CTL_RESET_REQ ci-dessus. C'est CETTE boucle qui empeche le
+    * DSP natif d'avoir une seule fenetre assez longue pour acquerir FB+SB.
+    *
+    * CE N'EST PAS UNE BEQUILLE : on ne fabrique aucun resultat DSP, aucune
+    * detection, aucune valeur d'API. On empeche seulement le banc d'interrompre
+    * son propre sujet toutes les ~8 s. Le juge reste intact : si le DSP ne
+    * trouve rien, il ne trouvera toujours rien, simplement sans etre coupe.
+    *
+    * CALYPSO_FBSB_TIMEOUT=<trames> ; absente = 100 = comportement d'origine.
+    * Champ uint16 cote protocole -> plafonne a 65535. En natif, viser large :
+    * 20000 trames ~= 30 min de mur a 10 Hz, soit ~92 s de temps GSM simule. */
+   {
+       static int fbsb_to = -1;
+       if (fbsb_to < 0) {
+           const char *e = getenv("CALYPSO_FBSB_TIMEOUT");
+           long v = (e && *e) ? strtol(e, NULL, 0) : 100;
+           if (v < 1) v = 1;
+           if (v > 65535) v = 65535;
+           fbsb_to = (int)v;
+           if (fbsb_to != 100)
+               LOGP(DCS, LOGL_NOTICE, "FBSB timeout force a %d trames "
+                    "(defaut 100) — fenetre de synchro allongee pour le "
+                    "DSP emule ; aucun resultat n'est fabrique\n", fbsb_to);
+       }
+       return l1ctl_tx_fbsb_req(ms, cs->arfcn,
+               L1CTL_FBSB_F_FB01SB, (uint16_t)fbsb_to, 0,
+               cs->ccch_mode,
+               cs->list[cs->arfci].rxlev);
+   }
 }

 /* this is called whenever the serving cell is unselectied */
```

2.153 /opt/GSM/osmo-operator/patches/osmocom-bb-firmware-gprs.patch

21152 octets, 649 lignes → 647 lignes (2 groupes compactés)

```
diff --git a/src/target/firmware/Makefile b/src/target/firmware/Makefile
index 64ed667f..0d8e610f 100644
--- a/src/target/firmware/Makefile
+++ b/src/target/firmware/Makefile
@@ -127,6 +127,9 @@ APPLICATIONS?=hello_world compal_dsp_dump layer1 loader rssi menu
 # Applications specific env requirements
 APP_loader_ENVIRONMENTS=compalram highram
 APP_rssi_ENVIRONMENTS=* -compalram
+# layer1 : avec le PDCH (l1gprs_fw.c, prim_pdtch.c) l image ne tient plus dans
+# les 64 Ko que le chargeur Compal accepte (LRAM) : highram via loader.compalram
+APP_layer1_ENVIRONMENTS=* -compalram
 APP_menu_ENVIRONMENTS=* -highram

 # Various objects that are currently linked into all applications
diff --git a/src/target/firmware/include/layer1/prim.h b/src/target/firmware/include/layer1/prim.h
index 4e34bb23..ea2715ba 100644
--- a/src/target/firmware/include/layer1/prim.h
+++ b/src/target/firmware/include/layer1/prim.h
@@ -25,6 +25,7 @@ void l1a_rach_req(uint16_t offset, uint8_t combined, uint8_t ra, uint8_t uic);
 /* Primitives raw scheduling sets */
 extern const struct tdma_sched_item nb_sched_set[];
 extern const struct tdma_sched_item nb_sched_set_ul[];
```

```
+extern const struct tdma_sched_item pdtch_sched_set_ul[];

 extern const struct tdma_sched_item tch_sched_set[];
 extern const struct tdma_sched_item tch_a_sched_set[];
diff --git a/src/target/firmware/layer1/Makefile b/src/target/firmware/layer1/Makefile
index 0a24e054..b1765b44 100644
--- a/src/target/firmware/layer1/Makefile
+++ b/src/target/firmware/layer1/Makefile
@@ -5,5 +5,5 @@ LIB_layer1_SRCS=avg.c agc.c afc.c toa.c sync.c tdma_sched.c tpu_window.c init.c
        l23_api.c mframe_sched.c sched_gsmtime.c async.c rfch.c

 LIB_layer1_SRCS += prim_pm.c prim_rach.c prim_tx_nb.c prim_rx_nb.c prim_fbsb.c \
-          prim_freq.c prim_utils.c prim_tch.c
+          prim_freq.c prim_utils.c prim_tch.c l1gprs_fw.c prim_pdtch.c

diff --git a/src/target/firmware/layer1/l23_api.c b/src/target/firmware/layer1/l23_api.c
index f4309e2a..2519a59c 100644
--- a/src/target/firmware/layer1/l23_api.c
+++ b/src/target/firmware/layer1/l23_api.c
@@ -44,6 +44,7 @@
 #include <calypso/dsp.h>

 #include <l1ctl_proto.h>
+#include <layer1/l1gprs_fw.h>

 /* the size we will allocate struct msgb* for HDLC */
 #define L3_MSG_HEAD 4
@@ -300,6 +301,26 @@ static void l1ctl_rx_dm_est_req(struct msgb *msg)

    /* figure out which MF tasks to enable */
    l1a_mftask_set(chan_nr2mf_task_mask(ul->chan_nr, NEIGH_MODE_PM));
+
+   /* [2026-10-15] PDCH : annoncer l intervalle TOUT DE SUITE. Sur le banc
+    * (QEMU + c54x_exe), le BSP ne joue l intervalle du PDCH qu une fois que le
+    * tap QEMU a vu un L1CTL_GPRS_DL_BLOCK_IND de ce TN. Le premier n arrivait
+    * qu apres le premier bloc PDTCH decode - lui-meme joue depuis TS0, donc
+    * perdu : les premiers blocs apres l assignation (commandes, polls)
+    * manquaient, T3190/T3168 et TBF abandonne. Une indication vide (meme
+    * forme qu un bloc en erreur, cf. l1gprs de trxcon) arme le BSP des
+    * l etablissement du PDCH. */
+   if (l1s.dedicated.type == GSM_DCHAN_PDCH) {
+       struct l1ctl_gprs_dl_block_ind *ind;
+       struct msgb *m = l1ctl_msgb_alloc(L1CTL_GPRS_DL_BLOCK_IND);
+       ind = (struct l1ctl_gprs_dl_block_ind *) msgb_put(m, sizeof(*ind));
+       memset(ind, 0, sizeof(*ind));
+       ind->hdr.fn = htonl(l1s.current_time.fn);
+       ind->hdr.tn = ul->chan_nr & 0x7;
+       ind->usf = 0xff;
+       l1_queue_for_l2(m);
+       printf("l1gprs: PDCH TN=%u etabli, annonce\n", ul->chan_nr & 0x7);
+   }
 }

 /* receive a L1CTL_DM_FREQ_REQ from L23 */
@@ -351,6 +372,7 @@ static void l1ctl_rx_crypto_req(struct msgb *msg)
 /* receive a L1CTL_DM_REL_REQ from L23 */
 static void l1ctl_rx_dm_rel_req(struct msgb *msg)
 {
+   l1gprs_fw_reset();  /* PDCH: drop TBFs and queued uplink blocks */
    printd("L1CTL_DM_REL_REQ\n");
    l1a_mftask_set(0);
    l1s.dedicated.type = GSM_DCHAN_NONE;
@@ -719,6 +741,19 @@ void l1a_l23_handler(void)
    case L1CTL_SIM_REQ:
        l1ctl_sim_req(msg);
        break;
```

```
+   case L1CTL_GPRS_UL_TBF_CFG_REQ:
+       l1gprs_fw_ul_tbf_cfg(msg);
+       break;
+   case L1CTL_GPRS_DL_TBF_CFG_REQ:
+       l1gprs_fw_dl_tbf_cfg(msg);
+       break;
+   case L1CTL_GPRS_UL_BLOCK_REQ:
+       l1gprs_fw_ul_block_req(msg);
+       /* queued for prim_pdtch.c (or freed there on error) */
+       goto exit_nofree;
+   default:
+       printf("l1a_l23_cb: unknown L1CTL msg_type 0x%02x\n", l1h->msg_type);
+       break;
    }

 exit_msgbfree:
diff --git a/src/target/firmware/layer1/mframe_sched.c b/src/target/firmware/layer1/mframe_sched.c
index 478fd20a..8ad23838 100644
--- a/src/target/firmware/layer1/mframe_sched.c
+++ b/src/target/firmware/layer1/mframe_sched.c
@@ -46,6 +46,7 @@ struct mframe_sched_item {
 #define NB_QUAD_FH_DL  NB_QUAD_DL
 #define NB_QUAD_UL nb_sched_set_ul
 #define NB_QUAD_FH_UL  NB_QUAD_UL
+#define PDTCH_QUAD_UL  pdtch_sched_set_ul
 #define NEIGH_PM   neigh_pm_sched_set

 /* BCCH Normal */
@@ -312,7 +313,11 @@ static const struct mframe_sched_item mf_gprs_pdtch[] = {
    { .sched_set = NB_QUAD_DL, .modulo = 13, .frame_nr = ⟨1⟩ },  ×3
    ⟨⟩ = (0) (4) (8)
-   /* NOTE: receive only task */
+   /* Uplink block periods, same frame numbers (3-TS offset in the TPU):
+    * prim_pdtch.c only transmits when L23 queued a block for that Fn. */
+   { .sched_set = PDTCH_QUAD_UL, .modulo = 13, .frame_nr = ⟨1⟩ },  ×3
    ⟨⟩ = (0) (4) (8)
    { .sched_set = NULL }
 };

diff --git a/src/target/firmware/layer1/prim_rx_nb.c b/src/target/firmware/layer1/prim_rx_nb.c
index d6dd82a5..6cee98ce 100644
--- a/src/target/firmware/layer1/prim_rx_nb.c
+++ b/src/target/firmware/layer1/prim_rx_nb.c
@@ -48,6 +48,7 @@
 #include <layer1/agc.h>

 #include <l1ctl_proto.h>
+#include <layer1/l1gprs_fw.h>

 struct l1s_rxnb_state {
    struct l1s_meas_hdr meas[4];
@@ -153,7 +154,16 @@ static int l1s_nb_resp(__unused uint8_t p1, uint8_t burst_id, uint16_t p3)
        /* copy actual data, skipping the information block [0,1,2] */
        dsp_memcpy_from_api(rxnb.di->data, &dsp_api.ndb->a_cd[3], 23, 0);

-       l1_queue_for_l2(rxnb.msg);
+       if (mf_task_id == MF_TASK_GPRS_PDTCH) {
+           /* PDCH: a CS-1 block (same coding as xCCH) goes to the
+            * GPRS state machine, not up as L1CTL_DATA_IND. */
+           l1gprs_fw_dl_block(rx_time.fn, tn, rxnb.di->data,
+                      rxnb.dl->fire_crc < 2,
+                      rxnb.dl->rx_level, num_biterr, rxnb.dl->snr);
+           msgb_free(rxnb.msg);
+       } else {
+           l1_queue_for_l2(rxnb.msg);
+       }
```

```
        rxnb.msg = NULL; rxnb.dl = NULL; rxnb.di = NULL;

        /* clear downlink task */
diff --git a/src/target/firmware/include/layer1/l1gprs_fw.h b/src/target/firmware/include/layer1/l1gprs_fw.h
new file mode 100644
index 00000000..2688e3f5
--- /dev/null
+++ b/src/target/firmware/include/layer1/l1gprs_fw.h
@@ -0,0 +1,33 @@
+/* l1gprs_fw.h - GPRS (PDCH) support in the Calypso layer1 firmware.
+ *
+ * A static-allocation port of src/shared/l1gprs.c (trxcon/virtphy) to the
+ * target: TBF registry, DL block filtering by TFI, RTS generation, and the
+ * uplink block queue consumed by prim_pdtch.c.  CS-1 only: the DSP xCCH
+ * coder/decoder (DDL/DUL tasks) is bit-exact with GPRS CS-1.
+ */
+#pragma once
+
+#include <stdint.h>
+#include <stdbool.h>
+
+struct msgb;
+
+/* L1A side (l23_api.c, main loop) */
+void l1gprs_fw_reset(void);
+int  l1gprs_fw_ul_tbf_cfg(struct msgb *msg);   /* L1CTL_GPRS_UL_TBF_CFG_REQ */
+int  l1gprs_fw_dl_tbf_cfg(struct msgb *msg);   /* L1CTL_GPRS_DL_TBF_CFG_REQ */
+int  l1gprs_fw_ul_block_req(struct msgb *msg); /* L1CTL_GPRS_UL_BLOCK_REQ: takes ownership of msg */
+
+/* L1S side (TDMA interrupt) */
+/* A downlink PDTCH block was decoded (or failed: crc_ok == 0, data ignored). */
+void l1gprs_fw_dl_block(uint32_t fn, uint8_t tn, const uint8_t *data,
+           bool crc_ok, uint8_t rx_lev, uint16_t num_biterr, uint16_t snr);
+/* Uplink block to transmit at exactly (fn, tn), or NULL.  Stale blocks are dropped. */
+struct msgb *l1gprs_fw_ul_dequeue(uint32_t fn, uint8_t tn);
+/* The block for (fn, tn) went out: L1CTL_GPRS_UL_BLOCK_CNF to L23. */
+void l1gprs_fw_ul_block_cnf(uint32_t fn, uint8_t tn, const uint8_t *data);
+
+/* Frame-number advance of the RTS.ind relative to the DL block that carried
+ * the USF (13 = next-but-one block period; 13 maps a block start onto a
+ * block start of the 52-multiframe).  Polls (RRBP) are at N+13 at least. */
+#define L1GPRS_FW_RTS_ADVANCE  13
diff --git a/src/target/firmware/layer1/l1gprs_fw.c b/src/target/firmware/layer1/l1gprs_fw.c
new file mode 100644
index 00000000..301a88f4
--- /dev/null
+++ b/src/target/firmware/layer1/l1gprs_fw.c
@@ -0,0 +1,324 @@
+/* l1gprs_fw.c - GPRS (PDCH) layer-1 state for the Calypso firmware.
+ *
+ * (C) 2026 osmo-operator bench. GPLv2+.
+ * Port of src/shared/l1gprs.c (C) 2022-2024 sysmocom, Vadim Yanitskiy,
+ * to static allocation and to the firmware's L1A/L1S split.
+ */
+#include <stdint.h>
+#include <stdio.h>
+#include <string.h>
+#include <errno.h>
+
+#include <arpa/inet.h>
+#include <asm/system.h>
+
+#include <osmocom/core/msgb.h>
+#include <osmocom/core/linuxlist.h>
+#include <osmocom/gsm/gsm_utils.h>
+
```

```
+#include <layer1/l23_api.h>
+#include <layer1/l1gprs_fw.h>
+#include <l1ctl_proto.h>
+
+#define MAX_TBF        8
+#define TDMA_FN_INVALID    0xffffffff
+#define CS1_BLOCK_LEN  23
+
+struct fw_tbf {
+   bool used;
+   bool uplink;
+   bool pending;       /* waiting for start_fn */
+   uint8_t ref;
+   uint8_t slotmask;
+   uint8_t dl_tfi;
+   uint32_t start_fn;
+};
+
+static struct fw_tbf tbfs[MAX_TBF];
+/* Uplink blocks queued by L23 (L1CTL_GPRS_UL_BLOCK_REQ), fn in cb[0], tn in cb[1] */
+static struct llist_head ul_queue = LLIST_HEAD_INIT(ul_queue);
+
+/* 0 when a == b, >0 when a is after b, <0 when a is before b (hyperframe wrap) */
+static int fn_cmp(uint32_t a, uint32_t b)
+{
+   uint32_t d = (a + GSM_MAX_FN - b) % GSM_MAX_FN;
+   if (d == 0)
+       return 0;
+   return (d < GSM_MAX_FN / 2) ? 1 : -1;
+}
+
+void l1gprs_fw_reset(void)
+{
+   unsigned long flags;
+   struct msgb *msg;
+
+   local_firq_save(flags);
+   memset(tbfs, 0, sizeof(tbfs));
+   while ((msg = msgb_dequeue(&ul_queue)))
+       msgb_free(msg);
+   local_irq_restore(flags);
+}
+
+static struct fw_tbf *find_tbf(bool uplink, uint8_t ref, bool pending)
+{
+   int i;
+   for (i = 0; i < MAX_TBF; i++) {
+       if (!tbfs[i].used || tbfs[i].uplink != uplink)
+           continue;
+       if (tbfs[i].ref != ref || tbfs[i].pending != pending)
+           continue;
+       return &tbfs[i];
+   }
+   return NULL;
+}
+
+static struct fw_tbf *alloc_tbf(void)
+{
+   int i;
+   for (i = 0; i < MAX_TBF; i++)
+       if (!tbfs[i].used)
+           return &tbfs[i];
+   return NULL;
+}
+
+/* Called from both contexts; the caller holds the IRQ lock */
```

```
+static void tbf_cfg(bool uplink, uint8_t ref, uint8_t slotmask, uint8_t dl_tfi, uint32_t start_fn)
+{
+   struct fw_tbf *t;
+
+   if (slotmask == 0x00) {
+       /* release */
+       while ((t = find_tbf(uplink, ref, false)) || (t = find_tbf(uplink, ref, true)))
+           t->used = false;
+       printf("l1gprs: %cL-TBF#%u released\n", uplink ? 'U' : 'D', ref);
+       return;
+   }
+   if (start_fn != TDMA_FN_INVALID) {
+       t = find_tbf(uplink, ref, true);
+       if (!t)
+           t = alloc_tbf();
+       if (!t) {
+           puts("l1gprs: no free TBF slot (pending)\n");
+           return;
+       }
+       *t = (struct fw_tbf) { .used = true, .uplink = uplink, .pending = true, .ref = ref,
+                       .slotmask = slotmask, .dl_tfi = dl_tfi, .start_fn = start_fn };
+       printf("l1gprs: %cL-TBF#%u pending slotmask=0x%02x start_fn=%u\n",
+              uplink ? 'U' : 'D', ref, slotmask, start_fn);
+       return;
+   }
+   t = find_tbf(uplink, ref, false);
+   if (!t)
+       t = alloc_tbf();
+   if (!t) {
+       puts("l1gprs: no free TBF slot\n");
+       return;
+   }
+   *t = (struct fw_tbf) { .used = true, .uplink = uplink, .pending = false, .ref = ref,
+                  .slotmask = slotmask, .dl_tfi = dl_tfi, .start_fn = TDMA_FN_INVALID };
+   printf("l1gprs: %cL-TBF#%u active slotmask=0x%02x tfi=%u\n",
+          uplink ? 'U' : 'D', ref, slotmask, dl_tfi);
+}
+
+/* Promote pending TBFs whose starting time has come (L1S context) */
+static void check_pending(uint32_t fn)
+{
+   int i;
+   for (i = 0; i < MAX_TBF; i++) {
+       struct fw_tbf *t = &tbfs[i], *a;
+       if (!t->used || !t->pending)
+           continue;
+       if (fn_cmp(fn, t->start_fn) < 0)
+           continue;
+       /* merge into an existing active TBF of the same ref, or promote */
+       a = find_tbf(t->uplink, t->ref, false);
+       if (a) {
+           a->slotmask = t->slotmask;
+           a->dl_tfi = t->dl_tfi;
+           t->used = false;
+       } else {
+           t->pending = false;
+       }
+       printf("l1gprs: %cL-TBF#%u becomes active (fn=%u)\n", t->uplink ? 'U' : 'D', t->ref, fn);
+   }
+}
+
+static unsigned int pdch_count(uint8_t tn, bool pending, uint32_t *dl_tfi_mask)
+{
+   unsigned int n = 0;
+   int i;
+   for (i = 0; i < MAX_TBF; i++) {
```

```
+       struct fw_tbf *t = &tbfs[i];
+       if (!t->used || t->pending != pending)
+           continue;
+       if (!(t->slotmask & (1 << tn)))
+           continue;
+       n++;
+       if (dl_tfi_mask && !t->uplink)
+           *dl_tfi_mask |= (1 << t->dl_tfi);
+   }
+   return n;
+}
+
+int l1gprs_fw_ul_tbf_cfg(struct msgb *msg)
+{
+   struct l1ctl_hdr *l1h = (struct l1ctl_hdr *) msg->data;
+   struct l1ctl_gprs_ul_tbf_cfg_req *req = (void *) l1h->data;
+   unsigned long flags;
+
+   if (msg->len < sizeof(*l1h) + sizeof(*req))
+       return -EINVAL;
+   local_firq_save(flags);
+   tbf_cfg(true, req->tbf_ref, req->slotmask, 0, ntohl(req->start_fn));
+   local_irq_restore(flags);
+   return 0;
+}
+
+int l1gprs_fw_dl_tbf_cfg(struct msgb *msg)
+{
+   struct l1ctl_hdr *l1h = (struct l1ctl_hdr *) msg->data;
+   struct l1ctl_gprs_dl_tbf_cfg_req *req = (void *) l1h->data;
+   unsigned long flags;
+
+   if (msg->len < sizeof(*l1h) + sizeof(*req))
+       return -EINVAL;
+   if (req->dl_tfi > 31)
+       return -EINVAL;
+   local_firq_save(flags);
+   tbf_cfg(false, req->tbf_ref, req->slotmask, req->dl_tfi, ntohl(req->start_fn));
+   local_irq_restore(flags);
+   return 0;
+}
+
+int l1gprs_fw_ul_block_req(struct msgb *msg)
+{
+   struct l1ctl_hdr *l1h = (struct l1ctl_hdr *) msg->data;
+   struct l1ctl_gprs_ul_block_req *req = (void *) l1h->data;
+   unsigned long flags;
+   uint32_t fn;
+
+   if (msg->len < sizeof(*l1h) + sizeof(*req) + CS1_BLOCK_LEN) {
+       printf("l1gprs: short UL BLOCK.req (len=%u)\n", msg->len);
+       msgb_free(msg);
+       return -EINVAL;
+   }
+   fn = ntohl(req->hdr.fn);
+   if (req->hdr.tn > 7) {
+       msgb_free(msg);
+       return -EINVAL;
+   }
+   msg->l3h = req->data;
+   msg->cb[0] = fn;
+   msg->cb[1] = req->hdr.tn;
+   local_firq_save(flags);
+   msgb_enqueue(&ul_queue, msg);
+   local_irq_restore(flags);
+   return 0;
```

```
+}
+
+struct msgb *l1gprs_fw_ul_dequeue(uint32_t fn, uint8_t tn)
+{
+   struct msgb *msg, *tmp;
+
+   llist_for_each_entry_safe(msg, tmp, &ul_queue, list) {
+       int c = fn_cmp((uint32_t) msg->cb[0], fn);
+       if (c < 0) {
+           /* the ship has sailed */
+           printf("l1gprs: drop stale UL block fn=%u (now %u)\n", (uint32_t) msg->cb[0], fn);
+           llist_del(&msg->list);
+           msgb_free(msg);
+           continue;
+       }
+       if (c == 0 && (uint8_t) msg->cb[1] == tn) {
+           llist_del(&msg->list);
+           return msg;
+       }
+   }
+   return NULL;
+}
+
+void l1gprs_fw_ul_block_cnf(uint32_t fn, uint8_t tn, const uint8_t *data)
+{
+   struct l1ctl_gprs_ul_block_cnf *cnf;
+   struct msgb *msg = l1ctl_msgb_alloc(L1CTL_GPRS_UL_BLOCK_CNF);
+
+   cnf = (void *) msgb_put(msg, sizeof(*cnf));
+   cnf->fn = htonl(fn);
+   cnf->tn = tn;
+   if (data)
+       memcpy(msgb_put(msg, CS1_BLOCK_LEN), data, CS1_BLOCK_LEN);
+   l1_queue_for_l2(msg);
+}
+
+/* Does the (CS-1) downlink block address one of our DL TBFs? Control blocks
+ * without optional octets always pass (3GPP TS 44.060 10.2.1 / 10.3.1). */
+static bool filter_dl_block(uint32_t dl_tfi_mask, const uint8_t *data)
+{
+   uint8_t dl_tfi;
+   switch (data[0] >> 6) {
+   case 0x00:  /* RLC data block */
+       dl_tfi = (data[1] >> 1) & 0x1f;
+       break;
+   case 0x02:  /* control block with optional octets */
+       dl_tfi = (data[2] >> 1) & 0x1f;
+       break;
+   case 0x01:  /* control block, no optional octets */
+       return true;
+   default:
+       return false;
+   }
+   return (dl_tfi_mask & (1 << dl_tfi)) != 0;
+}
+
+void l1gprs_fw_dl_block(uint32_t fn, uint8_t tn, const uint8_t *data,
+           bool crc_ok, uint8_t rx_lev, uint16_t num_biterr, uint16_t snr)
+{
+   struct l1ctl_gprs_dl_block_ind *ind;
+   struct l1ctl_gprs_rts_ind *rts;
+   uint32_t dl_tfi_mask = 0;
+   uint8_t usf = 0xff;
+   struct msgb *msg;
+   uint32_t ber10k;
+
```

```
+   check_pending(fn);
+
+   if (pdch_count(tn, false, &dl_tfi_mask) == 0) {
+       /* nothing configured on this PDCH (yet): L23 does not want to hear about it */
+       return;
+   }
+
+   ber10k = (uint32_t) num_biterr * 10000 / 456;
+   if (ber10k > 10000)
+       ber10k = 10000;
+
+   msg = l1ctl_msgb_alloc(L1CTL_GPRS_DL_BLOCK_IND);
+   ind = (void *) msgb_put(msg, sizeof(*ind));
+   ind->hdr.fn = htonl(fn);
+   ind->hdr.tn = tn;
+   ind->meas.ber10k = htons(ber10k);
+   ind->meas.ci_cb = htons((int16_t) (snr > 300 ? 300 : snr));
+   ind->meas.rx_lev = rx_lev;
+   ind->usf = 0xff;
+   if (crc_ok && data) {
+       if ((fn % 104) == 12) {
+           /* PTCCH/D: pass as-is */
+           memcpy(msgb_put(msg, CS1_BLOCK_LEN), data, CS1_BLOCK_LEN);
+       } else {
+           usf = data[0] & 0x07;
+           ind->usf = usf;
+           if (filter_dl_block(dl_tfi_mask, data))
+               memcpy(msgb_put(msg, CS1_BLOCK_LEN), data, CS1_BLOCK_LEN);
+       }
+   }
+   l1_queue_for_l2(msg);
+
+   /* Ready-to-send for the uplink block L1GPRS_FW_RTS_ADVANCE frames after
+    * this one.  The USF is the one just decoded: on the bench a lone UL TBF
+    * gets its USF in every block, and polls (RRBP) are fn-exact anyway. */
+   msg = l1ctl_msgb_alloc(L1CTL_GPRS_RTS_IND);
+   rts = (void *) msgb_put(msg, sizeof(*rts));
+   rts->fn = htonl((fn + L1GPRS_FW_RTS_ADVANCE) % GSM_MAX_FN);
+   rts->tn = tn;
+   rts->usf = usf;
+   l1_queue_for_l2(msg);
+}
diff --git a/src/target/firmware/layer1/prim_pdtch.c b/src/target/firmware/layer1/prim_pdtch.c
new file mode 100644
index 00000000..bac76f3a
--- /dev/null
+++ b/src/target/firmware/layer1/prim_pdtch.c
@@ -0,0 +1,115 @@
+/* prim_pdtch.c - PDTCH uplink (CS-1, four normal bursts, DUL task) for the
+ * Calypso firmware.  Mirror of prim_tx_nb.c, but a block is only sent when
+ * L23 queued one for exactly this frame number (L1CTL_GPRS_UL_BLOCK_REQ);
+ * otherwise the uplink block period is left silent.
+ *
+ * (C) 2026 osmo-operator bench. GPLv2+. */
+#include <stdint.h>
+#include <stdio.h>
+#include <string.h>
+
+#include <defines.h>
+#include <debug.h>
+#include <memory.h>
+#include <byteorder.h>
+#include <osmocom/gsm/gsm_utils.h>
+#include <osmocom/core/msgb.h>
+#include <calypso/dsp_api.h>
+#include <calypso/irq.h>
```

```
+#include <calypso/tpu.h>
+#include <calypso/tsp.h>
+#include <calypso/dsp.h>
+#include <calypso/timer.h>
+#include <comm/sercomm.h>
+
+#include <layer1/sync.h>
+#include <layer1/agc.h>
+#include <layer1/tdma_sched.h>
+#include <layer1/mframe_sched.h>
+#include <layer1/tpu_window.h>
+#include <layer1/l23_api.h>
+#include <layer1/rfch.h>
+#include <layer1/prim.h>
+#include <layer1/l1gprs_fw.h>
+
+#include <l1ctl_proto.h>
+
+static struct {
+   uint8_t active;     /* a block is being transmitted in this period */
+   uint32_t fn;        /* its first frame number */
+   uint8_t tn;
+   uint8_t data[23];
+} pdtch_ul;
+
+static int l1s_pdtch_tx_resp(__unused uint8_t p1, uint8_t burst_id, __unused uint16_t p3)
+{
+   if (!pdtch_ul.active)
+       return 0;
+
+   putchart('p');
+   dsp_api.r_page_used = 1;
+
+   if (burst_id == 3) {
+       pdtch_ul.active = 0;
+       l1gprs_fw_ul_block_cnf(pdtch_ul.fn, pdtch_ul.tn, pdtch_ul.data);
+   }
+   return 0;
+}
+
+static int l1s_pdtch_tx_cmd(__unused uint8_t p1, uint8_t burst_id, __unused uint16_t p3)
+{
+   uint16_t arfcn;
+   uint8_t tsc, tn;
+
+   rfch_get_params(&l1s.next_time, &arfcn, &tsc, &tn);
+
+   if (burst_id == 0) {
+       uint16_t *info_ptr = dsp_api.ndb->a_cu;
+       struct msgb *msg;
+
+       msg = l1gprs_fw_ul_dequeue(l1s.next_time.fn, tn);
+       if (!msg) {
+           pdtch_ul.active = 0;
+           return 0;
+       }
+       putchart('P');
+       pdtch_ul.active = 1;
+       pdtch_ul.fn = l1s.next_time.fn;
+       pdtch_ul.tn = tn;
+       memcpy(pdtch_ul.data, msg->l3h, sizeof(pdtch_ul.data));
+       msgb_free(msg);
+
+       /* Fill data block Header */
+       info_ptr[0] = (1 << B_BLUD);     // 1st word: Set B_BLU bit.
+       info_ptr[1] = 0;                 // 2nd word: cleared.
```

```
+       info_ptr[2] = 0;                // 3rd word: cleared.
+       /* Copy the actual data after the header */
+       dsp_memcpy_to_api(&info_ptr[3], pdtch_ul.data, 23, 0);
+   } else if (!pdtch_ul.active) {
+       return 0;
+   }
+
+   l1s_tx_apc_helper(arfcn);
+
+   /* DUL_DSP_TASK, four normal bursts: the ROM's xCCH coding is CS-1 */
+   dsp_load_tch_param(&l1s.next_time, SIG_ONLY_MODE, SDCCH_4, 0, 0, 0, tn);
+
+   dsp_load_tx_task(
+       dsp_task_iq_swap(DUL_DSP_TASK, arfcn, 1),
+       burst_id, tsc
+   );
+
+   l1s_tx_win_ctrl(arfcn | ARFCN_UPLINK, L1_TXWIN_NB, 0, 3);
+
+   return 0;
+}
+
+const struct tdma_sched_item pdtch_sched_set_ul[] = {
+   SCHED_ITEM_DT(l1s_pdtch_tx_cmd, 3, 2, 0),                       SCHED_END_FRAME(),
+   SCHED_ITEM_DT(l1s_pdtch_tx_cmd, 3, 2, 1),                       SCHED_END_FRAME(),
+   SCHED_ITEM(l1s_pdtch_tx_resp, -4, 2, 0),    SCHED_ITEM_DT(l1s_pdtch_tx_cmd, 3, 2, 2),   SCHED_END_FRAME(),
+   SCHED_ITEM(l1s_pdtch_tx_resp, -4, 2, 1),    SCHED_ITEM_DT(l1s_pdtch_tx_cmd, 3, 2, 3),   SCHED_END_FRAME(),
+                       SCHED_ITEM(l1s_pdtch_tx_resp, -4, 2, 2),    SCHED_END_FRAME(),
+                       SCHED_ITEM(l1s_pdtch_tx_resp, -4, 2, 3),    SCHED_END_FRAME(),
+   SCHED_END_SET()
+};
```

2.154 /opt/GSM/osmo-operator/patches/osmocom-bb-firmware-map-ld.patch

1449 octets, 24 lignes → 24 lignes

```
osmocom-bb firmware : -Wl,-Map,<fichier> au lieu de -Wl,-Map <fichier>

[2026-10-24] Le lien des .elf passe par gcc ($(LD) = $(CC)). « -Wl,-Map board/x.map »
ne transmet a ld QUE « -Map » ; le chemin du .map reste un argument de gcc, qui le prend
pour un fichier d entree et s arrete avant ld :
  arm-elf-gcc: board/compal_e88/hello_world.highram.map : No such file or directory
  make[1]: *** [Makefile.inc:139: board/compal_e88/hello_world.highram.elf] Error 1
Avec la virgule, option et fichier vont ensemble a ld. tools/osmo-firmwares-install.sh
(_fw_bb_corriger) le faisait par sed pour le supplement --gnuarm ; le patch couvre aussi
un « make firmware » tape a la main dans osmocom-bb/src.

diff --git a/src/target/firmware/Makefile.inc b/src/target/firmware/Makefile.inc
index 86e8f3ce..17549064 100644
--- a/src/target/firmware/Makefile.inc
+++ b/src/target/firmware/Makefile.inc
@@ -99,7 +99,7 @@ $(1)_$(2)_$(3)_OBJS+=board/$(2)/$(1).$(3).manifest.o
 # define compilation rule, also generates map file
 board/$(2)/$(1).$(3).elf board/$(2)/$(1).$(3).map: $$($(1)_$(2)_$(3)_OBJS) $$($(1)_$(2)_$(3)_LIBS)
 ↪  $$(ENV_$(3)_LDS)
    $$(Q_LD)$(CROSS_COMPILE)$(LD) $(LDFLAGS) -T $$(ENV_$(3)_LDS) -Bstatic \
-       -Wl,-Map board/$(2)/$(1).$(3).map -o board/$(2)/$(1).$(3).elf \
+       -Wl,-Map,board/$(2)/$(1).$(3).map -o board/$(2)/$(1).$(3).elf \
        -Wl,--start-group $$($(1)_$(2)_$(3)_OBJS) $$($(1)_$(2)_$(3)_LIBS) -Wl,--end-group

 # define size rule
```

2.155 /opt/GSM/osmo-operator/patches/osmocom-bb-force-rand-toy.patch

1210 octets, 30 lignes → 30 lignes

```
diff --git a/src/host/layer23/src/common/subscriber.c b/src/host/layer23/src/common/subscriber.c
index f6aa05d5..53645b04 100644
--- a/src/host/layer23/src/common/subscriber.c
+++ b/src/host/layer23/src/common/subscriber.c
@@ -16,6 +16,7 @@
  */

 #include <stdint.h>
+#include <stdlib.h>
 #include <errno.h>
 #include <string.h>
 #include <arpa/inet.h>
@@ -265,6 +266,17 @@ int gsm_subscr_generate_kc(struct osmocom_ms *ms, uint8_t key_seq, const uint8_t
     struct gsm_subscriber *subscr = &ms->subscr;
     struct osmobb_l23_subscr_sim_auth_resp_sig_data sd;
     int rc;
+   static const uint8_t forced_rand[16];
+
+   /* force RAND=0 for the fixed COMP128v1 toy table
+    * (see /root/deka/crack_toy.py) instead of the network's RAND.
+    * FORCE_RAND is a presence-only switch: its value is never read, so
+    * this can only ever force an all-zero RAND, never an arbitrary one.
+    * Unset (default): rand is untouched, normal behavior. */
+   if (getenv("FORCE_RAND")) {
+       LOGP(DMM, LOGL_NOTICE, "FORCE_RAND set: using RAND=0 (toy mode)\n");
+       rand = forced_rand;
+   }

     if (no_sim || subscr->sim_type == GSM_SIM_TYPE_NONE || !subscr->sim_valid) {
         LOGP(DMM, LOGL_INFO, "Sending dummy authentication response\n");
```

2.156 /opt/GSM/osmo-operator/patches/osmocom-bb-osmocon-kc-filter.patch

2335 octets, 62 lignes → 62 lignes

```
diff --git a/src/host/osmocon/osmocon.c b/src/host/osmocon/osmocon.c
index 547f49fe..d8921885 100644
--- a/src/host/osmocon/osmocon.c
+++ b/src/host/osmocon/osmocon.c
@@ -1281,6 +1281,57 @@ static int un_tool_read(struct osmo_fd *fd, unsigned int flags)
         c += rc;
     }

+   /* === FILTRE Kc en amont du firmware (chiffrement A5) ===
+    * Capture L1CTL_CRYPTO_REQ (mobile -> L1) et ecrit /dev/shm/calypso_kc
+    * pour qemu_wrap (cipher UL) + si_bridge/grgsm (decipher DL).
+    * Layout fichier (32o): [0..3]seq LE [4]algo [5]key_len=8 [6..13]Kc [14]key_seq(CKSN)
+    * NE TOUCHE PAS au firmware. */
+   if (length >= 10 && buf[0] == 0x15 /* L1CTL_CRYPTO_REQ */) {
+       uint8_t algo = buf[8];
+       uint8_t klen = buf[9];
+       unsigned i;
+       fprintf(stderr, "[osmocon] CRYPTO_REQ FILTRE: len=%u algo=%u key_len=%u key=",
+           length, algo, klen);
+       for (i = 10; i < length && i < 32; i++)
+           fprintf(stderr, "%02x", buf[i]);
+       fprintf(stderr, "\n");
+       if (algo) {
+           static uint32_t kc_seq = 0;
```

```
+           uint8_t kc[32];
+           /* key_seq (CKSN) : 9e octet quand key_len=9, sinon inconnu */
+           uint8_t key_seq = (klen == 9 && length >= 19) ? buf[18] : 0xff;
+           FILE *kf;
+           memset(kc, 0, sizeof(kc));
+           kc_seq++;
+           memcpy(kc, &kc_seq, 4);
+           kc[4] = algo;
+           kc[5] = 8;                 /* on stocke toujours 8 octets de Kc */
+           memcpy(kc + 6, &buf[10], 8);
+           kc[14] = key_seq;
+           kf = fopen("/dev/shm/calypso_kc", "wb");
+           if (kf) { fwrite(kc, 1, sizeof(kc), kf); fclose(kf); }
+           fprintf(stderr, "[osmocon] Kc ecrit (seq=%u algo=%u cksn=0x%02x)\n",
+               kc_seq, algo, key_seq);
+           if (klen != 8)
+               buf[9] = 8;        /* force key_len=8 (inert: corruption en aval) */
+       } else {
+           /* algo=0 : pas de chiffrement -> efface le Kc */
+           uint8_t zero[32];
+           FILE *kf;
+           memset(zero, 0, sizeof(zero));
+           kf = fopen("/dev/shm/calypso_kc", "wb");
+           if (kf) { fwrite(zero, 1, sizeof(zero), kf); fclose(kf); }
+       }
+   } else if (length >= 1 && (buf[0] == 0x05 /* DM_EST_REQ */ ||
+                buf[0] == 0x12 /* DM_REL_REQ */)) {
+       /* nouveau canal dedie ou release -> repart en clair */
+       uint8_t zero[32];
+       FILE *kf;
+       memset(zero, 0, sizeof(zero));
+       kf = fopen("/dev/shm/calypso_kc", "wb");
+       if (kf) { fwrite(zero, 1, sizeof(zero), kf); fclose(kf); }
+   }
+
    hdlc_send_to_phone(con->server->dlci, buf, length);

    return 0;
```

2.157 /opt/GSM/osmo-operator/patches/osmocom-bb-plmn-a6-reinsert.patch

574 octets, 14 lignes → 14 lignes

```
--- a/src/host/layer23/src/mobile/gsm322.c
+++ b/src/host/layer23/src/mobile/gsm322.c
@@ -1373,6 +1373,11 @@

    memset(&plmn->plmn, 0, sizeof(plmn->plmn));

+   /* back to the switched-on state: PLMN_SEARCH_END is only handled in A0.
+    * Left in A6 (SIM re-inserted without RPLMN, e.g. SAP reconnect) the
+    * search ends unhandled and the MS stays in PLMN search forever. */
+   new_a_state(plmn, GSM322_A0_NULL);
+
    /* initiate search at cell selection */
    LOGP(DSUM, LOGL_INFO, "Search for network\n");
    LOGP(DPLMN, LOGL_INFO, "Switch on, no RPLMN, start PLMN search "
```

2.158 /opt/GSM/osmo-operator/patches/osmocom-bb-sim-plmn-from-bcd.patch

774 octets, 20 lignes → 20 lignes

```
--- a/src/host/layer23/src/common/subscriber.c
+++ b/src/host/layer23/src/common/subscriber.c
@@ -930,7 +930,7 @@
        plmn = talloc_zero(ms, struct gsm_sub_plmn_list);
        if (!plmn)
            return -ENOMEM;
-       osmo_plmn_to_bcd(&data[0], &plmn->plmn);
+       osmo_plmn_from_bcd(&data[0], &plmn->plmn);
        llist_add_tail(&plmn->entry, &subscr->plmn_list);

        LOGP(DMM, LOGL_INFO, "received PLMN selector (mcc-mnc=%s) from SIM\n",
@@ -1027,7 +1027,7 @@
        na = talloc_zero(ms, struct gsm_sub_plmn_na);
        if (!na)
            return -ENOMEM;
-       osmo_plmn_to_bcd(&data[0], &na->plmn);
+       osmo_plmn_from_bcd(&data[0], &na->plmn);
        LOGP(DMM, LOGL_INFO, "received Forbidden PLMN %s from SIM\n",
            osmo_plmn_name(&na->plmn));
        na->cause = -1; /* must have a value, but SIM stores no cause */
```

2.159 /opt/GSM/osmo-operator/patches/osmocom-bb-transceiver-qemu-bench.patch

30756 octets, 853 lignes → 853 lignes

```
diff --git a/src/host/layer23/src/transceiver/app.h b/src/host/layer23/src/transceiver/app.h
index 7b8e25c..5e70dd6 100644
--- a/src/host/layer23/src/transceiver/app.h
+++ b/src/host/layer23/src/transceiver/app.h
@@ -26,6 +26,7 @@


 #include <stdint.h>
+#include <time.h>

 #include "l1ctl_link.h"

@@ -52,7 +53,24 @@ struct app_state

    /* Options */
    uint16_t arfcn_sync;
+   int no_sync;    /* [2026-10-19] mode BTS sans synchro sur une cellule (QEMU, pas d'air) */
+
+   /* [2026-10-21] telephone 2 (TS1, -2) : son compteur de trames est independant de celui du
+    * telephone 1 (chacun part de 0 a son boot). On mesure D = fn1 - fn2 sur les DATA_IND des deux
+    * (fn1 extrapole a l'instant du DATA_IND du 2 par le temps ecoule, 4.615 ms par trame), et on
+    * renumerote : BURST_REQ vers le 2 avec fn - D, indications du 2 avec fn + D. D est publie dans
+    * fn_delta_file pour l'air (pont/air.py) qui tague de meme les bursts du telephone 2. */
+   struct { uint32_t fn; struct timespec t; int valid; } clk[2];
+   int fn_delta, fn_delta_valid;
+   int fn_samples[64]; unsigned fn_n_samples, fn_n_total;
+   const char *fn_delta_file;
 };

+#define TRX_GSM_MAX_FN 2715648u
+/* fn du telephone 2 -> numerotation du telephone 1 (la BTS), et retour */
+#define TRX_FN_P2_TO_BTS(as, fn) ((uint32_t)(((int64_t)(fn) + (as)->fn_delta + TRX_GSM_MAX_FN) % TRX_GSM_MAX_FN))
+#define TRX_FN_BTS_TO_P2(as, fn) ((uint32_t)(((int64_t)(fn) - (as)->fn_delta + 2 * (int64_t)TRX_GSM_MAX_FN) %
 ↪  TRX_GSM_MAX_FN))
+int app_fn_delta_sample(struct app_state *as, int idx, uint32_t fn);
+

 #endif /* __TRX_APP_H__ */
diff --git a/src/host/layer23/src/transceiver/l1ctl.c b/src/host/layer23/src/transceiver/l1ctl.c
```

```
index 5bcabe3..e9af802 100644
--- a/src/host/layer23/src/transceiver/l1ctl.c
+++ b/src/host/layer23/src/transceiver/l1ctl.c
@@ -21,7 +21,9 @@
  * along with this program.  If not, see <http://www.gnu.org/licenses/>.
  */

+#include <stdio.h>
 #include <stdlib.h>
+#include <time.h>
 #include <string.h>

 #include <osmocom/core/select.h>
@@ -147,6 +149,75 @@ l1ctl_tx_bts_mode(struct l1ctl_link *l1l, uint8_t enabled, uint8_t *type,
     return l1l_send(l1l, msg);
 }

+/* [2026-10-21] mesure de D = fn1 - fn2 (voir app.h). idx 0 : telephone 1, 1 : telephone 2. */
+static void
+_fn_delta_write(struct app_state *as)
+{
+   char tmp[256];
+   FILE *f;
+
+   if (!as->fn_delta_file)
+       return;
+   snprintf(tmp, sizeof(tmp), "%s.tmp", as->fn_delta_file);
+   f = fopen(tmp, "w");
+   if (!f)
+       return;
+   fprintf(f, "%d\n", as->fn_delta);
+   fclose(f);
+   rename(tmp, as->fn_delta_file);
+}
+
+int
+app_fn_delta_sample(struct app_state *as, int idx, uint32_t fn)
+{
+   struct timespec now;
+   int64_t d, el_ns, fn1;
+   unsigned i, best = 0, best_n = 0;
+
+   clock_gettime(CLOCK_MONOTONIC, &now);
+   as->clk[idx].fn = fn;
+   as->clk[idx].t = now;
+   as->clk[idx].valid = 1;
+   if (idx != 1 || !as->clk[0].valid)
+       return 0;
+
+   /* fn du telephone 1 a cet instant : son dernier fn + les trames ecoulees depuis */
+   el_ns = (int64_t)(now.tv_sec - as->clk[0].t.tv_sec) * 1000000000LL + (now.tv_nsec - as->clk[0].t.tv_nsec);
+   if (el_ns < 0 || el_ns > 2000000000LL)
+       return 0;
+   fn1 = (int64_t)as->clk[0].fn + (el_ns + 2307692) / 4615385;
+   d = ((fn1 - (int64_t)fn) % TRX_GSM_MAX_FN + TRX_GSM_MAX_FN) % TRX_GSM_MAX_FN;
+   if (d > TRX_GSM_MAX_FN / 2)
+       d -= TRX_GSM_MAX_FN;
+   as->fn_samples[as->fn_n_samples++] = (int)d;
+   as->fn_n_total++;
+   if (as->fn_n_samples < (as->fn_delta_valid ? 64u : 16u))
+       return 0;
+
+   /* le mode des echantillons */
+   for (i = 0; i < as->fn_n_samples; i++) {
+       unsigned j, n = 0;
+       for (j = 0; j < as->fn_n_samples; j++)
```

```
+           if (as->fn_samples[j] == as->fn_samples[i])
+               n++;
+       if (n > best_n) { best_n = n; best = i; }
+   }
+   d = as->fn_samples[best];
+   as->fn_n_samples = 0;
+
+   if (!as->fn_delta_valid) {
+       as->fn_delta = (int)d; as->fn_delta_valid = 1;
+       LOGP(DL1C, LOGL_NOTICE, "Phone 2 frame offset: fn1 - fn2 = %d (%u/%u samples) -> written to %s\n",
+            as->fn_delta, best_n, 16, as->fn_delta_file ? as->fn_delta_file : "(nowhere)");
+       _fn_delta_write(as);
+   } else if (d - as->fn_delta >= 2 || as->fn_delta - d >= 2) {
+       LOGP(DL1C, LOGL_NOTICE, "Phone 2 frame offset drifted: %d -> %d (%u/64 samples)\n", as->fn_delta, (int)d,
 ↪  best_n);
+       as->fn_delta = (int)d;
+       _fn_delta_write(as);
+   }
+   return 0;
+}
+
 int
 l1ctl_tx_bts_burst_req(struct l1ctl_link *l1l,
                        uint32_t fn, uint8_t tn, struct burst_data *burst)
@@ -157,6 +228,13 @@ l1ctl_tx_bts_burst_req(struct l1ctl_link *l1l,
     if (!l1l->sync)
         return -EIO;

+   /* [2026-10-21] telephone 2 : dans SA numerotation de trames (fn - D) ; rien tant que D est inconnu */
+   if (l1l == &l1l->as->l1l[1]) {
+       if (!l1l->as->fn_delta_valid)
+           return 0;
+       fn = TRX_FN_BTS_TO_P2(l1l->as, fn);
+   }
+
     msg = _l1ctl_alloc(L1CTL_BTS_BURST_REQ);
     if (!msg)
         return -1;
@@ -184,7 +262,7 @@ _l1ctl_rx_bts_burst_nb_ind(struct l1ctl_link *l1l, struct msgb *msg)
 {
     struct l1ctl_bts_burst_nb_ind *bi;
     uint32_t fn;
-   int rc, i;
+   int rc = 0, i;
     sbit_t data[148], steal[2];
     float toa;
     int8_t rssi;
@@ -200,7 +278,13 @@ _l1ctl_rx_bts_burst_nb_ind(struct l1ctl_link *l1l, struct msgb *msg)

     fn = ntohl(bi->fn);

-   LOGP(DL1C, LOGL_INFO, "Normal Burst Indication (fn=%d)\n", fn);
+   /* [2026-10-21] telephone 2 : vers la numerotation de la BTS (fn + D) */
+   if (l1l == &l1l->as->l1l[1]) {
+       if (!l1l->as->fn_delta_valid) { rc = 0; goto exit; }
+       fn = TRX_FN_P2_TO_BTS(l1l->as, fn);
+   }
+
+   LOGP(DL1C, LOGL_INFO, "Normal Burst Indication (fn=%d tn=%d)\n", fn, bi->tn);

     toa = bi->toa * 1.0F;

@@ -244,6 +328,10 @@ _l1ctl_rx_bts_burst_ab_ind(struct l1ctl_link *l1l, struct msgb *msg)
     }

     fn = ntohl(bi->fn);
```

```
+   if (l1l == &l1l->as->l1l[1]) {
+       if (!l1l->as->fn_delta_valid) { rc = 0; goto exit; }
+       fn = TRX_FN_P2_TO_BTS(l1l->as, fn);
+   }

    rc = gsm_ab_ind_process(l1l->as, bi, data, &toa);
    if (rc < 0)
@@ -265,7 +353,7 @@ _l1ctl_rx_data_ind(struct l1ctl_link *l1l, struct msgb *msg)
 {
    struct l1ctl_info_dl *dl;
    struct l1ctl_data_ind *di;
-   int rc;
+   int rc = 0;

    dl = (struct l1ctl_info_dl *) msg->l1h;
    msg->l2h = dl->payload;
@@ -299,6 +387,8 @@ _l1ctl_rx_data_ind(struct l1ctl_link *l1l, struct msgb *msg)
    /* forward clock of fist l1ctl_link instance only */
    if (l1l == &l1l->as->l1l[0])
        rc = trx_clk_ind(l1l->trx, ntohl(dl->frame_nr));
+   /* [2026-10-21] et la mesure du decalage de trames entre les deux telephones */
+   app_fn_delta_sample(l1l->as, l1l == &l1l->as->l1l[0] ? 0 : 1, ntohl(dl->frame_nr));

 exit:
    msgb_free(msg);
@@ -380,6 +470,16 @@ l1ctl_recv(void *data, struct msgb *msg)

    case L1CTL_RESET_IND:
    case L1CTL_RESET_CONF:
+       if (l1l->as->no_sync) {
+           /* [2026-10-19] pas de cellule a ecouter : le compteur de trames
+            * du telephone fait foi, mode BTS des que la BTS a dit POWERON */
+           LOGP(DAPP, LOGL_INFO, "Reset received: --no-sync, BTS mode without cell sync.\n");
+           l1l->sync = 1;
+           if (l1l->trx->power)
+               l1ctl_tx_bts_mode(l1l, 1, l1l->trx->type, l1l->trx->bsic, l1l->trx->arfcn, l1l->trx->gain,
 ↪  l1l->tx_mask, l1l->rx_mask, l1l->trx->handover);
+           msgb_free(msg);
+           break;
+       }
        LOGP(DAPP, LOGL_INFO, "Reset received: Starting sync.\n");
        l1l->sync = 0;
        l1ctl_tx_fbsb_req(l1l, l1l->as->arfcn_sync, L1CTL_FBSB_F_FB01SB, 100, 0, CCCH_MODE_NONE);
diff --git a/src/host/layer23/src/transceiver/main.c b/src/host/layer23/src/transceiver/main.c
index a895ffc..86c8c5c 100644
--- a/src/host/layer23/src/transceiver/main.c
+++ b/src/host/layer23/src/transceiver/main.c
@@ -22,6 +22,7 @@
  */

 #include <stdlib.h>
+#include <unistd.h>
 #include <string.h>
 #include <getopt.h>
 #include <sched.h>
@@ -46,6 +47,25 @@ static int arfcn_sync = 0;
 static int daemonize = 0;
 static int rt_prio = -1;
 static int second_phone = 0;
+/* [2026-10-19] Banc QEMU (osmo-operator, start-native.sh --osmo-trx) : le
+ * telephone emule n'a aucune cellule sur laquelle se caler (FBSB), on passe
+ * en mode BTS des le reset ; un seul timeslot pour commencer (--ts-mask 0x01).
+ * [2026-10-20] --ts-mask ne touche que le masque RX : le masque TX reste 0xe3
+ * (TS 5,6,7,0,1), car prim_bts.c (l1s_bts_cmd) ne sert le TS0 BCCH que si
+ * tx_start == 5 (la fenetre TX du Calypso est decalee de 3 slots : SLOT 3 de
+ * la fenetre multiple ouverte en 5 tombe sur le TS0). Avec tx_mask 0x01,
```

```
+ * tx_start valait 0 et le firmware emettait tn 1 -> dummy burst a chaque trame
+ * (c54x_exe : « DUMMY 18000, FB 0, SB 0 »). --tx-mask pour le forcer quand meme. */
+static int no_sync = 0;
+static int ts_mask = -1;
+static int tx_mask_opt = -1;
+/* [2026-10-20] second telephone (TS1, --second-phone) : son socket L1CTL (banc QEMU : /tmp/osmocom_l2_3) */
+static const char *l1ctl2_path = "/tmp/osmocom_l2.2";
+/* [2026-10-21] ou publier fn1 - fn2 du telephone 2 (lu par pont/air.py) */
+static const char *fn_delta_file = "/tmp/osmocom_fn_delta_2";
+static const char *l1ctl_path = "/tmp/osmocom_l2";
+static int base_port = 5700;
+static int bsic_opt = -1;

 static void print_help(char *argv[])
 {
@@ -61,6 +81,14 @@ static void print_help(char *argv[])
         "  -p   --arfcn-sync-pcs   The ARFCN above is PCS\n"
         "  -2   --second-phone     Use second phone for TS 1\n"
         "  -r   --realtime PRIO    Set realtime scheduler with given prio\n"
+       "  -n   --no-sync          No cell sync (FBSB): BTS mode right after the phone reset (QEMU bench)\n"
+       "  -m   --ts-mask MASK     RX timeslot mask of phone 1 (default 0x01 = TS0 only)\n"
+       "  -M   --tx-mask MASK     TX timeslot mask of phone 1 (default 0xe3 = TS 5,6,7,0,1 : tx_start 5 serves
 ↪  TS0)\n"
+       "  -l   --l1ctl PATH       L1CTL socket of phone 1 (default /tmp/osmocom_l2)\n"
+       "  -L   --l1ctl2 PATH      L1CTL socket of phone 2, TS1 (default /tmp/osmocom_l2.2, with -2)\n"
+       "  -F   --fn-delta-file P  where to publish fn1 - fn2 of phone 2 (default /tmp/osmocom_fn_delta_2, with
 ↪  -2)\n"
+       "  -P   --base-port PORT   TRX UDP base port (default 5700)\n"
+       "  -b   --bsic BSIC        BSIC of the cell (osmo-bts-trx sends SETTSC, not SETBSIC: NCC<<3|BCC)\n"
         );
 }

@@ -78,9 +106,18 @@ static void handle_options(int argc, char **argv, struct app_state *as)
             { "arfcn-sync-pcs", 0, 0, 'p' },
             { "second-phone", 0, 0, '2' },
             { "realtime", 1, 0, 'r' },
+           { "no-sync", 0, 0, 'n' },
+           { "ts-mask", 1, 0, 'm' },
+           { "tx-mask", 1, 0, 'M' },
+           { "l1ctl", 1, 0, 'l' },
+           { "l1ctl2", 1, 0, 'L' },
+           { "fn-delta-file", 1, 0, 'F' },
+           { "base-port", 1, 0, 'P' },
+           { "bsic", 1, 0, 'b' },
+           { NULL, 0, 0, 0 },
         };

-       c = getopt_long(argc, argv, "hd:e:Dsa:p2r:",
+       c = getopt_long(argc, argv, "hd:e:Dsa:p2r:nm:M:l:L:F:P:b:",
             long_options, &option_idx);

         if (c == -1)
@@ -116,6 +153,30 @@ static void handle_options(int argc, char **argv, struct app_state *as)
         case 'r':
             rt_prio = atoi(optarg);
             break;
+       case 'n':
+           no_sync = 1;
+           break;
+       case 'm':
+           ts_mask = strtol(optarg, NULL, 0) & 0xff;
+           break;
+       case 'M':
+           tx_mask_opt = strtol(optarg, NULL, 0) & 0xff;
+           break;
+       case 'l':
```

```
+           l1ctl_path = optarg;
+           break;
+       case 'L':
+           l1ctl2_path = optarg;
+           break;
+       case 'F':
+           fn_delta_file = optarg;
+           break;
+       case 'P':
+           base_port = atoi(optarg);
+           break;
+       case 'b':
+           bsic_opt = atoi(optarg) & 0x3f;
+           break;
        default:
            fprintf(stderr, "Unknow option %s\n", optarg);
            exit(0);
@@ -148,10 +209,14 @@ int main(int argc, char *argv[])
        return -1;
    }
    handle_options(argc, argv, as);
-   if (!arfcn_sync) {
-       fprintf(stderr, "Use --arfcn-sync <ARFCN>\n");
+   if (!arfcn_sync && !no_sync) {
+       fprintf(stderr, "Use --arfcn-sync <ARFCN> (or --no-sync)\n");
        exit(0);
    }
+   as->no_sync = no_sync;
+   as->fn_delta_file = second_phone ? fn_delta_file : NULL;
+   if (second_phone)
+       unlink(fn_delta_file);  /* pas de D perime pour l'air */

    /* App state init */
    printf("%d\n", as->arfcn_sync);
@@ -168,34 +233,55 @@ int main(int argc, char *argv[])
        exit(-1);

    /* TRX interface with OpenBTS */
-   as->trx[0] = trx_alloc("127.0.0.1", 5700, as, 1);
-   if (!as->trx)
+   as->trx[0] = trx_alloc("127.0.0.1", base_port, as, 1);
+   if (!as->trx[0])
        exit(-1);
+   if (bsic_opt >= 0)
+       as->trx[0]->bsic = bsic_opt;

    /* Establish l1ctl link */
-   rv = l1l_open(&as->l1l[0], "/tmp/osmocom_l2", l1ctl_recv, &as->l1l[0]);
+   rv = l1l_open(&as->l1l[0], l1ctl_path, l1ctl_recv, &as->l1l[0]);
    if (rv)
        exit(-1);
    as->l1l[0].trx = as->trx[0];
    as->l1l[0].as = as;
    as->l1l[0].tx_mask = 0xe3; /* TS 5,6,7,0,1 */
    as->l1l[0].rx_mask = 0x01; /* TS 0 */
+   if (ts_mask >= 0)
+       as->l1l[0].rx_mask = ts_mask;
+   if (tx_mask_opt >= 0)
+       as->l1l[0].tx_mask = tx_mask_opt;
    as->trx[0]->l1l[0] = &as->l1l[0];
+   /* [2026-10-20] banc QEMU : le telephone 1 sert TOUS les intervalles de ses masques (TS0 et TS1
+    * avec 0x03) : les bursts de la BTS pour ces TS lui sont transmis et ses indications tn=1 acceptees */
+   if (!second_phone) {
+       int tn;
+       for (tn = 1; tn < 8; tn++)
+           if ((as->l1l[0].tx_mask | as->l1l[0].rx_mask) & (1 << tn))
```

```
+               as->trx[0]->l1l[tn] = &as->l1l[0];
+   }
+   printf("transceiver: l1ctl %s, TRXD base port %d, tx_mask 0x%02x rx_mask 0x%02x%s\n",
+       l1ctl_path, base_port, as->l1l[0].tx_mask, as->l1l[0].rx_mask,
+       no_sync ? ", no-sync (BTS mode at reset)" : "");

    /* Reset phone */
    l1ctl_tx_reset_req(&as->l1l[0], L1CTL_RES_T_FULL);

    if (second_phone) {
        /* Establish l1ctl link */
-       rv = l1l_open(&as->l1l[1], "/tmp/osmocom_l2.2", l1ctl_recv, &as->l1l[1]);
+       rv = l1l_open(&as->l1l[1], l1ctl2_path, l1ctl_recv, &as->l1l[1]);
        if (rv)
            exit(-1);
        as->l1l[1].trx = as->trx[0];
        as->l1l[1].as = as;
        as->l1l[1].tx_mask = 0x02; /* TS 1 */
        as->l1l[1].rx_mask = 0x02; /* TS 1 */
-       as->l1l[0].tx_mask = 0xe1; /* TS 5,6,7,0 */
-       as->l1l[0].rx_mask = 0x01; /* TS 0 */
+       /* [2026-10-20] le telephone 1 garde -M / -m s'ils ont ete donnes (banc QEMU : 0x01 / 0x01) */
+       if (tx_mask_opt < 0)
+           as->l1l[0].tx_mask = 0xe1; /* TS 5,6,7,0 */
+       if (ts_mask < 0)
+           as->l1l[0].rx_mask = 0x01; /* TS 0 */
+       printf("transceiver: second phone (TS1) on l1ctl %s\n", l1ctl2_path);
        as->trx[0]->l1l[1] = &as->l1l[1];

        /* Reset phone */
diff --git a/src/host/layer23/src/transceiver/trx.c b/src/host/layer23/src/transceiver/trx.c
index 3f51928..eb38b2a 100644
--- a/src/host/layer23/src/transceiver/trx.c
+++ b/src/host/layer23/src/transceiver/trx.c
@@ -265,11 +265,51 @@ _trx_ctrl_cmd_poweron(struct trx *trx, const char *cmd, const char *args)
 static int
 _trx_ctrl_cmd_settsc(struct trx *trx, const char *cmd, const char *args)
 {
-   LOGP(DTRX, LOGL_ERROR,
-       "TRX received SETTSC command ! "
-       "OpenBTS should be configured to send SETBSIC command !\n");
+   /* [2026-10-19] osmo-bts-trx envoie SETTSC (critique) puis SETBSIC ; le TSC
+    * est le BCC du BSIC, le firmware le derive lui-meme : on acquiesce. */
+   int tsc = atoi(args);

-   return _trx_ctrl_send_resp(trx, cmd, "%d", -1);
+   if (tsc < 0 || tsc > 7)
+       return _trx_ctrl_send_resp(trx, cmd, "%d", -1);
+
+   /* osmo-bts-trx n'envoie SETBSIC qu'en mode legacy : sans --bsic, on prend
+    * NCC 0 et BCC = TSC, sinon on verifie que le BCC donne suit le TSC. */
+   if (trx->bsic == BSIC_INVAL) {
+       trx->bsic = tsc;
+       LOGP(DTRX, LOGL_NOTICE, "SETTSC %d: no BSIC known, assuming NCC 0 -> BSIC %d (use --bsic)\n", tsc,
 ↪  trx->bsic);
+   } else if ((trx->bsic & 7) != tsc) {
+       LOGP(DTRX, LOGL_ERROR, "SETTSC %d does not match BCC of BSIC %d\n", tsc, trx->bsic);
+       return _trx_ctrl_send_resp(trx, cmd, "%d", -1);
+   } else
+       LOGP(DTRX, LOGL_INFO, "SETTSC %d accepted (BSIC %d)\n", tsc, trx->bsic);
+   return _trx_ctrl_send_resp(trx, cmd, "%d", 0);
+}
+
+/* [2026-10-19] Deux commandes d'osmo-bts-trx (>= 1.3) inconnues d'OpenBTS 2013 :
+ *  - SETFORMAT <ver> : negociation du format TRXD. Nous ne parlons que la v0 ;
+ *    reponse « RSP SETFORMAT 0 <ver> » (statut = version retenue, params =
```

```
+ *    la requete, ce que cmd_matches_rsp() d'osmo-bts compare) ;
+ *  - NOMTXPOWER (critique) : puissance nominale, « RSP NOMTXPOWER 0 <dBm> ». */
+#define TRX_NOM_TX_POWER_DBM   23
+
+static int
+_trx_ctrl_cmd_setformat(struct trx *trx, const char *cmd, const char *args)
+{
+   LOGP(DTRX, LOGL_INFO, "SETFORMAT %s requested, answering TRXD version 0\n", args);
+   return _trx_ctrl_send_resp(trx, cmd, "%d %s", 0, args);
+}
+
+static int
+_trx_ctrl_cmd_nomtxpower(struct trx *trx, const char *cmd, const char *args)
+{
+   return _trx_ctrl_send_resp(trx, cmd, "%d %d", 0, TRX_NOM_TX_POWER_DBM);
+}
+
+static int
+_trx_ctrl_cmd_rfmute(struct trx *trx, const char *cmd, const char *args)
+{
+   /* RFMUTE 0|1 (non critique) : rien a couper dans QEMU, on acquiesce */
+   return _trx_ctrl_send_resp(trx, cmd, "%d", 0);
 }

 static int
@@ -441,6 +481,9 @@ static const struct trx_cmd_handler trx_handlers[] = {
    { "POWEROFF",   _trx_ctrl_cmd_poweroff },
    { "POWERON",    _trx_ctrl_cmd_poweron },
    { "SETTSC", _trx_ctrl_cmd_settsc },
+   { "SETFORMAT",  _trx_ctrl_cmd_setformat },
+   { "NOMTXPOWER", _trx_ctrl_cmd_nomtxpower },
+   { "RFMUTE", _trx_ctrl_cmd_rfmute },
    { "SETBSIC",    _trx_ctrl_cmd_setbsic },
    { "SETPOWER",   _trx_ctrl_cmd_setpower },
    { "SETRXGAIN",  _trx_ctrl_cmd_setrxgain },
diff --git a/src/target/firmware/Makefile.inc b/src/target/firmware/Makefile.inc
index 9049898..47524dc 100644
--- a/src/target/firmware/Makefile.inc
+++ b/src/target/firmware/Makefile.inc
@@ -1,17 +1,17 @@

 #### TOOLCHAIN CONFIGURATION ####

-CROSS_COMPILE?=arm-elf-
+CROSS_COMPILE?=arm-none-eabi-

 CC=gcc
-LD=ld
+LD=gcc
 AR=ar
 SIZE=size
 OBJCOPY=objcopy

 DEBUGF=dwarf-2

-CFLAGS=-mcpu=arm7tdmi $(INCLUDES)
+CFLAGS=-mcpu=arm7tdmi -fcommon $(INCLUDES)
 CFLAGS += -Wall -Wextra -Wcast-align -Wimplicit -Wunused
 CFLAGS += -Wswitch -Wredundant-decls -Wreturn-type -Wshadow -Wnested-externs
 CFLAGS += -Wbad-function-cast -Wsign-compare -Waggregate-return
@@ -22,7 +22,7 @@ CFLAGS += -g$(DEBUGF)
 #ASFLAGS=-g$(DEBUGF) $(INCLUDES) -D__ASSEMBLY__
 ASFLAGS=$(INCLUDES) -D__ASSEMBLY__

-LDFLAGS = -nostartfiles -nostdlib -nodefaultlibs --gc-sections --cref
+LDFLAGS = -nostartfiles -nostdlib -nodefaultlibs -Wl,--gc-sections -Wl,--cref
```

```
 #### QUIET OUTPUT ####

@@ -95,8 +95,8 @@ $(1)_$(2)_$(3)_OBJS+=board/$(2)/$(1).$(3).manifest.o
 # define compilation rule, also generates map file
 board/$(2)/$(1).$(3).elf board/$(2)/$(1).$(3).map: $$($(1)_$(2)_$(3)_OBJS) $$($(1)_$(2)_$(3)_LIBS)
 ↪  $$(ENV_$(3)_LDS)
     $$(Q_LD)$(CROSS_COMPILE)$(LD) $(LDFLAGS) -T $$(ENV_$(3)_LDS) -Bstatic \
-       -Map board/$(2)/$(1).$(3).map -o board/$(2)/$(1).$(3).elf \
-       --start-group $$($(1)_$(2)_$(3)_OBJS) $$($(1)_$(2)_$(3)_LIBS) --end-group
+       -Wl,-Map,board/$(2)/$(1).$(3).map -o board/$(2)/$(1).$(3).elf \
+       -Wl,--start-group $$($(1)_$(2)_$(3)_OBJS) $$($(1)_$(2)_$(3)_LIBS) -Wl,--end-group

 # define size rule
 board/$(2)/$(1).$(3).size: board/$(2)/$(1).$(3).elf
diff --git a/src/target/firmware/include/stdint.h b/src/target/firmware/include/stdint.h
index 627403f..5997f43 100644
--- a/src/target/firmware/include/stdint.h
+++ b/src/target/firmware/include/stdint.h
@@ -19,7 +19,7 @@
 #include_next <stdint.h>
 #endif

-#ifndef __int8_t_defined
+#if (!defined(__int8_t_defined) && !defined(INT8_MAX))
 typedef signed char int8_t;
 typedef unsigned char uint8_t;

diff --git a/src/target/firmware/layer1/l23_api.c b/src/target/firmware/layer1/l23_api.c
index 6841f80..9cc87f1 100644
--- a/src/target/firmware/layer1/l23_api.c
+++ b/src/target/firmware/layer1/l23_api.c
@@ -603,12 +603,22 @@ static int l1ctl_bts_mode(struct msgb *msg)

     l1s.tx_power = ms_pwr_ctl_lvl(gsm_arfcn2band(l1s.bts.arfcn), 15);

-    l1a_mftask_set(0);
+    /* [2026-10-20] BTS_MODE est RENVOYE par l'appli hote a chaque SETRXGAIN, SETSLOT,
+     * HANDOVER/NOHANDOVER (trx.c) : osmo-bts-trx en envoie un juste apres le RACH
+     * (activation du canal). Refaire l1a_mftask_set(0) + mframe_enable() en pleine
+     * marche cassait la tache de synchro (prim_bts_sync.c) : plus de DATA_IND, donc plus
+     * d'IND CLOCK, et osmo-bts-trx sortait (« No more clock from transceiver »). Deja en
+     * mode BTS : on met a jour les parametres sans toucher aux taches. */
+    static int bts_actif;
+    if (!(bm->enabled && bts_actif))
+        l1a_mftask_set(0);
     if (bm->enabled) {
         int i;

-        mframe_enable(MF_TASK_BTS_SYNC);
-        mframe_enable(MF_TASK_BTS);
+        if (!bts_actif || !(l1s.mframe_sched.tasks_tgt & (1 << MF_TASK_BTS))) {
+            mframe_enable(MF_TASK_BTS_SYNC);
+            mframe_enable(MF_TASK_BTS);
+        }
         for (i = 0; i < 8; i++) {
             l1s.bts.type[i] = bm->type[i];
             l1s.bts.handover[i] = bm->handover[i];
@@ -648,6 +658,7 @@ static int l1ctl_bts_mode(struct msgb *msg)
         mframe_enable(MF_TASK_BCCH_NORM);
     }

+    bts_actif = bm->enabled ? 1 : 0;
     printf("BTS MODE: %u %u {%u,%u,%u,%u,%u,%u,%u,%u} "
         "TX %d..%d RX %d..%d\n", l1s.bts.bsic, l1s.bts.arfcn,
         bm->type[0], bm->type[1], bm->type[2], bm->type[3],
```

```
diff --git a/src/target/firmware/layer1/prim_bts.c b/src/target/firmware/layer1/prim_bts.c
index 91b18dc..02acc7d 100644
--- a/src/target/firmware/layer1/prim_bts.c
+++ b/src/target/firmware/layer1/prim_bts.c
@@ -139,7 +139,13 @@ l1s_bts_resp(uint8_t p1, uint8_t p2, uint16_t p3)
    struct gsm_time rx_time;

    /* Time of the command */
-   gsm_fn2gsmtime(&rx_time, l1s.current_time.fn - 2);
+   /* [2026-10-20] Banc QEMU : BTS_RX_LAG = 3 au lieu de 2. Le burst montant du mobile emule
+    * traverse l'air (anneau -> air.py -> UDP -> BSP) et arrive ~1 trame apres l'instant ou la
+    * fenetre RX de sa trame s'ouvre : 3 bursts sur 4 d'un bloc SDCCH manquaient. On lit donc le
+    * burst de la trame X une trame plus tard et on le marque X (le BSP de c54x_exe le joue a
+    * tick(X) + 1 : CALYPSO_BSP_TRX_DELTA = -2). Silicium : remettre 2. */
+#define BTS_RX_LAG 4   /* [2026-10-20 soir] 4 : deux trames de marge (delta -3 cote c54x_exe) */
+   gsm_fn2gsmtime(&rx_time, l1s.current_time.fn - BTS_RX_LAG);
        /* We're shifted in time since we RX in the 'next' frame */


@@ -165,9 +171,11 @@ l1s_bts_resp(uint8_t p1, uint8_t p2, uint16_t p3)
        bi->tn = l1s.bts.rx_start;

        /* Data (cut to 8 bits */
-       bi->toa = db->rx[1].cmd;
-       if (bi->toa > 68)
+       /* [2026-10-19] la garde testait bi->toa, tronque a 8 bits : 0x7719 passait
+        * (0x19 = 25) et j = 0xee32 sortait de la fenetre API (data abort). */
+       if (db->rx[1].cmd > 68)
            goto exit;
+       bi->toa = db->rx[1].cmd;
        for (i=0,j=(db->rx[1].cmd)<<1; i<2*88; i++,j++)
            bi->iq[i] = iq[j] >> 8;

@@ -261,6 +269,48 @@ l1s_bts_resp(uint8_t p1, uint8_t p2, uint16_t p3)
        }
    }

+   /* [2026-10-20] banc QEMU : le deuxieme intervalle (rx_start + 1), resultat pose par c54x_exe
+    * dans rx[2] / data[72..] (meme format que d[] ci-dessus : toa, pm, -, energie, -, bits) */
+   if (l1s.bts.rx_num > 1 && db->rx[2].cmd == DSP_EXT_RX_CMD_NB)
+   {
+       uint16_t *d = &db->data[72];
+       int rssi = agc_inp_dbm8_by_pm(d[1] >> 3) / 8;
+       int16_t toa = (int16_t)d[0] - 3;
+
+       db->rx[2].cmd = 0;
+       if (d[3] > 0x1000) {
+           struct msgb *msg;
+           struct l1ctl_bts_burst_nb_ind *bi;
+           int i;
+
+           msg = l1ctl_msgb_alloc(L1CTL_BTS_BURST_NB_IND);
+           if (!msg)
+               goto exit;
+
+           bi = (struct l1ctl_bts_burst_nb_ind *) msgb_put(msg, sizeof(*bi));
+           bi->fn = htonl(rx_time.fn);
+           bi->tn = (l1s.bts.rx_start + 1) & 7;
+           if (toa > -32 && toa < 32)
+               bi->toa = toa;
+           if (rssi < -110)
+               bi->rssi = -110;
+           else if (rssi < 0)
+               bi->rssi = rssi;
+           memset(bi->data, 0x00, sizeof(bi->data));
+           for (i=0; i<116; i++)
```

```
+           {
+               int sbit = 0x0008 << ((3 - (i & 3)) << 2);
+               int sword = i >> 2;
+               int dbit = 1 << (7 - (i & 7));
+               int dbyte = i >> 3;
+
+               if (d[5+sword] & sbit)
+                   bi->data[dbyte] |= dbit;
+           }
+           l1_queue_for_l2(msg);
+       }
+   }
+
 exit:
    /* We're done with this */
    dsp_api.r_page_used = 1;
@@ -289,6 +339,15 @@ l1s_bts_cmd(uint8_t p1, uint8_t p2, uint16_t p3)

    db->rx[0].cmd = 0;
    db->rx[0].data = 0x000;
+   /* [2026-10-19] rx[1] (offset du burst dans data[]) n'etait jamais remis a
+    * zero : sur le banc QEMU (c54x_exe) la page garde une copie de code ROM
+    * tant que le DSP n'y a rien ecrit, et l1s_bts_resp indexait data[] avec. */
+   db->rx[1].cmd = 0;
+   db->rx[1].data = 0x000;
+   /* [2026-10-20] banc QEMU : rx[2] est le resultat du DEUXIEME intervalle (rx_start + 1, le TCH),
+    * pose par c54x_exe (prothese montante TS1) dans data[72..] ; remis a zero a chaque trame */
+   db->rx[2].cmd = 0;
+   db->rx[2].data = 0x000;

    t3 = l1s.next_time.t3;

@@ -365,64 +424,69 @@ l1s_bts_cmd(uint8_t p1, uint8_t p2, uint16_t p3)
    /* TX side */
    /* ------- */

-   // FIXME put that in a loop over all TX bursts
-   static uint8_t SLOT, tn;
-   if (l1s.bts.tx_start == 5) {
-       SLOT = 3;
-       tn = 0;
-   } else {
-       SLOT = 0;
-       tn = 1;
-   }
+   /* [2026-10-20] Tous les slots de la fenetre TX, pas un seul : le slot p de la fenetre ouverte
+    * au slot TPU (tx_start + 5) & 7 part sur le TS descendant (tx_start + p) & 7 (le TX du Calypso
+    * est en avance de 3 slots). Sur le banc QEMU c54x_exe ne joue pas la fenetre multiple de la
+    * ROM : il publie lui-meme les bursts a partir de tx[p] (cmd + donnees) de cette page.
+    * Les donnees NB du slot p sont dans data[8 p .. 8 p + 7]. */
+   {
+   int p, nslots = l1s.bts.tx_num ? l1s.bts.tx_num : 1;
+   if (nslots > 8) nslots = 8;

    /* Reset all commands to dummy burst */
    for (i=0; i<8; i++)
        db->tx[i].cmd = DSP_EXT_TX_CMD_DUMMY;

-   /* Get the next burst */
-   type = trx_get_burst(l1s.next_time.fn, tn, data);
-
-   /* Program the TX commands */
-   if (t3 == 2 && (l1s.bts.type[tn] >> 1) != 2) {
-       // FIXME use dummy until TSC will be set individually
-       db->tx[SLOT].cmd = DSP_EXT_TX_CMD_DUMMY;
-   } else
```

```
-   switch (type) {
-   case BURST_FB:
-       db->tx[SLOT].cmd = DSP_EXT_TX_CMD_FB;
-       break;
-
-   case BURST_SB:
-       db->tx[SLOT].cmd = DSP_EXT_TX_CMD_SB;
-
-       sb = sb_build(
-           l1s.bts.bsic,                   /* BSIC */
-           l1s.next_time.t1,               /* T1   */
-           l1s.next_time.t2,               /* T2   */
-           (l1s.next_time.t3 - 1) / 10     /* T3'  */
-       );
-
-       db->sch[0] = sb;
-       db->sch[1] = sb >> 16;
-
-       break;
-
-   case BURST_DUMMY:
-       db->tx[SLOT].cmd = DSP_EXT_TX_CMD_DUMMY;
-       break;
-
-   case BURST_NB:
-       db->tx[SLOT].cmd = DSP_EXT_TX_CMD_NB;
-       db->tx[SLOT].data = sizeof(struct dsp_ext_db) / sizeof(uint16_t);
-
-       for (i=0; i<7; i++)
-           db->data[i] = (data[i<<1] << 8) | data[(i<<1)+1];
-       db->data[7] = data[14] << 8;
-       break;
-
-   default:
-       db->tx[SLOT].cmd = DSP_EXT_TX_CMD_DUMMY;
-       break;
+   for (p = 0; p < nslots; p++) {
+       uint8_t tn = (l1s.bts.tx_start + p) & 7;
+
+       /* Get the next burst */
+       type = trx_get_burst(l1s.next_time.fn, tn, data);
+
+       /* Program the TX commands */
+       if (t3 == 2 && (l1s.bts.type[tn] >> 1) != 2) {
+           // FIXME use dummy until TSC will be set individually
+           db->tx[p].cmd = DSP_EXT_TX_CMD_DUMMY;
+           continue;
+       }
+       switch (type) {
+       case BURST_FB:
+           db->tx[p].cmd = DSP_EXT_TX_CMD_FB;
+           break;
+
+       case BURST_SB:
+           db->tx[p].cmd = DSP_EXT_TX_CMD_SB;
+
+           sb = sb_build(
+               l1s.bts.bsic,                   /* BSIC */
+               l1s.next_time.t1,               /* T1   */
+               l1s.next_time.t2,               /* T2   */
+               (l1s.next_time.t3 - 1) / 10     /* T3'  */
+           );
+
+           db->sch[0] = sb;
+           db->sch[1] = sb >> 16;
+
```

```
+           break;
+
+       case BURST_DUMMY:
+           db->tx[p].cmd = DSP_EXT_TX_CMD_DUMMY;
+           break;
+
+       case BURST_NB:
+           db->tx[p].cmd = DSP_EXT_TX_CMD_NB;
+           db->tx[p].data = sizeof(struct dsp_ext_db) / sizeof(uint16_t) + 8 * p;
+
+           for (i=0; i<7; i++)
+               db->data[8 * p + i] = (data[i<<1] << 8) | data[(i<<1)+1];
+           db->data[8 * p + 7] = data[14] << 8;
+           break;
+
+       default:
+           db->tx[p].cmd = DSP_EXT_TX_CMD_DUMMY;
+           break;
+       }
+   }
    }

    /* Enable the task */
diff --git a/src/target/firmware/layer1/prim_bts_sync.c b/src/target/firmware/layer1/prim_bts_sync.c
index 60cb157..3ea69e5 100644
--- a/src/target/firmware/layer1/prim_bts_sync.c
+++ b/src/target/firmware/layer1/prim_bts_sync.c
@@ -166,17 +166,17 @@ static int l1s_nb_resp(__unused uint8_t p1, __unused uint8_t p2, uint16_t p3)
    rxnb.meas[0].snr = dsp_api.db_r->a_serv_demod[D_SNR];

    /* feed computed frequency error into AFC loop */
-   if (rxnb.meas[0].snr > AFC_SNR_THRESHOLD)
-       afc_input(rxnb.meas[0].freq_err, rf_arfcn, 1);
-   else
-       afc_input(rxnb.meas[0].freq_err, rf_arfcn, 0);
+   /* [2026-10-20] banc QEMU (c54x_exe) : NI AFC NI boucle de TA. Pas de cellule a suivre
+    * (transceiver --no-sync) ; quand un burst du mobile tombait dans cette fenetre TS0
+    * (freq_err=3967 mesure) les deux boucles decalaient frequence et TPU, la tache de
+    * synchro ne rendait plus rien, plus de DATA_IND donc plus d IND CLOCK et osmo-bts-trx
+    * sortait (« No more clock from transceiver »). On garde la mesure et le DATA_IND. */

    /* feed computed TOA into TA loop */
    printf("TOA=%d snr=%d (thres %d) freq_err=%d pm=%d\n",
        rxnb.meas[0].toa_qbit, rxnb.meas[0].snr, 2560,
        rxnb.meas[0].freq_err, rxnb.meas[0].pm_dbm8 / 8);
-   for (i = 0; i < 10; i++)
-       toa_input(rxnb.meas[0].toa_qbit << 2, rxnb.meas[0].snr);
+   (void)i;

    /* Tell the RF frontend to set the gain appropriately */
    rffe_compute_gain(rxnb.meas[0].pm_dbm8/8, CAL_DSP_TGT_BB_LVL);
@@ -225,18 +225,18 @@ static int l1s_nb_resp(__unused uint8_t p1, __unused uint8_t p2, uint16_t p3)
    rxnb.meas[0].snr = d[D_SNR];

    /* feed computed frequency error into AFC loop */
-   if (rxnb.meas[0].snr > AFC_SNR_THRESHOLD)
-       afc_input(rxnb.meas[0].freq_err, rf_arfcn, 1);
-   else
-       afc_input(rxnb.meas[0].freq_err, rf_arfcn, 0);
+   /* [2026-10-20] banc QEMU (c54x_exe) : NI AFC NI boucle de TA. Pas de cellule a suivre
+    * (transceiver --no-sync) ; quand un burst du mobile tombait dans cette fenetre TS0
+    * (freq_err=3967 mesure) les deux boucles decalaient frequence et TPU, la tache de
+    * synchro ne rendait plus rien, plus de DATA_IND donc plus d IND CLOCK et osmo-bts-trx
+    * sortait (« No more clock from transceiver »). On garde la mesure et le DATA_IND. */

    /* feed computed TOA into TA loop */
```

```
    printf("arfcn=%d tn=%d TOA=%d snr=%d (thres %d) freq_err=%d pm=%d\n",
        rf_arfcn, tn,
        rxnb.meas[0].toa_qbit, rxnb.meas[0].snr, 2560,
        rxnb.meas[0].freq_err, rxnb.meas[0].pm_dbm8 / 8);
-   for (i = 0; i < 10; i++)
-       toa_input(rxnb.meas[0].toa_qbit << 2, rxnb.meas[0].snr);
+   (void)i;

    /* Tell the RF frontend to set the gain appropriately */
    rffe_compute_gain(rxnb.meas[0].pm_dbm8/8, CAL_DSP_TGT_BB_LVL);
diff --git a/src/target/firmware/solve_envs.py b/src/target/firmware/solve_envs.py
index d6414d6..da1b0e1 100755
--- a/src/target/firmware/solve_envs.py
+++ b/src/target/firmware/solve_envs.py
@@ -1,4 +1,4 @@
-#!/usr/bin/env python
+#!/usr/bin/env python3

 import sys

@@ -34,7 +34,7 @@ def main(name, board_envs, app_envs):
    envs = solve(board_envs, app_envs)

    # Result
-   print ' '.join(envs)
+   print(' '.join(envs))


 if __name__ == '__main__':
```

2.160 /opt/GSM/osmo-operator/patches/pmaports-linux-postmarketos-stable-ppp.patch

1769 octets, 30 lignes → 30 lignes

```
diff --git a/device/main/linux-postmarketos-stable/APKBUILD b/device/main/linux-postmarketos-stable/APKBUILD
index 4ae7783d82..4146fa5a7b 100644
--- a/device/main/linux-postmarketos-stable/APKBUILD
+++ b/device/main/linux-postmarketos-stable/APKBUILD
@@ -133,5 +133,5 @@ a4fa95d30c48aeca7e082e1a65a7cb296f1d4561d5ef41d6ff22008d30cb7db0f513be3bc9fa8229

 ↪  e44dd245093beb04c3242273440b20bd5d1a43ee6b1cb74231f7cbd6daa942c573e130816dd5cf645c0ecc6186ee9d13048ebe7c1e43d15e0f21ea3f952
 ↪  config-stable.ppc64le

 ↪  25dc70e3cddf21d95e3267a8fb59804d77eb90f3eac55b81a5a267e9300edf12ca6603a5ac57a7f1b35780275dac23851e003a0ebfda7c2ce5de448819c
 ↪  config-stable.riscv64

 ↪  85b208a8c6d40080f050bb5fdfc63c20b0aa3beabad3166f82aed57441afee58996aa36793cb662bf669eac71efd6f2e91f29433faf3dfa642dab4c50e6
 ↪  config-stable.x86
-
 ↪  e50bea8bcefd1d25d64f662376799c50f6553a7c4881b041751c48d71756494a4506713aca92d1dd08645a69c296a1889784355424be2486be1089181b9
 ↪  config-stable.x86_64
]
 ↪  +1335c280137718cd2b38f6e588efbf7f2af55c1b7efc82ec603372fa950863e0a78dce855ba3d210a2594ee80db9cba0076e964d94239386f04417a9ae
 ↪  config-stable.x86_64
 "
diff --git a/device/main/linux-postmarketos-stable/config-stable.x86_64
 ↪  b/device/main/linux-postmarketos-stable/config-stable.x86_64
index 653356b9de..ca0c576c81 100644
--- a/device/main/linux-postmarketos-stable/config-stable.x86_64
+++ b/device/main/linux-postmarketos-stable/config-stable.x86_64
@@ -2841,7 +2841,13 @@ CONFIG_ACPI_MDIO=y
 # CONFIG_PCS_XPCS is not set
 # end of PCS device drivers
```

```
-# CONFIG_PPP is not set
+CONFIG_PPP=m
+CONFIG_PPP_BSDCOMP=m
+CONFIG_PPP_DEFLATE=m
+CONFIG_PPP_FILTER=y
+CONFIG_PPP_MULTILINK=y
+CONFIG_PPP_ASYNC=m
+CONFIG_PPP_SYNC_TTY=m
 # CONFIG_SLIP is not set
 CONFIG_USB_NET_DRIVERS=y
 # CONFIG_USB_CATC is not set
```

2.161 /opt/GSM/osmo-operator/patches/pmbootstrap-osmo-bench-qemu.patch

16351 octets, 341 lignes → 341 lignes

```
diff --git a/pmb/chroot/shutdown.py b/pmb/chroot/shutdown.py
index b49aaf7f..a3e307b8 100644
--- a/pmb/chroot/shutdown.py
+++ b/pmb/chroot/shutdown.py
@@ -19,6 +19,11 @@ def kill_adb() -> None:
     with closing(socket.socket(socket.AF_INET, socket.SOCK_STREAM)) as sock:
         sock.settimeout(2)
         if sock.connect_ex(("127.0.0.1", port)) == 0:
+            # [2026-09-07] osmo-operator: on this bench 127.0.0.1:5038 is
+            # Asterisk's AMI, not an adb server, and the native chroot has no
+            # adb - the kill would fail with 127 and abort every build.
+            if not (Chroot.native() / "usr/bin/adb").exists():
+                return
             pmb.chroot.root(["adb", "-P", str(port), "kill-server"])


diff --git a/pmb/commands/qemu.py b/pmb/commands/qemu.py
index 72b46c60..d2aadc99 100644
--- a/pmb/commands/qemu.py
+++ b/pmb/commands/qemu.py
@@ -61,6 +61,34 @@ def which_qemu(arch: Arch) -> str:
         )


+def _host_gpu_vendors() -> set[str]:
+    """PCI vendor ids of the host's DRM cards ("0x1002" = AMD, "0x10de" = NVIDIA)."""
+    vendors: set[str] = set()
+    drm = "/sys/class/drm"
+    try:
+        for card in os.listdir(drm):
+            vp = os.path.join(drm, card, "device", "vendor")
+            if card.startswith("card") and os.path.isfile(vp):
+                with open(vp) as fh:
+                    vendors.add(fh.read().strip().lower())
+    except OSError:
+        pass
+    return vendors
+
+
+def _host_cpu_has_virt() -> bool:
+    """True when the CPU exposes VT-x (vmx) or AMD-V (svm) - i.e. it is enabled in the firmware."""
+    try:
+        with open("/proc/cpuinfo") as fh:
+            for line in fh:
+                if line.startswith("flags"):
+                    flags = line.split(":", 1)[1].split()
+                    return "vmx" in flags or "svm" in flags
```

```
+    except OSError:
+        pass
+    return False
+
+
 def command_qemu(
     config: Config,
     arch: Arch,
@@ -160,7 +188,12 @@ def command_qemu(
     else:
         command += ["stdio"]

-    command += ["-drive", f"file={img_path},format=raw,if=virtio"]
+    # [2026-09-09] bootindex=0 on the root disk: with a fresh NVRAM, OVMF still
+    # ranks its own "UEFI PXEv4" (built-in network stack, no option ROM needed)
+    # ahead of the disk and spends a while in PXE before booting. QEMU's
+    # fw_cfg boot order, which OVMF honours, puts the disk first.
+    command += ["-drive", f"file={img_path},format=raw,if=none,id=osmo-root"]
+    command += ["-device", "virtio-blk-pci,drive=osmo-root,bootindex=0"]
     if img_path_2nd:
         command += ["-drive", f"file={img_path_2nd}" + ",format=raw,if=virtio"]

@@ -169,8 +202,23 @@ def command_qemu(
     else:
         command += ["-device", "virtio-mouse-pci"]
     command += ["-device", "virtio-keyboard-pci"]
-    command += ["-netdev", f"user,id=net,hostfwd=tcp:127.0.0.1:{port_ssh}-:22"]
-    command += ["-device", "virtio-net-pci,netdev=net"]
+    command += [
+        "-netdev",
+        # Plan d'adressage explicite du reseau utilisateur (slirp). Sans ces
+        # options QEMU prend des valeurs implicites et l'invite peut se
+        # retrouver sans bail DHCP : plus d'Internet ET plus de SSH, car le
+        # hostfwd n'a alors aucune IP invite ou livrer le trafic.
+        #   net=       le sous-reseau vu par l'invite
+        #   host=      la passerelle (QEMU lui-meme)
+        #   dhcpstart= la premiere IP distribuee -> l'invite est toujours .15
+        #   dns=       le resolveur interne de QEMU, qui relaie vers l'hote
+        f"user,id=net,net=10.0.2.0/24,host=10.0.2.2,dhcpstart=10.0.2.15,"
+        f"dns=10.0.2.3,hostfwd=tcp:127.0.0.1:{port_ssh}-:22",
+    ]
+    # romfile= drops the card's PXE option ROM. With it, a freshly created
+    # NVRAM ranks "UEFI PXEv4" ahead of the disk: the firmware runs iPXE,
+    # fails to find a server, and only then boots - a detour on every start.
+    command += ["-device", "virtio-net-pci,netdev=net,romfile="]

     # Check that arch is supported by pmb qemu
     if arch not in [
@@ -200,13 +248,48 @@ def command_qemu(
         case _:
             pass

+    # [2026-09-08] OPENGL ONLY WHEN IT CAN WORK. virtio-vga-gl + gl=on hands the
+    # guest's rendering to the host's Mesa as seen from this musl chroot. On an
+    # NVIDIA host there is no driver: EGL fails, QEMU falls back to software
+    # rendering and only warns. On an AMD host Mesa DOES find radeonsi, virgl
+    # engages for real against a libdrm/LLVM that do not match the host, and
+    # QEMU aborts or segfaults at launch - the usual "crashes on AMD".
+    # OSMO_PMOS_GL=0 turns GL off (virtio-vga, gl=off) ; unset, it is off on
+    # AMD hosts and on hosts without /dev/dri ; OSMO_PMOS_GL=1 forces it on.
+    gl_env = os.environ.get("OSMO_PMOS_GL", "")
+    if gl_env == "0":
+        use_qemu_gl = False
+    elif gl_env != "1" and use_qemu_gl:
+        vendors = _host_gpu_vendors()
+        if "0x1002" in vendors:
```

```
+            logging.info("osmo: AMD GPU on the host - QEMU without OpenGL (OSMO_PMOS_GL=1 to force it)")
+            use_qemu_gl = False
+        elif not os.path.exists("/dev/dri"):
+            logging.info("osmo: no /dev/dri on the host - QEMU without OpenGL")
+            use_qemu_gl = False
+
     # Configure display
     if qemu_display != "none":
+        # [2026-09-07] The screen's shape decides the phone's shape. Phosh lays
+        # itself out from the screen it is given: tall and narrow is a phone,
+        # wide is a tablet - same image, same session, two form factors. QEMU's
+        # virtio card takes that size as xres/yres, and the guest boots
+        # straight into it, so it is picked at launch and never changes under
+        # the running session. OSMO_PMOS_RES="<width>x<height>", empty or
+        # unset for whatever the card would pick on its own.
+        res = os.environ.get("OSMO_PMOS_RES", "")
+        geom = ""
+        if res:
+            try:
+                xres, yres = (int(v) for v in res.lower().split("x", 1))
+                geom = f",xres={xres},yres={yres}"
+            except ValueError:
+                logging.warning(f"WARNING: OSMO_PMOS_RES={res} is not <width>x<height>, ignored")
         match arch:
             case Arch.x86_64:
-                command += ["-device", "virtio-vga-gl"]
+                command += ["-device", f"{'virtio-vga-gl' if use_qemu_gl else 'virtio-vga'}{geom}"]
             case Arch.aarch64 | Arch.armv7:
-                command += ["-device", "virtio-gpu-gl"]
+                command += ["-device", "virtio-gpu-gl" if use_qemu_gl else "virtio-gpu-pci"]
             case Arch.riscv64 | Arch.ppc64le | Arch.loongarch64:
                 command += ["-device", "virtio-gpu-pci"]
             case _:
@@ -221,10 +304,22 @@ def command_qemu(
                 f"if=pflash,format=raw,readonly=on,file={edk2_chroot}/usr/share/AAVMF/AAVMF_CODE.fd",
             ]
         case Arch.x86_64:
+            # OVMF.fd on its own is read-only, so the firmware has nowhere to
+            # write its variables: it records no boot entry and drops into its
+            # setup menu instead of booting. Give it the code read-only and a
+            # writable copy of the variables, one per device.
+            ovmf = edk2_chroot / "usr/share/OVMF"
+            nvram = Chroot.native() / "home/pmos/rootfs" / f"{config.device}-ovmf-vars.fd"
+            if not nvram.exists():
+                pmb.helpers.run.root(["cp", ovmf / "OVMF_VARS.fd", nvram])
+            # QEMU runs as the local user, the copy is owned by root.
+            if not os.access(nvram, os.W_OK):
+                pmb.helpers.run.root(["chmod", "666", nvram])
             command += [
                 "-drive",
-                f"if=pflash,format=raw,readonly=on,file={edk2_chroot}/usr/share/OVMF/OVMF.fd",
+                f"if=pflash,unit=0,format=raw,readonly=on,file={ovmf}/OVMF_CODE.fd",
             ]
+            command += ["-drive", f"if=pflash,unit=1,format=raw,file={nvram}"]
         case Arch.loongarch64:
             command += [
                 "-drive",
@@ -243,11 +338,29 @@ def command_qemu(

     # Kernel Virtual Machine (KVM) support
     native = pmb.parse.deviceinfo().arch.is_native()
-    if use_qemu_kvm and native and os.path.exists("/dev/kvm"):
+    # [2026-09-08] KVM ONLY WHEN IT REALLY WORKS. /dev/kvm can exist while the
+    # CPU's virtualisation is off in the firmware (SVM on AMD boards ships off
+    # more often than not) or the node is not writable by this user : QEMU then
+    # dies at once ("Could not access KVM kernel module", "failed to initialize
```

```
+    # kvm: Operation not supported"). We check the flag and the access first,
+    # and fall back to TCG with -cpu max (the guest still boots, slower).
+    # OSMO_PMOS_KVM=0 forces TCG.
+    kvm_ok = (
+        use_qemu_kvm
+        and native
+        and os.environ.get("OSMO_PMOS_KVM", "1") != "0"
+        and os.access("/dev/kvm", os.R_OK | os.W_OK)
+        and _host_cpu_has_virt()
+    )
+    if kvm_ok:
         command += ["-enable-kvm"]
         command += ["-cpu", "host"]
     else:
+        if use_qemu_kvm and native and os.path.exists("/dev/kvm"):
+            logging.info("osmo: /dev/kvm is there but unusable (virtualisation off in the firmware ? not writable
 ↪  ?) - TCG")
         logging.info("WARNING: QEMU is not using KVM and will run slower!")
+        if arch == Arch.x86_64 and not qemu_cpu:
+            command += ["-cpu", "max"]

     if qemu_cpu:
         command += ["-cpu", qemu_cpu]
@@ -255,6 +368,19 @@ def command_qemu(
     display = qemu_display
     if display != "none":
         display += ",show-cursor=on,gl=" + ("on" if use_qemu_gl else "off")
+    # [2026-10-14] A GTK WINDOW THAT FITS THE SCREEN. The phone is 720x1440 and
+    # most monitors are 1080 high: without zoom-to-fit the GTK drawing area
+    # asks for the full guest size, the window manager clamps the window and
+    # the bottom of the phone is simply cut off. zoom-to-fit scales the frame
+    # (and the pointer with it) to whatever size the window gets. GTK is also
+    # the only display that leaves the host GPU alone when GL is off: the SDL
+    # display loads this chroot's Mesa and opens /dev/dri/renderD128 whatever
+    # its renderer, which is where the on-screen artefacts came from.
+    # OSMO_PMOS_GTK_OPTS replaces the extra options (empty: none).
+    if display.startswith("gtk"):
+        gtk_opts = os.environ.get("OSMO_PMOS_GTK_OPTS", "zoom-to-fit=on")
+        if gtk_opts:
+            display += "," + gtk_opts

     # Separate -show-cursor option is deprecated. If your host qemu fails here,
     # it's old (#1995).
@@ -264,9 +390,119 @@ def command_qemu(
     if qemu_audio:
         command += ["-audio", f"{qemu_audio},model=hda"]

+    command += osmo_bench_args(qemu_audio)
+
     return (command, env)


+def osmo_bench_args(qemu_audio: str | None) -> list[str]:
+    """Extra devices that put the VM on the osmocom bench: the modem, and the
+    voice bridge.
+
+    The modem (tools/osmo-phonesim-banc.py) is a PCI serial port, not ISA and
+    not USB. The UEFI firmware treats an ISA serial port as a console and
+    mistakes the modem's replies for keypresses, so it never boots; usb-serial
+    would need ftdi_sio, which this kernel doesn't have. 8250_pci is built in,
+    so the port shows up on its own as /dev/ttyS1, which is what ModemManager
+    probes. QEMU only listens here - the modem connects afterwards, once the
+    firmware is done booting (see osmo-pmos-setup).
+
+    There are TWO sound cards, because QEMU cannot push audio into a guest's
+    speaker: of an emulated card, playback goes out to a host sink and capture
```

```
+    comes in from a host source. The call's downlink lives on the host, so it
+    can only enter through the guest's capture side - never through its
+    speaker. One card alone therefore gives the guest the network's point of
+    view (it hears the far end on its microphone), not a handset's.
+
+    So: the "bridge" card is wired to the bench's voice path - gsm_audio
+    carries the downlink (RTP from osmo-mgw), so its monitor feeds that card's
+    capture; gsm_mic carries the uplink, so it takes what that card plays. The
+    "handset" card stays on the host's default devices, the real speakers and
+    microphone. Crossing the two inside the guest (see osmo-pmos-setup) puts
+    the VM in the middle of the voice path for real.
+
+    The slots are pinned so the guest can tell them apart by name alone:
+    0000:00:12.0 is the bridge, 0000:00:13.0 the handset.
+
+    Beware the pa backend: it looks for PulseAudio's pidfile, which doesn't
+    exist when PipeWire provides the service - there is only the socket, so we
+    name it explicitly. And an unknown backend doesn't error out, it segfaults;
+    this qemu only knows none, alsa, pa, sdl and wav.
+    """
+    port = os.environ.get("OSMO_PMOS_AT_PORT", "12346")
+    args: list[str] = [
+        "-chardev",
+        f"socket,id=gsmchr,host=127.0.0.1,port={port},server=on,wait=off",
+    ]
+    # [2026-09-08] virtio-serial, not a 16550. QEMU paces its emulated 16550
+    # at the programmed baud rate (115200 -> ~11 KB/s each way), so PPP data
+    # over the modem port crawled and every page filled a minute of queue.
+    # A virtconsole port has no baud rate: it shows up as /dev/hvc0 in the
+    # guest, a tty like any other, and ModemManager probes it thanks to the
+    # udev rule osmo-pmos-setup installs. OSMO_PMOS_SERIAL=pci brings the
+    # 16550 (ttyS1) back.
+    if os.environ.get("OSMO_PMOS_SERIAL", "virtio") == "pci":
+        args += ["-device", "pci-serial,chardev=gsmchr"]
+    else:
+        # Two ports, like a real modem: hvc0 is the command port (AT, URCs,
+        # calls, SMS), hvc1 the data port where ModemManager dials ATD*99 and
+        # runs PPP - so the phone keeps an AT channel while data is up. The
+        # data port listens on OSMO_PMOS_AT_PORT + 1 (12347 by default).
+        port_data = int(port) + 1
+        args += [
+            "-chardev",
+            f"socket,id=gsmdata,host=127.0.0.1,port={port_data},server=on,wait=off",
+            "-device",
+            "virtio-serial-pci,id=osmovser",
+            "-device",
+            "virtconsole,chardev=gsmchr,name=osmo.modem",
+            "-device",
+            "virtconsole,chardev=gsmdata,name=osmo.data",
+        ]
+
+    # --audio wins if it was asked for explicitly: it adds a card of its own.
+    if qemu_audio:
+        return args
+
+    sock = Path(os.environ.get("XDG_RUNTIME_DIR", f"/run/user/{os.getuid()}")) / "pulse/native"
+    if not sock.is_socket():
+        logging.warning(f"WARNING: no PulseAudio socket ({sock}), VM without sound")
+        return args
+
+    out = os.environ.get("OSMO_PMOS_SINK", "gsm_mic")
+    src = os.environ.get("OSMO_PMOS_SOURCE", "gsm_audio.monitor")
+    # [2026-09-09] The handset card is NAMED too, when the launcher says so:
+    # left to "the host's defaults", it landed on whatever was default when
+    # QEMU connected - the raw card, since the echo canceller (osmo_hp_ec /
+    # osmo_mic_ec) only came later with osmo-pmos-setup - and then had to be
```

```
+    # moved. osmo-pmos-qemu now loads the canceller first and passes its
+    # names here (OSMO_PMOS_HP / OSMO_PMOS_MIC); unset, the defaults apply.
+    combine = f"pa,id=combine,server={sock}"
+    hp = os.environ.get("OSMO_PMOS_HP", "")
+    mic = os.environ.get("OSMO_PMOS_MIC", "")
+    if hp:
+        combine += f",out.name={hp}"
+    if mic:
+        combine += f",in.name={mic}"
+    args += [
+        # The bridge: the call itself.
+        "-audiodev",
+        f"pa,id=gsm,server={sock},out.name={out},in.name={src}",
+        "-device",
+        "intel-hda,id=hdagsm,addr=0x12",
+        "-device",
+        "hda-duplex,bus=hdagsm.0,audiodev=gsm",
+        # The handset: the host's own speakers and microphone.
+        "-audiodev",
+        combine,
+        "-device",
+        "intel-hda,id=hdacombine,addr=0x13",
+        "-device",
+        "hda-duplex,bus=hdacombine.0,audiodev=combine",
+    ]
+    return args
+
+
 def resize_image(img_size_new: str, img_path: Path) -> None:
     """
     Truncates an image to a specific size. The value must be larger than the
```

2.162 /opt/GSM/osmo-operator/pont/init.py

2663 octets, 76 lignes → 76 lignes

```
import gc
import logging
import signal
import sys
import threading
import time

from . import config as _config
from .cipher import Cipher
from .downlink import Downlink, Feeder
from .gsm import load_timeslots, plan_for
from .record import Recorder
from .state import Dedicated, Tch
from .stats import Reporter, Stats
from .trx import Clock, Transmitter, Trx
from .uplink import TchScheduler, Uplink

log = logging.getLogger("pont")


def _guarded(name, fn):
    def run():
        while True:
            try:
                fn()
                log.warning("thread %s termine", name)
                return
            except Exception:
```

```
                log.exception("thread %s : exception, relance dans 1 s", name)
                time.sleep(1)
    return threading.Thread(target=run, name=name, daemon=True)


def main(argv=None):
    cfg = _config.parse(argv)
    logging.basicConfig(stream=sys.stdout, level=logging.INFO,
                        format="%(asctime)s [pont] %(message)s", datefmt="%H:%M:%S")
    gc.disable()
    gc.freeze()

    stats = Stats()
    clock = Clock()
    cipher = Cipher(cfg.kc_retention, stats)
    dedicated = Dedicated()
    tch = Tch(stats)
    record = Recorder(cfg, clock) if cfg.record else None
    trx = Trx(cfg, clock, stats, cipher, record, dedicated, tch)
    transmitter = Transmitter(cfg, clock, trx, stats)
    feeder = Feeder(cfg, stats)
    timeslots = load_timeslots(cfg.bsc_cfg)
    downlink = Downlink(cfg, stats, cipher, dedicated, tch, feeder, record, timeslots)
    uplink = Uplink(cfg, clock, stats, dedicated, tch, transmitter, feeder, cipher)
    scheduler = TchScheduler(cfg, clock, stats, tch, uplink, transmitter)

    log.info("pont TRX : ports %d/%d/%d, ARFCN %d, BSIC %d, avance UL %d trames",
             cfg.trx_base, cfg.trx_base + 1, cfg.trx_base + 2, cfg.arfcn, cfg.bsic, cfg.ul_fn_advance)
    for tn in sorted(timeslots):
        plan = plan_for(timeslots[tn])
        log.info("TS%d %-12s %s", tn, timeslots[tn], ("plan %s, %d sous-voies" % (plan.name, plan.n_sub)) if plan
↪   else "")
    log.info("QEMU : GSMTAP udp/%d, SCH udp/%d ; tap GSMTAP %s", cfg.gsmtap_port, cfg.sch_port,
             ("udp/%d" % cfg.tap_port) if cfg.tap else "coupe")

    threads = [
        _guarded("ctrl", trx.run_ctrl),
        _guarded("clock", trx.run_clock),
        _guarded("data", lambda: trx.run_data(downlink.dispatch)),
        transmitter, uplink, scheduler,
        Reporter(stats, clock, dedicated, record),
    ]
    if record:
        signal.signal(signal.SIGUSR1, record.split)
        threads.append(record)
    for t in threads:
        t.start()
    while True:
        time.sleep(3600)
```

2.163 /opt/GSM/osmo-operator/pont/air.py

19191 octets, 388 lignes → 388 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""air.py - the air between two emulated Calypsos: the bursts one emits, the other receives.

Two-instance setup (c54x_exe/run.sh INSTANCE=2): instance 1 is the TRX of BTS#0 (trx
firmware, start-native.sh --osmo-trx), instance 2 is the mobile.
Each c54x_exe publishes the final bursts its ROM emits into a ring
/dev/shm/calypso_tx_rom (pont.c, tx_tsp_publier): header w(4) slots(4), then 160-byte
slots: seq(4) fn(4) type(1) tsc(1) offset(1) src(1) bits(148). Each BSP (calypso_bsp.c)
expects the bursts it must receive as TRXDv0 over UDP (6702, CALYPSO_BSP_PORT):
```

```
tn(1) fn(4, big-endian) rssi(1) toa(2) bits(148, one 0/1 byte per bit).

This relay reads A's ring and sends it to B's BSP, and B's ring to A's BSP: the downlink
(FB, SB, dummy, NB from the TRX) goes to the mobile, the uplink (RACH, NB from the mobile)
goes to the TRX. It starts from the end of the rings (no replay) and counts what passes
every 5 s.

  python3 air.py --a-ring /dev/shm/calypso_tx_rom --a-port 6702 \\
                 --b-ring /dev/shm/calypso2/calypso_tx_rom --b-port 6712
From instance 2 (run.sh, private mount namespace) the ring of instance 1 is read at
/run/calypso-shm-hote/calypso_tx_rom.

Third Calypso, phone 2 of the TRX (TS1, instance 3: --c-ring / --c-port): its bursts go
to the mobile with tn=1; the mobile's bursts go to the TS0 TRX (A) while it is on the SDCCH,
to the TS1 TRX (C) once the ASSIGNMENT COMMAND has gone through. The air reads that
ASSIGNMENT COMMAND in the SDCCH/4 blocks of TS0 (pont.gsm, libosmocoding) and announces it
to the mobile's c54x_exe through --tch-cfg (pont/state.py format: seq, tn, tsc, arfcn);
a RACH from the mobile gives the SDCCH back (tn=0).

Phone 2 has its OWN frame counter (it starts from 0 at its boot, like phone 1): its bursts
leave its ring with its local fn, and its BSP expects uplink bursts in its own numbering.
The host transceiver measures D = fn1 - fn2 on the DATA_IND of both phones and writes it
to --fn-delta (/tmp/osmocom_fn_delta_2); the air tags C->B with fn + D and sends B->C with
fn - D, in tn 0 (phone 2's BSP files a tn=1 burst in the "other" slots, rx[2], which its
firmware does not read: for it TS1 is ITS slot). While D is unknown, nothing goes through C.
"""
import argparse
import os
import socket
import struct
import sys
import time

# The bridge's block decoder (libosmocoding via ctypes), to read the ASSIGNMENT COMMAND.
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
try:
    from pont import gsm
except Exception as _e:   # pragma: no cover - without libosmocoding the air still relays, without TCH
    gsm = None
    print("air : pont.gsm indisponible (%s) : pas de detection de l'ASSIGNMENT COMMAND" % _e, flush=True)

HDR = 8
SLOT = 160
TYPES = {0: "?", 1: "RACH", 2: "NB", 3: "SB", 4: "DUMMY", 5: "FB", 101: "TS1", 102: "TS2", 103: "TS3", 104: "TS4",
 ↪  105: "TS5", 106: "TS6", 107: "TS7"}


class Anneau:
    """A calypso_tx_rom ring; only what is written after our arrival is read."""

    def __init__(self, path, nom):
        self.path, self.nom = path, nom
        self.fd = -1
        self.seq = None          # last seq read
        self.slots = 256
        self.rate = 0
        # fn CONTINUITY. In TRX mode the published fn is the tick of the emitting c54x_exe
        # (the trx firmware does not set a_a5fn): it restarts from 0 when banc-trx is relaunched,
        # while the mobile's BSP (STREAM: one burst per frame, in fn order) expects a monotonic
        # sequence. We therefore relay fn + offset, the offset being realigned on every fn
        # step back so that the numbering seen by the receiver stays monotonic and continuous
        # (+1 from the last relayed fn).
        self.fn_dec = 0
        self.fn_dernier = None
        self.recalages = 0
```

```
def fn_continu(self, fn):
    out = (fn + self.fn_dec) % 2715648
    if self.fn_dernier is not None:
        d = (out - self.fn_dernier) % 2715648
       if d > 100000:                # step back (or absurd jump): realign after the last; d == 0 = same frame
            self.fn_dec = (self.fn_dernier + 1 - fn) % 2715648
            out = (fn + self.fn_dec) % 2715648
            self.recalages += 1
            print("air : anneau %s : fn recule (%d apres %d), decalage recale a +%d (%d recalages)"
                  % (self.nom, fn, self.fn_dernier, self.fn_dec, self.recalages), flush=True)
    self.fn_dernier = out
    return out

def _ouvrir(self):
    if self.fd >= 0:
        return True
    try:
        self.fd = os.open(self.path, os.O_RDONLY)
    except OSError:
        return False
    hdr = os.pread(self.fd, HDR, 0)
    if len(hdr) < HDR:
        os.close(self.fd); self.fd = -1
        return False
    w, slots = struct.unpack("<II", hdr)
    self.slots = slots or 256
    self.seq = w                        # start from the end
    print("air : anneau %s ouvert (%s), %d cases, seq=%d" % (self.nom, self.path, self.slots, w), flush=True)
    return True

def nouveaux(self):
    """Return the slots (seq, fn, type, tsc, offset, src, bits) written since the last call."""
    if not self._ouvrir():
        return []
    # When the other instance is relaunched (./start-native.sh --bts while the mobile runs),
    # its c54x_exe RECREATES the ring (new inode) and the old descriptor would read a deleted
    # file forever (A->B frozen, no more downlink for the mobile). Compare the inode periodically.
    self.tours = getattr(self, "tours", 0) + 1
    if self.tours % 200 == 0:
        try:
            if os.stat(self.path).st_ino != os.fstat(self.fd).st_ino:
                print("air : anneau %s recree (%s) : reouverture" % (self.nom, self.path), flush=True)
                os.close(self.fd); self.fd = -1
                if not self._ouvrir():
                    return []
        except OSError:
            pass
    try:
        hdr = os.pread(self.fd, HDR, 0)
    except OSError:
        os.close(self.fd); self.fd = -1
        return []
    if len(hdr) < HDR:
        return []
    w = struct.unpack("<I", hdr[:4])[0]
    if w == self.seq:
        return []
    if w < self.seq:                    # ring recreated (new run)
        print("air : anneau %s remis a zero (seq %d -> %d)" % (self.nom, self.seq, w), flush=True)
        self.seq = 0
    debut = self.seq + 1
    if w - debut >= self.slots:         # too far behind: jump to the most recent
        self.rate += (w - debut) - (self.slots - 1)
        debut = w - (self.slots - 1)
    out = []
    for s in range(debut, w + 1):
```

```
            off = HDR + ((s - 1) % self.slots) * SLOT
            try:
                case = os.pread(self.fd, SLOT, off)
            except OSError:
                break
            if len(case) < SLOT:
                break
            seq, fn = struct.unpack("<II", case[:8])
            typ, tsc, offset, src = case[8], case[9], case[10], case[11]
            if seq != s:                    # slot not written yet / overwritten
                continue
            out.append((seq, fn, typ, tsc, offset, src, case[12:160]))
        self.seq = w
        return out


class Cible:
    """The other instance's BSP, as TRXDv0 over UDP."""

    def __init__(self, host, port, nom, tn):
        self.addr, self.nom, self.tn = (host, port), nom, tn
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.envoyes = 0
        self.par_type = {}

    def envoyer(self, fn, typ, bits, tn=None):
        """tn: the timeslot announced in the TRXD header; by default the target's. The rings
        carry the burst's TS (bits 4-6 of src): it is relayed as is."""
        if tn is None:
            tn = self.tn
        pkt = bytes([tn & 7]) + struct.pack(">I", fn) + b"\x00\x00\x00" + bytes(b & 1 for b in bits)
        try:
            self.sock.sendto(pkt, self.addr)
        except OSError as e:
            print("air : envoi vers %s %s:%d : %s" % (self.nom, self.addr[0], self.addr[1], e), flush=True)
            return
        self.envoyes += 1
        self.par_type[typ] = self.par_type.get(typ, 0) + 1
        if tn:
            self.par_type[100 + tn] = self.par_type.get(100 + tn, 0) + 1   # count per timeslot (TS1..7)


class Decalage:
    """D = fn1 - fn2, published by the transceiver (--fn-delta-file); re-read when the file changes."""

    def __init__(self, chemin):
        self.chemin, self.d, self.mtime, self.prochain = chemin, None, None, 0.0
        self.changements = 0

    def lire(self):
        now = time.monotonic()
        if now < self.prochain or not self.chemin:
            return self.d
        self.prochain = now + 0.5
        try:
            st = os.stat(self.chemin)
        except OSError:
            return self.d
        if st.st_mtime == self.mtime and self.d is not None:
            return self.d
        try:
            d = int(open(self.chemin).read().strip())
        except (OSError, ValueError):
            return self.d
        self.mtime = st.st_mtime
        if d != self.d:
```

```
            self.changements += 1
            print("air : decalage de trames du telephone 2 : fn1 - fn2 = %d (%s, %d changements)" % (d,
             ↪  self.chemin, self.changements), flush=True)
            self.d = d
        return self.d


SDCCH4_DL = {22: 0, 23: 0, 24: 0, 25: 0, 26: 1, 27: 1, 28: 1, 29: 1, 32: 2, 33: 2, 34: 2, 35: 2, 36: 3, 37: 3, 38:
 ↪  3, 39: 3}


class Dedie:
    """The ASSIGNMENT COMMAND read on the SDCCH/4 of TS0, and the TCH announcement to the
    mobile (calypso_tch_cfg)."""

    def __init__(self, chemin):
        self.chemin = chemin
        self.blocs = {}          # subchannel -> [(fn, bits)]
        self.tn = None
        self.tsc = 7
        self.seq = 0
        self.decodes = 0
        self.rates = 0

    def _ecrire(self, tn, tsc, arfcn):
        if not self.chemin:
            return
        self.seq += 1
        buf = bytearray(16)
        buf[0:4] = int(self.seq).to_bytes(4, "little")
        buf[4] = tn & 0xff
        buf[5] = tsc & 0xff
        buf[6:8] = int(arfcn).to_bytes(2, "little")
        fd = os.open(self.chemin, os.O_CREAT | os.O_WRONLY, 0o644)
        try:
            os.pwrite(fd, bytes(buf), 0)
        finally:
            os.close(fd)

    def descendant(self, fn, typ, bits):
        """A TS0 burst from the TRX: accumulate SDCCH/4 blocks and look for the ASSIGNMENT COMMAND."""
        if gsm is None or typ != 2:
            return
        ss = SDCCH4_DL.get(fn % 51)
        if ss is None:
            return
        acc = self.blocs.setdefault(ss, [])
        if acc and (fn - acc[-1][0]) % 2715648 != 1:
            acc.clear()
        acc.append((fn, bytes(b & 1 for b in bits)))
        if len(acc) < 4:
            return
        quatre = acc[:]; acc.clear()
        try:
            l2 = gsm.xcch_decode([gsm.coded_from_burst(b) for _, b in quatre])
        except Exception as e:
            print("air : decodage SDCCH : %s" % e, flush=True); return
        if l2 is None:
            self.rates += 1; return
        self.decodes += 1
        off, mt = gsm.rr_message_type(l2)
        if mt == gsm.RR_ASSIGNMENT_COMMAND and len(l2) > off + 4:
            b0, b1, b2 = l2[off + 2], l2[off + 3], l2[off + 4]
            if (b1 >> 4) & 1:
                print("air : ASSIGNMENT COMMAND avec saut de frequence (fn=%u) : ignore" % fn, flush=True); return
            tn, tsc, arfcn = b0 & 0x07, (b1 >> 5) & 0x07, ((b1 & 0x03) << 8) | b2
```

```
            self.tn, self.tsc = tn, tsc
            self._ecrire(tn, tsc, arfcn)
            print("air : ASSIGNMENT COMMAND (fn=%u, SDCCH/4 ss%d) : TCH TS%d TSC=%d ARFCN=%d -> annonce %s (seq
             ↪  %d), montant du mobile vers le TRX du TS%d"
                  % (fn, ss, tn, tsc, arfcn, self.chemin, self.seq, tn), flush=True)
        elif mt == gsm.RR_CHANNEL_RELEASE and self.tn is not None:
            self.liberer("CHANNEL RELEASE sur le SDCCH")

    def liberer(self, raison):
        if self.tn is None:
            return
        tn = self.tn; self.tn = None
        self._ecrire(0, 0, 0)
        print("air : TCH TS%d rendu (%s) : montant du mobile vers le TRX du TS0 (seq %d)" % (tn, raison,
         ↪  self.seq), flush=True)


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--a-ring", required=True, help="anneau TX de A, le TRX du TS0 (ce que A emet)")
    ap.add_argument("--a-port", type=int, required=True, help="port UDP du BSP de A (ce que A recoit)")
    ap.add_argument("--b-ring", required=True, help="anneau TX de B, le mobile")
    ap.add_argument("--b-port", type=int, required=True, help="port UDP du BSP de B")
    ap.add_argument("--c-ring", default=None, help="anneau TX de C, le telephone 2 du TRX (TS1), optionnel")
    ap.add_argument("--c-port", type=int, default=None, help="port UDP du BSP de C")
    ap.add_argument("--tch-cfg", default=None, help="annonce du TCH au c54x_exe du mobile (calypso_tch_cfg de son
 ↪  /dev/shm)")
    ap.add_argument("--fn-delta", default="/tmp/osmocom_fn_delta_2", help="fichier ou le transceiver publie fn1 -
 ↪  fn2 du telephone 2 (avec --c-ring)")
    ap.add_argument("--c-toujours", action="store_true", help="relayer C->B meme hors TCH (defaut : seulement
 ↪  pendant un TCH assigne, les bursts TS1 dans la trame des SB cassaient la synchro du mobile le 21/10)")
    ap.add_argument("--c-tn", type=int, default=0, help="tn de l'en-tete TRXD des bursts du mobile envoyes au
 ↪  telephone 2 (0 : son intervalle principal)")
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--tn", type=int, default=0, help="timeslot annonce dans l'en-tete TRXD des bursts de A
 ↪  (defaut 0)")
    ap.add_argument("--b-fn-delta", type=int, default=int(os.environ.get("AIR_B_FN_DELTA", "0") or 0),
                    help="[21/10] trames ajoutees au fn des bursts du mobile B (defaut 0 : le fn publie par
                     ↪  c54x_exe est "
                          "le bon ; l'ecart de 2 vu le 21/10 venait d'une comparaison au RA_R de la L1 TI). Env
                           ↪  AIR_B_FN_DELTA")
    ap.add_argument("--sans", default="", help="types a ne pas relayer, ex. « ?,RACH »")
    ap.add_argument("--periode", type=float, default=0.002, help="periode de scrutation (s)")
    ap.add_argument("--stats", type=float, default=5.0, help="periode des comptes (s)")
    args = ap.parse_args()
    noms = {v: k for k, v in TYPES.items()}
    sans = {noms[t.strip()] for t in args.sans.split(",") if t.strip() in noms}

    a_ring, b_ring = Anneau(args.a_ring, "A"), Anneau(args.b_ring, "B")
    vers_b = Cible(args.host, args.b_port, "B", args.tn)   # what A emits -> B receives (TS0)
    vers_a = Cible(args.host, args.a_port, "A", args.tn)   # what B emits -> A receives
    c_ring = Anneau(args.c_ring, "C") if args.c_ring else None
    vers_b1 = Cible(args.host, args.b_port, "B(TS1)", 1) if c_ring else None   # what C emits -> B receives, tn 1
    vers_c = Cible(args.host, args.c_port, "C", 1) if (c_ring and args.c_port) else None
    dedie = Dedie(args.tch_cfg)
    decalage = Decalage(args.fn_delta if c_ring else None)
    c_sans_d = 0
    mobile_tch = 0.0          # time of the mobile's last burst on a dedicated TS (tn_m != 0):
                              # with a ciphered SDCCH the air cannot read the ASSIGNMENT COMMAND,
                              # the mobile itself tells it is on the TCH
    print("air : A %s -> udp %d (B) ; B %s -> udp %d (A) ; tn %d%s%s%s" %
          (args.a_ring, args.b_port, args.b_ring, args.a_port, args.tn,
           " ; C %s -> udp %d (B, tn 1) ; B -> udp %s (C) sur TCH" % (args.c_ring, args.b_port, args.c_port) if
            ↪  c_ring else "",
           " ; annonce TCH : %s" % args.tch_cfg if args.tch_cfg else "",
           " ; sans %s" % args.sans if sans else "")
```

```
          + (" ; fn de B %+d" % args.b_fn_delta if args.b_fn_delta else ""), flush=True)
    prochain = time.monotonic() + args.stats
    while True:
        bouge = False
        for seq, fn, typ, tsc, offset, src, bits in a_ring.nouveaux():
            bouge = True
            if typ in sans or offset == 0xff and typ == 0:
                continue
            fnc = a_ring.fn_continu(fn)
            tn_b = (src >> 4) & 7                       # the burst's TS (TS0: BCCH/CCCH/SDCCH, TS1: the TCH)
            vers_b.envoyer(fnc, typ, bits, tn_b)
            if tn_b == 0:
                dedie.descendant(fnc, typ, bits)
        if c_ring:
            d = decalage.lire()
            for seq, fn, typ, tsc, offset, src, bits in c_ring.nouveaux():
                bouge = True
                if typ in sans or offset == 0xff and typ == 0:
                    continue
                if d is None or (dedie.tn is None and not args.c_toujours and time.monotonic() - mobile_tch > 2.0):
                    c_sans_d += 1
                    continue
                vers_b1.envoyer((fn + d) % 2715648, typ, bits)
        for seq, fn, typ, tsc, offset, src, bits in b_ring.nouveaux():
            bouge = True
            if typ in sans or offset == 0xff and typ == 0:
                continue
            fnc = (b_ring.fn_continu(fn) + args.b_fn_delta) % 2715648
            tn_m = (src >> 4) & 7                       # the TS the mobile transmitted on (montant_tn_dedie)
            if typ == 1:                                    # RACH: the mobile is (back) on TS0
                if dedie.tn is not None:
                    dedie.liberer("RACH du mobile")
                vers_a.envoyer(fnc, typ, bits, 0)
            elif tn_m and vers_c is not None:
                mobile_tch = time.monotonic()
                d = decalage.lire()
                if d is None:
                    c_sans_d += 1
                    continue
                vers_c.envoyer((fnc - d) % 2715648, typ, bits, args.c_tn)   # TS1 TCH -> TRX phone 2, ITS numbering
            else:
                vers_a.envoyer(fnc, typ, bits, tn_m)       # single TRX phone: TS0 and TS1 by tn
        if not bouge:
            time.sleep(args.periode)
        if time.monotonic() >= prochain:
            prochain += args.stats
            def fmt(c):
                return " ".join("%s=%d" % (TYPES.get(t, str(t)), n) for t, n in sorted(c.par_type.items())) or "-"
            print("air : A->B %d (%s)  B->A %d (%s)%s  rates A %d B %d%s  SDCCH blocs %d/%d%s" %
                  (vers_b.envoyes, fmt(vers_b), vers_a.envoyes, fmt(vers_a),
                   "  C->B %d (%s)  B->C %d (%s)" % (vers_b1.envoyes, fmt(vers_b1), vers_c.envoyes if vers_c else
                    ↪  0, fmt(vers_c) if vers_c else "-") if c_ring else "",
                   a_ring.rate, b_ring.rate, " C %d" % c_ring.rate if c_ring else "",
                  dedie.decodes, dedie.decodes + dedie.rates, "  TCH TS%d" % dedie.tn if dedie.tn is not None else
↪  "")
                  + ("  D=%s%s" % (decalage.d, " (sans D : %d perdus)" % c_sans_d if c_sans_d else "") if c_ring
                   ↪  else ""), flush=True)


if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        pass
```

2.164 /opt/GSM/osmo-operator/pont/airmesh.py

28663 octets, 583 lignes → 583 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
#
# airmesh.py - the mesh is made ON THE BURSTS, not on the calls.
#
# ──────────────────────────────────────────────────────────────────────────────
# WHAT IT DOES
#
# What crosses the network is the burst. The radio medium becomes shared: the
# BTS of node 2 transmits, the mobile of node 1 hears it, synchronises on it,
# reads its SI, and does its location update FOR REAL. Everything else - BSC,
# MSC, HLR, MGW, Asterisk - stays enclosed in its node. Only the air is shared.
#
# ──────────────────────────────────────────────────────────────────────────────
# WHO DOES WHAT: QEMU emits the bursts, fake_trx translates, airmesh transports
#
# The bursts are not made here. They come from the Calypso running in QEMU:
# the firmware modulates, pont.py collects what the ARM drops in its sidebands,
# encodes it into bursts and puts it on TRXD. In the other direction it decodes
# the BTS downlink and hands it back to the firmware. THAT is the radio.
#
# fake_trx.py is the node TRANSLATOR: a burst switch linking N transceivers and
# distributing what one EMITS to all those whose RECEIVE frequency matches
# (burst_fwd.py, forward_msg). It makes nothing, it routes.
#
# airmesh is the transport between nodes. It plugs into the translator, takes
# what passes there, sends it to the peers, and injects what comes back.
#
#     ┌──────────────────────────── one node ────────────────────────────┐
#     │   QEMU/Calypso ──sidebands── pont.py ──TRXD 5700──┐             │
#     │   osmo-bts-trx ──────────────────────────TRXD 5700──┤             │
#     │   trxcon/mobile ─────────────────────────TRXD 6700──┤ fake        │
#     │   airmesh(mobile side) ──────────────────TRXD 6706──┤ _trx        │
#     │   airmesh(BTS side) ─────────────────────TRXD 6709──┘             │
#     └──────────────────────────────┬──────────────────────────────────┘
#                                    │  UDP AIRM (default 4739)
#                 ┌──────────────────┴──────────────────┐
#              node 2                                node 3
#
# WHY TWO ATTACHMENTS. fake_trx pairs on the FREQUENCY: a transceiver only
# hears what is emitted on its receive frequency. A single attachment point
# would hear only one direction, and a mobile that does not hear the BTS never
# attaches to it.
#   - mobile side: rx = downlink, tx = uplink. It HEARS what the local BTS
#     emits, and INJECTS the uplink of the remote mobiles towards it.
#   - BTS side   : rx = uplink, tx = downlink. It HEARS what the local mobiles
#     emit, and INJECTS the downlink of the remote BTSs towards them.
#
# ──────────────────────────────────────────────────────────────────────────────
# HOW IT ATTACHES WITHOUT CHANGING A LINE OF QEMU-SRC
#
# fake_trx creates one more transceiver per declared mobile (60-fake-trx.sh:
# --trx ue<n>@127.0.0.<g>:<port>, port = 6700 + (n-1) * 3). Those transceivers
# are "mobiles" only by name: fake_trx knows neither role nor band for them
# until they have sent RXTUNE and TXTUNE.
#
# So TWO more mobiles than the bench has are declared - N_MS + 2 - and airmesh
# takes their two places. It tunes the first like a mobile and the second the
# other way round, like a BTS. No change to the launcher, no qosmo-grgsm file
# touched: two slots already provided for, used differently.
#
# WHAT TO KNOW WHEN READING THEM: `pgrep -a fake_trx` shows two mobiles that do
```

```
# not exist, and the fake_trx log announces ue3 and ue4. This is intended. The
# name comes from the launcher.
#
# ──────────────────────────────────────────────────────────────────────────
# THE THREE DIFFICULTIES, AND WHAT IS DONE ABOUT EACH
#
# 1. THE CLOCK. A TDMA frame lasts 4.615 ms and carries a 22-bit number (FN)
#    that wraps every 3 h 28. Two nodes have two unrelated clocks: a burst
#    tagged fn=12345 on one designates no instant on the other.
#    WHAT IS DONE: OFFSET tracking. airmesh continuously measures the
#    difference between the local FN and the FN announced by each peer, and
#    REWRITES the FN at injection. Both clocks advance at the same nominal rate
#    (a 4.615 ms software timer on both sides), so the offset is almost
#    constant and the drift is caught up at every received burst.
#    WHAT WOULD BE BETTER: a master node that BROADCASTS the FN, the others
#    having no clock of their own. That requires taking over clck_gen; the
#    offset is enough as long as the nodes are on the same LAN. The function
#    _fn_offset() is the only place to replace.
#
# 2. THE DELAY. A burst must arrive BEFORE the local scheduler needs it. It is
#    therefore injected LOOKAHEAD frames ahead (default 6, i.e. ~28 ms): a LAN
#    round trip fits easily, a link with 30 ms latency does not. Bursts that
#    arrive too late are COUNTED, never silently dropped - a radio mesh losing
#    5 % of its bursts shows in the statistics, not in the BSC logs.
#
# 3. WHO HEARS WHOM. Without a graph, every mobile would hear every BTS of the
#    bench at full power: no cell, no reselection, nothing left to show.
#    data/air-mesh.txt gives the attenuation between nodes, in dB. Below a
#    floor, the burst is not transmitted - that is what makes a MESH and not a
#    bus.
#
# ──────────────────────────────────────────────────────────────────────────
# THE TWO MODES (AIRMESH_MODE)
#
#   fake  (default) - the burst goes through as is. Pure switching: no
#                     demodulation, no channel coding, the bit train comes out
#                     identical. Exact and cheap.
#   pont            - the burst goes through the Calypso chain: demodulation to
#                     L2 (libosmocoding), handed to QEMU through the sidebands,
#                     and what the ARM answers is re-modulated into bursts
#                     before leaving again on the mesh. pont.py holds that
#                     chain; airmesh hands over through a FIFO and does not
#                     touch it.
#
# ──────────────────────────────────────────────────────────────────────────
# WHAT IS NOT VALIDATED
#
# This file is written against the formats and port conventions read in
# osmocom-bb (trx_toolkit: transceiver.py, data_msg.py, burst_fwd.py) and
# against the TRXDv0 implemented on both sides by pont.py. The part to prove
# first is the clock offset tracking (point 1), the only one resting on an
# assumption - that both clck_gen advance at the same rate. The counters are
# there for that: "hors_fenetre" rising means point 1 or point 2 gives way.

import argparse, os, socket, struct, sys, threading, time, collections

# ── TRXD v0, as osmocom-bb writes it ──────────────────────────────────────
# Common header: byte 0 = (version << 4) | slot number, then FN on 4 bytes
# big-endian.  (data_msg.py, parse_hdr / append_hdr_to)
#   TxMsg (L1 -> transceiver): common header + pwr(1)            = 6 bytes
#   RxMsg (transceiver -> L1): common header + rssi(1) + toa(2)  = 8 bytes
# Then 148 bytes, one per bit (hard on transmit, soft on receive).
TRXD_VER      = 0
CHDR_LEN      = 5
TX_HDR_LEN    = CHDR_LEN + 1
RX_HDR_LEN    = CHDR_LEN + 3
```

```
BURST_LEN     = 148

# ── Our protocol between nodes ──────────────────────────────────────
# Deliberately flat and readable in tcpdump: a radio mesh is debugged by
# watching the packets go by, not by re-reading a serialisation schema.
#   magic(4) ver(1) dir(1) node(2) arfcn(2) tn(1) reserved(1)
#   fn(4) pwr(1) rssi(1) toa(2 signed) nbits(2)   = 22 bytes, then nbits bytes
AIRM_MAGIC    = b"AIRM"
AIRM_VER      = 1
AIRM_HDR      = ">4sBBHHBBIbbhH"
AIRM_HDR_LEN  = struct.calcsize(AIRM_HDR)
DIR_DL        = 0      # what a BTS emits
DIR_UL        = 1      # what a mobile emits

FN_MAX        = 2715648          # 26 * 51 * 2048, the GSM FN wrap-around
FRAME_S       = 4.615e-3


def arfcn_to_khz(arfcn, uplink=False):
    """Frequency in kHz. GSM900 / DCS1800, the two bands of the bench.

    fake_trx does not pair on the ARFCN but on the FREQUENCY: getting the band
    wrong gives no error, it gives a transceiver that simply never hears
    anything - the longest failure to track down.
    """
    if 512 <= arfcn <= 885:                     # DCS 1800
        dl = 1805200 + 200 * (arfcn - 512)
        return dl - 95000 if uplink else dl
    if 0 <= arfcn <= 124:                       # GSM 900
        dl = 935000 + 200 * arfcn
        return dl - 45000 if uplink else dl
    if 975 <= arfcn <= 1023:                    # E-GSM 900
        dl = 935000 + 200 * (arfcn - 1024)
        return dl - 45000 if uplink else dl
    raise ValueError("ARFCN hors des bandes connues : %d" % arfcn)


class Stats:
    """What to look at when the mesh "half works"."""
    def __init__(self):
        self.lock = threading.Lock()
        self.c = collections.Counter()

    def bump(self, k, n=1):
        with self.lock:
            self.c[k] += n

    def render(self):
        with self.lock:
            return " ".join("%s=%d" % (k, v) for k, v in sorted(self.c.items()))


ST = Stats()


class Topologie:
    """Who hears whom, and at what cost.

    data/air-mesh.txt, one edge per line:
        <node a> <node b> <attenuation dB> [uni]
    Without the word "uni" the edge holds in both directions. A pair absent
    from the file does not hear each other: it is a mesh, not a bus - two nodes
    only see each other if someone wrote it.
    """

    def __init__(self, chemin, defaut_db):
```

```
        self.aretes = {}
        self.defaut = defaut_db
        self.ouvert = False
        if chemin and os.path.isfile(chemin):
            self._charger(chemin)
            self.ouvert = True

    def _charger(self, chemin):
        with open(chemin, "r", encoding="utf-8", errors="replace") as f:
            for ligne in f:
                ligne = ligne.split("#", 1)[0].strip()
                if not ligne:
                    continue
                ch = ligne.split()
                if len(ch) < 3:
                    continue
                try:
                    a, b, att = int(ch[0]), int(ch[1]), float(ch[2])
                except ValueError:
                    continue
                self.aretes[(a, b)] = att
                if len(ch) < 4 or ch[3] != "uni":
                    self.aretes[(b, a)] = att

    def attenuation(self, src, dst):
        """dB between two nodes, or None if one does not hear the other.

        Without a file, everybody hears everybody at the default attenuation:
        that is the behaviour of a bus, useful for a first two-node trial, and
        it is ANNOUNCED at start-up so that it is not taken for a topology.
        """
        if not self.ouvert:
            return self.defaut
        return self.aretes.get((src, dst))


class TrxLien:
    """An L1 as seen from fake_trx: CTRL and DATA, exactly like trxcon.

    Port conventions read in transceiver.py (Transceiver.__init__): the
    transceiver BINDS base+1 (CTRL) and base+2 (DATA); the L1 binds base+101
    and base+102 and sends to the former. We are on the L1 side.
    """

    def __init__(self, nom, host, base, arfcn, rx_uplink):
        self.nom  = nom
        self.host = host
        self.base = base
        self.arfcn = arfcn
        self.rx_uplink = rx_uplink        # True: we listen to the uplink
        self.ctrl = self._sock(base + 101)
        self.data = self._sock(base + 102)
        self.ctrl_seq = 0
        self.slots = set()

    @staticmethod
    def _sock(port):
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind(("127.0.0.1", port))
        return s

    # ── CTRL ──────────────────────────────────────────────────────────
    def cmd(self, texte, attente=1.0):
        """Send a CTRL command and wait for its answer.
```

```
    The answer is AWAITED rather than firing the commands back to back:
    fake_trx answers RSP <cmd> <code>, and a non-zero code (frequency out
    of band, slot refused) would otherwise go unnoticed. A mistuned
    transceiver emits no error afterwards - it simply stays silent.
    """
    self.ctrl.sendto(("CMD " + texte + "\0").encode(), (self.host, self.base + 1))
    self.ctrl.settimeout(attente)
    try:
        rep, _ = self.ctrl.recvfrom(512)
    except socket.timeout:
        return None
    finally:
        self.ctrl.settimeout(None)
    return rep.rstrip(b"\0").decode(errors="replace")

def accorder(self, slots):
    """Full POWERON: tune both frequencies then open the slots.

    The order matters. fake_trx refuses SETSLOT on a transceiver that has
    no frequency, and POWERON on a transceiver without a slot is useless:
    it runs and hears nothing.
    """
    rx = arfcn_to_khz(self.arfcn, uplink=self.rx_uplink)
    tx = arfcn_to_khz(self.arfcn, uplink=not self.rx_uplink)
    for cmd in ("RXTUNE %d" % rx, "TXTUNE %d" % tx):
        rep = self.cmd(cmd)
        if rep is None or " 0" not in rep:
            raise RuntimeError("%s : %s refuse (%s)" % (self.nom, cmd, rep))
    for tn in slots:
        # type 1 = TCH/F for fake_trx; what matters here is that the slot
        # is ACTIVE, the real content is carried by the bursts themselves.
        self.cmd("SETSLOT %d 1" % tn)
        self.slots.add(tn)
    rep = self.cmd("POWERON")
    if rep is None or " 0" not in rep:
        raise RuntimeError("%s : POWERON refuse (%s)" % (self.nom, rep))

def eteindre(self):
    try:
        self.cmd("POWEROFF", attente=0.3)
    except OSError:
        pass

# -- DATA ---------------------------------------------------------------
def lire(self):
    """A burst fake_trx delivers to us: what the local neighbour EMITTED."""
    paquet, _ = self.data.recvfrom(1024)
    if len(paquet) < RX_HDR_LEN + BURST_LEN:
        ST.bump("rx_court")
        return None
    ver = paquet[0] >> 4
    if ver != TRXD_VER:
        ST.bump("rx_version_inattendue")
        return None
    tn   = paquet[0] & 0x0F
    fn   = struct.unpack(">L", paquet[1:5])[0]
    rssi = -paquet[5]
    toa  = struct.unpack(">h", paquet[6:8])[0]
    bits = paquet[RX_HDR_LEN:RX_HDR_LEN + BURST_LEN]
    return tn, fn, rssi, toa, bits

def ecrire(self, tn, fn, pwr, bits):
    """Inject a burst: fake_trx distributes it to whoever is tuned to it."""
    hdr = bytearray(TX_HDR_LEN)
    hdr[0] = (TRXD_VER << 4) | (tn & 0x0F)
    hdr[1:5] = struct.pack(">L", fn % FN_MAX)
```

```
        hdr[5] = pwr & 0xFF
        self.data.sendto(bytes(hdr) + bytes(bits), (self.host, self.base + 2))


class Maillage:
    def __init__(self, cfg):
        self.cfg = cfg
        self.noeud = cfg.node
        self.topo = Topologie(cfg.topologie, cfg.attenuation_defaut)
        self.pairs = cfg.pairs                      # [(id, ip, port)]
        self.stop = threading.Event()

        # The clock: last local FN seen, and estimated offset per peer.
        self._fn_local = 0
        self._fn_vu_a  = time.monotonic()
        self._ecart    = {}                         # peer id -> offset in frames
        self._verrou   = threading.Lock()

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.sock.bind((cfg.bind, cfg.port))

        # Mobile side: hears the local downlink, injects the remote uplink.
        self.ms  = TrxLien("airmesh-ms",  "127.0.0.1", cfg.trx_ms,
                           cfg.arfcn, rx_uplink=False)
        # BTS side: hears the local uplink, injects the remote downlink.
        self.bts = TrxLien("airmesh-bts", "127.0.0.1", cfg.trx_bts,
                           cfg.arfcn, rx_uplink=True)

    # ── Clock ──────────────────────────────────────────────────────────
    def _note_fn_local(self, fn):
        with self._verrou:
            self._fn_local = fn
            self._fn_vu_a  = time.monotonic()

    def _fn_maintenant(self):
        """The local FN, extrapolated from the last burst seen.

        We have no clock of our own: the only reference FN is the one fake_trx
        delivers with each burst. Between two bursts we advance by the watch -
        4.615 ms per frame. Enough for a six-frame lookahead; it would not be
        enough to timestamp anything.
        """
        with self._verrou:
            base, quand = self._fn_local, self._fn_vu_a
        return (base + int((time.monotonic() - quand) / FRAME_S)) % FN_MAX

    def _fn_offset(self, pair, fn_distant):
        """Offset between a peer's clock and ours, smoothed.

        This is THE fragile point of the mesh, and the only place to replace
        the day a master node broadcasts the FN: here, we only assume that both
        clocks advance at the same rate.
        """
        vu = (self._fn_maintenant() - fn_distant) % FN_MAX
        with self._verrou:
            ancien = self._ecart.get(pair)
            if ancien is None:
                self._ecart[pair] = vu
                return vu
            # Running average on the circular offset: without the wrap, an FN
            # wrap-around would make the offset jump by two million frames and
            # the mesh would go silent for three hours.
            d = (vu - ancien + FN_MAX // 2) % FN_MAX - FN_MAX // 2
            neuf = (ancien + d // 8) % FN_MAX
            self._ecart[pair] = neuf
```

```
        return neuf

# ── Sending to the peers ──────────────────────────────────────────
def _diffuser(self, sens, tn, fn, rssi, toa, bits):
    for (pid, ip, port) in self.pairs:
        att = self.topo.attenuation(self.noeud, pid)
        if att is None:
            ST.bump("hors_portee")
            continue
        r = max(-120, min(-1, int(rssi - att)))
        if r <= self.cfg.plancher:
            ST.bump("sous_plancher")
            continue
        paquet = struct.pack(AIRM_HDR, AIRM_MAGIC, AIRM_VER, sens,
                             self.noeud, self.cfg.arfcn, tn, 0,
                             fn % FN_MAX, 0, r, toa, len(bits)) + bytes(bits)
        try:
            self.sock.sendto(paquet, (ip, port))
            ST.bump("emis_dl" if sens == DIR_DL else "emis_ul")
        except OSError:
            ST.bump("emission_echec")

def _boucle_locale(self, lien, sens):
    """What the LOCAL neighbour emits goes to the peers."""
    while not self.stop.is_set():
        try:
            lu = lien.lire()
        except OSError:
            break
        if lu is None:
            continue
        tn, fn, rssi, toa, bits = lu
        self._note_fn_local(fn)
        self._diffuser(sens, tn, fn, rssi, toa, bits)

# ── Receiving from the peers ──────────────────────────────────────
def _boucle_maillage(self):
    while not self.stop.is_set():
        try:
            paquet, _ = self.sock.recvfrom(2048)
        except OSError:
            break
        if len(paquet) < AIRM_HDR_LEN:
            ST.bump("mesh_court")
            continue
        (magie, ver, sens, src, arfcn, tn, _res,
         fn, _pwr, rssi, toa, nbits) = struct.unpack(AIRM_HDR, paquet[:AIRM_HDR_LEN])
        if magie != AIRM_MAGIC or ver != AIRM_VER:
            ST.bump("mesh_etranger")
            continue
        bits = paquet[AIRM_HDR_LEN:AIRM_HDR_LEN + nbits]
        if len(bits) != nbits or nbits != BURST_LEN:
            ST.bump("mesh_tronque")
            continue
        if arfcn != self.cfg.arfcn:
            # A peer on another carrier: normal, and even the point - two
            # neighbouring operators do not occupy the same channel. Not
            # counted as an error.
            ST.bump("autre_arfcn")
            continue

        # Where this burst falls in OUR time.
        fn_local = (fn + self._fn_offset(src, fn)) % FN_MAX
        cible    = (self._fn_maintenant() + self.cfg.lookahead) % FN_MAX
        retard   = (cible - fn_local + FN_MAX // 2) % FN_MAX - FN_MAX // 2
        if retard > self.cfg.tolerance:
```

```
            ST.bump("hors_fenetre_tard")
            continue
        fn_inj = cible

        if sens == DIR_DL:
            # A remote downlink: a BTS from elsewhere. Injected through the
            # BTS side, the one whose tx the local mobiles listen to.
            self.bts.ecrire(tn, fn_inj, self._pwr(rssi), bits)
            ST.bump("injecte_dl")
        else:
            self.ms.ecrire(tn, fn_inj, self._pwr(rssi), bits)
            ST.bump("injecte_ul")

@staticmethod
def _pwr(rssi):
    """TxMsg carries an attenuation (0 = full power), not an RSSI.

    The attenuation already applied by the topology is carried over,
    bounded to a byte: this is what gives the mobile a reason to prefer
    one cell over another.
    """
    return max(0, min(255, -rssi - 40))

# ── Process life ─────────────────────────────────────────────────────
def _boucle_stats(self):
    while not self.stop.wait(self.cfg.stats_s):
        ecarts = " ".join("n%d:%+d" % (k, v if v < FN_MAX // 2 else v - FN_MAX)
                          for k, v in sorted(self._ecart.items()))
        print("[airmesh] fn=%d %s | ecarts %s" %
              (self._fn_maintenant(), ST.render(), ecarts or "aucun"),
              flush=True)

def demarrer(self):
    print("[airmesh] noeud %d  arfcn %d  pairs %s" %
          (self.noeud, self.cfg.arfcn,
           ", ".join("%d@%s:%d" % p for p in self.pairs) or "aucun"), flush=True)
    if not self.topo.ouvert:
        print("[airmesh] AUCUNE topologie (%s) : tous les pairs s'entendent "
              "a %.0f dB - c'est un bus, pas un maillage."
              % (self.cfg.topologie, self.cfg.attenuation_defaut), flush=True)
    if self.cfg.mode != "fake":
        print("[airmesh] mode '%s' : les bursts passent par la chaine "
              "Calypso de pont.py (FIFO %s)" % (self.cfg.mode, self.cfg.fifo),
              flush=True)

    slots = [int(x) for x in self.cfg.slots.split(",") if x != ""]
    self.ms.accorder(slots)
    self.bts.accorder(slots)

    fils = [
        threading.Thread(target=self._boucle_locale, args=(self.ms, DIR_DL),
                         name="local-dl", daemon=True),
        threading.Thread(target=self._boucle_locale, args=(self.bts, DIR_UL),
                         name="local-ul", daemon=True),
        threading.Thread(target=self._boucle_maillage, name="maillage", daemon=True),
        threading.Thread(target=self._boucle_stats, name="stats", daemon=True),
    ]
    for f in fils:
        f.start()
    try:
        while not self.stop.is_set():
            time.sleep(0.5)
    except KeyboardInterrupt:
        pass
    finally:
        self.stop.set()
```

```
            self.ms.eteindre()
            self.bts.eteindre()
            print("[airmesh] arret. %s" % ST.render(), flush=True)


def paire(txt):
    """--pair 2:192.168.1.12:4739"""
    ch = txt.split(":")
    if len(ch) != 3:
        raise argparse.ArgumentTypeError("attendu <noeud>:<ip>:<port>, recu %r" % txt)
    return (int(ch[0]), ch[1], int(ch[2]))


def main():
    p = argparse.ArgumentParser(
        prog="airmesh",
        description="Maillage de bursts GSM entre noeuds : le milieu radio devient commun.")
    p.add_argument("--node", type=int, required=True, help="numero de CE noeud")
    p.add_argument("--arfcn", type=int, default=int(os.environ.get("AIRMESH_ARFCN", "514")))
    p.add_argument("--pair", dest="pairs", action="append", type=paire, default=[],
                   metavar="NOEUD:IP:PORT", help="un pair du maillage (repetable)")
    p.add_argument("--bind", default=os.environ.get("AIRMESH_BIND", "0.0.0.0"))
    p.add_argument("--port", type=int, default=int(os.environ.get("AIRMESH_PORT", "4739")))
    # 6700 + (n-1)*3: the two mobile slots following the real ones.
    # With two real MS, the bench must run with N_MS=4 and airmesh takes 3 and 4.
    p.add_argument("--trx-ms", type=int, default=int(os.environ.get("AIRMESH_TRX_MS", "6706")),
                   help="base TRXD de l'emplacement pris cote mobile (defaut : MS 3)")
    p.add_argument("--trx-bts", type=int, default=int(os.environ.get("AIRMESH_TRX_BTS", "6709")),
                   help="base TRXD de l'emplacement pris cote BTS (defaut : MS 4)")
    p.add_argument("--slots", default=os.environ.get("AIRMESH_SLOTS", "0,1,2,3,4,5,6,7"))
    p.add_argument("--lookahead", type=int, default=int(os.environ.get("AIRMESH_LOOKAHEAD", "6")),
                   help="trames d'avance a l'injection (6 = ~28 ms)")
    p.add_argument("--tolerance", type=int, default=int(os.environ.get("AIRMESH_TOL", "2")),
                   help="trames de retard tolerees avant de compter le burst perdu")
    p.add_argument("--topologie", default=os.environ.get("AIRMESH_TOPO", "data/air-mesh.txt"))
    p.add_argument("--attenuation-defaut", type=float,
                   default=float(os.environ.get("AIRMESH_ATT", "20")))
    p.add_argument("--plancher", type=int, default=int(os.environ.get("AIRMESH_PLANCHER", "-110")),
                   help="dBm sous lesquels le burst n'est pas transmis")
    # fake: the burst goes through as is, airmesh is only a transport.
    # pont: it goes through the Calypso chain of pont.py - demodulation, handed
    # to QEMU, remodulation - before leaving again. This is the mode that
    # really makes the firmware transmit on behalf of the mesh.
    p.add_argument("--mode", choices=("fake", "pont"),
                   default=os.environ.get("AIRMESH_MODE", "fake"))
    p.add_argument("--fifo", default=os.environ.get("AIRMESH_FIFO", "/dev/shm/airmesh_pont"))
    p.add_argument("--stats-s", type=float, default=float(os.environ.get("AIRMESH_STATS", "10")))
    cfg = p.parse_args()

    try:
        Maillage(cfg).demarrer()
    except RuntimeError as e:
        print("[airmesh] %s" % e, file=sys.stderr)
        return 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

2.165 /opt/GSM/osmo-operator/pont/cipher.py

2445 octets, 86 lignes → 86 lignes

```
import logging
import os
import struct
import threading
import time

from . import gsm

log = logging.getLogger("pont")

KC_PATH = "/dev/shm/calypso_kc_l1"
KC_TTL = 0.1


class Cipher:
    def __init__(self, retention, stats):
        self.retention = retention
        self.stats = stats
        self._lock = threading.Lock()
        self._fd = None
        self._cache = None
        self._next = 0.0
        self._held = None
        # The BTS only ciphers the downlink after receiving the first ciphered
        # uplink frame (osmo-bts l1sap.c check_for_first_ciphrd). Until that is
        # observed, the downlink of the dedicated channel is in clear.
        self.dl_active = False

    def _read(self):
        now = time.monotonic()
        if now < self._next:
            return self._cache
        self._next = now + KC_TTL
        try:
            if self._fd is None:
                self._fd = os.open(KC_PATH, os.O_RDONLY)
            b = os.pread(self._fd, 32, 0)
        except OSError:
            if self._fd is not None:
                os.close(self._fd)
                self._fd = None
            self._cache = None
            return None
        if len(b) < 14:
            self._cache = None
            return None
        algo = b[4]
        kc = b[6:14]
        self._cache = (algo, kc) if 1 <= algo <= 3 and any(kc) else None
        return self._cache

    def current(self):
        with self._lock:
            key = self._read()
            if key:
                self._held = key
                return key
            if self.retention and self._held:
                return self._held
            return None

    def release(self, reason):
        with self._lock:
            if self._held is None:
                return
            self._held = None
```

```
            self.dl_active = False
        log.info("Kc lache (%s) : le lien repart en clair", reason)

    def confirm_dl(self, fn):
        with self._lock:
            if self.dl_active:
                return
            self.dl_active = True
        log.info("chiffrement descendant confirme par la BTS (fn=%u)", fn)

    def apply(self, burst, fn, uplink):
        key = self.current()
        if key is None:
            return burst
        dl, ul = gsm.a5_keystream(key[0], key[1], fn)
        if uplink:
            self.stats.a5_ul += 1
        else:
            self.stats.a5_dl += 1
        return gsm.a5_xor(burst, ul if uplink else dl)
```

2.166 /opt/GSM/osmo-operator/pont/config.py

3715 octets, 92 lignes → 92 lignes

```
import argparse
import os
from dataclasses import dataclass


def _env(name, default):
    return os.environ.get(name, default)


def _flag(name, default):
    return _env(name, default) not in ("0", "", "no")


@dataclass(frozen=True)
class Config:
    trx_bind: str
    trx_base: int
    dsp_port: int          # >0: copy every DL burst to the DSP (BSP UDP, 6702)
    arfcn: int
    bsic: int
    ul_fn_advance: int
    window_tol: int
    window_essais: int
    ul_retard_max: int
    rssi: int
    gsmtap_port: int
    sch_port: int
    tap: bool
    tap_port: int
    bsc_cfg: str
    clock_period: int
    kc_retention: bool
    record: bool
    record_ul: bool
    record_dl_path: str
    record_ul_path: str
    record_dl_fifo: str
    record_ul_fifo: str
    record_max_mb: int
```

```
    record_min_free_mb: int
    record_osr: int


def parse(argv=None):
    p = argparse.ArgumentParser(prog="pont", description="Pont TRX entre le BTS et la couche 1 gr-gsm de QEMU")
    p.add_argument("--arfcn", type=int, default=int(_env("PONT_ARFCN", "514")))
    p.add_argument("--bsic", type=int, default=int(_env("PONT_BSIC", "7")))
    p.add_argument("--trx-bind", default=_env("PONT_TRX_BIND", "127.0.0.1"))
    p.add_argument("--trx-base", type=int, default=int(_env("PONT_TRX_BASE", "5700")))
    p.add_argument("--dsp-port", type=int, default=int(_env("PONT_DSP_PORT", "0")),
                   help="port UDP du BSP du DSP C54x (c54x_exe --arm / qosmo-dsp) : 6702 ; 0 = coupe")
    p.add_argument("--bsc-cfg", default=_env("PONT_BSC_CFG", "/etc/osmocom/osmo-bsc.cfg"))
    p.add_argument("--no-tap", action="store_true", default=not _flag("PONT_TAP", "1"))
    p.add_argument("--no-record", action="store_true", default=not _flag("PONT_AIRREC", "1"))
    a = p.parse_args(argv)
    return Config(
        trx_bind=a.trx_bind,
        trx_base=a.trx_base,
        dsp_port=a.dsp_port,
        arfcn=a.arfcn,
        bsic=a.bsic,
        ul_fn_advance=int(_env("PONT_UL_FN_ADVANCE", "3")),
        window_tol=int(_env("PONT_WINDOW_TOL", "1")),
        # How many times to wait again for an uplink burst woken up too early
        # (clock slaved to the DSP, see trx.py Transmitter.run).
        window_essais=int(_env("PONT_WINDOW_ESSAIS", "12")),
        # Maximum lateness (frames) of an uplink burst still sent to the BTS,
        # see trx.py Transmitter.run. 0 = drop any late burst (package
        # default; pont_dsp.py, the DSP entry point, sets 26).
        ul_retard_max=int(_env("PONT_UL_RETARD_MAX", "0")),
        rssi=int(_env("PONT_RSSI", "60")),
        # 4730 is the target: the fixed port of the gr-gsm L1 of QEMU
        # (calypso_l1_grgsm.c). PONT_GSMTAP_PORT allows inserting
        # qosmo/tools/injecteur_bruit.py (grgsm target, 14730 -> 4730); pont.py
        # sets it by itself when BRUIT_MODE is ber or relais.
        gsmtap_port=int(_env("PONT_GSMTAP_PORT", "4730")),
        sch_port=4731,
        tap=not a.no_tap,
        tap_port=int(_env("PONT_TAP_PORT", "4729")),
        bsc_cfg=a.bsc_cfg,
        clock_period=int(_env("PONT_CLCK_PERIOD_FN", "51")),
        kc_retention=_flag("PONT_KC_RETENTION", "1"),
        record=not a.no_record,
        record_ul=_flag("PONT_AIRREC_UL", "1"),
        record_dl_path=_env("PONT_AIRREC_DL_PATH", "/root/record.cfile"),
        record_ul_path=_env("PONT_AIRREC_UL_PATH", "/root/record_ul.cfile"),
        record_dl_fifo=_env("PONT_AIRREC_DL_FIFO", "/tmp/iq_fft.fifo"),
        record_ul_fifo=_env("PONT_AIRREC_UL_FIFO", "/tmp/iq_fft_ms.fifo"),
        record_max_mb=int(_env("PONT_AIRREC_MAX_MB", "4096")),
        record_min_free_mb=int(_env("PONT_AIRREC_MINFREE_MB", "8192")),
        record_osr=int(_env("PONT_AIRREC_OSR", "4")),
    )
```

2.167 /opt/GSM/osmo-operator/pont/downlink.py

15057 octets, 359 lignes → 359 lignes

```
import logging
import os
import socket
import struct

from . import gsm
```

```
_dcch_gen = [0]   # generation of calypso_dcch_cfg (IMM ASS), see CHANNEL RELEASE
# RA of the TI firmware's last RACHs (dsp/uplink.py, noter_rach) and time of the last IMM ASS
# retained: see _imm_ass_dcch.
_rach_ra = []
_imm_ass_retenue = [0.0]
IMM_ASS_GARDE_S = float(os.environ.get("PONT_IMM_ASS_GARDE", "2.0"))


def noter_rach(ra):
    """A RACH from the TI firmware: the mobile accepts the IMM ASS answering one of its last 3 (44.018
     ↪  3.3.1.1.3)."""
    _rach_ra.append(ra)
    del _rach_ra[:-3]

log = logging.getLogger("pont")

TCH_DL_PATH = "/dev/shm/calypso_tch_dl"
TCH_DL_SLOTS = 16
TCH_DL_SLOT = 48
GSMTAP_HOST = "127.0.0.1"


class Feeder:
    def __init__(self, cfg, stats):
        self.cfg = cfg
        self.stats = stats
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    def l2(self, fn, chan, l2, tn=0, uplink=False, to_qemu=True):
        pkt = gsm.gsmtap_header(self.cfg.arfcn, fn, chan, tn, uplink) + bytes(l2)
        if not uplink and to_qemu:
            self.sock.sendto(pkt, (GSMTAP_HOST, self.cfg.gsmtap_port))
        if self.cfg.tap:
            self.sock.sendto(pkt, (GSMTAP_HOST, self.cfg.tap_port))

    def sch(self, fn):
        self.sock.sendto(b"SCH2" + struct.pack("<iii", self.cfg.bsic, fn, self.cfg.arfcn), (GSMTAP_HOST,
         ↪  self.cfg.sch_port))


class TchRing:
    def __init__(self):
        self.fd = None
        self.w = 0

    def publish(self, fr33, fn):
        if self.fd is None:
            self.fd = os.open(TCH_DL_PATH, os.O_CREAT | os.O_RDWR, 0o644)
            os.ftruncate(self.fd, 8 + TCH_DL_SLOTS * TCH_DL_SLOT)
            os.pwrite(self.fd, (0).to_bytes(4, "little") + TCH_DL_SLOTS.to_bytes(4, "little"), 0)
            self.w = 0
        self.w += 1
        rec = self.w.to_bytes(4, "little") + (fn & 0xffffffff).to_bytes(4, "little") + bytes(fr33)[:gsm.FR_BYTES]
        rec += bytes(TCH_DL_SLOT - len(rec))
        os.pwrite(self.fd, rec, 8 + ((self.w - 1) % TCH_DL_SLOTS) * TCH_DL_SLOT)
        os.pwrite(self.fd, self.w.to_bytes(4, "little"), 0)


class Downlink:
    def __init__(self, cfg, stats, cipher, dedicated, tch, feeder, record, timeslots):
        self.cfg = cfg
        self.stats = stats
        self.cipher = cipher
        self.dedicated = dedicated
        self.tch = tch
```

```
    self.feed = feeder
    self.record = record
    self.timeslots = timeslots
    self.blocks = {}
    self.ring = TchRing()
    self.tch_acc = []
    self.tch_last_fn = None
    self.sacch_acc = {}
    self._deciphered = {}
    tch.on_close.append(self._tch_reset)

def _tch_reset(self):
    self.tch_acc = []
    self.tch_last_fn = None
    self.sacch_acc = {}
    self._deciphered = {}

def dispatch(self, tn, fn, burst):
    if self.record:
        self.record.feed_dl(fn, tn, burst)
    if tn == 0 and fn % 51 in gsm.SCH_SLOTS_51:
        if gsm.sch_decode(burst):
            self.feed.sch(fn)
        return
    if tn == self.tch.active_tn():
        self._tch(tn, fn, burst)
        return
    phys = self.timeslots.get(tn, "")
    plan = gsm.plan_for(phys)
    if plan is not None:
        self._signalling(tn, fn, burst, plan)

def _active_subchannel(self, tn, plan):
    ded = self.dedicated.plan()
    if ded is None:
        return None, False
    dplan, ss, dtn = ded
    if dtn != tn:
        return None, True
    if dplan is not plan:
        return None, False
    return ss % plan.n_sub, False

def _signalling(self, tn, fn, burst, plan):
    m51 = fn % 51
    base = next((b for b in plan.block_bases if b <= m51 <= b + 3), None)
    if base is None:
        return
    fn0 = fn - (m51 - base)
    ss, elsewhere = self._active_subchannel(tn, plan)
    if elsewhere and base in plan.ded_bases:
        self.stats.dl_skipped += 1
        return
    if ss is not None:
        if base in plan.sdcch_dl and base != plan.sdcch_dl[ss]:
            return
        if base in plan.sacch_dl and (fn0 % 102) != plan.sacch_dl102[ss]:
            return
    if len(self.blocks) > 64:
        self.blocks.clear()
    if gsm.is_dummy(burst):
        self.stats.dl_dummy += 1
        return
    acc = self.blocks.setdefault((tn, fn0), [])
    acc.append((fn, burst))
    self.stats.dl_bursts += 1
```

```
    if len(acc) < 4:
        return
    del self.blocks[(tn, fn0)]
    l2 = self._decode(acc, base in plan.ded_bases)
    self.stats.block(tn, base, l2 is not None)
    if l2 is None:
        self.stats.dl_crc_fail += 1
        return
    self.stats.dl_blocks += 1
    if base in plan.sdcch_dl:
        self.feed.l2(fn0, gsm.GSMTAP_SDCCH4, l2, tn)
        self._rr_downlink(l2, fn0)
        return
    if base in plan.sacch_dl:
        self.feed.l2(fn0, gsm.GSMTAP_SACCH, l2, tn)
        return
    mt = None
    if len(l2) >= 3 and l2[1] == gsm.RR_PD:
        mt = l2[2]
    elif len(l2) >= 5 and l2[3] == gsm.RR_PD:
        mt = l2[4]
    if mt in (gsm.RR_IMMEDIATE_ASSIGNMENT, gsm.RR_IMMEDIATE_ASSIGNMENT_EXT):
        self.cipher.release("IMMEDIATE ASSIGNMENT")
        if mt == gsm.RR_IMMEDIATE_ASSIGNMENT:
            self._imm_ass_dcch(l2, fn0)
    if mt in gsm.SI_TYPES:
        self.feed.l2(fn0, gsm.GSMTAP_BCCH, l2, tn)
    elif mt in gsm.CCCH_TYPES:
        # EMPTY PAGINGS DO NOT GO TO THE QEMU L1. The L1
        # (calypso_l1_grgsm.c, feed_agch) keeps ONLY ONE CCCH block - the
        # last received, valid for 100 frames. osmo-bts fills every free
        # PCH block with a PAGING REQUEST 1 without identity: a real paging
        # (the mobile's TMSI) would be overwritten ~50 ms later, and the
        # firmware would only see it if its read of its paging group fell
        # within the window. The Wireshark tap keeps everything.
        self.feed.l2(fn0, gsm.GSMTAP_CCCH, l2, tn,
                     to_qemu=not gsm.is_empty_paging(l2))

def _clear(self, bursts):
    return [gsm.coded_from_burst(b) for _, b in bursts]

def _ciphered(self, bursts):
    if len(self._deciphered) > 64:
        self._deciphered.clear()
    out = []
    for fn, b in bursts:
        d = self._deciphered.get(fn)
        if d is None:
            d = self._deciphered[fn] = gsm.coded_from_burst(self.cipher.apply(b, fn, False))
        out.append(d)
    return out

def _decode(self, bursts, dedicated):
    # As in a real network: clear, then ciphered. The BTS enables downlink
    # ciphering only after the first ciphered uplink frame, so SI5/SI6 and
    # the first SDCCH frames arrive in clear even when the Kc is already
    # known. Decode in clear as long as it works, and lock the global
    # "downlink ciphered" state as soon as a block decodes only once
    # deciphered. All channels follow that state.
    if not dedicated or self.cipher.current() is None:
        return gsm.xcch_decode(self._clear(bursts))
    if not self.cipher.dl_active:
        l2 = gsm.xcch_decode(self._clear(bursts))
        if l2 is not None:
            return l2
    l2 = gsm.xcch_decode(self._ciphered(bursts))
```

```
        if l2 is not None:
            self.cipher.confirm_dl(bursts[0][0])
        return l2

    def _decode_tch(self, bursts):
        # Same rule for the TCH: speech follows the global state, a valid
        # FACCH (Fire CRC) serves as proof to switch the state.
        if self.cipher.current() is None:
            return gsm.tch_fr_decode(self._clear(bursts))
        if self.cipher.dl_active:
            return gsm.tch_fr_decode(self._ciphered(bursts))
        rc, fr = gsm.tch_fr_decode(self._clear(bursts))
        if rc == gsm.MACBLOCK_LEN:
            return rc, fr
        rc2, fr2 = gsm.tch_fr_decode(self._ciphered(bursts))
        if rc2 == gsm.MACBLOCK_LEN:
            self.cipher.confirm_dl(bursts[0][0])
            return rc2, fr2
        return rc, fr

    def _imm_ass_dcch(self, l2, fn):
        """TI firmware (Magnetite): no QEMU tap to tell the bench which dedicated channel the mobile
        goes to. The IMMEDIATE ASSIGNMENT (44.018 9.1.18) says it: octet 1 pseudo-length, 2 PD, 3 type,
        4 page mode, 5 chan_nr. SDCCH/4 = 0b00100SST, SDCCH/8 = 0b01SSSTTT.
        CALYPSO_PONT_IMM_DCCH=0 to disable; only with MS_MAGNETITE (the osmocom firmware has its tap).
        The mobile takes the FIRST one answering one of its RACHs and ignores the following ones. So the
        first one whose reference (octet 7, RA) is a RACH of the mobile is kept, then the others are
        ignored for PONT_IMM_ASS_GARDE s (the bridge reads the downlink ~30 frames before the mobile,
        which may still emit a RACH). Without a known RA (decoding failed), the last received wins."""
        if not os.environ.get("MS_MAGNETITE") or os.environ.get("CALYPSO_PONT_IMM_DCCH", "1") == "0":
            return
        if len(l2) < 5 or l2[1] != gsm.RR_PD:
            return
        cn = l2[4]
        if (cn >> 5) == 0b001:                              # SDCCH/4 + SACCH: 0b001SSTTT, subchannel 0..3
            kind, ss = 0, (cn >> 3) & 3
        elif (cn >> 6) == 0b01:                             # SDCCH/8 + SACCH, subchannel 0..7
            kind, ss = 1, (cn >> 3) & 7
        else:
            return
        if _rach_ra and len(l2) > 7:
            import time
            ra = l2[7]
            if ra not in _rach_ra or time.monotonic() - _imm_ass_retenue[0] < IMM_ASS_GARDE_S:
                log.info("IMMEDIATE ASSIGNMENT (fn=%u) : SDCCH TS%d SS%d pour RA 0x%02x, ignoree (RACH du mobile :
↪ %s)",
                         fn, cn & 7, ss, ra, " ".join("0x%02x" % r for r in _rach_ra) or "-")
                return
            _imm_ass_retenue[0] = time.monotonic()
            _rach_ra.clear()
        from .state import ecrire_dcch
        _dcch_gen[0] += 1                                   # cancels a pending deferred release
        ecrire_dcch(kind, ss, cn)
        log.info("IMMEDIATE ASSIGNMENT (fn=%u) : SDCCH/%d TS%d SS%d -> calypso_dcch_cfg (firmware TI)",
                 fn, 8 if kind else 4, cn & 7, ss)

    def _rr_downlink(self, l2, fn):
        off, mt = gsm.rr_message_type(l2)
        if mt == gsm.RR_ASSIGNMENT_COMMAND and len(l2) > off + 4:
            b0, b1, b2 = l2[off + 2], l2[off + 3], l2[off + 4]
            if (b1 >> 4) & 1:
                log.warning("ASSIGNMENT COMMAND avec saut de frequence : TCH non arme (fn=%u)", fn)
                return
            self.tch.arm(b0 & 0x07, (b1 >> 5) & 0x07, ((b1 & 0x03) << 8) | b2)
        elif mt == gsm.RR_CHANNEL_RELEASE:
            self.tch.release_requested("CHANNEL RELEASE")
```

```
        self._liberer_dcch_ti(fn, "CHANNEL RELEASE")

def _liberer_dcch_ti(self, fn, quoi):
    """TI firmware (Magnetite): no QEMU tap to say the mobile left the dedicated channel; the bridge
    itself releases calypso_dcch_cfg (and the TCH) after the CHANNEL RELEASE."""
    if not os.environ.get("MS_MAGNETITE") or os.environ.get("CALYPSO_PONT_IMM_DCCH", "1") == "0":
        return
    # DEFERRED. The bridge reads the downlink ~30 frames BEFORE the mobile (DL margin
    # +28..+35): releasing immediately would cut the SDCCH at the BSP before the mobile has
    # received this CHANNEL RELEASE, leaving it on a dead channel until link loss (~13 s),
    # deaf to paging meanwhile.
    # PONT_DCCH_REL_DELAI (s, default 1.5); an IMM ASS in the meantime cancels the release.
    # Also called from the FACCH: the CHANNEL RELEASE of a call arrives on the TCH.
    import threading
    from .state import ecrire_dcch, DCCH_RELEASED
    gen = _dcch_gen[0]
    delai = float(os.environ.get("PONT_DCCH_REL_DELAI", "1.5"))
    def _liberer():
        if _dcch_gen[0] == gen:
            self.tch.close("%s, liberation differee (firmware TI)" % quoi)
            ecrire_dcch(DCCH_RELEASED, 0, 0)
            log.info("%s (fn=%u) : calypso_dcch_cfg libere apres %.1f s (firmware TI)", quoi, fn, delai)
    t = threading.Timer(delai, _liberer)
    t.daemon = True
    t.start()

def _tch(self, tn, fn, burst):
    if gsm.is_dummy(burst):
        self.stats.dl_dummy += 1
        return
    if fn % 26 == gsm.sacch_tf_frame(tn):
        self._tch_sacch(tn, fn, burst)
        return
    if not gsm.is_tch_carrier(fn):
        return
    if self.tch_last_fn is not None and fn != gsm.next_tch_carrier(self.tch_last_fn):
        self.tch_acc = []
    self.tch_last_fn = fn
    self.tch_acc.append((fn, burst))
    if len(self.tch_acc) > 8:
        del self.tch_acc[0]
    if len(self.tch_acc) < 8 or gsm.tch_burst_index(fn) % 4 != 3:
        return
    rc, fr = self._decode_tch(self.tch_acc)
    if rc == gsm.FR_BYTES:
        self.stats.tch_dl += 1
        self.ring.publish(fr, fn)
        # REFERENCE FOR THE DSP SPEECH. Each FR frame the bridge decodes
        # itself from the BTS bursts, with its BTS fn: tools/comparer_parole.py
        # compares it with what the ROM delivers in a_dd
        # (/dev/shm/calypso_add_dl.bin, montant.c) to count the wrong bits
        # per class. At most 4000 frames per session.
        self._noter_parole(fn, fr)
    elif rc == gsm.MACBLOCK_LEN:
        self.stats.tch_facch_dl += 1
        l2 = fr[:gsm.MACBLOCK_LEN]
        self.feed.l2(fn, gsm.GSMTAP_TCH_F, l2, tn)
        if gsm.rr_message_type(l2)[1] == gsm.RR_CHANNEL_RELEASE:
            self.tch.release_requested("CHANNEL RELEASE (FACCH)")
            self._liberer_dcch_ti(fn, "CHANNEL RELEASE (FACCH)")
    else:
        self.stats.tch_crc += 1

def _noter_parole(self, fn, fr):
    n = getattr(self, "_parole_n", 0)
    if n >= 4000:
```

```
            return
        try:
            mode = "ab" if n else "wb"
            with open("/dev/shm/pont_tch_dl.bin", mode) as f:
                f.write(struct.pack("<I", fn & 0xffffffff) + bytes(fr[:gsm.FR_BYTES]))
        except OSError:
            return
        self._parole_n = n + 1

    def _tch_sacch(self, tn, fn, burst):
        start = gsm.sacch_tf_block_base(tn)
        block = fn - ((fn % 104 - start) % 104)
        acc = self.sacch_acc.setdefault(block, [])
        acc.append((fn, burst))
        if len(self.sacch_acc) > 4:
            for k in sorted(self.sacch_acc)[:-2]:
                del self.sacch_acc[k]
        if len(acc) < 4:
            return
        del self.sacch_acc[block]
        l2 = self._decode(acc, True)
        if l2 is None:
            self.stats.tch_crc += 1
            return
        self.stats.tch_sacch_dl += 1
        self.feed.l2(block, gsm.GSMTAP_TCH_ACCH, l2, tn)
```

2.168 /opt/GSM/osmo-operator/pont/gsm.py

9074 octets, 263 lignes → 264 lignes (1 groupes compactés)

```
import ctypes
import struct

HYPERFRAME = 26 * 51 * 2048
FRAME_DUR = 60.0 / 13000.0
MACBLOCK_LEN = 23
FR_BYTES = 33

GSMTAP_BCCH = 0x01
GSMTAP_CCCH = 0x04
GSMTAP_SDCCH4 = 0x07
GSMTAP_TCH_F = 0x09
GSMTAP_ACCH = 0x80
GSMTAP_SACCH = GSMTAP_SDCCH4 | GSMTAP_ACCH
GSMTAP_TCH_ACCH = GSMTAP_TCH_F | GSMTAP_ACCH

RR_PD = 0x06
RR_ASSIGNMENT_COMMAND = 0x2e
RR_ASSIGNMENT_COMPLETE = 0x29
RR_CHANNEL_RELEASE = 0x0d
RR_IMMEDIATE_ASSIGNMENT = 0x3f
RR_IMMEDIATE_ASSIGNMENT_EXT = 0x39
# Includes SI13 (0x00), SI2bis (0x02), SI2ter (0x03), SI2quater (0x07).
# As soon as BTS 0 has a neighbour in another band (BTS 1 DCS1800, ARFCN 614),
# osmo-bsc emits an SI2ter and announces it in the SI3 rest octets: the mobile
# requires it before selecting the cell (gsm322.c), so this filter must let it
# through. SI13 is needed for GPRS.
SI_TYPES = frozenset((0x19, 0x1a, 0x1b, 0x1c, 0x1d, 0x1e, 0x00, 0x02, 0x03, 0x07))
CCCH_TYPES = frozenset((0x3f, 0x39, 0x3a, 0x21, 0x22, 0x24))


def is_empty_paging(l2):
    """PAGING REQUEST TYPE 1 without identity: the filler block osmo-bts emits
```

```
    on every free PCH block (Mobile Identity of length 1, type 0 = "no
    identity"). Nothing to do with it on the mobile side."""
    return (len(l2) >= 6 and l2[1] == 0x06 and l2[2] == 0x21
            and l2[4] == 0x01 and (l2[5] & 0x07) == 0x00)

TSC7 = bytes((1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0))
RACH_SYNC = bytes(int(c) for c in "01001011011111111100110011010101000111100 0")
RACH_SLOTS_51 = frozenset([4, 5] + list(range(14, 37)) + [45, 46])
SCH_SLOTS_51 = frozenset((1, 11, 21, 31, 41))
DUMMY_BURST = bytes((
    0, 0, ⟨1⟩, ⟨2⟩, ⟨3⟩, 1, ⟨4⟩, ⟨5⟩, 0, ⟨6⟩, ⟨7⟩, 0, ⟨8⟩, ⟨9⟩, ⟨10⟩, ⟨11⟩, ⟨12⟩, ⟨13⟩, ⟨14⟩, ⟨15⟩, ⟨16⟩, ⟨17⟩, 0, ⟨18⟩, 0,
    ↪  ⟨19⟩, ⟨20⟩, ⟨21⟩, ⟨22⟩, 0, 0, 1, ⟨23⟩, ⟨24⟩, ⟨25⟩, ⟨26⟩, ⟨27⟩,  ×3
    ⟨⟩ = (0,1,1,1,1,1,1,1,1,0,1,1,0,0,0,0,1,1,0,0,1,1,1,0,0,0)
        (1,0,0,0,0,1,0,0,0,0,0,0,1,1,1,1,1,0,1,1,1,0,0,1,1,1,0)
        (1,0,1,1,0,0,1,1,0,1,1,1,0,1,0,0,1,1,0,0,1,1,1,0,0,1,1)
    1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0))
DUMMY_TOLERANCE = 8

_SOFT = bytes([127, 129] + [129] * 254)
_ONEBIT = bytes([0] + [1] * 255)

ubit = ctypes.c_int8
sbit = ctypes.c_int8
_cod = ctypes.CDLL("libosmocoding.so", use_errno=True)
_cod.gsm0503_xcch_encode.argtypes = [ctypes.POINTER(ubit), ctypes.POINTER(ctypes.c_uint8)]
_cod.gsm0503_xcch_decode.argtypes = [ctypes.POINTER(ctypes.c_uint8), ctypes.POINTER(sbit),
                                     ctypes.POINTER(ctypes.c_int), ctypes.POINTER(ctypes.c_int)]
_cod.gsm0503_rach_ext_encode.argtypes = [ctypes.POINTER(ubit), ctypes.c_uint16, ctypes.c_uint8, ctypes.c_bool]
_cod.gsm0503_rach_decode_ber.argtypes = [ctypes.POINTER(ctypes.c_uint8), ctypes.POINTER(sbit), ctypes.c_uint8,
                                       ctypes.POINTER(ctypes.c_int), ctypes.POINTER(ctypes.c_int)]
_cod.gsm0503_sch_decode.argtypes = [ctypes.POINTER(ctypes.c_uint8), ctypes.POINTER(sbit)]
_cod.gsm0503_tch_fr_encode.argtypes = [ctypes.POINTER(ubit), ctypes.POINTER(ctypes.c_uint8), ctypes.c_int,
 ↪  ctypes.c_int]
_cod.gsm0503_tch_fr_decode.argtypes = [ctypes.POINTER(ctypes.c_uint8), ctypes.POINTER(sbit), ctypes.c_int,
                                       ctypes.c_int, ctypes.POINTER(ctypes.c_int), ctypes.POINTER(ctypes.c_int)]
_gsm = ctypes.CDLL("libosmogsm.so", use_errno=True)
_gsm.osmo_a5.argtypes = [ctypes.c_int, ctypes.POINTER(ctypes.c_uint8), ctypes.c_uint32,
                         ctypes.POINTER(ubit), ctypes.POINTER(ubit)]


def normalize_bits(raw148):
    return raw148.translate(_ONEBIT)


def is_dummy(burst):
    errors = 0
    for a, b in zip(burst, DUMMY_BURST):
        if a != b:
            errors += 1
            if errors > DUMMY_TOLERANCE:
                return False
    return True


def coded_from_burst(burst):
    return burst[3:61] + burst[87:145]


def burst_from_coded(coded):
    return bytes(3) + bytes(coded[:58]) + TSC7 + bytes(coded[58:116]) + bytes(3)


def _soft(bursts):
    return b"".join(bytes(b).translate(_SOFT) for b in bursts)
```

```
def xcch_decode(bursts4):
    buf = (sbit * (4 * 116)).from_buffer_copy(_soft(bursts4))
    l2 = (ctypes.c_uint8 * MACBLOCK_LEN)()
    ne = ctypes.c_int()
    nb = ctypes.c_int()
    rc = _cod.gsm0503_xcch_decode(l2, buf, ctypes.byref(ne), ctypes.byref(nb))
    return bytes(l2) if rc == 0 else None


def xcch_encode(l2):
    data = (ctypes.c_uint8 * MACBLOCK_LEN)(*l2[:MACBLOCK_LEN])
    e = (ubit * (4 * 116))()
    _cod.gsm0503_xcch_encode(e, data)
    raw = bytes(e)
    return [burst_from_coded(raw[k * 116:(k + 1) * 116]) for k in range(4)]


def tch_fr_decode(bursts8):
    buf = (sbit * (8 * 116)).from_buffer_copy(_soft(bursts8))
    out = (ctypes.c_uint8 * FR_BYTES)()
    ne = ctypes.c_int()
    nb = ctypes.c_int()
    rc = _cod.gsm0503_tch_fr_decode(out, buf, 1, 0, ctypes.byref(ne), ctypes.byref(nb))
    return rc, bytes(out)


def tch_fr_encode_into(buf24, payload):
    if payload:
        data = (ctypes.c_uint8 * len(payload))(*payload)
        _cod.gsm0503_tch_fr_encode(buf24, data, len(payload), 1)
    else:
        _cod.gsm0503_tch_fr_encode(buf24, None, 0, 1)


def sch_decode(burst):
    d78 = burst[3:42] + burst[106:145]
    buf = (sbit * 78)(*[-127 if b else 127 for b in d78])
    sb = (ctypes.c_uint8 * 4)()
    return _cod.gsm0503_sch_decode(sb, buf) == 0


def rach_burst(ra, bsic):
    coded = (ubit * 40)()
    _cod.gsm0503_rach_ext_encode(coded, ctypes.c_uint16(ra), ctypes.c_uint8(bsic), False)
    return bytes(8) + RACH_SYNC + bytes(coded)[:36] + bytes(148 - 8 - 41 - 36)


def rach_ra(burst, bsic):
    """RA of an access burst (hard bits, 8 tail + 41 sync + 36 coded), None if the CRC
    (coloured by the BSIC) does not match."""
    d36 = burst[8 + 41:8 + 41 + 36]
    if len(d36) != 36:
        return None
    buf = (sbit * 36)(*[-127 if b else 127 for b in d36])
    ra = ctypes.c_uint8()
    nerr, nbits = ctypes.c_int(), ctypes.c_int()
    if _cod.gsm0503_rach_decode_ber(ctypes.byref(ra), buf, ctypes.c_uint8(bsic), ctypes.byref(nerr),
     ↪  ctypes.byref(nbits)):
        return None
    return ra.value


def a5_keystream(algo, kc, fn):
    key = (ctypes.c_uint8 * 8)(*kc)
    dl = (ubit * 114)()
    ul = (ubit * 114)()
```

```
    _gsm.osmo_a5(algo, key, ctypes.c_uint32(fn & 0xffffffff), dl, ul)
    return bytes(dl), bytes(ul)


def a5_xor(burst, ks):
    b = bytearray(burst)
    for i in range(57):
        b[3 + i] ^= ks[i]
        b[88 + i] ^= ks[57 + i]
    return bytes(b)


def gsmtap_header(arfcn, fn, chan, tn, uplink):
    a = arfcn | (0x4000 if uplink else 0)
    return struct.pack(">BBBBHbBIBBBB", 2, 4, 0x01, tn, a, 0, 0, fn, chan, 0, 0, 0)


def rr_message_type(l2):
    for off in (0, 3):
        if len(l2) > off + 1 and l2[off] == RR_PD:
            if off == 3 and (l2[2] >> 2) < 2:
                continue
            return off, l2[off + 1]
    return None, None


def tch_burst_index(fn):
    m = fn % 26
    return m if m < 12 else m - 1


def is_tch_carrier(fn):
    return fn % 26 not in (12, 25)


def next_tch_carrier(fn):
    n = (fn + 1) % HYPERFRAME
    while not is_tch_carrier(n):
        n = (n + 1) % HYPERFRAME
    return n


def sacch_tf_frame(tn):
    return 12 if tn % 2 == 0 else 25


def sacch_tf_block_base(tn):
    # 45.002 table 5: the SACCH/TF block of TN starts at frame 12 + 13*TN of the 104-multiframe
    return (12 + 13 * tn) % 104


def sacch_tf_block_start(fn, tn):
    return fn % 104 == sacch_tf_block_base(tn)


class SlotPlan:
    def __init__(self, name, sdcch_dl, sacch_dl, other_dl):
        self.name = name
        self.n_sub = len(sdcch_dl)
        self.sdcch_dl = tuple(sdcch_dl)
        self.sacch_dl = tuple(sacch_dl)
        half = len(sacch_dl)
        self.sacch_dl102 = tuple(sacch_dl[i % half] + (51 if i >= half else 0) for i in range(self.n_sub))
        self.ded_bases = tuple(sorted(set(self.sdcch_dl + self.sacch_dl)))
        self.block_bases = tuple(sorted(set(self.sdcch_dl + self.sacch_dl + tuple(other_dl))))
```

```
    def ul_base(self, ss):
        return (self.sdcch_dl[ss % self.n_sub] + 15) % 51


PLAN_SDCCH4 = SlotPlan("CCCH+SDCCH4", (22, 26, 32, 36), (42, 46), (2, 6, 12, 16))
PLAN_SDCCH8 = SlotPlan("SDCCH8", (0, 4, 8, 12, 16, 20, 24, 28), (32, 36, 40, 44), ())


def plan_for(phys):
    if phys.startswith("SDCCH8"):
        return PLAN_SDCCH8
    if phys.startswith("CCCH"):
        return PLAN_SDCCH4
    return None


def load_timeslots(path):
    ts = {}
    try:
        cur = None
        n_bts = 0
        for line in open(path, encoding="utf-8", errors="ignore"):
            s = line.strip()
            if s.startswith("bts "):
                n_bts += 1
                if n_bts > 1:
                    break
                cur = None
            elif s.startswith("timeslot "):
                cur = int(s.split()[1])
            elif s.startswith("phys_chan_config") and cur is not None:
                ts[cur] = s.split(None, 1)[1].strip()
                cur = None
    except OSError:
        pass
    return ts or {0: "CCCH+SDCCH4", 1: "SDCCH8", 2: "TCH/F", 3: "TCH/F"}
```

2.169 /opt/GSM/osmo-operator/pont/pont.py

1830 octets, 36 lignes → 36 lignes

```
#!/usr/bin/env python3
# pont.py - the bridge of the GRGSM setup (gr-gsm layer 1 inside QEMU).
#
# The bridge has two entry points on the same package:
#   pont.py           grgsm setup: QEMU receives the L2 blocks as GSMTAP
#                     (4730/4731), the bridge deciphers A5 for it.
#   pont_dsp.py       DSP setup (c54x_exe --arm, --dsp-port 6702): the bursts
#                     go to the C54x BSP.
# Both keep the A5 ciphering in the bridge. What differs between the two
# setups goes through the DEFAULTS set here, before the import, so that a
# change for the DSP cannot break the grgsm path. A variable set by the
# operator always wins (setdefault).
import os
import sys

# An uplink burst woken up after its frame is DROPPED (the DSP setup sends it,
# see trx.py Transmitter.run).
os.environ.setdefault("PONT_UL_RETARD_MAX", "0")

# NOISE INJECTOR (qosmo/tools/injecteur_bruit.py, grgsm target).
# The gr-gsm L1 of QEMU listens for GSMTAP on 4730, hard-coded: to insert the
# injector, the bridge is what aims elsewhere. BRUIT_MODE=ber or relais (the
# only modes that make sense on already decoded L2 blocks) aims at
```

```
# BRUIT_PORT_GRGSM (14730), where the injector listens
# (qosmo/run_modules/38-bruit.sh, c54x_exe/run.sh). Without BRUIT_MODE nothing
# changes. PONT_GSMTAP_PORT set by the operator (or by c54x_exe/run.sh) wins.
if os.environ.get("BRUIT_MODE", "") in ("ber", "relais"):
    os.environ.setdefault("PONT_GSMTAP_PORT", os.environ.get("BRUIT_PORT_GRGSM", "14730"))

sys.path[0] = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
from pont import main

if __name__ == "__main__":
    if any(a == "--dsp-port" or a.startswith("--dsp-port=") for a in sys.argv[1:]):
        sys.exit("pont.py est le pont du montage grgsm : pour le DSP, lancer pont/pont_dsp.py --dsp-port 6702")
    main()
```

2.170 /opt/GSM/osmo-operator/pont/pont_dsp.py

1496 octets, 31 lignes → 31 lignes

```
#!/usr/bin/env python3
# pont_dsp.py - the bridge of the DSP setup (c54x_exe --arm, bursts to the
# C54x BSP on --dsp-port 6702). The grgsm setup has its own entry point,
# pont.py, with its own defaults; both share the package (trx, downlink,
# uplink, A5 ciphering).
#
# This entry point starts pont.dsp.main, the real DSP bridge: same building
# blocks, but the dedicated state follows the DSP instead of the bridge's own
# decoding (see pont/dsp/__init__.py). pont/__init__.py and pont.py do not
# depend on it. Started by c54x_exe/run.sh in MODE=dsp (default PONT_PY), so
# also by start-native.sh --dsp.
import os
import sys

# An uplink burst woken up after its frame is still sent, up to 26 frames
# late: the BTS files it by its fn (see trx.py Transmitter.run).
os.environ.setdefault("PONT_UL_RETARD_MAX", "26")
# The Kc is kept after the release (pont/dsp/uplink.py): the retention is
# what keeps it readable once montant.c is back in clear.
os.environ.setdefault("PONT_KC_RETENTION", "1")
# The downlink also goes to the BSP of instance 4 (banc-rssi, udp 6762): with
# a classic BTS, the rssi application has no other source (the air only relays
# the TX ring of the instance 1 Calypso, here the mobile). Without rssi
# running, the copy goes to an empty port. PONT_DSP_COPIES= (empty) cuts it.
os.environ.setdefault("PONT_DSP_COPIES", "6762")

sys.path[0] = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
from pont.dsp import main

if __name__ == "__main__":
    main()
```

2.171 /opt/GSM/osmo-operator/pont/record.py

8097 octets, 217 lignes → 217 lignes

```
import logging
import os
import threading
import time

import numpy as np

from . import gsm
```

```
log = logging.getLogger("pont")

SAMPLES_PER_FRAME = 1250
SAMPLES_PER_SLOT_X4 = 625


class Direction:
    def __init__(self, name, path, fifo_path, max_bytes, min_free, osr, taps):
        self.name = name
        self.path = path
        self.fifo_path = fifo_path
        self.max_bytes = max_bytes
        self.min_free = min_free
        self.osr = osr
        self.taps = taps
        self.lock = threading.Lock()
        self.frames = {}
        self.fh = None
        self.bytes = 0
        self.off = False
        self.restart = False
        self.fifo_fd = None
        self.fifo_pending = b""
        self.fifo_retry = 0
        self.n_frames = 0
        self.n_bursts = 0
        self.n_fifo_drop = 0
        self.last_fn = None

    def feed(self, fn, tn, burst):
        if self.off:
            return
        with self.lock:
            # A LATE BURST GOES TO THE NEXT WRITTEN FRAME.
            # The ROM's uplink bursts (PONT_UL_RAW, pont/dsp/uplink.py) leave
            # at ITS fn, 12 to ~200 frames behind the bridge clock: filed under
            # an already written frame they would never be read back. For the
            # spectrum, the arrival time matters, not the fn.
            last = self.last_fn
            if last is not None and 0 <= (last - fn) % 2715648 < 1357824:
                fn = (last + 1) % 2715648
            self.frames.setdefault(fn, {})[tn] = burst

    def open(self):
        if self.fh is not None:
            self.fh.close()
            self.fh = None
        try:
            os.makedirs(os.path.dirname(self.path) or ".", exist_ok=True)
            st = os.statvfs(os.path.dirname(self.path) or ".")
            if st.f_bavail * st.f_frsize < self.min_free:
               log.warning("enregistrement %s coupe : moins de %d Mo libres", self.name, self.min_free // 1048576)
                return False
            self.fh = open(self.path, "wb", buffering=1024 * 1024)
        except OSError as e:
            log.warning("enregistrement %s impossible (%s)", self.name, e)
            return False
        self.bytes = 0
        log.info("enregistrement %s -> %s (buffer roulant de %d Mo)", self.name, self.path, self.max_bytes //
↪   1048576)
        return True

    def modulate(self, burst):
        nrz = 2.0 * np.frombuffer(bytes(burst), dtype=np.uint8).astype(np.float64) - 1.0
        prev = np.concatenate(([nrz[0]], nrz[:-1]))
```

```
    up = np.zeros(len(nrz) * self.osr)
    up[::self.osr] = nrz * prev
    phase = np.cumsum(np.convolve(up, self.taps, mode="same")) * (np.pi / 2.0)
    return np.exp(1j * phase).astype(np.complex64)

def _fifo(self, octets):
    if not self.fifo_path:
        return
    if self.fifo_fd is None:
        if self.n_frames < self.fifo_retry:
            return
        self.fifo_retry = self.n_frames + 217
        try:
            if not os.path.exists(self.fifo_path):
                os.mkfifo(self.fifo_path, 0o644)
            self.fifo_fd = os.open(self.fifo_path, os.O_WRONLY | os.O_NONBLOCK)
        except OSError:
            return
    data = self.fifo_pending + octets
    maxq = 3 * len(octets)
    if len(data) > maxq:
        data = data[len(data) - maxq:]
        self.n_fifo_drop += 1
    try:
        n = os.write(self.fifo_fd, data)
        self.fifo_pending = data[n:]
    except BlockingIOError:
        self.fifo_pending = data
    except OSError:
        os.close(self.fifo_fd)
        self.fifo_fd = None
        self.fifo_pending = b""

def write_frame(self, fn, spf, step, blen, silence):
    # THE FFT FIFO DOES NOT DEPEND ON THE DISK. fh is None when file
    # recording is cut (less than PONT_AIRREC_MINFREE_MB free: the case of
    # any live key, whose overlay lives in RAM). The FIFO feeds the I/Q
    # spectrum of the dashboard and the Conky, and the dashboard record
    # button reads THIS fifo: frames go to the FIFO even when the file stays
    # cut for lack of space.
    if self.off or (self.fh is None and not self.fifo_path):
        return
    with self.lock:
        slots = self.frames.pop(fn, None)
        self.last_fn = fn
        if len(self.frames) > 512:
            self.frames.clear()
    if slots:
        frame = np.zeros(spf, dtype=np.complex64)
        for tn, burst in slots.items():
            o = tn * step
            frame[o:o + blen] = self.modulate(burst)
            self.n_bursts += 1
    else:
        frame = silence
    octets = frame.tobytes()
    self._fifo(octets)
    self.n_frames += 1
    if self.fh is None:
        return
    try:
        self.fh.write(octets)
    except OSError as e:
        # The file is cut, not the FIFO: see the method header.
        log.warning("enregistrement %s : ecriture echouee (%s), coupe", self.name, e)
        try:
```

```
                self.fh.close()
            except OSError:
                pass
            self.fh = None
            return
        self.bytes += len(octets)
        if self.restart or self.bytes >= self.max_bytes:
            self.restart = False
            self.fh.flush()
            self.fh.seek(0)
            self.fh.truncate(0)
            self.bytes = 0


class Recorder(threading.Thread):
    def __init__(self, cfg, clock):
        super().__init__(name="record", daemon=True)
        self.cfg = cfg
        self.clock = clock
        osr = cfg.record_osr
        n = 4 * osr
        t = np.arange(-n, n + 1) / float(osr)
        g = np.exp(-2.0 * (np.pi ** 2) * (0.3 ** 2) * t ** 2 / np.log(2.0))
        taps = g / g.sum()
        max_bytes = cfg.record_max_mb * 1024 * 1024
        min_free = cfg.record_min_free_mb * 1024 * 1024
        self.dl = Direction("descendant", cfg.record_dl_path, cfg.record_dl_fifo, max_bytes, min_free, osr, taps)
        self.ul = Direction("montant", cfg.record_ul_path, cfg.record_ul_fifo, max_bytes, min_free, osr, taps)
        self.ul.off = not cfg.record_ul

    def feed_dl(self, fn, tn, burst):
        self.dl.feed(fn, tn, burst)

    def feed_ul(self, fn, tn, burst):
        self.ul.feed(fn, tn, burst)

    def split(self, *_):
        self.dl.restart = True
        self.ul.restart = True

    def summary(self):
        return " | IQ dl=%dtr/%dbu ul=%dtr/%dbu fifo_drop=%d/%d" % (
            self.dl.n_frames, self.dl.n_bursts, self.ul.n_frames, self.ul.n_bursts,
            self.dl.n_fifo_drop, self.ul.n_fifo_drop)

    def run(self):
        osr = self.cfg.record_osr
        if osr % 4:
            log.warning("enregistrement refuse : OSR=%d doit etre multiple de 4", osr)
            return
        # A direction whose file cannot be opened (space, permissions) keeps its
        # FIFO: it is switched off only if it has no FIFO to serve either.
        if not self.dl.open() and not self.dl.fifo_path:
            self.dl.off = True
        if not self.ul.off and not self.ul.open() and not self.ul.fifo_path:
            self.ul.off = True
        if self.dl.off and self.ul.off:
            return
        spf = SAMPLES_PER_FRAME * osr
        step = (SAMPLES_PER_SLOT_X4 * osr) // 4
        blen = 148 * osr
        silence = np.zeros(spf, dtype=np.complex64)
        nxt = (self.clock.fn() + 2) % gsm.HYPERFRAME
        while True:
            d = (self.clock.fn() - nxt) % gsm.HYPERFRAME
            if d > gsm.HYPERFRAME // 2 or d < 2:
```

```
                time.sleep(gsm.FRAME_DUR / 2.0)
                continue
            if d > 650:
                with self.dl.lock:
                    self.dl.frames.clear()
                with self.ul.lock:
                    self.ul.frames.clear()
                nxt = (self.clock.fn() + 2) % gsm.HYPERFRAME
                continue
            self.dl.write_frame(nxt, spf, step, blen, silence)
            self.ul.write_frame(nxt, spf, step, blen, silence)
            nxt = (nxt + 1) % gsm.HYPERFRAME
```

2.172 /opt/GSM/osmo-operator/pont/state.py

5230 octets, 165 lignes → 165 lignes

```
import logging
import os
import struct
import threading
import time

from . import gsm

log = logging.getLogger("pont")

DCCH_CFG = "/dev/shm/calypso_dcch_cfg"
TCH_CFG = "/dev/shm/calypso_tch_cfg"
DCCH_TTL = 0.1
DCCH_RELEASED = 0xFF


def ecrire_dcch(kind, ss, chan_nr):
    """calypso_dcch_cfg written by the bridge, in the QEMU tap format (calypso_dcch_tap.c):
    seq(4) kind(1) ss(1) tn(1) chan_nr(1), 16 bytes. The TI firmware (Magnetite) has no tap:
    without this announcement, montant.c never arms an SDCCH outside TS0 (SDCCH/8 on TS1).
    seq restarts above the one in the file."""
    seq = 0
    try:
        with open(DCCH_CFG, "rb") as f:
            b = f.read(4)
            if len(b) == 4:
                seq = struct.unpack("<I", b)[0]
    except OSError:
        pass
    b = struct.pack("<IBBBB", (seq + 1) & 0xFFFFFFFF, kind & 0xFF, ss & 7, chan_nr & 7, chan_nr & 0xFF) + bytes(8)
    # IN PLACE, no os.replace: montant.c and pont/state.py keep the file open (pread on their fd),
    # a new inode would never be read again.
    fd = os.open(DCCH_CFG, os.O_WRONLY | os.O_CREAT, 0o666)
    try:
        os.pwrite(fd, b, 0)
    finally:
        os.close(fd)


class Dedicated:
    def __init__(self):
        self._fd = None
        self._next = 0.0
        self._value = None
        self._seq = 0

    def read(self):
```

```
        now = time.monotonic()
        if now < self._next:
            return self._value
        self._next = now + DCCH_TTL
        try:
            if self._fd is None:
                self._fd = os.open(DCCH_CFG, os.O_RDONLY)
            b = os.pread(self._fd, 16, 0)
        except OSError:
            if self._fd is not None:
                os.close(self._fd)
                self._fd = None
            return self._value
        if len(b) < 8:
            return self._value
        seq = struct.unpack_from("<I", b, 0)[0]
        if not seq or seq == self._seq:
            return self._value
        self._seq = seq
        self._value = None if b[4] == DCCH_RELEASED else (b[4] & 1, b[5] & 7, b[6] & 7)
        return self._value

    def refresh(self):
        """Force the next read: the file may just have changed."""
        self._next = 0.0

    def plan(self):
        v = self.read()
        if v is None:
            return None
        kind, ss, tn = v
        return (gsm.PLAN_SDCCH8 if kind else gsm.PLAN_SDCCH4), ss, tn


class Tch:
    def __init__(self, stats):
        self.stats = stats
        self.lock = threading.Lock()
        self.tn = None
        self.tsc = 0
        self.open = False
        self.seq = 0
        self.on_close = []
        self.release_seen = None
        # time of the proof (prove): dsp/uplink.py only withdraws the TCH announcement on an
        # uplink SDCCH block after a grace delay, see TCH_ABANDON_GRACE.
        self.open_since = None

    def open_age(self):
        """Seconds since the mobile proved the TCH, None if it is not open."""
        with self.lock:
            return None if self.open_since is None else time.monotonic() - self.open_since

    def arm(self, tn, tsc, arfcn):
        with self.lock:
            self.seq += 1
            self.tn = tn
            self.tsc = tsc
            self.open = False
            self._write_cfg(tn, tsc, arfcn, self.seq)
        log.info("ASSIGNMENT COMMAND : TCH TN=%d TSC=%d ARFCN=%d arme, en attente du mobile", tn, tsc, arfcn)

    def prove(self, source):
        with self.lock:
            if self.tn is None or self.open:
                return
```

```
            self.open = True
            self.open_since = time.monotonic()
            tn = self.tn
        log.info("le mobile est sur le TCH TN=%d (%s) : montant TCH ouvert", tn, source)

    def release_requested(self, reason):
        with self.lock:
            if self.release_seen is not None:
                return
            self.release_seen = time.monotonic()
            tn = self.tn
        if tn is None:
            log.info("%s : en attente de la liberation du canal par le mobile", reason)
        else:
            log.info("%s : TCH TN=%d maintenu jusqu'a la liberation par le mobile", reason, tn)

    def release_overdue(self, delay):
        with self.lock:
            return self.release_seen is not None and time.monotonic() - self.release_seen > delay

    def close(self, reason):
        with self.lock:
            self.release_seen = None
            if self.tn is None:
                return
            tn = self.tn
            self.tn = None
            self.open = False
            self.open_since = None
            self._write_cfg(0, 0, 0, 0)
        log.info("TCH TN=%d ferme (%s)", tn, reason)
        for cb in self.on_close:
            cb()

    def active_tn(self):
        return self.tn

    def is_open(self):
        return self.open and self.tn is not None

    @staticmethod
    def _write_cfg(tn, tsc, arfcn, seq):
        buf = bytearray(16)
        buf[4] = tn & 0xff
        buf[5] = tsc & 0xff
        buf[6:8] = int(arfcn).to_bytes(2, "little")
        buf[0:4] = int(seq).to_bytes(4, "little")
        fd = os.open(TCH_CFG, os.O_CREAT | os.O_WRONLY, 0o644)
        try:
            os.pwrite(fd, bytes(buf), 0)
        finally:
            os.close(fd)
```

2.173 /opt/GSM/osmo-operator/pont/stats.py

5412 octets, 110 lignes → 110 lignes

```
import logging
import threading
import time

log = logging.getLogger("pont")
```

```
class Stats:
    def __init__(self):
        self.dl_bursts = 0
        self.dl_blocks = 0
        self.dl_crc_fail = 0
        self.dl_dummy = 0
        self.dl_skipped = 0
        self.ul_sent = 0
        self.ul_late = 0
        self.rach = 0
        self.tch_dl = 0
        self.tch_facch_dl = 0
        self.tch_sacch_dl = 0
        self.tch_crc = 0
        self.tch_ul = 0
        self.tch_ul_bursts = 0
        self.tch_ul_dropped = 0
        self.facch_ul = 0
        self.sacch_ul = 0
        self.a5_dl = 0
        self.a5_ul = 0
        # uplink xCCH blocks (SDCCH/SACCH of the SDCCH): bursts from the DSP ROM
        # (0x3f8a, pont/dsp/uplink.py) or coded by the host (gsm.xcch_encode); rom_ko = ROM
        # bursts present but which do not decode back to this L2.
        self.xcch_ul_rom = 0
        self.xcch_ul_hote = 0
        self.xcch_ul_rom_ko = 0
        # WHAT REALLY GOES TO THE ANTENNA, per path (PONT_UL_RAW=1).
        # The rach/xcch_ul_hote/a5_ul counters above also count the "ghost" host
        # path that ends in TransmitterPuits. air_* = bursts handed to the TRX.
        self.air_rom_rach = 0
        self.air_rom_xcch = 0      # ROM NBs outside TCH and PDCH (SDCCH/SACCH, ciphered by the ROM)
        self.air_rom_pdch = 0
        self.air_rom_tch = 0
        self.air_hote_tch = 0      # TCH coded by the host (PONT_UL_RAW_TCH=0)
        self.air_puits = 0         # host bursts dropped (never sent)
        # uplink TCH per sub-channel (dsp/uplink.py classe_tch): F FACCH, S SACCH/T, P speech.
        # rom = ROM bursts sent, rom_jete = ROM TCH bursts discarded (PONT_UL_RAW_TCH=0),
        # hote = bursts coded by the host and sent.
        self.tch_ul_rom = {"F": 0, "S": 0, "P": 0}
        self.tch_ul_rom_jete = {"F": 0, "S": 0, "P": 0}
        self.tch_ul_hote = {"F": 0, "S": 0, "P": 0}
        # uplink A5: rom = NB bursts marked ciphered by the coprocessor (calypso_a5.c),
        # hote_tch = TCH bursts ciphered by the bridge (cipher.py).
        self.a5_ul_rom = 0
        self.a5_ul_hote_tch = 0
        # ROM bursts of unknown type (sent), PDTCH blocks seen on the host SDCCH path
        # and dropped, xCCH blocks without ROM bursts in time and not recoded,
        # fn lateness ROM -> BTS clock of the TCH bursts (in frames).
        self.air_rom_inconnu = 0
        self.pdtch_ul_vu_hote = 0
        self.xcch_shadow_ko = 0
        self.tch_rom_retard_min = None
        self.tch_rom_retard_max = None
        self.sacch_t_rom_blocs = 0
        self.by_base = {}

    def block(self, tn, base, ok):
        st = self.by_base.setdefault((tn, base), [0, 0])
        st[0 if ok else 1] += 1


class Reporter(threading.Thread):
    def __init__(self, stats, clock, dedicated, record, period=5.0):
        super().__init__(name="stats", daemon=True)
        self.stats = stats
```

```
        self.clock = clock
        self.dedicated = dedicated
        self.record = record
        self.period = period

    def run(self):
        s = self.stats
        while True:
            time.sleep(self.period)
            log.info("STATS fn=%u | DL bursts=%d blocs=%d crc=%d dummy=%d hors_voie=%d | UL bursts=%d tard=%d
↪  rach=%d"
                     " | TCH dl=%d facch=%d sacch=%d crc=%d ul=%d bursts=%d perdus=%d | FACCH ul=%d SACCH ul=%d"
                     " | A5 dl=%d ul=%d | xCCH ul rom=%d hote=%d rom_ko=%d"
                     " | AIR rom rach=%d xcch=%d pdch=%d tch=%d | AIR hote tch=%d puits=%d"
                     " | TCH ul rom F=%d S=%d P=%d jete F=%d S=%d P=%d hote F=%d S=%d P=%d"
                     " | A5 ul rom=%d hote_tch=%d"
                     " | B inconnu=%d pdtch_hote_jete=%d xcch_ombre_ko=%d | TCH rom retard=%s..%s
                      ↪  sacch_t_blocs=%d%s",
                     self.clock.fn(), s.dl_bursts, s.dl_blocks, s.dl_crc_fail, s.dl_dummy, s.dl_skipped,
                     s.ul_sent, s.ul_late, s.rach, s.tch_dl, s.tch_facch_dl, s.tch_sacch_dl, s.tch_crc,
                     s.tch_ul, s.tch_ul_bursts, s.tch_ul_dropped, s.facch_ul, s.sacch_ul, s.a5_dl, s.a5_ul,
                     s.xcch_ul_rom, s.xcch_ul_hote, s.xcch_ul_rom_ko,
                     s.air_rom_rach, s.air_rom_xcch, s.air_rom_pdch, s.air_rom_tch, s.air_hote_tch, s.air_puits,
                     s.tch_ul_rom["F"], s.tch_ul_rom["S"], s.tch_ul_rom["P"],
                     s.tch_ul_rom_jete["F"], s.tch_ul_rom_jete["S"], s.tch_ul_rom_jete["P"],
                     s.tch_ul_hote["F"], s.tch_ul_hote["S"], s.tch_ul_hote["P"],
                     s.a5_ul_rom, s.a5_ul_hote_tch,
                     s.air_rom_inconnu, s.pdtch_ul_vu_hote, s.xcch_shadow_ko,
                     "-" if s.tch_rom_retard_min is None else s.tch_rom_retard_min,
                     "-" if s.tch_rom_retard_max is None else s.tch_rom_retard_max, s.sacch_t_rom_blocs,
                     self.record.summary() if self.record else "")
            ded = self.dedicated.plan()
            if ded:
                plan, ss, tn = ded
                log.info("canal dedie : %s SS=%d TS=%d (DL fn%%51=%d, UL fn%%51=%d)",
                         plan.name, ss, tn, plan.sdcch_dl[ss % plan.n_sub], plan.ul_base(ss))
            if s.by_base:
                log.info("blocs par TS/base51 (ok/echec) : %s",
                       " ".join("TS%d/%d:%d/%d" % (k[0], k[1], v[0], v[1]) for k, v in sorted(s.by_base.items())))
```

2.174 /opt/GSM/osmo-operator/pont/trx.py

24088 octets, 469 lignes → 469 lignes

```
import heapq
import logging
import os
import socket
import struct
import threading
import time

from . import gsm

log = logging.getLogger("pont")

# THE BENCH CLOCK FOLLOWS THE DSP, NOT THE WALL.
#
# In the DSP setup the C54x is emulated: it costs ~5.8 ms per TDMA frame
# against 4.615 ms of real time. The BTS runs on the clock this file sends it
# (IND CLOCK, run_clock). With a wall clock it would emit 1.25 frames for each
# one the DSP consumes: the BSP would replay an ever older ring, the IMMEDIATE
# ASSIGNMENT would arrive after T3126 and the dedicated channel would yield
# incomplete blocks ("Dropping frame with 110 bit errors").
```

```
#
# c54x_exe publishes the BTS frame its BSP requests (calypso_bsp.c,
# bsp_horloge_publier). The clock is slaved to it IN FREQUENCY, not by
# re-alignment: the effective frame duration (self.dur) follows the measured
# DSP rate, with a proportional correction holding the phase lead at AVANCE
# frames.
#
# Why not a simple re-alignment of t0: Transmitter.run() drops any burst
# whose frame is more than window_tol (1 frame) away from the clock at send
# time; a clock stepping forward by a few frames leaves that window every
# time, so no SABM reaches the BTS, hence no UA, hence "MDL-ERROR-IND cause 1"
# and no location update. A slaved clock must stay CONTINUOUS: only its speed
# changes, and t0 is rebased at each change so that fn() never jumps.
#
# PONT_HORLOGE=0 falls back to the wall clock; a missing file (setup without
# DSP, gr-gsm layer 1) also leaves the clock free, with nothing to configure.
HORLOGE = "/dev/shm/calypso_horloge"
#
# LOOP TUNING. The loop must run faster (~1.3 Hz) than the rate at which the
# DSP speed itself moves: a slower loop (~0.07 Hz) lets the phase enter a
# limit cycle of +/- 100 frames, and each half-turn where the error goes
# negative puts the bridge BEHIND the requested frame -- frames skipped that
# way are counted NOWHERE (neither "tard" nor "manques" on the BSP side).
# Do NOT "fix" that by raising PONT_HORLOGE_AVANCE: it hides the limit cycle
# behind a margin without removing it, and delays the whole uplink as much.
# The values are read from the environment so they can be swept without
# editing the code.
HORLOGE_PERIODE = float(os.environ.get("PONT_HORLOGE_PERIODE", "0.05"))
HORLOGE_KP = float(os.environ.get("PONT_HORLOGE_KP", "1.0"))
HORLOGE_PHASE_N = float(os.environ.get("PONT_HORLOGE_PHASE_N", "100.0"))
# SLOWDOWN CEILING. PONT_HORLOGE_MAX (default 4) bounds the factor by which the
# slaved clock may slow below real time. With the full ARM asm trace
# (start-native.sh --assembly-logs, ASSEMBLY_LOGS_FLAGS=in_asm,exec,nochain)
# the ARM+DSP pair runs at 66 ms per frame, 14 times real time, and the bridge
# would otherwise run thousands of frames ahead, the BSP no longer finding the
# bursts; c54x_exe/run.sh sets 20 when ASSEMBLY_LOGS=1. The ceiling stays low
# in normal operation: after a DSP jolt the rate measured over 50 ms explodes
# and the clock must not collapse.
# Above 4, run_clock shortens the IND CLOCK period, see below.
HORLOGE_MAX = float(os.environ.get("PONT_HORLOGE_MAX", "4.0"))
# IND CLOCK with a slow clock, see run_clock: maximum lead left to the BTS
# before the IND (= requested pause, BTS limit 50), fn-advance of the BTS
# (osmotrx fn-advance), and safeguard if the BTS stops delivering
# (osmo-bts-trx cuts after TRX_LOSS_FRAMES = 400 frames without IND).
CLCK_PAUSE_MAX = int(os.environ.get("PONT_CLCK_PAUSE_MAX", "20"))
CLCK_BTS_FN_ADVANCE = int(os.environ.get("PONT_BTS_FN_ADVANCE", "2"))
CLCK_SANS_IND_MAX = int(os.environ.get("PONT_CLCK_SANS_IND_MAX", "300"))
# CEILING OF THE PAUSE REQUESTED FROM THE BTS. osmo-bts-trx (scheduler_trx.c) pauses its timer for
# N+1 frames when the IND says it is N frames ahead (N <= MAX_FN_SKEW = 50; beyond that it re-aligns,
# "GSM clock skew", harmlessly); if two expirations are then more than 50 frames apart (230.75 ms),
# it EXITS ("PC clock skew too high"). A pause of 46 to 50 frames plus one frame of jitter is enough
# to kill it. If the BTS lead falls in the dangerous zone (> CEILING and <= MAX_FN_SKEW + margin), the
# IND carries fn_bts - CEILING instead of fn_trx: the pause is bounded and the remaining lead is
# absorbed by the following INDs (bursts only arrive earlier, the BSP ring keeps them). 0 = no ceiling.
CLCK_PAUSE_PLAFOND = int(os.environ.get("PONT_CLCK_PAUSE_PLAFOND", "38"))
CLCK_SKEW_ZONE = int(os.environ.get("PONT_CLCK_SKEW_ZONE", "60"))   # beyond, the BTS re-aligns on its own


class Clock:
    def __init__(self):
        self.t0 = time.monotonic()
        self.dur = gsm.FRAME_DUR
        self._verrou = threading.Lock()
        self._asservie = os.environ.get("PONT_HORLOGE", "1") not in ("0", "", "no")
        self._avance = int(os.environ.get("PONT_HORLOGE_AVANCE", "12"))
        self._fd = None
```

```
    self._echeance = 0.0
    self._t_prec = None
    self._dsp_prec = None
    self._dit = False

def _lire_dsp(self):
    """BTS frame the DSP requests, or None if it is not synchronised yet."""
    try:
        if self._fd is None:
            self._fd = os.open(HORLOGE, os.O_RDONLY)
        b = os.pread(self._fd, 16, 0)
    except OSError:
        if self._fd is not None:
            os.close(self._fd)
            self._fd = None
        return None
    if len(b) < 16:
        return None
    seq, cale, fn_bts, _tick = struct.unpack("<IIII", b)
    return fn_bts % gsm.HYPERFRAME if (seq and cale) else None

def _reguler(self, now):
    """Called under lock, at most once per HORLOGE_PERIODE."""
    if now < self._echeance:
        return
    self._echeance = now + HORLOGE_PERIODE
    dsp = self._lire_dsp()
    if dsp is None:
        self._t_prec = self._dsp_prec = None
        return
    if not self._dit:
        self._dit = True
        log.info("horloge asservie au DSP (%s, avance visee %d trames, ralentissement max x%g)",
                 HORLOGE, self._avance, HORLOGE_MAX)
    t_prec, dsp_prec = self._t_prec, self._dsp_prec
    self._t_prec, self._dsp_prec = now, dsp
    if t_prec is None:
        return
    avancees = (dsp - dsp_prec) % gsm.HYPERFRAME
    if avancees <= 0 or avancees > gsm.HYPERFRAME // 2:
        return                                  # DSP stopped or re-aligned: keep the speed
    dur_dsp = (now - t_prec) / avancees         # measured DSP rate
    # Phase error, in frames, signed: > 0 = the bench clock is ahead.
    f = (now - self.t0) / self.dur
    ecart = (int(f) - (dsp + self._avance)) % gsm.HYPERFRAME
    if ecart > gsm.HYPERFRAME // 2:
        ecart -= gsm.HYPERFRAME
    cible = dur_dsp * (1.0 + HORLOGE_KP * ecart / HORLOGE_PHASE_N)
    cible = min(max(cible, gsm.FRAME_DUR * 0.5), gsm.FRAME_DUR * HORLOGE_MAX)
    dur = 0.7 * self.dur + 0.3 * cible
    self.t0 = now - f * dur                     # rebase: fn() stays continuous
    self.dur = dur

def fn(self):
    now = time.monotonic()
    if not self._asservie:
        return int((now - self.t0) / self.dur) % gsm.HYPERFRAME
    with self._verrou:
        self._reguler(now)
        return int((now - self.t0) / self.dur) % gsm.HYPERFRAME

def time_of(self, fn):
    now = time.monotonic()
    if self._asservie:
        with self._verrou:
            self._reguler(now)
```

```
                t0, dur = self.t0, self.dur
            else:
                t0, dur = self.t0, self.dur
            cur = int((now - t0) / dur)
            delta = (fn - cur) % gsm.HYPERFRAME
            if delta > gsm.HYPERFRAME // 2:
                delta -= gsm.HYPERFRAME
            return t0 + (cur + delta) * dur

        def sleep_until_fn(self, fn):
            dt = self.time_of(fn) - time.monotonic()
            if dt > 0:
                time.sleep(dt)


    def _udp(bind_host, port):
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.bind((bind_host, port))
        return s


    class Trx:
        def __init__(self, cfg, clock, stats, cipher, record, dedicated=None, tch=None):
            self.cfg = cfg
            self.clock = clock
            self.stats = stats
            self.cipher = cipher
            self.record = record
            self.dedicated = dedicated
            self.tch = tch
            self.sk_clck = _udp(cfg.trx_bind, cfg.trx_base)
            self.sk_ctrl = _udp(cfg.trx_bind, cfg.trx_base + 1)
            self.sk_data = _udp(cfg.trx_bind, cfg.trx_base + 2)
            self.bts_data = None
            self.bts_clck = None
            self.fn_bts_vu = None          # fn of the last downlink burst received from the BTS (run_data)
            self._clck_t = 0.0             # last IND CLOCK: time, BTS fn seen, requested lead
            self._clck_vu = None
            self._clck_avance = 0
            self._clck_lent = False
            # COPIES OF THE DOWNLINK to other BSPs (PONT_DSP_COPIES, comma-separated UDP ports): a
            # Calypso listening to the cell without being the bridge's mobile, e.g. the rssi app of
            # instance 4 (banc-rssi, BSP udp 6762) with a classic BTS. Separate socket: a port without
            # a listener (rssi not started) must not change anything on the mobile's path.
            self.dsp_copies = [int(p) for p in os.environ.get("PONT_DSP_COPIES", "").replace(",", " ").split()
                               if p.isdigit() and int(p) != cfg.dsp_port]
            self.sk_copies = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) if self.dsp_copies else None
            self.copies_ul = os.environ.get("PONT_DSP_COPIES_UL", "1") != "0"

        def run_ctrl(self):
            while True:
                data, addr = self.sk_ctrl.recvfrom(1500)
                req = data.decode("latin1").strip("\0").strip()
                if not req.startswith("CMD"):
                    continue
                parts = req.split()
                cmd = parts[1] if len(parts) > 1 else ""
                args = (" " + " ".join(parts[2:])) if len(parts) > 2 else ""
                self.sk_ctrl.sendto(("RSP %s 0%s" % (cmd, args)).encode() + b"\0", addr)
                if self.bts_clck is None:
                    self.bts_clck = (addr[0], self.cfg.trx_base + 100)
                    log.info("BTS %s : horloge armee vers le port %d", addr[0], self.bts_clck[1])
                if cmd not in ("SETPOWER", "NOHANDOVER"):
                    log.info("CTRL %s%s", cmd, args)

        def run_clock(self):
```

```
# IND CLOCK WHEN THE CLOCK IS SLOW: WAIT FOR THE BTS TO RESUME.
# osmo-bts-trx (scheduler_trx.c) advances its frames at its own wall
# rate between two IND CLOCKs. At each IND it measures
# elapsed_fn = fn_trx - fn_bts; if negative, it PAUSES its timer for
# -elapsed_fn frames ("We were N FN faster than TRX, compensating"), and
# that pause is RE-ARMED FROM NOW by any IND received while it runs. Its
# timer kills the BTS if two expirations are more than MAX_FN_SKEW = 50
# frames apart ("PC clock skew too high"). With the fixed period of 51
# frames and k <= 4 the pause is 38 frames and ends 13 frames before
# the next IND; at k=14 it would be 47 frames, 3 frames from the next
# IND: untenable.
# Rule. As long as the fixed-period pause, period x (1 - 1/k), stays
# under CLCK_PAUSE_MAX (k <= ~1.9), nothing changes: one IND every 51
# frames, the pause ends well before the next. Beyond that, an IND is
# sent only when (1) the pause requested by the previous one has
# elapsed AND the BTS has started delivering bursts again (fn seen by
# run_data advances), and (2) the BTS is CLCK_PAUSE_MAX frames ahead of
# the clock again. After a pause N the lead is still N (1 - 1/k): the
# BTS only runs a few frames before the next IND, the requested pause
# stays ~CLCK_PAUSE_MAX whatever k and is never re-armed. No return to
# the fixed period in this regime: it would let the lead climb to
# 40-100 frames.
# Downlink bursts carry fn_bts + clock_advance (osmotrx fn-advance,
# 2 by default: main.c).
periode = self.cfg.clock_period
pause_max = CLCK_PAUSE_MAX
fn_adv = CLCK_BTS_FN_ADVANCE
demi = gsm.HYPERFRAME // 2
lent = False
while True:
    now = time.monotonic()
    if self.bts_clck:
        fn_trx = self.clock.fn()
        k = self.clock.dur / gsm.FRAME_DUR
        vu = self.fn_bts_vu
        attente = (now - self._clck_t) / gsm.FRAME_DUR
        lent = k > 1.0 and vu is not None and periode * (1.0 - 1.0 / k) > pause_max
        if lent != self._clck_lent:
            self._clck_lent = lent
            if lent:
              log.info("IND CLOCK : horloge lente (x%.1f), on attend la reprise de la BTS entre deux IND "
                       "(pause max %d trames)", k, pause_max)
            else:
                log.info("IND CLOCK : horloge x%.1f, retour a la periode fixe de %d trames", k, periode)
        if not lent:
            envoyer = attente >= periode
            avance = 0
        else:
            avance = ((vu - fn_adv - fn_trx) + demi) % gsm.HYPERFRAME - demi
            # BTS resumed: bursts arrive again. One or two may be in flight
            # right after the IND, hence the awaited pause (or 3 bursts, if
            # the BTS re-aligned without a pause: "GSM clock skew", or the
            # previous IND was sent before its first burst).
            if self._clck_vu is None:
                livre = 3
            else:
                livre = ((vu - self._clck_vu) + demi) % gsm.HYPERFRAME - demi
            reprise = livre >= 3 or (livre >= 1 and attente >= self._clck_avance)
            envoyer = (reprise and avance >= pause_max) or attente >= CLCK_SANS_IND_MAX
        if envoyer:
            fn_ind = fn_trx
            if CLCK_PAUSE_PLAFOND > 0 and vu is not None:
                av_bts = ((vu - fn_adv - fn_trx) + demi) % gsm.HYPERFRAME - demi
                if CLCK_PAUSE_PLAFOND < av_bts <= CLCK_SKEW_ZONE:
                    fn_ind = (fn_trx + av_bts - CLCK_PAUSE_PLAFOND) % gsm.HYPERFRAME
                    self._clck_bornees = getattr(self, "_clck_bornees", 0) + 1
```

```
                        if self._clck_bornees <= 20 or self._clck_bornees % 100 == 0:
                            log.warning("IND CLOCK : BTS %d trames devant, pause bornee a %d (fn %u au lieu de
↪  %u, "
                                        "%d fois)", av_bts, CLCK_PAUSE_PLAFOND, fn_ind, fn_trx, self._clck_bornees)
                    self.sk_clck.sendto(("IND CLOCK %u" % fn_ind).encode() + b"\0", self.bts_clck)
                    self._clck_t = now
                    self._clck_vu = vu
                    self._clck_avance = max(avance, 0)
                    self.stats_clck(avance)
            # Slaved clock (k > 1): poll every frame, otherwise the first lead
            # measurement after start-up comes 51 frames too late (lead 47 at
            # k=14, pause > 50: BTS stopped from the start).
            # In the slow regime, every half frame: the equilibrium pause is
            # r (k - 1), r being the detection delay of the resume (~1.5 frame
            # when polling per frame, i.e. 30 frames at k=20; ~1 per half frame).
            if lent:
                time.sleep(gsm.FRAME_DUR / 2)
            else:
                time.sleep(gsm.FRAME_DUR if self.clock.dur > gsm.FRAME_DUR else gsm.FRAME_DUR * periode)

    def stats_clck(self, avance):
        """Count the IND CLOCKs sent with a slow clock; log the long pauses."""
        if not self._clck_lent:
            return
        self._clck_n = getattr(self, "_clck_n", 0) + 1
        if avance > CLCK_PAUSE_MAX + 10 and getattr(self, "_clck_longues", 0) < 20:
            self._clck_longues = getattr(self, "_clck_longues", 0) + 1
            log.warning("IND CLOCK : la BTS avait %d trames d'avance (pause demandee > %d, limite BTS 50)",
                        avance, CLCK_PAUSE_MAX)

    def run_data(self, on_burst):
        while True:
            data, addr = self.sk_data.recvfrom(2000)
            self.bts_data = addr
            if len(data) < 6 + 148:
                continue
            tn = data[0] & 0x07
            fn = struct.unpack_from(">L", data, 1)[0]
            self.fn_bts_vu = fn            # where the BTS is (run_clock, slow clock)
            bits = gsm.normalize_bits(data[6:154])
            if self.cfg.dsp_port:
                # The same burst, as the DSP's BSP expects it
                # (calypso_bsp.c bsp_trxd_readable): 8 header bytes
                # [tn, fn BE32, attenuation, 0, 0] then 148 bits 0/1. The BSP
                # converts them itself to I/Q (cos table) and puts them in DARAM.
                #
                # THE DOWNLINK OF THE DEDICATED CHANNEL IS DECIPHERED HERE.
                # send_ul() CIPHERS the uplink (the bridge stands in for the
                # DSP's transmit path), and there is no A5 in the Calypso model
                # -- `d_a5mode` only exists in l1-grgsm/, nothing in l1-dsp/ --
                # so nobody else deciphers the downlink. A5 is symmetric:
                # applying the stream to the downlink deciphers it.
                # ONLY on the dedicated timeslot: the BCCH and the CCCH are
                # never ciphered, touching them would destroy camping.
                bits_dsp = bits
                if self.dechiffre_dl and self.cipher.dl_active and self._burst_dedie(tn, fn):
                    bits_dsp = self.cipher.apply(bits, fn, False)
                hdr = bytes([tn & 0x07]) + struct.pack(">L", fn) + bytes([data[5] if len(data) > 5 else 0, 0, 0])
                paquet = hdr + bytes(1 if b else 0 for b in bits_dsp)
                self.sk_data.sendto(paquet, ("127.0.0.1", self.cfg.dsp_port))
                for port in self.dsp_copies:
                    try:
                        self.sk_copies.sendto(paquet, ("127.0.0.1", port))
                    except OSError:
                        pass
            on_burst(tn, fn, bits)
```

```
    # Decipher the downlink before passing it to the BSP: yes for the gr-gsm
    # shunt, no in the DSP setup (TrxDsp) where the ROM does it itself.
    dechiffre_dl = True

    def _burst_dedie(self, tn, fn):
        """True if THIS burst belongs to the mobile's dedicated channel.

        Testing the timeslot alone (`tn == tn_dedie`) is not enough: in
        CCCH+SDCCH/4 the dedicated channel lives on TS0, which ALSO carries the
        FCCH, the SCH, the BCCH and the CCCH -- never ciphered, by definition:
        the mobile must be able to read them before it has a key. XORing all
        of TS0 towards the DSP would undo camping under the ongoing
        transaction. The same sorting as Downlink._signalling is done here: the
        block must be the one of the mobile's subchannel `ss`, SDCCH or SACCH.
        The read is cached by Dedicated (DCCH_TTL), so this can be called on
        every burst."""
        if self.tch is not None and self.tch.is_open():
            # On an assigned TCH the whole timeslot belongs to the transaction
            # (traffic, FACCH and SACCH): no subchannel to sort.
            if tn == self.tch.active_tn():
                return True
        if self.dedicated is None:
            return False
        ded = self.dedicated.plan()
        if ded is None:
            return False
        plan, ss, tn_ded = ded
        if tn != tn_ded:
            return False
        m51 = fn % 51
        base = next((b for b in plan.ded_bases if b <= m51 <= b + 3), None)
        if base is None:
            return False
        ss %= plan.n_sub
        if base in plan.sdcch_dl:
            return base == plan.sdcch_dl[ss]
        return (fn - (m51 - base)) % 102 == plan.sacch_dl102[ss]

    def send_ul(self, tn, fn, burst, cipher):
        if self.bts_data is None:
            return
        if cipher:
            burst = self.cipher.apply(burst, fn, True)
        hdr = bytes([tn & 0x07]) + struct.pack(">L", fn) + bytes([self.cfg.rssi & 0xFF]) + struct.pack(">h", 0)
        self.sk_data.sendto(hdr + bytes(255 if b else 0 for b in burst), self.bts_data)
        self.stats.ul_sent += 1
        # The UPLINK also goes to the copies (PONT_DSP_COPIES), marked 0xA5 at byte 6: the BSP
        # (calypso_bsp.c, CALYPSO_AIR_NIVEAUX) files it separately and only plays it in a window tuned
        # to the uplink frequency (rssi, UL view). PONT_DSP_COPIES_UL=0 disables it.
        if self.dsp_copies and self.copies_ul:
            paquet = bytes([tn & 0x07]) + struct.pack(">L", fn) + bytes([0, 0xA5, 0]) + bytes(1 if b else 0 for b
↪   in burst)
            for port in self.dsp_copies:
                try:
                    self.sk_copies.sendto(paquet, ("127.0.0.1", port))
                except OSError:
                    pass
        if self.record:
            self.record.feed_ul(fn, tn, burst)


class Transmitter(threading.Thread):
    def __init__(self, cfg, clock, trx, stats):
        super().__init__(name="tx", daemon=True)
        self.cfg = cfg
```

```
        self.clock = clock
        self.trx = trx
        self.stats = stats
        self.cond = threading.Condition()
        self.heap = []
        self.seq = 0

    def schedule(self, tn, fn_air, burst, cipher, essais=0):
        post = self.clock.time_of((fn_air - self.cfg.ul_fn_advance) % gsm.HYPERFRAME)
        with self.cond:
            self.seq += 1
            heapq.heappush(self.heap, (post, self.seq, tn, fn_air, burst, cipher, essais))
            self.cond.notify()

    def run(self):
        while True:
            with self.cond:
                while not self.heap:
                    self.cond.wait()
                post = self.heap[0][0]
                dt = post - time.monotonic()
                if dt > 0:
                    self.cond.wait(dt)
                    continue
                _, _, tn, fn_air, burst, cipher, essais = heapq.heappop(self.heap)
            cur = self.clock.fn()
            off = (fn_air - (cur + self.cfg.ul_fn_advance) + gsm.HYPERFRAME // 2) % gsm.HYPERFRAME -
↪   gsm.HYPERFRAME // 2
            # TOO EARLY IS NOT TOO LATE.
            # `post` is computed at queueing time, from the clock rate AT THAT
            # MOMENT. Since the clock follows the DSP (see Clock above), it no
            # longer advances at wall pace: the DSP stalls, the clock with it,
            # and the wake-up comes before the target frame has arrived. Such a
            # burst is early, not late: it is re-queued with a `post`
            # recomputed on the current clock; only a REALLY late burst is lost.
            if off > self.cfg.window_tol:
                if essais < self.cfg.window_essais:
                    self.schedule(tn, fn_air, burst, cipher, essais + 1)
                else:
                    self.stats.ul_late += 1
                continue
            if off < -self.cfg.window_tol:
                # LATE IS NOT LOST. osmo-bts-trx files an uplink burst by ITS
                # fn, compared with the last fn processed on the same logical
                # channel (scheduler.c, trx_sched_route_burst_ind); the arrival
                # time plays no part -- a real transceiver always delivers the
                # uplink after the fact. The lateness comes from the thread
                # wake-up (GIL shared with the I/Q recording) and from the
                # variations of the DSP-slaved clock, not from the BTS.
                # The burst is sent as long as it stays within the multiframe
                # (the BTS refuses beyond mf_period, "Too many contiguous TDMA
                # frames"). PONT_UL_RETARD_MAX=0 drops late bursts.
                self.stats.ul_late += 1
                if -off > self.cfg.ul_retard_max:
                    continue
            self.trx.send_ul(tn, fn_air, burst, cipher)
```

2.175 /opt/GSM/osmo-operator/pont/uplink.py

16425 octets, 407 lignes → 407 lignes

```
import collections
import ctypes
import logging
```

```
import os
import struct
import threading
import time

from . import gsm

log = logging.getLogger("pont")

SB_RACH = "/dev/shm/calypso_rach"
SB_SDCCH_UL = "/dev/shm/calypso_sdcch_ul"
SB_FACCH_UL = "/dev/shm/calypso_tch_facch_ul"
SB_SACCH_UL = "/dev/shm/calypso_tch_sacch_ul"
SB_TCH_UL = "/dev/shm/calypso_tch_ul"
TCH_UL_SLOT = 64
TCH_UL_FR_OFS = 16
POLL = gsm.FRAME_DUR / 8
TX_BURSTS = 24
BPLEN = 116
RELEASE_FALLBACK = 20.0
# How long to keep an uplink SDCCH block while waiting to know the dedicated
# channel (see _poll_sdcch).
SDCCH_ATTENTE = 1.0


class Sideband:
    def __init__(self, path, size):
        self.path = path
        self.size = size
        self.fd = None
        self.seq = 0

    def read(self):
        try:
            if self.fd is None:
                self.fd = os.open(self.path, os.O_RDONLY)
            return os.pread(self.fd, self.size, 0)
        except OSError:
            if self.fd is not None:
                os.close(self.fd)
                self.fd = None
            return b""

    def new_record(self):
        b = self.read()
        if len(b) < self.size:
            return None
        seq = struct.unpack_from("<I", b, 0)[0]
        if not seq or seq == self.seq:
            return None
        self.seq = seq
        return b

    def skip_pending(self):
        b = self.read()
        if len(b) >= 4:
            self.seq = struct.unpack_from("<I", b, 0)[0]


class TchUplinkRing:
    def __init__(self):
        self.fd = None
        self.last = 0

    def drain(self):
        try:
```

```
            if self.fd is None:
                self.fd = os.open(SB_TCH_UL, os.O_RDONLY)
            hdr = os.pread(self.fd, 8, 0)
        except OSError:
            if self.fd is not None:
                os.close(self.fd)
                self.fd = None
            return [], 0
        if len(hdr) < 8:
            return [], 0
        w, n = struct.unpack("<II", hdr)
        if not w or not n:
            return [], 0
        lost = 0
        if self.last == 0:
            self.last = w
        elif (w - self.last) > n:
            lost = (w - self.last) - n
            self.last = w - n
        frames = []
        while self.last < w:
            self.last += 1
            sl = os.pread(self.fd, TCH_UL_SLOT, 8 + ((self.last - 1) % n) * TCH_UL_SLOT)
            if len(sl) >= TCH_UL_FR_OFS + gsm.FR_BYTES:
                frames.append(sl[TCH_UL_FR_OFS:TCH_UL_FR_OFS + gsm.FR_BYTES])
        return frames, lost


class Uplink(threading.Thread):
    def __init__(self, cfg, clock, stats, dedicated, tch, transmitter, feeder, cipher):
        super().__init__(name="uplink", daemon=True)
        self.cfg = cfg
        self.clock = clock
        self.stats = stats
        self.dedicated = dedicated
        self.tch = tch
        self.cipher = cipher
        self.ded_active = False
        self.tx = transmitter
        self.feed = feeder
        self.sb_rach = Sideband(SB_RACH, 12)
        self.sb_sdcch = Sideband(SB_SDCCH_UL, 39)
        self.sb_facch = Sideband(SB_FACCH_UL, 39)
        self.sb_sacch = Sideband(SB_SACCH_UL, 39)
        self.ring = TchUplinkRing()
        self.q_lock = threading.Lock()
        self.q_facch = collections.deque(maxlen=8)
        self.q_voice = collections.deque(maxlen=4)
        self.sacch_l2 = None
        self.tch_epoch = -1
        self.sdcch_attente = None
        for sb in (self.sb_rach, self.sb_sdcch, self.sb_facch, self.sb_sacch):
            sb.skip_pending()

    def _next_fn(self, min_advance, accept):
        start = self.clock.fn() + min_advance
        for k in range(104):
            fn = (start + k) % gsm.HYPERFRAME
            if accept(fn):
                return fn
        return start % gsm.HYPERFRAME

    def _queue_facch(self, l2, ouverture=False):
        # A SAPI 0 UI ON THE TCH IS A BLOCK THAT IS NOT A FACCH.
        # A block whose first octet has the EA bit at 0 is not a LAPDm address:
        # lapdm.c:995 takes the frame for a short header (Bter) and passes it
```

```
    # up AS IS as UNIT DATA IND ("SAPI=0 UNIT DATA INDICATION:
    # unimplemented" at the BSC). That is the shape of an uplink SACCH
    # block: L1 header first (power, TA) -- 08 00 01 03 49 ... at power 8.
    # A FACCH is always format B and SAPI 0: address 0x01 or 0x03.
    # Any other block is discarded here, octets logged to find its source.
    # PONT_FACCH_TOUT=1 restores unfiltered sending.
    if l2[0] not in (0x01, 0x03) and os.environ.get("PONT_FACCH_TOUT", "0") != "1":
        log.info("FACCH montante ECARTEE (pas une adresse SAPI 0) : %s",
                 bytes(l2[:gsm.MACBLOCK_LEN]).hex(" "))
        return
    # Non-I frames (SABM, RR, REJ, DISC...) are rare: they are logged, as
    # is the first FACCH of each TCH.
    if (l2[1] & 0x01) or ouverture:
        log.info("FACCH montante %s : %s", "non-I" if l2[1] & 0x01 else "I (ouverture)",
                 bytes(l2[:gsm.MACBLOCK_LEN]).hex(" "))
    with self.q_lock:
        self.q_facch.append(bytes(l2[:gsm.MACBLOCK_LEN]))
    self.stats.facch_ul += 1
    self.feed.l2(self.clock.fn(), gsm.GSMTAP_TCH_F, l2[:gsm.MACBLOCK_LEN], self.tch.active_tn() or 0, True)

def _poll_rach(self):
    b = self.sb_rach.new_record()
    if b is None:
        return
    _, ra, bsic, fn = struct.unpack_from("<IBBxxI", b, 0)
    burst = gsm.rach_burst(ra, bsic)
    self.tx.schedule(0, self._next_fn(4, lambda f: f % 51 in gsm.RACH_SLOTS_51), burst, False)
    self.stats.rach += 1

def _route_sdcch(self, l2):
    """True if the uplink SDCCH block leaves as FACCH (consumed here).

    Split out of _poll_sdcch so that the DSP bridge (pont/dsp/uplink.py)
    can override it; the grgsm setup keeps this code."""
    tn_tch = self.tch.active_tn()
    if tn_tch is not None and not self.tch.is_open():
        if gsm.rr_message_type(l2)[1] == gsm.RR_ASSIGNMENT_COMPLETE:
            self.tch.prove("ASSIGNMENT COMPLETE")
            self._queue_facch(l2)
            return True
    elif self.tch.is_open():
        self._queue_facch(l2)
        return True
    return False

def _poll_sdcch(self):
    b = self.sb_sdcch.new_record()
    if b is not None:
        l2 = b[16:39]
        if self._route_sdcch(l2):
            return
        # The block WAITS for the channel to be known, it is not dropped.
        # Layer 1 publishes the SABM at the very moment it emits it, and
        # the dedicated channel is read here at most every DCCH_TTL
        # seconds: the first block of a connection would fall in that
        # window. The channel is re-read right away and the block is kept
        # for a moment if it is not armed yet.
        self.sdcch_attente = (l2, time.monotonic())
        self.dedicated.refresh()
    if self.sdcch_attente is None:
        return
    l2, depuis = self.sdcch_attente
    ded = self.dedicated.plan()
    if ded is None:
        if time.monotonic() - depuis > SDCCH_ATTENTE:
            log.info("SDCCH montant : aucun canal dedie apres %.1f s, bloc abandonne",
```

```
                    SDCCH_ATTENTE)
            self.sdcch_attente = None
        return
    plan, ss, tn = ded
    bursts = self._bursts_xcch(l2)
    if bursts is None:
        return          # the DSP setup is still waiting for the ROM's bursts (pont/dsp/uplink.py)
    self.sdcch_attente = None

    # SDCCH or SACCH? Layer 1 cannot tell: the firmware puts both in the
    # SAME a_cu buffer with the SAME task word (prim_tx_nb.c:118,
    # dsp_load_tx_task(DUL_DSP_TASK) in both cases); only its internal
    # MF_F_SACCH flag separates them, and it never reaches the RAM API. So
    # it is decided here, on the SHAPE of the block: an SDCCH block starts
    # with the LAPDm address (0x01 or 0x03), a SACCH block with its two L1
    # header octets (ordered power, TA), the address only coming at
    # position 2.
    #
    # The two kinds of block seen on the SDCCH path:
    #   01 3f 49 05 08 70 ...  -> SABM + LOCATION UPDATING REQUEST
    #   07 00 01 03 49 06 15   -> L1 header (pwr 7, TA 0) + MEASUREMENT REPORT
    # Sending the second on the uplink SDCCH frames would leave the uplink
    # SACCH empty: the BTS would see nothing on its slow link and drop the
    # channel a few seconds after establishment.
    #
    # SAPI 3: its LAPDm address is 0x0d / 0x0f, and a SAPI 3 frame without
    # information (UA, SABM, RR, DISC) carries at position 2 a length
    # indicator L=0, i.e. 0x01 -- exactly what the test above would take
    # for the SAPI 0 address of a SACCH block, e.g.
    #   0f 73 01 2b 2b ...     -> UA SAPI 3 (answer to the BTS's SABM)
    # Sent on the uplink SACCH, the BTS would decode it as "Unknown (SS)",
    # never establish SAPI 3, and the SMS CP-DATA would not leave.
    # Octet 0 = 0x0d / 0x0f is ambiguous (power 13 / 15 of a SACCH
    # header): SACCH is chosen only if the SACCH reading holds up --
    # TA <= 63 (6 bits) and a real LAPDm control field at position 3, not
    # the 0x2b padding that follows an empty SAPI 3 frame.
    sacch = l2[0] not in (0x01, 0x03) and l2[2] in (0x01, 0x03)
    if sacch and l2[0] in (0x0d, 0x0f):
        sacch = l2[1] <= 63 and l2[3] != 0x2b
    if sacch:
        base = (plan.sacch_dl102[ss % plan.n_sub] + 15) % 102
        fn0 = self._next_fn(4, lambda f: f % 102 == base)
        self.feed.l2(self.clock.fn(), gsm.GSMTAP_SACCH, l2, tn, True)
        self.stats.sacch_ul += 1
    else:
        base = plan.ul_base(ss)
        fn0 = self._next_fn(4, lambda f: f % 51 == base)
        self.feed.l2(self.clock.fn(), gsm.GSMTAP_SDCCH4, l2, tn, True)
    for j, burst in enumerate(bursts):
        self.tx.schedule(tn, (fn0 + j) % gsm.HYPERFRAME, burst, True)

def _bursts_xcch(self, l2):
    """The 4 bursts of an uplink SDCCH/SACCH block. Coded here by the host; the DSP bridge
    (pont/dsp/uplink.py) takes the ROM's. None = not ready yet, the block stays pending
    and _poll_sdcch presents it again on the next turn."""
    self.stats.xcch_ul_hote += 1
    return gsm.xcch_encode(l2)

def _poll_facch(self):
    tn = self.tch.active_tn()
    if tn is None:
        self.tch_epoch = -1
        return
    if self.tch_epoch != self.tch.seq:
        # DO NOT DROP THE FIRST FACCH AFTER ARMING.
        #
```

```
            # `skip_pending()` moves the cursor past EVERYTHING already written
            # in the side band. `tch.seq` is incremented by `tch.arm()`, called
            # when the bridge decodes the downlink ASSIGNMENT COMMAND -- and the
            # ASSIGNMENT COMPLETE is precisely THE FIRST thing the mobile emits
            # on the new TCH, so skipping the FACCH here would mark it "already
            # seen" and the BSC would end in "Assignment failed in state
            # WAIT_RR_ASS_COMPLETE, cause EQUIPMENT FAILURE: Timeout".
            #
            # The skip is kept for the SACCH (a stale measurement report is
            # useless) but the pending FACCH is PROCESSED. The residual risk --
            # replaying a FACCH of the previous session -- is bounded to one
            # block, which LAPDm rejects.
            # PONT_FACCH_SKIP=1 skips the pending FACCH as well.
            self.tch_epoch = self.tch.seq
            self.sb_sacch.skip_pending()
            if os.environ.get("PONT_FACCH_SKIP", "0") == "1":
                self.sb_facch.skip_pending()
                return
        b = self.sb_facch.new_record()
        if b is None:
            return
        l2 = b[16:39]
        # A block that is not a FACCH (see _queue_facch) does not open the TCH.
        if l2[0] not in (0x01, 0x03) and os.environ.get("PONT_FACCH_TOUT", "0") != "1":
            self._queue_facch(l2)
            return
        ouverture = not self.tch.is_open()
        self.tch.prove("FACCH montante%s" % (", ASSIGNMENT COMPLETE" if gsm.rr_message_type(l2)[1] ==
        ↪  gsm.RR_ASSIGNMENT_COMPLETE else ""))
        self._queue_facch(l2, ouverture)

    def _poll_sacch(self):
        b = self.sb_sacch.new_record()
        if b is None:
            return
        self.sacch_l2 = bytes(b[16:39])
        self.stats.sacch_ul += 1

    def _poll_release(self):
        active = self.dedicated.plan() is not None
        if self.ded_active and not active:
            reason = "canal libere par le mobile"
        elif self.tch.release_overdue(RELEASE_FALLBACK):
            reason = "CHANNEL RELEASE sans liberation du mobile depuis %ds" % RELEASE_FALLBACK
        else:
            self.ded_active = active
            return
        self.ded_active = active
        self.tch.close(reason)
        self.cipher.release(reason)

    def _poll_voice(self):
        if not self.tch.is_open():
            return
        frames, lost = self.ring.drain()
        self.stats.tch_ul_dropped += lost
        for fr in frames:
            with self.q_lock:
                if len(self.q_voice) == self.q_voice.maxlen:
                    self.stats.tch_ul_dropped += 1
                self.q_voice.append(bytes(fr))
            self.stats.tch_ul += 1

    def run(self):
        while True:
            self._poll_release()
```

```
            self._poll_rach()
            self._poll_sdcch()
            self._poll_facch()
            self._poll_sacch()
            self._poll_voice()
            time.sleep(POLL)


class TchScheduler(threading.Thread):
    def __init__(self, cfg, clock, stats, tch, uplink, transmitter):
        super().__init__(name="tch-tx", daemon=True)
        self.cfg = cfg
        self.clock = clock
        self.stats = stats
        self.tch = tch
        self.uplink = uplink
        self.tx = transmitter
        self.buf = (gsm.ubit * (TX_BURSTS * BPLEN))()
        self.mask = 0
        self.sent_sacch_block = None
        tch.on_close.append(self._reset)

    def _reset(self):
        ctypes.memset(self.buf, 0, TX_BURSTS * BPLEN)
        self.mask = 0
        self.sent_sacch_block = None
        with self.uplink.q_lock:
            self.uplink.q_facch.clear()
            self.uplink.q_voice.clear()

    def _load_block(self):
        ctypes.memmove(self.buf, ctypes.byref(self.buf, 4 * BPLEN), 20 * BPLEN)
        ctypes.memset(ctypes.byref(self.buf, 20 * BPLEN), 0, 4 * BPLEN)
        self.mask = (self.mask << 4) & 0xFFFFFFFF
        with self.uplink.q_lock:
            facch = self.uplink.q_facch.popleft() if self.uplink.q_facch else None
            voice = self.uplink.q_voice.popleft() if self.uplink.q_voice else None
        gsm.tch_fr_encode_into(self.buf, facch if facch is not None else voice)

    def _sacch(self, tn, fn_air):
        if not gsm.sacch_tf_block_start(fn_air, tn):
            return
        block = fn_air - (fn_air % 104)
        if self.uplink.sacch_l2 is None or self.sent_sacch_block == block:
            return
        self.sent_sacch_block = block
        for j, burst in enumerate(gsm.xcch_encode(self.uplink.sacch_l2)):
            self.tx.schedule(tn, (fn_air + 26 * j) % gsm.HYPERFRAME, burst, True)

    def run(self):
        fn_air = None
        while True:
            tn = self.tch.active_tn()
            if tn is None or not self.tch.is_open():
                fn_air = None
                time.sleep(0.05)
                continue
            if fn_air is None:
                fn_air = (self.clock.fn() + self.cfg.ul_fn_advance + 1) % gsm.HYPERFRAME
            else:
                fn_air = (fn_air + 1) % gsm.HYPERFRAME
            self.clock.sleep_until_fn((fn_air - self.cfg.ul_fn_advance) % gsm.HYPERFRAME)
            self._sacch(tn, fn_air)
            if not gsm.is_tch_carrier(fn_air):
                continue
            bid = gsm.tch_burst_index(fn_air) % 4
```

```
            if bid == 0:
                self._load_block()
            elif not (self.mask & 0x01):
                continue
            base = bid * BPLEN
            burst = gsm.burst_from_coded(bytes(self.buf)[base:base + BPLEN])
            self.mask |= 1 << bid
            self.tx.schedule(tn, fn_air, burst, True)
            self.stats.tch_ul_bursts += 1
```

2.176 /opt/GSM/osmo-operator/pont/voix_hote.py

4844 octets, 107 lignes → 107 lignes

```
"""voix_hote.py - the voice of the C139 Magnetite coded by the host (PONT_VOIX_HOTE=1).

The mask-ROM vocoder does not keep up: the emulated C54x runs at ~9.5 ms per frame when
the FR codec runs (budget 4.62 ms), the ROM decodes the downlink TCH as BFI (speech, FACCH,
SACCH) and never fills a_du. The bridge already decodes every FR frame the BTS sends
(Downlink._tch) and already codes the uplink TCH (TchScheduler). Only the PCM is added:

    DL: FR frame from the BTS --libgsm--> ring /dev/shm/pont_voix_dl --pacat--> gsm_audio
    UL: gsm_mic.monitor --parec--> ring /dev/shm/pont_voix_ul --libgsm--> uplink.q_voice

The DSP only does the channel: CALYPSO_VSP=0 (no voice port, no ROM codec). Enabled by
--gapk with --ms-magnetite (startup_scripts/14-demarrage.sh). FR frames are in the
libosmocoding net_order=1 format (TS 101 318, leading 0xD nibble), the one of libgsm.
"""
import ctypes
import logging
import os
import struct
import threading
import time

from . import vsp_audio

log = logging.getLogger("pont")

SHM_UL = os.environ.get("PONT_VOIX_SHM_UL", "/dev/shm/pont_voix_ul")
SHM_DL = os.environ.get("PONT_VOIX_SHM_DL", "/dev/shm/pont_voix_dl")
N = vsp_audio.N
ECH = 160                                   # 20 ms at 8 kHz, one FR frame

_gsm = ctypes.CDLL("libgsm.so.1")
_gsm.gsm_create.restype = ctypes.c_void_p
_gsm.gsm_decode.argtypes = [ctypes.c_void_p, ctypes.c_char_p, ctypes.POINTER(ctypes.c_short)]
_gsm.gsm_encode.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_short), ctypes.c_char_p]


class VoixHote:
    """Replaces the downlink TCH ring of the DSP bridge (DownlinkDsp.ring): publish() receives
    every FR frame decoded from the BTS. A thread codes the microphone and feeds the uplink TCH."""

    def __init__(self, uplink, tch, stats):
        self.uplink = uplink
        self.tch = tch
        self.stats = stats
        self.dec = _gsm.gsm_create()
        self.enc = _gsm.gsm_create()
        self.ul = vsp_audio.anneau(SHM_UL)
        self.dl = vsp_audio.anneau(SHM_DL)
        self.pcm = (ctypes.c_short * ECH)()
        self.n_dl = self.n_dl_ko = self.n_ul = 0
```

```
        # a single source for uplink speech: not the one of montant.c (a_du, /dev/shm/calypso_tch_ul)
        uplink._poll_voice = lambda: None

    # -- downlink ------------------------------------------------------------------------
    def publish(self, fr33, fn):
        if len(fr33) != 33 or _gsm.gsm_decode(self.dec, bytes(fr33), self.pcm) != 0:
            self.n_dl_ko += 1
            return
        w = struct.unpack_from("<I", self.dl, 0)[0]
        for i in range(ECH):
            struct.pack_into("<h", self.dl, 8 + 2 * ((w + i) % N), self.pcm[i])
        struct.pack_into("<I", self.dl, 0, (w + ECH) & 0xFFFFFFFF)   # counter last
        self.n_dl += 1

    # -- uplink --------------------------------------------------------------------------
    def _codeur(self):
        lu = struct.unpack_from("<I", self.ul, 0)[0]
        pcm = (ctypes.c_short * ECH)()
        out = ctypes.create_string_buffer(33)
        while True:
            w = struct.unpack_from("<I", self.ul, 0)[0]
            dispo = (w - lu) & 0xFFFFFFFF
            if dispo > N // 2:                      # behind (bridge paused, parec burst): catch up
                lu = (w - ECH) & 0xFFFFFFFF
                dispo = ECH
            if dispo < ECH:
                time.sleep(0.005)
                continue
            for i in range(ECH):
                pcm[i] = struct.unpack_from("<h", self.ul, 8 + 2 * ((lu + i) % N))[0]
            lu = (lu + ECH) & 0xFFFFFFFF
            if not self.tch.is_open():
                continue
            _gsm.gsm_encode(self.enc, pcm, out)
            with self.uplink.q_lock:
                if len(self.uplink.q_voice) == self.uplink.q_voice.maxlen:
                    self.stats.tch_ul_dropped += 1
                self.uplink.q_voice.append(out.raw)
            self.stats.tch_ul += 1
            self.n_ul += 1

    def _bilan(self):
        while True:
            time.sleep(30)
            log.info("voix hote : DL %d trames jouees (%d illisibles), UL %d trames codees en 30 s",
                     self.n_dl, self.n_dl_ko, self.n_ul)
            self.n_dl = self.n_dl_ko = self.n_ul = 0

    def start(self):
        vsp_audio.micro_reel()
        for nom, cible, args in (("voix-micro", vsp_audio.micro, (self.ul,)),
                                 ("voix-hp", vsp_audio.haut_parleur, (self.dl,)),
                                 ("voix-codeur", self._codeur, ()),
                                 ("voix-bilan", self._bilan, ())):
            threading.Thread(name=nom, target=cible, args=args, daemon=True).start()
        log.info("voix hote : FR de la BTS -> %s -> %s ; %s -> %s -> TCH montant (libgsm)",
                 SHM_DL, vsp_audio.SINK, vsp_audio.SOURCE, SHM_UL)
```

2.177 /opt/GSM/osmo-operator/pont/vsp_audio.py

7277 octets, 176 lignes → 176 lignes

```
"""vsp_audio.py - the voice port (VSP) of the Calypso DSP connected to PulseAudio.
```

```
c54x_exe models the SP0 serial port of the ROM (qosmo
hw/arm/calypso/l1-dsp/calypso_vsp.c, CALYPSO_VSP=1): for every sample it sets
DRR from the ring /dev/shm/calypso_pcm_ul and picks DXR up into
/dev/shm/calypso_pcm_dl, at exactly 8 kHz relative to the BTS fn. This module
does the rest, outside the DSP real-time loop:

    gsm_mic.monitor --parec--> calypso_pcm_ul      (the mobile's microphone)
    calypso_pcm_dl  --pacat--> gsm_audio            (its loudspeaker)

These are the points where GAPK plugs the mobile (alsa gsm_in / gsm_out) and
the ones the "voice" stage of banc-max injects into and listens to: the test
stays the same judge.

Rings: header uint32 written (count of samples written), uint32 size, then
size x int16. A single writer per ring. Started by c54x_exe/run.sh (step 5b)
when CALYPSO_VSP=1:  python3 -m pont.vsp_audio
"""
import fcntl
import mmap
import os
import struct
import subprocess
import sys
import termios
import threading
import time

SHM_UL = "/dev/shm/calypso_pcm_ul"
SHM_DL = "/dev/shm/calypso_pcm_dl"
N = 16000                       # = VSP_SHM_N (calypso_vsp.h)
TAILLE = 8 + 2 * N
TAUX = 8000
SOURCE = os.environ.get("VSP_SOURCE", "gsm_mic.monitor")
SINK = os.environ.get("VSP_SINK", "gsm_audio")
GIGUE = int(os.environ.get("VSP_GIGUE_MS", "60")) * TAUX // 1000
DL_CIBLE = int(os.environ.get("VSP_DL_CIBLE_MS", "150")) * TAUX // 1000   # pipe to pacat


def anneau(chemin):
    fd = os.open(chemin, os.O_RDWR | os.O_CREAT, 0o666)
    os.ftruncate(fd, TAILLE)
    m = mmap.mmap(fd, TAILLE)
    os.close(fd)
    struct.pack_into("<I", m, 4, N)
    return m


def journal(*a):
    print("[vsp-audio]", *a, flush=True)


def micro(ul):
    """parec -> UL ring (single writer)."""
    cmd = ["parec", f"--device={SOURCE}", "--format=s16le", f"--rate={TAUX}", "--channels=1",
           "--raw", "--latency-msec=20"]
    p = subprocess.Popen(cmd, stdout=subprocess.PIPE)
    journal("micro :", " ".join(cmd))
    w = struct.unpack_from("<I", ul, 0)[0]
    while True:
        bloc = p.stdout.read(320)                    # 20 ms
        if not bloc:
            journal("parec s'est arrete"); return
        for (ech,) in struct.iter_unpack("<h", bloc[: len(bloc) // 2 * 2]):
            struct.pack_into("<h", ul, 8 + 2 * (w % N), ech)
            w = (w + 1) & 0xFFFFFFFF
        struct.pack_into("<I", ul, 0, w)             # counter last
```

```
def en_attente(fd):
    """Bytes already written to the pipe to pacat and not yet read by it."""
    try:
        return struct.unpack("i", fcntl.ioctl(fd, termios.FIONREAD, b"\0\0\0\0"))[0]
    except OSError:
        return 0


def haut_parleur(dl):
    """DL ring -> pacat, with bounded delay.

    pacat consumes at exactly 8 kHz: silence added while the ring is empty
    (DSP behind), then DSP samples arriving in a burst, would pile up in the
    pipe (64 KiB = 4 s) without ever draining. The pipe is therefore only
    filled up to DL_CIBLE (default 150 ms); silence is only used if the pipe
    is about to run dry; the ring keeps at most GIGUE samples of delay (the
    surplus is dropped, once, and counted)."""
    cmd = ["pacat", "--playback", f"--device={SINK}", "--format=s16le", f"--rate={TAUX}",
           "--channels=1", "--raw", "--latency-msec=60"]
    p = subprocess.Popen(cmd, stdin=subprocess.PIPE)
    journal("haut-parleur :", " ".join(cmd))
    fd = p.stdin.fileno()
    cible = DL_CIBLE
    lu = None
    n_total = n_silence = n_jete = 0
    file_max = 0
    t_journal = time.monotonic()
    while True:
        w = struct.unpack_from("<I", dl, 0)[0]
        if lu is None:
            lu = (w - GIGUE) & 0xFFFFFFFF
        dispo = (w - lu) & 0xFFFFFFFF
        if dispo > N or dispo > GIGUE + cible:           # ring behind: catch up
            n_jete += (dispo - GIGUE) if dispo <= N else 0
            lu = (w - GIGUE) & 0xFFFFFFFF
            dispo = GIGUE
        file = en_attente(fd) // 2
        file_max = max(file_max, file)
        manque = cible - file
        if manque >= 160:
            k = min(dispo, manque)
            out = bytearray()
            for i in range(k):
                out += dl[8 + 2 * ((lu + i) % N): 10 + 2 * ((lu + i) % N)]
            lu = (lu + k) & 0xFFFFFFFF
            n_total += k
            if file + k < 320:                           # the pipe is about to run dry: silence
                out += bytes(2 * (320 - file - k)); n_silence += 320 - file - k
            if out:
                p.stdin.write(out); p.stdin.flush()
        time.sleep(0.01)
        if time.monotonic() - t_journal > 30:
            journal(f"DL : {n_total} echantillons du DSP, {n_silence} de silence, {n_jete} jetes (retard), "
                    f"tuyau vers pacat <= {file_max * 1000 // TAUX} ms (cible {cible * 1000 // TAUX} ms) en 30 s")
            n_total = n_silence = n_jete = file_max = 0; t_journal = time.monotonic()


def pactl(*args):
    r = subprocess.run(["pactl", *args], capture_output=True, text=True)
    return r.stdout if r.returncode == 0 else ""


def micro_reel():
    """The operator's real microphone up to gsm_mic, hence up to DRR.
```

```
    The wiring exists (lib/audio.sh: echo canceller osmo_mic_ec, then
    module-loopback osmo_mic_ec -> gsm_mic), but lib/audio.sh only sets the
    volume when the canceller is created; a hardware input at 0 % or a muted
    osmo_mic_ec leaves the DSP with only the tone injected by banc-max. Mute
    and volume are therefore checked again here, at every start of the voice
    port. VSP_MICRO_REEL=0 leaves the microphone as is."""
    if os.environ.get("VSP_MICRO_REEL", "1") == "0":
        return
    sources = [l.split("\t")[1] for l in pactl("list", "short", "sources").splitlines() if "\t" in l]
    materiel = [s for s in sources if s.startswith("alsa_input") and not s.endswith(".monitor")]
    micro = "osmo_mic_ec" if "osmo_mic_ec" in sources else (materiel[0] if materiel else None)
    if not micro:
        journal("micro reel : aucune entree materielle")
        return
    volume = os.environ.get("VSP_MICRO_VOLUME", os.environ.get("EC_MIC_VOLUME", "25%"))
    for s in dict.fromkeys(materiel[:1] + [micro]):
        pactl("set-source-mute", s, "0")
        if " 0%" in pactl("get-source-volume", s) or "/   0%" in pactl("get-source-volume", s):
            pactl("set-source-volume", s, volume)
    boucles = [l for l in pactl("list", "short", "modules").splitlines()
               if "module-loopback" in l and "sink=gsm_mic" in l]
    if not any(f"source={micro} " in l + " " for l in boucles):
        pactl("load-module", "module-loopback", f"source={micro}", "sink=gsm_mic",
              "sink_dont_move=true", "source_dont_move=true", "latency_msec=40")
        journal(f"micro reel : bouclage {micro} -> gsm_mic pose")
    journal(f"micro reel : {micro} -> gsm_mic (sourdine levee, volume {volume} si a 0)")


def main():
    ul, dl = anneau(SHM_UL), anneau(SHM_DL)
    journal(f"{SOURCE} -> {SHM_UL} ; {SHM_DL} -> {SINK} ; gigue {GIGUE} echantillons")
    micro_reel()
    threading.Thread(target=micro, args=(ul,), daemon=True).start()
    haut_parleur(dl)


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        sys.exit(0)
```

2.178 /opt/GSM/osmo-operator/pont/dsp/init.py

5062 octets, 111 lignes → 111 lignes

```
# pont/dsp - the bridge of the DSP setup (c54x_exe --arm, C54x BSP on udp/6702).
#
# The dedicated state of the pont/ package (TCH, ciphering, Kc) lives at the
# bridge's time, that is the BTS's. In DSP mode the BSP plays the frames at
# the C54x's time, 12 to ~200 frames later: a decision taken at decoding time
# would hit frames the mobile has not heard yet. This sub-package assembles
# the same building blocks with DSP-specific subclasses:
#   ClockDsp    the clock advance follows the real advance of the BTS
#               (osmotrx fn-advance and its jolts), measured continuously.
#   TchDsp      the ASSIGNMENT COMMAND is only an announcement; c54x_exe
#               switches the BSP on the firmware's TCH task (montant.c);
#               abandon() gives the SDCCH back after an ASSIGNMENT FAILURE.
#   TrxDsp      the TCH timeslot is deciphered from the announcement on.
#   UplinkDsp   an uplink SDCCH block is never a FACCH; the Kc is not
#               dropped at the release seen by the mobile.
#   FeederDsp,
#   DownlinkDsp no GSMTAP 4730/4731 nor calypso_tch_dl ring (grgsm).
```

```
# pont/__init__.py (and so pont.py, the grgsm setup) never imports this
# sub-package.
import gc
import logging
import os
import signal
import sys
import time

from .. import _guarded
from .. import config as _config
from ..cipher import Cipher
from ..gsm import load_timeslots, plan_for
from ..record import Recorder
from ..stats import Reporter, Stats
from ..trx import Transmitter
from ..uplink import TchScheduler
from .clock import ClockDsp
from .dedicated import DedicatedDsp
from .downlink import DownlinkDsp, FeederDsp
from .tch import TchDsp
from .trx import TrxDsp
from .uplink import UplinkDsp

log = logging.getLogger("pont")


def main(argv=None):
    cfg = _config.parse(argv)
    if not cfg.dsp_port:
        sys.exit("pont DSP : --dsp-port (ou PONT_DSP_PORT) est obligatoire, 6702 pour c54x_exe ; "
                 "pour le montage grgsm, lancer pont/pont.py")
    logging.basicConfig(stream=sys.stdout, level=logging.INFO,
                        format="%(asctime)s [pont] %(message)s", datefmt="%H:%M:%S")
    gc.disable()
    gc.freeze()

    stats = Stats()
    clock = ClockDsp()
    cipher = Cipher(cfg.kc_retention, stats)
    dedicated = DedicatedDsp()     # TCH kinds from the tap, see dsp/dedicated.py
    tch = TchDsp(stats)
    record = Recorder(cfg, clock) if cfg.record else None
    trx = TrxDsp(cfg, clock, stats, cipher, record, dedicated, tch)
    transmitter = Transmitter(cfg, clock, trx, stats)
    feeder = FeederDsp(cfg, stats)
    timeslots = load_timeslots(cfg.bsc_cfg)
    downlink = DownlinkDsp(cfg, stats, cipher, dedicated, tch, feeder, record, timeslots)
    uplink = UplinkDsp(cfg, clock, stats, dedicated, tch, transmitter, feeder, cipher)
    scheduler = TchScheduler(cfg, clock, stats, tch, uplink, transmitter)
    if os.environ.get("PONT_VOIX_HOTE", "0") == "1":
        # the Magnetite voice by the host (libgsm), see pont/voix_hote.py
        from ..voix_hote import VoixHote
        voix = VoixHote(uplink, tch, stats)
        downlink.ring = voix
        voix.start()
    if os.environ.get("PONT_UL_RAW", "0") == "1":
        # single pipe: the ROM bursts leave through `transmitter`, the ones the host still
        # codes (RACH, SDCCH, FACCH, SACCH, speech) go to the sink. See dsp/uplink.py RomTxRing.
        from .uplink import TransmitterPuits
        uplink.tx_rom = transmitter
        uplink.tx = TransmitterPuits(stats, transmitter, tch)   # the TCH goes to the real Transmitter (UL_RAW_TCH)
        scheduler.tx = uplink.tx
        log.info("PONT_UL_RAW=1 : montant = bursts finaux de la ROM (calypso_tx_rom) ; codage et A5 hote inactifs")

    log.info("pont TRX : ports %d/%d/%d, ARFCN %d, BSIC %d, avance UL %d trames",
```

```
            cfg.trx_base, cfg.trx_base + 1, cfg.trx_base + 2, cfg.arfcn, cfg.bsic, cfg.ul_fn_advance)
    log.info("pont DSP : bursts vers le BSP udp/%d, bascule TCH suivie par le firmware (montant.c), "
             "retard UL max %d trames, retention Kc %s",
             cfg.dsp_port, cfg.ul_retard_max, "oui" if cfg.kc_retention else "non")
    for tn in sorted(timeslots):
        plan = plan_for(timeslots[tn])
        log.info("TS%d %-12s %s", tn, timeslots[tn], ("plan %s, %d sous-voies" % (plan.name, plan.n_sub)) if plan
↪   else "")
    log.info("QEMU : pas de GSMTAP 4730/4731 (couche 1 gr-gsm desactivee) ; tap GSMTAP %s",
             ("udp/%d" % cfg.tap_port) if cfg.tap else "coupe")

    def _dl(tn, fn, bits):
        clock.note_dl(fn)       # real advance of the BTS, see dsp/clock.py
        downlink.dispatch(tn, fn, bits)

    threads = [
        _guarded("ctrl", trx.run_ctrl),
        _guarded("clock", trx.run_clock),
        _guarded("data", lambda: trx.run_data(_dl)),
        transmitter, uplink, scheduler,
        Reporter(stats, clock, dedicated, record),
    ]
    if record:
        signal.signal(signal.SIGUSR1, record.split)
        threads.append(record)
    for t in threads:
        t.start()
    while True:
        time.sleep(3600)
```

2.179 /opt/GSM/osmo-operator/pont/dsp/clock.py

5429 octets, 110 lignes → 110 lignes

```
# pont/dsp/clock.py - the DSP bridge clock: the advance follows the BTS.
#
# THE BTS ADVANCE IS CARRIED DYNAMICALLY OVER TO THE DSP.
#
# Clock (pont/trx.py) slaves the bench clock to "frame requested by the DSP
# + PONT_HORLOGE_AVANCE (12)". The BTS is the one transmitting, and its downlink
# bursts leave `osmotrx fn-advance` frames BEFORE that clock (2 by default),
# plus its jolts: "We missed 6 timers", "5 FN slower than TRX", then a block of
# catch-up. The margin actually useful to the BSP -- last frame received from
# the BTS minus frame requested by the DSP -- is 12 + fn-advance - jitter, which
# can fall below zero: frames without a burst are played as erasures, and a
# lost UA or FACCH block means "I frame ignored in state SABM_SENT" (the UA is
# missing, the following I frames arrive) or T200 on the assignment FACCH.
#
# ClockDsp measures at each regulation the real advance of the BTS over the
# clock (last downlink frame received - clock frame), keeps its MINIMUM over a
# sliding window (the BTS stalls are the dips), and sets
#     advance = PONT_MARGE_DL - min_BTS_advance
# so that "last frame received - frame requested" stays >= PONT_MARGE_DL.
# Changing `osmotrx fn-advance` live on the BTS is followed without a restart.
# The advance is bounded to [PONT_AVANCE_MIN, PONT_AVANCE_MAX]: beyond that the
# uplink leaves correspondingly later (Transmitter, PONT_UL_RETARD_MAX).
# PONT_MARGE_DL=0 falls back to Clock's fixed advance.
# Default 16: a margin of 8 doubles the frames without a burst (the 50 ms
# sampling does not see the worst BTS dips).
#
# THE BTS ADVANCE IS MEASURED AGAINST THE TARGET POINT, NOT THE CLOCK.
# Measured against the real clock (last received - clock), it would assume
# clock == DSP + advance. That is false as soon as the DSP waits for the BTS
# (CALYPSO_BSP_ATTENTE_MS): the DSP then sticks to the last received frame, the
```

```
# clock trails ~15 frames behind its target, the "BTS advance" read goes up and
# the target advance falls to the floor, leaving a holed downlink SDCCH (lost UA,
# the mobile repeats its SABM). So the measure is last received - (DSP +
# advance): the real margin, minus the advance already requested -- a closed
# loop on what must be held. Floor at 10 (the fixed advance 12 held at 1.6 %).
import collections
import logging
import os
import time

from .. import gsm
from ..trx import Clock

log = logging.getLogger("pont")

MARGE_DL = int(os.environ.get("PONT_MARGE_DL", "16"))
AVANCE_MIN = int(os.environ.get("PONT_AVANCE_MIN", "10"))
AVANCE_MAX = int(os.environ.get("PONT_AVANCE_MAX", "40"))
FENETRE_S = float(os.environ.get("PONT_MARGE_FENETRE", "3.0"))


class ClockDsp(Clock):
    def __init__(self):
        super().__init__()
        self._fn_dl = None                  # last downlink frame received from the BTS
        self._mesures = collections.deque() # (time, BTS advance over the clock)
        self._avance_dite = None

    def note_dl(self, fn):
        """Called by the TRXD thread on each downlink burst (no lock: a single
        integer assignment)."""
        prec = self._fn_dl
        if prec is None or 0 < (fn - prec) % gsm.HYPERFRAME < gsm.HYPERFRAME // 2:
            self._fn_dl = fn

    def _reguler(self, now):
        du = now >= self._echeance
        super()._reguler(now)                   # refreshes _dsp_prec (requested frame)
        dsp = self._dsp_prec
        if MARGE_DL > 0 and du and dsp is not None and self._fn_dl is not None:
            # BTS advance over the TARGET point (DSP + advance), not over the
            # real clock -- see the file header.
            avance_bts = (self._fn_dl - dsp - self._avance) % gsm.HYPERFRAME
            if avance_bts > gsm.HYPERFRAME // 2:
                avance_bts -= gsm.HYPERFRAME
            m = self._mesures
            m.append((now, avance_bts))
            while m and now - m[0][0] > FENETRE_S:
                m.popleft()
            creux = min(a for _, a in m)
            avance = min(max(MARGE_DL - creux, AVANCE_MIN), AVANCE_MAX)
            if avance != self._avance:
                self._avance = avance
                if self._avance_dite is None or abs(avance - self._avance_dite) >= 3:
                    self._avance_dite = avance
                    log.info("horloge : avance BTS mesuree %+d trames (creux sur %.0f s), "
                             "avance visee %d pour une marge DL >= %d",
                             creux, FENETRE_S, avance, MARGE_DL)
        self._mesurer_marge(now)

    def _mesurer_marge(self, now):
        """REAL margin seen by the BSP: last frame received - frame requested
        by the DSP (calypso_horloge). Logged every 10 s (min/mean): this is
        what must stay > 0, not the target advance."""
        dsp = self._dsp_prec
        if dsp is None or self._fn_dl is None:
```

```
            return
        marge = (self._fn_dl - dsp) % gsm.HYPERFRAME
        if marge > gsm.HYPERFRAME // 2:
            marge -= gsm.HYPERFRAME
        st = getattr(self, "_marges", None)
        if st is None:
            st = self._marges = [now, marge, 0, 0]
        st[1] = min(st[1], marge)
        st[2] += marge
        st[3] += 1
        if now - st[0] >= 10.0:
            log.info("horloge : marge DL reelle (recu - reclame par le DSP) min %+d moy %+.1f "
                     "sur %d mesures, avance visee %d", st[1], st[2] / st[3], st[3], self._avance)
            self._marges = [now, marge, 0, 0]
```

2.180 /opt/GSM/osmo-operator/pont/dsp/dedicated.py

4142 octets, 101 lignes → 101 lignes

```
# pont/dsp/dedicated.py - calypso_dcch_cfg as written by the QEMU tap of the
# DSP setup (qosmo hw/arm/calypso/calypso_dcch_tap.c).
#
# The tap also announces the TCH: kind 2 = TCH/F, 3 = TCH/H. Dedicated.read
# (pont/state.py) keeps only bit 0 of the kind: a kind 2 would become there an
# SDCCH/4 on the TCH's TS -- the uplink SDCCH of an ASSIGNMENT FAILURE would
# then be scheduled on TS2.
#
# Here the TCH is noted separately and plan() keeps returning the LAST SDCCH as
# long as the connection lives: this is what DownlinkDsp, TrxDsp and UplinkDsp
# expect, and it is the right channel if the mobile comes back to it
# (ASSIGNMENT FAILURE). Only 0xFF (return to the common channels) ends the
# connection: UplinkDsp._poll_release therefore does not see the arrival on
# the TCH as a release.
# pont.py (grgsm) keeps state.Dedicated: its layer 1 never writes a kind 2.
import logging
import os
import struct
import time

from ..state import DCCH_CFG, DCCH_RELEASED, DCCH_TTL, Dedicated

log = logging.getLogger("pont")

DCCH_TCH_F = 2
DCCH_TCH_H = 3
# Kind 4 = PDCH (GPRS): the l1gprs_fw.c firmware is on a PDCH; the QEMU tap
# (calypso_dcch_tap.c) publishes it from the L1CTL_GPRS_DL_BLOCK_IND.
DCCH_PDCH = 4


class DedicatedDsp(Dedicated):
    def __init__(self):
        super().__init__()
        self._tch = None        # (kind, ss, tn): the firmware is on the TCH
        self._pdch = None       # tn: the firmware is on a PDCH (GPRS)

    def read(self):
        now = time.monotonic()
        if now < self._next:
            return self._value
        self._next = now + DCCH_TTL
        try:
            if self._fd is None:
                self._fd = os.open(DCCH_CFG, os.O_RDONLY)
```

```
            b = os.pread(self._fd, 16, 0)
        except OSError:
            if self._fd is not None:
                os.close(self._fd)
                self._fd = None
            return self._value
        if len(b) < 8:
            return self._value
        seq = struct.unpack_from("<I", b, 0)[0]
        if not seq or seq == self._seq:
            return self._value
        self._seq = seq
        genre, ss, tn = b[4], b[5] & 7, b[6] & 7
        if genre == DCCH_RELEASED:
            self._value = None
            self._tch = None
            self._pdch = None
        elif genre == DCCH_PDCH:
            self._pdch = tn
            log.info("canal dedie : PDCH TS=%d annonce par le firmware (GPRS)", tn)
        elif genre in (DCCH_TCH_F, DCCH_TCH_H):
            self._pdch_quitte("TCH", tn)
            self._tch = (genre, ss, tn)
            if self._value is None:
                log.info("canal dedie : TCH/%s TS=%d SS=%d annonce par le firmware, aucun SDCCH connu",
                         "F" if genre == DCCH_TCH_F else "H", tn, ss)
            else:
                log.info("canal dedie : TCH/%s TS=%d SS=%d annonce par le firmware "
                         "(SDCCH SS=%d TS=%d garde pour un retour)",
                         "F" if genre == DCCH_TCH_F else "H", tn, ss, self._value[1], self._value[2])
        else:
            self._pdch_quitte("SDCCH", tn)
            if self._tch is not None:
                log.info("canal dedie : le firmware a quitte le TCH TS=%d pour le SDCCH SS=%d TS=%d",
                         self._tch[2], ss, tn)
            self._tch = None
            self._value = (genre & 1, ss, tn)
        return self._value

    def _pdch_quitte(self, quoi, tn):
        # The firmware cannot be on a PDCH AND on an SDCCH/TCH: _pdch is cleared here,
        # otherwise tn_dedie() (dsp/uplink.py), which tests the PDCH first, would send the
        # ROM bursts of the dedicated channel on the PDCH's TS.
        if self._pdch is not None:
            log.info("canal dedie : le firmware a quitte le PDCH TS=%d pour un %s TS=%d", self._pdch, quoi, tn)
            self._pdch = None

    def tch(self):
        """(kind, ss, tn) if the firmware is on a TCH according to the tap, else None."""
        self.read()
        return self._tch

    def pdch(self):
        """tn if the firmware is on a PDCH (GPRS) according to the tap, else None."""
        self.read()
        return self._pdch
```

2.181 /opt/GSM/osmo-operator/pont/dsp/downlink.py

985 octets, 28 lignes → 28 lignes

```
# pont/dsp/downlink.py - the downlink of the DSP bridge, without the grgsm outputs.
#
# In the DSP setup, QEMU runs with CALYPSO_DSP_EXTERN=1: its gr-gsm layer 1 is
```

```
# disabled, and it is the only listener of the GSMTAP udp/4730, the SCH
# udp/4731 and the speech ring /dev/shm/calypso_tch_dl (calypso_l1_grgsm.c:34).
# The DSP receives the raw bursts instead (Trx.run_data -> udp/6702). The
# Wireshark tap (udp/4729) is kept; the bridge's local decoding (statistics,
# ciphering confirmation, ASSIGNMENT COMMAND, CHANNEL RELEASE) is the same.
from ..downlink import Downlink, Feeder


class FeederDsp(Feeder):
    def l2(self, fn, chan, l2, tn=0, uplink=False, to_qemu=True):
        super().l2(fn, chan, l2, tn, uplink, to_qemu=False)

    def sch(self, fn):
        pass


class _NullRing:
    def publish(self, fr33, fn):
        pass


class DownlinkDsp(Downlink):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.ring = _NullRing()
```

2.182 /opt/GSM/osmo-operator/pont/dsp/tch.py

2122 octets, 51 lignes → 51 lignes

```
# pont/dsp/tch.py - the TCH as seen by the DSP bridge: an ANNOUNCEMENT, not a switch.
#
# In the DSP setup, /dev/shm/calypso_tch_cfg is read by c54x_exe
# (src/montant.c, scruter_tch). The BSP plays the frames at the DSP's time,
# 12 to ~200 frames later than the BTS: switching the BSP on the TCH as soon
# as the bridge decodes the ASSIGNMENT COMMAND would replace every frame not
# yet played -- the ASSIGNMENT COMMAND included, and TS0 with it -- by bursts
# of the TCH timeslot. c54x_exe therefore only switches at the first TCH task
# set by the firmware (montant.c, suivre_tache_tch); the write done here is
# only an announcement (timeslot, TSC).
#
# Same file, same format, same write moment as Tch.arm: the grgsm L1
# (calypso_l1_grgsm.c poll_tch_cfg) is not concerned, pont.py does not use
# this class. The only extra case, non-zero seq with tn=0, is the abandon.
import logging

from ..state import Tch

log = logging.getLogger("pont")


class TchDsp(Tch):
    def arm(self, tn, tsc, arfcn):
        with self.lock:
            self.seq += 1
            self.tn = tn
            self.tsc = tsc
            self.open = False
            self._write_cfg(tn, tsc, arfcn, self.seq)
        log.info("ASSIGNMENT COMMAND : TCH TN=%d TSC=%d ARFCN=%d annonce au DSP, "
                 "bascule du BSP a la premiere tache TCH du firmware", tn, tsc, arfcn)

    def abandon(self, reason):
        """The mobile stayed (or came back) on the SDCCH: withdraw the announcement.
```

```
        seq+1 with tn=0: c54x_exe gives the SDCCH back to the BSP WITHOUT
        dropping the Kc (the SDCCH still lives, the BTS still ciphers it).
        seq=0 remains the complete release (Tch.close)."""
        with self.lock:
            if self.tn is None:
                return
            tn = self.tn
            self.seq += 1
            self.tn = None
            self.open = False
            self.open_since = None
            self.release_seen = None
            self._write_cfg(0, 0, 0, self.seq)
        log.info("TCH TN=%d abandonne (%s) : retour sur le SDCCH", tn, reason)
        for cb in self.on_close:
            cb()
```

2.183 /opt/GSM/osmo-operator/pont/dsp/trx.py

1853 octets, 36 lignes → 36 lignes

```
# pont/dsp/trx.py - the Trx of the DSP bridge.
#
# The copy of every burst to the C54x BSP (UDP dsp_port, 8-byte header + 148
# bits 0/1) lives in Trx.run_data: pont.dsp.main refuses to start without
# --dsp-port, so that path is always taken here.
import os

from ..trx import Trx


class TrxDsp(Trx):
    # THE DSP DECIPHERS BY ITSELF. The mask-ROM programs the Calypso A5
    # coprocessor (XIO ports 0x2800..0x2818, see l1-dsp/calypso_a5.c) with
    # a_kc and a_a5fn, and applies the keystream to the demodulated burst.
    # Deciphering here as well would apply the keystream twice (= hand the
    # ciphered burst back to the decoder). The BSP therefore receives the
    # downlink as the BTS sent it. PONT_DSP_DECHIFFRE=1 deciphers here (core
    # without A5 model, or c54x_exe started with CALYPSO_A5=0).
    dechiffre_dl = os.environ.get("PONT_DSP_DECHIFFRE", "0") == "1"

    def _burst_dedie(self, tn, fn):
        """THE TCH IS DECIPHERED FROM THE ANNOUNCEMENT ON.

        Trx._burst_dedie only counts the TCH timeslot as dedicated once
        Tch.prove() has passed, that is after a first uplink frame of the
        mobile on the TCH. But osmo-bts ciphers the TCH in both directions
        from the channel activation on (osmo-bts-trx l1_if.c:479-483): between
        the firmware switch and prove(), the BSP would play ciphered bursts,
        and the first blocks of the BTS -- the UA answering the SABM of the
        ASSIGNMENT -- would be lost on the ROM side. Deciphering stays guarded
        by cipher.dl_active (Trx.run_data), confirmed on the SDCCH of the SAME
        session and reset at the IMMEDIATE ASSIGNMENT: a Kc kept from a
        previous session never touches a TCH in clear."""
        if self.tch is not None and tn == self.tch.active_tn():
            return True
        return super()._burst_dedie(tn, fn)
```

2.184 /opt/GSM/osmo-operator/pont/dsp/uplink.py

23560 octets, 459 lignes → 457 lignes (1 groupes compactés)

```
# pont/dsp/uplink.py - the uplink of the DSP bridge.
#
# The side bands are those of c54x_exe (src/montant.c), same paths and same
# format as in grgsm. Two rules differ, because in DSP mode the origin of each
# block is known and the release is read at the DSP's time.
import collections
import logging
import os
import struct
import time

from .. import gsm
from ..uplink import POLL, RELEASE_FALLBACK, Uplink

log = logging.getLogger("pont")

RR_ASSIGNMENT_FAILURE = 0x2f

# Uplink xCCH bursts emitted by the DSP ROM, published by c54x_exe (src/pont.c,
# tx_rom_publier): the ROM copies each burst from the interleaved buffer 0x4280 to data[0x3f8a]
# at its transmit frame. Ring: header w(u32, number written) n(u32, slots); 1024-byte slot:
# seq(u32) fn_burst0(u32) l2(23) .(1), 4 x 116 ubits (57 data, hl, hu, 57 data), then 4 x 116
# ubits of the uplink A5 keystream the ROM XORs onto the burst in ciphered mode (data[0x3f9b],
# calypso_a5.c). The ROM ciphers at the DSP's time (12 to ~200 frames after the BTS): its XOR is
# undone and the bridge re-ciphers at the air fn (Cipher.apply), as for a host-coded block.
SB_XCCH_UL_ROM = "/dev/shm/calypso_xcch_ul_rom"
ROM_SLOT = 1024
# The ROM's bursts come out 1 to ~5 frames after montant.c publishes the L2 (coding, then one
# frame per burst). Beyond that, the block leaves host-coded. PONT_UL_ROM=0: always the host.
ROM_ATTENTE = float(os.environ.get("PONT_UL_ROM_ATTENTE", "0.12"))

# THE SINGLE PIPE: THE ROM'S FINAL UPLINK BURSTS, AS IS, AT ITS FRAME NUMBER.
#
# c54x_exe (src/tsp_tx.c, pont.c tx_tsp_publier) publishes in calypso_tx_rom each burst the ROM
# has finished building in its TSP script for the ABB: training sequence, tails, I/Q inversion
# and A5 ciphering already applied by the ROM, 148 bits, with the frame number the ARM gave it
# for this burst (= the ciphering one: dsp_tester tx-sdcch-a5, 24/24). With PONT_UL_RAW=1 the
# bridge sends them to the BTS as is, at that fn, without host-side coding or A5: RACH on TS0,
# normal bursts on the TN of the dedicated channel (TCH if open, else the announced SDCCH). The
# BTS accepts an uplink burst whatever its lateness (osmo-bts trx_sched_ul_burst does not reject
# on fn); the classic host path keeps running for logs, stats and state (GSMTAP), but its bursts
# go to a sink (Transmitter replaced by TransmitterPuits). PONT_UL_RAW_DECAL=<n> adds n frames
# to the ROM's fn (0 by default: the ROM ciphers at this fn, the BTS deciphers at this fn).
SB_TX_ROM = "/dev/shm/calypso_tx_rom"
TX_ROM_SLOT = 160
UL_RAW = os.environ.get("PONT_UL_RAW", "0") == "1"
PDCH_FILTRE = os.environ.get("PONT_PDCH_FILTRE", "1") != "0"
# B5: uplink SACCH/T by the ROM. 0 (default) = measurement only (fn lateness,
# available ROM blocks).
UL_RAW_SACCH_T = os.environ.get("PONT_UL_RAW_SACCH_T", "0") == "1"
UL_RAW_DECAL = int(os.environ.get("PONT_UL_RAW_DECAL", "0"))
# THE TCH STAYS WITH THE HOST. With montant.c leaving B_BLUD to the ROM, the ROM does emit the
# SABM FACCH and the SACCH/T on the TCH, but NO speech burst (the `cc 0x8900,tc` serialiser of
# 0x8726 is never called for speech). Sending the ROM's TCH would therefore cut the uplink voice;
# mixing ROM and host bursts on the same TCH breaks the FACCH (osmo-bts decodes at bid 3 of each
# received burst and shifts its buffer at each bid 0). As long as the ROM does not emit speech:
# RACH and SDCCH/SACCH by the ROM, the whole TCH (FACCH, SACCH/T, speech) by the host.
# PONT_UL_RAW_TCH=1: everything by the ROM (uplink voice cut).
UL_RAW_TCH = os.environ.get("PONT_UL_RAW_TCH", "0") == "1"
# GRACE DELAY BEFORE ABANDONING THE TCH ON AN UPLINK SDCCH BLOCK (seconds).
# Right after the SABM FACCH, a block may still show up on the SDCCH path: it is the tail of
# the firmware's SDCCH task (ABORT after CAS_C), not a return to the SDCCH. Abandoning the TCH
# on it would leave the SABM uncoded in FACCH ("DL: >N200 of SABM" -> RR ASSIGNMENT FAILURE ->
# MSC "Assignment Failure, releasing call"), so during this delay such a block is ignored. A
# real return is announced by an ASSIGNMENT FAILURE, handled before and without delay.
TCH_ABANDON_GRACE = float(os.environ.get("PONT_TCH_ABANDON_GRACE", "2.0"))
```

```
# Magnetite (TI firmware): the ROM does not emit its bursts through the TSP script read by
# calypso_tx_rom, except the RACH that c54x_exe publishes there itself (tx_rach_vers_air). The SABM
# and the SACCH, captured in a_cu by montant.c (calypso_sdcch_ul), would go to the sink.
# PONT_UL_RAW_DCCH_HOTE=1: SDCCH and SACCH coded by the host and sent to the BTS, the rest of the
# ROM pipe unchanged.
UL_RAW_DCCH_HOTE = os.environ.get("PONT_UL_RAW_DCCH_HOTE", "0") == "1"
UL_TN_DEFAUT = int(os.environ["PONT_UL_TN_DEFAUT"]) if os.environ.get("PONT_UL_TN_DEFAUT", "") != "" else None


class RomTxRing:
    """Reader of calypso_tx_rom: (seq, fn, type, tsc, bits[148]) in write order."""
    def __init__(self):
        self.fd = None
        self.seq = 0            # last seq consumed

    def nouveaux(self):
        try:
            if self.fd is None:
                self.fd = os.open(SB_TX_ROM, os.O_RDONLY)
            hdr = os.pread(self.fd, 8, 0)
        except OSError:
            if self.fd is not None:
                os.close(self.fd)
                self.fd = None
            return []
        if len(hdr) < 8:
            return []
        w, n = struct.unpack("<II", hdr)
        if w <= self.seq:
            return []
        debut = max(self.seq + 1, w - n + 1)
        out = []
        for seq in range(debut, w + 1):
            case = os.pread(self.fd, TX_ROM_SLOT, 8 + ((seq - 1) % n) * TX_ROM_SLOT)
            if len(case) < TX_ROM_SLOT or struct.unpack_from("<I", case, 0)[0] != seq:
                continue
            fn, typ, tsc, _off, drap = struct.unpack_from("<IBBBB", case, 4)
            # flag bit 7: burst ciphered by the ROM (c54x_exe pont.c tx_tsp_publier)
            out.append((seq, fn, typ, tsc, bytes(case[12:12 + 148]), bool(drap & 0x80)))
        self.seq = w
        return out


def classe_tch(fn, burst):
    """Uplink burst of a TCH/F: 'S' SACCH/T (fn%26 = 12, or 25 depending on the TN),
    'F' FACCH (a stealing flag set: bits 60 / 87, either side of the TSC -- same place
    in the ROM's bursts), 'P' speech otherwise."""
    if fn % 26 in (12, 25):
        return "S"
    try:
        if burst[60] or burst[87]:
            return "F"
    except (IndexError, TypeError):
        pass
    return "P"


class TransmitterPuits:
    """Sink for host-coded bursts when PONT_UL_RAW=1: nothing leaves, except those of the TCH
    (PONT_UL_RAW_TCH=0, see UL_RAW_TCH), which go to the real Transmitter."""
    def __init__(self, stats, vrai=None, tch=None):
        self.stats = stats
        self.vrai = vrai
        self.tch = tch
        self.n = 0
```

```
    def schedule(self, tn, fn_air, burst, cipher, essais=0):
        if not UL_RAW_TCH and self.vrai is not None and self.tch is not None and tn == self.tch.active_tn():
            self.stats.air_hote_tch += 1
            self.stats.tch_ul_hote[classe_tch(fn_air, burst)] += 1
            if cipher:
                self.stats.a5_ul_hote_tch += 1
            return self.vrai.schedule(tn, fn_air, burst, cipher, essais)
        self.n += 1
        self.stats.air_puits += 1
  ×3
class RomXcchRing:
    def __init__(self):
        self.fd = None
        self.vus = set()
        self.dechiffres = 0     # blocks the ROM had ciphered (XOR undone here)

    def chercher(self, l2):
        """The 4 bursts (116 ubits each) of the most recent block whose RE-DECODED bits give
        `l2`; (None, n_ko) otherwise. n_ko = recent ROM blocks that do not re-decode (bad CRC)."""
        try:
            if self.fd is None:
                self.fd = os.open(SB_XCCH_UL_ROM, os.O_RDONLY)
            hdr = os.pread(self.fd, 8, 0)
        except OSError:
            if self.fd is not None:
                os.close(self.fd)
                self.fd = None
            return None, 0
        if len(hdr) < 8:
            return None, 0
        w, n = struct.unpack("<II", hdr)
        ko = 0
        for seq in range(w, max(0, w - n), -1):
            case = os.pread(self.fd, ROM_SLOT, 8 + ((seq - 1) % n) * ROM_SLOT)
            if len(case) < 32 + 8 * 116 or struct.unpack_from("<I", case, 0)[0] != seq or seq in self.vus:
                continue
            bits = case[32:32 + 4 * 116]
            flux = case[32 + 4 * 116:32 + 8 * 116]
            coded = [bits[k * 116:(k + 1) * 116] for k in range(4)]
            dec = gsm.xcch_decode(coded)
            chiffre = False
            if dec is None and any(flux):
                coded = [bytes(a ^ b for a, b in zip(bits[k * 116:(k + 1) * 116], flux[k * 116:(k + 1) * 116]))
                         for k in range(4)]
                dec = gsm.xcch_decode(coded)
                chiffre = True
            if dec is None:
                ko += 1
                self.n_ko = getattr(self, "n_ko", 0) + 1
                if self.n_ko <= 5:
                    log.info("xCCH ROM (0x3f8a) seq=%d : CRC faux en clair%s", seq,
                             " et apres XOR du flux A5" if any(flux) else ", aucun flux A5 joint")
                continue
            if bytes(dec[:gsm.MACBLOCK_LEN]) == bytes(l2[:gsm.MACBLOCK_LEN]):
                self.dechiffres += chiffre
                self.vus.add(seq)
                if len(self.vus) > 64:
                    self.vus = set(x for x in self.vus if x > w - n)
                return [gsm.burst_from_coded(c) for c in coded], ko
        return None, ko


class UplinkDsp(Uplink):
    rom = None
    rom_attente = None
```

```
    def _bursts_xcch(self, l2):
        """THE UPLINK SDCCH/SACCH BURSTS ARE THE ROM'S.

        The DSP ROM does the channel coding (FIRE, convolutional, interleaving); c54x_exe publishes
        the bursts it emits (data[0x3f8a]) in calypso_xcch_ul_rom. They are taken if they re-decode
        to this L2; while they are not out yet (ROM_ATTENTE), the block waits; beyond that, or if
        PONT_UL_ROM=0, it is coded by the host (counter xCCH ul hote)."""
        if os.environ.get("PONT_UL_ROM", "1") == "0":
            return Uplink._bursts_xcch(self, l2)
        if self.rom is None:
            self.rom = RomXcchRing()
        cle = bytes(l2[:gsm.MACBLOCK_LEN])
        if self.rom_attente is None or self.rom_attente[0] != cle:
            self.rom_attente = (cle, time.monotonic())
        bursts, ko = self.rom.chercher(l2)
        if bursts is not None:
            self.rom_attente = None
            self.stats.xcch_ul_rom += 1
            if self.stats.xcch_ul_rom <= 3 or self.rom.dechiffres == 1 and not getattr(self, "_dech_dit", False):
                self._dech_dit = self.rom.dechiffres > 0
                log.info("xCCH montant : bursts de la ROM (0x3f8a) pour %s%s", cle[:6].hex(" "),
                         " (chiffres par la ROM, XOR A5 defait, rechiffres par le pont au fn de l'air)"
                         if self.rom.dechiffres else "")
            return bursts
        if time.monotonic() - self.rom_attente[1] < ROM_ATTENTE:
            return None
        self.rom_attente = None
        self.stats.xcch_ul_rom_ko += ko
        if UL_RAW and not UL_RAW_DCCH_HOTE:
            # B2: under PONT_UL_RAW=1 the host copy would end in the sink, so it is not
            # coded. Empty list = _poll_sdcch keeps its effects (GSMTAP, sacch_ul, routing)
            # without scheduling anything. What goes on air is the ROM burst of calypso_tx_rom.
            self.stats.xcch_shadow_ko += 1
            if self.stats.xcch_shadow_ko <= 5:
                log.info("xCCH montant : %s sans bursts 0x3f8a apres %.0f ms (%d bloc(s) sans CRC) -- "
                         "non recode (PONT_UL_RAW=1, l'antenne a ceux de calypso_tx_rom)",
                         cle[:6].hex(" "), ROM_ATTENTE * 1000, ko)
            return []
        if self.stats.xcch_ul_hote < 5:
            log.info("xCCH montant : pas de bursts ROM pour %s apres %.0f ms (%d bloc(s) ROM sans CRC), code par
↪ l'hote",
                     cle[:6].hex(" "), ROM_ATTENTE * 1000, ko)
        return Uplink._bursts_xcch(self, l2)

    def _route_sdcch(self, l2):
        """A BLOCK FROM calypso_sdcch_ul IS NEVER A FACCH.

        In DSP mode, montant.c publishes on this path only the blocks of the
        SDCCH tasks (DUL/AUL, capture_sdcch_ul); those of the TCH go through
        calypso_tch_facch_ul (capture_tch_ul, TCHT). Uplink._route_sdcch would
        send them as FACCH as soon as tch.is_open(), on the TCH's TS.
        Such a block proves on the contrary that the mobile is on the SDCCH: if
        a TCH is announced and this is an ASSIGNMENT FAILURE, or the mobile had
        already moved to the TCH, the announcement is withdrawn (TchDsp.abandon;
        c54x_exe gives TS1 back to the BSP if it has not already done so on the
        firmware task). The block always leaves as SDCCH/SACCH.

        B1: on a PDCH, montant.c also publishes the GPRS a_cu blocks (DUL task)
        on this path. They are not SDCCH: they are dropped (counter pdtch_hote_jete).
        PONT_PDCH_FILTRE=0: keep them."""
        if PDCH_FILTRE and self.dedicated.pdch() is not None:
            self.stats.pdtch_ul_vu_hote += 1
            if self.stats.pdtch_ul_vu_hote == 1:
                log.info("voie SDCCH montante : blocs du PDCH TS%d (GPRS) jetes, la ROM les emet
↪ (PONT_PDCH_FILTRE=0 pour les garder)",
                         self.dedicated.pdch())
```

```
            return True
        if self.tch.active_tn() is not None:
            if gsm.rr_message_type(l2)[1] == RR_ASSIGNMENT_FAILURE:
                self.tch.abandon("ASSIGNMENT FAILURE sur le SDCCH")
            elif self.tch.is_open():
                age = self.tch.open_age()
                if age is not None and age < TCH_ABANDON_GRACE:
                    self._grace_vus = getattr(self, "_grace_vus", 0) + 1
                    if self._grace_vus <= 5:
                     log.info("bloc montant sur le SDCCH %.2f s apres l'ouverture du TCH : ignore (delai de grace "
                                "%.1f s, PONT_TCH_ABANDON_GRACE) -- queue de la tache SDCCH du firmware, pas un
                                 ↪  retour",
                                age, TCH_ABANDON_GRACE)
                    return False
                self.tch.abandon("bloc montant du mobile sur le SDCCH")
        return False

    def _poll_release(self):
        """THE Kc IS NOT DROPPED AT RELEASE.

        The release is read in calypso_dcch_cfg, published when the firmware
        goes back to the BCCH: at the DSP's time, behind the BTS, which still
        ciphers the SDCCH. Dropping the Kc (and dl_active) at that moment would
        leave the following SDCCH blocks undecodable. The Kc and dl_active fall
        at the next IMMEDIATE ASSIGNMENT, decoded at the BTS's time
        (Downlink._signalling), or with a new Kc."""
        active = self.dedicated.plan() is not None
        if self.ded_active and not active:
            reason = "canal libere par le mobile"
        elif self.tch.release_overdue(RELEASE_FALLBACK):
            reason = "CHANNEL RELEASE sans liberation du mobile depuis %ds" % RELEASE_FALLBACK
        else:
            self.ded_active = active
            return
        self.ded_active = active
        self.tch.close(reason)
        log.info("%s : Kc garde jusqu'a la prochaine IMMEDIATE ASSIGNMENT (la BTS chiffre encore)", reason)

    def run(self):
        """PONT_UL_RAW=1: see RomTxRing. The _poll_* keep running (logs, stats, state), their
        bursts go to the sink; the ROM's leave through self.tx_rom.trx.send_ul, IMMEDIATELY: their
        fn is final (it is the ROM's ciphering fn), the BTS files them by fn whatever their lateness
        (osmo-bts trx_sched_ul_burst), and the Transmitter window (PONT_UL_RETARD_MAX, 26 frames)
        would drop part of them once the BTS advance exceeds ~23 frames. The dedicated channel is
        force re-read (refresh) when a normal burst arrives without a known TN -- the 100 ms cache
        would lose the first SDCCH bursts, hence the SABM -- and bursts wait up to 0.3 s for the
        TN to be known."""
        if not UL_RAW:
            return Uplink.run(self)
        ring = RomTxRing()
        n = {0: 0, 1: 0, 2: 0}
        attente = collections.deque()
        perdus = 0
        log.info("PONT_UL_RAW=1 : bursts montants = ceux de la ROM (%s), emis a son fn%s, sans codage ni A5 cote
↪  hote",
                 SB_TX_ROM, (" %+d" % UL_RAW_DECAL) if UL_RAW_DECAL else "")

        def tn_dedie():
            # The TN is the one where the FIRMWARE is, according to the QEMU tap
            # (calypso_dcch_cfg): from the ASSIGNMENT COMMAND on it moves to the TCH and sends
            # there the SABM FACCH, the ASSIGNMENT COMPLETE, the speech; plan() keeps the SDCCH
            # "for a return" and tch.is_open() is only true once proven on the bridge side --
            # the first TCH bursts would otherwise leave on the SDCCH's TS.
            # PDCH (GPRS) first: the uplink PDTCH blocks coded by the ROM (DUL task of the
            # l1gprs_fw.c firmware) leave on the PDCH's TN, at their fn.
            pdch = self.dedicated.pdch()
```

```
            if pdch is not None:
                return pdch
            tchf = self.dedicated.tch()
            if tchf is None:
                self.dedicated.refresh()
                tchf = self.dedicated.tch()
                pdch = self.dedicated.pdch()
                if pdch is not None:
                    return pdch
            if tchf is not None:
                return tchf[2]
            if self.tch.is_open() and self.tch.active_tn() is not None:
                return self.tch.active_tn()
            ded = self.dedicated.plan()
            if ded is None:
                self.dedicated.refresh()
                ded = self.dedicated.plan()
            # Magnetite: no L1CTL tap, hence never a calypso_dcch_cfg. Its SDCCH bursts (TSC 7, 4
            # frames in a row, already in calypso_tx_rom with their fn) would be lost with "no known
            # dedicated channel". PONT_UL_TN_DEFAUT=<tn>: the timeslot to use without an
            # announcement (TS0, SDCCH/4).
            if ded is None and UL_TN_DEFAUT is not None:
                return UL_TN_DEFAUT
            return ded[2] if ded else None

        tch_dit = set()
        tch_rom = [0]

        def tn_tch():
            # TN of the TCH as the firmware sees it (QEMU tap), else the bridge's
            t = self.dedicated.tch()
            return t[2] if t is not None else self.tch.active_tn()

        def envoyer(seq, fn, typ, tsc, bits, tn, a5=False):
            st = self.stats
            est_tch = typ != 1 and tn is not None and tn == tn_tch()
            if est_tch:
                # B5: lateness of the ROM fn against the BTS clock (frames), and available SACCH/T
                r = (self.clock.fn() - fn) % gsm.HYPERFRAME
                if r > gsm.HYPERFRAME // 2:
                    r -= gsm.HYPERFRAME
                if st.tch_rom_retard_min is None or r < st.tch_rom_retard_min:
                    st.tch_rom_retard_min = r
                if st.tch_rom_retard_max is None or r > st.tch_rom_retard_max:
                    st.tch_rom_retard_max = r
                if classe_tch(fn, bits) == "S":
                    self._s_rom = getattr(self, "_s_rom", 0) + 1
                    st.sacch_t_rom_blocs = self._s_rom // 4
            if typ != 1 and not UL_RAW_TCH and tn == self.tch.active_tn():
                st.tch_ul_rom_jete[classe_tch(fn, bits)] += 1
                tch_rom[0] += 1          # see UL_RAW_TCH: the TCH is coded and emitted by the host
                if tch_rom[0] == 1:
                   log.info("bursts ROM du TCH TS%d non emis : FACCH, SACCH/T et parole restent codes par l'hote "
                            "(PONT_UL_RAW_TCH=0), la ROM n'emet pas encore la parole montante", tn)
                return
            self.tx_rom.trx.send_ul(tn, (fn + UL_RAW_DECAL) % gsm.HYPERFRAME, list(bits), False)
            if typ == 1:
                st.air_rom_rach += 1
                ra = gsm.rach_ra(bits, self.cfg.bsic)      # see downlink._imm_ass_dcch
                if ra is not None:
                    from ..downlink import noter_rach
                    noter_rach(ra)
                elif st.air_rom_rach <= 3:
                    log.info("RACH ROM #%d : RA illisible (BSIC %d), IMM ASS choisie sans reference", seq,
↪ self.cfg.bsic)
            elif tn == self.dedicated.pdch():
```

```
                st.air_rom_pdch += 1
            elif typ not in (1, 2):
                st.air_rom_inconnu += 1      # B4: emitted anyway (neighbour of the PDCH in GPRS)
                if st.air_rom_inconnu <= 5:
                    log.info("burst ROM #%d de type inconnu (%d) : TS%d fn=%u fn%%13=%d fn%%26=%d", seq, typ, tn,
                             fn, fn % 13, fn % 26)
            elif est_tch:
                st.air_rom_tch += 1
                st.tch_ul_rom[classe_tch(fn, bits)] += 1
            else:
                st.air_rom_xcch += 1
            if a5 and typ != 1:
                st.a5_ul_rom += 1
            if typ != 1 and self.dedicated.tch() is not None and tn not in tch_dit:
                tch_dit.add(tn)
                log.info("bursts ROM sur le TCH TS%d : parole, FACCH et SACCH montants codes et chiffres par la
↪ ROM", tn)
            k = typ if typ in n else 0
            n[k] += 1
            total = n[0] + n[1] + n[2]
            if total <= 5 or (typ == 1 and n[1] <= 5) or total % 1000 == 0:
                log.info("burst ROM #%d %s fn=%u TS%d%s (RACH %d, NB %d, ? %d, attendus %d, perdus %d)", seq,
                         "RACH" if typ == 1 else "NB" if typ == 2 else "?", fn, tn,
                         (" TSC%d" % tsc) if typ == 2 else "", n[1], n[2], n[0], len(attente), perdus)

        while True:
            self._poll_release()
            self._poll_rach()
            if UL_RAW_DCCH_HOTE:
                # SDCCH and SACCH coded by the host, really sent (the RACH stays with the ROM)
                puits, self.tx = self.tx, self.tx_rom
                try:
                    self._poll_sdcch()
                    self._poll_sacch()
                finally:
                    self.tx = puits
                self._poll_facch()
            else:
                self._poll_sdcch()
                self._poll_facch()
                self._poll_sacch()
            self._poll_voice()
            if attente:
                tn = tn_dedie()
                now = time.monotonic()
                while attente:
                    seq, fn, typ, tsc, bits, a5, t0 = attente[0]
                    if tn is not None:
                        attente.popleft()
                        envoyer(seq, fn, typ, tsc, bits, tn, a5)
                    elif now - t0 > 0.3:
                        attente.popleft()
                        perdus += 1
                        if perdus <= 5:
                            log.info("burst ROM #%d fn=%u (TSC %d) : aucun canal dedie connu apres 300 ms, perdu",
↪ seq, fn, tsc)
                    else:
                        break
            for seq, fn, typ, tsc, bits, a5 in ring.nouveaux():
                if typ == 1:
                    envoyer(seq, fn, typ, tsc, bits, 0)
                    continue
                tn = tn_dedie()
                if tn is None:
                    attente.append((seq, fn, typ, tsc, bits, a5, time.monotonic()))
                    continue
```

```
            envoyer(seq, fn, typ, tsc, bits, tn, a5)
        time.sleep(POLL)
```

2.185 /opt/GSM/osmo-operator/scripts/audio-chain.sh

5595 octets, 117 lignes → 117 lignes

```
#!/usr/bin/env bash
# ============================================================================
#  scripts/audio-chain.sh - closes the local audio chain once pulse is up
# ============================================================================
#  Three links of the audio chain are installed here, in order, idempotently:
#
#   1. /etc/asound.conf. Without it the ALSA PCMs `gsm_out` / `gsm_in` that
#      `mobile` opens do not exist, so the TCH voice never reaches gsm_audio.
#   2. the `gsm_mic` sink. configs/asound.conf points `pcm.gsm_in` at
#      `gsm_mic.monitor`: without this sink, gapk_io fails to open the capture
#      and gives up both directions (gapk_io.c) → call established, totally
#      silent.
#   3. the `module-loopback` gsm_audio.monitor → sound card. gsm_audio is a
#      null-sink: without a consumer the downlink voice is dropped by
#      construction (ALSA output SUSPENDED, pcm0p/sub0/status = "closed").
#
#  Usage: audio-chain.sh [timeout_s]   (default 30)
#  Disabled by AUDIO=0; the loopback alone by AUDIO_LOCAL_LOOPBACK=0.
#  Loopback latency: LOOPBACK_LATENCY_MSEC (default 20).
#  Always exit 0 - audio must never prevent the stack from coming up.
# ----------------------------------------------------------------------------
set -uo pipefail

# --reinit: unload the echo canceller, the loops and osmo_rec, then reinstall
# the chain (lib/audio.sh, reinit_audio). The remaining arguments are unchanged
# (PulseAudio wait delay, 30 s by default).
REINIT=0
if [ "${1:-}" = "--reinit" ]; then REINIT=1; shift; fi
# --appel debut|fin: the heavy chain (echo canceller, loops, osmo_rec) only
# during a call. Called by gapk-start.sh (auto mode) on « Endpoint actif » /
# « Endpoint disparu ». AUDIO_PENDANT_APPEL=0: the chain stays installed
# permanently.
APPEL=""
if [ "${1:-}" = "--appel" ]; then APPEL="${2:-}"; shift 2 2>/dev/null || shift; fi

HERE="${HERE:-$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)}"
export HERE
: "${PULSE_SOCK:=/var/run/pulse/native}"
export PULSE_SERVER="unix:${PULSE_SOCK}"

# shellcheck source=../lib/audio.sh
. "${HERE}/lib/audio.sh"

[ "${AUDIO:-1}" = "1" ] || { echo "[audio] desactive (AUDIO=0)"; exit 0; }

# ── 1. The ALSA mapping, before everything else ──────────────────────────
# `mobile` opens gsm_out/gsm_in: if /etc/asound.conf is missing, it hits
# "Unknown PCM gsm_out" and nothing ever reaches gsm_audio.
if [ -f "${HERE}/configs/asound.conf" ] \
   && ! cmp -s "${HERE}/configs/asound.conf" /etc/asound.conf 2>/dev/null; then
    if cp -f "${HERE}/configs/asound.conf" /etc/asound.conf 2>/dev/null; then
        echo "[audio] /etc/asound.conf deploye (PCM gsm_out/gsm_in)"
    else
        echo "[audio] /etc/asound.conf NON deploye (droits ?)" >&2
    fi
fi
```

```
# ── 2. Wait for the daemon ──────────────────────────────────────────
deadline="${1:-30}"
while [ "$deadline" -gt 0 ] && ! pactl info >/dev/null 2>&1; do
    sleep 1
    deadline=$(( deadline - 1 ))
done
if ! pactl info >/dev/null 2>&1; then
    echo "[audio] PulseAudio injoignable @ ${PULSE_SOCK} - chaine non fermee" >&2
    exit 0
fi

# ── 2 bis. Wait for a hardware output, not only the daemon ──────────
# On a freshly installed disk the daemon answers before the snd_hda_* modules
# have finished loading, so module-udev-detect has not yet declared any ALSA
# sink. module-udev-detect catches cards arriving later, but
# ensure_local_loopback runs only once: if it runs too early, the loopback is
# never installed. It therefore gets the same window as the daemon. Non-blocking
# beyond that: without any card (a bench without sound, a VM without audio
# device) we carry on - the null-sinks are enough for gapk_io, only the local
# listening is missing.
hw_deadline="${1:-30}"
while [ "$hw_deadline" -gt 0 ] \
      && ! pactl list short sinks 2>/dev/null | grep -q 'module-alsa-card'; do
    sleep 1
    hw_deadline=$(( hw_deadline - 1 ))
done

# ── 3. Both null-sinks, then the loopback ──────────────────────────
# load_gsm_sinks() loops over GSM_SINKS = gsm_audio + gsm_mic: this covers a
# system.pa that would declare only one of them.
[ "$REINIT" = 1 ] && reinit_audio
load_gsm_sinks
for s in gsm_audio gsm_mic; do
    if pactl list short sinks 2>/dev/null | grep -qw "$s"; then
        echo "[audio] sink $s present"
    else
        echo "[audio] sink $s MANQUANT - gapk_io va abandonner les deux sens" >&2
    fi
done

# The echo canceller first: the operator's speakers are osmo_hp_ec from boot,
# as with the VM (lib/audio.sh, speaker assignment). Without a VM the direct
# loopback follows; with one it is removed.
alleger_audio
if [ "${AUDIO_PENDANT_APPEL:-0}" = "1" ] && ! pmos_vm_audio_present; then
    case "$APPEL" in
    debut) ;;                                  # the chain is installed below
    fin)   reinit_audio; echo "[audio] fin d appel : chaine retiree"; exit 0 ;;
    *)     # at start / at boot: a fresh bench has no call in progress; the
           # first « Endpoint actif » of gapk-auto will install the chain.
           reinit_audio
           echo "[audio] chaine posee a la demande, au premier appel (AUDIO_PENDANT_APPEL=0 pour la garder)"
           exit 0 ;;
    esac
fi
ensure_echo_cancel
ensure_local_loopback
ensure_local_mic
ensure_record_mix
exit 0
```

2.186 /opt/GSM/osmo-operator/scripts/audio-diag.sh

8323 octets, 177 lignes → 177 lignes

```
#!/bin/bash
# audio-diag.sh - full audio diagnostic inside the container
# Usage: docker exec -ti osmo-operator-1 bash /etc/osmocom/audio-diag.sh
#
# Checks the whole chain: /dev/snd → ALSA → PulseAudio → mobile l1phy

set -euo pipefail

GREEN='\033[0;32m'; RED='\033[0;31m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; NC='\033[0m'; BOLD='\033[1m'

ok()   { echo -e "  ${GREEN}✓${NC} $*"; }
fail() { echo -e "  ${RED}✗${NC} $*"; }
warn() { echo -e "  ${YELLOW}⚠${NC} $*"; }

echo -e "${CYAN}${BOLD}═══════════════════════════════════════════════════${NC}"
echo -e "${CYAN}${BOLD}  Diagnostic audio - container $(hostname)${NC}"
echo -e "${CYAN}${BOLD}═══════════════════════════════════════════════════${NC}"
echo ""

# ── 1. /dev/snd ──────────────────────────────────────────────
echo -e "${BOLD}[1/7] /dev/snd${NC}"
if [ -d /dev/snd ]; then
    ok "/dev/snd present"
    ls -la /dev/snd/ 2>/dev/null | sed 's/^/       /'
else
    fail "/dev/snd ABSENT - le container n'a pas --device /dev/snd"
    echo "       Fix: ajouter --device /dev/snd au docker run"
fi
echo ""

# ── 2. ALSA ──────────────────────────────────────────────────
echo -e "${BOLD}[2/7] ALSA${NC}"
if command -v aplay >/dev/null 2>&1; then
    ok "aplay disponible"
    echo "       Cartes:"
    cat /proc/asound/cards 2>/dev/null | sed 's/^/       /' || fail "pas de /proc/asound/cards"
    echo ""
    echo "       Devices playback:"
    aplay -l 2>/dev/null | grep "^card" | sed 's/^/       /' || warn "aucun device playback"
    echo ""
    echo "       Devices capture:"
    arecord -l 2>/dev/null | grep "^card" | sed 's/^/       /' || warn "aucun device capture"
else
    fail "aplay non installe"
fi
echo ""

# ── 3. PulseAudio ────────────────────────────────────────────
echo -e "${BOLD}[3/7] PulseAudio${NC}"
if [ -n "${PULSE_SERVER:-}" ]; then
    ok "PULSE_SERVER=${PULSE_SERVER}"
    # Test the connection
    if command -v pactl >/dev/null 2>&1; then
        if pactl info >/dev/null 2>&1; then
            ok "PulseAudio accessible"
            pactl info 2>/dev/null | grep -E "Server|Default Sink|Default Source" | sed 's/^/       /'
        else
            fail "PulseAudio inaccessible (socket monte mais serveur non joignable)"
        fi
    else
        warn "pactl non installe - impossible de verifier PulseAudio"
        echo "       Le socket est monte mais on ne peut pas tester sans pactl"
        if [ -S "${PULSE_SERVER#unix:}" ] 2>/dev/null; then
            ok "Socket PulseAudio existe : ${PULSE_SERVER#unix:}"
        else
```

```
            fail "Socket PulseAudio introuvable"
        fi
    fi
else
    warn "PULSE_SERVER non defini"
    echo "       Sans PulseAudio, ALSA utilise directement hw:X,Y"
    echo "       Sur un desktop Linux moderne, le son passe par PulseAudio/PipeWire"
fi
echo ""

# ── 4. ALSA variables ──────────────────────────────────────────────
echo -e "${BOLD}[4/7] Variables d'environnement${NC}"
echo "       ALSA_CARD     = ${ALSA_CARD:-<non defini>}"
echo "       GAPK_ALSA_DEV = ${GAPK_ALSA_DEV:-<non defini>}"
echo "       ALSA_OUTPUT   = ${ALSA_OUTPUT:-<non defini>}"
echo "       ALSA_INPUT    = ${ALSA_INPUT:-<non defini>}"
echo "       PULSE_SERVER  = ${PULSE_SERVER:-<non defini>}"
echo ""

# ── 5. mobile.cfg tch-voice ────────────────────────────────────────
echo -e "${BOLD}[5/7] mobile.cfg tch-voice${NC}"
for cfg in /root/.osmocom/bb/mobile_group*.cfg /root/.osmocom/bb/mobile.cfg; do
    [ -f "$cfg" ] || continue
    echo "       ${cfg}:"
    if grep -q "io-handler l1phy" "$cfg" 2>/dev/null; then
        ok "io-handler l1phy"
        grep -E "alsa-(output|input)-dev" "$cfg" 2>/dev/null | sed 's/^/       /'
    elif grep -q "io-handler gapk" "$cfg" 2>/dev/null; then
        warn "io-handler gapk (audio via osmo-gapk, pas ALSA direct)"
    else
        warn "pas de bloc tch-voice - pas d'audio MS"
    fi
    break
done
echo ""

# ── 6. Quick ALSA test ─────────────────────────────────────────────
echo -e "${BOLD}[6/7] Test ALSA (bip 440Hz 0.2s)${NC}"
if [ -d /dev/snd ]; then
    if command -v speaker-test >/dev/null 2>&1; then
        echo "       Tentative speaker-test (2 secondes max)..."
        timeout 2 speaker-test -t sine -f 440 -l 1 -P 1 2>&1 | head -5 | sed 's/^/       /' || warn "speaker-test
        ↪  echoue"
    elif command -v aplay >/dev/null 2>&1; then
        # Generate a short beep
        python3 -c "
import struct, sys
rate=8000; dur=0.2; freq=440
samples = [int(32767 * __import__('math').sin(2*3.14159*freq*t/rate)) for t in range(int(rate*dur))]
header = struct.pack('<4sI4s4sIHHIIHH4sI', b'RIFF', 36+len(samples)*2, b'WAVE', b'fmt ', 16, 1, 1, rate, rate*2,
↪  2, 16, b'data', len(samples)*2)
sys.stdout.buffer.write(header + struct.pack('<%dh'%len(samples), *samples))
" 2>/dev/null | aplay -q 2>/dev/null && ok "Bip joue sur ALSA" || warn "aplay a echoue"
    fi
else
    fail "Pas de /dev/snd - test impossible"
fi
echo ""

# ── 7. Audio processes ─────────────────────────────────────────────
echo -e "${BOLD}[7/7] Processus audio actifs${NC}"
if pgrep -fa "mobile.*mobile" >/dev/null 2>&1; then
    ok "Processus mobile actif"
    pgrep -fa "mobile" 2>/dev/null | head -3 | sed 's/^/       /'
else
    warn "Processus mobile non detecte (pas encore demarre ?)"
```

```
fi
if pgrep -fa "osmo-gapk" >/dev/null 2>&1; then
    ok "osmo-gapk actif"
    pgrep -fa "osmo-gapk" 2>/dev/null | head -3 | sed 's/^/       /'
else
    echo "       osmo-gapk non actif (normal si l1phy utilise)"
fi
echo ""

# ── Summary ──────────────────────────────────────────────────────────
echo -e "${BOLD}═══ Resume ═══${NC}"
ISSUES=0

if [ ! -d /dev/snd ]; then
    fail "BLOQUANT: /dev/snd absent → pas d'audio possible"
    echo "       Fix: dans start-docker.sh, verifier que --device /dev/snd est dans le docker run"
    ISSUES=$((ISSUES+1))
fi

if [ -z "${PULSE_SERVER:-}" ] && [ -d /dev/snd ]; then
    warn "PulseAudio non configure"
    echo "       Sur un desktop moderne, le device 'default' ALSA necessite PulseAudio"
    echo "       Fix 1: monter le socket PulseAudio dans le container :"
    echo "         -v /run/user/\$(id -u)/pulse/native:/run/user/1000/pulse/native"
    echo "         -e PULSE_SERVER=unix:/run/user/1000/pulse/native"
    echo "       Fix 2: utiliser un device ALSA direct (bypass PulseAudio) :"
    echo "         Trouver le device: aplay -l"
    echo "         Puis: ALSA_OUTPUT=hw:0,0 ALSA_INPUT=hw:0,0 dans start-docker.sh"
    ISSUES=$((ISSUES+1))
fi

if [ -n "${PULSE_SERVER:-}" ]; then
    pulse_sock="${PULSE_SERVER#unix:}"
    if [ ! -S "$pulse_sock" ] 2>/dev/null; then
        fail "Socket PulseAudio monte mais fichier inexistant: $pulse_sock"
        echo "       Le socket PulseAudio de l'hote n'est pas accessible dans le container"
        echo "       Verifier le -v dans le docker run"
        ISSUES=$((ISSUES+1))
    fi
fi

if [ $ISSUES -eq 0 ]; then
    echo -e "  ${GREEN}${BOLD}Audio devrait fonctionner${NC}"
    echo "       Tester: depuis le VTY mobile → call 1 600"
    echo "       Le son sort sur les HP de l'hote via /dev/snd"
fi
echo ""
```

2.187 /opt/GSM/osmo-operator/scripts/banc-repro.sh

7782 octets, 164 lignes → 164 lignes

```
#!/bin/bash
# ================================================================================
#  banc-repro.sh — a reproducible experiment, from start to verdict
# ================================================================================
#  Intermittent call failures are diagnosed by replaying the same sequence
#  after each change and comparing comparable logs. This script freezes the
#  sequence: detached start, wait for camping, N identical calls, timestamped
#  collection, verdict per call.
#
#  It fixes nothing and guesses nothing. It produces a measurement.
#
#  Usage:
```

```
#     scripts/banc-repro.sh                     3 calls, with restart
#     scripts/banc-repro.sh --calls 5           5 calls
#     scripts/banc-repro.sh --no-restart        reuse the running stack
#     scripts/banc-repro.sh --dest 600          called number (default 600)
#
#  Output: a directory /root/repro-<timestamp>/ with all the logs of the run
#  and a resume.txt. Nothing is overwritten from one run to the next.
# ------------------------------------------------------------------------------
set -uo pipefail

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CALLS=3
DEST=600
RESTART=1
VTY=4247                # MS#1; MS#2 is on 4248
CALL_S=20               # duration of one call
GAP_S=10                # silence between two calls (the channel must be released)
BOOT_MAX=180            # maximum wait for camping

while [ $# -gt 0 ]; do
    case "$1" in
        --calls)      CALLS="${2:?}"; shift ;;
        --dest)       DEST="${2:?}"; shift ;;
        --vty)        VTY="${2:?}"; shift ;;
        --call-s)     CALL_S="${2:?}"; shift ;;
        --gap-s)      GAP_S="${2:?}"; shift ;;
        --no-restart) RESTART=0 ;;
        -h|--help)    sed -n '2,30p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo "option inconnue : $1" >&2; exit 2 ;;
    esac
    shift
done

STAMP="$(date +%Y%m%d-%H%M%S)"
OUT="/root/repro-$STAMP"
mkdir -p "$OUT"
LOGDIR=/run/user/0/osmo-nitb/logs

say() { printf '  %s\n' "$*"; }

# -- VTY: one command, one answer ---------------------------------------------
# `enable` then the command. The daemon gets one second to answer: nc closes as
# soon as stdin ends, and without this pause the command is sent but the answer
# is never read.
vty() {
    { printf 'enable\n%s\n' "$1"; sleep 1; } | timeout 6 nc 127.0.0.1 "$VTY" 2>/dev/null
}

# -- 1. Start -----------------------------------------------------------------
# CALYPSO_NO_ATTACH=1: run.sh does not attach to tmux at the end. Without it
# this script would stay blocked on an interactive session.
if [ "$RESTART" = 1 ]; then
    say "arret de la pile en cours"
    ( cd "$HERE" && CALYPSO_NO_ATTACH=1 ./start-native.sh --stop ) >"$OUT/stop.log" 2>&1
    sleep 3
    say "demarrage detache (CALYPSO_NO_ATTACH=1)"
    ( cd "$HERE" && CALYPSO_NO_ATTACH=1 setsid ./start-native.sh ) >"$OUT/start.log" 2>&1 &
else
    say "pile en cours reutilisee (--no-restart)"
fi

# -- 2. Wait for camping, do not assume it ------------------------------------
# The VTY is polled rather than waiting a fixed delay: depending on the machine
# camping takes 40 s or 2 min, and a fixed sleep gives either waste or calls
# started before the mobile is ready — "failures" that measure nothing.
say "attente du camp (C3), maximum ${BOOT_MAX}s"
```

```
camp=0
for i in $(seq 1 "$BOOT_MAX"); do
    if vty "show ms" 2>/dev/null | grep -q "C3 camped normally"; then camp=1; break; fi
    sleep 1
done
if [ "$camp" != 1 ]; then
    say "PAS DE CAMP apres ${BOOT_MAX}s — on s'arrete, les appels ne mesureraient rien"
    vty "show ms" > "$OUT/show-ms-echec.txt" 2>&1
    cp "$LOGDIR"/*.log "$OUT/" 2>/dev/null
    exit 1
fi
say "campe apres ${i}s"

# Marker: everything that follows in pont.log belongs to the experiment.
PONT_DEBUT=$(wc -l < /dev/shm/pont.log 2>/dev/null || echo 0)

# ── 3. The calls ──────────────────────────────────────────────────────
: > "$OUT/appels.txt"
for n in $(seq 1 "$CALLS"); do
    say "appel $n/$CALLS  -> $DEST"
    avant=$(wc -l < /dev/shm/pont.log 2>/dev/null || echo 0)
    vty "call 1 $DEST" > "$OUT/appel-$n-vty.txt" 2>&1
    sleep "$CALL_S"
    vty "call 1 hangup" >> "$OUT/appel-$n-vty.txt" 2>&1
    sleep 2

    # Verdict: in the slice of pont.log produced by this call, a
    # "DESARME (CHANNEL RELEASE" marks a TCH that lived; a "SANS ASSIGNMENT
    # COMPLETE" marks a mobile that did not switch. Cumulative counters are not
    # used, they would mix in the previous calls.
    tranche=$(tail -n +$((avant + 1)) /dev/shm/pont.log 2>/dev/null)
    if printf '%s' "$tranche" | grep -q "SANS ASSIGNMENT COMPLETE"; then
        verdict=ECHEC
    elif printf '%s' "$tranche" | grep -q "DESARME (CHANNEL RELEASE"; then
        verdict=OK
    else
        verdict=INDETERMINE
    fi
    printf 'appel %d : %s\n' "$n" "$verdict" | tee -a "$OUT/appels.txt"
    printf '%s' "$tranche" > "$OUT/appel-$n-pont.log"
    sleep "$GAP_S"
done

# ── 4. Collection ─────────────────────────────────────────────────────
say "collecte des journaux"
cp /dev/shm/pont.log            "$OUT/pont.log"            2>/dev/null
tail -n +$((PONT_DEBUT + 1)) /dev/shm/pont.log > "$OUT/pont-experience.log" 2>/dev/null
cp "$LOGDIR"/qemu.log           "$OUT/qemu.log"            2>/dev/null
cp "$LOGDIR"/qemu-tete.log      "$OUT/qemu-tete.log"       2>/dev/null
cp "$LOGDIR"/mobile.log         "$OUT/mobile.log"          2>/dev/null
journalctl -u osmo-bsc --since "-30min" --no-pager > "$OUT/osmo-bsc.log" 2>/dev/null
journalctl -u osmo-msc --since "-30min" --no-pager > "$OUT/osmo-msc.log" 2>/dev/null
for f in calypso_dcch_cfg calypso_kc_l1 calypso_tch_cfg; do
    [ -e "/dev/shm/$f" ] && od -A d -t x1 "/dev/shm/$f" > "$OUT/$f.hex" 2>/dev/null
done

# ── 5. Summary ────────────────────────────────────────────────────────
# The pont.py counters (STATS, DETAIL blocs) are real cumulative counts. The
# shunt ones are rate-limited SHUNT_LOG lines (e.g. `if (++n <= 3 || n % 2000 == 0)`):
# counting them would give a number that measures nothing. Only what is
# reliable goes here, and it is said.
{
    echo "=== experience $STAMP : $CALLS appels vers $DEST ==="
    cat "$OUT/appels.txt"
    echo
    echo "--- canaux dedies appris par le shunt (compteur reel, non limite) ---"
```

```
    grep -ac "DCCH #" "$OUT/qemu.log" 2>/dev/null
    grep -ah "DCCH #" "$OUT/qemu.log" 2>/dev/null | tail -5
    echo
    echo "--- pont : compteurs cumulatifs (FIABLES) ---"
    grep -a "STATS" "$OUT/pont.log" 2>/dev/null | tail -1
    grep -a "DETAIL blocs" "$OUT/pont.log" 2>/dev/null | tail -1
    grep -a "SOUS-VOIE ACTIVE" "$OUT/pont.log" 2>/dev/null | tail -1
    echo
    echo "--- reseau : echecs d'assignation ---"
    grep -ac "Assignment failed" "$OUT/osmo-bsc.log" 2>/dev/null
    echo
    echo "--- mobile : SAPI invalides ---"
    grep -ac "unsupported SAPI" "$OUT/mobile.log" 2>/dev/null
    echo
    echo "ATTENTION : les SHUNT_LOG du shunt sont limites en debit."
    echo "Ne pas utiliser 'grep -c' sur eux comme un compteur."
} > "$OUT/resume.txt" 2>&1

cat "$OUT/resume.txt"
say "tout est dans $OUT"
```

2.188 /opt/GSM/osmo-operator/scripts/call-diag.sh

7385 octets, 183 lignes → 183 lignes

```
#!/bin/bash
# call-diag.sh - call path diagnostic, MS → Asterisk echo test
# Usage: sudo docker exec -ti osmo-operator-1 bash /scripts/call-diag.sh
#
# Checks each link: MSC → MNCC → osmo-sip-connector → Asterisk

GREEN='\033[0;32m'; RED='\033[0;31m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; NC='\033[0m'; BOLD='\033[1m'

ok()   { echo -e "  ${GREEN}✓${NC} $*"; }
fail() { echo -e "  ${RED}✗${NC} $*"; }
warn() { echo -e "  ${YELLOW}⚠${NC} $*"; }

echo -e "${CYAN}${BOLD}═══ Diagnostic appel MS → Echo Test ═══${NC}"
echo ""

# ── 1. Core services ──────────────────────────────────────────
echo -e "${BOLD}[1/8] Services Osmocom${NC}"
for svc in osmo-msc osmo-bsc osmo-mgw osmo-stp osmo-hlr; do
    if systemctl is-active --quiet "$svc" 2>/dev/null; then
        ok "$svc actif"
    elif pgrep -f "$svc" >/dev/null 2>&1; then
        ok "$svc (process)"
    else
        fail "$svc INACTIF"
    fi
done
echo ""

# ── 2. MNCC socket ────────────────────────────────────────────
echo -e "${BOLD}[2/8] MNCC socket (/tmp/msc_mncc)${NC}"
if [ -S /tmp/msc_mncc ]; then
    ok "Socket MNCC existe"
    ls -la /tmp/msc_mncc | sed 's/^/       /'
else
    fail "Socket MNCC ABSENT - MSC ne cree pas le socket MNCC external"
    echo "       Verifier que osmo-msc.cfg contient : mncc external /tmp/msc_mncc"
fi
echo ""
```

```
# ── 3. osmo-sip-connector ──────────────────────────────────────────
echo -e "${BOLD}[3/8] osmo-sip-connector${NC}"
if pgrep -f "osmo-sip-connector" >/dev/null 2>&1; then
    ok "Processus actif"
    pgrep -fa "osmo-sip-connector" | head -1 | sed 's/^/       /'

    # Check that it listens on 5061
    if ss -tlnp 2>/dev/null | grep -q ":5061 "; then
        ok "Ecoute sur :5061 (cote SIP)"
    else
        warn "Pas en ecoute sur :5061"
    fi
else
    fail "osmo-sip-connector NON LANCE"
    echo ""
    echo "       C'est le pont MNCC ↔ SIP. Sans lui, aucun appel ne passe."
    echo "       Fix: systemctl start osmo-sip-connector"
    echo "       Ou:  osmo-sip-connector -c /etc/osmocom/osmo-sip-connector.cfg &"
fi
echo ""

# ── 4. Asterisk ────────────────────────────────────────────────────
echo -e "${BOLD}[4/8] Asterisk${NC}"
if pgrep -f "asterisk" >/dev/null 2>&1; then
    ok "Processus actif"

    if ss -ulnp 2>/dev/null | grep -q ":5060 "; then
        ok "Ecoute SIP sur :5060/udp"
    else
        fail "Pas en ecoute sur :5060/udp"
    fi

    # Check the PJSIP endpoints
    echo "       Endpoints PJSIP:"
    asterisk -rx "pjsip show endpoints" 2>/dev/null | grep -E "Endpoint:|Contact:" | head -10 | sed 's/^/       /'
     ↪  || warn "Impossible de lister les endpoints"

    # Check that the dialplan has 600
    echo ""
    echo "       Dialplan 600:"
    asterisk -rx "dialplan show 600@gsm_in" 2>/dev/null | head -10 | sed 's/^/       /' || warn "Extension 600
     ↪  introuvable dans gsm_in"
else
    fail "Asterisk NON LANCE"
    echo "       Fix: asterisk -cvvv &"
fi
echo ""

# ── 5. osmo-sip-connector config ──────────────────────────────────
echo -e "${BOLD}[5/8] Config osmo-sip-connector${NC}"
SIP_CFG="/etc/osmocom/osmo-sip-connector.cfg"
if [ -f "$SIP_CFG" ]; then
    echo "       Contenu:"
    cat "$SIP_CFG" | sed 's/^/       /'

    remote_line=$(grep "remote" "$SIP_CFG" | head -1)
    echo ""
    if echo "$remote_line" | grep -q "5060"; then
        ok "Remote SIP pointe vers :5060"
    else
        warn "Remote SIP: $remote_line"
    fi
else
    fail "Config osmo-sip-connector introuvable"
fi
```

```
echo ""

# ── 6. MGW ──────────────────────────────────────────────────────────
echo -e "${BOLD}[6/8] OsmoMGW (RTP)${NC}"
if bash -c "echo >/dev/tcp/127.0.0.1/4243" 2>/dev/null; then
    ok "VTY 4243 accessible"
    echo "       Endpoints:"
    (echo "show mgcp"; sleep 1) | telnet 127.0.0.1 4243 2>/dev/null \
        | grep -vE "^(Trying|Connected|Escape|Welcome|OsmoMGW)" \
        | head -15 | sed 's/^/       /' || true
else
    fail "VTY MGW 4243 inaccessible"
fi
echo ""

# ── 7. BTS-TRX ──────────────────────────────────────────────────────
echo -e "${BOLD}[7/8] osmo-bts-trx${NC}"
if pgrep -f "osmo-bts-trx" >/dev/null 2>&1; then
    ok "Processus actif"
else
    if systemctl is-active --quiet osmo-bts-trx 2>/dev/null; then
        ok "Service actif"
    else
        fail "osmo-bts-trx INACTIF"
        echo "       Sans BTS, pas de canal radio → pas d'appel"
        echo "       Fix: systemctl start osmo-bts-trx"
    fi
fi
echo ""

# ── 8. Mobile registration ──────────────────────────────────────────
echo -e "${BOLD}[8/8] Enregistrement mobile (MSC VTY)${NC}"
MSC_OUT=$( (echo "enable"; echo "show subscriber all"; sleep 1; echo "exit") \
    | telnet 127.0.0.1 4254 2>/dev/null \
    | grep -vE "^(Trying|Connected|Escape|Welcome|OsmoMSC)" || true)

if echo "$MSC_OUT" | grep -qiE "IMSI|subscriber"; then
    ok "Abonnes enregistres sur le MSC:"
    echo "$MSC_OUT" | grep -iE "IMSI|MSISDN|LAC" | head -10 | sed 's/^/       /'
else
    warn "Aucun abonne enregistre sur le MSC"
    echo "       Les MS ne sont pas encore attaches au reseau"
    echo "       Verifier : dans le VTY mobile → show ms"
    echo "       L'etat doit etre 'NORMAL service'"
fi
echo ""

# ── Summary ─────────────────────────────────────────────────────────
echo -e "${BOLD}═══ Chaine d'appel echo test ═══${NC}"
echo ""
echo "  mobile (call 1 600)"
echo "    ↓ TCH via fake_trx"
echo "  osmo-bts-trx → osmo-bsc → osmo-msc"
echo "    ↓ MNCC socket /tmp/msc_mncc"

if [ -S /tmp/msc_mncc ]; then
    echo -e "    ${GREEN}✓${NC} MNCC socket"
else
    echo -e "    ${RED}✗ MNCC socket MANQUANT${NC}"
fi

echo "  osmo-sip-connector (5061 → 5060)"
if pgrep -f "osmo-sip-connector" >/dev/null 2>&1; then
    echo -e "    ${GREEN}✓${NC} sip-connector"
else
    echo -e "    ${RED}✗ sip-connector MANQUANT ← PROBLEME PROBABLE${NC}"
```

```
fi

echo "  Asterisk (gsm_in context, exten 600 → Echo())"
if pgrep -f "asterisk" >/dev/null 2>&1; then
    echo -e "    ${GREEN}✓${NC} Asterisk"
else
    echo -e "    ${RED}✗ Asterisk MANQUANT${NC}"
fi

echo "    ↓ RTP via OsmoMGW"
echo "  osmo-bts-trx → fake_trx → trxcon → mobile"
echo "    ↓ l1phy decode TCH-FR"
echo "  ALSA → PulseAudio → HP"
echo ""
```

2.189 /opt/GSM/osmo-operator/scripts/calypso-mode.sh

7220 octets, 148 lignes → 148 lignes

```
#!/bin/bash
# ============================================================================
# scripts/calypso-mode.sh - who owns the Calypso (QEMU + TI ROM): voice or data.
# ============================================================================
#
# The Calypso layer 1 (osmocon, /tmp/osmocom_l2) accepts only one layer 2
# client. Two clients steal it from each other: each resets it (RESET FULL),
# the mobile loses the network (« No service ») and the modem does not attach.
#
#   voix   the osmocom-bb `mobile` (MS#1, VTY 4247): calls, SMS, USSD.
#   data   the osmocom-bb GPRS `modem` (IMSI of the mobile, 001010001000001):
#          GPRS attach, PDP context, tun_modem in the msgprs netns. The mobile
#          is stopped.
#
# Switching data -> voice restarts the mobile exactly as c54x_exe/run.sh had
# started it (command line, directory and log read from /proc before stopping
# it). Called by tools/osmo-phonesim-banc.py: ATD*99 from the phone -> data;
# call or SMS -> voice (a class B 2G: no data during voice). An incoming call
# cannot arrive in data mode: the mobile is not there to be paged.
#
# Usage: calypso-mode.sh data|voix|etat [--wait]
#   --wait   data: wait for the tun_modem address (PDP active, 240 s max)
#            voix: wait for the mobile in normal service (120 s max)
# State: /run/osmo-banc/calypso-mode (voix|data), mobile command line:
#        /run/osmo-banc/calypso-mobile.{cmd,cwd,log}
# ----------------------------------------------------------------------------
set -uo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
ETC="${OSMOCOM_CFG:-/etc/osmocom}"
SOCK="${CALYPSO_L2_SOCK:-/tmp/osmocom_l2}"
NETNS="${EGPRS_NETNS:-msgprs}"
ETAT=/run/osmo-banc
MODE_F="$ETAT/calypso-mode"
U_MODEM=osmo-modem-gprs
MOB_VTY="${OSMO_MOB_VTY_PORT:-4247}"
MODEM_BIN="$(command -v modem 2>/dev/null || echo /opt/GSM/osmocom-bb/src/host/layer23/src/modem/modem)"
mkdir -p "$ETAT"

log() { printf '[calypso-mode] %s\n' "$*"; }
mode_courant() { cat "$MODE_F" 2>/dev/null || echo voix; }

# Does the config file of a layer 2 client (mobile or modem) point at SOCK?
# Without « layer2-socket », osmocom-bb uses /tmp/osmocom_l2.
vise_sock() {
    local cfg="$1" s
```

```
    s="$(awk '$1=="layer2-socket"{print $2; exit}' "$cfg" 2>/dev/null)"
    [ "${s:-/tmp/osmocom_l2}" = "$SOCK" ]
}
# The PIDs of the `mobile` processes connected to SOCK
mobiles_sur_sock() {
    local p c
    for p in $(pgrep -x mobile); do
        c="$(tr '\0' '\n' < "/proc/$p/cmdline" 2>/dev/null | awk 'p{print; exit} $0=="-c"{p=1}')"
        [ -n "$c" ] || continue
        case "$c" in /*) ;; *) c="$(readlink "/proc/$p/cwd")/$c" ;; esac
        vise_sock "$c" && echo "$p"
    done
}
port_open() { timeout 1 bash -c "exec 3<>/dev/tcp/127.0.0.1/$1" 2>/dev/null; }
vty() {
    { sleep 0.3; printf 'enable\n'; local c; for c in "$@"; do sleep 0.3; printf '%s\n' "$c"; done; sleep 0.6; } \
        | timeout 6 nc -q1 127.0.0.1 "$MOB_VTY" 2>/dev/null | tr -d '\r'
}

vers_data() {
    local p n=0
    for p in $(mobiles_sur_sock); do
        # The exact mobile command line, to restart it identically
        tr '\0' '\n' < "/proc/$p/cmdline" > "$ETAT/calypso-mobile.cmd"
        readlink "/proc/$p/cwd" > "$ETAT/calypso-mobile.cwd"
        readlink "/proc/$p/fd/1" > "$ETAT/calypso-mobile.log" 2>/dev/null || true
        # SIGTERM is not enough: the mobile attempts a clean IMSI detach, which
        # it does not complete on a contended layer 1. 3 s, then SIGKILL.
        kill "$p" 2>/dev/null
        local w; for w in 1 2 3 4 5 6; do kill -0 "$p" 2>/dev/null || break; sleep 0.5; done
        kill -0 "$p" 2>/dev/null && kill -9 "$p" 2>/dev/null && log "mobile $p : SIGTERM ignore, SIGKILL"
        n=$((n+1)); log "mobile $p arrete (ligne gardee dans $ETAT/calypso-mobile.cmd)"
    done
    [ "$n" -gt 0 ] && sleep 1
    ip netns list 2>/dev/null | grep -qw "$NETNS" || ip netns add "$NETNS"
    local cfg="$ETC/modem-gprs-dsp.cfg"
    sed "s|layer2-socket .*|layer2-socket $SOCK|" "$HERE/configs/gprs/modem-gprs.cfg" > "$cfg"
    if ! systemctl is-active --quiet "$U_MODEM"; then
        systemctl reset-failed "$U_MODEM" >/dev/null 2>&1 || true
        systemd-run --quiet --unit="$U_MODEM" --collect -p KillMode=control-group -p
         ↪  CPUAffinity="${RT_CPUS:-0-15}" -p Nice=-10 \
            -p SyslogIdentifier="$U_MODEM" "$MODEM_BIN" -c "$cfg" \
            || { log "systemd-run $U_MODEM a echoue"; return 1; }
        log "modem GPRS lance sur $SOCK ($cfg)"
    fi
    echo data > "$MODE_F"
    if [ "$ATTENDRE" = 1 ]; then
        local t=0
        until ip netns exec "$NETNS" ip -4 -o addr show dev tun_modem 2>/dev/null | grep -q 'inet '; do
            [ $t -ge 240 ] && { log "pas de PDP apres 240 s (journalctl -u $U_MODEM)"; return 1; }
            sleep 2; t=$((t+2))
        done
        log "PDP actif en ${t}s : $(ip netns exec "$NETNS" ip -4 -o addr show dev tun_modem | awk '{print $4}')"
    fi
}

vers_voix() {
    if systemctl is-active --quiet "$U_MODEM"; then
        systemctl stop "$U_MODEM"; log "modem GPRS arrete"
    fi
    systemctl reset-failed "$U_MODEM" >/dev/null 2>&1 || true
    if [ -z "$(mobiles_sur_sock)" ]; then
        [ -s "$ETAT/calypso-mobile.cmd" ] || { log "aucune ligne de mobile gardee : relancer le banc (systemctl
         ↪  restart osmo-banc)"; echo voix > "$MODE_F"; return 1; }
        local -a cmd; mapfile -t cmd < "$ETAT/calypso-mobile.cmd"
        local cwd jrn
```

```
        cwd="$(cat "$ETAT/calypso-mobile.cwd" 2>/dev/null || echo /)"
        jrn="$(cat "$ETAT/calypso-mobile.log" 2>/dev/null)"
        case "$jrn" in /*) ;; *) jrn=/tmp/c54x-pont/mobile.log ;; esac
        # As c54x_exe/run.sh (step 4): chain priority, line-buffered output
        ( cd "$cwd" && exec setsid taskset -c "${RT_CPUS:-0-15}" nice -n -10 stdbuf -oL "${cmd[@]}" >> "$jrn" 2>&1
          ↪  < /dev/null ) &
        log "mobile relance : ${cmd[*]} (journal $jrn)"
    fi
    echo voix > "$MODE_F"
    if [ "$ATTENDRE" = 1 ]; then
        local t=0
        until port_open "$MOB_VTY" && vty "show ms" | grep -q 'service is normal'; do
            [ $t -ge 120 ] && { log "mobile pas en service normal apres 120 s"; return 1; }
            sleep 2; t=$((t+2))
        done
        log "mobile en service normal en ${t}s (VTY $MOB_VTY)"
    fi
}

ACTION="${1:-etat}"; ATTENDRE=0
[ "${2:-}" = "--wait" ] && ATTENDRE=1
# No layer 1, no switch: with the bench stopped (or not in DSP mode), a
# restarted mobile or modem would die at once (« Failed to create unix domain
# socket /tmp/osmocom_l2 »). Say so instead of starting for nothing.
if [ "$ACTION" = data ] || [ "$ACTION" = voix ]; then
    if [ ! -S "$SOCK" ]; then
        log "pas de couche 1 ($SOCK absent) : le banc DSP ne tourne pas - systemctl start osmo-banc, puis relancer"
        exit 3
    fi
fi
case "$ACTION" in
    data) vers_data ;;
    voix) vers_voix ;;
    etat)
        echo "mode    : $(mode_courant)"
        echo "mobile  : $(mobiles_sur_sock | tr '\n' ' ')"
        echo "modem   : $(systemctl is-active "$U_MODEM" 2>/dev/null)"
        echo "PDP     : $(ip netns exec "$NETNS" ip -4 -o addr show dev tun_modem 2>/dev/null | awk '{print $4}')"
          ↪  ;;
    *) echo "usage : $0 data|voix|etat [--wait]" >&2; exit 2 ;;
esac
```

2.190 /opt/GSM/osmo-operator/scripts/check.sh

2875 octets, 53 lignes → 52 lignes (1 groupes compactés)

```
# ── INTER-STP ──────────────────────────────────────────────────────
sudo docker exec -it osmo-inter-stp telnet 127.0.0.1 4239
# show cs7 instance 0 asp
# show cs7 instance 0 as
# show cs7 instance 0 route

# ── OPERATOR 1 - intra STP ─────────────────────────────────────────
sudo docker exec -it osmo-operator-1 telnet 127.0.0.1 4239
# show cs7 instance 0 asp
# show cs7 instance 0 as
# show cs7 instance 0 sccp users
# show cs7 instance 0 sccp connections

# ── OPERATOR 1 - MSC ───────────────────────────────────────────────
sudo docker exec -it osmo-operator-1 telnet 127.0.0.1 4254
# en
# show cs7 instance 0 asp
# show cs7 instance 0 sccp users
```

```
# show cs7 instance 0 sccp connections
# show subscriber all
# show connection all

# ── OPERATOR 1 - BSC ──────────────────────────────────────────────────
sudo docker exec -it osmo-operator-1 telnet 127.0.0.1 4242
# en
# show cs7 instance 0 asp
# show cs7 instance 0 sccp users
# show cs7 instance 0 sccp connections
# show bts 0
# show trx 0 0

# ── OPERATOR 2 - same thing ───────────────────────────────────────────
sudo docker exec -it osmo-operator-2 telnet 127.0.0.1 ⟨1⟩  ×3
    ⟨⟩ = (4239) (4254) (4242)

# ── ONE-LINER: dump everything without interaction────────────────────
for container in osmo-operator-1 osmo-operator-2; do
    echo "=== $container STP ==="
    sudo docker exec $container sh -c \
        'echo -e "show cs7 instance 0 asp\nshow cs7 instance 0 sccp users\nshow cs7 instance 0 sccp connections\n"
        ↪  | telnet 127.0.0.1 4239 2>/dev/null' \
        | grep -v "^Trying\|^Connected\|^Escape"

    echo "=== $container MSC ==="
    sudo docker exec $container sh -c \
        'echo -e "enable\nshow cs7 instance 0 sccp users\nshow cs7 instance 0 sccp connections\nshow subscriber
        ↪  all\n" | telnet 127.0.0.1 4254 2>/dev/null' \
        | grep -v "^Trying\|^Connected\|^Escape"
done

echo "=== inter-STP ASP ==="
sudo docker exec osmo-inter-stp sh -c \
    'echo -e "show cs7 instance 0 asp\nshow cs7 instance 0 as\n" | telnet 127.0.0.1 4239 2>/dev/null' \
    | grep -v "^Trying\|^Connected\|^Escape"
```

2.191 /opt/GSM/osmo-operator/scripts/dsp-rom-max.sh

14085 octets, 226 lignes → 226 lignes

```
#!/bin/bash
# ============================================================================
# scripts/dsp-rom-max.sh - workflow « the DSP does as much as possible in ROM ».
# ============================================================================
#
# The DSP bench (c54x_exe, TI mask-ROM) can replace some ROM processing with
# host reconstructions (pont.py, montant.c, qosmo): canned SB/TOA, RACH coded
# by the bridge, uplink xCCH coding by the bridge, A5 deciphering by the
# bridge... Each deviation is an environment switch (hacks_actifs() in
# c54x_exe/src/pont.c prints them at startup). This workflow:
#
#   1. profil     writes the « ROM max » profile to /etc/default/osmo-banc:
#                 every switch that has a ROM path is on ROM, no canned SB/TOA.
#                 Persistent: every boot of the bench reads it.
#   2. banc       restarts osmo-banc (--dsp) with this profile and waits for
#                 camping (mobile VTY 4247).
#   3. echelle    runs the ladder of /opt/GSM/tests/banc-max.sh --dsp on the
#                 running stack: LU, SMS MO/MT, voice call, then the
#                 « couverture » rung (tests/modules/99-couverture.sh), which
#                 decides, processing by processing, ROM or HOST from the logs.
#   4. bilan      reads couverture.txt back: what the ROM did, what stayed on
#                 the host despite the profile (to fix), and what has no ROM
#                 path wired (remaining work: uplink FACCH/SACCH-TCH/speech,
```

```
#                uplink A5 at the air fn, vocoding).
#
# Usage: dsp-rom-max.sh [run|profile|report|status] [--no-restart] [--only=camp,attache]
#   run (default)  1 -> 4            profile  write the profile only (1)
#   report         read the latest /root/banc-max-*/couverture.txt (4)
#   --no-restart   do not restart osmo-banc (the running stack already has the profile?)
#   --only=...     passed to banc-max.sh (e.g. --only=camp,attache,couverture)
#   --parole-rom   uplink TCH (speech, FACCH, SACCH/T) sent by the ROM:
#                  CALYPSO_NORM_TC=0 + CALYPSO_C54X_BCTC_NORM=1 (emulator: speech frame
#                  marker of PROM0 0x90a0) + PONT_UL_RAW_TCH=1.
#                  Default; --parole-hote reverts (and turns off --vocodeur-rom).
#   --vocodeur-rom the ROM FR codec does the vocoding (modelled SP0 voice port,
#                  calypso_vsp.c): BANC_VOCODEUR=rom (mobile in l1phy, without GAPK) +
#                  CALYPSO_VSP=1 (pont/vsp_audio.py <-> PulseAudio). Implies --parole-rom.
#                  + emulator: CALYPSO_C54X_BCD_CARRY=1 (BCD on C/NC, FR unpacking PDROM 0xe458)
#                  and CALYPSO_C54X_OVM_LOGIQUE=1 (SFTL/AND/OR/XOR/ROL do not saturate, PDROM
#                  header 0xe4c4) and CALYPSO_MAC=1 (MACR/MASR/MACA/MASA/MPYA per SPRU172C):
#                  without it the ROM FR decoder outputs a mirrored spectrum (1 kHz rendered
#                  at ~3 kHz); with it, its replay output has the libgsm spectrum.
#                  + CALYPSO_C54X_BCD_TC=1 (true TC on the 12 BCD tc/ntc: sign of the coder
#                  reflection coefficients, Schur PDROM 0xe763/0xedc5) and
#                  CALYPSO_C54X_BCTC_COMBO=1 (BC tc,nc,bio: ANDed conditions, isa 28). With
#                  these flags and the ASM shift of ADD/SUB src,ASM, the ROM FR coder is
#                  bit-exact with libgsm on ~99 % of the frames at replay.
#                  Default; --vocodeur-hote goes back to GAPK.
#   --fw-gprs      layer1 firmware with PDCH (layer1.highram.gprs.elf): the default;
#   --fw-stock     the stock firmware (without GPRS)
#
# Unit: services/osmo-dsp-rom-max.service (oneshot, started on demand: a test
# campaign has no business at boot; the profile, however, persists).
# Journal: journalctl -u osmo-dsp-rom-max; reports: /root/banc-max-<date>/
# ------------------------------------------------------------------------------
set -uo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
DEFAULTS=/etc/default/osmo-banc
TESTS=/opt/GSM/tests
FW_GPRS=/opt/GSM/firmware/board/compal_e88/layer1.highram.gprs.elf
# --vocodeur-rom (hence --parole-rom) by default: a machine without
# /etc/default/osmo-banc would otherwise run the ROM FR codec without its
# emulator fixes (uplink silence frames, silent echo test).
RESTART=1; ONLY=""; FWGPRS=1; PAROLE_ROM=1; VOCODEUR_ROM=1; ACTION=run
MOB_VTY=4247

if [ -t 1 ] && [ -z "${NO_COLOR:-}" ]; then C_OK=$'\033[32m'; C_KO=$'\033[31m'; C_WA=$'\033[33m';
 ↪  C_DIM=$'\033[2m'; C_B=$'\033[1m'; C_Z=$'\033[0m'
else C_OK=; C_KO=; C_WA=; C_DIM=; C_B=; C_Z=; fi
say()  { printf '  %s\n' "$*"; }
ok()   { printf '  %s[ OK ]%s %s\n' "$C_OK" "$C_Z" "$*"; }
ko()   { printf '  %s[ KO ]%s %s\n' "$C_KO" "$C_Z" "$*"; }
warn() { printf '  %s[ !! ]%s %s\n' "$C_WA" "$C_Z" "$*"; }
titre(){ printf '\n%s== %s ==%s\n' "$C_B" "$*" "$C_Z"; }
port_open() { timeout 1 bash -c "exec 3<>/dev/tcp/127.0.0.1/$1" 2>/dev/null; }

# ── 1. the ROM max profile ──────────────────────────────────────────────
# Each line: VARIABLE=value   # what it puts in the ROM (and the other value)
profil_lignes() { cat <<'P'
MONTANT_EFFACE_TASK_RA=0
MONTANT_ROM_UL=1
MONTANT_ROM_TX=1
UL_RAW=1
PONT_DSP_DECHIFFRE=0
CALYPSO_A5=1
MONTANT_REQREF=0
CALYPSO_C54X_BCTC_ETENDU=1
PONT_CAN_TOA=
PONT_CAN_SB_TOA=
```

```
PONT_CAN_SB=
P
}
profil_doc() { cat <<'P'
# ── profile « the ROM does the maximum » (scripts/dsp-rom-max.sh) ──────────
# MONTANT_EFFACE_TASK_RA=0  RACH: the ROM keeps the task and codes the access burst (1: the bridge codes it)
# MONTANT_ROM_UL=1          uplink SDCCH/SACCH: bursts coded by the ROM (0: host coding)
# MONTANT_ROM_TX=1          final ROM bursts published to the bridge (calypso_tx_rom)
# UL_RAW=1                  the bridge sends those bursts as is (0: recoding + A5 on the host)
# PONT_DSP_DECHIFFRE=0      downlink A5: the ROM deciphers, not the bridge
# CALYPSO_A5=1              A5 coprocessor of the silicon, modelled in c54x_exe (0: off)
# MONTANT_REQREF=0          IMMEDIATE ASSIGNMENT: BTS reference left as is (1: qosmo realigns it on last_rach)
# CALYPSO_C54X_BCTC_ETENDU=1  emulator: BC tc/ntc on the true TC after cmpr/bit/bitt and after cmpm/bitf + ST
 ↪  (0xb2c7, double I/Q buffer) (0: ACC heuristic)
# PONT_CAN_TOA= / PONT_CAN_SB_TOA= / PONT_CAN_SB=   nothing canned: FB/SB/TOA by the ROM
P
}
etape_profil() {
    titre "1. profil ROM max dans $DEFAULTS"
    # Without layer1.highram.gprs.elf the profile must not require it, otherwise
    # the bench refuses to start: fall back to the stock firmware.
    if [ "$FWGPRS" = 1 ] && { [ ! -r "$FW_GPRS" ] || [ ! -r "${FW_GPRS%.elf}.bin" ]; }; then
        warn "$FW_GPRS absent : firmware d origine (compiler le layer1 GPRS pour --fw-gprs)"
        FWGPRS=0
    fi
    touch "$DEFAULTS"
    # remove a previous version of the block, then write it again
    sed -i '/profil « la ROM fait le maximum »/,/^# ── fin profil ROM max/d' "$DEFAULTS"
    while IFS= read -r l; do [ -n "$l" ] && sed -i "/^${l%%=*}=/d" "$DEFAULTS"; done < <(profil_lignes)
    sed -i '/^FIRM-
     ↪  WARE_ELF=/d;/^FIRMWARE_BIN=/d;/^CALYPSO_NORM_TC=/d;/^CALYPSO_C54X_BCTC_NORM=/d;/^PONT_UL_RAW_TCH=/d;/^#
     ↪  parole montante
     ↪  ROM/d;/^BANC_VOCODEUR=/d;/^CALYPSO_VSP=/d;/^CALYPSO_C54X_BCD_CARRY=/d;/^CALYPSO_C54X_OVM_LOGIQUE=/d;/^CALYPSO_MAC=/d;/^
     ↪  vocodeur ROM/d' "$DEFAULTS"
    { profil_doc; profil_lignes
      if [ "$PAROLE_ROM" = 1 ]; then printf '# parole montante ROM (--parole-rom,
       ↪  defaut)\nCALYPSO_NORM_TC=0\nCALYPSO_C54X_BCTC_NORM=1\nPONT_UL_RAW_TCH=1\n'
      else printf '# parole montante hote (--parole-hote)\n'; fi
      if [ "$VOCODEUR_ROM" = 1 ]; then printf '# vocodeur ROM (--vocodeur-rom, de-
       ↪  faut)\nBANC_VOCODEUR=rom\nCALYPSO_VSP=1\nCALYPSO_C54X_BCD_CARRY=1\nCALYPSO_C54X_OVM_LOGIQUE=1\nCALYPSO_MAC=1\nCALYPSO
      else printf '# vocodeur hote, GAPK (--vocodeur-hote)\n'; fi
      if [ "$FWGPRS" = 1 ]; then printf 'FIRMWARE_ELF=%s\nFIRMWARE_BIN=%s\n' "$FW_GPRS" "${FW_GPRS%.elf}.bin"
      else printf '# FIRMWARE_ELF=%s   # --fw-gprs : layer1 avec PDCH (L1CTL_GPRS_*)\n' "$FW_GPRS"; fi
      printf '# ── fin profil ROM max ──\n'; } >> "$DEFAULTS"
    # No OSMO_BANC_ARGS="--dsp" here: in the unit, EnvironmentFile= takes
    # precedence over `systemctl set-environment` (launch.sh --grgsm) and the
    # bench would start in DSP whatever is asked. DSP is the start-native.sh
    # default.
    sed -i '/^OSMO_BANC_ARGS=/d' "$DEFAULTS"
    ok "profil pose ($(profil_lignes | grep -c .) commutateurs)"; grep -E '^[A-Z_]+=' "$DEFAULTS" | sed 's/^/
     ↪  /'
}

# ── 2. the bench ──────────────────────────────────────────────────────────
etape_banc() {
    titre "2. banc DSP (osmo-banc.service)"
    if [ "$RESTART" = 1 ]; then
        say "systemctl restart osmo-banc (plusieurs minutes : QEMU, C54x, camp)"
        systemctl reset-failed osmo-banc 2>/dev/null; systemctl restart osmo-banc || { ko "osmo-banc : journalctl
         ↪  -u osmo-banc -n 50"; return 1; }
    else say "--no-restart : pile en cours"; fi
    local t=0; while ! port_open $MOB_VTY; do [ $t -ge 180 ] && { ko "VTY du mobile DSP :$MOB_VTY absente apres
     ↪  180s"; return 1; }; sleep 3; t=$((t+3)); done
    ok "mobile DSP (VTY :$MOB_VTY) en ${t}s"
    local dsplog=/tmp/c54x-pont/dsp.log
    if [ -r "$dsplog" ]; then
```

```
        local h; h="$(grep -ao 'hacks[^)]*' "$dsplog" | head -1)"
        grep -aq 'laisse a la ROM : elle code l.access-burst' "$dsplog" && ok "RACH : tache laissee a la ROM" ||
         ↪  warn "RACH : pas de trace « laisse a la ROM » dans $dsplog"
        grep -aq 'rom-ul.*actif' "$dsplog" && ok "xCCH montant : bursts de la ROM publies" || warn "rom-ul : non
         ↪  actif dans $dsplog"
        grep -aq 'tx-rom.*actif\|calypso_tx_rom.*actif' "$dsplog" && ok "bursts finaux de la ROM : publies" || say
         ↪  "${C_DIM}(pas de ligne tx-rom actif lue)${C_Z}"
        [ -n "$h" ] && say "${C_DIM}$h${C_Z}"
    fi
    return 0
}

# ── 3. the ladder ──────────────────────────────────────────────────────
etape_echelle() {
    titre "3. echelle banc-max.sh --dsp --continue${ONLY:+ --only=$ONLY}"
    [ -x "$TESTS/banc-max.sh" ] || { ko "$TESTS/banc-max.sh absent"; return 1; }
    local log=/var/log/osmocom/dsp-rom-max-echelle.log
    mkdir -p /var/log/osmocom
    ( cd "$TESTS" && CALYPSO_NO_ATTACH=1 OSMO_NO_ATTACH=1 bash ./banc-max.sh --dsp --continue
     ↪  ${ONLY:+--only=$ONLY} ) > "$log" 2>&1
    local rc=$?
    grep -a 'ELEMENT MAX' "$log" | tail -1 | sed 's/^/  /'
    grep -aE '^\s+[a-z0-9-]+\s+(OK|ECHEC|SAUTE)' "$log" | sed 's/^/  /' | head -20
    [ "$rc" -eq 0 ] && ok "echelle : aucun barreau en echec" || warn "echelle : $rc barreau(x) en echec - $log"
    return 0
}

# ── 4. the ROM / host report ───────────────────────────────────────────
etape_bilan() {
    titre "4. bilan ROM / hote"
    local rap; rap="$(ls -t /root/banc-max-*/couverture.txt 2>/dev/null | head -1)"
    [ -n "$rap" ] || { ko "aucun /root/banc-max-*/couverture.txt (le barreau couverture n a pas joue)"; return 1; }
    say "rapport : $rap"
    python3 -I - "$rap" <<'PY'
import re, sys
txt = open(sys.argv[1], errors='replace').read()
m = re.search(r'chemin ROM / hote du traitement du signal.*?\n(.*?)\nchemin : (\d+) ROM, (\d+) hote, (\d+) mixte,
 ↪  (\d+) non observe', txt, re.S)
if not m:
    print("  [KO] section « chemin ROM / hote » absente du rapport"); sys.exit(1)
rom, hote, mixte, nobs = map(int, m.group(2, 3, 4, 5))
rows = []
for l in m.group(1).splitlines():
    r = re.match(r'^  (.{44}) (ROM|HOTE|MIXTE|NON OBSERVE)\s+(\d+)\s+(\d+)\s+(.*)$', l)
    if r: rows.append((r.group(1).strip(), r.group(2), int(r.group(3)), int(r.group(4)), r.group(5)))
attendu_rom = [r for r in rows if r[4].startswith('ROM')]
hote_constr = [r for r in rows if r[4].startswith('hote')]
print("  %-46s %-12s %s" % ("TRAITEMENT", "VERDICT", "rom/hote"))
for t, v, nr, nh, ch in rows:
    tag = {"ROM": "[ OK ]", "HOTE": "[ KO ]", "MIXTE": "[ !! ]", "NON OBSERVE": "[ -- ]"}[v]
    if v == "HOTE" and ch.startswith('hote'): tag = "[hote]"
    print("  %s %-39s %-12s %d/%d" % (tag, t[:39], v, nr, nh))
print()
print("  ROM %d, hote %d, mixte %d, non observe %d" % (rom, hote, mixte, nobs))
bad = [r for r in attendu_rom if r[1] in ("HOTE", "MIXTE")]
if bad:
    print("  A CORRIGER (chemin ROM attendu, l hote a pris la main) :")
    for t, v, nr, nh, ch in bad: print("    - %s : %s (rom=%d hote=%d) — %s" % (t, v, nr, nh, ch))
else:
    print("  [ OK ] tout ce qui a un chemin ROM est passe par la ROM")
print("  RESTE A BRANCHER SUR LA ROM (hote par construction) :")
for t, v, nr, nh, ch in hote_constr: print("    - %s — %s" % (t, ch))
PY
}

while [ $# -gt 0 ]; do
```

```
    case "$1" in
        run|profile|report|status) ACTION="$1" ;;
        --no-restart) RESTART=0 ;;
        --only=*) ONLY="${1#*=}" ;;
        --fw-gprs) FWGPRS=1 ;;
        --fw-stock) FWGPRS=0 ;;
        --parole-rom) PAROLE_ROM=1 ;;
        --parole-hote) PAROLE_ROM=0; VOCODEUR_ROM=0 ;;
        --vocodeur-rom) VOCODEUR_ROM=1; PAROLE_ROM=1 ;;
        --vocodeur-hote) VOCODEUR_ROM=0 ;;
        -h|--help) sed -n '2,36p' "$0"; exit 0 ;;
        *) echo "option inconnue : $1" >&2; exit 2 ;;
    esac; shift
done
[ "$(id -u)" -eq 0 ] || { echo "a lancer en root" >&2; exit 1; }
case "$ACTION" in
    profile) etape_profil ;;
    report)  etape_bilan ;;
    status)  systemctl is-active osmo-banc; grep -E '^[A-Z_]+=' "$DEFAULTS" 2>/dev/null | sed 's/^/  /'; port_open
     ↪  $MOB_VTY && echo "  mobile DSP : VTY :$MOB_VTY ouverte" ;;
    run)     etape_profil; etape_banc || exit 1; etape_echelle; etape_bilan ;;
esac
```

2.192 /opt/GSM/osmo-operator/scripts/egprs-dsp.sh

22850 octets, 378 lignes → 378 lignes

```
#!/bin/bash
# ============================================================================
# scripts/egprs-dsp.sh - EGPRS workflow of the bench, on the DSP chain (TI ROM).
# ============================================================================
#
# What this workflow brings up, in order, checking each step:
#
#   1. preflight    binaries, DSP ROM (7 files), c54x_exe, osmocom-bb modem;
#                   and the DSP gate: does the Calypso layer1 firmware handle
#                   PDCH (L1CTL_GPRS_*)? If not, the GPRS modem runs on a
#                   virtual layer 1 (virtphy), in the same PLMN/LAC/RA as the
#                   DSP mobile, which camps on BTS 0.
#   2. configs      /etc/osmocom: the virtual EGPRS BTS 2 in osmo-bsc.cfg
#                   (fragment configs/gprs/osmo-bsc-bts2-egprs.frag, reapplied
#                   after each --regen), osmo-bts-virtual-gprs.cfg,
#                   osmo-pcu-gprs.cfg, modem-gprs.cfg.
#   3. banc         osmo-banc.service (start-native.sh --dsp: core, BTS 0 on
#                   the C54x + TI ROM, SGSN/GGSN/PCU of BTS 0).
#   4. abonne       the modem IMSI in the HLR (comp128v1), idempotent.
#   5. cellule      osmo-bts-virtual (BTS 2, Um multicast on lo) + osmo-pcu
#                   (socket /tmp/pcu_bts3) as transient systemd units; waits
#                   for the OML of BTS 2 at the BSC and the NS-VC of the PCU.
#   6. modem        virtphy (-> /tmp/modem_l2) then osmocom-bb `modem`: GPRS
#                   attach (GMM) + PDP context (SM), tun in the msgprs netns;
#                   checked at the SGSN (mm-context, pdp-context) and with a
#                   ping to apn0 from the netns.
#
#   EGPRS_L1=dsp: no virtphy; the modem is started by c54x_exe/run.sh instead
#   of `mobile` (MOBILE= / MOBILE_CFG= in /etc/default/osmo-banc) on
#   /tmp/osmocom_l2. The script sets these two variables and says so; it does
#   not force a path the firmware does not have.
#
# Usage: egprs-dsp.sh [start|stop|status|check] [--l1 auto|virtphy|dsp]
#                     [--no-banc] [--regen-gprs] [--all]
#   start        bring everything up (default)  stop   tear down cell + modem
#   status       state + SGSN                   check  preflight only, starts nothing
#   --no-banc    leave osmo-banc alone          --all  (stop) also stop osmo-banc
```

```
#   --regen-gprs rewrite the 4 GPRS configs from configs/gprs even if present
#
# Persistent: services/osmo-egprs-dsp.service (Requires=osmo-banc) triggers this
# script at boot; OSMO_BANC_ARGS="--dsp" in /etc/default/osmo-banc.
# Journal: journalctl -u osmo-egprs-dsp, -u osmo-modem-gprs, -u osmo-pcu-gprs,
#          -u osmo-bts-virtual-gprs, -u osmo-virtphy-gprs.
# ------------------------------------------------------------------------------
set -uo pipefail

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CFG_SRC="$HERE/configs/gprs"
ETC="${OSMOCOM_CFG:-/etc/osmocom}"
BB="${OSMOCOM_BB:-/opt/GSM/osmocom-bb}"
C54X="${C54X_DIR:-/opt/GSM/c54x_exe}"
ROM_DIR="${OSMO_DSP_ROM_DIR:-/opt/GSM}"
L1="${EGPRS_L1:-auto}"              # auto | virtphy | dsp
WITH_BANC="${EGPRS_BANC:-1}"
NETNS="${EGPRS_NETNS:-msgprs}"
REGEN_GPRS=0
STOP_ALL=0

# The modem subscriber: the 3rd of the plan (01-abonnes-hlr: byte 15 = n, 16 = op).
IMSI="${EGPRS_IMSI:-001010001000001}"
KI="${EGPRS_KI:-00 11 22 33 44 55 66 77 88 99 aa bb cc dd 01 01}"
MSISDN="${EGPRS_MSISDN:-100101}"

SOCK_VIRT=/tmp/modem_l2
SOCK_DSP=/tmp/osmocom_l2
PCU_SOCK=/tmp/pcu_bts3

VTY_BSC=4242; VTY_HLR=4258; VTY_SGSN=4245
PCU_VTY_IP=127.0.0.2; VTY_PCU=4240

U_BTS=osmo-bts-virtual-gprs; U_PCU=osmo-pcu-gprs; U_VPHY=osmo-virtphy-gprs; U_MODEM=osmo-modem-gprs
UNITS=("$U_MODEM" "$U_VPHY" "$U_PCU" "$U_BTS")

# ── display ──────────────────────────────────────────────────────────────────
if [ -t 1 ] && [ -z "${NO_COLOR:-}" ]; then
    C_OK=$'\033[32m'; C_KO=$'\033[31m'; C_WA=$'\033[33m'; C_DIM=$'\033[2m'; C_B=$'\033[1m'; C_Z=$'\033[0m'
else C_OK=; C_KO=; C_WA=; C_DIM=; C_B=; C_Z=; fi
say()  { printf '  %s\n' "$*"; }
ok()   { printf '  %s[ OK ]%s %s\n' "$C_OK" "$C_Z" "$*"; }
ko()   { printf '  %s[ KO ]%s %s\n' "$C_KO" "$C_Z" "$*"; }
warn() { printf '  %s[ !! ]%s %s\n' "$C_WA" "$C_Z" "$*"; }
titre(){ printf '\n%s== %s ==%s\n' "$C_B" "$*" "$C_Z"; }
declare -A BILAN=()
bilan() { BILAN["$1"]="$2"; }

# ── tools ────────────────────────────────────────────────────────────────────
# The Osmocom VTY reads line by line: a pause between commands, otherwise it
# skips some (same as start-interstp.sh --status).
vty() {   # vty HOST PORT cmd...
    local host="$1" port="$2"; shift 2
    { sleep 0.4; printf 'enable\n'; local c; for c in "$@"; do sleep 0.3; printf '%s\n' "$c"; done
      sleep 0.8; printf 'exit\n'; sleep 0.2; } | timeout 10 nc -q1 "$host" "$port" 2>/dev/null | tr -d '\r'
}
port_open() { timeout 1 bash -c "exec 3<>/dev/tcp/${2:-127.0.0.1}/$1" 2>/dev/null; }
wait_until() {   # wait_until SECONDS "label" cmd...
    local to="$1" what="$2"; shift 2; local t=0
    while ! "$@" >/dev/null 2>&1; do
        [ "$t" -ge "$to" ] && { ko "$what : toujours rien apres ${to}s"; return 1; }
        sleep 2; t=$((t+2))
    done
    ok "$what (${t}s)"
}
unit_active() { systemctl is-active --quiet "$1"; }
```

```
unit_run() {   # unit_run NAME cmd...: transient unit, visible to systemctl/journalctl
    local u="$1"; shift
    if unit_active "$u"; then say "$u deja actif"; return 0; fi
    systemctl reset-failed "$u" >/dev/null 2>&1 || true
    systemd-run --quiet --unit="$u" --collect -p KillMode=control-group \
        -p SyslogIdentifier="$u" "$@" || { ko "systemd-run $u a echoue"; return 1; }
    say "$u lance : $*"
}
unit_stop() { unit_active "$1" && systemctl stop "$1" 2>/dev/null; systemctl reset-failed "$1" >/dev/null 2>&1 ||
 ↪  true; }

# ── 1. preflight ──────────────────────────────────────────────────────
DSP_GPRS=0
MODEM_BIN=""
etape_preflight() {
    titre "1. preflight"
    local rc=0 b
    for b in osmo-bts-virtual osmo-pcu virtphy nc systemd-run; do
        command -v "$b" >/dev/null 2>&1 && ok "$b : $(command -v "$b")" || { ko "$b introuvable"; rc=1; }
    done
    # osmocom-bb modem: from the PATH, otherwise the one of the tree; installed
    # (persistently) if it is only in the tree.
    MODEM_BIN="$(command -v modem 2>/dev/null || true)"
    if [ -z "$MODEM_BIN" ] && [ -x "$BB/src/host/layer23/src/modem/modem" ]; then
        install -m755 "$BB/src/host/layer23/src/modem/modem" /usr/local/bin/modem 2>/dev/null \
            && MODEM_BIN=/usr/local/bin/modem && say "modem installe dans /usr/local/bin (depuis $BB)"
    fi
    [ -n "$MODEM_BIN" ] && ok "modem : $MODEM_BIN" || { ko "modem (osmocom-bb layer23) absent : make -C
     ↪  $BB/src/host/layer23"; rc=1; }

    # The DSP chain: TI ROM + c54x_exe (the default bench of start-native.sh)
    local n=0 r
    for r in PROM0 PROM1 PROM2 PROM3 DROM PDROM Registers; do [ -f "$ROM_DIR/calypso_dsp.$r.bin" ] && n=$((n+1));
     ↪  done
    [ "$n" -eq 7 ] && ok "ROM DSP TI : $n/7 dans $ROM_DIR" || warn "ROM DSP : $n/7 dans $ROM_DIR - --dsp ne
     ↪  demarrera pas (c54x_exe/rom/fetch-rom.sh)"
    [ -x "$C54X/c54x_exe" ] && ok "c54x_exe : $C54X/c54x_exe" || warn "c54x_exe non compile : make -C $C54X"

    # The DSP gate: does the layer1 firmware (the one running on the emulated
    # ARM, in front of the ROM) handle the L1CTL GPRS primitives? Without them
    # no PDCH can be opened on this chain, whatever the ROM can do.
    if grep -q 'L1CTL_GPRS' "$BB/src/target/firmware/layer1/l23_api.c" 2>/dev/null; then
        DSP_GPRS=1; ok "porte DSP : le firmware layer1 traite L1CTL_GPRS_* -> PDCH possible sur le C54x"
    else
        DSP_GPRS=0; warn "porte DSP fermee : pas de L1CTL_GPRS_* dans firmware/layer1/l23_api.c -> le modem ira
         ↪  sur virtphy"
    fi
    grep -qi 'gprs' "$C54X"/src/*.c 2>/dev/null && ok "c54x_exe : du code GPRS present" \
        || say "${C_DIM}c54x_exe : aucune tache GPRS cote pont (src/*.c) - a faire quand la porte s ouvrira${C_Z}"
    say "${C_DIM}rappel : la ROM Calypso module en GMSK seulement (GPRS CS-1..4) ; la cellule annonce EGPRS,"
    say "le 8-PSK (MCS-5..9) restera au modem virtphy, jamais au C54x.${C_Z}"

    case "$L1" in
        auto) [ "$DSP_GPRS" = 1 ] && L1=dsp || L1=virtphy ;;
        dsp|virtphy) ;;
        *) ko "--l1 $L1 inconnu (auto|virtphy|dsp)"; rc=1 ;;
    esac
    say "couche 1 du modem : $C_B$L1$C_Z"
    [ "$rc" -eq 0 ] && bilan preflight OK || bilan preflight KO
    return $rc
}

# ── 2. configs ────────────────────────────────────────────────────────
BSC_CHANGE=0
pose_cfg() {   # pose_cfg SRC DST [sed-expr]
    local src="$CFG_SRC/$1" dst="$ETC/$2"
```

```
    [ -r "$src" ] || { ko "gabarit absent : $src"; return 1; }
    if [ -f "$dst" ] && [ "$REGEN_GPRS" = 0 ]; then say "$dst present (--regen-gprs pour le reposer)"; return 0; fi
    sed -e "${3:-}" "$src" > "$dst" && ok "$dst pose"
}
etape_configs() {
    titre "2. configs dans $ETC"
    local rc=0 sgsn_ip gsmtap_ip
    sgsn_ip="$(awk '$1=="listen"{print $2; exit}' "$ETC/osmo-sgsn.cfg" 2>/dev/null)"
    [ -n "$sgsn_ip" ] || { warn "pas de « listen » NS dans osmo-sgsn.cfg - 127.0.0.1"; sgsn_ip=127.0.0.1; }
    gsmtap_ip="$(awk '$1=="log" && $2=="gsmtap"{print $3; exit}' "$ETC/osmo-pcu.cfg" 2>/dev/null)"
    [ -n "$gsmtap_ip" ] || gsmtap_ip=127.0.0.1
    pose_cfg osmo-bts-virtual-gprs.cfg osmo-bts-virtual-gprs.cfg || rc=1
    pose_cfg osmo-pcu-gprs.cfg        osmo-pcu-gprs.cfg "s|__GSMTAP_IP__|$gsmtap_ip|g" || rc=1
    pose_cfg modem-gprs.cfg           modem-gprs.cfg || rc=1
    # DSP variant of the modem: same subscriber, L1CTL socket of the C54x
    # (MOBILE_CFG= in /etc/default/osmo-banc).
    pose_cfg modem-gprs.cfg modem-gprs-dsp.cfg "s|layer2-socket .*|layer2-socket $SOCK_DSP|" || rc=1

    # BTS 2 in osmo-bsc.cfg: a « bts 2 » block inserted before « msc 0 ».
    # --regen regenerates this file from configs/osmo-bsc.cfg (2 BTS): checked
    # at each start, re-inserted when needed.
    local bsc="$ETC/osmo-bsc.cfg"
    if [ ! -f "$bsc" ]; then ko "$bsc absent (start-native.sh --regen)"; rc=1
    elif grep -q '^ bts 2$' "$bsc"; then ok "BTS 2 EGPRS deja dans osmo-bsc.cfg"
    elif ! grep -q '^msc 0$' "$bsc"; then ko "pas de « msc 0 » dans $bsc : je ne sais pas ou inserer la BTS 2"; rc=1
    else
        cp -a "$bsc" "$bsc.avant-bts2-egprs"
        sed "s|__SGSN_IP__|$sgsn_ip|g" "$CFG_SRC/osmo-bsc-bts2-egprs.frag" > "$bsc.frag.$$"
        # Insert before « msc 0 », without touching the rest of the file.
        awk -v frag="$bsc.frag.$$" '
            /^msc 0$/ && !done { while ((getline l < frag) > 0) print l; done=1 }
            { print }' "$bsc.avant-bts2-egprs" > "$bsc"
        rm -f "$bsc.frag.$$"
        grep -q '^ bts 2$' "$bsc" && { ok "BTS 2 EGPRS inseree dans osmo-bsc.cfg (sauvegarde :
        ↪  $bsc.avant-bts2-egprs)"; BSC_CHANGE=1; } \
            || { ko "insertion de la BTS 2 ratee"; cp -a "$bsc.avant-bts2-egprs" "$bsc"; rc=1; }
    fi
    [ "$rc" -eq 0 ] && bilan configs OK || bilan configs KO
    return $rc
}

# ── 3. bench (osmo-banc.service, DSP chain) ──────────────────────────
etape_banc() {
    titre "3. banc DSP (osmo-banc.service)"
    if [ "$WITH_BANC" != 1 ]; then say "--no-banc : on suppose le banc deja monte"
    elif unit_active osmo-banc; then
        ok "osmo-banc actif"
        if [ "$BSC_CHANGE" = 1 ] && unit_active osmo-bsc; then
            warn "la BTS 2 vient d etre ajoutee : osmo-bsc relit sa config -> restart (le mobile DSP recampera)"
            systemctl restart osmo-bsc
        fi
    else
        say "systemctl start osmo-banc (start-native.sh --no-attach, DSP par defaut) - plusieurs minutes"
        systemctl start osmo-banc || { ko "osmo-banc : journalctl -u osmo-banc -n 50"; bilan banc KO; return 1; }
    fi
    wait_until 120 "VTY OsmoBSC :$VTY_BSC"  port_open "$VTY_BSC"  || { bilan banc KO; return 1; }
    wait_until 60  "VTY OsmoHLR :$VTY_HLR"  port_open "$VTY_HLR"  || { bilan banc KO; return 1; }
    wait_until 60  "VTY OsmoSGSN :$VTY_SGSN" port_open "$VTY_SGSN" || { bilan banc KO; return 1; }
    local mode; mode="$(vty 127.0.0.1 $VTY_BSC "show running-config" | awk '/^ bts 0$/{b=1} b && /gprs mode/{print
    ↪  $3; exit}')"
    [ "$mode" = egprs ] && ok "BTS 0 (C54x) : gprs mode egprs" || warn "BTS 0 : gprs mode ${mode:-?}"
    bilan banc OK
}

# ── 4. the modem subscriber in the HLR ──────────────────────────────
etape_abonne() {
```

```
    titre "4. abonne $IMSI (HLR :$VTY_HLR)"
    if vty 127.0.0.1 $VTY_HLR "subscriber imsi $IMSI show" | grep -q "IMSI: *$IMSI"; then
        ok "deja present"
    else
        vty 127.0.0.1 $VTY_HLR "subscriber imsi $IMSI create" \
            "subscriber imsi $IMSI update msisdn $MSISDN" \
            "subscriber imsi $IMSI update aud2g comp128v1 ki $(echo "$KI" | tr -d ' ')" >/dev/null
        vty 127.0.0.1 $VTY_HLR "subscriber imsi $IMSI show" | grep -q "IMSI: *$IMSI" \
            && ok "cree (MSISDN $MSISDN, comp128v1)" || { ko "creation ratee : telnet 127.0.0.1 $VTY_HLR"; bilan
             ↪  abonne KO; return 1; }
    fi
    bilan abonne OK
}

# ── 5. the packet cell: virtual BTS 2 + its PCU ──────────────────────────
etape_cellule() {
    titre "5. cellule EGPRS virtuelle (BTS 2, PCU $PCU_SOCK)"
    rm -f "$PCU_SOCK" 2>/dev/null
    unit_run "$U_BTS" osmo-bts-virtual -c "$ETC/osmo-bts-virtual-gprs.cfg" || { bilan cellule KO; return 1; }
    bts2_oml() { vty 127.0.0.1 $VTY_BSC "show bts 2" | grep -i 'OML' | grep -viE 'disconnected|not connected' |
     ↪  grep -qi 'connected'; }
    wait_until 60 "OML de la BTS 2 au BSC" bts2_oml || { warn "journalctl -u $U_BTS -n 30 ; show bts 2 sur
     ↪  :$VTY_BSC"; }
    unit_run "$U_PCU" osmo-pcu -c "$ETC/osmo-pcu-gprs.cfg" || { bilan cellule KO; return 1; }
    wait_until 30 "socket PCU $PCU_SOCK" test -S "$PCU_SOCK" || warn "la BTS 2 n a pas ouvert $PCU_SOCK"
    pcu_ns() { vty $PCU_VTY_IP $VTY_PCU "show ns" | grep -qiE 'ALIVE|UNBLOCKED'; }
    wait_until 60 "NS-VC PCU <-> SGSN (nsei 11)" pcu_ns || warn "NS : show ns sur $PCU_VTY_IP:$VTY_PCU ;
     ↪  journalctl -u $U_PCU"
    bilan cellule OK
}

# ── 6. the GPRS modem ────────────────────────────────────────────────────
etape_modem() {
    titre "6. modem GPRS (couche 1 : $L1)"
    ip netns list 2>/dev/null | grep -qw "$NETNS" || { ip netns add "$NETNS" && say "netns $NETNS cree"; }
    if [ "$L1" = dsp ]; then
        # QEMU + TI ROM: the modem takes the place of `mobile` on the same L1CTL
        # socket (/tmp/osmocom_l2, osmocon of the DSP bench). MOBILE=/MOBILE_CFG=
        # in /etc/default/osmo-banc is not used: start-native.sh rewrites the
        # file named by MOBILE_CFG with its `mobile` template (gps device...),
        # which `modem` rejects. The DSP bench mobile is therefore stopped and
        # modem started here, as a transient unit.
        local cfg="$ETC/modem-gprs-dsp.cfg"
        sed "s|layer2-socket .*|layer2-socket $SOCK_DSP|" "$CFG_SRC/modem-gprs.cfg" > "$cfg"
        # The layer1 firmware with PDCH (L1CTL_GPRS_*) and the final ROM bursts
        # (UL_RAW=1, pont/dsp/uplink.py): set for the next start of the bench.
        local fw=/opt/GSM/firmware/board/compal_e88/layer1.highram.gprs.elf df=/etc/default/osmo-banc
        if [ -r "$fw" ]; then
            touch "$df"
            grep -q "^FIRMWARE_ELF=$fw" "$df" || { sed -i '/^FIRMWARE_ELF=/d;/^FIRMWARE_BIN=/d' "$df"; printf
             ↪  'FIRMWARE_ELF=%s\nFIRMWARE_BIN=%s\n' "$fw" "${fw%.elf}.bin" >> "$df"; warn "firmware GPRS pose
             ↪  dans $df : « systemctl restart osmo-banc » pour le charger"; }
            grep -q '^UL_RAW=1' "$df" || { sed -i '/^UL_RAW=/d' "$df"; printf 'UL_RAW=1\n' >> "$df"; }
            tr -d '\0' < "/proc/$(pgrep -f 'qemu-system-arm.*calypso' | head -1)/cmdline" 2>/dev/null | grep -q
             ↪  "$fw" \
                && ok "QEMU tourne avec le firmware GPRS" || warn "QEMU ne tourne pas (encore) avec le firmware
                 ↪  GPRS : restart osmo-banc"
        else
            warn "firmware GPRS absent : $fw (make board/compal_e88/layer1.highram.elf dans
             ↪  osmocom-bb/src/target/firmware)"
        fi
        [ -S "$SOCK_DSP" ] || { ko "socket L1CTL $SOCK_DSP absente : le banc DSP (osmocon) ne tourne pas"; bilan
         ↪  modem KO; return 1; }
        # The bench must be in DSP mode: without c54x_exe the layer 1 is gr-gsm
        # (OSMO_BANC_ARGS=--grgsm set by the desktop launcher until reboot) and
        # there is no PDCH there.
```

```
        if ! pgrep -f 'c54x_exe --arm' >/dev/null; then
            ko "le banc tourne en gr-gsm (pas de c54x_exe) : $(systemctl show-environment 2>/dev/null | grep
             ↪  OSMO_BANC_ARGS || echo 'OSMO_BANC_ARGS non pose')"
            say "pour le DSP : systemctl set-environment OSMO_BANC_ARGS=--dsp && systemctl restart osmo-banc"
            bilan modem KO; return 1
        fi
        # Any `mobile` connected to the same L1CTL socket (mobile_pont of the DSP
        # bench, mobile_group1 of the fork...) would fight with the modem: each
        # resets the layer 1 (RESET FULL + PM_REQ versus RESET FULL + FBSB_REQ).
        local p c n=0
        for p in $(pgrep -x mobile); do
            c="$(tr '\0' ' ' < /proc/$p/cmdline 2>/dev/null | sed -n 's/.* -c \([^ ]*\).*/\1/p')"
            [ -n "$c" ] && grep -q "layer2-socket $SOCK_DSP\$" "$c" 2>/dev/null && kill "$p" 2>/dev/null &&
             ↪  n=$((n+1)) && say "mobile $p ($c) arrete : il occupait $SOCK_DSP"
        done
        [ "$n" -gt 0 ] && sleep 1
        unit_run "$U_MODEM" "$MODEM_BIN" -c "$cfg" || { bilan modem KO; return 1; }
        [ "$DSP_GPRS" = 1 ] || warn "porte DSP fermee : le modem campera, mais l attach echouera tant que le
         ↪  firmware layer1 ne traite pas L1CTL_GPRS_* (voir preflight)"
    else
        rm -f "$SOCK_VIRT"
        unit_run "$U_VPHY" virtphy -s "$SOCK_VIRT" -D lo || { bilan modem KO; return 1; }
        wait_until 20 "socket L1CTL $SOCK_VIRT" test -S "$SOCK_VIRT" || { bilan modem KO; return 1; }
        unit_run "$U_MODEM" "$MODEM_BIN" -c "$ETC/modem-gprs.cfg" || { bilan modem KO; return 1; }
    fi
    # Attach and PDP are read from the modem: « show mm-context » of the SGSN
    # does not show « GMM_REGISTERED ». Attach = « Attach success » from the
    # modem; PDP = tun_modem addressed in the netns, set by the modem at the
    # Activate PDP Context Accept.
    local t_debut; t_debut="$(date '+%Y-%m-%d %H:%M:%S')"
    attache() { journalctl -u "$U_MODEM" --since "$t_debut" -o cat 2>/dev/null | grep -q 'Attach success'; }
    wait_until 180 "GPRS attach de $IMSI (Attach success, P-TMSI attribue)" attache || {
       warn "journalctl -u $U_MODEM -n 40 ; show mm-context imsi $IMSI sur :$VTY_SGSN"; bilan modem KO; return 1; }
    pdp() { ip netns exec "$NETNS" ip -4 -o addr show dev tun_modem 2>/dev/null | grep -q 'inet '; }
    wait_until 90 "contexte PDP : adresse sur tun_modem (netns $NETNS)" pdp || { warn "pas de PDP : show
     ↪  pdp-context all ; journalctl -u $U_MODEM"; bilan modem KO; return 1; }
    ok "adresse du modem : $(ip netns exec "$NETNS" ip -4 -o addr show dev tun_modem | awk '{print $4}')"
    local apn_ip; apn_ip="$(ip -4 -o addr show apn0 2>/dev/null | awk '{print $4}' | cut -d/ -f1 | head -1)"
    if [ -n "$apn_ip" ]; then
        ip netns exec "$NETNS" ip route replace default dev tun_modem 2>/dev/null
        if ip netns exec "$NETNS" ping -c4 -W8 "$apn_ip" >/dev/null 2>&1; then ok "ping $apn_ip (apn0) depuis le
         ↪  netns $NETNS : la donnee passe"
        else warn "ping $apn_ip depuis le netns $NETNS rate (ip netns exec $NETNS ip route)"; fi
    fi
    bilan modem OK
}

# ── actions ──────────────────────────────────────────────────────────
do_stop() {
    titre "arret"
    local u; for u in "${UNITS[@]}"; do unit_stop "$u" && say "$u arrete"; done
    rm -f "$SOCK_VIRT" "$PCU_SOCK"
    [ "$STOP_ALL" = 1 ] && { systemctl stop osmo-banc; say "osmo-banc arrete"; }
    return 0
}
do_status() {
    titre "etat"
    local u; for u in osmo-banc "${UNITS[@]}"; do printf '  %-24s %s\n' "$u" "$(systemctl is-active "$u"
     ↪  2>/dev/null)"; done
    port_open "$VTY_SGSN" && { say "SGSN mm-context :"; vty 127.0.0.1 $VTY_SGSN "show mm-context all" | grep -E
     ↪  'IMSI|MM state|Routing|PDP' | sed 's/^/    /'
        say "SGSN pdp-context :"; vty 127.0.0.1 $VTY_SGSN "show pdp-context all" | grep -E 'IMSI|APN|Address|PDP'
         ↪  | sed 's/^/    /'; }
    port_open "$VTY_PCU" "$PCU_VTY_IP" && { say "PCU gprs (tbf) :"; vty $PCU_VTY_IP $VTY_PCU "show tbf all" | grep
     ↪  -vE '^(OsmoPCU|enable|exit|$)' | sed 's/^/    /' | head -12; }
    return 0
```

```
}
do_start() {
    etape_preflight || return 1
    etape_configs   || return 1
    etape_banc      || return 1
    etape_abonne    || true
    # The virtual cell only makes sense for a modem on virtphy: in dsp mode
    # (QEMU + TI ROM) the modem camps on BTS 0 of the C54x, like `mobile`.
    if [ "$L1" = virtphy ] || [ "${EGPRS_VIRTUAL_CELL:-0}" = 1 ]; then etape_cellule || true
    else say "cellule virtuelle (BTS 2) non montee : mode $L1 (EGPRS_VIRTUAL_CELL=1 pour l avoir quand meme)";
     ↪  bilan cellule "--"; fi
    etape_modem     || true
    titre "bilan"
    local k; for k in preflight configs banc abonne cellule modem; do printf '  %-10s %s\n' "$k"
     ↪  "${BILAN[$k]:-??}"; done
    say "${C_DIM}status : $0 status - arret : $0 stop - journaux : journalctl -u $U_MODEM -u $U_PCU -u $U_BTS -u
     ↪  $U_VPHY${C_Z}"
    return 0
}

ACTION=start
while [ $# -gt 0 ]; do
    case "$1" in
        start|stop|status|check) ACTION="$1" ;;
        --l1)        L1="${2:-auto}"; shift ;;
        --l1=*)      L1="${1#*=}" ;;
        --no-banc)   WITH_BANC=0 ;;
        --regen-gprs) REGEN_GPRS=1 ;;
        --all)       STOP_ALL=1 ;;
        -h|--help)   sed -n '2,40p' "$0"; exit 0 ;;
        *) echo "option inconnue : $1" >&2; exit 2 ;;
    esac; shift
done
[ "$(id -u)" -eq 0 ] || { echo "a lancer en root (systemd-run, ip netns)" >&2; exit 1; }
case "$ACTION" in
    start)  do_start ;;
    stop)   do_stop ;;
    status) do_status ;;
    check)  etape_preflight; etape_configs ;;
esac
```

2.193 /opt/GSM/osmo-operator/scripts/entrypoint.sh

1811 octets, 63 lignes → 63 lignes

```
#!/bin/bash
set -e

# --- TUN device ---
echo "[*] Initialisation du peripherique TUN pour osmo-ggsn..."
mkdir -p /dev/net
if [ ! -c /dev/net/tun ]; then
    # Create the device node (c = character, 10 = major, 200 = minor)
    mknod /dev/net/tun c 10 200
    chmod 666 /dev/net/tun
fi

# --- Environment ---
container=docker
export container

# A command is required
if [ $# -eq 0 ]; then
  echo >&2 'ERROR: No command specified. You probably want to run bash or a script.'
```

```
  exit 1
fi

# Export the variables for the systemd sessions
env > /etc/docker-entrypoint-env

# Create the startup service
quoted_args="$(printf " %q" "${@}")"
echo "${quoted_args}" > /etc/docker-entrypoint-cmd

cat >/etc/systemd/system/docker-entrypoint.service <<EOT
[Unit]
Description=Lancement de la stack Osmocom Simulee
After=network.target

[Service]
ExecStart=/bin/bash -exc "source /etc/docker-entrypoint-cmd"
StandardInput=tty-force
StandardOutput=inherit
StandardError=inherit
WorkingDirectory=/etc/osmocom
EnvironmentFile=/etc/docker-entrypoint-env

[Install]
WantedBy=multi-user.target
EOT

# Mask the conflicting systemd services
systemctl mask systemd-firstboot.service systemd-udevd.service systemd-modules-load.service \
               systemd-udevd-kernel systemd-udevd-control 2>/dev/null || true
systemctl enable docker-entrypoint.service

# Locate and exec systemd
if [ -x /lib/systemd/systemd ]; then
  exec /lib/systemd/systemd --show-status=false --unit=multi-user.target
elif [ -x /usr/lib/systemd/systemd ]; then
  exec /usr/lib/systemd/systemd --show-status=false --unit=multi-user.target
else
  echo >&2 'ERROR: systemd is not installed'
  exit 1
fi

# NOTE: RUN/COPY/EXPOSE directives belong in the Dockerfile, not in this script.
# exec systemd replaces the process - nothing after that line is executed.
```

2.194 /opt/GSM/osmo-operator/scripts/gapk-start.sh

36081 octets, 703 lignes → 703 lignes

```
#!/bin/bash
# gapk-start.sh - GSM audio manager (osmo-gapk ↔ ALSA/RTP)
#
# ═══════════════════════════════════════════════════════════════════════
# DUAL-PATH AUDIO ARCHITECTURE (osmo-nitb-for-calypso v2)
# ═══════════════════════════════════════════════════════════════════════
#
# PATH 1 - Baseband (mobile l1phy):
#   mobile process ← TCH frames → trxcon → fake_trx → osmo-bts-trx
#                  ↓ io-handler l1phy
#                  ALSA device (direct GSM-FR decoding, zero network latency)
#
#   → Enabled by tch-voice { io-handler l1phy } in mobile.cfg
#   → No gapk needed for this path
#   → Simulates a physical GSM handset
```

```
#
# PATH 2 - Network (MGW RTP):
#   BTS → BSC → MSC → MNCC → Asterisk → SIP → Linphone
#                          → MGCP → OsmoMGW → RTP
#                                             ↓
#                                        osmo-gapk (monitor/record/bridge)
#
#   → osmo-gapk hooks onto the MGW RTP ports
#   → Useful for: monitoring, recording, audio injection, tests
#   → Linphone receives the audio directly over SIP (no gapk needed)
#
# SUMMARY:
#   • MS → MS call (same operator)   : audio via l1phy ALSA (path 1)
#   • MS → Linphone call             : audio via Asterisk SIP (path 2)
#   • Network monitoring             : gapk auto/monitor mode (path 2)
#   • Call recording                 : gapk record mode (path 2)
#
# ═══════════════════════════════════════════════════════════════════════

# ═══════════════════════════════════════════════════════════════════════
# osmo-gapk CLI CONTRACT - REREAD BEFORE TOUCHING AN INVOCATION
# ═══════════════════════════════════════════════════════════════════════
# The contract (osmo-gapk --help, and src/app_osmo_gapk.c l.206-295):
#
#   -i FILE           FILE input              -o FILE      FILE output
#   -I HOST/PORT      RTP input               -O HOST/PORT RTP output
#   -a DEVICE         ALSA input              -A DEVICE    ALSA output
#   -f FORMAT         INPUT format            -g FORMAT    OUTPUT format
#   -p PT             RTP payload type in     -P PT        RTP payload type out
#   -t                throttle (one codec frame / 20 ms)
#
# Traps:
#   • `-i` is NOT `-I`. A URL passed to `-i` becomes a FILE NAME.
#   • `-f` alone is not enough: without `-g`, no output format is defined.
#   • The FR format name is "gsm" (.gsm file AND RFC3551 RTP payload), NOT
#     "gsmfr". `osmo-gapk --help` is authoritative; the full list is
#     gsm | rawpcm-s16le | rtp-efr | rtp-hr-etsi | rtp-hr-ietf | amr-* | ti-* |
#     racal-* | hr-ref-*.
#   • Only one codec is enc+dec in this build: `fr`. hr/efr/amr/pcm are
#     format-only (empty "enc dec" column in --help). The network uses
#     payload type 3 = GSM FR (osmo-mgw "payload-types:3=GSM").
#   • `-t` ONLY on a FILE input. On RTP the network paces, on ALSA the card
#     does: throttling on top desynchronises.
# ═══════════════════════════════════════════════════════════════════════

set -euo pipefail

GAPK_DEFAULT_FORMAT="${GAPK_FORMAT:-gsm}"
GAPK_DEFAULT_DEVICE="${GAPK_ALSA_DEV:-${ALSA_CARD:-default}}"
# CAPTURE device distinct from the PLAYBACK device. Capturing where the
# mobiles write (gsm_out) is the closed loop documented in /etc/asound.conf.
# The capture must target gsm_in (= monitor of the gsm_mic null-sink, which
# nobody feeds). NEVER let this default fall back to $GAPK_DEFAULT_DEVICE.
GAPK_DEFAULT_DEVICE_IN="${GAPK_ALSA_DEV_IN:-gsm_in}"
# RTP payload type of GSM FR (RFC3551). OsmoMGW negotiates "3=GSM".
GAPK_RTP_PT="${GAPK_RTP_PT:-3}"
GAPK_LOG_DIR="${GAPK_LOG_DIR:-/var/log/osmocom}"
GAPK_REC_DIR="${GAPK_REC_DIR:-/var/lib/gapk}"
# GSM frame period. osmo-gapk hardcodes it (option -t): this variable only
# documents the cadence in messages.
GAPK_FRAME_MS=20
MGW_VTY_HOST="127.0.0.1"
MGW_VTY_PORT="4243"
AUTO_POLL_INTERVAL="${GAPK_POLL_INTERVAL:-2}"

GAPK_PID_RX="/var/run/gapk-rx.pid"
```

```
GAPK_PID_TX="/var/run/gapk-tx.pid"
GAPK_AUTO_LOCK="/var/run/gapk-auto.lock"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; NC='\033[0m'; BOLD='\033[1m'

mkdir -p "$GAPK_LOG_DIR" "$GAPK_REC_DIR"

log_info()  { echo -e "${GREEN}[gapk]${NC} $*"; }
log_warn()  { echo -e "${YELLOW}[gapk]${NC} $*"; }
log_error() { echo -e "${RED}[gapk ERROR]${NC} $*" >&2; }
log_auto()  { echo -e "${CYAN}[gapk-auto]${NC} $(date '+%H:%M:%S') $*"; }
# The host's heavy audio chain (echo canceller, loops, recording) only runs
# during a call: set up on « Endpoint actif », torn down on « Endpoint
# disparu ». In the background, so as not to delay the polling. See
# scripts/audio-chain.sh --appel.
_audio_appel() {
    [ "${AUDIO_PENDANT_APPEL:-0}" = "1" ] || return 0
    local chaine="${OSMO_OPERATOR_DIR:-/opt/GSM/osmo-operator}/scripts/audio-chain.sh"
    [ -f "$chaine" ] || return 0
    bash "$chaine" --appel "$1" 3 >>"${GAPK_LOG_DIR:-/tmp}/audio-appel.log" 2>&1 &
}

# ── Checks ──────────────────────────────────────────────────────────────
check_gapk() {
    command -v osmo-gapk >/dev/null 2>&1 || {
        log_error "osmo-gapk introuvable."
        log_error "Verifier que le Dockerfile inclut l'etape osmo-gapk --enable-alsa."
        exit 1
    }
}

# No PCM of this chain is a sound card: gsm_out and gsm_in are `type pulse`
# (/etc/asound.conf) and end up in null-sinks, so the presence of /dev/snd
# says nothing (a server without a sound card is the project's target, and
# start-docker.sh does not pass `--device /dev/snd`). What WILL BE USED is
# tested instead: does the PCM open? Same gesture as assert_audio_devices()
# in lib/audio.sh - one second of silence, and a verdict on the real device.
check_alsa() {
    local dev="${1:-default}"
    command -v aplay >/dev/null 2>&1 || {
        log_warn "aplay absent - impossible de verifier le PCM '${dev}'"
        return 1
    }
    if timeout 5 aplay -D "$dev" -f S16_LE -r 8000 -c 1 -d 1 /dev/zero >/dev/null 2>&1; then
        return 0
    fi
    log_warn "PCM ALSA '${dev}' ne s'ouvre pas."
    [ -d /dev/snd ] || log_warn "  (/dev/snd absent - sans importance si le PCM est 'type pulse')"
    pactl info >/dev/null 2>&1 \
        || log_warn "  PulseAudio injoignable (PULSE_SERVER=${PULSE_SERVER:-non defini})"
    log_warn "  verifier /etc/asound.conf et 'pactl list short sinks' (gsm_audio + gsm_mic)"
    return 1
}

# ── Mobile audio mode detection ─────────────────────────────────────────
detect_audio_mode() {
    # The mobile carries ITS OWN audio chain as soon as an io-handler (gapk or
    # l1phy) is declared in one of its cfg files: it is the one reading gsm_in
    # and writing gsm_out. Every mobile*.cfg is inspected (mobile_group1.cfg,
    # mobile_faketrx_bts1.cfg...).
    #   mobile  : the mobile process does the audio (io-handler gapk|l1phy)
    #   l1phy   : same, kept for existing callers
    #   gapk    : no mobile has audio -> path 2 (RTP<->ALSA bridge)
    local cfg
    for cfg in "${OSMOCOM_HOME:-/root/.osmocom}"/bb/mobile*.cfg /root/.osmocom/bb/mobile*.cfg; do
```

```
        [ -f "$cfg" ] || continue
        grep -q "io-handler l1phy" "$cfg" 2>/dev/null && { echo "l1phy"; return 0; }
        grep -q "io-handler gapk"  "$cfg" 2>/dev/null && { echo "mobile"; return 0; }
    done
    echo "gapk"
}

# ── RTP addressing ──────────────────────────────────────────────────────
# osmo-gapk splits HOST/PORT at the SLASH (app_osmo_gapk.c:190, strtok(dup, "/")).
# A "127.0.0.1:16386" gives it host="127.0.0.1:16386" then strtok NULL ->
# -EINVAL -> "Invalid port". This whole script, its public CLI and the output
# of mode_mgw_ports speak IP:PORT. Normalised HERE, once, so that the visible
# contract of the script stays unchanged.
rtp_hostport() {
    local hp="$1"
    # already HOST/PORT: left alone. Otherwise last ':' -> '/'.
    case "$hp" in
        */*) echo "$hp" ;;
        *)   echo "${hp%:*}/${hp##*:}" ;;
    esac
}

# ── Background lifecycle ────────────────────────────────────────────────
run_bg() {
    local pid_file="$1"; shift
    local log_file="$1"; shift
    osmo-gapk "$@" >> "$log_file" 2>&1 &
    echo $! > "$pid_file"
    log_info "PID $(cat "$pid_file") → $(basename "$log_file")"
}

stop_pid() {
    local pid_file="$1"
    [ -f "$pid_file" ] || return 0
    local pid; pid=$(cat "$pid_file")
    if kill -0 "$pid" 2>/dev/null; then
        kill "$pid" 2>/dev/null && log_info "Arrete PID $pid" || true
    fi
    rm -f "$pid_file"
}

is_running() {
    local pid_file="$1"
    [ -f "$pid_file" ] && kill -0 "$(cat "$pid_file")" 2>/dev/null
}

# ═══════════════════════════════════════════════════════════════════════
# info - shows the current audio architecture
# ═══════════════════════════════════════════════════════════════════════
mode_info() {
    local audio_mode
    audio_mode=$(detect_audio_mode)

    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════════════════════╗"
    echo "║        Architecture audio osmo-nitb-for-calypso           ║"
    echo "╚═══════════════════════════════════════════════════════════╝"
    echo -e "${NC}"

    echo -e "  Mode mobile.cfg : ${BOLD}${audio_mode}${NC}"
    echo ""

    if [ "$audio_mode" = "l1phy" ]; then
        echo -e "  ${YELLOW}● Chemin 1 - Baseband (l1phy)${NC}"
        echo -e "    mobile.cfg a tch-voice l1phy mais fake_trx ne transporte"
        echo -e "    PAS le payload audio TCH (bursts downlink = zeros)."
```

```
        echo -e "    → Pas de son direct sur les MS virtuels."
        echo ""
        echo -e "  ${GREEN}● Chemin 2 - Reseau (SIP/RTP) - SEUL CHEMIN AUDIO${NC}"
        echo -e "    MS appelle → Asterisk → SIP → Linphone (audio natif)"
        echo -e "    Linphone appelle → Asterisk → MNCC → MSC → MS (signalisation)"
        echo -e "    → L'audio du lab passe par Linphone."
    else
        echo -e "  ${GREEN}● Mode gapk classique${NC}"
        echo -e "    osmo-gapk bridge RTP ↔ ALSA"
        echo -e "    → Requis pour l'audio des appels MS"
    fi

    echo ""
    echo -e "  ${BOLD}Linphone :${NC}"
    local inter_ip
    inter_ip=$(ip -4 addr show | grep '172\.20\.0\.' | awk '{print $2}' | cut -d/ -f1 | head -1 || echo
     ↪  "172.20.0.X")
    echo -e "    SIP server : ${CYAN}${inter_ip}:5060${NC}"
    echo -e "    linphone_A : user=${CYAN}linphone_A${NC}  pass=${CYAN}tester${NC}   → 100"
    echo -e "    linphone_B : user=${CYAN}linphone_B${NC}  pass=${CYAN}testerB${NC}  → 200"
    echo ""

    # MGW ports
    mode_mgw_ports "" 2>/dev/null || true
}

# ═══════════════════════════════════════════════════════════════════
# mgw-ports
# ═══════════════════════════════════════════════════════════════════
mode_mgw_ports() {
    local quiet="${1:-}"
    local mgw_out

    mgw_out=$( (echo "show mgcp"; sleep 1; echo "exit") | telnet "$MGW_VTY_HOST" "$MGW_VTY_PORT" 2>/dev/null \
        | grep -v "^Trying\|^Connected\|^Escape\|Welcome\|OsmoMGW[>#]\|Copyright\|License\|Contributions\|free
         ↪  software\|NO WARRANTY\|Based on") || true

    if [ -z "$mgw_out" ]; then
        [ -z "$quiet" ] && echo "(MGW non disponible ou aucun endpoint)"
        return 1
    fi

    # Actual OsmoMGW format:
    #   Virtual trunk 0 endpoint rtpbridge/1@mgw:
    #      CONN: (9/rtp C:51E1AF8F r=127.0.0.1:4072<->l=127.0.0.1:4068)
    #      CONN: (9/rtp C:96FA1465 r=127.0.0.1:30004<->l=127.0.0.1:4066)

    if [ -z "$quiet" ]; then
        echo -e "${CYAN}── Endpoints OsmoMGW actifs ──${NC}"
        local has_conn=false
        echo "$mgw_out" | while IFS= read -r line; do
            if echo "$line" | grep -q "endpoint.*@mgw"; then
                local ep; ep=$(echo "$line" | grep -oP 'rtpbridge/\S+')
                # Peek if next lines have CONN
                continue
            fi
            if echo "$line" | grep -q "CONN:.*r="; then
                has_conn=true
                local remote local_p
                remote=$(echo "$line" | grep -oP 'r=\K[0-9.]+:[0-9]+')
                local_p=$(echo "$line" | grep -oP 'l=\K[0-9.]+:[0-9]+')
                echo -e "  ${GREEN}●${NC} local=${CYAN}${local_p}${NC}  remote=${CYAN}${remote}${NC}"
            fi
        done
        # has_conn is assigned INSIDE the pipeline subshell (`echo | while`)
            # and lost on exit: the verdict is taken from mgw_out directly.
```

```
            if ! echo "$mgw_out" | grep -q "CONN:.*r="; then
            echo "  (aucune connexion active)"
        fi
    else
        # Machine mode: "LOCAL_PORT REMOTE_IP:REMOTE_PORT" per active connection
        echo "$mgw_out" | grep "CONN:.*r=" | while IFS= read -r line; do
            local remote local_p
            remote=$(echo "$line" | grep -oP 'r=\K[0-9.]+:[0-9]+')
            # OsmoMGW writes 'l=IP:PORT': the character class must include the
            # dot, otherwise grep stops at the first '.' and returns EMPTY, the
            # machine mode outputs NO line and mode_auto never launches gapk.
            # CAUTION: the SOURCE is here (scripts/), /etc/osmocom holds a COPY
            # regenerated at startup - patching the copy alone does not survive.
            local_p=$(echo "$line" | grep -oP 'l=\K[0-9.]+:[0-9]+' | cut -d: -f2)
            [ -n "$local_p" ] && [ -n "$remote" ] && echo "$local_p $remote"
        done
    fi
}

# ═══════════════════════════════════════════════════════════════════
# call - bidirectional ALSA ↔ RTP call (path 2 only)
# ═══════════════════════════════════════════════════════════════════
mode_call() {
    local rx_port="${1:?RTP_RX_PORT requis (port MGW downlink)}"
    local tx_dest="${2:?RTP_TX_DEST:PORT requis (dest MGW uplink)}"
    local fmt="${3:-$GAPK_DEFAULT_FORMAT}"
    local dev="${4:-$GAPK_DEFAULT_DEVICE}"
    local dev_in="${5:-$GAPK_DEFAULT_DEVICE_IN}"

    check_gapk
    check_alsa "$dev" || true

    stop_pid "$GAPK_PID_RX"
    stop_pid "$GAPK_PID_TX"

    local audio_mode; audio_mode=$(detect_audio_mode)
    if [ "$audio_mode" = "l1phy" ]; then
        echo -e "${YELLOW}${BOLD}NOTE: mobile.cfg utilise io-handler l1phy${NC}"
        echo -e "${YELLOW}L'audio MS passe directement via ALSA (pas via gapk).${NC}"
        echo -e "${YELLOW}Ce mode gapk bridge le RTP reseau (MGW) ↔ ALSA.${NC}"
        echo -e "${YELLOW}Pour les appels MS↔MS, l'audio est deja gere par mobile.${NC}"
        echo ""
    fi

    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════╗"
    echo "║   gapk - Bridge RTP reseau ↔ ALSA         ║"
    echo "╚═══════════════════════════════════════════╝"
    echo -e "${NC}"
    printf "  Codec     : ${CYAN}%s${NC}\n" "$fmt"
    printf "  Periph HP : ${CYAN}%s${NC}\n" "$dev"
    printf "  Periph mic: ${CYAN}%s${NC}\n" "$dev_in"
    printf "  RX ecoute : ${CYAN}0.0.0.0:%s${NC}  (MGW downlink → HP)\n" "$rx_port"
    printf "  TX envoi  : ${CYAN}%s${NC}  (micro → MGW uplink)\n" "$tx_dest"
    echo ""

    local log_rx="${GAPK_LOG_DIR}/gapk-rx.log"
    local log_tx="${GAPK_LOG_DIR}/gapk-tx.log"

    # RX: RTP (FR) → fr codec decoding → PCM → ALSA playback
    log_info "RX : rtp/${rx_port} → alsa:${dev}  [${fmt} → rawpcm-s16le]"
    run_bg "$GAPK_PID_RX" "$log_rx" \
        -I "0.0.0.0/${rx_port}" -f "$fmt" -p "$GAPK_RTP_PT" \
        -A "$dev"               -g rawpcm-s16le

    sleep 0.3
```

```
    # TX: ALSA capture → PCM → fr codec encoding → RTP (FR)
    log_info "TX : alsa:${dev_in} → rtp/${tx_dest}  [rawpcm-s16le → ${fmt}]"
    run_bg "$GAPK_PID_TX" "$log_tx" \
        -a "$dev_in"                    -f rawpcm-s16le \
        -O "$(rtp_hostport "$tx_dest")"  -g "$fmt" -P "$GAPK_RTP_PT"

    echo ""
    log_info "Bridge actif. Arreter : gapk-start.sh stop"
}

# ═══════════════════════════════════════════════════════════════════
# auto - watches the MGW, automatic gapk bridge
# ═══════════════════════════════════════════════════════════════════
mode_auto() {
    local fmt="${1:-$GAPK_DEFAULT_FORMAT}"
    local dev="${2:-$GAPK_DEFAULT_DEVICE}"
    local dev_in="${3:-$GAPK_DEFAULT_DEVICE_IN}"

    check_gapk
    check_alsa "$dev" || log_warn "ALSA absent - mode RTP-only"

    local audio_mode; audio_mode=$(detect_audio_mode)

    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════════════════╗"
    echo "║   gapk-auto - Integration OsmoMGW (chemin reseau)   ║"
    echo "╚═══════════════════════════════════════════════════════╝"
    echo -e "${NC}"

    if [ "$audio_mode" = "l1phy" ]; then
        log_auto "Mode l1phy detecte → audio MS via mobile process"
        log_auto "gapk surveille le chemin reseau (MGW RTP) uniquement"
        log_auto "Pour les appels MS↔MS, l'audio est gere nativement"
        echo ""
    fi

    log_auto "Codec=${fmt}  HP=${dev}  Mic=${dev_in}  Poll=${AUTO_POLL_INTERVAL}s"
    log_auto "Ctrl+C pour arreter"
    echo ""

    touch "$GAPK_AUTO_LOCK"
    local prev_rx_port=""

    cleanup_auto() {
        log_auto "Arret..."
        stop_pid "$GAPK_PID_RX"
        stop_pid "$GAPK_PID_TX"
        rm -f "$GAPK_AUTO_LOCK"
        exit 0
    }
    trap cleanup_auto SIGINT SIGTERM

    while [ -f "$GAPK_AUTO_LOCK" ]; do
        local first_active
        first_active=$(mode_mgw_ports "quiet" 2>/dev/null | head -1) || first_active=""

        if [ -n "$first_active" ]; then
            local rx_port tx_dest
            rx_port=$(echo "$first_active" | awk '{print $1}')
            tx_dest=$(echo "$first_active" | awk '{print $2}')

            if [ "$rx_port" != "$prev_rx_port" ]; then
                if [ -n "$prev_rx_port" ]; then
                    log_auto "Changement endpoint (${prev_rx_port} → ${rx_port})"
                    stop_pid "$GAPK_PID_RX"
```

```
            stop_pid "$GAPK_PID_TX"
        fi

        log_auto "Endpoint actif : RX=${rx_port}  TX=${tx_dest}"
        [ -z "$prev_rx_port" ] && _audio_appel debut

        local log_rx="${GAPK_LOG_DIR}/gapk-rx.log"
        local log_tx="${GAPK_LOG_DIR}/gapk-tx.log"

        # ── RX LEG: ARCHITECTURALLY IMPOSSIBLE, NOT LAUNCHED ──
        # `-I 0.0.0.0/$rx_port` fails with "Address already in use":
        # $rx_port is OsmoMGW's LOCAL port, it already holds it. Two
        # processes cannot listen on the same UDP port, and above all
        # the MGW does not SEND to that port - it sends to its remote
        # peer. Receiving a copy of an MGW stream requires the MGW to
        # address it to us, i.e. an extra MGCP connection on the
        # endpoint: that is settled on the MGCP side, never on gapk's.
        # TO LISTEN to the network audio: Asterisk already records
        # every call (MixMonitor, cf. sub-record in extensions.conf),
        # one file per direction: *-ul.wav / *-dl.wav.
        log_auto "RX non lancee : ${rx_port} appartient a OsmoMGW (voir le commentaire)"
        log_auto "  ecoute reseau -> enregistrements MixMonitor (*-ul.wav / *-dl.wav)"

        # ── TX LEG: SAME VERDICT AS THE RX, NOT LAUNCHED BY DEFAULT ──
        # $tx_dest is the REMOTE PEER of the first MGW connection - in
        # practice the RTP port of osmo-bts (127.0.0.1:16384, cf.
        # osmo-mgw.log « Got media info via SDP: port:16384 ... 3=GSM »
        # on rtpbridge/2). This leg would encode the mic (gsm_in) in
        # GSM-FR and SEND it TO THE BTS, on the same port as the MGW's
        # legitimate stream (the voice coming back from Asterisk): TWO
        # interleaved RTP streams - two SSRC, two clocks - in a single
        # 100 ms jitter buffer, with the mic fed straight into the
        # mobile's ear.
        # The mobile (io-handler gapk, alsa gsm_in/gsm_out) already
        # does the mic <-> TCH round trip over the radio: this leg is a
        # DUPLICATE by construction.
        # GAPK_AUTO_TX=1 re-enables it for path 2 only (no mobile with
        # audio); by default the supervisor only observes.
        if [ "${GAPK_AUTO_TX:-0}" = "1" ] && [ "$audio_mode" = "gapk" ]; then
            # TX: ALSA capture → PCM → fr codec (encode) → RTP (FR)
            run_bg "$GAPK_PID_TX" "$log_tx" \
                -a "$dev_in"                    -f rawpcm-s16le \
                -O "$(rtp_hostport "$tx_dest")"  -g "$fmt" -P "$GAPK_RTP_PT"
            log_auto "Audio actif (PID RX=$(cat $GAPK_PID_RX 2>/dev/null) TX=$(cat $GAPK_PID_TX
             ↪  2>/dev/null))"
        else
            log_auto "TX non lancee : ${tx_dest} est le pair du MGW (BTS/Asterisk), pas une entree libre"
          log_auto "  le montant est porte par le mobile (mode ${audio_mode}) - GAPK_AUTO_TX=1 pour forcer"
        fi

        prev_rx_port="$rx_port"
    fi

    # Only the TX leg is watched, and only if it was launched: the RX
    # is not launched (see above), requiring it would restart the pair
    # endlessly.
    if [ -f "$GAPK_PID_TX" ] && ! is_running "$GAPK_PID_TX"; then
        log_warn "Processus gapk mort - restart"
        stop_pid "$GAPK_PID_RX"; stop_pid "$GAPK_PID_TX"
        prev_rx_port=""
    fi
else
    if [ -n "$prev_rx_port" ]; then
        log_auto "Endpoint disparu - arret gapk"
        _audio_appel fin
        stop_pid "$GAPK_PID_RX"; stop_pid "$GAPK_PID_TX"
```

```
                prev_rx_port=""
            fi
        fi
        sleep "$AUTO_POLL_INTERVAL"
    done
}

# ═══════════════════════════════════════════════════════════════════
# monitor - passive RTP → ALSA listening
# ═══════════════════════════════════════════════════════════════════
mode_monitor() {
    local rx_port="${1:?RTP_RX_PORT requis}"
    local fmt="${2:-$GAPK_DEFAULT_FORMAT}"
    local dev="${3:-$GAPK_DEFAULT_DEVICE}"

    check_gapk; check_alsa "$dev" || true
    stop_pid "$GAPK_PID_RX"

    log_info "Monitor : rtp/${rx_port} → alsa:${dev}  [${fmt} → rawpcm-s16le]"
    exec osmo-gapk \
        -I "0.0.0.0/${rx_port}" -f "$fmt" -p "$GAPK_RTP_PT" \
        -A "$dev"               -g rawpcm-s16le
}

# ═══════════════════════════════════════════════════════════════════
# record - RTP → .gsm file
# ═══════════════════════════════════════════════════════════════════
mode_record() {
    local rx_port="${1:?RTP_RX_PORT requis}"
    local outfile="${2:-${GAPK_REC_DIR}/record_$(date '+%Y%m%d_%H%M%S').gsm}"
    local fmt="${3:-$GAPK_DEFAULT_FORMAT}"

    check_gapk; stop_pid "$GAPK_PID_RX"

    # File output in the SAME format as the input: the FR frames are archived
    # as is (.gsm readable by gapk playback and by grgsm_decode -d FR).
    log_info "Record : rtp/${rx_port} → ${outfile}  [${fmt}]"
    exec osmo-gapk \
        -I "0.0.0.0/${rx_port}" -f "$fmt" -p "$GAPK_RTP_PT" \
        -o "$outfile"           -g "$fmt"
}

# ═══════════════════════════════════════════════════════════════════
# playback - .gsm file → RTP
# ═══════════════════════════════════════════════════════════════════
mode_playback() {
    local infile="${1:?INPUT_FILE requis}"
    local tx_dest="${2:?RTP_TX_DEST:PORT requis}"
    local fmt="${3:-$GAPK_DEFAULT_FORMAT}"

    check_gapk
    [ -f "$infile" ] || { log_error "Fichier introuvable : $infile"; exit 1; }

    # -t MANDATORY here: the input is a FILE, nothing paces the reading.
    # Without throttle gapk drains the file at full speed and floods the
    # jitter buffer on the other side (minutes of audio sent in seconds).
    log_info "Playback : ${infile} → rtp/${tx_dest}  [${fmt}]"
    exec osmo-gapk -t \
        -i "$infile"                    -f "$fmt" \
        -O "$(rtp_hostport "$tx_dest")"  -g "$fmt" -P "$GAPK_RTP_PT"
}

# ═══════════════════════════════════════════════════════════════════
# loopback - ALSA mic → codec → ALSA speaker
# ═══════════════════════════════════════════════════════════════════
mode_loopback() {
```

```
    local fmt="${1:-$GAPK_DEFAULT_FORMAT}"
    local dev="${2:-$GAPK_DEFAULT_DEVICE}"
    local dev_in="${3:-$GAPK_DEFAULT_DEVICE_IN}"

    check_gapk
    check_alsa "$dev" || { log_error "ALSA requis."; exit 1; }

    echo -e "${YELLOW}${BOLD}⚠  UTILISEZ DES ECOUTEURS - risque de larsen !${NC}"
    # WHAT THIS MODE PROVES, AND WHAT IT DOES NOT.
    # osmo-gapk builds ONE chain "input format -> canonical -> output format":
    # the codec only appears on the side where the format IS a codec format.
    # PCM -> fr -> PCM is therefore NOT expressible in one invocation, and
    # $fmt is deliberately not used here. This loopback validates the ALSA
    # path (capture, cadence, devices), NOT the FR vocoder.
    # To test the vocoder: `record` then `playback` of the resulting .gsm.
    log_info "Loopback ALSA (sans vocodeur) : alsa:${dev_in} → alsa:${dev}"
    exec osmo-gapk \
        -a "$dev_in" -f rawpcm-s16le \
        -A "$dev"    -g rawpcm-s16le
}

# ═══════════════════════════════════════════════════════════════════
# stop / status
# ═══════════════════════════════════════════════════════════════════
mode_stop() {
    stop_pid "$GAPK_PID_RX"
    stop_pid "$GAPK_PID_TX"
    rm -f "$GAPK_AUTO_LOCK"
    pkill -x osmo-gapk 2>/dev/null && log_info "Toutes instances arretees" \
        || log_info "Aucune instance active"
}

mode_status() {
    local audio_mode; audio_mode=$(detect_audio_mode)
    echo -e "${CYAN}── Etat audio ──${NC}"
    echo -e "  Mode mobile.cfg : ${BOLD}${audio_mode}${NC}"
    echo ""

    local any=0
    for f in "$GAPK_PID_RX" "$GAPK_PID_TX"; do
        [ -f "$f" ] || continue
        local pid; pid=$(cat "$f")
        if kill -0 "$pid" 2>/dev/null; then
            local cmdline; cmdline=$(cat "/proc/${pid}/cmdline" 2>/dev/null \
                | tr '\0' ' ' | cut -c1-80 || echo "?")
            echo -e "  ${GREEN}●${NC} PID ${pid}  [$(basename "$f")]  ${cmdline}"
            any=1
        else
            echo -e "  ${RED}✗${NC} PID ${pid} mort"; rm -f "$f"
        fi
    done
    [ -f "$GAPK_AUTO_LOCK" ] && echo -e "  ${CYAN}●${NC} mode AUTO actif" && any=1
    [ $any -eq 0 ] && echo -e "  ${YELLOW}Aucune instance gapk active${NC}"

    if [ "$audio_mode" = "l1phy" ]; then
        echo ""
        echo -e "  ${GREEN}●${NC} Audio MS via l1phy (natif dans mobile process)"
        if pgrep -f "mobile.*mobile" >/dev/null 2>&1; then
            echo -e "  ${GREEN}●${NC} Processus mobile actif"
        else
            echo -e "  ${YELLOW}○${NC} Processus mobile non detecte"
        fi
    fi

    if pgrep -x osmo-gapk >/dev/null 2>&1; then
        echo ""; echo -e "${CYAN}Processus gapk :${NC}"
```

```
        ps -o pid,args --no-headers -C osmo-gapk 2>/dev/null | sed 's/^/  /' || true
    fi
}

mode_list_codecs() {
    check_gapk
    # `--list-codecs` does not exist in osmo-gapk: --help is queried directly,
    # it is the source of truth for the names.
    echo -e "${CYAN}Codecs et formats supportes (osmo-gapk --help) :${NC}"
    osmo-gapk --help 2>&1 | sed -n '/Supported codecs:/,$p'
}

mode_list_devices() {
    echo -e "${CYAN}── Peripheriques ALSA ──${NC}"
    echo "Lecture (HP) :"
    aplay -L 2>/dev/null | head -20 || echo "  (aplay indisponible)"
    echo ""; echo "Capture (micro) :"
    arecord -L 2>/dev/null | head -20 || echo "  (arecord indisponible)"
    echo ""; echo "Cartes son :"
    cat /proc/asound/cards 2>/dev/null || echo "  (relancer avec --device /dev/snd)"

    local audio_mode; audio_mode=$(detect_audio_mode)
    if [ "$audio_mode" = "l1phy" ]; then
        echo ""
        echo -e "${CYAN}── Config l1phy (mobile.cfg) ──${NC}"
        grep -A3 "tch-voice" /root/.osmocom/bb/mobile.cfg 2>/dev/null | sed 's/^/  /' || true
    fi
}

# ═══════════════════════════════════════════════════════════════════════
# Usage
# ═══════════════════════════════════════════════════════════════════════
usage() {
    local audio_mode; audio_mode=$(detect_audio_mode)
    cat <<EOF
Usage: gapk-start.sh <mode> [options]

  Mode audio mobile.cfg : ${audio_mode}
  $([ "$audio_mode" = "l1phy" ] && echo "  → Audio MS natif via l1phy. gapk = monitoring reseau uniquement." ||
↪  echo "  → Audio MS via gapk (mode classique).")

  info                    Affiche l'architecture audio et les IPs Linphone

  auto     [FORMAT] [ALSA_HP] [ALSA_MIC]
               Surveille MGW, bridge RTP↔ALSA auto (chemin reseau).

  call     <RX_PORT> <TX_DEST:PORT> [FORMAT] [ALSA_HP] [ALSA_MIC]
               Bridge bidirectionnel RTP reseau ↔ ALSA.

  monitor  <RX_PORT> [FORMAT] [ALSA_HP]
               Ecoute passive RTP → ALSA.

  record   <RX_PORT> [FICHIER] [FORMAT]
               Enregistre flux RTP → fichier .gsm.

  playback <FICHIER> <TX_DEST:PORT> [FORMAT]
               Injecte fichier .gsm → RTP.

  loopback [FORMAT] [ALSA_HP] [ALSA_MIC]
               Loopback ALSA mic→HP (ECOUTEURS OBLIGATOIRES).
               Ne traverse PAS le vocodeur - voir mode_loopback.

  mgw-ports     Endpoints RTP OsmoMGW actifs.
  stop          Arrete les instances gapk.
  status        Etat audio complet (l1phy + gapk).
  list-codecs   Codecs disponibles.
```

```
  list-devices  Peripheriques ALSA + config l1phy.

Formats : gsm (defaut, = GSM FR / RFC3551) | rawpcm-s16le | rtp-efr
          | rtp-hr-etsi | rtp-hr-ietf | amr-* | ti-* | racal-*
          "gsmfr", "gsmefr", "gsmhr", "pcm8", "pcm16" N'EXISTENT PAS :
          ces noms ont fait mourir gapk au demarrage jusqu'au 10/08/2026.
          Liste faisant foi : gapk-start.sh list-codecs
Codecs  : seul "fr" est encodeur ET decodeur dans ce build.

Variables : GAPK_FORMAT, GAPK_ALSA_DEV (HP), GAPK_ALSA_DEV_IN (micro),
            GAPK_RTP_PT, ALSA_CARD, GAPK_POLL_INTERVAL
EOF
}

# ── Dispatch ──────────────────────────────────────────────────────────
MODE="${1:-help}"; shift 2>/dev/null || true
case "$MODE" in
    info)          mode_info ;;
    auto)          mode_auto "$@" ;;
    call)          mode_call "$@" ;;
    monitor)       mode_monitor "$@" ;;
    record)        mode_record "$@" ;;
    playback)      mode_playback "$@" ;;
    loopback)      mode_loopback "$@" ;;
    mgw-ports)     mode_mgw_ports "" ;;
    stop)          mode_stop ;;
    status)        mode_status ;;
    list-codecs)   mode_list_codecs ;;
    list-devices)  mode_list_devices ;;
    help|-h|--help) usage ;;
    *) log_error "Mode inconnu : $MODE"; echo ""; usage; exit 1 ;;
esac
```

2.195 /opt/GSM/osmo-operator/scripts/ism_payload.py

15731 octets, 401 lignes → 401 lignes

```
#!/usr/bin/env python3
"""
ism_payload.py — the SMS PAYLOAD over an ISM radio link, the rest over IP.

Idea (start-native.sh --payload-ism option, cf. wiki/Payload-ISM.md): between
two osmo-operator nodes, the SIGNALLING (addresses, MSISDN, IMSI, SC-address,
SS7, SIP) keeps going over IP as with --wan. ONLY the message content (the SMS
text, extracted from the TPDU) travels over a radio link in an ISM band (433 /
868 / 915 MHz, 2.4 GHz — licence-free). The two relays coordinate over TCP;
the TCP carries a correlation TOKEN instead of the text, and the text arrives
over the radio, stamped with the same token.

This module knows no hardware: it talks to the DATA INTERFACE of your radio (a
serial device, a pair of named pipes, or the stdin/stdout of a program you
provide — a GNU Radio SDR flowgraph, rtl_433, a LoRa/RFM69/CC1101 module over
UART...). The backend is chosen by environment variable or by the [payload]
section of sms-routing.conf.

Frame (robust on a noisy link, no dependency outside stdlib): a JSON line
terminated by \\n, payload in hex, CRC32:
    {"t":"<token>","p":"<hex>","c":"<crc32hex>"}\\n
A truncated line or a wrong CRC is ignored (the radio may lose bytes).

BACKENDS (ISM_LINK / [payload] link=):
  loopback  all in memory, for a trial on a single machine (no radio)
  fifo      two named pipes: ISM_TX (write) and ISM_RX (read).
            Plug any radio tool onto them (socat, an SDR flowgraph...)
```

```
  serial    a /dev/tty* in raw read/write (ISM module over UART).
            Baud rate set by the setup script (stty), ISM_SERIAL=/dev/ttyUSB0
  exec      runs a TX command (frame on its stdin) and an RX command (frames
            on its stdout): ISM_TX_CMD / ISM_RX_CMD. The most general.

Variables: ISM_LINK, ISM_TX, ISM_RX, ISM_SERIAL, ISM_TX_CMD, ISM_RX_CMD,
ISM_PAYLOAD_TTL (seconds, default 120), ISM_RECV_TIMEOUT (default 30).

Self-test (no radio):
    python3 ism_payload.py --selftest
    python3 ism_payload.py --selftest --link fifo --tx /tmp/ism_a --rx /tmp/ism_b
"""

import os
import sys
import time
import json
import zlib
import queue
import shlex
import select
import threading
import subprocess
from typing import Optional, Dict, Tuple, Callable

DEFAULT_TTL = float(os.environ.get("ISM_PAYLOAD_TTL", "120"))
DEFAULT_RECV_TIMEOUT = float(os.environ.get("ISM_RECV_TIMEOUT", "30"))


def _log(msg: str):
    sys.stderr.write("[ism] %s\n" % msg)
    sys.stderr.flush()


def frame_encode(token: str, payload: bytes) -> bytes:
    """(token, payload) -> a frame line terminated by \\n."""
    hexp = payload.hex()
    crc = "%08x" % (zlib.crc32(payload) & 0xFFFFFFFF)
    obj = {"t": token, "p": hexp, "c": crc}
    return (json.dumps(obj, separators=(",", ":")) + "\n").encode("ascii")


def frame_decode(line: bytes) -> Optional[Tuple[str, bytes]]:
    """A frame line -> (token, payload), or None if invalid / wrong CRC."""
    try:
        obj = json.loads(line.decode("ascii", "replace").strip())
        token = obj["t"]
        payload = bytes.fromhex(obj["p"])
        if ("%08x" % (zlib.crc32(payload) & 0xFFFFFFFF)) != obj.get("c"):
            _log("trame rejetee : CRC faux (jeton %s)" % token)
            return None
        return token, payload
    except Exception:
        return None


# ============================================================================
# Backends: each exposes write_frame(bytes) and a read_frames() generator
# ============================================================================

class _Backend:
    def write_frame(self, data: bytes) -> None:
        raise NotImplementedError

    def read_line(self, timeout: float) -> Optional[bytes]:
        """One line (without the \\n), or None if nothing before timeout."""
```

```
        raise NotImplementedError

    def close(self) -> None:
        pass


class LoopbackBackend(_Backend):
    """TX writes directly into the RX queue: a trial without radio."""
    def __init__(self):
        self._q: "queue.Queue[bytes]" = queue.Queue()

    def write_frame(self, data: bytes) -> None:
        for line in data.splitlines():
            if line:
                self._q.put(line)

    def read_line(self, timeout: float) -> Optional[bytes]:
        try:
            return self._q.get(timeout=timeout)
        except queue.Empty:
            return None


class FifoBackend(_Backend):
    """Two named pipes: tx (write, toward the radio) and rx (read)."""
    def __init__(self, tx_path: str, rx_path: str):
        self.tx_path = tx_path
        self.rx_path = rx_path
        for p in (tx_path, rx_path):
            if p and not os.path.exists(p):
                os.mkfifo(p, 0o600)
        # Non-blocking read: rx is opened O_RDONLY|O_NONBLOCK and a dummy
        # writer is kept so as not to see EOF every time the pipe drains.
        self._rx_fd = os.open(rx_path, os.O_RDONLY | os.O_NONBLOCK)
        try:
            self._rx_keepalive = os.open(rx_path, os.O_WRONLY | os.O_NONBLOCK)
        except OSError:
            self._rx_keepalive = -1
        self._buf = b""

    def write_frame(self, data: bytes) -> None:
        # O_WRONLY may block if nobody reads: best-effort write.
        fd = os.open(self.tx_path, os.O_WRONLY | os.O_NONBLOCK)
        try:
            os.write(fd, data)
        except BlockingIOError:
            _log("fifo tx pleine, trame perdue")
        finally:
            os.close(fd)

    def read_line(self, timeout: float) -> Optional[bytes]:
        if b"\n" in self._buf:
            line, self._buf = self._buf.split(b"\n", 1)
            return line
        r, _, _ = select.select([self._rx_fd], [], [], timeout)
        if not r:
            return None
        try:
            chunk = os.read(self._rx_fd, 4096)
        except BlockingIOError:
            return None
        if not chunk:
            return None
        self._buf += chunk
        if b"\n" in self._buf:
            line, self._buf = self._buf.split(b"\n", 1)
```

```
            return line
        return None

    def close(self) -> None:
        for fd in (getattr(self, "_rx_fd", -1), getattr(self, "_rx_keepalive", -1)):
            if fd and fd >= 0:
                try:
                    os.close(fd)
                except OSError:
                    pass


class SerialBackend(_Backend):
    """A raw /dev/tty*. The baud rate is set outside (stty); bytes are read/written."""
    def __init__(self, dev: str):
        self.dev = dev
        self._fd = os.open(dev, os.O_RDWR | os.O_NONBLOCK | os.O_NOCTTY)
        self._buf = b""

    def write_frame(self, data: bytes) -> None:
        try:
            os.write(self._fd, data)
        except BlockingIOError:
            time.sleep(0.05)
            try:
                os.write(self._fd, data)
            except OSError:
                _log("serie tx occupee, trame perdue")

    def read_line(self, timeout: float) -> Optional[bytes]:
        if b"\n" in self._buf:
            line, self._buf = self._buf.split(b"\n", 1)
            return line
        r, _, _ = select.select([self._fd], [], [], timeout)
        if not r:
            return None
        try:
            chunk = os.read(self._fd, 4096)
        except BlockingIOError:
            return None
        if chunk:
            self._buf += chunk
        if b"\n" in self._buf:
            line, self._buf = self._buf.split(b"\n", 1)
            return line
        return None

    def close(self) -> None:
        try:
            os.close(self._fd)
        except OSError:
            pass


class ExecBackend(_Backend):
    """Runs a TX (frame on stdin) and an RX (frames on stdout): your SDR pipeline."""
    def __init__(self, tx_cmd: str, rx_cmd: str):
        self.tx_cmd = tx_cmd
        self.rx_cmd = rx_cmd
        self._rx = subprocess.Popen(shlex.split(rx_cmd), stdout=subprocess.PIPE,
                                    bufsize=0)

    def write_frame(self, data: bytes) -> None:
        # One TX per frame: simple and stateless; the command sends then returns.
        try:
            p = subprocess.Popen(shlex.split(self.tx_cmd), stdin=subprocess.PIPE)
```

```
            p.stdin.write(data)
            p.stdin.close()
            p.wait(timeout=30)
        except Exception as e:
            _log("exec tx echec : %s" % e)

    def read_line(self, timeout: float) -> Optional[bytes]:
        if self._rx.stdout is None:
            return None
        r, _, _ = select.select([self._rx.stdout], [], [], timeout)
        if not r:
            return None
        line = self._rx.stdout.readline()
        return line.rstrip(b"\n") if line else None

    def close(self) -> None:
        try:
            self._rx.terminate()
        except Exception:
            pass


def make_backend(cfg: Dict[str, str]) -> _Backend:
    link = (cfg.get("link") or os.environ.get("ISM_LINK") or "loopback").lower()
    if link == "loopback":
        return LoopbackBackend()
    if link == "fifo":
        tx = cfg.get("tx") or os.environ.get("ISM_TX") or "/tmp/osmo_ism_tx"
        rx = cfg.get("rx") or os.environ.get("ISM_RX") or "/tmp/osmo_ism_rx"
        return FifoBackend(tx, rx)
    if link == "serial":
        dev = cfg.get("serial") or os.environ.get("ISM_SERIAL") or "/dev/ttyUSB0"
        return SerialBackend(dev)
    if link == "exec":
        tx = cfg.get("tx_cmd") or os.environ.get("ISM_TX_CMD")
        rx = cfg.get("rx_cmd") or os.environ.get("ISM_RX_CMD")
        if not tx or not rx:
            raise ValueError("backend exec : ISM_TX_CMD et ISM_RX_CMD requis")
        return ExecBackend(tx, rx)
    raise ValueError("backend ISM inconnu : %r (loopback, fifo, serial, exec)" % link)


# ============================================================================
# PayloadLink: send(token, payload) / recv(token, timeout), RX in the background
# ============================================================================

class PayloadLink:
    """The ISM payload link, as seen by the SMS relay."""
    def __init__(self, cfg: Optional[Dict[str, str]] = None,
                 ttl: float = DEFAULT_TTL,
                 on_payload: Optional[Callable[[str, bytes], None]] = None):
        self.cfg = cfg or {}
        self.ttl = ttl
        self.backend = make_backend(self.cfg)
        self._store: Dict[str, Tuple[bytes, float]] = {}
        self._lock = threading.Lock()
        self._cv = threading.Condition(self._lock)
        self._on_payload = on_payload
        self._running = True
        self._rx = threading.Thread(target=self._rx_loop, daemon=True)
        self._rx.start()
        _log("liaison payload ISM prete (backend %s)"
             % (self.cfg.get("link") or os.environ.get("ISM_LINK") or "loopback"))

    def _rx_loop(self):
        while self._running:
```

```
            line = self.backend.read_line(timeout=1.0)
            if not line:
                self._prune()
                continue
            dec = frame_decode(line)
            if not dec:
                continue
            token, payload = dec
            with self._cv:
                self._store[token] = (payload, time.time())
                self._cv.notify_all()
            if self._on_payload:
                try:
                    self._on_payload(token, payload)
                except Exception as e:
                    _log("on_payload : %s" % e)

    def _prune(self):
        now = time.time()
        with self._lock:
            old = [t for t, (_, ts) in self._store.items() if now - ts > self.ttl]
            for t in old:
                del self._store[t]

    def send(self, token: str, payload: bytes) -> None:
        """Sends the payload over the radio, stamped with the token."""
        self.backend.write_frame(frame_encode(token, payload))

    def recv(self, token: str, timeout: float = DEFAULT_RECV_TIMEOUT) -> Optional[bytes]:
        """Waits for this token's payload, received over the radio (None if not in time)."""
        deadline = time.time() + timeout
        with self._cv:
            while token not in self._store:
                remaining = deadline - time.time()
                if remaining <= 0:
                    return None
                self._cv.wait(timeout=remaining)
            payload, _ = self._store.pop(token)
            return payload

    def close(self):
        self._running = False
        self.backend.close()


def new_token() -> str:
    """A short unique correlation token (12 hex digits)."""
    return os.urandom(6).hex()


# ═══════════════════════════════════════════════════════════════════════
# Autotest
# ═══════════════════════════════════════════════════════════════════════

def _selftest(cfg: Dict[str, str]) -> int:
    link = cfg.get("link") or os.environ.get("ISM_LINK") or "loopback"
    print("autotest liaison payload ISM — backend %s" % link)
    if link == "fifo":
        # two crossed links: A writes on tx and reads on rx; a second mirror
        # link replays what A sends back to A itself, to loop without radio.
        a = PayloadLink(dict(cfg))
        mirror_cfg = dict(cfg)
        mirror_cfg["tx"], mirror_cfg["rx"] = cfg.get("rx"), cfg.get("tx")
        b = PayloadLink(mirror_cfg)
        links = [a, b]
        tx, rx = a, b
```

```
    else:
        link_obj = PayloadLink(dict(cfg))
        links = [link_obj]
        tx = rx = link_obj

    ok = True
    for text in ["Bonjour banc", "éàü accents 2G", "x" * 140]:
        token = new_token()
        payload = text.encode("utf-8")
        tx.send(token, payload)
        got = rx.recv(token, timeout=5)
        good = got == payload
        ok = ok and good
        print("  %s jeton=%s  %r" % ("OK " if good else "KO ", token,
                                      text[:30] + ("…" if len(text) > 30 else "")))
    # CRC: a corrupted frame must be rejected
    bad = frame_encode("deadbeef", b"hello")
    assert frame_decode(bad.replace(b'"c":"', b'"c":"00')) is None
    print("  OK  trame a CRC faux rejetee")
    for l in links:
        l.close()
    print("resultat :", "OK" if ok else "ECHEC")
    return 0 if ok else 1


def main(argv):
    import argparse
    ap = argparse.ArgumentParser(description="Liaison payload ISM")
    ap.add_argument("--selftest", action="store_true", help="autotest (sans radio par defaut)")
    ap.add_argument("--link", help="loopback|fifo|serial|exec")
    ap.add_argument("--tx"); ap.add_argument("--rx")
    ap.add_argument("--serial"); ap.add_argument("--tx-cmd"); ap.add_argument("--rx-cmd")
    a = ap.parse_args(argv)
    cfg = {k: v for k, v in {
        "link": a.link, "tx": a.tx, "rx": a.rx, "serial": a.serial,
        "tx_cmd": a.tx_cmd, "rx_cmd": a.rx_cmd}.items() if v}
    if a.selftest:
        return _selftest(cfg)
    ap.print_help()
    return 0


if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```

2.196 /opt/GSM/osmo-operator/scripts/magnetite-c139.sh

5631 octets, 101 lignes → 101 lignes

```
#!/bin/bash
# magnetite-c139.sh - FreeCalypso Magnetite (c139 target, hybrid-ui-vo config) in qosmo.
#
# The real TI/TCS211 firmware (rebuilt by fc-magnetite with the TMS470
# compiler under Wine) instead of the osmocom-bb layer1:
#   - 4 MiB flash image = Compal FreeCalypso bootloader in sector 0 +
#     fwimage.bin at 0x10000 (/opt/GSM/firmware/magnetite/c139-flash.img);
#   - qosmo boots at 0 in the flash (no -kernel), CALYPSO_LCD=c139 puts the
#     96x64 colour screen of the C139 on uWire CS0 (calypso_uwire.c);
#   - RVTMUX traces of the firmware on the modem UART (-serial 0 -> pty), read
#     with rvinterf (/opt/GSM/fc-host-tools); IrDA UART on the 2nd pty.
#
#   scripts/magnetite-c139.sh [start|stop|status|log|traces|fsio|shell|gtk]
#     QEMU=/opt/GSM/qosmo/build/qemu-system-arm  FLASH=.../c139-flash.img
#     FLASH_RESET=1: start again from the pristine image (FFS erased)
```

```
#     RVINTERF=0: do not start rvinterf (decoded RVTMUX traces in $RUNDIR/rvinterf.log,
#       socket $RUNDIR/rvinterf.sock for fc-shell / fc-fsio -s)
#     QEMU_DEBUG=1: -d ${QEMU_DEBUG_FLAGS:-guest_errors,unimp} in qemu-debug.log
#     QEMU_CPU=ti925t (default): ARMv4T like the Calypso ARM7TDMI. With arm946
#       (ARMv5) the TI Thumb trampolines (pop {pc} without bit 0) run in ARM
#       -> prefetch abort at the first one.
#     CALYPSO_LCD_DEBUG=1: LCD commands on stderr
set -u
RUNDIR="${RUNDIR:-/dev/shm/magnetite}"
QEMU="${QEMU:-/opt/GSM/qosmo/build/qemu-system-arm}"
FLASH_SRC="${FLASH:-/opt/GSM/firmware/magnetite/c139-flash.img}"
HERE="$(cd "$(dirname "$0")/.." && pwd)"
cmd="${1:-start}"

vivant() { [ -f "$RUNDIR/qemu.pid" ] && kill -0 "$(cat "$RUNDIR/qemu.pid")" 2>/dev/null; }

case "$cmd" in
start)
    if vivant; then echo "deja lance (pid $(cat "$RUNDIR/qemu.pid"))"; exit 0; fi
    [ -x "$QEMU" ] || { echo "QEMU absent : $QEMU" >&2; exit 1; }
    [ -f "$FLASH_SRC" ] || { echo "image flash absente : $FLASH_SRC (voir doc/magnetite.md)" >&2; exit 1; }
    mkdir -p "$RUNDIR"
    # working copy: the firmware writes its FFS in the flash (0x3C0000, 3 x 64 KiB),
    # kept from one run to the next (IMEI, rfcap, config); FLASH_RESET=1 or a newer
    # source image (new build) replace it.
    if [ "${FLASH_RESET:-0}" != 0 ] || [ ! -f "$RUNDIR/flash.img" ] || [ "$FLASH_SRC" -nt "$RUNDIR/flash.img" ];
     ↪  then
        cp "$FLASH_SRC" "$RUNDIR/flash.img" && echo "flash de travail (re)creee depuis $FLASH_SRC"
    fi
    dbg=()
    [ "${QEMU_DEBUG:-0}" != 0 ] && dbg=(-d "${QEMU_DEBUG_FLAGS:-guest_errors,unimp}" -D "$RUNDIR/qemu-debug.log")
    # QEMU_GDB=1: gdb stub on tcp::${QEMU_GDB_PORT:-1240} (gdb-multiarch, symbols: fwimage.map addresses)
    [ "${QEMU_GDB:-0}" != 0 ] && dbg+=(-gdb "tcp::${QEMU_GDB_PORT:-1240}")
    [ "${QEMU_GDB_WAIT:-0}" != 0 ] && dbg+=(-S)     # CPU halted at start, gdb does « continue »
    [ -n "${QEMU_TRACE:-}" ] && dbg+=(-trace "$QEMU_TRACE")   # e.g. QEMU_TRACE='pflash*' (in qemu.log)
    CALYPSO_PACER_RATTRAPAGE="${CALYPSO_PACER_RATTRAPAGE:-0}" \
    CALYPSO_LCD=c139 CALYPSO_LCD_SHM="${CALYPSO_LCD_SHM:-/dev/shm/calypso_lcd}" \
    CALYPSO_KEYS_SOCK="${CALYPSO_KEYS_SOCK:-/tmp/calypso_keys.sock}" \
    "$QEMU" -M calypso -cpu "${QEMU_CPU:-ti925t}" -display none -parallel none \
        -serial pty -serial pty -monitor "unix:$RUNDIR/monitor.sock,server,nowait" \
        -drive if=pflash,file="$RUNDIR/flash.img",format=raw "${dbg[@]}" \
        > "$RUNDIR/qemu.log" 2>&1 &
    echo $! > "$RUNDIR/qemu.pid"
    for i in 1 2 3 4 5 6 7 8 9 10; do
        grep -q 'char device redirected' "$RUNDIR/qemu.log" 2>/dev/null && break; sleep 0.3
    done
    grep -o '/dev/pts/[0-9]*' "$RUNDIR/qemu.log" | head -1 > "$RUNDIR/modem.pty"
    grep -o '/dev/pts/[0-9]*' "$RUNDIR/qemu.log" | sed -n 2p > "$RUNDIR/irda.pty"
    RVINTERF_BIN="${RVINTERF_BIN:-/opt/GSM/fc-host-tools/rvinterf/lowlevel/rvinterf}"
    if [ "${RVINTERF:-1}" != 0 ] && [ -x "$RVINTERF_BIN" ] && [ -s "$RUNDIR/modem.pty" ]; then
        rm -f "$RUNDIR/rvinterf.sock"
        "$RVINTERF_BIN" -n -l "$RUNDIR/rvinterf.log" -s "$RUNDIR/rvinterf.sock" "$(cat "$RUNDIR/modem.pty")" \
            > "$RUNDIR/rvinterf.out" 2>&1 &
        echo $! > "$RUNDIR/rvinterf.pid"
    fi
    echo "qemu pid $(cat "$RUNDIR/qemu.pid")  UART modem (RVTMUX) $(cat "$RUNDIR/modem.pty")  IrDA $(cat
     ↪  "$RUNDIR/irda.pty")"
    echo "journal : $RUNDIR/qemu.log ; ecran : /usr/bin/python3 $HERE/tools/osmo-c123-gtk.py --modele c139"
    ;;
stop)
    vivant && kill "$(cat "$RUNDIR/qemu.pid")" && echo "arrete"
    [ -f "$RUNDIR/rvinterf.pid" ] && kill "$(cat "$RUNDIR/rvinterf.pid")" 2>/dev/null
    rm -f "$RUNDIR/qemu.pid" "$RUNDIR/rvinterf.pid"
    ;;
traces)
    tail -n "${2:-40}" "$RUNDIR/rvinterf.log"
```

```
    ;;
fsio|shell)
    # FreeCalypso fc-fsio / fc-shell on the rvinterf socket
    bin="/opt/GSM/fc-host-tools/rvinterf/$([ "$cmd" = fsio ] && echo etmsync/fc-fsio || echo asyncshell/fc-shell)"
    exec "$bin" -s "$RUNDIR/rvinterf.sock" "${@:2}"
    ;;
status)
    if vivant; then echo "en marche (pid $(cat "$RUNDIR/qemu.pid")), modem $(cat "$RUNDIR/modem.pty"
     ↪  2>/dev/null)"; else echo "arrete"; fi
    ;;
log)
    tail -n "${2:-40}" "$RUNDIR/qemu.log"
    ;;
gtk)
    # explicit --lcd/--keys: without them the window takes the first live non-TRX instance
    # (rssi, instance 4) and shows its screen instead of Magnetite.
    exec /usr/bin/python3 "$HERE/tools/osmo-c123-gtk.py" --modele c139 \
        --lcd "${CALYPSO_LCD_SHM:-/dev/shm/calypso_lcd}" --keys "${CALYPSO_KEYS_SOCK:-/tmp/calypso_keys.sock}"
         ↪  "${@:2}"
    ;;
*)
    echo "usage : $0 start|stop|status|log|traces|fsio [cmd]|shell [cmd]|gtk" >&2; exit 2
    ;;
esac
```

2.197 /opt/GSM/osmo-operator/scripts/osmo-banc-aliases.sh

4623 octets, 60 lignes → 59 lignes (1 groupes compactés)

```
# osmo-banc-aliases.sh - the Calypso bench aliases (sourced: /etc/profile.d/98-osmo-banc-aliases.sh)
# Two chains, four processes each, through c54x_exe/run.sh (MODE=dsp):
#   banc-trx     c54x_exe + qosmo (trx.highram.elf) + osmocon + transceiver: the Calypso is the
#                TRX of BTS#0 (TRXD 5700-5702); no bridge, it would hold the same ports
#   banc-mobile  INSTANCE 2: c54x_exe + qosmo (layer1) + osmocon + mobile, and the AIR towards
#                instance 1 (banc-mobile-stop/-status/-logs for this instance; c123-1 for its
#                screen)
#   banc-mobile-seul  single-Calypso setup: mobile + bridge towards osmo-bts-trx, instance 1
#   banc-rssi    c54x_exe + qosmo (rssi.highram.elf) + osmocon: the RSSI application alone,
#                emulated screen and keypad; the window: c123
#   banc-stop / banc-status / banc-logs   the same run.sh, --stop / --status / --logs
# The network (core, BSC, BTS, side-car) stays with ./start-native.sh; these aliases only bring
# up the Calypso chain, the network being already up.
# Everything at once, network included: ./start-native.sh --bts (the radio alone, = --osmo-trx)
#   or ./start-native.sh --bts --ms (the radio then the mobile over the air, = banc-trx then
#   banc-mobile).
alias banc-trx='MODE=dsp OSMO_TRX=1 bash /opt/GSM/c54x_exe/run.sh'
# banc-mobile = INSTANCE 2 (run.sh INSTANCE=2): second c54x_exe, QEMU, osmocon, mobile, and the
# air (pont/air.py) linking its bursts to those of instance 1 (banc-trx). Logs: /tmp/c54x-pont2.
alias banc-mobile='INSTANCE=2 MODE=dsp AIR=1 bash /opt/GSM/c54x_exe/run.sh'
alias banc-mobile-stop='INSTANCE=2 MODE=dsp bash /opt/GSM/c54x_exe/run.sh --stop'
alias banc-mobile-status='INSTANCE=2 MODE=dsp bash /opt/GSM/c54x_exe/run.sh --status'
alias banc-mobile-logs='INSTANCE=2 MODE=dsp bash /opt/GSM/c54x_exe/run.sh --logs'
# single-Calypso setup (mobile + bridge towards osmo-bts-trx), instance 1
alias banc-mobile-seul='MODE=dsp PONT=1 bash /opt/GSM/c54x_exe/run.sh'
# rssi on INSTANCE 4, separate from the rest of the bench: no environment copied from instance 1
# (the TRX), and not stopped by banc-stop / start-native --stop (banc_dsp_arreter_instances
# spares it). banc-rssi-stop stops it alone; c123-3 for its screen.
unalias banc-rssi 2>/dev/null   # an alias of the same name would expand in the function definition
# With a classic BTS (--bts-classic: pont_dsp.py alive), no air: it would relay the TX ring of
# instance 1, which is then the MOBILE (nothing idle, hence « Sync error ») and send the rssi
# bursts back to its BSP. The downlink comes through the bridge copy (PONT_DSP_COPIES, udp 6762).
# Otherwise (--bts), the air.
# Realistic air (calypso_bsp.c, CALYPSO_AIR_NIVEAUX): the cell is only heard on its ARFCN (radio
# plan), the downlink at CALYPSO_AIR_DL_DBM, the mobile uplink at CALYPSO_AIR_UL_DBM, noise
```

```
# elsewhere. CALYPSO_AIR_NIVEAUX=0 banc-rssi: everything at maximum, on every frequency.
function banc-rssi {
    local air=1 arfcn
    pgrep -f 'pont/pont_dsp\.py' >/dev/null && air=0
    arfcn="$( . /etc/osmocom/radio-plan.env 2>/dev/null; echo "${PLAN_ARFCN:-}")"
    INSTANCE=4 MODE=dsp FIRMWARE_APP=rssi AIR="${AIR:-$air}" \
        CALYPSO_AIR_NIVEAUX="${CALYPSO_AIR_NIVEAUX:-1}" CALYPSO_FAUX_AIR="${CALYPSO_FAUX_AIR:-$arfcn}" \
        bash /opt/GSM/c54x_exe/run.sh "$@"
}
# banc-rssi --stop (or banc-rssi-stop) stops the whole instance 4, leftovers included (run.sh
# arreter). A function, not an alias: an alias does not exist in a non-interactive shell
# (bash -lc).
unalias banc-rssi-stop 2>/dev/null
function banc-rssi-stop { banc-rssi --stop; }
alias banc-stop='MODE=dsp bash /opt/GSM/c54x_exe/run.sh --stop'
alias banc-status='MODE=dsp OSMO_TRX=1 bash /opt/GSM/c54x_exe/run.sh --status'
alias banc-logs='MODE=dsp bash /opt/GSM/c54x_exe/run.sh --logs'
# Bench numbering: c123 = TRX (instance 1, TS0), c123-1 = layer1 (the mobile, instance 2),
# c123-2 = trx2 (phone 2 of the TRX, TS1, instance 3), c123-3 = rssi (instance 4, banc-rssi).
alias c123='/usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py --instance 1'
alias c123-⟨1⟩='/usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py --instance ⟨2⟩'  ×3
    ⟨⟩ = (1,2) (2,3) (3,4)
alias c123-mobile='/usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py'   # the first live instance
↪  that is not a TRX
alias c123-tous='/usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py --toutes 1,2,3,4'   # + those
↪  appearing later
```

2.198 /opt/GSM/osmo-operator/scripts/osmo-disclaimer.sh

1029 octets, 15 lignes → 15 lignes

```
# osmo-disclaimer.sh — reminder shown when a shell opens in the container.
#
# Sourced, not executed: /root/.bashrc does `. /etc/osmo-disclaimer.sh` (symlink
# installed by Dockerfile.run to this file). The ISO counterpart is
# /etc/profile.d/01-osmo-disclaimer.sh, written by build-iso.sh.
#
# A repository file copied by `COPY scripts/* /etc/osmocom/` rather than a
# heredoc in Dockerfile.run: the classic docker builder (Step N/M, without
# BuildKit) does not support heredocs in RUN.
export OSMO_DISCLAIMER_SHOWN=1
printf "\n  \033[1;33mDisclaimer\033[0m - banc d'essai GSM/SS7 Osmocom (conteneur %s).\n" "$(hostname)"
printf "  A n'utiliser que sur un reseau radio \033[1mISOLE\033[0m (cage/attenuateur) ou\n"
printf "  sur une bande sous licence : emettre sur le spectre public est illegal.\n"
printf "  \033[1;33mDemarrer la pile radio :\033[0m cd /opt/GSM/osmo-operator && ./start-native.sh --force\n"
printf "  \033[0;36mSuivre :\033[0m tmux attach -t calypso   \033[0;36m(Ctrl-b puis d pour se detacher)\033[0m\n\n"
```

2.199 /opt/GSM/osmo-operator/scripts/osmo-gnuarm-profile.sh

827 octets, 16 lignes → 16 lignes

```
# osmo-gnuarm-profile.sh - the osmocom-bb gnuarm toolchain in the PATH (sourced:
# /etc/profile.d/97-osmo-gnuarm.sh)
# arm-elf-gcc 4.5.2 (binutils, newlib) lives in /root/gnuarm/install/bin, installed by the
# --gnuarm supplement (tools/osmo-firmwares-install.sh, copy of the osmocom-bb-trx image).
# The supplement builds pass this PATH to make; here it applies to every login shell:
# « cd osmocom-bb/src && make firmware » finds arm-elf-gcc without exporting anything.
# GNUARM_DIR relocates the toolchain. Without the directory (supplement not installed),
# nothing changes.
_osmo_gnuarm_bin="${GNUARM_DIR:-/root/gnuarm}/install/bin"
if [ -d "$_osmo_gnuarm_bin" ]; then
    case ":$PATH:" in
```

```
        *":$_osmo_gnuarm_bin:"*) ;;
        *) PATH="$PATH:$_osmo_gnuarm_bin"; export PATH ;;
    esac
fi
unset _osmo_gnuarm_bin
```

2.200 /opt/GSM/osmo-operator/scripts/osmo-start.sh

7209 octets, 156 lignes → 156 lignes

```
#!/bin/bash
# osmo-start.sh - sequenced start of the Osmocom core
#
# Dependency order:
#   STP (4239) → HLR (4258) → MGW (4243)
#                    ↓
#               MSC (4254) → BSC (4242)
#                    ↓
#          GGSN (4260) → SGSN (4245) → PCU (4239 bts)
#
# Each service waits for the VTY of the previous one before starting.
# BTS-TRX and SIP-connector are handled by run.sh (they depend on fake_trx / Asterisk).
#
# Called by: run.sh (step 3)

set -e

GREEN='\033[0;32m'; RED='\033[0;31m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; NC='\033[0m'

OPERATOR_ID="${OPERATOR_ID:-1}"

# ── Helper: wait for a TCP port to be open ──────────────────────────
wait_port() {
    local host="$1" port="$2" label="$3" timeout="${4:-30}"
    local elapsed=0
    echo -ne "  Attente ${label} (${host}:${port})"
    while ! bash -c "echo >/dev/tcp/${host}/${port}" 2>/dev/null; do
        sleep 1; elapsed=$((elapsed + 1))
        echo -n "."
        if [ "$elapsed" -ge "$timeout" ]; then
            echo -e " ${RED}TIMEOUT${NC}"
            return 1
        fi
    done
    echo -e " ${GREEN}OK${NC} (${elapsed}s)"
}

# ── Helper: start a service and check it ────────────────────────────
start_svc() {
    local svc="$1" vty_port="$2" label="$3" timeout="${4:-30}"

    echo -e "  ${CYAN}${label}${NC}"
    # ── reset-failed before start ──────────────────────────────────
    # systemd gives up on a service after StartLimitBurst restarts (10 on
    # osmo-bsc); a rejected config gets there in ten seconds. Afterwards fixing
    # the config is not enough: "systemctl start" answers "start request
    # repeated too quickly" and the service stays dead. The counter is therefore
    # cleared at each launch: this script is the start action, it must not
    # inherit earlier failures.
    systemctl reset-failed "$svc" 2>/dev/null || true
    systemctl start "$svc" || {
        echo -e "    ${RED}✗ systemctl start ${svc} echoue${NC}"
        journalctl -u "$svc" -n 20 --no-pager >&2
```

```
        return 1
    }

    if [ -n "$vty_port" ]; then
        wait_port 127.0.0.1 "$vty_port" "$label VTY" "$timeout" || {
            echo -e "    ${YELLOW}[WARN] VTY :${vty_port} non accessible apres ${timeout}s${NC}" >&2
            journalctl -u "$svc" -n 10 --no-pager >&2
            return 1
        }
    fi
}

# ═══════════════════════════════════════════════════════════════════

echo -e "${GREEN}=== Core Osmocom - Op${OPERATOR_ID} ===${NC}"
echo ""

# ── 1. TUN network (APN0 for GGSN) ──────────────────────────────────
echo -e "${CYAN}[1/4] Interface TUN${NC}"
if ip link show apn0 > /dev/null 2>&1; then
    ip link del dev apn0
fi
ip tuntap add dev apn0 mode tun
ip addr add 176.16.32.0/24 dev apn0
ip link set dev apn0 up
echo -e "  ${GREEN}✓${NC} apn0 up"

# ── 2. Signalling (STP → HLR → MGW) ─────────────────────────────────
#
# STP must be ready first: all SS7 goes through it.
# HLR must be ready before MSC: MSC connects to the HLR at startup.
# MGW must be ready before MSC: MSC opens an MGCP connection to the MGW.
# ═══════════════════════════════════════════════════════════════════
echo ""
echo -e "${CYAN}[2/4] Signalisation (STP → HLR → MGW)${NC}"

start_svc osmo-stp  4239 "OsmoSTP"  30 || true
start_svc osmo-hlr  4258 "OsmoHLR"  30 || {
    echo -e "  ${RED}[ERR] HLR indispensable pour MSC - abandon${NC}"
    exit 1
}
start_svc osmo-mgw  4243 "OsmoMGW"  20 || true

# ── 3. Core Network (MSC → BSC) ─────────────────────────────────────
#
# MSC must be ready before BSC: BSC connects to the MSC over the A interface.
# ═══════════════════════════════════════════════════════════════════
echo ""
echo -e "${CYAN}[3/4] Core Network (MSC → BSC)${NC}"

start_svc osmo-msc  4254 "OsmoMSC"  30 || true
start_svc osmo-bsc  4242 "OsmoBSC"  30 || true

# ── 4. Data (GGSN → SGSN → PCU) ─────────────────────────────────────
echo ""
echo -e "${CYAN}[4/4] Data (GGSN → SGSN → PCU)${NC}"

start_svc osmo-ggsn 4260 "OsmoGGSN" 20 || true
start_svc osmo-sgsn 4245 "OsmoSGSN" 20 || true
start_svc osmo-pcu  ""   "OsmoPCU"  0  || true
start_svc osmo-sip-connector  ""   "OsmoSIPConnector"  0  || true
sleep 0.5
chmod 777 /tmp/pcu_bts 2>/dev/null || true

# NOTE: osmo-bts-trx and osmo-sip-connector are intentionally absent here.
# run.sh starts them in the right order:
```

```
#   fake_trx → wait_udp 5700 → osmo-bts-trx  (avoids the TRX race condition)
#   Asterisk → osmo-sip-connector             (avoids MNCC connect before SIP UP)

# ── Summary ──────────────────────────────────────────────────────────────
echo ""
echo -e "${GREEN}=== Verification ===${NC}"
SERVICES="osmo-stp osmo-hlr osmo-mgw osmo-msc osmo-bsc osmo-ggsn osmo-sgsn osmo-pcu"
MORTS=""
for svc in $SERVICES; do
    if systemctl is-active --quiet "$svc"; then
        echo -e "  ${GREEN}✓${NC} ${svc}"
    else
        echo -e "  ${RED}✗${NC} ${svc}"
        MORTS="${MORTS} ${svc}"
    fi
done

echo ""
# Each start_svc carries a "|| true": intended, a missing GGSN must not take
# the rest down. The final line must still reflect dead services: a core
# without MSC does no attach, so it is said here rather than discovered later
# in "show bts".
if [ -n "$MORTS" ]; then
    echo -e "${RED}Coeur INCOMPLET -${NC}${MORTS}"
    for svc in $MORTS; do
        echo -e "  ${CYAN}journalctl -u ${svc} -n 30${NC}   ou   ${CYAN}tail -30 /var/log/osmocom/${svc}.log${NC}"
    done
    case "$MORTS" in
        *osmo-msc*|*osmo-bsc*)
            echo -e "  ${YELLOW}Sans MSC ni BSC, aucun mobile ne peut s'attacher.${NC}"
            echo -e "  ${YELLOW}Cause frequente : la config est refusee a la lecture. La ligne fautive${NC}"
            echo -e "  ${YELLOW}est nommee par :${NC} ${CYAN}osmo-msc -c /etc/osmocom/osmo-msc.cfg 2>&1 | grep -A2
             ↪  'Error occurred'${NC}" ;;
    esac
else
    echo -e "${GREEN}Core Osmocom pret. BTS et SIP connector geres par run.sh.${NC}"
fi
```

2.201 /opt/GSM/osmo-operator/scripts/osmo-wg-profile.sh

973 octets, 17 lignes → 17 lignes

```
# osmo-wg-profile.sh - « wg 10.99.0.N » to bring up the WAN tunnel (sourced:
# /etc/profile.d/97-osmo-wg.sh)
# The bench WireGuard plan (wiki/WAN.md): 10.99.0.1 is the hub, 10.99.0.N the nodes.
#   wg 1                                this node becomes the hub (10.99.0.1)
#   wg N HUB_IP[:PORT] HUB_PUB          this node becomes 10.99.0.N and joins the hub
#   wg peer PUBLIC_KEY N                (on the hub) accept a node
#   wg up / wg down / wg status / wg pub the tunnel already configured
# N or 10.99.0.N, either form.
# These forms go to tools/osmo-wg.sh (as root, through sudo); everything else (wg show,
# wg genkey, wg set...) goes to the real wg, unchanged. A function, not a link in the PATH:
# a « wg » in /usr/local/bin would shadow /usr/bin/wg for scripts.
wg() {
    case "${1:-}" in
        10.99.0.*|peer|up|down|status|pub|[1-9]|[1-9][0-9]|[12][0-9][0-9])
         ↪  /opt/GSM/osmo-operator/tools/osmo-wg.sh "$@" ;;
        *) command wg "$@" ;;
    esac
}
```

2.202 /opt/GSM/osmo-operator/scripts/pulse-gsm-setup.sh

4065 octets, 106 lignes → 106 lignes

```
#!/bin/bash
# pulse-gsm-setup.sh - Configure PulseAudio for osmo-gapk audio
#
# Audio chain:
#   osmo-gapk → ALSA "gsm_out" → asound.conf(type pulse, device gsm_audio)
#             → PulseAudio null-sink "gsm_audio"
#             → gsm_audio.monitor → parec → WebSocket → browser
#
# Called by run.sh before starting OsmocomBB/gapk.
# Requires: PULSE_SERVER set + socket mounted by start-docker.sh

set -u

TAG="[pulse-gsm]"
TIMEOUT="${PULSE_TIMEOUT:-30}"
SINK_NAME="gsm_audio"
# Silent sink used as the mobiles' microphone. Without it, asound.conf falls
# back to the default source (= gsm_audio.monitor) for capture, i.e. the output
# the mobiles just wrote: closed audio loop.
MIC_NAME="gsm_mic"
SINK_RATE=8000
SINK_CHANNELS=1
SINK_FORMAT="s16le"

log()  { echo "$TAG $*"; }
err()  { echo "$TAG ERROR: $*" >&2; }

# ── 1. Wait for PulseAudio ──────────────────────────────────────────
wait_pulse() {
    local i=0
    while [ $i -lt "$TIMEOUT" ]; do
        if pactl info >/dev/null 2>&1; then
            return 0
        fi
        sleep 1
        i=$((i + 1))
    done
    return 1
}

log "Attente connexion PulseAudio (${PULSE_SERVER:-non defini})..."

if ! wait_pulse; then
    err "PulseAudio injoignable apres ${TIMEOUT}s"
    err "Verifier que PULSE_SERVER est defini et le socket monte"
    exit 1
fi

log "PulseAudio connecte"

# ── 2. Create the gsm_audio null-sink ──────────────────────────────
if pactl list short sinks 2>/dev/null | grep -q "$SINK_NAME"; then
    log "Sink ${SINK_NAME} deja present"
else
    if pactl load-module module-null-sink \
        sink_name="$SINK_NAME" \
        format="$SINK_FORMAT" \
        rate="$SINK_RATE" \
        channels="$SINK_CHANNELS" \
        sink_properties=device.description=GSM_Audio >/dev/null 2>&1; then
        log "Sink ${SINK_NAME} cree (${SINK_FORMAT}/${SINK_RATE}Hz/${SINK_CHANNELS}ch)"
    else
```

```
        err "Echec creation sink ${SINK_NAME}"
        exit 1
    fi
fi

# ── 2bis. Silent sink "gsm_mic" (mobiles' microphone, see asound.conf) ──────
if pactl list short sinks 2>/dev/null | grep -q "$MIC_NAME"; then
    log "Sink ${MIC_NAME} deja present"
else
    pactl load-module module-null-sink sink_name="$MIC_NAME" \
        format="$SINK_FORMAT" rate="$SINK_RATE" channels="$SINK_CHANNELS" \
        sink_properties=device.description=GSM_Mic >/dev/null \
        && log "Sink ${MIC_NAME} cree (micro silencieux)" \
        || err "Echec creation sink ${MIC_NAME}"
fi

# ── 3. Check the monitor ────────────────────────────────────────────────────
if pactl list short sources 2>/dev/null | grep -q "${SINK_NAME}.monitor"; then
    log "Source ${SINK_NAME}.monitor disponible"
else
    err "Source ${SINK_NAME}.monitor introuvable"
    exit 1
fi

# ── 4. Prevent automatic suspension ─────────────────────────────────────────
# PulseAudio suspends idle sinks → osmo-gapk writes into the void and the host
# speakers must be toggled to wake the stream up

pactl suspend-sink "$SINK_NAME" 0 2>/dev/null || true
pactl suspend-source "${SINK_NAME}.monitor" 0 2>/dev/null || true

# Unload module-suspend-on-idle if it is loaded
SUSPEND_IDX=$(pactl list short modules 2>/dev/null | awk '/module-suspend-on-idle/ {print $1; exit}')
if [ -n "$SUSPEND_IDX" ]; then
    pactl unload-module "$SUSPEND_IDX" 2>/dev/null && \
        log "module-suspend-on-idle desactive" || true
fi

log "Sink ${SINK_NAME} force actif (pas de suspend)"

log "Configuration audio prete"
# NOTE: no gsm_audio → speakers loopback.
# Listening through parec → WebSocket → browser only.
# Sink kept active by suspend-sink 0 + removal of module-suspend-on-idle (step 4).
```

2.203 /opt/GSM/osmo-operator/scripts/run.sh

29043 octets, 541 lignes → 541 lignes

```
#!/bin/bash
# run.sh - Osmocom intra-container orchestrator
#
# PHY_MODE=faketrx (default): fake_trx → trxcon → mobile
# PHY_MODE=virtphy          : osmo-bts-virtual → virtphy → mobile
#
set -euo pipefail

LOG_FILE="/var/log/osmocom/run.sh.log"
mkdir -p "$(dirname "$LOG_FILE")"
exec > >(tee -a "$LOG_FILE") 2>&1

[[ -n "${DEBUG:-}" ]] && set -x

GREEN='\033[0;32m'; CYAN='\033[0;36m'; YELLOW='\033[1;33m'; RED='\033[0;31m'; NC='\033[0m'
```

```
# This file is DEPLOYED OUTSIDE THE REPO (/etc/osmocom/run.sh, /root/run.sh) and
# launched by scripts that do not necessarily export the manifest: under
# `set -u`, a bare ${GSM_ROOT} would kill the script here. Hence the explicit
# fallback values.
FAKETRX_PY="${FAKETRX_PY:-${FAKE_TRX:-${OSMOCOM_BB:-${GSM_ROOT:-/opt/GSM}/osmocom-bb}
 ↪  /src/target/trx_toolkit/fake_trx.py}}"
OPERATOR_ID="${OPERATOR_ID:-1}"; N_MS="${N_MS:-1}"; MOBILE_MODE="${MOBILE_MODE:-combined}"
PHY_MODE="${PHY_MODE:-faketrx}"   # faketrx | virtphy
# RUN_NO_PROCESS=1: only prepares/generates the configs (MS, TRX, handover) and
# exits WITHOUT launching any process (neither osmo-start, nor fake_trx/trxcon,
# nor mobile/asterisk/smsc). Used by the QEMU mode of start-docker.sh, which
# just wants the configs in place before launching ${OQC_ROOT}/run.sh.
RUN_NO_PROCESS="${RUN_NO_PROCESS:-0}"
MAX_MS=64; MAX_MS_PER_MOBILE=8; MS_PER_TRX=16
BB_PORT_BASE=6700; BB_PORT_STEP=3; BTS_PORT_BASE=5700
L2_SOCK_BASE="/tmp/osmocom_l2"; SAP_SOCK_BASE="/tmp/osmocom_sap"

[[ ! "$N_MS" =~ ^[0-9]+$ ]] || [ "$N_MS" -lt 1 ] || [ "$N_MS" -gt $MAX_MS ] && N_MS=1
[[ ! "$OPERATOR_ID" =~ ^[0-9]+$ ]] || [ "$OPERATOR_ID" -lt 1 ] || [ "$OPERATOR_ID" -gt 24 ] && exit 1
[[ "$MOBILE_MODE" != "combined" && "$MOBILE_MODE" != "split" ]] && MOBILE_MODE="combined"
[[ "$PHY_MODE" != "faketrx" && "$PHY_MODE" != "virtphy" ]] && PHY_MODE="faketrx"

N_TRX=$(( (N_MS + MS_PER_TRX - 1) / MS_PER_TRX ))
[ "$MOBILE_MODE" = "combined" ] && N_GROUPS=$(( (N_MS + MAX_MS_PER_MOBILE - 1) / MAX_MS_PER_MOBILE )) ||
 ↪  N_GROUPS=$N_MS

echo -e "  ${CYAN}Op${OPERATOR_ID}${NC}  N_MS=${N_MS}  mode=${MOBILE_MODE}  phy=${PHY_MODE}  TRX=${N_TRX}
 ↪  groups=${N_GROUPS}"

# ── Helpers ──────────────────────────────────────────────────────────
bb_port()  { echo $(( BB_PORT_BASE + ($1 - 1) * BB_PORT_STEP )); }
l2_sock()  { echo "${L2_SOCK_BASE}_${1}"; }
sap_sock() { echo "${SAP_SOCK_BASE}_${1}"; }
ms_imsi()  { printf '%s%s%04d%06d' "$_MCC" "$_MNC" "$OPERATOR_ID" "$1"; }
ms_imei()  { printf '3589250059%02d%02d0' "$OPERATOR_ID" "$1"; }
ms_ki()    { printf '00 11 22 33 44 55 66 77 88 99 aa bb cc dd %02x %02x' "$1" "$OPERATOR_ID"; }

wait_port() {
    local host=$1 port=$2 label=$3 timeout=${4:-60} elapsed=0
    echo -ne "  Attente ${label} (${host}:${port})"
    while ! bash -c "echo >/dev/tcp/${host}/${port}" 2>/dev/null; do
        sleep 1; elapsed=$((elapsed+1)); echo -n "."
        [ "$elapsed" -ge "$timeout" ] && echo -e " ${RED}TIMEOUT${NC}" && return 1
    done
    echo -e " ${GREEN}OK${NC} (${elapsed}s)"
}

wait_udp() {
    local port=$1 label=$2 timeout=${3:-30} elapsed=0
    echo -ne "  Attente ${label} (UDP ${port})"
    while ! ss -unlp | grep -q ":${port} "; do
        sleep 1; elapsed=$((elapsed+1)); echo -n "."
        [ "$elapsed" -ge "$timeout" ] && echo -e " ${RED}TIMEOUT${NC}" && return 1
    done
    echo -e " ${GREEN}OK${NC} (${elapsed}s)"
}

# ── TMUX ─────────────────────────────────────────────────────────────
SESSION="osmocom"; TMUX_SOCKET="/tmp/osmocom_tmux"; TMUX_OK=false

init_tmux() {
    echo -e "  ${CYAN}Init tmux...${NC}"
    rm -f "$TMUX_SOCKET"

    tmux -S "$TMUX_SOCKET" start-server 2>/dev/null || true
```

```
    sleep 0.5
    tmux -S "$TMUX_SOCKET" new-session -d -s "$SESSION" -n main 2>/dev/null || true
    sleep 0.5

    local i=0
    while [ ! -S "$TMUX_SOCKET" ] && [ $i -lt 30 ]; do sleep 0.3; i=$((i+1)); done

    if [ -S "$TMUX_SOCKET" ] && tmux -S "$TMUX_SOCKET" has-session -t "$SESSION" 2>/dev/null; then
        TMUX_OK=true
        echo -e "  ${GREEN}✓ tmux OK${NC}"
    else
        TMUX_OK=false
        echo -e "  ${RED}✗ tmux echec - mode background${NC}"
    fi
}

run_in_tmux() {
    local win=$1; shift; local cmd="$*"
    if [ "$TMUX_OK" = true ]; then
        tmux -S "$TMUX_SOCKET" new-window -t "$SESSION" -n "$win" 2>/dev/null || true
        sleep 0.2
        tmux -S "$TMUX_SOCKET" send-keys -t "${SESSION}:${win}" "$cmd" C-m 2>/dev/null || true
    else
        echo -e "  ${YELLOW}[bg]${NC} ${win}"
        bash -c "$cmd" >"/var/log/osmocom/${win}.log" 2>&1 &
    fi
}

# ── Config MS ──────────────────────────────────────────────────────
generate_ms_configs() {
    local base_cfg="/root/.osmocom/bb/mobile.cfg"
    [ -f "$base_cfg" ] || { echo -e "${RED}mobile.cfg introuvable${NC}"; return 1; }

    local ref_imsi ref_imei ref_ki ref_l2 ref_sap
    ref_imsi=$(grep -oP '^\s+imsi\s+\K[0-9]+' "$base_cfg" 2>/dev/null | head -1 || true)
    ref_imei=$(grep -oP '^\s+imei \K[0-9]+' "$base_cfg" 2>/dev/null | head -1 || true)
    ref_ki=$(grep -oP '^\s+ki comp128 \K[0-9a-f ]+' "$base_cfg" 2>/dev/null | head -1 | sed 's/[[:space:]]*$//' ||
     ↪  true)
    ref_l2=$(grep -oP 'layer2-socket \K\S+' "$base_cfg" 2>/dev/null | head -1 || echo "/tmp/osmocom_l2")
    ref_sap=$(grep -oP 'sap-socket \K\S+' "$base_cfg" 2>/dev/null | head -1 || true)

    if [ -z "$ref_imsi" ]; then
        ref_imsi=$(grep -E 'imsi [0-9]+' "$base_cfg" | head -1 | awk '{print $NF}')
        ref_imei=$(grep -E 'imei [0-9]+' "$base_cfg" | head -1 | awk '{print $2}')
        ref_ki=$(grep -E 'ki comp128' "$base_cfg" | head -1 | sed 's/.*ki comp128 //' | sed 's/[[:space:]]*$//')
    fi
    [ -z "$ref_imsi" ] || [ -z "$ref_imei" ] || [ -z "$ref_ki" ] && return 1

    _MCC=$(grep -E '^\s+rplmn ' "$base_cfg" | awk '{print $2}' | head -1); _MCC=${_MCC:-001}
    _MNC=$(grep -E '^\s+rplmn ' "$base_cfg" | awk '{print $3}' | head -1); _MNC=${_MNC:-01}
    echo -e "  ${CYAN}[ref]${NC} IMSI=${ref_imsi} MCC/MNC=${_MCC}/${_MNC}"

    local global_line; global_line=$(grep -n '^ms [0-9]' "$base_cfg" | head -1 | cut -d: -f1)
    [ -z "$global_line" ] && return 1
    local global_header; global_header=$(head -n $((global_line - 1)) "$base_cfg")
    local ms_template; ms_template=$(awk '/^ms [0-9]/{if(f)exit;f=1;p=1} p{print} /^(exit|end)$/&&p&&f{p=0;exit}'
     ↪  "$base_cfg")
    [ -z "$ms_template" ] && return 1

    _write_ms_block() {
        local outfile=$1 lnum=$2 gidx=$3
        local ni=$(ms_imsi "$gidx") ne=$(ms_imei "$gidx") nk=$(ms_ki "$gidx") nl=$(l2_sock "$gidx") ns=$(sap_sock
         ↪  "$gidx")
        local block; block=$(echo "$ms_template" | sed \
            -e "s|^ms [0-9]\+|ms ${lnum}|" \
            -e "s|layer2-socket ${ref_l2}|layer2-socket ${nl}|" \
```

```
            -e "s|imsi ${ref_imsi}|imsi ${ni}|" \
            -e "s|imei ${ref_imei}[[:space:]]*[0-9]*|imei ${ne}|" \
            -e "s|ki comp128 ${ref_ki}|ki comp128 ${nk}|")
        [ -n "$ref_sap" ] && block=$(echo "$block" | sed "s|sap-socket ${ref_sap}|sap-socket ${ns}|") \
            || block=$(echo "$block" | sed "/layer2-socket/a\\ sap-socket ${ns}")
        echo "$block" >> "$outfile"; echo "" >> "$outfile"
    }

    # If the file is bind-mounted (docker -v file:file), rm returns EBUSY.
    # In that case it cannot be deleted but it can be truncated and rewritten,
    # which is equivalent for our purpose.
    for f in /root/.osmocom/bb/mobile_ms*.cfg /root/.osmocom/bb/mobile_group*.cfg; do
        [ -e "$f" ] || continue
        if ! rm -f "$f" 2>/dev/null; then
            : > "$f" 2>/dev/null || \
                echo -e "  ${YELLOW}[warn] $f ni rm ni truncate (bind-mount RO ?)${NC}"
        fi
    done

    if [ "$MOBILE_MODE" = "split" ]; then
        for ms in $(seq 1 "$N_MS"); do
            echo "$global_header" > "/root/.osmocom/bb/mobile_ms${ms}.cfg"
            _write_ms_block "/root/.osmocom/bb/mobile_ms${ms}.cfg" 1 "$ms"
        done
    else
        local grp=1 gs=1
        while [ "$gs" -le "$N_MS" ]; do
            local ge=$((gs + MAX_MS_PER_MOBILE - 1)); [ "$ge" -gt "$N_MS" ] && ge=$N_MS
            local cfg="/root/.osmocom/bb/mobile_group${grp}.cfg"
            echo "$global_header" | sed "s|127\.0\.0\.1|127.0.0.${grp}|g" > "$cfg"
            local li=1; for ms in $(seq "$gs" "$ge"); do _write_ms_block "$cfg" "$li" "$ms"; li=$((li+1)); done
            echo -e "  ${CYAN}[Grp${grp}]${NC} IP=127.0.0.${grp} MS ${gs}-${ge}"
            grp=$((grp+1)); gs=$((ge+1))
        done
    fi
    echo -e "  ${GREEN}✓ Configs MS OK${NC}"
}

# ── TRX injection (fake_trx mode) ──────────────────────────────────────
inject_extra_trx() {
    [ "$1" -le 1 ] && return 0
    local bts="/etc/osmocom/osmo-bts-trx.cfg" bsc="/etc/osmocom/osmo-bsc.cfg"
    [ -f "$bts" ] || return 0
    local arfcn0; arfcn0=$(grep -E '^\s+arfcn [0-9]+' "$bsc" | head -1 | awk '{print $2}')
    local tmp; tmp=$(mktemp)
    awk -v n="$1" '/^ osmotrx ip local/&&!d{for(i=1;i<n;i++)printf" instance %d\n  osmotrx rx-gain 40\n  osmotrx
     ↪  tx-attenuation 50\n",i;d=1}{print}' "$bts" > "$tmp"; cp "$tmp" "$bts"; rm -f "$tmp"
    for t in $(seq 1 $(($1-1))); do
        local a=$((arfcn0 + t*2))
        printf ' trx %d\n  power-ramp max-initial 23000 mdBm\n  power-ramp step-size 2000 mdB\n  power-ramp
         ↪  step-interval 1\n  ms-power-control osmo\n  phy 0 instance %d\n' "$t" "$t" >> "$bts"
    done
    echo -e "  ${GREEN}✓ ${1} TRX injectes (osmo-bts-trx)${NC}"
}

# ── TRX injection (virtphy mode) ──────────────────────────────────────
inject_extra_trx_virtual() {
    [ "$1" -le 1 ] && return 0
    local bts="/etc/osmocom/osmo-bts-virtual.cfg"
    [ -f "$bts" ] || return 0

    # Add extra phy instances after "instance 0"
    local tmp; tmp=$(mktemp)
    local insert_done=0
    while IFS= read -r line; do
        echo "$line" >> "$tmp"
```

```
        if [ $insert_done -eq 0 ] && echo "$line" | grep -q '^ instance 0'; then
            for t in $(seq 1 $(($1-1))); do
                echo " instance ${t}" >> "$tmp"
            done
            insert_done=1
        fi
    done < "$bts"
    cp "$tmp" "$bts"; rm -f "$tmp"

    # Append the extra TRX at the end of the file
    for t in $(seq 1 $(($1-1))); do
        printf ' trx %d\n  power-ramp max-initial 23000 mdBm\n  power-ramp step-size 2000 mdB\n  power-ramp
         ↪  step-interval 1\n  ms-power-control osmo\n  phy 0 instance %d\n' "$t" "$t" >> "$bts"
    done
    echo -e "  ${GREEN}✓ ${1} TRX injectes (osmo-bts-virtual)${NC}"
}

# ── Handover ──────────────────────────────────────────────────────────
inject_handover() {
    local cfg="/etc/osmocom/osmo-bsc.cfg"; [ -f "$cfg" ] || return 0
    local tmp; tmp=$(mktemp)
    sed -e 's/^ handover 0/ handover 1/' -e 's/^ handover algorithm 1/ handover algorithm 2/' "$cfg" > "$tmp"
    awk '/^ handover algorithm 2/&&!d{print;print" handover2 window rxlev averaging 10\n handover2 window rxqual
     ↪  averaging 1\n handover2 window rxlev neighbor averaging 10\n handover2 power budget interval 6\n handover2
     ↪  power budget hysteresis 3\n handover2 maximum distance 9999\n handover2 assignment 1\n handover2
     ↪  tdma-measurement full";d=1;next}/^ handover1 /{next}{print}' "$tmp" > "${tmp}.2"
    cp "${tmp}.2" "$cfg"; rm -f "$tmp" "${tmp}.2"
    echo -e "  ${GREEN}✓ Handover algo 2${NC}"
}

# ── PulseAudio sinks (gsm_audio + gsm_mic) ────────────────────────────
# Docker: /scripts/  |  native: /etc/osmocom/  (same script, different path)
# Idempotent and NON FATAL: without PulseAudio the gapk audio is unavailable,
# but run.sh must never be killed (set -e) for that.
# $1 = PulseAudio wait timeout (default 30 s). The daemon is not started by
# this script but by lib/audio.sh (qemu pipeline): on the EARLY call [2d] it
# may not be there yet, hence a short timeout so as not to stall.
setup_pulse_sinks() {
    local PULSE_SETUP=/scripts/pulse-gsm-setup.sh
    [ -f "$PULSE_SETUP" ] || PULSE_SETUP=/etc/osmocom/pulse-gsm-setup.sh
    [ -f "$PULSE_SETUP" ] || { echo -e "  ${YELLOW}pulse-gsm-setup.sh introuvable - audio indispo${NC}"; return 0;
     ↪  }
    PULSE_TIMEOUT="${1:-30}" "$PULSE_SETUP" \
        || echo -e "  ${YELLOW}[warn] pulse-gsm-setup.sh echoue (audio indispo) - on continue${NC}"
}

# ── UNIVERSAL audio bridge ────────────────────────────────────────────
# gapk (network RTP → gsm_audio sink) + sound output. Called in faketrx/virtphy
# mode AND in no-process mode (qemu), which needs it for gsm_audio to be heard.
#
# Output: if the HOST pulse is reachable over TCP (relay opened by
# start-docker.sh, = WSLg→Windows on WSL, = session pulse on native Linux), a
# direct parec|paplay bridge is made (a single clock, no drift → no crackling).
# Otherwise, fallback local loopback to the container's default sink (card).
audio_bridge() {
    pactl info >/dev/null 2>&1 || { echo -e "  ${YELLOW}PulseAudio indisponible - bridge audio non lance${NC}";
     ↪  return 0; }

    # gapk : RTP reseau → sink gsm_audio
    local GAPK=/scripts/gapk-start.sh; [ -f "$GAPK" ] || GAPK=/etc/osmocom/gapk-start.sh
    if [ -f "$GAPK" ]; then
        run_in_tmux "gapk" "GAPK_ALSA_DEV=gsm_out bash '$GAPK' auto gsm gsm_out"
        echo -e "  ${GREEN}✓ gapk auto (RTP reseau → gsm_audio)${NC}"
    else
        echo -e "  ${YELLOW}gapk-start.sh absent - pas de bridge RTP${NC}"
    fi
```

```
    # Output to the host pulse if the TCP relay is open (HOST_AUDIO_RELAY is
    # set by start-docker.sh when the relay was enabled: tcp:<gw>:4713).
    local relay="${HOST_AUDIO_RELAY:-}"
    if [ -n "$relay" ] && pactl --server="$relay" info >/dev/null 2>&1; then
        # ── A SINGLE BRIDGE FOR THE WHOLE BENCH ──────────────────────────
        # The pkill below is not enough with SEVERAL containers: it runs
        # INSIDE the container, which has its own PID namespace, and never
        # sees the neighbour's bridge. Since all operators share the same
        # host PulseAudio, each would read the SAME gsm_audio sink and replay
        # it on the SAME output: the sound heard once per operator.
        #
        # The shared SERVER is therefore queried, not the local process
        # table: a client already connected under this name means a bridge
        # runs somewhere, whatever the container.
        local _bridge_name="osmo-gsm-bridge"
        if pactl --server="$relay" list clients 2>/dev/null \
             | grep -q "application.name = \"${_bridge_name}\""; then
            echo -e "  ${GREEN}✓ pont audio deja actif sur l hote (${relay}) - on n en ajoute pas un second${NC}"
            return 0
        fi
        # ── NO DUPLICATE WITH THE HOST LOOPBACK EITHER ───────────────────
        # The host already loads, on its side, a module-loopback
        # gsm_audio.monitor -> sound card (lib/audio.sh, ensure_local_loopback).
        # It is the SAME job as this bridge: same source, same output. Both
        # together, everything is heard TWICE - and the symptom survives a
        # container stop. The loopback is kept: it is host-side and much
        # shorter (20 ms against 250 ms for the TCP bridge).
        if pactl --server="$relay" list short modules 2>/dev/null \
             | grep -q 'source=gsm_audio.monitor'; then
            echo -e "  ${GREEN}✓ loopback gsm_audio deja en place sur l hote - pont TCP inutile${NC}"
            return 0
        fi
        pkill -f "paplay --server=${relay}" 2>/dev/null || true
        [ -f /run/host-audio.pid ] && kill -- "-$(cat /run/host-audio.pid)" 2>/dev/null || true
        setsid sh -c '
          while true; do
            parec -d gsm_audio.monitor --latency-msec=30 --format=s16le --rate=8000 --channels=1 \
              | paplay --server='"${relay}"' --client-name='"${_bridge_name}"' --latency-msec=250 --raw
↪   --format=s16le --rate=8000 --channels=1
            sleep 1
          done' >/var/log/osmocom/host-audio.log 2>&1 &
        echo $! > /run/host-audio.pid
        echo -e "  ${GREEN}✓ pont audio parec|paplay → hote (${relay})${NC}"
    else
        # Native fallback without relay: loopback to the default sink (card).
        if ! pactl list short modules 2>/dev/null | grep -q 'source=gsm_audio.monitor'; then
            pactl load-module module-loopback source=gsm_audio.monitor latency_msec=20 >/dev/null 2>&1 \
                && echo -e "  ${GREEN}✓ loopback gsm_audio → sink par defaut (carte)${NC}" \
                || echo -e "  ${YELLOW}loopback non charge (pas de sortie son ?)${NC}"
        else
            echo -e "  ${GREEN}✓ loopback gsm_audio deja actif${NC}"
        fi
    fi
}

# ═══════════════════════════════════════════════════════════════════════
# MAIN
# ═══════════════════════════════════════════════════════════════════════

# RUN_NO_PROCESS=1: tmux + configs + Osmocom core are prepared (osmo-start:
# STP/HLR/MGW/MSC/BSC... → the HLR is up and can be fed by start-docker.sh),
# but the radio/mobile/asterisk/smsc processes are NOT launched in the tmux.
[ "$RUN_NO_PROCESS" = "1" ] && echo -e "${YELLOW}=== RUN_NO_PROCESS=1 : core seul, PHY/mobile/asterisk/smsc NON
↪  lances ===${NC}"
```

```
echo -e "${GREEN}=== [1/10] tmux ===${NC}"
init_tmux
echo ""

echo -e "${GREEN}=== [2/10] Config MS ===${NC}"
generate_ms_configs
echo ""

echo -e "${GREEN}=== [2b] TRX ===${NC}"
if [ "$PHY_MODE" = "virtphy" ]; then
    inject_extra_trx_virtual "$N_TRX"
else
    inject_extra_trx "$N_TRX"
fi

echo -e "${GREEN}=== [2c] Handover ===${NC}"
inject_handover
echo ""

# ── [2d] PulseAudio audio - BEFORE the core, deliberately ──────────────────
# osmo-start.sh does `exit 1` if the HLR does not come up, and with
# `set -euo pipefail` (line 7) run.sh dies there: pulse-gsm-setup.sh - the only
# place that creates the gsm_mic null-sink - must run before, otherwise the
# gsm_in capture (= gsm_mic.monitor) never opens, gapk_io gives up BOTH
# directions, and every call is mute, echo-test 600 included.
# The audio has NO dependency on the core - so it is set up first.
# The call in [6b] stays in place (idempotent) for partial restarts.
echo -e "${GREEN}=== [2d] Audio PulseAudio (sinks) ===${NC}"
setup_pulse_sinks 3
echo ""

echo -e "${GREEN}=== [3/10] Core Osmocom ===${NC}"
/etc/osmocom/osmo-start.sh

# No-process mode: the core runs (HLR can be fed by start-docker.sh), we stop
# here without launching PHY/mobile/asterisk/smsc in the tmux. PulseAudio
# (gsm_audio sink) is configured anyway, because the pipeline launched next
# (e.g. ${OQC_ROOT}/run.sh) needs it for the gapk audio.
if [ "$RUN_NO_PROCESS" = "1" ]; then
    echo ""
    echo -e "${GREEN}=== Audio PulseAudio (no-process) ===${NC}"
    setup_pulse_sinks
    # Audio bridge ALSO in no-process: without it gapk is never launched and
    # the qemu pipeline has no audio.
    echo -e "${GREEN}=== Bridge audio (no-process / qemu) ===${NC}"
    audio_bridge
    echo ""
    echo -e "${GREEN}Core Osmocom pret (no-process). PHY/mobile/asterisk/smsc NON lances.${NC}"
    echo -e "  tmux : ${CYAN}tmux -S ${TMUX_SOCKET} attach -t ${SESSION}${NC}"
    exit 0
fi

# ═══════════════════════════════════════════════════════════════════════════
# PHY: fake_trx (TRXD) OR virtphy (multicast)
# ═══════════════════════════════════════════════════════════════════════════

if [ "$PHY_MODE" = "virtphy" ]; then
    # ──────────────────────────────────────────────────────────────────────
    # MODE VIRTPHY : osmo-bts-virtual → virtphy → mobile
    #
    #   mobile ↔ virtphy ↔ (multicast UDP) ↔ osmo-bts-virtual ↔ BSC
    #
    # No fake_trx, no trxcon.
    # ──────────────────────────────────────────────────────────────────────

    echo -e "${GREEN}=== [4/10] BTS Virtual ===${NC}"
```

```
    # Stop osmo-bts-trx if it was started by systemd
    systemctl stop osmo-bts-trx 2>/dev/null || true
    systemctl disable osmo-bts-trx 2>/dev/null || true

    run_in_tmux "bts" "osmo-bts-virtual -c /etc/osmocom/osmo-bts-virtual.cfg"
    wait_port 127.0.0.1 4241 "BTS-virtual" 30 || true

    echo -e "${GREEN}=== [5/10] virtphy (${N_MS} instances) ===${NC}"
    for ms in $(seq 1 "$N_MS"); do
        local_l2=$(l2_sock "$ms")
        cmd="virtphy -s ${local_l2}"
        pw=$(( (ms-1)/8 )); pl=$(( (ms-1)%8 ))
        win="virtphy${pw}"
        if [ "$pl" -eq 0 ]; then
            run_in_tmux "$win" "$cmd"
        elif [ "$TMUX_OK" = true ]; then
            tmux -S "$TMUX_SOCKET" split-window -v -t "${SESSION}:${win}" 2>/dev/null || true
            tmux -S "$TMUX_SOCKET" select-layout -t "${SESSION}:${win}" even-vertical 2>/dev/null || true
            tmux -S "$TMUX_SOCKET" send-keys -t "${SESSION}:${win}.${pl}" "$cmd" C-m 2>/dev/null || true
        else
            bash -c "$cmd" >"/var/log/osmocom/virtphy_ms${ms}.log" 2>&1 &
        fi
        [ $((ms % 8)) -eq 1 ] || [ "$ms" -eq "$N_MS" ] && echo -e "  ${CYAN}[ue${ms}]${NC} l2=${local_l2}"
    done
    # Let virtphy create the sockets before launching mobile
    sleep 2

    # No step 6 (trxcon) in virtphy mode

else
    # ──────────────────────────────────────────────────────────────────
    # MODE FAKETRX : fake_trx → trxcon → mobile
    #
    #   mobile ↔ trxcon ↔ (TRXD UDP) ↔ fake_trx ↔ osmo-bts-trx ↔ BSC
    # ──────────────────────────────────────────────────────────────────

    echo -e "${GREEN}=== [3b/10] osmo-bts-trx ===${NC}"
    # BTS launched BEFORE fake_trx: it polls the transceiver (POWERON) and
    # connects as soon as fake_trx answers. A possible systemd instance is
    # stopped to avoid a double BTS.
    systemctl stop osmo-bts-trx 2>/dev/null || true
    run_in_tmux "bts" "osmo-bts-trx -c /etc/osmocom/osmo-bts-trx.cfg"

    echo -e "${GREEN}=== [4/10] FakeTRX ===${NC}"
    # -s 20: real-time scheduling (SCHED_RR) for the transceiver and its TDMA
    # clock. Under normal scheduling, on a loaded host (srsRAN, QEMU, the
    # desktop), the 4.615 ms tick arrives 0.3 to 2 ms late dozens of times a
    # minute; the BTS compensates (« N FN slower/faster than TRX ») and the
    # speech loses frames.
    FAKETRX_CMD="python3 ${FAKETRX_PY} -s ${FAKETRX_RR_PRIO:-20} -b 127.0.0.1 -R 127.0.0.1 -r 127.0.0.1 -P
     ↪  ${BTS_PORT_BASE} -p ${BB_PORT_BASE}"
    for t in $(seq 1 $((N_TRX-1))); do
        FAKETRX_CMD+=" --trx bts${t}@127.0.0.1:${BTS_PORT_BASE}/${t}"
    done
    for ms in $(seq 2 "$N_MS"); do
        local_g=$(( (ms-1)/8 + 1 ))
        FAKETRX_CMD+=" --trx ue${ms}@127.0.0.${local_g}:$(bb_port $ms)"
    done
    run_in_tmux "faketrx" "$FAKETRX_CMD"
    wait_udp "$BTS_PORT_BASE" "fake_trx" 30 || true

    echo -e "${GREEN}=== [6/10] trxcon ===${NC}"
    for ms in $(seq 1 "$N_MS"); do
        local_l2=$(l2_sock "$ms"); local_port=$(bb_port "$ms")
        pw=$(( (ms-1)/8 )); pl=$(( (ms-1)%8 )); ug=$((pw+1)); gip="127.0.0.${ug}"
        cmd="trxcon ue${ms} -C 1 -i 127.0.0.1 -b ${gip} -p ${local_port} -s ${local_l2} -F 100"
```

```
        win="trxcon${pw}"
        if [ "$pl" -eq 0 ]; then
            run_in_tmux "$win" "$cmd"
        elif [ "$TMUX_OK" = true ]; then
            tmux -S "$TMUX_SOCKET" split-window -v -t "${SESSION}:${win}" 2>/dev/null || true
            tmux -S "$TMUX_SOCKET" select-layout -t "${SESSION}:${win}" even-vertical 2>/dev/null || true
            tmux -S "$TMUX_SOCKET" send-keys -t "${SESSION}:${win}.${pl}" "$cmd" C-m 2>/dev/null || true
        else
            bash -c "$cmd" >"/var/log/osmocom/trxcon_ue${ms}.log" 2>&1 &
        fi
        [ $((ms % 8)) -eq 1 ] || [ "$ms" -eq "$N_MS" ] && echo -e "  ${CYAN}[ue${ms}]${NC} ${gip}:${local_port}"
    done
fi

echo -e "${GREEN}=== [6b/10] Audio PulseAudio ===${NC}"
setup_pulse_sinks   # already done in [2d] - idempotent, safety net for partial restart
echo ""

# ── [6c] Bridge audio : gapk (RTP → gsm_audio) + sortie vers l'hote ──────────
echo -e "${GREEN}=== [6c/10] Bridge audio (gapk + sortie hote) ===${NC}"
audio_bridge
echo ""

echo -e "${GREEN}=== [7/10] Mobile ===${NC}"
if [ "$MOBILE_MODE" = "split" ]; then
    for ms in $(seq 1 "$N_MS"); do
        run_in_tmux "ue${ms}" "mobile -c /root/.osmocom/bb/mobile_ms${ms}.cfg"
    done
else
    for g in $(seq 1 "$N_GROUPS"); do
        if [ "$g" -gt 1 ]; then
            prev_ip="127.0.0.$((g-1))"
            echo -ne "  ${CYAN}Attente Grp$((g-1)) (${prev_ip}:4247)${NC}"
            r=0
            while ! bash -c "echo >/dev/tcp/${prev_ip}/4247" 2>/dev/null; do
                sleep 2; echo -n "."; r=$((r+1)); [ $r -ge 30 ] && break
            done
            echo -e " ${GREEN}OK${NC}"; sleep 3
        fi
        run_in_tmux "ue_g${g}" "mobile -c /root/.osmocom/bb/mobile_group${g}.cfg"
        echo -e "  ${GREEN}Groupe ${g} demarre${NC}"
    done
fi

echo -e "${GREEN}=== [9/10] Asterisk ===${NC}"
run_in_tmux "asterisk" "pkill asterisk 2>/dev/null; sleep 2; pkill -9 asterisk 2>/dev/null; sleep 1; rm -f
↪  /var/lib/asterisk/astdb.sqlite3; asterisk -cvvv"

echo -e "${GREEN}=== [10/10] SMSC ===${NC}"
if [ -f /etc/osmocom/smsc-start.sh ]; then
    run_in_tmux "smsc" "/etc/osmocom/smsc-start.sh"
else
    run_in_tmux "smsc" "echo 'SMSC non disponible'"
fi

# ── Cleanup: remove the useless main window ──────────────────────────────────
if [ "$TMUX_OK" = true ]; then
    sleep 1
    tmux -S "$TMUX_SOCKET" kill-window -t "${SESSION}:main" 2>/dev/null || true
fi

# ── Resume ───────────────────────────────────────────────────────────────────
echo ""
echo -e "${GREEN}╔═══════════════════════════════════════════════════════════════╗${NC}"
echo -e "${GREEN}║  Op${OPERATOR_ID} - ${N_MS} MS - ${MOBILE_MODE} - ${PHY_MODE} - ${N_TRX} TRX     ║${NC}"
echo -e "${GREEN}╚═══════════════════════════════════════════════════════════════╝${NC}"
```

```
echo ""
if [ "$PHY_MODE" = "virtphy" ]; then
    echo -e "  ${CYAN}PHY       :${NC} virtphy (osmo-bts-virtual, multicast)"
    echo -e "  ${CYAN}tmux      :${NC} bts  virtphy*  ue*  asterisk  smsc"
else
    echo -e "  ${CYAN}PHY       :${NC} faketrx (osmo-bts-trx, TRXD)"
    echo -e "  ${CYAN}BB ports  :${NC} $(bb_port 1) .. $(bb_port $N_MS)"
    echo -e "  ${CYAN}tmux      :${NC} faketrx  trxcon*  ue*  asterisk  smsc"
fi
if [ "$TMUX_OK" = true ]; then
    echo -e "  ${CYAN}Attach    :${NC} tmux -S ${TMUX_SOCKET} attach -t ${SESSION}"
    tmux -S "$TMUX_SOCKET" select-window -t "${SESSION}:ue_g1" 2>/dev/null \
        || tmux -S "$TMUX_SOCKET" select-window -t "${SESSION}:ue1" 2>/dev/null || true
fi
echo ""
echo -e "${GREEN}run.sh termine${NC}"
```

2.204 /opt/GSM/osmo-operator/scripts/send-mt-sms.sh

1008 octets, 29 lignes → 29 lignes

```
#!/bin/bash
# send-mt-sms.sh - send a local MT SMS through proto-smsc-sendmt
# Usage: ./send-mt-sms.sh <imsi> <message> [from_number]
set -euo pipefail

OPERATOR_ID="${OPERATOR_ID:-1}"
SC_ADDRESS="1999001${OPERATOR_ID}444"
SENDMT_SOCKET="/tmp/sendmt_socket"

if [ $# -lt 2 ]; then
    echo "Usage: $0 <imsi> <message> [from_number]"
    echo "  Ex: $0 001010000000001 'Bonjour!'"
    exit 1
fi

DEST_IMSI="$1"; MESSAGE="$2"; FROM="${3:-${SC_ADDRESS}}"

[ ! -S "$SENDMT_SOCKET" ] && { echo "ERREUR: $SENDMT_SOCKET absent"; exit 1; }

sms-encode-text "$MESSAGE" \
    | gen-sms-deliver-pdu "$FROM" \
    | proto-smsc-sendmt "$SC_ADDRESS" "$DEST_IMSI" "$SENDMT_SOCKET"

# This MT goes out over GSUP towards the radio: the software modems of the bench
# would never see it (« sms-over-gsup » bypasses the osmo-msc SMPP). They are
# notified here, otherwise a manual test only reaches the mobile.
"$(dirname "$0")/sms_notify.py" "$DEST_IMSI" "$FROM" "$MESSAGE" || true

echo "MT SMS envoye → IMSI=$DEST_IMSI"
```

2.205 /opt/GSM/osmo-operator/scripts/sms-interop-relay.py

38624 octets, 931 lignes → 931 lignes

```
#!/usr/bin/env python3
"""
sms-interop-relay.py — Inter-operator SMS relay via proto-smsc-proto

  - HLR VTY lookup with backoff retry (startup race condition)
  - TCP relay send with retry (3 attempts with delay)
  - Configurable TCP timeout (connect_timeout in sms-routing.conf)
```

```
  - N remote operators supported (no hardcoded limit)
  - TPDU error handling
  - Structured log with operator context
"""

import os
import sys
import json
import time
import socket
import struct
import select
import signal
import logging
import argparse
import threading
import subprocess
from pathlib import Path
from typing import Optional, Dict, Tuple

# ═══════════════════════════════════════════════════════════════════════
# Configuration
# ═══════════════════════════════════════════════════════════════════════

# NOTIFY THE SOFTWARE MODEMS. This MT leaves over GSUP toward the radio: the
# osmocom-bb mobile receives it, but the bench's software modems (Android via
# oFono, the postmarketOS VM via its PCI serial port) never see it -
# « sms-over-gsup » bypasses osmo-msc's SMPP path, the only place where they
# could have learnt about it. See sms_notify.py.
# sms_notify.py only lives in the repo, and this file may be the copy deployed
# in /etc/osmocom: both places are searched.
for _rep in (str(Path(__file__).resolve().parent), "/opt/GSM/osmo-operator/scripts"):
    if _rep not in sys.path:
        sys.path.insert(0, _rep)
try:
    from sms_notify import notifier as notifier_modems
except ImportError:                                  # sms_notify.py absent
    def notifier_modems(*a, **k):
        return 0

RELAY_TCP_PORT = 7890
MO_LOG_POLL_INTERVAL = 0.5
HLR_VTY_HOST = "127.0.0.1"
HLR_VTY_PORT = 4258
SENDMT_SOCKET = "/tmp/sendmt_socket"

# Retry config
HLR_LOOKUP_RETRIES = 3
HLR_LOOKUP_DELAY = 2.0
RELAY_SEND_RETRIES = 3
RELAY_SEND_DELAY = 3.0
RELAY_CONNECT_TIMEOUT = 10

# ─── ISM PAYLOAD (start-native.sh --payload-ism option) ──────────────────
# When active, the inter-node TCP DOES NOT CARRY the SMS text: it carries a
# TOKEN. The text leaves over the ISM radio link (scripts/ism_payload.py),
# stamped with the same token, and the receiving node fetches it from the radio
# before injecting the MT. Everything else (addresses, MSISDN, IMSI, SC-address)
# stays on IP. Enabled by the env PAYLOAD_ISM=1 or the [payload] section of
# sms-routing.conf.
PAYLOAD_ISM_RECV_TIMEOUT = float(os.environ.get("ISM_RECV_TIMEOUT", "30"))
_PAYLOAD_LINK = None
_PAYLOAD_LINK_LOCK = threading.Lock()
_PAYLOAD_CFG: Dict[str, str] = {}
```

```
def payload_ism_enabled() -> bool:
    if os.environ.get("PAYLOAD_ISM", "0") == "1":
        return True
    return _PAYLOAD_CFG.get("mode", "").lower() == "ism"


def get_payload_link():
    """The ISM payload link, built once (or None if unavailable)."""
    global _PAYLOAD_LINK
    if not payload_ism_enabled():
        return None
    with _PAYLOAD_LINK_LOCK:
        if _PAYLOAD_LINK is None:
            try:
                sys.path.insert(0, str(Path(__file__).resolve().parent))
                import ism_payload
                _PAYLOAD_LINK = ism_payload.PayloadLink(cfg=dict(_PAYLOAD_CFG))
                logging.info("Payload ISM actif (liaison %s)",
                             _PAYLOAD_CFG.get("link")
                             or os.environ.get("ISM_LINK") or "loopback")
            except Exception as e:
                logging.error("Payload ISM indisponible, repli sur IP : %s", e)
                _PAYLOAD_LINK = False  # « failure » marker, no retry
    return _PAYLOAD_LINK or None


def payload_new_token() -> str:
    import ism_payload
    return ism_payload.new_token()

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [RELAY-OP%(operator_id)s] %(levelname)s %(message)s',
    datefmt='%Y-%m-%dT%H:%M:%S'
)

# ═══════════════════════════════════════════════════════════════════
# TPDU Parser
# ═══════════════════════════════════════════════════════════════════

def decode_bcd_number(hex_bytes: bytes, num_digits: int) -> str:
    number = ""
    for byte in hex_bytes:
        lo = byte & 0x0F
        hi = (byte >> 4) & 0x0F
        if lo <= 9:
            number += str(lo)
        if hi <= 9 and len(number) < num_digits:
            number += str(hi)
    return number[:num_digits]


def parse_sms_submit_tpdu(hex_str: str) -> Optional[Dict]:
    try:
        data = bytes.fromhex(hex_str.strip())
    except ValueError:
        return None
    if len(data) < 4:
        return None

    first_octet = data[0]
    mti = first_octet & 0x03
    if mti != 0x01:
        return None

    vpf = (first_octet >> 3) & 0x03
```

```
    mr = data[1]
    da_len = data[2]
    da_toa = data[3]
    da_bytes = (da_len + 1) // 2

    if len(data) < 4 + da_bytes:
        return None

    da_digits = decode_bcd_number(data[4:4 + da_bytes], da_len)
    ton = (da_toa >> 4) & 0x07
    prefix = "+" if ton == 1 else ""

    offset = 4 + da_bytes
    result = {
        "mti": "SMS-SUBMIT",
        "mr": mr,
        "da_number": f"{prefix}{da_digits}",
        "da_ton": ton,
        "da_npi": da_toa & 0x0F,
        "da_raw": da_digits,
    }

    if offset < len(data):
        result["tp_pid"] = data[offset]; offset += 1
    if offset < len(data):
        result["tp_dcs"] = data[offset]; offset += 1

    if vpf == 2 and offset < len(data): offset += 1
    elif vpf == 3 and offset + 7 <= len(data): offset += 7
    elif vpf == 1 and offset + 7 <= len(data): offset += 7

    if offset < len(data):
        udl = data[offset]; result["tp_udl"] = udl; offset += 1
        dcs = result.get("tp_dcs", 0)
        if dcs == 0x00 and offset < len(data):
            result["user_data_hex"] = data[offset:].hex()

    return result


# ═══════════════════════════════════════════════════════════════
# MO SMS Log Parser
# ═══════════════════════════════════════════════════════════════

class MOLogEntry:
    def __init__(self):
        self.timestamp = ""
        self.imsi = ""
        self.sm_rp_mr = ""
        self.sm_rp_da = ""
        self.sm_rp_oa = ""
        self.sm_rp_oa_number = ""
        self.sm_rp_ui_len = 0
        self.tpdu_hex = ""
        self.parsed_tpdu = None

    @property
    def destination_number(self) -> str:
        if self.parsed_tpdu:
            return self.parsed_tpdu.get("da_raw", "")
        return ""

    @property
    def sender_number(self) -> str:
        return self.sm_rp_oa_number
```

```
    def __repr__(self):
        return f"MOLogEntry(from={self.sender_number}, to={self.destination_number}, imsi={self.imsi})"


def parse_mo_log_block(lines: list) -> Optional[MOLogEntry]:
    entry = MOLogEntry()
    for line in lines:
        line = line.strip()
        if not line: continue
        if line.endswith("Rx MO SM"):
            entry.timestamp = line.replace(" Rx MO SM", "")
        elif line.startswith("IMSI:"):
            entry.imsi = line.split(":", 1)[1].strip()
        elif line.startswith("SM-RP-MR:"):
            entry.sm_rp_mr = line.split(":", 1)[1].strip()
        elif line.startswith("SM-RP-DA:"):
            entry.sm_rp_da = line.split(":", 1)[1].strip()
        elif line.startswith("SM-RP-OA:"):
            parts = line.split()
            entry.sm_rp_oa = line.split(":", 1)[1].strip()
            if parts:
                entry.sm_rp_oa_number = parts[-1]
        elif line.startswith("SM-RP-UI:"):
            try:
                entry.sm_rp_ui_len = int(line.split(":")[1].strip().split()[0])
            except (ValueError, IndexError):
                pass
        elif all(c in "0123456789abcdefABCDEF" for c in line) and len(line) >= 4:
            entry.tpdu_hex = line
            entry.parsed_tpdu = parse_sms_submit_tpdu(line)

    if entry.imsi and entry.tpdu_hex:
        return entry
    return None


# ═══════════════════════════════════════════════════════════════════
# Routing Table — N operators supported
# ═══════════════════════════════════════════════════════════════════

class RoutingTable:
    def __init__(self, config_path: str):
        self.operators: Dict[str, str] = {}
        self.routes: list = []
        self.local_operator_id = "1"
        self.connect_timeout = RELAY_CONNECT_TIMEOUT
        self.retry_count = RELAY_SEND_RETRIES
        self.retry_delay = RELAY_SEND_DELAY
        self._load(config_path)

    def _load(self, path: str):
        section = None
        try:
            with open(path, 'r') as f:
                for line in f:
                    line = line.strip()
                    if not line or line.startswith('#'): continue
                    if line.startswith('[') and line.endswith(']'):
                        section = line[1:-1].lower(); continue
                    if '=' not in line: continue
                    key, val = line.split('=', 1)
                    key = key.strip()
                    val = val.split('#')[0].strip()

                    if section == 'operators':
                        # « ip » or « ip:port ». The explicit port serves the
```

```
                    # WAN: on a remote node, a single public port per
                    # operator can reach the right container's relay.
                    self.operators[key] = val
                elif section == 'routes':
                    # « op_id » or « op_id strip=N ». strip = number of
                    # DIALING CODE digits to remove before forwarding: the
                    # receiving node's HLR only knows the bare number.
                    parts = val.split()
                    op_id = parts[0] if parts else val
                    strip = 0
                    for extra in parts[1:]:
                        if extra.startswith('strip='):
                            try:
                                strip = int(extra.split('=', 1)[1])
                            except ValueError:
                                strip = 0
                    self.routes.append((key, op_id, strip))
                elif section == 'local':
                    if key == 'operator_id': self.local_operator_id = val
                elif section == 'relay':
                    if key == 'connect_timeout':
                        self.connect_timeout = int(val)
                    elif key == 'retry_count':
                        self.retry_count = int(val)
                    elif key == 'retry_delay':
                        self.retry_delay = int(val)
                elif section == 'payload':
                    # [payload] mode=ism, link=fifo|serial|exec|loopback,
                    # tx=/path, rx=/path, serial=/dev/ttyUSB0,
                    # tx_cmd=..., rx_cmd=..., recv_timeout=30
                    _PAYLOAD_CFG[key] = val

        self.routes.sort(key=lambda x: len(x[0]), reverse=True)
    except FileNotFoundError:
        logging.warning(f"Routing config not found: {path}, using defaults")

def lookup(self, msisdn: str) -> Optional[str]:
    clean = msisdn.lstrip('+')
    for prefix, op_id, _strip in self.routes:
        if clean.startswith(prefix):
            return op_id
    return None

def lookup_strip(self, msisdn: str) -> int:
    """Dialing code digits to remove for this destination (0 = none)."""
    clean = msisdn.lstrip('+')
    for prefix, _op_id, strip in self.routes:
        if clean.startswith(prefix):
            return strip
    return 0

def is_local(self, msisdn: str) -> bool:
    op = self.lookup(msisdn)
    return op == self.local_operator_id or op is None

def get_operator_ip(self, op_id: str) -> Optional[str]:
    """IP only (compatibility: the port is read with get_operator_target)."""
    target = self.operators.get(op_id)
    return target.split(':', 1)[0] if target else None

def get_operator_target(self, op_id: str) -> Optional[Tuple[str, int]]:
    """(ip, port) — default port RELAY_TCP_PORT if not given."""
    target = self.operators.get(op_id)
    if not target:
        return None
    if ':' in target:
```

```
            host, _, port = target.partition(':')
            try:
                return host, int(port)
            except ValueError:
                return host, RELAY_TCP_PORT
        return target, RELAY_TCP_PORT


# ═══════════════════════════════════════════════════════════════════
# HLR VTY — MSISDN → IMSI with retry
# ═══════════════════════════════════════════════════════════════════

def hlr_msisdn_to_imsi(msisdn: str,
                       hlr_host: str = HLR_VTY_HOST,
                       hlr_port: int = HLR_VTY_PORT,
                       retries: int = HLR_LOOKUP_RETRIES,
                       delay: float = HLR_LOOKUP_DELAY) -> Optional[str]:
    """Lookup MSISDN→IMSI via HLR VTY with retry on failure."""
    clean = msisdn.lstrip('+')

    for attempt in range(retries):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(5)
            s.connect((hlr_host, hlr_port))

            def recv_until_prompt(timeout=3):
                data = b""
                deadline = time.time() + timeout
                while time.time() < deadline:
                    try:
                        chunk = s.recv(4096)
                        if not chunk: break
                        data += chunk
                        decoded = data.decode("ascii", errors="replace")
                        lines = decoded.strip().splitlines()
                        if lines:
                            last = lines[-1].strip()
                            if last.endswith(">") or last.endswith("#"):
                                break
                    except socket.timeout:
                        break
                return data.decode("ascii", errors="replace")

            recv_until_prompt()
            s.sendall(b"enable\r\n")
            recv_until_prompt()

            cmd = f"show subscriber msisdn {clean}\r\n"
            s.sendall(cmd.encode())
            response = recv_until_prompt()
            s.close()

            for line in response.splitlines():
                stripped = line.strip()
                if stripped.startswith("IMSI:"):
                    imsi = stripped.split(":", 1)[1].strip()
                    if imsi.isdigit():
                        logging.info(f"HLR match: MSISDN {clean} → IMSI {imsi}")
                        return imsi

            # IMSI not found (not a connection error) — don't retry
            logging.warning(f"MSISDN {clean} not found in HLR (attempt {attempt+1})")
            if attempt < retries - 1:
                time.sleep(delay)
            continue
```

```
        except (ConnectionRefusedError, socket.timeout, OSError) as e:
            logging.warning(f"HLR VTY connection failed (attempt {attempt+1}/{retries}): {e}")
            try: s.close()
            except: pass
            if attempt < retries - 1:
                time.sleep(delay)
            continue
        except Exception as e:
            logging.error(f"HLR VTY unexpected error: {e}")
            try: s.close()
            except: pass
            if attempt < retries - 1:
                time.sleep(delay)

    logging.error(f"HLR lookup failed for MSISDN {clean} after {retries} attempts")
    return None


# ═══════════════════════════════════════════════════════════════════
# MT SMS Injection (local)
# ═══════════════════════════════════════════════════════════════════

def inject_mt_sms(dest_imsi: str, message_text: str, from_number: str,
                  sc_address: str, sendmt_socket: str = SENDMT_SOCKET) -> bool:
    # Wait for socket if not present yet (race at startup)
    for _ in range(5):
        if os.path.exists(sendmt_socket):
            break
        logging.warning(f"sendmt socket {sendmt_socket} not found, waiting 2s...")
        time.sleep(2)

    if not os.path.exists(sendmt_socket):
        logging.error(f"sendmt socket {sendmt_socket} still missing after wait")
        return False

    try:
        # Escape single quotes in message
        safe_msg = message_text.replace("'", "'\\''")
        cmd = (
            f"sms-encode-text '{safe_msg}' "
            f"| gen-sms-deliver-pdu {from_number} "
            f"| proto-smsc-sendmt {sc_address} {dest_imsi} {sendmt_socket}"
        )
        result = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=10)
        if result.returncode == 0:
            logging.info(f"MT SMS injected: IMSI={dest_imsi}")
            n = notifier_modems(dest_imsi, from_number, message_text)
            logging.info(f"Modems notified: {n}")
            return True
        else:
            logging.error(f"MT SMS injection failed (rc={result.returncode}): {result.stderr}")
            return False
    except Exception as e:
        logging.error(f"MT SMS injection error: {e}")
        return False


# ═══════════════════════════════════════════════════════════════════
# TCP Relay Server
# ═══════════════════════════════════════════════════════════════════

class RelayServer(threading.Thread):
    def __init__(self, port: int, operator_id: str, sc_address: str):
        super().__init__(daemon=True)
        self.port = port
```

```
    self.operator_id = operator_id
    self.sc_address = sc_address
    self.running = True

def run(self):
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(('0.0.0.0', self.port))
    server.listen(5)
    server.settimeout(1.0)
    logging.info(f"Relay server listening on port {self.port}")

    while self.running:
        try:
            conn, addr = server.accept()
            threading.Thread(target=self._handle_client, args=(conn, addr), daemon=True).start()
        except socket.timeout:
            continue
        except Exception as e:
            logging.error(f"Server error: {e}")
    server.close()

def _handle_client(self, conn: socket.socket, addr: tuple):
    try:
        conn.settimeout(10)
        data = b""
        while True:
            chunk = conn.recv(4096)
            if not chunk: break
            data += chunk
            if b'\n' in data: break

        if not data:
            conn.close(); return

        msg = json.loads(data.decode('utf-8').strip())
        if msg.get("type") == "mt_sms":
            self._handle_mt_sms(msg, conn)
        else:
            response = {"status": "error", "reason": "unknown message type"}
            conn.sendall((json.dumps(response) + '\n').encode())
    except Exception as e:
        logging.error(f"Client handler error from {addr}: {e}")
    finally:
        conn.close()

def _handle_mt_sms(self, msg: dict, conn: socket.socket):
    dest_msisdn = msg.get("dest_msisdn", "")
    from_number = msg.get("from_number", "")
    message_text = msg.get("message_text", "")

    # ISM payload: the text was not in the TCP, it arrives over the radio.
    # The token is awaited on the link before injecting the MT.
    if msg.get("payload_via") == "ism":
        token = msg.get("payload_token", "")
        link = get_payload_link()
        if link is None:
            response = {"status": "error",
                        "reason": "payload annonce ISM mais liaison radio absente ici"}
            conn.sendall((json.dumps(response) + '\n').encode())
            return
        payload = link.recv(token, timeout=PAYLOAD_ISM_RECV_TIMEOUT)
        if payload is None:
            logging.error("Payload ISM : jeton %s jamais recu par radio", token)
            response = {"status": "error",
                        "reason": f"payload ISM (jeton {token}) non recu par radio"}
```

```
                conn.sendall((json.dumps(response) + '\n').encode())
                return
            message_text = payload.decode("utf-8", "replace")
            logging.info("Payload ISM : texte recu par radio (jeton %s)", token)

        logging.info(f"Incoming interop MT: {from_number} → {dest_msisdn}")

        # Resolve with retries
        dest_imsi = hlr_msisdn_to_imsi(dest_msisdn)
        if not dest_imsi:
            response = {"status": "error", "reason": f"MSISDN {dest_msisdn} not found in HLR"}
            conn.sendall((json.dumps(response) + '\n').encode())
            return

        success = inject_mt_sms(
            dest_imsi=dest_imsi,
            message_text=message_text,
            from_number=from_number,
            sc_address=self.sc_address
        )

        response = {
            "status": "ok" if success else "error",
            "dest_imsi": dest_imsi,
            "reason": "" if success else "injection failed"
        }
        conn.sendall((json.dumps(response) + '\n').encode())


# ═══════════════════════════════════════════════════════════════════
# TCP Relay Client — with retry
# ═══════════════════════════════════════════════════════════════════

def send_interop_mt(target_ip: str, target_port: int, dest_msisdn: str,
                    from_number: str, message_text: str, sc_address: str,
                    sender_imsi: str = "",
                    retries: int = RELAY_SEND_RETRIES,
                    delay: float = RELAY_SEND_DELAY,
                    connect_timeout: int = RELAY_CONNECT_TIMEOUT) -> bool:
    """Send interop MT SMS with retry on TCP failure."""
    msg = {
        "type": "mt_sms",
        "dest_msisdn": dest_msisdn,
        "from_number": from_number,
        "message_text": message_text,
        "sc_address": sc_address,
        "sender_imsi": sender_imsi,
    }

    # ISM payload: the text leaves over the radio, the TCP only keeps a token.
    link = get_payload_link()
    if link is not None:
        try:
            token = payload_new_token()
            link.send(token, message_text.encode("utf-8"))
            msg["payload_via"] = "ism"
            msg["payload_token"] = token
            msg["message_text"] = ""   # the text does not go over IP
            logging.info("Payload ISM : texte emis par radio (jeton %s)", token)
        except Exception as e:
            # The radio failed: the SMS is not left mute, fall back to IP.
            logging.error("Payload ISM echoue, repli IP pour ce SMS : %s", e)

    for attempt in range(retries):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
            s.settimeout(connect_timeout)
            s.connect((target_ip, target_port))
            s.sendall((json.dumps(msg) + '\n').encode())

            data = b""
            while True:
                chunk = s.recv(4096)
                if not chunk: break
                data += chunk
                if b'\n' in data: break
            s.close()

            if data:
                response = json.loads(data.decode('utf-8').strip())
                if response.get("status") == "ok":
                    logging.info(f"Interop MT delivered: {from_number} → {dest_msisdn}")
                    return True
                else:
                    logging.error(f"Interop MT rejected: {response.get('reason')}")
                    # Don't retry if the remote explicitly rejected
                    return False
            else:
                logging.warning(f"Empty response from relay (attempt {attempt+1})")

        except (ConnectionRefusedError, socket.timeout, OSError) as e:
            logging.warning(f"Relay send to {target_ip}:{target_port} failed "
                            f"(attempt {attempt+1}/{retries}): {e}")
            try: s.close()
            except: pass

        if attempt < retries - 1:
            logging.info(f"Retrying in {delay}s...")
            time.sleep(delay)

    logging.error(f"Interop send failed after {retries} attempts to {target_ip}")
    return False


# ═══════════════════════════════════════════════════════════════════
# GSM 7-bit Decoder
# ═══════════════════════════════════════════════════════════════════

GSM7_BASIC = (
    "@£$¥èéùìòÇ\nØø\rÅå"
    "Δ_ΦΓΛΩΠΨΣΘΞ\x1bÆæßÉ"
    " !\"#¤%&'()*+,-./"
    "0123456789:;<=>?"
    "¡ABCDEFGHIJKLMNO"
    "PQRSTUVWXYZÄÖÑÜ§"
    "¿abcdefghijklmno"
    "pqrstuvwxyzäöñüà"
)

def decode_gsm7(data: bytes, num_chars: int) -> str:
    chars = []
    for i in range(num_chars):
        byte_offset = (i * 7) // 8
        bit_offset = (i * 7) % 8
        if byte_offset >= len(data): break
        val = (data[byte_offset] >> bit_offset) & 0x7F
        if bit_offset > 1 and byte_offset + 1 < len(data):
            val = ((data[byte_offset] >> bit_offset) |
                   (data[byte_offset + 1] << (8 - bit_offset))) & 0x7F
        if val < len(GSM7_BASIC):
            chars.append(GSM7_BASIC[val])
        else:
```

```
            chars.append('?')
    return ''.join(chars)


def extract_text_from_tpdu(tpdu_hex: str) -> str:
    parsed = parse_sms_submit_tpdu(tpdu_hex)
    if not parsed: return ""
    dcs = parsed.get("tp_dcs", 0)
    udl = parsed.get("tp_udl", 0)
    ud_hex = parsed.get("user_data_hex", "")
    if dcs == 0x00 and ud_hex:
        try:
            return decode_gsm7(bytes.fromhex(ud_hex), udl)
        except: pass
    return ""


def extract_text_via_sms_decode(tpdu_hex: str) -> str:
    try:
        result = subprocess.run(
            ['sms-pdu-decode', '-n'],
            input=tpdu_hex + '\n',
            capture_output=True, text=True, timeout=5
        )
        if result.returncode == 0:
            lines = result.stdout.strip().split('\n')
            text_lines = []
            found_length = False
            for line in lines:
                if found_length and line.strip():
                    text_lines.append(line)
                if line.strip().startswith('Length:'):
                    found_length = True
            if text_lines:
                return '\n'.join(text_lines).strip()
    except (FileNotFoundError, subprocess.TimeoutExpired):
        pass
    return extract_text_from_tpdu(tpdu_hex)


# ═══════════════════════════════════════════════════════════════════
# MO Log Watcher
# ═══════════════════════════════════════════════════════════════════

class MOLogWatcher(threading.Thread):
    def __init__(self, log_path: str, routing: RoutingTable,
                 operator_id: str, sc_address: str):
        super().__init__(daemon=True)
        self.log_path = log_path
        self.routing = routing
        self.operator_id = operator_id
        self.sc_address = sc_address
        self.running = True
        self._position = 0

    def run(self):
        logging.info(f"Watching MO log: {self.log_path}")
        while self.running and not Path(self.log_path).exists():
            time.sleep(2)
        if not self.running: return

        with open(self.log_path, 'r') as f:
            f.seek(0, 2)
            self._position = f.tell()

        logging.info(f"MO log watcher started (pos={self._position})")
```

```
    while self.running:
        try:
            self._check_new_entries()
        except Exception as e:
            logging.error(f"MO watcher error: {e}")
        time.sleep(MO_LOG_POLL_INTERVAL)

def _check_new_entries(self):
    try:
        with open(self.log_path, 'r') as f:
            f.seek(self._position)
            new_data = f.read()
            self._position = f.tell()
    except FileNotFoundError:
        return
    if not new_data: return

    blocks = []
    current_block = []
    for line in new_data.split('\n'):
        if "Rx MO SM" in line and current_block:
            blocks.append(current_block)
            current_block = [line]
        else:
            current_block.append(line)
    if current_block:
        blocks.append(current_block)

    for block in blocks:
        entry = parse_mo_log_block(block)
        if entry:
            self._route_mo_sms(entry)

def _route_mo_sms(self, entry: MOLogEntry):
    dest = entry.destination_number
    sender = entry.sender_number
    if not dest:
        logging.warning(f"MO SMS without parsable destination: {entry}")
        return

    logging.info(f"MO SMS: {sender} → {dest} (from IMSI {entry.imsi})")
    target_op = self.routing.lookup(dest)

    if target_op is None:
        logging.warning(f"No route for destination {dest}")
        return

    if target_op == self.operator_id:
        logging.info(f"Local delivery for {dest}")
        self._deliver_local(entry, dest)
        return

    target = self.routing.get_operator_target(target_op)
    if not target:
        logging.error(f"No IP for operator {target_op}")
        return
    target_ip, target_port = target

    # WAN dialing code: removed HERE, on the sending side. The receiving
    # node gets the bare number, the one its HLR knows; without this it
    # looks up « 3310001 » and answers « MSISDN not found in HLR ».
    strip = self.routing.lookup_strip(dest)
    dest_out = dest.lstrip('+')[strip:] if strip else dest
    if strip:
        logging.info(f"WAN strip {strip}: {dest} → {dest_out}")
```

```
        logging.info(f"Interop route: {dest} → Op{target_op} ({target_ip}:{target_port})")

        message_text = extract_text_via_sms_decode(entry.tpdu_hex)
        if not message_text:
            message_text = extract_text_from_tpdu(entry.tpdu_hex)
        if not message_text:
            logging.warning(f"Could not decode text from TPDU: {entry.tpdu_hex[:40]}...")
            message_text = "[undecoded]"

        success = send_interop_mt(
            target_ip=target_ip,
            target_port=target_port,
            dest_msisdn=dest_out,
            from_number=sender,
            message_text=message_text,
            sc_address=self.sc_address,
            sender_imsi=entry.imsi,
            retries=self.routing.retry_count,
            delay=self.routing.retry_delay,
            connect_timeout=self.routing.connect_timeout,
        )

        if success:
            logging.info(f"✓ Interop SMS: {sender} → {dest} (Op{target_op})")
        else:
            logging.error(f"✗ Interop SMS failed: {sender} → {dest} (Op{target_op})")

    def _deliver_local(self, entry: MOLogEntry, dest_msisdn: str):
        dest_imsi = hlr_msisdn_to_imsi(dest_msisdn)
        if not dest_imsi:
            logging.warning(f"Local MSISDN {dest_msisdn} not found in HLR")
            return

        message_text = extract_text_via_sms_decode(entry.tpdu_hex)
        if not message_text:
            message_text = extract_text_from_tpdu(entry.tpdu_hex)
        if not message_text:
            message_text = "[undecoded]"

        inject_mt_sms(
            dest_imsi=dest_imsi,
            message_text=message_text,
            from_number=entry.sender_number,
            sc_address=self.sc_address
        )


# ═══════════════════════════════════════════════════════════════════════
# Main
# ═══════════════════════════════════════════════════════════════════════

def main():
    parser = argparse.ArgumentParser(description='SMS Interop Relay')
    parser.add_argument('--config', '-c', default='/etc/osmocom/sms-routing.conf')
    parser.add_argument('--port', '-p', type=int, default=RELAY_TCP_PORT)
    parser.add_argument('--mo-log')
    parser.add_argument('--operator-id', default=os.environ.get('OPERATOR_ID', '1'))
    # --node-id: THE ROUTING KEY, and it is not the operator.
    #
    # With MSISDNs that start with the node (<node>00<op><ms>), sms-routing.conf
    # is indexed by NODE: operator 1 of node 1 and the one of node 2 would
    # otherwise carry the same key "1", and each would take the other's
    # numbers for local ones - an interop SMS would never be sent, it would be
    # injected into the local HLR which does not know the subscriber.
    #
    # The operator keeps what concerns it: the SC-address (the SMS centre
```

```
    # number, which a subscriber sees) and the MO log path. Changing them would
    # rename a file other tools follow, and a number mobiles already hold.
    parser.add_argument('--node-id',
                        default=os.environ.get('OSMO_WAN_NODE',
                               os.environ.get('WAN_NODE_ID', '')))
    args = parser.parse_args()

    op_id = args.operator_id
    node_id = args.node_id if args.node_id not in ('', '0') else op_id
    sc_address = f"1999001{op_id}444"
    mo_log = args.mo_log or f"/var/log/osmocom/mo-sms-op{op_id}.log"

    old_factory = logging.getLogRecordFactory()
    def record_factory(*a, **kw):
        record = old_factory(*a, **kw)
        record.operator_id = op_id
        return record
    logging.setLogRecordFactory(record_factory)

    logging.info(f"SMS Interop Relay starting")
    logging.info(f"  Operator   : {op_id}")
    logging.info(f"  Node       : {node_id}   (cle de routage dans sms-routing.conf)")
    logging.info(f"  SC-address : {sc_address}")
    logging.info(f"  MO log     : {mo_log}")
    logging.info(f"  TCP port   : {args.port}")

    routing = RoutingTable(args.config)
    routing.local_operator_id = node_id

    logging.info(f"  Operators  : {routing.operators}")
    logging.info(f"  Routes     : {len(routing.routes)} entries")
    logging.info(f"  Retry      : {routing.retry_count}x, delay={routing.retry_delay}s,
↪   timeout={routing.connect_timeout}s")

    # ISM payload: if requested, the radio link is brought up NOW so that the
    # receiver (RX) runs before the first SMS, and it is stated clearly.
    if payload_ism_enabled():
        link = get_payload_link()
        if link is not None:
            logging.info("  Payload    : ISM (texte par radio, signalisation par IP)")
        else:
            logging.warning("  Payload    : ISM demande mais liaison indisponible — repli IP")
    else:
        logging.info("  Payload    : IP (texte dans le relais TCP)")

    server = RelayServer(port=args.port, operator_id=op_id, sc_address=sc_address)
    server.start()

    watcher = MOLogWatcher(log_path=mo_log, routing=routing, operator_id=op_id, sc_address=sc_address)
    watcher.start()

    def signal_handler(sig, frame):
        logging.info("Shutting down...")
        watcher.running = False
        server.running = False
        sys.exit(0)

    signal.signal(signal.SIGINT, signal_handler)
    signal.signal(signal.SIGTERM, signal_handler)

    logging.info("Relay running. Ctrl+C to stop.")
    while True:
        time.sleep(1)


if __name__ == '__main__':
```

```
    main()
```

2.206 /opt/GSM/osmo-operator/scripts/sms-routing-setup.sh

26234 octets, 572 lignes → 572 lignes

```
#!/bin/bash
# sms-routing-setup.sh - Complete management of inter-operator SMS routing
#
# Exportable functions called from start-docker.sh:
#   sms_routing_generate  <op_id> <n_ops> <destdir> [op1_nms op2_nms ...]
#   sms_routing_validate  <conf_file>
#   sms_routing_summary   <n_ops>         [op1_nms op2_nms ...]
#
# COMMON formulas (identical to run.sh, hlr-feed-subscribers.sh):
#   MSISDN  = <node>00<op_id><ms_idx>       (100101, 100102, 100201...)
#   IMSI    = MCC(3) + MNC(2) + printf('%04d%06d', op_id, ms_idx)
#   KI      = 00 11 22 33 44 55 66 77 88 99 aa bb cc dd <ms_hex> <op_hex>
#
# Routing architecture:
#   ┌──────────────────────────────────────────────────────────────┐
#   │  MS (op1, ms1)  MSISDN=10001  sends to  MSISDN=20002 (op2, ms2)   │
#   │    → proto-smsc-daemon (op1) → MO log                        │
#   │    → sms-interop-relay.py (op1) : lookup prefix 20002 → op2  │
#   │    → TCP 172.20.0.12:7890                                    │
#   │    → sms-interop-relay.py (op2) : MSISDN→IMSI via HLR VTY    │
#   │    → proto-smsc-sendmt (op2) → HLR (op2) → MS (op2, ms2)     │
#   └──────────────────────────────────────────────────────────────┘
#
# Standalone usage (test/debug):
#   bash sms-routing-setup.sh generate 1 3 /tmp/sms-cfg 2 2 3
#   bash sms-routing-setup.sh validate /tmp/sms-cfg/sms-routing-op1.conf
#   bash sms-routing-setup.sh summary  3 2 2 3

set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

# ═══════════════════════════════════════════════════════════════════
# Internal helpers
# ═══════════════════════════════════════════════════════════════════

_sms_op_backbone_ip() { echo "172.20.0.$((10 + $1))"; }
# MSISDN = <node>00<operator><2-digit rank>: the first digit is the node
# address, it is what routes. The node comes from the radio plan written by
# generate_configs.sh, from the environment, or is 1.
_sms_node_id() {
    local n="${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}"
    [ -n "$n" ] || n="$(sed -n 's/^PLAN_NODE=//p' /etc/osmocom/radio-plan.env 2>/dev/null | tail -1)"
    case "$n" in [1-9]) ;; *) n=1 ;; esac
    printf '%s' "$n"
}
_sms_op_msisdn()      { echo $(( $(_sms_node_id) * 100000 + $1 * 100 + $2 )); }   # op_id ms_idx
_sms_op_ms_imsi()     {                                  # mcc mnc op_id ms_idx
    local mcc=$1 mnc=$2 op=$3 ms=$4
    printf '%s%s%04d%06d' "$mcc" "$mnc" "$op" "$ms"
}
_sms_sc_address()     { printf '1999001%s444' "$1"; }    # op_id

_log_ok()   { echo -e "  ${GREEN}✓${NC}  $*"; }
_log_warn() { echo -e "  ${YELLOW}⚠${NC}  $*"; }
_log_err()  { echo -e "  ${RED}✗${NC}  $*" >&2; }
```

```
# ═══════════════════════════════════════════════════════════════════
# sms_routing_generate
#
# Generates configs/sms-routing-op<N>.conf for each operator.
# Each operator's file contains ALL the routes (local + remote) and is mounted
# as /etc/osmocom/sms-routing.conf in the container.
#
# Usage:
#   sms_routing_generate <op_id> <n_ops> <destdir> [ms_counts...]
#   ms_counts: number of MS per operator (1 value per operator)
#              ex: "2 3 1" → op1=2MS, op2=3MS, op3=1MS
#
# Sortie :
#   <destdir>/sms-routing-op<op_id>.conf
#
# ═══════════════════════════════════════════════════════════════════
sms_routing_generate() {
    local op_id="$1"
    local n_ops="$2"
    local destdir="$3"
    shift 3
    local ms_counts=("$@")   # tableau : ms_counts[0]=nMS_op1, [1]=nMS_op2, ...

    mkdir -p "$destdir"
    local outfile="${destdir}/sms-routing-op${op_id}.conf"

    # ── OPERATOR NUMBERS DO NOT NECESSARILY START AT 1 ──────────────────
    # On the multi-operator bench the CONTAINERS carry ranks 2 and 3 - rank 1
    # is the native one (OP_ID_BASE=2). Enumerating 1..n_ops would give wrong
    # MSISDNs (100101 instead of 100201), wrong IMSIs, and a relay announced
    # at an address that is nobody's (172.20.0.11 is the native, not a
    # container).
    #
    # ⚠ ms_counts stays a 0-based POSITIONAL list: its first element
    # describes the first enumerated operator, whatever its rank. The
    # NUMBERING is shifted, not the indexing - hence the (i - _b).
    local _b="${OP_ID_BASE:-1}"
    local _last=$(( _b + n_ops - 1 ))
    local -a nms
    for i in $(seq "$_b" "$_last"); do
        nms[$i]=${ms_counts[$((i - _b))]:-1}
    done

    local mcc="${MCC:-001}"

    cat > "$outfile" << HEADER
# ═══════════════════════════════════════════════════════════════════
# sms-routing-op${op_id}.conf
# Generated by sms-routing-setup.sh - $(date '+%Y-%m-%d %H:%M:%S')
#
# Local operator   : ${op_id}
# Operator count   : ${n_ops}
# MS per operator  : $(for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_ops - 1 ))"); do printf
 ↪  'Op%s=%s ' "$i" "${nms[$i]}"; done)
#
# MSISDN routing:
#   MSISDN = <node> × 100000 + op_id × 100 + ms_idx
#   Longest-prefix match rule (the longest prefix wins)
#
# ═══════════════════════════════════════════════════════════════════

[local]
operator_id = ${op_id}
sc_address  = $(_sms_sc_address "$op_id")
hlr_vty_ip  = 127.0.0.1
hlr_vty_port = 4258
```

```
sendmt_socket = /tmp/sendmt_socket
mo_log = /var/log/osmocom/mo-sms-op${op_id}.log

[operators]
# operator_id = container_ip  (backbone network 172.20.0.0/24)
HEADER

    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_ops - 1 ))"); do
        printf '%s = %s\n' "$i" "$(_sms_op_backbone_ip "$i")" >> "$outfile"
    done

    cat >> "$outfile" << ROUTES_HEADER

[routes]
# Format: prefix = operator_id
# Longest-prefix match: the longest prefix wins.
# Strategy:
#   1. Exact MSISDN of each MS            → precise routing (max priority)
#   2. Short prefix per operator          → fallback for unknown MSISDNs
#   3. E.164 prefixes (+336XX...)           → international routing
#
ROUTES_HEADER

    # ── Exact routes per MS (maximum priority) ──────────────────────────
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_ops - 1 ))"); do
        local mnc; mnc=$(printf '%02d' "$i")
        printf '\n# ── Operateur %s (SC=%s) ──────────────────────────────────\n' \
               "$i" "$(_sms_sc_address "$i")" >> "$outfile"
        printf '# %s MS declares\n' "${nms[$i]}" >> "$outfile"

        for ms in $(seq 1 "${nms[$i]}"); do
            local msisdn; msisdn=$(_sms_op_msisdn "$i" "$ms")
            local imsi; imsi=$(_sms_op_ms_imsi "$mcc" "$mnc" "$i" "$ms")
            # IMSI comment for traceability
            printf '%-8s = %s   # IMSI=%s\n' "$msisdn" "$i" "$imsi" >> "$outfile"
        done

        # Short operator prefix (fallback for MSISDN outside the list)
        local op_prefix="${i}0000"
        printf '# Fallback operateur %s (MSISDN inconnu)\n' "$i" >> "$outfile"
        printf '%-8s = %s\n' "$op_prefix" "$i" >> "$outfile"

        # Fictional E.164 prefix: +336<op>X... (simulated French format)
        local e164_prefix; e164_prefix=$(printf '336%02d' "$i")
        printf '%-8s = %s   # E.164 +33 6%02d...\n' "$e164_prefix" "$i" "$i" >> "$outfile"
    done

    cat >> "$outfile" << ROUTES_FOOTER

[relay]
# TCP port on which this relay listens for incoming MT from other operators.
port = 7890
# Connection timeout toward a remote relay (seconds)
connect_timeout = 10
# Resend attempts if the remote relay is unavailable
retry_count = 3
retry_delay = 5
ROUTES_FOOTER

    _log_ok "sms-routing-op${op_id}.conf → ${outfile}"
}


# ═══════════════════════════════════════════════════════════════════════
# sms_routing_generate_all
#
```

```
# Generates the configs for ALL operators in a single pass.
# Called from start-docker.sh in bridge mode.
#
# Usage:
#   sms_routing_generate_all <n_ops> <destdir> [ms_counts...]
#
# ═══════════════════════════════════════════════════════════════════════
sms_routing_generate_all() {
    local n_ops="$1"
    local destdir="$2"
    shift 2
    local ms_counts=("$@")

    echo -e "${CYAN}${BOLD}── SMS Routing - generation configs (${n_ops} operateurs) ──${NC}"

    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_ops - 1 ))"); do
        sms_routing_generate "$i" "$n_ops" "$destdir" "${ms_counts[@]}"
    done

    echo -e "  ${GREEN}✓ Configs generees dans ${destdir}${NC}"
}


# ═══════════════════════════════════════════════════════════════════════
# sms_routing_validate
#
# Checks the consistency of an sms-routing.conf file:
#   • [local] section present
#   • [operators] section not empty
#   • [routes] section not empty
#   • No exact prefix collision between two different operators
#   • Local MSISDNs routed to the local operator
#
# Returns 0 if OK, 1 on errors.
#
# Usage:
#   sms_routing_validate <conf_file>
#
# ═══════════════════════════════════════════════════════════════════════
sms_routing_validate() {
    local conf="$1"
    local errors=0

    echo -e "${CYAN}Validation : ${conf}${NC}"

    if [ ! -f "$conf" ]; then
        _log_err "Fichier introuvable : $conf"
        return 1
    fi

    # ── Sections obligatoires ──────────────────────────────────────────
    for section in local operators routes; do
        if ! grep -q "^\[${section}\]" "$conf"; then
            _log_err "Section manquante : [${section}]"
            errors=$(( errors + 1 ))
        fi
    done

    # ── operator_id in [local] ─────────────────────────────────────────
    local local_op
    local_op=$(awk '/^\[local\]/,/^\[/' "$conf" \
               | grep -E '^\s*operator_id\s*=' \
               | head -1 | cut -d= -f2 | tr -d ' ')
    if [ -z "$local_op" ]; then
        _log_err "[local] : operator_id manquant"
        errors=$(( errors + 1 ))
```

```
else
    _log_ok "[local] operator_id = ${local_op}"
fi

# ── At least 1 operator declared ──────────────────────────────────────
local op_count
op_count=$(awk '/^\[operators\]/,/^\[/' "$conf" \
          | grep -cE '^\s*[0-9]+\s*=' || echo 0)
if [ "$op_count" -eq 0 ]; then
    _log_err "[operators] : aucun operateur declare"
    errors=$(( errors + 1 ))
else
    _log_ok "[operators] : ${op_count} operateur(s)"
fi

# ── Routes not empty ──────────────────────────────────────────────────
local route_count
route_count=$(awk '/^\[routes\]/,/^\[/' "$conf" \
              | grep -cE '^\s*[0-9]+\s*=' || echo 0)
if [ "$route_count" -eq 0 ]; then
    _log_err "[routes] : aucune route definie"
    errors=$(( errors + 1 ))
else
    _log_ok "[routes] : ${route_count} entree(s)"
fi

# ── Prefix collision: same MSISDN → two different operators ───────────
local collision_count=0
while IFS= read -r line; do
    [[ "$line" =~ ^[[:space:]]*# ]] && continue   # comment
    [[ "$line" =~ ^[[:space:]]*$ ]] && continue   # empty
    [[ "$line" =~ ^\[ ]]           && continue   # section

    local prefix op
    prefix=$(echo "$line" | cut -d= -f1 | tr -d ' ')
    op=$(echo "$line" | cut -d= -f2 | cut -d'#' -f1 | tr -d ' ')

    # Look for the same prefix with a different operator
    local other_op
    other_op=$(awk -v p="$prefix" -v o="$op" '
        /^\[routes\]/,/^\[/ {
            if ($0 ~ "^[[:space:]]*" p "[[:space:]]*=") {
                split($0, a, "=")
                gsub(/[[:space:]]/, "", a[2])
                sub(/#.*/, "", a[2])
                if (a[2] != o) { print a[2]; exit }
            }
        }' "$conf")

    if [ -n "$other_op" ]; then
        _log_err "Collision : prefixe ${prefix} → op${op} ET op${other_op}"
        collision_count=$(( collision_count + 1 ))
        errors=$(( errors + 1 ))
    fi
done < <(awk '/^\[routes\]/,/^\[/' "$conf")

[ "$collision_count" -eq 0 ] && _log_ok "Pas de collision de prefixes"

# ── Routes MSISDN locaux → operateur local ────────────────────────────
if [ -n "$local_op" ]; then
    local wrong_local=0
    while IFS= read -r line; do
        [[ "$line" =~ ^[[:space:]]*# ]] && continue
        [[ "$line" =~ ^[[:space:]]*$ ]] && continue
        [[ "$line" =~ ^\[ ]]           && continue
```

```
            local prefix op
            prefix=$(echo "$line" | cut -d= -f1 | tr -d ' ')
            op=$(echo "$line" | cut -d= -f2 | cut -d'#' -f1 | tr -d ' ')

            # An MSISDN starting with op_id should route to op_id
            if [[ "$prefix" =~ ^${local_op}[0-9]* ]] && [ "$op" != "$local_op" ]; then
                _log_warn "MSISDN local ${prefix} route vers op${op} ≠ op${local_op}"
                wrong_local=$(( wrong_local + 1 ))
            fi
        done < <(awk '/^\[routes\]/,/^\[/' "$conf")
        [ "$wrong_local" -eq 0 ] && _log_ok "MSISDN locaux correctement routes"
    fi

    # ── Resultat ──────────────────────────────────────────────────────────
    echo ""
    if [ "$errors" -eq 0 ]; then
        echo -e "  ${GREEN}${BOLD}✓ Validation OK${NC}"
        return 0
    else
        echo -e "  ${RED}${BOLD}✗ ${errors} erreur(s)${NC}"
        return 1
    fi
}


# ═══════════════════════════════════════════════════════════════════════════
# sms_routing_summary
#
# Prints the full routing table in readable form (debug / audit).
# Shows every registered MSISDN and its target operator.
#
# Usage:
#   sms_routing_summary <n_ops> [ms_counts...]
#
# ═══════════════════════════════════════════════════════════════════════════
sms_routing_summary() {
    local n_ops="$1"
    shift
    local ms_counts=("$@")

    local mcc="${MCC:-001}"

    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════════════════════════════╗"
    echo "║                  SMS Routing - Table complete                     ║"
    printf "║  %d operateur(s)                                                  ║\n" "$n_ops"
    echo "╚═══════════════════════════════════════════════════════════════════╝"
    echo -e "${NC}"

    # Table header
    printf "${BOLD}%-12s %-8s %-20s %-18s %-15s${NC}\n" \
           "MSISDN" "Op→" "IMSI" "Container IP" "SC-address"
    printf "%-12s %-8s %-20s %-18s %-15s\n" \
           "────────────" "────────" "────────────────────" "──────────────────" "───────────────"

    local total_ms=0
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_ops - 1 ))"); do
        # ms_counts is POSITIONAL (0-based) whereas i is an operator RANK,
        # which does not necessarily start at 1: index by (i - OP_ID_BASE),
        # not (i - 1), otherwise the neighbour's count is read.
        local n_ms=${ms_counts[$((i - ${OP_ID_BASE:-1}))]:-1}
        local mnc; mnc=$(printf '%02d' "$i")
        local container_ip; container_ip=$(_sms_op_backbone_ip "$i")
        local sc; sc=$(_sms_sc_address "$i")

        for ms in $(seq 1 "$n_ms"); do
```

```
            local msisdn; msisdn=$(_sms_op_msisdn "$i" "$ms")
            local imsi; imsi=$(_sms_op_ms_imsi "$mcc" "$mnc" "$i" "$ms")
            printf "%-12s ${CYAN}Op%-6s${NC} %-20s %-18s %-15s\n" \
                   "$msisdn" "$i" "$imsi" "$container_ip" "$sc"
            total_ms=$(( total_ms + 1 ))
        done

        # Separator line between operators
        [ "$i" -lt "$n_ops" ] && \
            printf "${YELLOW}%-12s %-8s %-20s %-18s %-15s${NC}\n" \
                   "──────────" "↓ Op$((i+1))" "" "" ""
    done

    echo ""
    printf "${BOLD}Total : %d MS  |  %d operateur(s)${NC}\n" "$total_ms" "$n_ops"

    echo ""
    echo -e "${CYAN}── Reseau inter-operateurs ──${NC}"
    for i in $(seq "${OP_ID_BASE:-1}" "$(( ${OP_ID_BASE:-1} + n_ops - 1 ))"); do
        # ms_counts is POSITIONAL (0-based) whereas i is an operator RANK,
        # which does not necessarily start at 1: index by (i - OP_ID_BASE).
        local n_ms=${ms_counts[$((i - ${OP_ID_BASE:-1}))]:-1}
        printf "  Op%-2s  %s  ← TCP 7890  (relay)  %d MS\n" \
               "$i" "$(_sms_op_backbone_ip "$i")" "$n_ms"
    done

    echo ""
    echo -e "${CYAN}── Exemple de flux SMS inter-op ──${NC}"
    if [ "$n_ops" -ge 2 ]; then
        local src_msisdn; src_msisdn=$(_sms_op_msisdn 1 1)
        local dst_msisdn; dst_msisdn=$(_sms_op_msisdn 2 1)
        printf "  %s (Op1) ──GSUP──► HLR(Op1) ──► proto-smsc-daemon\n" "$src_msisdn"
        printf "     ──► sms-interop-relay(Op1) [lookup %s → Op2]\n" "$dst_msisdn"
        printf "     ──TCP──► sms-interop-relay(Op2) @ %s:7890\n" "$(_sms_op_backbone_ip 2)"
        printf "     ──► HLR(Op2) VTY → IMSI lookup\n"
        printf "     ──► proto-smsc-sendmt(Op2) → MS\n"
    fi
    echo ""
}


# ═══════════════════════════════════════════════════════════════════════
# sms_routing_mount_args
#
# Returns the Docker -v arguments to mount the right operator's SMS config.
# Used in start_operator() of start-docker.sh.
#
# Usage:
#   vol_args+=$(sms_routing_mount_args <op_id> <destdir>)
#
# ═══════════════════════════════════════════════════════════════════════
sms_routing_mount_args() {
    local op_id="$1"
    local destdir="$2"
    local conf="${destdir}/sms-routing-op${op_id}.conf"

    if [ -f "$conf" ]; then
        echo "-v ${conf}:/etc/osmocom/sms-routing.conf"
    else
        echo "" # no volume if the file does not exist yet
    fi
}


# ═══════════════════════════════════════════════════════════════════════
# sms_routing_wait_ready
```

```
#
# Waits for an operator's SMS relay to be ready (TCP port 7890 listening).
# Optionally called after start_operator() to make sure the relay is
# available before injecting test SMS.
#
# Usage:
#   sms_routing_wait_ready <container_name> [timeout_sec]
#
# ============================================================================
sms_routing_wait_ready() {
    local container="$1"
    local timeout="${2:-60}"
    local elapsed=0

    echo -ne "  Attente relay SMS (${container}) "

    while [ "$elapsed" -lt "$timeout" ]; do
        if docker exec "$container" \
           bash -c "ss -tlnp | grep -q ':7890'" 2>/dev/null; then
            echo -e " ${GREEN}✓${NC}"
            return 0
        fi
        echo -n "."
        sleep 2
        elapsed=$(( elapsed + 2 ))
    done

    echo -e " ${YELLOW}timeout${NC}"
    return 1
}


# ============================================================================
# sms_routing_test_send
#
# Sends a test SMS between two MS of two different operators.
# Requires the containers to be started and the HLRs fed.
#
# Usage:
#   sms_routing_test_send <src_container> <src_imsi> <dst_msisdn> <message>
#
# ============================================================================
sms_routing_test_send() {
    local src_container="$1"
    local src_imsi="$2"
    local dst_msisdn="$3"
    local message="$4"

    echo -e "${CYAN}Test SMS :${NC} ${src_container}  IMSI=${src_imsi} → MSISDN=${dst_msisdn}"

    if ! docker ps --format '{{.Names}}' | grep -q "^${src_container}$"; then
        _log_err "Container ${src_container} non demarre"
        return 1
    fi

    docker exec "$src_container" bash -c "
        OPERATOR_ID=\${OPERATOR_ID:-1}
        SC_ADDRESS=\"\$(_sms_sc_address \$OPERATOR_ID)\" 2>/dev/null \
            || SC_ADDRESS=\"1999001\${OPERATOR_ID}444\"
        sms-encode-text '${message}' \
            | gen-sms-deliver-pdu \"\$SC_ADDRESS\" \
            | proto-smsc-sendmt \"\$SC_ADDRESS\" '${src_imsi}' /tmp/sendmt_socket
    " && _log_ok "SMS envoye" || _log_err "Echec envoi SMS"
}
```

```
# ═══════════════════════════════════════════════════════════════════
# CLI entry point (direct call from the command line)
# ═══════════════════════════════════════════════════════════════════
_usage() {
    cat << 'EOF'
Usage: sms-routing-setup.sh <commande> [args...]

Commandes :
  generate <op_id> <n_ops> <destdir> [ms_counts...]
      Genere sms-routing-op<op_id>.conf dans <destdir>.
      ms_counts : nombre de MS par operateur (ex: 2 3 1 pour 3 ops).

  generate-all <n_ops> <destdir> [ms_counts...]
      Genere les configs pour TOUS les operateurs.

  validate <conf_file>
      Valide un fichier sms-routing.conf et affiche les erreurs.

  summary <n_ops> [ms_counts...]
      Affiche la table de routage complete (audit / debug).

  wait-ready <container_name> [timeout_sec]
      Attend que le relay SMS soit actif dans un container.

  test-send <container> <src_imsi> <dst_msisdn> <message>
      Envoie un SMS de test via proto-smsc-sendmt.

Variables d'environnement :
  MCC   : Mobile Country Code (defaut: 001)

Exemples :
  bash sms-routing-setup.sh generate-all 3 /tmp/sms-cfg 2 2 3
  bash sms-routing-setup.sh validate /tmp/sms-cfg/sms-routing-op1.conf
  bash sms-routing-setup.sh summary 2 2 3
EOF
}

if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then
    # Executed directly (not sourced)
    CMD="${1:-help}"; shift 2>/dev/null || true
    case "$CMD" in
        generate)      sms_routing_generate    "$@" ;;
        generate-all)  sms_routing_generate_all "$@" ;;
        validate)      sms_routing_validate     "$@" ;;
        summary)       sms_routing_summary      "$@" ;;
        wait-ready)    sms_routing_wait_ready   "$@" ;;
        test-send)     sms_routing_test_send    "$@" ;;
        help|-h|--help) _usage ;;
        *)
            echo -e "${RED}Commande inconnue : ${CMD}${NC}" >&2
            _usage; exit 1 ;;
    esac
fi
```

2.207 /opt/GSM/osmo-operator/scripts/sms_notify.py

2835 octets, 78 lignes → 78 lignes

```
#!/usr/bin/env python3
"""sms_notify.py - notify the bench modems that an SMS arrives.

The bench osmo-msc runs with « sms-over-gsup »: an SM-RP received from the
radio goes to the HLR (gsm_04_11.c) without ever passing through
gsm340_rx_tpdu(), hence without reaching sms_route_mt_sms() nor the SMPP
```

```
server. The « smpp-first » and « route prefix ... 100101 » lines of osmo-msc.cfg
are dead configuration as long as sms-over-gsup is there: an ESME bound on port
2775 receives nothing. And the MT coming back over GSUP is pushed directly to
the subscriber over the radio path - the osmocom-bb mobile gets it, the
software modems (Android through oFono, the postmarketOS VM through its PCI
serial port) do not.

Hence this service path: where the gateway decides on a local delivery, it also
notifies the modems, which present the message to their system in the expected
AT form. The GSUP path is untouched, so the mobile keeps receiving as before -
nothing is duplicated here.

Two uses:
    import       : from sms_notify import notifier      (in the relay)
    command line : sms_notify.py <imsi> <from> <text>   (send-mt-sms.sh)
"""
import json
import os
import socket
import sys

# The modem inboxes. The same program (osmo-phonesim-banc.py) runs twice on
# this bench and therefore needs two ports: the Android modem (listen mode) and
# the VM modem (--connect mode).
DEFAUT = "127.0.0.1:12348,127.0.0.1:12349"
TIMEOUT = 2.0


def cibles():
    val = os.environ.get("OSMO_SMS_NOTIFY", DEFAUT)
    out = []
    for morceau in val.split(","):
        morceau = morceau.strip()
        if not morceau:
            continue
        hote, _, port = morceau.rpartition(":")
        try:
            out.append((hote or "127.0.0.1", int(port)))
        except ValueError:
            continue
    return out


def notifier(imsi, sender, text, log=None):
    """Hand a copy of the SMS to the modems. Best effort: a missing modem is
    not an error (the VM may be off), never raises."""
    charge = (json.dumps({"imsi": imsi, "from": sender, "text": text},
                         ensure_ascii=False) + "\n").encode()
    touches = 0
    for hote, port in cibles():
        try:
            s = socket.create_connection((hote, port), timeout=TIMEOUT)
        except OSError:
            continue                        # no modem on this port: normal
        try:
            s.sendall(charge)
            touches += 1
        except OSError:
            pass
        finally:
            s.close()
    if log:
        log("modems prevenus : %d" % touches)
    return touches
```

```
if __name__ == "__main__":
    if len(sys.argv) < 4:
        print("usage: sms_notify.py <imsi> <from> <texte>", file=sys.stderr)
        sys.exit(2)
    n = notifier(sys.argv[1], sys.argv[2], " ".join(sys.argv[3:]))
    print("modems prevenus : %d" % n)
```

2.208 /opt/GSM/osmo-operator/scripts/smsc-start.sh

5630 octets, 144 lignes → 144 lignes

```
#!/bin/bash
# smsc-start.sh - starts proto-smsc-daemon + the interop relay
# Waits for the HLR GSUP port, then checks the sendmt socket with retries.

set -euo pipefail

OPERATOR_ID="${OPERATOR_ID:-1}"
HLR_IP="${HLR_IP:-127.0.0.2}"
SMSC_IPA_NAME="SMSC-OP${OPERATOR_ID}"
MO_LOG="/var/log/osmocom/mo-sms-op${OPERATOR_ID}.log"
SENDMT_SOCKET="/tmp/sendmt_socket"
ROUTING_CONF="/etc/osmocom/sms-routing.conf"
RELAY_PORT="${RELAY_PORT:-7890}"

GREEN='\033[0;32m'; CYAN='\033[0;36m'; YELLOW='\033[0;33m'; RED='\033[0;31m'; NC='\033[0m'

echo -e "${GREEN}╔═══════════════════════════════════════════════════╗${NC}"
echo -e "${GREEN}║  SMS Gateway - Operateur ${OPERATOR_ID}                          ║${NC}"
echo -e "${GREEN}╚═══════════════════════════════════════════════════╝${NC}"
echo -e "  HLR         : ${CYAN}${HLR_IP}${NC}"
echo -e "  IPA name    : ${CYAN}${SMSC_IPA_NAME}${NC}"
echo -e "  MO log      : ${CYAN}${MO_LOG}${NC}"
echo -e "  MT socket   : ${CYAN}${SENDMT_SOCKET}${NC}"
echo -e "  Relay port : ${CYAN}${RELAY_PORT}${NC}"
echo -e "  Routing     : ${CYAN}${ROUTING_CONF}${NC}"

rm -f "$SENDMT_SOCKET"
mkdir -p /var/log/osmocom

# ── Wait for the HLR GSUP port (4222) ──────────────────────────
echo -ne "${YELLOW}[SMSC] Attente OsmoHLR GSUP (4222)"
retries=60
while [ $retries -gt 0 ]; do
    if bash -c "echo >/dev/tcp/${HLR_IP}/4222" 2>/dev/null; then
        echo -e " ${GREEN}✓${NC}"
        break
    fi
    # Fallback: also check with ss
    if ss -tln 2>/dev/null | grep -q ":4222 "; then
        echo -e " ${GREEN}✓${NC}"
        break
    fi
    echo -n "."
    sleep 1
    retries=$(( retries - 1 ))
done
if [ $retries -eq 0 ]; then
    echo -e " ${RED}TIMEOUT - HLR non disponible${NC}"
    echo -e "${YELLOW}[SMSC] Tentative de demarrage quand meme...${NC}"
fi

sleep 3
```

```
# ── 1. proto-smsc-daemon (background) ──────────────────────────────
echo -e "${GREEN}[1/2] proto-smsc-daemon${NC}"

# Check that the binary exists
if ! command -v proto-smsc-daemon >/dev/null 2>&1; then
    echo -e "  ${RED}proto-smsc-daemon introuvable dans PATH${NC}"
    echo -e "  ${YELLOW}SMS local uniquement via MSC VTY${NC}"
    # Still start the relay for incoming SMS over TCP
else
    proto-smsc-daemon "$HLR_IP" "$SMSC_IPA_NAME" "$MO_LOG" "$SENDMT_SOCKET" &
    DAEMON_PID=$!
    echo -e "  PID: ${CYAN}${DAEMON_PID}${NC}"

    # Wait for the sendmt socket, with timeout
    echo -ne "  Socket sendmt"
    for i in $(seq 1 30); do
        if [ -S "$SENDMT_SOCKET" ]; then
            echo -e " ${GREEN}✓${NC}"
            break
        fi
        echo -n "."
        sleep 1
    done
    if [ ! -S "$SENDMT_SOCKET" ]; then
        echo -e " ${YELLOW}non cree (les MT SMS ne fonctionneront pas)${NC}"
    fi
fi

# ── 2. Relay interop (background) ──────────────────────────────────
echo -e "${GREEN}[2/2] sms-interop-relay${NC}"

# Check that the script exists
RELAY_SCRIPT="/etc/osmocom/sms-interop-relay.py"
if [ ! -f "$RELAY_SCRIPT" ]; then
    echo -e "  ${RED}${RELAY_SCRIPT} introuvable${NC}"
    echo -e "  ${YELLOW}SMS inter-operateur desactive${NC}"
else
    # --node-id: the routing key of sms-routing.conf is the node, not the
    # operator (MSISDN = <node>00<operator><rank>). Without it the relay takes
    # the numbers of the other nodes for local ones and never sends.
    python3 "$RELAY_SCRIPT" \
        --config "$ROUTING_CONF" \
        --port "$RELAY_PORT" \
        --mo-log "$MO_LOG" \
        --operator-id "$OPERATOR_ID" \
        --node-id "${OSMO_WAN_NODE:-${WAN_NODE_ID:-1}}" &
    RELAY_PID=$!
    echo -e "  PID: ${CYAN}${RELAY_PID}${NC}"

    # Check that the TCP port is listening
    echo -ne "  Relay TCP ${RELAY_PORT}"
    for i in $(seq 1 15); do
        if ss -tlnp 2>/dev/null | grep -q ":${RELAY_PORT} "; then
            echo -e " ${GREEN}✓${NC}"
            break
        fi
        echo -n "."
        sleep 1
    done
fi

echo ""
echo -e "${GREEN}═══════════════════════════════════════════════════${NC}"
echo -e "${GREEN}  SMS Gateway actif - Op${OPERATOR_ID}${NC}"
echo -e "${GREEN}  MT local : send-mt-sms.sh <imsi> 'msg'${NC}"
echo -e "${GREEN}  MO log   : tail -f ${MO_LOG}${NC}"
```

```
echo -e "${GREEN}═══════════════════════════════════════════════${NC}"

cleanup() {
    echo -e "\n${YELLOW}[SMSC] Arret...${NC}"
    [ -n "${DAEMON_PID:-}" ] && kill $DAEMON_PID 2>/dev/null || true
    [ -n "${RELAY_PID:-}" ] && kill $RELAY_PID 2>/dev/null || true
    rm -f "$SENDMT_SOCKET"
    exit 0
}
trap cleanup SIGINT SIGTERM

# Wait for one of the processes to end
if [ -n "${DAEMON_PID:-}" ] && [ -n "${RELAY_PID:-}" ]; then
    wait -n $DAEMON_PID $RELAY_PID 2>/dev/null || true
elif [ -n "${DAEMON_PID:-}" ]; then
    wait $DAEMON_PID 2>/dev/null || true
elif [ -n "${RELAY_PID:-}" ]; then
    wait $RELAY_PID 2>/dev/null || true
else
    # No process to wait for - just sleep
    while true; do sleep 60; done
fi

echo -e "${YELLOW}[SMSC] Processus termine, arret...${NC}"
cleanup
```

2.209 /opt/GSM/osmo-operator/scripts/status.sh

338 octets, 14 lignes → 14 lignes

```
#!/bin/bash
cmd="${1:-status}"
set -ex
systemctl daemon-reload
systemctl $cmd osmo-hlr \
               osmo-msc \
               osmo-mgw \
               osmo-stp \
               osmo-bsc \
               osmo-ggsn \
               osmo-sgsn \
               osmo-sip-connector \
               osmo-bts-trx \
               osmo-pcu
```

2.210 /opt/GSM/osmo-operator/scripts/voix-forks.sh

4099 octets, 72 lignes → 72 lignes

```
#!/bin/bash
# ============================================================================
#  scripts/voix-forks.sh - the two voice settings that live in the forks
# ============================================================================
#
#  The bench voice depends on two lines that are not in osmo-operator: they are
#  in qosmo (run_modules, cfgs), a repository that the update resynchronises
#  with « git reset --hard FETCH_HEAD ». Any fix applied by hand in those trees
#  disappears at the next start. They are therefore reapplied, after each
#  fetch, from here - this file is the only one that knows how.
#
#  1. « io-tch-format rtp » in cfgs/mobile_group1.cfg.
#     This is the template that run.sh (run_modules/20-mobile-cfg.sh) copies
```

```
#     over ~/.osmocom/bb/mobile_group1.cfg at each start - it overwrites what
#     start-native.sh generated. Without this line the mobile reads and writes
#     its TCH frames in the Texas Instruments layout of real Calypsos, while
#     pont.py codes them with libosmocoding, i.e. in the RTP/RFC3551 layout
#     (33 bytes opened by the 0xD signature). Neither side understands the
#     other, no counter complains, and the voice comes out robotic.
#
#  2. CALYPSO_PULSE_LATENCY_MSEC in run_modules/70-l2.sh and
#     68-sidecar-mobile.sh. The mobile writes one GSM frame (160 samples,
#     20 ms) per snd_pcm_writei; if the ALSA-PulseAudio plugin buffer is too
#     short, each write returns -EPIPE, and pq_alsa.c answers with a silent
#     snd_pcm_prepare() that tears down and rebuilds the PulseAudio stream.
#     Measured with a writer paced at 20 ms like the mobile, over 10 s:
#         80 ms -> 16 rebuilds   120 -> 19   160 -> 1   200 -> 0   240 -> 0
#     During a real call: 50 per second. start-native.sh exports 320, which is
#     enough; the fork default is aligned for whoever runs run.sh alone.
#
#  Callers: update.sh (osmo_poser_voix_forks), the osmo-update service
#  generated by iso_modules/81-cloture-systeme.sh (right after the fetch), and
#  iso_modules/70-scripts.sh (on the ROOTFS, when building the ISO).
#
#  Usage: voix-forks.sh [prefix]     (prefix = a ROOTFS, empty by default)
#  Idempotent, no effect if the forks are absent, always exit 0.
#
#  Check: `grep io-tch-format ~/.osmocom/bb/mobile_group1.cfg` after a start;
#  and during an established call, the sink-input index of the mobile must stay
#  fixed (`pactl list short sink-inputs`, two readings 1 s apart).
# ------------------------------------------------------------------------------
set -u
ROOT="${1:-}"
VOIX_LATENCE="${CALYPSO_PULSE_LATENCY_MSEC_DEFAUT:-320}"
VOIX_TCH_FORMAT="${MS_TCH_FORMAT:-rtp}"

voix_forks_poser() {
    local f n fait=0
    for f in "$ROOT/opt/GSM/qosmo/cfgs/mobile_group1.cfg"; do
        [ -f "$f" ] || continue
        grep -q '^[[:space:]]*io-tch-format' "$f" && continue
        # Right after « io-handler », in the tch-voice block, same indentation.
        if sed -i '/^[[:space:]]*tch-voice[[:space:]]*$/,/^[[:space:]]*no shutdown/
         ↪  s/^\([[:space:]]*\)io-handler\(.*\)$/\1io-handler\2\n\1io-tch-format '"$VOIX_TCH_FORMAT"'/' "$f" \
           && grep -q "io-tch-format ${VOIX_TCH_FORMAT}" "$f"; then
            echo "  [voix] ${f#"$ROOT"} : io-tch-format ${VOIX_TCH_FORMAT} repose (voix robotisee sinon)"; fait=1
        else
            echo "  [voix] ${f#"$ROOT"} : io-tch-format NON pose - a regarder a la main" >&2
        fi
    done
    for f in "$ROOT/opt/GSM/qosmo/run_modules/70-l2.sh" \
             "$ROOT/opt/GSM/qosmo/run_modules/68-sidecar-mobile.sh"; do
        [ -f "$f" ] || continue
        n="$(sed -n 's/^: "${CALYPSO_PULSE_LATENCY_MSEC:=\([0-9]*\)}"/\1/p' "$f" | head -1)"
        [ -n "$n" ] || continue
        [ "$n" -ge "$VOIX_LATENCE" ] 2>/dev/null && continue
        sed -i "s|^: \"\${CALYPSO_PULSE_LATENCY_MSEC:=[0-9]*}\"|:
         ↪  \"\\\${CALYPSO_PULSE_LATENCY_MSEC:=${VOIX_LATENCE}}\"|" "$f" \
            && { echo "  [voix] ${f#"$ROOT"} : tampon du mobile ${n} -> ${VOIX_LATENCE} ms (flux PulseAudio
             ↪  remonte sinon)"; fait=1; }
    done
    return 0
}
voix_forks_poser
exit 0
```

2.211 /opt/GSM/osmo-operator/scripts/wslg-audio-bridge.sh

14512 octets, 306 lignes → 306 lignes

```
#!/bin/bash
# wslg-audio-bridge.sh - sends the audio of the Osmocom containers to the host
# speakers, from inside Docker. Works in two cases:
#
#   • WSL2 + WSLg  : the audio host = WSLg PulseServer (/mnt/wslg/PulseServer),
#                    which itself relays to Windows.
#   • native Linux : the audio host = the PulseAudio/PipeWire of the graphical
#                    session (/run/user/<uid>/pulse/native).
#
# Principle:
#   host-relay  → exposes the host pulse over TCP (module-native-protocol-tcp)
#   container   → direct audio bridge inside the container:
#                     parec(gsm_audio.monitor) | paplay(--server=tcp:<gw>:4713)
#
# module-tunnel-sink is not used: over TCP/WSL its clock drifts (latency
# climbs above 1 s → choppy sound). parec|paplay = a single clock server (the
# host's) + a plain byte pipe → clean audio.
#
# The container keeps its own PulseAudio; gsm_audio is untouched (the dashboard
# keeps reading gsm_audio.monitor).
#
# Usage:
#   wslg-audio-bridge.sh host-relay              # host: open pulse over TCP
#   wslg-audio-bridge.sh container <name> [gw]   # container: start the bridge
#   wslg-audio-bridge.sh test <name>             # 880Hz beep through gsm_audio
#   wslg-audio-bridge.sh down <name>             # stop the bridge
#   wslg-audio-bridge.sh all <name> [gw]         # host-relay + container + test
set -euo pipefail

TCP_PORT="${WSLG_TCP_PORT:-4713}"
# ACL: the whole private block 172.16/12 (covers the docker subnets gsm-inter/op)
TCP_ACL="${WSLG_TCP_ACL:-127.0.0.1;172.16.0.0/12}"

log()  { echo -e "  [host-audio] $*"; }
die()  { echo -e "  [host-audio][FAIL] $*" >&2; exit 1; }

# Host IP as seen from the container = gateway of its default route.
detect_gw() {
    docker exec "$1" sh -c "ip route 2>/dev/null | awk '/default/{print \$3; exit}'" 2>/dev/null || true
}

# User of the graphical session (owner of the native pulse).
# The criterion is not the name but the presence of the pulse socket: under
# `sudo -i` SUDO_USER is lost and `id -un` returns root, whose
# /run/user/0/pulse/native almost never exists. /run/user/* is scanned as a
# last resort. Same logic as session_pulse_user() in start-docker.sh.
session_user() {
    local u uid sock
    for u in "${HOST_PULSE_USER:-}" "${SUDO_USER:-}" "$(logname 2>/dev/null || true)" "$(id -un)"; do
        [ -n "$u" ] || continue
        uid="$(id -u "$u" 2>/dev/null)" || continue
        [ -S "/run/user/${uid}/pulse/native" ] && { echo "$u"; return 0; }
    done
    for sock in /run/user/*/pulse/native; do
        [ -S "$sock" ] || continue
        uid="${sock#/run/user/}"; uid="${uid%%/*}"
        u="$(getent passwd "$uid" 2>/dev/null | cut -d: -f1)"
        [ -n "$u" ] && { echo "$u"; return 0; }
    done
    echo "${HOST_PULSE_USER:-${SUDO_USER:-$(id -un)}}"   # nothing found: fallback
}
```

```
# Runs `pactl ...` on the native host pulse, as the session user.
pactl_native() {
    local u uid rt
    u="$(session_user)"; uid="$(id -u "$u")"; rt="/run/user/${uid}"
    if [ "$(id -un)" = "$u" ]; then
        XDG_RUNTIME_DIR="$rt" PULSE_SERVER="unix:${rt}/pulse/native" pactl "$@"
    else
        sudo -u "$u" XDG_RUNTIME_DIR="$rt" PULSE_SERVER="unix:${rt}/pulse/native" pactl "$@"
    fi
}

# Runs any pulse tool (pactl, parec, timeout parec...) against the host pulse,
# under WSLg as well as native. Generalises pactl_native.
host_pa() {
    local bin="$1"; shift
    if [ -S /mnt/wslg/PulseServer ]; then
        PULSE_SERVER="unix:/mnt/wslg/PulseServer" "$bin" "$@"
    else
        local u uid rt
        u="$(session_user)"; uid="$(id -u "$u")"; rt="/run/user/${uid}"
        if [ "$(id -un)" = "$u" ]; then
            XDG_RUNTIME_DIR="$rt" PULSE_SERVER="unix:${rt}/pulse/native" "$bin" "$@"
        else
            sudo -u "$u" XDG_RUNTIME_DIR="$rt" PULSE_SERVER="unix:${rt}/pulse/native" "$bin" "$@"
        fi
    fi
}

# ── Host: pick a microphone source that really carries signal ──────────────
#
# The browser captures the system default source; a default input that is
# present, unmuted, at a good volume can still deliver pure digital silence.
# `pactl` cannot tell the two apart: the source has to be listened to.
#
# No device name is hard-coded (they change from one machine to the next): the
# signal is measured. Deliberately binary criterion - a live microphone always
# has a noise floor, hence non-zero bytes; a misrouted input only outputs
# zeros. `tr -d '\0' | wc -c` is enough: no python/sox dependency, it runs
# wherever parec exists.
# Beware of the false negative: a suspended source takes ~2 s to deliver its
# first bytes. Judging on a short window would declare every idle device mute.
# Two distinct cases are therefore never confused:
#   - nothing captured          -> the device did not start (undetermined)
#   - captured but all zeros    -> digital silence (the real fault)
# Up to MIC_PROBE_BYTES are read, then the pipe is cut (head closes it, parec
# gets SIGPIPE): fast on a live device, only a dead one costs the timeout.
MIC_PROBE_SECS="${MIC_PROBE_SECS:-4}"
# Second chance for a device that returned nothing (verdict 2). A suspended
# source wakes up in ~2 s; a USB card behind a hub, or a bluetooth device that
# has to negotiate its profile, needs more.
MIC_PROBE_SECS_SLOW="${MIC_PROBE_SECS_SLOW:-12}"
MIC_PROBE_BYTES="${MIC_PROBE_BYTES:-16000}"   # 1 s @ 8 kHz s16le mono
MIC_MIN_BYTES="${MIC_MIN_BYTES:-4000}"        # below this: no verdict possible
MIC_MIN_NONZERO="${MIC_MIN_NONZERO:-100}"
MIC_MAX_PROBES="${MIC_MAX_PROBES:-8}"
# 1 = never touch the host system default (only report what was measured).
# Useful on a workstation: the selection below is permanent, it survives the
# stop of osmo-operator.
MIC_KEEP_DEFAULT="${MIC_KEEP_DEFAULT:-0}"

# Prints "<captured_bytes> <nonzero_bytes>" on stdout.
# $2 = window duration (default MIC_PROBE_SECS).
probe_source() {
    local src="$1" secs="${2:-$MIC_PROBE_SECS}" tmp raw nz
    tmp="$(mktemp)"
    # `|| true`: timeout exits 124, head causes a SIGPIPE - under
```

```
    # `set -o pipefail` either would kill the script.
    { host_pa timeout "$secs" parec -d "$src" \
        --format=s16le --rate=8000 --channels=1 2>/dev/null || true; } \
      | head -c "$MIC_PROBE_BYTES" > "$tmp" 2>/dev/null || true
    raw="$(wc -c < "$tmp")"; nz="$(tr -d '\000' < "$tmp" | wc -c)"
    rm -f "$tmp"
    echo "${raw:-0} ${nz:-0}"
}

# 0 = signal, 1 = digital silence, 2 = undetermined (nothing captured)
source_verdict() {
    local r raw nz secs="${2:-$MIC_PROBE_SECS}"
    r="$(probe_source "$1" "$secs")"; raw="${r%% *}"; nz="${r##* }"
    [ "$raw" -ge "$MIC_MIN_BYTES" ] || return 2
    [ "$nz"  -ge "$MIC_MIN_NONZERO" ] || return 1
    return 0
}

# Verdict 2 (nothing captured) is undetermined, not a fault: a device that is
# merely slow to start gets a longer look before another input is preferred.
verdict_retry() {
    local src="$1" rc=0
    source_verdict "$src" || rc=$?
    [ "$rc" = "2" ] || return "$rc"
    log "[mic]   ${src} : rien en ${MIC_PROBE_SECS}s - seconde chance sur ${MIC_PROBE_SECS_SLOW}s"
    rc=0; source_verdict "$src" "$MIC_PROBE_SECS_SLOW" || rc=$?
    return "$rc"
}

host_mic() {
    command -v parec >/dev/null 2>&1 \
        || { log "[mic] parec absent - selection micro ignoree"; return 0; }
    host_pa pactl info >/dev/null 2>&1 \
        || { log "[mic] pulse hote injoignable - selection micro ignoree"; return 0; }

    # Scope: this function sets the default of the host pulse. The dashboard
    # microphone is the one of the workstation displaying the page
    # (getUserMedia -> WebSocket -> pacat -> gsm_mic). The two only coincide
    # when browsing from the host machine. From another workstation, the page
    # selector (web/index.html, #mic-dev) decides, nothing else.
    log "[mic] selection de l'entree du PULSE DE L'HOTE (le navigateur, lui,"
    log "[mic]   utilise le micro de SON poste - selecteur dans le tableau de bord)."

    local cur rc; cur="$(host_pa pactl get-default-source 2>/dev/null || true)"
    if [ -n "$cur" ]; then
        # `|| rc=$?` is mandatory: under `set -e` a bare call of source_verdict
        # returning 1 (silence) or 2 (nothing captured) would kill the script
        # on the spot, without a log line.
        rc=0; verdict_retry "$cur" || rc=$?
        case "$rc" in
            0) log "[mic] source par defaut deja valide : ${cur}"; return 0 ;;
            1) log "[mic] defaut '${cur}' : SILENCE NUMERIQUE - recherche d'une autre entree..." ;;
            2) log "[mic] defaut '${cur}' : aucune donnee meme en ${MIC_PROBE_SECS_SLOW}s - recherche..." ;;
        esac
    fi

    if [ "$MIC_KEEP_DEFAULT" = "1" ]; then
        log "[mic] MIC_KEEP_DEFAULT=1 - defaut systeme laisse tel quel, pas de bascule."
        return 0
    fi

    local src i=0
    for src in $(host_pa pactl list short sources 2>/dev/null | awk '{print $2}' | grep -v '\.monitor$'); do
        [ "$src" = "$cur" ] && continue
        i=$((i + 1)); [ "$i" -gt "$MIC_MAX_PROBES" ] && break
        rc=0; verdict_retry "$src" || rc=$?        # see the set -e remark above
```

```
        if [ "$src" = "0" ]; then
            if host_pa pactl set-default-source "$src" >/dev/null 2>&1; then
                log "[mic] source par defaut -> ${src} (signal detecte)"
                # The switch is permanent: pulse keeps it after osmo-operator
                # stops. The previous value is logged so that going back is a
                # command, not an investigation.
                [ -n "$cur" ] && log "[mic]   ancien defaut : ${cur}"
                [ -n "$cur" ] && log "[mic]   restaurer : pactl set-default-source ${cur}"
                log "[mic]   (MIC_KEEP_DEFAULT=1 desactive cette bascule)"
                log "[mic]   un onglet qui tient deja le micro reste epingle sur"
                log "[mic]   l'ancien peripherique : recouper/relancer le bouton micro."
                return 0
            fi
        else
            log "[mic]   ${src} : $([ "$rc" = 1 ] && echo 'silence numerique' || echo 'aucune donnee')"
        fi
    done

    log "[mic][!] AUCUNE source micro ne produit de signal (${i} testee(s))."
    log "[mic][!]   -> micro coupe materiellement (touche/interrupteur) ?"
    log "[mic][!]   -> mauvais profil UCM : verifier 'pactl list cards' / alsamixer."
    log "[mic][!]   Sans ca l'echo-test rendra le HP mais jamais la voix."
    return 0
}

# ── Host: expose the host pulse over TCP (idempotent) ──────────────────
host_relay() {
    if [ -S /mnt/wslg/PulseServer ]; then
        # WSL2 + WSLg: world-accessible socket (root OK)
        export PULSE_SERVER="unix:/mnt/wslg/PulseServer"
        if pactl list short modules 2>/dev/null | grep -q "protocol-tcp.*port=${TCP_PORT}"; then
            log "relai TCP deja actif (WSLg) :${TCP_PORT}"; return 0
        fi
        pactl load-module module-native-protocol-tcp \
            "port=${TCP_PORT}" "auth-ip-acl=${TCP_ACL}" auth-anonymous=1 >/dev/null \
            || die "load module-native-protocol-tcp (WSLg)"
        log "relai TCP WSLg → Windows active :${TCP_PORT}"
    else
        # native Linux: pulse of the graphical session
        local u uid; u="$(session_user)"; uid="$(id -u "$u")"
        [ -S "/run/user/${uid}/pulse/native" ] \
           || die "pulse de session introuvable (/run/user/${uid}/pulse/native) - session audio demarree ? user=$u"
        if pactl_native list short modules 2>/dev/null | grep -q "protocol-tcp.*port=${TCP_PORT}"; then
            log "relai TCP deja actif (natif, user=$u) :${TCP_PORT}"; return 0
        fi
        pactl_native load-module module-native-protocol-tcp \
            "port=${TCP_PORT}" "auth-ip-acl=${TCP_ACL}" auth-anonymous=1 >/dev/null \
            || die "load module-native-protocol-tcp (natif)"
        log "relai TCP natif active (user=$u) :${TCP_PORT}"
    fi
}

# ── Container: supervised parec|paplay bridge to the host pulse ────────
container_bridge() {
    local name="$1" gw="${2:-}"
    [ -n "$name" ] || die "nom de conteneur requis"
    [ -z "$gw" ] && gw="$(detect_gw "$name")"
    [ -n "$gw" ] || die "gateway introuvable pour $name"
    docker exec -i "$name" sh -s -- "tcp:${gw}:${TCP_PORT}" <<'EOSH'
set -eu
RELAY="$1"
pactl info >/dev/null 2>&1 || { echo "  [host-audio][FAIL] pas de daemon pulse dans le conteneur" >&2; exit 1; }
# The host relay must be reachable
if ! pactl --server="$RELAY" info >/dev/null 2>&1; then
    echo "  [host-audio][FAIL] relai hote injoignable ($RELAY) - lancer 'host-relay' d'abord" >&2; exit 1
fi
```

```
# Stop a previous bridge (idempotent)
pkill -f "paplay --server=${RELAY}" 2>/dev/null || true
[ -f /run/host-audio.pid ] && kill -- "-$(cat /run/host-audio.pid)" 2>/dev/null || true
mkdir -p /var/log/osmocom
# Supervised bridge: restarts if parec/paplay dies. setsid = new group → clean kill.
setsid sh -c '
  while true; do
    parec -d gsm_audio.monitor --latency-msec=30 --format=s16le --rate=8000 --channels=1 \
      | paplay --server='"${RELAY}"' --latency-msec=250 --raw --format=s16le --rate=8000 --channels=1
    sleep 1
  done' >/var/log/osmocom/host-audio.log 2>&1 &
echo $! > /run/host-audio.pid
echo "  [host-audio] pont parec|paplay gsm_audio.monitor -> ${RELAY} (pgid $!)"
EOSH
}

# ── Container: test beep through gsm_audio (end-to-end proof) ──────────────
container_test() {
    docker exec -i "$1" sh -s <<'EOSH'
set -eu
ffmpeg -y -hide_banner -loglevel error -f lavfi -i "sine=frequency=880:duration=1" -ar 8000 -ac 1
 ↪  /tmp/host_test.wav
paplay -d gsm_audio /tmp/host_test.wav
echo "  [host-audio] bip 880Hz injecte dans gsm_audio -> hote"
EOSH
}

container_down() {
    docker exec -i "$1" sh -s <<'EOSH'
set -eu
[ -f /run/host-audio.pid ] && kill -- "-$(cat /run/host-audio.pid)" 2>/dev/null || true
pkill -f 'parec -d gsm_audio.monitor' 2>/dev/null || true
rm -f /run/host-audio.pid
echo "  [host-audio] pont arrete"
EOSH
}

cmd="${1:-}"; shift || true
case "$cmd" in
    host-relay) host_relay ;;
    host-mic)   host_mic ;;
    container)  container_bridge "${1:-}" "${2:-}" ;;
    test)       container_test "${1:-}" ;;
    down)       container_down "${1:-}" ;;
    all)        host_relay; host_mic; container_bridge "${1:-}" "${2:-}"; container_test "${1:-}" ;;
    *) echo "usage: $0 {host-relay|host-mic|container <name> [gw]|test <name>|down <name>|all <name> [gw]}" >&2;
     ↪  exit 2 ;;
esac
```

2.212 /opt/GSM/osmo-operator/startup_scripts/01-defauts-options.sh

34097 octets, 549 lignes → 549 lignes

```
# 01-defauts-options.sh - part of start.sh (defaults and command-line options)
# Sourced by start.sh in the main shell: same order, same variables.

# ── THE NUMBER OF SUBSCRIBERS DOES NOT GO THROUGH ~/.bashrc ────────────────
# /etc/osmocom/coeur.env (N_MS, written by build-iso.sh) is loaded here, once
# and for all, so that the icon (launch.sh -> pkexec), a systemd unit and
# start-multi.sh all see it - not only an interactive root shell. Without it
# run_modules/21-abonnes-hlr.sh falls back to N_MS=1, the HLR only has 100101,
# and the second mobile is created on the fly by "subscriber-create-on-demand
# 5" with a random five-digit MSISDN and NO Ki - unreachable and rejected at
# authentication. The ":=" of the file lets an N_MS=3 set in the environment
```

```
# win.
if [ -f "${OSMOCOM_CFG:-/etc/osmocom}/coeur.env" ]; then
    set -a; . "${OSMOCOM_CFG:-/etc/osmocom}/coeur.env"; set +a
fi
# --- options ----------------------------------------------------------------
DRY=0 VERBOSE=0 ACTION=start PROFILE="${CALYPSO_PROFILE:-faketrx-qemu}" FORCE=0
# SAP disabled by default: the mobile's SIM is the test SIM.
# --sap: the SIM is SERVED to the mobile (sim sap) by the softSIM SAP server,
# started here before the stack. SAP_TYPE: pcsc = the card in the PC/SC reader
# (HID Omnikey, sysmoISIM programmed by tools/osmo-sim.sh), sim = the software
# SIM sim-op1.xml (IMSI of MS#1, COMP128v1, Ki of the HLR), auto = pcsc if a
# card is in a reader, otherwise sim. SAP_SOCK: the socket (the one of MS#1).
: "${SAP_MODE:=0}"
: "${SAP_TYPE:=auto}"
: "${SAP_SOCK:=/tmp/osmocom_sap}"
# 1 as soon as the operator named a profile (--profile, or the mode as positional).
# Used by --dsp, which selects `qemu` ONLY if nobody chose before it.
PROFILE_CHOISI=0
[ -n "${CALYPSO_PROFILE:-}" ] && PROFILE_CHOISI=1
# WAN: never by default. --wan (or WAN_AUTO=1 in /etc/osmo-wan.conf, set by an
# ISO built with --wan) brings it up before handing over to run.sh.
WAN_MESH=0
# What the table will keep (WAN_AUTO_SAVE, network/wan-nodes.sh): 1 only if
# WAN_AUTO=1 brings up the WAN; a --wan on the command line applies to THIS
# launch and leaves WAN_AUTO=0, so that the next one starts without WAN.
WAN_AUTO_KEEP=0
# Inter-STP (SS7 hub) started standalone next to the WAN: auto = only if the
# hub does not exist yet on the network (no local address, no ping answer).
# --interstp forces, --no-interstp skips. One hub for the whole WAN: on the
# node that does not carry it, the remote hub is recognized and left alone.
INTERSTP=auto
# --wan FILE: node sheets to import (network/node-conf.sh).
WAN_IMPORT_FILES=()
# --regen-conf: the WAN table is rebuilt FROM SCRATCH. The table in place
# (/etc/osmo-wan.conf) survives restarts and keeps the nodes once seen; a
# switched-off peer leaves a route that goes PROHIB at the STP. The table is
# set aside (timestamped backup) and rebuilt from the command line
# (--wan-nodes), the sheets (--wan FILE) or the interactive question.
REGEN_WAN_CONF=0
# --payload-ism: the TEXT of the SMS goes over the ISM radio, the rest
# (signalling, SS7, SIP) over IP as with --wan. PAYLOAD_ISM_LINK = backend if
# given (--payload-ism=serial).
PAYLOAD_ISM=0
PAYLOAD_ISM_LINK=""
GEN_CONF_OUT=""
# --virtualbox: brings up the segment and the VMs before the WAN. Refused when
# running INSIDE a VM - the host drives VirtualBox (the script checks it).
VBOX_INTERCO=0
VBOX_NODES=""
VBOX_HOST_NODE=1
# --node: the number of THIS node, chosen at launch and not at build time.
# This is what allows a single ISO for the nine nodes.
NODE_ID=""
NODE_OP=1
HUB_IP=""
# The templates are NOT regenerated by default: run.sh would do it at startup
# and erase the SS7 identity set just before. --regen regenerates them for
# whoever wants to start again from the templates.
REGEN_GABARITS=0
# --menu: asks for the choices instead of guessing them. See the "Menu" block below.
MENU_MODE=0
# ── THE DEFAULT LAYER 1: THE DSP ──────────────────────────────────────────
# The bench starts in DSP mode (c54x_exe, TI mask-ROM) without being asked:
# this is the bench wanted at the first click, as a service as in a terminal,
# standalone as under osmo-multi (which starts the native through
# osmo-banc.service, hence through this default). --grgsm selects the gr-gsm
```

```
# layer 1. The environment wins (DSP_MODE=0 ./start-native.sh), as for the
# rest. If c54x_exe is missing, the bench falls back to gr-gsm and says so
# (below).
: "${DSP_MODE:=1}"
usage() {
    cat <<'USAGE'
Usage : ./start-native.sh [options] [mode]
  Modes (profils) :
    faketrx-qemu   coeur + BTS#0 QEMU + BTS#1 faketrx (defaut)
    faketrx        coeur + fake_trx + trxcon + mobile
    qemu           pipeline Calypso QEMU seul
    noproc         coeur seul (no-process)
    core           alias noproc
    hybrid         alias faketrx-qemu
  Options :
    --docker            banc en conteneurs : passe la main a
                        launch/start-docker.sh avec les autres arguments
    --dsp               (defaut) banc DSP C54x : le TMS320C54x tourne hors QEMU
                        (/opt/GSM/c54x_exe, mask-ROM TI) et decode lui-meme
                        FCCH/SCH/BCCH ; aucun decodeur gr-gsm n'est lance.
                        Le mobile campe (SI1-4, lai=001-01-1) et fait sa mise
                        a jour de localisation. La pile monte comme en mode
                        grgsm (meme profil hybride, coeur, BTS, side-car) ;
                        seuls les modules qemu,pty,osmocon,l2 sont retires et
                        repris par c54x_exe/run.sh. BANC_DSP=none pour n'avoir
                        que la pile.
    --grgsm             couche 1 gr-gsm dans QEMU (qosmo) + pont grgsm_exe
                        (l ancien defaut ; DSP_MODE=0 dans l environnement vaut pareil)
    --bts-classic       [2026-10-21] (alias : --bts-classique) la BTS CLASSIQUE : osmo-bts-trx parle a
↪   pont/pont_dsp.py
                        (TRXD 5700-5702), sans Calypso TRX dans QEMU ; le mobile emule
                        (QEMU + c54x_exe, MODE=dsp) est branche sur le pont. C'est le montage
                        par defaut de --dsp, nomme ; OSMO_TRX=0 force (meme si l environnement
                        l exporte). Exclusif avec --bts / --ts.
                            ./start-native.sh --bts-classic [--ms]           mobile osmocom-bb
                            ./start-native.sh --bts-classic --ms-magnetite   le C139 Magnetite
    --bts               [2026-10-20] (alias : --osmo-trx) le Calypso de QEMU est le
                        TRANSCEIVER de la BTS#0 : firmware trx.highram.elf
                        (osmocom-bb-transceiver, appli « trx ») + appli hote
                        `transceiver` (L1CTL -> TRXD 5700-5702) a la place de
                        mobile et du pont ; osmo-bts-trx s'y branche comme sur le
                        pont. Implique --dsp (c54x_exe/run.sh, OSMO_TRX=1). Par
                        defaut UN Calypso sert TS0 et TS1 (OSMO_TRX_TCH=1, masques
                        0x03) ; voir --ts.
    --ts N              [2026-10-20] intervalles de temps du TRX Calypso (implique --bts) :
                            --ts 1   TS0 seul (CCCH + SDCCH/4, pas d'appel) : OSMO_TRX_TCH=0
                            --ts 2   TS0 + TS1, UN CALYPSO PAR INTERVALLE : le TS1 (TCH/F,
                                     les appels) est servi par un deuxieme telephone TRX
                                     (c54x_exe/run.sh OSMO_TRX_TS1=1, instance 3,
                                     transceiver -2 -L /tmp/osmocom_l2_3) ; le premier
                                     garde le TS0 seul (OSMO_TRX_TCH=0). [21/10] PAS D'APPEL
                                     dans ce mode : le fn du 2e telephone n'est pas aligne
                                     (TCH descendant a ~80 bits faux, ASSIGNMENT FAILURE).
                            --ts 2/1 TS0 + TS1 par UN SEUL Calypso (OSMO_TRX_TCH=1, masques
                                     0x03, OSMO_TRX_TS1=0) : le montage qui a passe un
                                     appel audible le 20/10 21:08.
                        Sans --ts : comme --ts 2/1 (21/10 ; du 20/10 23:10 au 21/10 c'etait --ts 2).
    --ms                [2026-10-20] le telephone emule est un MOBILE. Seul (ou sans
                        option) : le montage habituel, mobile + pont vers osmo-bts-trx.
                        AVEC --bts : un DEUXIEME Calypso (c54x_exe/run.sh INSTANCE=2,
                        alias banc-mobile) campe sur le premier par l'air (pont/air.py) ;
                        il est lance quand le firmware trx est en mode BTS.
                            ./start-native.sh --bts          la radio de la BTS seule
                            ./start-native.sh --bts --ms     la BTS et le mobile, par l'air
    --ms-magnetite      [2026-10-21] comme --ms, mais le mobile est le VRAI firmware TI :
                        FreeCalypso Magnetite (cible c139, config hybrid-ui-vo, compile
```

```
                    avec le TMS470 sous Wine ; doc/magnetite.md) en flash a la place
                    du layer1 osmocom-bb (c54x_exe/run.sh FIRMWARE_FLASH=...). Pile
                    L23 et interface dans le firmware : pas d'osmocon ni de mobile,
                    rvinterf decode les traces, ecran couleur du C139 dans
                    tools/osmo-c123-gtk.py --modele c139. MAGNETITE_FLASH=<image>
                    (defaut /opt/GSM/firmware/magnetite/c139-flash.img).
                        ./start-native.sh --bts --ms-magnetite   la BTS et le C139 Magnetite, par l'air
                        ./start-native.sh --ms-magnetite         le C139 Magnetite seul, par le pont
--vocoder-rom       [2026-10-22] DEFAUT. Le codec FR de la mask-ROM du DSP vocode (port voix
                    SP0 modelise, CALYPSO_VSP=1, pont/vsp_audio.py) : BANC_VOCODEUR=rom.
                    Le C54x emule doit alors tenir le temps reel AVEC le codec.
--gapk              [2026-10-22] Le vocodeur tourne sur l'hote (osmo-gapk dans le mobile L23) :
                    BANC_VOCODEUR=hote, CALYPSO_VSP=0, TCH montant code par l'hote
                    (PONT_UL_RAW_TCH=0). Le DSP ne fait plus que le canal : il tient le temps
                    reel. Mobile osmocom seulement : le firmware TI (--ms-magnetite) vocode
                    toujours dans la ROM.
--sap, --softsim    SIM via SAP : serveur softSIM (Ruby, /opt/GSM/softsim) sur
                    /tmp/osmocom_sap, lance avant la pile ; le MS#1 passe en
                    « sim sap ». Sert la carte du lecteur PC/SC (HID Omnikey,
                    sysmoISIM : tools/osmo-sim.sh programmer) si une carte y
                    est, sinon la SIM logicielle sim-op1.xml (IMSI du MS#1).
--sap=pcsc          SIM via SAP : la carte du lecteur, obligatoirement
                    (--omnikey, --sim-reader : pareil)
--sap=sim           SIM via SAP : la SIM logicielle, meme avec une carte
--no-sap            pas de SIM via SAP (la SIM de test du mobile), par defaut
--launcher <bin>    lanceur C de QEMU a utiliser (defaut : /usr/local/bin/qosmo).
                    Il remplace l'appel direct a qemu-system-arm : memes
                    defauts, sockets et pty publies (<RUN_DIR>/modem.pty),
                    osmocon direct. Se compile dans
                    /opt/GSM/qosmo/tools/qosmo-launch (make ALIAS=qosmo install).
--list              affiche le plan (delegue a run.sh --list)
--dry-run           deroule sans effet de bord
--profile <nom>     force le profil
--stop              arrete la pile (delegue a run.sh --stop)
--status            interroge l'etat (delegue a run.sh --status)
--force             relance meme les modules deja demarres
--verbose           montre la sortie des modules
--assembly-logs     trace asm de l'ARM emule (= ASSEMBLY_LOGS=1) : QEMU
                    -d in_asm -D /var/tmp/c54x-pont/qemu-asm.log (lien
                    /tmp/c54x-pont/qemu-asm.log ; ASSEMBLY_LOGS_DIR change le
                    dossier, sur le disque : /tmp est un tmpfs de 4,7 Go),
                    chaque bloc une fois, a sa traduction, PLUS l'anneau des
                    blocs executes (plugin TCG qosmo/tools/qosmo-tbring) :
                    64 Gio creux dans le meme dossier (qemu-tbring), le PC de
                    chaque bloc execute, chaines compris, relu a chaud par
                    tbring-dump -n 500 -e <elf> -a qemu-asm.log. Rien de texte
                    dans la boucle : l'ARM garde son rythme. ASSEMBLY_LOGS_RING=0
                    coupe l'anneau. ASSEMBLY_LOGS_FLAGS
                    change les -d, ASSEMBLY_LOGS_FILTRE pose un -dfilter (ex.
                    0x00820000+0x40000). ASSEMBLY_LOGS_FLAGS=in_asm,exec,nochain
                    journalise aussi chaque execution en texte (150 Mo/s,
                    20 Go en 130 s ; QEMU_LOG_BUFFERED=4 Mio vide a chaque
                    arret de la VM) ; avant la correction de cpu_idle_park
                    du 2026-10-19 (qosmo calypso_trx.c) :
                    l'ARM passe de 0,4 a 62 ms par trame, le DSP en pas-a-pas
                    suit, l'horloge du pont (pont/trx.py) descend jusqu'a
                    PONT_HORLOGE_MAX x le temps reel (16 avec ASSEMBLY_LOGS=1,
                    4 sinon) et les IND CLOCK attendent la reprise de la BTS
                    (osmo-bts-trx s'arrete au-dela de 50 trames de pause).
                    Les temporisateurs de mobile (T3210 20 s) et du reseau
                    (T3101) restent muraux : a 14 x, l'attach peut encore
                    buter dessus. Runs du 2026-10-19 02:06 et 02:20.
--icount[=SPEC]     QEMU avec -icount SPEC (defaut : auto ; = QEMU_ICOUNT),
                    en --dsp (c54x_exe) comme en --grgsm (run_modules/40-qemu).
--no-attach         ne s'attache pas a tmux a la fin (= CALYPSO_NO_ATTACH=1 ;
                    c'est ce que pose osmo-banc.service : personne devant)
```

```
  --check-paths       verifie les dependances declarees
  --wan               monte le WAN a N noeuds (1 a 9) AVANT run.sh et demande :
                        nombre de noeuds, IP de chaque noeud, indicatif de
                        chaque noeud, numero du noeud construit par ce lancement
  --wan mynode1.conf [--wan mynode3.conf ...]
                      WAN a partir des FICHES des autres noeuds (ecrites par
                      leur run : /etc/osmocom/mynode<N>.conf). Ce noeud est
                      ajoute a la table avec son adresse locale ; aucune
                      question. Le maillage SIP + SMS suit comme avec --wan.
  --gen-conf [FICHIER] ecrit la fiche de CE noeud (defaut
                      /etc/osmocom/mynode<N>.conf) SANS lancer le banc. Le
                      run l'ecrit de toute facon, juste avant run.sh.
  --wan-nodes "1:IP:IND ..."   meme chose sans question (scriptable)
  --wan-id N          numero du noeud local (sinon deduit des IP locales)
  --wan-conf FICHIER  table a lire/ecrire (defaut /etc/osmo-wan.conf)
  --regen-conf        reconstruit la table WAN DE ZERO (implique --wan) : la
                      table en place est sauvegardee puis effacee, on repart
                      de --wan-nodes / des fiches / de la question. A utiliser
                      quand un pair eteint laisse une route PROHIB au STP.
  --payload-ism[=BACKEND]
                      le TEXTE des SMS entre noeuds part par une liaison RADIO
                      en bande ISM ; tout le reste (signalisation, SS7, SIP)
                      reste sur IP, comme --wan (a combiner avec --wan). Le
                      relais met un jeton dans le TCP et le texte sur la radio.
                      BACKEND = serial|exec|fifo|loopback. Sans BACKEND, lit la
                      section [payload] deja ecrite par network/setup-payload-ism.sh.
                      Peripherique/commandes par l'env : ISM_SERIAL, ISM_TX,
                      ISM_RX, ISM_TX_CMD, ISM_RX_CMD. Voir wiki/Payload-ISM.md.
  --node N            numero de CE noeud, 1 a 9. DEDUIT AUTOMATIQUEMENT s'il
  --menu              demande les choix au lieu de les deviner : profil,
                      noeud, operateur, inter-STP, WAN, regeneration des
                      configs. Les valeurs proposees sont
                      celles que le lanceur aurait prises tout seul, donc
                      valider sans rien changer equivaut a ne pas passer
                      l'option. Sans terminal, l'option est ignoree.
  --regen             regenere les configs depuis les gabarits (par defaut on
                      CONSERVE celles deja en place : run.sh les ecraserait)
                      est omis : environnement, puis /etc/osmo-role, puis la
                      table WAN comparee aux adresses locales. Cette option
                      ne sert qu'a forcer. Reecrit son identite SS7
                      (point codes 1.<noeud><op>.<role>, routing contexts) et
                      pointe son ASP sur l'inter-STP. Une seule ISO suffit
                      alors pour les neuf noeuds - le numero se choisit ici.
  --op N              operateur porte par ce noeud (defaut 1)
  --hub-ip ADRESSE    inter-STP a joindre (defaut : selon docker ou VM ; natif : 192.168.1.49)
  --interstp          lance le inter-STP en standalone sur ce noeud (force)
  --no-interstp       ne lance pas le inter-STP (defaut : auto, avec --wan)
  --air-mesh[=PORT]   maillage de BURSTS entre noeuds : le milieu radio
                      devient commun et un mobile peut entendre - et choisir -
                      la BTS d'un autre noeud. Les pairs viennent de la table
                      WAN, la portee de data/air-mesh.txt. Declare deux
                      mobiles de plus pour s'y brancher (voir pont/airmesh.py).
  --virtualbox[=N]    WAN entre CETTE machine et N-1 VM VirtualBox (implique
                      --wan). A lancer depuis l'hote, pas depuis une VM.
  --vbox-node N       numero de noeud porte par cette machine (defaut 1)
  --mobile            client de couche 2 = mobile (defaut) -- pile L2/L3
                      complete derriere le firmware QEMU
  --ccch_scan         client de couche 2 = ccch_scan -- ecoute le CCCH
  --bcch_scan         client de couche 2 = bcch_scan -- releve les BCCH
  --cell_log          client de couche 2 = cell_log -- journal de cellules
                      Ces quatre-la sont EXCLUSIFS et ECRASENT le mobile de
                      QEMU : ils prennent sa socket L1CTL et sa VTY.
  -h, --help          cette aide

Sans --wan : aucun WAN. Avec --wan le profil reste faketrx-qemu (hybride,
1 faketrx + 1 QEMU Calypso), qui est deja le defaut de ce script.
```

```
  Composer <indicatif><numero> joint ce numero sur le noeud correspondant.
Toute variable CALYPSO_* passee en prefixe est transmise a run.sh / QEMU :
  CALYPSO_LANG=en ./start-native.sh       CALYPSO_NO_ATTACH=1 ./start-native.sh
USAGE
}
while [ $# -gt 0 ]; do
    case "$1" in
        --list)        ACTION=list ;;
        --dry-run)     DRY=1 ;;
        --profile)     if [ $# -lt 2 ] || [ -z "${2:-}" ]; then
                           printf '%s\n\n' "--profile attend un nom de profil" >&2
                           usage >&2; exit 2
                       fi
                       PROFILE="$2"; PROFILE_CHOISI=1; shift ;;
        --stop)        ACTION=stop ;;
        --status)      ACTION=status ;;
        --force)       FORCE=1 ;;
        --verbose)     VERBOSE=1 ;;
        --assembly-logs) ASSEMBLY_LOGS=1 ;;
        --icount)      QEMU_ICOUNT=auto; export QEMU_ICOUNT ;;
        --icount=*)    QEMU_ICOUNT="${1#*=}"; export QEMU_ICOUNT ;;
        # Equivalent of CALYPSO_NO_ATTACH=1, as a command-line option so that
        # the systemd unit can say it on its ExecStart.
        --no-attach)   CALYPSO_NO_ATTACH=1; export CALYPSO_NO_ATTACH ;;
        --check-paths) ACTION=checkpaths ;;
        --wan)         WAN_MESH=1
                       # --wan FILE: the sheet of another node (network/node-conf.sh).
                       if [ -n "${2:-}" ] && [ -f "$2" ]; then WAN_IMPORT_FILES+=("$2"); shift; fi ;;
        --gen-conf)    ACTION=genconf
                       if [ -n "${2:-}" ] && [ "${2#-}" = "$2" ]; then GEN_CONF_OUT="$2"; shift; fi ;;
        --gen-conf=*)  ACTION=genconf; GEN_CONF_OUT="${1#*=}" ;;
        # --dsp changes NEITHER the fork NOR the profile: the stack comes up
        # exactly as in grgsm mode (core, BTS, side-car, tmux), and only the
        # four modules of the Calypso chain (qemu, pty, osmocon, l2) are handed
        # to /opt/GSM/c54x_exe, where the C54x runs outside QEMU on the TI
        # mask-ROM. One single QEMU tree, /opt/GSM/qosmo (run.sh and
        # run_modules are there); the layer 1 is c54x_exe (--dsp) or gr-gsm +
        # grgsm_exe (--grgsm).
        --dsp)         DSP_MODE=1 ;;
        --grgsm)       DSP_MODE=0 ;;
        # --osmo-trx: the DSP bench starts trx.highram.elf and the host
        # application transceiver (see c54x_exe/run.sh, OSMO_TRX=1); implies --dsp.
        --osmo-trx|--bts) DSP_MODE=1; OSMO_TRX=1; export OSMO_TRX; BTS_TRX_CHOISI="$1" ;;
        # --bts-classic: osmo-bts-trx <-> pont_dsp.py, no Calypso TRX (see the help)
        --bts-classic|--bts-classique) DSP_MODE=1; BTS_CLASSIC=1 ;;
        # --ts N: 1 = TS0 only; 2 = TS0 + TS1, one Calypso TRX per timeslot
        # (OSMO_TRX_TS1=1: instance 3 for TS1). Implies --bts. Exported below
        # (export_trx_ts).
        --ts)          if [ $# -lt 2 ] || ! [[ "${2:-}" =~ ^(1|2|2/1)$ ]]; then
                           printf '%s\n\n' "--ts attend 1 (TS0 seul), 2 (TS0 + TS1, un Calypso par intervalle) ou
                            ↪  2/1 (TS0 + TS1, un seul Calypso)" >&2
                           usage >&2; exit 2
                       fi
                       OSMO_TRX_TS="$2"; DSP_MODE=1; OSMO_TRX=1; export OSMO_TRX; BTS_TRX_CHOISI=--ts; shift ;;
        --ts=*)        OSMO_TRX_TS="${1#*=}"
                       [[ "$OSMO_TRX_TS" =~ ^(1|2|2/1)$ ]] || { printf '%s\n\n' "--ts attend 1, 2 ou 2/1 (recu :
                        ↪  $OSMO_TRX_TS)" >&2; usage >&2; exit 2; }
                       DSP_MODE=1; OSMO_TRX=1; export OSMO_TRX; BTS_TRX_CHOISI=--ts ;;
        # --ms: without --bts it is the default (mobile + bridge); with --bts,
        # instance 2 over the air
        --ms)          MS_AIR=1 ;;
        # --ms-magnetite: --ms with the TI firmware (Magnetite) in flash, see the help
        --ms-magnetite) MS_AIR=1; MS_MAGNETITE=1
                       MAGNETITE_FLASH="${MAGNETITE_FLASH:-/opt/GSM/firmware/magnetite/c139-flash.img}"
```

```
                [ -r "$MAGNETITE_FLASH" ] || { printf '%s\n' "--ms-magnetite : image flash absente :
                ↪  $MAGNETITE_FLASH (doc/magnetite.md : build fc-magnetite c139 hybrid-ui-vo)" >&2; exit
                ↪  2; }
                # NO export of MAGNETITE_FLASH here: exported, FIRMWARE_FLASH
                # would also reach instance 1 (the BTS radio), which would
                # boot Magnetite instead of the trx firmware. It is only
                # passed to the mobile (exec below).
                # MS_MAGNETITE IS exported: qosmo/run_modules/08-gabarits.sh
                # regenerates the templates in run.sh (another bash), and
                # without it generate_configs.sh sets SDCCH8 on TS1
                # (__TS1_PCHAN_BTS0__): IMM ASS on TS1, SABM of Magnetite
                # on TS0, LU in Timeout.
                export MS_MAGNETITE
                ;;
# vocoder: the DSP ROM (default, 14-demarrage.sh) or GAPK on the host.
# Exported here: /etc/default/osmo-banc (BANC_VOCODEUR=rom, CALYPSO_VSP=1,
# PONT_UL_RAW_TCH=1) only fills the gaps.
--vocoder-rom|--vocodeur-rom) BANC_VOCODEUR=rom; CALYPSO_VSP=1; export BANC_VOCODEUR CALYPSO_VSP ;;
--gapk|--vocodeur-hote) BANC_VOCODEUR=hote; CALYPSO_VSP=0; PONT_UL_RAW_TCH=0
                export BANC_VOCODEUR CALYPSO_VSP PONT_UL_RAW_TCH ;;
--sap)          SAP_MODE=1; export SAP_MODE ;;
--sap=*)        SAP_MODE=1; SAP_TYPE="${1#*=}"; export SAP_MODE SAP_TYPE ;;
--softsim)      SAP_MODE=1; export SAP_MODE ;;   # alias of --sap
--omnikey|--sim-reader) SAP_MODE=1; SAP_TYPE=pcsc; export SAP_MODE SAP_TYPE ;;
--no-sap)       SAP_MODE=0; export SAP_MODE ;;
--launcher)     QOSMO_LAUNCHER="${2:-}"; export QOSMO_LAUNCHER; shift ;;
--launcher=*)   QOSMO_LAUNCHER="${1#*=}"; export QOSMO_LAUNCHER ;;
--wan=*)        WAN_MESH=1
                if [ -f "${1#*=}" ]; then WAN_IMPORT_FILES+=("${1#*=}"); else WAN_NODES="${1#*=}"; fi ;;
--wan-nodes)    WAN_MESH=1; WAN_NODES="${2:-}"; shift ;;
--wan-nodes=*) WAN_MESH=1; WAN_NODES="${1#*=}" ;;
--wan-id)       WAN_MESH=1; WAN_NODE_ID="${2:-}"; shift ;;
--wan-id=*)     WAN_MESH=1; WAN_NODE_ID="${1#*=}" ;;
--wan-conf)     WAN_CONF_FILE="${2:-}"; shift ;;
--wan-conf=*)   WAN_CONF_FILE="${1#*=}" ;;
--payload-ism)    PAYLOAD_ISM=1; export PAYLOAD_ISM ;;
--payload-ism=*) PAYLOAD_ISM=1; PAYLOAD_ISM_LINK="${1#*=}"; export PAYLOAD_ISM ;;
--virtualbox)    WAN_MESH=1; VBOX_INTERCO=1 ;;
--virtualbox=*) WAN_MESH=1; VBOX_INTERCO=1; VBOX_NODES="${1#*=}" ;;
--vbox-node)     VBOX_HOST_NODE="${2:-1}"; shift ;;
--vbox-node=*)   VBOX_HOST_NODE="${1#*=}" ;;
--node)          NODE_ID="${2:-}"; shift ;;
--node=*)        NODE_ID="${1#*=}" ;;
--air-mesh)      AIR_MESH=1 ;;
--air-mesh=*)    AIR_MESH=1; AIRMESH_PORT="${1#*=}" ;;
--regen)         REGEN_GABARITS=1 ;;
--regen-conf)    WAN_MESH=1; REGEN_WAN_CONF=1 ;;
--menu)          MENU_MODE=1 ;;
--op)            NODE_OP="${2:-1}"; shift ;;
--op=*)          NODE_OP="${1#*=}" ;;
--hub-ip)        HUB_IP="${2:-}"; shift ;;
--hub-ip=*)      HUB_IP="${1#*=}" ;;
--interstp)      INTERSTP=1 ;;
--no-interstp)   INTERSTP=0 ;;
# GPRS MS (osmocom-bb modem) as second MS of the side-car fake_trx, with
# its PCU (/tmp/pcu_bts2): qosmo run_modules/{58,61,69,71}.
# INACTIVE by default: the bench starts without data (full DSP ROM
# alone, voice/SMS). --gprs-modem brings the data set up on demand.
--gprs-modem)    MODEM_GPRS=1 ;;
--no-gprs-modem) MODEM_GPRS=0 ;;
--mobile|--ccch_scan|--bcch_scan|--cell_log)
                # run_modules/70-l2.sh handles the four clients through
                # CALYPSO_L2_CLIENT; this is where one is chosen.
                # Only one at a time: they would fight over the L1CTL
                # socket (/tmp/osmocom_l2) and the VTY 4247, and the
                # second one started would stay mute forever.
```

```
                    _l2c="${1#--}"
                    if [ -n "${L2_CLIENT_CHOISI:-}" ] && \
                       [ "$L2_CLIENT_CHOISI" != "$_l2c" ]; then
                        printf '%s\n\n' \
                          "--$_l2c et --$L2_CLIENT_CHOISI sont exclusifs : un seul client de couche 2" >&2
                        usage >&2; exit 2
                    fi
                    L2_CLIENT_CHOISI="$_l2c" ;;
        -h|--help)  usage; exit 0 ;;
        faketrx-qemu|faketrx|qemu|noproc|core|hybrid)
            PROFILE="$1"; PROFILE_CHOISI=1
            [ "$PROFILE" = "hybrid" ] && PROFILE=faketrx-qemu
            [ "$PROFILE" = "core" ]   && PROFILE=noproc
            ;;
        *) printf 'option inconnue : %s\n\n' "$1" >&2; usage >&2; exit 2 ;;
    esac
    shift
done

# --bts-classic: the BTS is osmo-bts-trx + pont_dsp.py; an OSMO_TRX=1 inherited
# from the environment must not turn the Calypso TRX back on. --ms /
# --ms-magnetite remain valid: without OSMO_TRX, 14-demarrage.sh connects the
# mobile (or Magnetite, FIRMWARE_FLASH) to the bridge.
if [ "${BTS_CLASSIC:-0}" = 1 ]; then
    if [ -n "${BTS_TRX_CHOISI:-}" ]; then
        printf '%s\n\n' "--bts-classic et $BTS_TRX_CHOISI sont exclusifs : BTS classique (pont_dsp.py) ou Calypso
        ↪  TRX" >&2
        usage >&2; exit 2
    fi
    OSMO_TRX=0; export OSMO_TRX
fi

# --ts N -> variables of c54x_exe/run.sh (OSMO_TRX=1). Without --ts, run.sh
# keeps its default (OSMO_TRX_TCH=1: one Calypso, TS0 + TS1, masks 0x03).
# trx_ts_desc: the label of the summaries.
export_trx_ts() {
    # 2/1 and the default: one single Calypso for TS0 + TS1 (the only setup
    # that completes a call); 2: one Calypso per timeslot (LU OK, calls KO: fn
    # of phone 2 not aligned).
    case "${OSMO_TRX_TS:-}" in
        1)      export OSMO_TRX_TCH=0 OSMO_TRX_TS1=0 ;;
        2)      export OSMO_TRX_TCH=0 OSMO_TRX_TS1=1 ;;
        2/1|"") export OSMO_TRX_TCH=1 OSMO_TRX_TS1=0 ;;
    esac
}
trx_ts_desc() {
    if [ "${OSMO_TRX_TS1:-0}" = 1 ]; then
        printf 'TS0 + TS1 par un 2e Calypso (--ts 2 : OSMO_TRX_TS1=1, instance %s, transceiver -2)'
        ↪  "${OSMO_TRX_TS1_INSTANCE:-3}"
    elif [ "${OSMO_TRX_TCH:-0}" = 0 ]; then
        printf 'TS0 seul (--ts 1 : OSMO_TRX_TCH=0, masque 0x01)'
    else
        printf 'TS0 + TS1 par un seul Calypso (OSMO_TRX_TCH=1, masques 0x03)'
    fi
}
export_trx_ts

# The side-car GPRS set (fake_trx --trx, second PCU, trxcon + modem) reads
# MODEM_GPRS in the run.sh environment: exported here, 0 (no data) by default,
# 1 with --gprs-modem. The environment wins (MODEM_GPRS=1 ./start-native.sh).
export MODEM_GPRS="${MODEM_GPRS:-0}"

# --- --dsp keeps the default profile ----------------------------------------
# --dsp does not force the `qemu` profile: the DSP bench brings up the SAME
# stack as the grgsm mode, side-car included, and the hybrid is the default.
# An explicitly named profile always wins.
```

```
# --- where are we running? ---------------------------------------------------
# The question is not cosmetic: the addressing plan DEPENDS on it.
#   docker  → the inter-STP is the osmo-inter-stp container (172.20.0.10), the
#             node already has its IP when the daemons start.
#   VM/ISO  → the inter-STP is a separate machine (192.168.1.49 on the bench,
#             or a host-only 192.168.56.x depending on the setup), and the
#             node address comes from DHCP: it may be missing at launch.
# Mistaking one for the other gives an ASP that never attaches, with a
# "connection refused" towards an address that does not exist here as the
# only trace.
#
# /.dockerenv alone is not enough - it is true in any container. We also
# require /etc/docker-entrypoint-cmd, which scripts/entrypoint.sh drops: this
# pair identifies a container OF THIS REPOSITORY.
detect_runtime_env() {
    if [ -f /.dockerenv ] && [ -f /etc/docker-entrypoint-cmd ]; then
        echo docker; return
    fi
    if [ -f /.dockerenv ] || grep -qa 'docker\|containerd' /proc/1/cgroup 2>/dev/null; then
        echo docker; return
    fi
    # systemd-detect-virt EXITS WITH CODE 1 when it finds nothing. A
    # "$(cmd || echo none)" then produces TWO lines - the program's "none"
    # output plus the fallback - and no pattern matches: a physical machine
    # would be classified "vm". The exit code is ignored and only the first
    # line is kept.
    local v
    v="$(systemd-detect-virt 2>/dev/null | head -1)" || true
    v="${v:-none}"
    case "$v" in
        oracle|kvm|qemu|vmware|microsoft|xen|bochs|parallels) echo vm ;;
        none|"") echo bare ;;
        *) echo vm ;;
    esac
}
RUNTIME_ENV="${RUNTIME_ENV:-$(detect_runtime_env)}"

# --- which node are we? ------------------------------------------------------
# The node number decides the point codes, the routing context and the dial
# prefix. Typing it at every launch is an opportunity for mistakes - and a
# mistake here is invisible: the stack starts, it simply presents the SS7
# address of another node to the hub. So it is deduced, and the origin of the
# answer is SAID.
#
# Order, from the most explicit to the most deduced. The first one wins:
#   1. --node given on the command line
#   2. the environment - this is how start-docker.sh passes it to the container
#   3. /etc/osmo-role, frozen in the image by build-iso.sh --node N
#   4. the WAN table: the address of this node is in it, and we have it
#   5. nothing - the SS7 identity is then NOT touched, rather than invented
# Returns "number|origin". Not two variables: the function is called in a
# command substitution, hence in a SUBSHELL - any variable it set would stay
# there, and the caller would only print "?" afterwards.
detect_node_id() {
    local n

    n="${WAN_NODE_ID:-${OSMO_WAN_NODE:-}}"
    if [[ "$n" =~ ^[1-9]$ ]]; then echo "${n}|environnement"; return 0; fi

    if [ -r /etc/osmo-role ]; then
        n="$(awk -F= '/^OSMO_WAN_NODE=/{print $2}' /etc/osmo-role 2>/dev/null | tr -d ' \r')"
        if [[ "$n" =~ ^[1-9]$ ]]; then echo "${n}|/etc/osmo-role"; return 0; fi
    fi

    # The WAN table and our own addresses: the most reliable source in a VM,
```

```
    # where a single ISO serves the nine nodes and only the IP tells them apart.
    local conf="${WAN_CONF_FILE:-/etc/osmo-wan.conf}"
    if [ -r "$conf" ] && [ -r "$HERE/network/wan-nodes.sh" ]; then
        n="$(
            # Subshell: wan-nodes.sh sets variables and arrays that must not
            # leak into the launcher.
            set +u
            . "$HERE/network/wan-nodes.sh" 2>/dev/null || exit 0
            WAN_NODE_ID=0
            wan_nodes_load "$conf" 2>/dev/null || exit 0
            wan_nodes_detect_self 2>/dev/null || exit 0
            printf '%s' "$WAN_NODE_ID"
        )"
        if [[ "$n" =~ ^[1-9]$ ]]; then echo "${n}|table WAN (IP locale)"; return 0; fi
    fi

    return 1
}
```

2.213 /opt/GSM/osmo-operator/startup_scripts/02-noeud-couleurs.sh

4261 octets, 76 lignes → 76 lignes

```
# 02-noeud-couleurs.sh - part of start.sh (node identity, hub address, colors)
# Sourced by start.sh in the main shell: same order, same variables.
NODE_ID_SRC=""
if [ -n "$NODE_ID" ]; then
    NODE_ID_SRC="option --node"
else
    _detected="$(detect_node_id || true)"
    NODE_ID="${_detected%%|*}"
    [ -n "$NODE_ID" ] && NODE_ID_SRC="${_detected#*|}"
    unset _detected
fi
# The hub address arrives by the SAME path as the node: start-docker.sh sets
# it in the container environment (OSMO_HUB_IP, and INTER_STP_IP for older
# scripts). A container started by docker exec -d has no terminal, so the hub
# question is skipped; without this fallback HUB_IP would stay empty up to the
# configs. The fallback sits here, after the option loop, so that --hub-ip
# keeps the last word.
[ -n "$HUB_IP" ] || HUB_IP="${OSMO_HUB_IP:-${INTER_STP_IP:-}}"
# ── LAST FALLBACK: THE ADDRESS THE ASP ALREADY DIALS ──────────────────────
# Without HUB_IP, network/set-node-id.sh falls back to the default WAN hub,
# 192.168.1.49, computes the source that routes to it and writes it as the
# ASP local-ip. On a single-machine bench, however, the hub lives in a
# container and the ASP dials 127.0.0.1:2908 - the SCTP port the hub publishes
# on the loopback: the source computed for 192.168.1.49 does not lead to the
# hub and the ASP stays ASP_DOWN, with only a timeout in the log.
#
# The config KNOWS where the hub is - it is its remote-ip. It is read rather
# than guessed: source and destination then come from the same place, and
# `ip route get` returns the right local leg.
# --hub-ip and OSMO_HUB_IP keep priority; this fallback only covers silence.
if [ -z "$HUB_IP" ] && [ -r "${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp.cfg" ]; then
    HUB_IP="$(awk '/^[[:space:]]*asp[[:space:]]+asp-to-inter[[:space:]]/{a=1; next}
                   a && /^[[:space:]]*remote-ip[[:space:]]/{print $2; exit}' \
              "${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp.cfg" 2>/dev/null)"
    # A config never regenerated (fresh install: template copied as is) carries
    # the MARKER __INTER_STP_IP__, not an address; set-node-id.sh --hub-ip would
    # refuse it. Only an IPv4 counts.
    [[ "$HUB_IP" =~ ^[0-9]{1,3}(\.[0-9]{1,3}){3}$ ]] || HUB_IP=""
    [ -n "$HUB_IP" ] && echo "  hub        deduit du remote-ip de l ASP : $HUB_IP" >&2
fi
# For the scripts that only know two worlds: docker, or everything else.
```

```
case "$RUNTIME_ENV" in docker) NODE_MODE=docker ;; *) NODE_MODE=native ;; esac

# --- display (same as run.sh) -------------------------------------------------
if [ -t 1 ] && [ -z "${NO_COLOR:-}" ]; then
    TTY=1
    C_OK=$'\033[32m'; C_KO=$'\033[31m'; C_SK=$'\033[33m'
    C_DIM=$'\033[2m'; C_CYAN=$'\033[36m'; C_BOLD=$'\033[1m'; C_Z=$'\033[0m'
else
    TTY=0
    C_OK=""; C_KO=""; C_SK=""; C_DIM=""; C_CYAN=""; C_BOLD=""; C_Z=""
fi
# ── KILL THE BENCH PYTHONS, AND ONLY THEM ──────────────────────────────────
# No killall: only the python3 OF THE BENCH are killed, named one by one by
# their command line:
#   - fake_trx.py and the rest of the osmocom-bb trx_toolkit (virtual radio);
#   - pont/pont.py (the TRX-UDP bridge), pont/airmesh.py, pont/air.py,
#     pont/pont_dsp.py and the VSP audio bridge (pont.vsp_audio);
#   - the inter-operator SMS relay (scripts/smsc-start.sh);
#   - tools/osmo-ril-atmodem.py (the phone's AT modem);
#   - the gdb gateways of the Calypso instances (gdb-telnet.py).
# Any other python3 (desktop, pmbootstrap, a build next door) is left alone.
# CALYPSO_PY_BANC_EXTRA='pattern|pattern' adds patterns (ERE regex) without
# touching the script.
PY_BANC='fake_trx\.py|trx_toolkit/|pont/pont\.py|pont/airmesh\.py|sms-interop|osmo-ril-
↪  atmodem\.py|pont/air\.py|pont/pont_dsp\.py|pont\.vsp_audio|gdb-telnet\.py'
[ -n "${CALYPSO_PY_BANC_EXTRA:-}" ] && PY_BANC="$PY_BANC|$CALYPSO_PY_BANC_EXTRA"
killall_python() {
    local sig="${1:--TERM}" pid tue=0
    for pid in $(pgrep -x python3 2>/dev/null); do
        [ "$pid" = "$$" ] && continue
        tr '\0' ' ' < "/proc/$pid/cmdline" 2>/dev/null | grep -Eq "$PY_BANC" || continue
        kill "$sig" "$pid" 2>/dev/null && tue=1
    done
    [ "$tue" = 1 ]
}
```

2.214 /opt/GSM/osmo-operator/startup_scripts/03-affichage.sh

2378 octets, 52 lignes → 52 lignes

```
# 03-affichage.sh - part of start.sh (display helpers, globals.conf)
# Sourced by start.sh in the main shell: same order, same variables.
say_begin() {
    if [ $TTY -eq 1 ]; then
        printf '[ %s.. %s] %s' "$C_DIM" "$C_Z" "$1"
    else
        printf '[ .. ] %s\n' "$1"
    fi
}
say_end() { # $1=tag $2=color $3=label $4=detail
    if [ $TTY -eq 1 ]; then printf '\r\033[K'; fi
    printf '[%s%s%s] %s' "$2" "$1" "$C_Z" "$3"
    [ -n "${4:-}" ] && printf ' %s(%s)%s' "$C_DIM" "$4" "$C_Z"
    printf '\n'
}
banner() {
    echo -e "${C_CYAN}"
    echo "╔═══════════════════════════════════════════════════════════╗"
    echo "║   Calypso GSM - bootstrap → run.sh                        ║"
    echo "╚═══════════════════════════════════════════════════════════╝"
    echo -e "${C_Z}"
}
# --- 1. configuration ---------------------------------------------------------
# globals.conf - the network settings (MCC/MNC/ARFCN/KI/IMSI/A5...).
```

```
# Generated by ./generate_configs.sh on the host side; here it is only read.
# globals.conf does not use the ":=" idiom but plain assignment
# (`ENCRYPTION="a5 1"`) and is sourced with `set -a`: it OVERWRITES the
# environment (its own header says so: read last, its settings overwrite
# their namesake in the environment).
#
# The command line wins, as everywhere else in this script: the globals.conf
# variables that the caller has ALREADY set and non-empty are noted, the file
# is read, then they are set again. globals.conf stays the reference for
# everything not said; an empty value still means "computed value", so it
# overrides nothing.
if [ -r "$HERE/globals.conf" ]; then
    _gc_forcees=()
    while IFS= read -r _gc_v; do
        [ -n "${!_gc_v:-}" ] && _gc_forcees+=("$_gc_v=${!_gc_v}")
    done < <(sed -n 's/^\([A-Za-z_][A-Za-z0-9_]*\)=.*/\1/p' "$HERE/globals.conf")
    set -a
    . "$HERE/globals.conf"
    for _gc_kv in "${_gc_forcees[@]}"; do
        eval "${_gc_kv%%=*}=\${_gc_kv#*=}"
    done
    set +a
    [ ${#_gc_forcees[@]} -gt 0 ] && \
        printf '  %sglobals.conf%s surcharge par l environnement : %s\n' \
            "${C_DIM:-}" "${C_Z:-}" "${_gc_forcees[*]}"
    unset _gc_v _gc_kv
fi
```

2.215 /opt/GSM/osmo-operator/startup_scripts/04-environnement.sh

4093 octets, 78 lignes → 78 lignes

```
# 04-environnement.sh - part of start.sh (environment, encryption, TRX bridge)
# Sourced by start.sh in the main shell: same order, same variables.
say_begin "Chargement de l'environnement"
if [ -f "$HERE/environment/load.env" ]; then
    set -a; . "$HERE/environment/load.env"; set +a
    say_end " OK " "$C_OK" "Chargement de l'environnement" "environment/load.env"
elif [ -f "$HERE/env/load.env" ]; then
    set -a; . "$HERE/env/load.env"; set +a
    say_end " OK " "$C_OK" "Chargement de l'environnement" "env/load.env"
else
    # Minimal fallback (typical repository paths)
    : "${GSM_ROOT:=/opt/GSM}"
    : "${NITB_TREE:=$HERE}"
    # Same tree selection as environment/paths.env, replicated for the case
    # where load.env is missing (ISO, bare container). One QEMU tree: qosmo;
    # --dsp does not change tree, it hands the layer 1 to c54x_exe.
    : "${CALYPSO_FORK:=qosmo}"
    if [ -x "$HERE/../$CALYPSO_FORK/run.sh" ]; then
        : "${OQC_ROOT:=$(cd "$HERE/../$CALYPSO_FORK" && pwd)}"
    else
        : "${OQC_ROOT:=$GSM_ROOT/$CALYPSO_FORK}"
    fi
    say_end " OK " "$C_OK" "Chargement de l'environnement" "fallback minimal"
fi
: "${RUN_DIR:=/run/osmo-direct}"
# Fallback if load.env is missing. The logs stay under RUN_DIR (tmpfs): the
# tmux scrollback depends on it - cf. environment/paths.env.
: "${LOG_DIR:=$RUN_DIR/logs}"
# A5/1 by default: the encryption validated end to end on the model
# (Calypso ↔ BTS), and what start-docker.sh already imposes in its hand-off to
# this script, so that the direct launch and the start-docker.sh launch use
# the same encryption.
```

```
#
# WARNING: globals.conf is read BEFORE and is authoritative for this variable
# (its header says so). This ":=" is only a fallback when globals.conf is
# missing - the ISO, a bare container. The value that applies in practice
# comes from there.
: "${ENCRYPTION:=a5 1}"
# The default TRX bridge: a single transceiver (pont/pont.py) that presents
# itself to osmo-bts-trx as TRX-UDP, decodes the DL to GSMTAP 4730/4731 and
# encodes the UL. CALYPSO_BRIDGE is not declared in globals.conf (the
# CALYPSO_* "pass through, intact" according to its header): the environment
# really wins.
#   CALYPSO_BRIDGE=none ./start-native.sh   -> no bridge (QEMU + BTS only)
#
# TWO CHAINS, SELECTED BY --dsp / --grgsm. THE DEFAULT IS THE DSP (DSP_MODE=1
# in 01-defauts-options.sh). gr-gsm (qosmo + grgsm_exe) needs --grgsm: the
# layer 1 is then a gr-gsm decoder in QEMU, and that is the stack that goes
# end to end up to the voice call.
#
# `--dsp` does NOT designate a QEMU fork. The C54x runs OUTSIDE QEMU, in
# /opt/GSM/c54x_exe (TI mask-ROM, core emulated by l1-dsp), and QEMU is only
# the ARM, started with CALYPSO_DSP_EXTERN=1 from the /opt/GSM/qosmo tree.
# The five steps are in c54x_exe/run.sh:
#   1. c54x_exe --arm        the DSP, socket /tmp/calypso_dsp.sock + shared
#                            API RAM, DL bursts over UDP 6702
#   2. qemu-system-arm       the ARM + osmocom-bb layer1, gr-gsm layer 1 off
#   3. osmocon               romload on the serial pty, L1CTL relay
#   4. mobile                osmocom-bb layers 2/3
#   5. pont.py               TRXD 5700-5702 facing osmo-bts-trx, bursts to the
#                            DSP, uplink through the /dev/shm side-bands
# What it does: the mobile acquires FB and SB, decodes SI1-4 (lai=001-01-1),
# camps, and performs its location update.
#
# The DSP ROM is in /opt/GSM/c54x_exe/rom (and in /opt/GSM/calypso_dsp.*.bin,
# where c54x_exe reads it by default).
#
# --dsp: NO BRIDGE STARTED BY THIS SCRIPT. c54x_exe/run.sh starts its own
# (step 5) on 5700-5702; a second bridge on the same ports would die on
# "Address already in use" and leave a corpse at every start.
# `none` = nothing to start here. Do NOT use `ipc`: that value means
# "MS#1 through osmo-trx-ms-ipc, QEMU firmware not started".
# A value set by the operator is respected.
#
# The DSP being the default, `none` only applies if the DSP bench is REALLY
# started (DSP_BANC=1, decided right below): a missing c54x_exe falls back to
# gr-gsm, which needs its bridge.
```

2.216 /opt/GSM/osmo-operator/startup_scripts/05-run-sh.sh

4038 octets, 72 lignes → 72 lignes

```
# 05-run-sh.sh - part of start.sh (run.sh resolution, DSP bench, audio)
# Sourced by start.sh in the main shell: same order, same variables.
: "${MS_COUNT:=2}"
: "${HOST_IP:=127.0.0.1}"
mkdir -p "$RUN_DIR" "$LOG_DIR" /root/.osmocom/bb 2>/dev/null || true
# --- 2. path / binary detection ----------------------------------------------
say_begin "Resolution de run.sh"
# Order: explicit RUN_SH > resolved OQC_ROOT > historical path, so that RUN_SH
# can be overridden like everything else ("the command line always wins").
# THE DSP BENCH ADDS TO THE PIPELINE, IT DOES NOT REPLACE IT.
# RUN_SH stays the fork's one and brings everything up as usual (core, BTS,
# side-car and tmux are in the fork's run_modules); only the four modules of
# the Calypso chain that c54x_exe provides are removed from it (qemu, pty,
# osmocon, l2 - steps 2, 3 and 4 of its run.sh). The DSP bench is started
```

```
# right after, with its five steps.
# BANC_DSP=<path> to designate another one; BANC_DSP=none disables it.
DSP_BANC=0
DSP_MODULES_RETIRES="qemu,pty,osmocon,l2"
: "${BANC_DSP:=$GSM_ROOT/c54x_exe/run.sh}"
if [ "${DSP_MODE:-0}" = 1 ] && [ "$BANC_DSP" != none ] && [ -x "$BANC_DSP" ]; then
    DSP_BANC=1
elif [ "${DSP_MODE:-0}" = 1 ] && [ "$BANC_DSP" != none ]; then
    # The default is the DSP, but without its bench: say so, and the gr-gsm
    # chain takes over (CALYPSO_BRIDGE stays `pont`).
    printf '  %s!%s banc DSP absent (%s) - couche 1 gr-gsm a la place\n' "${C_KO:-}" "${C_Z:-}" "$BANC_DSP" >&2
fi
if [ "$DSP_BANC" = 1 ]; then
    : "${CALYPSO_BRIDGE:=none}"
fi
: "${CALYPSO_BRIDGE:=pont}"
: "${RUN_SH:=${OQC_ROOT:+$OQC_ROOT/run.sh}}"
: "${RUN_SH:=$GSM_ROOT/${CALYPSO_FORK:-qosmo}/run.sh}"
if [ ! -x "$RUN_SH" ]; then
    say_end "FAIL" "$C_KO" "Resolution de run.sh" "$RUN_SH introuvable ou non executable"
    printf '       %s→ Verifiez que %s existe et est executable%s\n' "$C_DIM" "$RUN_SH" "$C_Z"
    printf '       %s→ run.sh vit dans /opt/GSM/qosmo (repris de l ancien qosmo-grgsm)%s\n' "$C_DIM" "$C_Z"
    exit 1
fi
say_end " OK " "$C_OK" "Resolution de run.sh" "$RUN_SH"
RUN_ROOT="$(dirname "$RUN_SH")"

# ── THE DSP BENCH: ITS VOCABULARY ──────────────────────────────────────
# c54x_exe/run.sh only takes --stop | --status | --logs | --step N, nothing
# else: passing it --profile makes it exit on "option inconnue".
# The core network, the BTS and the side-car stay brought up by the fork's
# plan; this bench only provides the Calypso chain (its five steps).
# STOPPING DOES NOT REQUIRE --dsp. DSP_BANC is 0 when --dsp is not typed
# again, but c54x_exe, QEMU, osmocon, the mobile and pont.py would still hold
# the ports 5700-5702 and /tmp/calypso_dsp.sock. So the bench is stopped as
# soon as it exists; its --stop does no harm when it is not running.
#
# ⚠ EXPLICIT MODE=dsp ON EVERY CALL. start-native exports its own MODE (the
# profile: "faketrx-qemu"), and c54x_exe/run.sh reads the SAME variable to
# choose dsp|grgsm. Without it, it exits on
#     [run] ECHEC : MODE=faketrx-qemu inconnu (dsp|grgsm)
# and the bench is not stopped - silently, since the caller ignores its
# return code. Two scripts, two meanings for the same name: the bench's one
# is not renamed (run_real.sh and habits use it), the value is set each time.
# PulseAudio lightened at every start: speech-dispatcher stopped, speex
# canceller instead of webrtc, osmo_rec in 8 kHz mono (lib/audio.sh,
# alleger_audio). audio-chain.sh is idempotent; AUDIO_ALLEGER=0 to leave
# everything alone, AUDIO_AEC_METHOD=webrtc to keep the webrtc canceller.
audio_start() {
    [ -f "$HERE/scripts/audio-chain.sh" ] || return 0
    say_begin "Audio (PulseAudio allege)"
    if bash "$HERE/scripts/audio-chain.sh" 5 >/tmp/osmo-audio-chain.log 2>&1; then
        say_end " OK " "$C_OK" "Audio (PulseAudio allege)" "journal /tmp/osmo-audio-chain.log"
    else
        say_end " !! " "$C_KO" "Audio (PulseAudio allege)" "voir /tmp/osmo-audio-chain.log"
    fi
}
```

2.217 /opt/GSM/osmo-operator/startup_scripts/06-banc-dsp-menu.sh

33146 octets, 654 lignes → 654 lignes

```
# 06-banc-dsp-menu.sh - part of start.sh (DSP bench, --menu, mobile configs)
# Sourced by start.sh in the main shell: same order, same variables.
```

```
banc_dsp() {
    MODE=dsp bash "$BANC_DSP" "$@"
}
banc_dsp_arreter() {
    [ -x "$BANC_DSP" ] || return 0
    banc_dsp --stop "$@"
}
# ALL the Calypso instances (INSTANCE=N: /tmp/c54x-pontN, the over-the-air
# mobile = 2, phone 2 of the TRX = 3, trials = 4...). run.sh --stop (instance 1)
# already takes 2 and 3 down; this collects the rest, then the orphans by name
# (c54x_exe, Calypso QEMU, transceiver).
# banc-rssi (instance 4) and Magnetite (/dev/shm/magnetite, QEMU "-M calypso"
# like the bench) are SEPARATE: neither is stopped here. The protected pids
# are read from their pid files; the pkill below go through this filter
# instead of killing everything that matches.
banc_dsp_pids_proteges() {
    cat /tmp/c54x-pont4/*.pid /dev/shm/magnetite/*.pid 2>/dev/null | tr '\n' ' '
}
banc_dsp_tuer_sauf_proteges() {
    local nom p garde
    garde=" $(banc_dsp_pids_proteges) "
    # pgrep -x: process name (comm), not the command line: a shell containing this text is not targeted
    for nom in "$@"; do
        for p in $(pgrep -x -- "$nom" 2>/dev/null); do
            case "$garde" in *" $p "*) ;; *) kill "$p" 2>/dev/null || true ;; esac
        done
    done
}
banc_dsp_arreter_instances() {
    local d n
    for d in /tmp/c54x-pont[0-9]*; do
        [ -d "$d" ] || continue
        n="${d#/tmp/c54x-pont}"
        [ "$n" = 4 ] && continue                    # banc-rssi: stopped by banc-rssi-stop only
        ls "$d"/*.pid >/dev/null 2>&1 || continue
        [ -x "$BANC_DSP" ] && INSTANCE="$n" AIR=1 OSMO_TRX=0 MODE=dsp bash "$BANC_DSP" --stop >/dev/null 2>&1 ||
         ↪  true
        rm -f "$d"/*.pid
    done
    banc_dsp_tuer_sauf_proteges 'c54x_exe --arm' 'qemu-system-arm -M calypso' 'transceiver -l /tmp/osmocom_l2'
    for k in /tmp/calypso_keys[0-9]*.sock /tmp/osmocom_l2_[0-9]*; do
        case "$k" in /tmp/calypso_keys4.sock|/tmp/osmocom_l2_4) continue ;; esac
        rm -f "$k" 2>/dev/null
    done
    return 0
}

# --- 2b. C launcher of QEMU (qosmo) ------------------------------------------
# QEMU is not called directly by 40-qemu.sh: the qosmo tree carries a C
# launcher, tools/qosmo-launch, installed in /usr/local/bin/qosmo. It sets the
# working defaults (calypso machine + DSP ROMs, -cpu arm946, -gdb tcp::1234,
# -serial pty x2, -monitor unix:$RUN_DIR/...), reads l1s/last_rach in the
# ELF, and publishes stable links to the ptys ($RUN_DIR/modem.pty for
# osmocon, $RUN_DIR/irda.pty). Options: --bind, --trx-port, --iq-tee,
# --l1ctl, --monitor, --gdb (see `qosmo --help`).
# The name is passed to run.sh through QOSMO_LAUNCHER; if missing, 40-qemu.sh
# falls back to the historical qemu-system-arm line - said, without blocking.
say_begin "Lanceur QEMU"
: "${QOSMO_LAUNCHER:=/usr/local/bin/${CALYPSO_FORK:-qosmo}}"
export QOSMO_LAUNCHER
if [ -x "$QOSMO_LAUNCHER" ]; then
    say_end " OK " "$C_OK" "Lanceur QEMU" "$QOSMO_LAUNCHER ($("$QOSMO_LAUNCHER" -V 2>/dev/null | head -1))"
elif [ -f "$RUN_ROOT/tools/qosmo-launch/qosmo-launch.c" ] && command -v gcc >/dev/null 2>&1 \
     && make -s -C "$RUN_ROOT/tools/qosmo-launch" install >/dev/null 2>&1 && [ -x "$QOSMO_LAUNCHER" ]; then
    say_end " OK " "$C_OK" "Lanceur QEMU" "$QOSMO_LAUNCHER (compile a l'instant depuis
     ↪  $RUN_ROOT/tools/qosmo-launch)"
```

```
else
    say_end "WARN" "$C_SK" "Lanceur QEMU" "$QOSMO_LAUNCHER absent : 40-qemu.sh appellera qemu-system-arm
    ↪  directement"
    printf '       %s-> make -C %s/tools/qosmo-launch install%s\n' "$C_DIM" "$RUN_ROOT" "$C_Z"
fi
# == --menu: ASK instead of guessing ==========================================
# The launcher resolves the profile, the node, the inter-STP and the WAN on
# its own, which is what an ISO booting with nobody in front needs. In front
# of a bench, the same silent resolution forces one to re-read the script to
# know what was chosen, and to relaunch with the right options otherwise.
#
# This menu CHANGES no rule: it PRE-FILLS every question with the value the
# launcher would have taken alone, and only writes what the operator changes.
# Validating everything untouched gives exactly the default launch.
#
# It steps aside when nobody can answer (no terminal, --dry-run, NO_MENU=1
# set by start-docker.sh in front of a container): a question without a
# reader is a launch that freezes without saying so.
menu_interactif() {
    [ "${MENU_MODE:-0}" -eq 1 ] || return 0
    [ "${NO_MENU:-0}" = "1" ] && return 0
    [ "$DRY" -eq 1 ] && return 0
    [ -t 0 ] || { printf "  %smenu%s       ignore : pas de terminal\n" "${C_DIM:-}" "${C_Z:-}"; return 0; }

    local has_wt=0
    command -v whiptail >/dev/null 2>&1 && has_wt=1

    # One question, one default, one answer. Without whiptail, fall back to
    # read: a minimal ISO does not always have newt.
    _ask() {   # $1 = title  $2 = question  $3 = default
        local rep
        if [ "$has_wt" -eq 1 ]; then
            rep="$(whiptail --title "$1" --inputbox "$2" 11 70 "$3" 3>&1 1>&2 2>&3)" || rep="$3"
        else
            printf "  %s [%s] : " "$2" "$3" >&2
            read -r rep </dev/tty || rep=""
        fi
        printf "%s" "${rep:-$3}"
    }
    _yesno() { # $1 = title  $2 = question  $3 = default (0=yes 1=no)
        if [ "$has_wt" -eq 1 ]; then
            if [ "$3" -eq 0 ]; then whiptail --title "$1" --yesno "$2" 10 70; else
                whiptail --title "$1" --defaultno --yesno "$2" 10 70; fi
            return $?
        fi
        local d="o" rep
        [ "$3" -eq 0 ] || d="n"
        printf "  %s [%s] : " "$2" "$d" >&2
        read -r rep </dev/tty || rep=""
        case "${rep:-$d}" in [oOyY]*) return 0 ;; *) return 1 ;; esac
    }

    # One choice from a CLOSED list. `_ask` is free input: fine for a node
    # number, not for "gr-gsm or DSP", where any keystroke outside the list
    # gives a broken launch. whiptail --radiolist returns the chosen token;
    # without newt, the entries are numbered and a digit is read.
    #   $1 title  $2 question  $3 default  $4.. = token label token label...
    _menu() {
        local titre="$1" q="$2" def="$3"; shift 3
        local rep
        if [ "$has_wt" -eq 1 ]; then
            local args=() j l
            while [ $# -gt 0 ]; do
                j="$1"; l="$2"; shift 2
                if [ "$j" = "$def" ]; then args+=("$j" "$l" ON); else args+=("$j" "$l" OFF); fi
            done
```

```
        rep="$(whiptail --title "$titre" --radiolist "$q" 15 74 4 "${args[@]}" 3>&1 1>&2 2>&3)" || rep="$def"
    else
        local -a jetons=() i=1
        printf "  %s\n" "$q" >&2
        while [ $# -gt 0 ]; do
            jetons+=("$1")
            printf "    %d) %-12s %s\n" "$i" "$1" "$2" >&2
            shift 2; i=$((i+1))
        done
        printf "  choix [%s] : " "$def" >&2
        read -r rep </dev/tty || rep=""
        case "$rep" in
            ''|*[!0-9]*) rep="$def" ;;
            *) rep="${jetons[$((rep-1))]:-$def}" ;;
        esac
    fi
    printf "%s" "${rep:-$def}"
}

printf "  %smenu%s       --menu : les defauts proposes sont ceux du lancement automatique\n" \
    "${C_DIM:-}" "${C_Z:-}"

# ── 1. WHO DOES LAYER 1 ──────────────────────────────────────────
# The first choice, because it drives the second: the DSP has no use for
# a second base station.
_L1="$(_menu "Couche 1" \
    "Qui demodule ?\n\n  DSP     (defaut) le C54x execute la mask-ROM TI et decode lui-meme :\n          le
     ↪  mobile campe (SI1-4) et fait sa mise a jour de localisation.\n  gr-gsm  la pile va jusqu a l appel
     ↪  voix. Le DSP ne tourne pas." \
    "$([ "${DSP_MODE:-0}" = 1 ] && echo c54x_exe || echo grgsm_exe)" \
    c54x_exe  "DSP C54x hors QEMU, mask-ROM TI (defaut)" \
    grgsm_exe "couche 1 gr-gsm dans qosmo (pile complete)")"
case "$_L1" in c54x_exe) DSP_MODE=1 ;; *) DSP_MODE=0 ;; esac

# ── 2. TOPOLOGY: ALONE OR HYBRID ─────────────────────────────────
# Same axis as CALYPSO_PROFILE, said in bench terms:
#   seul     = profile `qemu`         -> one BTS, one QEMU
#   hybride  = profile `faketrx-qemu` -> + fake_trx side-car and an MS#2
# The default follows the choice above: hybrid in gr-gsm, alone in DSP.
_topo_def=hybride
[ "${DSP_MODE:-0}" = 1 ] && _topo_def=seul
[ "$PROFILE" = qemu ] && _topo_def=seul
_topo="$(_menu "Topologie" \
    "Combien de stations de base ?\n\n  seul     une BTS et un QEMU. C est ce qu il faut pour le DSP.\n
     ↪  hybride  + un side-car fake_trx et un second mobile, pour que\n           MS#2 appelle MS#1 sans
     ↪  sortir de la maquette." \
    "$_topo_def" \
    seul    "une BTS, un QEMU" \
    hybride "+ side-car fake_trx et MS#2")"
case "$_topo" in
    seul)    PROFILE=qemu ;;
    hybride) PROFILE=faketrx-qemu ;;
esac
PROFILE_CHOISI=1
unset _topo _topo_def

# The other profiles (faketrx alone, noproc) stay reachable, but from the
# command line: offering them here would weigh down the usual choice.
PROFILE="$(_ask "Profil" \
    "Profil de pile retenu (modifiable) :\n  faketrx-qemu = hybride   qemu = seul\n  faketrx  noproc  aussi
     ↪  acceptes" \
    "$PROFILE")"

# The node is already resolved above (--node, environment, /etc/osmo-role,
# WAN table): THAT result is proposed, not an arbitrary 1.
NODE_ID="$(_ask "Noeud SS7" \
```

```
        "Numero de ce noeud (1-9). Il fixe le point code 1.<noeud><op>.<role>,\nle routing context et les MSISDN
         ↪  <noeud>00<op><rang>." \
        "${NODE_ID:-1}")"
    NODE_OP="$(_ask "Operateur" "Operateur porte par ce noeud (1-9) :" "${NODE_OP:-1}")"
    HUB_IP="$(_ask "Inter-STP" \
        "Adresse du hub SS7 (vide = ne pas toucher au remote-ip en place) :" \
        "${HUB_IP:-$(awk -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}'
         ↪  "${ROLE_FILE:-/etc/osmo-role}" 2>/dev/null)}")"

    _yesno "WAN" "Monter le maillage WAN entre noeuds ?" "$([ "${WAN_MESH:-0}" -eq 1 ] && echo 0 || echo 1)" \
        && WAN_MESH=1 || WAN_MESH=0

    _yesno "Configurations" \
        "Regenerer /etc/osmocom et /etc/asterisk depuis les gabarits ?\n\nLa pile sera ARRETEE, les fichiers
         ↪  sauvegardes, puis reecrits\navec l'identite du noeud ci-dessus." \
        "$([ "${REGEN_GABARITS:-0}" -eq 1 ] && echo 0 || echo 1)" \
        && REGEN_GABARITS=1 || REGEN_GABARITS=0

    printf "  %smenu%s       profil=%s  noeud=%s  op=%s  hub=%s  wan=%s  regen=%s\n" \
        "${C_DIM:-}" "${C_Z:-}" "$PROFILE" "${NODE_ID:-?}" "$NODE_OP" \
        "${HUB_IP:-inchange}" "$WAN_MESH" "$REGEN_GABARITS"
}
menu_interactif

# Profile → mapping to the profiles expected by run.sh
case "$PROFILE" in
    faketrx-qemu|hybrid) CALYPSO_PROFILE=hybrid;   MODE=faketrx-qemu ;;
    faketrx)             CALYPSO_PROFILE=faketrx;  MODE=faketrx ;;
    qemu)                CALYPSO_PROFILE=calypso;  MODE=qemu ;;
    noproc|core)         CALYPSO_PROFILE=core;     MODE=noproc; RUN_NO_PROCESS=1 ;;
    *)                   CALYPSO_PROFILE=hybrid;   MODE=faketrx-qemu; PROFILE=faketrx-qemu ;;
esac
export CALYPSO_PROFILE MODE
export RUN_NO_PROCESS="${RUN_NO_PROCESS:-0}"
export ENCRYPTION MS_COUNT HOST_IP
# RAND set as a prefix (e.g. `RAND=0 ./start-native.sh`) forces the
# authentication challenge, through the getenv("RAND") gate of the osmo-hlr +
# osmo-msc patches. RAND must reach BOTH processes: osmo-hlr draws it in the
# vector (SRES/Kc computed on RAND=0 -> LU accepted) and osmo-msc sends it.
# RAND set on osmo-msc ALONE -> mismatch -> LU reject. RAND absent -> normal
# network. core_svc_start (run.sh) starts these daemons in two ways, both
# are covered:
#   - direct child (setsid, no systemd) -> inherits the env: exported.
#   - systemd service -> `systemctl start` does NOT inherit the env:
#     transient drop-in /run/systemd/system/<unit>.service.d/rand.conf +
#     daemon-reload.
[ -n "${RAND:-}" ] && export RAND
if command -v systemctl >/dev/null 2>&1; then
    _rand_changed=0
    for _u in osmo-hlr osmo-msc; do
        systemctl cat "$_u" >/dev/null 2>&1 || continue
        _dd="/run/systemd/system/$_u.service.d"
        [ -f "$_dd/rand.conf" ] && _rand_changed=1
        if [ -n "${RAND:-}" ]; then
            mkdir -p "$_dd" 2>/dev/null \
                && printf '[Service]\nEnvironment=RAND=%s\n' "$RAND" > "$_dd/rand.conf" \
                && _rand_changed=1
        else
            rm -f "$_dd/rand.conf" 2>/dev/null || true
        fi
    done
    [ "$_rand_changed" = 1 ] && systemctl daemon-reload 2>/dev/null || true
    # bounce only if the forcing changes, and HLR BEFORE MSC so that the MSC
    # fetches vectors at RAND=0 again; a normal run without RAND touches nothing.
    if [ -n "${RAND:-}" ] || [ "$_rand_changed" = 1 ]; then
        for _u in osmo-hlr osmo-msc; do
```

```
            systemctl is-active --quiet "$_u" 2>/dev/null && systemctl restart "$_u" 2>/dev/null || true
        done
    fi
    unset _u _dd _rand_changed
fi
# Layer 2 client: the command-line option OVERRIDES everything, and it is
# EXPORTED - a value set without export does not reach run.sh (same rule as
# ENCRYPTION, cf. generate_configs.sh).
if [ -n "${L2_CLIENT_CHOISI:-}" ]; then
    CALYPSO_L2_CLIENT="$L2_CLIENT_CHOISI"
fi
export CALYPSO_L2_CLIENT="${CALYPSO_L2_CLIENT:-mobile}"
# ── THE BUFFER OF THE `mobile` ALSA-PULSEAUDIO PLUGIN ──────────────────
# The mechanism is the one documented in qosmo/run_modules/70-l2.sh ("MUTE
# DOWNLINK"): the `mobile` writes one GSM frame (160 samples, 20 ms) per
# `snd_pcm_writei`, and if the plugin buffer is too short each write returns
# -EPIPE; pq_alsa.c answers with a silent `snd_pcm_prepare()`, which TEARS
# DOWN AND REBUILDS the PulseAudio stream. As soon as a consumer reads
# gsm_audio.monitor (our loopback to the speakers) the minimum latency of the
# sink rises to 26 ms, and short buffers rebuild the stream constantly
# (writer paced at 20 ms like the mobile, 10 s:
#     80 ms -> 16 rebuilds   120 -> 19   160 -> 1   200 -> 0   240 -> 0
# during a real call that is 50 rebuilds per second: stream destroyed at
# every frame, saturated voice). The value is above the first stable step
# (200 ms) with margin for jitter under load.
# The variable is exported HERE because update.sh re-fetches qosmo: its
# internal default may differ, this one wins (`:=`).
# CHECK, during an established call: the sink-input index of the mobile must
# stay FIXED (`pactl list short sink-inputs`, two readings 1 s apart).
export CALYPSO_PULSE_LATENCY_MSEC="${CALYPSO_PULSE_LATENCY_MSEC:-320}"
export LOG_DIR RUN_DIR
# --- 2c. ARM firmware: FIRMWARE_SRC ------------------------------------------
# Both launchers (qosmo/run.sh through bench.env, c54x_exe/run.sh) read
# FIRMWARE_ELF / FIRMWARE_BIN from the environment. FIRMWARE_SRC is the name
# the packaging scripts use (build-debs.sh, 71-debs-banc.sh); it is accepted
# here too so that a call
#     FIRMWARE_SRC=/path/layer1.xxx.elf ./start-native.sh
# loads that firmware into QEMU. A file = the ELF itself; a directory = the
# firmware/ tree (FIRMWARE_DIR, board/<board>/ below it).
# An explicit FIRMWARE_ELF keeps the upper hand.
if [ -n "${FIRMWARE_SRC:-}" ]; then
    if [ -f "$FIRMWARE_SRC" ]; then
        : "${FIRMWARE_ELF:=$FIRMWARE_SRC}"
    elif [ -d "$FIRMWARE_SRC" ]; then
        : "${FIRMWARE_DIR:=$FIRMWARE_SRC}"
        export FIRMWARE_DIR
    else
        printf '  %sFIRMWARE_SRC introuvable : %s%s\n' "$C_SK" "$FIRMWARE_SRC" "$C_Z" >&2
    fi
fi
if [ -n "${FIRMWARE_ELF:-}" ]; then
    # The .bin next to the ELF if it exists (qosmo --bin, c54x_exe); otherwise
    # the launchers apply their own default.
    [ -n "${FIRMWARE_BIN:-}" ] || [ ! -r "${FIRMWARE_ELF%.elf}.bin" ] || FIRMWARE_BIN="${FIRMWARE_ELF%.elf}.bin"
    export FIRMWARE_ELF FIRMWARE_BIN
fi
# --- 3. validation of the critical paths -------------------------------------
say_begin "Validation des chemins"
path_ok=1
_check() {
    local name="$1" val="${!1:-}"
    if [ -z "$val" ]; then
        printf '\n       %s%s non defini%s\n' "$C_DIM" "$name" "$C_Z"
        path_ok=0
    elif [ -e "$val" ]; then
        :
```

```
    else
        printf '\n       %s%s introuvable : %s%s\n' "$C_DIM" "$name" "$val" "$C_Z"
        path_ok=0
    fi
}
# Optional variables depending on the profile; only block if present and broken
# (DSP_PROM0 not checked: the ROM is not generated by qosmo - cf. run.sh
#  --check-paths and run_modules/00-prereqs.sh.)
for v in QEMU_BIN FIRMWARE_ELF OSMOCON; do
    # `[ -n x ] && _check || true` would swallow the verdict: _check could set
    # path_ok=0 without the loop letting it through. Explicit form.
    if [ -n "${!v:-}" ]; then _check "$v"; fi
done
if [ $path_ok -eq 1 ]; then
    say_end " OK " "$C_OK" "Validation des chemins"
else
    say_end "WARN" "$C_SK" "Validation des chemins" "certains chemins manquent (run.sh verifiera)"
fi
# --- 4. generation of the mobile configs ----------------------------------
# MS#1 (QEMU / main mobile) and MS#2 (faketrx side-car) for the hybrid profile.
BB_DIR="/root/.osmocom/bb"
mkdir -p "$BB_DIR"
# Possible sources for the mobile template
MS_TEMPLATE=""
for t in \
    "${OQC_ROOT:-}/cfgs/mobile_group1.cfg" \
    "$RUN_ROOT/cfgs/mobile_group1.cfg" \
    "$HERE/configs/mobile.cfg.template" \
    "$HERE/configs/mobile.cfg" \
    "$BB_DIR/mobile_group1.cfg" \
    "$BB_DIR/mobile.cfg"
do
    if [ -f "$t" ]; then MS_TEMPLATE="$t"; break; fi
done
# The MS SIM: "sap" for MS#1 when --sap is requested (its socket is the SAP
# server one, SAP_SOCK), "test" (the template's test SIM) otherwise.
ms_sim_mode() {
    if [ "${SAP_MODE:-0}" = 1 ] && [ "$1" = "${SAP_SOCK:-/tmp/osmocom_sap}" ]; then echo sap; else echo test; fi
}
generate_mobile_cfg() {
    local dest="$1" vty_port="$2" l2sock="$3" sapsock="$4" arfcn="$5" imsi="$6" ki="$7"
    # ALSA devices PER MOBILE. The template carries gsm_out/gsm_in for
    # everybody; keeping that would put both mobiles on the same sink and the
    # same source, i.e. the audio loop (cf. /etc/asound.conf).
    local aout="${8:-gsm_out}" ain="${9:-gsm_in}"
    # GUARD: the template list contains $BB_DIR/mobile.cfg, which is also a
    # DESTINATION. If it is the one selected, `sed "$tpl" > "$dest"` with
    # tpl == dest makes the shell truncate the file BEFORE sed reads it: an
    # EMPTY mobile.cfg, and the mobile no longer starts. Switch to the
    # fallback template rather than self-destruct.
    local tpl="$MS_TEMPLATE"
    if [ -n "$tpl" ] && [ "$tpl" -ef "$dest" ] 2>/dev/null; then
        printf '\n       %sMS_TEMPLATE == destination (%s) : template de secours%s\n' \
            "$C_DIM" "$dest" "$C_Z"
        tpl=""
    fi
    if [ -n "$tpl" ]; then
        # the template carries "sim sap": only the MS#1 of --sap keeps it
        # (delimiter #: the | of s|..| would break the \| alternation of sed)
        local _sim_sed=(-e "s#^\([[:space:]]*\)sim \(sap\|test\|reader\)\$#\1sim $(ms_sim_mode "$sapsock")#")
        sed \
            -e "s|bind 127.0.0.1 424[0-9]|bind 127.0.0.1 ${vty_port}|" \
            -e "s|layer2-socket /tmp/osmocom_l2[_0-9]*|layer2-socket ${l2sock}|" \
            -e "s|sap-socket /tmp/osmocom_sap[_0-9]*|sap-socket ${sapsock}|" \
            "${_sim_sed[@]}" \
            -e "s|stick [0-9]*|stick ${arfcn}|" \
```

```
            -e "s|^\([[:space:]]*\)imsi .*|\1imsi ${imsi}|" \
            -e "s|^\([[:space:]]*\)imei .*|\1imei $(rand_imei) 0|" \
            -e "s|^\([[:space:]]*\)ki .*|\1ki comp128 ${ki}|" \
            -e "s|alsa-output-dev .*|alsa-output-dev ${aout}|" \
            -e "s|alsa-input-dev .*|alsa-input-dev ${ain}|" \
            "$tpl" > "$dest"
    else
        # Minimal fallback template
        cat > "$dest" <<EOF
!
! mobile.cfg genere par start-native.sh (secours)
!
log stderr
 logging color 1
 logging print category 1
 logging timestamp 0
!
line vty
 no login
 bind 127.0.0.1 ${vty_port}
!
ms 1
 layer2-socket ${l2sock}
 sap-socket ${sapsock}
 sim $(ms_sim_mode "$sapsock")
 imsi ${imsi}
 ki comp128 ${ki}
 network-selection-mode auto
 stick ${arfcn}
!
EOF
    fi
    _ms_cfg_garantir_tch "$dest" "$aout" "$ain"
    _ms_cfg_garantir_log "$dest"
}

# ── THE tch-voice BLOCK, GUARANTEED, WHATEVER THE TEMPLATE ──────────────
# The template selected by the loop above is the FIRST one found, almost
# always qosmo/cfgs/mobile_group1.cfg - a file from another repository, which
# update.sh re-fetches, and which does not carry "io-tch-format rtp". The
# generation sed only SUBSTITUTES: what the template lacks, the result lacks.
#
# Without that line, the mobile reads and writes its TCH frames in the Texas
# Instruments layout of real Calypsos, while pont.py codes them with
# libosmocoding, i.e. in the RTP/RFC3551 layout (33 bytes opened by the 0xD
# signature). Both ends then misunderstand each other IN BOTH DIRECTIONS, no
# counter flinches, and the sound comes out robotic (cf. the comment in
# configs/mobile.cfg: downlink ring "d2 2c fd a2" against uplink ring
# "83 00 05 e1 ... ff ff ff ff", a frame with fields).
#
# The template is not fixed - it is not ours - the result is guaranteed: the
# four tch-voice settings are set here, and the whole block is created if
# missing (the fallback template above has none).
# MS_TCH_FORMAT for a bench that wants something other than "rtp".
_ms_cfg_garantir_tch() {
    local dest="$1" aout="${2:-gsm_out}" ain="${3:-gsm_in}" fmt="${MS_TCH_FORMAT:-rtp}"
    [ -f "$dest" ] || return 0
    awk -v aout="$aout" -v ain="$ain" -v fmt="$fmt" '
        BEGIN { vu = 0; dedans = 0 }
        # The block exists: our four settings are put at its head, once.
        /^[[:space:]]*tch-voice[[:space:]]*$/ && !vu {
            vu = 1; dedans = 1
            print " tch-voice"
            print "  io-handler gapk"
            print "  io-tch-format " fmt
            print "  alsa-output-dev " aout
```

```
            print "  alsa-input-dev " ain
            next
        }
        # Inside the block: the old copies of THESE four lines are removed,
        # everything else is kept - the other settings as well as the
        # comments, which document the measurements and are worth more
        # than the code.
        dedans {
            if ($0 ~
↪   /^[[:space:]][[:space:]]+(io-handler|io-tch-format|alsa-output-dev|alsa-input-dev)([[:space:]]|$)/) next
            if ($0 ~ /^[[:space:]][[:space:]]+[a-z]/ || $0 ~ /^[!#]/) { print; next }
            dedans = 0
        }
        # No block at all: it is placed right before "no shutdown".
        /^[[:space:]]*no shutdown[[:space:]]*$/ && !vu {
            vu = 1
            print " tch-voice"
            print "  io-handler gapk"
            print "  io-tch-format " fmt
            print "  alsa-output-dev " aout
            print "  alsa-input-dev " ain
        }
        { print }
    ' "$dest" > "${dest}.tch" 2>/dev/null && mv -f "${dest}.tch" "$dest" || rm -f "${dest}.tch"
}
# ── THE log stderr BLOCK, GUARANTEED, SAME SHAPE EVERYWHERE ──────────────
# qosmo/cfgs/mobile_group1.cfg and c54x_exe/mobile_pont.cfg carry a "log
# stderr" (color, timestamp, category); configs/mobile.cfg carries NONE -
# only the "log gsmtap". The file placed in /root/.osmocom/bb would otherwise
# depend on the template selected first, and mobile.log would come out either
# with colors and its 17-digit timestamp (what the osmo-fft-snap.py panel and
# rapport-run.sh can read) or with the libosmocore defaults plus the qosmo -d
# mask. THE block is set here, once, whatever the template: the seven
# categories the bench follows in debug, color per subsystem, compact
# timestamp. BEWARE of the two timestamps of libosmocore 1.12: "logging
# timestamp 1" = ctime ("Mon Oct 19 10:10:03 2026", 25 columns); "logging
# print extended-timestamp 1" = YYYYMMDDhhmmssnnn (17 digits, sortable, and
# the only one tail_lines() of the panel can strip). The second one is set.
# The gsmtap block is not touched.
MS_LOG_STDERR_BLOC=' logging filter all 1
 logging color 1
 logging timestamp 0
 logging print extended-timestamp 1
 logging print category 1
 logging level mm debug
 logging level rr debug
 logging level cc debug
 logging level sms debug
 logging level plmn debug
 logging level sim debug
 logging level sap debug'
_ms_cfg_garantir_log() {
    local dest="$1" existe=0
    [ -f "$dest" ] || return 0
    # Whether the block exists is known BEFORE reading: in mobile_group1.cfg,
    # "line vty" precedes "log stderr", and a single pass would insert a second
    # block before line vty before reaching the first one.
    grep -q '^log stderr[[:space:]]*$' "$dest" && existe=1
    awk -v bloc="$MS_LOG_STDERR_BLOC" -v existe="$existe" '
        BEGIN { vu = 0; dedans = 0 }
        # The block exists: it is replaced entirely, once.
        /^log stderr[[:space:]]*$/ && !vu {
            vu = 1; dedans = 1
            print "log stderr"; print bloc
            next
        }
```

```
        dedans {
            if ($0 ~ /^[[:space:]]+[a-z]/) next     # old line of the block
            dedans = 0                               # "!" or other: end of the block
        }
        # No block: it is placed before the first "log gsmtap", otherwise
        # before "line vty".
        /^(log gsmtap|line vty)([[:space:]]|$)/ && !existe && !vu {
            vu = 1
            print "log stderr"; print bloc; print "!"
        }
        { print }
        END { if (!vu) { print "!"; print "log stderr"; print bloc; print "!" } }
    ' "$dest" > "${dest}.log" 2>/dev/null && mv -f "${dest}.log" "$dest" || rm -f "${dest}.log"
}
# ⚠ THE RUN DOES NOT USE mobile.cfg FOR MS#1. The Calypso mobile runs with
#     mobile -c /root/.osmocom/bb/mobile_group1.cfg
# and mobile_group1.cfg is COPIED as is by qosmo (run_modules/20-mobile-cfg.sh,
# from $QEMU_TREE/cfgs): a static file, never derived from the operator. It
# carries the identity of operator 1 - IMSI 001010001000001, ARFCN 514 -
# whatever the operator; on operator 2 (HLR expects 002010002000001, BTS on
# 516) the mobile would show up with 001010001000001 on 514:
#     Mobile Subscriber of MS '1':
#      IMSI: 001010001000001
#      Status: U2_NOT_UPDATED  IMSI detached  LAI: invalid
#      Registered PLMN: MCC-MNC 001-01
#
# So MS#1 is written to BOTH files. mobile.cfg stays the reference (other
# modules read it), mobile_group1.cfg is the one the launcher opens. Check
# which one is running before trusting a setting set here:
#     pgrep -a mobile
# ── IMSI and Ki of the MS: the HLR plan, not the plan of operator 1 ────────
# On operator 2, whose network broadcasts another MNC, a mobile presenting
# the IMSI of an operator 1 subscriber has no record at the MSC nor the HLR:
# the Location Updating stays unanswered while the whole stack is up.
# The plan is the one of start-docker.sh ("HLR subscribers" section):
#     IMSI = MCC MNC <operator on 4 digits> <MS index on 6 digits>
#     Ki   = 00112233445566778899aabbccdd <MS index> <operator>
# MCC and MNC are re-read from the configuration ACTUALLY deployed - the one
# the BTS broadcasts - and not guessed again from a convention.
MS_OP_ID="${OPERATOR_ID:-1}"
MS_MCC="$(awk '$1=="network" && $2=="country" && $3=="code" {print $4; exit}' /etc/osmocom/osmo-bsc.cfg
 ↪  2>/dev/null || true)"
MS_MNC="$(awk '$1=="mobile" && $2=="network" && $3=="code" {print $4; exit}' /etc/osmocom/osmo-bsc.cfg
 ↪  2>/dev/null || true)"
# ── THE FALLBACKS FOLLOW THE PLAN, THEY DO NOT INVENT IT ───────────────────
# The fallbacks apply when osmo-bsc.cfg is not readable yet (configs not
# regenerated at that moment). The real plan (/var/tmp/osmo-annuaire.csv):
#   op 1 -> 001|01   op 2 -> 001|02   op 3 -> 001|03
# The WHOLE bench is in MCC 001 - the test PLMN, common to all operators -
# and the MNC is what tells them apart. A hard-coded MNC "01" would give
# operator 2 the MNC of operator 1 and ms_imsi() would produce
# 001|01|0002|000001: subscribers that reach the HLR WITHOUT MSISDN nor IMEI
# ("none" / "-------------"), which the VLR cannot attach to anyone.
# On the first launch of a fresh installation, osmo-bsc.cfg is still the
# template: it holds the MARKERS __MCC__ / __MNC__, which would otherwise be
# written as is into the mobile_*.cfg ("imsi __MCC____MNC__0001000002",
# refused by mobile) and exported to run.sh. Only digits count; otherwise,
# the fallback below.
[[ "$MS_MCC" =~ ^[0-9]{3}$ ]] || MS_MCC=""
[[ "$MS_MNC" =~ ^[0-9]{2,3}$ ]] || MS_MNC=""
[ -n "$MS_MCC" ] || MS_MCC="001"
[ -n "$MS_MNC" ] || MS_MNC="$(printf '%02d' "$MS_OP_ID")"
ms_imsi() { printf '%s%s%04d%06d' "$MS_MCC" "$MS_MNC" "$MS_OP_ID" "$1"; }

# ── THE FOURTH PROVISIONER ─────────────────────────────────────────────────
# run.sh embeds its own subscriber module (run_modules/21-abonnes-hlr.sh, in
```

```
# qosmo). It computes its IMSI as MCC+MNC+operator+rank - the same formula as
# ours - but reads MCC and MNC from /etc/osmocom/osmo-msc.cfg through a path
# that does not always succeed; it then falls back to its defaults, 001 and
# 01, and on operator 2 would build 001010002000001: a subscriber nobody
# presents, with an MSISDN of the old plan (20001) on top, which HOLDS that
# number and prevents the right subscriber from getting it.
#
# It honours MCC and MNC from the environment. With them set, its IMSI becomes
# exactly the one start-docker.sh has already provisioned - and its own state
# check (mod_abonnes_hlr_status) finds it present, so it does nothing. The
# duplicate is neutralized by its idempotence, without touching qosmo.
# WITHOUT "${MNC:-...}": 03-affichage.sh has already sourced globals.conf
# (MNC="01"), which would win over the deployed config. MS_MCC/MS_MNC ARE the
# deployed config.
export MCC="$MS_MCC"
export MNC="$MS_MNC"

# ── The radio plan: READ, not recomputed ──────────────────────────────
# generate_configs.sh writes it to /etc/osmocom/radio-plan.env together with
# the configurations. Re-reading it here guarantees that the mobile agrees
# with what the BTS REALLY broadcasts - not with a copied formula that would
# end up diverging. The fallbacks remain, for a bench set up without this file.
if [ -r /etc/osmocom/radio-plan.env ]; then
    # shellcheck disable=SC1091
    . /etc/osmocom/radio-plan.env
    printf '  %splan radio%s  lu dans /etc/osmocom/radio-plan.env\n' "${C_DIM:-}" "${C_Z:-}"
fi

# ── The two radio channels, and which MS attaches to which ────────────
# Hard-coded ARFCNs (514 and 516) are the values of operator 1, and of it
# alone. On operator 2 the main BTS moves to 516; with the side-car staying
# there too, two cells share the same frequency, their bursts mixed in a
# medium where fake_trx pairs by frequency: the mobile sends its RACH, no
# Immediate Assignment comes back.
#
# MS#1 follows the main BTS, whose ARFCN is read from the configuration
# ACTUALLY deployed - the BSC decides it and imposes it to the BTS through
# OML, so that is where the truth is.
MS_ARFCN1="${PLAN_ARFCN:-}"
[ -n "$MS_ARFCN1" ] || MS_ARFCN1="$(awk '/^ bts 0$/{b=1} b && /^ *arfcn /{print $2; exit}'
 ↪  /etc/osmocom/osmo-bsc.cfg 2>/dev/null || true)"
# Same guard as for MCC/MNC: a template not regenerated yet gives __ARFCN__.
[[ "$MS_ARFCN1" =~ ^[0-9]+$ ]] || MS_ARFCN1=""
[ -n "$MS_ARFCN1" ] || MS_ARFCN1=$(( 512 + MS_OP_ID * 2 ))

# MS#2 follows the side-car. That one is decided by US: the qosmo module
# 13-sidecar-cfg.sh reads SC_ARFCN and SC_UNIT_ID from the environment
# (run_modules/_lib/radio.sh: ": "${SC_ARFCN:=516}""). Setting them here
# avoids modifying qosmo, and keeps a single source per value - the same
# formulas as generate_configs.sh.
export SC_ARFCN="${SC_ARFCN:-${PLAN_ARFCN_BTS1:-$(( 612 + MS_OP_ID * 2 ))}}"
export SC_UNIT_ID="${SC_UNIT_ID:-${PLAN_UNIT_ID_BTS1:-$(( 6000 + MS_OP_ID * 10 + 2 ))}}"
MS_ARFCN2="$SC_ARFCN"
ms_ki()   { printf '00 11 22 33 44 55 66 77 88 99 aa bb cc dd %02x %02x' "$1" "$MS_OP_ID"; }

# Random IMEI, Luhn check digit computed - one per MS, never two identical in
# the same trace.
rand_imei() {
    local body d i sum=0 dbl parity
    body="35892500$(printf '%06d' "$(( (RANDOM << 15 | RANDOM) % 1000000 ))")"
    for (( i = ${#body} - 1; i >= 0; i-- )); do
        d=${body:i:1}
        parity=$(( (${#body} - 1 - i) % 2 ))
        if [ "$parity" -eq 0 ]; then
            dbl=$(( d * 2 )); [ "$dbl" -gt 9 ] && dbl=$(( dbl - 9 ))
            sum=$(( sum + dbl ))
```

```
        else
            sum=$(( sum + d ))
        fi
    done
    printf '%s%d' "$body" "$(( (10 - sum % 10) % 10 ))"
}
```

2.218 /opt/GSM/osmo-operator/startup_scripts/07-plan-mobile.sh

474 octets, 7 lignes → 7 lignes

```
# 07-plan-mobile.sh - part of start.sh (mobile plan summary)
# Sourced by start.sh in the main shell: same order, same variables.
printf '  %sPLMN MS%s    MCC %s  MNC %s  operateur %s  ->  IMSI %s / %s\n' \
    "${C_DIM:-}" "${C_Z:-}" "$MS_MCC" "$MS_MNC" "$MS_OP_ID" "$(ms_imsi 1)" "$(ms_imsi 2)"
printf '  %sradio%s      MS#1 sur ARFCN %s (BTS principale)   MS#2 sur ARFCN %s (side-car, unit-id %s)\n' \
    "${C_DIM:-}" "${C_Z:-}" "$MS_ARFCN1" "$MS_ARFCN2" "$SC_UNIT_ID"
```

2.219 /opt/GSM/osmo-operator/startup_scripts/08-mobile-ms1.sh

1234 octets, 24 lignes → 24 lignes

```
# 08-mobile-ms1.sh - part of start.sh (mobile.cfg of MS#1)
# Sourced by start.sh in the main shell: same order, same variables.
say_begin "Generation mobile MS#1"
MS1_CFG="${MOBILE_CFG_MS1_PATH:-$BB_DIR/mobile.cfg}"
generate_mobile_cfg "$MS1_CFG" \
    4247 \
    /tmp/osmocom_l2 \
    "${SAP_SOCK:-/tmp/osmocom_sap}" \
    "$MS_ARFCN1" \
    "$(ms_imsi 1)" \
    "$(ms_ki 1)"
# The copy the launcher really opens. It is REGENERATED rather than copied:
# generate_mobile_cfg also sets the VTY port, the sockets and the ARFCN, and a
# plain cp would propagate the static qosmo file.
# ── AND IT IS PROTECTED FROM BEING OVERWRITTEN ──────────────────────────
# Writing this file is not enough: run.sh REWRITES it right after. Its module
# 20-mobile-cfg.sh copies MOBILE_CFG_SRC over it, and that path defaults to
# $QEMU_TREE/cfgs/mobile_group1.cfg - the static qosmo file, frozen on
# operator 1 (imsi=001010001000001 arfcn=514), which the mobile would open.
# So OUR file is designated as the source: the module then copies the right
# content, and its idempotence check (cmp -s src dest) passes since both are
# identical. No change to qosmo.
export MOBILE_CFG_SRC="$MS1_CFG"
```

2.220 /opt/GSM/osmo-operator/startup_scripts/09-mobile-cfg-tmux.sh

18125 octets, 326 lignes → 326 lignes

```
# 09-mobile-cfg-tmux.sh - part of start.sh (mobile configs, TRX bridge,
# air mesh, summary, run.sh arguments, tmux session purge)
# Sourced by start.sh in the main shell: same order, same variables.
MS1_GROUP_CFG="$BB_DIR/mobile_group1.cfg"
if [ "$MS1_GROUP_CFG" != "$MS1_CFG" ]; then
    generate_mobile_cfg "$MS1_GROUP_CFG" \
        4247 \
        /tmp/osmocom_l2 \
        "${SAP_SOCK:-/tmp/osmocom_sap}" \
```

```
        "$MS_ARFCN1" \
        "$(ms_imsi 1)" \
        "$(ms_ki 1)"
    printf '  %sMS#1%s       aussi ecrit dans %s (le fichier que run.sh ouvre)\n' \
        "${C_DIM:-}" "${C_Z:-}" "$MS1_GROUP_CFG"
fi
say_end " OK " "$C_OK" "Generation mobile MS#1" "$MS1_CFG"
say_begin "Generation mobile MS#2 (faketrx)"
MS2_CFG="$BB_DIR/mobile_faketrx_bts1.cfg"
generate_mobile_cfg "$MS2_CFG" \
    4248 \
    /tmp/ms2_l2 \
    /tmp/ms2_sap \
    "$MS_ARFCN2" \
    "$(ms_imsi 2)" \
    "$(ms_ki 2)" \
    gsm_out gsm_in
say_end " OK " "$C_OK" "Generation mobile MS#2 (faketrx)" "$MS2_CFG"
# ── THE EMULATED SIM READS THE SAME FILE AS THE MOBILE ──────────────────
# CALYPSO_SIM_CFG is the ONLY environment variable the QEMU model reads
# (hw/arm/calypso/calypso_sim.c): it takes the "test-sim" section - imsi and
# ki comp128 - in mobile.cfg format, i.e. exactly the MS#1 file generated
# above. Without it, the emulated SIM keeps its default IMSI
# (001010000000001), unknown to the HLR provisioned with the plan.
# (The MS#2 side-car reads SC_MOBILE_CFG / the file itself.)
export CALYPSO_SIM_CFG="$MS1_CFG"

# --- TRX BRIDGE mode (CALYPSO_BRIDGE=pont): the in-house bridge is the transceiver
# The bridge (pont/pont.py) presents itself as a TRX-UDP transceiver to
# osmo-bts-trx (5700/5701/5702), decodes the DL bursts into L2 -> GSMTAP
# 4730/4731 towards the QEMU model, and encodes the UL from the sidebands
# /dev/shm/calypso_* -> TRXD -> BTS. It NEEDS osmo-bts-trx on the other side:
# qosmo/60-bts.sh launches it, and the barrier below waits for it before
# launching the bridge.
if [ "${CALYPSO_BRIDGE:-}" = pont ]; then
    export CALYPSO_BRIDGE
    # ── THE BRIDGE COMES FROM THE REPO ──────────────────────────────────
    # $HERE is the directory of THIS script: the bridge always travels with
    # it, never a copy outside the repo (which would drift silently from
    # pont/pont.py). PONT_PY remains available to try a variant, as an
    # explicit choice.
    # grgsm setup: its bridge is pont.py (pont_dsp.py is the DSP setup's,
    # launched by c54x_exe/run.sh). See pont/pont.py.
    _PONT="${PONT_PY:-$HERE/pont/pont.py}"

    # ── THE BRIDGE MUST KNOW WHICH CELL IT WORKS ON ─────────────────────
    # pont.py does the channel coding/decoding between the QEMU Calypso and
    # the BTS. It reads PONT_ARFCN and PONT_BSIC from its environment.
    #
    # The BSIC is not decorative: its low part (BCC) IS the training sequence
    # of the dedicated channels. With a wrong BSIC the correlation fails and
    # the blocks do not decode: the mobile sends its RACH, the network
    # answers, and the Immediate Assignment is never handed to the Calypso.
    #
    # They are taken from the radio plan published by generate_configs.sh,
    # with the same fallbacks as the mobiles.
    export PONT_ARFCN="${PONT_ARFCN:-${PLAN_ARFCN:-$MS_ARFCN1}}"
    export PONT_BSIC="${PONT_BSIC:-${PLAN_BSIC:-$(( MS_OP_ID * 7 % 64 ))}}"
    printf '  %spont TRX%s : cellule ARFCN %s BSIC %s\n' \
        "${C_DIM:-}" "${C_Z:-}" "$PONT_ARFCN" "$PONT_BSIC"
    # ARM THE LAUNCHER ONLY ON A REAL START.
    # This block runs BEFORE the `case "$ACTION"` below. On `--stop` (as on
    # --list/--status/--check-paths) the deferred launcher must not be armed:
    # `run.sh --stop` would kill it at once. A bridge ALREADY ALIVE is
    # stopped by the teardown (`f:pont/pont.py` in _td_patterns) and by
    # mod_teardown_stop.
```

```
    if [ "$ACTION" != start ]; then
        printf '  %spont TRX%s : non arme (action=%s, pas un demarrage)\n' \
               "${C_DIM:-}" "${C_Z:-}" "$ACTION"
    elif [ -r "$_PONT" ]; then
        # SINGLETON: kill a previous bridge, otherwise it holds 5700-5702 and
        # the run.sh teardown fails ("restes du run precedent : port:5700...").
        pkill -f "$_PONT" 2>/dev/null || true
        # KILLALL of the bench's python3.
        # The named pattern above misses any bridge launched under another
        # path (PONT_PY override, a .bak backup), which would make the
        # teardown fail on "port:5700 port:5701 port:5702".
        # WHAT IT TAKES DOWN, besides the bridge: fake_trx.py (the MS#2
        # transceiver), sms-interop-relay.py (inter-operator SMS routing,
        # OUTSIDE the run.sh stack so not relaunched automatically),
        # gsm_sniff.py and the tmux pane scripts. Acceptable HERE because this
        # is the path of a START (ACTION=start): the run.sh teardown stops
        # everything two seconds later anyway. Do NOT repeat this in a run.sh
        # module, where it would kill the timestamp filter of the current run.
        # CALYPSO_NO_KILLALL=1 to skip it.
        if [ "${CALYPSO_NO_KILLALL:-0}" != "1" ]; then
            printf '  %spont TRX%s : python3 du banc termines (fake_trx, pont, airmesh, relais SMS, modem AT)\n'
             ↪  "${C_DIM:-}" "${C_Z:-}"
            killall_python -9 || true
        fi
        sleep 1
        printf '  %spont TRX%s : transceiver TRX-UDP d osmo-bts-trx (pont/pont.py)\n' "${C_DIM:-}" "${C_Z:-}"
        # DEFERRED LAUNCH: run.sh first does its teardown (which checks that
        # 5700-5702 are FREE). So bind only afterwards, otherwise we block
        # ourselves.
        # LOG OUTSIDE LOG_DIR: run.sh archives/erases LOG_DIR during its
        # teardown; a pont.log opened before would end up on a DELETED inode
        # (invisible output, "(deleted)" in /proc/<pid>/fd/1). /dev/shm
        # survives.
        # setsid: the bridge must survive the end of the start-native shell.
        # The barrier waits for an EVENT, not a duration (the teardown time
        # depends on the size of the archived logs): osmo-bts-trx, the
        # bridge's TRX-UDP peer, only starts AFTER the teardown, so its
        # presence proves the ports are ours.
        # It waits for BOTH edges: the disappearance (the teardown happened -
        # without --stop the osmo-bts-trx of the PREVIOUS run is still there
        # and pgrep would succeed at once), then the appearance (the stack is
        # back up). With --stop the first edge is already crossed.
        ( _n=0
          while [ "$_n" -lt 180 ] && pgrep -x osmo-bts-trx >/dev/null 2>&1; do
              _n=$((_n + 1)); sleep 1
          done
          _dispar=$_n
          while [ "$_n" -lt 180 ] && ! pgrep -x osmo-bts-trx >/dev/null 2>&1; do
              _n=$((_n + 1)); sleep 1
          done
          if [ "$_n" -ge 180 ]; then
              echo "[pont] osmo-bts-trx jamais (re)vu apres 180 s - demarrage quand meme"
          else
              echo "[pont] osmo-bts-trx parti a ${_dispar}s puis revu a ${_n}s - la pile est debout, on binde"
          fi
          sleep 1
          pkill -f "$_PONT" 2>/dev/null; sleep 1
          exec setsid python3 -u "$_PONT" ) >/dev/shm/pont.log 2>&1 &
        disown $! 2>/dev/null || true
        printf '  %spont: journal%s /dev/shm/pont.log (hors LOG_DIR, efface par le teardown)\n' "${C_DIM:-}"
         ↪  "${C_Z:-}"
    else
        printf '  pont introuvable (%s)\n' "$_PONT"
    fi
fi
```

```
# --- BURST MESH (--air-mesh): the radio medium becomes shared ---------------
# Two nodes otherwise talk at the top of the stack - a SIP trunk, an SMS
# relay, M3UA towards a hub - each keeping its radio sealed: the mobile of
# operator 1 CANNOT hear the BTS of operator 2. Here the burst crosses:
# pont/airmesh.py takes what passes on the local fake_trx and exchanges it
# with the peers. See the airmesh.py header for details - notably why TWO
# transceiver slots are needed, and how they are obtained without modifying
# a line of qosmo.
if [ "${AIR_MESH:-0}" = "1" ]; then
    _AIRMESH="${AIRMESH_PY:-${HERE}/pont/airmesh.py}"
    [ -f "$_AIRMESH" ] || _AIRMESH=/opt/GSM/osmo-operator/pont/airmesh.py
    if [ ! -f "$_AIRMESH" ]; then
        printf '  %sair-mesh%s : airmesh.py introuvable - maillage de bursts non lance\n' "${C_KO:-}" "${C_Z:-}"
    else
        # TWO MORE MOBILES THAN THE BENCH HAS. fake_trx creates one
        # transceiver per declared mobile; airmesh takes the last two and
        # tunes them differently (one as a mobile, the other as a BTS).
        # Without this +2 there is no free slot and airmesh stays silent.
        _NMS_REEL="${N_MS:-2}"
        N_MS=$(( _NMS_REEL + 2 )); export N_MS
        _BBP="${BB_BASE_PORT:-6700}"; _BBS="${BB_PORT_STEP:-3}"
        _TRX_MS=$((  _BBP + (_NMS_REEL + 1 - 1) * _BBS ))   # slot N+1
        _TRX_BTS=$(( _BBP + (_NMS_REEL + 2 - 1) * _BBS ))   # slot N+2

        # The peers come from the WAN table: every node that is not us.
        _PAIRS=""
        for _tok in ${WAN_NODES:-}; do
            _pid="${_tok%%:*}"; _rest="${_tok#*:}"; _pip="${_rest%%:*}"
            [ -n "$_pid" ] && [ -n "$_pip" ] || continue
            [ "$_pid" = "${NODE_ID:-1}" ] && continue
            _PAIRS="$_PAIRS --pair ${_pid}:${_pip}:${AIRMESH_PORT:-4739}"
        done

        printf '  %sair-mesh%s   noeud %s  arfcn %s  emplacements TRXD %s et %s  pairs:%s\n' \
            "${C_DIM:-}" "${C_Z:-}" "${NODE_ID:-1}" "${ARFCN:-1}" \
            "$_TRX_MS" "$_TRX_BTS" "${_PAIRS:- aucun}"

        # Wait until fake_trx has BOUND the slot, not for a duration: it only
        # starts after the run.sh teardown, whose length depends on the size
        # of the archived logs. A fixed delay would make this launch a race -
        # same barrier as the bridge, just above.
        ( _n=0
          while [ "$_n" -lt 180 ] && ! ss -uln 2>/dev/null | grep -q ":$(( _TRX_MS + 1 ))\b"; do
              _n=$((_n + 1)); sleep 1
          done
          if [ "$_n" -ge 180 ]; then
              echo "[airmesh] emplacement $(( _TRX_MS + 1 )) jamais ouvert par fake_trx apres 180 s"
              echo "[airmesh] verifiez que le banc tourne bien avec N_MS=${N_MS} (deux de plus que ${_NMS_REEL})"
              exit 1
          fi
          echo "[airmesh] fake_trx a ouvert nos emplacements apres ${_n} s"
          sleep 1
          # shellcheck disable=SC2086
          exec setsid python3 -u "$_AIRMESH" \
              --node "${NODE_ID:-1}" --arfcn "${ARFCN:-1}" \
              --port "${AIRMESH_PORT:-4739}" \
              --trx-ms "$_TRX_MS" --trx-bts "$_TRX_BTS" \
              --topologie "${HERE}/data/air-mesh.txt" \
              $_PAIRS ) >/dev/shm/airmesh.log 2>&1 &
        printf '  %sair-mesh: journal%s /dev/shm/airmesh.log\n' "${C_DIM:-}" "${C_Z:-}"
    fi
fi

# --- 6. summary -------------------------------------------------------------
banner
printf '  %sprofil%s     %s (%s)\n' "$C_DIM" "$C_Z" "$CALYPSO_PROFILE" "$MODE"
```

```
printf '  %senviron.%s   %s\n' "$C_DIM" "$C_Z" \
    "$(case "$RUNTIME_ENV" in docker) echo "conteneur docker" ;; vm) echo "machine virtuelle" ;; *) echo "machine
    ↪  physique" ;; esac)"
if [ -n "$NODE_ID" ]; then
    printf '  %snoeud%s      %s  %s(%s)%s\n' "$C_DIM" "$C_Z" "$NODE_ID" "$C_DIM" "${NODE_ID_SRC:-?}" "$C_Z"
else
    printf '  %snoeud%s      %s\n' "$C_DIM" "$C_Z" "aucun - identite SS7 inchangee"
fi
printf '  %srun.sh%s     %s\n' "$C_DIM" "$C_Z" "$RUN_SH"
[ -z "${FIRMWARE_ELF:-}" ] || printf '  %sfirmware%s   %s%s\n' "$C_DIM" "$C_Z" "$FIRMWARE_ELF" \
    "${FIRMWARE_SRC:+ (FIRMWARE_SRC)}"
# IN --dsp, MS#1 IS NOT THE GENERATED FILE.
# The fork's `l2` module is removed from the plan: /root/.osmocom/bb/mobile.cfg
# is opened by nobody, and the running mobile is the DSP bench's
# (mobile_pont.cfg, VTY 4347). The summary names that file and that port.
if [ "$DSP_BANC" = 1 ] && [ "${OSMO_TRX:-0}" = 1 ]; then
    printf '  %sMS#1%s       non lance : le Calypso de QEMU est le TRX de la BTS#0 (--osmo-trx : trx.highram.elf +
    ↪  transceiver, %s)\n' \
        "$C_DIM" "$C_Z" "$(trx_ts_desc)"
elif [ "$DSP_BANC" = 1 ]; then
    _ms1_cfg="${MOBILE_CFG:-$(dirname "$BANC_DSP")/mobile_pont.cfg}"
    _ms1_vty="$(sed -n 's/^ *bind 127.0.0.1 \([0-9]*\).*/\1/p' "$_ms1_cfg" 2>/dev/null | head -1)"
    printf '  %sMS#1%s       %s  IMSI %s  ARFCN %s  VTY %s  %s(banc DSP)%s\n' \
        "$C_DIM" "$C_Z" "$_ms1_cfg" "$(ms_imsi 1)" "$MS_ARFCN1" "${_ms1_vty:-?}" "$C_DIM" "$C_Z"
    printf '  %s           %s non lance (module l2 retire du plan)%s\n' \
        "$C_DIM" "$MS1_CFG" "$C_Z"
else
    printf '  %sMS#1%s       %s  IMSI %s  ARFCN %s  VTY 4247\n' "$C_DIM" "$C_Z" "$MS1_CFG" "$(ms_imsi 1)"
    ↪  "$MS_ARFCN1"
fi
# THE MOBILE VTY PORT IS PUBLISHED, NOT GUESSED. In --dsp the mobile listens
# on 4347 (mobile_pont.cfg), otherwise on 4247 - and everything that opens
# "the mobile console" (launch.sh --vty, the osmo-panel.py inset, the
# osmo-egprs-web dashboard for the Kc and the bb1 VTY) reads it from
# /run/osmo-banc/ms1-vty (session tools) or from the systemd environment
# (OSMO_BB1_VTY_PORT, read by server.js when the dashboard restarts, below).
# Erased at stop.
MS1_VTY_PORT="${_ms1_vty:-4247}"
# THE MOBILE LOG IS PUBLISHED THE SAME WAY. It lives under LOG_DIR (qosmo,
# run_modules/70-l2.sh: "mobile ... >>LOG_DIR/mobile.log"), and in --dsp under
# the c54x_exe RUNDIR (run.sh: RUNDIR/mobile.log, /tmp/c54x-pont by default).
# LOG_DIR follows the launching uid (XDG_RUNTIME_DIR), so the real path is
# written to /run/osmo-banc/mobile-log and pushed into the systemd environment
# (OSMO_MOBILE_LOG) for the osmo-fft-snap.py inset. Erased at stop, with
# ms1-vty.
# --bts --ms / --ms-magnetite: the phone is INSTANCE 2 of run.sh
# (14-demarrage.sh, INSTANCE=2 AIR=1), its RUNDIR is /tmp/c54x-pont2;
# /tmp/c54x-pont is the TRX's, without mobile.
_ms1_rundir="${RUNDIR:-/tmp/c54x-pont}"
[ "${OSMO_TRX:-0}" = 1 ] && [ "${MS_AIR:-0}" = 1 ] && _ms1_rundir="${RUNDIR:-/tmp/c54x-pont2}"
MS1_LOG_NOM="mobile.log"
if [ "$DSP_BANC" = 1 ] && [ -n "${MS_MAGNETITE:-}" ]; then
    # Magnetite: no L23 mobile, the phone log is the L1/TI stack trace of rvinterf
    MS1_LOG="$_ms1_rundir/rvinterf.log"
    MS1_LOG_NOM="rvinterf"
elif [ "$DSP_BANC" = 1 ]; then
    MS1_LOG="$_ms1_rundir/mobile.log"
else
    MS1_LOG="$LOG_DIR/mobile.log"
fi
# Exported: the tmux ms1 window (qosmo tmux_modules/bench.sh) follows the
# phone log (rvinterf with Magnetite) instead of an empty mobile.log.
export MS1_LOG MS1_LOG_NOM
if [ "$ACTION" = "start" ] && [ "$DRY" -ne 1 ]; then
    mkdir -p /run/osmo-banc 2>/dev/null && chmod 755 /run/osmo-banc 2>/dev/null
```

```
    printf '%s\n' "$MS1_VTY_PORT" > /run/osmo-banc/ms1-vty 2>/dev/null && chmod 644 /run/osmo-banc/ms1-vty
     ↪  2>/dev/null
    printf '%s\n' "$MS1_LOG" > /run/osmo-banc/mobile-log 2>/dev/null && chmod 644 /run/osmo-banc/mobile-log
     ↪  2>/dev/null
    command -v systemctl >/dev/null 2>&1 && systemctl set-environment "OSMO_BB1_VTY_PORT=$MS1_VTY_PORT"
     ↪  "OSMO_MOBILE_LOG=$MS1_LOG" 2>/dev/null || true
fi
printf '  %sMS#2%s       %s  IMSI %s  ARFCN %s  VTY 4248\n' "$C_DIM" "$C_Z" "$MS2_CFG" "$(ms_imsi 2)" "$MS_ARFCN2"
printf '  %sjournaux%s   %s\n' "$C_DIM" "$C_Z" "$LOG_DIR"
printf '  %s%-10s%s %s\n' "$C_DIM" "$MS1_LOG_NOM" "$C_Z" "$MS1_LOG"
printf '  %schiffrement%s %s\n' "$C_DIM" "$C_Z" "$ENCRYPTION"
printf '\n'
# --- 7. actions delegated to run.sh ------------------------------------------
RUN_ARGS=()
[ "$DRY" -eq 1 ]     && RUN_ARGS+=(--dry-run)
[ "$FORCE" -eq 1 ]   && RUN_ARGS+=(--force)
[ "$VERBOSE" -eq 1 ] && RUN_ARGS+=(--verbose)
RUN_ARGS+=(--profile "$CALYPSO_PROFILE")
# --restart goes into RUN_ARGS, where it is visible in --dry-run and in the
# line printed just before the exec.
# CALYPSO_NO_RESTART=1 to start without resetting.
[ "${CALYPSO_NO_RESTART:-0}" = "1" ] || RUN_ARGS+=(--restart)
# run.sh already reads CALYPSO_NO_ATTACH (run.sh:8), but a variable is not
# visible in --dry-run nor in the line printed just before the exec: the
# option carries the same setting where one looks for it.
[ "${CALYPSO_NO_ATTACH:-0}" = "1" ] && RUN_ARGS+=(--no-attach)
# --icount and --assembly-logs towards run.sh, hence 40-qemu.sh (--grgsm
# path; in --dsp the qemu module is removed, c54x_exe/run.sh reads
# QEMU_ICOUNT and ASSEMBLY_LOGS itself). Same -d / -dfilter as the DSP bench.
case "${QEMU_ICOUNT:-}" in ''|0|off) ;; *) RUN_ARGS+=("--icount=$QEMU_ICOUNT") ;; esac
if [ "${ASSEMBLY_LOGS:-0}" = 1 ]; then
    RUN_ARGS+=(--asm)
    # in_asm alone by default: exec,nochain (one line per executed block)
    # makes the ARM about 150 times slower and breaks the attach, see the
    # --assembly-logs help.
    export QEMU_ASM_FLAGS="${ASSEMBLY_LOGS_FLAGS:-in_asm}"
    [ -n "${ASSEMBLY_LOGS_FILTRE:-}" ] && export QEMU_ASM_FILTRE="$ASSEMBLY_LOGS_FILTRE"
fi
# ── The tmux sessions of a previous run ──────────────────────────────────
# The run.sh teardown stops the processes but leaves the tmux session up. At
# the next start, its pane module RECREATES its windows in the surviving
# session: there are then TWO windows named "all", and tmux can no longer
# resolve the target (`tmux split-window -t calypso:all` -> "can't find
# window: all"); the splits fail and the barrier reports missing panes.
#
# So the sessions of the previous run are erased with it. Three conventions
# coexist depending on the launcher, all are covered; none needs to exist.
purge_sessions_tmux() {
    local n=0 s
    for s in calypso gapk osmo; do
        if tmux has-session -t "$s" 2>/dev/null; then
            tmux kill-session -t "$s" 2>/dev/null && n=$((n + 1))
        fi
    done
    if tmux -S /tmp/osmocom_tmux has-session -t osmocom 2>/dev/null; then
        tmux -S /tmp/osmocom_tmux kill-session -t osmocom 2>/dev/null && n=$((n + 1))
    fi
    [ "$n" -gt 0 ] && printf '  %ssessions tmux%s   %s session(s) du run precedent effacee(s)\n' \
        "${C_DIM:-}" "${C_Z:-}" "$n"
    return 0
}
```

2.221 /opt/GSM/osmo-operator/startup_scripts/10-identite-noeud.sh

10990 octets, 197 lignes → 197 lignes

```
# 10-identite-noeud.sh - part of start.sh (list/genconf/stop/status actions,
# node identity prompt, point-code display)
# Sourced by start.sh in the main shell: same order, same variables.
case "$ACTION" in
    list)
        if [ "$DSP_BANC" = 1 ]; then
            printf '  %sbanc DSP%s   %s retire du plan : %s ; %s les reprend (5 etapes)\n' \
                "$C_DIM" "$C_Z" "$RUN_SH" "$DSP_MODULES_RETIRES" "$BANC_DSP"
        fi
        # Without --restart: run.sh keeps the LAST action read, and RUN_ARGS
        # ends with --restart, which would stop and relaunch the stack instead
        # of printing the plan.
        _list_args=()
        for _a in "${RUN_ARGS[@]}"; do [ "$_a" = "--restart" ] || _list_args+=("$_a"); done
        exec env CALYPSO_PROFILE="$CALYPSO_PROFILE" bash "$RUN_SH" --list "${_list_args[@]}"
        ;;
    genconf)
        # ── THE NODE SHEET, WITHOUT LAUNCHING ──────────────────────────
        # The same one the run writes (just before run.sh): identity resolved
        # as at launch (--node, environment, /etc/osmo-role, WAN table), WAN
        # table re-read if it exists.
        . "$HERE/network/wan-nodes.sh"; . "$HERE/network/node-conf.sh"
        _gc_node="$NODE_ID"
        if [ -z "$_gc_node" ]; then
            _gc="$(detect_node_id)"; _gc_node="${_gc%%|*}"
        fi
        _gc_node="${_gc_node:-1}"
        wan_nodes_load "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" 2>/dev/null || true
        _gc_hub="${HUB_IP:-${OSMO_HUB_IP:-}}"
        [ -n "$_gc_hub" ] || _gc_hub="$(awk -F= '/^OSMO_HUB_IP=/{print $2}' /etc/osmo-role 2>/dev/null | tr -d '
        ↪  \r')"
        _gc_out="${GEN_CONF_OUT:-$(node_conf_path "$_gc_node")}"
        if node_conf_write "$_gc_out" "$_gc_node" "${NODE_OP:-1}" "$_gc_hub"; then
            say_end " OK " "$C_OK" "Fiche du noeud $_gc_node" "$_gc_out"
            printf '  %ssur un autre noeud :%s  ./start-native.sh --wan %s\n' "${C_DIM:-}" "${C_Z:-}" "$(basename
            ↪  "$_gc_out")"
            exit 0
        fi
        say_end " KO " "$C_KO" "Fiche du noeud" "ecriture impossible : $_gc_out"; exit 1
        ;;
    stop)
        rm -f /run/osmo-banc/ms1-vty /run/osmo-banc/mobile-log 2>/dev/null
        command -v systemctl >/dev/null 2>&1 && systemctl unset-environment OSMO_BB1_VTY_PORT OSMO_MOBILE_LOG
        ↪  2>/dev/null || true
        say_begin "Arret de la pile via run.sh"
        banc_dsp_arreter          # QEMU, osmocon, mobile, bridge: its job
        banc_dsp_arreter_instances
        bash "$RUN_SH" --stop --profile "$CALYPSO_PROFILE"
        # the softSIM SAP servers (--sap) and their socket
        for _p in $(ps -eo pid,args | awk '/[d]emo_server\.rb/ {print $1}'); do kill -TERM "$_p" 2>/dev/null ||
        ↪  true; done
        rm -f "${SAP_SOCK:-/tmp/osmocom_sap}" 2>/dev/null; unset _p
        say_end " OK " "$C_OK" "Arret de la pile via run.sh"
        purge_sessions_tmux
        # ── THE DASHBOARD, OPTIONALLY STOPPED WITH THE BENCH ──────────
        # Neither run.sh --stop nor the tmux purge know the dashboard (a node
        # server, WantedBy=multi-user.target), and killall_python does not see
        # it. PartOf=osmo-banc.service covers `systemctl stop|restart
        # osmo-banc`; this covers the direct call, used by the desktop
        # "Arreter" button, launch.sh --stop and start-multi.sh --stop.
        # --no-block: do not wait for systemd, and never block when this stop
        # is itself the ExecStop of osmo-banc.
```

```
        # By default the dashboard (:8080) stays online with the bench
        # stopped; CALYPSO_DASHBOARD_SUIT_BANC=1 stops it with the bench.
        if [ "${CALYPSO_DASHBOARD_SUIT_BANC:-0}" != 1 ]; then
            :
        elif [ -d /run/systemd/system ] && command -v systemctl >/dev/null 2>&1; then
            systemctl --no-block stop osmo-egprs-web.service 2>/dev/null || true
        else
            pkill -f "osmo-egprs-web/server.js" 2>/dev/null || true
        fi
        # Same safety net as run.sh, applied here too: this script can stop
        # the stack without going through the other, and an orphan
        # fake_trx.py is enough to make the next run fail on "UDP port 5720
        # already in use". Repeating it costs nothing and covers the case
        # where RUN_SH is missing or exits with an error.
        # SCOPE: the BENCH's python3 only (see killall_python).
        # CALYPSO_STOP_KILL_PYTHON=0 disables it.
        if [ "${CALYPSO_STOP_KILL_PYTHON:-1}" != 0 ]; then
            if killall_python -9; then
                say_end " OK " "$C_OK" "python3 du banc termines"
            fi
        fi
        exit 0
        ;;
    status)
        if [ "$DSP_BANC" = 1 ]; then
            printf '\n  %sbanc DSP%s  %s\n' "$C_DIM" "$C_Z" "$BANC_DSP"
            banc_dsp --status
            printf '\n'
        fi
        exec env CALYPSO_PROFILE="$CALYPSO_PROFILE" bash "$RUN_SH" --status --profile "$CALYPSO_PROFILE"
        ;;
    checkpaths)
        # The DSP bench has no --check-paths: each of its five steps checks
        # its own binary and exits naming what is missing. So show what they
        # will look for, rather than inventing an answer.
        if [ "$DSP_BANC" = 1 ]; then
            printf '  %sbanc DSP%s   %s\n' "$C_DIM" "$C_Z" "$BANC_DSP"
            for f in "$(dirname "$BANC_DSP")/c54x_exe" "$GSM_ROOT/qosmo/build/qemu-system-arm" \
                     "${FIRMWARE_ELF:-$GSM_ROOT/firmware/board/compal_e88/$([ "${OSMO_TRX:-0}" = 1 ] && echo trx
                     ↪  || echo layer1).highram.elf}" \
                     "$GSM_ROOT/osmocom-bb/src/host/osmocon/osmocon" \
                     "$([ "${OSMO_TRX:-0}" = 1 ] && echo
                     ↪  "$GSM_ROOT/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver" || echo
                     ↪  "$HERE/pont/pont_dsp.py")"; do
                [ -e "$f" ] && printf '   OK   %s\n' "$f" || printf '   KO   %s (absent)\n' "$f"
            done
            printf '\n'
        fi
        exec env CALYPSO_PROFILE="$CALYPSO_PROFILE" bash "$RUN_SH" --check-paths
        ;;
esac
# --- 7bis-menu. What is missing, asked ONCE, with the cache as default --------
# The three values that decide the SS7 identity - node number, operator, hub
# address - are deduced most of the time: environment passed by
# start-docker.sh, /etc/osmo-role frozen in the image, WAN table. When the
# deduction yields nothing, the stack would otherwise keep the template's point
# codes (1.1.2 everywhere, ASP towards 127.0.0.1) and the interconnection
# would be dead silently.
#
# So ask - but only for what is missing, proposing what the cache already
# knows. Without a terminal nothing is asked: the defaults apply, an ISO that
# boots on its own cannot stay blocked on a question.
ask_node_identity() {
    # NO_MENU=1 is what start-docker.sh sets in front of the launcher when
    # the choices were already made on the host. The tty guard just below
    # covers it today (start-docker.sh launches via docker exec -d, without a
```

```
    # terminal); this test aligns the SS7 identity on the same convention as
    # the stack menus of start-docker.sh, for an automatic launcher that has
    # a terminal.
    [ "${NO_MENU:-0}" = "1" ] && return 0
    [ -t 0 ] || return 0
    [ "$DRY" -eq 1 ] && return 0

    local cached_node cached_src cached_hub
    IFS='|' read -r cached_node cached_src <<< "$(detect_node_id)"

    # THE ENVIRONMENT IS NEVER ASKED FOR. That is how start-docker.sh passes
    # the node to a container: the answer is already known, and in a container
    # where nobody reads the terminal the question would come back empty.
    # NODE_ID and HUB_IP are taken from the environment by the node
    # resolution, much higher up, once - not here.
    cached_hub="$(awk -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' /etc/osmo-role 2>/dev/null)"
    [ -n "$cached_hub" ] || cached_hub="172.20.0.10"

    if command -v whiptail >/dev/null 2>&1; then
        [ -n "$NODE_ID" ] || NODE_ID="$(whiptail --title "Identite du noeud" \
            --inputbox "Numero de ce noeud (1-9)${cached_src:+   [$cached_src]}\nVide = ne pas toucher a
            ↪  l'identite SS7." \
            11 66 "${cached_node:-}" 3>&1 1>&2 2>&3)" || NODE_ID=""
        if [ -n "$NODE_ID" ] && [ -z "$HUB_IP" ]; then
            HUB_IP="$(whiptail --title "Inter-STP" \
                --inputbox "Adresse du hub SS7 (PC 0.0.0)\nVide = laisser le remote-ip deja present." \
                11 66 "$cached_hub" 3>&1 1>&2 2>&3)" || HUB_IP=""
        fi
    else
        if [ -z "$NODE_ID" ]; then
            printf '  Numero de ce noeud (1-9)%s [%s] : ' \
                "${cached_src:+ - $cached_src}" "${cached_node:-aucun}"
            read -r NODE_ID || NODE_ID=""
            [ -n "$NODE_ID" ] || NODE_ID="${cached_node:-}"
        fi
        if [ -n "$NODE_ID" ] && [ -z "$HUB_IP" ]; then
            printf '  Adresse de l inter-STP [%s] : ' "$cached_hub"
            read -r HUB_IP || HUB_IP=""
            [ -n "$HUB_IP" ] || HUB_IP="$cached_hub"
        fi
    fi
}
ask_node_identity

# == Configuration regeneration (--regen) ====================================
# --regen does the same thing as start-docker.sh on the docker side, and as
# build-iso.sh at build time - that is, the full sequence:
#
#   1. apply_config_templates      substitutes configs/*.cfg and *.conf into a
#                                  NEW directory (never in place: a half-done
#                                  substitution on /etc/osmocom leaves a bench
#                                  that no longer starts);
#   2. apply_native_post_patches   the NATIVE recipe of the ISO - sms-routing
#                                  by real MS, SGSN/MSC on the core's local
#                                  address, trimmed run.sh;
#   3. install_configs_native      installs the result into /etc/osmocom and
#                                  /etc/asterisk.
#
# IN A SUBSHELL. lib/gabarits.sh loads globals.conf and sets about thirty
# variables - MCC, ARFCN, HOST_IP, N_OPERATORS, KI... - which carry a
# different meaning here (HOST_IP is the bench address for the launcher, the
# core's internal address for the templates). Letting them leak would change
# the radio plan of a run because a regeneration was requested.
#
#   $1 node number   $2 operator number   $3 inter-STP (may be empty)
# -- THE DISPLAYED POINT CODE FOLLOWS THE PLAN ACTUALLY APPLIED ---------------
```

```
# set-node-id.sh receives --local outside WAN and writes 1.<op>.x; the WAN
# plan between machines is 1.<node><op>.x. The banner uses the same rule so
# that it never announces a plan other than the one applied.
_pc_affiche() {   # $1 = node, $2 = operator
    if [ "${WAN_MESH:-0}" != "1" ] && [ -z "${WAN_NODES:-}" ]; then
        printf '1.%s.x' "$2"          # local plan, the one applied
    else
        printf '1.%s%s.x' "$1" "$2"   # WAN plan, between machines
    fi
}
```

2.222 /opt/GSM/osmo-operator/startup_scripts/11-regen-configs.sh

14110 octets, 262 lignes → 262 lignes

```
# 11-regen-configs.sh - part of start.sh (--regen: native regeneration of
# /etc/osmocom and /etc/asterisk, node identity before the WAN)
# Sourced by start.sh in the main shell: same order, same variables.
regen_configs_natif() {
    local node="$1" op="$2" hub="$3"
    local tmp rc=0

    # ── The NATIVE plan, the one of build-iso.sh, not docker's ──────────────
    # 127.0.0.2 (__CONTAINER_IP__) carries the core: GGSN, SGSN, UPF, the BSC
    # nsvc. A loopback address ALWAYS exists and is ready before any service;
    # a NIC address may be missing when osmo-ggsn binds, and the bind fails
    # with no visible relation to the network.
    # 127.0.0.1 (__GATEWAY_IP__) receives the gsmtap, where tshark captures.
    local core_ip="${OSMO_NATIVE_CORE_IP:-127.0.0.2}"
    local gw_ip="${OSMO_NATIVE_GW_IP:-127.0.0.1}"

    # ── The identity FOLLOWS THE NODE ──────────────────────────────────────
    # 1.<node><op>.<role>, the plan of network/set-node-id.sh and of
    # build-iso.sh --node. Without the node in the point code, three machines
    # attached to the same hub would present 1.11.2 three times: a point code
    # is an ADDRESS, and two devices sharing one give silent, wrong routing.
    # The routing context follows the same rule: it is what the hub reads to
    # tell the ASs apart, and the local formula (op*100+50) yields the same
    # value on every node.
    # WAN plan only when there is a WAN (--wan, --wan-nodes). A single host
    # keeps the LOCAL plan 1.<op>.<role>, like set-node-id.sh --local at
    # startup.
    local mid="${node}${op}" wan_plan=0
    if [ "${WAN_MESH:-0}" = "1" ] || [ -n "${WAN_NODES:-}" ]; then wan_plan=1; else mid="$op"; fi
    local pc_msc="1.${mid}.1" pc_stp="1.${mid}.2" pc_bsc="1.${mid}.3"
    local shut="shutdown"
    [ -n "$hub" ] && shut="no shutdown"

    # ── The PLMN: the machine's, not the table's ───────────────────────────
    # Re-read from the installed osmo-msc.cfg. Taking the globals.conf
    # defaults would rename the network at every regeneration, and a mobile
    # already provisioned would no longer recognise its cell.
    local cfgdir="${OSMOCOM_CFG:-/etc/osmocom}"
    local mcc mnc opname
    mcc="$(awk '/^ *network country code /{print $4; exit}' "$cfgdir/osmo-msc.cfg" 2>/dev/null)"
    mnc="$(awk '/^ *mobile network code /{print $4; exit}'  "$cfgdir/osmo-msc.cfg" 2>/dev/null)"
    opname="$(awk '/^ *short name /{print $3; exit}'        "$cfgdir/osmo-msc.cfg" 2>/dev/null)"
    [ -n "$mcc" ]    || mcc="${MCC:-001}"
    [ -n "$mnc" ]    || mnc="${MNC:-01}"
    [ -n "$opname" ] || opname="${OP_NAME:-OsmoGSM}"

    # The number of MS comes from coeur.env, written by build-iso.sh: it says
    # how many subscribers the HLR provisions, hence how many SMS routes are
    # needed. Beyond that, the relay returns "No route for destination".
```

```
local n_ms="${N_MS:-}"
if [ -z "$n_ms" ] && [ -r "$cfgdir/coeur.env" ]; then
    n_ms="$(awk -F'[:=}]' '/N_MS:=/{print $3; exit}' "$cfgdir/coeur.env" 2>/dev/null)"
fi
[ -n "$n_ms" ] || n_ms=2

printf '  %splan natif%s coeur %s  gsmtap %s  PC %s  rctx %s  PLMN %s-%s (%s)  %s MS\n' \
    "${C_DIM:-}" "${C_Z:-}" "$core_ip" "$gw_ip" "$pc_stp" \
    "$(( node * 1000 + op * 100 + 50 ))" "$mcc" "$mnc" "$opname" "$n_ms"

tmp="$(mktemp -d)" || return 1

# ── The backup, BEFORE writing ──────────────────────────────────────
# A regeneration overwrites files that someone may have edited by hand on
# the bench; the previous state is kept as an archive.
local sav="${LOG_DIR:-/tmp}/osmocom-avant-regen-$(date +%Y%m%d-%H%M%S).tar.gz"
tar czf "$sav" -C / etc/osmocom etc/asterisk 2>/dev/null && \
    printf '  %ssauvegarde%s %s\n' "${C_DIM:-}" "${C_Z:-}" "$sav"

(
    cd "$HERE" || exit 1
    # BEFORE sourcing. lib/gabarits.sh pins ENCRYPTION to "a5 0" and
    # HOST_IP to 127.0.0.1 with ":=" when they are empty, and gc_load then
    # treats those values as the caller's choice.
    : "${ENCRYPTION:=a5 1}"
    HOST_IP="$core_ip"
    # The routing context and the ASP source address: the two overrides
    # build-iso.sh sets for a WAN node. 0.0.0.0 because the node address
    # comes from DHCP and is not necessarily up when osmo-stp starts -
    # binding to an absent address fails at launch.
    if [ "$wan_plan" = "1" ]; then
        export RCTX_INTER_OVERRIDE=$(( node * 1000 + op * 100 + 50 ))
    else
        export RCTX_INTER_OVERRIDE=$(( op * 100 + 50 ))
    fi
    export INTER_LOCAL_IP_OVERRIDE="0.0.0.0"
    # _apply_node_ss7_addressing (called at the end of
    # apply_config_templates) only replays set-node-id.sh if it knows WHO
    # it serves, and it reads that only here.
    export OSMO_ROLE=operator OSMO_WAN_NODE="$node"
    [ -n "$hub" ] && export OSMO_HUB_IP="$hub"
    unset OSMO_NO_REGEN

    # shellcheck source=lib/gabarits.sh
    . "$HERE/lib/gabarits.sh" || exit 1

    apply_config_templates "$tmp" \
        "$core_ip" "$gw_ip" \
        "$op" "$pc_msc" "$pc_stp" "$pc_bsc" \
        "$mcc" "$mnc" "$opname" \
        "${hub:-127.0.0.1}" "$shut" "1" || exit 1

    # The node and the WAN table: sms-routing.conf takes each node's
    # address from the table, and the MSISDNs (<node>00<operator><rank>)
    # from the node.
    apply_native_post_patches "$tmp" "$op" "$n_ms" "$core_ip" \
        "$node" "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" || exit 1

    # ── bb/ REMOVED on purpose ──────────────────────────────────────
    # install_configs_native would put $tmp/bb/mobile.cfg over
    # /root/.osmocom/bb/mobile.cfg. That file is already WRITTEN above by
    # generate_mobile_cfg, which alone sets the VTY port, the L1CTL
    # sockets and the ARFCN of EACH mobile. The template is frozen on
    # operator 1: installing it here would put both mobiles on the same
    # ARFCN and the same VTY.
    rm -rf "$tmp/bb"
```

```
        install_configs_native "$tmp" "" || exit 1
    ) || rc=1

    rm -rf "$tmp"
    return "$rc"
}

# ── The templates: kept, except with --regen ──────────────────────────
# By default nothing is touched: in a container, what start-docker.sh has just
# written - node point codes, inter-STP, active ASP - must not be replaced by
# the template values.
if [ "${REGEN_GABARITS:-0}" -eq 1 ]; then
    unset OSMO_NO_REGEN
    printf '  %sgabarits%s   regeneration demandee (--regen)\n' "${C_DIM:-}" "${C_Z:-}"

    # ── REGEN: STOP FIRST ─────────────────────────────────────────────
    # Regenerating /etc/osmocom under a LIVE stack is useless and misleading:
    # the daemons have already read their configuration, keep the old one in
    # memory, and the new files only take effect at the next start. A --regen
    # therefore stops the stack, then regenerates: the state is unambiguous.
    if [ -x "$RUN_SH" ] || [ -r "$RUN_SH" ]; then
        say_begin "Arret de la pile avant regeneration"
        banc_dsp_arreter >/dev/null 2>&1 || true
        bash "$RUN_SH" --stop --profile "$CALYPSO_PROFILE" >/dev/null 2>&1 || true
        say_end " OK " "$C_OK" "Arret de la pile avant regeneration"
        declare -F purge_sessions_tmux >/dev/null && purge_sessions_tmux
        # Same safety net as --stop: an orphan python3 (bridge, fake_trx,
        # trxcon) keeps its port and makes the next start fail on "UDP port
        # 5720 already in use". SCOPE: the bench's python3 only.
        if [ "${CALYPSO_STOP_KILL_PYTHON:-1}" != 0 ]; then
            killall_python -TERM && \
                say_end " OK " "$C_OK" "python3 du banc termines"
        fi
    fi

    # ── THE NODE, 1 BY DEFAULT ────────────────────────────────────────
    # A regeneration produces a COMPLETE identity, never a plan "without a
    # node" (without a number, apply_config_templates would yield the
    # template plan - point code 1.1.2 for every machine). So always number:
    # the node resolved above (--node, environment, /etc/osmo-role, WAN
    # table), and 1 when there is none. The configurations follow it: PC
    # 1.<node><op>.<role>, routing context <node>*1000+..., and the private
    # segment 192.168.<node+1>.x.
    REGEN_NODE="${NODE_ID:-1}"
    if ! [[ "$REGEN_NODE" =~ ^[1-9]$ ]]; then
        say_end " KO " "$C_KO" "--regen" "numero de noeud invalide : $REGEN_NODE (1 a 9)"
        exit 2
    fi
    REGEN_OP="${NODE_OP:-1}"
    if ! [[ "$REGEN_OP" =~ ^[1-9]$ ]]; then
        say_end " KO " "$C_KO" "--regen" "numero d'operateur invalide : $REGEN_OP (1 a 9)"
        exit 2
    fi
    # cached_hub is LOCAL to ask_node_identity: the hub is re-read from the
    # environment and /etc/osmo-role here, otherwise the ASP would be left in
    # shutdown after each regeneration.
    REGEN_HUB="${HUB_IP:-${OSMO_HUB_IP:-}}"
    [ -n "$REGEN_HUB" ] || REGEN_HUB="$(awk -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' \
        "${ROLE_FILE:-/etc/osmo-role}" 2>/dev/null)"

    say_begin "Regeneration des configurations (noeud $REGEN_NODE, operateur $REGEN_OP)"
    if [ "$DRY" -eq 1 ]; then
        say_end " -- " "$C_DIM" "Regeneration des configurations" "dry-run"
    elif regen_configs_natif "$REGEN_NODE" "$REGEN_OP" "$REGEN_HUB"; then
        say_end " OK " "$C_OK" "Regeneration des configurations" \
```

```
            "/etc/osmocom + /etc/asterisk  ·  PC $(_pc_affiche "$REGEN_NODE" "$REGEN_OP")"
    else
        say_end " KO " "$C_KO" "Regeneration des configurations" \
            "echec - les fichiers d'avant sont dans ${LOG_DIR:-/tmp}"
        exit 1
    fi

    # ── THE HLR FOLLOWS THE NUMBERING ──────────────────────────────────
    # A regeneration changes the MSISDNs (<node>00<operator><rank>): those in
    # the HLR date from the previous plan. The IMSI does not move - it does
    # not carry the node, precisely so that the Ki and authentication stay
    # valid. The subscriber step therefore also re-reads the MSISDN
    # (run_modules/21-abonnes-hlr.sh, mod_abonnes_hlr_status) and replays
    # "subscriber ... update msisdn" when it changed.
    #
    # It is also forced here: the stack has just been stopped, the HLR will
    # restart with run.sh, and the step must run even if a state cache
    # believes it already done.
    FORCE=1
    printf '  %sHLR%s        abonnes a reapprovisionner : MSISDN %s..%s\n' \
        "${C_DIM:-}" "${C_Z:-}" \
        "$(( REGEN_NODE * 100000 + REGEN_OP * 100 + 1 ))" \
        "$(( REGEN_NODE * 100000 + REGEN_OP * 100 + ${N_MS:-2} ))"

    # The chosen node holds for the rest of the launch: it is the one just
    # written into the files. Leaving it empty here would let the "SS7
    # identity" section below resolve differently and overwrite the identity
    # just applied.
    NODE_ID="$REGEN_NODE"
    NODE_OP="$REGEN_OP"
    [ -n "$REGEN_HUB" ] && HUB_IP="${HUB_IP:-$REGEN_HUB}"
    export OSMO_ROLE=operator
    export OSMO_WAN_NODE="$NODE_ID"
    [ -n "$HUB_IP" ] && export OSMO_HUB_IP="$HUB_IP"
elif [ "${RUNTIME_ENV:-native}" = "docker" ]; then
    # IN A CONTAINER only. The configs are already written there by
    # start-docker.sh, with the node identity: regenerating them would replace
    # them with the template values, and the interconnection would die.
    export OSMO_NO_REGEN=1
else
    # ON A VM without --regen: KEEP what is in place. The ISO generated these
    # files at build time, with the same recipe as regen_configs_natif;
    # redoing them at every start would change nothing and would overwrite
    # any edit made on the bench. --regen exists for that.
    if [ -n "$NODE_ID" ]; then
        export OSMO_ROLE=operator
        export OSMO_WAN_NODE="$NODE_ID"
        [ -n "$HUB_IP" ] && export OSMO_HUB_IP="$HUB_IP"
    fi
fi

# --- 7ter. Node identity (--node) ------------------------------------------------
# BEFORE the WAN and before run.sh: the point codes are read by osmo-stp,
# osmo-msc and osmo-bsc at their start. Changing them afterwards is useless.
#
# A SINGLE SOURCE OF TRUTH. start-docker.sh already computes a container's
# identity and passes it through the environment; it is reused when present
# rather than recomputed here.
if [ -z "$NODE_ID" ] && [ -n "${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}" ]; then
    NODE_ID="${OSMO_WAN_NODE:-$WAN_NODE_ID}"
    [ -n "$HUB_IP" ] || HUB_IP="${OSMO_HUB_IP:-}"
fi

# No identity while the WAN is requested: do NOT start silently. The stack
# would start with the template point codes (1.1.2 for everybody), two nodes
# would carry the same SS7 address, and no ASP would attach.
```

```
# --wan FILE without --node: this node takes the first number the sheets do
# not carry (the sheets say who is already there).
if [ -z "$NODE_ID" ] && [ "${#WAN_IMPORT_FILES[@]}" -gt 0 ]; then
    _used=" $(sed -n 's/^OSMO_NODE_ID=//p' "${WAN_IMPORT_FILES[@]}" 2>/dev/null | tr -d '" ' | tr '\n' ' ') "
    for _cand in 1 2 3 4 5 6 7 8 9; do
        case "$_used" in *" $_cand "*) ;; *) NODE_ID="$_cand"; break ;; esac
    done
    NODE_ID_SRC="premier numero libre d'apres les fiches"
    printf '  %sIdentite%s   noeud %s (%s ; --node N pour choisir)\n' "${C_DIM:-}" "${C_Z:-}" "$NODE_ID"
     ↪  "$NODE_ID_SRC"
fi
if [ -z "$NODE_ID" ] && [ "${WAN_MESH:-0}" -eq 1 ]; then
    say_end " KO " "$C_KO" "Identite SS7" "WAN demande mais aucun noeud : --node N, ou WAN_NODE_ID"
    exit 1
fi
```

2.223 /opt/GSM/osmo-operator/startup_scripts/12-pcap-softsim.sh

29684 octets, 541 lignes → 541 lignes

```
# 12-pcap-softsim.sh - part of start.sh (SS7 node identity, WAN mesh,
# inter-STP hub, local-ip of the ASP, tcpdump ring wrapper, softSIM stop)
# Sourced by start.sh in the main shell: same order, same variables.
if [ -n "$NODE_ID" ]; then
    if ! [[ "$NODE_ID" =~ ^[1-9]$ ]]; then
        say_end " KO " "$C_KO" "--node" "un chiffre de 1 a 9"; exit 2
    fi
    SETID="$HERE/network/set-node-id.sh"
    if [ ! -r "$SETID" ]; then
        say_end " KO " "$C_KO" "--node" "network/set-node-id.sh absent"; exit 1
    fi
    say_begin "Identite SS7 du noeud $NODE_ID"
    setid_args=(--node "$NODE_ID" --op "$NODE_OP" --mode "$NODE_MODE")
    # ── LOCAL PLAN OUTSIDE THE WAN ──────────────────────────────────
    # set-node-id.sh knows two plans: the WAN plan, MID=<node><op>
    # (node 1 / op 2 -> 1.12.2), and the LOCAL plan, MID=<op> (op 2 -> 1.2.2),
    # which is what the hub and the bench topology expect on a single host.
    # The WAN plan only makes sense BETWEEN MACHINES; without a WAN, one
    # machine carries everything and the local plan applies.
    if [ "${WAN_MESH:-0}" != "1" ] && [ -z "${WAN_NODES:-}" ]; then
        setid_args+=(--local)
    fi
    [ -n "$HUB_IP" ] && setid_args+=(--hub-ip "$HUB_IP")
    if [ "$DRY" -eq 1 ]; then
        say_end " -- " "$C_DIM" "Identite SS7 du noeud $NODE_ID" "dry-run"
        bash "$SETID" "${setid_args[@]}" --dry-run 2>&1 | sed 's/^/  /'
    elif bash "$SETID" "${setid_args[@]}" </dev/null > "${LOG_DIR:-/tmp}/set-node-id.log" 2>&1; then
        say_end " OK " "$C_OK" "Identite SS7" "noeud $NODE_ID · PC $(_pc_affiche "$NODE_ID" "$NODE_OP") · mode
         ↪  $NODE_MODE"
        # What was ACTUALLY written, re-read from the file.
        _pc_now="$(awk "/^cs7 instance/{c=1} c && /^ *point-code /{print \$2; exit}" \
                    "${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp.cfg" 2>/dev/null)"
        _hub_now="$(awk "/asp asp-to-inter/{f=1} f&&/remote-ip/{print \$2; exit}" \
                    "${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp.cfg" 2>/dev/null)"
        printf "  %sSS7%s        point-code %s   inter-STP %s\n" \
            "${C_DIM:-}" "${C_Z:-}" "${_pc_now:-?}" "${_hub_now:-?}"
    else
        say_end " KO " "$C_KO" "Identite SS7" "voir ${LOG_DIR:-/tmp}/set-node-id.log"
        exit 1
    fi
    # The node number also applies to voice and SMS: it is the same node.
    WAN_NODE_ID="$NODE_ID"
fi
```

```
# --- 7bis. N-node WAN ------------------------------------------------------
# HERE and not later: this process becomes run.sh (exec), there is no later.
# Asterisk is not started yet - setup-wan-mesh.sh sees it, writes the conf and
# restarts nothing; run.sh will start Asterisk with the WAN already in place.
#
# Regarding the templates: natively nothing reinstalls /etc/asterisk at start
# (install_configs_native has no caller from run.sh), so the WAN block written
# here survives run.sh. A template module, if any, must be followed by the
# WAN block, not preceded by it.
if [ -r "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" ] && [ "$WAN_MESH" -eq 0 ]; then
    # Table frozen in the image (ISO built with --wan): WAN_AUTO=1 says
    # "this system IS a node", the option need not be typed again.
    if grep -q '^WAN_AUTO=1' "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" 2>/dev/null; then
        WAN_MESH=1
        WAN_AUTO_KEEP=1
        printf '  %sWAN%s        table figee dans %s (WAN_AUTO=1)\n' \
            "$C_DIM" "$C_Z" "${WAN_CONF_FILE:-/etc/osmo-wan.conf}"
    fi
fi

# ── ISM PAYLOAD: the SMS text over radio, the rest over IP ──────────────────
# Only at start. Writes (if a backend is given) the [payload] section of
# sms-routing.conf via network/setup-payload-ism.sh, then exports
# PAYLOAD_ISM: the SMS relay (scripts/sms-interop-relay.py) then puts a token
# in the inter-node TCP and carries the text over the ISM radio. Without a
# backend, the section already written by setup-payload-ism.sh is reused.
if [ "${PAYLOAD_ISM:-0}" -eq 1 ] && [ "$ACTION" = "start" ]; then
    PISM_SH="$HERE/network/setup-payload-ism.sh"
    export PAYLOAD_ISM=1
    if [ "$DRY" -eq 1 ]; then
        say_begin "Payload ISM"
        if [ -n "$PAYLOAD_ISM_LINK" ]; then
            printf '  [dry-run] %s --%s%s\n' "$PISM_SH" "$PAYLOAD_ISM_LINK" \
                "$([ "$PAYLOAD_ISM_LINK" = serial ] && echo " ${ISM_SERIAL:-/dev/ttyUSB0}")"
        else
            printf '  [dry-run] section [payload] deja en place (sms-routing.conf)\n'
        fi
        say_end " -- " "$C_DIM" "Payload ISM" "dry-run"
    elif [ -n "$PAYLOAD_ISM_LINK" ]; then
        say_begin "Payload ISM ($PAYLOAD_ISM_LINK)"
        pism_args=("--$PAYLOAD_ISM_LINK")
        case "$PAYLOAD_ISM_LINK" in
            serial) pism_args=(--serial "${ISM_SERIAL:-/dev/ttyUSB0}" --baud "${ISM_SERIAL_BAUD:-9600}") ;;
            fifo)   pism_args=(--fifo --tx "${ISM_TX:-/tmp/osmo_ism_tx}" --rx "${ISM_RX:-/tmp/osmo_ism_rx}") ;;
            exec)   pism_args=(--exec ${ISM_TX_CMD:+--tx-cmd "$ISM_TX_CMD"} ${ISM_RX_CMD:+--rx-cmd "$ISM_RX_CMD"})
             ↪  ;;
        esac
        if [ -r "$PISM_SH" ] && bash "$PISM_SH" "${pism_args[@]}" \
                > "${LOG_DIR:-/tmp}/payload-ism.log" 2>&1; then
           say_end " OK " "$C_OK" "Payload ISM" "texte des SMS par radio ($PAYLOAD_ISM_LINK), signalisation par IP"
        else
            say_end " KO " "$C_KO" "Payload ISM" "voir ${LOG_DIR:-/tmp}/payload-ism.log ; le relais retombera sur
             ↪  IP"
        fi
    else
        say_begin "Payload ISM"
        if grep -rqs '^\[payload\]' /etc/osmocom/sms-routing.conf 2>/dev/null; then
            say_end " OK " "$C_OK" "Payload ISM" "section [payload] en place (sms-routing.conf)"
        else
            say_end " -- " "$C_DIM" "Payload ISM" "aucune section [payload] : lance d'abord $PISM_SH
             ↪  --serial|--exec|..."
        fi
    fi
fi

# --interstp only applies in the WAN: without --wan, say so.
```

```
if [ "$INTERSTP" = "1" ] && [ "$WAN_MESH" -ne 1 ] && [ "$ACTION" = "start" ]; then
    printf '  %s!%s --interstp sans --wan : pas de WAN, pas de hub lance (./start-native.sh --wan --interstp)\n'
     ↪  "${C_KO:-}" "${C_Z:-}" >&2
fi
if [ "$WAN_MESH" -eq 1 ] && [ "$ACTION" = "start" ]; then
    WAN_LIB="$HERE/network/wan-nodes.sh"
    WAN_MESH_SH="$HERE/network/setup-wan-mesh.sh"
    if [ ! -r "$WAN_LIB" ] || [ ! -r "$WAN_MESH_SH" ]; then
        say_end " KO " "$C_KO" "WAN" "network/wan-nodes.sh ou setup-wan-mesh.sh absent"
        exit 1
    fi
    # shellcheck source=network/wan-nodes.sh
    . "$WAN_LIB"
    export WAN_AUTO_SAVE="$WAN_AUTO_KEEP"
    # --regen-conf: start from scratch. The table in place is set aside
    # (never lost silently) then erased, BEFORE any wan_nodes_load: the rest
    # of the block rebuilds it from --wan-nodes, the sheets or the prompt,
    # without dragging along stale peers (PROHIB routes at the STP).
    if [ "$REGEN_WAN_CONF" -eq 1 ]; then
        _wc="${WAN_CONF_FILE:-/etc/osmo-wan.conf}"
        # The backup stays NEXT TO the table: /tmp (LOG_DIR by default) does
        # not survive a reboot, even less on the ISO in RAM.
        _sav="${_wc}.avant-regen-$(date +%Y%m%d-%H%M%S)"
        if [ "$DRY" -eq 1 ]; then
            [ -f "$_wc" ] && printf '  [dry-run] cp -p %s %s && rm -f %s\n' "$_wc" "$_sav" "$_wc"
        elif [ -f "$_wc" ]; then
            # No backup, no erasure: the table is never lost silently.
            if cp -p "$_wc" "$_sav" 2>/dev/null; then
                printf '  %sWAN%s        table mise de cote : %s\n' "$C_DIM" "$C_Z" "$_sav"
                rm -f "$_wc"
            else
               say_end " KO " "$C_KO" "WAN" "--regen-conf : sauvegarde impossible ($_sav), table laissee en place"
                exit 1
            fi
        fi
        printf '  %sWAN%s        table reconstruite de zero (--regen-conf)\n' "$C_DIM" "$C_Z"
        unset _wc _sav
    fi
    # --virtualbox: host-only segment + VM + table, before everything else.
    if [ "$VBOX_INTERCO" -eq 1 ] && [ -z "${WAN_NODES:-}" ]; then
        VB="$HERE/network/setup-vbox-interco.sh"
        [ -r "$VB" ] || { say_end " KO " "$C_KO" "WAN" "network/setup-vbox-interco.sh absent"; exit 1; }
        vb_args=(--host-node "$VBOX_HOST_NODE" --conf "${WAN_CONF_FILE:-/etc/osmo-wan.conf}")
        [ -n "$VBOX_NODES" ] && vb_args+=(--nodes "$VBOX_NODES")
        bash "$VB" "${vb_args[@]}" || exit 1
        wan_nodes_load "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" || exit 1
        WAN_NODES="$(wan_nodes_spec)"
        WAN_NODE_ID="$VBOX_HOST_NODE"
    fi

    if [ "${#WAN_IMPORT_FILES[@]}" -gt 0 ]; then
        # ── --wan mynode1.conf: THE TABLE COMES FROM THE SHEETS ──────────
        # Each ISO writes its sheet at run time (network/node-conf.sh); they
        # are imported over the table in place, this node is added with its
        # local address, and the usual mesh does the rest. No prompt:
        # everything that had to be asked is in the sheets.
        . "$HERE/network/node-conf.sh"
        wan_nodes_load "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" 2>/dev/null || true
        printf '  %sWAN%s        fiches de noeuds :\n' "$C_DIM" "$C_Z"
        for _nf in "${WAN_IMPORT_FILES[@]}"; do node_conf_import "$_nf" || exit 1; done
        [ -n "$NODE_ID" ] || { say_end " KO " "$C_KO" "WAN" "--wan FICHIER exige de connaitre ce noeud (--node
         ↪  N)"; exit 1; }
        WAN_NODE_ID="$NODE_ID"
        node_conf_self "$NODE_ID" || exit 1
        WAN_NODES="$(wan_nodes_spec)"
        wan_nodes_parse "$WAN_NODES" || exit 1
```

```
elif [ -n "${WAN_NODES:-}" ]; then
    wan_nodes_parse "$WAN_NODES" || exit 1
    # --node ALREADY SAYS which node this machine is: the same notion as
    # --wan-id, seen from SS7. Using it avoids failing on IP detection
    # when the embedded table is older than the machine - an ISO burnt
    # before the latest addressing plan.
    if [ "${WAN_NODE_ID:-0}" = "0" ] && [ -n "$NODE_ID" ]; then
        WAN_NODE_ID="$NODE_ID"
        printf '  %sWAN%s        noeud %s (repris de --node)\n' "$C_DIM" "$C_Z" "$WAN_NODE_ID"
    fi
    [ "${WAN_NODE_ID:-0}" != "0" ] || wan_nodes_detect_self || {
        printf '  %sWAN%s : aucune IP locale dans la table - passez --wan-id N\n' "$C_KO" "$C_Z"
        printf '  %s      (ou --node N : le numero de noeud vaut pour les deux)%s\n' "$C_DIM" "$C_Z"
        exit 1; }
else
    wan_nodes_load "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" 2>/dev/null || true
    # An ISO deployed on N machines carries the SAME table: each machine
    # recognises itself by its IP rather than requiring one image per node.
    wan_nodes_detect_self 2>/dev/null || true
    if [ "${WAN_NODE_COUNT:-0}" -lt 1 ] || [ "${WAN_NODE_ID:-0}" = "0" ]; then
        wan_nodes_prompt || exit 1
    fi
    # --node N: this machine IS node N. Its line in the table is reset to
    # the real address, and the table rewritten - without a prompt.
    if [ -n "$NODE_ID" ] && [ "$WAN_MESH" -eq 1 ]; then
        wan_self_refresh "$NODE_ID" "${HUB_IP:-${OSMO_HUB_IP:-}}" || exit 1
    fi
fi
wan_nodes_validate || exit 1
printf '\n'
wan_nodes_summary
printf '\n'
if [ "$DRY" -eq 1 ]; then
    printf '  [dry-run] %s --native --id %s\n' "$WAN_MESH_SH" "$WAN_NODE_ID"
else
    say_begin "WAN - application de la table"
    # --native / --docker: same distinction as for the point codes.
    if bash "$WAN_MESH_SH" "--$NODE_MODE" --id "$WAN_NODE_ID" \
            --nodes "$(wan_nodes_spec)" --ops 1 \
            --config "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" > "${LOG_DIR:-/tmp}/wan-mesh.log" 2>&1; then
        say_end " OK " "$C_OK" "WAN - noeud $WAN_NODE_ID, indicatif $(wan_local_ind)"
    else
        say_end " KO " "$C_KO" "WAN" "voir ${LOG_DIR:-/tmp}/wan-mesh.log"
    fi
fi

# --- 7quater. Standalone inter-STP (SS7 hub of the WAN) -------------------
# The hub is the only point where the nodes meet in SS7; without it, the
# ASP of each STP stays in shutdown. Standalone: a separate osmo-stp with
# its own config (start-interstp.sh), not a bench stack.
# Who launches it, without launching two:
#   --interstp       this node, always;
#   auto (default)   only if the hub address is ALREADY local, or if this
#                    node is node 1 and nobody answers in M3UA on
#                    HUB_IP:2908. Two nodes starting together do not each
#                    take the hub address (duplicate IP).
#   --no-interstp    never.
# "Present" is read on the M3UA port (SCTP 2908), not by ping: any host
# at that address, or a stopped hub, answers the ping.
INTERSTP_SH="$HERE/launch/start-interstp.sh"
if [ -z "${HUB_IP:-}" ]; then
    HUB_IP="$(awk -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}' /etc/osmo-role 2>/dev/null)"
    # The LAN default only applies to a native node; in a container the
    # hub is on the docker backbone (/etc/osmo-role, or start-interstp.sh).
    [ -n "$HUB_IP" ] || { [ "$NODE_MODE" = native ] && HUB_IP="192.168.1.49"; }
fi
```

```
    _m3ua_ecoute() {   # $1 IP: does a hub answer in M3UA (SCTP) on port 2908?
        python3 -c 'import socket,sys
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM,132); s.settimeout(2)
try: s.connect((sys.argv[1],2908)); sys.exit(0)
except Exception: sys.exit(1)' "$1" 2>/dev/null
    }
    if [ "$INTERSTP" != "0" ] && [ -n "${HUB_IP:-}" ] && [ -r "$INTERSTP_SH" ]; then
        hub_local=0; hub_up=0; hub_owned=0
        ip -4 -o addr show 2>/dev/null | awk '{split($4,a,"/"); print a[1]}' | grep -qxF "$HUB_IP" && hub_local=1
        _m3ua_ecoute "$HUB_IP" && hub_up=1
        # hub_owned: the address answers the ping but not in M3UA, and is not
        # ours - ANOTHER machine holds it; taking it here would create an IP
        # conflict on the LAN.
       if [ "$hub_up" = 0 ] && [ "$hub_local" = 0 ] && ping -c1 -W1 "$HUB_IP" >/dev/null 2>&1; then hub_owned=1; fi
        lancer=0
        if [ "$INTERSTP" = "1" ]; then lancer=1
        elif [ "$hub_up" = 1 ] && [ "$hub_local" = 0 ]; then lancer=0
        elif [ "$hub_local" = 1 ]; then lancer=1
        elif [ "${WAN_NODE_ID:-0}" = "1" ] && [ "$hub_owned" = 0 ]; then lancer=1
        fi
        if [ "$lancer" = 1 ]; then
            # The nodes with THEIR number (id:ip): a 1,3 table stays 1,3 on the hub.
            ips=""
            for id in "${WAN_NODE_LIST[@]}"; do ips="${ips}${id}:${WAN_IP[$id]} "; done
            if [ "$DRY" -eq 1 ]; then
                printf '  [dry-run] %s --no-save --ops 1 --ip %s --ips "%s"\n' "$INTERSTP_SH" "$HUB_IP" "$ips"
            else
                say_begin "Inter-STP standalone ($HUB_IP)"
                # --no-save: the WAN table of THIS node (WAN_AUTO=1, its number,
                # its prefixes) has just been written by setup-wan-mesh.sh;
                # the hub does not rewrite it.
                if bash "$INTERSTP_SH" --no-save --ops 1 \
                        --ip "$HUB_IP" --ips "$ips" > "${LOG_DIR:-/tmp}/interstp.log" 2>&1; then
                    say_end " OK " "$C_OK" "Inter-STP" "hub $HUB_IP:2908, $WAN_NODE_COUNT noeud(s)"
                else
                    say_end " KO " "$C_KO" "Inter-STP" "voir ${LOG_DIR:-/tmp}/interstp.log"
                fi
            fi
        elif [ "$hub_up" = 1 ]; then
            say_end " -- " "$C_DIM" "Inter-STP" "hub $HUB_IP deja en service (M3UA 2908) - non relance"
        elif [ "$hub_owned" = 1 ]; then
            say_end " KO " "$C_KO" "Inter-STP" "$HUB_IP repond au ping mais pas en SCTP 2908 : lancer
             ↪  start-interstp.sh SUR cette machine-la"
        else
            say_end " -- " "$C_DIM" "Inter-STP" "hub $HUB_IP muet, non lance ici (noeud ${WAN_NODE_ID:-?}, pas le
             ↪  1) - --interstp pour le forcer"
        fi
    fi

    # --- 7quinquies. SS7 addressing of the node (set_stp_ip.sh) ---------------
    # setup-wan-mesh.sh sets up VOICE and SMS between nodes; it does not touch
    # SS7. Without what follows, the inter-STP ASP would still target
    # 127.0.0.1, in shutdown, and the SS7 interconnection would never come up.
    #
    # set_stp_ip.sh is the only one that writes the three configs, the WAN
    # table and /etc/osmo-role in one go. It gets --no-ip: the local address
    # is already set (DHCP or WAN block above), and --restart would be absurd
    # here since run.sh starts the stack right after, with the corrected files.
    STP_IP_SH="$HERE/set_stp_ip.sh"
    if [ -r "$STP_IP_SH" ] && [ "${WAN_NODE_ID:-0}" != "0" ]; then
        self_ip="${WAN_IP[$WAN_NODE_ID]:-}"
        hub_ip="$HUB_IP"
        [ -n "$hub_ip" ] || hub_ip="$(awk -F= '/^OSMO_HUB_IP=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print v}'
         ↪  /etc/osmo-role 2>/dev/null)"

        stp_args=(--operator --node "$WAN_NODE_ID" --no-ip --conf-dir "${OSMOCOM_CFG:-/etc/osmocom}")
```

```
        [ -n "$self_ip" ] && stp_args+=(--ip "$self_ip")
        [ -n "$hub_ip" ]  && stp_args+=(--hub-ip "$hub_ip")

        if [ "$DRY" -eq 1 ]; then
            printf '  [dry-run] %s %s\n' "$STP_IP_SH" "${stp_args[*]}"
        else
            say_begin "Adressage SS7 - inter-STP ${hub_ip:-?}"
            # </dev/null: automatic call, never interactive (set_stp_ip.sh asks
            # for the hub address when it is missing - here nobody would
            # answer and the launch would hang).
            if bash "$STP_IP_SH" "${stp_args[@]}" </dev/null > "${LOG_DIR:-/tmp}/set-stp-ip.log" 2>&1; then
                say_end " OK " "$C_OK" "Adressage SS7" "noeud $WAN_NODE_ID -> inter-STP ${hub_ip:-?}"
            else
                # Non fatal: the local stack runs fine without interconnection.
                # SAY it, rather than suggesting a complete WAN.
                say_end " ~~ " "$C_SK" "Adressage SS7" "echec - voir ${LOG_DIR:-/tmp}/set-stp-ip.log"
            fi
        fi
    fi
fi

# --- 7quater-bis. Local SS7 hub (standalone bench) --------------------------
# Outside the WAN, the local STP keeps an ASP "asp-to-inter" towards an
# inter-STP. If that hub's address (remote-ip of asp-to-inter) is carried by
# THIS machine and nothing answers in M3UA (SCTP 2908), it is launched here,
# before run.sh: the STP's ASP attaches to it from its start. Node and
# operator come from the routing-key of as-inter (rctx = node*1000 + op*100 +
# 50). A remote hub is never touched; a local hub already in service is only
# touched if it still carries the nodes of a previous WAN (see below);
# --no-interstp disables, --no-save keeps the WAN table. Same logic as
# tests/modules/33-ss7.sh.
if [ "$ACTION" = "start" ] && [ "$WAN_MESH" -ne 1 ] && [ "$INTERSTP" != "0" ]; then
    _stp_cfg="${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp.cfg"
    _hub="$(awk '/^ *asp asp-to-inter /{f=1;next} f&&/^ *remote-ip /{print $2; exit} f&&/^ (asp|as) /{exit}'
     ↪  "$_stp_cfg" 2>/dev/null)"
    _rctx="$(awk '/^ *as as-inter /{f=1;next} f&&/^ *routing-key /{print $2; exit} f&&/^ (asp|as) /{exit}'
     ↪  "$_stp_cfg" 2>/dev/null)"
    if [ -n "$_hub" ] && [[ "$_rctx" =~ ^[0-9]+$ ]] && [ -r "$HERE/launch/start-interstp.sh" ] \
       && ip -4 -o addr show 2>/dev/null | awk '{split($4,a,"/"); print a[1]}' | grep -qxF "$_hub"; then
        _noeud=$(( _rctx / 1000 )); _op=$(( (_rctx / 100) % 10 ))
        # A hub still running with the table of a previous WAN declares the
        # ASs of the other nodes (as-n2op1, rctx 2150...), without ASP since
        # those nodes no longer attach, and announces them PROHIB to our STP
        # (dynamic route, re-announced by the hub). Outside the WAN, a local
        # hub carrying routing-keys other than ours is therefore relaunched
        # with our node only.
        _hub_cfg="${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp-interop.cfg"
        _hub_autres="$(awk -v k="$_rctx" '/^ *routing-key /{ if ($2 != k) print $2 }' "$_hub_cfg" 2>/dev/null | tr
         ↪  '\n' ' ')"
        if python3 -c 'import socket,sys
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM,132); s.settimeout(2)
s.connect((sys.argv[1],2908))' "$_hub" 2>/dev/null && [ -z "${_hub_autres// }" ]; then
            say_end " -- " "$C_DIM" "Hub SS7 local" "$_hub:2908 deja en service"
        elif [ "$DRY" -eq 1 ]; then
            printf '  [dry-run] start-interstp.sh --ip %s --ops %s --ips "%s:%s" --no-save\n' "$_hub" "$_op"
             ↪  "$_noeud" "$_hub"
        elif [ "$_noeud" -ge 1 ] && [ "$_noeud" -le 9 ] && [ "$_op" -ge 1 ]; then
            if [ -n "${_hub_autres// }" ]; then
                say_begin "Hub SS7 local ($_hub) - table perimee (rctx ${_hub_autres% }), relance avec le noeud
                 ↪  $_noeud seul"
            else
                say_begin "Hub SS7 local ($_hub)"
            fi
            if bash "$HERE/launch/start-interstp.sh" --ip "$_hub" --ops "$_op" --ips "$_noeud:$_hub" --no-save \
                    > "${LOG_DIR:-/tmp}/interstp.log" 2>&1 < /dev/null; then
                say_end " OK " "$C_OK" "Hub SS7 local" "$_hub:2908, noeud $_noeud op $_op (rctx $_rctx)"
```

```
            else
                say_end " KO " "$C_KO" "Hub SS7 local" "voir ${LOG_DIR:-/tmp}/interstp.log"
            fi
        fi
    fi
fi

# --- 7quater-ter. The ASP local-ip follows the current address ----------------
# The asp-to-inter ASP binds to a local-ip written in osmo-stp.cfg. A DHCP
# lease that changes the node address makes that bind fail silently: as-inter
# stays AS_DOWN. At EVERY start the local address (STP_LOCAL_IP) is recomputed
# and rewritten if it moved, before run.sh starts the stack. In WAN,
# set_stp_ip.sh takes care of it (7quinquies); here, the standalone case.
#   STP_LOCAL_IP = the local leg that routes to the hub (remote-ip of the ASP);
#   if needed (hub unreachable) the OSMO_NODE_IP of mynode<N>.conf (N = node
#   of the routing-key, rctx = node*1000 + op*100 + 50).
if [ "$ACTION" = "start" ] && [ "$WAN_MESH" -ne 1 ]; then
    _stp_cfg="${OSMOCOM_CFG:-/etc/osmocom}/osmo-stp.cfg"
    if [ -w "$_stp_cfg" ]; then
        _hub="$(awk '/^ *asp asp-to-inter /{f=1;next} f&&/^ *remote-ip /{print $2; exit} f&&/^ (asp|as) /{exit}'
         ↪  "$_stp_cfg" 2>/dev/null)"
        _lo_now="$(awk '/^ *asp asp-to-inter /{f=1;next} f&&/^ *local-ip /{print $2; exit} f&&/^ (asp|as) /{exit}'
         ↪  "$_stp_cfg" 2>/dev/null)"
        _rctx="$(awk '/^ *as as-inter /{f=1;next} f&&/^ *routing-key /{print $2; exit} f&&/^ (asp|as) /{exit}'
         ↪  "$_stp_cfg" 2>/dev/null)"
        STP_LOCAL_IP=""
        [ -n "$_hub" ] && STP_LOCAL_IP="$(ip route get "$_hub" 2>/dev/null | sed -n 's/.*src \([0-9.]*\).*/\1/p' |
         ↪  head -1)"
        if [ -z "$STP_LOCAL_IP" ] && [[ "$_rctx" =~ ^[0-9]+$ ]]; then
            _nconf="${OSMOCOM_CFG:-/etc/osmocom}/mynode$(( _rctx / 1000 )).conf"
            [ -r "$_nconf" ] && STP_LOCAL_IP="$(awk -F= '/^OSMO_NODE_IP=/{gsub(/[ \r\t]/,"",$2);v=$2} END{print
             ↪  v}' "$_nconf" 2>/dev/null)"
        fi
        if [ -n "$STP_LOCAL_IP" ] && [ "$STP_LOCAL_IP" != "$_lo_now" ]; then
            if [ "$DRY" -eq 1 ]; then
                printf '  [dry-run] osmo-stp.cfg local-ip %s -> %s\n' "${_lo_now:-?}" "$STP_LOCAL_IP"
            else
                say_begin "Adresse SS7 locale"
                # Bounded to the asp-to-inter block: the 127.0.0.x local-ip of
                # the local listener are not touched.
                sed -i "/^ *asp asp-to-inter /,/^ *as /{s/^\( *local-ip *\).*/\1$STP_LOCAL_IP/}" "$_stp_cfg"
                say_end " OK " "$C_OK" "Adresse SS7 locale" "local-ip ${_lo_now:-?} -> $STP_LOCAL_IP"
            fi
        fi
    fi
fi

# --- 7sexies. The tcpdump wrapper ---------------------------------------------
# The GSMTAP capture is launched by run_modules/75-gsmtap.sh, in qosmo:
# another repo, not modified. It writes an UNBOUNDED pcap, and the live root
# being a tmpfs, a busy capture ends up filling the RAM.
#
# The wrapper substitutes a ring. It is also installed by build-iso.sh, but
# an ISO already burnt does not know it, and a live without persistence loses
# it at each reboot: reinstalling it HERE, at every stack start, makes it
# effective without rebuilding or touching qosmo.
#
# TWO TRAPS, TWO COUNTERMEASURES - the whole point of this script:
#   -Z root : with -C/-W, tcpdump creates the next ring members AFTER
#             dropping its privileges. Without it: "Permission denied", and
#             NO capture - whereas without -C it opened the file before.
#   the link: with -C/-W, tcpdump numbers (capture.pcap0, .pcap1...); the
#             exact name requested never exists, and the module's barrier -
#             a "test -s <exact name>" - concludes failure on a running
#             capture. <exact name> -> current member is maintained.
#
```

```
# OSMO_NO_PCAP_RING=1 skips the installation.
install_pcap_wrapper() {
    local dst=/usr/local/bin/tcpdump tmp
    [ "${OSMO_NO_PCAP_RING:-0}" = "1" ] && return 0
    [ "$(id -u)" -eq 0 ] || return 0
    command -v tcpdump >/dev/null 2>&1 || return 0
    # The real binary must exist under /usr/bin or /usr/sbin AND be an ELF: a
    # script in its place (see the guard in the wrapper) would make the
    # wrapper recursive. In that case install nothing and say so.
    local real=/usr/bin/tcpdump; [ -x "$real" ] || real=/usr/sbin/tcpdump
    if [ ! -x "$real" ] || [ "$(head -c 2 "$real" 2>/dev/null)" = "#!" ]; then
        printf '  %s!%s pcap : %s n est pas le binaire tcpdump - anneau non arme (apt-get install --reinstall
        ↪  tcpdump)\n' \
            "${C_WARN:-}" "$C_Z" "$real" >&2
        return 0
    fi

    tmp="$(mktemp)" || return 0
    cat > "$tmp" <<'PCAPWRAP'
#!/bin/sh
# tcpdump - osmo-operator wrapper: enforces a ring on file captures.
# Installed by start-native.sh and by build-iso.sh. See their comments.
# Tunable: OSMO_PCAP_RING_MB (32), OSMO_PCAP_RING_FILES (5).
REAL=/usr/bin/tcpdump
[ -x "$REAL" ] || REAL=/usr/sbin/tcpdump
# If REAL is itself a script (this wrapper copied over the dpkg binary, see
# iso_modules/52-qemu.sh), exec "$REAL" would loop forever: no capture, and a
# `tcpdump --version` that never returns.
if [ "$(head -c 2 "$REAL" 2>/dev/null)" = "#!" ]; then
    echo "tcpdump (wrapper) : $REAL n'est pas le vrai binaire - reinstallez le paquet tcpdump" >&2
    exit 127
fi

has_w=0; has_ring=0; has_z=0; wfile=""; next_is_w=0
for a in "$@"; do
    if [ "$next_is_w" = 1 ]; then wfile="$a"; next_is_w=0; continue; fi
    case "$a" in
        -w)            has_w=1; next_is_w=1 ;;
        -w?*)          has_w=1; wfile="${a#-w}" ;;
        -C|-C*|-W|-W*) has_ring=1 ;;
        -Z|-Z*)        has_z=1 ;;
    esac
done

# Nothing to bound, or the caller already chose its ring: step aside.
if [ "$has_w" != 1 ] || [ "$has_ring" = 1 ] || [ -z "$wfile" ]; then
    exec "$REAL" "$@"
fi

# The link watcher. Launched BEFORE the exec: afterwards this process IS
# tcpdump. $$ survives the exec, so it follows its life exactly and stops
# with it.
( ppid=$$
  n=0
  while [ "$n" -lt 300 ]; do
      [ -e "${wfile}0" ] && break
      kill -0 "$ppid" 2>/dev/null || exit 0
      sleep 0.2; n=$((n + 1))
  done
  while kill -0 "$ppid" 2>/dev/null; do
      newest=$(ls -t "${wfile}"[0-9]* 2>/dev/null | head -1)
      if [ -n "$newest" ] && [ "$(readlink "$wfile" 2>/dev/null)" != "$newest" ]; then
          ln -sfn "$newest" "$wfile"
      fi
      sleep 2
  done ) >/dev/null 2>&1 &
```

```
if [ "$has_z" = 1 ]; then
    exec "$REAL" -C "${OSMO_PCAP_RING_MB:-32}" -W "${OSMO_PCAP_RING_FILES:-5}" "$@"
fi
exec "$REAL" -Z root -C "${OSMO_PCAP_RING_MB:-32}" -W "${OSMO_PCAP_RING_FILES:-5}" "$@"
PCAPWRAP

    # The wrapper is named "tcpdump" and lives in /usr/local/bin, which
    # precedes /usr/bin: it must therefore NEVER be installed over the real
    # binary, on pain of infinite recursion. Check where the real one points.
    case "$(command -v tcpdump)" in
        "$dst") ;;                      # already the wrapper, replace it
        /usr/bin/tcpdump|/usr/sbin/tcpdump|/bin/tcpdump) ;;
        *) rm -f "$tmp"; return 0 ;;    # unexpected location: do nothing
    esac

    if cmp -s "$tmp" "$dst" 2>/dev/null; then
        rm -f "$tmp"                    # already up to date, nothing to say
    else
        install -m 0755 "$tmp" "$dst" 2>/dev/null \
            && printf '  %spcap%s       anneau de capture arme (%s)\n' \
                "$C_DIM" "$C_Z" "$dst"
        rm -f "$tmp"
    fi
}
[ "$DRY" -eq 1 ] || install_pcap_wrapper

# --- 7ter. SYSTEMATIC STOP BEFORE START ------------------------------------
# Every start stops the stack first; otherwise the deferred bridge launcher
# starts from an ambiguous state.
#
# The bridge's barrier waits for osmo-bts-trx to know that ports 5700-5702
# are its own. If a previous run is still running, its osmo-bts-trx is
# already there: the barrier is crossed at once, the bridge binds IN THE
# MIDDLE of run.sh's teardown, which kills it (f:pont/pont.py is in
# _td_patterns) - and nothing relaunches it. The barrier waits for both edges
# (disappearance then appearance), but starting from a stopped stack makes
# the question moot.
#
# It also frees the ports: an orphan fake_trx or trxcon is enough to make the
# next start fail on "UDP port 5720 already in use". The killall covers the
# Python processes the registry does not know.
#
# CALYPSO_NO_AUTOSTOP=1 disables this behaviour.
# SAP: stops the servers launched by start_sap (softSIM demo_server.rb
# processes) and removes the SAP socket. The [d] pattern avoids matching
# itself.
kill_softsim() {
    local p sock="${SAP_SOCK:-/tmp/osmocom_sap}"
    for p in $(ps -eo pid,args | awk '/[d]emo_server\.rb/ {print $1}'); do
        kill -TERM "$p" 2>/dev/null || true
    done
    rm -f "$sock" 2>/dev/null || true
}
```

2.224 /opt/GSM/osmo-operator/startup_scripts/13-adresses.sh

19240 octets, 353 lignes → 353 lignes

```
# 13-adresses.sh - part of start.sh (pre-start stop, PCU socket, APN NAT,
# private addresses, dashboard, node sheet, oFono modem, SAP)
# Sourced by start.sh in the main shell: same order, same variables.
if [ "$DRY" -eq 0 ] && [ "${CALYPSO_NO_AUTOSTOP:-0}" != 1 ]; then
    say_begin "Arret de la pile avant demarrage"
```

```
    banc_dsp_arreter >/dev/null 2>&1 || true
    banc_dsp_arreter_instances
    bash "$RUN_SH" --stop --profile "$CALYPSO_PROFILE" >/dev/null 2>&1 || true
    declare -F purge_sessions_tmux >/dev/null && purge_sessions_tmux
    if [ "${CALYPSO_STOP_KILL_PYTHON:-1}" != 0 ]; then
        killall_python -TERM || true
    fi
    # SAP: the softSIM servers (demo_server.rb) and their socket.
    # CALYPSO_STOP_KILL_SOFTSIM=0 disables it.
    if [ "${CALYPSO_STOP_KILL_SOFTSIM:-1}" != 0 ]; then
        kill_softsim || true
    fi
    say_end " OK " "$C_OK" "Arret de la pile avant demarrage"
fi

# --- 7sexies. PCU socket (BTS <-> osmo-pcu) -----------------------------------
# The sockets of a previous run stay on disk; the PCU must find the main
# BTS socket (/tmp/pcu_bts). The harness removes the orphans before launch and
# prints the state, so that a mute GPRS can be read.
PCU_SOCK_SH="$HERE/network/pcu-socket.sh"
if [ "$DRY" -eq 0 ] && [ -r "$PCU_SOCK_SH" ]; then
    say_begin "Socket PCU"
    if bash "$PCU_SOCK_SH" --fix > "${LOG_DIR:-/tmp}/pcu-socket.log" 2>&1; then
        say_end " OK " "$C_OK" "Socket PCU" "voir ${LOG_DIR:-/tmp}/pcu-socket.log"
    else
        say_end " KO " "$C_KO" "Socket PCU" "suppression d'un orphelin impossible - voir
        ↪  ${LOG_DIR:-/tmp}/pcu-socket.log"
    fi
fi

# --- 7septies. APN Internet egress (NAT apn0 -> routing NIC) ------------------
APN_NAT_SH="$HERE/network/apn-nat.sh"
if [ "$DRY" -eq 0 ] && [ -r "$APN_NAT_SH" ]; then
    say_begin "Sortie Internet APN (NAT)"
    if bash "$APN_NAT_SH" > "${LOG_DIR:-/tmp}/apn-nat.log" 2>&1; then
        say_end " OK " "$C_OK" "Sortie Internet APN" "voir ${LOG_DIR:-/tmp}/apn-nat.log"
    else
        say_end " KO " "$C_KO" "Sortie Internet APN" "voir ${LOG_DIR:-/tmp}/apn-nat.log"
    fi
fi

# --- 8. launch: exec run.sh ---------------------------------------------------
# ── THE ADDRESSES THE CONFIGS REQUIRE MUST EXIST ──────────────────────────────
# A daemon that binds to an absent address does not start, and says so in ITS
# log, not ours:
#     osmo-ggsn: "gtp bind-ip 192.168.2.10" -> address not found locally
#     osmo-sgsn: "listen 192.168.2.10 23000" -> same
# These addresses come from the template generator (op_private_ip), not from
# the network: nothing else sets them.
#
# osmo-ip-plan.sh EXTRACTS them from the configs rather than recomputing them
# (the plan may come from another generator), picks the NIC that really
# provides Internet, and falls back to loopback when none leads anywhere. It
# does nothing in an image of this repo (OSMO_IMAGE=1): there, start-docker.sh
# / compose does the addressing. Any other container is treated as native.
if [ "$ACTION" = "start" ] && [ "$DRY" -ne 1 ]; then
    IPPLAN="$HERE/network/osmo-ip-plan.sh"
    if [ -r "$IPPLAN" ]; then
        say_begin "Adresses privees du noeud"
        _ipout="$(OSMOCOM_CFG="${OSMOCOM_CFG:-/etc/osmocom}" \
                  OSMO_WAN_NODE="${NODE_ID:-${OSMO_WAN_NODE:-}}" \
                  bash "$IPPLAN" --apply 2>&1)" || true
        say_end " OK " "$C_OK" "Adresses privees du noeud" \
            "$(printf '%s' "$_ipout" | tail -1 | sed 's/^osmo-ip-plan : //')"
    fi
fi
```

```
# ── THE DASHBOARD RESTARTS WITH THE BENCH ──────────────────────────────────
# Symmetric to the `stop)` action: the unit is PartOf=osmo-banc.service, but a
# direct launch does not go through osmo-banc, so it is restarted here. The
# dashboard also serves /psd, which tools/osmo-fft-snap.py polls for the
# wallpaper FFT. (fft-web/fft_web.py serves 8081: another server, not
# concerned by this block.)
# --no-block, as at stop: do not block on systemd. `restart` rather than
# `start`: an instance surviving from a previous run is taken over instead
# of serving the previous state.
# CALYPSO_NO_DASHBOARD=1 disables this start.
if [ "$ACTION" = "start" ] && [ "$DRY" -ne 1 ] && [ "${CALYPSO_NO_DASHBOARD:-0}" != 1 ]; then
    say_begin "Tableau de bord (:8080)"
    if [ -d /run/systemd/system ] && command -v systemctl >/dev/null 2>&1; then
        if systemctl --no-block restart osmo-egprs-web.service 2>/dev/null; then
            say_end " OK " "$C_OK" "Tableau de bord (:8080)" "osmo-egprs-web.service"
        else
            say_end " -- " "$C_KO" "Tableau de bord (:8080)" "unite absente ou refusee"
        fi
    elif [ -r /opt/GSM/osmo-egprs-web/server.js ] && command -v node >/dev/null 2>&1; then
        # Without systemd (container): direct launch, as osmo-launch.sh does.
        pkill -f "osmo-egprs-web/server.js" 2>/dev/null || true
        ( cd /opt/GSM/osmo-egprs-web && setsid node server.js \
            >>"${LOG_DIR:-/tmp}/web-dashboard.log" 2>&1 </dev/null & )
        say_end " OK " "$C_OK" "Tableau de bord (:8080)" "node server.js"
    else
        say_end " -- " "$C_KO" "Tableau de bord (:8080)" "ni systemd ni node/server.js"
    fi
fi

# ── THE NODE SHEET: THE RESULT OF THE RUN ──────────────────────────────────
# /etc/osmocom/mynode<N>.conf (network/node-conf.sh): what another node must
# know to reach this one (--wan mynode<N>.conf). Written here, once the SS7
# identity, the address and the prefix are definitively set.
# start-multi.sh rewrites it once the containers are up (--gen-conf).
if [ "$ACTION" = "start" ] && [ "$DRY" -ne 1 ]; then
    . "$HERE/network/wan-nodes.sh" 2>/dev/null
    . "$HERE/network/node-conf.sh" 2>/dev/null
    _nc_node="${NODE_ID:-${WAN_NODE_ID:-}}"; [[ "${_nc_node:-0}" =~ ^[1-9]$ ]] || _nc_node=1
    [ "${WAN_NODE_COUNT:-0}" -gt 0 ] || wan_nodes_load "${WAN_CONF_FILE:-/etc/osmo-wan.conf}" 2>/dev/null || true
    _nc_out="$(node_conf_path "$_nc_node")"
    if node_conf_write "$_nc_out" "$_nc_node" "${NODE_OP:-1}" "${HUB_IP:-${OSMO_HUB_IP:-}}" 2>/dev/null; then
        say_end " OK " "$C_OK" "Fiche du noeud $_nc_node" "$_nc_out  (pour un autre noeud : --wan $(basename
        ↪  "$_nc_out"))"
    else
        say_end " -- " "$C_DIM" "Fiche du noeud" "non ecrite ($_nc_out)"
    fi
fi

# ── THE MOBILE LINK: oFONO = THE BENCH MODEM ───────────────────────────────
# The bench phone - the postmarketOS/Phosh VM - is the SAME subscriber as the
# osmocom-bb mobile: 100101. Its ModemManager only talks to an AT modem; this
# bench has no dongle, so the modem is written here. The VM connects directly
# to osmo-phonesim-banc.py (--connect mode, see tools/osmo-pmos.sh); oFono
# remains useful to the host - it provides osmo-call and osmo-sms-send, and a
# second AT modem for debugging.
#
#   tools/osmo-phonesim-banc.py   the bench AT modem, oFono side (TCP 12345,
#                                 phonesim plugin). It reads the network on
#                                 the osmo-bsc VTY, sends SMS through the
#                                 osmo-msc VTY, places calls through Asterisk,
#                                 and receives SMS as an SMPP ESME (port 2775).
#   tools/osmo-ril-atmodem.py     the AT modem that rild consumes, backed by
#                                 oFono. It publishes its pty in
#                                 /run/osmo-ril/at-pty.
#
```

```
# None of this is essential to the bench: if oFono is missing, say so and
# continue. The bench must start even without the Android phone.
if [ "$ACTION" = "start" ] && [ "$DRY" -ne 1 ] && [ "${OSMO_RACCORD_MOBILE:-1}" = "1" ]; then
    say_begin "Raccord mobile (oFono)"
    _rm_detail=""
    if ! command -v ofonod >/dev/null 2>&1; then
        say_end " -- " "$C_DIM" "Raccord mobile (oFono)" "ofono absent - ignore"
    else
        # 1. oFono must know where to find the bench modem.
        if ! grep -q '^\[osmo\]' /etc/ofono/phonesim.conf 2>/dev/null; then
            printf '\n[osmo]\nAddress=127.0.0.1\nPort=12345\n' >> /etc/ofono/phonesim.conf
            _rm_detail="phonesim.conf pose"
        fi
        # 2. the AMI account: the modem originates calls through it (the
        #    Asterisk CLI has no "channel originate" here, the module
        #    res_clioriginate is not loaded - modules.conf is autoload=no).
        if [ -d /etc/asterisk/manager.d ] && [ ! -f /etc/asterisk/manager.d/osmo.conf ]; then
            install -m640 "$HERE/configs/manager-osmo.conf" \
                /etc/asterisk/manager.d/osmo.conf 2>/dev/null && \
                { chown asterisk:asterisk /etc/asterisk/manager.d/osmo.conf 2>/dev/null || true; }
            asterisk -rx "manager reload" >/dev/null 2>&1 || true
            _rm_detail="${_rm_detail:+$_rm_detail, }compte AMI pose"
        fi
        # 3. the bench modem, THEN oFono (the order matters: ofonod looks for
        #    the modem at startup and does not come back to it on its own).
        pgrep -f "$HERE/tools/osmo-phonesim-banc.py" >/dev/null 2>&1 || \
        # LAUNCHED OUTSIDE THE JOB TABLE.
        # `setsid` detaches the SESSION, not the job: bash would keep these
        # helpers in its table and report their death ("line NNNN: <pid>
        # Killed setsid ...") over the prompt after a --stop. `disown` is not
        # reliable in a non-interactive shell. The subshell is: bash sees only
        # IT as a job, it returns immediately with 0, and the grandchild is
        # adopted by init - nobody watches or signals it any more.
        # `</dev/null` as well: none of these helpers reads the terminal.
        # THE MOBILE VTY IS NOT ALWAYS 4247.
        # osmo-phonesim-banc.py falls back to OSMO_MOB_VTY_PORT=4247, the value
        # of /root/.osmocom/bb/mobile.cfg. In --dsp the running mobile is the
        # bench's (mobile_pont.cfg, VTY 4347): pass it the port actually
        # bound, read from the file.
            ( setsid env OSMO_MOB_VTY_PORT="${_ms1_vty:-4247}" \
                  "$HERE/tools/osmo-phonesim-banc.py" \
                  </dev/null >>/tmp/osmo-phonesim.log 2>&1 & )
        sleep 1
        systemctl restart ofono >/dev/null 2>&1 || true
        sleep 3
        # 4. power the modem on: oFono leaves it off until asked.
        /usr/bin/python3 - <<'PYON' >/dev/null 2>&1 || true
from gi.repository import Gio, GLib
import time
c = Gio.bus_get_sync(Gio.BusType.SYSTEM, None)
for prop in ("Powered", "Online"):
    try:
        c.call_sync("org.ofono", "/osmo", "org.ofono.Modem", "SetProperty",
                    GLib.Variant("(sv)", (prop, GLib.Variant("b", True))),
                    None, Gio.DBusCallFlags.NONE, 25000, None)
    except GLib.Error:
        pass
    time.sleep(2)
PYON
        # 5. the virtual AT modem (the pty a RIL consumes, or a debugging
        #    modem for tools/at-cmd.py).
        #    /usr/bin/python3 HARD-CODED, not the "env python3" shebang: the
        #    bench PATH puts the venv (/root/.env/bin) first, whose interpreter
        #    has no GObject bindings; PyGObject is installed SYSTEM-side. Same
        #    reason as the PYON block above.
        pgrep -f "$HERE/tools/osmo-ril-atmodem.py" >/dev/null 2>&1 || \
```

```
        # LAUNCHED OUTSIDE THE JOB TABLE.
        # `setsid` detaches the SESSION, not the job: bash would keep these
        # helpers in its table and report their death ("line NNNN: <pid>
        # Killed setsid ...") over the prompt after a --stop. `disown` is not
        # reliable in a non-interactive shell. The subshell is: bash sees only
        # IT as a job, it returns immediately with 0, and the grandchild is
        # adopted by init - nobody watches or signals it any more.
        # `</dev/null` as well: none of these helpers reads the terminal.
            ( setsid /usr/bin/python3 "$HERE/tools/osmo-ril-atmodem.py" \
                  </dev/null >>/tmp/osmo-ril.log 2>&1 & )
        sleep 1
        if pgrep -f "$HERE/tools/osmo-phonesim-banc.py" >/dev/null 2>&1; then
            say_end " OK " "$C_OK" "Raccord mobile (oFono)" \
                "modem /osmo${_rm_detail:+ - $_rm_detail} ; pty RIL : /run/osmo-ril/at-pty"
        else
            say_end " -- " "$C_DIM" "Raccord mobile (oFono)" "modem du banc non demarre (cf.
             ↪  /tmp/osmo-phonesim.log)"
        fi
    fi
fi

# ── THE DSP BENCH: MEASURED SETTINGS, THEN HAND-OFF ──────────────────────
# The values come from the bench (c54x_exe/run_real.sh; run.sh documents them
# one by one):
#   INSNS                    instruction budget per frame (see 14-demarrage.sh)
#   LOCKSTEP=1               QEMU only advances the frame once the DSP has
#                            finished the previous one; otherwise QEMU skips
#                            3 frames out of 4
#   IQ=none                  no synthetic cell: the bursts come from the real
#                            BTS, through pont.py
#   CALYPSO_BSP_STREAM=1     one TS0 burst per frame, in FN order
#   CALYPSO_RHEA_DMA_XFER=1  without it the API page is never filled: no SB,
#                            no BCCH, no SI
# Everything stays overridable: only what the operator did not set is set.

# ── SAP: THE SIM SERVED TO THE MOBILE OVER THE SAP SOCKET (--sap) ────────
# demo_server.rb server of the softSIM repo (/opt/GSM/softsim, Ruby 3.2+ shim:
# File.exists?, Fixnum), launched BEFORE the stack: the mobile (sim sap)
# connects to it at startup and stays (patch 0002: the server serves the next
# client after a disconnection, so a baseband restart needs nothing).
#   type pcsc : the card in the PC/SC reader (HID Omnikey 6121,
#               sysmoISIM-SJA5; gem smartcard). The card must carry the
#               IMSI/Ki of MS#1 (or its subscriber must be in the HLR):
#               tools/osmo-sim.sh verifier.
#   type sim  : the software SIM sim-op1.xml, regenerated at every launch
#               (IMSI of MS#1; COMP128v1 with SOFTSIM_KI = Ki of MS#1, patch 0001).
# In DSP, the running MS#1 is c54x_exe's (mobile_pont.cfg, sim test):
# $BB_DIR/mobile_pont_sap.cfg (sim sap, sap-socket) is derived from it and
# the bench receives it through MOBILE_CFG (SAP_DSP_MOBILE_CFG). Variables:
# SOFTSIM_DIR, SAP_SOCK, SAP_TYPE, SAP_FILE, SAP_LOG, SAP_VERBOSITY, SOFTSIM_KI.
# The python that has pyscard: the pySim venv's (root's PATH starts with
# /root/.env, the bridge venv, without pyscard), otherwise the system's.
sap_python() {
    local p="${PYSIM_VENV:-/opt/GSM/pysim-venv}/bin/python3"
    [ -x "$p" ] || p=/usr/bin/python3
    printf '%s' "$p"
}
sap_carte_presente() {
    "$(sap_python)" - <<'PYSC' 2>/dev/null
import sys
try:
    from smartcard.System import readers
except Exception:
    sys.exit(2)
for r in readers():
    try:
```

```
        c = r.createConnection(); c.connect(); c.disconnect(); sys.exit(0)
    except Exception:
        pass
sys.exit(1)
PYSC
}
sap_lecteur() {
    "$(sap_python)" -c 'from smartcard.System import readers; print(str(readers()[0]))' 2>/dev/null | sed "s/
     ↪  \[.*//"
}
SAP_DSP_MOBILE_CFG=""
start_sap() {
    [ "${SAP_MODE:-0}" = 1 ] || return 0
    local sdir="${SOFTSIM_DIR:-/opt/GSM/softsim}" sock="${SAP_SOCK:-/tmp/osmocom_sap}"
    local log="${SAP_LOG:-/tmp/osmo-sap.log}" type="${SAP_TYPE:-auto}" file="" detail=""
    local shim=/usr/local/share/softsim/ruby_compat.rb
    [ -f "$shim" ] || shim="$HERE/tools/softsim/ruby_compat.rb"
    say_begin "SIM via SAP"
    if [ ! -f "$sdir/src/demo_server.rb" ] || ! command -v ruby >/dev/null 2>&1; then
       say_end " KO " "$C_KO" "SIM via SAP" "softSIM ou ruby absent ($sdir) : sudo $HERE/tools/osmo-sim-install.sh"
        return 1
    fi
    if [ "$type" = auto ]; then
        if sap_carte_presente; then type=pcsc; else type=sim; fi
    fi
    case "$type" in
        pcsc)
            if ! sap_carte_presente; then
                say_end " KO " "$C_KO" "SIM via SAP" "aucune carte dans un lecteur PC/SC (pcscd actif ? Omnikey
                 ↪  branche ? tools/osmo-sim.sh lecteur)"
                return 1
            fi
            if ! ruby -e 'require "smartcard"' >/dev/null 2>&1; then
               say_end " KO " "$C_KO" "SIM via SAP" "gem smartcard absent : sudo $HERE/tools/osmo-sim-install.sh"
                return 1
            fi
            detail="carte du lecteur $(sap_lecteur)" ;;
        sim)
            file="${SAP_FILE:-$sdir/src/sim-op1.xml}"
            # the Ki of MS#1, the one 21-abonnes-hlr puts in the HLR (COMP128v1)
            export SOFTSIM_KI="${SOFTSIM_KI:-$(ms_ki 1 | tr -d ' ')}"
            detail="SIM logicielle $file (IMSI $(ms_imsi 1))" ;;
        *)
            say_end " KO " "$C_KO" "SIM via SAP" "SAP_TYPE=$type inconnu (pcsc, sim, auto)"
            return 1 ;;
    esac
    # In DSP: the "sim sap" copy of mobile_pont.cfg, for the bench only.
    if [ "${DSP_BANC:-0}" = 1 ]; then
        local _src="${MOBILE_CFG:-$(dirname "$BANC_DSP")/mobile_pont.cfg}" _dst="$BB_DIR/mobile_pont_sap.cfg"
        if [ -r "$_src" ]; then
            sed -e "s#^\([[:space:]]*\)sim \(test\|reader\)\$#\1sim sap#" \
                -e "s|^\([[:space:]]*\)sap-socket .*|\1sap-socket $sock|" \
               -e "s|^\([[:space:]]*\)stick [0-9]*\$|\1stick ${PLAN_ARFCN:-${ARFCN:-1}}|" "$_src" > "$_dst.tmp" \
                && mv -f "$_dst.tmp" "$_dst"
            grep -q "^[[:space:]]*sap-socket " "$_dst" \
                || sed -i "s|^\([[:space:]]*\)layer2-socket .*|&\n\1sap-socket $sock|" "$_dst"
            SAP_DSP_MOBILE_CFG="$_dst"
            detail="$detail ; MS#1 $_dst"
        else
            say_end " KO " "$C_KO" "SIM via SAP" "config du MS#1 du banc DSP illisible : $_src"
            return 1
        fi
    fi
    if [ $DRY -eq 1 ]; then
        say_end " -- " "$C_DIM" "SIM via SAP" "dry-run : type $type, ruby -r$shim demo_server.rb --unix
         ↪  $sock${file:+ --file $file} ($detail)"
```

```
        return 0
    fi
    # sim-op1.xml is regenerated at every launch: the server rewrites it at
    # every client disconnection, and an old file would keep the previous
    # ICCID / PLMN.
    if [ "$type" = sim ] && [ -z "${SAP_FILE:-}" ] && [ -f "$sdir/src/sim.xml" ] && [ -f
     ↪  "$HERE/tools/softsim/make_sim_op1.py" ]; then
        python3 "$HERE/tools/softsim/make_sim_op1.py" "$sdir/src/sim.xml" "$sdir/src/sim-op1.xml" \
            --imsi "$(ms_imsi 1)" >/dev/null 2>&1 || true
        [ -f "$file" ] || file="$sdir/src/sim.xml"
    fi
    rm -f "$sock"
    local args=(--type "$type" --socket unix --unix "$sock" --verbosity "${SAP_VERBOSITY:-3}")
    [ "$type" = sim ] && args+=(--file "$file")
    ( cd "$sdir/src" && nohup ruby -r"$shim" demo_server.rb "${args[@]}" >"$log" 2>&1 & )
    for _ in $(seq 1 50); do [ -S "$sock" ] && break; sleep 0.2; done
    if [ -S "$sock" ]; then
        say_end " OK " "$C_OK" "SIM via SAP" "type $type : $detail ; socket $sock, journal $log"
    else
        say_end " KO " "$C_KO" "SIM via SAP" "pas de socket $sock : voir $log"
        return 1
    fi
}
start_sap || { echo "SAP demande (--sap) mais non lance" >&2; exit 1; }
```

2.225 /opt/GSM/osmo-operator/startup_scripts/14-demarrage.sh

9966 octets, 158 lignes → 158 lignes

```
# 14-demarrage.sh - part of start.sh (hand-off to run.sh and the DSP bench)
# Sourced by start.sh in the main shell: same order, same variables.
if [ "$DSP_BANC" = 1 ]; then
    # ── THE FORK BRINGS UP EVERYTHING EXCEPT THE CALYPSO CHAIN ──────────────
    # The fork runs its usual plan; only the four modules that c54x_exe
    # provides (qemu, pty, osmocon, l2) are skipped, and the DSP bench is then
    # launched with its five steps. --no-attach is mandatory: without it the
    # fork attaches to tmux and control never comes back here.
    #
    # The bench settings come from the bench itself (c54x_exe/run_real.sh):
    #   INSNS                   instruction budget per frame (see below)
    #   LOCKSTEP=1              QEMU only advances the frame once the DSP has
    #                           finished the previous one; otherwise it skips
    #                           3 frames out of 4
    #   IQ=none                 no synthetic cell: the bursts come from the
    #                           real BTS, through pont.py
    #   CALYPSO_BSP_STREAM=1    one TS0 burst per frame, in FN order
    #   CALYPSO_RHEA_DMA_XFER=1 without it the API page is never filled:
    #                           no SB, no BCCH, no SI
    # Everything stays overridable: only what the operator did not set is set.
    export MODE=dsp PONT=1
    # --osmo-trx: transceiver instead of mobile, no bridge (c54x_exe/run.sh
    # forces PONT=0 itself; it is set here for the dry-run).
    if [ "${OSMO_TRX:-0}" = 1 ]; then export OSMO_TRX PONT=0; fi
    # ARM asm trace, read by c54x_exe/run.sh (step 2, QEMU -d).
    export ASSEMBLY_LOGS="${ASSEMBLY_LOGS:-0}" ASSEMBLY_LOGS_FLAGS ASSEMBLY_LOGS_FILTRE
    : "${IQ:=none}";        export IQ
    # INSNS is only a ceiling (the DSP stops at its IDLE, c54x_exe/src/pont.c
    # jouer_trame). TCH decoding needs up to ~87000 insn/frame with osmocom
    # layer1; below that the decoding spills over the next frame and corrupts
    # the speech Viterbi state. The real C54x has ~480000 cycles per frame.
    # Magnetite (--ms-magnetite): the TI firmware needs far more than osmocom
    # in TCH: at 120000 the DSP task queue overflows (d_error_status 20h/28h),
    # L1 sees its results late, restarts the DSP and the call to 600 drops
    # (Assignment Failure). 480000 leaves no unfinished tick.
```

```
if [ -n "${MS_MAGNETITE:-}" ]; then : "${INSNS:=480000}"; fi
: "${INSNS:=120000}";   export INSNS
: "${LOCKSTEP:=1}";     export LOCKSTEP
: "${CALYPSO_BSP_STREAM:=1}";    export CALYPSO_BSP_STREAM
: "${CALYPSO_RHEA_DMA_XFER:=1}"; export CALYPSO_RHEA_DMA_XFER
# TCH-VOICE in l1phy (ROM vocoder), not in gapk. The ROM profile of
# /etc/default/osmo-banc (scripts/dsp-rom-max.sh: BANC_VOCODEUR=rom,
# CALYPSO_VSP=1 and the vocoder emulator flags) is loaded here so that a
# manual launch gets the same profile as osmo-banc.service; the
# environment always wins. l1phy is the default: run.sh derives
# mobile_pont_rom.cfg from it (config_vocodeur_rom).
# BANC_VOCODEUR=hote switches back to gapk.
if [ -r /etc/default/osmo-banc ]; then
    while IFS='=' read -r _k _v; do
        case "$_k" in ''|\#*|*[!A-Za-z0-9_]*) continue ;; esac
        [ -n "${!_k+x}" ] || export "$_k=$_v"
    done < /etc/default/osmo-banc
    unset _k _v
fi
: "${BANC_VOCODEUR:=rom}"; export BANC_VOCODEUR
# --gapk with --ms-magnetite: no L23 mobile for GAPK, the voice is coded by
# the bridge (pont/voix_hote.py, libgsm); the DSP only does the channel
# (CALYPSO_VSP=0).
if [ "$BANC_VOCODEUR" = hote ] && [ -n "${MS_MAGNETITE:-}" ]; then
    : "${PONT_VOIX_HOTE:=1}"; export PONT_VOIX_HOTE
fi
: "${CALYPSO_VSP:=1}";     export CALYPSO_VSP
# The rest of the --vocodeur-rom profile, even without
# /etc/default/osmo-banc: the FR codec emulator flags and PONT_UL_RAW_TCH.
# Without them the ROM receives the microphone but codes the silence
# frame, and the 600 echo test stays mute. Same values as
# scripts/dsp-rom-max.sh profile --vocodeur-rom (the default).
if [ "$BANC_VOCODEUR" = rom ]; then
    for _kv in CALYPSO_NORM_TC=0 CALYPSO_C54X_BCTC_NORM=1 PONT_UL_RAW_TCH=1 \
               CALYPSO_C54X_BCD_CARRY=1 CALYPSO_C54X_OVM_LOGIQUE=1 CALYPSO_MAC=1 \
               CALYPSO_C54X_BCD_TC=1 CALYPSO_C54X_BCTC_COMBO=1; do
        _k="${_kv%%=*}"; [ -n "${!_k+x}" ] || export "$_kv"
    done
    unset _kv _k
fi
RUN_ARGS+=(--skip "$DSP_MODULES_RETIRES" --no-attach)
if [ $DRY -eq 1 ]; then
    say_begin "Transmission a run.sh puis au banc DSP"
    say_end " -- " "$C_DIM" "Transmission a run.sh puis au banc DSP" "dry-run"
    printf '  1. CALYPSO_PROFILE=%s bash %s %s\n' "$CALYPSO_PROFILE" "$RUN_SH" "${RUN_ARGS[*]}"
    printf '  2. MODE=dsp PONT=%s IQ=%s INSNS=%s LOCKSTEP=%s BANC_VOCODEUR=%s%s%s bash %s\n' \
        "$PONT" "$IQ" "$INSNS" "$LOCKSTEP" "$BANC_VOCODEUR" \
        "${QEMU_ICOUNT:+ QEMU_ICOUNT=$QEMU_ICOUNT}" \
        "$([ "${ASSEMBLY_LOGS:-0}" = 1 ] && echo " ASSEMBLY_LOGS=1")" "$BANC_DSP"
    [ "${OSMO_TRX:-0}" = 1 ] && printf '     OSMO_TRX=1 : trx.highram.elf + transceiver --no-sync (pas de
    ↪  mobile, pas de pont) ; %s\n' "$(trx_ts_desc)"
    [ "${OSMO_TRX_TS1:-0}" = 1 ] && printf '     OSMO_TRX_TS1=1 : INSTANCE=3 OSMO_TRX_PHONE=2 bash %s   (le 2e
    ↪  Calypso TRX, TS1, avant le transceiver)\n' "$BANC_DSP"
    # Classic BTS (--bts-classic or default): the mobile is on the bridge,
    # Magnetite through FIRMWARE_FLASH.
    [ "${OSMO_TRX:-0}" != 1 ] && printf '     BTS classique : osmo-bts-trx <-> pont_dsp.py ; mobile %s\n' \
        "$([ -n "${MS_MAGNETITE:-}" ] && echo "Magnetite (FIRMWARE_FLASH=$MAGNETITE_FLASH)" || echo
        ↪  "osmocom-bb (mobile + layer1)")"
    [ "${OSMO_TRX:-0}" = 1 ] && [ "${MS_AIR:-0}" = 1 ] && printf '  3. INSTANCE=2 AIR=1 MODE=dsp %sbash %s
    ↪  (le mobile par l air, quand le firmware trx est en mode BTS)\n'
    ↪  "${MS_MAGNETITE:+FIRMWARE_FLASH=$MAGNETITE_FLASH }" "$BANC_DSP"
    exit 0
fi
# The instance 1 mobile reads mobile_pont.cfg as is (run.sh only rewrites
# the stick for instances 2+): align its stick ARFCN with the plan's cell,
# otherwise FBSB fails in a loop in the multi-operator containers.
```

```
    _cfg_pont="$(dirname "$BANC_DSP")/mobile_pont.cfg"
    _arfcn_pont="${PLAN_ARFCN:-${MS_ARFCN1:-${ARFCN:-1}}}"
    if [ -w "$_cfg_pont" ] && [[ "$_arfcn_pont" =~ ^[0-9]+$ ]]; then
        sed -i "s|^\([[:space:]]*\)stick [0-9]*\$|\1stick ${_arfcn_pont}|" "$_cfg_pont"
    fi
    # Same template, same gap for the SIM: imsi/ki/rplmn are taken from
    # mobile.cfg, the card that start-docker generates per operator, so the HLR
    # of the operator's container knows the IMSI with its auth data.
    _sim_src="/root/.osmocom/bb/mobile.cfg"
    if [ -w "$_cfg_pont" ] && [ -r "$_sim_src" ]; then
        for _k in imsi "ki comp128" rplmn; do
            _l="$(grep -m1 -E "^[[:space:]]+${_k} " "$_sim_src")" || continue
            _l="${_l//|/\\|}"
            sed -i "s|^[[:space:]]\+${_k} .*\$|${_l}|" "$_cfg_pont"
        done
    fi
    unset _cfg_pont _arfcn_pont _sim_src _k _l
    audio_start
    say_begin "Transmission a run.sh (sans $DSP_MODULES_RETIRES)"
    if ! env CALYPSO_PROFILE="$CALYPSO_PROFILE" bash "$RUN_SH" "${RUN_ARGS[@]}"; then
        say_end " KO " "$C_KO" "Transmission a run.sh" "le plan du fork a echoue, banc DSP non lance"
        exit 1
    fi
    say_end " OK " "$C_OK" "Transmission a run.sh" "profil=$CALYPSO_PROFILE, chaine Calypso laissee au banc DSP"
    say_begin "Banc DSP C54x"
    say_end " OK " "$C_OK" "Banc DSP C54x" "$BANC_DSP  (INSNS=$INSNS LOCKSTEP=$LOCKSTEP IQ=$IQ$([ "${OSMO_TRX:-0}"
     ↪  = 1 ] && echo " OSMO_TRX=1 : trx.highram.elf + transceiver, $(trx_ts_desc)"))"
    # Full hand-off: this process becomes c54x_exe/run.sh (five steps, then it
    # returns; the logs stay in /tmp/c54x-pont). MODE=dsp is exported just
    # above and overrides the profile's.
    # --sap: the "sim sap" copy of mobile_pont.cfg, for the bench only.
    # --bts --ms: instance 1 (the BTS radio) first, then, once the trx
    # firmware has switched to BTS mode (osmo-bts-trx sent POWERON:
    # "BTS MODE: 7 514" in osmocon.log), instance 2 (the mobile + the air),
    # which takes over.
    if [ "${OSMO_TRX:-0}" = 1 ] && [ "${MS_AIR:-0}" = 1 ]; then
        bash "$BANC_DSP" || exit 1
        say_begin "Mobile par l'air (instance 2)"
        _ok=0
        for _i in $(seq 1 "${OSMO_TRX_BTS_ATTENTE:-150}"); do
            grep -aq 'BTS MODE: [0-9]* [1-9]' /tmp/c54x-pont/osmocon.log 2>/dev/null && { _ok=1; break; }
            sleep 1
        done
        if [ "$_ok" = 1 ]; then
            say_end " OK " "$C_OK" "Mobile par l'air (instance 2)" "firmware trx en mode BTS apres ${_i}s ;
             ↪  INSTANCE=2 AIR=1 $BANC_DSP"
        else
            say_end " !! " "$C_KO" "Mobile par l'air (instance 2)" "le firmware trx n'est pas en mode BTS
             ↪  (osmo-bts-trx absente ?) ; on lance quand meme"
        fi
        sleep 10
        # Explicit OSMO_TRX=0: OSMO_TRX=1 is exported (the TRX is instance 1)
        # and instance 2 must not inherit it (it would start a second
        # transceiver on the same UDP port). The over-the-air mobile runs the
        # stock layer1 (the profile's FIRMWARE_ELF is dropped; an explicit
        # FIRMWARE_SRC keeps precedence).
        # --ms-magnetite: FIRMWARE_FLASH (exported at parsing) replaces layer1
        # for instance 2.
        exec env -u FIRMWARE_ELF -u FIRMWARE_BIN ${FIRMWARE_SRC:+FIRMWARE_ELF="$FIRMWARE_ELF"} \
            ${MS_MAGNETITE:+FIRMWARE_FLASH="$MAGNETITE_FLASH"} \
            INSTANCE=2 AIR=1 OSMO_TRX=0 ${SAP_DSP_MOBILE_CFG:+MOBILE_CFG="$SAP_DSP_MOBILE_CFG"} bash "$BANC_DSP"
    fi
    exec env ${MS_MAGNETITE:+FIRMWARE_FLASH="$MAGNETITE_FLASH"}
     ↪  ${SAP_DSP_MOBILE_CFG:+MOBILE_CFG="$SAP_DSP_MOBILE_CFG"} bash "$BANC_DSP"
fi
```

2.226 /opt/GSM/osmo-operator/startup_scripts/15-fin.sh

554 octets, 13 lignes → 13 lignes

```
# 15-fin.sh - part of start.sh (hand-off to run.sh)
# Sourced by start.sh in the main shell: same order, same variables.
[ $DRY -eq 1 ] || audio_start
say_begin "Transmission a run.sh"
if [ $DRY -eq 1 ]; then
    say_end " -- " "$C_DIM" "Transmission a run.sh" "dry-run"
    printf '  commande : CALYPSO_PROFILE=%s bash %s %s\n' \
        "$CALYPSO_PROFILE" "$RUN_SH" "${RUN_ARGS[*]}"
    exit 0
fi
say_end " OK " "$C_OK" "Transmission a run.sh" "profil=$CALYPSO_PROFILE"
# Full hand-off: this process becomes run.sh
exec bash "$RUN_SH" "${RUN_ARGS[@]}"
```

2.227 /opt/GSM/osmo-operator/tools/at-cmd.py

5183 octets, 141 lignes → 141 lignes

```
#!/usr/bin/env python3
# at-cmd.py - TALK TO THE AT MODEM BY HAND.
#
# Counterpart of tools/osmo-ril-atmodem.py: that one plays the modem role
# (backed by oFono), this one plays the RIL role. It is the tool to check,
# without Android and without rild, that the virtual modem answers what it
# should - and also what gets plugged on a REAL modem (an EC25 on /dev/ttyUSB2)
# to compare the answers.
#
#   at-cmd.py                     opens an interactive session
#   at-cmd.py AT+CREG?            one command, the answer, exit
#   at-cmd.py -d /dev/ttyUSB2 AT+CSQ   on a physical modem
#   at-cmd.py --sms 100102 "salut"     the complete AT+CMGS dialogue (with the ^Z)
#   at-cmd.py --call 100102            ATD100102;
#
# By default it talks to the virtual modem, whose link is published in
# /run/osmo-ril/at-pty.
import argparse
import os
import sys
import termios
import time

DEV = os.environ.get("OSMO_AT_DEV", "/run/osmo-ril/at-pty")


def open_port(dev, baud=115200):
    fd = os.open(dev, os.O_RDWR | os.O_NOCTTY)
    try:                                    # un pty n a pas de vitesse, un tty si
        attrs = termios.tcgetattr(fd)
        attrs[0] = attrs[1] = attrs[3] = 0          # iflag, oflag, lflag : brut
        attrs[2] = termios.CS8 | termios.CREAD | termios.CLOCAL
        speed = getattr(termios, "B%d" % baud, termios.B115200)
        attrs[4] = attrs[5] = speed
        termios.tcsetattr(fd, termios.TCSANOW, attrs)
    except termios.error:
        pass
    return fd


def send(fd, line, wait=1.0, quiet=False):
```

```
    """Sends a command and returns everything that arrives until silence."""
    os.write(fd, (line + "\r\n").encode())
    return read_until(fd, wait, quiet)


def read_until(fd, wait, quiet=False):
    """Reads until OK/ERROR, or until the delay runs out."""
    import select
    out, end = "", time.time() + wait
    while time.time() < end:
        r, _, _ = select.select([fd], [], [], 0.1)
        if not r:
            continue
        try:
            chunk = os.read(fd, 4096).decode(errors="replace")
        except OSError:
            break
        if not chunk:
            break
        out += chunk
        if not quiet:
            sys.stdout.write(chunk)
            sys.stdout.flush()
                # Final answer: no need to wait for the rest of the delay.
        for fin in ("\r\nOK\r\n", "\r\nERROR\r\n", "+CME ERROR", "+CMS ERROR"):
            if out.endswith(fin) or fin in out[-40:]:
                return out
    return out


def main():
    ap = argparse.ArgumentParser(description="client AT (modem virtuel ou vrai modem)")
    ap.add_argument("cmd", nargs="*", help="commande AT (sinon : session interactive)")
    ap.add_argument("-d", "--device", default=DEV, help="port (defaut %s)" % DEV)
    ap.add_argument("-t", "--timeout", type=float, default=2.0, help="delai de lecture")
    ap.add_argument("--sms", nargs=2, metavar=("NUMERO", "TEXTE"),
                    help="envoie un SMS : AT+CMGF=1, AT+CMGS, le texte, puis Ctrl-Z")
    ap.add_argument("--call", metavar="NUMERO", help="ATD<numero>;")
    ap.add_argument("--hangup", action="store_true", help="ATH")
    ap.add_argument("--status", action="store_true",
                    help="l etat en une passe : CPIN, CFUN, CREG, COPS, CSQ, CLCC")
    a = ap.parse_args()

    if not os.path.exists(a.device):
        print("[at-cmd] %s introuvable - osmo-ril-atmodem.py tourne ?" % a.device,
              file=sys.stderr)
        return 1
    try:
        fd = open_port(a.device)
    except OSError as e:
        print("[at-cmd] %s : %s" % (a.device, e), file=sys.stderr)
        return 1

    try:
        if a.status:
            for c in ("AT", "AT+CPIN?", "AT+CFUN?", "AT+CREG?", "AT+COPS?",
                      "AT+CSQ", "AT+CLCC"):
                print("--- %s" % c)
                send(fd, c, a.timeout)
            return 0
        if a.sms:
            num, text = a.sms
            send(fd, "AT+CMGF=1", a.timeout)
            print('--- AT+CMGS="%s"' % num)
            os.write(fd, ('AT+CMGS="%s"\r\n' % num).encode())
            read_until(fd, 1.0)                     # l invite « > »
```

```
            os.write(fd, (text + "\x1a").encode())  # le texte puis Ctrl-Z
            read_until(fd, max(a.timeout, 5.0))
            return 0
        if a.call:
            send(fd, "ATD%s;" % a.call, a.timeout)
            return 0
        if a.hangup:
            send(fd, "ATH", a.timeout)
            return 0
        if a.cmd:
            send(fd, " ".join(a.cmd), a.timeout)
            return 0

                # interactive session
        print("[at-cmd] %s - une commande par ligne, Ctrl-D pour sortir" % a.device)
        while True:
            try:
                line = input("AT> ").strip()
            except (EOFError, KeyboardInterrupt):
                print()
                return 0
            if not line:
                continue
            if line.lower() in ("quit", "exit"):
                return 0
            if not line.upper().startswith("AT"):
                line = "AT" + line
            send(fd, line, a.timeout)
    finally:
        os.close(fd)


if __name__ == "__main__":
    sys.exit(main())
```

2.228 /opt/GSM/osmo-operator/tools/conky-osmo-status.sh

28553 octets, 498 lignes → 498 lignes

```
#!/bin/bash
# conky-osmo-status.sh - the lines of the bench Conky (configs/conky/osmo-conky.conf)
#   role | vue | net | hub | core | radio | lte | subs | services
# Every output is conky text (execpi): ${color2} green, ${color3} red,
# ${color4} yellow. Fast and without hard dependency: whatever is missing shows "-".
set -u
# GLYPHS IN DEJAVU SANS, TEXT IN UBUNTU. The Conky is entirely in Ubuntu
# (osmo-conky.conf), which has neither ● ○ nor ↓ ↑ ◄ ► ☺: they render as
# squares. Each glyph is therefore wrapped in a switch to DejaVu Sans (FS)
# followed by a return to the default font (FU = ${font}, i.e. Ubuntu).
# AT THE SAME SIZE (9) as the default font, and that matters: Conky computes
# its window height with the default font but DRAWS each line at the height
# of the tallest font met on it. With DejaVu at 8 (or 7, or Mono) every line
# with dots is drawn 6 px lower than computed, and after twenty lines the
# bottom of the Conky falls out of the window (21 px per line for 15
# computed). At 9 the metrics match and the window fits.
FS="\${font ${OSMO_CONKY_FS:-DejaVu Sans:size=9}}"; FU='${font}'
OK="${FS}\${color2}●\${color}${FU}"; KO="${FS}\${color3}○\${color}${FU}"; WARN="${FS}\${color4}●\${color}${FU}"
# One smiley for EACH stop of the bench (op1, op2, op3, hub, 4G): green smile
# when the stop is running, grey frown when it is down.
SMILE="${FS}\${color2}☺\${color}${FU}"; SAD="${FS}\${color5}☹\${color}${FU}"
C1='${color1}'; C='${color}'; C2='${color2}'; AR='${alignr}'
have() { command -v "$1" >/dev/null 2>&1; }
# ── pgrep -x COMPARES AGAINST THE NAME TRUNCATED TO 15 CHARACTERS ──────────
# The kernel keeps only 15 characters of process name (comm): osmo-sip-connector
```

```
# is « osmo-sip-connec », proto-smsc-daemon is « proto-smsc-daem ». `pgrep -x`
# with the full name never matches, so the pattern is truncated the same way.
# ── THE OPERATOR SELECTED IN THE PANEL ──────────────────────────────────────
# tools/osmo-panel.py (arrows < >) writes /run/osmo-fft/operator:
# OP=, MODE=native|docker, IP=, NAME=osmo-operator-N. On a container, every
# probe (pgrep, port, sqlite) runs INSIDE it (docker exec): the Conky follows
# the panel. Without the file, or in native mode: the local machine.
OP_FILE="${OSMO_FFT_DIR:-/run/osmo-fft}/operator"
OP_ID=1; OP_MODE=native; OP_NAME=""
if [ -r "$OP_FILE" ]; then
    OP_ID="$(sed -n 's/^OP=//p' "$OP_FILE" | head -1)"; OP_ID="${OP_ID:-1}"
    OP_MODE="$(sed -n 's/^MODE=//p' "$OP_FILE" | head -1)"; OP_MODE="${OP_MODE:-native}"
    OP_NAME="$(sed -n 's/^NAME=//p' "$OP_FILE" | head -1)"
fi
# in_op CMD...: run the command here, or inside the operator's container.
if [ "$OP_MODE" = docker ] && [ -n "$OP_NAME" ]; then
    in_op() { docker exec "$OP_NAME" "$@" 2>/dev/null; }
else
    in_op() { "$@"; }
fi
port_open() {
    if [ "$OP_MODE" = docker ]; then in_op timeout 1 bash -c "echo >/dev/tcp/127.0.0.1/$1" 2>/dev/null
    else timeout 1 bash -c "echo >/dev/tcp/127.0.0.1/$1" 2>/dev/null; fi
}
alive_x() { in_op pgrep -x "${1:0:15}" >/dev/null 2>&1; }
# ── IS THE BENCH UP? ────────────────────────────────────────────────────────
# Without a bench, the « Coeur GSM », « Radio / MS », « Abonnes » and
# « Services » sections would show thirty red dots - a wall of alarms saying
# one thing: nothing runs. That hides what actually matters when the bench
# is down: the network, the disk space, the machine.
#
# When the stack is down these sections therefore render NOTHING: the Conky
# falls back on the machine, and the titles stay as landmarks. The Network
# section is the only one that ALWAYS speaks - it depends on no operator, and
# it is precisely what one looks at when the bench does not come up.
#
# The criterion: the STP, BSC or MSC of the displayed operator. These are the
# processes that exist as soon as the core is up, in both worlds (native and
# container) - and in_op looks for them in the right place.
banc_debout() { alive_x osmo-stp || alive_x osmo-bsc || alive_x osmo-msc; }
# Pattern on the full command line (python scripts, paths).
alive_f() { in_op pgrep -f "$1" >/dev/null 2>&1; }

# ── THE MATRIX: DISPLAY (instant) AND MEASUREMENT (in the background) ───────
MATRIX_FILE="${OSMO_FFT_DIR:-/run/osmo-fft}/ss7-matrix"
MATRIX_LOCK="${OSMO_FFT_DIR:-/run/osmo-fft}/.ss7-matrix.lock"
MATRIX_TTL="${OSMO_MATRIX_TTL:-300}"

matrice() {
    local now age=999999 mtime=0
    now="$(date +%s)"
    [ -f "$MATRIX_FILE" ] && {
        mtime="$(stat -c %Y "$MATRIX_FILE" 2>/dev/null || echo 0)"
        age=$(( now - mtime ))
    }
    # ── A MEASUREMENT TAKEN BEFORE THE BENCH IS NOT A MEASUREMENT ──────────
    # A measurement taken at startup, before osmo-multi has launched
    # osmo-operator-2/3, reports the docker operators as failed because their
    # containers do not exist yet. The cache must not outlive what it
    # describes: any measurement older than the last start of osmo-banc or
    # osmo-multi is stale, whatever its age. `systemctl show` answers without
    # privilege.
    local t u
    for u in osmo-banc.service osmo-multi.service; do
        t="$(systemctl show -p ActiveEnterTimestamp --value "$u" 2>/dev/null)"
        [ -n "$t" ] || continue
```

```
        t="$(date -d "$t" +%s 2>/dev/null)" || continue
        [ -n "$t" ] && [ "$mtime" -lt "$t" ] && age=$(( MATRIX_TTL + 1 ))
    done
    # Stale (or never measured): launch the measurement DETACHED and return at
    # once. The lock is a mkdir: atomic, and it goes away with the directory
    # even if the measurement is killed (trap).
    local encours=0
    if [ "$age" -gt "$MATRIX_TTL" ]; then
        ( setsid "$0" --refresh-matrix >/dev/null 2>&1 & ) 2>/dev/null
        encours=1
    fi
    [ -d "$MATRIX_LOCK" ] && encours=1
    if [ -s "$MATRIX_FILE" ]; then
        cat "$MATRIX_FILE"
        # ss7_check.sh takes a good minute: SAY that what is shown is old and
        # that a measurement is running, rather than letting a stale KO pass
        # for the current state.
        if [ "$encours" = 1 ]; then
            echo "  \${color4}mesure en cours\${color} - affiche : il y a $(( age / 60 ))m$(( age % 60 ))s"
        fi
    else
        echo "  ${C1}matrice SS7${C} : premiere mesure en cours..."
    fi
}

refresh_matrice() {
    mkdir "$MATRIX_LOCK" 2>/dev/null || return 0     # one measurement at a time
    trap 'rmdir "$MATRIX_LOCK" 2>/dev/null' EXIT
    local dir out
    dir="$(cd "$(dirname "$0")/.." && pwd)"
    [ -x "$dir/checks/ss7_check.sh" ] || return 0
    # NO_COLOR + stripping of ANSI sequences: repainted with the Conky colours
    # (${color1..4}), as Conky does not understand terminal codes.
    out="$(cd "$dir" && NO_COLOR=1 timeout 240 ./checks/ss7_check.sh 2>/dev/null \
           | sed -e 's/\x1b\[[0-9;]*m//g')"
    [ -n "$out" ] || return 0
    {
        # The table: the header line (the OpN columns) then one line per
        # operator, up to the blank line that closes the block.
        printf '%s\n' "$out" \
        | awk '/MATRICE DE CONNECTIVITE/{f=1; next}
               f && /^[[:space:]]*$/ && seen {exit}
               f && /Op/ {seen=1; print}' \
        | sed -e 's/[[:space:]]*$//' \
              -e 's/\bself\b/${color1}self${color}/g' \
              -e 's/\bvia\b/${color2}via${color}/g' \
              -e 's/\bFAIL\b/${color3}FAIL${color}/g'
        # The verdict, as ss7_check phrases it.
        printf '%s\n' "$out" | awk '/SS7 (OK|DEGRADE|KO)/{sub(/^[[:space:]]+/,""); print; exit}' \
        | sed -e 's/^SS7 OK/${color2}SS7 OK${color}/' \
              -e 's/^SS7 DEGRADE/${color4}SS7 DEGRADE${color}/' \
              -e 's/^SS7 KO/${color3}SS7 KO${color}/' \
              -e 's/\([0-9]\+\) pass/${color2}\1 pass${color}/' \
              -e 's/\([0-9]\+\) fail/${color3}\1 fail${color}/' \
              -e 's/\([0-9]\+\) warn/${color4}\1 warn${color}/'
    } > "$MATRIX_FILE.tmp" 2>/dev/null && mv -f "$MATRIX_FILE.tmp" "$MATRIX_FILE"
}

case "${1:-}" in
--refresh-matrix) refresh_matrice; exit 0 ;;
role)
    r="$(awk -F= '/^OSMO_ROLE=/{print $2}' /etc/osmo-role 2>/dev/null)"
    n="$(awk -F= '/^OSMO_NODE=|^NODE_ID=/{print $2}' /etc/osmo-role 2>/dev/null | head -1)"
    h="$(awk -F= '/^OSMO_HUB_IP=/{print $2}' /etc/osmo-role 2>/dev/null)"
    live=""; [ -e /run/live/rootfs/filesystem.squashfs ] && live=" \${color4}LIVE${C}"
    opl=""; [ "$OP_MODE" = docker ] && opl=" \${color2}op $OP_ID${C} ($OP_NAME)"
```

```
    echo "${r:-operateur}${n:+ noeud $n}${h:+ · hub $h}$live$opl" ;;
net)
    ip -o -4 addr show up 2>/dev/null | awk '$2!="lo" && $2!~/^(veth|br-|docker|apn)/ {print $2, $4}' | sort -u |
     ↪  head -5 \
    | while read -r ifc addr; do
        printf '${color6}%s${color} ${alignr}${color2}${downspeedf %s}${color} K/s %s  ${color4}${upspeedf
         ↪  %s}${color} K/s %s\n%-8s ${color1}%s${color}\n' \
            "$ifc" "$ifc" "${FS}\${color2}↓\${color}${FU}" "$ifc" "${FS}\${color4}↑\${color}${FU}" "" "$addr"
      done
    # This section NEVER collapses (see banc_debout): when the bench is down,
    # it is the one being read. If the filter above let nothing through - only
    # veth/docker, or no address at all - say so, rather than rendering an
    # empty section that would pass for a broken Conky.
    if [ -z "$(ip -o -4 addr show up 2>/dev/null | awk '$2!="lo" && $2!~/^(veth|br-|docker|apn)/')" ]; then
        if [ -n "$(ip -o -4 addr show up 2>/dev/null | awk '$2!="lo"')" ]; then
            echo "${WARN} aucune carte physique - seulement les interfaces du banc"
        else
            echo "${WARN} aucune interface avec adresse"
        fi
    fi ;;
core)
    banc_debout || exit 0
    # name:process:VTY port
    for e in HLR:osmo-hlr:4258 MSC:osmo-msc:4254 BSC:osmo-bsc:4242 STP:osmo-stp:4239 \
             MGW:osmo-mgw:4243 SGSN:osmo-sgsn:4245 GGSN:osmo-ggsn:4260 PCU:osmo-pcu:4240 \
             BTS:osmo-bts-trx:4241 SIP:osmo-sip-connector:4256 SMSC:proto-smsc-daemon:0 PBX:asterisk:0; do
        IFS=: read -r name proc port <<< "$e"
        if alive_x "$proc"; then
            if [ "$port" != 0 ] && ! port_open "$port"; then s="$WARN"; else s="$OK"; fi
        else s="$KO"; fi
        printf '%s %-5s' "$s" "$name"
        i=$((${i:-0}+1)); [ $((i % 4)) -eq 0 ] && echo
    done
    # No extra blank line when the twelve dots fill the rows exactly.
    [ $((i % 4)) -eq 0 ] || echo ;;
radio)
    banc_debout || exit 0
    phy="-"
    pgrep -f 'qemu-system-arm' >/dev/null 2>&1 && phy="qemu (Calypso emule)"
    pgrep -f 'fake_trx.py' >/dev/null 2>&1 && phy="faketrx"
    pgrep -x virtphy >/dev/null 2>&1 && phy="virtphy"
    pgrep -f 'pont/pont(_dsp)?\.py' >/dev/null 2>&1 && phy="$phy + pont"
    echo "PHY ${C1}${phy}${C}"
    # ── WHAT EACH DOT REALLY LOOKS FOR ──────────────────────────────
    # The ISO bench runs in faketrx + pont: there is NO osmo-trx binary, the
    # BTS transceiver is pont/pont.py and the MS#2 one is fake_trx.py. gapk
    # only exists as osmo-gapk, launched by gapk-start.sh for the duration of
    # a call; outside a call, the watcher (gapk-start.sh auto) IS the nominal
    # state. gr-gsm is replaced by qosmo-grgsm (gsm_sniff.py): grgsm_decode
    # is not installed.
    s=""
    { alive_f 'osmo-trx' || alive_f 'pont/pont(_dsp)?\.py' || alive_f 'fake_trx\.py'; } && s="$s$OK TRX     " ||
     ↪  s="$s$KO TRX     "
    alive_x mobile                  && s="$s$OK MOBILE  " || s="$s$KO MOBILE  "
    alive_x trxcon                  && s="$s$OK TRXCON  " || s="$s$KO TRXCON  "
    alive_f 'qemu-system-arm'       && s="$s$OK QEMU    " || s="$s$KO QEMU    "
    if alive_x osmo-gapk; then s="$s$OK GAPK    "
    elif alive_f 'gapk-start\.sh'; then s="$s$OK GAPK    "
    else s="$s$KO GAPK    "; fi
    { alive_f 'grgsm_(decode|livemon)' || alive_f 'gsm_sniff\.py' || alive_x qosmo-grgsm; } \
        && s="$s$OK GRGSM  " || s="$s$KO GRGSM  "
    echo "$s"
    arfcn="$(in_op cat /etc/osmocom/osmo-bsc.cfg 2>/dev/null | awk '/^ *arfcn /{print $2; exit}')"
    plmn="$(in_op cat /etc/osmocom/osmo-msc.cfg 2>/dev/null | awk '/network country code/{c=$4} /mobile network
     ↪  code/{n=$4} END{if(c) print c"-"n}')"
```

```
    a5="$(in_op cat /etc/osmocom/osmo-msc.cfg 2>/dev/null | awk '/encryption a5/{$1="";$2="";print; exit}' | sed
     ↪  's/^ *//')"
    echo "PLMN ${C1}${plmn:--}${C}  ARFCN ${C1}${arfcn:--}${C}  A5 ${C1}${a5:--}${C}" ;;
subs)
    banc_debout || exit 0
    db=/var/lib/osmocom/hlr.db
    # The container does not necessarily have sqlite3: its database is read
    # through a copy. Together with its -wal: osmo-hlr runs in WAL mode, so the
    # LUs are not yet written back into hlr.db (the copy alone would report
    # « 0 attached »). One copy per user, so a root-owned copy cannot block
    # the session user's conky.
    if [ "$OP_MODE" = docker ]; then
        dbl="/tmp/.conky-hlr-$(id -u)-${OP_NAME}.db"
        rm -f "$dbl-wal" "$dbl-shm"
        if docker cp "$OP_NAME:$db" "$dbl" >/dev/null 2>&1; then
            docker cp "$OP_NAME:$db-wal" "$dbl-wal" >/dev/null 2>&1
            db="$dbl"
        fi
    fi
    # ── THE HLR STATE IS NOT READ FROM ITS DATABASE ──────────────────────
    # The database outlives the daemon: it would still show subscribers
    # « attached » to a VLR that is gone. The daemon is checked first; the
    # database only says what is PROVISIONED.
    if ! alive_x osmo-hlr; then
        _n=""
        have sqlite3 && [ -r "$db" ] && \
            _n="$(sqlite3 "$db" 'select count(*) from subscriber;' 2>/dev/null)"
        echo "HLR ${KO} arrete${_n:+ ${AR}${_n} abonnes en base}"
    elif have sqlite3 && [ -r "$db" ]; then
        tot="$(sqlite3 "$db" 'select count(*) from subscriber;' 2>/dev/null)"
        att="$(sqlite3 "$db" "select count(*) from subscriber where vlr_number is not null and vlr_number != '';"
         ↪  2>/dev/null)"
        last="$(sqlite3 "$db" 'select max(last_lu_seen) from subscriber;' 2>/dev/null)"
        echo "HLR ${C1}${tot:-0}${C} abonnes · ${C2}${att:-0}${C} rattaches ${AR}LU ${last:--}"
        sqlite3 "$db" "select imsi, coalesce(msisdn,'-'), case when vlr_number is not null and vlr_number != ''
         ↪  then 1 else 0 end from subscriber order by imsi limit 6;" 2>/dev/null \
        | awk -F'|' -v ok="$OK" -v ko="$KO" '{printf "%s %s ${alignr}%s\n", ($3==1?ok:ko), $1, $2}'
    else
        echo "HLR ${KO} base introuvable (${db})"
    fi ;;
vue)
    # ── THE SELECTOR: ◄ STOP ► , AND THE VIEW IT DRIVES ──────────────────
    # The Conky carries ONE selector and ONE view: the stops are those of
    # « osmo-op --list-vue » (op1, the docker operators that are RUNNING, the
    # hub if it runs, the 4G if it is installed) and the view changes with the
    # stop - 2G sections for an operator, SS7 matrix for the hub, EPC core and
    # LTE radio for the 4G.
    #
    # Conky does NOT receive clicks (that is the whole point of
    # tools/osmo-panel.py): the ◄ ► arrows are moved with those of the panel,
    # with Ctrl+AltGr+Left/Right (shortcut set by the ISO,
    # iso_modules/80-chroot.sh), or from the command line - « osmo-op --next »
    # and « --prev ».
    _op="${DIR:-/opt/GSM/osmo-operator}/tools/osmo-op.sh"
    [ -x "$_op" ] || _op="/opt/GSM/osmo-operator/tools/osmo-op.sh"
    [ -x "$_op" ] || { echo "${WARN} osmo-op.sh introuvable"; exit 0; }
    mapfile -t VUES < <("$_op" --list-vue 2>/dev/null)
    [ "${#VUES[@]}" -gt 0 ] || { echo "${WARN} aucun arret (osmo-op --list-vue)"; exit 0; }
    # ── THE SELECTED STOP MAY HAVE VANISHED ──────────────────────────────
    # If the container of the selected operator was stopped, the stop no
    # longer exists and the Conky would show the view of an absent operator.
    # Fall back on the first stop - and WRITE it, so the panel and the arrows
    # follow instead of staying on a ghost.
    _cur="$OP_ID"; _mode="$OP_MODE"; _connu=0
    for l in "${VUES[@]}"; do
        set -- $l
```

```
        [ "$1" = "$_cur" ] && { _connu=1; _mode="$2"; }
    done
    if [ "$_connu" = 0 ]; then
        set -- ${VUES[0]}
        _cur="$1"; _mode="$2"
        "$_op" --set "$_cur" >/dev/null 2>&1 || true
    fi
    # The selector - only when there is more than one place to go.
    if [ "${#VUES[@]}" -gt 1 ]; then
        sel=""
        for l in "${VUES[@]}"; do
            set -- $l
            idx="$1"; mode="$2"; etat="${4:-arrete}"   # $3 = the IP, or « - »
            case "$mode" in
                interstp) nom="hub" ;;
                lte)      nom="4G" ;;
                *)        nom="op$idx" ;;
            esac
            # A smiley before each stop: green smile if it runs, grey frown
            # otherwise; the selected stop between ◂ ▸.
            [ "$etat" = actif ] && p="$SMILE" || p="$SAD"
            if [ "$idx" = "$_cur" ]; then
                sel="$sel ${FS}\${color4}◂\${color}${FU}$p${C2}$nom${C}${FS}\${color4}▸\${color}${FU}"
            else
                sel="$sel $p${C1}$nom${C}"
            fi
        done
        echo "${sel# }${AR}\${color5}Ctrl+AltGr+${FS}\${color4}◂▸\${color}${FU}"
    fi
    # The view.
    T='${font Ubuntu:bold:size=10}${color1}'; TF='${font}${color}'
    case "$_mode" in
        interstp)
            echo "${T}Hub inter-STP${TF}"
            "$0" hub ;;
        lte)
            echo "${T}4G / LTE\${color4} Open5GS + srsRAN${TF}"
            "$0" lte ;;
        *)
            # ── NO TITLE WITHOUT SOMETHING BELOW IT ──────────────────────
            # These three sections render NOTHING when the stack is down
            # (banc_debout, above): the titles would then stand alone, three
            # lines that read like a broken Conky. They are only printed when
            # there is something to put below them, and the « nothing at all »
            # case is said in one line - the useful one.
            _vide=1
            for e in "Coeur GSM:core" "Radio / MS:radio" "Abonnes (HLR):subs"; do
                _out="$("$0" "${e##*:}" 2>/dev/null)"
                [ -n "$_out" ] || continue
                _vide=0
                echo "${T}${e%%:*}${TF}"
                echo "$_out"
            done
            [ "$_vide" = 0 ] || echo "${C1}banc a l arret${C} - ${C2}./start-native.sh${C}" ;;
    esac
    ;;
ops|hub)
    # ── THE INTER-STP HUB, AND THE CONNECTIVITY MATRIX ──────────────────
    # The operator selector lives in « vue » above; only what concerns the hub
    # remains here, and it is the « hub » view. (« ops » is still accepted as
    # an alias: older ISOs call it by that name.)
    # The inter-STP hub: present only when it runs (multi bench only).
    if have docker && docker ps --format '{{.Names}}' 2>/dev/null | grep -qx osmo-inter-stp; then
        _pc="$(awk -F= '/^MULTI_HUB_PC=/{print $2}' /etc/osmocom/osmo-multi.conf 2>/dev/null)"
        _hi="$(awk -F= '/^MULTI_HUB_IP=/{print $2}' /etc/osmocom/osmo-multi.conf 2>/dev/null)"
        echo "$OK inter-STP ${C1}${_hi:-172.20.0.10}${C} PC ${C1}${_pc:-0.0.0}${C}"
```

```
        # ── THE CONNECTIVITY MATRIX, CACHED ──────────────────────────────
        # It comes from checks/ss7_check.sh - THE source, the one that really
        # queries the VTYs and counts the routes. It is not recomputed here:
        # two implementations of the same question always end up disagreeing,
        # and the copy is the one people believe.
        #
        # BUT ss7_check.sh takes a good minute (31 tests, several VTY telnets
        # per operator, each with its timeouts). An `execpi` calling it would
        # FREEZE THE WHOLE CONKY at every refresh - clock, CPU, network, all
        # of it would stop during the measurement. Hence the cache: the last
        # result is DISPLAYED, and the next measurement is triggered IN THE
        # BACKGROUND once it has aged. The conky never blocks; at worst the
        # matrix is a minute late, which does not matter for a topology that
        # does not move.
        matrice
    fi ;;
lte)
    # ── THE BENCH 4G: Open5GS (the EPC core) + srsRAN over ZeroMQ ──────────
    # The bench carries the 4G through tools/osmo-lte.sh: the eight Open5GS
    # daemons (+ mongod, without which the HSS and the PCRF die at init), the
    # srsRAN eNB and UE linked by ZeroMQ, S1 on 36412 and ogstun for the user
    # plane.
    #
    # The probes are those of tools/osmo-epc.sh and tools/osmo-lte.sh - THE
    # sources - but WITHOUT their ZeroMQ throughput measurement: that one
    # sleeps one second between two counter reads, and an execpi that sleeps
    # FREEZES the whole Conky (same reason as the SS7 matrix cache above).
    # Everything is native here: Open5GS is not containerised, so no in_op.
    _o5gs=""
    for p in "${OPEN5GS_PREFIX:-}" /opt/LTE/open5gs/install /root/open5gs/install; do
        [ -n "$p" ] && [ -x "$p/bin/open5gs-mmed" ] && { _o5gs="$p"; break; }
    done
    _srs=""
    for b in /usr/local/bin/srsenb "${OSMO_SRSRAN_BUILD:-/opt/LTE/srsRAN_4G/build}/srsenb/src/srsenb"; do
        [ -x "$b" ] && { _srs="$b"; break; }
    done
    have srsenb && _srs="${_srs:-srsenb}"
    if [ -z "$_o5gs" ] && [ -z "$_srs" ]; then
        echo "${C1}4G non installee${C} - ${C2}osmo-lte-install --build${C}"; exit 0
    fi
    # ── A STOPPED 4G DOES NOT DESERVE THIRTEEN RED DOTS ─────────────────────
    # Same rule as banc_debout for the 2G: when nothing runs, one line saying
    # what to type is worth more than a wall of alarms.
    _debout=0
    for d in open5gs-mmed srsenb srsue; do
        pgrep -x "$d" >/dev/null 2>&1 && _debout=1
    done
    if [ "$_debout" = 0 ]; then
        # The subscriber console lives its own life: it only reads MongoDB and
        # is precisely used to provision with the core off. If it listens, say so.
        _w=""
        ss -tanH state listening "( sport = :9999 )" 2>/dev/null | grep -q . \
            && _w="  ${AR}${OK} console ${C1}:9999${C}"
        echo "${C1}4G a l arret${C} - ${C2}osmo-lte start${C}${_w}"; exit 0
    fi
    # The core: the eight daemons in osmo-epc.sh start order, mongod - the HSS
    # and the PCRF read the subscribers from it and DIE without it (the MME
    # then answers 3002 on S6a and rejects every attach) - and the subscriber
    # console (open5gs-webui.service, :9999), which reads the same database.
    i=0
    for e in MME:open5gs-mmed HSS:open5gs-hssd SGWC:open5gs-sgwcd SGWU:open5gs-sgwud \
             SMF:open5gs-smfd UPF:open5gs-upfd PCRF:open5gs-pcrfd NRF:open5gs-nrfd \
             MONGO:mongod WEBUI:@9999; do
        IFS=: read -r name proc <<< "$e"
        # « @port »: a console, not a daemon - the Open5GS WebUI is a node
        # process, its name says nothing. Check what it LISTENS on instead.
```

```
        case "$proc" in
            @*) ss -tanH state listening "( sport = :${proc#@} )" 2>/dev/null | grep -q . \
                    && s="$OK" || s="$KO" ;;
            *)  pgrep -x "$proc" >/dev/null 2>&1 && s="$OK" || s="$KO" ;;
        esac
        # %-6s and not %-5s: « MONGO » is five characters and would touch the
        # next dot.
        printf '%s %-6s' "$s" "$name"
        i=$((i+1)); [ $((i % 5)) -eq 0 ] && echo
    done
    [ $((i % 5)) -eq 0 ] || echo
    # The radio and the links: eNB, UE, S1 (SCTP 36412), ZeroMQ, ogstun.
    s=""
    pgrep -x srsenb >/dev/null 2>&1 && s="$s$OK eNB   " || s="$s$KO eNB   "
    pgrep -x srsue  >/dev/null 2>&1 && s="$s$OK UE    " || s="$s$KO UE    "
    ss -San 2>/dev/null | grep -q "ESTAB.*:36412" && s="$s$OK S1    " || s="$s$KO S1    "
    _zp="${OSMO_ZMQ_PORT:-2000}"
    ss -tanH state established "( sport = :$_zp or dport = :$_zp )" 2>/dev/null | grep -q . \
        && s="$s$OK ZMQ   " || s="$s$KO ZMQ   "
    ip -br link show ogstun >/dev/null 2>&1 && s="$s$OK TUN" || s="$s$KO TUN"
    echo "$s"
    # The cell as declared by enb.conf - the one that was used if readable
    # (root), the repo one otherwise: it is the source of the copy.
    _cfg=""
    for f in "${OSMO_SRSRAN_DIR:-/root/.config/srsran}/enb.conf" \
             "${DIR:-/opt/GSM/osmo-operator}/configs/srsran/enb.conf" \
             /opt/GSM/osmo-operator/configs/srsran/enb.conf; do
        [ -r "$f" ] && { _cfg="$f"; break; }
    done
    mcc=""; mnc=""; earfcn=""; prb=""
    if [ -n "$_cfg" ]; then
        eval "$(awk -F= '
            /^[[:space:]]*mcc[[:space:]]*=/       {gsub(/[[:space:]]/,"",$2); print "mcc="$2}
            /^[[:space:]]*mnc[[:space:]]*=/       {gsub(/[[:space:]]/,"",$2); print "mnc="$2}
            /^[[:space:]]*dl_earfcn[[:space:]]*=/ {gsub(/[[:space:]]/,"",$2); print "earfcn="$2}
            /^[[:space:]]*n_prb[[:space:]]*=/     {gsub(/[[:space:]]/,"",$2); print "prb="$2}' "$_cfg")"
    fi
    # The PRB count IS the channel width: display the one that is read.
    case "$prb" in 6) bw=1.4;; 15) bw=3;; 25) bw=5;; 50) bw=10;; 75) bw=15;; 100) bw=20;; *) bw="";; esac
    # The UE address lives in a netns: « ip -n » requires privilege. Under
    # the session (non-root uid) there is none - say nothing rather than
    # suggesting the UE is not attached.
    ue="$(ip -n "${OSMO_LTE_NETNS:-ue1}" -4 -o addr show 2>/dev/null | awk '/tun_srsue/{print $4; exit}')"
    echo "PLMN ${C1}${mcc:--}-${mnc:--}${C}  EARFCN ${C1}${earfcn:--}${C}${bw:+  ${C1}${bw}${C} MHz}${ue:+ ${AR}UE
     ↪  ${C1}${ue}${C}}"
    ;;
services)
    # Bench down: a single line, the useful one - the failed units. The rest
    # (web, pulse, tmux, docker) says nothing beyond « nothing runs », which
    # the absence of the sections above has already said.
    if ! banc_debout; then
        f="$(systemctl --failed --no-legend 2>/dev/null | wc -l)"
        [ "$f" -gt 0 ] && echo "${WARN} ${f} service(s) systemd en echec" \
                       || echo "${C1}banc a l arret${C} - ${C2}./start-native.sh${C}"
        exit 0
    fi
    s=""
    port_open 8080 || port_open 8443 && s="$s$OK web " || s="$s$KO web "
    pgrep -x pulseaudio >/dev/null 2>&1 && s="$s$OK pulse " || s="$s$KO pulse "
    # ── NO tmux OR docker DOTS ──────────────────────────────────────
    # The Conky runs under the SESSION account, so these two probes would
    # answer « what do I see » rather than « what runs on this machine »:
    #   tmux   : the bench opens its sessions as ROOT, on /tmp/tmux-0/default;
    #            `tmux list-sessions` as the user looks at
    #            /tmp/tmux-1001/default, which does not exist.
    #   docker : « permission denied ... /var/run/docker.sock » as long as the
```

```
    #            account is not in the docker group, hence zero containers.
    # Neither says anything useful about the bench: the Coeur GSM and Radio
    # sections already say whether the stack is up, and the operator line
    # (`ops` sub-command) says which containers run - and that one can tell
    # « stopped » from « invisible ».
    have nvidia-smi && s="$s$OK nvidia "
    echo "$s"
    # ── AND NAME WHAT FAILS ──────────────────────────────────────────
    # A bare « N systemd service(s) failed » sends the reader looking for which
    # one by hand; the name makes the diagnosis immediate (e.g. a unit masked
    # by the image that stays `failed` until `systemctl reset-failed`).
    # `systemctl --failed --no-legend` starts with a BULLET (●), not the name:
    # take the field that looks like a unit, whatever its position.
    f="$(systemctl --failed --no-legend 2>/dev/null \
         | awk '{for(i=1;i<=NF;i++) if($i ~ /\.(service|socket|target|timer|mount|path)$/) {print $i; break}}' \
         | sed 's/\.service$//' | paste -sd' ' -)"
    [ -n "$f" ] && echo "${WARN} en echec : ${C1}${f}${C}" || echo "${OK} systemd sans echec"
    ;;
*) echo "usage: $0 role|vue|net|hub|core|radio|lte|subs|services" >&2; exit 2 ;;
esac
```

2.229 /opt/GSM/osmo-operator/tools/daram_sweep.sh

5221 octets, 101 lignes → 101 lignes

```
#!/bin/bash
# tools/daram_sweep.sh - sweep CALYPSO_BSP_DARAM_ADDR in full mode and compare
# the activity of the FB-det correlator (fb0_att / snr / toa / pm).
# Goal: decide empirically which DARAM area unblocks the FB-det.
#
# Robust: run.sh is started NO_ATTACH IN THE FOREGROUND (it returns on its own
# after starting qemu, which survives orphaned). run.sh is NOT backgrounded
# with & (that breaks the log pipe and qemu dies). qemu is checked to be alive
# before measuring; otherwise it is reported.
#
# Usage (container): /opt/GSM/daram_sweep.sh
# Env: ADDRS (default "0x0080 0x2a00 0x0060"), SETTLE (post-boot wait, 50s)
set -u
RUN=/opt/GSM/qosmo-grgsm/run.sh
QLOG=/root/qemu.log
ADDRS="${ADDRS:-0x0080 0x2a00 0x0060}"
SETTLE="${SETTLE:-50}"
# Emulation profile of the sweep. The profiles in environment/modes.env are
# empty | shunt_legit | shunt_legit_no_inject | native | native_twl |
# native_twl_host_demod | native_helped.
MODE="${MODE:-${CALYPSO_MODE:-native}}"
declare -A R_FB R_SB R_NZ R_BEST R_ALIVE R_STATES R_TOT

kill_all() {
    tmux kill-session -t calypso 2>/dev/null
    for p in $(pgrep -f qemu-system-arm); do kill -9 "$p" 2>/dev/null; done
    pkill -9 -f osmo-bts-trx 2>/dev/null; pkill -9 -f osmo-trx-ipc 2>/dev/null
    pkill -9 -f "mobile" 2>/dev/null; pkill -9 -f osmocon 2>/dev/null
    sleep 4
}

for a in $ADDRS; do
    echo "############ DARAM_ADDR=$a (profil $MODE) ############ $(date +%H:%M:%S)"
    kill_all
    if pgrep -f qemu-system-arm >/dev/null; then
        echo "  !! un qemu survit a kill_all : la supervision le relance." >&2
        echo "     Arretez la pile (CALYPSO_NO_RESTART=1 ou ./run.sh --stop) avant le balayage." >&2
        exit 3
    fi
```

```
    : > "$QLOG"
    # run.sh NO_ATTACH returns on its own after the launch (~30-60s). timeout = safety net.
    echo "  launch run.sh (NO_ATTACH, avant-plan, le boot prend ~30-60s)..."
    timeout 150 env CALYPSO_NO_ATTACH=1 CALYPSO_MODE="$MODE" CALYPSO_ICOUNT=off \
      CALYPSO_NO_RESTART=1 CALYPSO_BSP_DARAM_ADDR="$a" "$RUN" \
      </dev/null >"/tmp/sweep_$a.out" 2>&1
    rc_run=$?
    # Check that the profile was actually applied: with an unknown mode run.sh
    # aborts at its second module, the previous qemu (restarted by the
    # supervision) keeps writing to the SAME log, and the sweep would measure
    # that instance instead.
    if grep -qa "CALYPSO_MODE inconnu\|sequence aborted" "/tmp/sweep_$a.out"; then
        echo "  !! run.sh a avorte : profil '''$MODE''' refuse par environment/modes.env" >&2
        sed -n '''/valeurs :/,+2p''' "/tmp/sweep_$a.out" | sed "s/^/     /" >&2
        echo "     Relancez avec MODE=<profil valide> $0" >&2
        exit 2
    fi
    echo "  run.sh rendu (code=$?). qemu doit tourner orphelin. settle ${SETTLE}s..."
    alive=no
    for i in $(seq 1 $((SETTLE/5))); do
        sleep 5
        if pgrep -f qemu-system-arm >/dev/null; then alive=yes; else alive=no; fi
        # stop early once non-trivial FB data has been seen
        grep -qa "real DSP path" "$QLOG" 2>/dev/null && [ "$i" -ge 4 ] && break
    done

    fb=$(grep -a "real DSP path" "$QLOG" 2>/dev/null | grep -oE "fb0_att=[0-9]+" | grep -oE "[0-9]+" | sort -n |
     ↪  tail -1)
    sb=$(grep -a "real DSP path" "$QLOG" 2>/dev/null | grep -oE "sb_att=[0-9]+"  | grep -oE "[0-9]+" | sort -n |
     ↪  tail -1)
    # Two FBSB line formats are accepted: the correlator writes
    #   data[](det=0 toa=0 pm=0 ang=0 snr=0x0000) api[](...)
    # or "last(snr=... toa=... ang=... pm=...)". Matching only one of them
    # would report "non0=0 / 1er_corr=none" for every address, including those
    # where the correlator sees the I/Q well.
    NUL='(det=0 toa=0 pm=0 ang=0 snr=0x0000)|snr=0 toa=0 ang=0 pm=0'
    nz=$(grep -a "real DSP path" "$QLOG" 2>/dev/null | grep -avcE "$NUL")
    best=$(grep -a "real DSP path" "$QLOG" 2>/dev/null | grep -avE "$NUL" \
             | grep -oE "(data\[\]\(det=[0-9-]+ toa=[0-9-]+ pm=[0-9-]+ ang=[0-9-]+
              ↪  snr=0x[0-9a-fA-F]+\))|(last\(snr=[0-9-]+ toa=[0-9-]+ ang=[0-9-]+ pm=[0-9a-fx]+\))" \
             | head -1)
    # Total number of hits: the detection rate matters as much as the first
    # success (19 % is not enough to lock the synchronisation).
    tot=$(grep -ac "real DSP path" "$QLOG" 2>/dev/null)
    states=$(grep -aoE 'state=[A-Z0-9_]+' "$QLOG" 2>/dev/null | sort | uniq -c | tr '\n' ' ')
    qlines=$(wc -l < "$QLOG" 2>/dev/null)
    R_FB[$a]="${fb:-0}"; R_SB[$a]="${sb:-0}"; R_NZ[$a]="${nz:-0}"
    R_BEST[$a]="${best:-none}"; R_ALIVE[$a]="$alive"; R_STATES[$a]="${states:-<aucun>}"
    R_TOT[$a]="${tot:-0}"
    echo "  -> qemu_vivant=$alive  qemu.log=${qlines}l  fb0_att=${fb:-0}  sb_att=${sb:-0}  non0=${nz:-0}/${tot:-0}"
    echo "  -> etats: ${states:-<aucun>}"
    echo "  -> 1er correlateur non-nul: ${best:-none}"
done

kill_all
echo
echo "================= COMPARAISON ================="
printf "%-9s %-7s %-9s %-8s %-12s %s\n" "ADDR" "alive" "fb0_att" "sb_att" "non0/total" "1er_corr"
for a in $ADDRS; do
    printf "%-9s %-7s %-9s %-8s %-12s %s\n" \
      "$a" "${R_ALIVE[$a]}" "${R_FB[$a]}" "${R_SB[$a]}" \
      "${R_NZ[$a]}/${R_TOT[$a]}" "${R_BEST[$a]}"
done
echo
echo "Gagnant = fb0_att>0 et/ou non0>0 (correlateur voit l'I/Q). alive=no => boot KO."
```

2.230 /opt/GSM/osmo-operator/tools/dockerfile-to-install-md.sh

13170 octets, 283 lignes → 283 lignes

```
#!/usr/bin/env bash
# dockerfile-to-install-md.sh - the Dockerfile, step by step, as an "install from scratch" Markdown.
#
# The Dockerfile is the only complete and tested description of the installation
# (apt packages, compiled Osmocom stack, osmocom-bb + firmware, qosmo, gr-gsm, node, LTE...).
# Rather than maintaining by hand a second document that drifts, it is generated:
#   each FROM becomes a chapter, each RUN a bash block to paste as is in a root shell,
#   each COPY a cp from the repository, the Dockerfile comments become the prose.
#
#   tools/dockerfile-to-install-md.sh                     # Dockerfile -> doc/install-from-scratch.md
#   tools/dockerfile-to-install-md.sh Dockerfile.run -o doc/x.md
#   tools/dockerfile-to-install-md.sh --annexe doc/c54x_exe-depuis-ubuntu-vierge.md
#
# Deliberately simple (embedded python3, no dependency).
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
DOCKERFILE="$HERE/Dockerfile"
OUT="$HERE/doc/install-from-scratch.md"
ANNEXES=()
LANG_DOC=fr
NOTES=""
while [ $# -gt 0 ]; do
    case "$1" in
        -o|--out)    OUT="$2"; shift ;;
        --annexe)    ANNEXES+=("$2"); shift ;;
        # --lang en: English prose; the (French) Dockerfile comments are replaced by the
        # per-step notes of --notes <file> (sections "## <step name>"), the command blocks stay.
        --lang)      LANG_DOC="$2"; shift ;;
        --notes)     NOTES="$2"; shift ;;
        -h|--help)   sed -n '2,15p' "$0"; exit 0 ;;
        *)           DOCKERFILE="$1" ;;
    esac
    shift
done
[ -r "$DOCKERFILE" ] || { echo "Dockerfile illisible : $DOCKERFILE" >&2; exit 1; }
mkdir -p "$(dirname "$OUT")"
python3 - "$DOCKERFILE" "$OUT" "$HERE" "$LANG_DOC" "$NOTES" "${ANNEXES[@]}" <<'EOF'
import re, sys, datetime, os
src, out, root, lang, notes_file, *annexes = sys.argv[1:]
EN = (lang == "en")
notes = {}
if notes_file and os.path.isfile(notes_file):
    cur = None
    for l in open(notes_file, encoding="utf-8").read().split("\n"):
        m = re.match(r"^## (\S+)", l)
        if m:
            cur = m.group(1); notes[cur] = []
        elif cur is not None:
            notes[cur].append(l)
    notes = {k: "\n".join(v).strip() for k, v in notes.items()}
lines = open(src, encoding="utf-8", errors="replace").read().split("\n")

# 1. re-join the continuations (\ at end of line), keep the comments that precede
items = []          # (comments:list[str], instr:str, args:str)
pending = []
i = 0
while i < len(lines):
    l = lines[i]
    if not l.strip():
        if pending and items and not items[-1][1]:
            pass
        i += 1
```

```
        continue
    if l.lstrip().startswith("#"):
        # keep the indentation after the "#": indented lines (diagrams, lists
        # "RAND=0 -> ...") are rendered verbatim, in a block
        pending.append(l.lstrip()[1:].rstrip())
        i += 1
        continue
    buf = [l]
    # continuation: as long as the last NON-comment line ends with "\"; Docker accepts
    # "# ..." lines in the middle of a continued RUN, they do not carry a "\"
    def derniere_utile():
        for b in reversed(buf):
            if not b.lstrip().startswith("#"):
                return b
        return ""
    while derniere_utile().rstrip().endswith("\\") and i + 1 < len(lines):
        i += 1
        buf.append(lines[i])
    i += 1
    text = "\n".join(buf)
    m = re.match(r"\s*([A-Z]+)\s*(.*)", text, re.S)
    if not m:
        continue
    items.append((pending, m.group(1), m.group(2)))
    pending = []

def prose(comments):
    """Les commentaires du Dockerfile, en paragraphes. Les titres « N. ... » et « -- ... -- » deviennent des
↪ sous-titres."""
    outl = []
    if EN:
        return outl
    para = []
    def flush():
        if para:
            outl.append(" ".join(para)); outl.append("")
            para.clear()
    verb = []
    def flush_verb():
        if verb:
            flush(); outl.append("```text"); outl.extend(verb); outl.append("```"); outl.append("")
            verb.clear()
    for c in comments:
        if c.startswith("  ") and c.strip():        # ligne indentee : verbatim
            verb.append(c[1:] if c.startswith(" ") else c)
            continue
        flush_verb()
        c = c.strip()
        if c.startswith("syntax="):
            continue
        s = re.sub(r"^[—═=\-]{3,}\s*", "", c); s = re.sub(r"\s*[—═=\-]{3,}$", "", s)
        if not s:
            flush(); continue
        if re.match(r"^(\d+\.\s+|[A-ZÉ][^a-z]{6,}$)", s) or (c.startswith("——") or c.startswith("---") or
↪ c.startswith("══")):
            flush(); outl.append("#### " + s.strip(" :")); outl.append("")
            continue
        para.append(s)
    flush_verb(); flush()
    return outl

def strip_mounts(cmd):
    cmd = re.sub(r"--mount=\S+(\s*\\\n\s*)?", "", cmd)
    return cmd

def run_block(args):
```

```
    cmd = strip_mounts(args).strip()
    # comments inside RUNs: they are already bash (#), keep them
    return ["```bash", cmd, "```", ""]

doc = []
if EN:
    doc.append("# Installing from scratch (generated from the Dockerfile)")
    doc.append("")
    doc.append("> Generated by `tools/dockerfile-to-install-md.sh --lang en` on %s from `%s`. Do not edit by hand:
 ↪  fix the Dockerfile, the script or `%s`, then regenerate. The French version (`doc/install-from-scratch.md`)
 ↪  carries the full rationale written in the Dockerfile comments; this one keeps the commands and a short English
 ↪  note per stage."
              % (datetime.date.today().isoformat(), os.path.relpath(src, root), os.path.relpath(notes_file, root)
 ↪  if notes_file else "the notes file"))
    doc.append("")
    doc.append("""## How to read this page

The osmo-operator Dockerfile is the complete, tested description of the installation. This page replays it
step by step on a bare machine (Ubuntu 24.04, as root), without Docker:

- every `FROM ... AS <name>` of the Dockerfile is a chapter; on a single machine the stages simply follow
  each other and the `COPY --from=<stage>` instructions are not needed (everything is already in place);
- every `RUN` is a bash block to paste as is into a root shell, in order;
- every `ARG`/`ENV` becomes an `export` to set before the blocks that follow (in Docker an `ARG` does not
  survive a `FROM`; here it just stays in the shell);
- every `COPY` becomes a `cp` from a checkout of the osmo-operator repository (`$REPO`, `/opt/GSM/osmo-operator`
 ↪  by default).

The `.deb` package cache (`osmo-deb install <name> <version>`) is an optimisation of the Docker build:
without the cache the command fails and the `if ! osmo-deb install ...; then ... fi` branch builds from
source, which is exactly what a bare machine needs. `osmo-deb pack` then wraps the result (`make install`);
without `/var/cache/osmo-debs` it just installs.

```bash
export REPO=/opt/GSM/osmo-operator      # checkout of the repository (git clone
 ↪  https://github.com/bbaranoff/osmo-operator)
export ROOT=/opt/GSM
mkdir -p "$ROOT"
```
""")
fr = []
fr.append("# Installation depuis zero (generee depuis le Dockerfile)")
fr.append("")
fr.append("> Fichier genere par `tools/dockerfile-to-install-md.sh` le %s depuis `%s`. Ne pas editer a la main :
 ↪ corriger le Dockerfile ou le script, puis regenerer."
          % (datetime.date.today().isoformat(), os.path.relpath(src, root)))
fr.append("")
fr.append("""## Comment lire cette page

Le Dockerfile d'osmo-operator est la description complete et testee de l'installation. Cette page le rejoue
etape par etape sur une machine nue (Ubuntu 24.04, en root), sans Docker :

- chaque `FROM ... AS <nom>` du Dockerfile est un chapitre ; sur une seule machine les etapes se suivent
  simplement, les `COPY --from=<etape>` ne sont plus necessaires (tout est deja en place) ;
- chaque `RUN` est un bloc bash a coller tel quel dans un shell root, dans l'ordre ;
- chaque `ARG`/`ENV` devient un `export` a poser avant les blocs qui suivent (les `ARG` ne traversent pas
  un `FROM` dans Docker, ici on les garde dans le shell) ;
- chaque `COPY` devient un `cp` depuis une copie du depot osmo-operator (`$REPO`, par defaut
 ↪ `/opt/GSM/osmo-operator`).

Le cache de paquets `.deb` (`osmo-deb install <nom> <version>`) est une optimisation du build Docker :
sans cache, la commande echoue et la branche `if ! osmo-deb install ...; then ... fi` compile depuis les
sources, ce qui est exactement ce qu'on veut sur une machine nue. `osmo-deb pack` emballe ensuite le
resultat (`make install`) ; sans `/var/cache/osmo-debs` il se contente d'installer.

```bash
```
```

```
export REPO=/opt/GSM/osmo-operator      # copie du depot (git clone https://github.com/bbaranoff/osmo-operator)
export ROOT=/opt/GSM
mkdir -p "$ROOT"
```
""")

if not EN:
    doc.extend(fr)

table of contents
stages = [(k, a) for (c, k, a) in items if k == "FROM"]
doc.append("## Table of contents" if EN else "## Sommaire")
doc.append("")
n = 0
for c, k, a in items:
    if k == "FROM":
        n += 1
        m = re.match(r"(\S+)(?:\s+AS\s+(\S+))?", a, re.I)
        base, name = m.group(1), (m.group(2) or m.group(1))
        if EN:
            doc.append("%d. [%s](#stage-%d--%s) (from `%s`)" % (n, name, n, name.lower().replace("_", "-"), base))
        else:
            doc.append("%d. [%s](#etape-%d--%s) (a partir de `%s`)" % (n, name, n, name.lower().replace("_", "-"),
↪ base))
doc.append("")

n = 0
for comments, k, a in items:
    if k == "FROM":
        n += 1
        m = re.match(r"(\S+)(?:\s+AS\s+(\S+))?", a, re.I)
        base, name = m.group(1), (m.group(2) or m.group(1))
        doc.append("")
        doc.append(("## Stage %d : %s" if EN else "## Etape %d : %s") % (n, name))
        doc.append("")
        doc.extend(prose(comments))
        if EN and notes.get(name):
            doc.append(notes[name]); doc.append("")
        if base == "scratch":
            doc.append("_Docker stage without content of its own (`scratch` image): nothing to do on a machine._"
↪ if EN else "_Etape Docker sans contenu propre (image `scratch`) : rien a faire sur une machine._")
            doc.append("")
        else:
            doc.append(("_In Docker: `FROM %s`. On a machine, carry on in the same system._" if EN else "_Dans
↪ Docker : `FROM %s`. Sur une machine, on continue dans le meme systeme._") % base)
            doc.append("")
        continue
    doc.extend(prose(comments))
    if k in ("ARG", "ENV"):
        body = a.replace("\\\n", "\n").strip()
        exports = []
        for kv in re.findall(r"([A-Za-z_][A-Za-z0-9_]*)=((?:\"[^\"]*\")|(?:'[^']*')|\S*)", body):
            exports.append("export %s=%s" % (kv[0], kv[1] if kv[1] != "" else '""'))
        if not exports and body:
            exports.append("export " + body)
        doc.append("```bash"); doc.extend(exports); doc.append("```"); doc.append("")
    elif k == "RUN":
        doc.extend(run_block(a))
    elif k == "COPY" or k == "ADD":
        parts = a.split()
        opts = [p for p in parts if p.startswith("--")]
        files = [p for p in parts if not p.startswith("--")]
        frm = [o for o in opts if o.startswith("--from=")]
        if frm:
```
```

```
            doc.append(("_Copy between Docker stages (`%s`): on a single machine %s is already in place._" if EN
↪  else "_Copie entre etapes Docker (`%s`) : sur une seule machine, %s est deja en place._") % (a.replace("\n", "
↪  "), ", ".join("`%s`" % f for f in files[:-1])))
            doc.append("")
        elif len(files) >= 2:
            dst = files[-1]
            doc.append("```bash")
            for s in files[:-1]:
                if dst.endswith("/"):
                    doc.append('mkdir -p %s && cp -r "$REPO"/%s %s' % (dst, s, dst))
                else:
                    doc.append('mkdir -p "$(dirname %s)" && cp -r "$REPO"/%s %s' % (dst, s, dst))
            doc.append("```"); doc.append("")
    elif k == "WORKDIR":
        doc.append("```bash"); doc.append("cd %s" % a.strip()); doc.append("```"); doc.append("")
    elif k in ("STOPSIGNAL", "ENTRYPOINT", "CMD", "EXPOSE", "LABEL", "HEALTHCHECK", "USER", "VOLUME", "SHELL"):
        doc.append(("_Docker metadata, no equivalent on a machine: `%s %s`._" if EN else "_Metadonnee Docker, sans
↪  equivalent sur une machine : `%s %s`._") % (k, a.replace("\n", " ").strip()))
        doc.append("")
    else:
        doc.append(("_Unhandled instruction: `%s %s`._" if EN else "_Instruction non traitee : `%s %s`._") % (k,
↪  a[:80]))
        doc.append("")

for an in annexes:
    if not os.path.isfile(an):
        continue
    doc.append("")
    doc.append("---")
    doc.append("")
    doc.append(("# Appendix: %s" if EN else "# Annexe : %s") % os.path.basename(an))
    doc.append("")
    doc.append(("_Contents of `%s`, included as is._" if EN else "_Contenu de `%s`, repris tel quel._") %
↪  os.path.relpath(an, root))
    doc.append("")
    txt = open(an, encoding="utf-8", errors="replace").read()
    # shift the headings down one level
    txt = re.sub(r"^(#{1,5}) ", lambda m: "#" + m.group(1) + " ", txt, flags=re.M)
    doc.append(txt)

open(out, "w", encoding="utf-8").write("\n".join(doc).rstrip() + "\n")
print("%s : %d lignes, %d etapes FROM, %d RUN" % (out, len(doc), len(stages), sum(1 for c,k,a in items if
↪  k=="RUN")))
EOF
```

2.231 /opt/GSM/osmo-operator/tools/fft.sh

9365 octets, 199 lignes → 199 lignes

```
#!/usr/bin/env bash
# tools/fft.sh - "pretty" FFT of the Calypso I/Q stream, read DIRECTLY in the docker.
#
# Reads /tmp/record.cfile (the CONTINUOUS record ring, written by record_drain.py
# which drains iq_record.fifo in O_RDWR - DECOUPLED from the real-time relay
# loop) IN the container, fetches it as fc32 (complex64), and displays on the
# HOST's X: averaged PSD (home-made Welch) + scrolling waterfall. Loops (FuncAnimation).
#
# /!\ There is no 'fifo' mode (LIVE tap of /tmp/iq_fft.fifo): opening that fifo
#     for reading forces the IPC device to write it and disturbs the real-time
#     relay -> the grgsm decode stops (si-bridge 'fini') and camping freezes.
#     record.cfile is near-live and has NO effect on the pipeline.
#
# Usage:
#   ./tools/fft.sh                      # tail of record.cfile (default, non-intrusive)
```

```
#   FFT_SRC=sweep ./tools/fft.sh        # sweep the whole record ring
#   ./tools/fft.sh --save spectre.png   # one PNG capture (if no X)
#   CONTAINER=osmo-operator-1 CFILE=/tmp/record.cfile RATE=1083333 \
#   NSAMP=262144 REFRESH=1.0 ARFCN=514 ./tools/fft.sh
#
set -euo pipefail
export CONTAINER="${CONTAINER:-osmo-operator-1}"
export FFT_SRC="${FFT_SRC:-tail}"   # tail (defaut, NON-perturbant) | sweep - lisent record.cfile
# NB: no 'fifo' mode (LIVE tap of /tmp/iq_fft.fifo): opening that fifo for
# reading forces the IPC device to write it, which disturbs the real-time relay
# loop and CUTS the grgsm decode (si-bridge 'fini') -> camping freezes.
# record.cfile is read instead, fed by record_drain.py (decoupled, always-on).
export CFILE="${CFILE:-/dev/shm/dsp_iq.cfile}"    # entree DSP shunt = MS (le mobile qui repond a la BTS ; cfile
 ↪  fc32 en /dev/shm RAM). Fichier qui GRANDIT -> tail-c lit le frais. (override CFILE=/dev/shm/record.cfile = BTS)
export RATE="${RATE:-1083333}"      # 4 SPS natif = 26e6/24
export NSAMP="${NSAMP:-32768}"      # complex samples par fenetre (256 KB) - court = ca gigote
export REFRESH="${REFRESH:-0.25}"   # periode de rafraichissement (s) - court = fluide
export ARFCN="${ARFCN:-514}"        # juste pour le titre (DCS1800)
export DISPLAY="${DISPLAY:-:0}"
export FFT_SAVE=""
[ "${1:-}" = "--save" ] && export FFT_SAVE="${2:?--save needs a path}"

# guards
command -v docker >/dev/null || { echo "docker introuvable"; exit 1; }
case "$FFT_SRC" in
  sweep|tail) ;;
  *) echo "[fft] FFT_SRC='$FFT_SRC' invalide. Le mode 'fifo' (tap live) a ete retire"
     echo "[fft] (il perturbe le relai temps-reel et coupe le decode grgsm). -> tail."
     export FFT_SRC=tail ;;
esac
docker exec "$CONTAINER" test -f "$CFILE" \
  || { echo "record absent $CONTAINER:$CFILE (lance ./run.sh d'abord ; record_drain.py alimente record.cfile)";
   ↪  exit 1; }

python3 - <<'PYEOF'
import os, sys, subprocess, numpy as np
import matplotlib
SAVE = os.environ.get("FFT_SAVE","")
if SAVE:
    matplotlib.use("Agg")
else:
    for bk in ("TkAgg","Qt5Agg","GTK3Agg"):
        try: matplotlib.use(bk); break
        except Exception: continue
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

CONT  = os.environ["CONTAINER"]; CFILE = os.environ["CFILE"]
RATE  = float(os.environ["RATE"]); NS = int(os.environ["NSAMP"])
REFRESH = float(os.environ["REFRESH"]); ARFCN = os.environ["ARFCN"]
NBYTES = NS*8                       # complex64 = 8 o/sample
NFFT  = 4096                        # resolution FFT (segment Welch)
WROWS = 120                         # lignes du waterfall

win = np.hanning(NFFT).astype(np.float32)
freqs = np.fft.fftshift(np.fft.fftfreq(NFFT, 1.0/RATE)) / 1e3   # kHz

SRC  = os.environ.get("FFT_SRC","tail")    # tail (defaut) | sweep - lisent record.cfile
if SRC not in ("sweep","tail"): SRC = "tail"   # 'fifo' retire (tap live nocif)
BS   = 4096; NBLK = max(1, NBYTES//BS)

# ---- file modes: sweep the ring (sweep) or read the tail of record.cfile
#      (fed by record_drain.py, decoupled from the real-time relay loop).
def _filesize():
    try:
        return int(subprocess.run(["docker","exec",CONT,"stat","-c","%s",CFILE],
```

```
                                capture_output=True,timeout=5).stdout or 0)
    except Exception: return NBYTES
def _ringsize():
    # ring size published by record_drain (modulo for the LIVE wrap)
    try:
        v = int(subprocess.run(["docker","exec",CONT,"cat",CFILE+".ring"],
                               capture_output=True,timeout=5).stdout or 0)
        if v > 0: return v
    except Exception: pass
    return _filesize()
RING = _ringsize()
_off = [0]

def _dd(skip_blk, count_blk):
    return subprocess.run(["docker","exec",CONT,"dd","if="+CFILE,"bs="+str(BS),
        "skip="+str(skip_blk),"count="+str(count_blk),"status=none"],
        capture_output=True, timeout=8).stdout

def grab():
    """Rend NBYTES d'I/Q complex64 (le plus frais), depuis record.cfile.
    SRC=tail  : derniers octets (defaut, near-live, non-perturbant).
    SRC=sweep : balaye le ring record (offset qui avance)."""
    if SRC == "sweep":
        skip = _off[0] // BS
        try:
            raw = subprocess.run(["docker","exec",CONT,"dd","if="+CFILE,
                "bs="+str(BS),"skip="+str(skip),"count="+str(NBLK),"status=none"],
                capture_output=True, timeout=8).stdout
        except Exception:
            return None
        _off[0] += NBYTES
        if _off[0] + NBYTES > RING: _off[0] = 0
    else:
        # LIVE (tail): read the fresh window ENDING at the write offset
        # published by record_drain (<cfile>.off). With the ring, seek(0) puts
        # the fresh data at `w` (middle), NOT at the tail. Block-aligned + wrap.
        try:
            w = int(subprocess.run(["docker","exec",CONT,"cat",CFILE+".off"],
                                   capture_output=True, timeout=5).stdout or -1)
        except Exception:
            w = -1
        try:
            if w >= 0:
                wblk = w // BS; start_blk = wblk - NBLK
                if start_blk >= 0:
                    raw = _dd(start_blk, NBLK)
                else:                                  # fenetre a cheval sur le wrap
                    ring_blk = RING // BS; n_end = -start_blk
                    raw = _dd(ring_blk - n_end, n_end) + _dd(0, wblk)
            else:                                       # repli : pas d'offset publie
                raw = subprocess.run(["docker","exec",CONT,"tail","-c",str(NBYTES),CFILE],
                                     capture_output=True, timeout=8).stdout
        except Exception:
            return None
    n = (len(raw)//8)*8
    if n < NFFT*8: return None
    return np.frombuffer(raw[:n], dtype=np.complex64)

def welch_psd(iq):
    """PSD moyennee (segments NFFT, overlap 50%, fenetre Hann) en dB"""
    step = NFFT//2; nseg = max(1,(len(iq)-NFFT)//step + 1)
    acc = np.zeros(NFFT, dtype=np.float64)
    for i in range(nseg):
        seg = iq[i*step:i*step+NFFT]
        if len(seg) < NFFT: break
        sp = np.fft.fftshift(np.fft.fft(seg*win))
```

```
        acc += (sp.real**2 + sp.imag**2)
    psd = acc/max(1,nseg)
    return 10*np.log10(psd/ (NFFT*RATE) + 1e-20)

# ---- "pretty" figure: PSD (top) + waterfall (bottom) vertically ALIGNED.
#      GridSpec 2x2: col 0 = the 2 plots (SAME width), col 1 = colorbar (only on
#      the waterfall row) -> the colorbar does not shrink the waterfall, so x=0
#      falls on the SAME vertical in the PSD and the waterfall. sharex links the axes. ----
plt.style.use("dark_background")
fig = plt.figure(figsize=(11,8))
gs = fig.add_gridspec(2, 2, width_ratios=[40, 1], height_ratios=[1, 1.4],
                      hspace=0.18, wspace=0.015)
axp = fig.add_subplot(gs[0, 0])
axw = fig.add_subplot(gs[1, 0], sharex=axp)   # x partage -> aligne avec la PSD
cax = fig.add_subplot(gs[1, 1])               # colorbar dans sa colonne (n'affecte pas axw)
fig.suptitle(f"Calypso I/Q live - {CONT}:{CFILE}  |  ARFCN={ARFCN}  Fs={RATE/1e6:.4f} MHz",
             fontsize=11, color="#7fd")
(line,) = axp.plot(freqs, np.full(NFFT,-120), lw=0.5, color="#39f")
axp.set_xlim(freqs[0],freqs[-1]); axp.set_ylim(-130,-30)
axp.set_ylabel("PSD (dB)"); axp.grid(alpha=0.25)
axp.axvline(0, color="#f55", lw=0.7, alpha=0.7)
plt.setp(axp.get_xticklabels(), visible=False)   # l'axe x est porte par la waterfall en bas

wf = np.full((WROWS,NFFT), -120.0)
im = axw.imshow(wf, aspect="auto", origin="upper", cmap="turbo",
                extent=[freqs[0],freqs[-1],0,WROWS], vmin=-120, vmax=-50,
                interpolation="bilinear")
axw.set_ylabel("temps (frames)"); axw.set_xlabel("offset (kHz)")
axw.axvline(0, color="#fff", lw=0.7, alpha=0.5)  # meme 0 vertical que la PSD
fig.colorbar(im, cax=cax, label="dB")
fig.subplots_adjust(left=0.08, right=0.92, top=0.92, bottom=0.08)

def update(_):
    iq = grab()
    if iq is None: return line, im
    psd = welch_psd(iq)
    line.set_ydata(psd)
    lo,hi = np.percentile(psd,5), np.percentile(psd,99.5)
    axp.set_ylim(lo-5, hi+8)
    global wf
    wf = np.roll(wf,1,axis=0); wf[0,:] = psd
    im.set_data(wf); im.set_clim(lo, hi+5)
    return line, im

if SAVE:
    update(0)                       # une passe
    fig.savefig(SAVE, dpi=110, facecolor=fig.get_facecolor())
    print("ecrit", SAVE)
else:
    ani = FuncAnimation(fig, update, interval=int(REFRESH*1000),
                        blit=False, cache_frame_data=False)
    print("FFT live - ferme la fenetre pour quitter.")
    plt.show()
PYEOF
```

2.232 /opt/GSM/osmo-operator/tools/fft2.sh

687 octets, 14 lignes → 14 lignes

```
#!/usr/bin/env bash
# tools/fft2.sh - FFT of the 2nd cfile. Reuses tools/fft.sh with another CFILE.
#
# tools/fft.sh  = dsp_iq.cfile = MS  (Calypso DSP input: the mobile answering the BTS)
# tools/fft2.sh = record.cfile = BTS (what the BTS transmits)
```

```
# -> SAME signal at both ends of the chain (BTS TX <-> MS RX); the difference
#    between the two spectra = what the chain (shunt/gr-gsm) does to the signal.
#
# Usage: ./tools/fft2.sh            (default CFILE=/dev/shm/record.cfile)
#        CFILE=... ./tools/fft2.sh
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
export CFILE="${CFILE:-/dev/shm/record.cfile}"
export ARFCN="${ARFCN:-BTS-record}"
exec bash "$HERE/fft.sh" "$@"
```

2.233 /opt/GSM/osmo-operator/tools/fft_global.sh

5673 octets, 146 lignes → 146 lignes

```
#!/usr/bin/env bash
# tools/fft_global.sh - "pretty" FFT of a LOCAL cfile given as argument (static file).
#
# Derived from tools/fft.sh (osmo-operator) but WITHOUT docker / ring / live tail:
# reads a cfile directly on the host, sweeps it in a loop, and shows an
# averaged PSD (home-made Welch) + scrolling waterfall (FuncAnimation).
#
# Usage:
#   ./tools/fft_global.sh my.cfile
#   ./tools/fft_global.sh my.cfile --save spectre.png
#   RATE=1083333 NSAMP=32768 REFRESH=0.25 FORMAT=cf32 ARFCN=514 ./tools/fft_global.sh my.cfile
#
# FORMAT: cf32 (complex64, default, what the shunt writes) | cs8 | cs16
#         (osmo-trx record.cfile is often cs8 -> FORMAT=cs8)
set -euo pipefail

CFILE="${1:?usage: tools/fft_global.sh <fichier.cfile> [--save out.png]}"
[ -f "$CFILE" ] || { echo "[fft] introuvable: $CFILE"; exit 1; }
export CFILE="$(readlink -f "$CFILE")"

export RATE="${RATE:-1083333}"      # 4 SPS natif = 26e6/24
export NSAMP="${NSAMP:-32768}"      # complex samples par fenetre
export REFRESH="${REFRESH:-0.25}"   # periode de rafraichissement (s)
export ARFCN="${ARFCN:-514}"        # juste pour le titre
export FORMAT="${FORMAT:-cf32}"     # cf32 | cs8 | cs16
export DISPLAY="${DISPLAY:-:0}"
export FFT_SAVE=""
[ "${2:-}" = "--save" ] && export FFT_SAVE="${3:?--save needs a path}"

python3 - <<'PYEOF'
import os, numpy as np
import matplotlib
SAVE = os.environ.get("FFT_SAVE","")
if SAVE:
    matplotlib.use("Agg")
else:
    for bk in ("TkAgg","Qt5Agg","GTK3Agg"):
        try: matplotlib.use(bk); break
        except Exception: continue
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

CFILE = os.environ["CFILE"]
RATE  = float(os.environ["RATE"]); NS = int(os.environ["NSAMP"])
REFRESH = float(os.environ["REFRESH"]); ARFCN = os.environ["ARFCN"]
FORMAT = os.environ.get("FORMAT","cf32").lower()
NFFT  = 4096                        # resolution FFT
WROWS = 120                         # lignes du waterfall

# ---- sample format -> raw dtype + bytes per complex sample ----
```

```
if FORMAT in ("cf32","fc32","complex64"):
    RAWDT, ITEMS = np.float32, 2          # I,F float32
elif FORMAT in ("cs16","sc16","int16"):
    RAWDT, ITEMS = np.int16, 2
elif FORMAT in ("cs8","sc8","int8"):
    RAWDT, ITEMS = np.int8, 2
else:
    raise SystemExit(f"FORMAT inconnu: {FORMAT} (cf32|cs16|cs8)")
SAMP_BYTES = np.dtype(RAWDT).itemsize * ITEMS     # octets / sample complexe
FSIZE = os.path.getsize(CFILE)
NSAMP_TOT = FSIZE // SAMP_BYTES
print(f"[fft] {CFILE}  {FSIZE} o  format={FORMAT}  {NSAMP_TOT} samples  Fs={RATE/1e6:.4f} MHz")
if NSAMP_TOT < NFFT:
    raise SystemExit("[fft] fichier trop court pour une FFT")

win = np.hanning(NFFT).astype(np.float32)
freqs = np.fft.fftshift(np.fft.fftfreq(NFFT, 1.0/RATE)) / 1e3   # kHz

_off = [0]    # offset courant en SAMPLES (balayage en boucle)

def grab():
    """Lit NS samples complexes a l'offset courant, avance, boucle a la fin."""
    start = _off[0]
    if start + NS > NSAMP_TOT:
        start = 0
    raw = np.fromfile(CFILE, dtype=RAWDT, count=NS*ITEMS,
                      offset=start*SAMP_BYTES)
    _off[0] = start + NS
    if len(raw) < NFFT*ITEMS:
        return None
    raw = raw.astype(np.float32)
    iq = raw[0::2] + 1j*raw[1::2]
    if RAWDT == np.int8:   iq /= 128.0
    if RAWDT == np.int16:  iq /= 32768.0
    return iq.astype(np.complex64)

def welch_psd(iq):
    """PSD moyennee (segments NFFT, overlap 50%, Hann) en dB."""
    step = NFFT//2; nseg = max(1,(len(iq)-NFFT)//step + 1)
    acc = np.zeros(NFFT, dtype=np.float64)
    used = 0
    for i in range(nseg):
        seg = iq[i*step:i*step+NFFT]
        if len(seg) < NFFT: break
        sp = np.fft.fftshift(np.fft.fft(seg*win))
        acc += (sp.real**2 + sp.imag**2); used += 1
    psd = acc/max(1,used)
    return 10*np.log10(psd/(NFFT*RATE) + 1e-20)

# ---- "pretty" figure: PSD (top) + waterfall (bottom) aligned ----
plt.style.use("dark_background")
fig = plt.figure(figsize=(11,8))
gs = fig.add_gridspec(2, 2, width_ratios=[40, 1], height_ratios=[1, 1.4],
                      hspace=0.18, wspace=0.015)
axp = fig.add_subplot(gs[0, 0])
axw = fig.add_subplot(gs[1, 0], sharex=axp)
cax = fig.add_subplot(gs[1, 1])
fig.suptitle(f"FFT - {os.path.basename(CFILE)}  |  ARFCN={ARFCN}  Fs={RATE/1e6:.4f} MHz  fmt={FORMAT}",
             fontsize=11, color="#7fd")
(line,) = axp.plot(freqs, np.full(NFFT,-120), lw=0.5, color="#39f")
axp.set_xlim(freqs[0],freqs[-1]); axp.set_ylim(-130,-30)
axp.set_ylabel("PSD (dB)"); axp.grid(alpha=0.25)
axp.axvline(0, color="#f55", lw=0.7, alpha=0.7)
plt.setp(axp.get_xticklabels(), visible=False)

wf = np.full((WROWS,NFFT), -120.0)
```

```
im = axw.imshow(wf, aspect="auto", origin="upper", cmap="turbo",
                extent=[freqs[0],freqs[-1],0,WROWS], vmin=-120, vmax=-50,
                interpolation="bilinear")
axw.set_ylabel("temps (fenetres)"); axw.set_xlabel("offset (kHz)")
axw.axvline(0, color="#fff", lw=0.7, alpha=0.5)
fig.colorbar(im, cax=cax, label="dB")
fig.subplots_adjust(left=0.08, right=0.92, top=0.92, bottom=0.08)

def update(_):
    iq = grab()
    if iq is None: return line, im
    psd = welch_psd(iq)
    line.set_ydata(psd)
    lo,hi = np.percentile(psd,5), np.percentile(psd,99.5)
    axp.set_ylim(lo-5, hi+8)
    global wf
    wf = np.roll(wf,1,axis=0); wf[0,:] = psd
    im.set_data(wf); im.set_clim(lo, hi+5)
    return line, im

if SAVE:
    update(0)
    fig.savefig(SAVE, dpi=110, facecolor=fig.get_facecolor())
    print("ecrit", SAVE)
else:
    ani = FuncAnimation(fig, update, interval=int(REFRESH*1000),
                        blit=False, cache_frame_data=False)
    print("FFT - ferme la fenetre pour quitter.")
    plt.show()
PYEOF
```

2.234 /opt/GSM/osmo-operator/tools/full_qmd.sh

20801 octets, 591 lignes → 591 lignes

```
#!/bin/bash
# full_qmd.sh - bundle the repository as Quarto: docs + tests + .h + .c + .py
# + .sh, each file in a typed code block, "sketchy" theme.
#
# TWO MODES
#   MODE=single (default): one .qmd. Simple, but Deno/Quarto breaks
#                          ("Maximum call stack size exceeded") above
#                          ~1-2 MB, especially with embed-resources.
#   MODE=site            : multi-page Quarto project (recommended above 1 MB).
#                          Sections split into pages of at most SPLIT_KB,
#                          built-in full-text search, global index.
#
# Everything goes into ONE directory ($OUTDIR, default ./<dir>-qmd/, <dir> =
# the script's directory name): .qmd, sketchy.css, sk-filter.html, quarto
# output; a second run does not rescan its own output. Known limitation: the
# scan starts from the script's own directory (tools/); SCOPE is relative to it.
#
# Usage:
#   MODE=site ./full_qmd.sh              # -> ./<dir>-qmd/ (multi-page)
#   ./full_qmd.sh                        # -> ./<dir>-qmd/<dir>-full.qmd
#   ./full_qmd.sh out.qmd                # explicit path (single mode)
#   OUTDIR=/tmp/x ./full_qmd.sh          # elsewhere
#   PL4Y_SECTION=calypso ./full_qmd.sh   # target pl4y.store section
#   SCOPE=hw/arm/calypso ./full_qmd.sh   # only this subdir
#   EXCLUDE='build|pc-bios' ./full_qmd.sh
#   MAX_KB=256 ./full_qmd.sh             # per-file truncation
#   SPLIT_KB=300 MODE=site ./full_qmd.sh # max page size
#   EMBED=true ./full_qmd.sh             # self-contained html (may crash)
#   RENDER_MERMAID=0 ./full_qmd.sh       # .mmd as code instead of diagram
```

```
#   RENDER=0 MODE=site ./full_qmd.sh     # .qmd sources only, no render

set -uo pipefail

case "${1:-}" in -h|--help) sed -n '2,30p' "$0"; exit 0;; esac

MODE="${MODE:-single}"
# One output directory for BOTH modes, named after the script's directory, so
# bundles of different repositories can sit side by side without collision.
OUTDIR="${OUTDIR:-./$(basename "$(cd "$(dirname "$0")" && pwd)")-qmd}"
# pl4y.store section targeted by this bundle: written to .pl4y-section at the
# root of the output directory; build.mjs (in pl4y) reads it to know under
# which URL to publish.
PL4Y_SECTION="${PL4Y_SECTION:-}"
OUT="${1:-}"
SCOPE="${SCOPE:-.}"
TITLE="${TITLE:-Calypso QEMU}"
SUBTITLE="${SUBTITLE:-Full source bundle - DSP C54x / osmocom-bb / SHUNT_LEGIT}"
MAX_KB="${MAX_KB:-512}"
SPLIT_KB="${SPLIT_KB:-400}"
RENDER_MERMAID="${RENDER_MERMAID:-1}"
EMBED="${EMBED:-false}"
# In site mode rendering is the default: the produced directory is then
# directly usable in pl4y/bundles/ (it contains _site/, HTML). Without
# rendering it is only a Quarto project that nothing downstream can publish.
# RENDER=0 generates the .qmd sources only.
if [ "$MODE" = "site" ]; then RENDER="${RENDER:-1}"; else RENDER="${RENDER:-0}"; fi
EXCLUDE_RE="${EXCLUDE:-subprojects|build|pc-
 ↪  bios|tests/functional|tests/qtest|tests/unit|tests/migration|tests/qemu-
 ↪  iotests|node_modules|\.git|\.pytest_cache}"

HERE="$(cd "$(dirname "$0")" && pwd)"
cd "$HERE"

# The directory exists before anything else: both modes write into it.
mkdir -p "$OUTDIR" || { echo "ERROR: mkdir '$OUTDIR'" >&2; exit 1; }
OUTDIR="$(cd "$OUTDIR" && pwd)"

# Section slug: explicit via PL4Y_SECTION, otherwise derived from the
# repository. qemu-calypso is published under /calypso/, hence the single
# special case.
if [ -z "$PL4Y_SECTION" ]; then
    case "$(basename "$HERE")" in
        qemu-calypso) PL4Y_SECTION="calypso" ;;
        *)            PL4Y_SECTION="$(basename "$HERE")" ;;
    esac
fi
printf '%s\n' "$PL4Y_SECTION" > "$OUTDIR/.pl4y-section"

# Without argument the .qmd goes into $OUTDIR. An explicit argument is
# honoured: `./full_qmd.sh /tmp/x.qmd` writes there (css/js next to it).
if [ -z "$OUT" ]; then
    OUT="$OUTDIR/$(basename "$HERE")-full.qmd"
elif [ -d "$OUT" ]; then
    OUT="${OUT%/}/$(basename "$HERE")-full.qmd"
fi

MAXB=$(( MAX_KB * 1024 ))
SPLITB=$(( SPLIT_KB * 1024 ))
TMPD="$(mktemp -d)"
trap 'rm -rf "$TMPD"' EXIT

GITREV="$(git rev-parse --short HEAD 2>/dev/null || echo 'n/a')"
GITBRANCH="$(git rev-parse --abbrev-ref HEAD 2>/dev/null || echo 'n/a')"
NOW="$(date -Iseconds)"
```

```
# --- target + self-exclusion ------------------------------------------------
SELF_FIXED=""      # file excluded from the scan (single mode)
SELF_PREFIX=""     # directory prefix excluded from the scan (site mode)

# $OUTDIR is excluded from the scan in BOTH modes, so sketchy.css,
# sk-filter.html and the *_files/ directory of a previous render are never
# swallowed as sources.
SELF_PREFIX="${OUTDIR#"$HERE"/}/"
[ "$SELF_PREFIX" = "$OUTDIR/" ] && SELF_PREFIX=""   # $OUTDIR outside the repository

if [ "$MODE" = "site" ]; then
    DEST=""
else
    mkdir -p "$(dirname "$OUT")" || { echo "ERROR: mkdir pour '$OUT'" >&2; exit 1; }
    : > "$OUT" || { echo "ERROR: cannot write '$OUT'" >&2; exit 1; }
    OUT_ABS="$(cd "$(dirname "$OUT")" && pwd)/$(basename "$OUT")"
    SELF_FIXED="${OUT_ABS#"$HERE"/}"
    DEST="$OUT"
fi

echo "=== full_qmd.sh ==="
echo "Mode     : $MODE"
echo "Scope    : $SCOPE"
echo "Output   : $([ "$MODE" = site ] && echo "$OUTDIR/" || echo "$OUT")"
echo "Exclude  : $EXCLUDE_RE"
echo "Max/file : ${MAX_KB} ko$([ "$MODE" = site ] && echo " | page: ${SPLIT_KB} ko")"
echo

# ------------------------------------------------------------------- helpers --

_lang_for() {
    case "${1##*.}" in
        c|h)        echo c ;;
        cpp|cc|cxx) echo cpp ;;
        py)         echo python ;;
        sh|bash)    echo bash ;;
        md|qmd)     echo markdown ;;
        mmd)        echo mermaid ;;
        *)          echo default ;;
    esac
}

_fence_for() {
    local maxrun n
    maxrun=$(grep -oE '^`{3,}' "$1" 2>/dev/null | awk '{ if (length($0) > m) m = length($0) } END { print m+0 }')
    n=3; [ "${maxrun:-0}" -ge 3 ] && n=$(( maxrun + 1 ))
    printf '%*s' "$n" '' | tr ' ' '`'
}

_anchor_for() {
    printf 'f-%s' "$(printf '%s' "$1" | tr 'A-Z' 'a-z' | sed 's/[^a-z0-9]\+/-/g; s/^-//; s/-$//')"
}

_human() {
    if command -v numfmt >/dev/null 2>&1; then numfmt --to=iec --suffix=B "$1"; else echo "$1 B"; fi
}

_self_filter() {
    { if [ -n "$SELF_FIXED" ]; then grep -vxF "$SELF_FIXED"; else cat; fi; } \
  | { if [ -n "$SELF_PREFIX" ]; then grep -vF "$SELF_PREFIX"; else cat; fi; }
}

_find_into() {
    local excl="$1"; shift
    local args=() p first=1
    for p in "$@"; do
```

```
        if [ "$first" -eq 1 ]; then args+=( -name "$p" ); first=0
        else args+=( -o -name "$p" ); fi
    done
    find "$SCOPE" -type f \( "${args[@]}" \) 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" \
        | { if [ -n "$excl" ]; then grep -vE "$excl"; else cat; fi; } \
        | sed 's|^\./||' \
        | _self_filter \
        | sort > "$TMPD/list"
    wc -l < "$TMPD/list"
}

# ------------------------------------------------------------ emit one file

_add() {
    local f="$1" lang fence size nlines anchor ext eff
    ext="${f##*.}"
    lang="$(_lang_for "$f")"
    size=$(wc -c < "$f" 2>/dev/null || echo 0)
    nlines=$(wc -l < "$f" 2>/dev/null || echo 0)
    anchor="$(_anchor_for "$f")"

    {
        printf '\n## `%s` {#%s}\n\n' "$f" "$anchor"
        printf '::: {.file-meta}\n[.%s]{.tag} %s lignes - %s\n:::\n\n' "$ext" "$nlines" "$(_human "$size")"
    } >> "$DEST"

    if [ "$size" -eq 0 ]; then
        printf '::: {.callout-note appearance="simple"}\nFichier vide.\n:::\n' >> "$DEST"
        eff=0
    elif ! grep -Iq . "$f" 2>/dev/null; then
        printf '::: {.callout-warning appearance="simple"}\nFichier binaire - contenu omis.\n:::\n' >> "$DEST"
        eff=0
    else
        fence="$(_fence_for "$f")"
        if [ "$lang" = "mermaid" ] && [ "$RENDER_MERMAID" = "1" ]; then
            printf '%s{mermaid}\n' "$fence" >> "$DEST"
        else
            printf '%s{.%s .numberLines filename="%s"}\n' "$fence" "$lang" "$f" >> "$DEST"
        fi
        if [ "$size" -gt "$MAXB" ]; then
            head -c "$MAXB" "$f" >> "$DEST"
            printf '\n\n... [TRONQUE a %s ko - %s au total] ...\n' "$MAX_KB" "$(_human "$size")" >> "$DEST"
            eff=$MAXB
        else
            cat "$f" >> "$DEST"
            eff=$size
        fi
        printf '\n%s\n' "$fence" >> "$DEST"
    fi

    # manifest: section | page | file | anchor | lines | size
    printf '%s\t%s\t%s\t%s\t%s\t%s\n' \
        "$SECNUM" "$(basename "${DEST%.qmd}")" "$f" "$anchor" "$nlines" "$(_human "$size")" \
        >> "$TMPD/manifest"

    PAGE_BYTES=$(( PAGE_BYTES + eff + 400 ))
}

# ------------------------------------------------------------ site mode: page

_new_page() {
    local base
    base="$(printf 'sec%s-%02d' "$SECNUM" "$PART")"
    DEST="$OUTDIR/$base.qmd"
    printf '%s\t%s\t%s\t%s\n' "$SECNUM" "$SECTITLE" "$base.qmd" "partie $PART" >> "$TMPD/pages"
```

```
    cat > "$DEST" <<EOF
---
title: "$SECTITLE$([ "$PART" -gt 1 ] && printf ' - partie %s' "$PART")"
---

EOF
    PAGE_BYTES=0
}

# ---------------------------------------------------------------- one section

_section() {
    SECNUM="$1"; SECTITLE="$2"
    local extra_excl="$3" n f fsize
    shift 3
    n=$(_find_into "$extra_excl" "$@")
    echo "Section $SECNUM : $n files"

    if [ "$MODE" = "site" ]; then
        PART=1; _new_page
        while IFS= read -r f; do
            [ -n "$f" ] || continue
            fsize=$(wc -c < "$f" 2>/dev/null || echo 0)
            [ "$fsize" -gt "$MAXB" ] && fsize=$MAXB
            if [ "$PAGE_BYTES" -gt 0 ] && [ $(( PAGE_BYTES + fsize )) -gt "$SPLITB" ]; then
                PART=$(( PART + 1 )); _new_page
            fi
            _add "$f"
        done < "$TMPD/list"
    else
        {
            printf '\n\n# %s {#sec-%s}\n\n' "$SECTITLE" "$SECNUM"
            printf '::: {.section-count}\n**%s fichiers**\n:::\n' "$n"
        } >> "$DEST"
        PAGE_BYTES=0
        while IFS= read -r f; do
            [ -n "$f" ] && _add "$f"
        done < "$TMPD/list"
    fi
}

# ---------------------------------------------------------------------- the CSS

_write_css() {
    cat > "$1" <<'SKETCHYCSS'
@import
 ↪  url('https://fonts.googleapis.com/css2?family=Caveat:wght@600;700&family=Architects+Daughter&family=JetBrains+Mono:wght@400

:root{
  --sk-ink:#20242e;
  --sk-paper:#fffdf6;
  --sk-red:#e4572e;
  --sk-teal:#17968a;
  --sk-yellow:#ffd469;
  --sk-rough:255px 12px 225px 15px / 15px 225px 15px 255px;
  --sk-rough2:15px 225px 15px 255px / 255px 15px 225px 15px;
}

body{
  background-color:var(--sk-paper);
  background-image:
    linear-gradient(rgba(32,36,46,.05) 1px, transparent 1px),
    linear-gradient(90deg, rgba(32,36,46,.05) 1px, transparent 1px);
  background-size:28px 28px;
  font-family:'Architects Daughter','Neucha',ui-rounded,system-ui,sans-serif;
  color:var(--sk-ink);
```

```
}

h1.title{
  font-family:'Caveat',cursive; font-size:3.6rem; line-height:1.05;
  display:inline-block; padding:.1em .5em; transform:rotate(-1.1deg);
  background:var(--sk-yellow); border:3px solid var(--sk-ink);
  border-radius:var(--sk-rough); box-shadow:6px 6px 0 var(--sk-ink);
}
.subtitle{ font-size:1.2rem; opacity:.75; transform:rotate(.4deg); display:inline-block; }

h1:not(.title){
  font-family:'Caveat',cursive; font-size:2.9rem; margin-top:3.2rem;
  border-bottom:4px solid var(--sk-ink); border-radius:var(--sk-rough2);
  padding-bottom:.15em; transform:rotate(-.35deg);
}
h2{
  font-family:'JetBrains Mono',monospace; font-size:1.05rem; font-weight:700;
  margin-top:2.6rem; padding:.35em .7em; display:inline-block;
  background:#fff; border:2.5px dashed var(--sk-ink);
  border-radius:var(--sk-rough); box-shadow:3px 3px 0 rgba(23,150,138,.35);
}
h2 code{ background:none; color:var(--sk-ink); padding:0; font-size:1em; }

div.sourceCode, pre:not(.sourceCode){
  border:2.5px solid var(--sk-ink);
  border-radius:var(--sk-rough);
  box-shadow:5px 5px 0 rgba(32,36,46,.9), 10px 10px 0 rgba(228,87,46,.18);
  background:#fffef9 !important;
  transform:rotate(-.12deg);
}
code, pre, code span{ font-family:'JetBrains Mono',ui-monospace,monospace !important; }
div.sourceCode{ max-height:70vh; overflow:auto; }
code.sourceCode > span > a:first-child::before{ color:rgba(32,36,46,.28); }

.code-with-filename .code-with-filename-file{
  font-family:'Caveat',cursive !important; font-size:1.2rem; font-weight:700;
  background:var(--sk-yellow) !important; color:var(--sk-ink) !important;
  border:2.5px solid var(--sk-ink); border-bottom:none;
  border-radius:12px 110px 6px 6px / 40px 8px 6px 6px;
  transform:rotate(-.5deg); display:inline-block; padding:.15em .9em;
}
.code-with-filename .code-with-filename-file pre{ border:none; box-shadow:none; background:none; }

.file-meta{ font-size:.9rem; opacity:.8; margin:.1rem 0 .8rem .2rem; }
.file-meta .tag{
  background:var(--sk-teal); color:#fff; padding:.05em .55em; margin-right:.5em;
  border-radius:var(--sk-rough2); font-family:'JetBrains Mono',monospace; font-size:.8rem;
}
.section-count{
  display:inline-block; padding:.1em .8em; background:#fff;
  border:2px solid var(--sk-red); color:var(--sk-red);
  border-radius:var(--sk-rough2); transform:rotate(.7deg); margin-bottom:1rem;
}
.bundle-index table{ font-family:'JetBrains Mono',monospace; font-size:.82rem; width:100%; }
.bundle-index{
  border:2.5px solid var(--sk-ink); border-radius:var(--sk-rough);
  padding:.6rem 1rem; background:#fff; box-shadow:5px 5px 0 rgba(23,150,138,.3);
}
.bundle-index a{ text-decoration:none; }
.bundle-index tr:hover{ background:rgba(255,212,105,.35); }

#TOC, nav[role=doc-toc]{ font-family:'JetBrains Mono',monospace; font-size:.78rem; }
#TOC a.active, nav[role=doc-toc] a.active{ background:var(--sk-yellow); border-radius:var(--sk-rough2); }
#quarto-sidebar{ font-family:'JetBrains Mono',monospace; font-size:.85rem; }

.sk-filter{ position:sticky; top:0; z-index:50; padding:.4rem 0 .8rem; background:var(--sk-paper); }
```

```
.sk-filter input{
  width:100%; font-family:'Caveat',cursive; font-size:1.4rem; padding:.25em .8em;
  background:#fff; border:2.5px solid var(--sk-ink); border-radius:var(--sk-rough);
  box-shadow:4px 4px 0 rgba(228,87,46,.35); outline:none;
}
.sk-filter input:focus{ box-shadow:4px 4px 0 var(--sk-teal); }

a{ color:var(--sk-red); text-decoration-style:wavy; text-decoration-thickness:1px; }
blockquote{ border-left:4px solid var(--sk-teal); }
SKETCHYCSS
}

_write_filter_js() {
    cat > "$1" <<'FILTERJS'
<script>
document.addEventListener('DOMContentLoaded', function () {
  var box = document.getElementById('sk-filter');
  if (!box) return;
  var rows = Array.from(document.querySelectorAll('.bundle-index tbody tr'));
  var toc  = Array.from(document.querySelectorAll('#TOC a, nav[role=doc-toc] a'));
  box.addEventListener('input', function () {
    var q = box.value.trim().toLowerCase();
    rows.forEach(function (r) {
      r.style.display = (!q || r.textContent.toLowerCase().includes(q)) ? '' : 'none';
    });
    toc.forEach(function (a) {
      var li = a.closest('li') || a;
      li.style.display = (!q || a.textContent.toLowerCase().includes(q)) ? '' : 'none';
    });
  });
});
</script>
FILTERJS
}

: > "$TMPD/manifest"
: > "$TMPD/pages"

# ============================================================ SINGLE MODE =====

if [ "$MODE" != "site" ]; then

cat >> "$OUT" <<EOF
---
title: "$TITLE"
subtitle: "$SUBTITLE"
date: "$NOW"
date-format: "YYYY-MM-DD HH:mm"
lang: fr
format:
  html:
    theme: [sketchy]
    css: sketchy.css
    toc: true
    toc-depth: 2
    toc-location: left
    toc-title: "Fichiers"
    toc-expand: 1
    number-sections: false
    anchor-sections: true
    smooth-scroll: true
    code-copy: true
    code-overflow: scroll
    code-block-bg: true
    highlight-style: arrow
    embed-resources: $EMBED
```

```
    include-after-body: sk-filter.html
    grid:
      sidebar-width: 340px
      body-width: 1150px
      margin-width: 140px
    mermaid:
      theme: neutral
---

::: {.sk-filter}
<input id="sk-filter" type="text" placeholder="filtrer les fichiers (ex: dsp, .c, shunt)..." autocomplete="off">
:::

::: {.callout-note appearance="simple" icon=false}
**Bundle plat du depot** - doc, tests, headers, sources, python, shell.
Scope \`$SCOPE\` - git \`$GITBRANCH@$GITREV\` - genere $NOW
- exclusions \`$EXCLUDE_RE\` - troncature ${MAX_KB} ko/fichier.
:::
EOF

_write_css "$(dirname "$OUT")/sketchy.css"
_write_filter_js "$(dirname "$OUT")/sk-filter.html"

fi

# ============================================================ SITE MODE =====

if [ "$MODE" = "site" ]; then
    _write_css "$OUTDIR/sketchy.css"
    _write_filter_js "$OUTDIR/sk-filter.html"
fi

# ------------------------------------------------------------------ sections

_section 1 "1 - Documentation" ''                                  '*.md' '*.mmd' '*.qmd'
_section 2 "2 - Tests"         ''                                  'test_*.py' 'conftest.py'
_section 3 "3 - Headers"       ''                                  '*.h'
_section 4 "4 - Sources"       ''                                  '*.c' '*.cpp'
_section 5 "5 - Python"        '/tests/|test_.*\.py$|conftest\.py$' '*.py'
_section 6 "6 - Shell"         ''                                  '*.sh' '*.bash'

# ------------------------------------------------------- index + site config --

if [ "$MODE" = "site" ]; then

    # index.qmd: global table, cross-page links
    {
        cat <<EOF
---
title: "$TITLE"
subtitle: "$SUBTITLE"
date: "$NOW"
date-format: "YYYY-MM-DD HH:mm"
---

::: {.callout-note appearance="simple" icon=false}
**Bundle du depot** - doc, tests, headers, sources, python, shell.
Scope \`$SCOPE\` - git \`$GITBRANCH@$GITREV\` - genere $NOW
- exclusions \`$EXCLUDE_RE\` - troncature ${MAX_KB} ko/fichier - pages de ${SPLIT_KB} ko max.
:::

::: {.sk-filter}
<input id="sk-filter" type="text" placeholder="filtrer les fichiers (ex: dsp, .c, shunt)..." autocomplete="off">
:::
EOF
        cursec=""
```

```
        while IFS=$'\t' read -r sec page f anchor nlines size; do
            if [ "$sec" != "$cursec" ]; then
                [ -n "$cursec" ] && printf '\n:::\n'
                title="$(awk -F'\t' -v s="$sec" '$1==s {print $2; exit}' "$TMPD/pages")"
                printf '\n# %s\n\n::: {.bundle-index}\n\n' "$title"
                printf '| Fichier | Lignes | Taille |\n|:---|---:|---:|\n'
                cursec="$sec"
            fi
            printf '| [`%s`](%s.html#%s) | %s | %s |\n' "$f" "$page" "$anchor" "$nlines" "$size"
        done < "$TMPD/manifest"
        printf '\n:::\n'
    } > "$OUTDIR/index.qmd"

    # _quarto.yml
    {
        cat <<EOF
project:
  type: website
  output-dir: _site

website:
  title: "$TITLE"
  search:
    location: sidebar
    type: textbox
  sidebar:
    style: floating
    collapse-level: 1
    contents:
      - text: "Index"
        href: index.qmd
EOF
        cursec=""
        while IFS=$'\t' read -r sec title page part; do
            if [ "$sec" != "$cursec" ]; then
                printf '      - section: "%s"\n        contents:\n' "$title"
                cursec="$sec"
            fi
            printf '          - href: %s\n            text: "%s"\n' "$page" "$part"
        done < "$TMPD/pages"

        cat <<EOF

format:
  html:
    theme: [sketchy]
    css: sketchy.css
    toc: true
    toc-depth: 2
    toc-location: right
    toc-title: "Fichiers"
    number-sections: false
    anchor-sections: true
    smooth-scroll: true
    code-copy: true
    code-overflow: scroll
    code-block-bg: true
    highlight-style: arrow
    embed-resources: false
    include-after-body: sk-filter.html
    mermaid:
      theme: neutral
EOF
    } > "$OUTDIR/_quarto.yml"
fi
```

```
# ---------------------------------------------------------------- footer

if [ "$MODE" != "site" ]; then
cat >> "$OUT" <<EOF

# Fin du bundle

::: {.section-count}
genere $NOW - \`$(basename "$0")\`
:::
EOF
fi

echo
echo "=== DONE ==="
if [ "$MODE" = "site" ]; then
    echo "Output : $OUTDIR/  ($(ls "$OUTDIR"/*.qmd | wc -l) pages, $(du -sh "$OUTDIR" | cut -f1))"
    echo "Rendu  : (cd \"$OUTDIR\" && quarto render)"
    echo "Preview: (cd \"$OUTDIR\" && quarto preview)"
else
    echo "Output : $OUT ($(du -h "$OUT" | cut -f1), $(wc -l < "$OUT") lignes)"
    echo "         + sketchy.css + sk-filter.html dans $(dirname "$OUT")/"
    echo "Rendu  : quarto render \"$OUT\"   (sortie dans $(dirname "$OUT")/)"
    if [ "$(wc -c < "$OUT")" -gt 1500000 ]; then
        echo
        echo "!! $(du -h "$OUT" | cut -f1) dans un seul document : Deno risque de casser"
        echo "   (RangeError: Maximum call stack size exceeded). Utilise MODE=site."
    fi
fi

if [ "$RENDER" = "1" ]; then
    if command -v quarto >/dev/null 2>&1; then
        if [ "$MODE" = "site" ]; then (cd "$OUTDIR" && quarto render); else quarto render "$OUT"; fi
    else
        echo "quarto introuvable dans le PATH - rendu ignore." >&2
    fi
fi
```

2.235 /opt/GSM/osmo-operator/tools/full_text.sh

5936 octets, 166 lignes → 166 lignes

```
#!/bin/bash
# full_text.sh - concatenate docs + tests + .h + .c + .py + .sh, in that order,
# into a single raw .txt (ASCII separators, no markdown fences), for LLM
# ingestion or a flat archive. Relative paths are taken from the script's
# directory (it cd's there first).
#
# Usage:
#   ./full_text.sh                          # -> ./calypso-full.txt
#   ./full_text.sh out.txt                  # custom output
#   SCOPE=hw/arm/calypso ./full_text.sh     # only this subdir
#   EXCLUDE='build|pc-bios' ./full_text.sh  # extra excludes

set -uo pipefail

OUT="${1:-./calypso-full.txt}"
SCOPE="${SCOPE:-.}"
EXCLUDE_RE="${EXCLUDE:-subprojects|build|pc-
↪ bios|tests/functional|tests/qtest|tests/unit|tests/migration|tests/qemu-
↪ iotests|node_modules|\.git|\.pytest_cache}"

# If $OUT is a directory, append the default file name.
if [ -d "$OUT" ]; then
```

```
    OUT="${OUT%/}/calypso-full.txt"
fi

HERE="$(cd "$(dirname "$0")" && pwd)"
cd "$HERE"

echo "=== full_text.sh ==="
echo "Scope    : $SCOPE"
echo "Output   : $OUT"
echo "Exclude  : $EXCLUDE_RE"
echo

: > "$OUT" || { echo "ERROR: cannot write to '$OUT' (permission ? path invalide ?)" >&2; exit 1; }

# ---- Header ----
cat >> "$OUT" <<EOF
================================================================================
Calypso QEMU - Full text bundle
Generated : $(date -Iseconds)
Scope     : $SCOPE
Sections  : 1.docs  2.tests  3.headers  4.sources  5.python  6.shell
================================================================================

EOF

# Helper : add a file with ASCII separator
_add() {
    local f="$1"
    local rel="${f#./}"
    local size=$(wc -c < "$f" 2>/dev/null || echo 0)
    local nlines=$(wc -l < "$f" 2>/dev/null || echo 0)
    {
        echo ""
        echo "================================================================================"
        echo "FILE: $rel"
        echo "SIZE: $size bytes, $nlines lines"
        echo "================================================================================"
        cat "$f"
        echo ""
    } >> "$OUT"
}

# Find files of a given pattern, filtered by EXCLUDE_RE
_find() {
    find "$SCOPE" -type f \( $1 \) 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" \
        | sort
}

# ---- 1) Documentation ----
{
    echo ""
    echo "################################################################################"
    echo "# SECTION 1 : DOCUMENTATION (.md, .mmd, .qmd)"
    echo "################################################################################"
} >> "$OUT"
DOCS=$(find "$SCOPE" -type f \( -name "*.md" -o -name "*.mmd" -o -name "*.qmd" \) 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" | sort)
N=$(echo "$DOCS" | grep -c . || echo 0)
echo "Section 1 : $N files"
echo "Total docs files : $N" >> "$OUT"
for f in $DOCS; do _add "$f"; done

# ---- 2) Tests ----
{
    echo ""
```

```
    echo "################################################################################"
    echo "# SECTION 2 : TESTS (tests/*.py)"
    echo "################################################################################"
} >> "$OUT"
TESTS=$(find "$SCOPE" -type f \( -name "test_*.py" -o -name "conftest.py" \) 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" | sort)
N=$(echo "$TESTS" | grep -c . || echo 0)
echo "Section 2 : $N files"
echo "Total tests files : $N" >> "$OUT"
for f in $TESTS; do _add "$f"; done

# ---- 3) Headers ----
{
    echo ""
    echo "################################################################################"
    echo "# SECTION 3 : HEADERS (.h)"
    echo "################################################################################"
} >> "$OUT"
HDRS=$(find "$SCOPE" -type f -name "*.h" 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" | sort)
N=$(echo "$HDRS" | grep -c . || echo 0)
echo "Section 3 : $N files"
echo "Total headers files : $N" >> "$OUT"
for f in $HDRS; do _add "$f"; done

# ---- 4) Sources ----
{
    echo ""
    echo "################################################################################"
    echo "# SECTION 4 : SOURCES (.c, .cpp)"
    echo "################################################################################"
} >> "$OUT"
SRCS=$(find "$SCOPE" -type f \( -name "*.c" -o -name "*.cpp" \) 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" | sort)
N=$(echo "$SRCS" | grep -c . || echo 0)
echo "Section 4 : $N files"
echo "Total sources files : $N" >> "$OUT"
for f in $SRCS; do _add "$f"; done

# ---- 5) Python scripts (outside tests/) ----
{
    echo ""
    echo "################################################################################"
    echo "# SECTION 5 : PYTHON SCRIPTS (hors tests/)"
    echo "################################################################################"
} >> "$OUT"
PYS=$(find "$SCOPE" -type f -name "*.py" 2>/dev/null \
       | grep -vE "$EXCLUDE_RE|/tests/|test_.*\.py$|conftest\.py$" | sort)
N=$(echo "$PYS" | grep -c . || echo 0)
echo "Section 5 : $N files"
echo "Total python files : $N" >> "$OUT"
for f in $PYS; do _add "$f"; done

# ---- 6) Shell scripts ----
{
    echo ""
    echo "################################################################################"
    echo "# SECTION 6 : SHELL SCRIPTS (.sh)"
    echo "################################################################################"
} >> "$OUT"
SHS=$(find "$SCOPE" -type f \( -name "*.sh" -o -name "*.bash" \) 2>/dev/null \
        | grep -vE "$EXCLUDE_RE" | sort)
N=$(echo "$SHS" | grep -c . || echo 0)
echo "Section 6 : $N files"
echo "Total shell files : $N" >> "$OUT"
for f in $SHS; do _add "$f"; done
```

```
# ---- Footer ----
{
    echo ""
    echo "================================================================================"
    echo "END OF BUNDLE - generated $(date -Iseconds)"
    echo "================================================================================"
} >> "$OUT"

echo
echo "=== DONE ==="
echo "Output : $OUT"
echo "Size   : $(du -h "$OUT" | cut -f1)"
echo "Lines  : $(wc -l < "$OUT")"
```

2.236 /opt/GSM/osmo-operator/tools/glava-build.sh

5865 octets, 115 lignes → 115 lignes

```
#!/bin/bash
# glava-build.sh - GLava + the "NCS Spectrum" module, built and installed.
#
#     tools/glava-build.sh            on the machine (/), as root
#     tools/glava-build.sh $ROOTFS    in the ISO rootfs (iso_modules/85-glava.sh)
#
# The desktop audio visualizer: the "NCS Spectrum" circle (particles turning
# around a disc, driven by the sound coming out of the speakers), placed as a
# desktop background. Two repositories, pinned to a commit:
#   - GLava (GPL-3): no package in noble, built here. The NCS module needs a
#     patch of glava/render.c (image textures for the particles,
#     imageLoad/imageStore - OpenGL >= 4.2), provided by the NCS repository;
#   - NCS_Spectrum_GLava: ncs.glsl and ncs/ go next to the GLava modules, in
#     /etc/xdg/glava. This repository has NO licence: it is cloned at build
#     time, not copied into ours.
# rc.glsl (system) is set as the NCS README asks: module ncs, OpenGL 4.5 /
# GLSL 450, setmirror (both channels), desktop window without border or
# click. Position and size come from tools/osmo-glava.sh, which computes them
# from the screen at startup.
#
# Built on the HOST (same Ubuntu as the image), installed through DESTDIR: no
# -dev packages to install and then remove in the rootfs.
set -euo pipefail

DEST="${1:-}"
DIR="$(cd "$(dirname "$0")/.." && pwd)"
GLAVA_REPO="${GLAVA_REPO:-https://gitlab.com/wild-turtles-publicly-release/glava/glava.git}"
GLAVA_COMMIT="${GLAVA_COMMIT:-e03a8a5b721f393dd7b80738a6ac5c9bccc8f050}"
NCS_REPO="${NCS_REPO:-https://github.com/Roonil/NCS_Spectrum_GLava.git}"
NCS_COMMIT="${NCS_COMMIT:-0984455086557584e53da12915e9dbd24255fed2}"

manque=""
for t in git meson ninja cc pkg-config patch; do
    command -v "$t" >/dev/null 2>&1 || manque="$manque $t"
done
for h in pulse/simple.h X11/Xlib.h X11/extensions/Xcomposite.h X11/extensions/Xrender.h X11/extensions/shape.h
 ↪  GL/gl.h; do
    [ -f "/usr/include/$h" ] || manque="$manque $h"
done
if [ -n "$manque" ]; then
    echo "glava-build : manque$manque" >&2
    echo "  apt-get install -y git meson ninja-build gcc pkg-config patch libpulse-dev libx11-dev libxext-dev
     ↪  libxrender-dev libxcomposite-dev libgl-dev" >&2
    exit 1
fi
```

```
TMP="$(mktemp -d)"
trap 'rm -rf "$TMP"' EXIT
clone() {   # depot commit dossier
    git init -q "$3"
    git -C "$3" fetch -q --depth 1 "$1" "$2"
    git -C "$3" checkout -q FETCH_HEAD
}
clone "$GLAVA_REPO" "$GLAVA_COMMIT" "$TMP/glava"
clone "$NCS_REPO" "$NCS_COMMIT" "$TMP/ncs"

patch -s -d "$TMP/glava" glava/render.c < "$TMP/ncs/imageOperations.patch"
# -Dobs=false: the OBS plugin needs libobs, unused here
meson setup "$TMP/glava/build" "$TMP/glava" --prefix /usr -Dobs=false >/dev/null
ninja -C "$TMP/glava/build" >/dev/null
DESTDIR="$DEST" ninja -C "$TMP/glava/build" install >/dev/null

XDG="$DEST/etc/xdg/glava"
cp -r "$TMP/ncs/ncs" "$TMP/ncs/ncs.glsl" "$XDG/"
# Particles in ORANGE (the one of the BTS -> Core diagram of the wallpaper,
# 255/140/40), at the same intensity as the original blue (0.0118, 0.1412,
# 0.3412): the blend is additive, a stronger colour saturates to white.
sed -i 's/^#define color vec3(0.0118, 0.1412, 0.3412)/#define color vec3(0.3412, 0.1873, 0.0535)/' "$XDG/ncs.glsl"
grep -q '^#define color vec3(0.3412, 0.1873, 0.0535)' "$XDG/ncs.glsl" \
    || echo "glava-build : couleur NCS non changee (ncs.glsl a bouge ?)" >&2
# The bottom band (bars module, started by osmo-glava next to the circle):
# thick bars (12 px, gap 3), light blue, transparent background.
sed -i -e 's/^#define BAR_WIDTH 5$/#define BAR_WIDTH 12/' \
       -e 's/^#define BAR_GAP 1$/#define BAR_GAP 3/' \
       -e 's/^#define USE_ALPHA 0$/#define USE_ALPHA 1/' \
       -e 's/^#define COLOR @fg:mix(#3366b2, #a0a0b2, clamp(d \/ GRADIENT, 0, 1))$/#define COLOR @fg:mix(#78c3ff,
        ↪  #d6eeff, clamp(d \/ GRADIENT, 0, 1))/' \
       "$XDG/bars.glsl"
grep -q '^#define COLOR @fg:mix(#78c3ff' "$XDG/bars.glsl" \
    || echo "glava-build : couleur des barres non changee (bars.glsl a bouge ?)" >&2
sed -i -e 's/^#request mod bars/#request mod ncs/' \
       -e 's/^#request setdecorated true/#request setdecorated false/' \
       -e 's/^#request setmirror false/#request setmirror true/' \
       -e 's/^#request setversion 3 3/#request setversion 4 5/' \
       -e 's/^#request setshaderversion 330/#request setshaderversion 450/' \
       -e 's/^#request setxwintype "normal"/#request setxwintype "desktop"/' \
       -e 's/^#request setclickthrough false/#request setclickthrough true/' \
       "$XDG/rc.glsl"

install -Dm755 "$DIR/tools/osmo-glava.sh" "$DEST/usr/local/bin/osmo-glava"
# A USER SERVICE. Circle AND bars are in the same cgroup:
#   systemctl --user stop glava      stops both (and removes the mixing)
#   systemctl --user start glava     restarts them
#   systemctl --user disable glava   no more GLava at login (this account)
# Enabled for all accounts (graphical-session.target.wants in
# /etc/systemd/user): starts with the graphical session, which provides DISPLAY.
rm -f "$DEST/etc/xdg/autostart/osmo-glava.desktop"
install -d "$DEST/etc/systemd/user/graphical-session.target.wants"
cat > "$DEST/etc/systemd/user/glava.service" <<'UNIT'
[Unit]
Description=GLava : cercle NCS et barres du bas (micro + haut-parleurs)
PartOf=graphical-session.target
After=graphical-session.target pulseaudio.service pipewire-pulse.service

[Service]
ExecStart=/usr/local/bin/osmo-glava
ExecStopPost=/usr/local/bin/osmo-glava --retirer-mix
Restart=on-failure
RestartSec=5

[Install]
```

```
WantedBy=graphical-session.target
UNIT
chmod 644 "$DEST/etc/systemd/user/glava.service"
ln -sfn ../glava.service "$DEST/etc/systemd/user/graphical-session.target.wants/glava.service"

if [ -n "$DEST" ]; then chroot "$DEST" ldconfig 2>/dev/null || true; else ldconfig; fi
echo "GLava + NCS : $(git -C "$TMP/glava" rev-parse --short HEAD) / $(git -C "$TMP/ncs" rev-parse --short HEAD) ->
 ↪  ${DEST:-/}"
```

2.237 /opt/GSM/osmo-operator/tools/globals-complet.py

9127 octets, 200 lignes → 200 lignes

```
#!/usr/bin/env python3
# The TEMPLATES of the web page (web/, served by tools/osmo-reglages.py):
#
#   globals.conf.example          network settings of globals.conf + LAUNCHER variables
#                                 (start.sh, startup_scripts/*.sh): global to the chain
#   instances/instance.env.example  DSP BENCH variables (c54x_exe/run.sh): ONE VALUE PER
#                                 Calypso INSTANCE (1 = BTS/TRX or MS on the bridge,
#                                 2 = QEMU MS over the air, 3 = TRX TS1), written by the
#                                 page in instances/instance-N.env and read by run.sh
#   python3 tools/globals-complet.py            # (re)generates both templates
#   python3 tools/globals-complet.py --voir     # prints them without writing anything
#
# Each variable carries its default as a comment (file:line) and, when the scripts
# compare it to fixed values (0/1, dsp/grgsm, ...), a "choix : a | b | c" line: the page
# turns it into a drop-down menu. EMPTY value = script default. globals.conf is sourced
# with `set -a` after option parsing (03-affichage.sh): a variable already set and
# non-empty (command line) keeps its value; the page writes launcher variables left
# empty as comments (exported empty, they would break the launch).
# Excluded: the launcher internal state (C_*, *_CHOISI, computed paths), variables
# read without a colon (${VAR-x}: empty and absent differ). For run.sh, only the
# variables it renormalises itself (VAR="${VAR:-x}", export included): an empty value
# never reaches the C.

import argparse, glob, os, re, shutil, time

RACINE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
GLOBALS = os.path.join(RACINE, "globals.conf")
EXEMPLE = GLOBALS + ".example"
INSTANCES = os.path.join(RACINE, "instances")
EXEMPLE_INST = os.path.join(INSTANCES, "instance.env.example")
RUN_DSP = "/opt/GSM/c54x_exe/run.sh"
DEBUT = "# >>> VARIABLES DU LANCEUR : bloc genere par tools/globals-complet.py (ne pas editer les marqueurs) >>>"
FIN = "# <<< FIN DES VARIABLES DU LANCEUR <<<"

INTERNES = {"RUNTIME_ENV", "OQC_ROOT", "NITB_TREE", "RAND", "NODE_ID_SRC", "GEN_CONF_OUT", "SAP_FILE",
            "SOFTSIM_KI", "HUB_IP", "NO_COLOR", "RUN_SH", "HERE", "RUNDIR", "INSTANCE", "CALYPSO_NS"}
# choices by hand (the scripts do not compare all of them to fixed values)
CHOIX = {
    "BAND": "GSM900 | DCS1800 | PCS1900 | GSM850",
    "ENCRYPTION": "a5 0 | a5 1 | a5 3 | a5 0 1 | a5 0 1 3",
    "SIM_ALGO": "comp128 | comp128v2 | comp128v3 | xor | milenage",
    "CALYPSO_PROFILE": "faketrx-qemu | faketrx | qemu | noproc",
    "MODE": "dsp | grgsm",
    "OSMO_TRX_TS": "1 | 2 | 2/1",
    "SAP_TYPE": "auto | pcsc | softsim",
}
DEFAUT = re.compile(r'\$\{([A-Z][A-Z0-9_]*):([-=])')
RENORMALISE = re.compile(r'^\s*(?:export\s+)?([A-Z][A-Z0-9_]*)="?\$\{\1:-')


def defaut_apres(ligne, i):
```

```
    """The default text of a ${VAR:-...} from index i, with balanced braces."""
    prof, j = 1, i
    while j < len(ligne) and prof:
        if ligne[j] == "{":
            prof += 1
        elif ligne[j] == "}":
            prof -= 1
        j += 1
    return ligne[i:j - 1]


def extraire(fichier, seulement_renormalisees=False):
    vus = {}
    for n, l in enumerate(open(fichier, encoding="utf-8", errors="replace").read().split("\n"), 1):
        if l.lstrip().startswith("#"):
            continue
        m_r = RENORMALISE.match(l)
        if seulement_renormalisees and not m_r:
            continue
        for m in DEFAUT.finditer(l):
            v = m.group(1)
            if seulement_renormalisees and v != m_r.group(1):
                continue
            if v not in vus:
                vus[v] = (defaut_apres(l, m.end()), n)
    return vus


def sans_deux_points(fichiers):
    r = set()
    for f in fichiers:
        r |= set(re.findall(r'\$\{([A-Z][A-Z0-9_]*)[-=]', open(f, encoding="utf-8", errors="replace").read()))
    return r


def choix_auto(textes):
    """The fixed values the scripts compare each variable to: [ "$VAR" = 1 ], != "off"..."""
    comp = {}
    pat = re.compile(r'\$\{?([A-Z][A-Z0-9_]*)(?::-[^}]*)?\}?"?\s*(?:=|==|!=)\s*"?([A-Za-z0-9_./-]+)"?')
    for t in textes:
        for l in t.split("\n"):
            if l.lstrip().startswith("#"):
                continue
            for m in pat.finditer(l):
                comp.setdefault(m.group(1), []).append(m.group(2))
    return comp


def choix_de(var, defaut, comp):
    if var in CHOIX:
        return CHOIX[var]
    vals = []
    for v in comp.get(var, []):
        if v not in vals:
            vals.append(v)
    if defaut in ("0", "1") and all(v in ("0", "1") for v in vals):
        return "0 | 1"
    if len(vals) >= 2 and all(len(v) <= 24 for v in vals):
        if defaut and defaut not in vals and re.match(r'^[A-Za-z0-9_./-]+$', defaut):
            vals.insert(0, defaut)
        return " | ".join(vals)
    return ""


def titre(nom):
    return "# ── %s %s" % (nom, "─" * max(3, 58 - len(nom)))
```

```
def bloc_variables(sources, exclues, comp, ligne_var):
    deja, lignes, n = set(), [], 0
    for f, renorm in sources:
        vars_f = extraire(f, renorm)
        garde = [(v, d, l) for v, (d, l) in vars_f.items()
                 if v not in exclues and v not in deja and not v.startswith("C_") and not v.endswith("_CHOISI")]
        if not garde:
            continue
        nom = os.path.relpath(f, RACINE) if f.startswith(RACINE) else f
        lignes += ["", titre(nom), ""]
        for v, d, l in garde:
            deja.add(v)
            d = d.replace("\n", " ")
            lignes.append("# defaut : %s   (%s:%d)" % (d if d else "(vide)", os.path.basename(f), l))
            c = choix_de(v, d, comp)
            if c:
                lignes.append("# choix : %s" % c)
            lignes.append(ligne_var(v))
            n += 1
    return lignes, n


def tete_globals():
    texte = open(GLOBALS, encoding="utf-8").read()
    if DEBUT in texte and FIN in texte:
        tete, reste = texte.split(DEBUT, 1)
        queue = reste.split(FIN, 1)[1]
    else:
        tete, queue = texte, ""
            # does the template carry the network settings EMPTY of value? No: the lines are
            # kept as they are, the choices are added by hand; the page reads globals.conf.
    sortie = []
    for l in tete.rstrip("\n").split("\n"):
        m = re.match(r"^([A-Z_][A-Z0-9_]*)=", l)
        if m and m.group(1) in CHOIX and (not sortie or not sortie[-1].startswith("# choix :")):
            sortie.append("# choix : " + CHOIX[m.group(1)])
        sortie.append(l)
    declarees = set(re.findall(r"^([A-Z_][A-Z0-9_]*)=", tete + queue, re.M))
    return "\n".join(sortie), queue.strip("\n"), declarees


def generer():
    modules = [os.path.join(RACINE, "start.sh")] + sorted(glob.glob(os.path.join(RACINE, "startup_scripts",
↪   "*.sh")))
    textes = [open(f, encoding="utf-8", errors="replace").read() for f in modules + [RUN_DSP] if os.path.exists(f)]
    comp = choix_auto(textes)
    tete, queue, declarees = tete_globals()
    exclues = sans_deux_points(modules + [RUN_DSP]) | INTERNES | declarees
    lignes, n = bloc_variables([(m, False) for m in modules], exclues, comp, lambda v: '%s=""' % v)
    corps = [DEBUT, "#", "#  Valeur VIDE = defaut du script (en commentaire). Regenerer : python3
↪   tools/globals-complet.py", "#"] + lignes + ["", FIN]
    globals_txt = tete + "\n\n" + "\n".join(corps) + ("\n\n" + queue if queue else "") + "\n"

    inst_tete = [
        "# instances/instance.env.example — patron des reglages PAR INSTANCE Calypso (c54x_exe/run.sh).",
        "# Genere par tools/globals-complet.py ; la page web (tools/osmo-reglages.py) ecrit instance-N.env :",
        "#   1 = la radio de la BTS (TRX Calypso, --bts) ou le mobile sur le pont (--bts-classique)",
        "#   2 = le mobile QEMU par l air (--bts --ms, --ms-magnetite)",
        "#   3 = le 2e TRX Calypso, TS1 (--ts 2)",
        "# run.sh lit instances/instance-$INSTANCE.env au demarrage, sous la forme : \"${VAR:=valeur}\" :",
        "# l environnement (start-native.sh, ligne de commande) garde la priorite, la valeur du fichier vient",
        "# ensuite, puis le defaut de run.sh. Valeur VIDE = defaut.",
    ]
```

```
    lignes_i, n_i = bloc_variables([(RUN_DSP, True)], sans_deux_points([RUN_DSP]) | INTERNES, comp, lambda v:
↪   '%s=""' % v)
    inst_txt = "\n".join(inst_tete + lignes_i) + "\n"
    return globals_txt, n, inst_txt, n_i


if __name__ == "__main__":
    a = argparse.ArgumentParser(description="patrons de la page de reglages")
    a.add_argument("--voir", action="store_true")
    o = a.parse_args()
    g, n, i, n_i = generer()
    if o.voir:
        print(g)
        print(i)
    else:
        for chemin, texte in ((EXEMPLE, g), (EXEMPLE_INST, i)):
            os.makedirs(os.path.dirname(chemin), exist_ok=True)
            if os.path.exists(chemin):
                shutil.copy2(chemin, chemin + ".bak-" + time.strftime("%Y%m%d-%H%M%S"))
            open(chemin, "w", encoding="utf-8").write(texte)
        print("globals.conf.example : %d variables du lanceur ; instances/instance.env.example : %d variables de
        ↪  run.sh" % (n, n_i))
```

2.238 /opt/GSM/osmo-operator/tools/gnome-extensions-install.sh

2162 octets, 41 lignes → 41 lignes

```
#!/bin/bash
# gnome-extensions-install.sh - GNOME Shell extensions enabled by default.
#
#     tools/gnome-extensions-install.sh            on the machine (/), as root
#     tools/gnome-extensions-install.sh $ROOTFS    in the ISO rootfs
#                                                  (iso_modules/85-extensions-gnome.sh)
#
# EasyScreenCast (GPL-3, extensions.gnome.org no. 690): a top bar menu for the
# GNOME screen recorder (sound, webcam, area, timer). No package: the zip of
# version 53 (GNOME 46 to 48) is downloaded, checked against its sha256 and
# installed for all accounts in /usr/share/gnome-shell/extensions.
# The browser bridge chrome-gnome-shell (gnome-browser-connector) is an apt
# package: it is in the desktop list (iso_modules/80-chroot.sh).
#
# "Enabled by default" = enabled-extensions in a schema override, in the bare
# section AND :ubuntu (the ubuntu session reads that one first - same reason
# as the wallpaper and the dock in 80-chroot.sh). An account that already has
# its own list (gsettings set ...) keeps it: the override is only a default.
set -euo pipefail

DEST="${1:-}"
ESC_UUID="EasyScreenCast@iacopodeenosee.gmail.com"
ESC_URL="https://extensions.gnome.org/download-extension/$ESC_UUID.shell-extension.zip?version_tag=63091"
ESC_SHA256="e0fce1b69cf05eb365fe3d4a3e8c627c6a93beffc18a042b7896249961bc6777"

TMP="$(mktemp -d)"
trap 'rm -rf "$TMP"' EXIT
curl -fsSL -o "$TMP/esc.zip" "$ESC_URL"
echo "$ESC_SHA256  $TMP/esc.zip" | sha256sum -c --quiet -

EXT="$DEST/usr/share/gnome-shell/extensions/$ESC_UUID"
rm -rf "$EXT"
install -d "$EXT"
python3 -I -c "import sys, zipfile; zipfile.ZipFile(sys.argv[1]).extractall(sys.argv[2])" "$TMP/esc.zip" "$EXT"
chmod -R u=rwX,go=rX "$EXT"

SCH="$DEST/usr/share/glib-2.0/schemas"
```

```
printf "[org.gnome.shell]\nenabled-extensions=['%s']\n\n[org.gnome.shell:ubuntu]\nenabled-extensions=['%s']\n" \
    "$ESC_UUID" "$ESC_UUID" > "$SCH/99-osmo-extensions.gschema.override"
if [ -n "$DEST" ]; then chroot "$DEST" glib-compile-schemas /usr/share/glib-2.0/schemas; else
 ↪  glib-compile-schemas "$SCH"; fi
echo "EasyScreenCast v53 -> ${DEST:-/}usr/share/gnome-shell/extensions, active par defaut"
```

2.239 /opt/GSM/osmo-operator/tools/grgsm_relay_decode.py

3080 octets, 70 lignes → 70 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
#
# tools/grgsm_relay_decode.py - gr-gsm decoder on the relayed CONTINUOUS I/Q
# (full-grgsm mode). Reads the BTS I/Q relayed by calypso-ipc-device (UDP 5810,
# fc32), feeds it to the complete gr-gsm receiver (gsm.receiver: FCCH/SCH
# sync + GMSK demod), decodes the BCCH/CCCH and sends the L2 as GSMTAP to
# the shunt listener (4730) -> feed_si -> a_cd -> the mobile (on osmocon) camps.
#
# It is a DECODER (not a transceiver): no trxcon needed. The chain is the one
# of grgsm_decode but with a UDP source instead of a file.
#
#   udp_source(5810, fc32) -> lpf -> gsm.receiver(osr) -> C0 bursts ->
#       gsm_bcch_ccch_demapper -> control_channels_decoder -> GSMTAP(4730)
#
# osr MUST match the SPS of osmo-trx (tx-sps=1 -> osr=1). If the sync does not
# lock at osr=1, switch osmo-trx to SPS=4 and CALYPSO_TRX_OSR=4.
import os, sys

from gnuradio import gr, network, filter
from gnuradio.filter import firdes
from gnuradio.fft import window
from gnuradio import gsm

# OSR = oversampling seen by gsm.receiver. It needs ~4 to lock the FCCH/SCH
# and the GMSK timing recovery. The relayed I/Q is at 1 SPS (osmo-trx
# tx-sps=1) -> it is RESAMPLED 1->OSR in the flowgraph (no device surgery,
# no SPS=4 on the osmo-trx side).
OSR        = int(os.environ.get("CALYPSO_TRX_OSR", "4"))
IN_SPS     = int(os.environ.get("CALYPSO_TRX_IN_SPS", "1"))   # SPS du relais (osmo-trx)
SYM_RATE   = 1625000.0 / 6.0                       # 270833.33 Hz
SAMP_RATE  = SYM_RATE * OSR                         # rate après resampling
RX_PORT    = int(os.environ.get("CALYPSO_TRX_IQ_RX_PORT", "5810"))
GSMTAP_HOST = os.environ.get("GSMTAP_HOST", "127.0.0.1")
GSMTAP_PORT = int(os.environ.get("CALYPSO_SHUNT_GSMTAP_PORT", "4730"))
TS         = int(os.environ.get("TS", "0"))

tb = gr.top_block("grgsm relay decode")

# Source: relayed continuous I/Q (fc32, 625 samples/datagram = 5000 B). src_zeros
# -> continuous stream when there is no data (the receiver does not stall).
src = network.udp_source(gr.sizeof_gr_complex, 1, RX_PORT,
                         0, 5000, False, True, False)

# Resampling 1->OSR (IN_SPS->OSR): gives gsm.receiver the ~4 SPS it expects.
resamp = filter.rational_resampler_ccc(interpolation=OSR, decimation=IN_SPS)

lpf = filter.fir_filter_ccf(1, firdes.low_pass(
    1, SAMP_RATE, 125e3, 5e3, window.WIN_HAMMING, 6.76))

rx  = gsm.receiver(OSR, [0], [])              # cell_allocation [0], tseq []
dm  = gsm.gsm_bcch_ccch_demapper(TS)
dec = gsm.control_channels_decoder()
snk = network.socket_pdu("UDP_CLIENT", GSMTAP_HOST, str(GSMTAP_PORT), 10000)
```

```
tb.connect((src, 0), (resamp, 0))
tb.connect((resamp, 0), (lpf, 0))
tb.connect((lpf, 0), (rx, 0))
tb.msg_connect((rx, 'C0'), (dm, 'bursts'))
tb.msg_connect((dm, 'bursts'), (dec, 'bursts'))
tb.msg_connect((dec, 'msgs'), (snk, 'pdus'))

print("[grgsm-relay-decode] I/Q udp:%d (osr=%d, %.0f Hz) -> GSMTAP %s:%d -> feed_si"
      % (RX_PORT, OSR, SAMP_RATE, GSMTAP_HOST, GSMTAP_PORT), flush=True)

tb.start()
try:
    tb.wait()
except KeyboardInterrupt:
    tb.stop(); tb.wait()
```

2.240 /opt/GSM/osmo-operator/tools/grub-theme-render.py

6762 octets, 163 lignes → 163 lignes

```
#!/usr/bin/env python3
# grub-theme-render.py - builds the "osmo" GRUB theme.
#
#     python3 tools/grub-theme-render.py --out /boot/grub/themes/osmo
#
# Like the Plymouth theme (tools/plymouth-render.py): THE THEME IS GENERATED,
# NOT STORED. The drawing is the code; change a colour, rerun the script.
# Called by iso_modules/86-finitions.sh (installed system) and
# iso_modules/90-iso.sh (live menu).
#
# WHAT IS SHOWN:
#   - as background, the city of the wallpaper (configs/gsm-lab-ville.jpg) blurred
#     and darkened, in the same dark purple -> blue gradient;
#   - at the top, "osmo-operator" and, in orange, "localhost · no radio emission"
#     (the frame of the desktop diagram);
#   - in the middle, the menu in a translucent panel with a blue border; the
#     selected entry is a pill in a purple -> blue gradient;
#   - at the bottom, the countdown (orange) and the keyboard help.
#
# The fonts (.pf2) are made by grub-mkfont (grub-common) from Ubuntu
# (fonts-ubuntu): the same family as the desktop.
import argparse
import os
import subprocess
from PIL import Image, ImageDraw, ImageFilter, ImageFont

REPO = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
FONTS = "/usr/share/fonts/truetype/ubuntu"
W, H = 1920, 1080
ORANGE = (255, 140, 40)
VIOLET = (58, 30, 110)
BLEU = (70, 150, 245)
BORD = (110, 175, 255)

p = argparse.ArgumentParser()
p.add_argument("--out", required=True)
p.add_argument("--fond", default=os.path.join(REPO, "configs", "gsm-lab-ville.jpg"))
a = p.parse_args()
os.makedirs(a.out, exist_ok=True)
out = lambda n: os.path.join(a.out, n)


def police(f, t):
```

```
    return ImageFont.truetype(os.path.join(FONTS, f), t)


# ── background ──────────────────────────────────────────────────────
src = Image.open(a.fond).convert("RGB")
r = max(W / src.width, H / src.height)
src = src.resize((round(src.width * r), round(src.height * r)), Image.LANCZOS)
x0, y0 = (src.width - W) // 2, (src.height - H) // 2
fond = src.crop((x0, y0, x0 + W, y0 + H)).filter(ImageFilter.GaussianBlur(30))
# veil: dark purple at the top left -> night blue at the bottom right
voile = Image.new("RGB", (W, H))
dv = ImageDraw.Draw(voile)
for i in range(W + H):
    t = i / (W + H)
    c = (round(22 * (1 - t) + 6 * t), round(8 * (1 - t) + 14 * t), round(44 * (1 - t) + 40 * t))
    dv.line([(i, 0), (i - H, H)], fill=c, width=2)
fond = Image.blend(fond, voile, 0.62)

d = ImageDraw.Draw(fond)
titre = police("Ubuntu-B.ttf", 74)
d.text((W / 2, 150), "osmo-operator", font=titre, fill=(245, 245, 250), anchor="mm")
sous = police("Ubuntu-M.ttf", 30)
d.text((W / 2, 222), "localhost  ·  no radio emission", font=sous, fill=ORANGE, anchor="mm")
lw = d.textlength("localhost  ·  no radio emission", font=sous)
d.line([(W / 2 - lw / 2, 252), (W / 2 + lw / 2, 252)], fill=ORANGE + (255,), width=2)
fond.save(out("background.png"), optimize=True)


# ── menu panel (9 pieces: menu_*.png) ──────────────────────────────
def neuf(prefixe, taille, rayon, remplissage, bord, epais):
    """Draws a rounded rectangle and cuts it into 9 pieces for GRUB."""
    s = taille
    im = Image.new("RGBA", (3 * s, 3 * s), (0, 0, 0, 0))
    ImageDraw.Draw(im).rounded_rectangle((0, 0, 3 * s - 1, 3 * s - 1), radius=rayon,
                                         fill=remplissage, outline=bord, width=epais)
    noms = {"nw": (0, 0), "n": (1, 0), "ne": (2, 0), "w": (0, 1), "c": (1, 1),
            "e": (2, 1), "sw": (0, 2), "s": (1, 2), "se": (2, 2)}
    for nom, (i, j) in noms.items():
        im.crop((i * s, j * s, (i + 1) * s, (j + 1) * s)).save(out("%s_%s.png" % (prefixe, nom)))


neuf("menu", 24, 22, (12, 8, 30, 175), BORD + (200,), 2)

# ── selected entry (select_w / select_c / select_e): gradient pill ──────────
HS = 44
pastille = Image.new("RGBA", (400, HS), (0, 0, 0, 0))
masque = Image.new("L", (400, HS), 0)
ImageDraw.Draw(masque).rounded_rectangle((0, 0, 399, HS - 1), radius=HS // 2, fill=255)
deg = Image.new("RGBA", (400, HS))
dd = ImageDraw.Draw(deg)
for x in range(400):
    t = x / 399
    dd.line([(x, 0), (x, HS)], fill=tuple(round(VIOLET[k] * (1 - t) + BLEU[k] * t) for k in range(3)) + (235,))
pastille.paste(deg, (0, 0), masque)
pastille.crop((0, 0, HS // 2, HS)).save(out("select_w.png"))
pastille.crop((HS // 2, 0, 400 - HS // 2, HS)).save(out("select_c.png"))
pastille.crop((400 - HS // 2, 0, 400, HS)).save(out("select_e.png"))

# ── fonts ──────────────────────────────────────────────────────────
# -r: ASCII, Latin-1 and Latin extended A (the accents of the entries), the
# middle dot of the help. -n sets the name: "Osmo Regular 24".
plages = "0x20-0x7E,0xA0-0x17F,0x2022-0x2022"
for fichier, nom, taille in (("Ubuntu-R.ttf", "Osmo", 24), ("Ubuntu-M.ttf", "OsmoM", 22),
                             ("Ubuntu-R.ttf", "Osmo", 18)):
    subprocess.run(["grub-mkfont", "-s", str(taille), "-n", nom, "-r", plages,
                    "-o", out("%s-%d.pf2" % (nom.lower(), taille)), os.path.join(FONTS, fichier)],
```

```
                  check=True)

# ── theme.txt ──────────────────────────────────────────────────────────
theme = """# GENERE par tools/grub-theme-render.py - ne pas modifier a la main.
title-text: ""
desktop-image: "background.png"
desktop-color: "#0b0820"
terminal-font: "Osmo Regular 18"
terminal-box: "menu_*.png"

+ boot_menu {
    left = 27%
    top = 33%
    width = 46%
    height = 42%
    menu_pixmap_style = "menu_*.png"
    item_font = "Osmo Regular 24"
    item_color = "#c9cde0"
    selected_item_font = "OsmoM Regular 22"
    selected_item_color = "#ffffff"
    selected_item_pixmap_style = "select_*.png"
    item_height = 44
    item_spacing = 8
    item_padding = 18
    icon_width = 0
    icon_height = 0
    item_icon_space = 0
    scrollbar = false
}

+ label {
    id = "__timeout__"
    left = 0
    top = 84%
    width = 100%
    align = "center"
    font = "Osmo Regular 24"
    color = "#ff8c28"
    text = "Démarrage automatique dans %d s"
}

+ label {
    left = 0
    top = 91%
    width = 100%
    align = "center"
    font = "Osmo Regular 18"
    color = "#8f96b8"
    text = "flèches  choisir    •    Entrée  démarrer    •    e  modifier    •    c  console"
}
"""
with open(out("theme.txt"), "w") as f:
    f.write(theme)
print(a.out)
```

2.241 /opt/GSM/osmo-operator/tools/make-docker-image.sh

413 octets, 10 lignes → 10 lignes

```
#!/bin/bash
# tools/make-docker-image.sh - build the osmocom-nitb docker image.
#
# Delegates to build.sh: apt-fast, docker compose v2 + buildx among the
# dependencies, .deb cache on the host (/var/cache/osmo-debs). All arguments
```

```
# are passed through unchanged.
#
#   sudo ./tools/make-docker-image.sh [--no-cache] [--lite] [--stp]
set -euo pipefail
exec bash "$(dirname "$(readlink -f "$0")")/../build.sh" "$@"
```

2.242 /opt/GSM/osmo-operator/tools/matrix.sh

6122 octets, 119 lignes → 119 lignes

```
#!/usr/bin/env bash
# tools/matrix.sh <burst|si> - DYNAMIC MATRIX of the 51-multiframe trellis (mobile side).
#
# Two complementary "time/channel" views of the 2 FFTs (frequency):
#   tools/matrix.sh burst  -> BURST dynamics (TRXD ts0 DL :5702):
#                       rows = FN%51 (channel structure), columns = time,
#                       colour = empty / dummy(idle) / real(active).
#   tools/matrix.sh si     -> dynamics of the decoded SI/L2 (GSMTAP :4729/:4730):
#                       rows = FN%51, colour = message type (SI/IMM/UA/PAGING/?).
#
# PASSIVE sniff (AF_PACKET via gsm_sniff.py in the container, steals nothing, no
# fifo) -> pipe -> matplotlib rendering on the host (X). Close the window to quit.
#
# Usage: ./tools/matrix.sh burst    |    ./tools/matrix.sh si
#        CONTAINER=osmo-operator-66 TS=0 W=240 ./tools/matrix.sh burst
set -euo pipefail
MODE="${1:?usage: tools/matrix.sh <burst|si>}"
case "$MODE" in burst|si) ;; *) echo "mode invalide: $MODE (burst|si)"; exit 1;; esac
export MODE
export CONTAINER="${CONTAINER:-osmo-operator-66}"
export TS="${TS:-0}"          # timeslot a afficher (0 = canal combine CCCH+SDCCH4)
export W="${W:-240}"          # colonnes (multiframes d'historique)
export DISPLAY="${DISPLAY:-:0}"
PY="${PY:-/root/.env/bin/python3}"; command -v "$PY" >/dev/null 2>&1 || PY=python3

SNIFF=/opt/GSM/qosmo-grgsm/opt-gsm-scripts/gsm_sniff.py
docker exec "$CONTAINER" test -f "$SNIFF" || { echo "gsm_sniff absent dans $CONTAINER"; exit 1; }

# gsm_sniff (container, passive) -> stdout -> host python (live matrix rendering)
docker exec "$CONTAINER" sh -c "SNIFF_IFACE=lo stdbuf -oL python3 $SNIFF $MODE" \
| "$PY" - <<'PYEOF'
import sys, os, re, threading, collections
import numpy as np, matplotlib
matplotlib.use("TkAgg") if not os.environ.get("MATRIX_SAVE") else matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.colors import ListedColormap, BoundaryNorm

MODE = os.environ.get("MODE","burst"); TS = int(os.environ.get("TS","0")); W = int(os.environ.get("W","240"))
DUMMY = "00011111011011101100000101001001"   # debut du dummy burst GSM (idle)

# --- SI/L2 type -> colour code (si mode) ---
SI_CODE = {"?":1,"SI1":2,"SI2":3,"SI3":4,"SI4":5,"SI2bis":6,"SI2ter":7,"SI13":8,
           "PAGING-REQ-1":9,"PAGING-REQ-2":9,"PAGING-REQ-3":9,
           "IMM-ASSIGN":10,"IMM-ASSIGN-EXT":10,"IMM-ASSIGN-REJ":10}
# LAPDm (SDCCH/SACCH) detected by control byte: tagged "DEDIE"
SI_LABELS = ["·","?","SI1","SI2","SI3","SI4","SI2b","SI2t","SI13","PAG","IMM","DEDIE"]
SI_COLORS = ["#101018","#888","#1f77b4","#2ca02c","#17becf","#9467bd","#8c564b","#e377c2",
             "#7f7f7f","#ff7f0e","#d62728","#ffd700"]

mat = np.zeros((51, W)); curmf=[None]; lock=threading.Lock()

re_ts  = re.compile(r"ts(\d)\b")
re_m51 = re.compile(r"%51=(\d+)")
```

```
re_fn  = re.compile(r"\bfn=(\d+)|\bFN=(\d+)")
re_148 = re.compile(r"\b([01]{148})\b")
re_si  = re.compile(r"\b(SI2bis|SI2ter|SI1[3]?|SI[2-4]|IMM-ASSIGN-?\w*|PAGING-REQ-\d)\b")
re_port= re.compile(r":(\d{4})\b")

def reader():
    for line in sys.stdin:
        m51 = re_m51.search(line);  mts = re_ts.search(line)
        if not m51: continue
        if mts and int(mts.group(1)) != TS:   # filtre timeslot (defaut ts0)
            # in si mode the :4730 lines do not always carry ts -> keep
            if MODE=="burst": continue
        fnm = re_fn.search(line); fn = int(fnm.group(1) or fnm.group(2)) if fnm else None
        r = int(m51.group(1)) % 51
        if MODE=="burst":
            mb = re_148.search(line)
            if "NOPE" in line or "vide" in line: v = 0.0          # burst vide
            elif mb and mb.group(1).startswith(DUMMY): v = 1.0    # dummy (idle)
            elif mb: v = 2.0                                       # reel (actif)
            else: v = 0.0
        else:
            ms = re_si.search(line)
            if ms: v = float(SI_CODE.get(ms.group(1), 1))
            elif "?" in line:                                      # LAPDm dedie (SDCCH/SACCH UA/I)
                v = 11.0 if (22<=r<=25 or 26<=r<=39 or 42<=r<=49) else 1.0
            else: v = 1.0
        mf = fn//51 if fn is not None else (curmf[0] or 0)
        with lock:
            if curmf[0] is None: curmf[0]=mf
            while mf > curmf[0]:                # nouvelle multiframe -> shift gauche
                mat[:, :-1] = mat[:, 1:]; mat[:, -1] = 0; curmf[0]+=1
            if mf == curmf[0]:
                mat[r, -1] = v

threading.Thread(target=reader, daemon=True).start()

plt.style.use("dark_background")
fig, ax = plt.subplots(figsize=(12,7))
title = ("BURSTS ts%d (·=vide  bleu=dummy/idle  rouge=ACTIF)" % TS) if MODE=="burst" \
        else "SI / L2 decodes (type par couleur)"
fig.suptitle("Matrice dynamique 51-multiframe - %s  [%s]" % (title, os.environ.get("CONTAINER","")),
             color="#7fd", fontsize=11)
if MODE=="burst":
    cmap = ListedColormap(["#101018","#1f77b4","#d62728"]); norm = BoundaryNorm([-.5,.5,1.5,2.5], cmap.N)
else:
    cmap = ListedColormap(SI_COLORS); norm = BoundaryNorm([i-.5 for i in range(len(SI_COLORS)+1)], cmap.N)
im = ax.imshow(mat, aspect="auto", origin="lower", cmap=cmap, norm=norm, interpolation="nearest")
ax.set_ylabel("FN % 51  (FCCH=0,10..  SCH=1,11..  BCCH=2-5  CCCH=6-19  SDCCH=22-39  SACCH=42-49)")
ax.set_xlabel("temps (multiframes 51) →")
# channel band markers (combined BCCH+SDCCH4 ts0)
for y in (0,10,20,30,40): ax.axhline(y, color="#444", lw=.4)
for y0,y1,lbl,c in [(2,5,"BCCH","#17becf"),(22,25,"SDCCH/0","#ffd700"),(42,45,"SACCH/0","#ff7f0e")]:
    ax.axhspan(y0-.5,y1+.5, xmin=0, xmax=0.015, color=c, alpha=.8)
if MODE=="si":
    from matplotlib.patches import Patch
    ax.legend(handles=[Patch(color=SI_COLORS[i], label=SI_LABELS[i]) for i in range(1,len(SI_LABELS))],
              loc="upper left", ncol=4, fontsize=7, framealpha=.3)

def update(_):
    with lock: im.set_data(mat.copy())
    return (im,)
ani = FuncAnimation(fig, update, interval=250, blit=False, cache_frame_data=False)
print("[matrix:%s] fenetre live - ferme pour quitter." % MODE, flush=True)
plt.show()
PYEOF
```

2.243 /opt/GSM/osmo-operator/tools/osmo-banc-cmd.sh

1982 octets, 40 lignes → 40 lignes

```
#!/bin/bash
# osmo-banc-cmd.sh - the bench aliases and functions (scripts/osmo-banc-aliases.sh) as commands.
#
# banc-rssi, banc-trx, banc-mobile... are defined in /etc/profile.d/98-osmo-banc-aliases.sh,
# which only login shells source: nothing from Alt+F2, sudo, a script or a shell that does
# not read profile.d. This script is linked under each of these names in /usr/local/bin; it
# sources the alias file and replays its own name with its arguments. The definition stays
# in ONE place: scripts/osmo-banc-aliases.sh.
#
#   tools/osmo-banc-cmd.sh --install [DIR]   install the links (default /usr/local/bin; DIR
#                                            may be a $ROOTFS/usr/local/bin) for each banc-*
#                                            and c123-* of the file (c123 itself is a direct
#                                            link to tools/osmo-c123-gtk.py, without
#                                            --instance). The links point to
#                                            $OSMO_BANC_TREE/tools (default: /opt/GSM/osmo-operator)
REPO=/opt/GSM/osmo-operator
SELF="$(readlink -f "${BASH_SOURCE[0]}")"
ALIASES="$(dirname "$SELF")/../scripts/osmo-banc-aliases.sh"
[ -f "$ALIASES" ] || ALIASES="$REPO/scripts/osmo-banc-aliases.sh"

noms() {
    sed -n -e 's/^alias \([a-z0-9-]*\)=.*/\1/p' -e 's/^function \([a-z0-9-]*\) .*/\1/p' "$ALIASES" \
        | grep -E '^(banc-|c123-)' | sort -u
}

if [ "${1:-}" = "--install" ]; then
    dest="${2:-/usr/local/bin}"
    install -d "$dest"
    for n in $(noms); do ln -sf "${OSMO_BANC_TREE:-$REPO}/tools/osmo-banc-cmd.sh" "$dest/$n"; done
    echo "commandes du banc -> $dest : $(noms | tr '\n' ' ')"
    exit 0
fi

nom="$(basename "$0")"
shopt -s expand_aliases
# shellcheck source=../scripts/osmo-banc-aliases.sh
. "$ALIASES"
type "$nom" >/dev/null 2>&1 || { echo "$nom : inconnu de $ALIASES" >&2; exit 127; }
# eval: an alias is only expanded when a NEW line is read, after the source
eval "$nom \"\$@\""
```

2.244 /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py

27103 octets, 620 lignes → 620 lignes

```
#!/usr/bin/python3
# -*- coding: utf-8 -*-
"""osmo-c123-gtk.py - the emulated Motorola C123, as a photo: embedded LCD screen, clickable keys.

The QEMU of the repository (qosmo) has no gtk backend; it publishes the ST7558
screen in /dev/shm/calypso_lcd (hw/arm/calypso/calypso_st7558.c) and reads the
keys on a Unix datagram socket (/tmp/calypso_keys.sock, calypso_armio.c). This
GTK3 window does the rest:
  - the photo of the phone (data/c123.jpg or .png; without photo, a diagram);
    --modele c139: data/c139.jpg + data/c139-zones.json and the 96x64 colour
    screen published by calypso_uwire.c (FreeCalypso Magnetite c139);
    auto (default) = from the magic of the shared image;
  - the 96x65 screen redrawn on every change, embedded in the "lcd" zone;
  - a click on a key of the photo = press/release to QEMU; the PC keyboard
```

```
    works too (0-9, * or s, # or h, arrows, Enter = menu, keypad Enter or
    o = ok, Esc or p = power, F1/F2 = soft keys).
Several instances (c54x_exe/run.sh INSTANCE=N: screen
/dev/shm/calypsoN/calypso_lcd, keys /tmp/calypso_keysN.sock). Without
--instance, the window picks the first live instance that is NOT a TRX of the
BTS (trx firmware: /tmp/c54x-pontN/transceiver.log or "BTS MODE" in
osmocon.log) - the --bts --ms setup puts the mobile in instance 2. The power
key (red, Esc, p) switches the firmware off as soon as it is pressed (trx as
well as layer1: "Powering off due to keypress"): it is only sent after a LONG
PRESS (%.1f s), and never to a TRX instance (a power sent to the TRX instance
switches the BTS radio off and the BTS loses its clock).
Zones: data/c123-zones.json (x, y, l, h normalised). `--calibrate` redefines
them with the mouse: for each announced zone, click the top-left corner then
the bottom-right corner; `s` saves, `q` quits.

  /usr/bin/python3 tools/osmo-c123-gtk.py [--image PHOTO] [--zones JSON]
        [--instance N] [--lcd /dev/shm/calypso_lcd] [--keys /tmp/calypso_keys.sock]
        [--scale 2.5] [--calibrate] [--power-libre] [--toutes [1,2,3]]
(system python3: the one of /root/.env has no PyGObject.)
"""
import argparse
import json
import os
import socket
import struct
import sys
import time

try:
    import gi
except ModuleNotFoundError:
    # started with the venv python (/root/.env: no PyGObject): re-execute with the
    # system python, which has gi; "pip install gi" is not the right package.
    if os.path.realpath(sys.executable) != os.path.realpath("/usr/bin/python3") and
     ↪  os.path.exists("/usr/bin/python3"):
        os.execv("/usr/bin/python3", ["/usr/bin/python3", os.path.abspath(__file__)] + sys.argv[1:])
    sys.stderr.write("PyGObject (gi) introuvable : apt install python3-gi gir1.2-gtk-3.0, et lancer avec
 ↪  /usr/bin/python3\n")
    sys.exit(1)
gi.require_version("Gtk", "3.0")
gi.require_version("GdkPixbuf", "2.0")
from gi.repository import Gtk, Gdk, GLib, GdkPixbuf  # noqa: E402
import cairo  # noqa: E402

HERE = os.path.dirname(os.path.realpath(__file__))
DATA = os.path.join(HERE, "..", "data")
LCD_W, LCD_H, LCD_PAGES, LCD_HDR = 96, 65, 9, 32
POWER_APPUI_LONG_S = 1.2       # hold time on power before sending it to the firmware
__doc__ = __doc__ % POWER_APPUI_LONG_S
ORDRE_ZONES = ["lcd", "lsb", "rsb", "up", "left", "menu", "right", "down", "ok", "power",
               "1", "2", "3", "4", "5", "6", "7", "8", "9", "star", "0", "hash"]
LIBELLES = {"lsb": "—", "rsb": "—", "up": "▲", "down": "▼", "left": "◀", "right": "▶",
            "menu": "●", "ok": "☎", "power": "⏻", "star": "*", "hash": "#"}
CLAVIER = {  # GDK key name -> C123 key
    "0": "0", "1": "1", "2": "2", "3": "3", "4": "4", "5": "5", "6": "6", "7": "7", "8": "8", "9": "9",
    "KP_0": "0", "KP_1": "1", "KP_2": "2", "KP_3": "3", "KP_4": "4", "KP_5": "5", "KP_6": "6",
    "KP_7": "7", "KP_8": "8", "KP_9": "9", "asterisk": "star", "KP_Multiply": "star", "s": "star",
    "numbersign": "hash", "h": "hash", "Up": "up", "Down": "down", "Left": "left", "Right": "right",
    "Return": "menu", "KP_Enter": "ok", "o": "ok", "Escape": "power", "p": "power",
    "F1": "lsb", "F2": "rsb",
}


def rundir_instance(n):
    return "/tmp/c54x-pont" if n == 1 else "/tmp/c54x-pont%d" % n
```

```
def chemins_instance(n):
    """(screen, key socket) of instance n (c54x_exe/run.sh: INSTANCE=1 without suffix)."""
    if n == 1:
        return "/dev/shm/calypso_lcd", "/tmp/calypso_keys.sock"
    return "/dev/shm/calypso%d/calypso_lcd" % n, "/tmp/calypso_keys%d.sock" % n


def instance_vivante(n):
    try:
        with open(os.path.join(rundir_instance(n), "dsp.pid")) as f:
            pid = int(f.read().strip() or 0)
        os.kill(pid, 0)
        return True
    except (OSError, ValueError):
        return False


def instance_est_trx(n):
    """True if instance n runs the trx firmware (radio of the BTS): a power switches it off and the
    BTS loses its clock. Clues: transceiver.log in its RUNDIR (run.sh OSMO_TRX=1), or "BTS MODE" on
    its console (trx.highram prints BTS MODE: ... from the L1CTL_RESET on)."""
    rd = rundir_instance(n)
    if os.path.isfile(os.path.join(rd, "transceiver.log")):
        return True
    try:
        with open(os.path.join(rd, "osmocon.log"), "rb") as f:
            return b"BTS MODE:" in f.read(1 << 20)
    except OSError:
        return False


def instance_par_defaut():
    """The first live instance that is not a TRX (the mobile of the --bts --ms setup is 2);
    else the first live one; else 1."""
    vivantes = [n for n in range(1, 9) if instance_vivante(n)]
    for n in vivantes:
        if not instance_est_trx(n):
            return n
    return vivantes[0] if vivantes else 1


def magic_lcd(path):
    """The magic of the shared image: "C123LCD" (mono ST7558) or "C139LCD" (colour), None if absent."""
    try:
        with open(path, "rb") as f:
            m = f.read(7)
    except OSError:
        return None
    return m.decode("ascii", "replace") if m in (b"C123LCD", b"C139LCD") else None


class Lcd:
    """The shared image: 32-byte header (magic, width, height, pages or bytes per pixel, flags,
    counter) then the pixels. Two magics: "C123LCD" (calypso_st7558.c: 96 x 9 pages, 1 bit per pixel,
    bit 0 at the top) and "C139LCD" (calypso_uwire.c, Magnetite c139: 96x64 little-endian RGB565)."""

    def __init__(self, path):
        self.path = path
        self.seq = -1
        self.flags = 0
        self.pages = b""
        self.w, self.h, self.modele = LCD_W, LCD_H, "c123"
        self.surface = cairo.ImageSurface(cairo.FORMAT_RGB24, self.w, self.h)
        self.vivant = False
        self._peindre()
```

```
def lire(self):
    try:
        with open(self.path, "rb") as f:
            buf = f.read(LCD_HDR + 96 * 64 * 2)
    except OSError:
        if self.vivant:
            self.vivant = False
            return True
        return False
    if len(buf) < LCD_HDR + 8 or buf[:7] not in (b"C123LCD", b"C139LCD"):
        return False
    w, h, pages, flags, _pad, seq = struct.unpack_from("<HHHBBI", buf, 8)
    modele = "c139" if buf[:7] == b"C139LCD" else "c123"
    taille = w * h * pages if modele == "c139" else w * pages
    if not (1 <= w <= 256 and 1 <= h <= 256) or len(buf) < LCD_HDR + taille:
        return False
    if seq == self.seq and self.vivant:
        return False
    if (w, h, modele) != (self.w, self.h, self.modele):
        self.w, self.h, self.modele = w, h, modele
        self.surface = cairo.ImageSurface(cairo.FORMAT_RGB24, w, h)
    self.seq, self.flags, self.pages, self.vivant = seq, flags, buf[LCD_HDR:LCD_HDR + taille], True
    self._peindre()
    return True

def _peindre(self):
    if self.modele == "c139":
        self._peindre_c139()
        return
    inverse, eteint = bool(self.flags & 1), bool(self.flags & 2)
    on, off = (0x1a2a1a, 0xc8d8b0) if not inverse else (0xc8d8b0, 0x1a2a1a)
    if eteint or not self.vivant:
        on = off = 0x9aa892
    data = self.surface.get_data()
    stride = self.surface.get_stride()
    pages = self.pages
    for y in range(self.h):
        page, mask = y >> 3, 1 << (y & 7)
        base = page * self.w
        for x in range(self.w):
            bit = pages[base + x] & mask if pages else 0
            c = on if bit else off
            o = y * stride + x * 4
            data[o:o + 4] = struct.pack("<I", c)
    self.surface.mark_dirty()

def _peindre_c139(self):
    """RGB565 -> RGB24; without QEMU: uniform grey."""
    data = self.surface.get_data()
    stride = self.surface.get_stride()
    src = self.pages
    for y in range(self.h):
        for x in range(self.w):
            if src and self.vivant:
                i = (y * self.w + x) * 2
                p = src[i] | (src[i + 1] << 8)
                r, g, b = (p >> 11) & 0x1f, (p >> 5) & 0x3f, p & 0x1f
                c = ((r << 3 | r >> 2) << 16) | ((g << 2 | g >> 4) << 8) | (b << 3 | b >> 2)
            else:
                c = 0x9aa892
            o = y * stride + x * 4
            data[o:o + 4] = struct.pack("<I", c)
    self.surface.mark_dirty()
```

```
class Clavier:
    """The datagram socket of calypso_armio.c: "press X", "release X", "tap X ms"."""

    def __init__(self, path):
        self.path = path
        self.sock = socket.socket(socket.AF_UNIX, socket.SOCK_DGRAM)
        self.erreur = None

    def envoyer(self, cmd, touche):
        try:
            self.sock.sendto(("%s %s\n" % (cmd, touche)).encode(), self.path)
            self.erreur = None
            return True
        except OSError as e:
            self.erreur = str(e)
            return False


FENETRES = []


def fenetre_fermee(_w):
    """A window closed; Gtk.main stops with the last one."""
    if _w in FENETRES:
        FENETRES.remove(_w)
    if not FENETRES:
        Gtk.main_quit()


class Fenetre(Gtk.Window):
    def __init__(self, args):
        # --modele c139 (FreeCalypso Magnetite): photo data/c139.jpg, zones data/c139-zones.json,
        # colour screen "C139LCD"; auto = from the magic of the shared image, c123 by default.
        self.modele = args.modele
        if self.modele == "auto":
            self.modele = "c139" if magic_lcd(args.lcd) == "C139LCD" else "c123"
        if not args.zones:
            args.zones = os.path.join(DATA, "%s-zones.json" % self.modele)
        self.modele_force = args.modele != "auto"
        self.args = args
        super().__init__(title=self._titre())
        self.zones = self._charger_zones(args.zones)
        self.pixbuf = self._charger_image(args.image)
        self.lcd = Lcd(args.lcd)
        self.clavier = Clavier(args.keys)
        self.enfoncees = set()
        # a Magnetite C139 is never a TRX of the BTS (trx traces of an old run do not count)
        self.trx = instance_est_trx(args.instance) if args.instance and self.modele != "c139" else False
        self.power_depuis = None      # instant of the press on power (long press in progress)
        self.power_envoyee = False    # press power already sent to the firmware
        self.calibrage = [] if args.calibrate else None   # [(zone, x1, y1)] in progress
        self.cal_index = 0
        self.cal_premier = None

        self.lw, self.lh = self._taille()

        boite = Gtk.Box(orientation=Gtk.Orientation.VERTICAL)
        self.add(boite)
        self.zone = Gtk.DrawingArea()
        self.zone.set_size_request(self.lw, self.lh)
        self.zone.add_events(Gdk.EventMask.BUTTON_PRESS_MASK | Gdk.EventMask.BUTTON_RELEASE_MASK
                             | Gdk.EventMask.POINTER_MOTION_MASK)
        self.zone.connect("draw", self.dessiner)
        self.zone.connect("button-press-event", self.clic)
        self.zone.connect("button-release-event", self.relacher)
        boite.pack_start(self.zone, True, True, 0)
```

```
    self.statut = Gtk.Label(xalign=0)
    self.statut.set_margin_start(6)
    boite.pack_start(self.statut, False, False, 2)
    self.connect("key-press-event", self.touche_pc)
    self.connect("key-release-event", self.touche_pc)
    self.connect("destroy", fenetre_fermee)
    GLib.timeout_add(40, self.tic)
    self._statut()

# --- loading -------------------------------------------------------------
def _titre(self):
    role = "TRX de la BTS" if (self.args.instance and instance_est_trx(self.args.instance)) else "mobile"
    if self.modele == "c139":
        role = "mobile Magnetite"
    titre = "Motorola %s - Calypso emule" % self.modele.upper()
    if self.args.instance:
        titre += " - instance %d (%s)" % (self.args.instance, role)
    return titre

def _taille(self):
    if self.pixbuf:
        iw, ih = self.pixbuf.get_width(), self.pixbuf.get_height()
    else:
        iw, ih = 300, 630
    scale = self.args.scale or min(2.5, 900.0 / ih)
    return int(iw * scale), int(ih * scale)

def _changer_modele(self, modele):
    """The shared screen changes magic (e.g. --bts --ms-magnetite: instance 2 becomes a
    C139 Magnetite after the window opened): photo, zones, size and title follow."""
    self.modele = modele
    self.args.zones = os.path.join(DATA, "%s-zones.json" % modele)
    self.zones = self._charger_zones(self.args.zones)
    self.pixbuf = self._charger_image(None)
    self.lw, self.lh = self._taille()
    self.zone.set_size_request(self.lw, self.lh)
    self.resize(self.lw, self.lh + 24)
    self.set_title(self._titre())
    self._statut("modele %s (ecran %dx%d)" % (modele, self.lcd.w, self.lcd.h))

def _charger_zones(self, path):
    with open(path) as f:
        z = json.load(f)
    return {k: v for k, v in z.items() if not k.startswith("_")}

def _charger_image(self, path):
   candidats = [path] if path else [os.path.join(DATA, self.modele + ext) for ext in (".jpg", ".png", ".jpeg")]
    for p in candidats:
        if p and os.path.isfile(p):
            try:
                return GdkPixbuf.Pixbuf.new_from_file(p)
            except GLib.Error as e:
                print("photo illisible %s : %s" % (p, e), file=sys.stderr)
    print("pas de photo (%s) : schema dessine a la place ; posez la photo du %s dans data/%s.jpg"
          % (", ".join(c for c in candidats if c), self.modele.upper(), self.modele), file=sys.stderr)
    return None

# --- geometry ------------------------------------------------------------
def rect(self, nom):
    x, y, w, h = self.zones[nom]
    return x * self.lw, y * self.lh, w * self.lw, h * self.lh

def zone_sous(self, px, py):
    for nom in self.zones:
        if nom == "lcd":
            continue
```

```
        x, y, w, h = self.rect(nom)
        if x <= px <= x + w and y <= py <= y + h:
            return nom
    return None

# --- drawing ------------------------------------------------------------
def dessiner(self, _w, cr):
    if self.pixbuf:
        cr.save()
        cr.scale(self.lw / self.pixbuf.get_width(), self.lh / self.pixbuf.get_height())
        Gdk.cairo_set_source_pixbuf(cr, self.pixbuf, 0, 0)
        cr.paint()
        cr.restore()
    else:
        self._schema(cr)
    # the screen
    x, y, w, h = self.rect("lcd")
    cr.save()
    cr.translate(x, y)
    cr.scale(w / self.lcd.w, h / self.lcd.h)
    cr.set_source_surface(self.lcd.surface, 0, 0)
    cr.get_source().set_filter(cairo.FILTER_NEAREST)
    cr.paint()
    cr.restore()
    if not self.lcd.vivant:
        cr.set_source_rgba(0, 0, 0, 0.55)
        cr.rectangle(x, y, w, h)
        cr.fill()
        cr.set_source_rgb(1, 1, 1)
        cr.set_font_size(max(10, h / 6))
        cr.move_to(x + 6, y + h / 2)
        cr.show_text("QEMU absent")
    # pressed keys
    for nom in self.enfoncees:
        if nom in self.zones:
            x, y, w, h = self.rect(nom)
            cr.set_source_rgba(1.0, 0.85, 0.1, 0.45)
            cr.rectangle(x, y, w, h)
            cr.fill()
    # calibration: zones already set + zone in progress
    if self.calibrage is not None:
        cr.set_line_width(1.5)
        for nom in self.zones:
            x, y, w, h = self.rect(nom)
            cr.set_source_rgba(0.1, 0.6, 1.0, 0.8)
            cr.rectangle(x, y, w, h)
            cr.stroke()
            cr.move_to(x + 2, y + h - 2)
            cr.set_font_size(10)
            cr.show_text(nom)
        if self.cal_premier:
            cr.set_source_rgba(1, 0, 0, 0.9)
            cr.arc(self.cal_premier[0], self.cal_premier[1], 4, 0, 6.3)
            cr.fill()
    return False

def _schema(self, cr):
    cr.set_source_rgb(0.82, 0.82, 0.84)
    cr.rectangle(0, 0, self.lw, self.lh)
    cr.fill()
    cr.set_source_rgb(0.25, 0.25, 0.28)
    cr.rectangle(self.lw * 0.04, self.lh * 0.02, self.lw * 0.92, self.lh * 0.96)
    cr.fill()
    for nom in self.zones:
        if nom == "lcd":
            continue
```

```
        x, y, w, h = self.rect(nom)
        cr.set_source_rgb(0.85, 0.85, 0.87)
        cr.rectangle(x, y, w, h)
        cr.fill()
        cr.set_source_rgb(0.1, 0.1, 0.1)
        cr.set_font_size(max(9, h * 0.6))
        cr.move_to(x + w * 0.3, y + h * 0.75)
        cr.show_text(LIBELLES.get(nom, nom))

# --- input -------------------------------------------------------------
def clic(self, _w, ev):
    if self.calibrage is not None:
        return self._clic_calibrage(ev)
    nom = self.zone_sous(ev.x, ev.y)
    if nom:
        self.appuyer(nom)
    return True

def relacher(self, _w, ev):
    if self.calibrage is not None:
        return True
    for nom in list(self.enfoncees):
        self.lacher(nom)
    self.zone.queue_draw()
    return True

# --- keys: press/release to QEMU, power on long press -----------------
def appuyer(self, nom):
    if nom in self.enfoncees:
        return
    self.enfoncees.add(nom)
    if nom == "power" and not self.args.power_libre:
        self.power_depuis = time.monotonic()
        self.power_envoyee = False
        if self.trx:
            self._statut("power ignoree : l'instance %s est le TRX de la BTS (un power l'eteint et la BTS perd
             ↪  son horloge)"
                         % self.args.instance)
        else:
            self._statut("power : maintenir %.1f s pour eteindre le firmware" % POWER_APPUI_LONG_S)
    else:
        self.clavier.envoyer("press", nom)
        self._statut("touche %s" % nom)
    self.zone.queue_draw()

def lacher(self, nom):
    if nom not in self.enfoncees:
        return
    self.enfoncees.discard(nom)
    if nom == "power" and not self.args.power_libre:
        if self.power_envoyee:
            self.clavier.envoyer("release", nom)
        elif not self.trx:
            self._statut("power relachee avant %.1f s : rien envoye" % POWER_APPUI_LONG_S)
        self.power_depuis = None
        self.power_envoyee = False
    else:
        self.clavier.envoyer("release", nom)
    self.zone.queue_draw()

def touche_pc(self, _w, ev):
    nom_gdk = Gdk.keyval_name(ev.keyval)
    if self.calibrage is not None:
        if ev.type == Gdk.EventType.KEY_PRESS and nom_gdk == "s":
            self._sauver_zones()
        elif ev.type == Gdk.EventType.KEY_PRESS and nom_gdk == "q":
```

```
                Gtk.main_quit()
            return True
        nom = CLAVIER.get(nom_gdk)
        if not nom:
            return False
        if ev.type == Gdk.EventType.KEY_PRESS:
            self.appuyer(nom)
        else:
            self.lacher(nom)
        return True

    # --- calibration ----------------------------------------------------------
    def _clic_calibrage(self, ev):
        if self.cal_index >= len(ORDRE_ZONES):
            self._statut("calibrage termine : s = sauver, q = quitter")
            return True
        nom = ORDRE_ZONES[self.cal_index]
        if self.cal_premier is None:
            self.cal_premier = (ev.x, ev.y)
            self._statut("%s : coin bas-droit" % nom)
        else:
            x1, y1 = self.cal_premier
            x2, y2 = ev.x, ev.y
            self.zones[nom] = [round(min(x1, x2) / self.lw, 4), round(min(y1, y2) / self.lh, 4),
                               round(abs(x2 - x1) / self.lw, 4), round(abs(y2 - y1) / self.lh, 4)]
            self.cal_premier = None
            self.cal_index += 1
            if self.cal_index < len(ORDRE_ZONES):
                self._statut("%s : coin haut-gauche" % ORDRE_ZONES[self.cal_index])
            else:
                self._statut("calibrage termine : s = sauver, q = quitter")
        self.zone.queue_draw()
        return True

    def _sauver_zones(self):
        z = {"_doc": "Zones cliquables de la photo du C123, [x, y, largeur, hauteur] normalises ; "
                     "recalibrer : osmo-c123-gtk.py --modele %s --calibrate" % self.modele}
        z.update({k: self.zones[k] for k in ORDRE_ZONES if k in self.zones})
        with open(self.args.zones, "w") as f:
            json.dump(z, f, indent=2)
        self._statut("zones sauvees dans %s" % self.args.zones)

    # --- loop -----------------------------------------------------------------
    def tic(self):
        if self.lcd.vivant and not self.modele_force and self.lcd.modele != self.modele:
            self._changer_modele(self.lcd.modele)
        if self.lcd.lire():
            self.zone.queue_draw()
            self._statut()
        if (self.power_depuis is not None and not self.power_envoyee and not self.trx
                and time.monotonic() - self.power_depuis >= POWER_APPUI_LONG_S):
            self.power_envoyee = True
            self.clavier.envoyer("press", "power")
            self._statut("power envoyee a l'instance %s (appui long)" % self.args.instance)
        return True

    def _statut(self, msg=None):
        if self.calibrage is not None and msg is None:
            msg = "calibrage : %s : coin haut-gauche" % ORDRE_ZONES[0]
        etat = "LCD %s (v%d)" % ("vivant" if self.lcd.vivant else "absent", max(self.lcd.seq, 0))
        clav = "touches -> %s%s" % (self.args.keys, " (%s)" % self.clavier.erreur if self.clavier.erreur else "")
        inst = "instance %s%s · " % (self.args.instance, " (TRX de la BTS)" if self.trx else "") if
↪   self.args.instance else ""
        self.statut.set_text("%s%s · %s%s" % (inst, etat, clav, " · " + msg if msg else ""))
```

```
def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--modele", choices=["auto", "c123", "c139"], default="auto",
                    help="telephone : c123 (ST7558 mono, osmocom-bb) ou c139 (ecran couleur, FreeCalypso Magnetite)
                     ↪  ; "
                         "auto = d'apres l'ecran partage (defaut)")
    ap.add_argument("--image", help="photo du telephone (defaut data/<modele>.jpg|png)")
    ap.add_argument("--zones", default=None, help="zones cliquables (defaut data/<modele>-zones.json)")
    ap.add_argument("--instance", type=int, default=0,
                    help="instance c54x_exe (1 = /dev/shm/calypso_lcd, N = /dev/shm/calypsoN/... et
                     ↪  /tmp/calypso_keysN.sock) ; "
                         "defaut : la premiere instance vivante qui n'est pas un TRX de la BTS")
    ap.add_argument("--lcd", default=os.environ.get("CALYPSO_LCD_SHM"), help="ecran partage (prime sur
↪   --instance)")
    ap.add_argument("--keys", default=os.environ.get("CALYPSO_KEYS_SOCK"), help="socket des touches (prime sur
↪   --instance)")
    ap.add_argument("--power-libre", action="store_true",
                    help="touche power envoyee des l'appui, meme a un TRX (defaut : appui long de %.1f s, jamais a
                     ↪  un TRX)"
                         % POWER_APPUI_LONG_S)
    ap.add_argument("--toutes", nargs="?", const="1,2,3,4", metavar="N,M,...",
                    help="une fenetre par instance (defaut 1,2,3,4 : TRX TS0, mobile, TRX TS1, rssi ; plus toute
                     ↪  autre instance vivante, et celles qui "
                         "apparaissent ensuite : guet toutes les 2 s), cote a cote")
    ap.add_argument("--scale", type=float, default=0, help="facteur d'agrandissement de la photo (defaut : hauteur
↪   900 px max)")
    ap.add_argument("--calibrate", action="store_true", help="redefinir les zones a la souris")
    ap.add_argument("--autotest", action="store_true", help="sans fenetre : lit l'image LCD une fois et sort")
    args = ap.parse_args()
    if args.toutes:
        if args.calibrate or args.autotest:
            ap.error("--toutes ne va pas avec --calibrate / --autotest")
        voulues = [int(x) for x in args.toutes.replace(" ", ",").split(",") if x]
        for n in range(1, 9):
            if n not in voulues and instance_vivante(n):
                voulues.append(n)
        ouvertes = {}
        pos = [0]

        def ouvrir(n):
            a = argparse.Namespace(**vars(args))
            a.instance, a.lcd, a.keys = n, None, None
            a.lcd, a.keys = chemins_instance(n)
            if not args.scale:
                a.scale = 1.1                    # four phones side by side on a 1080p screen
            f = Fenetre(a)
            FENETRES.append(f)
            ouvertes[n] = f
            f.move(pos[0], 0)
            pos[0] += f.lw + 24
            f.show_all()

        for n in voulues:
            ouvrir(n)

        # an instance started AFTER the window (banc-rssi, INSTANCE=4...) also gets a window:
        # instances 1..8 are polled every 2 s and those that appear are opened.
        def guetter():
            for n in range(1, 9):
                if n not in ouvertes and instance_vivante(n):
                    print("instance %d apparue : fenetre ouverte" % n, file=sys.stderr)
                    ouvrir(n)
            return True
        GLib.timeout_add(2000, guetter)
        Gtk.main()
        return 0
```

```
    if not args.instance and not (args.lcd and args.keys):
        args.instance = instance_par_defaut()
    if args.instance:
        lcd, keys = chemins_instance(args.instance)
        args.lcd = args.lcd or lcd
        args.keys = args.keys or keys
        if instance_est_trx(args.instance):
            print("instance %d : firmware trx (TRX de la BTS) ; touche power refusee sans --power-libre" %
             ↪  args.instance,
                  file=sys.stderr)
    if args.autotest:
        lcd = Lcd(args.lcd)
        ok = lcd.lire()
        print("lcd %s : vivant=%s seq=%d flags=%d" % (args.lcd, lcd.vivant, lcd.seq, lcd.flags))
        return 0 if ok else 1
    f = Fenetre(args)
    FENETRES.append(f)
    f.show_all()
    Gtk.main()
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

2.245 /opt/GSM/osmo-operator/tools/osmo-dino.py

10413 octets, 210 lignes → 210 lignes

```
#!/usr/bin/env python3
# osmo-dino.py - the ANIMATED DINO ("Coureur d'ombres" game), placed under Conky.
#
# Replaces osmo-moon.py (the rider/moon gif): same window of DESKTOP type
# (Gdk.WindowTypeHint.DESKTOP - under all windows, above the wallpaper,
# undecorated, sticky, transparent), same top-right column, same width 400,
# stuck BELOW Conky. But the content is an HTML/canvas page
# (configs/conky/dino.html) rendered by WebKit2: the game runs in "attract"
# mode on its own (the dino runs, Sun/Moon, computed shadows) - no interaction
# required, like an animation. Header/footer are hidden by an injected
# stylesheet to keep only the frame of the game.
#
# Settings through environment variables:
#   OSMO_DINO_HTML   path of the html  (default: configs/conky/dino.html)
#   OSMO_DINO_W      width in px       (default: 400, the width of Conky)
#   OSMO_DINO_H      height in px      (default: 260 ~ 8/5 ratio of the frame)
#   OSMO_DINO_GAP_X  right margin      (default: 24, the gap_x of Conky)
#   OSMO_DINO_GAP_Y  y of the game top (default: 680 = under Conky, which goes
#                    down to about 580)
#
# Started by /usr/local/bin/osmo-desktop-panel (like osmo-panel.py + conky).
import os
import subprocess
import sys

# THE DESKTOP WINDOW REQUIRES X11, NOT WAYLAND.
# Under Wayland, GTK3 ignores window.move() AND Gdk.WindowTypeHint.DESKTOP: the
# compositor places the window where it wants (in the middle, in front). The
# protocol gives the client no way to place itself at absolute coordinates
# (it would need gtk-layer-shell, absent on mutter). XWayland is used instead,
# where X11 positioning is honoured, exactly like the neighbouring Conky
# (own_window type desktop). Only forced when a DISPLAY exists: without an X
# server, the default behaviour is better than a failure at startup.
X11_FORCE = os.environ.get("XDG_SESSION_TYPE") == "wayland" and bool(os.environ.get("DISPLAY"))
if X11_FORCE:
```

```
    os.environ["GDK_BACKEND"] = "x11"

import gi

gi.require_version("Gtk", "3.0")
gi.require_version("Gdk", "3.0")
gi.require_version("WebKit2", "4.1")
from gi.repository import Gdk, GLib, Gtk, WebKit2  # noqa: E402

# THE DESKTOP TYPE DOES NOT WORK ON XWAYLAND. There, a window of type
# _NET_WM_WINDOW_TYPE_DESKTOP is put by GNOME Shell in the desktop layer,
# UNDER the full-screen icons window (DING extension) - which swallows all
# clicks: the panel would show but its buttons would not respond. A NORMAL
# window is kept instead, simply held below the others (keep_below + stick +
# out of the taskbar): same rendering, and the clicks arrive. On native X11
# (the live key) the DESKTOP type works and is kept.
# APPLICATION MODE (--app, or OSMO_DINO_APP=1).
# The dino is not placed on the desktop: it is in the APPLICATION MENU
# (data/desktop/osmo-dino.desktop), and opens like any program. What
# changes, and nothing else: an ORDINARY window - decorated, resizable, in
# the taskbar, centred, neither sticky nor held below - and the page rendered
# as is, without the stylesheet that trims the frame to fit a 400x260
# corner. The full screen button of the page does not have to delegate to
# osmo-dino-play: this window CAN grow.
APP_MODE = os.environ.get("OSMO_DINO_APP") == "1" or "--app" in sys.argv[1:]

TYPE_HINT = Gdk.WindowTypeHint.NORMAL if (X11_FORCE or APP_MODE) else Gdk.WindowTypeHint.DESKTOP
# "BELOW" DOES NOT WORK ON XWAYLAND EITHER: with _NET_WM_STATE_BELOW, GNOME
# Shell puts the XWayland window under its own desktop layer (native Wayland),
# which is full screen and takes all clicks - the window stays visible but
# completely inert. Under XWayland "always below" is therefore given up:
# ordinary window, sticky and out of the taskbar. It comes to the front when
# clicked, the price to pay for being clickable at all. Set
# OSMO_DESKTOP_BELOW=1 to get the window under everything (clicks lost);
# on native X11 nothing changes.
KEEP_BELOW = (not APP_MODE) and ((not X11_FORCE) or os.environ.get("OSMO_DESKTOP_BELOW") == "1")

REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
HTML = os.environ.get("OSMO_DINO_HTML", os.path.join(REPO, "configs/conky/dino.html"))
WIN_W = int(os.environ.get("OSMO_DINO_W", "880" if APP_MODE else "400"))  # widget: the width of Conky
WIN_H = int(os.environ.get("OSMO_DINO_H", "560" if APP_MODE else "260"))
GAP_X = int(os.environ.get("OSMO_DINO_GAP_X", "24"))  # same gap_x as Conky
GAP_Y = int(os.environ.get("OSMO_DINO_GAP_Y", "720"))  # under Conky, which goes down to about 700

# Only the frame of the game is kept: no title nor footer, frame flush to the edge.
INJECT_CSS = (
    "header,footer{display:none!important}"
    "body{padding:0!important;margin:0!important;min-height:auto!important;"
    "gap:0!important;background:var(--ink)!important}"
    ".frame{width:100%!important;max-width:none!important;border:0!important;"
    "border-radius:0!important;box-shadow:none!important;aspect-ratio:auto!important;"
    "height:100vh!important}"
)


class Dino(Gtk.Window):
    def __init__(self):
        super().__init__(title="osmo-dino")
        if not os.path.exists(HTML):
            print(f"[dino] html introuvable: {HTML}", file=sys.stderr)
            sys.exit(1)
        self.w, self.h = WIN_W, WIN_H
        self._plein = False

        if APP_MODE:
            # A window like the others: the window manager places it, resizes it
```

```
    # and puts it in the taskbar. No move(), hence no _pin either - nothing
    # tries to put it back in a corner it no longer has.
    self.x = self.y = 0
    print(f"[dino] application {self.w}x{self.h} ({HTML})", flush=True)
    self.set_type_hint(TYPE_HINT)
    self.set_default_size(self.w, self.h)
    self.set_position(Gtk.WindowPosition.CENTER)
    self.set_icon_name("osmo-dino")
else:
    # top-right column of the primary monitor, like Conky (top_right)
    disp = Gdk.Display.get_default()
    mon = disp.get_primary_monitor() or disp.get_monitor(0)
    g = mon.get_geometry()
    self.x = g.x + g.width - GAP_X - self.w
    self.y = g.y + GAP_Y
    print(f"[dino] {self.w}x{self.h} @ {self.x},{self.y} ({HTML})", flush=True)

    # window of DESKTOP type (cf. osmo-panel.py / osmo-moon.py)
    self.set_type_hint(TYPE_HINT)
    self.set_decorated(False)
    self.set_resizable(False)
    self.set_keep_below(KEEP_BELOW)
    self.set_skip_taskbar_hint(True)
    self.set_skip_pager_hint(True)
    self.stick()
    self.set_default_size(self.w, self.h)
    self.set_size_request(self.w, self.h)
    self.move(self.x, self.y)

screen = self.get_screen()
visual = screen.get_rgba_visual()
if visual and screen.is_composited():
    self.set_visual(visual)
self.set_app_paintable(True)
self.connect("destroy", Gtk.main_quit)

# WebView: injected stylesheet (hides header/footer) + transparent
# background (the body of the page stays dark, on purpose).
ucm = WebKit2.UserContentManager()
# The stylesheet is only for the widget: it hides header and footer to
# keep only the frame of the game in 400x260. As an application, the
# page is shown whole, as in a browser.
if not APP_MODE:
    ucm.add_style_sheet(WebKit2.UserStyleSheet(
        INJECT_CSS, WebKit2.UserContentInjectedFrames.ALL_FRAMES,
        WebKit2.UserStyleLevel.USER, None, None))
# THE FULL SCREEN BUTTON OF THE WIDGET. A page cannot enlarge a GTK
# window, and this 400x260 widget has nothing to enlarge anyway: it is a
# corner of the desktop. The page therefore knows it is a "widget"
# (OSMO_HOST) and, on click, sends a WebKit message - the REAL game is
# then opened, full screen, in its own window.
# 'widget' makes the page delegate full screen (dino.html l.519). As an
# application, the host CAN grow: 'app' is announced, and the {full:...}
# message is handled below by _on_message.
ucm.add_script(WebKit2.UserScript(
    "window.OSMO_HOST='app';" if APP_MODE else "window.OSMO_HOST='widget';",
    WebKit2.UserContentInjectedFrames.ALL_FRAMES,
    WebKit2.UserScriptInjectionTime.START, None, None))
ucm.connect("script-message-received::osmo", self._on_message)
ucm.register_script_message_handler("osmo")
self.view = WebKit2.WebView.new_with_user_content_manager(ucm)
self.view.set_background_color(Gdk.RGBA(0, 0, 0, 0))
self.add(self.view)
self.view.load_uri(GLib.filename_to_uri(HTML, None))

self.show_all()
```

```
        self._pos = (self.x, self.y)
        if not APP_MODE:
            self._pin()
            self.connect("map-event", self._pin)
            GLib.timeout_add(500, self._pin)

    # REPOSITION AFTERWARDS. Even under X11, the window manager may move the
    # window when it maps it (mutter does so for undecorated windows it does
    # not recognise as desktop). A single move() before show_all() does not
    # hold: the window is repositioned on map-event, then one last time half
    # a second later.
    def _pin(self, *_a):
        self.move(*self._pos)
        return False

    def _on_message(self, _ucm, result):
        """{full:...}: as an application, THIS window; as a widget, the one of the game."""
        if APP_MODE:
            # The page sends {full:true} or {full:false} - followed.
            try:
                val = result.get_js_value()
                plein = bool(val.object_get_property("full").to_boolean())
            except Exception:
                plein = not self._plein
            self._plein = plein
            self.fullscreen() if plein else self.unfullscreen()
            return
        for cand in ("/usr/local/bin/osmo-dino-play",):
            if os.path.exists(cand):
                subprocess.Popen([cand, "--fullscreen"], stdin=subprocess.DEVNULL,
                                 stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
                                 start_new_session=True)
                return
        print("[dino] osmo-dino-play absent (tools/osmo-extras-install.sh le pose)", flush=True)


def main():
    Dino()
    Gtk.main()


if __name__ == "__main__":
    main()
```

2.246 /opt/GSM/osmo-operator/tools/osmo-drivers.sh

5778 octets, 112 lignes → 112 lignes

```
#!/bin/bash
# ============================================================================
#  osmo-drivers - the NVIDIA graphics driver: status, installation, update
# ============================================================================
#  The counterpart, on the running system (live or installed), of the
#  "Graphics drivers" page of the installer: shows the card, the loaded module
#  and what is installed, and lets you install or update.
#
#  THE BENCH DRIVER IS FIXED: nvidia-driver-610, a named package:
#      sudo apt install nvidia-driver-610
#  ubuntu-drivers does not decide anything; it is only used, if present, to
#  inform (the list and the "recommended" shown by --status).
#  OSMO_NVIDIA_DRIVER selects another package.
#
#      sudo ./tools/osmo-drivers.sh            whiptail menu
#      sudo ./tools/osmo-drivers.sh --status   status only, as text
```

```
#      sudo ./tools/osmo-drivers.sh --install [driver]   (default: nvidia-driver-610)
#      sudo ./tools/osmo-drivers.sh --update
# ============================================================================
set -u
RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'
export DEBIAN_FRONTEND=noninteractive

[ "$(id -u)" -eq 0 ] || exec pkexec --disable-internal-agent "$0" "$@" 2>/dev/null || exec sudo "$0" "$@"

need() { command -v "$1" >/dev/null 2>&1; }
# NVDRV: the driver this script installs. A package name, not a lookup.
NVDRV="${OSMO_NVIDIA_DRIVER:-nvidia-driver-610}"
# ubuntu-drivers is not a dependency: it is not installed for its own sake.
# If it is already there, --status uses it to show Ubuntu's "recommended" -
# for information, next to the driver the bench actually installs.

gpu()       { lspci -d 10de: 2>/dev/null | head -1 | cut -d: -f3- | sed 's/^ //; s/ (rev.*//'; }
reco()      { ubuntu-drivers devices 2>/dev/null | awk '/^driver *:/ && /recommended/{print $3; exit}'; }
avail()     { { echo "$NVDRV"; ubuntu-drivers list 2>/dev/null | awk '/^nvidia-driver-/{print $1}' | sed
 ↪  's/,.*//'; } | sort -u; }
installed() { dpkg -l 'nvidia-driver-*' 2>/dev/null | awk '/^ii/{print $2}'; }
loaded()    { lsmod 2>/dev/null | awk '$1=="nvidia"||$1=="nouveau"{print $1}' | paste -sd, -; }

status_text() {
    local g r
    g="$(gpu)"; r="$(reco)"
    echo "Carte NVIDIA : ${g:-aucune (lspci vendor 10de)}"
    echo "Module charge : $(loaded)"
    echo "Pilote du banc: $NVDRV  ($(apt-cache policy "$NVDRV" 2>/dev/null | awk '/Candidate:|Candidat/{print $2;
     ↪  exit}'))"
    echo "Recommande Ubuntu : ${r:-inconnu (ubuntu-drivers absent ou pas de reseau)}"
    echo "Installe      : $(installed | paste -sd' ' -)"
    [ -z "$(installed)" ] && echo "                (aucun pilote proprietaire)"
    echo "Disponibles   :"
    local d st
    for d in $(avail); do
        st=""
        [ "$d" = "$NVDRV" ] && st="$st pilote du banc"
        [ "$d" = "$r" ] && st="$st recommande Ubuntu"
        dpkg -s "$d" >/dev/null 2>&1 && st="$st installe"
        echo "   $d${st:+  [$st ]}"
    done
    if need nvidia-smi; then echo; nvidia-smi --query-gpu=name,driver_version,memory.used,memory.total
     ↪  --format=csv,noheader 2>/dev/null | sed 's/^/nvidia-smi : /'; fi
}

do_install() {
    local what="${1:-$NVDRV}"
    [ "$what" = recommended ] && what="$NVDRV"   # ancien nom d option, meme resultat
    if ! lspci -n 2>/dev/null | grep -qi 10de:; then
        echo -e "${RED}Aucune carte NVIDIA sur cette machine : rien a installer.${NC}"; return 1
    fi
    apt-get update -qq || { echo -e "${RED}Pas de reseau : les pilotes viennent des depots Ubuntu.${NC}"; return
     ↪  1; }
    echo -e "${CYAN}apt install $what...${NC}"
    if ! apt-get install -y "$what"; then
        echo -e "${RED}$what non installe.${NC} Il vit dans multiverse :"
        echo -e "   ${CYAN}sudo add-apt-repository multiverse && sudo apt install $what${NC}"
        return 1
    fi
    update-initramfs -u >/dev/null 2>&1 || true
    echo -e "${GREEN}Termine. Redemarrez pour charger le module nvidia.${NC}"
}

do_update() {
```

```
    local pk; pk="$(dpkg -l 'nvidia-*' 'libnvidia-*' 'linux-modules-nvidia-*' 2>/dev/null | awk '/^ii/{print $2}'
    ↪  | paste -sd' ' -)"
    [ -n "$pk" ] || { echo -e "${YELLOW}Aucun paquet NVIDIA installe : rien a mettre a jour (--install d
    ↪  abord).${NC}"; return 1; }
    apt-get update -qq || { echo -e "${RED}Pas de reseau.${NC}"; return 1; }
    echo -e "${CYAN}Mise a jour de : $pk${NC}"
    # shellcheck disable=SC2086
    apt-get install -y --only-upgrade $pk
    update-initramfs -u >/dev/null 2>&1 || true
    echo -e "${GREEN}Termine.${NC}"
}

case "${1:-}" in
    --status)  status_text; exit 0 ;;
    --install) do_install "${2:-$NVDRV}"; exit $? ;;
    --update)  do_update; exit $? ;;
esac

need whiptail || { status_text; echo; echo "whiptail absent : --install [pilote|recommended] / --update"; exit 0; }
while :; do
    items=(status "Etat : carte, module charge, pilote du banc, installes")
    for d in $(avail); do
        st="installer"; dpkg -s "$d" >/dev/null 2>&1 && st="deja installe - reinstaller"
        [ "$d" = "$NVDRV" ] && st="$st (pilote du banc)"
        items+=("$d" "$st")
    done
    items+=(update "Mettre a jour les paquets NVIDIA installes")
    items+=(quit "Quitter")
    choice=$(whiptail --title "Pilotes graphiques - $(gpu | cut -c1-50)" \
        --menu "Pilote du banc : $NVDRV. Machine : $(hostname). Module charge : $(loaded)" 22 78 12 "${items[@]}"
        ↪  3>&1 1>&2 2>&3) || exit 0
    case "$choice" in
        status)      status_text | whiptail --title "Etat des pilotes" --textbox /dev/stdin 24 78 ;;
        update)      do_update; read -rp "Entree pour continuer..." _ ;;
        quit|"")     exit 0 ;;
        *)           do_install "$choice"; read -rp "Entree pour continuer..." _ ;;
    esac
done
```

2.247 /opt/GSM/osmo-operator/tools/osmo-epc.sh

10814 octets, 216 lignes → 216 lignes

```
#!/bin/bash
# ============================================================================
# osmo-epc.sh - THE BENCH 4G CORE: Open5GS (EPC).
#
# Open5GS provides a complete EPC, and above all the SGs interface to OsmoMSC
# that allows CSFB (voice and SMS from LTE). Addressing plan: the UEs are in
# 10.45.0.0/16 on ogstun (NOT 172.17.0.0/24, which overlaps docker0).
#
# OPEN5GS LIVES IN /opt/LTE. The prefix is /opt/LTE/open5gs/install (the
# Dockerfile, the osmo-build-open5gs .deb and tools/osmo-lte-install.sh put it
# there); the old /root/open5gs/install path is still accepted, as a last
# resort, for a machine not yet migrated. The configs ARE PART OF THE
# REPOSITORY: configs/open5gs/*.yaml, installed by osmo-lte-install.sh in
# $PREFIX/etc/open5gs.
#
#   osmo-epc start        mongod, ogstun + NAT, then the daemons in order
#   osmo-epc stop         stops the daemons (ogstun stays)
#   osmo-epc restart
#   osmo-epc status       one state per daemon, ogstun, SGs
#   osmo-epc subscribers  loads the repository subscribers (configs/open5gs/
#                         subscribers.json) into MongoDB if they are missing
```

```
#   osmo-epc log [name]   the log of a daemon (mme by default), live
#
# Variables: OPEN5GS_PREFIX, SRS_WAN (NAT output interface; by default the
# one of the default route), OPEN5GS_UESUBNET, OPEN5GS_TUNADDR.
# ============================================================================
set -u

REPO="${OSMO_REPO:-/opt/GSM/osmo-operator}"
# The prefix: the variable, else /opt/LTE, else the old /root.
_prefix() {
    local p
    for p in "${OPEN5GS_PREFIX:-}" /opt/LTE/open5gs/install /root/open5gs/install; do
        [ -n "$p" ] && [ -x "$p/bin/open5gs-mmed" ] && { echo "$p"; return 0; }
    done
    echo "${OPEN5GS_PREFIX:-/opt/LTE/open5gs/install}"
}
PREFIX="$(_prefix)"
BIN="$PREFIX/bin"
ETC="$PREFIX/etc/open5gs"
LOG="$PREFIX/var/log/open5gs"
UESUBNET="${OPEN5GS_UESUBNET:-10.45.0.0/16}"
TUNADDR="${OPEN5GS_TUNADDR:-10.45.0.1/16}"
# The NAT output interface: SRS_WAN, else the one of the default route
# (not hard-coded: another machine does not have the same card).
WAN="${SRS_WAN:-$(ip -4 route show default 2>/dev/null | awk '/default/ {print $5; exit}')}"
# Start order: the NRF (the SMF SBIs look for it), the HSS before the MME
# (Diameter), the user planes before the control planes attached to them.
DAEMONS="open5gs-nrfd open5gs-hssd open5gs-sgwcd open5gs-sgwud open5gs-smfd open5gs-upfd open5gs-pcrfd
 ↪  open5gs-mmed"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'
_ok()   { echo -e "  ${GREEN}✓${NC} $*"; }
_warn() { echo -e "  ${YELLOW}!${NC} $*"; }
_err()  { echo -e "  ${RED}✗${NC} $*" >&2; }
_root() { [ "$(id -u)" -eq 0 ] && return 0; _err "il faut root (tun, NAT, mongod) : sudo $0 $*"; return 1; }

[ -x "$BIN/open5gs-mmed" ] || { _err "Open5GS introuvable ($BIN/open5gs-mmed) - « osmo-lte-install --build » ou
 ↪  dpkg -i osmo-build-open5gs"; [ "${1:-}" = status ] || exit 1; }

# MONGOD FIRST. The HSS and the PCRF read the subscribers in MongoDB and DIE
# at initialisation if it is not running ("Failed to connect to server
# [mongodb://localhost/open5gs]"): the MME then answers 3002 on S6a and
# rejects every attach (EMM cause 15). The service may be "disabled": it is
# started here, every time.
_mongod() {
    systemctl is-active --quiet mongod && return 0
    if systemctl start mongod 2>/dev/null; then
        _ok "mongod demarre"; sleep 2
    else
        _err "mongod ne demarre pas (HSS et PCRF ne tiendront pas) : apt install mongodb-org"
        return 1
    fi
}

# The repository subscribers, loaded if missing. The field slice[].session[].smf
# (127.0.0.200, WebUI default value) is REMOVED: it takes precedence over
# gtpc.client.smf of mme.yaml and the SGW-C would send the Create Session to
# 127.0.0.200, where nobody answers (GTP Timeout, RRC Release).
epc_subscribers() {
    _root subscribers || return 1
    _mongod || return 1
    local dump="$REPO/configs/open5gs/dump/open5gs" j="$REPO/configs/open5gs/subscribers.json" n
    command -v mongosh >/dev/null 2>&1 || { _warn "mongosh absent : abonnes non poses"; return 0; }
    n="$(mongosh --quiet open5gs --eval 'print(db.subscribers.countDocuments())' 2>/dev/null | tail -1)"
    # Empty database and a mongodump in the repository (configs/open5gs/dump:
    # subscribers, WebUI accounts, sessions): restore EVERYTHING as is (BSON:
```

```
    # the sqn Longs and the ObjectIds stay what they are).
    if [ "${n:-0}" = "0" ] && [ -f "$dump/subscribers.bson" ] && command -v mongorestore >/dev/null 2>&1; then
        mongorestore --quiet --db open5gs "$dump" >/dev/null 2>&1 \
            && _ok "base open5gs restauree depuis le depot (mongodump)" \
            || _warn "mongorestore a echoue - on essaie le JSON"
    fi
    [ -s "$j" ] || return 0
    mongosh --quiet open5gs --eval "
        const subs = JSON.parse(require('fs').readFileSync('$j', 'utf8'));
        let n = 0;
        for (const s of subs) {
            // JSON.stringify d un Long donne {high, low, unsigned} : on le refait Long.
            if (s.security && s.security.sqn && typeof s.security.sqn === 'object')
                s.security.sqn = Long.fromBits(s.security.sqn.low, s.security.sqn.high);
            delete s.mme_timestamp;
            if (db.subscribers.countDocuments({ imsi: s.imsi }) === 0) { db.subscribers.insertOne(s); n++; }
        }
        db.subscribers.updateMany({ 'slice.session.smf': { \$exists: true } }, { \$unset: {
↪   'slice.$[].session.$[].smf': '' } });
        print('abonnes en base : ' + db.subscribers.countDocuments() + ' (' + n + ' ajoute(s) depuis le depot)');
    " 2>&1 | sed 's/^/  /'
}

setup_tun() {
    if ! ip link show ogstun >/dev/null 2>&1; then
        ip tuntap add name ogstun mode tun && _ok "ogstun cree"
    fi
    ip addr replace "$TUNADDR" dev ogstun
    ip link set ogstun up
    sysctl -q -w net.ipv4.ip_forward=1
    if [ -n "$WAN" ]; then
        iptables -t nat -C POSTROUTING -s "$UESUBNET" -o "$WAN" -j MASQUERADE 2>/dev/null \
            || iptables -t nat -A POSTROUTING -s "$UESUBNET" -o "$WAN" -j MASQUERADE
        _ok "ogstun $TUNADDR, NAT $UESUBNET -> $WAN"
    else
        _warn "ogstun $TUNADDR, pas de route par defaut : pas de NAT (SRS_WAN=<iface>)"
    fi
    setup_forward
}

# THE FORWARD, WITHOUT WHICH THE NAT IS USELESS. Docker sets a DROP FORWARD
# policy on the host: a packet from the UE (10.45.0.2, via ogstun) to the
# Internet would be NATed then dropped, without refusal or trace (seen from
# the pmOS phone: PPP up, address assigned, gateway ping OK, nothing beyond).
# Same mechanism as start-docker.sh for the WAN: DOCKER-USER is consulted
# before docker's rules and docker does not rewrite it; everything entering
# or leaving through ogstun is allowed there. Replayed at every "osmo-epc
# start" (hence every "osmo-lte start"), and by osmo-pmos-qemu at every VM
# launch.
setup_forward() {
    local spec
    iptables -N DOCKER-USER 2>/dev/null || true
    for spec in "-i ogstun" "-o ogstun"; do
        # shellcheck disable=SC2086
        iptables -C DOCKER-USER $spec -j ACCEPT 2>/dev/null \
            || iptables -I DOCKER-USER 1 $spec -j ACCEPT 2>/dev/null \
            || { _warn "FORWARD ($spec) : iptables refuse - la data des UE n ira pas plus loin que l hote"; return
             ↪  0; }
    done
    # The DOCKER-USER rules only matter if FORWARD goes through them: docker
    # does that (-A FORWARD -j DOCKER-USER); without docker, skip the step.
    iptables -C FORWARD -j DOCKER-USER 2>/dev/null || iptables -I FORWARD 1 -j DOCKER-USER 2>/dev/null || true
    _ok "FORWARD ogstun <-> $WAN autorise (DOCKER-USER)"
}

epc_start() {
```

```
    _root start || return 1
    _mongod || true
    epc_subscribers >/dev/null 2>&1 || true
    mkdir -p "$LOG"
    setup_tun
    local d y
    for d in $DAEMONS; do
        if pgrep -x "$d" >/dev/null; then _ok "$d deja en cours"; continue; fi
        # -D: daemon. The config is the one of $ETC (the prefix), installed
        # from the repository by osmo-lte-install.sh. open5gs-nrfd reads
        # nrf.yaml, not nrfd.yaml: -c must point to the right file, otherwise
        # the fallback without -c (the path compiled into the binary) would
        # start it and a different OPEN5GS_PREFIX would never read its configs.
        # stdout ALSO goes to /dev/null: -D does not detach the descriptors,
        # the daemon would keep the "osmo-epc start | sed" pipe of osmo-lte.sh
        # open and sed would never see its end.
        y="${d#open5gs-}"; y="$ETC/${y%d}.yaml"
        if "$BIN/$d" -D -c "$y" >/dev/null 2>&1; then :; else
            "$BIN/$d" -D >/dev/null 2>&1
        fi || _err "ECHEC $d (voir $LOG/$(basename "$y" .yaml).log)"
        sleep 0.5
        # "started" IS NOT ENOUGH: -D returns before the daemon has read its
        # config. An MME without freeDiameter/mme.conf (see configs/open5gs/
        # freeDiameter/mme.conf) reports started and then dies at once - the S1
        # does not listen, srsUE receives ConnectionReject, and nobody says
        # why. So check that it is STILL there; otherwise replay it for 3 s in
        # the foreground to report the reason (its FATAL / ERROR lines).
        if pgrep -x "$d" >/dev/null; then _ok "$d demarre"
        else
            _err "$d est mort a l init :"
            timeout 3 "$BIN/$d" -c "$y" 2>&1 | grep -aE 'FATAL|ERROR' | grep -v backtrace | head -3 | sed 's/^/
            ↪  /'
        fi
    done
    sleep 1
    ss -np 2>/dev/null | grep -q 29118 && _ok "SGs : association vers OsmoMSC etablie" \
        || _warn "SGs : pas encore d association (osmo-msc lance ?)"
}

epc_stop() {
    _root stop || return 1
    local d n=0
    for d in $DAEMONS; do pgrep -x "$d" >/dev/null && { pkill -TERM -x "$d"; n=$((n + 1)); }; done
    sleep 1
    for d in $DAEMONS; do pgrep -x "$d" >/dev/null && pkill -KILL -x "$d"; done
    [ "$n" -gt 0 ] && _ok "$n demon(s) Open5GS arrete(s)" || _ok "rien ne tournait"
}

epc_status() {
    echo -e "\033[1mOpen5GS - le coeur 4G du banc\033[0m  ($PREFIX)"
    local d
    for d in $DAEMONS; do
        pgrep -x "$d" >/dev/null && _ok "$d" || _warn "$d : arrete"
    done
    systemctl is-active --quiet mongod && _ok "mongod actif" || _warn "mongod arrete"
    ip -br addr show ogstun 2>/dev/null | sed 's/^/  /' || _warn "ogstun absent"
    ss -San 2>/dev/null | grep -q "ESTAB.*:36412" && _ok "S1 : un eNB est associe" || _warn "S1 : aucun eNB"
}

case "${1:-status}" in
    start)       epc_start ;;
    stop)        epc_stop ;;
    restart)     epc_stop; sleep 1; epc_start ;;
    status)      epc_status ;;
    subscribers) epc_subscribers ;;
    log)         exec tail -F "$LOG/${2:-mme}.log" ;;
```

```
    *) sed -n '/^#   osmo-epc start/,/^#   osmo-epc log/p' "$0" | sed 's/^# \{0,2\}//'; exit 2 ;;
esac
```

2.248 /opt/GSM/osmo-operator/tools/osmo-extras-install.sh

21299 octets, 470 lignes → 470 lignes

```
#!/bin/bash
# ============================================================================
# osmo-extras-install.sh - GAMES + MEDIA, installed once for TWO paths.
#
# Same logic as the rest of the repo: what must exist both in the SUPPLEMENT
# (addition.sh, on an already installed machine) and in the NATIVE image (the
# ISO, iso_modules/80-chroot.sh) is written ONCE here, and both paths source
# this script and call osmo_extras_install. Nothing is fatal: a package
# missing from the apt mirror must not stop the rest.
#
# What it installs:
#   - [addition.sh only, not the ISO: OSMO_EXTRAS_JEUX=0 skips them, ~470 MB]
#     two FREE games, data included, available on Ubuntu noble:
#       Doom  : freedoom (free WADs) + chocolate-doom (the engine the
#               freedoom2 launcher knows how to find; dsda-doom is not
#               recognised);
#       Quake : openarena (ioquake3 engine, Quake III, ~410 MB of data).
#     Not installed: gzdoom and openra (absent from noble), quakespasm
#     (stops at startup without the proprietary pak0.pak), Unreal (no free
#     version);
#   - kodi (mediacenter);
#   - wmctrl: the panel (osmo-panel.py) needs it to DOCK the launched window
#     onto the FFT frame;
#   - the playable DINO (osmo-dino-play, fullscreen or docked) from the same
#     configs/conky/dino.html as the desktop animation;
#   - YOUTUBE in a real Firefox (Google login possible) with uBlock Origin
#     forced by enterprise POLICY (policies.json) - not a third-party client,
#     so the Google login is kept;
#   - the missing .desktop files (youtube, dino) and two GNOME application
#     FOLDERS: « Jeux » and « Media ».
#
# Idempotent: can be rerun without damage. Usable in a session (gsettings
# available) AND in the ISO chroot (no session: the gschema override is used
# instead, read by the live session at boot).
# ============================================================================

# These colours may already exist (addition.sh defines them): no complaint.
: "${GREEN:=}"; : "${YELLOW:=}"; : "${CYAN:=}"; : "${BOLD:=}"; : "${NC:=}"

# The repo, to find dino.html. addition.sh sets DIR; the ISO sets REPO or
# nothing (the repo is copied into the image at a fixed path).
OSMO_EXTRAS_REPO="${DIR:-${REPO:-/opt/GSM/osmo-operator}}"

_ex_say()  { echo -e "  ${CYAN}\xe2\x86\x92${NC} $*"; }
_ex_ok()   { echo -e "      ${GREEN}\xe2\x9c\x93${NC} $*"; }
_ex_warn() { echo -e "      ${YELLOW}!${NC} $*"; }

# -- THE PACKAGES ------------------------------------------------------------
# Non-fatal package by package: on a mirror where one is missing (universe
# not enabled, version), the others still go through.
_osmo_extras_apt() {
    export DEBIAN_FRONTEND=noninteractive
    apt-get update -y >/dev/null 2>&1 || true
    # wireshark asks through debconf whether to allow capture without root:
    # answer NO (the panel launches it as root anyway, see osmo-wireshark-root)
    # so the install goes through without a question.
    echo "wireshark-common wireshark-common/install-setuid boolean false" \
```

```
        | debconf-set-selections 2>/dev/null || true
    local p
    for p in wmctrl xdotool \
             qemu-system-x86 qemu-system-gui qemu-system-modules-opengl ovmf sshpass kpartx \
             kodi \
             wireshark \
             gir1.2-webkit2-4.1; do
        if dpkg -s "$p" >/dev/null 2>&1; then
            _ex_ok "$p deja present"
        elif apt-get install -y --no-install-recommends "$p" >/dev/null 2>&1; then
            _ex_ok "$p installe"
        else
            _ex_warn "$p indisponible (apt) - ignore"
        fi
    done
    # linphone: the package name depends on the version (linphone-desktop
    # recent, linphone older). Take the first one that installs.
    if dpkg -s linphone-desktop >/dev/null 2>&1 || dpkg -s linphone >/dev/null 2>&1; then
        _ex_ok "linphone deja present"
    elif apt-get install -y --no-install-recommends linphone-desktop >/dev/null 2>&1 \
      || apt-get install -y --no-install-recommends linphone >/dev/null 2>&1; then
        _ex_ok "linphone installe"
    else
        _ex_warn "linphone indisponible (apt) - ignore"
    fi
    # ofono: telephony stack (daemon + test tools)
    if dpkg -s ofono >/dev/null 2>&1; then
        _ex_ok "ofono deja present"
    elif apt-get install -y --no-install-recommends ofono >/dev/null 2>&1; then
        _ex_ok "ofono installe"
    else
        _ex_warn "ofono indisponible (apt) - ignore"
    fi
}

# -- THE GAMES: SUPPLEMENT ONLY ------------------------------------------------
# Kept out of the ISO: ~470 MB for two games. addition.sh --extras installs
# them (OSMO_EXTRAS_JEUX=1, the default); iso_modules/80-chroot.sh passes 0.
_osmo_extras_jeux() {
    export DEBIAN_FRONTEND=noninteractive
    local p
    for p in freedoom chocolate-doom openarena; do
        if dpkg -s "$p" >/dev/null 2>&1; then
            _ex_ok "$p deja present"
        elif apt-get install -y --no-install-recommends "$p" >/dev/null 2>&1; then
            _ex_ok "$p installe"
        else
            _ex_warn "$p indisponible (apt) - ignore"
        fi
    done
}

# -- YOUTUBE: uBLOCK ORIGIN FORCED BY FIREFOX POLICY ---------------------------
# NOT a third-party client (it would lose the Google login): the real Firefox,
# with uBlock Origin installed by default for ALL profiles through the
# enterprise policy. `install_url` points at the AMO-signed XPI;
# `installation_mode force_installed` imposes it without a click. The Google
# login stays Firefox's own.
#
# The Mozilla .deb Firefox reads /etc/firefox/policies/policies.json AND
# <install>/distribution/policies.json - both are written so as not to depend
# on the install path.
_osmo_extras_firefox_ublock() {
    local pol
    read -r -d '' pol <<'POLICIES'
{
```

```
  "policies": {
    "ExtensionSettings": {
      "uBlock0@raymondhill.net": {
        "installation_mode": "force_installed",
        "install_url": "https://addons.mozilla.org/firefox/downloads/latest/ublock-origin/latest.xpi"
      }
    },
    "DisableAppUpdate": false
  }
}
POLICIES
    local d
    for d in /etc/firefox/policies \
             /usr/lib/firefox/distribution \
             /opt/firefox/distribution \
             /usr/lib/firefox-esr/distribution; do
        # /etc/firefox: always; the <install>/distribution ones: only if the
        # parent directory (hence Firefox) exists.
        case "$d" in
            /etc/firefox/policies) mkdir -p "$d" ;;
            *) [ -d "$(dirname "$d")" ] || continue; mkdir -p "$d" ;;
        esac
        printf '%s\n' "$pol" > "$d/policies.json"
    done
    _ex_ok "uBlock Origin force par politique Firefox (login Google conserve)"
}

# ── THE /usr/local/bin LAUNCHERS ──────────────────────────────────────
# Each launcher accepts --fullscreen. Without it: normal window (the panel
# docks it onto the frame via wmctrl). With it: straight to fullscreen.
_osmo_extras_lanceurs() {
    # YOUTUBE: Firefox, dedicated profile, on youtube.com. Google login OK,
    # uBlock already forced by the policy above.
    cat > /usr/local/bin/osmo-youtube <<'YT'
#!/bin/bash
# osmo-youtube - Firefox (Google login possible) + uBlock Origin (policy),
# on YouTube. --fullscreen => fullscreen kiosk.
set -u
URL="https://www.youtube.com"
PROFDIR="${HOME:-/root}/.mozilla/osmo-youtube"
mkdir -p "$PROFDIR"
FF="$(command -v firefox || command -v firefox-esr || true)"
[ -n "$FF" ] || { command -v zenity >/dev/null 2>&1 && \
    zenity --error --text="Firefox introuvable" 2>/dev/null; exit 1; }
if [ "${1:-}" = "--fullscreen" ]; then
    exec "$FF" --profile "$PROFDIR" --no-remote --kiosk "$URL"
else
    exec "$FF" --profile "$PROFDIR" --no-remote --new-window "$URL"
fi
YT
    chmod 755 /usr/local/bin/osmo-youtube

    # Playable DINO: the same dino.html, but in an interactive WebKit window
    # (not the "attract" animation of osmo-dino.py). --fullscreen => fullscreen.
    cat > /usr/local/bin/osmo-dino-play <<'DINO'
#!/usr/bin/env python3
# osmo-dino-play - the dino game (configs/conky/dino.html), playable.
# Without argument: 900x560 window. With --fullscreen: fullscreen.
import os, sys, gi
gi.require_version("Gtk", "3.0"); gi.require_version("WebKit2", "4.1")
from gi.repository import Gtk, GLib, WebKit2
REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
HTML = os.environ.get("OSMO_DINO_HTML", os.path.join(REPO, "configs/conky/dino.html"))
full = "--fullscreen" in sys.argv
class W(Gtk.Window):
    def __init__(self):
```

```
        super().__init__(title="Cavalier d'ombres")
        self.set_default_size(900, 560)
        self.connect("destroy", Gtk.main_quit)
        # The page's fullscreen button talks to us through a WebKit message
        # ({full:true|false}): WE enlarge or restore the window - a page
        # cannot do it on its own.
        ucm = WebKit2.UserContentManager()
        ucm.connect("script-message-received::osmo", self.on_message)
        ucm.register_script_message_handler("osmo")
        self.view = WebKit2.WebView.new_with_user_content_manager(ucm)
        self.add(self.view)
        if os.path.exists(HTML):
            # #full: the page starts fullscreen (its button brings it back
            # into the window). Without --fullscreen it stays in the window.
            self.view.load_uri(GLib.filename_to_uri(HTML, None) + ("#full" if full else ""))
        else:
            self.view.load_html("<h2 style='font-family:sans-serif'>dino.html introuvable</h2>", None)
        self.show_all()
        if full:
            self.fullscreen()
    def on_message(self, _ucm, result):
        try:
            on = result.get_js_value().to_string().find("true") >= 0 \
                 if not hasattr(result.get_js_value(), "object_get_property") \
                 else result.get_js_value().object_get_property("full").to_boolean()
        except Exception:
            on = True
        (self.fullscreen if on else self.unfullscreen)()
W(); Gtk.main()
DINO
    chmod 755 /usr/local/bin/osmo-dino-play

    # WIRESHARK AS ROOT: capture needs privileges (the install set setuid to
    # false). The launcher is a repo file, tools/osmo-wireshark-root.sh: root
    # via pkexec (graphical prompt), immediate capture with the bench
    # LTE + SCTP + GSM filter, 2G + 4G + phone (call, SMS, data) display
    # filter and filter buttons. /usr/local/bin/wireshark points at it:
    # « wireshark » at the keyboard is this one (the binary stays
    # /usr/bin/wireshark, which it calls).
    [ -f "$OSMO_EXTRAS_REPO/tools/osmo-wireshark-root.sh" ] \
        && install -m 755 "$OSMO_EXTRAS_REPO/tools/osmo-wireshark-root.sh" /usr/local/bin/osmo-wireshark-root
    ln -sfn /usr/local/bin/osmo-wireshark-root /usr/local/bin/wireshark

    # OFONO: telephony stack. No native GUI; open a terminal that makes sure
    # the daemon runs (root) and lists the modems.
    cat > /usr/local/bin/osmo-ofono <<'OF'
#!/bin/bash
# osmo-ofono - starts ofonod if needed and lists the modems (terminal).
set -u
SUDO=""; [ "$(id -u)" -ne 0 ] && SUDO="sudo -E"
$SUDO systemctl start ofono 2>/dev/null || $SUDO ofonod 2>/dev/null &
sleep 1
if [ -x /usr/share/ofono/scripts/list-modems ]; then
    /usr/share/ofono/scripts/list-modems
else
    command -v ofonoctl >/dev/null 2>&1 && ofonoctl list || \
        echo "ofono demarre (aucun outil de liste trouve)"
fi
echo; read -n1 -rsp 'ofono - une touche pour fermer...'
OF
    chmod 755 /usr/local/bin/osmo-ofono

    # THE BENCH PHONE: the postmarketOS / Phosh VM. postmarketOS comes with
    # ModemManager, Calls and Chatty, and plugs onto the REAL bench modem
    # (tools/osmo-phonesim-banc.py). tools/osmo-pmos.sh does the rest.
    cat > /usr/local/bin/osmo-pmos <<'PM'
```

```
#!/bin/bash
# osmo-pmos - the bench postmarketOS phone (see tools/osmo-pmos.sh).
exec "${OSMO_REPO:-/opt/GSM/osmo-operator}/tools/osmo-pmos.sh" "$@"
PM
    chmod 755 /usr/local/bin/osmo-pmos

    cat > /usr/local/bin/osmo-sms-send <<'SM'
#!/bin/bash
# osmo-sms-send <number> <text...> - sends an SMS through oFono via the bridge.
set -u
[ $# -ge 2 ] || { echo "usage: osmo-sms-send <numero> <texte...>"; exit 2; }
NUM="$1"; shift
# /usr/bin/python3 hard-coded: under the bench PATH, « env python3 » lands on
# the venv, which has no GObject bindings (see start-native.sh).
exec /usr/bin/python3 "${OSMO_REPO:-/opt/GSM/osmo-operator}/tools/osmo-ofono-bridge.py" --once "sms $NUM $*"
SM
    chmod 755 /usr/local/bin/osmo-sms-send

    cat > /usr/local/bin/osmo-call <<'CA'
#!/bin/bash
# osmo-call <number> | answer | hangup - calls through oFono via the bridge.
set -u
B="${OSMO_REPO:-/opt/GSM/osmo-operator}/tools/osmo-ofono-bridge.py"
P=/usr/bin/python3        # not the venv: it has no GObject bindings
case "${1:-}" in
    answer) exec "$P" "$B" --once "answer" ;;
    hangup) exec "$P" "$B" --once "hangup" ;;
    "")     echo "usage: osmo-call <numero> | answer | hangup"; exit 2 ;;
    *)      exec "$P" "$B" --once "call $1" ;;
esac
CA
    chmod 755 /usr/local/bin/osmo-call

    _ex_ok "lanceurs poses : osmo-youtube, osmo-dino-play, osmo-wireshark-root, osmo-ofono,"
    _ex_ok "                 osmo-pmos, osmo-sms-send, osmo-call"
}

# ── THE MISSING .desktop FILES + THE « Jeux » / « Media » FOLDERS ──────────
# The games and kodi ALREADY ship their .desktop (io.github.freedoom.Phase1/2,
# openarena.desktop, kodi.desktop). Only the missing ones are added (youtube,
# dino-jeu), then everything is SORTED into GNOME folders.
_osmo_extras_desktops() {
    install -d /usr/share/applications

    cat > /usr/share/applications/osmo-youtube.desktop <<'YTD'
[Desktop Entry]
Type=Application
Name=YouTube
Comment=YouTube dans Firefox (login Google) avec uBlock Origin
Exec=/usr/local/bin/osmo-youtube
Icon=firefox
Terminal=false
Categories=AudioVideo;Video;Network;
Keywords=youtube;video;media;
YTD

    cat > /usr/share/applications/osmo-dino-jeu.desktop <<'DIND'
[Desktop Entry]
Type=Application
Name=Cavalier d'ombres
Comment=Le jeu du dino, jouable
Exec=/usr/local/bin/osmo-dino-play
Icon=applications-games
Terminal=false
Categories=Game;ArcadeGame;
Keywords=dino;jeu;game;
```

```
DIND

    cat > /usr/share/applications/osmo-wireshark-root.desktop <<'WSD'
[Desktop Entry]
Type=Application
Name=Wireshark (banc)
GenericName=Analyseur reseau
Comment=Wireshark en root, capture immediate du banc, filtre 2G + 4G + telephone (signalisation, appel, SMS, data)
 ↪  sans les battements de coeur ; boutons Banc / 2G / 4G / Appel / SMS / Data / Attach
Exec=/usr/local/bin/osmo-wireshark-root %f
Icon=org.wireshark.Wireshark
Terminal=false
StartupNotify=true
Categories=Network;Monitor;
Keywords=wireshark;capture;pcap;reseau;lte;sctp;gsm;gsmtap;2g;4g;sms;appel;
WSD

    cat > /usr/share/applications/osmo-ofono.desktop <<'OFD'
[Desktop Entry]
Type=Application
Name=oFono
Comment=Pile telephonie oFono - demarre le daemon et liste les modems
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "oFono" -- /usr/local/bin/osmo-ofono
Icon=phone
Terminal=false
Categories=Network;Telephony;
Keywords=ofono;telephonie;modem;
OFD

    cat > /usr/share/applications/osmo-pmos.desktop <<'PMD'
[Desktop Entry]
Type=Application
Name=Telephone (postmarketOS)
Comment=Le telephone du banc - Phosh, ModemManager, appels et SMS reels
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --root --title "Telephone (postmarketOS)" --
 ↪  /usr/local/bin/osmo-pmos up
Icon=phone
Terminal=false
Categories=Network;Telephony;
Keywords=postmarketos;phosh;telephone;sms;appel;modem;
Actions=Etat;Arreter;

[Desktop Action Etat]
Name=Etat du telephone
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --keep-open --title "Etat du telephone" --
 ↪  /usr/local/bin/osmo-pmos status

[Desktop Action Arreter]
Name=Arreter le telephone
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --close-on-exit --title "Arret du telephone" --
 ↪  /usr/local/bin/osmo-pmos stop
PMD

    # The two pmbootstrap launchers (see update.sh, osmo_poser_pmos): the VM
    # and its format, then the modem and the voice.
    local ps
    for ps in qemu setup; do
        [ -f "$OSMO_EXTRAS_REPO/tools/osmo-pmos-$ps.sh" ] \
            && install -m 755 "$OSMO_EXTRAS_REPO/tools/osmo-pmos-$ps.sh" "/usr/local/bin/osmo-pmos-$ps"
    done
    # TWO FORMATS, TWO ICONS. Phosh lays itself out after the screen it is
    # given: tall and narrow is a phone; wide is a tablet - same image, same
    # session. The QEMU graphics card carries that size (OSMO_PMOS_RES, read
    # by the pmbootstrap patch) and the system boots inside it: the format
    # is chosen AT LAUNCH and does not change under an open session. Hence
    # two entries rather than a setting.
```

```
    local pf pn pr
    for pf in smartphone:720x1440 tablette:1280x800; do
        pn="${pf%%:*}"; pr="${pf#*:}"
        cat > "/usr/share/applications/osmo-pmos-$pn.desktop" <<PMD
[Desktop Entry]
Type=Application
Name=postmarketOS - $pn (banc)
Comment=Le telephone du banc, modem et voix branches tout seuls, en format $pn ($pr) - modem branche sur le banc
 ↪  GSM
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "Telephone du banc (VM)" -- env
 ↪  OSMO_PMOS_RES=$pr /usr/local/bin/osmo-pmos-qemu
Icon=$([ "$pn" = tablette ] && echo video-display || echo phone)
Terminal=false
Categories=Network;Telephony;System;
Keywords=postmarketos;pmos;qemu;modem;gsm;telephone;$pn;
Actions=Setup;SansModem;

[Desktop Action Setup]
Name=Rebrancher le modem et la voix (osmo-pmos-setup)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "Modem et voix du telephone" --
 ↪  /usr/local/bin/osmo-pmos-setup

[Desktop Action SansModem]
Name=Demarrer sans modem (VM nue)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "Telephone du banc (VM)" -- env
 ↪  OSMO_PMOS_RES=$pr OSMO_PMOS_MODEM=0 /usr/local/bin/osmo-pmos-qemu
PMD
        chmod 644 "/usr/share/applications/osmo-pmos-$pn.desktop"
    done

    chmod 644 /usr/share/applications/osmo-youtube.desktop \
              /usr/share/applications/osmo-dino-jeu.desktop \
              /usr/share/applications/osmo-wireshark-root.desktop \
              /usr/share/applications/osmo-ofono.desktop \
              /usr/share/applications/osmo-pmos.desktop

    # ── GNOME APPLICATION FOLDERS: « Jeux » and « Media » ──────────────────
    # A gschema override is read both by the live session (ISO) and by an
    # installed session; it is the only way that ALSO works in a chroot (no
    # dbus). The expected .desktop files are listed - a missing one is ignored
    # by GNOME, as for the dock favourites.
    local ov=/usr/share/glib-2.0/schemas/99-osmo-extras.gschema.override
    cat > "$ov" <<'OVR'
[org.gnome.desktop.app-folders]
folder-children=['Jeux', 'Media', 'Telephone', 'Outils']

[org.gnome.desktop.app-folders.folders:/org/gnome/desktop/app-folders/folders/Jeux/]
name='Jeux'
apps=['io.github.freedoom.Phase2.desktop', 'io.github.freedoom.Phase1.desktop', 'openarena.desktop',
 ↪  'osmo-dino-jeu.desktop']

[org.gnome.desktop.app-folders.folders:/org/gnome/desktop/app-folders/folders/Media/]
name='Media'
apps=['kodi.desktop', 'osmo-youtube.desktop']

[org.gnome.desktop.app-folders.folders:/org/gnome/desktop/app-folders/folders/Telephone/]
name='Telephone'
apps=['linphone.desktop', 'org.linphone.desktop', 'osmo-ofono.desktop', 'osmo-pmos.desktop']

[org.gnome.desktop.app-folders.folders:/org/gnome/desktop/app-folders/folders/Outils/]
name='Outils'
apps=['osmo-wireshark-root.desktop', 'org.wireshark.Wireshark.desktop', 'wireshark.desktop']
OVR
    glib-compile-schemas /usr/share/glib-2.0/schemas 2>/dev/null || true

    command -v gtk-update-icon-cache >/dev/null 2>&1 && \
```

```
        gtk-update-icon-cache -f -q /usr/share/icons/hicolor 2>/dev/null || true
    update-desktop-database /usr/share/applications 2>/dev/null || true
    _ex_ok "menus poses ; dossiers d applications « Jeux » et « Media »"
}

# ── ENTRY POINT ──────────────────────────────────────────────────────────
osmo_extras_install() {
    [ "$(id -u)" -eq 0 ] || { _ex_warn "osmo_extras_install demande root"; return 0; }
    if [ "${OSMO_EXTRAS_JEUX:-1}" = "1" ]; then
        _ex_say "jeux + media (Doom, Quake III, Kodi, YouTube, dino) ..."
    else
        _ex_say "media (Kodi, YouTube, dino) - jeux Doom/Quake sautes (OSMO_EXTRAS_JEUX=0) ..."
    fi
    _osmo_extras_apt
    [ "${OSMO_EXTRAS_JEUX:-1}" = "1" ] && _osmo_extras_jeux
    _osmo_extras_firefox_ublock
    _osmo_extras_lanceurs
    _osmo_extras_desktops
    echo -e "  ${GREEN}\xe2\x9c\x93${NC} jeux + media poses (dossiers ${BOLD}Jeux${NC} / ${BOLD}Media${NC}, et
     ↪  dans l encart du bureau)"
    return 0
}

# Run directly (not sourced): ./osmo-extras-install.sh
if [ "${BASH_SOURCE[0]}" = "${0}" ]; then
    osmo_extras_install
fi
```

2.249 /opt/GSM/osmo-operator/tools/osmo-fft-snap.py

49476 octets, 1048 lignes → 1048 lignes

```
#!/usr/bin/env python3
# osmo-fft-snap.py - the LIVE PANEL of the desktop: the wallpaper frame
# (tools/wallpaper-render.py, box 510,220-1410,1010 in 1920x1080), in TWO
# HALVES, each FADING to its bench once it is ready:
#
#   +-----------------------------------------------------------+
#   |  I/Q spectrum of the srsRAN UE (UL) |  srsUE console      |   4G
#   |  spectrum + waterfall (ZMQ 2001)    |  (tmux srsran:ue)   |
#   +-----------------------------------------------------------+
#   |  I/Q spectrum of the mobile (UL)    |  mobile.log (QEMU)  |   2G
#   |  spectrum + waterfall               |  last lines         |
#   +-----------------------------------------------------------+
#
# TWO HALVES, TWO BENCHES, TWO IMAGES. The frame is 790 px high and split in
# two: the top belongs to the 4G, the bottom to the 2G. Each half has ITS
# fade: as long as its bench is not there, it shows the comic strip the
# wallpaper carries at that spot (two images: a random geek comic at the top,
# Calvin & Hobbes at the bottom - tools/osmo-wallpaper.sh). One bench on, one
# live half; both, the whole panel.
#
# THE 4G IS LISTENED TO WITHOUT TAKING ANYTHING FROM IT. The srsRAN virtual
# radio is ZeroMQ in REQ/REP: the receiver ASKS for samples, and the
# transmitter ANSWERS it - once. A third party asking on the UE port (2001)
# would STEAL answers from the eNB, which would no longer have the whole
# uplink: attach fails, and with fail_on_disconnect its radio stops. So NO
# ZeroMQ socket is opened. The TCP traffic of the loopback interface is read
# through a raw capture (AF_PACKET, hence the unit runs as root), the port
# 2001 stream is followed and the ZMTP frames are cut out of it: each UE
# answer is [empty frame][long frame = N complex64 samples]. Nothing is sent
# back, nothing is consumed: srsue and srsenb cannot notice. The UE only
# transmits in bursts (PRACH, PUSCH...), and 11.52 MS/s make 92 MB/s on the
# loopback: only a 250 ms window is read per image and the PEAK of each
```

```
# spectrum bin over that window is kept, otherwise only silence between two
# bursts would be seen.
#
# 2G FFT source: the dashboard, http://127.0.0.1:8080/psd?src=ms - the same
# JSON as the FFT tab (fft1 view): freqs, psd (dB), dr, arfcn. The log: the
# stack's mobile.log (QEMU MS#1), read from the tail. On the 4G side, the log
# is the srsue console, captured in the « srsran » tmux that carries it
# (tools/osmo-lte.sh), and /tmp/ue.log as fallback.
#
#   /run/osmo-fft/panel.png   900x790   (wallpaper coordinates; osmo-panel.py
#                                        scales it to the screen)
#
# Launched by osmo-fft-snap.service. Depends only on Pillow and the DejaVu
# fonts, present on the image as on the host; numpy for the 4G FFT (without
# it, the top half says so and stays on its comic strip).
import glob
import json
import os
import re
import socket
import struct
import subprocess
import sys
import threading
import time
import urllib.request

from PIL import Image, ImageDraw, ImageFont

URL = os.environ.get("OSMO_FFT_URL", "http://127.0.0.1:8080/psd")
# DOWNLINK AS BACKGROUND, UPLINK ON TOP. The uplink alone (src=ms) only has
# signal when the mobile transmits (RACH, dedicated channel): in between the
# panel would show a flat floor. The downlink (src=bts, /tmp/iq_fft.fifo
# filled by pont.py) continuously carries the BCCH, the CCCH and hence the
# paging. It is drawn permanently, and the uplink is overlaid in orange as
# soon as it carries a burst.
# OSMO_FFT_2G=ms shows the uplink alone.
FFT_2G = os.environ.get("OSMO_FFT_2G", "bts+ms")
UL_ACTIF_DB = float(os.environ.get("OSMO_FFT_UL_ACTIF_DB", "20"))
OUT = os.environ.get("OSMO_FFT_DIR", "/run/osmo-fft")
PERIOD = float(os.environ.get("OSMO_FFT_PERIOD", "1"))
FADE_S = float(os.environ.get("OSMO_FFT_FADE", "3"))
# Opacity of the bench OVER the background image, once the fade is done.
#
# 1.0: with a lower value the comic strip would show through the log and the
# spectrum (the black strokes of the drawing between the mobile.log lines, the
# spectrum on a patterned background) and both would be lost. The fade stays:
# it is what moves from the image to the bench and back (FADE_S), and during
# those three seconds the two overlap. OSMO_FFT_OPACITY=0.86 lets the image
# show through for whoever wants to try.
OPACITY = max(0.0, min(1.0, float(os.environ.get("OSMO_FFT_OPACITY", "1.0"))))
# State written by tools/osmo-wallpaper.sh: is there a strip in the wallpaper?
# Without network, gocomics does not render the image and the wallpaper goes
# out WITHOUT strip - the frame is then empty, and blending the bench over
# nothing only dulls the spectrum and the log. In that case the panel goes to
# full opacity: no png, but mobile.log and the FFT stay sharp.
STRIP_STATE = os.environ.get("OSMO_WP_STATE", "/var/cache/osmo-wallpaper/strip.state")
WALLPAPER = os.environ.get("OSMO_WP_FILE", "/usr/share/backgrounds/gsm-lab-wallpaper.jpg")
# ── WHAT LIES UNDER THE PANEL: THE WHOLE ILLUSTRATION ──────────────────
# The default wallpaper is an image without frame, and the GNOME desktop
# carries what the operator puts there (configs/gsm-lab-fond.jpg): cutting a
# box out of it would give an enlarged piece of sky and hill, meaningless.
# In « entier » mode the panel therefore shows the bench illustration WHOLE
# (pylons, sun, hill), fitted into its box - the top and bottom left free by
# the aspect ratio are filled with the same image, blurred and darkened,
# rather than two black bands.
```

```
#
# The composed render keeps its path: when osmo-wallpaper.sh has run and
# written FOND= in strip.state, THAT file is cropped - its frames are exactly
# where the box expects them. OSMO_FFT_FOND=decoupe forces the crop on
# $OSMO_WP_FILE.
# ONE SCENE. The illustration and the GNOME wallpaper come from the same
# drawing (tools/paysage-render.py): the wallpaper is the bare landscape, the
# illustration is the BOX crop of that landscape WITH the pylons. It thus
# already has the W x H format (900/790): « entier » fits it without band,
# and the antennas look planted into the desktop by the panel. The shipped
# files are a retouched version of that render, chosen on the bench.
ILLUSTRATION = os.environ.get("OSMO_FFT_IMAGE", "/usr/share/backgrounds/gsm-lab-wallpaper.jpg")
# NO IMAGE IN THE CENTRE. By default (« aucun ») the panel background is
# TRANSPARENT: bench down, the desktop shows; bench up, the two panes
# (spectrum, log) sit on top at PANEL_ALPHA opacity - not less, a spectrum
# over a light landscape is unreadable. panel.png is RGBA, osmo-panel.py
# displays it on a window that is itself transparent.
# OSMO_FFT_FOND=entier (the illustration) or decoupe give the image backgrounds.
FOND_MODE = os.environ.get("OSMO_FFT_FOND", "aucun")
PANEL_ALPHA = max(0.0, min(1.0, float(os.environ.get("OSMO_FFT_PANEL_ALPHA", "0.8"))))
PANEL = (8, 10, 14, int(round(255 * PANEL_ALPHA)))
# ── WHERE THE MOBILE LOG IS ──────────────────────────────────────────
# ONE PATH, PUBLISHED BY start-native.sh. mobile.log lives under LOG_DIR
# (= RUN_DIR/logs: /run/user/<uid>/osmo-nitb/logs, or /tmp/osmo-nitb without
# XDG_RUNTIME_DIR - environment/paths.env), and in --dsp under the c54x_exe
# RUNDIR (/tmp/c54x-pont). start-native.sh writes the actual path to
# /run/osmo-banc/mobile-log (like ms1-vty for the VTY port) and into the
# systemd environment (OSMO_MOBILE_LOG).
# Order: the published file; OSMO_MOBILE_LOG; else the most recent of the
# known locations; else the default. Resolved at every render: the log
# appears AFTER the panel is launched.
MOBILE_LOG_CANDIDATS = ("/run/user/*/osmo-nitb/logs/mobile.log",
                        "/tmp/osmo-nitb/logs/mobile.log",
                        "/tmp/c54x-pont/mobile.log")
MOBILE_LOG_DEFAUT = "/run/user/0/osmo-nitb/logs/mobile.log"


def mobile_log_path():
    # THE PUBLISHED FILE FIRST. OSMO_MOBILE_LOG is frozen at service start,
    # while /run/osmo-banc/mobile-log is rewritten at every bench launch
    # (e.g. rvinterf.log after a --ms-magnetite).
    try:
        with open("/run/osmo-banc/mobile-log") as f:
            publie = f.read().strip()
        if publie:
            return publie
    except OSError:
        pass
    force = os.environ.get("OSMO_MOBILE_LOG")
    if force:
        return force
    vus = []
    for motif in MOBILE_LOG_CANDIDATS:
        for p in glob.glob(motif):
            try:
                vus.append((os.stat(p).st_mtime, p))
            except OSError:
                pass
    return max(vus)[1] if vus else MOBILE_LOG_DEFAUT


# The operator selected in the panel (tools/osmo-panel.py, arrows): OP=,
# MODE=, IP=, NAME=, DASH=. Absent or native: the local dashboard and log.
OP_FILE = os.path.join(OUT, "operator")
# The panel box in the wallpaper (wallpaper-render.py, main()):
# THE FOUR FILES THAT KNOW IT MUST STAY IN AGREEMENT -
```

```
# wallpaper-render.py, osmo-fft-snap.py, osmo-panel.py, osmo-launcher.py.
BOX = (510, 220, 1410, 1010)
W, H = BOX[2] - BOX[0], BOX[3] - BOX[1]
# The two halves, in image coordinates: (top y, bottom y). The wallpaper
# draws two frames separated by ECART px (the two images), and each live half
# covers exactly its own.
ECART = 8
HH = (H - ECART) // 2
HAUT = (0, HH)
BAS = (H - HH, H)
PAD = 18
# FOOT = 0: NO FOOTER BAND. The content - spectrum, waterfall, log - takes
# the whole height, and only the buttons remain, laid on top in the corner
# (they carry the operator number, so a footer line would say nothing more).
# FOOT stays named: the geometry is expressed with it, and setting it to 30
# gives a footer band.
FOOT = 0
FONT_DIR = "/usr/share/fonts/truetype/dejavu"
ANSI = re.compile(r"\x1b\[[0-9;]*[A-Za-z]|\x07")
# ── THE LOG COLOURS ARE ALREADY IN THE LOG ──────────────────────────────
# mobile.log is written WITH its ANSI codes (osmocom colours by subsystem:
# « \033[0;31m<0004> gsm322.c:... \033[0;m »). The colour the log carries is
# rendered as is, with the Tango palette - the one of the image's
# gnome-terminal profile (85-installeur-bureau.sh) - so the reading matches
# the terminal.
SGR = re.compile(r"\x1b\[([0-9;]*)m")
TANGO = {
    "30": (85, 87, 83),    "31": (204, 0, 0),     "32": (78, 154, 6),
    "33": (196, 160, 0),   "34": (52, 101, 164),  "35": (117, 80, 123),
    "36": (6, 152, 154),   "37": (211, 215, 207),
}
TANGO_VIF = {
    "30": (128, 130, 126), "31": (239, 41, 41),   "32": (138, 226, 52),
    "33": (252, 233, 79),  "34": (114, 159, 207), "35": (173, 127, 168),
    "36": (52, 226, 226),  "37": (238, 238, 236),
}
LOG_DEFAUT = (200, 210, 225)


def couleur_ansi(brut):
    """The colour a terminal would give this line. None if it has none."""
    for m in SGR.finditer(brut):
        codes = [c for c in m.group(1).split(";") if c]
        if not codes or codes == ["0"]:
            continue                        # reset: not a colour
        table = TANGO_VIF if "1" in codes else TANGO
        for c in codes:
            if c in table:
                return table[c]
            if c.startswith("9") and len(c) == 2 and c[1] in "01234567":
                return TANGO_VIF["3" + c[1]]   # 90-97: the ECMA-48 bright colours
    return None


def font(name, size):
    try:
        return ImageFont.truetype(os.path.join(FONT_DIR, name), size)
    except OSError:
        return ImageFont.load_default()


F_TITLE = font("DejaVuSansMono-Bold.ttf", 15)
F_SMALL = font("DejaVuSansMono.ttf", 12)
F_LOG = font("DejaVuSansMono.ttf", 12)
```

```
def palette_sdr(v):
    """SDR palette, 0..1 -> RGB: navy, blue, cyan, green, yellow, red (the same as the
    FFT tab, sdr)."""
    ancres = [(0,0,40), (0,20,150), (0,90,255), (0,200,230), (0,210,80), (170,235,0), (255,230,0), (255,130,0),
    (230,0,0)]
    stops = [(i / (len(ancres) - 1), c) for i, c in enumerate(ancres)]
    v = 0.0 if v < 0 else 1.0 if v > 1 else v
    for i in range(len(stops) - 1):
        a, ca = stops[i]
        b, cb = stops[i + 1]
        if v <= b:
            t = (v - a) / (b - a) if b > a else 0
            return tuple(int(ca[j] + (cb[j] - ca[j]) * t) for j in range(3))
    return stops[-1][1]


def operator():
    """{'OP','MODE','IP','NAME','DASH'} of the selected operator, native by default."""
    op = {"OP": "1", "MODE": "native", "IP": "", "NAME": "", "DASH": URL.rsplit("/", 1)[0]}
    try:
        with open(OP_FILE) as f:
            for line in f:
                k, _, v = line.strip().partition("=")
                if k in op:
                    op[k] = v
    except OSError:
        pass
    return op


def fetch(src):
    op = operator()
    base = URL if op["MODE"] != "docker" else op["DASH"].rstrip("/") + "/psd"
    req = urllib.request.Request(f"{base}?src={src}&t={int(time.time() * 1000)}",
                                 headers={"Cache-Control": "no-store"})
    with urllib.request.urlopen(req, timeout=2) as r:
        return json.loads(r.read().decode("utf-8", "replace"))


def normalize(psd, dr):
    top = max(psd)
    floor = top - (dr if dr and dr > 0 else 40)
    span = top - floor if top > floor else 1
    return [min(1.0, max(0.0, (p - floor) / span)) for p in psd]


def resample(vals, n):
    """Brings a list down to n columns (max per group: a peak stays visible)."""
    if len(vals) <= n:
        return vals + [vals[-1]] * (n - len(vals)) if vals else [0.0] * n
    out = []
    step = len(vals) / n
    for i in range(n):
        a, b = int(i * step), max(int((i + 1) * step), int(i * step) + 1)
        out.append(max(vals[a:b]))
    return out


_strip = {"mtime": None, "present": True, "fond": ""}


def _lire_strip_state():
    """Rereads strip.state (osmo-wallpaper.sh) when it changes: STRIP= and FOND=."""
    try:
        mt = os.stat(STRIP_STATE).st_mtime
    except OSError:
```

```
        _strip.update(mtime=None, present=True, fond="")
        return
    if mt == _strip["mtime"]:
        return
    present, fond = True, ""
    try:
        with open(STRIP_STATE) as fh:
            for line in fh:
                k, _, v = line.strip().partition("=")
                if k == "STRIP":
                    present = v.strip().lower() != "non"
                elif k == "FOND":
                    fond = v.strip()
    except OSError:
        pass
    _strip.update(mtime=mt, present=present, fond=fond)


def strip_present():
    """Does the wallpaper of the day carry a strip? (default: yes, no guessing)"""
    _lire_strip_state()
    return _strip["present"]


def _source_fond():
    """(file, mode): the composed render to CROP, or the illustration to FIT."""
    _lire_strip_state()
    f = _strip["fond"]
    if _strip["present"] and f and os.path.isfile(f):
        return f, "decoupe"
    if FOND_MODE == "decoupe":
        return WALLPAPER, "decoupe"
    if FOND_MODE == "aucun":
        return None, "aucun"
    return (ILLUSTRATION if os.path.isfile(ILLUSTRATION) else WALLPAPER), "entier"


def _ajuster(im):
    """The WHOLE image in W x H: fitted and centred; the rest of the box is the
    same image filling it, blurred and darkened (no black bands)."""
    from PIL import ImageEnhance, ImageFilter
    im.thumbnail((2 * W, 2 * H), Image.LANCZOS)       # 1800x1580 (or more): fast enough
    sw, sh = im.size
    s = max(W / sw, H / sh)
    fond = im.resize((max(W, int(sw * s + 0.5)), max(H, int(sh * s + 0.5))), Image.LANCZOS)
    x0, y0 = (fond.width - W) // 2, (fond.height - H) // 2
    fond = fond.crop((x0, y0, x0 + W, y0 + H)).filter(ImageFilter.GaussianBlur(18))
    fond = ImageEnhance.Brightness(fond).enhance(0.5)
    s = min(W / sw, H / sh)
    dev = im.resize((max(1, int(sw * s)), max(1, int(sh * s))), Image.LANCZOS)
    fond.paste(dev, ((W - dev.width) // 2, (H - dev.height) // 2))
    return fond


# ── THE PANEL BACKGROUND, REREAD WHEN IT CHANGES ──────────────────────
# The mtime of the chosen file is followed (and the file itself:
# osmo-wallpaper.sh may change it on the day it runs).
_base = {"cle": None, "img": None}


def base_image():
    src, mode = _source_fond()
    try:
        mt = os.stat(src).st_mtime if src else None
    except OSError:
        mt = None
```

```
    cle = (src, mode, mt)
    if cle != _base["cle"] or _base["img"] is None:
        img = None
        if mt is not None:
            try:
                wp = Image.open(src).convert("RGB")
                if mode == "decoupe":
                    if wp.size != (1920, 1080):
                        wp = wp.resize((1920, 1080), Image.LANCZOS)
                    img = wp.crop(BOX)
                else:
                    img = _ajuster(wp)
                print(f"[fft-snap] fond de l encart : {src} ({mode})", flush=True)
            except Exception as e:  # unreadable background: dark frame, no strip
                print(f"[fft-snap] fond {src} : {e}", file=sys.stderr, flush=True)
        if img is None and mode == "aucun":
            img = Image.new("RGBA", (W, H), (0, 0, 0, 0))     # nothing: the desktop
        elif img is None:
            img = Image.new("RGB", (W, H), (20, 24, 36))
            for m in (HAUT, BAS):     # the two frames, as the wallpaper draws them
                ImageDraw.Draw(img).rounded_rectangle((0, m[0], W - 1, m[1] - 1), radius=26,
                                                      outline=(200, 200, 210), width=2)
        _base.update(cle=cle, img=img.convert("RGBA"))
    return _base["img"]


def base_moitie(m):
    """The piece of background under one half of the panel (HAUT or BAS)."""
    return base_image().crop((0, m[0], W, m[1]))


# ── THE MOBILE LOG ──────────────────────────────────────────────────────
# The name of the file read in each container (mobile.log, rvinterf.log...),
# filled by tail_lines for the panel title.
JOURNAL_DOCKER_NOM = {}


def tail_lines(path, n, width):
    op = operator()
    if op["MODE"] == "docker":
        # `docker logs` SHOWS NOTHING OF AN OPERATOR, exactly as for the hub
        # (see HUB_LOGS below): the stack is launched by `docker exec` in a
        # tmux and its output goes to FILES, not to the container's standard
        # stream, which only carries the two entrypoint lines. The file is
        # read INSIDE the container; `docker logs` remains the last resort for
        # an image that would launch the operator differently.
        data = ""
        # THE PATH PUBLISHED IN THE CONTAINER, NOT THE HOST ONE. The container
        # publishes its log in /run/osmo-banc/mobile-log too (start-native.sh):
        # it is read there, and failing that the most recent NON-EMPTY of the
        # known locations (DSP, instance 2, Magnetite, qosmo). The first line
        # returned carries the file name (@@path) for the panel title.
        choix = (
            'p=$(cat /run/osmo-banc/mobile-log 2>/dev/null); '
            'if [ -n "$p" ] && [ -s "$p" ]; then f="$p"; else f=""; '
            'for x in $(ls -t /tmp/c54x-pont/mobile.log /tmp/c54x-pont2/mobile.log '
            '/tmp/c54x-pont/rvinterf.log /tmp/c54x-pont2/rvinterf.log '
            '/tmp/osmo-nitb/logs/mobile.log /run/user/*/osmo-nitb/logs/mobile.log 2>/dev/null); do '
            '[ -s "$x" ] && { f="$x"; break; }; done; fi; '
            '[ -n "$f" ] || exit 3; echo "@@$f"; tail -n ' + str(n + 5) + ' "$f"')
        try:
            out = subprocess.run(["docker", "exec", op["NAME"], "sh", "-c", choix],
                                 capture_output=True, text=True, timeout=4)
        except Exception as e:
            return [(f"{op['NAME']} : journal inaccessible ({type(e).__name__})", None)]
        if out.returncode == 0 and out.stdout.startswith("@@"):
```

```
            nom, _, data = out.stdout.partition("\n")
            JOURNAL_DOCKER_NOM[op["NAME"]] = os.path.basename(nom[2:].strip())
        elif "permission denied" in (out.stderr or "").lower():
            return [("journal indisponible (docker : droit refuse -", None),
                    ("  le compte n est pas dans le groupe docker)", None)]
        if not data:
            try:
                r = subprocess.run(["docker", "logs", "--tail", str(n + 5), op["NAME"]],
                                   capture_output=True, text=True, timeout=3)
                data = (r.stdout + r.stderr)
            except Exception as e:
                return [(f"{op['NAME']} : journal inaccessible ({type(e).__name__})", None)]
    else:
        try:
            with open(path, "rb") as f:
                f.seek(0, 2)
                size = f.tell()
                f.seek(max(0, size - 16384))
                data = f.read().decode("utf-8", "replace")
        except OSError:
            return [(f"{os.path.basename(path)} : pas encore de journal", None)]
    # The RAW line is kept long enough to read its colour, then cleaned.
    brutes = [l for l in data.splitlines() if ANSI.sub("", l).strip()]
    out = []
    for brut in brutes[-n:]:
        col = couleur_ansi(brut)
        l = ANSI.sub("", brut).rstrip()
        # The osmocom timestamp (20260904135215721) eats 17 columns for nothing.
        l = re.sub(r"^\d{17}\s+", "", l)
        out.append((l[:width], col))
    return out or [("(journal vide)", None)]


# ── THE BENCH: SPECTRUM ON THE LEFT, LOG ON THE RIGHT ──────────────────
history = []


# ── THE HUB VIEW: SS7, NOT AN FFT ──────────────────────────────────────
# An M3UA hub has no radio: no /psd to query, no ARFCN, no mobile.log. An
# empty FFT on the central node of the bench would be useless precisely when
# one is looking for why SS7 does not pass.
#
# What a hub does have: a connectivity matrix (who talks to whom), its ASPs,
# its routes, and a log. That is what is shown.
#
# The matrix comes from the CACHE filled by tools/conky-osmo-status.sh
# (--refresh-matrix, from checks/ss7_check.sh): the measurement takes a
# minute, it has no place in a 1 Hz render loop. The same file as the Conky
# is read - one measurement, two displays. The Conky ${colorN} tags are
# stripped here and rendered as Pillow colours.
MATRIX_FILE = os.path.join(OUT, "ss7-matrix")
CONKY_TAG = re.compile(r"\$\{color[0-9]?\}")
COUL_SS7 = {"self": (88, 166, 255), "via": (63, 185, 80), "FAIL": (248, 81, 73)}


def matrice_lignes():
    """The matrix cache, without the Conky tags. [] if it does not exist."""
    try:
        with open(MATRIX_FILE) as f:
            return [CONKY_TAG.sub("", l.rstrip("\n")) for l in f if l.strip()]
    except OSError:
        return []


# The places where the hub writes, in the order they are tried.
#
```

```
# `docker logs osmo-inter-stp` is EMPTY by design: the container's command is
# `sleep infinity`, and osmo-stp is launched AFTERWARDS by `docker exec` in a
# tmux, its output going through a `tee` to a file. None of that crosses the
# container's standard stream. So the file is read INSIDE the container;
# docker logs remains the last resort, for an image that would launch the
# STP differently.
HUB_LOGS = ("/tmp/osmo-stp.log", "/var/log/osmocom/osmo-stp.log")


def hub_journal(nom, n, width):
    """The last lines of the hub STP. Without docker (or without permission),
    say so rather than leaving an empty frame - an empty frame reads like a
    hub failure, when it is the probe whose arm is not long enough."""
    for chemin in HUB_LOGS:
        try:
            out = subprocess.run(["docker", "exec", nom, "tail", "-n", str(n), chemin],
                                 capture_output=True, text=True, timeout=4)
        except Exception as e:
            return [(f"journal indisponible : {e}", (212, 153, 34))]
        if out.returncode == 0 and out.stdout.strip():
            return [(ANSI.sub("", l)[:width], None) for l in out.stdout.splitlines()[-n:]]
        if "permission denied" in (out.stderr or "").lower():
            return [("journal indisponible (docker : droit refuse -",
                     (212, 153, 34)), ("  le compte n est pas dans le groupe docker)", (212, 153, 34))]
    try:
        out = subprocess.run(["docker", "logs", "--tail", str(n), nom],
                             capture_output=True, text=True, timeout=4)
        lignes = (out.stdout + out.stderr).splitlines()[-n:]
        if lignes:
            return [(ANSI.sub("", l)[:width], None) for l in lignes]
    except Exception:
        pass
    return [("pas de journal : " + " ni ".join(HUB_LOGS), (139, 148, 158)),
            ("le hub tourne-t-il ?  docker ps --filter name=" + nom, (139, 148, 158))]


def render_interstp(op):
    """The panel when the selection is the inter-STP: matrix on the left, hub
    log on the right. Same frame, same margins and same footer as the radio
    view - only the content changes, so the eye keeps its landmarks when
    moving from an operator to the hub."""
    img = base_moitie(BAS).copy()
    d = ImageDraw.Draw(img)
    # ── THE PANE COVERS THE WHOLE FRAME, BORDER INCLUDED ──────────────────
    # The PAD margin would otherwise let base_image() show through - the piece
    # of wallpaper, hence the `glass_panel` painted there by
    # wallpaper-render.py: a lighter fill (20,24,36) and above all a BORDER
    # (200,200,210), a light rounded frame all around the panel.
    #
    # When the bench runs, the panel is a solid pane: it covers the WHOLE
    # frame, same radius as the wallpaper one (26) to land exactly on it.
    # The wallpaper frame stays visible when the bench is down - it then
    # frames the image of the day, and there it makes sense.
    d.rounded_rectangle((0, 0, W - 1, HH - 1), radius=26, fill=PANEL)
    x0, y0, x1, y1 = PAD, PAD, W - PAD, HH - PAD - FOOT
    d.rounded_rectangle((x0 - 6, y0 - 6, x1 + 6, y1 + 6), radius=8, fill=PANEL)
    split = x0 + int((x1 - x0) * 0.56)

    d.text((x0, y0), "Matrice de connectivite  ·  via inter-STP", font=F_TITLE, fill=(88, 166, 255))
    tag = f"PC {os.environ.get('MULTI_HUB_PC', '0.0.0')}"
    d.text((split - 12 - d.textlength(tag, font=F_SMALL), y0 + 2), tag, font=F_SMALL, fill=(63, 185, 80))

    lignes = matrice_lignes()
    yy = y0 + 30
    if not lignes:
        d.text((x0, yy), "premiere mesure en cours (checks/ss7_check.sh)...",
```

```
            font=F_SMALL, fill=(139, 148, 158))
    for ligne in lignes:
        xx = x0
        # Each word is painted in its colour: self/via/FAIL carry the meaning.
        for mot in re.split(r"(\s+)", ligne):
            if not mot:
                continue
            col = COUL_SS7.get(mot.strip(), (200, 208, 220))
            if mot.strip().startswith("SS7"):
                col = (63, 185, 80) if "OK" in ligne else (212, 153, 34)
            d.text((xx, yy), mot, font=F_LOG, fill=col)
            xx += d.textlength(mot, font=F_LOG)
        yy += 17
        if yy > y1 - 14:
            break

    # The hub log, on the right, exactly where the mobile one lives.
    lx0 = split
    d.line((lx0 - 6, y0, lx0 - 6, y1), fill=(30, 36, 48))
    d.text((lx0, y0), f"journal  ·  {op['NAME']}", font=F_TITLE, fill=(88, 166, 255))
    line_h = 15
    n = max(1, (y1 - (y0 + 26)) // line_h)
    cols = max(10, int((x1 - lx0) / 7.3))
    for i, (l, col) in enumerate(hub_journal(op["NAME"], n, cols)):
        if col is None:
            col = LOG_DEFAUT
            if re.search(r"error|fail|reject|lost|down", l, re.I):
                col = TANGO_VIF["31"]
            elif re.search(r"ASP|AS |active|established|route", l, re.I):
                col = TANGO_VIF["32"]
        d.text((lx0, y0 + 26 + i * line_h), l, font=F_LOG, fill=col)

    return img


def render_live(data, data_ul=None):
    img = base_moitie(BAS).copy()
    d = ImageDraw.Draw(img)
    # ── THE PANE COVERS THE WHOLE FRAME, BORDER INCLUDED ──────────────────
    # Same rule as render_interstp(): a solid pane over the whole frame, same
    # radius as the wallpaper frame (26), so that nothing of the wallpaper
    # `glass_panel` (fill and border) shows through the margins.
    d.rounded_rectangle((0, 0, W - 1, HH - 1), radius=26, fill=PANEL)
    x0, y0, x1, y1 = PAD, PAD, W - PAD, HH - PAD - FOOT
    d.rounded_rectangle((x0 - 6, y0 - 6, x1 + 6, y1 + 6), radius=8, fill=PANEL)
    split = x0 + int((x1 - x0) * 0.56)
    # Spectrum
    titre = ("Spectre I/Q du mobile  ·  montant (UL)" if FFT_2G == "ms"
             else "Spectre I/Q  ·  DL + UL")
    d.text((x0, y0), titre, font=F_TITLE, fill=(88, 166, 255))
    arfcn = data.get("arfcn", "?") if data else "?"
    tag = f"ARFCN {arfcn}"
    d.text((split - 12 - d.textlength(tag, font=F_SMALL), y0 + 2), tag, font=F_SMALL, fill=(63, 185, 80))
    sx0, sx1 = x0, split - 12
    sw = sx1 - sx0
    py0 = y0 + 24
    psd_h = 110
    py1 = py0 + psd_h
    d.rectangle((sx0, py0, sx1, py1), fill=(12, 16, 24))
    for gy in range(1, 4):
        yy = py0 + gy * psd_h // 4
        d.line((sx0, yy, sx1, yy), fill=(24, 30, 40))
    if data and "psd" in data:
        vals = resample(normalize(data["psd"], data.get("dr", 40)), sw)
        pts = [(sx0 + x, py1 - 1 - int(v * (psd_h - 4))) for x, v in enumerate(vals)]
        for x, y in pts:
```

```
            d.line((x, y, x, py1 - 1), fill=(30, 70, 110))
        d.line(pts, fill=(88, 210, 255), width=1)
        ul = data_ul.get("psd") if data_ul else None
        if ul and max(ul) - min(ul) >= UL_ACTIF_DB:
            # Uplink burst: its curve on top, and in the waterfall the max of
            # both, so that a RACH or an SDCCH block leaves its trace there.
            vul = resample(normalize(ul, data_ul.get("dr", 40)), sw)
            d.line([(sx0 + x, py1 - 1 - int(v * (psd_h - 4))) for x, v in enumerate(vul)],
                   fill=(255, 166, 60), width=1)
            vals = [max(a, b) for a, b in zip(vals, vul)]
        history.insert(0, vals)
    else:
        d.text((sx0 + 8, py0 + 8), "pas de flux", font=F_SMALL, fill=(248, 81, 73))
    # Waterfall: one line per image, the most recent at the top
    wy0 = py1 + 4
    wf_h = y1 - wy0
    del history[wf_h:]
    wf = Image.new("RGB", (sw, wf_h), (8, 10, 14))
    px = wf.load()
    for row, line in enumerate(history):
        for x in range(min(sw, len(line))):
            px[x, row] = palette_sdr(line[x])
    img.paste(wf, (sx0, wy0))
    # Log
    d = ImageDraw.Draw(img)
    lx0 = split
    d.line((lx0 - 6, y0, lx0 - 6, y1), fill=(30, 36, 48))
    op = operator()
    # The title names the file read: rvinterf.log (TI traces) with --ms-magnetite
    line_h = 15
    n = max(1, (y1 - (y0 + 26)) // line_h)
    cols = max(10, int((x1 - lx0) / 7.3))
    # The lines first: for a container, reading them is what says which file
    # carries the log (JOURNAL_DOCKER_NOM), and the title names it.
    lignes = tail_lines(mobile_log_path(), n, cols)
    d.text((lx0, y0), (f"{os.path.basename(mobile_log_path())}  ·  QEMU MS#1" if op["MODE"] != "docker"
                       else f"{JOURNAL_DOCKER_NOM.get(op['NAME'], 'journal')}  ·  {op['NAME']}  ·  MS#1"),
           font=F_TITLE, fill=(88, 166, 255))
    for i, (l, col) in enumerate(lignes):
        if col is None:
            # Log without colours (docker logs, a redirection that ate them):
            # fall back on keyword matching.
            col = LOG_DEFAUT
            if re.search(r"error|fail|reject|lost", l, re.I):
                col = TANGO_VIF["31"]
            elif re.search(r"attach|answer|call|SMS|proceed", l, re.I):
                col = TANGO_VIF["32"]
        d.text((lx0, y0 + 26 + i * line_h), l, font=F_LOG, fill=col)
    # The pane is built ON a copy of the wallpaper (base_image): whatever the
    # render does not paint lets the background show, so the pane is painted
    # to the edge in a single colour (no footer band, FOOT = 0). What a
    # footer caption would say - which operator, which log - is carried by
    # the buttons themselves (« Dashboard op2 », « VTY 4247 op2 ») and by the
    # title of the right column.
    return img


# ── THE 4G: THE srsRAN UE, RAW-CAPTURED ON THE LOOPBACK ──────────────────
# See the header: no ZeroMQ socket, the TCP of the UE port is read as it
# passes on lo and the ZMTP frames are cut out of it. Settings:
#   OSMO_LTE_TAP_PORT    the « tx » ZMQ port of srsue (2001; 2000 = the eNB,
#                        hence the downlink, for whoever prefers to see it)
#   OSMO_LTE_TAP_IFACE   the interface (lo)
#   OSMO_LTE_TAP_WINDOW  the listening window per image, in s (0.25)
#   OSMO_SRSRAN_UE_CONF  the ue.conf to read for the EARFCN and the rate
#   OSMO_SRSRAN_TMUX     the tmux (-L) carrying srsue, window « ue »
```

```
#   OSMO_SRSRAN_UE_LOG   the srsue log, failing the tmux
LTE_PORT = int(os.environ.get("OSMO_LTE_TAP_PORT", "2001"))
LTE_IFACE = os.environ.get("OSMO_LTE_TAP_IFACE", "lo")
LTE_FENETRE = float(os.environ.get("OSMO_LTE_TAP_WINDOW", "0.25"))
LTE_NFFT = 1024
LTE_UE_CONF = os.environ.get("OSMO_SRSRAN_UE_CONF", "/root/.config/srsran/ue.conf")
LTE_TMUX = os.environ.get("OSMO_SRSRAN_TMUX", "srsran")
# tools/osmo-lte.sh launches srsue with --log.filename=/tmp/osmo-lte-ue.log
# (which overrides the « filename = /tmp/ue.log » of ue.conf) and its STANDARD
# OUTPUT into /tmp/osmo-lte-ue.console - that is where « Found PLMN », « RRC
# Connected », « Network attach successful » are. The tmux -L srsran only
# exists if it was opened by hand.
LTE_UE_LOG = os.environ.get("OSMO_SRSRAN_UE_LOG", "/tmp/osmo-lte-ue.log")
LTE_UE_CONSOLE = os.environ.get("OSMO_SRSRAN_UE_CONSOLE", "/tmp/osmo-lte-ue.console")
LTE_UE_SOURCES = [LTE_UE_CONSOLE, LTE_UE_LOG, "/tmp/ue.log"]
# The marker of a ZMTP answer from the UE: the empty frame of the REQ/REP
# (0x01 = « there is another one », size 0), then the header of a long frame
# (0x02, 8-byte big-endian size, whose first five bytes are zero as long as
# it is under 2^24 bytes - a one-second frame would be 92 M).
ZMTP_MARQUE = b"\x01\x00\x02\x00\x00\x00\x00\x00"
try:
    import numpy as np
except ImportError:      # the 4G half will say so; the 2G does not need it
    np = None


def _tcp_ecoute(port):
    """Does someone listen on this TCP port? (read of /proc/net/tcp, no privilege)"""
    for f in ("/proc/net/tcp", "/proc/net/tcp6"):
        try:
            with open(f) as fh:
                next(fh)
                for l in fh:
                    c = l.split()
                    if c[3] == "0A" and int(c[1].rsplit(":", 1)[1], 16) == port:
                        return True
        except (OSError, StopIteration, ValueError, IndexError):
            pass
    return False


def srsran_ue_conf():
    """{'earfcn', 'srate'} from ue.conf; reasonable defaults otherwise."""
    conf = {"earfcn": "?", "srate": 11.52e6}
    try:
        section = ""
        with open(LTE_UE_CONF) as fh:
            for l in fh:
                l = l.strip()
                if l.startswith("["):
                    section = l
                elif l.startswith("dl_earfcn") and section == "[rat.eutra]":
                    conf["earfcn"] = l.split("=", 1)[1].strip()
                elif l.startswith("device_args"):
                    m = re.search(r"base_srate=([0-9.eE+]+)", l)
                    if m:
                        conf["srate"] = float(m.group(1))
    except (OSError, ValueError):
        pass
    return conf


class SondeLte(threading.Thread):
    """Listens for the 4G half: etat = {psd (dB, LTE_NFFT bins, centre in the
    middle), crete (max amplitude seen), trames, erreur, t}. Only runs when
    the UE port is open - otherwise nothing to listen to, and nothing to open."""
```

```
def __init__(self):
    super().__init__(daemon=True)
    self.etat = {"psd": None, "crete": 0.0, "trames": 0, "erreur": None, "t": 0.0}
    self.hann = (np.hanning(LTE_NFFT).astype(np.float32) if np is not None else None)

def run(self):
    while True:
        if np is None:
            self.etat = dict(self.etat, psd=None, erreur="numpy absent : pas de FFT 4G")
            time.sleep(10)
            continue
        if not _tcp_ecoute(LTE_PORT):
            self.etat = dict(self.etat, psd=None, trames=0, erreur=None)
            time.sleep(PERIOD)
            continue
        try:
            self.capture()
        except PermissionError:
            self.etat = dict(self.etat, psd=None, erreur="prise brute refusee : root (CAP_NET_RAW) requis")
            time.sleep(10)
        except Exception as e:
            self.etat = dict(self.etat, psd=None, erreur=f"sonde : {type(e).__name__}: {e}")
            time.sleep(2)
        time.sleep(max(0.1, PERIOD - LTE_FENETRE))

def capture(self):
    s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(0x0003))
    try:
        s.bind((LTE_IFACE, 0))
        s.settimeout(0.2)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 4 << 20)
        buf, attendu, crete_psd, crete, trames = b"", None, None, 0.0, 0
        fin = time.time() + LTE_FENETRE
        while time.time() < fin:
            try:
                pkt, adr = s.recvfrom(70000)
            except socket.timeout:
                continue
            # On lo each packet passes twice (outgoing then incoming):
            # only the incoming one is kept (pkttype 4 = PACKET_OUTGOING).
            if adr[2] == 4 or len(pkt) < 34 or pkt[12:14] != b"\x08\x00" or pkt[23] != 6:
                continue
            tcp = 14 + (pkt[14] & 0xF) * 4
            sport, _dport, seq = struct.unpack("!HHI", pkt[tcp:tcp + 8])
            if sport != LTE_PORT:
                continue
            charge = pkt[tcp + (pkt[tcp + 12] >> 4) * 4:]
            if not charge:
                continue
            if attendu is not None and seq != attendu:
                buf = b""            # gap in the stream: resynchronise
            attendu = (seq + len(charge)) & 0xFFFFFFFF
            buf += charge
            while True:
                i = buf.find(ZMTP_MARQUE)
                if i < 0:
                    buf = buf[-len(ZMTP_MARQUE):]
                    break
                taille = int.from_bytes(buf[i + 3:i + 11], "big")
                if taille % 8 or taille > (64 << 20):
                    buf = buf[i + 1:]            # false marker inside the data
                    continue
                if len(buf) < i + 11 + taille:
                    buf = buf[i:]
                    break
```

```
                    x = np.frombuffer(buf[i + 11:i + 11 + taille], dtype=np.complex64)
                    buf = buf[i + 11 + taille:]
                    n = len(x) // LTE_NFFT
                    if n == 0:
                        continue
                    trames += 1
                    blocs = x[:n * LTE_NFFT].reshape(n, LTE_NFFT) * self.hann
                    p = np.abs(np.fft.fftshift(np.fft.fft(blocs, axis=1), axes=1)) ** 2
                    p = p.max(axis=0) / (LTE_NFFT * LTE_NFFT)
                    crete_psd = p if crete_psd is None else np.maximum(crete_psd, p)
                    crete = max(crete, float(np.max(np.abs(x))))
        finally:
            s.close()
        if trames == 0:
            self.etat = dict(self.etat, psd=None, trames=0, erreur=None, t=time.time())
        else:
            self.etat = {"psd": (10 * np.log10(crete_psd + 1e-20)).tolist(), "crete": crete,
                         "trames": trames, "erreur": None, "t": time.time()}


def _queue_fichier(chemin, n):
    """The last n non-empty lines of a text file, without ANSI colours;
    [] if it is absent or empty."""
    try:
        with open(chemin, "rb") as f:
            f.seek(0, 2)
            f.seek(max(0, f.tell() - 262144))
            data = f.read().decode("utf-8", "replace")
    except OSError:
        return []
    # RRC measurements (« MEAS: New measurement serving cell ») arrive every
    # 20 ms: they would drown the attach. Only the events are kept.
    # Last line still being written: it is not shown truncated.
    if data and not data.endswith("\n"):
        data = data[:data.rfind("\n") + 1]
    # srsue writes its progress bars with \r: only the last segment counts.
    lignes = [ANSI.sub("", l.split("\r")[-1]).rstrip() for l in data.splitlines()]
    return [l for l in lignes if l.strip() and " MEAS:" not in l][-n:]


def srsran_journal(n, width):
    """The last lines of the srsue console: the « ue » window of the tmux
    carrying it if it exists, otherwise its standard output
    (osmo-lte-ue.console: PLMN found, RRC, attach) COMPLETED by its log
    (osmo-lte-ue.log: RRC/NAS at info level, see configs/srsran/ue.conf)."""
    try:
        out = subprocess.run(["tmux", "-L", LTE_TMUX, "capture-pane", "-p", "-t", f"{LTE_TMUX}:ue"],
                             capture_output=True, text=True, timeout=2)
        lignes = [ANSI.sub("", l).rstrip() for l in out.stdout.splitlines()]
        lignes = [l for l in lignes if l.strip()]
        if out.returncode == 0 and lignes:
            return [(l[:width], None) for l in lignes[-n:]]
    except Exception:
        pass
    # Console first (the essentials fit in a few lines), the log takes the
    # rest of the room: both truncated from the top, keeping the most recent.
    console = _queue_fichier(LTE_UE_CONSOLE, max(1, n // 3))
    journal = _queue_fichier(LTE_UE_LOG, n)
    if not console and not journal:
        journal = _queue_fichier("/tmp/ue.log", n)
    if console or journal:
        lignes = console + journal[-max(0, n - len(console)):] if journal else console
        return [(l[:width], None) for l in lignes[-n:]]
    return [("console srsue introuvable :", (139, 148, 158)),
            (f"  ni tmux -L {LTE_TMUX} (fenetre ue), ni " + ", ni ".join(LTE_UE_SOURCES),
             (139, 148, 158))]
```

```
historique_lte = []


def render_lte(etat):
    """The top half: the uplink spectrum of the srsRAN UE on the left, its
    console on the right. Same frame, same margins as the 2G: one panel, two
    benches."""
    img = base_moitie(HAUT).copy()
    d = ImageDraw.Draw(img)
    d.rounded_rectangle((0, 0, W - 1, HH - 1), radius=26, fill=PANEL)
    x0, y0, x1, y1 = PAD, PAD, W - PAD, HH - PAD - FOOT
    d.rounded_rectangle((x0 - 6, y0 - 6, x1 + 6, y1 + 6), radius=8, fill=PANEL)
    split = x0 + int((x1 - x0) * 0.56)
    conf = srsran_ue_conf()
    d.text((x0, y0), "Spectre I/Q du UE srsRAN  ·  UL", font=F_TITLE, fill=(88, 166, 255))
    tag = f"EARFCN {conf['earfcn']} · ZMQ {LTE_PORT}"
    d.text((split - 12 - d.textlength(tag, font=F_SMALL), y0 + 2), tag, font=F_SMALL, fill=(63, 185, 80))
    sx0, sx1 = x0, split - 12
    sw = sx1 - sx0
    py0 = y0 + 24
    psd_h = 110
    py1 = py0 + psd_h
    d.rectangle((sx0, py0, sx1, py1), fill=(12, 16, 24))
    for gy in range(1, 4):
        yy = py0 + gy * psd_h // 4
        d.line((sx0, yy, sx1, yy), fill=(24, 30, 40))
    # The axis: the sample rate, from -srate/2 to +srate/2 around the carrier.
    demi = conf["srate"] / 2e6
    for k, frac in ((f"-{demi:.2f}", 0.0), ("0", 0.5), (f"+{demi:.2f} MHz", 1.0)):
        tx = sx0 + int(frac * sw)
        tx = min(max(tx - d.textlength(k, font=F_SMALL) * frac, sx0), sx1 - d.textlength(k, font=F_SMALL))
        d.text((tx, py1 + 2), k, font=F_SMALL, fill=(90, 100, 120))
    psd = etat.get("psd")
    if etat.get("erreur"):
        d.text((sx0 + 8, py0 + 8), etat["erreur"], font=F_SMALL, fill=(248, 81, 73))
        vals = None
    elif psd is None:
        d.text((sx0 + 8, py0 + 8), "pas de flux (l eNB ne demande rien a l UE)", font=F_SMALL,
               fill=(248, 81, 73))
        vals = None
    else:
        # Scale: the top follows the peak, never below -60 dB - so the
        # silence between two bursts stays flat instead of being stretched.
        haut = max(max(psd), -60.0)
        vals = resample([min(1.0, max(0.0, (p - (haut - 50)) / 50)) for p in psd], sw)
        pts = [(sx0 + x, py1 - 1 - int(v * (psd_h - 4))) for x, v in enumerate(vals)]
        for x, y in pts:
            d.line((x, y, x, py1 - 1), fill=(30, 70, 110))
        d.line(pts, fill=(88, 210, 255), width=1)
        if etat.get("crete", 0.0) < 1e-6:
            d.text((sx0 + 8, py0 + 8), "silence : le UE n emet pas (veille)", font=F_SMALL,
                   fill=(139, 148, 158))
    historique_lte.insert(0, vals if vals is not None else [0.0] * sw)
    wy0 = py1 + 18
    wf_h = y1 - wy0
    del historique_lte[wf_h:]
    wf = Image.new("RGB", (sw, wf_h), (8, 10, 14))
    px = wf.load()
    for row, line in enumerate(historique_lte):
        for x in range(min(sw, len(line))):
            px[x, row] = palette_sdr(line[x])
    img.paste(wf, (sx0, wy0))
    d = ImageDraw.Draw(img)
    lx0 = split
```

```
    d.line((lx0 - 6, y0, lx0 - 6, y1), fill=(30, 36, 48))
    d.text((lx0, y0), "console srsUE  ·  osmo-lte-ue.console + .log", font=F_TITLE, fill=(88, 166, 255))
    line_h = 15
    n = max(1, (y1 - (y0 + 26)) // line_h)
    cols = max(10, int((x1 - lx0) / 7.3))
    for i, (l, col) in enumerate(srsran_journal(n, cols)):
        if col is None:
            col = LOG_DEFAUT
            if re.search(r"reject|fail|error|release|lost|timeout", l, re.I):
                col = TANGO_VIF["31"]
            elif re.search(r"attach|found (plmn|cell)|complete|connected|network attach", l, re.I):
                col = TANGO_VIF["32"]
        d.text((lx0, y0 + 26 + i * line_h), l, font=F_LOG, fill=col)
    return img


def write(img):
    os.makedirs(OUT, exist_ok=True)
    tmp = os.path.join(OUT, ".panel.tmp.png")
    img.save(tmp, "PNG")
    os.replace(tmp, os.path.join(OUT, "panel.png"))


def main():
    print(f"[fft-snap] {URL} + {mobile_log_path()} (2G, bas) + prise brute {LTE_IFACE}:{LTE_PORT} (4G, haut)"
          f" -> {OUT}/panel.png toutes les {PERIOD}s (fondu {FADE_S}s depuis {WALLPAPER})", flush=True)
    sonde = SondeLte()
    sonde.start()
    # One fade per half: « bas » follows the 2G (or the hub), « haut » the 4G.
    alpha = {"bas": 0.0, "haut": 0.0}
    step = PERIOD / FADE_S if FADE_S > 0 else 1.0
    last_state = {"bas": None, "haut": None}
    while True:
        t = time.time()
        op_courant = operator()
        # The hub has no /psd: querying it would never return anything and
        # the panel would stay on the strip. Its view is « ready » as soon as
        # it is selected - the matrix and its log make the content.
        if op_courant.get("MODE") == "interstp":
            data, data_ul, ready = None, None, True
        else:
            data = data_ul = None
            try:
                data = fetch("ms" if FFT_2G == "ms" else "bts")
            except Exception:
                data = None
            if FFT_2G != "ms":
                try:
                    data_ul = fetch("ms")
                except Exception:
                    data_ul = None
                # No downlink (bridge without DL FIFO): the uplink alone.
                if not (data and "psd" in data) and data_ul and "psd" in data_ul:
                    data, data_ul = data_ul, None
            ready = bool(data) and "psd" in data
        # The 4G is « there » as soon as srsue holds its ZMQ port: the
        # spectrum may be empty (no burst), but the panel then shows that
        # silence, with the UE console next to it - precisely what one wants
        # to see while waiting for the attach.
        prets = {"bas": ready, "haut": _tcp_ecoute(LTE_PORT)}
        for m, nom in (("bas", "banc 2G"), ("haut", "banc 4G (srsue)")):
            if prets[m] != last_state[m]:
                print(f"[fft-snap] {nom} {'pret : fondu vers le spectre' if prets[m] else 'absent : retour a la
                ↪  bande dessinee'}",
                      flush=True)
                last_state[m] = prets[m]
```

```
            target = 1.0 if prets[m] else 0.0
            alpha[m] = (min(target, alpha[m] + step) if target > alpha[m]
                        else max(target, alpha[m] - step))
        try:
            img = base_image().copy()
            # ALWAYS a blend, even once the fade is done: it is what lets the
            # strip show through under the bench. Without a strip behind,
            # nothing to show: full opacity.
            op = OPACITY if strip_present() else 1.0
            if alpha["bas"] <= 0.0:
                history.clear()
            else:
                live = (render_interstp(op_courant)
                        if op_courant.get("MODE") == "interstp" else render_live(data, data_ul))
                img.paste(Image.blend(base_moitie(BAS), live, alpha["bas"] * op), (0, BAS[0]))
            if alpha["haut"] <= 0.0:
                historique_lte.clear()
            else:
                live = render_lte(sonde.etat)
                img.paste(Image.blend(base_moitie(HAUT), live, alpha["haut"] * op), (0, HAUT[0]))
            write(img)
        except Exception as e:
            print(f"[fft-snap] rendu : {e}", file=sys.stderr, flush=True)
        time.sleep(max(0.2, PERIOD - (time.time() - t)))


if __name__ == "__main__":
    main()
```

2.250 /opt/GSM/osmo-operator/tools/osmo-firmwares-install.sh

23122 octets, 367 lignes → 367 lignes

```
#!/bin/bash
# ============================================================================
# osmo-firmwares-install.sh - THE PHONE FIRMWARES OUTSIDE THE ISO (supplements)
#
# Sourced by addition.sh (--gnuarm, --magnetite, --trx, --fernvale, --simcom).
# Each function is IDEMPOTENT (an already cloned tree is not re-cloned, an
# already built binary is not rebuilt without OSMO_FW_REFAIRE=1) and nothing
# is fatal: a missing mirror must not stop the other supplements.
#
#   osmo_gnuarm_install     the historical osmocom-bb toolchain: arm-elf-gcc 4.5.2,
#                           PULLED from the osmocom-bb-trx Docker image (not rebuilt)
#                           into /root/gnuarm, and the libmpfr.so.4 -> libmpfr.so.6
#                           link its cc1 requires
#   osmo_magnetite_install  FreeCalypso Magnetite for the Motorola C139 (the real
#                           TI firmware): Wine + TI TMS470 environment, pinned git
#                           sources, bench patches, 4 MiB flash image
#   osmo_calypso_fw_install the bench Calypso firmware (layer1 GPRS, rssi...)
#                           rebuilt with gnuarm from osmocom-bb eac4eed1 + patches
#   osmo_trx_install        osmocom-bb-trx: the published Docker image (host side,
#                           ubuntu 16.04) and the trx firmware (active TX) built
#                           with gnuarm
#   osmo_fernvale_install   MediaTek MT6260 (Fernvale): fernly, fernvale-nuttx,
#                           osmocom-bb-mtk-layer1, with arm-none-eabi
#   osmo_simcom_install     the SIMCom documentation and SDKs (~3.4 GB)
# ============================================================================

: "${GREEN:=}"; : "${YELLOW:=}"; : "${RED:=}"; : "${CYAN:=}"; : "${BOLD:=}"; : "${NC:=}"
: "${GSM_ROOT:=/opt/GSM}"
: "${REPO:=${DIR:-$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)}}"
: "${OSMO_FW_JOBS:=$(nproc 2>/dev/null || echo 4)}"
OSMO_FW_LOG="${OSMO_FW_LOG:-/var/log/osmo-firmwares}"
```

```
_fw_ok()   { echo -e "  ${GREEN}✓${NC} $*"; }
_fw_warn() { echo -e "  ${YELLOW}!${NC} $*"; }
_fw_err()  { echo -e "  ${RED}✗${NC} $*"; }
_fw_log()  { mkdir -p "$OSMO_FW_LOG" 2>/dev/null; echo "$OSMO_FW_LOG/$1.log"; }

# apt without questions, once per run of the supplement
_fw_apt() {
    if [ -z "${_FW_APT_UPDATE:-}" ]; then apt-get update -qq >/dev/null 2>&1 || true; _FW_APT_UPDATE=1; fi
    DEBIAN_FRONTEND=noninteractive apt-get install -y -q "$@" >>"$(_fw_log apt)" 2>&1
}

# git clone if missing; nothing otherwise (an existing work tree is not touched)
_fw_clone() {   # url dest [branche]
    local url="$1" dest="$2" br="${3:-}"
    if [ -d "$dest/.git" ]; then _fw_ok "$(basename "$dest") deja present ($dest)"; return 0; fi
    git clone -q ${br:+-b "$br"} "$url" "$dest" >>"$(_fw_log git)" 2>&1 \
        && _fw_ok "$(basename "$dest") clone ($url${br:+ $br})" \
        || { _fw_err "clone de $url impossible (voir $(_fw_log git))"; return 1; }
}

# THE osmocom-bb .map ISSUE. The .map file is requested at link time in two
# forms, both broken as soon as the link goes through the gcc driver ($(CC)):
#   "-Map board/..."       (jolly/testing, MTK): gcc reads -Map as -M -ap;
#   "-Wl,-Map board/..."   (upstream osmocom-bb, eac4eed1): -Map goes to ld but
#                            the NAME goes to gcc, as an input file: "board/
#                            compal_e88/hello_world.highram.map: No such file".
# Both become -Wl,-Map,<file> (comma). Through $(LD) directly (gnuarm on
# jolly/testing) "-Map file" is right: left untouched. The script solve_envs.py
# is Python 2 (print without parentheses): without python2 on the host, no
# environment is resolved and the firmware comes out without .bin, silently.
_fw_bb_corriger() {   # racine/src/target/firmware
    local fw="$1" mk="$1/Makefile.inc"
    if [ -f "$mk" ] && grep -q '^\s*-Wl,-Map board' "$mk"; then
        sed -i 's|^\(\s*\)-Wl,-Map board/|\1-Wl,-Map,board/|' "$mk"
        _fw_ok "Makefile.inc : -Wl,-Map <fichier> -> -Wl,-Map,<fichier>"
    fi
    if [ -f "$mk" ] && grep -q '^\s*-Map board' "$mk" \
       && grep -B2 '^\s*-Map board' "$mk" | grep -q '\$(CC)'; then
        sed -i 's|^\(\s*\)-Map board/\$(2)/\$(1)\.\$(3)\.map |\1-Wl,-Map,board/$(2)/$(1).$(3).map |' "$mk"
        _fw_ok "Makefile.inc : -Map -> -Wl,-Map,<fichier> (lien par gcc)"
    fi
    if [ -f "$fw/solve_envs.py" ] && grep -q "^\s*print ' '" "$fw/solve_envs.py"; then
        sed -i "1s|python\$|python3|; s|^\(\s*\)print ' '.join(envs)|\1print(' '.join(envs))|" "$fw/solve_envs.py"
        _fw_ok "solve_envs.py passe en Python 3"
    fi
}

# ── gnuarm ──────────────────────────────────────────────────────────────
# THE TOOLCHAIN IS NOT REBUILT: gnu-arm-installer (binutils, gcc 4.5.2,
# newlib 1.19 from sources) takes a long time and depends on mirrors that
# disappear (newlib has left sources.redhat.com). The osmocom-bb-trx image,
# already pulled for --trx, carries the same toolchain, built once and for
# all: /root/gnuarm/install. It is copied out of the image (docker create +
# docker cp, without running a container) to $GNUARM_DIR/install. Its
# binaries come from Ubuntu 16.04: on noble they only lack libmpfr.so.4, the
# link installed below. Tested: compiling and linking a .c with newlib
# (-mcpu=arm7tdmi) with arm-elf-gcc 4.5.2.
GNUARM_DIR="${GNUARM_DIR:-/root/gnuarm}"
OSMOCOM_BB_TRX_IMAGE="${OSMOCOM_BB_TRX_IMAGE:-ghcr.io/bbaranoff/osmocom-bb-trx:main}"
osmo_gnuarm_install() {
    echo -e "${BOLD}== gnuarm (arm-elf-gcc 4.5.2, image $OSMOCOM_BB_TRX_IMAGE) ==${NC}"
    if [ ! -x "$GNUARM_DIR/install/bin/arm-elf-gcc" ]; then
        if ! command -v docker >/dev/null 2>&1 || ! docker info >/dev/null 2>&1; then
           _fw_err "docker absent ou arrete : gnuarm vient de l image $OSMOCOM_BB_TRX_IMAGE (addition.sh --docker)"
            return 1
```

```
        fi
        local log c; log="$(_fw_log gnuarm)"
        docker pull "$OSMOCOM_BB_TRX_IMAGE" >>"$log" 2>&1 \
            || { _fw_err "docker pull $OSMOCOM_BB_TRX_IMAGE en echec (voir $log)"; return 1; }
        c="$(docker create "$OSMOCOM_BB_TRX_IMAGE" 2>>"$log")" \
            || { _fw_err "docker create en echec (voir $log)"; return 1; }
        mkdir -p "$GNUARM_DIR"
        rm -rf "$GNUARM_DIR/install"
        docker cp -q "$c:/root/gnuarm/install" "$GNUARM_DIR/install" >>"$log" 2>&1
        local rc=$?
        docker rm -f "$c" >/dev/null 2>&1
        [ "$rc" = 0 ] || { _fw_err "docker cp de /root/gnuarm/install en echec (voir $log)"; return 1; }
        _fw_ok "gnuarm copie depuis l image ($GNUARM_DIR/install, $(du -sh "$GNUARM_DIR/install" | cut -f1))"
    fi
    # cc1 of gcc 4.5.2 is linked to libmpfr.so.4, which noble no longer provides
    # (libmpfr.so.6, same ABI for what it uses): without this link, "cc1: error
    # while loading shared libraries: libmpfr.so.4".
    if [ ! -e /usr/local/lib/libmpfr.so.4 ]; then
        local m6; m6="$(ls /usr/lib/*-linux-gnu/libmpfr.so.6 2>/dev/null | head -1)"
        if [ -n "$m6" ]; then ln -sf "$m6" /usr/local/lib/libmpfr.so.4 && ldconfig && _fw_ok "lien libmpfr.so.4 ->
         ↪  $m6"
        else _fw_warn "libmpfr.so.6 introuvable : apt install libmpfr6"; fi
    fi
    if echo 'int main(void){return 0;}' | "$GNUARM_DIR/install/bin/arm-elf-gcc" -x c -c -o /dev/null -
     ↪  2>/dev/null; then
        _fw_ok "arm-elf-gcc $("$GNUARM_DIR/install/bin/arm-elf-gcc" -dumpversion) operationnel
         ↪  ($GNUARM_DIR/install/bin)"
    else
        _fw_err "arm-elf-gcc ne compile pas (cc1, libmpfr ?)"; return 1
    fi
    # $GNUARM_DIR/install/bin in the PATH of every login shell
    # (scripts/osmo-gnuarm-profile.sh -> /etc/profile.d/97-osmo-gnuarm.sh)
    if [ -f "${REPO:-/opt/GSM/osmo-operator}/scripts/osmo-gnuarm-profile.sh" ]; then
        install -m644 "${REPO:-/opt/GSM/osmo-operator}/scripts/osmo-gnuarm-profile.sh"
         ↪  /etc/profile.d/97-osmo-gnuarm.sh \
            && _fw_ok "PATH : $GNUARM_DIR/install/bin (/etc/profile.d/97-osmo-gnuarm.sh, nouveaux shells)"
    fi
}

# ── Magnetite (C139) ─────────────────────────────────────────────────
# NOTHING FROM freecalypso.org: its FTP caps at ~18 KB/s (52 MB of TI
# environment = 45 min). github.com/bbaranoff/fc-magnetite carries the converted
# hg history (tip 9394305d4ff5, upstream under the bench patches) and, in
# freecalypso-env/, the Wine environment, the Compal bootloader and freecalypso-tools.
FC_MAGNETITE_GIT="${FC_MAGNETITE_GIT:-https://github.com/bbaranoff/fc-magnetite}"
FC_ENV_URL="${FC_ENV_URL:-https://raw.githubusercontent.com/bbaranoff/fc-magnetite/main/freecalypso-env}"
osmo_magnetite_install() {
    echo -e "${BOLD}== FreeCalypso Magnetite (C139) : Wine + TI TMS470 ==${NC}"
    local fcm="$GSM_ROOT/fc-magnetite" fht="$GSM_ROOT/fc-host-tools" wdl="$GSM_ROOT/fc-wine-dl" \
          bdl="$GSM_ROOT/fc-dl" out="$GSM_ROOT/firmware/magnetite" log; log="$(_fw_log magnetite)"
    # 32-bit Wine: the TMS470 (cl470, asm470, lnk470) is a 32-bit Windows binary
    dpkg --print-foreign-architectures | grep -qx i386 || dpkg --add-architecture i386
    _FW_APT_UPDATE=""
    _fw_apt git wine wine64 wine32:i386 xz-utils wget build-essential \
        || { _fw_err "installation de Wine en echec (voir $(_fw_log apt))"; return 1; }
    _fw_ok "wine $(wine --version 2>/dev/null)"
    # the sources: Magnetite (the git repository, freecalypso-env/ included) and the
    # FreeCalypso host tools (rvinterf, fc-shell, fc-fsio: run.sh needs them). An old
    # hg clone stays as is: the environment is then taken from FC_ENV_URL.
    if [ ! -d "$fcm/.hg" ] && [ ! -d "$fcm/.git" ]; then
        git clone -q "$FC_MAGNETITE_GIT" "$fcm" >>"$log" 2>&1 \
            || { _fw_err "git clone fc-magnetite en echec (voir $log)"; return 1; }
        _fw_ok "fc-magnetite clone ($FC_MAGNETITE_GIT)"
    fi
    # _fc_env <file> <destination>: the copy from the clone, else the GitHub repository
    _fc_env() {
```

```
        [ -s "$2" ] && return 0
        if [ -s "$fcm/freecalypso-env/$1" ]; then cp "$fcm/freecalypso-env/$1" "$2"
        else wget -q -O "$2" "$FC_ENV_URL/$1" >>"$log" 2>&1 || { rm -f "$2"; return 1; }; fi
    }
    if [ ! -f "$fht/Makefile" ]; then
        mkdir -p "$fht" "$wdl"
        _fc_env freecalypso-tools.tar.xz "$wdl/freecalypso-tools.tar.xz" \
            && tar xJf "$wdl/freecalypso-tools.tar.xz" -C "$fht" --strip-components=1 >>"$log" 2>&1 \
            || _fw_warn "freecalypso-tools : extraction en echec"
    fi
    [ -x "$fht/rvinterf/lowlevel/rvinterf" ] || make -C "$fht" -j"$OSMO_FW_JOBS" >>"$log" 2>&1 \
        || _fw_warn "outils hote FreeCalypso : make en echec (voir $log)"
    # the bench patches (forced ARFCN, auto registration, SMS, MMI): a single file,
    # regenerated by `hg diff` in fc-magnetite (patches/fc-magnetite-banc.patch)
    local p="$REPO/patches/fc-magnetite-banc.patch"
    if [ -f "$p" ]; then
        if (cd "$fcm" && patch -p1 -R --dry-run -s -f <"$p" >/dev/null 2>&1); then _fw_ok "patchs du banc deja
         ↪  appliques"
        elif (cd "$fcm" && patch -p1 -s -f <"$p" >>"$log" 2>&1); then _fw_ok "patchs du banc appliques ($(basename
         ↪  "$p"))"
        else _fw_warn "$(basename "$p") ne s applique pas sur cet arbre (voir $log)"; fi
    fi
    # the TI environment installed (compiler, cygwin, perl, java) in ~/.wine/drive_c
    mkdir -p "$wdl" "$bdl" "$out"
    local f
    for f in installed-env.tar.xz nowhine.c winebuild.sh README; do
        _fc_env "$f" "$wdl/$f" || _fw_warn "telechargement $f en echec"
    done
    if [ ! -d "$HOME/.wine/drive_c/Dvpt_tools" ]; then
        WINEDEBUG=-all wineboot -i >>"$log" 2>&1
        tar xJf "$wdl/installed-env.tar.xz" -C "$HOME/.wine/drive_c" >>"$log" 2>&1 \
            && _fw_ok "environnement TI pose dans $HOME/.wine/drive_c" \
            || { _fw_err "environnement TI : extraction en echec"; return 1; }
        WINEDEBUG=-all wineboot -u >>"$log" 2>&1
    fi
    [ -x /usr/local/bin/nowhine ] || gcc -O2 -o /usr/local/bin/nowhine "$wdl/nowhine.c" >>"$log" 2>&1 \
        || _fw_warn "nowhine : compilation en echec"
    # the build: c139, hybrid-ui-vo. WITHOUT -j: the TMS470 under Wine writes
    # temporary files with fixed names, two parallel cl470 step on each other.
    # THE BENCH VARIANT: DWNLD 0 (FC_MAGNETITE_DWNLD), -nopatch directory. The DSP
    # patch downloaded by the TI L1 (DWNLD 1) fails under the emulation
    # (LA_LoadPage: race on LA_TIMEOUT); the bench runs on this build, checked
    # byte for byte against c139-flash.img. L1_DYN_DSP_DWNLD stays at 1 (at 0,
    # l1audio_async.c does not compile). FC_MAGNETITE_DWNLD=1: the original build.
    local dwnld="${FC_MAGNETITE_DWNLD:-0}" suf="" b
    [ "$dwnld" = 0 ] && suf="-nopatch"
    b="$fcm/build-c139-hybrid-ui-vo$suf"
    if [ ! -s "$b/fwimage.bin" ] || [ "${OSMO_FW_REFAIRE:-0}" = 1 ]; then
        ( cd "$fcm" && (cd helpers && make) && ./configure.sh --clean c139 hybrid-ui-vo ${suf:+SUFFIX=$suf} \
            && sed -i "s/^#define DWNLD [01]\$/#define DWNLD $dwnld/" "$b/config/swconfig.cfg" \
            && cd "$b" && WINEDEBUG=-all make ) \
            >>"$log" 2>&1 || { _fw_err "compilation Magnetite en echec (voir $log)"; return 1; }
    fi
    _fw_ok "fwimage.bin (DWNLD $dwnld) : $(stat -c %s "$b/fwimage.bin") octets, $b"
    # the 4 MiB flash image: FreeCalypso Compal bootloader at 0, fwimage.bin at
    # 0x10000, FFS at 0x3C0000 (IMEI, afcparams, rfcap written by fc-fsio). An
    # EXISTING image only changes its firmware area: the bench bootloader and FFS
    # stay as they are (the bootloader area of the bench image differs by 4 bytes
    # from the FreeCalypso file, at vector 0x10; the bench boots with it). The old
    # image is copied to .avant-supplement.
    _fc_env compal-flash-boot-for-fc.bin "$bdl/compal-flash-boot-for-fc.bin" \
        || _fw_warn "bootloader Compal : telechargement en echec"
    [ -f "$out/c139-flash.img" ] && cp -p "$out/c139-flash.img" "$out/c139-flash.img.avant-supplement"
    python3 - "$bdl/compal-flash-boot-for-fc.bin" "$b/fwimage.bin" "$out/c139-flash.img" <<'EOF' \
        && _fw_ok "image flash $out/c139-flash.img" || _fw_err "assemblage de l image flash en echec"
import os, sys
```

```
boot, fw, img_path = sys.argv[1:4]
f = open(fw, "rb").read()
if os.path.exists(img_path) and os.path.getsize(img_path) == 4 << 20:
    img = bytearray(open(img_path, "rb").read())              # bench bootloader + FFS kept
    img[0x10000:0x3C0000] = b"\xff" * (0x3C0000 - 0x10000)
else:
    img = bytearray(b"\xff" * (4 << 20))
    b = open(boot, "rb").read(); img[0:len(b)] = b
img[0x10000:0x10000 + len(f)] = f
open(img_path, "wb").write(img)
EOF
    echo -e "  ${CYAN}→${NC} lancer : ${BOLD}./start.sh --bts-classique --ms-magnetite --gapk${NC} (appels :
     ↪  ENCRYPTION=\"a5 0\" et --regen)"
}

# ── osmocom-bb-trx ──────────────────────────────────────────────────
osmo_trx_install() {
    echo -e "${BOLD}== osmocom-bb-trx (jolly/testing, TX) ==${NC}"
    # host side (layer23 --enable-transceiver, libosmocore 1.1.0): the published
    # image (github.com/bbaranoff/osmocom-bb-trx, ubuntu 16.04 + gcc 4.9/5 + gnuarm)
    if command -v docker >/dev/null 2>&1 && docker info >/dev/null 2>&1; then
        if docker pull "$OSMOCOM_BB_TRX_IMAGE" >>"$(_fw_log trx)" 2>&1; then _fw_ok "image $OSMOCOM_BB_TRX_IMAGE"
        else _fw_warn "docker pull $OSMOCOM_BB_TRX_IMAGE en echec (voir $(_fw_log trx))"; fi
    else
        _fw_warn "docker absent ou arrete : l image osmocom-bb-trx n est pas tiree (addition.sh --docker)"
    fi
    # the trx firmware, with the gnuarm toolchain, in a separate tree (the bench
    # one, osmocom-bb-transceiver, keeps its patches and is not touched)
    osmo_gnuarm_install || return 1
    local t="$GSM_ROOT/osmocom-bb-trx" log; log="$(_fw_log trx)"
    _fw_apt autoconf automake libtool pkg-config python3 git   # libosmocore cote cible
    _fw_clone https://gitea.osmocom.org/phone-side/osmocom-bb "$t" jolly/testing || return 1
    sed -i 's/^#CFLAGS += -DCONFIG_TX_ENABLE/CFLAGS += -DCONFIG_TX_ENABLE/' "$t/src/target/firmware/Makefile"
    _fw_bb_corriger "$t/src/target/firmware"
    if [ ! -s "$t/src/target/firmware/board/compal_e88/trx.highram.bin" ] || [ "${OSMO_FW_REFAIRE:-0}" = 1 ]; then
        ( cd "$t/src" && PATH="$GNUARM_DIR/install/bin:$PATH" make -j"$OSMO_FW_JOBS" firmware ) >>"$log" 2>&1 \
            || { _fw_err "firmware trx en echec (voir $log)"; return 1; }
    fi
    if [ -s "$t/src/target/firmware/board/compal_e88/trx.highram.bin" ]; then
        _fw_ok "firmware trx (TX) : $t/src/target/firmware/board/compal_e88/trx.{highram,e88loader,e88flash}.bin"
    else
        _fw_err "trx.highram.bin absent apres make (voir $log)"; return 1
    fi
}

# ── bench Calypso firmware (layer1 GPRS, rssi...) with gnuarm ──────
# Rebuilds the bench osmocom-bb tree (OSMOCOM_BB_REF + patches/osmocom-bb-*.patch,
# including the GPRS layer1) with arm-elf-gcc 4.5.2: "cd src && make", the whole
# tree (host tools, target libosmocore, firmware). layer1 only exists in highram:
# with the PDCH it no longer fits in the 64 KB of the Compal loader
# (target/firmware/Makefile). OSMO_FW_INSTALLER=1 installs layer1/rssi
# .highram.{elf,bin} in /opt/GSM/firmware, layer1 also under the name
# layer1.highram.gprs.* (the one dsp-rom-max.sh and egprs-dsp.sh read); the
# previous ones remain as .avant-<date>.
OSMOCOM_BB_REF="${OSMOCOM_BB_REF:-eac4eed1}"
osmo_calypso_fw_install() {
    echo -e "${BOLD}== firmware Calypso du banc (osmocom-bb, gnuarm) ==${NC}"
    osmo_gnuarm_install || return 1
    local t="$GSM_ROOT/osmocom-bb" fw log; log="$(_fw_log calypso-fw)"
    _fw_apt autoconf automake libtool pkg-config python3 git
    if [ ! -d "$t/.git" ]; then
        _fw_clone https://gitea.osmocom.org/phone-side/osmocom-bb "$t" || return 1
        git -C "$t" checkout -q "$OSMOCOM_BB_REF" && _fw_ok "osmocom-bb epingle sur $OSMOCOM_BB_REF"
    fi
    OSMO_PATCHES_ONLY=osmocom-bb GSM_ROOT="$GSM_ROOT" REPO="$REPO" bash "$REPO/tools/osmo-patches-apply.sh"
     ↪  "$GSM_ROOT" \
```

```
        || _fw_warn "un correctif osmocom-bb ne s applique pas (ci-dessus)"
    fw="$t/src/target/firmware"
    _fw_bb_corriger "$fw"
    if ( cd "$t/src" && PATH="$GNUARM_DIR/install/bin:$PATH" make -j"$OSMO_FW_JOBS" ) >>"$log" 2>&1; then
        _fw_ok "osmocom-bb : make complet dans $t/src (gnuarm)"
    else
        _fw_err "make dans $t/src en echec (voir $log)"; return 1
    fi
    local b="$fw/board/compal_e88" ok=1 a
    for a in layer1 rssi; do
        if [ -s "$b/$a.highram.bin" ]; then _fw_ok "$a.highram.bin ($b)"; else _fw_err "$a.highram.bin absent
        ↪ (voir $log)"; ok=0; fi
    done
    if [ "$ok" = 1 ] && [ "${OSMO_FW_INSTALLER:-0}" = 1 ]; then
        local dst="$GSM_ROOT/firmware/board/compal_e88" d n x; d="$(date +%Y%m%d-%H%M)"
        mkdir -p "$dst"
        for a in layer1 rssi; do
            for n in "$a.highram" $([ "$a" = layer1 ] && echo layer1.highram.gprs); do
                for x in elf bin; do
                    [ -f "$dst/$n.$x" ] && cp -p "$dst/$n.$x" "$dst/$n.$x.avant-$d"
                    cp -f "$b/$a.highram.$x" "$dst/$n.$x"
                done
            done
        done
        _fw_ok "pose dans $dst (anciens en .avant-$d)"
    elif [ "$ok" = 1 ]; then
        echo -e "  ${CYAN}→${NC} le banc garde le firmware livre ; OSMO_FW_INSTALLER=1 pour poser celui-ci dans
        ↪ $GSM_ROOT/firmware"
    fi
    [ "$ok" = 1 ]
}

# ── Fernvale (MT6260) ─────────────────────────────────────────────────
osmo_fernvale_install() {
    echo -e "${BOLD}== Fernvale (MediaTek MT6260) ==${NC}"
    local log; log="$(_fw_log fernvale)"
    _fw_apt gcc-arm-none-eabi binutils-arm-none-eabi libnewlib-arm-none-eabi build-essential \
            libusb-1.0-0-dev genromfs python3 autoconf automake libtool pkg-config git \
        || _fw_warn "paquets arm-none-eabi : installation incomplete (voir $(_fw_log apt))"
    local fl="$GSM_ROOT/fernly" nx="$GSM_ROOT/fernvale-nuttx" mtk="$GSM_ROOT/osmocom-bb-mtk-layer1"
    _fw_clone https://github.com/bbaranoff/fernly "$fl"
    _fw_clone https://github.com/bbaranoff/fernvale-nuttx "$nx"
    _fw_clone https://github.com/bbaranoff/osmocom-bb-mtk-layer1 "$mtk" mtk-layer1
    # fernly: the reverse-engineering OS and its USB loader (fernly-usb-loader, native)
    if [ -d "$fl" ]; then
        if ( cd "$fl" && make CROSS_COMPILE=arm-none-eabi- ) >>"$log" 2>&1; then
            _fw_ok "fernly : $fl/build/firmware.bin + fernly-usb-loader"
        else _fw_err "fernly : make en echec (voir $log)"; fi
        [ -f "$fl/95-fernvale-test.rules" ] && install -D -m644 "$fl/95-fernvale-test.rules"
        ↪ /etc/udev/rules.d/95-fernvale-test.rules \
            && udevadm control --reload 2>/dev/null; true
    fi
    # NuttX (kosagi-fernvale/nsh): the build fixes are already in the repository
    if [ -d "$nx/nuttx" ]; then
        if [ ! -s "$nx/nuttx/nuttx.bin" ] || [ "${OSMO_FW_REFAIRE:-0}" = 1 ]; then
            ( cd "$nx/nuttx/tools" && ./configure.sh kosagi-fernvale/nsh && cd .. \
                && make CROSSDEV=arm-none-eabi- ARCROSSDEV=arm-none-eabi- ) >>"$log" 2>&1 \
                || _fw_err "NuttX : make en echec (voir $log)"
        fi
        [ -s "$nx/nuttx/nuttx.bin" ] && _fw_ok "NuttX : $nx/nuttx/nuttx.bin"
    fi
    # osmocom-bb layer1 MTK (bringup + layer1 MT6260)
    if [ -d "$mtk/src" ]; then
        _fw_bb_corriger "$mtk/src/target/firmware"
        if ( cd "$mtk/src" && make mtk-firmware CROSS_HOST=arm-none-eabi ) >>"$log" 2>&1 \
           && [ -s "$mtk/src/target/firmware/board/fernvale/layer1.mt6260.bin" ]; then
```

```
            _fw_ok "layer1 MTK : $mtk/src/target/firmware/board/fernvale/{bringup,layer1}.mt6260.bin"
        else _fw_err "layer1 MTK : make mtk-firmware en echec (voir $log)"; fi
    fi
    echo -e "  ${CYAN}→${NC} charger : ${BOLD}$fl/build/fernly-usb-loader -s /dev/fernvale
     ↪  $fl/build/usb-loader.bin <payload.bin>${NC}"
}

# ── SIMCom ──────────────────────────────────────────────────────────────
osmo_simcom_install() {
    echo -e "${BOLD}== SIMCom : documentation et SDK ==${NC}"
    # ~3.4 GB (rar archives of the SDKs, reports): a clone without history
    local d="$GSM_ROOT/simcom"
    if [ -d "$d/.git" ]; then _fw_ok "simcom deja present ($d)"; return 0; fi
    git clone -q --depth 1 https://github.com/bbaranoff/simcom "$d" >>"$(_fw_log git)" 2>&1 \
        && _fw_ok "simcom clone dans $d ($(du -sh "$d" 2>/dev/null | cut -f1))" \
        || _fw_err "clone de bbaranoff/simcom en echec"
    _fw_apt p7zip-full unrar 2>/dev/null || true   # the SDKs ship as .rar / .7z
}
```

2.251 /opt/GSM/osmo-operator/tools/osmo-fond-antenne.py

3933 octets, 92 lignes → 92 lignes

```
#!/usr/bin/env python3
# "Bench running" variant of the wallpaper: without the ubuntu|osmocom banner and
# the NO RF sign, to leave room for the banner and Conky. Only the light pixels of
# the logos / sign are erased (the background behind them is dark); they are filled
# with a normalised blur of the dark neighbours, plus a light grain.
#
# Input: the output of tools/osmo-fond-repos.py (2560x1600); the erased areas are
# in the coordinates of that image. See the header of osmo-fond-repos.py for the
# complete chain.
import sys
import numpy as np
from PIL import Image, ImageFilter

SRC, DST = sys.argv[1], sys.argv[2]
im = np.asarray(Image.open(SRC).convert("RGB"), float)
H, W, _ = im.shape
yy, xx = np.mgrid[0:H, 0:W]


def flou(a, s):
    return np.stack([np.asarray(Image.fromarray(np.clip(a[..., c], 0, 255).astype(np.uint8))
                                .filter(ImageFilter.GaussianBlur(s)), float) for c in range(3)], -1)


def flou_m(m, s):
    return np.asarray(Image.fromarray((np.clip(m, 0, 1) * 255).astype(np.uint8))
                      .filter(ImageFilter.GaussianBlur(s)), float) / 255


def dilate(m, n):
    return np.asarray(Image.fromarray((m * 255).astype(np.uint8)).filter(ImageFilter.MaxFilter(n))) > 0


lum = im.max(axis=2)
clair = lum > 55

# zone 1: ubuntu | osmocom | GSM...LTE banner - light pixels only
zone = np.zeros((H, W), bool); zone[115:355, 175:1630] = True
trou = dilate(clair & zone, 23)
# zone 2: NO RF sign (disc of the barred circle + text + stroke) - whole area,
# the orange background behind the text cannot be told apart from the stroke
```

```
cx, cy, r = 2205, 285, 268
norf = (xx - cx) ** 2 + (yy - cy) ** 2 < r ** 2
norf[475:700, 2045:2415] = True
norf = dilate(norf, 9)

# fill: mean of the neighbouring dark pixels, at several scales
garde = (~trou).astype(float)
rempli = np.zeros_like(im); poids = np.zeros((H, W))
for s, seuil in ((6, .15), (14, .15), (30, .15), (60, .1), (120, .03), (250, 0.003)):
    num = flou(im * garde[..., None], s)
    den = flou_m(garde, s)
    ok = (den > seuil) & (poids == 0)
    rempli[ok] = num[ok] / den[ok][:, None]
    poids[ok] = 1

# NO RF sign: smooth membrane (diffusion from the edge, at 1/8), without the
# steps of the blur scales, then the brush strokes of a 690 px zone on the
# left (banner already erased, city tower avoided), faded towards the bottom
def membrane(img, trou, f=8, n=400):
    h, w = H // f, W // f
    p = np.stack([np.asarray(Image.fromarray(img[..., c].astype(np.uint8)).resize((w, h), Image.BOX), float)
                  for c in range(3)], -1)
    t = np.asarray(Image.fromarray((trou * 255).astype(np.uint8)).resize((w, h), Image.BOX)) > 0
    q = p.copy(); q[t] = p[~t].mean(axis=0)
    for _ in range(n):
        v = (np.roll(q, 1, 0) + np.roll(q, -1, 0) + np.roll(q, 1, 1) + np.roll(q, -1, 1)) / 4
        q[t] = v[t]
    return np.stack([np.asarray(Image.fromarray(np.clip(q[..., c], 0, 255).astype(np.uint8))
                                .resize((W, H), Image.BICUBIC), float) for c in range(3)], -1)

# grain: light noise, blurred by one pixel, so as not to leave a flat area
rng = np.random.default_rng(23)
bruit = rng.normal(0, 5, (H, W))
bruit = np.asarray(Image.fromarray(np.clip(bruit + 128, 0, 255).astype(np.uint8))
                   .filter(ImageFilter.GaussianBlur(0.8)), float) - 128
rempli = rempli + bruit[..., None] * np.array([1.0, 0.6, 1.0])

m = flou_m(trou.astype(float), 3)
m = np.maximum(m, trou)[..., None]
out = im * (1 - m) + rempli * m

lisse = membrane(out, norf)
don = np.roll(out, 690, axis=1)
coups = don - flou(don, 22)
attenue = np.clip((660 - yy) / 80, 0, 1)[..., None]
remp2 = lisse + 0.8 * coups * attenue + bruit[..., None] * np.array([1.0, 0.6, 1.0])
m2 = flou_m(dilate(norf, 21).astype(float), 10)
m2 = np.maximum(m2, norf)[..., None]
out = out * (1 - m2) + remp2 * m2
Image.fromarray(np.clip(out, 0, 255).astype(np.uint8)).save(DST, optimize=True)
print(DST)
```

2.252 /opt/GSM/osmo-operator/tools/osmo-fond-bleu.py

8953 octets, 163 lignes → 163 lignes

```
#!/usr/bin/env python3
# "Telecom Operator" wallpaper in blue: the orange brush strokes of the background
# turn into a dark purple -> light blue gradient (by pixel luminance).
# Kept orange / red: the ubuntu logo, OPERATOR, the barred NO RF sign and the
# phones -> BTS -> BSC -> MSC -> Core diagram.
#
# Input: the output of tools/osmo-fond-repos.py (2560x1600); the protected
# zones are in the coordinates of that image. The chain is:
```

```
#   tools/osmo-fond-repos.py configs/gsm-lab-source.png /tmp/repos.png
#   tools/osmo-fond-bleu.py /tmp/repos.png /tmp/bleu.png
#   tools/osmo-fond-antenne.py /tmp/bleu.png /tmp/antenne.png
#
# Without argument: the whole chain at once, from configs/gsm-lab-source.png
# to configs/gsm-lab-ville.jpg and configs/gsm-lab-antenne.jpg.
#
# OSMO_FOND_ORANGE_HAUT=1: the "bench running" variant (the one without the
# NO RF) mixes both, the original orange at the top fading in a gradient into
# the blue towards the bottom. The chain without argument applies it to
# gsm-lab-antenne.jpg only; gsm-lab-ville.jpg (idle) stays blue.
import os
import subprocess
import sys
import tempfile
import numpy as np
from PIL import Image, ImageDraw, ImageFilter, ImageFont

if len(sys.argv) == 1:
    outils = os.path.dirname(os.path.abspath(__file__))
    configs = os.path.join(os.path.dirname(outils), "configs")
    with tempfile.TemporaryDirectory() as tmp:
        repos, bleu, mix, antenne = (os.path.join(tmp, n) for n in ("repos.png", "bleu.png", "mix.png",
 ↪  "antenne.png"))
        for script, src, dst, env in (("osmo-fond-repos.py", os.path.join(configs, "gsm-lab-source.png"), repos,
         ↪  {}),
                                      ("osmo-fond-bleu.py", repos, bleu, {"OSMO_FOND_ORANGE_HAUT": "0"}),
                                      ("osmo-fond-bleu.py", repos, mix, {"OSMO_FOND_ORANGE_HAUT": "1"}),
                                      ("osmo-fond-antenne.py", mix, antenne, {})):
            subprocess.run([sys.executable, os.path.join(outils, script), src, dst], check=True,
                           env={**os.environ, **env})
        for src, nom in ((bleu, "gsm-lab-ville.jpg"), (antenne, "gsm-lab-antenne.jpg")):
            Image.open(src).convert("RGB").save(os.path.join(configs, nom), quality=92)
            print(os.path.join(configs, nom))
    sys.exit(0)
if len(sys.argv) != 3:
    sys.exit("usage : osmo-fond-bleu.py [ENTREE.png SORTIE.png]")

SRC, DST = sys.argv[1], sys.argv[2]
im = Image.open(SRC).convert("RGB")
W, H = im.size
k = W / 2560
a = np.asarray(im, float) / 255
r, g, b = a[..., 0], a[..., 1], a[..., 2]
yy, xx = np.mgrid[0:H, 0:W]

# hue (-180..180 degrees) and saturation
mx, mn = a.max(2), a.min(2)
c = mx - mn
h = np.zeros_like(mx)
nz = c > 1e-6
i = nz & (mx == r); h[i] = ((g - b)[i] / c[i]) % 6
i = nz & (mx == g) & (mx != r); h[i] = (b - r)[i] / c[i] + 2
i = nz & (mx == b) & (mx != r) & (mx != g); h[i] = (r - g)[i] / c[i] + 4
h = h * 60
h = np.where(h > 180, h - 360, h)
sat = np.where(mx > 0, c / np.maximum(mx, 1e-6), 0)

# "orange" weight: from burgundy (-70) to orange-yellow (60), saturated pixels
w = np.clip((60 - h) / 15, 0, 1) * np.clip((h + 70) / 30, 0, 1) * np.clip((sat - .25) / .25, 0, 1)

# protected zones: only the vivid pixels (logo, letters, strokes) are kept there,
# the darker brush strokes behind turn blue like the rest
P = np.zeros((H, W))
for x0, y0, x1, y1 in [(192, 128, 380, 310),      # logo ubuntu
                       (200, 470, 1060, 720),     # OPERATOR
```

```
                      (2045, 475, 2415, 700),    # NO RF + trait
                      (1320, 1075, 1660, 1305),  # schema : telephones + BTS
                      (1660, 1130, 2440, 1335)]: # schema : BSC / MSC / Core (pas le reflet au-dessus)
    P[int(y0 * k):int(y1 * k), int(x0 * k):int(x1 * k)] = 1
P[(xx - 2205 * k) ** 2 + (yy - 285 * k) ** 2 < (268 * k) ** 2] = 1   # cercle barré
vif = ((mx > .62) & (sat > .5)).astype(np.uint8) * 255
vif = np.asarray(Image.fromarray(vif).filter(ImageFilter.MaxFilter(5)), float) / 255
P = np.asarray(Image.fromarray((P * vif * 255).astype(np.uint8))
               .filter(ImageFilter.GaussianBlur(2 * k)), float) / 255
w = w * (1 - P)
if os.environ.get("OSMO_FOND_ORANGE_HAUT", "0") == "1":
        # pure orange on the first fifth, smoothstep fade down to 3/5: blue below
    u = np.clip((yy / H - 0.2) / 0.4, 0, 1)
    w = w * (u * u * (3 - 2 * u))

# navy -> cyan gradient, following the luminance
# (a lighter navy and some cyan rather than dark purple -> light blue)
L = 0.3 * r + 0.59 * g + 0.11 * b
t = np.clip(L / 0.6, 0, 1)[..., None]
MARINE, CYAN = np.array([35, 70, 140]) / 255, np.array([80, 225, 255]) / 255
neuf = (MARINE * (1 - t) + CYAN * t) * (0.55 + 0.45 * np.clip(mx / 0.9, 0, 1))[..., None] * 1.15
out = a * (1 - w[..., None]) + np.clip(neuf, 0, 1) * w[..., None]

# TITLE IN UBUNTU SANS. "TELECOM OPERATOR" is brush painted in the source: the
# letters (white or orange, inside the title frame, dilated to take their dark
# outline) are erased by filling with a wider and wider normalised convolution
# (no OpenCV on the bench), then rewritten lower in the default Ubuntu font.
X0, Y0, X1, Y1 = (round(v * k) for v in (160, 375, 1065, 708))
L = 0.3 * out[..., 0] + 0.59 * out[..., 1] + 0.11 * out[..., 2]
omx, omn = out.max(2), out.min(2)
osat = np.where(omx > 0, (omx - omn) / np.maximum(omx, 1e-6), 0)
lettre = np.zeros((H, W), bool)
lettre[Y0:Y1, X0:X1] = (((L > .38) & (osat < .45)) |
                        ((out[..., 0] > .45) & (out[..., 0] > out[..., 1] * 1.3) & (osat > .4)))[Y0:Y1, X0:X1]
M = np.asarray(Image.fromarray((lettre * 255).astype(np.uint8))
               .filter(ImageFilter.MaxFilter(9)).filter(ImageFilter.MaxFilter(5))) > 127
connu = (~M).astype(float)
for rayon in (2, 4, 8, 16, 32, 64):
    num = np.asarray(Image.fromarray((np.clip(out * connu[..., None], 0, 1) * 255).astype(np.uint8))
                     .filter(ImageFilter.GaussianBlur(rayon * k)), float) / 255
    den = np.asarray(Image.fromarray((connu * 255).astype(np.uint8))
                     .filter(ImageFilter.GaussianBlur(rayon * k)), float) / 255
    sel = M & (den > .02) & (connu < .5)
    out[sel] = (num / np.maximum(den[..., None], 1e-3))[sel]
    connu[sel] = 1

# a bit lighter: gamma on the whole background (darks rise more than lights)
out = np.clip(out, 0, 1) ** float(os.environ.get("OSMO_FOND_GAMMA", "0.82"))
img = Image.fromarray((np.clip(out, 0, 1) * 255).astype(np.uint8))
TITRE_FONTE = "/usr/share/fonts/truetype/ubuntu/UbuntuSans[wdth,wght].ttf"
ft = ImageFont.truetype(TITRE_FONTE, round(150 * k))
ft.set_variation_by_axes([100, 800])
# colours measured on the painted letters: TELECOM off-white, OPERATOR orange gradient
def titre_degrade(xy, texte, haut, bas):
    masque = Image.new("L", img.size, 0)
    ImageDraw.Draw(masque).text(xy, texte, font=ft, fill=255, anchor="lt")
    y0_, y1_ = masque.getbbox()[1], masque.getbbox()[3]
    tt = np.clip((np.arange(img.size[1]) - y0_) / max(1, y1_ - y0_), 0, 1)[:, None, None]
    couleur = np.array(haut) * (1 - tt) + np.array(bas) * tt
    couche = Image.fromarray(np.broadcast_to(couleur, (img.size[1], img.size[0], 3)).astype(np.uint8))
    img.paste(couche, (0, 0), masque)
titre_degrade((round(200 * k), round(400 * k)), "TELECOM", (249, 247, 245), (248, 246, 244))
titre_degrade((round(200 * k), round(545 * k)), "OPERATOR", (248, 112, 2), (247, 99, 2))

# frame around the diagram: everything runs locally, no radio emission. The
# title is placed in a gap of the top stroke (like a fieldset legend).
```

```
ORANGE = (255, 140, 40)
SANS = "/usr/share/fonts/truetype/ubuntu/Ubuntu[wdth,wght].ttf"
fonte = ImageFont.truetype(SANS, round(30 * k))
fonte.set_variation_by_axes([100, 500])
titre = "localhost  ·  no radio emission"
x0, y0, x1, y1 = (round(v * k) for v in (1300, 1035, 2465, 1345))
calque = Image.new("RGBA", img.size, (0, 0, 0, 0))
d = ImageDraw.Draw(calque)
d.rounded_rectangle((x0, y0, x1, y1), radius=round(18 * k),
                    outline=ORANGE + (235,), width=max(2, round(3 * k)))
tl = d.textlength(titre, font=fonte)
tx = x0 + round(40 * k)
d.rectangle((tx - round(14 * k), y0 - round(6 * k), tx + tl + round(14 * k), y0 + round(6 * k)), fill=(0, 0, 0, 0))
img = img.convert("RGBA")
# the gap must show the background: the stroke under the title is erased
trou = Image.new("L", img.size, 255)
ImageDraw.Draw(trou).rectangle((tx - round(14 * k), y0 - round(6 * k), tx + tl + round(14 * k), y0 + round(6 *
 ↪  k)), fill=0)
# the laptop corner is IN FRONT of the frame: no stroke on its shell
ImageDraw.Draw(trou).polygon([(round(x * k), round(y * k)) for x, y in
                              ((1150, 1312), (1482, 1356), (1505, 1385), (1150, 1395))], fill=0)
calque.putalpha(Image.fromarray(np.minimum(np.asarray(calque.getchannel("A")), np.asarray(trou))))
img.alpha_composite(calque)
ImageDraw.Draw(img).text((tx, y0), titre, font=fonte, fill=ORANGE + (255,), anchor="lm")
img.convert("RGB").save(DST, optimize=True)
print(DST)
```

2.253 /opt/GSM/osmo-operator/tools/osmo-fond-repos.py

5717 octets, 121 lignes → 121 lignes

```
#!/usr/bin/env python3
# "Telecom Operator" wallpaper: upscaled to 2560x1600, terminal and GSM line redrawn sharp.
#
# Source image: configs/gsm-lab-source.png (1586x992, AI generated: invented
# terminal text, "GSM | ... | MSC" line). Regenerate the idle wallpaper, then
# its "bench running" variant:
#   tools/osmo-fond-repos.py configs/gsm-lab-source.png /tmp/repos.png
#   tools/osmo-fond-bleu.py /tmp/repos.png /tmp/bleu.png     (orange -> blue background)
#   tools/osmo-fond-antenne.py /tmp/bleu.png /tmp/antenne.png
# and store both as JPEG (quality 92) under configs/gsm-lab-ville.jpg and
# configs/gsm-lab-antenne.jpg (the names read by tools/osmo-fond.py).
# The coordinates (laptop screen, GSM line) are those of THIS source image.
import sys
import numpy as np
from PIL import Image, ImageDraw, ImageFont, ImageFilter

SRC, DST = sys.argv[1], sys.argv[2]
W, H = 2560, 1600
MONO = "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf"
MONO_B = "/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf"
SANS = "/usr/share/fonts/truetype/ubuntu/Ubuntu[wdth,wght].ttf"

src = Image.open(SRC).convert("RGB")
sx, sy = W / src.width, H / src.height
up = src.resize((W, H), Image.LANCZOS).filter(ImageFilter.UnsharpMask(2, 60, 2))


def coeffs(dst_pts, src_pts):
    """PERSPECTIVE coefficients: output point (dst) -> input point (src)."""
    A, b = [], []
    for (x, y), (u, v) in zip(dst_pts, src_pts):
        A.append([x, y, 1, 0, 0, 0, -u * x, -u * y]); b.append(u)
        A.append([0, 0, 0, x, y, 1, -v * x, -v * y]); b.append(v)
```

```
    return np.linalg.solve(np.array(A, float), np.array(b, float)).tolist()


def efface(img, masque, sigma):
    """Fills the masked pixels with a normalised blur of the neighbouring pixels."""
    a = np.asarray(img, float)
    m = (np.asarray(masque) > 0)
    garde = Image.fromarray(((~m) * 255).astype(np.uint8))
    num = np.stack([np.asarray(Image.fromarray((a[..., c] * ~m).astype(np.uint8))
                               .filter(ImageFilter.GaussianBlur(sigma)), float) for c in range(3)], -1)
    den = np.asarray(garde.filter(ImageFilter.GaussianBlur(sigma)), float)[..., None] / 255
    rempli = num / np.maximum(den, 1e-3)
    a[m] = rempli[m]
    return Image.fromarray(np.clip(a, 0, 255).astype(np.uint8))


# ── laptop terminal ──────────────────────────────────────────────────
Q = [(184 * sx, 507 * sy), (642 * sx, 521 * sy), (682 * sx, 793 * sy), (218 * sx, 806 * sy)]
FW, FH = 1480, 960
F = [(0, 0), (FW, 0), (FW, FH), (0, FH)]

plat = up.transform((FW, FH), Image.PERSPECTIVE, coeffs(F, Q), Image.BICUBIC)
lum = np.asarray(plat.convert("L"))
masque = (lum > 38).astype(np.uint8) * 255
masque[:80] = 0                                      # garde la barre de titre
masque = Image.fromarray(masque).filter(ImageFilter.MaxFilter(13))
plat = efface(plat, masque, 18)

VERT, BLANC, GRIS = (138, 226, 52), (225, 225, 225), (150, 150, 160)
lignes = [
    [("ubuntu@osmocom:~$ ", VERT, 1), ("systemctl status osmo-bsc", BLANC, 0)],
    [("● ", VERT, 0), ("osmo-bsc.service - Osmocom BSC", BLANC, 0)],
    [("     Loaded: loaded (/lib/systemd/system/osmo-bsc.service; enabled)", BLANC, 0)],
    [("     Active: ", BLANC, 0), ("active (running)", VERT, 1)],
    [("   Main PID: 2483 (osmo-bsc)", BLANC, 0)],
    [("      Tasks: 12 (limit: 2300)", BLANC, 0)],
    [("     Memory: 42.6M", BLANC, 0)],
    [("     CGroup: /system.slice/osmo-bsc.service", BLANC, 0)],
    [("             └─2483 /usr/bin/osmo-bsc", BLANC, 0)],
    [],
    [("ubuntu@osmocom:~$ ", VERT, 1), ("systemctl is-active osmo-bts-virtual", BLANC, 0),
     ("  # NO RF", GRIS, 0)],
    [("active", VERT, 1)],
    [],
    [("ubuntu@osmocom:~$ ", VERT, 1)],
]
fonte = {0: ImageFont.truetype(MONO, 31), 1: ImageFont.truetype(MONO_B, 31)}
calque = Image.new("RGBA", (FW, FH), (0, 0, 0, 0))
d = ImageDraw.Draw(calque)
y = 108
for morceaux in lignes:
    x = 58
    for texte, coul, gras in morceaux:
        d.text((x, y), texte, font=fonte[gras], fill=coul + (255,))
        x += d.textlength(texte, font=fonte[gras])
    if morceaux and morceaux[0][0].startswith("ubuntu@") and len(morceaux) == 1:
        d.rectangle([x + 2, y + 2, x + 18, y + 36], fill=(150, 230, 190, 255))   # curseur
    y += 50
# light halo, like a lit screen
halo = calque.filter(ImageFilter.GaussianBlur(4))
plat = plat.convert("RGBA")
plat.alpha_composite(Image.eval(halo, lambda v: v // 3))
plat.alpha_composite(calque)

retour = coeffs(Q, F)
colle = plat.transform((W, H), Image.PERSPECTIVE, retour, Image.BICUBIC)
```

```
forme = Image.new("L", (FW, FH), 255).transform((W, H), Image.PERSPECTIVE, retour, Image.BICUBIC)
up.paste(colle.convert("RGB"), (0, 0), forme)

# ── "GSM | GPRS | EDGE | LTE" line under the osmocom logo ──────────────────
bx0, by0, bx1, by1 = int(615 * sx), int(176 * sy), int(995 * sx), int(208 * sy)
zone = up.crop((bx0, by0, bx1, by1))
mz = Image.fromarray(((np.asarray(zone.convert("L")) > 42) *
 ↪  255).astype(np.uint8)).filter(ImageFilter.MaxFilter(11))
up.paste(efface(zone, mz, 10), (bx0, by0))

ff = ImageFont.truetype(SANS, 33)
ff.set_variation_by_axes([100, 400])
texte = "GSM   |   GPRS   |   EDGE   |   LTE"
d = ImageDraw.Draw(up)
espace = 4                                          # interlettrage comme l original
largeur = sum(d.textlength(c, font=ff) + espace for c in texte) - espace
x, cy = (bx0 + bx1) / 2 - largeur / 2, (by0 + by1) / 2
for c in texte:
    d.text((x, cy), c, font=ff, fill=(215, 215, 220), anchor="lm")
    x += d.textlength(c, font=ff) + espace

up.save(DST, optimize=True)
print(DST, up.size)
```

2.254 /opt/GSM/osmo-operator/tools/osmo-fond.py

9833 octets, 248 lignes → 248 lignes

```
#!/usr/bin/env python3
# osmo-fond.py - THE BENCH WALLPAPER, IN TWO STATES, WITH A FADE BETWEEN THEM.
#
#   osmo-fond ville        the idle desktop: the city by the water, without
#                          antenna (configs/gsm-lab-ville.jpg)
#   osmo-fond antenne      a bench is running: the SAME city, the 2G/4G mast
#                          raised in front, the icons going up
#                          (configs/gsm-lab-antenne.jpg)
#   osmo-fond etat         prints the current state (ville / antenne / autre)
#   osmo-fond <file>       any image, same fade
#
#   --duree S     length of the fade (default 2.4 s); --sec: no fade
#   --pas N       number of intermediate images (default 12)
#
# WHY. Bench choice: idle, the city alone; when the 2G (standalone run) or
# the 4G (multi run) is STARTED, the antenna rises in the landscape - with a
# fade, not at once - and it goes away when everything stops. Two callers:
#   - tools/osmo-launcher.py, on the CLICK on a run (immediately, even before
#     the service is up);
#   - tools/osmo-panel.py, which already knows when a bench goes up or down
#     (montrer(actif)): covers a launch from a terminal, and the return to
#     the city on stop.
#
# Both may follow each other closely: the tool is IDEMPOTENT (already on the
# target: nothing), and only one fade at a time per session (lock).
#
# HOW. GNOME Shell itself does a short cross-fade (~1 s) on every change of
# picture-uri. It is relied upon and stretched: Pillow renders N intermediate
# images (linear blend of the two wallpapers) in
# $XDG_RUNTIME_DIR/osmo-fond/, pushed one by one through gsettings at a
# regular interval; the last value set is the REAL file
# (/usr/share/backgrounds/gsm-lab-*.jpg), not a cache image. The images are
# kept (tmpfs, a few MB): the reverse fade reads them backwards.
#
# USER. gsettings talks to the SESSION bus: the tool must run as the desktop
# user. Started as root (start-docker.sh, a service), it re-executes itself
```

```
# under the account that has a gnome-shell and a /run/user/<uid>/bus.
import argparse
import fcntl
import hashlib
import os
import pwd
import signal
import subprocess
import sys
import time
from urllib.parse import quote, unquote, urlparse

REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
BG_DIR = "/usr/share/backgrounds"
# Name -> file installed by the install (80-chroot.sh / 80-bureau.sh), else the repo.
ETATS = {
    "ville":   ("gsm-lab-ville.jpg",   "le bureau au repos"),
    "antenne": ("gsm-lab-antenne.jpg", "un banc tourne"),
}
SCHEMA = "org.gnome.desktop.background"
CLES = ("picture-uri", "picture-uri-dark")


def dire(msg):
    print("[fond] " + msg, flush=True)


def fichier_etat(nom):
    f = ETATS[nom][0]
    for d in (BG_DIR, os.path.join(REPO, "configs")):
        p = os.path.join(d, f)
        if os.path.isfile(p):
            return p
    return None


# ── run as the desktop user ──────────────────────────────────────────
def _session_bus(uid):
    p = "/run/user/%d/bus" % uid
    return "unix:path=" + p if os.path.exists(p) else None


def se_relancer_en_utilisateur():
    """Root without a session bus: find the desktop and re-execute under it."""
    if os.geteuid() != 0:
        return
    if os.environ.get("DBUS_SESSION_BUS_ADDRESS") and _session_bus(0):
        return                      # the desktop belongs to root (the live key)
    candidats = []
    su = os.environ.get("SUDO_USER")
    if su:
        candidats.append(su)
    try:
        out = subprocess.run(["ps", "-eo", "user:32=,comm="], capture_output=True, text=True).stdout
        candidats += [l.split()[0] for l in out.splitlines() if l.strip().endswith("gnome-shell")]
    except OSError:
        pass
    for nom in candidats:
        try:
            pw = pwd.getpwnam(nom)
        except KeyError:
            continue
        bus = _session_bus(pw.pw_uid)
        if not bus:
            continue
        if pw.pw_uid == 0:
```

```
            os.environ["DBUS_SESSION_BUS_ADDRESS"] = bus
            os.environ.setdefault("XDG_RUNTIME_DIR", "/run/user/0")
            return
        env = {"DBUS_SESSION_BUS_ADDRESS": bus, "XDG_RUNTIME_DIR": "/run/user/%d" % pw.pw_uid,
               "HOME": pw.pw_dir, "USER": pw.pw_name, "LOGNAME": pw.pw_name,
               "PATH": os.environ.get("PATH", "/usr/local/bin:/usr/bin:/bin"),
               "OSMO_REPO": REPO, "OSMO_FOND_RELANCE": "1"}
        os.execvp("runuser", ["runuser", "-u", pw.pw_name, "--", "env"] +
                  ["%s=%s" % kv for kv in env.items()] +
                  [sys.executable, os.path.abspath(__file__)] + sys.argv[1:])
    dire("aucun bureau GNOME trouve : rien a faire")
    sys.exit(0)


# ── gsettings ──────────────────────────────────────────────────────
def gs_get(cle):
    try:
        v = subprocess.run(["gsettings", "get", SCHEMA, cle], capture_output=True, text=True,
 ↪  timeout=5).stdout.strip()
    except (OSError, subprocess.TimeoutExpired):
        return ""
    return v.strip("'")


def gs_set(uri):
    for cle in CLES:
        subprocess.run(["gsettings", "set", SCHEMA, cle, uri], timeout=5)


def uri_de(path):
    return "file://" + quote(os.path.abspath(path))


def chemin_de(uri):
    if not uri.startswith("file://"):
        return None
    return unquote(urlparse(uri).path)


def etat_courant():
    """(state, path): ville / antenne / "<target> en cours" (image of an
    interrupted fade - an osmo-fond killed in flight, cf. start-native.sh
    killall_python) / autre."""
    p = chemin_de(gs_get("picture-uri")) or ""
    for nom, (f, _) in ETATS.items():
        if os.path.basename(p) == f:
            return nom, p
    for nom in ETATS:
        if "/osmo-fond/" in p and os.path.basename(p).startswith(nom + "-"):
            return nom + " en cours", p
    return "autre", p


# ── the fade ───────────────────────────────────────────────────────
def _cache_dir():
    base = os.environ.get("XDG_RUNTIME_DIR") or "/run/user/%d" % os.getuid()
    if not os.path.isdir(base):
        base = "/tmp"
    d = os.path.join(base, "osmo-fond")
    os.makedirs(d, mode=0o700, exist_ok=True)
    return d


def rendre_etapes(src, dst, pas, cible):
    """The intermediate images src -> dst (without both ends); None if impossible."""
    try:
```

```
        from PIL import Image
    except ImportError:
        return None
    try:
        a = Image.open(src).convert("RGB")
        b = Image.open(dst).convert("RGB")
    except OSError:
        return None
    if a.size != b.size:
        # bring the source to the target format, "zoomed" like GNOME
        W, H = b.size
        r = max(W / a.width, H / a.height)
        a = a.resize((max(1, round(a.width * r)), max(1, round(a.height * r))), Image.LANCZOS)
        l, t = (a.width - W) // 2, (a.height - H) // 2
        a = a.crop((l, t, l + W, t + H))
    key = hashlib.sha1(("%s:%s:%s:%s:%d" % (src, os.path.getmtime(src), dst, os.path.getmtime(dst),
↪   pas)).encode()).hexdigest()[:10]
    d = _cache_dir()
    etapes = []
    for i in range(1, pas):
        p = os.path.join(d, "%s-%s-%02d.jpg" % (cible, key, i))
        if not os.path.isfile(p):
            Image.blend(a, b, i / pas).save(p, quality=88)
        etapes.append(p)
    return etapes


def poser(cible, duree, pas, sec):
    nom = cible if cible in ETATS else "autre"
    dst = fichier_etat(cible) if cible in ETATS else os.path.abspath(cible)
    if not dst or not os.path.isfile(dst):
        dire("image introuvable : %s" % (dst or cible))
        return 1
    etat, src = etat_courant()
    if (etat == nom and nom != "autre") or (src and os.path.abspath(src) == dst):
        dire("deja « %s » : rien a faire" % nom)
        return 0
    # an interrupted fade: restart from the image where it stopped (src)
    # only one fade at a time per session
    verrou = open(os.path.join(_cache_dir(), "verrou"), "w")
    try:
        fcntl.flock(verrou, fcntl.LOCK_EX | fcntl.LOCK_NB)
    except OSError:
        dire("un fondu est deja en cours")
        return 0
    etapes = None
    if not sec and duree > 0 and pas > 1 and src and os.path.isfile(src):
        etapes = rendre_etapes(src, dst, pas, nom)
    try:
        if etapes:
            dire("fondu %s -> %s (%d images, %.1f s)" % (etat, nom, len(etapes), duree))
            dt = duree / (len(etapes) + 1)
            for p in etapes:
                gs_set(uri_de(p))
                time.sleep(dt)
        else:
            dire("%s -> %s (fondu GNOME seul)" % (etat, nom))
    finally:
        # whatever happens (error, SIGTERM), the REAL file is set
        gs_set(uri_de(dst))
    return 0


def main():
    ap = argparse.ArgumentParser(description="fond d ecran du banc : ville / antenne, en fondu")
    ap.add_argument("cible", help="ville | antenne | etat | <fichier image>")
```

```
    ap.add_argument("--duree", type=float, default=2.4, help="duree du fondu en secondes (defaut 2.4)")
    ap.add_argument("--pas", type=int, default=12, help="images intermediaires (defaut 12)")
    ap.add_argument("--sec", action="store_true", help="sans fondu")
    a = ap.parse_args()
    se_relancer_en_utilisateur()
    # SIGTERM during the fade: the finally of poser() sets the target
    signal.signal(signal.SIGTERM, lambda *_: sys.exit(143))
    if a.cible == "etat":
        etat, p = etat_courant()
        print("%s\t%s" % (etat, p))
        return 0
    return poser(a.cible, a.duree, a.pas, a.sec)


if __name__ == "__main__":
    sys.exit(main())
```

2.255 /opt/GSM/osmo-operator/tools/osmo-gcc-alternatives.sh

1326 octets, 24 lignes → 24 lignes

```
#!/bin/bash
# osmo-gcc-alternatives.sh - gcc-9 / gcc-11 / gcc-13 in update-alternatives (g++ as slave).
#
# The experimental osmocom-bb branches (jolly/testing, burst_ind) only build with gcc-9
# (see Dockerfile, stage bb). This script is THE definition, called by
# iso_modules/80-chroot.sh (in the chroot, after apt), install_modules/80-bureau.sh and
# update.sh.
#
# Priorities: gcc-9 (90), gcc-11 (110), gcc-13 (130). A choice already made
# (update-alternatives --config gcc) is kept; on creation, gcc-13 - the kernel compiler,
# for DKMS. Only the compilers present are registered: no apt, no network here.
[ "$(id -u)" -eq 0 ] || { echo "osmo-gcc-alternatives : il faut root" >&2; exit 1; }
command -v update-alternatives >/dev/null 2>&1 || exit 0
neuf=0
update-alternatives --list gcc >/dev/null 2>&1 || neuf=1
for v in 9 11 13; do
    [ -x "/usr/bin/gcc-$v" ] && [ -x "/usr/bin/g++-$v" ] || continue
    update-alternatives --list gcc 2>/dev/null | grep -qx "/usr/bin/gcc-$v" && continue
    update-alternatives --install /usr/bin/gcc gcc "/usr/bin/gcc-$v" "$(( v * 10 ))" \
        --slave /usr/bin/g++ g++ "/usr/bin/g++-$v" >/dev/null 2>&1 \
        && echo "  [outils] alternative gcc-$v posee"
done
[ "$neuf" = 1 ] && [ -x /usr/bin/gcc-13 ] && update-alternatives --set gcc /usr/bin/gcc-13 >/dev/null 2>&1
exit 0
```

2.256 /opt/GSM/osmo-operator/tools/osmo-glava.sh

4133 octets, 78 lignes → 78 lignes

```
#!/bin/bash
# osmo-glava - start the NCS Spectrum visualizer (GLava) on the desktop.
#
# Installed by tools/glava-build.sh in /usr/local/bin, started by the user
# service glava.service (systemctl --user stop|start glava). One GLava NCS per
# user.
#
# The window is a square of 26 % of the screen height, placed above the city
# of the wallpaper (gsm-lab-ville.jpg): x = 49 %, y = 21 %. On 2560x1600 that
# is 420 px at (1250, 330). Computed here from the primary screen rather than
# fixed in rc.glsl: the same ISO runs on different screens.
#
```

```
# MICROPHONE AND SPEAKERS. GLava reads only ONE PulseAudio source. One is built
# for it: a null sink "osmo_glava", into which two module-loopback copy the
# monitor of the default output (what the speakers play) and the default
# source (the microphone - on the bench, osmo_mic_ec, the echo-cancelled
# microphone). GLava listens to osmo_glava.monitor. The microphone goes ONLY
# into this null sink, never to the speakers: no feedback. The default devices
# are read at startup (login).
# The guard only targets the NCS instance (-r setgeometry...): another instance
# (glava -m bars, for example) does not prevent it from starting.
#
MIX=osmo_glava
# --retirer-mix: called when the service stops (ExecStopPost of glava.service),
# removes the two loopbacks then the null sink (in that order: the sink takes
# its loopbacks with it, a second removal would fail).
if [ "${1:-}" = "--retirer-mix" ]; then
    pactl list short modules 2>/dev/null \
        | awk -v m="$MIX" '$0 ~ ("sink(_name)?=" m "([[:space:]]|$)") {print $1}' \
        | sort -rn | while read -r id; do pactl unload-module "$id"; done
    exit 0
fi
pgrep -u "$(id -u)" -f '^glava -r setgeometry' >/dev/null && exit 0
sleep "${OSMO_GLAVA_DELAI:-6}"      # comme conky : le bureau GNOME doit etre peint

if command -v pactl >/dev/null 2>&1; then
    if ! pactl list short sinks 2>/dev/null | grep -q "^[0-9]*[[:space:]]$MIX[[:space:]]"; then
        pactl load-module module-null-sink sink_name=$MIX \
            sink_properties=device.description=GLava-micro+HP >/dev/null
        # The speakers = the monitor of the HARDWARE output. @DEFAULT_MONITOR@
        # is not enough: at login the default sink may be osmo_hp_ec (the bench
        # echo cancellation, 8 kHz mono, GSM voice only) and GLava would see
        # neither the browser nor anything else. The ALSA sink receives
        # everything, osmo_hp_ec included (its master).
        hp=$(pactl get-default-sink 2>/dev/null)
        case "$hp" in alsa_output.*|bluez_sink.*) ;;
            *) hp=$(pactl list short sinks 2>/dev/null | awk '$2 ~ /^(alsa_output|bluez_sink)\./ {print $2;
             ↪  exit}') ;;
        esac
        hp=${hp:+$hp.monitor}
        pactl load-module module-loopback source="${hp:-@DEFAULT_MONITOR@}" sink=$MIX \
            source_dont_move=true sink_dont_move=true latency_msec=30 >/dev/null
        pactl load-module module-loopback source=@DEFAULT_SOURCE@ sink=$MIX \
            source_dont_move=true sink_dont_move=true latency_msec=30 >/dev/null
    fi
    SOURCE=(-r "setsource \"$MIX.monitor\"")
else
    SOURCE=()
fi

# THE BOTTOM BAND: a second instance, bars module, over the whole free width
# (right of the dock: _NET_WORKAREA) and 8.75 % of the height (140 px on 1600),
# stuck to the bottom edge. Same source as the circle.
lancer_barres() {   # W H
    pgrep -u "$(id -u)" -f '^glava -m bars' >/dev/null && return 0
    local zone x0 h
    zone=$(xprop -root _NET_WORKAREA 2>/dev/null | sed -n 's/.*= \([0-9]*\), .*/\1/p')
    x0=${zone:-0}
    h=$(( $2 * 875 / 10000 ))
    setsid -f glava -m bars -r "setgeometry $x0 $(( $2 - h )) $(( $1 - x0 )) $h" "${SOURCE[@]}" \
        >/dev/null 2>&1
}

ecran=$(xrandr --current 2>/dev/null | sed -n 's/.* connected primary \([0-9]*\)x\([0-9]*\)+.*/\1 \2/p' | head -1)
[ -n "$ecran" ] || ecran=$(xrandr --current 2>/dev/null | sed -n 's/.*current \([0-9]*\) x \([0-9]*\),.*/\1 \2/p'
 ↪  | head -1)
read -r W H <<< "${ecran:-1920 1080}"
cote=$(( H * 26 / 100 ))
```

```
lancer_barres "$W" "$H"
exec glava -r "setgeometry $(( W * 49 / 100 )) $(( H * 21 / 100 )) $cote $cote" "${SOURCE[@]}" "$@"
```

2.257 /opt/GSM/osmo-operator/tools/osmo-gtk-run.py

24665 octets, 574 lignes → 574 lignes

```
#!/usr/bin/env python3
# osmo-gtk-run.py - ONE COMMAND, ONE GTK WINDOW: the terminal becomes a log.
#
#     osmo-gtk-run.py [--title T] [--icon FILE] [--root] [--tty] [--progress]
#                     [--keep-open] [--cwd DIR] -- COMMAND [ARGS...]
#
# NO FAVOURITE OPENS A TERMINAL. The dock and desktop icons (supplements,
# update, Claude, deka, pmOS phone, 4G, drivers, the multi log, the tmux
# console, the VTY of the mobile...) all go through HERE: a GTK window of the
# bench (same look as the launcher bar, tools/osmo-launcher.py), which shows
# the output of the command, says when it is finished and with which status,
# and closes.
#
# THE END OF THE SCRIPT CLOSES THE WINDOW. A finished favourite has nothing
# more to show: exit 0, the window leaves after 1.5 s; on failure, the console
# is unfolded and a 15 s countdown lets the error be read ("garder ouverte"
# stops it). --keep-open: the window stays, for what is opened TO READ
# ("Etat 4G", "Etat du telephone"). --close-on-exit is accepted and changes
# nothing (it is the default).
#
# TWO WAYS TO SHOW THE COMMAND:
#   - log (default): stdout + stderr of the command in a text view (ANSI
#     colours stripped, automatic scrolling, "copier"). For anything that
#     only writes: journalctl -f, a status, a service being started.
#   - --tty: a REAL terminal (VTE widget) in the window. For what asks
#     questions (whiptail, sudo, "Entree pour continuer"), animates the line
#     (update.sh tests `[ -t 1 ]` and exits without tty), or is interactive
#     end to end (claude, tmux attach, telnet). It is the terminal, but INSIDE
#     the GTK, with the same status band; without gir1.2-vte-2.91, falls back
#     to the log and says so.
#
# --progress: THE PROGRESS BAR, NOT THE SHELL. For the launches (2G, multi,
#   4G, pmOS phone) a scrolling shell is not wanted: a small window with a
#   bar, the name of the current step, the last line - and a "console"
#   button that UNFOLDS the complete output (a real VTE terminal, or the log
#   without VTE) for whoever wants to see. The command runs in a
#   pseudo-terminal held here: it believes it is in a terminal (colours,
#   questions, pkexec), what it writes is read on the fly for the bar and
#   passed to the console. The progress comes from the "[n/m]" markers the
#   bench scripts print (run.sh: [3/10] Core Osmocom; osmo-lte, osmo-pmos);
#   without marker, the bar pulses and shows the last line. End: full bar
#   (green) or red with the status.
#
# --root: the command runs as root. Already root: nothing to do. Otherwise
# pkexec (the desktop password window) with DISPLAY, XAUTHORITY and the proxy
# variables of the session - pkexec cleans the environment, and without this
# forwarding the git clones of the supplements fail behind a proxy. Without
# pkexec: sudo (which asks in the terminal with --tty, and cannot ask in log
# mode: sudo -n, the failure is read in the window).
#
# Started by the .desktop of the repository (data/desktop/), the launchers
# installed by iso_modules/85-installeur-bureau.sh, addition.sh, update.sh,
# and by the GTK bars of the desktop (osmo-launcher.py, osmo-panel.py) for
# their interactive entries.
import argparse
import fcntl
import os
```

```
import re
import shlex
import shutil
import signal
import struct
import subprocess
import sys
import termios
import threading


def _root_prefix(tty):
    """argv prefix to execute as root: nothing if already root, pkexec with
    the useful environment otherwise, sudo as a last resort."""
    if os.geteuid() == 0:
        return []
    if shutil.which("pkexec") and os.environ.get("DISPLAY"):
        env = ["DISPLAY=" + os.environ.get("DISPLAY", ""),
               "XAUTHORITY=" + os.environ.get("XAUTHORITY", "")]
        for v in ("http_proxy", "https_proxy", "ftp_proxy", "no_proxy",
                  "HTTP_PROXY", "HTTPS_PROXY", "FTP_PROXY", "NO_PROXY"):
            if os.environ.get(v):
                env.append(v + "=" + os.environ[v])
        if not tty:
            env += ["NO_COLOR=1", "TERM=dumb"]
        return ["pkexec", "env"] + env
    if shutil.which("sudo"):
        return ["sudo", "-E"] if tty else ["sudo", "-n", "-E"]
    return []


def _parse():
    ap = argparse.ArgumentParser(description="montre une commande dans une fenetre GTK")
    ap.add_argument("--title", default="osmo-operator")
    ap.add_argument("--icon", default=None, help="fichier d icone (PNG/SVG)")
    ap.add_argument("--root", action="store_true", help="en root (pkexec, ou sudo)")
    ap.add_argument("--tty", action="store_true", help="un terminal (VTE) plutot qu un journal")
    ap.add_argument("--progress", action="store_true",
                    help="une barre de progression, la console repliee derriere un bouton")
    ap.add_argument("--close-on-exit", action="store_true", help="(defaut, garde pour les anciens .desktop)")
    ap.add_argument("--keep-open", action="store_true", help="la fenetre reste ouverte a la fin de la commande")
    ap.add_argument("--cwd", default=None)
    ap.add_argument("--width", type=int, default=980)
    ap.add_argument("--height", type=int, default=600)
    ap.add_argument("cmd", nargs=argparse.REMAINDER)
    a = ap.parse_args()
    if a.cmd and a.cmd[0] == "--":
        a.cmd = a.cmd[1:]
    if not a.cmd:
        ap.error("aucune commande (osmo-gtk-run.py --title T -- commande args)")
    return a


ARGS = _parse()


def _exec_direct(pourquoi):
    # No desktop, or no GTK: executed as is, in the current terminal - better
    # the command without window than no command.
    print("[osmo-gtk-run] %s : execution directe" % pourquoi, file=sys.stderr)
    argv = (_root_prefix(True) if ARGS.root else []) + ARGS.cmd
    if ARGS.cwd:
        os.chdir(ARGS.cwd)
    os.execvp(argv[0], argv)
```

```
if not os.environ.get("DISPLAY") and not os.environ.get("WAYLAND_DISPLAY"):
    _exec_direct("pas de bureau (DISPLAY)")

try:
    import gi
    gi.require_version("Gtk", "3.0")
    gi.require_version("Gdk", "3.0")
    from gi.repository import Gdk, GLib, Gtk  # noqa: E402
except (ImportError, ValueError) as _e:
    _exec_direct("GTK indisponible (%s)" % _e)

try:
    gi.require_version("Vte", "2.91")
    from gi.repository import Vte  # noqa: E402
except (ValueError, ImportError):
    Vte = None

ANSI = re.compile(r"\x1b\[[0-9;?]*[A-Za-z]|\x1b\][^\x07]*\x07|\r")
MAX_CHARS = 400_000          # beyond that, the beginning is cut: the log is not an archive

CSS = b"""
.osmo-run { background-color: #0b0e14; }
.osmo-run label.title { color: #58a6ff; font: bold 10pt "Ubuntu"; }
.osmo-run label.state { color: #8b949e; font: 9pt "Ubuntu"; }
.osmo-run label.state.ok { color: #3fb950; }
.osmo-run label.state.ko { color: #f85149; }
.osmo-run button { background: rgba(22,27,34,0.9); color: #e6edf3;
                   border: 1px solid rgba(88,166,255,0.3); border-radius: 6px;
                   padding: 2px 10px; font: 9pt "Ubuntu"; min-height: 0; }
.osmo-run button:hover { border-color: #58a6ff; }
.osmo-run textview, .osmo-run textview text { background-color: #0b0e14; color: #d8e0ea;
                   font-family: "Ubuntu Mono"; font-size: 10pt; }
.osmo-run .cmd { color: #6e7681; font: 8pt "Ubuntu Mono"; }
.osmo-run progressbar trough { background: rgba(22,27,34,0.95); border: 1px solid rgba(88,166,255,0.25);
                   border-radius: 6px; min-height: 14px; }
.osmo-run progressbar progress { background: linear-gradient(90deg, #58a6ff, #3fe08a); border-radius: 6px;
                   min-height: 14px; }
.osmo-run progressbar.ko progress { background: #f85149; }
.osmo-run progressbar text { color: #e6edf3; font: bold 9pt "Ubuntu"; }
.osmo-run label.etape { color: #e6edf3; font: 10pt "Ubuntu"; }
.osmo-run label.derniere { color: #8b949e; font: 9pt "Ubuntu Mono"; }
"""
ETAPE = re.compile(r"\[\s*(\d+)\s*/\s*(\d+)\s*\]")
COMPACT = (640, 190)


class Run(Gtk.Window):
    def __init__(self, a):
        super().__init__(title=a.title)
        self.a = a
        self.proc = None            # log: Popen
        self.child_pid = None       # tty: pid of the VTE child
        self.fini = False
        self.progress = bool(a.progress)
        self.master = None          # progress: the pty of the command
        self.fraction = None        # progress: None as long as no [n/m] marker
        self._lbuf = ""
        self.set_default_size(*(COMPACT if self.progress else (a.width, a.height)))
        if a.icon and os.path.exists(a.icon):
            try:
                self.set_icon_from_file(a.icon)
            except Exception:
                pass
        self.connect("destroy", self._quit)
        prov = Gtk.CssProvider()
        prov.load_from_data(CSS)
```

```
Gtk.StyleContext.add_provider_for_screen(Gdk.Screen.get_default(), prov,
                                         Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION)
col = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=4)
col.get_style_context().add_class("osmo-run")
col.set_border_width(6)

head = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=8)
t = Gtk.Label(label=a.title)
t.get_style_context().add_class("title")
self.state = Gtk.Label(label="en cours ...")
self.state.get_style_context().add_class("state")
head.pack_start(t, False, False, 0)
head.pack_start(self.state, False, False, 0)
self.b_close = Gtk.Button(label="fermer")
self.b_close.connect("clicked", lambda *_x: self.destroy())
head.pack_end(self.b_close, False, False, 0)
self.rebours = None         # seconds before closing, None = no countdown
self.b_garder = Gtk.Button(label="garder ouverte")
self.b_garder.set_tooltip_text("annule la fermeture automatique")
self.b_garder.connect("clicked", lambda *_x: self._garder())
head.pack_end(self.b_garder, False, False, 0)
if not a.tty:
    b_copy = Gtk.Button(label="copier")
    b_copy.set_tooltip_text("copie tout le journal dans le presse-papier")
    b_copy.connect("clicked", self._copier)
    head.pack_end(b_copy, False, False, 0)
self.b_stop = Gtk.Button(label="■ stop")
self.b_stop.set_tooltip_text("envoie SIGTERM a la commande")
self.b_stop.connect("clicked", lambda *_x: self.stop())
head.pack_end(self.b_stop, False, False, 0)
col.pack_start(head, False, False, 0)

cmdl = Gtk.Label(label="$ " + " ".join(shlex.quote(c) for c in a.cmd))
cmdl.get_style_context().add_class("cmd")
cmdl.set_halign(Gtk.Align.START)
cmdl.set_ellipsize(3)       # Pango.EllipsizeMode.END
col.pack_start(cmdl, False, False, 0)

# progress: the bar, the step, the last line, then the console
# FOLDED (Gtk.Revealer) behind the "console" button
if self.progress:
    self.bar = Gtk.ProgressBar()
    self.bar.set_show_text(True)
    self.bar.set_text("demarrage...")
    col.pack_start(self.bar, False, False, 2)
    self.l_etape = Gtk.Label(label="")
    self.l_etape.get_style_context().add_class("etape")
    self.l_etape.set_halign(Gtk.Align.START)
    self.l_etape.set_ellipsize(3)
    col.pack_start(self.l_etape, False, False, 0)
    self.l_der = Gtk.Label(label="")
    self.l_der.get_style_context().add_class("derniere")
    self.l_der.set_halign(Gtk.Align.START)
    self.l_der.set_ellipsize(3)
    col.pack_start(self.l_der, False, False, 0)
    self.b_console = Gtk.ToggleButton(label="▾ console")
    self.b_console.set_tooltip_text("montre / cache la sortie complete de la commande")
    self.b_console.set_halign(Gtk.Align.START)
    self.b_console.connect("toggled", self._toggle_console)
    col.pack_start(self.b_console, False, False, 0)
    self.rev = Gtk.Revealer()
    self.rev.set_transition_type(Gtk.RevealerTransitionType.SLIDE_DOWN)
    self.rev.set_reveal_child(False)
    col.pack_start(self.rev, True, True, 0)
    GLib.timeout_add(120, self._pulse)
```

```
    # in progress mode, the VTE terminal is the console as soon as it exists
    # (the command is in a pty anyway); otherwise the log
    self.tty = bool((a.tty or self.progress) and Vte is not None)
    if self.tty:
        self.term = Vte.Terminal()
        self.term.set_scrollback_lines(20000)
        self.term.set_scroll_on_output(False)
        self.term.set_scroll_on_keystroke(True)
        try:
            from gi.repository import Pango
            self.term.set_font(Pango.FontDescription("Ubuntu Mono 10"))
        except Exception:
            pass
        fg, bg = Gdk.RGBA(), Gdk.RGBA()
        fg.parse("#d8e0ea")
        bg.parse("#0b0e14")
        self.term.set_colors(fg, bg, None)
        if self.progress:
            # what is typed in the console goes to the pty of the command
            self.term.connect("commit", self._tty_commit)
            self.term.connect("size-allocate", self._tty_resize)
        else:
            self.term.connect("child-exited", self._tty_exit)
        sc = Gtk.ScrolledWindow()
        sc.set_policy(Gtk.PolicyType.NEVER, Gtk.PolicyType.AUTOMATIC)
        sc.add(self.term)
        (self.rev if self.progress else col).add(sc) if self.progress else col.pack_start(sc, True, True, 0)
    else:
        sc = Gtk.ScrolledWindow()
        sc.set_policy(Gtk.PolicyType.AUTOMATIC, Gtk.PolicyType.AUTOMATIC)
        self.tv = Gtk.TextView()
        self.tv.set_editable(False)
        self.tv.set_cursor_visible(False)
        self.tv.set_monospace(True)
        self.tv.set_wrap_mode(Gtk.WrapMode.WORD_CHAR)
        self.tv.set_left_margin(6)
        self.tv.set_right_margin(6)
        sc.add(self.tv)
        self.sc = sc
        if self.progress:
            self.rev.add(sc)
        else:
            col.pack_start(sc, True, True, 0)
    self.add(col)
    self.show_all()
    self.b_garder.hide()
    if a.tty and Vte is None:
        self._append("[osmo-gtk-run] VTE (gir1.2-vte-2.91) absent : journal au lieu du terminal, "
                     "les questions de la commande ne pourront pas etre repondues.\n")
    GLib.idle_add(self._start)

# ── start ──────────────────────────────────────────────────────────
def _argv(self):
    return (_root_prefix(self.tty) if self.a.root else []) + list(self.a.cmd)

def _start(self):
    argv = self._argv()
    cwd = self.a.cwd or None
    if self.progress:
        return self._start_pty(argv, cwd)
    if self.tty:
        env = dict(os.environ)
        env.setdefault("TERM", "xterm-256color")
        envv = ["%s=%s" % kv for kv in env.items()]
        try:
            self.term.spawn_async(Vte.PtyFlags.DEFAULT, cwd, argv, envv,
```

```
                                GLib.SpawnFlags.DEFAULT, None, None, -1, None,
                                self._tty_spawned)
        except Exception as e:
            self.term.feed(("lancement impossible : %s\r\n" % e).encode())
            self._done(127)
        self.term.grab_focus()
        return False
    env = dict(os.environ, NO_COLOR="1", TERM="dumb")
    try:
        self.proc = subprocess.Popen(argv, cwd=cwd, stdin=subprocess.DEVNULL,
                                     stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
                                     start_new_session=True, env=env)
    except OSError as e:
        self._append("lancement impossible : %s\n" % e)
        self._done(127)
        return False
    threading.Thread(target=self._pump, daemon=True).start()
    return False

# ── progress: the command in a pty held here ──────────────────────────
def _start_pty(self, argv, cwd):
    env = dict(os.environ)
    env.setdefault("TERM", "xterm-256color")
    try:
        master, slave = os.openpty()
        fcntl.ioctl(master, termios.TIOCSWINSZ, struct.pack("HHHH", 40, 120, 0, 0))

        def enfant():
            # its own group + the pty as controlling terminal
            os.setsid()
            try:
                fcntl.ioctl(0, termios.TIOCSCTTY, 0)
            except OSError:
                pass
        self.proc = subprocess.Popen(argv, cwd=cwd, stdin=slave, stdout=slave, stderr=slave,
                                     preexec_fn=enfant, env=env, close_fds=True)
        os.close(slave)
        self.master = master
    except OSError as e:
        self._append("lancement impossible : %s\n" % e)
        self._done(127)
        return False
    threading.Thread(target=self._pump_pty, daemon=True).start()
    return False

def _pump_pty(self):
    while True:
        try:
            data = os.read(self.master, 4096)
        except OSError:            # EIO: the command closed its side
            break
        if not data:
            break
        GLib.idle_add(self._feed_pty, data)
    rc = self.proc.wait()
    GLib.idle_add(self._done, rc)

def _feed_pty(self, data):
    text = data.decode("utf-8", "replace")
    if self.tty:
        self.term.feed(data)       # VTE reads the ANSI itself
    else:
        self._append(text)
    # the progress markers, line by line
    self._lbuf += ANSI.sub("", text)
    while "\n" in self._lbuf:
```

```
        line, self._lbuf = self._lbuf.split("\n", 1)
        self._ligne(line)
    return False

def _ligne(self, line):
    line = line.strip()
    if not line:
        return
    m = ETAPE.search(line)
    if m:
        n, t = int(m.group(1)), int(m.group(2))
        if t > 0:
            self.fraction = min(1.0, max(0.0, n / t))
            self.bar.set_fraction(self.fraction)
            self.bar.set_text("%d / %d" % (n, t))
        etape = (line[:m.start()] + line[m.end():]).strip(" =:-·.")
        self.l_etape.set_text(etape or line)
    else:
        self.l_der.set_text(line[:200])

def _pulse(self):
    if self.fini:
        return False
    if self.fraction is None:
        self.bar.pulse()
    return True

def _tty_commit(self, _term, text, _size):
    if self.master is not None:
        try:
            os.write(self.master, text.encode("utf-8", "replace"))
        except OSError:
            pass

def _tty_resize(self, term, _alloc):
    if self.master is None:
        return
    try:
        fcntl.ioctl(self.master, termios.TIOCSWINSZ,
                    struct.pack("HHHH", max(1, term.get_row_count()), max(1, term.get_column_count()), 0, 0))
    except OSError:
        pass

def _toggle_console(self, b):
    on = b.get_active()
    b.set_label("▴ console" if on else "▾ console")
    self.rev.set_reveal_child(on)
    if on:
        self.resize(self.a.width, self.a.height)
        if self.tty:
            self.term.grab_focus()
    else:
        GLib.timeout_add(300, lambda: (self.resize(*COMPACT), False)[1])

def _tty_spawned(self, _term, pid, error, *_a):
    if error is not None or not pid or pid < 0:
        self.term.feed(("lancement impossible : %s\r\n" % error).encode())
        self._done(127)
        return
    self.child_pid = pid

def _tty_exit(self, _term, status):
    # status: waitpid format
    rc = (status >> 8) & 0xff if (status & 0xff) == 0 else 128 + (status & 0x7f)
    self._done(rc)
```

```
def _pump(self):
    try:
        for chunk in iter(lambda: self.proc.stdout.read1(4096), b""):
            GLib.idle_add(self._append, chunk.decode("utf-8", "replace"))
    except Exception:
        pass
    rc = self.proc.wait()
    GLib.idle_add(self._done, rc)

# ── show ────────────────────────────────────────────────────────
def _append(self, text):
    if self.tty:
        self.term.feed(text.replace("\n", "\r\n").encode())
        return False
    buf = self.tv.get_buffer()
    adj = self.sc.get_vadjustment()
    en_bas = adj.get_value() + adj.get_page_size() >= adj.get_upper() - 40
    buf.insert(buf.get_end_iter(), ANSI.sub("", text))
    n = buf.get_char_count()
    if n > MAX_CHARS:
        buf.delete(buf.get_start_iter(), buf.get_iter_at_offset(n - MAX_CHARS))
    if en_bas:
        self.tv.scroll_to_mark(buf.get_insert(), 0.0, True, 0.0, 1.0)
    return False

def _done(self, rc):
    if self.fini:
        return False
    self.fini = True
    ctx = self.state.get_style_context()
    if rc == 0:
        self.state.set_text("termine")
        ctx.add_class("ok")
    else:
        self.state.set_text("termine (statut %d)" % rc)
        ctx.add_class("ko")
    if self.progress:
        if self._lbuf.strip():
            self._ligne(self._lbuf)
        if rc == 0:
            self.bar.set_fraction(1.0)
            self.bar.set_text("termine")
        else:
            self.bar.set_fraction(self.fraction if self.fraction is not None else 1.0)
            self.bar.set_text("echec (statut %d)" % rc)
            self.bar.get_style_context().add_class("ko")
            # the failure is read in the console: unfold it
            if not self.b_console.get_active():
                self.b_console.set_active(True)
        if self.master is not None:
            try:
                os.close(self.master)
            except OSError:
                pass
            self.master = None
    self.b_stop.set_sensitive(False)
    if not self.a.keep_open:
        if rc == 0:
            GLib.timeout_add(1500, lambda: (self.destroy(), False)[1])
        else:
            self.rebours = 15
            self.b_garder.show()
            self._tic()
            GLib.timeout_add(1000, self._tic)
    return False
```

```
    def _tic(self):
        """Failure: countdown in the band, then closing."""
        if self.rebours is None:
            return False
        if self.rebours <= 0:
            self.destroy()
            return False
        base = self.state.get_text().split(" - ")[0]
        self.state.set_text("%s - fermeture dans %d s" % (base, self.rebours))
        self.rebours -= 1
        return True

    def _garder(self):
        self.rebours = None
        self.state.set_text(self.state.get_text().split(" - ")[0])
        self.b_garder.hide()

    def _copier(self, *_a):
        buf = self.tv.get_buffer()
        texte = buf.get_text(buf.get_start_iter(), buf.get_end_iter(), False)
        Gtk.Clipboard.get(Gdk.SELECTION_CLIPBOARD).set_text(texte, -1)

    # ── stop ──────────────────────────────────────────────────────────
    def stop(self):
        if self.tty and not self.progress:
            if self.child_pid:
                try:
                    os.killpg(os.getpgid(self.child_pid), signal.SIGTERM)
                except OSError:
                    try:
                        os.kill(self.child_pid, signal.SIGTERM)
                    except OSError:
                        pass
            return
        if self.proc is not None and self.proc.poll() is None:
            try:
                os.killpg(self.proc.pid, signal.SIGTERM)
            except OSError:
                pass

    def _quit(self, *_a):
        # The window leaves: the command too. An orphan journalctl -f or tmux
        # attach has no reason to outlive what was being watched.
        self.stop()
        Gtk.main_quit()


def main():
    Run(ARGS)
    GLib.unix_signal_add(GLib.PRIORITY_DEFAULT, signal.SIGTERM, lambda *_x: (Gtk.main_quit(), False)[1])
    Gtk.main()


if __name__ == "__main__":
    main()
```

2.258 /opt/GSM/osmo-operator/tools/osmo-info

958 octets, 19 lignes → 19 lignes

```
#!/bin/bash
# osmo-info - ouvre la page d aide du banc (le « ? » du dock).
#
# [2026-10-23] PAR HTTP, PAS PAR file://. Firefox est un snap : son confinement
```

```
# ne lit pas /usr/share, et l ancien Exec=xdg-open file:///usr/share/... ouvrait
# une page « fichier introuvable ». Le dashboard (8080) sert la page ; la copie
# de /usr/share reste en repli si le dashboard ne repond pas, recopiee alors
# dans ~/Documents (le snap ne lit pas les dossiers caches du home).
set -u
URL="${OSMO_INFO_URL:-http://127.0.0.1:8080/info.html}"
PORT="${URL##*:}"; PORT="${PORT%%/*}"
if (exec 3<>"/dev/tcp/127.0.0.1/${PORT}") 2>/dev/null; then
    exec 3>&- 2>/dev/null
    exec xdg-open "$URL"
fi
echo "Dashboard injoignable sur $PORT - repli sur une copie locale" >&2
D="$(xdg-user-dir DOCUMENTS 2>/dev/null || echo "$HOME/Documents")/osmo-operator"
mkdir -p "$D" && cp -f /usr/share/osmo-operator/info.html "$D/help.html" 2>/dev/null
exec xdg-open "file://$D/help.html"
```

2.259 /opt/GSM/osmo-operator/tools/osmo-launcher.py

28092 octets, 614 lignes → 614 lignes

```
#!/usr/bin/env python3
# osmo-launcher.py - THE BOTTOM LAUNCHER BAR, clickable, by category.
#
# Same window mechanics as osmo-panel.py / osmo-dino.py: a window of DESKTOP
# type (Gdk.WindowTypeHint.DESKTOP - under the application windows, above the
# wallpaper, transparent, sticky, undecorated), placed at the very BOTTOM of
# the screen. It renders, as TRANSPARENT RECTANGLES grouped by category,
# everything that is started from the desktop of the bench:
#
#   Banc      : run standalone · run multi · run deka · Dashboard · tmux · VTY
#   Jeux      : Doom (freedoom) · Quake (OpenArena) · Dino
#   Media     : Kodi · YouTube
#   Telephone : Linphone
#   Outils    : Wireshark (root)
#
# Each application has THREE gestures:
#   - click on the name -> starts the application FITTED ON THE FFT FRAME
#                          (normal window moved/resized on the panel, via wmctrl);
#   - button  □  -> starts in FULL SCREEN;
#   - button  □  -> starts and SENDS TO THE TOP (LAB GSM zone): writes a
#                          command that osmo-topzone.py reads to fade the
#                          banner and host the window.
#
# ── THE "run" ENTRIES DO NOT OPEN A TERMINAL ──────────────────────────────
# run standalone / run multi / run deka run IN THE BACKGROUND, without
# window: their output goes to a file ($XDG_RUNTIME_DIR/osmo-launcher/<name>.log),
# and the bar UNFOLDS ("▴ journal" button, or automatically on the click on
# a run) to show that log, live, above the buttons. One button per run
# already started switches from one to the other; "▾" folds. Privileges go
# through pkexec (desktop password window), as launch.sh does for the
# actions without terminal.
#
# Placement is done by spotting the NEW window that appeared after the
# launch (diff of the wmctrl ids): robust even for applications that fork
# (firefox, kodi). Started by /usr/local/bin/osmo-desktop-panel.
import os
import re
import shlex
import shutil
import signal
import subprocess
import sys
import time
```

```
# SAME CONSTRAINT AS osmo-panel.py / osmo-dino.py (see their notes): under
# Wayland, GTK3 ignores move() and the DESKTOP type, and once on XWayland the
# pair DESKTOP type + "always below" makes the GNOME Shell desktop layer eat
# the clicks. X11 backend, normal window.
X11_FORCE = os.environ.get("XDG_SESSION_TYPE") == "wayland" and bool(os.environ.get("DISPLAY"))
if X11_FORCE:
    os.environ["GDK_BACKEND"] = "x11"

import gi

gi.require_version("Gtk", "3.0")
gi.require_version("Gdk", "3.0")
from gi.repository import Gdk, GLib, Gtk  # noqa: E402

TYPE_HINT = Gdk.WindowTypeHint.NORMAL if X11_FORCE else Gdk.WindowTypeHint.DESKTOP
KEEP_BELOW = (not X11_FORCE) or os.environ.get("OSMO_DESKTOP_BELOW") == "1"

FW, FH = 1920, 1080
# The FFT panel (tools/wallpaper-render.py): where an application started
# "in the panel" is fitted. Same box as osmo-panel.py.
FFT_BOX = (510, 220, 900, 790)
# The top LAB GSM zone (card of the wallpaper): the target of "send to the top".
TOP_BOX = (320, 60, 1110, 500)

RUN = os.environ.get("XDG_RUNTIME_DIR") or "/run/user/%d" % os.getuid()
RUN = RUN if os.path.isdir(RUN) else "/tmp"
PID_FILE = os.path.join(RUN, "osmo-launcher.pid")
# The channel to osmo-topzone.py: one line "HOST <winid>" or "HOST FS <winid>".
TOPZONE_CMD = os.path.join(RUN, "osmo-topzone.cmd")
# The logs of the "run" entries: one file per action, overwritten at each launch.
LOG_DIR = os.path.join(RUN, "osmo-launcher")
LOG_H = 300                     # height of the unfolded log, in px
LOG_W_MAX = 1240
ANSI = re.compile(r"\x1b\[[0-9;?]*[A-Za-z]|\x1b\][^\x07]*\x07|\r")

REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
INFO_PAGE = "file:///usr/share/osmo-operator/info.html"
if not os.path.exists("/usr/share/osmo-operator/info.html"):
    INFO_PAGE = "file://" + os.path.join(REPO, "configs/info.html")

CSS = b"""
.osmo-launch { background: none; background-color: transparent; border: none;
               box-shadow: none; padding: 0; margin: 0; }
/* [2026-10-11] TRANSPARENCE TOTALE : les cadres de categorie n ont plus de fond ;
   textes et boutons portent une ombre pour rester lisibles sur le paysage. */
.osmo-cat { background: rgba(0,0,0,0); border: 1px solid rgba(239,230,210,0.18);
            border-radius: 10px; padding: 4px 8px; margin: 0 4px; }
.osmo-cat > label.title { color: #c9d4ee; font: 8pt "Ubuntu"; text-shadow: 0 1px 2px rgba(0,0,0,0.9); }
.osmo-app button { background: rgba(22,27,34,0.35); color: #e6edf3; text-shadow: 0 1px 2px rgba(0,0,0,0.9);
                   border: 1px solid rgba(88,166,255,0.25); border-radius: 6px;
                   padding: 1px 8px; font: 9pt "Ubuntu"; min-height: 0; }
.osmo-app button:hover { background: rgba(33,38,45,0.85); border-color: #58a6ff; }
.osmo-app button.mini { padding: 1px 5px; color: #a9b7de; font: 8pt "Ubuntu"; }
.osmo-app button.journal { color: #58a6ff; }
.osmo-help { background: #d62828; color: #ffffff; border: 2px solid #ff6b6b; border-radius: 999px;
             font: bold 22pt "Ubuntu"; min-width: 46px; min-height: 46px; padding: 0; margin: 0 6px; }
.osmo-help:hover { background: #ef3b3b; }
.osmo-log { background: rgba(10,12,18,0.93); border: 1px solid rgba(88,166,255,0.35);
            border-radius: 10px; padding: 6px 8px; margin: 0 0 6px 0; }
.osmo-log label.title { color: #58a6ff; font: bold 9pt "Ubuntu"; }
.osmo-log label.state { color: #8b949e; font: 8pt "Ubuntu"; }
.osmo-log button { background: rgba(22,27,34,0.7); color: #e6edf3;
                   border: 1px solid rgba(88,166,255,0.25); border-radius: 6px;
                   padding: 0 7px; font: 8pt "Ubuntu"; min-height: 0; }
.osmo-log button.actif { border-color: #58a6ff; color: #58a6ff; }
.osmo-log textview, .osmo-log textview text { background-color: transparent; color: #d8e0ea;
```

```
                font-family: "Ubuntu Mono"; font-size: 9pt; }
"""


# ── THE SCREEN GEOMETRY (like osmo-panel.py) ──────────────────────────
def screen_geom():
    disp = Gdk.Display.get_default()
    mon = disp.get_primary_monitor() or disp.get_monitor(0)
    g = mon.get_geometry()
    sw, sh = g.width, g.height
    try:
        from gi.repository import Gio
        mode = Gio.Settings.new("org.gnome.desktop.background").get_string("picture-options")
    except Exception:
        mode = "zoom"
    if mode == "stretched":
        sx, sy, ox, oy = sw / FW, sh / FH, 0, 0
    else:
        s = min(sw / FW, sh / FH) if mode == "scaled" else max(sw / FW, sh / FH)
        sx = sy = s
        ox, oy = (FW * s - sw) / 2, (FH * s - sh) / 2
    return (g.x, g.y, sw, sh), (sx, sy, ox, oy)


def box_to_screen(box):
    (gx, gy, sw, sh), (sx, sy, ox, oy) = screen_geom()
    bx, by, bw, bh = box
    return (gx + int(bx * sx - ox), gy + int(by * sy - oy), int(bw * sx), int(bh * sy))


# ── wmctrl: PLACE THE NEW WINDOW ──────────────────────────────────────
def _wmctrl_ids():
    try:
        out = subprocess.run(["wmctrl", "-l"], capture_output=True, text=True, timeout=3).stdout
        return {ln.split()[0] for ln in out.splitlines() if ln.strip()}
    except Exception:
        return set()


def _place(winid, geom):
    x, y, w, h = geom
    subprocess.run(["wmctrl", "-i", "-r", winid, "-b",
                    "remove,maximized_vert,maximized_horz,fullscreen"], timeout=3)
    subprocess.run(["wmctrl", "-i", "-r", winid, "-e", f"0,{x},{y},{w},{h}"], timeout=3)


def _fullscreen(winid):
    subprocess.run(["wmctrl", "-i", "-r", winid, "-b", "add,fullscreen"], timeout=3)


def spawn(argv):
    return subprocess.Popen(argv, stdin=subprocess.DEVNULL, stdout=subprocess.DEVNULL,
                            stderr=subprocess.DEVNULL, start_new_session=True)


GTK_RUN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "osmo-gtk-run.py")


def _terminal_argv(cmd, titre="Banc GSM"):
    """argv that opens `cmd` (shell string) in a VTE terminal placed INSIDE a
    GTK window of the bench (osmo-gtk-run.py --tty). No terminal emulator:
    no entry of the bar opens one."""
    return [sys.executable, GTK_RUN, "--tty", "--title", titre, "--", "bash", "-lc", cmd]


def _root_prefix():
```

```
    """argv prefix to execute as root WITHOUT terminal: nothing if already root,
    pkexec (desktop password window) otherwise, sudo -n as a last resort
    (it cannot ask: without NOPASSWD it fails at once, and the log says so)."""
    if os.geteuid() == 0:
        return []
    if shutil.which("pkexec") and os.environ.get("DISPLAY"):
        return ["pkexec", "env", "DISPLAY=" + os.environ.get("DISPLAY", ""),
                "XAUTHORITY=" + os.environ.get("XAUTHORITY", ""), "NO_COLOR=1", "TERM=dumb"]
    return ["sudo", "-n", "-E"]


LAUNCHER = None     # the window, so that the Actions can hand it a log


class Action:
    """An application: how to start it - window, terminal, or background run with log."""
    def __init__(self, argv=None, term_cmd=None, fs_argv=None, log_cmd=None, root=False, slug=None):
        self.argv = argv          # normal graphical window
        self.term_cmd = term_cmd  # command to open in a terminal
        self.fs_argv = fs_argv    # full screen variant if the application handles it
        self.log_cmd = log_cmd    # shell command in the background, output in a log (no terminal)
        self.root = root          # log_cmd: as root (pkexec)
        self.slug = slug          # name of the log
        self.nom = slug or ""     # the button label (title of the tty window)

    def launch(self, mode):
        """mode: 'frame' (fitted on the panel) | 'full' (full screen) | 'top'."""
        if self.log_cmd is not None:
            if LAUNCHER is not None:
                LAUNCHER.run_logged(self)
            return
        before = _wmctrl_ids()
        if self.term_cmd is not None:
            spawn(_terminal_argv(self.term_cmd, self.nom or "Banc GSM"))
        elif mode == "full" and self.fs_argv:
            spawn(self.fs_argv)
        else:
            spawn(self.argv)
        GLib.timeout_add(400, self._settle, before, mode, 0)

    def _settle(self, before, mode, tries):
        new = _wmctrl_ids() - before
        # our own desktop windows are ignored
        new = {w for w in new}
        if not new:
            if tries < 25:  # ~10 s
                GLib.timeout_add(400, self._settle, before, mode, tries + 1)
            return False
        winid = sorted(new)[-1]
        if mode == "full":
            _fullscreen(winid)
        elif mode == "top":
            try:
                with open(TOPZONE_CMD + ".tmp", "w") as f:
                    f.write("HOST %s\n" % winid)
                os.replace(TOPZONE_CMD + ".tmp", TOPZONE_CMD)
            except OSError:
                _place(winid, box_to_screen(TOP_BOX))
        else:  # frame
            _place(winid, box_to_screen(FFT_BOX))
        return False


# ── THE CATALOGUE ──────────────────────────────────────────────────
def _sudo():
    return "" if os.geteuid() == 0 else "sudo -E "
```

```
def catalogue():
    tmux = os.environ.get("TMUX_SESSION", "calypso")
    vty = os.environ.get("MS_VTY_PORT", "")
    if not vty:
        try:                      # 4347 with --dsp: start-native.sh publishes the port
            vty = open("/run/osmo-banc/ms1-vty").read().strip() or "4247"
        except OSError:
            vty = "4247"
    dash = os.environ.get("DASH_PORT", "8080")
    browser = "firefox"
    for b in ("firefox", "chromium", "xdg-open"):
        if shutil.which(b):
            browser = b
            break
    # The three "run" entries: in the background, log in the bar (see the header).
    #   standalone : launch.sh --service starts osmo-banc.service and SAYS so
    #                (notification); then the log of the unit is followed.
    #   multi      : launch.sh --multi (osmo-multi.service, several minutes)
    #                IN PARALLEL with the log of the unit, which shows the
    #                containers coming up - run in sequence, the log would only
    #                start at the end of the startup.
    #   deka       : deka-start.sh (installed by addition.sh --opencl), which
    #                already tees into /var/log/deka.log.
    run_standalone = (f"{REPO}/launch.sh --service; echo; "
                      "exec journalctl -u osmo-banc -f -n 40 --no-pager -o cat")
    run_multi = (f"{REPO}/launch.sh --multi & "
                 "journalctl -u osmo-multi -f -n 20 --no-pager -o cat & wait")
    run_deka = ("if [ -x /root/deka/deka-start.sh ]; then exec /root/deka/deka-start.sh; "
                "else echo 'deka non installe : icone « Supplements » (addition.sh --opencl)'; fi")
    return [
        ("Banc", [
            ("run standalone", Action(log_cmd=run_standalone, root=True, slug="standalone")),
            ("run multi",      Action(log_cmd=run_multi, root=True, slug="multi")),
            ("run deka",       Action(log_cmd=run_deka, root=True, slug="deka")),
            ("Dashboard",      Action(argv=[browser, f"http://127.0.0.1:{dash}"])),
            # tmux, VTY, oFono, pmOS: interactive, hence a terminal - VTE, in a GTK
            # window of the bench, which stays open at the end.
            ("tmux",           Action(term_cmd=f"tmux attach -t {tmux} || tmux -S /tmp/osmocom_tmux attach -t
↪ osmocom || echo 'pas de session tmux'")),
           (f"VTY {vty}",     Action(term_cmd=f"telnet 127.0.0.1 {vty} || {{ echo; echo 'VTY injoignable'; }}")),
        ]),
        # Doom and Quake are only installed by addition.sh --extras (not in the
        # ISO): an entry only appears if its binary is there.
        ("Jeux", [j for j in [
            ("Doom",   Action(argv=["/usr/games/chocolate-doom", "-iwad", "/usr/share/games/doom/freedoom2.wad",
↪ "-window"],
                              fs_argv=["/usr/games/chocolate-doom", "-iwad", "/usr/share/games/doom/freedoom2.wad",
↪ "-fullscreen"])),
            ("Quake",  Action(argv=["/usr/games/openarena", "+set", "r_fullscreen", "0"],
                              fs_argv=["/usr/games/openarena", "+set", "r_fullscreen", "1"])),
            ("Dino",   Action(argv=["/usr/local/bin/osmo-dino-play"],
                              fs_argv=["/usr/local/bin/osmo-dino-play", "--fullscreen"])),
        ] if not j[1].argv[0].startswith("/usr/games/") or os.path.exists(j[1].argv[0])]),
        ("Media", [
            ("Kodi",    Action(argv=["kodi"], fs_argv=["kodi", "--fullscreen"])),
            ("YouTube", Action(argv=["/usr/local/bin/osmo-youtube"],
                               fs_argv=["/usr/local/bin/osmo-youtube", "--fullscreen"])),
        ]),
        ("Telephone", [
            ("Linphone", Action(argv=["sh", "-c", "linphone || linphone-desktop"])),
            ("oFono",    Action(term_cmd="/usr/local/bin/osmo-ofono")),
            # The phone of the bench: a postmarketOS/Phosh VM, with its ModemManager
            # plugged on the modem of the bench (tools/osmo-pmos.sh).
            ("pmOS",     Action(term_cmd="/usr/local/bin/osmo-pmos up")),
```

```
        ]),
        ("Outils", [
            ("Wireshark", Action(argv=["/usr/local/bin/osmo-wireshark-root"])),
        ]),
    ]


class Launcher(Gtk.Window):
    def __init__(self):
        super().__init__(title="osmo-launcher")
        (gx, gy, sw, sh), _ = screen_geom()
        self.geo = (gx, gy, sw, sh)
        self.set_type_hint(TYPE_HINT)
        self.set_decorated(False)
        self.set_resizable(False)
        self.set_keep_below(KEEP_BELOW)
        self.set_skip_taskbar_hint(True)
        self.set_skip_pager_hint(True)
        self.stick()
        screen = self.get_screen()
        visual = screen.get_rgba_visual()
        if visual and screen.is_composited():
            self.set_visual(visual)
        self.set_app_paintable(True)
        self.connect("draw", self._draw)
        self.connect("destroy", self._quit)

        prov = Gtk.CssProvider()
        prov.load_from_data(CSS)
        Gtk.StyleContext.add_provider_for_screen(screen, prov, Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION)

        # ── the run logs ──────────────────────────────────────────
        self.runs = {}          # slug -> {"proc": Popen|None, "titre": str, "log": path, "bouton": Gtk.Button}
        self.cur = None         # displayed slug
        self._shown = ""        # last text pushed into the view
        self.deplie = False
        os.makedirs(LOG_DIR, exist_ok=True)

        col = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=0)
        col.get_style_context().add_class("osmo-launch")
        col.set_valign(Gtk.Align.END)
        self.revealer = Gtk.Revealer()
        self.revealer.set_transition_type(Gtk.RevealerTransitionType.SLIDE_UP)
        self.revealer.set_transition_duration(180)
        self.revealer.add(self._log_box(sw))
        col.pack_start(self.revealer, False, False, 0)

        row = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=2)
        row.set_halign(Gtk.Align.CENTER)
        row.set_valign(Gtk.Align.END)
        for cat, apps in catalogue():
            row.pack_start(self._cat_box(cat, apps), False, False, 0)
        # The big red "?": the Info page (each module, icon / bar / CLI).
        aide = Gtk.Button(label="?")
        aide.get_style_context().add_class("osmo-help")
        aide.set_valign(Gtk.Align.CENTER)
        aide.set_tooltip_text("Info : comment lancer chaque module (icone, barre, CLI)")
        aide.connect("clicked", lambda *_a: spawn(["xdg-open", INFO_PAGE]))
        row.pack_end(aide, False, False, 0)
        col.pack_start(row, False, False, 0)
        self.add(col)

        # bottom band, full width, ~72 px; unfolded: + the log
        self.h = 74
        self._taille(self.h)
        self.show_all()
```

```
    self._taille(self.h)
    GLib.timeout_add(700, self._refresh_log)

def _taille(self, h):
    gx, gy, sw, sh = self.geo
    self.set_size_request(sw, h)
    self.resize(sw, h)
    self.move(gx, gy + sh - h)

def _draw(self, _w, cr):
    import cairo
    cr.set_operator(cairo.OPERATOR_SOURCE)
    cr.set_source_rgba(0, 0, 0, 0)
    cr.paint()
    return False

def _quit(self, *_a):
    self.stop_runs()
    Gtk.main_quit()

# ── the log: the box above the buttons ──────────────────────────
def _log_box(self, sw):
    box = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=4)
    box.get_style_context().add_class("osmo-log")
    box.set_halign(Gtk.Align.CENTER)
    box.set_size_request(min(sw - 60, LOG_W_MAX), LOG_H)
    head = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=6)
    self.lbl_titre = Gtk.Label(label="journal")
    self.lbl_titre.get_style_context().add_class("title")
    self.lbl_etat = Gtk.Label(label="")
    self.lbl_etat.get_style_context().add_class("state")
    head.pack_start(self.lbl_titre, False, False, 0)
    head.pack_start(self.lbl_etat, False, False, 0)
    self.onglets = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=4)
    head.pack_end(self._bouton("▾ replier", self.replier), False, False, 0)
    b_stop = self._bouton("■ stop", self._stop_courant)
    b_stop.set_tooltip_text("arrete la commande en cours (pas le banc : launch.sh --stop)")
    head.pack_end(b_stop, False, False, 0)
    head.pack_end(self.onglets, False, False, 0)
    box.pack_start(head, False, False, 0)
    sc = Gtk.ScrolledWindow()
    sc.set_policy(Gtk.PolicyType.AUTOMATIC, Gtk.PolicyType.AUTOMATIC)
    self.tv = Gtk.TextView()
    self.tv.set_editable(False)
    self.tv.set_cursor_visible(False)
    self.tv.set_monospace(True)
    self.tv.set_wrap_mode(Gtk.WrapMode.WORD_CHAR)
    self.tv.set_left_margin(6)
    self.tv.set_right_margin(6)
    sc.add(self.tv)
    box.pack_start(sc, True, True, 0)
    return box

@staticmethod
def _bouton(label, cb):
    b = Gtk.Button(label=label)
    b.connect("clicked", lambda *_a: cb())
    return b

def deplier(self, slug=None):
    if slug:
        self.cur = slug
        for s, r in self.runs.items():
            ctx = r["bouton"].get_style_context()
            (ctx.add_class if s == slug else ctx.remove_class)("actif")
        self.lbl_titre.set_text("journal : " + self.runs[slug]["titre"])
```

```
        self._shown = ""
    if not self.deplie:
        self.deplie = True
        self._taille(self.h + LOG_H + 10)
        self.revealer.set_reveal_child(True)
    self._refresh_log()

def replier(self):
    if not self.deplie:
        return
    self.deplie = False
    self.revealer.set_reveal_child(False)
    GLib.timeout_add(200, lambda: (self._taille(self.h), False)[1])

def basculer(self):
    if self.deplie:
        self.replier()
    elif self.runs:
        self.deplier(self.cur or next(iter(self.runs)))
    else:
        self.lbl_titre.set_text("journal : aucun run lance")
        self.deplier()

# ── start a run in the background ──────────────────────────────
def run_logged(self, action):
    slug = action.slug or re.sub(r"\W+", "-", action.log_cmd)[:24]
    log = os.path.join(LOG_DIR, slug + ".log")
    r = self.runs.get(slug)
    if r is None:
        b = self._bouton(slug, lambda s=slug: self.deplier(s))
        self.onglets.pack_start(b, False, False, 0)
        b.show()
        r = self.runs[slug] = {"proc": None, "titre": slug, "log": log, "bouton": b}
    self._stop(r)
    # 2G (standalone) or 4G (multi): the antenna rises in the wallpaper from
    # the click on (tools/osmo-fond.py, fade) - osmo-panel.py does the same
    # when it sees the bench up, without effect if it already is.
    if slug in ("standalone", "multi"):
        fond = os.path.join(REPO, "tools", "osmo-fond.py")
        if os.path.exists(fond):
            spawn([sys.executable, fond, "antenne"])
    argv = (_root_prefix() if action.root else []) + ["bash", "-lc", action.log_cmd]
    env = dict(os.environ, NO_COLOR="1", TERM="dumb")
    try:
        fh = open(log, "w")
        fh.write("$ %s\n" % action.log_cmd)
        fh.flush()
        r["proc"] = subprocess.Popen(argv, stdin=subprocess.DEVNULL, stdout=fh, stderr=subprocess.STDOUT,
                                     start_new_session=True, env=env)
        fh.close()
    except OSError as e:
        with open(log, "a") as fh:
            fh.write("lancement impossible : %s\n" % e)
    self.deplier(slug)

@staticmethod
def _stop(r):
    p = r.get("proc")
    if p is None or p.poll() is not None:
        r["proc"] = None
        return
    try:
        os.killpg(p.pid, signal.SIGTERM)
    except OSError:
        pass
    r["proc"] = None
```

```
def _stop_courant(self):
    if self.cur and self.cur in self.runs:
        self._stop(self.runs[self.cur])
        with open(self.runs[self.cur]["log"], "a") as fh:
            fh.write("\n[stop]\n")

def stop_runs(self):
    for r in self.runs.values():
        self._stop(r)

def _refresh_log(self):
    if not self.deplie or not self.cur or self.cur not in self.runs:
        return True
    r = self.runs[self.cur]
    p = r.get("proc")
    if p is not None and p.poll() is not None:
        self.lbl_etat.set_text("termine (statut %d)" % p.returncode)
    elif p is not None:
        self.lbl_etat.set_text("en cours ...")
    else:
        self.lbl_etat.set_text("")
    try:
        with open(r["log"], "rb") as fh:
            fh.seek(0, os.SEEK_END)
            taille = fh.tell()
            fh.seek(max(0, taille - 48 * 1024))
            data = fh.read().decode("utf-8", "replace")
    except OSError:
        data = ""
    data = ANSI.sub("", data)
    if data != self._shown:
        self._shown = data
        buf = self.tv.get_buffer()
        buf.set_text(data)
        fin = buf.get_end_iter()
        self.tv.scroll_to_iter(fin, 0.0, False, 0.0, 1.0)
    return True

# ── the buttons ──────────────────────────────────────────────────
def _cat_box(self, cat, apps):
    box = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=2)
    box.get_style_context().add_class("osmo-cat")
    lbl = Gtk.Label(label=cat)
    lbl.get_style_context().add_class("title")
    lbl.set_halign(Gtk.Align.START)
    box.pack_start(lbl, False, False, 0)
    approw = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=3)
    for name, action in apps:
        approw.pack_start(self._app_widget(name, action), False, False, 0)
    if cat == "Banc":
        wrap = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=0)
        wrap.get_style_context().add_class("osmo-app")
        bj = Gtk.Button(label="▴ journal")
        bj.get_style_context().add_class("journal")
        bj.set_tooltip_text("deplie / replie le journal des run")
        bj.connect("clicked", lambda *_a: self.basculer())
        wrap.pack_start(bj, False, False, 0)
        approw.pack_start(wrap, False, False, 0)
    box.pack_start(approw, False, False, 0)
    return box

def _app_widget(self, name, action):
    wrap = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=0)
    wrap.get_style_context().add_class("osmo-app")
    b = Gtk.Button(label=name)
```

```
        action.nom = name
        b.connect("clicked", lambda *_a: action.launch("frame"))
        wrap.pack_start(b, False, False, 0)
        if action.log_cmd is not None:
            # a run has no window to fit nor to send to the top: a single button
            b.set_tooltip_text(f"{name} - en fond, journal dans la barre")
            return wrap
        b.set_tooltip_text(f"{name} - clic: cale sur le cadre")
        # □ full screen
        bf = Gtk.Button(label="□")
        bf.get_style_context().add_class("mini")
        bf.set_tooltip_text("plein ecran")
        bf.connect("clicked", lambda *_a: action.launch("full"))
        wrap.pack_start(bf, False, False, 0)
        # □ send to the top (LAB GSM zone)
        bt = Gtk.Button(label="□")
        bt.get_style_context().add_class("mini")
        bt.set_tooltip_text("envoyer en haut (zone LAB GSM)")
        bt.connect("clicked", lambda *_a: action.launch("top"))
        wrap.pack_start(bt, False, False, 0)
        return wrap


def single_instance():
    try:
        old = int(open(PID_FILE).read().strip())
        if old != os.getpid():
            os.kill(old, signal.SIGTERM)
            time.sleep(0.3)
    except (OSError, ValueError):
        pass
    try:
        with open(PID_FILE, "w") as f:
            f.write(str(os.getpid()))
    except OSError as e:
        print("osmo-launcher : verrou %s non ecrit (%s)" % (PID_FILE, e), file=sys.stderr)


if __name__ == "__main__":
    single_instance()
    LAUNCHER = Launcher()
    # SIGTERM (restart by the desktop guardian, single_instance): the
    # journalctl -f started in the background must not outlive the bar.
    GLib.unix_signal_add(GLib.PRIORITY_DEFAULT, signal.SIGTERM, lambda *_a: (LAUNCHER._quit(), False)[1])
    Gtk.main()
```

2.260 /opt/GSM/osmo-operator/tools/osmo-lte-install.sh

30484 octets, 494 lignes → 494 lignes

```
#!/bin/bash
# ============================================================================
# osmo-lte-install.sh - THE BENCH 4G, INSTALLED ONCE FOR THE THREE PATHS.
#
# This script is the SINGLE place where the 4G (srsRAN over ZeroMQ, Open5GS)
# is installed; the three paths call it:
#   - addition.sh            (an already installed machine)     : --all
#   - update.sh              (the catch-up)                      : --configs --launchers
#   - iso_modules/72-lte-pmos.sh (the ISO build, in chroot)   : --deps --configs --launchers
#     (the binaries there come from the docker build .deb files:
#      osmo-build-srsran, osmo-build-open5gs, laid down by 50-injection-image.sh)
#
# WHERE THINGS LIVE:
#   /opt/LTE/srsGUI             srsGUI (Qt5 + Qwt): the real-time plots of
```

```
#                               srsenb / srsue. BUILT BEFORE srsRAN - its cmake
#                               looks for it at configure time - and installed
#                               in /usr/local/lib/libsrsgui
#   /opt/LTE/srsRAN_4G          srsRAN sources and build (ZeroMQ, GUI),
#                               binaries in /usr/local/bin (srsenb, srsue, srsepc)
#   /opt/LTE/open5gs            Open5GS sources; install prefix
#                               /opt/LTE/open5gs/install (bin, etc/open5gs, var/log)
#   /root/.config/srsran        enb.conf ue.conf sib.conf rr.conf rb.conf user_db.csv
#                               = the repo's configs/srsran (srs* read root's HOME)
#   /usr/local/bin/osmo-lte     the launcher (tools/osmo-lte.sh)
#   /usr/local/bin/osmo-epc     the core (tools/osmo-epc.sh), alias open5gs-epc.sh
#
#   osmo-lte-install                = --deps --debs --configs --launchers
#   osmo-lte-install --all          same, plus --build if the binaries are missing
#   osmo-lte-install --build        builds srsRAN and Open5GS in /opt/LTE
#   osmo-lte-install --deps         the apt packages (runtime + build), mongodb-org
#   osmo-lte-install --debs         installs the cached .deb files (/var/cache/osmo-debs)
#   osmo-lte-install --configs      installs the repo configs
#   osmo-lte-install --launchers    osmo-lte, osmo-epc in /usr/local/bin
#                                   (+ the open5gs-webui unit, enabled)
#   osmo-lte-install --webui        builds the Open5GS WebUI (console :9999)
#   osmo-lte-install --force        (with --configs) overwrites existing configs
#
# Idempotent, non-fatal package by package. Usable in a session AND in the ISO
# chroot (no systemd: the systemctl calls fail there without stopping).
# ============================================================================
set -u

REPO="${OSMO_REPO:-${DIR:-/opt/GSM/osmo-operator}}"
LTE="${OSMO_LTE_ROOT:-/opt/LTE}"
O5GS="$LTE/open5gs"
O5GS_PREFIX="${OPEN5GS_PREFIX:-$O5GS/install}"
SRS="$LTE/srsRAN_4G"
SRSGUI="$LTE/srsGUI"
SRS_DIR="${OSMO_SRSRAN_DIR:-/root/.config/srsran}"
DEB_CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
SRS_REF="${OSMO_SRSRAN_REF:-release_25_10}"
# srsGUI has neither tag nor release: it is master. The .deb version is
# therefore named here, once, and the three paths reuse it (Dockerfile,
# packaging/snapshot-lte-debs.sh, lte_debs).
SRSGUI_REF="${OSMO_SRSGUI_REF:-master}"
SRSGUI_VER="${OSMO_SRSGUI_VER:-0.1+git}"
O5GS_REF="${OSMO_OPEN5GS_REF:-v2.8.0}"
JOBS="${JOBS:-$(nproc)}"
# ── THE srsRAN ISA: PORTABLE, NOT « native » ──────────────────────────────
# The srsRAN_4G CMakeLists sets GCC_ARCH to « native » by default, and its
# cmake/modules/FindSSE.cmake decides on -mavx/-mavx2/-mfma/-mavx512 through
# check_c_source_runs - that is, by RUNNING the test on the building machine.
# The binaries thus come out tailored for that processor; on a CPU without
# AVX2 (a VM that does not expose the full set, an older laptop, a different
# CI runner) the first such instruction raises SIGILL. An ISO moves from one
# machine to another by definition: it cannot be built « for the build
# machine ».
# A common baseline is therefore fixed (SSE4.1/SSE4.2, every x86_64 since
# 2009 - and srsRAN requires SSE4.1 anyway, see its « no SIMD instructions
# found ») and the AVX/AVX2/FMA/AVX512 detections, which only measure the
# host, are switched off.
# OSMO_SRSRAN_ARCH=native restores host-tuned binaries (faster, valid on THIS
# machine only); any other value is passed as is to -march= (x86-64,
# nehalem, haswell...). A -march the local gcc does not know is ignored by
# ADD_C_COMPILER_FLAG_IF_AVAILABLE: the compiler's x86-64 baseline is then
# used, portable as well.
SRS_ARCH="${OSMO_SRSRAN_ARCH:-x86-64-v2}"

# ── THE srsRAN .deb VERSION CARRIES ITS ISA ───────────────────────────────
# The ISA is part of the version: a native binary cannot be mistaken for a
```

```
# portable one, the .deb cache (locally and in the GitHub workflow restore
# keys) keeps both without one shadowing the other, and nothing is rebuilt
# once the right .deb is there.
srs_deb_version() {
    local v a
    v="${SRS_REF#release_}"; v="${v//_/.}"
    a="$(printf '%s' "$SRS_ARCH" | tr -c 'A-Za-z0-9.' '-')"   # osmo-deb: [A-Za-z0-9.+~-]
    printf '%s+zmq+%s\n' "$v" "$a"
}

: "${GREEN:=}"; : "${YELLOW:=}"; : "${CYAN:=}"; : "${RED:=}"; : "${BOLD:=}"; : "${NC:=}"
_l_say()  { echo -e "  ${CYAN}→${NC} $*"; }
_l_ok()   { echo -e "      ${GREEN}✓${NC} $*"; }
_l_warn() { echo -e "      ${YELLOW}!${NC} $*"; }
_l_err()  { echo -e "      ${RED}✗${NC} $*" >&2; }

# ── WILL THIS srsenb RUN ANYWHERE BUT HERE? ──────────────────────────────
# An already installed srsenb - or one from a cached .deb built on ANOTHER
# machine - may be tailored for the CPU that built it, and nothing betrays
# it: it exists, its libraries resolve, --help answers. It only shows at the
# first real launch, as an « Illegal instruction (core dumped) » without a
# line of log (see SRS_ARCH above). So look at what the binary CONTAINS: a
# single %ymm (AVX/AVX2) or %zmm (AVX512) instruction condemns it outside its
# machine of origin. Without objdump (binutils absent) no verdict: what is
# there is kept, and no hour-long rebuild is triggered on a doubt.
_srs_isa_portable() {
    local b; b="$(command -v srsenb 2>/dev/null)" || return 0
    [ -n "$b" ] || return 0
    command -v objdump >/dev/null 2>&1 || return 0
    ! objdump -d "$b" 2>/dev/null | grep -qE '%[yz]mm[0-9]'
}

# ── THE PACKAGES ─────────────────────────────────────────────────────────
# Runtime (what ldd of srsenb and open5gs-mmed asks for) AND build: the ISO is
# a workshop, « make » must be possible again in /opt/LTE. mongodb-org comes
# from the MongoDB repository (noble/8.0): the key and the source are laid
# down if missing. Non-fatal package by package.
_lte_apt_mongo_source() {
    local k=/usr/share/keyrings/mongodb-server-8.0.gpg
    local s=/etc/apt/sources.list.d/mongodb-org-8.0.list
    [ -s "$s" ] && [ -s "$k" ] && return 0
    command -v curl >/dev/null 2>&1 && command -v gpg >/dev/null 2>&1 || { _l_warn "curl/gpg absents : pas de
     ↪  depot MongoDB"; return 1; }
    if curl -fsSL --retry 3 https://www.mongodb.org/static/pgp/server-8.0.asc | gpg --dearmor --yes -o "$k"
     ↪  2>/dev/null; then
        echo "deb [ arch=amd64,arm64 signed-by=$k ] https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0
         ↪  multiverse" > "$s"
        _l_ok "depot MongoDB 8.0 pose ($s)"
    else
        _l_warn "cle MongoDB injoignable : mongodb-org ne s installera pas (HSS/PCRF sans base)"
        return 1
    fi
}

lte_deps() {
    export DEBIAN_FRONTEND=noninteractive
    _l_say "paquets de la 4G (srsRAN ZeroMQ + Open5GS + MongoDB)"
    _lte_apt_mongo_source || true
    apt-get update -y >/dev/null 2>&1 || true
    local p
    # qtbase5/qwt/boost: srsGUI (the real-time plots of srsenb and srsue).
    # Without them the srsGUI cmake fails and srsRAN builds without plots -
    # nothing stops, but the constellation would never be seen.
    # libpcsclite-dev: srsUE reads a REAL SIM card through PC/SC
    # (srsue/hdr/stack/upper/pcsc_usim.h -> winscard.h); the srsRAN tree
    # shipped by osmo-build-srsran carries a CMakeCache where PCSC is ACTIVE,
```

```
    # so any rebuild needs winscard.h.
    for p in libzmq5 libzmq3-dev libboost-program-options-dev libmbedtls-dev libconfig++-dev libfftw3-dev
     ↪  libsctp-dev lksctp-tools cmake \
              qtbase5-dev libqt5opengl5-dev libqwt-qt5-dev libboost-system-dev libboost-thread-dev
 ↪  libboost-test-dev \
              libpcsclite-dev \
              meson ninja-build flex bison libgnutls28-dev libgcrypt20-dev libssl-dev libidn-dev libmongoc-dev
 ↪  libbson-dev \
              libyaml-dev libnghttp2-dev libmicrohttpd-dev libcurl4-gnutls-dev libtins-dev libtalloc-dev
 ↪  libc-ares-dev \
              mongodb-org mongodb-mongosh mongodb-database-tools; do
        dpkg -s "$p" >/dev/null 2>&1 && continue
        apt-get install -y --no-install-recommends "$p" >/dev/null 2>&1 \
            && _l_ok "$p" || _l_warn "$p indisponible (apt) - ignore"
    done
    # mongod: ENABLED at boot (osmo-epc starts it anyway if it sleeps).
    # In a chroot without systemd, enable still creates the link.
    systemctl enable mongod >/dev/null 2>&1 && _l_ok "mongod active au demarrage" || true
}

# ── THE DOCKER BUILD .deb FILES ──────────────────────────────────────────
# osmo-build-libzmq, osmo-build-srsran, osmo-build-open5gs: produced by the
# Dockerfile (osmo-deb pack) or by a snapshot of the reference machine
# (packaging/snapshot-lte-debs.sh). Installed only if the binaries are missing.
lte_debs() {
    local n deb suite
    suite="$(. /etc/os-release 2>/dev/null; echo "${VERSION_CODENAME:-noble}")"
    # srsgui BEFORE srsran: it is the plot library, and srsRAN looks for it at
    # ITS configure time (see lte_build).
    for n in libzmq srsgui srsran open5gs; do
        case "$n" in
            libzmq)  [ -e /usr/local/lib/libzmq.so.5 ] || [ -e /usr/lib/x86_64-linux-gnu/libzmq.so.5 ] && continue
             ↪  ;;
            srsgui)  [ -e /usr/local/lib/libsrsgui.so ] || [ -e /usr/local/lib/libsrsgui.a ] && continue ;;
            srsran)  command -v srsenb >/dev/null 2>&1 && continue ;;
            open5gs) [ -x "$O5GS_PREFIX/bin/open5gs-mmed" ] && continue ;;
        esac
        deb="$(ls -1 "$DEB_CACHE"/osmo-build-${n}_*"~${suite}_$(dpkg --print-architecture).deb" 2>/dev/null | tail
         ↪  -1)"
        [ -n "$deb" ] || { _l_warn "$n : pas de binaire ni de .deb dans $DEB_CACHE (osmo-lte-install --build)";
         ↪  continue; }
        dpkg -i --force-overwrite "$deb" >/dev/null 2>&1 && _l_ok "$n pose depuis $(basename "$deb")" || _l_warn
         ↪  "dpkg -i $(basename "$deb") a echoue"
    done
    ldconfig 2>/dev/null || true
}

# ── THE BUILD, IN /opt/LTE ───────────────────────────────────────────────
# srsRAN_4G with ZeroMQ (Ubuntu's libzmq3-dev is enough), Open5GS through
# meson with the /opt/LTE/open5gs/install prefix. Under OSMO_DEB=1 (the
# Dockerfile) each step is packed into a .deb via osmo-deb pack; otherwise
# make/ninja install directly.
lte_build() {
    mkdir -p "$LTE"
    local pack=""
    [ "${OSMO_DEB:-0}" = "1" ] && command -v osmo-deb >/dev/null 2>&1 && pack=1
    # ── THE .deb VERSIONS: WRITTEN ONCE, FOR EVERYONE ──────────────────
    # The names that count are those of the Dockerfile and of
    # packaging/snapshot-lte-debs.sh: osmo-build-srsran_25.10+zmq+<isa>,
    # osmo-build-open5gs_2.8.0+git. They are derived here from the same git
    # refs, with dots (osmo-deb refuses an underscore in a version).
    local srs_ver o5gs_ver
    srs_ver="$(srs_deb_version)"                                  # release_25_10 -> 25.10+zmq+x86-64-v2
    o5gs_ver="${O5GS_REF#v}+git"                                  # v2.8.0        -> 2.8.0+git
    # ── srsGUI FIRST: srsRAN IS THE ONE LOOKING FOR IT ─────────────────
    # The real-time plots of srsenb and srsue (constellation, spectrum, PDSCH)
```

```
# come from srsGUI (github.com/srsran/srsGUI, Qt5 + Qwt), a SEPARATE
# library - and the srsRAN cmake looks for it AT ITS OWN CONFIGURE TIME.
# Installed afterwards it would be useless: it is therefore built HERE,
# just before, and srsRAN is configured with ENABLE_GUI=ON below.
#
# NON-FATAL, ON PURPOSE: without Qt, without network, or if the srsGUI
# cmake fails, a warning is printed and the build goes on. ENABLE_GUI=ON
# then breaks nothing - the srsRAN cmake notices the absence and does not
# enable the plots. A bench without a constellation window is still a
# bench; a bench that no longer builds is not.
if [ ! -e /usr/local/lib/libsrsgui.so ] && [ ! -e /usr/local/lib/libsrsgui.a ]; then
    _l_say "srsGUI $SRSGUI_REF -> $SRSGUI (les traces de srsenb / srsue)"
    if [ ! -d "$SRSGUI/.git" ]; then
        git clone --depth 1 -b "$SRSGUI_REF" https://github.com/srsran/srsGUI "$SRSGUI" >/dev/null 2>&1 \
          || git clone --depth 1 https://github.com/srsran/srsGUI "$SRSGUI" >/dev/null 2>&1 \
          || _l_warn "srsGUI : clone impossible - srsRAN se compilera sans traces"
    fi
    if [ -d "$SRSGUI" ]; then
        # ── -DBOOST_TIMER_ENABLE_DEPRECATED: WITHOUT IT, NOTHING COMES OUT
        # srsGUI does not build on Boost 1.83 (noble): its TESTS
        # (test/cxx/*_test.cpp) include boost/progress.hpp and
        # boost/timer.hpp, which are deprecated and #error out - and
        # `make install` goes through the `all` target, tests included, so
        # one test that does not compile blocks the whole install. Boost
        # itself says what to do (« You can define
        # BOOST_TIMER_ENABLE_DEPRECATED to suppress this error »): it is
        # defined, rather than removing targets from the upstream project.
        ( cd "$SRSGUI" && mkdir -p build && cd build \
          && cmake -DCMAKE_BUILD_TYPE=Release \
                   -DCMAKE_CXX_FLAGS=-DBOOST_TIMER_ENABLE_DEPRECATED .. >/dev/null 2>&1 \
          && make -j"$JOBS" >/dev/null 2>&1 \
          && if [ -n "$pack" ]; then OSMO_DEB_SRC_ROOT="$LTE" OSMO_DEB_SRC="$SRSGUI" osmo-deb pack srsgui
           ↪  "$SRSGUI_VER" make install
             else make install >/dev/null 2>&1; fi ) \
          && { ldconfig; _l_ok "srsGUI : /usr/local/lib/libsrsgui"; } \
          || _l_warn "srsGUI : compilation echouee - srsRAN se compilera sans traces"
    fi
else
    _l_ok "srsGUI deja la (/usr/local/lib/libsrsgui)"
fi
# ── A srsenb WITHOUT PLOTS DOES NOT FIX ITSELF ──────────────────────
# A machine that already has srsRAN - or got it from osmo-build-srsran -
# may carry binaries without plots even with libsrsgui freshly installed
# ten lines above. So check what the binary is LINKED against, not only
# whether it exists.
_srs_lie_gui() {
    local b; b="$(command -v srsenb 2>/dev/null)" || return 1
    [ -n "$b" ] && ldd "$b" 2>/dev/null | grep -q libsrsgui
}
if ! command -v srsenb >/dev/null 2>&1 \
   || { [ -e /usr/local/lib/libsrsgui.so ] && ! _srs_lie_gui; } \
   || { [ "$SRS_ARCH" != native ] && ! _srs_isa_portable; }; then
    if command -v srsenb >/dev/null 2>&1; then
        if [ "$SRS_ARCH" != native ] && ! _srs_isa_portable; then
            _l_say "srsRAN_4G : recompilation, srsenb porte de l AVX (taille pour le CPU du build, SIGILL
             ↪  ailleurs)"
        else
            _l_say "srsRAN_4G : recompilation, srsenb n est pas lie a srsGUI"
        fi
    else
        _l_say "srsRAN_4G $SRS_REF (ZeroMQ, -march=$SRS_ARCH) -> $SRS"
    fi
    if [ ! -d "$SRS/.git" ]; then
        git clone --depth 1 -b "$SRS_REF" https://github.com/srsran/srsRAN_4G "$SRS" || { _l_err "clone
         ↪  srsRAN_4G"; return 1; }
    fi
```

```
        # ── THE CMAKE CACHE LIES ABOUT THE ISA ──────────────────────────────
        # HAVE_AVX2, HAVE_FMA, HAVE_AVX512 are CACHE variables set by the
        # check_c_source_runs of FindSSE.cmake. -DENABLE_AVX2=OFF prevents the
        # TEST from running, it does not erase a previous answer: on an
        # already configured tree, the CMakeLists `if (HAVE_AVX2)` would reread
        # the cache and put -mavx2 back. So start from an empty build - the
        # flags change anyway, every object has to be rebuilt.
        rm -rf "$SRS/build"
        # -DGCC_ARCH: the common baseline (see SRS_ARCH above). The four
        # ENABLE_* switch off the detections that only measure the host;
        # SSE4.1 stays active (ENABLE_SSE=ON by default), otherwise cmake
        # stops on « no SIMD instructions found » and srsRAN loses its -Ofast.
        _srs_isa=(-DGCC_ARCH="$SRS_ARCH" -DENABLE_AVX=OFF -DENABLE_AVX2=OFF -DENABLE_FMA=OFF -DENABLE_AVX512=OFF)
        [ "$SRS_ARCH" = native ] && _srs_isa=(-DGCC_ARCH=native)
        ( cd "$SRS" && mkdir -p build && cd build \
         && cmake -DENABLE_ZEROMQ=ON -DENABLE_GUI=ON -DENABLE_UHD=OFF -DENABLE_BLADERF=OFF -DENABLE_SOAPYSDR=OFF \
                  "${_srs_isa[@]}" \
                  -DCMAKE_BUILD_TYPE=Release .. >/dev/null \
          && make -j"$JOBS" >/dev/null \
          && if [ -n "$pack" ]; then OSMO_DEB_SRC_ROOT="$LTE" OSMO_DEB_SRC="$SRS" osmo-deb pack srsran "$srs_ver"
           ↪  make install
             else make install >/dev/null; fi ) \
          && { ldconfig; _l_ok "srsRAN : $(command -v srsenb)"; } || { _l_err "srsRAN : compilation echouee";
           ↪  return 1; }
    else
        _l_ok "srsRAN deja la ($(command -v srsenb))$(_srs_lie_gui && echo ' + traces srsGUI')"
    fi
    if [ ! -x "$O5GS_PREFIX/bin/open5gs-mmed" ]; then
        _l_say "Open5GS $O5GS_REF -> $O5GS (prefixe $O5GS_PREFIX)"
        if [ ! -d "$O5GS/.git" ]; then
            git clone --depth 1 -b "$O5GS_REF" https://github.com/open5gs/open5gs "$O5GS" || { _l_err "clone
             ↪  open5gs"; return 1; }
        fi
        ( cd "$O5GS" \
          && { [ -f build/build.ninja ] || meson setup build --prefix="$O5GS_PREFIX" >/dev/null; } \
          && ninja -C build -j"$JOBS" >/dev/null \
          && if [ -n "$pack" ]; then OSMO_DEB_SRC_ROOT="$LTE" OSMO_DEB_SRC="$O5GS" osmo-deb pack open5gs
           ↪  "$o5gs_ver" ninja -C build install
             else ninja -C build install >/dev/null; fi ) \
         && { ldconfig; _l_ok "Open5GS : $O5GS_PREFIX/bin"; } || { _l_err "Open5GS : compilation echouee"; return
           ↪  1; }
    else
        _l_ok "Open5GS deja la ($O5GS_PREFIX/bin/open5gs-mmed)"
    fi
}

# ── THE freeDiameter CERTIFICATES ──────────────────────────────────────────
# The bench Diameter peers are in No_TLS, but freeDiameter loads TLS_Cred /
# TLS_CA at init and refuses to start without them. Open5GS ships its own
# (configs/open5gs/tls of its repo) through the same install_conf that does
# nothing under DESTDIR (see lte_configs): ours are therefore generated, once
# (self-signed CA + one certificate per daemon), without touching existing ones.
lte_tls() {
    local t="$O5GS_PREFIX/etc/open5gs/tls" n
    command -v openssl >/dev/null 2>&1 || { _l_warn "openssl absent : pas de certificats freeDiameter (apt install
     ↪  openssl)"; return 0; }
    mkdir -p "$t"
    if [ ! -s "$t/ca.crt" ] || [ ! -s "$t/ca.key" ]; then
        openssl req -x509 -newkey rsa:2048 -nodes -days 3650 -subj "/CN=ca.localdomain" \
            -keyout "$t/ca.key" -out "$t/ca.crt" >/dev/null 2>&1 || { _l_warn "CA freeDiameter : openssl a
             ↪  echoue"; return 0; }
    fi
    for n in mme hss pcrf smf; do
        [ -s "$t/$n.crt" ] && [ -s "$t/$n.key" ] && continue
        openssl req -newkey rsa:2048 -nodes -subj "/CN=$n.localdomain" -keyout "$t/$n.key" -out "$t/$n.csr"
         ↪  >/dev/null 2>&1 \
```

```
        && openssl x509 -req -in "$t/$n.csr" -CA "$t/ca.crt" -CAkey "$t/ca.key" -CAcreateserial -days 3650 -out
        ↪  "$t/$n.crt" >/dev/null 2>&1 \
        || _l_warn "certificat freeDiameter $n : openssl a echoue"
        rm -f "$t/$n.csr"
    done
    chmod 600 "$t"/*.key 2>/dev/null || true
}

# ── THE CONFIGS ──────────────────────────────────────────────────────────
# The repo ones, as they run on the reference bench:
#   configs/srsran/*   -> /root/.config/srsran   (eNB 0x19B, TAC 7, EARFCN 3350,
#                         MME 127.0.0.2, ZeroMQ 2000/2001 at 11.52e6, SIB3+SIB7,
#                         UE 001010001000001, APN srsapn, netns ue1)
#   configs/open5gs/*.yaml -> $PREFIX/etc/open5gs (MME 127.0.0.2, SGs towards
#                         OsmoMSC 127.0.0.1, TAC 7 / LAC 1, UE 10.45.0.0/16)
# An existing config is not overwritten (the bench's live settings), except
# with --force: it is kept as .bak-osmo next to it.
lte_configs() {
    local force="${1:-0}" f dst
    mkdir -p "$SRS_DIR"
    for f in "$REPO"/configs/srsran/*; do
        [ -f "$f" ] || continue
        dst="$SRS_DIR/$(basename "$f")"
        if [ -f "$dst" ] && [ "$force" != "1" ]; then continue; fi
        [ -f "$dst" ] && ! cmp -s "$f" "$dst" && cp -a "$dst" "$dst.bak-osmo"
        install -m644 "$f" "$dst"
    done
    _l_ok "srsRAN : configs dans $SRS_DIR ($(ls "$SRS_DIR" | grep -c '\.conf$') .conf, user_db.csv)"
    # The BINARY is what says whether Open5GS is installed: an Open5GS from
    # the cached .deb, or an install whose etc/ was cleaned (the lite ISO
    # cleans /opt/LTE), has its binaries without the example yaml files. The
    # etc/ directory is created here.
    if [ -x "$O5GS_PREFIX/bin/open5gs-mmed" ] || [ -d "$O5GS_PREFIX/etc/open5gs" ] || [ "$force" = "1" ]; then
        mkdir -p "$O5GS_PREFIX/etc/open5gs" "$O5GS_PREFIX/var/log/open5gs"
        for f in "$REPO"/configs/open5gs/*.yaml; do
            [ -f "$f" ] || continue
            dst="$O5GS_PREFIX/etc/open5gs/$(basename "$f")"
            # The yaml files carry the /opt/LTE/open5gs/install prefix: another
            # prefix (OPEN5GS_PREFIX) is substituted on the fly.
           if [ -f "$dst" ] && [ "$force" != "1" ] && grep -q 'osmo-operator' "$dst" 2>/dev/null; then continue; fi
            [ -f "$dst" ] && ! cmp -s "$f" "$dst" && cp -a "$dst" "$dst.bak-osmo"
            sed "s#/opt/LTE/open5gs/install#$O5GS_PREFIX#g" "$f" > "$dst"
        done
        _l_ok "Open5GS : configs dans $O5GS_PREFIX/etc/open5gs"
        # freeDiameter AND the TLS certificates: what « ninja install » does
        # NOT lay down under DESTDIR (install_conf of meson.build: nothing if
        # DESTDIR is defined), hence never in the osmo-build-open5gs .deb -
        # without them the MME, HSS, PCRF and SMF die at init (« Unable to
        # open configuration file … freeDiameter/mme.conf »). See
        # configs/open5gs/freeDiameter/mme.conf. The .fdx directory depends on
        # the triplet (lib/x86_64-linux-gnu on amd64): substituted as well.
        local fdd fdlib
        fdd="$O5GS_PREFIX/etc/freeDiameter"; mkdir -p "$fdd"
        fdlib="$(ls -d "$O5GS_PREFIX"/lib/*/freeDiameter "$O5GS_PREFIX"/lib/freeDiameter 2>/dev/null | head -1)"
        for f in "$REPO"/configs/open5gs/freeDiameter/*.conf; do
            [ -f "$f" ] || continue
            dst="$fdd/$(basename "$f")"
           if [ -f "$dst" ] && [ "$force" != "1" ] && grep -q 'osmo-operator' "$dst" 2>/dev/null; then continue; fi
            [ -f "$dst" ] && ! cmp -s "$f" "$dst" && cp -a "$dst" "$dst.bak-osmo"
            sed "s#/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter#${fdlib:-$O5GS_PREFIX/lib/x86_64-
            ↪  linux-gnu/freeDiameter}#g; s#/opt/LTE/open5gs/install#$O5GS_PREFIX#g" "$f" >
            ↪  "$dst"
        done
        lte_tls
        _l_ok "freeDiameter : $(ls "$fdd" 2>/dev/null | grep -c '\.conf$') .conf dans $fdd, TLS dans
        ↪  $O5GS_PREFIX/etc/open5gs/tls"
```

```
    else
        _l_warn "Open5GS pas installe (pas de $O5GS_PREFIX/bin/open5gs-mmed) : ses configs seront posees apres
        ↪  --build / --debs"
    fi
}

# ── THE LAUNCHERS ──────────────────────────────────────────────────────
lte_launchers() {
    local t
    for t in osmo-lte osmo-epc; do
        [ -f "$REPO/tools/$t.sh" ] || { _l_warn "$REPO/tools/$t.sh absent"; continue; }
        chmod 755 "$REPO/tools/$t.sh"
        ln -sfn "$REPO/tools/$t.sh" "/usr/local/bin/$t"
    done
    # The core's older name, kept: habits and the bench notes use it.
    ln -sfn "$REPO/tools/osmo-epc.sh" /usr/local/bin/open5gs-epc.sh
    _l_ok "lanceurs : /usr/local/bin/osmo-lte, osmo-epc (alias open5gs-epc.sh)"
    # ── THE 4G SUBSCRIBER CONSOLE: INSTALLED AND ENABLED BY DEFAULT ──────
    # Like the 2G dashboard (osmo-egprs-web). The unit still does not start
    # until the WebUI is built: its ConditionPathExists skips it, without
    # failure. « --webui » builds it. A MASKED unit is left alone: that is the
    # way to say no.
    if [ -f "$REPO/services/open5gs-webui.service" ]; then
        install -m644 "$REPO/services/open5gs-webui.service" \
                /etc/systemd/system/open5gs-webui.service 2>/dev/null || true
        systemctl daemon-reload 2>/dev/null || true
        if [ "$(systemctl is-enabled open5gs-webui 2>/dev/null)" != masked ]; then
            systemctl enable open5gs-webui >/dev/null 2>&1 || true
        fi
        _l_ok "console 4G : open5gs-webui.service posee et activee (:9999)"
    fi
}

# ── THE OPEN5GS WEBUI: THE 4G SUBSCRIBER CONSOLE ──────────────────────────
# The WebUI provisions LTE subscribers (otherwise done by hand in MongoDB or
# with « osmo-epc subscribers », which replays the repo JSON). It lives in the
# Open5GS SOURCES (webui/), not in the install prefix: a machine installed
# from the bench .deb files (osmo-build-open5gs) does NOT have it. It is
# fetched with a sparse checkout - the whole open5gs repo for one Next.js
# directory would be absurd - and built in place. It is npm: it needs the
# network and takes a few minutes. Hence a separate sub-command, outside the
# default path.
lte_webui() {
    local dir="$O5GS/webui" tmp="$LTE/.open5gs-webui-src"
    command -v node >/dev/null 2>&1 && command -v npm >/dev/null 2>&1 \
        || { _l_warn "WebUI Open5GS : node/npm absents - rien a construire"; return 1; }
    if [ ! -f "$dir/package.json" ]; then
        _l_say "WebUI Open5GS $O5GS_REF -> $dir"
        rm -rf "$tmp"
        # sparse: only webui/ is fetched (and blobs on demand).
        if git clone --depth 1 --filter=blob:none --sparse -b "$O5GS_REF" \
               https://github.com/open5gs/open5gs "$tmp" >/dev/null 2>&1 \
           && ( cd "$tmp" && git sparse-checkout set webui >/dev/null 2>&1 ) \
           && [ -d "$tmp/webui" ]; then
            mkdir -p "$O5GS"
            rm -rf "$dir"; mv "$tmp/webui" "$dir"; rm -rf "$tmp"
        else
            rm -rf "$tmp"; _l_err "WebUI Open5GS : recuperation des sources echouee"; return 1
        fi
    fi
    # `npm install` and not `npm ci --omit=dev`: Next.js builds WITH its
    # development dependencies, and the repo does not always ship an
    # up-to-date package-lock.
    ( cd "$dir" && npm install --no-audit --no-fund >/dev/null 2>&1 \
      && npm run build >/dev/null 2>&1 ) \
        || { _l_err "WebUI Open5GS : npm install / build echoue (voir $dir)"; return 1; }
```

```
    [ -f "$dir/server/index.js" ] || { _l_err "WebUI Open5GS : pas de server/index.js apres build"; return 1; }
    _l_ok "WebUI Open5GS construit ($dir) - console des abonnes sur :9999"
    systemctl restart open5gs-webui >/dev/null 2>&1 || true
}

osmo_lte_install() {
    local deps=0 debs=0 configs=0 launchers=0 build=0 all=0 force=0 webui=0 a rc=0
    for a in "$@"; do case "$a" in
        --deps) deps=1 ;; --debs) debs=1 ;; --configs) configs=1 ;; --launchers) launchers=1 ;;
        --build) build=1 ;; --all) all=1 ;; --force) force=1 ;; --webui) webui=1 ;;
        # The Dockerfile asks for the name instead of copying it: written
        # once, and changing OSMO_SRSRAN_ARCH changes both.
        --srs-deb-version) srs_deb_version; return 0 ;;
        *) echo "osmo-lte-install : option inconnue $a" >&2; return 2 ;;
    esac; done
    if [ $((deps + debs + configs + launchers + build + all + webui)) -eq 0 ]; then deps=1; debs=1; configs=1;
     ↪  launchers=1; fi
    if [ "$all" = 1 ]; then deps=1; debs=1; configs=1; launchers=1; fi
    [ "$deps" = 1 ] && lte_deps
    [ "$debs" = 1 ] && lte_debs
    # « ! command -v srsenb » is not enough to decide on a build: a srsenb
    # that is PRESENT but tailored for another machine's CPU would get through
    # --all (hence addition.sh and update.sh). Same question as elsewhere:
    # will this binary run HERE?
    if [ "$build" = 1 ] || { [ "$all" = 1 ] && { ! command -v srsenb >/dev/null 2>&1 \
            || { [ "$SRS_ARCH" != native ] && ! _srs_isa_portable; } \
            || [ ! -x "$O5GS_PREFIX/bin/open5gs-mmed" ]; }; }; then
        # A failed build does not stop the rest (configs, launchers: the bench
        # must stay usable without the 4G) but the return code reports it.
        # The callers handle it (`|| true`, `|| inst_hint`): addition.sh,
        # update.sh, install_modules/50-build.sh.
        lte_build || rc=1
    fi
    [ "$configs" = 1 ] && lte_configs "$force"
    [ "$launchers" = 1 ] && lte_launchers
    # The WebUI goes through npm and the network: never in the default path,
    # only on request (or with --all, which is already the long path).
    if [ "$webui" = 1 ] || [ "$all" = 1 ]; then lte_webui || rc=1; fi
    return $rc
}

# Sourced (addition.sh, update.sh) or executed (ISO, command line).
if [ "${BASH_SOURCE[0]}" = "$0" ]; then
    [ "$(id -u)" -eq 0 ] || { echo "osmo-lte-install : root requis" >&2; exit 1; }
    osmo_lte_install "$@"
fi
```

2.261 /opt/GSM/osmo-operator/tools/osmo-lte.sh

19881 octets, 397 lignes → 397 lignes

```
#!/bin/bash
# ============================================================================
# osmo-lte.sh - THE BENCH 4G: Open5GS (the core), srsENB and srsUE on the
# ZeroMQ radio, started IN ORDER AND ON TIME.
#
# WHY A LAUNCHER, when three command lines are enough. Because the three
# lines have two traps:
#
#   1. "fail_on_disconnect=true" on the eNB. It is the argument of the doc,
#      and it does what it says: when the eNB receives NO sample from the UE
#      for 2000 ms (ZMQ_TIMEOUT_MS, rf_zmq_imp_trx.h), its radio STOPS. The
#      process stays, the S1 to the core stays, but nothing goes out on ZeroMQ
#      any more (a hundred bytes each way, the ZMTP handshake and nothing
```

```
#      else). An srsUE started ONE MINUTE after the eNB finds a dead radio:
#      "Attaching UE..." and never "Found Cell". Here the UE starts half a
#      second after the eNB.
#
#   2. The SIB3. sib.conf programs the SIBs sent through "si_mapping_info";
#      putting [ 7 ] (the SIB7, the GERAN list of the CSFB) INSTEAD of [ 3 ]
#      removes the SIB3. srsUE only deems a cell "suitable" IF it has the
#      SIB3 (srsue rrc.cc, cell_selection_criteria: has_sib3()): "Failed to
#      configure serving cell", T3410 expires, five attempts, nothing. It
#      must be [ 3, 7 ] - checked before starting.
#
# THE CORE IS OPEN5GS (tools/osmo-epc.sh: mongod, ogstun, the eight daemons,
# the SGs to OsmoMSC for the CSFB). The MME listens for the S1 on 127.0.0.2 -
# that is what enb.conf says (mme_addr). And the "ue1" netns of the UE lives
# in /run: it disappears at reboot, it is recreated here.
#
# THE CONFIGS ARE THOSE OF THE REPOSITORY (configs/srsran/*.conf, user_db.csv),
# installed in root's ~/.config/srsran by tools/osmo-lte-install.sh. They
# carry the ZeroMQ radio (device_name / device_args: ports 2000/2001,
# base_srate 11.52e6): they are NOT repeated on the command line, except
# OSMO_LTE_SRATE to force another rate. The srs* are started FROM this
# directory: sib.conf / rr.conf / rb.conf are resolved there, and the .ctxt
# (the NAS context srsue saves in its cwd) lives there too.
#
#   osmo-lte start      core (osmo-epc), then eNB, then UE (refused if one already runs)
#   osmo-lte start-gui  THE SAME LAUNCH, WITH THE srsGUI TRACES ON SCREEN
#                       (constellation, spectrum, PDSCH): --gui.enable on the eNB
#                       and the UE, and a DISPLAY. Never through systemd - a
#                       system unit has no screen; it is a direct launch.
#   osmo-lte traces     THE TRACES ALONE: restarts eNB and UE with srsGUI, WITHOUT
#                       touching the Open5GS core (which stays up, subscribers
#                       included). srsGUI is not a separate app: these are Qt
#                       windows opened by srsenb/srsue themselves (--gui.enable),
#                       they cannot be attached to an already running eNB - hence
#                       the restart of the radio, and of the radio only.
#                       Nothing running? Then it is a full start-gui.
#   osmo-lte stop       stops eNB and UE (the core stays: "osmo-epc stop")
#   osmo-lte restart    stop then start
#                       start / stop / restart without root: the script
#                       RE-RUNS ITSELF through pkexec (sudo fallback) instead
#                       of refusing - "osmo-lte stop" typed in a session
#                       terminal does stop the 4G.
#   osmo-lte status     processes, S1, ZeroMQ throughput, UE address
#   osmo-lte log        the logs, live
#   osmo-lte toggle     THE ICON: the osmo-lte service (systemd) - starts it if
#                       stopped, stops it if running (pkexec without root).
#                       "running" = the service active OR the daemons (srsenb,
#                       srsue, open5gs-mmed) up: a 4G started by "sudo osmo-lte
#                       start" or start-gui, unknown to the unit, is also
#                       stopped by the icon. The stop turns EVERYTHING off (eNB,
#                       UE, core).
#   osmo-lte up|down    the service, explicitly (right-click Actions)
#   osmo-lte journal    journalctl -u osmo-lte -f
#   osmo-lte install    installs the configs and the launchers (tools/osmo-lte-install.sh)
# ============================================================================
set -u

REPO="${OSMO_REPO:-/opt/GSM/osmo-operator}"
SRS_DIR="${OSMO_SRSRAN_DIR:-/root/.config/srsran}"
NETNS="${OSMO_LTE_NETNS:-ue1}"
ZMQ_ENB="${OSMO_ZMQ_PORT:-2000}"
ZMQ_UE="${OSMO_ZMQ_PORT_UE:-2001}"
SRATE="${OSMO_LTE_SRATE:-}"           # vide = ce que disent enb.conf / ue.conf
LOGDIR="${OSMO_LTE_LOGDIR:-/tmp}"
BUILD="${OSMO_SRSRAN_BUILD:-/opt/LTE/srsRAN_4G/build}"
EPC="${OSMO_EPC:-/usr/local/bin/osmo-epc}"
```

```
[ -x "$EPC" ] || EPC="$REPO/tools/osmo-epc.sh"
MME_S1="${OSMO_LTE_MME:-127.0.0.2}"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'
_say()  { echo -e "  ${CYAN}→${NC} $*"; }
_ok()   { echo -e "  ${GREEN}✓${NC} $*"; }
_warn() { echo -e "  ${YELLOW}!${NC} $*"; }
_err()  { echo -e "  ${RED}✗${NC} $*" >&2; }

_root() { [ "$(id -u)" -eq 0 ] && return 0; _err "il faut root (netns, tun, SCTP) : sudo $0 $*"; return 1; }

# THE ICON GESTURE. The 4G lives in services/osmo-lte.service (Open5GS + eNB
# + UE); the desktop icon (data/desktop/osmo-lte.desktop) calls "osmo-lte
# toggle": one click starts, the next one stops. From a session without
# root, pkexec opens the password window (sudo fallback); the result goes
# out as a notification, there is no terminal.
SVC="${OSMO_LTE_SERVICE:-osmo-lte.service}"
ICONE=/usr/share/osmo-operator/icons/osmo-lte.svg
_notif() { command -v notify-send >/dev/null 2>&1 && notify-send -i "$ICONE" "4G du banc" "$1" 2>/dev/null ||
 ↪  true; _say "$1"; }
_lte_debout() {
    case "$(systemctl is-active "$SVC" 2>/dev/null)" in active|activating) return 0 ;; esac
    local d
    for d in srsenb srsue open5gs-mmed; do pgrep -x "$d" >/dev/null 2>&1 && return 0; done
    return 1
}
# What _svc does AS ROOT, in a single pass (a single password window): the
# unit, then what it did not carry.
_svc_root() {
    local act="$1" rc=0
    case "$act" in
        start)
            # the unit log scrolls during the start: it is what the launch
            # window reads ([n/4] markers of lte_start)
            journalctl -u "$SVC" -f -n 0 -o cat 2>/dev/null & local _j=$!
            systemctl start "$SVC"; rc=$?
            sleep 1; kill "$_j" 2>/dev/null; wait "$_j" 2>/dev/null ;;
        stop)
            case "$(systemctl is-active "$SVC" 2>/dev/null)" in
                active|activating) systemctl stop "$SVC"; rc=$? ;;
            esac
            # a 4G started outside the unit (sudo osmo-lte start, start-gui):
            # turned off the same way - eNB, UE, then the core
            if _lte_debout; then
                lte_stop || rc=1
                "$EPC" stop || rc=1
            fi ;;
    esac
    return "$rc"
}
_svc() {
    local act="$1" rc
    if [ "$act" = toggle ]; then
        if _lte_debout; then act=stop; else act=start; fi
    fi
    [ "$act" = start ] && _notif "demarrage de la 4G (Open5GS, eNB, UE)... une trentaine de secondes"
    if [ "$(id -u)" -eq 0 ]; then _svc_root "$act"; rc=$?
    elif command -v pkexec >/dev/null 2>&1; then pkexec /bin/bash "$0" _svc-root "$act"; rc=$?
    else sudo /bin/bash "$0" _svc-root "$act"; rc=$?; fi
    if [ "$rc" -eq 0 ]; then
        [ "$act" = start ] && _notif "4G en marche : UE attache dans $NETNS (osmo-lte status)" || _notif "4G
         ↪  arretee (eNB, UE et coeur)"
    else
        _notif "echec ($act, code $rc) - journalctl -u $SVC"
    fi
    return "$rc"
```

```
}

# start / stop / restart WITHOUT root: re-run elevated, DISPLAY kept (pkexec
# clears it, and the icon / the session terminal have one).
_eleve() {
    [ "$(id -u)" -eq 0 ] && return 0
    _say "elevation (pkexec) pour « $* »"
    if command -v pkexec >/dev/null 2>&1; then
        exec pkexec /usr/bin/env DISPLAY="${DISPLAY:-}" XAUTHORITY="${XAUTHORITY:-}" /bin/bash "$0" "$@"
    fi
    exec sudo /bin/bash "$0" "$@"
}

# ── THE "WITH TRACES" MODE (srsGUI) ──────────────────────────────────────
# srsRAN is built with ENABLE_GUI=ON (srsGUI, installed before it by
# tools/osmo-lte-install.sh): srsenb and srsue CAN trace their constellation,
# their spectrum and their PDSCH in real time. They only DO it when asked -
# "[gui] enable" of enb.conf/ue.conf is false, and it is not changed: the
# bench 4G normally starts through a systemd service, which has no screen,
# and a Qt window that cannot open would make a working start fail.
#
# Hence TWO modes, and the normal mode stays EXACTLY what it was:
#     "start"      what the service does, nothing on screen
#     "start-gui"  the same launch + --gui.enable=1 on both
#   (these two labels deliberately do not begin with "osmo-lte start": the
#    help at the bottom of the file is a `sed` range on that pattern, and a
#    second occurrence would make it print the whole script.)
# The only difference in lte_start is an empty array when GUI=0.
GUI=0

# Qt needs an X server and a cookie. The traces mode is started from the
# desktop (icon, session terminal) but RUNS AS ROOT: DISPLAY and XAUTHORITY
# do not always follow (pkexec clears the environment, sudo too depending on
# env_reset). They are recovered: the X socket of the running server, and
# the cookie of the ACTIVE graphical session.
_display_ou_rien() {
    if [ -z "${DISPLAY:-}" ]; then
        local s n=""
        for s in /tmp/.X11-unix/X*; do
            [ -S "$s" ] || continue
            n="${s##*/X}"; break
        done
        [ -n "$n" ] || { _err "aucun serveur X (/tmp/.X11-unix vide) : le mode traces demande un ecran - « $0
        ↪  start » sans traces"; return 1; }
        export DISPLAY=":$n"
        _say "DISPLAY absent : on prend $DISPLAY"
    fi
    if [ -z "${XAUTHORITY:-}" ] || [ ! -r "${XAUTHORITY:-}" ]; then
        local uid f
        uid="$(loginctl list-sessions --no-legend 2>/dev/null | awk '$0 ~ / active / {print $2; exit}')"
        [ -n "$uid" ] || uid="$(loginctl list-sessions --no-legend 2>/dev/null | awk 'NR==1{print $2}')"
        for f in "/run/user/$uid/gdm/Xauthority" "/run/user/$uid/.Xauthority" \
                 "$(getent passwd "${uid:-0}" | cut -d: -f6)/.Xauthority"; do
            [ -n "$uid" ] || break
            [ -r "$f" ] && { export XAUTHORITY="$f"; break; }
        done
    fi
    # Qt would follow WAYLAND_DISPLAY if it lingers in the environment, and
    # would fail as root on the user's wayland socket: xcb is imposed.
    export QT_QPA_PLATFORM=xcb
    # An srsenb not linked to srsGUI accepts --gui.enable without showing
    # anything: say it rather than let the user search.
    local b
    b="$(_bin srsenb 2>/dev/null)"
    if [ -n "$b" ] && ! ldd "$b" 2>/dev/null | grep -q libsrsgui; then
        _warn "srsenb n est pas lie a srsGUI : aucune trace ne s affichera - « osmo-lte-install --build »"
```

```
    fi
    return 0
}

# The traces mode is started from the session, not from systemd: it is the
# one that fetches root, taking the screen along (same gesture as the bench
# icon, launch/osmo-launch.sh: pkexec env DISPLAY=... XAUTHORITY=...).
_root_avec_display() {
    local act="${1:-start-gui}"
    [ "$(id -u)" -eq 0 ] && return 0
    _display_ou_rien || return 1
    _say "elevation (pkexec) en gardant $DISPLAY"
    if command -v pkexec >/dev/null 2>&1; then
        exec pkexec /usr/bin/env DISPLAY="$DISPLAY" XAUTHORITY="${XAUTHORITY:-}" \
             QT_QPA_PLATFORM=xcb "$0" "$act"
    fi
    exec sudo DISPLAY="$DISPLAY" XAUTHORITY="${XAUTHORITY:-}" QT_QPA_PLATFORM=xcb "$0" "$act"
}

# The installed binary first, the local build otherwise.
_bin() {
    local n="$1"
    command -v "$n" 2>/dev/null && return 0
    for f in "$BUILD/srs${n#srs}/src/$n" "$BUILD/$n/src/$n"; do [ -x "$f" ] && { echo "$f"; return 0; }; done
    return 1
}

# The SIB3 must be programmed (trap no. 2 of the header).
_verifier_sib() {
    local sib="$SRS_DIR/sib.conf"
    [ -f "$sib" ] || { _warn "pas de $sib : srsENB prendra ses valeurs par defaut"; return 0; }
    local map
    map="$(grep -oE 'si_mapping_info\s*=\s*\[[^]]*\]' "$sib" | head -1)"
    if [ -n "$map" ] && ! printf '%s' "$map" | grep -qE '\[\s*(3|[0-9]+\s*,\s*)*3(\s*,|\s*\])'; then
        _err "sib.conf : « $map » ne programme pas le SIB3 - srsUE ne s attachera jamais (il faut [ 3, 7 ])"
        return 1
    fi
    _ok "sib.conf : $map"
}

# The configs: those of ~/.config/srsran, else those of the repository, installed.
_configs() {
    [ -f "$SRS_DIR/enb.conf" ] && [ -f "$SRS_DIR/ue.conf" ] && return 0
    [ -d "$REPO/configs/srsran" ] || { _err "pas de configs srsRAN ($SRS_DIR, ni $REPO/configs/srsran)"; return 1;
    ↪  }
    mkdir -p "$SRS_DIR"
    cp -n "$REPO/configs/srsran/"* "$SRS_DIR/"
    _ok "configs srsRAN posees depuis le depot dans $SRS_DIR"
}

lte_tourne() { pgrep -x "$1" >/dev/null 2>&1; }

lte_start() {
    _root start || return 1
    local enb ue
    enb="$(_bin srsenb)" || { _err "srsenb introuvable (dpkg -i osmo-build-srsran, ou osmo-lte-install --build)";
    ↪  return 1; }
    ue="$(_bin srsue)"   || { _err "srsue introuvable"; return 1; }
    for p in srsenb srsue; do
        lte_tourne "$p" && { _err "$p tourne deja (pid $(pgrep -x "$p" | head -1)) - « $0 restart » pour repartir
        ↪  propre"; return 1; }
    done
    _configs || return 1
    _verifier_sib || return 1
    # the desktop follows: the antenna rises in the wallpaper (tools/osmo-fond.py
    # re-runs itself under the desktop user; the return to the city, at stop,
```

```
    # is done by osmo-panel.py when nothing runs any more)
    [ -x "$REPO/tools/osmo-fond.py" ] && ("$REPO/tools/osmo-fond.py" antenne >/dev/null 2>&1 &)
    ip netns list 2>/dev/null | grep -qw "$NETNS" || { ip netns add "$NETNS" && _ok "espace reseau $NETNS cree"; }

    # The "[n/4]" markers: the desktop launch window (tools/osmo-gtk-run.py
    # --progress) uses them as its progress bar.
    echo "[1/4] coeur Open5GS"
    # The core: Open5GS, through osmo-epc (idempotent: what already runs stays).
    "$EPC" start | sed 's/^/  /'
    for _ in $(seq 1 20); do ss -Sln 2>/dev/null | grep -q "$MME_S1:36412 " && break; sleep 0.25; done
    ss -Sln 2>/dev/null | grep -q "$MME_S1:36412 " && _ok "MME : S1 en ecoute sur $MME_S1" \
        || _warn "MME : le S1 n ecoute pas sur $MME_S1 (voir « osmo-epc status », osmo-epc log mme)"

    export HOME=/root                       # srs* lisent ~/.config/srsran : celui de root
    cd "$SRS_DIR" || return 1               # sib.conf & co. relatifs, et le .ctxt de srsue
    rm -f .ctxt                             # un vieux contexte fait echouer l attach apres un redemarrage du coeur

    # The traces: nothing at all when GUI=0 - the normal mode is intact.
    local -a gui_opt=()
    [ "$GUI" = 1 ] && gui_opt=(--gui.enable=1)
    local -a enb_rf=() ue_rf=()
    if [ -n "$SRATE" ]; then
        enb_rf=(--rf.device_name=zmq --rf.device_args="fail_on_disconnect=true,tx_port=tcp://*:$ZMQ_ENB
        ↪  ,rx_port=tcp://localhost:$ZMQ_UE,id=enb,base_srate=$SRATE")
        ue_rf=(--rf.device_name=zmq
        ↪  --rf.device_args="tx_port=tcp://*:$ZMQ_UE,rx_port=tcp://localhost:$ZMQ_ENB,id=ue,base_srate=$SRATE")
    fi
    echo "[2/4] station de base srsENB"
    # The eNB, and the UE right behind: see trap no. 1.
    setsid "$enb" "$SRS_DIR/enb.conf" "${enb_rf[@]}" "${gui_opt[@]}" \
        --log.filename="$LOGDIR/osmo-lte-enb.log" >"$LOGDIR/osmo-lte-enb.console" 2>&1 </dev/null &
    sleep 0.5
    echo "[3/4] mobile srsUE"
    setsid "$ue" "$SRS_DIR/ue.conf" "${ue_rf[@]}" "${gui_opt[@]}" \
        --gw.netns="$NETNS" --log.filename="$LOGDIR/osmo-lte-ue.log" >"$LOGDIR/osmo-lte-ue.console" 2>&1
        ↪  </dev/null &
    _ok "srsENB puis srsUE lances (ZeroMQ $ZMQ_ENB/$ZMQ_UE, UE dans $NETNS${SRATE:+, base_srate
    ↪  $SRATE})${gui_opt:+ ${GREEN}avec les traces srsGUI sur $DISPLAY${NC}}"

    echo "[4/4] attach du mobile"
    _say "attente de l attach (30 s max)..."
    local i ip
    for i in $(seq 1 60); do
        ip="$(ip -n "$NETNS" -4 -o addr show 2>/dev/null | awk '/tun_srsue/ {print $4}')"
        [ -n "$ip" ] && break
        sleep 0.5
    done
    if [ -n "$ip" ]; then
        _ok "UE attache : $ip (tun_srsue dans $NETNS) en $((i / 2)) s"
    else
        _err "pas d attach en 30 s - voir $LOGDIR/osmo-lte-ue.console et « $0 status »"
        grep -E "Found|Attach|fail|Reject" "$LOGDIR/osmo-lte-ue.console" | tail -4 | sed 's/^/      /'
        return 1
    fi
}

lte_stop() {
    _root stop || return 1
    local p n=0
    for p in srsue srsenb; do
        if lte_tourne "$p"; then pkill -x "$p"; n=$((n + 1)); fi
    done
    sleep 2
    for p in srsue srsenb; do lte_tourne "$p" && pkill -9 -x "$p"; done
    [ "$n" -gt 0 ] && _ok "$n processus srs* arrete(s)" || _say "rien ne tournait"
    # The S1 takes a moment to close on the SCTP side; the next eNB needs it.
```

```
    sleep 1
}

lte_status() {
    echo -e "\033[1msrsRAN sur ZeroMQ + Open5GS - la 4G du banc\033[0m"
    local p
    for p in open5gs-mmed srsenb srsue; do
        lte_tourne "$p" && _ok "$p : pid $(pgrep -x "$p" | head -1)" || _warn "$p : arrete"
    done
    ss -San 2>/dev/null | grep -q "ESTAB.*:36412" && _ok "S1 : eNB associe au MME" || _warn "S1 : pas d
     ↪  association eNB-MME"
    # The eNB -> UE samples leave from the local socket :$ZMQ_ENB of the eNB
    # (bytes_sent); what it RECEIVES is only the UE requests.
    local a b
    a="$(ss -tin state established "( sport = :$ZMQ_ENB )" 2>/dev/null | grep -oE "bytes_sent:[0-9]+" | head -1 |
     ↪  cut -d: -f2)"
    if [ -n "$a" ]; then
        sleep 1
        b="$(ss -tin state established "( sport = :$ZMQ_ENB )" 2>/dev/null | grep -oE "bytes_sent:[0-9]+" | head
         ↪  -1 | cut -d: -f2)"
        local ko=$(( (${b:-0} - a) / 1024 ))
        if [ "$ko" -gt 100 ]; then _ok "ZeroMQ : $ko Ko/s d echantillons eNB -> UE"
        else _warn "ZeroMQ : $ko Ko/s - la radio de l eNB est morte (fail_on_disconnect) : « $0 restart »"; fi
    else
        _warn "ZeroMQ : aucun lien eNB <-> UE"
    fi
    local ip
    ip="$(ip -n "$NETNS" -4 -o addr show 2>/dev/null | awk '/tun_srsue/ {print $4}')"
    [ -n "$ip" ] && _ok "UE : attache, $ip dans $NETNS" || _warn "UE : pas attache (pas de tun_srsue dans $NETNS)"
}

case "${1:-status}" in
    start|stop|restart) _eleve "$@" ;;      # hors root : on repasse ici en root
esac
case "${1:-status}" in
    start)   lte_start ;;
    start-gui|gui|start-display)
        # The traces mode: root WITH the screen, then the normal launch + GUI=1.
        _root_avec_display || exit 1
        _display_ou_rien || exit 1
        GUI=1
        lte_start ;;
    traces|gui-seul)
        # The traces without restarting the whole 4G: only the radio restarts
        # with srsGUI; lte_start leaves the core as is (osmo-epc start is
        # idempotent). Started outside the unit: "osmo-lte down" / the icon
        # know how to stop a radio started this way (_svc_root).
        _root_avec_display traces || exit 1
        _display_ou_rien || exit 1
        GUI=1
        if lte_tourne srsenb || lte_tourne srsue; then
            _say "radio relancee avec les traces srsGUI - le coeur Open5GS reste debout"
            lte_stop
        fi
        lte_start ;;
    stop)    lte_stop ;;
    restart) lte_stop; lte_start ;;
    status)  lte_status ;;
    log)     exec tail -F "$LOGDIR"/osmo-lte-{enb,ue}.console ;;
    toggle|bascule) _svc toggle ;;
    _svc-root) _root _svc-root || exit 1; _svc_root "$2" ;;
    up)      _svc start ;;
    down)    _svc stop ;;
    journal) exec journalctl -u "$SVC" -f --no-pager ;;
    install) shift; exec bash "$REPO/tools/osmo-lte-install.sh" "$@" ;;
    *)       sed -n '/^#   osmo-lte start/,/^#   osmo-lte install/p' "$0" | sed 's/^# \{0,2\}//'; exit 2 ;;
```

```
esac
```

2.262 /opt/GSM/osmo-operator/tools/osmo-moon.py

6913 octets, 161 lignes → 161 lignes

```
#!/usr/bin/env python3
# osmo-moon.py - the ANIMATED rider/moon, placed under Conky (top-right column).
#
# Same window mechanism as tools/osmo-panel.py: a GTK window of DESKTOP type
# (Gdk.WindowTypeHint.DESKTOP) - under all windows, above the wallpaper,
# undecorated, sticky, transparent (argb). It has NOTHING TO DO with the Conky
# configuration (osmo-conky.conf): it is a SEPARATE element, aligned VERTICALLY
# with Conky (same top-right column, same width 400), stuck BELOW it.
#
# The gif (820x560, 240 frames) is resized PROPORTIONALLY to the requested
# width (default 400, like Conky): height = width * 560/820. The animation is
# played frame by frame (GdkPixbuf.PixbufAnimation + iterator), each frame
# being scaled before display.
#
# Settings through environment variables:
#   OSMO_MOON_GIF     path of the gif    (default: configs/conky/cavalier_lune.gif)
#   OSMO_MOON_W       width in px        (default: 400, the width of Conky)
#   OSMO_MOON_GAP_X   right margin       (default: 24, the gap_x of Conky)
#   OSMO_MOON_GAP_Y   y of the gif top   (default: 600 = under Conky; to adjust
#                                          to the real height of Conky)
#
# Started by /usr/local/bin/osmo-desktop-panel (like osmo-panel.py + conky).
import os
import sys

# THE DESKTOP WINDOW REQUIRES X11, NOT WAYLAND.
# Under Wayland, GTK3 ignores window.move() AND Gdk.WindowTypeHint.DESKTOP: the
# compositor places the window where it wants (in the middle, in front). The
# protocol gives the client no way to place itself at absolute coordinates
# (it would need gtk-layer-shell, absent on mutter). XWayland is used instead,
# where X11 positioning is honoured, exactly like the neighbouring Conky
# (own_window type desktop). Only forced when a DISPLAY exists: without an X
# server, the default behaviour is better than a failure at startup.
X11_FORCE = os.environ.get("XDG_SESSION_TYPE") == "wayland" and bool(os.environ.get("DISPLAY"))
if X11_FORCE:
    os.environ["GDK_BACKEND"] = "x11"

import gi

gi.require_version("Gtk", "3.0")
gi.require_version("Gdk", "3.0")
gi.require_version("GdkPixbuf", "2.0")
from gi.repository import Gdk, GdkPixbuf, GLib, Gtk  # noqa: E402

# THE DESKTOP TYPE DOES NOT WORK ON XWAYLAND. There, a window of type
# _NET_WM_WINDOW_TYPE_DESKTOP is put by GNOME Shell in the desktop layer,
# UNDER the full-screen icons window (DING extension) - which swallows all
# clicks: the panel would show but its buttons would not respond. A NORMAL
# window is kept instead, simply held below the others (keep_below + stick +
# out of the taskbar): same rendering, and the clicks arrive. On native X11
# (the live key) the DESKTOP type works and is kept.
TYPE_HINT = Gdk.WindowTypeHint.NORMAL if X11_FORCE else Gdk.WindowTypeHint.DESKTOP
# "BELOW" DOES NOT WORK ON XWAYLAND EITHER: with _NET_WM_STATE_BELOW, GNOME
# Shell puts the XWayland window under its own desktop layer (native Wayland),
# which is full screen and takes all clicks - the window stays visible but
# completely inert. Under XWayland "always below" is therefore given up:
# ordinary window, sticky and out of the taskbar. It comes to the front when
# clicked, the price to pay for being clickable at all. Set
```

```
# OSMO_DESKTOP_BELOW=1 to get the window under everything (clicks lost);
# on native X11 nothing changes.
KEEP_BELOW = (not X11_FORCE) or os.environ.get("OSMO_DESKTOP_BELOW") == "1"

import cairo  # noqa: E402

REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
GIF = os.environ.get("OSMO_MOON_GIF", os.path.join(REPO, "configs/conky/cavalier_lune.gif"))
WIN_W = int(os.environ.get("OSMO_MOON_W", "400"))     # same width as Conky
GAP_X = int(os.environ.get("OSMO_MOON_GAP_X", "24"))  # same gap_x as Conky
GAP_Y = int(os.environ.get("OSMO_MOON_GAP_Y", "600")) # under Conky (to adjust)


def target_size(anim):
    """Imposed width, height PROPORTIONAL to the native ratio of the gif."""
    gw, gh = anim.get_width(), anim.get_height()
    w = WIN_W
    h = max(1, round(w * gh / gw))
    return w, h


class Moon(Gtk.Window):
    def __init__(self):
        super().__init__(title="osmo-moon")
        try:
            self.anim = GdkPixbuf.PixbufAnimation.new_from_file(GIF)
        except GLib.Error as e:
            print(f"[moon] gif introuvable/illisible: {GIF} ({e})", file=sys.stderr)
            sys.exit(1)
        self.w, self.h = target_size(self.anim)

        # top-right column of the primary monitor, like Conky (top_right)
        disp = Gdk.Display.get_default()
        mon = disp.get_primary_monitor() or disp.get_monitor(0)
        g = mon.get_geometry()
        self.x = g.x + g.width - GAP_X - self.w
        self.y = g.y + GAP_Y
        print(f"[moon] {self.w}x{self.h} @ {self.x},{self.y} (gif
         ↪  {self.anim.get_width()}x{self.anim.get_height()})",
              flush=True)

        # window of DESKTOP type (cf. osmo-panel.py)
        self.set_type_hint(TYPE_HINT)
        self.set_decorated(False)
        self.set_resizable(False)
        self.set_keep_below(KEEP_BELOW)
        self.set_skip_taskbar_hint(True)
        self.set_skip_pager_hint(True)
        self.stick()
        self.set_default_size(self.w, self.h)
        self.set_size_request(self.w, self.h)
        self.move(self.x, self.y)

        screen = self.get_screen()
        visual = screen.get_rgba_visual()
        if visual and screen.is_composited():
            self.set_visual(visual)
        self.set_app_paintable(True)
        self.connect("draw", self.on_draw)
        self.connect("destroy", Gtk.main_quit)

        self.image = Gtk.Image()
        self.add(self.image)

        # animated playback: frame iterator, proportional scaling
        self.it = self.anim.get_iter(None)
```

```
        self.show_all()
        self._pos = (self.x, self.y)
        self._pin()
        self.connect("map-event", self._pin)
        GLib.timeout_add(500, self._pin)
        self._render()

    # REPOSITION AFTERWARDS. Even under X11, the window manager may move the
    # window when it maps it (mutter does so for undecorated windows it does
    # not recognise as desktop). A single move() before show_all() does not
    # hold: the window is repositioned on map-event, then one last time half
    # a second later.
    def _pin(self, *_a):
        self.move(*self._pos)
        return False
    def on_draw(self, _w, cr):
        cr.set_operator(cairo.OPERATOR_SOURCE)
        cr.set_source_rgba(0, 0, 0, 0)
        cr.paint()
        return False

    def _render(self):
        self.it.advance(None)
        frame = self.it.get_pixbuf()
        scaled = frame.scale_simple(self.w, self.h, GdkPixbuf.InterpType.BILINEAR)
        self.image.set_from_pixbuf(scaled)
        delay = self.it.get_delay_time()   # ms; -1 = still image
        GLib.timeout_add(delay if delay > 0 else 100, self._render)
        return False


def main():
    Moon()
    Gtk.main()


if __name__ == "__main__":
    main()
```

2.263 /opt/GSM/osmo-operator/tools/osmo-ofono-bridge.py

10730 octets, 274 lignes → 274 lignes

```
#!/usr/bin/env python3
# osmo-ofono-bridge.py - CALLS AND SMS OF THE BENCH, FROM THE COMMAND LINE.
#
# Drives oFono from a terminal or an icon. The phone of the bench is the
# postmarketOS/Phosh VM, which has ModemManager and talks AT to the modem of
# the bench (tools/osmo-phonesim-banc.py --connect); this program only handles
# the oFono side:
#
#   osmo-sms-send <number> <text>    -> org.ofono.MessageManager.SendMessage
#   osmo-call <number>|answer|hangup -> org.ofono.VoiceCallManager
#
# Started without --once, it HOLDS the line: it logs the SMS and calls the
# bench pushes to it, publishes the state in /run/osmo-ril/state.json (the
# panel and Conky read it) and executes the commands dropped in the queue by
# the two launchers above.
#
# DATA. Not yet in service on the bench (no PDP context established). The path
# is nevertheless WIRED here: --data activates the oFono context and publishes
# the obtained interface. Without --data nothing is touched.
#
# Settings through environment variables:
```

```
#   OSMO_OFONO_MODEM    oFono modem path        (default: the first found)
#   OSMO_BRIDGE_SPOOL   command queue           (default: /run/osmo-ril/spool)
#   OSMO_BRIDGE_STATE   published state (json)  (default: /run/osmo-ril/state.json)
#
import argparse
import json
import os
import sys
import time

from gi.repository import Gio, GLib

MODEM = os.environ.get("OSMO_OFONO_MODEM", "")
RUN = os.environ.get("OSMO_RIL_DIR", "/run/osmo-ril")
SPOOL = os.environ.get("OSMO_BRIDGE_SPOOL", os.path.join(RUN, "spool"))
STATE = os.environ.get("OSMO_BRIDGE_STATE", os.path.join(RUN, "state.json"))

OFONO = "org.ofono"
IF_MANAGER = "org.ofono.Manager"
IF_MODEM = "org.ofono.Modem"
IF_SMS = "org.ofono.MessageManager"
IF_VOICE = "org.ofono.VoiceCallManager"
IF_CALL = "org.ofono.VoiceCall"
IF_CONN = "org.ofono.ConnectionManager"
IF_CTX = "org.ofono.ConnectionContext"


def log(*a):
    print("[ril]", *a, flush=True)


# ── THE BUS ──────────────────────────────────────────────────────────
class Bus:
    """The bare minimum of system D-Bus, without dbus-python (absent from the
    image): Gio is enough and comes with PyGObject, already there for the panel."""

    def __init__(self):
        self.c = Gio.bus_get_sync(Gio.BusType.SYSTEM, None)

    def call(self, path, iface, method, params=None, timeout=20000):
        return self.c.call_sync(OFONO, path, iface, method, params, None,
                                Gio.DBusCallFlags.NONE, timeout, None)

    def subscribe(self, iface, signal, cb, path=None):
        return self.c.signal_subscribe(OFONO, iface, signal, path, None,
                                       Gio.DBusSignalFlags.NONE, cb)


# ── THE PUBLISHED STATE ──────────────────────────────────────────────
def publish(**kw):
    """What the rest of the bench reads: the panel and Conky."""
    try:
        os.makedirs(RUN, exist_ok=True)
        cur = {}
        if os.path.exists(STATE):
            with open(STATE) as f:
                cur = json.load(f)
        cur.update(kw)
        cur["ts"] = int(time.time())
        tmp = STATE + ".tmp"
        with open(tmp, "w") as f:
            json.dump(cur, f)
        os.replace(tmp, STATE)
    except Exception as e:                      # jamais fatal
        log("etat non publie :", e)
```

```
# ── THE BRIDGE ──────────────────────────────────────────────────────────
class Bridge:
    def __init__(self, bus, modem, with_data=False):
        self.bus = bus
        self.modem = modem
        self.with_data = with_data
        self.calls = {}

        # -- modem
    def power_on(self):
        for prop in ("Powered", "Online"):
            try:
                self.bus.call(self.modem, IF_MODEM, "SetProperty",
                              GLib.Variant("(sv)", (prop, GLib.Variant("b", True))))
            except GLib.Error as e:
                log("modem %s : %s" % (prop, e.message.split(":")[-1].strip()))

        # -- SMS
    def on_sms(self, _c, _s, _p, _i, _sig, params):
        text = params[0]
        info = params[1]
        sender = info.get("Sender", "?")
        when = info.get("LocalSentTime") or info.get("SentTime") or ""
        log("SMS de %s : %s" % (sender, text))
        publish(last_sms={"from": sender, "text": text, "when": when})

        # -- calls
    def on_call_added(self, _c, _s, _p, _i, _sig, params):
        path, props = params[0], params[1]
        num = props.get("LineIdentification", "?")
        state = props.get("State", "?")
        self.calls[path] = num
        log("appel %s (%s) %s" % (num, state, path))
        publish(call={"number": num, "state": state})

    def on_call_removed(self, _c, _s, _p, _i, _sig, params):
        path = params[0]
        num = self.calls.pop(path, "?")
        log("appel termine %s" % num)
        publish(call={"number": num, "state": "ended"})

        # -- data (wired, not enabled: see the header)
    def data_up(self):
        try:
            ctxs = self.bus.call(self.modem, IF_CONN, "GetContexts").unpack()[0]
        except GLib.Error as e:
            log("pas de ConnectionManager :", e.message.split(":")[-1].strip())
            return
        if not ctxs:
            log("aucun contexte PDP declare sur le modem")
            return
        path, props = ctxs[0]
        log("contexte %s (apn=%s)" % (path, props.get("AccessPointName", "?")))
        try:
            self.bus.call(path, IF_CTX, "SetProperty",
                          GLib.Variant("(sv)", ("Active", GLib.Variant("b", True))))
            props = self.bus.call(path, IF_CTX, "GetProperties").unpack()[0]
        except GLib.Error as e:
            log("contexte non active :", e.message.split(":")[-1].strip())
            publish(data={"up": False, "error": e.message})
            return
        st = props.get("Settings", {})
        log("data : if=%s addr=%s gw=%s" % (st.get("Interface"), st.get("Address"),
                                            st.get("Gateway")))
        publish(data={"up": True, "interface": st.get("Interface"),
```

```
                "address": st.get("Address"), "gateway": st.get("Gateway"),
                "dns": st.get("DomainNameServers", [])})

    # -- commands coming from osmo-sms-send / osmo-call
def command(self, line):
    try:
        verb, rest = (line.split(" ", 1) + [""])[:2]
    except ValueError:
        return
    verb = verb.strip().lower()
    if verb == "sms":
        num, _, text = rest.partition(" ")
        if not num or not text:
            log("usage: sms <numero> <texte>")
            return
        try:
            self.bus.call(self.modem, IF_SMS, "SendMessage",
                          GLib.Variant("(ss)", (num, text)))
            log("SMS envoye a %s" % num)
        except GLib.Error as e:
            log("envoi refuse :", e.message.split(":")[-1].strip())
    elif verb == "call":
        try:
            self.bus.call(self.modem, IF_VOICE, "Dial",
                          GLib.Variant("(ss)", (rest.strip(), "default")))
        except GLib.Error as e:
            log("appel refuse :", e.message.split(":")[-1].strip())
    elif verb == "answer":
        for path in list(self.calls):
            try:
                self.bus.call(path, IF_CALL, "Answer")
            except GLib.Error as e:
                log("decrochage refuse :", e.message.split(":")[-1].strip())
    elif verb == "hangup":
        try:
            self.bus.call(self.modem, IF_VOICE, "HangupAll")
        except GLib.Error as e:
            log("raccrochage refuse :", e.message.split(":")[-1].strip())
    elif verb == "data-up":
        self.data_up()
    else:
        log("commande inconnue : %s" % line.strip())

def watch_spool(self):
    """The queue: one line = one command. A fifo would be nicer but a plain
    file can be read and written from any script, including by hand during
    a session."""
    os.makedirs(RUN, exist_ok=True)
    if not os.path.exists(SPOOL):
        open(SPOOL, "w").close()
        os.chmod(SPOOL, 0o666)

    def tick():
        try:
            with open(SPOOL, "r+") as f:
                lines = f.read().splitlines()
                if lines:
                    f.seek(0)
                    f.truncate()
            for line in lines:
                if line.strip():
                    self.command(line)
        except Exception as e:
            log("file :", e)
        return True
```

```
        GLib.timeout_add(500, tick)


def first_modem(bus):
    modems = bus.call("/", IF_MANAGER, "GetModems").unpack()[0]
    if not modems:
        return ""
    return modems[0][0]


def main():
    ap = argparse.ArgumentParser(description="appels et SMS du banc par oFono")
    ap.add_argument("--data", action="store_true",
                    help="active aussi le contexte PDP (pas encore en service sur le banc)")
    ap.add_argument("--once", metavar="CMD",
                    help="passe une commande au pont deja lance (sms/call/answer/hangup)")
    args = ap.parse_args()

    if args.once:
        os.makedirs(RUN, exist_ok=True)
        with open(SPOOL, "a") as f:
            f.write(args.once.rstrip("\n") + "\n")
        return 0

    try:
        bus = Bus()
    except GLib.Error as e:
        print("[ril] bus systeme injoignable : %s" % e, file=sys.stderr)
        return 1

    modem = MODEM or first_modem(bus)
    if not modem:
        print("[ril] aucun modem oFono (ofonod demarre ? modem branche ?)", file=sys.stderr)
        publish(modem=None)
        return 1
    log("modem", modem)
    publish(modem=modem)

    br = Bridge(bus, modem, with_data=args.data)
    br.power_on()
    bus.subscribe(IF_SMS, "IncomingMessage", br.on_sms, path=modem)
    bus.subscribe(IF_SMS, "ImmediateMessage", br.on_sms, path=modem)
    bus.subscribe(IF_VOICE, "CallAdded", br.on_call_added, path=modem)
    bus.subscribe(IF_VOICE, "CallRemoved", br.on_call_removed, path=modem)
    br.watch_spool()
    if args.data:
        br.data_up()

    log("en ligne (SMS + appels ; data %s)" % ("demandee" if args.data else "non cablee"))
    GLib.MainLoop().run()
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

2.264 /opt/GSM/osmo-operator/tools/osmo-op.sh

11164 octets, 229 lignes → 229 lignes

```
#!/bin/bash
# ============================================================================
#  osmo-op - WHICH OPERATOR THE SCREEN IS LOOKING AT
# ============================================================================
```

```
#  The bench desktop has ONE panel (tools/osmo-panel.py) and ONE Conky
#  (tools/conky-osmo-status.sh), but a multi bench carries three operators
#  plus the hub. The choice lives in a single file, /run/osmo-fft/operator,
#  which both read at every refresh:
#
#      OP=2  MODE=docker  IP=172.20.0.12  NAME=osmo-operator-2
#      DASH=http://172.20.0.12:8080
#
#  This script is what WRITES it, and the only definition of "the operator
#  list" - the panel and the Conky refer to it, they do not recompute it.
#
#      osmo-op                 the current operator
#      osmo-op --list          the operators and their state
#      osmo-op --list-vue      the VISIBLE stops (what is running) and their state
#      osmo-op --next / --prev the next / previous operator  (the arrows)
#      osmo-op --set N         operator N
#
#  STOPPED OPERATORS COUNT TOO. The screen must be able to look at a stopped
#  operator: that is precisely when it is needed (to see why it did not
#  start). The state (active / stopped) is shown, not filtered; --list-vue
#  only keeps what is running, for the arrow cycle.
#
#  NO INVENTED OPERATORS, however. Without /etc/osmocom/osmo-multi.conf -
#  so on a single-operator bench - the list returns the native one only,
#  and the panel hides its arrows: there is nowhere to go. The arrows only
#  appear when the topology really declares several operators.
#  The hub (index "hub") and the 4G (index "lte") are extra stops after the
#  2G operators: no number, no FFT, their own view in the Conky.
# ============================================================================
set -u
RUN="${OSMO_FFT_DIR:-/run/osmo-fft}"
OP_FILE="$RUN/operator"
MULTI_CONF="${MULTI_CONF:-/etc/osmocom/osmo-multi.conf}"
DASH_PORT="${DASH_PORT:-8080}"
HUB_NAME="${MULTI_HUB_NAME:-osmo-inter-stp}"
M3UA_PORT="${MULTI_M3UA_PORT:-2908}"

# -- THE LIST: "idx mode ip" per line ----------------------------------------
# The topology first (MULTI_OPS of osmo-multi.conf, written by addition.sh);
# otherwise op 1 native + the next ones in docker, on the addressing plan of
# the hub (172.20.0.1<idx>, see start-multi.sh).
liste() {
    local specs="" idx mode ip
    if [ -r "$MULTI_CONF" ]; then
        specs="$(sed -nE 's/^MULTI_OPS="?([^"]*)"?.*/\1/p' "$MULTI_CONF" | head -1)"
    fi
    if [ -n "$specs" ]; then
        for s in $specs; do
            IFS=: read -r idx mode ip _ <<< "$s"
            [ -n "$idx" ] || continue
            echo "$idx ${mode:-docker} ${ip:-}"
        done
        # -- AND THE HUB, IN LAST POSITION ---------------------------------
        # The inter-STP is the fourth stop, after op3, before coming back to
        # op1: it is the node queried when the SS7 goes wrong. Its "index" is
        # its name (hub) and not a digit: it does not number an operator, and
        # the panel renders it as an SS7 view (the matrix, the ASPs, its log)
        # instead of the I/Q spectrum - an M3UA hub has no radio.
        _hub_ip="$(sed -nE 's/^MULTI_HUB_IP=([^ #]*).*/\1/p' "$MULTI_CONF" | head -1)"
        echo "hub interstp ${_hub_ip:-172.20.0.10}"
    else
        # No topology: a single operator, the native one. Nothing to cycle.
        echo "1 native "
    fi
    # -- AND THE 4G, IF INSTALLED ON THIS MACHINE ----------------------------
    # The bench carries a complete 4G (tools/osmo-lte.sh: Open5GS, srsENB and
```

```
    # srsUE on ZeroMQ). It is the last stop - after the 2G operators and the
    # hub - and the Conky gives it its own view (the EPC core daemons, the
    # eNB, the UE, the S1, the cell). Its "index" is "lte", like the hub's is
    # "hub": it does not number a 2G operator. No container, no FFT:
    # everything is native on the host.
    lte_installee && echo "lte lte "
}

# Is the 4G installed here? The binaries, where tools/osmo-lte-install.sh puts
# them (or the local srsRAN build).
lte_installee() {
    [ -x /usr/local/bin/srsenb ] && return 0
    [ -x "${OPEN5GS_PREFIX:-/opt/LTE/open5gs/install}/bin/open5gs-mmed" ] && return 0
    [ -x "${OSMO_SRSRAN_BUILD:-/opt/LTE/srsRAN_4G/build}/srsenb/src/srsenb" ]
}

# actif IDX MODE [IP]: is the operator running?
#
# "STOPPED" must not mean "I AM NOT ALLOWED TO SEE". The Conky and the panel
# run under the SESSION account, not root, and that account is not necessarily
# in the `docker` group:
#     $ docker ps
#     permission denied while trying to connect to the docker API at
#     unix:///var/run/docker.sock
# `docker ps 2>/dev/null | grep -qx` would then return false and the screen
# would report op2 and op3 STOPPED while `docker ps` as root shows them "Up".
# A probe that confuses "absent" and "invisible" lies.
#
# The two cases are told apart: if docker does not answer US, fall back to a
# probe that needs no privilege - the operator dashboard, on its IP on the
# hub network. The account is added to the `docker` group elsewhere
# (installer/calamares/modules/users.conf, addition.sh); this fallback covers
# the already open session, where the group has not taken effect yet.
actif() {
    if [ "$2" = lte ]; then
        # The 4G is running as soon as the core or the radio runs (osmo-lte status).
        pgrep -x open5gs-mmed >/dev/null 2>&1 || pgrep -x srsenb >/dev/null 2>&1 \
            || pgrep -x srsue >/dev/null 2>&1
    elif [ "$2" = interstp ]; then
        if docker ps --format '{{.Names}}' 2>/dev/null | grep -q .; then
            docker ps --format '{{.Names}}' 2>/dev/null | grep -qx "${HUB_NAME}"
        else
            # Without docker access: the hub M3UA port, in SCTP - a TCP probe
            # would be silent there, hence `ss` rather than a /dev/tcp.
            ss -an 2>/dev/null | awk -v p=":${M3UA_PORT}" \
               '$1=="sctp" && index($5,p) {f=1} END{exit !f}'
        fi
    elif [ "$2" = native ]; then
        pgrep -x osmo-bsc >/dev/null 2>&1 || pgrep -x osmo-stp >/dev/null 2>&1
    elif docker ps --format '{{.Names}}' 2>/dev/null | grep -q .; then
        docker ps --format '{{.Names}}' 2>/dev/null | grep -qx "osmo-operator-$1"
    elif [ -n "${3:-}" ]; then
        timeout 1 bash -c "echo >/dev/tcp/$3/${DASH_PORT}" 2>/dev/null
    else
        return 1
    fi
}

courant() { sed -n 's/^OP=//p' "$OP_FILE" 2>/dev/null | head -1; }

ecrire() {
    local idx="$1" mode="$2" ip="$3"
    # ── THE DIRECTORY BELONGS TO THE BENCH, NOT TO WHOEVER COMES FIRST ──────
    # /run/osmo-fft is normally created by osmo-fft-snap.service
    # (RuntimeDirectory, 0775, group sudo): the panel and the probes run under
    # the SESSION account and must be able to write there. This `mkdir -p` may
```

```
    # create it BEFORE the unit, as root; the same permissions as the unit are
    # applied so that the session probes (osmo-ts-probe.py among them) can
    # write. In session (non root) the chgrp fails harmlessly, the directory
    # already belongs to the right account.
    if [ ! -d "$RUN" ]; then
        mkdir -p "$RUN" 2>/dev/null || true
        chmod 0775 "$RUN" 2>/dev/null || true
        chgrp sudo "$RUN" 2>/dev/null || true
    fi
    local nom="osmo-operator-$idx"
    [ "$mode" = interstp ] && nom="$HUB_NAME"
    [ "$mode" = lte ] && nom="osmo-lte"
    printf 'OP=%s\nMODE=%s\nIP=%s\nNAME=%s\nDASH=http://%s:%s\n' \
        "$idx" "$mode" "$ip" "$nom" "${ip:-127.0.0.1}" "$DASH_PORT" > "$OP_FILE.tmp" \
        && mv -f "$OP_FILE.tmp" "$OP_FILE"
}

# ── THE VISIBLE STOPS: WHAT REALLY RUNS ON THIS MACHINE ──────────────────
# On a bench where only op1 is up, the cycle only shows what is running - a
# single `docker ps` tells it for the three. A stopped operator is therefore
# not reachable with the arrows; `osmo-op --set 2` still goes there, and
# `osmo-op --list` keeps showing ALL of them with their state - that is where
# "op2 arrete" is read.
#
# Two exceptions, which are not docker operators: the first stop (the native
# operator, THIS machine - its bench may be stopped without disappearing) and
# the 4G (installed here, running or not: that view is precisely the one that
# says how to start it).
liste_vue() {
    local premier=1 noms="" docker_ok=0
    if noms="$(docker ps --format '{{.Names}}' 2>/dev/null)"; then docker_ok=1; fi
    while read -r idx mode ip; do
        if [ "$premier" = 1 ] || [ "$mode" = lte ]; then
            echo "$idx $mode $ip"
        elif [ "$docker_ok" = 0 ]; then
            # docker does not answer us (not in the group): we do NOT know,
            # and a probe that confuses "absent" and "invisible" lies. Keep
            # the stop rather than wrongly hiding it.
            echo "$idx $mode $ip"
        elif [ "$mode" = interstp ]; then
            printf '%s\n' "$noms" | grep -qx "$HUB_NAME" && echo "$idx $mode $ip"
        else
            printf '%s\n' "$noms" | grep -qx "osmo-operator-$idx" && echo "$idx $mode $ip"
        fi
        premier=0
    done < <(liste)
}

# shift +1 / -1 in the list, wrapping around.
decale() {
    local pas="$1" cur n i=0 sel=0
    cur="$(courant)"; cur="${cur:-1}"
    mapfile -t L < <(liste_vue)
    n="${#L[@]}"; [ "$n" -gt 0 ] || return 1
    for i in "${!L[@]}"; do
        [ "${L[$i]%% *}" = "$cur" ] && sel="$i"
    done
    sel=$(( (sel + pas + n) % n ))
    # shellcheck disable=SC2086
    set -- ${L[$sel]}
    ecrire "$1" "$2" "${3:-}"
    echo "op $1 (${2})"
}

case "${1:-}" in
    --list|-l)
```

```
        while read -r idx mode ip; do
            if actif "$idx" "$mode" "$ip"; then e="actif"; else e="arrete"; fi
            [ "$idx" = "$(courant)" ] && e="$e  <- ecran"
            printf '  op %-2s %-7s %-13s %s\n' "$idx" "$mode" "${ip:-(hote)}" "$e"
        done < <(liste) ;;
    --list-vue)
        # "idx mode ip etat" - THE list the Conky builds its selector on, so
        # that screen and arrows stop at the same places.
        while read -r idx mode ip; do
            if actif "$idx" "$mode" "$ip"; then e=actif; else e=arrete; fi
            # The IP may be empty (the host): a dash replaces it, otherwise the
            # line only has three fields and the reader takes the state for an
            # address.
            echo "$idx $mode ${ip:--} $e"
        done < <(liste_vue) ;;
    --next|-n) decale 1 ;;
    --prev|-p) decale -1 ;;
    --set|-s)
        [ $# -ge 2 ] || { echo "usage: $0 --set N" >&2; exit 2; }
        while read -r idx mode ip; do
            if [ "$idx" = "$2" ]; then ecrire "$idx" "$mode" "$ip"; echo "op $idx ($mode)"; exit 0; fi
        done < <(liste)
        echo "op $2 : absent de la topologie ($MULTI_CONF)" >&2; exit 1 ;;
    ""|--current|-c)
        [ -r "$OP_FILE" ] && cat "$OP_FILE" || echo "OP=1"$'\n'"MODE=native" ;;
    *) sed -n '2,30p' "$0" | sed 's/^# \{0,1\}//'; exit 2 ;;
esac
```

2.265 /opt/GSM/osmo-operator/tools/osmo-panel.py

43298 octets, 907 lignes → 907 lignes

```
#!/usr/bin/env python3
# osmo-panel.py - the LIVE PANEL of the desktop, clickable version.
#
# A GTK window of type DESKTOP (the same mechanism as Ubuntu's DING icons:
# below every window, above the background, and it receives clicks - which
# Conky cannot do), placed EXACTLY on the Calvin & Hobbes frame of the
# wallpaper (tools/wallpaper-render.py: box 510,220-1410,1010 in 1920x1080),
# scaled to the screen. It shows /run/osmo-fft/panel.png
# (tools/osmo-fft-snap.py: the strip of the day fading to « mobile FFT +
# mobile.log » once the bench is ready) and, at the bottom, a small menu:
#
#   [ Dashboard ]  firefox http://<operator>:8080
#   [ tmux ]       a terminal attached to the bench session
#   [ VTY 4247 ]   a terminal on telnet 127.0.0.1 4247 (mobile console)
#   [ ZMQ ]        the state of the srsRAN virtual radio (ZeroMQ): green when
#                  the entry points listen, grey otherwise. A click opens a
#                  terminal showing who listens and who is connected.
#   IP 192.168.x.y the address of THIS machine on the network - the one to
#                  write into a srsRAN running elsewhere, or to give to
#                  another bench node. A click copies it to the clipboard.
#   [ < ] op 2/3 [ > ]   ONLY in multi-operator: moves from one operator to
#                  the next. The choice is written to /run/osmo-fft/operator;
#                  the FFT render (osmo-fft-snap.py) and the Conky at the top
#                  right (conky-osmo-status.sh) read it and follow.
#
# Multi-operator = /etc/osmocom/osmo-multi.conf (addition.sh) AND at least one
# osmo-operator-N container running (docker ps). Without that: no arrows.
#
# Launched by the osmo-conky.desktop autostart (with the osmo-conky.conf Conky).
#
# THE PANEL IS ONLY THERE WHEN A BENCH RUNS. Bench down, the window is HIDDEN
# (not merely transparent: the button bar would sit in the middle of an empty
```

```
# desktop). It appears as soon as a 2G (native osmo-bsc, an osmo-operator-N
# container, the hub) or a 4G (open5gs-mmed, srsenb, srsue) is up, and goes
# away when everything stops. The process itself stays: osmo-desktop-panel
# counts it as alive and does not start a second one. OSMO_PANEL_TOUJOURS=1
# keeps it on screen whatever happens.
import os
import re
import shlex
import signal
import socket
import shutil
import subprocess
import sys
import threading
import time

import cairo
# THE DESKTOP WINDOW REQUIRES X11, NOT WAYLAND.
# Under Wayland, GTK3 ignores window.move() AND Gdk.WindowTypeHint.DESKTOP:
# the compositor places the window where it wants (centred, in front). The
# protocol gives the client no way to sit at absolute coordinates (it would
# take gtk-layer-shell, absent on mutter). So go through XWayland, where X11
# positioning is honoured, exactly like the neighbouring Conky (own_window
# type desktop). Only forced when a DISPLAY exists: without an X server, the
# default behaviour is better than a failure at startup.
X11_FORCE = os.environ.get("XDG_SESSION_TYPE") == "wayland" and bool(os.environ.get("DISPLAY"))
if X11_FORCE:
    os.environ["GDK_BACKEND"] = "x11"

import gi

gi.require_version("Gtk", "3.0")
gi.require_version("Gdk", "3.0")
gi.require_version("GdkPixbuf", "2.0")
from gi.repository import Gdk, GdkPixbuf, Gio, GLib, Gtk  # noqa: E402

# AND THEN THE DESKTOP TYPE NO LONGER WORKS. On XWayland, a
# _NET_WM_WINDOW_TYPE_DESKTOP window is put by GNOME Shell in the desktop
# layer, BELOW the fullscreen icon window (DING extension) - which swallows
# every click: the panel would show but its buttons would not respond. So a
# NORMAL window is kept, simply held below the others (keep_below + stick +
# out of the taskbar): same rendering, and the clicks arrive. On native X11
# (the live key) the DESKTOP type works and is kept.
TYPE_HINT = Gdk.WindowTypeHint.NORMAL if X11_FORCE else Gdk.WindowTypeHint.DESKTOP
# ... AND NEITHER DOES « BELOW ». Second half of the same trap: with
# _NET_WM_STATE_BELOW, GNOME Shell puts the XWayland window under its own
# desktop layer (native Wayland), which is fullscreen and takes every click -
# the window stays visible but fully inert. Under XWayland « always below »
# is therefore given up: ordinary window, sticky and out of the taskbar. It
# comes to the front when clicked, which is the price for being clickable at
# all. OSMO_DESKTOP_BELOW=1 restores the window-below-everything behaviour
# (clicks lost); on native X11 nothing changes.
KEEP_BELOW = (not X11_FORCE) or os.environ.get("OSMO_DESKTOP_BELOW") == "1"

RUN = os.environ.get("OSMO_FFT_DIR", "/run/osmo-fft")
IMG = os.path.join(RUN, "panel.png")
OP_FILE = os.path.join(RUN, "operator")
# THE INSTANCE LOCK IS PER SESSION, NOT PER MACHINE. /run/osmo-fft is created
# root:root by systemd (RuntimeDirectory of osmo-fft-snap.service) and is not
# writable by the desktop user of an installed system. XDG_RUNTIME_DIR
# (/run/user/<uid>) belongs to the user, and one panel per session is exactly
# what is to be counted.
_RUNDIR = os.environ.get("XDG_RUNTIME_DIR") or "/run/user/%d" % os.getuid()
PID_FILE = os.path.join(_RUNDIR if os.path.isdir(_RUNDIR) else RUN, "osmo-panel.pid")
ETAT_FILE = os.path.join(_RUNDIR if os.path.isdir(_RUNDIR) else RUN, "osmo-banc.etat")
REPO_DIR = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
```

```
MULTI_CONF = os.environ.get("MULTI_CONF", "/etc/osmocom/osmo-multi.conf")
TMUX_SESSION = os.environ.get("TMUX_SESSION", "calypso")
DASH_PORT = int(os.environ.get("DASH_PORT", "8080"))
# The VTY of the mobile client (osmocom-bb « mobile »). 4247 for MS#1; MS#2 is
# on 4248 - see start-native.sh, which assigns them.
# 4247, or 4347 in --dsp: start-native.sh publishes the actual port in
# /run/osmo-banc/ms1-vty. MS_VTY_PORT forces it; else the file; else 4247.
def vty_port():
    if os.environ.get("MS_VTY_PORT"):
        return int(os.environ["MS_VTY_PORT"])
    try:
        with open("/run/osmo-banc/ms1-vty") as f:
            return int(f.read().strip())
    except (OSError, ValueError):
        return 4247


VTY_PORT = vty_port()
# The panel box in the wallpaper (1920x1080): x, y, width, height.
# 790 high: two halves (4G on top, 2G at the bottom), see
# tools/osmo-fft-snap.py. The four files that know it must stay in agreement
# (list in tools/wallpaper-render.py).
BOX = (510, 220, 900, 790)
FW, FH = 1920, 1080

# THE BAR HAS NO BACKGROUND. The footer is already painted by
# osmo-fft-snap.py; a second background behind the buttons would show a
# visible edge across the panel footer. The buttons have their own background
# (`.osmo-bar button`) and stand out well enough on their own.
CSS = b"""
/* `background-color: transparent` IS NOT ENOUGH. Adwaita does not only paint
   a colour on a box: it sets a `background-image` (a gradient), plus a border
   and a shadow. Neutralising the colour alone leaves the image - a lighter
   band behind the buttons, over the footer painted by osmo-fft-snap.py. All
   four are switched off.
   `background: none` MUST come before `background-color`, otherwise the
   shorthand property puts the default colour back afterwards. */
.osmo-bar { background: none; background-image: none; background-color: transparent;
            border: none; box-shadow: none; padding: 0; margin: 0; }
.osmo-bar button { background: #161b22; color: #e6edf3; border: 1px solid #30363d;
                   border-radius: 6px; padding: 1px 8px; font: 9pt "DejaVu Sans Mono"; min-height: 0; }
.osmo-bar button:hover { background: #21262d; border-color: #58a6ff; }
.osmo-bar button.arrow { font: 15pt "DejaVu Sans Mono"; font-weight: bold;
                         padding: 0 14px; min-width: 34px; color: #3fb950; }
.osmo-bar button.arrow:hover { border-color: #ff9a3c; color: #ff9a3c; }
.osmo-bar label { color: #8b949e; font: 9pt "DejaVu Sans Mono"; }
.osmo-bar label.op { color: #3fb950; font-weight: bold; font-size: 11pt; }
/* The virtual radio: green when it listens, grey when it is down.
   The colour IS the information - it is what one reads at a glance. */
.osmo-bar button.zmq-on  { color: #3fb950; border-color: #238636; }
.osmo-bar button.zmq-off { color: #6e7681; }
.osmo-bar label.ip { color: #58a6ff; font-weight: bold; }
"""


# ── THE GEOMETRY: WHERE THE FRAME IS ON THIS SCREEN ──────────────────────
def geometry():
    """(x, y, w, h) of the strip frame on screen, according to the wallpaper
    display mode (zoom = cover and crop, scaled = contain, stretched)."""
    disp = Gdk.Display.get_default()
    mon = disp.get_primary_monitor() or disp.get_monitor(0)
    g = mon.get_geometry()
    sw, sh = g.width, g.height
    try:
        mode = Gio.Settings.new("org.gnome.desktop.background").get_string("picture-options")
    except Exception:
```

```
        mode = "zoom"
    if mode == "stretched":
        sx, sy, ox, oy = sw / FW, sh / FH, 0, 0
    else:
        s = min(sw / FW, sh / FH) if mode == "scaled" else max(sw / FW, sh / FH)
        sx = sy = s
        ox, oy = (FW * s - sw) / 2, (FH * s - sh) / 2
    bx, by, bw, bh = BOX
    return (g.x + int(bx * sx - ox), g.y + int(by * sy - oy), int(bw * sx), int(bh * sy)), (sw, sh, mode)


# ── THE OPERATORS ──────────────────────────────────────────────────
# STOPPED OPERATORS COUNT TOO. The screen must be able to settle on a stopped
# operator - that is precisely when it is needed for diagnosis. So the WHOLE
# topology is returned, with the state (up) next to each entry, and the panel
# footer reads "op 3/3 arrete" instead of hiding the button.
# ON THE OTHER HAND, NO ARROWS OUTSIDE MULTI-OPERATOR. Without
# /etc/osmocom/osmo-multi.conf - a single-operator bench - there is nothing
# to navigate to: the list returns the native one only, and update_op() hides
# the arrows. No operators 2 and 3 are invented.
# Same list and same rule as tools/osmo-op.sh, the command-line version (and
# the reference definition).


M3UA_PORT = int(os.environ.get("MULTI_M3UA_PORT", "2908"))

# ── THE srsRAN VIRTUAL RADIO (ZeroMQ) ──────────────────────────────
# srsRAN without hardware: the « zmq » driver replaces the radio card with two
# ZeroMQ sockets - one carries the samples from the station to the mobile,
# the other from the mobile to the station (tcp://IP:2000 and :2001 by
# default). The bench uses it to give the phone a REAL radio layer: srsUE
# attaches to the station, and the phone modem (postmarketOS, via
# tools/osmo-pmos.sh) talks to that mobile.
# NOTHING about the state is guessed: look at who listens on those ports.
ZMQ_PORTS = [int(x) for x in os.environ.get("OSMO_ZMQ_PORTS", "2000,2001").split(",") if x.strip()]


def _sctp_ecoute(port):
    """Does an SCTP socket listen on this port? Direct read of
    /proc/net/sctp/eps - no privilege, and `ss` may not be installed."""
    try:
        with open("/proc/net/sctp/eps") as f:
            for line in f.readlines()[1:]:
                ch = line.split()
                # LPORT column: the 5th field in the kernel format.
                if len(ch) > 4 and ch[4] == str(port):
                    return True
    except OSError:
        pass
    try:
        out = subprocess.run(["ss", "-an"], capture_output=True, text=True, timeout=3).stdout
        return any(l.startswith("sctp") and f":{port}" in l for l in out.splitlines())
    except Exception:
        return False


def _tcp_ecoute(port):
    """Does a TCP socket listen on this port? /proc/net/tcp{,6} is read
    directly: no privilege, and `ss` may be missing. The 4th column is
    « 0A » (TCP_LISTEN) and the local address is « HEX:PORT »."""
    hexport = "%04X" % port
    for f in ("/proc/net/tcp", "/proc/net/tcp6"):
        try:
            with open(f) as fh:
                for line in fh.readlines()[1:]:
                    ch = line.split()
```

```
                if len(ch) > 3 and ch[3] == "0A" and ch[1].endswith(":" + hexport):
                    return True
        except OSError:
            continue
    return False


def zmq_etat():
    """(active, detail) of the srsRAN virtual radio.

    Active = at least one of the ZeroMQ ports listens. The detail names those
    that listen: on a half-started bench (station launched, mobile not yet),
    it is the only way to see which of the two is missing.
    """
    ouverts = [p for p in ZMQ_PORTS if _tcp_ecoute(p)]
    if not ouverts:
        return False, "aucun point d entree ZeroMQ (srsRAN arrete)"
    manque = [p for p in ZMQ_PORTS if p not in ouverts]
    detail = "ecoute : " + ", ".join(str(p) for p in ouverts)
    if manque:
        detail += " ; MANQUE : " + ", ".join(str(p) for p in manque)
    return True, detail


def ip_locale():
    """The address of this machine on the network - the one given to others.

    « ip route get » names the interface carrying the default route without
    sending a single packet: it is the right answer even with no network at
    the other end, and it does not depend on a hard-coded interface name.
    """
    try:
        out = subprocess.run(["ip", "-4", "route", "get", "1.1.1.1"],
                             capture_output=True, text=True, timeout=2).stdout
        m = re.search(r"\bsrc\s+(\d+\.\d+\.\d+\.\d+)", out)
        if m:
            return m.group(1)
    except Exception:
        pass
    # No default route (isolated bench): the first non-local address.
    try:
        out = subprocess.run(["ip", "-4", "-br", "addr"], capture_output=True,
                             text=True, timeout=2).stdout
        for ligne in out.splitlines():
            m = re.search(r"(\d+\.\d+\.\d+\.\d+)/", ligne)
            if m and not m.group(1).startswith("127."):
                return m.group(1)
    except Exception:
        pass
    return "127.0.0.1"


def ip_toutes():
    """All the addresses, for the tooltip: a bench has several (the network,
    the docker bridges, apn0...) and the one that matters depends on who is
    asking."""
    try:
        out = subprocess.run(["ip", "-4", "-br", "addr"], capture_output=True,
                             text=True, timeout=2).stdout.strip()
        return out or "aucune adresse"
    except Exception:
        return "adresses illisibles"


def operators():
    """List [(idx, mode, ip, up)]: the whole topology, running or not."""
```

```
ops = []
try:
    with open(MULTI_CONF) as f:
        txt = f.read()
except OSError:
    txt = ""
m = re.search(r'^MULTI_OPS="?([^"\n]*)"?', txt, re.M)
specs = m.group(1).split() if m else []
if not specs:
    # No multi-operator topology: a single operator, the native one.
    # update_op() will hide the arrows - there is nowhere to go.
    specs = ["1:native:"]
# `docker ps` FAILS UNDER THE SESSION ACCOUNT if it is not in the `docker`
# group ("permission denied ... /var/run/docker.sock"). « absent » and
# « invisible » are not confused: when docker does not answer, the
# operator's dashboard is probed instead, which needs no privilege. Same
# rule as tools/osmo-op.sh.
running = set()
docker_ok = False
try:
    r = subprocess.run(["docker", "ps", "--format", "{{.Names}}"], capture_output=True,
                       text=True, timeout=5)
    docker_ok = r.returncode == 0
    running = set(r.stdout.split())
except Exception:
    docker_ok = False

def reachable(ip):
    if not ip:
        return False
    try:
        with socket.create_connection((ip, DASH_PORT), timeout=0.4):
            return True
    except OSError:
        return False

for spec in specs:
    parts = spec.split(":")
    if len(parts) < 3:
        continue
    idx, mode, ip = parts[0], parts[1], parts[2]
    if mode == "native":
        up = subprocess.run(["pgrep", "-x", "osmo-bsc"], capture_output=True).returncode == 0
    elif docker_ok:
        up = f"osmo-operator-{idx}" in running
    else:
        up = reachable(ip)
    ops.append((idx, mode, ip, up))
if not any(o[1] == "native" for o in ops):
    ops.insert(0, ("1", "native", "", False))
# ── THE HUB IS THE LAST STOP OF THE CYCLE ──────────────────────────
# The node that links the operators is the one queried when SS7 goes
# wrong. The panel renders it as an SS7 view (connectivity matrix + hub
# log) and not as a spectrum: an M3UA hub has no radio, an FFT would mean
# nothing there. See tools/osmo-fft-snap.py, render_interstp().
# Same rule as tools/osmo-op.sh, the reference definition.
if specs and any(o[1] == "docker" for o in ops):
    hub_ip = ""
    m = re.search(r'^MULTI_HUB_IP="?([^"\s#]*)', txt, re.M)
    if m:
        hub_ip = m.group(1)
    hub = os.environ.get("MULTI_HUB_NAME", "osmo-inter-stp")
    # THE HUB HAS NO DASHBOARD: the inter-STP only routes M3UA, so the
    # `reachable()` fallback on :8080 would say nothing. Without docker
    # access, look at what it REALLY listens on: an SCTP socket on the
    # M3UA port. A TCP probe is mute there (it is SCTP), hence the read
```

```
        # of /proc/net/sctp.
        up = hub in running if docker_ok else _sctp_ecoute(M3UA_PORT)
        ops.append(("hub", "interstp", hub_ip or "172.20.0.10", up))
    return ops


# ── IS THERE A BENCH TO SHOW? ──────────────────────────────────────
# 2G: an operator of the topology running (native, container or hub - the
# operators() list, which already knows). 4G: one of the daemons the Conky
# looks at to say « 4G up » (tools/conky-osmo-status.sh, lte) - same names,
# same rule. `pgrep -x`: the exact NAME, not the command line (a
# `tail -F srsue.log` or this panel itself do not count).
LTE_DEMONS = os.environ.get("OSMO_LTE_DEMONS", "open5gs-mmed|srsenb|srsue")
PANEL_TOUJOURS = os.environ.get("OSMO_PANEL_TOUJOURS") == "1"


def lte_actif():
    try:
        out = subprocess.run(["pgrep", "-x", LTE_DEMONS], capture_output=True,
                             text=True, timeout=5).stdout.split()
    except Exception:
        return False
    # pgrep also lists ZOMBIES (a crashed srsue not yet reaped by its parent):
    # a dead process holds no bench, it is not counted.
    for pid in out:
        try:
            with open(f"/proc/{pid}/stat") as f:
                if f.read().rsplit(")", 1)[1].split()[0] != "Z":
                    return True
        except (OSError, IndexError):
            return True       # unreadable: assume alive rather than hiding a bench
    return False


def banc_actif(ops):
    """True if a 2G (an operator or the hub running) or a 4G is running."""
    return PANEL_TOUJOURS or any(o[3] for o in ops) or lte_actif()


# ── THE FILE IS THE SHARED SOURCE, NOT AN OUTPUT ──────────────────
# The keyboard shortcut, `osmo-op --next` and other sessions change
# /run/osmo-fft/operator too. The panel therefore READS it, and only WRITES
# it on a click of its own arrows - rewriting it on every refresh would
# cancel a choice made elsewhere.
def read_operator():
    """The selected operator, whoever selected it (panel arrows, keyboard
    shortcut, `osmo-op`). None if the file does not exist."""
    try:
        with open(OP_FILE) as f:
            for line in f:
                if line.startswith("OP="):
                    return line[3:].strip() or None
    except OSError:
        pass
    return None


def write_operator(op):
    idx, mode, ip = op[0], op[1], op[2]
    # NAME is the CONTAINER name: it is how osmo-fft-snap.py and the Conky get
    # in (docker exec). For the hub it is not « osmo-operator-hub » - that
    # container does not exist - but osmo-inter-stp. Same rule as
    # tools/osmo-op.sh.
    nom = (os.environ.get("MULTI_HUB_NAME", "osmo-inter-stp") if mode == "interstp"
           else f"osmo-operator-{idx}")
    os.makedirs(RUN, exist_ok=True)
```

```
    tmp = OP_FILE + ".tmp"
    with open(tmp, "w") as f:
        f.write(f"OP={idx}\nMODE={mode}\nIP={ip}\nNAME={nom}\n"
                f"DASH=http://{ip or '127.0.0.1'}:{DASH_PORT}\n")
    os.replace(tmp, OP_FILE)


def spawn(argv):
    subprocess.Popen(argv, stdin=subprocess.DEVNULL, stdout=subprocess.DEVNULL,
                     stderr=subprocess.DEVNULL, start_new_session=True)


def terminal(cmd, titre="osmo-panel"):
    """Opens `cmd` (shell string) in a VTE terminal placed INSIDE a bench GTK
    window (tools/osmo-gtk-run.py --tty). No terminal emulator is involved:
    no panel entry opens one."""
    root = "" if os.geteuid() == 0 else "sudo -E "
    gtk_run = os.path.join(os.path.dirname(os.path.abspath(__file__)), "osmo-gtk-run.py")
    spawn([sys.executable, gtk_run, "--tty", "--title", titre, "--", "bash", "-c", root + cmd])


class Panel(Gtk.Window):
    def __init__(self):
        super().__init__(title="osmo-panel (osmo-egprs)")
        (x, y, w, h), (sw, sh, mode) = geometry()
        print(f"[panel] ecran {sw}x{sh} ({mode}) : encart a {x},{y} {w}x{h}", flush=True)
        self.geo = (x, y, w, h)
        self.set_type_hint(TYPE_HINT)
        self.set_decorated(False)
        self.set_resizable(False)
        self.set_keep_below(KEEP_BELOW)
        self.set_skip_taskbar_hint(True)
        self.set_skip_pager_hint(True)
        self.stick()
        self.set_default_size(w, h)
        self.set_size_request(w, h)
        self.move(x, y)
        screen = self.get_screen()
        visual = screen.get_rgba_visual()
        if visual and screen.is_composited():
            self.set_visual(visual)
        self.set_app_paintable(True)
        self.connect("draw", self.on_draw)
        self.connect("destroy", Gtk.main_quit)

        prov = Gtk.CssProvider()
        prov.load_from_data(CSS)
        Gtk.StyleContext.add_provider_for_screen(screen, prov, Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION)

        self.image = Gtk.Image()
        overlay = Gtk.Overlay()
        overlay.add(self.image)

        bar = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=6)
        bar.get_style_context().add_class("osmo-bar")
        bar.set_halign(Gtk.Align.END)
        bar.set_valign(Gtk.Align.END)
        scale = w / BOX[2]
        bar.set_margin_end(int(18 * scale))
        bar.set_margin_bottom(int(14 * scale))
        b_dash = self.b_dash = Gtk.Button(label="Dashboard")
        b_dash.set_tooltip_text("firefox sur le tableau de bord de l operateur choisi")
        b_dash.connect("clicked", self.on_dash)
        b_tmux = self.b_tmux = Gtk.Button(label="tmux")
        b_tmux.set_tooltip_text("terminal attache a la session du banc (Ctrl-b d pour detacher)")
        b_tmux.connect("clicked", self.on_tmux)
```

```
    # The mobile VTY: the console where one types « show ms »,
    # « call 100102 »... the third place opened by hand ten times a session.
    b_vty = self.b_vty = Gtk.Button(label=f"VTY {VTY_PORT}")
    b_vty.set_tooltip_text(f"terminal : telnet 127.0.0.1 {VTY_PORT} (console du mobile)")
    b_vty.connect("clicked", self.on_vty)
    # The srsRAN virtual radio, and the machine address. Both answer the
    # same question - « where do I plug the phone? ».
    b_zmq = self.b_zmq = Gtk.Button(label="ZMQ")
    b_zmq.connect("clicked", self.on_zmq)
    self.l_ip = Gtk.Label(label="IP —")
    self.l_ip.get_style_context().add_class("ip")
    # A label does not receive clicks: wrap it in an event box.
    ip_box = Gtk.EventBox()
    ip_box.add(self.l_ip)
    ip_box.connect("button-press-event", self.on_ip)
    ip_box.set_tooltip_text("l adresse de cette machine - un clic la copie")
    self.ip_box = ip_box

    self.b_prev = Gtk.Button(label="◀")
    self.b_next = Gtk.Button(label="▶")
    self.b_prev.get_style_context().add_class("arrow")
    self.b_next.get_style_context().add_class("arrow")
    self.b_prev.set_tooltip_text("operateur precedent (op1, op2 ...)")
    self.b_next.set_tooltip_text("operateur suivant (op1, op2 ...)")
    self.b_prev.connect("clicked", self.on_prev)
    self.b_next.connect("clicked", self.on_next)
    self.l_op = Gtk.Label(label="")
    self.l_op.get_style_context().add_class("op")
    for wdg in (b_dash, b_tmux, b_vty, b_zmq, ip_box,
                self.b_prev, self.l_op, self.b_next):
        bar.pack_start(wdg, False, False, 0)
    overlay.add_overlay(bar)
    self.add(overlay)

    self.ops = []
    self.cur = 0
    self.mtime = None
    self._pos = (x, y)
    self.connect("map-event", self._pin)
    # The children are shown, the WINDOW only if a bench runs
    # (refresh_ops, below): a show_all() here would flash it on screen
    # for a second before hiding it.
    overlay.show_all()
    self.update_op()          # show_all showed everything: re-hide the arrows outside multi
    # None: unknown state - the first refresh_ops writes the bench state
    # (wallpaper, state file) even when it is « stopped ».
    self._visible = None
    self.refresh_ops()
    self.refresh_image()
    GLib.timeout_add(1000, self.refresh_image)
    GLib.timeout_add(2000, self.refresh_ops)
    self.refresh_zmq()
    GLib.timeout_add(3000, self.refresh_zmq)

# ── show / hide according to the bench ──────────────────────────
def montrer(self, actif):
    if actif == self._visible:
        return
    self._visible = actif
    self._etat_banc(actif)
    if actif:
        print("[panel] un banc tourne : encart a l ecran", flush=True)
        self.show()
        self._pin()
        GLib.timeout_add(500, self._pin)
    else:
```

```
        print("[panel] plus de banc : encart cache", flush=True)
        self.hide()

# THE DESKTOP FOLLOWS THE BENCH, NOT ONLY THE PANEL. At the same moment:
# (1) the wallpaper fades from the city alone to the city with the
# antenna (tools/osmo-fond.py), and back when stopped; (2) the state is
# written to $XDG_RUNTIME_DIR/osmo-banc.etat ("1"/"0"), which
# osmo-topzone.py reads to fade the « Telecom Operator » title to the LAB
# GSM banner. osmo-launcher.py also launches the wallpaper on the CLICK
# of a run (earlier than the detection here); osmo-fond is idempotent,
# both coexist.
def _etat_banc(self, actif):
    try:
        with open(ETAT_FILE + ".tmp", "w") as f:
            f.write("1\n" if actif else "0\n")
        os.replace(ETAT_FILE + ".tmp", ETAT_FILE)
    except OSError as e:
        print("[panel] etat du banc non ecrit : %s" % e, flush=True)
    fond = os.path.join(REPO_DIR, "tools", "osmo-fond.py")
    if os.path.exists(fond):
        spawn([sys.executable, fond, "antenne" if actif else "ville"])
    self._conky_fondu(actif)

# THE CONKY FOLLOWS TOO, WITH A FADE. Bench down, the dashboard has
# nothing to say (« banc a l arret - ./start-native.sh » twice): it is
# faded out, and comes back with the bench. No kill: the osmo-desktop-panel
# guardian would restart a conky within five seconds. Its X11 WINDOW is
# acted on instead: opacity (_NET_WM_WINDOW_OPACITY, honoured by mutter)
# going down in a dozen steps, then unmap (xdotool); on return, map then
# the opacity goes back up. Without xdotool, opacity 0 is enough to hide
# it. The conky may be restarted meanwhile (new window): _conky_veille
# puts the window back in the wanted state on every refresh_ops round.
def _conky_fenetre(self):
    try:
        out = subprocess.run(["xdotool", "search", "--classname", "^conky$"],
                             capture_output=True, text=True, timeout=3).stdout.split()
    except (OSError, subprocess.TimeoutExpired):
        out = []
    if not out:
        try:
            for l in subprocess.run(["wmctrl", "-lx"], capture_output=True, text=True,
             ↪  timeout=3).stdout.splitlines():
                f = l.split()
                if len(f) > 2 and f[2].lower().startswith("conky."):
                    out.append(str(int(f[0], 16)))
        except (OSError, subprocess.TimeoutExpired, ValueError):
            pass
    return out[0] if out else None

@staticmethod
def _conky_opacite(win, v):
    subprocess.run(["xprop", "-id", win, "-f", "_NET_WM_WINDOW_OPACITY", "32c",
                    "-set", "_NET_WM_WINDOW_OPACITY", str(int(max(0.0, min(1.0, v)) * 0xFFFFFFFF))],
                   stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=3)

def _conky_fondu(self, actif, pas=12, duree=1.2):
    if os.environ.get("OSMO_CONKY_TOUJOURS") == "1":
        return
    win = self._conky_fenetre()
    self._conky_voulu = actif
    if not win:
        return
    xdo = shutil.which("xdotool")

    def anim():
        try:
```

```
            if actif:
                self._conky_opacite(win, 0.0)
                if xdo:
                    subprocess.run([xdo, "windowmap", win], timeout=3)
                for i in range(1, pas + 1):
                    self._conky_opacite(win, i / pas)
                    time.sleep(duree / pas)
                subprocess.run(["xprop", "-id", win, "-remove", "_NET_WM_WINDOW_OPACITY"],
                               stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=3)
            else:
                for i in range(pas - 1, -1, -1):
                    self._conky_opacite(win, i / pas)
                    time.sleep(duree / pas)
                if xdo:
                    subprocess.run([xdo, "windowunmap", win], timeout=3)
        except (OSError, subprocess.TimeoutExpired):
            pass
        self._conky_win = win
    threading.Thread(target=anim, daemon=True).start()

# A fade killed in flight would leave the wallpaper on an intermediate
# image. Every 10 s the wallpaper URI is reread: if it is not the file
# expected for the current state, osmo-fond is relaunched (idempotent, it
# resumes from the image it was at).
def _fond_veille(self):
    if self._visible is None or os.environ.get("OSMO_FOND_TOUJOURS") == "1":
        return
    t = time.monotonic()
    if t - getattr(self, "_fond_t", 0) < 10:
        return
    self._fond_t = t
    attendu = "gsm-lab-antenne.jpg" if self._visible else "gsm-lab-ville.jpg"
    try:
        uri = subprocess.run(["gsettings", "get", "org.gnome.desktop.background", "picture-uri"],
                             capture_output=True, text=True, timeout=5).stdout.strip().strip("'")
    except (OSError, subprocess.TimeoutExpired):
        return
    if uri.endswith("/" + attendu) or "/osmo-fond/" not in uri and "gsm-lab-" not in uri:
        return          # in place, or a wallpaper chosen by the user: left alone
    fond = os.path.join(REPO_DIR, "tools", "osmo-fond.py")
    if os.path.exists(fond):
        print("[panel] fond d ecran a reprendre : %s" % os.path.basename(uri), flush=True)
        spawn([sys.executable, fond, "antenne" if self._visible else "ville"])

def _conky_veille(self):
    # a conky restarted (other window) while the bench is down: hide it at
    # once; and if it vanished then reappeared on return, show it
    voulu = getattr(self, "_conky_voulu", None)
    if voulu is None or voulu or os.environ.get("OSMO_CONKY_TOUJOURS") == "1":
        return
    win = self._conky_fenetre()
    if not win:
        return
    # the X id may be REUSED by a restarted conky: look at the actual
    # window state, not only its number
    try:
        mappee = "IsViewable" in subprocess.run(["xwininfo", "-id", win], capture_output=True,
                                                text=True, timeout=3).stdout
    except (OSError, subprocess.TimeoutExpired):
        mappee = False
    if win != getattr(self, "_conky_win", None) or (mappee and shutil.which("xdotool")):
        self._conky_fondu(False, pas=1, duree=0)

# RE-PLACE AFTERWARDS. Even under X11, the window manager may move the
# window when it maps it (mutter does so for undecorated windows it does
# not recognise as desktop). A single move() before show_all() does not
```

```
# hold: re-place on map-event, then one last time half a second later.
def _pin(self, *_a):
    self.move(*self._pos)
    return False
def on_draw(self, _w, cr):
    cr.set_operator(cairo.OPERATOR_SOURCE)
    cr.set_source_rgba(0, 0, 0, 0)
    cr.paint()
    return False

# ── the image ──────────────────────────────────────────────────────
def refresh_image(self):
    try:
        mt = os.stat(IMG).st_mtime
    except OSError:
        return True
    if mt == self.mtime:
        return True
    self.mtime = mt
    _, _, w, h = self.geo
    try:
        pb = GdkPixbuf.Pixbuf.new_from_file_at_scale(IMG, w, h, False)
        self.image.set_from_pixbuf(pb)
    except GLib.Error:
        pass  # file being written: next second
    return True

# ── the operators ──────────────────────────────────────────────────
def refresh_ops(self):
    ops = operators()
    changed = ops != self.ops
    if changed:
        sel = self.ops[self.cur] if self.ops and self.cur < len(self.ops) else None
        self.ops = ops
        self.cur = next((i for i, o in enumerate(ops) if sel and o[0] == sel[0]), 0)
    # Did someone else choose? (keyboard shortcut, `osmo-op`, another
    # session). If so, follow it - without rewriting the file.
    want = read_operator()
    if want is not None:
        i = next((i for i, o in enumerate(self.ops) if o[0] == want), None)
        if i is not None and i != self.cur:
            self.cur = i
            changed = True
    if changed:
        self.update_op(write=(want is None))
    self.montrer(banc_actif(self.ops))
    self._conky_veille()
    self._fond_veille()
    return True

def update_op(self, write=True):
    global VTY_PORT
    # The arrows are ALWAYS there as soon as the topology has more than
    # one operator - running or not (see operators()). A stopped operator
    # is said in the label, it does not make the button disappear.
    multi = len(self.ops) > 1
    for wdg in (self.b_prev, self.b_next, self.l_op):
        wdg.set_visible(multi)
    if self.ops:
        op = self.ops[self.cur]
        etat = "" if op[3] else " arrete"
        libelle = "inter-STP" if op[1] == "interstp" else f"op {op[0]}/{len(self.ops)}"
        self.l_op.set_text(f"{libelle}{etat}")
        self.l_op.set_tooltip_text(
            "\n".join(f"op {o[0]} {o[1]} {o[2] or '(hote)'} "
                      f"{'actif' if o[3] else 'arrete'}" for o in self.ops))
```

```
        # The three buttons target the selected operator: they SAY so in
        # their label, so pushing the arrows shows which operator they
        # will land on.
        hub = op[1] == "interstp"
        suff = (" hub" if hub else f" op{op[0]}") if len(self.ops) > 1 else ""
        if hub:
            cible = os.environ.get("MULTI_HUB_NAME", "osmo-inter-stp")
        elif op[1] == "docker":
            cible = f"osmo-operator-{op[0]}"
        else:
            cible = f"{op[2] or '127.0.0.1'}"
        self.b_dash.set_label("Dashboard" + suff)
        self.b_dash.set_tooltip_text(f"firefox http://{op[2] or '127.0.0.1'}:{DASH_PORT} ({cible})")
        self.b_tmux.set_label("tmux" + suff)
        self.b_tmux.set_tooltip_text(f"terminal sur la session tmux de {cible}")
        VTY_PORT = vty_port()
        self.b_vty.set_label(f"VTY {VTY_PORT}" + suff)
        self.b_vty.set_tooltip_text(f"terminal : telnet 127.0.0.1 {VTY_PORT} dans {cible}")
        # `write=False` when only FOLLOWING the file: rewriting it
        # identically is useless, and rewriting it with our own idea would
        # cancel a choice made at the keyboard.
        if write:
            write_operator(op)

    def on_prev(self, *_):
        if self.ops:
            self.cur = (self.cur - 1) % len(self.ops)
            self.update_op()

    def on_next(self, *_):
        if self.ops:
            self.cur = (self.cur + 1) % len(self.ops)
            self.update_op()

    # ── the virtual radio and the address, refreshed with the rest ──────────
    def refresh_zmq(self):
        actif, detail = zmq_etat()
        ctx = self.b_zmq.get_style_context()
        ctx.remove_class("zmq-on" if not actif else "zmq-off")
        ctx.add_class("zmq-on" if actif else "zmq-off")
        self.b_zmq.set_label("ZMQ" if actif else "ZMQ ○")
        self.b_zmq.set_tooltip_text("radio virtuelle srsRAN (ZeroMQ)\n%s\nun clic : qui ecoute, qui est connecte"
         ↪  % detail)
        ip = ip_locale()
        self.l_ip.set_text("IP %s" % ip)
        self.l_ip.set_tooltip_text("adresse de cette machine (un clic la copie)\n\n%s" % ip_toutes())
        return True

    def on_zmq(self, *_):
        ports = "|".join(str(p) for p in ZMQ_PORTS)
        # ss first (it names the process), netstat as fallback.
        cmd = (f"echo 'Radio virtuelle srsRAN (ZeroMQ) - ports {', '.join(str(p) for p in ZMQ_PORTS)}'; echo;"
               f" (ss -lntp 2>/dev/null || netstat -lntp 2>/dev/null) | grep -E ':({ports})\\b' || echo 'personne
                ↪  n ecoute';"
               f" echo; echo 'connexions etablies :';"
               f" (ss -ntp 2>/dev/null || netstat -ntp 2>/dev/null) | grep -E ':({ports})\\b' || echo 'aucune';"
               f" echo; echo 'processus srsRAN :'; pgrep -a 'srsenb|srsue|srsepc|gnb|srsran' || echo 'aucun';"
               f" echo; read -n1 -rsp 'touche pour fermer...'")
        terminal(cmd)

    def on_ip(self, *_):
        """Copy the address: that is what it is for nine times out of ten -
        pasting it into a srsRAN configuration file or on another host."""
        ip = ip_locale()
        try:
            Gtk.Clipboard.get(Gdk.SELECTION_CLIPBOARD).set_text(ip, -1)
```

```
            self.l_ip.set_text("IP %s ✓" % ip)
            GLib.timeout_add(1200, lambda: (self.l_ip.set_text("IP %s" % ip), False)[1])
        except Exception:
            pass
        return True

    # ── the buttons ──────────────────────────────────────────────────
    def on_dash(self, *_):
        op = self.ops[self.cur] if self.ops else ("1", "native", "", False)
        url = f"http://{op[2] or '127.0.0.1'}:{DASH_PORT}"
        for browser in ("firefox", "chromium", "xdg-open"):
            if subprocess.run(["which", browser], capture_output=True).returncode == 0:
                spawn([browser, url])
                return

    def on_vty(self, *_):
        # telnet, not nc: it is what the Osmocom documentation gives, and the
        # VTY expects a real line-by-line terminal. The final `read` keeps the
        # window open when the port is closed - otherwise it would close
        # before the reason could be read.
        #
        # A CONTAINER'S VTY IS INSIDE THE CONTAINER: the port is NOT published
        # on the operator's docker network (start-docker.sh only publishes
        # what the host needs). So enter the container, exactly as for tmux.
        VTY_PORT = vty_port()
        op = self.ops[self.cur] if self.ops else ("1", "native", "", False)
        if op[1] == "interstp":
            # The hub has neither mobile nor VTY 4247: its console is the STP (4239).
            c = os.environ.get("MULTI_HUB_NAME", "osmo-inter-stp")
            cmd = (f"docker exec -it {c} telnet 127.0.0.1 4239"
                   f" || {{ echo; echo 'VTY 4239 injoignable dans {c} (hub arrete ?)'; read -r _; }}")
        elif op[1] == "docker":
            c = f"osmo-operator-{op[0]}"
            cmd = (f"docker exec -it {c} telnet 127.0.0.1 {VTY_PORT}"
                   f" || {{ echo; echo 'VTY {VTY_PORT} injoignable dans {c} (operateur arrete ?)';"
                   f" read -r _; }}")
        else:
            cmd = (f"telnet 127.0.0.1 {VTY_PORT}"
                   f" || {{ echo; echo 'VTY 127.0.0.1:{VTY_PORT} injoignable - banc arrete ?'; read -r _; }}")
        terminal(cmd)

    def on_tmux(self, *_):
        # start-docker.sh opens a session named « osmo » on the DEFAULT socket
        # in the container ("docker exec -ti osmo-operator-2 tmux attach -t
        # osmo"); the « osmocom » session on /tmp/osmocom_tmux is the other
        # layout. Both are tried, in order, before concluding.
        op = self.ops[self.cur] if self.ops else ("1", "native", "", False)
        if op[1] == "interstp":
            c = os.environ.get("MULTI_HUB_NAME", "osmo-inter-stp")
            cmd = (f"docker exec -it {c} bash"
                   f" || {{ echo; echo 'hub {c} injoignable'; read -r _; }}")
        elif op[1] == "docker":
            c = f"osmo-operator-{op[0]}"
            cmd = (f"docker exec -it {c} tmux attach -t osmo"
                   f" || docker exec -it {c} tmux -S /tmp/osmocom_tmux attach -t osmocom"
                   f" || docker exec -it {c} tmux attach"
                   f" || {{ echo; echo 'pas de session tmux dans {c} (operateur arrete ?)';"
                   f" echo 'docker ps -a --filter name={c}'; read -r _; }}")
        else:
            cmd = (f"tmux attach -t {TMUX_SESSION}"
                   f" || tmux -S /tmp/osmocom_tmux attach -t osmocom"
                   f" || {{ echo; echo 'pas de session tmux « {TMUX_SESSION} » - banc arrete ?'; read -r _; }}")
        terminal(cmd)


def single_instance():
```

```
    try:
        old = int(open(PID_FILE).read().strip())
        if old != os.getpid():
            os.kill(old, signal.SIGTERM)
            time.sleep(0.3)
    except (OSError, ValueError):
        pass
    # A lock that cannot be written is no reason not to show the panel: at
    # worst two windows, never a crash dialog.
    try:
        with open(PID_FILE, "w") as f:
            f.write(str(os.getpid()))
    except OSError as e:
        print("osmo-panel : verrou %s non ecrit (%s)" % (PID_FILE, e), file=sys.stderr)


if __name__ == "__main__":
    single_instance()
    Panel()
    Gtk.main()
```

2.266 /opt/GSM/osmo-operator/tools/osmo-patches-apply.sh

2557 octets, 51 lignes → 51 lignes

```
#!/bin/bash
# osmo-patches-apply.sh - apply patches/*.patch on the trees under $GSM_ROOT.
#
# Idempotent: a patch already present (git apply -R --check) is skipped, a
# patch that does not apply is reported, never fatal. Used by the native
# install (install_modules/40-patches.sh), the ISO (iso_modules/70-scripts.sh,
# on the rootfs), update.sh (catch-up on an installed machine) and the
# --calypso-fw supplement (tools/osmo-firmwares-install.sh).
#
#   osmo-patches-apply.sh [GSM_ROOT]        default: /opt/GSM
#   OSMO_PATCHES_ONLY=osmocom-bb            only handle the patches of this tree
#   OSMO_PATCHES_QUIET=1                    only print applications and failures
#
# Exit code: 0 if nothing failed, 1 if at least one patch does not apply.
set -u
GSM_ROOT="${1:-${GSM_ROOT:-/opt/GSM}}"
REPO="${REPO:-$(cd "$(dirname "$(readlink -f "${BASH_SOURCE[0]}")")/.." && pwd)}"
: "${GREEN:=}"; : "${YELLOW:=}"; : "${RED:=}"; : "${NC:=}"

# tree targeted by a patch, from its prefix
osmo_patch_target() {
    case "$(basename "$1")" in
        osmo-bts-*)               printf '%s\n' "$GSM_ROOT/osmo-bts" ;;
        osmo-msc-*)               printf '%s\n' "$GSM_ROOT/osmo-msc" ;;
        osmo-hlr-*)               printf '%s\n' "$GSM_ROOT/osmo-hlr" ;;
        osmo-trx-*)               printf '%s\n' "$GSM_ROOT/osmo-trx" ;;
        osmocom-bb-transceiver-*) printf '%s\n' "$GSM_ROOT/osmocom-bb-transceiver" ;;
        osmocom-bb-*)             printf '%s\n' "$GSM_ROOT/osmocom-bb" ;;
        grgsm-*)                  printf '%s\n' "$GSM_ROOT/gr-gsm" ;;
        qosmo-*)                  printf '%s\n' "$GSM_ROOT/qosmo" ;;
        c54x_exe-*)               printf '%s\n' "$GSM_ROOT/c54x_exe" ;;
        *)                        printf '' ;;
    esac
}

rc=0; n=0
for p in "$REPO"/patches/*.patch; do
    [ -f "$p" ] || continue
    t="$(osmo_patch_target "$p")"
```

```
    [ -n "$t" ] && [ -d "$t" ] || continue
    if [ -n "${OSMO_PATCHES_ONLY:-}" ] && [ "$(basename "$t")" != "$OSMO_PATCHES_ONLY" ]; then continue; fi
    if git -C "$t" apply -R --check "$p" 2>/dev/null; then
        [ "${OSMO_PATCHES_QUIET:-0}" = 1 ] || echo "  deja applique : $(basename "$p")"
    elif git -C "$t" apply --check "$p" 2>/dev/null && git -C "$t" apply "$p" 2>/dev/null; then
        echo -e "  ${GREEN}applique${NC} : $(basename "$p") -> $t"; n=$((n+1))
    else
        echo -e "  ${YELLOW}ne s applique pas${NC} : $(basename "$p") sur $t" >&2; rc=1
    fi
done
[ "${OSMO_PATCHES_QUIET:-0}" = 1 ] && [ $n = 0 ] || echo "  $n correctif(s) applique(s) ($GSM_ROOT)"
exit $rc
```

2.267 /opt/GSM/osmo-operator/tools/osmo-phonesim-banc.py

124888 octets, 2812 lignes → 2812 lignes

```
#!/usr/bin/env python3
# osmo-phonesim-banc.py - THE BENCH MODEM, AS SEEN BY oFONO.
#
# THE COMPLETE CHAIN, AND WHERE THIS FILE SITS IN IT:
#
#   Android → rild → libreference-ril.so → pty AT → osmo-ril-atmodem.py → oFono
#                                                                          ↑
#                                                  THIS PROGRAM (AT over TCP) ┘
#                                                          ↓
#                                          the bench: osmo-bsc / osmo-msc VTY
#
# oFono does not make a modem: it DRIVES one. On this bench there is no
# dongle, so oFono must be given something that behaves like a modem: that is
# the role of the phonesim plugin, which opens a TCP connection and speaks
# plain AT over it. This program holds the other end of that connection and
# answers by reading THE BENCH.
#
# Setting up:
#   1. /etc/ofono/phonesim.conf:
#          [osmo]
#          Address=127.0.0.1
#          Port=12345
#   2. this program (it must listen BEFORE ofonod looks for the modem)
#   3. systemctl restart ofono  ->  a modem appears in org.ofono.Manager
#
# What the modem reports comes from osmo-bsc (VTY 4242): the network (MCC/MNC,
# name), the ARFCN, the registration state. A stopped bench = a modem "not
# registered", which is the truth, not a failure.
import json
import os
import re
import socket
import socketserver
import subprocess
import sys
import threading
import time

REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
HOST = os.environ.get("OSMO_PHONESIM_HOST", "127.0.0.1")
PORT = int(os.environ.get("OSMO_PHONESIM_PORT", "12345"))
BSC_VTY = (os.environ.get("OSMO_BSC_VTY_HOST", "127.0.0.1"),
           int(os.environ.get("OSMO_BSC_VTY_PORT", "4242")))
MSC_VTY = (os.environ.get("OSMO_MSC_VTY_HOST", "127.0.0.1"),
           int(os.environ.get("OSMO_MSC_VTY_PORT", "4254")))
# The osmocom-bb mobile. IT is subscriber 100101, and its audio bridge
# (gsm_audio / gsm_mic) is what the VM relays: without it, no voice.
```

```
#
# THE DEFAULT PORT IS A LIST, NOT A SINGLE VALUE. The QEMU fork mobile
# (mobile.cfg) binds 4247, the DSP bench one (c54x_exe/mobile_pont.cfg) binds
# 4347. Without OSMO_MOB_VTY_PORT, 4247 then 4347 are tried at every
# connection; when the variable is set, it is the only address tried.
_MOB_HOST = os.environ.get("OSMO_MOB_VTY_HOST", "127.0.0.1")
if os.environ.get("OSMO_MOB_VTY_PORT"):
    MOB_VTY = (_MOB_HOST, int(os.environ["OSMO_MOB_VTY_PORT"]))
else:
    MOB_VTY = [(_MOB_HOST, 4247), (_MOB_HOST, 4347)]
MS = os.environ.get("OSMO_MS_NAME", "1")
# The subscriber this modem EMBODIES: the one of the Android phone. On the
# bench it is 100101 (IMSI 001010001000001), as shown by "show subscriber
# msisdn 100101" on the osmo-msc VTY.
IMSI = os.environ.get("OSMO_MS_IMSI", "001010001000001")
MSISDN = os.environ.get("OSMO_MS_MSISDN", "100101")
# SMS reception goes through the osmo-msc SMPP server (port 2775): an ESME is
# already declared in the bench conf (esme msc_tester / osmocom1).
SMPP_HOST = os.environ.get("OSMO_SMPP_HOST", "127.0.0.1")
SMPP_PORT = int(os.environ.get("OSMO_SMPP_PORT", "2775"))
SMPP_ID = os.environ.get("OSMO_SMPP_ID", "msc_tester")
SMPP_PW = os.environ.get("OSMO_SMPP_PASSWORD", "osmocom1")
SMPP_ON = os.environ.get("OSMO_SMPP", "1") == "1"
# Does an incoming SMS ALSO go to the osmocom-bb mobile? Yes by default:
# Android and the mobile are the same bench subscriber, and both are wanted
# in parallel.
DUPLIQUE = os.environ.get("OSMO_SMS_DUPLIQUE", "1") == "1"
# The inboxes for SMS delivered by the bench gateway. This program runs
# TWICE (the Android modem in listen mode, the VM one in --connect mode): each
# needs its own port. scripts/sms_notify.py notifies them, and
# scripts/sms-interop-relay.py calls it.
SMS_IN_HOST = os.environ.get("OSMO_SMS_IN_HOST", "127.0.0.1")
SMS_IN_LISTEN = int(os.environ.get("OSMO_SMS_IN_PORT", "12348"))
SMS_IN_CONNECT = int(os.environ.get("OSMO_SMS_IN_PORT_VM", "12349"))
# How a call requested by the Android phone becomes a bench call.
# {num} = the dialled number, {msisdn} = our number. The Local channel enters
# the Asterisk dialplan ("internal" context, the one carrying 100/500/600 and
# the GSM gateway), then switches back to our own number: the bench thus sets
# up a real call between the dialled number and the mobile.
CTX = os.environ.get("OSMO_CALL_CONTEXT", "internal")
# And the contexts through which the bench CALLS us. An incoming call shows
# up in the AMI as "Event: DialBegin / Context: <ctx> / DestExten: 100101".
# ALL contexts through which a call can reach us are accepted:
#   gsm_in      the gsm_msc endpoint, a call COMING FROM THE RADIO
#               (100102 -> 100101);
#   interop_in  the interop trunk (endpoint interop_trunk_opN), a call from
#               ANOTHER operator;
#   internal    a Linphone;
#   zadarma_in  the [zadarma] / [zadarma_pbx] trunk (pjsip.conf), a call from
#               the outside world via the Zadarma operator.
# Without the right context entrant() is never called and NO RING goes to
# oFono/ModemManager: the osmocom-bb mobile is paged by the MSC but the
# handset stays silent and the MSC eventually releases (MNCC cause DEST_OOO).
# The filter that matters remains DestExten == our MSISDN: the second leg of
# our own outgoing calls dials the REMOTE number, never ours.
IN_CTX = {c.strip() for c in os.environ.get(
    "OSMO_CALL_IN_CONTEXT", "gsm_in,interop_in,internal,zadarma_in").split(",") if c.strip()}
# THE CALL GOES THROUGH THE AMI, NOT THE CLI. "channel originate" comes from
# the res_clioriginate module, absent from this installation (modules.conf is
# autoload = no). The AMI is active in manager.conf; its account is installed
# by start-native.sh from configs/manager-osmo.conf.
AMI_HOST = os.environ.get("OSMO_AMI_HOST", "127.0.0.1")
AMI_PORT = int(os.environ.get("OSMO_AMI_PORT", "5038"))
AMI_USER = os.environ.get("OSMO_AMI_USER", "osmo")
AMI_PASS = os.environ.get("OSMO_AMI_PASSWORD", "osmocom1")
```

```
def clip(num):
    """The caller number as it must be PRESENTED, and its 27.007 type.

    A call from the outside via Zadarma arrives with an E.164 caller, but the
    dialplan strips the "+" before handing the call to the MSC ([zadarma_in]:
    Set(CALLERID(num)=${FILTER(0-9,...)})), because GSM signalling cannot
    carry it to the handset. The AMI thus gives "33656678074".

    The "+" is PUT BACK here, at presentation time, because 27.007 has a field
    for it: the number type. 145 = international (the "+" is implicit and the
    phone displays it again), 129 = unknown/national. Both are returned, and
    the caller gets back its +33656678074 form.

    Rule: a number already prefixed with "+" is international. Otherwise, ten
    digits or more WITHOUT a leading zero is international too - bench numbers
    are six digits (100101) and a French national number starts with 0, so
    neither can be mistaken for an international one.
    """
    n = (num or "").strip()
    if not n:
        return "", 129
    if n.startswith("+"):
        return n, 145
    if len(n) >= 10 and n.isdigit() and not n.startswith("0"):
        return "+" + n, 145
    return n, 129


_PROMPT = re.compile(rb'[\r\n][A-Za-z0-9_.\-]+(?:\([^)]*\))?[>#]\s*$')


AT_LOG = os.environ.get("OSMO_AT_LOG_OFONO", "/var/log/osmo-at-ofono.log")
_atlog = None


def log(*a):
    print("[banc]", *a, flush=True)


def atlog(sens, texte):
    """The log of THIS port: what oFono sends to the bench modem and what it
    answers. The counterpart of /var/log/osmo-at.log, which is the other end
    of the chain (the Android RIL). Together they tell the whole journey."""
    global _atlog
    if _atlog is None:
        # FALL BACK TO /tmp. The instance serving the VM runs as the ordinary
        # user and cannot write to /var/log: without the fallback it would
        # lose its whole AT trace, the one that says what ModemManager does
        # with a RING or a NO CARRIER.
        for chemin in (AT_LOG, "/tmp/osmo-at-ofono-%s.log" % os.environ.get("USER", "x")):
            try:
                _atlog = open(chemin, "a", buffering=1)
                try:
                    os.chmod(chemin, 0o644)
                except OSError:
                    pass
                if chemin != AT_LOG:
                    print("[banc] trace AT dans %s" % chemin, file=sys.stderr)
                break
            except OSError as e:
                print("[banc] journal %s impossible : %s" % (chemin, e), file=sys.stderr)
                _atlog = False
    if not _atlog:
        return
    stamp = time.strftime("%H:%M:%S") + ".%03d" % (int(time.time() * 1000) % 1000)
    for ligne in texte.replace("\r", "\n").split("\n"):
```

```
        if ligne.strip():
            try:
                _atlog.write("%s %s %s\n" % (stamp, sens, ligne.strip()))
            except OSError:
                return


# ── THE BENCH, READ THROUGH ITS VTY ─────────────────────────────────────
class Vty:
    """A VTY connection kept open (same mechanics as tools/osmo-ts-probe.py:
    read up to the prompt, never sleep)."""

    def __init__(self, addr):
        # One address, or a list tried in order (see MOB_VTY).
        self.addrs = addr if isinstance(addr, list) else [addr]
        self.addr = self.addrs[0]
        self.s = None
        self.lock = threading.Lock()

    def _connect(self):
        for addr in self.addrs:
            try:
                self.s = socket.create_connection(addr, timeout=1.0)
                self.s.settimeout(1.0)
                self.addr = addr
                self._read()
                return True
            except OSError:
                self.s = None
        return False

    def _read(self, limit=262144):
        buf = b""
        while True:
            try:
                chunk = self.s.recv(4096)
            except socket.timeout:
                break
            except OSError:
                self.s = None
                return None
            if not chunk:
                self.s = None
                return None
            buf += chunk
            if _PROMPT.search(buf) or len(buf) > limit:
                break
        return buf

    def cmd(self, text):
        with self.lock:
            if self.s is None and not self._connect():
                return None
            try:
                self.s.sendall(text.encode() + b"\r\n")
            except OSError:
                self.s = None
                if not self._connect():
                    return None
                try:
                    self.s.sendall(text.encode() + b"\r\n")
                except OSError:
                    self.s = None
                    return None
            out = self._read()
            return out.decode(errors="replace") if out else None
```

```
BSC = Vty(BSC_VTY)
MSC = Vty(MSC_VTY)
MOB = Vty(MOB_VTY)


def mobile(action, essais=1):
    """A call command on the mobile. Returns True if it was accepted.

    WHY THE MOBILE, AND NOT THE VM. An ATD from the VM makes Asterisk start
    TWO legs: one plays the requested service (the echo of 600), the other
    calls 100101 - that is the MOBILE, not the VM. Without an answer that leg
    falls on Congestion(), gsm_audio stays at zero and nothing is heard - yet
    that bridge is what the VM relays. The mobile must therefore answer, and
    there is nobody to press the key: it is done through its VTY.
    """
    # WAIT FOR THE LEG TO ARRIVE BEFORE ANSWERING. A "call 1 answer" sent
    # while the leg is still in paging writes "% No alerting call" on ALL the
    # mobile's VTYs - including the user's. The mobile says where it stands in
    # "show ms": as long as its RR layer is "idle", nothing has arrived yet,
    # so keep quiet.
    if action == "answer":
        for _ in range(essais * 2):
            out = MOB.cmd("show ms %s" % MS) or ""
            if "radio resource layer state: idle" not in out:
                break
            time.sleep(0.5)
        # Between the paging response and the ringing there remain one to
        # three seconds: all attempts (essais) are kept, not just a few.
    # The mobile confirms with "% (MS 1) % Call is connected" - a message that
    # starts with "%", like its errors. Only "No alerting call" / "No call"
    # (nothing to answer yet) justifies a retry; "connected" or any other
    # reply means success.
    for _ in range(essais):
        MOB.cmd("enable")
        out = MOB.cmd("call %s %s" % (MS, action))
        if out is not None and "No alerting call" not in out and "No call" not in out:
            return True
        time.sleep(0.5)
    return False


def mobile_decroche():
    """The answer, insistently: the leg towards the subscriber takes a moment
    to make the mobile ring, and a single attempt comes too early."""
    if mobile("answer", essais=30):
        log("mobile : decroche")
        return True
    log("mobile : rien a decrocher")
    return False


class Banc:
    """The bench state, refreshed on demand and cached for one second - a
    modem is queried in bursts when the RIL starts (CREG, COPS, CSQ...), no
    need to make four VTY round trips for the same answer."""

    def __init__(self):
        self._t = 0
        self._v = {}

    def state(self):
        if time.time() - self._t < 1.0:
            return self._v
        v = {"up": False, "mcc": "001", "mnc": "01", "name": "Osmocom",
```

```
             "arfcn": "", "lac": 0, "cid": 0}
        txt = BSC.cmd("show network")
        if txt:
            v["up"] = True
            # The actual OsmoBSC format, as seen on the bench:
            #   "BSC is on MCC-MNC 001-01 and has 2 BTS"
            m = re.search(r'MCC-MNC\s+(\d+)-(\d+)', txt)
            if m:
                v["mcc"], v["mnc"] = m.group(1), m.group(2)
            m = re.search(r'Network Name.*?:\s*(\S.*)', txt)
            if m:
                v["name"] = m.group(1).strip()
        txt = BSC.cmd("show bts 0")
        if txt:
            # "ARFCNs: 514" on this bench (the "s" and the colon matter).
            m = re.search(r'ARFCNs?:?\s+(\d+)', txt)
            if m:
                v["arfcn"] = m.group(1)
            m = re.search(r'LAC\s+(\d+)', txt)
            if m:
                v["lac"] = int(m.group(1))
            m = re.search(r'(?:Cell ID|CI)\s+(\d+)', txt)
            if m:
                v["cid"] = int(m.group(1))
        self._v, self._t = v, time.time()
        return v


BANC = Banc()


# ── THE 4G: srsRAN OVER THE ZEROMQ VIRTUAL RADIO ──────────────────────────
# THE PHONE IS ON 4G, AND DROPS TO 2G TO TALK.
# The bench has two radios: the osmocom 2G (osmo-bts-trx + the osmocom-bb
# mobile, which IS subscriber 100101) and an srsRAN 4G without hardware
# (srsENB <-> srsUE over ZeroMQ, ports 2000/2001, srsEPC behind). The eNB
# announces the 2G to its UEs through the SIB7 of sib.conf (the GERAN list,
# mapped into SIB1 by si_mapping_info = [7]): that is CSFB, "CS fallback" - a
# UE camped on LTE, which has no voice, drops to GSM for the duration of a
# call or an SMS, then climbs back.
#
# This modem does what the chip of a real phone would do in front of these
# two networks: it camps on the 4G as long as it is there (+CEREG registered,
# AcT 7, TAC/ECI of the eNB), and as soon as a CS service starts - an ATD, a
# RING, an SMS either way - it announces it has moved to 2G (URC +CREG with
# LAC/CI of the BTS and AcT 0); once the service ends, it climbs back. Phosh
# therefore shows "4G", then "2G" during the call, then "4G".
#
# THE TRUTH OF THE 4G COMES FROM THE BENCH, as that of the 2G comes from the
# osmo-bsc VTY:
#   - the eNB is there if the srsENB ZeroMQ port (2000) listens;
#   - the UE is ATTACHED if srsUE has brought up its tun (tun_srsue) with an
#     address - it does so in its own network namespace (--gw.netns=ue1),
#     which is where we look. No right to look there (not root): stick to
#     the eNB. OSMO_LTE_TEMOIN=enb to look ONLY at the eNB, even as root.
# The identities (MCC/MNC, TAC, ECI, APN, the GERAN ARFCNs of the SIB7) are
# read from the srsRAN files themselves (OSMO_SRSRAN_DIR, else root's
# ~/.config/srsran, else /etc/srsran): what the modem reports is what the
# eNB transmits.
ZMQ_PORT = int(os.environ.get("OSMO_ZMQ_PORT", "2000"))
LTE_NETNS = os.environ.get("OSMO_LTE_NETNS", "ue1")
LTE_TUN = os.environ.get("OSMO_LTE_TUN", "tun_srsue")
LTE_TEMOIN = os.environ.get("OSMO_LTE_TEMOIN", "ue")          # ue | enb
LTE_ON = os.environ.get("OSMO_LTE", "1") == "1"
# The name under which the 4G appears on the phone. srsRAN does not transmit
# a network name; one is given, distinct from that of the 2G ("Osmocom", read
```

```
# from osmo-bsc), so that the network list shows TWO entries.
LTE_NAME = os.environ.get("OSMO_LTE_NAME", "Osmocom 4G")
# DATA OVER 2G: THE GPRS MODEM ON THE CALYPSO (QEMU + TI ROM).
# Without 4G, the phone's data goes out through tun_modem, the tunnel of the
# osmocom-bb `modem` (PDP context, netns msgprs), NATed behind its address;
# the GGSN delivers it all on apn0, NATed to the Internet like the 4G SGi.
# The Calypso has only ONE layer-2 client: scripts/calypso-mode.sh gives it
# to the modem (ATD*99, data) or to the mobile (call, SMS: voice).
# OSMO_GPRS_DATA=0: disabled.
GPRS_DATA = os.environ.get("OSMO_GPRS_DATA", "1") == "1"
GPRS_NETNS = os.environ.get("OSMO_GPRS_NETNS", "msgprs")
GPRS_TUN = os.environ.get("OSMO_GPRS_TUN", "tun_modem")
GPRS_NET = os.environ.get("OSMO_GPRS_NET", "176.16.32.0/24")
CALYPSO_MODE = os.environ.get("OSMO_CALYPSO_MODE",
                              os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "scripts",
 ↪  "calypso-mode.sh"))
CALYPSO_MODE_F = "/run/osmo-banc/calypso-mode"


def calypso_mode():
    """voix | data: who owns the Calypso (scripts/calypso-mode.sh)."""
    try:
        with open(CALYPSO_MODE_F) as f:
            return f.read().strip() or "voix"
    except OSError:
        return "voix"


def calypso(mode, attendre):
    """Give the Calypso to the mobile (voix) or to the GPRS modem (data).
    attendre: wait until the mobile is in service / the PDP is active
    (blocking, 2 to 4 min max)."""
    if not os.path.exists(CALYPSO_MODE):
        log("calypso : %s absent, pas de bascule %s" % (CALYPSO_MODE, mode))
        return False
    cmd = ["bash", CALYPSO_MODE, mode] + (["--wait"] if attendre else [])
    if not attendre:
        subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, start_new_session=True)
        log("calypso : bascule en %s lancee" % mode)
        return True
    r = subprocess.run(cmd, capture_output=True, text=True, timeout=300)
    for l in (r.stdout or "").splitlines():
        log("calypso : %s" % l)
    return r.returncode == 0
# The vendor announced to AT+CGMI: it decides the ModemManager plugin, hence
# the way data is done (see the replies to +CGMI and +GTRNDIS).
VENDOR = os.environ.get("OSMO_MODEM_VENDOR", "Fibocom")


def _tcp_ecoute(port, host="127.0.0.1"):
    try:
        s = socket.create_connection((host, port), timeout=0.3)
        s.close()
        return True
    except OSError:
        return False


class Lte:
    """The bench 4G: what srsRAN transmits, and whether our UE is on it."""

    def __init__(self):
        self.dir = self._repertoire()
        self.cfg = self._lire_conf()
        self._t = 0
        self._v = {}
```

```
    self._dernier = None
    self._geran_verifie = False

@staticmethod
def _repertoire():
    cands = [os.environ.get("OSMO_SRSRAN_DIR", ""), "/root/.config/srsran",
             os.path.expanduser("~/.config/srsran"), "/etc/srsran"]
    for d in cands:
        if d and os.path.isfile(os.path.join(d, "enb.conf")):
            return d
    return None

def _texte(self, nom):
    if not self.dir:
        return ""
    try:
        with open(os.path.join(self.dir, nom), errors="replace") as f:
            return f.read()
    except OSError:
        return ""

@staticmethod
def _val(texte, cle, defaut):
    """key = value" (ini or libconfig, 0x accepted), or the default."""
    m = re.search(r'^\s*%s\s*=\s*"?([0-9A-Za-z_.]+)' % re.escape(cle), texte, re.M)
    if not m:
        return defaut
    v = m.group(1)
    if isinstance(defaut, int):
        try:
            return int(v, 0)
        except ValueError:
            return defaut
    return v

def _lire_conf(self):
    enb, rr, epc, sib = (self._texte(n) for n in ("enb.conf", "rr.conf", "epc.conf", "sib.conf"))
    c = {"mcc": self._val(enb, "mcc", "001"), "mnc": self._val(enb, "mnc", "01"),
         "enb_id": self._val(enb, "enb_id", 0x19B),
         "earfcn": self._val(enb, "dl_earfcn", 3350),
         "tac": self._val(rr, "tac", self._val(epc, "tac", 7)),
         "cell_id": self._val(rr, "cell_id", 1),
         "apn": self._val(epc, "apn", "srsapn"),
         "geran": [], "sib7_annonce": False}
    # The SIB7: the GERAN list the eNB gives its UEs for CSFB, and
    # whether it is scheduled in SIB1 (otherwise it is not transmitted,
    # and a real UE would not know there is a 2G next door).
    m = re.search(r'sib7\s*=\s*\{(.*?)\n\}', sib, re.S)
    if m:
        bloc = m.group(1)
        # The GERAN group of the SIB7 is start_arfcn PLUS the list that
        # follows (36.331, CarrierFreqsGERAN): both are announced.
        deb = re.search(r'start_arfcn\s*=\s*(\d+)', bloc)
        if deb:
            c["geran"].append(int(deb.group(1)))
        lst = re.search(r'explicit_list_of_arfcns\s*=\s*\(([^)]*)\)', bloc)
        if lst:
            c["geran"] += [int(x) for x in re.findall(r'\d+', lst.group(1))]
    m = re.search(r'si_mapping_info\s*=\s*\[([^\]]*)\]', sib)
    c["sib7_annonce"] = bool(m and re.search(r'\b7\b', m.group(1)))
    return c

def _ue_attache(self):
    """True/False if the srsUE tun could be inspected, None otherwise."""
    # The tun is in the network namespace if srsue was started with
    # --gw.netns, on the host otherwise (plain "srsue"): BOTH are checked.
```

```
    # True as soon as it is seen; None if nowhere could be looked at (not
    # root for the namespace, and no tun on the host).
    essais = []
    if LTE_NETNS:
        essais.append(["ip", "-n", LTE_NETNS, "-4", "-o", "addr", "show"])
    essais.append(["ip", "-4", "-o", "addr", "show"])
    vu_quelque_part = False
    ns_ferme = False
    for cmd in essais:
        try:
            r = subprocess.run(cmd, capture_output=True, text=True, timeout=3)
        except (OSError, subprocess.TimeoutExpired):
            continue
        if r.returncode != 0:
            # "ip -n ue1" is refused without root (Operation not
            # permitted) while the namespace exists: the tun may be there
            # unseen. Do not conclude "not attached" from the host view
            # alone.
            if "-n" in cmd and LTE_NETNS and os.path.exists("/run/netns/" + LTE_NETNS):
                ns_ferme = True
            continue
        vu_quelque_part = True
        if re.search(r'\b%s\b.*\binet\b' % re.escape(LTE_TUN), r.stdout):
            return True
    if ns_ferme:
        # Without root, look through the srsue threads: the GW thread
        # did setns() into the namespace, and /proc/<pid>/task/<tid>/net/
        # is readable by everyone. fib_trie lists the local addresses
        # there: the tun shows up with its address once the attach
        # succeeded.
        vu = self._ue_attache_par_proc()
        if vu is not None:
            return vu
        return None
    return False if vu_quelque_part else None

def _ue_attache_par_proc(self):
    """True/False from /proc/<srsue pid>/task/*/net, None if no srsue
    or nothing readable."""
    try:
        r = subprocess.run(["pgrep", "-x", "srsue"], capture_output=True, text=True, timeout=3)
    except (OSError, subprocess.TimeoutExpired):
        return None
    pids = r.stdout.split()
    if not pids:
        return False
    lu = False
    for pid in pids:
        try:
            tids = os.listdir("/proc/%s/task" % pid)
        except OSError:
            continue
        for tid in tids:
            base = "/proc/%s/task/%s/net/" % (pid, tid)
            try:
                with open(base + "dev") as f:
                    dev = f.read()
            except OSError:
                continue
            lu = True
            if LTE_TUN + ":" not in dev:
                continue
            try:
                with open(base + "fib_trie") as f:
                    trie = f.read()
            except OSError:
```

```
                    continue
                # A LOCAL address other than 127.x in the tun's namespace:
                # srsue received it from the EPC.
                if re.search(r'^\s+\|--\s+(?!127\.)\d+\.\d+\.\d+\.\d+\n\s+/32 host LOCAL', trie, re.M):
                    return True
        return False if lu else None

    def state(self):
        if time.time() - self._t < 1.0:
            return self._v
        v = dict(self.cfg)
        v["eci"] = ((v["enb_id"] & 0xFFFFF) << 8) | (v["cell_id"] & 0xFF)
        v["enb"] = LTE_ON and _tcp_ecoute(ZMQ_PORT)
        v["ue"] = self._ue_attache() if v["enb"] else False
        if LTE_TEMOIN == "enb":
            v["ok"] = v["enb"]
        else:
            v["ok"] = v["enb"] and v["ue"] is not False
        etat = (v["enb"], v["ok"])
        if etat != self._dernier:
            if v["ok"]:
                log("4G : eNB en ligne (ZeroMQ %d), UE %s - TAC %04X, ECI %08X, APN %s"
                    % (ZMQ_PORT, "attache" if v["ue"] else "non observable, on suit l eNB",
                       v["tac"], v["eci"], v["apn"]))
            elif not v["enb"]:
                log("4G : eNB absent (rien n ecoute sur le port ZeroMQ %d)" % ZMQ_PORT)
            else:
              log("4G : eNB en ligne mais srsUE pas attache (pas de %s avec une adresse, ni dans %s ni sur l hote)"
                    % (LTE_TUN, LTE_NETNS or "-"))
            self._dernier = etat
        self._v, self._t = v, time.time()
        return v

    def resume(self):
        c = self.cfg
        if not self.dir:
            log("4G : aucune configuration srsRAN trouvee (OSMO_SRSRAN_DIR) - valeurs par defaut")
        else:
            log("4G : configuration srsRAN dans %s" % self.dir)
        log("4G : PLMN %s%s, EARFCN %d, TAC %04X, eNB 0x%X, APN %s"
            % (c["mcc"], c["mnc"], c["earfcn"], c["tac"], c["enb_id"], c["apn"]))
        if c["geran"]:
            log("4G : SIB7 annonce la 2G sur ARFCN %s%s"
                % (", ".join(str(a) for a in c["geran"]),
                   "" if c["sib7_annonce"] else
                   " - MAIS le SIB7 n est pas dans si_mapping_info du SIB1 : il n est pas emis"))
        else:
            log("4G : pas de SIB7 (liste GERAN) dans sib.conf - pas de CSFB annonce aux UE")

    def verifier_geran(self, arfcn_bts):
        """Once: is the 2G announced by the SIB7 the one of the bench?"""
        if self._geran_verifie or not arfcn_bts:
            return
        self._geran_verifie = True
        try:
            a = int(arfcn_bts)
        except ValueError:
            return
        if self.cfg["geran"] and a not in self.cfg["geran"]:
            log("4G : ATTENTION, le SIB7 annonce ARFCN %s mais la BTS du banc est sur %d - "
                "un vrai UE ne trouverait pas la 2G pour son CSFB (sib.conf : explicit_list_of_arfcns)"
                % (", ".join(str(x) for x in self.cfg["geran"]), a))


LTE = Lte()
```

```
# ── ASTERISK, THROUGH THE AMI ──────────────────────────────────────────
def _ami(actions, wait=3.0):
    """Open an AMI session, play the actions, return the raw reply."""
    try:
        s = socket.create_connection((AMI_HOST, AMI_PORT), timeout=5)
    except OSError as e:
        return None, "AMI injoignable (%s)" % e
    try:
        s.settimeout(wait)
        s.recv(4096)                                  # the banner
        s.sendall(("Action: Login\r\nUsername: %s\r\nSecret: %s\r\n\r\n"
                   % (AMI_USER, AMI_PASS)).encode())
        rep = s.recv(4096).decode(errors="replace")
        if "Success" not in rep:
            return None, "connexion AMI refusee"
        for act in actions:
            s.sendall(act.encode())
        out = b""
        end = time.time() + wait
        while time.time() < end:
            try:
                chunk = s.recv(4096)
            except socket.timeout:
                break
            if not chunk:
                break
            out += chunk
            if b"Response: Success" in out or b"Response: Error" in out:
                break
        return out.decode(errors="replace"), ""
    except OSError as e:
        return None, str(e)
    finally:
        try:
            s.sendall(b"Action: Logoff\r\n\r\n")
            s.close()
        except OSError:
            pass


def ami_originate(num):
    """Give birth to the call: the dialled number is called in the bench
    dialplan, and once answered it is connected to our own number - hence to
    the subscriber's mobile. Returns (ok, channel or reason)."""
    chan = "Local/%s@%s" % (num, CTX)
    # ECHO_NO_PROMPT=1: Asterisk's 600 skips its announcement (demo-echotest)
    # and goes straight to Echo: with the recording of leg ;1 (sub-record),
    # the Playback -> Echo sequence makes Asterisk send a burst of RTP
    # silence packets towards the BTS. See the comment of 600 in
    # configs/extensions.conf; no effect on the other numbers. The echo test
    # announcement is played instead on the mobile leg (A() option of Dial,
    # via DIAL_OPTS), where it does not trigger the burst.
    extra = "Variable: DIAL_OPTS=A(demo-echotest)\r\n" if num == "600" else ""
    act = ("Action: Originate\r\n"
           "Channel: %s\r\n"
           "Context: %s\r\n"
           "Exten: %s\r\n"
           "Priority: 1\r\n"
           "CallerID: %s\r\n"
           "Variable: ECHO_NO_PROMPT=1\r\n"
           "%s"
           "Async: true\r\n\r\n" % (chan, CTX, MSISDN, MSISDN, extra))
    out, err = _ami([act], wait=5.0)
    if out is None:
        return False, err
```

```
    if "Response: Error" in out:
        m = re.search(r"Message: (.+)", out)
        return False, (m.group(1).strip() if m else "refus d Asterisk")
    return True, chan


def ami_hangup(channel):
    if not channel:
        return
    _ami(["Action: Hangup\r\nChannel: %s\r\n\r\n" % channel], wait=2.0)


def _bloc(txt):
    """An AMI event block ("Key: value" per line) returned as a dict."""
    d = {}
    for ligne in txt.split("\r\n"):
        cle, sep, val = ligne.partition(":")
        if sep:
            d[cle.strip()] = val.strip()
    return d


class AmiEcoute(threading.Thread):
    """The ear on Asterisk.

    WITHOUT IT, NEITHER RINGING NOR HANGUP. It is what emits the RING for an
    incoming call (otherwise ModemManager has nothing to show) and what
    learns that a call has ended (otherwise the call screen stays displayed
    when the other party hangs up).

    What is watched, as seen on the bench for a call from 100102 to 100101:
        Event: DialBegin      Context: gsm_in
        CallerIDNum: 100102   DestExten: 100101
        DestChannel: PJSIP/gsm_msc-0000000f
    then Hangup on that same channel for the end. Our own outgoing calls also
    dial our number: without the est_notre_jambe() filter the modem would
    ring itself.
    """

    daemon = True

    def run(self):
        while True:
            try:
                self._session()
            except OSError as e:
                log("AMI : lien perdu (%s)" % e)
            time.sleep(5)

    def _session(self):
        s = socket.create_connection((AMI_HOST, AMI_PORT), timeout=5)
        try:
            s.settimeout(None)
            s.recv(4096)                                    # the banner
            s.sendall(("Action: Login\r\nUsername: %s\r\nSecret: %s\r\n"
                       "Events: on\r\n\r\n" % (AMI_USER, AMI_PASS)).encode())
            log("AMI : a l ecoute des appels")
            buf = b""
            while True:
                d = s.recv(8192)
                if not d:
                    return
                buf += d
                while b"\r\n\r\n" in buf:
                    blk, buf = buf.split(b"\r\n\r\n", 1)
                    self._evenement(_bloc(blk.decode(errors="replace")))
```

```
        finally:
            try:
                s.close()
            except OSError:
                pass

    def _evenement(self, e):
        modem = CURRENT
        if modem is None:
            return
        ev = e.get("Event")
        if ev == "DialBegin" and e.get("Context") in IN_CTX \
                and e.get("DestExten") == MSISDN:
            # The second leg of our own calls dials OUR number: without this
            # test, the phone rings for the call it has just placed.
            if modem.est_notre_jambe(e.get("Channel") or ""):
                return
            modem.entrant(e.get("CallerIDNum") or "", e.get("DestChannel") or "")
        elif ev == "Newchannel":
            # ami_originate stores "Local/600@internal", which is the
            # REQUEST. Asterisk creates "Local/600@internal-0000000b;1": an
            # Action: Hangup needs that real name, noted as soon as it
            # appears. LATCH THE FIRST, NOT THE LAST: a Local channel comes
            # in PAIRS ("...;1" and "...;2", both events in the same
            # millisecond), so the name is replaced only while it still
            # equals the REQUEST (c["origine"]).
            chan = e.get("Channel") or ""
            for c in modem.calls:
                ref = c.get("channel")
                if ref and chan.startswith(ref) and chan != ref \
                        and c.get("channel") == c.get("origine"):
                    c["channel"] = chan
        elif ev == "Hangup" or (ev == "DialEnd" and e.get("DialStatus") != "ANSWER"):
            for chan in (e.get("Channel"), e.get("DestChannel")):
                if chan:
                    modem.fin_appel(chan)


# ── DATA: A PPP SERVER AT THE END OF THE AT PORT ─────────────────────────
# WHY PPP, AND NOTHING ELSE. ModemManager, facing a modem that has only an AT
# port, knows a single way to do data: dial ATD*99***1#, wait for CONNECT,
# and let NetworkManager run pppd on the serial port. The plugins that can do
# without PPP (Fibocom +GTRNDIS, Huawei ^NDISDUP, Cinterion ^SWWAN) all
# require a USB network port - either to recognise the modem by its USB id,
# or to number the port (bInterfaceNumber); on a QEMU PCI serial port, none
# applies. So the phone must HAVE PPP (the ppp_generic/ppp_async modules,
# built for its kernel) and this modem must speak PPP at the other end: that
# is this block.
#
# What is done, and nothing more (RFC 1661, 1662, 1332):
#   - asynchronous HDLC framing (0x7e, 0x7d escape, FCS-16);
#   - LCP: MRU, ACCM, magic number, PFC and ACFC are accepted, the rest is
#     rejected (including authentication: the bench asks for none);
#     Echo-Requests are answered; a Terminate-Request ends the session;
#   - IPCP: the phone is GIVEN its address and DNS by Configure-Nak (this is
#     how an operator assigns them), VJ compression is refused;
#   - IPv6CP and CCP: Protocol-Reject, pppd does without;
#   - IP packets go into a tun, and that tun is placed IN THE NETWORK
#     NAMESPACE OF srsUE (--gw.netns) with a NAT towards tun_srsue: each
#     packet of the phone then crosses the 4G radio, the eNB, the EPC and the
#     SGi. If srsUE runs on the host (no netns), routing still goes through
#     tun_srsue, but the return takes the local shortcut - the UE and the
#     EPC being the same machine, the kernel delivers internally what is
#     addressed to a local address. This is said in the log.
import fcntl
import random
```

```
import struct

# The PPP link addresses must NOT be those the EPC gives the UE (with
# open5gs, srsUE receives 10.45.0.2): the phone's address would then be LOCAL
# on tun_srsue in the same namespace, and the NAT return would be delivered
# to the host instead of going back to the phone. A separate /30 that nothing
# else uses.
PPP_LOCAL = os.environ.get("OSMO_PPP_LOCAL", "10.99.0.1")      # us, the network
PPP_PEER = os.environ.get("OSMO_PPP_PEER", "10.99.0.2")        # the phone
PPP_DNS = [d.strip() for d in os.environ.get("OSMO_PPP_DNS", "8.8.8.8,8.8.4.4").split(",") if d.strip()]
PPP_IF = os.environ.get("OSMO_PPP_IF", "ppp-pmos")


def _sgi_net_defaut():
    """The UE subnet, for the host's SGi -> Internet egress: the one of
    smf.yaml with open5gs (10.45.0.0/16), 172.16.0.0/24 with srsEPC."""
    # The bench prefix (/opt/LTE/open5gs/install, the one of osmo-epc.sh) is
    # among the candidates: without it the MASQUERADE set here would not
    # cover the open5gs UEs (10.45.0.0/16).
    for f in ([os.path.join(os.environ["OPEN5GS_PREFIX"], "etc/open5gs/smf.yaml")] if
     ↪  os.environ.get("OPEN5GS_PREFIX") else []) + [
            "/opt/LTE/open5gs/install/etc/open5gs/smf.yaml",
            "/root/open5gs/install/etc/open5gs/smf.yaml", "/etc/open5gs/smf.yaml"]:
        try:
            m = re.search(r'^\s*-\s*subnet:\s*(\d+\.\d+\.\d+\.\d+/\d+)', open(f).read(), re.M)
        except OSError:
            continue
        if m:
            return m.group(1)
    return "172.16.0.0/24"


PPP_SGI_NET = os.environ.get("OSMO_LTE_SGI_NET") or _sgi_net_defaut()

_FCS_TAB = []
for _b in range(256):
    _v = _b
    for _ in range(8):
        _v = (_v >> 1) ^ 0x8408 if _v & 1 else _v >> 1
    _FCS_TAB.append(_v)


def ppp_fcs(data, fcs=0xFFFF):
    for b in data:
        fcs = (fcs >> 8) ^ _FCS_TAB[(fcs ^ b) & 0xFF]
    return fcs


def _sh(*cmd):
    """A system command, without raising: returns (rc, output)."""
    try:
        r = subprocess.run(cmd, capture_output=True, text=True, timeout=5)
        return r.returncode, (r.stdout + r.stderr).strip()
    except (OSError, subprocess.TimeoutExpired) as e:
        return 1, str(e)


class Ppp:
    LCP, IPCP, IP, IPV6CP, CCP = 0xC021, 0x8021, 0x0021, 0x8057, 0x80FD
    CONF_REQ, CONF_ACK, CONF_NAK, CONF_REJ = 1, 2, 3, 4
    TERM_REQ, TERM_ACK, CODE_REJ, PROTO_REJ, ECHO_REQ, ECHO_REP = 5, 6, 7, 8, 9, 10

    def __init__(self, modem):
        self.modem = modem
        self.magic = struct.pack(">I", random.getrandbits(32))
        self.buf = bytearray()
```

```
    self.in_frame = False
    self.esc = False
    self.brut = bytearray()             # what arrives OUTSIDE a frame (an AT?)
    self.ident = 0
    self.lcp_ack_recu = self.lcp_ack_donne = self.lcp_ouvert = False
    self.ipcp_ack_recu = self.ipcp_ack_donne = self.ipcp_ouvert = False
    self.tun = None
    self.ns = None
    self.fini = False
    self.lock = threading.Lock()
    self.octets = [0, 0]                # upstream, downstream
    self._motif_fin = None              # set by terminer()

def terminer(self, motif):
    """WE close the session: Terminate-Request to pppd, which answers
    Terminate-Ack and stops; if it does not answer within a second, close
    anyway. fin() will say NO CARRIER on the port, and ModemManager takes
    over again in command mode."""
    if self.fini or self._motif_fin:
        return
    self._motif_fin = motif
    self._paquet(self.LCP, self.TERM_REQ, self._prochain(), b"")
    t = threading.Timer(1.0, lambda: self.fin(motif))
    t.daemon = True
    t.start()

# -- the HDLC layer
def feed(self, data):
    for b in data:
        if b == 0x7E:
            if self.in_frame and len(self.buf) >= 4:
                self._trame(bytes(self.buf))
            self.buf.clear()
            self.in_frame = True
            self.esc = False
            self.brut.clear()
            continue
        if not self.in_frame:
            # Outside a frame: the port may have switched back to AT
            # (ModemManager taking over after pppd died). "AT" followed
            # by an end of line, and the port is given back.
            self.brut += bytes([b])
            if b in (0x0D, 0x0A):
                ligne = self.brut.decode(errors="replace").strip().upper()
                self.brut.clear()
                if ligne.startswith("AT") or ligne == "+++":
                    self.fin("commande AT recue en mode donnees (%s)" % ligne)
                    return
            if len(self.brut) > 64:
                del self.brut[:32]
            continue
        if self.esc:
            self.buf.append(b ^ 0x20)
            self.esc = False
        elif b == 0x7D:
            self.esc = True
        else:
            self.buf.append(b)

def _trame(self, f):
    if ppp_fcs(f) != 0xF0B8:
        return                          # bad FCS: drop
    f = f[:-2]
    if f[:2] == b"\xff\x03":
        f = f[2:]
    if not f:
```

```
            return
        if f[0] & 1:                        # one-byte protocol (PFC)
            proto, pkt = f[0], f[1:]
        else:
            proto, pkt = struct.unpack(">H", f[:2])[0], f[2:]
        if proto == self.IP:
            self._vers_tun(pkt)
        elif proto == self.LCP:
            self._lcp(pkt)
        elif proto == self.IPCP:
            self._ipcp(pkt)
        else:
            self._proto_rejet(proto, pkt)

    def envoyer(self, proto, payload):
        f = b"\xff\x03" + struct.pack(">H", proto) + payload
        fcs = ppp_fcs(f) ^ 0xFFFF
        f += bytes([fcs & 0xFF, fcs >> 8])
        out = bytearray([0x7E])
        for b in f:
            if b < 0x20 or b in (0x7D, 0x7E):
                out += bytes([0x7D, b ^ 0x20])
            else:
                out.append(b)
        out.append(0x7E)
        with self.lock:
            try:
                self.modem.wfile.write(bytes(out))
                self.modem.wfile.flush()
            except OSError:
                self.fin("port ferme")

    # -- the negotiation packets
    @staticmethod
    def _options(data):
        opts, i = [], 0
        while i + 2 <= len(data):
            t, ln = data[i], data[i + 1]
            if ln < 2 or i + ln > len(data):
                break
            opts.append((t, data[i + 2:i + ln]))
            i += ln
        return opts

    @staticmethod
    def _opt(t, v):
        return bytes([t, 2 + len(v)]) + v

    def _paquet(self, proto, code, ident, body):
        self.envoyer(proto, bytes([code, ident]) + struct.pack(">H", 4 + len(body)) + body)

    def _prochain(self):
        self.ident = (self.ident + 1) & 0xFF or 1
        return self.ident

    def demarrer(self):
        log("PPP : session ouverte (CONNECT), le telephone doit lancer pppd")
        self._lcp_requete()

    def _lcp_requete(self):
        self._paquet(self.LCP, self.CONF_REQ, self._prochain(),
                     self._opt(2, b"\x00\x00\x00\x00") + self._opt(5, self.magic))

    def _lcp(self, pkt):
        if len(pkt) < 4:
            return
```

```
    code, ident, ln = pkt[0], pkt[1], struct.unpack(">H", pkt[2:4])[0]
    body = pkt[4:ln]
    if code == self.CONF_REQ:
        rejet = b""
        for t, v in self._options(body):
            if t not in (1, 2, 5, 7, 8):     # MRU, ACCM, magic, PFC, ACFC
                rejet += self._opt(t, v)
        if rejet:
            self._paquet(self.LCP, self.CONF_REJ, ident, rejet)
        else:
            self._paquet(self.LCP, self.CONF_ACK, ident, body)
            self.lcp_ack_donne = True
        if not self.lcp_ack_recu:
            self._lcp_requete()
    elif code == self.CONF_ACK:
        self.lcp_ack_recu = True
    elif code in (self.CONF_NAK, self.CONF_REJ):
        # Do not insist on the ACCM: the magic number is enough.
        self._paquet(self.LCP, self.CONF_REQ, self._prochain(), self._opt(5, self.magic))
    elif code == self.TERM_REQ:
        self._paquet(self.LCP, self.TERM_ACK, ident, body)
        self.fin("le telephone a termine la session PPP")
        return
    elif code == self.TERM_ACK and self._motif_fin:
        self.fin(self._motif_fin)
        return
    elif code == self.ECHO_REQ:
        self._paquet(self.LCP, self.ECHO_REP, ident, self.magic + body[4:])
    if self.lcp_ack_recu and self.lcp_ack_donne and not self.lcp_ouvert:
        self.lcp_ouvert = True
        log("PPP : LCP ouvert, on passe a IPCP")
        self._ipcp_requete()

def _proto_rejet(self, proto, pkt):
    if proto in (self.IPV6CP, self.CCP) or self.lcp_ouvert:
        self._paquet(self.LCP, self.PROTO_REJ, self._prochain(),
                     struct.pack(">H", proto) + pkt[:8])

def _ipcp_requete(self):
    self._paquet(self.IPCP, self.CONF_REQ, self._prochain(),
                 self._opt(3, socket.inet_aton(PPP_LOCAL)))

def _ipcp(self, pkt):
    if len(pkt) < 4:
        return
    code, ident, ln = pkt[0], pkt[1], struct.unpack(">H", pkt[2:4])[0]
    body = pkt[4:ln]
    if code == self.CONF_REQ:
        rejet, nak = b"", b""
        dns = PPP_DNS + PPP_DNS[:1]
        for t, v in self._options(body):
            if t == 3:                    # the phone's address
                if v != socket.inet_aton(PPP_PEER):
                    nak += self._opt(3, socket.inet_aton(PPP_PEER))
            elif t == 129:                # primary DNS
                if v != socket.inet_aton(dns[0]):
                    nak += self._opt(129, socket.inet_aton(dns[0]))
            elif t == 131:                # secondary DNS
                if v != socket.inet_aton(dns[1]):
                    nak += self._opt(131, socket.inet_aton(dns[1]))
            else:                         # VJ compression and the rest
                rejet += self._opt(t, v)
        if rejet:
            self._paquet(self.IPCP, self.CONF_REJ, ident, rejet)
        elif nak:
            self._paquet(self.IPCP, self.CONF_NAK, ident, nak)
```

```
            else:
                self._paquet(self.IPCP, self.CONF_ACK, ident, body)
                self.ipcp_ack_donne = True
            if not self.ipcp_ack_recu:
                self._ipcp_requete()
        elif code == self.CONF_ACK:
            self.ipcp_ack_recu = True
        elif code in (self.CONF_NAK, self.CONF_REJ):
            self._ipcp_requete()
        elif code == self.TERM_REQ:
            self._paquet(self.IPCP, self.TERM_ACK, ident, body)
        if self.ipcp_ack_recu and self.ipcp_ack_donne and not self.ipcp_ouvert:
            self.ipcp_ouvert = True
            self._tun_monter()

    # -- the tun, and its hookup to the radio
    def _tun_monter(self):
        TUNSETIFF, IFF_TUN, IFF_NO_PI = 0x400454CA, 0x0001, 0x1000
        try:
            fd = os.open("/dev/net/tun", os.O_RDWR)
            fcntl.ioctl(fd, TUNSETIFF, struct.pack("16sH", PPP_IF.encode(), IFF_TUN | IFF_NO_PI))
        except OSError as e:
            log("PPP : impossible de creer %s (%s) - il faut root ; la session reste sans IP" % (PPP_IF, e))
            return
        self.tun = fd
        self.ns = None
        _sh("ip", "addr", "replace", PPP_LOCAL, "peer", PPP_PEER + "/32", "dev", PPP_IF)
        _sh("ip", "link", "set", PPP_IF, "up", "mtu", "1500")
        if not self._radio_brancher():
            if GPRS_DATA:
                log("PPP : IPCP ouvert, %s au telephone ; sortie radio en attente du PDP GPRS (%s dans %s)"
                    % (PPP_PEER, GPRS_TUN, GPRS_NETNS))
                threading.Thread(target=self._attendre_radio, daemon=True).start()
            elif not self._raccourci_test():
                log("PPP : IPCP ouvert, %s au telephone, mais pas de %s : la data n a pas de sortie radio"
                    % (PPP_PEER, LTE_TUN))
        threading.Thread(target=self._depuis_tun, daemon=True).start()

    def _raccourci_test(self):
        """TEST SHORTCUT (no radio, no GGSN), OSMO_PPP_SHORTCUT=1: the phone
        goes out through the host's default interface, NATed behind its
        address, as the srsEPC SGi would do. Real EGPRS goes through the
        SGSN/GGSN; this path only serves to validate DNS, NAT and the
        phone's IP stack."""
        if os.environ.get("OSMO_PPP_SHORTCUT", "0") != "1":
            return False
        rc, up = _sh("sh", "-c", "ip -4 route show default | awk '{for(i=1;i<=NF;i++) if($i==\"dev\") print
↪ $(i+1); exit}'")
        if not up:
            return False
        _sh("sysctl", "-qw", "net.ipv4.ip_forward=1")
        rc, _ = _sh("iptables", "-w", "-t", "nat", "-C", "POSTROUTING", "-s", PPP_PEER, "-o", up, "-j",
↪ "MASQUERADE")
        if rc != 0:
            _sh("iptables", "-w", "-t", "nat", "-A", "POSTROUTING", "-s", PPP_PEER, "-o", up, "-j", "MASQUERADE")
        _sh("iptables", "-w", "-N", "DOCKER-USER")
        for spec in (("-s", PPP_PEER), ("-d", PPP_PEER)):
            rc, _ = _sh("iptables", "-w", "-C", "DOCKER-USER", *spec, "-j", "ACCEPT")
            if rc != 0:
                _sh("iptables", "-w", "-I", "DOCKER-USER", "1", *spec, "-j", "ACCEPT")
        rc, _ = _sh("iptables", "-w", "-C", "FORWARD", "-j", "DOCKER-USER")
        if rc != 0:
            _sh("iptables", "-w", "-I", "FORWARD", "1", "-j", "DOCKER-USER")
        log("PPP : IPCP ouvert, %s au telephone - RACCOURCI DE TEST : sortie directe par %s (sans GGSN)"
            % (PPP_PEER, up))
        return True
```

```
    def _radio_choix(self):
        """(netns or None, tun, network to NAT towards the Internet): the 4G
        if srsUE has its tun, else the GPRS PDP of the modem on the Calypso,
        else None."""
        if LTE_NETNS:
            rc, o = _sh("ip", "-n", LTE_NETNS, "-4", "-o", "addr", "show", LTE_TUN)
            if rc == 0 and "inet" in o:
                return LTE_NETNS, LTE_TUN, PPP_SGI_NET
        rc, o = _sh("ip", "-4", "-o", "addr", "show", LTE_TUN)
        if rc == 0 and "inet" in o:
            return None, LTE_TUN, PPP_SGI_NET
        if GPRS_DATA and GPRS_NETNS:
            rc, o = _sh("ip", "-n", GPRS_NETNS, "-4", "-o", "addr", "show", GPRS_TUN)
            if rc == 0 and "inet" in o:
                return GPRS_NETNS, GPRS_TUN, GPRS_NET
        return None

    def _radio_brancher(self):
        """Hook PPP_IF onto the available radio egress. False if none."""
        choix = self._radio_choix()
        if choix is None or self.tun is None:
            return False
        ns, tun, sgi = choix
        if ns != self.ns:
            cur = ["ip", "-n", self.ns] if self.ns else ["ip"]
            _sh(*cur, "link", "set", PPP_IF, "netns", ns if ns else "1")
            self.ns = ns
        ip = ["ip", "-n", self.ns] if self.ns else ["ip"]
        ex = ["ip", "netns", "exec", self.ns] if self.ns else []
        _sh(*ip, "addr", "replace", PPP_LOCAL, "peer", PPP_PEER + "/32", "dev", PPP_IF)
        _sh(*ip, "link", "set", PPP_IF, "up", "mtu", "1500")
        _sh(*ex, "sysctl", "-qw", "net.ipv4.ip_forward=1")
        _sh(*ex, "sysctl", "-qw", "net.ipv4.conf.%s.rp_filter=0" % PPP_IF)
        if self.ns:
            _sh(*ip, "route", "replace", "default", "dev", tun)
        else:
            _sh("ip", "route", "replace", "default", "dev", tun, "table", "45")
            rc, o = _sh("ip", "rule", "show")
            if PPP_PEER not in o:
                _sh("ip", "rule", "add", "from", PPP_PEER, "lookup", "45", "priority", "4500")
        rc, _ = _sh(*ex, "iptables", "-w", "-t", "nat", "-C", "POSTROUTING", "-s", PPP_PEER, "-o", tun, "-j",
↪   "MASQUERADE")
        if rc != 0:
            _sh(*ex, "iptables", "-w", "-t", "nat", "-A", "POSTROUTING", "-s", PPP_PEER, "-o", tun, "-j",
↪   "MASQUERADE")
        # The Internet egress of the UE network (4G SGi or GGSN pool)
        rc, up = _sh("sh", "-c", "ip -4 route show default | awk '{for(i=1;i<=NF;i++) if($i==\"dev\") print
↪   $(i+1); exit}'")
        if up:
            _sh("sysctl", "-qw", "net.ipv4.ip_forward=1")
           rc, _ = _sh("iptables", "-w", "-t", "nat", "-C", "POSTROUTING", "-s", sgi, "-o", up, "-j", "MASQUERADE")
            if rc != 0:
                _sh("iptables", "-w", "-t", "nat", "-A", "POSTROUTING", "-s", sgi, "-o", up, "-j", "MASQUERADE")
            _sh("iptables", "-w", "-N", "DOCKER-USER")
            for spec in (("-s", sgi), ("-d", sgi)):
                rc, _ = _sh("iptables", "-w", "-C", "DOCKER-USER", *spec, "-j", "ACCEPT")
                if rc != 0:
                    _sh("iptables", "-w", "-I", "DOCKER-USER", "1", *spec, "-j", "ACCEPT")
            rc, _ = _sh("iptables", "-w", "-C", "FORWARD", "-j", "DOCKER-USER")
            if rc != 0:
                _sh("iptables", "-w", "-I", "FORWARD", "1", "-j", "DOCKER-USER")
        log("PPP : IPCP ouvert, %s au telephone ; %s %s, la data passe par %s (%s)"
            % (PPP_PEER, PPP_IF, "dans l espace %s" % self.ns if self.ns else "sur l hote",
               tun, "GPRS, Calypso + ROM TI" if tun == GPRS_TUN else "4G, srsUE"))
        return True
```

```
    def _attendre_radio(self):
        """The GPRS modem's PDP arrives 30 to 120 s after ATD*99 (Calypso
        switch, attach, PDP): the egress is hooked up as soon as it is there."""
        t0 = time.time()
        while not self.fini and self.tun is not None and time.time() - t0 < 300:
            if self._radio_brancher():
                return
            time.sleep(2)
        if not self.fini and self.tun is not None:
            log("PPP : toujours pas de PDP GPRS apres 300 s (scripts/calypso-mode.sh etat)")

    def _vers_tun(self, pkt):
        if self.tun is None:
            return
        try:
            os.write(self.tun, pkt)
            self.octets[0] += len(pkt)
        except OSError:
            pass

    def _depuis_tun(self):
        fd = self.tun
        while not self.fini and fd is not None:
            try:
                pkt = os.read(fd, 2048)
            except OSError:
                return
            if not pkt:
                return
            self.octets[1] += len(pkt)
            self.envoyer(self.IP, pkt)

    def _tun_demonter(self):
        fd, self.tun = self.tun, None
        if fd is None:
            return
        ip = ["ip", "-n", self.ns] if self.ns else ["ip"]
        _sh(*ip, "link", "del", PPP_IF)
        if not self.ns:
            _sh("ip", "rule", "del", "from", PPP_PEER, "lookup", "45")
        try:
            os.close(fd)
        except OSError:
            pass

    def fin(self, motif):
        if self.fini:
            return
        self.fini = True
        log("PPP : fin (%s) - %d octets montes, %d descendus" % (motif, self.octets[0], self.octets[1]))
        self._tun_demonter()
        self.modem.ppp_fini()


# ── THE AT MODEM ──────────────────────────────────────────────────────
class AtHandler(socketserver.StreamRequestHandler):
    """One modem per connection: oFono opens a single one."""

    role = "commande"             # "commande" (hvc0) or "data" (hvc1, see DATA)

    def setup(self):
        super().setup()
        global CURRENT, DATA
        if self.role == "data":
            DATA = self
```

```
    else:
        CURRENT = self        # oFono opens a single connection: this one
    self.echo = True          # a real modem starts with echo on
    self.cmee = 1
    self.creg_mode = 0
    self.cops_format = 2          # 0 = long name, 1 = short name, 2 = MCC/MNC
    self.calls = []               # the calls in progress, for +CLCC
    self.sms_target = None
    # How this system wants SMS announced. mt=2: the message pushed as
    # is; mt=1: stored, and only the index is signalled. Start with 2,
    # and AT+CNMI= tells the truth.
    self.cnmi_mt = 2
    self.cmgf = 0             # 0 = PDU (what ModemManager uses), 1 = text
    # The 4G and CSFB (see the Lte class). Three URC modes, one per
    # registration (CS, GPRS, EPS); ws46 = the technologies the system
    # ALLOWS (27.007 § 5.9: 12 GSM only, 28 E-UTRAN only, 31 both) - this
    # is Phosh's "preferred network" setting; _cs_hold holds the 2G for
    # the duration of the CS service, _retour is the timer that brings us
    # back to 4G; _rat_annonce, the last technology announced, to signal
    # only the changes.
    self.cgreg_mode = 0
    self.cereg_mode = 0
    self.ws46 = 31
    self._cs_hold = False
    self._retour = None
    self._rat_annonce = None
    self.data_cid = 0           # the PDP context "connected" by +GTRNDIS (0 = none)
    self.ppp = None             # the PPP session in progress (ATD*99), see the Ppp class
    self._urc_attente = []      # the URCs held back during data (see out)
    self._urc_lock = threading.Lock()
    self.port_commande = True   # False between NO CARRIER and the next AT command
    log("oFono s est connecte depuis %s (port de %s)" % (self.client_address, self.role))
    if self.role != "data":
        threading.Thread(target=self._veille, daemon=True).start()

def finish(self):
    """The link is down: this modem is no longer "the" modem.

    CURRENT is cleared here so that on_passerelle_sms() and the RINGs of
    AmiEcoute do not write into a closed socket (out() swallows the
    OSError). The inbox keeps the message: it comes out at the next
    AT+CMGL.
    """
    global CURRENT, DATA
    if CURRENT is self:
        CURRENT = None
    if DATA is self:
        DATA = None
    if self.ppp is not None:
        self.ppp.fin("lien AT ferme")
    # THE CALLS DIE WITH THE LINK. The AMI only talks to CURRENT, so a
    # hangup would never reach this handler once a new one is born:
    # hang up on the Asterisk side and clear, otherwise the _sonne loop
    # would run forever and the other party would stay hanging.
    for c in list(self.calls):
        log("lien ferme : appel %s abandonne, raccroche cote Asterisk"
            % (c.get("channel") or c.get("num") or "?"))
        ami_hangup(c.get("channel"))
    self.calls.clear()
    t, self._retour = self._retour, None
    if t:
        t.cancel()
    try:
        super().finish()
    except OSError:
        pass
```

```
def ppp_fini(self):
    """The PPP session is over: the port becomes an AT port again, and
    says so like a modem (NO CARRIER) - ModemManager then restarts its
    probe and the bearer becomes "disconnected"."""
    self.ppp = None
    self.data_cid = 0
    # NO CARRIER goes out as is: it is what gives the port back. Then, as
    # long as the phone has not sent its first AT command, the port still
    # belongs to the stopping pppd: anything emitted would go into the
    # void (the +CREG of the 4G return, a +CMTI). Hold them (see out).
    self.port_commande = False
    atlog(">", "NO CARRIER")
    try:
        self.wfile.write(b"\r\nNO CARRIER\r\n")
        self.wfile.flush()
    except OSError:
        pass

def _tag(self, sens):
    return sens if self.role != "data" else sens + "d"      # ">d": data port

def out(self, text):
    # A SINGLE PORT, HENCE NO URC DURING PPP. The bench modem has only
    # one serial port: once ATD*99 has gone through, everything written
    # to it enters the PPP stream, and pppd drops RING, +CLIP, +CMTI as
    # noise. What must be announced during data waits here, and goes out
    # as soon as the port is back in command mode (see _urc_liberer,
    # called after the first AT command that follows). csfb_debut() cuts
    # the data for that: on a real network too, CSFB suspends data for
    # the duration of the CS service.
    if self.ppp is not None or not self.port_commande:
        with self._urc_lock:
            if text not in self._urc_attente:
                self._urc_attente.append(text)
        atlog(self._tag("~"), text + "   (retenu : port en mode donnees)")
        return
    atlog(self._tag(">"), text)
    try:
        self.wfile.write(("\r\n%s\r\n" % text).encode())
        self.wfile.flush()
    except OSError:
        pass

def _urc_liberer(self):
    """The port is in command mode: emit what was waiting."""
    with self._urc_lock:
        attente, self._urc_attente = self._urc_attente, []
    for text in attente:
        self.out(text)

# -- the access technology: 4G at rest, 2G for the duration of a CS service
def _modem(self):
    """The phone state lives on the command port: the data port refers
    to it."""
    if self.role == "data" and CURRENT is not None:
        return CURRENT
    return self

def cs_actif(self):
    m = self._modem()
    return bool(m.calls) or m.sms_target is not None or m._cs_hold

def appel_en_cours(self):
    return bool(self._modem().calls)
```

```
def rat(self):
    """lte" or "gsm": where the phone is camped right now."""
    if self.role == "data" and CURRENT is not None:
        return CURRENT.rat()
    if self.ws46 == 12:                  # the system asked for GSM only
        return "gsm"
    if not LTE.state()["ok"]:
        return "gsm"
    if self.cs_actif():
        return "gsm"                     # CSFB: we went down
    return "lte"

def _reg(self, tag):
    """(stat, lac or tac, ci or eci, act) for +CREG, +CGREG or +CEREG in
    the current technology. act None = no position to give."""
    st = BANC.state()
    lte = LTE.state()
    if self.rat() == "lte":
        # Combined attach (EPS + IMSI attach via SGs): CS is registered
        # if the 2G is there, and everything is said with TAC/ECI, AcT 7.
        stat_cs = 1 if st["up"] else (2 if lte["ok"] else 0)
        stat = stat_cs if tag == "+CREG" else 1
        return stat, lte["tac"], lte["eci"], 7
    reg = 1 if st["up"] else 0
    if tag == "+CEREG":
        # On 2G, EPS is suspended (CSFB) or absent: no position.
        return (2 if lte["ok"] else 0), 0, 0, None
    return reg, st["lac"], st["cid"], 0

def _ligne_reg(self, tag, mode, urc=False):
    stat, lac, ci, act = self._reg(tag)
    tete = "%s: " % tag + ("" if urc else "%d," % mode)
    if mode >= 2 and act is not None and stat in (1, 5):
        return '%s%d,"%04X","%08X",%d' % (tete, stat, lac, ci, act)
    return "%s%d" % (tete, stat)

def annonce_rat(self):
    """Signal a technology change through the registration URCs the
    system asked for (AT+CREG=2 etc.). Order matters: the LAST
    announcement gives the AcT that ModemManager keeps, so end with the
    one carrying the current technology."""
    if self.role == "data":
        return                           # URCs go to the command port
    rat = self.rat()
    prev, self._rat_annonce = self._rat_annonce, rat
    if prev == rat:
        return
    if prev is not None:
        log("technologie : %s -> %s" % ("4G" if prev == "lte" else "2G",
                                        "4G" if rat == "lte" else "2G"))
    ordre = ("+CEREG", "+CGREG", "+CREG") if rat == "gsm" else ("+CREG", "+CGREG", "+CEREG")
    modes = {"+CREG": self.creg_mode, "+CGREG": self.cgreg_mode, "+CEREG": self.cereg_mode}
    for tag in ordre:
        if modes[tag] >= 1:
            self.out(self._ligne_reg(tag, modes[tag], urc=True))

def _veille(self):
    """The 4G may come up or go down while the phone does nothing: check
    every two seconds, and verify once that the 2G announced by the SIB7
    is indeed the one of the bench."""
    while CURRENT is self:
        try:
            st = BANC.state()
            if st["up"]:
                LTE.verifier_geran(st["arfcn"])
            self.annonce_rat()
```

```
        except Exception as e:      # never at the cost of the modem itself
            log("veille 4G : %s" % e)
        time.sleep(2)

def csfb_debut(self, motif, couper_data=True):
    """A CS service starts: go down to 2G right away, and cancel a 4G
    return that would be under way. couper_data: the PPP session is
    closed (a call: 2G has no data during voice; an SMS on a single port:
    the port must be given back to the URCs)."""
    t, self._retour = self._retour, None
    if t:
        t.cancel()
    self._cs_hold = True
    if GPRS_DATA and calypso_mode() == "data":
        # Class B: the Calypso's GPRS data gives way to voice. The mobile
        # comes back (same line as c54x_exe/run.sh) and we wait for it to
        # be in service before letting the call go.
        log("calypso : %s - le Calypso repasse au mobile (voix)" % motif)
        for m in (self, DATA):
            if m is not None and m.ppp is not None:
                m.ppp.terminer("voix : le Calypso quitte le GPRS")
        calypso("voix", attendre=True)
    if self._rat_annonce == "lte":
        st = BANC.state()
        log("CSFB : %s - on quitte la 4G pour la 2G (ARFCN %s, LAC %d, CI %d)"
            % (motif, st["arfcn"] or "?", st["lac"], st["cid"]))
    # Data is suspended for the duration of the CS service - and above
    # all it is the only way to give the port back to the URCs (see out).
    # NetworkManager restarts ATD*99 by itself once the UE is back on 4G.
    if couper_data:
        for m in (self, DATA):
            if m is not None and m.ppp is not None:
                log("CSFB : data suspendue (fin de la session PPP) le temps du service CS")
                m.ppp.terminer("CSFB : data suspendue le temps du service CS")
    self.annonce_rat()

def csfb_fin_apres(self, delai=2.0):
    """The CS service is over: in <delai> seconds, if another one has not
    started, climb back to 4G - the time a real UE takes to reselect its
    LTE cell once the call is hung up."""
    t, self._retour = self._retour, None
    if t:
        t.cancel()
    self._retour = threading.Timer(delai, self._retour_lte)
    self._retour.daemon = True
    self._retour.start()

def _retour_lte(self):
    self._retour = None
    if self.calls or self.sms_target is not None:
        return                          # another CS service took over
    self._cs_hold = False
    if self._rat_annonce == "gsm" and LTE.state()["ok"] and self.ws46 != 12:
        log("CSFB : service CS termine, retour sur la 4G")
    self.annonce_rat()

def ok(self):
    self.out("OK")

def err(self, code=100):
    self.out("+CME ERROR: %d" % code if self.cmee else "ERROR")

def handle(self):
    buf = b""
    while True:
        try:
```

```
            data = self.rfile.read1(4096)
        except (OSError, ValueError):
            break
        if not data:
            break
        if self.ppp is not None:
            # Data mode: everything goes to PPP, nothing is echoed.
            self.ppp.feed(data)
            buf = b""
            continue
        if self.echo:
            try:
                self.wfile.write(data)
                self.wfile.flush()
            except OSError:
                pass
        buf += data
        while True:
            m = re.search(rb"[\r\n\x1a]", buf)
            if not m:
                break
            line, buf = buf[:m.start()], buf[m.end():]
            text = line.decode(errors="replace").strip()
            if text:
                atlog(self._tag("<"), text)
            if text:
                self.port_commande = True     # the phone speaks AT: the port is ours
                # The held URCs go out BEFORE the command: if it is
                # ATD*99, the port goes back to data right after, and a
                # held "back to 4G" +CEREG would stay locked behind the
                # PPP.
                if self.ppp is None and self._urc_attente:
                    self._urc_liberer()
            if self.sms_target is not None and m.group() == b"\x1a":
                self.sms(text)
            elif text:
                self.command(text)
    log("oFono s est deconnecte")

# -- the incoming calls, pushed by AmiEcoute
def est_notre_jambe(self, chan):
    """Is this channel a leg of a call WE placed?

    ami_originate() starts "Local/<num>@internal" on one side and, on
    the other, dials OUR OWN number (Exten: MSISDN) - this is how the
    bench connects the other party to the subscriber. That second leg
    thus produces, for each of our outgoing calls:
        DialBegin  Context: internal  DestExten: 100101
    which is exactly the signature of an incoming call ("internal" is in
    IN_CTX so that a Linphone call can reach us). Our legs are therefore
    recognised by the channel THAT DIALS (the Local we requested), not by
    the context - a real incoming call always arrives through another
    channel. Without this filter the phone would ring for the call it has
    just placed and hang up by itself when that leg ends.
    """
    if not chan:
        return False
    for c in self.calls:
        ref = c.get("origine")
        if ref and chan.startswith(ref):
            return True
    return False

def entrant(self, num, chan):
    """The bench calls us: ring until it stops."""
    if any(c.get("channel") == chan for c in self.calls):
```

```
            return
        c = {"num": num, "state": "incoming", "channel": chan}
        # The paging comes through the 4G (SGs), the ringing plays on the
        # 2G: go down BEFORE the first RING. With a separate data port, data
        # stays up: refusing ATD*99 during the ringing would push
        # NetworkManager into backoff (4 failures, then 5 min without retry).
        self.csfb_debut("appel entrant de %s" % (num or "inconnu"), couper_data=(DATA is None))
        self.calls.append(c)
        log("appel entrant de %s (%s)" % (num or "inconnu", chan))
        threading.Thread(target=self._sonne, args=(c,), daemon=True).start()

    def _sonne(self, c):
        """RING every three seconds, and +CLIP right behind to give the
        number - this is what ModemManager expects to present the call
        (AT+CLIP=1 is among the settings accepted)."""
        while c in self.calls and c["state"] == "incoming" and CURRENT is self:
            self.out("RING")
            if c["num"]:
                _n, _t = clip(c["num"])
                self.out('+CLIP: "%s",%d,,,,0' % (_n, _t))
            time.sleep(3)

    def fin_appel(self, chan):
        """Asterisk hung up this channel. This path is what closes Phosh's
        call screen when the other party hangs up."""
        for c in list(self.calls):
            ref = c.get("channel")
            # Our outgoing calls keep "Local/600@internal", which Asterisk
            # expands to "Local/600@internal-0000001a;1": compare by prefix.
            # Incoming ones carry the exact name.
            if ref and chan.startswith(ref):
                self.calls.remove(c)
                log("appel termine (%s)" % chan)
                self.out("NO CARRIER")
        if not self.calls:
            self.csfb_fin_apres()

    def deliver_sms(self, sender, text):
        """Deliver an incoming SMS THE WAY THE SYSTEM ASKED FOR.

        The message is always stored: even in mt=2 the system may want to
        read it again (AT+CMGL at startup). Only then is the URC chosen.
        """
        pdu = pdu_deliver(sender, text)
        idx = inbox_ranger(sender, text, pdu)
        # An SMS is a CS service on this bench (it comes from the MSC): go
        # down to 2G to receive it, climb back three seconds later. With a
        # separate data port, data stays up: the URCs have their port. On a
        # single port, data must be cut for +CMTI to get through.
        self.csfb_debut("SMS entrant de %s" % (sender or "inconnu"), couper_data=(DATA is None))
        self.csfb_fin_apres(3.0)
        if self.cnmi_mt == 2:
            tpdu_len = len(pdu) // 2 - 1      # without the leading SMSC octet
            self.out("+CMT: ,%d\r\n%s" % (tpdu_len, pdu))
            log("SMS remis directement (+CMT) : %s" % text)
        else:
            self.out('+CMTI: "SM",%d' % idx)
            log("SMS range en SM[%d], index signale (+CMTI) : %s" % (idx, text))

    # -- the three commands that go with the inbox
    def _sms_ligne(self, m, entete):
        """A CMGR/CMGL entry, in the current mode."""
        if self.cmgf:                         # text mode
            stat = "REC READ" if m["lu"] else "REC UNREAD"
            return '%s"%s","%s",,"%s"\r\n%s' % (entete, stat, m["sender"],
                                                m["ts"], m["text"])
```

```
    stat = 1 if m["lu"] else 0            # PDU mode: 0 unread, 1 read
    return "%s%d,,%d\r\n%s" % (entete, stat, len(m["pdu"]) // 2 - 1,
                               m["pdu"])

# -- sending an SMS through the bench
def sms(self, text):
    num, self.sms_target = self.sms_target, None
    if num == "<pdu>":
        try:
            num, text = parse_submit(text.strip())
        except (ValueError, IndexError) as e:
            log("PDU illisible (%s) : %s" % (e, text[:60]))
            self.err()
            return
    self.csfb_debut("SMS vers %s" % num, couper_data=(DATA is None))
    out = MSC.cmd('subscriber msisdn %s sms sender msisdn %s send %s'
                  % (num, MSISDN, text))
    self.csfb_fin_apres(3.0)
    if out is None:
        log("SMS non envoye : osmo-msc injoignable")
        self.err()
        return
    log("SMS a %s : %s" % (num, text))
    self.out("+CMGS: 1")
    self.ok()

def command(self, line):
    cmd = line.upper()
    if not cmd.startswith("AT"):
        return
    ub = cmd[2:]
    body = line[2:]
    st = BANC.state()

    if ub in ("", "&F", "Z", "V1", "Q0", "S0=0", "+CMEE=1", "+CMEE=2",
              "+CLIP=1", "+COLP=0", "+CSSN=0,0", "+CMOD=0", "+CSNS=0",
              "+CSCS=\"IRA\"",
              "+CSCS=\"UCS2\"", "+CGEREP=1,0", "+CVHU=0", "&C1", "&D2"):
        self.ok()
        return
    if ub == "E0":
        self.echo = False; self.ok(); return
    if ub == "E1":
        self.echo = True; self.ok(); return

    if ub in ("+CGMI", "+GMI"):
        # "Fibocom", AND WHY. ModemManager picks its plugin from this
        # name. The generic plugin can only do data through PPP on the
        # AT port. The Fibocom plugin, when the modem has a NETWORK PORT
        # beside the AT port and answers AT+GTRNDIS=?, brings up an ECM
        # bearer: +GTRNDIS=1,<cid> to connect, address by DHCP on the
        # network port. That is what this bench does: QEMU's second
        # network card (tools/osmo-pmos-data.sh) is that port, and it
        # leads into srsUE's network namespace - hence onto the 4G radio.
        self.out(VENDOR); self.ok(); return
    if ub in ("+CGMM", "+GMM"):
        self.out("osmo-banc"); self.ok(); return
    if ub in ("+CGMR", "+GMR"):
        self.out("1.0"); self.ok(); return
    if ub in ("+CGSN", "+GSN"):
        self.out("350000000000000"); self.ok(); return
    if ub == "+CIMI":
        self.out(IMSI); self.ok(); return
    if ub == "+CPIN?":
        self.out("+CPIN: READY"); self.ok(); return
    if ub.startswith("+CNUM"):
```

```
        self.out('+CNUM: "","%s",129' % MSISDN); self.ok(); return

    if ub.startswith("+CFUN"):
        if ub.endswith("?"):
            self.out("+CFUN: %d" % (1 if st["up"] else 4))
        self.ok(); return

    if ub.startswith(("+CREG", "+CGREG", "+CEREG")):
        # THE THREE REGISTRATIONS, AND THE 4G. +CREG is circuit (voice,
        # SMS), +CGREG the 2G/3G packet, +CEREG the EPS - LTE. The modem
        # tells the truth: registered on the EPS when srsUE is attached,
        # and each reply carries the current technology (last field:
        # 0 = GSM, 7 = E-UTRAN). During a call or an SMS, the reply
        # switches to 2G (CSFB) and the EPS becomes "searching".
        # THE "TEST" FORM IS NOT A SETTING. The atmodem driver first
        # probes AT+CREG=? and expects the LIST of supported modes; a
        # plain OK makes it conclude the modem cannot register (oFono:
        # "Unable to initialize Network Registration", no
        # NetworkRegistration interface at all). Same trap for +CGREG
        # (data): "GPRS not supported on this device".
        tag = ub.split("=")[0].rstrip("?")
        if ub.endswith("=?"):
            self.out("%s: (0-2)" % tag); self.ok(); return
        if "=" in ub:
            try:
                mode = int(ub.split("=")[1][0])
            except (ValueError, IndexError):
                mode = 0
            if tag == "+CREG":
                self.creg_mode = mode
            elif tag == "+CGREG":
                self.cgreg_mode = mode
            else:
                self.cereg_mode = mode
            self.ok(); return
        mode = {"+CREG": self.creg_mode, "+CGREG": self.cgreg_mode,
                "+CEREG": self.cereg_mode}[tag]
        self.out(self._ligne_reg(tag, mode))
        self.ok(); return

    if ub.startswith("+COPS"):
        # oFono queries the operator TWICE, in two formats: it sets
        # AT+COPS=3,2 then AT+COPS? for the numeric code (MCC/MNC), and
        # AT+COPS=3,0 then AT+COPS? for the name. The requested format
        # must be remembered, otherwise the name stays empty on the oFono
        # side.
        # The last field is the technology: 7 on 4G, 0 on 2G.
        rat = self.rat()
        act = 7 if rat == "lte" else 0
        nom = LTE_NAME if rat == "lte" else st["name"]
        if ub.endswith("=?"):
            # THE NETWORK SCAN ("Select a network" in Phosh). A real
            # scan sees a cell as soon as it TRANSMITS, not only when
            # attached to it: the 4G is listed as soon as the eNB is
            # online. It carries a name distinct from the 2G (LTE_NAME):
            # two "Osmocom" entries would merge into one on screen.
            # stat: 2 = current, 1 = available.
            lte = LTE.state()
            liste = []
            if lte["enb"]:
                liste.append('(%d,"%s","%s","%s%s",7)'
                             % (2 if rat == "lte" else 1, LTE_NAME, LTE_NAME, lte["mcc"], lte["mnc"]))
            if st["up"]:
                liste.append('(%d,"%s","%s","%s%s",0)'
                             % (2 if rat == "gsm" else 1, st["name"], st["name"], st["mcc"], st["mnc"]))
            self.out('+COPS: %s,,(0,1,3,4),(0,1,2)' % ",".join(liste))
```

```
            self.ok(); return
        if ub.endswith("?"):
            if self.cops_format == 2:
                self.out('+COPS: 0,2,"%s%s",%d' % (st["mcc"], st["mnc"], act))
            else:
                self.out('+COPS: 0,%d,"%s",%d' % (self.cops_format, nom, act))
            self.ok(); return
        m = re.match(r'\+COPS=3,(\d)', ub)
        if m:
            self.cops_format = int(m.group(1))
            self.ok(); return
        # The SELECTION: AT+COPS=0 (automatic) or AT+COPS=1,<fmt>,<oper>
        # [,<act>]. Choosing the 4G entry of the list means camping on
        # LTE; the 2G entry, on GSM; automatic gives both back.
        #
        # THIS IS THE ONLY LEVER THE USER HAS. Phosh's "network type"
        # setting goes through ModemManager, which REFUSES to set it on
        # an AT-command modem ("Setting allowed modes not supported" -
        # it reads AT+WS46=? for the list, but never emits AT+WS46=<n>).
        # What remains is the manual network selection, and MM always
        # does it NUMERICALLY, WITHOUT technology: 'AT+COPS=1,2,"00101"'.
        # Both radios carry the same MCC/MNC, so the number does not say
        # which one is wanted. Settled by usage: automatic (+COPS=0) is
        # already "4G as long as it is there"; asking for MANUAL here can
        # only mean one thing - stay on 2G. A name (format 0 or 1) or an
        # explicit technology are taken at their word.
        m = re.match(r'\+COPS=(\d)(?:,(\d),"?([^",]*)"?(?:,(\d))?)?', ub)
        if m:
            mode, fmt, oper, actsel = (int(m.group(1)), m.group(2),
                                       m.group(3) or "", m.group(4))
            if mode == 0:
                self.ws46 = 31
            elif mode == 1:
                if actsel is not None:
                    self.ws46 = 28 if actsel == "7" else 12
                elif fmt in ("0", "1") and oper:
                    self.ws46 = 28 if oper.strip().upper() == LTE_NAME.upper() else 12
                else:
                    self.ws46 = 12
           log("selection de reseau : %s" % {31: "automatique (2G + 4G)", 28: "la 4G", 12: "la 2G"}[self.ws46])
            self.annonce_rat()
        self.ok(); return

    if ub.startswith("+WS46"):
        # 27.007 § 5.9: the technology ALLOWED by the system - this is
        # Phosh's "network type" setting (2G / 4G / automatic).
        # 12 = GSM only, 28 = E-UTRAN only, 31 = GERAN + E-UTRAN.
        if ub.endswith("=?"):
            self.out("+WS46: (12,28,31)"); self.ok(); return
        if ub.endswith("?"):
            self.out("+WS46: %d" % self.ws46); self.ok(); return
        try:
            v = int(re.sub(r"\D", "", ub.split("=", 1)[1]) or "31")
        except (ValueError, IndexError):
            self.err(); return
        if v not in (12, 28, 31):
            self.err(); return
        self.ws46 = v
        log("technologies autorisees : %s" % {12: "2G seule", 28: "4G seule", 31: "2G + 4G"}[v])
        self.annonce_rat()
        self.ok(); return

    if ub.startswith("+CSQ"):
        # Bench running = strong signal; bench stopped = no signal.
        # The virtual 4G is "stronger" than the 2G: it shows.
        rat = self.rat()
```

```
        self.out("+CSQ: %d,99" % (28 if rat == "lte" else 24 if st["up"] else 99))
        self.ok(); return

    if ub.startswith("+CESQ"):
        # Extended quality (27.007 § 8.69): rxlev/ber for 2G, rsrq/rsrp
        # for 4G, 255 = not applicable. On 4G: rsrq 20 (-9.5 dB), rsrp 70
        # (-71 dBm); on 2G: rxlev 24.
        if ub.endswith("=?"):
            self.out("+CESQ: (0-63,99),(0-7,99),(0-96,255),(0-49,255),(0-34,255),(0-97,255)")
            self.ok(); return
        if self.rat() == "lte":
            self.out("+CESQ: 99,99,255,255,20,70")
        else:
            self.out("+CESQ: %d,99,255,255,255,255" % (24 if st["up"] else 99))
        self.ok(); return

    if ub.startswith("+GTRNDIS"):
        # FIBOCOM'S ECM BEARER (see +CGMI). ModemManager probes
        # AT+GTRNDIS=? at initialisation; if answered, and it has a
        # network port grouped with this AT port, data no longer goes
        # through PPP: +GTRNDIS=1,<cid> "connects", +GTRNDIS=0,<cid>
        # disconnects, +GTRNDIS? gives the state, and the address comes
        # by DHCP on the network port (dnsmasq in srsUE's namespace,
        # tools/osmo-pmos-data.sh).
        if ub.endswith("=?"):
            self.out("+GTRNDIS: (0,1),(1-11)"); self.ok(); return
        if ub.endswith("?"):
            self.out("+GTRNDIS: %d,%d" % (1 if self.data_cid else 0, self.data_cid or 1))
            self.ok(); return
        m = re.match(r'\+GTRNDIS=(\d)(?:,(\d+))?', ub)
        if not m:
            self.err(); return
        cid = int(m.group(2) or 1)
        if m.group(1) == "1":
            if not LTE.state()["ok"] and not st["up"]:
                log("data : +GTRNDIS refuse, aucun reseau")
                self.err(30)                # 30 = no network service
                return
            self.data_cid = cid
            log("data : porteur ECM connecte (cid %d, APN %s) - l adresse vient par DHCP "
                "sur le port reseau, la data passe par %s"
                % (cid, LTE.state()["apn"], "la 4G (srsUE)" if self.rat() == "lte" else "la 2G"))
        else:
            self.data_cid = 0
            log("data : porteur ECM coupe (cid %d)" % cid)
        self.ok(); return

    if ub == "+CGDCONT?":
        # The PDP context: the APN is that of srsEPC (epc.conf), the
        # phone's data is on the 4G - even if, without PPP, the system
        # cannot activate it through this port (see ATD*99).
        self.out('+CGDCONT: 1,"IP","%s","0.0.0.0",0,0' % LTE.state()["apn"])
        self.ok(); return
    if ub == "+CGATT?":
        self.out("+CGATT: %d" % (1 if (LTE.state()["ok"] or st["up"]) else 0))
        self.ok(); return
    if ub == "+CGACT?":
        self.out("+CGACT: 1,%d" % (1 if (self.data_cid or self.ppp) else 0)); self.ok(); return
    if ub == "+CGACT=?":
        self.out("+CGACT: (0,1)"); self.ok(); return
    if ub.startswith("+CEMODE"):
        # The EPS mode of operation (24.301 § 4.3): 1 = "CS/PS mode 1,
        # voice centric" - the phone that drops to 2G to talk. Exactly
        # this bench.
        if ub.endswith("=?"):
            self.out("+CEMODE: (0-3)")
```

```
    elif ub.endswith("?"):
        self.out("+CEMODE: 1")
    self.ok(); return

# ── WHAT oFONO ASKS AT INITIALISATION ──────────────────────────
# As soon as Powered=true, oFono runs through the whole inventory of
# a real modem - character sets, SMS services, and above all a long
# series of AT+CRSM (reading the SIM card files). Answering ERROR to
# all that makes oFono give up; so answer like a modem whose SIM does
# not have these files, which is THE TRUTH here: there is no card,
# the identity comes from the bench (AT+CIMI), not from a physical
# SIM.
if ub == "+CSCS=?":
    self.out('+CSCS: ("IRA","GSM","UCS2")'); self.ok(); return
if ub.startswith("+CSMS"):
    # 1,1,1: sent, received and broadcast SMS supported.
    self.out("+CSMS: 1,1,1"); self.ok(); return
if ub.startswith("+CSIM"):
    # FLATLY REFUSE APDU ACCESS. ModemManager probes the card with
    # AT+CSIM (raw APDU commands: file selection, IMSI read...).
    # Answering "OK" without data would leave it waiting for a reply
    # that never comes, and it would register no modem. Announcing
    # the command as unsupported makes it fall back to the ordinary
    # commands (+CIMI, +CPIN?), which are answered - there is no real
    # card here, the identity comes from the bench.
    self.err(4)          # 4 = operation not supported
    return
if ub.startswith("+CRSM"):
    # 148,4 = SW1 0x94 / SW2 0x04, "file not found": the normal reply
    # of a SIM that does not carry the requested file.
    self.out("+CRSM: 148,4"); self.ok(); return
# THE COMMAND THAT DECIDES EVERYTHING: AT+GCAP. Through it oFono
# asks the modem what it can do; without "+CGSM" in the reply, it
# does not enable the network layer and NEVER sends AT+CREG - the
# NetworkRegistration interface then stays absent.
if ub in ("+GCAP", "+CGCAP"):
    self.out("+GCAP: +CGSM,+FCLASS,+DS,+ES"); self.ok(); return

# ── THE INBOX, SEEN THROUGH AT ─────────────────────────────────
# THE TEST FORM FIRST. "AT+CMGF=?" starts with "+CMGF=": it must be
# matched before the prefix test below, which would answer a bare OK
# (and, for AT+CNMI=?, reset cnmi_mt to 2).
if ub == "+CMGF=?":
    self.out("+CMGF: (0,1)"); self.ok(); return
if ub == "+CNMI=?":
    self.out("+CNMI: (0-2),(0-3),(0,2),(0,1),(0,1)"); self.ok(); return
if ub.startswith("+CMGF="):
    # Remember the mode: CMGR and CMGL do not answer the same way in
    # text and in PDU, and answering in the wrong mode gets the
    # message dropped.
    self.cmgf = 1 if ub[6:].strip().startswith("1") else 0
    self.ok(); return
if ub == "+CMGF?":
    self.out("+CMGF: %d" % self.cmgf); self.ok(); return

if ub.startswith("+CNMI="):
    # THE COMMAND THAT DECIDES EVERYTHING FOR INCOMING SMS. The
    # second field (mt) says whether the system wants the message
    # pushed (2) or only its index (1). ModemManager asks for 1.
    champs = ub[6:].split(",")
    try:
        self.cnmi_mt = int(champs[1]) if len(champs) > 1 else 2
    except ValueError:
        self.cnmi_mt = 2
    log("annonce des SMS entrants : mt=%d (%s)"
        % (self.cnmi_mt,
```

```
            "+CMT direct" if self.cnmi_mt == 2 else "+CMTI puis CMGR"))
        self.ok(); return
    if ub == "+CNMI?":
        self.out("+CNMI: 2,%d,2,1,0" % self.cnmi_mt); self.ok(); return

    if ub.startswith("+CNMA"):
        # The acknowledgement of a "+CMT:". Nothing to do: the message
        # has already left the bench. But OK MUST be answered, otherwise
        # the system believes delivery failed and turns off direct
        # announcement.
        self.ok(); return

    if ub.startswith("+CMGR="):
        try:
            idx = int(re.sub(r"\D", "", ub[6:]) or "0")
        except ValueError:
            self.err(); return
        m = inbox_get(idx)
        if m is None:
            self.err(321)            # 321 = invalid memory index
            return
        self.out(self._sms_ligne(m, "+CMGR: "))
        m["lu"] = True
        self.ok(); return

    if ub.startswith("+CMGL"):
        if ub.endswith("=?"):
            self.out('+CMGL: (0,1,2,3,4)' if not self.cmgf else
                     '+CMGL: ("REC UNREAD","REC READ","ALL")')
            self.ok(); return
        arg = ub[6:].strip().strip('"').upper() if "=" in ub else "4"
        # 0 / "REC UNREAD": only the unread ones. Anything else: all.
        que_non_lus = arg in ("0", "REC UNREAD")
        with INBOX_LOCK:
            liste = [m for m in INBOX if not (que_non_lus and m["lu"])]
        for m in liste:
            self.out(self._sms_ligne(m, "+CMGL: %d," % m["idx"]))
        for m in liste:
            m["lu"] = True
        self.ok(); return

    if ub.startswith("+CMGD"):
        if ub.endswith("=?"):
            with INBOX_LOCK:
                idxs = ",".join(str(m["idx"]) for m in INBOX)
            self.out("+CMGD: (%s),(0-4)" % idxs); self.ok(); return
        champs = ub[6:].split(",")
        try:
            idx = int(re.sub(r"\D", "", champs[0]) or "0")
        except ValueError:
            self.err(); return
        drapeau = champs[1].strip() if len(champs) > 1 else "0"
        with INBOX_LOCK:
            if drapeau in ("1", "2", "3", "4"):
                INBOX[:] = []            # "delete all", per the flag
            else:
                INBOX[:] = [m for m in INBOX if m["idx"] != idx]
        self.ok(); return

    if ub == "+CSCS?":
        self.out('+CSCS: "IRA"'); self.ok(); return
    if ub.startswith("+CPMS"):
        # Three SMS memories, all reported as the same inbox. The count
        # must be TRUE: a system that reads "0 messages" will not fetch
        # what has just been stored.
        with INBOX_LOCK:
```

```
            n = len(INBOX)
        if ub.endswith("=?"):
            self.out('+CPMS: ("SM"),("SM"),("SM")')
        elif ub.startswith("+CPMS="):
            self.out('+CPMS: %d,%d,%d,%d,%d,%d'
                     % (n, INBOX_MAX, n, INBOX_MAX, n, INBOX_MAX))
        else:
            self.out('+CPMS: "SM",%d,%d,"SM",%d,%d,"SM",%d,%d'
                     % (n, INBOX_MAX, n, INBOX_MAX, n, INBOX_MAX))
        self.ok(); return
    if ub == "+CSMS?":
        self.out("+CSMS: 0,1,1,1"); self.ok(); return
    if ub == "+CGDCONT=?":
        self.out('+CGDCONT: (1-4),"IP",,,(0-2),(0-4)'); self.ok(); return
    if ub in ("+CMUT?", "+CMUT=?"):
        self.out("+CMUT: 0"); self.ok(); return
    if ub in ("+CLVL?", "+CLVL=?"):
        self.out("+CLVL: 5" if ub.endswith("?") and "=" not in ub else "+CLVL: (0-9)")
        self.ok(); return

    if ub in ("+CPINR", "+CUAD") or ub.startswith(("+PTTY", "+VTD", "+CCWA",
                                                   "+CIND", "+CMER", "+CPBS",
                                                   "+CSCB", "+CGATT", "+CPMS")):
        self.ok(); return

    if ub == "+CLCC=?":
        # THE COMMAND THAT DECIDES THE HANGUP. ModemManager first asks
        # AT+CLCC=? and, on ERROR, concludes that this modem cannot
        # return its call list - it then stops the periodic re-read and
        # has nothing left to learn that a call has ended. A real modem
        # answers OK.
        self.ok(); return

    if ub == "+CLCC":
        for i, c in enumerate(self.calls, start=1):
            # 27.007: idx, direction, state, mode, multiparty, number,
            # type. Direction 0 = outgoing, 1 = incoming. State 0 =
            # active, 2 = dialling, 4 = incoming ringing.
            sens = 1 if c["state"] == "incoming" else 0
            etat = {"active": 0, "incoming": 4}.get(c["state"], 2)
            _n, _t = clip(c["num"])
            self.out('+CLCC: %d,%d,%d,0,0,"%s",%d'
                     % (i, sens, etat, _n, _t))
        self.ok(); return

    if ub.startswith("+CMGS="):
        # Two forms: text mode (the number in quotes, for a manual test)
        # and PDU mode (a length - what oFono sends). In both cases the
        # rest arrives after the "> " prompt.
        m = re.search(r'"([^"]+)"', body)
        self.sms_target = m.group(1) if m else "<pdu>"
        try:
            self.wfile.write(b"\r\n> ")
            self.wfile.flush()
        except OSError:
            pass
        return

    if ub == "A":
        # ANSWER. The call ALREADY goes to the bench subscriber - the VM
        # and the mobile are the same 100101, as for SMS. The line cannot
        # be "taken": note that the call is in progress, and the voice
        # comes through the audio bridge (gsm_audio), which the VM
        # already relays.
        for c in self.calls:
            if c["state"] == "incoming":
```

```
            c["state"] = "active"
            log("appel de %s decroche" % (c["num"] or "inconnu"))
            threading.Thread(target=mobile_decroche, daemon=True).start()
            self.ok(); return
    self.err(); return

if cmd.startswith("ATD"):
    num = re.sub(r"[^0-9+*#]", "", body.split(";")[0][1:])
    if not num:
        self.err(); return
    if not num.startswith("*99") and ("*" in num or "#" in num):
        # A USSD CODE IS NOT A CALL. Phosh's dialler sends "*#100#"
        # as ATD: refused (27.007: operation not allowed); USSD goes
        # through AT+CUSD, below.
        log("ATD%s : code USSD/SS, pas un appel - refuse" % num)
        self.err(3); return
    if num.startswith("*99"):
        # ATD*99***<cid>#: DATA, by PPP on this port (see the Ppp
        # class). Answer CONNECT and the port switches to data mode:
        # NetworkManager starts pppd in the phone, and we hold the
        # other end. The phone's kernel must have
        # ppp_generic/ppp_async (built for it, see tools/osmo-pmos.sh)
        # - without them pppd dies and the port comes back to AT by
        # itself (Ppp.feed recognises an "AT").
        if not LTE.state()["ok"] and not st["up"]:
            log("data : ATD%s refuse, aucun reseau" % num)
            self.err(30)                # 30 = no network service
            return
        # NO DATA DURING A CS SERVICE. On 2G, a mobile in a call has
        # no data (class B); and here, with a SINGLE port, accepting
        # ATD*99 in the middle of a call would put the port back in
        # data mode: the NO CARRIER of the other party hanging up and
        # the +CEREG of the return to 4G would stay held behind the
        # PPP. NetworkManager retries by itself after the service.
        if DATA is None and self.cs_actif():
            log("data : ATD%s refuse le temps du service CS (CSFB)" % num)
            self.out("NO CARRIER")
            return
        m = re.search(r"\*\*\*(\d+)#", num)
        self.data_cid = int(m.group(1)) if m else 1
        log("data : ATD%s -> CONNECT, PPP sur le port AT (cid %d, APN %s), par %s"
            % (num, self.data_cid, LTE.state()["apn"],
               "la 4G (srsUE)" if self.rat() == "lte" else "la 2G"))
        # CONNECT AFTER THE PDP, like a real modem. Answering CONNECT
        # right away would leave the phone's PPP session without an
        # egress during attach + PDP (1 to 3 min on the Calypso).
        # The data port waits for the switch and the PDP; on failure,
        # NO CARRIER, and NetworkManager asks again by itself.
        if GPRS_DATA and self.rat() == "gsm":
            pdp = _sh("ip", "-n", GPRS_NETNS, "-4", "-o", "addr", "show", GPRS_TUN)
            if not (pdp[0] == 0 and "inet" in pdp[1]):
                log("data : ATD%s - le Calypso passe au GPRS, CONNECT a l arrivee du PDP" % num)
                if not calypso("data", attendre=True):
                    log("data : pas de PDP GPRS, NO CARRIER (le telephone redemandera)")
                    self.out("NO CARRIER")
                    return
        self.out("CONNECT")
        self.port_commande = False
        self.ppp = Ppp(self)
        self.ppp.demarrer()
        return
    # CSFB: the call is made on 2G. Go down BEFORE dialling, as a
    # real UE leaves LTE on the Extended Service Request.
    self.csfb_debut("appel vers %s" % num, couper_data=(DATA is None))
    # "origine" = the REQUEST name of the Local channel
    # ("Local/600@internal"), which Asterisk expands to
```

```
    # "...-0000000b;1" and ";2". It is how our own legs are
    # recognised in the AMI events (cf. est_notre_jambe); "channel"
    # will be replaced by the real name.
    # RECORDED BEFORE DIALLING, NOT AFTER: the AMI events arrive on
    # ANOTHER connection (AmiEcoute) and the DialBegin of our second
    # leg (Exten 100101) can precede the return of ami_originate;
    # est_notre_jambe must already find the call.
    chan = "Local/%s@%s" % (num, CTX)
    appel = {"num": num, "state": "active", "channel": chan, "origine": chan}
    self.calls.append(appel)
    ok, detail = ami_originate(num)
    if not ok:
        log("appel vers %s refuse : %s" % (num, detail))
        if appel in self.calls:
            self.calls.remove(appel)
        self.csfb_fin_apres()
        self.err(); return
    log("appel vers %s etabli par Asterisk" % num)
    threading.Thread(target=mobile_decroche, daemon=True).start()
    self.ok()
    return

if ub == "+CHLD=?":
    # 0 reject/release, 1 release the active ones, 1x release call x.
    self.out('+CHLD: (0,1,1x)'); self.ok(); return

if ub in ("H", "+CHUP") or ub.startswith("+CHLD="):
    # ModemManager hangs up with +CHLD (27.007 § 7.13: 0 = reject the
    # incoming, 1 = release the active ones, 1x = release call x),
    # not with ATH. Without call hold on this bench, all three come
    # down to the same thing - hang up. +CHLD must be handled here
    # and not fall into the general "any ASSIGNMENT is accepted
    # without effect" rule below.
    #
    # WE hang up, so WE announce it, right away, once per call:
    # self.calls is cleared here, so fin_appel() finds nothing to
    # remove when Asterisk tears the channel down and would never
    # emit the NO CARRIER. fin_appel() stays in place for the other
    # direction (the other party hangs up first) and does not
    # duplicate: the list is empty when its event arrives.
    for c in self.calls:
        ami_hangup(c.get("channel", ""))
    n = len(self.calls)
    self.calls.clear()
    mobile("hangup")            # the subscriber is the mobile: it too
    for _ in range(n):
        self.out("NO CARRIER")
    self.csfb_fin_apres()       # nothing left in CS: back to 4G
    self.ok(); return

if ub.startswith("+CUSD=1,"):
    # USSD, a minimum: *#100# returns the subscriber's number (as on
    # the osmocom-bb mobile); any other code is "released by the
    # network" (+CUSD: 2), without error nor drop to 2G.
    m = re.search(r'"([^"]*)"', body)
    code = m.group(1) if m else ""
    self.ok()
    if code.replace("*", "").replace("#", "") == "100":
        self.out('+CUSD: 0,"Votre numero : %s",15' % MSISDN)
    else:
        log("USSD %s : pas de service, on libere" % code)
        self.out("+CUSD: 2")
    return
if ub == "+CBC":
    self.out("+CBC: 0,80"); self.ok(); return       # powered, 80 %
if ub == "+CLCK=?":
```

```
            self.out('+CLCK: ("SC","AO","OI","AI")'); self.ok(); return

        # THE GENERAL RULE, AND WHY IT IS SAFE. At initialisation oFono runs
        # through some thirty SETTINGS (AT+CRC=1, AT+CNAP=1, AT+CDIP=1,
        # AT+CSCS="GSM"...) whose reply it never reads beyond the OK;
        # refusing them makes it abandon the whole initialisation. So
        # anything that is an ASSIGNMENT is accepted - a modem that accepts
        # a setting it does not apply lies to nobody, it has nothing to
        # report about it. An unknown READ (AT+XXX? or AT+XXX=?), on the
        # other hand, stays refused: there, an invented reply would make
        # oFono believe in a capability that does not exist.
        if "=" in ub and not ub.endswith("=?"):
            log("reglage accepte sans effet : %s" % line)
            self.ok(); return

        log("lecture non geree (refusee) : %s" % line)
        self.err()


# ── SMS RECEPTION: SMPP ESME ON osmo-msc ──────────────────────────────
# Sending an SMS is done through the VTY ("subscriber ... sms ... send"), but
# RECEIVING requires the MSC to push something to us: the VTY notifies
# nothing. For that osmo-msc exposes an SMPP server (port 2775) to which we
# connect as an ESME. Each SMS routed to us arrives as deliver_sm, and is
# presented to the modem in the form the AT world expects: "+CMT:" with the
# message as a PDU, preceded by the URC.
#
# The protocol is binary but only a handful of messages are used:
# bind_transceiver, enquire_link (the heartbeat), deliver_sm and its reply.
# No library: it would be one more dependency on the image for three
# structures.
SMPP_BIND_TRANSCEIVER = 0x00000009
SMPP_BIND_TRANSCEIVER_RESP = 0x80000009
SMPP_DELIVER_SM = 0x00000005
SMPP_DELIVER_SM_RESP = 0x80000005
SMPP_ENQUIRE_LINK = 0x00000015
SMPP_ENQUIRE_LINK_RESP = 0x80000015


def _cstr(b, i):
    """A zero-terminated string, and the index right after it."""
    j = b.index(b"\x00", i)
    return b[i:j].decode(errors="replace"), j + 1


def gsm7_pack(text):
    """The GSM 7-bit alphabet, packed seven octets for eight characters -
    the default encoding of an SMS."""
    # The EXACT mirror of gsm7_unpack: septets are stacked into a bit buffer
    # and an octet is output as soon as there are eight bits.
    septets = [ord(c) & 0x7F for c in text]
    out = bytearray()
    cur = bits = 0
    for sep in septets:
        cur |= sep << bits
        bits += 7
        while bits >= 8:
            out.append(cur & 0xFF)
            cur >>= 8
            bits -= 8
    if bits:
        out.append(cur & 0xFF)
    return bytes(out), len(septets)


# ── THE INBOX ─────────────────────────────────────────────────────────
```

```
# WHY SMS MUST BE STORED. The modem does not decide alone how it announces an
# incoming SMS: the system tells it through AT+CNMI=<mode>,<mt>,... Two
# worlds:
#   mt=2: "push me the message right away"      -> URC "+CMT:" + the PDU
#   mt=1: "store it and only tell me where"     -> URC '+CMTI: "SM",<i>'
#         then the system reads it with AT+CMGR=<i>.
# ModemManager (postmarketOS VM) negotiates mt=1 ("AT+CNMI=2,1,2,1,0") and
# even lists the inbox at startup with AT+CMGL=4. Hence this store, and the
# commands that go with it (CMGR, CMGL, CMGD).
INBOX = []                    # [{idx, pdu, sender, text, lu, ts}]
INBOX_LOCK = threading.Lock()
INBOX_MAX = 20                # what +CPMS announces as capacity


INBOX_DERNIER = [0]           # the last index assigned (see inbox_ranger)


def inbox_ranger(sender, text, pdu):
    """Store an SMS and return its index, like an "SM" memory.

    THE INDEXES ROTATE. ModemManager sometimes keeps a deleted message in
    ITS list (its deletion failed on its side, "SMS storage currently
    locked, try again later", although the modem said OK), and a +CMTI on an
    index it believes already known is ignored. So the NEXT free index is
    always taken, rotating over the 20: an index is reused only after 19
    others."""
    with INBOX_LOCK:
        pris = {m["idx"] for m in INBOX}
        if len(pris) >= INBOX_MAX:
            INBOX.pop(0)      # inbox full: the oldest gives way
            pris = {m["idx"] for m in INBOX}
        idx = INBOX_DERNIER[0]
        for _ in range(INBOX_MAX):
            idx = idx % INBOX_MAX + 1
            if idx not in pris:
                break
        INBOX_DERNIER[0] = idx
        INBOX.append({"idx": idx, "pdu": pdu, "sender": sender, "text": text,
                      "lu": False, "ts": time.strftime("%y/%m/%d,%H:%M:%S+08")})
        INBOX.sort(key=lambda m: m["idx"])
    return idx


def inbox_get(idx):
    with INBOX_LOCK:
        for m in INBOX:
            if m["idx"] == idx:
                return m
    return None


def semi_octets(num):
    """A number in swapped nibbles, as in a PDU."""
    digits = re.sub(r"\D", "", num)
    if len(digits) % 2:
        digits += "F"
    return bytes(int(digits[i + 1] + digits[i], 16) for i in range(0, len(digits), 2))


def scts_maintenant():
    """The timestamp of an SMS-DELIVER (23.040 § 9.2.3.11): six pairs of
    digits in swapped nibbles, then the time zone in quarters of an hour.

    A real modem gives the network time; the bench time is given, which is
    the same. An all-zero timestamp (year 00 / month 00 / day 00) is not a
    valid date and Chatty may drop the message when reading it.
```

```
    """
    t = time.localtime()
    out = bytearray(int(("%02d" % v)[::-1], 16) for v in
                    (t.tm_year % 100, t.tm_mon, t.tm_mday,
                     t.tm_hour, t.tm_min, t.tm_sec))
    quarts = -(time.altzone if t.tm_isdst else time.timezone) // 900
    octet = int(("%02d" % abs(quarts))[::-1], 16)
    if quarts < 0:
        octet |= 0x08                 # the sign, bit 3 of the first nibble
    out.append(octet)
    return bytes(out)


def pdu_deliver(sender, text):
    """A complete SMS-DELIVER, as a modem returns it on "+CMT:"."""
    digits = re.sub(r"\D", "", sender)
    oa = bytes([len(digits), 0x81]) + semi_octets(sender)   # 0x81: national number
    ud, n = gsm7_pack(text)
    scts = scts_maintenant()
    pdu = b"\x00" + b"\x04" + oa + b"\x00\x00" + scts + bytes([n]) + ud
    return pdu.hex().upper()


def gsm7_unpack(data, n):
    """The inverse of gsm7_pack: the packed septets become characters again."""
    out, cur, bits = [], 0, 0
    for b in data:
        cur |= b << bits
        bits += 8
        while bits >= 7 and len(out) < n:
            out.append(chr(cur & 0x7F))
            cur >>= 7
            bits -= 7
    return "".join(out)


def parse_submit(pdu_hex):
    """Read an SMS-SUBMIT and return (number, text).

    oFono does NOT send SMS in text mode: it sends "AT+CMGS=<length>" then
    a PDU, so decoding is required here.
    """
    b = bytes.fromhex(pdu_hex)
    i = 0
    i += 1 + b[0]                     # SMSC length, then skip it
    first = b[i]; i += 1              # type octet
    i += 1                            # message reference
    ndigits = b[i]; i += 1
    i += 1                            # address type
    nbytes = (ndigits + 1) // 2
    num = ""
    for octet in b[i:i + nbytes]:
        num += "%d%d" % (octet & 0x0F, octet >> 4)
    num = num[:ndigits]
    i += nbytes
    i += 1                            # protocol
    dcs = b[i]; i += 1
    if first & 0x10:                  # validity period present
        i += 1
    udl = b[i]; i += 1
    ud = b[i:]
    if dcs & 0x0C == 0x08:            # UCS2
        text = ud[:udl].decode("utf-16-be", errors="replace")
    else:
        text = gsm7_unpack(ud, udl)
    return num, text
```

```
class Smpp(threading.Thread):
    """The ESME: binds to the MSC, keeps the link, and delivers SMS to the modem."""

    daemon = True

    def __init__(self, deliver_cb):
        super().__init__(name="smpp")
        self.deliver = deliver_cb
        self.seq = 0
        self.s = None

    def _pack(self, cmd_id, body=b"", status=0):
        self.seq += 1
        return (len(body) + 16).to_bytes(4, "big") + cmd_id.to_bytes(4, "big") + \
            status.to_bytes(4, "big") + self.seq.to_bytes(4, "big") + body

    def _bind(self):
        body = (SMPP_ID.encode() + b"\x00" + SMPP_PW.encode() + b"\x00" +
                b"osmo\x00" + b"\x34" + b"\x00\x00" + b"\x00")
        self.s.sendall(self._pack(SMPP_BIND_TRANSCEIVER, body))
        head = self._recv(16)
        if not head:
            return False
        ln = int.from_bytes(head[:4], "big")
        cmd = int.from_bytes(head[4:8], "big")
        status = int.from_bytes(head[8:12], "big")
        self._recv(ln - 16)
        if cmd == SMPP_BIND_TRANSCEIVER_RESP and status == 0:
            log("SMPP : lie a osmo-msc comme « %s »" % SMPP_ID)
            return True
        log("SMPP : liaison refusee (statut %d)" % status)
        return False

    def _recv(self, n):
        buf = b""
        while len(buf) < n:
            try:
                chunk = self.s.recv(n - len(buf))
            except OSError:
                return None
            if not chunk:
                return None
            buf += chunk
        return buf

    def run(self):
        while True:
            try:
                self.s = socket.create_connection((SMPP_HOST, SMPP_PORT), timeout=5)
                self.s.settimeout(None)
                if not self._bind():
                    self.s.close()
                    time.sleep(5)
                    continue
                self._loop()
            except OSError as e:
                log("SMPP : %s" % e)
            finally:
                try:
                    if self.s:
                        self.s.close()
                except OSError:
                    pass
            time.sleep(5)          # the MSC restarts, come back to it
```

```
    def _loop(self):
        while True:
            head = self._recv(16)
            if not head:
                return
            ln = int.from_bytes(head[:4], "big")
            cmd = int.from_bytes(head[4:8], "big")
            seq = head[12:16]
            body = self._recv(ln - 16) if ln > 16 else b""
            if body is None:
                return
            if cmd == SMPP_ENQUIRE_LINK:
                self.s.sendall(b"\x00\x00\x00\x10" +
                               SMPP_ENQUIRE_LINK_RESP.to_bytes(4, "big") +
                               b"\x00\x00\x00\x00" + seq)
            elif cmd == SMPP_DELIVER_SM:
                try:
                    self._on_deliver(body)
                except (ValueError, IndexError) as e:
                    log("SMPP : deliver_sm illisible (%s)" % e)
                self.s.sendall(b"\x00\x00\x00\x11" +
                               SMPP_DELIVER_SM_RESP.to_bytes(4, "big") +
                               b"\x00\x00\x00\x00" + seq + b"\x00")

    def _on_deliver(self, b):
        i = 0
        _svc, i = _cstr(b, i)
        i += 2                                  # source ton and npi
        src, i = _cstr(b, i)
        i += 2                                  # destination ton and npi
        dst, i = _cstr(b, i)
        i += 3                                  # esm_class, protocol, priority
        _sched, i = _cstr(b, i)
        _valid, i = _cstr(b, i)
        i += 3                                  # registered, replace, coding
        i += 1                                  # sm_default_msg_id
        ln = b[i]; i += 1
        text = b[i:i + ln].decode("latin-1", errors="replace")
        log("SMS entrant de %s pour %s : %s" % (src, dst, text))
        self.deliver(src, text)


class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True


CURRENT = None                  # the modem currently opened by oFono
# THE MODEM HAS TWO PORTS. On a single port, once ATD*99 has gone through,
# nothing flows in AT any more: the phone can neither call nor send an SMS
# while data is up ("No AT port available"), and incoming ones only arrive
# by cutting the PPP. A real modem has a command port and a data port. Here
# too: CURRENT holds the command port (hvc0, URCs, calls, SMS); DATA holds
# the data port (hvc1, the one ModemManager marks ID_MM_PORT_TYPE_AT_PPP and
# on which it dials ATD*99). Both are AtHandlers on the same bench state; the
# data port refers to CURRENT for the phone state (technology, call in
# progress). Without --data, single port.
DATA = None                     # the data port, if there is one


def on_smpp_sms(sender, text):
    """An SMS arrives from the bench. Two recipients, not one:

    THE ANDROID PHONE IS THE SAME SUBSCRIBER AS THE MOBILE. The bench has
    only one 100101, and it must ring on BOTH sides - in Android through the
```

```
    RIL, and on the osmocom-bb mobile. "smpp-first" diverts the SMS to us
    BEFORE internal delivery, so the mobile would see nothing: it is handed
    over by us, through the MSC VTY command, which delivers directly to the
    mobile without going through SMPP again - no loop possible.
    OSMO_SMS_DUPLIQUE=0 to keep only one (Android alone).
    """
    if CURRENT is not None:
        CURRENT.deliver_sms(sender, text)
    else:
        log("oFono n est pas connecte : rien a remettre a Android")
    if DUPLIQUE:
        out = MSC.cmd("subscriber msisdn %s sms sender msisdn %s send %s"
                      % (MSISDN, sender, text))
        if out is None:
            log("copie au mobile impossible : osmo-msc injoignable")
        else:
            log("copie remise au mobile %s" % MSISDN)


# ── THE INBOX: THE SMS THE GATEWAY HANDS TO US ──────────────────────────
# WHY A SECOND PATH, BESIDE SMPP. The bench runs with "sms-over-gsup"
# (osmo-msc.cfg). In that mode, an SM-RP received from the radio goes to the
# HLR WITHOUT ever passing through gsm340_rx_tpdu(): sms_route_mt_sms() is
# not called, hence smpp_try_deliver() neither. "smpp-first" and the "route
# prefix ... 100101" lines of the conf are dead configuration as long as
# sms-over-gsup is there - the ESME above is BOUND to port 2775 and receives
# NOTHING. And the MT that comes back through GSUP is pushed directly to the
# subscriber over the radio: the osmocom-bb mobile gets it, the software
# modems do not.
#
# The gateway therefore notifies the modems where it decides local delivery
# (scripts/sms-interop-relay.py, inject_mt_sms), through
# scripts/sms_notify.py. NOTHING is duplicated here, unlike the SMPP path:
# the mobile already has its copy over the radio, sending it again would
# make it ring twice.
class SmsInHandler(socketserver.StreamRequestHandler):
    """One JSON line: {"imsi": ..., "from": ..., "text": ...}."""

    def handle(self):
        try:
            ligne = self.rfile.readline(65536)
        except OSError:
            return
        try:
            msg = json.loads(ligne.decode("utf-8", errors="replace"))
        except ValueError as e:
            log("passerelle : message illisible (%s)" % e)
            return
        imsi = str(msg.get("imsi") or "")
        if imsi and imsi != IMSI:
            return                      # that SMS is for another subscriber
        on_passerelle_sms(str(msg.get("from") or ""), str(msg.get("text") or ""))
        try:
            self.wfile.write(b'{"ok":true}\n')
        except OSError:
            pass


def on_passerelle_sms(sender, text):
    """No system at the end of the line? Store anyway: the VM may be
    booting, and ModemManager lists the inbox with AT+CMGL=4 as soon as it
    connects. An SMS must not be lost because the modem was absent."""
    if CURRENT is not None:
        CURRENT.deliver_sms(sender, text)
        return
    idx = inbox_ranger(sender, text, pdu_deliver(sender, text))
```

```
    log("SMS de la passerelle range en SM[%d], personne au bout du fil : %s"
        % (idx, text))


def boite_entree(port):
    """Open the inbox. Port already taken = the other modem instance has it:
    say so and go on, everything else in the modem works without it."""
    try:
        srv = Server((SMS_IN_HOST, port), SmsInHandler)
    except OSError as e:
        log("boite d entree SMS sur %d impossible (%s)" % (port, e))
        return
    log("boite d entree SMS sur %s:%d" % (SMS_IN_HOST, port))
    threading.Thread(target=srv.serve_forever, daemon=True).start()


class ClientHandler(AtHandler):
    """The same modem, but WE go towards the port.

    WHY THIS MODE. When the modem is plugged into a virtual machine serial
    port, the VM's UEFI firmware treats that port as a console: it reads the
    modem's replies as key presses, believes the boot is being interrupted
    and opens its menu - the VM never boots. The remedy is that NOBODY is at
    the end of the line while the firmware works: QEMU listens
    (server=on,wait=off), and we connect once the system is loaded.
    """

    def __init__(self, sock, addr, role="commande"):
        # StreamRequestHandler.setup() expects self.request and builds
        # connection/rfile/wfile itself: just give it the socket.
        self.role = role
        self.request = sock
        self.client_address = addr
        self.server = None
        self.setup()
        try:
            self.handle()
        finally:
            self.finish()


def vm_prete(port):
    """Does the VM's system answer? A witness port is used (the virtual
    machine's SSH, 2222 by default).

    WHY WAIT. The VM's UEFI firmware reads any serial port as a console: if
    the modem connects while it works, its replies pass for key presses and
    the boot stops on the menu. The --connect mode loops every five seconds,
    so it must connect ONLY once the system is up.
    """
    try:
        s = socket.create_connection(("127.0.0.1", port), timeout=2)
        s.close()
        return True
    except OSError:
        return False


def connect_to(dest, role="commande"):
    """dest = "host:port": connect and hold the modem on that link."""
    host, _, port = dest.rpartition(":")
    temoin = int(os.environ.get("OSMO_VM_READY_PORT", "2222"))
    deja_dit = False
    while True:
        if temoin and not vm_prete(temoin):
            log("la VM n est pas prete (port %d ferme) - on patiente" % temoin)
```

```
            while not vm_prete(temoin):
                time.sleep(5)
            log("VM prete")
        try:
            s = socket.create_connection((host or "127.0.0.1", int(port)), timeout=10)
        except OSError as e:
            if not deja_dit:
                log("connexion a %s (port de %s) impossible (%s) - on reessaie toutes les 5 s"
                    % (dest, role, e))
                deja_dit = True
            time.sleep(5)
            continue
        deja_dit = False
        # THE TIMEOUT IS FOR CONNECTING, NOT FOR HOLDING THE LINK.
        # create_connection(timeout=10) leaves that timeout ON the socket:
        # the next read would raise socket.timeout after ten seconds of
        # silence and the modem would unplug itself. A modem is silent most
        # of the time - ModemManager only queries it in bursts. So switch
        # back to blocking, like the SMPP link above.
        s.settimeout(None)
        log("connecte a %s (port de %s)" % (dest, role))
        try:
            ClientHandler(s, (host, int(port)), role)
        except OSError as e:
            log("lien rompu : %s" % e)
        finally:
            try:
                s.close()
            except OSError:
                pass
        log("lien ferme, on repasse dans 5 s")
        time.sleep(5)


def main():
    LTE.resume()
    if SMPP_ON:
        Smpp(on_smpp_sms).start()
    AmiEcoute().start()
    data = None
    for i, a in enumerate(sys.argv):
        if a == "--data" and i + 1 < len(sys.argv):
            data = sys.argv[i + 1]
    for i, a in enumerate(sys.argv):
        if a == "--connect" and i + 1 < len(sys.argv):
            boite_entree(SMS_IN_CONNECT)
            if data:
                threading.Thread(target=connect_to, args=(data, "data"), daemon=True).start()
            return connect_to(sys.argv[i + 1])
    boite_entree(SMS_IN_LISTEN)
    srv = Server((HOST, PORT), AtHandler)
    log("modem du banc en ecoute sur %s:%d" % (HOST, PORT))
    log("declarer dans /etc/ofono/phonesim.conf :")
    log("  [osmo]")
    log("  Address=%s" % HOST)
    log("  Port=%d" % PORT)
    log("puis : systemctl restart ofono")
    try:
        srv.serve_forever()
    except KeyboardInterrupt:
        pass
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

2.268 /opt/GSM/osmo-operator/tools/osmo-pmos-build.sh

16290 octets, 280 lignes → 280 lignes

```
#!/bin/bash
# ============================================================================
# osmo-pmos-build.sh - THE postmarketOS KERNEL WITH PPP, THE IMAGE, AND THE VM
# THAT GOES WITH IT.
#
# WHY. The phone 4G data (ATD*99 -> CONNECT) needs PPP: the stock
# linux-postmarketos-stable kernel has no PPP ("# CONFIG_PPP is not set") and
# refuses unsigned modules (lockdown). Modules built WITH the kernel are
# signed by the build key, so the kernel is rebuilt with
# CONFIG_PPP/CONFIG_PPP_ASYNC, then the image.
#
# TWO PATHS TO THIS KERNEL:
#   1. THE SHORTCUT: /opt/user_interface/kernel/pmos/ (installed by the
#      osmo-build-pmos-kernel .deb, in the ISO and in /var/cache/osmo-debs)
#      carries the already built apk, its index, its public key and the
#      pmaports commit. They are copied into the pmbootstrap work directory and
#      "pmbootstrap install" takes that kernel: ZERO compilation.
#   2. THE SOURCES: no apk, or --rebuild -> pmbootstrap build of the kernel
#      (15 to 40 min).
#
# Run under the SESSION ACCOUNT (pmbootstrap refuses root; it asks for sudo
# itself), in a terminal:
#   osmo-pmos-build                 kernel + image + smartphone VM (720x1440)
#   osmo-pmos-build tablette        same, VM in tablet format (1280x800)
#   osmo-pmos-build --no-install    the kernel only
#   osmo-pmos-build --no-launch     kernel + image, without starting the VM
#   osmo-pmos-build --rebuild       recompile even if the apk with PPP is already there
#   osmo-pmos-build --init          only prepare pmbootstrap (config, pmaports, kernel)
#   JOBS=8 osmo-pmos-build          fewer cores (srsRAN runs alongside)
#
# PMBOOTSTRAP AND ITS WORK DIRECTORY. pmbootstrap is the one of
# /opt/user_interface/pmos/pmbootstrap (patched: patches/pmbootstrap-osmo-
# bench-qemu.patch - the modem on a PCI serial port, the bench sound card, the
# screen size, and port 5038 which is not an adb). The work directory is the
# one of the account's pmbootstrap_v3.cfg, else ~/.local/var/pmbootstrap
# (OSMO_PMB_WORK for another). Without config, one is written from the
# template /opt/user_interface/pmos/pmbootstrap_v3.cfg: Phosh, fr_FR,
# hostname pmos-gsm, and the modem packages (ppp, networkmanager-ppp...).
# pmaports is cloned at the kernel commit (pmaports.commit) and receives the
# PPP patch: same pkgver/pkgrel as the apk, otherwise pmbootstrap does not prefer it.
# ============================================================================
set -u

UI="${OSMO_UI_ROOT:-/opt/user_interface}"
PM="$UI/pmos"
KD="$UI/kernel/pmos"
PMB="${PMB:-}"
for _p in "$PMB" "$PM/pmbootstrap/pmbootstrap.py" "$(command -v pmbootstrap 2>/dev/null)"
 ↪  "$HOME/.local/bin/pmbootstrap"; do
    [ -n "$_p" ] && [ -x "$_p" ] && { PMB="$_p"; break; }
done
CFG="$HOME/.config/pmbootstrap_v3.cfg"
WORK="${OSMO_PMB_WORK:-}"
[ -z "$WORK" ] && [ -f "$CFG" ] && WORK="$(sed -n 's/^work *= *//p' "$CFG" | head -1)"
[ -z "$WORK" ] && WORK="$HOME/.local/var/pmbootstrap"
PMAPORTS="$(sed -n 's/^aports *= *//p' "$CFG" 2>/dev/null | head -1)"
[ -n "$PMAPORTS" ] || PMAPORTS="$WORK/cache_git/pmaports"
KCFG="$PMAPORTS/device/main/linux-postmarketos-stable/config-stable.x86_64"
IMG="$WORK/chroot_native/home/pmos/rootfs/qemu-amd64.img"
JOBS="${JOBS:-$(nproc)}"
PASS="${OSMO_PMOS_PASSWORD:-147147}"
FORMAT=smartphone
```

```
INSTALL=1
LAUNCH=1
REBUILD=0
INIT_ONLY=0

for a in "$@"; do
    case "$a" in
        smartphone|tablette) FORMAT="$a" ;;
        --no-install) INSTALL=0 ;;
        --no-launch)  LAUNCH=0 ;;
        --rebuild)    REBUILD=1 ;;
        --init)       INIT_ONLY=1 ;;
        *) echo "argument inconnu : $a (smartphone|tablette|--no-install|--no-launch|--rebuild|--init)" >&2; exit
         ↪  2 ;;
    esac
done

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; CYAN='\033[0;36m'; NC='\033[0m'
say()  { echo -e "${CYAN}→${NC} $*"; }
ok()   { echo -e "${GREEN}✓${NC} $*"; }
warn() { echo -e "${YELLOW}!${NC} $*"; }
die()  { echo -e "${RED}✗${NC} $*" >&2; exit 1; }

# ROOT ACCEPTED. The ISO session IS root (see osmo-pmos-qemu): pmbootstrap
# accepts --as-root; under a regular account, nothing changes.
PMB_OPTS=(); [ "$(id -u)" -eq 0 ] && { PMB_OPTS=(--as-root); warn "en root : pmbootstrap --as-root (dossier de
 ↪  travail de root)"; }
pmb() { "$PMB" "${PMB_OPTS[@]}" "$@"; }
[ -n "$PMB" ] && [ -x "$PMB" ] || die "pmbootstrap introuvable ($PM/pmbootstrap manquant : osmo-pmos-install)"

# ── 0. THE ACCOUNT'S pmbootstrap CONFIG ──────────────────────────────
if [ ! -f "$CFG" ]; then
    [ -f "$PM/pmbootstrap_v3.cfg" ] || die "pas de $CFG ni de gabarit $PM/pmbootstrap_v3.cfg"
    mkdir -p "$(dirname "$CFG")" "$WORK"
    sed "s#@WORK@#$WORK#g; s#@APORTS@#$PMAPORTS#g" "$PM/pmbootstrap_v3.cfg" > "$CFG"
    ok "config pmbootstrap ecrite : $CFG (work = $WORK)"
fi
mkdir -p "$WORK"
# The work directory version number: without it, pmbootstrap asks for
# "init". It is the one of pmb/config/__init__.py (work_version).
if [ ! -f "$WORK/version" ]; then
    _wv="$(grep -m1 -oE '^work_version *= *[0-9]+' "$(dirname "$PMB")/pmb/config/__init__.py" 2>/dev/null | grep
     ↪  -oE '[0-9]+$')"
    echo "${_wv:-8}" > "$WORK/version"
fi

# ── 1. pmaports, AT THE KERNEL COMMIT, with the PPP patch ────────────
_commit="$(cat "$KD/pmaports.commit" 2>/dev/null || true)"
# ONE osmo-pmos-build AT A TIME. Two launches (the icon clicked while a first
# clone is running) would step on each other: the second would see a .git
# without commit - the one of the clone IN PROGRESS - and erase it. Lock on
# the work directory, and never touch a clone that still has its git.
exec 9>"$WORK/.osmo-pmos-build.lock"
flock -n 9 || die "un autre osmo-pmos-build tourne deja (verrou $WORK/.osmo-pmos-build.lock) : attends-le, ou
 ↪  ferme-le"
# An interrupted clone (network cut, window closed) leaves a .git WITHOUT
# commit, and the build would die further down on "kernel config not found".
# A repository without HEAD, and without git running on it, is a missing one.
if [ -d "$PMAPORTS/.git" ] && ! git -C "$PMAPORTS" rev-parse --verify HEAD >/dev/null 2>&1 \
   && ! pgrep -f "git(-remote-https)? .*$PMAPORTS" >/dev/null 2>&1; then
    warn "pmaports : clone vide ou interrompu, on le refait ($PMAPORTS)"
    rm -rf "$PMAPORTS"
fi
# PMAPORTS WITHOUT GITLAB WHEN POSSIBLE. The full clone (hundreds of MB, slow
# GitLab) is unreliable from the live system, and git then erases everything.
# In order: the copy installed by the ISO (88-lte-pmos.sh: $PM/pmaports, at
```

```
# the kernel commit, no network); else A SINGLE commit fetched with
# --depth 1 (a few tens of MB); the full clone only if the commit is unknown.
if [ ! -d "$PMAPORTS/.git" ]; then
    mkdir -p "$(dirname "$PMAPORTS")"
    if [ -d "$PM/pmaports/.git" ] && git -C "$PM/pmaports" rev-parse --verify HEAD >/dev/null 2>&1; then
        say "pmaports : copie de celui de l ISO ($PM/pmaports -> $PMAPORTS)..."
        cp -a "$PM/pmaports" "$PMAPORTS" || die "copie de pmaports"
        # pmbootstrap reads channels.cfg in origin/main: the ref must exist.
        git -C "$PMAPORTS" rev-parse -q --verify origin/main >/dev/null 2>&1 \
            || git -C "$PMAPORTS" update-ref refs/remotes/origin/main HEAD
        ok "pmaports au commit $(git -C "$PMAPORTS" rev-parse --short HEAD) (celui de l ISO)"
    elif [ -n "$_commit" ] && [ "$_commit" != inconnu ]; then
        say "pmaports : le commit $_commit du noyau, seul (fetch --depth 1)..."
        ( git init -q "$PMAPORTS" && cd "$PMAPORTS" \
          && git remote add origin https://gitlab.postmarketos.org/postmarketOS/pmaports.git \
          && git fetch -q --depth 1 origin "$_commit" && git checkout -q -B main FETCH_HEAD \
          && git update-ref refs/remotes/origin/main FETCH_HEAD ) \
          || { rm -rf "$PMAPORTS"; die "pmaports : fetch du commit $_commit impossible (reseau ?) - relance quand
           ↪  GitLab repond"; }
        ok "pmaports au commit $_commit (celui du noyau)"
    else
        say "clone de pmaports ($PMAPORTS)..."
        git clone https://gitlab.postmarketos.org/postmarketOS/pmaports.git "$PMAPORTS" || die "clone pmaports"
    fi
fi
[ -f "$KCFG" ] || die "config noyau introuvable : $KCFG"
if grep -q '^CONFIG_PPP_ASYNC=m' "$KCFG"; then
    ok "config noyau : PPP deja active ($(grep -c '^CONFIG_PPP' "$KCFG") symboles)"
else
    say "config noyau : ajout de PPP"
    cp -a "$KCFG" "$KCFG.avant-ppp"
    _patch=""
    for _p in "$KD/pmaports-linux-postmarketos-stable-ppp.patch"
     ↪  "$PM/patches/pmaports-linux-postmarketos-stable-ppp.patch"; do
        [ -f "$_p" ] && { _patch="$_p"; break; }
    done
    if [ -n "$_patch" ] && git -C "$PMAPORTS" apply --check "$_patch" 2>/dev/null; then
        git -C "$PMAPORTS" apply "$_patch" && ok "patch pmaports applique ($_patch)"
    else
        python3 - "$KCFG" <<'PY' || die "patch de la config impossible"
import sys
p = sys.argv[1]; s = open(p).read()
old = "# CONFIG_PPP is not set\n"
new = ("CONFIG_PPP=m\nCONFIG_PPP_BSDCOMP=m\nCONFIG_PPP_DEFLATE=m\nCONFIG_PPP_FILTER=y\n"
       "CONFIG_PPP_MULTILINK=y\nCONFIG_PPP_ASYNC=m\nCONFIG_PPP_SYNC_TTY=m\n")
if old not in s:
    sys.exit("pas de « # CONFIG_PPP is not set » dans la config : a regarder a la main")
open(p, "w").write(s.replace(old, new, 1))
PY
        ok "config noyau : PPP=m, PPP_ASYNC=m (sauvegarde : $KCFG.avant-ppp)"
    fi
fi
# The checksum: abuild checks the sha512 of each source file, and the modified
# config no longer has the one of the APKBUILD ("FAILED, Use 'abuild
# checksum'"). It is always updated, it costs nothing.
APKB="$(dirname "$KCFG")/APKBUILD"
somme="$(sha512sum "$KCFG" | cut -d' ' -f1)"
if grep -q "^$somme  $(basename "$KCFG")\$" "$APKB"; then
    ok "APKBUILD : sha512 de la config a jour"
else
    sed -i "s/^[0-9a-f]\{128\}  $(basename "$KCFG")\$/$somme  $(basename "$KCFG")/" "$APKB" \
        && ok "APKBUILD : sha512 de la config mis a jour" \
        || die "sha512 de la config non remplace dans $APKB"
fi

# ── 2. THE pmbootstrap SETTINGS: Phosh, not the console ──────────────────
```

```
ui="$(pmb config ui 2>/dev/null | tail -1)"
if [ "$ui" != "phosh" ]; then
    warn "pmbootstrap config ui = « $ui » : un install ferait une VM sans bureau"
    pmb config ui phosh || die "impossible de poser ui=phosh"
fi
ok "ui = phosh, device = $(pmb config device 2>/dev/null | tail -1), jobs = $JOBS, work = $WORK"

# ── 3. THE KERNEL: THE SHORTCUT FIRST ──────────────────────────────────
# The apk of the osmo-build-pmos-kernel .deb, copied into the local pmbootstrap
# repository with its index and its key: "pmbootstrap install" prefers it to
# the online repository (same version, local package). Else an apk already
# built here, newer than the config, which has ppp_async.ko. Else compile.
# --rebuild forces the compilation.
_a_ppp() { grep -q 'ppp_async\.ko' < <(tar tzf "$1" 2>/dev/null); }
if [ "$REBUILD" = 0 ] && [ -f "$KD"/linux-postmarketos-stable-*.apk ]; then
    read -r _chan _parch _ver < "$KD/apk.info" 2>/dev/null || { _chan=edge; _parch=x86_64; }
    _dst="$WORK/packages/$_chan/$_parch"
    mkdir -p "$_dst" "$WORK/config_apk_keys"
    for f in "$KD"/linux-postmarketos-stable-*.apk "$KD"/APKINDEX.tar.gz; do
        [ -f "$f" ] && [ ! -f "$_dst/$(basename "$f")" ] && cp -f "$f" "$_dst/"
    done
    cp -n "$KD"/*.rsa.pub "$WORK/config_apk_keys/" 2>/dev/null || true
    touch "$_dst"/linux-postmarketos-stable-*.apk 2>/dev/null || true   # plus recent que la config (fichier du
     ↪  chroot, uid 12345 : pas grave)
    ok "noyau PPP pris dans $KD (raccourci : pas de compilation) -> $_dst"
fi
apk="$(find "$WORK/packages" -name 'linux-postmarketos-stable-[0-9]*.apk' -newer "$KCFG" 2>/dev/null | head -1)"
if [ "$REBUILD" = 0 ] && [ -n "$apk" ] && _a_ppp "$apk"; then
    ok "noyau deja construit avec PPP : $apk (--rebuild pour recompiler)"
else
    [ "$INIT_ONLY" = 1 ] && { warn "pas d apk PPP ; --init ne compile pas (relancer sans --init)"; exit 0; }
    sudo -v || die "sudo refuse"
    say "build du noyau (long : compter 15 a 40 min)..."
    pmb -y -j "$JOBS" build --force linux-postmarketos-stable \
        || die "build echoue - voir $WORK/log.txt (pmbootstrap log)"
    apk="$(find "$WORK/packages" -name 'linux-postmarketos-stable-[0-9]*.apk' -newer "$KCFG" 2>/dev/null | head
     ↪  -1)"
    [ -n "$apk" ] || die "aucun .apk du noyau dans $WORK/packages"
fi
_a_ppp "$apk" && ok "noyau : $apk contient ppp_async.ko" || die "le paquet $apk n a PAS ppp_async.ko - la config n
 ↪  a pas ete prise"
[ "$INIT_ONLY" = 1 ] && { ok "pmbootstrap pret (--init)"; exit 0; }
[ "$INSTALL" = 1 ] || { ok "fini (--no-install)"; exit 0; }

# ── 4. THE IMAGE ───────────────────────────────────────────────────────
# The image the VM uses is written: the VM must not be running on it.
# An image already provided (/opt/user_interface/pmos/image/qemu-amd64.img.zst,
# the reference VM) is decompressed instead of an install: that is shortcut
# no. 2 - OSMO_PMOS_FRESH=1 to install anyway.
if pgrep -f "qemu-system-x86_64.*$(basename "$IMG")" >/dev/null 2>&1; then
    die "la VM tourne encore sur $IMG : arrete-la (osmo-pmos-qemu stop) puis relance"
fi
sudo -v || die "sudo refuse"
# ── THE REFERENCE IMAGE: PRESENT, OR DOWNLOADED ────────────────────────
# If the .zst is missing (ISO built without it) or CORRUPT (zstd -t says so:
# a truncated copy, a 404 saved as is, a key removed too early), it is taken
# from the GitHub release "pmos-image" - the one the workflow embeds in the
# ISO. 1.7 GB, resumable (curl -C -), tested before being put in place.
# OSMO_PMOS_IMAGE_URL for another address, empty to download nothing (falls
# back to pmbootstrap install).
ZST="$PM/image/qemu-amd64.img.zst"
: "${OSMO_PMOS_IMAGE_URL=https://github.com/bbaranoff/osmo-operator/releases/download/pmos-image/qemu-
 ↪  amd64.img.zst}"
pmos_zst_ok() { [ -f "$ZST" ] && zstd -t "$ZST" >/dev/null 2>&1; }
if [ "${OSMO_PMOS_FRESH:-0}" != 1 ] && [ ! -f "$IMG" ] && command -v zstd >/dev/null 2>&1 \
   && ! pmos_zst_ok && [ -n "$OSMO_PMOS_IMAGE_URL" ] && command -v curl >/dev/null 2>&1; then
```

```
    [ -f "$ZST" ] && warn "image de reference illisible ($ZST) - retelechargee"
    say "telechargement de l image de reference (1,7 Go, reprise possible) : $OSMO_PMOS_IMAGE_URL"
    sudo install -d "$(dirname "$ZST")"
    if sudo curl -fL -C - --retry 3 --retry-delay 5 -o "$ZST.part" "$OSMO_PMOS_IMAGE_URL" \
       && zstd -t "$ZST.part" >/dev/null 2>&1; then
        sudo mv -f "$ZST.part" "$ZST" && sudo chmod a+r "$ZST"
        ok "image de reference recuperee : $ZST ($(du -h "$ZST" | cut -f1))"
    else
        sudo rm -f "$ZST.part" "$ZST"
        warn "telechargement impossible ou archive invalide - install classique (pmbootstrap install)"
    fi
fi
if [ "${OSMO_PMOS_FRESH:-0}" != 1 ] && [ ! -f "$IMG" ] && pmos_zst_ok; then
    say "image de reference decompressee ($PM/image/qemu-amd64.img.zst -> $IMG)..."
    mkdir -p "$(dirname "$IMG")"
    zstd -d --sparse -o "$IMG" "$PM/image/qemu-amd64.img.zst" && ok "image prete : $IMG (VM de reference)" \
        || { rm -f "$IMG"; warn "decompression echouee - install classique"; }
fi
if [ ! -f "$IMG" ] || [ "${OSMO_PMOS_FRESH:-0}" = 1 ]; then
    say "installation de l image (nouvelle VM, mot de passe $PASS)..."
    pmb -y install --password "$PASS" || die "install echoue - voir $WORK/log.txt"
    ok "image prete : $IMG"
fi
[ "$LAUNCH" = 1 ] || { ok "fini (--no-launch) - « osmo-pmos-qemu $FORMAT » branche VM, modem et voix"; exit 0; }

# ── 5. THE VM, AND THE MODEM AS SOON AS IT ANSWERS ──────────────────────
# osmo-pmos-qemu keeps control (SDL window) and its watcher plugs the modem
# and the voice by itself (osmo-pmos-setup) as soon as the VM SSH answers.
QEMU="$(command -v osmo-pmos-qemu 2>/dev/null || echo "$PM/bin/osmo-pmos-qemu.sh")"
export OSMO_PMOS_FROM_BUILD=1     # osmo-pmos-qemu ne doit pas nous rappeler
exec "$QEMU" "$FORMAT"
```

2.269 /opt/GSM/osmo-operator/tools/osmo-pmos-install.sh

15197 octets, 275 lignes → 275 lignes

```
#!/bin/bash
# ============================================================================
# osmo-pmos-install.sh - THE BENCH SMARTPHONE UI, INSTALLED IN /opt/user_interface
# ONCE FOR THE THREE PATHS (addition.sh, update.sh, the ISO build).
#
# The bench phone (the postmarketOS/Phosh VM, its modem, its PPP kernel) lives
# in ONE place:
#
#   /opt/user_interface/
#     pmos/                       THE SMARTPHONE UI
#       bin/                      osmo-pmos.sh osmo-pmos-qemu.sh osmo-pmos-setup.sh
#                                 osmo-pmos-build.sh osmo-phonesim-banc.py at-cmd.py
#                                 (copies from the repository; /usr/local/bin/osmo-pmos*
#                                 point there)
#       pmbootstrap/              pmbootstrap, PATCHED (patches/pmbootstrap-osmo-bench-qemu.patch)
#                                 -> /usr/local/bin/pmbootstrap
#       patches/                  the two patches (pmbootstrap, pmaports PPP)
#       pmbootstrap_v3.cfg        the config template (Phosh, fr, modem packages)
#       image/qemu-amd64.img.zst  (optional) the reference VM, compressed
#       README
#     kernel/                     THE pmOS PPP KERNEL .deb
#       osmo-build-pmos-kernel_*.deb
#       pmos/                     what the .deb installs: apk, APKINDEX, .pub key,
#                                 pmaports.commit, patch (read by osmo-pmos-build)
#
#   osmo-pmos-install               = --layout --launchers --kernel
#   osmo-pmos-install --layout      bin/, patches/, template, pmbootstrap (clone + patch if missing)
#   osmo-pmos-install --launchers   /usr/local/bin/osmo-pmos*, pmbootstrap, .desktop
```

```
#   osmo-pmos-install --kernel       dpkg -i of the kernel .deb (kernel/ or /var/cache/osmo-debs)
#   osmo-pmos-install --image <img>  stores a VM image (zstd compressed) in pmos/image/
#   osmo-pmos-install --image telecharger   takes it from the GitHub release pmos-image (also
#                                    the automatic fallback if <img> is missing or corrupt)
#
# Idempotent; usable in session AND in the ISO chroot (no systemctl, no
# network required if pmbootstrap is already there).
# ============================================================================
set -u

REPO="${OSMO_REPO:-${DIR:-/opt/GSM/osmo-operator}}"
UI="${OSMO_UI_ROOT:-/opt/user_interface}"
PM="$UI/pmos"
KDIR="$UI/kernel"
DEB_CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
PMB_GIT="${OSMO_PMB_GIT:-https://gitlab.postmarketos.org/postmarketOS/pmbootstrap.git}"
# The PPP kernel .deb is also published on GitHub (Git LFS):
# https://github.com/bbaranoff/pmos_ppp_kernel - the fallback when neither
# /opt/user_interface/kernel nor /var/cache/osmo-debs has it (a machine other
# than the reference one, an ISO build on another machine).
KERNEL_URL="${OSMO_PMOS_KERNEL_URL:-
 ↪  https://media.githubusercontent.com/media/bbaranoff/pmos_ppp_kernel/main/osmo-build-pmos-
 ↪  kernel_7.2.3.0+ppp~noble_amd64.deb}"

: "${GREEN:=}"; : "${YELLOW:=}"; : "${CYAN:=}"; : "${RED:=}"; : "${NC:=}"
_p_say()  { echo -e "  ${CYAN}→${NC} $*"; }
_p_ok()   { echo -e "      ${GREEN}✓${NC} $*"; }
_p_warn() { echo -e "      ${YELLOW}!${NC} $*"; }

pmos_layout() {
    _p_say "l UI smartphone dans $PM"
    install -d "$PM/bin" "$PM/patches" "$KDIR"
    local f
    for f in osmo-pmos.sh osmo-pmos-qemu.sh osmo-pmos-setup.sh osmo-pmos-build.sh osmo-phonesim-banc.py
     ↪  at-cmd.py; do
        [ -f "$REPO/tools/$f" ] && install -m755 "$REPO/tools/$f" "$PM/bin/$f"
    done
    for f in pmbootstrap-osmo-bench-qemu.patch pmaports-linux-postmarketos-stable-ppp.patch; do
        [ -f "$REPO/patches/$f" ] && install -m644 "$REPO/patches/$f" "$PM/patches/$f"
    done
    [ -f "$REPO/configs/pmos/pmbootstrap_v3.cfg" ] && install -m644 "$REPO/configs/pmos/pmbootstrap_v3.cfg"
     ↪  "$PM/pmbootstrap_v3.cfg"
    # pmbootstrap: the one already there (the osmo-build-pmbootstrap .deb of the
    # Dockerfile, or a previous install), else a clone. Then the bench patch,
    # if not already in (osmo_bench_args in pmb/commands/qemu.py).
    if [ ! -f "$PM/pmbootstrap/pmbootstrap.py" ]; then
        if git clone -q "$PMB_GIT" "$PM/pmbootstrap" 2>/dev/null; then
            _p_ok "pmbootstrap clone ($PMB_GIT)"
        else
            _p_warn "pmbootstrap : pas de clone possible (hors ligne ?) - osmo-pmos-build le redemandera"
        fi
    fi
    if [ -f "$PM/pmbootstrap/pmbootstrap.py" ]; then
        # "already patched" does not mean "up to date": the osmo-build-pmbootstrap
        # .deb carries the patch of one date, the repository the current one.
        # On a git clone, start again from the original files and apply THE
        # current patch - idempotent; without git, rely on the markers.
        # Marker of the most recent patch: OSMO_PMOS_HP (the named "combine"
        # card); the older one is bootindex=0.
        if [ -d "$PM/pmbootstrap/.git" ] && ! grep -q 'OSMO_PMOS_HP' "$PM/pmbootstrap/pmb/commands/qemu.py"
         ↪  2>/dev/null \
           && git -C "$PM/pmbootstrap" checkout -q -- pmb 2>/dev/null \
           && git -C "$PM/pmbootstrap" apply "$PM/patches/pmbootstrap-osmo-bench-qemu.patch" 2>/dev/null; then
            _p_ok "pmbootstrap : patch du banc remis a jour (carte combine nommee, bootindex du disque, modem
             ↪  serie PCI, son du banc)"
        elif grep -q 'osmo_bench_args' "$PM/pmbootstrap/pmb/commands/qemu.py" 2>/dev/null; then
```

```
            _p_ok "pmbootstrap deja patche (osmo_bench_args$(grep -q 'OSMO_PMOS_HP'
             ↪  "$PM/pmbootstrap/pmb/commands/qemu.py" 2>/dev/null && echo ', carte combine nommee'))"
        elif git -C "$PM/pmbootstrap" apply --check "$PM/patches/pmbootstrap-osmo-bench-qemu.patch" 2>/dev/null;
         ↪  then
            git -C "$PM/pmbootstrap" apply "$PM/patches/pmbootstrap-osmo-bench-qemu.patch" && _p_ok "pmbootstrap
             ↪  patche (modem serie PCI, son du banc, ecran, port 5038)"
        else
            _p_warn "le patch pmbootstrap ne s applique pas sur cette version - a regarder (git -C $PM/pmbootstrap
             ↪  apply --check)"
        fi
        find "$PM/pmbootstrap" -name '__pycache__' -type d -exec rm -rf {} + 2>/dev/null || true
        find "$PM/pmbootstrap" -name '*.bak-*' -delete 2>/dev/null || true
    fi
    cat > "$PM/README" <<EOF
L UI SMARTPHONE DU BANC - une VM postmarketOS/Phosh branchee sur le modem
du banc (tools/osmo-phonesim-banc.py). Pose par osmo-pmos-install
(depot : $REPO, tools/osmo-pmos-install.sh).

  bin/                 les scripts (osmo-pmos, osmo-pmos-qemu, osmo-pmos-setup,
                       osmo-pmos-build) - /usr/local/bin/* y pointent
  pmbootstrap/         pmbootstrap patche pour le banc (patches/)
  pmbootstrap_v3.cfg   gabarit : ~/.config/pmbootstrap_v3.cfg du compte de session
  image/               (optionnel) qemu-amd64.img.zst, la VM de reference
  ../kernel/           le .deb du noyau PPP, et kernel/pmos/ son contenu

Premiere fois (l icone le fait toute seule ; root accepte : --as-root est passe) :
  osmo-pmos-build            noyau (raccourci : l apk de ../kernel/pmos), image, VM
  osmo-pmos-qemu smartphone  ensuite : la VM, le modem et la voix
Mot de passe de la VM : 147147.
EOF
    # Readable and executable by the session account: pmbootstrap is run under
    # it, not under root.
    chmod -R a+rX "$PM" "$KDIR" 2>/dev/null || true
    _p_ok "$PM : bin/ ($(ls "$PM/bin" | wc -l) scripts), patches/, gabarit, pmbootstrap$( [ -f
     ↪  "$PM/pmbootstrap/pmbootstrap.py" ] && echo '' || echo ' (absent)')"
}

pmos_launchers() {
    local f n
    for f in osmo-pmos osmo-pmos-qemu osmo-pmos-setup osmo-pmos-build; do
        [ -f "$PM/bin/$f.sh" ] && ln -sfn "$PM/bin/$f.sh" "/usr/local/bin/$f"
    done
    [ -f "$PM/pmbootstrap/pmbootstrap.py" ] && ln -sfn "$PM/pmbootstrap/pmbootstrap.py" /usr/local/bin/pmbootstrap
    # Terminal=false everywhere: each Exec goes through tools/osmo-gtk-run.py
    # (bench GTK window, with a VTE terminal inside when the command talks or
    # asks questions) - no favourite opens a terminal.
    # The icons: the VM in both formats, the stop, the modem re-plug (same
    # .desktop files as update.sh / osmo-extras-install.sh, one source here).
    install -d /usr/share/applications
    cat > /usr/share/applications/osmo-pmos.desktop <<'PMD'
[Desktop Entry]
Type=Application
Name=Le telephone du banc (postmarketOS)
Comment=Une VM postmarketOS/Phosh branchee sur le modem du banc - osmo-pmos up
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --root --title "Telephone (postmarketOS)" --
 ↪  /usr/local/bin/osmo-pmos up
Icon=phone
Terminal=false
Categories=Network;Telephony;
Keywords=postmarketos;pmos;qemu;modem;gsm;telephone;
Actions=status;stop;build;
[Desktop Action status]
Name=Etat du telephone (osmo-pmos status)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --keep-open --title "Etat du telephone" --
 ↪  /usr/local/bin/osmo-pmos status
[Desktop Action stop]
```

```
Name=Arreter le telephone (osmo-pmos stop)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --close-on-exit --title "Arret du telephone" --
 ↪  /usr/local/bin/osmo-pmos stop
[Desktop Action build]
Name=Construire l image (noyau PPP + pmbootstrap install)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Image postmarketOS (build)" --
 ↪  /usr/local/bin/osmo-pmos-build --no-launch
PMD
    for n in smartphone tablette; do
        local r=720x1440 ic=phone; [ "$n" = tablette ] && { r=1280x800; ic=tablet; }
        cat > "/usr/share/applications/osmo-pmos-$n.desktop" <<PMD
[Desktop Entry]
Type=Application
Name=Telephone du banc - $n
Comment=La VM postmarketOS au format $n ($r), le modem et la voix branches tout seuls
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Telephone du banc (VM)" -- env OSMO_PMOS_RES=$r
 ↪  /usr/local/bin/osmo-pmos-qemu
Icon=$ic
Terminal=false
Categories=Network;Telephony;
Keywords=postmarketos;pmos;qemu;modem;gsm;telephone;$n;
Actions=stop;setup;nomodem;
[Desktop Action stop]
Name=Arreter le telephone (osmo-pmos-qemu stop)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --close-on-exit --title "Arret du telephone" --
 ↪  /usr/local/bin/osmo-pmos-qemu stop
[Desktop Action setup]
Name=Rebrancher le modem et la voix (osmo-pmos-setup)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Modem et voix du telephone" --
 ↪  /usr/local/bin/osmo-pmos-setup
[Desktop Action nomodem]
Name=VM seule, sans modem
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --tty --title "Telephone du banc (VM)" -- env OSMO_PMOS_RES=$r
 ↪  OSMO_PMOS_MODEM=0 /usr/local/bin/osmo-pmos-qemu
PMD
    done
    chmod 644 /usr/share/applications/osmo-pmos*.desktop
    update-desktop-database /usr/share/applications >/dev/null 2>&1 || true
    _p_ok "lanceurs : /usr/local/bin/osmo-pmos, osmo-pmos-qemu, osmo-pmos-setup, osmo-pmos-build, pmbootstrap ; 3
     ↪  .desktop"
}

pmos_kernel() {
    local deb
    if [ -f "$KDIR"/pmos/linux-postmarketos-stable-*.apk ]; then
        _p_ok "noyau PPP deja pose : $KDIR/pmos/$(basename "$(ls "$KDIR"/pmos/linux-postmarketos-stable-*.apk |
         ↪  head -1)")"
        return 0
    fi
    deb="$(ls -1 "$KDIR"/osmo-build-pmos-kernel_*.deb "$DEB_CACHE"/osmo-build-pmos-kernel_*.deb 2>/dev/null |
     ↪  head -1)"
    if [ -z "$deb" ] && [ -n "$KERNEL_URL" ] && command -v curl >/dev/null 2>&1; then
        install -d "$KDIR"
        deb="$KDIR/$(basename "$KERNEL_URL")"
        _p_say "telechargement du noyau PPP ($KERNEL_URL)..."
        if curl -fsSL --retry 3 -o "$deb.part" "$KERNEL_URL" && dpkg-deb --info "$deb.part" >/dev/null 2>&1; then
            mv -f "$deb.part" "$deb"; cp -f "$deb" "$DEB_CACHE/" 2>/dev/null || true
        else
            rm -f "$deb.part"; deb=""
        fi
    fi
    if [ -z "$deb" ]; then
        _p_warn "pas de .deb du noyau pmOS ($KDIR, $DEB_CACHE, GitHub) : osmo-pmos-build compilera le noyau (long)"
        return 0
    fi
```

```
    dpkg -i --force-overwrite "$deb" >/dev/null 2>&1 && _p_ok "noyau PPP pose depuis $(basename "$deb")" ||
     ↪  _p_warn "dpkg -i $(basename "$deb") a echoue"
    [ -f "$KDIR/$(basename "$deb")" ] || cp -f "$deb" "$KDIR/"
}

# THE IMAGE MAY COME FROM THE NETWORK. --image without a file behind it
# (missing), or a corrupt .zst (zstd -t), or --image telecharger: take the
# GitHub release "pmos-image", the same one the ISO embeds. Resumable, archive
# tested before being put in place.
PMOS_IMAGE_URL="${OSMO_PMOS_IMAGE_URL-https://github.com/bbaranoff/osmo-operator/releases/download/pmos-
 ↪  image/qemu-amd64.img.zst}"
pmos_image_telecharger() {
    local dst="$1"
    [ -n "$PMOS_IMAGE_URL" ] || { _p_warn "pas d URL d image (OSMO_PMOS_IMAGE_URL vide)"; return 1; }
    command -v curl >/dev/null 2>&1 || { _p_warn "curl absent - image non telechargee"; return 1; }
    command -v zstd >/dev/null 2>&1 || { _p_warn "zstd absent - image non verifiee, non telechargee"; return 1; }
    install -d "$(dirname "$dst")"
    _p_say "telechargement de l image de reference (1,7 Go, reprise possible) : $PMOS_IMAGE_URL"
    if curl -fL -C - --retry 3 --retry-delay 5 -o "$dst.part" "$PMOS_IMAGE_URL" \
       && zstd -t "$dst.part" >/dev/null 2>&1; then
        mv -f "$dst.part" "$dst"
        return 0
    fi
    rm -f "$dst.part"
    _p_warn "telechargement impossible ou archive invalide ($PMOS_IMAGE_URL)"
    return 1
}

pmos_image() {
    local src="$1" dst="$PM/image/qemu-amd64.img.zst"
    install -d "$PM/image"
    if [ "$src" = telecharger ] || [ ! -f "$src" ]; then
        [ "$src" = telecharger ] || _p_warn "image introuvable : $src - telechargement a la place"
        pmos_image_telecharger "$dst" || return 1
        chmod a+r "$dst"
        _p_ok "image de reference : $dst ($(du -h "$dst" | cut -f1))"
        return 0
    fi
    case "$src" in
        *.zst) if command -v zstd >/dev/null 2>&1 && ! zstd -t "$src" >/dev/null 2>&1; then
                   _p_warn "image corrompue : $src (zstd -t) - telechargement a la place"
                   pmos_image_telecharger "$dst" || return 1
               elif [ "$(readlink -f "$src")" != "$(readlink -f "$dst")" ]; then
                   cp -f "$src" "$dst"
               fi ;;
        *) command -v zstd >/dev/null 2>&1 || { _p_warn "zstd absent"; return 1; }
           _p_say "compression de $src (fichier creux : seuls les blocs ecrits comptent)..."
           zstd -T0 -3 --sparse -f -o "$dst" "$src" || return 1 ;;
    esac
    chmod a+r "$dst"
    _p_ok "image de reference : $dst ($(du -h "$dst" | cut -f1))"
}

osmo_pmos_install() {
    local layout=0 launchers=0 kernel=0 image="" a prev=""
    for a in "$@"; do
        if [ "$prev" = "--image" ]; then image="$a"; prev=""; continue; fi
        case "$a" in
            --layout) layout=1 ;; --launchers) launchers=1 ;; --kernel) kernel=1 ;; --image) prev="--image" ;;
            *) echo "osmo-pmos-install : option inconnue $a" >&2; return 2 ;;
        esac
    done
    if [ $((layout + launchers + kernel)) -eq 0 ] && [ -z "$image" ]; then layout=1; launchers=1; kernel=1; fi
    [ "$layout" = 1 ] && pmos_layout
    [ "$kernel" = 1 ] && pmos_kernel
    [ "$launchers" = 1 ] && pmos_launchers
```

```
    [ -n "$image" ] && pmos_image "$image"
    return 0
}

if [ "${BASH_SOURCE[0]}" = "$0" ]; then
    [ "$(id -u)" -eq 0 ] || { echo "osmo-pmos-install : root requis" >&2; exit 1; }
    osmo_pmos_install "$@"
fi
```

2.270 /opt/GSM/osmo-operator/tools/osmo-pmos-qemu.sh

21592 octets, 387 lignes → 387 lignes

```
#!/bin/bash
# osmo-pmos-qemu - postmarketOS in QEMU, with the bench modem and its voice.
#
# The QEMU command line is NOT rebuilt here: "pmbootstrap qemu" is called,
# with virtio-vga-gl and "gl=on" (a virtio-vga forced into software rendering
# covers the screen with colour blocks).
#
# What pmbootstrap does for the bench lives INSIDE it, in
# pmb/commands/qemu.py, function osmo_bench_args():
#   - the bench modem on a PCI serial port (neither ISA nor USB: the UEFI
#     firmware takes the answers of an ISA port for keystrokes and never
#     boots, and this kernel has no ftdi_sio for USB). 8250_pci is built in:
#     the port appears on its own as /dev/ttyS1, which ModemManager probes;
#   - the sound card wired to the bench voice bridge rather than to the
#     default devices: gsm_audio carries the downlink (the osmo-mgw RTP), so
#     its monitor feeds the VM CAPTURE; gsm_mic carries the uplink, so it
#     receives what the VM PLAYS.
#
# QEMU only LISTENS on the modem port: osmo-phonesim-banc.py connects to it,
# and only once the firmware has passed - otherwise it does not boot. That
# is the role of osmo-pmos-setup, to rerun after each bench restart:
# start-native.sh kills the VM modem on the way.
#
# The display stays SDL, the one pmbootstrap sets up: virtio-vga-gl and
# "sdl,gl=on". OSMO_PMOS_DISPLAY=gtk switches to GTK, =none for a VM without
# a window.
#
# Settings: OSMO_PMOS_AT_PORT (12346), OSMO_PMOS_SINK (gsm_mic),
# OSMO_PMOS_SOURCE (gsm_audio.monitor), OSMO_PMOS_MEM (4096),
# OSMO_PMOS_DISPLAY (sdl), OSMO_PMOS_DISK (16G), OSMO_PMOS_MODEM (1: the
# modem and the voice are connected on their own as soon as the VM answers;
# 0: bare VM).
# THE PASSWORD, ASKED FIRST. pmbootstrap needs root: it detaches the loops
# ("sudo losetup --json --list"), mounts the chroot, installs OVMF in it.
# Launched from the icon, nobody is there to answer sudo. So the password is
# asked first, with a real terminal (hence "Terminal=true" in
# osmo-pmos.desktop), and the window is kept open on failure - otherwise the
# error vanishes with it.
# THE FORMAT, CHOSEN AT LAUNCH. Phosh lays itself out according to the
# screen: tall and narrow is a smartphone; wide, a tablet. The QEMU virtio
# card carries this size (xres/yres, set by the pmbootstrap patch from
# OSMO_PMOS_RES), and the system boots in it - the format does not change
# under an open session. The format is the first argument (the desktop icons
# go through the variable):
#   osmo-pmos-qemu smartphone      720x1440   (the default)
#   osmo-pmos-qemu tablette        1280x800
#   osmo-pmos-qemu 1024x768        any <width>x<height> size
# STOP, AND ONE VM AT A TIME. QEMU refuses an image already open and the
# ports 2222 / 12346 already in use, and its message goes to the console,
# not the log. So:
#   osmo-pmos-qemu stop     clean shutdown (poweroff over SSH, 30 s),
```

```
#                           otherwise SIGTERM then SIGKILL on QEMU;
#                           disconnects the bench modem
#                           (osmo-phonesim-banc.py --connect), which
#                           osmo-pmos-setup reconnects at the next start.
#   osmo-pmos-qemu status   says whether the VM runs (pid) and whether its
#                           SSH answers.
# And at launch, if a VM already runs, say so and stop there.
PMOS_SSH_PORT="${OSMO_PMOS_SSH_PORT:-2222}"
PMOS_SSH="sshpass -p ${OSMO_PMOS_PASS:-147147} ssh -p $PMOS_SSH_PORT -T -o StrictHostKeyChecking=no -o
 ↪  UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o ConnectTimeout=4 -o PreferredAuthentications=password
 ↪  ${OSMO_PMOS_USER:-user}@127.0.0.1"
# lib/audio.sh: the assignment of the speakers and microphone (a single
# source of truth, see "L AFFECTATION DES HAUT-PARLEURS" there).
# Here: the echo canceller is loaded BEFORE QEMU, so that the VM "combine"
# card is born directly on it (OSMO_PMOS_HP / OSMO_PMOS_MIC, read by the
# pmbootstrap patch); the host direct loopbacks are removed when the VM
# starts (if the voice goes through it) and RESTORED when it stops.
AUDIO_LIB="${OSMO_REPO:-/opt/GSM/osmo-operator}/lib/audio.sh"
if [ -f "$AUDIO_LIB" ]; then
    # shellcheck source=../lib/audio.sh
    . "$AUDIO_LIB"
else
    AUDIO_LIB=""
fi
# The real QEMU (not the "sh -c" wrapping it): the one holding the image.
pmos_qemu_pids() {
    pgrep -f '^[^ ]*ld-musl[^ ]* .*qemu-system-x86_64 .*rootfs/qemu-amd64\.img' 2>/dev/null
    pgrep -f '^[^ ]*qemu-system-x86_64 .*rootfs/qemu-amd64\.img' 2>/dev/null
}
pmos_ssh_ok() { $PMOS_SSH 'echo ok' 2>/dev/null | grep -q ok; }
pmos_status() {
    local pids; pids="$(pmos_qemu_pids | sort -u | tr '\n' ' ')"
    if [ -z "$pids" ]; then echo "osmo-pmos-qemu: aucune VM postmarketOS ne tourne"; return 1; fi
    echo "osmo-pmos-qemu: VM postmarketOS en marche (QEMU pid $pids)"
    pmos_ssh_ok && echo "osmo-pmos-qemu: SSH repond sur le port $PMOS_SSH_PORT" \
                || echo "osmo-pmos-qemu: SSH ne repond pas encore (port $PMOS_SSH_PORT)"
    pgrep -f '[o]smo-phonesim-banc.py --connect' >/dev/null && echo "osmo-pmos-qemu: modem du banc branche" \
                                                            || echo "osmo-pmos-qemu: modem du banc NON branche"
    return 0
}
pmos_stop() {
    local pids i
    pids="$(pmos_qemu_pids | sort -u | tr '\n' ' ')"
    if [ -z "$pids" ]; then
        echo "osmo-pmos-qemu: aucune VM postmarketOS a arreter"
    else
        if pmos_ssh_ok; then
            echo "osmo-pmos-qemu: extinction propre par SSH (poweroff)..."
            $PMOS_SSH "echo ${OSMO_PMOS_PASS:-147147} | sudo -S -p '' poweroff" >/dev/null 2>&1
            for i in $(seq 1 30); do
                sleep 1
                [ -z "$(pmos_qemu_pids)" ] && break
            done
        fi
        if [ -n "$(pmos_qemu_pids)" ]; then
            echo "osmo-pmos-qemu: QEMU encore la, SIGTERM (pid $pids)"
            # shellcheck disable=SC2086
            kill -TERM $pids 2>/dev/null
            for i in $(seq 1 5); do sleep 1; [ -z "$(pmos_qemu_pids)" ] && break; done
        fi
        if [ -n "$(pmos_qemu_pids)" ]; then
            echo "osmo-pmos-qemu: QEMU ne meurt pas, SIGKILL"
            # shellcheck disable=SC2086
            kill -KILL $(pmos_qemu_pids) 2>/dev/null
            sleep 1
        fi
```

```
        [ -z "$(pmos_qemu_pids)" ] && echo "osmo-pmos-qemu: VM arretee" \
                                   || { echo "osmo-pmos-qemu: la VM tourne toujours (droits ? relancer en root)";
                                    ↪  return 1; }
    fi
    # The bench modem was connected to the serial port of THIS VM:
    # disconnect it, osmo-pmos-setup reconnects it at the next start.
    # The bench runs as root since osmo-pmos-setup (the PPP tun requires
    # it): under the session account pkill cannot, sudo -n can if the
    # password is still cached.
    if pkill -f '[o]smo-phonesim-banc.py --connect' 2>/dev/null \
       || sudo -n pkill -f '[o]smo-phonesim-banc.py --connect' 2>/dev/null; then
        echo "osmo-pmos-qemu: modem du banc debranche"
    fi
    # An orphan "pmbootstrap qemu" (VM killed under it) is useless.
    # Pattern anchored on the interpreter: a broad "pkill -f" would also
    # kill the terminal of anyone with these words in their command line.
    pkill -f '^[^ ]*python[0-9.]* [^ ]*pmbootstrap qemu' 2>/dev/null || true
    # VM gone: the host resumes its direct listening (gsm_audio -> speakers).
    [ -n "$AUDIO_LIB" ] && { ensure_local_loopback; ensure_local_mic; }
    return 0
}
# From an icon (Terminal=true), the window would close before one can read:
# keep it a few seconds when on a real terminal.
pmos_fin() { local rc=$1; [ -t 0 ] && read -r -t 8 -p "Entree pour fermer (8 s) " _; exit "$rc"; }
# TOGGLE: a single action on the icon. VM running -> power it off
# (pmos_stop); VM stopped -> launch it, exactly as without argument. This is
# what the icon does (osmo-pmos toggle -> here).
case "${1:-}" in
    stop|arret|arreter|off|down) pmos_stop; pmos_fin $? ;;
    status|etat)                 pmos_status; pmos_fin $? ;;
    toggle|bascule)
        if [ -n "$(pmos_qemu_pids)" ]; then
            echo "osmo-pmos-qemu: le telephone tourne - on l eteint"
            pmos_stop; pmos_fin $?
        fi
        echo "osmo-pmos-qemu: le telephone est arrete - on le lance"
        shift ;;
esac

case "${1:-}" in
    smartphone|telephone|phone) export OSMO_PMOS_RES=720x1440; shift ;;
    tablette|tablet)            export OSMO_PMOS_RES=1280x800; shift ;;
    [0-9]*x[0-9]*)              export OSMO_PMOS_RES="$1"; shift ;;
esac
export OSMO_PMOS_RES="${OSMO_PMOS_RES:-720x1440}"
echo "osmo-pmos-qemu: format ${OSMO_PMOS_RES} ($([ "$OSMO_PMOS_RES" = 1280x800 ] && echo tablette || echo
 ↪  smartphone))"
if [ -n "$(pmos_qemu_pids)" ]; then
    pmos_status
    echo "osmo-pmos-qemu: une seule VM a la fois - « osmo-pmos-qemu stop » pour l arreter, puis relancer"
    read -r -p "Entree pour fermer " _
    exit 1
fi

PMB="$(command -v pmbootstrap || echo "$HOME/.local/bin/pmbootstrap")"
if ! sudo -v; then
    echo "osmo-pmos-qemu: sudo refuse - pmbootstrap ne peut pas preparer l image" >&2
    read -r -p "Entree pour fermer " _
    exit 1
fi
# AS ROOT TOO. The ISO session opens as root (gdm3,
# iso_modules/84-comptes.sh), and pmbootstrap refuses to run as root unless
# --as-root is passed: it is passed when root, and the work directory is
# then root's.
PMB_OPTS=(); [ "$(id -u)" -eq 0 ] && PMB_OPTS=(--as-root)
# THE PHONE DATA REALLY GETS OUT, AT EVERY LAUNCH. docker sets the FORWARD
```

```
# policy to DROP on the host, which would silently drop the UE packets
# (ogstun, after NAT). osmo-epc.sh (setup_forward) sets the rule when the 4G
# core starts; it is set again here, before the VM, in case the core started
# before this script or docker recreated its chains in between. DOCKER-USER:
# consulted before the docker rules, never rewritten by it.
pmos_forward() {
    local su=(); [ "$(id -u)" -eq 0 ] || su=(sudo -n)
    "${su[@]}" iptables -N DOCKER-USER 2>/dev/null || true
    local spec
    for spec in "-i ogstun" "-o ogstun"; do
        # shellcheck disable=SC2086
        "${su[@]}" iptables -C DOCKER-USER $spec -j ACCEPT 2>/dev/null \
            || "${su[@]}" iptables -I DOCKER-USER 1 $spec -j ACCEPT 2>/dev/null \
            || { echo "osmo-pmos-qemu: FORWARD ($spec) refuse - la data du telephone s arretera a l hote" >&2;
             ↪  return 0; }
    done
    "${su[@]}" iptables -C FORWARD -j DOCKER-USER 2>/dev/null \
        || "${su[@]}" iptables -I FORWARD 1 -j DOCKER-USER 2>/dev/null || true
    echo "osmo-pmos-qemu: FORWARD ogstun autorise (DOCKER-USER) - la data du telephone sort vers Internet"
}
pmos_forward
# THE WORK DIRECTORY IS THE CONFIG'S: on the ISO it is
# ~/.local/var/pmbootstrap (template installed by 88-lte-pmos.sh in /root and
# /home/osmocom, instantiated by osmo-pmos-build). ~/test is still accepted.
PMB_WORK="${OSMO_PMB_WORK:-$(sed -n 's/^work *= *//p' "$HOME/.config/pmbootstrap_v3.cfg" 2>/dev/null | head -1)}"
[ -n "$PMB_WORK" ] || PMB_WORK="$HOME/.local/var/pmbootstrap"
[ -d "$PMB_WORK/chroot_native" ] || [ ! -d "$HOME/test/chroot_native" ] || PMB_WORK="$HOME/test"
IMG_PMB="$(ls -1 "$PMB_WORK"/chroot_native/home/pmos/rootfs/*.img 2>/dev/null | head -1)"
# NO IMAGE: BUILD IT, HERE. On the first click on the ISO icon there is no
# config, no work directory, and the reference image is still a .zst in
# /opt/user_interface/pmos/image. osmo-pmos-build does all of that (config
# from the template, pmaports, PPP kernel, decompressed image) then COMES
# BACK here (exec osmo-pmos-qemu, OSMO_PMOS_FROM_BUILD=1 to avoid looping).
if [ -z "$IMG_PMB" ] && [ "${OSMO_PMOS_FROM_BUILD:-0}" != 1 ]; then
    BUILD="$(command -v osmo-pmos-build 2>/dev/null || echo /opt/user_interface/pmos/bin/osmo-pmos-build.sh)"
    if [ -x "$BUILD" ]; then
        echo "osmo-pmos-qemu: pas d image postmarketOS dans $PMB_WORK - premiere fois : osmo-pmos-build (long,
         ↪  reseau)"
        exec "$BUILD" "$([ "$OSMO_PMOS_RES" = 1280x800 ] && echo tablette || echo smartphone)"
    fi
    echo "osmo-pmos-qemu: pas d image postmarketOS dans $PMB_WORK, et pas d osmo-pmos-build : « pmbootstrap
     ↪  install » a la main" >&2
fi

# THE MODEM, BY DEFAULT. The phone without a modem is only a VM: no call,
# no SMS, no data. A watcher waits for the VM SSH to answer (so that the
# firmware has passed, see the header: connecting earlier blocks the boot)
# and runs osmo-pmos-setup - the bench modem on the serial port, then the
# voice. Its full trace goes to a file; only the useful lines come back
# here, prefixed, in the middle of the VM serial console.
# OSMO_PMOS_MODEM=0 for a bare VM.
SETUP="${OSMO_PMOS_SETUP:-/usr/local/bin/osmo-pmos-setup}"
[ -x "$SETUP" ] || SETUP="${OSMO_REPO:-/opt/GSM/osmo-operator}/tools/osmo-pmos-setup.sh"
SETUP_LOG="${XDG_RUNTIME_DIR:-/tmp}/osmo-pmos-setup.log"
# DISK SPACE, CHECKED AT EVERY LAUNCH. The watcher always runs (modem or
# not): once the VM is up it checks that the root fills the whole disk.
# Normally the pmOS initramfs already did it at boot
# (pmos.force-partition-resize on the kernel line); otherwise - image
# rebuilt without that parameter - it extends partition 2 then the ext4
# itself, online, over SSH. Then the modem if OSMO_PMOS_MODEM=1.
pmos_place() {
    # The VM-side script travels in base64: three levels of quoting otherwise.
    local b64; b64="$(base64 -w0 <<'PLACE'
set -e
d=$(blockdev --getsize64 /dev/vda); p=$(blockdev --getsize64 /dev/vda2)
s=$(sfdisk -d /dev/vda 2>/dev/null | sed -n 's/^.*vda2 : start= *\([0-9]*\).*/\1/p')
```

```
# room behind partition 2 (> 64 MiB)? extend it, online
if [ $((d - s*512 - p)) -gt 67108864 ]; then
    echo "partition 2 : $((p/1048576)) Mio sur un disque de $((d/1048576)) Mio - extension"
    parted -f -s /dev/vda resizepart 2 100% >/dev/null 2>&1 \
        || echo ", +" | sfdisk --no-reread -N2 /dev/vda >/dev/null 2>&1 || true
    partprobe /dev/vda 2>/dev/null || true
fi
resize2fs /dev/vda2 2>&1 | grep -v '^resize2fs [0-9]' || true
df -h / | tail -1 | awk '{print "place : racine " $2 ", libre " $4 " (" $5 " pris)"}'
PLACE
)"
    $PMOS_SSH "echo ${OSMO_PMOS_PASS:-147147} | sudo -S -p '' sh -c \"\$(echo $b64 | base64 -d)\"" 2>/dev/null
}
GUETTEUR=""
(
    for _ in $(seq 1 120); do          # 10 min: the image may need rebuilding
        sleep 5
        pmos_ssh_ok || continue
        echo "osmo-pmos-qemu: VM levee - verification de la place sur le disque"
        pmos_place | sed 's/^/osmo-pmos-qemu: /'
        if [ "${OSMO_PMOS_MODEM:-1}" = "1" ] && [ -x "$SETUP" ]; then
            echo "osmo-pmos-qemu: branchement du modem et de la voix ($SETUP)"
            "$SETUP" >"$SETUP_LOG" 2>&1
            grep -E 'modem|ATTENTION|descendant|montant|Modem|DNS' "$SETUP_LOG" \
                | sed 's/^/osmo-pmos-qemu: /'
            echo "osmo-pmos-qemu: trace complete dans $SETUP_LOG"
        elif [ -x "$SETUP" ]; then
            # WITHOUT A MODEM, THE SOUND ANYWAY. A bare VM would output its
            # audio on the BRIDGE card (0x12 -> gsm_mic): mute for the
            # operator. --voix only does the sound part (handset by default,
            # loopbacks, mutes).
            echo "osmo-pmos-qemu: VM sans modem - le son seul ($SETUP --voix)"
            "$SETUP" --voix >"$SETUP_LOG" 2>&1
            grep -E 'ATTENTION|descendant|montant|sourdines' "$SETUP_LOG" | sed 's/^/osmo-pmos-qemu: /'
        fi
        exit 0
    done
    echo "osmo-pmos-qemu: la VM n a pas repondu en SSH en 10 min - place non verifiee, modem NON branche" \
         "(relancer osmo-pmos-setup a la main)"
) &
GUETTEUR=$!
if [ "${OSMO_PMOS_MODEM:-1}" = "1" ] && [ -x "$SETUP" ]; then
    echo "osmo-pmos-qemu: le modem du banc sera branche des que la VM repond (OSMO_PMOS_MODEM=0 pour l eviter)"
else
    echo "osmo-pmos-qemu: VM sans modem (OSMO_PMOS_MODEM=0 ou osmo-pmos-setup absent)"
fi
# VM gone: the watcher has nothing left to wait for.
trap '[ -n "$GUETTEUR" ] && kill "$GUETTEUR" 2>/dev/null' EXIT

# THE DISK: 16 GB, not 2. The pmbootstrap image is 2 GB by default and fills
# up after apk add/upgrade. --image-size is passed AT EVERY LAUNCH:
# pmbootstrap only does a truncate (sparse file, costs only what is written;
# it only refuses to SHRINK, hence the computation below which keeps the
# current size if larger), the pmOS initramfs extends the partition and the
# ext4 at boot, and the watcher above checks the result. OSMO_PMOS_DISK for
# another size.
DISK="${OSMO_PMOS_DISK:-16G}"
# IMG_PMB: resolved above, from the config's work directory.
if [ -n "$IMG_PMB" ]; then
    cur_m=$(( ($(stat -c %s "$IMG_PMB") + 1048575) / 1048576 ))
    case "$DISK" in *G) want_m=$(( ${DISK%G} * 1024 )) ;; *M) want_m=${DISK%M} ;; *) want_m=$cur_m ;; esac
    [ "$want_m" -lt "$cur_m" ] && DISK="${cur_m}M"
    echo "osmo-pmos-qemu: disque de la VM $DISK ($IMG_PMB, ${cur_m} Mio avant)"
fi
# ROBUST TO RELUCTANT HOSTS (AMD first). The pmbootstrap patch already
# disables OpenGL on an AMD GPU and KVM without SVM; a VM that DIES AT ONCE
```

```
# (less than 25 s, non-zero code) is relaunched with fallbacks (see below).
# Each attempt is announced; the last error code is the one kept. A VM that
# ran more than 25 s then stopped is not relaunched: that is a shutdown.
# THE VM SPEAKERS, NAMED BEFORE IT STARTS. The "combine" card
# (osmo_bench_args) takes the host default devices when QEMU connects. The
# echo canceller is loaded HERE, the direct loopbacks are handled (removed
# if the voice goes through the VM), and the names are given to the
# pmbootstrap patch: the card is born in the right place.
if [ -n "$AUDIO_LIB" ] && [ "${OSMO_PMOS_RELAI:-1}" = "1" ]; then
    ensure_echo_cancel 2>/dev/null | sed 's/^ *//; s/^/osmo-pmos-qemu: /'
    # THE HOST LISTENING IS KEPT AT LAUNCH. The voice does not go through
    # the VM by default (see lib/audio.sh, "QUI PORTE LA VOIX"): the direct
    # loopbacks ARE the path, and removing them here would mute the bench
    # during the whole phone boot.
    # OSMO_PMOS_VOIX_VM=1: osmo-pmos-setup then writes the marker and removes
    # these loopbacks, once the VM is ready to replace them.
    if [ "${OSMO_PMOS_VOIX_VM:-0}" = "1" ]; then
        remove_direct_loopbacks 2>/dev/null | sed 's/^ *//; s/^/osmo-pmos-qemu: /'
    else
        { ensure_local_loopback; ensure_local_mic; } 2>/dev/null | sed 's/^ *//; s/^/osmo-pmos-qemu: /'
    fi
    _hp="$(audio_hp_sink)"; _mic="$(audio_mic_source)"
    [ -n "$_hp" ]  && export OSMO_PMOS_HP="${OSMO_PMOS_HP:-$_hp}"
    [ -n "$_mic" ] && export OSMO_PMOS_MIC="${OSMO_PMOS_MIC:-$_mic}"
    echo "osmo-pmos-qemu: carte « combine » de la VM -> HP ${OSMO_PMOS_HP:-defaut}, micro ${OSMO_PMOS_MIC:-defaut}"
fi
# THE HOST QEMU, NOT THE CHROOT'S. The pmbootstrap QEMU runs in a musl
# chroot without the machine's NVIDIA drivers: even with virtio-vga-gl and
# gl=on, its EGL fails and the rendering falls back to software - broken
# display. The host's (Ubuntu, glibc) loads libEGL_nvidia. It needs
# qemu-system-gui and qemu-system-modules-opengl (ui-sdl, ui-opengl,
# virtio-vga-gl). OSMO_PMOS_HOST_QEMU=0 goes back to the chroot QEMU.
QEMU_MODDIR=/usr/lib/x86_64-linux-gnu/qemu
if [ -z "${OSMO_PMOS_HOST_QEMU:-}" ]; then
    if [ -e "$QEMU_MODDIR/ui-opengl.so" ] && [ -e "$QEMU_MODDIR/ui-sdl.so" ] && [ -e
     ↪  "$QEMU_MODDIR/hw-display-virtio-vga-gl.so" ]; then
        OSMO_PMOS_HOST_QEMU=1
    else
        OSMO_PMOS_HOST_QEMU=0
        echo "osmo-pmos-qemu: QEMU de l hote sans modules OpenGL (apt install qemu-system-gui
         ↪  qemu-system-modules-opengl) - QEMU du chroot"
    fi
fi
HOST_QEMU_OPT=(); [ "$OSMO_PMOS_HOST_QEMU" = 1 ] && HOST_QEMU_OPT=(--host-qemu)
_lance() {
    "$PMB" "${PMB_OPTS[@]}" qemu \
        "${HOST_QEMU_OPT[@]}" \
        --memory "${OSMO_PMOS_MEM:-4096}" \
        --image-size "$DISK" \
        --display "${OSMO_PMOS_DISPLAY:-sdl}" \
        "$@"
}
_mort_tot() { [ "$1" -ne 0 ] && [ $(( $(date +%s) - t0 )) -lt 25 ]; }
# OPENGL FIRST, ALL THE WAY. Software rendering (virtio-vga without GL) is
# exactly the colour-block display, so the fallbacks try the chroot QEMU,
# then GTK, then without KVM, ALWAYS with OpenGL; software rendering only
# comes last, and says so. OSMO_PMOS_GL=1 is set by default (NVIDIA host:
# pmbootstrap would otherwise disable it in some cases).
export OSMO_PMOS_GL="${OSMO_PMOS_GL:-1}"
t0=$(date +%s); _lance "$@"; rc=$?
if _mort_tot "$rc" && [ "$OSMO_PMOS_HOST_QEMU" = 1 ]; then
    echo "osmo-pmos-qemu: le QEMU de l hote est mort aussitot ($rc) - relance avec celui du chroot
     ↪  (OSMO_PMOS_HOST_QEMU=0)"
    OSMO_PMOS_HOST_QEMU=0; HOST_QEMU_OPT=(); t0=$(date +%s); _lance "$@"; rc=$?
fi
if _mort_tot "$rc" && [ "${OSMO_PMOS_DISPLAY:-sdl}" = sdl ]; then
```

```
    echo "osmo-pmos-qemu: QEMU est mort aussitot ($rc) - relance en fenetre GTK, OpenGL garde
     ↪  (OSMO_PMOS_DISPLAY=gtk)"
    export OSMO_PMOS_DISPLAY=gtk; t0=$(date +%s); _lance "$@"; rc=$?
fi
if _mort_tot "$rc" && [ "${OSMO_PMOS_KVM:-}" != 0 ]; then
    echo "osmo-pmos-qemu: encore mort ($rc) - relance SANS KVM, OpenGL garde (OSMO_PMOS_KVM=0, lent)"
    export OSMO_PMOS_KVM=0; t0=$(date +%s); _lance "$@"; rc=$?
fi
if _mort_tot "$rc" && [ "$OSMO_PMOS_GL" != 0 ]; then
    echo "osmo-pmos-qemu: encore mort ($rc) - DERNIER RECOURS : sans OpenGL (OSMO_PMOS_GL=0) - l affichage sera en
     ↪  paves"
    export OSMO_PMOS_GL=0; t0=$(date +%s); _lance "$@"; rc=$?
fi
# The VM stopped (window closed, poweroff from Phosh, failure): the host
# direct loopback comes back - without it, nobody listens to gsm_audio.
[ -n "$AUDIO_LIB" ] && { ensure_local_loopback; ensure_local_mic; } 2>/dev/null | sed 's/^ *//;
 ↪  s/^/osmo-pmos-qemu: /'
[ "$rc" -eq 0 ] || read -r -p "pmbootstrap a echoue ($rc). Entree pour fermer " _
exit "$rc"
```

2.271 /opt/GSM/osmo-operator/tools/osmo-pmos-setup.sh

24068 octets, 428 lignes → 428 lignes

```
#!/bin/bash
# osmo-pmos-setup - prepares the postmarketOS VM for the bench modem.
#
# Run from the host, with the VM started (SSH on 2222). It applies,
# PERSISTENTLY:
#   - the DNS: postmarketOS goes through systemd-resolved, which has no server
#     under QEMU (hence "no network" while the IP and route are fine);
#   - the udev rule that tells ModemManager to probe the modem port (a PCI
#     serial port), otherwise it ignores it;
#   - the wired link hidden from the phone, so that data goes through the
#     mobile connection (PPP of the bench);
#   - the voice wiring (--voix): host audio and the two sound cards of the VM.
set -u
PORT="${OSMO_PMOS_SSH_PORT:-2222}"
PASS="${OSMO_PMOS_PASS:-147147}"
USER_VM="${OSMO_PMOS_USER:-user}"
ssh_vm() { sshpass -p "$PASS" ssh -p "$PORT" -T -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
 ↪  LogLevel=ERROR \
    -o ConnectTimeout=8 -o PreferredAuthentications=password "$USER_VM@127.0.0.1" "$@"; }
# lib/audio.sh: the assignment of the host speakers and microphone (echo
# canceller, direct loopbacks) - a single source of truth, shared with
# audio-chain.sh (boot), start-native and osmo-pmos-qemu.
AUDIO_LIB="${OSMO_REPO:-/opt/GSM/osmo-operator}/lib/audio.sh"
if [ -f "$AUDIO_LIB" ]; then
    # shellcheck source=../lib/audio.sh
    . "$AUDIO_LIB"
else
    AUDIO_LIB=""
fi
# THE VOICE AS A FUNCTION, callable alone (osmo-pmos-setup --voix).
# Without a modem (OSMO_PMOS_MODEM=0) the VM must still be switched to its
# "combine" card, otherwise the default output stays the BRIDGE (0x12), whose
# playback goes into gsm_mic. osmo-pmos-qemu calls --voix as soon as SSH
# answers when it does not connect the modem.
voix() {
# THE VOICE, AND WHY TWO SOUND CARDS ARE NEEDED. QEMU cannot push sound into
# a guest's speaker: from an emulated card, PLAYBACK goes to a host sink and
# CAPTURE comes from a host source. The call downlink (gsm_audio) lives host
# side: it can only enter the VM through its capture, never through its
# speaker. With a single card the VM has the NETWORK's point of view - it
```

```
# hears the peer in its microphone - and Calls stays mute.
#
# So the VM has two cards, set by pmbootstrap (osmo_bench_args):
#   0000:00:12.0  the BRIDGE  : capture = gsm_audio.monitor (the downlink),
#                               playback = gsm_mic (the uplink);
#   0000:00:13.0  the HANDSET : the host default devices, the real speakers
#                               and the real microphone.
# They are crossed INSIDE the VM: bridge -> handset to hear, handset ->
# bridge to speak. The VM is then really in the voice path.
#
# And the two DIRECT host loopbacks (gsm_audio.monitor to the speakers,
# microphone to gsm_mic) are cut: otherwise the voice arrives TWICE, once
# directly and once through the VM with its added latency - a plain echo.
# OSMO_PMOS_RELAI=0 to leave the audio alone.
if [ "${OSMO_PMOS_RELAI:-1}" = "1" ]; then
    echo "--- voix"
    # THE SPEAKER AND MICROPHONE ASSIGNMENT COMES FROM lib/audio.sh, the
    # single source of truth (see "L AFFECTATION DES HAUT-PARLEURS" there):
    # the host direct loopbacks are removed, the echo canceller is loaded (or
    # already there, osmo-pmos-qemu loads it before QEMU) and becomes the
    # default, and the QEMU "combine" streams go to it - audio_hp_sink /
    # audio_mic_source.
    HP_SINK=""; MIC_SRC=""
    if [ -n "$AUDIO_LIB" ]; then
        ensure_echo_cancel
        ensure_record_mix
        # THE VOICE STAYS ON THE HOST (see lib/audio.sh, "QUI PORTE LA
        # VOIX"). OSMO_PMOS_VOIX_VM=1 routes it through the VM: the marker
        # is then written BEFORE calling the functions, so that they remove
        # the direct loopbacks instead of re-adding them.
        if [ "${OSMO_PMOS_VOIX_VM:-0}" = "1" ]; then
            : > "$PMOS_VOIX_VM_MARQUEUR" 2>/dev/null
            remove_direct_loopbacks
            echo "  la voix passe PAR LA VM (OSMO_PMOS_VOIX_VM=1) - ~800 ms d aller-retour"
        else
            rm -f "$PMOS_VOIX_VM_MARQUEUR" 2>/dev/null
            ensure_local_loopback
            ensure_local_mic
            echo "  la voix reste sur l hote (40 ms) - OSMO_PMOS_VOIX_VM=1 pour la faire passer par la VM"
        fi
        HP_SINK="$(audio_hp_sink)"; MIC_SRC="$(audio_mic_source)"
        echo "  haut-parleurs de l operateur : ${HP_SINK:-aucun} ; micro : ${MIC_SRC:-aucun}"
    else
        echo "  ATTENTION : lib/audio.sh introuvable (OSMO_REPO ?) - l audio de l hote n est pas regle"
    fi

    # THE MUTES THAT COME BACK. The downlink crosses four volume controls,
    # and ONE on mute is enough for the voice to vanish with no loopback
    # missing. Two are host side: the QEMU "combine" stream (the VM output to
    # the speakers) and the speaker sink. PulseAudio REMEMBERS them by
    # application name (module-stream-restore): once muted, the stream comes
    # back muted at every launch. So they are unmuted at every setup.
    for i in $(pactl list sink-inputs short 2>/dev/null | cut -f1); do
        pactl set-sink-input-mute "$i" 0 2>/dev/null
        pactl set-sink-input-volume "$i" 100% 2>/dev/null
    done
    for o in $(pactl list source-outputs short 2>/dev/null | cut -f1); do
        pactl set-source-output-mute "$o" 0 2>/dev/null
    done
    # The QEMU streams of the "combine" card (media.name=combine) on the
    # operator's speakers and microphone - already there when osmo-pmos-qemu
    # named them at launch (OSMO_PMOS_HP / OSMO_PMOS_MIC), otherwise moved.
    if [ -n "$MIC_SRC" ]; then
        for so in $(pactl list source-outputs 2>/dev/null | awk '/Source Output #/{id=$3} /media.name =
        ↪  "combine"/{print id}' | tr -d '#'); do
            pactl move-source-output "$so" "$MIC_SRC" 2>/dev/null
```

```
    done
fi
if [ -n "$HP_SINK" ]; then
    for si in $(pactl list sink-inputs 2>/dev/null | awk '/Sink Input #/{id=$3} /media.name = "combine"/{print
    ↪  id}' | tr -d '#'); do
        pactl move-sink-input "$si" "$HP_SINK" 2>/dev/null
    done
fi
pactl set-source-mute @DEFAULT_SOURCE@ 0 2>/dev/null
pactl set-sink-mute @DEFAULT_SINK@ 0 2>/dev/null && echo "  sourdines de l hote levees (haut-parleur et micro)"
# speech-dispatcher (dummy module) plays a loud, low dummy message in the
# speakers when an application requests speech synthesis: it is sent to
# the osmo_tts_off trash sink (see lib/audio.sh); stream-restore remembers
# it for the following streams.
pactl list short sinks 2>/dev/null | grep -qw osmo_tts_off \
    || pactl load-module module-null-sink sink_name=osmo_tts_off format=s16le rate=8000 channels=1
    ↪  sink_properties=device.description=TTS_off >/dev/null 2>&1
for si in $(pactl list sink-inputs 2>/dev/null | awk '/Sink Input #/{id=$3} /application.name =
↪  "speech-dispatcher/{print id}' | tr -d '#'); do
    pactl move-sink-input "$si" osmo_tts_off 2>/dev/null && echo "  flux speech-dispatcher detourne des
    ↪  haut-parleurs (osmo_tts_off)"
done
ssh_vm "VOIX_VM='${OSMO_PMOS_VOIX_VM:-0}'; "'
    pci() { pactl list "$1" short 2>/dev/null | grep "pci-0000_00_$2\.0" | grep -v monitor | head -n1 | cut
↪  -f2; }
    PONT_OUT=$(pci sinks 12);  PONT_IN=$(pci sources 12)
    COMB_OUT=$(pci sinks 13);  COMB_IN=$(pci sources 13)
    if [ -z "$PONT_IN" ] || [ -z "$COMB_OUT" ]; then
        echo "  ATTENTION : les deux cartes son ne sont pas la - VM lancee avant le patch ?"
        pactl list sinks short 2>/dev/null
        exit 0
    fi
    # The handset becomes the default device: otherwise the Phosh ringtone
    # and notifications would go into the call uplink.
    pactl set-default-sink "$COMB_OUT"
    pactl set-default-source "$COMB_IN"
    # The two other downlink volume controls are here: Phosh readily
    # mutes the handset (volume key, silent mode) and the voice dies
    # there, loopbacks in place. Unmute them.
    for c in "$PONT_OUT" "$COMB_OUT"; do
        pactl set-sink-mute "$c" 0 2>/dev/null
        pactl set-sink-volume "$c" 100% 2>/dev/null
    done
    for c in "$PONT_IN" "$COMB_IN"; do
        pactl set-source-mute "$c" 0 2>/dev/null
    done
    # IDEMPOTENT. If both loopbacks are already there, pinned, with the
    # right ends, they are left alone (reloading them during a call
    # would cut the voice for half a second).
    deja=$(pactl list short modules 2>/dev/null | grep -c "module-loopback.*sink_dont_move=true")
    if [ "$deja" = "2" ] \
       && pactl list short modules 2>/dev/null | grep -q "source=$PONT_IN.*sink=$COMB_OUT" \
       && pactl list short modules 2>/dev/null | grep -q "source=$COMB_IN.*sink=$PONT_OUT"; then
        echo "  bouclages deja en place et epingles - inchanges"
        exit 0
    fi
    # THE VM PULSE MUST NOT SUSPEND ITS CARDS. module-suspend-on-idle
    # puts a sink to sleep as soon as nobody plays: QEMU then CLOSES its
    # host-side streams, and each wake-up renegotiates the latency of the
    # two emulated cards, which breaks the voice. The loopbacks keep the
    # cards awake, but not before they are loaded: the module is removed.
    for m in $(pactl list modules short 2>/dev/null | grep module-suspend-on-idle | cut -f1); do
        pactl unload-module "$m" 2>/dev/null && echo "  mise en veille des cartes desactivee
↪  (module-suspend-on-idle)"
    done
    # CROSSING THE TWO CARDS IS NOT THE DEFAULT. Without it, the VM does
```

```
        # not carry the voice: it keeps its cards for the ringtone, the
        # notifications and the Calls application, and the operator hears the
        # call through the host (40 ms instead of ~800). So the loopbacks left
        # by a previous setup are REMOVED, and only re-added if the host asked
        # for it (OSMO_PMOS_VOIX_VM=1, passed here as the VOIX_VM variable).
        for m in $(pactl list modules short 2>/dev/null | grep module-loopback | cut -f1); do
            pactl unload-module "$m" 2>/dev/null
        done
        if [ "${VOIX_VM:-0}" != "1" ]; then
            echo "  cartes non croisees : la voix reste sur l hote"
            exit 0
        fi
        # Idempotent: the previous loopbacks are removed before re-adding.
        # adjust_time=0 AND 400 ms: the mobile writes its audio in 20 ms
        # frames at the L1 pace; with the default rate adjustment
        # (adjust_time=10), module-loopback resamples constantly and the
        # voice breaks up. A VM lacks the regularity of a sound card: a
        # fixed rate plus a 400 ms buffer gives a stable delay, imperceptible
        # on a call.
        for m in $(pactl list modules short 2>/dev/null | grep module-loopback | cut -f1); do
            pactl unload-module "$m" 2>/dev/null
        done
        # PINNED (dont_move): callaudiod / Phosh change the default device AT
        # EVERY CALL (journal: "card has no voice profile and no usable
        # sink") and PulseAudio then moves every unpinned stream - the two
        # loopbacks would go elsewhere as soon as the call is answered.
        pactl load-module module-loopback source="$PONT_IN" sink="$COMB_OUT" latency_msec=400 adjust_time=0
↪   sink_dont_move=true source_dont_move=true >/dev/null \
            && echo "  descendant : pont -> combine (on entend)"
        pactl load-module module-loopback source="$COMB_IN" sink="$PONT_OUT" latency_msec=400 adjust_time=0
↪   sink_dont_move=true source_dont_move=true >/dev/null \
            && echo "  montant    : combine -> pont (on parle)"
    ' 2>&1 | tail -n 6
fi
}

# WAIT FOR SSH. Launched by hand while the VM is still booting, everything
# would fail: wait up to 90 s.
for _ in $(seq 1 45); do
    ssh_vm 'echo ok' 2>/dev/null | grep -q ok && break
    [ "$_" -eq 1 ] && echo "  la VM ne repond pas encore en SSH (port $PORT), on attend..."
    sleep 2
done
ssh_vm 'echo ok' 2>/dev/null | grep -q ok || { echo "  ATTENTION : pas de SSH sur le port $PORT - la VM est-elle
↪   lancee ?"; exit 1; }
if [ "${1:-}" = "--voix" ] || [ "${1:-}" = "voix" ]; then voix; exit 0; fi

ssh_vm "echo $PASS | sudo -S -p '' sh -c '
    mkdir -p /etc/systemd/resolved.conf.d
    # DNSSEC=no: the internal QEMU DNS (10.0.2.3) does not sign its answers,
    # and resolved would refuse everything (DNSSEC validation failed).
    printf \"[Resolve]\nDNS=10.0.2.3 1.1.1.1\nFallbackDNS=8.8.8.8\nDNSSEC=no\nDNSOverTLS=no\n\" \
        > /etc/systemd/resolved.conf.d/osmo.conf
    systemctl restart systemd-resolved 2>/dev/null

    # The modem arrives on a PCI serial port: 8250_pci is built into this
    # kernel, so there is no module to load (ftdi_sio does not exist here).

    mkdir -p /etc/udev/rules.d
    # TWO PORTS, ONE MODEM. hvc0 = command, hvc1 = data
    # (ID_MM_PORT_TYPE_AT_PPP: where ModemManager dials ATD*99), and the same
    # ID_MM_PHYSDEV_UID so that MM groups them into one modem instead of
    # seeing two. ttyS[1-9]: the 16550 of OSMO_PMOS_SERIAL=pci.
    {
```

```
      printf \"ACTION==\\\"add|change\\\", SUBSYSTEM==\\\"tty\\\", KERNEL==\\\"hvc0\\\",
↪  ENV{ID_MM_DEVICE_PROCESS}=\\\"1\\\", ENV{ID_MM_PHYSDEV_UID}=\\\"osmo-banc\\\",
↪  ENV{ID_MM_PORT_TYPE_AT_PRIMARY}=\\\"1\\\"\n\"
      printf \"ACTION==\\\"add|change\\\", SUBSYSTEM==\\\"tty\\\", KERNEL==\\\"hvc1\\\",
↪  ENV{ID_MM_DEVICE_PROCESS}=\\\"1\\\", ENV{ID_MM_PHYSDEV_UID}=\\\"osmo-banc\\\",
↪  ENV{ID_MM_PORT_TYPE_AT_PPP}=\\\"1\\\"\n\"
      printf \"ACTION==\\\"add|change\\\", SUBSYSTEM==\\\"tty\\\", KERNEL==\\\"ttyS[1-9]\\\",
↪  ENV{ID_MM_DEVICE_PROCESS}=\\\"1\\\", ENV{ID_MM_TTY_BAUDRATE}=\\\"115200\\\"\n\"
    } > /etc/udev/rules.d/99-osmo-modem.rules
    udevadm control --reload 2>/dev/null
    udevadm trigger --subsystem-match=tty 2>/dev/null
    # The modem port is a virtconsole (/dev/hvc0): the systemd getty
    # generator opens a login prompt on the first virtualisation console,
    # which would answer the modem. It is masked, the port is the modem only.
    systemctl mask --now serial-getty@hvc0.service getty@hvc0.service serial-getty@hvc1.service
↪  getty@hvc1.service >/dev/null 2>&1

    # DATA: pppd and the NetworkManager PPP plugin. Without them, ATD*99
    # ends in CONNECT then NO CARRIER one second later (PPP failed to start:
    # libnm-ppp-plugin.so is not installed). The kernel must have PPP
    # (build-pm.sh); the modules load at boot.
    # The wired link has no default route (see the next block): for apk it
    # is restored for the duration of this block, with the QEMU DNS, and the
    # osmo-filaire service removes it right after.
    ip route replace default via 10.0.2.2 dev eth0 2>/dev/null
    resolvectl dns eth0 10.0.2.3 2>/dev/null; resolvectl default-route eth0 yes 2>/dev/null
    apk info -e ppp networkmanager-ppp >/dev/null 2>&1 \
        || apk add ppp networkmanager-ppp 2>&1 | tail -n 1
    # networkmanager and networkmanager-ppp MUST be the same version: NM
    # looks for the plugin in /usr/lib/NetworkManager/<its version>/.
    NMV=\$(apk info -v 2>/dev/null | grep -oE \"^networkmanager-[0-9.]+\" | head -n1 | sed
↪  \"s/^networkmanager-//\")
    NMP=\$(apk info -v 2>/dev/null | grep -oE \"^networkmanager-ppp-[0-9.]+\" | head -n1 | sed
↪  \"s/^networkmanager-ppp-//\")
    # ALL the networkmanager-* sub-packages (wwan, ppp, ...): each installs
    # its plugin in the directory of ITS version; a single one lagging, and
    # NM no longer sees the modem (wwan) or no longer starts pppd (ppp).
    [ \"\$NMV\" = \"\$NMP\" ] || apk upgrade \$(apk info 2>/dev/null | grep -E \"^networkmanager\") ppp 2>&1 |
↪  tail -n 1
    # The journal is bounded: unbounded it fills the 2 GB disk.
    mkdir -p /etc/systemd/journald.conf.d
    printf \"[Journal]\nSystemMaxUse=48M\n\" > /etc/systemd/journald.conf.d/osmo.conf
    journalctl --vacuum-size=40M >/dev/null 2>&1; rm -rf /var/cache/apk/*
    printf \"ppp_generic\nppp_async\n\" > /etc/modules-load.d/osmo-ppp.conf
    modprobe ppp_async 2>/dev/null

    systemctl restart ModemManager 2>/dev/null
    sleep 3
    mmcli --scan-modems 2>/dev/null | tail -n 1
'" 2>&1 | tail -n 4

# DATA THROUGH OUR 4G ONLY. The QEMU virtio card (the "wired" link) would
# give the phone a default route and a DNS through the QEMU NAT, and
# everything would go through it. The card cannot be removed from QEMU: it is
# through it (hostfwd 2222) that this script talks to the VM. So it is made
# INVISIBLE to the phone: NetworkManager no longer manages it (virtio_net
# driver unmanaged), a small service leaves it only its management address
# 10.0.2.15 without default route or DNS, resolved no longer points to the
# QEMU DNS. The phone has only one way out: the mobile connection (APN
# srsapn, created here if missing, automatic), hence the bench PPP, hence the
# radio. OSMO_PMOS_FILAIRE=1 restores the wired link (managed by NM, QEMU
# DNS). The switch cuts for a second the address we are connected through:
# it runs detached in the VM, and we wait for SSH to come back.
FILAIRE="${OSMO_PMOS_FILAIRE:-0}"
{ echo "$PASS"; echo "FILAIRE=$FILAIRE"; cat <<'GUEST'
set -u
```

```
CONF=/etc/NetworkManager/conf.d/90-osmo-filaire.conf
SVC=/etc/systemd/system/osmo-filaire.service
BIN=/usr/local/bin/osmo-filaire
mkdir -p /etc/NetworkManager/conf.d /etc/systemd/resolved.conf.d
if [ "$FILAIRE" = 0 ]; then
    printf '[keyfile]\nunmanaged-devices=driver:virtio_net\n' > "$CONF"
    cat > "$BIN" <<'EOB'
#!/bin/sh
# osmo-filaire - the QEMU virtio card keeps its management address (SSH from
# the host through hostfwd), without default route or DNS: the phone data
# goes through the mobile connection, hence through the bench PPP.
for d in /sys/class/net/*; do
    n=${d##*/}
    drv=$(basename "$(readlink "$d/device/driver" 2>/dev/null)" 2>/dev/null)
    [ "$drv" = virtio_net ] || continue
    ip link set "$n" up
    ip addr replace 10.0.2.15/24 dev "$n"
    ip route del default dev "$n" 2>/dev/null
    resolvectl revert "$n" 2>/dev/null     # no QEMU DNS through this link
done
exit 0
EOB
    chmod 755 "$BIN"
    cat > "$SVC" <<'EOS'
[Unit]
Description=osmo-filaire - carte virtio en adresse de gestion seule (data par la 4G)
After=NetworkManager.service
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/bin/osmo-filaire
[Install]
WantedBy=multi-user.target
EOS
    systemctl daemon-reload; systemctl enable osmo-filaire >/dev/null 2>&1
    printf '[Resolve]\nDNSSEC=no\nDNSOverTLS=no\n' > /etc/systemd/resolved.conf.d/osmo.conf
    # A single mobile profile: the one the user already created (srsapn) is
    # used as is, otherwise one is created.
    GSM=$(nmcli -t -f NAME,TYPE con show 2>/dev/null | awk -F: '$2 == "gsm" {print $1; exit}')
    if [ -z "$GSM" ]; then
        nmcli con add type gsm ifname '*' con-name 'Osmocom 4G' apn srsapn \
            connection.autoconnect yes ipv4.route-metric 50 >/dev/null 2>&1 && GSM='Osmocom 4G'
    fi
    # NetworkManager is RESTARTED: apk may have just updated it with the PPP
    # plugin, and the daemon still in memory looks for the plugin of ITS
    # version number. The configuration file being in place, it restarts
    # with the wired link unmanaged.
    # Detached through systemd-run: a plain "setsid ... &" dies with the SSH
    # session (the VM logind kills the user processes at disconnection).
    systemd-run --quiet --collect --unit "osmo-filaire-bascule-$$" sh -c "
        sleep 1
        systemctl restart NetworkManager
        sleep 2
        /usr/local/bin/osmo-filaire
        systemctl restart systemd-resolved
        [ -n '$GSM' ] && nmcli con up '$GSM' >/dev/null 2>&1
        exit 0
    "
    echo "  filaire cache au telephone : adresse de gestion seule, data par la connexion mobile (APN srsapn)"
else
    rm -f "$CONF" "$BIN" "$SVC"; systemctl daemon-reload
    printf '[Resolve]\nDNS=10.0.2.3 1.1.1.1\nFallbackDNS=8.8.8.8\nDNSSEC=no\nDNSOverTLS=no\n' >
↪ /etc/systemd/resolved.conf.d/osmo.conf
    systemd-run --quiet --collect --unit "osmo-filaire-bascule-$$" sh -c '
        sleep 1
        systemctl restart NetworkManager
```

```
        systemctl restart systemd-resolved
    '
    echo "  filaire rendu au telephone (OSMO_PMOS_FILAIRE=1)"
fi
GUEST
} | ssh_vm "sudo -S -p '' sh -s" 2>/dev/null | tail -n 2
for _ in $(seq 1 20); do
    sleep 1
    ssh_vm 'echo ok' 2>/dev/null | grep -q ok && break
done
ssh_vm 'echo ok' 2>/dev/null | grep -q ok || echo "  ATTENTION : la VM ne repond plus en SSH apres le basculement
 ↪  reseau"

# The bench modem now connects to the port QEMU keeps open. Only HERE, with
# the system booted: connected from the start, the UEFI firmware takes its
# answers for keystrokes and never boots (see the osmo-pmos-qemu header).
PORT_AT="${OSMO_PMOS_AT_PORT:-12346}"
REPO="${OSMO_REPO:-/opt/GSM/osmo-operator}"
# THE MODEM AS ROOT: the phone data is a PPP link the bench turns into a tun
# (/dev/net/tun + TUNSETIFF), placed in the srsUE network namespace with a
# NAT towards tun_srsue. All of that requires root. osmo-pmos-qemu has
# already done sudo -v in this terminal; otherwise ask if there is a
# terminal, and fall back to the user, saying so.
BANC_LOG="/tmp/osmo-phonesim-vm-$(id -un).log"
BANC_SUDO=""
if [ "$(id -u)" -ne 0 ]; then
    if sudo -n true 2>/dev/null || { [ -t 0 ] && sudo -v; }; then
        BANC_SUDO="sudo -n"
    else
        echo "  ATTENTION : pas de root pour le modem - la data PPP n aura pas de sortie radio"
    fi
fi
if pgrep -f "[o]smo-phonesim-banc.py --connect" >/dev/null 2>&1; then
    echo "  modem deja branche sur la VM"
else
    # The AT trace of THIS modem in its own file: as root, the two bench
    # instances would otherwise both write to /var/log/osmo-at-ofono.log and
    # their dialogues would mix.
    # sudo BEFORE setsid: the password cache is attached to the terminal,
    # and setsid detaches from it - sudo launched under setsid would ask for
    # the password again ("a terminal is required").
    # --data: the data port (hvc1, QEMU chardev on PORT_AT+1), see the
    # two-port modem in osmo-phonesim-banc.py.
    # THE RADIO EGRESS IS CHOSEN BY THE BENCH (_radio_choix in
    # osmo-phonesim-banc.py): 4G first (tun_srsue in the ue1 netns),
    # otherwise the GPRS PDP of the osmocom-bb modem (tun_modem in msgprs).
    # Nothing is forced here.
    $BANC_SUDO setsid nohup env OSMO_AT_LOG_OFONO=/tmp/osmo-at-vm.log \
        OSMO_PPP_DNS="${OSMO_PPP_DNS:-192.168.1.254,1.1.1.1}" \
        "$REPO/tools/osmo-phonesim-banc.py" --connect "127.0.0.1:$PORT_AT" --data "127.0.0.1:$((PORT_AT + 1))" \
        >>"$BANC_LOG" 2>&1 </dev/null &
    sleep 3
    pgrep -f "[o]smo-phonesim-banc.py --connect" >/dev/null 2>&1 \
        && echo "  modem du banc branche sur la VM (port $PORT_AT)" \
        || echo "  ATTENTION : le modem ne s est pas branche (cf. $BANC_LOG)"
fi
# MODEMMANAGER PROBES AFTER THE CONNECTION. The MM restart above happens
# BEFORE the bench is on the port: the hvc0 probe finds nothing, MM drops
# the port and never probes it again. A second restart, with the modem
# connected, and it sees it.
# And NetworkManager AFTER ModemManager: restarted while MM is absent, NM
# would log "error creating ModemManager client" and never see the modem
# afterwards - no gsm device, no PPP. Detached through systemd-run: the NM
# restart cuts the SSH session that requests it.
ssh_vm "echo $PASS | sudo -S -p '' systemd-run --quiet --collect --unit osmo-mm-puis-nm sh -c 'systemctl restart
 ↪  ModemManager; sleep 6; systemctl restart NetworkManager'" >/dev/null 2>&1
```

```
for _ in $(seq 1 20); do
    sleep 1
    ssh_vm 'echo ok' 2>/dev/null | grep -q ok && break
done
sleep 8

voix

echo "--- etat"
# ModemManager has just been restarted: it probes the port for a few
# seconds, and "No modems were found" read too early is misleading. Give it
# up to 40 s.
ssh_vm 'for i in $(seq 1 20); do mmcli -L 2>/dev/null | grep -q Modem/ && break; sleep 2; done
ls /dev/hvc0 /dev/hvc1 /dev/ttyS[1-9] 2>/dev/null | head -n 4
getent hosts alpinelinux.org >/dev/null 2>&1 && echo "DNS OK" || echo "DNS KO"
mmcli -L 2>/dev/null | tail -n 1
mmcli -m any 2>/dev/null | grep -E "state:|access tech|primary port|ports" | sed "s/^ *|//"' 2>&1 | tail -n 8
```

2.272 /opt/GSM/osmo-operator/tools/osmo-pmos.sh

20470 octets, 431 lignes → 431 lignes

```
#!/bin/bash
# ============================================================================
# osmo-pmos.sh - THE BENCH PHONE: A postmarketOS / Phosh VM.
#
# postmarketOS comes with ModemManager, Calls and Chatty: a real Linux phone,
# and a real phone connects to a MODEM. The bench has one -
# tools/osmo-phonesim-banc.py, which speaks 27.007 on one side and drives the
# bench on the other (osmo-bsc VTY, osmo-msc SMPP, Asterisk AMI). This script
# lays the wire between the two.
#
#   Phosh / Calls / Chatty
#        ↕  D-Bus
#   ModemManager (in the VM)
#        ↕  /dev/ttyS1  (serial port of the VM)
#   QEMU  -serial tcp:127.0.0.1:12346,server=on,wait=off
#        ↕  TCP
#   osmo-phonesim-banc.py --connect 127.0.0.1:12346   (the bench modem)
#        ↕  VTY / SMPP / AMI
#   osmo-bsc · osmo-msc · Asterisk · the osmocom-bb mobile
#
# WHY WE GO TOWARDS QEMU, and not the reverse: the VM's UEFI firmware reads
# any serial port as a console. A modem already chatty during boot makes it
# believe the boot is being interrupted, and it stops on its menu. So QEMU
# LISTENS (server=on,wait=off) and the modem connects once the system is up -
# the modem's --connect mode, which waits for the VM's SSH port to answer
# before connecting.
#
#   osmo-pmos up         the icon's gesture: the pmbootstrap VM (osmo-pmos-qemu),
#                        smartphone|tablette form factor as argument; stop and status go there too
#   osmo-pmos install    qemu packages + the postmarketOS image + the preparation
#   osmo-pmos fetch      downloads (or resumes) the image and checks its sha256
#   osmo-pmos provision  password, sshd, and the ModemManager rule for the AT port
#   osmo-pmos start      starts the VM AND the bench modem that goes with it
#   osmo-pmos toggle     switch: shuts the VM down if running, starts it otherwise (the icon)
#   osmo-pmos stop       stops both
#   osmo-pmos status     where each piece stands
#   osmo-pmos shell      a shell in the VM (ssh)
#   osmo-pmos mm         what ModemManager sees of the modem, INSIDE the VM
#   osmo-pmos at         talk to the bench modem by hand (tools/at-cmd.py)
#   osmo-pmos log        the VM's AT trace, live
#
# THE PASSWORD IS 147147. It is the one of the official postmarketOS images,
```

```
# and it is SET AGAIN at each provision so that the live key, the installed
# system and a re-downloaded image all answer the same. It opens the Phosh
# session (the lock screen), the "user" account and ssh. It is not a secret:
# this is a demonstration bench.
# ============================================================================
set -u

REPO="${OSMO_REPO:-/opt/GSM/osmo-operator}"
VAR="${OSMO_PMOS_DIR:-/var/lib/osmo-pmos}"
IMG="$VAR/pmos.img"
RUNDIR="${OSMO_PMOS_RUN:-/run/osmo-pmos}"
PIDF="$RUNDIR/qemu.pid"
MDMPID="$RUNDIR/modem.pid"

# The phone password. 147147 everywhere: lock screen, "user" account, ssh.
# Changeable for a particular demonstration, but that is not the use case -
# see the header.
PMOS_PASS="${OSMO_PMOS_PASSWORD:-147147}"
PMOS_USER="${OSMO_PMOS_USER:-user}"

# The VM serial port (the modem), and the SSH port. The second ALSO serves
# as a witness: the modem only connects once it answers (see the header).
AT_PORT="${OSMO_PMOS_AT_PORT:-12346}"
SSH_PORT="${OSMO_PMOS_SSH_PORT:-2222}"

# The image. The official images live in dated directories; "the most
# recent" is resolved on the fly, and everything is overridable:
#   OSMO_PMOS_IMG_URL   the exact URL of a .img.xz image
#   OSMO_PMOS_IMG_FILE  an image ALREADY downloaded (nothing is downloaded)
PMOS_BRANCHE="${OSMO_PMOS_BRANCHE:-v25.12}"
PMOS_APPAREIL="${OSMO_PMOS_APPAREIL:-generic-x86_64}"
PMOS_IHM="${OSMO_PMOS_IHM:-phosh}"
PMOS_BASE="https://images.postmarketos.org/bpo/$PMOS_BRANCHE/$PMOS_APPAREIL/$PMOS_IHM/"

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'
_say()  { echo -e "  ${CYAN}→${NC} $*"; }
_ok()   { echo -e "  ${GREEN}✓${NC} $*"; }
_warn() { echo -e "  ${YELLOW}!${NC} $*"; }
_err()  { echo -e "  ${RED}✗${NC} $*" >&2; }

_root() {
    [ "$(id -u)" -eq 0 ] && return 0
    _err "cette commande demande root : sudo osmo-pmos $*"
    return 1
}

# ── THE PACKAGES ──────────────────────────────────────────────────────────
# qemu-system-x86 (the VM), ovmf (the UEFI firmware: the postmarketOS
# generic-x86_64 image does not boot in BIOS), xz-utils (the image is
# compressed), and sshpass for "osmo-pmos shell" without typing the password
# every time.
_pmos_paquets() {
    export DEBIAN_FRONTEND=noninteractive
    local p manque=()
    for p in qemu-system-x86 qemu-system-gui qemu-system-modules-opengl ovmf xz-utils curl sshpass; do
        dpkg -s "$p" >/dev/null 2>&1 || manque+=("$p")
    done
    [ "${#manque[@]}" -eq 0 ] && { _ok "paquets deja la (qemu, ovmf, xz, curl, sshpass)"; return 0; }
    _say "installation : ${manque[*]}"
    apt-get update -y >/dev/null 2>&1 || true
    for p in "${manque[@]}"; do
        apt-get install -y --no-install-recommends "$p" >/dev/null 2>&1 \
            && _ok "$p installe" || _warn "$p indisponible (apt) - ignore"
    done
}
```

```
_ovmf() {
    local f
    for f in /usr/share/OVMF/OVMF_CODE_4M.fd /usr/share/OVMF/OVMF_CODE.fd \
             /usr/share/ovmf/OVMF.fd /usr/share/qemu/OVMF.fd; do
        [ -f "$f" ] && { echo "$f"; return 0; }
    done
    return 1
}

# ── THE IMAGE ──────────────────────────────────────────────────────────
# Resolve the most recent dated directory, download with resume (curl -C -),
# and VERIFY the sha256 published next to it. An image truncated by a network
# cut gives a VM that does not boot, without saying why.
_pmos_url() {
    [ -n "${OSMO_PMOS_IMG_URL:-}" ] && { echo "$OSMO_PMOS_IMG_URL"; return 0; }
    local index date fichier
    index="$(curl -fsSL "$PMOS_BASE" 2>/dev/null)" || return 1
    date="$(printf '%s' "$index" | grep -o 'href="[0-9]\{8\}-[0-9]\{4\}/"' \
            | sed 's/href="//; s|/"||' | sort -r | head -1)"
    [ -n "$date" ] || return 1
    index="$(curl -fsSL "$PMOS_BASE$date/" 2>/dev/null)" || return 1
    fichier="$(printf '%s' "$index" | grep -o 'href="[^"]*\.img\.xz"' \
              | sed 's/href="//; s/"//' | head -1)"
    [ -n "$fichier" ] || return 1
    echo "$PMOS_BASE$date/$fichier"
}

pmos_fetch() {
    _root fetch || return 1
    install -d "$VAR" || return 1
    if [ -n "${OSMO_PMOS_IMG_FILE:-}" ]; then
        [ -f "$OSMO_PMOS_IMG_FILE" ] || { _err "OSMO_PMOS_IMG_FILE : $OSMO_PMOS_IMG_FILE introuvable"; return 1; }
        _say "image fournie : $OSMO_PMOS_IMG_FILE"
        case "$OSMO_PMOS_IMG_FILE" in
            *.xz) xz -dc "$OSMO_PMOS_IMG_FILE" > "$IMG" ;;
            *)    cp -f "$OSMO_PMOS_IMG_FILE" "$IMG" ;;
        esac
        _ok "image posee : $IMG"
        return 0
    fi
    if [ -f "$IMG" ] && [ "${OSMO_PMOS_REFETCH:-0}" != "1" ]; then
        _ok "image deja la : $IMG ($(du -h "$IMG" | cut -f1)) - OSMO_PMOS_REFETCH=1 pour la reprendre"
        return 0
    fi
    local url; url="$(_pmos_url)" || { _err "index postmarketOS injoignable ($PMOS_BASE) - donnez
    ↪  OSMO_PMOS_IMG_URL ou OSMO_PMOS_IMG_FILE"; return 1; }
    local xzf="$VAR/$(basename "$url")"
    _say "telechargement : $url"
    _say "1,5 Go environ - la reprise est possible (relancez la commande)"
    curl -fL -C - -o "$xzf" "$url" || { _err "telechargement interrompu - relancez, il reprend"; return 1; }
    if curl -fsSL -o "$xzf.sha256" "$url.sha256" 2>/dev/null; then
        _say "verification sha256..."
        ( cd "$VAR" && sha256sum -c "$(basename "$xzf").sha256" >/dev/null 2>&1 ) \
            && _ok "sha256 conforme" \
            || { _err "sha256 NON conforme : image corrompue, on ne la deballe pas"; return 1; }
    else
        _warn "pas de sha256 publie - on continue sans verifier"
    fi
    _say "decompression (compter quelques minutes)..."
    xz -dc "$xzf" > "$IMG" || { _err "decompression impossible"; return 1; }
    # The image comes out at the exact size; a phone needs some room for its
    # logs and messages. The file is sparse: these 8 GiB cost nothing until
    # used.
    qemu-img resize -f raw "$IMG" 8G >/dev/null 2>&1 || true
    _ok "image prete : $IMG"
```

```
    [ "${OSMO_PMOS_GARDER_XZ:-0}" = "1" ] || rm -f "$xzf" "$xzf.sha256"
}

# ── PREPARING THE IMAGE ──────────────────────────────────────────────
# Mount the image's root partition on the host (losetup -P) and install three
# things. Safer than an interactive first boot: the VM is ready BEFORE being
# powered on, and a re-downloaded image goes through here again.
pmos_provision() {
    _root provision || return 1
    [ -f "$IMG" ] || { _err "pas d image : osmo-pmos fetch d abord"; return 1; }
    local loop mnt="$VAR/mnt" part
    loop="$(losetup -f --show -P "$IMG")" || { _err "losetup impossible"; return 1; }
    # The root is the largest partition of the image (the first is the ESP).
    part="$(lsblk -brno NAME,SIZE "$loop" | tail -n +2 | sort -k2 -n | tail -1 | cut -d' ' -f1)"
    [ -n "$part" ] || { losetup -d "$loop"; _err "aucune partition dans l image"; return 1; }
    install -d "$mnt"
    if ! mount "/dev/$part" "$mnt" 2>/dev/null; then
        losetup -d "$loop"
        _err "/dev/$part non montable (image chiffree ? partition inattendue)"
        return 1
    fi
    local rc=0
    if [ ! -d "$mnt/etc" ]; then
        _err "/dev/$part ne ressemble pas a la racine du systeme"
        rc=1
    else
        # 1. THE PASSWORD, written in /etc/shadow. No chroot: the image is
        #    musl/Alpine and its /bin/sh does not have the same libraries as
        #    the host. openssl builds the SHA-512 digest that shadow expects.
        local hash
        hash="$(openssl passwd -6 "$PMOS_PASS" 2>/dev/null)"
        if [ -n "$hash" ] && [ -f "$mnt/etc/shadow" ]; then
            local u
            for u in root "$PMOS_USER"; do
                grep -q "^$u:" "$mnt/etc/shadow" || continue
                # Field 2 is the digest; awk rather than sed because a hash
                # contains "/".
                awk -F: -v u="$u" -v h="$hash" 'BEGIN{OFS=":"} $1==u{$2=h} {print}' \
                    "$mnt/etc/shadow" > "$mnt/etc/shadow.osmo" \
                    && mv "$mnt/etc/shadow.osmo" "$mnt/etc/shadow"
            done
            chmod 640 "$mnt/etc/shadow" 2>/dev/null || true
            _ok "mot de passe ${BOLD}$PMOS_PASS${NC} pose pour root et $PMOS_USER"
        else
            _warn "mot de passe non pose (openssl absent ou /etc/shadow introuvable)"
        fi

        # 2. THE AT PORT AS SEEN BY ModemManager. Without this rule nothing
        #    happens, SILENTLY: ModemManager does not probe a random ttyS (its
        #    default policy is "strict" - it needs a udev marker), and the
        #    Phosh screen stays on "no modem".
        #    The serial port QEMU adds is a pci-serial (vendor 0x1b36, "Red
        #    Hat, Inc."): the motherboard's ttyS0-3 have no PCI parent and
        #    are therefore not touched by this rule.
        install -d "$mnt/etc/udev/rules.d"
        cat > "$mnt/etc/udev/rules.d/77-osmo-modem-at.rules" <<'UDEV'
# The bench modem, presented to the VM as a PCI serial port by QEMU.
# ID_MM_DEVICE_PROCESS : "probe this port" (ModemManager ignores it otherwise).
# ID_MM_PORT_TYPE_AT   : "this is a command port", not a data port.
SUBSYSTEM=="tty", KERNEL=="ttyS*", ATTRS{vendor}=="0x1b36", \
    ENV{ID_MM_DEVICE_PROCESS}="1", ENV{ID_MM_PORT_TYPE_AT}="1", \
    ENV{ID_MM_TTY_BAUDRATE}="115200"
UDEV
        _ok "regle udev posee : ModemManager sondera le port AT du banc"

        # 3. SSH. It serves as a witness ("the VM is up"), as a debugging
```

```
        #    shell, and it is how the modem knows when to connect.
        if [ -d "$mnt/etc/init.d" ] && [ -x "$mnt/etc/init.d/sshd" ]; then
            install -d "$mnt/etc/runlevels/default"
            ln -sf /etc/init.d/sshd "$mnt/etc/runlevels/default/sshd" 2>/dev/null || true
            _ok "sshd active au demarrage de la VM"
        else
            _warn "sshd absent de l image - « osmo-pmos shell » ne marchera pas"
        fi
    fi
    umount "$mnt" 2>/dev/null || true
    losetup -d "$loop" 2>/dev/null || true
    return $rc
}

# ── START ──────────────────────────────────────────────────────────────
pmos_start() {
    echo "[2/3] machine virtuelle"
    _root start || return 1
    [ -f "$IMG" ] || { _err "pas d image : osmo-pmos install d abord"; return 1; }
    install -d "$RUNDIR"
    if pmos_tourne; then _ok "la VM tourne deja (pid $(cat "$PIDF"))"; else
        local bios; bios="$(_ovmf)" || { _err "firmware UEFI introuvable (paquet ovmf) - l image generic-x86_64 ne
         ↪  demarre qu en UEFI"; return 1; }
        local accel=tcg
        [ -w /dev/kvm ] && accel=kvm
        [ "$accel" = tcg ] && _warn "pas de /dev/kvm : la VM tournera en emulation (lent mais fonctionnel)"
        # -serial tcp:...,server=on,wait=off: QEMU LISTENS and waits for
        # nobody. The modem comes once the system is up (see the header).
        # wait=on would block the boot of the VM itself.
        # The phone format (720x1440) does not exist on every QEMU: fall back
        # to the default resolution if the setting is refused.
        local gpu="virtio-vga"
        local res="${OSMO_PMOS_RES:-720x1440}"
        local xr="${res%x*}" yr="${res#*x}"
        local base=(
            qemu-system-x86_64
            -name osmo-pmos
            -machine q35 -accel "$accel" -m "${OSMO_PMOS_RAM:-2048}" -smp "${OSMO_PMOS_CPU:-2}"
            -bios "$bios"
            -drive "if=virtio,format=raw,file=$IMG"
            -device virtio-tablet-pci -device virtio-keyboard-pci
            -netdev "user,id=n0,hostfwd=tcp:127.0.0.1:$SSH_PORT-:22"
            -device virtio-net-pci,netdev=n0
            -device pci-serial,chardev=at
            -chardev "socket,id=at,host=127.0.0.1,port=$AT_PORT,server=on,wait=off"
            -display "${OSMO_PMOS_DISPLAY:-gtk}"
            -pidfile "$PIDF"
            -daemonize
        )
        if ! "${base[@]}" -device "$gpu,xres=$xr,yres=$yr" 2>"$RUNDIR/qemu.err"; then
            _warn "definition $res refusee par ce QEMU - on garde la sienne"
            "${base[@]}" -device "$gpu" 2>"$RUNDIR/qemu.err" || {
                _err "QEMU n a pas demarre :"; sed 's/^/      /' "$RUNDIR/qemu.err" >&2; return 1; }
        fi
        _ok "VM postmarketOS demarree (ssh 127.0.0.1:$SSH_PORT, modem 127.0.0.1:$AT_PORT)"
        _say "mot de passe : ${BOLD}$PMOS_PASS${NC} (ecran de verrouillage, compte $PMOS_USER, ssh)"
    fi
    pmos_modem_start
}

# The bench modem, host side. --connect mode: it waits for the VM's SSH to
# answer, then connects to the serial port QEMU keeps open.
pmos_modem_start() {
    echo "[3/3] modem du banc"
    if pgrep -f "osmo-phonesim-banc.py --connect 127.0.0.1:$AT_PORT" >/dev/null 2>&1; then
        _ok "le modem du banc est deja branche sur le port $AT_PORT"
```

```
        return 0
    fi
    # /usr/bin/python3 and not the shebang: launched from the bench, the PATH
    # puts the venv first and the interpreter found there lacks the bindings
    # these tools need (same trap as in start-native.sh).
    OSMO_VM_READY_PORT="$SSH_PORT" setsid /usr/bin/python3 \
        "$REPO/tools/osmo-phonesim-banc.py" --connect "127.0.0.1:$AT_PORT" \
        >>/tmp/osmo-phonesim-vm.log 2>&1 &
    echo $! > "$MDMPID"
    _ok "modem du banc lance (il se branchera des que la VM repondra en ssh)"
    _say "journal : /tmp/osmo-phonesim-vm.log ; trace AT : /var/log/osmo-at-ofono.log"
}

pmos_tourne() {
    [ -f "$PIDF" ] || return 1
    kill -0 "$(cat "$PIDF" 2>/dev/null)" 2>/dev/null
}

pmos_stop() {
    _root stop || return 1
    pkill -f "osmo-phonesim-banc.py --connect 127.0.0.1:$AT_PORT" 2>/dev/null \
        && _ok "modem du banc arrete" || true
    if pmos_tourne; then
        kill "$(cat "$PIDF")" 2>/dev/null
        sleep 2
        pmos_tourne && kill -9 "$(cat "$PIDF")" 2>/dev/null
        _ok "VM arretee"
    else
        _say "la VM ne tournait pas"
    fi
    rm -f "$PIDF" "$MDMPID"
}

pmos_status() {
    echo -e "${BOLD}postmarketOS / Phosh - le telephone du banc${NC}"
    if pmos_tourne; then _ok "VM : en marche (pid $(cat "$PIDF"))"
    else _warn "VM : arretee"; fi
    [ -f "$IMG" ] && _ok "image : $IMG ($(du -h "$IMG" 2>/dev/null | cut -f1))" \
                  || _warn "image : absente (osmo-pmos install)"
    if pgrep -f "osmo-phonesim-banc.py --connect 127.0.0.1:$AT_PORT" >/dev/null 2>&1; then
        _ok "modem du banc : lance"
    else
        _warn "modem du banc : arrete"
    fi
    if timeout 2 bash -c "</dev/tcp/127.0.0.1/$SSH_PORT" 2>/dev/null; then
        _ok "ssh de la VM : ouvert sur 127.0.0.1:$SSH_PORT (mot de passe $PMOS_PASS)"
    else
        _warn "ssh de la VM : ferme (la VM demarre encore ?)"
    fi
    if grep -q . /var/log/osmo-at-ofono.log 2>/dev/null; then
        echo -e "  ${CYAN}dernieres lignes du dialogue AT :${NC}"
        tail -6 /var/log/osmo-at-ofono.log | sed 's/^/      /'
    fi
}

pmos_shell() {
    command -v sshpass >/dev/null 2>&1 \
        && exec sshpass -p "$PMOS_PASS" ssh -o StrictHostKeyChecking=no \
                -o UserKnownHostsFile=/dev/null -p "$SSH_PORT" "$PMOS_USER@127.0.0.1" "$@"
    exec ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null \
             -p "$SSH_PORT" "$PMOS_USER@127.0.0.1" "$@"
}

# What ModemManager sees, from the VM: the only place that tells whether the
# bench modem was recognised, and under which number.
pmos_mm() {
```

```
    pmos_shell "sudo mmcli -L; echo; sudo mmcli -m 0 2>/dev/null || true"
}

pmos_install() {
    echo "[1/3] image postmarketOS"
    _root install || return 1
    _pmos_paquets
    pmos_fetch     || return 1
    pmos_provision || return 1
    echo
    _ok "${BOLD}pret${NC} - « osmo-pmos start » allume le telephone"
    _say "mot de passe : ${BOLD}$PMOS_PASS${NC}"
}

# THE ICON ACTION GOES TO pmbootstrap. The dock icon (osmo-pmos.desktop) and
# osmo-launcher.py call "osmo-pmos up"; the bench VM is the pmbootstrap one
# (tools/osmo-pmos-qemu.sh: modem on a PCI serial port, voice bridge sound
# cards, screen format), not the image downloaded here into
# /var/lib/osmo-pmos. up/start, stop/down and status go to the pmbootstrap
# launcher as soon as it is present; the rest (fetch, provision, shell, mm,
# at, log) keeps its meaning here.
QEMU_LANCEUR="${OSMO_PMOS_QEMU:-/usr/local/bin/osmo-pmos-qemu}"
[ -x "$QEMU_LANCEUR" ] || QEMU_LANCEUR="$REPO/tools/osmo-pmos-qemu.sh"
if [ -x "$QEMU_LANCEUR" ]; then
    case "${1:-status}" in
        up|start)  shift; exec "$QEMU_LANCEUR" "$@" ;;
        toggle|bascule) shift; exec "$QEMU_LANCEUR" toggle "$@" ;;
        stop|down) exec "$QEMU_LANCEUR" stop ;;
        status)    exec "$QEMU_LANCEUR" status ;;
    esac
fi
case "${1:-status}" in
    install)   pmos_install ;;
    # "up" = the icon action: install if the image is missing, then power on.
    # The image (1.5 GB to download) is NOT in the ISO - it would fit poorly
    # in a squashfs that must live in RAM - so the first launch is long,
    # once. The packages are already in the image
    # (tools/osmo-extras-install.sh installs them at build).
    up)
        # The "[n/3]" markers printed by the functions make the progress bar
        # of the launch window (osmo-gtk-run.py --progress).
        if [ ! -f "$IMG" ]; then
            _say "premier lancement : il faut d abord poser l image postmarketOS"
            pmos_install || exit 1
        fi
        pmos_start
        ;;
    fetch)     pmos_fetch ;;
    provision) pmos_provision ;;
    start)     pmos_start ;;
    stop|down) pmos_stop ;;
    status)    pmos_status ;;
    shell)     shift; pmos_shell "$@" ;;
    mm)        pmos_mm ;;
    at)        shift; exec /usr/bin/python3 "$REPO/tools/at-cmd.py" "$@" ;;
    log)       exec tail -f /var/log/osmo-at-ofono.log ;;
    *)
        sed -n '/^#   osmo-pmos/,/^#   osmo-pmos log/p' "$0" | sed 's/^# \{0,2\}//'
        exit 2
        ;;
esac
```

2.273 /opt/GSM/osmo-operator/tools/osmo-reglages.py

20790 octets, 504 lignes → 504 lignes

```
#!/usr/bin/env python3
# Web page to configure the Osmocom stack of the bench.
#
#   python3 tools/osmo-reglages.py [--port 8090] [--bind 127.0.0.1]
#   puis http://127.0.0.1:8090/
#
# The page is in web/ (index.html, style.css, app.js); this script serves it and provides
# the /api/... API. Four tabs:
#   globals.conf : form on the model of globals.conf.example (tools/globals-complet.py:
#                  network settings + all launcher variables), values read from and
#                  written back to globals.conf
#   Demons       : running-config of each daemon read through the VTY, each line editable
#                  live (configure terminal + node path + line), "no", add, write memory,
#                  free VTY console
#   Fichiers     : raw editing of configs/*.cfg and /etc/osmocom/*.cfg (backup)
#   Etat         : osmo-* processes, ports, useful counters
# Nothing is restarted: VTY changes are immediate, globals.conf and the files are
# used by the next run. Listens on 127.0.0.1 only: the VTY console executes anything.
# Standard library only.

import argparse, glob, html, json, os, re, shutil, socket, subprocess, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs

RACINE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
GLOBALS = os.path.join(RACINE, "globals.conf")
EXEMPLE = GLOBALS + ".example"
WEB = os.path.join(RACINE, "web")
TYPES = {".html": "text/html", ".css": "text/css", ".js": "application/javascript", ".svg": "image/svg+xml",
 ↪  ".png": "image/png"}
DOSSIERS_CFG = [os.path.join(RACINE, "configs"), "/etc/osmocom", "/root/.osmocom/bb"]
PROMPT = re.compile(rb"[\r\n][^\r\n]*[>#] ?$")


# ── VTY ──────────────────────────────────────────────────────────

class Vty:
    def __init__(self, hote, port, delai=4.0):
        self.s = socket.create_connection((hote, port), 2)
        self.s.settimeout(delai)
        self.lire()
        self.cmd("enable")

    def lire(self):
        b = b""
        while True:
            try:
                d = self.s.recv(65536)
            except socket.timeout:
                break
            if not d:
                break
            b += d
            b = re.sub(rb"\xff[\xfb-\xfe].", b"", b)  # negociation telnet (IAC WILL/WONT/DO/DONT)
            if PROMPT.search(b):
                break
        return b.decode("utf-8", "replace")

    def cmd(self, ligne):
        self.s.sendall(ligne.encode() + b"\r\n")
        r = self.lire().replace("\r", "")
                # strip the echo and the final prompt
```

```
        l = r.split("\n")
        if l and l[0].strip() == ligne.strip():
            l = l[1:]
        if l and re.search(r"[>#] ?$", l[-1]):
            l = l[:-1]
        return "\n".join(l)

    def fermer(self):
        try:
            self.s.sendall(b"exit\r\n")
            self.s.close()
        except OSError:
            pass


_cache_vty = {"t": 0, "liste": []}


def demons_vty():
    """Listening ports of the osmo-* and of mobile, probed: the VTY answers with a banner."""
    if time.time() - _cache_vty["t"] < 15:
        return _cache_vty["liste"]
    out = subprocess.run(["ss", "-ltnpH"], capture_output=True, text=True).stdout
    vus, liste = set(), []
    for l in out.splitlines():
        m = re.search(r"\s(\d+\.\d+\.\d+\.\d+):(\d+)\s.*users:\(\(\"([^\"]+)\",pid=(\d+)", l)
        if not m:
            continue
        hote, port, nom, pid = m.group(1), int(m.group(2)), m.group(3), m.group(4)
        if not (nom.startswith("osmo") or nom == "mobile") or (hote, port) in vus:
            continue
        vus.add((hote, port))
        try:
            s = socket.create_connection((hote, port), 0.5)
            s.settimeout(0.6)
            b = b""
            try:
                while len(b) < 4096:
                    d = s.recv(4096)
                    if not d:
                        break
                    b += d
                    if PROMPT.search(b):
                        break
            except socket.timeout:
                pass
            s.close()
        except OSError:
            continue
        t = b.decode("latin-1", "replace")
        if "VTY" not in t and not re.search(r"[>#] ?$", t):
            continue  # port CTRL / GSUP / autre
        invite = re.findall(r"([\w.-]+)[>#] ?$", t)
        liste.append({"hote": hote, "port": port, "nom": nom, "pid": pid,
                      "invite": invite[-1] if invite else nom})
    liste.sort(key=lambda d: (d["nom"], d["hote"], d["port"]))
    _cache_vty.update(t=time.time(), liste=liste)
    return liste


def arbre_config(texte):
    """running-config -> list of lines {txt, prof, chemin[], noeud}."""
    lignes = []
    for brut in texte.split("\n"):
        if not brut.strip() or brut.strip() == "!" or brut.startswith("Current configuration"):
            continue
```

```
        prof = len(brut) - len(brut.lstrip(" "))
        lignes.append([prof, brut.strip()])
    sortie, pile = [], []
    for i, (prof, txt) in enumerate(lignes):
        while pile and pile[-1][0] >= prof:
            pile.pop()
        noeud = i + 1 < len(lignes) and lignes[i + 1][0] > prof
        sortie.append({"txt": txt, "prof": prof, "chemin": [p[1] for p in pile], "noeud": noeud})
        if noeud:
            pile.append((prof, txt))
    return sortie


def appliquer(hote, port, chemin, lignes):
    v = Vty(hote, port)
    journal = []
    try:
        journal.append(("configure terminal", v.cmd("configure terminal")))
        for c in chemin:
            journal.append((c, v.cmd(c)))
        for l in lignes:
            journal.append((l, v.cmd(l)))
        journal.append(("end", v.cmd("end")))
    finally:
        v.fermer()
    erreur = any(r.lstrip().startswith("%") for _, r in journal)
    return {"ok": not erreur,
            "journal": "\n".join("> " + c + ("\n" + r if r.strip() else "") for c, r in journal)}


def console(hote, port, lignes):
    v = Vty(hote, port, delai=6.0)
    try:
        out = []
        for l in lignes:
            if l.strip():
                out.append("> " + l + "\n" + v.cmd(l))
        return "\n".join(out)
    finally:
        v.fermer()


# ── globals.conf ──────────────────────────────────────────

LIGNE_VAR = re.compile(r"^([A-Z_][A-Z0-9_]*)=(.*)$")


def _valeur(brut):
    v = brut.strip()
    if len(v) >= 2 and v[0] == v[-1] and v[0] in "\"'":
        v = v[1:-1]
    return v


def _valeurs(chemin):
    r = {}
    if os.path.exists(chemin):
        for l in open(chemin, encoding="utf-8"):
            m = LIGNE_VAR.match(l.rstrip("\n"))
            if m:
                r[m.group(1)] = _valeur(m.group(2))
    return r


def _patron():
    return EXEMPLE if os.path.exists(EXEMPLE) else GLOBALS
```

```
def lire_globals(patron=None, actuelles=None):
    """The template fields (order, section, comment, default, choices) with the current values."""
    if actuelles is None:
        actuelles = _valeurs(GLOBALS)
    champs, section, doc = [], "", []
    for l in open(patron or _patron(), encoding="utf-8"):
        l = l.rstrip("\n")
        m = re.match(r"#\s*──\s*(.+?)\s*─", l)
        if m:
            section, doc = m.group(1), []
            continue
        if l.startswith("#"):
            if section:
                doc.append(l.lstrip("# ").rstrip())
            continue
        m = LIGNE_VAR.match(l)
        if m:
            cle = m.group(1)
            defaut, choix = "", []
            for d in doc:
                md = re.match(r"defaut : (.*?)\s+\(\S+:\d+\)$", d)
                if md:
                    defaut = "" if md.group(1) == "(vide)" else md.group(1)
                mc = re.match(r"choix : (.*)$", d)
                if mc:
                    choix = [c.strip() for c in mc.group(1).split("|") if c.strip()]
            texte = " ".join(d for d in doc if not d.startswith("defaut :") and not d.startswith("choix :"))
            val = actuelles.get(cle, _valeur(m.group(2)))
            champs.append({"cle": cle, "val": val, "section": section, "doc": texte, "defaut": defaut, "choix":
↪   choix})
        doc = []
        # file variables absent from the template: kept, at the end of the list
    connues = {c["cle"] for c in champs}
    for cle, val in actuelles.items():
        if cle not in connues:
           champs.append({"cle": cle, "val": val, "section": "Hors patron", "doc": "", "defaut": "", "choix": []})
    return champs


# ── instances (c54x_exe/run.sh, one file per Calypso) ────────

INSTANCES = os.path.join(RACINE, "instances")
PATRON_INST = os.path.join(INSTANCES, "instance.env.example")
ROLES = {1: "Instance 1 — BTS : TRX Calypso (--bts) ou mobile sur le pont (--bts-classique)",
         2: "Instance 2 — MS QEMU par l air (--bts --ms / --ms-magnetite)",
         3: "Instance 3 — 2e TRX Calypso, TS1 (--ts 2)"}
LIGNE_INST = re.compile(r'^: "\$\{([A-Z_][A-Z0-9_]*):=(.*)\}"$')


def _fichier_instance(n):
    return os.path.join(INSTANCES, "instance-%d.env" % n)


def _valeurs_instance(n):
    r = {}
    chemin = _fichier_instance(n)
    if os.path.exists(chemin):
        for l in open(chemin, encoding="utf-8"):
            m = LIGNE_INST.match(l.rstrip("\n"))
            if m:
                r[m.group(1)] = m.group(2)
    return r
```

```
def lire_instance(n):
    if not os.path.exists(PATRON_INST):
        return []
    return lire_globals(PATRON_INST, _valeurs_instance(n))


def ecrire_instance(n, nouvelles):
    """instance-N.env on the model of the template: `: "${VAR:=value}"` (the environment keeps priority)."""
    valeurs = _valeurs_instance(n)
    valeurs.update({k: str(v).replace('"', "").replace("\n", " ") for k, v in nouvelles.items()})
    sortie = ["# instances/instance-%d.env — %s\n" % (n, ROLES.get(n, "")),
              "# ecrit par la page web (tools/osmo-reglages.py) ; lu par c54x_exe/run.sh (INSTANCE=%d)\n" % n]
    vues = set()
    for l in open(PATRON_INST, encoding="utf-8"):
        m = LIGNE_VAR.match(l.rstrip("\n"))
        if m:
            cle = m.group(1)
            val = valeurs.get(cle, "")
            vues.add(cle)
            sortie.append(('#%s\n' % cle) if val == "" else ': "${%s:=%s}"\n' % (cle, val))
        elif not l.startswith("# instances/instance.env.example") and not l.startswith("# Genere par"):
            sortie.append(l if l.endswith("\n") else l + "\n")
    for k in valeurs:
        if k not in vues and valeurs[k] != "":
            sortie.append(': "${%s:=%s}"\n' % (k, valeurs[k]))
    os.makedirs(INSTANCES, exist_ok=True)
    sauvegarde(_fichier_instance(n))
    with open(_fichier_instance(n), "w", encoding="utf-8") as f:
        f.writelines(sortie)


def sauvegarde(chemin):
    if os.path.exists(chemin):
        shutil.copy2(chemin, chemin + ".bak-" + time.strftime("%Y%m%d-%H%M%S"))


def ecrire_globals(nouvelles):
    """Rewrites globals.conf on the model of the template: same lines and comments, current + new values."""
    valeurs = _valeurs(GLOBALS)
    valeurs.update({k: str(v).replace('"', "").replace("\n", " ") for k, v in nouvelles.items()})
        # in the launcher block (markers of tools/globals-complet.py), an EMPTY variable is
        # written as a comment: undefined. Exported empty by `set -a`, the launcher variables
        # would break the launch. Only the entered values become active.
    sortie, vues, bloc = [], set(), False
    for l in open(_patron(), encoding="utf-8"):
        if l.startswith("# >>> VARIABLES DU LANCEUR"):
            bloc = True
        elif l.startswith("# <<< FIN DES VARIABLES DU LANCEUR"):
            bloc = False
        m = LIGNE_VAR.match(l.rstrip("\n"))
        if m:
            cle = m.group(1)
            val = valeurs.get(cle, "")
            vues.add(cle)
            sortie.append(('#%s=""\n' % cle) if (bloc and val == "") else '%s="%s"\n' % (cle, val))
        else:
            sortie.append(l if l.endswith("\n") else l + "\n")
    reste = [k for k in valeurs if k not in vues]
    if reste:
        sortie.append("\n# ── Hors patron (gardees de l ancien globals.conf) ──────────\n")
        sortie += ['%s="%s"\n' % (k, valeurs[k]) for k in reste]
    sauvegarde(GLOBALS)
    with open(GLOBALS, "w", encoding="utf-8") as f:
        f.writelines(sortie)
```

```
# ── Fichiers ──────────────────────────────────────────

def fichiers_cfg():
    r = []
    for d in DOSSIERS_CFG:
        for p in sorted(glob.glob(os.path.join(d, "*.cfg")) + glob.glob(os.path.join(d, "*.conf"))):
            r.append(p)
    return r


def fichier_autorise(p):
    p = os.path.realpath(p)
    return p in [os.path.realpath(x) for x in fichiers_cfg()]


# "Configs osmo" tab: each /etc/osmocom/osmo-*.cfg, its template in configs/ (start.sh
# regenerates /etc/osmocom from configs/ on every run: an edit of /etc only lives until
# then) and the daemon currently reading it (-c <file> in its command line).
PROTEGES = "/etc/osmocom/.proteges"   # lib/gabarits.sh install_configs_native n'ecrase pas ces fichiers


def proteges():
    try:
        with open(PROTEGES, encoding="utf-8") as f:
            return {l.split("#")[0].strip() for l in f} - {""}
    except FileNotFoundError:
        return set()


def proteger(nom, on):
    if "/" in nom or not nom:
        raise ValueError("nom invalide")
    if (nom in proteges()) == on:
        return
    lignes = []
    if os.path.exists(PROTEGES):
        with open(PROTEGES, encoding="utf-8") as f:
            lignes = [l for l in f if l.split("#")[0].strip() != nom]
    else:
        lignes = ["# Fichiers de /etc/osmocom que start.sh ne regenere pas depuis configs/ (un par ligne).\n"]
    if on:
        lignes.append(nom + "\n")
    with open(PROTEGES, "w", encoding="utf-8") as f:
        f.writelines(lignes)


def configs_osmo():
    cmds = subprocess.run(["ps", "-eo", "pid,args"], capture_output=True, text=True).stdout.splitlines()[1:]
    prot = proteges()
    r = []
    for p in sorted(glob.glob("/etc/osmocom/osmo-*.cfg")):
        nom = os.path.basename(p)
        gab = os.path.join(RACINE, "configs", nom)
        st = os.stat(p)
        pid = next((c.split(None, 1)[0] for c in cmds
                    if re.search(r"(^|\s)-c\s+" + re.escape(p) + r"(\s|$)", c.split(None, 1)[-1])), None)
        r.append({"nom": nom[:-4], "etc": p, "gabarit": gab if os.path.exists(gab) else None,
                  "pid": pid, "taille": st.st_size, "mtime": int(st.st_mtime), "protege": nom in prot})
    return r


# ── Etat ──────────────────────────────────────────────

def etat():
    ps = subprocess.run(["ps", "-eo", "pid,etimes,rss,pcpu,args", "--sort=args"],
                        capture_output=True, text=True).stdout.splitlines()
```

```
    procs = []
    for l in ps[1:]:
        c = l.split(None, 4)
        if len(c) == 5 and re.search(r"(^|/)(osmo-[\w-]+|mobile|qemu-system-arm|c54x_exe)\b", c[4]) \
                and "osmo-reglages" not in c[4]:
            procs.append({"pid": c[0], "age": int(c[1]), "rss": int(c[2]) // 1024,
                          "cpu": c[3], "cmd": c[4][:160]})
    return {"procs": procs, "vty": demons_vty()}


# ── HTTP ──────────────────────────────────────────────────────

class H(BaseHTTPRequestHandler):
    def log_message(self, *a):
        pass

    def envoyer(self, code, corps, type_="application/json"):
        if not isinstance(corps, bytes):
            corps = (json.dumps(corps) if type_ == "application/json" else corps).encode()
        self.send_response(code)
        self.send_header("Content-Type", type_ + "; charset=utf-8")
        self.send_header("Cache-Control", "no-store")
        self.send_header("Content-Length", str(len(corps)))
        self.end_headers()
        self.wfile.write(corps)

    def corps_json(self):
        n = int(self.headers.get("Content-Length") or 0)
        return json.loads(self.rfile.read(n) or b"{}")

    def do_GET(self):
        u = urlparse(self.path)
        q = {k: v[0] for k, v in parse_qs(u.query).items()}
        try:
            if u.path == "/" or not u.path.startswith("/api/"):
                nom = "index.html" if u.path == "/" else u.path.lstrip("/")
                chemin = os.path.realpath(os.path.join(WEB, nom))
                if not chemin.startswith(os.path.realpath(WEB) + os.sep) or not os.path.isfile(chemin):
                    return self.envoyer(404, {"err": "introuvable"})
                with open(chemin, "rb") as f:
                    return self.envoyer(200, f.read(), TYPES.get(os.path.splitext(chemin)[1],
                     ↪  "application/octet-stream"))
            if u.path == "/api/globals":
                return self.envoyer(200, lire_globals())
            if u.path == "/api/instances":
                return self.envoyer(200, [{"n": n, "role": r, "fichier": _fichier_instance(n),
                                           "existe": os.path.exists(_fichier_instance(n))} for n, r in ROLES.items()])
            if u.path == "/api/instance":
                return self.envoyer(200, lire_instance(int(q.get("n", "1"))))
            if u.path == "/api/demons":
                if q.get("rafraichir"):
                    _cache_vty["t"] = 0
                return self.envoyer(200, demons_vty())
            if u.path == "/api/config":
                v = Vty(q["hote"], int(q["port"]), delai=6.0)
                try:
                    t = v.cmd("show running-config")
                finally:
                    v.fermer()
                return self.envoyer(200, {"lignes": arbre_config(t), "brut": t})
            if u.path == "/api/fichiers":
                return self.envoyer(200, fichiers_cfg())
            if u.path == "/api/configs-osmo":
                return self.envoyer(200, configs_osmo())
            if u.path == "/api/fichier":
                if not fichier_autorise(q["p"]):
```

```
                    return self.envoyer(403, {"err": "fichier hors liste"})
                with open(q["p"], encoding="utf-8", errors="replace") as f:
                    return self.envoyer(200, {"texte": f.read()})
            if u.path == "/api/etat":
                return self.envoyer(200, etat())
            self.envoyer(404, {"err": "?"})
        except Exception as e:
            self.envoyer(500, {"err": "%s: %s" % (type(e).__name__, e)})

    def do_POST(self):
        u = urlparse(self.path)
        try:
            j = self.corps_json()
            if u.path == "/api/globals":
                ecrire_globals(j)
                return self.envoyer(200, {"ok": True})
            if u.path == "/api/instance":
                ecrire_instance(int(j["n"]), j["valeurs"])
                return self.envoyer(200, {"ok": True, "fichier": _fichier_instance(int(j["n"]))})
            if u.path == "/api/appliquer":
                return self.envoyer(200, appliquer(j["hote"], int(j["port"]), j["chemin"], j["lignes"]))
            if u.path == "/api/console":
                return self.envoyer(200, {"sortie": console(j["hote"], int(j["port"]), j["lignes"])})
            if u.path == "/api/fichier":
                if not fichier_autorise(j["p"]):
                    return self.envoyer(403, {"err": "fichier hors liste"})
                sauvegarde(j["p"])
                with open(j["p"], "w", encoding="utf-8") as f:
                    f.write(j["texte"])
                                                        # edited here: kept across the regeneration
                protege = os.path.dirname(os.path.realpath(j["p"])) == "/etc/osmocom"
                if protege:
                    proteger(os.path.basename(j["p"]), True)
                return self.envoyer(200, {"ok": True, "protege": protege})
            if u.path == "/api/protege":
                proteger(os.path.basename(j["nom"]), bool(j["on"]))
                return self.envoyer(200, {"ok": True})
            self.envoyer(404, {"err": "?"})
        except Exception as e:
            self.envoyer(500, {"err": "%s: %s" % (type(e).__name__, e)})


if __name__ == "__main__":
    a = argparse.ArgumentParser(description="Page web de reglage Osmocom")
    a.add_argument("--port", type=int, default=8090)
    a.add_argument("--bind", default="127.0.0.1")
    o = a.parse_args()
    print("Reglages Osmocom : http://%s:%d/" % (o.bind, o.port))
    ThreadingHTTPServer((o.bind, o.port), H).serve_forever()
```

2.274 /opt/GSM/osmo-operator/tools/osmo-ril-atmodem.py

19424 octets, 486 lignes → 486 lignes

```
#!/usr/bin/env python3
# osmo-ril-atmodem.py - THE VIRTUAL AT MODEM THAT PLUGS oFono ON THE RIL.
#
# THE CHAIN, AND THE GAP FILLED HERE.
#
#   Android -> rild -> libreference-ril.so -> AT serial port -> THIS PROGRAM -> oFono -> bench
#
# An Android RIL (libreference-ril.so) only opens a serial port and sends
# 3GPP 27.007 on it (AT+CFUN, AT+CREG?, ATD..., AT+CMGS...). oFono is not a
# modem: it is a modem consumer, with a D-Bus API. The two never meet -
```

```
# unless someone PLAYS THE MODEM ROLE. That is what this program does: it
# opens a pseudo-terminal, behaves like a 27.007 modem seen from the port,
# and executes each command on oFono.
#
#   AT+CREG?        <- the registration state read on org.ofono.NetworkRegistration
#   ATD100102;      <- org.ofono.VoiceCallManager.Dial
#   ATA / ATH       <- Answer / HangupAll
#   AT+CLCC         <- the list of current calls (GetCalls)
#   AT+CMGS         <- org.ofono.MessageManager.SendMessage
#   +CMTI / RING    <- pushed to the port when oFono signals an SMS or a call
#
# WHAT IT IS FOR. The phone of the bench - the postmarketOS VM - plugs
# DIRECTLY on the modem of the bench (tools/osmo-phonesim-banc.py --connect).
# This program is the AT modem backed by oFono: through it, with
# tools/at-cmd.py on /run/osmo-ril/at-pty, what oFono really answers is
# checked by hand - and it is the port to give to any software expecting
# a modem.
#
# Settings:
#   OSMO_RIL_DIR      where to publish the link to the pty (default /run/osmo-ril)
#   OSMO_OFONO_MODEM  the oFono modem to drive (default: the first found)
#   OSMO_AT_ECHO      1 to echo from the start (ATE1)
import os
import pty
import re
import sys
import termios
import time
import tty

from gi.repository import Gio, GLib

RUN = os.environ.get("OSMO_RIL_DIR", "/run/osmo-ril")
LINK = os.path.join(RUN, "at-pty")
MODEM = os.environ.get("OSMO_OFONO_MODEM", "")
# THE LOG OF THE AT PORT. Everything passing between the RIL and this modem,
# in both directions, timestamped to the millisecond. It is the tool to see
# WHERE a sequence stops - a command answered badly silently blocks all the
# rest. "tail -f /var/log/osmo-at.log" while driving the client.
AT_LOG = os.environ.get("OSMO_AT_LOG", "/var/log/osmo-at.log")

OFONO = "org.ofono"
IF_MANAGER = "org.ofono.Manager"
IF_MODEM = "org.ofono.Modem"
IF_SMS = "org.ofono.MessageManager"
IF_VOICE = "org.ofono.VoiceCallManager"
IF_CALL = "org.ofono.VoiceCall"
IF_NETREG = "org.ofono.NetworkRegistration"
IF_SIM = "org.ofono.SimManager"


def log(*a):
    print("[at]", *a, flush=True)


_atlog = None


def atlog(sens, texte):
    """sens: '<' what the RIL sends, '>' what the modem answers."""
    global _atlog
    if _atlog is None:
        try:
            _atlog = open(AT_LOG, "a", buffering=1)   # line by line: direct
            os.chmod(AT_LOG, 0o644)                   # readable without being root
        except OSError as e:
```

```
            print("[at] journal %s impossible : %s" % (AT_LOG, e), file=sys.stderr)
            _atlog = False
    if not _atlog:
        return
    stamp = time.strftime("%H:%M:%S") + ".%03d" % (int(time.time() * 1000) % 1000)
    for ligne in texte.replace("\r", "\n").split("\n"):
        if ligne.strip():
            try:
                _atlog.write("%s %s %s\n" % (stamp, sens, ligne.strip()))
            except OSError:
                return


# ── oFONO ──────────────────────────────────────────────────────────────
class Ofono:
    """The bare minimum of oFono, in Gio (no dbus-python on the image)."""

    def __init__(self):
        self.c = Gio.bus_get_sync(Gio.BusType.SYSTEM, None)
        self.modem = MODEM or self._first_modem()

    def _first_modem(self):
        try:
            modems = self.call("/", IF_MANAGER, "GetModems").unpack()[0]
        except GLib.Error:
            return ""
        return modems[0][0] if modems else ""

    def call(self, path, iface, method, params=None, timeout=20000):
        return self.c.call_sync(OFONO, path, iface, method, params, None,
                                Gio.DBusCallFlags.NONE, timeout, None)

    def props(self, iface, path=None):
        try:
            return self.call(path or self.modem, iface, "GetProperties").unpack()[0]
        except GLib.Error:
            return {}

    def calls(self):
        try:
            return self.call(self.modem, IF_VOICE, "GetCalls").unpack()[0]
        except GLib.Error:
            return []

    def subscribe(self, iface, signal, cb, path=None):
        return self.c.signal_subscribe(OFONO, iface, signal, path, None,
                                       Gio.DBusSignalFlags.NONE, cb)


# ── THE MODEM SEEN FROM THE SERIAL PORT ────────────────────────────────
# The 27.007 states expected by the RIL. What oFono says is translated.
_CREG_STATE = {"unregistered": 0, "registered": 1, "searching": 2,
               "denied": 3, "unknown": 4, "roaming": 5}


class AtModem:
    def __init__(self, ofono, fd):
        self.o = ofono
        self.fd = fd
        self.buf = b""
        self.echo = os.environ.get("OSMO_AT_ECHO") == "1"
        self.cmee = 0
        self.creg_mode = 0          # 0 = no URC, 1/2 = registration URC
        self.cmgf = 0               # 0 = PDU (what the RIL wants), 1 = text
        self.sms_target = None      # number pending between AT+CMGS and the ^Z
```

```
# -- writing to the RIL
def send(self, text):
    atlog(">", text)
    try:
        os.write(self.fd, ("\r\n%s\r\n" % text).encode())
    except OSError as e:
        log("ecriture impossible :", e)

def ok(self):
    self.send("OK")

def err(self, code=100):
    self.send("+CME ERROR: %d" % code if self.cmee else "ERROR")

# -- reading from the RIL
def feed(self, data):
    if self.echo:
        try:
            os.write(self.fd, data)
        except OSError:
            pass
    self.buf += data
    while True:
        m = re.search(rb"[\r\n\x1a]", self.buf)
        if not m:
            return
        line, self.buf = self.buf[:m.start()], self.buf[m.end():]
        end = self.buf[m.start():m.end()] if False else m.group()
        text = line.decode(errors="replace").strip()
        if text:
            atlog("<", text)
        if end == b"\x1a" or self.sms_target is not None:
            self.on_sms_body(text, submitted=(end == b"\x1a"))
        elif text:
            self.on_line(text)

# -- the body of an SMS, between AT+CMGS=... and Ctrl-Z
def on_sms_body(self, text, submitted):
    if not submitted:
        return                      # still waiting for the Ctrl-Z
    num, self.sms_target = self.sms_target, None
    if not num:
        self.err()
        return
    try:
        self.o.call(self.o.modem, IF_SMS, "SendMessage",
                    GLib.Variant("(ss)", (num, text)))
        self.send("+CMGS: 1")
        self.ok()
        log("SMS envoye a %s" % num)
    except GLib.Error as e:
        log("envoi refuse :", e.message.split(":")[-1].strip())
        self.err()

# -- an AT command
def on_line(self, line):
    cmd = line.upper()
    if not cmd.startswith("AT"):
        return
    body = line[2:]
    ub = cmd[2:]

    # the basics the RIL sends at startup
    if ub in ("", "&F", "Z", "E0", "E1", "+CMEE=1", "+CMEE=2", "V1", "Q0",
              "S0=0", "+CSCS=\"UCS2\"", "+CSCS=\"IRA\"", "+CMGF=0",
              "+CNMI=1,2,0,1,0", "+CGEREP=1,0", "+CLIP=1", "+COLP=0",
```

```
          "+CSSN=0,0", "+CSNS=0", "+CMOD=0"):
    if ub == "E0":
        self.echo = False
    elif ub == "E1":
        self.echo = True
    elif ub.startswith("+CMEE"):
        self.cmee = 1
    elif ub == "+CMGF=0":
        self.cmgf = 0
    self.ok()
    return

# modem identity: what the RIL shows in "about"
if ub in ("+CGMI", "+GMI"):
    self.send("Osmocom"); self.ok(); return
if ub in ("+CGMM", "+GMM"):
    self.send("osmo-ril-atmodem"); self.ok(); return
if ub in ("+CGMR", "+GMR"):
    self.send("1.0"); self.ok(); return
if ub in ("+CGSN", "+GSN"):
    self.send(self.o.props(IF_MODEM).get("Serial", "000000000000000"))
    self.ok(); return

# the SIM
if ub == "+CPIN?":
    sim = self.o.props(IF_SIM)
    self.send("+CPIN: READY" if sim.get("Present", True) else "+CME ERROR: 10")
    self.ok(); return
if ub == "+CIMI":
    self.send(self.o.props(IF_SIM).get("SubscriberIdentity", "001010000000000"))
    self.ok(); return
if ub.startswith("+CRSM") or ub.startswith("+CCID") or ub.startswith("+CSIM"):
    self.err(); return       # the RIL can do without it

# the radio
if ub.startswith("+CFUN"):
    if ub.endswith("?"):
        on = self.o.props(IF_MODEM).get("Online", False)
        self.send("+CFUN: %d" % (1 if on else 4))
        self.ok(); return
    want = ub.rstrip().endswith("1")
    try:
        self.o.call(self.o.modem, IF_MODEM, "SetProperty",
                    GLib.Variant("(sv)", ("Online", GLib.Variant("b", want))))
    except GLib.Error as e:
        log("CFUN refuse :", e.message.split(":")[-1].strip())
    self.ok(); return

# network registration
if ub.startswith("+CREG") or ub.startswith("+CGREG") or ub.startswith("+CEREG"):
    tag = "+CREG" if ub.startswith("+CREG") else ("+CGREG" if ub.startswith("+CGREG") else "+CEREG")
    if "=" in ub:
        try:
            self.creg_mode = int(ub.split("=")[1][0])
        except (ValueError, IndexError):
            self.creg_mode = 0
        self.ok(); return
    net = self.o.props(IF_NETREG)
    st = _CREG_STATE.get(str(net.get("Status", "unknown")), 4)
    lac = int(net.get("LocationAreaCode", 0) or 0)
    cid = int(net.get("CellId", 0) or 0)
    if self.creg_mode >= 2:
        self.send("%s: %d,%d,\"%04X\",\"%08X\"" % (tag, self.creg_mode, st, lac, cid))
    else:
        self.send("%s: %d,%d" % (tag, self.creg_mode, st))
    self.ok(); return
```

```
if ub.startswith("+COPS"):
    net = self.o.props(IF_NETREG)
    if ub.endswith("?"):
        self.send("+COPS: 0,0,\"%s\",0" % net.get("Name", "Osmocom"))
    self.ok(); return

if ub.startswith("+CSQ"):
    # oFono gives a strength in %, 27.007 expects 0..31
    pct = int(net_strength(self.o))
    self.send("+CSQ: %d,99" % min(31, round(pct * 31 / 100)))
    self.ok(); return

# calls
if cmd.startswith("ATD") and (cmd.endswith(";") or ";" in cmd):
    num = re.sub(r"[^0-9+*#]", "", body.split(";")[0][1:])
    try:
        self.o.call(self.o.modem, IF_VOICE, "Dial",
                    GLib.Variant("(ss)", (num, "default")))
        self.ok()
        log("appel vers %s" % num)
    except GLib.Error as e:
        log("appel refuse :", e.message.split(":")[-1].strip())
        self.err()
    return
if ub == "A":
    for path, props in self.o.calls():
        if props.get("State") == "incoming":
            try:
                self.o.call(path, IF_CALL, "Answer")
            except GLib.Error:
                pass
    self.ok(); return
if ub in ("H", "+CHUP") or ub.startswith("+CHLD"):
    try:
        self.o.call(self.o.modem, IF_VOICE, "HangupAll")
    except GLib.Error:
        pass
    self.ok(); return
if ub == "+CLCC":
    for i, (path, props) in enumerate(self.o.calls(), start=1):
        st = {"active": 0, "held": 1, "dialing": 2, "alerting": 3,
              "incoming": 4, "waiting": 5}.get(props.get("State", ""), 6)
        direction = 1 if props.get("State") in ("incoming", "waiting") else 0
        num = props.get("LineIdentification", "")
        self.send("+CLCC: %d,%d,%d,0,0,\"%s\",129" % (i, direction, st, num))
    self.ok(); return

# SMS
if ub.startswith("+CMGS="):
    # In text mode the number is between quotes; in PDU the RIL sends a
    # length, and the number is in the PDU - not handled here (the oFono
    # bridge takes care of it as long as there is no rild).
    m = re.search(r'"([^"]+)"', body)
    if m:
        self.sms_target = m.group(1)
        try:
            os.write(self.fd, b"\r\n> ")
        except OSError:
            pass
        return
    self.err(); return
if ub.startswith("+CMGF"):
    self.cmgf = 1 if ub.rstrip().endswith("1") else 0
    self.ok(); return
if ub.startswith("+CSCA"):
```

```
            self.send("+CSCA: \"\",129"); self.ok(); return

        # ── WHAT THE ANDROID RIL ASKS AT INITIALISATION ──────────────────
        # Sequence sent by libreference-ril when it opens this port: ATE0Q0V1,
        # AT+CTEC?, AT+WNAM, AT+CCWA=1, AT+CMUT=0, AT+CSSN=0,1, AT+CSCS="HEX",
        # AT+CUSD=1... If these are refused, libreference-ril stays blocked at the
        # very beginning of its sequence and never publishes its interface - the
        # framework then sees no radio service.
        if ub in ("E0Q0V1", "E1Q0V1", "Q0V1", "E0V1"):
            self.echo = ub.startswith("E1")
            self.ok(); return
        if ub.startswith("+CTEC"):
            # "current technology" of the reference-ril: 0 = GSM, what the bench
            # actually does.
            self.send("+CTEC: 0,1" if ub.endswith("?") else "+CTEC: 0")
            self.ok(); return
        if ub.startswith("+WNAM"):
            self.send('+WNAM: "%s"' % (self.o.props(IF_NETREG).get("Name") or "Osmocom"))
            self.ok(); return
        if ub.startswith("+CSCS"):
            if ub.endswith("=?"):
                self.send('+CSCS: ("IRA","GSM","UCS2","HEX")')
            elif ub.endswith("?"):
                self.send('+CSCS: "HEX"')
            self.ok(); return

        # Same rule as the modem on the oFono side (tools/osmo-phonesim-banc.py):
        # an ASSIGNMENT is accepted - a modem that accepts a setting without
        # applying it tells nothing false; an unknown READ is refused, because
        # an invented answer would suggest a capability that does not exist.
        if "=" in ub and not ub.endswith("=?"):
            log("reglage accepte sans effet : %s" % line)
            self.ok(); return

        log("lecture non geree (refusee) : %s" % line)
        self.err()

    # ── oFono events pushed to the RIL ──────────────────────────────────
    def on_incoming_sms(self, _c, _s, _p, _i, _sig, params):
        info = params[1]
        log("SMS entrant de %s" % info.get("Sender", "?"))
        self.send("+CMTI: \"SM\",1")

    def on_call_added(self, _c, _s, _p, _i, _sig, params):
        props = params[1]
        if props.get("State") == "incoming":
            num = props.get("LineIdentification", "")
            self.send("+CLIP: \"%s\",129" % num)
            self.send("RING")

    def on_netreg_changed(self, _c, _s, _p, _i, _sig, params):
        if self.creg_mode and params[0] in ("Status", "CellId", "LocationAreaCode"):
            net = self.o.props(IF_NETREG)
            st = _CREG_STATE.get(str(net.get("Status", "unknown")), 4)
            self.send("+CREG: %d" % st)


# ── THE TEST MODEM ──────────────────────────────────────────────────────
# Without an oFono modem (bench stopped, EC25 unplugged), the program would
# have nothing to drive and would exit at once - impossible to tune the AT
# dialogue or to check what the RIL will receive. --essai therefore replaces
# oFono with default answers: the modem answers like a switched-off but polite
# modem (CFUN 0, CREG 0, no call), and EVERY action command is logged instead
# of being executed. It is a test bench for the mechanics, never a substitute
# for the real path.
class OfonoEssai:
```

```
    modem = "/essai/modem0"

    def props(self, iface, path=None):
        if iface == IF_SIM:
            return {"Present": True, "SubscriberIdentity": "001010000000001"}
        if iface == IF_NETREG:
            return {"Status": "unregistered", "Name": "Osmocom", "Strength": 0}
        if iface == IF_MODEM:
            return {"Online": False, "Serial": "000000000000001"}
        return {}

    def calls(self):
        return []

    def call(self, path, iface, method, params=None, timeout=20000):
        log("[essai] %s.%s ignore" % (iface.rsplit(".", 1)[-1], method))
        return GLib.Variant("()", ())

    def subscribe(self, *a, **kw):
        return 0


def net_strength(o):
    return o.props(IF_NETREG).get("Strength", 0) or 0


def main():
    essai = "--essai" in sys.argv or os.environ.get("OSMO_AT_ESSAI") == "1"
    o = None
    if not essai:
        try:
            o = Ofono()
        except GLib.Error as e:
            print("[at] bus systeme injoignable : %s" % e, file=sys.stderr)
            return 1
        if not o.modem:
            print("[at] aucun modem oFono (ofonod demarre ? modem branche ?)\n"
                  "[at] --essai pour ouvrir quand meme un modem de mise au point",
                  file=sys.stderr)
            return 1
        log("modem oFono", o.modem)
    else:
        o = OfonoEssai()
        log("MODE ESSAI : aucun oFono derriere, reponses par defaut")

    master, slave = pty.openpty()
    name = os.ttyname(slave)
    tty.setraw(master)
    # The RIL opens the stable link, not the /dev/pts/N of the day.
    os.makedirs(RUN, exist_ok=True)
    try:
        if os.path.islink(LINK) or os.path.exists(LINK):
            os.remove(LINK)
        os.symlink(name, LINK)
    except OSError as e:
        log("lien %s non pose : %s" % (LINK, e))
    log("modem AT virtuel sur %s (lien : %s)" % (name, LINK))
    log("a donner a rild :  rild -l /vendor/lib64/libreference-ril.so -d %s" % LINK)

    m = AtModem(o, master)
    o.subscribe(IF_SMS, "IncomingMessage", m.on_incoming_sms, path=o.modem)
    o.subscribe(IF_VOICE, "CallAdded", m.on_call_added, path=o.modem)
    o.subscribe(IF_NETREG, "PropertyChanged", m.on_netreg_changed, path=o.modem)

    def readable(_src, _cond):
        try:
```

```
            data = os.read(master, 4096)
        except OSError:
            return True
        if data:
            m.feed(data)
        return True

    GLib.io_add_watch(master, GLib.IO_IN, readable)
    GLib.MainLoop().run()
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

2.275 /opt/GSM/osmo-operator/tools/osmo-sim-install.sh

8085 octets, 163 lignes → 163 lignes

```
#!/bin/bash
# ============================================================================
# osmo-sim-install.sh - THE BENCH SIM, installed once for TWO paths.
#
# Same logic as tools/osmo-extras-install.sh: what must exist both in the
# SUPPLEMENT (addition.sh --sim, on an already installed machine) and in the
# NATIVE install (the ISO, iso_modules/80-chroot.sh, in the chroot) is written
# ONCE here; both paths source this script and call osmo_sim_install.
# Nothing is fatal: a missing package must not stop the rest.
#
# What it installs:
#   - pcscd + libccid: the PC/SC daemon and the CCID driver of the HID Omnikey
#     reader (6121, reads the ATR of the sysmoISIM-SJA5); pcsc-tools
#     (pcsc_scan) and opensc (opensc-tool -l/-a) to see the reader and the ATR;
#   - Ruby + ruby-libxml + the smartcard gem: softSIM (/opt/GSM/softsim), the
#     SAP server that serves either a software SIM (sim.xml, type sim) or the
#     card in the reader (type pcsc) to the osmocom-bb mobile ("sim sap");
#   - pySim (Osmocom, from its git: the PyPI "pySim" is another project) in
#     its venv /opt/GSM/pysim-venv: pySim-read, pySim-prog, pySim-shell, to
#     read and program the sysmoISIM-SJA5 (ADM1 key required to write);
#   - sysmo-usim-tool (sysmocom) in /opt/GSM/sysmo-usim-tool: the tool that
#     writes IMSI, Ki, algorithm (COMP128v1) and OPc on the sysmoISIM-SJA5;
#   - softSIM itself if missing (clone + patches tools/softsim/*.patch + Ruby
#     3.2 shim), as iso_modules/51-softsim.sh does for the ISO;
#   - pcscd.socket enabled (except in chroot: no systemd, the unit is enabled
#     at first boot by its preset).
#
# Idempotent: can be re-run without harm. Variables:
#   OSMO_SIM_PYSIM=0      do not install pySim
#   OSMO_SIM_SOFTSIM=0    do not install softSIM
#   PYSIM_VENV            the pySim venv (default /opt/GSM/pysim-venv)
#   SOFTSIM_DIR           the softSIM tree (default /opt/GSM/softsim)
#   REPO / DIR            the osmo-operator repository (softSIM patches, Ruby shim)
# ============================================================================

: "${GREEN:=}"; : "${YELLOW:=}"; : "${CYAN:=}"; : "${BOLD:=}"; : "${NC:=}"

_sim_repo() {
    local d
    for d in "${REPO:-}" "${DIR:-}" /opt/GSM/osmo-operator "$(cd "$(dirname "${BASH_SOURCE[0]}")/.." 2>/dev/null
     ↪  && pwd)"; do
        [ -n "$d" ] && [ -f "$d/tools/softsim/ruby_compat.rb" ] && { printf '%s' "$d"; return 0; }
    done
    return 1
}
```

```
_sim_say()  { echo -e "  ${GREEN}✓${NC} $*"; }
_sim_warn() { echo -e "  ${YELLOW}!${NC} $*" >&2; }

# In chroot (ISO): no running systemd, no reader to test.
_sim_in_chroot() { [ "$(stat -c %d:%i / 2>/dev/null)" != "$(stat -c %d:%i /proc/1/root/. 2>/dev/null)" ]; }

osmo_sim_install_apt() {
    local pkgs="pcscd libccid pcsc-tools opensc libpcsclite1 libpcsclite-dev
                python3 python3-venv python3-pip python3-pyscard swig
                ruby ruby-libxml ruby-dev build-essential git"
    export DEBIAN_FRONTEND=noninteractive
    local apt=apt-get; command -v apt-fast >/dev/null 2>&1 && apt=apt-fast
    # shellcheck disable=SC2086
    if $apt install -y --no-install-recommends $pkgs >/dev/null 2>&1; then
        _sim_say "paquets SIM : pcscd, libccid, pcsc-tools, opensc, pyscard, ruby"
    else
        # one by one: a package missing from the mirror must not block the others
        local p; for p in $pkgs; do
            $apt install -y --no-install-recommends "$p" >/dev/null 2>&1 || _sim_warn "paquet absent : $p"
        done
    fi
    if ! _sim_in_chroot && command -v systemctl >/dev/null 2>&1; then
        systemctl enable --now pcscd.socket >/dev/null 2>&1 && _sim_say "pcscd.socket actif" \
            || _sim_warn "pcscd.socket non active"
    fi
}

osmo_sim_install_gem() {
    command -v gem >/dev/null 2>&1 || { _sim_warn "gem absent : pas de softSIM pcsc"; return 0; }
    if gem list -i smartcard >/dev/null 2>&1; then
        _sim_say "gem smartcard deja present"
    elif gem install smartcard --no-document >/dev/null 2>&1; then
        _sim_say "gem smartcard installe (softSIM en lecteur PC/SC)"
    else
        _sim_warn "gem smartcard non installe (softSIM pcsc indisponible)"
    fi
}

osmo_sim_install_pysim() {
    [ "${OSMO_SIM_PYSIM:-1}" = 1 ] || return 0
    local venv="${PYSIM_VENV:-/opt/GSM/pysim-venv}"
    local repo="${OSMO_PYSIM_REPO:-https://gitea.osmocom.org/sim-card/pysim.git}"
    if [ -x "$venv/bin/pySim-shell.py" ] && [ "${OSMO_SIM_PYSIM_UPDATE:-0}" != 1 ]; then
        _sim_say "pySim deja present : $venv"
        return 0
    fi
    python3 -m venv "$venv" >/dev/null 2>&1 || { _sim_warn "venv pySim impossible ($venv)"; return 0; }
    # pyscard needs swig + libpcsclite-dev to build (PKGS above).
    if "$venv/bin/pip" install -q --disable-pip-version-check --no-cache-dir "git+$repo" >/dev/null 2>&1; then
        _sim_say "pySim (Osmocom) : $venv/bin/pySim-{read,prog,shell}.py"
    else
        _sim_warn "pySim non installe (reseau ? swig/libpcsclite-dev ?)"
    fi
}

osmo_sim_install_sysmo_tool() {
    local d="${SYSMO_TOOL:-/opt/GSM/sysmo-usim-tool}"
    if [ -f "$d/sysmo-isim-tool.sja5.py" ]; then _sim_say "sysmo-usim-tool deja present : $d"; return 0; fi
    if GIT_TERMINAL_PROMPT=0 git clone -q --depth 1
     ↪  "${OSMO_SYSMO_TOOL_REPO:-https://gitea.sysmocom.de/sysmocom/sysmo-usim-tool.git}" "$d" 2>/dev/null; then
        _sim_say "sysmo-usim-tool (sysmo-isim-tool.sja5.py) : $d"
    else
        rm -rf "$d"; _sim_warn "sysmo-usim-tool : clone impossible (reseau ?) - pySim-prog servira"
    fi
}
```

```
osmo_sim_install_softsim() {
    [ "${OSMO_SIM_SOFTSIM:-1}" = 1 ] || return 0
    local sdir="${SOFTSIM_DIR:-/opt/GSM/softsim}" repo p
    repo="$(_sim_repo)" || { _sim_warn "depot osmo-operator introuvable : softSIM non pose"; return 0; }
    if [ ! -f "$sdir/src/demo_server.rb" ]; then
        rm -rf "$sdir"
        if GIT_TERMINAL_PROMPT=0 git clone -q --depth 1
         ↪  "${OSMO_SOFTSIM_REPO:-https://gitea.osmocom.org/sim-card/softsim.git}" "$sdir" 2>/dev/null; then
            _sim_say "softSIM clone : $sdir"
        else
            rm -rf "$sdir"; _sim_warn "softSIM : clone impossible (reseau ?)"; return 0
        fi
    fi
    install -Dm644 "$repo/tools/softsim/comp128.rb" "$sdir/src/lib/comp128.rb"
    for p in "$repo"/tools/softsim/*.patch; do
        [ -f "$p" ] || continue
        if git -C "$sdir" apply --check -R "$p" >/dev/null 2>&1; then
            :  # deja applique
        elif git -C "$sdir" apply "$p" >/dev/null 2>&1; then
            _sim_say "softSIM : patch $(basename "$p") applique"
        else
            _sim_warn "softSIM : patch $(basename "$p") ne s applique pas"
        fi
    done
    install -Dm644 "$repo/tools/softsim/ruby_compat.rb" /usr/local/share/softsim/ruby_compat.rb
    if [ -f "$sdir/src/sim.xml" ]; then
        python3 "$repo/tools/softsim/make_sim_op1.py" "$sdir/src/sim.xml" "$sdir/src/sim-op1.xml" \
            --imsi "${OSMO_SIM_IMSI:-001010001000001}" >/dev/null 2>&1 \
            && _sim_say "softSIM : sim-op1.xml (IMSI ${OSMO_SIM_IMSI:-001010001000001})"
    fi
    _sim_say "softSIM pret : $sdir (shim /usr/local/share/softsim/ruby_compat.rb)"
}

osmo_sim_install() {
    echo -e "${CYAN}SIM du banc : lecteur PC/SC (Omnikey), softSIM (SAP), pySim${NC}"
    osmo_sim_install_apt
    osmo_sim_install_gem
    osmo_sim_install_pysim
    osmo_sim_install_sysmo_tool
    osmo_sim_install_softsim
    if ! _sim_in_chroot && command -v opensc-tool >/dev/null 2>&1; then
        local r; r="$(timeout 10 opensc-tool -l 2>/dev/null | sed -n '3,$p')"
        [ -n "$r" ] && echo -e "  ${BOLD}lecteurs :${NC}\n$(printf '%s\n' "$r" | sed 's/^/    /')" \
                    || echo -e "  ${YELLOW}aucun lecteur PC/SC branche${NC}"
    fi
    echo -e "  ${BOLD}suite :${NC} tools/osmo-sim.sh lecteur | lire | programmer | sap  (voir wiki/SIM.md)"
    return 0
}

# Run directly: do the work (otherwise, source + osmo_sim_install).
if [ "${BASH_SOURCE[0]}" = "$0" ]; then
    [ "$(id -u)" -eq 0 ] || { echo "osmo-sim-install.sh : a lancer en root (sudo)" >&2; exit 1; }
    osmo_sim_install
fi
```

2.276 /opt/GSM/osmo-operator/tools/osmo-sim.sh

20216 octets, 372 lignes → 372 lignes

```
#!/bin/bash
# ============================================================================
# osmo-sim.sh - THE BENCH SIM IN A PC/SC READER (HID Omnikey).
#
```

```
# The card (sysmocom sysmoISIM-SJA5, in an OMNIKEY 6121) is read and
# programmed with pySim (Osmocom, venv /opt/GSM/pysim-venv), then SERVED to
# the osmocom-bb mobile by the softSIM SAP server in pcsc mode: this is what
# start-native.sh --sap does when a card is in the reader.
#
#   tools/osmo-sim.sh lecteur                the PC/SC readers, the ATR, the card type
#   tools/osmo-sim.sh lire                   everything the card says (pySim-read)
#   tools/osmo-sim.sh programmer [options]   writes the bench identity on the card
#   tools/osmo-sim.sh verifier  [options]    does the card carry the bench identity,
#                                            does the HLR know it: ready for --sap?
#   tools/osmo-sim.sh hlr [--creer] [options] the card subscriber in the HLR
#   tools/osmo-sim.sh sap [--socket S]       serves the card over SAP (foreground),
#                                            what start-native.sh --sap does by itself
#   tools/osmo-sim.sh shell                  pySim-shell on the card (ADM1 if known)
#
# THE BENCH IDENTITY (same plan as qosmo/run_modules/21-abonnes-hlr.sh and
# start-native.sh, section "ms_imsi / ms_ki"):
#   IMSI   = MCC MNC <operator on 4> <MS on 6>           001 01 0001 000001
#   Ki     = 00112233445566778899aabbccdd <MS> <operator>   ...dd 01 01
#   MSISDN = node*100000 + operator*100 + MS                  100101
#   algo   = COMP128v1 (osmo-hlr: aud2g comp128v1)
# Options: --ms N  --op N  --mcc 001  --mnc 01  --imsi I  --ki K  --msisdn M
#          --lecteur N (PC/SC index, default 0)  --dry-run  --oui (no question)
#
# WRITING goes through the sysmocom tool, sysmo-usim-tool (sysmo-isim-tool.sja5.py,
# SYSMO_TOOL=/opt/GSM/sysmo-usim-tool): IMSI, Ki, COMP128v1 algorithm, OPc. Without
# it (or OSMO_SIM_PYSIM_PROG=1): pySim-prog then pySim-shell for the algorithm.
#
# THE ADM1 KEY: writing on a sysmoISIM requires the ADM1 key of THE card (8
# digits, provided by sysmocom with the card: CSV file / order e-mail). Three
# wrong attempts BLOCK the card for good: the script never guesses. Sources,
# in order: --adm1 K, OSMO_SIM_ADM1, the file /etc/osmocom/sim-adm1 (one line,
# or "ICCID key" per line for several cards), ~/.config/osmo/sim-adm1.
# ============================================================================
set -u

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
PYSIM_VENV="${PYSIM_VENV:-/opt/GSM/pysim-venv}"
SOFTSIM_DIR="${SOFTSIM_DIR:-/opt/GSM/softsim}"
SAP_SOCK="${SAP_SOCK:-/tmp/osmocom_sap}"
HLR_VTY_PORT="${HLR_VTY_PORT:-4258}"
HLR_DB="${HLR_DB:-/var/lib/osmocom/hlr.db}"
OSMOCOM_CFG="${OSMOCOM_CFG:-/etc/osmocom}"
# The python that has pyscard: the one of the pySim venv (root's profile puts
# /root/.env, the bridge venv, first in the PATH, and it has no pyscard), else the
# system one.
PY="$PYSIM_VENV/bin/python3"; [ -x "$PY" ] || PY=/usr/bin/python3

if [ -t 1 ]; then
    C_OK=$'\033[32m'; C_KO=$'\033[31m'; C_W=$'\033[33m'; C_B=$'\033[1m'; C_D=$'\033[2m'; C_Z=$'\033[0m'
else
    C_OK=; C_KO=; C_W=; C_B=; C_D=; C_Z=
fi
ok()   { printf '  %s✓%s %s\n' "$C_OK" "$C_Z" "$*"; }
ko()   { printf '  %s✗%s %s\n' "$C_KO" "$C_Z" "$*" >&2; }
warn() { printf '  %s!%s %s\n' "$C_W" "$C_Z" "$*" >&2; }
die()  { ko "$*"; exit 1; }

usage() { sed -n '2,/^# ====*$/p' "$0" | sed -n '2,$p' | sed 's/^# \{0,1\}//' | sed '$d'; }

# -- the identity plan ------------------------------------------------------
MS=1 OP=1 MCC="" MNC="" IMSI="" KI="" MSISDN="" READER="${OSMO_SIM_READER:-0}"
DRY=0 OUI=0 ADM1="${OSMO_SIM_ADM1:-}" CREER=0 SOCK="$SAP_SOCK" VERB="${SAP_VERBOSITY:-3}"

plan_mcc() {
    [ -n "$MCC" ] && { printf '%s' "$MCC"; return; }
```

```
    sed -n 's/^[[:space:]]*network country code[[:space:]]\{1,\}\([0-9]*\).*/\1/p' \
        "$OSMOCOM_CFG/osmo-msc.cfg" 2>/dev/null | head -1 | grep . || printf '001'
}
plan_mnc() {
    [ -n "$MNC" ] && { printf '%s' "$MNC"; return; }
    sed -n 's/^[[:space:]]*mobile network code[[:space:]]\{1,\}\([0-9]*\).*/\1/p' \
        "$OSMOCOM_CFG/osmo-msc.cfg" 2>/dev/null | head -1 | grep . || printf '01'
}
plan_node() {
    local n="${OSMO_WAN_NODE:-${WAN_NODE_ID:-}}"
    [ -n "$n" ] || n="$(sed -n 's/^PLAN_NODE=//p' "$OSMOCOM_CFG/radio-plan.env" 2>/dev/null | tail -1)"
    case "$n" in [1-9]) ;; *) n=1 ;; esac
    printf '%s' "$n"
}
plan_imsi()   { [ -n "$IMSI" ] && printf '%s' "$IMSI" || printf '%s%s%04d%06d' "$(plan_mcc)" "$(plan_mnc)" "$OP"
 ↪  "$MS"; }
plan_ki()     { [ -n "$KI" ] && printf '%s' "$KI" || printf '00112233445566778899aabbccdd%02x%02x' "$MS" "$OP"; }
plan_msisdn() { [ -n "$MSISDN" ] && printf '%s' "$MSISDN" || printf '%d' $(( $(plan_node) * 100000 + OP * 100 + MS
 ↪  )); }

# ── the tools ──────────────────────────────────────────────────────────
pysim() {  # pysim read|prog|shell args...
    local t="$1"; shift
    local bin="$PYSIM_VENV/bin/pySim-$t.py"
    [ -x "$bin" ] || die "pySim absent ($bin) : sudo $HERE/tools/osmo-sim-install.sh"
    "$bin" -p "$READER" "$@"
}
pcscd_pret() {
    if command -v systemctl >/dev/null 2>&1 && ! systemctl is-active --quiet pcscd pcscd.socket 2>/dev/null; then
        systemctl start pcscd.socket >/dev/null 2>&1 || true
    fi
}
# The readers, and whether they hold a card: "N Yes|No name"
lecteurs() {
    "$PY" - <<'PYSC' 2>/dev/null
from smartcard.System import readers
for i, r in enumerate(readers()):
    try:
        c = r.createConnection(); c.connect(); atr = bytes(c.getATR()).hex(); c.disconnect()
        print(i, "Yes", atr, str(r))
    except Exception:
        print(i, "No", "-", str(r))
PYSC
}
carte_presente() { lecteurs | awk -v r="$READER" '$1==r && $2=="Yes"{f=1} END{exit !f}'; }
exiger_carte() {
    pcscd_pret
    "$PY" -c 'import smartcard' 2>/dev/null \
        || die "pyscard absent ($PY) : sudo $HERE/tools/osmo-sim-install.sh"
    [ -n "$(lecteurs)" ] || die "aucun lecteur PC/SC (pcscd actif ? Omnikey branche ? libccid ?)"
    carte_presente || die "pas de carte dans le lecteur $READER :$(lecteurs | sed 's/^/\n    /')"
}
# The ICCID and the IMSI of the card (pySim-read), "iccid imsi"
carte_identite() {
    local out
    out="$(timeout 90 "$PYSIM_VENV/bin/pySim-read.py" -p "$READER" 2>/dev/null)" || return 1
    printf '%s %s %s' \
        "$(printf '%s\n' "$out" | sed -n 's/^ICCID: *//p' | head -1)" \
        "$(printf '%s\n' "$out" | sed -n 's/^IMSI: *//p' | head -1)" \
        "$(printf '%s\n' "$out" | sed -n 's/^Autodetected card type: *//p' | head -1 | tr ' ' '_')"
}

# The ADM1 key: --adm1, OSMO_SIM_ADM1, /etc/osmocom/sim-adm1, ~/.config/osmo/sim-adm1
adm1_trouver() {  # $1 = ICCID de la carte
    local f k
    [ -n "$ADM1" ] && { printf '%s' "$ADM1"; return 0; }
```

```
    for f in "$OSMOCOM_CFG/sim-adm1" "$HOME/.config/osmo/sim-adm1"; do
        [ -r "$f" ] || continue
        k="$(awk -v ic="$1" 'NF==2 && $1==ic {print $2; exit} NF==1 && $1 !~ /^#/ {one=$1} END{if(!k && one) print
         ↪  one}' "$f")"
        [ -n "$k" ] && { printf '%s' "$k"; return 0; }
    done
    return 1
}

vty() {  # vty cmd... -> sortie du VTY HLR
    local c; { echo enable; for c in "$@"; do echo "$c"; done; echo exit; } \
        | timeout 10 socat -T 5 - "TCP:127.0.0.1:$HLR_VTY_PORT,crlf" 2>/dev/null
}
hlr_connait() {  # $1 = IMSI
    if [ -r "$HLR_DB" ] && command -v sqlite3 >/dev/null 2>&1; then
        [ "$(sqlite3 -readonly "$HLR_DB" "select count(*) from subscriber where imsi='$1';" 2>/dev/null)" -ge 1 ]
         ↪  2>/dev/null && return 0
    fi
    command -v socat >/dev/null 2>&1 && vty "subscriber imsi $1 show" | grep -q "IMSI: $1"
}

# ── the commands ──────────────────────────────────────────────────────
cmd_lecteur() {
    pcscd_pret
    local l; l="$(lecteurs)"
    [ -n "$l" ] || die "aucun lecteur PC/SC (pcscd actif ? Omnikey branche ? libccid ?)"
    printf '%slecteurs PC/SC :%s\n' "$C_B" "$C_Z"
    printf '%s\n' "$l" | while read -r i yes atr name; do
        if [ "$yes" = Yes ]; then
            printf '  %s%s%s  %s\n      carte : ATR %s\n' "$C_B" "$i" "$C_Z" "$name" "$atr"
        else
            printf '  %s%s%s  %s\n      %spas de carte%s\n' "$C_B" "$i" "$C_Z" "$name" "$C_D" "$C_Z"
        fi
    done
    if carte_presente && [ -x "$PYSIM_VENV/bin/pySim-read.py" ]; then
        read -r iccid imsi type <<<"$(carte_identite)"
       printf '  lecteur %s : %s  ICCID %s  IMSI %s\n' "$READER" "${type:-type_inconnu}" "${iccid:-?}" "${imsi:-?}"
    fi
}

cmd_lire() { exiger_carte; pysim read; }

cmd_shell() {
    exiger_carte
    local k; k="$(adm1_trouver "$(carte_identite | cut -d' ' -f1)")" || true
    if [ -n "$k" ]; then pysim shell -a "$k" "$@"; else pysim shell "$@"; fi
}

# pySim-prog (identity, PLMN, SMSP, Ki) then pySim-shell: the 2G algorithm of
# the sysmoISIM. pySim-prog only writes the Ki (bytes 1..16 of EF.SIM_AUTH_KEY),
# not the configuration byte: the card would stay in Milenage and the HLR
# (COMP128v1) would answer "authentication failed". Byte 0: bits 7-6 free,
# bit 5 SRES derivation 1, bit 4 OPc, bits 3-0 algorithm (1 = COMP128v1, 4 =
# Milenage): 0x11. Written in DF.SYSTEM/EF.SIM_AUTH_KEY (6f20) and
# ADF.USIM/EF.USIM_AUTH_KEY_2G (af22), the two 2G files of the card. OP/OPc:
# 16 zero bytes.
cmd_programmer() {
    exiger_carte
    local imsi ki msisdn mcc mnc iccid cur_imsi type
    imsi="$(plan_imsi)"; ki="$(plan_ki)"; msisdn="$(plan_msisdn)"; mcc="$(plan_mcc)"; mnc="$(plan_mnc)"
    case "$imsi" in [0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]) ;; *) die "IMSI
     ↪  a 15 chiffres attendu : $imsi" ;; esac
```

```
case "$ki" in [0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-
 ↪  fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-
 ↪  F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-
 ↪  9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F][0-9a-fA-F]) ;; *) die "Ki : 32 caracteres hexa attendus : $ki"
 ↪  ;; esac
read -r iccid cur_imsi type <<<"$(carte_identite)"
[ -n "$iccid" ] || die "la carte ne repond pas a pySim-read (lecteur $READER)"
printf '%sla carte%s   : %s  ICCID %s  IMSI actuel %s\n' "$C_B" "$C_Z" "${type:-?}" "$iccid" "${cur_imsi:-?}"
printf '%sle banc%s    : IMSI %s  PLMN %s-%s  Ki %s  MSISDN %s  COMP128v1\n' "$C_B" "$C_Z" "$imsi" "$mcc"
 ↪  "$mnc" "$ki" "$msisdn"
local adm=""
if [ "$DRY" = 0 ]; then
    adm="$(adm1_trouver "$iccid")" || die "cle ADM1 inconnue pour l ICCID $iccid : --adm1 K, OSMO_SIM_ADM1, ou
     ↪  $OSMOCOM_CFG/sim-adm1 (voir l en-tete). Elle vient de sysmocom avec la carte. --dry-run montre sans
     ↪  ecrire."
    case "$adm" in [0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]) ;; *) die "ADM1 : 8 chiffres attendus (une
     ↪  sysmoISIM), lu « ${#adm} caracteres »" ;; esac
    if [ "$OUI" = 0 ]; then
        printf '%sEcrire ces valeurs sur la carte %s ? Trois ADM1 faux bloquent la carte. [o/N] %s' "$C_W"
         ↪  "$iccid" "$C_Z"
        read -r rep; case "$rep" in o|O|oui|y|Y) ;; *) echo "abandon"; exit 1 ;; esac
    fi
fi
local sysmo="${SYSMO_TOOL:-/opt/GSM/sysmo-usim-tool}/sysmo-isim-tool.sja5.py"
if [ "$type" = sysmoISIM-SJA5 ] && [ -f "$sysmo" ] && [ "${OSMO_SIM_PYSIM_PROG:-0}" != 1 ]; then
    # 1. The sysmocom tool (sysmo-usim-tool, sysmo-isim-tool.sja5.py): IMSI,
    # MNC length, Ki (2G/3G/4G5G), 2G algorithm COMP128v1 (3G Milenage),
    # OPc. It is the one that knows how to set the algorithm; pySim-prog
    # does not.
    printf '%s1. sysmo-isim-tool.sja5 (sysmocom)%s\n' "$C_B" "$C_Z"
    if [ "$DRY" = 1 ]; then
        printf '    %s -a <ADM1> -J %s -N %s -K %s -T COMP128v1:MILENAGE -C %s\n' "$(basename "$sysmo")"
         ↪  "$imsi" "${#mnc}" "$ki" "00000000000000000000000000000000"
        ok "dry-run : rien n a ete ecrit"; return 0
    fi
    local out
    if ! out="$("$PY" "$sysmo" -a "$adm" -J "$imsi" -N "${#mnc}" -K "$ki" -T COMP128v1:MILENAGE \
                -C 00000000000000000000000000000000 2>&1)"; then
        printf '%s\n' "$out" | tail -12 | sed 's/^/    /' >&2
        die "sysmo-isim-tool a echoue (ADM1 refusee ? carte retiree ?)"
    fi
    printf '%s\n' "$out" | grep -E 'programmed|algorithm|2g:|3g:|IMSI|Error|rror' | sed 's/^/    /'
    # 2. read back the algorithm and the Ki with the same tool
    printf '%s2. relecture (algorithme, Ki)%s\n' "$C_B" "$C_Z"
    out="$("$PY" "$sysmo" -a "$adm" -t -k 2>&1)"
   printf '%s\n' "$out" | grep -q '2g: 1=COMP128v1' || { printf '%s\n' "$out" | tail -8 | sed 's/^/    /' >&2;
     ↪  die "COMP128v1 non relu sur la carte"; }
    printf '%s\n' "$out" | grep -q "2g: Key: $ki" || { printf '%s\n' "$out" | tail -8 | sed 's/^/    /' >&2;
     ↪  die "Ki non relu sur la carte"; }
    ok "COMP128v1 et Ki relus sur la carte (2g)"
else
    # Without the sysmocom tool (or OSMO_SIM_PYSIM_PROG=1): pySim-prog then pySim-shell.
    # 1. pySim-prog: ICCID kept, IMSI, PLMN (HPLMN, PLMNsel), SPN, SMSP, MSISDN, Ki
    local args=(-t sysmoISIM-SJA5 -s "$iccid" -i "$imsi" -k "$ki" -o 00000000000000000000000000000000
                -x "$mcc" -y "$mnc" --mnclen "${#mnc}" -n "Osmo $OP" --msisdn "$msisdn" --acc 0001)
    [ "$type" = sysmoISIM-SJA5 ] || args=("${args[@]:2}")   # autre carte : laisser pySim la reconnaitre
    [ "$DRY" = 1 ] && args+=(--dry-run)
    [ -n "$adm" ] && args+=(-a "$adm")
    printf '%s1. pySim-prog%s\n' "$C_B" "$C_Z"
    local out
    if ! out="$(pysim prog "${args[@]}" 2>&1)"; then
        printf '%s\n' "$out" | tail -15 >&2
        die "pySim-prog a echoue (ADM1 refusee ? carte retiree ?)"
    fi
    printf '%s\n' "$out" | grep -E '^( > |Programming|Dry Run|Autodetected)' | sed 's/^/    /'
    [ "$DRY" = 1 ] && { ok "dry-run : rien n a ete ecrit"; return 0; }
```

```
        # 2. pySim-shell: the 2G algorithm = COMP128v1 in both authentication files.
        # verify_adm receives the key: without argument it looks for it in a CSV by
        # ICCID ("Unable to find card key data"), -a is not enough.
        printf '%s2. pySim-shell : algorithme COMP128v1%s\n' "$C_B" "$C_Z"
        local script; script="$(mktemp)"
        cat >"$script" <<EOF
verify_adm $adm
select MF
select DF.SYSTEM
select EF.SIM_AUTH_KEY
update_binary 11${ki}00000000000000000000000000000000
read_binary_decoded
select MF
select ADF.USIM
select EF.USIM_AUTH_KEY_2G
update_binary 11${ki}00000000000000000000000000000000
read_binary_decoded
EOF
        out="$(pysim shell -a "$adm" --noprompt --script "$script" 2>&1)"; rm -f "$script"
        if printf '%s\n' "$out" | grep -q -i -e 'SW Match Error' -e 'Exception' -e 'EXCEPTION' -e 'Error'; then
            printf '%s\n' "$out" | grep -i -e error -e exception | head -5 | sed 's/^/    /' >&2
            die "l ecriture de l algorithme a echoue (voir ci-dessus) ; l identite (etape 1) est ecrite"
        fi
        local n; n="$(printf '%s\n' "$out" | grep -c '"algorithm": "comp128v1"')"
        [ "$n" -ge 1 ] || { printf '%s\n' "$out" | tail -8 | sed 's/^/    /' >&2; die "COMP128v1 non relu sur la
        ↪  carte"; }
        ok "COMP128v1 relu dans $n fichier(s) d authentification"
    fi
    # 3. read back
    read -r iccid cur_imsi type <<<"$(carte_identite)"
    [ "$cur_imsi" = "$imsi" ] || die "relecture : IMSI $cur_imsi au lieu de $imsi"
    ok "carte programmee : IMSI $imsi, Ki ${ki:0:8}..., COMP128v1, MSISDN $msisdn"
    cmd_hlr
}

cmd_hlr() {
    local imsi ki iccid type
    if [ -n "$IMSI" ]; then imsi="$IMSI"; else
        exiger_carte; read -r iccid imsi type <<<"$(carte_identite)"
        [ -n "$imsi" ] || die "IMSI illisible sur la carte"
    fi
    if hlr_connait "$imsi"; then
        ok "HLR : l abonne $imsi existe ($HLR_DB / VTY $HLR_VTY_PORT)"
        return 0
    fi
    if [ "$CREER" = 1 ]; then
        ki="$(plan_ki)"
        command -v socat >/dev/null 2>&1 || die "socat absent : impossible d ecrire dans le VTY HLR"
        vty "subscriber imsi $imsi create" "subscriber imsi $imsi update msisdn $(plan_msisdn)" \
            "subscriber imsi $imsi update aud2g comp128v1 ki $ki" >/dev/null
        hlr_connait "$imsi" && ok "HLR : abonne $imsi cree (COMP128v1, Ki ${ki:0:8}..., MSISDN $(plan_msisdn))" \
                            || die "HLR : creation de $imsi echouee (osmo-hlr lance ? VTY $HLR_VTY_PORT ?)"
        return 0
    fi
    if [ "$imsi" = "$(plan_imsi)" ]; then
        warn "HLR : $imsi inconnu pour l instant - c est l IMSI du MS#$MS, start-native.sh le cree au lancement
        ↪  (21-abonnes-hlr)"
    else
        warn "HLR : $imsi inconnu - tools/osmo-sim.sh hlr --creer --ki <Ki de la carte> (ou programmer la carte au
        ↪  plan du banc)"
    fi
    return 1
}

cmd_verifier() {
    exiger_carte
```

```
    local iccid imsi type rc=0
    read -r iccid imsi type <<<"$(carte_identite)"
    [ -n "$imsi" ] || die "IMSI illisible sur la carte (lecteur $READER)"
    ok "carte : ${type:-?}  ICCID $iccid  IMSI $imsi"
    if [ "$imsi" = "$(plan_imsi)" ]; then ok "IMSI = celui du MS#$MS de l operateur $OP (plan du banc)"
    else warn "IMSI $imsi ≠ plan du banc $(plan_imsi) : tools/osmo-sim.sh programmer, ou hlr --creer"; rc=1; fi
    cmd_hlr || rc=1
    if [ -f "$SOFTSIM_DIR/src/demo_server.rb" ] && ruby -e 'require "smartcard"' >/dev/null 2>&1; then
        ok "softSIM pcsc : $SOFTSIM_DIR, gem smartcard present"
    else warn "softSIM pcsc indisponible : sudo $HERE/tools/osmo-sim-install.sh"; rc=1; fi
    if [ $rc -eq 0 ]; then ok "prete : ./start-native.sh --sap (la carte est servie par SAP au mobile)"
    else warn "pas encore prete pour --sap"; fi
    return $rc
}

cmd_sap() {
    exiger_carte
    [ -f "$SOFTSIM_DIR/src/demo_server.rb" ] || die "softSIM absent ($SOFTSIM_DIR) : sudo
     ↪  $HERE/tools/osmo-sim-install.sh"
    ruby -e 'require "smartcard"' >/dev/null 2>&1 || die "gem smartcard absent : sudo
     ↪  $HERE/tools/osmo-sim-install.sh"
    local shim=/usr/local/share/softsim/ruby_compat.rb
    [ -f "$shim" ] || shim="$HERE/tools/softsim/ruby_compat.rb"
    for p in $(ps -eo pid,args | awk '/[d]emo_server\.rb/ {print $1}'); do kill "$p" 2>/dev/null; done
    rm -f "$SOCK"
    ok "serveur SAP (softSIM, type pcsc) sur $SOCK - Ctrl-C pour arreter ; le mobile : sim sap + sap-socket $SOCK"
    cd "$SOFTSIM_DIR/src" && exec ruby -r"$shim" demo_server.rb --type pcsc --socket unix --unix "$SOCK"
     ↪  --verbosity "$VERB"
}

# ── command line ──────────────────────────────────────────────────────
CMD="${1:-}"; [ $# -gt 0 ] && shift
REST=()
while [ $# -gt 0 ]; do
    case "$1" in
        --ms)       MS="${2:?}"; shift ;;
        --op)       OP="${2:?}"; shift ;;
        --mcc)      MCC="${2:?}"; shift ;;
        --mnc)      MNC="${2:?}"; shift ;;
        --imsi)     IMSI="${2:?}"; shift ;;
        --ki)       KI="${2:?}"; shift ;;
        --msisdn)   MSISDN="${2:?}"; shift ;;
        --adm1)     ADM1="${2:?}"; shift ;;
        --lecteur|-p) READER="${2:?}"; shift ;;
        --socket)   SOCK="${2:?}"; shift ;;
        --verbosity) VERB="${2:?}"; shift ;;
        --dry-run)  DRY=1 ;;
        --oui|-y)   OUI=1 ;;
        --creer)    CREER=1 ;;
        -h|--help)  usage; exit 0 ;;
        *)          REST+=("$1") ;;
    esac
    shift
done
case "$CMD" in
    lecteur|lecteurs) cmd_lecteur ;;
    lire)             cmd_lire ;;
    programmer)       cmd_programmer ;;
    verifier)         cmd_verifier ;;
    hlr)              cmd_hlr ;;
    sap)              cmd_sap ;;
    shell)            cmd_shell "${REST[@]}" ;;
    ''|-h|--help)     usage; exit 0 ;;
    *)                die "commande inconnue : $CMD (lecteur, lire, programmer, verifier, hlr, sap, shell)" ;;
esac
```

2.277 /opt/GSM/osmo-operator/tools/osmo-supplements-firstboot.sh

2283 octets, 55 lignes → 55 lignes

```
#!/bin/bash
# osmo-supplements-firstboot.sh - the supplements chosen in the installer, at first boot.
#
# The Calamares "Supplements" page (installer/calamares/modules/
# packagechooser-supplements.conf) only writes /etc/osmo-supplements:
#   OSMO_SUPPLEMENTS="--multi --opencl ..."   the addition.sh flags
#   HUB_IMAGE="..."                           the origin of the operator image (multi)
# osmo-supplements.service runs this script once the network is up. On success the
# file becomes /etc/osmo-supplements.fait and the unit disables itself. On failure it
# is retried at the next boot, three times at most (/etc/osmo-supplements.essais), then
# /etc/osmo-supplements.echec: "sudo addition.sh" remains available by hand.
# Log: /var/log/osmo-supplements.log (and journalctl -u osmo-supplements).
set -u
CONF=/etc/osmo-supplements
ESSAIS=/etc/osmo-supplements.essais
LOG=/var/log/osmo-supplements.log
REPO="${OSMO_REPO:-/opt/GSM/osmo-operator}"
MAX="${OSMO_SUPPLEMENTS_ESSAIS:-3}"

_fin() {  # _fin <suffixe> : range la config et desactive l unite
    mv -f "$CONF" "$CONF.$1"
    rm -f "$ESSAIS"
    systemctl disable osmo-supplements.service >/dev/null 2>&1 || true
}

[ -f "$CONF" ] || exit 0
# shellcheck disable=SC1090
. "$CONF"
[ -n "${OSMO_SUPPLEMENTS:-}" ] || { _fin fait; exit 0; }

n=$(( $(cat "$ESSAIS" 2>/dev/null || echo 0) + 1 ))
echo "$n" > "$ESSAIS"

# network-online.target does not guarantee Internet (slow DHCP, Wi-Fi not yet associated):
# wait until GitHub answers, five minutes at most.
for _ in $(seq 1 60); do
    curl -fsS -m 5 -o /dev/null https://github.com && break
    sleep 5
done

echo "=== $(date '+%F %T') essai $n/$MAX : addition.sh $OSMO_SUPPLEMENTS ===" >>"$LOG"
# shellcheck disable=SC2086
# OSMO_COMPTE: the installer account (docker group, icons), nobody being logged in
# yet; otherwise the first human account (uid 1000 and above).
[ -n "${OSMO_COMPTE:-}" ] || OSMO_COMPTE="$(getent passwd | awk -F: '$3>=1000 && $3<60000 {print $1; exit}')"
env HUB_IMAGE="${HUB_IMAGE:-}" OSMO_COMPTE="${OSMO_COMPTE:-}" bash "$REPO/addition.sh" $OSMO_SUPPLEMENTS >>"$LOG"
 ↪  2>&1 </dev/null
rc=$?
echo "=== $(date '+%F %T') fin : statut $rc ===" >>"$LOG"

if [ "$rc" -eq 0 ]; then
    _fin fait
elif [ "$n" -ge "$MAX" ]; then
    _fin echec
fi
exit "$rc"
```

2.278 /opt/GSM/osmo-operator/tools/osmo-telecom-boot.sh

3568 octets, 75 lignes → 75 lignes

```
#!/bin/bash
# ============================================================================
# osmo-telecom-boot.sh - apply the "no telecom service at boot" policy
#
# Installs services/80-osmo-operator.preset in
# <root>/usr/lib/systemd/system-preset/ then disables, among the units
# PRESENT, those targeted by this preset (systemctl --root for an ISO rootfs).
# Nothing is stopped: a running service keeps running until it is stopped, it
# simply no longer starts at boot.
#
#   tools/osmo-telecom-boot.sh                this machine
#   tools/osmo-telecom-boot.sh --root DIR     a rootfs (ISO chroot)
#   tools/osmo-telecom-boot.sh --garder U...  units to leave enabled
#                                             (e.g. osmo-interstp for the
#                                             interstp role, osmo-banc if
#                                             OSMO_ISO_BANC=1)
#   tools/osmo-telecom-boot.sh --liste        only print what would be done
# ============================================================================
set -uo pipefail
DIR="$(cd "$(dirname "$(readlink -f "$0")")/.." && pwd)"
PRESET="$DIR/services/80-osmo-operator.preset"
ROOT=""; LISTE=0; GARDER=()
while [ $# -gt 0 ]; do
    case "$1" in
        --root)   ROOT="${2%/}"; shift ;;
        --garder) shift; while [ $# -gt 0 ] && [ "${1#--}" = "$1" ]; do GARDER+=("${1%.service}"); shift; done;
        ↪  continue ;;
        --liste)  LISTE=1 ;;
        -h|--help) sed -n '2,19p' "$0"; exit 0 ;;
    esac
    shift
done
[ -f "$PRESET" ] || { echo "preset introuvable : $PRESET" >&2; exit 1; }

# the "disable" patterns of the preset
mapfile -t MOTIFS < <(awk '$1 == "disable" {print $2}' "$PRESET")

# the units present on the target that match a pattern
unites=()
for d in "$ROOT/etc/systemd/system" "$ROOT/lib/systemd/system" "$ROOT/usr/lib/systemd/system"; do
    [ -d "$d" ] || continue
    for f in "$d"/*.service; do
        [ -e "$f" ] || continue
        u="$(basename "$f")"
        for m in "${MOTIFS[@]}"; do
            # shellcheck disable=SC2053
            [[ $u == $m ]] && { unites+=("$u"); break; }
        done
    done
done
mapfile -t unites < <(printf '%s\n' "${unites[@]}" | sort -u | grep -v '^$')

[ "$LISTE" = 1 ] || install -D -m644 "$PRESET" "$ROOT/usr/lib/systemd/system-preset/80-osmo-operator.preset"
sc=(systemctl); [ -n "$ROOT" ] && sc+=(--root="$ROOT")
n=0
for u in "${unites[@]}"; do
    for g in "${GARDER[@]}"; do [ "${u%.service}" = "$g" ] && continue 2; done
    if [ -n "$ROOT" ]; then
        # rootfs: enabled = a *.wants/ link to the unit (is-enabled is unreliable outside systemd)
        compgen -G "$ROOT/etc/systemd/system/*.wants/$u" >/dev/null || continue
        etat=enabled
    else
        etat="$("${sc[@]}" is-enabled "$u" 2>/dev/null)"
        case "$etat" in enabled|enabled-runtime|linked) ;; *) continue ;; esac
    fi
    if [ "$LISTE" = 1 ]; then echo "  desactiverait : $u ($etat)"; n=$((n + 1)); continue; fi
```

```
    if "${sc[@]}" disable "$u" >/dev/null 2>&1; then echo "  boot : $u desactive"; n=$((n + 1))
    else
        # chroot without systemd: remove the links ourselves (what disable does)
        find "$ROOT/etc/systemd/system" -path "*.wants/$u" -delete 2>/dev/null && echo "  boot : $u desactive
        ↪  (liens)" && n=$((n + 1))
    fi
done
[ "${#GARDER[@]}" -gt 0 ] && echo "  boot : gardes actives : ${GARDER[*]}"
if [ "$LISTE" = 1 ]; then echo "  politique telecom au boot : $n unite(s) a desactiver"
else echo "  politique telecom au boot : $n unite(s) desactivee(s), preset 80-osmo-operator.preset pose"; fi
exit 0
```

2.279 /opt/GSM/osmo-operator/tools/osmo-topzone.py

15543 octets, 370 lignes → 370 lignes

```
#!/usr/bin/env python3
# osmo-topzone.py - THE TOP "LAB GSM" ZONE, live and clickable.
#
# Replaces the LAB GSM card formerly BAKED into the wallpaper
# (tools/wallpaper-render.py, run with OSMO_LIVE_BANNER=1 so that it does not
# draw it). It is a window of DESKTOP type (like osmo-panel.py) placed EXACTLY
# on the box of the card (320,60 .. 1430,560 in 1920x1080), which displays
# configs/conky/labgsm.html through WebKit.
#
# TWO ROLES:
#   1. IDLE (nothing hosted): the banner animates its 8 timeslots. It reads
#      $OSMO_FFT_DIR/timeslots.json (written by tools/osmo-ts-probe.py: osmo-bsc
#      config + VTY) and makes any timeslot carrying bursts BLINK IN ORANGE -
#      a TCH only blinks when there is voice. ARFCN / IMSI / A5 are the LIVE
#      values. The burst chronogram flickers with the activity.
#   2. HOSTING: when osmo-launcher.py sends an application "to the top", it
#      writes "HOST <winid>" in $XDG_RUNTIME_DIR/osmo-topzone.cmd. The banner
#      is then FADED and the application window is placed on the zone. A small
#      bar above (always in front) carries a close CROSS and a FULL SCREEN /
#      TOP RECTANGLE toggle.
import json
import os
import subprocess
import sys
import time

# SAME CONSTRAINT AS osmo-panel.py / osmo-dino.py (see their notes): under
# Wayland, GTK3 ignores move() and the DESKTOP type; once on XWayland,
# "always below" makes the GNOME Shell desktop layer swallow the clicks.
# X11 backend, normal window.
X11_FORCE = os.environ.get("XDG_SESSION_TYPE") == "wayland" and bool(os.environ.get("DISPLAY"))
if X11_FORCE:
    os.environ["GDK_BACKEND"] = "x11"

import gi

gi.require_version("Gtk", "3.0")
gi.require_version("Gdk", "3.0")
gi.require_version("WebKit2", "4.1")
from gi.repository import Gdk, GLib, Gtk, WebKit2  # noqa: E402

# THE BANNER STAYS AT THE BACK. Unlike the panel and the launcher, it is
# WANTED stuck to the desktop, behind everything: it is a decoration, not a
# control panel. On XWayland this costs the clicks (GNOME Shell puts DESKTOP
# type windows under its icons layer, which swallows them) - so window
# hosting in the top zone does not respond to the mouse. That is the choice
# made here; OSMO_TOPZONE_CLIQUABLE=1 to get a normal window back (in front,
# but clickable) when it is needed.
```

```
_CLIQUABLE = os.environ.get("OSMO_TOPZONE_CLIQUABLE") == "1"
TYPE_HINT = Gdk.WindowTypeHint.NORMAL if _CLIQUABLE else Gdk.WindowTypeHint.DESKTOP
KEEP_BELOW = not _CLIQUABLE

FW, FH = 1920, 1080
# THE EMPTY BOTTOM IS REMOVED, NOT THE CONTENT.
# The box (1110x500, i.e. 1644x740 on the bench screen) is much taller than
# what the card fills; the page is written in vh/vw units (#card is
# height:100%, titles 8.5vh, labels 1.3vh...), so shortening the window or
# zooming the view to 0.5 would SHRINK the whole content.
# What is done: the page is rendered AT ITS ORIGINAL SIZE (the vh keep the
# reference height below, so the content is unchanged), the frame stops after
# its content (injected CSS: #card becomes height:auto), and the window is
# fitted to the height really occupied, measured IN the page once loaded.
# No empty space, and not one pixel of content lost.
TOP_BOX = (320, 60, 1110, 500)          # LAB GSM card (x,y,w,h) - the RENDERING
# The stylesheet that cuts the empty space. Note: #card is in
# "position:absolute; inset:0" - it is not a height that stretches it but its
# bottom:0. Setting height:auto changes NOTHING as long as the bottom is not
# released; both are needed.
INJECT_CSS = "#card{bottom:auto!important;height:auto!important}"
REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
HTML = os.environ.get("OSMO_LABGSM_HTML", os.path.join(REPO, "configs/conky/labgsm.html"))
FFT_RUN = os.environ.get("OSMO_FFT_DIR", "/run/osmo-fft")


# ── WHERE TO READ: THE SAME RULE AS THE PROBE, RE-EVALUATED ON EVERY TURN ──
# The SAME rule on both sides (tools/osmo-ts-probe.py, run_dir()), word for
# word: the first directory that exists and is writable, among $OSMO_FFT_DIR
# and /tmp - re-evaluated on every read, because /run/osmo-fft is a
# RuntimeDirectory that comes and goes with its service.
def ts_json():
    for d in (FFT_RUN, "/tmp"):
        if os.path.isdir(d) and os.access(d, os.W_OK):
            return os.path.join(d, "timeslots.json")
    return "/tmp/timeslots.json"
RUN = os.environ.get("XDG_RUNTIME_DIR") or "/run/user/%d" % os.getuid()
CMD_FILE = os.path.join(RUN if os.path.isdir(RUN) else "/tmp", "osmo-topzone.cmd")
# THE BENCH STATE, WRITTEN BY osmo-panel.py ("1": a 2G or a 4G is running,
# "0": nothing). Bench stopped, the banner fades and the zone only carries the
# "Telecom Operator" title; when the 2G starts, the title fades into the
# banner (labgsm.html, osmoBanc). File absent: idle.
ETAT_FILE = os.path.join(RUN if os.path.isdir(RUN) else "/tmp", "osmo-banc.etat")


def banc_etat():
    try:
        return open(ETAT_FILE).read().strip() == "1"
    except OSError:
        return False


def screen_geom():
    disp = Gdk.Display.get_default()
    mon = disp.get_primary_monitor() or disp.get_monitor(0)
    g = mon.get_geometry()
    sw, sh = g.width, g.height
    try:
        from gi.repository import Gio
        mode = Gio.Settings.new("org.gnome.desktop.background").get_string("picture-options")
    except Exception:
        mode = "zoom"
    if mode == "stretched":
        sx, sy, ox, oy = sw / FW, sh / FH, 0, 0
    else:
        s = min(sw / FW, sh / FH) if mode == "scaled" else max(sw / FW, sh / FH)
```

```
        sx = sy = s
        ox, oy = (FW * s - sw) / 2, (FH * s - sh) / 2
    return (g.x, g.y, sw, sh), (sx, sy, ox, oy)


def box_to_screen(box):
    (gx, gy, sw, sh), (sx, sy, ox, oy) = screen_geom()
    bx, by, bw, bh = box
    return (gx + int(bx * sx - ox), gy + int(by * sy - oy), int(bw * sx), int(bh * sy))


def _place(winid, geom):
    x, y, w, h = geom
    subprocess.run(["wmctrl", "-i", "-r", winid, "-b",
                    "remove,maximized_vert,maximized_horz,fullscreen"], timeout=3)
    subprocess.run(["wmctrl", "-i", "-r", winid, "-e", f"0,{x},{y},{w},{h}"], timeout=3)


def _fullscreen(winid, on=True):
    subprocess.run(["wmctrl", "-i", "-r", winid, "-b",
                    ("add" if on else "remove") + ",fullscreen"], timeout=3)


def _close_win(winid):
    subprocess.run(["wmctrl", "-i", "-c", winid], timeout=3)


class Strip(Gtk.Window):
    """Control bar ALWAYS IN FRONT (cross + full screen toggle)."""
    def __init__(self, on_close, on_toggle):
        super().__init__(type=Gtk.WindowType.TOPLEVEL)
        self.set_decorated(False)
        self.set_keep_above(True)
        self.set_skip_taskbar_hint(True)
        self.set_skip_pager_hint(True)
        self.set_type_hint(Gdk.WindowTypeHint.UTILITY)
        self.set_accept_focus(False)
        prov = Gtk.CssProvider()
        prov.load_from_data(b".strip{background:rgba(19,16,24,0);border-radius:8px}"
                            b".strip button{color:#efe6d2;background:rgba(40,34,52,.5);text-shadow:0 1px 2px
                             ↪  rgba(0,0,0,.9);"
                            b"border:1px solid rgba(239,230,210,.25);border-radius:6px;"
                            b"margin:2px;padding:0 8px;font:11pt 'DejaVu Sans Mono'}"
                            b".strip button:hover{border-color:#ff9a3c}")
        Gtk.StyleContext.add_provider_for_screen(
            self.get_screen(), prov, Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION)
        box = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=0)
        box.get_style_context().add_class("strip")
        self.b_mode = Gtk.Button(label="▢")
        self.b_mode.set_tooltip_text("plein écran / rectangle haut")
        self.b_mode.connect("clicked", lambda *_a: on_toggle())
        b_close = Gtk.Button(label="×")
        b_close.set_tooltip_text("fermer et rendre la bannière")
        b_close.connect("clicked", lambda *_a: on_close())
        box.pack_start(self.b_mode, False, False, 0)
        box.pack_start(b_close, False, False, 0)
        self.add(box)

    def show_at(self, x, y):
        self.show_all()
        self.set_keep_above(True)
        self.move(x, y)


class TopZone(Gtk.Window):
    def __init__(self):
```

```
    super().__init__(title="osmo-topzone")
    x, y, w, h = box_to_screen(TOP_BOX)
    self.box_geo = (x, y, w, h)
    self.hosted = None       # winid of the hosted application
    self.fs = False          # full screen mode of the hosted application
    self.set_type_hint(TYPE_HINT)
    self.set_decorated(False)
    self.set_resizable(False)
    self.set_keep_below(KEEP_BELOW)
    self.set_skip_taskbar_hint(True)
    self.set_skip_pager_hint(True)
    self.stick()
    self.set_default_size(w, h)
    self.set_size_request(w, h)
    self.move(x, y)
    screen = self.get_screen()
    visual = screen.get_rgba_visual()
    if visual and screen.is_composited():
        self.set_visual(visual)
    self.set_app_paintable(True)
    self.connect("destroy", Gtk.main_quit)

    ucm = WebKit2.UserContentManager()
    ucm.add_style_sheet(WebKit2.UserStyleSheet(
        INJECT_CSS, WebKit2.UserContentInjectedFrames.ALL_FRAMES,
        WebKit2.UserStyleLevel.USER, None, None))
    self.view = WebKit2.WebView.new_with_user_content_manager(ucm)
    # Once the page is loaded, it is asked for the height the card REALLY
    # occupies and the window is fitted to it.
    self.view.connect("load-changed", self.on_load)
    self.view.set_background_color(Gdk.RGBA(0, 0, 0, 0))
    self.add(self.view)
    if os.path.exists(HTML):
        self.view.load_uri(GLib.filename_to_uri(HTML, None))
    else:
       self.view.load_html("<h1 style='color:#fff;font-family:sans-serif'>labgsm.html introuvable</h1>", None)

    self.strip = Strip(self.on_close, self.on_toggle)

    self._cmd_mtime = None
    self._etat_mtime = None
    self._etat = False           # what the page shows: False = the title
    self.show_all()
    self.move(x, y)
    # The file is looked at five times per second and the state is pushed
    # ONLY if it changed: reactive, and no useless JS.
    self._ts_mtime = None
    GLib.timeout_add(200, self.push_state)
    GLib.timeout_add(500, self.poll_cmd)

# ── fit the window to the useful height ──────────────────────────
def on_load(self, _view, event):
    if event != WebKit2.LoadEvent.FINISHED:
        return
    # the bench state as it is, WITHOUT fade (the page starts "idle")
    self._banc(banc_etat(), instant=True)
    # A hair of margin under the frame, otherwise the bottom border grazes
    # the edge of the window.
    self.view.run_javascript(
        "(function(){var c=document.getElementById('card')||document.body;"
        "return Math.ceil(c.getBoundingClientRect().bottom)+2;})()",
        None, self.on_height, None)

def on_height(self, view, res, _data):
    try:
        val = view.run_javascript_finish(res).get_js_value().to_double()
```

```
    except Exception as e:                      # page replaced, JS cut...
        print("[topzone] hauteur non mesuree : %s" % e, file=sys.stderr, flush=True)
        return
    h = int(val)
    x, y, w, h_full = self.box_geo
    if not (0 < h < h_full):                    # absurd measurement: everything is kept
        print("[topzone] hauteur mesuree %d ignoree (fenetre %d)" % (h, h_full),
              file=sys.stderr, flush=True)
        return
    print("[topzone] hauteur utile %d px (au lieu de %d)" % (h, h_full), flush=True)
    self.box_geo = (x, y, w, h)
    self.set_size_request(w, h)
    self.resize(w, h)
    self.move(x, y)

# ── idle: push the timeslot state into the page ──────────────────────
def push_state(self):
    chemin = ts_json()
    try:
        mt = (chemin, os.stat(chemin).st_mtime)
    except OSError:
        mt = None
    if mt is not None and mt == self._ts_mtime:
        return True                     # nothing new since the last turn
    self._ts_mtime = mt
    try:
        with open(chemin) as f:
            state = f.read().strip()
    except OSError:
        state = ""
    if state:
        js = "window.__osmoLive=true; if(window.osmoUpdate){osmoUpdate(%s);}" % state
        try:
            self.view.run_javascript(js, None, None, None)
        except Exception:
            pass
    return True

def _fade(self, on):
    try:
        self.view.run_javascript("if(window.osmoFade)osmoFade(%s);" % ("true" if on else "false"),
                                 None, None, None)
    except Exception:
        pass

# ── "Telecom Operator" title (idle) <-> banner (bench up) ────────────
def _banc(self, on, instant=False):
    self._etat = on
    try:
        self.view.run_javascript("if(window.osmoBanc)osmoBanc(%s,%s);"
                                 % ("true" if on else "false", "true" if instant else "false"),
                                 None, None, None)
    except Exception:
        pass

def poll_etat(self):
    try:
        mt = os.stat(ETAT_FILE).st_mtime
    except OSError:
        mt = None
    if mt == self._etat_mtime:
        return
    self._etat_mtime = mt
    on = banc_etat()
    if on != self._etat:
        print("[topzone] %s" % ("un banc tourne : banniere" if on else "banc a l arret : titre"), flush=True)
```

```
            self._banc(on)

    # ── hosting: read the osmo-launcher -> osmo-topzone channel ──────────────
    def poll_cmd(self):
        self.poll_etat()
        try:
            mt = os.stat(CMD_FILE).st_mtime
        except OSError:
            return True
        if mt == self._cmd_mtime:
            return True
        self._cmd_mtime = mt
        try:
            line = open(CMD_FILE).read().strip()
        except OSError:
            return True
        parts = line.split()
        if len(parts) >= 2 and parts[0] == "HOST":
            self.host(parts[-1], full=("FS" in parts))
        return True

    def host(self, winid, full=False):
        self.hosted = winid
        self.fs = full
        self._fade(True)
        x, y, w, h = self.box_geo
        if full:
            _fullscreen(winid, True)
        else:
            _place(winid, self.box_geo)
        # the control bar, in the top-right corner of the zone
        self.strip.b_mode.set_label("▭" if full else "□")
        self.strip.show_at(x + w - 74, max(0, y - 2))

    def on_toggle(self):
        if not self.hosted:
            return
        self.fs = not self.fs
        if self.fs:
            _fullscreen(self.hosted, True)
            self.strip.b_mode.set_label("▭")
        else:
            _place(self.hosted, self.box_geo)
            self.strip.b_mode.set_label("□")
        x, y, w, h = self.box_geo
        self.strip.show_at(x + w - 74, max(0, y - 2))

    def on_close(self):
        if self.hosted:
            _close_win(self.hosted)
        self.hosted = None
        self.fs = False
        self.strip.hide()
        self._fade(False)


if __name__ == "__main__":
    TopZone()
    Gtk.main()
```

2.280 /opt/GSM/osmo-operator/tools/osmo-ts-probe.py

17876 octets, 446 lignes → 446 lignes

```
#!/usr/bin/env python3
# osmo-ts-probe.py - WHO CARRIES BURSTS RIGHT NOW? Writes the state of the 8
# timeslots that the LAB GSM banner (configs/conky/labgsm.html, via
# osmo-topzone.py) blinks in orange.
#
# Two sources:
#   1. THE osmo-bsc CONF: phys_chan_config of each timeslot (CCCH+SDCCH4,
#      SDCCH8, TCH/F ...). Gives the TYPE, hence the label of each cell and
#      whether a TCH can carry VOICE. Looked up in, whichever exists:
#         $OSMO_BSC_CFG, /etc/osmocom/osmo-bsc.cfg, $OSMO_REPO/configs/osmo-bsc.cfg
#   2. THE osmo-bsc VTY (telnet 127.0.0.1 4242, `show lchan summary`): which
#      logical channels are ESTABLISHED, on which timeslot, of which type. An
#      established TCH = voice -> the timeslot blinks. No VTY (bench stopped) ->
#      bsc=false, the banner switches to demonstration mode.
#
# Output: $OSMO_FFT_DIR/timeslots.json (default /run/osmo-fft, /tmp as
# fallback - see run_dir(), the SAME rule as the reader), rewritten every
# period, of the form:
#   {"bsc":true,"arfcn":"ARFCN 514 · DCS 1800",
#    "ts":[{"n":0,"type":"CCCH+SDCCH4","active":false,"voice":false}, ...]}
import json
import os
import re
import socket
import sys
import time

REPO = os.environ.get("OSMO_REPO", "/opt/GSM/osmo-operator")
# ── WHERE TO WRITE: THE SAME RULE AS THE READER, RE-EVALUATED EVERY TURN ────
# The reader (tools/osmo-topzone.py) looks ONLY at $OSMO_FFT_DIR then /tmp, so
# the rule is the same here, word for word: the first directory that EXISTS
# AND IS WRITABLE, between $OSMO_FFT_DIR and /tmp.
#
# "writable" and not only "exists": /run/osmo-fft is created by
# osmo-fft-snap.service (RuntimeDirectory, 0775, group sudo), but it may also
# have been created earlier by another tool as 0755 root:root, in which case
# the session probe cannot write there.
#
# The rule is RE-EVALUATED AT EVERY WRITE: that directory is a
# RuntimeDirectory, it disappears when the unit stops and comes back when it
# restarts. A probe that resolved its path once would otherwise keep writing
# into the void until the next restart.
RUN_PREFERE = os.environ.get("OSMO_FFT_DIR", "/run/osmo-fft")


def run_dir():
    for d in (RUN_PREFERE, "/tmp"):
        if os.path.isdir(d) and os.access(d, os.W_OK):
            return d
    return "/tmp"
VTY_HOST = os.environ.get("OSMO_BSC_VTY_HOST", "127.0.0.1")
VTY_PORT = int(os.environ.get("OSMO_BSC_VTY_PORT", "4242"))
# THE BANNER FOLLOWS ONE PRECISE BTS, AND ONLY ONE. The eight cells are the
# TDMA frame of ONE carrier: BTS 0, TRX 0 - the bench BTS. On a multi-operator
# bench (several BTS declared in the same osmo-bsc) the VTY lines of the other
# BTS/TRX are ignored, so a channel established on BTS 1 never lights a cell
# of BTS 0.
BTS_NR = int(os.environ.get("OSMO_BTS_NR", "0"))
TRX_NR = int(os.environ.get("OSMO_TRX_NR", "0"))


def cfg_path():
    for p in (os.environ.get("OSMO_BSC_CFG"),
              "/etc/osmocom/osmo-bsc.cfg",
              os.path.join(REPO, "configs/osmo-bsc.cfg")):
        if p and os.path.exists(p):
```

```
            return p
    return None


def read_types():
    """[(n, phys_chan_config)] of the first trx of the followed bts in the conf."""
    p = cfg_path()
    types = {}
    arfcn = None
    if not p:
        return types, arfcn
    try:
        txt = open(p, errors="replace").read()
    except OSError:
        return types, arfcn
    # The block of THE followed bts (BTS_NR), not "the first one found": on a
    # multi-BTS conf the phys_chan_config differ from one BTS to another, and
    # the cell labels must be those of the carrier being displayed.
    blocks = re.split(r'\n\s*bts\s+(\d+)', txt)
    block = txt
    # re.split with a group yields [before, "0", block0, "1", block1, ...]
    for i in range(1, len(blocks) - 1, 2):
        if int(blocks[i]) == BTS_NR:
            block = blocks[i + 1]
            break
    else:
        if len(blocks) > 2:
            block = blocks[2]                # no BTS_NR: the first one
    # arfcn of the first trx (a template placeholder __ARFCN__ is ignored)
    m = re.search(r'\n\s*arfcn\s+(\d+)', block)
    if m:
        arfcn = m.group(1)
    # first trx only
    trx0 = re.split(r'\n\s*trx\s+\d+', block)
    scope = trx0[1] if len(trx0) > 1 else block
    # phys_chan_config is not necessarily the line that FOLLOWS "timeslot N":
    # a comment ("! ...") may sit between the two. Take the first
    # phys_chan_config of the timeslot block, up to the next timeslot.
    for tm in re.finditer(r'\n\s*timeslot\s+(\d+)((?:\n(?!\s*timeslot\s)[^\n]*)*)', scope):
        pm = re.search(r'\n\s*phys_chan_config\s+(\S+)', tm.group(2))
        if pm:
            types[int(tm.group(1))] = pm.group(1)
    return types, arfcn


# ── THE VTY CONNECTION THAT STAYS OPEN ──────────────────────────────
# The connection is kept across probe turns, and the reply is read up to the
# VTY prompt instead of sleeping a fixed delay: the reply is processed as soon
# as it is complete, usually within a few milliseconds, so a channel that gets
# established blinks its cell without a visible lag.
_PROMPT = re.compile(rb'[\r\n][A-Za-z0-9_.\-]+(?:\([^)]*\))?[>#]\s*$')


class Vty:
    def __init__(self):
        self.s = None

    def _connect(self):
        try:
            self.s = socket.create_connection((VTY_HOST, VTY_PORT), timeout=1.0)
            self.s.settimeout(1.0)
            self._read_until_prompt()          # the welcome banner
            return True
        except OSError:
            self.s = None
            return False
```

```
    def close(self):
        if self.s is not None:
            try:
                self.s.close()
            except OSError:
                pass
            self.s = None

    def _read_until_prompt(self, limit=262144):
        buf = b""
        while True:
            try:
                chunk = self.s.recv(4096)
            except socket.timeout:
                break                           # return what we have
            except OSError:
                self.close()
                return None
            if not chunk:
                self.close()
                return None
            buf += chunk
            if _PROMPT.search(buf) or len(buf) > limit:
                break
        return buf

    def cmd(self, text):
        """Output of a command, or None if the VTY is not reachable."""
        if self.s is None and not self._connect():
            return None
        try:
            self.s.sendall(text.encode() + b"\r\n")
        except OSError:
            self.close()
            if not self._connect():            # a single retry
                return None
            try:
                self.s.sendall(text.encode() + b"\r\n")
            except OSError:
                self.close()
                return None
        out = self._read_until_prompt()
        if out is None:
            return None
        return out.decode(errors="replace")


VTY = Vty()


def vty_lchan():
    """Return (ok, {ts: {'active':bool,'voice':bool}}) from the osmo-bsc VTY."""
    act = {}
    text = VTY.cmd("show lchan summary")
    if text is None:
        return False, act
    # typical lines: "BTS 0, TRX 0, Timeslot 2, ... Lchan 0, Type TCH/F, State ESTABLISHED ..."
    for line in text.splitlines():
        m = re.search(r'Timeslot\s+(\d+)', line)
        if not m:
            continue
        # The BTS and TRX of the line: a line from another BTS (or another
        # carrier) does not concern this banner. A line that does not name
        # them is kept - old output format, better to show than to lose the
        # information.
```

```
        mb = re.search(r'BTS\s+(\d+)', line)
        mx = re.search(r'TRX\s+(\d+)', line)
        if (mb and int(mb.group(1)) != BTS_NR) or (mx and int(mx.group(1)) != TRX_NR):
            continue
        ts = int(m.group(1))
        d = act.setdefault(ts, {"active": False, "voice": False})
        d["active"] = True
        typ = ""
        mt = re.search(r'Type\s+(\S+)', line)
        if mt:
            typ = mt.group(1).upper()
        state = line.upper()
        established = ("ESTABLISHED" in state or "ACTIVE" in state)
        if "TCH" in typ and established:
            d["voice"] = True
    return True, act


def vty_text(cmd):
    """Raw output of an osmo-bsc VTY command, or None if unreachable."""
    return VTY.cmd(cmd)


def vty_identity():
    """MCC/MNC, A5, arfcn, band LIVE from the VTY (the most up to date). {} if none."""
    ident = {}
    net = vty_text("show network")
    if net:
        m = re.search(r'Country Code[:\s]+(\d+).*?Network Code[:\s]+(\d+)', net, re.S)
        if m:
            ident["mcc"], ident["mnc"] = m.group(1), m.group(2)
        m = re.search(r'[Ee]ncryption[:\s]+A5/?(\d)', net)
        if m:
            ident["a5"] = m.group(1)
    bts = vty_text("show bts 0")
    if bts:
        m = re.search(r'\bARFCN[:\s]+(\d+)', bts) or re.search(r'\bC0[:\s]+(\d+)', bts)
        if m:
            ident["arfcn"] = m.group(1)
        m = re.search(r'[Bb]and[:\s]+(\S+)', bts)
        if m:
            ident["band"] = m.group(1)
    return ident


def net_from_cfg():
    """Fallback: MCC/MNC/A5/band from the conf file, placeholders ignored."""
    p = cfg_path()
    out = {}
    if not p:
        return out
    try:
        txt = open(p, errors="replace").read()
    except OSError:
        return out
    def grab(rx, key):
        m = re.search(rx, txt)
        if m and "__" not in m.group(1):
            out[key] = m.group(1)
    grab(r'\n\s*network country code\s+(\S+)', "mcc")
    grab(r'\n\s*mobile network code\s+(\S+)', "mnc")
    grab(r'\n\s*encryption\s+a5\s+(\S+)', "a5")
    grab(r'\n\s*band\s+(\S+)', "band")
    return out
```

```
def build():
    types, cfg_arfcn = read_types()
    if not types:
        # reasonable default without a conf: the classic bench topology
        types = {0: "CCCH+SDCCH4", 1: "SDCCH8", 2: "TCH/F", 3: "TCH/F",
                 4: "TCH/F", 5: "TCH/F", 6: "TCH/F", 7: "TCH/F"}
    ok, act = vty_lchan()
    ts = []
    for n in range(8):
        a = act.get(n, {})
        ts.append({"n": n, "type": types.get(n, "NONE"),
                   "active": bool(a.get("active")), "voice": bool(a.get("voice"))})
    out = {"bsc": ok, "ts": ts}

    # ── THE LIVE VALUES: ARFCN / IMSI (MCC-MNC) / A5 / band ──────────────
    # The VTY wins (real state of the running network); the conf file is the
    # fallback; the repo template does not count (__MCC__ placeholders ignored).
    ident = {}
    if cfg_arfcn and "__" not in cfg_arfcn:
        ident["arfcn"] = cfg_arfcn
    ident.update(net_from_cfg())
    if ok:
        ident.update(vty_identity())  # live overrides the file

    band = ident.get("band", "")
    band_txt = ""
    if band:
        b = str(band).upper().replace("DCS", "DCS ").replace("PCS", "PCS ")
        band_txt = " · " + b
    if ident.get("arfcn"):
        out["arfcn"] = "ARFCN %s%s" % (ident["arfcn"], band_txt)
    if ident.get("mcc") and ident.get("mnc"):
        out["imsi"] = "%s-%s" % (str(ident["mcc"]).zfill(3), str(ident["mnc"]).zfill(2))
    if ident.get("a5"):
        out["a5"] = str(ident["a5"])
    out.update(live_info(ts))
    return out


# ── THE BENCH IN REAL TIME, FOR THE BANNER ──────────────────────────────
# The banner receives the whole bench - the core daemons, the radio, the HLR
# subscribers, the calls in progress, how long the stack has been up - read
# cheaply (/proc, the HLR sqlite database read-only) every 2 s: the rest of
# the probe runs four times per second, and daemons that live for hours do
# not need to be recounted at that rate. Names are those the kernel keeps
# (comm, 15 characters) - "osmo-sip-connector" reads "osmo-sip-connec" there,
# as in tools/conky-osmo-status.sh.
COEUR = (("HLR", "osmo-hlr"), ("MSC", "osmo-msc"), ("BSC", "osmo-bsc"), ("STP", "osmo-stp"),
         ("MGW", "osmo-mgw"), ("BTS", "osmo-bts-trx"), ("SIP", "osmo-sip-connector"),
         ("SMSC", "proto-smsc-daemon"), ("PBX", "asterisk"))
RADIO = (("MOBILE", "mobile"), ("TRXCON", "trxcon"), ("QEMU", "qemu-system-arm"),
         ("GAPK", "osmo-gapk"))
HLR_DB = os.environ.get("OSMO_HLR_DB", "/var/lib/osmocom/hlr.db")
_live = {"quand": 0.0, "val": {}}


def _procs():
    """(comm names, command lines, {comm: pid}) of all processes."""
    comms, cmds, pids = set(), [], {}
    for p in os.listdir("/proc"):
        if not p.isdigit():
            continue
        try:
            with open("/proc/%s/comm" % p) as f:
                c = f.read().strip()
            with open("/proc/%s/cmdline" % p, "rb") as f:
```

```
                cmds.append(f.read(300).replace(b"\0", b" ").decode(errors="replace"))
            except OSError:
                continue
            comms.add(c)
            pids.setdefault(c, int(p))
    return comms, cmds, pids


def _depuis(pid):
    """How many seconds this process has been running (0 if unreadable)."""
    try:
        with open("/proc/%d/stat" % pid) as f:
            champs = f.read().rsplit(")", 1)[1].split()
        start = int(champs[19]) / os.sysconf("SC_CLK_TCK")
        with open("/proc/uptime") as f:
            up = float(f.read().split()[0])
        return max(0, int(up - start))
    except (OSError, ValueError, IndexError):
        return 0


def live_info(ts):
    now = time.time()
    if now - _live["quand"] < 2.0:
        return _live["val"]
    val = {}
    try:
        comms, cmds, pids = _procs()
        core = {n: (c[:15] in comms) for n, c in COEUR}
        radio = {n: (c[:15] in comms) for n, c in RADIO}
        phy = ""
        if any("qemu-system-arm" in c for c in cmds):
            phy = "Calypso émulé (QEMU)"
        if any("fake_trx.py" in c for c in cmds):
            phy = "faketrx"
        if any(re.search(r"pont/pont(_dsp)?\.py", c) for c in cmds):
            phy = (phy + " + pont") if phy else "pont"
        # The ISO bench has NO osmo-trx: the transceiver is faketrx
        # (fake_trx.py) and pont (pont.py / pont_dsp.py) - shown as such.
        # GAPK: the osmo-gapk binary only exists for the duration of a call;
        # outside a call the gapk-start.sh watcher IS the nominal state.
        radio["FAKETRX"] = any("fake_trx.py" in c for c in cmds)
        radio["PONT"] = any(re.search(r"pont/pont(_dsp)?\.py", c) for c in cmds)
        radio["GAPK"] = radio.get("GAPK", False) or any("gapk-start.sh" in c for c in cmds)
        subs = {}
        if core["HLR"] and os.path.exists(HLR_DB):
            try:
                import sqlite3
                con = sqlite3.connect("file:%s?mode=ro" % HLR_DB, uri=True, timeout=0.2)
                tot = con.execute("select count(*) from subscriber").fetchone()[0]
                att = con.execute("select count(*) from subscriber where vlr_number is not null "
                                  "and vlr_number != ''").fetchone()[0]
                con.close()
                subs = {"total": tot, "attached": att}
            except Exception:  # noqa: BLE001 - locked or unreadable db: no subscribers
                subs = {}
        depuis = 0
        for c in ("osmo-bsc", "osmo-stp", "osmo-msc"):
            if c in pids:
                depuis = _depuis(pids[c])
                break
        val = {"core": core, "radio": radio, "phy": phy, "subs": subs,
               "calls": sum(1 for t in ts if t.get("voice")), "up": depuis}
    except Exception as e:  # noqa: BLE001 - the banner lives without this info
        print("[ts-probe] infos banc : %s" % e, file=sys.stderr, flush=True)
    _live.update(quand=now, val=val)
```

```
    return val


def main():
    # 0.25 s: short enough for a channel lighting up to look "immediate" to
    # the eye, long enough not to hammer the osmo-bsc VTY (4 "show lchan
    # summary" per second on an already open connection).
    period = float(os.environ.get("OSMO_TS_PERIOD", "0.25"))
    try:
        os.makedirs(RUN_PREFERE, exist_ok=True)
    except OSError:
        pass                                # run_dir() falls back to /tmp
    tmp = None
    dernier = (None, 0.0)                   # (message, when) - see below
    while True:
        try:
            out = build()
            dest = os.path.join(run_dir(), "timeslots.json")
            # -- ONE TEMPORARY FILE PER PROCESS ------------------------------
            # Two probes may run at the same time (e.g. a widget surviving
            # from a previous X session). The PID in the temporary name keeps
            # the write atomic even with several writers: each one renames
            # its own temporary file only.
            tmp = "%s.%d.tmp" % (dest, os.getpid())
            with open(tmp, "w") as f:
                json.dump(out, f)
            os.replace(tmp, dest)
            tmp = None
        except Exception as e:  # noqa: BLE001 - a probe must never die
            # An orphan temporary file must not stay in /run.
            if tmp:
                try:
                    os.unlink(tmp)
                except OSError:
                    pass
                tmp = None
            # -- THE SAME FAULT IS NOT LOGGED FOUR TIMES PER SECOND ----------
            # A lasting failure (directory not accessible) would flood the log
            # with identical lines. Log the first one, then one per minute as
            # long as nothing changes.
            msg = "%s (destination %s)" % (e, run_dir())
            now = time.time()
            if msg != dernier[0] or now - dernier[1] > 60:
                print("[ts-probe] %s" % msg, file=sys.stderr, flush=True)
                dernier = (msg, now)
        time.sleep(period)


if __name__ == "__main__":
    main()
```

2.281 /opt/GSM/osmo-operator/tools/osmo-wallpaper.sh

17341 octets, 314 lignes → 314 lignes

```
#!/bin/bash
# osmo-wallpaper.sh - set the bench wallpaper with its two comic strips of
# the day embedded (Calvin & Hobbes at the bottom, a random geek comic at
# the top). Called by osmo-wallpaper.service (daily timer + boot), by the
# GNOME autostart, and by hand:
#     sudo /usr/local/sbin/osmo-wallpaper
#
# 1. The strip of the day comes from gocomics.com, as the "hellogist" gist does
#    (curl of the day's page, og:image tag, curl of the image). Without network,
```

```
#    the last cached strip is reused; without cache, the wallpaper goes without strip.
# 2. tools/wallpaper-render.py composes photo + LAB GSM map + strip.
# 3. The PNG is written under TWO names: the fixed file the GNOME schema
#    designates by default (gsm-lab-wallpaper.png, what a session sees when it
#    opens), and a DATED file pushed into every open session via gsettings -
#    a different URI forces GNOME Shell to reload, which rewriting the same
#    file does not guarantee.
set -u
# Overridable from the environment (tests off the machine: OSMO_WP_OUT=... etc.).
REPO="${OSMO_WP_REPO:-/opt/GSM/osmo-operator}"
RENDER="$REPO/tools/wallpaper-render.py"
# The file format is not fixed. The tower photo is shipped AS IS (a full
# definition PNG): forcing .jpg would re-encode and degrade it to satisfy a
# name. The first configs/wallpaper/tower.* that exists is used.
TOWER="$(ls "$REPO"/configs/wallpaper/tower.* 2>/dev/null | head -1)"
OUT="${OSMO_WP_OUT:-/usr/share/backgrounds/gsm-lab-wallpaper.png}"
DATED_DIR="${OSMO_WP_DATED_DIR:-/usr/share/backgrounds/osmo-lab}"
CACHE="${OSMO_WP_CACHE:-/var/cache/osmo-wallpaper}"
NO_SESSION="${OSMO_WP_NO_SESSION:-0}"
DAY="$(date +%F)"
STRIP="$CACHE/calvin_${DAY}.gif"
UA="Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
H=(-H "Accept: text/html,*/*;q=0.8" -H "Accept-Language: fr,en;q=0.7" -H "Referer: https://www.gocomics.com/")

[ "$(id -u)" -eq 0 ] || { echo "root requis : sudo $0" >&2; exit 1; }
[ -f "$RENDER" ] && [ -f "$TOWER" ] || { echo "[wallpaper] $RENDER ou $TOWER absent" >&2; exit 1; }
mkdir -p "$CACHE" "$DATED_DIR"

# ── 1. THE IMAGE OF THE DAY ──────────────────────────────────────────
# SEVERAL SOURCES. gocomics.com has an anti-robot check (Bunny Shield) in
# front of its pages: curl often only receives "Establishing a secure
# connection", never the og:image tag. So a source is drawn among SEVERAL,
# and the first one that answers is used. All were checked with bare curl,
# without key or account:
#   calvin   gocomics.com     - kept at the head of the list: still works
#                               from the networks the shield lets through
#   xkcd     xkcd.com         - official JSON (info.0.json), the most reliable
#   apod     apod.nasa.gov    - the astronomy picture of the day, simple HTML
#   bing     bing.com         - the photo of the day, JSON HPImageArchive
#   turnoff  turnoff.us       - programmer comics (Daniel Stori)
#
# A different source at each launch: the wallpaper changes even when the strip
# of the day did not. OSMO_WP_SOURCE=xkcd forces one; OSMO_WP_SOURCES
# restricts the list.
# TWO IMAGES, TWO FRAMES. The wallpaper inset is split in two
# (tools/wallpaper-render.py): the BOTTOM is Calvin & Hobbes, the frame of the
# 2G bench; the TOP, the frame of the 4G bench, carries a GEEK comic drawn at
# random at each launch - an xkcd picked at random in the whole collection, or
# the latest turnoff.us. Each frame has its own source list and its own draw;
# without network, each reuses its last cached image.
#   OSMO_WP_SOURCES_BAS   (calvin)                   OSMO_WP_SOURCE_BAS  forces one
#   OSMO_WP_SOURCES_HAUT  (xkcd_alea turnoff xkcd)   OSMO_WP_SOURCE_HAUT forces one
# OSMO_WP_SOURCES / OSMO_WP_SOURCE (the single-image setting) apply to the TOP.
SOURCES_BAS="${OSMO_WP_SOURCES_BAS:-calvin}"
SOURCES_HAUT="${OSMO_WP_SOURCES_HAUT:-${OSMO_WP_SOURCES:-xkcd_alea turnoff xkcd}}"
[ -n "${OSMO_WP_SOURCE_BAS:-}" ] && SOURCES_BAS="$OSMO_WP_SOURCE_BAS"
[ -n "${OSMO_WP_SOURCE_HAUT:-${OSMO_WP_SOURCE:-}}" ] && SOURCES_HAUT="${OSMO_WP_SOURCE_HAUT:-$OSMO_WP_SOURCE}"

_get() { curl -sL --max-time 30 -A "$UA" "$@" 2>/dev/null; }
# An image, and not a disguised error page: `file` looks at the content.
_pose_image() {   # url fichier referer
    _get -H "Referer: ${3:-}" -o "$2.tmp" "$1" || { rm -f "$2.tmp"; return 1; }
    if [ -s "$2.tmp" ] && file -b "$2.tmp" | grep -qiE 'image|GIF|PNG|JPEG'; then
        mv -f "$2.tmp" "$2"; return 0
    fi
    rm -f "$2.tmp"; return 1
```

```
}

# Each source writes the image in $1 and its credit line in $2. A source that
# draws at random puts in NOM_RETENU a name carrying the draw ("xkcd-1234"):
# the cache file and the wallpaper URI inherit it, so the screen changes.
NOM_RETENU=""
src_calvin() {
    local page url
    page="$(_get "${H[@]}" "https://www.gocomics.com/calvinandhobbes/$(date +%Y/%m/%d)")"
    if [ -z "$page" ]; then
        echo "[wallpaper] calvin : gocomics.com injoignable (reseau, DNS, proxy ?)"; return 1
    fi
    if printf '%s' "$page" | grep -qiE 'bunny-shield|Establishing a secure
     ↪  connection|challenge-platform|cf-browser-verification'; then
        echo "[wallpaper] calvin : gocomics.com rend une page anti-robot ($(printf '%s' "$page" | wc -c) octets)"
        return 1
    fi
    # The direct featureassets URL first (robust to HTML), og:image next -
    # the tag the "hellogist" gist reads.
    url="$(printf '%s' "$page" | grep -oE 'https://featureassets\.gocomics\.com/assets/[0-9a-f]+' | head -1)"
    [ -n "$url" ] || url="$(printf '%s' "$page" | grep -oP '<meta property="og:image" content="\K[^"]+' | head -1)"
    [ -n "$url" ] || { echo "[wallpaper] calvin : page recue, aucune image dedans (le site a change ?)"; return 1; }
    _pose_image "$url" "$1" "https://www.gocomics.com/" || return 1
    printf 'Calvin & Hobbes  ·  Bill Watterson  ·  %s  ·  gocomics.com\n' "$DAY" > "$2"
}
_xkcd_json() {   # json -> image + credit ($1 json, $2 fichier, $3 credit)
    local url num titre
    url="$(printf '%s' "$1" | grep -oP '"img":\s*"\K[^"]+')"
    [ -n "$url" ] || return 1
    num="$(printf '%s' "$1" | grep -oP '"num":\s*\K[0-9]+')"
    titre="$(printf '%s' "$1" | grep -oP '"safe_title":\s*"\K[^"]+')"
    _pose_image "$url" "$2" "https://xkcd.com/" || return 1
    printf 'xkcd #%s  ·  %s  ·  Randall Munroe  ·  xkcd.com\n' "$num" "$titre" > "$3"
    NOM_RETENU="xkcd-$num"
}
src_xkcd() { _xkcd_json "$(_get https://xkcd.com/info.0.json)" "$1" "$2"; }
# A RANDOM xkcd in the whole collection: the latest number says how many there
# are, and one is drawn. 404 does not exist (it is a joke of the author).
src_xkcd_alea() {
    local dernier num
    dernier="$(_get https://xkcd.com/info.0.json | grep -oP '"num":\s*\K[0-9]+')"
    [ -n "$dernier" ] && [ "$dernier" -gt 1 ] || return 1
    num=$(( (RANDOM * 32768 + RANDOM) % dernier + 1 ))
    [ "$num" -eq 404 ] && num=405
    _xkcd_json "$(_get "https://xkcd.com/$num/info.0.json")" "$1" "$2"
}
src_apod() {
    local page src
    page="$(_get https://apod.nasa.gov/apod/astropix.html)"
    src="$(printf '%s' "$page" | grep -oiP '<img[^>]+src="\K[^"]+' | head -1)"
    [ -n "$src" ] || return 1
    case "$src" in http*) ;; *) src="https://apod.nasa.gov/apod/${src#/}" ;; esac
    _pose_image "$src" "$1" "https://apod.nasa.gov/apod/" || return 1
    printf 'NASA · Astronomy Picture of the Day  ·  %s  ·  apod.nasa.gov\n' "$DAY" > "$2"
}
src_bing() {
    local j url cr
    j="$(_get "https://www.bing.com/HPImageArchive.aspx?format=js&idx=0&n=1&mkt=fr-FR")"
    url="$(printf '%s' "$j" | grep -oP '"url":"\K[^"]+')"
    [ -n "$url" ] || return 1
    case "$url" in http*) ;; *) url="https://www.bing.com${url}" ;; esac
    cr="$(printf '%s' "$j" | grep -oP '"copyright":"\K[^"]+')"
    _pose_image "$url" "$1" "https://www.bing.com/" || return 1
    printf 'Bing · image du jour  ·  %s\n' "${cr:-bing.com}" > "$2"
}
src_turnoff() {
```

```
    local url
    url="$(_get https://turnoff.us/feed.xml | grep -oP '<img[^>]+src=.\K[^"'"'"' ]+' | head -1)"
    [ -n "$url" ] || return 1
    _pose_image "$url" "$1" "https://turnoff.us/" || return 1
    printf 'turnoff.us  ·  Daniel Stori  ·  turnoff.us\n' > "$2"
}

# Random order: `shuf` if present, a home-made shuffle otherwise (busybox,
# pruned image). Without it the same source would always be queried first, and
# "a different source at each launch" would mean nothing.
# The source of the previous draw ($2) goes LAST: otherwise one draw in five
# lands on it and the wallpaper does not change - while that is precisely how
# one sees that a new bench has started. If it is the only one available, it
# still comes out (it is at the tail, not excluded).
_melange() {   # "sources" derniere -> l ordre, sur une ligne
    local cand="" s ordre=""
    for s in $1; do [ "$s" = "$2" ] || cand="$cand $s"; done
    [ -n "$cand" ] || cand="$1"
    if command -v shuf >/dev/null 2>&1; then
        ordre="$(printf '%s\n' $cand | shuf | tr '\n' ' ')"
    else
        for s in $cand; do
            if [ $((RANDOM % 2)) -eq 0 ]; then ordre="$s $ordre"; else ordre="$ordre $s"; fi
        done
    fi
    [ -n "$2" ] && ordre="$ordre $2"
    printf '%s' "$ordre"
}

# The draw of a frame: the first source of the list ($1) that answers,
# avoiding the one of the previous draw ($2). Result in R_STRIP / R_CREDIT /
# R_SRC; 1 if none answers.
tirer() {
    local s f c
    R_STRIP=""; R_CREDIT=""; R_SRC=""
    for s in $(_melange "$1" "$2"); do
        f="$CACHE/strip_${DAY}_${s}.img"; c="$CACHE/strip_${DAY}_${s}.credit"
        # Already downloaded today: do not ask the site again. Not for a random
        # draw (xkcd_alea): there, it is a new number every time.
        case "$s" in
            *_alea) ;;
            *) if [ -s "$f" ]; then
                   R_STRIP="$f"; R_CREDIT="$(cat "$c" 2>/dev/null || true)"; R_SRC="$s"
                   echo "[wallpaper] image du $DAY : source ${s} (deja en cache)"; return 0
               fi ;;
        esac
        NOM_RETENU=""
        if declare -F "src_$s" >/dev/null && "src_$s" "$f" "$c"; then
            if [ -n "$NOM_RETENU" ] && [ "$NOM_RETENU" != "$s" ]; then
                mv -f "$f" "$CACHE/strip_${DAY}_${NOM_RETENU}.img"
                mv -f "$c" "$CACHE/strip_${DAY}_${NOM_RETENU}.credit" 2>/dev/null
                f="$CACHE/strip_${DAY}_${NOM_RETENU}.img"; c="$CACHE/strip_${DAY}_${NOM_RETENU}.credit"
                s="$NOM_RETENU"
            fi
            R_STRIP="$f"; R_CREDIT="$(cat "$c" 2>/dev/null || true)"; R_SRC="$s"
            echo "[wallpaper] image du $DAY : source ${s}"; return 0
        fi
    done
    return 1
}
# Fallback: the last cached image whose name matches the pattern ($1),
# excluding one file ($2, the image of the other frame). Empty if none.
_dernier_cache() {
    local f
    for f in $(ls -1t $CACHE/$1 2>/dev/null); do
        [ -s "$f" ] && [ "$f" != "$2" ] && { printf '%s' "$f"; return 0; }
```

```
    done
    return 1
}

DERNIERE_BAS="$(awk -F= '/^SOURCE=/{print $2}' "$CACHE/strip.state" 2>/dev/null)"
DERNIERE_HAUT="$(awk -F= '/^SOURCE_HAUT=/{print $2}' "$CACHE/strip.state" 2>/dev/null)"

# ── THE BOTTOM: Calvin & Hobbes ──────────────────────────────────────
STRIP_BAS=""; CREDIT_BAS=""; SRC_BAS=""
if tirer "$SOURCES_BAS" "$DERNIERE_BAS"; then
    STRIP_BAS="$R_STRIP"; CREDIT_BAS="$R_CREDIT"; SRC_BAS="$R_SRC"
else
    # gocomics behind its shield: the last cached Calvin, whatever its date
    # (calvin_*.gif: the name used before the random draw).
    STRIP_BAS="$(_dernier_cache 'strip_*_calvin.img' '' || _dernier_cache 'calvin_*.gif' '' || true)"
    if [ -n "$STRIP_BAS" ]; then
        CREDIT_BAS="$(cat "${STRIP_BAS%.img}.credit" 2>/dev/null || true)"; SRC_BAS="calvin"
        echo "[wallpaper] bas : calvin ne repond pas - dernier Calvin en cache : ${STRIP_BAS##*/}"
    elif tirer "apod bing" ""; then
        STRIP_BAS="$R_STRIP"; CREDIT_BAS="$R_CREDIT"; SRC_BAS="$R_SRC"
        echo "[wallpaper] bas : aucun Calvin, ni en ligne ni en cache - image du jour a la place"
    else
        STRIP_BAS="$(_dernier_cache 'strip_*.img' '' || true)"
        [ -n "$STRIP_BAS" ] && CREDIT_BAS="$(cat "${STRIP_BAS%.img}.credit" 2>/dev/null || true)"
    fi
fi

# ── THE TOP: the geek comic, at random ──────────────────────────────
STRIP_HAUT=""; CREDIT_HAUT=""; SRC_HAUT=""
if tirer "$SOURCES_HAUT" "$DERNIERE_HAUT"; then
    STRIP_HAUT="$R_STRIP"; CREDIT_HAUT="$R_CREDIT"; SRC_HAUT="$R_SRC"
else
    STRIP_HAUT="$(_dernier_cache 'strip_*_xkcd*.img' "$STRIP_BAS" || _dernier_cache 'strip_*_turnoff.img'
    ↪  "$STRIP_BAS" \
                || _dernier_cache 'strip_*.img' "$STRIP_BAS" || true)"
    if [ -n "$STRIP_HAUT" ]; then
        CREDIT_HAUT="$(cat "${STRIP_HAUT%.img}.credit" 2>/dev/null || true)"
        SRC_HAUT="$(basename "$STRIP_HAUT" .img | sed 's/^strip_[0-9-]*_//')"
        echo "[wallpaper] haut : aucune source ne repond - derniere BD en cache : ${STRIP_HAUT##*/}"
    else
        echo "[wallpaper] haut : aucune source ne repond et le cache est vide - cadre sans image."
        echo "[wallpaper] Deposez n importe quelle image dans $CACHE/strip_${DAY}_local.img et relancez."
    fi
fi
STRIP="$STRIP_BAS"; SRC_RETENUE="$SRC_BAS"

args=()
if [ -n "$STRIP_BAS" ] && [ -s "$STRIP_BAS" ]; then
    args+=(--strip "$STRIP_BAS")
    [ -n "$CREDIT_BAS" ] && args+=(--credit "$CREDIT_BAS")
fi
if [ -n "$STRIP_HAUT" ] && [ -s "$STRIP_HAUT" ]; then
    args+=(--strip-haut "$STRIP_HAUT")
    [ -n "$CREDIT_HAUT" ] && args+=(--credit-haut "$CREDIT_HAUT")
fi
args+=(--date "$DAY")
# ── WHAT THE PANEL MUST KNOW ─────────────────────────────────────────
# Without image, the wallpaper has nothing to show at that place: the frame is
# empty. The panel (tools/osmo-fft-snap.py) composes the bench IN TRANSPARENCY
# over this frame; it is told what is behind it, and without image it switches
# to full opacity.
# STRIP/SOURCE = the bottom, STRIP_HAUT/SOURCE_HAUT = the top.
# FOND = the composed render: it is what the panel crops when it exists;
# without this line (raw wallpaper, inactive timer) the panel shows the whole
# bench illustration.
printf 'STRIP=%s\nDATE=%s\nSOURCE=%s\nSTRIP_HAUT=%s\nSOURCE_HAUT=%s\nFOND=%s\n' \
```

```
    "$([ -n "$STRIP_BAS" ] && [ -s "$STRIP_BAS" ] && echo oui || echo non)" "$DAY" "${SRC_BAS:-}" \
    "$([ -n "$STRIP_HAUT" ] && [ -s "$STRIP_HAUT" ] && echo oui || echo non)" "${SRC_HAUT:-}" \
    "$OUT" \
    > "$CACHE/strip.state"
# The cache only keeps the last 14 images (two per launch).
ls -1t "$CACHE"/strip_*.img 2>/dev/null | tail -n +15 | while read -r _old; do
    rm -f "$_old" "${_old%.img}.credit"
done
ls -1t "$CACHE"/calvin_*.gif 2>/dev/null | tail -n +15 | xargs -r rm -f

# ── 2. the render ──────────────────────────────────────────────────────
# THE NAME CARRIES THE SOURCE, AND THAT IS WHAT MAKES THE SCREEN CHANGE.
# GNOME Shell only reloads on a URI change: rewriting the file under its old
# name repaints nothing. A name with only the day would give the SAME path
# for two renders on the same day. So the name carries the retained source:
# it changes at each launch (see the draw above, which avoids the previous
# one), so the URI changes, so the screen follows.
DATED="$DATED_DIR/gsm-lab-${DAY}${SRC_BAS:+-$SRC_BAS}${SRC_HAUT:+-$SRC_HAUT}.png"
python3 "$RENDER" --tower "$TOWER" "${args[@]}" --out "$DATED" || exit 1
cp -f "$DATED" "$OUT.tmp" && mv -f "$OUT.tmp" "$OUT"
chmod 644 "$OUT" "$DATED"
# The last three renders are kept. The current file is NEVER deleted: a
# session that just received its URI is still reading it.
ls -1t "$DATED_DIR"/gsm-lab-*.png 2>/dev/null | tail -n +4 | grep -vxF "$DATED" | xargs -r rm -f

# ── 3. the open sessions ──────────────────────────────────────────────
# A GNOME session reads the schema when it opens; an already open session only
# follows dconf. The dated URI is pushed into each one (root on the live key,
# the Calamares user on disk).
[ "$NO_SESSION" = "1" ] && exit 0
for bus in /run/user/*/bus; do
    [ -S "$bus" ] || continue
    uid="${bus#/run/user/}"; uid="${uid%/bus}"
    user="$(id -nu "$uid" 2>/dev/null)" || continue
    runuser -u "$user" -- env DBUS_SESSION_BUS_ADDRESS="unix:path=$bus" XDG_RUNTIME_DIR="/run/user/$uid" \
        sh -c "gsettings set org.gnome.desktop.background picture-uri 'file://$DATED' ; \
               gsettings set org.gnome.desktop.background picture-uri-dark 'file://$DATED'" \
        2>/dev/null && echo "[wallpaper] session de $user : $DATED" && _pushed=1
done
# DING (desktop icons) sometimes keeps the previous image over the wallpaper:
# restart it (the GNOME extension restarts it at once).
[ "${_pushed:-0}" = "1" ] && pkill -f "extensions/ding@rastersoft.com/app/ding.js" 2>/dev/null
exit 0
```

2.282 /opt/GSM/osmo-operator/tools/osmo-wg.sh

5994 octets, 159 lignes → 159 lignes

```
#!/bin/bash
# ============================================================================
# osmo-wg.sh - THE WAN WIREGUARD TUNNEL (wiki/WAN.md §3) IN ONE COMMAND.
#
# The addressing plan is fixed (10.99.0.0/24):
#   10.99.0.1   the hub (co-located inter-STP), the only one that LISTENS (udp 51820)
#   10.99.0.N   the other nodes, which call the hub
# One command per node, and one line to copy on the hub per node added.
# The keys are generated HERE, on the machine: never in the ISO (every live
# system would share the same keys).
#
#   wg 1          (= wg 10.99.0.1)       this node becomes the hub
#   wg N HUB_IP[:PORT] HUB_PUB           this node becomes 10.99.0.N (N = 2..254)
#                                        and joins the hub; prints the
#                                        "wg peer ..." line to type on the hub
#   wg peer PUBLIC_KEY N                 (on the hub) accept this node
```

```
#   wg up | wg down                     bring the installed tunnel up / down
#   wg status | wg pub                  N or 10.99.0.N, either form anywhere
# "wg" is the function of /etc/profile.d/97-osmo-wg.sh: it routes these forms
# here, everything else (wg show, wg genkey...) goes to the real wg.
# ============================================================================
set -u

IF="${OSMO_WG_IF:-wg0}"
PORT="${OSMO_WG_PORT:-51820}"
NET=10.99.0
HUB="$NET.1"
DIR=/etc/wireguard
CONF="$DIR/$IF.conf"

RED='\033[0;31m'; GREEN='\033[0;32m'; CYAN='\033[0;36m'; NC='\033[0m'
_ok()  { echo -e "  ${GREEN}✓${NC} $*"; }
_say() { echo -e "  ${CYAN}→${NC} $*"; }
_err() { echo -e "  ${RED}✗${NC} $*" >&2; }

usage() { sed -n '13,20p' "$0" | sed 's/^# \{0,1\}//'; exit "${1:-1}"; }

case "${1:-}" in ""|-h|--help) usage 0 ;; esac
if [ "$(id -u)" -ne 0 ]; then
    exec sudo "$0" "$@"
fi
command -v wg >/dev/null 2>&1 && command -v wg-quick >/dev/null 2>&1 \
    || { _err "wireguard-tools absent : apt install wireguard-tools"; exit 1; }

# _ip_ok 10.99.0.N: N in 1..254
_ip_ok() {
    [[ "$1" =~ ^10\.99\.0\.([0-9]{1,3})$ ]] && [ "${BASH_REMATCH[1]}" -ge 1 ] && [ "${BASH_REMATCH[1]}" -le 254 ]
}

_cles() {
    install -d -m700 "$DIR"
    if [ ! -s "$DIR/$IF.priv" ]; then
        (umask 077; wg genkey > "$DIR/$IF.priv")
        _ok "cle privee generee ($DIR/$IF.priv)"
    fi
    wg pubkey < "$DIR/$IF.priv" > "$DIR/$IF.pub"
}

# SCTP (M3UA, SGs) goes through the tunnel: opened to the subnet, no more.
_parefeu() {
    iptables -C INPUT -i "$IF" -s "$NET.0/24" -j ACCEPT 2>/dev/null \
        || iptables -I INPUT -i "$IF" -s "$NET.0/24" -j ACCEPT
    if [ "${1:-}" = hub ]; then
        iptables -C INPUT -p udp --dport "$PORT" -j ACCEPT 2>/dev/null \
            || iptables -I INPUT -p udp --dport "$PORT" -j ACCEPT
    fi
    if command -v ufw >/dev/null 2>&1 && ufw status 2>/dev/null | grep -q '^Status: active'; then
        ufw allow in on "$IF" from "$NET.0/24" >/dev/null
        [ "${1:-}" = hub ] && ufw allow "$PORT/udp" >/dev/null
    fi
}

_monter() {
    systemctl enable "wg-quick@$IF" >/dev/null 2>&1
    systemctl restart "wg-quick@$IF" || { _err "wg-quick@$IF ne demarre pas : journalctl -u wg-quick@$IF"; exit 1;
    ↪ }
    _ok "$IF monte, et a chaque demarrage (wg-quick@$IF)"
}

_ecrire() {  # _ecrire <contenu> : remplace wg0.conf, l ancien garde en .bak
    [ -f "$CONF" ] && cp -a "$CONF" "$CONF.bak" && _say "ancien $CONF garde en $CONF.bak"
    systemctl stop "wg-quick@$IF" 2>/dev/null
```

```
    (umask 077; printf '%s\n' "$1" > "$CONF")
}

hub() {
    _cles
    # SaveConfig: the peers added by "wg peer" survive a restart
    _ecrire "[Interface]
Address    = $HUB/24
ListenPort = $PORT
PrivateKey = $(cat "$DIR/$IF.priv")
SaveConfig = true"
    _parefeu hub
    _monter
    echo
    _say "HUB $HUB pret. Sur chaque autre noeud :"
    echo "      wg N <IP publique de cette machine>:$PORT $(cat "$DIR/$IF.pub")      (N = 2..254 : 10.99.0.N)"
}

noeud() {
    local ip="$1" ep="$2" pub="$3"
    [[ "$ep" == *:* ]] || ep="$ep:$PORT"
    [[ "$pub" =~ ^[A-Za-z0-9+/]{42,43}=$ ]] || { _err "cle publique du hub invalide : $pub"; exit 1; }
    _cles
    _ecrire "[Interface]
Address    = $ip/24
PrivateKey = $(cat "$DIR/$IF.priv")

[Peer]
# the hub ($HUB)
PublicKey  = $pub
Endpoint   = $ep
AllowedIPs = $NET.0/24
PersistentKeepalive = 25"
    _parefeu
    _monter
    echo
    _say "NOEUD $ip pret. Sur le hub ($HUB), tapez :"
    echo "      wg peer $(cat "$DIR/$IF.pub") ${ip##*.}"
    _say "puis ici : ping -c3 $HUB"
}

peer() {
    local pub="$1" ip="$2"
    [ -f "$CONF" ] && grep -q "^Address *= *$HUB/" "$CONF" \
        || { _err "ce noeud n est pas le hub ($HUB) : « wg 1 » d abord"; exit 1; }
    wg show "$IF" >/dev/null 2>&1 || _monter
    wg set "$IF" peer "$pub" allowed-ips "$ip/32" || exit 1
    wg-quick save "$IF" >/dev/null 2>&1
    _ok "pair $ip accepte (et enregistre dans $CONF)"
}

# "wg 5" = "wg 10.99.0.5" (and "wg peer KEY 5")
_ip() { [[ "$1" =~ ^[0-9]{1,3}$ ]] && echo "$NET.$1" || echo "$1"; }
case "${1:-}" in
    peer) [ $# -ge 3 ] && set -- "$1" "$2" "$(_ip "$3")" "${@:4}" ;;
    [0-9]*) set -- "$(_ip "$1")" "${@:2}" ;;
esac

case "${1:-}" in
    ""|-h|--help) usage 0 ;;
    up)     [ -f "$CONF" ] || { _err "pas de $CONF : wg 1 (hub) ou wg N ... (noeud)"; exit 1; }
            _monter; wg show "$IF" | sed -n '1,4p' ;;
    status) wg show "$IF" 2>/dev/null || _say "$IF absent" ;;
    down)   systemctl disable --now "wg-quick@$IF" && _ok "$IF arrete (et plus au demarrage)" ;;
    pub)    [ -s "$DIR/$IF.pub" ] && cat "$DIR/$IF.pub" || { _err "pas de cle : wg 1 (hub) ou wg N ... (noeud)";
     ↪  exit 1; } ;;
```

```
    peer)
        [ $# -eq 3 ] && _ip_ok "$3" && [ "$3" != "$HUB" ] || usage
        peer "$2" "$3" ;;
    *)
        _ip_ok "$1" || usage
        if [ "$1" = "$HUB" ]; then
            [ $# -eq 1 ] || usage
            hub
        else
            [ $# -eq 3 ] || { _err "un noeud doit connaitre le hub : wg ${1##*.} HUB_IP[:PORT]
            ↪  CLE_PUBLIQUE_DU_HUB"; exit 1; }
            noeud "$1" "$2" "$3"
        fi ;;
esac
```

2.283 /opt/GSM/osmo-operator/tools/osmo-wireshark-root.sh

7411 octets, 120 lignes → 120 lignes

```
#!/bin/bash
# osmo-wireshark-root - Wireshark as root on the bench: 2G, 4G and the phone
# (signalling, call, SMS, data) at once.
#
# ONE ICON, ALREADY LISTENING TO THE RIGHT TRAFFIC (wireshark-common/
# install-setuid is false on purpose, so capture needs root). Here:
#   - root through pkexec, hence the GRAPHICAL PASSWORD PROMPT of the GNOME
#     polkit agent (--disable-internal-agent forbids the text agent);
#     xhost lets root open its window on the X11 display of the session,
#     QT_QPA_PLATFORM=xcb forces it through X even under Wayland;
#   - immediate capture (-k) on all interfaces (-i any: the loopback where
#     the whole core lives, plus the network where GSMTAP and the Abis of a
#     real BTS go) with the bench BPF filter (-f, what is RECORDED):
#       whole SCTP      M3UA of osmo-stp (2905), SGs (29118), S1AP (36412),
#                       Diameter when it runs over SCTP - A, Lb, S1 signalling;
#       GSM             GSMTAP udp 4729 (the air, and the osmocom logs in gsmtap),
#                       Abis/IPA tcp 3002-3003, Gb udp 23000-23001 (osmo-pcu),
#                       GTP udp 2123/2152 (sgsn <-> ggsn), MGCP udp 2427-2428,
#                       SIP 5060-5061, RTP of osmo-mgw (4002-16001) and of
#                       asterisk (30000-30199);
#       LTE             S1AP through SCTP, GTP-C/U udp 2123/2152 (mme, sgwc,
#                       smf, upf), PFCP udp 8805, Diameter tcp 3868 (S6a, Gx),
#                       SBI http tcp 7777 (nrf <-> smf).
#     Left out on purpose: the VTYs (tcp 42xx), the srsRAN ZeroMQ radio
#     (tcp 2000/2001: IQ samples, MB per second), Prometheus.
#   - a DISPLAY FILTER (-Y, what is SHOWN: everything stays in the capture,
#     one click in the bar removes it) that only keeps the 2G / 4G network
#     and the phone signalling:
#       2G   GSMTAP (Um: RR, MM, CC, SMS, GMM/SM of the phone), LAPDm,
#            Abis RSL/OML, A (SCTP/M3UA/SCCP/BSSAP/BSSMAP/DTAP), GSUP
#            (osmo-hlr), Gb (NS/BSSGP/LLC/SNDCP), GTP, MGCP, SMPP;
#       4G   S1AP + NAS-EPS (EMM/ESM), GTPv2 (S11), GTP-U, PFCP, Diameter
#            (S6a, Gx), SBI HTTP/2, SGs (CSFB and SMS over SGs), and LTE RRC /
#            MAC / RLC / PDCP when opening an srsRAN pcap;
#       call    DTAP CC, MGCP, SIP, RTP / RTCP / DTMF (rtpevent);
#       SMS     gsm_sms wherever it goes (DTAP, SGs, NAS), SMPP.
#     Without the noise that drowns all this: SCTP SACK / HEARTBEAT, M3UA
#     BEAT, Diameter Device-Watchdog, GTP / GTPv2 Echo, PFCP Heartbeat, and
#     on the air the System Information and Measurement Report messages.
#   - FILTER BUTTONS in the Wireshark bar (Banc, 2G, 4G, Appel, SMS, Data,
#     Attach) to see only one thing: installed in root's profile
#     (~root/.config/wireshark/dfilter_buttons) if there are none yet.
#   - with arguments (a .pcap opened from the desktop, or the options passed
#     by launch.sh), nothing is touched: they are passed through unchanged.
# /usr/local/bin/wireshark is a link to this script: "wireshark" typed at the
```

```
# keyboard or from the icon is this one. The real binary stays in /usr/bin.
# OSMO_WS_FILTRE (BPF), OSMO_WS_DFILTRE (display; empty = show everything) and
# OSMO_WS_IFACE change the filter, the display or the interface.
set -u
WS=/usr/bin/wireshark
[ -x "$WS" ] || { command -v zenity >/dev/null 2>&1 && \
    zenity --error --text="Wireshark introuvable (apt install wireshark)" 2>/dev/null; exit 1; }

# What is recorded (BPF).
FILTRE="${OSMO_WS_FILTRE:-sctp\
 or udp port 4729 or tcp port 3002 or tcp port 3003 or udp port 23000 or udp port 23001\
 or udp port 2123 or udp port 2152 or udp port 2427 or udp port 2428\
 or port 5060 or port 5061 or udp portrange 4002-16001 or udp portrange 30000-30199\
 or udp port 8805 or tcp port 3868 or tcp port 7777}"
IFACE="${OSMO_WS_IFACE:-any}"

# What is shown (Wireshark display filter). The parentheses: first the bench
# protocols, then, subtracted, the heartbeat noise.
D_PROTOS="sctp || m3ua || sccp || bssap || gsm_a.bssmap || gsm_a.dtap || gsm_a.rr || gsm_a.ccch || gsm_a.sacch ||
 ↪  gsm_a.gm || gsm_sms || gsm_map || tcap || gsup || sgsap || smpp\
 || gsmtap || lapdm || gsm_abis_rsl || gsm_abis_oml\
 || gprs-ns || bssgp || llcgprs || sndcp || gtp || gtpv2 || pfcp\
 || s1ap || nas-eps || lte_rrc || mac-lte || rlc-lte || pdcp-lte || diameter || http2\
 || mgcp || sip || rtp || rtcp || rtpevent"
D_BRUIT="(sctp && !(sctp.chunk_type == 0) && sctp.chunk_type in {3, 4, 5})\
 || (m3ua.message_class == 3 && m3ua.message_type in {3, 6})\
 || diameter.cmd.code == 280\
 || gtp.message in {1, 2} || gtpv2.message_type in {1, 2} || pfcp.msg_type in {1, 2}\
 || gsm_a.dtap.msg_rr_type in {0x00, 0x02, 0x03, 0x05, 0x06, 0x07, 0x15, 0x19, 0x1a, 0x1b, 0x1c, 0x1d, 0x1e}"
DFILTRE="${OSMO_WS_DFILTRE-($D_PROTOS) && !($D_BRUIT)}"

# The filter bar buttons, in root's profile (root is the one capturing). An
# existing file (the user made their own) is respected.
osmo_ws_boutons() {
    local f="${HOME:-/root}/.config/wireshark/dfilter_buttons"
    [ -e "$f" ] && return 0
    mkdir -p "${f%/*}" 2>/dev/null || return 0
    cat > "$f" <<BTN
# Filter buttons of the osmo-operator bench (osmo-wireshark-root).
"TRUE","Banc","$DFILTRE","Reseau 2G + 4G et telephone, sans les battements de coeur"
"TRUE","2G","(gsmtap || lapdm || gsm_abis_rsl || gsm_abis_oml || bssap || gsm_a.bssmap || gsm_a.dtap || gsup ||
 ↪  gprs-ns || bssgp || llcgprs || sndcp || mgcp || smpp) && !($D_BRUIT)","Um, Abis, A, Gb, GSUP, MGCP"
"TRUE","4G","(s1ap || nas-eps || gtpv2 || pfcp || diameter || http2 || sgsap || lte_rrc || mac-lte || pdcp-lte) &&
 ↪  !($D_BRUIT)","S1, S11, Sx, S6a/Gx, SBI, SGs"
"TRUE","Appel","gsm_a.dtap.protocol_discriminator == 3 || mgcp || sip || rtpevent || sgsap ||
 ↪  nas-eps.nas_msg_emm_type == 0x4c || gsm_a.bssmap.msgtype in {0x01, 0x02, 0x03, 0x20, 0x21, 0x22, 0x52}","CC
 ↪  (DTAP), MGCP, SIP, DTMF, CSFB (SGs, Extended Service Request), paging, assignation et liberation BSSMAP"
"TRUE","SMS","gsm_sms || gsm_a.dtap.protocol_discriminator == 9 || smpp || sgsap.msg_type in {0x07, 0x08}","SMS
 ↪  en DTAP, SGs, NAS et SMPP"
"TRUE","Data","gsm_a.gm || llcgprs || sndcp || bssgp || gprs-ns || gtp || gtpv2 || pfcp ||
 ↪  nas-eps.nas_msg_esm_type || gsm_a.dtap.protocol_discriminator in {8, 10}","GMM/SM, Gb, GTP, PFCP, ESM"
"TRUE","Attach","gsm_a.dtap.protocol_discriminator == 5 || gsm_a.dtap.msg_gmm_type || nas-eps.nas_msg_emm_type ||
 ↪  gsup || diameter.cmd.code in {316, 318} || gsm_a.bssmap.msgtype in {0x52, 0x57, 0x20}","MM, GMM, EMM, GSUP,
 ↪  S6a (ULR/AIR), Paging / Complete Layer 3 / Clear"
BTN
    return 0
}

if [ $# -eq 0 ]; then
    set -- -i "$IFACE" -k -f "$FILTRE"
    [ -n "$DFILTRE" ] && set -- "$@" -Y "$DFILTRE"
fi

if [ "$(id -u)" -eq 0 ]; then
    osmo_ws_boutons
    exec "$WS" "$@"
```

```
fi

# Not root: re-run ourselves as root (to install the buttons in root's
# profile), then Wireshark.
MOI="$(readlink -f "$0" 2>/dev/null || echo "$0")"
[ -x "$MOI" ] || MOI="$WS"
command -v xhost >/dev/null 2>&1 && xhost +SI:localuser:root >/dev/null 2>&1
if command -v pkexec >/dev/null 2>&1; then
    exec pkexec --disable-internal-agent env \
        DISPLAY="${DISPLAY:-:0}" \
        XAUTHORITY="${XAUTHORITY:-$HOME/.Xauthority}" \
        WAYLAND_DISPLAY="${WAYLAND_DISPLAY:-}" \
        XDG_RUNTIME_DIR="${XDG_RUNTIME_DIR:-}" \
        QT_QPA_PLATFORM=xcb \
        HOME=/root \
        "$MOI" "$@"
fi
exec sudo -E HOME=/root "$MOI" "$@"
```

2.284 /opt/GSM/osmo-operator/tools/overlay-draw.py

24696 octets, 635 lignes → 635 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# overlay-draw.py - Inkscape-style vector drawing, as a transparent OVERLAY
# ALWAYS ON TOP of the desktop. Everything fits in ONE window: the full-screen
# canvas + a vertical toolbar stuck to the BOTTOM RIGHT.
#
# Pen (default tool): HOLD the left click to draw freehand; the stroke stays
# on top and can be erased (eraser, or the "clear all" box in the bottom
# right corner).
#
# Toolbar (from bottom to top, stuck to the right and bottom edges):
#   [C]  Clear all   (red box, in the corner)
#   Eraser · Colour (click = next colour) · − · +  (thickness)
#   Ellipse · Rectangle · Segment · Pen · Move · Undo
#   Click-through · Quit
#
# Wheel     = stroke thickness.
# Keyboard  : b pen · l segment · r rect · e ellipse · x eraser · s move
#             z undo · Del deletes the selection · Space click-through · Esc/q quit
#
# X11 + compositor (GNOME/mutter) required for per-pixel transparency.
import gi
gi.require_version('Gtk', '3.0')
from gi.repository import Gtk, Gdk, GLib
import cairo
import math
import copy
import time
import os

GRAB_R = 14
HIT_TOL = 8
BS = 46          # side of a toolbar button
GAP = 6          # gap between buttons
MARGIN = 12      # margin to the right / bottom edges
DOT_R = 24       # radius of the reminder dot (dormant state)
DOT_MARGIN = 22  # margin of the dot to the edges
HOLD_MS = 220    # hold time of the 2nd click to wake up from the dot

SWATCHES = [
    (0.00, 0.82, 1.00), (1.00, 0.58, 0.00), (0.20, 0.90, 0.35),
```

```
    (1.00, 0.25, 0.35), (1.00, 0.90, 0.10), (0.75, 0.45, 1.00),
    (1.00, 1.00, 1.00), (0.05, 0.05, 0.05),
]

# toolbar order, from the bottom-right CORNER UPWARDS
TOOLBAR = ["clear", "eraser", "color", "width_dn", "width_up",
           "ellipse", "rect", "line", "pencil", "select", "undo",
           "pass", "quit"]
TOOL_IDS = {"pencil", "line", "rect", "ellipse", "eraser", "select", "node"}


def dist(ax, ay, bx, by):
    return math.hypot(ax - bx, ay - by)


def dist_point_seg(px, py, ax, ay, bx, by):
    dx, dy = bx - ax, by - ay
    L2 = dx * dx + dy * dy
    if L2 == 0:
        return dist(px, py, ax, ay)
    t = max(0.0, min(1.0, ((px - ax) * dx + (py - ay) * dy) / L2))
    return dist(px, py, ax + t * dx, ay + t * dy)


def obj_bbox(o):
    xs = [p[0] for p in o["pts"]]; ys = [p[1] for p in o["pts"]]
    return min(xs), min(ys), max(xs), max(ys)


def obj_hit(o, x, y, tol):
    t = o["pts"]; typ = o["type"]; tol = tol + o["width"] / 2.0
    if typ in ("line", "path"):
        if len(t) == 1:
            return dist(x, y, t[0][0], t[0][1]) <= tol
        return any(dist_point_seg(x, y, a[0], a[1], b[0], b[1]) <= tol
                   for a, b in zip(t, t[1:]))
    x0, y0, x1, y1 = obj_bbox(o)
    if o.get("fill") and x0 - tol <= x <= x1 + tol and y0 - tol <= y <= y1 + tol:
        return True
    if typ == "rect":
        v = (abs(x - x0) <= tol or abs(x - x1) <= tol) and (y0 - tol <= y <= y1 + tol)
        h = (abs(y - y0) <= tol or abs(y - y1) <= tol) and (x0 - tol <= x <= x1 + tol)
        return v or h
    if typ == "ellipse":
        cx, cy = (x0 + x1) / 2, (y0 + y1) / 2
        rx, ry = max((x1 - x0) / 2, 1), max((y1 - y0) / 2, 1)
        d = math.sqrt(((x - cx) / rx) ** 2 + ((y - cy) / ry) ** 2)
        return abs(d - 1.0) <= (tol / min(rx, ry) + 0.06)
    return False


def obj_translate(o, dx, dy):
    o["pts"] = [(px + dx, py + dy) for px, py in o["pts"]]


def _hex(c):
    return "#%02x%02x%02x" % (int(c[0] * 255), int(c[1] * 255), int(c[2] * 255))


class Overlay(Gtk.Window):
    def __init__(self):
        super().__init__(type=Gtk.WindowType.TOPLEVEL)
        self.set_title("overlay-draw")
        self.set_app_paintable(True)
        screen = self.get_screen()
        vis = screen.get_rgba_visual()
```

```
    self.has_alpha = vis is not None
    if self.has_alpha:
        self.set_visual(vis)
    self.set_decorated(False)
    self.set_keep_above(True)
    self.set_skip_taskbar_hint(True)
    self.set_skip_pager_hint(True)
    self.stick()

    # document state
    self.objects = []
    self.selected = None
    self.tool = "pencil"           # the pen by default
    self.color_idx = 0
    self.color = SWATCHES[0]
    self.width = 4.0
    self.fill = False
    self.passthrough = False
    self.undo_stack = []
    self.redo_stack = []
    self.status = ""
    self.status_until = 0
    self.dormant = False           # tucked away in the corner dot
    self._dot_pressed = False      # 2nd click on the dot held?

    # interaction state
    self.drag = None
    self.start = None
    self.temp = None
    self.pen_pts = None
    self.buttons = []              # [(id, x, y, w, h)] recomputed on render

    self.add_events(
        Gdk.EventMask.BUTTON_PRESS_MASK | Gdk.EventMask.BUTTON_RELEASE_MASK
        | Gdk.EventMask.POINTER_MOTION_MASK | Gdk.EventMask.KEY_PRESS_MASK
        | Gdk.EventMask.SCROLL_MASK)
    self.connect("draw", self.on_draw)
    self.connect("button-press-event", self.on_press)
    self.connect("button-release-event", self.on_release)
    self.connect("motion-notify-event", self.on_motion)
    self.connect("key-press-event", self.on_key)
    self.connect("scroll-event", self.on_scroll)
    self.connect("destroy", Gtk.main_quit)

    self.fullscreen()
    self.show_all()
    GLib.timeout_add(300, lambda: (self.set_keep_above(True), False)[1])

# ── toolbar: geometry and clicks ──────────────────────────────────
def layout_buttons(self, W, H):
    self.buttons = []
    x = W - MARGIN - BS
    for i, bid in enumerate(TOOLBAR):
        y = H - MARGIN - BS - i * (BS + GAP)
        self.buttons.append((bid, x, y, BS, BS))

def button_at(self, x, y):
    for bid, bx, by, bw, bh in self.buttons:
        if bx <= x <= bx + bw and by <= y <= by + bh:
            return bid
    return None

def toolbar_region(self):
    reg = cairo.Region()
    for _bid, bx, by, bw, bh in self.buttons:
        reg.union(cairo.RectangleInt(int(bx), int(by), int(bw), int(bh)))
```

```
    return reg

def full_region(self):
    # this binding refuses None for "no shape": input is given back to the
    # whole window with a full region (equivalent to no shape).
    a = self.get_allocation()
    return cairo.Region(cairo.RectangleInt(0, 0, max(a.width, 1), max(a.height, 1)))

def do_button(self, bid):
    if bid in TOOL_IDS:
        self.tool = bid
        self.finish_pen()
    elif bid == "clear":
        self.clear_all()
    elif bid == "color":
        self.color_idx = (self.color_idx + 1) % len(SWATCHES)
        self.color = SWATCHES[self.color_idx]
        self.apply_style()
    elif bid == "width_dn":
        self.set_width(self.width - 1)
    elif bid == "width_up":
        self.set_width(self.width + 1)
    elif bid == "undo":
        self.undo()
    elif bid == "pass":
        self.set_passthrough(not self.passthrough)
    elif bid == "quit":
        self.go_dormant()
    self.queue_draw()

# ── dormant state: everything tucks into a dot in the bottom-right corner ─
def dot_center(self):
    a = self.get_allocation()
    return a.width - DOT_MARGIN - DOT_R, a.height - DOT_MARGIN - DOT_R

def dot_region(self):
    cx, cy = self.dot_center()
    r = DOT_R + 6
    return cairo.Region(cairo.RectangleInt(int(cx - r), int(cy - r),
                                           int(2 * r), int(2 * r)))

def go_dormant(self):
    # the cross (X) does not quit: it tucks away. The drawing disappears, only
    # the dot remains, always on top. The work is kept in memory.
    self.dormant = True
    self.drag = None
    self.temp = None
    self.pen_pts = None
    gw = self.get_window()
    if gw is not None:
        gw.input_shape_combine_region(self.dot_region(), 0, 0)
    self.queue_draw()

def wake(self):
    self.dormant = False
    self.passthrough = False
    gw = self.get_window()
    if gw is not None:
        gw.input_shape_combine_region(self.full_region(), 0, 0)
    self.set_keep_above(True)
    self.queue_draw()

def _dot_wake_if_held(self):
    # called HOLD_MS after the 2nd click: wake up only if it is still
    # pressed -> "double-click whose second press stays long".
    if self.dormant and self._dot_pressed:
```

```
        self.wake()
    return False

# ── document helpers ──────────────────────────────────────────────
def new_obj(self, typ, pts):
    return {"type": typ, "pts": pts, "stroke": self.color,
            "width": self.width, "fill": self.color if self.fill else None}

def topmost_at(self, x, y):
    for o in reversed(self.objects):
        if obj_hit(o, x, y, HIT_TOL):
            return o
    return None

def set_width(self, v):
    self.width = max(1.0, min(40.0, v))
    self.apply_style()
    self.flash("épaisseur %d" % self.width)

def apply_style(self):
    o = self.selected
    if o is not None and o in self.objects:
        o["stroke"] = self.color
        o["width"] = self.width
        self.queue_draw()

def flash(self, msg):
    self.status = msg
    self.status_until = time.time() + 1.5
    self.queue_draw()

# ── history ───────────────────────────────────────────────────────
def push_undo(self):
    self.undo_stack.append(copy.deepcopy(self.objects))
    if len(self.undo_stack) > 120:
        self.undo_stack.pop(0)
    self.redo_stack.clear()

def undo(self):
    if self.undo_stack:
        self.redo_stack.append(copy.deepcopy(self.objects))
        self.objects = self.undo_stack.pop()
        self.selected = None

def redo(self):
    if self.redo_stack:
        self.undo_stack.append(copy.deepcopy(self.objects))
        self.objects = self.redo_stack.pop()
        self.selected = None

def delete_selected(self):
    if self.selected in self.objects:
        self.push_undo()
        self.objects.remove(self.selected)
        self.selected = None
        self.queue_draw()

def clear_all(self):
    if self.objects:
        self.push_undo()
        self.objects = []
        self.selected = None
        self.flash("tableau effacé")

# ── click-through: only the toolbar stays clickable ──────────────
def set_passthrough(self, on):
```

```
    self.passthrough = on
    gw = self.get_window()
    if gw is not None:
        gw.input_shape_combine_region(self.toolbar_region() if on else self.full_region(), 0, 0)
    self.flash("clic-à-travers " + ("ON" if on else "OFF"))

# ── mouse input ─────────────────────────────────────────────────
def on_press(self, w, e):
    x, y = e.x, e.y
    if self.dormant:
        cx, cy = self.dot_center()
        if e.button == 1 and dist(x, y, cx, cy) <= DOT_R + 6:
            self._dot_pressed = True
            if e.type == Gdk.EventType._2BUTTON_PRESS:
                GLib.timeout_add(HOLD_MS, self._dot_wake_if_held)
        return True
    bid = self.button_at(x, y)
    if bid is not None and e.button == 1:
        self.do_button(bid)
        return True
    if self.passthrough:
        return False
    if e.button == 3:
        self.finish_pen()
        return True
    if e.button != 1:
        return False

    tool = self.tool
    if tool == "select":
        o = self.topmost_at(x, y)
        self.selected = o
        if o is not None:
            self.push_undo()
            self.drag = ("move",)
            self.start = (x, y)
        self.queue_draw()
        return True
    if tool == "eraser":
        o = self.topmost_at(x, y)
        if o is not None:
            self.push_undo()
            self.objects.remove(o)
            if self.selected is o:
                self.selected = None
        self.drag = ("erase",)
        self.queue_draw()
        return True
    if tool == "pencil":                       # the pen
        self.push_undo()
        self.temp = self.new_obj("path", [(x, y)])
        self.drag = ("free",)
        return True
    if tool in ("rect", "ellipse", "line"):
        self.push_undo()
        self.temp = self.new_obj(tool, [(x, y), (x, y)])
        self.drag = ("create",)
        self.start = (x, y)
        return True
    return False

def on_motion(self, w, e):
    if self.dormant:
        return False
    if self.drag is None:
        return False
```

```
    x, y = e.x, e.y
    k = self.drag[0]
    if k == "move" and self.selected:
        obj_translate(self.selected, x - self.start[0], y - self.start[1])
        self.start = (x, y)
    elif k == "free" and self.temp:
        self.temp["pts"].append((x, y))
    elif k == "create" and self.temp:
        self.temp["pts"][1] = (x, y)
    elif k == "erase":
        o = self.topmost_at(x, y)
        if o is not None:
            self.objects.remove(o)
    self.queue_draw()
    return True

def on_release(self, w, e):
    if self.dormant:
        self._dot_pressed = False
        return True
    if self.drag and self.drag[0] in ("free", "create") and self.temp:
        keep = True
        if self.temp["type"] == "line":
            p = self.temp["pts"]
            keep = dist(p[0][0], p[0][1], p[1][0], p[1][1]) >= 3
        if self.temp["type"] == "path" and len(self.temp["pts"]) < 2:
            keep = False
        if keep:
            self.objects.append(self.temp)
            self.selected = self.temp
    self.temp = None
    self.drag = None
    self.queue_draw()
    return True

def finish_pen(self):
    if self.pen_pts and len(self.pen_pts) >= 2:
        self.objects.append(self.new_obj("path", self.pen_pts[:]))
    self.pen_pts = None
    self.queue_draw()

def on_scroll(self, w, e):
    if e.direction == Gdk.ScrollDirection.UP:
        self.set_width(self.width + 1)
    elif e.direction == Gdk.ScrollDirection.DOWN:
        self.set_width(self.width - 1)
    return True

def on_key(self, w, e):
    k = e.keyval
    if k in (Gdk.KEY_Escape, Gdk.KEY_q):
        if self.pen_pts is not None:
            self.finish_pen()
        else:
            self.destroy()
    elif k in (Gdk.KEY_Delete, Gdk.KEY_BackSpace):
        self.delete_selected()
    elif k == Gdk.KEY_z:
        self.undo(); self.queue_draw()
    elif k == Gdk.KEY_y:
        self.redo(); self.queue_draw()
    elif k == Gdk.KEY_space:
        self.set_passthrough(not self.passthrough)
    else:
        m = {Gdk.KEY_b: "pencil", Gdk.KEY_l: "line", Gdk.KEY_r: "rect",
             Gdk.KEY_e: "ellipse", Gdk.KEY_x: "eraser", Gdk.KEY_s: "select"}
```

```
        if k in m:
            self.tool = m[k]; self.finish_pen(); self.queue_draw()
    return True

# ── object rendering ──────────────────────────────────────────────
def draw_obj(self, cr, o, ghost=False):
    pts = o["pts"]; r, g, b = o["stroke"]; a = 0.45 if ghost else 0.97
    cr.set_line_cap(cairo.LINE_CAP_ROUND); cr.set_line_join(cairo.LINE_JOIN_ROUND)

    def path():
        typ = o["type"]
        if typ in ("line", "path"):
            cr.move_to(*pts[0])
            for p in pts[1:]:
                cr.line_to(*p)
        elif typ == "rect":
            x0, y0, x1, y1 = obj_bbox(o); cr.rectangle(x0, y0, x1 - x0, y1 - y0)
        elif typ == "ellipse":
            x0, y0, x1, y1 = obj_bbox(o)
            cx, cy = (x0 + x1) / 2, (y0 + y1) / 2
            rx, ry = max((x1 - x0) / 2, .5), max((y1 - y0) / 2, .5)
            cr.save(); cr.translate(cx, cy); cr.scale(rx, ry)
            cr.arc(0, 0, 1, 0, 2 * math.pi); cr.restore()

    if o.get("fill") and o["type"] in ("rect", "ellipse", "path"):
        path(); fr, fg, fb = o["fill"]
        cr.set_source_rgba(fr, fg, fb, 0.3 if ghost else 0.5); cr.fill()
    path(); cr.set_source_rgba(0, 0, 0, 0.5)
    cr.set_line_width(o["width"] + 2.5); cr.stroke()
    path(); cr.set_source_rgba(r, g, b, a)
    cr.set_line_width(o["width"]); cr.stroke()

def on_draw(self, w, cr):
    a = self.get_allocation()

    # background
    cr.set_operator(cairo.OPERATOR_SOURCE)
    if self.dormant:
        cr.set_source_rgba(0, 0, 0, 0.0)      # only the dot
    else:
        cr.set_source_rgba(0, 0, 0, 0.0 if self.has_alpha else 0.08)
    cr.paint()
    cr.set_operator(cairo.OPERATOR_OVER)

    if self.dormant:
        self.draw_dot(cr)
        return False

    self.layout_buttons(a.width, a.height)
    for o in self.objects:
        self.draw_obj(cr, o)
    if self.temp:
        self.draw_obj(cr, self.temp, ghost=True)

    # selection
    sel = self.selected
    if sel is not None and sel in self.objects and self.tool == "select":
        x0, y0, x1, y1 = obj_bbox(sel)
        cr.set_dash([5, 4]); cr.set_line_width(1); cr.set_source_rgba(1, 1, 1, .8)
        cr.rectangle(x0 - 4, y0 - 4, x1 - x0 + 8, y1 - y0 + 8); cr.stroke(); cr.set_dash([])

    self.draw_toolbar(cr)

    if self.status and time.time() < self.status_until:
        self._text(cr, self.status, 18, 30, (1, 1, 1))
    return False
```

```
# ── the reminder dot (dormant state) ──────────────────────────────────
def draw_dot(self, cr):
    cx, cy = self.dot_center()
    cr.arc(cx, cy, DOT_R, 0, 2 * math.pi)          # dark halo
    cr.set_source_rgba(0, 0, 0, 0.5); cr.fill()
    cr.arc(cx, cy, DOT_R - 3, 0, 2 * math.pi)      # coloured dot
    cr.set_source_rgba(*self.color, 0.95); cr.fill()
    cr.arc(cx, cy, DOT_R - 3, 0, 2 * math.pi)
    cr.set_source_rgba(1, 1, 1, 0.85); cr.set_line_width(1.5); cr.stroke()
    # small pen stroke as a sign
    cr.set_source_rgba(1, 1, 1, 0.95)
    cr.set_line_width(2.4); cr.set_line_cap(cairo.LINE_CAP_ROUND)
    cr.move_to(cx - 10, cy + 5)
    cr.curve_to(cx - 3, cy - 10, cx + 3, cy + 10, cx + 10, cy - 5)
    cr.stroke()

# ── toolbar drawn in the canvas ───────────────────────────────────────
def draw_toolbar(self, cr):
    for bid, x, y, w, h in self.buttons:
        active = (bid == self.tool)
        # button background
        self._round_rect(cr, x, y, w, h, 9)
        if bid == "clear":
            cr.set_source_rgba(0.80, 0.12, 0.16, 0.92)
        elif active:
            cr.set_source_rgba(0.16, 0.55, 0.95, 0.95)
        else:
            cr.set_source_rgba(0.14, 0.15, 0.18, 0.85)
        cr.fill()
        self._round_rect(cr, x, y, w, h, 9)
        cr.set_source_rgba(1, 1, 1, 0.22); cr.set_line_width(1); cr.stroke()
        self.draw_icon(cr, bid, x, y, w, h)

def draw_icon(self, cr, bid, x, y, w, h):
    cx, cy = x + w / 2, y + h / 2
    white = (0.95, 0.95, 0.97)
    cr.set_line_width(2.2); cr.set_line_cap(cairo.LINE_CAP_ROUND)
    cr.set_source_rgba(*white, 1)

    if bid == "clear":
        self._glyph(cr, "C", cx, cy, 20)
    elif bid == "eraser":
        cr.save(); cr.translate(cx, cy); cr.rotate(-0.5)
        cr.rectangle(-11, -6, 22, 12); cr.stroke()
        cr.move_to(0, -6); cr.line_to(0, 6); cr.stroke(); cr.restore()
    elif bid == "color":
        cr.arc(cx, cy, 12, 0, 2 * math.pi)
        cr.set_source_rgba(*self.color, 1); cr.fill()
        cr.arc(cx, cy, 12, 0, 2 * math.pi)
        cr.set_source_rgba(1, 1, 1, .8); cr.set_line_width(1.5); cr.stroke()
    elif bid == "width_dn":
        cr.move_to(cx - 9, cy); cr.line_to(cx + 9, cy); cr.stroke()
    elif bid == "width_up":
        cr.move_to(cx - 9, cy); cr.line_to(cx + 9, cy)
        cr.move_to(cx, cy - 9); cr.line_to(cx, cy + 9); cr.stroke()
    elif bid == "ellipse":
        cr.save(); cr.translate(cx, cy); cr.scale(1.4, 1.0)
        cr.arc(0, 0, 9, 0, 2 * math.pi); cr.restore(); cr.stroke()
    elif bid == "rect":
        cr.rectangle(cx - 11, cy - 8, 22, 16); cr.stroke()
    elif bid == "line":
        cr.move_to(cx - 11, cy + 9); cr.line_to(cx + 11, cy - 9); cr.stroke()
    elif bid == "pencil":                          # pen: small wave
        cr.move_to(cx - 12, cy + 6)
        cr.curve_to(cx - 4, cy - 12, cx + 4, cy + 12, cx + 12, cy - 6)
```

```
            cr.stroke()
        elif bid == "select":
            cr.move_to(cx - 8, cy - 10); cr.line_to(cx - 8, cy + 8)
            cr.line_to(cx - 3, cy + 3); cr.line_to(cx + 1, cy + 11)
            cr.line_to(cx + 4, cy + 9); cr.line_to(cx, cy + 1)
            cr.line_to(cx + 7, cy + 1); cr.close_path(); cr.stroke()
        elif bid == "undo":
            cr.arc(cx, cy + 1, 9, math.radians(40), math.radians(300)); cr.stroke()
            cr.move_to(cx - 9, cy - 4); cr.line_to(cx - 12, cy + 2)
            cr.line_to(cx - 4, cy + 1); cr.close_path(); cr.fill()
        elif bid == "pass":
            cr.move_to(cx - 11, cy); cr.line_to(cx + 8, cy)
            cr.move_to(cx + 2, cy - 6); cr.line_to(cx + 10, cy); cr.line_to(cx + 2, cy + 6)
            cr.stroke()
            if self.passthrough:
                cr.set_source_rgba(0.2, 0.9, 0.35, 1)
                cr.arc(x + w - 9, y + 9, 4, 0, 2 * math.pi); cr.fill()
        elif bid == "quit":
            cr.move_to(cx - 8, cy - 8); cr.line_to(cx + 8, cy + 8)
            cr.move_to(cx + 8, cy - 8); cr.line_to(cx - 8, cy + 8); cr.stroke()

        # thickness value shown on the − / + buttons
        if bid == "width_up":
            self._text(cr, "%d" % self.width, x + 4, y - 4, (0.8, 0.85, 0.9), 12)

    def _round_rect(self, cr, x, y, w, h, r):
        cr.new_sub_path()
        cr.arc(x + w - r, y + r, r, -math.pi / 2, 0)
        cr.arc(x + w - r, y + h - r, r, 0, math.pi / 2)
        cr.arc(x + r, y + h - r, r, math.pi / 2, math.pi)
        cr.arc(x + r, y + r, r, math.pi, 1.5 * math.pi)
        cr.close_path()

    def _glyph(self, cr, s, cx, cy, size):
        cr.select_font_face("sans-serif", cairo.FONT_SLANT_NORMAL, cairo.FONT_WEIGHT_BOLD)
        cr.set_font_size(size)
        xb, yb, tw, th, _, _ = cr.text_extents(s)
        cr.move_to(cx - tw / 2 - xb, cy - yb - th / 2 + th)
        cr.show_text(s)

    def _text(self, cr, s, x, y, rgb, size=15):
        cr.select_font_face("monospace", cairo.FONT_SLANT_NORMAL, cairo.FONT_WEIGHT_BOLD)
        cr.set_font_size(size)
        cr.move_to(x + 1, y + 1); cr.set_source_rgba(0, 0, 0, .8); cr.show_text(s)
        cr.move_to(x, y); cr.set_source_rgba(*rgb, 1); cr.show_text(s)

    # ── export (bonus, key not mapped in the toolbar to stay compact) ──
    def export(self):
        ts = time.strftime("%Y%m%d-%H%M%S")
        base = os.path.expanduser("~/overlay-draw-%s" % ts)
        a = self.get_allocation()
        surf = cairo.ImageSurface(cairo.FORMAT_ARGB32, a.width, a.height)
        cr = cairo.Context(surf)
        for o in self.objects:
            self.draw_obj(cr, o)
        surf.write_to_png(base + ".png")
        self.flash("exporté %s.png" % base)


def main():
    Overlay()
    Gtk.main()


if __name__ == "__main__":
    main()
```

2.285 /opt/GSM/osmo-operator/tools/paysage-render.py

22882 octets, 451 lignes → 448 lignes (1 groupes compactés)

```
#!/usr/bin/env python3
# paysage-render.py - ONE scene, TWO images: the bench wallpaper and the
# illustration of the central inset, cropped out of it.
#
#   python3 tools/paysage-render.py [--scale 2] \
#       --fond configs/gsm-lab-fond.jpg --encart configs/gsm-lab-wallpaper.jpg
#
# Both images come from the SAME drawing, so the inset does not look like a
# hole in the wallpaper:
#
#   - the WALLPAPER is the whole scene, in 16:9 (3840x2160 at --scale 2): a
#     low-poly landscape in the manner of the Ubuntu wallpaper "Province of the
#     south of France" (Delaunay triangulation over a smooth painting, flat
#     facets), with the bench subject: three pylons, the sun, the hills;
#   - the INSET is the exact crop of the box (510,220 .. 1410,1010) of the
#     1920x1080 frame in that scene. osmo-launcher.py / osmo-panel.py place the
#     inset window at that box, projected on the screen with the SAME
#     transformation GNOME applies to the wallpaper in "zoom" mode: the inset
#     image therefore lands pixel for pixel on the wallpaper it covers, whatever
#     the screen. osmo-fft-snap.py fits it "whole" into W x H: since it already
#     has that aspect ratio, no bands.
#
# The composition is designed for the bench screen: the big tree on the left,
# behind the dock and the icons; the village and the cypresses on the right,
# behind the (translucent) conky; the pylons and the sun INSIDE the inset box;
# the foot of the image (fields) under the launcher bar.
#
# Depends only on Pillow, numpy and scipy (Delaunay), present on the ISO image
# as on the host. Deterministic rendering (fixed seed).

import argparse
import math
import random

import numpy as np
from PIL import Image, ImageDraw, ImageFilter
from scipy.spatial import Delaunay

FW, FH = 1920, 1080              # the reference frame (tools/osmo-launcher.py)
BOX = (510, 220, 1410, 1010)     # the inset (osmo-fft-snap.py: BOX)

# ── THE PALETTE (that of the "Province" wallpaper, measured) ──────────────
P = {
    "ciel_haut": (62, 140, 155), "ciel_bas": (138, 184, 186),
    "horizon": (226, 214, 176), "soleil": (252, 238, 160), "halo": (238, 200, 96),
    "nuage": (234, 228, 210), "nuage_clair": (244, 240, 228), "nuage_ombre": (196, 196, 184),
    "nuage_bleu": (166, 188, 190),
    "lointain": (112, 152, 156), "lointain2": (92, 134, 138),
    "colline": (122, 142, 76), "colline2": (150, 164, 88), "colline3": (96, 118, 66),
    "ocre": (214, 168, 70), "ble": (230, 196, 92), "orange": (216, 124, 52),
    "rouge": (176, 84, 44), "brun": (108, 48, 36), "olive": (142, 150, 70),
    "vert": (104, 130, 60), "vert_sombre": (62, 86, 52), "route": (232, 222, 190),
    "feuille": (74, 104, 60), "feuille2": (108, 134, 64), "feuille3": (44, 66, 44),
    "feuille_rouge": (110, 42, 30), "tronc": (70, 40, 30), "tronc2": (38, 24, 20),
    "cypres": (50, 70, 48), "cypres2": (32, 46, 34),
    "mur": (226, 200, 140), "mur_ombre": (190, 160, 100), "toit": (178, 96, 54),
    "pylone": (34, 42, 48), "pylone2": (58, 70, 78), "parabole": (236, 236, 228),
}


# LESS COLOUR, MORE RAW. The palette above is the bold one of the "Province"
# wallpaper; behind the dark panels of the bench it is pulled towards its grey
```

```
# (SATURATION: 1.0 = as is, 0 = grey) and darkened slightly; the facets are
# larger and more uneven.
SATURATION = 0.55
SOMBRE = 0.93


def rabattre(c):
    lum = 0.299 * c[0] + 0.587 * c[1] + 0.114 * c[2]
    return tuple(int(max(0, min(255, (lum + (v - lum) * SATURATION) * SOMBRE))) for v in c)


P = {k: rabattre(v) for k, v in P.items()}


def lerp(a, b, t):
    return tuple(int(round(a[i] + (b[i] - a[i]) * t)) for i in range(3))


def shade(c, k):
    return tuple(max(0, min(255, int(round(v * k)))) for v in c)


# ── THE LANDSCAPE LINES (in 1920x1080 frame coordinates) ──────────────────
def ligne(x, base, amp, freqs, phase=0.0):
    """A soft silhouette: sum of sines."""
    y = base
    for i, (a, f) in enumerate(zip(amp, freqs)):
        y += a * math.sin(f * x / FW * 2 * math.pi + phase + i * 1.7)
    return y


HORIZON = 640.0
LIGNES = {
    # name: (function y(x), colour)
    "lointain": (lambda x: ligne(x, 612, (46, 20, 7), (1.3, 3.1, 7.0), 0.4), P["lointain"]),
    "lointain2": (lambda x: ligne(x, 648, (30, 12, 5), (2.2, 4.3, 9.0), 3.6), P["lointain2"]),
    "colline": (lambda x: ligne(x, 690, (40, 16, 6), (0.9, 2.6, 6.0), 1.2), P["colline"]),
    "colline2": (lambda x: ligne(x, 735, (30, 14, 5), (1.1, 3.4, 8.0), 3.3), P["colline2"]),
    "colline3": (lambda x: ligne(x, 770, (24, 10, 4), (1.4, 2.9, 7.5), 0.9), P["colline3"]),
    "ocre": (lambda x: ligne(x, 820, (26, 10, 4), (0.8, 2.2, 5.0), 2.6), P["ocre"]),
    "ble": (lambda x: ligne(x, 870, (20, 8, 4), (1.2, 3.0, 6.5), 1.5), P["ble"]),
    "orange": (lambda x: ligne(x, 925, (24, 10, 4), (0.7, 2.4, 5.5), 4.0), P["orange"]),
    "rouge": (lambda x: ligne(x, 985, (18, 8, 4), (1.0, 2.8, 6.0), 0.2), P["rouge"]),
    "brun": (lambda x: ligne(x, 1040, (14, 6, 3), (1.3, 3.2, 7.0), 2.9), P["brun"]),
}
ORDRE = ["lointain", "lointain2", "colline", "colline2", "colline3",
         "ocre", "ble", "orange", "rouge", "brun"]


def bande_polygone(f, s, pas=24):
    """The polygon (scale s) under line f, down to the bottom of the frame."""
    pts = [(x * s, f(x) * s) for x in range(-40, FW + 41, pas)]
    return pts + [((FW + 40) * s, (FH + 40) * s), (-40 * s, (FH + 40) * s)]


# ── THE SMOOTH PAINTING (what the triangulation will facet) ───────────────
def ciel(w, h, s):
    """Sky gradient, warm towards the horizon, and the sun halo."""
    yy, xx = np.mgrid[0:h, 0:w].astype(np.float32)
    t = np.clip(yy / (HORIZON * s), 0, 1)[..., None]
    haut = np.array(P["ciel_haut"], np.float32)
    bas = np.array(P["ciel_bas"], np.float32)
    hor = np.array(P["horizon"], np.float32)
    img = haut + (bas - haut) * np.clip(t / 0.75, 0, 1)
    th = np.clip((t - 0.62) / 0.38, 0, 1) ** 1.6
    img = img + (hor - img) * th
```

```
    # the halo: a low sun, at the centre of the inset
    sx, sy, r = SOLEIL
    d = np.sqrt(((xx - sx * s) ** 2) + ((yy - sy * s) ** 2) * 1.35) / (r * s)
    halo = np.exp(-(d / 3.0) ** 2)[..., None] * 0.6
    halo += np.exp(-(d / 1.35) ** 2)[..., None] * 0.45
    img = img + (np.array(P["halo"], np.float32) - img) * np.clip(halo, 0, 1)
    return img


SOLEIL = (935, 548, 82)           # centre x, centre y, radius (1920x1080 frame)


def nuages(rng, s):
    """Flat clouds, in stretched banks, like paper cut-outs."""
    out = []
    bancs = [(230, 140, 640, 70), (1520, 110, 560, 64), (860, 190, 460, 52),
             (1250, 290, 360, 40), (470, 320, 420, 44), (1720, 350, 300, 32),
             (1020, 400, 260, 26), (320, 450, 260, 22), (1480, 450, 220, 18)]
    for (cx, cy, lw, lh) in bancs:
        for k in range(2):
            pts = []
            n = rng.randint(7, 10)
            ox, oy = rng.uniform(-lw * 0.2, lw * 0.2), rng.uniform(-lh * 0.3, lh * 0.3) - k * lh * 0.18
            for i in range(n):
                a = 2 * math.pi * i / n
                rx = lw * (0.5 - 0.14 * k) * rng.uniform(0.8, 1.05)
                ry = lh * (0.5 - 0.1 * k) * rng.uniform(0.7, 1.15)
                pts.append(((cx + ox + rx * math.cos(a)) * s, (cy + oy + ry * math.sin(a)) * s))
            c = [P["nuage"], P["nuage_clair"]][k]
            out.append((pts, c))
        # a bluish shadow under the bank
        pts = [((cx + lw * 0.45 * math.cos(a) - lw * 0.08) * s, (cy + lh * 0.55 + lh * 0.22 * math.sin(a)) * s)
               for a in np.linspace(0, 2 * math.pi, 7, endpoint=False)]
        out.insert(len(out) - 2, (pts, P["nuage_bleu"]))
    return out


def peinture(w, h, s, rng):
    img = Image.fromarray(np.clip(ciel(w, h, s), 0, 255).astype(np.uint8))
    d = ImageDraw.Draw(img)
    nuag = nuages(rng, s)
    for pts, c in nuag:
        d.polygon(pts, fill=c)
    # the sun disc (faceted later, then redrawn sharp on top)
    sx, sy, r = SOLEIL
    d.ellipse((sx * s - r * s, sy * s - r * s, sx * s + r * s, sy * s + r * s), fill=P["soleil"])
    for nom in ORDRE:
        f, c = LIGNES[nom]
        d.polygon(bande_polygone(f, s), fill=c)
    # a light road winding from the bottom up to the village, on the right
    route = [(1130, 1100), (1200, 1010), (1340, 930), (1500, 860), (1620, 790), (1700, 740), (1740, 712)]
    larg = [40, 32, 26, 18, 12, 8, 5]
    gauche, droite = [], []
    for i, (x, y) in enumerate(route):
        xp, yp = route[max(0, i - 1)]
        xn, yn = route[min(len(route) - 1, i + 1)]
        dx, dy = xn - xp, yn - yp
        n = math.hypot(dx, dy) or 1.0
        nx, ny = -dy / n, dx / n          # the normal
        gauche.append(((x + nx * larg[i]) * s, (y + ny * larg[i]) * s))
        droite.append(((x - nx * larg[i]) * s, (y - ny * larg[i]) * s))
    d.polygon(gauche + droite[::-1], fill=P["route"])
    # patches of green and olive in the fields (distant groves, hedges)
    for (cx, cy, rw, rh, c) in [(260, 790, 90, 18, P["vert"]), (1180, 760, 120, 14, P["vert_sombre"]),
                                (640, 805, 70, 12, P["olive"]), (1800, 700, 90, 16, P["vert"]),
                                (120, 905, 160, 30, P["olive"]), (480, 940, 120, 22, P["vert_sombre"]),
```

```
                              (1950, 870, 140, 28, P["olive"]), (80, 1010, 130, 26, P["vert_sombre"])]:
        pts = [((cx + rw * math.cos(a) * rng.uniform(0.8, 1.1)) * s,
                (cy + rh * math.sin(a) * rng.uniform(0.7, 1.2)) * s)
               for a in np.linspace(0, 2 * math.pi, 8, endpoint=False)]
        d.polygon(pts, fill=c)
    return img, nuag


# ── THE POINTS, THEN THE FACETS ──────────────────────────────────────
def points_scene(w, h, s, rng, nuag):
    pts = []
    # an irregular scatter: wide in the sky, tight in the fields
    y = 0.0
    while y < FH + 60:
        pas = 210 if y < 520 else (96 if y < 800 else 78)
        x = -40.0
        while x < FW + 60:
            pts.append(((x + rng.uniform(-0.6, 0.6) * pas) * s, (y + rng.uniform(-0.6, 0.6) * pas) * s))
            x += pas
        y += pas * 0.85
    # the silhouettes: points along each line, so that the facets stop
    # sharply there (a ridge stays a ridge)
    for nom in ORDRE:
        f, _ = LIGNES[nom]
        x = -40.0
        while x < FW + 40:
            pts.append((x * s, f(x) * s))
            x += rng.uniform(34, 64)
    # the outline of the clouds and the sun
    for poly, _ in nuag:
        pts.extend(poly)
    sx, sy, r = SOLEIL
    for a in np.linspace(0, 2 * math.pi, 14, endpoint=False):
        pts.append(((sx + r * math.cos(a)) * s, (sy + r * math.sin(a)) * s))
    for a in np.linspace(0, 2 * math.pi, 9, endpoint=False):
        pts.append(((sx + 2.2 * r * math.cos(a)) * s, (sy + 1.6 * r * math.sin(a)) * s))
    # the corners
    pts += [(-60 * s, -60 * s), ((FW + 60) * s, -60 * s), (-60 * s, (FH + 60) * s), ((FW + 60) * s, (FH + 60) * s)]
    return np.array(pts, np.float64)


def facettes(peint, pts, rng):
    """Each Delaunay triangle takes the average colour of the painting
    under it, slightly varied: the flat facets of the low-poly."""
    arr = np.asarray(peint).astype(np.float32)
    h, w = arr.shape[:2]
    tri = Delaunay(pts)
    out = Image.new("RGB", (w, h), P["ciel_haut"])
    d = ImageDraw.Draw(out)
    # barycentric sampling points (inside the triangle)
    bary = np.array([(1 / 3, 1 / 3, 1 / 3), (0.6, 0.2, 0.2), (0.2, 0.6, 0.2), (0.2, 0.2, 0.6),
                     (0.45, 0.45, 0.1), (0.1, 0.45, 0.45), (0.45, 0.1, 0.45)], np.float32)
    for simplex in tri.simplices:
        p = pts[simplex]
        samp = bary @ p
        xs = np.clip(samp[:, 0], 0, w - 1).astype(int)
        ys = np.clip(samp[:, 1], 0, h - 1).astype(int)
        cols = arr[ys, xs]
        # the median rather than the mean: a triangle straddling a ridge takes
        # the colour of the side that fills it, not a grey mix
        c = np.median(cols, axis=0)
        k = rng.uniform(0.90, 1.10)
        c = np.clip(c * k, 0, 255).astype(int)
        d.polygon([tuple(q) for q in p], fill=tuple(int(v) for v in c))
    return out
```

```
# ── THE SUBJECTS, SHARP, ON TOP ──────────────────────────────────────────
def pylone(d, s, x, base, haut, larg, deck=False, paraboles=(), antennes=True):
    """A lattice pylon: two uprights that narrow, cross braces, a platform
    and dishes for the big one."""
    c, c2 = P["pylone"], P["pylone2"]
    ht = base - haut
    n = max(6, int(ht / 42))
    pts_g, pts_d = [], []
    for i in range(n + 1):
        t = i / n
        y = base - ht * t
        lw = larg * (1 - 0.82 * t) + 2
        pts_g.append((x - lw, y))
        pts_d.append((x + lw, y))
    lw_px = max(2, int(2.2 * s))
    # uprights
    d.line([(px * s, py * s) for px, py in pts_g], fill=c, width=lw_px)
    d.line([(px * s, py * s) for px, py in pts_d], fill=c, width=lw_px)
    # X braces and landings
    for i in range(n):
        (gx0, gy0), (gx1, gy1) = pts_g[i], pts_g[i + 1]
        (dx0, dy0), (dx1, dy1) = pts_d[i], pts_d[i + 1]
        d.line([(gx0 * s, gy0 * s), (dx1 * s, dy1 * s)], fill=c2, width=max(1, int(1.1 * s)))
        d.line([(dx0 * s, dy0 * s), (gx1 * s, gy1 * s)], fill=c2, width=max(1, int(1.1 * s)))
        d.line([(gx0 * s, gy0 * s), (dx0 * s, dy0 * s)], fill=c, width=max(1, int(1.3 * s)))
    # the foot: a polygonal base
    d.polygon([((x - larg - 6) * s, base * s), ((x + larg + 6) * s, base * s),
               ((x + larg + 2) * s, (base - 10) * s), ((x - larg - 2) * s, (base - 10) * s)], fill=c)
    if deck:
        y = haut + ht * 0.30
        d.polygon([((x - larg * 1.5) * s, y * s), ((x + larg * 1.5) * s, y * s),
                   ((x + larg * 1.15) * s, (y - 22) * s), ((x - larg * 1.15) * s, (y - 22) * s)], fill=c)
        d.polygon([((x - larg * 1.3) * s, (y - 22) * s), ((x + larg * 1.3) * s, (y - 22) * s),
                   ((x + larg * 0.8) * s, (y - 36) * s), ((x - larg * 0.8) * s, (y - 36) * s)], fill=c2)
        d.polygon([((x - larg * 1.5) * s, y * s), ((x + larg * 1.5) * s, y * s),
                   ((x + larg * 0.9) * s, (y + 24) * s), ((x - larg * 0.9) * s, (y + 24) * s)], fill=c2)
    for (dy, cote, r) in paraboles:
        y = haut + ht * dy
        lw = larg * (1 - 0.82 * (base - y) / ht) + 2
        cx = x + cote * (lw + r * 0.7)
        d.ellipse(((cx - r * 0.55) * s, (y - r) * s, (cx + r * 0.55) * s, (y + r) * s), fill=P["parabole"])
        d.ellipse(((cx - r * 0.25) * s, (y - r * 0.85) * s, (cx + r * 0.35) * s, (y + r * 0.85) * s), fill=(214,
↪   214, 206))
    if antennes:
        # the mast at the top
        d.line([(x * s, haut * s), (x * s, (haut - ht * 0.14) * s)], fill=c, width=max(1, int(1.6 * s)))
        d.polygon([((x - 3) * s, (haut - ht * 0.14) * s), ((x + 3) * s, (haut - ht * 0.14) * s),
                   (x * s, (haut - ht * 0.21) * s)], fill=c)
        for k in (0.04, 0.09):
            y = haut + ht * k
            d.line([((x - 9) * s, y * s), ((x + 9) * s, y * s)], fill=c, width=max(1, int(1.3 * s)))
            d.rectangle(((x - 11) * s, (y - 5) * s, (x - 8) * s, (y + 5) * s), fill=c)
            d.rectangle(((x + 8) * s, (y - 5) * s, (x + 11) * s, (y + 5) * s), fill=c)


def arbre(d, s, rng, x, base, hauteur, envergure):
    """The big tree on the left: twisted trunk, branches, foliage in shards."""
    tx = x
    # trunk in several tilting segments
    seg = [(tx, base), (tx + 10, base - hauteur * 0.25), (tx - 8, base - hauteur * 0.5),
           (tx + 14, base - hauteur * 0.7), (tx + 2, base - hauteur * 0.86)]
    larg = [34, 26, 20, 15, 10]
    for i in range(len(seg) - 1):
        (x0, y0), (x1, y1) = seg[i], seg[i + 1]
        l0, l1 = larg[i], larg[i + 1]
```

```
        d.polygon([((x0 - l0) * s, y0 * s), ((x0 + l0) * s, y0 * s), ((x1 + l1) * s, y1 * s), ((x1 - l1) * s, y1 *
↪  s)],
                  fill=P["tronc"])
        d.polygon([((x0 - l0) * s, y0 * s), ((x0 - l0 * 0.3) * s, y0 * s), ((x1 - l1 * 0.3) * s, y1 * s), ((x1 -
↪  l1) * s, y1 * s)],
                  fill=P["tronc2"])
    # the roots
    d.polygon([((tx - 60) * s, base * s), ((tx + 70) * s, base * s), ((tx + 30) * s, (base - 24) * s), ((tx - 28) *
↪  s, (base - 26) * s)],
              fill=P["tronc"])
    # branches
    branches = [((tx - 4, base - hauteur * 0.55), (tx - 120, base - hauteur * 0.78), 9),
                ((tx + 8, base - hauteur * 0.68), (tx + 150, base - hauteur * 0.9), 8),
                ((tx + 2, base - hauteur * 0.84), (tx - 70, base - hauteur * 1.02), 6),
                ((tx + 2, base - hauteur * 0.86), (tx + 60, base - hauteur * 1.05), 6),
                ((tx - 120, base - hauteur * 0.78), (tx - 210, base - hauteur * 0.88), 5),
                ((tx + 150, base - hauteur * 0.9), (tx + 260, base - hauteur * 0.98), 5)]
    for (a, b, w) in branches:
        d.line([(a[0] * s, a[1] * s), (b[0] * s, b[1] * s)], fill=P["tronc"], width=int(w * s))
    # foliage: triangular shards around a few centres
    centres = [(tx - 180, base - hauteur * 0.95, 150), (tx + 40, base - hauteur * 1.05, 190),
               (tx + 230, base - hauteur * 1.0, 160), (tx - 60, base - hauteur * 1.15, 130),
               (tx + 120, base - hauteur * 1.2, 120), (tx - 240, base - hauteur * 0.8, 90),
               (tx + 300, base - hauteur * 0.9, 100)]
    couleurs = [P["feuille"], P["feuille2"], P["feuille3"], P["feuille"], P["feuille2"], P["feuille"],
                P["feuille3"], P["feuille_rouge"]]
    for (cx, cy, r) in centres:
        for _ in range(26):
            a = rng.uniform(0, 2 * math.pi)
            rr = r * rng.uniform(0.2, 1.0)
            px, py = cx + rr * math.cos(a), cy + rr * math.sin(a) * 0.75
            n = rng.randint(3, 5)
            size = rng.uniform(28, 70)
            pts = []
            for i in range(n):
                aa = 2 * math.pi * i / n + rng.uniform(-0.3, 0.3)
                pts.append(((px + size * math.cos(aa)) * s, (py + size * 0.8 * math.sin(aa)) * s))
            c = rng.choice(couleurs)
            d.polygon(pts, fill=shade(c, rng.uniform(0.85, 1.12)))


def cypres(d, s, x, base, h, w):
    d.polygon([((x - w) * s, base * s), ((x + w) * s, base * s), ((x + w * 0.6) * s, (base - h * 0.55) * s),
               (x * s, (base - h) * s), ((x - w * 0.6) * s, (base - h * 0.55) * s)], fill=P["cypres"])
    d.polygon([((x - w) * s, base * s), ((x - w * 0.1) * s, base * s), ((x - w * 0.05) * s, (base - h * 0.6) * s),
               (x * s, (base - h) * s), ((x - w * 0.6) * s, (base - h * 0.55) * s)], fill=P["cypres2"])


def maison(d, s, x, base, w, h):
    d.polygon([(x * s, base * s), ((x + w) * s, base * s), ((x + w) * s, (base - h) * s), (x * s, (base - h) * s)],
↪  fill=P["mur"])
    d.polygon([((x + w * 0.62) * s, base * s), ((x + w) * s, base * s), ((x + w) * s, (base - h) * s), ((x + w *
↪  0.62) * s, (base - h) * s)],
              fill=P["mur_ombre"])
    d.polygon([((x - w * 0.08) * s, (base - h) * s), ((x + w * 1.08) * s, (base - h) * s),
               ((x + w * 0.95) * s, (base - h * 1.35) * s), ((x + w * 0.05) * s, (base - h * 1.35) * s)],
↪  fill=P["toit"])
    d.polygon([((x - w * 0.08) * s, (base - h) * s), ((x + w * 0.5) * s, (base - h) * s),
               ((x + w * 0.45) * s, (base - h * 1.35) * s), ((x + w * 0.05) * s, (base - h * 1.35) * s)],
↪  fill=shade(P["toit"], 1.15))
    # windows and door, in shadow
    for fx in (0.14, 0.40):
        d.rectangle(((x + w * fx) * s, (base - h * 0.78) * s, (x + w * (fx + 0.1)) * s, (base - h * 0.55) * s),
↪  fill=P["cypres2"])
    d.rectangle(((x + w * 0.27) * s, (base - h * 0.38) * s, (x + w * 0.37) * s, base * s), fill=P["cypres2"])
 ×4
```

```
def dessiner(scale, seed=7, antennes=True):
    rng = random.Random(seed)
    w, h = FW * scale, FH * scale
    peint, nuag = peinture(w, h, scale, rng)
    pts = points_scene(w, h, scale, rng, nuag)
    img = facettes(peint, pts, rng)

    # the sharp sun (a faceted disc), then the land goes back in front
    avant = img.copy()
    s = scale
    d = ImageDraw.Draw(img)
    sx, sy, r = SOLEIL
    d.polygon([((sx + r * math.cos(a)) * s, (sy + r * math.sin(a)) * s)
               for a in np.linspace(0, 2 * math.pi, 11, endpoint=False)], fill=P["soleil"])
    masque = Image.new("L", img.size, 0)
    ImageDraw.Draw(masque).polygon(bande_polygone(LIGNES["lointain"][0], scale), fill=255)
    img.paste(avant, (0, 0), masque)
    # the village on the right (behind the conky), on the "colline" ridge
    fc = LIGNES["colline"][0]
    maison(d, s, 1705, fc(1725) + 6, 84, 42)
    for (x, hh, ww) in [(1660, 120, 10), (1680, 96, 7), (1820, 104, 8)]:
        cypres(d, s, x, fc(x) + 10, hh, ww)
    # THE PYLONS ARE ONLY IN THE INSET. The wallpaper is the bare landscape;
    # the middle window is what plants the antennas in it, within its box -
    # all three are therefore entirely inside BOX (510..1410, 220..1010).
    if antennes:
        fl2 = LIGNES["lointain2"][0]
        pylone(d, s, 592, fl2(592) + 4, 505, 7, antennes=True)
        pylone(d, s, 705, fl2(705) + 6, 420, 11, antennes=True)
        pylone(d, s, 1255, fc(1255) + 2, 330, 30, deck=True,
               paraboles=((0.42, -1, 18), (0.50, 1, 15), (0.60, -1, 12)))
    # the big tree on the left, behind the dock and the icons
    fo = LIGNES["ocre"][0]
    arbre(d, s, rng, 170, fo(170) + 40, 560, 300)
    # tufts of grass at the foot, under the launcher bar
    for _ in range(60):
        x = rng.uniform(-20, FW + 20)
        y = rng.uniform(1030, 1085)
        hh = rng.uniform(10, 26)
        c = shade(P["brun"] if y > 1040 else P["rouge"], rng.uniform(0.7, 0.95))
       d.polygon([((x - 5) * s, y * s), ((x + 5) * s, y * s), ((x + rng.uniform(-6, 6)) * s, (y - hh) * s)], fill=c)
    return img


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--scale", type=int, default=2, help="2 : fond 3840x2160, encart 1800x1580")
    ap.add_argument("--fond", required=True, help="le fond d ecran GNOME (scene entiere)")
    ap.add_argument("--encart", required=True, help="l illustration de l encart (decoupe BOX)")
    ap.add_argument("--seed", type=int, default=7)
    a = ap.parse_args()

    # two renders of the same seed: the bare landscape (wallpaper) and the
    # same with the pylons, of which only the inset box is kept
    img = dessiner(a.scale, a.seed, antennes=False)
    img.save(a.fond, "JPEG", quality=92, optimize=True)
    s = a.scale
    enc = dessiner(a.scale, a.seed, antennes=True).crop((BOX[0] * s, BOX[1] * s, BOX[2] * s, BOX[3] * s))
    enc.save(a.encart, "JPEG", quality=92, optimize=True)
    print(f"fond   : {a.fond} {img.size}")
    print(f"encart : {a.encart} {enc.size} (boite {BOX} x{s})")


if __name__ == "__main__":
    main()
```

2.286 /opt/GSM/osmo-operator/tools/plymouth-render.py

8688 octets, 188 lignes → 188 lignes

```
#!/usr/bin/env python3
# plymouth-render.py - builds the images of the "osmo-bts" Plymouth theme.
#
#     python3 tools/plymouth-render.py --out configs/plymouth/osmo-bts
#
# THE THEME IS GENERATED, NOT STORED. A Plymouth theme is a set of PNGs:
# keeping them in the repository means some thirty binaries that can neither
# be read nor fixed in a diff. Here the drawing IS the code - change a colour,
# rerun the script, and the next ISO inherits it. iso_modules/86-finitions.sh
# calls it at build time.
#
# WHAT IS SHOWN AT BOOT (pure 2D):
#
#                        (  (  ( • )  )  )
#                              /|\
#                             / | \         a stylised antenna in the middle,
#                            /__|__\        thin circles opening from
#                                           its top
#                           LAB GSM
#                 ————————— (bar) —————————
#
#   A flat background, a single antenna in thick rounded strokes, and
#   concentric circles with a sharp stroke that open and fade out.
#
# --gif FILE also writes an animated preview (background + 24 frames, reduced),
# for the wiki or to judge the rendering without rebooting. It does NOT go in
# --out: everything in the theme folder goes into the initrd.
#
# Depends only on Pillow and the DejaVu fonts, like wallpaper-render.py.
import argparse
import math
import os

from PIL import Image, ImageDraw, ImageFont

W, H = 1920, 1080
FONT_DIR = "/usr/share/fonts/truetype/dejavu"
BG_TOP, BG_BOT = (8, 12, 26), (16, 26, 52)
CYAN, GREEN, VIOLET, GREY = (88, 166, 255), (63, 185, 80), (166, 122, 255), (139, 148, 158)
NEON = (64, 224, 255)            # the cyan of the rings: vivid, like a neon
FRAMES = 24


def font(name, size):
    try:
        return ImageFont.truetype(os.path.join(FONT_DIR, name), size)
    except OSError:
        return ImageFont.load_default()


def background():
    """FLAT night blue background (pure 2D, no vertical gradient)."""
    return Image.new("RGB", (W, H), BG_TOP)


def mast(d, cx, top_y, h, col=(200, 210, 225)):
    """The antenna, in pure 2D: a mast, two legs and two cross bars, thick
    strokes with round ends, and a bright dot at the top (where the waves start)."""
    w, lw = h * 0.26, 7
    foot = top_y + h
    def seg(a, b):
        d.line([a, b], fill=col, width=lw)
```

```
        for x, y in (a, b):
            d.ellipse([x - lw / 2, y - lw / 2, x + lw / 2, y + lw / 2], fill=col)
    seg((cx, top_y + 18), (cx, foot))
    seg((cx, top_y + 30), (cx - w, foot))
    seg((cx, top_y + 30), (cx + w, foot))
    for t in (0.55, 0.82):                      # cross bars between the legs
        y = top_y + 30 + (foot - top_y - 30) * t
        dx = w * t
        seg((cx - dx, y), (cx + dx, y))
    r = 10
    d.ellipse([cx - r, top_y - r, cx + r, top_y + r], fill=NEON)
    return (cx, top_y)


def ripples(d, at, phase, rmax=300, n=4, col=NEON):
    """The waves, in pure 2D: concentric CIRCLES centred on the antenna, which
    open and fade out as they move away. A single sharp stroke per circle: no
    perspective ellipse, cone, halo or light core."""
    x, y = at
    for k in range(n):
        t = ((phase + k / n) % 1.0)
        r = 28 + t * rmax
        a = int(255 * (1 - t) ** 0.9)
        d.ellipse([x - r, y - r, x + r, y + r], outline=(col[0], col[1], col[2], a), width=4)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", required=True)
    ap.add_argument("--template", default=None,
                    help="le gabarit osmo-bts.script (defaut : celui du depot)")
    ap.add_argument("--gif", default=None,
                    help="ecrit aussi un apercu anime (GIF) a ce chemin")
    ap.add_argument("--gif-width", type=int, default=960,
                    help="largeur de l apercu GIF (defaut 960)")
    a = ap.parse_args()
    os.makedirs(a.out, exist_ok=True)

    bg = background()
    d = ImageDraw.Draw(bg)
    ant = mast(d, W // 2, int(H * 0.36), int(H * 0.25))
    f_title, f_sub = font("DejaVuSans-Bold.ttf", 64), font("DejaVuSansMono.ttf", 24)
    d.text((W / 2, 900), "LAB GSM", font=f_title, fill=(230, 237, 243), anchor="mm")
    d.text((W / 2, 952), "reseau 2G complet, sans materiel  ·  osmocom  ·  calypso emule",
           font=f_sub, fill=GREY, anchor="mm")
    bg.save(os.path.join(a.out, "background.png"), "PNG", optimize=True)

    # The frames: only the BAND between the two antennas, in RGBA. Plymouth
    # draws them over the background - one image per frame rather than a
    # complete background redrawn 24 times (24 x 1.3 MB would not fit well in
    # an initrd). The band encloses the WHOLE circles (maximum radius + stroke):
    # a circle is never cut by its edge. Outside the strokes it is transparent,
    # hence light in the initrd.
    R, M = 230, 12
    bx0, bx1 = ant[0] - 28 - R - M, ant[0] + 28 + R + M
    strip_y = ant[1] - 28 - R - M
    strip_h = 2 * (28 + R + M)
    tip = (ant[0] - bx0, ant[1] - strip_y)                # the top of the antenna, in the band
    frames = []
    for i in range(FRAMES):
        t = i / FRAMES
        im = Image.new("RGBA", (int(bx1 - bx0), strip_h), (0, 0, 0, 0))
        dd = ImageDraw.Draw(im)
        # the waves, and nothing else
        ripples(dd, tip, t, rmax=R, n=3)
        im.save(os.path.join(a.out, "burst-%02d.png" % i), "PNG", optimize=True)
```

```
        frames.append(im)

    # The animated preview: the background, the band drawn over it in place, one
    # image per frame, at ~12 fps as in the theme (one frame every 4 passes at
    # 50 Hz). Reduced: a 1920x1080 GIF of 24 images weighs several MB for
    # nothing.
    if a.gif:
        gw = max(160, a.gif_width)
        gh = int(H * gw / W)
        seq = []
        for im in frames:
            full = bg.convert("RGBA")
            full.alpha_composite(im, (int(bx0), int(strip_y)))
            seq.append(full.convert("RGB").resize((gw, gh), Image.LANCZOS)
                       .quantize(colors=128, method=Image.Quantize.MEDIANCUT))
        seq[0].save(a.gif, save_all=True, append_images=seq[1:], loop=0,
                    duration=int(1000 * 4 / 50), optimize=True)
        print("[plymouth] apercu : %s (%dx%d, %d images)" % (a.gif, gw, gh, FRAMES))

    # The progress bar: a background and a dot running over it.
    bar = Image.new("RGBA", (520, 8), (255, 255, 255, 0))
    ImageDraw.Draw(bar).rounded_rectangle([0, 0, 519, 7], 4, fill=(38, 48, 76, 255))
    bar.save(os.path.join(a.out, "progress-box.png"), "PNG")
    dot = Image.new("RGBA", (64, 8), (0, 0, 0, 0))
    ImageDraw.Draw(dot).rounded_rectangle([0, 0, 63, 7], 4, fill=CYAN + (255,))
    dot.save(os.path.join(a.out, "progress-bar.png"), "PNG")
    # The LUKS passphrase entry field (theme script: it has to be drawn here).
    # Without these two images, an encrypted installation asks for its
    # passphrase on an empty screen.
    ent = Image.new("RGBA", (620, 56), (0, 0, 0, 0))
    ImageDraw.Draw(ent).rounded_rectangle([0, 0, 619, 55], 10, fill=(18, 26, 48, 235),
                                          outline=CYAN + (170,), width=2)
    ent.save(os.path.join(a.out, "entry.png"), "PNG")
    bul = Image.new("RGBA", (18, 18), (0, 0, 0, 0))
    ImageDraw.Draw(bul).ellipse([0, 0, 17, 17], fill=(230, 237, 243, 255))
    bul.save(os.path.join(a.out, "bullet.png"), "PNG")
    # ── THE SCRIPT IS WRITTEN FROM HERE TOO ──────────────────────────────
    # It needs to know WHERE the band between the two antennas is; these
    # numbers were just computed above. Copying them by hand into the .script
    # would guarantee they diverge at the first adjustment.
    tpl = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                       "..", "configs", "plymouth", "osmo-bts", "osmo-bts.script")
    tpl = a.template or os.path.normpath(tpl)
    if os.path.isfile(tpl):
        with open(tpl, encoding="utf-8") as f:
            sc = f.read()
        for k, v in (("@@BX@@", bx0 / W), ("@@BY@@", strip_y / H),
                     ("@@BW@@", (bx1 - bx0) / W), ("@@BH@@", strip_h / H)):
            sc = sc.replace(k, "%.4f" % v)
        with open(os.path.join(a.out, "osmo-bts.script"), "w", encoding="utf-8") as f:
            f.write(sc)
        print("[plymouth] script : bande a x=%.3f y=%.3f  %.3f x %.3f (fractions d ecran)"
              % (bx0 / W, strip_y / H, (bx1 - bx0) / W, strip_h / H))
    else:
        print("[plymouth] gabarit %s introuvable - script NON regenere" % tpl)
    print("[plymouth] %s : fond + %d frames + barre + saisie" % (a.out, FRAMES))


if __name__ == "__main__":
    main()
```

2.287 /opt/GSM/osmo-operator/tools/screenfetch-art.py

6288 octets, 180 lignes → 180 lignes

```
#!/usr/bin/env python3
# screenfetch-art.py - writes configs/screenfetch/osmo-lab.sh: the logo in
# COLOUR PIXEL ART that screenfetch displays when each terminal opens
# (osmo-banner, iso_modules/86-finitions.sh: "screenfetch -a").
#
#     python3 tools/screenfetch-art.py
#
# A dish (the BTS) on its green base, its arm and its red feed, and the
# phone next to it, green screen. The drawing is COMPUTED here (ellipses,
# segments) on a pixel grid, then rendered as half blocks "▀": one
# character = two pixels (top colour in the foreground, bottom colour in
# the background), in true 24-bit colours.
# screenfetch requires lines of the same VISIBLE width: each line is W
# columns, the colours are \e[...m sequences (zero width) interpreted by
# the printf of screenfetch.
import math
import os

W, H = 30, 30
PAL = {
    "k": (16, 16, 24),     # contour
    "d": (84, 96, 130),    # parabole, creux
    "m": (138, 148, 180),  # parabole, flanc
    "l": (200, 208, 228),  # parabole, reflet
    "t": (216, 204, 160),  # bras de l illuminateur
    "r": (200, 70, 60),    # rouge : illuminateur et rotule
    "R": (232, 110, 100),
    "g": (140, 180, 140),  # socle
    "G": (90, 138, 90),
    "s": (150, 156, 176),  # support
    "p": (60, 66, 88),     # telephone, boitier
    "P": (96, 104, 134),
    "e": (70, 220, 120),   # telephone, ecran
    "E": (150, 255, 190),
    "y": (255, 176, 0),    # ondes
}
g = [["."] * W for _ in range(H)]


def put(x, y, c):
    x, y = int(round(x)), int(round(y))
    if 0 <= x < W and 0 <= y < H:
        g[y][x] = c


def line(x0, y0, x1, y1, c, w=1):
    n = int(max(abs(x1 - x0), abs(y1 - y0)) * 2) + 1
    for i in range(n + 1):
        t = i / n
        for dx in range(w):
            for dy in range(w):
                put(x0 + (x1 - x0) * t + dx, y0 + (y1 - y0) * t + dy, c)


def ell(cx, cy, a, b, ang):
    ca, sa = math.cos(ang), math.sin(ang)

    def f(x, y):
        dx, dy = x - cx, y - cy
        u, v = dx * ca + dy * sa, -dx * sa + dy * ca
        return (u / a) ** 2 + (v / b) ** 2
    return f


def paint_ell(f, c, only=None):
    for y in range(H):
```

```
        for x in range(W):
            if f(x, y) <= 1 and (only is None or g[y][x] in only):
                g[y][x] = c


# -- the dish: light shell, dark hollow shifted towards the top left ------
ang = math.radians(40)
coque = ell(15, 9, 10.8, 5.6, ang)
creux = ell(13.8, 8.2, 8.8, 4.0, ang)
bord = ell(15, 9, 12.0, 6.8, ang)
paint_ell(bord, "k")
paint_ell(coque, "m")
for y in range(H):                      # reflet : le flanc haut de la coque
    for x in range(W):
        if coque(x, y) <= 1 and creux(x, y) > 1 and creux(x + 1.4, y + 1.4) <= 1:
            g[y][x] = "l"
paint_ell(creux, "d")
# -- feed arm: from the hollow up to the right, red tip -------------------
line(14, 9, 22, 2.5, "k", 3)
line(14.5, 9.5, 21.6, 3.2, "t", 1)
for (x, y) in ((22, 1), (23, 1), (22, 2), (23, 2), (24, 2), (23, 3), (24, 3)):
    put(x, y, "k")
for (x, y) in ((22, 2), (23, 2), (23, 3)):
    put(x, y, "r")
put(23, 2, "R")
# -- support: from under the dish to the joint, then to the base ----------
line(11, 14, 8, 18, "k", 4)
line(11.5, 14.5, 9, 18, "s", 2)
for y in range(18, 26):                 # pied trapezoidal
    half = 2 + (y - 18) * 0.55
    for x in range(int(8 - half) - 1, int(8 + half) + 2):
        put(x, y, "k")
for y in range(18, 26):
    half = 2 + (y - 18) * 0.55
    for x in range(int(8 - half), int(8 + half) + 1):
        put(x, y, "s" if x < 8 else "m")
for (x, y) in ((7, 17), (8, 17), (9, 17), (6, 18), (10, 18), (6, 19), (10, 19), (7, 20), (8, 20), (9, 20)):
    put(x, y, "k")
for (x, y) in ((7, 18), (8, 18), (9, 18), (7, 19), (9, 19)):
    put(x, y, "r")
put(8, 19, "k")
# -- green base -----------------------------------------------------------
for x in range(1, 20):
    put(x, 25, "k"); put(x, 29, "k")
for y in range(26, 29):
    put(1, y, "k"); put(19, y, "k")
    for x in range(2, 19):
        put(x, y, "g" if y < 28 else "G")
for x in range(2, 19):
    put(x, 26, "l" if x % 7 else "g")
# -- the phone, on the right, green screen, with its waves ----------------
for y in range(17, 28):
    for x in range(22, 29):
        put(x, y, "k")
for y in range(18, 27):
    for x in range(23, 28):
        put(x, y, "p")
for y in range(19, 22):
    for x in range(24, 27):
        put(x, y, "e")
put(24, 19, "E"); put(25, 19, "E")
for (x, y) in ((24, 23), (26, 23), (24, 24), (26, 24), (24, 25), (26, 25)):
    put(x, y, "P")
for (x, y) in ((25, 23), (25, 24), (25, 25)):
    put(x, y, "P")
line(27, 17, 28, 14, "k", 1)            # antenne
```

```
for (x, y) in ((24, 12), (26, 11), (25, 14), (28, 12)):
    put(x, y, "y")                      # ondes qui partent du telephone


def cell(top, bot):
    def fg(c): return "\\e[38;2;%d;%d;%dm" % PAL[c]
    def bg(c): return "\\e[48;2;%d;%d;%dm" % PAL[c]
    if top == "." and bot == ".":
        return " "
    if bot == ".":
        return fg(top) + "▀\\e[0m"
    if top == ".":
        return fg(bot) + "▄\\e[0m"
    return fg(top) + bg(bot) + "▀\\e[0m"


EN_TETE = r'''#!/bin/bash
# configs/screenfetch/osmo-lab.sh - pixel-art logo of the lab for "screenfetch -a".
# GENERATED by tools/screenfetch-art.py: edit the drawing there, not here.
# Sourced by screenfetch (asciiText): startline, fulloutput (one line = one
# trailing %s, @W@ visible columns each), labelcolor / textcolor.
#
# The colours of the INFO lines are not set here (screenfetch has already
# built them when it sources this file): osmo-banner filters the output
# (user yellow, host green; root red, host yellow).
labelcolor=$'\e[1;33m'
textcolor=$'\e[0m'
startline="0"
logowidth="@W@"
fulloutput=(
'''


def main():
    out = []
    for r in range(0, H, 2):
        s = "".join(cell(g[r][x], g[r + 1][x]) for x in range(W))
        out.append('"' + s + '  %s"')
    dst = os.path.normpath(os.path.join(os.path.dirname(os.path.abspath(__file__)),
                                        "..", "configs", "screenfetch", "osmo-lab.sh"))
    os.makedirs(os.path.dirname(dst), exist_ok=True)
    with open(dst, "w") as f:
        f.write(EN_TETE.replace("@W@", str(W + 2)) + "\n".join(out) + ")\n")
    print("[screenfetch-art] %s : %d lignes de %d colonnes" % (dst, H // 2, W + 2))


if __name__ == "__main__":
    main()
```

2.288 /opt/GSM/osmo-operator/tools/status.sh

identique à /opt/GSM/osmo-operator/scripts/status.sh

2.289 /opt/GSM/osmo-operator/tools/vbox-wan-lab.sh

34003 octets, 652 lignes → 652 lignes

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════════════
# tools/vbox-wan-lab.sh - N VirtualBox machines, one WAN node each
#
# Sets up a complete WAN on a single machine: N VMs (1 to 9, 3 by default)
# boot the SAME osmo-operator ISO, on a shared internal network, and each one
```

```
# recognises itself as the node whose IP is its own. From an MS of VM 2,
# dialing <code of VM 1> + the number reaches an MS of VM 1.
#
# WHY A SINGLE ISO FOR N VMs
# The node number is NOT frozen in the image: the table is, and
# wan_nodes_detect_self compares the local IPs to that table at boot.
# One image per VM would cost N one-hour builds for a single conf line.
#
# VirtualBox's DHCP is what makes the IP predictable: a fixed address per
# (VM, NIC), so the ISO always finds the same node number. Without it, a VM
# would change identity with each lease.
#
# Usage:
#   tools/vbox-wan-lab.sh check                # is the host ready? (run first)
#   tools/vbox-wan-lab.sh create   [--nodes 3] [--iso path.iso] [--build-iso]
#   tools/vbox-wan-lab.sh start | stop | status | destroy
#   tools/vbox-wan-lab.sh usb                  # plugged devices + filters
#   tools/vbox-wan-lab.sh usb --vm 2 --add 0bda:2838:RTL-SDR
#   tools/vbox-wan-lab.sh table                # prints the table without creating anything
#
# create options:
#   --nodes N        number of VMs/nodes, 1 to 9 (default 3)
#   --iso FILE       ISO to boot (default: the most recent in the repo)
#   --build-iso      builds the ISO with the table already inside (sudo, long)
#   --subnet PREFIX  /24 prefix of the segment (default 192.168.56 - the only
#                    range VirtualBox accepts as host-only without configuration)
#   --ram MB         memory per VM (default 4096)
#   --cpus N         vCPU per VM (default 2)
#   --indicatifs "11 22 33"   dialing codes, one per node (default 11 22 33 ...)
#   --no-nat         removes the 2nd adapter (NAT) - no Internet access in the VMs
#
# Hardware passed to the VMs (everything enabled by default):
#   --no-usb         no USB controller at all
#   --usb-filter vid:pid[:name]  captures this device (repeatable, added to the
#                    known list: RTL-SDR, HackRF, bladeRF, LimeSDR, USRP B2xx,
#                    Calypso FT232/CP210x/PL2303/CH340 cables...)
#   --no-audio       no sound card (the GSM audio chain needs one)
#   --audio-driver X pulse | alsa | none (default: detected)
#   --no-vrde        no RDP remote console
#   --share PATH     shared folder (requires the guest additions in the ISO)
# ═══════════════════════════════════════════════════════════════════════════
set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
# shellcheck source=network/wan-nodes.sh
. "$DIR/network/wan-nodes.sh"

NODES=3
# 192.168.56: the range VirtualBox allows out of the box in host-only
# (/etc/vbox/networks.conf is required for any other). It is also used for the
# internal network, where nothing imposes it, so that both modes look alike -
# switching from one to the other must not change the node addresses.
SUBNET="192.168.56"
RAM=4096
CPUS=2
ISO=""
BUILD_ISO=0
WITH_NAT=1
INDS=""
NETNAME="osmowan"
VM_PREFIX="osmo-wan-"

# intnet  : the VMs talk to each other, the host SEES NOTHING of the segment.
```

```
# hostonly: the host has a leg on it - essential when the host is itself a
#           WAN node (start-docker.sh --virtualbox). VirtualBox then only
#           allows 192.168.56.0/21, unless /etc/vbox/networks.conf.
# Node number carried by the HOST itself (empty = none, the N nodes are VMs).
# Once set, the segment must be host-only: the host needs a leg on it to talk
# to its peers.
# SS7 hub ISO. Given → one more VM, at ${SUBNET}.1, which ONLY routes M3UA
# between the nodes. Absent → no inter-node SS7, only voice and SMS.
INTERSTP_ISO=""
INTERSTP_VM="osmo-wan-interstp"
HOST_NODE=""
NET_MODE="intnet"
HOSTONLY_IF=""
WITH_USB=1
WITH_AUDIO=1
WITH_VRDE=1
AUDIO_DRIVER=""
SHARE=""
EXTRA_FILTERS=()
USB_VM=""
USB_ADD=""
FIX=0

# ── Devices captured automatically ──────────────────────────────────────
# Without a filter, VirtualBox leaves EVERYTHING to the host: plugging an SDR
# dongle while the VM runs does nothing at all, silently. One filter per model
# settles it once and for all - the device switches to the VM as soon as it is
# plugged, even hot.
#
# Indicative list: "vbox-wan-lab.sh usb" tells the TRUTH about what is plugged
# and under which vid:pid. Anything missing is added with
# --usb-filter vid:pid:name, without touching this file.
USB_KNOWN=(
    "0bda:2838:RTL-SDR (RTL2838)"
    "0bda:2832:RTL-SDR (RTL2832U)"
    "1d50:6089:HackRF One"
    "1d50:604b:HackRF Jawbreaker"
    "1d50:6066:bladeRF x40/x115"
    "2cf0:5250:bladeRF 2.0 micro"
    "1d50:6108:LimeSDR-USB"
    "2500:0020:USRP B200"
    "2500:0021:USRP B210"
    "2500:0022:USRP B2xx"
    "0403:6001:Cable FT232 (Calypso)"
    "10c4:ea60:Cable CP210x (Calypso)"
    "067b:2303:Cable PL2303 (Calypso)"
    "1a86:7523:Cable CH340 (Calypso)"
    "16c0:0762:Osmocom SIMtrace"
    "1d50:60e3:Osmocom SIMtrace2"
)

ACTION="${1:-}"; [ $# -gt 0 ] && shift || true
while [ $# -gt 0 ]; do
    case "$1" in
        --nodes)       NODES="${2:-3}"; shift ;;
        --nodes=*)     NODES="${1#*=}" ;;
        --iso)         ISO="${2:-}"; shift ;;
        --iso=*)       ISO="${1#*=}" ;;
        --build-iso)   BUILD_ISO=1 ;;
        --subnet)      SUBNET="${2:-}"; shift ;;
        --subnet=*)    SUBNET="${1#*=}" ;;
        --ram)         RAM="${2:-}"; shift ;;
        --ram=*)       RAM="${1#*=}" ;;
        --cpus)        CPUS="${2:-}"; shift ;;
        --cpus=*)      CPUS="${1#*=}" ;;
        --indicatifs)  INDS="${2:-}"; shift ;;
```

```
        --indicatifs=*) INDS="${1#*=}" ;;
        --interstp-iso)   INTERSTP_ISO="${2:-}"; shift ;;
        --interstp-iso=*) INTERSTP_ISO="${1#*=}" ;;
        --no-nat)      WITH_NAT=0 ;;
        --net-mode)    NET_MODE="${2:-}"; shift ;;
        --net-mode=*)  NET_MODE="${1#*=}" ;;
        --hostonly)    NET_MODE="hostonly" ;;
        --host-node)   HOST_NODE="${2:-}"; NET_MODE="hostonly"; shift ;;
        --host-node=*) HOST_NODE="${1#*=}"; NET_MODE="hostonly" ;;
        --fix)         FIX=1 ;;
        --no-usb)      WITH_USB=0 ;;
        --no-audio)    WITH_AUDIO=0 ;;
        --no-vrde)     WITH_VRDE=0 ;;
        --audio-driver)   AUDIO_DRIVER="${2:-}"; shift ;;
        --audio-driver=*) AUDIO_DRIVER="${1#*=}" ;;
        --usb-filter)     EXTRA_FILTERS+=("${2:-}"); shift ;;
        --usb-filter=*)   EXTRA_FILTERS+=("${1#*=}") ;;
        --share)       SHARE="${2:-}"; shift ;;
        --share=*)     SHARE="${1#*=}" ;;
        --vm)          USB_VM="${2:-}"; shift ;;
        --vm=*)        USB_VM="${1#*=}" ;;
        --add)         USB_ADD="${2:-}"; shift ;;
        --add=*)       USB_ADD="${1#*=}" ;;
        --net)         NETNAME="${2:-}"; shift ;;
        --net=*)       NETNAME="${1#*=}" ;;
        -h|--help)     sed -n '2,33p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo -e "${RED}option inconnue : $1${NC}" >&2; exit 2 ;;
    esac
    shift
done

[[ "$NODES" =~ ^[1-9]$ ]] || { echo -e "${RED}--nodes doit valoir 1 a 9${NC}" >&2; exit 2; }

VBM="$(command -v VBoxManage || command -v vboxmanage || true)"
[ -n "$VBM" ] || { echo -e "${RED}VBoxManage introuvable - installez VirtualBox.${NC}" >&2; exit 1; }
vbm() { "$VBM" "$@" 2>&1 | grep -v 'vboxdrv\|vboxconfig\|recompile the kernel\|You will not be able\|available for
 ↪  the current kernel\|load. Please\|^$' || true; }

# ── `pipefail` + `grep -q` trap ──────────────────────────────────────────
# `grep -q` exits at the FIRST match; the pipeline producer then gets SIGPIPE
# and dies with 141, and under `set -o pipefail` the pipeline returns 141.
# "Found" would read as "not found", intermittently - depending on whether the
# output fitted in the pipe buffer. So read the whole input, discard the output.
qgrep() { grep "$@" >/dev/null; }

# What the host can do - determined ONCE, used everywhere.
HAS_EXTPACK=0
"$VBM" list extpacks 2>/dev/null | qgrep 'Usable:.*true' && HAS_EXTPACK=1
if [ -z "$AUDIO_DRIVER" ]; then
    if pactl info >/dev/null 2>&1 || [ -S "/run/user/$(id -u)/pulse/native" ]; then AUDIO_DRIVER="pulse"
    elif [ -d /proc/asound ]; then AUDIO_DRIVER="alsa"
    else AUDIO_DRIVER="none"; fi
fi
USB_FILTER_COUNT=0

vm_name() { echo "${VM_PREFIX}$1"; }
node_ip()  { echo "${SUBNET}.$((10 + $1))"; }

# ── Node table ──────────────────────────────────────────────────────────
build_table() {
    local i ind
    local -a ind_arr=()
    [ -n "$INDS" ] && read -r -a ind_arr <<< "$INDS"
    WAN_IP=(); WAN_IND=(); WAN_NODE_LIST=()
    for i in $(seq 1 "$NODES"); do
        ind="${ind_arr[$((i-1))]:-$(wan_default_ind "$i")}"
```

```
        # An addressing plan without exception: node N → .1N, VM or host. The
        # host simply gets its own on the host-only interface instead of
        # receiving it through DHCP.
        WAN_IP[$i]="$(node_ip "$i")"
        WAN_IND[$i]="$ind"; WAN_NODE_LIST+=("$i")
    done
    WAN_NODE_COUNT="$NODES"; WAN_NODE_ID="${HOST_NODE:-1}"
    wan_nodes_validate || exit 1
}

find_iso() {
    local latest
    latest="$(ls -t "$DIR"/osmo-operator-*.iso 2>/dev/null | head -1)"
    [ -n "$latest" ] && { echo "$latest"; return 0; }
    return 1
}

# ── Host-only network: the host takes a leg on the segment ──────────────
# Needed as soon as the host is itself a WAN node. With an "internal" network
# the VMs talk to each other but the host sees NOTHING - a docker lab on the
# host could not reach its peers, silently.
setup_hostonly() {
    local existing
    existing="$("$VBM" list hostonlyifs 2>/dev/null | awk -F': *' '/^Name:/{print $2; exit}')"
    if [ -n "$existing" ]; then
        HOSTONLY_IF="$existing"
    else
        "$VBM" hostonlyif create >/dev/null 2>&1
        HOSTONLY_IF="$("$VBM" list hostonlyifs 2>/dev/null | awk -F': *' '/^Name:/{print $2; exit}')"
    fi
    [ -n "$HOSTONLY_IF" ] || { echo -e "${RED}creation de l'interface host-only impossible${NC}" >&2; return 1; }

    # The interface carries the host node address (.1N), not a separate one:
    # what the table announces is exactly what the peers will reach.
    local host_ip; host_ip="$(node_ip "${HOST_NODE:-1}")"
    if ! "$VBM" hostonlyif ipconfig "$HOSTONLY_IF" --ip "$host_ip" --netmask 255.255.255.0 >/dev/null 2>&1; then
        # VirtualBox refuses any range outside 192.168.56.0/21 as long as
        # /etc/vbox/networks.conf does not allow it. Explicit message:
        # otherwise only an "E_ACCESSDENIED" that points at nothing.
        echo -e "${RED}VirtualBox refuse ${host_ip} sur ${HOSTONLY_IF}.${NC}" >&2
        echo -e "  Les reseaux host-only sont limites a ${CYAN}192.168.56.0/21${NC} par defaut." >&2
        echo -e "  Soit : ${CYAN}--subnet 192.168.56${NC}" >&2
        echo -e "  Soit, pour autoriser votre plage :" >&2
        echo -e "    ${CYAN}echo '* ${SUBNET}.0/24' | sudo tee -a /etc/vbox/networks.conf${NC}" >&2
        return 1
    fi
    echo -e "  ${GREEN}✓${NC} ${HOSTONLY_IF} @ ${CYAN}${host_ip}${NC} (l'hote sur le segment, noeud
     ↪  ${HOST_NODE:-1})"
}

# ── VM hardware: USB, sound, clipboard, remote console ──────────────────
# A lab node is a RADIO mock-up: without an SDR dongle passed to the VM and
# without a sound card, only a network core remains. That is why everything
# is enabled by default here, unlike an ordinary test VM.
apply_hardware() {
    local vm="$1" idx="$2" f vid pid name i

    # ── USB ──────────────────────────────────────────────────────────────
    if [ "$WITH_USB" -eq 1 ]; then
        # xHCI (USB 3) requires the Extension Pack; without it fall back to
        # OHCI (USB 1.1), enough for a Calypso serial cable but NOT for an
        # SDR dongle - the throughput does not keep up and the capture drops
        # without a clear message.
        if [ "$HAS_EXTPACK" -eq 1 ]; then
            vbm modifyvm "$vm" --usb-xhci on
        else
```

```
            vbm modifyvm "$vm" --usb-ohci on
        fi
        i=0
        for f in "${USB_KNOWN[@]}" ${EXTRA_FILTERS[@]+"${EXTRA_FILTERS[@]}"}; do
            vid="${f%%:*}"; name="${f#*:}"; pid="${name%%:*}"; name="${name#*:}"
            [ "$name" = "$pid" ] && name="usb-${vid}-${pid}"
            vbm usbfilter add "$i" --target "$vm" --name "$name" \
                --vendorid "$vid" --productid "$pid" >/dev/null 2>&1 && i=$((i+1))
        done
        USB_FILTER_COUNT="$i"
    fi

    # ── Sound ──────────────────────────────────────────────────────────
    # INPUT AND OUTPUT: the GSM audio chain (osmo-gapk) carries a call both
    # ways. A VM with output only gives a one-way call, which is very hard to
    # diagnose - it looks like a codec problem.
    if [ "$WITH_AUDIO" -eq 1 ]; then
        vbm modifyvm "$vm" --audio-driver "$AUDIO_DRIVER" --audio-controller hda \
            --audio-enabled on --audio-in on --audio-out on
    else
        vbm modifyvm "$vm" --audio-enabled off
    fi

    # ── Comfort: clipboard and drag-and-drop ──────────────────────────
    vbm modifyvm "$vm" --clipboard-mode bidirectional --drag-and-drop bidirectional

    # ── Remote console ────────────────────────────────────────────────
    # The lab starts headless: without this, no way to see the console of a
    # VM that refuses to boot.
    if [ "$WITH_VRDE" -eq 1 ] && [ "$HAS_EXTPACK" -eq 1 ]; then
        vbm modifyvm "$vm" --vrde on --vrde-address 127.0.0.1 --vrde-port "$((3900 + idx))"
    fi

    # ── Shared folder ─────────────────────────────────────────────────
    if [ -n "$SHARE" ]; then
        vbm sharedfolder add "$vm" --name osmo --hostpath "$SHARE" --automount
    fi
}

# ── create ─────────────────────────────────────────────────────────────
do_create() {
    build_table
    echo -e "${CYAN}${BOLD}"
    echo "╔═══════════════════════════════════════════════════════════════╗"
    printf "║   Banc WAN VirtualBox - %d noeud(s), reseau interne "%-8s ║\n" "$NODES" "${NETNAME}""
    echo "╚═══════════════════════════════════════════════════════════════╝"
    echo -e "${NC}"
    wan_nodes_summary
    echo ""

    if [ "$BUILD_ISO" -eq 1 ]; then
        echo -e "${GREEN}[1/4] Construction de l'ISO avec la table embarquee...${NC}"
        echo -e "  ${CYAN}sudo $DIR/build-iso.sh --wan --wan-nodes \"$(wan_nodes_spec)\"${NC}"
        sudo "$DIR/build-iso.sh" --wan --wan-nodes "$(wan_nodes_spec)" \
            || { echo -e "${RED}build-iso.sh a echoue${NC}"; exit 1; }
        ISO="$(find_iso)" || { echo -e "${RED}ISO introuvable apres build${NC}"; exit 1; }
    elif [ -z "$ISO" ]; then
        ISO="$(find_iso)" || {
            echo -e "${RED}Aucune ISO. Passez --iso chemin.iso, ou --build-iso.${NC}" >&2; exit 1; }
        echo -e "${GREEN}[1/4] ISO : ${CYAN}${ISO}${NC}"
        echo -e "  ${YELLOW}Verifiez qu'elle a ete construite avec --wan et CETTE table,${NC}"
        echo -e "  ${YELLOW}sinon les VM demarreront sans WAN (start-native.sh --wan a la main).${NC}"
    else
        echo -e "${GREEN}[1/4] ISO : ${CYAN}${ISO}${NC}"
    fi
    [ -r "$ISO" ] || { echo -e "${RED}ISO illisible : $ISO${NC}" >&2; exit 1; }
```

```
# ── Network + fixed-address DHCP ──────────────────────────────────────
if [ "$NET_MODE" = "hostonly" ]; then
    echo -e "${GREEN}[2/4] Reseau host-only (l'hote est un noeud) + DHCP...${NC}"
    setup_hostonly || exit 1
    NETNAME="HostInterfaceNetworking-${HOSTONLY_IF}"
else
    echo -e "${GREEN}[2/4] Reseau interne "${NETNAME}" + DHCP...${NC}"
fi
if "$VBM" list dhcpservers 2>/dev/null | qgrep "NetworkName:.*${NETNAME}"; then
    vbm dhcpserver modify --network="$NETNAME" \
        --server-ip="${SUBNET}.254" --netmask=255.255.255.0 \
        --lower-ip="${SUBNET}.100" --upper-ip="${SUBNET}.200" --enable
else
    vbm dhcpserver add --network="$NETNAME" \
        --server-ip="${SUBNET}.254" --netmask=255.255.255.0 \
        --lower-ip="${SUBNET}.100" --upper-ip="${SUBNET}.200" --enable
fi
# The DHCP is at .254, NOT .1: .1 is reserved for the inter-STP, the SS7
# hub shared by the nodes. All node configs name that address literally
# - it cannot depend on a lease.
echo -e "  ${GREEN}✓${NC} DHCP ${SUBNET}.254 · inter-STP reserve en ${CYAN}${SUBNET}.1${NC}"

# ── VM ────────────────────────────────────────────────────────────────
echo -e "${GREEN}[3/4] Creation des ${NODES} VM...${NC}"
for i in $(seq 1 "$NODES"); do
    if [ "$i" = "$HOST_NODE" ]; then
        echo -e "  ${CYAN}-${NC} noeud ${i} : c'est CETTE machine ($(node_ip "$i")), pas de VM"
        continue
    fi
    local vm; vm="$(vm_name "$i")"
    if "$VBM" list vms 2>/dev/null | qgrep "\"${vm}\""; then
        echo -e "  ${YELLOW}⚠${NC} ${vm} existe deja - ignoree (destroy pour repartir a neuf)"
        continue
    fi
    vbm createvm --name "$vm" --ostype Ubuntu_64 --register
    vbm modifyvm "$vm" --memory "$RAM" --cpus "$CPUS" --ioapic on \
        --boot1 dvd --boot2 disk --boot3 none --boot4 none \
        --graphicscontroller vmsvga --vram 64 \
        --rtc-use-utc on --paravirt-provider kvm \
        --nested-hw-virt on
    # NIC1 = the WAN. THIS card is the one the table addresses.
    if [ "$NET_MODE" = "hostonly" ]; then
        vbm modifyvm "$vm" --nic1 hostonly --host-only-adapter1 "$HOSTONLY_IF" --nictype1 virtio
    else
        vbm modifyvm "$vm" --nic1 intnet --intnet1 "$NETNAME" --nictype1 virtio
    fi
    apply_hardware "$vm" "$i"
    if [ "$WITH_NAT" -eq 1 ]; then
        # NIC2 = NAT: Internet egress (apt, osmo-update) and SSH forwarding
        # from the host. It carries NO WAN traffic - its 10.0.2.15 is not
        # in the table, so the node detection is not fooled.
        vbm modifyvm "$vm" --nic2 nat --nictype2 virtio
        vbm modifyvm "$vm" --natpf2 "ssh${i},tcp,127.0.0.1,$((2220 + i)),,22"
        # Dashboard and spectra of the lab, reachable from the host without
        # going through the WAN segment (which must stay clean).
        vbm modifyvm "$vm" --natpf2 "web${i},tcp,127.0.0.1,$((8080 + (i-1)*10)),,8080"
        vbm modifyvm "$vm" --natpf2 "fft${i},tcp,127.0.0.1,$((8081 + (i-1)*10)),,8081"
    fi
    vbm storagectl "$vm" --name IDE --add ide
    vbm storageattach "$vm" --storagectl IDE --port 0 --device 0 --type dvddrive --medium "$ISO"
    # Fixed address: it is what gives the node its IDENTITY.
    vbm dhcpserver modify --network="$NETNAME" --vm="$vm" --nic=1 \
        --fixed-address="$(node_ip "$i")" \
        || echo -e "  ${YELLOW}⚠${NC} adresse fixe refusee pour ${vm} - VirtualBox < 6.1 ?"
```

```
        echo -e "  ${GREEN}✓${NC} ${vm}  ${CYAN}$(node_ip "$i")${NC}  indicatif ${CYAN}${WAN_IND[$i]}${NC}$([
        ↪  "$WITH_NAT" -eq 1 ] && echo "  ssh 127.0.0.1:$((2220 + i))")"
    done

    # ── Inter-STP VM: the SS7 hub, at the reserved .1 address ──────────────
    if [ -n "$INTERSTP_ISO" ]; then
        if "$VBM" list vms 2>/dev/null | qgrep "\"${INTERSTP_VM}\""; then
            echo -e "  ${YELLOW}⚠${NC} ${INTERSTP_VM} existe deja - ignoree"
        elif [ ! -r "$INTERSTP_ISO" ]; then
            echo -e "  ${RED}✗${NC} ISO inter-STP illisible : ${INTERSTP_ISO}"
        else
            vbm createvm --name "$INTERSTP_VM" --ostype Ubuntu_64 --register
            # The hub carries neither radio nor audio: it routes M3UA. So it
            # gets no sound, no USB, nor the 4 GB of a full node.
            vbm modifyvm "$INTERSTP_VM" --memory 1536 --cpus 1 --ioapic on \
                --boot1 dvd --boot2 disk --boot3 none --boot4 none \
                --graphicscontroller vmsvga --vram 16 --rtc-use-utc on \
                --paravirt-provider kvm --audio-enabled off
            if [ "$NET_MODE" = "hostonly" ]; then
                vbm modifyvm "$INTERSTP_VM" --nic1 hostonly --host-only-adapter1 "$HOSTONLY_IF" --nictype1 virtio
            else
                vbm modifyvm "$INTERSTP_VM" --nic1 intnet --intnet1 "$NETNAME" --nictype1 virtio
            fi
            [ "$WITH_NAT" -eq 1 ] && vbm modifyvm "$INTERSTP_VM" --nic2 nat --nictype2 virtio \
                --natpf2 "sshstp,tcp,127.0.0.1,2229,,22"
            vbm storagectl "$INTERSTP_VM" --name IDE --add ide
            vbm storageattach "$INTERSTP_VM" --storagectl IDE --port 0 --device 0 \
                --type dvddrive --medium "$INTERSTP_ISO"
            vbm dhcpserver modify --network="$NETNAME" --vm="$INTERSTP_VM" --nic=1 \
                --fixed-address="${SUBNET}.1" \
                || echo -e "  ${YELLOW}⚠${NC} adresse fixe refusee pour ${INTERSTP_VM}"
            echo -e "  ${GREEN}✓${NC} ${INTERSTP_VM}  ${CYAN}${SUBNET}.1${NC}  hub SS7 (PC 0.0.0)"
        fi
    fi

    echo -e "${GREEN}[4/4] Materiel passe aux VM :${NC}"
    if [ "$WITH_USB" -eq 1 ]; then
        echo -e "  USB        : ${CYAN}$([ "$HAS_EXTPACK" -eq 1 ] && echo 'xHCI (USB 3)' || echo 'OHCI (USB 1.1,
        ↪  sans Extension Pack)')${NC}, ${USB_FILTER_COUNT} filtre(s)"
        if ! id -nG "${SUDO_USER:-$USER}" 2>/dev/null | qgrep -w vboxusers; then
            echo -e "  ${YELLOW}⚠ ${SUDO_USER:-$USER} n'est pas dans le groupe vboxusers :${NC}"
            echo -e "    ${YELLOW}les peripheriques USB resteront a l'hote, sans message d'erreur.${NC}"
            echo -e "    ${CYAN}sudo usermod -aG vboxusers ${SUDO_USER:-$USER}${NC}  puis rouvrir la session"
        fi
    else
        echo -e "  USB        : ${YELLOW}desactive (--no-usb)${NC}"
    fi
    echo -e "  Son        : ${CYAN}${AUDIO_DRIVER}${NC} (entree + sortie)"
    [ "$WITH_VRDE" -eq 1 ] && [ "$HAS_EXTPACK" -eq 1 ] \
        && echo -e "  Console    : ${CYAN}RDP 127.0.0.1:390N${NC} (N = numero de noeud)"
    [ -n "$SHARE" ] && echo -e "  Partage    : ${CYAN}${SHARE}${NC} → /media/sf_osmo (additions invite requises)"
    echo ""
    if ! grep -q '^vboxdrv ' /proc/modules 2>/dev/null; then
        echo ""
        echo -e "  ${YELLOW}vboxdrv n'est pas charge : les VM sont creees mais ne demarreront pas.${NC}"
        echo -e "  ${CYAN}sudo /sbin/vboxconfig${NC}  (recompile le module pour le noyau courant)"
    fi
    do_hint
}

do_hint() {
    build_table
    echo ""
    echo -e "  ${BOLD}Demarrer :${NC} ${CYAN}$0 start${NC}"
    echo ""
    echo -e "  ${BOLD}Dans chaque VM (autologin root) :${NC}"
```

```
    echo -e "    ${CYAN}/opt/GSM/osmo-operator/start-native.sh${NC}"
    echo -e "    → la table WAN figee dans l'ISO est appliquee toute seule (WAN_AUTO=1)."
    echo -e "    → sans ISO --wan : ${CYAN}/opt/GSM/osmo-operator/start-native.sh --wan${NC}"
    echo ""
    echo -e "  ${BOLD}Appeler d'un noeud a l'autre :${NC}"
    local a b
    a="${WAN_NODE_LIST[0]}"; b="${WAN_NODE_LIST[$(( ${#WAN_NODE_LIST[@]} > 1 ? 1 : 0 ))]}"
    echo -e "    depuis un MS du noeud ${a} : ${CYAN}${WAN_IND[$b]}10001${NC} → MS 10001 du noeud ${b}"
    echo -e "    SMS : meme numerotation (${CYAN}${WAN_IND[$b]}10001${NC})"
    echo ""
    echo -e "  ${BOLD}Diagnostic dans une VM :${NC} ${CYAN}/opt/GSM/osmo-operator/checks/wan_ss7_check.sh${NC}"
    echo ""
}

do_each() {  # $1 = vboxmanage command on each existing VM
    local i vm found=0
    # The hub first at start: a node coming up before it gets its SCTP
    # refused and does not necessarily retry.
    for i in interstp $(seq 1 9); do
        if [ "$i" = "interstp" ]; then vm="$INTERSTP_VM"; else vm="$(vm_name "$i")"; fi
        "$VBM" list vms 2>/dev/null | qgrep "\"${vm}\"" || continue
        found=1
        case "$1" in
            start)   vbm startvm "$vm" --type headless && echo -e "  ${GREEN}▶${NC} ${vm}" ;;
            stop)    vbm controlvm "$vm" acpipowerbutton >/dev/null 2>&1 || true
                     echo -e "  ${YELLOW}■${NC} ${vm}" ;;
            destroy) vbm controlvm "$vm" poweroff >/dev/null 2>&1 || true
                     sleep 1
                     vbm unregistervm "$vm" --delete >/dev/null 2>&1 || true
                     echo -e "  ${RED}✗${NC} ${vm} supprimee" ;;
            status)
                    local st ip; st="$("$VBM" showvminfo "$vm" --machinereadable 2>/dev/null | awk -F'"'
                     ↪  '/^VMState=/{print $2}')"
                    if [ "$i" = "interstp" ]; then ip="${SUBNET}.1 (hub SS7)"; else ip="$(node_ip "$i")"; fi
                    printf '  %-20s %-10s %s\n' "$vm" "${st:-?}" "$ip" ;;
        esac
    done
    [ "$found" = "1" ] || echo -e "  ${YELLOW}aucune VM ${VM_PREFIX}* - lancez "$0 create"${NC}"
}

# The REAL groups of an open session of $1: read /proc/<pid>/status of one of
# its processes. `id -nG` would not do - it queries the database, not the
# session, and would answer "yes" right after a usermod while nothing has
# changed for the programs already running.
session_has_group() {
    local user="$1" grp="$2" gid pid
    gid="$(getent group "$grp" | cut -d: -f3)"
    [ -n "$gid" ] || return 1
    for pid in $(pgrep -u "$user" 2>/dev/null | head -40); do
        [ -r "/proc/$pid/status" ] || continue
        awk -v g="$gid" '/^Groups:/ { for (i=2; i<=NF; i++) if ($i == g) { found=1 } }
                         END { exit !found }' "/proc/$pid/status" && return 0
    done
    return 1
}

# ── check: can the host run the bench? ──────────────────────────────
do_check() {
    local rc=0 u="${SUDO_USER:-$USER}"
    echo -e "${BOLD}Hote VirtualBox${NC}"
    printf '─%.0s' $(seq 1 60); echo ""

    echo -e "  ${GREEN}✓${NC} VBoxManage $("$VBM" --version 2>/dev/null | tail -1)"

    if grep -q '^vboxdrv ' /proc/modules 2>/dev/null; then
        echo -e "  ${GREEN}✓${NC} module vboxdrv charge"
```

```
    else
        echo -e "  ${RED}✗${NC} vboxdrv non charge - aucune VM ne demarrera"
        echo -e "      ${CYAN}sudo /sbin/vboxconfig${NC}"
        if command -v mokutil >/dev/null 2>&1 && mokutil --sb-state 2>/dev/null | qgrep -i enabled; then
           echo -e "     ${YELLOW}Secure Boot est actif : le module doit etre signe et sa cle enrolee (MOK).${NC}"
        fi
        rc=1
    fi

    if [ "$HAS_EXTPACK" -eq 1 ]; then
        echo -e "  ${GREEN}✓${NC} Extension Pack - USB 2.0/3.0 et console RDP disponibles"
    else
        echo -e "  ${YELLOW}⚠${NC} pas d'Extension Pack : USB 1.1 seulement (insuffisant pour une cle SDR), pas de
         ↪  RDP"
    fi

    # Two distinct states, and that is the WHOLE difficulty of this point:
    #   • /etc/group may already contain the user...
    #   • ...while the current SESSION still runs with the old group list.
    #     Processes inherit their groups at login: `usermod` does not reach
    #     them retroactively. VirtualBox launched from that session still
    #     cannot open /dev/vboxusb - and the error shown speaks of a "busy"
    #     device, never of groups.
    # So look at BOTH: the database, and the real groups of a live process
    # of the user.
    if id -nG "$u" 2>/dev/null | qgrep -w vboxusers; then
        if session_has_group "$u" vboxusers; then
            echo -e "  ${GREEN}✓${NC} ${u} est dans vboxusers, et sa session l'a bien pris"
        else
            echo -e "  ${YELLOW}⚠${NC} ${u} est dans vboxusers, mais ${BOLD}sa session ouverte ne l'a pas
             ↪  encore${NC}"
            echo -e "      Les processus gardent les groupes recus au login."
            echo -e "      ${CYAN}Deconnectez-vous et reconnectez-vous${NC} (ou redemarrez) - puis relancez ce
             ↪  check."
            echo -e "      Pour tester tout de suite, sans quitter la session : ${CYAN}newgrp vboxusers${NC}"
            rc=1
        fi
    else
        echo -e "  ${RED}✗${NC} ${u} n'est PAS dans vboxusers - les filtres USB ne capteront rien"
        if [ "$FIX" -eq 1 ] && [ "$(id -u)" -eq 0 ]; then
            usermod -aG vboxusers "$u" && echo -e "      ${GREEN}✓ ajoute${NC} - deconnectez-vous et
             ↪  reconnectez-vous pour l'activer"
        else
            echo -e "      ${CYAN}sudo usermod -aG vboxusers ${u}${NC}  puis rouvrir la session"
            echo -e "      (ou relancez : ${CYAN}sudo $0 check --fix${NC})"
        fi
        rc=1
    fi

    echo -e "  ${GREEN}✓${NC} son : pilote ${CYAN}${AUDIO_DRIVER}${NC}"

    if find_iso >/dev/null 2>&1; then
        echo -e "  ${GREEN}✓${NC} ISO : $(find_iso)"
    else
        echo -e "  ${YELLOW}⚠${NC} aucune ISO dans le depot - ${CYAN}$0 create --build-iso${NC}"
    fi

    echo ""
    echo -e "${BOLD}Peripheriques USB branches${NC}"
    printf '─%.0s' $(seq 1 60); echo ""
    do_usb_list
    echo ""
    [ $rc -eq 0 ] && echo -e "  ${GREEN}Hote pret.${NC}" || echo -e "  ${YELLOW}Corrigez les points ci-dessus
     ↪  avant de creer le banc.${NC}"
    return $rc
}
```

```
# Lists the host devices and says which ones a bench filter would capture.
do_usb_list() {
    local out vid pid name known f kv kp
    out="$("$VBM" list usbhost 2>/dev/null)"
    if [ -z "$out" ]; then echo -e "  ${YELLOW}aucun peripherique USB visible${NC}"; return 0; fi
    echo "$out" | awk -v RS='' '{print}' | while IFS= read -r line; do :; done
    printf '%s\n' "$out" | awk '
        /^VendorId:/  { v=$0; sub(/.*0x/,"",v); sub(/ .*/,"",v); vid=v }
        /^ProductId:/ { p=$0; sub(/.*0x/,"",p); sub(/ .*/,"",p); pid=p }
        /^Product:/   { pr=$0; sub(/^Product: */,"",pr); prod=pr }
        /^Manufacturer:/ { m=$0; sub(/^Manufacturer: */,"",m); man=m }
        /^Current State:/ { st=$0; sub(/^Current State: */,"",st)
                            if (vid != "") printf "%s:%s|%s|%s|%s\n", vid, pid, man, prod, st
                            vid=""; pid=""; prod=""; man="" }
    ' | while IFS='|' read -r ids man prod st; do
        known=""
        for f in "${USB_KNOWN[@]}" ${EXTRA_FILTERS[@]+"${EXTRA_FILTERS[@]}"}; do
            kv="${f%%:*}"; kp="${f#*:}"; kp="${kp%%:*}"
            [ "${kv}:${kp}" = "$ids" ] && { known="${f#*:*:}"; break; }
        done
        if [ -n "$known" ]; then
            printf '  ✓ %-12s %-34s → capte (%s)\n' "$ids" "${prod:-$man}" "$known"
        else
            printf '    %-12s %-34s %s\n' "$ids" "${prod:-$man}" "$st"
        fi
    done
    echo ""
    echo -e "  ${CYAN}Pour capter un peripherique absent de la liste :${NC}"
    echo -e "    $0 usb --vm 1 --add ${CYAN}vid:pid:nom${NC}"
}

do_usb() {
    if [ -n "$USB_ADD" ]; then
        [ -n "$USB_VM" ] || { echo -e "${RED}--add exige --vm N${NC}" >&2; exit 2; }
        local vm vid pid name idx
        vm="$(vm_name "$USB_VM")"
       "$VBM" list vms 2>/dev/null | qgrep "\"${vm}\"" || { echo -e "${RED}${vm} n'existe pas${NC}" >&2; exit 1; }
        vid="${USB_ADD%%:*}"; name="${USB_ADD#*:}"; pid="${name%%:*}"; name="${name#*:}"
        [ "$name" = "$pid" ] && name="usb-${vid}-${pid}"
        idx="$("$VBM" showvminfo "$vm" --machinereadable | grep -c '^USBFilterName')"
        vbm usbfilter add "$idx" --target "$vm" --name "$name" --vendorid "$vid" --productid "$pid"
        echo -e "  ${GREEN}✓${NC} ${vm} captera ${CYAN}${vid}:${pid}${NC} (${name})"
        echo -e "  ${CYAN}Actif au prochain demarrage de la VM, ou au prochain branchement.${NC}"
    else
        do_usb_list
    fi
}

case "$ACTION" in
    check)   do_check ;;
    usb)     do_usb ;;
    create)  do_create ;;
    start)   echo -e "${GREEN}Demarrage...${NC}"; do_each start ;;
    stop)    echo -e "${YELLOW}Arret...${NC}";    do_each stop ;;
    destroy) echo -e "${RED}Suppression...${NC}"; do_each destroy
             vbm dhcpserver remove --network="$NETNAME" >/dev/null 2>&1 || true ;;
    status)  echo -e "${BOLD}VM du banc WAN :${NC}"; do_each status ;;
    table)   build_table; wan_nodes_summary
             echo ""; echo "  --wan-nodes \"$(wan_nodes_spec)\"" ;;
    ""|-h|--help) sed -n '2,33p' "$0" | sed 's/^# \{0,1\}//' ;;
    *) echo -e "${RED}action inconnue : $ACTION${NC}" >&2; exit 2 ;;
esac
```

2.290 /opt/GSM/osmo-operator/tools/vty-menu.sh

5981 octets, 156 lignes → 156 lignes

```
#!/bin/bash
# the mobile VTY port: 4247, or 4347 with --dsp (published by start-native.sh)
BB_VTY="$(tr -dc 0-9 < /run/osmo-banc/ms1-vty 2>/dev/null)"; BB_VTY="${BB_VTY:-4247}"

# Save the current terminal configuration
OLD_SETTINGS=$(stty -g)
# Save the initial colours (if supported)
INITIAL_BG=$(printf '\e]11;?\a')

# "Nord Theme" colour palette
set_kernel_profile() {
    # Change background and text
    echo -ne "\e]11;#2E3440\a" # Fond bleu-gris fonce
    echo -ne "\e]10;#D8DEE9\a" # Texte blanc casse
    clear
}

reset_terminal_profile() {
    # 1. Restore the original colours of the user profile
    echo -ne "\e]110\a" # Reset Foreground
    echo -ne "\e]111\a" # Reset Background
    echo -ne "\e]112\a" # Reset cursor color

    # 2. System command to reset the terminal
    tput reset
    tput sgr0

    # 3. Restore the stty settings
    stty "$OLD_SETTINGS"

    clear
    echo "--- Session SDR terminee, profil restaure ---"
}

# Matching whiptail theme
export NEWT_COLORS='
  root=,rgb:2e/34/40
  window=rgb:d8/de/e9,rgb:3b/42/52
  shadow=,rgb:1a/1c/23
  title=rgb:88/c0/d0,rgb:3b/42/52
  button=rgb:2e/34/40,rgb:81/a1/c1
  actbutton=rgb:d8/de/e9,rgb:5e/81/ac
  listbox=rgb:d8/de/e9,rgb:3b/42/52
  actlistbox=rgb:88/c0/d0,rgb:43/4c/5e
'

set_kernel_profile
trap reset_terminal_profile EXIT

connect_vty() {
    tput cnorm # S'assure que le curseur est visible
    expect ./tools/vty-connect.exp "$1" "$3" "$4"
    [[ $? -ne 0 ]] && { echo -e "\n[!] Service hors ligne."; sleep 1; }
}

# ...

menu_services() {
    local OP=$1
    while true; do
        # Option 7 is the Baseband
        CHOICE=$(whiptail --title " OPERATOR CONTROL : $OP " --menu "" 20 62 9 \
```

```
            "1" "MSC (4254)" "2" "BSC (4242)" "3" "HLR (4258)" \
            "4" "MGW (4243)" "5" "GGSN (4260)" "6" "SGSN (4245)" \
            "7" "STP (4239)" "8" "BASEBAND ($BB_VTY)" \
            "T" "TMUX - la pile en direct (session osmo)" \
            "R" "<< BACK" 3>&1 1>&2 2>&3)

        [[ -z "$CHOICE" || "$CHOICE" == "R" ]] && break

        case $CHOICE in
            1) connect_vty "$OP" "MSC" 4254 "msc" ;;
            2) connect_vty "$OP" "BSC" 4242 "bsc" ;;
            3) connect_vty "$OP" "HLR" 4258 "hlr" ;;
            4) connect_vty "$OP" "MGW" 4243 "mgw" ;;
            5) connect_vty "$OP" "GGSN" 4260 "gg" ;;
            6) connect_vty "$OP" "SGSN" 4245 "sg" ;;
            7) connect_vty "$OP" "STP" 4239 "stp" ;;
            T|t)
                # The session start-docker.sh opens in each operator container.
                # The launch terminal is taken by osmo-operator-1; this is how
                # the others are reached, without remembering the docker
                # command.
                tput cnorm
                # Several conventions coexist depending on the container:
                #   osmocom on /tmp/osmocom_tmux  the stack started by run.sh
                #   calypso / gapk on the default socket  the QEMU pipeline
                # They are tried in that order rather than imposing one: a
                # session announced but not found ("no sessions") would
                # suggest the stack is dead while it is running.
                _sock=""; _sess=""
                if docker exec "$OP" tmux -S /tmp/osmocom_tmux has-session -t osmocom 2>/dev/null; then
                    _sock="-S /tmp/osmocom_tmux"; _sess="osmocom"
                elif docker exec "$OP" tmux has-session -t calypso 2>/dev/null; then
                    _sess="calypso"
                elif docker exec "$OP" tmux has-session -t osmo 2>/dev/null; then
                    _sess="osmo"
                fi
                if [ -n "$_sess" ]; then
                    echo "  Session ${_sess} - detacher : Ctrl-b puis d"
                    # shellcheck disable=SC2086
                    docker exec -ti "$OP" tmux $_sock attach -t "$_sess"
                else
                   whiptail --msgbox "Pas de session tmux 'osmo' dans $OP.\n\nLe journal reste lisible :\n  docker
                    ↪  exec $OP tail -f /var/log/osmocom/run.sh.log" 12 66
                fi
                ;;
            8)
                # Multi-group Baseband handling (127.0.0.1, 127.0.0.2, etc.)
                local GRP_IP
                GRP_IP=$(whiptail --title " BASEBAND GROUP SELECT " --inputbox "Target IP (127.0.0.X) :" 10 40
                ↪  "127.0.0.1" 3>&1 1>&2 2>&3)
                if [[ -n "$GRP_IP" ]]; then
                    # Pass the group IP to the expect script
                    tput cnorm
                    expect ./tools/vty-connect.exp "$OP" "$BB_VTY" "bb" "$GRP_IP"
                fi
                ;;
        esac
    done
}

# --- MAIN LOOP (guarded) ---
while true; do
    MAIN_TYPE=$(whiptail --title " OSMO-SDR STACK MANAGER " --menu "Select Category :" 15 60 3 \
        "CORE" "Global Infrastructure (Inter-STP)" \
        "OPS"  "Network Operators (MSC/BSC/STP...)" \
        "EXIT" "Quit Manager" 3>&1 1>&2 2>&3)
```

```
    [[ -z "$MAIN_TYPE" || "$MAIN_TYPE" == "EXIT" ]] && break

    if [[ "$MAIN_TYPE" == "CORE" ]]; then
        # Check whether the inter-stp is alive
        if docker ps --format '{{.Names}}' | grep -q "osmo-inter-stp"; then
            docker exec -ti osmo-inter-stp telnet 127.0.0.1 4239
        else
            whiptail --msgbox "Erreur : osmo-inter-stp n'est pas lance." 10 50
        fi
    else
        # Get the active operators
        OPS=$(docker ps --format "{{.Names}}" | grep "osmo-operator" | sort)

        if [[ -z "$OPS" ]]; then
            whiptail --msgbox "Aucun operateur detecte ! Verifiez start-docker.sh" 10 50
            continue
        fi

        MENU_LIST=()
        while read -r line; do MENU_LIST+=("$line" "Status: Active"); done <<< "$OPS"

        OP_CHOICE=$(whiptail --title " OPERATOR SELECTION " --menu "Select Operator Node :" 20 70 10 \
            "${MENU_LIST[@]}" 3>&1 1>&2 2>&3)

        [[ -n "$OP_CHOICE" ]] && menu_services "$OP_CHOICE"
    fi
done

reset_terminal_profile
```

2.291 /opt/GSM/osmo-operator/tools/wallpaper-render.py

22343 octets, 444 lignes → 444 lignes

```
#!/usr/bin/env python3
# wallpaper-render.py - composes the bench wallpaper (1920x1080).
#
#   python3 wallpaper-render.py --tower configs/wallpaper/tower.png \
#           [--strip calvin.gif] [--date 2026-09-04] --out gsm-lab-wallpaper.png
#
# Layout, designed so that nothing overlaps on the screen:
#
#   +--------+-----------------------------------------+-------------------+
#   | dock + | LAB GSM card (x 320..1430, y 60..560)    | conky (400 px,    |
#   | desktop|                                         | gap 24, y 40..660)|
#   | icons  | inset (x 510..1410, CENTRED, y 220..1010)|                   |
#   | (top   |   top   : random geek comic / 4G spectrum|                   |
#   | left)  |   bottom: Calvin & Hobbes / 2G spectrum  |                   |
#   +--------+-----------------------------------------+-------------------+
#
# The pylon of the photo is in the MIDDLE (see background/TOWER_X): both
# panels are translucent and let it show through.
# The two side columns only receive the photo. On the left (x < 320): the
# GNOME dock (~70 px) then the desktop icons, top left (DING, start-corner
# top-left, ~100 px cells). On the right (x >= 1440): conky (top_right,
# gap_x 24, 400 px wide, cf. configs/conky/osmo-conky.conf).
#
# Without --strip (no network at build time, or at boot), the strip area
# stays empty: the wallpaper is complete without it. The strip of the day is
# placed by tools/osmo-wallpaper.sh (osmo-wallpaper service + timer).
#
# Depends only on Pillow (python3-pil) and the DejaVu fonts (fonts-dejavu),
# present on the image as on the host.
```

```
# THE INSET IS CENTRED, IN TWO FRAMES. The box (510, 220, 1410, 1010), i.e.
# 900x790, is CENTRED on the 1920 px (510 + 900/2 = 960) and stops before
# x=1440, where the conky column starts. It is split into two frames of 391 px
# separated by 8 px: the top one carries a randomly drawn geek comic (and the
# srsRAN UE spectrum when it runs), the bottom one Calvin & Hobbes (and the
# 2G spectrum). It overlaps the LAB GSM top zone (osmo-topzone.py,
# y 60..~400): intended, the banner stays a backdrop behind it.
# THE FOUR FILES THAT KNOW THIS BOX MUST STAY IN AGREEMENT:
#   tools/wallpaper-render.py  (the two frames drawn in the wallpaper)
#   tools/osmo-fft-snap.py     (the 900x790 image painted into it)
#   tools/osmo-panel.py        (the clickable GTK window placed on top)
#   tools/osmo-launcher.py     (where an app launched "in the panel" docks)
BOX_HAUT = (510, 220, 1410, 611)
BOX_BAS = (510, 619, 1410, 1010)
import argparse
import datetime
import os
import sys

from PIL import Image, ImageChops, ImageDraw, ImageEnhance, ImageFilter, ImageFont

W, H = 1920, 1080
FONT_DIR = "/usr/share/fonts/truetype/dejavu"


def font(name, size):
    try:
        return ImageFont.truetype(os.path.join(FONT_DIR, name), size)
    except OSError:
        return ImageFont.load_default()


def lerp(a, b, t):
    return tuple(int(a[i] + (b[i] - a[i]) * t) for i in range(3))


def gradient_h(w, h, stops):
    """Horizontal band w x h, gradient between the `stops` colours."""
    img = Image.new("RGB", (w, h))
    px = img.load()
    n = len(stops) - 1
    for x in range(w):
        t = x / max(1, w - 1) * n
        i = min(int(t), n - 1)
        c = lerp(stops[i], stops[i + 1], t - i)
        for y in range(h):
            px[x, y] = c
    return img


def rounded(draw, box, r, fill=None, outline=None, width=1):
    draw.rounded_rectangle(box, radius=r, fill=fill, outline=outline, width=width)


# ── ERASING THE PHOTO'S DEFECTS ──────────────────────────────────────────
# A backlit phone photo carries three marks, obvious once the pylon is in the
# middle and enlarged:
#
#   1. A MAGENTA VEIL. The sun in the lens throws a purplish diagonal halo
#      across the sky - a diaphragm ghost. On a night-blue background it reads
#      as a dirty streak.
#   2. CHROMATIC FRINGING. Dark uprights against a burnt sky take a purple
#      edge (lateral aberration). The sharpening that follows would REINFORCE
#      it: the defect gets sharpened along with the detail.
#   3. JPEG BLOCKING. The sky is a nearly flat gradient: exactly what JPEG
```

```
#      renders worst (8x8 squares and bands). Enlargement spreads them into
#      visible plates.
#
# Three targeted corrections, in this order - and BEFORE sharpening, otherwise
# what is about to be erased gets sharpened first.
#
# OSMO_WP_CLEAN=1 enables them (off by default, see below).
def nettoyer_photo(im):
    """Magenta defringing + sky smoothing, without touching the lattice."""
    # DEFAULT: DO NOTHING. This cleanup suits a backlit phone photo with flare,
    # fringing and JPEG blocking. The current photo is clean, and above all it
    # is PINK: a sunset sky, where green legitimately sits below the mean of
    # red and blue over half the image. The magenta correction would see a
    # defect everywhere and REPAINT THE SKY grey. The code stays, documented:
    # OSMO_WP_CLEAN=1 turns it on for a photo that needs it.
    if os.environ.get("OSMO_WP_CLEAN", "0") != "1":
        return im

    # ── 1 and 2. THE MAGENTA, VEIL AND FRINGES, IN ONE GESTURE ──────────────
    # A sky has R < G < B. The flare halo and the fringing share the same
    # signature: GREEN falls below the mean of red and blue (the definition
    # of magenta). Green is therefore raised up to that mean wherever it sits
    # below it. Nothing in the scene (sky and steel) is legitimately magenta,
    # so nothing legitimate is touched. A one-pass correction, no mask to tune.
    r, g, b = im.split()
    moy = ImageChops.add(r.point(lambda v: v // 2), b.point(lambda v: v // 2))
    g = ImageChops.lighter(g, moy)
    im = Image.merge("RGB", (r, g, b))

    # ── 3. THE SKY SMOOTHED, THE LATTICE INTACT ─────────────────────────────
    # The whole image cannot be blurred: the pylon would lose what the
    # sharpening gives it right after. So a DETAIL mask is built - the
    # difference between the image and its blurred version, i.e. the edges -
    # widened (MaxFilter) to cover the borders, then softened so that the
    # transition does not show. The blur is applied only WHERE THERE IS NO
    # DETAIL: the sky, and the sky alone.
    flou = im.filter(ImageFilter.GaussianBlur(2.2))
    gris = im.convert("L")
    ecart = ImageChops.difference(gris, gris.filter(ImageFilter.GaussianBlur(2.2)))
    masque = ecart.point(lambda v: 255 if v > 4 else 0)
    masque = masque.filter(ImageFilter.MaxFilter(5)).filter(ImageFilter.GaussianBlur(1.6))
    return Image.composite(im, flou, masque)


# ── WHERE THE PYLON LANDS ON THE SCREEN ─────────────────────────────────────
# The photo is anchored on the PYLON, not on the sun: the mast column of the
# source photo (TOWER_SRC) is brought to TOWER_X (0.62: between the LAB GSM
# card and the conky column - the exact centre, 0.5, would enlarge the photo
# too much and push the sun out of frame). The panels placed there are
# translucent, so the pylon reads through them.
#
# THE SCALE IS DERIVED FROM THE WANTED POSITION, not simply "cover the
# screen": the smallest scale that puts the pylon in place WITHOUT a gap on
# either side. The mirrored edge below is only a safety net for a photo with
# a different framing.
# x of the mast, and reference size of configs/wallpaper/tower.jpg.
# The mast sits at 34.2 % of the width, measured by looking for the most
# STRUCTURED column of the central band (sum of image-minus-blur detail,
# column by column). The ratio is what matters: background() rescales it to
# the real file size, so replacing the photo only requires this line.
# The photo is used AS IS, in PNG, at its native resolution (5824x3264): the
# render DOWNSCALES it anyway (scale ~0.57 to place the mast), and downscaling
# from full resolution gives a better result than downscaling twice.
TOWER_SRC = (1992, 5824, 3264)
TOWER_X = float(os.environ.get("OSMO_WP_TOWER_X", "0.62"))
```

```
def background(tower_path, tower_x=None, sun_y_at=984):
    """The photo as background: the pylon IN THE MIDDLE (tower_x, fraction of
    the screen width; TOWER_X by default). Darkened at the centre (the panels
    sit on top) and tinted night blue."""
    tower = nettoyer_photo(Image.open(tower_path).convert("RGB"))
    tw, th = tower.size
    src_x, ref_w, _ref_h = TOWER_SRC
    # The mast reference follows the real file size: a photo replaced by
    # another of the same framing stays correct.
    src_x = src_x * tw / ref_w
    target = (TOWER_X if tower_x is None else tower_x) * W
    # Three constraints: cover the height, and overflow neither left nor
    # right once the mast is brought to `target`.
    scale = max(H / th,
                target / max(src_x, 1),
                (W - target) / max(tw - src_x, 1))
    tower = tower.resize((int(tw * scale) + 1, int(th * scale) + 1), Image.LANCZOS)
    # ── SHARPENING THE PYLON ──────────────────────────────────────────
    # The pylon is a dark silhouette against a bright sky; three gestures, in
    # this order, and not one more:
    #   1. a wide, soft unsharp mask: it gives back the edge of the uprights
    #      and the braces without digging into the grain of the sky;
    #   2. the contrast, slightly: the silhouette stands out from the halo;
    #   3. the sharpness, a hair: the cables and the antenna panels.
    # Applied BEFORE the tint and the darkening: sharpening afterwards would
    # sharpen a blue veil, not the photo.
    #
    # The three values scale with OSMO_WP_SHARPEN (0 disables them): on
    # another photo, or a very dense screen, the right dose is not the same,
    # and an over-sharpened image is worse than a soft one. The default 0.45
    # is a light touch: the photo is DOWNSCALED by the render, and downscaling
    # already sharpens; a stronger value gives halos on the guy wires and a
    # grainy sky.
    _shp = float(os.environ.get("OSMO_WP_SHARPEN", "0.45"))
    if _shp > 0:
        tower = tower.filter(ImageFilter.UnsharpMask(radius=2.4 * _shp,
                                                     percent=int(135 * _shp), threshold=3))
        tower = ImageEnhance.Contrast(tower).enhance(1 + 0.16 * _shp)
        tower = ImageEnhance.Sharpness(tower).enhance(1 + 0.35 * _shp)
    x0 = int(round(src_x * scale - target))
    x0 = max(0, min(x0, tower.width - W))
    # Vertically, the sun stays where it was: bottom left of the strip.
    sun_y = int(600 * scale * th / _ref_h)
    y0 = max(0, min(sun_y - sun_y_at, tower.height - H))
    bg = Image.new("RGB", (W, H))
    part = tower.crop((x0, y0, min(tower.width, x0 + W), y0 + H))
    bg.paste(part, (0, 0))
    gap = W - part.width
    if gap > 0:
        # Should not happen (the scale is computed for it), but a photo with
        # another framing could force it: mirror of the edge, flat sky.
        edge = tower.crop((tower.width - gap, y0, tower.width, y0 + H)).transpose(Image.FLIP_LEFT_RIGHT)
        bg.paste(edge, (part.width, 0))
    # Night-blue tint + darkening: the pylon is at the centre, hence UNDER
    # both panels - they are translucent, it must show through without
    # turning the text grey. Hence a soft (70), symmetric darkening at the
    # centre.
    tint = Image.new("RGB", (W, H), (10, 18, 40))
    bg = Image.blend(bg, tint, 0.18)
    mask = gradient_h(W, 1, [(0, 0, 0), (70, 70, 70), (70, 70, 70), (0, 0, 0)]).resize((W, H))
    dark = Image.new("RGB", (W, H), (6, 10, 24))
    bg = Image.composite(dark, bg, mask.convert("L"))
    return bg
```

```
def glass_panel(base, box, radius=26, alpha=185, fill=(13, 22, 44), border=(90, 120, 200), blur=14):
    """Translucent panel, slightly blurred background underneath (frosted glass)."""
    x0, y0, x1, y1 = box
    if blur > 0:
        region = base.crop(box).filter(ImageFilter.GaussianBlur(blur))
        base.paste(region, (x0, y0))
    overlay = Image.new("RGBA", base.size, (0, 0, 0, 0))
    d = ImageDraw.Draw(overlay)
    rounded(d, box, radius, fill=fill + (alpha,), outline=border + (110,), width=2)
    base.paste(Image.alpha_composite(base.convert("RGBA"), overlay).convert("RGB"))


def gradient_text(base, xy, text, fnt, stops):
    """Text filled with a horizontal gradient."""
    x, y = xy
    l, t, r, b = fnt.getbbox(text)
    w, h = r - l, b - t
    mask = Image.new("L", (w + 4, h + 4), 0)
    ImageDraw.Draw(mask).text((-l + 2, -t + 2), text, font=fnt, fill=255)
    grad = gradient_h(w + 4, h + 4, stops)
    base.paste(grad, (x, y), mask)
    return w, h


def card(base, box, arfcn="514", band="DCS 1800"):
    x0, y0, x1, y1 = box
    # Very transparent card: the pylon and the sky stay visible behind the
    # text (the strip, on the other hand, keeps an opaque background to stay
    # readable).
    glass_panel(base, box, alpha=38, blur=0)
    d = ImageDraw.Draw(base)
    pad = 44
    x = x0 + pad
    y = y0 + 34

    # Header
    d.ellipse((x, y + 4, x + 14, y + 18), fill=(63, 185, 160))
    d.text((x + 28, y), "SOFTWARE DEFINED RADIO  ·  UM INTERFACE  ·  GR-GSM",
           font=font("DejaVuSansMono.ttf", 15), fill=(150, 170, 210))
    tag = f"ARFCN {arfcn}  ·  {band}"
    tf = font("DejaVuSansMono-Bold.ttf", 15)
    tw = d.textlength(tag, font=tf)
    rounded(d, (x1 - pad - tw - 28, y - 8, x1 - pad, y + 28), 10, outline=(63, 185, 160), width=2)
    d.text((x1 - pad - tw - 14, y), tag, font=tf, fill=(120, 230, 210))

    # Title
    y += 46
    # Title in plain purple (no gradient).
    tw, th = gradient_text(base, (x - 4, y), "LAB GSM", font("DejaVuSans-Bold.ttf", 86),
                           [(150, 100, 245), (150, 100, 245)])
    d = ImageDraw.Draw(base)
    y += th + 20
    sub = font("DejaVuSans.ttf", 22)
    subb = font("DejaVuSans-Bold.ttf", 22)
    line1 = [("Un réseau 2G complet, ", sub), ("sans matériel", subb),
             (" — baseband ", sub), ("Calypso", subb), (" émulé, cœur ", sub), ("Osmocom", subb), (",", sub)]
    line2 = [("de la synchro radio à l'appel voix chiffré en ", sub), ("A5/1", subb),
             ("  ·  couche 1 ", sub), ("gr-gsm", subb), (" (SDR)", sub)]
    for line in (line1, line2):
        cx = x
        for txt, f in line:
            d.text((cx, y), txt, font=f, fill=(215, 225, 245))
            cx += d.textlength(txt, font=f)
        y += 32

    # TDMA frame
```

```
    y += 14
    d.text((x, y), "TRAME TDMA  ·  8 TIMESLOTS  ·  4,615 MS", font=font("DejaVuSansMono.ttf", 14),
           fill=(150, 170, 210))
    y += 26
    slots = [("TS0", "FCCH·SCH·BCCH"), ("TS1", "SDCCH"), ("TS2", "TCH/F"), ("TS3", "TCH/F"),
             ("TS4", "TCH/F"), ("TS5", "TCH/F"), ("TS6", "PDCH"), ("TS7", "PDCH")]
    inner = (x1 - pad) - x
    gap = 10
    sw = (inner - gap * 7) // 8
    # TALL CELLS, NOT WIDE - same gesture as in the live banner
    # (configs/conky/labgsm.html): the width (sw, computed from the card) is
    # fixed; only sh is tall, and the text keeps its size, vertically centred
    # in the cell.
    sh = 148
    for i, (name, use) in enumerate(slots):
        sx = x + i * (sw + gap)
        active = i == 0
        rounded(d, (sx, y, sx + sw, y + sh), 10,
                fill=(58, 62, 150) if active else (24, 34, 70),
                outline=(120, 130, 240) if active else (60, 75, 130), width=2)
        nf = font("DejaVuSansMono-Bold.ttf", 16)
        uf = font("DejaVuSansMono.ttf", 11)
        cy = y + sh / 2
        d.text((sx + (sw - d.textlength(name, font=nf)) / 2, cy - 24), name, font=nf, fill=(235, 240, 255))
        d.text((sx + (sw - d.textlength(use, font=uf)) / 2, cy + 2), use, font=uf,
               fill=(120, 230, 210) if active else (140, 160, 200))
    y += sh + 8
    d.text((x, y), "0 µs", font=font("DejaVuSansMono.ttf", 12), fill=(120, 140, 180))
    mid = "1 burst = 577 µs"
    d.text((x + inner / 2 - d.textlength(mid, font=font("DejaVuSansMono.ttf", 12)) / 2, y), mid,
           font=font("DejaVuSansMono.ttf", 12), fill=(120, 140, 180))
    end = "4615 µs"
    d.text((x1 - pad - d.textlength(end, font=font("DejaVuSansMono.ttf", 12)), y), end,
           font=font("DejaVuSansMono.ttf", 12), fill=(120, 140, 180))

    y += 22

    # Chips at the bottom
    chips = [("IMSI 001-01", (150, 170, 210)), ("A5/1 ✓", (120, 230, 210)), ("COMP128v1 ✓", (120, 230, 210)),
             ("TCH/F · GAPK fr", (120, 230, 210)), ("gr-gsm · trxcon · osmo-bts-trx", (150, 170, 210))]
    cf = font("DejaVuSansMono-Bold.ttf", 13)
    cx = x
    for txt, col in chips:
        cw = d.textlength(txt, font=cf) + 22
        rounded(d, (cx, y, cx + cw, y + 28), 8, outline=col, width=1)
        d.text((cx + 11, y + 6), txt, font=cf, fill=col)
        cx += cw + 10
    site = "bbaranoff.github.io"
    sf = font("DejaVuSansMono-Bold.ttf", 14)
    d.text((x1 - pad - d.textlength(site, font=sf), y + 6), site, font=sf, fill=(180, 200, 240))


def strip_panel(base, box, strip_path, date_str, credit=None):
    """The image of the day, in a rounded white frame.

    `credit` is the footer line: it NAMES the source. osmo-wallpaper.sh draws
    the source at random, so the caption must come from the caller; the
    Calvin & Hobbes caption is only the default.
    """
    x0, y0, x1, y1 = box
    glass_panel(base, box, fill=(20, 24, 36), alpha=210, border=(200, 200, 210))
    d = ImageDraw.Draw(base)
    pad = 18
    # ── NO SOURCE LINE IN THE FRAME BY DEFAULT ──────────────────────────
    # The footer would sit UNDER the inset's button bar (Dashboard / tmux /
    # VTY, plus the arrows), which lives exactly there, and this frame is the
```

```
    # BENCH INSET: as soon as the stack comes up it carries a spectrum and a
    # log, where a photo credit makes no sense. The image therefore fills the
    # whole frame. OSMO_WP_CREDIT=1 adds the line back.
    foot = 30 if os.environ.get("OSMO_WP_CREDIT") == "1" else 0
    strip = Image.open(strip_path)
    strip.seek(0)
    strip = strip.convert("RGB")
    sw, sh = strip.size
    aw, ah = (x1 - x0) - 2 * pad, (y1 - y0) - 2 * pad - foot
    scale = min(aw / sw, ah / sh)
    strip = strip.resize((int(sw * scale), int(sh * scale)), Image.LANCZOS)
    sx = x0 + pad + (aw - strip.width) // 2
    sy = y0 + pad
    # White frame behind the strip (GIFs have irregular margins).
    d.rounded_rectangle((sx - 6, sy - 6, sx + strip.width + 6, sy + strip.height + 6), radius=8,
                        fill=(255, 255, 255))
    base.paste(strip, (sx, sy))
    if foot == 0:
        return
    d = ImageDraw.Draw(base)
    cap = credit or f"Calvin & Hobbes  ·  Bill Watterson  ·  {date_str}  ·  gocomics.com"
    cf = font("DejaVuSansMono.ttf", 13)
    # ── THE CAPTION MUST NOT LEAVE THE FRAME ──────────────────────────────
    # The live inset (osmo-fft-snap.py) covers exactly the frame and nothing
    # more, so anything overflowing would stay visible beside the spectrum.
    # Cut at the usable width, with an ellipsis. Measure then truncate, rather
    # than guessing a number of characters: the font is monospaced here, but
    # the caption may contain accents and symbols.
    avail = (x1 - x0) - 2 * pad
    if d.textlength(cap, font=cf) > avail:
        while cap and d.textlength(cap + "...", font=cf) > avail:
            cap = cap[:-1]
        cap = cap.rstrip(" ·") + "..."
    d.text((x0 + pad, y1 - foot + 6), cap, font=cf, fill=(160, 170, 190))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--tower", required=True)
    ap.add_argument("--strip", default=None, help="l image du cadre du BAS (Calvin & Hobbes)")
    ap.add_argument("--strip-haut", default=None, help="l image du cadre du HAUT (la BD geek)")
    ap.add_argument("--credit-haut", default=None)
    ap.add_argument("--date", default=datetime.date.today().isoformat())
    ap.add_argument("--credit", default=None,
                    help="ligne de pied sous l image (source, auteur). "
                         "Defaut : la legende Calvin & Hobbes.")
    ap.add_argument("--out", required=True)
    ap.add_argument("--arfcn", default="514")
    ap.add_argument("--band", default="DCS 1800")
    ap.add_argument("--tower-x", type=float, default=None,
                    help="ou poser le pylone, en fraction de la largeur d ecran "
                         "(0.5 = milieu, le defaut ; 0.17 = l ancien calage a gauche)")
    a = ap.parse_args()

    base = background(a.tower, a.tower_x)
    # THE LAB GSM CARD IS NOT BAKED INTO THE WALLPAPER BY DEFAULT.
    # tools/osmo-topzone.py (live WebKit widget) renders it, with the blinking
    # timeslots and the live ARFCN/IMSI/A5 values; a card painted here as well
    # would show twice behind the live one. OSMO_LEGACY_BANNER=1 bakes the
    # static card for a standalone wallpaper (a bench without the live desktop).
    if os.environ.get("OSMO_LEGACY_BANNER", "0") == "1":
        card(base, (320, 60, 1430, 560), a.arfcn, a.band)
    # Two frames, two images. A frame without an image is still drawn (empty):
    # the spectrum will cover it, and empty it says "nothing to show here".
    poses = {}
    for nom, box, chemin, credit in (("bas", BOX_BAS, a.strip, a.credit),
```

```
                                    ("haut", BOX_HAUT, a.strip_haut, a.credit_haut)):
        poses[nom] = False
        if chemin and os.path.isfile(chemin):
            try:
                strip_panel(base, box, chemin, a.date, credit)
                poses[nom] = True
            except Exception as e:  # corrupt GIF, HTML page instead of an image...
                print(f"[wallpaper] image du {nom} ignoree : {e}", file=sys.stderr)
        else:
            glass_panel(base, box, fill=(20, 24, 36), alpha=210, border=(200, 200, 210))
    tmp = a.out + ".tmp.png"
    base.save(tmp, "PNG", optimize=True)
    os.replace(tmp, a.out)
    print(f"[wallpaper] {a.out} ({W}x{H}, bas={'oui' if poses['bas'] else 'non'}, "
          f"haut={'oui' if poses['haut'] else 'non'})")


if __name__ == "__main__":
    main()
```

2.292 /opt/GSM/osmo-operator/tools/softsim/0001-simos-comp128v1-a38.patch

3323 octets, 103 lignes → 103 lignes

```
diff --git a/src/demo_server.rb b/src/demo_server.rb
index e89b69d..4bbe12d 100755
--- a/src/demo_server.rb
+++ b/src/demo_server.rb
@@ -107,23 +107,30 @@ when "unix"
 else
   raise "please defined which socket to use"
 end
-# wait for a client to connect
-io = socket.accept
+# serve clients one after the other: a new connection (mobile "sim sap") must
+# not need a server restart after the previous one disconnected
+loop do
+  # wait for a client to connect
+  puts "[sap] en attente d une connexion sur #{@unix || @port}"
+  io = socket.accept
+  puts "[sap] client connecte"

-case @type
-when "pcsc"
-  require_relative 'pcsc_server'
-  server = PCSCServer.new(io)
-when "sim"
-  require_relative 'simos_server'
-  server = SIMServer.new(io)
-else
-  raise "unkown server type"
-end
+  case @type
+  when "pcsc"
+    require_relative 'pcsc_server'
+    server = PCSCServer.new(io)
+  when "sim"
+    require_relative 'simos_server'
+    server = SIMServer.new(io, @file)
+  else
+    raise "unkown server type"
+  end

-# starting the server
```

```
-server.start
+  # starting the server
+  server.start

-# close IO when finished
-io.close
-server.close
+  # close IO when finished
+  io.close
+  server.close
+  puts "[sap] device disconnected : pret pour une reconnexion"
+end
diff --git a/src/simos_server.rb b/src/simos_server.rb
index 3c7a52a..80f6691 100644
--- a/src/simos_server.rb
+++ b/src/simos_server.rb
@@ -165,6 +165,31 @@ class SIMServer < Server
           sw = [0x90,0x00]
         end
       end
+    when 0xb2 # READ RECORD
+      # P1 = numero d enregistrement, P2 = 0x04 (absolu, P1 valide), P3 = longueur
+      # (Le) de l enregistrement, pour un EF lineaire fixe (ou cyclique) seulement.
+      type = file_info
+      if type[:type]!="EF" then
+        # no EF selected
+        sw = [0x94,0x00]
+      elsif !["linear fixed","cyclic"].include?(type[:structure]) then
+        # file is inconsitent with the command
+        sw = [0x94,0x08]
+      elsif request[3]!=0x04 then
+        # incorrect parameter P2 (seul le mode absolu est gere)
+        sw = [0x6b,0x00]
+      else
+        body = @selected.find_first("./body").content.hex2arr
+        record = request[2]
+        length = request[4]
+        if length==0 or record==0 or (record-1)*length+length>body.length then
+          # record not found / out of range
+          sw = [0x94,0x02]
+        else
+          data = body[(record-1)*length,length]
+          sw = [0x90,0x00]
+        end
+      end
     when 0xd6 # UPDATE BINARY
       # is an transparent ef selected ?
       type = file_info
@@ -205,6 +230,14 @@ class SIMServer < Server
         if tuple then
           @response = tuple["sres"].hex2arr+tuple["kc"].hex2arr
           sw = [0x9f,@response.length]
+        elsif ENV["SOFTSIM_KI"] && ENV["SOFTSIM_KI"].length == 32
+          # COMP128v1 avec le Ki de la SIM (patch local, lib/comp128.rb) : repond a
+          # n importe quel RAND du HLR, pas seulement aux triplets enregistres.
+          require_relative 'lib/comp128'
+          ki = [ENV["SOFTSIM_KI"]].pack("H*")
+          sres, kc = Comp128.v1(ki, request[5,16].pack("C*"))
+          @response = sres.unpack("C*") + kc.unpack("C*")
+          sw = [0x9f,@response.length]
         else
           # memory problem
           sw = [0x92,0x40]
```

2.293 /opt/GSM/osmo-operator/tools/softsim/0002-sap-serve-next-client.patch

500 octets, 15 lignes → 15 lignes

```
diff --git a/src/sap/common.rb b/src/sap/common.rb
index 9054e9f..a1d3862 100644
--- a/src/sap/common.rb
+++ b/src/sap/common.rb
@@ -183,7 +183,9 @@ class SAP
       rescue EOFError
         $stderr.puts "device disconnected"
         @io.close
-        exit 0
+        # return, not exit: demo_server.rb serves the next client (the mobile
+        # reconnects after every baseband restart)
+        return
       end
       # parse the message from @buffer_in to @messages_in
       parse_messages
```

2.294 /opt/GSM/osmo-operator/tools/softsim/0003-sap-log-timestamps.patch

1929 octets, 40 lignes → 40 lignes

```
diff --git a/src/sap/common.rb b/src/sap/common.rb
--- a/src/sap/common.rb
+++ b/src/sap/common.rb
@@ -454,7 +454,10 @@
   def log (group,message,level)
     if $verbosity and $verbosity>=level then
       color = 95-level
-      puts "\e[1m\e[#{color}m[#{group}]\e[0m #{message}"
+      # timestamp (ms) : lets one measure the time between two APDUs
+      ts = Time.now.strftime("%H:%M:%S.%L")
+      puts "#{ts} \e[1m\e[#{color}m[#{group}]\e[0m #{message}"
+      $stdout.flush
     end
   end

diff --git a/src/sap/server.rb b/src/sap/server.rb
--- a/src/sap/server.rb
+++ b/src/sap/server.rb
@@ -138,9 +138,18 @@
         set_state :processing_apdu_request
         # apdu should return APDU response byte array, or error result code
         raise "no APDU request in message" unless message[:payload].size==1
-        log("APDU","> #{hex(message[:payload][0][:value])}",1)
-        apdu_result = apdu(message[:payload][0][:value])
-        log("APDU","< #{hex(apdu_result)}",1)
+        ins_names = {0xa4=>"SELECT",0xc0=>"GET RESPONSE",0xb0=>"READ BINARY",
+                     0xb2=>"READ RECORD",0xd6=>"UPDATE BINARY",0xdc=>"UPDATE RECORD",
+                     0x88=>"RUN GSM ALGORITHM",0xf2=>"STATUS",0x20=>"VERIFY CHV",
+                     0x26=>"DISABLE CHV",0x28=>"ENABLE CHV",0x24=>"CHANGE CHV",
+                     0x2c=>"UNBLOCK CHV",0x04=>"INVALIDATE",0x44=>"REHABILITATE",
+                     0xfa=>"SLEEP",0x12=>"FETCH",0x14=>"TERMINAL RESPONSE"}
+        apdu_in = message[:payload][0][:value]
+        ins = ins_names[apdu_in[1]] || "INS 0x%02x" % apdu_in[1].to_i
+        log("APDU","> #{ins} #{hex(apdu_in)}",1)
+        t0 = Time.now
+        apdu_result = apdu(apdu_in)
+        log("APDU","< #{hex(apdu_result)} (#{ins}, #{((Time.now-t0)*1000).round(2)} ms)",1)
         payload = []
         if apdu_result.kind_of?(Array) then
           # ["ResultCode",["OK, request processed correctly"]]
```

2.295 /opt/GSM/osmo-operator/tools/softsim/0004-pcsc-disconnect-idempotent.patch

905 octets, 29 lignes → 29 lignes

```
diff --git a/src/pcsc_server.rb b/src/pcsc_server.rb
index ab3820d..082381d 100644
--- a/src/pcsc_server.rb
+++ b/src/pcsc_server.rb
@@ -104,9 +104,22 @@ class PCSCServer < Server
   end

   # release card and context
+  # Called twice per client: on DISCONNECT_REQ (sap/server.rb) and again by
+  # demo_server.rb (server.close) once the socket is gone. The second call hit
+  # the smartcard gem with a nil handle (TypeError) and killed the server, so the
+  # mobile could never reconnect. Idempotent now, and a lost reader is not fatal.
   def disconnect
-    @card.disconnect
-    @context.release
+    begin
+      @card.disconnect(:leave) if @card
+    rescue Exception => e
+      $stderr.puts "pcsc: disconnect ignored (#{e.class}: #{e.message})"
+    end
+    @card = nil
+    begin
+      @context.release if @context
+    rescue Exception
+    end
+    @context = nil
   end

   # get ATR
```

2.296 /opt/GSM/osmo-operator/tools/softsim/make_sim_op1.py

3857 octets, 97 lignes → 97 lignes

```
#!/usr/bin/env python3
# tools/softsim/make_sim_op1.py - generates the sim.xml of the HLR subscriber SIM
# from the softSIM demonstration sim.xml.
#
#   make_sim_op1.py <base sim.xml> <output> [--imsi 001010001000001]
#
# Replaces the EF_IMSI (6f07) of both DFs with the given IMSI, BCD encoded as in
# TS 51.011 (byte 1: digit 1 + odd indicator + IMSI type). The Ki is NOT
# written here: it comes from SOFTSIM_KI (read by the patched server).
import re
import sys

DEMO_IMSI_BODY = "084906202771401469"  # IMSI de demonstration 460027217044196


def imsi_body(imsi):
    d = [int(c) for c in imsi]
    if len(d) != 15:
        raise SystemExit("IMSI a 15 chiffres attendu : %r" % imsi)
    out = [(d[0] << 4) | 0x09]  # chiffre 1, indicateur impair, type IMSI (1)
    for k in range(1, 15, 2):
        lo = d[k]
        hi = d[k + 1] if k + 1 < 15 else 0xF
        out.append((hi << 4) | lo)
```

```
    return "08" + "".join("%02x" % b for b in out)


def iccid_body(imsi, mcc, mnc):
    """20-digit ICCID (89 + MCC + MNC + end of the IMSI + 00 + Luhn key), in
    swapped-nibble BCD as in TS 51.011: the demonstration sim.xml carries an
    ICCID with invalid nibbles (e, a) that the mobile displays as > and :."""
    d = [int(c) for c in "89" + mcc + mnc + imsi[5:] + "00"]
    tot = 0
    for k, v in enumerate(reversed(d)):  # le chiffre de controle s ajoute apres
        if k % 2 == 0:
            v *= 2
            v = v - 9 if v > 9 else v
        tot += v
    d.append((10 - tot % 10) % 10)
    return "".join("%x%x" % (d[k + 1], d[k]) for k in range(0, 20, 2))


def plmn_bcd(mcc, mnc):
    """PLMN on 3 bytes (TS 31.102): MCC1 MCC2 / MNC3 MCC3 / MNC2 MNC1."""
    m = [int(c) for c in mcc]
    n = [int(c) for c in mnc]
    n3 = n[2] if len(n) == 3 else 0xF
    return "%02x%02x%02x" % ((m[1] << 4) | m[0], (n3 << 4) | m[2], (n[1] << 4) | n[0])


def set_file_body(s, fid, fn):
    """Replaces the <body> of file fid (DF and every place) with fn(old_hex)."""
    pat = re.compile(r'(<file id="%s"[^>]*>.*?<body>)([0-9a-fA-F]*)(</body>)' % fid, re.S)
    return pat.sub(lambda m: m.group(1) + fn(m.group(2)) + m.group(3), s)


def set_plmn(s, mcc, mnc):
    """Replaces the PLMN of the files that carry it (PLMNsel, FPLMN, LOCI, SPN)."""
    p = plmn_bcd(mcc, mnc)
        # EF_PLMNsel: first entry = PLMN, the rest ff (same length)
    s = set_file_body(s, "6f30", lambda b: p + "ff" * (len(b) // 2 - 3))
        # EF_FPLMN: no forbidden PLMN
    s = set_file_body(s, "6f7b", lambda b: "ff" * (len(b) // 2))
        # EF_LOCI: LAI = PLMN (bytes 4..6 of the body, after the TMSI)
    def loci(b):
        raw = bytearray.fromhex(b)
        raw[4:7] = bytes.fromhex(p)
        return raw.hex()
    s = set_file_body(s, "6f7e", loci)
        # EF_SPN: operator name, display indicator 00
    s = set_file_body(s, "6f46", lambda b: "00" + "Test".encode().hex() + "ff" * (len(b) // 2 - 5))
    return s


def main(argv):
    if len(argv) < 3:
        raise SystemExit(__doc__)
    src, dst = argv[1], argv[2]
    imsi = argv[argv.index("--imsi") + 1] if "--imsi" in argv else "001010001000001"
        # check: the encoder reproduces the demonstration IMSI
    assert imsi_body("460027217044196") == DEMO_IMSI_BODY
    body = imsi_body(imsi)
    s = open(src, encoding="utf-8").read()
    s, n = re.subn(r"<body>%s</body>" % DEMO_IMSI_BODY, "<body>%s</body>" % body, s)
    if n == 0:
        raise SystemExit("EF_IMSI de demonstration introuvable dans %s" % src)
    mcc = imsi[:3]
    mnc = imsi[3:5]  # MNC a 2 chiffres, comme l IMSI du plan
    s = set_plmn(s, mcc, mnc)
        # EF_ICCID (2fe2): 10 bytes, valid BCD
```

```
    s = set_file_body(s, "2fe2", lambda b: iccid_body(imsi, mcc, mnc))
    open(dst, "w", encoding="utf-8").write(s)
    print("EF_IMSI %s -> %s (%d occurrence(s))" % (imsi, body, n))


if __name__ == "__main__":
    main(sys.argv)
```

2.297 /opt/GSM/osmo-operator/web/app.js

15303 octets, 308 lignes → 308 lignes

```
// Osmocom settings - web/index.html, served by tools/osmo-reglages.py (API /api/...)
'use strict';

const $ = s => document.querySelector(s);
const esc = s => String(s).replace(/[&<>"]/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;' }[c]));

function msg(t, ok = true) {
  const m = $('#msg');
  m.textContent = t;
  m.className = ok ? 'ok' : 'err';
  clearTimeout(msg.t);
  msg.t = setTimeout(() => (m.textContent = ''), 6000);
}

async function api(u, corps) {
  const r = await fetch(u, corps === undefined ? {} :
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(corps) });
  const j = await r.json();
  if (!r.ok) throw new Error(j.err || r.status);
  return j;
}

/* ── tabs ── */
document.querySelectorAll('nav button').forEach(b => b.onclick = () => {
  document.querySelectorAll('nav button, .onglet').forEach(x => x.classList.remove('actif'));
  b.classList.add('actif');
  $('#' + b.dataset.o).classList.add('actif');
  try { localStorage.setItem('onglet', b.dataset.o); } catch (e) { }
  if (b.dataset.o === 'etat') chargerEtat();
});

/* ── globals.conf / instance forms ──
 * A field = {cle, val, section, doc, defaut, choix[]}. Empty = script default (greyed). When the scripts
 * compare the variable against fixed values (choix), a drop-down: "(defaut)", the choices, and the
 * current value if it is not one of them. "autre…" switches to free input. */
function dessinerForm(champs, conteneur, filtre, posees) {
  const f = filtre.trim().toLowerCase();
  const parSec = new Map();
  for (const c of champs) {
    if (f && !(c.cle + ' ' + c.doc + ' ' + c.section).toLowerCase().includes(f)) continue;
    if (posees && c.val === '') continue;
    if (!parSec.has(c.section)) parSec.set(c.section, []);
    parSec.get(c.section).push(c);
  }
  const champ = c => {
    const id = conteneur.id + '_' + c.cle;
    let ctrl;
    if (c.choix && c.choix.length) {
      const opts = ['<option value="">(défaut' + (c.defaut ? ' : ' + esc(c.defaut) : '') + ')</option>'];
      const vals = c.choix.slice();
      if (c.val !== '' && !vals.includes(c.val)) vals.unshift(c.val);
```

```
      for (const v of vals) opts.push(`<option value="${esc(v)}"${v === c.val ? ' selected' :
      ↪  ''}>${esc(v)}</option>`);
      opts.push('<option value="\u0000autre">autre…</option>');
      ctrl = `<select id="${id}" data-cle="${c.cle}" data-orig="${esc(c.val)}">${opts.join('')}</select>`;
    } else {
      ctrl = `<input id="${id}" data-cle="${c.cle}" data-orig="${esc(c.val)}" value="${esc(c.val)}"
↪  placeholder="${esc(c.defaut || '')}">`;
    }
    return `<div class="champ${c.val !== '' ? ' pose' : ''}"><label
     ↪  for="${id}">${c.cle}<small>${esc(c.doc)}</small></label>${ctrl}</div>`;
  };
  conteneur.innerHTML = [...parSec].map(([s, l]) =>
    `<div class="carte"><h2>${esc(s || 'Divers')} <small>(${l.length})</small></h2><div
     ↪  class="grille">${l.map(champ).join('')}</div></div>`
  ).join('') || '<p class="aide">Aucun réglage ne correspond.</p>';
  conteneur.querySelectorAll('input, select').forEach(x => {
    const maj = () => x.classList.toggle('modifie', x.value !== x.dataset.orig);
    x.oninput = maj;
    x.onchange = () => {
      if (x.tagName === 'SELECT' && x.value === '\u0000autre') {
        // switch to free input, same id/cle
        const i = document.createElement('input');
        i.id = x.id; i.dataset.cle = x.dataset.cle; i.dataset.orig = x.dataset.orig; i.value = x.dataset.orig;
        i.oninput = () => i.classList.toggle('modifie', i.value !== i.dataset.orig);
        x.replaceWith(i); i.focus();
        return;
      }
      maj();
    };
  });
}
const modifies = conteneur => {
  const v = {};
  conteneur.querySelectorAll('input.modifie, select.modifie').forEach(x => (v[x.dataset.cle] = x.value));
  return v;
};

/* ── globals.conf ── */
let champs = [];
async function chargerGlobals() { champs = await api('/api/globals'); dessinerGlobals(); }
const dessinerGlobals = () => dessinerForm(champs, $('#g-form'), $('#g-filtre').value, $('#g-modifiees').checked);
$('#g-filtre').oninput = dessinerGlobals;
$('#g-modifiees').onchange = dessinerGlobals;
$('#g-enr').onclick = async () => {
  const v = modifies($('#g-form'));
  if (!Object.keys(v).length) return msg('Rien de modifié');
  try { await api('/api/globals', v); msg('globals.conf réécrit : ' + Object.keys(v).join(', '));
   ↪  chargerGlobals(); }
  catch (e) { msg(e.message, false); }
};

/* ── Calypso instances (instances/instance-N.env) ── */
let champsInst = [];
async function chargerInstances() {
  const l = await api('/api/instances');
  $('#i-sel').innerHTML = l.map(i => `<option value="${i.n}">${esc(i.role)}${i.existe ? '' : ' — (pas encore de
   ↪  fichier)'}</option>`).join('');
  try { const n = localStorage.getItem('instance'); if (n) $('#i-sel').value = n; } catch (e) { }
  chargerInstance();
}
async function chargerInstance() {
  champsInst = await api('/api/instance?n=' + $('#i-sel').value);
  try { localStorage.setItem('instance', $('#i-sel').value); } catch (e) { }
  dessinerInstance();
}
```

```
const dessinerInstance = () => dessinerForm(champsInst, $('#i-form'), $('#i-filtre').value,
 ↪  $('#i-modifiees').checked);
$('#i-sel').onchange = chargerInstance;
$('#i-filtre').oninput = dessinerInstance;
$('#i-modifiees').onchange = dessinerInstance;
$('#i-enr').onclick = async () => {
  const v = modifies($('#i-form'));
  if (!Object.keys(v).length) return msg('Rien de modifié');
  try {
    const r = await api('/api/instance', { n: +$('#i-sel').value, valeurs: v });
    msg(r.fichier + ' écrit : ' + Object.keys(v).join(', '));
    chargerInstances();
  } catch (e) { msg(e.message, false); }
};

/* ── daemons (VTY) ── */
let demons = [], cour = null;

async function chargerDemons(force) {
  demons = await api('/api/demons' + (force ? '?rafraichir=1' : ''));
  const avant = $('#d-sel').value;
  $('#d-sel').innerHTML = demons.map((d, i) =>
    `<option value="${i}">${esc(d.nom)} — ${d.hote}:${d.port} (${esc(d.invite)})</option>`).join('');
  if (avant && demons[avant]) $('#d-sel').value = avant;
  if (demons.length) chargerCfg();
}
const demon = () => demons[$('#d-sel').value];

async function chargerCfg() {
  const d = demon();
  if (!d) return;
  $('#d-cfg').textContent = 'lecture…';
  try { cour = await api(`/api/config?hote=${d.hote}&port=${d.port}`); dessinerCfg(); }
  catch (e) { $('#d-cfg').textContent = e.message; }
}

function dessinerCfg() {
  const f = $('#filtre').value.trim().toLowerCase();
  $('#d-cfg').innerHTML = cour.lignes.map((l, i) => {
    if (f && !(l.txt + ' ' + l.chemin.join(' ')).toLowerCase().includes(f)) return '';
    return `<div class="l${l.noeud ? ' noeud' : ''}" style="padding-left:${l.prof * 14}px">
      <input class="t" data-i="${i}" value="${esc(l.txt)}">
      <button class="b2" data-a="ok" data-i="${i}" title="appliquer">✓</button>
      ${l.noeud ? `<button class="b2" data-a="aj" data-i="${i}" title="ajouter une ligne dans ce
 ↪  noeud">+</button>` : ''}
      <button class="b2" data-a="no" data-i="${i}" title="envoyer « no … »">no</button></div>`;
  }).join('');
  $('#d-cfg').querySelectorAll('input.t').forEach(x => {
    x.oninput = () => x.classList.toggle('modifie', x.value !== cour.lignes[x.dataset.i].txt);
    x.onkeydown = e => {
      if (e.key === 'Enter') envoyer(+x.dataset.i, [x.value]);
      if (e.key === 'Escape') { x.value = cour.lignes[x.dataset.i].txt; x.classList.remove('modifie'); }
    };
  });
  $('#d-cfg').querySelectorAll('button').forEach(b => b.onclick = () => {
    const i = +b.dataset.i, l = cour.lignes[i], x = $(`#d-cfg input[data-i="${i}"]`);
    if (b.dataset.a === 'ok') envoyer(i, [x.value]);
    if (b.dataset.a === 'no' && confirm('Envoyer « no ' + l.txt + ' » ?')) envoyer(i, ['no ' + l.txt]);
    if (b.dataset.a === 'aj') { const n = prompt('Ligne à ajouter dans « ' + l.txt + ' » :'); if (n) envoyer(i,
     ↪  [n], true); }
  });
}

async function envoyer(i, lignes, dedans) {
  const d = demon(), l = cour.lignes[i], chemin = dedans ? [...l.chemin, l.txt] : l.chemin;
  try {
```

```
    const r = await api('/api/appliquer', { hote: d.hote, port: d.port, chemin, lignes });
    $('#d-jrn').textContent = `[${new Date().toLocaleTimeString()}] ${d.nom}
     ↪  ${d.hote}:${d.port}\n${r.journal}\n\n` + $('#d-jrn').textContent;
    msg(r.ok ? 'Appliqué' : 'Erreur VTY — voir le journal', r.ok);
    if (r.ok) chargerCfg();
  } catch (e) { msg(e.message, false); }
}

async function consoleVty(lignes) {
  const d = demon();
  if (!d) return;
  $('#c-out').textContent = '…';
  try { $('#c-out').textContent = (await api('/api/console', { hote: d.hote, port: d.port, lignes })).sortie; }
  catch (e) { $('#c-out').textContent = e.message; }
}

$('#d-sel').onchange = chargerCfg;
$('#d-ref').onclick = chargerCfg;
$('#d-scan').onclick = () => chargerDemons(true);
$('#filtre').oninput = () => cour && dessinerCfg();
$('#c-go').onclick = () => consoleVty($('#c-in').value.split('\n'));
$('#c-in').onkeydown = e => { if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) $('#c-go').click(); };
document.querySelectorAll('[data-c]').forEach(b => b.onclick = () => consoleVty([b.dataset.c]));
$('#d-write').onclick = () => {
  const d = demon();
  if (d && confirm('write memory sur ' + d.nom + ' ? (réécrit son fichier de config)')) consoleVty(['write
   ↪  memory']);
};

/* ── files ── */
async function chargerFichiers() {
  const l = await api('/api/fichiers');
  $('#f-sel').innerHTML = l.map(p => `<option>${esc(p)}</option>`).join('');
  chargerFichier();
}
async function chargerFichier() {
  const t = $('#f-txt');
  t.value = (await api('/api/fichier?p=' + encodeURIComponent($('#f-sel').value))).texte;
  t.dataset.orig = t.value;
  t.classList.remove('modifie');
}
$('#f-sel').onchange = chargerFichier;
$('#f-txt').oninput = e => e.target.classList.toggle('modifie', e.target.value !== e.target.dataset.orig);
$('#f-enr').onclick = async () => {
  try { await api('/api/fichier', { p: $('#f-sel').value, texte: $('#f-txt').value }); msg('Enregistré');
   ↪  chargerFichier(); }
  catch (e) { msg(e.message, false); }
};

/* ── osmo configs: /etc/osmocom/osmo-*.cfg, live file or configs/ template ── */
const osmo = { liste: [], courant: null, vue: 'etc' };
function dessinerOsmo() {
  const f = $('#o-filtre').value.trim().toLowerCase();
  $('#o-liste').innerHTML = osmo.liste.filter(c => !f || c.nom.includes(f)).map(c =>
    `<li class="${osmo.courant && osmo.courant.nom === c.nom ? 'actif' : ''}" data-n="${esc(c.nom)}">` +
    `<span class="pt ${c.pid ? 'on' : ''}" title="${c.pid ? 'démon en marche (pid ' + c.pid + ')' : 'aucun démon
     ↪  ne lit ce fichier'}"></span>` +
    `${esc(c.nom)}${c.protege ? ' <span title="protégé : start.sh ne le réécrit pas">🔒</span>' : ''}` +
    `${c.gabarit ? '' : ' <small>sans gabarit</small>'}</li>`).join('');
  document.querySelectorAll('#o-liste li').forEach(li => li.onclick = () => ouvrirOsmo(li.dataset.n));
}
async function chargerOsmo() {
  osmo.liste = await api('/api/configs-osmo');
  dessinerOsmo();
  if (!osmo.courant && osmo.liste.length) {
    let n = null;
```

```
    try { n = localStorage.getItem('osmo'); } catch (e) { }
    ouvrirOsmo(osmo.liste.some(c => c.nom === n) ? n : osmo.liste[0].nom);
  }
}
async function ouvrirOsmo(nom, vue) {
  const t = $('#o-txt');
  if (t.classList.contains('modifie') && !confirm('Modifications non enregistrées : les abandonner ?')) return;
  osmo.courant = osmo.liste.find(c => c.nom === nom);
  osmo.vue = vue || (osmo.vue === 'gabarit' && osmo.courant.gabarit ? 'gabarit' : 'etc');
  try { localStorage.setItem('osmo', nom); } catch (e) { }
  const c = osmo.courant, p = osmo.vue === 'gabarit' ? c.gabarit : c.etc;
  $('#o-titre').textContent = c.nom + '.cfg';
  $('#o-v-etc').classList.toggle('actif', osmo.vue === 'etc');
  $('#o-v-gab').classList.toggle('actif', osmo.vue === 'gabarit');
  $('#o-v-gab').disabled = !c.gabarit;
  $('#o-prot').textContent = c.protege ? '🔒 Protégé' : '🔓 Non protégé';
  $('#o-prot').classList.toggle('actif', c.protege);
  $('#o-aide').textContent = osmo.vue === 'gabarit'
    ? c.gabarit + (c.protege ? ' — ' + c.etc + ' est protégé : ce gabarit ne s\'applique plus tant que la
    ↪  protection reste.'
                           : ' — start.sh en régénère ' + c.etc + ' (marqueurs __X__ remplacés).')
    : c.etc + (c.protege ? ' — protégé : start.sh ne le réécrit pas. ' : c.gabarit ? ' — réécrit depuis le gabarit
    ↪  au prochain run ; Enregistrer le protège. ' : ' — ') +
      (c.pid ? 'Lu par le démon pid ' + c.pid + ' : effet à son redémarrage.' : 'Aucun démon ne le lit en ce
↪  moment.');
  t.value = (await api('/api/fichier?p=' + encodeURIComponent(p))).texte;
  t.dataset.orig = t.value;
  t.dataset.p = p;
  t.classList.remove('modifie');
  dessinerOsmo();
}
$('#o-filtre').oninput = dessinerOsmo;
$('#o-txt').oninput = e => e.target.classList.toggle('modifie', e.target.value !== e.target.dataset.orig);
$('#o-v-etc').onclick = () => osmo.courant && ouvrirOsmo(osmo.courant.nom, 'etc');
$('#o-v-gab').onclick = () => osmo.courant && ouvrirOsmo(osmo.courant.nom, 'gabarit');
$('#o-rech').onclick = () => { $('#o-txt').classList.remove('modifie'); chargerOsmo().then(() => osmo.courant &&
↪  ouvrirOsmo(osmo.courant.nom, osmo.vue)); };
$('#o-enr').onclick = async () => {
  const t = $('#o-txt');
  try {
    const r = await api('/api/fichier', { p: t.dataset.p, texte: t.value });
    msg(t.dataset.p + ' enregistré (.bak horodaté)' + (r.protege ? ', protégé contre la régénération' : ''));
    t.classList.remove('modifie');
    await chargerOsmo();
    ouvrirOsmo(osmo.courant.nom, osmo.vue);
  } catch (e) { msg(e.message, false); }
};
$('#o-prot').onclick = async () => {
  const c = osmo.courant;
  if (!c) return;
  if (c.protege && !confirm(c.nom + '.cfg redeviendra réécrit depuis le gabarit au prochain run. Continuer ?'))
  ↪  return;
  try {
    await api('/api/protege', { nom: c.nom + '.cfg', on: !c.protege });
    msg(c.nom + '.cfg ' + (c.protege ? 'rendu au gabarit' : 'protégé'));
    await chargerOsmo();
    ouvrirOsmo(c.nom, osmo.vue);
  } catch (e) { msg(e.message, false); }
};

/* ── status ── */
const duree = s => s > 3600 ? (s / 3600 | 0) + 'h' + String((s % 3600) / 60 | 0).padStart(2, '0') : s > 60 ? (s /
↪  60 | 0) + 'm' : s + 's';
async function chargerEtat() {
  const e = await api('/api/etat');
  $('#e-ps').innerHTML = '<tr><th>PID</th><th>Âge</th><th>RSS Mo</th><th>CPU %</th><th>Commande</th></tr>' +
```

```
    e.procs.map(p => `<tr><td>${p.pid}</td><td>${duree(p.age)}</td><td>${p.rss}</td><td>${p.cpu}</td><td><code>
 ↪  ${esc(p.cmd)}</code></td></tr>`).join('');
  $('#e-vty').innerHTML = '<tr><th>Démon</th><th>Adresse</th><th>Invite</th><th>PID</th></tr>' +
    e.vty.map(d => `<tr><td>${esc(d.nom)}</td><td>${d.hote}:${d.port}</td><td>${esc(d.invite)}</td><td>${d.pid}
 ↪  </td></tr>`).join('');
}

/* ── start-up ── */

chargerGlobals();
chargerInstances().catch(e => msg(e.message, false));
chargerDemons().catch(e => msg(e.message, false));
chargerFichiers();
chargerOsmo().catch(e => msg(e.message, false));
try { const o = localStorage.getItem('onglet'); if (o) document.querySelector(`nav
 ↪  button[data-o="${o}"]`)?.click(); } catch (e) { }
```

2.298 /opt/GSM/osmo-operator/web/index.html

5143 octets, 120 lignes → 120 lignes

```
<!doctype html>
<html lang="fr">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Réglages Osmocom</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
  <h1>Réglages Osmocom</h1>
  <nav>
    <button data-o="reseau" class="actif">globals.conf</button>
    <button data-o="osmo">Configs osmo</button>
    <button data-o="instances">Instances Calypso</button>
    <button data-o="demons">Démons (VTY)</button>
    <button data-o="fichiers">Fichiers</button>
    <button data-o="etat">État</button>
  </nav>
  <span id="msg"></span>
</header>

<main>
  <!-- globals.conf (network + launcher), modelled on globals.conf.example -->
  <section id="reseau" class="onglet actif">
    <div class="barre">
      <button class="b" id="g-enr">Enregistrer globals.conf</button>
      <input id="g-filtre" placeholder="filtrer (ex. ARFCN, SAP, WAN)">
      <label class="case"><input type="checkbox" id="g-modifiees"> seulement les valeurs posées</label>
      <span class="aide">Vide = défaut du script (grisé). Pris en compte au prochain run ; sauvegarde .bak
       ↪  horodatée.</span>
    </div>
    <div id="g-form"></div>
  </section>

  <!-- each /etc/osmocom/osmo-*.cfg: menu on the left, live file or configs/ template on the right -->
  <section id="osmo" class="onglet">
    <div class="osmo">
      <div class="carte o-menu">
        <h2>/etc/osmocom</h2>
        <input id="o-filtre" placeholder="filtrer (bsc, bts, pcu…)">
        <ul id="o-liste"></ul>
      </div>
```

```
    <div class="carte o-edit">
      <div class="barre serre">
        <strong id="o-titre">—</strong>
        <span class="bascule">
          <button class="b2" id="o-v-etc" data-v="etc">Fichier actif</button>
          <button class="b2" id="o-v-gab" data-v="gabarit">Gabarit configs/</button>
        </span>
        <button class="b" id="o-enr">Enregistrer</button>
        <button class="b2" id="o-prot" title="un fichier protégé n'est plus réécrit par start.sh depuis
        ↪  configs/"> Protégé</button>
        <button class="b2" id="o-rech">↻ Recharger</button>
      </div>
      <p class="aide" id="o-aide"></p>
      <textarea id="o-txt" rows="34" spellcheck="false"></textarea>
    </div>
  </div>
</section>

<!-- one file per Calypso instance (c54x_exe/run.sh): instances/instance-N.env -->
<section id="instances" class="onglet">
  <div class="barre">
    <select id="i-sel"></select>
    <button class="b" id="i-enr">Enregistrer l'instance</button>
    <input id="i-filtre" placeholder="filtrer (ex. CALYPSO_BSP, INSNS, AIR)">
    <label class="case"><input type="checkbox" id="i-modifiees"> seulement les valeurs posées</label>
    <span class="aide">Réglages du banc DSP de <em>cette</em> instance (pas globaux) : la ligne de commande
    ↪  garde la priorité, puis ce fichier, puis le défaut de run.sh.</span>
  </div>
  <div id="i-form"></div>
</section>

<!-- running-config of each daemon, through the VTY -->
<section id="demons" class="onglet">
  <div class="barre">
    <select id="d-sel"></select>
    <button class="b2" id="d-ref">↻ Recharger</button>
    <button class="b2" id="d-scan">Rechercher les démons</button>
    <input id="filtre" placeholder="filtrer (ex. arfcn, bts 0)">
    <button class="b2" id="d-write" title="écrit le fichier de config du démon">write memory</button>
  </div>
  <div class="flex2">
    <div class="carte"><h2>running-config — clic pour modifier, Entrée pour appliquer</h2><div id="d-cfg"
    ↪  class="cfg"></div></div>
    <div>
      <div class="carte">
        <h2>Console VTY</h2>
        <textarea id="c-in" rows="5" placeholder="show bts 0&#10;configure terminal&#10; network&#10;
        ↪  ..."></textarea>
        <div class="barre serre">
          <button class="b" id="c-go">Exécuter</button>
          <button class="b2" data-c="show running-config">running-config</button>
          <button class="b2" data-c="show rate-counters">rate-counters</button>
          <button class="b2" data-c="show bts 0">bts 0</button>
          <button class="b2" data-c="show subscriber cache">abonnés</button>
          <button class="b2" data-c="show talloc-context application brief">talloc</button>
        </div>
        <pre id="c-out"></pre>
      </div>
      <div class="carte"><h2>Journal des modifications</h2><pre id="d-jrn"></pre></div>
    </div>
  </div>
</section>

<!-- raw editing of the config files -->

<section id="fichiers" class="onglet">
```

```
    <div class="barre">
      <select id="f-sel"></select>
      <button class="b" id="f-enr">Enregistrer</button>
      <span class="aide">Sauvegarde .bak horodatée ; effet au prochain démarrage du démon.</span>
    </div>
    <textarea id="f-txt" rows="34" spellcheck="false"></textarea>
  </section>

  <section id="etat" class="onglet">
    <div class="carte"><h2>Processus</h2><table id="e-ps"></table></div>
    <div class="carte"><h2>VTY détectées</h2><table id="e-vty"></table></div>
  </section>
</main>
<script src="app.js"></script>
</body>
</html>
```

2.299 /opt/GSM/osmo-operator/web/style.css

5048 octets, 81 lignes → 81 lignes

```
/* Reglages Osmocom - web/index.html */
:root {
  --fond: #f6f7f9; --carte: #fff; --texte: #1d2330; --doux: #667085; --bord: #d9dde5;
  --accent: #2f6fde; --ok: #1f8a4c; --err: #c0362c; --modif: #fff4d6; --pose: #e8f1ff; --code: #f0f2f6;
}
@media (prefers-color-scheme: dark) {
  :root {
    --fond: #14171c; --carte: #1c2027; --texte: #e3e6eb; --doux: #8d96a5; --bord: #2e3440;
    --accent: #6b9cff; --ok: #4cc27e; --err: #ff6b5e; --modif: #3a3220; --pose: #1d2a40; --code: #232833;
  }
}
* { box-sizing: border-box; }
body { margin: 0; background: var(--fond); color: var(--texte); font: 14px/1.45 system-ui, sans-serif; }

header {
  display: flex; gap: 4px; align-items: center; flex-wrap: wrap; padding: 10px 16px;
  border-bottom: 1px solid var(--bord); background: var(--carte); position: sticky; top: 0; z-index: 2;
}
header h1 { font-size: 15px; margin: 0 16px 0 0; }
nav button { background: none; border: 0; padding: 6px 12px; border-radius: 6px; color: var(--doux); cursor:
↪  pointer; font: inherit; }
nav button.actif { background: var(--code); color: var(--texte); font-weight: 600; }
#msg { margin-left: auto; font-size: 13px; }
.ok { color: var(--ok); } .err { color: var(--err); }

main { padding: 16px; max-width: 1300px; margin: auto; }
.onglet { display: none; } .onglet.actif { display: block; }
.carte { background: var(--carte); border: 1px solid var(--bord); border-radius: 8px; padding: 14px;
↪  margin-bottom: 14px; }
h2 { font-size: 13px; text-transform: uppercase; letter-spacing: .04em; color: var(--doux); margin: 0 0 10px; }
.aide { color: var(--doux); font-size: 13px; }

.barre { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; margin-bottom: 12px; }
.barre.serre { margin: 8px 0 0; }
.barre select { width: auto; min-width: 240px; }
#g-filtre, #filtre { max-width: 300px; }
.case { display: inline-flex; gap: 6px; align-items: center; font: 13px system-ui; color: var(--doux); }
.case input { width: auto; }

/* formulaire globals.conf */
.grille { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 10px 18px; }
.champ label { display: block; font: 600 12.5px ui-monospace, monospace; }
.champ label small { display: block; font: 12px system-ui; color: var(--doux); font-weight: 400; }
```

```
.champ.pose input { background: var(--pose); }
input, select, textarea {
  font: 13px ui-monospace, monospace; background: var(--fond); color: var(--texte);
  border: 1px solid var(--bord); border-radius: 5px; padding: 5px 7px; width: 100%;
}
input::placeholder { color: var(--doux); opacity: .7; }
input.modifie, textarea.modifie { background: var(--modif) !important; }

button.b { background: var(--accent); color: #fff; border: 0; border-radius: 6px; padding: 6px 12px; cursor:
 ↪  pointer; font: inherit; }
button.b2 { background: var(--code); color: var(--texte); border: 1px solid var(--bord); border-radius: 6px;
 ↪  padding: 4px 10px; cursor: pointer; font: inherit; }

/* running-config */
.cfg { font: 13px ui-monospace, monospace; }
.cfg .l { display: flex; align-items: center; gap: 6px; padding: 1px 0; }
.cfg .l .t { flex: 1; padding: 2px 6px; border: 1px solid transparent; background: none; width: auto; }
.cfg .l .t:hover { border-color: var(--bord); }
.cfg .l.noeud > .t { font-weight: 700; color: var(--accent); }
.cfg .l button { visibility: hidden; }
.cfg .l:hover button, .cfg .l .t.modifie ~ button { visibility: visible; }

pre { background: var(--code); padding: 10px; border-radius: 6px; overflow: auto; max-height: 420px; margin: 8px 0
 ↪  0; white-space: pre-wrap; }
table { border-collapse: collapse; width: 100%; font-size: 13px; }
td, th { border-bottom: 1px solid var(--bord); padding: 4px 8px; text-align: left; vertical-align: top; }
td code { font-size: 12px; word-break: break-all; }
.flex2 { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
@media (max-width: 800px) { .flex2 { grid-template-columns: 1fr; } main { padding: 16px 12px; } }
/* [2026-10-22] onglet Configs osmo : menu des /etc/osmocom/osmo-*.cfg a gauche, editeur a droite */
.osmo { display: grid; grid-template-columns: 240px 1fr; gap: 14px; align-items: start; }
.o-menu { position: sticky; top: 64px; }
.o-menu ul { list-style: none; margin: 8px 0 0; padding: 0; max-height: 70vh; overflow: auto; }
.o-menu li { padding: 5px 8px; border-radius: 5px; cursor: pointer; font: 13px ui-monospace, monospace; display:
 ↪  flex; gap: 8px; align-items: center; }
.o-menu li:hover { background: var(--code); }
.o-menu li.actif { background: var(--pose); font-weight: 600; }
.o-menu li small { color: var(--doux); font: 11px system-ui; }
.pt { width: 8px; height: 8px; border-radius: 50%; background: var(--bord); flex: none; }
.pt.on { background: var(--ok); }
.bascule button.actif { background: var(--accent); color: #fff; border-color: var(--accent); }
.bascule button:disabled { opacity: .45; cursor: default; }
@media (max-width: 720px) { .osmo { grid-template-columns: 1fr; } .o-menu { position: static; } }
#o-prot.actif { background: var(--ok); color: #fff; border-color: var(--ok); }
```